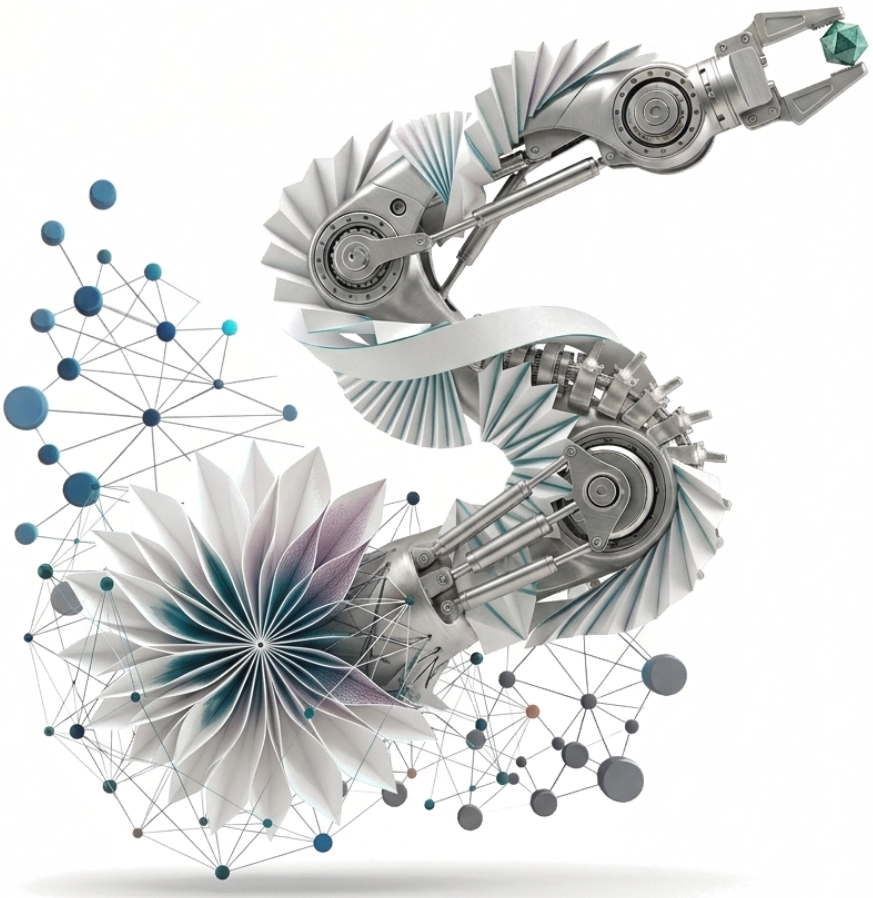


Physical AI

Machine Learning Systems

Vijay Janapa Reddi

Physical AI

Machine Learning Systems That Sense and Act

DRAFT

DRAFT

Physical AI

Machine Learning Systems That Sense and Act

Vijay Janapa Reddi
Harvard University

DRAFT

The MIT Press
Cambridge, Massachusetts
London, England

Contents

Preface **xiii**
Who This Book Is For . . . xiii
Why a Physical AI Textbook . . . xiii
The Causal Boundary and the Four Laws . . . xiv
What This Book Covers . . . xiv
How to Read This Book . . . xv
What This Book Is Not . . . xvii
A Note on Durability . . . xvii
An Open-Source Textbook . . . xvii
A Living Textbook . . . xvii
Prerequisites . . . xviii

Dedication **xix**

Author's Note **xxi**

About the Author **xxiii**
Why I Am Writing This Book: The First-Principles Systems Stance . . . xxiii
Research Background and Systems Path . . . xxiv
The Kit and Laboratory Development . . . xxiv

Reader Guide and Prerequisites **xxv**
The Technical Starting Point . . . xxv
Curricular Reading Tracks . . . xxvi
What the Book Introduces . . . xxvi
How to Read the Numbers . . . xxvii
What We Deliberately Omit . . . xxvii

Acknowledgments **xxix**

A Note on AI Assistance **xxxi**

Notation **xxxiii**

Main Text

Chapter 1 The Causal Boundary **3**
1.1 Artificial Intelligence Beyond Glass . . . 4
1.2 From Prediction to Action . . . 6
1.3 The Question This Book Answers . . . 9
1.4 Defining the Causal Boundary . . . 11
1.5 Three Machine Classes . . . 17
1.6 The Physical AI Scope Test . . . 18
1.7 The Machine in Five Levels . . . 21

1.8 When the Responsibilities Fail Together . . . 23
1.9 Where Safety, Security, and Accountability Meet . . . 26
1.10 The Four Bedrock Laws . . . 28
1.11 The Architecture of This Book . . . 30
1.12 Fallacies and Pitfalls . . . 32
1.13 Summary . . . 33
1.14 Historical Perspective and Further Reading . . . 34

Part I: The Machine Anatomy

The Machine Anatomy **39**

Chapter 2 The Physical Body **41**
2.1 The Five Physical Budgets . . . 42
2.2 Measurement Freshness . . . 44
2.3 Kinetic Momentum . . . 48
2.4 Actuator Transmission Limits . . . 53
2.5 Thermal Duty Cycles . . . 65
2.6 Electrical Power Integrity . . . 71
2.7 Measuring a Machine's Own Limits . . . 75
2.8 Which Physical Budget Binds First . . . 79
2.9 Fallacies and Pitfalls . . . 82
2.10 Summary . . . 85
2.11 Historical Perspective and Further Reading . . . 87

Chapter 3 The Cognitive Brain **89**
3.1 The Embodied Brain . . . 90
3.2 The Classical Wall . . . 92
3.3 Learned Representations . . . 94
3.4 The Cost of a Fresh Evaluation . . . 99
3.5 Chunk Supply and Evidence Validity . . . 107
3.6 Cognitive Vulnerabilities . . . 118
3.7 Fallacies and Pitfalls . . . 122
3.8 Summary . . . 124
3.9 Historical Perspective and Further Reading . . . 125

Chapter 4 The Nervous System **129**
4.1 The Multi-Rate Bridge . . . 130
4.2 The Independence Requirement and the Two-Processor Hypothesis . . . 132
4.3 Multi-Rate Cadences . . . 136
4.4 Chunk Lease Sizing from the Stopping Budget . . . 141
4.5 Real-Time Command Transport and Bus Determinism . . . 147
4.6 Actuation Authority . . . 154
4.7 Hardware Isolation and Fault Containment . . . 157
4.8 Invariant Checking . . . 159
4.9 The Handoffs Matrix . . . 160
4.10 Fallacies and Pitfalls . . . 166
4.11 Summary . . . 167
4.12 Historical Perspective and Further Reading . . . 169

Part II: Teaching the Machine

Teaching the Machine **173**

Chapter 5 Physical Data **175**
5.1 Endogenous Experience . . . 176

5.2 Mechanics of Endogenous Covariate Shift . . . 177
5.3 Recovery and Compounding Error . . . 178
5.4 Physical Telemetry Sources and Collection Paths . . . 180
5.5 Action Label Selection across the Control Boundary . . . 183
5.6 Teleoperation Demonstration Costs . . . 189
5.7 Collection Policy Coverage . . . 196
5.8 Intervention and Recovery Data . . . 199
5.9 Cross-Embodiment Transfer . . . 204
5.10 Dataset Degradation and Physical Distribution Shift . . . 210
5.11 The Dataset Schema . . . 215
5.12 Fallacies and Pitfalls . . . 218
5.13 Summary . . . 220
5.14 Historical Perspective and Further Reading . . . 222

Chapter 6 Policy Training **225**
6.1 Dual Failure Modes of Policy Training . . . 226
6.2 Policy Training Regimes . . . 227
6.3 Reinforcement Learning and Environment Interaction . . . 242
6.4 The Simulator as a System Component . . . 246
6.5 Physical Channels of the Sim-to-Real Gap . . . 252
6.6 Real-Robot Fine-Tuning and Residual Control . . . 260
6.7 Training Failure Modes and Permission-Path Interceptors . . . 263
6.8 The Policy Manifest . . . 264
6.9 Fallacies and Pitfalls . . . 268
6.10 Summary . . . 271
6.11 Historical Perspective and Further Reading . . . 273

Chapter 7 Closed-Loop Evaluation **275**
7.1 From Clean Runs to Bounded Claims . . . 276
7.2 What a Physical Success Rate Measures . . . 277
7.3 Closed-Loop Dynamics . . . 281
7.4 Evidence Regimes . . . 287
7.5 Fixing the Trial Protocol . . . 291
7.6 Confidence from a Finite Campaign . . . 292
7.7 Trial Budgets, Stopping Rules, and Comparisons . . . 295
7.8 The Evaluation Record . . . 300
7.9 Verification Ceilings and the Exposure Wall . . . 306
7.10 Evaluation Pitfalls and Metric Masking . . . 315
7.11 Fallacies and Pitfalls . . . 317
7.12 Summary . . . 320
7.13 Historical Perspective and Further Reading . . . 322

Part III: Running the Machine

Running the Machine **327**

Chapter 8 Sensor Perception **329**
8.1 Choosing Evidence for a Spatial Decision . . . 330
8.2 Capture Physics and Calibration . . . 332
8.3 Dating and Placing the Measurement . . . 341
8.4 Bounding Observation Age . . . 345
8.5 Ingestion Overhead and Memory Bus Contention . . . 351
8.6 Spatial Error and Clearance Bounds . . . 354
8.7 Perception Failure Modes and Sensor Degradation . . . 361
8.8 The Observation Contract . . . 366
8.9 Fallacies and Pitfalls . . . 370

8.10 Summary . . . 372
8.11 Historical Perspective and Further Reading . . . 374

Chapter 9 Spatial Memory **377**
9.1 Spatial Belief Persistence and Expiry . . . 378
9.2 Estimating State from Dated Measurements . . . 380
9.3 The Spatial Memory Handoff and State Invariants . . . 386
9.4 State Epochs and Temporal Anchoring . . . 388
9.5 Coordinate Transform Expiry and Localization Drift . . . 390
9.6 Certified Spatial Assertions in Scene Memory . . . 395
9.7 Forward State Propagation and Occlusion Decay . . . 406
9.8 Observability Ceilings and Estimation Limits . . . 415
9.9 Spatial Belief Failure Modes and Map Corruption . . . 417
9.10 The Belief Record Schema . . . 420
9.11 Fallacies and Pitfalls . . . 424
9.12 Summary . . . 425
9.13 Historical Perspective and Further Reading . . . 428

Chapter 10 Grounded Intent **431**
10.1 Grounded Affordances . . . 432
10.2 Task Operators and Geometric Samplers . . . 436
10.3 Semantic Request Translation to Geometric Proposals . . . 437
10.4 The Intent Handoff and Bounded Authority . . . 446
10.5 Expiring Goals and Temporal Validity Leases . . . 448
10.6 Admissible Goal Sets . . . 452
10.7 Intent Grounding Failure Modes and Their Checks . . . 457
10.8 The Intent Lease Specification and Contract . . . 459
10.9 Fallacies and Pitfalls . . . 465
10.10 Summary . . . 467
10.11 Historical Perspective and Further Reading . . . 469

Chapter 11 Trajectory Planning **471**
11.1 Continuous Trajectory Generation . . . 472
11.2 The Planning Handoff and the Trajectory Record . . . 473
11.3 Replacement Timing and the Planning Horizon . . . 477
11.4 Kinodynamic Feasibility . . . 482
11.5 Behavior at the Seam . . . 494
11.6 Choosing a Trajectory Generator . . . 501
11.7 Trajectory Planning Failure Modes and Infeasible Seams . . . 507
11.8 The Trajectory Record . . . 509
11.9 Fallacies and Pitfalls . . . 510
11.10 Summary . . . 512
11.11 Historical Perspective and Further Reading . . . 514

Chapter 12 Safety Enforcement **517**
12.1 The Per-Tick Gatekeeper . . . 518
12.2 The Tracker Handoff and Dynamic Reference Envelopes . . . 519
12.3 Stopping Envelopes . . . 521
12.4 Authority Exhaustion and Emergency Fallback . . . 530
12.5 Safe Sets as Conditional Permission . . . 534
12.6 Refusing Proposals and Enforcing State Invariants . . . 537
12.7 The Minimal Intervention Principle and Optimization Filters . . . 539
12.8 Admission, Cyclic Checking, and Monitor Independence . . . 548
12.9 The Fallback Ladder . . . 550
12.10 Runtime Enforcement Failure Modes . . . 558
12.11 The Enforcement Record . . . 559

12.12 Fallacies and Pitfalls . . . 563
12.13 Summary . . . 565
12.14 Historical Perspective and Further Reading . . . 567

Chapter 13 Workload Placement **571**
13.1 The Silicon Substrate . . . 572
13.2 Two Paths and the Resources They Share . . . 573
13.3 Contention for Shared Resources . . . 580
13.4 Thermal and Timing Coupling . . . 585
13.5 The Boundary Contract . . . 589
13.6 Silicon Isolation Decisions . . . 591
13.7 Empirical Verification of Silicon Isolation . . . 596
13.8 The Placement Record . . . 601
13.9 Fallacies and Pitfalls . . . 604
13.10 Summary . . . 605
13.11 Historical Perspective and Further Reading . . . 607

Part IV: Governing the Machine

Governing the Machine **611**

Chapter 14 Supervisory Intervention **613**
14.1 Shared Authority . . . 614
14.2 Human Authority, Reaction Time, and Clearance . . . 616
14.3 Authority Transitions . . . 626
14.4 Transfer Without Discontinuity . . . 633
14.5 Authenticating an Intervention . . . 637
14.6 Authority Handover Failure Modes . . . 639
14.7 Trustworthy Operational Logs . . . 642
14.8 The Authority Record . . . 643
14.9 Fallacies and Pitfalls . . . 648
14.10 Summary . . . 650
14.11 Historical Perspective and Further Reading . . . 653

Chapter 15 Adversarial Verification **657**
15.1 The Claim a Fault Trial Must Test . . . 658
15.2 The Operating Envelope . . . 660
15.3 One F1 Trial from Injection to Record . . . 664
15.4 Fault Oracles and Acceptance . . . 667
15.5 Extending the Trial to the Envelope . . . 671
15.6 Deriving the Fault List . . . 673
15.7 The Qualification Ladder . . . 681
15.8 Fault Injection on Hardware . . . 686
15.9 Searching Beyond the First Trial . . . 690
15.10 Interpreting Campaign Statistics . . . 695
15.11 Fault Records for Release . . . 701
15.12 Fallacies and Pitfalls . . . 704
15.13 Summary . . . 706
15.14 Historical Perspective and Further Reading . . . 708

Chapter 16 Deployment Release **711**
16.1 The Decision to Deploy . . . 712
16.2 Physical Criteria for Release . . . 713
16.3 Safety Cases and Claims . . . 717
16.4 The Three Verdicts . . . 725
16.5 The Verdict's Standing Conditions . . . 729

16.6 Adjudicating the Safety Case . . . 733
16.7 Adversarial Review . . . 736
16.8 The Release Manifest . . . 739
16.9 Maintaining Release Through Updates . . . 743
16.10 Fallacies and Pitfalls . . . 749
16.11 Summary . . . 751
16.12 Historical Perspective and Further Reading . . . 754

Chapter 17 Responsible Physical AI **757**
17.1 Who Answered for the Cruise Collision . . . 758
17.2 Decisions Behind an Admitted Action . . . 759
17.3 Oversight a Person Can Perform . . . 760
17.4 Permission in Stages . . . 763
17.5 When Detection Depends on Who Is Detected . . . 764
17.6 Who Bears the Risk . . . 767
17.7 Data About People . . . 768
17.8 The Labor the Machine Moves . . . 770
17.9 The Approval Record . . . 771
17.10 Fallacies and Pitfalls . . . 773
17.11 Summary . . . 774
17.12 Historical Perspective and Further Reading . . . 775

Conclusion

Chapter 18 Engineering Physical AI **779**
18.1 From a Better Prediction to Useful Service . . . 780
18.2 Generalizing to Unfamiliar Systems . . . 783
18.3 The Epistemic Frontier . . . 788
18.4 Choosing the Next Capability . . . 801
18.5 Fallacies and Pitfalls . . . 805
18.6 Summary . . . 807

Back Matter

Appendices **813**

References **813**

Appendix A Machine Learning Background **839**

Appendix B Control and Dynamics **881**

Appendix C Systems and Hardware **923**

Appendix D The Sense–Plan–Act Paradigm **979**

Glossary **991**

Index **1003**

DRAFT

0

Preface

Who This Book Is For

This book is for senior undergraduates and beginning graduate students in computer science, robotics, electrical engineering, and machine learning, and for practicing engineers who build, deploy, or certify physical AI systems. It is written for the engineer who has watched an embodied policy succeed in simulation or a scripted demonstration and now has to ship a machine that moves mass among people, day after day. That machine must not collide with a coworker, damage a payload, burn out a motor winding, or run away on a network timeout.

The reader should understand basic programming in Python and machine learning intuition, and be comfortable with introductory Newtonian physics, linear algebra, and basic probability. The book assumes no prior book in this series. It develops physical AI concepts where they are first used and derives the timing, control, and hardware-placement relationships needed to reason about embodied workloads without requiring specialist background in nonlinear control theory or application-specific integrated circuit (ASIC) microarchitecture.

Why a Physical AI Textbook

A reasonable engineer might look at the current wave of robot foundation models and ask a skeptical question. *Is physical artificial intelligence a genuine systems discipline with durable principles, or is it simply classical robotics with a transformer bolted on, or an end-to-end neural network running on an onboard computer?*

The skepticism is fair. Much of what is written about embodied AI highlights impressive single-task demonstrations, such as a dexterous hand folding laundry or a humanoid walking through a lab. Demonstration, however, is not deployment. In the field, failure rarely comes from the raw expressiveness of the model. What fails is the system around it, as when an observation arrives 50 milliseconds too late, an action proposal demands more torque than the joint drives can deliver, or an unmodeled delay turns a braking command into an impact.

Traditional machine learning systems center on disembodied prediction. A model evaluates static inputs in an open loop and returns advisory labels, rankings, or tokens without acting directly on the world. Because a person or downstream service must still decide what to do with that output, an erroneous prediction can be discarded or recalculated before it causes harm, and systems engineering centers on balancing training cost against serving latency, memory, and throughput. Agentic machine learning systems move from single predictions to sequential action: a model formulates multi-step plans, inspects feedback, and invokes software tools in a closed loop. As long as that loop stays inside a digital environment, however, virtual time can be paused, execution can be sandboxed, and failed transactions can be rolled back.

Physical AI systems bring agency into continuous physics. When an autonomous model commands actuators, computation crosses the causal boundary into mass, momentum, friction, and heat. In continuous physics, virtual sandboxes do not exist: time cannot pause while a foundation model deliberates, and Newtonian work cannot be rolled back after an impact. A larger model cannot shorten a stopping distance, cool an electric winding, or recover clearance spent while it computed. Physical AI therefore requires an architectural discipline. Learned models propose, while an independent permission path, with its own timing and its own fallback, decides what may cross into the physical world.

This book teaches how to design, measure, and govern that machine. Its central question is what an embodied task needs around a learned model to produce useful physical service supported by

the required evidence. Readers should leave able to translate physical limits into computational budgets, design sensorimotor policies that close the loop under real-time deadlines, enforce runtime assurance independently of the learned proposer, and justify release through an assurance case that links claims to empirical evidence.

The Causal Boundary and the Four Laws

The unit of engineering in this book is the **causal boundary**, the interface where an admitted digital command becomes current in a motor winding, torque on a shaft, and physical force delivered to the world. Accuracy on a held-out dataset is not the measure of a physical AI system. The measure is whether closed-loop actions achieve task goals within verified physical limits, supported by evidence independent of the learned model's self-report, and at what cost in latency, energy, thermal margin, and compute.

Four governing laws anchor the book's architecture. Each law pairs a physical or statistical constraint with the systems mechanism required to bound it:

- **The First Law: Irreversibility.** Software cannot roll back physical consequences. Every check that admits a command must finish before the machine spends its remaining clearance, so physical limits become strict budgets in time, distance, torque, heat, and voltage.
- **The Second Law: Endogenous Data.** A physical machine's actions choose its next observations. The states a deployed policy visits are conditioned on its own previous actions, so competence is a claim bounded by the operating envelope that closed-loop evaluation actually verified.
- **The Third Law: Proposal Is Not Permission.** A learned model may propose motion; only an independent permission path may admit it. The permission path evaluates state whose age is bounded, executes within a deterministic deadline set by the plant physics, and maintains a feasible fallback that does not depend on the learned proposer.
- **The Fourth Law: Permission Must Be Earned.** Permission to operate is granted per operating condition on recorded evidence, and it expires when that evidence does.

The first of these is a physical law. The other three are design principles, the embodied counterparts to covariate shift, runtime assurance, and the assurance case. This book elevates them to bedrock laws because a production machine that violates them incurs predictable physical risks.

Two relations quantify these obligations throughout the book. **Delay is distance** governs the stopping budget: if a machine moving at velocity v takes reaction time τ_{delay} to sense, compute, and actuate before decelerating at rate a_{brake}, the total stopping distance is $d_{\text{stop}} = v \cdot \tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}}$. Every millisecond of latency is distance spent before braking can begin. **Evidence expiration** governs the validity of state: an observation or state estimate carries an expiration timestamp derived from the plant's maximum rate of change, after which it cannot justify a safety assertion.

The machine's architecture separates five roles: the **Brain** (learned models that propose goals and motion), the **Nervous System** (deterministic communication and the independent permission path), the **Body** (drives, transmissions, windings, and power systems), the **Boundary** (register writes and sensor captures), and the **World** (continuous physical dynamics, obstacles, and people). A warehouse mobile manipulator that navigates an aisle shared with workers, opens a spring-latched door, and handles a fragile payload serves as the running reference machine in every chapter. A humanoid example at the end of each part tests whether the reasoning survives a body that couples balance, joint torque, and thermal dissipation.

What This Book Covers

The book builds understanding in physical dependency order: **anatomy, teaching, running, governing**. Each part adds responsibilities that the preceding material prepares the reader to evaluate, and reference appendices supply the supporting mathematical, control, and hardware derivations:

- **Introduction: The Causal Boundary** (chapter 1) prices delay at the causal boundary, defines the five-level machine architecture, and establishes the four governing laws.
- **Part I: The Machine Anatomy** turns the first law into physical budgets. Chapter 2 measures what the body can do before damage, chapter 3 characterizes the learned proposer and how

quickly its evidence ages, and chapter 4 derives how frequently the permission path must tick and how long a proposal may drive the actuators.

- **Part II: Teaching the Machine** applies the second law to sensorimotor learning. Chapter 5 records demonstrations with their operating conditions, chapter 6 analyzes training regimes and the dependencies they hide, and chapter 7 bounds the policy's competence through finite-trial closed-loop evaluation.
- **Part III: Running the Machine** applies the third law at runtime. It follows dated sensor measurements through perception (chapter 8), spatial memory and state estimation (chapter 9), intent grounding (chapter 10), and trajectory planning (chapter 11) to runtime enforcement (chapter 12), and tests whether real-time hardware keeps that path on time (chapter 13).
- **Part IV: Governing the Machine** applies the fourth law to operational authority. Chapter 14 governs how authority passes between the machine and an operator. Chapter 15 injects physical and computational faults to prove containment. Chapter 16 constructs the assurance case that justifies release under named conditions. Chapter 17 names who answers for each decision behind an admitted action, which human roles a person can actually perform, and which people bear the machine's risk and appear in its records.
- **Conclusion: Engineering Physical AI** (chapter 18) synthesizes the complete methodology, transfers the reasoning to an unfamiliar machine, and establishes the open research frontier.

Reference appendices provide background on machine learning foundations (Appendix A), control and dynamics (Appendix B), systems and hardware (Appendix C), sensorimotor troubleshooting (section D.6), and a unified glossary. Each appendix is organized as a lookup reference so that readers can consult a specific derivation without losing the thread of the main text.

How to Read This Book

The book is written to be read in order. Each chapter uses only ideas introduced earlier or defined where they appear, and each owns its concepts. Every chapter opens with the physical problem it solves, develops the mechanism as the answer, and closes with a summary and a handoff to the next chapter. Classical systems mechanisms (watchdog leases, safety filters, interlocks, fail-safe fallbacks, and real-time scheduling) appear as concrete solutions to physical hazards rather than as abstract theory.

Readers with a particular focus can tailor their depth after the introduction. Four tracks correspond to the primary engineering roles that collaborate on a physical AI deployment:

- **Core systems curriculum**: Follow the complete sequence from chapter 1 through chapter 18 to integrate physical limits, learning, runtime design, and operational evidence.
- **ML systems and learned runtimes**: Focus on the common foundation, sensorimotor training and evaluation (chapter 5 through chapter 7), perception and planning (chapter 8 through chapter 13), and release (chapter 16).
- **Embedded systems and silicon**: Focus on the common foundation, runtime placement (chapter 13), communication timing (chapter 4), and fault verification (chapter 15).
- **Robotics, control, and assurance**: Focus on the common foundation, state estimation and planning (chapter 9 through chapter 11), runtime enforcement (chapter 12), human intervention (chapter 14), and responsibility for the people around the machine (chapter 17).

Evidence flows from physical anatomy through training and runtime execution to operational release, and the chapter dependencies across the six divisions of the book follow that progression (figure P.1). Every arrow in the map represents a concrete record or physical bound handed forward from one engineering stage to the next.

Marginal reading aids and navigation pointers

The margins carry small visual aids beside the prose where a quantitative relationship, scale gap, or threshold is easiest to see. Each visual follows one of five recurring forms:

- **Ladders** span orders of magnitude (such as sensor ingestion bandwidths or execution loop rates).

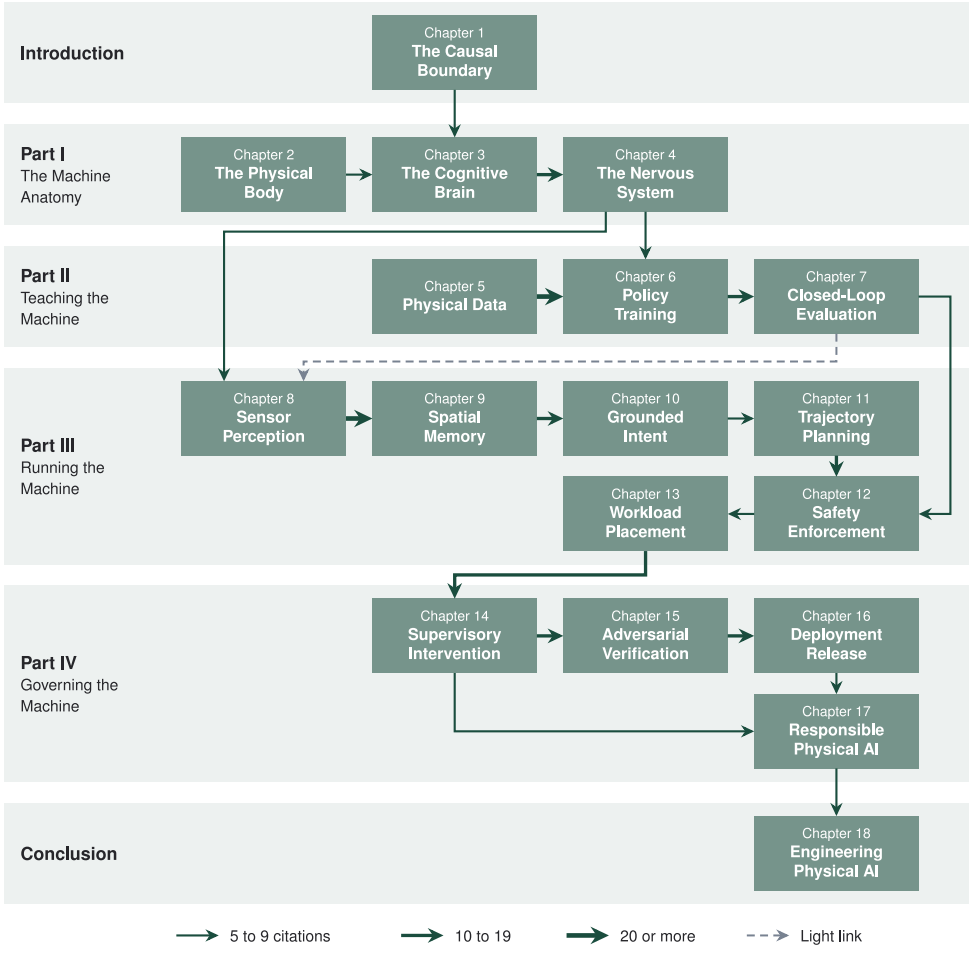
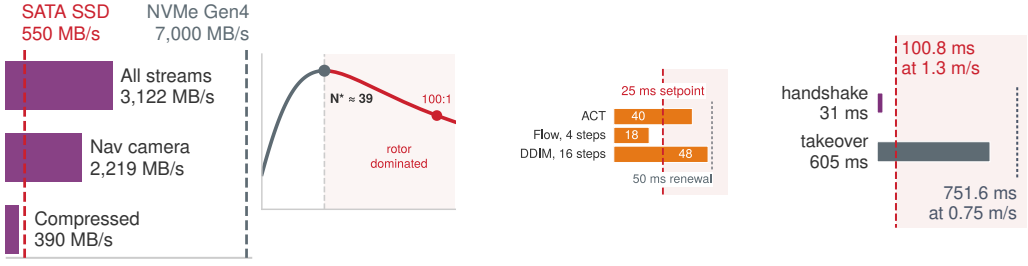


Figure P.1: Chapter Dependency Map: Arrows show chapter dependencies across the book's divisions. Solid-arrow weight follows the citation-count bands in the legend: 5–9, 10–19, and 20 or more citations; dashed arrows mark light links. Transitive links are omitted for clarity.

- **Threshold knees** mark the point where a physical relationship changes regime (such as reflected rotor inertia dominating gear trains or contact stiffness spiking forces).
- **Budget envelopes** set competing demands against a deadline (such as diffusion denoising steps against control periods, or operator takeover latencies).
- **Sequence strips** illustrate multi-phase timing (such as receding-horizon chunk execution or fault handshakes).
- **Causal chains** trace fault propagation from sensor error to physical consequence.



Alongside these visual notes, unnumbered **marginal navigation pointers** trace architectural connections across chapters. Three symbols distinguish whether a pointer looks back to a foundation, ahead to a downstream consumer, or across to a contrasting design:

- ↩ **Backward pointer:** Directs the reader back to the earlier chapter where a physical bound, hardware contract, or clock cadence was established.
- ↪ **Forward pointer:** Directs the reader ahead to the later chapter, or to an appendix, where the idea is developed, measured, or enforced.
- ⇄ **Contrast:** Marks a place where two chapters or machine classes treat the same problem differently.

A pointer only orients. When an argument depends on another chapter, the reference stays in the text.

What This Book Is Not

This is not a ROS 2 or middleware manual. Middleware and messaging frameworks change, whereas the physical requirements they must satisfy (bounded latency, deterministic serialization, and fault isolation) do not. The book therefore introduces real-time protocols and message queues only as mechanisms to meet stated physical deadlines. Nor is it a mechanical CAD manual or a motor design treatise, because it characterizes actuators, transmissions, and thermal limits through the software budgets they impose rather than through their mechanical fabrication. For the same reason, simulation appears here not as a benchmark leaderboard, where open-loop scores stand in for deployment, but as a tool for generating hypotheses and training data. Its claims must still be verified on the physical machine. Software agents that act solely behind digital APIs without physical actuators remain outside our scope.

A Note on Durability

Robot form factors, neural network architectures, and motor drive silicon will evolve, some before this book reaches print. The text therefore emphasizes the principles that survive those transitions: the causal boundary as the unit of engineering, the four governing laws, the separation between learned proposers and deterministic permission paths, and the discipline of grounding operational authority in empirical evidence. Current platforms and models appear throughout to make the concepts concrete, but the principles stand independently of any specific vendor, model, or hardware platform. Where a question is actively unsettled in the field, such as how best to verify generalist policies against rare corner cases, the book presents the problem as an open engineering challenge rather than settled doctrine.

An Open-Source Textbook

This book is open source. Its full text, figures, and build system are available at mlsysbook.ai/git, and contributions are welcome from every reader. That openness is deliberate. Physical AI draws on robotics, control, machine learning, and embedded systems, whose knowledge rarely sits in one place. A textbook that joins them should be available to everyone working across those boundaries, regardless of geography or institutional affiliation.

Readers help most by fixing an error, suggesting a clearer explanation, adding a worked example, or reporting where a claim does not hold on real hardware. Issues and pull requests provide the mechanism for that collaboration.

A Living Textbook

This copy is a snapshot of a book that continues to change. The field keeps moving, and so does my understanding of it, so I keep revising the online edition at **mlsysbook.ai**. The principles in this book are chosen to last. Examples, numbers, and explanations may have improved since this version was published, and the online release notes record what changed.

Prerequisites

The worked examples and derivations draw on five foundational areas. Readers should be comfortable with:

- programming in Python and working with numeric arrays;
- core machine learning concepts, including supervised training, loss functions, and inference;
- introductory Newtonian mechanics (position, velocity, acceleration, force, torque, inertia);
- linear algebra (vectors, matrices, coordinate transformations); and
- undergraduate probability and basic statistics.

Deeper background in classical control theory, real-time operating systems, or robotics kinematics is beneficial but not required; the book develops the necessary control, estimation, and real-time concepts where they are first needed. Every specialized tool is introduced from first principles at the point where a concrete systems decision depends on it.

DRAFT

0

Dedication

For my students,
who ask the question
I have not thought of yet,

and for the researchers and systems engineers
who take up the challenge of giving machines
safe and capable hands in an open physical world.

DRAFT

0

Author's Note

When a server returns an error for a page request, the reader can try again. A robot approaching an obstacle keeps moving while its computer decides whether to brake. Restarting the computation cannot recover the distance already traveled.

In a page-serving application, an unhandled exception or late result may produce a log entry, a dropped frame, or a retry button. In physical AI, software commands matter, momentum, and energy. The unit of engineering is the action crossing the causal boundary. Delay is paid in millimeters, and failure results in cracked composite, burned motor coils, or physical harm.

To build machines that survive this reality, we must give up the idea that a high-capacity network can drive a physical plant, the apparatus being controlled, directly. In the machine this book builds, the learned model proposes; only the permission path permits, and only for as long as its evidence holds.

I write these chapters as an active systems researcher exploring how the laws of computing intersect with the laws of Newtonian mechanics, thermal dissipation, and functional safety. Concrete systems tasks, hardware measurements, and first-principles napkin math guide our progression; formal continuous-control derivations and proof machinery provide optional depth in the appendices.

This book is an open, working notebook. Step across the causal boundary with me, and let us explore how to engineer machines that sense, act, and survive in the physical world.

— Vijay Janapa Reddi
Cambridge, Massachusetts
2026

DRAFT

0

About the Author

Vijay Janapa Reddi is the Gordon McKay Professor of Electrical Engineering and Computer Engineering at Harvard University and a Visiting Professor at ETH Zurich in the Integrated Systems Laboratory (IIS), D-ITET. He is a founding member and Vice President of MLCommons and the author of the open-source reference text *Machine Learning Systems*.

His research sits at the intersection of computer architecture, machine learning systems, and edge robotics, focusing on how computing hardware and learned software co-design to interact with the physical world. Over the past decade, his work has concentrated on democratizing applied systems through open-source education and reproducible benchmarks, connecting first-principles silicon constraints directly to accessible, bench-verified learning.

Why I Am Writing This Book: The First-Principles Systems Stance

Every textbook reflects the intellectual lineage, biases, and lens of its author. I want to make the viewpoint behind this book explicit.

I am not a classical control theorist, nor a mechanical kinematics specialist, and I do not pretend to be one. I bring the perspective of a computer systems architect and machine learning systems (MLSys) researcher going after first principles.

My intuition for this domain dates back to my undergraduate mechatronics capstone. As an electrical and computer engineering student, that capstone was the first time I had to integrate analog circuits, sensor conditioning, microcontrollers, motor drivers, mechanical linkages, and embedded software into a single working machine. It was the moment I realized that software cannot be designed in isolation from physical reality: motor coils heat up, power rails droop under sudden acceleration, and sensor measurements age the instant physical transduction occurs. For over twenty years, my work has sat at the boundary where software meets silicon and energy constraints: mobile System-on-Chip performance architecture and edge AI silicon at Google, co-founding MLCommons and MLPerf to set empirical benchmarking standards for AI accelerators, and helping start TinyML, which compresses neural networks to run on milliwatt microcontrollers.

Physical AI brings me back to that mechatronics capstone, with one difference. In classical mechatronics, we closed the loop using hand-crafted, deterministic logic: PID controllers, state machines, and analytical models. In Physical AI, we are embedding high-capacity neural networks (foundation models, diffusion policies, and learned decoders) into that exact same unforgiving loop. Their proposals may be sampled or deterministic, but neither form alone establishes that an action is safe. The engineering challenge now extends beyond making the machine move to closing the loop safely around a learned brain, using deterministic systems and safety principles.

That background shapes what I want students to understand. A model's proposed action must pass through a computing system before the physical machine can carry it out. I want readers to trace that path and ask what could prevent an action from being carried out as intended. The timing of computation, the body's capabilities, and the assumptions behind command checks all matter to that assessment. The explanations should make these interactions understandable before specialized mathematics or implementation detail is needed.

This textbook is my attempt to share that foundational systems perspective with the next generation of engineers, roboticists, and computer scientists. It is the book I wish my own students had when bridging machine learning systems to the physical world.

Research Background and Systems Path

This first-principles systems work spans three major eras. Each era extended the same hardware-to-software discipline to a tighter physical operating envelope:

- **Silicon Reality & Mobile Architecture:** Grounded in computer architecture, low-power microarchitectures, and mobile System-on-Chip (SoC) design (including edge AI silicon performance architecture at Google). This work addressed the physical realities of computing: memory bus contention under high-bandwidth camera streams, thermal throttling under strict power and thermal envelopes, and interrupt latencies under hard real-time deadlines.
- **ML Systems & The TinyML Movement:** Bridging algorithmic machine learning abstractions to silicon execution. He co-founded MLPerf to establish the global standard for benchmarking ML hardware and software, and pioneered the TinyML movement, developing the Harvard CS249r curriculum and the HarvardX TinyML program (reaching over 100,000 learners globally) to bring deep neural models onto milliwatt microcontrollers.
- **Closing the Sense-and-Act Loop in Physical AI:** Extending this systems perspective to machines whose learned models propose physical actions. The focus is on how sensing, computation, and the body's response interact, and on the conditions under which a proposed action can be executed.

The Kit and Laboratory Development

Development of the hands-on bench laboratories and the reference hardware platform (the Physical AI Kit on Arduino UNO Q Dual-Brain) is a collaboration with Dr. Andrea Mattia Garavagno, postdoctoral researcher in the Integrated Systems Laboratory (IIS), D-ITET at ETH Zurich. This work aims to translate the systems ideas in the book into firmware and software exercises that students and practitioners can reproduce and examine on the bench.

DRAFT

0

Reader Guide and Prerequisites

Physical AI: Machine Learning Systems That Sense and Act is an independent textbook for advanced undergraduate students, graduate researchers, and practicing engineers who want to build machines that sense and act. Readers may arrive from machine learning, software systems, electrical engineering, or robotics. The common goal is to connect learned behavior to physical execution and explain the conditions under which that connection can be trusted.

The Technical Starting Point

A physical AI systems engineer must reason across disciplines, but need not enter with specialist training in all of them. The starting point is programming experience, basic machine learning intuition, and introductory mechanics, supported by undergraduate mathematics. Prior study of another volume in this series is not required.

The following competencies support the worked examples. Each area supplies a practical skill that the chapters build on rather than a prerequisite in advanced theory:

- **Programming**: Read and modify a short program, follow its data flow, and interpret experimental output. The executable calculations use Python; familiarity with C or C++ helps with implementation examples but is not the entry requirement.
- **Machine learning intuition**: Distinguish training from inference, understand a learned model as a mapping fitted from data, and recognize why performance on held-out examples may differ from deployment. Experience building a large model or using a particular framework is not required.
- **Mathematics**: Work with algebra, vectors, matrices, derivatives, and elementary probability. These tools support coordinate transformations, rates of change, and uncertainty estimates. More specialized notation is introduced with the mechanism that needs it.
- **Physical reasoning**: Relate force to acceleration, speed to distance traveled, and electrical power to energy use. Introductory Newtonian mechanics supplies the starting intuition for stopping, contact, and actuator limits.
- **Experimental practice**: Interpret a measurement with its units and conditions, distinguish an assumption from an observation, and compare a calculation with a test. The book develops this practice into evidence for operating a machine.

Specialized control theory, robot kinematics, real-time scheduling, and accelerator architecture are subjects of study here, rather than a second set of admission requirements. Some derivations require a slower pass or background reading. The local appendices provide machine learning background (Appendix A), control and dynamics (Appendix B), and systems and hardware (Appendix C). When a failure spans perception, planning, and actuation, section D.6 maps the symptom to measurements that distinguish competing causes.

Learning across disciplinary boundaries

Readers from different backgrounds enter this loop from different sides. An ML practitioner starts from the consequences of a proposed action and works toward the mechanics and timing that constrain it. An embedded systems engineer starts from a hardware deadline and learns how training data and policy behavior shape the workload that must meet it. A roboticist starts from a model of the plant and examines what changes when a learned proposer supplies goals and

trajectories. Each reader brings one part of the foundation and meets the others where the system connects them.

On a first pass through a dense derivation, identify the physical quantity being bounded, the assumptions the bound needs, and the decision it changes. Then work through the example before returning to the derivation. A formal guarantee may require that deeper mathematical treatment; understanding the systems decision still begins with the concrete failure the mechanism is meant to prevent.

Curricular Reading Tracks

The sequential path is the default for learning the whole discipline. Specialist tracks select an emphasis within that path. They do not make the common physical model or the evidence limits optional.

All tracks begin with chapter 1 and the machine anatomy in chapter 2, chapter 3, and chapter 4. These establish physical budgets, learned proposals, and the independent path that decides whether a proposal may act. Later chapters repeatedly use those foundations. After that common start, readers can concentrate their deeper study along one of four role-specific tracks (table P.1).

Table P.1: Curricular reading tracks: Four structured reading tracks through this book.

Track	Primary Audience	Core Chapter Progression	Primary Systems Competency
Core systems curriculum	Students and engineers learning the full discipline	Complete sequence from chapter 1 through chapter 18	Integrate physical limits, learning, runtime design, and operating evidence
ML systems and learned runtimes	ML engineers and researchers	Common start; chapter 5 through chapter 7; chapter 8 through chapter 13; chapter 16 and chapter 18	Develop and evaluate learned behavior within physical and computational limits
Embedded systems and silicon	Embedded engineers and computer architects	Common start; evaluation limits in chapter 7; chapter 8 through chapter 13; chapter 14 through chapter 18	Keep sensing, permission, and actuation timely under interference and faults
Robotics, control, and assurance	Roboticists and control engineers	Common start; chapter 5 through chapter 7; chapter 9 through chapter 18	Integrate learned behavior with state estimation, feasible motion, and conditional guarantees

When a track omits a chapter from detailed study, its opening, worked handoff, and summary provide the context needed downstream. For example, a planning exercise still needs dated observations and a current state estimate; a release decision still needs evidence about human authority and fault response. Instructors can select technical depth while retaining those dependencies. The full sequence contains more detail than every course will cover at equal depth.

What the Book Introduces

The curriculum builds toward an end-to-end engineering judgment about a machine performing a specified task. Its parts contribute different capabilities:

1. In Part I, determine which physical budget binds, estimate what the learned proposer can supply, and derive the timing requirements that connect computation to actuation.
2. In Part II, collect demonstrations with their operating conditions, choose a training regime, and evaluate behavior in the feedback loop where actions change subsequent observations. Use failures to collect new experience and test whether a changed candidate expands competence without losing established behavior.
3. In Part III, turn measurements into dated state estimates, grounded goals, and feasible trajectories; then check proposals against physical limits and place the runtime on resources that can meet its deadlines.
4. In Part IV, design human intervention, challenge the integrated system with faults, and justify useful service within the conditions the evidence covers, assessing task performance and physical containment separately.

The conclusion synthesizes those engineering capabilities and tests whether they transfer to an unfamiliar machine. The records accumulated along the way preserve the reasoning behind a design: what was measured, what was assumed, which decision followed, and what would invalidate it. Filling in a record is useful only when the underlying engineering argument is sound.

How to Read the Numbers

The evidence behind a number determines what it can support. Four categories distinguish observations and calculations from ratings and illustrative inputs; a fifth label identifies a design choice:

- **Measured**: A calibrated observation of a particular machine. It names its instrument, its operating conditions, its sample count, and, for a distribution, the percentile it reports. Only a measured number, or a number derived from measured ones, can support a safety margin.
- **Derived**: Computed from stated inputs through a physical or statistical model. It carries that model's assumptions, input provenance, and propagated uncertainty. A result derived from illustrative inputs remains illustrative evidence.
- **Datasheet**: A manufacturer's rating under the manufacturer's conditions. It says what to expect and what to test, and it is not evidence about the unit on the bench.
- **Illustrative**: Selected to show arithmetic or scaling. It supports no margin, timeout, or release decision.

A budget or requirement, such as a lease duration or a clearance margin, belongs to none of the four. It is a design decision, labeled **chosen**, and it means something only when it is checked against the measured or derived number it must fit, as a chosen lease is checked against the derived ceiling on pre-brake delay.

A worked design in this book shows a decision procedure. When its inputs are illustrative, the decision it reaches is illustrative too. It shows how the machine's speed, lease, or verdict would be set, and it would bind a real machine only after its physical and timing premises are qualified. Some examples stipulate a supporting evidence packet to teach how successful learning or restricted service could be justified. Such branches are hypothetical throughout; their assumed trial counts and outcomes are not measurements of the book's machine.

When a number decides something, such as a speed limit, a timeout, or a release, the book keeps it as a **limit record** with six elements. The record pairs the nominal value (or distribution) and its measurement uncertainty with the operating conditions under which that value holds (payload, temperature, supply voltage, surface, concurrent workload, and firmware). Alongside the engineering margin between the operating target and the breakdown threshold, it names the runtime detector that catches an excursion while the machine runs and records the value's provenance, meaning its label and the instrument or model that produced it. A limit holds only under its stated conditions. When wear, heat, a new payload, or a new build changes them, the limit must be established again before it is reused.

Before using a worked value to justify a decision, establish both its category and the conditions attached to it, because an unlabeled value does not become a measured limit merely by appearing in a calculation. Every result holds only under the assumptions stated with its example. A statistical bound from finitely many trials depends on its sampling assumptions and proves nothing beyond them. A measured percentile describes only the tested workload rather than a true worst case. In a release argument (chapter 16 and chapter 18), each premise therefore carries an explicit status: established by qualified measurements or derivations, assumed, decided by accountable judgment, unsupported, or outside the threat model.

What We Deliberately Omit

The book develops specialist mechanisms to the depth needed for systems decisions, introducing each topic from first principles inside the chapter whose decision depends on it and stopping there. In the anatomy and learning chapters, model quantization is introduced where it sets the intent model's latency (section 3.4.2), while reinforcement learning fundamentals and the pretrain-then-post-train recipe appear when the arm learns the spring-latched door task (section 6.2.1, section 6.2.8). In the runtime chapters, perception stops at the qualified measurement and hands state estimation

to spatial memory (chapter 9). Spatial memory builds from the discrete and extended Kalman filters, multi-target tracking, and delayed updates (section 9.2) to localization and SLAM in the planning map (section 9.5.1). Task and motion planning grounds abstract goals in feasible geometry (section 10.2), while manipulator kinematics, time parameterization, and mobile-base planning build the trajectory record (section 11.2, section 11.4.5, section 11.6.1) and worst-case execution time analysis tests hardware isolation (section 13.7). Finally, the governance chapters introduce importance sampling where rare-failure rates must be estimated (section 15.10.2) and functional safety standards where the release argument is assembled (section 16.3.4).

What the book omits is the remainder of each of those fields. It does not provide an exhaustive catalog of manipulator geometries or grasp analysis, or a complete course in nonlinear or optimal control. It omits the convergence theory of reinforcement learning and the construction of large-scale SLAM back ends. It also leaves out the internals of static timing-analysis tools and a comprehensive theory of deep learning optimization, and it is not a substitute for the text of a safety standard. Specialized derivations appear where they explain a bound or trade-off, and the appendices provide supporting background.

Implementation examples expose timing, memory, communication, and actuation mechanisms. They are not tutorials for a particular robotics middleware package or simulation product. The intended result is the ability to reason about a new machine and recognize when its design needs additional specialist analysis or measurement.

DRAFT

0

Acknowledgments

PLACEHOLDER. This book's acknowledgments are still to be written, recording the people whose expertise, interviews, and industry collaboration shaped its development.

DRAFT

DRAFT

0

A Note on AI Assistance

Working draft

This text is an evolving working draft. As I collect ideas and organize this cross-disciplinary material, these chapters represent an active work in progress. While I develop each section carefully from foundational control literature, robotics principles, and verified computation cells, I have not yet reached the stage of stress-testing these frameworks with external domain specialists. I am sharing this draft openly to organize my thoughts and invite early community feedback as the curriculum takes shape.

This book is released as an open-access, living textbook; it is not yet under contract with a commercial publisher. In digital machine learning, computation operates behind glass: an unhandled exception produces a log trace, and latency costs user patience. In Physical AI, computation commands matter, momentum, and current. Latency is paid in millimeters of uncontrolled travel, and software decisions carry direct kinetic consequences.

Crossing that boundary required reconciling two engineering traditions that rarely speak the same language. On one side stands classical robotics and control theory, built on state-space dynamics, kinematics, and formal stability proofs. On the other stands modern machine learning systems, built on statistical optimization, memory hierarchies, and massive hardware acceleration.

Bridging these disciplines is difficult because their literatures are deeply siloed. In developing this curriculum, I used generative AI as an interactive research translator and cross-domain synthesis engine. Large language models served as a lateral search tool, querying across disparate bodies of robotics, control, and systems literature to connect classical methods with modern learned representations. The tool helped surface historical precedents, identify relevant mathematical formulations across disciplines, and explore alternative pedagogical framings.

Yet an AI tool cannot test whether a control loop survives contact with physical hardware. Discovering mathematical parallels between neural networks and control theory is easy; deciding which abstractions hold under real sensor noise, actuator saturation, and hard millisecond deadlines requires human systems judgment. Translating these cross-disciplinary connections into a disciplined, first-principles textbook required months of intensive analysis, iterative drafting, and building executable models.

Authorship in this domain is stewardship: uniting two historically divided fields into a cohesive curriculum and assuming responsibility for the result. The breakthroughs belong to the control theorists, roboticists, and systems researchers cited throughout these pages. My name is on the cover so that readers and engineers have a single human being to hold accountable, someone to reach out to for debates, clarifications, and corrections.

Because physical systems carry real-world consequences, this text should be approached as an evolving working draft. While our executable Python cells verify numerical consistency and automated test suites check cross-references, I am actively refining these connections before formally stress-testing them with robotics and control specialists. I invite readers, roboticists, and control engineers to engage with these ideas, report errors on GitHub, and help refine this material as the curriculum takes shape. The principle holds with absolute finality at the physical boundary: **the tool can propose, and the tool can tutor, but only the human engineer can ratify and take responsibility.**

DRAFT

0

Notation

Physical AI draws on three disciplines at once. Machine learning supplies the foundation models, vision-language-action architectures, diffusion policies, and action chunking that propose what a robot should do. Robotics and control supply the mechanics that decide what it can do: articulated dynamics, the Lie group $SE(3)$, contact and friction, Lyapunov stability, control barrier functions, and actuator electromechanics. Embedded and real-time systems supply the machinery that decides when it happens, from interconnect and memory contention through zero-order hold, real-time scheduling, leases, hardware watchdogs, and functional safety.

Because each discipline developed its mathematical conventions independently over decades, combining them creates symbol collisions that cyber-physical systems engineering has long lived with. For example, writing $\nabla_\theta \pi_\theta(\mathbf{a} \mid \mathbf{q})$ when θ also denotes joint angles, or writing $\mathbf{a}_t$ for an action when $\mathbf{a}$ denotes Cartesian acceleration or barrier constraints, confuses spatial mechanics with parameter optimization and control execution authority.

The sections that follow state the cross-discipline disambiguation rules, the governing relationships of embodied systems, and the symbol tables used throughout this book. Every symbol of the series core keeps its foundational meaning: T is time, BW bandwidth, L_{lat} latency, R_{peak} peak rate, O operations, I arithmetic intensity, D_{vol} bytes moved, $\mathcal{L}$ loss, η learning rate, and $\mathcal{O}(\cdot)$ asymptotic cost. Serving and memory arithmetic on the robot's onboard computer uses those symbols, with a subscript naming the instance (BW_{mem}, O_{prefill}).

Cross-Discipline Disambiguation

When physical plant dynamics meet deep neural networks and real-time embedded hardware, common mathematical shorthand breaks down. The conventions below resolve these cross-domain collisions systematically:

Symbol	Canonical Convention	Colliding Meaning	Disambiguation Rule
θ	Learned model (policy) parameters, as in π_θ	Joint angle, trajectory polynomial $\theta(t)$, heading, azimuth, attitude or mounting error, clock offset, or thermal resistance θ_{JA}	**Bare θ is always the parameter collection of a learned model**, set italic (never bold), as the series core defines it. A geometric angle always carries a subscript that names it: θ_{joint} for one joint (θ_{motor} and θ_{link} on either side of a reducer, θ_{enc} for an encoder reading), θ_{head} for planar heading, θ_{az} for azimuth, θ_{att} for attitude error, θ_{ext} for extrinsic misalignment, θ_{tilt} for tilt from vertical, θ_{tol} for an orientation tolerance, and θ_e for the electrical rotor angle; increments keep the subscript ($\Delta\theta_{\text{joint}}$, $\delta\theta_{\text{enc}}$). Joint configurations are the generalized coordinates $\mathbf{q} \in \mathbb{R}^n$ with entries q_i, and a joint trajectory polynomial is $q(t) = \sum a_k t^k$ with boundary values q_0, q_1 (never $\theta(t)$). A clock offset is θ_{offset} (in seconds). Thermal resistance is R_{th} or $R_{\text{th,JA}}$ (in K/W), never θ_{JA}.
a vs. $\mathbf{u}_{\text{nom}}$ vs. $\mathbf{u}^*$ vs. τ	Four-stage authority chain ($\mathbf{a}_t \to \mathbf{u}_{\text{nom}} \to \mathbf{u}^* \to \tau$)	Generic "action" symbol colliding with deceleration a_{brake} or a barrier constraint row	**Disambiguate execution authority**: $\mathbf{a}_t$ is the unprivileged policy action (dimensionless/normalized); $\mathbf{u}_{\text{nom}}$ is the proposed plant control setpoint; $\mathbf{u}^* \in \mathcal{U}_{\text{adm}}$ is the certified, CBF-filtered control command; $\tau \in \mathbb{R}^n$ is realized physical actuator torque. Control commands are bold $\mathbf{u}$ (with time index $\mathbf{u}_t$). Credible linear deceleration is a_{brake} (m/s^2); stacked barrier constraints are $\mathbf{A}_{\text{cbf}}(\mathbf{x})\mathbf{u} \le \mathbf{b}_{\text{cbf}}(\mathbf{x})$, and a single constraint row is $\mathbf{a}_{\text{cbf}}(\mathbf{x})^\top \mathbf{u} \le b_{\text{cbf}}(\mathbf{x})$.

Symbol	Canonical Convention	Colliding Meaning	Disambiguation Rule
τ **vs.** τ	Actuator torque vector $\tau \in \mathbb{R}^n$ (N · m or N)	Time constants (τ_e, τ_{th}), delays (τ_{delay}, τ_{ZOH}), observation age, lease durations, or real-time task names	**Bare or bold τ is always torque** ($\tau \in \mathbb{R}^n$, scalar joint torque τ_i, component torques such as τ_{motor}, τ_{hold}, τ_{bias}), with $\tau_{\text{motor}} = K_t i_q$ at the motor. A subscripted Greek τ marks a time only for these named constants and delays: τ_{delay} (pre-brake delay, excluding Δ_{da}) and its ceiling $\tau_{\text{delay,max}}$, $\tau_e = L_{\text{phase}}/R_{\text{phase}}$ (electrical), $\tau_{\text{th}} = R_{\text{th}} C_{\text{th}}$ (thermal, with stage constants such as $\tau_{\text{th,cu}}$ and $\tau_{\text{th,case}}$), τ_{fall} (balance divergence), τ_{ZOH} (sample-and-hold delay), τ_{ev} (evidence horizon), and τ_{blend} (authority blend window). Every other interval uses Latin T or t: observation age is t_{age}, never τ; leases are T_{lease}, $T_{\text{lease,man}}$, and T_{admin}; compute and fieldbus times are T_{comp} and T_{bus}; a pure transport delay is T_d. A real-time task is named "task i", never τ_i.
x vs. s **vs. p vs.** $\mathbf{x}_p$	Mechanical plant state $\mathbf{x} = [\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top \in \mathbb{R}^{2n}$	World state $s \in \mathcal{S}$, 3D position $\mathbf{p} = [x, y, z]^\top$, image point, or ViT image patch $\mathbf{x}_p$	**Continuous mechanical plant state is bold vector** $\mathbf{x} \in \mathbb{R}^{2n}$. Ground-truth latent world state is italic s_t (a mixed-type state, never bold). 3D Cartesian position is vector $\mathbf{p} \in \mathbb{R}^3$, including a camera-frame point $\mathbf{p}_c$ and a voxel center $\mathbf{p}_{\text{vox}}$; the translation of a rigid transform is $\mathbf{t}$. An image point in homogeneous pixel coordinates is $\tilde{\mathbf{m}} = [u, v, 1]^\top$, never $\mathbf{x}$ or $\mathbf{u}$. Vision transformer patch embeddings use explicitly subscripted $\mathbf{x}_p$ or latent feature $\mathbf{z}$.
P	Model parameter count P (series core)	Power (P_{loss}, P_{peak}), estimation covariance $\mathbf{P}$, failure probability or rate, tail-latency percentile, or 3D point $\mathbf{p}$	Use P for parameter count. Electrical/mechanical power always carries a subscript (P_{loss}, P_{peak}, P_{heat}, in Watts); the estimator's error covariance is bold matrix $\mathbf{P}$, and a generic Gaussian covariance is Σ; a per-trial probability is lowercase italic p ($p_{\text{fail}}, p_{\text{target}}$), a rate per hour is λ ($\lambda_{\text{system},h}, \lambda_{\text{target}}$), and an event probability is $\Pr(\cdot)$; percentiles are upright p50, p95, and p99 (never P_{99}); 3D Cartesian position is bold lowercase $\mathbf{p} = [x, y, z]^\top$.
B	Batch size B (series core)	Bandwidth (BW), Body frame ($\mathcal{F}_B$), bus transfer size, SPRT boundary, or input matrix $\mathbf{B}$	Batch size is B; sustained bandwidth is BW (per the single-device decomposition), with a subscript naming the path (BW_{mem}, BW_{chirp}); coordinate reference frames use calligraphic script ($\mathcal{F}_W, \mathcal{F}_B, \mathcal{F}_E$); the bytes of bus transaction k are $D_{\text{vol}\,k}$; SPRT boundaries are Λ_{up} and Λ_{lo}; state-space control input matrix is bold capital $\mathbf{B} \in \mathbb{R}^{n \times m}$.
α	Bumpless transfer blending function $\alpha(\hat{t}) \in [0, 1]$	CBF decay (γ_{cbf}), significance (α_{sig}), angular acceleration (α), copper tempco (α_{Cu}), or noise schedule ($\bar{\alpha}_k$)	Authority transfer uses scalar function $\alpha(\hat{t}) \in [0, 1]$ of normalized time $\hat{t} \in [0, 1]$; barrier linear decay uses $\gamma_{\text{cbf}} > 0$; statistical confidence significance uses $\alpha_{\text{sig}} = 1 - \text{CL}$; angular acceleration is bold vector $\boldsymbol{\alpha} \in \mathbb{R}^3$ (rad/s^2); copper resistance tempco is α_{Cu}; diffusion noise schedules use indexed $\bar{\alpha}_k$.
$\mathbf{C}$	Coriolis matrix $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{n \times n}$	Safe set $\mathcal{C}$, conformal prediction set, assurance claim, state-space output matrix, controllability matrix, capacitance ($C_{\text{th}}, C_{\text{bus}}$), CoM $\mathbf{c}$, or shield coverage c_h	Coriolis/centrifugal forces use matrix function $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$, always written with its arguments; forward-invariant safe sets use calligraphic $\mathcal{C} \subset \mathbb{R}^{2n}$, defined on the plant state, and the conformal prediction set $\mathcal{C}$ of target grounding shares that glyph but never appears beside a safe set; the top-level claim of an assurance argument is italic C; the state-space output map is $\mathbf{H}$ (the same map as the Kalman measurement map), never $\mathbf{C}$; the controllability matrix is $\mathbf{C}_{\text{ctrb}}$; electrical/thermal capacitance uses italic C with a subscript; center of mass is bold vector $\mathbf{c} \in \mathbb{R}^3$; safety shield coverage of hazard class h uses $c_h \in [0, 1]$.
$\mathbf{A}$	Action chunk $\mathbf{A}_t$, $\mathbf{A}_{t:t+H}$, and its denoising iterate $\mathbf{A}_t^k$	Continuous LTI state matrix, barrier constraint matrix	Bold $\mathbf{A}$ with a time or denoising index is the action chunk. The continuous LTI state matrix is $\mathbf{A}_{\text{sys}}$ (its discretizations $\mathbf{A}_{\text{FE}}$ and $\mathbf{A}_{\text{BE}}$, the discrete Kalman transition $\mathbf{F}$); stacked barrier constraints use $\mathbf{A}_{\text{cbf}}$.
$\mathbf{v}$	Velocity ($\mathbf{v}$, end-effector velocity $\mathbf{v}_e$)	Measurement noise, voxel	Bold $\mathbf{v}$ is a velocity. Measurement noise is $\mathbf{v}_{\text{meas}}$ with covariance Σ_{meas} (bold $\mathbf{R}$ is reserved for rotation); a voxel is addressed by its center $\mathbf{p}_{\text{vox}}$.

Symbol	Canonical Convention	Colliding Meaning	Disambiguation Rule
K	Camera intrinsic calibration matrix $\mathbf{K} \in \mathbb{R}^{3\times3}$	Contact stiffness (k_{contact}), control gains (K_p, K_d, LQR gain), motor constants (K_t, K_e), ensembling window K, or diffusion step k	Camera intrinsics use bold matrix $\mathbf{K}$; contact stiffness is k_{contact}, with body-region stiffness k_H (N/m); feedback control gains use capitalized scalars K_p, K_d, and a state-feedback gain matrix is $\mathbf{K}_{\text{lqr}}$; motor torque/back-EMF constants use K_t, K_e (per joint K_{t_i}); temporal ensembling window length uses integer K; the number of denoising or solver evaluations per chunk is N_{eval}, never K; diffusion noise steps use lowercase index k; the Kalman gain is $\mathbf{G}_k$.
M	Articulated inertia matrix $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n\times n}$	Mass (m, m_{eff}), memory footprint (M_{weights}, M_{KV}), or fleet size (N_{fleet})	Articulated generalized mass/inertia uses bold matrix $\mathbf{M}(\mathbf{q})$; physical body mass is lowercase italic m (kg); effective impact mass is m_{eff}; hardware memory storage uses capital M with descriptive subscript (in bytes); robot fleet size uses N_{fleet}.
T	Time (series core)	Temperature	**Bare T is always time.** A temperature always carries a subscript that names where it is measured: T_{winding} (winding), T_{core}, T_{j} (junction), T_{case}, T_{amb} (ambient), T_{limit} (rated limit), T_{trip}, T_{∞} (equilibrium), and T_0 (the reference temperature of α_{Cu}). A temperature rise is ΔT (in K); a time interval is Δt, never ΔT.
SIL (abbreviation)	Safety integrity level (IEC 61508 family), a grade of a safety function	Software-in-the-loop, the first rung of the verification ladder	**Spell out at first use in each chapter.** "SIL 2" and similar graded forms always mean safety integrity level; the verification ladder writes software-in-the-loop (SIL) only where the ladder's abbreviations MIL, SIL, PIL, and HIL appear together, and never beside a safety integrity level.

Governing Relationships

Physical intelligence operates subject to thermodynamic, mechanical, kinematic, and communications constraints. The relationships below state those constraints in the symbols this book uses. They are working relationships, not laws: the book's four bedrock laws are the ones section 1.10 states.

The sense-to-actuation delay budget

Delay converts directly into travel before any retarding force begins. The pre-brake delay τ_{delay} runs from the capture of the frame that first shows a hazard to retarding force, excluding the dispatch-to-admission interval Δ_{da}, which the stopping budget charges against its timing allowance. The stop it leaves, whose travel is $d_{\text{suffix}}(v)$, must fit inside the clear distance, and the spare that remains must pay for $v\,\Delta_{\text{da}}$:

$$\tau_{\text{delay}} = t_{\text{age}} + T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}$$

$$v\,\tau_{\text{delay}} + d_{\text{suffix}}(v) + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} \le D_{\text{clear}}$$

Inference time is spent on the proposal side of the proposal boundary. It ages the evidence behind a proposal, which the permission path checks against the proposal's expiry. It is neither a term of τ_{delay} nor part of the permission loop's period T_{tick}, because the fallback does not wait for the proposer.

The appendices regroup the same interval for two specific stop paths, and each form defines its own terms. For a stop triggered by lease expiry, $\tau_{\text{delay}} = T_{\text{lease}} + t_{\text{detect}} + T_{\text{hold}} + t_{\text{age}} + T_{\text{act}}$; for a stop triggered by a hazard detector, $\tau_{\text{delay}} = t_{\text{age}} + t_{\text{detect}} + T_{\text{compute}} + T_{\text{bus}} + T_{\text{act}}$.

Symbol	Definition	Unit	Notes
τ_{delay}	**Pre-Brake Delay**	ms	Time from capture of the frame that first shows a hazard to the onset of retarding force, excluding Δ_{da}; the machine travels $v(\tau_{\text{delay}} + \Delta_{\text{da}})$ before braking begins. Its allowed ceiling is $\tau_{\text{delay,max}}$.

Symbol	Definition	Unit	Notes
t_{age}	**Observation Age**	ms	Age of the state the permission path acts on, from the exposure midpoint at the sensor to the decision, which enters τ_{delay}. Its perception-side terms are the exposure midpoint $\frac{1}{2}t_{\text{exp}}$, sensor readout, transfer, and perception processing; a bound on it is $t_{\text{age,max}}$.
T_{lease}	**Chunk Lease**	ms	Longest interval an admitted chunk may keep driving the actuators without renewal, which enters τ_{delay}; chosen, then checked against the ceiling the stopping budget derives.
T_{chunk}	**Chunk Renewal Period**	ms	Interval between successful admissions of the chunk policy; with T_{jitter} it sets the floor of the lease rule, $T_{\text{chunk}} + T_{\text{jitter}} < T_{\text{lease}}$ (equation 4.4).
T_{jitter}	**Renewal-Jitter Allowance**	ms	Allowance for variation in the interval between successful admissions; chosen, then checked against measured renewal intervals.
T_{tick}	**Permission-Loop Period**	ms	Period of the permission loop, which enters τ_{delay} because a hazard may wait up to one period before it is checked.
T_{bus}	**Command Transport Time**	ms	Fieldbus delivery time from the permission path to the drive, which enters τ_{delay}.
T_{act}	**Actuator Response Time**	ms	Time from a delivered stop command to retarding force, including gate-drive and current rise, which enters τ_{delay}.
t_{detect}	**Detection Interval**	ms	Recognition or expiry-detection time after the decision-start epoch, not already charged to t_{age}; a term of the lease-expiry and hazard-detection forms of τ_{delay}.
T_{hold}	**Fallback Hold**	ms	Optional, separately authorized hold or fault-confirmation interval before the stop command reaches the drive; zero when the stop path has none.
t_{infer}	**Proposer Inference Time**	ms	Time for one complete evaluation of the learned proposer (prefill and decode); spent above the proposal boundary and outside the permission loop's period.
Δ_{da}	**Dispatch-to-Admission Interval**	ms	Elapsed time from perception dispatch to the admission of the candidate it conditions; evidence keeps aging through it. Not a term of τ_{delay}; its travel $v\,\Delta_{\text{da}}$ is charged against the stopping budget's timing allowance.
$T_{\Sigma,1}, T_{\Sigma,2}$	**Takeover Response Time**	ms, s	Human response plus console transport for an in-loop ($T_{\Sigma,1}$) and an out-of-loop ($T_{\Sigma,2}$) takeover; it adds to τ_{delay} while the machine waits for the person.
T_{deadline}	**Deadline**	ms, s	A duration budget a computation or handover must meet (the series symbol); qualifiers name the instance, as in $T_{\text{deadline,fallback}}$, the latest permissible wait before the fallback must begin. A deadline stated as a clock instant is lowercase, t_{deadline} or $t_{\text{deadline,stop}}$.
T_{timeout}	**Handover Timeout**	s	Timer an arbiter sets for an unacknowledged handover, $T_{\text{timeout}} = \max(0, \min(T_{\text{config}}, T_{\text{deadline,fallback}}))$, where T_{config} is the protocol ceiling.
t_{commit}	**Commitment Instant**	s	Instant after which a receding-horizon plan's prefix is committed and can no longer be replaced by a new proposal.
t_{onset}	**Hazard Onset**	s	Instant at which a hazard first becomes physically present, the reference from which detection and stopping intervals are measured.

DRAFT

Leases

A lease is authority that lapses unless it is renewed. The book uses six kinds:

Lease	Symbol	Distinction
Execution lease	remaining $T_{\text{lease,rem}}$	Any authority an admitted command source holds to keep driving the actuators without renewal; it begins at admission, and its expiry invokes the state-matched fallback.
Chunk lease	T_{lease}	The execution lease of the autonomous chunk policy, renewed by each admitted chunk; it enters τ_{delay}.
Manual lease	$T_{\text{lease,man}}$	The execution lease of a human operator in manual mode, renewed by authenticated heartbeats; it replaces the chunk lease in τ_{delay} while the operator drives.
Administrative lease	T_{admin}	The host-silence bound inside an intent lease: how long the application processor may hold a goal without renewing it.
Intent lease	expiry $t_{\text{intent_exp}}$	The record that grants a grounded target and effort ceilings, never a path; it expires at the earliest of its evidence horizon, its administrative lease, and its stopping deadline.
Hardware lease	(mechanism)	The countdown timer on the safety microcontroller that enforces any of these expiries even when the process that issued the authority has hung.

Single-device execution time

In a serial model without overlap, one evaluation of a learned proposer on the robot's onboard accelerator takes a time that splits into three terms: moving bytes through memory, executing operations, and a fixed overhead. The chunk-cadence and proposer-inference relationships below reuse these symbols, with a subscript naming the instance (BW_{mem}, O_{prefill}):

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Memory Time}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute Time}} + \underbrace{L_{\text{lat}}}_{\text{Latency Overhead}}$$

DRAFT

Symbol	Definition	Unit	Notes
T	**Execution Time**	seconds	Total execution time of one evaluation; bare T is always time.
BW	**Bandwidth**	bytes/s	Sustained bytes per second across a named path; never bare B, which is batch size B.
D_{vol}	**Data Volume**	bytes	Bytes moved through memory in one evaluation; never bare D, which is dataset size.
η_{hw}	**Efficiency**	—	Fraction of the peak rate an evaluation achieves, $0 \leq \eta_{\text{hw}} \leq 1$; never bare η, which is the learning rate.
L_{lat}	**Latency**	seconds	Fixed overhead per evaluation (kernel launch, transfer setup); the subscript keeps it distinct from the loss $\mathcal{L}$.
O	**Operations**	FLOPs	Floating-point operations one evaluation executes; a subscript names the phase (O_{prefill}). Asymptotic cost is $\mathcal{O}(\cdot)$.
R_{peak}	**Peak Rate**	FLOP/s	Peak arithmetic rate of the processor at the stated precision; never bare P, which is parameter count.

Action-chunk cadence and proposer inference

Large neural policies that stream their weights from memory may exceed a kilohertz control period. Action chunking predicts an action trajectory of horizon H at a lower cognitive frequency f_{policy}, while the chunk's waypoints are consumed at f_{exec}. In the serial model with one weight-streaming pass per chunk and no additional fixed overhead, memory streaming must fit inside the chunk execution window after reserving the compute term T_{compute} of the single-device decomposition:

$$t_{\text{stream}} = \frac{M_{\text{weights}}}{\text{BW}_{\text{mem}}} \leq \frac{H}{f_{\text{exec}}} - T_{\text{compute}}$$

A decoder that streams its nonresident weights once per emitted token spends, for S_{out} emitted tokens, the inference time

$$t_{\text{infer}} \approx \max\left(\frac{O_{\text{prefill}}}{\eta_{\text{hw}} R_{\text{peak}}}, \frac{M_{\text{weights}}}{\text{BW}_{\text{mem}}}\right) + (S_{\text{out}} - 1)\frac{M_{\text{weights}}}{\text{BW}_{\text{mem}}}$$

which caps the fresh-evaluation rate at $f \leq \text{BW}_{\text{mem}}/M_{\text{weights}}$ when every evaluation streams the weights. The symbols are those of the single-device decomposition: O_{prefill} counts the prefill's operations, R_{peak} is the processor's peak rate at the activation precision, η_{hw} is the fraction of it the prefill achieves, and I is the arithmetic intensity that decides which term binds.

Symbol	Definition	Unit	Notes
H	**Horizon**	Time steps	Number of contiguous future action steps generated in a single policy forward pass ($H = 16$ on the mobile manipulator; 16 to 64 steps in the literature).
d_a	**Action Dimension**	Integer	Number of components of one action $\mathbf{a}_t$; a chunk $\mathbf{A}_{t:t+H}$ has $H \times d_a$ entries.
BW_{mem}	**Sustained Memory Bandwidth**	bytes/s	Achievable memory throughput of the SoC or accelerator memory subsystem (e.g., LPDDR5, HBM); the core's BW at the memory interface.
t_{stream}	**Parameter Streaming Time**	ms	Time required to stream non-resident model weights into execution registers.
f_{exec}	**Action Execution Rate**	Hz	Temporal consumption rate of chunked waypoints along the planned trajectory (100 Hz to 1 kHz).
S_{out}	**Emitted Token Count**	tokens	Output tokens an autoregressive decoder emits per evaluation; the core's sequence length S for the decoded part. Never D, which is dataset size.

DRAFT

Stopping, braking, and safe-set invariants

For the stopping-based safety envelope, the machine must retain enough clearance to arrest its approach before crossing the protected boundary. The relationship below specializes the resident stop to one-dimensional motion starting and ending at zero acceleration, with symmetric jerk ramps and a constant-deceleration segment. The stopping distance combines travel during the pre-brake delay with braking travel; the delay-budget inequality adds the spatial margins:

$$d_{\text{stop}}(v) = v\,\tau_{\text{delay}} + d_{\text{suffix}}(v), \qquad d_{\text{suffix}}(v) = \frac{v^2}{2a_{\text{brake}}} + \frac{v\,a_{\text{brake}}}{2j_{\text{max}}} \quad (v \geq a_{\text{brake}}^2/j_{\text{max}})$$

Symbol	Definition	Unit	Notes
d_{stop}	**Stopping Distance**	m	Distance required to bring the plant to rest from speed v.
T_{stop}	**Stopping Duration**	s	Elapsed time from braking onset to standstill in the protective-separation calculation; distinct from controller sampling period.
D_{clear}	**Clear Distance**	m	Clear distance from the machine to the boundary it must not cross; the subscript keeps it distinct from dataset size D.
v	**Speed at Detection**	m/s	Speed of the machine at the moment the hazard is detected; in stopping, pre-brake, and impact algebra, the scalar travel or approach speed v.
v_{max}	**Speed Bound**	m/s	Largest speed the machine, a tracked object, or an approaching body is credited with in a budget; the bounding speed in $\Delta t_{\text{fresh}} \leq e_{\text{max}}/v_{\text{max}}$.
a_{brake}	**Credible Deceleration**	m/s^2	Validated low tail of the deceleration the machine achieves under its stated conditions, not the datasheet peak.

Symbol	Definition	Unit	Notes
$a_{\max}$	**Profile Acceleration Bound**	$\mathrm{m/s^2}$	Largest acceleration or deceleration a planned or interpolated profile may command; for the base's resident stop it is set to a_{brake}.
a_{net}	**Net Braking Deceleration**	$\mathrm{m/s^2}, \mathrm{rad/s^2}$	Linear or angular deceleration left after a gravity or load bias torque τ_{bias} is subtracted from the actuator's braking capacity, in the direction of travel.
$d_{\text{suffix}}(v)$	**Resident Stop Distance**	m	Travel of the resident stop from speed v: $v^2/(2a_{\text{brake}})$ for constant-deceleration braking; for the jerk-limited stop of equation 11.17, that term plus the jerk ramps' $v\,a_{\text{brake}}/(2j_{\max})$ once $v \geq a_{\text{brake}}^2/j_{\max}$.
δ_{loc}	**Localization Bound**	m	Bound on localization and state-estimation error.
δ_{margin}	**Protective Clearance**	m	Fixed, nonzero protective clearance between the machine envelope and the boundary.
ϵ_{track}	**Tracking-Error Bound**	m	Bound on the deviation between the commanded and the executed trajectory, stated by the enforcement record.
v_h	**Human Walking-Speed Bound**	m/s	Walking-speed bound for a person approaching the machine, used in the form its safety standard states.
$j_{\max}$	**Jerk Limit**	$\mathrm{m/s^3}$	Maximum rate of change of deceleration; sets the jerk ramps of the resident stop in $d_{\text{suffix}}(v)$ (equation 11.17) and bounds the bumpless-transfer window τ_{blend}.

Zero-order hold and phase margin

At crossover frequencies well below the sampling frequency, holding discrete control commands constant over sampling period $T_s = 1/f_s$ contributes approximately half a sample period of delay. With fixed modeled enforcer and fieldbus delays, the relationship below estimates phase-margin erosion; its phase-lag term is in radians, so all phase margins must use the same angular unit before subtraction:

$$\tau_{\text{ZOH}} = \frac{T_s}{2}, \qquad \Delta\phi = \omega_c \left(\frac{T_s}{2} + T_{\text{comp}} + T_{\text{bus}} \right), \qquad \text{PM}_{\text{net}} = \text{PM}_{\text{plant}} - \Delta\phi > 0$$

Symbol	Definition	Unit	Notes
τ_{ZOH}	**ZOH Transport Delay**	s	Equivalent continuous delay introduced by discrete Zero-Order Hold reconstruction ($T_s/2$).
T_s, f_s	**Sampling Period and Rate**	s, Hz	Discrete controller execution period and frequency ($f_s = 1/T_s$).
T_{comp}	**Enforcer Compute Time**	s	Time the enforcer takes to evaluate its constraints within one control period; with T_{bus} it adds to the ZOH delay.
T_d	**Pure Transport Delay**	s	A fixed delay in a feedback or communication channel, contributing phase lag ωT_d at frequency ω.
ω_c	**Open-Loop Crossover Frequency**	rad/s	Frequency at which open-loop gain $\lvert L(j\omega_c)\rvert = 1$ (0 dB).
$\Delta\phi$	**Phase Lag Degradation**	rad / deg	Phase erosion at crossover ($\Delta\phi \approx 28.65^\circ \cdot (\omega_c/f_s)$ from the hold alone) depleting closed-loop stability margin.

Actuator electrical and thermal limits

Actuators are constrained by copper winding heating, DC bus rail droop, and magnetic saturation. Sustainable torque output is governed by the coupled electromechanical balance, written per joint with scalar torque:

$$V_{\text{bus}} = L_{\text{phase}} \frac{di_q}{dt} + R(T_{\text{winding}})\, i_q + K_e \omega_m, \qquad C_{\text{th}} \frac{dT_{\text{winding}}}{dt} = i_q^2 R(T_{\text{winding}}) - \frac{T_{\text{winding}} - T_{\text{amb}}}{R_{\text{th}}}, \qquad \tau = \eta_{\text{gear}} N_{\text{gear}} K_t i_q$$

Symbol	Definition	Unit	Notes
K_t, K_e	**Torque and Back-EMF Constants**	$\mathrm{N\cdot m/A}, \mathrm{V\cdot s/rad}$	Motor electromechanical coupling constants linking winding current to rotor shaft torque and rotor speed to back-EMF voltage ($K_t \approx K_e$ in SI units).
i_q	**Torque-Producing Current**	A	Quadrature-axis current of a field-oriented drive, $\tau_{\text{motor}} = K_t i_q$. Bare I is never a current, because the series core reserves it for arithmetic intensity.
I_{phase}	**Motor Phase RMS Current**	A	RMS phase current driving the stator windings of a brushless PM motor; an uppercase I with a subscript names an RMS value or rating.
i_{phase}	**Instantaneous Phase Current**	A	Signed instantaneous electrical phase current, positive from inverter node into winding; distinct from RMS phase current and torque-producing dq current.
$I_{\text{cont}}, I_{\text{peak}}$	**Continuous / Peak Current**	A	RMS current ratings bounded by winding insulation class and inverter MOSFET dissipation.
$L_{\text{phase}}, R_{\text{phase}}$	**Phase Inductance and Resistance**	H, Ω	Stator winding inductance and DC resistance; they set $\tau_e = L_{\text{phase}}/R_{\text{phase}}$.
$R(T_{\text{winding}})$	**Temperature-Dependent Resistance**	Ω	Stator copper resistance at winding temperature T_{winding}: $R(T_{\text{winding}}) = R_0[1+\alpha_{\text{Cu}}(T_{\text{winding}} - T_0)]$ with $\alpha_{\text{Cu}} = 0.00393\ \mathrm{K}^{-1}$.
R_{th}	**Thermal Resistance**	K/W	Lumped thermal resistance from stator windings to the ambient environment; path segments carry a second subscript ($R_{\text{th,JA}}$, $R_{\text{th,JC}}$, $R_{\text{th,wc}}$, $R_{\text{th,ca}}$) and add in series.
C_{th}	**Stator Thermal Capacitance**	J/K	Lumped heat capacity of the motor core, windings, and potting compound.
τ_e	**Electrical Time Constant**	s	Winding current rise time constant $\tau_e = L_{\text{phase}}/R_{\text{phase}}$; current reaches $1-e^{-1}$ of its target in τ_e.
τ_{th}	**Thermal Time Constant**	s	Actuator heating characteristic time constant ($\tau_{\text{th}} = R_{\text{th}}C_{\text{th}}$, typically 60–300 s); a single stage takes a second subscript ($\tau_{\text{th,cu}}$, $\tau_{\text{th,case}}$).
$T_{\text{winding}}, T_{\text{amb}}$	**Winding and Ambient Temperature**	K or °C	The thermal model's state and boundary temperatures; the other temperatures follow the subscript rule of the disambiguation table (T_j, T_{case}, T_{limit}, T_∞, T_0).
ΔT	**Joulean Temperature Rise**	K	Thermal rise above ambient; $\Delta T = P_{\text{loss}}R_{\text{th}}$ gives the steady-state rise for constant loss power in the lumped model. Transient rise follows the thermal balance; never a time interval.
D_{cycle}	**Actuator Duty Cycle**	$[0, 1]$	Ratio of active pulse duration to cycle period ($t_{\text{on}}/(t_{\text{on}}+t_{\text{off}})$) preventing thermal runaway.

Shielded hazards and the exposure bound

Catastrophic failure rates in embodied systems cannot be certified through stochastic testing alone. For a defined hazard class h, verified interception coverage and independently bounded residual faults constrain the system's hazard rate. Under a constant-rate Poisson exposure model, zero-failure exposure follows the Rule of Three:

$$\lambda_{\text{system},h} \le \lambda_{\text{proposal},h}(1-c_h) + \lambda_{\text{other}} \le \lambda_{\text{target}}, \qquad T \ge \frac{-\ln \alpha_{\text{sig}}}{\lambda_{\text{target}}}$$

Here T is the zero-incident operating exposure, in hours. For independent trials with a common success probability, per-trial campaigns use the same bound in probability form, $n \ge \ln \alpha_{\text{sig}} / \ln p_{\text{target}} \approx 3/(1-p_{\text{target}})$.

Symbol	Definition	Unit	Notes
$\lambda_{\text{system},h}$	**System Hazard Rate**	h^{-1}	Certified dangerous failure rate of the integrated cyber-physical machine for hazard class h.

Symbol	Definition	Unit	Notes
$\lambda_{\text{proposal},h}$	**Hazardous Proposal Rate**	h^{-1}	Arrival rate of unshielded hazardous action proposals the policy emits before the permission layer, measured against a counterfactual oracle.
c_h	**Shield Interception Coverage**	Dimensionless	Verified conditional probability that the shield intercepts a hazardous proposal of class h, evaluated over the states in which the policy errs.
λ_{other}	**Hardware and Common-Cause Rate**	h^{-1}	Independently bounded hardware-fault and common-cause hazard rate the shield cannot intercept; the floor $\lambda_{\text{system},h}$ reaches when $c_h = 1$.
λ_{target}	**Target Hazard Rate**	h^{-1}	Tolerable dangerous failure rate assigned to the safety function by risk assessment, in the band its governing standard defines.
C_{diag}	**Diagnostic Coverage**	Dimensionless	Rate-weighted fraction of hardware faults whose hazardous effect is mitigated, $\lambda_{\text{unmitigated}} = \lambda_{\text{raw}}(1 - C_{\text{diag}})$; the hardware counterpart of c_h, which covers policy proposals.
$p_{\text{fail}}, p_{\text{target}}$	**Per-Trial Probabilities**	Dimensionless	Per-trial failure probability, and the per-trial success probability a campaign must demonstrate; trial counts follow $n \geq \ln \alpha_{\text{sig}} / \ln p_{\text{target}}$.
α_{sig}	**Statistical Significance Level**	Dimensionless	Type I error threshold ($\alpha_{\text{sig}} = 1 - \text{CL}$, e.g., $\alpha_{\text{sig}} = 0.05$ for 95% confidence where $-\ln(0.05) \approx 3.0$).
β	**Type II Error Allowance**	Dimensionless	Probability of rejecting an acceptable candidate ($1 - \beta$ is the power); with α_{sig} it sets the SPRT boundaries Λ_{up} and Λ_{lo}. Diffusion schedules use indexed β_s; a KL weight β stays local to its objective.
n	**Zero-Failure Trial Count**	Trials	Required number of incident-free trials to substantiate p_{target}; for exposure measured in hours, the bound is on T instead.
Λ_m	**Sequential Log-Likelihood Ratio**	Dimensionless	After k failures in m trials, $\Lambda_m = k \ln(p_1/p_0) + (m-k) \ln \frac{1-p_1}{1-p_0}$ for per-trial failure probabilities p_0 (unacceptable) and p_1 (acceptable); SPRT stops when it leaves $(\Lambda_{\text{lo}}, \Lambda_{\text{up}})$.
$\Lambda_{\text{up}}, \Lambda_{\text{lo}}$	**SPRT Boundaries**	Dimensionless	Upper and lower stopping thresholds on Λ_m, $\Lambda_{\text{up}} \approx \ln((1-\beta)/\alpha_{\text{sig}})$ and $\Lambda_{\text{lo}} \approx \ln(\beta/(1-\alpha_{\text{sig}}))$; never bare A and B.
C	**Assurance Claim**	Proposition	Top-level safety claim of an assurance argument. Under an independent-premise probability model, its confidence $\Pr(C)$ is the product of the premise confidences for a conjunctive argument, and one minus the product of the legs' failure probabilities for a multi-legged one.

The embodied energy corollary

Untethered mobile manipulators and autonomous robots operate under strict battery energy reserves and peak thermal dissipation ceilings. Total mission energy (distinct from the ML workload's E_{total}) splits across mechanical actuation work, edge neural computation, and auxiliary sensor payloads. The compute energy expended by edge inference system-on-chip (SoC) devices decomposes into data movement and arithmetic operations according to the core energy corollary:

$$E_{\text{mission}} = E_{\text{act}} + E_{\text{total}} + E_{\text{aux}}, \qquad E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$$

For constant loss power in the single-node thermal model, resistive heating in stator windings $P_{\text{loss}} \approx \sum i_q^2 R(T_{\text{winding}})$ sets the steady-state rise $\Delta T = P_{\text{loss}} R_{\text{th}}$. Transient heating during shorter acceleration or contact bursts follows the coupled first-order thermal balance above.

Plant State, Actuation, and Kinematics

The symbols below describe the mechanical configuration, velocities, forces, and inertia of articulated and mobile mechanisms. Units for inertia and generalized loads specialize to revolute coordinates;

mixed revolute–prismatic plants require entrywise units consistent with their generalized coordinates.

Symbol	Definition	Unit	Notes
$s_t \in \mathcal{S}$	**Physical World State**	Mixed	Ground-truth continuous state of the machine, environment, and physical contacts at time t, and the state of the decision process in the learning chapters; italic, because it mixes types.
$\mathbf{x}$	**Mechanical Plant State**	Mixed: rad or m for positions; rad/s or m/s for velocities	Generalized state vector $\mathbf{x} = [\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top \in \mathbb{R}^{2n}$ containing positions and velocities.
$\mathbf{q} \in \mathbb{R}^n$	**Generalized Coordinates**	rad or m	Degrees of freedom describing mechanical articulation (joint angles and prismatic link extensions); entries q_i.
$\dot{\mathbf{q}} \in \mathbb{R}^n$	**Generalized Velocities**	rad/s or m/s	First time derivative of generalized coordinates.
$\ddot{\mathbf{q}} \in \mathbb{R}^n$	**Generalized Accelerations**	$\mathrm{rad/s^2}$ or $\mathrm{m/s^2}$	Second time derivative of generalized coordinates.
$\tau \in \mathbb{R}^n$	**Actuator Torque Vector**	N · m or N	Generalized forces and torques delivered by actuators at mechanical joints; a subscript names a source, as in the contact torque τ_c.
τ	**Commanded Joint Torque**	N · m	Scalar torque commanded at one joint in single-joint derivations such as the kinetic barrier; bounds and disturbances keep a subscript ($\tau_{\max}$, τ_{bias}).
$\mathbf{M}(\mathbf{q})$	**Generalized Inertia Matrix**	$\mathrm{kg \cdot m^2}$	Symmetric positive-definite mass/inertia matrix of the articulated multibody plant.
$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$	**Coriolis / Centrifugal Matrix**	N · m · s/rad	Matrix satisfying $\dot{\mathbf{M}} - 2\mathbf{C}$ skew-symmetry (passivity property).
$\mathbf{g}(\mathbf{q})$	**Gravitational Torque Vector**	N · m	Generalized gravitational load acting on links as a function of configuration; a single joint's share is τ_g.
$\mathbf{J}(\mathbf{q})$	**Manipulator Jacobian**	Mixed	Linear mapping $\mathbf{v}_e = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$ from joint rates to operational task-space velocity.
$w(\mathbf{q})$	**Yoshikawa Manipulability**	Depends on task coordinates and scaling	Singularity proximity metric $w(\mathbf{q}) = \sqrt{\det(\mathbf{J}\mathbf{J}^\top)}$ vanishing at kinematic singular configurations.
e_{bemf}	**Back-EMF Voltage**	V	Opposing voltage $e_{\text{bemf}} = K_e \omega_m$ developed across stator coils by rotor rotation.
$J_{\text{rotor}}, J_{\text{load}}$	**Rotor and Load Inertia**	$\mathrm{kg \cdot m^2}$	Moment of inertia of motor rotor shaft and coupled mechanical load.
J_{ref}	**Reflected Rotor Inertia**	$\mathrm{kg \cdot m^2}$	Effective inertia $J_{\text{ref}} = N_{\text{gear}}^2 J_{\text{rotor}}$ amplified quadratically by gearbox ratio.
J_{eff}	**Effective Joint Inertia**	$\mathrm{kg \cdot m^2}$	Total inertia seen at one joint, link and payload plus J_{ref}; the inertia in single-joint torque and acceleration budgets.
N_{gear}	**Gear Reduction Ratio**	Dimensionless	Transmission reduction ratio ($N_{\text{gear}} = \omega_m/\omega_j \gg 1$) between motor rotor and joint output shaft; never bare N.
N^*	**Optimal Inertia-Matching Ratio**	Dimensionless	Ratio $N^* = \sqrt{J_{\text{load}}/J_{\text{rotor}}}$ maximizing load angular acceleration under peak torque.
η_{gear}	**Transmission Efficiency**	Dimensionless	Mechanical power transmission efficiency ($0 < \eta_{\text{gear}} \leq 1$); never bare η, which is the learning rate.
$\tau_{\text{peak}}, \tau_{\text{hold}}$	**Continuous / Peak / Holding Torque**	N · m	Thermally continuous holding torque τ_{hold} and transient peak acceleration torque.
m	**Mass**	kg	Physical body or payload mass, lowercase italic; effective and reduced impact masses carry subscripts (m_{eff}, m_{red}).
g	**Gravitational Acceleration**	$\mathrm{m/s^2}$	Scalar magnitude of gravitational acceleration; the vector $\mathbf{g}_0$ and the configuration-dependent torque $\mathbf{g}(\mathbf{q})$ are bold.
p	**Linear Momentum**	kg · m/s	Scalar momentum mv of a moving body; distinct from the bold position $\mathbf{p}$ and from a distribution $p(\cdot)$.

Symbol	Definition	Unit	Notes
r	**Wheel Radius**	m	Rolling radius of a driven wheel, which converts wheel torque to floor force and speed to wheel rate ($\omega = v/r$); the barrier relative degree r appears only in barrier analysis.
ω	**Angular Speed**	rad/s	Magnitude of a rigid body's angular velocity ω; not the crossover frequency ω_c.
v	**Base Forward Speed**	m/s	Forward linear speed of the mobile base along its heading, the speed input of the unicycle model; not the speed at the instant a hazard is detected.
$\dot{\theta}_{\text{head}}$	**Base Heading Rate**	rad/s	Rate of change of a mobile base's heading angle, the turn input of the unicycle model.
$\dot{y}$	**Base Lateral Velocity**	m/s	Lateral velocity of a mobile base in the plane.
$\dot{x}$	**Base Longitudinal Velocity**	m/s	Forward velocity of a mobile base in the plane.
v	**Body Speed**	m/s	Magnitude of the linear velocity of a moving body or machine; the scalar speed, with the vector velocity bold.
$\dot{\tau}$	**Commanded Joint Torque Slew**	$\text{N}\cdot\text{m/s}$	Time derivative of the torque at a single joint; for constant effective inertia it equals J_{eff} times the joint jerk.
τ_{cont}	**Continuous Torque**	$\text{N}\cdot\text{m}$	Largest motor torque a winding can sustain indefinitely at its temperature limit under the stated cooling; set by heat rejection, and at the engineering ceiling it gives the installed rating.
i_d	**Direct-Axis Current**	A	Current of a field-oriented drive along the rotor field, which produces no torque and is normally regulated to zero; not the torque-producing quadrature current.
ω_e	**Electrical Angular Speed**	rad/s	Rotor speed times the pole-pair count; the electrical frequency of the drive, not the mechanical shaft speed.
θ_e	**Electrical Rotor Angle**	rad	Mechanical rotor angle times the pole-pair count, the angle a field-oriented drive uses to rotate between stator and rotor frames.
θ_{enc}	**Encoder Angle**	rad	Joint angle an encoder reports; a sample index appears as a subscript, as in θ_{enc_k}.
FK	**Forward Kinematics Map**	Function	Map from joint positions to the positions of points on the robot's links in the base frame.
n	**Generalized Coordinate Count**	Dimensionless	Number of generalized coordinates of a mechanism, its degrees of freedom (the joints of a serial arm); the plant state has twice this many entries.
θ_{head}	**Heading Angle**	rad	Heading of a planar mobile base in the unicycle model, the direction its forward speed points.
v_{conv}	**Conveyor Speed**	m/s	Speed of the belt that carries a workpiece past the pick station; not the bound on a takeaway conveyor, v_{conv}.
ϕ_{drive}	**Driven-Wheel Load Fraction**	Dimensionless	Fraction of a vehicle's total weight carried by its driven wheels; it scales the traction ceiling on braking deceleration.
q	**Joint Coordinate**	rad or m	One entry of the generalized coordinates $\mathbf{q}$, a single joint's angle or extension.
q	**Joint Position**	rad or m	Position of a single joint along its axis, an angle for a revolute joint and a length for a prismatic one; its time derivatives are $\dot{q}$ and $\ddot{q}$.
θ_{joint}	**Joint Angle**	rad	Angle of one revolute joint, a scalar; the joint-space vector is $\mathbf{q}$.
$\dot{q}$	**Joint Velocity**	rad/s or m/s	Velocity of one joint, a scalar; the vector of all joints is $\dot{\mathbf{q}}$.
ω	**Joint Angular Speed**	rad/s	Angular speed of a single arm joint; a subscript zero marks the initial value.
$\ddot{q}$	**Joint Acceleration**	rad/s^2 or m/s^2	Acceleration of one joint, a scalar; the vector of all joints is $\ddot{\mathbf{q}}$.
j_{joint}	**Joint Jerk**	rad/s^3	Jerk of one joint's trajectory, the third time derivative of its position; a scalar for one joint.

Symbol	Definition	Unit	Notes
n	**Joint Count**	Dimensionless	Number of joints of a manipulator, the length of its joint-position vector; it shares its glyph with the zero-failure trial count of the validation chapters.
i	**Joint Index**	Index	Position of one joint axis of a manipulator, from one to the number of joints.
τ_{motor}	**Motor Torque**	$\mathrm{N \cdot m}$	Torque a motor produces at its shaft before the transmission, the torque constant times the torque-producing current; the joint output is τ_{out}.
τ_{out}	**Joint Output Torque**	$\mathrm{N \cdot m}$	Torque a joint delivers at the transmission output: the transmission efficiency η_{gear} times the gear ratio N_{gear} times the motor torque τ_{motor}.
τ_{fric}	**Joint Friction Torque**	$\mathrm{N \cdot m}$	Total friction torque of a joint, referred to its output link.
$\tau_{\max}$	**Joint Torque Limit**	$\mathrm{N \cdot m}$	Largest torque the actuator of one joint can deliver, indexed by joint; it bounds τ at the same joint and is not the transient burst rating.
$\boldsymbol{\tau}_{\max}$	**Joint Torque Upper Limit**	$\mathrm{N \cdot m}$	Per-joint upper bounds on the torque the actuators can deliver, as a vector; the vector counterpart of the scalar $\tau_{\max}$.
$\dot{\boldsymbol{\tau}}$	**Joint Torque Slew**	$\mathrm{N \cdot m/s}$	Time derivative of the joint torque vector; it couples to jerk through the derivative of the manipulator equation.
$\dot{\mathbf{q}}_{\max}$	**Joint Velocity Limit**	rad/s or m/s	Per-joint bounds on the magnitude of the generalized velocities; a planned trajectory and every admitted command must stay within them.
$\mathbf{e}$	**Joint Tracking Error**	rad or m	Vector of differences between the desired and the measured joint positions; not the state estimation error, which shares the glyph.
r_{arm}	**Lever Arm Length**	m	Distance from a rotating joint to the point it carries, which converts a joint rotation into tool travel; a scalar length.
σ	**Jacobian Singular Value**	Mixed	Singular value of the manipulator Jacobian $\mathbf{J}$, the tool speed a unit joint rate produces along one singular direction; units follow the Jacobian rows.
λ	**Inverse Kinematics Damping**	Mixed	Damping length that damped-least-squares inverse kinematics adds to bound the joint step near a singularity; it enters squared, and along a singular direction the gain never exceeds one over twice its value. The ridge-regression analogue of weight decay.
λ_{pm}	**Permanent-Magnet Flux Linkage**	$\mathrm{V \cdot s}$	Magnetic flux from the rotor magnets that links each stator winding; with the pole-pair count it sets the torque constant.
$\mathbf{f}$	**Plant Dynamics Function**	Function	Vector field of a plant's equations of motion, giving the state rate from the state, the control input, and the plant parameters; not the unforced drift part of control-affine dynamics, which shares its glyph.
m	**Plant Parameters**	Record	Physical and hardware properties of the machine in one trial or rollout, such as tolerances, wear, payload mass, state of charge, and calibration; not the mass m of a body.
$\dot{\mathbf{x}}$	**Plant State Rate**	Dimensionless per unit time	Time derivative of the mechanical plant state; the decorated form of the plant state, and not the Cartesian velocity of the tool, which shares its glyph.
$\mathcal{X}$	**Plant State Space**	Set	Set of all states the mechanical plant can occupy, positions together with velocities.
ω_m	**Rotor Angular Speed**	rad/s	Angular speed of a motor rotor about its axis; it sets the back-EMF, and on a direct-drive hub motor it equals the wheel's angular speed.
N_{poles}	**Rotor Pole-Pair Count**	Dimensionless	Number of magnetic pole pairs on a motor rotor; multiplying the mechanical rotor speed by it gives the electrical frequency of the drive.
v_q	**Quadrature-Axis Voltage**	V	Voltage the inverter of a field-oriented drive applies along the quadrature axis, the axis that produces torque.
θ_{tilt}	**Tilt Angle**	rad	Angular deflection of a body from vertical; zero when the body is upright.

Symbol	Definition	Unit	Notes
$\dot{\tau}_{\max}$	**Torque Rate Limit**	$\mathrm{N}\cdot\mathrm{m/s}$	Largest rate of change of joint output torque a drive accepts. Divided by the effective joint inertia it bounds joint jerk.
K_{joint}	**Transmission Torsional Stiffness**	$\mathrm{N}\cdot\mathrm{m/rad}$	Torsional stiffness of the transmission between rotor and output link, the compliance that couples motor to load.
ω_z	**Yaw Rate**	rad/s	Angular velocity of a body about its vertical axis, one component of the body angular velocity ω.

Contact Mechanics and Centroidal Dynamics

Contact turns a free-moving body into a constrained one. The symbols here describe the forces a contact can transmit, the friction cone that bounds them, and the centroidal quantities that sum them over the whole robot.

Symbol	Definition	Unit	Notes
$\mathbf{S} \in \mathbb{R}^{n\times(n+6)}$	**Actuation Selection Matrix**	Matrix	Matrix $[\mathbf{0}, \mathbf{I}]$ projecting generalized forces onto actuated joints in floating-base systems.
$\mathbf{f}_k \in \mathbb{R}^3$	**Contact Force Vector**	N	Physical contact force exerted by environment on contact patch k ($\mathbf{f}_k = [\mathbf{f}_{k,\parallel}^\top, f_{k,\perp}]^\top$); always indexed by the contact.
f_n	**Finger normal force**	N	Normal force one finger of a grasp presses against the object at its contact.
$\mathbf{f}_t$	**Finger tangential force**	N	Tangential friction force one finger of a grasp carries at its contact, bounded by the friction cone times f_n.
$\mathbf{J}_{\mathbf{c}_k}(\mathbf{q})$	**Contact Point Jacobian**	Mixed	Mapping joint velocities to contact patch Cartesian linear velocity: $\dot{\mathbf{p}}_k = \mathbf{J}_{c_k}(\mathbf{q})\dot{\mathbf{q}}$.
$f_{k,\perp}$	**Normal Contact Force**	N	Unilateral compressive force ($f_{k,\perp} = \mathbf{f}_k^\top \hat{\mathbf{n}}_k \geq 0$) perpendicular to contact plane.
$\mathbf{f}_{k,\parallel}$	**Tangential Contact Shear**	N	Friction traction force tangential to contact plane ($\mathbf{f}_{k,\parallel} = \mathbf{f}_k - f_{k,\perp}\hat{\mathbf{n}}_k$).
μ	**Coulomb Friction Coefficient**	Dimensionless	Static/kinetic coefficient bounding friction cone: $\|\mathbf{f}_{k,\parallel}\| \leq \mu f_{k,\perp}$.
$\mathbf{c} \in \mathbb{R}^3$	**Center of Mass (CoM)**	m	Spatial coordinates of the total articulated body mass centroid ($\mathbf{c} = \frac{1}{m}\sum m_i \mathbf{p}_i$).
$\mathbf{h}_{\text{lin}}$	**Centroidal Linear Momentum**	$\mathrm{kg}\cdot\mathrm{m/s}$	Total plant linear momentum ($\mathbf{h}_{\text{lin}} = m\dot{\mathbf{c}}$), governed by $\dot{\mathbf{h}}_{\text{lin}} = \sum \mathbf{f}_k + m\mathbf{g}_0$, with gravitational acceleration $\mathbf{g}_0$.
$\mathbf{h}_{\text{ang}}$	**Centroidal Angular Momentum**	$\mathrm{kg}\cdot\mathrm{m}^2/\mathrm{s}$	Angular momentum about CoM: $\dot{\mathbf{h}}_{\text{ang}} = \sum(\mathbf{p}_k - \mathbf{c}) \times \mathbf{f}_k + \tau_k$.
$\mathbf{p}_{\text{ZMP}} \in \mathbb{R}^2$	**Zero Moment Point (ZMP)**	m	Ground-plane point where net horizontal tipping moment vanishes; must reside in $\mathrm{Conv}(\{\mathbf{p}_k\})$.
$e \in [0,1]$	**Coefficient of Restitution**	Dimensionless	Kinematic velocity rebound ratio during non-smooth impact ($\dot{q}^+ = -e\dot{q}^-$).
k_{contact}, k_H	**Contact Stiffness**	N/m	Elastic stiffness of a structural contact, and of the human body region H in an impact (with its effective mass m_H), in the form ISO/TS 15066 states.
m_{eff}	**Effective Impact Mass**	kg	Apparent normal mass at a specified contact point and configuration. For a single frictionless normal impact, it is the reciprocal of the positive scalar Delassus coefficient formed from the contact normal Jacobian and generalized inertia matrix (section B.7).
m_{red}	**Reduced Impact Mass**	kg	Two-body reduced mass of a collision, $m_{\text{red}} = (1/m_{\text{eff}} + 1/m_H)^{-1}$, which sets the transferred energy.
F_{unbraked}	**Unbraked Impact Force**	N	Transient collision force peak ($F_{\text{unbraked}} = v_{\max}\sqrt{k_H m_{\text{red}}}$) governed by ISO/TS 15066.
a_{arm}	**Arm Stopping Deceleration**	$\mathrm{m/s}^2$	Deceleration the arm must achieve after braking onset to stop within the latch force limit.

Symbol	Definition	Unit	Notes
m_H	**Body Region Mass**	kg	Effective mass of a human body region, here the hand, in the ISO/TS 15066 body model.
F_{cmd}	**Brake Command Contact Force**	N	Contact force against the latch at the instant the brake command issues, after the contact loop responds; distinct from a commanded traction force.
F_{cmd}	**Braking Onset Contact Force**	N	Contact force reached against the latch plate at physical braking onset.
$\dot{\mathbf{h}}_{\text{ang}}$	**Centroidal Angular Momentum Derivative**	$\mathrm{N \cdot m}$	Time derivative of the centroidal angular momentum, the net external moment about the center of mass: contact-force moments plus the contact torques.
$\dot{\mathbf{h}}_{\text{lin}}$	**Centroidal Linear Momentum Derivative**	N	Time derivative of the centroidal linear momentum, the net external force on the robot: the sum of contact forces plus mass times gravity.
v_{approach}	**Contact Approach Speed**	m/s	Speed at which the robot approaches and strikes a contact surface; contact force rises at stiffness times this speed.
$\dot{x}$	**Contact Direction Velocity**	m/s	Velocity of the robot's contact point along the contact direction.
F_{contact}	**Contact Force**	N	Force between the robot and the environment at a contact, along the contact direction; the per-point vector is $\mathbf{f}_k$.
T_c	**Contact Loop Response Time**	s	Time the contact-control loop takes to respond after its tripwire fires.
$\hat{\mathbf{n}}$	**Contact Normal**	Dimensionless	Unit vector normal to the contact plane at one contact, pointing into the robot; indexed by contact.
x	**Contact Normal Position**	m	Position of a tool or link along the contact normal, whose change from first contact is the contact penetration in the spring contact model.
i	**Contact Point Index**	Index	Position of one contact point among the contacts of a grasp.
$\mathbf{r}$	**Contact Point Offset**	m	Position of a grasp contact relative to the reference point of the grasped object, the moment arm of its force; indexed by contact.
x	**Contact Point Position**	m	Position of the robot's contact point along the contact direction.
τ	**Contact Point Torque**	$\mathrm{N \cdot m}$	Moment the environment exerts on the robot at one contact patch, the torque part of the contact wrench; indexed by contact. Not the joint torque vector τ, which shares the glyph.
F_{trip}	**Contact Tripwire Force**	N	Contact force at which the permission path ends an attempt on the machine, set below the structural damage limit.
$\mathbf{w}$	**Contact Wrench**	Varies	Force and moment the environment applies at a contact, stacked into one vector; force rows in newtons, moment rows in newton-meters. Privileged in simulation.
$\mathbf{w}_{\max}$	**Contact Wrench Ceiling**	Varies	Largest contact force and torque an intent lease permits at the tool, stacked as a wrench; units mix newtons and newton-meters.
x_{des}	**Desired Contact Position**	m	Setpoint position of the robot's contact point along the contact direction.
$\dot{x}_{\text{des}}$	**Desired Contact Velocity**	m/s	Setpoint velocity of the robot's contact point along the contact direction.
k_{eff}	**Effective Series Stiffness**	N/m	Stiffness of a contact whose two sides deflect in series, the reciprocal of the sum of their compliances; the softer side dominates.
$\mathbf{F}_{\text{ext}}$	**External Wrench**	Mixed	Force and moment the environment applies at the end effector, in newtons for the force and newton-meters for the moments; it maps into joint torques through the transposed manipulator Jacobian.
τ_{ext}	**External Contact Torque Vector**	$\mathrm{N \cdot m}$	Joint torques produced by external contact forces acting on the manipulator.
τ_{ext}	**External Joint Torque**	$\mathrm{N \cdot m}$	Torque the environment applies at one joint's output; the scalar counterpart of the external contact torque vector τ_{ext}.

Symbol	Definition	Unit	Notes
$\mu_{\min}$	**Friction Coefficient Floor**	Dimensionless	Lowest friction coefficient a braking or contact premise assumes the surface provides; an unobserved oil spill can invalidate it.
$f_{\max}$	**Grasp Force Ceiling**	N	Largest grasp contact force the object tolerates without crushing or deforming.
$f_{\min}$	**Grasp Force Floor**	N	Smallest grasp contact force that holds the object against slip; the grasp must stay between this floor and $f_{\max}$.
$\mathbf{G}$	**Grasp Map**	Matrix	Linear map from contact forces to the wrench they exert on the grasped object; indexed by contact when one contact is meant; not the Kalman gain.
$h_{\mathbf{cg}}$	**Center-of-Gravity Height**	m	Height of a vehicle's center of gravity above the floor; it sets the tip-over ceiling on deceleration.
$\mathbf{g}_0$	**Gravity Vector**	$\mathrm{m/s^2}$	Gravitational acceleration as a vector pointing down with the magnitude of surface gravity; the scalar is the gravitational acceleration, and it is not the vector of gravitational joint torques.
x	**Impedance-Controlled Position**	m	Measured position of the point an impedance controller regulates; its deviation from x_{des}, times the stiffness, is the spring term.
$\dot{x}$	**Impedance-Controlled Velocity**	m/s	Measured velocity of the regulated point, the time derivative of x.
$x_{\mathbf{des}}$	**Impedance Equilibrium Position**	m	Virtual rest point of the emulated spring that an impedance controller pulls the controlled point toward.
$\dot{x}_{\mathbf{des}}$	**Impedance Equilibrium Velocity**	m/s	Velocity of the virtual equilibrium; its difference from $\dot{x}$, times the damping, is the damper term.
E_k	**Kinetic Energy**	J	Energy a moving body carries, one half its mass times its speed squared (plus rotational terms where stated); not the standard's limit on energy transferred to a person.
z_c	**Center-of-Mass Height**	m	Height of a body's center of mass above its support or contact point; it sets the natural frequency of the linear inverted pendulum, $\sqrt{g/z_c}$, and is not a camera-frame coordinate.
δ	**Penetration Depth**	m	Depth by which one body penetrates or compresses another at a contact, measured along the contact normal; penalty and compliant contact models compute normal force from it. Shares its glyph with the Huber threshold of the training chapters.
$\dot{\delta}$	**Penetration Rate**	m/s	Time rate of change of the penetration depth δ at a contact; zero at the instant of maximum compression.
$v_{\mathbf{latch}}$	**Latch Approach Speed**	m/s	Speed at which the arm presses into the cage-door latch plate; the enforcement and release chapters use the same latch scenario.
$F_{\mathbf{lim}}$	**Latch Force Limit**	N	Structural force limit of the door latch the arm pushes against.
$k_{\mathbf{latch}}$	**Latch Plate Stiffness**	N/m	Spring stiffness of the cage-door latch plate the arm compresses.
$d_{\mathbf{lead}}$	**Leading Support Distance**	m	Horizontal distance from a vehicle's center of gravity forward to its leading ground support; it shrinks when an arm extends forward.
C	**Intrusion Distance**	m	Distance a body part can intrude past the sensing field before detection; a term of the ISO/TS 15066 protective separation distance.
S_h	**Person Approach Travel**	m	Distance a person travels toward the robot during the robot's reaction and stopping time; a term of the ISO/TS 15066 protective separation distance.
Z_d	**Person Position Uncertainty**	m	Position uncertainty of the person from the sensing system; a term of the ISO/TS 15066 protective separation distance.

Symbol	Definition	Unit	Notes
d_{budget}	**Known-Component Stop Distance**	m	Stopping travel that the known components of the stopping budget add up to at a given speed on a dry floor: delay travel, braking, jerk ramps, and fixed overheads; the clear distance minus it is the spare the stopping claim holds in reserve.
v_n^-	**Pre-Impact Normal Velocity**	m/s	Normal contact velocity just before an impact, negative while the bodies approach; the restitution law links it to the post-impact velocity.
v_n^+	**Post-Impact Normal Velocity**	m/s	Normal contact velocity just after an impact, equal to minus the restitution coefficient times the pre-impact normal velocity.
k_{robot}	**Robot Structural Stiffness**	N/m	Stiffness of the manipulator side of a contact, its pads, linkages, and joint elasticity acting as one series spring; it combines in series with the stiffness of the struck body.
a_{slip}	**Slip Acceleration Bound**	m/s^2	Bound on the residual acceleration of a tracked object slipping relative to its carrier; it widens the evidence horizon of a target estimate.

Geometry, Transformations, and Perception

Every measurement a robot uses arrives in some frame, and every command leaves in another. These symbols name the frames, the rigid transforms between them, and the camera models that project the world onto a sensor.

Symbol	Definition	Unit	Notes
$SE(3)$	**Special Euclidean Group**	Lie Group	Group of 3D rigid-body spatial transformations (translations and rotations: $\mathbb{R}^3 \rtimes SO(3)$).
$SO(3)$	**Special Orthogonal Group**	Lie Group	Group of 3×3 proper orthogonal rotation matrices ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = +1$).
$\mathcal{F}_W, \mathcal{F}_B, \mathcal{F}_E$	**Coordinate Reference Frames**	Reference	Fixed World frame ($\mathcal{F}_W$), mobile robot Base/body frame ($\mathcal{F}_B$), and End-effector frame ($\mathcal{F}_E$).
$\mathbf{p} \in \mathbb{R}^3$	**Cartesian 3D Position**	m	Position vector $[x, y, z]^\top$ expressing point coordinates in a stated frame.
$\mathbf{t} \in \mathbb{R}^3$	**Rigid-Transform Translation**	m	Translation part of a rigid transform between two frames (the offset of one frame's origin in the other).
$\mathbf{R} \in SO(3)$	**Directional Cosine Matrix**	Matrix	3×3 orthonormal matrix parameterizing spatial attitude and orientation.
$\mathbf{q}_{\text{quat}} \in \mathbb{H}$	**Unit Quaternion**	Dimensionless	Hypercomplex orientation vector $[w, x, y, z]^\top$ satisfying $\|\mathbf{q}_{\text{quat}}\| = 1$.
$\mathbf{T} \in SE(3)$	**Pose**	Matrix	Rigid-body pose of a named frame, such as a grasp, the tool center point, or a target ($\mathbf{T}_{\text{grasp}}$, $\mathbf{T}_{\text{TCP}}$); the transform between two frames is $\mathbf{T}_{AB}$ below.
$\mathbf{T}_{AB} \in SE(3)$	**Homogeneous Transform**	Matrix	4×4 matrix $\begin{bmatrix} \mathbf{R}_{AB} & \mathbf{t}_{AB} \\ \mathbf{0}^\top & 1 \end{bmatrix}$ mapping coordinates from frame B into frame A.
$\omega, \alpha \in \mathbb{R}^3$	**Angular Velocity & Acceleration**	$\text{rad/s}, \text{rad/s}^2$	Spatial angular velocity and angular acceleration vectors of a rigid body.
$\mathbf{r}_{P/O} \in \mathbb{R}^3$	**Sensor Lever-Arm Vector**	m	Displacement vector from body rotation origin O to displaced sensor location P; its length, or a link's lever arm, is the scalar r_{arm}.
$\mathbf{F}, \mathbf{E}$	**Fundamental and Essential Matrices**	Matrix	Two-view epipolar constraint $\tilde{\mathbf{m}}_2^\top \mathbf{F} \tilde{\mathbf{m}}_1 = 0$ with $\mathbf{E} = [\mathbf{t}]_\times \mathbf{R}$; the subscript keeps it distinct from the transition matrix $\mathbf{F}$.

Symbol	Definition	Unit	Notes
$\mathbf{K} \in \mathbb{R}^{3\times3}$	**Pinhole Camera Intrinsics**	Pixels	Calibration matrix $\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$ parameterizing focal lengths and principal center.
$\mathbf{K}^{-\top}$	**Inverse-transpose intrinsic matrix**	Matrix	Transpose of the inverse of the camera intrinsic matrix $\mathbf{K}$, which carries an essential matrix into the pixel-coordinate fundamental matrix.
$\mathbf{p}_c = [X_c, Y_c, Z_c]^\top$	**Camera-Frame 3D Coordinate**	m	Metric coordinates of a spatial point in the optical camera frame (Z_c is optical depth).
$[u, v]^\top$, $\tilde{\mathbf{m}}$	**Image Plane Pixel Coordinates**	Pixels	Discrete sensor pixel coordinates ($u \sim f_x \frac{X_c}{Z_c} + c_x$, $v \sim f_y \frac{Y_c}{Z_c} + c_y$), italic as scalars; the homogeneous image point is $\tilde{\mathbf{m}} = [u, v, 1]^\top$, never bold $\mathbf{u}$.
t_{exp}	**Exposure Duration**	ms	Integration time of one camera frame; the frame's timestamp belongs at its midpoint, $t_{\text{exp}}/2$ after exposure starts.
T_{inv}	**Belief Invalidation Horizon**	ms	Time since the last supporting evidence after which a belief can no longer be shown to lie inside its task tolerance; a belief expires at $t_0 + T_{\text{inv}}$.
d_{free}	**Free-Space Depth**	m	Distance from a hull's surface to the nearest cell that is occupied, unknown, or expired, read from an ESDF built with all three treated as obstacles; the budget motion may spend there (section 9.6.1).
t_{row}	**Rolling Shutter Row Readout**	µs	Exposure time delta between adjacent CMOS sensor scanlines; row y starts at $t(y) = t_{\text{start}} + y\, t_{\text{row}}$.
$\Delta\mathbf{x}(y)$	**Rolling Shutter Distortion**	m	Kinematic skew displacement $\Delta\mathbf{x} = \mathbf{v} \cdot y \cdot t_{\text{row}}$ induced by high-velocity platform motion.
$\mathbf{tsdf}(\mathbf{p}_{\text{vox}})$	**Truncated Signed Distance**	$[-1, +1]$	Normalized volumetric distance $\text{tsdf}(\mathbf{p}_{\text{vox}}) = \text{clamp}(d(\mathbf{p}_{\text{vox}})/\mu_{\text{tsdf}}, -1, 1)$ of the voxel centered at $\mathbf{p}_{\text{vox}}$ to the nearest surface.
μ_{tsdf}	**TSDF Truncation Band**	m	Metric thickness of the spatial surface interface band stored in voxel memory.
$D_t(\mathbf{p}_{\text{vox}}), W_t(\mathbf{p}_{\text{vox}})$	**TSDF Accumulated Volume**	Mixed	Running weighted signed distance and accumulated visibility weight at the voxel centered at $\mathbf{p}_{\text{vox}}$.
$e_0, e_{\max}$	**Initial and Ceiling Error**	m	Position error of an estimate when its evidence is fresh, and the largest error the task tolerates; E_0 and $E_{\max}$ stay energies.
r_{tol}	**Positional Task Tolerance**	m	Radius within which a target estimate must lie for the task to succeed; paired with the orientation tolerance θ_{tol}.
$\mathcal{G}_{\text{goal}}$	**Geometric Goal Set**	Subset of $SE(3)$	Admissible terminal poses within the positional and angular tolerances of the task; the spatial component of an intent lease $\mathcal{I}_{\text{intent}}$.
d	**Distance to Target**	m	Straight-line distance from the measured end-effector position to the admitted target, $\|\mathbf{p}_{\text{target}} - \mathbf{p}_0\|$, in the intent-lease reachability test; distinct from the hidden width d.
θ_{att}	**Attitude Error**	rad	Angular error in an estimated attitude, which leaks gravity into the horizontal acceleration.
θ_{az}	**Azimuth Angle**	rad	Angle of a direction measured in the horizontal plane about the vertical axis; the book may state values in degrees.
$\mathbf{T}^{\text{cam}}_{\text{base}}$	**Base-to-Camera Transform**	Matrix	Rigid transform mapping camera coordinates into the robot base frame.
$\mathbf{p}_{\text{body}}$	**Body-Frame Point**	m	Position of a perceived point expressed in the robot body frame, the camera-frame point carried through the extrinsics.
Σ_{body}	**Body-Frame Point Covariance**	m^2	Covariance of a perceived point's position error in the robot body frame, the spatial covariance an observation contract carries; rotated camera-frame covariance plus the extrinsic term, not the covariance of a state estimator.

Symbol	Definition	Unit	Notes
$\mathbf{R}_{bc}$	**Camera Extrinsic Rotation**	Dimensionless	Rotation part of a camera's extrinsic calibration, taking camera-frame vectors into the body frame; the rotation block of $\mathbf{T}_{bc}$.
$\mathbf{T}_{bc}$	**Camera Extrinsic Transform**	Matrix	Rigid transform of a camera's extrinsic calibration, mapping camera-frame coordinates into the body frame; mixed rotation and translation blocks.
Σ_c	**Camera-Frame Point Covariance**	m^2	Covariance of a point's position error in the camera's optical frame, after unprojection of its pixel and depth measurement; rotated into the body frame by $\mathbf{R}_{bc}$.
x	**Cartesian X Coordinate**	m	First Cartesian component of a three-dimensional position.
y	**Cartesian Y Coordinate**	m	Second Cartesian component of a three-dimensional position.
z	**Cartesian Z Coordinate**	m	Third Cartesian component of a three-dimensional position.
C_{inset}	**Clearance Inset**	m	Separation a consumer holds between an estimated obstacle position and the machine's envelope so the true obstacle stays the required clearance away, except with a stated tail probability; a field of the observation contract.
σ_z	**Depth Standard Deviation**	m	Standard deviation of the depth a stereo or range sensor reports for one point; for stereo it grows with the square of the range.
σ_d	**Disparity Standard Deviation**	pixels	Standard deviation of the disparity a stereo matcher measures; it feeds the stereo depth term σ_z.
ϕ	**Elevation Angle**	rad	Angle of a direction above the horizontal plane; not the encoder parameters of a variational autoencoder, which render the same glyph.
Σ_{ext}	**Extrinsic Calibration Covariance**	m^2	Covariance of a sensor's calibrated mounting extrinsic.
θ_{ext}	**Extrinsic Misalignment Angle**	rad	Angular error of a sensor's extrinsic calibration relative to its true mounting.
f	**Focal Length**	pixels	Focal length of a pinhole camera in pixels, the scale from normalized image coordinates to pixels; a rectified pair shares one value.
f_x	**Horizontal Focal Length**	pixels	Camera focal length in pixels along the horizontal image axis, one entry of the intrinsics.
t_{start}	**Frame Exposure Start Time**	s	Instant at which the first row of a rolling-shutter frame begins its exposure.
t_{readout}	**Frame Readout Time**	s	Time a camera takes to read its pixels out after exposure.
t_{capture}	**Evidence Capture Instant**	s	Instant, on a monotonic clock, at which the sensor evidence behind an observation or proposal was captured; observation age and proposal validity are measured from it.
t_0	**Evidence Epoch**	s	Capture instant of the most recent measurement that constrained a stored belief; republishing or predicting never advances it.
t_{fresh}	**Fresh Evidence Time**	s	Capture time of the fresh physical evidence that renews a belief's evidence epoch t_0.
t_{now}	**Current Time**	s	Wall-clock time at which a consumer evaluates a stored record; evidence age is the current time minus t_0.
t_{age}	**Dispatch Observation Age**	s	Elapsed time from the sensor exposure midpoint to dispatch of the processed observation. Current consumer evidence age additionally includes elapsed time after dispatch.
$\mathcal{C}_{\text{free}}$	**Free Configuration Space**	Set	Configurations at which the robot touches no obstacle, the configuration space minus the obstacle set.
ρ	**Inverse Depth**	$1/\mathrm{m}$	Reciprocal of a point's depth along the optical axis, the parameterization that lets a landmark at infinity enter an estimator at zero; linear in disparity.
d	**Obstacle Separation Distance**	m	Distance between the machine and the nearest obstacle or boundary at a given time, the signal a clearance predicate scores; written as a function of time.

Symbol	Definition	Unit	Notes
L	**Occupancy Log-Odds**	Dimensionless	Natural logarithm of the odds of a voxel's occupancy probability, the quantity an occupancy map stores per cell; indexed by step, with the prior at step zero.
$\mathbf{p}_{\text{meas}}$	**Measured Tracked Position**	m	Measured Cartesian position of the tracked point, which the tracking error compares against the reference.
t_{last}	**Last Measurement Time**	s	Capture time of the most recent measurement that updated a track or map cell.
$\sigma_{v,0}$	**Initial Velocity Standard Deviation**	m/s	Standard deviation of the velocity error of an estimate, here at the onset of occlusion.
σ_u	**Pixel Standard Deviation**	Pixels	Standard deviation of a measured image position, in pixels; the image-plane noise that propagates into depth and position error.
c_x	**Principal Point Column**	Pixels	Horizontal pixel coordinate of the principal point, where the optical axis meets the image.
c_y	**Principal Point Row**	Pixels	Vertical pixel coordinate of the principal point, where the optical axis meets the image.
b	**Stereo Baseline**	m	Distance between the optical centers of a stereo camera pair; depth is focal length times baseline over disparity.
d	**Stereo Disparity**	Pixels	Horizontal pixel offset between the images of one scene point in the two views of a rectified stereo pair; not a travel distance or a sightline range, which share the glyph.
z	**Target Range**	m	Distance from a camera to the target it observes, measured along the viewing direction; stereo depth error grows with its square.
$\mathbf{p}_{\text{target}}$	**Target Position**	m	Position of the target an end effector is commanded toward, in the robot base frame; a decorated form of the Cartesian position.
E	**Position Error Bound**	m	Worst-case position error of a target estimate as a function of the time elapsed since its evidence was captured; it grows while the target is occluded.
e	**Position Error Envelope**	m	Conservative bound on the position error of a target estimate along one coordinate axis of the base frame; written with the axis index.
z	**Range Observation**	Record	One range observation a sensor returns at a time step; indexed by step, with the history as the sequence of them.
$\mathcal{M}$	**Spatial Memory Map**	Set	Occupancy map in spatial memory that stores the obstacle geometry around the machine; not a decision-process tuple.
$\mathbf{R}_{AB}$	**Transform Rotation Block**	Matrix	Rotation block of the homogeneous transform from frame B to frame A, its upper-left three-by-three part.
$\mathbf{t}_{AB}$	**Transform Translation Block**	m	Translation block of the homogeneous transform from frame B to frame A, the position of the origin of frame B expressed in frame A.
$\mathbf{J}_{\text{unproj}}$	**Unprojection Jacobian**	Mixed	Jacobian of a camera-frame point with respect to its pixel coordinates and depth; it carries measurement covariance into Cartesian covariance. Not the manipulator Jacobian $\mathbf{J}$.
f_y	**Vertical Focal Length**	Pixels	Camera focal length expressed in pixels along the vertical image axis.
$\mathbf{T}_{wb}$	**World-from-Body Transform**	Matrix	Rigid transform giving the pose of the vehicle body frame in the world frame; it maps body coordinates into world coordinates.
$\mathbf{p}_{\text{world}}$	**World-Frame Point**	m	Position of an observed point expressed in the world frame; compare $\mathbf{p}_c$, the same point in the camera frame.
Σ_{depth}	**Perception Depth Covariance**	m^2	Covariance of the three-dimensional position error of a point lifted from stereo, lidar, or depth data; a matrix, not the scalar depth variance.
c	**Speed of Light in Vacuum**	m/s	About 3×10^8 m/s, the upper limit on how fast a signal crosses a chip or link; on-chip wires carry signals at a fraction of it.

Control, Observers, and Safety Filters

The control symbols follow state-space convention. A plant has a state, an input, and a measured output; an observer estimates the state from that output; and a safety filter, such as a control barrier function solved as a quadratic program, edits the input before it reaches the actuators.

Symbol	Definition	Unit	Notes
$\mathbf{A_{sys}}, \mathbf{B}$	**Linear State-Space Matrices**	Matrix	Continuous LTI plant: $\dot{\mathbf{x}} = \mathbf{A}_{\text{sys}}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{w}$, observed output $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{v}_{\text{meas}}$ through the measurement map $\mathbf{H}$ below.
$\mathbf{w}(t), \mathbf{v_{meas}}(t)$	**Process and Measurement Noise**	Vector	Zero-mean Gaussian noise vectors with covariance matrices $\mathbf{Q} \succeq 0$ and $\Sigma_{\text{meas}} \succ 0$; bold $\mathbf{R}$ is reserved for rotation.
$\mathbf{F}, \mathbf{H}$	**Transition and Measurement Maps**	Matrix	Discrete state transition $\hat{\mathbf{x}}_{k\|k-1} = \mathbf{F}\hat{\mathbf{x}}_{k-1\|k-1}$ and measurement map $\mathbf{y}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_{\text{meas}}$ of a Kalman filter.
$\mathbf{G}_k$	**Kalman Gain**	Matrix	Weight on the innovation $\nu_k = \mathbf{y}_k - \mathbf{H}\hat{\mathbf{x}}_{k\|k-1}$ in the update step; bold $\mathbf{K}$ is reserved for camera intrinsics.
$\mathbf{P} \in \mathbb{R}^{n \times n}$	**Estimation Error Covariance**	Matrix	Symmetric positive-definite matrix $\mathbb{E}[(\mathbf{x}-\hat{\mathbf{x}})(\mathbf{x}-\hat{\mathbf{x}})^\top]$ ($\mathbf{P}(t)$ continuous, $\mathbf{P}_{k\|k}$ discrete) evolving via matrix Riccati equation; $\mathbf{P}_{\text{est}}$ where an LQR cost matrix is also in scope.
$\mathbf{O_{obs}}, \mathbf{O_{NL}}$	**Observability Matrices**	Matrix	Kalman observability matrix $[\mathbf{H}^\top, (\mathbf{H}\mathbf{A}_{\text{sys}})^\top, \ldots]^\top$ and nonlinear Lie matrix $[d\mathbf{h}, d L_{\mathbf{f}}\mathbf{h}, \ldots]^\top$; calligraphic $\mathcal{O}(\cdot)$ stays asymptotic cost. The dual controllability matrix is $\mathbf{C}_{\text{ctrb}}$.
$\mathbf{C_{ctrb}}$	**Controllability Matrix**	Mixed	Kalman block matrix formed from the input matrix and its products with powers of the system matrix; full row rank characterizes unconstrained LTI controllability.
$\partial\mathcal{C}$	**Safety Boundary Manifold**	Boundary	Zero level set $\{\mathbf{x} \mid h(\mathbf{x}) = 0\}$, the boundary of the safe set $\mathcal{C}$, separating nominal operation from irreversible hazard.
$h(\mathbf{x}), h$	**Control Barrier Function (CBF)**	Scalar	Continuously differentiable function whose 0-superlevel set is the safe set $\mathcal{C}$. Written $h(\mathbf{x})$ on the plant state, and as the function head h when it takes other arguments or is differentiated ($\dot{h}$, $L_{\mathbf{f}}h$, $h(\theta_{\text{clear}}, \omega)$).
$\mathcal{C} \subset \mathbb{R}^{2n}$	**Forward-Invariant Safe Set**	Set	Superlevel set $\mathcal{C} = \{\mathbf{x} \mid h(\mathbf{x}) \geq 0\}$ of plant states where safety invariants are strictly maintained; a qualifier names a variant ($\mathcal{C}_{\text{contact}}$, $\mathcal{C}_{\text{discrete}}$).
$\mathbf{f}(\mathbf{x}), \mathbf{g}(\mathbf{x})$	**Control-Affine Drift and Input Fields**	Vector, Matrix	The control-affine model $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$ of barrier analysis; these letters hold only inside that model, where the gravity torque $\mathbf{g}(\mathbf{q})$ and contact forces $\mathbf{f}_k$ do not appear.
$L_{\mathbf{f}}h(\mathbf{x})$	**Lie Derivative along Drift**	Barrier units/s	Directional gradient derivative $\nabla h(\mathbf{x})\mathbf{f}(\mathbf{x})$ along unforced plant dynamics; its units are those of the barrier divided by time.
$L_{\mathbf{g}}h(\mathbf{x})$	**Lie Derivative along Control**	Vector	Directional control effectiveness gradient $\nabla h(\mathbf{x})\mathbf{g}(\mathbf{x})$.
r	**Barrier Relative Degree**	Integer	Number of differentiations required before control input $\mathbf{u}$ appears ($L_{\mathbf{g}}L_{\mathbf{f}}^{r-1}h \neq 0$).
$h_{\text{kinetic}}(\mathbf{x})$	**Kinetic Inset Barrier Function**	Scalar	Relative-degree-1 barrier: $h_{\text{kinetic}}(\mathbf{x}) = h(\mathbf{x}) - v^2/(2a_{\text{brake}})$ incorporating stopping distance.
$\kappa(\cdot), \kappa$	**Class-$\mathcal{K}$ Rate Function**	Scalar function	Extended class-$\mathcal{K}$ function in the barrier condition $\dot{h} \geq -\kappa(h)$, written as the function head κ when applied; the book uses the linear choice $\kappa(h) = \gamma_{\text{cbf}}h$.
γ_{cbf}	**Linear Barrier Decay Rate**	s^{-1}	Coefficient of the linear class-$\mathcal{K}$ choice, $\dot{h} + \gamma_{\text{cbf}}h \geq 0$; bare γ is the reinforcement-learning discount factor.
$\mathbf{u_{nom}} \in \mathcal{U}_{\text{adm}}$	**Nominal Control Command**	N·m / rad	Unverified control candidate proposed across the proposal boundary by the learned policy.
$\mathbf{u}^* \in \mathcal{U}_{\text{adm}}$	**Certified Control Action**	N·m / rad	Privileged, safety-filtered control command resolved by the real-time active-set CBF-QP.

DRAFT

Symbol	Definition	Unit	Notes
$\mathbf{r}$	**Reference Position**	m	Commanded Cartesian position reference the plant tracks, $\mathbf{r}(t)$; the feedback tracker bounds the tracked point's position error by $\|\mathbf{p}(t)-\mathbf{r}(t)\| \le \epsilon_{\text{track}}$, with both positions expressed in the same frame. Whole-body clearance requires a declared output map or metric that bounds swept-volume displacement. Elsewhere bold $\mathbf{r}$ is local (a moment arm, ellipsoid semi-axes).
$\alpha(\hat{t})$	**Bumpless Transfer Blend**	$[0,1]$	C^2 quintic polynomial $6\hat{t}^5-15\hat{t}^4+10\hat{t}^3$ of normalized time $\hat{t}\in[0,1]$, ramping between prior and candidate controllers.
α'	**Blend slope**	Dimensionless	Derivative of the bumpless-transfer blend weight α with respect to normalized progress.
τ_{blend}	**Blending Transition Window**	s	Duration of bumpless transfer; for two fixed, latched acceleration commands, bounded by $\tau_{\text{blend,min}} = 1.875\|\Delta\mathbf{u}\|/j_{\max}$. Torque or position commands require their own compatible command-rate bounds.
K_p, K_d, K_i	**PID / Feedback Gains**	Mixed	Proportional, derivative, and integral gains for joint-level or operational-space error stabilization.
$\mathbf{K}_{\text{lqr}}, \mathbf{Q}_x, \mathbf{R}_u$	**LQR Gain and Weights**	Matrix	State-feedback gain $\mathbf{u}^* = -\mathbf{K}_{\text{lqr}}\mathbf{x}$ and the state and control weights of the quadratic cost; the subscripts keep them distinct from intrinsics, process noise, and rotation.
N_{mpc}	**MPC Prediction Horizon**	Control steps	Number of predicted discrete control steps in the finite-horizon MPC formulation; distinct from physical-trial count and learned action-chunk horizon.
$\mathbf{D}_{\text{virt}}, \mathbf{K}_{\text{virt}}$	**Virtual Damping & Stiffness Matrices**	Matrix	Target compliance matrices in impedance control: $\tau_{\text{safe}} = -\mathbf{D}_{\text{virt}}\dot{\mathbf{q}} - \mathbf{K}_{\text{virt}}\Delta\mathbf{q} + \mathbf{g}(\mathbf{q})$.
q_c	**Acceleration Noise Spectral Density**	m^2/s^3	Continuous power spectral density of the zero-mean white acceleration noise per axis in a constant-velocity motion model; sets how fast the predicted position uncertainty grows.
$\mathcal{W}$	**Admissible Disturbance Set**	Set	Set of disturbance values a falsification search may inject; the search is bounded to it.
$\mathbf{u}$	**Admitted Control Command**	Per interface	Control command the permission path applies to the actuators after admitting, clipping, or replacing a policy proposal; indexed by step. Not $\mathbf{u}_{\text{nom}}$ (before the filter) or $\mathbf{u}^*$ (the filter's projection).
ω	**Angular Frequency**	rad/s	Frequency of a sinusoidal signal, the free variable of a frequency response; evaluated at the crossover frequency when a delay's phase lag is charged to the phase margin.
$\mathbf{A}_{\text{BE}}$	**Backward Euler Transition Matrix**	Matrix	Discrete-time transition matrix that backward (implicit) Euler integration produces, the inverse of the identity minus the system matrix times the step; unconditionally stable but numerically dissipative.
b_{cbf}	**Barrier Constraint Bound**	Varies	Bound on the right side of one affine control-barrier constraint, one entry of $\mathbf{b}_{\text{cbf}}$; its unit follows the barrier function rate.
$\mathbf{a}_{\text{cbf}}$	**Barrier Constraint Normal**	Varies	Normal vector of a single affine control-barrier constraint on the control input, equal to $-L_{\mathbf{g}}h$ transposed; one row of $\mathbf{A}_{\text{cbf}}$.
$\dot{h}$	**Barrier Function Rate**	m/s	Time derivative of the control barrier function along the closed-loop dynamics, the rate in the barrier condition; a dot is not an allowed decoration on h, so it has its own entry.
λ	**Barrier Projection Multiplier**	Varies	Lagrange multiplier of the barrier half-space constraint in the CBF-QP projection; starred at the optimum, where it is zero when the nominal command is already safe.
E	**Belief Error Bound**	Varies	Upper bound on the discrepancy between a stored belief and the true state after a given elapsed time without new evidence; the unit is that of the stored quantity (meters for a position, radians for an angle).

Symbol	Definition	Unit	Notes
t_{start}	**Bumpless Transfer Start Instant**	s	Instant at which a bumpless-transfer blend begins, the zero of its normalized progress; other chapters use the same letters for exposure or sweep starts, different quantities.
u	**Channel Command**	Varies	Scalar command of one actuator channel, such as the base acceleration or one joint torque; the unit is the channel's command unit. Scalar per-channel form of $\mathbf{u}$.
$\mathcal{K}$	**Class-K Function Set**	Set	Class of continuous strictly increasing functions that are zero at zero; named in "extended class-K function."
$\mathbf{u}$	**Control Command**	Per interface	Control command vector sent toward the actuators, whatever its source or stage; indices name instances. The stages are $\mathbf{u}_{\text{nom}}$ and $\mathbf{u}^*$.
$\mathbf{u}$	**Control Input**	Per interface	Control input applied to the plant, the command a controller or safety filter sends to the actuators; units depend on the command interface (torques, setpoints).
m	**Control Input Dimension**	Count	Number of components of the control input vector; not the mass m.
$\mathbf{u}_{\min}$	**Control Input Lower Limit**	Per interface	Per-component lower limit of the control input the actuators can deliver.
$\mathcal{U}$	**Control Input Set**	Set	Set of control inputs the controller may choose from, a subset of the input space; the actuator box and the barrier-feasible subset are narrower sets inside it.
$\mathbf{u}_{\max}$	**Control Input Upper Limit**	Vector	Per-component upper limit of the control input the actuators can deliver.
t	**Control Step Index**	Dimensionless	Position of one discrete control step, or setpoint, in a robot's control timeline; a subscript on per-step quantities, bare in expressions such as a clipped next step.
$\mathbf{K}_d$	**Derivative Gain Matrix**	Matrix	Derivative feedback gains acting on the tracking error rate in joint-space PD or PID control; its diagonal entries are the per-joint derivative gains.
C	**Differentiability Class**	Set	Functions whose derivatives up to the order written as the superscript exist and are continuous; C^2 means continuous position, velocity, and acceleration of a blended trajectory.
f	**Discrete Plant Dynamics**	Function	Discrete-time map from the current state and applied command to the next state; the deterministic counterpart of the plant's transition kernel.
$L_{\mathbf{f}}$	**Drift Lie Derivative Operator**	Operator	Rate of change of a scalar function of the state under the unforced dynamics; it is applied repeatedly for higher-order derivatives.
v_{drift}	**Drift Speed Bound**	m/s	Bound on the unobserved speed at which a target drifts relative to its last measured position; not the bound on the state-estimate drift, v_{drift}.
v_{drift}	**Drift Velocity Bound**	m/s	Bound on the unmodeled velocity at which an unobserved estimate drifts from the true state.
v_{conv}	**Conveyor Speed Bound**	m/s	Largest speed at which the takeaway conveyor carries a target; the bound used when sizing a catch window, not the belt speed v_{conv} of a single pick.
$\hat{\mathbf{x}}$	**Estimated Plant State**	Mixed	Estimate of the mechanical plant state reconstructed from sensor observations; its error covariance is a separate symbol.
$\hat{\mathbf{p}}$	**Estimated Position**	m	Position of a point that a filter propagates under its nominal model.
$\hat{\mathbf{v}}$	**Estimated Velocity**	m/s	Velocity of a body that a filter propagates under its nominal model.
T	**Finite Time Horizon**	s	Length of the finite window from time zero to the horizon over which an optimization, rollout, test, or temporal-logic operator is evaluated; not the wall-clock time to complete a unit of work.

Symbol	Definition	Unit	Notes
$\mathbf{A_{FE}}$	**Forward Euler Transition Matrix**	Matrix	Discrete-time transition matrix that forward Euler integration produces, identity plus the system matrix times the step; its spectral radius exceeds one for an undamped oscillator at any step size.
h	**Hermite Basis Function**	Function	Cubic Hermite basis polynomial that weights an endpoint position or velocity in spline interpolation; indexed by two digits.
j	**Imaginary Unit**	Dimensionless	Square root of minus one, written j by the engineering convention rather than i.
v	**Inductive Voltage**	V	Voltage across a coil, the inductance times the rate of change of its current.
g	**Inequality Constraint Function**	Varies	Constraint function of an optimization problem, whose nonpositive values define the feasible set; it appears in the interior-point barrier.
$L_{\mathbf{g}}$	**Input Lie Derivative Operator**	Operator	Lie derivative along the input matrix field, mapping a scalar function of the state to its sensitivity to the control input.
s	**Integration Time Instant**	s	Time instant running over an integration interval, the dummy variable when the interval ends at the time t; it keeps the integrand's time distinct from the upper limit.
t'	**Integration Dummy Time**	s	Dummy time variable of integration inside a time integral, distinct from its limits; a primed time would be a decorated quantity, so this entry names the bound variable.
s	**Laplace Variable**	$1/\mathrm{s}$	Complex frequency variable of the Laplace transform, standing for differentiation in time; read as the imaginary unit times the angular frequency for sinusoidal motion.
$\lambda_{\max}$	**Largest Eigenvalue**	Varies	Largest eigenvalue of a symmetric matrix, applied to that matrix; its unit is that of the matrix entries.
$\mathbf{P}$	**LQR Cost-to-Go Matrix**	Varies	Symmetric solution of the regulator algebraic Riccati equation (the discrete one in discrete-time LQR), whose quadratic form is the optimal LQR cost-to-go; shares its glyph with the estimation error covariance.
$\mathbf{u}_{0:N_{\mathbf{mpc}}-1}$	**MPC Control Sequence**	Varies	Sequence of control inputs over a model-predictive-control horizon, one per step, that the optimizer chooses; only the first is applied before the next re-solve.
$\mathbf{h}$	**Nonlinear Output Function**	Varies	Output function of a nonlinear state-space model, mapping the state to the measurement vector $\mathbf{y}$; not the barrier function nor the bias forces of the floating-base equations.
L	**Open-Loop Transfer Function**	Dimensionless	Open-loop transfer function of a feedback loop, evaluated along the imaginary axis for frequency response; never the loss.
$\mathbf{K}_p$	**PID Proportional Gain Matrix**	Varies	Proportional gain matrix of multi-joint PD or PID feedback, the stiffness acting on the position error.
$\mathbf{y}$	**Measurement Vector**	Varies	Measurement vector a sensor or detector reports at one step.
ν	**Measurement Innovation**	Varies	Difference between a measurement and the measurement the predicted state implies; its normalized square is the NIS.
$\Sigma_{\mathbf{meas}}$	**Measurement Noise Covariance**	Varies	Covariance of the noise on a sensor measurement vector, such as a pixel position with depth or a Kalman filter measurement; units are those of the measurement squared.
$\Delta t_{\mathbf{asym}}$	**Path Delay Asymmetry**	s	Difference between the forward and reverse one-way delays of a clock-synchronization exchange, which biases the estimated clock offset by half its value; two-way time transfer assumes it is zero.
$\hat{\mathbf{x}}_{k\mid k-1}$	**Predicted State Estimate**	Vector	State estimate at step k predicted from measurements through the previous step, before the measurement update; the step index is part of the symbol.
$\mathbf{P}_{k\mid k-1}$	**Predicted Error Covariance**	Matrix	Error covariance at step k predicted from measurements through the previous step, before the update; the conditioned form of the estimation error covariance.
$\hat{\mathbf{x}}_{k-1\mid k-1}$	**Previous Updated State Estimate**	Vector	State estimate at the previous step after that step's update, the input to the next prediction.

Symbol	Definition	Unit	Notes
$\mathbf{P}_{k-1\|k-1}$	**Previous Updated Error Covariance**	Matrix	Error covariance at the previous step after that step's update, the input to the next prediction.
$\mathbf{Q}$	**Process Noise Covariance**	Matrix	Covariance of the process noise vector; the predict step adds it to the error covariance.
$\hat{\mathbf{x}}$	**State Estimate**	Vector	Estimate of the state vector that a recursive estimator maintains; conditioned forms with the step index baked in have their own symbols.
$\mathbf{K}_p$	**Proportional Gain Matrix**	Matrix	Matrix of proportional feedback gains acting on the tracking error vector in joint-space PD or PID control; its diagonal entries are the per-joint proportional gains, and it is not the camera calibration matrix.
$\mathcal{U}_{\text{safe}}$	**Safe Control Set**	Set	Set of control inputs that satisfy every active barrier constraint and the actuator limits at the current state; it is empty when saturation leaves no command that keeps the barrier condition, which selects a fallback.
$\mathcal{X}_{\text{safe}}$	**Safe State Constraint Set**	Set	Set of plant states that satisfy the state constraints of a model predictive control problem, such as joint limits and obstacle margins; polyhedral, and not the barrier safe set.
d_{safe}	**Safe Separation Distance**	m	Smallest separation from an obstacle that a clearance requirement allows.
s	**Signal Trajectory**	Function	Trajectory of a system's real-valued signals over continuous time, the trace a temporal logic formula is evaluated on; two trajectories are told apart by subscripts.
ϕ	**Signal Temporal Logic Formula**	Record	Signal temporal logic formula a signal must satisfy, such as a safety invariant or a reachability deadline; subscripts index subformulas.
f	**Signal Temporal Logic Predicate Function**	Function	Real-valued function of a signal whose sign decides an atomic temporal-logic predicate; its unit is the physical unit of that predicate's margin, and it is indexed per predicate.
ρ	**Signal Temporal Logic Robustness**	Function	Signed margin by which a signal satisfies (positive) or violates (negative) a temporal logic formula at a time; it carries the units of the atomic predicates when they agree and is dimensionless after a declared normalization. Not a correlation coefficient.
δ	**Small Change**	Operator	Small change, increment or error in the quantity it prefixes; the companion of the finite change for first-order changes.
δ	**Small Perturbation**	Operator	Small change or error in the quantity it prefixes, treated to first order, as in a perturbation of a signal or a time; not a labeled localization bound that also renders δ.
a	**Polynomial Trajectory Coefficient**	Varies	Coefficient of one power of time in a polynomial joint trajectory segment, indexed by the power it multiplies; its unit is the joint unit divided by seconds to that power, and it is not an acceleration.
k	**Setpoint Sample Index**	Index	Position of one discrete setpoint sample in a sampled trajectory.
Δt_{step}	**Setpoint Step Interval**	s	Time between consecutive setpoints of an action chunk, the spacing at which the controller consumes them; the reciprocal of the action execution rate.
d_s	**State Space Dimension**	Dimensionless	Number of continuous coordinates in the task-relevant state.
$t_{\text{stop_end}}$	**Stop Completion Instant**	s	Instant, on the permission clock, at which the matched stop of an admitted trajectory record reaches rest; the commitment instant plus the stopping duration.
$t_{\text{deadline,stop}}$	**Stopping Deadline**	s	Latest instant at which authority can be revoked and a worst-case stop still ends within the verified clearance.
L_{lip}	**Policy Lipschitz Bound**	Varies	Lipschitz bound of a learned policy over a stated operating domain, the largest ratio of action change to state change; it gives no guarantee unless established for that domain, and times the sensor error bound it bounds the action error.
$\mathbf{r}$	**Task Tolerance Semi-Axes**	m	Principal semi-axes of the ellipsoid within which terminal position error must lie; the vector of r values.

DRAFT

Symbol	Definition	Unit	Notes
r	**Task Tolerance Semi-Axis**	m	Semi-axis of the task tolerance ellipsoid along one coordinate axis, written with the axis index.
$\mathcal{X}_f$	**Terminal Invariant Set**	Set	Plant states from which an invariant fallback control law keeps the system safe indefinitely, the required endpoint of a receding horizon. On an embodied system a validated stop that reaches rest within verified clearance realizes it.
$\mathbf{e}$	**Tracking Error Vector**	Vector	Commanded reference state minus measured state of a feedback controller, acted on by $\mathbf{K}_p$. Not the error feedback residual, which shares the glyph.
$\dot{\mathbf{e}}$	**Tracking Error Rate**	Vector	Time derivative of the tracking error vector, the velocity error of a feedback controller; multiplied by $\mathbf{K}_d$ in PD control, so a step in it produces a torque step.
δ	**Unit Impulse Function**	s^{-1}	Dirac delta, the idealized unit impulse of zero width and unit area; with time as its argument it carries inverse seconds.
$\mathbf{P}_{k\|k}$	**Updated Error Covariance**	Matrix	Error covariance at a step after incorporating that step's measurement, the posterior counterpart of $\mathbf{P}$.
$\hat{\mathbf{x}}_{k\|k}$	**Updated State Estimate**	Vector	State estimate at a step after incorporating that step's measurement, the posterior estimate of a Kalman filter.
d	**Derivative Operator**	Operator	Differential operator of an ordinary derivative, written over the differential in a Leibniz quotient. Prefer writing the derivative so the d sits before a variable.
Δ	**Finite Change**	Operator	Finite change in the quantity it prefixes, between two states, configurations, or points in time. Differs from the increment fused into named entries such as ΔT.
s	**Path Coordinate**	Dimensionless	Scalar coordinate along a geometric path, from zero at its start to one at its end, that a timing law maps to time; not a Laplace variable.

DRAFT

Learned Policies for Robotics

Learned policies propose actions rather than issue them. The symbols below cover how those policies are trained and sampled, including diffusion denoising, action chunking, and vision-language-action models, and how their imitation and reinforcement-learning objectives are written.

Symbol	Definition	Unit	Notes
π_θ	**Policy**	Distribution	Deep policy network parameterized by weights θ (italic), mapping observation tensors to action distributions.
$\mathbf{o}_t$	**Multimodal Observation**	Mixed	Synchronized tokenized sensor packet (RGB images, depth latents, joint encoders, tactile arrays).
$\mathbf{a}_t \in \mathcal{A}$	**Cognitive Policy Action**	Normalized	Unprivileged action emitted by neural policy π_θ (normalized joint delta, target pose, or gripper command).
K	**Ensembling Window Length**	Time steps	Number of overlapping past chunks blended in receding horizon execution ($K \leq H$).
$N_{\mathbf{eval}}$	**Decoder Evaluation Count**	Count	Dependent forward passes of a generative action head per chunk (denoising or ODE-solver steps); distinct from the ensembling window K and the step index k.
$\mathbf{z} \in \mathbb{R}^{d_z}$	**Latent Style / State Embedding**	Dimensionless	Compressed semantic representation or CVAE latent style variable in ACT policies.
$\bar{\alpha}_k$	**Diffusion Schedule Parameter**	$[0, 1]$	Cumulative noise variance schedule parameter $\bar{\alpha}_k = \prod_{s=1}^{k}(1-\beta_s)$ at diffusion step k.
$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	**Diffusion Gaussian Noise**	Tensor	Standard normal Gaussian noise corrupted onto clean action trajectories.
$\epsilon_\theta(\mathbf{o}_t, \mathbf{A}_t^k, k)$	**Noise Prediction Network**	Tensor	Neural network predicting added noise in Denoising Diffusion Probabilistic Models (DDPM).

Symbol	Definition	Unit	Notes
$\mathbf{A}^k$	**Noised Action Chunk**	Tensor	Action chunk after k forward-diffusion steps, $\sqrt{\bar{\alpha}_k}\,\mathbf{A}^0 + \sqrt{1-\bar{\alpha}_k}\,\epsilon$; the denoising iterate the reverse process refines.
$\mathcal{L}_{\text{diff}}(\theta)$	**Diffusion Matching Loss**	Scalar	Mean-squared error objective: $\mathbb{E}[\|\epsilon - \epsilon_\theta(\mathbf{o}_t, \mathbf{A}_t^k, k)\|^2]$ optimized across denoising steps.
$\mathcal{L}_{\text{ACT}}(\theta, \phi)$	**ACT Variational Objective**	Scalar	CVAE objective combining trajectory reconstruction $\mathcal{L}_{\text{recon}}$ with latent KL-regularization $\beta\mathcal{D}_{\text{KL}}$.
$q_\phi(\mathbf{z} \mid \mathbf{o}, \mathbf{A})$	**CVAE Recognition Encoder**	Distribution	Variational encoder in Action Chunking with Transformers parameterizing latent distribution $\mathcal{N}(\mu, \Sigma)$; ϕ here is the encoder's parameter set.
t_{div}	**Divergence Instant**	s	Time at which the autonomous policy's rollout begins to leave its supported states before a takeover; the start of the interval excluded from imitation targets.
e_{div}	**Trajectory Divergence Residual**	m	Signed lateral gripper-position error between measured and planned kinematics in the declared fixture frame for the latch example.
δ_{tol}	**Divergence Tolerance**	m	Positive threshold for the declared trajectory-divergence residual; expressed in that residual's physical unit.
r	**Reward**	Scalar	Per-step reward $r(s, \mathbf{a})$ that policy search maximizes; the stage cost of control is its negative. The barrier relative degree r also renders r but appears only in barrier analysis.
g	**Residual Gate**	$\{0, 1\}$	Deterministic, independently evaluated gate $g(s)$ that admits a learned residual correction only inside the certified contact region and is zero elsewhere.
N	**Trial Count**	Trials	Number of physical trials (rollouts) in a learning or evaluation campaign, which sets its wall-clock cost; distinct from the zero-failure trial count n.
ϵ	**Single-Step Imitation Error**	$[0, 1]$	Bounded probability of an unforced policy error during an individual execution step; bold $\boldsymbol{\epsilon}$ is diffusion noise, the conformal miscoverage rate is ϵ, and other error bounds carry a subscript (ϵ_{track}).
S	**Behavioral-Cloning Horizon**	Control steps	Episode length in control steps over which imitation error compounds; not the action-chunk horizon H, not a duration T, and not the token sequence length S.
$\mathcal{O}(S^2\epsilon)$	**Compounding Covariate-Drift Bound**	Task cost	Worst-case excess cost of fixed-archive behavioral cloning, which grows quadratically over a rollout of S control steps with per-step error ϵ.
$\mathbf{W}_{\text{patch}}, \mathbf{E}_{\text{pos}}$	**ViT Projection & Positional Bias**	Matrix	Convolutional/linear patch projection matrix and spatial coordinate position embedding tensors.
$\mathbf{Z}$	**Multimodal Fused Token Stream**	Tensor	Concatenated token sequence $[\mathbf{z}_{\text{vision}}, \mathbf{z}_{\text{text}}, \mathbf{z}_{\text{proprio}}]$ ingested by embodied transformer backbones.
M_{KV}	**KV-Cache Memory**	bytes	Autoregressive key-value cache memory footprint at the embodied batch size $B = 1$ ($2 \times N_L \times d \times S \times s_{\text{elem}}$ with full multi-head attention); the core's quantity.
$\mathcal{I}_{\text{intent}}$	**Intent Lease**	Tuple	$(\mathcal{G}_{\text{goal}}, [t_{\text{adm}}, t_{\text{intent_exp}}], \mathbf{w}_{\max})$: the admitted goal set, its validity interval, and its effort ceilings; it names a target, never a path. Calligraphic $\mathcal{L}$ stays the loss.
$t_{\text{intent_exp}}$	**Intent-Lease Expiry**	s	$t_{\text{intent_exp}} = \min(t_0 + \tau_{\text{ev}},\, t_{\text{adm}} + T_{\text{admin}},\, t_{\text{deadline,stop}})$, where $t_{\text{deadline,stop}}$ is the instant by which a stop must begin; see the lease table above.
τ_{ev}	**Evidence Horizon**	ms	Time for which a target estimate stays within its task tolerance without fresh evidence.
$\mathcal{C}$	**Conformal Prediction Set**	Set	Calibrated set of referent candidates that cannot be excluded at miscoverage rate ϵ; a singleton admits the referent, and a larger set refuses. It shares the glyph of the safe set $\mathcal{C}$, but the two never appear together.
ϵ	**Conformal Miscoverage Rate**	Probability	Allowed marginal probability that the conformal prediction set misses the true referent under the calibration procedure's exchangeability assumptions; coverage is at least $1 - \epsilon$.

Symbol	Definition	Unit	Notes
T_{wait}	**External Wait**	Seconds	Time the machine spends waiting for a remote operator's response during a handover or intervention.
ϵ_N	**Aggregated Surrogate Loss**	Dimensionless	Best-in-class surrogate imitation loss on the states the learner visited, aggregated over the rounds so far; differs from the error measured on expert-visited states.
$\hat{\pi}$	**Aggregation Learned Policy**	Function	Policy trained in one round of corrective dataset aggregation, one of the sequence the bound selects from; carries a round index in the appendix.
H_{exec}	**Chunk Execution Horizon**	Count	Number of actions of each predicted chunk the controller executes before the policy is queried again; at most the chunk horizon.
i	**Chunk Step Index**	Index	Position of one step within an action chunk, counted from zero.
α_{CQL}	**Conservative Penalty Weight**	Dimensionless	Weight of the conservative penalty that pushes down values of unsupported actions in conservative Q-learning.
$\mathcal{A}$	**Continuous Action Space**	Set	Set of continuous actions a robot policy may emit, a subset of real space of the action dimension.
$\mathbf{a}$	**Continuous Action Vector**	Varies	Continuous action vector of the robot's decision process, written without a step index: the argument of a policy, value function, reward, or action distribution, or an action recorded from a demonstrator. A star marks the demonstrated action; it lives in $\mathcal{A}$.
Q_ϕ	**Critic Action-Value Estimate**	Function	Return a learned critic predicts for a state and an action; the action counterpart of the critic's state-value estimate.
t_{pass}	**Decoder Pass Time**	s	Time of one dependent forward pass of a generative action head, one denoising or ODE-solver step; summed over the decoder evaluations in the proposer inference time.
p	**Demonstration Action Distribution**	Distribution	Conditional distribution of the actions demonstrators took given an observation, the target a behavioral cloning decoder models; its mean minimizes squared error and can fall between modes (mode averaging).
$\mathcal{D}_{\text{demo}}$	**Demonstration Dataset**	Set	Recorded observation-action pairs that an imitation loss averages over.
$\hat{\mathbb{E}}$	**Empirical Timestep Average**	Operator	Sample average over the timesteps of a batch of rollouts, the estimate of an expectation.
m	**Ensembling Decay Rate**	Dimensionless	Decay factor per chunk index of exponential temporal-ensembling weights; its size sets how fast newer chunks enter the blend.
$\hat{\pi}$	**Imitation-Learned Policy**	Function	Policy learned from expert demonstrations or expert-labeled states, compared against the expert policy; indexed by aggregation round in DAgger.
β	**Latent KL Weight**	Dimensionless	Weight on the KL divergence between the latent posterior and prior in a conditional variational autoencoder objective; not the RLHF reference penalty coefficient.
q	**Latent Posterior Distribution**	Distribution	Distribution over the latent style variable that a conditional VAE encoder infers from a demonstration chunk and its observation; compared with p in the KL term.
M	**Mixture Component Count**	Dimensionless	Number of components of a mixture distribution.
m	**Mixture Component Index**	Index	Position of one component of a mixture distribution, from one to the component count M; it indexes the weight, mean, and covariance of a mixture density network.
μ_m	**Mixture Component Mean**	Varies	Mean vector of one component of a Gaussian mixture, output by a mixture density network for the given input.
Σ_m	**Mixture Component Covariance**	Varies	Covariance matrix of one component of a Gaussian mixture, output by a mixture density network for the given input.
t_{enc}	**Observation Encoding Time**	s	Time to encode one observation, visual backbone and proprioception, once per policy proposal; the first term of the proposal latency.

Symbol	Definition	Unit	Notes
$\mathbf{o}_{t-H_{\text{hist}}:t}$	**Observation History Window**	Varies	Sliding window of the robot's most recent onboard observations, from the history length before the current step up to the current step; one row per observation step.
H_{hist}	**Observation history length**	Steps	Number of past steps the observation history window holds before the current one.
$\mathbf{a}$	**Robot Action**	Vector	Action vector a learned robot policy emits, written without a step index as the argument of a policy, value function, or action distribution; the same quantity as the cognitive policy action, which carries the step index.
$\mathbf{o}$	**Robot Observation**	Vector	Observation vector a robot policy receives from its sensors, written without a step index.
a_{req}	**Requested Action Tap**	Record	First stream of the four-stream action tap, the raw policy or operator request before kinematic mapping.
$\mathbf{a}'$	**Successor Action Vector**	Vector	Continuous action vector taken in the successor state of a transition, the next action a Bellman backup evaluates.
k	**Return Step Offset**	Index	Number of control steps after the current step at which a reward arrives, the summation index of a discounted return.
T	**Rollout Horizon**	Dimensionless	Number of control steps in one executed rollout, the index of its final state; a step count, not a duration.
θ_{old}	**Rollout Policy Parameters**	Vector	Parameters of the policy that collected the current batch of rollouts, frozen while the updated parameters are optimized; the denominator policy of the probability ratio.
$\mathcal{L}^{\text{CLIP}}$	**PPO Clipped Surrogate Objective**	Dimensionless	Clipped surrogate objective that proximal policy optimization maximizes, the empirical mean of the smaller of the ratio-weighted advantage and its clipped counterpart; a whole-batch objective, not the per-token term of the language model variant.
s_{priv}	**Privileged Simulator State**	Record	Ground-truth simulator quantities such as friction and contact wrenches that a deployed policy cannot measure; a subset of the physical world state exposed by the simulator.
ξ	**Simulation Parameter Vector**	Varies	Physical parameters that define one simulated environment: link masses, centers of mass, friction, joint damping, transport delays, and sensor noise; sampled in domain randomization, with the nominal point marked by a zero subscript.
$\mathbf{z}_{\text{proprio}}$	**Proprioceptive Token Embeddings**	Vector	Token embeddings of the robot's joint and force state fed to an embodied transformer; one block of the fused multimodal token stream.
p_{data}	**Static Data Distribution**	Distribution	Fixed data distribution from which a static benchmark draws its held-out examples.
$\mathbf{z}_{\text{text}}$	**Text Token Embeddings**	Vector	Token embeddings of the language instruction fed to an embodied transformer, one block of the fused token stream.
$\mathbf{z}_{\text{vision}}$	**Vision Token Embeddings**	Vector	Token embeddings an embodied transformer computes from camera images, one block of the fused token stream.
p_{patch}	**Patch Size**	Pixels	Side length of one square image patch a vision transformer tokenizes, typically 14 or 16 pixels.
$\mathbf{z}$	**Patch Token Embedding**	Vector	Latent token embedding a vision transformer forms from one image patch.
V_ϕ	**Critic Return Estimate**	Dimensionless	Remaining-return estimate a learned critic network predicts for an interaction history; the baseline of PPO, which GRPO replaces with the group mean reward.
s	**Decision Process State**	Record	Complete environment state of a decision process, including interaction history and variables the agent cannot observe. A record, not a number.
u	**Deviation Disadvantage**	Dimensionless	Largest extra expected cost of one deviation from the reference action followed by reference recovery, at any state the learner can reach; an assumption about the task, in normalized cost units.

Symbol	Definition	Unit	Notes
γ	**Discount Factor**	Dimensionless	Factor by which a reward arriving one step later is weighted in the return, between zero and one; one means undiscounted.
p	**Environment Transition Distribution**	Distribution	Distribution the environment induces over the next state given the current state and the action taken.
$\hat{A}$	**Estimated History Advantage**	Dimensionless	Estimate of how much an action taken after a given interaction history beats the policy value at that history.
Q	**Expected History-Action Return**	Function	Expected remaining return under a policy after taking a given action at a given interaction history and then following the policy.
V	**Expected Remaining Return**	Function	Expected remaining return under a policy from a given interaction history at a decision step; written with the policy as a superscript.
J	**Expected Return Objective**	Function	Expected return of a policy as a function of its parameters, the objective reinforcement learning maximizes. Not the imitation cost sum J.
J	**Expected Task Cost Sum**	Function	Expected sum over the horizon of the normalized immediate task costs a policy incurs; the imitation bounds compare it for a learner and the reference. Not monetary.

Embedded Real-Time Systems, Power, and Hardware Safety

Response-time analysis keeps the Liu–Layland letters of its literature, but only inside that model and only with a task index i, j, or k. The letters are C_i (worst-case execution time), B_i (blocking), I_i (interference), J_i (release jitter), T_i (period), R_i (worst-case response time), and D_i (relative deadline, which is T_{deadline} of task i in the series' terms). They combine as in $R_i = C_i + B_i + I_i \le D_i$. Outside that model these letters keep their table meanings.

DRAFT

Symbol	Definition	Unit	Notes
$L_{\text{bus}}, R_{\text{bus}}$	**Power Rail Inductance / ESR**	$\mu\text{H}, \text{m}\Omega$	Parasitic inductance and equivalent series resistance of the power harness feeding the inverter; with C_{bus} they set the droop ΔV_{droop} under a current step.
C_{bus}	**DC Bus Bulk Capacitance**	μF	Inverter decoupling capacitor bank buffering transient inductive voltage collapse.
t_{queue}	**Memory Bus Queue Delay**	ms	Contention latency $\sum_k D_{\text{vol}\,k}/\text{BW}_{\text{peak}} + N_{\text{conflict}}\Delta t_{\text{conflict}}$ across shared SoC crossbars.
Δt_{fresh}	**Telemetry Freshness Bound**	ms	Maximum allowable age of sensor telemetry before an interlock trips: $\Delta t_{\text{fresh}} \le e_{\max}/v_{\max}$.
θ_{offset}	**Clock Offset**	µs	Offset of one clock from another, estimated from a timestamp exchange as $\theta_{\text{offset}} = [(t_2 - t_1) - (t_4 - t_3)]/2$; its one-way path delay is T_{prop}.
σ_{jitter}	**Timing Jitter**	µs	Standard deviation of a nominally periodic event's timing about its schedule.
T_{blackout}	**Synchronization Blackout**	s	Interval during which a node receives no synchronization message and free-runs on its own clock.
t_{acq}	**Acquisition Epoch**	s	Instant a sensor transduced one observation, such as a camera frame's exposure midpoint or an encoder latch; the evidence epoch of a belief is the latest acquisition epoch among the measurements that constrained it.
t_{off}	**Actuator Recovery Time**	s	Recovery duration between actuator bursts in one duty cycle; the burst itself is t_{on}.
t_{on}	**Actuator Burst Time**	s	Duration an actuator runs at burst power in one duty cycle; pairs with the recovery time t_{off}.
v_{aisle}	**Aisle Travel Speed**	m/s	Speed the mobile base is set to travel in a warehouse aisle, chosen below the derived dry-floor speed ceiling; it is the speed at detection in the aisle stopping budget.

Symbol	Definition	Unit	Notes
t_{recv}	**Arrival Timestamp**	s	Instant a sensor sample arrives in host memory, when its direct memory access transfer completes; later than the capture timestamp by exposure, readout, transport, and transfer.
D_{avail}	**Available Braking Distance**	m	Clear distance minus the localization bound and the protective clearance margin; the distance braking and blind travel may use.
a	**Available Deceleration**	m/s^2	Deceleration a machine actually has, the smaller of its credible braking deceleration and the traction limit, friction coefficient times g.
$\Delta t_{\text{conflict}}$	**Bank Conflict Penalty**	s	Time one memory bank or row conflict adds to a queued transfer, about a precharge plus an activation.
d_{blind}	**Blind Travel Distance**	m	Distance a machine travels during the pre-brake delay, before retarding force begins: speed at detection times that delay.
R_{brake}	**Brake Chopper Resistance**	Ω	Resistance of the absorption resistor a dynamic brake chopper switches across the bus; must dissipate the peak regenerated power at the clamp voltage.
d_{brake}	**Braking Distance**	m	Distance a machine travels from brake onset to rest, speed squared over twice the credible deceleration; excludes travel during the pre-brake delay.
t_{capture}	**Capture Timestamp**	s	Midpoint of the sensor's exposure or firing window, the instant the observation describes; the letter t carrying index zero.
T_{case}	**Case Temperature**	°C	Temperature of a motor or package case.
$\tau_{\text{th,case}}$	**Case Thermal Time Constant**	s	Thermal time constant of the motor case stage of a multi-node motor thermal model.
V_{clamp}	**Chopper Clamp Voltage**	V	Bus voltage at which a dynamic brake chopper switches its absorption resistor across the bus; above the full-charge voltage and below the converter and switch ratings.
ϵ_{sync}	**Clock Conversion Bound**	s	Bound on the error of converting a sensor's capture timestamp to the clock of the safety microcontroller that hosts the permission path; measured from offset tails, path asymmetry, and drift, and added to the observation age at dispatch.
b_{clk}	**Clock Conversion Intercept**	s	Intercept of the affine mapping that converts a capture-clock time to the safety microcontroller's clock.
a	**Clock Conversion Slope**	Dimensionless	Slope of the affine mapping that converts a capture-clock time to the safety microcontroller's clock; near one, the drift between the clocks.
T_{comm}	**Console Communication Time**	s	Time the operator console and link add to a takeover: video transport to the operator plus the returning protocol exchange; shares its glyph with the gradient communication time of training, a different quantity.
λ	**Constant Hazard Rate**	h^{-1}	Constant rate of hazardous failures per operating hour of a machine, the rate a failure-free test exposure bounds from above; assumes the flat middle of the bathtub curve.
V_{uvlo}	**Converter Lockout Voltage**	V	Input voltage at which a DC-DC converter shuts off (undervoltage lockout); bus droop below it resets the computation the converter supplies.
T_0	**Copper Reference Temperature**	°C	Temperature at which a winding has its reference resistance, the base of the copper temperature coefficient.
α_{Cu}	**Copper Temperature Coefficient**	K^{-1}	Fractional rise in a copper winding's resistance per kelvin; about 0.00393 per kelvin for annealed copper at 20 °C.
$\tau_{\text{th,cu}}$	**Copper Thermal Time Constant**	s	Thermal time constant of the winding copper stage in a multi-node motor thermal model.
$R_{\text{th,ca}}$	**Core-to-Ambient Thermal Resistance**	K/W	Convective thermal resistance from a motor's stator core to the ambient air; it rises when an enclosure removes airflow.

DRAFT

Symbol	Definition	Unit	Notes
T_{∞}	**Equilibrium Winding Temperature**	°C	Temperature a winding approaches asymptotically under a constant loss power: ambient plus loss power times the thermal resistance in the first-order model.
$V_{\mathbf{bus}}$	**DC Bus Voltage**	V	Voltage of the DC bus that feeds a motor inverter.
$V_{\mathbf{rail}}$	**DC Rail Voltage**	V	Voltage of the shared DC power rail that feeds the compute and the motor inverters; the bus of V_{bus} is the inverter's side of it.
$V_{\mathbf{droop}}$	**Drooped Supply Voltage**	V	Supply voltage a core sees at the bottom of a droop event.
$V_{\mathbf{final}}$	**Final Bus Voltage**	V	DC bus voltage at the end of a regenerative stop when the bus capacitance alone absorbs the returned energy; a screening endpoint, since exceeding a component rating means the stop needs another energy path.
$t_{\mathbf{prop}}$	**Gate Propagation Delay**	s	Propagation delay of a logic gate as a function of its supply voltage; the alpha-power law makes it grow as the supply approaches the transistor threshold voltage.
$P_{\mathbf{heat}}$	**Heat Dissipation Power**	W	Power the hardware at one signal handoff dissipates as heat, the energy-and-heat limit of the handoff matrix; from under a watt on the safety microcontroller to tens of watts on the proposer accelerators.
$t_{\mathbf{deadline}}$	**Deadline Instant**	s	Clock instant by which an observation or action must be complete.
$T_{\mathbf{deadline,fallback}}$	**Fallback Deadline**	s	Latest permissible wait after a handover request before the fallback must begin, from the stopping inequality at the current speed and clearance; it clips the handover timeout ceiling T_{config}.
$T_{\mathbf{config}}$	**Handover Timeout Ceiling**	s	Configured protocol ceiling on the handover timeout, which the clearance-derived $T_{\text{deadline,fallback}}$ may shorten but never extend.
$a_{\mathbf{enf}}$	**Enforced Command Tap**	Record	Third stream of the four-stream action tap, the command the permission check admitted.
$V_{\mathbf{initial}}$	**Initial Bus Voltage**	V	DC bus voltage at the moment a regenerative deceleration begins; the worst case is a full pack, when the battery refuses charge.
$V_{\mathbf{nom}}$	**Nominal Supply Voltage**	V	Nominal regulated supply voltage of a core rail.
I_{max}	**Inverter Current Limit**	A	Largest current an inverter bridge lets flow, set to protect its transistors and the stator.
$I_{\mathbf{load}}$	**Load Current**	A	Current the loads draw from a bus node.
T_{j}	**Junction Temperature**	°C	Temperature of a silicon die at its transistor junctions; written as a function of time when it relaxes after a power step. A temperature always carries a subscript, since the bare letter is time.
$R_{\mathbf{th,JC}}$	**Junction-to-Case Thermal Resistance**	K/W	Thermal resistance from a silicon junction to the case of its package.
$R_{\mathbf{th,JA}}$	**Junction-to-Ambient Thermal Resistance**	K/W	Lumped thermal resistance from a silicon junction to the ambient air, the sum of the junction-to-case, interface-material, and chassis paths; not the motor stator-to-ambient path.
Ω	**Ohm Unit**	Unit	SI unit of electrical resistance, written upright after a prefix as in milliohms.
n	**Periodic Task Count**	Dimensionless	Number of periodic tasks in a real-time task set; it sets the Liu and Layland utilization bound, which falls toward the natural logarithm of two as the count grows.
$t_{\mathbf{MCU}}$	**Microcontroller Clock Time**	s	Time read on the safety microcontroller's monotonic clock.
$a_{\mathbf{cmd}}$	**Mapped Command Tap**	Record	Second stream of the four-stream action tap, the kinematically mapped command before it crosses to the safety microcontroller.
$a_{\mathbf{meas}}$	**Measured Response Tap**	Record	Fourth stream of the four-stream action tap, the plant response measured by joint encoders and current shunts.

Symbol	Definition	Unit	Notes
$t_{\mathbf{adm}}$	**Lease Admission Instant**	s	Instant at which the permission path admits an intent lease, the start of its validity interval.
$t_{\mathbf{plan_exp}}$	**Nominal Plan Expiry**	s	Instant, on the permission clock, at which the nominal setpoints of an admitted trajectory record end; never executed when the matched stop begins at commitment.
$t_{\mathbf{age},\max}$	**Observation Age Budget**	s	Configured upper bound on observation age above which a consumer refuses a state.
$t_{\mathbf{age},\max}$	**Maximum Observation Age**	s	Largest observation age a consumer tolerates before acting on the observation is no longer safe or stable; the phase-margin ceiling in the control chapters and the freshness threshold in the memory and machine-learning chapters.
$T_{\mathbf{mission}}$	**Mission Duration**	s	Duration of one mission of a mobile robot, the unit of operation over which a per-mission failure probability is allocated.
$E_{\mathbf{mission}}$	**Mission Energy**	J	Total energy a machine draws to complete one physical mission; distinct from the machine-learning workload's total energy.
$T_{\mathbf{sensory}}$	**Operator Sensory Time**	s	Time an operator needs to perceive a takeover stimulus, the perceptual stage of a simple reaction (Model Human Processor perceptual cycle).
$T_{\mathbf{cognitive}}$	**Operator Cognitive Time**	s	Time an operator needs to decide on a response to a takeover stimulus, the cognitive stage of a simple reaction (Model Human Processor cognitive cycle).
$T_{\mathbf{motor}}$	**Operator Motor Time**	s	Time an operator needs to move the controls once a response is chosen, the motor stage of a simple reaction; not the electrical time of a motor.
$t_{\mathbf{stabilize}}$	**Operator Stabilization Instant**	s	Time after a takeover at which the operator's corrections settle into a steady return toward the nominal corridor.
$\theta_{\mathbf{tol}}$	**Orientation Tolerance**	rad	Largest angular orientation error a task accepts.
N	**Observed Cycle Count**	Dimensionless	Number of operating cycles, each one observation of the monitored quantity, over which exceedances are counted.
k	**Observed Trial Failure Count**	Dimensionless	Number of failures observed among the trials of a test campaign, fixed-sample or sequential.
$\mathcal{D}$	**Operational Test Distribution**	Distribution	Declared joint distribution of trial conditions an evaluation campaign samples, the population its success probability describes; each stratum of a stratified test matrix is its own distribution, and it is not a dataset.
$T_{\mathbf{pre}}$	**Pre-Trigger Window Duration**	s	Length of the ring-buffer window before an intervention trigger that the logger freezes and keeps; sized from the out-of-loop takeover latency and rounded up to whole seconds.
$\tau_{\mathbf{delay},\max}$	**Pre-Brake Delay Ceiling**	s	Largest pre-brake delay that still lets the machine stop within its clearance: clearance minus margins and braking distance, divided by speed; the ceiling the lease rule subtracts the loop, transport, actuator, and observation times from.
$T_{\mathbf{compute}}$	**Stop Decision Compute Bound**	s	Bound on the stop-decision computation and any still-required input transfer that a hazard-triggered stop spends inside the pre-brake delay; a budget term, not operations over a sustained rate.
T_{Σ}	**Supervisory Takeover Latency**	s	Total time from a takeover demand to the first checked human command: the human sensory, cognitive, and motor times plus console transport and handshake; the generic form of the in-loop and out-of-loop design values.
$T_{\mathbf{lease,rem}}$	**Remaining Execution Lease**	s	Time left on the active execution lease, the lease length minus the time elapsed since its last accepted renewal; a field of the per-tick evidence tuple.

DRAFT

Symbol	Definition	Unit	Notes
L	**Replacement Latency**	s	Time from the capture of the observation a candidate trajectory rests on to that candidate's admission into the execution queue; a latency, not a period, and not the loss that bare L names.
$\mathcal{H}$	**Signal Handoff**	Record	One of the five canonical signal handoffs of the machine, from sensing to perception through enforcer to actuator, at which interface limits are budgeted; written with its position.
ϵ_{sync}	**Synchronization Error Bound**	s	Bound on the error of converting a timestamp from one node's clock to another's; the clock-conversion error of the observation contract, not an estimated clock offset.
T_{prop}	**PTP Propagation Delay**	s	One-way network path delay between two clocks, estimated from a symmetric two-way Precision Time Protocol exchange.
i	**Real-Time Task Index**	Index	Position of one periodic task in a real-time task set, the index of its period, deadline, and execution time; written bare when the text names the task.
B	**Task Blocking Time**	s	Longest time a real-time task can wait for a lower-priority task to release a shared resource; written with its task index, and under the priority ceiling protocol at most one lower-priority critical section.
c_{start}	**Read-Start Sequence Count**	Dimensionless	Value of a sequence-lock counter that a reader loads before copying the shared slot; the read is valid only if it is even and equals the end count.
c_{end}	**Read-End Sequence Count**	Dimensionless	Value of a sequence-lock counter that a reader loads after copying the shared slot; a change from the start count means a writer preempted the copy and the snapshot is torn.
t_{RCD}	**Row Activation Delay**	s	Minimum time from activating a DRAM row to the first column access to it, the row-to-column delay (JEDEC tRCD).
t_{RP}	**Row Precharge Time**	s	Minimum time a DRAM bank needs to close its open row by precharge before another row can be activated (JEDEC tRP); a row conflict costs it plus the row activation delay.
λ_{raw}	**Raw Hardware Fault Rate**	h^{-1}	Rate of hazardous hardware faults before any diagnostic mitigates them, the sum of the component fault rates; usually quoted in failures in time.
L_{10}	**Rated Fatigue Life**	Dimensionless	Cumulative load-cycle count at which 10% of a population of identical components is expected to have failed; the subscript ten is part of the symbol.
I_{source}	**Source Current**	A	Current arriving at a bus node through the source path of battery, harness, and switches; it differs from the load current during a transient, and the bus capacitance supplies the difference.
V_{source}	**Source Voltage**	V	Unloaded voltage of the supply that feeds a bus node through the source path; minus the drop across the rail resistance and inductance it gives the bus voltage.
R_0	**Reference Winding Resistance**	Ω	Winding resistance at the reference temperature, the base of copper's linear resistance law.
T_{core}	**Stator Core Temperature**	K or °C	Temperature of the stator core in a two-node motor thermal model.
S_p	**Protective Separation Distance**	m	Minimum distance a speed-and-separation monitor must keep between a person and a robot; the sum of the human travel, robot reaction travel, robot stopping travel, intrusion, and the two uncertainty terms of ISO/TS 15066.
S_r	**Robot Reaction Travel**	m	Distance a robot travels during its reaction time before stopping begins; a term of the protective separation distance.
S_s	**Robot Stopping Travel**	m	Distance a robot travels while stopping; a term of the protective separation distance.
Z_r	**Robot Position Uncertainty**	m	Position uncertainty of the robot; a term of the protective separation distance.
m	**Sequential Trial Count**	Dimensionless	Number of trials a sequential test has observed so far, the index of its running log-likelihood ratio.

Symbol	Definition	Unit	Notes
X	**Tail-Risk Observation**	Varies	Monitored physical quantity whose rare large values a tail model describes, such as a contact load or a control-loop latency, treated as a random variable; in the unit of the quantity monitored, and indexed for one independent observation.
x	**Tail-Risk Observation Level**	Varies	Level of the monitored quantity at which its distribution or density is evaluated, or one observed value of it; the lower cutoff of a power-law tail is marked by a zero subscript.
ξ	**Tail Shape Parameter**	Dimensionless	Shape parameter of an extreme value tail model, positive for a power-law tail, zero for an exponential tail, and negative for a bounded tail; shared by the generalized extreme value and generalized Pareto models.
x_F	**Tail Support Upper Endpoint**	Varies	Right endpoint of the support of the monitored quantity's distribution, possibly infinite; in the unit of the monitored quantity.
ϵ	**Target Exceedance Probability**	Dimensionless	Per-sample probability of a catastrophic exceedance that a design margin is sized against; a probability per sampled cycle, not a rate per operating hour.
I	**Task Interference Time**	s	Processor time that higher-priority tasks and interrupt handlers take from a real-time task within its response window. Written with its task index; it depends on the response time itself, so it is found by a fixed-point recurrence.
T	**Task Period**	s	Interval between successive releases of a periodic real-time task, written with its task index.
D	**Task Relative Deadline**	s	Longest time after its release by which a job of a real-time task must complete, written with its task index. The series-wide deadline is T_{deadline}.
U	**Task Set Utilization**	Dimensionless	Share of processor time a set of periodic real-time tasks demands, the sum over tasks of C/T.
C	**Worst-Case Execution Time**	s	Worst-case execution time of one real-time task in response-time analysis; used only inside that analysis and only with a task index.
R	**Worst-Case Response Time**	s	Longest time from a real-time task's release to its completion; used only with a task index.
t_{start}	**Tracking Start Instant**	s	Instant, on the permission clock, at which nominal tracking of an admitted trajectory record begins; the first of the record's four deadlines.
t_{skew}	**Timestamp Skew**	s	Skew between the time stamps of two records or clocks. The book writes it with the finite-change prefix, Δt_{skew}.
t_{last}	**Track Last Update Time**	s	Time of the most recent observation associated with a track.
t_{current}	**Current Wall-Clock Time**	s	Wall-clock time at which a monitor evaluates staleness; a point in time, like t, fixed at the moment of the check.
T	**Failure Observation Interval**	s	Length of the time interval over which failures are counted under the Poisson failure model; a generic window, not the length of one job.
λ	**Component Failure Rate**	h^{-1}	Rate at which one component fails; the system-wide rate is the component count times this rate under independent, identical components.
$\lambda_{\text{unmitigated}}$	**Unmitigated Hardware Fault Rate**	h^{-1}	Rate of hardware faults whose hazardous effect no diagnostic mitigates, the rate compared with the target hazard rate.
T_{lower}	**Watchdog Earliest Service Time**	s	Time after the last reload before which a windowed watchdog rejects a service attempt as too early, the lower end of its service window. With T_{upper} it bounds the valid window.
T_{upper}	**Watchdog Latest Service Time**	s	Time after the last reload by which a windowed watchdog must be serviced, past which it trips the safe state.

Symbol	Definition	Unit	Notes
T_{trip}	**Throttle Trip Temperature**	°C	Junction temperature at or above which a hardware thermal governor reduces the processor clock; paired with a lower clear temperature for hysteresis.
V_{th}	**Transistor Threshold Voltage**	V	Gate-source voltage at which a transistor begins to conduct. The headroom between supply and threshold sets gate delay in the alpha-power law.
α	**Velocity Saturation Exponent**	Dimensionless	Technology-dependent exponent of the alpha-power law, by which drain current grows with gate overdrive above threshold; it falls from 2 for a long channel toward 1 as carrier velocity saturates.
$R_{\text{th,wc}}$	**Winding-to-Core Thermal Resistance**	K/W	Conductive thermal resistance from a motor winding to its stator core; in series with the core-to-ambient resistance it makes the total R_{th}.
I	**Winding Current**	A	Electric current flowing through a motor winding, which heats it at the rate of current squared times resistance.
I	**Conductor Current**	A	Electrical current flowing through a power-distribution conductor; resistive loss is its square times the conductor resistance.
P_{loss}	**Winding Loss Power**	W	Resistive power dissipated in the motor windings.
T_{limit}	**Winding Temperature Limit**	°C	Largest winding temperature the insulation class and derating policy allow.
µ	**Micro Unit Prefix**	Prefix	SI prefix for one millionth, written before a unit such as seconds; it only scales the unit after it and is never a quantity, a service rate, or a mean.
V	**Supply Voltage**	V	Voltage that powers a digital circuit and that voltage and frequency scaling lowers; dynamic power scales with V^2.
$L_{\text{parasitic}}$	**Parasitic Loop Inductance**	H	Unintended inductance of the current loop that feeds an accelerator core rail; it turns current slew into voltage droop.

Deep Learning Foundations and Edge Serving

Embodied robotic platforms increasingly run large multimodal foundation models, such as vision-language-action (VLA) backbones and cross-attention policy decoders, alongside deterministic real-time control stacks. Serving these models on an embedded edge computer requires tracking both the neural architecture dimensions and the memory footprint those dimensions occupy:

Symbol	Definition	Unit	Notes
d_{head}	**Attention Head Dimension**	Integer	Per-head hidden dimension in attention layers.
d	**Hidden Dimension**	Integer	Hidden feature (model) width d of a transformer layer or representation; the distance to a target, also d, is d and appears only in the intent-lease reachability test.
$p(\cdot)$	**Distribution**	Distribution	Probability mass or density $p(x)$, conditional $p(y \mid x)$, or an observation model $p(\mathbf{o} \mid s)$; lowercase avoids the collision with P (parameter count).
$\Pr(E)$	**Event Probability**	Dimensionless	Probability of an event E, as in $\Pr(\text{error by step } t)$; use $p(\cdot)$ for distributions.
$d_{\text{in}}, d_{\text{out}}$	**Layer Input and Output Widths**	Integer	Number of input and output features of a dense layer, which holds $d_{\text{in}} d_{\text{out}}$ weights.
η	**Learning Rate**	Scalar	Step size for the optimizer; never bare for hardware efficiency, which is η_{hw}.
$\mathcal{L}$	**Loss Function**	Scalar	The objective minimized during training; a subscript or superscript names the objective ($\mathcal{L}_{\text{MSE}}$, $\mathcal{L}_{\text{chunk}}$, $\mathcal{L}^{\text{CLIP}}$).
θ	**Model Parameters**	Vector	Trainable parameter vector or weights θ of a learned model, such as a policy.
N_L	**Number of Layers**	Integer	Total number of layers in a network.

Symbol	Definition	Unit	Notes
S	**Sequence Length**	tokens	Input context or sequence length S in tokens. The behavioral-cloning horizon S also renders S but counts control steps; the two never share an equation.
M_{weights}	**Weight Memory**	bytes	Memory occupied by model weights ($P \times$ bytes-per-parameter).
s_{elem}	**Element Storage Size**	bytes/element	Bytes per stored tensor element at the deployed precision.
I	**Arithmetic Intensity**	FLOP/byte	Operations per byte moved from off-chip memory, O/D_{vol}; it decides whether an evaluation is bound by memory or by compute.
N_{eff}	**Effective Sample Size**	Dimensionless	Number of independent observations that carry the same information about a mean as a serially correlated stream.
e	**Environment Parameters**	Record	External physical variables of one trial or rollout, such as floor friction, illuminance, obstacle geometry, and air temperature; written non-bold, so a record rather than a vector.
ϕ	**Evaluation Predicate**	Function	Binary predicate that maps a realized trial trajectory to its task outcome, zero or one.
z_ϵ	**Exceedance Return Level**	Varies	Level of the monitored quantity exceeded with the target probability per sampled cycle; its subscript is that exceedance probability.
u	**Exceedance Threshold**	Varies	High threshold on a performance metric above which peaks-over-threshold modeling fits exceedances; it carries the metric's units (meters for a distance overshoot).
H	**Generalized Pareto Distribution**	Function	Cumulative distribution function of the excess over a high threshold, parameterized by a scale and a tail shape.
$\tilde{\sigma}$	**Generalized Pareto Scale**	Varies	Scale parameter of the generalized Pareto distribution fitted to exceedances over a threshold.
$p_{\text{fail, upper}}$	**Failure Upper Confidence Limit**	Dimensionless	One-sided upper confidence limit on the per-trial failure probability from a finite campaign.
p	**Hypothesized Trial Failure Probability**	Dimensionless	Per-trial failure probability a sequential probability ratio test specifies under one of its two hypotheses, indexed 0 for the unacceptable null and 1 for the acceptable alternative.
h	**Hazard Class Index**	Dimensionless	Index of one defined hazard class.
N_{fleet}	**Fleet Robot Count**	Dimensionless	Number of robots in a deployed fleet; not the accelerator count of a data center fleet.
Σ	**Gaussian Covariance Matrix**	Matrix	Covariance matrix of a multivariate Gaussian; entries carry the products of the component units.
μ	**Gaussian Mean Vector**	Vector	Mean vector of a multivariate Gaussian, paired with Σ.
O_{prefill}	**Prefill Operations**	FLOP	Arithmetic operations one evaluation's prefill executes over all of its input tokens; the compute-bound phase of a fresh evaluation, and an instance of the operation count.
f	**Proposer Evaluation Rate**	Hz	Rate of fresh proposer evaluations when every evaluation streams the weights.
σ	**Softmax Function**	Function	Function that maps a vector of logits to a probability vector that sums to one; normalization by construction, not calibrated confidence, and not the sigmoid.
x	**Model Input**	Record	Input of one example, the data a model receives to produce a prediction; its label partner is $\mathbf{y}$. Not a scalar feature x_i of a neuron.
$\hat{y}$	**Model Prediction**	Dimensionless	Scalar prediction a model outputs for one example, compared with its label by the loss; the vector of class probabilities is $\hat{\mathbf{y}}$.
$\mathbf{z}$	**Logit Vector**	Vector	Unnormalized score for every class that a classifier outputs before softmax; its entries are z.
z	**Output Logit**	Dimensionless	Raw score a model outputs for one class before the softmax, indexed by class as z_i.
$\mathcal{L}_{\text{val}}$	**Validation Loss**	Scalar	Loss a trained model incurs on held-out validation data; the outer objective of neural architecture search, never the training loss.
$\mathbf{W}$	**Weight Matrix**	Matrix	Matrix of learned weights one layer multiplies its input by, or the kernel tensor of a convolution layer; a subscript names the layer, as in $\mathbf{W}_1$.

DRAFT

Symbol	Definition	Unit	Notes
$\mathbf{J}$	**Function Jacobian**	Matrix	Matrix of partial derivatives of a differentiated function's outputs with respect to its inputs; not the manipulator Jacobian of a robot arm.
$\mathbf{0}$	**Zero Matrix or Vector**	Matrix	Matrix or vector whose entries are all zero; bold, so it is not the scalar zero.
H	**Image Height**	pixels	Number of pixel rows in an image or its segmentation mask; shares its glyph with horizons but counts rows.
W	**Image Width**	pixels	Number of pixel columns in an image or its segmentation mask.
$\mathbb{I}$	**Indicator Function**	Function	Function equal to one when its condition holds and zero otherwise.
$\mathcal{D}_{\mathrm{KL}}$	**KL Divergence**	nats or bits	Expected extra code length from encoding one distribution with a code built for another; asymmetric, so $\mathcal{D}_{\mathrm{KL}}(p\|q) \neq \mathcal{D}_{\mathrm{KL}}(q\|p)$, and distillation uses the teacher-to-student order.
H	**Shannon Entropy**	bits	Average uncertainty of a probability distribution, the expected negative log probability of its outcomes; the log base sets the unit, base two for bits and natural log for nats.
H	**Statistical Hypothesis**	Record	A hypothesis about a population: H_0 the null, H_1 the alternative, and $H_{(i)}$ the hypothesis with the i-th smallest p-value. A record, not a number.
$z_{1-\alpha_{\mathbf{sig}}/2}$	**Significance Critical Quantile**	Dimensionless	Standard-normal quantile at one minus half the significance level, the two-sided critical value of a release test; about 1.96 at a significance level of 0.05.
s'	**Next Decision-Process State**	Record	State a decision process moves to after an action is taken in the current state, as in the transition law $p(s' \mid s, a)$; the successor of the current state.
s	**Policy Observed State**	Record	State or observation a policy conditions its choice of action on, the conditioning argument in $\pi_\theta(a \mid s)$.
a	**Policy Action**	Record	Action a policy chooses given the state it observes, the chosen argument in $\pi_\theta(a \mid s)$; agentic rollouts use their own rollout-action symbol.
ρ	**Policy Probability Ratio**	Dimensionless	Ratio of a sampled token's probability under the current policy to its probability under the behavior policy that generated it; clipped to an interval of half-width ϵ_{clip} around one.
ϵ_{clip}	**Ratio Clipping Width**	Dimensionless	Half-width of the interval around one to which the clipped surrogate limits the probability ratio; between zero and one.
Q^π	**Policy History-Action Return**	Dimensionless	Expected remaining return under the current policy after taking a given action at a given interaction history and then following that policy, written $Q^\pi(h_k, a_k)$.
V^π	**Policy Remaining Return**	Dimensionless	Expected remaining return under the current policy from a given interaction history, written $V^\pi(h_k)$; the baseline the advantage subtracts.
A^π	**Policy History Advantage**	Dimensionless	Return of a given action at a given interaction history minus the remaining return at that history, $A^\pi(h_k, a_k) = Q^\pi(h_k, a_k) - V^\pi(h_k)$.
d_π	**State Distribution**	Distribution	Average distribution of states a policy visits over the horizon; the per-step form carries a superscript, d_π^t.
d_{π^*}	**Reference State Distribution**	Distribution	Average distribution of states the reference policy visits over the horizon; the reference-policy counterpart of d_π.
τ	**Trajectory**	Record	Ordered record of the states, actions, and observations of one delegated task, $\tau = (s_0, a_0, o_1, \ldots, s_H)$; a record, not a number.
K	**Class Count**	Count	Number of classes a classifier distinguishes, or vocabulary entries it scores; the length of its softmax output.
y	**Example Label**	Record	Label paired with one input, treated as a random variable; a bold form is the label written as a vector.

Latin matrices and vectors are set in bold (**W**, **x**); scalars, dimensions, indices, and individual matrix entries stay italic (N, d, i, W_{ij}). The generic parameter symbol θ is the one conventional exception and stays italic. Local matrix algebra follows the same bold rule: matrix operands are bold (**A**, **B**, **C**), as in general matrix multiply (GEMM) $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$. The bold form marks a matrix operand and is distinct from the italic scalar batch size B, which remains batch size in scalar contexts.

A GEMM's scalar dimensions are M, N, and K (**A** is $M \times K$, **B** is $K \times N$). Asymptotic cost is written $\mathcal{O}(\cdot)$, which keeps it distinct from the operation count O.

Evaluation and Statistical Evidence

A physical trial is expensive, so the evaluation chapters count carefully. These symbols name the trials themselves, the samples a metric is computed over, and the threshold exceedances that tail estimates are built from.

Symbol	Definition	Unit	Notes
n_{cal}	**Calibration Example Count**	Count	Number of held-out calibration examples, and so of nonconformity scores, used to set a conformal prediction threshold. Not the quantization calibration set.
N	**Evaluation Sample Count**	Count	Number of examples in the evaluation set over which an accuracy or calibration metric is computed; it sets the sampling error. Not the training set size D.
X	**Discrete Random Variable**	Record	Discrete random variable, the uncertain quantity whose outcomes a distribution describes; uppercase for the variable, lowercase for one of its outcomes.
c	**Trial Condition**	Record	Condition of one physical trial: initial plant state, environment parameters, plant parameters, computational configuration, and termination protocol; drawn from the operational test distribution.
n	**Trial Index**	Index	Position of one physical trial within a learning campaign, counted from one.
p	**Trial Success Probability**	Dimensionless	True probability that one trial succeeds under the declared evaluation protocol and test distribution.
N_u	**Threshold Exceedance Count**	Count	Number of trials whose performance metric exceeds the threshold.
y	**Threshold Excess**	Varies	Amount by which the monitored quantity exceeds the threshold, one recorded exceedance; it carries the unit of the monitored quantity. Its random-variable form is Y.
Y	**Threshold Excess Variable**	Varies	Excess of the monitored quantity over the threshold, conditional on exceeding it, treated as a random variable.

Units and Precision

- Physical units: This book uses SI (metric) units throughout, including meters, kilograms, seconds, watts, and °C, consistent with standard engineering and scientific practice. Where source data was originally reported in imperial units, the book converts to SI and notes the original values parenthetically. A space always separates the number from the unit (for example, 100 ms, 2 TB/s).
- Data and memory: This book uses **decimal SI prefixes only**: KB = 10^3 bytes, MB = 10^6, GB = 10^9, TB = 10^{12}. Binary-prefixed units do not appear in prose; model sizes, activation footprints, throughputs, and transfer volumes are reported in decimal units (for example, 2 TB/s, 102 MB). Hardware memory capacities are the one exception: they keep the vendor's nameplate figure and label. A device modeled here with a binary 80 GB capacity holds 80×2^{30} bytes; usable capacity and allocation headroom remain separate.
- Compute: The notation distinguishes operation counts from rates. Total work uses FLOPs (for example, GFLOPs, TFLOPs), while throughput uses FLOP/s with decimal prefixes (for example, GFLOP/s, TFLOP/s).
  - 1 TFLOP = 10^{12} FLOPs
  - 1 TFLOP/s = 10^{12} FLOPs per second
  - Arithmetic intensity conventionally uses FLOP/byte as a unit ratio (floating-point operations per byte moved). This is a ratio unit, not a total-work symbol or throughput symbol.

- Power: Power carries a subscript (P_{avg}, P_{device}, P_{cluster}); bare P remains parameter count.
- Latency percentiles: Written p50, p95, and p99 in upright type, inside math as well as in prose.
- Currency: Dollar amounts use the dollar sign (`$`); unless otherwise noted, dollar-denominated costs are U.S. dollars (USD).
- Precision:
  - **FP64**: Double precision (8 bytes)
  - **FP32**: Single precision (4 bytes)
  - **TF32**: TensorFloat-32 (19-bit Tensor Core compute mode; inputs and storage remain FP32)
  - **FP16**: Half precision (2 bytes, standard range)
  - **BF16**: Brain float (2 bytes, wide dynamic range)
  - **FP8**: Quarter precision (1 byte, E4M3 or E5M2 format)
  - **FP4**: 4-bit floating-point format
  - **INT8**: 8-bit integer (1 byte)
  - **INT4**: 4-bit integer; lower integer precisions follow the same uppercase `INTn` pattern (for example, INT3, INT2)

Quick Reference: Resolving Collisions

Throughout this book, two single-letter symbols recur in both machine learning equations and hardware performance budgets. The most frequent symbol collisions across the machine learning, control, and embedded systems literatures are resolved as follows:

Symbol	ML Meaning	Systems Meaning	Book Convention
B	**Batch Size**	Bandwidth	**Batch Size**. Use BW for bandwidth.
P	**Parameters**	Peak FLOP/s	**Parameters**. Use R_{peak} for peak rate.

DRAFT

As a general principle, ML conventions take precedence for single letters, while systems concepts get subscripts or multi-letter symbols. This preserves compatibility with the ML literature while distinguishing the robotics and embedded systems meanings used in this book.

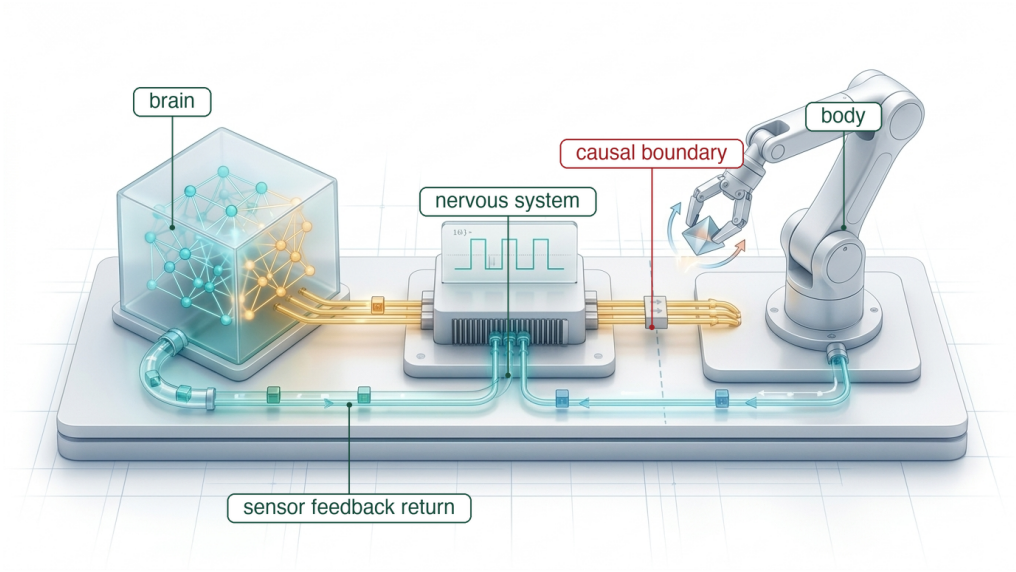
DRAFT

MAIN TEXT

DRAFT

1

The Causal Boundary

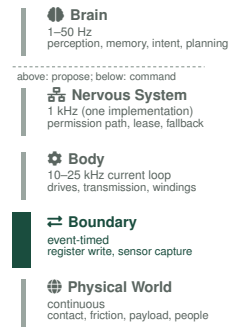


1.1 Artificial Intelligence Beyond Glass
1.2 From Prediction to Action
1.3 The Question This Book Answers
1.4 Defining the Causal Boundary
1.5 Three Machine Classes
1.6 The Physical AI Scope Test
1.7 The Machine in Five Levels
1.8 When the Responsibilities Fail Together
1.9 Where Safety, Security, and Accountability Meet
1.10 The Four Bedrock Laws
1.11 The Architecture of This Book
1.12 Fallacies and Pitfalls
1.13 Summary
1.14 Historical Perspective and Further Reading

Purpose

Why can a better prediction still produce a worse physical action?

A learned model can recognize a task correctly and still act too late to stop a moving machine or prevent excessive contact force. Once a command produces motion or heat, replacing the prediction cannot undo the work already delivered. The engineer must therefore connect model quality to the body's limits, the time available to respond, and the evidence supporting the next action. An independent permission path must be able to refuse a proposal and carry out a feasible fallback even when the model stalls. At the causal boundary, where commands become physical work, irreversibility makes these responsibilities part of the system's architecture.



Learning Objectives

- Trace how machine learning systems evolved from disembodied prediction through digital agency to physical embodiment, and explain why that progression gives data, compute, evaluation, and deployment physical stakes
- Explain why an action that crosses the causal boundary requires reasoning about consequences that software rollback cannot undo
- Calculate how response delay consumes a moving machine's stopping clearance
- Apply the scope test to distinguish physical AI from advisory learning and classical control
- Distinguish learned proposals from permission to execute them in the five-level machine
- Diagnose how perception, response timing, and missing redundancy interacted in the Tempe collision
- Relate the four laws to their established names in the field and to the book's four parts

1.1 Artificial Intelligence Beyond Glass

When an image classifier mislabels a warehouse photograph on a server, a downstream filter can discard the label before any machine moves; when a mobile robot's policy misjudges aisle clearance and drives its wheel motors, the chassis is already accelerating toward the rack. Across offline advisory prediction, digital agency, and physical embodiment, computation acquires different consequences as its outputs enter an action loop, and physical AI diverges from the other two in how agency, time, and recovery behave (table 1.1).

In the offline or advisory case, a model delivers predictions against a static dataset before any downstream action closes a feedback loop. Whether classifying an image, predicting customer churn, estimating a bounding box, or ranking search results, the model operates in open loop. Its input distribution is exogenous, because the predictions it emits do not alter the process that generates later test examples. Because those requests arrive independently, throughput-oriented serving can batch many inputs together ($B \gg 1$) across pipeline stages, performing many arithmetic operations for every byte of weights fetched from memory (its arithmetic intensity). The advisory model exercises no agency of its own. Before a person or downstream service acts on the output, an incorrect label or irrelevant search result can be ignored, filtered, or recalculated without having done physical work.

The emergence of agentic machine learning marked an architectural transition from passive, single-shot prediction to active, sequential decision-making. Endowed with reasoning loops and test-time deliberation, an agentic model formulates multi-step plans, maintains internal state across execution traces, and autonomously invokes external tools. These tools execute database queries, call web APIs, run generated code in execution environments, and coordinate software subagents. In this regime, the execution loop is closed. An action taken at step t alters the state of the software environment, so subsequent observations o_{t+1} depend directly on prior decisions a_t. The system possesses genuine agency.

Within a controlled software environment, this agency can operate inside a protective envelope, "behind the glass." Such environments can grant agentic systems three structural privileges, though external services and consequential tool calls may fall outside that envelope:

- **Elastic virtual time**: A controlled software environment can wait while the model computes. Whether an agent requires 200 ms or 30 s of test-time compute to plan its next tool call, the environment can hold its simulated state until the action is emitted. External services continue evolving on their own clocks.
- **Transactional rollback and sandboxing**: Tool execution can occur within isolated containers, virtual machines, or transactional databases. When an action produces an error inside this envelope, software exceptions catch the fault, database transactions roll back, or the container restarts from a clean snapshot. These mechanisms cannot undo consequences already delivered outside it.

- **Discrete symbolic interfaces**: Actions are structured strings, JSON schemas, or API payloads whose syntax and validity can be verified by deterministic compilers and linters before execution.

Physical artificial intelligence represents the arrival of agency in the physical world. A physical AI system does not invoke a software API inside a virtual sandbox; its outputs cross the **causal boundary** into physical actuators. A learned model's output becomes a setpoint (the target value a low-level controller drives toward), the setpoint becomes electrical current in a motor phase winding (one of the stationary copper coils whose current turns the motor), and the current becomes electromagnetic torque. The torque accelerates physical mass through gearboxes, linkages, and contact surfaces sharing space with people and objects. That physical embodiment strips away every digital privilege:

- **Continuous time**: Physical time advances under Newtonian mechanics regardless of processor state. The **physical plant**, the apparatus being controlled such as a robot's wheeled base or arm, does not pause while a neural network deliberates. In a moving robot, computational latency is not a software delay. A machine moving at speed v that takes a time τ_{delay} to begin braking covers $v\,\tau_{\text{delay}}$ toward an obstacle under unguided momentum. Delay is physical distance.
- **Mechanical and thermodynamic irreversibility**: Physical state transitions admit no software rollback. Once software commands current through an inverter (the power electronics that switch a DC supply into the motor's phase currents), electromagnetic force accelerates mass, dissipating irreversible Joule heat ($P_{\text{phase}} = I_{\text{phase}}^2 R_{\text{phase}}$ in each phase winding) and altering mechanical contact states. Kinetic energy ($E_k = \frac{1}{2}mv^2$) cannot be garbage-collected, and a mechanical impact cannot be undone by restoring a checkpoint. A command that has moved a machine can be answered only by another physical action, which itself consumes time, clearance, and energy.
- **Single-batch, hard real-time execution**: On the running machine, each new action proposal depends on observations shaped by prior motion, so the live loop does not accumulate independent observations into a throughput batch ($B = 1$). Control loops must also meet hard real-time deadlines across several coupled timescales: motor current loops at tens of kilohertz, loops near a kilohertz that check motion and regulate contact force, learned policies renewing at tens of hertz, and slower models interpreting the scene a few times per second. Section 1.7 gives the running machine's rates. A missed deadline does not degrade user experience; it can destabilize the physical plant.

Losing these privileges forces an architectural response. Deep neural networks are statistical function approximators, vulnerable to distribution shift, sensor noise, and hallucination, so they cannot be granted unmediated authority over physical actuators. Physical AI therefore separates proposal from permission. High-capacity learned models act only as *proposers* in an unprivileged tier (the Brain). An independent, deterministic *permission path* (in the Nervous System) checks each proposal against the machine's physical limits and *admits* (accepts for execution) only the proposals that pass. It also keeps an executable fallback ready on isolated real-time hardware. The permission path does all of this before any action crosses the causal boundary into the plant (the Body).

Table 1.1: Three deployment regimes of machine learning systems: The selected arrangements progress from offline advisory prediction to digital agency to physical embodiment. The unit of engineering shifts from advisory prediction to digital tool execution, and finally to irreversible physical action across the causal boundary.

Deployment Regime	Execution Substrate	Primary Output	Environmental Coupling	Time Semantics	Failure & Recovery	Safety & Authority Model
Offline/advisory prediction	Cloud servers / accelerators	Advisory prediction (label, score, token)	Open-loop (static dataset, exogenous data)	Batchable in the selected serving case	Discard or recompute before downstream action	Advisory; downstream software or human decides

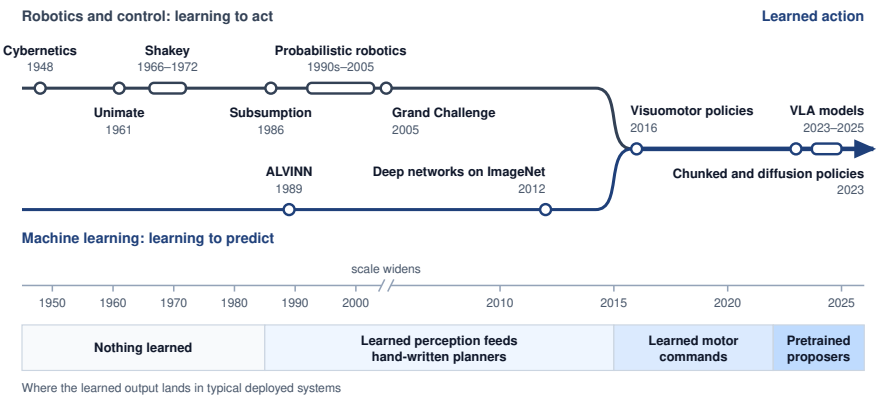


Figure 1.1: From prediction to action: Robotics and control (top) learned to act on the physical world, and machine learning (bottom) learned to predict from data; the two lines merge in learned action. The band beneath the time axis shows where the learned output lands in typical deployed systems of each era, from no learned component, to learned perception feeding hand-written planners, to learned motor commands, to pretrained models that propose motion. The time axis widens after 2005.

Deployment Regime	Execution Substrate	Primary Output	Environmental Coupling	Time Semantics	Failure & Recovery	Safety & Authority Model
Agentic ML	Digital software environments	Tool execution (API payload, code snippet)	Closed-loop in software (actions alter state)	Elastic virtual time in controlled environments	Software exception, container restart, DB rollback	Sandboxed permission; API scopes and virtual gates
Physical AI	Embodied physical plant	Physical force and motion (current, torque)	Closed-loop in physics (actions alter world)	Inexorable continuous time; delay is distance	Irreversible work; energy dissipation; damage	Proposal vs. permission; deterministic runtime veto

Physical AI did not arrive solely through the scaling of language models. It represents the convergence of two long-standing engineering traditions: robotics and control, which learned to act reliably on a physical world that pushes back, and machine learning, which learned to extract representations from data at scale. Over eighty years, the learned model has moved step by step from the periphery of the machine toward its actuators, and each step shifted what the surrounding systems architecture had to guarantee. That history explains what physical AI inherits from classical control and digital learning, and why its central systems challenge is new.

1.2 From Prediction to Action

The acting line begins with feedback. Wiener's cybernetics described animals and machines alike as systems that measure the result of their own action and use it to correct the next one (Wiener 1948), and control theory soon learned to estimate a moving system's state from noisy measurements (Rudolph Emil Kalman 1960). The first industrial robots needed little of this at the level of the task. In 1961 a Unimate arm began stacking hot die-cast parts at General Motors by replaying step-by-step commands stored on a magnetic drum (Computer History Museum 1961). Nothing in that program could notice a person in the arm's path, so protecting people meant keeping them out of its reach. The machine acted, but it did not perceive, and nothing in it was learned. Robotics and control learning to act and machine learning mastering prediction run as the upper and lower lanes of figure 1.1 until they merge.

Shakey, developed at SRI from 1966 through 1972, joined perception to planning (Nilsson 1984). It took camera images and range-finder readings, kept a model of its surroundings, and used a planner called STRIPS to compose sequences of actions that would achieve a stated goal. This organization, later summarized as sense-plan-act (S·P·A), treats acting as the last step of a computation. Shakey moved slowly, and its designers treated its motion as running asynchronously to the programs

directing it (Nilsson 1984). The world was assumed to wait while the computer thought. Shakey also carried a detail that anticipates this book's architecture. When one of its catwhiskers touched an obstacle, the robot stopped by a reflex action beneath the planner.

Brooks inverted that organization. His subsumption architecture built control from layers of simple, asynchronous behaviors, each coupling sensing to action. Higher layers could suppress the outputs of lower ones, but the lower layers continued to run as higher ones were added (Brooks 1986). A fast layer that keeps working while a slow one is busy or wrong is an ancestor of the permission path this book develops. Neither organization, however, made perception or dexterous motion easy to write by hand, and the planners and behavior layers alike stalled on exactly the skills a toddler has. That difficulty has a name. **Moravec's paradox**[1] observes that sensorimotor skills people perform with little conscious effort can be difficult to reproduce computationally, which is why learning became attractive for physical tasks.

↪ The sense-plan-act paradigm, and the failure taxonomy it organizes, is developed in Appendix D.

Learning had already entered the loop. In 1989 ALVINN, a small neural network, mapped camera and laser range images to the direction a vehicle should steer, and followed real roads under some field conditions (Pomerleau 1989). Through the 1990s robotics also made uncertainty explicit. Probabilistic robotics represents what a machine believes about itself and its surroundings as probability distributions, updated as actions and measurements arrive (Thrun et al. 2005). The DARPA Grand Challenge tested these ideas at scale. In the first race, in 2004, no vehicle covered more than a small fraction of the desert course. In 2005 five vehicles finished, and the winner, Stanford's Stanley, relied predominantly on machine learning and probabilistic reasoning (Thrun et al. 2006). In systems of this kind, learning mostly estimated what the world was like, and hand-written planners and controllers decided what to do. A decade later, the DARPA Robotics Challenge Finals asked robots to work at a simulated disaster site, driving a vehicle, opening a door, turning a valve, and climbing stairs. The organizers deliberately degraded the communications link to the remote human operators, so that each robot had to carry out parts of every task under supervised autonomy rather than direct teleoperation (Krotkov et al. 2017). Even at that stage, a person remained close to the loop.

1 | **Moravec's Paradox**: Moravec (1988) observed that computers could be made to perform at an adult level on intelligence tests or at checkers far more easily than they could be given a one-year-old's perception and mobility, and he traced the gap to evolutionary age, since sensorimotor skill was refined over a far longer history than abstract reasoning. For a physical machine the hard part is therefore the part that touches the causal boundary, perceiving a changing world and acting on it before a deadline, which is where learning and this book both concentrate.

Deep learning then changed how much a machine could perceive. In 2012 a deep convolutional network trained on GPUs classified ImageNet photographs with error well below the previous state of the art (Krizhevsky et al. 2012), and learned detectors spread into the perception systems of robots and automated vehicles. The architecture around them usually stayed modular, with a learned percept feeding a planner written as rules or as an optimization. The learned outputs were still predictions, yet no person stood between them and the brakes. The Tempe vehicle examined in section 1.8 was a machine of this kind.

↪ ALVINN's missing recovery examples, and the compounding error they expose, are developed in section 5.3.

Learning reached the actuator itself in the following years. Levine and colleagues trained convolutional networks that mapped raw camera images directly to the torques at a robot's motors, for tasks such as screwing a cap onto a bottle (Levine et al. 2016). Grasping systems learned from hundreds of thousands of their own attempts, gathered by groups of robot arms working in parallel (Levine et al. 2018; Kalashnikov et al. 2018). Deep reinforcement learning (training a network by trial, error, and a numerical reward) in simulation produced controllers for a legged robot and a dexterous hand that transferred to hardware (Hwangbo et al. 2019; OpenAI et al. 2019). With that step the data changed character. A robot that learns from its own attempts chooses the states it will see, so its training data depends on its own behavior.

Imitation learning, which trains a policy to reproduce recorded demonstrations, made human demonstration a practical route to new manipulation skills, and two ideas shaped the policies now in use. Action chunking predicts a short sequence of future actions from each observation, a design its authors motivated partly by the way a cloned policy's errors compound over time (Zhao et al. 2023). Diffusion policies generate action sequences by iterative denoising, which lets one policy represent the several different ways a demonstrator might complete the same task (Chi et al. 2025). The learned output had become a short plan of future motion, executed by controllers beneath it.

The latest step changed where learned proposals come from. In digital software systems, foundation models matured as agentic reasoning engines capable of decomposing abstract goals, generating plans, and invoking external tools. Bringing that agentic reasoning into physical embodiment required grounding multimodal representations in spatial geometry, continuous dynamics, and motor coordination. Large multitask datasets collected on many robots prepared the ground (Brohan,

Brown, Carbajal, Chebotar, Dabis, et al. 2023; Open X-Embodiment Collaboration et al. 2024), and between 2023 and 2025 several groups showed that a model pretrained on web-scale images and text can be fine-tuned to emit robot actions directly. These **vision-language-action (VLA) models** share one mechanism. A pretrained vision-language backbone encodes the camera images and an instruction, and an action head converts the backbone's output into motor commands. The head either decodes actions as discrete tokens, the way a language model decodes words (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024), or produces a continuous *action chunk*, a short sequence of future setpoints from one inference (Black et al. 2024). A backbone with billions of parameters cannot run at a controller's rate, so recent designs split the work across rates. A slow model interprets the scene, and a fast action model or on-robot decoder emits chunks that controllers execute (Bjorck et al. 2025; Gemini Robotics Team 2025b). These systems are dated instances of one step, in which a pretrained general-purpose backbone, rather than a policy trained for one task on one robot, proposes the motion. The sizes and rates their authors report are collected later in table 3.2. The running machine adopts the same hierarchy with two proposers: an *intent model*, the slow vision-language model that reads the scene and chooses the goal, and a *chunk policy*, the fast learned policy that emits action chunks toward that goal. Their rates are fixed on its machine card (section 1.4.2). Because a general-purpose learned model can now propose motion for a general-purpose machine, deciding which component may *permit* that motion becomes a design question for every machine that adopts one.

↪ The memory-bound cost of splitting the work between a slow intent model and a fast chunk policy on the running machine is developed in section 3.5.2.

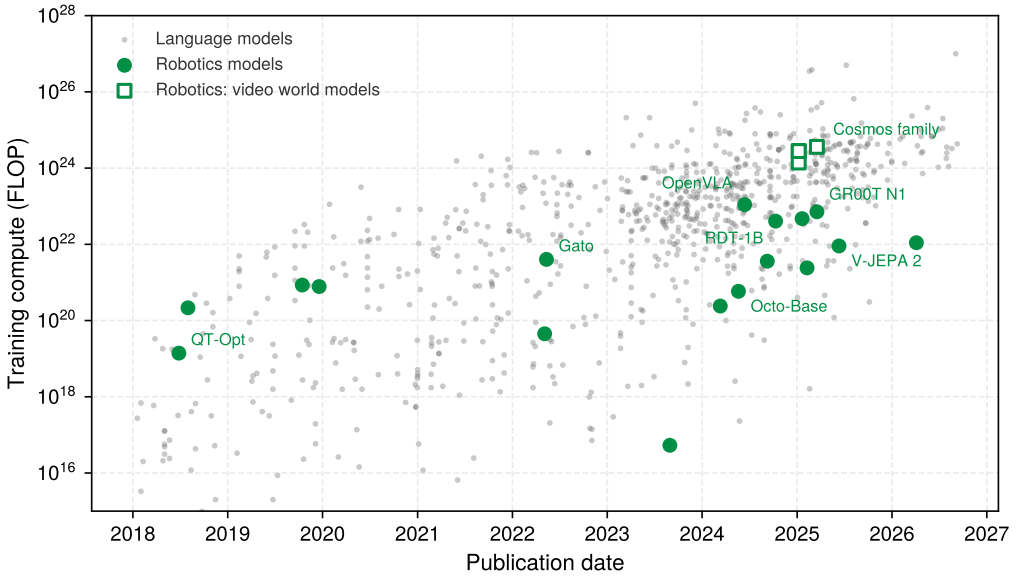


Figure 1.2: Robot models train on a sliver of frontier compute: Estimated training compute, on a logarithmic axis, of the models the Epoch AI database tags as language models (gray) and as robotics models (green), from 2018. Robotics models that generate video of the world rather than actions are drawn as open squares and labeled by family. The open robot action policies sit orders of magnitude below the largest language models of the same year; on this record, the scaling that drove language models has not been applied to learned action. Compute values are Epoch's estimates (Epoch AI 2026).

The scale of that transfer from web-scale pretraining is easy to overstate. In the Epoch AI database, the open robot action policies trained on hundreds to thousands of times less compute than the largest language models of their year (figure 1.2) (Epoch AI 2026). The gap is about 350 times less for a 2024 vision-language-action model and about 7,000 times less for a 2025 open robot foundation model, and only 25 robotics models in the database carry a compute estimate at all. Physical AI is therefore not a smaller copy of the language-model scaling story. Its learned proposers are bounded by what can be collected from machines acting in the world and by what can run on the machine itself, two constraints that Parts I and II of this book treat as first-class.

The name *physical AI* is recent, and industry usage since 2024 applies it broadly to almost any AI that perceives and acts in the physical world (NVIDIA 2026c). The history above runs through

two research traditions that cover much of the same ground. **Embodied AI** descends from Brooks's argument that intelligent behavior arises from a body situated in and interacting with its environment rather than from abstract representation alone (Brooks 1991). **Cyber-physical systems** name the integration of computation and networking with physical processes (Lee 2008), and they carry the timing and verification discipline of embedded engineering (Lee and Seshia 2017). Embodied AI supplies much of the learning this book uses, and cyber-physical systems supply much of its timing and verification practice. The book's subject is where the two meet at an actuator, and the definition given below is deliberately narrower than industry usage, narrow enough that the scope test in section 1.6 can apply it to a given machine.

Read as one story, this history moves the learned model step by step toward the actuator, as the band beneath the time axis of figure 1.1 records. At first nothing was learned. Then learned percepts fed hand-written planners, learned policies emitted motor commands, and finally pretrained general-purpose models began to propose motion for many kinds of machine. At each step, a guarantee that separation or human inspection used to supply had to move inside the machine. The familiar concerns of machine learning systems engineering come along, and each acquires physical stakes. Data stops being a fixed corpus because the machine's own actions choose what it will observe next. Compute stops being a matter of throughput and cost alone because a late answer is paid for in stopping distance. Evaluation stops being a score on held-out examples because the competence that matters appears only in the closed loop. Deployment stops being a routine version rollout because a release grants a machine authority over motion near people. When a learned output becomes an irreversible action, none of these concerns can be settled inside the model. They belong to the whole machine, and they meet at the boundary where its commands become physical work.

1.3 The Question This Book Answers

That convergence is why the problem now faces the whole field rather than a few robotics specialists. The central question of this book is why a better prediction can still produce a worse physical action, and what the rest of the machine must supply so that a learned proposal becomes useful work within justified limits. The book's answer is an argument in four linked steps. The body's physical limits bound what any proposal may ask for. The learned proposer's competence holds only over the states that its data and closed-loop evaluation covered. For both reasons, an independent permission path, rather than the model itself, must decide what may execute at each tick. The evidence gathered about the whole machine then bounds the operating conditions under which that permission is granted. The argument first requires a shared vocabulary. That vocabulary covers the causal boundary and the cost of delay at it, the scope of physical AI, a five-level model of the machine that separates proposal from permission, and the four laws that organize the rest of the book.

One machine carries the argument through every chapter. A warehouse mobile manipulator must collect an object from a conveyor, open a spring-latched cage door, and carry its load through an aisle shared with people. These tasks join abilities often studied separately. The machine must recognize an unfamiliar object, choose a grasp, coordinate its arm and base, and respond when contact or the scene differs from expectation. Learning makes varied behavior possible; engineering must turn that behavior into useful, repeatable physical work.

This book develops the ability to design, teach, integrate, evaluate, and maintain such a machine within justified operating limits. Its recurring warehouse machine is an illustrative engineering design, not a report of a qualified deployment. The same task connects physical budgets, learning experiments, runtime decisions, and evidence for service. When the evidence is incomplete, the engineer must identify what measurement, new experience, or design change would support the next capability.

Consider the base approaching the end of a storage rack while its model processes a camera image. A worker steps into the aisle before the next proposal returns. The computer can still discard that proposal, but the robot has continued moving throughout the calculation, so choosing an action requires reasoning about both the prediction and the physical state in which it will execute. Three things must therefore hold together: the model must recognize a useful action, the execution mechanism must judge it while the observation remains relevant, and the body must be able to deliver the admitted motion or a feasible alternative. A stronger prediction alone cannot recover

clearance already consumed during computation, which is why the engineering problem leads from physical limits to the architecture that checks each proposed action.

Building such a machine requires contributions from four disciplines that normally work apart. Machine learning supplies perception and action proposals. Computer systems determine how quickly those proposals compute and move across memory (Williams et al. 2009). Mechanics and control establish which motions the plant can physically sustain. Embedded systems enforce the real-time scheduling and bus rules that connect a proposal to an actuator. All four responsibilities meet at a single runtime question: whether the proposed action may execute under the machine's current conditions. This book develops the reasoning needed to make and justify that decision.

DRAFT

Definition 1.1: Physical artificial intelligence

Physical artificial intelligence is the engineering of closed-loop machines in which learned models influence, without synchronous human approval, physical actions that change subsequent observations.

1. **Significance**: Model performance becomes a property of an operating machine. Its actions must fit the plant's physical limits and the time available to respond.
2. **Distinction**: The scope test later in this chapter combines a learned model, consequential physical feedback, and delegated authority. Neither a model used only for advice nor an automated machine without a learned component meets all three criteria.
3. **Common pitfall**: Treating the body as a peripheral device for a model. A useful proposal still needs an independent permission decision based on the current state, physical limits, and available fallback.

The argument starts where the definition does, with the difference between a proposed action and its physical effect. A short stopping calculation then puts a number on what delay costs at the causal boundary. A documented collision later in the chapter shows these responsibilities failing together, and four laws then distill them.

1.4 Defining the Causal Boundary

That difference between a proposed action and its physical effect separates advisory computing from physical AI. An advisory model that renders an inference on a dashboard or returns a JSON payload across a network has no direct electrical or mechanical coupling to physical actuators: an operator or downstream software filter can inspect, validate, or discard its output before any physical energy is expended. Digital errors are not benign (an operator who acts on flawed guidance can cause serious damage), but advisory software cannot directly alter physical state. Once an automated system acquires the delegated authority of definition 1.1 to turn a computed proposal directly into physical motion, safety arbitration cannot remain an external afterthought. It must live inside the machine architecture, at the causal boundary between digital intent and physical consequence.

1.4.1 From a proposed action to motion

In modern computational frameworks, executing `action_chunk = policy(observation)` produces a floating-point tensor. That tensor represents an uncommitted proposal; converting it into physical motion requires an entire chain of intermediate controllers, bus interfaces, and power electronics. The causal boundary is the exact interface through which an admitted digital command begins driving physical change in the plant.

A learned policy typically proposes a trajectory of future joint positions, target velocities, or motor torques. Those numerical arrays remain purely informational descriptions until a low-level controller validates, accepts, and begins executing them. The internal representation and predictive horizon of such action sequences are developed in section 3.5.1; here the systems distinction is between proposing an action and authorizing its irreversible consequences.

That authorization is governed by the permission path, an independent mechanism defined in section 1.7 that alone holds the authority to pass commands to motor drives. The permission path verifies whether a proposed trajectory satisfies physical kinematic constraints, obeys velocity and acceleration bounds, and can complete safely within current time allowances. Once admitted, trajectory setpoints feed into dedicated motor controllers. A position loop compares each setpoint with the measured joint position and computes a correction, and a current controller sets the motor current that produces the torque the correction needs.

Consider an electric motor driven by pulse-width modulation. A software write updates a compare register in a microcontroller timer, which sets how long each power switch stays on in every switching period. The resulting on-off pattern drives the inverter's transistor switches, which turn the DC supply into current in the stator windings (the stationary coils). That current sets up a magnetic field that pulls on the magnets of the rotor (the spinning part) to produce torque, which accelerates the

↪ The bus timing and jitter bounds on command transport from the permission path to the drives are developed in section 4.5.

rotor shaft, transmission gears, and mechanical payload. The progression is asymmetric. Rewriting a planned setpoint in memory requires negligible energy and can be undone before the next clock cycle, but modifying a command that has already energized a winding requires physical work performed against mechanical inertia over finite time. A subsequent command can apply brakes or reverse drive current, but that corrective maneuver requires its own physical distance and must obey strict actuator limits.

Definition 1.2: Causal boundary

Causal boundary is the interface through which a digital command produces physical forces, currents, or flows.

1. **Significance**: A proposal can be rejected before it changes the plant. Once executed, it can alter the environment, dissipate energy, and change subsequent observations.
2. **Distinction**: Restoring a command in memory does not restore the machine's prior physical state. Recovery requires another physical action, with its own limits and consequences.
3. **Common pitfall**: Assuming that canceling a software task stops the motion it initiated. The machine needs a feasible stopping or holding response even when its proposer stops computing.

Offline supervised learning evaluates models against static datasets where predictions never influence future test inputs. An embodied robot cannot rely on this decoupling. When the mobile base steers its chassis, its onboard cameras immediately capture a different visual field. When its manipulator nudges a carton, the object's spatial coordinates and contact forces change. When the vehicle cruises along an aisle, its closing distance to personnel shifts in real time. The closed-loop feedback introduced in section 1.2 applies directly to the learned model, because an acting machine reshapes the very environment that produces its future training and inference inputs.

In any embodied system, the environment evolves through closed-loop dynamics: the subsequent physical state s_{t+1} and incoming sensory observation $\mathbf{o}_{t+1}$ depend directly on the action $\mathbf{a}_t$ issued by the policy inference:

$$s_{t+1} = f(s_t, \mathbf{a}_t), \quad \mathbf{o}_{t+1} \sim p(\mathbf{o} \mid s_{t+1})$$

Here f represents the physical plant and environmental dynamics (formulated without process noise for this introductory overview), and p denotes the observation model transforming underlying physical states into discrete sensor measurements. Because each state transition feeds the next observation, a flawed action proposal can steer the machine into configurations its training data never covered. In those out-of-distribution states the model's outputs degrade further, and the machine departs faster from its nominal trajectory. Advisory output stops at an informational interface. In physical AI the loop closes through the permission path, the motor actuators, and the plant, so the model's own actions irreversibly reshape the observations $\mathbf{o}_t$ it receives next (figure 1.3).[2] Imitation learning formalizes this compounding divergence as **covariate shift**. Whereas independent errors in open-loop prediction simply add up across the task horizon S (the number of control steps the task lasts), a single mistake in closed-loop execution can push the machine into an unfamiliar state that causes elevated error rates across every remaining step of the rollout, so the expected cost of mistakes can grow quadratically with the horizon, $\mathcal{O}(S^2\epsilon)$, rather than linearly (Ross et al. 2011). Section 5.2 develops the mechanics of this endogenous covariate shift. Contractive feedback controllers (stabilizing low-level tracking loops that pull perturbed states back toward the reference trajectory) or well-damped passive mechanics can absorb minor trajectory errors, although verifying behavior over long horizons still requires closed-loop evaluation rather than static offline benchmark scoring.

1.4.2 The running machine

To trace how a proposed action crosses the causal boundary on the warehouse mobile manipulator of section 1.3, the book needs one set of numbers that every later chapter can share. With them, the

2 **Recovery Data (ALVINN)**: The neural network that steered Carnegie Mellon's Navlab autonomous van directly from camera imagery confronted this challenge in early trials: because human test drivers kept the van centered in the driving lane, training logs contained no examples of recovering from the road shoulder, causing the system to veer off whenever it drifted off center. To synthesize recovery demonstrations without hazardous trials, Pomerleau (1991) transformed recorded road images using synthetic lateral shifts and calculated corresponding steering corrections back toward the lane center. While synthetic shifts work for simple geometric offsets, a physical policy must still be trained over the states its own actions visit.

↪ Corrective aggregation and the intervention data that defend a policy against covariate shift are developed in section 5.8.

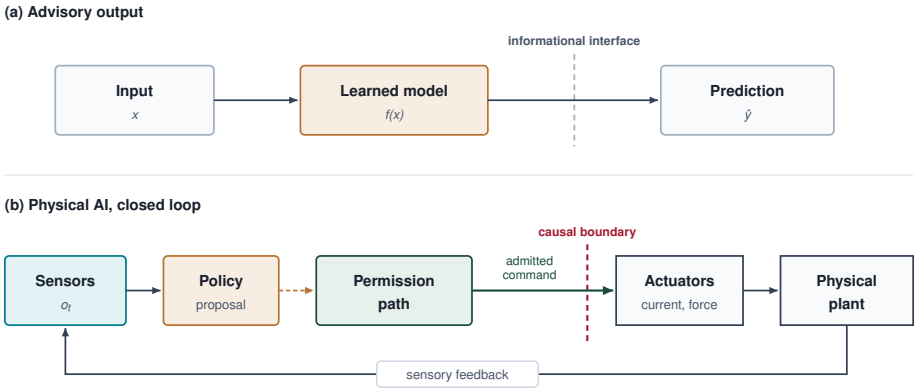


Figure 1.3: The closed causal loop: Advisory output (top) stops at an informational interface. In physical AI (bottom), permitted model proposals reach hardware registers, where electrical current produces force and motion. Those actions alter the environment and subsequent sensor observations.

stop chapter 2 derives, the timing chapter 4 budgets, and the evidence Part IV demands all describe the same robot. Table 1.2 fixes the fifteen parameters of the machine that later derivations use most, each with the evidence label the reader guide defines (measured, derived, datasheet, or illustrative, or chosen for a design decision). The stopping budget that follows uses five of them, the drive limit, the credible deceleration, the clear distance, the pre-brake delay, and the chunk lease inside that delay. This chapter needs only those five and, in one pitfall, the in-loop takeover time. The other rows use ideas (the jerk-limited stop, the intent model's prefill and decode, human takeover studies) that the section named in each row's Basis column develops. They appear here so that every chapter reads the same values. Subsequent worked examples cite the values by parameter name. For complete hardware specifications, the machine data sheet in section C.14.1 catalogs every other input alongside its programmatic registry key.

Table 1.2: The running machine's card: The fifteen parameters of the warehouse mobile manipulator that later calculations use most. A derived value built on illustrative inputs is illustrative evidence. Values print from the MLSysIM registry when the book is built.

Parameter	Value	Category	Basis
Loaded mass m	368 kg	Derived	Base, arm, and a full tote rack; an upper bound (section 2.1)
Drive limit $v_{\max}$	1.5 m/s	Illustrative	Class profile of a warehouse base
Aisle speed v_{aisle}	1.3 m/s	Chosen	Set below the dry-floor speed ceiling (section 12.3.1)
Credible deceleration a_{brake}	2 m/s^2	Illustrative	Loaded, on a dry floor whose grip supplies a friction coefficient $\mu \geq 0.204$ if all the weight bears on the driven wheels, or 0.250 at the share the drive motors allow; tip-over is unchecked until the center of gravity is measured (section 2.3.1)
Stop jerk bound $j_{\max}$	7.11 m/s^3	Derived	Stipulated for the jerk-limited stop, whose deceleration bound is $a_{\max} = a_{\text{brake}}$ (section 11.5.2)
Rack-end clear distance D_{clear}	1.10 m	Illustrative	Clearance the stop must fit inside
Pre-brake delay τ_{delay}	133.6 ms	Derived	Observation age, chunk lease, tick, bus cycle, and brake onset (section 8.4.1); excludes the dispatch-to-admission interval, which the spare pays for (section 13.6.2)
Permission tick T_{tick}	1 ms	Derived	Period of the chosen permission-loop rate (section 4.3.1)
Chunk lease T_{lease}	60 ms	Chosen	Longest an admitted chunk may drive without renewal (section 4.4)
Setpoint step Δt	25 ms	Chosen	Spacing of the chunk policy's setpoints (section 3.5.1)

Parameter	Value	Category	Basis
Chunk replan period	50 ms	Derived	A whole number of setpoint steps
Intent-model latency	336 ms	Derived	Prefill plus decode on the application processor (section 3.4.1)
Intent-model rate	2 Hz	Chosen	Largest rate whose period covers the latency
In-loop takeover $T_{\Sigma,1}$	605 ms	Derived	Slow-bound human reaction plus console video and handshake (section 14.2.2)
Out-of-loop takeover $T_{\Sigma,2}$	4.47 s	Derived	Take-over time of the 90th-percentile study condition plus video and handshake

With these parameters fixed, every later design problem on this platform starts from the same physical foundation. It asks which margin a commanded action approaches and whether the machine can execute an assured protective response before that margin is exhausted. The first margin to price is stopping clearance, and four rows of the card, the drive limit, the credible deceleration, the clear distance, and the pre-brake delay, are enough to price it.

1.4.3 Delay consumes stopping clearance

When the loaded warehouse base cruises down an aisle at speed v and a worker steps out from behind a rack, stopping requires clearance across two distinct phases. The first is the distance the base travels before braking forces engage. The second is the distance it travels while mechanical braking brings its momentum to zero. Assuming steady initial velocity v, uniform deceleration a_{brake} once brakes engage, and a stationary obstacle, the elementary stopping distance satisfies the kinematic budget:

$$d_{\text{stop}} = \underbrace{v\tau_{\text{delay}}}_{\text{travel during response delay}} + \underbrace{\frac{v^2}{2a_{\text{brake}}}}_{\text{mechanical dissipation}} \leq D_{\text{clear}} \tag{1.1}$$

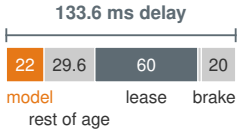
Here v denotes initial forward speed, τ_{delay} represents total pre-brake response latency, a_{brake} is the sustained braking deceleration, and D_{clear} is the usable line-of-sight clearance. The pre-brake delay τ_{delay} spans the time from when an obstacle first appears in a sensor exposure to the physical onset of retarding force. Specifically, it sums the age of the sensor observation upon handoff and the unexpired lease duration of any currently executing motion chunk (the lease is the longest an admitted chunk may drive before the permission path must act again). It also adds one discrete permission-path evaluation tick (the period at which the permission path re-checks the motion it has admitted), one bus communication cycle (the time to carry a command to the motor drives), and mechanical brake onset latency.

The lease belongs in the worst case even though the permission path re-checks every tick, because of what that check can see. Each tick tests the admitted chunk against limits the permission path already holds (the chunk's freshness, its lease, and the kinematic bounds, measured against the wheel and joint encoders). The permission path has no camera pipeline of its own. On the detection path this budget models, evidence that a worker has stepped out therefore reaches the brakes only through the learned perception path and a replacement proposal. If the proposer stalls or fails to answer the worker, the most recently admitted chunk keeps driving until its lease expires, and only then does the permission path command the stop. The term deliberately excludes one interval, the dispatch-to-admission interval (the time while the learned policy computes a replacement proposal and the permission path admits it). Its length depends on where the proposer and the permission path run and how they communicate, which are decisions this chapter has not yet made. Section 13.6.2 therefore allocates it, and the finished budget pays for it from the spare this elementary budget leaves.

The budget has two terms that grow differently with speed: travel during the delay grows linearly ($v\tau_{\text{delay}}$), and travel while shedding kinetic energy grows with the square of speed ($v^2/(2a_{\text{brake}})$). Equating them gives the crossover speed $v^* = 2a_{\text{brake}}\tau_{\text{delay}}$. Below v^*, delay travel is the larger term; above it, braking travel dominates. Shortening sensing, computation, and signaling reduces the delay term, while lowering speed reduces both terms: halving speed halves the delay travel and

quarters the braking travel. Which change saves more clearance depends on the reductions available, not on term dominance alone. Solving equation 1.1 for v gives the fastest speed whose stop fits inside a verified clear distance, a ceiling section 2.3.2 derives with the allowances this elementary form omits.

Every input to the budget comes from the running machine. Its speed, deceleration, clear distance, and pre-brake delay are rows of the machine card (table 1.2), and the five terms of that delay come from the same MLSysIM registry, so the notebook's givens are the ones every later chapter refines.



The learned backbone is 22 ms of the 133.6 ms delay; the lease costs more.

Napkin Math 1.1: Does the base stop inside its clearance?

Problem: The loaded base cruises toward a rack end at its drive limit, and a worker may step out at the end of the rack. The permission path must stop the base inside the rack-end clear distance on the machine card. Does the elementary stopping budget of equation 1.1 fit, which term dominates, and how much of the delay belongs to the learned model?

Variables:

- Speed $v = 1.5$ m/s, the drive limit (illustrative).
- Credible deceleration $a_{\text{brake}} = 2$ m/s², loaded on a dry floor (illustrative).
- Clear distance $D_{\text{clear}} = 1.10$ m (illustrative).
- Pre-brake delay $\tau_{\text{delay}} = 133.6$ ms (derived). It is the sum of five registry terms. The first is the observation age of the camera frame that first shows the worker, 51.6 ms, which includes the 22 ms perception backbone (the rest is half the exposure, sensor readout, transfer, image processing, and the handoff to the proposer, itemized in section 8.4.1). The second is the chunk lease of 60 ms, the longest an admitted command may drive before the permission path must act. The remaining terms are one permission tick of 1 ms, one bus cycle of 1 ms, and a brake onset of 20 ms. It excludes the dispatch-to-admission interval, for the reason given with equation 1.1.

Math:

$$\tau_{\text{delay}} = 51.6 + 60 + 1 + 1 + 20 = 133.6 \text{ ms}$$

$$v\tau_{\text{delay}} = 1.5 \text{ m/s} \times 0.1336 \text{ s} = 200.4 \text{ mm}$$

$$\frac{v^2}{2a_{\text{brake}}} = \frac{(1.5 \text{ m/s})^2}{2 \times 2 \text{ m/s}^2} = 562.5 \text{ mm}$$

$$d_{\text{stop}} = 200.4 + 562.5 = 762.9 \text{ mm} \leq D_{\text{clear}} = 1100 \text{ mm}, \quad D_{\text{clear}} - d_{\text{stop}} = 337.1 \text{ mm}$$

$$v^* = 2a_{\text{brake}}\tau_{\text{delay}} = 2 \times 2 \text{ m/s}^2 \times 0.1336 \text{ s} \approx 0.53 \text{ m/s}$$

Result: The elementary budget fits, with 337.1 mm to spare. The drive limit sits above the crossover speed, so braking dominates: it needs 2.8× the travel of the delay, and the delay adds 35.6 percent to the braking distance. At half speed, 0.75 m/s, delay travel halves to 100.2 mm while braking quarters to 140.6 mm, for a stop of 240.8 mm.

Systems insight: The learned model's backbone accounts for 22 ms of the delay, or 33 mm of travel; the chunk lease alone costs 90 mm. Each added millisecond of delay anywhere in the chain costs 1.5 mm at this speed, so a slower but more accurate model is a distance cost the budget must absorb. The spare is not yet a safety margin. Section 12.3.1 adds the localization bound, a protective margin, tracking error, and the longer stop of a jerk-limited brake (one that limits jerk, the rate of change of deceleration, by ramping its braking up instead of stepping it). Together they exceed the clear distance at the drive limit, which is why the card sets the aisle speed at 1.3 m/s.

The running machine's drive limit lies well above the crossover speed the notebook computed (figure 1.4), so braking, not delay, sets most of its stop, and the sum still clears the dashed clear-distance line before any later allowance is added. These values are representative parameters and a

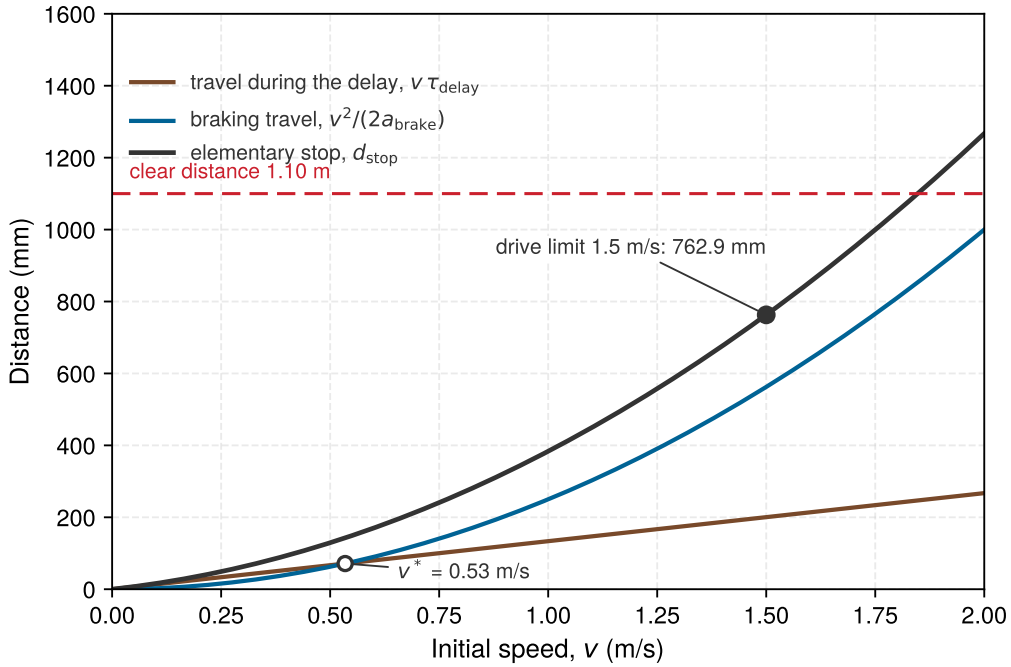


Figure 1.4: Delay travel and braking travel against speed: The elementary stopping budget of the loaded base, split into travel during the 133.6 ms pre-brake delay, which grows linearly with speed, and braking travel at 2 m/s², which grows with its square. The two terms are equal at the crossover speed v^*; above it braking dominates. At the 1.5 m/s drive limit the elementary stop fits inside the 1.10 m clear distance, before the allowances that later chapters add.

derived latency profile, not measured stopping distances for a real chassis, but they already show the two levers a designer holds. Shortening response latency contracts the linear term, whereas lowering cruising speed contracts both, the braking term quadratically. Selecting a model purely on offline accuracy overlooks the first lever, because added inference latency is added travel.

The budget just computed models one particular detection path, and naming it shows which learned computation it charges. The worker is seen by the learned perception stack, so the perception backbone sits inside the observation age. The chunk policy's own compute time does not appear, because the lease, not the policy, bounds how long an admitted chunk may keep driving once the proposer falls silent. A machine can also carry a second, independent protective path with its own sensing, timing, power, and fallback logic, such as a certified safety scanner wired to the drives. A stop that path triggers bypasses perception dispatch and the lease entirely. The backbone and lease terms leave its delay, and its own sensing and switching latency take their place, while the braking travel and the allowances stay. Under that decoupled architecture, the protective path's latency, rather than the execution time of the neural network, governs the protective stop. Industrial automated guided vehicles (AGVs) implement this separation today, with safety standards mandating certified safety scanners that halt the machine before human contact, independent of fleet management or guidance computers (International Organization for Standardization 2023b). This book layers learned sensorimotor proposers atop deterministic safety kernels and analyzes the architectural contracts required between them.

↪ The scanner path's own timing budget, and how the software stopping budget maps onto certified safety functions, are developed in section 12.9.3.

Deploying this budget in real-world facilities requires several additional physical allowances. Beyond the allowances the notebook's systems insight names (localization, a protective margin, tracking error, and the longer jerk-limited stop), real-world controllers must also account for unmodeled initial acceleration and degraded tire-floor traction. Dynamic obstacles complicate the clearance budget even further: if a pedestrian walks toward the mobile robot at speed v_h, the relative closing distance contracts during both the pre-brake delay and the mechanical deceleration phase. For this reason, functional safety standards mandate adding a pedestrian approach-speed allowance

to the vehicle's own braking distance (International Organization for Standardization 2024a). Later chapters formalize these dynamic constraints and allocate delay budgets across compute hardware. Even before they do, a learned policy's behavior is acceptable only inside the real-time envelope that plant momentum sets.

1.4.4 Irreversibility and recovery

The stopping budget prices one consequence of irreversibility. The other is what recovery becomes. A database transaction that aborts restores the state it started from, but a gripper that has permanently deformed a workholding fixture, or a winding whose insulation has degraded from overheating, has no earlier state to return to. Recovery on a physical machine is therefore a new trajectory, not a reversal.

Corrective commands can brake the motors, release a grasp, or plan a diversion, but each is itself motion, limited by the torque the actuators can deliver, by friction, and by the speeds and accelerations the drives can reach. Which limit governs depends on what the machine is doing. For the mobile base it is stopping clearance under momentum. When the arm closes its jaws on a fragile component or braces against an assembly fixture, it is contact force, the flex of the gears and belts between motor and gripper (transmission compliance), and heat in the windings. Because recovery is constrained motion, the permission path must check before it admits an action that a feasible recovery from the resulting state will still exist; checking afterward is too late.

1.5 Three Machine Classes

↪ The contact loads a transmission delivers to a gripped part or a latch, the arm's limiting quantity here, are developed in section 2.4.4.

As the warehouse mobile manipulator travels down an aisle, its primary engineering challenge is kinetic: preserving enough clearance to stop safely before an obstacle. When the same robot presses against a door latch, the governing constraint shifts to contact mechanics, regulating interaction forces to avoid damaging the mechanism or the workpiece. When holding a suspended payload stationary, the machine performs zero mechanical work, yet its actuators must sustain continuous stall torque without exhausting thermal headroom or browning out the shared electrical supply. Because each phase of the task spends a different physical margin, the book names three machine classes by the margin each one spends (figure 1.5).

The first class, **mass-dominated mobility** (Class 1), spends stopping clearance. A mobile base, delivery vehicle, or aerial drone carries significant momentum and cannot halt instantaneously. Forward velocity, control and sensing latency, and available braking deceleration or reverse thrust jointly dictate the spatial clearance required to arrest or redirect its motion.

The second class, **contact-dominated manipulation** (Class 2), spends force and deformation margin. When an articulated arm presses against a stiff fixture, minute position displacements can generate destructive contact forces over fractions of a millimeter. Physical contact is both the means of task success (seating a part or opening a latch requires deliberate force) and the immediate hazard. The engineering task is to supply the wrenches (the combined forces and torques) needed to complete the assembly while guaranteeing that contact forces never exceed the mechanical yield limits of the payload or the manipulator.

The third class, **energy- and thermal-dominated operation** (Class 3), spends heat and electrical supply margins. Supporting a heavy load or resisting gravity demands sustained motor current, driving continuous Joule heating in actuator windings even when joints remain stationary. Concurrent peak current draws from drive inverters, compute accelerators, and sensor payloads can induce voltage sag across a shared DC bus. The system must sustain these thermal and electrical loads across the entire operational profile while reserving sufficient energy and thermal headroom to execute a safe protective abort.

These three classes identify dominant physical constraints rather than mutually exclusive hardware categories. A practical robot transitions across all three regimes throughout a single task. The running machine integrates a wheeled base, an articulated arm, navigation lidar, and wrist cameras. An end-to-end learned policy can coordinate simultaneous base trajectories and arm proposals (Fu et al. 2024), but the integrated machine must enforce the safety envelopes of every regime concurrently. A fixed-base arm still faces thermal and bus-voltage limits during prolonged holding, just as a wheeled base encounters contact constraints when docking against a workstation.

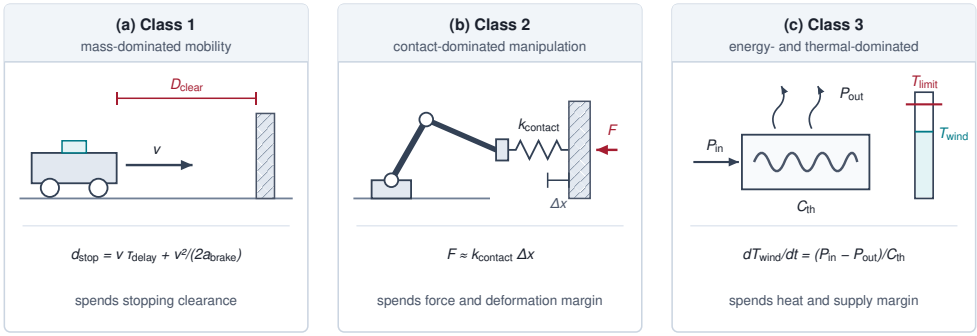


Figure 1.5: Three machine classes: Each class is named for the constraint that dominates it and the margin it spends: (a) mass-dominated mobility (Class 1) spends stopping clearance, set by momentum and braking; (b) contact-dominated manipulation (Class 2) spends force and deformation margin, set by contact stiffness and the strength of the object and mechanism; and (c) energy- and thermal-dominated operation (Class 3) spends heat and supply margin, set by Joule heating, thermal capacitance, and power delivery.

Systems Perspective 1.1: The humanoid crucible

A humanoid robot exposes the tight coupling across all three regimes. Catching a shifted load perturbs dynamic balance, requiring an immediate recovery step that demands peak joint torque. That torque surge draws heavy current from the shared DC bus and injects rapid thermal energy into the joint windings. If the power distribution network sags or thermal protection limits current, the recovery step fails and the machine falls. Shielding the onboard computing stack from reset does not by itself supply the transient mechanical energy needed to stabilize the body. The part-ending humanoid examples throughout this book test how each systems principle survives these tightly coupled physical demands.

↪ The multi-contact balance mechanics a humanoid's recovery step depends on are developed in section B.9.

Identifying whether a task spends stopping clearance, contact force, or thermal headroom tells the engineer which physical budget binds, regardless of how the controller is implemented. Whether the machine also requires the proposal-and-permission architecture of physical AI, rather than a classical feedback loop or an offline tool, depends on a separate test of how its decisions are generated and authorized.

1.6 The Physical AI Scope Test

A robotic arm in a fulfillment center can follow a fixed motion profile under a proportional-integral-derivative (PID) controller (a classical hand-tuned feedback law that corrects in proportion to the present error, its accumulated sum, and its rate of change). It can also execute an offline trajectory pre-screened and loaded by a human technician, or stream joint torques directly from a learned policy running on real-time sensor frames. While all three regimes actuate identical electric motors and gear trains, they impose different architectural obligations on the computing system. The scope test establishes the operational boundaries that define physical AI throughout this book.

The scope of physical AI is defined by the simultaneous intersection of three structural criteria:

1. **Learned Statistical Model:** The system relies on a policy, perception encoder, or world model whose runtime behavior emerges from training data rather than an inspectable closed-form specification, as seen in deep neural networks or diffusion policies (Chi et al. 2025). Correctness cannot be formally proven across all state configurations from source code alone; engineers must establish operational validity empirically across a defined operating domain. By contrast, a linear plant model fitted through classical system identification,[3] where parameter uncertainty yields provable stability margins, falls outside this category. A high-capacity neural policy falls within it.
2. **Consequential Physical Feedback:** The system's actions alter the material state of the surrounding physical world, and those state changes directly govern what its sensors measure in

[3] **System Identification**: The control-engineering practice of fitting low-order dynamic models to measured input-output time series. When this fitting establishes bounded parameter uncertainty under stated modeling assumptions, control engineers can use those bounds to prove stability guarantees before deploying the controller. Modern high-dimensional neural networks lack tractable closed-form uncertainty bounds and therefore require empirical verification across explicitly specified operating domains.

subsequent time steps under classical mechanics, thermodynamics, or electromagnetism. The environment cannot be paused or rolled back, and energy dissipated into it cannot be recalled.

3. **Delegated Authority:** The system's outputs reach execution without synchronous human approval for each control cycle. In a machine that also meets the second criterion, that execution is actuation, so the authority is physical. A learned component holds this authority whenever its output (a raw percept, a semantic state estimate, or a proposed motion trajectory) can propagate to live actuators without such approval. The learned network need not write directly to motor driver registers; the system possesses delegated authority whenever learned signals actively shape downstream actuation commands. Adding downstream safety checks or deterministic rate limiters does not vacate this authority, provided accepted proposals still drive hardware motion.

Together, these three criteria delineate the domain of physical AI. The three-set Venn diagram in figure 1.6 formalizes this boundary, locating physical AI at the central intersection where a learned model, consequential physical feedback, and delegated authority converge. Each pairwise overlap holds systems that meet only two of the three criteria. Advisory systems place a learned model inside a physical process, as when a technician inspects learned toolpaths before loading them onto a mill, but a person still approves each action. Digital autonomy executes learned outputs without human approval, as in an automated trading engine, but its actions land in software rather than in physics. Classical control closes a physical feedback loop automatically, as in a PID speed governor, but relies on an analytical control law rather than a learned model. Excluding these pairwise overlaps restricts the design problem to systems where statistical uncertainty collides directly with physical irreversibility.

A learned model alone does not make a machine a physical AI system. Consider an offline trajectory generator that optimizes smooth paths using a learned diffusion model. If a manufacturing engineer must inspect, validate, and load those toolpaths onto a CNC mill prior to cutting metal, the statistical model operates only as an advisory tool. The human retains ultimate physical authority. Similarly, a vision model that highlights defects on an assembly line display assists human inspection without closing the physical loop.

Conversely, delegated authority does not require end-to-end neural control. Most production autonomous systems adopt modular architectures where deep neural networks handle perception while deterministic trajectory planners compute actual actuator setpoints. These systems satisfy the delegated-authority test through perception: if an unvetted object detection or bounding-box estimate triggers an automated swerve or emergency braking maneuver, the learned representation directly dictates plant dynamics. The autonomous test vehicle in the Tempe collision examined in section 1.8 is such a machine: changing classifications in its modular perception fed the path predictions that decided whether it would brake.

Shared autonomy falls within physical AI whenever a learned sub-system modifies live motion before a human operator can intervene. In a robotic surgical console equipped with learned haptic virtual fixtures, for example, the assistive policy exerts real forces on the master tool manipulators to guide the surgeon around delicate anatomical structures. Even if the surgeon holds administrative responsibility for the procedure, high-bandwidth force cues alter physical tissue contact well within normal human neurological reaction latency (typically 150 ms to 250 ms). Because the machine enforces learned spatial guidance dynamically without requiring the surgeon to review and acknowledge each micro-correction, it exercises delegated authority over physical motion.

When classifying an ambiguous machine, engineers must evaluate each of the three criteria independently. Implementing an independent deterministic veto, such as an analog brake relay triggered by a certified safety lidar, does not strip a machine of delegated authority, because learned proposals still govern all nominal motion up to the intervention threshold. In contrast, requiring a human operator to screen and authorize every trajectory command before transmission reliably removes that authority. Once a design satisfies all three criteria, it qualifies as physical AI, and its computing architecture must respect the hard timing and causal boundaries established in section 1.4.

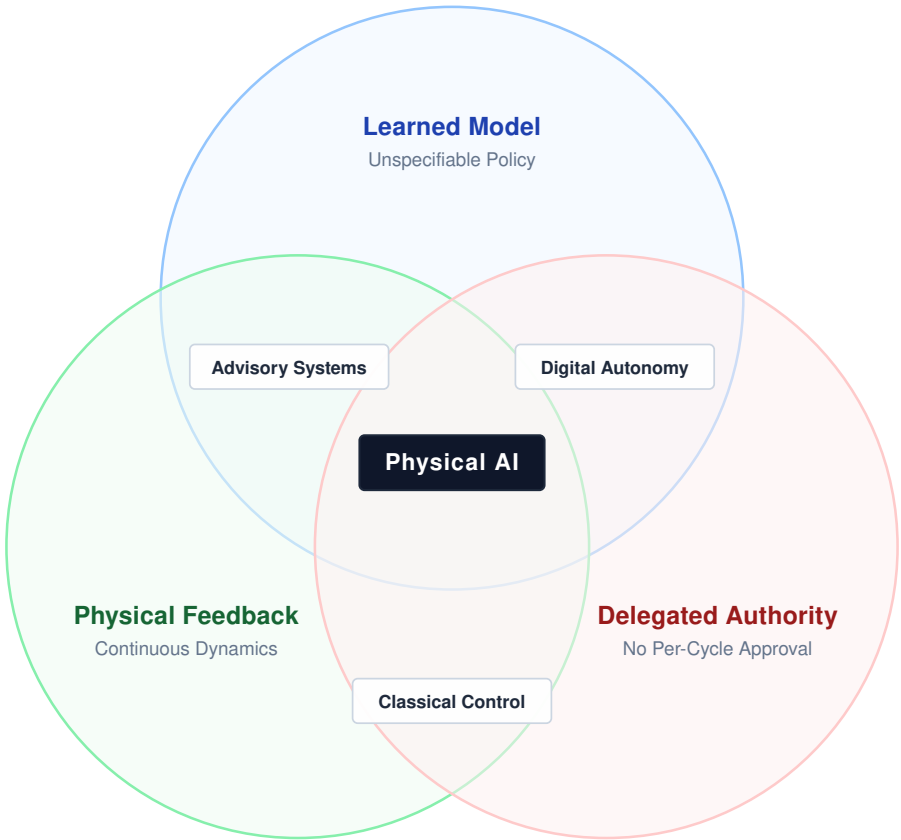


Figure 1.6: The physical AI scope test: Physical AI combines a learned component, consequential physical feedback, and delegated authority to let learned outputs reach actuation without human approval. An independent veto belongs inside this intersection because accepted proposals can still move the plant. The pairwise overlaps locate advisory systems, digital autonomy, and classical control.

Checkpoint 1.1: Binding budgets and the scope test

- ☐ **Delay vs. braking**: Explain why response delay and active braking consume separate portions of stopping clearance, and identify which one dominates above the crossover speed.
- ☐ **Stopping budget estimate**: Using the machine card, estimate how far the elementary stop at the 1.5 m/s drive limit grows if the 60 ms chunk lease doubles, and determine whether it still fits inside the 1.10 m clear distance.
- ☐ **Binding margins**: Identify the relevant physical margin when the warehouse machine travels, presses a latch, and holds a load.
- ☐ **Scope criteria**: Apply all three scope criteria to a new machine, including one whose learned output passes through an independent veto.

Satisfying the scope test binds statistical uncertainty directly to hardware consequences. To design systems that operate safely under these demands, computer systems engineers must map how timing, state estimation, and control authority distribute along the execution path from raw sensor frames to physical actuator motion.

1.7 The Machine in Five Levels

When the running machine's chunk policy stalls mid-aisle, something other than the policy must still be able to stop the base. When the policy emits a chunk that would carry the arm through a coworker's hand, something other than the policy must refuse it. An embodied machine therefore needs an architecture that prevents learned models from writing directly to the actuators they could damage, while still letting those models guide physical behavior.

The Body establishes what motion, force, and mechanical power the hardware can physically sustain. The Brain generates learned proposals intended to operate within those capabilities. Mediating between them, the Nervous System coordinates communication and control, preserving both the temporal margin and the hardware authority required to intercept and refuse an inadmissible command. Adding the interface where commands become current and the environment the machine acts on gives a five-level model that runs from learned computation down to the physical world:

1. **The Brain (Level 1):** The learned models that interpret sensory observations and propose candidate goals, trajectories, or torques. Generating a proposed trajectory or sequence of setpoints confers no intrinsic authority to execute it.
2. **The Nervous System (Level 2):** The deterministic communication buses, real-time schedulers, and control loops that bridge proposals to physical execution. Its permission path evaluates, filters, or rejects proposals against fresh sensory evidence and invariant physical limits, and it executes a fallback shown feasible from the current state under the stated sensing, plant, and response-time assumptions when the proposer falters.
3. **The Body (Level 3):** The physical plant, including motor drives, transmissions, electromagnetic windings, structural linkages, and energy storage, that delivers mechanical action.
4. **The Boundary (Level 4):** The hard physical and electrical interface where digital registers translate into actuator voltages or currents, and where physical phenomena register across sensor transducers as raw measurements.
5. **The World (Level 5):** The external physical environment (surfaces, payloads, obstacles, ambient lighting, and human collaborators), which evolves continuously under the laws of physics regardless of whether the machine measures it.

The list orders the levels by abstraction, from learned computation down to physics, not along the path a command travels. A command leaves the Nervous System, crosses the Boundary at a register write, and only then drives current through the Body's windings. The Boundary is listed after the Body because it gathers the transducers at both edges of the machine, the register writes that energize the windings and the sensor captures that turn light, force, and motion into numbers. Those are the places where the machine exchanges energy and information with the World. The actuation half of this level is the causal boundary defined earlier in the chapter. The proposal boundary, defined below, is not a level at all. It is the line between the Brain and the Nervous System that separates proposing from permitting.

A concrete physical AI implementation often maps this hierarchy across distinct computational tiers. Learned proposers run on high-throughput accelerator hosts (such as an application processor paired with a GPU or neural processing unit). The permission path executes on an independent real-time microcontroller or dedicated real-time operating system (RTOS) core. The current loops that regulate motor-phase current run directly on dedicated motor drives. This architectural isolation is rooted in strict timing guarantees and fault-containment boundaries rather than specific silicon families or nominal operating system labels.

Governance does not constitute a sixth level in this real-time control hierarchy. Instead, it serves as the operational and lifecycle envelope enclosing all five. Governance determines which entity holds valid command authority, what verification evidence licenses the machine to operate, and when that license lapses and must be renewed. Governance transitions unfold across the human and operational timescales of safety interventions, policy rollouts, and maintenance releases, rather than the millisecond rhythms of real-time control loops.

↪ The loaded test of the two-processor split that realizes this isolation is developed in section 13.7.

↪ The authority state machine that hands each actuator channel to exactly one owner during a transfer is developed in section 14.3.

Definition 1.3: Proposal boundary

Proposal boundary is the line between a learned model, which may only propose an action, and the **permission path**, an independent mechanism with bounded latency that the model cannot delay or corrupt and that alone may permit the action to reach an actuator. The permission path validates every proposal against fresh sensory evidence, the stopping envelope (the state space from which the machine can safely decelerate to a halt prior to obstacle collision), and a fallback trajectory shown feasible from the current state. On the running machine, the algorithmic admission checks remain an unrated performance layer (one not certified to a functional-safety rating) above separate certified protective functions. Section 12.9.3 explains that distinction. If any condition is violated, the permission path rejects the proposal within the established response bound.

1. **Significance**: Prevents learned model outputs from directly manipulating actuators. This architectural protection remains dependable only as long as the permission path's independent sensing, dynamic models, and fallback maneuvers remain physically valid.
2. **Distinction**: Even a deep reinforcement learning policy trained to emit joint torques possesses zero raw actuator-write authority; its output remains a mere proposal until explicitly admitted by the permission path.
3. **Common pitfall**: Treating the permission path as a cosmetic post-processing filter or output clamp, rather than an autonomous supervisory mechanism capable of seizing control when the learned proposer hangs, violates deadlines, emits pathological values, or falls silent.

The proposal boundary determines who holds the authority to approve an action, whereas the causal boundary marks the physical interface where an admitted command alters the state of the plant. The Nervous System bridges this divide, verifying proposals against hard constraints before energizing the drives.

Readers steeped in classical control theory will recognize this design as a supervisory architecture. Specifically, the proposal boundary implements runtime assurance. In classic Simplex architectures, a complex, high-performance controller (here, the learned policy) is paired with a certified baseline controller, coordinated by decision logic that determines which subsystem drives the plant (Seto et al. 1998; Sha 2001). The lineage table in section 1.10 (table 1.3) maps this book's terms to their established names once the four laws are stated.

↪ Runtime assurance for physical AI, which pairs the learned proposer with a resident protective controller and an output selector, is developed in section 12.1.

Hardware fault isolation alone does not guarantee safety, because architectural independence also constrains the permission path's sensor inputs. The learned detection path used in the stopping budget prices response delay, but its lease cannot protect against a perception error that produces timely, unsafe proposals. If an enforcement monitor validates a proposed trajectory using the very same learned perception pipeline that synthesized it, a single perception error will corrupt both components simultaneously. The sensing pipelines that grant execution authority must therefore maintain distinct failure modes from the learned models they supervise.

Across the five stacked levels in figure 1.7, characteristic operating rates span four orders of magnitude. The continuous physical World sits at the bottom, followed by the event-timed Boundary, the 10–25 kHz current loops of the mechanical Body, and the 1 kHz permission loop of the Nervous System. At the top, the hierarchical Brain operates from 40 Hz setpoints to 2 Hz intent models. The prominent gap between Brain and Nervous System marks the foundational proposal boundary: components above this line may only formulate proposals, while only deterministic subsystems below it possess the authority to command physical actuators. The operating frequencies charted across the levels reflect the reference architecture analyzed throughout this book, rather than immutable physical constants for every robotic platform. Subsequent chapters derive these timing budgets directly from task dynamics and mechanical plant characteristics. What remains fundamental across all physical AI systems is the proposal boundary that severs learned computation from unchecked execution authority.

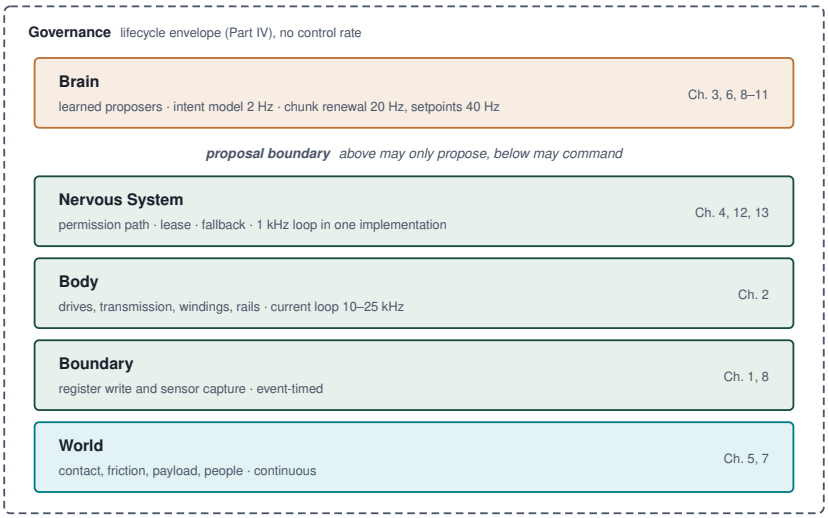


Figure 1.7: The five-level machine: Each band carries its pinned rate and the chapters that work on it. The wider gap between Brain and Nervous System marks the proposal boundary, above which a component may only propose. A chapter-opening stack with no level highlighted marks a governance chapter. The permission-loop rate shown is one implementation.

Deploying capable physical AI requires balancing expressive policy proposals with deterministic execution guarantees. This separation of concerns does not demand rejecting every imperfect proposal out of hand; real-time intervention mechanisms can project a trajectory onto safe constraints or smoothly blend corrections, preserving task progress while respecting physical invariants. Systems evaluation must therefore disentangle what the learned proposer achieves in isolation, how effectively the integrated machine executes under routine supervisory filtering, and whether the fallback mechanisms themselves introduce secondary hazards.

Distributing responsibilities across these five levels gives the rest of the book its structure. It also gives a way to diagnose real failures by asking which level failed, rather than reducing every physical malfunction to a deficiency in neural network weights.

↪ The minimal-intervention optimization filters that project a proposal onto its constraints are developed in section 12.7.

1.8 When the Responsibilities Fail Together

A warehouse model that recognizes an obstacle too late cannot recover the clearance already consumed. In a road vehicle, the same timing problem can combine with tracking errors, delayed intervention, and missing redundancy. The 2018 Tempe collision provides a documented example of these interacting failures; it does not establish that one prescribed hardware architecture would have prevented them. The National Transportation Safety Board (NTSB) attributed the crash primarily to the vehicle operator's distraction. The design choices traced below explain why the automated system had made that single, unmonitored person the last line of defense.

On a March night in 2018, a modified Volvo XC90 operating Uber Advanced Technologies Group's developmental automated driving system (ADS) struck and fatally injured a pedestrian pushing a bicycle across a road in Tempe, Arizona (National Transportation Safety Board 2019).[4] The vehicle carried a roof-mounted lidar, forward and side radars, surround cameras, and GPS antennas, all routed to ADS compute and data-storage racks in the cargo bay (figure 1.8). The ADS first detected the pedestrian 5.6 s before impact, but detection alone did not provide a correct path prediction (National Transportation Safety Board 2019). The failure ran through perception and path prediction, a response rule that withheld braking, and a human operator the design relied on as its last countermeasure.

4 **Reading the Tempe Record**: Times in this section count backward from impact at T, as the event table of NTSB/HAR-19/03 does, and speeds are those the ADS itself recorded (National Transportation Safety Board 2019). The report also cautions that communication delay leaves unclear exactly when the planned slowdown began. Reading each time as an interval before impact, rather than as a clock time, is what lets it be converted into distance traveled at the recorded speed, the conversion this section applies.

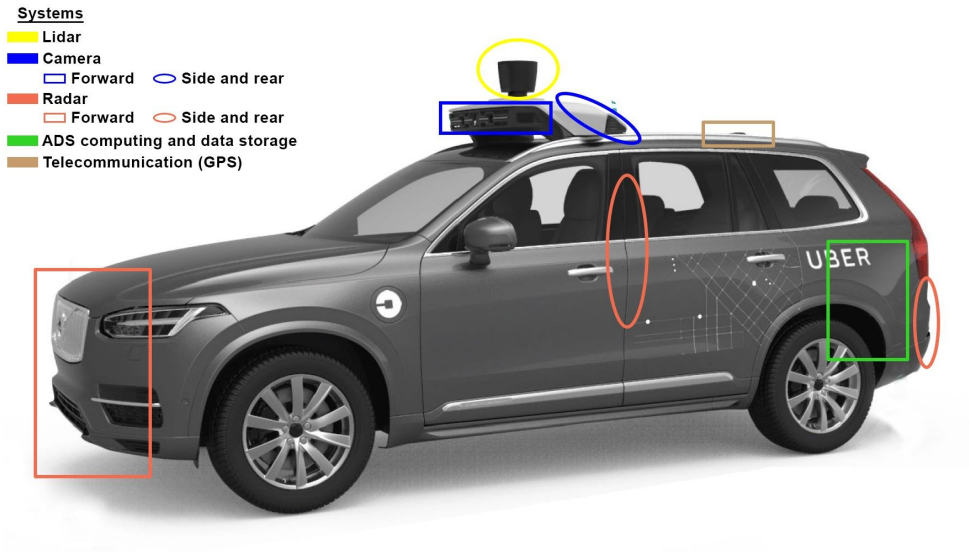


Figure 1.8: Tempe test vehicle sensor locations: The NTSB diagram identifies the developmental ADS vehicle's lidar, camera, radar, computing, and telecommunications locations. (Source: NTSB, public domain).

1.8.1 Tracking history loss under classification churn

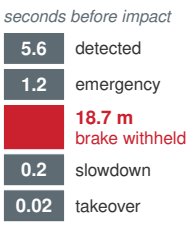
Despite continuing to track the detected object, the ADS did not brake in response to it because its classification kept shifting among vehicle, bicycle, and other before $T-1.2$ s (National Transportation Safety Board 2019). Whenever the classification changed, the ADS path predictor did not use the object's earlier locations, and for an "other" object it could assume a static path.[5] The loss of earlier locations matters because successive positions provide evidence of motion across the roadway; a changing semantic label must not silently discard that evidence. Because each relabeling shortened the track, the resulting path predictions did not place the pedestrian in the SUV's lane until $T-1.2$ s (National Transportation Safety Board 2019).

1.8.2 Action suppression and lost stopping distance

The ADS included an action-suppression interval intended to reduce responses to false alarms while the system checked the situation and allowed operator intervention (National Transportation Safety Board 2019). That software choice directly consumed time available for a physical response: at $T-1.2$ s, with the vehicle traveling at 43.2 mph, the ADS recognized an emergency and entered its one-second action-suppression period with braking withheld. At $T-0.2$ s, suppression ended with the vehicle at 40.5 mph. The ADS then initiated a plan for gradual slowdown and sounded an alert; it did not request emergency braking. The NTSB notes that communication delay leaves unclear whether that slowdown began before the operator disengaged the ADS at $T-0.02$ s (National Transportation Safety Board 2019).

The first term of the warehouse stopping budget applies directly. Averaging the two recorded speeds over the 1 s of suppression, the vehicle traveled about 18.7 m while emergency braking was withheld, distance that no later action could recover. The speed still fell during that second because a slowdown for the planned right turn at the next intersection was already under way; what the suppression withheld was the emergency response (National Transportation Safety Board 2019). The emergency itself was defined by a chosen limit. The ADS treated a situation as an emergency when avoiding a collision required more than 7 m/s² (0.71 g) of deceleration or a jerk[6] beyond its limit, and it capped its own automated braking at that deceleration. Police determined that the vehicle could decelerate at an average of 0.92 g, about 9.0 m/s², on that road surface (National Transportation Safety Board 2019). The difference between a chosen permission limit and a physical one is the distinction the book's numbers keep throughout. The calculation stops short of a counterfactual stop, because the report does not establish when a different design would have begun braking. Detection,

[5] **Tracking-History Loss**: The NTSB record states only that a changed classification caused the predictor to omit the object's earlier locations; it does not describe a Kalman-filter reset or a zero-velocity initialization, so neither should be read into it (National Transportation Safety Board 2019). The consequence follows from the omission alone. Each relabeling shortened the evidence window from which motion is estimated, so an object crossing the lane began again, after each change, as one with no recorded history of crossing.



Recognition came seconds after detection; suppression then spent 18.7 m with the brake withheld.

correct path prediction, hazard recognition, and physical braking are distinct events; reporting the first detection time alone conceals the failures between them.

War Story 1.1: The disabled safety redundancy (2018)

Context: On March 18, 2018, a developmental ADS on a 2017 Volvo XC90 struck and killed a pedestrian crossing a road in Tempe, Arizona (National Transportation Safety Board 2019).

Mechanism: As traced above, classification changes discarded the object's tracking history, and the one-second suppression withheld braking until only a gradual slowdown was planned. The design did not use maximum braking solely to mitigate an unavoidable collision (National Transportation Safety Board 2019).

Impact: The vehicle struck and fatally injured the pedestrian. The late gradual-slowdown plan could not recover the lost stopping opportunity.

Response: The NTSB determined that the probable cause was the vehicle operator's failure to monitor the driving environment and the ADS because she was visually distracted throughout the trip by her personal cell phone. It found three contributing factors at Uber ATG, all a consequence of an inadequate safety culture: inadequate safety risk assessment procedures, ineffective oversight of vehicle operators, and a lack of adequate mechanisms for addressing operators' automation complacency. The impaired pedestrian's crossing outside a crosswalk and the Arizona Department of Transportation's insufficient oversight of automated vehicle testing also contributed. The NTSB further found that deactivating the Volvo collision warning and automatic emergency braking, which Uber ATG had done over possible radar interference, removed a layer of safety redundancy, and it recommended stronger safety risk management for testing automated vehicles (National Transportation Safety Board 2019).

Systems lesson: Classification uncertainty, a delayed response rule, and reliance on an inattentive operator can combine. The design made the operator the primary countermeasure in an emergency, yet no mechanism monitored her attention in real time: authority rested on evidence nobody collected. A design needs timely hazard assessment and a response that can mitigate impact when avoidance is no longer possible; the investigation does not prescribe a particular hardware architecture.

The investigation supports a broader architectural requirement. A developmental ADS needs safety redundancy and a timely mitigation path even when perception or path prediction fails, and simulation tests by Volvo that the NTSB cites suggest that the factory emergency braking might have prevented or mitigated this collision. The report does not establish when a separate permission path would have triggered or where it would have stopped the vehicle.[7]

Tempe illustrates failure *before* impact, where classification changes and a software suppression delay prevented timely braking. The 2023 Cruise robotaxi incident marks the opposite pole *after* impact. A hit-and-run driver's car threw a pedestrian into the path of the driverless vehicle, which braked hard but struck her and stopped. It then began a pullover toward a minimal-risk condition[8] and, because its sensing did not detect a person beneath the vehicle, moved about 20 ft (6.1 m) at up to 7.7 mph while she was pinned beneath it. This account comes from a review commissioned by the company's boards, whose focus was the company's disclosures to regulators (Quinn Emanuel Urquhart and Sullivan, LLP 2024).

Neither failure reduces to an inaccurate model. The first spent stopping distance before braking began, and no later action could recover it. The second chose motion after the impact without fresh evidence about the space it moved through. Together they show what must hold before the machine acts and what a fallback may do once harm has begun. Both also had to be reconstructed afterward from records of what the machine perceived and when, so accountability, like safety, depends on the path that authorizes motion.

6 **Jerk Limit**: Jerk, met in the stopping notebook as the rate of change of deceleration, is measured in m/s^3. A jerk limit makes a brake ramp its deceleration up instead of stepping it, which spares occupants, loads, and transmissions a shock but costs stopping distance, because full deceleration arrives later. The warehouse machine's card carries a jerk bound for the same reason, and the machine's finished stopping budget pays for the ramp.

↪ The stopping envelope, which charges every interval of delayed braking against the clear distance, is developed in section 12.3.

7 **Safety-Critical Standards**: Three standards divide the ground. ISO 26262 addresses hazards caused by faults in a vehicle's electrical and electronic systems (International Organization for Standardization 2018); ISO 21448, safety of the intended functionality (SOTIF), addresses hazards that arise when a function works as designed but performs insufficiently, as perception can (International Organization for Standardization 2022); and UL 4600 sets out the safety case for autonomous products with no human driver to fall back on (UL Standards & Engagement 2023). Tempe falls on the SOTIF side, since the recorded failures were in classification and path prediction rather than in a faulted component, and none of the three prescribes a processor architecture. Section 16.3.4 maps the standards that govern the running machine.

8 **Minimal-Risk Condition**: In automated driving, the stable, usually stopped, state a system brings a vehicle to when a trip cannot or should not continue. Reaching it can itself require motion, as a pullover to the curb does, and in the Cruise case that motion was the harm. A fallback trajectory therefore needs the same permission check as any other proposal, judged against fresh evidence about the space it will move through.

↪ The Cruise pullover, and the fallback rule that forbids renewed motion while the space beneath the vehicle is unverified, are developed in section 12.9.1.

↪ Who answered for the Cruise collision, and the decisions behind an admitted action that no per-tick log records, are developed in section 17.1.

Checkpoint 1.2: Forensics of boundary and authority failures

Verify how the Tempe mechanisms connect to the architecture:

- ☐ **Classification changes and path prediction**: Explain how classification changes caused the ADS path predictor to omit earlier object locations, despite continued tracking.
- ☐ **Software suppression vs. kinematic clearance**: Distinguish the $T - 1.2$ s recognition time from the one-second suppression period, and explain why the gradual-slowdown plan at $T - 0.2$ s could not restore the time already spent.
- ☐ **Safety redundancy**: Explain why disabling factory collision warning and emergency braking removed a layer of protection from a developmental ADS.

1.9 Where Safety, Security, and Accountability Meet

Tempe and Cruise both turned on what the machine knew when it moved or failed to brake, and both had to be reconstructed afterward from what it had recorded. The same three questions (whether the evidence behind a motion was sound, whether anything could refuse it, and whether the decision can be traced) also describe an attack and an audit. In enterprise software and distributed systems, safety, security, and accountability usually live in separate tiers. Security defends network boundaries with cryptographic tokens, reliability frameworks mitigate component failures through retries and circuit breakers, and accountability records transactions to append-only databases. In physical AI, all three converge on the causal boundary, the interface where software commands meet physical actuators. Consider the running machine crossing a warehouse aisle. It needs a valid stopping decision whether its forward cameras are obscured by dirt, an attacker has injected spoofed packets onto its internal Controller Area Network (CAN) bus, or a vision-language-action policy has hallucinated an unobstructed path. In each case the hazard is kinetic, a collision with racking or a worker. Because momentum cannot be rolled back like a failed database transaction, admission control cannot simply confirm who dispatched a command. It must determine whether executing that command now would violate the machine's physical limits.

An ordinary environmental change challenges this boundary as directly as an attack. The running machine's credible deceleration of 2 $\mathrm{m/s^2}$ rests on the premise, stated on its card, that the floor supplies a friction coefficient of at least 0.204. When the floor cannot supply the friction the brakes ask for, traction rather than the brakes caps deceleration, and braking distance grows in inverse proportion to the available friction. A film of spilled hydraulic fluid can drop tire-floor friction from about 0.60 on the aisle's dry concrete to 0.05 (an illustrative pair). A vehicle that brakes at the traction limit would see its braking distance grow by a factor of 12. The running machine asks far less of a dry floor than it supplies, but 0.05 falls below its premise. Its credible deceleration therefore no longer holds, and every clearance envelope and speed profile computed from that deceleration a moment earlier is invalid. The learned planner, processing camera streams that do not reveal the transparent slick, may continue to output an aggressive velocity command with high nominal confidence. Safety verification cannot treat the model's internal confidence as proof of physical feasibility. The machine needs independent, real-time evidence from the plant and its surroundings to detect the divergence. Such evidence includes wheel-slip telemetry, disagreement between commanded and inertial-measurement-unit (IMU) acceleration, and motor current transients. The machine can then restrict motion using a response feasible under the updated traction estimate. If the machine has already consumed the clearance that response needs, detection can support harm mitigation but cannot establish collision avoidance.

Security threats target this same permission machinery by corrupting the evidence used to authorize motion. An adversary does not need to crack cryptographic keys or modify model weights to destabilize a physical agent; distorting the sensory inputs that inform admission decisions is sufficient. Projecting structured light patterns onto camera lenses, spoofing ultra-wideband (UWB) localization beacons, or injecting rogue packets into sensor buses can deceive the controller into approving destructive commands, even while the neural network executes its inference pipeline without numerical fault. Isolating the proposer from the permission path offers little defense if both subsystems ingest the same corrupted telemetry. Securing the causal boundary therefore

requires validating the evidence itself. The validation consists of cross-checking heterogeneous sensor streams (for example, corroborating camera-based visual odometry against wheel encoders and inertial sensors), enforcing cryptographic message authentication across bus transceivers, and maintaining fail-operational plausibility checks that reject sensor readings deviating from physical conservation laws.

In digital systems, a cryptographic capability token or access control list entry grants authority to modify a file regardless of processor load or ambient conditions. In physical AI, **physical authority is state-dependent**. Cryptographic credentials establish identity and organizational rights to request an action, but physical admissibility depends entirely on the instantaneous operational state of the machine and its workspace. A joint velocity command of 2 rad/s may represent a routine transit when an industrial manipulator operates in open free space, yet that identical command becomes unacceptable if the arm is within 2 mm of a mechanical hard stop or a human coworker. Evaluating physical authority requires testing the proposed action against instantaneous kinematics, payload inertia, motor thermal headroom, and state estimation uncertainty. Admission control must also remain coupled to an executable fallback. If the permission gate rejects a proposed command because the machine approaches an operational boundary, the controller must immediately dispatch a dynamically feasible contingency maneuver, such as controlled regenerative braking or a compliant hold that yields to contact. The maneuver halts or redirects the plant without exceeding mechanical stress limits.

↪ A well-formed packet that carries a false measurement becomes a fault trial in the injection campaign developed in section 15.6.3.

Accountability demands an audit trail that reconstructs the causal chain connecting sensory evidence to physical outcomes. When an embodied machine causes a collision or near-miss, post-incident analysis requires timestamp and clock-conversion uncertainty small enough to distinguish the relevant sensing, admission, dispatch, and physical-response events across multiple data streams. These streams include the raw observations ingested by the planner and the action proposed by the learned policy. They also include the state estimates and dynamic constraints evaluated by the admission gate, the filtered command dispatched to the motor drives, and the measured response from joint encoders and force-torque sensors. Command logs record the intended trajectory, while sensor telemetry measures the plant's actual mechanical response. Recording dispatched setpoints alone does not confirm that an actuator executed them; an open-circuit fault, a tripped thermal breaker, or mechanical gearbox jamming will cause the physical plant to diverge immediately from commanded behavior. Both streams must be logged against synchronized hardware clocks (such as the IEEE 1588 Precision Time Protocol) so that investigators can distinguish between a planning failure, an admission gate error, and an electromechanical actuator breakdown. Reconstruction is necessary but does not by itself say who answers for an action. The decisions that set the machine's envelope, its staffing, and its permission to operate are made before any tick, and they have to be recorded on purpose (section 17.2).

Safety, security, and accountability ultimately converge on three unified demands at the permission path guarding the causal boundary:

↪ **Protocol:** The IEEE 1588 Precision Time Protocol exchange that aligns these clocks is developed in section C.7.

1. **Fresh, authenticated physical state**: The permission gate must operate on telemetry fresh enough to bound physical uncertainty and robust enough against tampering or degradation to justify admitting an action.
2. **Deterministic authority to intervene**: The admission mechanism needs uncompromised physical authority to veto or clamp infeasible proposals and inject safe contingency maneuvers, regardless of the learned model's internal confidence or identity.
3. **Synchronized causal telemetry**: The system must keep an immutable record of what was proposed, what state was evaluated, what commands were dispatched, and how the machine responded.

The incidents above map onto these demands. Tempe's predictor discarded evidence it already had, and the factory emergency braking that could have intervened independently had been disabled; Cruise moved without fresh evidence about the space beneath it. Both investigations then depended on the third demand, the recorded telemetry from which the events were reconstructed. The permission path is therefore where a physical machine's safety, security, and accountability obligations are discharged, and anchoring probabilistic proposals to checks against physical limits

is what lets learned capability operate within justified bounds. What those checks must respect, and why, is the subject of four laws.

↪ The operational log that survives physical failure, from which an investigator reconstructs each admitted action, is developed in section 14.7.

1.10 The Four Bedrock Laws

Every check on the permission path answers to constraints that neither a learned policy nor a software supervisor can negotiate away. Whether the warehouse machine is braking for a coworker in an aisle, pressing a spring-latched cage door, or carrying an unmodeled payload, it must respect the mechanical limits of its plant, adapt to dynamic environments altered by its own motion, and arbitrate between speculative model outputs and physically safe actuation. Four bedrock laws organize these engineering responsibilities, each anchoring a major part of this book:

1. **The first law: irreversibility.** Software cannot roll back physical consequences. In pure digital systems, an erroneous transaction can be aborted and state restored from a checkpoint. In physical space, however, work performed on a plant is thermodynamic and permanent. A motor command that collides with a fixture, tears a cable, or fractures a fragile payload cannot be retracted. Every admissibility check must therefore complete before the machine consumes its remaining clearance margin. As the stopping distance budget demonstrates, processing latency directly expends physical clearance, forcing the mechanical actuators to brake over an ever-shrinking distance. Reversing a drive voltage may eventually return an arm to its starting coordinates, but it cannot mend cracked structural material or recover energy lost to friction and heat. Part I translates this physical reality into kinetic budgets for the robot's body and deterministic timing constraints for its low-level controllers.
2. **The second law: endogenous data.** In classic offline machine learning, data samples are drawn independently from a stationary distribution. In embodied systems, data generation is endogenous. Every control action alters the physical state and determines what sensors will observe next. If an arm nudges an object during an approach, the object shifts; if a mobile base drifts off its nominal path, the subsequent visual frames deviate from the training demonstrations. Single-step prediction accuracy on an offline benchmark cannot guarantee closed-loop behavior, because minor execution errors compound across the task horizon, the covariate shift met with the closed causal loop (section 1.4.1). Part II frames data collection, model training, and policy evaluation as a continuous improvement cycle: diagnose behavioral failures, curate targeted recovery demonstrations, retrain candidate policies, and systematically evaluate both performance gains and unintended regressions. Empirical evidence from past rollouts bounds what can be claimed about present capability; only qualified new experience can expand that envelope.
3. **The third law: proposal is not permission.** A learned model operates only as a proposer, never as an unconstrained executor. An architecturally independent permission path (a runtime safety filter or an assurance monitor, for example) must arbitrate every proposed control command before that command reaches hardware actuators. Granting permission requires three prerequisites: fresh state estimates from verified sensors, an explicit check that the command respects every physical safety envelope, and an immediately executable fallback maneuver if the proposal is delayed, missing, or unsafe. This separation of authority must remain robust against system faults, so a runaway or stalled model thread must never be allowed to delay, starve, or corrupt the deterministic safety logic guarding the plant. Part III constructs the real-time execution runtime around this division between speculative proposal and independent, bounded permission.
4. **The fourth law: permission must be earned.** An embodied machine earns permission to operate only as far as empirical evidence supports its performance under stated conditions. Flawless execution across a handful of laboratory trials does not justify autonomy across diverse surface friction, variable payloads, degraded lighting, or sensor degradation. Every deployment safety case must explicitly demarcate the system's operational design domain (ODD),[9] articulate the environmental assumptions underlying those boundaries, and specify fail-safe degradation policies whenever runtime conditions drift beyond verified bounds. The 2018 Tempe collision showed the cost of ignoring this boundary. The system architecture relied on an in-vehicle safety operator as the primary emergency countermeasure, yet lacked any mechanism to monitor whether the operator was actually attending to the roadway

[9] **Operational Design Domain (ODD)**: The automated-driving term for the stated environmental and operating conditions (roadway types, speed ranges, weather, lighting) under which a system is designed to work, and the starting scope of a UL 4600 safety case (UL Standards & Engagement 2023). This book applies the concept to the payloads, surface friction, operating speeds, and disturbances covered by the machine's empirical testing. Operating outside that tested domain has no evidence behind it, which is why section 5.7.1 organizes the operating domain as an explicit scenario table against which collected training data are audited.

(National Transportation Safety Board 2019). Part IV formalizes command authority, fault injection testing, and release criteria so that epistemic and physical uncertainty bound what the machine is permitted to do.

Of these four, only the first is a physical law. Irreversibility follows from Newtonian mechanics and thermodynamics. Work performed on a plant transfers energy and raises entropy, and no software architecture can undo that transition. The remaining three are engineering design principles, distilled from decades of systems engineering, industrial robotics, and aerospace failures. A team could build an architecture that violates them, for example by piping unvalidated network outputs straight to motor amplifiers, but it would incur severe and predictable risks. The book elevates all four to bedrock laws because it treats them as mandatory constraints for any production embodied system.

None of these three architectural principles is entirely novel. Each has a rich foundation in computer systems, control theory, and safety-critical engineering. Table 1.3 maps this book's terminology to the established literature, alongside the canonical sources and documented real-world failures each principle distills, including the Tempe and Cruise incidents. Where the book's terms introduce tighter operational precision, as when tying state freshness directly to the dynamic stopping budget, the book retains specialized systems phrasing. When searching broader engineering literature, the classical terms serve as the standard index.

Table 1.3: Lineage of the four laws: Each law and key term of this book against the field's established name, its canonical source, and a documented failure it distills. Only the first law is physical; the other three are design principles, which is why each has an independent literature. Runtime assurance enters with the proposal boundary, and section 1.14 traces its lineage together with the safety filter and the assurance case.

Book's law or term	Kind	Established name in the field	Canonical source	Documented failure it distills
First law: irreversibility (the causal boundary)	Physical	Irreversible work on the plant; the reaction-plus-braking stopping budget	Mechanics; section 2.3	Tempe (2018): the one-second action suppression let the vehicle travel about 18.7 m with emergency braking withheld (National Transportation Safety Board 2019)
Second law: endogenous data	Design principle	Covariate shift; compounding error in imitation learning	Ross, Gordon, and Bagnell (Ross et al. 2011)	A learning result rather than an accident: a cloned policy's errors can compound with the task horizon (section 5.2)
Third law: proposal is not permission (proposal boundary, permission path)	Design principle	Runtime assurance and its Simplex architecture; safety filter	Sha (Sha 2001); Hsu, Hu, and Fisac (Hsu et al. 2024)	Therac-25: software took over protection that hardware interlocks had provided (Leveson and Turner 1993); Mars Pathfinder (1997): through priority inversion, lower-priority work delayed a deadline-critical bus task until the scheduler's completion check reset the computer (Reeves 1997)
Fresh evidence (an input to every permission check)	Design principle	Age of information	Kaul, Yates, and Gruteser (Kaul et al. 2012)	While not a pure staleness failure, Tempe (2018) shows the neighboring hazard of discarded evidence: each object reclassification dropped historical tracks, forcing path prediction to compute trajectories on truncated sensor history (National Transportation Safety Board 2019)
Fourth law: permission must be earned	Design principle	Assurance case; operational design domain (ODD)	UL 4600 (UL Standards & Engagement 2023)	Tempe (2018): the design made the operator the primary countermeasure in an emergency, yet nothing monitored her attention in real time (National Transportation Safety Board 2019); Cruise (2023): a post-collision pullover moved the vehicle without evidence that the space beneath it was clear, because its detection system did not identify the pedestrian there (Quinn Emanuel Urquhart and Sullivan, LLP 2024)

The proposal boundary is where all four laws meet in one architecture. Irreversibility requires that every check complete before a command alters the plant, and endogenous data prevents offline training metrics from certifying a proposer in the closed loop. For these reasons, the third law (proposal is not permission) places that check on an independent permission path that the model cannot delay or corrupt. The fourth law (permission must be earned) then confines admitted proposals to the operating conditions that empirical evidence has verified. Scaling up model capacity does not relax these constraints; on the contrary, it intensifies physical pressure on the system. Although general machine learning methods deliver superior performance as compute expands (Sutton 2019), the running mobile machine must supply its onboard compute within a finite electrical and thermal budget. Every additional neural layer, attention head, or diffusion step draws against fixed physical allocations of memory bandwidth, electrical power, and thermal dissipation. Inference latency on the stopping path is travel that braking cannot recover. On the running machine at its drive limit, each millisecond costs 1.5 mm of clearance, as the stopping notebook showed. This compute, thermal, and temporal ceiling forms the physical wall confronting every learned proposer.

↪ The memory-bandwidth and latency bottlenecks confronting onboard learned proposers are developed in section 3.4.

Finally, rejecting an inadmissible command must have an explicit physical realization. In traditional software, an unhandled exception or failed assertion can simply terminate a thread. On a moving robot, halting a software process leaves physical momentum untouched. A refusal is meaningful only if the execution runtime immediately dispatches an admissible fallback (a dynamic braking maneuver, a compliant hold, or a controlled stop) that the physical body can safely execute within its remaining stopping clearance. Translating the four laws and their runtime mechanisms into an engineered system takes the four parts of this book, each producing records that later parts rely on.

1.11 The Architecture of This Book

Moving a learned proposal, such as the running machine's trajectory to open a warehouse cage door, from an abstract task requirement to safe, justified physical operation requires a disciplined four-stage progression. The engineer must first bound what the hardware plant can physically achieve and characterize what the learned proposer has demonstrated under rigorous testing. The engineer must then build an isolated runtime monitor to intercept live proposals and establish the operational conditions under which continued execution remains justified. Each stage corresponds to one of the four laws. The system of records illustrated in figure 1.9 formalizes this progression, capturing operating assumptions and empirical evidence so that downstream runtime decisions can inspect and rely on them. Conceptually, these records trace two converging pipelines. One strand establishes offline physical limits and verified competence; the second processes online sensor observations and real-time candidate actions. Both pipelines converge at the permission decision, where the system admits or rejects a proposed trajectory. In section 18.1, this architecture synthesizes both strands to resolve the book's central challenge of engineering dependable, high-performance physical systems out of unverified learned models and unforgiving mechanical hardware.

A record is a machine-checkable artifact that downstream safety and runtime decisions can verify and trust. The initial stopping calculation (notebook 1.1) already sketches the first record in the chain: a limit record, with the six elements the reader guide introduces (nominal value, operating conditions, uncertainty, margin, runtime detector, and provenance). Its nominal value is the derived stopping distance under stated operating conditions of loaded payload mass, dry concrete, a stationary obstacle, and constant approach speed. Because those inputs are illustrative scalars rather than calibrated distributions, however, its uncertainty remains unquantified. The apparent buffer between the obstacle and the machine is therefore only an arithmetic difference between provisional figures rather than a verified safety margin. The calculation also lacks a runtime detector (no sensor yet monitors whether floor traction drops or cargo shifts outside the assumed envelope, section 2.7.3), and its provenance rests on baseline assumptions rather than calibrated hardware logs. An ungrounded calculation becomes operational engineering evidence only when all six fields are populated and backed by empirical data. Every numbered box across the four part bands in figure 1.9 establishes a machine-checkable record of this rigor for a distinct architectural decision.

Part I, The Machine Anatomy, translates physical irreversibility into rigid architectural limits. Chapter 2 profiles the mechanical plant to quantify its physical budgets (measurement freshness,

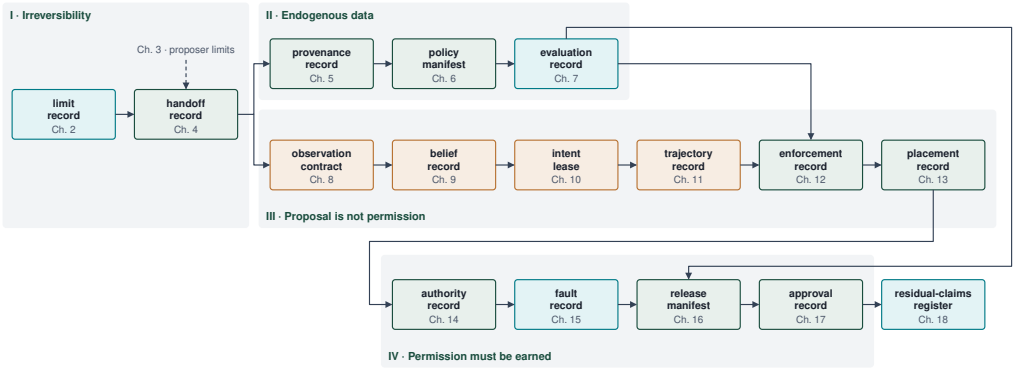


Figure 1.9: The spine of the book: Each part is led by one law, and each chapter of Parts I to IV except The Cognitive Brain (Chapter 3) produces a record that a later one consumes. What the plant can do and what the proposer has shown it can do run as one strand, what the proposer proposes now as another, and the two meet at the permission decision.

stopping distance, deliverable torque, winding heat, and supply voltage). Chapter 3 measures the throughput and inference latency of the learned proposer to determine what it can deliver before deadlines expire. Chapter 4 links compute to the actuators across deterministic buses, structuring a permission path whose timing guarantees derive from the plant's stopping requirements. Together, the limit record and handoff record make every physical and temporal assumption explicit. Part II, Teaching the Machine, confronts the reality that physical actions continuously alter future sensor observations. Chapter 5, chapter 6, and chapter 7 build the evidentiary chain required to collect demonstrations, train control policies, and bound policy performance to a declared operational design domain. Across this pipeline, the provenance record, policy manifest, and evaluation record document both the empirical competence demonstrated by the model and the operational boundaries where its statistical guarantees break down.

Part III, Running the Machine, enforces at runtime the separation between proposing an action and permitting it to execute. From chapter 8 through chapter 13, the chapters trace candidate commands through the online runtime. They qualify sensor measurements in observation contracts and maintain state estimates in a belief record. They bound a goal with an intent lease (a time-limited grant of a target, never of a path, and distinct from the chunk lease that bounds one motion segment). They check candidate trajectories against real-time physical constraints before admission and dispatch, then test whether the hardware keeps the permission path independent. Part IV, Governing the Machine, enforces the principle that permission must be earned. Chapter 14, chapter 15, and chapter 16 govern how authority transfers between the machine and a human operator, how fault injection tests whether protective responses hold, and how deployment decisions are assembled into release manifests. Chapter 17 then names who answers for each decision behind an admitted action. It also tests whether each human role the design relies on is one a person can perform, and it counts the people who bear the machine's risk and appear in its records. Finally, chapter 18 brings the framework to bear on an unfamiliar machine, synthesizing what an engineering team can construct and formally justify while identifying the residual risks that require further mitigation. Across these parts, the warehouse mobile manipulator introduced here provides a single concrete machine on which to ground each architectural decision. Its records render design assumptions fully inspectable, though an engineer must remember that assembling a record documents an explicit claim rather than guaranteeing its physical truth. The system record register in section C.14.2 catalogs all sixteen records, noting their producing chapters and the specific invariants they bind, while the machine card in section 1.4.2 specifies the foundational physical parameters of the robot itself. Together, this four-stage route forms a repeatable discipline for making learned behavior dependable in physical space, guarding against the common misconceptions that arise when digital assumptions are carried across the causal boundary.

1.12 Fallacies and Pitfalls

The misconceptions that follow arise when assumptions from pure software engineering, statistical learning, or classical control theory are imported into embodied machines. In digital systems, an erroneous state can typically be trapped, rolled back, or restarted. Across the causal boundary, however, software outputs become physical forces that alter the world irrevocably. Preventing physical harm requires hard deterministic guarantees that learning objectives, human operators, and asynchronous recovery routines cannot provide on their own.

Fallacy: *Safety constraints can be trained into the policy.*

Engineering teams often attempt to enforce physical safety by adding penalty terms, barrier penalties, or negative rewards to the training objective. While penalty shaping suppresses undesirable actions over the training distribution, it provides no mathematical guarantee on the actions emitted during deployment. Deep neural networks remain uncalibrated function approximators when exposed to out-of-distribution inputs, sensor degradation, or novel environmental dynamics. A statistical loss function optimizes expected performance, but safety requires hard invariants that hold on every control cycle. For this reason, deterministic safety enforcement must sit downstream of the policy, operating as an independent permission path that filters proposed commands before they reach actuator driver registers, exactly where the third law places the boundary gate.

Pitfall: *Treating human supervisory intervention as instantaneous in shared-autonomy systems.*

In shared-autonomy designs, such as a robotic surgical tool providing haptic guidance from a learned anatomical model, system designers sometimes treat the human operator as a failsafe catch-all. This assumption ignores the physiological limits of human reaction time, which ranges between 150 ms and 250 ms for simple tactile or visual reflexes and grows under cognitive load. Once console video and a handshake are added, the running machine's card budgets an in-loop takeover at 605 ms. Over that interval, an actuator running on a 1 kHz control loop can execute hundreds of unchecked cycles, generating enough force to tear soft tissue or damage a mechanical workpiece before the operator can intervene. Delegating authority to a human supervisor does not eliminate the machine's capacity for harm during the supervisory lag. The permission path must enforce local force, velocity, and displacement envelopes autonomously, preserving the established physical envelopes under their operating assumptions without relying on timely human overrides.

Pitfall: *Assuming software latency merely degrades performance gracefully.*

In purely digital systems, inference latency or scheduler jitter increases response time or degrades service quality. In physical systems, timing is a functional correctness requirement. If a learned policy stalls, whether from accelerator kernel contention, memory allocation pauses, or network serialization delays, allowing its previous command to persist across subsequent control cycles turns stale intent into physical danger. For example, a mobile manipulator that repeats its last forward-velocity command during a 100 ms inference hiccup can collide with an unmapped obstacle. Actuation commands must therefore carry bounded temporal validity, enforced by hardware or real-time software watchdogs. When that validity window (the chunk lease of the stopping budget) expires without a verified update, the low-level controller must begin a fallback within its established response budget. The selected stop or hold must remain feasible for the current mechanism, state, and surviving resources.

Fallacy: *Assuming physical actuation is reversible through subsequent software compensation.*

Transactional software architectures rely on rollbacks and compensation routines to undo erroneous operations. In embodied systems, thermodynamics and materials science break this equivalence. If a robotic arm applies excessive torque to a workpiece, issuing a reverse motor command cannot unbend plastically deformed metal, repair fractured composite, or recover the electrical energy dissipated as heat in the motor windings ($I^2_{\text{phase}} R_{\text{phase}}$). Anomaly detection routines that react after physical contact can only truncate ongoing damage; they cannot undo the mechanical work already transferred across the causal boundary. Preventive verification, such as checking proposed trajectories against workspace boundaries, torque limits, and kinetic energy caps before dispatch, must serve as the primary defense. Reactive recovery strategies remain necessary for unexpected

disturbances, but they function as a secondary backstop rather than a substitute for preventive enforcement.

1.13 Summary

Physical AI converges two historically distinct engineering disciplines: classical robotics and modern machine learning. As learned models move progressively closer to direct actuator control, the boundary between digital computation and physical reality sharpens at the causal boundary, the interface where an admitted command becomes an irreversible physical effect. In purely software systems, state rollbacks and retries can recover from algorithmic faults; in the physical world, software cannot undo kinetic energy, momentum, or impact. Safe recovery requires an executable, physically feasible countermeasure.

Physical dynamics also do not pause while neural networks compute. Because a machine continues to travel during sensing and processing delays, safety requires deterministic admission checks upstream of any consequential actuation. As the warehouse mobile manipulator's stopping budget and the Tempe collision show, prediction quality offers no protection once response delay has spent the clearance. The architecture resolves this tension by separating proposal from permission and giving an independent, bounded-latency permission path the authority to refuse. Four laws, one physical and three design principles, organize the framework the rest of the book develops, while the elementary stopping budget and its crossover speed form this chapter's quantitative core (table 1.4).

Table 1.4: Key equations of the causal boundary: The elementary stopping budget and its crossover speed, with the place each is derived.

Equation	What it bounds or decides	Derived
$d_{\text{stop}} = v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} \leq D_{\text{clear}}$	Whether a moving machine can stop within its observed clearance: distance traveled during response delay plus braking distance, prior to allowances added in later chapters	equation 1.1
$v^* = 2a_{\text{brake}}\tau_{\text{delay}}$	The crossover speed at which delay distance equals braking distance; below v^*, delay travel is larger, while above it, braking travel is larger	From equation 1.1, section 1.4.3

Key Takeaways: Engineering across the causal boundary

- **Useful behavior requires an integrated machine**: Physical AI combines learned decision-making, physical dynamics, and delegated mechanical authority. Delivering reliable autonomous operation requires co-designing task competence with physical body limits, real-time compute runtimes, and the operational evidence supporting deployment.
- **Response delay consumes physical margin**: The elementary stopping budget $d_{\text{stop}} = v\tau_{\text{delay}} + v^2/(2a_{\text{brake}}) \leq D_{\text{clear}}$ partitions stopping distance into latency-induced travel and mechanical braking. Below the crossover speed $v^* = 2a_{\text{brake}}\tau_{\text{delay}}$, response delay dominates stopping distance; above v^*, braking deceleration dominates. For the warehouse mobile manipulator at its drive limit, pre-braking travel adds 35.6 percent to braking distance. The decisive delay belongs to whichever path triggers the protective response, which can operate independently of the learned proposer's inference time.
- **Proposal and permission serve distinct architectural roles**: The learned proposer (the Brain) generates goal-directed actions, while an independent permission path admits, modifies, or rejects them before they cross the causal boundary. Safety enforcement relies on validated sensor inputs, physical kinematic envelopes, strict real-time deadlines, and verifiable fallback maneuvers.
- **Evidence governs safe operation and systematic refinement**: Four laws structure this book. Irreversibility is a physical law; the other three (endogenous data, proposal is not permission, and permission must be earned) are design principles that translate covariate

shift, runtime assurance, and the assurance case into systems practice. Together they link physical hardware, embodied learning, real-time execution, and operational governance, establishing verified service within defined operational envelopes while identifying the targeted evaluations needed to expand them.

Before judging whether a proposed action is right, an engineer should establish what physical response would remain if the proposal were wrong, stale, or never arrived, and whether the machine still holds the clearance, the time, and the authority to carry that response out. A design that cannot answer from its own measured limits has handed safety to prediction quality, and the stopping budget showed that prediction quality cannot buy back clearance once delay has spent it. The answer therefore starts from what the plant can do under stated conditions, never from what the model believes.

1.14 Historical Perspective and Further Reading

In physical artificial intelligence, moving learned models directly into the actuation loop creates severe timing and safety hazards. The engineering principles that resolve these challenges originate in embedded systems, real-time control, and safety engineering. General-purpose computing treats execution time as a mere performance optimization, assuming that correctness depends solely on logical input-output relationships. Cyber-physical systems theory overturned this assumption, holding that physical dynamics and computational processes must be modeled jointly because a command dispatched at the wrong time is physically incorrect (Lee 2008). This insight forms the foundation of the causal boundary. Similarly, separating command proposal from execution permission stems from runtime assurance architectures. The Simplex architecture demonstrated that an industrial control system could safely run an unverified, high-performance controller if paired with a certified baseline controller and deterministic switching logic capable of taking back the plant before safety boundaries were violated (Seto et al. 1998; Sha 2001). Modern safety filters in embodied machine learning generalize this design, validating neural network proposals against mathematical certificates of safe states (control barrier functions or reachability envelopes, which chapter 12 develops) before granting actuator permission (Hsu et al. 2024).

Many foundational safety principles in physical systems emerged from forensic investigations of catastrophic failures. The Therac-25 radiation therapy disasters demonstrated the fatal peril of replacing mechanical interlocks with software routines (Leveson and Turner 1993). When race conditions bypassed software checks, the machine administered lethal radiation overdoses, proving that software reliability does not imply system safety. Systems-theoretic safety subsequently recast accidents not as isolated component breakdowns, but as failures of the overall control structure to enforce safety constraints on the interactions among components, feedback loops, and human operators (Leveson 2011). This shift led directly to the assurance case, a structured argument demonstrating that concrete evidence satisfies defined safety claims within explicit operating environments (Kelly 1998). The UL 4600 standard adopted this structure as its central framework for autonomous products (UL Standards & Engagement 2023), abandoning arbitrary road-testing mileage in favor of verifiable, claim-based arguments. Public accident records, particularly the National Transportation Safety Board investigation of the 2018 Tempe autonomous vehicle collision (National Transportation Safety Board 2019), remain the field's most complete documentation of how safety arguments unravel when classification changes, a response delay chosen in software, and reliance on a human fallback interact in the field.

- **Cyber-physical systems** (Lee 2008). Lee articulates why classical computing abstractions fail when computation interacts with physical processes. General-purpose computer architectures deliberately abstract away time, treating execution speed as a performance consideration rather than a correctness specification. Lee demonstrates that for physical interactions, the timing of an actuation command is inseparable from its physical meaning, providing the conceptual foundation for treating the causal boundary as an explicit systems interface.
- **Runtime assurance** (Sha 2001). Sha's paper on the Simplex architecture is the place to see how the switching logic decides when to take the plant back from the unverified controller, and

why that logic must be simpler than what it supervises. It is the pattern behind this book's separation of proposal from permission.

- **Therac-25** (Leveson and Turner 1993). Leveson and Turner's forensic analysis traces the specific race conditions and the development and reporting practices that let the overdoses recur, detail the summary above omits. It remains the essential case study in why protective functions need justification independent of the software they supervise.
- **Systems-theoretic safety** (Leveson 2011). Leveson introduces the STAMP accident model, reframing safety as a continuous control problem rather than a collection of component failure probabilities. Accidents occur when the control structure fails to enforce safety constraints across component interactions, sensor delays, and feedback loops. This text provides the theoretical basis for enforcing safety invariants as dynamic runtime constraints rather than static design assumptions.
- **The Tempe investigation** (National Transportation Safety Board 2019). The National Transportation Safety Board report on the fatal 2018 Tempe autonomous vehicle crash offers the most thorough empirical record of an embodied AI safety failure. The report details how classification changes that discarded tracking history, a one-second action suppression meant to limit responses to false alarms, the separate deactivation of the vehicle's factory emergency braking over possible radar interference, and reliance on an operator whose attention nothing monitored combined in the collision. The event timeline and sensor telemetry provide vital reading for anyone building real-time perception-action loops.

What's Next: From the causal boundary to physical limits

A permission decision needs a physical limit against which to judge the proposed action. Chapter 2 begins Part I with the five budgets the permission path must check before it commits torque (measurement freshness, stopping distance, deliverable torque, winding heat, and supply voltage) and asks which binds first for a given maneuver, under what stress, and on what evidence. Each budget ends as a limit record, the six-element form this chapter sketched from its stopping calculation. Chapter 3 then examines the learned proposer's capabilities and timing, and chapter 4 uses both sets of constraints to design the handoff and independent permission path. The progression starts with what the machine can sustain and ends with how computation may safely request it.

DRAFT

I

THE MACHINE ANATOMY

DRAFT

Part I

DRAFT

1

The Machine Anatomy

Every physical AI system acts through a body with limited capacity to move, stop, exert force, and dissipate heat. The first law of section 1.10 holds that this apparatus cannot take back the work it delivers, and Part I turns that law into numbers. It asks how quickly the machine's evidence ages, how far the machine travels before it stops, how much torque its drives can deliver, and how much winding heat and supply-voltage drop they tolerate. The Body establishes what motion the hardware can physically support. The Brain, the learned proposer, establishes what computation can supply and how long its answers stay true. The Nervous System must therefore connect them within the time the stopping budget allows. Following this physical order lets the engineer derive an architectural requirement before selecting its implementation.

Principle 1: The irreversibility law

Invariant: Once a permitted command crosses the causal boundary, the energy it delivers becomes motion, strain, or heat that no software action can recall. At steady speed, during a pre-brake delay τ_{delay} the machine travels $v\,\tau_{\text{delay}}$ before any response begins. A stop therefore requires $d_{\text{stop}} \leq D_{\text{clear}}$, with every term of d_{stop} taken from a measured limit of this machine under stated conditions (section 2.3).

Implication: Each part of the machine carries budgets in time, distance, torque, heat, and voltage. The checks needed to admit a command must finish before its execution and inside those budgets. A design is admissible only when it can say which budget binds first, how that budget was established, and what the machine does when it is exhausted.

- **The Physical Body (chapter 2):** Decides which physical budget binds first and how its value was established, and produces the limit record from which every later record descends.
- **The Cognitive Brain (chapter 3):** Decides what a learned proposer can deliver on time and what memory and latency that costs, and derives the machine's two-speed split, a slow intent model above a fast chunk policy. It produces no record of its own; its proposer limits, and the payload every proposal must carry, enter the handoff record.
- **The Nervous System (chapter 4):** Decides where the permission path must run to stay independent of the proposer, how often it must check, and how long a proposal may drive the actuators. It produces the handoff record together with the header that every proposal carries. The independence it requires leads it to adopt a two-processor split as the hypothesis the machine is built on; whether that split holds on real silicon is decided in chapter 13.

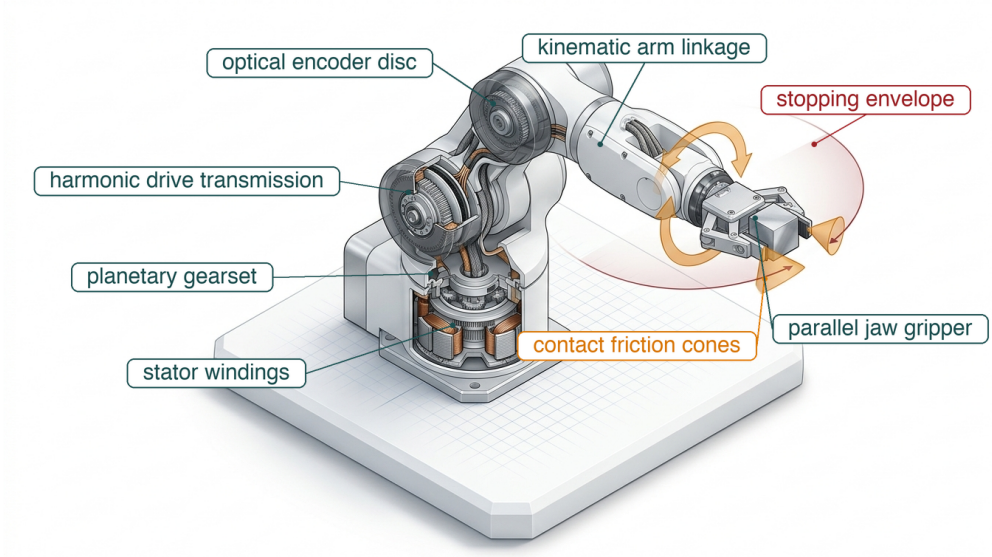
Part I hands forward three linked artifacts across these chapters. On the physical side, chapter 2 establishes the limit record, the six-element form in which every budget that can end in damage states its operating conditions and its evidence category. The chapter then fills that form on the running machine for the credible deceleration behind the stopping distance, the drive torque that must deliver it, and the input lockout of the converter on the shared drive bus. Against those physical bounds, chapter 3 characterizes the proposer's supply by pricing how many future setpoints one chunk covers, how long a new chunk takes, and how quickly the evidence behind it ages, showing why a chunk keeps the arm supplied without keeping that evidence fresh. Joining demand and supply, chapter 4 produces the handoff record: the lease the permission path derives from the stopping budget, the tick at which it checks, the handoffs between proposer and permission path,

and a worked budget that couples all three chapters on the running machine. That worked example selects a candidate configuration. Qualification still requires bounds on each interval, including dispatch to admission, evidence that the protective response remains physically available, and loaded tests showing that the permission path's isolation from the proposer holds. Part II inherits both records as preconditions on the demonstrations the machine collects while its permission path is watching.

DRAFT

2

The Physical Body

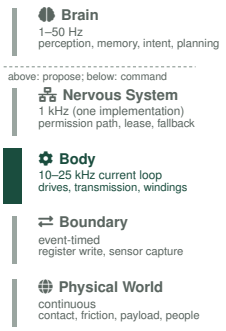


2.1 The Five Physical Budgets
2.2 Measurement Freshness
2.3 Kinetic Momentum
2.4 Actuator Transmission Limits
2.5 Thermal Duty Cycles
2.6 Electrical Power Integrity
2.7 Measuring a Machine's Own Limits
2.8 Which Physical Budget Binds First
2.9 Fallacies and Pitfalls
2.10 Summary
2.11 Historical Perspective and Further Reading

Purpose

Why can a correctly chosen action still demand more than the machine's body can deliver?

A mobile manipulator can recognize an obstacle correctly and still strike it if its body cannot stop within the remaining distance. The same machine can hold a payload without moving while its motor windings overheat, or lose its controller when a sudden demand for current pulls the supply voltage too low. These failures arise from limits that prediction accuracy alone cannot reveal. Physical AI engineering requires connecting a proposed action to the motion, heat, and electrical demand it creates, then establishing which limit the assembled machine reaches first. Measurements must also establish the conditions under which those limits hold, because a change in payload, floor surface, cooling, or battery state changes what the machine can deliver. The body's limits therefore govern what the permission path may admit before a command crosses the causal boundary and produces irreversible physical work.



Learning Objectives

- Calculate observation freshness limits from measurement age and the machine's allowable motion
- Calculate stopping distance and the admissible speed ceiling from pre-brake delay, credible deceleration, and clearance overheads
- Evaluate actuator and transmission choices against response limits, reflected inertia, and mechanical impedance
- Estimate sustainable motor duty cycles from heat generation, cooling, and operating conditions
- Budget electrical supply margins for simultaneous actuation, computation, and regenerative braking
- Specify the elements of a limit record that states a measured limit's conditions, uncertainty, margin, runtime detector, and evidence category
- Evaluate maneuver-specific binding budgets from qualified evidence and justify holding up the permission supply through covered faults

2.1 The Five Physical Budgets

Every action a learned model proposes ultimately manifests as electrical current flowing through a motor winding, and from that instant physics governs execution. Machine learning practice rarely contends with this transition. Policies trained in simulation typically interact with idealized plants where commanded torque appears instantaneously, heat never accumulates in windings, and direct current (DC) bus voltages never sag under load. A physical machine honors none of these simplifications. The **physical body** is the coupled assembly of actuators, mechanical transmissions, structural linkages, power electronics, and sensors that translates digital commands into mechanical motion. Its physical dynamics bound what even a mathematically optimal policy can achieve. An emergency brake command requires finite time to develop mechanical retarding force and clearance distance to dissipate kinetic energy. Similarly, commanding a manipulator to hold a heavy payload motionless draws continuous phase current, dissipating resistive heat into the stator even though the joint does zero mechanical work.

The proposal boundary in section 1.7 assigns the permission path responsibility for admitting action, and the body provides the concrete physical envelopes that make that gating decision meaningful. Before committing a command to hardware registers, the permission path must evaluate five physical budgets. It must confirm that the sensor observation remains fresh, that the moving machine can stop within available clearance, that the actuator can deliver the required torque trajectory, that the winding can tolerate the resulting thermal rise, and that the electrical bus can support the transient current demand. These mechanisms form a direct causal chain. Sensor and pipeline latencies consume allowable stopping distance. Arresting momentum demands delivered actuator torque. Generating that torque dissipates heat in motor coils and pulls current from a shared power rail. For any given maneuver, the governing systems question is which of these physical budgets binds first, under what operational stress, and on what empirical evidence.

1. **Measurement freshness** (t_{age}): The observation must remain valid until the mechanical plant responds. Latency accumulated across sensor exposure, data transport, neural inference, and low-level drive dispatch consumes allowable tracking margins as the physical world continues to move.
2. **Kinetic momentum** (E_k, p): A moving machine carries kinetic energy ($E_k = \frac{1}{2}mv^2$) and linear or angular momentum ($p = mv$). Bringing the plant to rest demands applying retarding force over finite distance and time; safe operating velocity must therefore preserve a stopping envelope well within available physical clearance.
3. **Actuator bandwidth and reflected inertia** ($N_{\text{gear}}^2 J_{\text{rotor}}$): Winding inductance and supply voltage limit the rate at which torque can change. A gearhead with reduction ratio N_{gear} multiplies

delivered torque by N_{gear}, but it also scales the motor rotor's inertia J_{rotor} by N_{gear}^2 as seen at the joint output (section 2.4.3), which constrains instantaneous acceleration.

4. **Thermal capacity** ($\hat{T}_{\text{wind}}$): Resistive losses ($P_{\text{loss}} = i_q^2 R_{\text{eff}}$, with R_{eff} the winding's effective loss resistance) deposit thermal energy directly into the stator windings and inverter switching stages. Heat dissipation limits sustained continuous torque, while the thermal mass of the coils constrains the magnitude, duration, and repetition rate of transient torque bursts.
5. **Electrical power integrity** (V_{rail}): Onboard compute clusters and motor inverters draw from a shared DC power rail. Peak motor accelerations must not drag the bus voltage below digital logic thresholds, while regenerative braking must provide an electrical sink so returned kinetic energy does not overvoltage the bus.

Under the **simultaneous admission invariant**, motion permission requires a non-negative margin in every budget before the permission path commits torque to hardware registers. Each margin must be checked in its own units; time, distance, torque, temperature, and voltage are separate constraints, not interchangeable quantities:

$$
\begin{aligned}
t_{\text{age}} &\leq \Delta t_{\text{fresh}}, \\
d_{\text{stop}} &\leq D_{\text{clear}}, \\
|\tau| &\leq \tau_{\text{out,max}}, \\
\hat{T}_{\text{wind}} &\leq T_{\text{limit}}, \\
V_{\text{rail,min}} &\geq V_{\text{uvlo}}.
\end{aligned}
$$

Here τ is requested torque and $\tau_{\text{out,max}}$ is the maximum deliverable output torque under current electrical and thermal conditions. The estimated winding temperature $\hat{T}_{\text{wind}}$ must remain below its limit, while the lowest rail voltage $V_{\text{rail,min}}$ must remain above the undervoltage-lockout threshold V_{uvlo} at which the supply regulator shuts down. The fifth line checks only undervoltage. Permission must also require regenerative braking to stay below the qualified upper operating-voltage limit of the supplied components, with an available absorption or dissipation path. Two lines state their budgets in derived form. The kinetic budget appears as the stopping distance it demands, $d_{\text{stop}} \leq D_{\text{clear}}$. The actuator line checks only torque magnitude, because the slew-rate limit that winding inductance and reflected inertia impose enters the stopping line instead, through the brake-onset term of the pre-brake delay τ_{delay}. All five inequalities must hold simultaneously. A surplus in four budgets cannot compensate for a deficit in the fifth. A fresh observation cannot prevent a stop from overrunning if kinetic energy exceeds available clearance, and abundant battery energy cannot prevent an actuator from burning out if thermal dissipation limits are violated.

> Systems Perspective 2.1: Why physical limits bound physical AI architecture
>
> The permission path requires concrete bounds on physical behavior, not abstract software promises. Accepting a torque command does not guarantee that the inverter can deliver the requested current against the back electromotive force (back-EMF) the spinning motor induces in its own windings, nor does cutting motor power guarantee that friction will arrest the plant before it breaches a clearance zone. The five physical budgets bridge this gap, tying software proposals to verifiable, empirical responses under declared operating conditions.

The warehouse mobile manipulator that this book follows draws on all five. Loaded to at most 368 kg and moving at its 1.5 m/s drive limit (illustrative; see the Reader Guide), it carries 414 J of kinetic energy through aisles it shares with people. Each budget establishes an empirical operating boundary on the assembled machine under declared thermal and electrical stress. Each ends the chapter as a limit record, the six-element record that section 1.11 sketched from the stopping notebook: a value, the conditions under which it holds, its uncertainty, the margin defended, a runtime detector, and the provenance of its evidence. The first budget is the age of the measurement on which every command rests, which runs from physical transduction, well before any software process receives the raw sensor data.

2.2 Measurement Freshness

Software benchmarks in machine learning frequently report latency as isolated accelerator execution time, for example 18.5 ms for the vision backbone of the arm's wrist-camera grasp path on an edge neural processing unit (NPU), a different pipeline from the navigation backbone that the stopping budget of section 1.4.3 charges. This narrow focus fosters the illusion of an agile 54.1 Hz control loop. In physical AI systems, however, that compute kernel represents only one intermediate segment in an end-to-end silicon-to-plant dataflow pipeline. A sensor measurement begins decaying the moment physical transduction occurs; what governs physical performance is its total age when torque actually appears at the actuator joint. That physical age spans the entire interval from the moment photons strike a photodiode array, magnetic flux sweeps past a Hall sensor, or strain deflects a piezoresistor to the moment motor inverter switches build stator magnetic flux that alters link acceleration. If a perception policy evaluates an obstacle bounding box in 18.5 ms, while exposure integration, sensor readout, Peripheral Component Interconnect Express (PCIe) transport, kernel scheduling, fieldbus dispatch, and motor current rise consume an additional 16.5 ms at the median, the true physical age of the observation when the joint accelerates is 35 ms. The distribution tail extends farther still.

Throughout that capture-to-onset window, the physical environment continues to evolve unobserved. In distributed data systems, read-after-write consistency windows govern whether a client observes stale or linearized state. In physical AI, the physical plant acts as an autonomous, unpausable concurrent writer: spatial position $\mathbf{p}(t)$ evolves along continuous trajectories at velocity $\mathbf{v}(t)$. An observation age t_{age} converts directly into unobserved physical displacement $\Delta\mathbf{p} \approx \int_0^{t_{\text{age}}} \mathbf{v}(t)\,dt$. High-speed machines inevitably act on historical state.

The rate of world-state change dictates how long a measurement remains valid before acting on it introduces intolerable error. Let $e_{\max}$ denote the maximum acceptable error in a tracked state, such as a joint angle or the position of a part on a conveyor, and let $v_{\max}$ denote the fastest credible velocity at which that state can evolve. The freshness deadline Δt_{fresh} is the upper bound on measurement age before a control action based on that measurement becomes invalid, defined by the inequality

$$\Delta t_{\text{fresh}} \leq \frac{e_{\max}}{v_{\max}} \tag{2.1}$$

Definition 2.1: Freshness deadline

Freshness deadline (Δt_{fresh}) is the largest age, counted from the capture of a measurement to the onset of the actuator's response, that the measurement of a moving target may reach before the target's motion alone spends the allowable position error $e_{\max}$ (equation 2.1).

1. **Significance**: It converts a spatial tolerance into a time budget that every stage from exposure to torque onset must share. A 12 mm tolerance on a target moving at 0.20 m/s leaves 60 ms for sensing, inference, transport, and current rise together.
2. **Distinction**: Unlike an inference-latency budget, which times one stage, the deadline bounds the age at onset; unlike a control-loop period, it is set by the target's speed and the tolerance rather than by the loop rate. When the machine moves toward a stationary obstacle instead, the same age becomes the first term of the pre-brake delay τ_{delay} and is paid as blind travel $v\tau_{\text{delay}}$ in the stopping distance.
3. **Common pitfall**: Treating the deadline as a property of the sensor or the model. It changes whenever the target moves faster or the tolerance shrinks, so a pipeline qualified on a slow conveyor can miss on a faster one without any change in its own timing.

↪ The same deadline applied to a stored belief, which lapses once its region falls out of view, is developed in section 9.7.

The running machine meets this bound at its pick station, where the arm grasps a mug from a moving conveyor. The belt carries the mug at speeds up to $v_{\text{conv}} = 0.20$ m/s, and the gripper mechanism tolerates $r_{\text{tol}} = 15$ mm of position error. Initial handle-pose estimation already consumes $e_0 = 3$ mm of this margin. Consequently, the mug's physical motion may spend at most $e_{\max} = r_{\text{tol}} - e_0 = 12$ mm, with $v_{\max} = v_{\text{conv}}$. Notebook 2.1 evaluates this resulting deadline against the ages

an integrated camera-to-joint pipeline delivers in practice. The deadline governs any target that moves while its observation ages. For moving workpieces, it limits motion-induced tracking drift under bounded relative velocity and valid initial observations. Sensing and state-estimation noise consume separate slices of the tolerance budget, just as e_0 does here. Conversely, when the robot approaches a stationary obstacle, such as a person standing in a warehouse aisle, the target does not move. Instead, the mobile base drives toward the person during the latency interval. Observation age then converts directly into unguided vehicular travel before braking begins, which section 2.3 charges against the kinetic stopping envelope.

2.2.1 Transduction physics, quantization noise, and Nyquist limits

If a 12-bit magnetic encoder jumps by a single count between two 1 ms servo ticks, backward differencing reports a sudden 88°/s velocity spike. If an unshielded cooling fan vibrates an inertial unit above half its sampling rate, the vibration folds into the low-frequency band and looks like chassis acceleration. Every sensor trades error tolerance against time: it must resolve the physical state of interest, and the analog filtering, sampling, or serial scanning needed to extract a clean signal must fit inside the freshness deadline. Four physical mechanisms set this trade:

1. **Optical Encoders and Velocity Quantization Noise:** An encoder measures joint position in discrete angular increments. An n-bit encoder discretizes the continuous 2π rad circle into an angular resolution of $\delta\theta_{\text{enc}} = \frac{2\pi}{2^n}$. For an 18-bit absolute optical encoder, $\delta\theta_{\text{enc}} \approx 2.4 \times 10^{-5}$ rad (0.0014°); for a 12-bit magnetic encoder, $\delta\theta_{\text{enc}} \approx 1.53 \times 10^{-3}$ rad (0.088°). Feedback controllers, which compute each command from the measured difference between the desired and the actual state, estimate joint velocity from successive position samples across a discrete control cycle Δt:

   $$\dot{\theta}_{\text{enc}_k} \approx \frac{\theta_{\text{enc}_k} - \theta_{\text{enc}_{k-1}}}{\Delta t}$$

   Backward differencing acts as an effective high-pass filter, scaling high-frequency position fluctuations by $1/\Delta t$. Under an idealized model where successive quantization errors are independent and uniformly distributed across one encoder count, each sample has variance $(\delta\theta_{\text{enc}})^2/12$. Differencing two independent samples sums their variances to $(\delta\theta_{\text{enc}})^2/6$, yielding a velocity-error standard deviation of $\sigma_{\dot{\theta}_{\text{enc}}} = \frac{\delta\theta_{\text{enc}}}{\sqrt{6}\,\Delta t}$. At a 1 kHz servo update rate ($\Delta t = 1$ ms), a single-count jump on a 12-bit encoder (1.53 mrad) registers as an abrupt velocity spike of 1.53 rad/s (88°/s). If routed directly into velocity feedback loops or neural policy state inputs, these discrete impulses cause severe torque chatter and excite mechanical resonances. Downstream low-pass filters or state observers (models that infer a quantity such as velocity from a stream of measurements) smooth this quantization noise, but filter group delay introduces phase lag, consuming milliseconds of the freshness budget.
2. **Inertial Measurement Units (IMUs) and Nyquist Aliasing:** An inertial measurement unit measures linear acceleration and angular velocity. Sampling must isolate macroscopic body dynamics from high-frequency structural vibration. Under the Nyquist-Shannon sampling theorem (Nyquist 1928; Shannon 1949), continuous signals require a sampling frequency strictly greater than twice their highest spectral component ($f_s > 2f_{\text{max}}$). Mechanical sources, including gear mesh harmonics, cooling fans, and motor inverter switching, generate vibrations reaching kilohertz frequencies. Without adequate analog anti-aliasing filters before the analog-to-digital converter (ADC), structural vibrations above the Nyquist frequency fold back into baseband frequencies (1–20 Hz). The state estimator, which combines sensor readings into an estimate of the machine's motion, cannot distinguish these aliased artifacts from genuine chassis acceleration, introducing cumulative drift into dead-reckoning integrations. Raising downstream software loop rates cannot reconstruct information already corrupted at the sensor hardware interface.
3. **Tactile Sensor Arrays and Scanning Bandwidth:** Tactile arrays measure contact pressure across a robot gripper using discrete sensing elements called taxels. To minimize cabling across robotic wrists, hardware architectures multiplex taxels across shared row and column lines. Scanning these elements sequentially forces an entire frame to span the multiplexing period. This serial scan time adds directly to the measurement age and limits the bandwidth

for detecting rapid contact events, such as the onset of localized slip. Increasing the taxel density without scaling the underlying ADC readout rate produces older tactile observations.

4. **Cameras and Rolling-Shutter Readout:** A rolling-shutter CMOS camera exposes and transfers pixel rows sequentially, so a single frame spans its entire readout duration. Image timestamps must therefore anchor to a defined reference, such as the exposure midpoint of a designated row, rather than the frame-arrival interrupt.

Across all four mechanisms, a measurement begins aging at physical transduction, milliseconds before any software driver receives a buffer. Bounding how much of the freshness deadline remains at actuation therefore requires tracing the signal across every stage between transduction and motor current.

↪ Rolling-shutter distortion on the navigation camera, and its compensation before images enter the state estimator, is developed in section 8.2.

2.2.2 The end-to-end sense-to-actuation latency pipeline

A measurement's physical age begins at **capture**, the moment of physical transduction, rather than when an operating system process first receives the data buffer. Three subsequent boundary events demarcate the pipeline from transduction to mechanical work. *Dispatch* is when perception publishes dated perceptual evidence to the action proposer. *Admission* is when the real-time permission path validates and accepts a proposal built using that evidence. *Onset* is when electrical current in the motor stator windings begins driving physical acceleration at the joint. The four events recur throughout the book, and table 4.1 later budgets the intervals between them for the running machine. For the freshness deadline, capture and onset form the bounding endpoints, and the deadline limits the cumulative delay between them.

An illustrative camera-to-joint pipeline reveals how latency compounds across physical and digital boundaries (table 2.1). Between capture and dispatch, the CMOS sensor integrates exposure and reads out raw scanlines over MIPI CSI-2.[1] Direct memory access (DMA) transfers the frame into host memory, and the vision backbone executes. In this representative architecture, the policy network executes in the same inference pass as the backbone, spanning the dispatch boundary. Between dispatch and admission, inter-process communication (IPC) transmits the proposed trajectory to the safety microcontroller, a dedicated real-time processor that executes the deterministic permission path. Between admission and onset, an EtherCAT fieldbus[2] delivers target setpoints to individual motor drives, where power-inverter gate drivers switch to ramp up stator current. Neural inference occupies one of the eight stages. Under memory bus contention and scheduling delays, the peripheral system stages expand the latency tail substantially more than the neural network execution itself.

Systems engineers sometimes estimate worst-case pipeline latency by summing the 99th-percentile (p99) latencies of individual stages. However, the percentile of a sum does not equal the sum of the percentiles.[3] In embedded system-on-chip (SoC) architectures, shared hardware resources couple stage delays. For example, PCIe DMA bursts during camera readout contend for memory-controller bandwidth with neural accelerators, while CPU cache evictions during kernel message passing induce thread scheduling jitter. The end-to-end distribution of sense-to-torque latency must therefore be profiled empirically on the integrated hardware platform under concurrent production workloads. Furthermore, an empirical percentile remains a statistical measurement; it does not replace the formal worst-case bounds required for hard real-time safety guarantees. Under continuous inference load, each stage in table 2.1 carries its own tail, and the two bottom rows separate the arithmetic column sums from the end-to-end age the system actually delivers.

Table 2.1: Illustrative sense-to-actuation latency decomposition: Stage and system-level timings show why the arithmetic sum of stage percentiles is not the end-to-end tail. The example's p99.9 of 68.4 ms exceeds the 60 ms freshness deadline of the mug on the pick-station conveyor, although its median fits.

Pipeline Stage	Physical Mechanism / Contention Source	p50 (ms)	p99 (ms)	Max (ms)
1–2. Sensor Exposure & Readout	60 Hz rolling shutter exposure midpoint	11.2	16.6	16.7
3–4. Host Transport & DMA	PCIe DMA transfer into host memory	1.4	3.2	6.8
5. Vision Backbone & Policy	Onboard accelerator inference (memory throttling tail)	18.5	24.1	38.4

[1] **MIPI CSI-2 (Camera Serial Interface 2)**: The Mobile Industry Processor Interface camera link carries raw frames from the image sensor to the system-on-chip and marks the start and end of each frame with short packets. Latching a hardware timestamp on the frame-start packet ties the frame to a fixed offset from its exposure rather than to the moment software first sees it, which is what lets transduction time, not arrival time, anchor the age budget.

[2] **EtherCAT**: An industrial Ethernet fieldbus in which one frame passes through every drive on the segment, each node reading and writing its own slice of the frame as it passes. Frame length and node count, not contention for the wire, fix the cycle time, which is why the fieldbus row of table 2.1 has equal p50 and p99. Section 4.5.3 sizes that cycle for the running machine.

[3] **Quantile Convolution**: In general, the quantile of a sum of delays does not equal the sum of their individual quantiles ($Q_p(X+Y) \neq Q_p(X) + Q_p(Y)$). Even when stages are independent, adding their p99 values is not a worst-case bound, and shared buses and queues correlate the stages so that the end-to-end tail must be measured across the whole pipeline.

Pipeline Stage	Physical Mechanism / Contention Source	p50 (ms)	p99 (ms)	Max (ms)
6. OS Scheduling & IPC	Kernel preemption and mailbox hand-off to safety microcontroller	0.6	2.1	12.3
7. Fieldbus Transmission	Deterministic EtherCAT cyclic packet exchange	1	1	2
8. Actuator Current Rise	Gate driver switching and stator flux buildup	2.3	3	4.5
Arithmetic Stage Sum	Sum of each column's stage values; not an end-to-end quantile	**35**	**50**	**80.7**
End-to-End Age	System-level profile, capture to onset; max bounded within the illustrative profile	**35**	**46.2**	$\leq$ **80.7**

The 50 ms sum of individual stage percentiles overstates the system-level p99 latency of 46.2 ms because pipeline stages rarely experience worst-case contention simultaneously. For the same reason, the sum of the stage maxima enters the end-to-end row only as an upper bound within the stated illustrative profile and conditions. A measured maximum needs a separately justified allowance or bound before it can support a hard real-time deadline. Coupling pushes the other way. Stages that slow down together, because they contend for the same memory controller or scheduler, fatten the end-to-end tail toward the stage sum, while stages that vary independently usually leave that sum pessimistic, as here. The end-to-end median in the table is set equal to the stage-median sum, a simplification of the illustrative profile that a measured one need not share. Notebook 2.1 evaluates how this illustrative profile behaves against the conveyor's physical deadline.

Napkin Math 2.1: The freshness deadline

Problem: The arm grasps a mug riding the pick-station conveyor, acting on camera observations of the handle. Suppose the grasp path had the illustrative age profile of table 2.1. Does it deliver those observations within the freshness deadline, and how far does the mug move at the median and at the tail of the observation age?

Variables:

- Grasp tolerance $r_{\text{tol}} = 15$ mm, of which the initial estimate spends $e_0 = 3$ mm (illustrative)
- Conveyor speed $v_{\max} = v_{\text{conv}} = 0.20$ m/s (illustrative)
- Median backbone time 18.5 ms, median end-to-end age 35 ms, and p99.9 age 68.4 ms (illustrative profile of table 2.1)

Math:

1. **Deadline**: $e_{\max} = r_{\text{tol}} - e_0 = 12$ mm, so $\Delta t_{\text{fresh}} = e_{\max}/v_{\max} = 12$ mm / 0.20 m/s $= 60$ ms (equation 2.1).
2. **Backbone alone**: 18.5 ms is less than 60 ms, so the one stage a benchmark times fits with room to spare.
3. **Median chain**: 0.20 m/s $\times$ 35 ms $= 7$ mm of mug travel, 5 mm inside the tolerance.
4. **Tail**: 0.20 m/s $\times$ 68.4 ms $= 13.7$ mm, 1.7 mm past the tolerance.

Result: Inference and the median chain both meet the deadline, but the stated p99.9 age exceeds it. The profile therefore does not establish that 99.9 percent of grasps meet the freshness deadline.

Systems insight: The whole path from exposure to torque onset spends the freshness deadline, not the model alone, and its tail, not its median, decides whether the grasp holds. When the machine instead drives toward a stationary obstacle, the same age enters the pre-brake delay τ_{delay} as its first term and is paid as blind travel $v\,\tau_{\text{delay}}$, the first term of the stopping distance that section 2.3 sets against the clear distance.

Excessive feedback delay can also degrade closed-loop tracking or destabilize motor control loops,[4]

[4] **Transport Delay and Feedback Stability**: Transport delay makes a controller act on historical state. Whether delayed corrections destabilize the loop depends on the plant, controller, and total response. Section B.5.5 develops the phase-margin calculation; delay alone does not establish a particular oscillation or damage outcome.

but the freshness calculation requires no frequency-domain model: at the latency tail, the mug has already moved beyond mechanical capture tolerance. System architectures must therefore preserve the observation's true physical capture timestamp. Hardware timers synchronized through **IEEE 802.1AS**, the generalized Precision Time Protocol (gPTP),[5] or hardware capture strobes associated with the MIPI CSI-2 receiver provide a common physical timebase. The sensor's exposure and readout characteristics determine how that hardware timestamp maps to the measurement midpoint. Once latched, this capture timestamp accompanies the observation through every subsequent compute kernel.

At the admission boundary, the real-time permission path computes the observation's total elapsed age, $t_{\text{age}} = t_{\text{current}} - t_{\text{transduction}}$, and adds the bounded remaining admission-to-onset interval for fieldbus dispatch and motor current rise. Capture and clock-synchronization uncertainty must also fit the qualified age budget. The permission path rejects the proposed trajectory update if its projected onset age exceeds the freshness deadline. Rejecting a stale trajectory replacement does not require cutting motor torque immediately; an executing command may continue while its qualified physical margin remains valid. If that margin expires before fresh, valid observations arrive, the permission path triggers a qualified fallback maneuver within the remaining stopping distance (Brooks 1986).

[5] **IEEE 802.1AS (gPTP)**: The generalized Precision Time Protocol of IEEE 802.1AS is a profile of the IEEE 1588 Precision Time Protocol (IEEE 2020a) for time-sensitive Ethernet networks (IEEE 2020b). It synchronizes clocks by exchanging packets timestamped in hardware at the network interface, which keeps the operating-system scheduler's jitter out of the measurement. The age $t_{\text{age}} = t_{\text{current}} - t_{\text{transduction}}$ means something only when the camera, the host, and the drives read one timebase; an offset between their clocks enters the freshness budget as an error that no timestamp reveals.

> **Systems Perspective 2.2: The publish timestamp anti-pattern**
>
> A widespread architectural bug in robotics systems is timestamping sensor observations inside user-space publishers, such as a Robot Operating System 2 (ROS 2) node transmitting over Data Distribution Service (DDS) middleware. Latching timestamps in application software discards the duration of sensor exposure, rolling-shutter integration, sensor readout, DMA transfers, and kernel queueing. The latency budget therefore loses stages 1 through 4 of table 2.1 (exposure, readout, transport, and DMA), along with any kernel queueing before the publisher runs. Hardware strobes must latch timestamps directly at the physical silicon interface (such as MIPI CSI-2 Start-of-Frame interrupts or Ethernet physical-layer transceiver timestamps synchronized via IEEE 802.1AS) before data enters operating-system queues.

Bounding measurement age ensures that control policies act on valid observations rather than historical artifacts, but freshness alone cannot guarantee collision avoidance. Throughout that capture-to-onset interval, the body continues moving under unguided momentum, consuming physical clearance before braking can begin. Halting that physical mass requires dissipating its stored kinetic energy across real distance, an irreversible mechanical process governed by momentum rather than processor clock rates.

2.3 Kinetic Momentum

When the loaded mobile manipulator drives down a warehouse aisle at its 1.5 m/s drive limit and a pedestrian steps out from behind a rack 1.10 m ahead, the capture-to-onset age from section 2.2 becomes unguided travel toward the pedestrian. Even after the brake command arrives, halting the base is not instantaneous: the moving chassis carries linear momentum ($p = mv$) and stores kinetic energy ($E_k = \frac{1}{2}mv^2$) that must be dissipated across real space through mechanical work ($W = \int F\,dx$). Stopping distance grows linearly with pre-brake delay but quadratically with velocity, so above the crossover speed of the elementary stopping budget, the braking term rather than the delay dominates the stop.

2.3.1 Kinematic stopping distance and pre-brake delay

As in the elementary stopping budget, a stop divides into two phases, pre-brake delay and active braking. During the pre-brake delay τ_{delay}, the vehicle travels forward without any retarding force. This latency accounts for the age of the sensory observation when perception hands it downstream, followed by the communication transit, safety validation, and actuator-onset intervals required before mechanical braking begins. It excludes the dispatch-to-admission interval, in which the proposer and the permission path turn that observation into an admitted command, because section 12.3.1 charges that interval separately. An independent protective response need not wait for a motion

planning proposal; safety enforcement can trigger immediately along a fast-path monitor. The safety budget must trace whichever concrete execution path initiates braking. Furthermore, if an obstacle can enter the path between sensing cycles, the budget must include an allowance for camera exposure and sampling intervals. The budget conservatively credits zero retarding force during this total pre-brake delay and assumes approximately constant forward speed v, which gives the blind travel distance

$$d_{\text{blind}} = v\,\tau_{\text{delay}}$$

Once the selected braking mechanism engages, the system enters the active braking phase. Mechanical work done by the retarding braking force F_{brake} over the physical braking distance d_{brake} must dissipate the machine's stored kinetic energy, $W = \Delta E_k$. How that retarding force arises depends on the physical hardware. A powered motor stop requires functioning drive inverters, sufficient bus-voltage headroom, and a circuit capable of absorbing or dissipating returned electrical power. A mechanical brake, by contrast, operates independently of electrical power but carries distinct engagement latencies and thermal dissipation limits. In any safety certification, the credible deceleration must reflect the specific mechanism available during the fault scenario under analysis.

↩ The two-phase stopping budget is established in section 1.4.3.

In an embodied machine, stored kinetic energy includes rotating powertrain components as well as the translating chassis. A component spinning at angular velocity ω stores rotational energy $\frac{1}{2}J\omega^2$, where its moment of inertia J (in $\text{kg}\cdot\text{m}^2$) measures resistance to rotational acceleration. It plays the same role that mass plays in translational energy $\frac{1}{2}mv^2$. A transmission with **gear ratio** N_{gear} turns the motor shaft N_{gear} times for every revolution of the wheel, multiplying motor torque by N_{gear} while dividing speed by N_{gear}. The machine's total kinetic energy sums the translational energy of the chassis and payload ($E_{k,\text{trans}} = \frac{1}{2}mv^2$) and the rotational energy of its wheels, gearboxes, and motor rotors ($E_{k,\text{rot}} = \sum_i \frac{1}{2}J_{\text{rotor}_i}{\omega_{m_i}}^2$). Because a drive with gear ratio N_{gear_i} spins its rotor at angular speed $\omega_{m_i} = N_{\text{gear}_i}\frac{v}{r}$, the rotor's rotational kinetic energy reflects into the translational frame scaled by the square of the gear ratio:

$$E_{k,\text{total}} = \frac{1}{2}mv^2 + \sum_i \frac{1}{2}J_{\text{rotor}_i}\left(N_{\text{gear}_i}\frac{v}{r}\right)^2 = \frac{1}{2}\left(m + \sum_i \frac{{N_{\text{gear}_i}}^2 J_{\text{rotor}_i}}{r^2}\right)v^2$$

This formulation reveals that the effective translational inertia of the machine is $m_{\text{eff}} = m + \sum_i \frac{{N_{\text{gear}_i}}^2 J_{\text{rotor}_i}}{r^2}$. For a platform decelerating to rest under sustained retarding force $F_{\text{brake}} = m_{\text{eff}}a_{\text{brake}}$, the work-energy theorem balances mechanical work against total kinetic energy:

$$m_{\text{eff}}a_{\text{brake}}d_{\text{brake}} = \frac{1}{2}m_{\text{eff}}v^2$$

Dividing both sides by effective mass yields the kinematic braking travel:

$$d_{\text{brake}} = \frac{v^2}{2a_{\text{brake}}}$$

If an engineer evaluates braking capacity using bare chassis mass alone ($a_{\text{nominal}} = F_{\text{brake}}/m$), they omit the kinetic energy stored in the spinning motor rotors. The result overestimates deceleration by the factor m_{eff}/m and underestimates stopping travel by $\frac{m_{\text{eff}}-m}{m} = \sum_i \frac{{N_{\text{gear}_i}}^2 J_{\text{rotor}_i}}{mr^2}$. On the running machine, this correction remains modest. Its two driven wheels use direct-drive hub motors ($N_{\text{gear}_i} = 1$), and because no portion of a hub motor lies outside its tire circumference, each rotor term J_{rotor_i}/r^2 cannot exceed the motor's 9.5 kg total mass. The rotating parts add at most 5.2 percent to the 368 kg loaded mass. However, because the inertia reflection scales with ${N_{\text{gear}_i}}^2$, this rotational overhead becomes substantial on lightweight robot platforms driven through high-ratio planetary or harmonic gearboxes.

The parameter a_{brake} is the credible sustained deceleration the physical plant can reliably exert against the environment, not an idealized rating copied from an actuator datasheet. The retarding force $F_{\text{brake}} = m_{\text{eff}}a_{\text{brake}}$ must fit beneath three independent physical ceilings. First, tire-floor traction imposes a strict friction ceiling. Only the braked wheels transmit retarding shear to the ground,

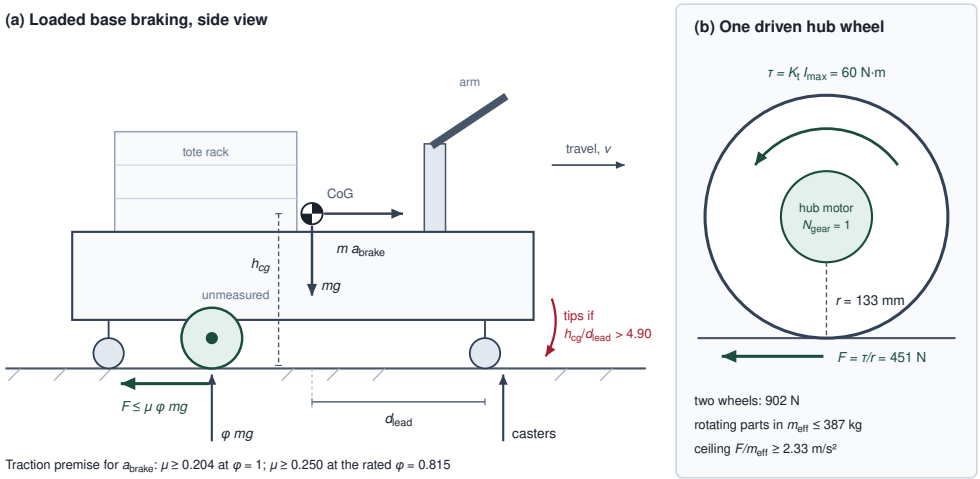


Figure 2.1: Three ceilings on the braking base: (a) Only the driven wheels brake against the floor, so friction there caps the retarding force at $\mu\phi_{\text{drive}}mg$; the credible 2 m/s² needs $\mu \geq 0.204$ with all the weight on those wheels and 0.250 at their rated load share. The inertial load ma_{brake} acts at the center of gravity, a height h_{cg} above the floor, and pitches the body about its leading support unless $h_{\text{cg}}/d_{\text{lead}} \leq 4.90$; neither length has been measured. (b) Each direct-drive hub wheel turns its 60 N·m current-limited torque into a floor force τ/r, 902.3 N for the pair, and the spinning parts are already counted in m_{eff}. The layout is schematic, not to scale.

and friction there can reach at most μ times the normal force they support, $\phi_{\text{drive}}mg$. This caps deceleration at $a_{\text{brake}} \leq \mu g\phi_{\text{drive}}$, where μ is the tire-floor friction coefficient and ϕ_{drive} is the fraction of total vehicle weight carried by the driven wheels. Second, vehicle pitch stability limits deceleration through dynamic load transfer. While braking force acts at the floor contact patch, inertial force acts forward through the center of gravity at height h_{cg} above it. Taking moments about the leading ground support, a horizontal distance d_{lead} ahead of the center of gravity, the overturning inertial moment $ma_{\text{brake}}h_{\text{cg}}$ opposes the restoring gravitational moment mgd_{lead}. The platform stays on all wheels only so long as $a_{\text{brake}} \leq g\, d_{\text{lead}}/h_{\text{cg}}$. Raising or extending a robotic arm alters both h_{cg} and d_{lead} in real time, making this pitch-stability boundary state-dependent. Third, the electromechanical actuators limit retarding force through the maximum torque their inverter current limit allows, a ceiling that section 2.4.2 derives from the motor torque constant. If an inverter restricts phase current, or if the back-EMF of the spinning rotor leaves the inverter too little headroom to control current, the drive cannot deliver the required braking torque. Deceleration a_{brake} degrades silently, expanding braking travel without software notification. The rotor's rotational inertia is already counted in the effective mass m_{eff} and must not be charged a second time. These three ceilings act simultaneously on the loaded chassis (figure 2.1), and notebook 2.3 checks them against the running machine's numbers once the next section derives the drive's torque ceiling.

The constant-deceleration model requires a level surface and a qualified braking response. A conservative implementation charges the transient interval required to develop full retarding force into τ_{delay}, then applies a certified lower bound a_{brake}. The actuator brake-onset allowance (T_{act}) enters that delay once. Any subsequent calculation that models a finite torque ramp must reconcile its ramp travel with this allowance rather than add the same interval twice. Furthermore, empirical stopping trials across declared operating conditions establish the credible deceleration, its distribution, and its safety margin. A sample percentile alone does not constitute a certified lower bound.

In unstructured environments, an autonomous machine cannot allocate its entire clearance to nominal stopping dynamics. Obstacle tracking carries state estimation uncertainty, sensor calibration drifts over time, and localization algorithms introduce position error. The stopping distance d_{stop} that the system defends must therefore include a spatial localization uncertainty bound δ_{loc} and a minimum physical clearance margin δ_{margin}:[6]

6 **Protective Separation Distance**: For speed and separation monitoring of robot arms, the collaborative-application requirements that ISO 10218-2:2025 incorporates from ISO/TS 15066 (which itself remains published and under revision) write the protective separation distance as $S_p = S_h + S_r + S_s + C + Z_d + Z_r$: the person's travel, the robot's travel during its reaction time and while stopping, an intrusion distance set by the sensing, and the position uncertainties of person and robot (International Organization for Standardization 2016, 2025b; Marvel and Norcross 2017). The robot's terms map onto equation 2.2, with $v\,\tau_{\text{delay}}$ as S_r, $v^2/2a_{\text{brake}}$ as S_s, and δ_{loc} as Z_r, although ISO 10218-2:2025 excludes the hazards specific to a manipulator riding a mobile platform, the running machine's case. For a learned policy, an unbounded compute tail lengthens the reaction time, which enlarges both S_h and S_r and forces the protective field outward.

$$d_{\text{stop}} = v\,\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} \tag{2.2}$$

The sum $\delta_{\text{loc}} + \delta_{\text{margin}}$ is fixed spatial overhead that must remain unoccupied when the body comes to a complete rest.

↪ The localization bound δ_{loc} and its dependence on map corrections are developed in section 9.5.1.

Definition 2.2: Dynamic stopping distance

Dynamic stopping distance (d_{stop}) is the minimum physical clearance required to bring a moving mass to a complete halt from its speed v when the hazard is first imaged. It sums three parts. The first is the travel during the pre-brake delay τ_{delay}, which spans the observation age of the frame that shows the hazard and the lease, tick, bus, and brake-onset intervals that follow until retarding force begins (the decomposition of section 1.4.3), and which excludes the separately charged dispatch-to-admission interval. The second is the braking travel under the credible deceleration a_{brake}. The last is the localization bound δ_{loc} and protective clearance δ_{margin}, as equation 2.2 states.

1. **Significance**: Quantifies the non-negotiable physical spatial margin that software proposals must clear before motion permission can be granted.
2. **Distinction**: Unlike static geometric obstacle clearance in classical kinematics, dynamic stopping distance depends on the machine's own speed and on every delay before braking, so each velocity setpoint and each delay term changes it.
3. **Common pitfall**: Assuming an emergency stop command halts the machine instantaneously, ignoring that moving mass carries kinetic energy ($E_k = \frac{1}{2}mv^2$) that requires finite mechanical work and braking distance to dissipate.

Three cases on the running machine show how the terms trade against one another (figure 2.2). With only the terms the body owns, the stop needs 742.5 mm. Charging the full 133.6 ms pre-brake delay of the elementary stopping budget (section 1.4.3), whose terms later chapters measure one by one, gives that elementary stop plus the 150 mm fixed overhead, or 912.9 mm. That stop still fits the 1.10 m clear distance. Raising speed above the drive limit with that same delay pushes the stop to 1,417.2 mm, past the rack end, because speed enters the blind travel linearly and the braking travel quadratically. Notebook 2.2 works the first case term by term.

2.3.2 Dynamic velocity ceilings and clearance budgeting

Because braking travel grows quadratically with speed while blind travel scales linearly with delay (figure 2.2), autonomous systems cannot rely on static, scene-independent speed limits. They require a dynamic velocity ceiling parameterized by instantaneous obstacle clearance. For a stationary obstacle detected at distance D_{clear}, such as a person standing at an aisle intersection or a dropped storage tote, safe operation demands that defended stopping distance remain strictly within the available clear path: $d_{\text{stop}} \leq D_{\text{clear}}$. This constraint is how the first law, irreversibility (principle 1), binds every motion cycle, with each parameter in d_{stop} grounded in the machine's verified physical limits. Using conservative bounds for actuation delay and retarding acceleration, the permission path solves this inequality to determine the maximum permissible command velocity v_{max}. The symbol is the one that equation 2.1 used for the speed of a tracked target, but here it bounds the machine's own speed. Taking the positive physical root of the resulting quadratic relation[7] yields:

$$v_{\text{max}} = -a_{\text{brake}}\tau_{\text{delay}} + \sqrt{(a_{\text{brake}}\tau_{\text{delay}})^2 + 2a_{\text{brake}}(D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}})} \tag{2.3}$$

Whenever perceptual range degrades or an obstacle obstructs the trajectory, D_{clear} contracts, continuously lowering the velocity ceiling. When available distance collapses to zero, the allowable forward velocity becomes zero. If the machine is already in motion, however, issuing a zero-velocity setpoint cannot instantaneously eliminate its accumulated kinetic energy. A negative available distance signals that the platform has already breached safe stopping margins, demanding immediate, full-authority emergency braking.

[7] **Kinematic Velocity Root**: Grouping the fixed distance terms into the available braking distance $D_{\text{avail}} = D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}}$ gives the quadratic inequality $\frac{1}{2a_{\text{brake}}}v^2 + \tau_{\text{delay}}v - D_{\text{avail}} \leq 0$. Multiplying by $2a_{\text{brake}}$ gives $v^2 + 2a_{\text{brake}}\tau_{\text{delay}}v - 2a_{\text{brake}}D_{\text{avail}} \leq 0$. Applying the quadratic formula and taking the positive root sets the speed ceiling.

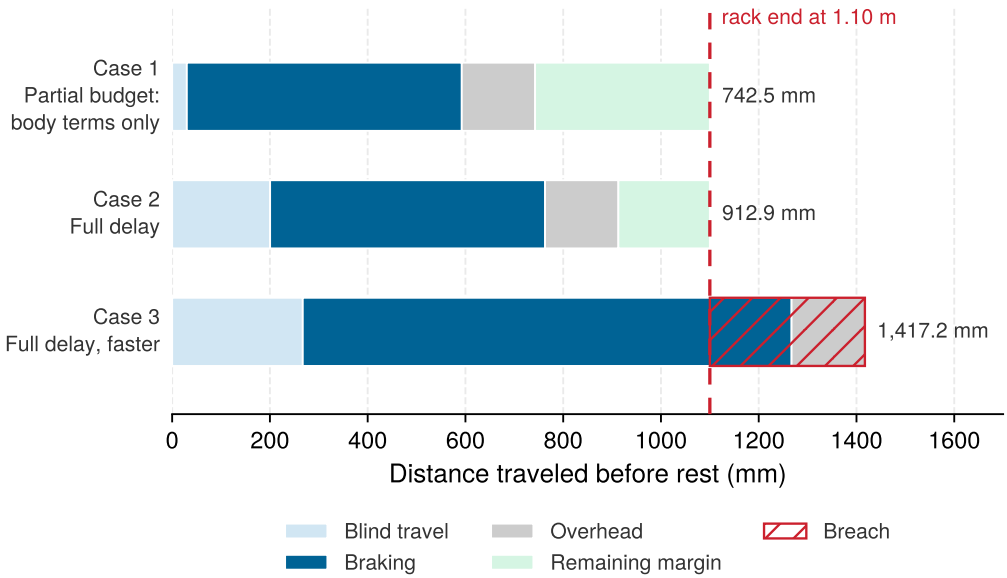


Figure 2.2: Stopping distance of the warehouse mobile manipulator: Each bar splits the stop into blind travel during the pre-brake delay, braking travel under the credible deceleration, and the fixed overhead of the localization bound and protective clearance, against the 1.10 m rack-end clear distance. Case 1 counts only the terms the body owns at the 1.5 m/s drive limit and stops within 742.5 mm. Case 2 charges the full 133.6 ms pre-brake delay of the elementary stopping budget (the kinematic roofline); at 912.9 mm it still fits, although later chapters add a stop profile and a tracking bound. Case 3 raises speed to a hypothetical 2 m/s, above the drive limit, with the same delay; braking travel grows by 77.8 percent and the stop, at 1,417.2 mm, breaches the clear distance.

DRAFT

Napkin Math 2.2: The body's stopping distance

Problem: The loaded warehouse mobile manipulator drives toward a rack end at its drive limit, and a person stands in the aisle at the clear distance. The body owns three terms of equation 2.2: braking travel, the fixed overhead, and the brake onset that enters τ_{delay}. How much of the clear distance do these terms use, what speed would they alone permit, and how do speed and added delay change the answer?

Variables:

- Loaded mass at most 368 kg (derived upper bound, full tote rack) and drive limit $v = 1.5$ m/s (illustrative)
- Clear distance $D_{\text{clear}} = 1.10$ m (illustrative)
- Credible deceleration $a_{\text{brake}} = 2$ m/s² (illustrative; loaded, dry floor)
- Localization bound $\delta_{\text{loc}} = 50$ mm (illustrative) and protective clearance $\delta_{\text{margin}} = 100$ mm (chosen)
- Brake onset $T_{\text{act}} = 20$ ms (illustrative)

Math:

1. Braking travel: For the same credible deceleration, mass cancels from the work-energy balance. The loaded machine's 414 J sets the energy the stopping system must absorb or dissipate, while the specified deceleration sets the braking distance. A different payload can change that deceleration and requires a new bound:

$$d_{\text{brake}} = \frac{v^2}{2a_{\text{brake}}} = \frac{(1.5m/s)^2}{2 \times 2m/s^2} = 562.5mm$$

The stopping distance so far is 562.5 mm.

2. Fixed overhead:

$$\delta_{\text{loc}} + \delta_{\text{margin}} = 50mm + 100mm = 150mm$$

The running total rises to 712.5 mm.

3. Brake onset: Between the brake command and the first retarding force, the base still moves at full speed:

$$vT_{\text{act}} = (1.5m/s) \times (20ms) = 30mm$$

$$d_{\text{stop}}\big|_{\text{body}} = 562.5mm + 150mm + 30mm = 742.5mm$$

The body's own terms leave 357.5 mm of the 1.10 m clear distance for every delay the body does not own.

4. The body-only ceiling: Solving equation 2.2 for speed with $\tau_{\text{delay}} = T_{\text{act}}$:

$$v_{\text{max}} = -a_{\text{brake}}T_{\text{act}} + \sqrt{(a_{\text{brake}}T_{\text{act}})^2 + 2a_{\text{brake}}(D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}})} \approx 1.91m/s$$

The body alone would permit 1.91 m/s, above the drive limit. The remaining delay terms, added in later chapters, spend part of this headroom.

5. Speed vs. delay: At a hypothetical 2 m/s, above the drive limit, the braking term alone grows to 1,000 mm, 77.8 percent more than at the drive limit. At the drive limit, each additional 100 ms of pre-brake delay adds 150 mm.

Systems insight: Platform velocity is the most sensitive variable in the stopping budget. Accelerating from 1.5 m/s to 2 m/s adds 437.5 mm in braking travel, as much distance as 291.7 ms of additional pre-brake delay at the drive limit. A velocity setpoint may therefore rise only after re-evaluating the clearance quadratic. Because physical braking distance scales with v^2, granting an aggressive velocity setpoint expands the defended stopping envelope and rapidly exhausts the clear distance margin defended by $d_{\text{stop}} \leq D_{\text{clear}}$.

2.3.3 Applying the velocity ceiling

A dynamic speed ceiling protects the machine only when an independent check on the permission path tests each new velocity request against it before execution. The learned policy proposes an action. The safety check evaluates clearance, delay, and available deceleration to determine whether that request fits the stopping budget. Requesting a lower speed alters future trajectory points but cannot erase the distance required to stop from the current speed. Consequently, the check must evaluate the platform's instantaneous state alongside candidate commands, keeping the machine inside the region from which it can still stop between control ticks under strict bounds on timing jitter, obstacle motion, and actuator response. That per-tick validation belongs to the permission path; section 12.3.2 formalizes it around the jerk-limited deceleration profiles executed on the physical platform. What the physical body contributes is the ceiling itself: a clearance-dependent boundary grounded in physical constraints that drops whenever traction degrades, actuator response slows, or localization uncertainty expands.

Enforcing this speed limit assumes the permission path can demand the deceleration a_{brake} whenever safety requires it. Commanding rapid deceleration, however, requires sudden, large mechanical torque from the drive motors and gearboxes. If the permission path requests a torque step that exceeds what the actuator bandwidth budget allows, the mechanical body cannot deliver the required deceleration, invalidating the model-based stopping ceiling and risking collision. The same drives also shape what happens if the stop falls short. The reflected rotor energy counted in m_{eff} then reaches the obstacle and the gear teeth through the transmission's compliance, so the stopping calculation cannot substitute for a contact-load assessment.

2.4 Actuator Transmission Limits

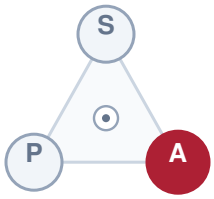
When a policy writes a 50 N·m torque setpoint to a joint register or dispatches an emergency brake command over a Controller Area Network (CAN) bus or EtherCAT, the drive electronics may clamp

↪ The chunk lease, the time an admitted trajectory segment may keep executing without a replacement, is one of the delay terms that spend this headroom. Its sizing from this stopping distance is developed in section 4.4.

the current at half that request while returning a clean acknowledgement packet in sub-millisecond time. In digital software, an execution target either accepts an instruction or raises a fault; in physical hardware, stator winding inductance, DC bus voltage headroom, magnetic saturation, and transmission elasticity bound what the joint actually delivers. The systems engineering challenge is to specify an actuator contract at the physical plant interface, written in delivered force, torque, slew-rate limits, response latency, and tracking error rather than digital setpoints, and to expose those limits directly to the runtime permission path before an unachievable command causes a collision.

2.4.1 Actuation mechanisms and the actuator contract

What that actuator contract must bound depends on how the joint converts stored energy into mechanical force. Electric drives dominate mobile manipulation and power the running machine of this book, where current in copper windings pulls on rotor magnets through a gear train. Other robots rely on pressurized fluid or calibrated mechanical springs, and each mechanism trades peak force density against bandwidth, compliance, and maintenance overhead.



On the sense-plan-act (S·P·A) triad (Appendix D), the actuator contract sits at the Act node, where a command must become delivered force.

Across the broader field, three non-rigid or fluid alternatives illustrate how the physical medium reshapes the contract:

- **Hydraulic servovalves**: For heavy machinery, construction robots, and high-load dynamic platforms, fluid power trades magnetic force for pressure. A cylinder's force is supply pressure times piston area, $F_{\text{cyl}} = pA$, so raising the pressure lets a compact actuator deliver a large force. An electric motor's force, by contrast, is bounded by the magnetic field its magnets and copper can sustain. The miniature servovalves on hydraulic legged robots respond fast, with bandwidths of hundreds of hertz (Scully 2015), so the valve is rarely the slow element. Hydraulic control is bounded instead by fluid compressibility (oil bulk modulus β), nonlinear orifice flow, and the plumbing, pumps, and cooling needed to keep the fluid within its operating range. That maintenance burden is one reason Boston Dynamics retired its hydraulic Atlas humanoid in 2024 in favor of an all-electric successor (Boston Dynamics 2024).
- **Pneumatics and soft actuation**: Pneumatic artificial muscles and elastomeric chambers provide physical compliance that naturally cushions impacts. However, gas compressibility, valve flow restrictions, and acoustic transport delays limit closed-loop bandwidth to tens of hertz. Stored pneumatic energy can also discharge violently during rapid venting or line rupture. Safety validation requires qualifying the envelope of peak chamber pressure, flow rates, and measured contact dynamics.
- **Series elastic actuators (SEAs)**: By placing a calibrated spring directly between the gearbox and the output link (as in the ANYbotics ANYdrive), an SEA incorporates mechanical compliance into the drivetrain itself. The spring acts as a low-pass mechanical filter: it isolates delicate gear teeth from shock loads and decouples the large reflected rotor inertia ($N_{\text{gear}}^2 J_{\text{rotor}}$) from the environment during impact. Its measured deflection also serves as a direct force sensor, since spring stiffness times deflection yields output torque (Pratt and Williamson 1995). Rather than dissipating energy, the spring stores elastic potential ($E = \frac{1}{2} k \Delta\theta_{\text{joint}}^2$) at the joint and can return that energy into an obstacle. The engineering trade-off is reduced closed-loop position bandwidth and low-frequency structural resonance. The safety system and permission path must therefore monitor physical spring deflection directly instead of assuming rigid load tracking.

2.4.2 Electrical time constants, back-EMF, and current clamping

The running machine's wheels and arm joints are electric, so the electric drive is the plant whose limits the stopping budget depends on. To a policy, a torque command is a number written to a register; to the motor, it is a current that must be built up in a coil. Three results from introductory physics describe that conversion (Lynch and Park 2017). First, a wire carrying current across a magnetic field feels a force proportional to the current. Wound around a rotor, those forces produce a torque proportional to current, $\tau_{\text{motor}} = K_t I$, where the **torque constant** K_t (in $\text{N} \cdot \text{m/A}$) collects the magnet strength and the winding geometry. Second, the same wires, swept through the same field by the spinning rotor, have a voltage induced in them. A spinning motor is therefore also a

generator, and its **back-EMF** (back electromotive force) $e_{\text{bemf}} = K_e\omega_m$ opposes the supply. Energy conservation ties the two constants together. The electrical power the motor converts, $e_{\text{bemf}}\,I$, equals the mechanical power it delivers, $\tau_{\text{motor}}\omega_m$, so K_e and K_t are the same number in SI units. A three-phase drive's current and voltage conventions scale them by constant factors, which is why both must be identified under one convention. Third, the winding is a coil, and a coil opposes any change in its current, $v = L_{\text{phase}}\,dI/dt$ for inductance L_{phase}. Current, and therefore torque, cannot jump.

The budget needs two results from the motor controller: the torque delivered for a given current and the delay before that current appears. In a three-phase motor, field-oriented control (FOC) measures the winding currents and adjusts the inverter voltages so that the torque-producing current i_q stays at right angles to the rotor's magnetic field as the rotor turns. Torque is then $\tau_{\text{motor}} = K_t i_q$.[8] The controller splits the stator current into two components that turn with the rotor. The quadrature, or q-axis, current i_q lies at right angles to the rotor field and produces torque. The direct, or d-axis, current i_d lies along the field, produces none, and is normally held at zero. The coordinate transformations inside the drive appear in section 2.4.6.

The delay between commanding a torque and delivering it begins in the stator windings. Adding the three effects around one winding circuit, the supply voltage must cover the resistive drop, the inductive voltage, and the back-EMF, which is Kirchhoff's voltage law for the torque-producing current:

$$L_{\text{phase}}\frac{di_q(t)}{dt} + R_{\text{phase}}i_q(t) = v_q(t) - e_{\text{bemf}}(t)$$

where L_{phase} is phase inductance, R_{phase} is phase resistance, $v_q(t)$ is the applied quadrature voltage from the inverter bridge, $i_q(t)$ is the quadrature-axis (torque-producing) current, and $e_{\text{bemf}}(t) = K_e\omega_m(t)$ is the back-EMF. The stator electrical time constant is:

$$\tau_e = \frac{L_{\text{phase}}}{R_{\text{phase}}}$$

DRAFT

If the inverter applied an uncontrolled voltage step, the current would approach its steady state exponentially along $i_q(t) = I_{\text{final}}(1 - e^{-t/\tau_e})$, taking an entire time constant τ_e to reach 63 percent of the target. A closed-loop current controller drives the winding much faster. It applies full bus voltage until current reaches the setpoint, forcing a ramp rate of nearly $(V_{\text{bus}} - e_{\text{bemf}})/L_{\text{phase}}$. At that point the current loop throttles the voltage to maintain regulation. On the running machine, this electrical rise is rapid. The hub motor windings (line inductance 1.885 mH, line resistance 0.7 Ω) yield $\tau_e = 2.7$ ms. Starting from rest on a depleted 42 V pack, the initial current slope gives a resistance-free rise estimate of 1.1 ms, only 5.6 percent of the 20 ms brake onset budgeted for the platform. The resistive drop slows the ramp as current builds, so this estimate is optimistic and the full winding response must be measured. When braking from forward speed, the rotor continues spinning in the positive direction while the controller reverses the sign of the torque-producing current to oppose motion, so the positive back-EMF reinforces the inverter's negative current drive, accelerates the current ramp further, and reduces the terminal voltage required to sustain peak braking current from $e_{\text{bemf}} + I_{\text{max}}R$ to $e_{\text{bemf}} - I_{\text{max}}R$. In this architecture, electrical rise time does not bottleneck emergency braking. Tire traction, maximum inverter current, and voltage headroom at high speed govern the stopping distance, as detailed in notebook 2.3.

Torque tracks current directly, which means available voltage headroom also dictates the maximum torque slew rate:

$$\frac{d\tau_{\text{motor}}}{dt} = K_t\frac{di_q}{dt} \le \frac{K_t(V_{\text{bus}} - K_e\omega_m)}{L_{\text{phase}}}$$

Inductance and voltage conventions must align consistently: line-to-line values reported on manufacturer datasheets pair with the full DC bus voltage. Per-phase values modeled under space-vector modulation scale by an amplitude of approximately $V_{\text{bus}}/\sqrt{3}$, the modulation voltage limit stated in section C.9. At low wheel speeds, ample voltage headroom enables steep torque ramps. As rotational velocity climbs, back-EMF consumes that headroom, throttling the rate at which the inverter can inject current. An unconstrained policy setpoint requesting an instantaneous torque step is transformed by the motor hardware into a finite physical ramp; any demand exceeding this

8 **Field-Oriented Control (FOC)**: Holding the stator current at right angles to the rotor field is the arrangement that a brushed DC motor's commutator enforces mechanically. An AC machine controlled this way behaves like a DC motor whose torque is set by a single current, i_q. That is what lets a policy command torque at all, because its request becomes a current target whose arrival the winding inductance and the voltage headroom then pace.

maximum slew rate produces severe tracking error. The safety stopping budget must account for this rate-limited response.

Voltage headroom bounds the rate of current rise; power electronics define the maximum current that can safely flow. Inverter bridges enforce a strict current ceiling I_{max} to protect the switching power transistors from thermal failure and prevent magnetic core saturation in the stator. For a motor coupled to a mechanical transmission with gear ratio N_{gear} and mechanical efficiency η_{gear}, this ceiling caps joint output torque at:

$$\tau_{\text{out,max}} = \eta_{\text{gear}} N_{\text{gear}} K_t I_{\text{max}} \tag{2.4}$$

This current limit settles a central safety requirement: whether the drives can sustain the deceleration demanded by the supervisory permission path. A motor drive may silently clamp current setpoints while continuing to acknowledge network commands; the safety path must interrogate measured output currents and status flags rather than inferring torque delivery from successful packet transmission. If emergency braking demands more torque than this ceiling allows, the assumed deceleration model breaks down. Any change to inverter current limits therefore invalidates the robot's qualified stopping envelope and requires recomputing the stopping limit record (section 2.7).

DRAFT

Napkin Math 2.3: Can the drives deliver the credible deceleration?

The running machine brakes through two driven wheels, each a direct-drive 10-inch hub motor (the ZLTECH ZLLG10ASM800, with no gearbox, so $N_{\text{gear}} = 1$ and $\eta_{\text{gear}} \approx 1$) (Shenzhen Zhongling Technology 2022) with rolling radius $r = 133$ mm. Its torque constant $K_t = 2.4$ N·m/A is the datasheet's peak torque divided by its peak current, and the drive's current limit is set at that peak, $I_{\max} = 25$ A. Casters carry the rest of the weight.

Problem: Does the loaded machine's credible deceleration $a_{\text{brake}} = 2$ m/s² fit under the traction, tip-over, and drive-torque ceilings, and does the drive keep control of its current at the 1.5 m/s drive limit?

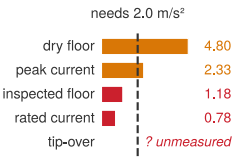
Math:

1. **Traction**: Only the driven wheels brake, so $a_{\text{brake}} \le \mu g \phi_{\text{drive}}$. With all the weight on them ($\phi_{\text{drive}} = 1$), the floor must supply $\mu \ge a_{\text{brake}}/g = 0.204$. The hub motors are rated to carry 300 kg per pair, so on the 368 kg machine $\phi_{\text{drive}} \le 0.815$ and the floor must supply at least $\mu = 0.250$. The dry floor ($\mu = 0.60$) meets that whenever the driven wheels carry at least a fraction 0.34 of the weight, and at the rated load share it supports $\mu g \phi_{\text{drive}} = 4.80$ m/s². The inspected floor, a sealed low-traction surface whose friction a facility inspection certifies only down to $\mu = 0.12$, supports only 1.18 m/s² even with every kilogram on the driven wheels.
2. **Tip-over**: The machine stays on its wheels while $a_{\text{brake}} \le g\, d_{\text{lead}}/h_{\text{cg}}$, so the credible deceleration requires $h_{\text{cg}}/d_{\text{lead}} \le g/a_{\text{brake}} = 4.90$. The machine's specification gives neither the loaded center-of-gravity height nor its distance to the leading support, and extending the arm forward shortens d_{lead}. This ceiling cannot yet be checked.
3. **Drive torque**: Each wheel delivers $K_t I_{\max} = 60$ N·m, so the floor sees $F = 2K_t I_{\max}/r = 902.3$ N. With the rotating-mass bound of section 2.3.1, $m_{\text{eff}} \le 387$ kg, and the torque ceiling is $F/m_{\text{eff}} \ge 2.33$ m/s², at least 16.6 percent above the credible deceleration. With the current limit set at the rated 8.5 A instead, each wheel gives its rated 20 N·m and the ceiling falls to 0.78 m/s².
4. **Voltage at the drive limit**: At 1.5 m/s the wheel turns at $\omega_m = v_{\max}/r = 107.7$ rpm, and the line-to-line back-EMF is $e_{\text{bemf}} = K_e \omega_m = 28.2$ V. Braking current flows with the back-EMF, so holding $I_{\max}$ through the 0.7 Ω line resistance requires only $e_{\text{bemf}} - I_{\max} R = 10.7$ V at the terminals. Even a depleted 42 V pack sits 13.8 V above e_{bemf}. Driving at the same current would need $e_{\text{bemf}} + I_{\max} R = 45.7$ V, more than the depleted pack supplies.

Result: The drives clear 2 m/s² by at least 16.6 percent, but only with the current limit at the motor's peak. Traction holds on the dry floor if the driven wheels carry at least a fraction 0.34

of the weight, and it fails on the inspected floor. The tip-over ceiling remains unverified until the loaded center of gravity is measured, by weighing each wheel on corner scales or by a tilt test, with the arm stowed and extended.

Systems insight: The credible deceleration is a conjunction of three ceilings owned by different parts of the machine: the floor, the mass distribution that the arm changes, and the drive configuration. A current limit lowered to protect the windings cuts the torque ceiling below the stopping premise without any software change, so the configured current limit belongs in the limit record's conditions.



A 2 m/s² stop must clear every ceiling; two fall short and one is unmeasured.

Current regulation also incurs electrical and switching losses. Any stator current vector misaligned with the quadrature axis produces resistive heating without contributing shaft torque. Inverter pulse-width modulation and required protective dead times introduce further nonlinear voltage distortion.[9] Consequently, dynamic torque tracking must be verified through physical dynamometer measurements on the assembled actuator rather than inferred from current setpoints alone.

2.4.3 Reflected rotor inertia and optimal inertia matching

Once electromagnetic torque leaves the motor shaft, mechanical gearing introduces a dynamic asymmetry between torque and inertia. High-reduction joints employ gear ratios between $50{:}1$ and $160{:}1$ to multiply the output torque of compact, lightweight motors. While this reduces the motor torque required by a factor of N_{gear}, the motor rotor must spin N_{gear} times faster than the actuated joint. When an external contact backdrives the joint, it must accelerate that rotor at N_{gear} times the joint's angular acceleration. Because rotational kinetic energy scales with the square of velocity ($\frac{1}{2} J_{\text{rotor}} \omega_m^2$), the rotor's physical inertia J_{rotor} resists load-side acceleration multiplied by the square of the gear ratio (definition 2.3).

Through a 100:1 transmission, a small 50 g rotor imposes the apparent inertia of a heavy 11.2 kg flywheel with a 10 cm radius at the output shaft (Lynch and Park 2017). When the end-effector collides with a rigid surface, the contact force must instantly decelerate this equivalent flywheel through the drivetrain, generating extreme peak stresses that can fracture gear teeth at their roots, as documented in figure 2.4.

Definition 2.3: Reflected rotor inertia

Reflected rotor inertia (J_{ref}) is the apparent rotational inertia of an actuator rotor at the load-side joint output, scaling with the square of the transmission gear ratio:

$$J_{\text{ref}} = N_{\text{gear}}^2 J_{\text{rotor}}$$

1. **Significance**: In high-reduction transmissions ($N_{\text{gear}} \geq 100$) it dominates the apparent load, capping joint acceleration $\ddot{q}$ and setting impact shock loads.
2. **Distinction**: Unlike rotor inertia J_{rotor} or link inertia J_{load}, it acts through transmission stiffness and backlash, so backdriving from the environment meets an N_{gear}^2 barrier.
3. **Common pitfall**: Ignoring reflected rotor inertia during collision and contact modeling. Under sudden obstacle impact, the reflected kinetic energy of the spinning rotor concentrates stress directly on gear teeth before drive current can be interrupted.

The combined equation of motion for a geared joint driving load inertia J_{load} with motor torque τ_{motor} is:

$$\ddot{q} = \frac{N_{\text{gear}} \tau_{\text{motor}}}{J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}}}$$

Maximizing delivered load acceleration $\ddot{q}$ with respect to transmission ratio N_{gear}, by setting $d\ddot{q}/dN_{\text{gear}} = 0$, gives $J_{\text{load}} = N_{\text{gear}}^2 J_{\text{rotor}}$: acceleration peaks when the reflected rotor inertia equals the load inertia. Solving for N_{gear} establishes the classical **inertia matching ratio**:[10]

[9] **Inverter Dead Time**: A short hardware-inserted interval in which both switches of an inverter half-bridge are off, so that the high-side and low-side switches of one leg never conduct together and short the DC bus, a failure called shoot-through. The interval distorts the applied voltage most at low speed and near each current zero crossing, so delivered torque departs from the request exactly where slow, fine contact force is commanded.

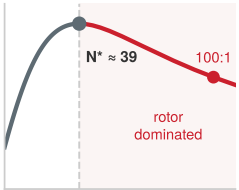
[10] **Inertia Matching**: At N^* the reflected rotor inertia equals the load inertia, the mechanical twin of impedance matching in a circuit, where a source delivers the most power to a load whose impedance equals its own. The analogy carries the same caveat: the matched ratio maximizes acceleration, not efficiency or safety, so practical arms choose lower ratios to protect gear teeth from shock, keep the joint backdrivable, and raise top speed.

$$N^* = \sqrt{\frac{J_{\text{load}}}{J_{\text{rotor}}}} \tag{2.5}$$

Below N^*, joint acceleration is torque-limited: increasing N_{gear} multiplies motor torque to accelerate the heavy load faster. Above N^*, more gearing *decreases* delivered joint acceleration, because the motor expends most of its electromagnetic torque accelerating its own spinning rotor ($N_{\text{gear}}^2 J_{\text{rotor}}$) rather than the external link. For the 50 g rotor above driving an illustrative link of $J_{\text{load}} = 8.5 \times 10^{-3}$ kg·m², the matched ratio is $N^* \approx 39$. At 100:1 the same motor torque delivers only 67.7 percent of the matched-ratio acceleration, because $\ddot{q}(N_{\text{gear}})/\ddot{q}(N^*) = 2\tilde{N}/(1+\tilde{N}^2)$ with $\tilde{N} = N_{\text{gear}}/N^*$. The extra gearing has cost 32.3 percent of the joint's acceleration.

The gear ratio also magnifies the mechanical resistance an external contact meets when attempting to move the link. This resistance to imposed motion constitutes the joint's mechanical impedance. Because link inertia, joint damping, torsional compliance, and seal friction all shape this response, the gear ratio alone does not establish contact forces or structural fracture limits. Transmission selection must balance delivered joint acceleration against the impedance presented to accidental impacts.

Abrupt changes in commanded torque excite the torsional elasticity of geared joints before the physical link responds. Bounding joint acceleration and jerk moderates this excitation, but software trajectory smoothing cannot override mechanical resonance. Trajectory generators formulate control profiles that respect these physical boundaries (Lynch and Park 2017). The hardware platform must provide the empirical response rates and contact stiffness limits required to tune them.



Past $N^* \approx 39$, gearing chokes acceleration; 100:1 keeps 67.7 percent.

↪ Trajectory timing under joint torque and jerk bounds is developed in section 11.4.5.

2.4.4 Transmission compliance, contact impedance, and tooth fracture

Mechanical gearboxes introduce backlash and elasticity that decouple the motor from the driven link. Upon direction reversal, the motor shaft traverses mechanical clearance before the gear teeth engage the opposing flank. Concurrently, as illustrated by the disassembled strain wave gear set in figure 2.3, torsional compliance in gear teeth and flexible elements[11] transforms the rigid link into an elastic system (Spong 1987). Its dynamics follow the flexible-joint formulation given in section B.5.4. Step changes in torque excite this structural spring, driving the drivetrain into limit-cycle[12] chatter. Motor-side encoders conceal this deadband completely. The motor rotates smoothly while the output shaft remains pinned, causing link position q_{out} to diverge from $q_{\text{motor}}/N_{\text{gear}}$ by a load-dependent deflection δ_{lost}. The motor controller observes flawless setpoint tracking while the end-effector stalls, eventually experiencing an impulsive impact when the gear teeth make contact. Repeated shock cycles culminate in the root fatigue failure shown in figure 2.4.

The strain wave gear of figure 2.3 packs a high reduction into a compact joint by deforming its flexspline continuously, and that elasticity has a cost. This intentional compliance decouples rotor position from link position under dynamic loading: while the motor encoder registers smooth rotation, the flexspline winds up like a torsion spring, introducing unmeasured end-effector deflection.

A tooth broken off at its root (figure 2.4) is the end state of that loading. Such failures show why dynamic impact loads, rather than static payload ratings alone, must govern actuator and transmission selection: repeated torque spikes and unmodeled contact shocks concentrate destructive stresses directly at the tooth root fillet. While smooth motion plans mitigate drivetrain vibration, safe physical interaction depends on measuring output torque and enforcing strict mechanical load boundaries.

[11] **Strain Wave Gearing**: Musser (1959) patented the mechanism, now sold as the Harmonic Drive: an elliptical wave generator flexes a thin toothed cup inside a rigid ring that carries two more teeth, so a single stage yields a high ratio with near-zero backlash. The flexing that makes the ratio possible is also the joint's torsional compliance, which makes the link elastic.

[12] **Limit Cycle**: A self-sustained oscillation of fixed amplitude that a nonlinear feedback loop settles into, here excited by the backlash deadband as the motor crosses the gap, the teeth strike, and the controller corrects back across it. Unlike an instability, it does not grow, so it can run unnoticed through a shift while every cycle adds a tooth impact to the fatigue count.

> **Systems Perspective 2.3: Impedance control for contact safety**
>
> When a position-controlled robot encounters a rigid surface (contact stiffness $k_{\text{contact}} \geq 10^5$ N/m), a minor setpoint tracking error of $\Delta x = 1$ mm escalates the contact force to $F_{\text{contact}} = k_{\text{contact}} \Delta x = 100$ N within milliseconds as the inverter commands maximum current against the obstacle. On the mobile manipulator, the gripper contacting the spring-latched door strikes a steel plate with stiffness 4.0×10^5 N/m. Once impact occurs, contact force ramps at $F_{\text{contact}} = k_{\text{contact}} v_{\text{approach}} t$: 12 N/ms under a guarded approach velocity of 0.03 m/s, and 40 N/ms at an unconstrained 0.10 m/s, rapidly exceeding the latch's 100 N structural limit.

Figure 2.3: Strain wave gear set: The three parts of a Harmonic Drive strain wave gear, disassembled: the elliptical wave generator with its thin-race ball bearing (top left), the flexspline cup with external teeth on its open rim (top right), and the rigid circular spline ring with internal teeth (bottom). The wave generator bends the flexspline so that its teeth mesh with the circular spline at two opposite zones; because the flexspline carries slightly fewer teeth, each turn of the wave generator advances it by only that difference, which gives a high reduction with little backlash. (Credit: Pieceofmetalwork, Wikimedia Commons, CC BY-SA 4.0.)

Impedance control (Hogan 1985) reformulates contact interactions away from pure position or force control. An impedance takes motion in and returns force; an admittance takes force in and returns motion. Because a rigid environment behaves as an admittance, yielding motion only when pushed, the manipulator is made to behave as an impedance, which answers a deviation in position and velocity with a restoring force ($F_{\text{contact}} = Z(\dot{x})$, force as a function of the imposed motion). The joint drive controller realizes this relationship by emulating a **virtual spring-damper system**:

$$F_{\text{contact}} = K_p(x_{\text{des}} - x) + K_d(\dot{x}_{\text{des}} - \dot{x})$$

The stiffness K_p multiplies the position error and the damping K_d the velocity error, the roles a proportional and a derivative gain play in a feedback controller. Tuning them in software causes the arm to yield compliantly under tracking errors, shedding peak impact loads. However, closed-loop impedance behavior is physically bounded by current-loop update rates, inverter voltage headroom, and unmodeled joint elasticity. Algebraic control laws cannot alter the unyielding inertia of the rotor during the first milliseconds of an impact. The runtime permission path must therefore enforce approach speed limits derived from physical contact qualification rather than relying on software compliance alone. The impedance command interface, through which a learned policy proposes these gains and the permission path validates them before admission, is defined in section 4.6.1.

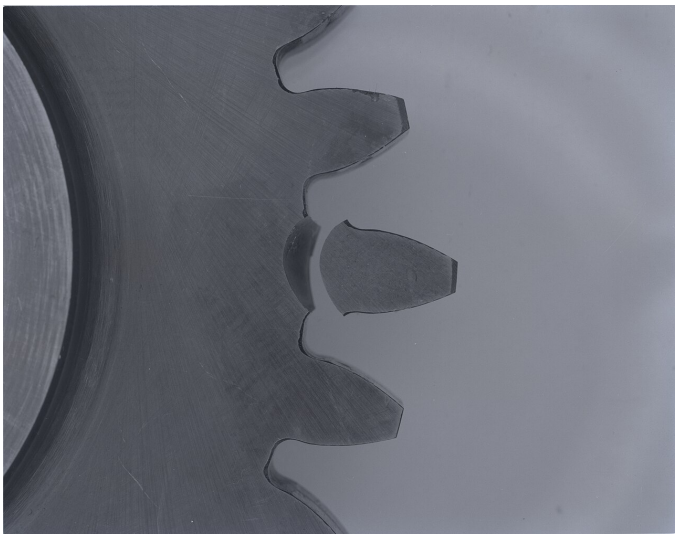


Figure 2.4: Broken gear tooth: A spur gear from gear research at the NASA Lewis (now Glenn) Research Center, photographed in 1974, with one tooth broken away at its root and the fragment lying in the gap. The root is where a tooth's bending stress concentrates, so repeated or impulsive loading that the transmission passes to the teeth ends here. (Credit: Martin Brown, NASA Glenn Research Center, via the U.S. National Archives, NARA 17417956, public domain.)

2.4.5 Actuation regimes and gear reduction trade-offs

The scaling disparity underlying inertia matching, in which output torque scales linearly with ratio ($\tau_{\text{out}} = \eta_{\text{gear}} N_{\text{gear}} \tau_{\text{motor}}$) while reflected inertia scales quadratically (N^2_{gear}), divides robotic actuation into three primary regimes. These regimes trade continuous torque density against mechanical backdrivability, distinguished by gear design, stage count, and reduction ratio (table 2.2): at the lowest ratios, **quasi-direct drive (QDD)** pairs a high-torque motor with a single compact planetary stage; at intermediate ratios, multistage planetary gearboxes reduce motor size at the cost of moderate reflected inertia and backlash; and at the highest ratios, strain-wave gears pack large torque multiplication into a compact joint at the cost of extreme reflected inertia and torsional compliance.

Table 2.2: Actuator transmission trade-off: Reduction ratio (N_{gear}), reflected rotor inertia scaling (N^2_{gear}), and the consequence for learned control policies across quasi-direct-drive (a single low-ratio planetary stage), multistage planetary, and strain wave transmissions. The strain wave range is that of one catalog series (Harmonic Drive AG 2014).

Transmission	Gear Ratio (N_{gear})	Reflected Inertia (N^2_{gear})	Implication for Learned Policies
Quasi-Direct Drive (single-stage planetary)	$3:1$–$10:1$ ($N_{\text{gear}} \leq 10$)	$N^2_{\text{gear}} \leq 100$ (Low)	Lower reflected inertia supports backdriving and current-based torque estimation; contact response still requires qualification.
Multistage Planetary	$15:1$–$50:1$ ($N_{\text{gear}} \approx 30$)	$N^2_{\text{gear}} \approx 225$–$2{,}500$ (Moderate)	Semi-backdrivable; a smaller motor than QDD for the same output torque; gear backlash requires load-side joint sensing.
Strain Wave (Harmonic Drive)	$50:1$–$160:1$ ($N_{\text{gear}} \geq 50$)	$N^2_{\text{gear}} \geq 2{,}500$–$25{,}600$ (Extreme)	High reflected inertia; zero initial backlash; torsional compliance and impact loading require qualification.

Actuator selection must satisfy continuous torque targets and permissible contact dynamics simultaneously. Direct-drive configurations eliminate gear backlash entirely, but require large, heavy motors to produce holding torque (Asada and Kanade 1983). Increasing the gear ratio permits a much smaller motor to produce equivalent output torque, but introduces reflected inertia, friction, and transmission compliance.

Motor thermal limits and torque requirements intersect in an electromagnetic figure of merit. Stator resistive losses follow $P_{\text{loss}} = i_q^2 R_{\text{eff}}$, where R_{eff} is the winding's effective loss resistance, while electromagnetic torque follows $\tau_{\text{motor}} = K_t i_q$. Combining these expressions reveals that resistive dissipation scales with the square of motor torque: $P_{\text{loss}} = (\tau_{\text{motor}}/K_m)^2$. Here the **motor constant** $K_m = K_t/\sqrt{R_{\text{eff}}}$ (in $\text{N}\cdot\text{m}/\sqrt{\text{W}}$) captures the motor's torque-producing capacity per square root of dissipated heat. Motor datasheets specify this parameter directly (TQ-Systems 2014). Section 2.5.1 relates R_{eff} to the phase and line-to-line resistances of section 2.4.2. When driving a mechanical transmission of ratio N_{gear} and efficiency η_{gear}, the motor must supply an internal torque of $\tau_{\text{out}}/(\eta_{\text{gear}} N_{\text{gear}})$, resulting in:

$$P_{\text{loss}} = \left(\frac{\tau_{\text{out}}}{\eta_{\text{gear}} N_{\text{gear}} K_m} \right)^2, \qquad J_{\text{ref,in}} = N_{\text{gear}}^2 J_{\text{in}}$$

where J_{in} represents total input inertia (the rotor plus the transmission input stage), and $J_{\text{ref,in}}$ is that combined inertia reflected to the joint output. Increasing the reduction ratio N_{gear} reduces winding heat by N_{gear}^2 while multiplying reflected inertia by N_{gear}^2. Consequently, for a selected motor and gearbox geometry, their product is invariant to the gear ratio:

$$P_{\text{loss}}\, J_{\text{ref,in}} = \frac{\tau_{\text{out}}^2\, J_{\text{in}}}{\eta_{\text{gear}}^2 K_m^2} \tag{2.6}$$

Adjusting N_{gear} relocates the actuator along a fixed trade-off curve between thermal dissipation and reflected inertia. Improving both characteristics concurrently requires selecting a motor with a superior ratio of K_m^2/J_{in}. Notebook 2.4 evaluates this trade-off for the mobile manipulator's arm joints.

Napkin Math 2.4: What does the gear ratio cost?

The running machine's arm is a seven-joint, torque-sensing manipulator in the class of the Franka Emika Panda (Franka Emika GmbH 2018), whose joints use strain-wave gears (section C.14.1). Franka does not publish the motors in those joints, so this example, like the thermal examples of section 2.5, uses a sourced stand-in. The stand-in is a TQ-RoboDrive ILM 70x18 frameless motor (TQ-Systems 2014) with $K_t = 0.187$ N·m/A and effective loss resistance $R_{\text{eff}} = 0.495\ \Omega$, driving a CSG-25 strain-wave gear (Harmonic Drive AG 2014), whose catalog lists one input inertia for all of that size's ratios. Rotor plus gear input give $J_{\text{in}} = 7.34 \times 10^{-5}$ kg·m². The joint must hold the arm's maximum joint torque $\tau_{\text{out}} = 87$ N·m through a gear of efficiency $\eta_{\text{gear}} = 0.85$ (illustrative).

Problem: How do the copper loss and the reflected inertia change if the same motor drives the joint through 50:1, 100:1, or a quasi-direct-drive ratio of 10:1?

Math:

1. **Motor constant**: $K_m = K_t/\sqrt{R_{\text{eff}}} = 0.266$ N·m/√W, the value the motor's data sheet publishes.
2. **At 50:1**: The motor supplies $\tau_{\text{out}}/(\eta_{\text{gear}} N_{\text{gear}}) = 2.05$ N·m, dissipating $(\tau_{\text{motor}}/K_m)^2 = 59.3$ W, and the joint reflects $N_{\text{gear}}^2 J_{\text{in}} = 0.183$ kg·m².
3. **At 100:1**: The motor supplies 1.02 N·m, the loss falls fourfold to 14.8 W, and the reflected inertia rises fourfold to 0.734 kg·m². The product $P_{\text{loss}} J_{\text{ref,in}}$ is unchanged.
4. **At 10:1**: The motor would have to supply 10.2 N·m, 2.5× its 4.05 N·m peak torque. No current can make this motor hold the load.

Result: Doubling the ratio buys a fourfold cut in holding heat with a fourfold rise in the inertia that an impact must accelerate, and lowering the ratio toward quasi-direct drive fails outright with this motor. The gear ratio moves the joint along a fixed trade; only the motor sets the trade itself.

Systems insight: A low-ratio, backdrivable joint requires a motor with a much larger torque per square root of watt, the route quasi-direct drive takes. A high-ratio arm joint instead accepts the reflected inertia and recovers contact sensitivity by measuring torque at its output.

Mechanical backdrivability measures the ease with which an external load applied at the joint output rotates the motor shaft backward through the transmission. The external torque required to backdrive a geared joint is:

$$\tau_{\text{ext}} = N_{\text{gear}}^2 J_{\text{rotor}} \ddot{q} + \tau_{\text{fric}}$$

where τ_{fric} denotes total joint friction referred to the output link. This friction varies dynamically with operating state (Lynch and Park 2017): breakaway static friction dominates at rest, Coulomb friction opposes motion at constant levels, and viscous damping grows with velocity. The Stribeck effect, a dip in friction just after sliding begins, shapes the response at near-zero speeds. Furthermore, gear tooth mesh friction scales with transmitted load, ambient temperature, and lubrication aging. At low velocities, friction dominates the resistance of high-reduction gearboxes; during sudden impacts, reflected rotor inertia dominates. As a result, backdrivability must be verified through empirical load testing on the assembled joint rather than inferred from the reduction ratio alone.

Quasi-direct drive actuation takes the low-ratio route of notebook 2.4, using large-diameter, high-torque motors so a single modest reduction stage suffices (Seok et al. 2015; Wensing et al. 2017; Katz et al. 2019). Minimizing reflected inertia and internal friction maximizes mechanical transparency, the fidelity with which external interaction forces translate into measurable motor motion and winding currents. The penalty is a substantially heavier motor demand for any given output torque.

Mechanical transparency also dictates what motor currents reveal about contact states. In high-friction gearboxes, motor current primarily overcomes internal transmission losses rather than external loads. Conversely, when transmission friction and inertial resistance remain small, output torque simplifies to:

$$\tau_{\text{out}} \approx \eta_{\text{gear}} N_{\text{gear}} K_t i_q$$

This relationship enables **proprioceptive sensing**, allowing the controller to estimate interaction forces directly from motor current without specialized external sensors. However, accurate estimation requires rigorous calibration across operating temperatures and speeds; current setpoints measure motor electromagnetic effort, portions of which are absorbed by acceleration and drivetrain losses. While backdrivability allows external collisions to displace the joint without shearing gear teeth, low gearing and motor current sensing do not guarantee safe contact forces. High link velocities can still impart severe impulsive shock before the current controller can respond. Safe interaction mandates qualifying the total closed-loop contact response.

High-reduction joints overcome transmission friction by sensing torque directly at the joint output. Strain gauge torque sensors mounted on the output side of strain wave gearboxes measure delivered torque directly, allowing closed-loop current controllers to cancel gearbox friction and cogging torque (Lynch and Park 2017). The running machine's arm adopts this approach, integrating torque sensors within each of its seven joints (Franka Emika GmbH 2018) to achieve transparency through measurement rather than low mechanical reduction. The engineering cost includes increased joint complexity, elevated mass, and the torsional compliance of the strain wave flexsplines, which complicates high-bandwidth motion tracking.

Whether mechanical transparency originates by construction (quasi-direct drive) or by active measurement (output torque sensing), contact operations require qualifying joint compliance, closed-loop torque response, and dynamic sensing fidelity against the operating environment. Dual-encoder architectures and load-side torque sensors expose internal transmission windup and mechanical backlash that motor-side encoders conceal. When load-side deflection departs from the qualified dynamic envelope, the runtime safety system can immediately intercede and reject subsequent motion commands.

2.4.6 Supporting drive and impedance models

Deliverable motor torque and contact shock isolation rest on two compact models of how the drive and transmission transform effort and motion: the rotating-frame torque law behind $\tau_{\text{motor}} = K_t i_q$ (section 2.4.2), and the series-spring impedance that isolates reflected rotor inertia during a fast

impact (section 2.4.4). Inside the physical drive, FOC holds the d-axis current i_d of section 2.4.2 at zero, which minimizes stator losses and makes output torque proportional to the q-axis current alone (section C.9 states the voltage equations that govern both currents in the rotating frame):

$$\tau_{\text{motor}} = \frac{3}{2} N_{\text{poles}} \lambda_{\text{pm}} i_q = K_t i_q$$

where N_{poles} is the number of magnetic pole pairs on the rotor and λ_{pm} is the permanent-magnet flux linkage, the magnetic flux from the rotor magnets that threads each winding. The $\frac{3}{2}$ scaling factor arises from the amplitude-invariant transformation convention adopted in this chapter. Motor drive firmware must enforce a transformation convention consistent with the identified torque constant K_t.

For frequency-domain dynamic analysis, mechanical impedance quantifies the joint's physical resistance to imposed environmental motion. The Laplace variable s stands for differentiation in time, so that for a sinusoidal motion of angular frequency ω it can be read as $s = j\omega$. An impedance term sJ grows with frequency like an inertia, and a term K/s falls with frequency like a spring. The apparent mechanical impedance $Z_{\text{out}}(s) = \tau_{\text{ext}}(s)/\dot{q}(s)$ evaluated at the joint output flange couples the load dynamics directly to the reflected rotor dynamics through the drivetrain's torsional stiffness K_{joint}. An external torque applied at the joint flange splits between two paths that share the same output velocity $\dot{q}(s)$, one accelerating the output link inertia J_{load} directly and the other twisting the transmission spring against the reflected rotor. Along that second path, the transmission spring and the reflected rotor act in series so both stages bear equal transmitted torque while their relative angular velocities sum, causing the spring impedance K_{joint}/s and reflected rotor impedance to combine through their reciprocal sum before adding to the output link impedance (Spong 1987):

$$Z_{\text{out}}(s) = sJ_{\text{load}} + D_{\text{joint}} + \left[\frac{1}{sN_{\text{gear}}^2 J_{\text{rotor}} + N_{\text{gear}}^2 D_{\text{motor}}} + \frac{s}{K_{\text{joint}}} \right]^{-1}$$

DRAFT

The coefficients D_{joint} and D_{motor} account for viscous damping in the output bearings and motor shaft. At low interaction frequencies, the series spring behaves rigidly relative to the rotor, and the environment experiences the aggregate inertia $J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}}$ assumed in rigid-body kinematic models. Above the transmission's natural resonance frequency, the bracketed impedance term simplifies toward K_{joint}/s: external collision forces compress the joint compliance against the link inertia J_{load}, temporarily isolating the rotor's reflected inertia. This series compliance cushions sudden impacts in flexible drivetrains and series elastic actuators, reinforcing why the flexible-joint dynamic model is necessary to calculate contact loads during manipulation.

Checkpoint 2.1: Actuator limits

The drives decide whether the stopping premise can be delivered, and the transmission decides what that delivery costs in heat and inertia.

- ☐ **Torque ceiling**: Can you compute the deceleration ceiling of the two hub drives when their current limit is set at the motor's rated current rather than its peak, and explain why a current limit lowered to protect the windings breaks the stopping premise without any software change?
- ☐ **Current rise**: Can you estimate how long the hub-motor current takes to reach its limit from rest on a depleted pack, and explain why back-EMF at speed helps a braking current but can starve a driving one?
- ☐ **Inertia matching**: Can you explain why the gear ratio that maximizes link acceleration makes reflected rotor inertia equal load inertia ($N_{\text{gear}}^2 J_{\text{rotor}} = J_{\text{load}}$), and why oversizing the gearbox for static torque lowers dynamic joint acceleration?
- ☐ **Heat vs. reflected inertia**: Can you explain why, for a given motor and gear, the ratio cannot reduce holding heat and reflected inertia together, and what must change to improve both?

Operating near these electrical and mechanical ceilings exacts an inevitable thermodynamic toll. Every ampere driving the motor against reflected rotor inertia or sustaining peak clamping torque dissipates resistive heat ($P_{\text{loss}} = i_q^2 R_{\text{eff}}$) in the stator windings. That thermal energy accumulates directly in the motor's thermal mass, bounding the operational duration of peak torque before winding insulation suffers accelerated thermal degradation.

2.5 Thermal Duty Cycles

Holding a load motionless at full reach places deceptive demands on an articulated arm such as the mobile manipulator's. Maintaining a static load against gravity demands continuous phase current through the stator coils, actively converting electrical energy into accumulating heat. Heat arrives in seconds and leaves in minutes, making duty cycle a primary design variable.

2.5.1 Thermal capacity and insulation breakdown thresholds

Although a stationary joint performs zero mechanical work ($W = \int F \cdot dx = 0$), sustaining its holding torque against gravity requires continuous phase current to produce magnetic Lorentz force ($\tau_{\text{motor}} = K_t i_q$). The lumped thermal model uses i_q in the same convention as its torque constant K_t. Here R_{eff} denotes the effective aggregate loss resistance for that current, so total winding copper loss is:

$$P_{\text{loss}} = i_q^2 R_{\text{eff}} = \frac{\tau_{\text{motor}}^2}{K_t^2} R_{\text{eff}}$$

For the amplitude-invariant drive convention above, with balanced phase currents and $i_d = 0$, this effective resistance is $R_{\text{eff}} = \frac{3}{2} R_{\text{phase}}$, where R_{phase} is resistance per phase. Three resistances of one winding therefore appear in this chapter. The phase resistance R_{phase} belongs in the Kirchhoff equation of section 2.4.2. Datasheets quote the line-to-line value measured between two terminals, which for a star-connected winding spans two phases, $2R_{\text{phase}}$. The hub-motor current-rise example used that line value with the line inductance, whose ratio gives the same time constant. The thermal calculations use $R_{\text{eff}} = \frac{3}{2} R_{\text{phase}}$, three quarters of the line-to-line value, and every winding resistance in them, including $R(T_{\text{winding}})$, is this effective resistance. The following illustrative thermal examples stipulate it and the matching current convention.

The copper-loss equation ties torque to current and current to heat. Commanded torque scales linearly with current ($\tau_{\text{motor}} = K_t i_q$), but stator copper loss scales quadratically with torque (τ_{motor}^2). Demanding twice the torque requires four times the thermal dissipation. Because heat has no fast exit, thermal accumulation limits how long an arm can maintain static poses or execute dynamic bursts without mechanical brakes.

Reducing motor current to shed heat also reduces holding torque. A loaded arm therefore needs a response that preserves support, such as transferring the load to a mechanical brake, rather than assuming that thermal derating leaves the task unchanged. Excess winding temperature can damage electrical insulation, while excess rotor temperature can permanently reduce magnet strength. Those material limits bound the allowable temperature trajectory.

The winding and housing respond on different timescales. Copper heats rapidly under current, while the larger core and housing absorb and reject heat more slowly. A case temperature measurement can therefore lag the winding hotspot during a torque burst. The stator in figure 2.5 failed from overheating, and its damage sits where this model puts the heat: in the copper windings and the insulation between them.

A thermal model follows the same energy accounting as the stopping calculation: energy accumulates where the rate entering exceeds the rate leaving. Here the winding receives electrical losses and transfers heat to the core; the core transfers heat to the surroundings. Thermal capacitance measures how much energy raises a component's temperature, while thermal resistance measures how strongly the heat-flow path resists that transfer. Representing the winding and core separately gives the two-node lumped-parameter thermal ordinary differential equation (ODE) system:

$$C_{\text{wind}} \frac{dT_{\text{winding}}(t)}{dt} = P_{\text{loss}}(t) - \frac{T_{\text{winding}}(t) - T_{\text{core}}(t)}{R_{\text{th,wc}}}$$

$$C_{\text{core}} \frac{dT_{\text{core}}(t)}{dt} = \frac{T_{\text{winding}}(t) - T_{\text{core}}(t)}{R_{\text{th,wc}}} - \frac{T_{\text{core}}(t) - T_{\text{amb}}}{R_{\text{th,ca}}}$$

Figure 2.5: Motor stator damaged by overheating: The opened stator of an electric motor that failed from overheating, showing the copper phase windings bound with tape at left, the insulating slot liners where the windings leave the laminated iron core, and the end bell with its bearing seat at right. The windings are where resistive heat ($P_{\text{loss}} = i_q^2 R_{\text{eff}}$) is generated, and the insulation between them is what accumulated heat ages. (Credit: K. Kudelina, Wikimedia Commons, CC BY 4.0.)

where $P_{\text{loss}}(t) = i_q^2(t) R(T_{\text{winding}}) = \frac{\tau_{\text{motor}}^2(t)}{K_t^2} R(T_{\text{winding}})$ is the Joule dissipation source deposited directly into the copper coils, C_{wind} is the small thermal capacitance of the winding, C_{core} is the large thermal capacitance of the laminated stator core and housing, $R_{\text{th,wc}}$ is the conductive thermal resistance from winding to core, and $R_{\text{th,ca}}$ is the convective thermal resistance from core to ambient air.

This separation of timescales creates a dangerous observability gap. Because $C_{\text{wind}} \ll C_{\text{core}}$, under a sudden peak torque burst or sustained stall against gravity, the stator core temperature T_{core} remains virtually clamped over the initial seconds by its large thermal inertia. The internal copper winding temperature surges at the initial adiabatic rate:

$$\left.\frac{dT_{\text{winding}}}{dt}\right|_{t=0} \approx \frac{P_{\text{loss}}}{C_{\text{wind}}}$$

Under sustained peak torque, the internal copper hotspot T_{winding} can climb past the 155 °C Class F rating within tens of seconds (notebook 2.5) while an external thermistor bonded to the motor casing still reads far below the hotspot. The class temperature is not a cliff at which the insulation fails. IEC 60085 assigns it as the hot-spot temperature at which an insulation system reaches its rated life (International Electrotechnical Commission 2007), and insulation aging[13] proceeds as a chemical reaction whose rate grows exponentially with temperature (Dakin 1948). Every excursion above the rating therefore spends insulation life faster than the rating assumes, and the accumulated aging culminates in the insulation charring and turn-to-turn electrical failures documented in figure 2.5. A thermal budget must count accumulated exposure as well as the instantaneous peak.

A first-order approximation can summarize heating over a characterized operating range, with an effective winding temperature T_{winding} and fitted thermal parameters:

$$C_{\text{th}} \frac{dT_{\text{winding}}(t)}{dt} = P_{\text{loss}}(t) - \frac{T_{\text{winding}}(t) - T_{\text{amb}}}{R_{\text{th}}}$$

where $R_{\text{th}} = R_{\text{th,wc}} + R_{\text{th,ca}}$ represents the winding-to-ambient steady-state thermal resistance. The effective capacitance C_{th} sets the fitted time constant $\tau_{\text{th}} = R_{\text{th}} C_{\text{th}}$. This approximation does not re-

[13] **Thermal Aging as an Exchange Rate**: The engineering practice predates the chemistry. Montsinger (1930) proposed loading transformers by the temperature their insulation reaches rather than by current alone, and Dakin (1948) later explained the exponential dependence as Arrhenius reaction kinetics. The consequence is an exchange rate rather than a threshold. Time spent above the rating costs life at a rate that grows exponentially with the excess, so the limit record must count time at temperature rather than merely the peak.

solve a fast winding hotspot separately from the housing. Its transient predictions require validation before they can bound a burst.

At thermal steady state ($dT_{\text{winding}}/dt = 0$), heat generation balances heat dissipation to ambient:

$$P_{\text{loss,ss}} = \frac{T_{\text{limit}} - T_{\text{amb}}}{R_{\text{th}}}$$

Substituting the copper loss identity $P_{\text{loss}} = \frac{\tau_{\text{motor}}^2}{K_t^2} R(T_{\text{limit}})$ reveals the **continuous thermal torque ceiling**:

$$\tau_{\text{cont}} = K_t \sqrt{\frac{T_{\text{limit}} - T_{\text{amb}}}{R(T_{\text{limit}}) R_{\text{th}}}} \tag{2.7}$$

Continuous torque is limited by thermal dissipation, not by the peak current ratings of the drive electronics or the battery. The arm's stand-in motor, for example, is rated for 4.05 N·m peak but only 1.25 N·m continuous, a ratio of 3.2 (TQ-Systems 2014). A brief torque burst of several seconds causes a rapid transient temperature rise that stays within the insulation rating ($T_{\text{limit}} = 155$ °C) if its peak does. However, sustained holding torque or high-frequency cycling drives the winding toward an asymptotic thermal equilibrium $T_\infty = T_{\text{amb}} + \bar{P}_{\text{loss}} R_{\text{th}}$. If that equilibrium exceeds the rating, continuous operation ages the winding insulation far faster than its rated life allows and ends in phase-to-phase short circuits.

The loss term must also change with temperature. Copper's positive temperature coefficient ($\alpha_{\text{Cu}} \approx 0.00393$ K^{-1}) increases the effective winding resistance by 47.2 percent from 20 °C to 140 °C, according to $R(T_{\text{winding}}) = R_0[1 + \alpha_{\text{Cu}}(T_{\text{winding}} - T_0)]$. That upper temperature is the **engineering ceiling** T_{ceil}, a working limit chosen below the 155 °C insulation rating T_{limit} to leave room for the error of the thermal model. The burst horizons and the stall derating below are checked against the rating itself. A schedule of repeated bursts is held to the ceiling, as is the installed continuous rating that the datasheet pitfall of section 2.9 obtains by evaluating equation 2.7 at T_{ceil} in place of T_{limit}. Holding the same torque at the ceiling therefore dissipates 47.2 percent more winding power than at the reference. A fixed-resistance calculation omits this feedback. The burst example below uses that approximation explicitly, and the subsequent stall calculation restores it.

Datasheet thermal ratings are optimistic because they are measured on an unconstrained motor mounted to a large heat sink in free air at 20°C or 25°C. Installed in a sealed chassis beside power electronics, battery packs, and other joints, the motor loses convective airflow, which raises R_{th}. Neighboring loads raise its local ambient above the test ambient before its own coils carry current. Thermal margins must therefore be budgeted from heating and cooling time constants measured with the enclosure sealed and adjacent loads at their p99 power draw.

2.5.2 Transient thermal bursts and periodic duty cycles

Because thermal capacity integrates power over time, a peak torque[14] or peak current rating is meaningless without four parameters: the initial winding temperature $T_{\text{winding}}(0)$, the allowed duration t_{on}, the ambient T_{amb}, and the recovery time t_{off}. An actuator that can produce three times its continuous torque sustains it only while the integral of copper loss ($P_{\text{loss}} = i_q^2 R_{\text{eff}}$) stays within the margin below the insulation limit. From a cold winding, that can last tens of seconds, and from a heat-soaked one a fraction of that (notebook 2.5).

> Napkin Math 2.5: Thermal accumulation across duty cycles
>
> This example uses the arm's sourced stand-in joint of notebook 2.4: a CSG-25-50 strain-wave gear (Harmonic Drive AG 2014) driven by a TQ-RoboDrive ILM 70x18 frameless motor (TQ-Systems 2014), whose rated current is $I_{\text{cont}} = 6.7$ A. The motor has effective aggregate loss resistance $R_{\text{eff}} = 0.495\ \Omega$, lumped thermal capacitance $C_{\text{th}} = 120$ J/K, and installed thermal resistance $R_{\text{th}} = 2.5$ K/W (both thermal values illustrative, because the motor's data sheet leaves them to the installation). It runs in an enclosed ambient $T_{\text{amb}} = 40$ °C with Class F insulation rated to 155 °C. The thermal time constant is $\tau_{\text{th}} = R_{\text{th}} C_{\text{th}} = 300$ s.

[14] **Duty Type and RMS Torque**: IEC 60034-1 rates a motor for a duty type: S1 is continuous running at constant load long enough to reach thermal equilibrium, and S3 is an intermittent periodic duty of identical load-and-rest cycles too short to reach it (International Electrotechnical Commission 2022). Because copper loss grows with τ_{motor}^2, the torque that heats the winding over a cycle short compared with the thermal time constant is the root-mean-square value $\tau_{\text{rms}} = \sqrt{\frac{1}{T}\int_0^T \tau_{\text{motor}}^2\, dt}$, and it is the RMS, not the peak, that must stay under the continuous rating. A peak rating means nothing without its duty type, which is why figure 2.6 draws the S1 and S3 regions separately.

Problem: How does the initial winding temperature change how long a threefold rated-current burst can last before the insulation limit?

Math:

1. **Continuous equilibrium**: At $I_{\text{cont}} = 6.7$ A the winding settles at $T_{\text{eq}} = T_{\text{amb}} + I_{\text{cont}}^2 R_{\text{eff}} R_{\text{th}} = 95.6$ °C, below the limit.
2. **Peak burst**: At $I_{\text{peak}} = 20.1$ A the dissipation rises ninefold to $P_{\text{peak}} = 200.0$ W, whose asymptote $T_\infty = T_{\text{amb}} + P_{\text{peak}} R_{\text{th}} = 540.0$ °C lies far above the limit.
3. **Survival horizons**: With $T_{\text{winding}}(t) = T_\infty - (T_\infty - T_{\text{winding}}(0))\, e^{-t/\tau_{\text{th}}}$, the time to reach the limit is
$$t_{\text{limit}} = \tau_{\text{th}} \ln \frac{T_\infty - T_{\text{winding}}(0)}{T_\infty - T_{\text{limit}}}$$
which gives 78.4 s from a cold start at ambient and 43.1 s from a heat-soaked start at T_{eq}.

Result: The heat-soaked joint holds the burst 45.1 percent less time than the cold one, so peak torque cannot be budgeted without tracking thermal history.

Systems insight: Peak torque is a stock of heat margin that thermal history spends, not a fixed rating. The admission check $\hat{T}_{\text{wind}} \le T_{\text{limit}}$ must use the current estimate of the winding temperature, and these first-order horizons are optimistic, because the winding hotspot of the two-node model runs ahead of the lumped temperature.

For repeated cycles of period $t_{\text{cycle}} = t_{\text{on}} + t_{\text{off}}$, the engineering ceiling $T_{\text{ceil}} = 140$ °C sets a cycle-average power ceiling $\bar{P}_{\max} = (T_{\text{ceil}} - T_{\text{amb}})/R_{\text{th}} = 40$ W. With peak dissipation during t_{on} and none during t_{off}, this average-power screen bounds the duty cycle $D_{\text{cycle}} = t_{\text{on}}/t_{\text{cycle}}$ by

$$D_{\text{cycle}} \le \frac{T_{\text{ceil}} - T_{\text{amb}}}{P_{\text{peak}} R_{\text{th}}} \tag{2.8}$$

which here is 20.0 percent, so a 3 s burst calls for a recovery of $t_{\text{off}} = t_{\text{on}}(1 - D_{\text{cycle}})/D_{\text{cycle}} \approx 12$ s. The average does not bound the hottest instant. Because heat arrives in seconds during bursts but dissipates over minutes during recovery, the peak transient hotspot temperature during periodic pulsing strictly exceeds the time-averaged calculation,[15] reaching 142.0 °C (above the ceiling) before the rise of $R(T_{\text{winding}})$ adds more. Average-power screening is optimistic, and a burst schedule is certified only against the transient hotspot trajectory.

[15] **Periodic steady-state thermal peak**: Over a repeating cycle of period $t_{\text{cycle}} = t_{\text{on}} + t_{\text{off}}$, summing the heating and cooling exponentials gives the periodic peak temperature rise: $\Delta T_{\text{peak}} = P_{\text{peak}} R_{\text{th}} \frac{1 - e^{-t_{\text{on}}/\tau_{\text{th}}}}{1 - e^{-t_{\text{cycle}}/\tau_{\text{th}}}}$. As the cycle period $t_{\text{cycle}} \to 0$, this peak approaches the average-power screen ($D_{\text{cycle}} P_{\text{peak}} R_{\text{th}}$); for any finite burst length, the peak winding temperature exceeds the average.

2.5.3 Torque-speed operating envelopes and back-EMF roll-off

The burst calculation limits duration, while the back-EMF relation in section 2.4.2 limits torque at speed. A feasible maneuver must satisfy both. At low speed, the drive can supply the current needed for a burst, but the winding temperature limits how long it may do so. At higher speed, back-EMF consumes voltage headroom and can prevent that current from appearing at all. On the arm's stand-in joint motor, voltage headroom and thermal history bound the usable torque-speed region simultaneously (figure 2.6). Because steady-state operation at full supply voltage splits the bus voltage between the resistive drop $i_q R_{\text{phase}}$ and the back-EMF $K_e \omega_m$, rising torque demands more current, consumes a larger resistive voltage drop, and pulls the motor's voltage-limited speed linearly down from 2,110 rpm at no load to 1,570 rpm at peak torque, while the burst that a cold winding sustains for 78.4 s reaches the insulation class in only 43.1 s from continuous-duty equilibrium.

The continuous-duty region (S_1) permits sustained operation under the declared cooling conditions. The larger intermittent region (S_3) permits periodic bursts only with the starting temperature, duration, and recovery interval established above. Both regions narrow when speed consumes the drive's voltage headroom. Their intersection with the current thermal state, rather than the catalog peak, gives the available torque for the next maneuver.

A simulator that uses only a fixed torque clamp can therefore teach a policy an unavailable action: maximum torque while the rotor is already near its voltage-limited speed, or a repeated burst without recovery. The actuator model and permission limits must include speed and thermal history.

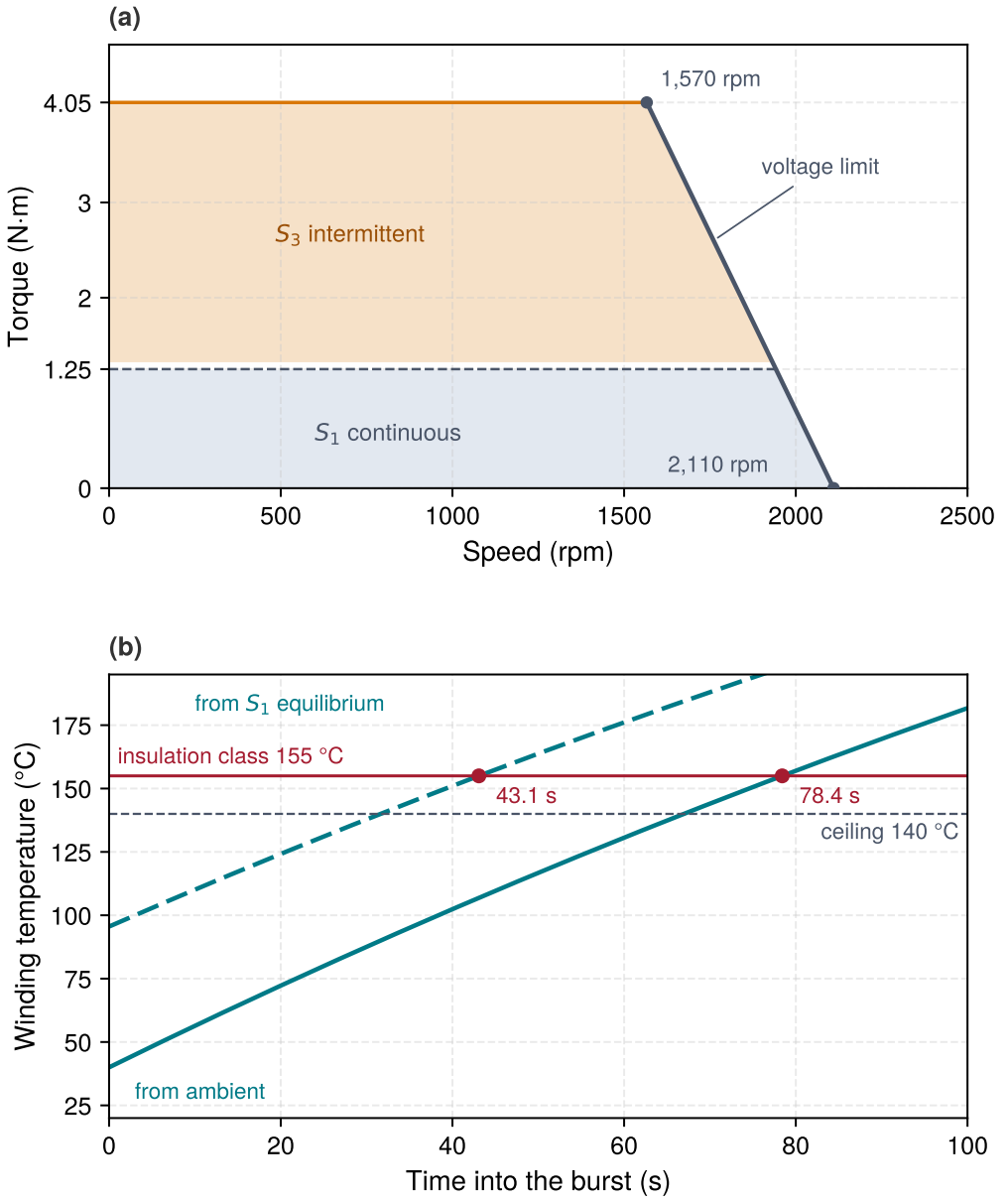


Figure 2.6: Torque, speed, and time on one joint motor: (a) The TQ-RoboDrive ILM 70x18 of the arm's stand-in joint on its 48 V supply. Continuous duty S_1 extends up to the 1.25 N·m rated torque; intermittent duty S_3 extends to the 4.05 N·m peak. Both end where back-EMF plus the resistive drop uses up the supply, from the 2,110 rpm no-load speed down to 1,570 rpm at peak torque; winding inductance, neglected here, pulls that boundary lower. (b) The winding temperature of the threefold-current burst of notebook 2.5, a first-order rise toward 540.0 °C with $\tau_{\text{th}} = 300$ s. From ambient the burst reaches the 155 °C insulation class after 78.4 s; from the continuous-duty equilibrium it gets there after 43.1 s. The thermal values are illustrative, because the data sheet leaves them to the installation.

Temperature-dependent winding resistance further shrinks the available region during a sustained hold.

The standard remedy in legged-robot sim-to-real transfer replaces the clamp with an actuator model identified on the hardware. For the series elastic actuators of the ANYmal quadruped, Hwangbo et al. (2019) trained a small network on a few minutes of logged joint data to map a short

history of position errors and velocities to the torque the actuator actually delivered. The network then ran inside the simulator in place of an ideal actuator. The ideal model, which assumes any commanded torque appears instantly, failed to predict the measured torque. The same structure carries the envelope of this section into training: an actuator model that also takes speed, bus voltage, and winding temperature as inputs lets the simulator deny the torque the hardware cannot deliver.

↪ How actuator dynamics mismatch is measured and closed in the training simulator is developed in section 6.5.

2.5.4 Copper resistance feedback and stall thermal runaway

The copper-resistance rise calculated in section 2.5.1 must enter the runtime estimate as well as the design calculation. An external thermistor can lag the winding hotspot, so a thermal observer combines measured current with the characterized heating and cooling model. A discrete update of the lumped energy balance is:

$$\hat{T}_{\text{wind}_{k+1}} = \hat{T}_{\text{wind}_k} + \frac{\Delta t}{C_{\text{th}}} \left[{i_q}_k^2 R(\hat{T}_{\text{wind}_k}) - \frac{\hat{T}_{\text{wind}_k} - T_{\text{amb}}}{R_{\text{th}}} \right]$$

where $R(\hat{T}_{\text{wind}_k}) = R_0[1 + \alpha_{\text{Cu}}(\hat{T}_{\text{wind}_k} - T_0)]$. The update interval, model error, and temperature uncertainty must fit the thermal margin. As the estimate approaches its ceiling, the permission path reduces admissible effort or transfers the load to a qualified support mechanism. Removing holding current without securing a gravity-loaded arm would exchange a thermal fault for uncontrolled motion. A static hold shows how copper's feedback changes the sustainable torque (notebook 2.6).

Napkin Math 2.6: Static holding torque and stall derating

The arm's elbow, the same Panda-class stand-in joint as notebook 2.5, is asked to hold the arm's maximum joint torque $\tau_{\text{hold}} = 87$ N·m at stall ($\omega_m = 0$) through its $N_{\text{gear}} = 50{:}1$ strain-wave gear ($\eta_{\text{gear}} = 0.85$, illustrative). Its motor has $K_t = 0.187$ N·m/A and cold effective aggregate loss resistance $R_0 = 0.495\ \Omega$ at $T_0 = 20$ °C. The illustrative installed thermal resistance is $R_{\text{th}} = 1.80$ K/W, lower than the 2.5 K/W of the burst example and so a better-cooled installation. The Class F limit is 155 °C, and the limb ambient is $T_{\text{amb}} = 40$ °C.

Problem: With copper's resistance rising with temperature, where does the stalled winding settle, and what holding torque is thermally sustainable?

Math:

1. **Stall current**: $\tau_{\text{motor}} = \tau_{\text{hold}}/(\eta_{\text{gear}} N_{\text{gear}}) = 2.05$ N·m, so $i_q = \tau_{\text{motor}}/K_t = 10.9$ A and the cold dissipation is $P_0 = i_q^2 R_0 = 59.3$ W.
2. **Runaway**: With $R(T_{\text{winding}}) = R_0[1 + \alpha_{\text{Cu}}(T_{\text{winding}} - T_0)]$, the steady rise satisfies

   $$\Delta T = i_q^2 R_0 \left[1 + \alpha_{\text{Cu}}(T_{\text{amb}} - T_0 + \Delta T)\right] R_{\text{th}}$$

   Rearranging terms yields $\Delta T = \frac{i_q^2 R_0 [1 + \alpha_{\text{Cu}}(T_{\text{amb}} - T_0)] R_{\text{th}}}{1 - i_q^2 R_0 R_{\text{th}} \alpha_{\text{Cu}}}$. The denominator establishes the **thermal runaway criterion**: if the positive feedback loop gain satisfies $i_q^2 R_0 R_{\text{th}} \alpha_{\text{Cu}} \geq 1$, this fixed-current, linear-resistance model cannot settle to a stable thermal equilibrium. Above one, heat generation increases with temperature faster than heat removal, producing exponential growth. At exactly one, the temperature-dependent terms cancel and the remaining positive heating causes linear growth. In this stall scenario, the feedback factor is $0.4196 < 1$, so temperature settles at a finite (yet fatal) equilibrium given by $\Delta T = 115.16 + 0.4196\ \Delta T$. This yields $\Delta T = 198.4$ K and drives the winding to 238.4 °C, far above its rating.
3. **Derating**: At the limit the allowable rise is 115 K, so the sustainable dissipation is $P_{\text{loss,ss}} = \Delta T_{\text{limit}}/R_{\text{th}} = 63.89$ W. At the hot resistance $R(T_{\text{limit}}) \approx 0.758\ \Omega$, that allows $I_{\text{cont}} = \sqrt{P_{\text{loss,ss}}/R(T_{\text{limit}})} \approx 9.18$ A and a hold of $\tau_{\text{hold,max}} = \eta_{\text{gear}} N_{\text{gear}} K_t I_{\text{cont}} \approx 73.0$ N·m.

Result: Held without control, the stall drives the winding toward 238.4 °C, and staying within the insulation rating requires a 16.1 percent derating, from 87 N·m to 73.0 N·m.

Systems insight: At stall the motor does no mechanical work, so every watt it draws becomes heat, and copper's rising resistance makes that heat grow with temperature. Continuous gravity holding above the thermally sustainable torque therefore needs another support mechanism, such as a friction brake or counterweight, in the premise the permission path admits; holding within that torque can remain electrically sustained under the stated thermal conditions.

Thermal capacity governs how long an actuator can sustain torque before winding insulation degrades or thermal runaway occurs, bounding continuous holds and transient bursts by the plant's ability to reject heat. Motor windings, however, do not generate those thermal loads in isolation. Delivering the current pulses that drive these torque trajectories simultaneously stresses the upstream power distribution network, shifting the engineering constraint from heat dissipation in copper to transient voltage stability on the shared direct-current bus.

2.6 Electrical Power Integrity

When the mobile manipulator swerves hard to avoid a collision, every drive motor, joint inverter, and accelerator demands peak current at once from one direct-current distribution bus. In simulation, current flows from an idealized source. On hardware, delivery is bounded by finite capacity, source impedance, and transient voltage stability. A machine that browns out mid-motion fails physically; the electrical supply is a shared resource.

Rail sag, regenerative overvoltage, and thermal accumulation are manifestations of one bounded energy network on different time scales. Of the three, voltage moves fastest, within microseconds to milliseconds whenever current demand exceeds what the source can deliver at that instant. Compute, sensing, and motor inverters draw from a common bus, so when a policy commands coordinated accelerations across several joints, the sum of their currents changes the voltage delivered to every node on the machine.

2.6.1 Power distribution impedance, transient droop, and brownout resets

The mobile machine's battery, harnesses, connectors, and protection switches all contribute source impedance.[16] Local capacitors supply part of a sudden load increase, so current drawn by the loads need not equal current arriving through the harness. For a source feeding one bus node through resistance and inductance,

$$V_{\text{source}}(t) - V_{\text{bus}}(t) = R_{\text{bus}} I_{\text{source}}(t) + L_{\text{bus}} \frac{dI_{\text{source}}}{dt} \tag{2.9}$$

where R_{bus} is the lumped source resistance, L_{bus} is the loop inductance, and I_{source} is the current through that source path. The local capacitor bank C_{bus} supplies the difference between load current I_{load} and source current. In the ideal-capacitor model, charge removed from that bank lowers the same bus voltage:

$$V_{\text{bus}}(0) - V_{\text{bus}}(t) = \frac{1}{C_{\text{bus}}} \int_0^t [I_{\text{load}}(s) - I_{\text{source}}(s)]\, ds.$$

These equations jointly determine the voltage response; the capacitor's voltage change is not an additional drop to add to equation 2.9.

Compute and motion must be budgeted together because they share this path. An obstacle response can require fresh inference at the same time as a large drive-current change, so testing each load separately misses the combined step that the supply must support. Harness inductance limits how quickly source current can rise. Local capacitors supply the initial current deficit, while harness resistance continues to lower the rail after source current catches up with a sustained load. Once the currents settle, the inductive voltage is zero and the sustained drop is $I_{\text{source}} R_{\text{bus}}$. The worked example below screens that sustained condition. It does not establish the minimum voltage during the transient. If the actual voltage crosses a regulator's undervoltage threshold, computation or communication may stop or reset. Power qualification must establish the voltage at the supplied devices during coincident load and the physical response if those devices lose power.

[16] **Source Resistance**: The lumped source resistance is $R_{\text{bus}} = R_{\text{cell}} + R_{\text{harness}} + R_{\text{switch}}$, the sum of battery cell, harness and connector, and protection-switch on-state resistances. The cell term is the one that moves. It rises as the pack ages and as it cools, so a sustained screen passed at commissioning can fail on a cold morning years later. That drift belongs in the rail's limit record as a condition, with a detector that measures the drop under load.

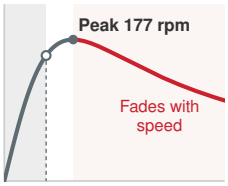
↪ Sizing the bus capacitance against that droop, and the undervoltage lockout it must avoid, is developed in section C.10.

2.6.2 Unpowered states and unmanaged fallback dynamics

A brownout reset turns the electrical fault into a mechanical one. Voltage supervisors pull the reset line on microcontrollers and gate drivers, clocks halt, register configurations vanish, and communication buses float. An acceleration command issued just before the reset stays physically active until the gate-driver stage collapses. If the driver then floats, phase current ceases, torque falls to zero, and the body coasts under gravity and inertia into workpieces or hardstops. If it instead shorts the low-side switches for dynamic braking, back-EMF ($e_{\text{bemf}} = K_e\omega_m$) produces an immediate, abrupt counter-torque that jerks the mechanics at the joint limits.

Power removal is an unmanaged state transition, not an automatic return to safety. Cutting power leaves an actuator in one of three unpowered states, namely free coasting, phase-short dynamic braking, or spring-engaged mechanical braking, and each enforces a distinct mechanical behavior. Free coasting simply allows gravity to accelerate unconstrained joints without any resistance. An electromechanical friction brake requires 30 ms to 80 ms for its mechanical spring to overcome magnetic coil decay and engage its friction pads, during which gravity accelerates an overhead arm downward.

Phase-short dynamic braking generates a counter-torque that at low speed grows in proportion to angular velocity ($\tau_{\text{motor}} \approx \frac{K_t K_e}{R_{\text{phase}}}\omega_m$) and gives zero holding torque at zero velocity. The proportionality does not last. As speed rises, the winding inductance limits the short-circuit current. The shorted winding is a resistance and an inductance driven by the back-EMF $K_e\omega_m$, which alternates at the electrical speed $N_{\text{poles}}\omega_m$. An inductance opposes an alternating current in proportion to its frequency, with reactance $N_{\text{poles}}\omega_m L_{\text{phase}}$, and that opposition adds to the resistance in quadrature. The current is therefore $K_e\omega_m / \sqrt{R_{\text{phase}}^2 + (N_{\text{poles}}\omega_m L_{\text{phase}})^2}$. The braking torque, the power dissipated in R_{phase} divided by ω_m, is $K_t K_e \omega_m R_{\text{phase}} / (R_{\text{phase}}^2 + (N_{\text{poles}}\omega_m L_{\text{phase}})^2)$. That torque peaks where the electrical speed $N_{\text{poles}}\omega_m$ equals $R_{\text{phase}}/L_{\text{phase}}$ and falls beyond it, so above that speed a phase short brakes less the faster the joint turns. For the base's hub motors that peak lies at 177.3 rpm of the wheel, above the 107.7 rpm of the drive limit, so braking torque still rises across the base's speed range, though not strictly in proportion to speed. A fast, high-ratio joint motor has no such guarantee.



Phase-short braking peaks at 177.3 rpm, above the base's 107.7 rpm limit.

Industrial machine stop categories and drive safety functions sequence these states. The machinery standard defines three stop categories (International Electrotechnical Commission 2016a): remove power at once, stop under control and then remove power, or stop under control and keep power applied. Each category still requires specific physical evidence before a safety case can credit it (table 12.3). Beneath every category lies one physical fact. Once torque is removed, a joint enters one of the three unpowered states above. A drive's Safe Torque Off (STO) therefore leaves a moving joint in free coast until a brake engages. The trajectory a machine follows after an electrical cutoff must be measured under full mechanical payload, not deduced from circuit diagrams. The choice of which safe fallback state the machine should enter belongs to system safety architecture, but the physical dynamics of an unpowered joint are unalterable mechanical constraints of the body.

↪ The placement of Safe Torque Off in the machine's hardwired interlock is developed in section 4.6.

2.6.3 Regenerative overvoltage surges and dynamic brake choppers

Regenerative deceleration drives the bus toward its opposite boundary, overvoltage, as part of the machine's kinetic energy returns electrically. Returned braking energy needs an absorption or dissipation path, and a battery cannot be assumed to accept it under every state of charge, temperature, or protection state. If the battery management system disconnects charge input, the remaining bus components must handle the regenerated energy without exceeding their voltage ratings.

Under the conservative assumption that the battery management system has disconnected and the local bus capacitance C_{bus} must absorb the entire returned kinetic energy E_{regen} without dissipation, the terminal DC bus voltage rises to:

$$V_{\text{final}} = \sqrt{V_{\text{initial}}^2 + \frac{2E_{\text{regen}}}{C_{\text{bus}}}} \tag{2.10}$$

where V_{initial} is the bus voltage when deceleration begins (section C.10 derives this energy balance). The worst case is a full pack, the state in which the battery management system refuses further charge.

The capacitor-only endpoint is a screening calculation: if it exceeds a component rating, the stop requires another energy path. A dynamic brake chopper switches an absorption resistor across the bus when voltage reaches a chosen clamp threshold. Its resistor must handle both the peak regenerated power and the pulse energy, with enough recovery between stops. The worked bus calculation below sizes the peak-power requirement for the warehouse machine.

Power integrity is an architectural constraint. The design sets coordinated power budgets and current slew-rate limits across accelerators and drives. The permission path arbitrates simultaneous multi-axis accelerations so that their dI/dt transients do not compound. Brake choppers protect the inverters from regenerative feedback during emergency stops. Notebook 2.7 works both halves of that budget, the bus's sag and the stop's regenerative surge, on the mobile manipulator's 48 V drive bus.

Napkin Math 2.7: DC bus voltage sag and regenerative surge

The mobile manipulator's lithium-ion pack feeds a 47.7 V nominal drive bus, with the voltage envelope of a commercial 48 V-class mobile robot's pack (Mobile Industrial Robots 2025). The bus carries the wheel drives, the arm's joint drives, and a 48-to-24 V converter that supplies the application processor and control electronics. The converter shuts off when its input falls to 33 V (MEAN WELL 2026).

Problem: Under the coincident current step of an inference burst and the drive motors, and then a stop from the drive limit, does the bus remain within the operating limits of the converter and the motor inverters?

Math:

1. **Parameters**:
   - Machine mass, travel speed, and credible deceleration: $m = 368$ kg (upper bound, full tote rack), $v = 1.5$ m/s, $a_{\text{brake}} = 2$ m/s²
   - Bus characteristics: pack voltage from 54 V at full charge to 42 V depleted, source resistance $R_{\text{bus}} = 60$ mΩ (illustrative), bulk link capacitance $C_{\text{bus}} = 22{,}000$ µF
   - Operating limits: converter lockout $V_{\text{uvlo}} = 33$ V, converter continuous input maximum 67.2 V, inverter switch rating $V_{\text{breakdown}} = 100$ V
2. **Sustained-load screen**:
   - As an illustrative coincident load, the two drive motors and the arm's joint drives add 68.6 A while the application processor, drawing its 60 W thermal design power (TDP) through the converter at 91 percent efficiency, adds 1.4 A. Their aggregate increase is $\Delta I = 70$ A. Assume the stated pack voltage is the unloaded source voltage and this coincident demand is the total load in the screen.
   - After source current settles to the increased load, the additional resistive drop is:

$$\Delta V_{\text{sustained}} = \Delta I R_{\text{bus}} = 4.2V$$

   - The resulting sustained bus voltage at full charge and when depleted is:

$$V_{\text{bus, sustained}} = 54V - 4.2V = 49.8V \text{ (full)}, \quad 42V - 4.2V = 37.8V \text{ (depleted)}$$

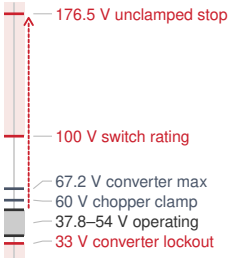
   - Even from the depleted pack, the bus stays 4.8 V above the converter's lockout; it would take a 150 A step to spend the 9 V between the depleted pack and the lockout. This screen does not establish the transient minimum, which inductance and capacitor depletion push lower.
3. **Regenerative surge**:

- During a stop from the drive limit, the loaded machine's kinetic energy must be dissipated:
$$E_k = \tfrac{1}{2}mv^2 = \tfrac{1}{2}\,(368kg)(1.5m/s)^2 = 414J$$
- A full pack is the worst case, because that is when the battery management system refuses charge. If it disconnects charge input and an illustrative 75 percent of kinetic energy returns to the bus ($E_{\text{regen}} = 310.5$ J), an unprotected, lossless capacitor-only energy balance would end at:
$$V_{\text{final}} = \sqrt{V_{\text{full}}^2 + \frac{2E_{\text{regen}}}{C_{\text{bus}}}} \approx 176.5V$$
- *Failure mechanism*: Raising the capacitor from full charge to the 60 V clamp absorbs only 7.5 J of the 310.5 J, so long before the bus reached that endpoint it would cross the converter's input maximum and then the 100 V switch rating.

4. **Dynamic braking clamp**:
   - Under constant deceleration over $T_{\text{stop}} = 750$ ms, speed falls linearly, so the regenerated power, force times speed, falls linearly from its peak to zero. The peak is twice the average $E_{\text{regen}}/T_{\text{stop}}$:
$$P_{\text{regen, peak}} \approx \frac{2E_{\text{regen}}}{t_{\text{stop}}} = 828W$$
   - The clamp $V_{\text{clamp}} = 60$ V sits above the full-charge voltage, so it never fires while charging, and within the converter's input range. To hold it there, the brake chopper must divert this peak power through an absorption resistor. A resistor across the bus at the clamp voltage dissipates $V_{\text{clamp}}^2/R_{\text{brake}}$, which must be at least the peak power:
$$R_{\text{brake}} \le \frac{V_{\text{clamp}}^2}{P_{\text{regen, peak}}} \approx 4.35\Omega$$

Result: The sustained-load screen leaves the bus at 49.8 V from a full pack and 37.8 V from a depleted one, 4.8 V above the converter lockout. Unbuffered regenerative braking would drive the bus past the converter's input maximum and the 100 V switch rating, which calls for a dynamic brake chopper whose resistor is no larger than 4.35 Ω and absorbs an 828 W peak (satisfied by an illustrative 3.9 Ω, 1 kW component).

Systems insight: Mechanical motion and electrical power are bidirectionally coupled. Acceleration requires a supply that sustains the coincident current demand; deceleration requires an adequate absorption or dissipation path. On this 48 V bus the depleted pack still sits 9 V above the converter lockout, more than twice this step's resistive drop, so the sustained screen passes. The transient minimum, pulse energy, repetition, cooling, and component tolerances still require qualification.



The bus must stay between the 33 V lockout and the clamp; an unclamped stop reaches 176.5 V.

Checkpoint 2.2: Freshness, stopping, heat, and supply

The actuator budget was checked in checkpoint 2.1. Before measuring these limits on an assembled machine, verify your understanding of the other four:

- ☐ **Freshness deadline**: Can you calculate the maximum allowable measurement age for the mug riding the pick-station conveyor at 0.20 m/s, with a 15 mm grasp tolerance of which the initial estimate already spends 3 mm? Can you explain why that age is counted from

exposure on the sensor rather than from the start of inference, and why the base's stop for a standing person instead charges age as blind travel?

- ☐ **Velocity ceiling**: Can you solve $d_{\text{stop}} \leq D_{\text{clear}}$ for the admissible speed on the running machine's numbers, and explain why doubling the pre-brake delay expands the stopping distance linearly while doubling travel speed expands the braking term quadratically?
- ☐ **Thermal horizons and runaway**: Can you explain why an actuator can produce three times its rated torque for seconds but overheats its winding insulation if held there for minutes, and distinguish a sustained stall that reaches an excessive finite equilibrium from one in which copper's resistance feedback causes thermal runaway?
- ☐ **Rail sag and regeneration**: Can you explain how a coincident compute and drive current step sags a shared rail through resistance, inductance, and capacitor depletion, and why a stop from speed pumps energy back into the bus that a brake chopper must divert?

Preserving power integrity ensures that transient acceleration spikes do not trigger brownout resets and that regenerative braking does not overvoltage the bus, completing the analytical formulation of the five physical budgets. Calculating these envelopes from datasheets and circuit models, however, cannot guarantee operational safety on a physical robot. In deployment, source impedances rise as batteries age, contact friction varies across facility floors, and enclosed packaging degrades thermal dissipation. Defending these budgets in production requires measuring what the integrated hardware actually delivers under live operational stress.

2.7 Measuring a Machine's Own Limits

A brake that stops an unloaded base on a cool laboratory floor has not established how the loaded mobile manipulator stops after repeated maneuvers. Characterization must connect each calculation to measurements under the conditions in which the machine will use it. A dynamometer, a test stand that applies a controlled mechanical load, isolates actuator behavior, just as the instrumented propulsion test stand in figure 2.7 evaluates an assembled electromechanical powertrain under live aerodynamic and structural loads. Integrated trials then establish what changes when the actuator shares a chassis, cooling path, and power supply with the rest of the machine. A defensible limit states what was measured, how uncertain the result is, and which conditions must remain true for a later decision to rely on it.

2.7.1 Dynamometer testing and instrument intrusion

On a dynamometer, a controlled load machine resists the actuator while torque, speed, current, and voltage are measured on a shared timebase. Four tests connect the earlier models to measurable parameters:

- **Torque and back-EMF transduction (K_t and K_e)**: By commanding the inverter's Field-Oriented Control (FOC) current loop across a sweep of quadrature phase currents i_q while the dynamometer absorbs torque at constant speed, engineers measure electromechanical torque transduction:

  $$\tau_{\text{motor}} = K_t i_q$$

  Driving the unpowered motor at known speed and measuring open-circuit voltage supplies the corresponding back-EMF constant $K_e = e_{\text{bemf}}/\omega_m$. These tests establish the conversion factors used in section 2.4.2.
- **Electrical rise time and inductive bandwidth ($\tau_e = L_{\text{phase}}/R_{\text{phase}}$)**: With the rotor held so that the test current produces no torque, the inverter applies a small voltage step V_{step} across the winding. The current response:

  $$i_{\text{phase}}(t) = \frac{V_{\text{step}}}{R_{\text{phase}}}\left(1 - e^{-t/\tau_e}\right)$$

  reveals the electrical time constant. The step must stay small, because its steady current $V_{\text{step}}/R_{\text{phase}}$ flows through a winding of low resistance: the full 42 V of a depleted pack across

Figure 2.7: An instrumented propulsion test stand: An electric motor with its propeller, speed controller, and wiring secured at the top of the Airvolt test stand at the NASA Armstrong Flight Research Center in 2016, instrumented to collect data on the propulsion system under load. A stand like this measures what a data sheet only asserts, on the assembled drive under stated conditions. (Credit: Lauren Hughes, NASA Armstrong Flight Research Center, AFRC2016-0100-05, public domain.)

a hub motor's 0.7 Ω would drive 60 A, 2.4× the motor's peak current. The current probe and acquisition path must resolve the rise; otherwise the instrument smooths the response being measured and hides the actuator limit.

- **Enclosed thermal resistance (R_{th}) and continuous torque ceilings**: A heating trial under the installed cooling conditions measures the winding temperature rise and loss power, establishing $R_{\text{th}} = \Delta T / P_{\text{loss}}$. The continuous torque ceiling from section 2.5.1 is then:

$$\tau_{\text{cont}} = K_t \sqrt{\frac{T_{\text{limit}} - T_{\text{amb}}}{R(T_{\text{limit}}) R_{\text{th}}}}$$

  The test must reproduce the enclosure and neighboring heat loads under which that ceiling will be used. An open-bench rating overstates continuous capacity whenever the installed shroud restricts airflow or adjacent actuators warm the mounting frame.

- **Backdrive friction and impact dynamics**: By driving the actuator output shaft with the dynamometer load motor, the testbed measures the torque needed to backdrive the joint across speed, load, and temperature:

$$\tau_{\text{ext}} = N_{\text{gear}}^2 J_{\text{rotor}} \ddot{q} + \tau_{\text{fric}}$$

  The measured response establishes whether the transmission can yield under the intended external load and maps the friction term that section 2.4.5 shows is not a constant. Contact qualification must additionally exercise the relevant compliance and impact conditions. The backdrive model alone does not predict tooth failure.

Measurement can change the plant. A series current shunt lowers the supplied voltage, and an embedded temperature probe changes a local heat-flow path. These effects belong in the uncertainty budget along with sensor resolution, bandwidth, and clock alignment.

The sensor mechanisms in section 2.2.1 also affect characterization. Velocity estimated from encoder samples by backward differencing trades resolution against latency, and that trade-off must enter the reported uncertainty and response time. Vibration above the sensor's sampled bandwidth can likewise appear as slower motion:

$$f_{\text{alias}} = |f_{\text{vib}} - k f_s|$$

The integer k identifies the sampling-frequency multiple that folds the vibration into the observed band. Qualification must distinguish that measurement artifact from the motion used to establish the limit.

A limit means nothing without the context of its test. Every recorded limit states the payload, ambient temperature, supply voltage, enclosure state, firmware version, and concurrent computational workload of the run. A thermal time constant measured on an open bench does not transfer to the same actuator in a sealed fairing in sunlight, because enclosing the motor raises R_{th} and with it $\tau_{\text{th}} = R_{\text{th}}C_{\text{th}}$. The hotter ambient also shrinks the margin below the insulation rating. A rail characterization that omits whether vision-language model inference (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) was running during motor stalls describes an idle board rather than the combined draw of full compute and peak torque.

2.7.2 Zero-failure bounds and wear over life

The absence of a failure during testing bounds the failure rate statistically; it does not establish safety. If the mobile manipulator completes $n = 3{,}000$ loaded stops without a brownout or an overrun, the data support only a per-stop failure rate below roughly $3/n$, the rule of three, which is about one stop in 1,000 at 95 percent confidence. The bound holds only if the trials are independent and stationary, so that one trial does not influence another and the failure probability stays the same from trial to trial.

↪ The rule of three, and the trial campaigns it sizes, is developed in section 7.6.

Thermal history and wear can change the conditions between trials:

- **Heating across consecutive trials**: Repeated stops heat the windings, changing resistance and the available torque response. Trials taken at different starting temperatures do not characterize the same operating condition.
- **Rising battery resistance over life**: Battery aging increases source resistance, reducing the voltage margin under a current step.
- **Growing gearbox backlash**: Transmission wear changes reversal response and tracking error, invalidating a limit measured on a less worn assembly.

These changes make characterization a standing obligation. **Lifecycle degradation** changes the conditions behind a limit, so its record must identify when recalibration or a reduced operating envelope is required.

2.7.3 The six elements of a limit record

The stopping and power calculations become usable by other layers only when their assumptions travel with their values. A **limit record** carries those assumptions together with the evidence that supports the limit. The learned proposer uses the limits to constrain candidate actions, while the independent permission path checks them before actuation. Recording a value does not certify it: the illustrative entries below still require measurements and uncertainty estimates before they can support operation. The six elements in table 2.3 make that missing evidence visible for the mobile manipulator's credible deceleration, the drive torque ceiling that must support it, and the drive bus's undervoltage lockout.

↪ The handoff record that carries each limit record to the permission path is developed in section 4.9.

Two of the elements are easily conflated. Uncertainty describes what is not known about the measurement (observation noise, instrument tolerance, sensor bandwidth, and calibration drift). Margin is the physical clearance the machine defends against what is not known. The provenance element then says how far the value itself can be trusted, by naming its evidence category. Every number that governs a machine falls in one of five categories: a direct measurement, a value derived from measurements through a stated model, a datasheet rating, an illustrative example, or a budget the designer chose. Only the first two can support a safety margin or a timeout. A value computed from datasheet inputs, such as the drive torque ceiling below, keeps the datasheet's weaker standing until measurements replace its inputs. A chosen budget, such as the protective clearance δ_{margin}, binds only once it is checked against the measured number it must fit.

Table 2.3: The limit record on the mobile manipulator: The six elements of a limit record, filled in for the credible deceleration that sets the stopping distance, for the drive torque ceiling that must support it, and for the converter lockout that bounds the shared drive bus. None of the three yet has samples or an established uncertainty, and the provenance element names the measurement that would replace each.

Element	What the record states	Credible deceleration a_{brake}	Drive torque ceiling	Drive-bus converter lockout
Value	The value or empirical distribution, with its sample count and the tail percentiles it reports	2 m/s², taken as the credible low tail of loaded stops; no samples yet	At least 2.33 m/s² from 60 N·m per wheel; computed from datasheet values, no samples yet	33 V at the 48-to-24 V converter input; no samples on the assembled bus
Conditions	The context listed earlier under which it was established, plus the surface for any limit that depends on traction	Loaded to at most 368 kg; $\mu\,\phi_{\text{drive}} \geq 0.204$, so $\mu \geq 0.250$ at the hub motors' rated load share and never on the inspected floor ($\mu = 0.12$); tip-over geometry not yet measured; usable powered braking and energy absorption; tires at commissioning wear	Drive current limit at 25 A; both driven wheels in traction; rotating mass within the 9.5 kg per-wheel bound; windings and bus within their rated ranges	Sustained coincident load on the 48 V bus; the permission path isolated on its held-up rail; drive power and physical stopping behavior require separate qualification
Uncertainty	The measurement uncertainty of each instrument, with its observation point and bandwidth	Not established	Datasheet tolerances only; the torque constant's tolerance is not published	Not established
Margin	The engineering margin separating the operating target from the breakdown threshold	No justified deceleration margin yet; the chosen 100 mm protective clearance must be checked against the eventual uncertainty bound	At least 16.6% above a_{brake}; none if the current limit falls to the rated 8.5 A	4.8 V from a depleted pack under sustained coincident load; transient minimum not established
Detector	The runtime check that catches an excursion while the machine runs, such as a phase-current sensor, an optical interrupter, or a supply-rail monitor	Deceleration monitor comparing commanded with measured deceleration from the wheel encoders and the Bosch BMI088 inertial unit, on every stop	Drive saturation flag and measured phase current against the configured limit, read on every stop	Bus-voltage monitor flags undervoltage; the permission path selects a response whose power and mechanical premises still hold
Provenance	The evidence category and the instrument or model that produced the value	Illustrative, from the base's class profile; to be replaced by loaded stopping trials	Datasheet standing: computed from the hub motor's datasheet values; to be replaced by wheel-torque measurements at the current limit on a loaded braking trial	Converter datasheet; source resistance illustrative; to be replaced by a bus characterization under coincident load

The detector element matters most once the machine leaves the test stand, because operation can carry a limit outside the conditions under which it was established. The saturation duration against which the Schiaparelli lander's inertial unit had been accepted was such a limit. The lander's loss in 2016 shows what follows when a limit's conditions are exceeded in operation and nothing downstream checks the result.

War Story 2.1: ExoMars Schiaparelli IMU saturation (2016)

Context: On October 19, 2016, the European Space Agency's ExoMars Schiaparelli lander executed its atmospheric entry and descent toward the Martian surface, relying on an onboard Guidance, Navigation, and Control (GNC) computer to fuse Inertial Measurement Unit (IMU) gyroscopes with a Doppler radar altimeter (ESA Investigation Board 2017).

Mechanism: About three minutes after atmospheric entry, the parachute inflated and set the lander oscillating. The IMU measured a pitch rate larger than expected and raised a saturation flag, which persisted longer than the 15 ms assumed when the unit was accepted. While the flag was set, the GNC software integrated the saturation threshold rate as though it were the true rate, although the lander was in reality oscillating, and its attitude estimate drifted by about 165°, nearly upside down. When the radar altimeter later returned valid slant ranges, the software projected them onto that attitude and computed a negative altitude. No onboard check tested that altitude for plausibility.

Impact: Taking the negative altitude as a valid estimate, the guidance logic released the parachute and backshell, fired the braking thrusters for 3 seconds instead of the planned 30 seconds, and activated its on-ground systems as if it had landed. The lander was still about 3.7 km above the surface. It fell freely for about half a minute and struck the ground at about 150 m/s (540 km/h).

Response: The ESA inquiry board recommended robust sanity checks in the onboard software, among them that altitude cannot be negative and cannot change from 3.7 km to a negative value in one second, and a design that handles IMU saturation explicitly, for example by inhibiting IMU inputs during parachute deployment.

Systems lesson: A physical body cannot transcend the saturation ranges of its sensory transducers. When a sensor's physical limit (a gyroscope's rate ceiling or an accelerometer's dynamic range) is exceeded, a state estimator that integrates the saturated measurement as if it were valid drifts away from the physical state. The saturation duration against which the IMU had been accepted was a limit with conditions, and once descent exceeded them, no detector checked the estimate before it commanded an irreversible action.

Checkpoint 2.3: Limit records

A limit governs a decision only together with the conditions, evidence, and detector that travel with it.

- ☐ **Uncertainty vs. margin**: Can you distinguish the uncertainty of a measured deceleration from the margin the machine defends, and explain why only a measured value, or one derived from measurements, can support that margin?
- ☐ **Zero-failure bound**: Can you estimate how many failure-free loaded stops would bound the per-stop failure rate below one in 10,000 at 95 percent confidence, and state the two assumptions about the trials that the bound requires?
- ☐ **Record conditions**: Can you name the condition of the credible-deceleration record that a lowered drive current limit, the inspected floor, and an arm extended forward would each violate?
- ☐ **Detector**: Can you explain why the Schiaparelli lander's accepted saturation duration failed as a limit in flight, and which detector in table 2.3 plays the missing role for the credible deceleration?

Measured and recorded one at a time, the five limits describe the body budget by budget, each under its own conditions. In operation, however, a single maneuver draws on all five budgets at once, and a change in one budget's state can erode the margin of another. Which limit binds first therefore depends on both the machine's physical class and the specific command under evaluation.

2.8 Which Physical Budget Binds First

A single command from an action-chunking policy to accelerate a loaded mobile manipulator draws concurrently against all five physical budgets, which is why the simultaneous admission invariant of section 2.1 admits it only when every margin is non-negative at once. The command executes against a camera frame that has already aged during neural inference, so that age enters the pre-brake delay.

At the same time, the acceleration injects kinetic energy into the chassis, which friction brakes or regenerative drives must eventually dissipate. Generating that motion forces the arm's geared joints to accelerate reflected rotor inertia, while the base's direct-drive hubs add their rotating inertia to the stopping load. The phase currents that produce those torques dump resistive Joule heat into stator windings, which shed thermal energy only over minutes. The same currents pull transient current from the shared DC rail that powers the onboard compute host. Each of the five budgets pairs a governing physical law against the software assumption it invalidates, exposing a characteristic vulnerability by machine class and demanding a hardware-enforced runtime invariant (table 2.4).

2.8.1 Which budget binds in each machine class

A mobile manipulator integrates two of the machine classes from section 1.5 onto a single chassis: a Class 1 wheeled base governed primarily by chassis momentum and stopping distance, and a Class 2 articulated arm constrained by reflected rotor inertia, contact compliance, and static holding heat. While the classes identify the primary vulnerabilities, the instantaneous maneuver determines which budget binds first. For instance, holding a heavy payload in a cantilevered pose can drive joint windings toward their thermal dissipation limit over tens of seconds. Conversely, a sudden multi-axis acceleration burst coupled with full neural network execution can deplete electrical bus margins within milliseconds. The coupled calculations below compare the evidence behind the running machine's limits and identify a stopping qualification priority. They do not establish an ordering of all five physical margins for an unspecified maneuver.

Table 2.4: Cross-class physical constraint matrix: The five budgets, the law that governs each, the assumption a software-trained policy makes about it, the class characteristically vulnerable, and the runtime invariant that answers it.

Physical Budget / Factor	Governing First Principle & Scaling Law	Software Illusion vs. Hardware Ground Truth	Characteristic Vulnerability by Class	Systems Architect Takeaway & Runtime Invariant
1. Measurement Freshness & Age	$\Delta t_{\text{fresh}} \leq \frac{e_{\max}}{v_{\max}}$; $\delta\theta_{\text{enc}} = \frac{2\pi}{2^n}$; end-to-end age measured from exposure	*Illusion:* High inference rate (50 Hz) guarantees real-time control. *Truth:* Age begins at exposure; encoder differentiation amplifies quantization noise; vibration harmonics alias into IMU attitude drift.	**Class 1 & 2:** Erodes clearance margins and destabilizes high-gain contact loops.	Tag frames with hardware timestamps; enforce $t_{\text{age}} \leq$ the budgeted age at motor registers; drop stale commands.
2. Momentum & Stopping Distance	$d_{\text{stop}} = v\,\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} \leq D_{\text{clear}}$; $m_{\text{eff}} = m + \sum_i \frac{N_{\text{gear}_i}^2 J_{\text{rotor}\,i}}{r^2}$	*Illusion:* Deceleration can be commanded instantaneously. *Truth:* Braking distance scales quadratically (v^2); retarding torque must decelerate reflected rotor inertia; the stopping system must absorb or dissipate energy.	**Class 1 (Mobility):** Primary constraint. Doubling speed quadruples the braking term.	Solve clearance quadratic $v_{\max}(D_{\text{clear}}, \tau_{\text{delay}})$ on the safety microcontroller at the rate the stopping budget sets; clamp policy velocity.
3. Actuator & Reflected Inertia	$J_{\text{ref}} = N_{\text{gear}}^2 J_{\text{rotor}}$; $\tau_{\text{motor}} = K_t i_q$; $\tau_e = \frac{L_{\text{phase}}}{R_{\text{phase}}}$; $\frac{d\tau_{\text{motor}}}{dt} \leq \frac{K_t(V_{\text{bus}} - e_{\text{bemf}})}{L_{\text{phase}}}$	*Illusion:* Motor torque tracks policy tensors instantaneously. *Truth:* Current rise limits response; gearboxes square rotor inertia (N_{gear}^2); high ratios increase resistance to rapid contact motion.	**Class 2 (Manipulation):** Contact stiffness and reflected inertia constrain interaction.	Limit acceleration and torque slew; measure load-side response; select gearing and compliance for the qualified contact envelope.

Physical Budget / Factor	Governing First Principle & Scaling Law	Software Illusion vs. Hardware Ground Truth	Characteristic Vulnerability by Class	Systems Architect Takeaway & Runtime Invariant
4. Thermal Limits & Duty Cycle	$C_{\text{th}}\frac{dT_{\text{winding}}}{dt} = \frac{\tau_{\text{motor}}^2}{K_t^2}R(T_{\text{winding}}) - \frac{\Delta T}{R_{\text{th}}}$; $\tau_{\text{cont}} = K_t\sqrt{\frac{T_{\text{limit}}-T_{\text{amb}}}{R(T_{\text{limit}})R_{\text{th}}}}$	*Illusion:* Motors deliver catalog peak torque continuously. *Truth:* Torque is current, current is heat ($P_{\text{loss}} \propto \tau_{\text{motor}}^2$); continuous torque is limited by heat rejection at the characterized winding-temperature limit T_{limit}.	**Class 3 (Thermal) & Class 2:** Holding above the qualified continuous torque can overheat the windings.	Run discrete I^2t thermal observers on the safety microcontroller; dynamically derate speed and torque when hotspot models approach limits.
5. Electrical Power Integrity	$V_{\text{source}} - V_{\text{bus}} = R_{\text{bus}}I_{\text{source}} + L_{\text{bus}}\frac{dI_{\text{source}}}{dt}$; $C_{\text{bus}}\frac{dV_{\text{bus}}}{dt} = I_{\text{source}} - I_{\text{load}}$; $V_{\text{final}} = \sqrt{V_{\text{initial}}^2 + \frac{2E_{\text{regen}}}{C_{\text{bus}}}}$	*Illusion:* Power rails act as infinite, ideal voltage sources. *Truth:* Bus impedance collapses rail voltage under combined compute+motor bursts; braking surges overvoltage.	**Class 1 & 3:** Peak multi-axis acceleration trips microcontroller brownout resets; regen surges destroy inverter FETs.	Enforce coordinated compute-actuation power budgets and dI_{source}/dt limits; install dynamic brake choppers; characterize unpowered fallbacks.

2.8.2 Cross-budget coupling and permission rail isolation

When an autonomous agent commands an aggressive multi-axis acceleration, the physical margins do not degrade in isolation. They deplete together. Drive inverters draw substantial current to overcome both chassis inertia and reflected rotor inertia. This transient current surge causes the shared DC bus voltage to sag, even as resistive losses ($P_{\text{loss}} = i_q^2 R_{\text{eff}}$) steadily deposit thermal energy into motor windings. Because motor phase currents are pulse-width modulated by inverters and do not map directly to battery draw, systems architects must evaluate input current at the primary DC bus. This coincident load step establishes the true operating baseline across all subsystems.

On the 48 V drive bus, the sustained electrical screen passes under its stated assumptions. As shown in the coincident power step of notebook 2.7, a depleted battery pack at 42 V sags to 37.8 V under the coincident load step. Even under that sustained step, the rail keeps a 4.8 V buffer above the 33 V undervoltage lockout (UVLO) threshold of the 48-to-24 V converter, although the sustained screen does not establish the transient minimum.

The stopping dynamics expose a concrete qualification priority. Under emergency braking, the drive motors clear the required deceleration by only 16.6 percent, and only with phase current at the motor's catalog peak. The chassis tip-over boundary has not been measured at all. For the arm, the optimistic stand-in thermal model gives a heat-soaked joint a threefold current burst of 43.1 s before its winding reaches the insulation rating. Additionally, the chassis stopping distance at the full 1.5 m/s travel limit leaves 357.5 mm of clearance to accommodate perception latency, communication jitter, and tracking variance. The whole stopping budget inherits the credible-deceleration premise, which rests on the peak current limit and the unmeasured center of gravity, so that premise requires qualification before relying on the stop. The transient rail minimum and installed arm thermal response remain unresolved too. Ranking the budgets requires a specified maneuver and duration, with comparably qualified limits for each.

Modern SoC architectures deploy dynamic voltage and frequency scaling (DVFS) algorithms to regulate die temperature and energy consumption. While DVFS protects silicon from thermal degradation, its timing penalties propagate directly into physical workspace clearance:

SoC thermal dissipation $\longrightarrow$ DVFS clock throttling $\longrightarrow$ vision latency stretch $\longrightarrow$ observation age inflation $\longrightarrow$ pre-brake travel expa

When sustained motor currents and high-throughput perception pipelines elevate temperatures across the shared enclosure, the host kernel throttles the clock frequencies of onboard CPU and GPU accelerators. On the running machine, this thermal intervention stretches the navigation backbone that the stopping budget charges, not the grasp path of table 2.1, from 22 ms to 33 ms. This delay adds 11 ms directly to the end-to-end observation age t_{age}, inflating the pre-braking reaction time τ_{delay}. When traveling at the maximum base velocity of 1.5 m/s, this algorithmic latency injects 16.5 mm of unmonitored chassis travel before emergency brakes can even begin mechanical engagement.

In this manner, a standard operating-system power policy designed for digital silicon silently shrinks the robot's stopping margin.

↪ Thermal clock throttling on the shared die, and the placement decisions that bound its effect on the safety path, are developed in section 13.4.

Surviving a sustained bus voltage drop does not qualify the shared power rail to support safety-critical control channels. Transient dips during motor reversals remain difficult to bound completely, and electrical faults such as an open circuit breaker, a battery management system (BMS) overcurrent disconnect, or a blown DC-DC stage drop bus voltage to zero instantly. The running machine addresses these common supply faults with a **dual-rail architecture** that decouples motor dynamics from safety enforcement. In this design, the primary shared bus powers high-current traction inverters and steps down voltage for the primary application processor. An isolated, reverse-blocked permission rail separately powers the safety microcontroller, deterministic fieldbus transceivers, and holding-brake release coils. Under the drive-rail faults covered by its isolation and hold-up qualification, this separate supply preserves the permission loads. It does not make the detector, controller, or communication path infallible.

The permission rail incorporates local capacitive energy storage to preserve control availability during a loss of its upstream supply. For the stated constant 70 W permission load, ideal energy sizing gives an onboard 2.025 F supercapacitor bank 140 J of usable energy between its minimum nominal charge voltage (21.5 V) and regulator dropout (18 V), targeting 2 s of hold-up. Establishing a minimum available duration requires qualification of initial charge, capacitance tolerance and aging, conversion losses, equivalent series resistance, current demand, and regulator behavior. The 750 ms stop from the drive limit plus the 80 ms slow end of spring-brake engagement is a comparison for the powered stopping model only. Total main-battery failure does not establish that this model remains available. The qualified control interval must cover detection and decision, the duration of the response available under the fault, and brake engagement when those stages are sequential, or their actual sequence when they overlap. Section 12.9 checks the hold-up against the longest stop its fallback ladder commands.

Under the qualified rail and intact detector and controller assumptions, maintaining permission-rail power preserves the opportunity to detect power loss and initiate a response, but it does not supply the mechanical energy or high-current paths necessary for controlled regenerative braking. A controlled, powered stop requires intact inverter switching, sufficient bus voltage, and an operational energy dump, such as a dynamic brake chopper or a regenerative battery sink, to absorb kinetic deceleration energy. When a catastrophic power fault removes the supply or absorption path needed for a powered stop, that stopping model no longer applies. Under such conditions, stopping distances must be governed by the unpowered physical fallbacks analyzed in section 2.6.2. A held-up permission rail does not change what the mechanical brakes deliver. Powering their release coils from that rail lets the permission logic choose when the spring brakes engage, but it does not give them the deceleration of a powered drive stop. Systems architects must treat stopping performance as conditional: isolate the safety supervisory rail, and then characterize the exact deceleration capabilities that remain active across every distinct electrical fault state. Downstream runtime enforcement and hardware qualification must preserve these underlying electrical premises.

2.9 Fallacies and Pitfalls

Physical limits rarely respect subsystem boundaries. In embodied machine learning systems, an architecture whose individual component budgets balance cleanly on paper can fail catastrophically when thermal rise, bus latency, power supply sag, and mechanical dynamics interact under compound operational load.

Fallacy: *Model inference time defines the measurement freshness latency.*

Consider an engineering team that compresses its vision backbone, cutting neural network inference time from 30 ms to 10 ms. While this optimization provides a threefold compute speedup, the physical robot still executes actions based on visual state that traversed camera exposure, sensor readout, DMA transfers, and kernel queuing delays. Once the model outputs an action, that command must traverse a real-time network fieldbus before motor drive inverters can slew coil currents and produce physical force. A 45 ms scheduling or transport jitter spike elsewhere along this pipeline eclipses the 20 ms compute gain, consuming safety margins before the robot can react. Real-time control requires budgeting and bounding the worst-case tail of the entire capture-to-onset

pipeline across synchronized hardware clocks, rather than certifying loop safety from isolated model benchmarks.

Architect's countermeasure: Measure the true capture-to-onset latency tail on synchronized hardware clocks; never certify real-time safety from neural network inference benchmarks alone.

Fallacy: *Software step commands deliver instantaneous physical torque.*

Reinforcement learning policies trained in simulation routinely issue piecewise-constant torque or position setpoints at 50 Hz, implicitly assuming the physical actuator behaves as an ideal, memoryless force source. In physical reality, an electric drive is an electromechanical dynamic system governed by electromagnetic and inertial limits. Developing torque requires inner Field-Oriented Control (FOC) loops whose current rise rate is strictly bounded by stator winding electrical time constants ($\tau_e = L_{\text{phase}}/R_{\text{phase}}$) and DC bus voltage headroom ($\frac{d\tau_{\text{motor}}}{dt} \leq \frac{K_t(V_{\text{bus}} - e_{\text{bemf}})}{L_{\text{phase}}}$). Mechanically, a high-ratio gearhead amplifies rotor inertia quadratically ($J_{\text{ref}} = N_{\text{gear}}^2 J_{\text{rotor}}$), imposing a steep low-pass filter on achievable link acceleration.

When a discrete tensor update demands an abrupt torque step, substantial electromagnetic effort is diverted into accelerating the spinning rotor rather than moving the external link. Transmission compliance and joint elasticity convert these high-frequency torque transients into structural vibration and limit-cycle oscillations. Simultaneously, repeated high-current corrections waste energy as resistive heat dissipation ($P_{\text{loss}} = \frac{\tau_{\text{motor}}^2}{K_t^2} R_{\text{eff}}$), rapidly burning through the actuator's thermal margin. Systems architects must filter and shape setpoints against actual plant dynamics, validating commanded trajectories against delivered physical response rather than assuming numerical smoothness in software guarantees smooth physical execution.

Architect's countermeasure: Bound acceleration and torque slew using measured actuator response, include electrical current rise and reflected inertia in the model, and qualify the chosen transmission and controller for the intended contact task.

Pitfall: *Treating isolated component datasheets as operational system guarantees.*

Hardware component datasheets report performance envelopes measured under idealized bench conditions. When a motion planner adopts a motor vendor's continuous torque rating or an IMU vendor's nominal noise floor as operational bounds, it ignores the harsh realities of system packaging. A motor catalog rating reflects vendor test benches (section 2.5), typically an open motor bolted to an enormous aluminum heatsink in still air at $25\ ^\circ$C, not a sealed robotic joint housing tucked inside an unventilated composite limb on a chassis dissipating heat from a 60 W application processor.

In physical deployment, an actuator's continuous torque is strictly constrained by steady-state heat dissipation up to the winding's engineering ceiling T_{ceil}, so the installed rating is equation 2.7 evaluated at the ceiling rather than the insulation rating:

$$\tau_{\text{cont}} = K_t \sqrt{\frac{T_{\text{ceil}} - T_{\text{amb}}}{R(T_{\text{ceil}}) R_{\text{th}}}}$$

Given a 140 °C engineering ceiling, shifting the motor from the vendor's 25 °C bench into the 40 °C interior of an enclosed chassis strips away 6.7 percent of continuous torque. Because thermal dissipation scales inversely with the square root of thermal resistance ($1/\sqrt{R_{\text{th}}}$), eliminating forced convection and heatsink mass degrades capacity even further: doubling R_{th} inside the arm housing cuts total usable continuous torque by 34.1 percent relative to the catalog rating. If trajectory planners treat catalog continuous torque as achievable duty, persistent copper losses ($P_{\text{loss}} = \frac{\tau_{\text{motor}}^2}{K_t^2} R_{\text{eff}}$) will exceed heat dissipation rates, triggering thermal cutouts or permanently degrading winding insulation.

Sensors suffer the same systemic degradation: inertial measurement units and optical encoders calibrated on quiet granite surface plates suffer severe vibration rectification, thermal drift, and mechanical aliasing once bolted near high-frequency switching inverters and planetary gearsets. System limits must reflect empirical measurements collected on the fully assembled machine under peak operational stress rather than pristine standalone datasheets.

Architect's countermeasure: Calibrate actuator continuous torque ceilings and sensor noise profiles on fully assembled, enclosed hardware across the entire operational temperature envelope rather than copying open-bench catalog numbers into planner configuration files.

Pitfall: *Treating dynamic voltage and frequency scaling (DVFS) as a transparent software optimization.*

In server computing, DVFS operates as an invisible operating-system service that reduces clock frequency and core voltages to prevent silicon junction overtemperature. In an embodied system, however, DVFS directly degrades the real-time control contract. Consider a mobile manipulator whose high-current drive motors heat a shared structural chassis during intense maneuvering. As heat conducts into the compute bay, the application processor throttles its clock frequency to avoid thermal shutdown.

The system kernel treats clock throttling as an isolated power-management decision. However, a downclocked vision backbone adds observation age, and observation age at speed is unmodeled blind travel before the safety monitor can command braking. Section 2.8.2 prices that travel on the running machine. Although thermal throttling successfully shields the processor silicon, the vehicle's dynamic stopping envelope silently expands beyond calibrated safety margins.

Architect's countermeasure: Feed SoC thermal and clock-frequency telemetry directly into the motion safety monitor, dynamically derating maximum travel speed whenever compute throttling inflates perception age.

Pitfall: *Scaling operational velocity without recalculating quadratic braking distances and thermal dissipation.*

Facility operators frequently seek to increase robot throughput by raising maximum cruise velocity in software configurations. This operational change couples two punishing physical nonlinearities. First, the braking term grows with the square of speed, which is why Case 3 of figure 2.2 breaches the clear distance at 2 m/s.

Second, absorbing the kinetic energy of the faster machine demands large dynamic braking currents that dump heat directly into motor stator windings and drive electronics via Joule heating ($P_{\text{loss}} = i_q^2 R_{\text{eff}}$). An isolated emergency stop performed during commissioning may pass thermal checks when the machine starts cold. However, when an autonomous vehicle repeats that higher-speed duty cycle across an eight-hour shift, winding temperatures compound across successive maneuvers, rapidly outstripping the chassis thermal dissipation budget. System planners must treat operating velocity as a gated admission parameter: any commanded speed increase requires re-evaluating both the expanded quadratic stopping envelope and the cumulative multi-cycle thermal duty budget.

Architect's countermeasure: Treat operating speed as an admission parameter that re-evaluates both quadratic stopping envelopes and multi-cycle thermal accumulation before granting velocity increases.

Fallacy: *Compute accelerators and motor inverters operate on independent, ideal power rails.*

Software architectures typically treat accelerator power consumption and motor drive power as decoupled subsystems, verifying each against isolated power supply ratings. In physical mobile machines, however, both systems draw from a shared, unregulated DC battery bus.

Consider a scenario where a heavy neural network vision burst coincides precisely with the drive motors accelerating to push the mobile base over an uneven threshold. On the shared 47.7 V electrical bus, this coincident current surge demands a combined 70 A step. Drawn through the 60 mΩ source resistance of cell, harness, and switch, this sudden draw induces a sustained 4.2 V voltage sag. That sag consumes nearly half of the 9 V voltage margin separating a depleted battery pack from the 33 V UVLO threshold of the application processor's DC-DC converter, even before harness inductance and bulk capacitor discharge deepen the dynamic transient. A step of 150 A, or the identical current demand drawn from an aging or cold battery with sufficiently elevated internal impedance, reaches the UVLO threshold and can interrupt the primary processor's supply.

If safety microcontrollers and brake solenoids share that unbuffered power rail, the resulting brownout crashes the safety supervisor simultaneously. The machine is plunged into the uncontrolled, unpowered fail states described in section 2.6 while carrying significant physical momentum, rather than executing an orderly, controlled deceleration. Software power budgets must acknowledge electrical coupling at the physical bus. System qualification demands testing coincident worst-case

compute and motor loads at minimum battery state of charge, while ensuring safety permission logic and electro-mechanical brakes operate on an isolated, held-up power rail (section 2.8).

Architect's countermeasure: Isolate the permission logic and brake releases, then qualify both their hold-up and the physical stopping response under coincident motor and compute load at minimum battery state of charge. A powered stop also requires the drive rail and energy-absorption path to remain usable.

Fallacy: *Passing isolated single-variable bench tests guarantees physical survival under compound operational stress.*

Development teams often qualify physical AI systems through single-variable stress testing: a robotic manipulator verifies maximum payload capacity inside a 20°C air-conditioned laboratory, then undergoes thermal validation inside a 45°C environmental chamber with an unloaded gripper. Neither benchmark, however, validates whether the manipulator can lift maximum payload in high ambient heat while operating from a depleted battery pack.

Physical failure modes couple nonlinearly across subsystem domains. Under simultaneous operational stress, manipulating maximum payload demands sustained high motor currents that dramatically increase resistive winding losses ($P_{\text{loss}} = i_q^2 R_{\text{eff}}$). An ambient temperature of 45°C severely compresses the thermal gradient required to reject that heat to the environment, accelerating stator insulation degradation. Concurrently, a low battery state of charge depresses DC bus voltage, stripping away the voltage headroom required to overcome motor back-EMF (e_{bemf}) and severely degrading current slew rates ($\frac{di_q}{dt}$). An arm that moves smoothly in isolated tests may experience joint stall, severe tracking errors, or premature thermal shutdown under their compound interaction. Hardware and software qualification cannot rely on orthogonal single-variable test sweeps. Engineering teams must validate boundary envelopes on fully integrated machines subjected to simultaneous peak stresses across payload, thermal ambient, battery state of charge, and control frequency.

Architect's countermeasure: Test boundary envelopes under worst-case simultaneous physical stress (maximum payload, elevated ambient temperature, depleted battery, and high-frequency actuation) to catch multi-budget coupling failures.

2.10 Summary

Every motion plan in physical AI must ultimately satisfy hardware invariants. Five coupled budgets translate a high-level trajectory request into concrete engineering constraints: observation age, stopping distance, delivered torque, winding temperature, and supply voltage. Because these budgets are tightly coupled, no single limit can be evaluated in isolation. Consider the warehouse robot operating on a 48 V drive bus: while the sustained electrical draw clears the converter lockout even from a depleted battery pack, the stopping budget rests on fragile assumptions. The machine's credible deceleration margin is narrow and holds only while the drive current limit stays at the motor's peak. Furthermore, its traction model depends on dynamic wheel loading, and its anti-tip threshold lacks empirical center-of-mass validation. Isolating and sustaining the safety permission path on a dedicated capacitor-backed supply protects digital control availability during bus transients. Certifying the physical safety envelope, however, still requires verifying the actual drive power or passive braking dynamics available during an electrical fault.

To make these boundaries enforceable, a physical limit cannot be treated as a bare scalar. Instead, systems engineers structure each boundary as a limit record that documents valid operating conditions, measurement uncertainty, safety margins, runtime detection mechanisms, and calibration provenance. On the reference mobile manipulator traveling at its 1.5 m/s drive limit, mechanical stopping dynamics consume 742.5 mm of the 1.10 m clearance distance. That figure already includes the localization bound and protective clearance, so the remaining 357.5 mm is a design budget. It covers the delay terms the body does not own (observation age, chunk lease, permission tick, and bus cycle), stopping-profile shaping, and the tracking bound that later chapters add. These are the first three rows of the stopping budget, which table 12.1 completes term by term, with the chapter that prices each one. These analytical values guide initial system architecture, while physical characterization of the fully assembled robot establishes the empirical envelopes that production controllers enforce across the governing relations of all five budgets (table 2.5).

Table 2.5: Key equations of the physical body: Each load-bearing equation of the chapter, the limit it bounds or the decision it settles, and the equation where it is derived; $D_{\text{avail}} = D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}}$.

Equation	What it bounds or decides	Derived in
$\Delta t_{\text{fresh}} \leq e_{\max} / v_{\max}$	The largest observation age before a moving target's own motion spends the position tolerance	equation 2.1
$d_{\text{stop}} = v\,\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}}$	Whether the body stops within the clear distance, $d_{\text{stop}} \leq D_{\text{clear}}$	equation 2.2
$v_{\max} = -a_{\text{brake}}\tau_{\text{delay}} + \sqrt{(a_{\text{brake}}\tau_{\text{delay}})^2 + 2a_{\text{brake}}D_{\text{avail}}}$	The admissible speed for a given clearance, pre-brake delay, and credible deceleration	equation 2.3
$\tau_{\text{out,max}} = \eta_{\text{gear}} N_{\text{gear}} K_t I_{\max}$	The largest torque the configured current limit allows, and so whether a_{brake} is deliverable	equation 2.4
$N^* = \sqrt{J_{\text{load}} / J_{\text{rotor}}}$	The gear ratio that maximizes joint acceleration for a given motor torque	equation 2.5
$P_{\text{loss}} J_{\text{ref,in}} = \tau_{\text{out}}^2 J_{\text{in}} / (\eta_{\text{gear}}^2 K_m^2)$	The fixed trade between holding heat and reflected inertia that the gear ratio cannot escape	equation 2.6
$\tau_{\text{cont}} = K_t \sqrt{(T_{\text{limit}} - T_{\text{amb}}) / (R(T_{\text{limit}})\, R_{\text{th}})}$	The torque a winding sustains indefinitely under the declared cooling	equation 2.7
$D_{\text{cycle}} \leq (T_{\text{ceil}} - T_{\text{amb}}) / (P_{\text{peak}}\, R_{\text{th}})$	An average-power screen on a burst schedule, optimistic against the transient peak	equation 2.8
$V_{\text{source}} - V_{\text{bus}} = R_{\text{bus}} I_{\text{source}} + L_{\text{bus}}\, dI_{\text{source}}/dt$	Rail droop under a coincident current step, against the undervoltage lockout	equation 2.9
$V_{\text{final}} = \sqrt{V_{\text{initial}}^2 + 2E_{\text{regen}}/C_{\text{bus}}}$	The capacitor-only bus voltage after a regenerative stop, and so whether a brake chopper is needed	equation 2.10

Key Takeaways: The physical body and invariant budgets

- **Age begins at physical transduction**: The data freshness budget spans the entire timeline from physical photon or contact arrival to mechanical force delivery. Because stage latencies vary dynamically, an end-to-end tail percentile is never the simple sum of individual stage percentiles. Robust software must preserve hardware capture timestamps, account for downstream actuation lag, and reject any action proposal whose underlying sensory evidence will be stale when the actuator responds.
- **Speed sets the stopping envelope**: The clearance quadratic in speed determines an admissible velocity ceiling only while runtime state, stopping mechanics, and credible deceleration satisfy its mathematical premises. In turn, credible deceleration cannot exceed the strictest bound among tire traction, vehicle tip-over thresholds, and available motor braking torque.
- **Gearing trades torque for reflected inertia**: A transmission multiplies actuator torque linearly by the gear ratio N_{gear}, but reflects rotor inertia back to the joint by N_{gear}^2. Because physical motion also depends on current rise times, inverter voltage headroom, mechanical compliance, and joint friction, software acceptance of a setpoint command never guarantees physical trajectory tracking.
- **Thermal history limits available torque**: Resistive copper loss ($P_{\text{loss}} = i_q^2 R_{\text{eff}}$) scales quadratically with current, and winding resistance increases as internal temperatures climb. Actuator burst schedules must therefore respect transient thermal peaks rather than relying on average power limits. Furthermore, reducing holding current to alleviate thermal strain requires mechanical brakes or counterbalancing mechanisms capable of supporting static gravity loads.
- **The supply rail is shared**: Motor inverters and digital compute share a common DC bus. Coincident acceleration demands induce inductive rail droop, rapidly depleting voltage headroom above converter undervoltage lockouts, while unclamped regenerative braking can elevate bus voltage and damage inverter stages. An isolated, capacitor-

backed power domain preserves safety permission logic through transient brownouts. However, completing a controlled emergency stop still requires sufficient braking torque, energy absorption, and stopping clearance; otherwise, the machine must rely entirely on a qualified unpowered response.

- **A limit is a record, not a number**: An engineering threshold safely governs an execution decision only when coupled with its valid operating regime, measurement uncertainty, safety margin, fault detector, and calibration provenance. Moreover, these limits lapse as hardware degrades (rising thermal resistance, increasing battery internal impedance, transmission backlash wear), so they require continuous lifecycle monitoring and recalibration.

Every budget in this chapter ends the same way, as a number the permission path can compare against a proposal before that proposal becomes current in a winding. That comparison is how the first law, irreversibility (principle 1), enters the machine as engineering rather than as a warning. Work delivered across the causal boundary cannot be recalled, so each command must remain subject to refusal before it produces physical work. Refusing it takes a limit that states its value, the conditions under which it holds, how uncertain it is, and what detects its violation. In operation, each relevant limit record must support the proposed maneuver under its actual conditions. On the running machine, the electrical budget passes only its sustained screen, and the arm's illustrative thermal model gives a burst lasting tens of seconds. The credible deceleration rests on peak drive current, a dry floor, and an unmeasured center of gravity, and every stopping calculation inherits those conditions. When a budget looks comfortable, the first question is which condition of its record the next maneuver could violate, and the second is whether a detector would notice before the stop is needed.

2.11 Historical Perspective and Further Reading

The freshness budget rests on foundations established well before the advent of digital control. Nyquist (1928) quantified the transmission capacity of telegraph channels relative to available bandwidth. Shannon (1949) formalized the sampling theorem in the form used throughout this chapter: a continuous signal band-limited below half the sampling frequency is completely determined by discrete samples, whereas unattenuated higher-frequency dynamics fold back into the passband as aliasing distortion. Clock synchronization subsequently migrated from software operating systems directly into physical networking silicon. The Precision Time Protocol (IEEE 2020a) and its profile for time-sensitive Ethernet (IEEE 2020b) capture packet timestamps at the physical layer interface. They allow an embodied system to preserve each sensor measurement's capture timestamp under bounded capture and clock-synchronization uncertainty as it travels through the compute fabric.

Modern actuator architectures trace back to two distinct engineering lineages. The first lineage eliminated mechanical transmissions entirely to maximize backdrivability. Asada and Kanade (1983) developed early direct-drive robot arms to bypass the nonlinear friction, backlash, and torque ripple inherent in high-ratio gearboxes. Three decades later, the MIT Cheetah quadruped revisited this architecture with the quasi-direct drive of section 2.4.5, coupling high-torque-density motors with low-ratio single-stage planetary gearsets to keep most of that transparency while adding impact resilience (Seok et al. 2015; Wensing et al. 2017). The second lineage treated transmission compliance not as an unwanted defect, but as an explicit design variable. Musser (1959) patented the strain wave gear (harmonic drive), whose compact form factor and high reduction ratio trade off torsional stiffness for torque density. To analyze these mechanics, Spong (1987) introduced the flexible-joint model, demonstrating how elastic coupling dynamically decouples motor inertia from the load link. Moving beyond rigid position tracking, Hogan (1985) reframed physical contact as an exchange of energy governed by a programmed mechanical impedance. Building directly on this concept, Pratt and Williamson (1995) introduced series elastic actuators, deliberately placing an elastic element between the gearbox and the output link so that spring deflection provides reliable force sensing while shielding gear teeth from shock impacts. Learned policies successfully mastered these compliant systems once Hwangbo et al. (2019) replaced idealized torque sources in simulation with compact neural networks identified directly from physical hardware.

Thermal and electrical budgets descend from classical electrical machine design and power systems engineering. Montsinger (1930) demonstrated that operational limits for electrical machines should be governed by winding insulation temperature rather than instantaneous current thresholds alone. Dakin (1948) modeled this insulation degradation as a chemical reaction obeying Arrhenius kinetics, proving that thermal aging accelerates exponentially with operating temperature. Modern international standards such as IEC 60085 (International Electrotechnical Commission 2007) still formalize this trade-off between operating temperature and motor operational lifespan. Similarly, standardized stop categories and drive safety functions (International Electrotechnical Commission 2016a, 2016b) codify decades of industrial safety practice for removing power from moving physical mass under fault conditions. Physical AI introduces a new challenge: learned policy proposers operate without built-in guarantees regarding these physical budgets. Deterministic safety monitors must therefore enforce these boundaries directly, translating thermal, electrical, and kinematic constraints into explicit limit records checked along the permission path.

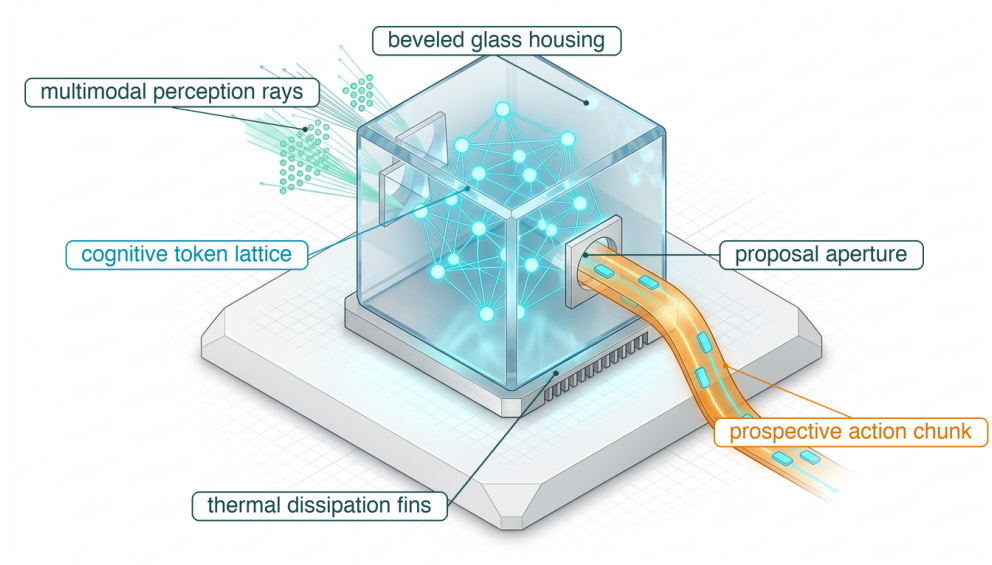
- **Mechanics and actuation from first principles** (Lynch and Park 2017). A comprehensive derivation of rigid-body dynamics, kinematic chains, gearing, and contact mechanics.
- **Impedance control** (Hogan 1985). The seminal paper establishing contact manipulation as dynamic interaction rather than pure position or force control, introducing the virtual spring-damper formulations used to regulate interaction forces.
- **Flexible-joint dynamics** (Spong 1987). The analytical foundation modeling elastic transmission compliance as a torsional spring, demonstrating why motor-side encoders cannot certify link motion or guarantee end-effector trajectory tracking.
- **Series elastic actuators** (Pratt and Williamson 1995). The deliberate introduction of mechanical compliance to trade control bandwidth for impact tolerance, energy storage, and accurate force sensing.
- **Proprioceptive actuator design** (Wensing et al. 2017). Details how low-ratio transmissions paired with high-torque electric motors allow motor current to serve as a reliable contact-force signal, highlighting the associated thermal limits and torque-density trade-offs.
- **Learned actuator models** (Hwangbo et al. 2019). Demonstrates that compact neural networks trained on minutes of empirical hardware data close the reality gap between idealized simulator torques and nonlinear physical actuators.

What's Next: From body limits to brain proposals

Every budget in this chapter now ends as a limit record, and all five can hold while the machine still does nothing useful. At the pick station the observation can be fresh, the stop inside its clearance, the windings cool, and the bus above lockout, yet nothing in the body says which object on the conveyor is the mug or where its handle is. That judgment comes from a learned proposer, and chapter 3 opens with the cost of supplying it in time. A model large enough to recognize unfamiliar objects streams its weights through the same onboard processor that sensing and control depend on, so it renews its proposal far more slowly than the servo loops run. The chapter derives the two-speed brain that the machine card commits to, a slow intent model above a fast chunk policy, and shows why an action chunk keeps the arm supplied without keeping its evidence fresh. The freshness deadline and stopping budget derived here are the windows those proposals must arrive within, and chapter 4 joins the two, admitting each chunk only under a bounded lease that the body's limit records define.

3

The Cognitive Brain

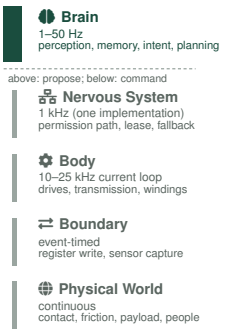


3.1 The Embodied Brain
3.2 The Classical Wall
3.3 Learned Representations
3.4 The Cost of a Fresh Evaluation
3.5 Chunk Supply and Evidence Validity
3.6 Cognitive Vulnerabilities
3.7 Fallacies and Pitfalls
3.8 Summary
3.9 Historical Perspective and Further Reading

Purpose

Why can a robot recognize what to do yet still be too late to act?

Learned models are what let a physical machine handle objects and scenes that no engineer enumerated, and they are also the largest and least predictable computation the machine carries. A model that recognizes the right object still has to move its weights through an onboard processor that sensing and control also depend on, and a larger model that recognizes more takes longer to answer. While it computes, the world keeps moving, so a correct answer can arrive after the conditions that justified it have changed. Predicting a sequence of future motions keeps the actuators supplied between model updates, but it cannot make those motions respond to observations the model has not yet received, and a proposal that arrives on time can still be wrong in ways the model's own confidence does not reveal. An engineer who treats the learned model as a function that returns actions cannot size the processor, set the rates at which proposals are renewed, or say how long the machine may keep moving on an answer that has gone stale. Because motion beyond the causal boundary cannot be recalled, every learned proposal must remain a request that an independent permission path can refuse in time.



Learning Objectives

- Compare classical analytical pipelines and learned representations on coverage, sensory robustness, and formal verifiability
- Calculate the memory wall bound on a model's fresh-evaluation rate ($f \leq \text{BW}_{\text{mem}}/M_{\text{weights}}$) under the embodied batch size 1 constraint, and explain why weight-only quantization shortens the bandwidth-bound decode but not the compute-bound prefill
- Estimate key-value cache expansion from multi-camera tokenization, hidden dimensions, and retained history windows
- Explain how action chunking (H) amortizes off-chip weight transfers without improving closed-loop reaction latency, and why that gap requires a second, smaller proposer
- Compare autoregressive, one-pass, and iterative action decoders by the sequential evaluations and off-chip transfers one chunk costs
- Apply replanning period, inference latency, and target drift to size an action chunk for a dynamic target
- Evaluate the interface between a slow and a fast proposer by what the permission path can check against its own evidence
- Diagnose cognitive failure modes: semantic hallucinations, uncalibrated softmax overconfidence, perceptual chattering, and expired intent leases
- Identify the trajectory, timestamps, and validity information a proposer must hand to independent real-time verification

3.1 The Embodied Brain

DRAFT

The body's limit records tell the permission path what it must refuse, but they cannot say what the machine should do next. That choice belongs to the Brain, the deliberative layer that interprets incoming sensor telemetry and proposes what the physical body should execute next. Over the past decade, physical AI architectures have transitioned from hand-authored geometric pipelines and finite-state machines to learned models, notably vision-language-action (VLA) architectures that map natural language instructions and camera pixels directly to candidate end-effector motions for objects no engineer explicitly modeled. This shift places the machine's most computationally intensive workloads on its least predictable path. A deep neural proposer requires billions of operations, contends for shared memory bus bandwidth with low-latency sensing and control tasks, and predicts future actions for an environment that continues to evolve during inference. A physical constraint governs this interaction: a learned proposal remains useful only if it arrives, and remains physically valid, within the time window permitted by environmental dynamics. Furthermore, every candidate trajectory remains strictly a proposal. The hard-real-time permission path beneath the proposal boundary holds exclusive authority to validate and admit candidate motions before they actuate physical hardware.

The warehouse mobile manipulator illustrates this boundary concretely. The robot can operate within every torque, thermal, voltage, and braking limit of its body and meet the freshness deadline that the body's budgets set for this same pick-station grasp. It can still fail to grasp an unfamiliar ceramic mug traveling along an active conveyor. Low-level limits safeguard physical actuators against structural and electrical damage, but they cannot interpret natural language instructions, recognize a target handle amid visual clutter, or plan an obstacle-free trajectory around a coworker's arm. A learned vision-action model supplies this missing semantic interpretation, which must then be projected into metric Cartesian space using calibrated depth sensing and forward kinematics despite specular reflections, dynamic shadows, and transient occlusions. Because this proposer's training coverage does not establish a closed-loop safety guarantee, and its execution demands must fit within the throughput, memory bandwidth, and thermal envelope of the onboard processor, every generated action sequence remains an unverified candidate until the permission path checks it against fresh telemetry and established safety envelopes.

↩ The mobile manipulator's torque, thermal, voltage, and braking limits are established in section 2.1.

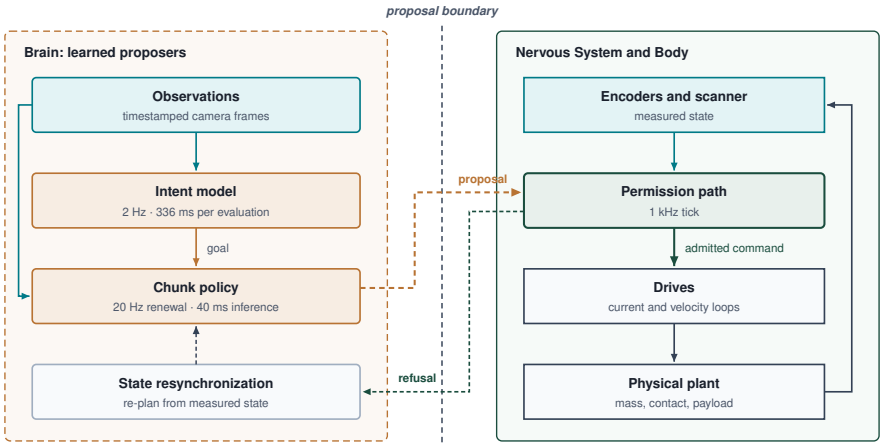


Figure 3.1: Two-speed brain architecture: Above the proposal boundary, two learned proposers run at their own rates, an intent model at 2 Hz and a chunk policy renewing at 20 Hz. Below it, the permission path checks each proposal against measured state on a 1 kHz tick, and the drives close their own current and velocity loops. A proposal carries no authority until it is admitted; a refusal returns the proposers to a plan from measured state.

This division exposes a timing dilemma, because physical dynamics do not pause while inference runs. As the vision model evaluates sensory frames, the conveyor belt advances, and the manipulator must continue tracking previously admitted waypoints without stalling or jerking. Deploying a larger foundation model can enhance open-world object recognition, but the resulting inference latency delays subsequent updates. Systems architects must therefore decouple three distinct temporal quantities: the observation frequency at which the proposer ingests fresh sensor frames, the trajectory horizon spanning the future waypoints contained within each proposal, and the validity window over which those predictions remain physically accurate. Synthesizing longer action horizons can bridge execution between inference cycles, but generating additional waypoints does not refresh the perceptual evidence that produced them.

The machine card of section 1.4.2 already commits the running machine to one answer, the **two-speed brain architecture**: a slow intent model revised at 2 Hz above a fast chunk policy renewed at 20 Hz. Both propose to a permission path that checks measured state on a 1 kHz tick (figure 3.1). The card states these rates; this chapter prices and checks that design. The slow intent model names a target. The fast chunk policy emits an action chunk, the short sequence of future setpoints from one inference that section 1.2 introduced, so the drives below have motion to track between evaluations (Zhao et al. 2023; Chi et al. 2025). Three claims govern this split. Because no single large model can renew its proposal fast enough to supply the arm, the fast policy must emit multi-step chunks, yet a chunk keeps the arm supplied without keeping the evidence behind it fresh. Consequently, the permission path can evaluate and refuse the slow model's intent alongside the fast model's candidate motion only when that intent arrives as a target checkable against independent depth measurements and time: a metric pose stamped with the capture time of the frame that generated it (section 3.5.3).

A first estimate shows why one model cannot do both jobs. If every fresh evaluation must stream all of a model's weights from off-chip memory, its renewal rate can be no higher than the sustained bandwidth divided by the bytes of weights, a ceiling that falls in proportion to parameter count (figure 3.2). The intent model stores its weights as 4-bit integers (INT4, priced in section 3.4.2) rather than 16-bit floating point (FP16), which raises its ceiling fourfold. Both of the machine's proposers still sit below their ceilings, because reading the input tokens in one pass through the model (the *prefill*) and then decoding the output add work that weight streaming alone ignores. Neither approaches the kilohertz rate at which the permission path and the servo loops run. The design problem is therefore deciding which computation can finish before the observation it started from goes stale. Section 3.4.1 derives the ceiling and the work above it; the figure is a preview of where the derivation lands.

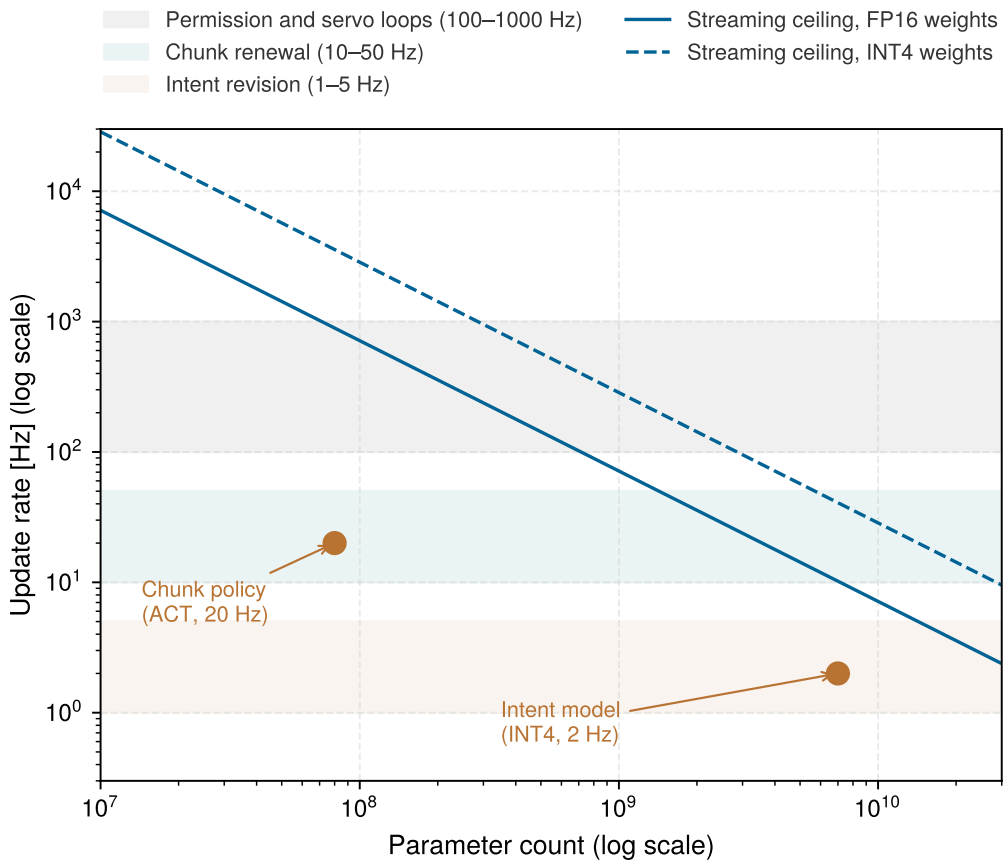


Figure 3.2: Cadence regimes on the machine's processor: The weight-streaming ceiling $f \leq \mathrm{BW}_{\mathrm{mem}}/M_{\mathrm{weights}}$ on the mobile manipulator's application processor for FP16 and INT4 weights (16-bit floating-point and 4-bit integer representations, section 3.4.2) plotted against model parameter counts, alongside the operating points for the machine's intent model and chunk policy. This ceiling establishes an upper bound on throughput rather than an achieved operating frequency, because input token prefill, vision encoding, and autoregressive generation consume execution cycles beyond raw DRAM weight traffic (section 3.4.1).

Testing the two-speed design takes five steps. Classical pipelines built on hand-authored geometry and rigid state machines show why a learned proposer is needed at all, because open-world variability exhausts manual rule sets (section 3.2). Learned representations restore semantic breadth but pay for it in tokens, cache, and bus traffic (section 3.3). Pricing one fresh evaluation of the intent model on the application processor then exposes the memory wall and what quantization can and cannot do about it (section 3.4). Sizing the chunks that keep the arm supplied next separates the motion a chunk supplies from the time its evidence stays true (section 3.5). Because a proposal can arrive on schedule and still be wrong, the last step examines the failures that leave the permission path to refuse an on-time proposal (section 3.6).

3.2 The Classical Wall

In a fixed industrial workcell, a robotic gripper can approach the exact same fixture along an identical trajectory on every production cycle. Engineers can specify such repetitive motions through analytical geometry, rigid-body equations of motion, and deterministic state transitions. Classical control theory and traditional robotics flourish within this analytical formulation (Wiener 1948; Rudolph Emil Kalman 1960; Craig 2005). The engineering challenge sharpens, however, when the work moves from a rigid fixture to an unconstrained scenario, such as retrieving a randomly oriented mug from

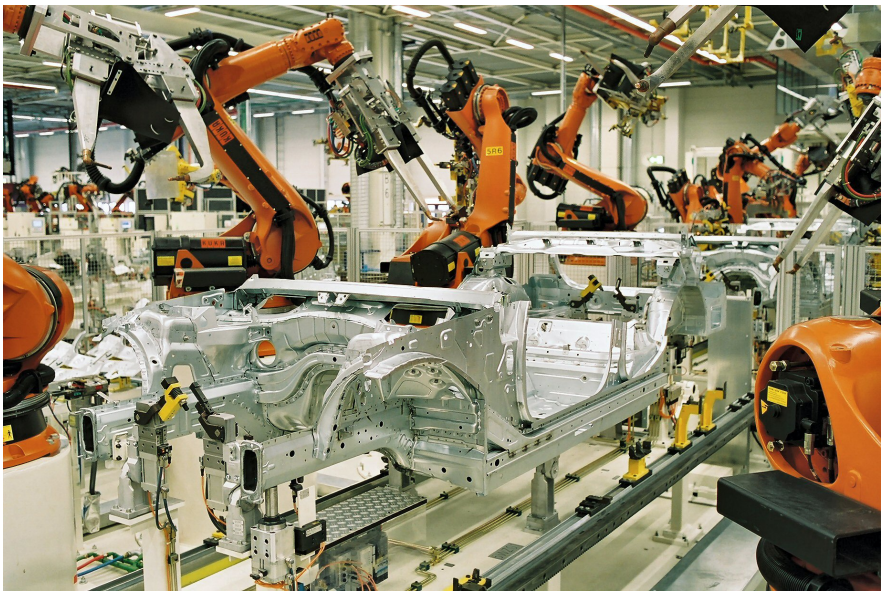


Figure 3.3: Industrial spot-welding workcell: Multiple six-degree-of-freedom articulated robots execute preprogrammed spot welding paths on stamped automotive chassis at the BMW Leipzig plant. The calibrated setting uses repeatable fixtures and safety interlock cages to constrain the conditions for those preprogrammed motions. (Credit: BMW Werk Leipzig, Wikimedia Commons, CC BY-SA 2.0 DE.)

a moving conveyor belt. There the engineer cannot enumerate every combination of object geometry, pose, and physical contact in advance.

Analytical kinematics and dynamics map target end-effector[1] trajectories into joint angles, velocities, and actuator torques.[2] Classical perception routines register incoming sensor point clouds against reference computer-aided design (CAD) models, while a synchronous finite-state machine steps the manipulator through sequential phases: approaching, aligning, grasping, and retracting. In a calibrated workcell, these analytical components reliably solve bounded manipulation tasks under explicit operational assumptions. Their computational latency and real-time execution bounds depend strictly on the chosen numerical solver, processor throughput, and contact dynamics model.

Where the operational environment remains structured and bounded, classical methods provide dependable behavior within their modeled operating envelopes. In a body shop (figure 3.3), the chassis sits in submillimeter tooling fixtures, and controlled lighting and interlock fencing constrain variation under stated fixture, access, and model assumptions. This controlled regime permits explicit physical modeling, servo timing analysis, and formal Lyapunov stability analysis.[3] Establishing selected trajectory tracking error bounds ($\|\mathbf{e}(t)\| \leq \epsilon_{\text{track}}$), stability properties, or software execution guarantees requires justified model, implementation, and execution evidence; the absence of training data does not establish those properties.

When an embodied machine steps outside the structured factory cage into unstructured, open-world environments, however, this analytical paradigm hits the **classical wall** (Brooks 1991; Levine et al. 2018). Physical reality defies closed-form geometric modeling: real objects deform under contact, ambient illumination shifts unpredictably, and scenes present near-infinite visual and mechanical variety. A rescue test arena of tilted ramps and a victim dummy on a high shelf (figure 3.4) has none of the fixtures that made the welding cell analyzable: no part arrives at a known pose, and no template describes the terrain. In such unmapped, irregular settings, preauthored geometric templates and rigid state machines lose state coverage and fail to generalize.

Outside structured fixtures, the analytical perception-and-control pipeline breaks down whenever scenes vary beyond its hand-coded rules. A rigid CAD template fails on deformable cloth or irregular food items; glare or cast shadows can wipe out the depth edges needed to separate adjacent parcels; and an unexpected obstacle invalidates the guard conditions of a preauthored grasping state machine.

1 | **End-Effector**: The tool, gripper, or hand at the distal tip of a manipulator, the only part of the arm meant to touch the workpiece. Workspace and keep-out checks are written against its pose rather than against joint angles, which is why the proposal payload later in this chapter carries end-effector poses alongside joint targets.

2 | **Manipulator Equation**: Classical control computes joint torques from $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \tau$, with a configuration-dependent inertia matrix $\mathbf{M}$, Coriolis and centripetal terms $\mathbf{C}$, and gravity $\mathbf{g}$. Torque is linear in inertia, and a link's inertia grows with the square of its distance from the joint, so an outstretched arm needs more torque for the same angular acceleration. Unmodeled contact, friction, or deformable objects break this rigid-body model, which is the gap learned proposers enter; section B.2.4 shows how classical control inverts this model to compute torque.

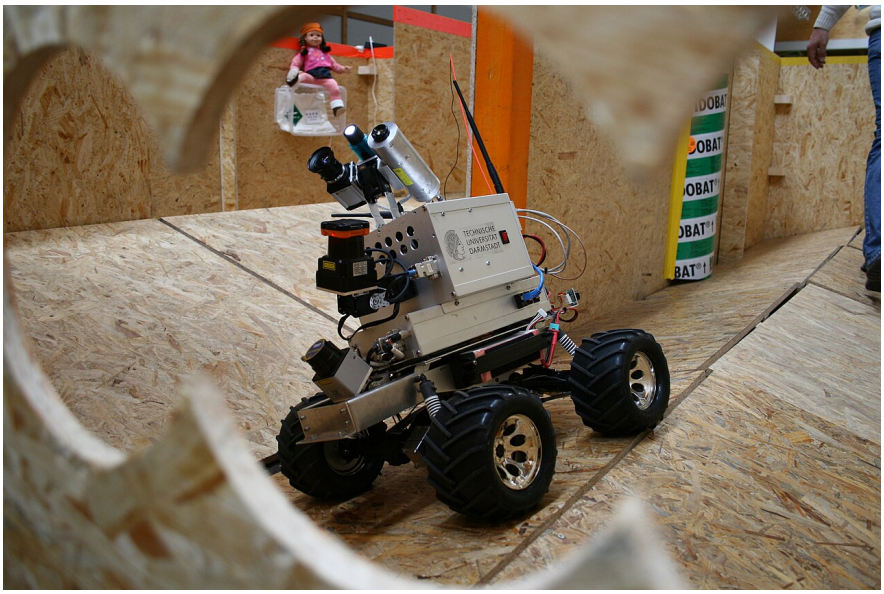


Figure 3.4: Unstructured search-and-rescue robot: A four-wheeled rescue robot carrying cameras and a laser scanner on a pan-tilt mast works through a test arena of tilted ramps and narrow openings, with a victim dummy placed on a high shelf. In such unmapped environments, preauthored geometric models and rigid state machines can lose coverage; learned policies may help infer spatial affordances and traversability from sensory streams. (Credit: Hector, Technische Universität Darmstadt, at the 2010 RoboCup German Open; photo by Mike1024, Wikimedia Commons, public domain.)

Engineers can add templates, heuristic thresholds, and recovery branches to patch each failure, but the cases multiply faster than they can be tuned and tested.

Machine learning exits this trap by learning from diverse physical interactions and demonstrations rather than enumerating geometric rules by hand. Neural models extract graspable features and motion policies directly from high-dimensional sensor streams without a bespoke CAD model for every object. Those learned models do not replace the downstream analytical controller: deterministic feedback loops still enforce kinematic bounds and track trajectories at high rates. Instead, learning inserts a compute-heavy proposal stage whose memory traffic, arithmetic cost, and latency must fit inside the timing contract of the real-time controller.

3.3 Learned Representations

When commanded to pick an unfamiliar ceramic mug from a conveyor, a learned model relates the text prompt to visual features of the handle and proposes a plausible grasp region without an exact CAD model. VLA models combine multi-camera imagery, natural-language instructions, and proprioceptive robot state to produce candidate actions (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024). These systems descend from early embodied language models such as SayCan and PaLM-E, which emitted high-level semantic plans rather than low-level motor commands (Ahn et al. 2022; Driess et al. 2023). The operational value of a VLA stems from broad generalization across diverse objects and ambient environments acquired during large-scale pretraining, rather than formal guarantees of correct scene interpretation.

Teleoperated demonstrations can likewise teach coordinated sensorimotor policies for tasks whose contacts and structural deformations resist analytic kinematic modeling. In a bimanual teleoperation workcell (figure 3.5), an operator moves low-cost leader arms while follower arms with parallel grippers reproduce the motion, and overhead and wrist cameras record what the followers saw. Those recordings become the training data for policies that later act alone (Zhao et al. 2023; Chi et al. 2025). The running machine's chunk policy takes its size from ACT, which was trained on exactly such recordings. What such a policy learns from them is the action chunk itself, a short stretch of the operator's motion predicted from one view. How demonstrations become a policy

[3] **Lyapunov Stability**: Aleksandr Lyapunov's 1892 method establishes stability near an equilibrium using a positive-definite, energy-like function whose derivative along the modeled dynamics is nonpositive; asymptotic convergence requires stronger decrease or invariance conditions (Slotine and Li 1991). Applying the method requires a justified closed-loop dynamics model and verified certificate conditions; a learned proposer does not acquire such a certificate merely from training. The barrier certificates of section 12.5 reuse the idea to keep the machine inside a safe set rather than at a point.

↩ The causal boundary, where an admitted command becomes physical work, is established in section 1.4.

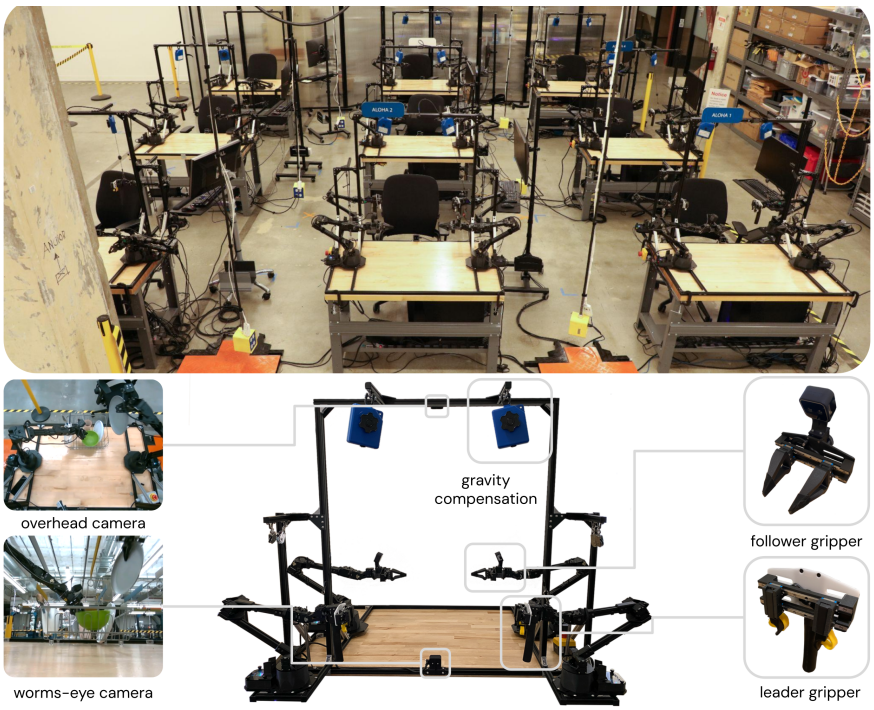


Figure 3.5: Bimanual teleoperation workcell: Dual-arm teleoperation setup featuring two six-degree-of-freedom follower arms, two leader arms for kinesthetic teleoperation, multi-view cameras (overhead and wrist-mounted), and parallel grippers mounted to an extruded aluminum frame. The physical AI policy ingests synchronized multi-camera streams to predict continuous end-effector trajectories, and the permission path must refuse any proposal that would cause self-collision or motor over-torquing during closed-loop execution. (Credit: ALOHA 2 Team, ALOHA 2: An Enhanced Low-Cost Hardware for Bimanual Teleoperation, arXiv:2405.02292, CC BY 4.0.)

belongs to chapter 6; this chapter prices what the trained policy costs to run. Although learned representations tolerate sensory variations captured across training distributions, executing any proposed motion safely still demands calibrated spatial geometry and strict physical boundary enforcement. Empirical task success demonstrates capability over tested operating envelopes, but out-of-distribution shifts in illumination, surface friction, or payload dynamics can quickly induce behavioral drift or outright failure.

The computational cost of this capability depends directly on the model's underlying representation. A transformer does not read pixels or words directly; it reads *tokens*, embedding vectors that each represent one slice of input, such as an image patch, a word fragment, or a joint angle. Across a VLA model, that token sequence flows from camera patches and prompt embeddings through self-attention to candidate action chunks (figure 3.6). Multi-view camera frames consume significant memory bandwidth during ingestion, historical context expands key-value (KV) cache footprints in memory, and the inference engine must produce subsequent actions before the physical state diverges from the initial observation. Evaluating an embodied learned model alongside a classical perception-planning pipeline requires assessing both task generalization and the end-to-end latency required to refresh action proposals.

To grasp the mug, the model turns camera images and a language instruction into candidate trajectories in three learned stages: tokenization, attention across the tokens in a backbone, and an action decoder that projects the backbone's output into proposed motion. A proposal boundary separates all three from actuation, so every candidate action passes independent permission checks before it reaches the motor drives. Evaluating this pipeline from a systems perspective means tracing what each stage stores in memory, transfers across the memory bus, and delivers to the controllers below it:

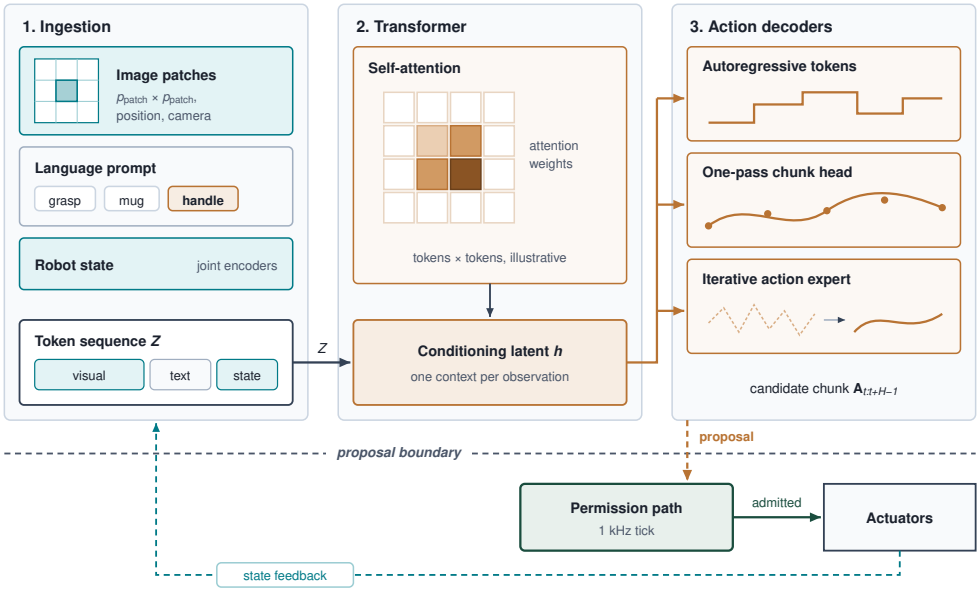


Figure 3.6: Embodied Vision-Language-Action (VLA) architecture: A representative dataflow. (1) Camera patches receive 2D image-position and camera identifiers alongside language and robot-state tokens; metric depth still requires calibrated evidence. (2) Self-attention mixes the combined token sequence; the colored grid illustrates internal attention weights, not a verified affordance or grasp pose. (3) Autoregressive token decoders emit tokens sequentially, whereas one-pass chunk heads and iterative action experts generate candidate multi-step chunks ($\mathbf{A}_{t:t+H-1}$) in one pass or over several refinement steps. (4) An independent permission path checks candidate motion against sensed state and modeled limits before actuation. (Synthesized from Brohan et al. (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), Kim et al. (Kim et al. 2024), and Octo Model Team (Octo Model Team et al. 2024)).

1. *Sensory ingestion and spatial tokenization (the inputs).* A vision transformer (ViT) frontend (Dosovitskiy et al. 2021) slices each incoming camera frame into a grid of non-overlapping pixel patches of size $p_{\text{patch}} \times p_{\text{patch}}$ (typically 14×14 or 16×16 pixels). Each patch is linearly projected into a vector of width d, augmented with a position embedding (learned in the SigLIP and DINOv2 encoders that OpenVLA uses (Kim et al. 2024), sinusoidal in some other designs), and tagged with a learned camera identifier.
   In an illustrative three-camera configuration (left wrist, right wrist, and an overhead torso mount) capturing 224×224 images, the image frontend prepares a buffer for each view. Each frame yields $(224/14)^2 = 256$ spatial patches. Across 3 cameras, ingestion injects 768 visual tokens into the model's sequence on every perceptual step. When concatenated with tokenized natural language commands (about 32 tokens) and proprioceptive joint state embeddings (about 16 tokens), the initial context runs slightly past 800 tokens, which the estimates below round to $S \approx 800$. In 16-bit floating-point (FP16) precision (2 bytes per element) with hidden dimension $d = 4096$, the token embeddings for one such step occupy
   $$M_{\text{embed}} = S \times d \times 2\,\text{bytes} = 800 \times 4096 \times 2 \approx 6.55\,\text{MB}$$
   a few megabytes, negligible beside the multi-gigabyte weights that section 3.4.1 prices. Tokens therefore cost the machine through the arithmetic every weight performs on each of them and through the cache they leave behind, not through the storage of the embeddings themselves. Neither 2D image coordinates nor internal attention patterns constitute metric depth. Projecting an end-effector into physical 3D space requires calibrated extrinsic camera matrices and depth telemetry.
2. *Multimodal alignment and attention (the backbone).* For the illustrative backbone, the token sequence passes through $N_L = 32$ transformer layers where standard multi-head self-attention computes pairwise token interactions (Vaswani et al. 2017). To relate each new token to the earlier ones, every layer keeps a *key* and a *value* vector for every token it has already

processed, the **KV cache**. The arithmetic of attention grows with the square of the sequence length ($\mathcal{O}(S^2 d)$ FLOPs), but the memory the cache occupies grows only linearly with it, as equation 3.1 gives:

$$M_{\text{KV}} = 2 \times N_L \times d \times S \times s_{\text{elem}} \tag{3.1}$$

The estimate assumes full-width keys and values, meaning every attention head in every layer stores its own. For a 32-layer model with $d = 4096$ using FP16 storage, each context token then claims 524,288 bytes, about 0.524 MB, of KV cache, so an observation step of several hundred tokens holds hundreds of megabytes.

Whether the machine pays that charge more than once depends on what the model keeps between evaluations. Single-observation VLAs such as OpenVLA rebuild their context from each new frame and carry no cache across control steps, and the running machine's intent model is one of them. Only a design that conditions on retained history rereads the cache on every step, and section 3.5.2 prices what retaining several seconds of history would cost. Such rereading is not free, because the processor uses a unified memory architecture (UMA), in which the CPU, the GPU, and the camera interfaces share one DRAM. Every byte of cache read is bus time taken from weight streaming and from the direct memory access (DMA) transfers that deliver sensor frames. Grouped-query attention (GQA) narrows the cache by letting several query heads share one key and value; cross-attention can instead limit which retained representations the decoder reads. Either saving depends on the architecture, and action chunking does not provide it.

Attention weights are also not geometry. A strong weight between the language tokens "mug handle" and a visual patch does not establish a graspable affordance[4] or collision-free clearance, and the KV cache does not keep track of an object while it is occluded. The formal patch-embedding, position-encoding, and attention formulations appear in section A.3.2.

3. *Specialized action decoders (the outputs).* Instead of generating open-ended conversational text, the final hidden representation $\mathbf{h}_{\text{context}} \in \mathbb{R}^d$ is routed to specialized action decoders. The running machine's chunk policy proposes base velocity and arm targets together in one whole-body stream (section 3.5.2), so its decoder is a manipulation decoder whose outputs include the base. Separate navigation models that decode waypoints over a topological map of the building (figure 3.7) follow the same proposal rules but are not part of this machine. Manipulation (VLA) models diverge sharply in how they bridge semantic representations to physical motion: Discrete action decoders map each action dimension to a categorical token by dividing its continuous range into bins (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024). Autoregressive decoders such as RT-2 and OpenVLA then generate those tokens sequentially, requiring one complete model evaluation per action dimension (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024). However, discretization does not require sequential decoding. RT-1 predicts all action tokens simultaneously in a single forward pass, and its authors demonstrated that autoregressively conditioning each action dimension on preceding dimensions yielded no accuracy gains while more than doubling inference latency (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023).

   What separates modern decoders in systems practice is the computational cost of generating a trajectory chunk and the volume of model weights reread during inference. Three architectural families dominate implementations as of 2026:

   - *Autoregressive tokens.* Each token requires a separate sequential evaluation. With one token per action dimension per control step, predicting a chunk of H timesteps across d_a dimensions demands Hd_a sequential evaluations, causing inference latency to scale with both horizon and degree of freedom. Per-step binning also breaks down at high control frequencies: consecutive continuous setpoints differ negligibly, producing runs of identical token targets from which next-token training learns almost nothing. Frequency-space action sequence tokenization (FAST) resolves this scaling bottleneck by applying a discrete cosine transform across each action dimension over the chunk horizon, representing the trajectory as a linear combination of cosine basis functions (Pertsch et al. 2025). Smooth physical motions concentrate energy in low-frequency coefficients, allowing aggressive

[4] **Affordance**: Gibson's term for the action possibilities an environment offers an agent, defined jointly by object properties and agent physical capabilities (Gibson 1979). A handle affords a grasp only when its physical dimensions and clearance accommodate the robot's gripper geometry and kinematics. While self-attention weights across image patches highlight semantically relevant features such as a handle, they do not certify mechanical grasp stability or kinematic reachability, the failure mode section 3.6.1 analyzes.

truncation. FAST quantizes these coefficients and compresses them with byte-pair encoding, which repeatedly merges the most frequent adjacent pair of symbols into a new token, tying token counts directly to trajectory information complexity rather than Hd_a. On a 50 Hz bimanual task, FAST compressed 700 raw binned tokens to 53, achieving a 13.2× reduction (Pertsch et al. 2025). Token generation nevertheless remains sequential. The authors report an inference latency of approximately 750 ms per chunk for FAST-π_0, compared to 100 ms for the flow-matching variant on an identical desktop GPU.

- *One-pass chunk heads.* Action Chunking with Transformers (ACT) regresses an entire trajectory chunk in a single feedforward decoder pass (Zhao et al. 2023). OpenVLA-OFT adapts an autoregressive VLA into a one-pass generator by passing placeholder action tokens through bidirectional self-attention and directly regressing continuous values for all action steps in parallel, increasing throughput by 26× on 8-step chunks (Kim et al. 2025). However, deterministic regression heads trained with mean squared error struggle with multimodal demonstration data: when presented with two equally valid paths around an obstacle, a naive regression loss averages them into an infeasible trajectory directly through the obstacle (Chi et al. 2025). Section 6.2.3 develops this *mode averaging* from the training side. ACT mitigates it by conditioning its decoder on a latent style variable, a code that records which of several demonstrated ways of doing the task a trajectory follows, learned with a conditional variational autoencoder (CVAE, section A.5.1).
- *Iterative action experts.* Diffusion Policy formulates action generation as iterative denoising, starting a candidate chunk from random noise and repeatedly removing the noise a network predicts, adapting continuous diffusion processes originally developed for generative vision models (Ho et al. 2020; Chi et al. 2025). The π_0 architecture pairs a 300M-parameter flow-matching[5] **action expert** with a pretrained vision-language model, yielding 3.3B total parameters (Black et al. 2024). The large vision-language backbone processes perceptual observations once and caches its key-value activations; the action expert then generates the continuous trajectory chunk across 10 numerical integration steps while attending to that static cache. Each integration step is a forward Euler step. The expert starts the candidate from Gaussian noise, evaluates a learned velocity field that says which direction moves it toward a plausible action chunk, and moves the candidate a small distance that way; after 10 steps the candidate is the chunk. Each step requires a forward pass only through the lightweight action expert and its cross-attention reads into the cached backbone KV activations, avoiding any re-evaluation of the multi-billion-parameter vision-language backbone. Subsequent designs such as $\pi_{0.5}$ and GR00T N1 employ this exact decoupling (Black, Brown, et al. 2025; Bjorck et al. 2025), with $\pi_{0.5}$ combining FAST-token pretraining on the backbone with flow-expert execution at runtime.

[5] **Flow Matching**: A method that trains a neural network to predict a continuous velocity field carrying samples from Gaussian noise to the target data distribution along nearly straight paths (Lipman et al. 2022). Because those paths are straight, numerical solvers can generate samples in a handful of Euler steps, whereas older denoising diffusion models needed hundreds of sequential steps (Ho et al. 2020). On an embedded robot processor, the step count directly sets latency because each solver step reread the action expert's weights and cached visual context from off-chip memory.

Across all three families, the continuous output forms a multi-step trajectory chunk:

$$\mathbf{A}_{t:t+H-1} = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}] \in \mathbb{R}^{H \times d_a}$$

Here H specifies the future prediction horizon in discrete setpoints and d_a denotes the degrees of freedom per action; section 3.5.1 states the formal definition (3.2) once the memory wall has shown why chunks are needed. Published horizons span from tens to hundreds of setpoints: Diffusion Policy predicts 16 steps and executes the initial 8 (Chi et al. 2025), π_0 predicts 50 steps (Black et al. 2024), and ACT predicts 100 steps at 50 Hz (Zhao et al. 2023). The structural gap between prediction and execution horizons resurfaces in section 3.5.2 as the interface between asynchronous trajectory generation and real-time permission filtering. A trajectory chunk provides an open-loop sequence of future setpoints derived from a single sensory context, amortizing inference latency without incorporating intervening sensor observations. Section 3.5.1 evaluates the computational efficiency of these three decoder families on target silicon, while section A.5.1 gives the discrete tokenization rule and details how generative decoders capture multimodal demonstration distributions without averaging conflicting trajectories.

4. *The proposal boundary (isolation from actuators).* Whether the model plans navigation or manipulation, its output is an unprivileged candidate proposal strictly outside the proposal boundary

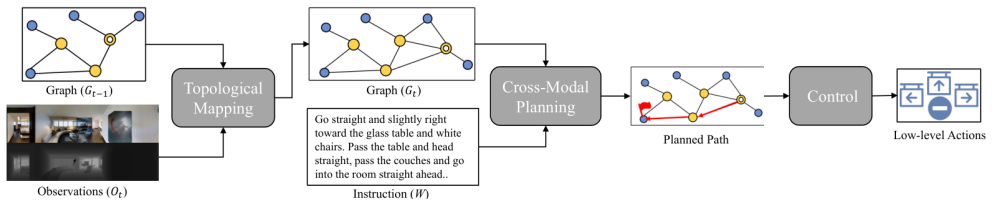


Figure 3.7: Topological vision-language navigation: Embodied navigation architecture executing natural-language spatial instructions. Panoramic visual observations are encoded into perceptual tokens, matched against prompt embeddings, and cross-referenced with a dynamically updated topological navigation graph. The action decoder prunes unreachable nodes and emits metric waypoint offsets $(\Delta x, \Delta y)$ and heading setpoints to guide the chassis along traversable semantic frontiers without requiring an exact geometric prior map. (Adapted from An et al. (2023), arXiv:2304.03047, CC BY 4.0.)

(definition 1.3). Under the concrete machine model established in section 1.7, the foundation model runs in unprivileged user space on the host application processor and has no direct memory-mapped access to pulse-width modulation (PWM) registers or motor coil current drivers. Instead, the Brain hands candidate motion, stamped with the capture times of the sensory frames that generated it, to a permission path whose kilohertz loops a slow inference cannot preempt or bypass. That path refuses proposals that would collide, violate joint limits, or drive the arm into a kinematic singularity (an arm configuration in which the joints lose the ability to move the end effector in some direction). Section 4.6 isolates this gating logic on a dedicated real-time safety microcontroller, completely segregated from the application processor hosting the learned model.

The choice of sensory representation and action decoder dictates the memory traffic and latency profile of every control cycle. Maintaining historical token context accumulates KV cache overhead that consumes memory bus bandwidth, while running multi-billion-parameter backbones on resource-constrained embedded systems demands heavy weight streaming from off-chip DRAM. An embodied agent needs both semantic breadth and rapid trajectory renewal, and these costs suggest that one monolithic model on an embedded processor cannot deliver both. Whether it can is a question of numbers, and the place to get them is one fresh evaluation of the machine's intent model on its own application processor.

3.4 The Cost of a Fresh Evaluation

At the pick station the mobile manipulator's arm consumes a new setpoint every few tens of milliseconds, while the mug keeps moving along the conveyor. Whether one large model could supply those setpoints depends on how long one fresh evaluation of it takes on the application processor, and that time depends on which resource the evaluation exhausts first, the memory bus or the arithmetic units. The intent model's evaluation is priced first, together with what quantization can buy back; section 3.5 then checks the motion buffer that the selected split relies on against the target's drift. The single large-model cases below are alternatives used to expose the trade-off, not changes to the selected machine.

After ingestion (the tokenization of section 3.3), four learned stages turn those tokens into action proposals, and each hands the next a record that ages (figure 3.8). Across the proposal boundary, an isolated safety microcontroller (MCU) running the permission path on a 1 kHz tick decides whether candidate setpoints become admitted drive commands. These stages describe what the machine must compute, not how its networks are divided; the three learned stages of section 3.3 describe the inside of one transformer. On the running machine, the intent model recognizes the mug and names the target (perception and intent), a tracker carries the mug's estimate from frame to frame (memory), and the chunk policy, reading its own camera frames, turns the target into setpoints (planning). *Memory* here means the belief the machine keeps about the world, not DRAM. The chapters of Part III, beginning with chapter 8, develop each stage in depth; here they matter only for what each hands downstream and how quickly that record goes stale.

Perception grounds what it recognizes in metric coordinates relative to the base, so that the mug has a 3D position and orientation (an $SE(3)$ pose)[6] and a graspable handle rather than an abstract semantic label; its output ages from the moment of capture. Memory carries those estimates through

6 **SE(3) Pose**: SE(3), the special Euclidean group in three dimensions, is the set of rigid-body poses, a rotation plus a translation with six degrees of freedom; "special" excludes mirror reflections, which no rigid object can undergo. A pose can be compared with measured depth and aged from its capture time, while a semantic label such as "mug handle" can be neither, which is why the handoff to planning is a pose and not a label.

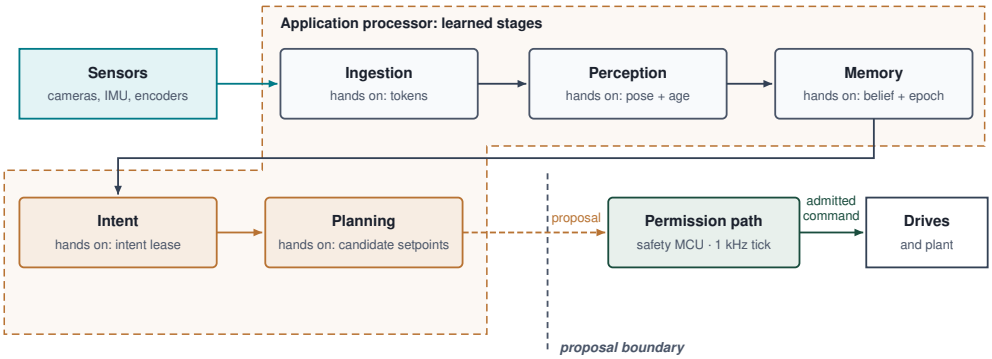


Figure 3.8: The physical AI cognitive flow: The learned stages run on the application processor and each hands the next a record that ages: ingestion produces tokens, perception a metric pose with its age, memory a belief with its evidence epoch, intent an intent lease, and planning candidate setpoints. Everything up to planning is proposal. Only the permission path, on its own 1 kHz tick across the proposal boundary, can turn a proposal into an admitted command for the drives.

occlusion and must let each one expire when the evidence behind it does. Intent decomposes a task such as "take the ceramic mug off the conveyor and hand it to the coworker at the packing station" into subgoals, and because the mug moves while the intent model deliberates, it hands the planner a target that expires, an intent lease in the vocabulary of Part III. Planning converts that target into candidate setpoints for the base and the arm, the proposals the permission path will judge.

↪ The observation contract that dates each perception output from its capture epoch is developed in section 8.8.

↪ The intent lease and the deadlines that end it are developed in section 10.8.

These stages, and the ingestion before them, all run on the application processor and draw on the same unified memory bus, which is where the machine's cadence is decided. On an edge robotics system-on-chip (SoC), memory bandwidth, arithmetic rate, and power envelope are fixed allocations, and model weights, sensor transfers, and host computation share that bandwidth. A policy cannot budget the full bus for itself while assuming that new camera frames arrive without interference, and scaling the model up only draws harder against the same ceiling. The transfer bound below is therefore a lower bound on evaluation time. The worked comparison later uses an assumed sustained fraction of peak bandwidth. Section 13.3 models how weight fetches, camera transfers, and telemetry writes queue for the shared memory controller.

↩ That every added layer of onboard compute draws against fixed memory bandwidth, electrical power, and thermal dissipation is established in section 1.10.

Published measurements on the machine's own module show how far apart those costs sit before any of that arithmetic. The MLPerf Inference edge suite times one query at a time, its single-stream scenario, for a fixed set of reference models (Reddi et al. 2019). Vendor submissions for the 64 GB Jetson AGX Orin place those reference latencies directly against the three cadences of the running machine (figure 3.9).

Only three of the seven reference models answer inside the 50 ms chunk period: classification in 0.64 ms, question answering, and detection in 11.7 ms. A six-billion-parameter language model takes 4.2 s to produce one summary, 8× the 500 ms intent period, and the span from fastest to slowest is about 16,000×. The figure is a measured version of the two-speed argument. A language-model proposer on this module can revise intent only at a fraction of the rate the arm consumes setpoints. The same submission also shows that the cost is not fixed by the silicon alone. The language model's latency was 10.2 s one round earlier on identical hardware, so the software stack sets the cost as much as the silicon does. Which bytes that software moves per evaluation is what the memory-wall analysis below quantifies.

3.4.1 The memory wall

A multi-billion-parameter model takes hundreds of milliseconds on a processor rated at tens of tera-operations per second. Its weights do not fit in on-chip static RAM (SRAM) or cache, so every fresh evaluation must stream them from off-chip low-power DRAM (LPDDR5) across a narrower shared bus. On-chip SRAM and caches can reuse weights and intermediate state without another external transfer only while the working set fits the memory allocated to that workload. Nominal chip-wide capacity does not guarantee that even a small action head stays resident when competing

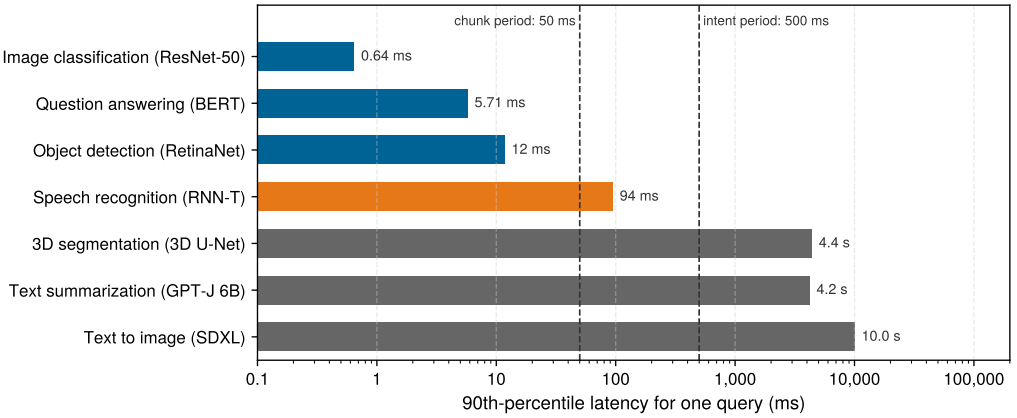


Figure 3.9: Measured single-query latency on the application processor: The 90th-percentile time to answer one query, on a logarithmic axis, for each MLPerf Inference edge reference model on the 64 GB Jetson AGX Orin, the module in the running machine. Dashed lines mark the machine's chunk renewal period and intent revision period; blue bars finish within one chunk period, the orange bar within the intent period, and gray bars within neither. The reference models span four orders of magnitude on the same silicon, so which proposer a design can afford is fixed by the model class before any tuning. Vendor submissions, latest round per model (v3.1, or v4.1 for the two generative models); data from the MLCommons inference results repositories (Reddi et al. 2019).

tasks share the processor. When operands cannot reach the arithmetic units as fast as those units can multiply them, memory bandwidth, not arithmetic speed, limits the update rate. That ceiling is the memory wall, and on a moving robot its cost is a delayed proposal while the body keeps traveling.

Consider a robot that generates continuous motor actions one discrete step at a time through an autoregressive foundation model, such as a multi-billion-parameter VLA. Its weights are *nonresident*, too large to stay in on-chip memory, so each token step streams them across the external DRAM bus. In single-token decoding with FP16 weights and request batch size $B = 1$, the dense weight projections become matrix-vector operations ($\mathbf{y} = \mathbf{W}\mathbf{x}$), in which each fetched weight takes part in a single multiply-add before it is discarded. The bus, not the arithmetic, then sets the pace. The sustained memory bandwidth $\mathrm{BW}_{\mathrm{mem}}$ and model weight footprint M_{weights} bound the maximum fresh-evaluation frequency, as equation 3.2 states:

$$f \leq \frac{\mathrm{BW}_{\mathrm{mem}}}{M_{\mathrm{weights}}} \tag{3.2}$$

This bound must use the bandwidth actually available to the policy and the weights it transfers at the selected precision. It prices only the weight transfer. A fresh evaluation first reads its input tokens (image patches and the instruction) through the backbone in one pass, the **prefill**. That pass reuses every fetched weight across all of those tokens, so arithmetic rather than bandwidth can bind it. Notebook 3.1 prices both terms for the mobile manipulator's intent model. When a decoder emits one token per action dimension, each action dimension requires another token; if each token revisits off-chip parameters, that decode latency scales linearly with degrees of freedom.

Single-token decoding at batch size 1 Servers can combine independent requests so that a weight matrix multiplies several input vectors at once, reusing fetched weights. A robot waiting for its next action token often has only one such request. Waiting for future observations to form a batch would add delay, and sequentially dependent action tokens cannot all be evaluated at once by an autoregressive decoder. This limits reuse in those projections. It does not make every operation in an embodied model a matrix-vector product. Image patches, multiple camera views, prompt tokens, and parallel action decoders can expose matrix-matrix work within one robot's request. The one-use-per-fetch estimate therefore describes the single-token weight sweep, not the complete perception-and-action workload.

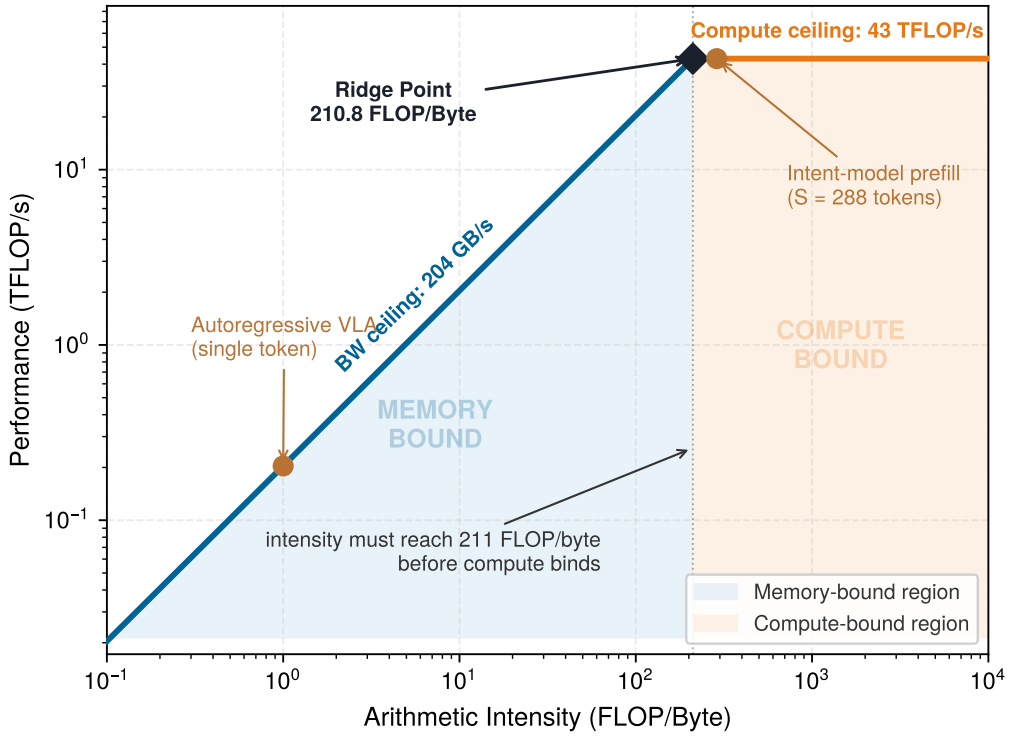


Figure 3.10: Illustrative embodied roofline: Arithmetic intensity (operations per byte moved from off-chip memory, equation 3.3) against attainable throughput using the application processor's dense FP16 GPU ceiling and rounded peak memory bandwidth. Single-token weight streaming sits deep in the memory-bound region, while the intent model's prefill, which reuses each fetched weight across all of its input tokens, lies past the ridge, the intensity at which the bandwidth roof meets the compute roof, where compute binds. Chunk length alone does not determine arithmetic intensity; chunked workloads require measured operations and memory traffic. A separate real-time loop executes accepted waypoints between fresh model updates.

Whether a phase of the evaluation meets the memory wall or the arithmetic limit is decided by how much work it does per byte it moves, its **arithmetic intensity** (I), defined in equation 3.3 as:

$$I = \frac{\text{FLOPs}}{\text{Bytes transferred from off-chip memory}} \tag{3.3}$$

For the dense weight projections of single-token decoding at batch size 1, each a matrix-vector product $\mathbf{y} = \mathbf{W}\mathbf{x}$, counting weight traffic alone gives an optimistic estimate, because activation and context traffic add further bytes. In FP16 each weight occupies 2 bytes and participates in exactly one multiply-accumulate operation (2 FLOPs), so

$$I_{\text{GEMV, 16-bit}} = \frac{2\,\text{FLOPs}}{2\,\text{Bytes}} = 1.0\,\text{FLOP/byte}$$

The roofline model (Williams et al. 2009) turns that ratio into a bound (figure 3.10). It plots attainable throughput against arithmetic intensity as two roofs. Below a crossover, the ridge point, throughput is capped by a sloped roof equal to bandwidth times the intensity; above it, by a flat roof equal to the processor's peak arithmetic rate. It is read the same way as the stopping-budget plot of section 1.4.3, where delay travel and braking travel trade dominance at a crossover speed. Both plots show two regimes and the point at which one gives way to the other.

The mobile manipulator's application processor is a Jetson AGX Orin module (section C.14.1). In its 60 W maximum-performance power mode, its dense FP16 GPU Tensor Core ceiling is 43

TFLOP/s; the aggregate sparse INT8 rating cannot be used for this FP16 workload. Using the registry's rounded peak memory bandwidth of 204 GB/s, the hardware ridge point occurs at:

$$I_{\text{ridge}} = \frac{R_{\text{peak}}}{\text{BW}} = \frac{43TFLOP/s}{204GB/s} \approx 210.8FLOP/byte$$

For this weight-streaming example, $I_{\text{GEMV, 16-bit}} = 1.0\,\text{FLOP/byte}$ is far below 210.8 FLOP/byte, so memory bandwidth sets the roofline ceiling. Attainable throughput is bounded by the peak bandwidth times $I_{\text{GEMV, 16-bit}}$, or 0.204 TFLOP/s, about 0.47 percent of the dense FP16 peak; the sustained bandwidth that the notebook below uses lowers it further. That ratio is a workload throughput bound, not a measurement of how many cores are idle. Both roofs also assume the maximum-performance mode. A battery-powered base often runs a lower power mode, down to 15 W on this module, where lower clocks drop both the compute and the bandwidth roof; the chassis thermal budget sets the mode the machine can sustain.

Definition 3.1: Memory wall

Memory wall is the limit on a workload's rate that arises when its operands cannot reach the arithmetic units as fast as those units could consume them, so that the bytes moved from off-chip memory, not the operations performed, set the time per evaluation.

1. **Significance**: A proposer whose weights do not stay on chip must stream them at least once per fresh evaluation, which gives equation 3.2. On the machine's application processor, the intent model's 14 GB of FP16 weights at 142.8 GB/s sustained allow at most 10.2 Hz before any vision or decoding work, two orders of magnitude below the 1 kHz permission tick.
2. **Distinction**: A compute bound yields to faster arithmetic; the memory wall yields only to fewer bytes per evaluation (fewer or narrower weights, or weights that stay resident on chip) or to more bandwidth. It is also distinct from memory capacity: a model can fit in DRAM and still be too slow to stream.
3. **Common pitfall**: A frequent misconception is that a processor's peak arithmetic rating predicts its batch-1 inference rate. Single-token weight streaming on this processor reaches only about 0.47 percent of its dense FP16 peak, so the rating overstates the attainable rate by orders of magnitude.

On the mobile manipulator's application processor, then, the memory wall[7] caps the intent model's fresh-evaluation rate, before any vision or decoding work, far below the 100–1000 Hz at which stabilization, joint impedance control, and reflex-level collision avoidance run (notebook 3.1). Each of those loop rates is set by the physical time constant of the motion its loop must stabilize, a derivation carried out in section 4.3. A model renewed that slowly can say where the arm should go, but it cannot close the loop on moving mass, so it proposes waypoints and a faster loop below it tracks them. That conclusion is about timing only. The separate requirement that learned outputs remain proposals, even when they arrive in time, comes from the failures of section 3.6.

Repeated off-chip transfers also consume memory-system energy. Their power cost depends on the DRAM, controller, access pattern, and transfer rate, so a bandwidth calculation alone does not establish battery drain or actuator heating. Whatever the processor dissipates is heat released inside the same sealed chassis as the joint motors, and it raises the ambient temperature against which section 2.5.1 rates each motor's continuous torque.

A fresh evaluation is not a single point on the roofline. Its prefill reuses each fetched weight across all of its input tokens and lands past the ridge, while its decode returns to the memory-bound point, one full weight sweep per token. For a decoder that emits S_{out} tokens one after another, equation 3.4 adds the two phases:

$$t_{\text{infer}} \approx \max\left(\frac{O_{\text{prefill}}}{\eta_{\text{hw}} R_{\text{peak}}}, \frac{M_{\text{weights}}}{\text{BW}_{\text{mem}}}\right) + (S_{\text{out}} - 1)\,\frac{M_{\text{weights}}}{\text{BW}_{\text{mem}}} \tag{3.4}$$

[7] **Wulf and McKee**: Their short paper named the memory wall (Wulf and McKee 1995), observing that processor speed was improving far faster than DRAM latency, so that memory access would eventually dominate run time whatever the processor did. Caches postponed the wall for workloads that reuse their data, but a batch-1 model on a robot meets it directly. When each weight is used once per fetch, the bus, not the arithmetic, sets how often a fresh proposal can arrive.

Here O_{prefill} counts the prefill's operations, R_{peak} is the processor's peak arithmetic rate at the activation precision, and η_{hw} is the fraction of that rate the prefill achieves. The prefill cannot finish before its own weight sweep, hence the maximum; the prefill emits the first token, and each of the other $S_{\text{out}} - 1$ tokens costs one more sweep. Notebook 3.1 prices both phases for the machine's intent model.

Prefill creates KV state that decode extends within one evaluation. Figure 3.11 follows a single observation through prefill, first-token sampling, and subsequent decode passes, then marks the boundary at which the running machine rebuilds its context.

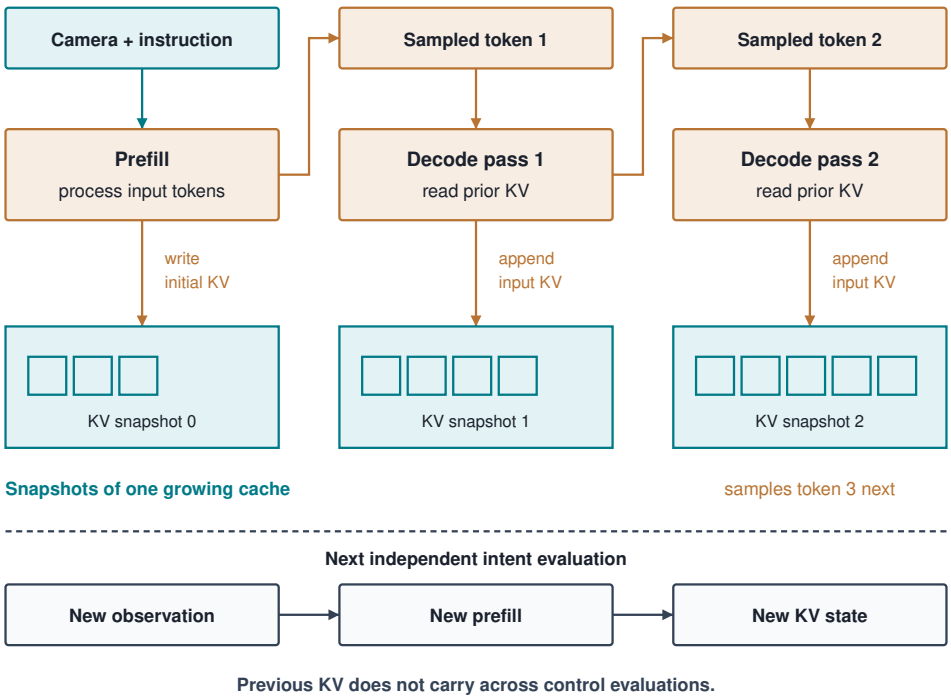


Figure 3.11: KV State Within One Intent Evaluation: Prefill processes camera and instruction tokens and supplies the output from which the first token is sampled. Each subsequent decode pass takes the previously sampled token, reads prior keys and values, and appends the input token's keys and values before sampling the next output. The cache panels are successive snapshots of one growing cache, with schematic token counts. The running machine's single-observation intent model begins the next independent evaluation with new context rather than retaining this cache across control evaluations.

Cache reuse avoids recomputing the prefix projections during that evaluation. It neither refreshes the captured observation nor turns token state into a spatial memory of the room. A model that deliberately retains observation history has a different memory and bandwidth obligation.

Napkin Math 3.1: Pricing one intent-model evaluation

Problem: The mobile manipulator's intent model is a 7B-parameter vision-language model sized after OpenVLA (Kim et al. 2024), running on the application processor. Where OpenVLA decodes a step of end-effector motion, the machine's model decodes the same number of tokens to name a target pose (section 3.5.3). How long does one fresh evaluation take, which resource binds each part of it, and what renewal rate can the machine therefore state?

Variables:

- Parameters $P = 7$B in $N_L = 32$ layers of width $d = 4096$ (Datasheet). The decoder emits $S_{\text{out}} = 7$ tokens, one per output dimension, one after another (Datasheet).
- Input tokens $S = 288$: 256 patches from one 224×224 view (Derived) plus 32 instruction tokens (Illustrative).
- Dense FP16 peak $R_{\text{peak}} = 43$ TFLOP/s (Datasheet), of which the prefill achieves $\eta_{\text{hw}} = 50$ percent (Illustrative).
- Peak bandwidth 204 GB/s (Datasheet), of which 70 percent is sustained (Illustrative), so $\text{BW}_{\text{mem}} = 142.8$ GB/s.
- Stored weights: FP16 (2 bytes per parameter) or INT4 (half a byte), with activations kept in 16 bits (Chosen).

Math:

The memory wall. FP16 weights occupy 14 GB. If every evaluation streams them once, the bound above gives

$$t_{\text{sweep}} = \frac{M_{\text{weights}}}{\text{BW}_{\text{mem}}} = \frac{14GB}{142.8GB/s} \approx 98.0ms, \qquad f \leq \frac{1}{t_{\text{sweep}}} \approx 10.2Hz$$

The prefill binds on arithmetic. All S input tokens pass through the backbone together, costing two FLOPs per parameter per token (each weight multiplies one input value and adds the product to a running sum) plus a small attention term:

$$O_{\text{prefill}} \approx 2NS + 2N_L dS^2 = 4.03 \times 10^{12} + 2.17 \times 10^{10} \approx 4.05 \times 10^{12} \text{ FLOP}$$

Each fetched weight now serves S tokens, so the intensity is about $S \approx 288$ FLOP/byte, beyond the 210.8 FLOP/byte ridge of figure 3.10. Arithmetic binds, and the weight sweep runs underneath it:

$$t_{\text{prefill}} = \frac{O_{\text{prefill}}}{\eta_{\text{hw}} R_{\text{peak}}} = \frac{4.05 \times 10^{12} \text{ FLOP}}{0.5 \times 43TFLOP/s} \approx 188.5ms$$

The decode binds on bandwidth. The prefill emits the first action token. Each of the remaining $S_{\text{out}} - 1 = 6$ tokens is a matrix-vector pass that rereads every weight:

$$t_{\text{FP16}} = t_{\text{prefill}} + (S_{\text{out}} - 1)\, t_{\text{sweep}} \approx 189ms + 6 \times 98.0ms \approx 189ms + 588ms = 777ms$$

That is about 1.3 Hz.

INT4 weights shrink only the bandwidth term. Storing the weights in four bits cuts them to 3.5 GB and each sweep to 24.5 ms, but the prefill still multiplies 16-bit activations at the FP16 rate:

$$t_{\text{INT4}} \approx 189ms + 6 \times 24.5ms \approx 189ms + 147ms = 336ms, \qquad f \leq 2.98Hz$$

Sensitivity. Parallel decoding (Kim et al. 2025) moves the action tokens into the prefill ($S + S_{\text{out}} = 295$) and removes the decode sweeps, giving 194 ms, only 2.9 percent inside a 200 ms period with the processor busy for all of it. Three camera views ($S \approx 800$) raise the INT4 total to 676 ms.

Result: At INT4 one evaluation takes about 336 ms. A published measurement of the same model and quantization on this processor reports 336 ms per frame (Franklin 2024), a gap of 0.1 percent. That agreement is closer than two illustrative factors deserve. It shows that the model has the right terms, not that η_{hw} and the sustained fraction are exactly right. A 3 Hz rate would need a 333 ms period, so the machine runs its intent model at 2 Hz, one evaluation per 500 ms.

Systems insight: The binding resource depends on the phase. Prefill reuses weights across tokens and is compute-bound; decode rereads them per token and is bandwidth-bound (Pope et al. 2023). Weight-only quantization attacks only the second, so after INT4 the compute-bound prefill is most of the latency, and the next gain must come from fewer input tokens, lower-precision arithmetic that the processor's kernels support, or a faster compute roof. Section 3.4.2 develops that choice and its accuracy cost.

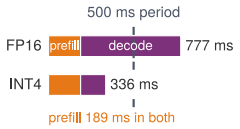
3.4.2 Quantizing the intent model

The memory wall leaves two levers on the weight term, fewer weights or fewer bytes per weight. A smaller model may lose task recognition capability, which requires task evidence, so the selected design keeps the 7B-parameter model and pulls the second lever. **Quantization** stores each weight as a low-precision integer plus a shared scale, and reconstructs an approximate weight when the kernel needs it. In the common symmetric form, the weights are divided into groups of b_{block}, each group keeps one 16-bit scale s, and each weight w_i becomes a b-bit integer q_i (Jacob et al. 2018; Lin et al. 2024):

$$s = \frac{\max_{i \in \text{group}} |w_i|}{2^{b-1} - 1}, \qquad q_i = \text{clamp}\left(\text{round}\left(\frac{w_i}{s}\right), -2^{b-1}, 2^{b-1} - 1\right), \qquad \hat{w}_i = s\, q_i$$

Rounding moves each reconstructed weight $\hat{w}_i$ at most $s/2$ from w_i, so one large weight in a group coarsens the step for all the others; choosing which weights to protect from that error is where quantization methods differ. The stored size is $b/8$ bytes per weight plus the scales' share. At $b = 4$ with the 128-weight groups that activation-aware weight quantization (AWQ) uses (Lin et al. 2024), the scales add 3.1 percent to the four-bit payload, which the half-byte-per-parameter figure used here leaves out.

Two families of quantization act on different terms of the latency. *Weight-only* quantization, the machine's choice, restores each weight to 16 bits inside the kernel just before the multiply. It shrinks the bytes every decode sweep rereads, but the arithmetic still runs at the 16-bit rate, so the compute-bound prefill does not move. *Weight-and-activation* quantization makes both operands integers, so the multiplications themselves run on integer units and can shorten the prefill as well. It pays off only when three conditions hold: the processor has fast integer units for that precision, the runtime has kernels for the model's layer shapes, and the activations tolerate the coarser grid. A format without a kernel is only a smaller file. Evaluating the intent model at three weight precisions with the prefill-plus-decode model of notebook 3.1 shows how closely the two-term estimate tracks measurements of the same model on the same module (table 3.1) (Franklin 2024).



INT4 shrinks the decode; the 189 ms prefill it cannot touch now dominates.

Table 3.1: Weight-only quantization of the intent model: Computed latency uses the sustained bandwidth and the illustrative prefill utilization of section 3.4.1; measured latency and action scores are the Jetson AI Lab means over BridgeData V2 episodes on the 64 GB Jetson AGX Orin module (Franklin 2024). The episodes were included in training; the reported score is one minus the mean range-normalized root-mean-square error against their recorded actions, not agreement with FP16 or a closed-loop task-success rate. The source labels its 8-bit build FP8, but runs it in a mode that stores 8-bit weights and computes in 16 bits, the same weight-only form the computed row assumes. The FP16 measurement used a general-purpose runtime (Hugging Face) rather than the optimized one (MLC) used for the other two.

Weight format	Bytes per parameter	Weights	Weight sweep	Computed latency	Measured latency	Action score (1 – mean NRMSE)
FP16	2	14 GB	98.0 ms	777 ms	840 ms	95.3%
8-bit	1	7 GB	49.0 ms	483 ms	471 ms	95.2%
INT4 (deployed)	0.5	3.5 GB	24.5 ms	336 ms	336 ms	90.1%

The computed and measured columns agree within 2.5 percent for the 8-bit format and 0.1 percent for INT4. The model underestimates the FP16 run by 7.5 percent, consistent with that run's slower runtime, so the comparison supports the model's two terms rather than its exact constants. The table also shows where weight-only quantization stops paying. Each halving of the bytes halves the sweep, but the 188.5 ms prefill is untouched, and at INT4 it is already 56 percent of the latency. Even weights that cost no bytes at all would leave the prefill, so no weight-only format can lift the intent model above 5.3 Hz. Beyond that ceiling, the gain must come from integer arithmetic in the prefill, fewer input tokens, or a processor with a higher compute roof.

The last column carries the cost. On this module the INT4 build's action score fell from 95.3 percent to 90.1 percent (Franklin 2024). OpenVLA's authors, measuring task success on a real robot with a desktop GPU, found a different pattern. INT4 weights kept 71.9 percent success against 71.3 percent for bfloat16, while INT8 weights through a general-purpose quantization library fell to 58.1 percent. The cause was time, not rounding. The library's added quantization operations cost more than the saved bytes, so the INT8 policy ran at 1.2 Hz on the evaluation GPU, against

the 5 Hz control rate of its training data. The robot therefore met each observation later than the demonstrations had (Kim et al. 2024). On the module an 8-bit format was nearly free and INT4 reduced the offline action score; on the desktop GPU the 8-bit build lost success through its slower loop and INT4 was nearly free. The two 8-bit results used different software and kernels, and the two studies scored different tasks, so the pair shows that a bit width alone does not predict the outcome; the quantization method, the kernels, the control rate they allow, and the task together do.

A quantized checkpoint is therefore a different model. It must be evaluated on the machine's own contact tasks at the deployed precision, carry its own calibration, and be named as such by the `model_id` of the proposal payload (section 3.5.4). The footprint saving also has a systems value. Dropping from 14 GB to 3.5 GB of weights returns memory that retained context and perception also need, though how much of the computed latency saving survives when the cameras and the chunk policy share the bus must be measured under load. Even with that saving, quantization leaves the machine's intent model at 2 Hz, one fresh proposal every 500 ms, while the arm below it consumes setpoints tens of times faster. No lever on the weight term closes a gap that large, and the gap raises two questions the weights cannot answer: how the arm stays supplied between evaluations, and how long whatever supplies it stays true.

Checkpoint 3.1: The memory wall and its levers

One fresh evaluation of a large proposer pays a compute-bound prefill and a bandwidth-bound decode, and each lever moves only one of them.

- ☐ **Weight-streaming ceiling**: Given a model's parameter count, its bytes per parameter, and a sustained bandwidth, can you compute the ceiling of equation 3.2 and say what it leaves out?
- ☐ **Prefill vs. decode**: Can you explain, from arithmetic intensity, why the intent model's prefill lands past the ridge of figure 3.10 while each decode token sits on the sloped roof?
- ☐ **Weight-only quantization**: Which term of equation 3.4 does INT4 shrink, and why does that leave a rate ceiling that no weight-only format can pass?
- ☐ **Shorter-input estimate**: The INT4 evaluation is a 189 ms prefill of $S = 288$ tokens plus 6 sweeps of 24.5 ms. Treating the prefill as proportional to S, estimate whether halving the input tokens would fit one evaluation inside the 200 ms period of a 5 Hz rate.

3.5 Chunk Supply and Evidence Validity

One INT4 evaluation of the intent model takes 336 ms. Over that interval the mug keeps riding the conveyor, and an arm tracking a new setpoint every few tens of milliseconds would have nothing to track if each proposal named a single setpoint. Predicting a sequence of setpoints per evaluation solves the supply problem but raises a second one that the memory wall never touched. Every setpoint in such a sequence descends from one observation, so the sequence can outlast the evidence behind it. Sizing those sequences for the machine's two proposers leads to three further questions: how long the supply stays true, what the slow proposer should hand the fast one, and what a proposal must carry for the permission path to refuse it.

3.5.1 Action chunking

If a proposer emits only one future setpoint per evaluation and takes longer than one setpoint step to finish the next inference pass, the controller runs out of planned motion while the processor is still computing. To keep dense waypoints flowing despite slow full-model inference, modern physical AI policies employ **action chunking** (definition 3.2), predicting a multi-step trajectory segment in a single pass (Zhao et al. 2023; Chi et al. 2025). Each chunk crosses the proposal boundary as one packet, reusing one evaluation across several outputs, though it does not make the next fresh camera observation arrive any sooner.

Definition 3.2: Temporal action chunking

Temporal action chunking is the policy formulation that predicts an entire horizon of H future continuous actions $\mathbf{A}_{t:t+H-1} = [\mathbf{a}_t, \dots, \mathbf{a}_{t+H-1}]$ in a single neural evaluation or diffusion rollout, amortizing parameter memory transfers across extended physical execution windows.

1. **Significance**: Chunking divides the weight traffic per supplied setpoint by H, 16 on the mobile manipulator, but leaves the delay before a new observation can change the motion at one renewal period plus one inference, 90 ms for its chunk policy. It does not make fresh observations or policy decisions arrive at the waypoint rate; a downstream controller tracks the accepted trajectory between model updates.
2. **Distinction**: Unlike single-step policies $\mathbf{a}_t \sim \pi(\cdot \mid \mathbf{o}_t)$, which condition only on the current observation and re-evaluate the full neural network at every control step, action chunking deliberately generates multi-step feedforward trajectories. It executes only the start of each before the next replaces it (receding-horizon execution), and it blends across overlapping chunks to reconcile slow neural deliberation with fast closed-loop control.
3. **Common pitfall**: Executing action chunks open-loop, without smoothing across chunk boundaries or a check by the permission path. Without smooth blending across overlapping chunk boundaries, switching between independently generated horizons injects acceleration jumps and torque shocks that excite mechanical gearbox resonances.

Predicting a horizon[8] of H actions per forward pass spreads the multi-gigabyte DRAM weight streaming cost over $H \times \Delta t_{\text{step}}$ seconds of physical execution. On the mobile manipulator's application processor, a 7B FP16 model (14 GB) generating single-step actions at the 40 Hz setpoint rate would demand 40×14 GB $= 560$ GB/s of memory bandwidth, 3.9 times the 142.8 GB/s it can sustain. A chunk of $H = 16$ waypoints cuts that demand to 35 GB/s, 24.5 percent of the sustained bandwidth, if one sweep supplies each full chunk. More frequent replanning, decoder work, and competing memory traffic consume additional bandwidth. The saving is in weight traffic per supplied waypoint, not an equal gain in arithmetic intensity: predicting a chunk adds decoder work, and its position on the roofline depends on the operations performed and the bytes actually transferred. Predicting a sequence also changes how a learned policy uses recent actions, so this supply calculation establishes only the weight traffic for the stated schedule, not that the policy attends to the right evidence or recovers from an unfamiliar contact.

8 | **Receding Horizon**: Model predictive control optimizes over a fixed horizon, executes only the first part of the plan, and then replans from the newly measured state, so the horizon recedes as time advances (Mayne et al. 2000). A chunked policy is the learned analog and inherits the same rule: the unexecuted tail of a chunk is a buffer that the next evaluation may overwrite, never a commitment the machine owes to finish.

Supply rate vs. closed-loop reaction latency On the running machine, the smaller chunk policy follows its own illustrative schedule, in which three quantities separate the rate at which setpoints reach the servos from the delay before a new observation changes a proposal:

- *Setpoint supply cadence* ($f_{\text{exec}} = 1/\Delta t_{\text{step}} = 40$ Hz, $\Delta t_{\text{step}} = 25$ ms): The rate at which the trajectory queue delivers discrete waypoints to the joint servo loop.
- *Policy replanning rate* ($f_{\text{replan}} = 20$ Hz, period $T_{\text{chunk}} = 50$ ms): The cadence at which the smaller chunk policy begins an evaluation from fresh sensory frames. The period is chosen as a whole number of setpoint steps, here 2, so that every renewal begins on a setpoint boundary.
- *Closed-loop reaction latency* ($T_{\text{reaction}} = T_{\text{chunk}} + t_{\text{infer}} = 50$ ms $+ 40$ ms $= 90$ ms): The worst sampling-phase wait plus the assumed inference time until a revised proposal is available, excluding further admission and physical response delay.

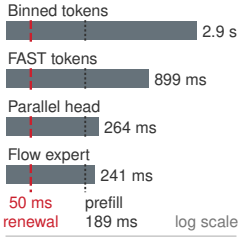
In this illustrative schedule, a disturbance immediately after capture can wait up to 90 ms for a revised learned proposal, before any additional admission and actuation delay. The independent permission path need not wait for that revision. Current qualified evidence, meaning measurements whose error bound and age have been established for this use (section 8.6 develops how), can trigger a protective response while the policy computes. Its ability to detect the relevant obstacle and complete a feasible response needs its own timing and sensing evidence. Chunking keeps candidate waypoints available; it establishes neither that response nor its deadline.

Pricing the decoder families The three decoder families of section 3.3 differ in how they turn one observation into a chunk, and the machine's processor prices that difference. Suppose the intent-sized backbone, with its deployed INT4 weights, produced one fresh chunk of $H = 16$ setpoints of 7

values each. Every family first pays the same compute-bound prefill of the observation, about 189 ms, and then the families part ways:

- Binned autoregressive tokens need 112 sequential decode steps, each rereading the backbone weights, for about 2.9 s.
- FAST tokens cut the sequence to about 30 tokens, the count its authors report per arm for a one-second chunk and therefore generous for this shorter chunk, about 899 ms, still sequential.
- A parallel head folds every action into one longer prefill, about 264 ms.
- A π_0-sized action expert of 600 MB in 16-bit weights integrates 10 steps. Each step rereads the expert and the backbone's 151 MB of cached keys and values rather than the backbone weights, about 5.3 ms per step on bandwidth, for about 241 ms in all. Section A.5.2 derives the noise-prediction and flow-matching objectives these experts train on.

The decoder choice spans 12× in latency on the same backbone, the cost that parallel decoding and action experts were introduced to remove (Kim et al. 2025; Black et al. 2024). In the expert design, weights of hundreds of megabytes far exceed the processor's on-chip caches, so each refinement step pays another off-chip transfer. A head small enough to stay resident would change that term. Reducing the number of refinement steps shortens the decoder provided the proposals keep the required task quality. Neither change touches the prefill of a fresh observation. None of the four therefore renews within the chunk policy's 50 ms period, because the prefill alone exceeds it. A large backbone that reads each observation fresh cannot also be the fast renewer on this processor, whatever its decoder. Deployed dual systems reach the same conclusion by assigning renewal to a separate, smaller model, as table 3.2 records, and the machine does the same with its ACT chunk policy.



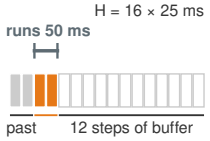
No decoder renews within 50 ms; the prefill alone is too slow.

Whatever the decoder, chunking creates one more problem, **chunk seam continuity**. Because each action chunk is generated independently from rolling sensory observations, the tail of chunk k and the head of chunk $k+1$ may show slight positional and velocity discrepancies. If a controller concatenated these chunks directly, the resulting step-discontinuities in acceleration would inject large jerk ($\dddot{\mathbf{q}}$) into the physical linkages, inducing mechanical shock, acoustic vibration, gear-tooth wear, and motor overcurrent faults. To reduce discontinuities between overlapping chunks, systems can implement **temporal ensembling**, blending overlapping trajectory predictions via exponentially weighted averaging:

$$\mathbf{a}_t = \frac{\sum_{k=0}^{K-1} w_k \, \mathbf{a}_t^{(k)}}{\sum_{k=0}^{K-1} w_k}, \quad w_k = \exp(-m \cdot k)$$

where $\mathbf{a}_t^{(k)}$ is the action for step t predicted by the k-th oldest of the K chunks that cover step t ($k=0$ is the oldest), and $m > 0$ is a tunable weighting rate. Following ACT, the oldest prediction receives the largest weight, which favors smoothness over responsiveness; a smaller m lets new observations enter the blend faster (Zhao et al. 2023). The formula assumes that the policy is queried at every control step, as ACT does, so that $K = \min(t+1, H)$. The mobile manipulator's chunk policy renews every 2 setpoints instead, and because its 50 ms renewal period is a whole number of 25 ms steps, each new chunk starts on a setpoint boundary. Because imitation-learned policies are trained on recorded demonstrations to predict motion starting from the instant the camera frame was captured, the waypoints use capture-relative times rather than starting when inference finishes, so elapsed entries are not newly executed when a chunk arrives. The first setpoint is dated at capture, and each following interval advances by 25 ms, so the last interval reaches the capture-relative horizon. By the time the 40 ms evaluation completes and the chunk arrives, the first 2 entries are already past; the next 2 setpoints precede replacement. The same formula then applies with k counting renewals rather than steps. A 16-setpoint chunk renewed every 2 setpoints gives at most $K = 8$ overlapping predictions at any step. A renewal period that fell between setpoints would leave each chunk's first action off the grid, and the blend would first have to interpolate every chunk onto common setpoint times. Below the proposal boundary, the permission path interpolates the admitted waypoints into a setpoint for every bus cycle, and the drives close their own velocity and current loops on those setpoints.

↪ The interpolation that turns admitted waypoints into per-cycle setpoints, and the fallback it executes when waypoints stop arriving, is developed in section 4.4.2.



Each chunk runs 2 of its 16 steps; later chunks overlap the rest.

Ensembling has two limits. It averages, and the average of two valid predictions need not be valid, the same mode averaging that defeats a regression head. If an older chunk passes left of an obstacle and a newer one passes right, the blend steers between them (Black, Galliker, et al. 2025). It also ignores what executes while the policy computes. With the assumed 40 ms inference and 25 ms setpoints, the arm consumes 2 setpoints of the previous chunk before the new one exists, so no new prediction can change them. *Real-time chunking* treats those setpoints as committed. It freezes the part of the previous chunk that will execute during inference and has a flow or diffusion policy fill in the rest of the new chunk so that it agrees with that frozen prefix, rather than blending afterward (Black, Galliker, et al. 2025). The method arrives from the learning side at this chapter's distinction, in which the committed prefix is motion already admitted and the tail is a buffer that the next evaluation may still rewrite.

↪ The chunk-prediction objective and the modal commitment it buys in training are developed in section 6.2.5.

Systems Perspective 3.1: The temporal ensembling jerk trap

Exponentially weighted temporal ensembling averages overlapping predictions, but averaging sampled setpoints alone guarantees neither continuous position nor continuous velocity between samples or at chunk boundaries. The interpolator on the permission path determines those properties. If successive chunks disagree in slope, a controller can receive a step in commanded velocity even when position remains continuous. In joint proportional-derivative (PD) tracking ($\tau = \mathbf{K}_p\mathbf{e} + \mathbf{K}_d\dot{\mathbf{e}}$), that velocity jump produces a finite torque step. With an illustrative derivative gain for one joint, the diagonal entry $K_d = 40\,\mathrm{N\cdot m\cdot s/rad}$ of $\mathbf{K}_d$, and an illustrative velocity discontinuity $\Delta\dot{q} = 0.5\,\mathrm{rad/s}$, the derivative term changes by $\Delta\tau = K_d\Delta\dot{q} = 20\,\mathrm{N\cdot m}$. Dividing by the servo period would instead describe a torque slew rate, not a torque. Abrupt torque changes can excite mechanical resonances or exceed drive limits. Interpolation must match position, velocity, and acceleration at adjoining segments, the property called C^2 continuity, while enforcing the machine's velocity, acceleration, and torque limits. Smoothness alone does not establish that the requested motion is feasible.

↪ The quintic spline that matches position, velocity, and acceleration across a chunk seam is developed in section 11.4.4.

A chunked policy therefore has two rates that a single "action rate" conflates: the spacing of the waypoints it supplies and the interval between the fresh observations that change them. Supplying more waypoints from one sensory context does not refresh the observation on which they depend. A downstream controller can use current state measurements while tracking admitted waypoints between renewals, and its feedback corrects tracking errors within its capabilities, but a newly perceived obstacle still requires a timely checking or replanning response. The mobile manipulator's own budget makes this distinction quantitative, first for supply and then for the time the evidence behind a chunk stays true.

3.5.2 Two proposers and the evidence horizon

Chunking reduces the weight traffic per waypoint, but the decoder pricing showed that each fresh context still waits for the large backbone's prefill, whatever the decoder. That prefill floor and the weight-streaming ceiling of figure 3.2 explain why embodied models in the published literature split into two parameter regimes rather than converging on a single model size (figure 3.12). Across 1,526 language and robotics models with disclosed parameter counts in the Epoch AI catalog between 2018 and 2026 (1,484 language and 42 robotics models) (Epoch AI 2026), language models climbed continuously past hundreds of billions of parameters. Embodied vision-language-action backbones followed that curve upward, from RT-1 (35M) to GR00T N1 (2.2B), π_0 (3.3B), OpenVLA (7B), Helix System 2 (7B), RT-2 (55B), and PaLM-E (562B), to inherit web-scale semantic and visual breadth (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Bjorck et al. 2025; Black et al. 2024; Kim et al. 2024; Figure AI 2025; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Driess et al. 2023). Alongside those backbones, a compact tier of visuomotor chunk policies and action experts, including ACT (80M), Octo-Base (93M), Helix System 1 (80M), and the π_0 action expert (300M), stayed near 10^8 parameters (88× smaller than the machine's 7B intent model) so that onboard processors can renew trajectories at tens of hertz (Zhao et al. 2023; Octo Model Team et al. 2024).

This two-tier split, a slow intent model above a fast chunk policy, can now be budgeted on the warehouse mobile manipulator, and the chunks the fast policy supplies can be checked against how

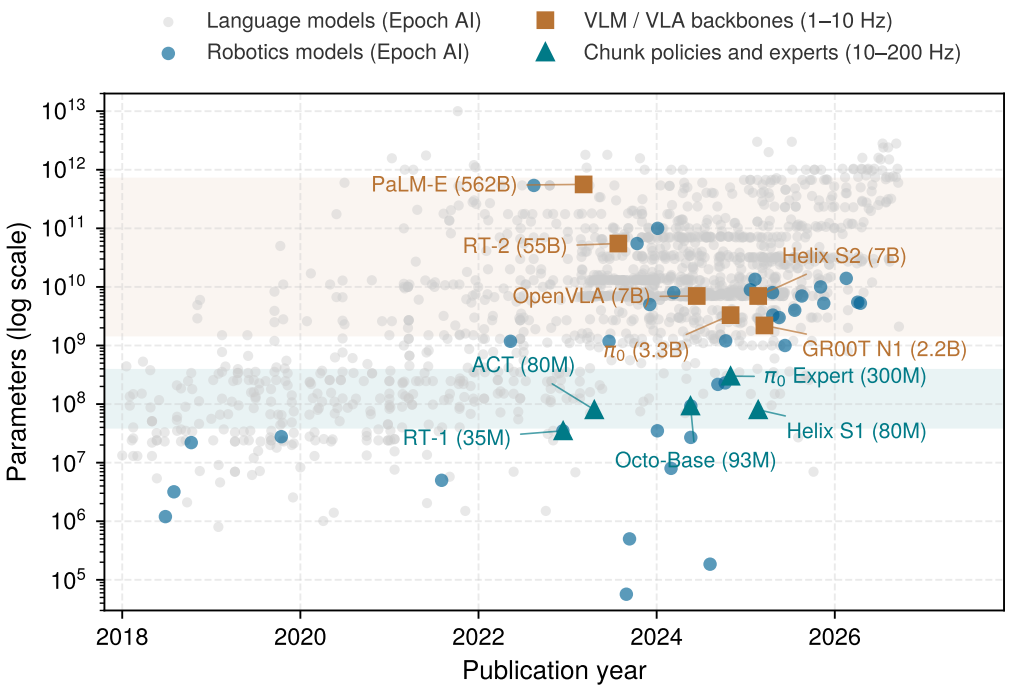


Figure 3.12: Parameter scaling of language and robotics models, 2018 to 2026: Each point plots one model with a disclosed parameter count on a logarithmic axis against its publication date. Language models scale continuously past 10^{11} parameters, while robotics models after 2022 bifurcate into multi-billion-parameter vision-language-action backbones that supply open-world semantic grounding and compact 80M–300M visuomotor chunk policies and action experts that fit real-time renewal budgets on embedded processors. Catalog series from Epoch AI (2026); component sizes for ACT, Helix System 1, and the π_0 action expert from Zhao et al. (2023), Figure AI (2025), and Black et al. (2024).

long they stay true. Budgeting that split also uses a term that the stopping budget of section 1.4.3 already charged, a lease, authority that lapses unless it is renewed. The motion the chunk policy proposes runs under a chunk lease, the longest an admitted chunk may keep driving the actuators without an admitted successor. Section 4.4 sizes that lease against the stopping budget, so that a late renewal is contained by that lease rather than by the length of the chunk. Notebook 3.2 applies the memory bound to the split and checks whether chunking alone could give the larger model the required renewal cadence.

Napkin Math 3.2: The renewal gap

Problem: Chunking spreads one weight sweep across many setpoints. Can it let the intent model renew proposals within the chunk policy's 50 ms period, or does the machine need a second, smaller proposer, and how much of the shared bus do the two proposers then take?

Variables:

- Intent model: 14 GB of FP16 or 3.5 GB of INT4 weights (Derived), an INT4 evaluation of 336 ms that decodes $S_{\text{out}} = 7$ tokens (Derived, notebook 3.1), run at 2 Hz (Chosen).
- Chunk policy: 80M parameters, ACT's size (Datasheet), 0.16 GB in FP16 (Derived), renewed at 20 Hz (Chosen).
- Chunk of $H = 16$ setpoints at $\Delta t_{\text{step}} = 25$ ms (Chosen); sustained bandwidth $\text{BW}_{\text{mem}} = 142.8$ GB/s (Derived).

Math:

Amortization. Counting weight traffic only, as section 3.5.1 did, and setting aside the prefill's arithmetic and the other decode sweeps, one FP16 sweep that emitted a whole chunk would be shared by H setpoints:

$$t_{\text{amort}} = \frac{t_{\text{stream}}}{H} = \frac{98.0ms}{16 \text{ setpoints}} \approx 6.1ms \text{ per setpoint}$$

Renewal. A fresh chunk still waits for a whole INT4 evaluation, prefill and decode together, while the chunk policy streams its own weights in about a millisecond; the rest of its inference target goes to vision encoding and the decoder's arithmetic:

$$\frac{t_{\text{INT4}}}{T_{\text{chunk}}} = \frac{336ms}{50ms} \approx 6.7 \text{ renewal periods}, \qquad \frac{0.16GB}{142.8GB/s} \approx 1.12ms$$

Bus share. The INT4 intent model streams its weights S_{out} times per evaluation, once under the prefill and once per remaining token (equation 3.4); the chunk policy streams its weights once per renewal:

$$S_{\text{out}}\, M_{\text{INT4}}\, f_{\text{intent}} + M_{\text{chunk}}\, f_{\text{chunk}} = 7 \times 3.5GB \times 2Hz + 0.16GB \times 20Hz = 49GB/s + 3.2GB/s = 52.2GB/s$$

Result: Amortization cuts the weight time per setpoint to 6.1 ms, but each fresh intent-model chunk arrives 6.7 renewal periods after its capture. The chunk policy's sweep takes 2.2 percent of its period. Together the two proposers average 52.2 GB/s of weight traffic, 37 percent of the sustained bandwidth.

Systems insight: Amortization is a supply lever, not a renewal lever, so the machine renews with a second, smaller proposer. The average hides the timing. For 147 ms of every 500 ms, the intent model's decode sweeps are modeled at the full sustained bandwidth, so a chunk-policy evaluation that overlaps them competes for the same bus, and its latency target holds only if it is measured under that contention. A late renewal is not free: the arm keeps moving on the previous chunk, and the chunk lease bounds how far.

The selected design follows. The 80M-parameter chunk policy uses ACT as its size reference and is assumed to be adapted to the mobile manipulator. It proposes base velocity and arm joint targets together at 20 Hz (a 50 ms replanning period), emitting waypoints spaced at $\Delta t_{\text{step}} = 25$ ms (a 40 Hz supply rate), within a 40 ms p99 inference target that includes vision. The permission path interpolates admitted waypoints into per-cycle setpoints and the drives close their velocity and current loops at their own faster rates. The intent model revises its target at 2 Hz. That is a candidate architecture, not a measured timing result. Loaded execution must establish the inference target, and a percentile target still permits overruns; the chunk lease contains a missed renewal.

What retained history would cost The intent model as deployed reads one frame and keeps nothing between evaluations, so the budget above is complete for it. A variant that conditioned on several seconds of history would add memory pressure that the weights alone do not show, and the cost is worth knowing before anyone proposes one. The per-token charge from section 3.3, 0.52 MB of full-width FP16 keys and values across all 32 layers through equation 3.1, now scales with history. One synchronized set of 3 224×224 frames ($3 \times 256 = 768$ patch tokens) adds 402.7 MB. This is a standard multi-head attention estimate for a hypothetical multi-camera extension, not a measured cache footprint for the published OpenVLA checkpoint. Counting language and proprioception tokens along with the visual patches raises each step to $S \approx 800$ tokens, so each observation step adds 419.4 MB of KV cache. Retaining an unpruned 5 s historical context window at 10 Hz would yield 21.0 GB of KV cache. This hypothetical history exceeds the published OpenVLA checkpoint's configured context window; the calculation exposes the storage cost of such an extension, rather than claiming that the checkpoint supports it. Combined with the intent model's 14 GB of weights, this example claims 35.0 GB, about half of the application processor's 68.7 GB (the module sold as 64 GB counts in binary gigabytes, 64×2^{30} bytes) before runtime overhead. Perception, mapping, and the chunk policy also need that memory. Even where memory capacity permits, reading a large

cache is further traffic on that same bus. Bounding retained history therefore serves both capacity and timely renewal. Spatial pooling or cross-attention can reduce context traffic; fixed-horizon action output does not by itself bound the observation history retained by the backbone.

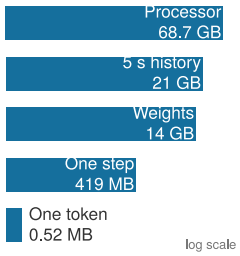
The split fixes how often each proposer renews. The moving target fixes how long each renewal stays true. At the pick station the arm takes the mug from a moving conveyor, and a chunk is useful only while the mug stays within the grasp tolerance r_{tol} of where the chunk's evidence placed it, starting from the handle estimate's initial error e_0. Untracked, the belt's steady speed v_{conv} uses up the margin $r_{\text{tol}} - e_0$ within the freshness deadline $e_{\max}/v_{\max}$ of section 2.2, here with $e_{\max} = r_{\text{tol}} - e_0$ and $v_{\max} = v_{\text{conv}}$. A tracker removes that steady motion. It is the memory stage of the pipeline, re-estimating the mug's position and velocity from each new frame, and a chunk computed from its estimate aims at where the mug will be if the belt keeps its speed. What that extrapolation cannot foresee is a change of speed after capture, the mug slipping on the belt at a residual acceleration a_{slip}. The chunk's setpoints were fixed at its capture, so later frames do not correct them, and extrapolation over an evidence age T drifts by $\frac{1}{2} a_{\text{slip}} T^2$. Setting that drift equal to the margin gives the **evidence horizon** of the tracked mug, the longest time the evidence behind a chunk keeps the mug within tolerance, in equation 3.5:

$$\tau_{\text{ev}} = \sqrt{\frac{2\,(r_{\text{tol}} - e_0)}{a_{\text{slip}}}} \tag{3.5}$$

When inference meets the stated target and no further admission delay is charged, a chunk is replaced one renewal period plus one inference after the capture it was computed from, the reaction latency $T_{\text{chunk}} + t_{\text{infer}}$ of section 3.5.1. That span must fit inside τ_{ev}, and the chunk must carry enough setpoints to bridge it, as equation 3.6 requires:

$$h_{\min} = \left\lceil \frac{T_{\text{chunk}} + t_{\text{infer}}}{\Delta t_{\text{step}}} \right\rceil \tag{3.6}$$

Together, equation 3.5 and equation 3.6 distinguish the setpoint buffer a chunk carries from the duration its evidence remains valid. Notebook 3.3 applies both bounds to the machine's schedule.



5 s of retained context outweighs the 14 GB of weights.

Napkin Math 3.3: Buffer, not permission

Problem: The chunk policy renews every 50 ms, is assumed to need 40 ms per evaluation, and emits 16 setpoints 25 ms apart. Does the schedule depend on tracking the mug, how many setpoints must a chunk carry, and how much of each chunk may the arm execute on the evidence behind it?

Variables:

- Conveyor speed $v_{\text{conv}} = 0.20$ m/s and residual slip acceleration $a_{\text{slip}} = 0.50$ m/s² (Illustrative).
- Grasp tolerance $r_{\text{tol}} = 15$ mm (Chosen) and initial handle error $e_0 = 3$ mm (Illustrative), a margin of 12 mm.
- Renewal period $T_{\text{chunk}} = 50$ ms, setpoint step $\Delta t_{\text{step}} = 25$ ms, and $H = 16$ (Chosen); inference $t_{\text{infer}} = 40$ ms, a p99 target that includes vision (Illustrative).

Math:

Untracked deadline. The belt alone uses up the margin in

$$\frac{r_{\text{tol}} - e_0}{v_{\text{conv}}} = \frac{12\ \text{mm}}{0.20\ \text{m/s}} = 60\ \text{ms}$$

Tracked horizon. With the steady motion removed, equation 3.5 gives

$$\tau_{\text{ev}} = \sqrt{\frac{2 \times 12\ \text{mm}}{0.50\ \text{m/s}^2}} \approx 219\ \text{ms}$$

Age at replacement and supply. equation 3.6 and the drift over the evidence age since capture give

$$T_{\text{chunk}} + t_{\text{infer}} = 50 + 40 = 90\,\text{ms}, \quad \tfrac{1}{2} a_{\text{slip}} T^2 = \tfrac{1}{2} \times 0.50 \times (0.09\,\text{s})^2 \approx 2.0\,\text{mm}, \quad h_{\text{min}} = \lceil \tfrac{90}{25} \rceil = 4$$

The whole chunk. Executed open-loop to its end, one chunk would carry the mug to

$$H\,\Delta t_{\text{step}} = 16 \times 25\,\text{ms} = 400\,\text{ms}, \quad e_0 + \tfrac{1}{2} \times 0.50 \times (0.40\,\text{s})^2 = 3 + 40 = 43\,\text{mm} > 15\,\text{mm}$$

Sensitivity. Renewing at 10 Hz instead raises the age at replacement to 140 ms, still inside τ_{ev}, and h_{min} to 6.

Result: Untracked, the mug leaves tolerance after 60 ms, before the first replacement at 90 ms, so this schedule depends on the tracker. Tracked, the evidence holds for about 219 ms: 4 setpoints bridge each renewal, and the 90 ms evidence age at replacement permits drift of only 2.0 mm. The 400 ms chunk outlasts its evidence by 181 ms.

Systems insight: The chunk is sized for supply, not for execution. Its tail beyond τ_{ev} keeps setpoints flowing if a renewal slips, but it is not permission to finish the motion, because the drift it would carry the arm through cannot be taken back once the gripper closes on the wrong place. The permission path therefore ages each chunk's evidence against τ_{ev}, and when renewal stops, the chunk lease, not the chunk length, bounds how far the arm may keep going.

On one clock, the schedule looks like figure 3.13. With direct replacement, each chunk is in force for only 50 ms before its successor replaces it, so motion computed from that capture ends when the capture is 90 ms old, inside the 219 ms evidence horizon. Temporal ensembling instead retains contributions from older chunks. A blend must exclude contributions whose evidence has expired and carry the oldest capture time among the contributions it retains; the direct-replacement timing strip does not bound the age of that blend. If the chunk policy overruns its inference target, the notebook's assumption no longer holds, and if renewal stops altogether, a further setpoint remains eligible only while both its evidence and the active chunk lease remain valid. If the implementation uses temporal ensembling, it must filter expired contributions before blending and date the result from the oldest retained evidence; seam continuity and feasibility still require independent checks.

Under the stated assumptions, the conveyor calculation supplies the quantities needed to connect cognition to physical permission. One chunk covers 400 ms of motion, a new chunk is assumed to meet its 40 ms inference target, and the evidence stays within the stated grasp tolerance for about 219 ms. The Nervous System must combine these proposer limits with the body's measured limits to establish its handoff record. That requires more information than a trajectory tensor alone. Timely delivery and usable evidence bound how long a proposal can remain eligible; they do not establish that the proposed grasp was correct.

3.5.3 Handing intent to the chunk policy

Whether the permission path can audit the slow intent model's goal alongside the fast chunk policy's trajectory depends on what crosses the interface between the two tiers. That two-tier split is not peculiar to this machine. Helix runs a 7B-parameter vision-language model at 7 to 9 Hz above an 80M-parameter policy at 200 Hz (Figure AI 2025), and GR00T N1 runs its vision-language model at 10 Hz above a flow-matching transformer at 120 Hz (Bjorck et al. 2025). Across these and other published designs (table 3.2), the reported rates bracket the machine's own 2 Hz and 20 Hz. What separates them is less their rates than what the slow model hands the fast one, and that choice decides what the permission path can check.

Deployed systems use three kinds of interface between the two tiers. With a *latent vector*, Helix's slow model passes a single continuous latent that conditions the fast policy (Figure AI 2025), and GR00T N1's policy attends to token embeddings taken from an intermediate layer of its vision-language model (Bjorck et al. 2025). Because a latent is trained end to end, it carries whatever the fast policy finds useful and costs no decoding. Other architectures pass a *language subtask*: $\pi_{0.5}$ first predicts the next subtask as text and then generates actions conditioned on it (Black, Brown, et al. 2025), while the orchestrating model of Gemini Robotics 1.5 sends its action model natural-language step instructions (Gemini Robotics Team 2025a). Still others emit an explicit *geometric target*, as in

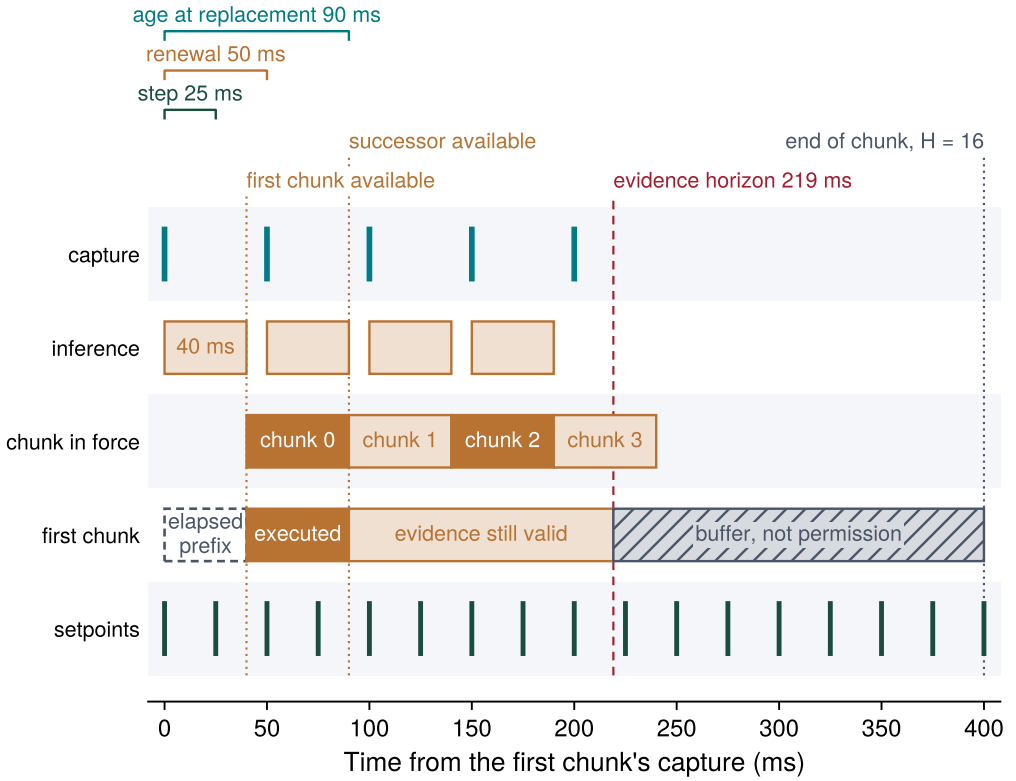


Figure 3.13: Chunk supply against the evidence horizon: With direct replacement, time runs from the capture that the first chunk is computed from. The chunk policy renews every 50 ms and needs 40 ms per evaluation, so each chunk is in force for one renewal period and is replaced when its evidence is 90 ms old. The first chunk predicts a capture-relative horizon of 400 ms, including the prefix already elapsed before arrival, but the tracked mug stays within the grasp tolerance only for the 219 ms evidence horizon; the hatched tail is supply that keeps the setpoint stream full if renewal slips, not permission to execute it.

Gemini Robotics-ER, which predicts points, two- and three-dimensional boxes, trajectories, and grasps with coordinates in the scene (Gemini Robotics Team 2025b).

The permission path can check only a claim it can compare with its own evidence, and the three interfaces differ exactly there. A latent vector means something only to the network that reads it. No independent checker can tell whether it refers to the mug or to the photograph of a mug on a carton (section 3.6.1), and it carries no position that could age against the evidence horizon τ_{ev}. A text subtask can be checked against the task predicates the permission path has admitted, but "pick up the mug" names no location, so it can be neither compared with measured depth nor aged against drift. A geometric target allows both checks. It can be compared with calibrated depth and keep-out geometry, and aged from the capture time of the frame it came from. This geometric interface lets the machine's intent lease be enforced, because that lease names a grounded target, never a path, and expires at the earliest of the time its evidence stays valid and the deadlines the permission path sets. With a latent or a text interface, the permission path can still check each chunk's motion under the chunk lease, but the target behind that motion lacks this independent grounded-position check.

The mobile manipulator therefore has its intent model emit a target: a position, an orientation, and a gripper command, the same seven values OpenVLA decodes as a step of motion, together with the capture time of the frame it read. The timing shows why that target cannot stand alone. One evaluation takes 336 ms, longer than the 219 ms evidence horizon of the tracked mug, so the pose the intent model read is stale before its target arrives. The intent model therefore contributes which object to take and approximately where, and the tracker of section 3.5.2 supplies where that object is now. The tracker binds the target to its own latest estimate of the mug, and the 80M-parameter

chunk policy aims at that fresh pose. The two leases then age against different clocks. The intent lease ages from the capture time of the tracker's latest estimate and is renewed each time the tracker confirms the same mug, so it lapses when tracking is lost or a permission-path deadline passes. Each chunk carries the capture time of the oldest observation behind it, normally the frame the chunk policy itself read, and its evidence ages from that time against τ_{ev}.

The geometric bottleneck costs capability. A target pose cannot say how to fold a shirt or where to regrasp a cable, information a latent carries without a schema, so a checkable interface limits the tasks a single target can express. The machine accepts that limit in exchange for an intent the permission path can refuse. The same reasoning applies one level down to the chunk policy's action space. Joint targets can be checked directly against joint limits and end-effector poses against workspace keep-out regions, which is why the proposal payload of section 3.5.4 carries both.

Table 3.2: State of practice in learned proposers, as of 2026: Published decoder mechanisms, sizes, and rates, with the interface each dual system passes from its slow model to its fast one. Rates and latencies are as the authors report them on different hardware, so they compare designs, not processors; none was measured on the machine's application processor.

System (year)	Decoder mechanism	Reported sizes and rates	Slow-to-fast interface
OpenVLA (2024) (Kim et al. 2024)	Autoregressive binned tokens, one per action dimension	7B; about 6 Hz on a desktop GPU	None (single model)
OpenVLA-OFT (2025) (Kim et al. 2025)	Parallel decoding, continuous regression head	109.7 Hz against 4.2 Hz throughput for OpenVLA, 8-step chunks	None (single model)
π_0 (2024) (Black et al. 2024)	Vision-language backbone with a flow-matching action expert	3.3B including a 300M expert; $H = 50$, 10 integration steps; 73 ms per chunk on a desktop GPU; control up to 50 Hz	None (single model)
π_0-FAST (2025) (Pertsch et al. 2025)	Autoregressive compressed (DCT and byte-pair) tokens	About 30 tokens per arm per one-second chunk (60 for two arms); about 750 ms per chunk against about 100 ms for π_0	None (single model)
$\pi_{0.5}$ (2025) (Black, Brown, et al. 2025)	One model: text subtask, then flow-matching action expert	10 integration steps; control at 50 Hz; one text subtask decoded before each action chunk	Language subtask
Helix (2025) (Figure AI 2025)	Vision-language model above a cross-attention visuomotor transformer	7B at 7–9 Hz; 80M at 200 Hz over 35 degrees of freedom; onboard embedded GPUs	Latent vector
GR00T N1 (2025) (Bjorck et al. 2025)	Vision-language model above a flow-matching transformer	2.2B total, 1.34B in the language model; 10 Hz and 120 Hz; $H = 16$, 4 steps; 63.9 ms per chunk on a data center GPU	Intermediate-layer token embeddings (latent)
Gemini Robotics (2025) (Gemini Robotics Team 2025b, 2025a)	Cloud-hosted backbone with an on-robot action decoder; embodied-reasoning model as orchestrator	Backbone under 160 ms; about 250 ms end to end; 50 Hz control with chunks	Points, boxes, and grasps (ER); language step instructions (1.5)
This machine	INT4 intent model above an ACT chunk policy	7B at 2 Hz (336 ms computed); 80M at 20 Hz, $H = 16$ at 25 ms	Target pose with capture time

3.5.4 Information the proposer must supply

The conveyor budget gives each payload field a physical purpose. The receiver needs the candidate motion, the capture time from which its evidence ages, and the conditions limiting its validity. Bundling that information with delivery metadata defines the proposal payload rather than a new evidence record in the book's record chain. Section 4.4.1 wraps this payload in the proposal header the permission path validates, and the Nervous System's handoff record combines this supply with the body's limits.

The **proposal payload** $\mathcal{P}$ is the typed candidate motion that the unprivileged Brain sends to the real-time Nervous System, packaged with the capture time of its evidence, the time after which it lapses, and the provenance the receiver needs to check it:

$$\mathcal{P} = \langle t_{\text{capture}}, t_{\text{valid}}, H, \Delta t_{\text{step}}, \mathbf{Q}_{1:H}, \dot{\mathbf{Q}}_{1:H}, \mathbf{X}_{1:H}, \mathbf{g}_{1:H}, \text{model_id}, \text{lease_id}, \hat{p}, \text{crc} \rangle$$

The capture and validity times let the receiver compare the motion with freshness and calibration requirements it establishes independently; for the conveyor pick, t_{valid} can lie no later than $t_{\text{capture}} + \tau_{\text{ev}}$.

The payload requests motion. It is neither execution permission nor the handoff evidence record, and a payload that omits its observation time or validity horizon invites the receiver to execute a proposal computed from a scene that no longer exists.

The fields in table 3.3 illustrate the information needed to interpret a candidate trajectory, rather than prescribing a second wire format. The proposer supplies trajectory and observation information. Validity claims require supporting evidence, while the receiver applies the separately established chunk lease. A `lease_id` associates the request with that permission process; it does not let the proposer grant itself authority. The header and handoff record specify delivery, admission, and expiration.

Table 3.3: Illustrative proposal payload fields: Candidate-motion information and associated metadata. The proposal header and handoff record belong to the Nervous System; this illustration adds no separate qualification record.

Field	Type	Unit	Systems Semantics and Verification Rule
`t_capture`	`uint64_t`	ns	Hardware timestamp of oldest sensory observation conditioning the proposal.
`t_valid`	`uint64_t`	ns	Monotonic time after which proposal expires ($\leq t_{\text{capture}} + \tau_{\text{ev}}$).
`horizon_H`	`uint16_t`	count	Number of prospective trajectory setpoints ($H = 16$ on this machine).
`delta_t`	`uint32_t`	µs	Temporal interval between consecutive waypoints ($\Delta t_{\text{step}} =$ 25,000 µs, a 40 Hz supply rate).
`q_traj`	`float32[H][n]`	rad	Candidate joint trajectory array across n degrees of freedom.
`q_dot_traj`	`float32[H][n]`	rad/s	Velocity setpoints; checked for continuity when adjoining chunks meet.
`x_cartesian`	`position + orientation[H]`	m, quat	End-effector target poses; verified against workspace keep-out boundaries.
`gripper_cmd`	`float32[H]`	$[0, 1]$	Gripper closure target; evaluated against grasp force and slip limits.
`model_id`	`uint32_t`	identifier	Identifier resolving to the model architecture, weights checkpoint, and calibration profile.
`confidence`	`float32`	$[0, 1]$	Score $\hat{p}$ from the method named in the calibration profile (softmax margin for token decoders; sample dispersion, ensemble disagreement, or a conformal set for continuous decoders); absent when no method is qualified.
`lease_id`	`uint64_t`	identifier	Identifier linking the proposal to a time-limited permission record.
`crc32`	`uint32_t`	checksum	Integrity checksum protecting shared-memory ring buffer payload.

↪ **Byte layout:** The wire format that carries a chunk's evidence time, expiry, horizon, and joint setpoints, with offsets, types, and checks, is given in section C.15.1.

Checkpoint 3.2: Chunk supply and evidence horizons

Before analyzing runtime failure modes, check that you can separate what a chunk supplies from how long the evidence behind it holds.

- ☐ **Amortization vs. renewal**: Can you explain why a chunk cuts the intent model's weight time per setpoint to 6.1 ms yet leaves each fresh intent-model chunk several renewal periods late, and why that forces a second proposer?
- ☐ **KV cache growth**: Can you use equation 3.1 to size the cache for a retained history, and explain why reading that cache competes with current sensor transfers on the same bus?
- ☐ **Supply vs. validity**: Can you explain why a chunk must carry at least $h_{\min}$ setpoints to bridge one renewal, why the evidence age of motion from one observation must stay inside τ_{ev}, and why the remaining setpoints are a buffer rather than a plan?
- ☐ **Slip estimate**: If residual slip on the conveyor quadrupled, by what factor would equation 3.5 shrink τ_{ev}, and would the 90 ms age at replacement still fit inside it?

3.6 Cognitive Vulnerabilities

A chunk proposed by a mobile manipulator's policy can arrive within its 40 ms execution budget, with every joint setpoint smooth and within kinematic limits, and yet still collide the gripper into the rim of the mug. The trajectory meets its real-time deadline, but its physical content is wrong, and the most consequential failures of learned physical AI systems sit in that gap. Learned foundation models expand task coverage across open-world environments, but no general proof of their integrated, closed-loop behavior across unconstrained state spaces is available. Engineering practice must instead analyze verified components within explicitly bounded operating regions.

Without a verified input domain and sensitivity bound, small sensing changes to a learned proposer can yield unexpectedly large changes in proposed motion, a sensitivity demonstrated early by imperceptible image perturbations that flipped image classification outputs (Szegedy et al. 2013). Benchmark success does not establish closed-loop stability, timing bounds, or physical safety under unseen operating conditions; those guarantees require independent evidence across the model, the software runtime, and the physical plant.[9] Even when a valid sensitivity bound exists, it characterizes only the relationship between input perturbations and output deviations over a specified domain. It provides no proof that the nominal proposed action is collision-free or physically admissible, meaning analytical bounds can inform, but never replace, an independent permission decision.

A numerical interface easily traps malformed tensors, NaN values, and out-of-range floating-point setpoints, but it cannot detect a wrong grasp plan. Both classical pipelines and learned models can emit well-formed yet wrong commands; learning, however, adds failures rooted in high-dimensional latent features, out-of-distribution hallucinations, and uncalibrated confidence scores. Independent safety monitors can reject proposals that contradict measured geometry or joint limits, but they cannot infer true task intent. Safe operation in the physical world requires examining each of these failure modes and stating the evidence checks needed beyond packet formatting.

[9] **Lipschitz Continuity in Neural Policies**: Although neural networks with smooth activations are continuous on compact domains, a sensitivity bound $L_{\text{lip}} = \sup_{\mathbf{x}_1 \neq \mathbf{x}_2} \frac{\|\pi(\mathbf{x}_1) - \pi(\mathbf{x}_2)\|}{\|\mathbf{x}_1 - \mathbf{x}_2\|}$ must be established over a stated operating region. Where that bound holds, sensor error $\|\Delta\mathbf{x}\| \leq \epsilon$ bounds the action change by $\|\Delta\mathbf{a}\| \leq L_{\text{lip}}\epsilon$; an unverified L_{lip} gives no usable action-error guarantee. Full derivations appear in section A.4.2.

3.6.1 Semantic hallucinations

Consider a mobile manipulator clearing items near a conveyor. A discarded shipping carton carries a high-resolution printed photograph of a ceramic mug. The robot's vision backbone attends to the visual texture, and the policy proposes an end-effector trajectory that closes the parallel jaws around the printed handle. The trajectory tensor is numerically flawless: smooth, within joint velocity and acceleration bounds, and clear of modeled obstacles. The handle it targets, however, is flat ink on cardboard. In web-scale vision models, attention maps track statistical correlations across 2D pixel arrays rather than measured 3D geometry.

Independent geometric evidence exposes this hallucination directly. A runtime validator compares the proposed grasp pose against calibrated depth returns to check whether physical relief supports the handle. If the depth sensor shows a flat cardboard surface within measurement resolution, the runtime rejects the proposal; if sensor dropouts or occlusion leave the region uncertain, the grasp remains unverified and must not be admitted. In neither case can a 2D attention map stand in for a 3D depth measurement. The proposer must therefore supply both its target coordinate frame and the supporting observation timestamps so that downstream checks can separate a physically grounded motion plan from an unverified visual guess.

↪ How depth error is bounded and qualified before a spatial claim can be trusted is developed in section 8.6.

3.6.2 The softmax illusion and out-of-distribution overconfidence

A natural temptation is to filter out hallucinations like the cardboard mug, or frames degraded by intense specular glare, dynamic shadows, and water droplets on camera lenses, by rejecting proposals whose reported confidence falls below a threshold. That defense fails when deployment interfaces conflate mathematical normalization with the physical probability of task success. In discrete classification, classification-based tokenization, or categorical action heads, raw output logits are transformed via the softmax operator:[10]

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

whose outputs sum to one purely by algebraic construction. When an embodied machine encounters those out-of-distribution (OOD) observations or novel visual textures, the network can still emit

[10] **Softmax** (soft arg max): Bridle (1990) named the normalized exponential "softmax" because it is a differentiable version of the winner-take-all operation that picks the largest input. A function built to pick a winner keeps doing so far from its training data, pushing its top output toward one whenever one logit separates from the rest, whether or not the input resembles anything the model has seen.

confidence scores near one for physically disastrous actions. Modern deep networks are often overconfident even on in-distribution inputs (Guo et al. 2017), and networks constructed from rectified linear units provably approach maximum confidence on inputs far from their training distribution (Hein et al. 2019).

Calibration evaluates how closely a model's predicted confidence aligns with observed empirical success rates under specified conditions. A confidence score is useful for selecting or abstaining from a proposal only to the extent that this relationship holds for the deployed task. Section A.2.3 formalizes binned expected calibration error and post-hoc temperature scaling. From a systems perspective, the execution runtime must bind calibration metadata directly to the model identity (`model_id`), preventing supervisory layers from mistaking uncalibrated algebraic certainty for physical permission.

In modern physical AI systems, populating the action payload's `confidence` field is rarely as straightforward as extracting a softmax probability, because contemporary chunking policies do not operate as discrete classifiers. Action Chunking with Transformers (ACT), for example, decodes each multi-step chunk deterministically at inference time by clamping its latent style variable to the prior mean (Zhao et al. 2023). Similarly, diffusion and flow-matching policies iteratively denoise trajectories from Gaussian noise vectors. Neither architecture natively emits a normalized score. For these continuous decoders, confidence must be derived through an explicit, qualified estimation technique. Three primary architectures are used in practice:

- *Sample dispersion.* The system draws multiple trajectory chunks for the identical sensory observation by varying random diffusion noise seeds or sampling latent vectors. Measuring geometric dispersion across these trajectories describes the policy's sampled output variability. Wide dispersion can reflect several valid motions as well as uncertainty; it does not by itself isolate uncertainty caused by missing knowledge.
- *Ensemble disagreement.* Multiple independently trained policy heads evaluate the same sensory input simultaneously. Downstream monitors measure the variance among their trajectory proposals, treating significant disagreement as an indicator of model uncertainty.
- *Conformal thresholds.* To convert heuristic dispersion or disagreement metrics into actionable statistical guarantees, the runtime applies inductive conformal prediction. Using a held-out calibration dataset, the system establishes an empirical threshold such that, under the assumption of exchangeability,[11] the predicted set covers the calibrated target with a chosen marginal coverage probability. Specifically, each candidate trajectory receives a nonconformity score measuring its residual error or disagreement, and the conformal set collects all candidates whose scores fall within the quantile threshold measured on the calibration trials, so high sensory ambiguity widens the set to include distinct trajectories while pairing the threshold with an admissibility filter can leave no candidate in the set at all. Physical validity still requires independent checks against current geometry and plant limits. If the conformal set is empty or contains mutually conflicting actions, the runtime refuses the candidate or abstains; if active motion is no longer admissible, it requires a physically qualified response or operator intervention (Ren et al. 2023; Angelopoulos and Bates 2023).

[11] **Exchangeability**: The conformal guarantee assumes only that the calibration trials and the new trial are exchangeable, a weaker condition than independent and identically distributed data (Vovk et al. 2005). A deployed robot can break even that weaker assumption because its actions change the scenes it meets next, the endogenous data of the second law (section 1.10). A threshold calibrated under one policy therefore has to be re-established after every policy or deployment change, and its stated coverage lapses in between.

Each uncertainty estimation strategy incurs distinct hardware and memory-bandwidth costs. Generating multiple samples from a single observation can reuse visual backbone features and batch each decoding iteration through the policy head, sharing a weight read across samples. Iterative decoders still revisit their weights across refinement steps, and the balance between arithmetic and memory traffic depends on the batch and kernels. In contrast, an ensemble of disparate models requires reading separate sets of weights across the memory bus for every head, multiplying memory bandwidth traffic by the number of heads. The action payload must therefore treat the confidence scalar $\hat{p}$ as meaningful only when accompanied by the identifier of the method that produced it and the corresponding calibration record, both resolved through `model_id`. A model lacking a qualified estimation pipeline must omit the confidence score entirely rather than emit an ungrounded default value.

The stipulated values in table 3.4 illustrate how environmental and hardware shifts can open a gap between confidence and success; they describe nominal illustrative regimes rather than a single deployed benchmark. The near-OOD row is a *covariate shift*, a change in the inputs the

deployed model meets while the task stays the same. Across all shifted regimes, reported confidence significantly outstrips actual task success. A supervisory filter relying solely on an uncalibrated score threshold would mistakenly admit hazardous trajectories. Choosing an appropriate supervisory response depends on the integrity of the remaining sensor streams and the physical capabilities of the machine; a confidence discrepancy alone cannot dictate whether the robot should slow down, collect additional sensor frames, or execute an emergency stop.

Table 3.4: Illustrative Confidence–Success gaps under distribution shift: Stipulated confidence and task-success rates for four operating regimes, with candidate responses requiring separate physical justification. These values are illustrative, not test results.

Distribution Shift Regime	Physical & Environmental Trigger	Conf. vs. Success (conf vs. success)	Confidence–Success Gap (conf − acc)	Downstream Supervisory Action
In-Distribution (Nominal Baseline)	Clean optics, structured laboratory lighting, nominal workpiece textures	conf $= 0.91$, acc $= 0.89$	$+0.02$	Apply independent limits before nominal execution
Near-OOD Shift (Covariate Shift)	Specular reflections, lens water droplets, dynamic shadow transitions	conf $= 0.89$, acc $= 0.52$	$+0.37$	Abstain or seek new evidence; permit probing only with a qualified contact policy
Far-OOD Shift (Semantic Shift)	Uncataloged deformable obstacles, human limb intrusion into workspace	conf $= 0.78$, acc $= 0.08$	$+0.70$	Reject proposal; use feasible controlled response
Sensor Degradation (Hardware Drift)	Photodetector thermal noise (> 75°C), focal blur from vibration	conf $= 0.84$, acc $= 0.31$	$+0.53$	Restrict operation to modes supported despite the sensor degradation

DRAFT

3.6.3 Perceptual chattering and covariate shift

When sensory observations hover near a boundary between competing object classes or grasp modes, small frame-to-frame noise flips the winning label back and forth, a failure known as *perceptual chattering*. If a motion planner treats each label flip as an abrupt change in the physical world, it discards and resets its trajectory on every frame. In the fatal 2018 Tempe autonomous vehicle collision, the National Transportation Safety Board record documents this failure in the automated driving system: the perception module alternated between classifying a pedestrian pushing a bicycle as a vehicle, a bicycle, and "other." After each reclassification the path predictor dropped the object's earlier locations, preventing the motion planner from seeing that the obstacle was crossing the vehicle's path (National Transportation Safety Board 2019). Covariate shift, the near-OOD regime of table 3.4 in which the inputs change while the task stays the same, pushes more frames toward those boundaries; section 5.2 develops it.

In robotic manipulation, perceptual chattering injects rapid direction reversals into joint trajectory setpoints. Over a discrete control interval Δt, a sharp step in commanded velocity demands an instantaneous acceleration $\Delta\dot{\mathbf{q}}/\Delta t$ beyond what the actuators can deliver. PD joint controllers react to the resulting velocity errors by commanding large opposing motor torques, the same derivative-gain torque step that the temporal-ensembling jerk trap of section 3.5.1 priced at a single chunk seam, now repeated at every flip. The resulting current spikes, tracking errors, and heat strain the power stage and gearbox, risking protective drive faults or structural vibration. Physical AI runtimes suppress perceptual chattering with three defenses:

1. *Temporal hysteresis and debouncing.* Requiring a newly observed class or spatial mode to persist across N consecutive perception frames before committing a state change, just like switch debouncing[12] in embedded firmware.
2. *Recursive state filtering.* Fusing successive observations into a running estimate so that a single noisy frame cannot overwrite an established world state.

↩ The tracking-history loss under classification changes in the 2018 Tempe collision is established in section 1.8.1.

12 **Debouncing**: The contacts of a mechanical switch bounce for milliseconds when pressed, so embedded firmware accepts a new switch state only after it has persisted for a set interval. The same N-frame rule applied to class labels suppresses chattering, but it delays every genuine change by N frames, time charged to the observation-age budget before any proposal is renewed.

3. *Rate and jerk limiting.* Enforcing joint acceleration and jerk bounds in the real-time controller so that high-level label flicker cannot reach the motor shafts as torque shocks.

3.6.4 When intent expires

↪ Recursive state estimation, which accumulates evidence across frames instead of trusting the latest one, is developed in section 9.2.

Even when perception neither hallucinates nor chatters, the physical scene continues to move after a valid goal is issued. Notebook 3.3 already showed a chunk outliving its evidence, and the same failure applies one level up, to intent. Renewing a high-level goal identifier, or retransmitting a previous trajectory, does not refresh the observation that placed the mug. High-capacity intent models, which deliberate over long time horizons at low update rates, must therefore provide goals whose validity can be continuously audited against fresh, real-time telemetry while high-frequency reactive policies stream immediate motion.

The execution receiver must distinguish between rejecting a stale proposal and terminating an active, executing trajectory. Dropping an incoming proposal that failed its arrival deadline or validity check does not automatically invalidate the chunk currently running on the motors. The active chunk continues executing so long as its own chunk lease, the intent lease governing its target, the evidence behind it, its state bounds, and its safety margins all remain valid. If the active chunk lease or the governing intent lease expires, or the evidence behind the active chunk lapses, the supervisory permission path must end nominal tracking and engage the qualified fallback unless a valid successor restores the required authority. Because holding position, decelerating to a controlled stop, and removing torque require distinct physical preconditions, section 12.9 develops the formal fallback ladder across these operational regimes.

Each of the four vulnerabilities ends the same way. Hallucination, overconfidence, chattering, and expiry cannot be reliably ruled out by the proposer's own output and require checks against independently qualified evidence, which is why the opening comparison of classical and learned methods resolves into a partition rather than a choice. Physical AI architectures assign semantic interpretation to the learned proposer and execution authority to the permission path (table 3.5). Learned foundation models supply flexible task coverage across unstructured environments, but their lack of closed-loop formal guarantees requires treating their outputs as unprivileged proposals. Conversely, classical analytical checks enforce strict physical invariances and real-time bounds, yet their guarantees remain conditional on accurate kinematic models and fresh sensor data. Safety therefore rests on the interface, which must never let an unverified proposal bypass the checks of the permission path.

Table 3.5: Classical analytical methods vs. learned foundation models: Trade-offs across generalizability, verifiability, pacing, and fallback, with the synthesis that partitions proposal capability from execution authority.

Systems Dimension	Classical Analytical Methods	Learned Foundation Models	Physical AI Architectural Synthesis
Foundational Principle	Analytical differential equations, geometry, CAD models, Lyapunov stability	High-capacity neural representations fitted to empirical multimodal data	**Hybrid Hierarchy:** Learned intent proposals gated by the permission path
Generalizability & Perception	**Model-dependent:** Explicit geometry and features may lose coverage under shift	**Data-dependent:** Open-vocabulary grounding can improve coverage but may fail under shift	**Unprivileged Deliberation:** Foundation models synthesize continuous semantic goals
Mathematical Verifiability	**Conditional:** Selected properties can be proved within model assumptions	**Limited:** Empirical behavior needs separate bounds and runtime evidence	**Proposal Boundary:** Gate checks modeled limits using independent evidence
Temporal Pacing	**Boundable:** Selected controllers support real-time deadlines under stated execution assumptions	**Multi-Rate:** Full inference, chunk refresh, and waypoint execution have distinct rates	**Split Proposers:** A small chunk policy renews while the intent model revises goals
Failure Modes & Fallback	Implementation defects; model mismatch; failure on uncataloged objects	Semantic hallucinations; out-of-distribution overconfidence; expired intent	**Conditional Fallback:** Reject detected violations and execute a feasible response

Checkpoint 3.3: Proposals that arrive on time and are wrong

A proposal can meet every deadline and still target the wrong thing, so the receiver needs evidence the proposer cannot supply about itself.

- ☐ **Classical vs. learned**: For the welding cell and the conveyor mug pick, which approach would you use to propose motion in each, and which properties could you still prove or measure for each?
- ☐ **Interface checkability**: Can you explain why the permission path can enforce this geometric intent lease on a target pose with a capture time but not on an ungrounded latent vector alone?
- ☐ **Hallucination vs. staleness**: For the grasp on the carton photograph and for a smooth chunk that arrives after τ_{ev}, which payload field or independent measurement lets the receiver refuse each?
- ☐ **Threshold estimate**: Assuming every attempt in a regime receives its tabulated confidence score, if the receiver admitted every proposal scoring above the far-OOD confidence in table 3.4, which regimes would pass, and what stipulated fraction of near-OOD attempts would fail?

3.7 Fallacies and Pitfalls

Learned models expand the behavioral repertoire of robotic systems, but their predictions remain ungrounded hypotheses until verified against physical dynamics and environmental constraints. In deployed systems, critical failures emerge when software architectures treat statistical confidence, numerical formatting, or kinematic smoothness as an unconditional license to command hardware. Avoiding these hazards requires separating candidate generation from deterministic safety enforcement.

DRAFT

Fallacy: *An engineering team must choose entirely between formal classical safety proofs and learned empirical capability.*

Engineers often frame robot control as a mutual exclusion: either adopt a classical controller with provable stability guarantees, or deploy an expressive neural policy that achieves broad task coverage through empirical learning. Consider an assembly plant routing flexible wiring harnesses. A classical model-based controller enforces rigid kinematic constraints and workspace boundaries, yet it struggles to model the nonlinear dynamics of deformable cables. Conversely, a learned diffusion policy navigates cable deformation successfully, but occasionally proposes motions that exceed physical clearance limits.

The proposal boundary dissolves this dilemma. The diffusion policy proposes, and an independent minimal-intervention filter corrects any command that would cross a clearance limit before it reaches the actuators. The correction preserves the modeled safety constraint without guaranteeing completion of the harness insertion, so systems architects must track task completion, proposer error distribution, and protective intervention frequency as independent performance metrics.

↪ The minimal-intervention filter, which replaces an inadmissible command with the smallest correction that satisfies the physical constraints, is developed in section 12.7.

Pitfall: *Allowing a neural model's high confidence to bypass deterministic geometric boundaries.*

A cleanroom manipulator handling reagent vials mistakes specular glare on a transparent glass partition for open workspace and plans a reach directly through the barrier. The resulting action packet satisfies standard transport validation: joint coordinates are finite and in range, the command arrives on schedule, and the neural proposer reports an output confidence of $\hat{p} = 0.98$. However, a high score indicates only that one output dominated the alternatives. Section 3.6.2 showed that a network can be most confident on inputs far from anything it was trained on. The same specular reflection that deceived the perceptual front-end also corrupts the confidence head, producing an assertive proposal for an impossible motion.

Actuation permission must always derive from deterministic geometric validation rather than statistical self-assessment. Before energizing motor drives, the real-time execution monitor must validate candidate trajectories against qualified local geometric boundaries derived from independent

sensors, such as acoustic rangefinders, time-of-flight sensors, or calibrated structural keep-out zones. Because an isolated microcontroller or real-time checker lacks the memory and compute budget to evaluate a full host-resident 3D occupancy grid, the boundary handoff contract must deliver explicit spatial bounds along with verified validity intervals before motors receive current.

Fallacy: *Raw softmax probabilities or predicted confidence scores reflect true physical success rates.*

An industrial bin-picking policy can assign near-unity scores to grasp candidates that fail immediately under altered lighting or surface conditions. The softmax function merely transforms unbounded real-valued logits into an artificial probability distribution that sums to one across the discrete candidate set. This mathematical normalization quantifies relative preference across the model's training distribution rather than mechanical stability in physical contact.

Post-hoc calibration methods, such as temperature scaling, align predicted probabilities with empirical accuracy only while operating under conditions identical to the calibration distribution. As the illustrative gaps in table 3.4 depict, calibration can degrade sharply when lighting, surface friction, or geometry shifts out of distribution. Relying on an abstention threshold applied to raw model scores will silently allow unstable grasps or collisions during operational shifts. Production systems must backstop model proposals with independent physical verification, including tactile slip sensing, force-torque thresholds, and deterministic geometric checks.

Pitfall: *Coupling high-frequency semantic classifications directly to motor setpoints without continuous filtering.*

A delivery robot navigates past reflective tarps flapping over traffic cones. At every perception update (20 Hz), the vision pipeline alternates its discrete classification between an open corridor, a rigid obstacle, and a compliant drape. If the motion planner maps these discrete classification switches directly into motor setpoints, perceptual noise commands instantaneous velocity reversals. Each reversal is a step in velocity reference, which a PD servo loop turns into the derivative-gain torque step that the temporal ensembling jerk trap computed for a single chunk seam, repeated at every flip. Chattering commands of this kind saturate motor drive currents, cause mechanical backlash wear, and excite dangerous structural resonances throughout the chassis.

Physical architectures must decouple discrete semantic classification from continuous motor control. Engineers introduce temporal debouncing, Bayesian belief filters, or hysteresis thresholds to suppress semantic switching noise. Furthermore, the permission path's interpolator must hold setpoints within explicit acceleration and jerk limits before dispatching them to the drives. Even when the resulting trajectory appears smooth, the real-time execution engine must verify dynamic feasibility against hardware torque and thermal limits.

Fallacy: *A numerically valid and smooth action chunk can safely be executed whenever it arrives.*

Modern action-chunking architectures generate sequences of future setpoints to bridge the rate mismatch between slow neural inference on an application processor and fast motor control loops. Consider a mobile manipulator grasping an item from a moving conveyor. A host policy computes a smooth, multi-step arm trajectory toward the target, but operating system scheduling jitter, memory paging, or bus stalls delay packet delivery beyond the sensor evidence horizon. When the action chunk finally reaches the motor controller, its waypoints remain mathematically continuous and within joint kinematic limits. Meanwhile, conveyor motion has shifted the actual object outside the grasp envelope.

Executing an outdated chunk causes the arm to track a phantom path. A motor controller must reject stale packets regardless of their internal smoothness. Action chunks must carry explicit timestamps and temporal validity leases tied to the freshness of their underlying sensor observations. The real-time receiver may execute an active trajectory only while its lease remains valid. If an expected chunk fails to arrive before the lease expires, the controller must execute a qualified deceleration or compliant hold to keep the hardware within safe operational limits. Kinematic smoothness cannot compensate for expired temporal validity.

3.8 Summary

Learned representations give the mobile manipulator what hand-authored geometry could not, the ability to recognize and act on objects no engineer enumerated, and the machine pays for that breadth in memory traffic, arithmetic, and time. A fresh evaluation splits into a compute-bound prefill and a bandwidth-bound decode, so weight-only quantization shortens one term and leaves the other, and a large intent model on an embedded processor renews only a few times per second. Supply and validity are therefore separate budgets. A chunk keeps the arm supplied between renewals, but every setpoint in it descends from one observation, so the time the target stays within tolerance, not the length of the chunk, bounds how much of it may run. What the slow proposer hands the fast one decides what the permission path can check, and because a proposal can arrive on time and still be wrong, every proposal must carry its capture time, validity limit, and provenance so that it remains refusable. Six core quantitative relations govern these trade-offs across the chapter, each bounding a distinct stage of proposal generation and expiration (table 3.6).

Table 3.6: Key equations of the cognitive brain: The bounds that price a learned proposer on the machine's processor and size its chunks against a moving target, with the place each is derived.

Equation	What it bounds or decides	Derived
$f \leq \mathrm{BW}_{\mathrm{mem}} / M_{\mathrm{weights}}$	The memory wall: the peak renewal frequency when every evaluation cycle streams nonresident weights from memory before performing additional work	equation 3.2
$I = \mathrm{FLOPs/bytes\ transferred}$	Arithmetic intensity vs. the processor ridge point, determining whether a workload phase is bandwidth-bound or compute-bound (batch-1 decode sits near one FLOP per byte; prefill approaches S FLOPs per byte)	equation 3.3
$t_{\mathrm{infer}} \approx \max\left(\frac{O_{\mathrm{prefill}}}{\eta_{\mathrm{hw}} R_{\mathrm{peak}}}, \frac{M_{\mathrm{weights}}}{\mathrm{BW}_{\mathrm{mem}}}\right) + (S_{\mathrm{out}} - 1)\frac{M_{\mathrm{weights}}}{\mathrm{BW}_{\mathrm{mem}}}$	The latency of a complete evaluation, combining compute-bound prefill with bandwidth-bound decode sweeps; weight-only quantization reduces only the second term	equation 3.4
$M_{\mathrm{KV}} = 2 \times N_L \times d \times S \times s_{\mathrm{elem}}$	The memory capacity occupied and reread by retained context, scaling linearly with the preserved token sequence length	equation 3.1
$h_{\mathrm{min}} = \left\lceil (T_{\mathrm{chunk}} + t_{\mathrm{infer}}) / \Delta t_{\mathrm{step}} \right\rceil$	The minimum setpoint count an action chunk must deliver to keep actuators supplied across a complete replanning and inference cycle	equation 3.6
$\tau_{\mathrm{ev}} = \sqrt{2(r_{\mathrm{tol}} - e_0)/a_{\mathrm{slip}}}$	The evidence horizon during which target tracking remains within physical tolerance; the total latency span $T_{\mathrm{chunk}} + t_{\mathrm{infer}}$ must fit inside this limit	equation 3.5

Key Takeaways: Learned proposers and deliberation limits

- **Memory traffic and arithmetic both bound fresh inference**: When an evaluation cycle streams nonresident weights from memory, the total weight footprint and sustained memory bandwidth establish a hard latency floor. Fetching the intent model's 14 GB of FP16 weights requires 98.0 ms. Input token prefill operates in the compute-bound regime, whereas per-token autoregressive decoding is bandwidth-bound. Consequently, even with INT4 weights, a single evaluation consumes approximately 336 ms. This supports the selected intent renewal rate of 2 Hz, which is far too slow to drive real-time actuator chunk renewal directly.
- **Chunking supplies motion without refreshing evidence**: Generating a chunk of 16 setpoints amortizes a single backbone evaluation across 400 ms of candidate motion. Although this amortizes memory bus traffic across waypoints, the robot executes that motion without incorporating new visual evidence until the subsequent policy evaluation completes.

- **Supply and validity bound different horizons**: In the conveyor pick task, actuator continuity requires a minimum supply horizon of 4 setpoints to bridge neural renewal intervals, whereas target drift constrains the sensory evidence horizon to approximately 219 ms. A deeper queue of setpoints cannot extend that validity. A setpoint stays eligible only while the evidence behind it holds and the active chunk lease runs, and when renewal stops, the chunk lease bounds how far the arm continues.
- **Retained context competes with current sensing**: Retaining the hypothetical five-second history at the stated sampling rate occupies 21.0 GB of key-value cache memory. Because storing and rereading past attention state directly contends for DRAM bandwidth with incoming sensory observations and proposal generation, temporal context retention must be explicitly managed within the system timing and memory budget.
- **Proposer interfaces dictate runtime verifiability**: An opaque latent embedding passed from a high-level intent model to a low-level policy cannot be cross-referenced with measured workspace geometry or aged against target drift. Conversely, an explicit Cartesian target pose accompanied by an observation timestamp provides a checkable contract. This auditable geometric interface allows the machine's permission path to enforce its intent lease alongside the high-rate chunk lease.
- **Reported confidence requires independent physical validation**: A softmax probability or normalized confidence score emitted by a learned model cannot guarantee physical execution success under real-world distribution shift. While calibration validates score reliability across known operating regimes, runtime safety demands independent geometric constraints, kinematic limits, and physical contact monitoring.
- **Proposals must remain rejectable**: Trajectory proposals must bundle explicit observation timestamps, kinematic assumptions, and validity envelopes alongside their raw setpoints. The Nervous System combines these proposer constraints with real-time body state measurements to validate trajectory headers, manage handoffs between successive chunks, and enforce fail-safe execution windows.

The conveyor pick separates two kinds of inadequacy that call for different fixes. A proposer that cannot renew before its evidence expires has design remedies: fewer input tokens, a smaller or differently quantized model, more bandwidth, or a slower target. A proposer that renews in time and still proposes a wrong grasp is not repaired by any of them, which is why its output stays a proposal however fast it arrives. Both cases end at the first law (principle 1). Motion executed on evidence that has gone stale, or toward an affordance that does not exist, delivers energy that cannot be recalled. The time a proposal may keep driving the machine therefore has to be set below the proposer, from the body's stopping budget, rather than by the proposer's own schedule.

3.9 Historical Perspective and Further Reading

The two-speed architecture developed throughout this chapter, which couples a slow deliberative proposer with a high-frequency execution loop that continuously services the actuators, is robotics' recurring answer to a timing constraint. Physical machines have wrestled with this division since Shakey's deliberative planning (Nilsson 1984), Brooks' reactive subsumption layers (Brooks 1986), and ALVINN's direct learned steering (Pomerleau 1989). The classical industrial pipelines examined in section 3.2 retained explicit geometric representations by restricting robot operation to structured, calibrated workcells where rigid kinematic models hold, while end-to-end learning gained broad behavioral coverage from data at the cost of formal analytical guarantees.

↩ The lineage from Shakey's deliberative planning through Brooks' subsumption layers to ALVINN's learned steering is established in section 1.2.

The hardware bottlenecks that govern modern policy inference stem from a parallel evolution in computer architecture. Wulf and McKee (1995) identified the widening memory wall, observing that processor speed was improving far faster than DRAM access time. Williams et al. (2009) formalized this relationship into the roofline model, establishing operational arithmetic intensity, the ratio of floating-point operations performed to memory bytes transferred, as the metric that reveals whether an execution phase is memory-bandwidth-bound or compute-bound. Modern autoregressive models collide directly with these bandwidth ceilings. The Transformer architecture requires every newly generated token to attend over cached keys and values from all preceding context tokens (Vaswani et

al. 2017). As Pope et al. (2023) demonstrated through analytical modeling of distributed Transformer inference, generating one token at a time splits execution into a compute-bound prompt prefill phase, which processes all input tokens together, and a memory-bandwidth-bound autoregressive decoding phase. In embodied setups, where real-time interactive control requires generating action tokens with a batch size of 1, the memory wall dominates the decode, as notebook 3.1 prices for the machine and checks against a published measurement on its processor. To alleviate this memory traffic, integer-only quantization demonstrated that low-precision arithmetic could run efficiently on mobile processors with small accuracy loss on the vision networks it evaluated (Jacob et al. 2018). As the intent-model measurements of table 3.1 show, however, the cost of a given bit width depends on the method, the kernels, and the task. Modern weight-only formats such as AWQ protect salient weight channels while cutting the bytes transferred across the memory bus on each decoding pass (Lin et al. 2024).

Learned proposers emerged when these sequence architectures were adapted to robot sensorimotor trajectories. The Vision Transformer demonstrated that self-attention could replace convolutional feature extractors on raw image patches without losing spatial coherence (Dosovitskiy et al. 2021). RT-1 and RT-2 extended this foundation to robotics by tokenizing continuous physical actions directly into the vocabulary of large vision-language models (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), while OpenVLA established an open, reproducible seven-billion-parameter architecture that enabled systematic hardware profiling and fine-tuning (Kim et al. 2024). However, evaluating large transformer backbones autoregressively at every single control step proved computationally impractical for low-latency physical loops. Action Chunking with Transformers (ACT) and Diffusion Policy resolved this bottleneck by shifting the unit of prediction from isolated steps to short trajectories (Zhao et al. 2023; Chi et al. 2025), amortizing inference latency over multiple timesteps and reducing the compounding of per-step errors. Advancing this approach, π_0 coupled a high-capacity vision-language backbone with a flow-matching action expert capable of generating high-frequency motor commands (Black et al. 2024). Real-time action chunking hid the inference delay behind execution by freezing the setpoints that run during inference and generating the rest of the next chunk to agree with them (Black, Galliker, et al. 2025), a formulation that recasts classical receding-horizon model predictive control through the lens of deep learning (Mayne et al. 2000). Modern embodied platforms, including Helix (Figure AI 2025) and GR00T N1 (Bjorck et al. 2025), adopt the same two-speed separation. The contract between the two tiers remains an open design question. Deployed systems pass latent vectors, language, or geometry from the slow model to the fast one (table 3.2). This chapter's case for a checkable geometric target, together with the invariants a real-time monitor must enforce on it, is one position in that question rather than a settled standard.

Six works trace this lineage from reactive robotics to modern vision-language-action architectures:

- Brooks (1986) introduces the subsumption architecture, organizing autonomous robots into layered, asynchronous behaviors that each close their own sensorimotor loop. The paper presents the foundational systems argument that fast reactive loops must never block on deliberative planning, establishing the conceptual basis for modern two-speed execution.
- Williams et al. (2009) introduces the roofline model, a visual bound relating arithmetic intensity to processor compute and memory bandwidth ceilings. The paper provides the standard methodology for diagnosing whether an inference kernel is memory-bound or compute-bound before undertaking hardware optimization.
- Pope et al. (2023) models Transformer inference from first principles, demonstrating how batch size, sequence length, and tensor partitioning dictate whether execution binds on memory bandwidth or arithmetic execution units. This analysis underpins the performance characterization of batch-size-1 policy inference explored throughout this chapter.
- Zhao et al. (2023) introduces Action Chunking with Transformers (ACT) and temporal ensembling for precise bimanual manipulation. The authors demonstrate that predicting action sequences rather than individual steps dampens compounding tracking errors and stabilizes learned policies, establishing the trajectory chunking approach that real-time learned policies now rely on.
- Chi et al. (2025) develops Diffusion Policy, demonstrating why multimodal human demonstrations with multiple valid solutions require expressive generative decoders rather than

unimodal regression. The paper explicitly characterizes the systems cost of iterative refinement, quantifying the computational latency required to evaluate multiple diffusion steps per action chunk.

- Kim et al. (2024) provides a complete, open-source seven-billion-parameter vision-language-action model, detailing its training pipeline, data mixture, and deployment trade-offs. The paper includes 4-bit and 8-bit quantization benchmarks, which make it a common reference point for profiling learned embodied policies on edge hardware.

What's Next: From learned proposals to the permission path

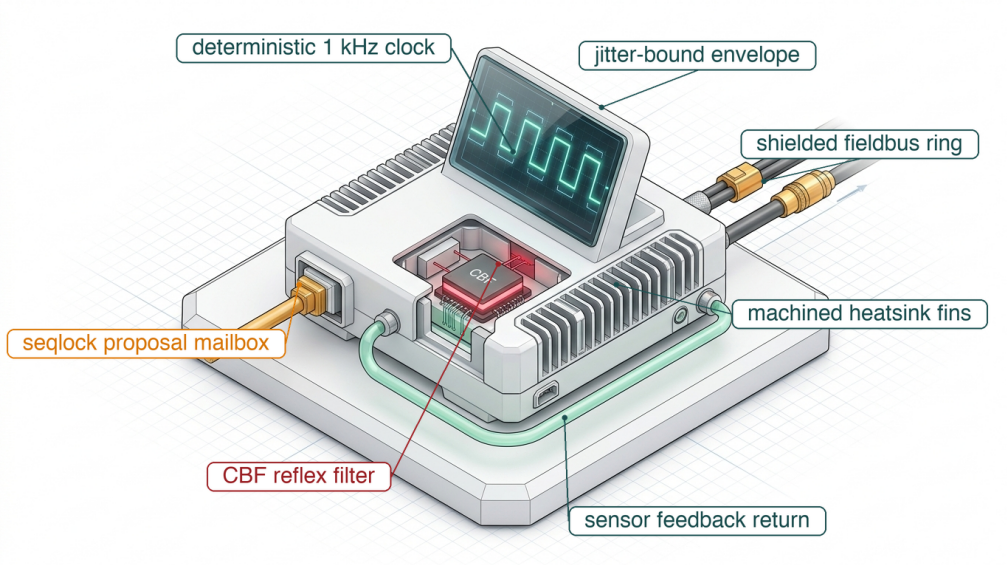
A physical machine cannot allow a candidate proposal to continue driving hardware indefinitely when its successor fails to arrive. The mobile manipulator's chunk policy targets a proposal period of 50 ms, whereas the intent model updates at a slower cadence, and either level can stall, miss its deadline, or misjudge the target object. A check that waits on the proposer it checks stalls along with it. Chapter 4 therefore begins with where the permission path must run to stay independent of the learned models. It next derives each tier's rate from a physical time constant and passes proposals between tiers without letting a slow writer block a fast reader. Only then does it price the chunk lease from the body's stopping budget and the proposer's renewal floor, ensuring that an admitted chunk commands the joints for a bounded interval before a deterministic fail-safe response begins.

DRAFT

DRAFT

4

The Nervous System

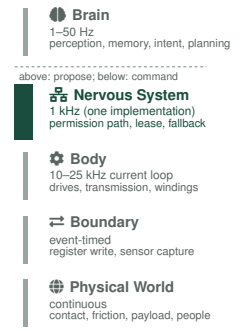


4.1 The Multi-Rate Bridge
4.2 The Independence Requirement and the Two-Processor Hypothesis
4.3 Multi-Rate Cadences
4.4 Chunk Lease Sizing from the Stopping Budget
4.5 Real-Time Command Transport and Bus Determinism
4.6 Actuation Authority
4.7 Hardware Isolation and Fault Containment
4.8 Invariant Checking
4.9 The Handoffs Matrix
4.10 Fallacies and Pitfalls
4.11 Summary
4.12 Historical Perspective and Further Reading

Purpose

Why must a moving machine keep checking its commands when its learned model stops responding?

A learned model can pause for a fraction of a second while it interprets a scene, but the machine carrying it keeps moving, and a loaded base covers tens of centimeters in that pause. Every delay between a decision and the current it becomes is therefore paid in clearance. A general-purpose computer hides such delays behind queues, retries, and priorities. A moving machine cannot, because a command that arrives late acts on a body that has already moved, and a check that waits on the model it checks protects nothing once that model stalls. Treating timing and communication as implementation details produces machines whose safeguards share a processor, a supply rail, or a memory bus with the model they guard, and so fail together with it. A machine that acts on learned decisions therefore needs a timed path between proposal and actuator whose delays are bounded, whose checks never wait on the proposer, and whose grant of authority to each proposal expires on its own. At the causal boundary, time granted to a proposal becomes motion that software cannot recall.



Learning Objectives

- Explain why the permission path's independence from the proposer is a requirement to be demonstrated rather than a property of separate processors
- Derive update rates and loop placement from actuator time constants, the phase lag of sampling and transport delay, and proposal renewal limits
- Evaluate mailbox exchanges for bounded reads, complete payloads, and proposal freshness when the writer stalls
- Size a chunk lease between the renewal floor and the pre-brake ceiling, and explain why a host heartbeat cannot renew it
- Distinguish measured latency percentiles from response-time bounds that include computation, blocking, and interference
- Compare position, velocity, torque, and impedance commands by what each leaves the permission path to check
- Evaluate interface timing violations using handoff records that distinguish allocated budgets, supporting evidence, and unresolved bounds

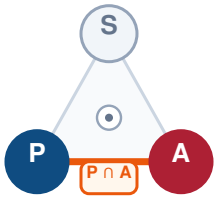
4.1 The Multi-Rate Bridge

The running machine joins two computational regimes that run on clocks roughly three orders of magnitude apart. A deep neural network processes camera frames into action proposals over tens to hundreds of milliseconds, with latency that fluctuates from frame to frame. An electric motor drive, by contrast, regulates winding current every few tens of microseconds, in a loop that can never pause because the mass it moves keeps its momentum whether or not a new command arrives. The architectural layer between them, the machine's **Nervous System**, routes action proposals downward toward the actuators, streams physical telemetry back up, and arbitrates whether any proposal is permitted to move the hardware. Industrial motion control, avionics, and automotive engineering solved similar coordination problems decades ago for classical feedback controllers whose worst-case execution times were statically bounded before deployment (Kopetz 2011). Learned proposers break those classical assumptions. Their inference latencies have long tails, their software runtimes can stall unexpectedly, and their outputs can fail in ways that cannot be exhaustively enumerated at design time.

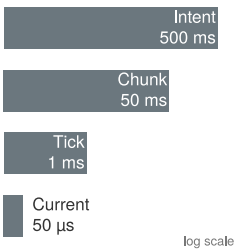
This asymmetry sets the chapter's central design question, which chapter 3 left open with two proposers (a chunk policy and an intent model, either of which can stall, miss its deadline, or misjudge the target). The question is how long the physical plant may keep executing the last valid proposal when an inference step stalls or drops a frame. That duration cannot be an arbitrary software timeout or the proposer's own schedule; it must follow from the machine's stopping distance and mechanical limits.

On the warehouse mobile manipulator, that gap appears every time the vision-language-action (VLA) policies plan a grasp on a conveyor-belt mug while the brushless motor drives regulate winding current thousands of times per second. If motor regulation ever blocks on deliberative reasoning, or contends for shared memory without a bound on its wait, a heavy inference burst can push the control loop past its hard deadline. Moving mass and electromagnetic coils force one conclusion. *The Brain needs time to deliberate, while the Body needs continuous control.*

Coupling these domains across a shared blocking execution path forces the fast loop to inherit the tail latency of the slow one. If the motor control loop stalls waiting for inference or drains an outdated FIFO queue of stale setpoints, the resulting actuation commands arrive at a physical body that has already moved past the target state. Unmanaged transport delay (T_d) also injects phase lag directly into the feedback loop, adding $\Delta\phi = \omega T_d$ radians at angular frequency ω. That lag erodes the loop's phase margin, the extra lag it can absorb before its corrections start to reinforce the motion instead of damping it (made precise in section 4.3.1). The Nervous System must bound these end-to-end transport delays deterministically while ensuring that high-rate local control remains autonomous whenever high-level deliberation stalls.



On the sense-plan-act (S·P·A) triad (Appendix D), proposals cross from Plan to Act only through the permission path.



Each tier runs at least ten times faster than the one above it, and none may wait on a slower one.

To enforce this isolation, the Nervous System splits work across four tiers, separating what each tier controls alongside how often it runs. Each tier runs decoupled from the tiers above it, so fast safety and stabilization loops never wait on slower deliberative tiers:

1. **Actuator current and velocity regulation:** Dedicated motor drives close the inner winding-current and joint-velocity loops locally in hardware. Field-oriented control (FOC)[1] aligns the stator voltage vector with the rotor's magnetic flux in every carrier period (the switching period of the drive's pulse-width modulation, or PWM). The winding then produces the commanded torque within a few current-loop samples.
2. **Permission and position supervision:** The deterministic real-time supervisory controller verifies proposal freshness and kinematic limits, sends admitted position setpoints across the fieldbus, checks tracking error against physical safety envelopes, and commands a fail-safe stopping trajectory if proposal authority expires.
3. **Action-chunk policy:** The learned mid-level policy emits a short sequence of future waypoints (an action chunk), giving local controllers a continuous motion segment to track between discrete inference steps.
4. **Intent planning:** Slower vision-language or task-planning models read camera frames and language instructions to set high-level goals, steering the action-chunk policy without directly touching drive setpoints.

[1] **Field-Oriented Control (FOC)**: The drive rotates the three measured phase currents into a frame that turns with the rotor's magnetic field, where torque and flux become two slowly varying currents that ordinary proportional-integral loops can hold. The rotation needs a fresh rotor angle in every carrier period, so the loop cannot wait on anything slower than its own sensors. That is why the current loop runs inside the drive, beside the encoder and the current shunts, and why no proposer, however fast, ever sits inside it.

Section 4.3.1 gives each tier its rate, while one rule holds all four together and comes first: a stalled or crashed proposer must never block or delay a lower tier's checks. Conventional computing architectures break that rule because they rely on shared processors, unified memory buses, and multi-level caches managed by general-purpose operating systems. In such environments, hardware isolation is weak. When a large deep neural network streams gigabytes of model weights from DRAM across a shared memory fabric, its high-throughput direct memory access (DMA) bursts contend directly with real-time telemetry and control packets. Because hardware bus transactions cannot be preempted by software thread priorities, control packets queue up at the memory controller. A control path that shares an execution environment with machine learning runtimes is also exposed to operating-system jitter, because kernel page faults, garbage collection cycles, driver interrupts, and thermal frequency throttling can each stall execution for tens of milliseconds. Physical stability therefore demands an architecture in which local control loops keep updating the actuators on schedule whatever delays the inference path suffers.

The physical numbers show why the fast tiers cannot wait. An electromagnetic motor winding has an electrical time constant ($\tau_e = L_{\text{phase}}/R_{\text{phase}}$) of a few milliseconds, 2.7 ms for the running machine's direct-drive hub motors (section 2.4.2), while the moving links carry substantial kinetic energy ($E_k = \frac{1}{2}mv^2$). Delaying or dropping an actuation command by a fraction of a second can exhaust the clearance before a new inference cycle finishes. On the warehouse robot the mismatch appears in two mechanical domains:

- The base (Class 1) cruises at up to its 1.5 m/s drive limit (illustrative; see the Reader Guide). At this speed, it travels 60 mm during a single 40 ms action-chunk inference, and 503 mm across one 336 ms intent-model inference cycle. Unless an independent supervisory process monitors plant telemetry and commands braking, the chassis travels half a meter open-loop before deliberative planning produces an updated command, nearly half the rack-end clear distance on the machine card.
- The arm (Class 2) couples motor rotors to joints through high-ratio gearboxes, reflecting rotor inertia back to the joint output as $N^2_{\text{gear}} J_{\text{rotor}}$, the scaling derived in section 2.4.3. The drives keep closing their own joint loops when the proposer stalls, but they keep tracking the last admitted command. If that command was set for a grasp on a conveyor mug or a turn of a door latch the scene no longer matches, the reflected inertia and the drives together keep pressing the mechanism into the contact. Contact forces then escalate rapidly. Whether this interaction breaches structural limits depends on the contact compliance, surface friction, and mechanical response times alongside the reflected inertia.

↩ The intent model's inference latency, priced from its compute-bound prefill and its bandwidth-bound weight sweeps, is established in section 3.4.1.

Software discipline, operating system thread priorities, and runtime exception handlers cannot by themselves keep the plant safe. Safety requires an explicit, physically deterministic computing, memory, and communication fabric. That fabric forms the Nervous System of the five-level machine

model (section 1.7), hosting the dedicated permission path so the physical plant retains a measured, feasible response whenever higher-level inference pauses or fails. Its primary design requirement is an interface whose temporal bounds, command authority, and fault responses can be verified directly against the machine's physical safety budget. Enforcement algorithms later turn these requirements into per-tick admission decisions, while the mechanisms of this layer fix the contracts those decisions must satisfy.

Building and pricing that fabric proceeds from silicon isolation to the system handoff record. Section 4.2 shows why the timed path to the actuators must remain independent of unbounded interference and stalls from deliberative inference, stating the two-processor split that meets this requirement as a hypothesis the machine's silicon must still confirm under load. Section 4.3 derives the rate of each tier from a physical time constant and passes proposals between tiers through a mailbox that never blocks the faster reader. Section 4.4 then answers the chapter's central question with the *chunk lease*, the bounded interval during which an admitted proposal may keep driving the actuators, priced in the stopping budget. Section 4.5 bounds the computation and the drive network (the *fieldbus*) on which that price depends. Because bounded timing still requires exclusive control of the actuators and survival under host faults, section 4.6 and section 4.7 keep the permission path the sole writer of the drives and keep it alive when the proposer fails, while section 4.8 states the execution contract its checks must meet however fast the proposer runs. Finally, the handoff record of section 4.9 locates every limit at one of five interfaces, coupling this layer's lease to the stopping distance of the Body and the renewal rate of the Brain.

↪ The per-tick admission decisions that must satisfy these contracts are developed in section 12.1.

4.2 The Independence Requirement and the Two-Processor Hypothesis

When an inference model suffers a transient stall or drops a frame, the robot can safely reject the stale proposal and fall back to local control. When a permission check stalls, however, the delay burns directly through the machine's stopping budget, regardless of whether the processor eventually catches up. The two workloads therefore optimize for different guarantees. A neural chunk policy needs high compute throughput and memory bandwidth to stream motion proposals at its target renewal rate, whereas the permission path needs a strict, provable upper bound on the time required to validate and dispatch each step.

Standard operating-system schedulers decide which ready task executes next, but software priority alone cannot guarantee deterministic timing. Raising a control task's priority reduces processor contention, yet it cannot prevent microarchitectural delays. A sudden page fault suspends execution while the virtual memory subsystem resolves the missing page. Lock contention on shared kernel primitives leaves a real-time thread waiting behind uncritical background tasks. Memory requests stall at the arbiter while DMA transfers from a GPU or neural accelerator saturate the system interconnect. Long inference latency tails[2] therefore threaten closed-loop stability even when average proposal latency remains well within nominal bounds.

Bounding the permission tick against those microarchitectural stalls requires separating the parts of its cycle using the standard terms of real-time scheduling (Buttazzo 2011; Kopetz 2011). The permission tick runs as a periodic task i released once every **period** T_i, set here to 1 ms. Each release is a single job that must finish before its relative **deadline** D_i, measured from the release instant and set here to 400 µs. The deadline is shorter than the period so that each admitted setpoint is ready within the cycle that computed it rather than one tick later, a delay whose cost section 4.3.1 prices. It also leaves the rest of the period free for lower-priority work. A deadline is *hard* when a single miss can cause a physical hazard, as on the permission tick, where one late check adds one more millisecond of unchecked motion. The total wall-clock time from task release to job completion is its **response time** R_i. The part of that response time spent running the task's own instructions is bounded above by its **worst-case execution time (WCET)**[3] C_i.

> Definition 4.1: Worst-case execution time
>
> ***Worst-case execution time*** is the longest time a task can take to run to completion on a given processor, over every admissible input and every state its caches, pipeline, and buses can be in, with no other task competing for the processor.

[2] **Inference Latency Distribution**: The median and p99 alone do not fix the exceedance probability at a given deadline, which must be measured or modeled separately. Tail-model assumptions and exceedance estimates require representative workload data and an account of dependence and uncertainty; section A.10 develops these conditions.

[3] **Worst-Case Execution Time (WCET)**: A static WCET is a bound proved over every path through the binary and every state the processor's caches and pipeline can be in; analyzers compute it by abstract interpretation of the code against a model of that hardware (Wilhelm et al. 2008). Caches, branch predictors, and shared buses make that model either pessimistic or intractable, so a timed path earns a tight bound by avoiding them. The bounded sources listed in section 4.5.1 therefore favor tightly coupled memory and fixed pipelines, giving up average speed for a bound that analysis can prove.

1. **Significance**: It is the C_i term of every response bound in this chapter, and every margin above it is sized on it. The permission tick declares 250 µs against an unloaded run of 135 µs. The 115 µs between them is a claim that analysis must justify before the 400 µs deadline, and the stopping budget behind it, can rely on the number.
2. **Distinction**: Unlike a mean or a high percentile such as p99, which describe the executions a test happened to reach, the WCET covers every admissible path and processor state. A declared WCET is a third quantity, a number the design commits to, which evidence must show is no smaller than the true one.
3. **Common pitfall**: Recording the largest measured execution time as the WCET. A measured maximum can refute a declared bound but never establish it, because a test campaign reaches only some of the processor states in which the code can run (Wilhelm et al. 2008).

The rest of the response time is spent waiting rather than running. **Blocking** B_i is the time a task spends waiting for a lower-priority task to release a shared hardware or software resource, whereas **interference** I_i is the time taken by higher-priority tasks, kernel interrupts, and exception handlers that preempt the job. Because blocking and interference vary from one release to the next, the exact timing of recurring events shifts from cycle to cycle, a variation called **jitter**. Task releases, sensor samples, and job completions each have their own jitter, so a tick released on schedule can still miss its completion deadline if its internal wait times fluctuate. Later, section 4.5.1 combines C_i, B_i, and I_i into a provable bound on R_i.

Placed on one axis, the terms separate an illustrative unloaded observation from the bound the schedule assumes (figure 4.1). The unloaded run of 135 µs is an illustrative observation, not a reported measurement. The 115 µs above it, up to the declared 250 µs, is a claim that WCET analysis must certify. The slack from there to the 400 µs deadline is all the room left for blocking and interference. Underneath, the base covers 1.5 mm during every tick at its 1.5 m/s drive limit, so each of these microsecond terms is ultimately paid in millimeters.

Blocking introduces an architectural vulnerability that priority schemes alone cannot resolve. In **priority inversion**, a high-priority task waits for a shared resource held by a low-priority task, while medium-priority tasks that need no shared resource preempt the lock holder and so starve the high-priority task for as long as they run. Only a real-time locking protocol, such as priority inheritance or priority ceiling emulation, can bound that wait. The recurring resets of the 1997 Mars Pathfinder spacecraft are the canonical case of an inversion left unbounded; war story 4.1 in section 4.5.1 traces it once the response-time bound it violated is in hand.

POSIX real-time extensions (such as `SCHED_FIFO` priority scheduling and memory locking via `mlockall()`), real-time Linux kernels (`PREEMPT_RT`), and dual-kernel architectures (such as Xenomai) target specific software sources of delay. Locking physical memory via `mlockall()` eliminates demand paging inside the timed control loop. Assigning processor affinity confines threads to specific CPU cores to avoid migration latency and cache thrashing. Priority inheritance elevates a low-priority lock holder's priority whenever a high-priority task requests its mutex. These operating-system mechanisms, however, guarantee determinism only when supported by proper configuration, bounded system calls, and verified timing evidence.

↪ The scheduling, interrupt, and memory mechanisms whose formal contribution enters a response bound are developed in section C.3.

Shared hardware introduces delays that operating-system software priorities cannot control. A control task cannot read an optical encoder buffer while a high-bandwidth peripheral monopolizes a shared PCI Express (PCIe) link or system memory bus. Any physical infrastructure that two processing domains share, whether system memory, power regulation, a thermal path, or a reference oscillator, can delay or silence the permission path without a single software fault. Cataloging those physical couplings and gathering the evidence required to verify their isolation belongs to placement (section 13.2). The requirement that analysis tests can be stated now. Under any workload the proposer can generate, the permission path must meet its response bound and keep its authority to refuse an unsafe motion. This chapter calls that requirement independence.

The running machine meets the independence requirement by putting the two paths on two processors, a hypothesis it must still test (figure 4.2). An application processor runs perception, the 2 Hz intent model, and the 20 Hz chunk policy. A separate safety microcontroller unit (MCU) runs the

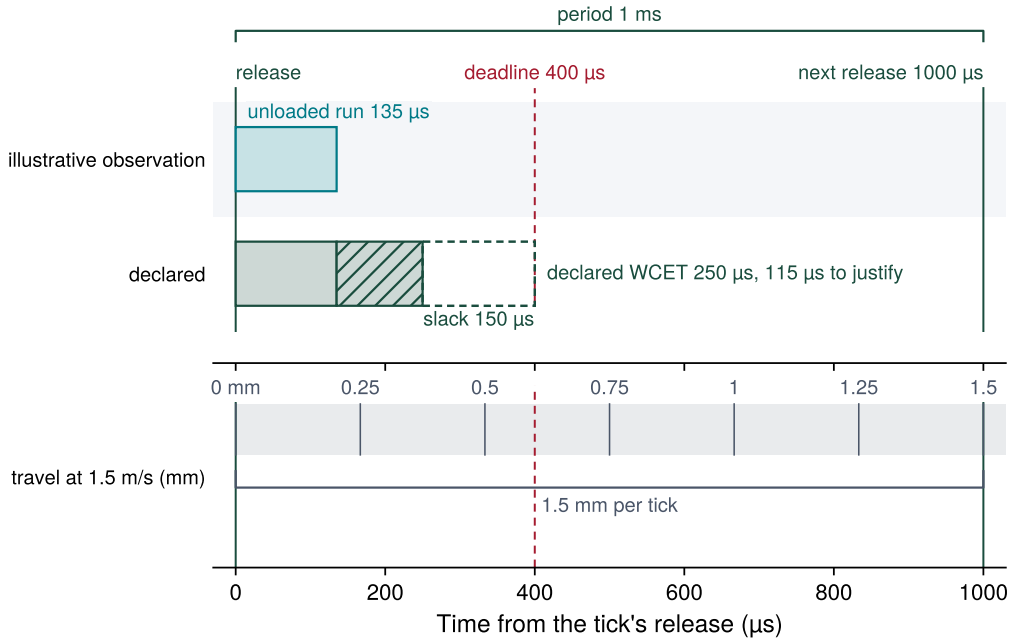


Figure 4.1: One permission tick: A job is released at the start of each 1 ms period and must finish by its 400 µs deadline. The unloaded run of 135 µs is an illustrative observation, not a reported measurement; the declared WCET of 250 µs is the bound the schedule assumes, and the hatched 115 µs between them is what WCET analysis must justify. The slack to the deadline is the budget for blocking and interference. The bottom row counts the base's travel on the same axis, 1.5 mm per tick at the 1.5 m/s drive limit.

permission path and alone drives the motors. Between them runs a board link, a serial peripheral interface (SPI) or Controller Area Network with Flexible Data-rate (CAN FD) bus carrying framed packets with cyclic redundancy checks (CRCs) and sequence numbers. The link is treated as a *black channel*, one the endpoints do not trust, so each frame carries its own integrity and timing checks (section 4.4.1). The two paths share no processor cores, caches, or memory buses, and the learned proposers on the host have no memory-mapped input/output (MMIO) access to the drive registers. Candidate chunks cross the link into the MCU's own SRAM through a double buffer. A receive DMA transfer fills a private receive buffer. The receive interrupt copies each checked frame into a single mailbox slot guarded by a sequence lock, or *seqlock*, whose protocol section 4.3.2 develops. A 1 kHz permission tick reads that slot, checks each proposal's freshness, lease, and kinematic limits against measured encoder feedback, and is the only code that can send an admitted setpoint to the drives. The host is an unprivileged client; the microcontroller, with a short and cycle-counted loop, is the part whose worst-case execution time can be bounded.

Physical separation onto separate chips represents one realization. A single system-on-chip can also host both paths, provided that on-chip interconnects, cache partitioning, and memory controllers demonstrate equivalent response bounds and fault containment under stress. Choosing between discrete silicon and partitioned on-chip cores is the central placement trade-off, and until loaded experiments settle it, physical separation remains a foundational design hypothesis. Scheduling both paths on shared processor cores introduces the problem of *mixed-criticality scheduling*, in which each task carries execution-time bounds of graduated assurance levels. The operating system guarantees each deadline only under the bound its own criticality demands, which allows the proposer to overrun while the permission path executes deterministically (Vestal 2007). Sharing memory on a single die also demands explicit memory-bandwidth regulation to prevent tensor accelerators from starving real-time safety routines.

↪ Mixed-criticality scheduling, memory-bandwidth regulation, and the other remedies for shared-die contention are developed in section 13.6.

Designating an embedded controller as a *safety microcontroller* claims far more than physical separation on a circuit board. It denotes a device developed, qualified, and assessed under the

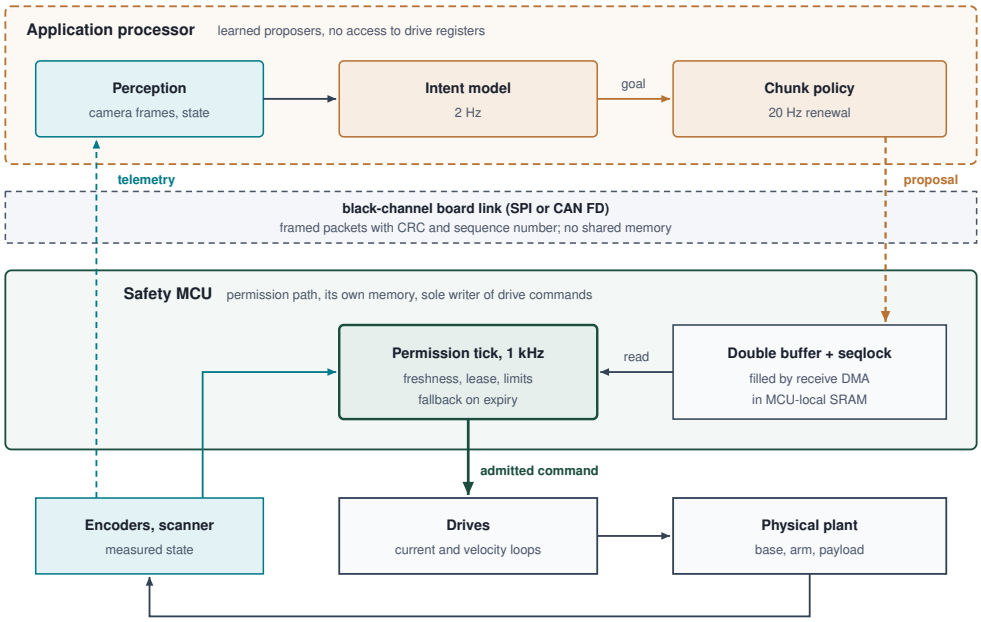


Figure 4.2: The two-processor implementation: The application processor runs the learned proposers and has no access to the drive registers. Proposals cross a black-channel board link as framed packets into a double buffer in the microcontroller's own memory. The receive DMA fills a private receive buffer, and the receive interrupt copies each checked frame into the seqlock-guarded mailbox slot, so the processors share no memory. The permission tick reads the latest proposal, checks it against measured state, freshness, and the lease, and sends only an admitted command to the drives, which close their own current and velocity loops. Measured state returns to the application processor as telemetry over the same link.

IEC 61508 functional-safety standard, accompanied by a safety manual specifying its certified integrity level and the operational conditions that support depends on (International Electrotechnical Commission 2010b). At the silicon level, a safety MCU integrates specialized hardware redundancy to detect internal faults autonomously. Dual processor cores execute in lockstep while a comparator flags divergent execution. Error-correcting codes (ECC) protect internal SRAM and flash memories against single-event upsets. A hardware memory protection unit (MPU) confines each task to its assigned memory range. Analog monitors continuously watch the clock and supply voltages. Built-in self-tests run at power-up and periodically during operation to verify internal logic. An external watchdog timer, clocked by its own oscillator and serviced within a strict time window or through challenge-response messages, monitors the core. Because the watchdog relies on an independent clock source, a physical fault that stops the microcontroller's oscillator cannot stall the watchdog, allowing it to detect that silent processor failure and command a qualified out-of-band response. Successful physical containment still depends on the response path, its surviving power, and the actuator premises.

Two different protections meet at this microcontroller, and keeping them apart explains the standards that follow. The chunk lease protects the stopping budget against a proposer that stalls or misbehaves. The rated claim that the machine will not strike a person rests elsewhere, on the independent protective path of section 1.4.3 (a certified safety scanner wired to the drives, with its own sensing, timing, and power) and on the drives' certified stop functions. Hardware diagnostic features alone do not confer a safety rating on the robot. Under functional safety standards, integrity belongs to a specific safety function, such as revoking actuator authority when a lease expires, rather than to an individual chip. The industrial standard IEC 61508 classifies safety functions by Safety Integrity Level (SIL 1 through SIL 4), where each numerical increment demands approximately an order-of-magnitude reduction in the allowable rate of dangerous failure (International Electrotechnical Commission 2010a). The machinery standard ISO 13849-1 grades safety functions by

Performance Level (PL a through PL e) (International Organization for Standardization 2023a). A Performance Level combines four architectural factors. The structural Category ranges from single-channel architectures using well-tried components (Categories B and 1), through single channels with periodic diagnostic testing (Category 2), to dual-channel redundant designs in which no single hardware fault defeats safety enforcement (Categories 3 and 4). Component reliability is quantified as the mean time to dangerous failure. Diagnostic coverage defines the fraction of dangerous failures detected by internal testing. Physical defenses guard against common-cause failure, where a shared disturbance disables redundant channels simultaneously. The safety microcontroller contributes certified failure rates and diagnostic coverage to this calculation, while the target Performance Level is dictated by the severity and exposure of the physical hazard the safety function must mitigate.

↪ The Performance Level each certified stop and speed function of the running machine must reach is developed in section 12.9.3.

The physical architecture of the robot makes a deliberate engineering trade-off that functional safety certification must evaluate. Its permission path executes real-time performance tasks, validating trajectory proposals and interpolating admitted waypoints into drive setpoints, alongside the safety function intended for qualification: revoking actuator authority and commanding a safe stop when a chunk lease expires or the host heartbeat falls silent. The algorithmic admission checks themselves carry no formal safety rating, which is why the protective claim stays with the scanner and the drive stop circuits (section 12.9.3). Standard functional-safety practice keeps complex control algorithms out of the certified safety channel, relying instead on an independent monitoring channel to satisfy Category 3 and Category 4 redundancy requirements. Modern motor drives supply part of that second channel through integrated hardware safety functions such as Safe Stop 1 (SS1), which brakes the motor under drive control and then removes torque.

↪ How Safe Stop 1 and the other drive safety functions realize the industrial stop categories is developed in section 12.9.2.

Any hardware split must keep the permission path the sole writer of the actuators and bound its check time across all expected hardware and software faults. Putting the two paths on separate chips is the cleanest way to start, though empirical timing evidence under load must still confirm that isolation. Separate processors, however, run on asynchronous clocks at very different rates, so the link between them must pass trajectory updates continuously without ever forcing the fast permission reader to wait on the slower host writer.

4.3 Multi-Rate Cadences

A motor drive cannot wait for the next camera-based proposal before regulating winding current. Conversely, forcing a learned policy to evaluate at current-loop rates wastes compute without generating fresher sensory observations. Decoupling these cadences allows each workload to meet its distinct physical or computational deadline, provided the boundary interface preserves data freshness and never blocks local real-time control.

4.3.1 Physical time constants and sampling phase lag

The mobile manipulator's motor inverters regulate winding current every 50 µs (20 kHz), while its chunk policy requires 40 ms at p99 latency to ingest camera frames and generate an action chunk. No single cadence can serve both the winding's electromagnetic dynamics and the cost of deep visual inference. The four tiers named in section 4.1 therefore each take their rate from a different time constant:

1. **Inner Commutation and Current Loop (20 kHz, 50 µs cycle):** The PWM carrier frequency, selected to balance switching losses, current ripple, and acoustic noise, fixes the sampling instants. Phase currents are sampled once per carrier period at its midpoint, so a 20 kHz carrier yields $T_s = 50$ µs. The stator winding's electrical time constant establishes the plant pole that the current regulator must compensate. For the hub motors, $\tau_e = L_{\text{phase}}/R_{\text{phase}} = 2.7$ ms (section 2.4.2) places that pole near 59 Hz. The closed-loop current bandwidth is tuned well above this pole and at least a factor of ten below the sampling rate, reaching at most about 2 kHz (Franklin et al. 1998). This loop executes directly within drive silicon and commands electromagnetic torque through the motor controller.
2. **Permission and Position-Supervision Tick (1 kHz, 1000 µs $\pm$ 10 µs cycle):** The rate here is set by distance rather than by a plant pole. Each tick of detection delay is base travel the stopping budget must absorb, and each tick must also supply the drives a fresh position setpoint. At one tick per millisecond both demands are met with the margins quantified below, while the drives close the subordinate velocity and current loops.

3. **Action-Chunk Policy Stream (20 Hz, 50 ms cycle):** Bridging observation age and mechanical execution, the local policy emits receding-horizon action chunks ($H = 16$ waypoints spaced 25 ms apart). On this machine, the policy is a compact visuomotor network following the Action Chunking with Transformers (ACT) architecture, which decodes an entire chunk in a single forward pass. Section 3.5.1 priced it against the iterative diffusion and flow-matching heads that trade more passes for expressiveness. Each pass must stream the model's weights from memory on the application processor's accelerator, which caps the pass rate near $f \leq \mathrm{BW}_{\mathrm{mem}}/M_{\mathrm{weights}}$. Because one pass yields all 16 setpoints, that sweep is paid once per chunk rather than once per setpoint, which is what lets the policy renew at this rate.
4. **Multimodal Intent Planner (2 Hz, 500 ms cycle):** A high-capacity VLA model turns camera frames and a natural-language instruction into task goals and navigation targets. Its period is set by the prefill and weight sweeps of section 3.4.1, not by any plant dynamics. It may therefore steer the chunk policy but never hold a chunk lease, the authority to keep driving the actuators that section 4.4 sizes in tens of milliseconds.

The two proposer cadences reflect mainstream action-chunking and VLA architectures as of 2026, and their frequencies will shift as models and accelerators evolve. The method developed below does not depend on them. It anchors the lease's lower limit to what the proposer can supply and its upper limit to the physical stopping budget.

The current-loop rate follows directly from the winding's electrical time constant $L_{\mathrm{phase}}/R_{\mathrm{phase}}$, and the loop closes within the drive's local inverter. The harder timing question arises one level up, at the joint. There, inertia and structural compliance set the required closed-loop bandwidth, and every digital stage between sensor and actuator (sampling, computation, bus transport) exacts a penalty in phase lag. The crossover frequency ω_c marks the point where open-loop gain equals unity. At this frequency, the phase margin[4] defines the maximum additional phase lag the system can absorb before feedback transitions from stabilizing attenuation to self-reinforcing oscillation.

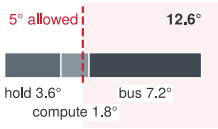
Excess phase lag causes corrective control torques to arrive out of phase with physical joint motion, eventually turning stabilizing negative feedback into unstable positive feedback (Nyquist 1932). Closing a feedback loop across the permission tick accumulates three distinct sources of latency. The discrete sample-and-hold introduces an effective transport delay equal to half the sample period ($T_s/2$). The enforcer requires compute time (T_{comp}) to evaluate constraints. The resulting command then incurs fieldbus transit latency (T_{bus}) before reaching the drive. Because a pure delay T_d removes ωT_d radians of phase across all frequencies, these stages collectively erode the phase margin at crossover according to equation 4.1, the total-delay relation given in section B.5.5:

$$\Delta\phi = \omega_c \left(\frac{T_s}{2} + T_{\mathrm{comp}} + T_{\mathrm{bus}} \right) \tag{4.1}$$

Consider an illustrative joint-loop crossover of 20 Hz and a design phase erosion allowance of 5°. At a 20 Hz crossover, each 1 ms of pure delay along the loop spans that fraction of a full 360° oscillation cycle and costs 7.2° of phase margin. A 1 ms tick's sample-and-hold accounts for 3.6°, which fits comfortably within the budget. However, the 1 ms fieldbus transit cycle contributes 7.2° on its own, and the enforcer's declared worst-case execution time of 250 µs adds another 1.8°. The combined delay of 1.75 ms imposes 12.6° of phase lag, 2.5× the allowance. If the controller delays output release until the subsequent tick boundary, total phase erosion escalates to 18° over 2.5 ms. Because the phase budget tolerates at most 0.69 ms of total delay, less than a single bus cycle, no increase in tick frequency can keep a 20 Hz loop closed across this communication bus within its erosion allowance. The loop might remain stable, but it would spend phase margin the design never allocated to delay. Given 1.75 ms of unavoidable transit and processing latency, a 1 ms tick stays within the chosen phase erosion allowance only at crossover frequencies no greater than 7.9 Hz. Reducing the tick rate to 500 Hz increases the sample-and-hold penalty alone to 7.2°, whereas driving the tick frequency toward infinity still leaves the bus transport penalty untouched. Loop placement, rather than clock scaling, resolves this architectural constraint.

Consequently, the machine closes its high-rate joint velocity and current loops directly within the decentralized motor drives, where co-located encoder sampling bypasses the fieldbus entirely. The centralized 1 ms permission tick operates strictly as a supervisor and trajectory setpoint generator. This decouples the tick from joint-level phase margin constraints, replacing them with two distinct

4 **Phase Margin**: Bode (1945) introduced gain and phase margins to measure how far a feedback amplifier sits from instability, turning the stability criterion of Nyquist (1932) into a practical design margin. In cyber-physical loops, delay erodes the modeled phase reserve and can destabilize the loop under its nominal assumptions. Establishing actual damping and stability requires the complete sampled plant, compensation, and timing response. Because a pure transport or compute delay T_d subtracts $\omega_c T_d$ radians of phase regardless of cause, bus cycles, compute stages, and zero-order holds all draw from the same finite phase budget.



The bus cycle alone spends more phase than the loop can spare.

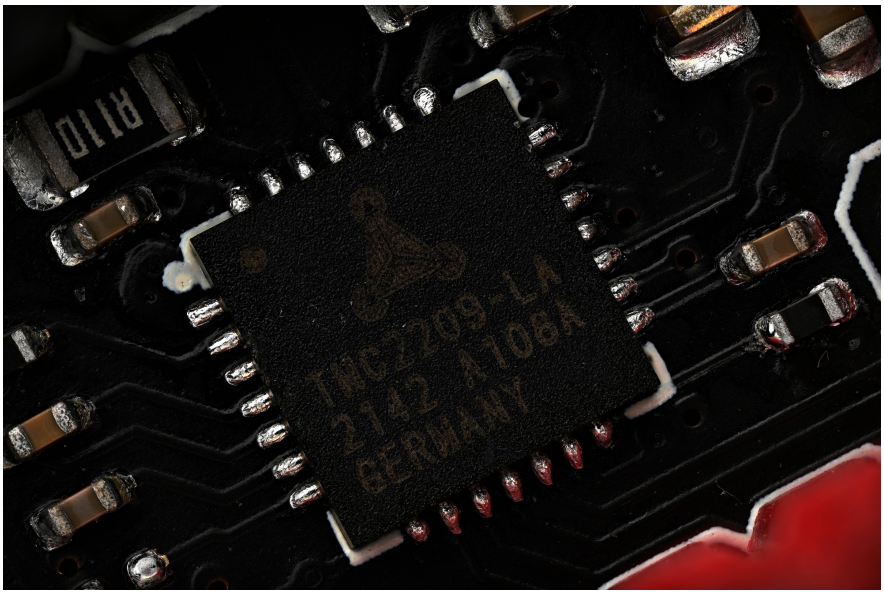


Figure 4.3: Integrated motor-driver silicon: A Trinamic TMC2209 stepper-motor driver in a quad flat no-lead (QFN) package, with a low-value current-sense resistor (marked R110) beside it on the board. The chip carries its own power transistors and current regulation, so the winding current is measured and switched next to the motor it drives rather than by the host processor. A photograph shows the placement; the timing of the current loop is a property of the silicon that the datasheet and measurement must establish. (Credit: Phiarc, Wikimedia Commons, CC BY-SA 4.0.)

systems requirements. First, as a safety supervisor, each millisecond of detection delay incurs 1.5 mm of unbraked base travel at maximum drive velocity, directly consuming clearance from the stopping budget (notebook 4.1). Second, as a setpoint source, the tick must evaluate the admitted trajectory frequently enough to supply smooth kinematic references to the drives. Transport delay along this reference path introduces tracking lag rather than eroding the drive loop's phase margin, because the reference sits outside the feedback loop. Any outer supervisory corrections closed across the bus, such as cartesian compliance trims, must restrict their crossover bandwidth below 7.9 Hz.

Closing those loops in the drive is a hardware arrangement rather than merely a firmware setting. The fastest loop sits beside its sensor and its switch, so phase current is measured and switched next to the winding with no host processor in the loop. Dedicated PWM timers in the same silicon enforce the gap between complementary switch transitions that keeps the power bridge from shorting.[5] The running machine's brushless hub drives follow this arrangement at a larger scale; the small stepper-motor driver of figure 4.3 shows it on one chip.

[5] **Dead-Time Delay**: The interval in which both switches of a half-bridge leg are off, so the leg never shorts the DC bus, is the inverter dead time introduced in section 2.4.2. Dedicated pulse-width modulation timers insert it in hardware (100–500 ns) on every transition, and drive firmware applies online volt-second compensation for the voltage error it introduces.

Perception pipelines feeding the policy stream execute at intermediate cadences constrained by optical physics, sensor exposure durations, and relative scene velocity. In a typical 300 lux warehouse lighting environment requiring a 15 ms exposure, the robot base traveling at its 1.5 m/s drive limit covers $\Delta x = v t_{\text{exp}} = 22.5$ mm while the shutter is open. Resulting image-plane blur interacts with scene depth and optical geometry, setting physical limits on feature tracking. The achievable signal-to-noise ratio, spatial motion blur, and image preprocessing latency jointly establish the feasible sensing cadence for visual control, a trade-off chapter 8 develops. For the permission path the exposure matters as time. An observation is dated at its exposure midpoint, so half the exposure is already part of its age before any processing begins, and that age is the first interval the lease budget must charge (table 4.1).

The chunk-policy generation rate must satisfy the proposal-supply budget set in section 3.5.1. For our mobile manipulator, a 20 Hz renewal cycle paired with an $H = 16$ waypoint horizon (Zhao et al. 2023) bridges visual perception and high-rate actuation. The 50 ms renewal period matches 2 discrete setpoint steps of 25 ms, so every scheduled renewal lands on a setpoint boundary, provided the handoff between the two clocks never makes the faster reader wait.

4.3.2 Lock-free and wait-free mailbox exchanges

If a 1 kHz safety loop blocks on a mutual exclusion lock held by a 20 Hz neural network runtime, one page fault or scheduling hiccup in the runtime becomes an unbounded priority inversion, and the hard real-time controller misses its cycle deadline. Because the tiers run on different processors at different rates, every exchange across a rate boundary must give the faster reader bounded, nonblocking access. Panel (b) of figure 4.4 follows the remedy developed here, the mailbox, through a clean exchange, a write preempted by the tick, and a stalled host, and in none of the three does the reader wait. Panel (a), which shows what the tick does when the proposer stops renewing altogether, returns in section 4.4 once the lease that governs it is defined.

Standard inter-process communication libraries rarely provide deterministic execution bounds out of the box. Message serialization, dynamic memory allocation, intermediate queueing, and thread scheduling can each introduce unpredictable delivery stalls. While shared memory eliminates buffer copying and kernel context switches, it requires concurrency controls to prevent the reader from ingesting partially written, inconsistent command payloads. The communication protocol must strictly bound the consumer's execution time, even when the producer stalls at the most unfavorable point in its write cycle.

While a first-in, first-out (FIFO) queue guarantees sequential message delivery, message ordering is the wrong semantic model for physical motion control. If the control consumer falls behind the rate of production, queued motion plans represent progressively stale physical state estimates. Backpressure, the queue's way of slowing a producer, cannot pause moving mass. The real-time enforcer requires the freshest complete proposal, paired with a strict temporal deadline. This principle does not prevent the controller from buffering the future waypoints of an already admitted trajectory; those waypoints follow an explicit execution schedule governed by a valid lease. Rather, it prevents treating the delayed consumption of historical commands as valid control inputs. Real-time distributed systems distinguish these two communication semantics. An *event message* is queued and consumed exactly once, where dropping a message loses state history. A *state message* represents current environmental state and updates in place, allowing new observations to overwrite older ones without queuing (Kopetz 2011). The proposal mailbox implements state-message semantics via a single-slot seqlock, following the non-blocking write protocol of Kopetz and Reisinger (1993) to ensure mutual independence between writer progress and reader execution.

A baseline nonblocking exchange uses a **single-producer single-consumer (SPSC) seqlock mailbox**[6] in the microcontroller's own static memory. The proposal first crosses the board-level link as a framed packet, which a DMA transfer lands in a private receive buffer. A receive interrupt outside the tick checks the frame's CRC and sequence number and copies the proposal into the mailbox slot. Both participants therefore execute on the microcontroller, the receive interrupt as writer and the permission tick as reader, and the application processor never touches mailbox memory.

The writer increments the sequence counter to an odd value, copies the checked frame into the slot, and increments the counter to an even value using store-release memory ordering. The reader makes exactly one bounded read attempt per control cycle and rejects the snapshot if the counter begins odd or changes before the read finishes. A snapshot that passes becomes the active chunk only after its integrity, freshness, and physical-admissibility checks succeed, and the tick then interpolates it into setpoints (section 4.4.2). A rejected read leaves the previously admitted chunk in force, so the reader never blocks on the writer. Because the timer tick has higher priority than the receive interrupt, a tick that arrives mid-write sees the odd counter and discards the torn read in $\mathcal{O}(1)$ time without spinning. It keeps executing the active chunk for as long as its lease and evidence remain valid. A crashed or stalled application processor cannot leave the counter odd, because it never writes the slot. Its failure appears only as an absence of incoming frames, and the active chunk's lease expires on schedule.

Formally, the reader validates the snapshot using two boundary counter observations:

$$c_{\text{start}} \equiv c_{\text{end}} \equiv 0 \pmod 2 \quad \text{and} \quad c_{\text{start}} = c_{\text{end}},$$

where an odd count signals an active write and a value mismatch signals preemption during the copy. Unlike mutexes or semaphores, this protocol never suspends either participant. Because the reader never retries within a single tick, its computational overhead remains strictly $\mathcal{O}(1)$ regardless of writer activity. Correct execution requires that memory loads occur in strict programmatic

[6] **Versioned Read**: Lamport (1977) showed that a reader can detect a concurrent, overlapping write without acquiring mutual exclusion locks. By incrementing a version counter before and after updating payload data, and having the reader sample the counter in reverse order, any mismatch reliably flags a torn read.

↪ The boundary contract that keeps the application processor out of the microcontroller's mailbox memory is developed in section 13.5.

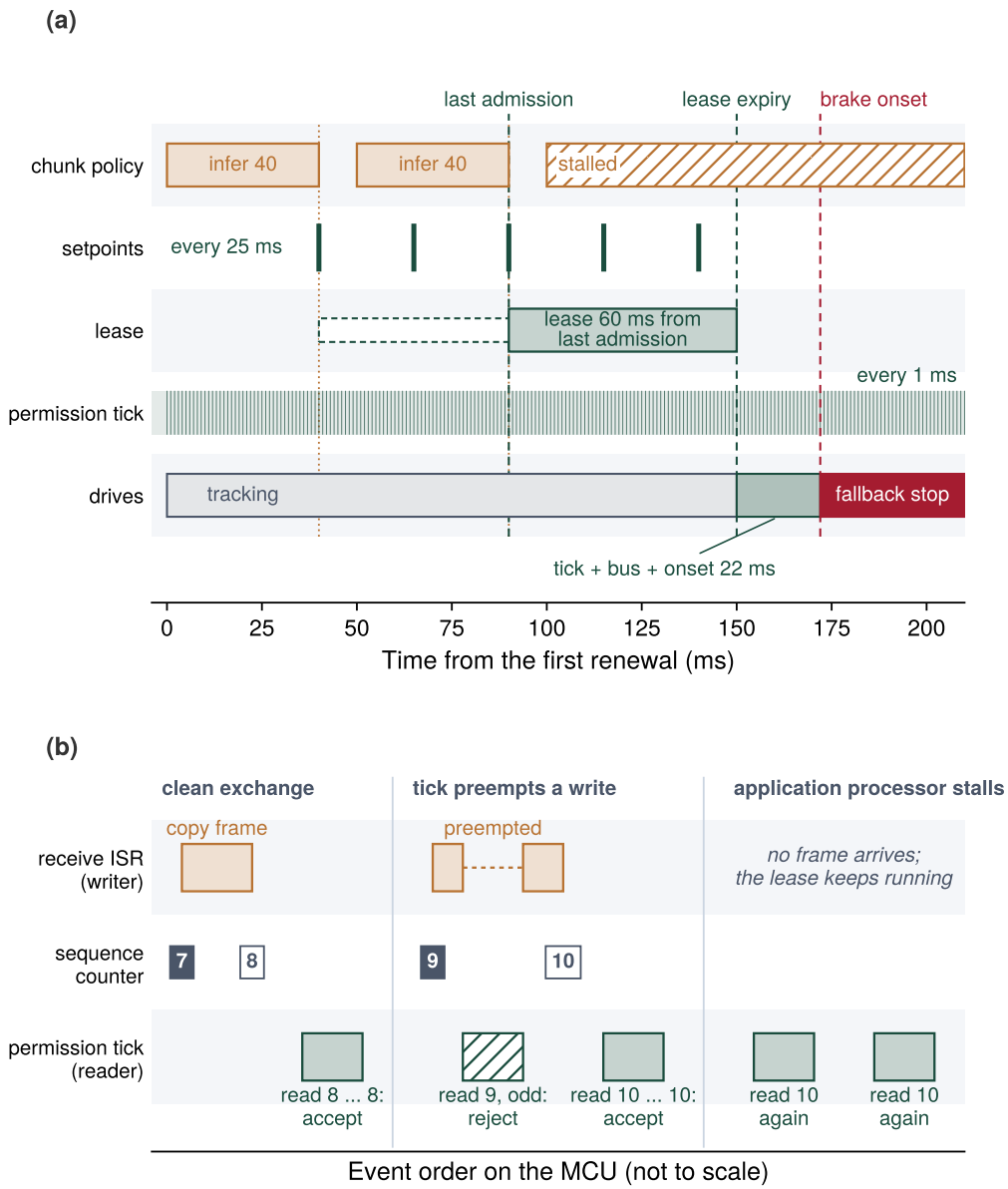


Figure 4.4: Multi-rate cadence and the mailbox exchange: (a) The running machine's clocks during a proposer stall, to scale. The chunk policy renews every 50 ms and needs 40 ms; the admitted chunk supplies a setpoint every 25 ms. When the third renewal never completes, the tick keeps executing the admitted chunk only until the 60 ms lease from the last admission lapses; the tick, bus, and brake onset then put the fallback stop 82 ms after that admission. (b) The single-slot seqlock in the microcontroller's own memory, in event order. The receive interrupt makes the counter odd, copies a checked frame, and makes it even; the tick accepts a copy only when it reads the same even value before and after. A tick that preempts a write reads an odd value and keeps its last validated chunk, and a stalled application processor leaves the slot unchanged while the lease runs.

order. Within a single core arbitrating between an interrupt handler and the periodic tick, compiler optimization barriers prevent instruction reordering. However, when a DMA engine or a second processor core writes the slot directly, hardware memory barriers become mandatory. On Arm Cortex-M profile cores, this requires the full-system data memory barrier instruction (`DMB`) (ARM Limited 2012). Omitting this barrier allows out-of-order execution pipelines to speculatively load

payload fields before validating the sequence counter, silently allowing torn data into the control loop.

A **wait-free three-slot mailbox**[7] can avoid read/write collisions by assigning separate slots to the writer, the published proposal, and the reader. Atomic publication transfers slot ownership; it must never let the writer overwrite the reader's active slot. Unlike the one-attempt seqlock, this design can provide a complete published snapshot without discarding a collided read. Its extra storage and ownership protocol must still be checked on the target hardware. Simpson's four-slot mechanism reaches the same guarantee, a complete snapshot for a reader that never waits, using only single-bit control variables written with ordinary stores and no atomic read-modify-write instruction (Simpson 1990). Neither design makes an old proposal fresh or extends its lease.

Sequential diagnostic telemetry, by contrast, operates under event semantics. A single-producer single-consumer (SPSC) ring buffer preserves an ordered log of events until its bounded capacity fills, whereas the proposal mailbox exposes only the latest complete candidate state. The permission interface relies on these bounded-access guarantees to eliminate wait states; it intentionally discards historical command queues.

No robotics middleware setting can stand in for the chunk lease, because none derives its parameters from the physical stopping budget and none executes on the consumer's own hardware clock at the point of actuation. Middleware can still approximate the mailbox's state-message semantics, and the mapping shows how far configuration goes. In ROS 2, communication relies on the Data Distribution Service (DDS) or equivalent publish-subscribe transports (Macenski et al. 2022), whose Quality-of-Service (QoS) policies reflect real-time contracts only partially (Object Management Group 2015). Configuring a *keep-last* history depth of one provides state-message semantics by retaining only the freshest sample. Selecting *best-effort* reliability drops lost packets rather than triggering retransmissions, avoiding the burst of stale samples that *reliable* delivery releases after network blips. A DDS *deadline* policy raises notification events when sample inter-arrival times exceed a threshold, acting as an alerting heartbeat without enforcing authority revocation. Similarly, a DDS *lifespan* policy purges samples whose age exceeds a configured threshold, but this mechanism relies on synchronized network clocks, defaults to arrival timestamps when synchronization degrades, and controls only message transport rather than granted actuation authority. Callback scheduling within middleware executors also introduces variable dispatch jitter, requiring dedicated response-time analyses across end-to-end processing chains (Casini et al. 2019). At the direct hardware interface, the ros2_control framework implements a nonblocking realtime buffer using a single try-lock attempt. If the writer holds the mutex, the read aborts and the controller retains its prior command (ros2_control Development Team 2025), which mimics the one-attempt semantics of a seqlock. Regardless of middleware abstractions, the physical permission path must independently enforce its own freshness checks and lease bounds.

Because the receive interrupt write routine is brief and resumes immediately after the tick completes, an access collision discards at most one read attempt without delaying the tick. However, if the upstream link halts completely, the mailbox slot remains unchanged across thousands of consecutive ticks. Bounded reader execution time does not guarantee bounded publication latency. The system therefore requires an explicit physical policy governing how long a previously admitted motion chunk may continue commanding actuators. Under the first law, irreversibility (principle 1), every millisecond that a stalled proposal retains actuation authority produces physical displacement that cannot be retracted. That exposure has to be priced against the stopping budget, which makes the length of a proposal's authority the next design decision.

[7] **Wait-Free**: Herlihy (1991) defined an implementation as wait-free when every operation finishes in a bounded number of its own steps, whatever the other processes do. That is stronger than lock-free progress, which guarantees only that some operation completes while a particular one may retry indefinitely. The one-attempt seqlock read is bounded but can come back empty-handed; the slot mailbox is bounded and always returns a complete snapshot, and the extra slot is the price of that second property.

↪ The memory-ordering primitives, cache-line padding, and slot-ownership protocols that implement both exchanges are developed in section C.8.

↪ The inverse failure, middleware that reports healthy delivery while transmitting stale payload data, is developed in section 9.9.

4.4 Chunk Lease Sizing from the Stopping Budget

A learned policy runtime can stall for many reasons: a page fault, thermal throttling on the vision accelerator, memory fragmentation, or an uncaught exception in perception preprocessing. The robot does not stop when its software does. A loaded base keeps its momentum ($p = mv$) and covers 1.5 mm every millisecond at the drive limit, while the downstream controller keeps converting the remaining waypoints of the last admitted chunk into setpoints. To keep an unrenewed chunk from driving the machine into an obstacle, the permission path must end that chunk's authority on its own hardware clock, whether or not the proposer ever reports its failure. How long it should wait before doing so is a question about distance, not software.

Definition 4.2: Chunk lease

Chunk lease is the interval, counted on the permission path's own clock from a chunk's admission, during which that admitted chunk may keep driving the actuators without a successful replacement; when it expires, the permission path ends the chunk's authority and commands the state-matched fallback, a stop profile chosen from the machine's measured speed and pose.

1. **Significance**: The lease enters the pre-brake delay τ_{delay} of the stopping budget, so each millisecond of it costs v of clear distance, 1.5 mm at the running machine's drive limit. The machine's 60 ms lease, with the tick and bus cycle that detect and deliver its expiry, adds 93 mm to the stopping budget (notebook 4.1).
2. **Distinction**: Unlike a heartbeat timeout, which shows only that the host is alive, a lease is renewed only by a freshly admitted proposal. Its length is derived from the plant rather than configured: above the proposer's renewal floor and below the stopping budget's ceiling. Unlike an evidence deadline, it bounds how long one motion segment may execute, not how long the observation behind it stays valid.
3. **Common pitfall**: Lengthening the lease to ride out proposer hiccups. A longer lease buys continuity with clearance, and a lease that reaches the pre-brake ceiling commits the machine to motion it can no longer stop in time.

In classical distributed systems, a lease grants bounded access to a shared digital resource, such as a distributed lock or cache entry, expiring automatically if the leaseholder crashes (Gray and Cheriton 1989). In physical AI, the leased resource is the kinetic authority to move mass through space. Because physical motion cannot be undone, the duration of a lease must be formally derived from the physical stopping budget. The distance covered during the lease, the detection latency, the fieldbus transit, and the brake onset, together with the braking distance itself, must fit within the available clearance D_{clear} after the localization uncertainty and protective margin are subtracted. Rearranging the stopping-distance relation of equation 2.2, $v\,\tau_{\text{delay}} + v^2/2a_{\text{brake}} + \delta_{\text{loc}} + \delta_{\text{margin}} \le D_{\text{clear}}$, and dividing the remaining pre-braking distance by vehicle velocity yields the **pre-brake ceiling**, representing the maximum tolerable delay τ_{delay} before physical braking deceleration must commence:

$$\tau_{\text{delay}} \le \frac{D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - v^2/2a_{\text{brake}}}{v} \tag{4.2}$$

where v represents operating velocity, a_{brake} denotes verified braking deceleration, and δ_{loc} and δ_{margin} represent the localization uncertainty and safety clearance of that formulation. As operating speed v decreases, the required braking distance shrinks quadratically while the velocity divisor decreases linearly, causing the pre-brake ceiling to expand toward infinity at rest. At maximum operating speed, however, every millisecond allocated to the lease path consumes exactly v units of stopping clearance.

Panel (a) of figure 4.4 draws the lease on the running machine's own clocks. The chunk policy renews twice and then stalls. The tick keeps executing the last admitted chunk only until the lease counted from that admission lapses, and the next tick, one bus cycle, and the brake onset then bring the fallback stop. Notebook 4.1 evaluates equation 4.2 for the mobile manipulator at its drive limit and prices that sequence inside it.

Napkin Math 4.1: The price of a lease

Problem: In the illustrative aisle scenario, the warehouse mobile manipulator cruises at its drive limit toward a person who has stepped out from a rack end and stopped. The permission path admits one last chunk and then receives no admissible replacement. How much time remains before braking must begin, and how much of it, and of the clear distance, does the chunk lease spend?

Variables:

- Speed $v = 1.5$ m/s, the drive limit (illustrative); braking deceleration $a_{\text{brake}} = 2$ m/s², loaded on a dry floor (illustrative).
- Clear distance $D_{\text{clear}} = 1.10$ m (illustrative), less localization error $\delta_{\text{loc}} = 50$ mm (illustrative) and protective clearance $\delta_{\text{margin}} = 100$ mm (chosen).
- Brake onset $T_{\text{act}} = 20$ ms (illustrative); permission tick $T_{\text{tick}} = 1$ ms (chosen); bus cycle $T_{\text{bus}} = 1$ ms (illustrative).
- Chunk lease $T_{\text{lease}} = 60$ ms; renewal period $T_{\text{chunk}} = 50$ ms with a 5 ms variation allowance T_{jitter} (all chosen).

Math: The braking term and the ceiling of equation 4.2:

$$\frac{v^2}{2a_{\text{brake}}} = \frac{(1.5\ \text{m/s})^2}{2 \times 2\ \text{m/s}^2} = 562.5\ \text{mm}$$

$$\tau_{\text{delay}} \leq \frac{1100 - 150 - 562.5\ \text{mm}}{1.5\ \text{m/s}} = 258.3\ \text{ms}$$

The stopping-budget terms that section 2.3 already charged (braking, the localization and clearance overheads, and onset travel $vT_{\text{act}} = 30$ mm) hold 742.5 mm of the clear distance. The lease path, lease plus the tick that notices its expiry plus the bus cycle that delivers the stop, is the next term:

$$v\,(T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}}) = 1.5\ \text{m/s} \times (60 + 1 + 1)\ \text{ms} = 93\ \text{mm}$$

$$742.5 + 93 = 835.5\ \text{mm}, \quad 1100 - 835.5 = 264.5\ \text{mm}$$

In time, from the last admission to brake onset, and against the ceiling:

$$62 + 20 = 82\ \text{ms}, \quad 258.3 - 82 = 176.3\ \text{ms}$$

Continuous tracking also sets a floor under the lease:

$$T_{\text{chunk}} + T_{\text{jitter}} = 50 + 5 = 55\ \text{ms} < T_{\text{lease}} = 60\ \text{ms}$$

Systems insight: A chunk lease is a term in the stopping budget, not a timeout chosen for convenience. Each millisecond added to it costs 1.5 mm of clear distance at the drive limit, distance that the observation age and the stop profile still need, so the lease sits just above its renewal floor rather than anywhere under the ceiling.

The renewal floor is the lease duration needed to cover the budgeted inter-arrival interval between successful admissions without starving trajectory execution. Because this floor cannot rely solely on the model's 99th-percentile (p99) inference latency, it must accommodate bus transit, validation overhead, and mailbox synchronization jitter. If a renewal fails to arrive before the lease expires, the system terminates trajectory tracking rather than granting an unearned extension. The 176.3 ms remaining under the pre-brake ceiling is not spare margin either. It must still absorb the observation's age at admission, which section 4.9.1 charges against it, and the jerk limits of the braking profile.

Every charge in this budget is an interval between the four boundary events introduced in section 2.2.2. *Capture* dates an observation at its exposure midpoint, *dispatch* hands that evidence to the proposer, *admission* accepts a chunk built on it and starts the chunk's lease, and *brake onset* marks the first physical braking force at the drives. In the worst-case stall scenario, an obstacle enters the path just after *capture* of the camera frame that produces the last healthy chunk, which therefore travels from *capture* through *dispatch* to *admission* without reflecting the hazard. When the subsequent renewal stalls, the robot continues tracking that last admitted chunk from *admission* until its lease expires and *brake onset* engages, chaining all three intervals end to end. Between these four events lie three intervals, each derived in exactly one chapter (table 4.1). Only row 3, the stalled-proposer response, is derived here, from the lease notebook. Rows 1 and 2 enter Part I as allocations: the observation age that section 8.4.1 will build stage by stage, and the dispatch-to-admission interval

that section 13.6.2 will charge to the board link and the validator. Later chapters charge time against these events and this table rather than keeping their own stage lists.

Table 4.1: The capture-to-brake-onset timing budget of the running machine: Four events bound three intervals, and each interval is derived in one chapter. Rows 1 and 3 together form the 133.6 ms pre-brake delay of the stopping budget; row 2 is the allocation that budget must still pay for. Row 2 charges the whole inference after dispatch, although its vision stage already lies in row 1, so the sum is conservative. Every value is an allocation or an illustrative stage time to qualify on the loaded machine, not a measurement.

Interval	Opens at	Closes at	What it contains	Allocation	Derived in
1. Observation age t_{age}	capture	dispatch	Half the exposure, readout, transport, DMA, image signal processing, perception backbone, hand-off to the proposer	51.6 ms	section 8.4.1
2. Dispatch to admission Δ_{da}	dispatch	admission	Proposer inference (charged in full), board link, deserialization, whole-plan validation, activation, wait for the next tick	63.3 ms	section 13.6.2
1 + 2. Replacement latency L_{lat}	capture	admission	Rows 1 and 2; validation is counted once, inside row 2	**114.9 ms**	section 11.3.1
3. Stalled-proposer response	admission	brake onset	Chunk lease, one permission tick, one fieldbus cycle, brake onset	82 ms	section 4.4
1 + 2 + 3. Capture to brake onset	capture	brake onset	All three intervals	**196.9 ms**	section 12.3.1

The table also shows why the stopping budget and the lease rule charge different ages. The stopping budget's pre-brake delay sums rows 1 and 3 and leaves row 2 to be paid from the spare that the elementary budget of section 1.4.3 left. The reason is that row 2's length depends on where the proposer and the permission path run, a decision section 13.6.2 makes. A renewal cannot defer that payment. A replacement chunk spends row 2 before its lease even starts, so the evidence age that the lease rule of section 4.9.1 charges at admission is rows 1 and 2 together.

4.4.1 Supervisory timers and proposal wire contracts

Granting a fresh lease at admission without charging the time the observation already spent in inference and board-link transit lets the drives keep tracking a trajectory whose sensory evidence is already stale. Waiting on a single timeout while a hung policy thread sits inside a live host kernel lets them keep tracking a trajectory whose producer has stopped updating. Evidence keeps aging from capture onward, and admitting a chunk starts its lease on the microcontroller's clock without rolling that age back. Neither the dispatch timestamp nor the newly granted lease accounts for the dispatch-to-admission interval, so the evidence-age check must charge row 2 of table 4.1 explicitly. Authority over an admitted chunk then ends at the earliest of four events: its evidence freshness deadline passes, its lease expires without a successful replacement, the host heartbeat times out, or an independent safety reflex such as the protective scanner path detects an obstacle. None of these waits for the others, and which fires first depends on the scene. At the conveyor, the tracked evidence horizon that section 3.5.2 computed is several times longer than the lease, so there a stalled chunk loses its authority to the lease. The chunk lease bounds how long one admitted motion segment may execute. How long the strategic goal stays meaningful is a separate grant, the intent lease of section 10.8, and neither is the nominal planning horizon.

↩ The freshness deadline that bounds how old evidence may be at actuation is established in section 2.2.

Three independent supervisory timers guard the exchange, and each monitors a distinct failure domain:

1. **The chunk lease ($T_{\text{lease}} = 60$ ms):** Bounds how long an admitted proposal may command actuators. Only a freshly validated, admitted proposal chunk renews the lease. Its duration is derived directly from the physical stopping budget, sitting between the renewal floor and the pre-brake ceiling (notebook 4.1).
2. **The host heartbeat ($f_{\text{heartbeat}} = 100$ Hz, timeout 30 ms):** Asserts that the application processor operating system kernel and user-space runtime remain alive. Expiry of this timeout revokes the active proposal without waiting for the chunk lease. Heartbeat pulses **never renew the**

chunk lease, because a live process can easily suffer algorithmic livelock or produce corrupted, out-of-distribution setpoints.

3. **The microcontroller self-watchdog ($T_{\text{self}} = 5$ ms):** Monitors whether the permission path completes its required checks within the watchdog interval. The permission loop feeds this hardware watchdog only upon successfully completing all verification checks in a tick; a hung enforcer trips the watchdog even if timer interrupts continue firing (section 4.10).

A watchdog refresh confirms only that the specific task servicing the timer is still executing. In *Bookout v. Toyota* (2013), litigation centered on whether a timer-tick interrupt could continue petting a watchdog while application-level control tasks had hung (Barr 2013). Although federal investigations by NASA and NHTSA identified no electronic mechanism capable of causing large unintended accelerations (NASA Engineering and Safety Center 2011), the dispute highlights a real-time principle. Watchdog refresh points must be coupled to the completion of safety-critical algorithmic cycles rather than independent periodic interrupts.

Real-time safety requires wrapping the proposer's payload in a transport and authority envelope. The system enforces chunk leases through the **proposal header**, a structured wire contract that enables the permission path to validate candidate trajectories without placing trust in host application software. The header encapsulates a monotonically increasing sequence number and the evidence timestamp of the source sensory frame (including clock conversion error bounds). It also carries the host dispatch timestamp and the requested lease duration (which the permission path caps at the lease it derives, so a proposer can ask for less authority but never more). A cryptographic parent-record digest supports provenance tracking. The remaining fields are the coordinate frame identifier, the payload type enum, and a 32-bit cyclic redundancy check (CRC-32). For the default payload type the payload consists of H kinematic waypoints specifying joint coordinates and velocities. The payload-type field is where other command forms announce themselves, so an impedance target (section 4.6.1) carries its stiffness and damping gains beside its equilibrium waypoints. In this layered architecture, the learned policy expresses kinematic intent, the permission enforcer converts admitted waypoints into position setpoints, and the drives regulate joint velocity and motor current. The payload deliberately omits raw torque commands; while direct-torque policies require alternate wire representations and specialized admission checks, their outputs remain untrusted proposals. In all cases, an admitted proposal retains execution authority only until the earlier of its lease expiration and its evidence freshness deadline, regardless of whether cancellation packets arrive across the network.

↩ The proposal payload, the candidate motion and the evidence times it carries, is established in section 3.5.4.

Industrial safety networking adopted the same transport safeguards decades before the advent of learned robotic proposers. Functional-safety fieldbus standards such as IEC 61784-3 establish the **black-channel principle**. Transmission media, network controllers, and intermediate switches are treated as untrusted, fallible channels, which requires safety-critical packets to encapsulate self-contained integrity and timing protections verified exclusively at communication endpoints (International Electrotechnical Commission 2021). Fieldbus profiles like PROFIsafe and Safety over EtherCAT (FSoE) exemplify this architecture. FSoE protects frames against corruption using CRCs, against repetition and packet loss using sequence counters, against misrouting using unique connection identifiers, and against transit delay using watchdog timers. Missing or delayed frames force the receiver into a deterministic safe state (EtherCAT Technology Group 2019). The proposal header mirrors this design field for field: checksums guard integrity, sequence numbers detect drops, payload identifiers ensure proper dispatch, and the chunk lease replaces the watchdog timer. The physical link between host application processor and safety microcontroller operates as an untrusted black channel; end-to-end header checks allow the permission path to enforce its declared physical admission contract without trusting host operating system integrity. Classical real-time theory provides the complementary semantic foundation. Kopetz defines a computing system's internal representation of a dynamic physical entity as a *real-time image*, which remains *temporally accurate* only during a bounded physical duration following the initial observation (Kopetz 2011). The proposal's evidence deadline represents this temporal accuracy window applied to sensory inputs, while the chunk lease bounds the duration of derived actuation. This architecture adds one thing to these precedents, the grounding of temporal validity. Whereas industrial fieldbuses configure watchdog timeouts empirically and real-time images reflect passive plant dynamics, the

chunk lease is derived analytically from the physical stopping budget above and the proposer's renewal floor below (equation 4.2, notebook 4.1).

↪ **Byte layout:** The proposal header's and chunk payload's fields, types, and offsets are given in section C.15.1.

Every record crossing the proposal boundary expresses temporal quantities referenced to the safety microcontroller's monotonic clock. Upstream producers operating on independent oscillator domains map timestamps using a versioned affine transformation and transmit explicit synchronization error bounds, which the safety consumer adds to all age evaluations. Wall-clock calendar time is restricted to diagnostic logging and software release expiration; it never enters real-time safety gating.

Individual boundary records guard against three orthogonal failure modes through three distinct mechanisms, only two of them cryptographic. A checksum across the header and payload, a CRC rather than a cryptographic function, detects transmission bit flips and memory corruption during transit, leaving origin authentication unaddressed. Similarly, a cryptographic digest of the parent record establishes computational lineage without authenticating the sender. Cryptographic message authentication codes (MACs) appear exclusively where sender identity conveys explicit operational authority: network teleoperation commands, immutable authority audit logs, and release artifacts verified during deployment. Learned model proposals omit cryptographic signatures entirely, because the permission enforcer validates candidates based on physical admissibility and temporal freshness rather than origin. A cryptographic key stored inside an untrusted host environment provides no assurance of kinematic safety.

↪ How a teleoperation command's authentication tag is verified and priced against the fallback deadline is developed in section 14.5.

The reader's possible failures fall into four classes (table 4.2), and none of them lets a failed producer make the permission path wait. The classes keep the exchange from blocking. They do not by themselves contain the plant, because the fallback each one selects still needs bounded timing and a validated stop. Even a valid admitted chunk still has to be stepped from the proposer's coarse waypoint spacing down to the permission tick's rate.

Table 4.2: Boundary contract failure classes and deterministic fallback matrix: Exchange faults across the proposal boundary by class, with the fallback each selects. Fallback paths are selected without waiting on host recovery.

Failure Class	Reader Detection Condition	Physical Systems Fallback Action	Bounded Response
Torn / In-Flight	Odd c_{start} or changed c_{end}	Reject current snapshot; use admissible prior chunk or fallback	One bounded attempt; measure duration
Malformed	Checksum or configured command bound fails	Reject candidate; retain admissible active chunk or use fallback	Measure check and response
Stale	Past proposal expiry or evidence deadline (local clock)	Reject stale candidate; end active authority if its own deadline passed	Deadline from plant budget
Missing	Sequence gap	Record audit gap; evaluate current record on its own merits	One tick for flag

4.4.2 Rate interpolation and boundary fallback execution

Holding each admitted waypoint constant for its full 25 ms spacing until the next waypoint arrives injects staircase position steps into the drives rather than a smooth trajectory, delaying reference tracking by half the step spacing on average and exciting high-frequency structural resonances. Bridging the learned policy's renewal rate (20 Hz) and the real-time permission tick (1 ms) without those disturbances requires guarding against three sampled-data failure modes: reference lag, spectral aliasing, and buffer starvation. Because the zero-order hold's latency sits entirely on the feedforward reference path outside the feedback loop, it appears as tracking error rather than eroding feedback phase margins (section 4.3.1). Confusing reference lag with closed-loop phase lag often leads engineers to misdiagnose tracking errors as plant instability. To eliminate discrete steps across chunk boundaries, the permission enforcer constructs a quintic Hermite spline bridge matching boundary position, velocity, and acceleration. These six boundary constraints guarantee C^2 continuity across chunk transitions, meaning position, velocity, and acceleration all stay continuous. Evaluating this admitted spline at 1 kHz yields continuous position references, smooth velocity feeds, and feedforward acceleration profiles for the underlying joint controllers.

↩ Action chunking, which produces the trajectory segments this interpolator densifies, is established in section 3.5.1.

Spectral aliasing occurs when unmodeled high-frequency mechanical vibration or sensor noise, sampled too slowly, reappears as a false low-frequency signal in the slower streams that estimators and the tick consume. The architecture mitigates aliasing through front-end analog filtering followed by multi-rate digital decimation,[8] low-pass filtering a fast sample stream before keeping only one sample in several, within the high-rate microcontroller domain. Classical sampled-data control models assume constant, deterministic sampling periods. Temporal jitter, a sampling interval that varies from cycle to cycle, degrades phase margin more severely than a constant delay of equivalent mean duration (Cervin et al. 2003). Discrete trajectory execution also assumes setpoints replenish continuously before downstream buffers drain. The permission path enforces these invariants via cycle-deterministic hardware timer interrupts and real-time buffer-depth accounting. If the upstream planner stalls and the trajectory buffer reaches its terminal waypoint, the enforcer rejects stale extrapolation and commands a validated local stopping profile.

An overrun costs something different at each of the four tiers:

- **Current loop (20 kHz):** Its jitter allowance is below $10\,\mu s$ (table 4.3). A timing overrun, gate-drive desynchronization, or current spike trips a hardware fault line that disables the inverter's PWM stages.
- **Permission tick (1 kHz):** A deadline overrun by the enforcer software sends the machine to the local deceleration stop.
- **Chunk policy (20 Hz):** Inference latency is stochastic (40 ms at p99), so some chunks arrive late. A late chunk costs nothing until the active chunk's lease or evidence deadline runs out, and the enforcer never waits on the proposer's optimistic estimate of when it will finish. When either deadline passes, the enforcer commands the state-matched fallback.
- **Intent planner (2 Hz):** Its goals expire under a separate *intent lease*, a longer grant of authority over the goal itself rather than over any motion (developed in section 10.8). When it lapses, the system holds station instead of chasing a stale goal.

The lease notebook charged the tick and the bus cycle as fixed terms, 1 ms and 1 ms, as though each were certain to finish on time. That premise is the next thing to establish. It requires a worst-case response bound for every stage between the mailbox and the drives, and no benchmark or faster processor substitutes for one.

4.5 Real-Time Command Transport and Bus Determinism

A profiling campaign can run the permission tick for days without a missed deadline and still not establish the tick's worst case. An empirical percentile says how often sampled executions stayed under a threshold. It says nothing about executions that meet a combination of competing tasks, memory contention, or peripheral delay the campaign never produced. The lease budget needs the other kind of number: a response-time bound that accounts for every unit of permitted computation and queuing delay along the critical path under defined operating conditions. In an embodied system, timing determinism protects both stability and safety, because delayed feedback erodes the controller's phase margin and delayed actuation consumes stopping clearance before the chassis can decelerate. Bounding the tick on the microcontroller comes first; bounding the drive network that carries its admitted setpoints to the motors follows the same logic (Kopetz 2011).

4.5.1 Response time analysis and execution variance

A permission tick whose own instructions finish in 250 µs can still miss its 400 µs deadline if a lower-priority link driver masks interrupts while re-arming a DMA descriptor or a burst of higher-priority timer interrupts preempts the tick mid-cycle. Response Time Analysis (RTA) bounds a task's total response time by summing the three constituents introduced in section 4.2: execution time, blocking delay, and preemption interference (Joseph and Pandya 1986; Audsley et al. 1993). To guarantee schedulability, each term requires an analytical upper bound under specified operating conditions:

$$R_i = C_i + B_i + I_i \leq D_i \tag{4.3}$$

where C_i is the worst-case execution time (WCET) of task i, B_i is the maximum blocking time spent waiting for lower-priority tasks to vacate shared critical sections, I_i is the maximum interference

8 **Filter Group Delay**: Digital smoothing filters commonly applied in machine learning pipelines introduce significant phase lag when placed inside feedback control loops. An N-tap moving-average finite impulse response (FIR) filter imposes an average group delay of $(N-1)/2$ sample intervals. Consequently, a 9-tap moving average operating at the 1 ms permission tick delays feedback signals by 4 ms. At the illustrative 20 Hz crossover frequency from section 4.3.1, this delay burns 28.8° of phase, 5.8× the design allowance. In real-time control paths, decimation and filter topologies must be selected for minimal phase lag rather than cosmetic smoothing.

from preempting higher-priority tasks, timer interrupts, and DMA transfers, and D_i is the relative deadline.

Each term demands distinct evidentiary backing. Formal static timing analysis against an accurate cycle-level processor model can establish a bound on the WCET C_i. High-watermark profiling under stress-bench conditions instead characterizes observed execution times; it can refute a declared WCET but cannot establish one. The blocking term B_i represents the longest critical section of any lower-priority task that can lock a shared resource required by task i. The priority ceiling protocol establishes a provable bound on B_i. In this protocol, each synchronization lock carries a priority ceiling equal to the highest priority of any task that may acquire it. A task may acquire a lock only if its priority strictly exceeds the priority ceilings of all locks currently held by other tasks. Furthermore, any lock holder that blocks a higher-priority task temporarily inherits that task's elevated priority. Under these constraints, a task is blocked by at most a single lower-priority critical section across its entire execution (Sha et al. 1990). An immediate priority ceiling protocol (IPCP), which instantly elevates a lock holder to the resource ceiling upon acquisition, guarantees the same blocking bound with lower runtime overhead. When a real-time task avoids shared software mutexes altogether through lock-free data structures, the blocking term B_i reduces to the longest interval during which lower-priority driver code masks processor interrupts or holds a non-preemptible hardware peripheral transaction. Finally, the interference term I_i aggregates the execution time of all higher-priority tasks and interrupt service routines released during the response window. Because a longer response time provides a wider window for higher-priority tasks to re-trigger, calculating I_i requires solving a fixed-point recurrence relation. Section C.3 details this formulation. Equation 4.3 defines the overarching budget that the recurrence resolves, and notebook 4.2 evaluates that budget for the permission tick.

Napkin Math 4.2: The response-time margin

Problem: The permission tick must finish within its deadline after every release. With its declared WCET and illustrative bounds on blocking and interference, does it meet the deadline, and what happens to the margin when one term loses its bound?

Variables:

- Declared WCET $C_i = 250$ µs and deadline $D_i = 400$ µs (both chosen).
- Blocking $B_i = 50$ µs, the longest section in which lower-priority code masks the tick, such as the link driver re-arming its DMA descriptor (illustrative). The mailbox adds nothing to B_i, because the seqlock never makes the tick wait for the receive interrupt.
- Interference $I_i = 40$ µs, the timer and bus interrupts that can preempt the tick within one response window (illustrative).

Math:

$$R_i = C_i + B_i + I_i = 250 + 50 + 40 = 340\ \mu\text{s} \le D_i = 400\ \mu\text{s}, \quad D_i - R_i = 60\ \mu\text{s}$$

Systems insight: The margin is only as dependable as its weakest constituent bound. For instance, moving the mailbox copy into a low-priority task guarded by a standard mutex without priority inheritance or priority ceilings leaves B_i completely unbounded, because medium-priority work can preempt the lock holder indefinitely. This is the Mars Pathfinder pathology of war story 4.1. Similarly, permitting unscheduled DMA traffic on the tick's memory path destroys the upper bound on I_i. If even a single term loses its bound, the tick lacks a defensible worst-case response time, leaving the stopping budget's T_{tick} parameter with no engineering foundation.

↪ How host memory traffic would inflate the permission tick's interference if both paths shared one die is developed in section 13.3.

Neither a low average latency nor a modest processor utilization can substitute for that analytical bound,[9] because a single missed deadline degrades the machine across multiple architectural layers. When a deadline slips, distributed motor drives must hold their previous position setpoint for an entire additional bus cycle. Consequently, the actuator setpoint lags by a full period (1 ms) rather than the conventional half-period delay associated with a zero-order hold. Repeated deadline

[9] **Rate-Monotonic Scheduling**: Liu and Layland (1973) established that static-priority rate-monotonic scheduling guarantees periodic task schedulability when total processor utilization satisfies $U \le n(2^{1/n} - 1)$, converging to $\ln(2) \approx 0.693$ for large n. The utilization bound is sufficient, not necessary, so a task set that exceeds it may still meet every deadline; with shared-resource blocking and deadlines shorter than periods, only the exact response-time test of section C.3 decides.

misses manifest as timing jitter, which Cervin et al. (2003) analyze as a direct erosion of phase margin in digital feedback loops closed through the tick. Within the safety subsystem, a delayed tick evaluates an expired lease or safety violation late; at the drive velocity limit, each 1 ms delay incurs an irreversible loss of 1.5 mm in physical stopping clearance.

Bounding these response terms requires categorizing every potential source of timing variability according to whether an analytical bound exists:

1. **Bounded Sources:** Hardware mechanisms that admit analytical upper bounds from vendor architectural specifications. Examples include deterministic instruction pipelines, round-robin interconnect arbitration, zero-wait-state tightly coupled memory (TCM/SRAM), and deterministic vectored interrupt controllers on real-time microcontrollers.
2. **Experimentally Characterized Sources:** Microarchitectural behaviors that cannot be proved strictly from first principles but can be enveloped empirically under frozen hardware configurations. Examples include bounded cache-line misses under hardware cache partitioning, write-buffer draining latencies, periodic DRAM auto-refresh pauses, and thermal throttling profiles stabilized by closed-loop active cooling.
3. **Unbounded Sources:** Software and hardware mechanisms that have no finite mathematical worst-case bound and must be purged from the real-time critical path. Examples include operating-system page faults under demand paging, preemptive thread synchronization lacking priority inheritance, dynamic heap allocation over fragmented memory pools, and unscheduled peripheral bus contention.

A credible timing guarantee must therefore explicitly enumerate all permissible operations along the execution path. The hardware partitioning and core isolation strategies from section 4.2 eliminate broad classes of timing interference. A **zero-allocation static architecture** pre-allocates all buffers and queues at boot time to eliminate memory-management overhead. Neither strategy can eliminate contention for shared memory buses. Response-time analysis must account for every remaining shared-resource transaction, because a high task priority alone never guarantees real-time execution, as the Mars Pathfinder resets showed.

War Story 4.1: The Mars Pathfinder priority inversion (1997)

Context: In July 1997, shortly after landing on Mars, the Mars Pathfinder spacecraft's avionics computer ran the VxWorks real-time operating system with preemptive priority-based scheduling across mission tasks (Reeves 1997).

Mechanism: A high-priority thread (`bc_dist`) distributed critical spacecraft bus telemetry, while a low-priority thread (`ASI/MET`) published meteorological sensor observations through a shared pipe. The mutex serializing access to this pipe was not explicitly declared in mission application code; rather, it was embedded deep within the operating system's `select()` implementation to protect internal file-descriptor queues (Reeves 1997). When `ASI/MET` acquired this internal mutex, long-running medium-priority communications tasks preempted it. Because these medium-priority tasks outranked the meteorological thread, they starved it of CPU cycles, preventing it from releasing the mutex while `bc_dist` remained blocked, an unbounded priority inversion.

Impact: The blocked `bc_dist` thread missed its execution deadline. When the supervisory `bc_sched` watchdog initiated the subsequent bus cycle and detected that `bc_dist` had not completed, it flagged a critical timing fault and commanded a total system reset. Although telemetry collected prior to the reset survived in non-volatile storage, the resets recurred repeatedly during operational surface maneuvers.

Response: NASA engineers diagnosed the pathology on an identical ground testbed using VxWorks execution tracing, subsequently uploading an inflight software patch that enabled priority inheritance (Sha et al. 1990) on the underlying mutex semaphore.

Systems lesson: Preempting a low-priority lock holder with medium-priority work blocks high-priority consumers indefinitely. Priority inheritance resolves this vulnerability by temporarily elevating the lock holder's priority to match that of the blocked high-priority task, allowing it

to complete its critical section promptly. In robotic actuator and motor-control paths, engineers must either rigorously bound blocking duration (B_i) using priority inheritance protocols or eliminate shared locks altogether through lock-free communication structures.

4.5.2 Event-triggered and time-triggered bus architectures

Once the permission tick finishes its check on the microcontroller, the admitted setpoint still has to cross a shared physical wire to reach the motor drives, where the same blocking and interference questions reappear. If a high-priority servo command arrives at a shared bus just after a lower-priority diagnostic frame has started transmitting, the command cannot preempt the bits already on the wire and must wait for the frame to finish. Physical buses handle shared medium access in one of two ways. On an event-triggered bus, nodes transmit whenever new data is ready and resolve collisions through dynamic bus arbitration. On a time-triggered bus, a fixed global schedule assigns dedicated transmission windows to each node in advance. Neither design guarantees bounded delivery by its name alone; establishing a worst-case bound requires pricing payload sizes, competing traffic, error-recovery overhead, and device controller delays.

The Controller Area Network (CAN) and its CAN FD variant resolve message collisions via bitwise priority arbitration. While a high-priority frame preempts lower-priority traffic competing for an idle bus, it must wait for any frame already in transmission to finish. Switching to CAN FD accelerates payload serialization by elevating the data-phase bitrate, yet it retains the rule that an in-progress frame cannot be preempted. The blocking duration depends on the longest eligible frame at its nominal and data-phase rates. A valid worst-case response budget must therefore combine both the frame's own transmission time and the maximum duration of a lower-priority transmission already in progress.

EtherCAT adopts an on-the-fly time-triggered model: an Ethernet coordinator issues a single master frame that traverses all connected motor drives in a physical ring. Dedicated hardware controllers in each drive read incoming setpoints and write sensor feedback directly into designated bit offsets as the Ethernet frame streams through the physical layer. This architecture eliminates arbitration contention across the motor drives, while distributed clock synchronization synchronizes local PWM switching cycles. Time-Sensitive Networking (TSN) achieves comparable determinism over switched Ethernet through IEEE 802.1Qbv time-aware traffic shapers. A TSN delivery guarantee relies on synchronized network gates, bounded clock skew, and strict admission controls that gate best-effort traffic.

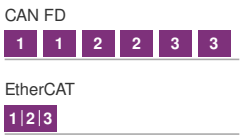
For an integrated mobile manipulator, the core architectural question is whether cyclic actuator commands and feedback fit within the allocated bus period while preserving sufficient margin for MCU control loop execution and motor drive processing. High-bandwidth perception streams (such as camera and LiDAR data) and asynchronous supervisory middleware require separate networks or prioritized channels to prevent unmodeled queuing delays from invading the motor command path. Industrial controllers build that separation into the hardware (figure 4.5), with ports that serve only the deterministic drive network and local I/O attached through terminals on the controller itself rather than through host software. The running machine does not use this controller; the photograph shows the pattern its own permission path follows.

Clock synchronization aligns the sampling and actuation instants across distributed drives;[10] however, phase alignment reveals nothing about the transport latency required for a newly computed setpoint to traverse the physical bus. Even if every drive clock fires simultaneously, the actuators will merely execute stale commands if transit delays stall delivery. A sound design therefore requires independent bounds for both command transport latency and multi-axis synchronization jitter, starting with whether the serialized frames fit inside the cycle at all.

4.5.3 Comparing bus occupancy on CAN FD and EtherCAT

If all nine servo axes of the mobile manipulator share a single CAN FD bus, arbitration headers and worst-case bit stuffing push the command and feedback frames alone past the 1 ms bus cycle, before any controller delay or bus contention is counted. To keep each link within its cycle budget, communication links on the mobile manipulator separate into three tiers by rate and scope:

↪ The protocol engines and analytical response-time models of concrete fieldbus deployments are developed in section C.11.



CAN FD arbitrates a separate frame per message; EtherCAT carries every drive in one frame.

[10] **Distributed Clock**: EtherCAT Distributed Clocks (DC) synchronize distributed drive nodes to a reference clock by measuring propagation delays around the ring. Receive-time latches at each node let the coordinator discipline every node's timer, so the hardware `SYNC0` events that start the drives' update cycles occur in close alignment. Clock alignment alone does not bound the complete control cycle, which must also account for controller response time and PWM gate update latency.

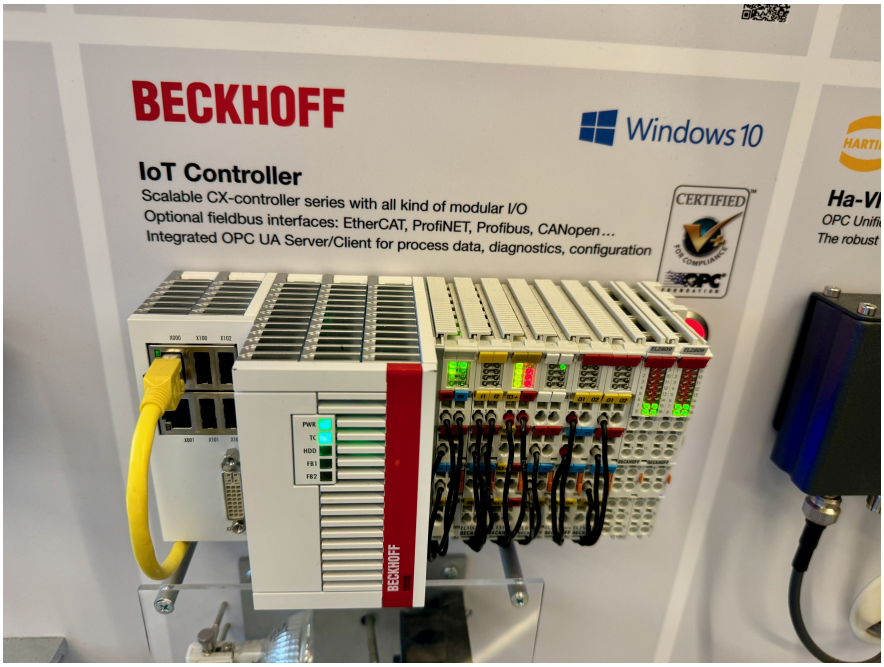


Figure 4.5: Real-time distributed controller with modular I/O: Dedicated embedded real-time controller and adjacent hardware I/O modules. The unit acts as a deterministic bus coordinator, synchronizing distributed sensor acquisition and motor drive terminals via hardware-scheduled Ethernet telegrams. (Credit: Oleg Yunakov, Wikimedia Commons, CC BY-SA 4.0.)

1. **Board-Level Peripheral Interconnects (SPI, Memory-Mapped I/O):** Synchronous SPI links and memory-mapped register transfers link local microcontrollers directly to joint position encoders, current-shunt analog-to-digital converters (ADCs), and inverter gate drivers with sub-microsecond latency ($< 1\,\mu s$) along rigid, hardware-timed schedules.
2. **Deterministic Drive Command Buses (EtherCAT, TSN Ethernet):** Deterministic industrial networks transport cyclic joint position setpoints and motor feedback across all distributed drive inverters within an allocated 1 ms bus cycle (T_{bus}), maintaining sub-microsecond synchronization jitter ($\sigma_{\text{jitter}} < 100$ ns).
3. **Chassis and Auxiliary Diagnostic Buses (CAN FD):** Priority-arbitrated buses service lower-rate peripherals, such as battery management systems (BMS), thermal monitors, and chassis diagnostics, where arbitration delays remain acceptable under verified worst-case utilization bounds.

None of these bounds belongs to a protocol acronym alone. Each guarantee depends on the physical network topology, transceiver hardware, payload geometries, and cyclic schedules. The first check is whether every servo axis can exchange its command and feedback payloads within the permission path's allocated bus cycle budget.

Napkin Math 4.3: Fieldbus serialization before schedulability

Problem: The mobile manipulator's permission path exchanges position setpoints and feedback with its servo axes once per bus cycle. How much of that cycle do CAN FD frames, or one EtherCAT frame, occupy before controller and device delays are counted, and which buses pass this necessary check?

Variables:

- 9 servo axes, 7 arm joints and 2 drive wheels (datasheet), exchanged every 1 ms bus cycle (illustrative).

- An 8-byte record per axis in each direction, a 32-bit position and two 16-bit words (chosen).
- CAN FD at 1 Mbps nominal and 5 Mbps in the data phase (chosen). A CAN frame carries one message, so each axis needs a command frame and a feedback frame. The arbitration field and the trailer run at the nominal rate, 34 bits; the data-length code, payload, stuff count, and 17-bit CRC run at the data rate, 113 bits (International Organization for Standardization 2015a; Robert Bosch GmbH 2012). Both counts include the worst-case 21 dynamic stuff bits, because a bound must assume the bit pattern that lengthens the frame most.
- EtherCAT at 100 Mbps (datasheet), one frame carrying every axis's command and feedback with a 64-byte header, and a forwarding delay of 0.6 µs per node, the shift-register pass-through of a slave controller that never buffers the frame (both illustrative).

Math: One CAN FD frame, then all frames on one bus and on an arm bus:

$$t_{\text{frame}} = \frac{34\ \text{bits}}{1\ \text{Mbps}} + \frac{113\ \text{bits}}{5\ \text{Mbps}} = 34 + 22.6 = 56.6\ \mu\text{s}$$

$$18 \times 56.6 = 1018.8\ \mu\text{s} > 1000\ \mu\text{s}, \qquad 14 \times 56.6 = 792.4\ \mu\text{s}$$

The base's 4 frames take 226.4 µs. For EtherCAT, serialization plus forwarding:

$$\frac{(64 + 9 \times 16)\ \text{B} \times 8}{100\ \text{Mbps}} + 9 \times 0.6\ \mu\text{s} = 16.64 + 5.4 = 22.04\ \mu\text{s}$$

Result: One CAN FD bus fills 101.9 percent of the cycle on serialization alone. A bus per limb passes, the arm bus at 79.2 percent with 207.6 µs left for blocking, competing frames, and retransmissions. EtherCAT occupies 2.2 percent and leaves 978.0 µs unallocated by this calculation.

Systems insight: CAN FD's accelerated data phase cannot salvage a single shared bus topology. Every transmitted frame still incurs arbitration and trailer bit times at the lower nominal rate, and CAN requires distinct command and feedback frames for each joint axis. EtherCAT, in contrast, aggregates all axes into a single circulating telegram processed on the fly. Neither occupancy figure represents an end-to-end response bound. A rigorous CAN FD analysis must incorporate message arbitration, non-preemptive blocking, and retransmissions, whereas an EtherCAT budget must include coordinator cycle jitter, slave hardware propagation, and drive firmware latencies. Any unmodeled delay that creeps into the physical command path consumes stopping clearance at a rate of $v\,\Delta t_{\text{bus}}$.

4.5.4 Bandwidth, response time, and multi-axis synchronization jitter

A network link with plenty of raw bandwidth can still deliver a joint setpoint late if frames queue behind camera traffic. A bus that delivers setpoints quickly on average can still twist a coordinated multi-joint motion if the drives apply their updates at slightly different instants. Analyzing serialization occupancy separates three metrics that industrial datasheets often conflate: raw bandwidth, command response time, and multi-axis synchronization jitter. Bandwidth governs the total payload volume the link can carry per unit time. Command response time measures the complete latency, including queueing, arbitration, transmission, and drive input decoding, before an actuator applies a setpoint. Synchronization jitter bounds the time spread across multi-axis updates. A network design can improve any one of these metrics while leaving the others unconstrained.

Set against the cycle, the notebook's occupancies separate into one failure and three passes (figure 4.6). Only the single shared CAN FD bus fails, needing 1018.8 µs before any contention or retransmission is counted. The per-limb buses (792.4 µs for the arm) and EtherCAT (22.0 µs) leave room for controller execution and clock synchronization. Staying within the cycle budget provides neither a synchronization-jitter bound nor a complete response bound. A defensible CAN FD analysis requires upper bounds on asynchronous message interference and non-preemptive blocking. An EtherCAT path demands explicit master cycle jitter, slave hardware propagation, and

drive firmware latencies. A TSN schedule requires reserved transmission windows and bounded clock-synchronization errors.[11] These factors are required inputs to the response-time equation, not inherent guarantees achieved by selecting a protocol standard.

Actuator commands also demand strict temporal freshness. Even when delivered comfortably within a deterministic transport window, a setpoint computed from stale perception is functionally hazardous. The evidence timestamp and lease expiry embedded in the command proposal remain safeguards even over nanosecond-synchronized fieldbuses. The transport budget must reserve adequate headroom for physical actuation to execute, and the real-time permission path must reject any setpoint whose authoritative validity window has expired.

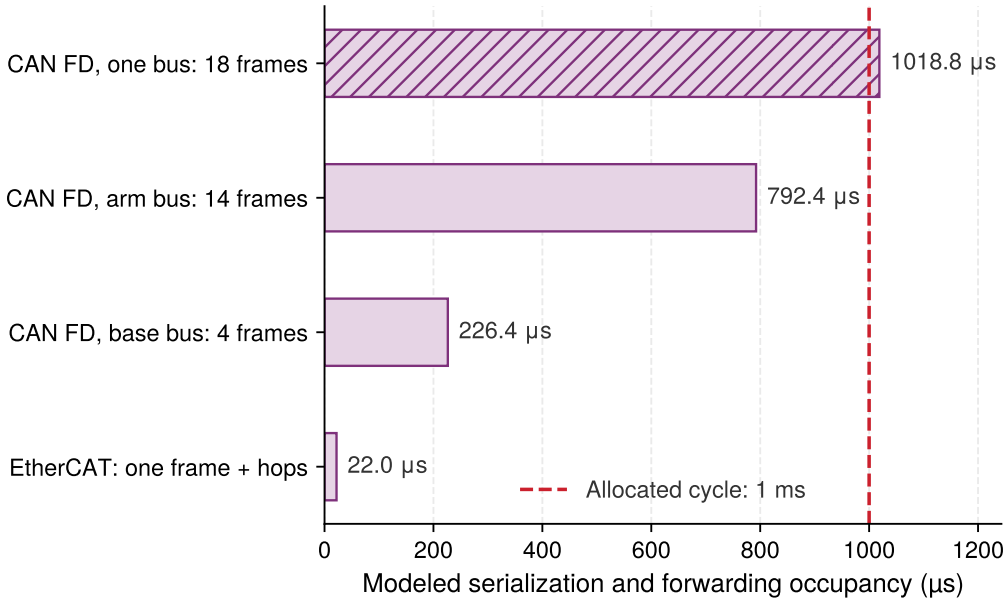


Figure 4.6: Scenario bus occupancy before response qualification: Modeled serialization and forwarding terms for the configured servo-axis example, command and feedback included, compared with the allocated bus cycle. One CAN FD bus for all axes exceeds the allocation before other delays are counted. A CAN FD bus per limb and EtherCAT leave unallocated time, but controller response, competing traffic, device delays, and fault allowances still need qualification. These calculated values are not measured end-to-end response or synchronization bounds.

Checkpoint 4.1: Response bounds and command transport

The lease budget counts the permission tick and the bus cycle as bounded terms. These items test whether you can supply the bounds it assumes.

- ☐ **Percentile vs. bound**: Can you explain why a measured p99 response time, however long the test that produced it, may enter the handoff record only as a measured distribution and never as a proven bound?
- ☐ **Response-time estimate**: If the longest tick-masking section in the link driver grew from 50 µs to 120 µs, with the tick's WCET and interference unchanged, would the tick still meet its 400 µs deadline, and by how much would it pass or miss?
- ☐ **Bus occupancy estimate**: With one command frame and one feedback frame per axis at 56.6 µs each, how many axes can one CAN FD bus serve within a 1 ms cycle before any blocking is counted?
- ☐ **Synchronization vs. latency**: Can you explain why tightly synchronized drive clocks do not bound how long a newly issued command takes to reach the drives?

[11] **Time-Sensitive Networking**: IEEE 802.1Qbv Time-Sensitive Networking provides bounded Ethernet delivery by configuring time-aware traffic shapers across network switches. High-priority control frames travel inside dedicated, pre-scheduled transmission windows, while asynchronous high-bandwidth traffic (such as uncompressed camera streams) is held back at the switch gates. This scheduled time-division multiplexing lets camera streams and microsecond servo commands share a physical link without packet collisions or switch-queue buildup.

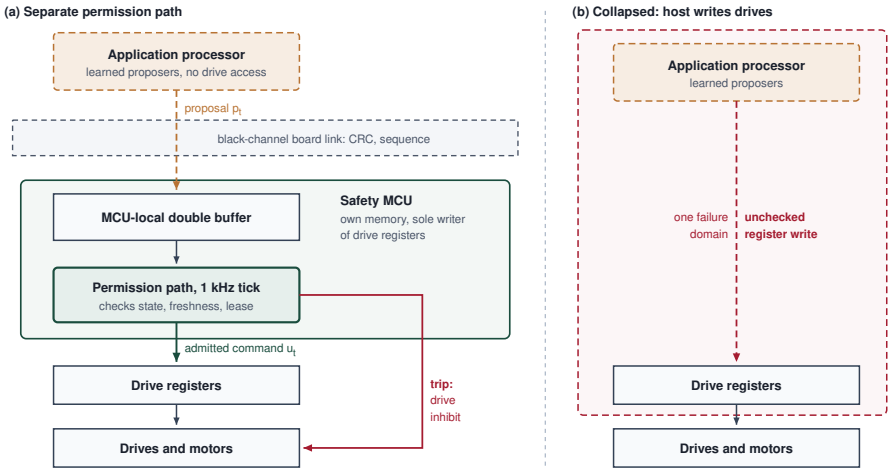


Figure 4.7: Hardware proposal boundary: (a) The application processor proposes across a black-channel board link into the safety microcontroller's own double buffer; the permission path on that microcontroller is the sole writer of the drive registers and can trip a drive inhibit. (b) When the application processor can write the drive registers directly, an unchecked write reaches the motors, and proposer and drives share a single failure domain.

A fieldbus path qualified under declared traffic and fault conditions bounds command delivery within its allocated cycle, but on-time delivery says nothing about whether the command should have been sent. That depends on which node may issue setpoints at all, and on whether each setpoint is checked against the machine's physical limits before it leaves.

4.6 Actuation Authority

A shared mailbox cannot protect a mobile manipulator if the host processor retains a diagnostic route, an auxiliary bus master, or a JTAG debug port that writes directly to a drive. The silicon partition established in section 4.2 therefore requires exclusive physical control over every electrical path capable of energizing the motor windings. Actuation authority is this exclusive physical ability to modify drive states, encompassing PWM configuration registers, current references, gate-driver outputs, and bridge-enable lines. Under this architecture, the host publishes candidate proposals p_t and the safety microcontroller's permission path alone turns them into admitted commands u_t. The difference between holding that authority and merely appearing to is a question of wiring (figure 4.7). When the permission path is the sole writer of the drive registers and can trip a drive inhibit, every motion passes its checks. When the host can also write those registers, one unchecked write reaches the motors, and proposer and drives share a single failure domain.

The microcontroller continuously evaluates physical safety invariants: verifying that commanded joint positions lie within certified reach envelopes, that joint velocities stay within qualified limits, and that projected decelerations remain safely within dynamic stopping clearance ($d_{\text{stop}} \leq D_{\text{clear}}$). When a proposal satisfies all invariants, the microcontroller admits the command and interpolates waypoints into cyclic position setpoints for each bus cycle, which the drives track through their internal velocity and current loops. When a candidate proposal violates an invariant or expires, the enforcer refuses it ($\perp$). A refusal does not automatically revoke a previously admitted, actively executing trajectory chunk. Instead, the active chunk continues running as long as its state, sensor evidence, and chunk lease remain valid. When those conditions lapse or fail, the enforcer terminates execution authority and transitions the system into a controlled deceleration or hold. In parallel, a separate hardware trip zone stands ready to cut motor drive torque upon catastrophic electrical faults, though de-energizing an inverter does not by itself brake moving mass.[12]

At the causal boundary the root cause of an aberrant proposal does not matter. A remote memory exploit in a host operating system daemon, an adversarial optical perturbation that induces a vision model hallucination, and an arithmetic overflow in an acceleration tensor engine can all produce invalid proposals for the same boundary filter to reject. A proposal that passes the physical checks

↪ How exclusive authority over each actuator channel passes between the learned policy, an operator, and the fallback is developed in section 14.3.2.

12 **Hardware Trip Zone**: Microcontroller hardware trip zones couple analog overcurrent and overvoltage comparators directly to timer output logic. Upon fault detection, configured trip logic may force gate signals high-impedance or activate a validated braking mode. These have different mechanical consequences: torque removal can leave the plant coasting, while dynamic braking requires a rated energy path.

↪ The refusal authority and the state invariants it checks are developed in section 12.6.

may still pursue the wrong task, so admission does not establish semantic correctness. Because the enforcer evaluates physical invariant bounds against direct sensor feedback from optical encoders and current shunts rather than inspecting internal neural network weights, it requires no internal model of why the proposer failed. Because the host lacks bus-master privileges to actuator registers, a compromised host has no bypass around this check. Complete protection against a rogue host nonetheless requires four physical foundations: independent sensing, an accurate kinematic and dynamic limit model, sufficient actuator capability, and a physically feasible fallback trajectory.

Controlled stopping and torque removal impose different engineering obligations. Powered braking actively uses the drive inverters to decelerate the plant to zero velocity before disabling torque. Immediate torque removal, by contrast, de-energizes the motor windings instantly, which can leave a wheeled base coasting uncontrollably or allow a vertically loaded manipulator arm to collapse under gravity. Maintaining a held position requires either continuous motor torque or a certified mechanical restraint. At the actuation authority interface, the systems designer must explicitly designate the physical fallback response and reserve both the execution time and thermal energy budget that it demands.

↩ The causal boundary, where a command becomes current in a winding and mass begins to move, is established in section 1.4.1.

A refusal that leaves no admissible active continuation carries an unavoidable physical cost. When the enforcer rejects an invalid proposal and commands dynamic braking on a fast-moving joint, the physical braking path must supply sufficient torque, clearance, and thermal dissipation capacity to arrest the moving mass. Over an arrest interval spanning several hundred milliseconds, the motor winding resistance, dynamic brake chopper resistor, or friction brake lining must absorb the joint's accumulated kinetic energy within the thermal limits detailed in section 2.5. This dissipation must occur without degrading winding insulation or permitting the manipulator to breach its physical clearance boundary.

If the safety microcontroller itself hangs or suffers a clock failure, it cannot execute a software-directed stop. A hardwired, out-of-band interlock must therefore supervise the system and enforce a qualified fallback state. Safe Torque Off (STO) accomplishes this by disabling gate-driver optocouplers or charge pumps through hardware logic, preventing the inverter from generating rotating magnetic fields.[13] STO does not isolate the high-voltage DC bus from the mains, nor does it guarantee that momentum will dissipate within the stopping clearance allocated for powered deceleration. Complete electrical isolation requires an upstream safety contactor, and containing gravitational drop or mechanical drift requires a certified electromechanical brake. A dual-channel interlock circuit (the relay network of figure 4.8) decouples torque inhibition entirely from the host software stack. Safety qualification must still verify that the system remains resilient against common-cause failures in power rails, clock crystals, and harness wiring.

↪ The stopping-distance model this braking path must satisfy is developed in section C.13.

In figure 4.8, dual positive-break E-stop contacts (S_1, S_2) drive independent force-guided relays (K_1, K_2), which route dual STO enable signals into the motor drive. Series-wired normally closed external device monitoring (EDM) contacts prevent system restart if either relay fails to open. At the bottom, a dedicated brake supply drops a spring-applied mechanical brake directly onto the motor shaft upon power loss, enforcing torque removal ($\tau_{\text{motor}} = 0$) and holding restraint without software or processor intervention. However, engineers must not confuse a static holding brake with a dynamic stopping brake. Under normal operational sequences, the drive decelerates the motor to zero velocity electrically before engaging the spring brake to hold position. Dropping a holding brake onto a spinning shaft during a high-speed emergency stop subjects the brake pads and hub to severe friction heating and mechanical shock, requiring independent thermal rating and stopping-distance validation. The stop-category selections and timing sequences depend directly on these electromechanical limits.

[13] **Safe Torque Off (STO)**: An STO implementation removes torque-producing drive signals through a hardware interlock. An IEC 61800-5-2 integrity rating depends on the actual devices, diagnostic coverage, wiring, and validation, not on the circuit pattern alone (International Electrotechnical Commission 2016b).

> Systems Perspective 4.1: Hardware authority invariant
>
> If an unverified learned policy has direct memory-mapped write access to actuator gate-driver registers, the machine has no mediated proposal path, and its next out-of-distribution input reaches the motors unchecked.

Write privilege is a physical reality dictated by memory mapping, bus arbitration, and physical trace layout, never a configurable software flag. The host processor holds no write privilege to drive

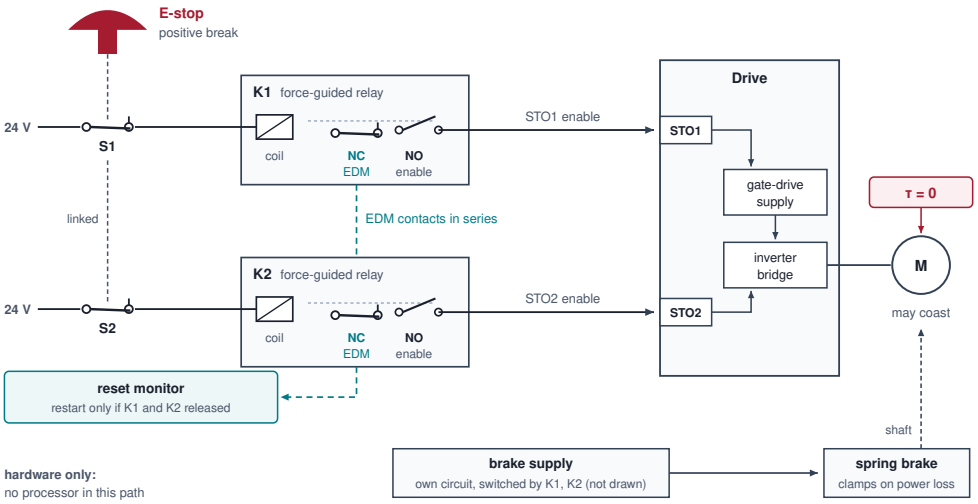


Figure 4.8: Illustrative out-of-band interlock and STO path: Each emergency-stop contact holds its own force-guided relay, and each relay's enable contact holds one of the drive's two STO inputs, so a single welded contact or shorted wire leaves the other channel able to remove torque. The EDM contacts report a relay that failed to release before restart. Removing torque leaves the motor free to coast; the spring-applied brake is a separate circuit that clamps when its supply is removed. The diagram shows a hardware response path, not a certified integrity level or a stopping distance (synthesized from ISO 13849-1 and IEC 61800-5-2).

registers. While hardware partitioning determines which processor may issue physical commands to an actuator, the engineer must still decide what physical quantity those admitted commands specify, because each choice leaves a different set of failure modes for the permission path to catch.

4.6.1 What a proposal may command

A digital motor drive accepts setpoints across a nested cascade of control loops: position, velocity, and winding current, with current running innermost at the highest bandwidth. Sending raw current targets directly to the drive bypasses its position and velocity loops and leaves the permission path to predict how gravity, inertia, and external contact will move the joint. Sending pure position targets lets the drive bound its own speed and current, at the cost of ramping winding current against an unmodeled obstacle until the inverter saturates or the mechanism breaks. The level at which a proposal enters this cascade therefore decides which physical states the drive hardware regulates on its own and which quantities the permission path must verify before admitting the command. Four command interfaces dominate physical AI systems, each placing a different prediction and checking burden on the enforcer.

A *position target* q_{des} delegates the entire trajectory tracking problem to the drive's internal cascaded loops. Because the drive firmware enforces its own configured speed and current limits, the enforcer's verification burden remains relatively compact. It validates that the target lies within the geometric reach envelope, bounds the discrete step between consecutive targets (which position gains convert directly into velocity and torque demands), and ensures adequate dynamic stopping clearance. However, a pure position target cannot modulate interaction force. When encountering an unmodeled rigid obstacle, a stiff proportional-integral position loop interprets physical contact as a tracking error to be eliminated, ramping winding current until the inverter saturates or the mechanical structure breaks. The impedance-control analysis in section 2.4.4 quantified this exact failure mode on a cage-door latch.

A *velocity target* $\dot{q}_{\text{des}}$ shifts kinematic integration to the host proposer, bypassing the drive's position loop. Because the proposal omits an explicit geometric endpoint, the enforcer cannot simply validate a target coordinate against static workspace limits. Instead, the enforcer forward-integrates the commanded velocity over the duration of the active chunk lease to predict the joint's worst-case

state at lease expiration. It then checks both the reach envelope and the dynamic stopping distance against this forward projection. As with pure position control, contact force remains unregulated, and unmodeled collisions will produce maximum drive torque.

A *torque target* τ_{des}, delivered as a direct current reference, bypasses both outer loops. The drive regulates only electromagnetic winding current. Consequently, all kinematic state evolution, including velocity, position, and braking distance, becomes an emergent consequence of plant dynamics, gravity, and external contact forces rather than an enforced setpoint. The enforcer must bound the resulting state evolution and preserve a feasible fallback, using a plant model or effort-space constraints, while constraining torque slew rates and tracking the cumulative mechanical energy injected into the system. Torque control offers maximum responsiveness for dynamic locomotion and manipulation, but it imposes the heaviest computational burden on the permission path while providing the least explicit intent.

An *impedance target* specifies a virtual equilibrium pose alongside stiffness gains K_p and damping gains K_d, so the actuator behaves as a virtual spring-damper (Hogan 1985), the same model introduced in section 2.4.4:

$$F = K_p(x_{\text{des}} - x) + K_d(\dot{x}_{\text{des}} - \dot{x})$$

Its dynamic dual, an *admittance target*, measures external contact force via load cells and commands motion in response. Both formulations explicitly regulate the dynamic relationship between motion and force, a coupling that position, velocity, and torque interfaces leave unconstrained. This capability makes impedance and admittance formulations standard in contact-rich manipulation tasks. The enforcer's verification role expands accordingly: it must evaluate both the equilibrium setpoint and the commanded stiffness gains. Under quasi-static contact, the maximum contact force is bounded by the product of the commanded stiffness and the largest displacement the environment can impose. Consequently, a stiffness parameter that exceeds the structural limits of the workpiece or fixture is just as inadmissible as a trajectory crossing a workspace wall. However, steady-state gains do not govern transient impact forces, which depend heavily on approach velocity, rotor inertia, structural compliance, and control-loop latency. Section 12.7.2 establishes the energy and contact-force barriers needed to bound these transient impact phases.

On the reference mobile manipulator, the enforcer admits position waypoints, interpolating them into fine setpoints for each bus cycle while the motor drives close their own internal velocity and current loops. When the manipulator engages the sprung latch of an equipment cage door, the host switches to an impedance command, introducing stiffness and damping parameters that the real-time tick must validate before transmission. No command interface eliminates the verification burden of the permission path: shifting the command interface inward, from position setpoints toward raw current references, strips away explicit geometric intent and forces the safety layer to predict complex electrodynamic behavior. Write privilege and command filtering protect the physical plant only if the enforcer also maintains physical independence from the host whose proposals it evaluates. If the safety microcontroller and the host application processor share a power rail, a common system clock, or a unified memory bus, a failure in the host can brown out the microcontroller, corrupt its timing, or deadlock the bus. Any of these silences the safety layer without issuing a single illegal drive command.

4.7 Hardware Isolation and Fault Containment

A large current transient during a neural accelerator burst can pull down a shared power rail and brown out or reset the permission processor without warning, while memory bus contention on a shared interconnect can delay safety checks past the hard real-time execution tick. Even when safety enforcement executes on a dedicated processor, physical coupling to the proposal engine can silently undermine software isolation. Containing faults across the two paths therefore requires isolating or bounding shared physical resources across four domains: execution hardware, reference oscillators, power distribution rails, and memory hierarchies.[14] Where any of those resources remain shared, the architecture must demonstrate that cross-domain interference and declared common-cause faults cannot prevent the permission path from meeting its execution deadline or initiating its qualified fallback.

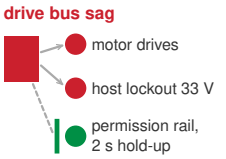
The running machine's two-processor architecture addresses each of these four fault paths with an explicit hardware design choice. The permission path executes from private memory over a

[14] **Freedom from Interference (FFI)**: ISO 26262 defines FFI in its vocabulary, Part 1, as the absence of cascading failures between two or more elements that could lead to the violation of a safety requirement (International Organization for Standardization 2018). Part 6 lists the software interference it covers (timing and execution, memory, and exchange of information), whereas shared power, clocks, and heat are common-cause and cascading faults that Part 9 treats in its dependent-failure analysis and that IEC 61508 addresses as independence between elements (International Electrotechnical Commission 2010b). A neural policy sharing silicon with the permission path therefore needs both arguments, FFI for its software and a dependent-failure analysis for the resources underneath.

dedicated board-level link, clocks from an independent oscillator, and draws power from the isolated rail specified in section 2.8.2 rather than from the primary 47.7 V drive bus feeding the application processor. Diode-isolated storage capacitors hold up this safety rail for 2 s. If a transient load fault pulls the drive bus down to the host converter's undervoltage lockout threshold at 33 V, the safety microcontroller stays powered long enough to capture the undervoltage interrupt, log the event, and command a graceful fallback while the primary host resets. This holds only if the required detector, actuator power, and braking path survive the fault.

Keeping the logic powered does not guarantee motor torque or braking capability. Bringing a moving machine to a controlled stop requires sufficient DC bus energy, healthy inverter stages, and adequate obstacle clearance. If electrical drive power collapses entirely, the system must execute a qualified unpowered fallback whose mechanical brake engagement delay and subsequent stopping motion fit within the pre-allocated physical envelope. These isolation guarantees are design assertions that must be empirically verified on physical hardware. Every shared dependency must be carried forward into the formal verification handoff record.

↪ The four substrate couplings that board layout must sever, and the fault-injection experiments that test their isolation, are developed in section 13.2.



A sag on the shared bus reaches every load; the permission rail rides through on its own.

Substrate isolation ensures that the permission path stays alive when the host crashes, but the enforcer's downstream action depends entirely on how the proposer failed: through unresponsiveness or through corruption. Unresponsiveness occurs when an inference pipeline deadlocks, thread scheduling misses a deadline, or kernel memory fragmentation stalls process execution, yielding no new trajectory packets. A complete host crash trips the hardware heartbeat timer. Conversely, the host operating system might remain healthy while the user-space neural policy pipeline hangs; in that scenario, the proposer stops emitting admissible action chunks, allowing the active chunk's lease or evidence deadline to lapse. These complementary deadlines form the supervisory timing hierarchy established in section 4.4.1.

Corruption occurs when the proposer continues running and satisfies its heartbeat deadline, but outputs invalid or hazardous setpoints. Examples include kinematically unreachable joint configurations, step discontinuities in velocity requests, or floating-point NaNs and infinities caused by out-of-distribution visual tokens or matrix overflow. The enforcer handles these two failure profiles through orthogonal mechanisms: unresponsiveness triggers trajectory revocation as soon as the chunk lease or the heartbeat timer expires, whereas corruption prompts an immediate invariant rejection at the boundary filter upon the arrival of the malformed packet. A live heartbeat therefore never extends motion authority; only an admitted chunk renews it, and authority ends when the chunk lease runs out. Whether the chunk policy can renew quickly enough to keep the machine moving, and why the slower intent model supplies goals rather than holding chunk leases itself, is the coupled budget of section 4.9.1.

When active motion authority lapses without an admissible continuation, the permission path must execute an explicit degradation strategy rather than cutting motor bus power unconditionally. As detailed in section 2.6.2, de-energizing an inverter drops the actuators into uncontrolled freewheeling or dynamic short-circuit braking, neither of which produces an orderly, predictable stop. Instead, the safety path evaluates the plant's instantaneous kinematic state and selects the safest feasible deceleration profile. An optional brief continuation along the last validated trajectory is permissible only while its state estimates, sensor evidence validity, chunk lease window, and obstacle clearance margins all remain valid.

Once that lease expires, any separately authorized fallback delay T_{hold} is absorbed into the total reaction latency τ_{delay}. It must fit, together with the evidence age, within the 176.3 ms left under the pre-brake ceiling once the lease path and brake onset are charged at the drive limit (notebook 4.1). If measured latency bounds exceed this headroom, the enforcer must initiate deceleration sooner. Controlled braking uses the available actuator torque characterized in section 2.4 to arrest motion within the remaining clearance. On gravity-loaded manipulators, motion arrest must be backed by engaging rated mechanical holding brakes once the joints reach standstill, preventing gravitational backdriving when drive power is subsequently removed.

After the physical plant comes to a complete halt, the architecture executes a **recovery handshake** before restoring autonomous proposal authority. The safety microcontroller asserts a persistent `FAULT_HELD` latch in its exported status register and publishes the plant's confirmed physical rest pose $(\mathbf{q}_{\text{rest}}, \dot{\mathbf{q}} = \mathbf{0})$. In turn, the application processor must discard all pending action chunks,

invalidate internal state tied to the interrupted motion, capture fresh sensor observations tagged with synchronized hardware timestamps, and formulate a newly aligned trajectory proposal.

This recovery sequence must also purge any proposal-side state that assumes the robot executed its planned trajectory to completion. For autoregressive policies and VLA models, this requires flushing token buffers and the key-value (KV) attention cache; retaining pre-fault keys and values would bias future action generation toward a physical trajectory that was interrupted. However, clearing software caches does not by itself establish physical continuity. Before deasserting the `FAULT_HELD` latch and restoring proposal authority, the safety microcontroller inspects the opening waypoint $\mathbf{a}_0$ of the incoming proposal, verifying that it aligns with the robot's measured physical rest pose within a strict spatial bound ($||\mathbf{a}_0 - \mathbf{q}_{\text{rest}}|| \leq \epsilon_{\text{sync}}$).

Architectural independence cannot simply be read off a schematic or block diagram; it must be empirically validated against identified common-cause faults on the physical board. Similarly, the chunk lease that constrains a malfunctioning proposer derives directly from physical dynamics, sized from the stopping margin that remains after detection delays and actuator responses. Together, these defenses bound a proposer that crashes, stalls, or publishes corrupted actions. The separation leaves one question open, whether a neural policy engineered to execute deterministically at the enforcer's own update rate still requires an external runtime monitor.

↪ The fault injection that tests this independence on the physical board is developed in section 15.8.

4.8 Invariant Checking

A proposer fast enough to keep pace with the permission tick invites a shortcut. Learned locomotion controllers already emit joint targets within an order of magnitude of a millisecond, and a designer may be tempted to let such a policy write the motor registers directly. Rate does not confer authority. A fast network emitting joint targets or torques carries the same statistical uncertainty as a large model generating waypoints a few times a second. Even when policy and enforcer share one period, the network's output remains a proposal that must pass the proposal boundary before it changes current in a winding. What the permission path can still guarantee when a learned policy commands at control rate, and what it gives up, is examined in section 12.8.1.

The interface must reject an inadmissible candidate without waiting for the producer, and recheck the active command against current physical state. Position and velocity limits ($\mathbf{q} \in [\mathbf{q}_{\min}, \mathbf{q}_{\max}]$ and $|\dot{\mathbf{q}}| \leq \dot{\mathbf{q}}_{\max}$) are necessary but do not by themselves establish a feasible stop. A torque-slew constraint, $|\dot{\tau}| = |(\tau_k - \tau_{k-1})/\Delta t| \leq \dot{\tau}_{\max}$, can limit command changes, but a command that meets the slew limit must still satisfy the other physical limits. Contact may additionally require accounting for exchanged energy, $\int \tau^\top \dot{\mathbf{q}}\, dt$.

↪ The passivity framework that accounts for energy exchanged in contact is developed in section B.10.

Choosing and composing those filters is the work of chapter 12, which constructs the admissibility calculation and selects a feasible continuation, hold, or stop. What the Nervous System fixes is the execution contract those filters must run under, whatever they compute. On the running machine the contract has numbers. The recheck of the active command must finish inside the tick's 400 μs deadline under the response bound of equation 4.3. It must read measured wheel and joint encoder state rather than commanded setpoints (the fallacy of section 4.10 shows what the commanded setpoints miss). It must also leave the safety microcontroller the only writer of the drive registers. Neither fast inference nor a fast rejection routine establishes that contract on its own.

The same reasoning covers the one proposer that is not learned. A remote operator is a third proposer. Its commands pass the same permission path, and network delay spends the same pre-brake budget. Neither speed nor a human origin gives a proposal authority; only passing the permission path's checks does, and each check spends part of a budget that the next section lays out handoff by handoff.

↪ How an operator's reaction and link delays are charged against the stopping budget is developed in section 14.2.2.

> Checkpoint 4.2: Hardware isolation and invariant checking
>
> Before turning to the handoffs between proposer and actuator, verify your understanding of the exchange, hardware isolation, and pre-actuation invariant defenses:
>
> - ☐ **Nonblocking exchange**: Can you explain how the seqlock reader detects a torn read in one bounded attempt, and why a writer stopped mid-write cannot extend the active spline's state, evidence, or lease validity?

- ☐ **Independence as a hypothesis**: Can you state the independence requirement the permission path must meet, and explain why the two-processor split satisfies it only once loaded tests on the machine's silicon support the claim?
- ☐ **Chunk lease cost**: Can you explain why the chunk lease must be sized from stopping distance and clearance rather than chosen as a software timeout, why a heartbeat never renews it, and how much clear distance lengthening it from 60 ms to 100 ms would spend at the 1.5 m/s drive limit?
- ☐ **Command interface**: Can you state what the permission path must check or predict for a position, a velocity, a torque, and an impedance target, and why a velocity target leaves it to predict motion over the whole lease?
- ☐ **Rate matching and authority**: Can you explain why a policy that runs at the plant's own rate still produces only proposals, and what timing, state, and fallback obligations remain?

4.9 The Handoffs Matrix

When a mobile manipulator overruns a stop, the diagnosis begins by locating the fault at one of five canonical handoffs between the photons collected by its cameras and the current delivered to its drives. A cloud service governs its interfaces with statistical service-level objectives (SLOs), such as a p99 request latency; when a tail event violates that target, the runtime absorbs the delay through retries, request rerouting, or graceful degradation. The five handoffs of an embodied machine define **hard cyber-physical service-level objectives** instead. Software cannot recall physical momentum or undo resistive heating in an actuator winding, so no retry mechanism can mask an unbudgeted interface violation.

Upstream, sensing hardware transfers raw transducer signals to perception pipelines. Perception yields intermediate geometric and semantic representations that fuse into an estimated world state. This state feeds the learned policy to produce an action plan, which in turn submits candidate trajectories to the safety enforcer. Finally, the approved command passes from the enforcer to the physical motor drives. Across each boundary, five interface limits govern feasibility: latency (L_{lat}), bandwidth (BW), energy and heat (P_{heat}), determinism (σ_{jitter}), and fault domain isolation ($\mathcal{F}$). The five handoffs and five interface limits form a five-by-five matrix mapping every signal path to the physical budgets of the machine. These interface limits are distinct from the body's five mechanical budgets, which enter the matrix primarily through its latency and energy columns. The contract that records these budgets, the handoff record defined below, joins the physical limit records of section 2.7.3 to the proposer limits of the Brain. Latency, renewal rate, and evidence age are therefore written in one schema.

↩ The body's five mechanical budgets are established in section 2.1.

The allocation in table 4.3 illustrates which limits are design-limiting at each handoff, using chosen design budgets rather than empirical measurements. The matrix gathers the physical budgets of the machine's mechanical, cognitive, and sensory systems, together with the sensor capture that the five-level machine model places at the Boundary. In the Boundary row ($\mathcal{H}_1$), where the physical world becomes digital measurement, bandwidth and latency bind first: raw multi-camera pixel streams saturate the serializer-deserializer (SerDes) lanes, and the exposure window imposes a strict floor on image age before serialization even begins. The Brain rows ($\mathcal{H}_2$, $\mathcal{H}_3$) bind first on energy and latency, reflecting the heavy inference costs of the learned proposers. The Body row ($\mathcal{H}_5$) binds first on determinism and heat, because the drives' current loops must hold their timing jitter below $10\,\mu$s, and every ampere delivered dissipates heat across the windings and power inverter stages.

The Nervous System row ($\mathcal{H}_4$) forms the architectural bridge coupling the Brain and the Body. The proposal boundary between the learned planner and the enforcer carries an action proposal (p_t), typically structured as a compact spline parameterization or waypoint chunk. For the compact payload assumed in this grid, the required throughput is modest (BW < 100 kB/s), so bandwidth does not bind; instead, latency, determinism (σ_{jitter}), and fault domain isolation ($\mathcal{F}$) govern the interface. The permission path isolates stochastic model proposals using nonblocking mailboxes and expiring leases. On the mobile manipulator, the latency limit is the chunk lease, which consumes 60 ms of the 258.3 ms pre-brake ceiling available at the drive limit (section 4.4), while the active

command is rechecked within each 1 ms permission tick. Full trajectory admission has a separate computation and response budget and must complete before any candidate chunk activates; the tick interval does not imply that arbitrary occupancy queries or whole-trajectory collision checks fit within that brief slice. Fault isolation relies on a private peripheral bus, an independent oscillator, and the isolated, held-up power rail detailed in section 4.7.

From the causal boundary perspective, any interface failure localizes to the specific handoff and budget it violates. Semantic mistakes or flawed physical assumptions can still escape an otherwise timely interface; the matrix does not substitute for task-level evaluation or hazard analysis. When diagnosing an over-budget stop, trajectory overrun, or subsystem fault, engineers navigate table 4.3 through a three-step operational root-cause procedure:

1. *Locate the boundary seam:* Identify which physical handoff ($\mathcal{H}_1$–$\mathcal{H}_5$) recorded the initial timing excursion, sequence gap, torn payload, or thermal trip.
2. *Check the binding dimension:* Query the specific physical limit ($L_{\text{lat}}, \text{BW}, P_{\text{heat}}, \sigma_{\text{jitter}}, \mathcal{F}$) designated as *BINDING* for that handoff cell, identifying the primary resource bottleneck governing the interface.
3. *Evaluate like-for-like evidence:* Compare runtime diagnostic traces directly against the recorded bound kind (`requirement`, `proven bound`, or `measured distribution`), verifying whether the failure stemmed from an unmodeled microarchitectural tail event or a direct physical budget violation.

A handoff diagnosis must strictly compare like evidence. For the $\mathcal{H}_2$ state-estimation path, suppose engineers allocate a 30.0 ms latency budget and measure p99 $= 24.5$ ms during a baseline hardware-in-the-loop run, yielding a sampled margin of $+5.5$ ms. Under thermal throttling, a subsequent run measuring p99 $= 85.0$ ms reveals a -55.0 ms deficit against that design budget. The appropriate remedy is to reduce or reallocate computational workload while preserving independent safety enforcement during revalidation. A sampled p99 latency diagnoses a performance regression; it does not constitute a worst-case timing proof.

DRAFT

Table 4.3: Handoffs-by-limits synthesis grid: A five-by-five matrix mapping five canonical signal handoffs ($\mathcal{H}_1$–$\mathcal{H}_5$) across the Boundary (section 1.7), Brain (chapter 3), Body (chapter 2), and Nervous System (chapter 4) against five physical limits ($L_{\text{lat}}, \text{BW}, P_{\text{heat}}, \sigma_{\text{jitter}}, \mathcal{F}$). Cells marked *BINDING* are the budgets that drive selection; *Non-binding* means the scenario budget meets that dimension with margin. These illustrative allocations organize the timing and isolation evidence still required; a non-binding cell is not a qualification verdict.

Canonical Handoff Boundary	Latency Limit (L_{lat})	Bandwidth Limit (BW)	Energy & Heat Limit (P_{heat})	Determinism Limit (σ_{jitter})	Fault Domain Isolation ($\mathcal{F}$)
$\mathcal{H}_1$: **Sensing** → **Perception** *(Boundary)*	**BINDING** $L_{\text{lat}} \leq 16.7$ ms, one frame period (sensor age dated at the frame's middle row; the oldest row is older, section 8.8)	**BINDING** BW ≥ 1.2 GB/s (multi-camera 4K uncompressed streams)	*Non-binding* $P_{\text{heat}} < 3$ W (sensor head dissipation isolated)	**BINDING** $\sigma_{\text{jitter}} \leq 1.0$ ms (frame-sync triggers prevent skew)	*Non-binding* Defended by SerDes hardware & DMA ring buffers
$\mathcal{H}_2$: **Perception** → **State** *(Brain)*	**BINDING** $L_{\text{lat}} \leq 30.0$ ms (camera and inertial motion estimate & feature window)	*Non-binding* BW ≈ 10–50 MB/s (compressed image features & 3D occupancy cells)	**BINDING** $P_{\text{heat}} \approx$ 15–35 W (edge vision backbone accelerator envelope)	*Non-binding* Defended by sliding-window state estimator buffers	**BINDING** Spatial memory isolation: MPUs prevent memory corruption
$\mathcal{H}_3$: **State** → **Plan** *(Brain)*	**BINDING** chunk policy 40 ms p99 inside its 50 ms renewal period (section 3.5.2); autoregressive intent model 336 ms inside its 500 ms period (section 3.4)	*Non-binding* BW < 5 MB/s (low-dimensional state & prompt embeddings)	**BINDING** $P_{\text{heat}} \approx$ 30–75 W (transformer/diffusion matrix accelerators)	*Non-binding* Defended by receding-horizon chunking & spline buffers	**BINDING** Host faults must not halt the independent permission path

Canonical Handoff Boundary	Latency Limit (L_{lat})	Bandwidth Limit (BW)	Energy & Heat Limit (P_{heat})	Determinism Limit (σ_{jitter})	Fault Domain Isolation ($\mathcal{F}$)
$\mathcal{H}_4$: **Plan** → **Enforcer** *(Nervous System)*	**BINDING** chunk lease 60 ms (candidate allocation; age at admission from table 4.1); active-command checks within each 1 ms tick; full admission budget recorded separately	*Non-binding* Defended by compact parameterized splines (BW $<$ 100 kB/s)	*Non-binding* $P_{\text{heat}} < 0.5$ W (fixed-point constraint evaluation on MCU)	**BINDING** $\sigma_{\text{jitter}} \leq 100\,\mu$s (cycle-deterministic packet reception on MCU)	**BINDING** Hardware boundary: private bus, separate oscillator, isolated held-up rail
$\mathcal{H}_5$: **Enforcer** → **Actuator** *(Body)*	**BINDING** $L_{\text{lat}} \leq 1$ ms (servo interval 1 ms; current loop 50 µs)	*Non-binding* BW ≈ 1–5 Mbps (cyclic packets over deterministic command bus)	**BINDING** I^2R copper winding dissipation, MOSFET switching losses	**BINDING** $\sigma_{\text{jitter}} < 10\,\mu$s (hard real-time PWM phase alignment & FOC)	**BINDING** Galvanic isolation, out-of-band analog E-stops, STO circuits

Definition 4.3: Handoff record

Handoff record is the record that states one interface limit at one of the five handoffs between sensing and actuation: its value, the conditions under which it holds, its uncertainty and margin, the detector that catches an excursion, its provenance, and whether the value is a requirement, a proven bound, or a measured distribution.

1. **Significance**: It is the second record in the record chain, after the limit record of the Body, and the first that ties a timing claim to the interface where the time is spent. On the running machine its $\mathcal{H}_4$ entry is the 60 ms chunk lease, recorded as a requirement whose margin depends on the intervals of table 4.1.
2. **Distinction**: Unlike a limit record, which describes a physical envelope of the plant such as credible deceleration, a handoff record describes an interface between two subsystems and names the clock on which its times are expressed. Unlike a latency in a profiling log, it states its bound kind, so a percentile cannot pass for a worst case.
3. **Common pitfall**: Recording a bench p99 as a proven bound. The record then carries a number that the permission path trusts in conditions the bench never reached.

Every boundary condition in the grid is recorded this way, so the budgets survive changes of hardware, firmware, and model. Each entry carries the six elements of a limit record and adds the fields that locate the limit at a handoff and state what evidence supports it (table 4.4).

Table 4.4: Conceptual fields of the handoff record: The six limit-record elements of section 2.7.3, extended with the handoff, bound kind, and clock domain that locate a limit in the machine and state what evidence supports it.

Field Name	Physical SI Units	Structural Invariant & Description
`signal_id`	dimensionless	Unique alphanumeric identifier (for example, `SIG_VIO_01`, `SIG_TRQ_AX1`).
`source_subsystem`	dimensionless	Originating hardware or software module (`Camera_SerDes`, `Planner_VLA`).
`destination_subsystem`	dimensionless	Receiving hardware or software module (`State_Estimator`, `Safety_MCU`).
`handoff`	dimensionless	The handoff the signal crosses ($\mathcal{H}_1$ to $\mathcal{H}_5$).
`physical_dimension`	dimensionless	Target limit category (L_{lat}, BW, P_{heat}, σ_{jitter}, $\mathcal{F}$).

Field Name	Physical SI Units	Structural Invariant & Description
`value`	SI units (ms, MB/s, W, µs)	Allocated budget, or the measured distribution it is checked against; for $\mathcal{H}_4$, the chunk lease derived from the stopping budget.
`conditions`	as recorded	Payload, temperature, supply voltage, firmware build, and concurrent workload under which the value holds; the value lapses when they change.
`uncertainty`	same as `value`	Error bound of the instrument or model that produced the value.
`margin`	same as `value`	Engineering margin between the operating target and the breakdown threshold.
`detector`	dimensionless	Runtime monitor that catches an excursion (for example, a lease timer, a rail-voltage monitor, or a phase-current sensor).
`provenance`	dimensionless	Source of the value, including its evidence category (measured, derived, datasheet, or illustrative).
`bound_kind`	dimensionless	`requirement`, `proven bound`, or `measured distribution`; only a proven bound may stand in for a worst case.
`clock_domain`	dimensionless	Clock on which every time-valued field is expressed; a handoff that feeds the permission path uses the safety microcontroller's clock.
`verification_timestamp`	UTC timestamp	Date of bench falsification and sign-off; a log field, never an input to an age test.

The four timing events of table 4.1 sit on this grid's handoffs, capture within $\mathcal{H}_1$, dispatch at $\mathcal{H}_2$, admission at $\mathcal{H}_4$, and brake onset at $\mathcal{H}_5$. The grid's latency column therefore follows the same single execution path as the timing table, and every time in it is expressed on the safety microcontroller's clock. Together, those rows show that the chunk lease at $\mathcal{H}_4$ cannot be chosen in isolation from the stopping distance below it or the inference renewal rate above it.

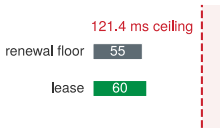
4.9.1 Coupling the distance, lease, and supply budgets

Shortening the chunk lease to save stopping distance, without checking how fast the proposer renews, causes the lease to expire before the next chunk arrives, so the robot stutters from stop to stop. Lengthening the lease to ride out slow renewals lets the robot overrun its rack-end clearance before braking begins. The machine's physical, cognitive, and nervous subsystems therefore meet at a single shared budget, with the chunk lease at their point of convergence. In section 2.3, the loaded machine moving at its 1.5 m/s drive limit had to come to a complete stop within the 1.10 m clearance available at a rack end. Accounting for deceleration mechanics and fixed braking overheads left 258.3 ms for all computational and communication activities preceding brake onset. Section 4.4 then allocated a portion of this window: when a proposer stalls, it retains motion authority for one chunk lease before the emergency stop command reaches the motor drives one tick and one bus cycle later, bringing the running stopping budget to 835.5 mm.

Because the proposer needs enough time to renew that lease before it expires, section 3.4 sets the lower bound from what each learned model can supply. The intent model's INT4 weights require 24.5 ms just to stream across the memory bus. A complete inference pass of compute-bound prefill followed by bandwidth-bound autoregressive decoding consumes 336 ms, so at 2 Hz the intent model cannot renew proposals on tens-of-milliseconds timescales. By contrast, the action chunking policy streams its parameters in 1.12 ms, and its target 20 Hz cadence supplies a fresh trajectory chunk every 50 ms. Its illustrative 40 ms p99 inference latency makes this cadence a feasible nominal supply target rather than a guaranteed worst-case bound, so intercepting late, dropped, or corrupt renewals remains the responsibility of the independent permission path.

These constraints dictate the **lease rule** for the machine. For continuous, uninterrupted trajectory tracking, the lease duration must exceed the proposer's renewal interval plus an allowance for tail jitter. Simultaneously, to guarantee physical safety, the lease must fit within the pre-brake ceiling of equation 4.2 once detection latency, bus transport, actuator brake onset, and sensor evidence age at admission are deducted:

$$T_{\text{chunk}} + T_{\text{jitter}} < T_{\text{lease}} \leq \frac{D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - v^2/2a_{\text{brake}}}{v} - T_{\text{tick}} - T_{\text{bus}} - T_{\text{act}} - t_{\text{age}} \tag{4.4}$$



The lease must outlast renewal and expire well before the stopping budget runs out.

where T_{chunk} is the chunk policy's nominal renewal period, T_{jitter} is the margin allocated for inter-arrival timing variations, and t_{age} is charged as the evidence's age at admission, rows 1 and 2 of table 4.1 together, not its age at dispatch alone. This inequality captures the coupled cyber-physical budget in a single mathematical relation. The left side reflects compute supply, bounded by the neural proposer. The first term on the right reflects physical mechanics, governed by vehicle speed, workspace clearance, and braking deceleration. The remaining subtractive terms account for detection and communication delays, actuator brake onset, and the observation age accumulated across the perception pipeline. A faster vehicle, a less aggressive brake, or a tighter workspace lowers the upper bound; a slower or more jitter-prone neural network raises the lower bound. If the two bounds cross, no lease can preserve both uninterrupted proposal tracking and the stopping guarantee at that velocity; the permission path must interrupt tracking before authority expires.

On the mobile manipulator, the lower bound evaluates to the 55 ms renewal floor derived in notebook 4.1. The upper bound provides 236.3 ms prior to deducting observation age; subtracting the 51.6 ms capture-to-dispatch age allocated in row 1 of table 4.1 leaves 184.7 ms. Because the observation must also traverse the dispatch-to-admission pipeline before execution, charging that interval's budget from table 4.1 increases total evidence age at admission to 114.9 ms, leaving an effective upper limit of 121.4 ms. This is the same subtraction as the notebook's, taken in a different order. The notebook charged the lease first and left 176.3 ms under the ceiling for evidence age and the stop profile; the lease rule charges the age first and leaves this limit for the lease. The selected 60 ms lease sits just above the lower renewal bound, reflecting the physical reality that every additional millisecond of lease duration consumes 1.5 mm of stopping clearance at maximum drive velocity. This value remains an unverified allocation until the physical vehicle demonstrates that all constituent intervals fit within budget under load. Artificially inflating the lease duration to mask software latency spikes merely consumes stopping clearance, trading mechanical safety margin for compute slack.

Because each chunk packages base velocities and arm joint targets together, the architecture enforces a single shared chunk lease across both subsystems. When that lease expires, each physical body must execute a kinematically feasible, state-matched fallback maneuver. The base's 258.3 ms pre-brake ceiling governs this allocation; it does not imply that the arm possesses surplus margin. The arm analysis in section 11.2 evaluates stopping distance and response travel against its own localized workspace clearances. Validating the shared lease requires verifying timing, evidence age, dynamic state, and fallback limits simultaneously across both base and manipulator. The goal the intent model supplies is governed by a different timer, the intent lease, which limits how long a high-level goal remains active before it must be renewed and is sized in its own chapter, not from this stopping budget.

↪ The intent lease and its contract are developed in section 10.8.

Checkpoint 4.3: The handoffs matrix and physical budgets

Before testing the lease on a humanoid and reviewing the engineering fallacies, verify your understanding of system-wide interface contracts:

- ☐ **Signal handoff classification**: Can you identify the five canonical handoffs ($\mathcal{H}_1$–$\mathcal{H}_5$) bridging physical plant mechanics to high-level deliberative reasoning?
- ☐ **Binding physical constraints**: Can you explain why sensory handoffs bind first on bandwidth and exposure latency, whereas actuation handoffs bind first on determinism and heat?
- ☐ **Handoff record provenance**: Can you describe why every numerical limit in an autonomous system must record its evidence category, operating conditions, bound kind, and verification status?
- ☐ **The lease rule**: Can you state the two limits on the chunk lease, one set by successful-admission interarrival and one by the pre-brake ceiling, and explain why a p99 inference time alone proves neither?

Systems Perspective 4.2: A deadline set by a fall

Question: Does the split among proposer, lease, and permission path still hold when the deadline comes from a fall rather than a stop?

An upright biped diverges from balance on the characteristic timescale $\tau_{\text{fall}} = \sqrt{z_c/g}$, where z_c is its center-of-mass height. The biped is modeled in the sagittal plane as an inverted pendulum of point mass m pivoting above a contact point at height z_c (the linear inverted pendulum model that Kajita et al. (2001) introduced for walking-pattern generation). Newton's second law for a small angular deflection θ_{tilt} from vertical then yields:

$$mz_c^2\ddot{\theta}_{\text{tilt}} = mgz_c \sin\theta_{\text{tilt}} \implies \ddot{\theta}_{\text{tilt}} \approx \frac{g}{z_c}\theta_{\text{tilt}}$$

The characteristic equation $\ddot{\theta}_{\text{tilt}} - \omega_0^2\theta_{\text{tilt}} = 0$ yields natural divergence modes $e^{\pm\omega_0 t}$ with natural frequency $\omega_0 = \sqrt{g/z_c}$. The growing exponential mode governs how quickly an unrecovered perturbation tilts out of control, with characteristic divergence timescale:

$$\tau_{\text{fall}} = \frac{1}{\omega_0} = \sqrt{\frac{z_c}{g}}$$

For an adult-scale humanoid with its center of mass near hip height ($z_c = 0.9$ m) and gravitational acceleration $g = 9.81\,\text{m/s}^2$, this evaluates to $\tau_{\text{fall}} = \sqrt{0.9/9.81} \approx 0.303\,\text{s} = 303\,\text{ms}$ (about 303 ms). This figure represents a characteristic divergence rate, not an absolute recovery deadline. The actual time available to recover depends on the biped's initial angular deflection, its angular velocity, support polygon geometry, and available kinematic reach. Nonetheless, this timescale demonstrates why deliberative inference cannot be placed in the feedback loop governing local balance: an explicit, state-dependent recovery calculation must define the hard real-time deadline.

The architectural partition between proposer, lease, and permission path remains valid for bipeds, but the recovery deadline becomes dynamic and state-dependent. Rather than bringing drives to a standstill, balance recovery requires placing a foot at the dynamic capture point (Pratt et al. 2006), a process demanding both finite swing time and peak joint torque. The interface lease must accommodate these physical demands:

- **The lease is sized from the dynamic capture-point deadline.** On a wheeled base, an expired lease triggers mechanical braking to bring the vehicle to rest ($v \cdot T_{\text{lease}}$). On an underactuated biped, locking the joints mid-stride makes the fall worse, because engaging holding brakes while the center of mass moves forward causes the torso to pivot forward into an uncontrolled ground impact. Where reach, swing-time, torque, and energy limits permit recovery, the lease fallback action must therefore execute an **emergency capture-point step**. The chunk lease, permission tick, fieldbus transit, and observation age must all fit within the dynamic recovery budget while reserving sufficient time to swing and plant the foot. The characteristic fall timescale frames this constraint but cannot substitute for dynamic simulation.
- **The proposer must meet the recovery budget.** An inference renewal period that consumes a major fraction of the characteristic fall timescale cannot safely participate in posture stabilization. High-level intent models must remain decoupled from balance reflexes. Whether a mid-level chunk policy can safely control locomotion depends on whether its renewal interval and tail jitter leave adequate margin for fallback step execution.
- **Actuator thermal limits and supply sag constrain the recovery step.** Executing an emergency recovery step demands simultaneous peak torque across multiple leg joints. This surge draws a large transient current, inducing resistive heating in motor windings and risking DC bus voltage collapse (section 2.6.1). Consequently, the permission path must evaluate candidate recovery trajectories against instantaneous thermal and electrical reserves before admission.

Whether the deadline comes from a warehouse rack end or a biped's fall, the handoff matrix and the lease rule only protect the machine if the hardware and firmware underneath them honor the boundaries the equations assume. In practice, subtle implementation shortcuts in watchdog servicing, calibration ports, state feedback, and middleware configurations routinely bypass those boundaries without triggering a compile or bench error.

4.10 Fallacies and Pitfalls

A block diagram draws a clean boundary between high-level deliberation and low-level real-time enforcement, but physical boards and firmware routinely couple the two through shared power rails, common clock domains, interrupt lines, or auxiliary diagnostic ports. When a planning subsystem crashes or misbehaves, those hidden couplings decide whether the safety enforcement loop actually survives.

Pitfall: *Servicing a safety watchdog inside a high-priority timer interrupt rather than an application-verified safety loop.*

The mobile manipulator relies on a hardware self-watchdog to trigger the Safe Torque Off path of section 4.6 if it receives no feed within 5 ms. Because an enforcer that stops executing can no longer command a coordinated stop, this immediate hardware cutoff is required. A dangerous design mistake occurs when engineers service this watchdog from a high-priority hardware timer interrupt rather than directly from the safety loop. If the lower-priority enforcement task deadlocks on a mutex or crashes, the hardware timer interrupt continues firing every 1 ms tick. The watchdog remains fed while the enforcer has silently lost its ability to evaluate torque limits, kinematic clearances, and lease budgets.

In distributed systems engineering, this error is known as the confusion of liveness with correctness. A node responding to network ping heartbeats can be livelocked, thrashing on memory allocation, or deadlocked on an internal mutex. Servicing a watchdog must never signify mere scheduler activity. Instead, every refresh pulse must be conditioned on the successful, sequential retirement of all required verification stages, for example by requiring a monotonic task-completion token before a windowed hardware watchdog is serviced. An independent timer tick confirms only that interrupt dispatching still operates; it provides zero evidence that the controller has safely evaluated the machine's physical state (section 4.4.1).

Architect's countermeasure: Bind watchdog refresh pulses strictly to the verified completion of the safety loop's invariant checks, deploying windowed hardware watchdogs that trip on both execution underruns and spurious early feeds.

Pitfall: *Leaving auxiliary diagnostic or calibration interfaces unmediated in production operation.*

Suppose the mobile manipulator validates base velocity proposals through its permission path, yet its drive-wheel motor drives retain an unmonitored serial calibration interface. If a diagnostic daemon writes a 35.0 A test value directly into an inverter register through that port, the proposal mailbox continues reporting zero velocity. Checking the mailbox reveals nothing about the bypassed command, yet the drive wheels produce torque that no motion safety check ever evaluated or approved. Actuation authority is not defined by a logical software diagram; it encompasses every physical wire, bus transceiver, and digital interface capable of changing a power-stage register. Service interfaces must therefore incorporate operational interlocks or hardware jumpers that prevent calibration writes from bypassing the enforcer during production runs.

Architect's countermeasure: Electrically isolate, physically jumper, or cryptographically interlock all auxiliary diagnostic and calibration ports during operational modes, routing all register writes exclusively through the real-time enforcer.

Fallacy: *Evaluating safety barrier invariants on commanded trajectory setpoints is equivalent to evaluating physical plant state.*

Suppose an arm controller evaluates safety using a control barrier function (Ames et al. 2019), a scalar function h of the state space that remains positive within the safe set and is kept from going negative by filtering unacceptable control inputs. To conserve fieldbus bandwidth, an engineer might evaluate h using commanded joint positions rather than load-side encoder feedback. If a transmission gear

strips or a drive belt shears, the trajectory interpolator continues generating valid, smooth setpoints. The enforcer calculates that the system remains safely inside the barrier set ($h(\mathbf{q}_{\text{cmd}}) > 0$), even while the decoupled arm collapses into an obstacle. Safety enforcement belongs at the causal boundary, which means it must ingest ground-truth physical state measurements through dependable sensory channels and account for sensor noise and latency. Auditing the command stream reveals what the planner intended, but only independent physical feedback captures where the machinery actually moves.

Architect's countermeasure: Close safety barrier evaluations strictly around verified, independent sensor measurements (such as optical load encoders and current shunts) rather than planned trajectory setpoints.

↪ The control barrier function and the safety filter that keeps it nonnegative are developed in section 12.5.

Fallacy: *Nominal, fault-free operating logs are sufficient to prove physical architectural isolation.*

Logging thousands of operating hours without a missed deadline demonstrates nominal stability, but clean telemetry cannot expose hidden cross-subsystem couplings that awaken only during failure modes. Subsystem isolation often breaks down under unmodeled physical interactions, such as supply-rail voltage sags when multiple inverters switch simultaneously, bus contention storms, or memory corruption spilling across addressing domains. Rigorous qualification requires deliberate fault injection: inducing power transients, flooding communication buses, corrupting policy memory, and mechanically locking motor shafts on a dynamometer. Each experiment must evaluate an explicit, quantitative response bound, such as locked-rotor detection clamping motor current within 2.0 ms, rather than treating the mere absence of an operating system crash as validation. Fault experiments support the isolation claim within their stated fault and load coverage when the system maintains invariant enforcement or reaches the specified fallback within the quantitative response bounds. They do not establish coverage of untested fault combinations.

Architect's countermeasure: Validate fault-domain independence through active fault-injection regimes (power sags, clock glitches, bus flooding, and memory corruption) with quantitative pass-fail response bounds.

DRAFT

Pitfall: *Assuming zero packet loss in robotics middleware equates to deterministic real-time delivery.*

A publish-subscribe transport configured for reliable delivery attempts to recover lost packets, but successful recovery does not establish deterministic real-time delivery. After congestion or interference, a reliable transport delivers the burst of queued, obsolete setpoints that section 4.3.2 traced, each describing a robot state that no longer exists. While switching to a freshest-sample policy prevents queue backlogs, dropping stale frames does not inherently bound scheduling jitter, serialization delay, or driver execution latency. The cycle-critical actuation path requires two separate guarantees: an engineered worst-case delivery bound across the physical communication bus and an explicit freshness deadline enforced by the receiver before executing any torque command.

Architect's countermeasure: Budget the complete transport response under declared traffic and fault conditions, enforce deadline-based freshness at the consumer, and isolate cycle-critical commands from supervisory middleware traffic regardless of the bus architecture.

4.11 Summary

Chapter 2 derived the physical stopping budget for the loaded machine, and chapter 3 established the operational cadences for proposal generation and evidence age. This chapter introduced the real-time permission path designed to enforce those constraints, establishing a two-processor architecture whose physical isolation still awaits evaluation under load. To guarantee deterministic safety, the permission path holds exclusive write access to the motor drive registers. It communicates with the unprivileged proposal engine through a lock-free mailbox that never stalls the high-frequency control tick, while communication and computation delays mandate closing joint-level feedback loops directly within the drives. By evaluating command interfaces, from joint positions to operational-space impedances, the permission path prices each check against available stopping clearance. Combining these constraints yields a bounded time window during which an action proposal is permitted to drive physical actuators. On the running machine, this analysis produces a chunk lease that exceeds the proposer's renewal interval while fitting within the pre-brake delay budget. The resulting handoff record documents this lease alongside its validity conditions and evidence

requirements. This lease and the mechanisms behind it remain claims until they are verified under worst-case compute jitter, bus contention, malformed proposals, sensor faults, and degraded braking. Table 4.5 summarizes the four governing relations that define the nervous system's timing, lease duration, and response guarantees, along with the engineering decisions they resolve.

Table 4.5: Key equations of the nervous system: The four relations behind the chapter's rate, lease, and response decisions, with the place each is derived. $\tau_{\text{delay,max}}$ is the right side of the pre-brake ceiling.

Equation	What it bounds or decides	Derived
$\Delta\phi = \omega_c \left(\frac{T_s}{2} + T_{\text{comp}} + T_{\text{bus}} \right)$	The phase lost to sample-and-hold, computation, and transport delay at crossover, dictating whether a control loop can close across the network bus or must reside inside the drive	equation 4.1
$\tau_{\text{delay}} \leq \frac{D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - v^2/2a_{\text{brake}}}{v}$	The pre-brake ceiling: the maximum allowable delay before braking onset that still guarantees stopping within available clearance	equation 4.2
$T_{\text{chunk}} + T_{\text{jitter}} < T_{\text{lease}} \leq \tau_{\text{delay,max}} - T_{\text{tick}} - T_{\text{bus}} - T_{\text{act}} - t_{\text{age}}$	The lease rule, bounding chunk lease duration between the proposer's renewal floor and the ceiling imposed by detection, transport, brake onset, and evidence age	equation 4.4
$R_i = C_i + B_i + I_i \leq D_i$	Worst-case response-time schedulability for hard real-time tasks, verifying that execution, blocking, and interference fit within the task deadline	equation 4.3

Key Takeaways: The permission path and determinism

- **The permission path alone writes the drives:** Learned models act only as unprivileged proposal engines (p_t). Write access to the motor drive registers belongs to the deterministic real-time enforcer alone, hosted on silicon whose isolation from the proposer must be verified experimentally.
- **Cadence is dictated by physics, not software:** The four-tier concurrency hierarchy (20 kHz FOC current regulation, 1 kHz kinematic enforcer checks, 20 Hz action-chunk renewal, and 2 Hz multimodal deliberation on the running machine) is governed by physical time constants ($\tau_e = L_{\text{phase}}/R_{\text{phase}}$), total loop delay, exposure horizons, and model execution budgets. The total loop delay appears as a phase lag $\Delta\phi = \omega_c(T_s/2 + T_{\text{comp}} + T_{\text{bus}})$, which keeps the joint loops in the drives.
- **The exchange never waits, and every proposal expires:** The permission path reads the mailbox in a single bounded attempt per tick, retaining the currently active, independently admissible chunk if a write collision occurs. Every proposal carries an evidence timestamp and an explicit expiration horizon that terminates its authority even if an explicit cancellation never arrives.
- **Stopping dynamics dimension the chunk lease:** For the mobile manipulator operating at its drive limits, the pre-brake ceiling is 258.3 ms. Sizing the lease at 60 ms places it just above the proposer's renewal floor. The interval from the final admitted chunk to brake onset takes 82 ms. Its lease path brings the stopping budget's running total to 835.5 mm of the 1.10 m clear distance before evidence age and the remaining stop margins are charged (row 4 of table 12.1).
- **Worst-case bounds outrank empirical distributions:** Tail-latency percentiles such as p99 cannot establish real-time safety. Deterministic enforcement requires bounding worst-case computation, blocking, and interference ($R_i = C_i + B_i + I_i \leq D_i$) under rigorously qualified operating conditions.
- **The permission path is only as independent as what it shares:** If the permission path shares a clock source, power rail, memory bus, or silicon die with the proposal engine, faults in the proposer can compromise the enforcer. The two-processor split is an archi-

tectural hypothesis that requires empirical validation through stress testing and fault injection (section 13.7), not an assumed property of separate software processes.

- **The command interface sets what the permission path must check:** Joint position targets express kinematic intent that the enforcer can verify geometrically, but they cannot bound contact forces. Velocity interfaces require a bounded motion projection, while torque interfaces require dynamic bounds and a feasible fallback. Impedance targets bound steady-state interaction forces, but only if the enforcer validates the stiffness and damping gains alongside the reference trajectory.
- **Locate interface regressions at their handoff:** The system handoff record maps every timing and clearance limit to one of five architectural boundaries, binding each to operational preconditions, evidence categories, and bound types. When performance regresses, engineers can isolate the offending subsystem against its allocated budget using like-for-like evidence.

Physical stopping limits, inference renewal rates, and the Nervous System's response delays converge on one governing parameter, how long an admitted proposal may command the actuators. When a timing question arises on a new machine, the first diagnostic is to price that parameter, converting the lease path to distance at the operating speed and comparing it with what the stopping budget has left. A lease that cannot fit between the renewal floor and the pre-brake ceiling cannot sustain uninterrupted proposal tracking at that speed, and a faster processor helps only if it lets the renewal period, and with it the floor, shrink.

This chapter closes Part I, and with it the argument that the first law, irreversibility (principle 1), set in motion. The Body turned irreversibility into distances and limit records, the Brain into proposals that arrive late and expire, and the Nervous System into a rule of authority. Motion that cannot be undone may be granted only for as long as the stopping budget can still guarantee the stop. The chunk lease is that rule written as a number, and the handoff record is where the number is kept. Everything the machine later learns or is permitted to do inherits this bound, including the demonstrations it collects while the permission path is watching.

4.12 Historical Perspective and Further Reading

The phase-margin arguments that keep high-bandwidth joint feedback loops inside local motor drives predate digital computing. Nyquist (1932) established the foundational stability criterion that determines whether a closed-loop feedback amplifier will oscillate, and Bode (1945) formalized these dynamics into the gain and phase margins that control engineers budget today. When mechanical and analog controllers transitioned to software, sampled-data control theory formalized how digital sample-and-hold stages, discrete computation steps, and communication transport introduce pure time delays that consume this margin (Franklin et al. 1998). High-bandwidth feedback-loop placement in this chapter follows directly from this phase accounting: a control loop must close physically close to its sensors and actuators because any transport latency erodes phase margin that high-bandwidth physical dynamics cannot spare.

Predictable execution and real-time scheduling supply the second pillar of this architecture. Liu and Layland (1973) proved that assigning static priorities monotonically by rate guarantees schedulability for periodic task sets up to a theoretical utilization bound. Subsequent response-time analysis transformed this sufficient condition into an exact test by summing worst-case execution time, resource blocking, and higher-priority task interference (Joseph and Pandya 1986; Audsley et al. 1993). When shared resources threatened these guarantees with priority inversion, Sha et al. (1990) bounded the duration of blocking through priority inheritance and the priority ceiling protocol. The Mars Pathfinder mission in 1997 demonstrated precisely what the absence of such bounds costs in production, when unmanaged priority inversions triggered repeated system resets on the Martian surface (Reeves 1997). Extending predictability to distributed embedded systems, Kopetz (2011) established the foundations of time-triggered design. That work argued that safety-critical systems must remain statically analyzable prior to execution, and it introduced the concepts of state messages and temporal accuracy that govern the proposal headers in this chapter.

The lock-free coordination between the high-level policy proposer and the low-level safety permission path descends from foundational work on sharing state without waiting. Lamport (1977) demonstrated that a concurrent reader can detect an overlapping write using version sequence numbers alone. Simpson (1990) developed the asynchronous four-slot mechanism for flight-control avionics running on unrelated clocks, Herlihy (1991) formalized the theoretical hierarchy of wait-free synchronization, and Kopetz and Reisinger (1993) formulated the non-blocking write protocol specifically for real-time readers. Industrial networks confronted similar timing and isolation challenges on the wire: Controller Area Network (CAN) bus architectures resolved medium access through deterministic, bitwise priority arbitration (International Organization for Standardization 2015a). The functional-safety profiles of IEC 61784-3 established the black-channel principle, which encapsulates integrity and freshness guarantees within the messages so that the transmission medium itself need not be trusted (International Electrotechnical Commission 2021). Finally, the concept of bounded temporal leases originated in distributed systems, where Gray and Cheriton (1989) restricted how long a client could rely on cached data so that an unresponsive client blocked the server for at most one lease period. Physical AI systems adapt this lease mechanism to an entirely different physical objective: rather than preserving distributed cache consistency, the lease is bounded by the kinetic stopping budget and renewal deadlines of the safety monitor.

For readers seeking deeper exploration, six works trace the lineage from classical feedback theory to modern real-time permission paths:

- Franklin et al. (1998) provides the definitive treatment of digital and sampled-data control, analyzing how zero-order hold circuits, computation time, and transport latencies introduce phase lag. This analysis forms the theoretical foundation for the multi-rate cadence decisions in section 4.3.
- Liu and Layland (1973) is a concise, classic paper that remains the clearest demonstration of how rate-monotonic priority assignment guarantees schedulability under worst-case task arrival patterns.
- Sha et al. (1990) formalizes the priority inheritance and priority ceiling protocols alongside their mathematical blocking bounds. This work explains both the recovery strategy deployed on Mars Pathfinder and the derivation of the blocking term B_i in equation 4.3.
- Buttazzo (2011) offers an authoritative textbook treatment of hard real-time systems, fixed-priority and deadline-driven scheduling, and response-time analysis, complete with concrete task sets that readers can verify by hand.
- Kopetz (2011) develops real-time architectures from the time-triggered perspective. Its chapters on real-time entities, temporal accuracy, and communication clarify why our proposal mailbox exchanges idempotent state messages rather than transient event queues.
- Simpson (1990) presents the four-slot mechanism in its original flight-critical avionics context, demonstrating how four memory slots decouple an asynchronous reader and writer without locking, waiting, or sample loss.

What's Next: From anatomy to closed-loop learning

When an embodied machine modifies or rejects a human command before execution, the recorded demonstration no longer reflects unconstrained intent. The Body, the Brain, and the Nervous System establish physical limits, learned proposals, and a timed permission path. During data collection, such as teleoperated openings of the spring-latched cage door, that path may clamp, rate-limit, or refuse an operator's request, while the body may deliver motion that differs from the admitted command. The limit and handoff records preserve the conditions behind those interventions. In Part II, chapter 5 begins from the reality that a physical machine manufactures its own training data, so each executed action changes the observations that follow. Each record carries what the operator requested, what the permission path admitted, and what the body delivered. It then establishes what a demonstration must retain so that a policy can learn the task without mistaking the collecting machine's limitations for intended behavior.

II

TEACHING THE MACHINE

DRAFT

DRAFT

4

Teaching the Machine

Part I gave the warehouse mobile manipulator a measured body, a chunked proposer slot, and a permission path that bounds every motion segment by a chunk lease. Teaching that proposer a contact skill, such as unlatching a spring-latched cage door, exposes a difficulty that offline machine learning benchmarks never face. In a static benchmark, a classifier reads held-out examples drawn from a fixed distribution ($x \sim p_{\text{data}}$) whose next stored example never changes when the model predicts. On a physical machine, every action the permission path admits moves the arm, alters the contact state, and shapes the observations that follow. That closed-loop dependency is the second bedrock law of section 1.10, a systems design principle that turns covariate shift into an evidentiary discipline across data collection, training, and evaluation.

> Principle 2: Endogenous data
>
> **Invariant**: A physical agent's actions choose its next observations ($s_{t+1} \sim p(s \mid s_t, \mathbf{a}_t)$), so the states a deployed policy visits need not be the states it was trained or tested on. For behavioral cloning, the worst-case cost of that drift can grow with the square of the horizon (Ross et al. 2011), as equation 5.1 states.
>
> **Implication**: A learned proposer's competence is a claim about a declared region of states, established in closed loop with the permission path in place. Visiting a state during collection, recording an attributable training target for it, and testing competence there in closed loop are three separate steps. Missing states call for more collection, and a finite set of trials bounds the resulting claim rather than proving it outright.

Part II traces the cage-door policy across the machine Part I built. During collection the permission path can clamp or refuse an operator's command, and the body can deliver motion that differs from the admitted one. A demonstration must therefore show what was requested, what the permission path allowed, and what the body actually did. Part I's limit and handoff records become preconditions on the data. Training then turns the supported behavior into candidate policies, one cloned from the demonstrations and one distilled from a simulator teacher, each of which improves its training score while hiding a dependency the hardware can expose. Closed-loop trials test a candidate under declared conditions, showing how data collection, training, and evaluation fit together. When a displaced latch plate breaks the nominal policy, a recovery campaign collects attributable corrections, trains an updated candidate, and tests both nominal regression and the wider latch range (using stipulated trial counts rather than measurements from a physical robot).

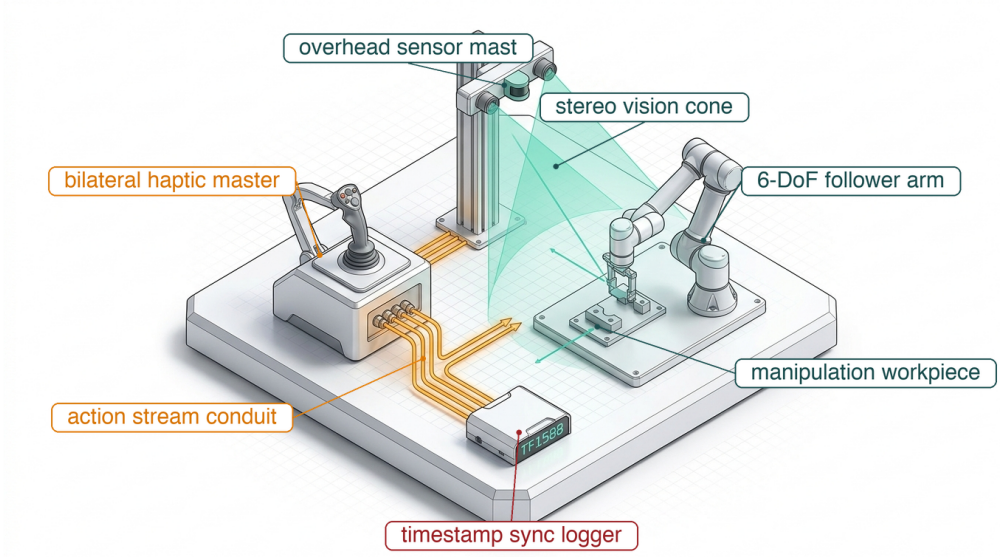
- **Physical Data (chapter 5):** Decides which conditions the demonstrations support and what they cost to collect, and produces the provenance record of the demonstrations, which lists the conditions the data does not cover as known absences.
- **Policy Training (chapter 6):** Decides which training regime fits the available experience and which dependency each regime leaves hidden, and produces the policy manifest, whose declared operating domain cannot exceed the conditions the provenance record supports.
- **Closed-Loop Evaluation (chapter 7):** Decides what a finite set of closed-loop trials supports about the proposer, and produces the evaluation record for a target operational design domain (ODD) inside the manifest's declared one, separating proposer autonomy, whole-machine task completion, and protective outcomes.

Part II hands forward an evaluation record: a claim about the learned proposer confined to a target ODD, the finite-sample bound that supports it, a catalog of the conditions it does not cover, and the runtime monitors the rest of the machine must supply. With it, the first of the record chain's two strands, plant limits and evaluated competence, is complete. That record identifies the candidate from chapter 6, the experience and labels documented in chapter 5, and the independent trials supporting its evaluated domain. While the initial archive bounds what the current evidence can support, new collection and re-qualification can widen that domain later. Part III relies on the current evaluation record rather than assuming that a policy manifest alone establishes competence. Its permission path checks the physical premises for each action and restricts or refuses proposals when those premises lack support. Any fallback still requires a response the machine can physically execute.

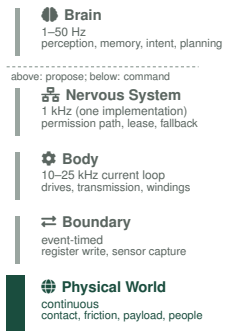
DRAFT

5

Physical Data



5.1 Endogenous Experience
5.2 Mechanics of Endogenous Covariate Shift
5.3 Recovery and Compounding Error
5.4 Physical Telemetry Sources and Collection Paths
5.5 Action Label Selection across the Control Boundary
5.6 Teleoperation Demonstration Costs
5.7 Collection Policy Coverage
5.8 Intervention and Recovery Data
5.9 Cross-Embodiment Transfer
5.10 Dataset Degradation and Physical Distribution Shift
5.11 The Dataset Schema
5.12 Fallacies and Pitfalls
5.13 Summary
5.14 Historical Perspective and Further Reading



Purpose

Why can a large archive of successful demonstrations leave a robot unable to recover from its first mistake?

A successful demonstration shows how one machine completed a task under one set of conditions. It shows almost nothing about what to do after a slip, a shifted fixture, or a worn gripper, yet a learned policy's first error carries it into exactly those states, where the archive holds no instruction. Repeating the demonstration enlarges the archive without filling that gap. The recorded motion also carries the operator's delay, the machine's limits, and every correction the permission path applied, so a training process that cannot separate these influences learns a safeguard's intervention or a mechanical limitation as if it were skill. Gathering the missing experience costs operator time and component life, and the collection itself must stay within the machine's physical limits. Physical data must therefore be engineered, from the choice of experiences to the meaning, timing, and physical origin of every label and the conditions the archive leaves uncovered. Because every collected action crosses the causal boundary, its irreversible costs decide what a machine can learn and where its evidence stops.

Learning Objectives

- Explain how endogenous covariate shift yields the conditional quadratic imitation-cost bound, and distinguish dataset occupancy from evaluated closed-loop competence
- Select the action tap a dataset imitates and align multi-rate sensor and action streams to hardware exposure-midpoint timestamps within the task's spatial tolerance
- Estimate usable demonstration yield from collection time, operator rest, and physical reset overhead, and budget the mechanical component wear a campaign consumes
- Identify collection gaps by joining empirical state-action occupancy against a required scenario table
- Separate failed autonomous actions from admissible human recovery across the divergence, takeover, and stabilization timestamps of an intervention episode
- Evaluate whether an embodiment's mechanical, sensing, and timing assumptions transfer to another machine without falling into the normalization fallacy
- Detect silent calibration and plant drift in a demonstration archive using kinematic reprojection and repeated excitation trajectories
- Design an immutable provenance record linking each observation to its four-stream action taps ($a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}}$) and to the collecting plant

5.1 Endogenous Experience

In most machine learning, the dataset exists before the engineering begins. A text corpus or an image benchmark is gathered independently of the model, costs little per additional example, and does not change in response to anything the model predicts. A physical machine has no such corpus. Collecting demonstrations on the robot consumes operator time and component life; robot-free and synthetic sources avoid that robot wear but still require qualification for the target machine. Every telemetry record reflects who commanded the action, what the permission path admitted, and how the physical body moved. Because each executed action changes the physical state of the system, it dictates what subsequent observations can reveal. Physical data is therefore an engineered artifact. The central challenge of physical data engineering is deciding which experiences to collect, and what to record about them, so that a policy learns the task rather than the idiosyncrasies of its collection. Answering this challenge requires four elements: accounting for collection costs in operator time and hardware wear, defining an unambiguous meaning for every recorded action label, establishing a metric for state-space coverage independent of the archive's internal contents, and explicitly auditing what the archive omits.

Consider a warehouse mobile manipulator tasked with opening a spring-latched cage door. The robot can flawlessly enforce its low-level motion limits and yet fail because it lacks the operational experience to unlatch the door. During a demonstration, a human operator commands motion, the safety permission path checks and filters it, and the robot's physical body drives against the mechanical latch. The onboard sensor log captures the output of that entire coupled loop. The limits and handoffs recorded earlier for this machine, in the limit record of section 2.7.3 and the handoff record of section 4.9, therefore become preconditions on the training data fed to the learned policy proposer. Every logged command was shaped by them before the body moved. Choosing which physical experiences to collect demands a rigorous accounting of how control inputs shift future sensory states. Choosing what to imitate requires cleanly separating the operator's raw request from the safety-filtered command and the measured physical response.

A policy fit by supervised imitation to nominal, clean teleoperated demonstrations of door openings (behavioral cloning, defined below) can execute flawlessly in simulation yet jam the latch during its first autonomous deployment. The primary culprit is the distribution shift that the machine's own closed-loop actions induce. Unlike passive inference on a fixed benchmark, an action executed within an embodied Markov decision process (MDP), in which the next state depends on the current state and action, physically alters the robot's spatial trajectory, camera viewpoint, and contact states. That displacement changes the probability distribution of every subsequent sensory observation ($s_{t+1} \sim p(s \mid s_t, \mathbf{a}_t)$).

↪ The Markov decision process, with its transition model and reward, is developed for the latch task in section 6.2.1.

Definition 5.1: Endogenous covariate shift

Endogenous covariate shift is the closed-loop distribution discrepancy wherein an embodied agent's executed actions alter its subsequent physical states and sensory observations ($s_{t+1} \sim p(s \mid s_t, \mathbf{a}_t)$), driving the system into state-space regimes unsupported by its training demonstrations.

1. **Significance**: Explains how offline behavioral cloning can fail during closed-loop physical execution: a perturbation may drive the system into states where the policy lacks training support.
2. **Distinction**: Unlike exogenous dataset shift caused by changing ambient lighting or seasonal weather, endogenous shift is actively generated by the agent's own closed-loop actions altering its subsequent observation distribution.
3. **Common pitfall**: Attempting to fix closed-loop policy failure by collecting more nominal expert demonstrations, which only increases data density along the success manifold without providing the corrective recovery data needed to escape boundary excursions.

This distribution shift is how the second law, endogenous data (principle 2), shows up in data collection, where the machine's own actions choose the states its archive must later answer for. An archive composed exclusively of flawless executions without recovery trajectories becomes an operational liability. The moment the autonomous policy commits its first sub-millimeter tracking error, the robot enters states never encountered during training. Two distinct physical mechanisms govern how fast that departure grows (section 5.2).

5.2 Mechanics of Endogenous Covariate Shift

When an image classifier misidentifies a dog as a cat, the mistake does not alter the pixels of the next photograph in the test set. When a door-opening policy misses the latch centerline by a millimeter, that motion shifts the camera viewpoint, changes the contact geometry, and alters every observation that follows. Classical empirical risk minimization assumes the first setting, drawing independent and identically distributed (i.i.d.) samples from an exogenous, stationary distribution $\mathcal{D}$. Even when passive machine learning studies *covariate shift*, it treats the shift as an open-loop observational change: the marginal input distribution $p(x)$ shifts (such as evaluating a daytime-trained vision model at night) while the conditional data-generating function $p(y \mid x)$ stays fixed. Because a passive model's predictions $\hat{Y}$ exert no physical back-action on the world, an inference error never mutates the next input. Imitation learning on a machine operates inside the closed-loop counterpart to this setting.

In an embodied physical system, two distinct mechanisms distort what the learner encounters. The first shifts the states that the policy visits even when the hardware stays unchanged; the second alters how the physical plant responds to any given control input:

↩ The closed loop through which a flawed proposal carries the machine into states missing from its training distribution is established in section 1.4.1.

1. **Endogenous state distribution shift**: The policy's executed actions $\mathbf{a}_t = \pi_\theta(\mathbf{o}_t)$ directly drive the physical state transition $s_{t+1} \sim p(s \mid s_t, \mathbf{a}_t)$, choosing the sensory observations $\mathbf{o}_{t+1}$ the robot sees next. A single control error moves the body into an unvisited region of the state space and changes the incoming observation distribution on the next tick.
2. **Active physical plant drift**: The physical plant is a continuous dynamical system governed by differential equations of motion $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}; \theta_{\text{plant}}(t))$ whose parameters drift over operational time. Actuator Joule heating ($i_q^2 R(T_{\text{winding}})$) raises winding temperature and copper resistance via $R(T_{\text{winding}}) = R_0(1 + \alpha_{\text{Cu}}\Delta T)$, compressing the motor drive's voltage headroom and capping peak torque at high joint speeds. Over extended operating cycles, continuous loading degrades strain-wave gearbox lubrication, lowering torsional stiffness and expanding mechanical lost motion. Compliant elastomer gripper pads wear down and deform, shrinking the Coulomb friction cone ($\|\mathbf{f}_{k,\parallel}\| \leq \mu f_{k,\perp}$). Finally, physical workpieces vary in mass, center of gravity, and surface texture across trials, perturbing both the inertial loads the arm accelerates and the gravitational torques the joints sustain.

Together, the policy's closed-loop actions and the plant's response induce an *endogenous state visitation distribution* $p^{\pi}(s)$, the expected fraction of time steps that a rollout of policy π spends in state s. When an agent is trained by offline behavioral cloning on demonstrations from an expert policy π^*, empirical risk minimization fits the imitation objective only over the expert's occupancy distribution $p^{\pi^*}(s)$. During autonomous execution, the learned policy instead acts under its own induced distribution $p^{\pi}(s)$. Wherever p^{π} departs from p^{π^*}, the policy encounters states that hold no demonstrated corrective action.

Definition 5.2: Behavioral cloning

Behavioral cloning is imitation learning posed as supervised learning: a policy is fit to map each recorded observation to the action the demonstrator took there, with the loss weighted by the demonstrator's own state occupancy p^{π^*}.

1. **Significance**: Training error is measured only on states the demonstrator visited, while deployment cost accrues on the states the learned policy visits. Under a 0–1 action loss (one whenever the policy's action differs from the expert's, zero otherwise) the worst-case excess cost can therefore grow with the square of the horizon S, the episode length in control steps, rather than linearly, as the next section derives.
2. **Distinction**: Unlike corrective aggregation, which obtains expert labels at states the learner itself reaches (section 5.8 develops it), behavioral cloning never observes the consequences of its own mistakes; unlike reinforcement learning, it needs demonstrations but no reward.
3. **Common pitfall**: Reading a low held-out loss as evidence of closed-loop competence. A held-out split drawn from the same demonstrations shares their occupancy, so it cannot contain the states the policy's first error reaches.

Because behavioral cloning weights its loss solely by the demonstrator's occupancy p^{π^*}, a single step off the demonstrated path places the policy in states where training never penalized an error. How fast that first departure compounds over an episode depends on whether the archive holds any actions that steer the machine back (section 5.3).

5.3 Recovery and Compounding Error

Opening a latch from a nominal centered approach and recovering from an off-axis sideways slip require different physical actions (figure 5.1). Every retained demonstration guides the gripper forward along the narrow corridor centered on the strike plate, which populates the archive with thousands of forward-progress actions. The archive contains zero demonstrations that start from a gripper displaced a few millimeters toward the fixture and return it to the centerline. Because a skilled human operator rarely lets the gripper drift into those boundary regions, and because teleoperation runs that drifted noticeably were aborted or discarded during curation, nothing in the training data constrains the policy's output at such a displaced state.[1] The **recoverability problem** is this absence of demonstrated corrective actions that steer the system back into the nominal corridor.

Whether this missing return action leads to task failure depends on the closed-loop dynamics of the complete physical plant. Because the robot's physical state encompasses joint coordinates, joint velocities, motor drive currents, contact wrenches (combined three-axis contact forces and torques), and raw camera streams, a nominal demonstration traces an infinitesimally thin manifold through a high-dimensional state space. Almost any unmodeled disturbance displaces the system off this path. If the physical plant and its low-level joint controllers inherently restore small lateral displacements toward the centerline through compliant passive dynamics, an imperfect policy output near the corridor boundary will naturally attenuate. If the system dynamics lack this self-centering stability, the tracking error will persist or compound until a learned recovery policy, the safety permission path, or a human supervisor intervenes. While production demonstration archives often contain occasional human corrections, their coverage cannot be assumed; it must be audited against an explicit scenario table (section 5.7).

[1] **ALVINN (Autonomous Land Vehicle in a Neural Network)**: Pomerleau's road-following network learned to steer from a human driver who kept the van centered, so its training images held almost no off-center views (Pomerleau 1989). His remedy shifted each camera image sideways in software and corrected the steering label to match, synthesizing recovery examples about 20 years before Dataset Aggregation (DAgger, section 5.8) (Pomerleau 1991). A warp could fill that gap because the missing state was only a viewpoint of a flat road, whereas a gripper displaced toward the fixture also lacks contact forces that no image transform can manufacture, so its recoveries must be demonstrated or collected.

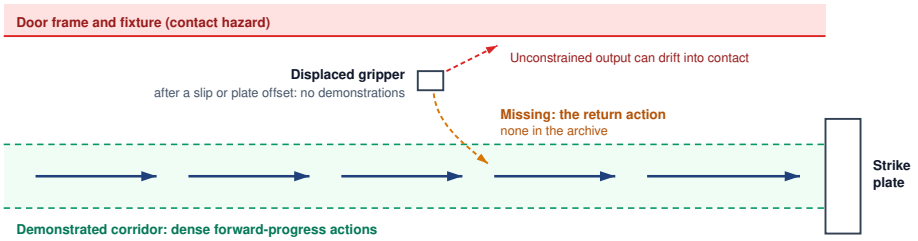


Figure 5.1: The latch corridor seen from above: Retained demonstrations supply dense forward-progress actions inside a narrow corridor centered on the strike plate. A gripper displaced toward the fixture sits where the archive holds no demonstration, so nothing in the data specifies the action that returns it to the corridor, and an unconstrained output can carry it farther toward contact.

The quadratic worst-case scaling of behavioral cloning can be derived through a discrete summation over horizon steps. Suppose an autonomous agent follows the expert trajectory until encountering its first execution error at step $k \in \{1, \dots, S\}$. If this error propels the system into an unsupported state from which it never recovers, the agent incurs an adverse task cost at every remaining step $(S-k)$. Assigning each time step an upper bound ϵ on the probability of committing a first error and normalizing the per-step cost to unity ($C_t = 1$) yields the following worst-case cost accumulation. This discrete summation serves as an upper-bound derivation rather than an exact expectation over independent failure events:

$$\sum_{k=1}^{S} \epsilon(S-k) = \epsilon\left(S^2 - \sum_{k=1}^{S} k\right) = \epsilon\left(S^2 - \frac{S(S+1)}{2}\right) = \frac{S(S-1)}{2}\epsilon \approx \frac{1}{2}S^2\epsilon.$$

In open-loop supervised learning, by contrast, if per-step inference errors were independent and exerted zero feedback on subsequent model inputs, an error rate of ϵ per step would scale linearly over S steps as $\sum_{t=1}^{S} \epsilon = S\epsilon$. The extra factor of S appearing in $\mathcal{O}(S^2\epsilon)$ is the mathematical signature of closed-loop endogenous feedback.

This quadratic bound characterizes the accumulation of task cost over time, not the physical trajectory of the machine. The result does not imply that spatial tracking errors grow quadratically as t^2. Closed-loop contact forces, mechanical compliance, and actuator saturation govern how physical trajectories evolve; the derivation formalizes only the cost of operating in states where the policy lacks training support.

Writing $J(\pi)$ and $J(\pi^*)$ for the expected cumulative task costs of the learned policy π and expert policy π^* over those S steps, let ϵ bound the single-step action disagreement across expert-visited states under a 0–1 action loss: $\mathbb{E}_{s\sim p^{\pi^*}}[\mathbb{I}(\pi(s) \neq \pi^*(s))] \leq \epsilon$. Assuming per-step task costs lie in $[0, 1]$, behavioral cloning admits the worst-case bound (equation 5.1) derived by Ross and Bagnell (2010) and restated by Ross et al. (2011):

$$J(\pi) - J(\pi^*) \leq S^2\epsilon. \tag{5.1}$$

The published theorem carries a constant of one rather than the sketch's one half; both are worst-case bounds, and their quadratic dependence on the horizon is the same.

In practice, this theoretical bound can exceed the maximum possible cost achievable in a finite episode. Because per-step costs are bounded in $[0, 1]$, the excess cost is trivially bounded by S; hence, $\min(S, S^2\epsilon)$ represents the operational upper bound. Once the quadratic expression surpasses S, it provides no meaningful prediction regarding physical failure rates or contact severity. The practical engineering value of this bound lies in exposing why behavioral cloning has no defense in states its demonstrations never reached, rather than in predicting exact physical collision rates.

The actual scaling with horizon length depends heavily on the chosen loss function, the action space, and the closed-loop plant stability. Consequently, equation 5.1 reflects a classical result for discrete 0–1 loss rather than an absolute law for all imitation architectures. When trained under

↪ The full derivation of the quadratic bound, with its loss and cost assumptions, is developed in section A.5.

logarithmic loss (minimizing the negative log-probability assigned to expert actions) over a well-specified policy class containing the expert, behavioral cloning can achieve a sample complexity that does not grow with the horizon length. This result holds provided total reward is bounded and the policy class capacity is controlled (Foster et al. 2024). Continuous action spaces, however, introduce the opposite challenge. Even when the physical closed loop is exponentially stable (small mechanical perturbations decay at a fixed rate) and the expert policy is smooth and deterministic, any smooth, deterministic imitator can accumulate closed-loop execution errors that scale exponentially with horizon length relative to its training loss (Simchowitz et al. 2025). The same analysis identifies two ways around this compounding (Simchowitz et al. 2025). The first changes the policy class. A policy that is non-smooth, that is non-Markovian (its action depends on recent history rather than solely the current observation), or whose action distribution is multimodal (it can place probability on several distinct actions at once, rather than averaging them) escapes the result. Action chunking and diffusion decoders have these properties. The second changes the data, using demonstrations deliberately spread across a wide tube surrounding the nominal trajectory. That second remedy is a data-collection decision, and section 5.8 returns to it as noise injection during teleoperation. Because the mobile manipulator operates in a continuous control space, closed-loop success ultimately hinges on whether the combined policy and low-level controllers actively restore a displaced gripper to the nominal path.

↩ Action chunking, in which the policy emits a short sequence of setpoints per inference, is established in section 3.5.1.

Collecting more nominal expert demonstrations does not resolve endogenous covariate shift. Adding another thousand hours of clean, centered door openings merely increases sample density along the nominal trajectory; the empirical support outside this narrow tube remains strictly zero ($p^{\pi^*}(s) = 0$). This pitfall is the *nominal demonstration trap*. Beyond this coverage limitation, logging physical robot experience carries direct hardware costs. While a web crawler scrapes static text with zero marginal mechanical wear, every recorded second of robot telemetry consumes actuator fatigue life, heats motor windings (dT_{winding}/dt), wears down gearbox teeth, and depletes battery reserves. In practical data-collection campaigns, required pauses for operator fatigue, sensor recalibration, scene resets, and discarded trials often leave less than one quarter of scheduled workcell time as usable demonstration data (section 5.6). Physical telemetry is scarce, wear-intensive, and costly; collecting it without modeling closed-loop recovery dynamics compromises policy training.

↩ Actuator thermal dissipation (dT_{winding}/dt) and winding damage limits are established in section 2.5.

A physical command also changes meaning as it crosses the control stack toward the actuators. In a system partitioned by the proposal boundary, an operator's or policy's requested action is mapped into a trajectory setpoint, checked and potentially clamped by the safety permission path, and executed by the physical plant to produce a measured response. Logging only the raw requested action trains the policy on commands the plant may be physically incapable of executing; logging only the measured plant response teaches the policy to copy link flexure, gear backlash, and contact stalls as if they were intentional human skill. A physical data-collection campaign must therefore settle two decisions before recording its first trajectory. The first is which collection path generates the actions (section 5.4). The second is which of the four representations of one command (the request, the mapped setpoint, the admitted command, or the measured response) the dataset preserves as its training label (section 5.5).

↩ The proposal boundary, which separates a learned proposer from the permission path that alone may admit an action, is established in section 1.7.

5.4 Physical Telemetry Sources and Collection Paths

When an engineer records demonstrations of the warehouse cage-door latch, the choice of collection rig decides which contact forces, hesitations, and recovery maneuvers ever reach the archive. Guiding the robot arm by hand gives the demonstrator direct tactile feel of the binding latch, but the demonstrator's torso blocks the onboard cameras. Puppeteering the arm over a network clears the camera view, but display delay makes the operator inch forward in a stop-and-go hunt. Running a scripted spline produces clean, repeatable approaches that jam the moment the strike plate shifts by a millimeter. Because every attempt on the physical robot spends facility hours, joint life, and human effort, choosing a collection path trades collection cost against the behaviors the resulting archive can support. Seven collection paths appear in practice, six on physical hardware and a seventh in simulation:

1. In **kinesthetic teaching**, a colocated human demonstrator physically grasps and guides the unpowered or gravity-compensated links of the mechanism. The logging pipeline records joint encoder positions at the native controller rate without network transport delays. How-

ever, human biomechanics govern the motion's bandwidth, moving far more slowly than the controller's sampling loop, while labor costs scale $1:1$ with demonstrator time. The demonstrator's body also frequently occludes onboard camera views, introducing systematic visual distribution shifts relative to autonomous operation.

2. In **teleoperation**, a remote operator manipulates an input device to puppet the follower robot across a network fabric.[2] Teleoperation interfaces differ in how operator inputs map to robot actuators and how much physical contact information reflects back to the human. Joint-space leader-follower arms such as ALOHA (Zhao et al. 2023) and GELLO (P. Wu et al. 2024) mirror the follower's joint topology, often at reduced scale, streaming joint angles directly to the follower without inverse kinematics (computing the joint angles that place the tool at a commanded pose). In contrast, tracked spatial controllers and rate-controlled space mice command Cartesian tool poses or velocities that require inverse kinematics solvers at runtime, while force-reflecting bilateral arms actively transmit measured contact forces back to the operator's hands. Table 5.2 compares these interfaces and names the artifact each leaves in the recording.
3. In **scripted collection**, a deterministic trajectory generator executes automated motion primitives, such as cubic splines or impedance-governed state machines. Ingestion rates are high and predictable (50–500 Hz), but trajectories traverse rigid, repetitive manifolds. Because these paths fail to generate corrective recovery data during off-nominal contact, the manipulator risks jamming whenever workpiece dimensions deviate by even a millimeter.
4. In **exploratory collection**, an active neural policy or reinforcement learning agent selects actions to maximize an information-theoretic objective, such as epistemic entropy or intrinsic curiosity. Although data throughput matches the robot's peak policy cadence (50–100 Hz), unconstrained exploratory actions risk high-energy collisions, tripping hardware safety tripwires or degrading mechanical gearboxes through repeated high-torque shock loads.
5. In **autonomous deployment logging**, production policies control actuators during routine factory or warehouse operation, streaming multimodal telemetry into local circular buffers in situ. This channel yields zero-marginal-labor data at the system's full execution rate (10–50 Hz for high-level policies, 1 kHz for joint servos). However, runtime safety barriers heavily censor the recorded state distribution, capturing almost exclusively nominal operating regimes while excluding boundary failures.
6. In **robot-free demonstration**, the operator carries a hand-held gripper fitted with a wrist camera and performs tasks directly in the physical environment, as in the Universal Manipulation Interface (UMI) (Chi et al. 2024). Collection incurs zero robot operating hours or actuator wear, allowing a single demonstrator to gather data across locations where no robot is deployed. The engineering cost shifts to label synthesis: because the hand-held rig records its own Cartesian trajectory and visual feed without robot joint states, actions must be expressed relative to the gripper's pose frame and executed downstream through the target robot's controller. The deployed robot's observation and actuation latencies must also be calibrated to match the recording timing (section 5.5.2).

A seventh source removes physical hardware entirely. Simulation pipelines eliminate mechanical wear by substituting a numerical contact model for the physical plant, generating synthetic demonstrations at scale. Trajectory generators can expand a few hundred human seed demonstrations across novel object poses and scene layouts, replaying them in physics engines to synthesize tens of thousands of varied trajectories (Mandlekar et al. 2023). Downstream policies can then co-train on simulated and real telemetry at a specified mixing ratio. However, these synthetic streams require rigorous metadata tagging: records must log their source generator, physics engine build, integrator step size, and contact solver parameters. The dataset's scenario table, the list of required operating conditions that section 5.7.1 builds, must also flag every condition covered solely by synthetic rollouts, ensuring that training diagnostics can isolate simulation artifacts from measured physical plant behavior. The provenance requirements for synthetic telemetry appear under dataset degradation (section 5.10.2).

Because the collection source determines how the machine interacts with its physical environment, these data streams occupy disjoint state-action manifolds even when executing identical manipula-

[2] **Bilateral Telemanipulation**: Raymond Goertz's 1949 master-slave manipulator at Argonne National Laboratory moved tongs behind radiation shielding in exact correspondence with a master handle, and its mechanical linkage returned the forces at the tongs to the operator's hand (Goertz 1949; Hokayem and Spong 2006). Remote manipulation therefore began with contact force inside the loop. A demonstration recorded through an interface that drops that force channel cannot contain the contact corrections the operator would have made by feel.

tion tasks. Consider releasing the warehouse mobile manipulator's cage-door latch. A colocated demonstrator guiding the arm feels the latch bind through direct tactile resistance and immediately adjusts wrist compliance, whereas a remote teleoperator watching a compressed video stream lacks haptic feedback and falls into step-and-wait hunting, pausing repeatedly to check strike-plate contact. By contrast, a scripted controller advances along a fixed geometric path at constant velocity and produces the same rigid contact-force spike on every repetition, while an exploratory policy approaches at oblique angles that bind the latch until safety tripwires abort the trial. Far from representing noisy variations of a single ground-truth trajectory, these recorded streams reflect distinct dynamical systems with different bandwidths, transport latencies, and mechanical compliance profiles, as summarized in table 5.1.

Table 5.1: Systems trade-offs across physical demonstration and telemetry sources: Seven collection sources compared across control dynamics, contact awareness, human labor economics, and the boundary conditions that collection must satisfy.

Physical Data Source	Ingestion Bandwidth & Cadence	Contact Compliance & Force Awareness	Scaling Labor Economics	Collection Boundary Conditions to Check
Kinesthetic teaching	Controller rate; motion bandwidth set by the human arm	High (direct physical feel of binding and compliance)	Very high (1 : 1 dedicated human demonstrator)	Human contact with links; qualified force and speed limits
Teleoperation (bilateral/isomorphic)	Moderate (20–50 Hz, network + human latency)	Moderate to high (depends on haptic force feedback)	High (1 : 1 human operator per active cell)	Bounded delay and visibility; isolation and qualified response
Scripted primitives	High (50–500 Hz, deterministic trajectory generators)	Low (rigid trajectories unless impedance-governed)	Low (automated once authored; engineering upfront)	Fixture mismatch and contact; guarded trajectory
Exploratory policy	High (50–100 Hz, active neural exploration)	Variable (learned or randomized contact behaviors)	Very low (fully autonomous hardware execution)	Bounded exploration; qualified containment
Autonomous deployment logging	Full system rate (10–50 Hz policy, 1 kHz joint servos)	Operational (reflects live deployed closed-loop control)	Zero marginal labor (harvested passively during production)	Qualified operating domain and runtime enforcement
Robot-free hand-held device	Device camera and pose rate; no robot joint stream	High (operator feels real contact through the device)	Moderate (one person per device; no robot time or wear)	No robot actuation during capture; target execution separately qualified
Synthetic generation	Simulator step rate, limited by compute	Modeled (contact set by engine and parameters)	Low after seed demonstrations are recorded	No physical actuation during generation; model fidelity and target qualification

The cheapest sources in table 5.1 also produce the highest volume of records. This abundance frequently invites a misconception borrowed from disembodied machine learning, where empirical scaling laws correlate model capability directly with raw token count or pretraining dataset volume (N). In physical telemetry, treating every logged control step as an independent observation is a category error. An edge logger sampling at 1 kHz records 3.6×10^6 state-action pairs every hour, yet adjacent timesteps carry near-total temporal autocorrelation ($\rho \to 1.0$) because successive kinematic readings sampled 1 ms apart differ only by sub-millimeter structural displacements and sensor quantization noise. For a stationary scalar telemetry statistic with geometrically decaying autocorrelation $\rho_k = \rho^k$, as characterized by a first-order autoregressive AR(1) process, the large-sample *effective sample size* (ESS)[3] for estimating its mean is:

$$N_{\text{eff}} = N \frac{1-\rho}{1+\rho}. \tag{5.2}$$

Under this autoregressive model, an autocorrelation of $\rho = 0.999$ across 3.6×10^6 raw records yields only 1,800 statistically independent observations. While lag-1 correlation alone does not govern ESS across every non-stationary telemetry stream, high-frequency sampling still cannot substitute for operational diversity. Whereas consecutive tokens in language modeling represent high-entropy transitions across a large vocabulary, high-rate robot telemetry mostly oversamples

[3] **Effective Sample Size**: The factor $(1-\rho)/(1+\rho)$ is the effective-sample-size correction for the mean of a first-order autoregressive series, the correction climatologists apply to serially correlated weather records before testing a trend (Thiébaux and Zwiers 1984). At the correlation used here it turns an hour of permission-tick records into about 1,800 independent ones, so a step count overstates the independent information available for estimating this statistic's mean by three orders of magnitude before coverage is even considered.

the same physical trajectory. It can resolve fast contact transients when they occur, but raising the logger's clock frequency never populates an unvisited scenario cell. Physical dataset coverage expands instead through independent physical episodes, diverse workpiece placements, varying illumination, and shifting friction coefficients. A curated archive containing several hundred episodes that systematically sweep initial geometric and mechanical conditions provides far richer empirical support than millions of repetitive timesteps collected across a static test bench.

5.4.1 Human demonstration idiosyncrasies and mechanical artifacts

When a behavioral cloning policy is trained directly on raw teleoperated joint velocities, the deployed robot arm often chatters its harmonic drives at 8–12 Hz or stalls for 300 ms a few millimeters in front of the latch strike plate. Neither defect comes from the network architecture. Because a human operator executed the training motions, the recorded telemetry embeds the demonstrator's neuromuscular bursts, perceptual delays, and visual blind spots alongside the intended trajectory:

- **Neuromuscular sub-movements and physiological tremor:** Human motor control operates through discrete motor unit recruitment bursts rather than continuous, smooth trajectories. These neuromuscular dynamics introduce corrective sub-movements at 2–3 Hz overlaid with involuntary physiological tremor at 8–12 Hz. If a high-frequency behavioral cloning policy trains on raw joint velocities without low-pass filtering, the neural network imitates this biological tremor as high-frequency torque chatter, accelerating mechanical fatigue in the robot's harmonic drives and gearboxes.
- **Visual latency and step-and-wait hunting:** When an operator guides a manipulator toward a contact fixture using 2D camera monitors, monocular depth ambiguity and video compression latency force an unnatural *step-and-wait* hunting cycle. The operator inches the gripper forward by a few millimeters, pauses for 200–400 ms to visually confirm clearance or contact across the video stream, and only then resumes motion. An imitation policy trained on uncurated velocity streams treats these latency-induced pauses as deliberate actions, yielding deployed behaviors that stutter, oscillate, or stall mid-trajectory.
- **Attentional narrowing and gaze-conditioned bias:** Human demonstrators naturally focus visual attention on the active tool center point (TCP). Because the operator rarely tracks the peripheral links of the arm during fine manipulation, the robot's elbow or shoulder links frequently sweep perilously close to structural supports or table edges. An imitation policy trained on these demonstrations inherits trajectories that graze workspace obstacles, yet lacks the human's peripheral situational awareness or rapid collision avoidance reflexes.

Because an ingestion logger records every physical tremor, visual hesitation, and marginal joint clearance as part of the demonstration, imitation policies reproduce these flaws unless the training pipeline filters or abstracts them. Choosing a telemetry source decides what raw experience reaches the archive; choosing which control signal serves as the action label decides whether the policy learns the operator's intent, the safety filter's clamp, or the plant's compliance (section 5.5).

5.5 Action Label Selection across the Control Boundary

Suppose a teleoperator pushes the gripper into the cage-door strike plate aggressively enough to trip a velocity clamp, or stalls against the latch while the wrist motor holds torque. A logger that records only the operator's raw command teaches the policy to repeat a move the machine refused to execute, whereas a logger that records only the measured encoder velocity teaches it that stalling against a latch is a deliberate command to stop. By the time the gripper contacts the strike plate, a single operator motion has become four different numerical signals across two decoupled processing domains:

- The unprivileged **Linux application processor** executes perception pipelines, handles video frame ingestion, runs teleoperation input mapping, and computes the neural policy forward pass ($\pi_\theta(\mathbf{o}_t)$).
- On the permission side of the proposal boundary, a hard real-time **safety microcontroller (MCU)** runs joint and permission checks at the chosen 1 kHz cadence. Dedicated motor-drive logic closes the faster field-oriented current loop; its current samples remain distinct from the telemetry retained at each permission tick.[4]

[4] **Hardware Input Capture**: On real-time MCUs, a timer's input-capture unit latches encoder counts and current-shunt conversions at a hardware edge, and Direct Memory Access (DMA) moves them to memory without waiting on software. Sampling jitter stays within microseconds, so the denominator of a numerically differentiated velocity or acceleration carries no scheduling noise.

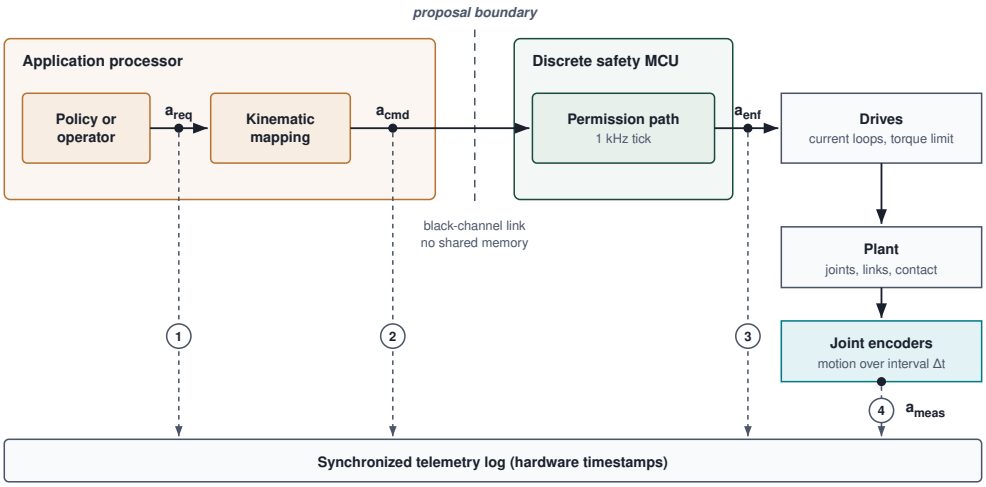


Figure 5.2: Four-stream action tap tuple: Four places a command can be logged, from raw policy intent on the application processor through to measured plant response. Which tap a dataset records decides what the model learns to imitate. The four-tap decomposition is this book's construct; the teleoperation rigs it instruments follow Zhao et al. (2023) and P. Wu et al. (2024).

5.5.1 The four-stream action tap tuple

If a dataset keeps only one signal from those two processing domains, any discrepancy between what the operator asked for and how the arm actually moved becomes impossible to diagnose. A command must therefore be logged at each of the four causal stages it passes through. The **four-stream action tap** $(a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}})$ records all four in one hardware-synchronized telemetry log (figure 5.2). Two are logged on the unprivileged Linux host before the black-channel link, and two on the discrete safety MCU after the 1 kHz permission check. Logged together, the taps keep interventions and tracking errors auditable.

Dividing telemetry logging across the unprivileged Linux host and the hard real-time MCU captures four distinct physical signals:

- **Tap 1: The requested action (a_{req}, "Asked"):** Logged in user space on the application processor, this represents unconstrained human operator input (from a leader arm or joystick) or the raw forward-pass output from an unverified policy, tagged with its native coordinate frame and issue timestamp prior to software validation. This proposal reflects unadulterated human or policy intent, but it frequently demands infeasible joint accelerations or violates spatial bounding envelopes.
- **Tap 2: The mapped command (a_{cmd}, "Mapped"):** The kinematically retargeted setpoint (such as Cartesian end-effector velocities converted to joint space setpoints via inverse kinematics), incorporating coordinate frame transformations, trajectory interpolation, and workspace limits before transmission across the black-channel board link to the safety MCU.
- **Tap 3: The enforced command (a_{enf}, "Allowed"):** Logged on the bare-metal MCU on the permission side of the boundary, this represents the command after the permission path has checked or modified the proposal. The checks include deterministic Control Barrier Functions (CBFs, safety filters that keep states within a forward-invariant safe set), velocity and acceleration clamps, torque saturation envelopes, and collision guards. Whichever tap a dataset chooses as its label, the imitation loss has the same form; written for the admitted command, it is:

$$\mathcal{L}_{\text{policy}}(\theta) = \mathbb{E}_{(\mathbf{o}_t, \mathbf{a}_{\text{enf},t}) \sim \mathcal{D}_{\text{demo}}} \left[\mathcal{L}(\pi_\theta(\mathbf{o}_t), \mathbf{a}_{\text{enf},t}) \right],$$

  Here the training dataset must contain curated, correctly attributed targets. Passing the permission check does not make a command an expert demonstration, because a clamp or emergency stop may be a safeguard's response to a poor request. Override flags and companion

taps determine which records can serve as imitation targets and which remain evidence of intervention.

- **Tap 4: The measured response (a_{meas}, "Done"):** The physical plant reaction measured by optical joint encoders and motor current shunt resistors over the executed control tick, recording actual mechanical displacement and electrical effort. This measured response serves as proprioceptive state input ($\mathbf{o}_{\text{proprio},t+1}$) for subsequent policy steps rather than an action label.

These distinctions become critical when commanded intent and physical motion diverge. If a teleoperator requests an aggressive movement that the safety permission path clamps, the resulting state transition follows the enforced command (a_{enf}), not the unconstrained request (a_{req}). Pairing that transition only with the request conceals the safety intervention and teaches a policy to repeat demands the physical machine cannot execute.

↪ Control barrier functions and the forward-invariant safe sets they enforce are developed in section 12.5.

At the latch, the opposite ambiguity arises. Contact resistance can bring measured joint velocities to zero ($\dot{\mathbf{q}}_{\text{meas}} \approx \mathbf{0}$) while the motors continue exerting holding torque against the strike plate. Treating that response as a deliberate zero-velocity target loses the distinction between maintaining effort against the latch and choosing to stop. The appropriate imitation target depends on the policy's action interface; the other taps explain how that target became physical motion.

Retaining all four streams is what makes any single choice of label interpretable. A policy trained on the request alone learns commands the permission path would have clamped, and one trained on the measured response alone learns the arm's compliance, backlash, and external disturbances as if they were intended control. A log that keeps only the final encoder state or only the operator's input discards exactly the stages at which software and safety hardware changed the command. The tuple is also only as reliable as its shared clock. The application processor and the MCU must stamp their taps in one hardware-synchronized timebase, such as the IEEE 1588 Precision Time Protocol (PTP), written t_{PTP}. A few milliseconds of skew between them misaligns each visual observation from the torque it prompted (section 5.5.2).

For the latch campaign, the imitation target is the mapped command whenever it represents an expert request that the permission path accepted unchanged. In that case a_{cmd} and a_{enf} carry the same value, so the choice between the two taps matters only for commands the permission path modified. A modified command remains a separate, attributed intervention; a curated recovery can later supply an expert target at that state. Each training pair therefore carries two times. The observation is dated when its physics was true, at the exposure midpoint (the instant halfway through the interval over which the camera integrates light), and the command is dated when the operator issued it. The gap between them is part of the demonstration and is recorded rather than erased, because it contains the operator's own reaction time (section 5.6.1). The alignment developed below supplies the first of these times.

5.5.2 Multi-rate time synchronization and exposure-midpoint alignment

↪ The capture timestamp, taken in hardware at the exposure midpoint, is enforced for every sensor in section 8.3.

Recording the right action tap still leaves open when each recorded observation was physically true, because a robot's sensors run at different rates and transport delays. A naive data loader that latches the latest available sensor sample pairs each action with a camera frame whose optical exposure was centered 25–40 ms before the frame arrived in host memory, recording that delayed image as the present state. Trained on those pairs, the policy learns to map historical visual states to present actions, baking an artificial phase lag into its weights that shows up at deployment as overshoot and chattering against the latch. Across the machine, cameras integrate photons over exposure windows of 10–35 ms at 30–60 Hz, 6-axis force-torque sensors stream over a Serial Peripheral Interface (SPI) or Controller Area Network (CAN) bus at 100–500 Hz, joint and permission telemetry is retained at the MCU's chosen 1 kHz tick while motor-drive current sampling serves the faster local current loop, and neural policy models dispatch setpoints at 10–50 Hz. Unlike the operator's reaction time, this pairing error belongs to the logging pipeline rather than the demonstration, so stamping each sample at its physical capture instant lets the alignment pipeline remove it.

To quantify the physical hazard of temporal misalignment, consider the spatial tracking error $\Delta\mathbf{x}_{\text{sync}}$ injected into the dataset when an observation and an action are paired across an uncompensated phase lag Δt_{lag}:

$$\Delta\mathbf{x}_{\text{sync}}(t) = \int_{t-\Delta t_{\text{lag}}}^{t} \mathbf{v}(t')\,dt' \approx \mathbf{v}(t) \cdot \Delta t_{\text{lag}}. \tag{5.3}$$

The latch campaign moves far more slowly than an arm in transit, and notebook 5.1 applies equation 5.3 at both speeds.

Napkin Math 5.1: The pairing lag

Problem: A logger pairs each action with a camera frame whose exposure midpoint lies an uncompensated 35 ms earlier. Does the resulting pairing error fit the 2 mm latch clearance at transit speed and at the latch campaign's two approach speeds, and how much lag can each approach tolerate?

Variables:

- Uncompensated lag $\Delta t_{\text{lag}} = 35$ ms (illustrative; exposure-to-arrival delay of an untimed logger)
- Latch clearance $c = 2$ mm (illustrative)
- Transit speed 0.50 m/s (illustrative); guarded approach 0.03 m/s and faster approach 0.10 m/s (illustrative scenario speeds of the running machine)

Math:

1. **Transit**: 0.50 m/s $\times$ 35 ms $=$ 17.5 mm, 8.8× the clearance.
2. **Guarded approach**: 0.03 m/s $\times$ 35 ms $=$ 1.05 mm, inside the clearance.
3. **Faster approach**: 0.10 m/s $\times$ 35 ms $=$ 3.5 mm, outside it.
4. **Lag ceilings**: $\Delta t_{\text{lag,max}} = c/v_{\text{app}}$, so 2 mm / 0.03 m/s $=$ 66.7 ms and 2 mm / 0.10 m/s $=$ 20 ms.

Result: The same lag fits the modeled pairing-error allocation at the guarded approach and exceeds it at the faster one.

Systems insight: A timing error becomes a position error at the speed of the motion, just as observation age becomes blind travel $v\,\tau_{\text{delay}}$ in the stopping budget derived in section 2.3.1. The faster approach cannot be collected without the exposure-midpoint alignment of the four-stage pipeline in section 5.5.2.3.

When an imitation learning model trains on misaligned pairs, it fits a relationship in which the action appears to precede the visual evidence that prompted it. During closed-loop deployment, that artificial phase lag appears as overshoot, control hunting, and chattering against rigid surfaces. Because displacement grows linearly with lag, each approach speed has a lag ceiling equal to the clearance divided by the speed, and figure 5.3 places those ceilings beside a second timing error the notebook did not count. A rolling-shutter sensor exposes its pixel rows one after another rather than all at once, so the bottom row of a frame is captured later than the top (section 5.5.2.2 quantifies the delay). A full-frame rolling-shutter skew of 15 ms to 33 ms is comparable to the uncompensated lag, so a logger that stamps every frame correctly can still breach the faster approach's 20 ms ceiling through readout alone.

5.5.2.1 Hardware timestamps vs. software arrival timestamps

The 35 ms pairing lag in notebook 5.1 arises when a logger stamps a sensor packet with the clock reading at the instant a Linux thread wakes up to read it rather than when the hardware captured the sample. Even on the 1.0 ms kinematic stream, half a millisecond of kernel scheduling jitter cuts a nominal tick in half or stretches it by half, doubling or shrinking the finite-difference velocity label for the exact same physical motion. Both errors come from confusing when an optical or electrical event happened in hardware with when the operating system noticed it in software:

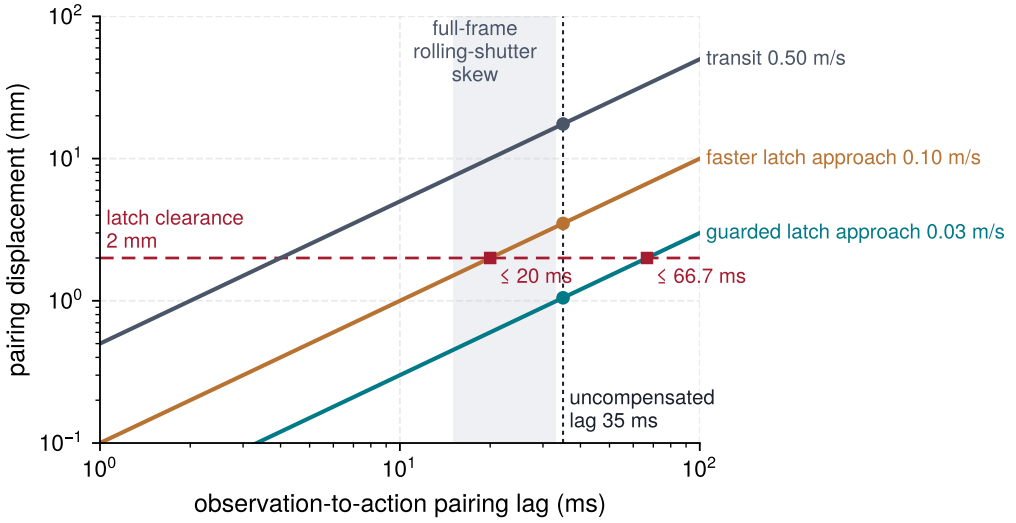


Figure 5.3: One lag, three verdicts: Pairing an observation with an action across an uncompensated lag displaces the recorded state by speed times lag. At 35 ms the transit pairing is 17.5 mm off, the faster latch approach exceeds the 2 mm clearance, and the guarded approach stays inside it. The clearance tolerates at most 20 ms of lag at 0.10 m/s and 66.7 ms at 0.03 m/s; a full-frame rolling-shutter skew alone spans 15 ms to 33 ms.

- **Software arrival timestamps (t_{host}):** In naive logging pipelines, an image frame or CAN packet is timestamped when the Linux kernel handles the device interrupt or when a user-space thread dequeues the buffer from Video4Linux2 (V4L2) or a socket. This host timestamp incorporates camera pixel readout time, USB 3 or MIPI CSI-2 bus serialization, Direct Memory Access (DMA) latency, and operating system scheduling jitter:

$$t_{\text{host}} = t_{\text{mid}} + \frac{t_{\text{exp}}}{2} + t_{\text{readout}} + t_{\text{bus}} + t_{\text{driver_irq}} + \delta_{\text{jitter}}.$$

  Host timestamps lag physical reality by 20–45 ms. In an untuned system, operating system scheduling jitter ($\delta_{\text{jitter}} = \pm 5$–$15$ ms), driven by kernel preemption latency, CPU C-state power wake-up transitions, and USB host controller interrupt coalescing (where packets are held in hardware buffers until a threshold count or timer expires), corrupts numerical differentiation. When joint velocities are computed via backward difference:

$$\dot{\mathbf{q}}_k = \frac{\mathbf{q}_k - \mathbf{q}_{k-1}}{t_k - t_{k-1}},$$

  a nominal 1 kHz sampling interval ($\Delta t = 1.0$ ms) perturbed by software timestamp jitter $\delta_{\text{jitter}} = \pm 0.5$ ms causes the denominator to fluctuate between 0.5 ms and 1.5 ms. The effect is doubling the inferred velocity at the shorter interval and reducing it by one third at the longer interval, for the same measured displacement. If thread preemption reverses packet dequeuing order ($t_k < t_{k-1}$), the computed velocity reverses sign, injecting spurious velocity spikes into the training targets.

- **Hardware PTP timestamps (t_{PTP}):** Hardware timestamps isolate the physical event from host scheduling delays. IEEE 1588 Precision Time Protocol (PTP) estimates clock offset across distributed nodes using a deterministic two-way message exchange.[5] The reference clock sends at t_1 and receives the reply at t_4; the remote clock receives at t_2 and replies at t_3. Under symmetric path delays, the propagation delay and offset are:

$$T_{\text{prop}} = \frac{(t_4 - t_1) - (t_3 - t_2)}{2}, \qquad \theta_{\text{offset}} = \frac{(t_2 - t_1) - (t_4 - t_3)}{2}.$$

  Because the forward timestamp difference $t_2 - t_1 = T_{\text{prop}} + \theta_{\text{offset}}$ adds the remote clock offset to the one-way propagation delay while the return difference $t_4 - t_3 = T_{\text{prop}} - \theta_{\text{offset}}$ subtracts

5 **Precision Time Protocol (PTP)**: IEEE 1588 was first published in 2002 for test-and-measurement and industrial-automation networks, and its current revision dates from 2019 (IEEE 2020a). It reaches sub-microsecond agreement only when both endpoints timestamp packets in hardware at the network interface. Its weak point is the symmetric-path assumption in the offset formula, because a link whose two directions differ in delay biases the computed offset by half that difference, and every observation-action pairing in the archive inherits the bias.

that same offset, adding the two differences cancels θ_{offset} to isolate the propagation delay and subtracting them cancels T_{prop} to isolate the clock offset. Hardware timestamping bounds uncertainty, but a network packet timestamp does not automatically equal an optical exposure midpoint. Production logs must preserve the capture event, clock domain identity, and bounded conversion error. The key collection decision is whether that uncertainty fits within the task's spatial alignment tolerance.

↪ **Protocol:** The IEEE 1588 message exchange, its clock servo, and its path-asymmetry uncertainty are developed in section C.7.

5.5.2.2 Rolling shutter skew across camera streams

When a camera with a rolling shutter pans past the cage door or rides on a moving link, straight vertical bars bend into diagonal arcs because the bottom pixel row is exposed up to 33 ms later than the top row. This distortion is **rolling shutter skew**: low-cost consumer sensors and webcams expose pixel rows one after another rather than all at once. For a sensor with H vertical scanlines and line readout duration t_{row}, row $y \in [0, H-1]$ begins its exposure at $t_{\text{scan}}(y)$, and its exposure midpoint, half an exposure later, depends linearly on its vertical coordinate:

$$t_{\text{scan}}(y) + \frac{t_{\text{exp}}}{2} = t_{\text{start}} + y\, t_{\text{row}} + \frac{t_{\text{exp}}}{2}.$$

Across a full frame, the temporal skew between the top row ($y = 0$) and bottom row ($y = H-1$) accumulates to:

$$\Delta t_{\text{skew}} = (H-1)\, t_{\text{row}}, \tag{5.4}$$

which for common sensors lies between about 15 ms and 33 ms, the upper end being one full frame period at 30 Hz. When the robot arm or camera moves at 0.50 m/s, the bottom of the manipulator in the image is captured while the physical link has moved up to 16.5 mm relative to the top of the link in the same frame. Even at the faster latch approach of 0.10 m/s, the worst-case skew moves the gripper 3.3 mm within one frame, exceeding the 2 mm clearance. This geometric warping (the "jello effect") deforms straight edges into curved arcs and breaks rigid-body geometry.

Independent rolling shutters can depict the same moving arm at different poses within nominally simultaneous frames. Hardware-triggered global shutters simplify alignment; rolling shutters require a row-time model and a motion bound or correction. Sensor choice must follow the task tolerance rather than an unyielding demand for a single shutter architecture.

↪ The same row timing as a spatial error on a moving base, charged against clearance and removed by per-row motion compensation, is developed in section 8.2.

5.5.2.3 A four-stage alignment pipeline

Hardware timestamps fix when each sample was taken, and the row-time model fixes when each part of an image was exposed. A collection pipeline still has to turn those facts into training pairs whose pairing error fits inside the clearance budget of notebook 5.1. It can implement that alignment contract in four stages:

1. **Hardware-triggered optical exposure midpoint:** The real-time MCU dispatches hardware GPIO sync pulses or PTP-synchronized trigger packets to every triggerable camera. The rig mixes shutter types. The wrist camera's global shutter exposes all rows together, while the navigation camera and the external cameras' color sensors read out row by row, so their frames also need the row-time model above to assign each row its own midpoint. The camera latches the trigger at t_{start}, and the driver calculates the true optical midpoint $t_{\text{mid}} = t_{\text{start}} + t_{\text{exp}}/2$ in hardware PTP clock units (t_{PTP}), avoiding host arrival time as the exposure timestamp.[6]
2. **Lock-free time-sorted jitter compensation:** Because network and bus transfers experience variable transport delay δ_{jitter}, incoming sensor packets are stamped with their hardware PTP capture timestamp and inserted into a lock-free, time-sorted jitter ring buffer in shared memory. An asynchronous consumer thread advances a playhead lagging real-time by a fixed, bounded window ($T_{\text{buf}} = 15$–25 ms), absorbing transport bursts and re-ordering packets into monotonic hardware timestamp sequence.
3. **Cubic Hermite kinematic interpolation:** Because joint encoders sample at 1 kHz (1 ms period) while cameras capture at 30 Hz or 60 Hz, the camera midpoint t_{mid} falls between two discrete MCU telemetry ticks t_k and t_{k+1}. The synchronization thread interpolates joint angles

[6] **Hardware Exposure Triggering**: A dedicated GPIO or TTL pulse from the real-time microcontroller starts every camera's exposure on the same electrical edge, instead of a software trigger issued through the host's driver stack. Triggered global-shutter cameras then begin exposing within microseconds of one another, which leaves the exposure duration, not the trigger path, as the dominant term in the midpoint timestamp.

$\mathbf{q}(t_{\text{mid}})$ using cubic Hermite splines across the bounding kinematic records $(t_k, \mathbf{q}_k, \dot{\mathbf{q}}_k)$ and $(t_{k+1}, \mathbf{q}_{k+1}, \dot{\mathbf{q}}_{k+1})$:

$$\mathbf{q}(t_{\text{mid}}) = h_{00}(\lambda)\mathbf{q}_k + h_{10}(\lambda)\Delta t_s \dot{\mathbf{q}}_k + h_{01}(\lambda)\mathbf{q}_{k+1} + h_{11}(\lambda)\Delta t_s \dot{\mathbf{q}}_{k+1},$$

where $\Delta t_s = t_{k+1} - t_k = 1$ ms, $\lambda = (t_{\text{mid}} - t_k)/\Delta t_s \in [0,1]$ is the fraction of the tick elapsed at the exposure midpoint, and $h_{ij}(\lambda)$ are the cubic Hermite basis functions (section B.5). This gives a smooth state estimate between valid samples. The interpolation error still needs a bound, especially across contact or missing telemetry.

4. **Pre-serialization alignment:** Container formats store multi-rate topics with the timestamps provided by the logger. Alignment may occur during capture or in a versioned preprocessing pass. Preserve the raw timestamps and interpolation provenance in either case, so the training pair carries an estimated state and its uncertainty rather than an assertion of exact simultaneity.

The same pairing contract governs deployment. A policy trained on aligned pairs must also be fed aligned observations and have its actions executed when the training data implies. UMI measures each observation stream's latency and aligns all streams to the slowest, usually the camera. It then sends commands ahead of time to cover the arm's and gripper's execution latency, and it discards actions whose execution time has already passed. With latency matching disabled, the same policy's motion became visibly jittery (Chi et al. 2024). UMI also expresses actions relative to the gripper's current pose rather than in a fixed robot frame, which removes the dependence on a base calibration and lets one policy run on more than one arm.

> Checkpoint 5.1: Action labels and pairing time
>
> A training pair is only as trustworthy as the tap it imitates and the time it assigns to each observation.
>
> - ☐ **Clamped request**: A teleoperator requests a motion that the permission path clamps. Which tap does the resulting state transition follow, and where does the record keep the evidence of the clamp?
> - ☐ **Stalled latch**: Measured joint velocity is zero while the motors hold torque against the latch. Why is a_{meas} the wrong imitation target at that step?
> - ☐ **Unchanged mapped command**: Under what condition can a_{cmd} serve as the expert target, and what happens to the pair when that condition fails?
> - ☐ **Lag budget**: A tighter strike plate leaves only 1 mm of clearance. At the faster approach of 0.10 m/s, what is the largest uncompensated lag the pairing tolerates, and does a logger with the 35 ms lag of notebook 5.1 meet it?

A tap and a timestamp are only as good as the loop that produced them. In teleoperation that loop runs through a human, and the interface between human and machine shapes the recorded dynamics, storage bandwidth, and hardware wear long before the data reaches a model (section 5.6).

5.6 Teleoperation Demonstration Costs

When an operator guides a leader arm to unlatch the warehouse cage door, every corrective nudge must traverse a multi-stage physical loop before the robot can respond. Teleoperation is a data-generation pipeline with a human embedded in the real-time feedback loop. It charges for each demonstration in three currencies: the phase lag and stability margin of the loop itself, the write bandwidth and storage the session demands, and the cell time and component life a campaign consumes. The loop comes first (figure 5.4). The operator sees the scene on a display after a delay t_{disp} and perceives, decides, and moves the leader in t_{hum}. The command crosses to the follower in t_{cmd}. The follower's actuators then accelerate the links against inertia, drivetrain friction, and contact in t_{mech}, before the camera shows the result on the next cycle.

↪ Where a deployment moves operators' labor, and which uses of their recorded sessions it must bar, is developed in section 17.8.

By mapping each leader joint directly to the corresponding follower joint ($\mathbf{q}_{\text{leader}} \equiv \mathbf{q}_{\text{follower}}$), this isomorphic leader-follower topology sidesteps numerical inverse kinematics and algorithmic

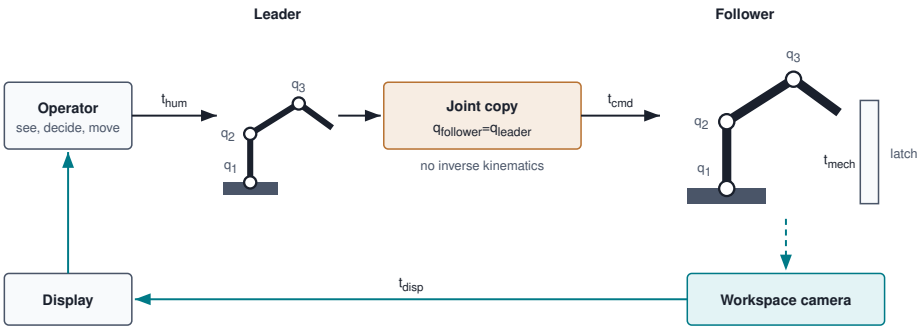
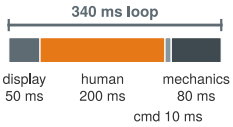


Figure 5.4: Kinematically isomorphic teleoperation loop: The operator moves a leader arm whose joints mirror the follower's, and the host copies joint angles across, as in low-cost leader-follower rigs such as GELLO (P. Wu et al. 2024). Because the leader shares the follower's joint structure, the interface needs no inverse kinematics and inherits none of its singularities (configurations where joints align and the arm loses a direction of motion), preserving the operator's direct joint-level coordination. The loop still closes through the operator: human reaction, command transfer, mechanical response, and display delay all enter the recording as phase lag, so the demonstration carries the operator's delay as if it were policy.

kinematic singularities. Even without software kinematic conversion, however, every recorded trajectory remains bound to the physical transport and neuromuscular delays of the complete loop, placing hard upper bounds on corrective bandwidth.

5.6.1 Human-in-the-loop latency and bilateral passivity breakdown



The operator's own reaction is 58.8 percent of the 340 ms correction loop.

Suppose the follower gripper grazes the side of the latch strike plate at time t. If a logger naively pairs the observation $\mathbf{o}_t$ captured at that instant with the command the operator issued at t, it pairs a contact state with an action chosen for a trajectory state 250 ms earlier, before the contact ever happened. A behavioral cloning policy trained on those pairs learns that delay as intentional strategy and, when run autonomously at high control frequencies, over-corrects into destructive oscillations. That stale pairing arises because every stage of the teleoperation loop adds phase delay. In an illustrative reference setup with a high-speed camera and a wired local network, display pipeline processing takes $t_{\text{disp}} = 50$ ms across sensor exposure, H.264 decoding, and display raster refresh. Human visual perception, motor planning, and neuromuscular transmission add an operator response latency of $t_{\text{hum}} = 200$ ms. Command quantization, packetization, and fieldbus transport add $t_{\text{cmd}} = 10$ ms. Actuator torque ramp-up against link inertia then adds a mechanical response latency of $t_{\text{mech}} = 80$ ms before the arm reaches the commanded acceleration. Their sum is the **teleoperation round-trip latency** t_{loop}, the elapsed time from a physical contact disturbance to the corrective actuator response:

$$t_{\text{loop}} = t_{\text{disp}} + t_{\text{hum}} + t_{\text{cmd}} + t_{\text{mech}}. \tag{5.5}$$

Following the contact at time t through those four stages shows why the recorded command lags the state. The operator sees the error on the display 50 ms later and commands a physical correction at $t +$ 250 ms. The follower arm finally pushes back at $t +$ 340 ms, after drifting under momentum and contact compliance throughout the interval. Round-trip latency is therefore a property of the resulting dataset rather than just an artifact of the rig, because it injects a systematic phase lag into every recorded demonstration, dominated by the operator's own reaction time (58.8 percent of the loop in this setup).

This delay differs in kind from the sensor-pipeline delay of section 5.5.2. That one was a pairing artifact of the logger, and exposure-midpoint stamping lets the alignment pipeline account for it. The operator's delay is a real property of the demonstrator, so no timestamp correction can remove it, and silently erasing it would misstate what the demonstration contains. The logger therefore keeps both times of every pair (the observation's exposure midpoint and the command's issue time) and records the session's measured loop latency. Training can then treat the observation-to-action interval as a declared part of the demonstration, either by conditioning the policy on enough

observation history to span it or by pairing each command with the earlier frame the operator was answering. Deployment can then match the interval it was trained with, as UMI's latency matching does (section 5.5.2.3). Because the operator's reaction varies from command to command, no single fixed shift recovers the exact frame each command answered, which is why the latch campaign records the interval instead of baking one correction into the archive.

Force reflection introduces a deeper stability challenge into the collection interface. In **bilateral teleoperation**, the operator manipulates the leader device while measured contact forces from the follower arm reflect back across a delayed communication channel. Over a closed motion cycle of period T_{cycle}, the net mechanical energy delivered at that port is:

$$\Delta E = \int_0^{T_{\text{cycle}}} F_{\text{contact}}(t - T_d) v(t)\, dt.$$

The sign of ΔE depends on the force–velocity phase relationship. Under the declared port orientation, energy delivered in excess of received and initially stored energy violates passivity;[7] a positive value over one cycle alone does not establish that excess. In an undelayed elastic contact, the reflected force opposes the leader handle during compression and drops as the handle rebounds, whereas transmission delay T_d causes peak contact force to continue pushing the leader handle after the operator has already reversed direction, aligning force with rebound velocity $v(t)$ to pump net mechanical energy into each oscillation cycle and excite contact instability (Anderson and Spong 1989). The wave-variable transformation derived in section B.10 keeps the ideal delayed channel passive and lossless under its constant-delay model, while passivity observers (monitors that track the energy a port has generated and dissipate any excess) restore passivity by injecting variable damping. Preserving channel passivity imposes a force-fidelity cost at contact. Injected damping, and the explicit low-pass filtering often added to delayed force channels, suppress the fastest components of the reflected force first, so a stabilized channel behaves as a low-pass filter on contact. In data collection, this imposes an unavoidable label-fidelity trade-off. The filtering that stabilizes delayed haptic feedback also attenuates the sharp contact transients the operator needs to demonstrate precise tactile insertion. System builders must log the active feedback architecture and filter cutoffs alongside raw trajectories to document this phase distortion.

The mechanical interface therefore filters which behaviors can reach the archive at all. A force-reflecting leader allows an operator to feel mechanical binding and ease off insertion forces. A visual-only Cartesian rate controller instead forces the step-and-wait hunting of section 5.4.1, because the operator must wait for display frames to confirm each small motion. A backdriven joint-space leader such as ALOHA, recorded at 50 Hz (Zhao et al. 2023), occupies an intermediate design point, providing inertial and friction feedback from the leader mechanism itself without reflecting follower contact forces. Across all these interfaces, human operators instinctively decelerate prior to contact, because closing a high-gain corrective loop around round-trip latency t_{loop} would erode phase margin to zero. Table 5.2 categorizes the dominant teleoperation architectures and the specific data collection artifacts they produce.

7 | **Passivity**: A passive system cannot deliver more energy through its ports than it has received or initially stored. A well-posed, power-conserving interconnection of passive components preserves their combined passivity inequality. Stronger stability conclusions require bounds on stored energy and appropriate strictness or detectability conditions, as developed in section B.10. Anderson and Spong (1989) showed that a delayed communication channel need not be passive and restored passivity for any constant delay with a scattering transformation, and Niemeyer and Slotine (1991) recast the idea as wave variables. The price is paid in force fidelity at contact, and the demonstrations record that price.

Table 5.2: Teleoperation interface mechanisms for physical demonstration collection: Mapping and feedback determine what the operator can demonstrate, and the last column names the artifact to check on a given rig, task, and operator.

Interface and feedback	Command mapping	Contact information available to operator	Possible collection artifact to check
Force-reflecting bilateral leader	Leader motion mapped to follower joints; measured follower force reflected to leader	Vision and physical contact force, subject to link delay and fidelity	Contact corrections and oscillation when delayed force reflection loses passivity
Backdriven joint-space leader (ALOHA) (Zhao et al. 2023)	Leader joint positions copied to follower; demonstration recorded at 50 Hz	Vision and leader mechanism feel, without active follower force reflection	Joint-space posture preferences and unobserved follower contact forces
Rate-controlled 3D mouse	Device deflection mapped to Cartesian velocity, then inverse kinematics	Primarily visual feedback unless a separate haptic channel exists	Step-and-wait motion and zero-velocity pauses under delayed feedback

Interface and feedback	Command mapping	Contact information available to operator	Possible collection artifact to check
Tracked spatial controller	Tracked hand pose mapped to tool pose with clutching and scale	Visual feedback; optional vibration is not directional force reflection	Clutch discontinuities and pose-tracking gaps
Instrumented exoskeleton	Wearable joint motion retargeted to robot joints	Depends on whether active force feedback is fitted	Biomechanical calibration and retargeting bias

5.6.2 Ingestion rate and the preservation requirement

The loop's latency is the first cost a demonstration carries; the second is what it takes to write everything the loop produced to disk without silent data loss. If an onboard logger tries to stream raw multi-camera video and 1 kHz kinematics onto a SATA solid-state drive, write buffers overflow within seconds, dropping either the camera frames or the four action taps of section 5.5.1 that explain what the permission path did. Whether an edge collection computer can sustain a teleoperated session without dropping samples is storage-bus arithmetic: the aggregate write rate set against the sequential write bandwidth of the drive.

Napkin Math 5.2: The cheapest stream

Problem: One teleoperated door-opening session on the warehouse mobile manipulator logs every camera and every axis raw. What sustained write rate and storage does it demand, and what share of the bytes carries the four action taps?

Variables (camera geometry is datasheet; logging formats and shift length are chosen):

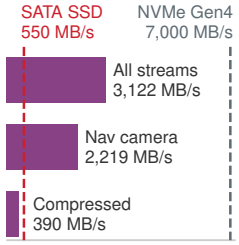
- **Navigation camera:** 4056 by 3040 px at 60 Hz, logged as 24-bit RGB (3 bytes/px).
- **Wrist camera:** 1440 by 1080 px at 60 Hz, logged the same way.
- **External cameras:** 2 RGB-D cameras viewing the door, color at 1920 by 1080 px and 30 Hz, with depth aligned to the color frame as raw `uint16` millimeters (2 bytes/px).
- **Kinematics:** 9 axes (two base wheels, seven arm joints) logged at 1 kHz. Each step stores, for every axis, 6 values of 4 bytes (position and velocity, which form the measured response a_{meas}; measured torque; and the requested, mapped, and enforced commands), plus one 8-byte timestamp.
- **Shift:** 8 hours of collection on one machine.

Math:

1. **Kinematic stream**: $9 \times 6 \times 4\ \text{B} + 8\ \text{B} = 224\ \text{B}$ per step, and at 1 kHz that is 0.22 MB/s.
2. **Navigation camera**: $4056 \times 3040\ \text{px} \times 3\ \text{B} \times 60\ \text{Hz} = 2219.44$ MB/s. The wrist camera's product is 279.94 MB/s.
3. **External pair**: $2 \times 1920 \times 1080\ \text{px} \times (3\ \text{B} + 2\ \text{B}) \times 30\ \text{Hz} = 622.08$ MB/s, of which 373.25 MB/s is color and 248.83 MB/s depth.
4. **Total and storage**: The streams sum to 3121.68 MB/s, about 3.12 GB/s; over one hour that is 11.24 TB, and over the shift 89.9 TB.

Result: Raw logging needs a sustained 3.12 GB/s write path, and the kinematic stream is 0.007 percent of the bytes.

Systems insight: The navigation camera alone writes 4.0× what a SATA SSD sustains (550 MB/s). The full session fits within a Gen4 NVMe drive's 7 GB/s, but only by consuming storage at 11.24 TB per hour. Intra-frame video compression at an illustrative 8× brings the rate to about 390.2 MB/s (1.40 TB per hour) at the cost of encoder latency and artifacts in the frames the policy will train on. The kinematic stream carries all four action taps, so the record that preserves the causal chain is the cheapest stream to log, and a logger that drops data under buffer pressure must never drop it first.

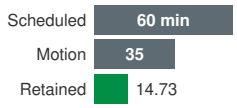


The navigation camera alone writes 4.0× what a SATA SSD sustains; only the compressed session fits under it.

Keeping that write rate under the drive ceiling with video compression trades storage bandwidth against visual fidelity, because lossy encoders discard fine spatial detail at the latch edge and inject blocking artifacts into the frames the policy trains on. A session can be stored in custom HDF5 files, ROS 2 MCAP bags (a container format for multimodal robotics data), or standardized schemas such as LeRobot (Cadene et al. 2024). Whichever format is used, the logger must flush complete chunks to non-volatile storage before a power drop or process halt can truncate the write buffer. It must also carry all four control taps, hardware timestamps, clock-domain identifiers, missing-sample flags, and spatial calibration parameters through every format conversion. Otherwise, a file that deserializes without error proves only that its byte layout is intact, not that its sensory and motor streams remain physically synchronized.

5.6.3 Operational yield and mechanical fatigue budgeting

Sizing the NVMe drive settles what one hour in the workcell writes to disk, but less than a quarter of that scheduled hour survives as usable demonstration motion. Before the arm records a single latch opening, a scheduled hour in the workcell pays for bus synchronization, camera and torque-sensor calibration, and mandatory operator rest. The active motion window that remains is further divided among door-reset strokes, mid-trial contact trips, and offline quality filtering. Because every attempt, whether retained or discarded, flexes the wrist cable harness and reverses load on the harmonic drive, a collection campaign is bounded as much by component fatigue life as by facility reservations.



In this illustrative session, setup, rest, resets, and rejected attempts leave under a quarter of the scheduled hour as retained demonstrations.

The ultimate metric of collection efficiency is **demonstration yield**: the number of validated, policy-ready trajectory records produced in a scheduled session after accounting for cell startup, sensor calibration, operator rest, mechanical resets, and offline quality filtering. Given an active motion window T_{motion}, an attempt-and-reset cycle duration $t_{\text{cyc}} = t_{\text{att}} + t_{\text{rst}}$, and an end-to-end retention rate R, the session yields:

$$Y_{\text{usable}} = \left\lfloor \frac{T_{\text{motion}}}{t_{\text{cyc}}} \right\rfloor R. \tag{5.6}$$

Mechanical fatigue accumulates per physical motion cycle rather than per facility hour. This illustrative accounting assigns the same stress-cycle counts to failed and successful attempts and assumes component ratings expressed under the L_{10} convention. Under it, the rating is the cumulative cycle count at which 10 percent of an identical component population is statistically expected to fail.[8] Notebook 5.3 evaluates an illustrative door-opening collection session and its full campaign in both currencies: facility hours and component fatigue cycles.

[8] L_{10} **Rated Life**: The convention comes from rolling-bearing fatigue theory, where Lundberg and Palmgren (1949) modeled the scatter of bearing life as a probability of survival and rated a bearing by the life that 90 percent of a population reaches. A rating chosen that way is a population statistic, not a guarantee for one part. A fleet of ten collection arms, each run to its cable's rated life, should therefore expect about one cable to fail before the budget says it will.

Napkin Math 5.3: The flex-cable ceiling

Problem: The latch campaign must retain 5,000 demonstrations. How much scheduled cell time does that cost, and which wrist component's rated life does it spend fastest?

Variables (illustrative except the chosen session length and target):

- Session $T_{\text{sched}} = 60$ min (chosen), leaving an active window $T_{\text{motion}} = 35$ min after startup, calibration, and operator rest
- Attempt $t_{\text{att}} = 25$ s, one teleoperated opening; reset $t_{\text{rst}} = 15$ s, the door re-latched by its fixture
- In-trial failure rate 20 percent; post-collection quality filtering 15 percent
- Wear per attempt: $n_{\text{gear}} = 14$ torque reversals of the wrist's strain-wave gear; $n_{\text{cable}} = 8$ flex cycles of its internal cable bundle (6 while opening, 2 on the return stroke)
- Rated lives $L_{\text{gear}} = 5{,}000{,}000$ stress cycles and $L_{\text{cable}} = 200{,}000$ bending cycles

Math:

1. **Session**: $t_{\text{cyc}} = 25\text{ s} + 15\text{ s} = 40$ s, and $\lfloor 35\text{ min} / 40\text{ s} \rfloor = 52$ attempts, with 20 s left idle.
2. **Yield** (equation 5.6): $R = (1 - 0.20)(1 - 0.15) = 0.68$, so $Y_{\text{usable}} = 52 \times 0.68 = 35.36$ episodes, 14.73 min of validated motion, 24.6 percent of the scheduled hour.

3. **Campaign**: $\lceil$ 5,000 / 0.68 $\rceil$ = 7,353 attempts at 52 per session, or 141.4 h of scheduled cell time, 4.07× the 5,000 × 25 s of motion the campaign keeps.
4. **Wear**: 7,353 × 14 = 102,942 gear cycles, 2.1 percent of L_{gear}; 7,353 × 8 = 58,824 cable bends, 29.4 percent of L_{cable}.

Result: Less than a quarter of each scheduled hour survives as retained demonstration time, and one campaign spends far more of the cable's rated life than of the gear's.

Systems insight: The internal flex cable consumes the larger fraction of its assumed rating, informing maintenance planning rather than setting an individual replacement schedule. Three campaigns on one machine would consume most of its rated life, and an intermittent conductor break corrupts the very sensor streams being recorded. Because every attempt costs wear whether or not it is retained, the retention rate R divides both bills, while a faster reset shortens only the cell time.

Operator rest requirements and fatigue rates depend heavily on interface ergonomics and cognitive task difficulty; teams must empirically calibrate sustained operator throughput and retention on their specific hardware. A collection campaign must therefore be budgeted in both validated trajectory time and consumed fatigue life, not in facility clock hours. Table 5.3 breaks down the time and mechanical wear budgets of the illustrative session stage by stage.

↪ Per-cycle wear accounting for trial-based learning on the machine is developed in section 6.3.

Table 5.3: Physical demonstration collection budget across a scheduled 60-minute session: Illustrative facility clock time, yield losses, and component wear by stage for one session of the latch campaign ($N_{\text{att}} = 52$ attempts of $t_{\text{att}} = 25$ s, reset $t_{\text{rst}} = 15$ s). The 20 s of active time left after the last full cycle is idle and has no row. Less than a quarter of the hour survives as retained demonstration data.

Session Stage / Activity	Allocated Duration (Δt)	Wall-Clock Share (%)	Physical Operation & Control State	Hardware Stress & Wear Mechanism	Net Trajectory Output (N_{episodes} / T_{yield})
Cell Startup & Bus Sync	8 min (480 s)	13.3%	Bus synchronization (EtherCAT/CANopen), safety watchdog verification, optical homing	Power-on electrical inrush; thermal warm-up in motor windings and sensor bridges	$N_{\text{episodes}} = 0$ ep, 0.0 min yield
Sensor Calibration & Nulling	7 min (420 s)	11.7%	Camera intrinsic/extrinsic sweep, joint-torque bias nulling, encoder homing	Static holding torque; sensor bridge thermal stabilization; zero dynamic gear wear	$N_{\text{episodes}} = 0$ ep, 0.0 min yield
Operator Rest & Ergonomics	10 min (600 s)	16.7%	Mandatory operator breaks to mitigate visual fatigue, cognitive saturation, and tremor drift	Passive standby; electromagnetic brakes engaged or zero-velocity holding current	$N_{\text{episodes}} = 0$ ep, 0.0 min yield
Physical Scene Resets	13 min (780 s)	21.7%	Door closed and re-latched by its fixture, arm return stroke (15 s per attempt)	52 return strokes × 2 bends = 104 cable flex cycles; zero contact payload	$N_{\text{episodes}} = 0$ ep, 0.0 min yield
Active Teleoperated Attempts	21.67 min (1,300 s)	36.1%	Real-time human teleoperation guiding handle approach, latch release, and door swing (25 s per attempt)	52 × 14 = 728 torque reversals on harmonic drive; 52 × 6 = 312 cable flex cycles; contact monitored against the 15 N tripwire	$N_{\text{episodes}} = 52$ raw attempts, 21.67 min gross motion

Session Stage / Activity	Allocated Duration (Δt)	Wall-Clock Share (%)	Physical Operation & Control State	Hardware Stress & Wear Mechanism	Net Trajectory Output (N_{episodes} / T_{yield})
In-Trial Failures (Labeled)	-4.33 min (-260 s)	-7.2%	Handle slip, latch bind, contact tripwire above 15 N (20% loss)	High-force contact transients; abrasive wear on gripper silicone pads	-10.4 ep from success yield; valid logs kept
Post-Collection QA Filtering	-2.60 min (-156 s)	-4.3%	Offline filtering: link latency >100 ms, visual occlusion, operator hesitation (15% loss)	Zero hardware wear (offline computational validation pass)	-6.24 ep discarded
Net Validated Dataset Yield	**14.73 min (884 s)**	**24.6%**	High-quality synchronized multi-modal trajectories $(s_t, \mathbf{a}_t)$ admitted into training buffer	Consumes 2.1% of wrist gear life and 29.4% of internal flex cable life per 5,000-episode campaign	$Y_{\textbf{usable}} = \mathbf{35.36}$ episodes

Public collection shows what that yield arithmetic produces at community scale. The open-source LeRobot toolchain records teleoperated sessions in a common format and publishes them to the Hugging Face Hub with an auto-generated metadata card (Cadene et al. 2024). Figure 5.5 plots, from the Hub's own listing, how many such datasets exist and how many episodes each holds.

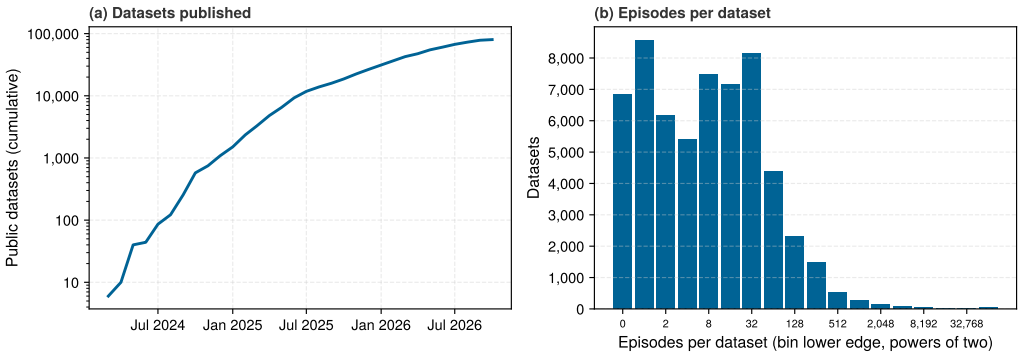


Figure 5.5: Many datasets, few episodes each: (a) Cumulative count of public datasets tagged with the LeRobot format on the Hugging Face Hub, by creation month; only datasets still public at retrieval are counted, so earlier values are lower bounds. (b) Episodes per dataset for the datasets whose card records a count, in power-of-two bins on a logarithmic axis. The count of datasets has grown by orders of magnitude while the typical dataset remains a single short session. Data retrieved from the Hub API on 2026-10-12.

The Hub held 1,085 such datasets at the end of 2024, 26,548 a year later, and 80,132 by the retrieval date. Episode counts tell a different story. Of the 59,031 datasets whose card records one, 58 percent hold fewer than 16 episodes, and only 2 percent hold 512 or more. The median dataset falls in the 8 to 15 episode bin, less than the roughly 35 validated demonstrations of the illustrative hour in table 5.3. Community data therefore grows wide rather than deep. The archive gains embodiments, scenes, and operators far faster than it gains repeated coverage of any one machine's states, and pooling it inherits the transfer problem of section 5.9.1.

The three costs compound into a fourth that no bill shows. Teleoperated demonstrations are bounded by what a human operator can perceive through video displays, coordinate through mechanical input devices, and stabilize against closed-loop delays. A campaign that pays for every attempt in cell time and cable life spends its budget where the operator is comfortable. The resulting dataset reflects the behavioral envelope of the coupled human-machine composite rather than the

robot's full physical dynamics. That selection bias decides which situations the archive covers, and measuring it requires a standard outside the archive (section 5.7).

5.7 Collection Policy Coverage

Every logged latch record carries an ingestion cost, an attributed action label, and a hardware timestamp, yet those fields say nothing about which physical situations the archive covers. In an illustrative contact task where a gripper aligns with a workpiece edge across contact angles $\theta_{\text{contact}} \in [-15.0^\circ, +15.0^\circ]$ and normal forces up to 20.0 N, binning at 1.0° and 1.0 N forms an operational grid of 600 discrete cells. Suppose two collection policies each record 1,000,000 transitions at 100 Hz with identical sensors. A cautious teleoperator keeps contact within $\pm 2.0^\circ$ of perpendicular under 4.0 to 6.0 N of normal force, populating 8 cells and leaving 98.7 percent of the grid empty. An exploratory policy approaches across the full angular range, slips, and loads the contact between 1.0 and 18.0 N, filling 510 cells and leaving 15.0 percent empty. Although both datasets have identical record counts, storage sizes, and hardware timestamps, they answer different operational questions. Record count therefore fails as a measure twice over: correlated control steps hold few independent samples (equation 5.2), and even independent samples support only the cells where they land.

That disparity arises because a physical dataset comes from running a closed-loop policy on hardware and logging the resulting transitions, violating the independent-and-identically-distributed assumption of standard empirical risk minimization (section 5.2). Instead, it traces an **empirical state-action occupancy distribution** (Spencer et al. 2021) $p^{\pi_{\text{col}}}(s, \mathbf{a})$ generated by a specific collecting policy π_{col} under its own control biases, interface delays, and risk tolerance. Formally, imitation learning minimizes prediction error weighted solely by this empirical occupancy:

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{(s,\mathbf{a})\sim p^{\pi_{\text{col}}}} \left[\ell(\pi_\theta(s), \mathbf{a}) \right].$$

The learned policy π_θ therefore receives supervisory gradient updates only over the empirical support hull $\text{supp}(p^{\pi_{\text{col}}}) = \{s \in \mathcal{S} \mid p^{\pi_{\text{col}}}(s) > 0\}$. Outside this narrow support hull, the network's outputs are unconstrained by training demonstrations.

Supervisory labels exist only where the collecting policy visited and commanded an actuator action. In unvisited states, the dataset contains neither an optimal control response nor proof that a safe trajectory exists. Supervised loss functions cannot distinguish an inherently unstable physical configuration from a benign unvisited state or an outright collision hazard. Whether an autonomous policy generalizes in such regions, fails silently, or triggers a safety stop cannot be deduced from the archive alone; resolving that uncertainty requires targeted recovery demonstrations, expert relabeling, or explicit run-time gating. Consider a policy trained exclusively on the cautious teleoperation data. If unexpected surface friction perturbs the gripper to contact angle $\theta_{\text{contact}} = +4.5^\circ$, outside the eight demonstrated cells, the policy network evaluates an input unconstrained by demonstrations. One control step later, its ungrounded action drives the contact to $+6.0^\circ$ at 11.0 N, another state lacking corrective labels in the training corpus. Compounding errors push the manipulator progressively farther from the nominal corridor until the gripper jams against the fixture. This cascade illustrates endogenous covariate shift (definition 5.1) triggered by a coverage gap.

5.7.1 Choosing a tractable coverage table

The contact-angle example uses a compact grid because it isolates the parameters relevant to a single mating feature. Extending that dense discretization across all robot state variables triggers the **curse of dimensionality in continuous physical interaction**.[9] Consider a mobile manipulator with a 7-degree-of-freedom (7-DoF) arm, a 2-DoF differential-drive base, a 6-axis force-torque sensor at the wrist, and a 6-DoF target workpiece pose. The continuous state space has dimension:

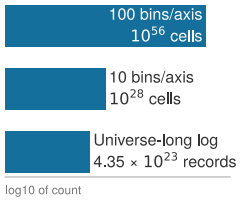
$$d_s = 7 \text{ (joint angles)} + 7 \text{ (joint velocities)} + 2 \text{ (wheel speeds)} + 6 \text{ (wrench components)} + 6 \text{ (workpiece pose)} = 28.$$

If an engineering team partitions each dimension into a crude $n_{\text{bin}} = 10$ bins (36° per angle bin and 10 N per wrench bin, far too coarse to guide contact tasks whose tolerances are a few millimeters), the discretized state space contains $n_{\text{bin}}^{d_s} = 10^{28}$ cells. A fleet of 1,000 robots logging at 1 kHz since the universe began, about 13.8 billion years ago, would hold 4.35×10^{23} records, enough to visit at most 0.004 percent of those cells even if every record landed in a different one. At a resolution

[9] **Curse of Dimensionality**: Bellman coined the phrase for dynamic programming, whose value tables grow exponentially with the number of state variables (Bellman 1957). A raw physical coverage table meets the same combinatorial barrier. Dynamic programming scaled by approximating value functions over compact feature spaces; a physical coverage table stays tractable the same way, by listing the few task parameters that determine success rather than the dense continuous state.

fine enough to guide contact ($n_{\text{bin}} = 100$ bins per axis), the count grows to 10^{56} cells. Brute-force observation cannot cover a continuous physical state space; a coverage table must describe the task with a few parameters instead.

Evaluating dataset completeness against the empirical extremes of collected values is fundamentally circular. A collection logger that records forces exclusively between 4.0 N and 6.0 N trivially reports full coverage of that observed interval. Coverage must instead be evaluated against the operational conditions that deployment demands, formulated before any demonstration data is collected.



A fleet logging since the universe began could visit at most 0.004 percent of even the coarse grid.

Definition 5.3: Operational design domain

Operational design domain (ODD) is the declared set of conditions (object poses, surface conditions, payloads, speeds, and disturbances) under which a machine is intended to operate, stated independently of any data collected within it.

1. **Significance**: The ODD serves as the denominator for every coverage claim. For example, the latch archive, as the join later in this section shows, populates only a single cell of the scenario table its door mechanism requires, a fraction that no volume of redundant nominal episodes can increase. Subsequent verification, evaluation, and release records can assert operational authority only over explicitly supported regions of the ODD.
2. **Distinction**: Deployment requirements define the ODD, whereas the collecting policy defines empirical occupancy. Coverage measured against observed telemetry extremes merely restates empirical occupancy in ODD terminology, producing an illusion of completeness. A **scenario table** partitions the continuous ODD into structured discrete cells so that deployment requirements and empirical occupancy can be joined.
3. **Common pitfall**: Defining the ODD post hoc based on conditions where the machine has already operated successfully rather than where it is required to function. Letting the dataset define its own requirements masks known unvisited failure modes.

In high-dimensional continuous systems, a scenario table avoids voxelizing raw camera pixel streams or continuous joint spaces, which would trigger an intractable combinatorial explosion. Instead, it parameterizes low-dimensional task invariants and environmental boundaries, such as workpiece pose offsets (± 5 mm), surface friction coefficients, approach speeds, lighting variations (100–1000 lux), and payload masses (0–2 kg). These parameters ground verification in concrete physical operating constraints. While this relational join identifies where the dataset holds empirical experience, populating a cell does not demonstrate that a model trained on those samples executes successfully. A populated cell provides supervisory targets, but verifying autonomous competence still requires closed-loop policy evaluation.

The missing recovery actions identified in figure 5.1 demonstrate why even a tractable scenario table must list off-nominal perturbations as cells of their own. Repeatedly executing a flawless motion samples progress along the nominal path, whereas corrective demonstrations supply actions that return the machine after an unexpected displacement. A sound scenario table distinguishes recovery experiences from nominal runs rather than treating repeated nominal cycles as coverage of the adjacent state space.

Unsupported operational regimes are exposed by a **support envelope relational join**, which cross-references empirical occupancy $p^{\pi_{\text{col}}}(s, \mathbf{a})$ (where the machine went) against the scenario table $\mathcal{S}_{\text{req}}$ (where the machine must be able to go). The table declares a count unit (transitions, independent episodes, or retained demonstrations), an eligibility and retention rule, and a minimum in that same unit. For each cell, the join tallies N_{cell} under this policy and partitions the table into cells meeting the minimum ($N_{\text{cell}} \geq N_{\text{min}}$), low-count cells, and unvisited blind spots ($N_{\text{cell}} = 0$). A populated cell has a nonzero count; it need not meet the minimum. The contact-angle example counts transitions, while the latch campaign's table counts retained demonstrations. These bins define collection categories. The threshold N_{min} represents a declared data requirement rather than a test of policy competence; neither an occupied cell nor a large sample count confers autonomous deployment authority.

A held-out validation split drawn from the same collecting policy can evaluate action prediction accuracy over that occupancy while remaining completely blind to gaps in the required operational domain. The comparison in table 5.4 separates collection coverage metrics from downstream policy outcomes. Its loss row is the root-mean-square error, in radians, of the predicted action on held-out records, and the cautious archive posts the lower value precisely because a narrow, repetitive archive is easier to fit. Low validation loss does not demonstrate recovery capability in unvisited conditions, just as wide exploratory coverage does not prove that collected corrective labels are dynamically stabilizing. Closed-loop evaluation must test the candidate policy directly against the declared task population.

Table 5.4: Empirical coverage and validation loss metrics across operational scenario bins: Illustrative counts and held-out losses compare two collection distributions; neither column supplies measured closed-loop capability.

Metric	Cautious Teleoperation ($p^{\pi_{\text{teleop}}}$)	Exploratory Policy ($p^{\pi_{\text{expl}}}$)	Systems Consequence
Recorded Transitions	1,000,000 transitions (100 Hz)	1,000,000 transitions (100 Hz)	Equivalent dataset scale
Operational Grid Coverage	8 / 600 bins (1.3%)	510 / 600 bins (85.0%)	Populated collection cells
Zero-Count Blind Spots ($N_{\text{cell}} = 0$)	592 / 600 bins (98.7%)	90 / 600 bins (15.0%)	Collection gaps
Held-Out RMS Action Error ($\mathcal{L}_{\text{val}}$)	0.001 rad	0.012 rad	Validation loss is non-diagnostic
Closed-Loop Capability	Not established by occupancy	Not established by occupancy	Requires candidate evaluation
Collection Decision	Target missing cells	Inspect labels and remaining gaps	Occupancy guides collection

The cage-door latch campaign illustrates this relational join for the running machine. Its six-cell example records occupancy, without supplying per-cell counts or a minimum-count requirement. Every retained episode approaches the strike plate at the guarded speed of 0.03 m/s with the plate aligned within ±0.5 mm of nominal, and every successful demonstration stays below the illustrative 15 N detection threshold. This observation does not itself qualify the collection response. Evaluated against an uncritical table containing only the nominal plate position at the guarded speed, the archive appears exhaustive. Against the scenario table the deployment actually demands, three plate positions (nominal and ±3 mm) crossed with two approach speeds (0.03 m/s and 0.10 m/s), the identical dataset populates only one cell of six. Every off-nominal plate displacement and every fast approach at 0.10 m/s contains zero demonstrations. The raw logged telemetry does not change between these two assessments; the difference lies entirely in the standard of audit applied. The five unvisited cells therefore travel with the dataset into training as documented operational absences, establishing that the current demonstrations provide no empirical foundation for competence claims in those regimes. Targeted collection can acquire the missing recovery trajectories, but downstream training and closed-loop testing must verify what the resulting policy achieves. Figure 5.6 sets the two audits side by side and adds a third, the stipulated recovery collection `latch-recovery-02` of section 5.8, which fills one more cell and still leaves four known absences.

Checkpoint 5.2: Empirical occupancy and scenario table joins

Before evaluating policy coverage across an operational design domain, audit dataset occupancy and scenario-table counts against these criteria:

- ☐ **Empirical support vs. validation loss**: Explain why an imitation policy can achieve near-zero validation loss on held-out test data while leaving 98 percent of the ODD's required cells unsupported.
- ☐ **Unsupported states**: What can an unvisited scenario cell establish about policy behavior, and what additional evidence or refusal decision is needed?

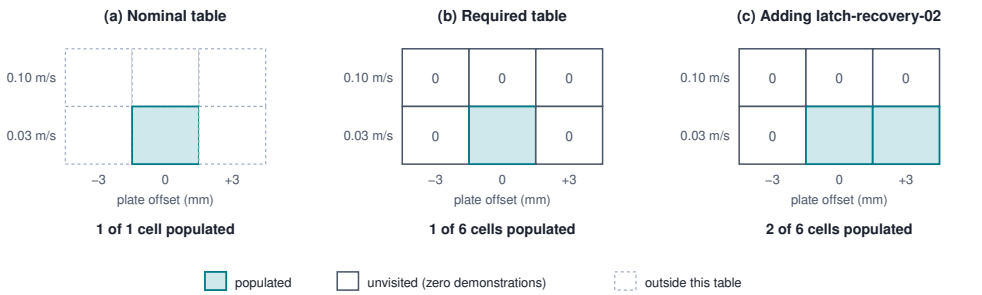


Figure 5.6: Scenario-table join for the latch: The same retained demonstrations joined against tables of strike-plate offset and approach speed. Against a table holding only the nominal plate at 0.03 m/s (a), the archive looks complete. Against the table the door needs (b), it fills one cell of six and the other five hold zero demonstrations. The stipulated recovery collection `latch-recovery-02` (c) adds the positive-offset guarded cell and leaves four known absences. Occupancy describes available experience, not evaluated competence or permission to act.

- ☐ **Scenario table joins**: Distinguish cells meeting the declared minimum, low-count cells, and unvisited cells when joining telemetry against an operational scenario table.
- ☐ **Table growth**: If the latch scenario table adds a third approach speed between the two it holds, how many cells does it contain, and how many of them are known absences for the original archive?

Logging additional nominal demonstrations along an unperturbed trajectory cannot eliminate coverage blind spots. Exposing the policy to recovery dynamics requires forcing the system toward its operational boundaries, either through deliberate demonstrator noise injection or through expert supervisory interventions (Kelly et al. 2019; Laskey et al. 2017). Before collecting either, a team must know why each empty cell is empty, and recorded interventions add a harder question, because they mix the learner's commands and the expert's in one trace (section 5.8).

5.8 Intervention and Recovery Data

A scenario cell, such as the cage-door latch displaced by $+3$ mm, may hold zero demonstrations in the training archive. That zero count alone cannot tell an engineer whether the robot never drifted there, whether the 15 N contact tripwire aborted every attempt that reached it, whether a safety supervisor took over beforehand, or whether an ingestion script threw the failed episodes away. Treating those four absences as interchangeable either trains the policy to imitate a safety abort or discards the exact boundary states needed for recovery. An absent state in an empirical log arises from one of four physical mechanisms during collection:

1. **Unvisited nominal path**: The collecting policy never visited the state because nominal rollouts remained entirely within the primary task corridor. This absence represents unmonitored nominal operation, verifiable by cross-referencing trajectory logs against the scenario table.
2. **Permission-path interception**: An active safety barrier or deterministic reflex in the permission path detected an impending boundary violation and truncated the motion before reaching the state, as the 15 N contact tripwire does during a misaligned latch attempt. The permission path logs record this barrier trip directly.
3. **Human supervisory takeover**: A human safety operator perceived a developing hazard and disengaged the autonomous policy before contact occurred. The permission path's log flags this disengagement explicitly.
4. **Post-collection censoring**: The machine entered the hazardous state and collided with an obstacle or jammed a joint, but an automated ingestion script discarded the episode as corrupt or incomplete. Dataset inspection cannot detect this absence because the failure transitions were purged before the training arrays were serialized to disk.

Each mechanism produces an identical zero count in the scenario table, yet each calls for a different response. An unvisited cell calls for targeted collection. An intercepted state belongs to the permission path, which already refuses it, and is not a target for the policy to learn. A takeover leaves learner-visited states that curation can turn into corrective supervision. A censored episode is a pipeline defect, recoverable only if the raw recording still exists. Conflating them leads to wrong conclusions about closed-loop policy stability, and the third mechanism needs the most care.

Human takeovers and automated disengagements introduce a selection trap into fleet data. An intervention occurs when the autonomous policy enters a region of state space it handles poorly and its tracking error or uncertainty crosses a threshold. A trajectory recorded after takeover therefore shows an expert recovering from an edge-case disturbance, not nominal task execution. The distribution of states visited during these rescues, p^{takeover}, is conditioned on policy failure. Treating intervention logs as if they were drawn from the nominal distribution p^{π^*} distorts the training distribution toward states that exist only because the prior policy failed.

Appending a rescued episode directly to a behavioral cloning dataset can make unsafe policy commands look like expert demonstrations. Before takeover, the learned policy may issue commands that compound heading error or contact force; after takeover, the expert issues corrective commands from states the learner visited. Those drift states are valuable for corrective aggregation (expert labeling of the states the learner itself reaches, developed below) when an expert can label a safe action there. If a pipeline trains on the entire recorded trajectory without curation, it commits **learner-action contamination**. The failing policy's own commands enter the dataset as expert labels. The network then learns to answer the visual cues of an approaching hazard (such as the gripper sliding off the handle or the base drifting toward a rack upright) with the high-risk commands that produced them, which accelerates divergence rather than correcting it. Learner-action contamination corrupts the training targets directly by injecting errant commands, whereas **causal confusion** occurs when a policy trained on uncorrupted expert targets locks onto spurious correlations (Haan et al. 2019). The driving example from that work is a brake indicator on the dashboard. The light turns on because the expert braked, so it predicts the braking label almost perfectly, and a policy can learn to brake when the light is on rather than when an obstacle appears. At the latch, a measured joint current that climbs because the operator has already begun to push could play the same role, a consequence of the label that a policy mistakes for its cause.

Separating these labels requires locating the **divergence point** where the autonomous policy departed the nominal path, rather than treating takeover as the first error. The divergence precedes the takeover by at least the time a supervisor needs to respond. A remote operator who was not watching continuously must notice the hazard, reorient to the scene, and take control. The running machine's card in section 1.4.2 budgets that out-of-loop takeover at 4.47 s, from published take-over studies plus console video and handshake (section 14.2.2 derives it). Edge loggers therefore keep a circular ring buffer spanning a fixed lookback window $T_{\text{pre}} = 5$ s, that takeover time rounded up to whole seconds. Because dynamic memory allocation during safety-critical events introduces non-deterministic latency spikes, the buffer is pre-allocated in locked memory (`mlock`) or non-volatile storage and populated deterministically at the permission tick rate (1 kHz). When a human safety supervisor hits an override button or an MCU safety enforcer trips a barrier, the pipeline latches the intervention timestamp t_{takeover} and freezes the preceding ring buffer. An out-of-loop supervisory takeover takes longer than the teleoperation loop's continuous correction (section 5.6); at an illustrative 100 Hz, that delay spans hundreds of control cycles, all logged under the learner's authority.

↪ The authority record, which logs on every control cycle which entity held authority and the root cause of any preemption, is developed in section 14.8.

Detecting the divergence instant t_{div}, estimated below as $\hat{t}_{\text{div}}$, in offline ingestion pipelines requires searching backwards in time from the physical override timestamp across this synchronized pre-trigger telemetry buffer. For the latch, $e_{\text{div}}(t)$ is the lateral gripper-position error between measured and planned kinematics in the declared fixture frame, and δ_{tol} is its positive tolerance in meters. The divergence estimate is:

$$\hat{t}_{\text{div}} = \min\Big\{ t \in [t_{\text{takeover}} - T_{\text{pre}}, t_{\text{takeover}}] \;\Big|\; |e_{\text{div}}(t)| > \delta_{\text{tol}} \vee \operatorname{Tr}(\Sigma_\pi(t)) > \sigma^2_{\text{thresh}} \Big\}. \tag{5.7}$$

The pipeline scans the lookback window for the first threshold crossing. The policy ensemble covariance is computed after normalizing each action component by its declared scale, so its trace and variance threshold are dimensionless. In unstructured contact manipulation where no static reference trajectory exists, divergence detection instead evaluates declared physical residuals against admissible safety envelope boundaries, such as velocity-to-obstacle clearance margins or anomalous contact force derivatives dF/dt. The detection thresholds are scenario-specific and calibrated to the sensor noise floor. The minimum defines an estimate only when the retained window contains a threshold crossing from below; an empty set or a residual already above threshold at the start of the window leaves the onset unresolved.

The protocol of **intervention curation** separates learner commands, expert labels on learner-visited states, and post-takeover recovery at timestamps the permission path logs for every takeover, the entries of the authority record that chapter 14 formalizes. The takeover instant t_{takeover} is the permission tick at which the authority commit revokes the policy's ownership of the arm, and the three curation rules below are measured from it:

1. **Quarantine drift commands:** The actions issued by the failing autonomous policy during the drift window $[t_{\text{div}}, t_{\text{takeover}})$ are retained in the forensic audit log with their safety flags, but masked out ($\mathcal{M}_{\text{train}} = 0$) from imitation training targets.
2. **Preserve drift states for relabeling:** The physical states visited during $[t_{\text{div}}, t_{\text{takeover}})$ capture off-nominal boundary conditions that are absent from nominal demonstrations. These states are admitted to training only when an automated expert, such as a planner with privileged state, supplies corrective labels ($\mathbf{a}^*(s)$); a person replaying a logged state cannot reliably say what they would have commanded while closing the loop on it.
3. **Admit only stabilized recovery:** The post-takeover trajectory is not one demonstration. It opens with the enforcer's blend, in which the enforcer, still the only writer to the drives, ramps its single admitted command smoothly from the learner's last checked reference to the operator's. The actuators therefore never see a step, and no two sources ever command them at once. The operator's reorientation follows. Neither interval is expert behavior, and both stay masked. Only the segment after the operator's measured stabilization $t_{\text{stabilize}}$, the instant the operator's corrections settle into a steady return toward the corridor, up to recovery at t_{rec}, and only where the episode passes task-validity review, becomes an expert recovery demonstration. It teaches the policy how to steer from boundary states back to the nominal manifold. Section 14.8.3 defines the blend, reorientation, and stabilization intervals and the criterion that ends reorientation.

For the latch campaign, consider a stipulated follow-up collection, `latch-recovery-02`, at the positive 3 mm strike-plate offset and the guarded 0.03 m/s approach. The same 15 N tripwire remains active. An expert takes over from a misaligned approach, withdraws and reseats the gripper, then releases the latch. The stipulated archive contains valid, reviewed recovery sequences; this is a worked teaching branch, not a report of collected trials or measured improvement.

The three rules decide what this archive keeps. The unsuccessful learner commands are retained for diagnosis, expert relabeling is used only where a corrective target is available, and the expert's recovery is admitted only from its stabilized, reviewed segment. A new provenance digest binds those labels to the displaced fixture, the original plant contract, and the collection safeguards. The original archive remains immutable. Its successor now contains experience for the positive-offset guarded cell, while the negative-offset and faster-approach cells remain known absences. Chapter 6 uses this archive to construct the candidate `latch-bc-recovery-02`, and closed-loop evaluation must test recovery and nominal-task regressions before any expanded competence claim follows.

The states between divergence and takeover are the second law, endogenous data, made concrete. The learner's own control actions chose them, and the original nominal archive supplied no corrective supervision there; they are exactly the off-corridor states of figure 5.1, now visited and logged. Curated, they supply the missing return actions. Training on nominal expert demonstrations alone leaves the compounding-error problem of section 5.3 in place, bounded in the worst case by the quadratic growth of equation 5.1, $\mathcal{O}(S^2\epsilon)$. Labeling the states the learner itself reaches changes how that bound grows.

↪ The qualification of `latch-bc-recovery-02` against fresh nominal and displaced-plate trials is developed in section 7.8.1.

In **corrective aggregation**, such as Dataset Aggregation (DAgger) (Ross et al. 2011), an expert labels states visited by the learner, including eligible pre-takeover drift states. Training on states sampled from the learner's induced distribution ($s \sim p^{\pi}$), rather than solely the expert's distribution ($s \sim p^{\pi^*}$), directly counters the covariate shift that nominal demonstrations leave unaddressed. The resulting performance guarantee depends on a constant determined by plant dynamics and task geometry rather than the learning algorithm. After N rounds of aggregation, at least one policy $\hat{\pi}$ in the sequence satisfies

$$J(\hat{\pi}) \leq J(\pi^*) + u\,S\,\epsilon_N + \mathcal{O}(1), \tag{5.8}$$

where ϵ_N represents the surrogate loss of the best single policy on the aggregated learner-visited states (a different quantity from the per-step disagreement ϵ of equation 5.1). The symbol u is the **recoverability constant**, the maximum increase in the expert's remaining cost-to-go incurred by a single errant action before the expert resumes control.[10] This bound scales linearly with the task horizon S only while u remains bounded, which is to say only while the expert can undo any single mistake at a bounded price. If u scales in proportion to S, the product $uS\epsilon_N$ reverts to $\mathcal{O}(S^2\epsilon_N)$, wiping out the linear advantage over behavioral cloning; Ross et al. (2011) observed this exact failure mode in their predecessor forward-training scheme.

[10] **Corrective Aggregation Error Bound**: The bound is Theorem 3.2 of Ross et al. (2011). It holds for some policy in the sequence of N iterations, with N of order uS up to logarithmic factors, and u bounds the change in the expert's cost-to-go after any single action. Section A.5 states the finite-sample form, in which the same constant appears as $u_{\max}$.

For a horizon of $S = 500$ steps (5 s at 100 Hz) and $\epsilon = 0.001$, the behavioral cloning bound $S^2\epsilon = 250$. The aggregation term $u\,S\epsilon_N$ in equation 5.8 evaluates to 0.5 for a recoverable task with $u = 1$ and matching per-step error. In this setting, the quadratic cloning bound equals half the episode's maximum budget, while the leading aggregation term is less than the cost of a single control step. This comparison is conceptual, since the two error terms evaluate losses over different state distributions and both bounds omit problem-dependent leading constants. The quadratic bound becomes vacuous whenever ϵ exceeds $1/S = 0.002$, at which point $S^2\epsilon$ exceeds the maximum possible episode cost of S. Figure 5.7 joins the two halves of this section, the feedback loop that makes cloning's worst case quadratic and the curation timeline that turns one takeover into labels the linear bound can use.

The worst case, where u grows in proportion to the remaining horizon, is the first law, irreversibility (principle 1), written in the language of imitation bounds. Once the plant enters an unrecoverable state, such as a bent latch, a sheared cable harness, or a dropped brittle workpiece, no subsequent expert command can restore the lost objective. In these failure modes, the cost penalty spans the entire remaining task horizon ($u \sim S$), and aggregating additional learner states cannot recover linear error bounds. A policy cannot learn recovery from states where recovery is physically impossible; the deterministic permission path must prevent entry into those states altogether. The latch campaign draws this boundary explicitly. Withdrawing and reseating a misaligned gripper is recoverable, so `latch-recovery-02` records corrections there. Contact forces approaching the illustrative 100 N component damage threshold are unrecoverable; the illustrative 15 N tripwire is a detection threshold, not a contact-force ceiling. Its use requires a qualified collection-response envelope accounting for detection, command and actuator response, and peak-force overshoot below the damage threshold. Without that evidence, the lower threshold alone cannot authorize collection. Those damaging states belong to the safety-enforcer domain rather than the policy learning targets.

Gathering expert supervision on learner-visited states takes several architectural forms, and the curation rules detailed above instantiate one of them. Classical DAgger requires an expert to query and label every state visited during autonomous execution, an impossible requirement for a human supervisor when the policy issues commands at control rates. The framework of **human-gated aggregation** resolves this bottleneck by allowing a human supervisor to take over only when necessary, appending labels exclusively from the resulting intervention episodes (Kelly et al. 2019). The three curation rules refine this approach by keeping quarantined learner commands, relabeled drift states, and expert recovery actions under separate provenance. Where standard human-gated aggregation retains the entire human takeover stream, rule 3 also filters out the transitional controller blend and operator reorientation intervals. Two refinements address the scarcity of supervisor attention. One queries the expert only when policy uncertainty or novelty exceeds a threshold, spending a fixed intervention budget where the policy is least sure (Hoque et al. 2022). The other upweights the sparse recovery segments relative to the dense nominal trajectories during training, so a few minutes of rescue are not drowned by hours of nominal motion (Mandlekar et al. 2020).

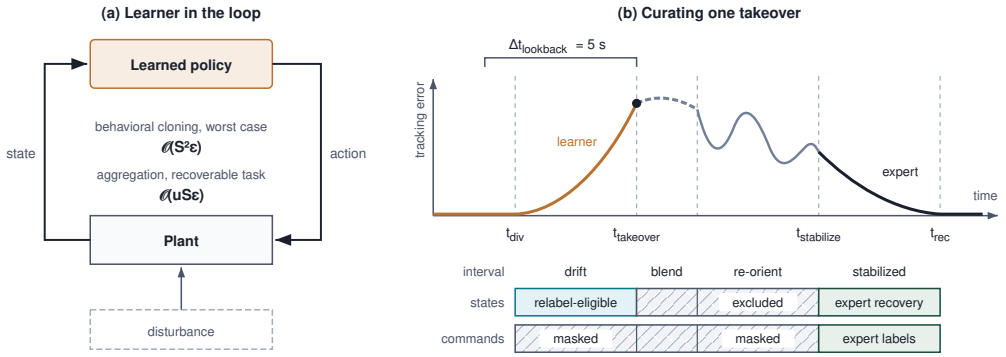


Figure 5.7: Covariate shift and intervention curation: (a) The learned policy's own actions choose the states it sees next. The worst-case cost bound grows as $\mathcal{O}(S^2\epsilon)$ for behavioral cloning and as $\mathcal{O}(uS\epsilon_N)$ for aggregation on a recoverable task; these are bound orders, not measured drift. (b) One takeover. The 5 s lookback reaches back past the divergence point; states between divergence and takeover are eligible for expert relabeling while the learner's commands there are masked. After takeover, the enforcer's blend and the operator's reorientation are retained for diagnosis but excluded from expert imitation targets, and only the reviewed segment from measured stabilization to recovery becomes an expert demonstration. Based on Ross et al. (2011) and Spencer et al. (2021).

A team can also avoid online interventions entirely, and this is the wide-tube remedy promised in section 5.3. Injecting small noise perturbations into the teleoperator's commands pushes the robot off the nominal path during ordinary collection, so the operator's corrections back toward the corridor appear in the initial demonstration set itself (Laskey et al. 2017). Each perturbation consumes hardware life from the fatigue budget in section 5.6.3 and must stay within recoverable bounds. Ingestion pipelines therefore log the collection mode of every episode explicitly (nominal, noise-injected, or gated intervention), because the three modes put different kinds of states and labels into the archive.

Learner-action contamination compounds across policy generations. Suppose a first policy drifts toward a joint torque limit until a supervisor intervenes with an override pendant. A successor trained on those uncurated drift-window commands as expert demonstrations reinforces the behavior that produced the departure, drives harder toward the limit in deployment, and forces operators to intervene earlier and more often. A pipeline that monitors only offline validation loss misses the cascade, because loss on the mislabeled validation split keeps improving as the network memorizes the corrupt labels while physical stability degrades.

Preventing this cascade requires the permission path to record every takeover or safety-barrier trip as an explicit entry in the authority record.[11] A boolean disengagement tag overlaid on video is not enough. The ingestion pipeline requires a populated pre-trigger ring buffer matching the declared lookback window (5 s), hardware-synchronized sensor observations and actuator taps, monotonic override timestamps, unique authority identifiers, and precise timestamps marking the blend completion and supervisor stabilization. Without this synchronized pre-trigger telemetry, ingestion pipelines cannot locate the divergence point t_{div} and therefore cannot isolate errant learner commands from the boundary states eligible for expert relabeling.

> **Checkpoint 5.3: Intervention curation and error bounds**
>
> Before curating human intervention logs for policy retraining, audit compounding error dynamics and timestamp attribution against these criteria:
>
> - ☐ **Takeover vs. divergence timestamps**: Why retain learner-visited drift states for possible expert relabeling while excluding their unsafe learner commands from expert imitation targets?

11 **Event Data Recorder**: US vehicle rules under 49 CFR Part 563 require an automotive event data recorder to preserve a fixed window of telemetry recorded prior to a crash trigger, recognizing that the root cause of an incident precedes the trigger event itself (National Highway Traffic Safety Administration 2024a). Human supervisory takeovers have the same temporal structure, because the initial divergence t_{div} always precedes the override timestamp t_{takeover}. A logging pipeline that begins capturing telemetry only upon trigger activation has already discarded the critical drift data required for policy curation.

- ☐ **Retraining cascade progression**: Describe how training successive policy generations on uncurated intervention logs accelerates policy degradation despite improving training metrics.
- ☐ **Conditional error bounds**: Which assumptions distinguish the behavioral-cloning quadratic worst-case cost bound from DAgger's linear-horizon result?
- ☐ **Bound threshold**: For an episode twice as long as the 5 s comparison, at the same 100 Hz control rate, above what per-step error ϵ does the quadratic bound stop constraining the cost?

Even with expert labels on learner-visited states, the resulting demonstration archive remains tethered to the kinematics and dynamics of the collecting machine. Pooling demonstrations across robot arms, bases, and end-effectors raises the cross-embodiment questions in section 5.9.

5.9 Cross-Embodiment Transfer

Borrowing manipulation demonstrations from another robot arm looks effortless in tensor space, because both datasets store image frames and seven-element action vectors. That ease disappears when a trajectory recorded on a wide-stroke, 50 Hz gripper drives a narrower, 20 Hz gripper into its mechanical endstops or misses a moving conveyor part by centimeters. Physical demonstration trajectories never represent a task in the abstract. Every recorded stream of sensor observations and actuator commands embeds the link lengths, joint velocity limits, camera optical paths, bus latencies, and contact-surface compliances of the logging hardware directly into its numbers. Reusing demonstration datasets across robots therefore requires separating environmental task invariants from machine-specific realizations, checking kinematic and dynamic compatibility, and resolving topological mismatches. This applies whenever the warehouse manipulator supplements its local latch demonstrations with data from foreign arms and grippers.

5.9.1 Embodiment contracts and the normalization fallacy

Suppose a training pipeline rescales gripper commands from two different robots to the unit interval $[0, 1]$ so a single network head can predict both. A normalized command that opens a wide-stroke donor gripper far enough to clear a workpiece will open a narrower target gripper to only a fraction of that span, driving its fingers straight into the part. That failure occurs because an observation-action trajectory $(\mathbf{o}_t, \mathbf{a}_t)$ permanently encodes the **embodiment contract** between a specific machine and its physical surroundings. This contract covers the sensor geometry and optical calibration and the sampling period and transport latency of the perception bus. It also covers the coordinate frames and kinematic limits of the actuators, the torque-speed saturation curves of the motors (including reflected rotor inertia $N_{\text{gear}}^2 J_{\text{rotor}}$ in geared transmissions), and the friction and compliance of the contact interface. For the running machine's recorded teleoperation, the contract specifies a rolling-shutter navigation camera at 4056 by 3040 px and 60 Hz, a global-shutter wrist camera at 1440 by 1080 px on its dedicated bracket. The contract also specifies a 9-axis kinematic log sampled at the 1 kHz permission tick (notebook 5.2). Every latch demonstration in that archive reflects that camera suite, that sensor bracket, and that exact arm. Altering any parameter in this contract alters the mapping between commanded inputs and physical state transitions. Even when two robots follow an identical Cartesian tool trajectory $\mathbf{x}(t)$, their required actuator torques diverge. Written per joint, Newton's second law for a serial manipulator yields the manipulator equation, met earlier as the model classical control inverts:

$$\tau = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \tau_{\text{ext}},$$

↩ The manipulator equation, which classical control inverts to compute joint torques, is established in section 3.2.

where $\mathbf{q}$ denotes joint positions, $\mathbf{M}(\mathbf{q})$ is the configuration-dependent mass matrix, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ collects Coriolis and centrifugal forces, $\mathbf{g}(\mathbf{q})$ is the gravitational torque vector, and τ_{ext} represents torques exerted by external contact forces. Section 11.4.1 builds this equation up from the single-joint case. Because every term depends directly on link masses, rotor inertias, and gear ratios, the same tool trajectory demands different torque profiles across different robot embodiments.

↩ Reflected rotor inertia, $N_{\text{gear}}^2 J_{\text{rotor}}$, and its effect on joint dynamics are established in section 2.4.3.

While the embodiment contract defines the physics of a single machine, the **cross-embodiment compatibility contract** determines whether trajectories logged under one embodiment contract can

safely serve as action targets on another. This compatibility contract defines the forward kinematics mapping (translating joint angles to tool pose), control-rate transformations (Δt), and actuator effort bounds. These bounds prevent a high-force profile or rapid spatial velocity demonstrated on a rigid industrial arm from driving a lightweight cobot beyond its structural limits. Large pooled corpora rarely provide such guarantees. Open X-Embodiment maps action fields into a coarsely aligned seven-dimensional end-effector format and normalizes each dataset independently. However, it deliberately leaves coordinate frames unaligned and allows action vectors to represent either absolute or relative poses and velocities across different sub-corpora (Open X-Embodiment Collaboration et al. 2024). A multi-task model trained on this pool learns to interpret identical action vectors differently based on embodiment tokens. In contrast, DROID sidesteps embodiment mismatch altogether by collecting all data on a single standardized robot model (Khazatsky et al. 2024). Demonstrations from diverse robots can enrich representation learning without an explicit compatibility contract, but such a contract becomes strictly mandatory the moment borrowed actions are imitated directly as supervisory labels or dispatched to a physical plant.

How much work the compatibility contract has to do is visible in the largest pooled robot corpus. The Open X-Embodiment collection merges independently recorded datasets into one training mixture and publishes, for each, its episode count, robot morphology, scene type, and command rate (Open X-Embodiment Collaboration et al. 2024). Figure 5.8 weights each dataset by its episodes.

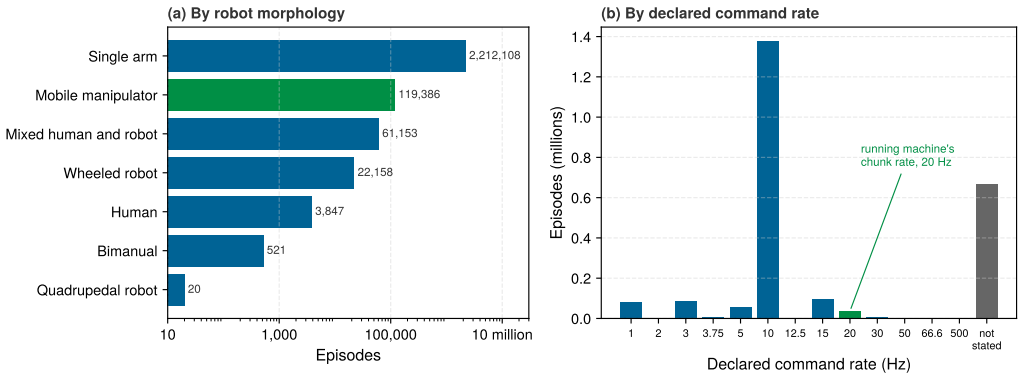


Figure 5.8: What a pooled corpus actually contains: Episodes of the Open X-Embodiment collection, weighted by dataset size, (a) by robot morphology on a logarithmic axis and (b) by the command rate each dataset declares. Single-arm table-top manipulation dominates, and the declared rates cluster at or below 10 Hz; a large share declares no rate at all because its actions are primitive skills or quasi-static moves. Green marks the running machine's class in (a) and its chunk renewal rate in (b). Data from the collection's published dataset overview (Open X-Embodiment Collaboration et al. 2024).

Of the 2.4 million episodes across 72 datasets, 91 percent come from single arms and 89 percent from table-top scenes. Mobile manipulators, the class of the running machine, contribute 5 percent, and two datasets supply 95 percent of those. The command rates are as uneven: 57 percent of episodes were logged at 10 Hz, 28 percent declare no rate, and only 2 percent were recorded at or above the machine's 20 Hz chunk rate. Corpus size therefore says little about coverage of one machine's contract. Almost every episode in the mixture needs the rate transformation and morphology mapping of the compatibility contract before it can serve as an action target here.

A data-reuse pipeline must separate environmental task invariants from embodiment-specific realizations. A task invariant is a geometric or dynamical condition imposed by the physical process, for example bringing a compliant probe into contact with a terminal pad within 5° of the surface normal while sustaining a normal contact force between 2.0 N and 5.0 N. An embodiment realization is the concrete sequence of joint encoder counts, pulse-width modulation duty cycles, perception transport latencies, and sensor pixel rasters used to achieve that condition on a particular test rig. While the contact goal exists in world coordinates, the dataset stores only the realization. When a model trains directly on raw hardware realizations, it internalizes the idiosyncratic mechanics of the source testbed, including structural flexure, drive backlash, and sensor mounting oscillations.

Consider transferring a demonstration dataset collected on Machine A to Machine B for an identical pick-and-place task:

- **Machine A** operates a parallel-jaw gripper with a stroke of 0 to 100 mm, a 50 Hz command rate (20 ms period), a continuous grasp force limit of 40 N, and a wrist camera mounted 45 mm behind the tool center point with a 15 ms transport delay.
- **Machine B** operates a compact gripper with a stroke of 0 to 60 mm, a 20 Hz command rate (50 ms period), a continuous force limit of 15 N, and a wrist camera mounted 20 mm behind the tool center point with a 35 ms transport delay.

Projecting Machine A's demonstrations directly through the physical constraints of Machine B reveals immediate points of hardware failure. For a workpiece of width 75 mm, Machine A commands a gripper aperture of 80 mm. That command exceeds Machine B's mechanical stroke by 20 mm, driving its leadscrew into the mechanical endstop. During the hold phase, Machine A applies 30 N of clamping force; executing that command on Machine B doubles its continuous rating of 15 N and forces its motor amplifier into current saturation. In the time domain, subsampling the 50 Hz trajectory onto Machine B's 20 Hz bus aliases rapid contact dynamics and increases the half-period average hold age $\Delta t/2$ from 10 ms to 25 ms, introducing 15 ms of additional zero-order hold phase lag (section B.5.5). Compounded by the 20 ms difference in camera transport delay (35 ms $-$ 15 ms), the added low-frequency phase-equivalent delay is 35 ms, of which the camera difference contributes 20 ms. At the assumed constant approach velocity of 0.10 m/s, that phase approximation corresponds to an equivalent displacement of 3.5 mm. Actual contact overrun requires measured or bounded sampling phase, capture and transport, computation, actuation, and plant response; the phase-equivalent value does not establish when the first correction dispatches. Meanwhile, the 25 mm disparity in wrist camera mounting introduces optical parallax that shifts the apparent position of the workpiece boundary across the camera raster.

Standard machine learning pipelines often try to reconcile these discrepancies by normalizing observation and action channels to $[0,1]$ or $[-1,1]$. That rescaling aligns tensor ranges while leaving the physical mismatch intact. Normalizing gripper travel to $[0,1]$ turns Machine A's safe 80 mm opening ($\tilde{a} = 0.80$) into a 48 mm opening on Machine B (0.80×60 mm) that drives its fingers into the 75 mm workpiece during approach. Normalizing feature ranges rescales numbers, but it cannot lengthen a leadscrew, lower motor coil resistance, or recover bus transport delay. Demonstration trajectories plotted in normalized unit spaces may look identical, yet only the subset inside the physical intersection (apertures below 60 mm, forces under 15 N, and command frequencies below 20 Hz) can run on the target hardware without saturation or contact damage. Even within that intersection, two arms with different joint layouts can collide with the workspace when they share a single Cartesian command (section 5.9.2).

5.9.2 Kinematic topology mismatches and null-space projection

Even when end-effectors, force ratings, and control rates align, differences in arm geometry can break a transferred trajectory. When a 7-DoF arm's demonstration is compressed into a 6-DoF Cartesian hand displacement so it can be pooled with other robots, the log discards the arm's elbow posture; replaying that 6-DoF stream through a target robot's inverse kinematics solver can swing the elbow into a shelf upright or drive a lighter arm through a singularity, even though the hand follows the exact demonstrated path. Combining robots with different kinematic structures introduces two topological conflicts:

1. **Action space incompatibility**: A 7-DoF arm records joint positions $\mathbf{q} \in \mathbb{R}^7$, a bimanual workstation records $\mathbf{q} \in \mathbb{R}^{14}$, and a mobile base logs planar velocities $(\dot{x}, \dot{y}, \dot{\theta}_{\text{head}})$. Because joint-space commands cannot transfer directly across distinct kinematic arrangements, multi-embodiment models project action sequences into a common Cartesian end-effector displacement:

$$\Delta \mathbf{x}_t = (\Delta x, \Delta y, \Delta z, \Delta \phi_{\text{roll}}, \Delta \phi_{\text{pitch}}, \Delta \phi_{\text{yaw}}) \in \mathbb{R}^6, \quad a_{\text{gripper}} \in [0,1],$$

   where the final three elements represent local roll, pitch, and yaw increments under a declared rotation convention. This displacement vector is not itself a member of the Special Euclidean group $SE(3)$; any numerical integration must handle angle wrapping and representation singularities. Projecting into Cartesian space standardizes spatial geometry, but it strips away actuator dynamics. A Cartesian step that an industrial arm executes effortlessly near its nominal pose can force a lighter arm through a kinematic singularity, a configuration in which

aligned joints leave the tool unable to move in some direction. The manipulator Jacobian $\mathbf{J}(\mathbf{q})$, a configuration-dependent matrix that section 11.2.1 develops on the machine's own arm, maps joint velocities to tool spatial velocities through $\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$. When $\mathbf{J}(\mathbf{q})$ loses row rank at a singularity, no finite joint velocity can generate motion along the lost Cartesian direction. Commanded displacements near such poses demand joint velocities that immediately exceed the target machine's actuator velocity bounds.

2. **Null-space projection and elbow swivel ambiguity**: Away from singularities, mapping a 7-DoF arm into a 6-DoF Cartesian delta projects out the one-dimensional null space ($7-6=1$ redundant degree of freedom). For a redundant manipulator with Jacobian $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6\times 7}$, the general kinematic relationship between Cartesian velocity $\dot{\mathbf{x}}$ and joint velocity $\dot{\mathbf{q}}$ is:

$$\dot{\mathbf{q}} = \mathbf{J}^{\dagger}(\mathbf{q})\dot{\mathbf{x}} + \left(\mathbf{I} - \mathbf{J}^{\dagger}(\mathbf{q})\mathbf{J}(\mathbf{q})\right)\dot{\mathbf{q}}_0,$$

where $\mathbf{J}^{\dagger} = \mathbf{J}^{\top}(\mathbf{J}\mathbf{J}^{\top})^{-1}$ is the Moore-Penrose pseudoinverse (the minimum-norm joint velocity producing $\dot{\mathbf{x}}$ whenever $\mathbf{J}$ has full row rank), and the projection operator $(\mathbf{I} - \mathbf{J}^{\dagger}\mathbf{J})$ filters an arbitrary joint-velocity bias $\dot{\mathbf{q}}_0 \in \mathbb{R}^7$ into self-motion that leaves the end-effector still. For an anthropomorphic arm with shoulder and wrist geometry that admits this construction, a fixed end-effector pose in $SE(3)$ and a fixed shoulder anchor leave an elbow orbit around the shoulder-to-wrist axis, parameterized by the **elbow swivel angle** ψ. A general redundant arm instead requires its own configuration parameter or the full joint vector; one redundant degree of freedom does not imply this circular geometry. Discarding this internal degree of freedom to impose a uniform 6-DoF action representation erases the human operator's postural intent. When the target robot receives this 6-DoF command stream, its local inverse kinematics (IK) solver must resolve the redundancy using an arbitrary mathematical heuristic, such as joint-limit centering or manipulability maximization. Without the original swivel angle, the solver may adopt an elbow posture that drives joints into hard stops or sweeps the elbow into nearby shelving, fixtures, or personnel during a motion that was entirely collision-free in the source environment. Preserving the applicable redundancy parameter, or the complete joint-state vector, within the trajectory's provenance metadata retains the operator's postural intent for target-specific retargeting and collision checks. The running mobile manipulator archive maintains this intent directly: its arm has redundant kinematics, and its logger captures all 9 axes of the arm and base at the 1 kHz permission tick. Any pooled 6-DoF delta generated from a latch-servicing trajectory can always be traced back to the demonstrator's original elbow posture.

Reusing physical data across different robots therefore requires two validation gates at the ingestion boundary, corresponding to two distinct roles in training:

- The **pretraining admission gate** evaluates whether a trajectory can safely contribute to general representation learning. This gate validates data provenance, observation integrity, coordinate frame conventions, action semantics (reference frame, absolute vs. relative representations, position vs. velocity modes), and sampling rates. In particular, source actions admitted at this gate need not be physically executable on the target robot. Multi-embodiment foundation models rely on this gate to ingest broad visual and task structures across dozens of robot morphologies before fine-tuning on target hardware (Open X-Embodiment Collaboration et al. 2024; Black et al. 2024).
- The **target-label gate** applies whenever recorded actions serve as direct imitation labels or execution setpoints on the physical plant. This gate maps the source trajectory through target forward kinematics to verify coordinate consistency (Lynch and Park 2017). It confirms that commanded joint velocities and accelerations remain within the target plant's dynamic envelope (Tedrake 2023). It also verifies that task-critical features remain within the field of view and dynamic range of the target sensors. When commanded trajectories violate kinematic reach or motor torque limits, an ingestion pipeline cannot simply clamp the values to target boundaries. Clamping distorts velocity profiles, alters phase relationships, and destroys the causal link between observed spatial errors and corrective control outputs. Trajectories that satisfy the target dynamics pass untouched; trajectories that admit valid geometric retargeting through inverse kinematics are relabeled; and trajectories that demand unobtainable actuator authority or rely on unobservable states are rejected.

↪ Forward kinematics, the manipulator Jacobian, its singularities, and the pseudoinverse are developed in section B.6.1.

Evaluating the two-gripper scenario of section 5.9.1 through this target-label gate resolves each trajectory segment systematically. The initial opening phase must be rejected rather than relabeled, because satisfying the task invariant (clearing the 75 mm workpiece) demands an aperture that Machine B cannot physically achieve. The 30 N grasp hold is rejected on identical grounds, as it exceeds Machine B's continuous motor rating. Conversely, the 50 Hz trajectory can be resampled and relabeled onto Machine B's 20 Hz control bus only if the slower command stream preserves the task-critical contact dynamics and remains within the target's verified tracking and force limits. The data record must also log the added zero-order hold lag and camera transport delay so that policy training accounts for the altered timing. An ingestion pipeline lacking this gate injects dynamically invalid trajectories into the training distribution. A policy trained on such unvetted data fails in deployment not because of neural network underfitting, but because its training labels were physically valid only on the donor hardware. For the latch-servicing campaign, the target plant is the mobile manipulator's own arm and gripper. Every borrowed trajectory proposed as a latch training label must pass this gate against the mobile manipulator's verified limit record, regardless of the donor platform's reported capabilities.

↩ The limit record, which states each measured limit with its operating conditions, uncertainty, and margin, is established in section 2.7.3.

The warehouse conveyor pick demonstrates a transfer mismatch rooted in task dynamics rather than robot kinematics. At the sorting station, the arm grasps a workpiece moving along a belt at 0.20 m/s. The workpiece translates 5 mm during each 25 ms control step and advances 80 mm over a single 16-step action chunk of 400 ms, a displacement of 5.3× the allowable 15 mm grasp tolerance. A borrowed tabletop demonstration recorded on a stationary bench defines its grasp trajectory entirely in the static table frame. If a policy imitates that trajectory unchanged at the conveyor, an open-loop action chunk aims at the spatial location the part occupied when the chunk was generated. The target-label gate can adapt such a static trajectory only by transforming the end-effector path into the moving workpiece coordinate frame. This coordinate transformation requires both the instantaneous object pose in the donor metadata and the conveyor velocity in the target environment record. An external trajectory lacking this telemetry can enter the pipeline solely through the pretraining admission gate. Teleoperation data collected directly on the warehouse machine requires no such mathematical rescue, as the human operator generated the demonstration commands with the conveyor in motion.

5.9.3 Choosing pooled experience for the target task

Pretraining on a million pooled trajectories from foreign robots does not help the warehouse manipulator open a displaced latch if the training mixture drowns out the machine's own contact dynamics. The same holds if the mixture adds ten thousand repetitions of a single tabletop scene instead of covering new operating cells. Large-scale pooled corpora such as DROID (Khazatsky et al. 2024), BridgeData v2 (Walke et al. 2023), and Open X-Embodiment (Open X-Embodiment Collaboration et al. 2024) offer broad scene diversity and multi-robot demonstrations, but their value to a specific deployment depends on what information survives the admission gates of section 5.9.2. Even when kinematic mismatches prevent direct imitation of donor joint commands, foreign demonstrations can supply visual representations, object affordances, and high-level task structures, provided the campaign measures how those additions fill its scenario table.

Empirical scaling studies since 2024 support the premise behind the scenario table, that policies generalize with the number of distinct environmental conditions in their data rather than with repetitions of the same ones. In an extensive empirical evaluation spanning more than 40,000 demonstrations collected with handheld grippers, policy success across novel environments and unseen objects scaled as a power law with the count of distinct training environments and objects. Adding further demonstrations within existing environments yielded sharply diminishing returns once a modest baseline was achieved. A dataset collecting 50 demonstrations across each of 32 environments achieved 90 percent on novel environment-object pairs (Lin et al. 2025). Similarly, evaluating mobile manipulation across residential settings revealed steady performance gains as the training distribution expanded from 3 to 104 homes. At 104 locations, the generalized model matched the performance of an in-domain baseline trained exclusively within the evaluation homes (Black, Brown, et al. 2025). Viewed through the scenario table, the primary scaling axis is the count of distinct operational cells populated by the corpus. The declared minimum coverage threshold $N_{\min}$ identifies cells that still need collection; it does not establish that additional trajectories within

a populated cell have no value. Populating operational cells is necessary but not sufficient on its own. As Belkhale et al. (2023) demonstrate, dataset utility depends on two additional quality metrics: action divergence (the discrepancy between expert actions and learned policy outputs at reachable states) and transition diversity (the stochasticity of outcomes from given state-action pairs). Uncurated state diversity does not automatically yield performance improvements.

These demonstration corpora also diverge in how they acquire operational diversity. Open X-Embodiment aggregates 60 distinct datasets across 22 different robotic embodiments with different, unaligned action spaces; despite this lack of structural alignment, joint pretraining improved success rates across the smaller constituent datasets (Open X-Embodiment Collaboration et al. 2024). DROID and AgiBot World instead hold the embodiment contract constant while maximizing environmental and task diversity. DROID deploys a single manipulator model across 564 in-the-wild environments (Khazatsky et al. 2024). AgiBot World scales this principle with a homogeneous fleet of more than 100 identical robots, producing pretraining data that demonstrated superior downstream transfer compared to Open X-Embodiment (AgiBot-World-Contributors et al. 2025). A homogeneous fleet removes nominal kinematic mismatches, but each unit must still satisfy the target-label gate through verified calibration and dynamic compatibility. The fleet also remains vulnerable to the shared kinematic blind spots of that specific platform. Conversely, a multi-robot corpus can broaden perceptual learning through the pretraining gate; its actions become target labels only after passing the target-label gate. Balancing these disparate data streams requires deliberate weighting within the training mixture; dynamically weighting datasets to safeguard the worst-performing target domain significantly outperforms uniform data sampling across Open X-Embodiment (Hejna et al. 2025).

Placing these landmark robot archives alongside community uploads and web-scale foundation-model corpora exposes both how physical collection has scaled and where its ceiling sits (figure 5.9). Across 11 major physical collection campaigns since 2015 (figure 5.9, panel a), robot datasets progressed from single-lab grasping farms (50,000 to 800,000 trials in a fixed bin (Levine et al. 2018; Kalashnikov et al. 2018)) through multi-scene teleoperation (60,096 trajectories in BridgeData V2 and 76,000 in DROID (Walke et al. 2023; Khazatsky et al. 2024)) to million-episode pooled and fleet archives (2,419,193 episodes in Open X-Embodiment and 1,001,552 in AgiBot World (Open X-Embodiment Collaboration et al. 2024; AgiBot-World-Contributors et al. 2025)). Those pooled campaigns sit five orders of magnitude above the 8–15 episode median of single-session community uploads in figure 5.5. Even million-episode robot campaigns, however, confront the cross-domain gap in panel b. Across 902 language, vision, and robotics models with disclosed training dataset sizes in the Epoch AI catalog (Epoch AI 2026), frontier language and vision corpora reach 10^{13} to 10^{14} tokens or images, whereas π_0's $10{,}000$-hour robot pretraining corpus (1,000,000,000 state-action steps (Black et al. 2024)) sits 14,800× below DeepSeek-V3's 14.8 trillion tokens.

Because every physical trajectory is throttled by real-time motor execution and session yield (equation 5.6), robot learning cannot close a four-order-of-magnitude data gap by brute-force teleoperation alone. Deployed pipelines instead inherit broad semantic priors from web-pretrained vision-language backbones, use synthetic expansion or co-training where provenance allows, and spend their scarce physical episodes on the cells that scenario-table coverage and compatibility gates confirm they need, as the benchmark systems in table 5.5 illustrate.

Table 5.5: State of practice in robot demonstration data, as of 2026: Named systems illustrate the mechanisms of this chapter at one point in time; each scale and result is as reported by the cited paper, measured on that paper's own tasks and not comparable across rows.

System (year)	Collection regime	Reported scale	What it shows for the scenario table
Open X-Embodiment (2024) (Open X-Embodiment Collaboration et al. 2024)	Heterogeneous pool of existing datasets	60 datasets, 22 embodiments; RT-1-X 50% higher mean success than the original methods	Breadth without aligned actions; target labels require the target-label gate
DROID (2024) (Khazatsky et al. 2024)	One robot model, distributed in-the-wild collection	76,000 trajectories, 350 h, 564 scenes	Scene diversity under a single embodiment contract

System (year)	Collection regime	Reported scale	What it shows for the scenario table
UMI (2024) (Chi et al. 2024)	Robot-free hand-held grippers	Relative-trajectory actions; latency matched at deployment	Collection decoupled from robot time and wear
Data scaling study (2025) (Lin et al. 2025)	Hand-held grippers across many environments	40,000 demonstrations; power law in environments and objects	Fill more cells before adding episodes per cell
AgiBot World (2025) (AgiBot-World-Contributors et al. 2025)	Homogeneous fleet of identical robots	1,001,552 trajectories, 217 tasks; 30% average gain over pretraining on Open X-Embodiment	Shared embodiment; unit compatibility still requires verification
$\pi_{0.5}$ (2025) (Black, Brown, et al. 2025)	Mobile manipulators in homes, co-trained with other sources	About 400 h in homes; 3 to 104 training locations	Generalization grows with the number of locations
MimicGen (2023) (Mandlekar et al. 2023)	Synthetic generation from seed demonstrations	Over 50,000 from about 200 human demonstrations, 18 tasks	Cells filled only by synthetic data must be marked as such
Sim-and-real co-training (2025) (Maddukuri et al. 2025)	Simulated and real demonstrations mixed in training	38% average gain in real-robot success	Synthetic data helps when its provenance is kept

The core engineering decision is whether incorporating compatible pooled experience improves target-task policy performance relative to an identical baseline trained strictly on in-domain data. Rigorous evaluation requires fixing the target task distribution and test protocol, maintaining full data provenance, and benchmarking both nominal task completion and recovery behavior. Merely increasing the number of embodiments in a dataset does not guarantee positive transfer. In the mobile manipulator's displaced-latch campaign, the control baseline is a policy trained exclusively on 5,000 retained teleoperation demonstrations from the machine itself, populating a single cell within the six-cell scenario table. An expanded multi-robot corpus justifies its training overhead and ingestion complexity only if the resulting policy improves robustness on long-tail target conditions without regressing nominal latch-servicing performance.

While cross-embodiment compatibility contracts guard the ingestion boundary against hardware mismatches, demonstration data collected on the native machine can fail through internal mechanisms. Teleoperator drift, sensor calibration drift, fixture flexure, and misclassified execution failures corrupt physical datasets without altering tensor dimensions or triggering schema errors (section 5.10).

5.10 Dataset Degradation and Physical Distribution Shift

A demonstration archive can pass every schema check and fill its scenario table on paper while silently rotting from within. A technician may bump a camera mount by 3 mm between shifts, a hot motor winding may lose peak torque during an afternoon session, or a curation script may mistake dropped force-sensor packets for a clean trajectory. Because physical datasets capture closed-loop interactions between controllers and hardware rather than static observations, verifying an archive requires auditing the mechanical, optical, and temporal meaning of recorded trajectories alongside array shapes and scenario-table counts.

5.10.1 Silent plant drift and kinematic reprojection verification

Suppose afternoon latch demonstrations begin missing the strike plate by two millimeters even though the same operator is running the same cell. Before an engineer can decide whether to relabel the afternoon runs, recalibrate the wrist camera, or service the arm, the pipeline must separate three sources of drift that occur inside a single supported scenario cell. These are changes in the physical plant, changes in the human operator's control habits, and silent shifts in sensor calibration.

5.10.1.1 Passive covariate shift vs. physical dynamics shift

In those afternoon runs, the first question is whether the world around the arm changed or the arm itself changed, which is the distinction between passive observation shift and active plant

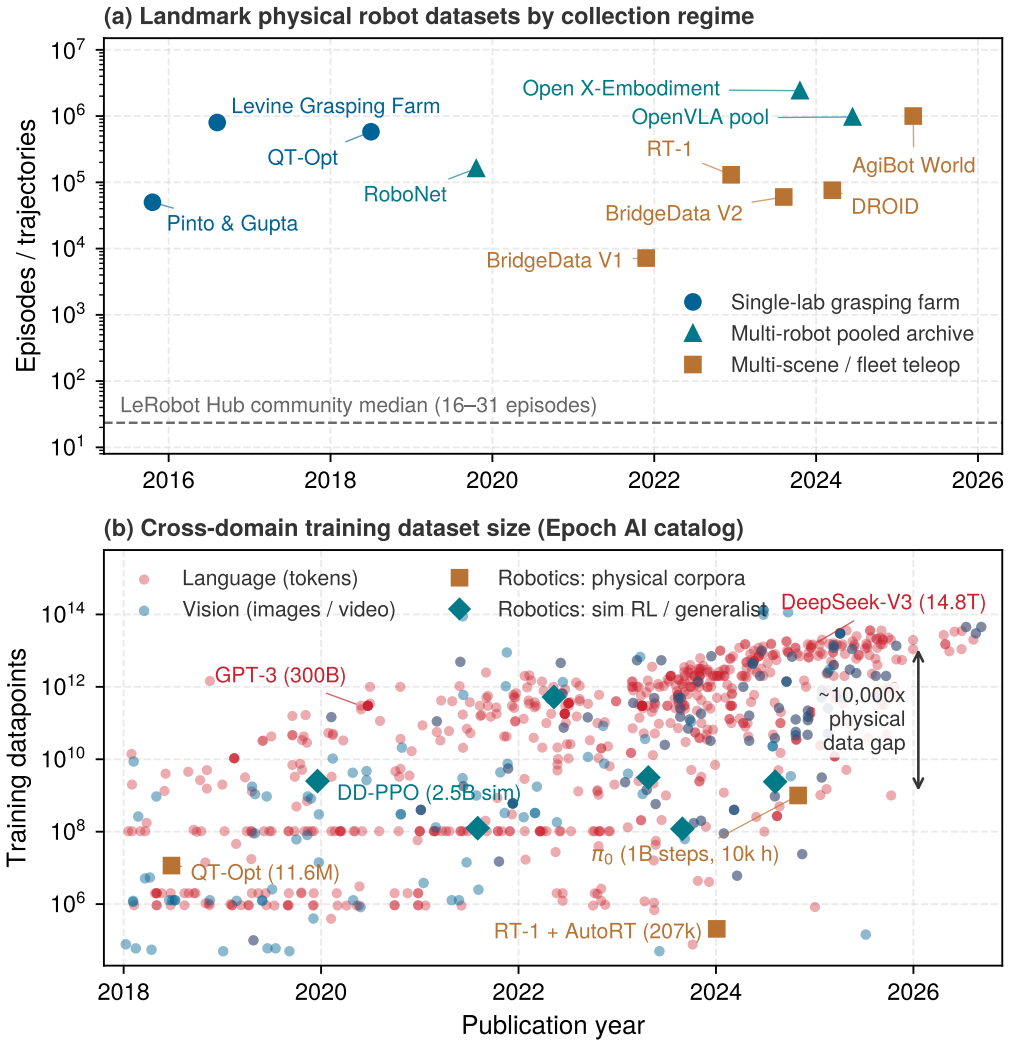


Figure 5.9: Physical robot demonstration scaling and the cross-domain data gap: (a) Reported trajectory or episode counts of 11 landmark physical robot corpora from 2015 to 2025 on a logarithmic axis, grouped by collection regime alongside the median episode bin of community LeRobot Hub uploads (figure 5.5). (b) Training dataset sizes in datapoints across language, vision, and robotics models in the Epoch AI catalog from 2018 to 2026 (Epoch AI 2026). While multi-robot pooling and homogeneous fleets pushed physical archives past 10^6 episodes, real-world robot corpora remain four orders of magnitude smaller than web-scraped language and vision datasets because physical collection is bounded by real-time actuation.

drift introduced in section 5.2. Shifting afternoon sunlight across the cage door alters camera pixel intensities without changing how the joints respond to a torque command, whereas hot motor windings, worn gearbox teeth, abraded gripper pads, and varying door friction alter the plant dynamics directly. Before an ingestion pipeline pools or partitions demonstration episodes, it must determine which underlying physical conditions have shifted. Motor heating shows why the answer depends on where in its envelope the machine operated. A hot winding has higher resistance, which leaves the drive less voltage headroom. Inside the current-regulated region of the torque-speed envelope the drive still delivers the commanded torque, so the modeled voltage-headroom effect alone does not distinguish slow, hot and slow, cold demonstrations; other thermal and plant changes still require checking. Near the voltage limit at high joint speed, the same resistance rise caps the current and lowers the available torque, as section 5.2 described, so a fast demonstration recorded hot reflects a weaker plant than one recorded cold.

↩ The torque-speed envelope and the back-EMF roll-off at high speed are established in section 2.5.3.

That dependence is why pipelines stratify episodes before pooling them. Stratifying by measured actuator temperatures, maintenance logs, and workpiece wear enables engineers to verify whether identical actuation commands yield consistent plant dynamics. The standard diagnostic tool for this verification is a repeatable excitation trajectory. At the start of each operating shift, the manipulator replays a predetermined joint-space trajectory while stationary. If the tracking error residual exceeds an established threshold, the pipeline flags the plant as mechanically altered. Logging timestamps alone cannot establish that a machine degraded during an operational shift; dataset curation pipelines must justify pooling across sessions based on verified physical invariants and intended training objectives rather than superficial uniformity of array dimensions.

Demonstration collection policies drift just as readily as mechanical hardware. Across 141.4 h of scheduled cell time, teleoperators naturally adapt their control strategies. An operator who initially moved cautiously, executing small corrective sub-movements and pausing deliberately before striking a latch plate, gradually develops muscle memory. Over time, that operator rounds corners, anticipates contact forces, and exploits system momentum. These mature demonstrations rely on feedforward compensation and tight timing rather than reflecting changes in joint friction. Partitioning datasets into temporal windows and monitoring episode durations, corrective jerk counts, command amplitudes, and teleoperator intervention rates exposes these subtle shifts in human behavior. This behavioral tracking reveals whether later trajectories contribute advanced manipulation skills or introduce an entirely distinct behavioral regime that requires segregated dataset splits or specialized behavioral labels.

Sensor calibration also deteriorates silently across operating sessions. Standard storage containers such as HDF5, Zarr, and Apache Parquet preserve array topologies, data types, and chunk structures, but they cannot verify whether stored coordinates correspond to physical reality. If a robot is serviced, bumped by personnel, or re-zeroed between collection sessions, joint encoder offsets and external camera poses shift. A camera mount displaced by merely 3 mm moves workpiece pixels across image frames while the stored arrays remain syntactically valid. Clock synchronization drift across distributed fieldbus controllers introduces comparable corruption, skewing the temporal alignment between visual frames and tactile force measurements by tens of milliseconds.

↩ Distributed-clock synchronization across fieldbus drive nodes is established in section 4.5.2.

To detect this silent drift, automated ingestion pipelines rely on **kinematic reprojection verification**, formulated below and evaluated across a photographic mosaic in figure 5.10. Let $\mathbf{q} \in \mathbb{R}^n$ denote the joint angles latched from optical encoders at the optical exposure midpoint. Forward kinematics $\mathbf{FK}(\mathbf{q})$ maps these angles to the 3D surface vertices of the robot's links in the base coordinate frame, $\mathbf{p}_{\text{base}} \in \mathbb{R}^3$. The camera's extrinsic spatial transformation $\mathbf{T}_{\text{cam}\leftarrow\text{base}} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3)$ maps base coordinates into the camera's optical frame ($\mathbf{p}_c = \mathbf{R}\mathbf{p}_{\text{base}} + \mathbf{t}$). Under the pinhole camera model parameterized by intrinsic matrix $\mathbf{K} \in \mathbb{R}^{3\times3}$,[12] link points project to homogeneous image pixel coordinates $\tilde{\mathbf{m}} = [u, v, 1]^\top$:

$$Z_c \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K}\left(\mathbf{R}\,\mathbf{FK}(\mathbf{q}) + \mathbf{t}\right) = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \left(\mathbf{R}\,\mathbf{FK}(\mathbf{q}) + \mathbf{t}\right),$$

[12] **Camera Calibration**: In practice $\mathbf{K}$ is estimated from a few views of a planar checkerboard, the method Zhang (2000) introduced, and the extrinsics come from a separate hand-eye calibration. Both describe one mounting at one time. The reprojection residual is therefore a calibration test repeated every session, not a setup step performed once.

where Z_c is the point's depth along the camera's optical axis, the scale that division removes when the projection $\text{proj}_{\mathbf{K}}$ maps a camera-frame point to its pixel. An automated check run on every recording session evaluates the reprojection residual between the projected link wireframe and the observed physical arm in the image:

$$\epsilon_{\text{reproj}} = \left\| \tilde{\mathbf{m}}_{\text{observed}} - \text{proj}_{\mathbf{K}}\left(\mathbf{T}_{\text{cam}\leftarrow\text{base}}\,\mathbf{FK}(\mathbf{q})\right) \right\|_2 . \tag{5.9}$$

If mechanical vibration, fastener relaxation, or an accidental impact displaces the camera bracket by even $\Delta\mathbf{t} = 3$ mm or tilts it by extrinsic misalignment $\Delta\theta_{\text{ext}} = 0.5^\circ$, the reprojection residual ϵ_{reproj} rises from its calibrated baseline of roughly one pixel to a large geometric offset. The focal length and working distance determine the size of that offset. While the underlying HDF5 or Zarr arrays

Figure 5.10: Camera-to-robot calibration check on DROID scenes: Randomly chosen DROID scenes with a robot mask rendered from the robot's URDF mesh at the recorded joint angles and projected through each camera's audited calibration, from the dataset's own post-hoc recalibration (Khazatsky et al. 2024). Where the mask and the visible arm coincide, the extrinsics are consistent with the joint encoders; a displaced camera shows up as a mask that misses the arm, before any corrupted trajectory reaches the training pipeline. Source: Khazatsky et al., Fig. 13, CC BY 4.0.

remain syntactically flawless, an end-to-end visuomotor policy trained on shifted frames learns an erroneous spatial mapping between pixel coordinates and the end-effector tool center point (TCP). During closed-loop autonomous deployment, the policy issues reaching commands that miss physical targets by the uncompensated calibration offset, converting valid demonstrations into persistent latching jams or collisions with fixtures.

Figure 5.10 displays this verification pipeline across a 7×7 grid of workspace scenes from the DROID dataset. In each, a colored silhouette of the robot arm, rendered from its Unified Robot Description Format (URDF) kinematic mesh at the recorded joint angles, is projected through each camera's extrinsic calibration matrix and overlaid directly onto the raw photographic frame. Where the mask and the physical arm coincide, camera extrinsics and joint encoders are geometrically aligned; where the colored silhouette departs from the links, the check exposes camera mount slippage before corrupted trajectory pairs enter training. The DROID corpus showed how easily mount drift escapes standard checks. Although its cameras were initially calibrated with checkerboard targets during data collection, independent re-analysis revealed that rendered kinematic masks barely overlapped visible robot links across numerous recorded episodes (Lu et al. 2025). The DROID research team subsequently evaluated stored extrinsics by measuring mask intersection-over-union, discarded failed sessions, re-estimated camera poses using learning-based hand-eye estimators, and published updated calibrations for approximately 36,000 episodes (Khazatsky et al. 2024). This calibration error shipped within a major open-source corpus because it was transparent to standard file format parsers. To prevent these silent failures, ingestion architectures must bind every recorded episode to an immutable calibration identifier and real-time clock synchronization metrics. Trajectories collected under conflicting geometric or temporal calibrations force neural policies to average incompatible sensorimotor distributions within identical state spaces.

5.10.2 Sim-to-real transfer artifacts in rigid-body simulators

In a rigid-body simulator with a relaxed soft-contact solver, a scripted gripper can interpenetrate a jammed latch plate by a millimeter and slide it open without binding. A behavioral cloning policy trained on those synthetic trajectories learns to push straight through resistance that stalls the physical arm. Because synthetic data generation replaces hardware wear with numerical approximations, degradation in a simulated dataset comes from the solver's contact model rather

than from slipped camera brackets or fatigued operators. Discrete-time physics engines resolve contact constraints once per integration step, and the chosen formulation (whether strict complementarity constraints (Stewart and Trinkle 1996), convex soft-contact relaxations (Todorov et al. 2012), or penalty-based spring-damper models) sets penetration depths, slip thresholds, and drivetrain compliance. Whereas reinforcement learning agents actively exploit numerical quirks to maximize synthetic reward, behavioral cloning policies passively inherit whatever unphysical contact impulses, penetrations, or frictionless transitions the simulator produces. Co-training on simulated and physical demonstrations can nonetheless improve real-world policy success despite discrepancies between the two regimes (table 5.5) (Maddukuri et al. 2025). A physical ingestion pipeline cannot enforce exact contact fidelity across arbitrary geometries; its job is to preserve provenance. Every synthetic trajectory must record its simulation engine, software version, contact solver parameters, and integration time step. The scenario table also flags cells populated solely by synthetic rollouts (table 5.6), so downstream training and diagnostics can separate solver artifacts from hardware behavior (section 6.4.2).

↪ The decomposition of transfer discrepancies into dynamics, sensing, appearance, and timing channels, sized from paired command-response traces, is developed in section 6.5.

5.10.3 Rig-to-rig divergence across nominally identical fleets

When a worn secondary manipulator with 0.35° of gearbox backlash and a 22 ms torque-response lag records latch demonstrations for a fleet, the human operator unconsciously adds aggressive lead compensation to overcome the deadband. Replaying or imitating those demonstrations on a newly commissioned arm with 0.02° of backlash drives the new arm into limit-cycle oscillations. Even among factory-identical robots of the same model number, individual units differ in gearbox wear, thermal dissipation, and motor-controller filter tunings. To keep fleet-wide pooling from mixing incompatible plants, data ingestion pipelines must bind each unit's embodiment contract (its kinematic parameters, actuator serial numbers, controller firmware revisions, and identified dynamic models) to every recorded file. They then filter out trajectories from units that violate the target platform's compatibility bounds.

Aggregating fleet data also exposes a second failure mode: confusing physical task failures with telemetry logging faults. A failed attempt is a complete, physically valid sequence in which the robot tried a task and missed the goal. Discarding those episodes introduces survivorship bias[13] and strips the dataset of the boundary states, recovery paths, and negative outcomes needed to train stabilizing policies. A failed recording, by contrast, is an infrastructure fault, such as a dropped network packet, a torn image frame, or a ring-buffer overrun. Ingesting a corrupted recording without tracking gaps presents missing or uneven sample intervals as a continuous fixed-rate stream, corrupting finite-difference state derivatives. Data pipelines must therefore sort every session into one of three categories: a retained success, a retained physical failure, or an excluded logging fault tagged with diagnostic error codes. In the door-latching campaign, an episode halted by the 15 N contact tripwire is a valid physical failure and stays in the provenance archive, whereas a run in which the 6-axis force-torque sensor dropped packets is quarantined with its bus error code. Retaining a physical failure does not make its commands expert imitation targets; only stabilized, reviewed recovery segments or expert-relabeled states qualify under the intervention curation rules.

13 **Survivorship Bias**: During the Second World War, Abraham Wald of Columbia's Statistical Research Group estimated the vulnerability of combat aircraft from the damage on the planes that came back; the planes that were hit in the most vulnerable places were the ones missing from the sample (Mangel and Samaniego 1984). A demonstration archive filtered to successes has the same hole. The attempts that reached the failure boundary are exactly the ones absent, so the archive understates how often the boundary is reached and holds no states from it.

Even when every packet arrives intact, a fleet interface can still corrupt an archive by recording numbers in the wrong physical units or coordinate frame. Because standard schema validators check only data types and value ranges, such mismatches pass ingestion silently until a downstream consumer interprets the numbers under a different convention (war story 5.1).

War Story 5.1: Mars Climate Orbiter unit mismatch (1999)

Context: During its nine-month cruise to Mars in 1998 and 1999, the Mars Climate Orbiter fired small thrusters to desaturate its reaction wheels, and the navigation team on the ground folded the resulting velocity changes into the trajectory models used to plan each correction maneuver (Stephenson et al. 1999).

Mechanism: A ground software file (called "Small Forces" in the investigation) reported thruster impulse in pound-force-seconds ($\mathrm{lbf \cdot s}$), while the trajectory models that consumed it assumed newton-seconds ($\mathrm{N \cdot s}$) as the interface specification required. Because $1\ \mathrm{lbf \cdot s} =$

$4.45\ \mathrm{N \cdot s}$, every desaturation event entered the models too small by a factor of 4.45. The values were well formed, in range, and of the expected type, so no schema check could flag them.

Impact: The accumulated error was never resolved during cruise. At orbit insertion the spacecraft passed about 57 km above the surface instead of the planned 226 km and was lost.

Response: The Mishap Investigation Board traced the loss to the missing unit conversion in the ground file. It recommended that the Mars Polar Lander project verify consistent units throughout design and operations and audit all data transferred between JPL and Lockheed Martin Astronautics for compliance with the interface specification, along with stronger navigation review and anomaly escalation.

Systems lesson: A number carries its meaning only together with its units and its frame. A robot dataset runs the same risk on every action tap and every calibration. A record that stores a value without the units, frame, and calibration identifier it was produced under lets a mismatched rig pass every tensor check, which is why the provenance record of section 5.11 binds each tap to them.

5.10.4 Operational design domain blind spots and unsafe interpolation

If the warehouse manipulator contacts the cage door during a fast $0.10\ \mathrm{m/s}$ approach, an unpopulated cell in its scenario table, a policy trained on the `latch-recovery-02` archive can trigger the same withdraw-and-reseat motion that worked during a slow $0.03\ \mathrm{m/s}$ approach. The motion yanks the gripper back before impact strain has dissipated and bends the latch pin. Encountering an unobserved operational state does not make a neural controller pause; the network continues emitting commands, often replaying familiar recovery primitives in regimes where their physical preconditions fail. Each recovery primitive therefore needs its own scenario-table coverage for the physical states in which it executes, rather than a log of the failure conditions that trigger it. Collection supplies experience, and closed-loop evaluation must establish competence in the conditions where the policy is to operate. A qualified permission restriction or physical constraint must exclude regimes that remain unsupported. The 2023 Cruise pullover recounted in section 1.8 is the same failure at road scale. A familiar maneuver, the pullover toward a minimal-risk condition, ran in a state no one had covered, with a pedestrian beneath the vehicle (section 12.9.1 gives the full account).

Sensor calibrations, human operator habits, actuator wear states, and operational design domain boundaries define the physical semantics of every recorded trajectory. If data ingestion pipelines strip these physical metadata to generate flat, uniform tensors, downstream evaluations cannot determine whether an autonomous failure stems from model capacity limitations or from unmodeled distribution shifts embedded within the training corpus. The structured provenance schema of section 5.11 defines the mechanisms that preserve this physical context throughout the dataset lifecycle.

5.11 The Dataset Schema

Consider an audit of two trajectory files from the latch campaign, each holding the exact same number of state-action pairs logged at the same frequency. File A records trajectories from four authorized operators across several days, with no permission-path clamps and consistent fingertip pads throughout. File B records a single operator, logs 620 steps where safety logic clamped joint velocities, and spans an unannounced force-sensor re-zeroing midway through the session. A standard validation script checking only tensor dimensions and value ranges accepts both files without complaint. Treating them as interchangeable training data introduces silent, corrupting bugs. File B requires explicit intervention attribution and recalibration metadata before an engineer can interpret its action labels correctly. Neither file's tensor geometry reveals which physical latch conditions the recorded data actually covers.

The dataset schema organizes the contextual evidence required to make ingestion decisions safely. It preserves the physical origin and timing of every signal, logs collection costs and dropouts, and documents which physical conditions the archive covers or omits. These fields reflect the mechanical and operational realities of the collection cell. While complete metadata cannot guarantee that a

learned policy will operate successfully in closed loop, omitting it ensures that downstream pipelines will misinterpret physical signals.

The **provenance record** is the immutable, machine-readable artifact bound directly to a demonstration archive. It links every recorded trajectory to the specific plant, sensor calibrations, component wear state, four action taps, collection denominators, and scenario-table join ($N_{\text{supported}}/N_{\text{req}}$), including its count unit, eligibility and retention rule, and minimum, under which data collection occurred. Standard tensor shapes and training loss curves cannot validate a physical dataset. By recording an archive's empirical boundaries, the provenance record allows the policy manifest (the record that training attaches to each candidate policy) and the safety case to distinguish demonstrated historical experience from the operational competence that physical testing must establish. Datasheets for datasets (Gebru et al. 2021) standardize documentation for static corpora; embodied archives additionally require tracking physical wear, permission-path interventions, and tap histories across the causal boundary. This record must travel with the trajectory data itself. Because detachable sidecar files easily separate from their payloads during format conversions, engineers serialize the record as an immutable, hashed header within the container's root metadata, such as an MCAP top-level metadata record or a Zarr root attribute dictionary.

↪ The sixteen-record chain, in which the provenance record is the third record, is developed in section C.14.2.

The provenance record chains Part I's hardware records directly to Part II's training and evaluation records. It inherits the actuator and thermal bounds from the limit record and the clock domains and jitter budgets from the handoff record, while the proposal boundary determines which entity held control at each of the four action taps. As episodes are logged, the collection runtime adds the plant identifier, calibration matrices, component wear counts, yield denominators, and scenario-table join. The runtime then hands that complete record forward to the policy manifest, so closed-loop evaluation can test the policy against the exact conditions its data covered.

↩ The handoff record, which extends each limit with the handoff and clock domain that locate it in the machine, is established in section 4.9.

Every trajectory admitted into the training corpus must identify its collection source, collector policy, and execution authority. For human teleoperation, the record logs the operator identifier, the interface hardware, and transport timing characteristics, for example a 6-DoF leader arm sampled at 100 Hz with 0.8 ms of input jitter. Storing an operator identifier, or capturing human faces on peripheral camera streams, makes an episode a record of people as well as of the robot. The schema therefore documents the legal and institutional basis under which each person was recorded, the authorized deployment purposes, and strict retention schedules. This governance ensures that an episode collected to teach latch insertion is not repurposed to evaluate the worker who demonstrated it. When data originates from an autonomous or mixed controller, the record stores the exact policy checkpoint, the planner version, and the active enforcement configuration, including Cartesian bounding boxes, joint velocity limits, and contact-force clamping thresholds. Trajectories must also record the active retention rule, documenting whether the runtime committed an episode automatically upon task completion, retained it after explicit operator review, or flagged it for inclusion as an instructive failure recovery.

↪ Purpose limits and retention schedules for records that capture people are enforced in section 17.7.1.

A physical machine cannot be identified by a generic catalog name or model string. In real hardware, two nominally identical robot arms behave differently because of manufacturing tolerances and component aging, so the data record specifies the individual plant through its intrinsic and extrinsic sensor calibrations, embodiment contract (section 5.9.1), motor-drive firmware revisions, and component wear state. Even replaceable contact surfaces alter the governing dynamics. A parallel gripper fitted with compliant silicone pads of Shore 30A hardness (a durometer scale for rubbers, on which higher numbers are stiffer) produces vastly different friction and compliance than the same mechanism fitted with worn Shore 60A polyurethane pads under identical joint commands. Maintenance milestones such as cable retensioning, drive-belt replacements, gearbox swaps, and camera remounts therefore belong directly in the machine state record. If technicians recalibrated joint offsets between morning and afternoon collection shifts, the dataset exposes that boundary, so downstream imitation learning does not attempt to resolve two conflicting kinematic mappings into a single policy.

Collection denominators reconcile every physical attempt as a retained success, a retained failure, or an excluded recording, while exposing the wall-clock time spent resetting the scene. Alongside those yield counts, the four action taps of section 5.5.1, override flags, and exclusion codes preserve the critical distinction between demonstrated behavior and permission-path intervention. When safety logic intervenes to clamp velocity or limit contact forces, the data pipeline must at-

tribute those adjustments correctly rather than misinterpreting supervisory interventions as expert demonstrations.

The scenario join records dataset-supported cells (configurations containing retained demonstrations) and catalogs known absences where no empirical data exists. The count-policy digest binds the count unit, eligibility and retention rule, and declared minimum; when a minimum is specified, the record distinguishes occupied cells from those meeting it. Here, dataset support reflects historical logging coverage and must not be conflated with the evaluated support of a deployed policy. For the latch campaign, the record below connects collection cost to coverage; table 5.6 maps its field groups to the decisions they support.

- **Denominators:** 52 attempts per scheduled session at a net retention rate of 0.68, so 5,000 retained successful demonstrations cost 7,353 attempts over 141.4 h of scheduled cell time.
- **Retention rule:** an attempt whose contact force crosses the 15 N tripwire triggers the qualified collection response and is kept as a labeled failure, outside the demonstration count, when its telemetry is valid; a recording that fails quality filtering is excluded with its exclusion code.
- **Scenario table:** plate offset (nominal, +3 mm, –3 mm) crossed with approach speed (0.03 m/s, 0.10 m/s), six cells, held with the count and retention policy in the record as a digest rather than as a copy.
- `supported_cells`**:** the nominal plate at 0.03 m/s, the only cell the original retained demonstrations occupy.
- `known_absences`**:** the other five cells, every displaced plate and every approach at 0.10 m/s.

DRAFT

Table 5.6: Demonstration dataset provenance schema: Evidence retained and the admission check applied to each field group of the provenance record; the embodiment, sensor calibration, and contact wear groups bind the individual plant that collected the data.

Provenance record	Evidence retained	Admission decision
Session	Collector, operator, policy version, source type (physical, robot-free, or synthetic with generator and engine versions), collection mode (nominal, noise-injected, or gated intervention), sample rate, and timing jitter.	Check operator authorization and the session jitter budget.
Embodiment	Robot identity, geometry, gearing, backlash, firmware, and the redundancy parameter (elbow swivel angle or full joint vector).	Check reach, dynamics, and joint limits on the target plant.
Sensor calibration	Camera geometry, clock synchronization, and force/torque bias.	Check calibration age and timebase error.
Contact wear	End-effector material, hardness, friction, flex cycles, and maintenance history.	Revalidate contact data after material or compliance changes.
Four action taps	Requested, mapped, enforced, and measured values, each with its units and frame, plus override and clamp codes.	Compare aligned values; attribute modified commands before choosing labels.
Transition timing	Times of exposure midpoint, action issue, mapping, enforcement, response start, and response end, with clock IDs and synchronization errors.	Check causal order and the declared response window.
Collection denominators	Scheduled cell time, motion time, attempts, retained successes and failures, excluded recordings, and task configurations.	Reconcile attempts against retained and excluded episodes.
Scenario support	Scenario-table and count-policy digest (unit, eligibility, retention, minimum), occupied cells, cells meeting any declared minimum, synthetic-only cells, and known absences.	Identify collection gaps; require separate evidence for deployment claims.
Consent and retention	People recorded (operator and any bystander in view), the consent or other basis for recording each, the declared purposes, the retention period, and who may read the record.	Admit an episode only for a purpose its basis covers; delete or de-identify it when retention ends.

The policy manifest of section 6.8 cites each contributing archive by digest, including `latch-recovery-02` in the stipulated follow-up branch. The original record continues to describe the original campaign; new experience receives new provenance rather than rewriting old absences. Operator and override history also informs supervisory evaluation, while coverage and calibration evidence contributes to the release argument. Binding the metadata to the trajectory container helps preserve these relationships through ingestion, but cryptographic hashes do not automatically prevent misuse. Data loaders and ingestion pipelines must verify that trajectory arrays remain bound to their declared schema and calibration headers before training begins, because skipping any part of that check leads to the recurring collection failures cataloged next (section 5.12).

↪ The release manifest, which binds this provenance record and its operating-domain coverage into the release argument, is developed in section 16.8.

5.12 Fallacies and Pitfalls

In pure software systems, data collection can often treat storage growth as an unvarnished positive, since more web pages, text tokens, or labeled images generally broaden coverage. In physical artificial intelligence, however, datasets are tightly coupled to the mechanical dynamics, calibration state, and physical limits of the collection hardware. A dataset that passes every structural schema check, with zero dropped packets and consistent tensor dimensions, can still leave a deployed machine entirely blind to critical operational states. The operational distribution of physical data reflects three intertwined factors: the collecting policy (whether driven by human teleoperation or automated planning), the physical capabilities and wear of the collection rig, and the curation filters applied before admission to the archive. Neglecting these physical constraints leads to predictable engineering fallacies and costly operational pitfalls.

Fallacy: *Massive sample volume compensates for lack of operational state-space coverage.*

Suppose a surface-tracking policy must operate reliably across 500 distinct combinations of normal force, surface friction, and contact angle, but the archive covers only 45. Logging another collection run that adds $72{,}000$ control steps by repeating nominal trajectories through those same 45 states inflates the database, but it leaves physical coverage unchanged. High-frequency sampling in familiar regions merely records the same dynamic trajectory more densely, adding zero evidence about the unvisited friction regimes or steep contact angles. Ingestion pipelines that report growing data volume and zero dropped packets can create a false sense of security while leaving downstream policies brittle. A collection campaign earns its cost by addressing a declared coverage, label-quality, or evidence deficit. A populated cell may still need more or better demonstrations, while nominal repetition alone supplies no evidence about unvisited regimes.

Architect's countermeasure: Gate dataset admission on a support envelope relational join against an explicit scenario table, rather than step counts or raw byte volume. Require that new data collection campaigns specify which coverage, label-quality, or evidence deficit they target, and track scenario coverage ($N_{\text{supported}}/N_{\text{req}}$) as a primary engineering KPI alongside validation loss.

Pitfall: *Assuming physical demonstration labels survive mechanical or contact hardware maintenance.*

Archived demonstrations do not retain physical validity when the underlying hardware changes, even if array shapes and data schemas remain identical. Consider what happens when a technician replaces a parallel gripper's compliant fingertips with stiffer pads and re-torques an adjacent camera bracket. The historical dataset still passes every automated pipeline check: sensor channels, array dimensions, and datatypes match expected specifications. If the camera bracket shifts, a known transformation matrix allows mathematical reprojection of visual coordinates. Contact mechanics, however, cannot be reprojected geometrically. The stiffer pads alter normal compliance and contact friction, causing grasp slip at substantially lower shear thresholds than the compliant material tolerated. Telemetry recorded before the swap captures the physical behavior of an obsolete body; training a closed-loop force controller on those historical tactile profiles will cause grasping failures on the modified hardware. Data provenance must track the full mechanical and calibration state of the collection rig so that any hardware alteration immediately triggers targeted review and recollected baselines.

Architect's countermeasure: Bind every trajectory file to an immutable hardware state hash that captures camera extrinsics, motor serial numbers, and contact surface material specifications. When maintenance events alter contact stiffness or camera mountings, invalidate historical tactile and force labels for that cell until an automated revalidation protocol verifies dynamic equivalence.

Pitfall: *Filtering out aborted safety interventions to sanitize the demonstration dataset.*

Purging aborted demonstrations to produce a flawless training set introduces severe survivorship bias into physical policies. Consider an automated peg-insertion pipeline spanning 1,000 recorded attempts. Of these, 180 runs triggered a safety cutoff when contact forces reached or exceeded a 15.0 N trip threshold, leaving 820 nominal runs that succeeded with an 82 percent yield. A curation script discards the 180 aborted and recovery episodes, preserving only the smooth runs where forces stayed comfortably below 8.0 N. When deployed in production, minor surface misalignment inevitably pushes an insertion force to 9.5 N. Although this force remains below the 15.0 N trip threshold, the policy encounters a state outside its training support. Because the sanitization script erased every observation between nominal tracking and failure, the archive supplies no supervision for these near-boundary dynamics, so recovery behavior remains unsupported. Pruning near-boundary and intervention data strips away the exact state-action pairs the policy needs to regain stability after a disturbance.

Architect's countermeasure: Implement three-way intervention curation at data ingestion. Quarantine learner commands between divergence (t_{div}) and takeover (t_{takeover}) for safety auditing only. Preserve the visited boundary states for expert relabeling via corrective aggregation (DAgger). Admit the subsequent human recovery as an expert sequence only after the operator's measured stabilization ($t_{\text{stabilize}}$), masking the blend and reorientation that precede it.

Pitfall: *Merging demonstrations collected across kinematically asymmetric leader interfaces.*

Merging trajectories across disparate teleoperation input interfaces can quietly bake kinematic dead zones into the dataset. Suppose two operators collect snap-fit assembly demonstrations using different leader devices to puppet identical follower arms. The first leader mechanism features detented wrist mechanics that restrict roll to two discrete positions ($\theta_{\text{roll}} \in \{0.0^\circ, 90.0^\circ\}$). The second leader uses a continuous gimbal that mechanically limits roll travel to $[-15.0^\circ, +15.0^\circ]$. Merging the two collections yields 4,000 episodes and increases overall sample volume, but neither interface can express orientations between 20.0° and 70.0°. Even though the follower arm can navigate that full range, the combined archive contains zero training targets within this 50.0° gap. Pooling data from different input devices cannot fill gaps that neither device can mechanically traverse. Downstream imitation policies trained on the combined dataset will fail whenever initial conditions or part tolerances demand intermediate orientations.

Architect's countermeasure: Apply the cross-embodiment compatibility contract to the leader interface itself, checking before recording that the leader and follower share the required reachable poses, joint motion ranges, and ability to move in the required directions. Enforce continuous orientation coverage checks across task-space degrees of freedom during data QA to detect mechanical dead-zones.

Fallacy: *Physical data collection scales linearly with labor hours just like internet scraping scales with compute.*

Treating physical data collection as an easily parallelized, linear process like web scraping ignores the mechanical entropy and human fatigue inherent to physical experimentation. Teams often extrapolate campaign timelines from early pilot runs, for example by projecting that an initial pace of 50 successful episodes per operator-hour will scale linearly across weeks of shifts. In reality, continuous operation degrades both hardware and teleoperator consistency. Repetitive contact impacts cause backlash and tooth wear in strain-wave gearboxes, thermal expansion and sensor drift cause uncalibrated joint encoders to drift away from physical ground truth, and human fatigue introduces latency spikes and jerky trajectories. As wear accumulates, the proportion of episodes passing quality filtering steadily drops, even while scheduled labor hours remain constant. Pilot runs conducted on pristine hardware systematically overstate long-term collection yield. Campaign planning must budget for routine recalibration, mechanical maintenance, ergonomic rests, and discarded runs alongside raw operator hours.

Architect's countermeasure: Budget collection campaigns in net validated demonstration yield ($Y_{\text{usable}} = \lfloor T_{\text{motion}} / t_{\text{cyc}} \rfloor \cdot R$) and component fatigue cycles (L_{10} ratings for strain-wave gears and internal flex harnesses), rather than gross scheduled operator hours. Estimate the overhead multiplier from the campaign's calibration, reset, retention, and rest assumptions; validate it against sustained collection yield.

5.13 Summary

A physical machine manufactures its own training data, so every record carries the loop that produced it: the operator's request, the permission path's admission, and the body's response, all under the machine's current calibration and wear. Collecting that data is costly in a way scraping is not. In the latch campaign, only 24.6 percent of each scheduled hour survives as retained demonstration motion once startup, calibration, operator rest, resets, failed attempts, and quality filtering are paid for, and the wrist's flex cable consumes the larger fraction of its assumed fatigue rating. Because every attempt spends cell time and component life whether or not it is kept, and because correlated control steps hold few independent samples, a campaign earns its cost by addressing a declared coverage, label-quality, or evidence deficit, including inadequate evidence within a populated cell.

Which states the archive holds is a question the archive cannot answer about itself. Coverage is a join between empirical occupancy and a scenario table derived from the declared operational design domain, and against the table the door actually demands, the baseline latch archive fills one cell of six. Inside even a populated cell, a cloned policy's first error carries it off the demonstrated corridor into states where nothing in the archive constrains its output, which is why behavioral cloning's worst-case cost grows with the square of the horizon. Interventions supply the missing learner-visited states, but only after curation at the divergence, takeover, and stabilization timestamps

separates the learner's commands from the expert's recovery. Aggregating expert labels on those states makes the worst-case cost linear in the horizon only while the recoverability constant stays bounded; where a mistake cannot be undone, the state belongs to the permission path, not to the dataset.

A label is only as meaningful as the tap, clock, and plant behind it. Imitating the request teaches commands the permission path would have clamped, imitating the measured response teaches compliance and backlash as intent, and pairing either with a misdated frame converts timing error into position error at the speed of the motion. Range normalization cannot carry a label to another embodiment, and a bumped camera bracket or worn pad silently changes what a stored tuple means. The provenance record binds each archive to its taps, timestamps, plant, calibration, collection denominators, and known absences, so that training and evaluation can tell demonstrated experience from evaluated competence. Table 5.7 summarizes the load-bearing equations introduced throughout this chapter, detailing the physical boundaries each expression defines, the architectural decisions it governs, and its formal derivation.

Table 5.7: Key equations of physical data: Each load-bearing equation of the chapter, the limit it bounds or the decision it settles, and the equation where it is derived.

Equation	What it bounds or decides	Derived in
$J(\pi) - J(\pi^*) \leq S^2\epsilon$	The worst-case excess cost of cloning on expert states under a 0–1 action loss; the bound becomes uninformative once $\epsilon > 1/S$	equation 5.1
$J(\hat{\pi}) \leq J(\pi^*) + u\,S\,\epsilon_N + \mathcal{O}(1)$	The cost after aggregating expert labels on learner-visited states, linear in S only while the recoverability constant u stays bounded	equation 5.8
$N_{\text{eff}} = N(1-\rho)/(1+\rho)$	The effective number of independent observations contained in a serially correlated telemetry stream for a single statistic	equation 5.2
$\Delta\mathbf{x}_{\text{sync}} \approx \mathbf{v}\,\Delta t_{\text{lag}}$	The spatial error introduced by inter-sensor pairing latency, defining the maximum allowable delay for a specified task clearance	equation 5.3
$\Delta t_{\text{skew}} = (H-1)\,t_{\text{row}}$	The temporal skew across a rolling-shutter frame, which end-effector motion translates into geometric image distortion	equation 5.4
$t_{\text{loop}} = t_{\text{disp}} + t_{\text{hum}} + t_{\text{cmd}} + t_{\text{mech}}$	The round-trip phase lag in teleoperation between an observed state and the physical execution of the human correction	equation 5.5
$Y_{\text{usable}} = \lfloor T_{\text{motion}}/t_{\text{cyc}} \rfloor R$	The yield of usable demonstrations per scheduled session, determining the cell time and mechanical wear required for a campaign target	equation 5.6
$\hat{t}_{\text{div}}$: first threshold crossing, when captured, in $[t_{\text{takeover}} - T_{\text{pre}},\, t_{\text{takeover}}]$	The estimated onset of trajectory drift before human takeover, determining whether the lookback window captures the causal error	equation 5.7
$\epsilon_{\text{reproj}} = \lVert \tilde{\mathbf{m}}_{\text{observed}} - \text{proj}_{\mathbf{K}}(\mathbf{T}_{\text{cam}\leftarrow\text{base}}\,\mathbf{FK}(\mathbf{q})) \rVert_2$	The extrinsic reprojection residual used to verify that camera calibration remains aligned with kinematic joint encoders	equation 5.9

Key Takeaways: Physical data collection and provenance

- **Coverage measures distinct physical configurations rather than sample volume:** Repeated nominal trajectories add millions of telemetry rows without expanding scenario coverage. Adjacent control steps are so strongly autocorrelated that an hour of permission-tick records holds only about 1,800 independent samples for a slowly varying statistic, and even independent samples support only the cells where they land. An engineering team must measure dataset completeness by joining logged trajectories against an explicit scenario table that spans the operational design domain, rather than relying on raw row counts or the archive's own observed extremes.

- **Offline validation loss cannot detect coverage deficits:** Because conventional train-validation splits sample from the same underlying trajectory distribution, a policy can achieve near-zero validation loss while leaving critical operational cells unvisited. Only auditing the dataset against an independent scenario table reveals these silent omissions before deployment.
- **Every action tap represents a distinct physical control signal:** An operator's commanded request, the resolved kinematic setpoint, the safety-filtered actuator command, and the measured joint response diverge whenever hardware limiters, friction, or permission guards intervene. Logging each tap alongside synchronized timestamps ensures the supervisory target remains causal and preserves unambiguous evidence of physical interventions.
- **Preserve aborted runs and interventions with rigorous causal attribution:** Discarding failed attempts discards the most valuable boundary-adjacent states in the operational envelope. Autonomous commands preceding a human takeover must be archived for fault analysis and counterfactual relabeling rather than misattributed as expert demonstrations, while the subsequent corrective maneuver should be reviewed, trimmed, and stored as an explicit recovery trajectory.
- **Physical telemetry remains bound to specific hardware embodiments:** While optical perspectives can be reprojected across camera geometries, dynamic properties such as joint friction, structural compliance, and gearbox backlash are embedded directly into recorded force and torque signals. Simple range normalization masks these physical discrepancies without resolving them, requiring thorough re-calibration and verification whenever data is transferred across hardware platforms.
- **Targeted recovery data broadens operational competence under new provenance:** An immutable dataset records the boundary conditions of a specific collection campaign, but engineers can systematically expand policy capabilities by appending verified recovery sequences under separate provenance tags. Asserting an expanded operational claim, however, requires end-to-end closed-loop validation on the physical machine under those target conditions.

Each latch archive in this chapter ended the same way, with a short list of supported cells and a longer list of known absences. That ratio is not a defect of one campaign. It is what the second law, endogenous data (principle 2), costs in practice. A machine whose actions choose its next states can only record the states its collecting policy chose, and every additional state it records is paid for in cell time and component life. The known absences in the provenance record are the price list. They say which experience the machine never bought, and therefore which claims no amount of training on this archive can support.

Read that way, the engineering question shifts from how much data a campaign collected to which absences it retired and what each one cost. The displaced-plate cell that `latch-recovery-02` fills moves from the absence list to the supported list under new provenance. The faster approach, which no one collected, stays on the list, and the permission path must keep refusing it until evidence arrives.

5.14 Historical Perspective and Further Reading

Demonstration in robotics began not as data collection for machine learning, but as teleoperation for hazardous environments. At Argonne National Laboratory, Raymond Goertz developed mechanically coupled master-slave manipulators to handle radioactive materials behind heavy shielding (Goertz 1949). These early systems linked the operator's handle directly to remote mechanical tongs through cables and counterweights; because the transmission was entirely mechanical, contact forces against objects reflected instantly and passively back to the operator's hand.

The systems challenge shifted radically when electrical servos and digital communication networks replaced mechanical linkages. Introducing transmission latency into the bilateral loop between the operator interface and the actuated follower can inject artificial energy into the feedback channel,

quickly driving the coupled system into violent instability. Anderson and Spong (1989) proved that even small transmission delays can destabilize a bilateral teleoperator and established a framework that preserves delayed-channel passivity under its constant-delay model. Stability of the complete loop also depends on the connected systems and their interconnection conditions. Niemeyer and Slotine (1991) reformulated this remedy through wave variables, transforming power variables (force and velocity) into traveling wave representations that behave as passive transmission lines under the constant-delay model. In their comprehensive survey, Hokayem and Spong (2006) trace the progression from Goertz's early linkages to modern delay-robust control schemes. Every teleoperation data-collection interface operates under the compromise these works describe, balancing faithful haptic transparency against the conditions needed to preserve passivity. The physical trajectories, contact transitions, and actuation commands captured in an embodied training archive inherently reflect the compromises of the collection hardware.

Synthesizing feedback controllers directly from recorded demonstrations followed an equally grounded trajectory. In one of the earliest end-to-end vision-based driving systems, ALVINN mapped camera frames to steering angles for an autonomous van (Pomerleau 1989, 1991). ALVINN immediately ran into the recovery gap. Because expert drivers kept the van centered within the lane, the recorded dataset contained virtually no examples of drifting off course or recovering from road shoulders. Pomerleau resolved this issue through an early form of data augmentation, synthetically warping camera images and computing the corresponding corrective steering angles to teach the network how to return to the road center.

As the field matured, Argall et al. (2009) unified robot learning from demonstration into a comprehensive framework, formalizing how data collection methods shape policy representations and analyzing the kinematic correspondence problem between the demonstrator and the physical robot. The theoretical mechanics of compounding execution errors were established by Ross and Bagnell (2010), who proved that naive behavioral cloning yields an error bound that grows quadratically with the task horizon ($\mathcal{O}(S^2\epsilon)$) under covariate shift. To escape this quadratic penalty, Ross et al. (2011) introduced Dataset Aggregation (DAgger), demonstrating that rolling out the learner while querying the expert along the induced state distribution achieves an optimal linear bound ($\mathcal{O}(S\epsilon)$), provided the environment satisfies a finite recoverability condition. Practical robotics deployments adapted this theoretical machinery to human supervisors who cannot label millions of counterfactual query states. Human-gated DAgger (Kelly et al. 2019) transferred control to a human expert only during critical trajectory deviations, while DART (Laskey et al. 2017) injected continuous stochastic noise into the actuators during teleoperation to force human demonstrators to execute recovery maneuvers naturally. The empirical intervention strategies and recovery curation workflows in section 5.8 trace directly to these foundational methods.

The accounting of state-space coverage and sampling validity borrows from older statistical and control foundations. Bellman (1957) named the curse of dimensionality to describe the exponential explosion of discrete state spaces in dynamic programming. In environmental time series, climatologists demonstrated that serially correlated records dramatically reduce the effective degrees of freedom, requiring explicit adjustments to compute true sample sizes (Thiébaux and Zwiers 1984). Simultaneously, Abraham Wald's wartime analysis of returning aircraft showed that evaluating only surviving samples systematically blinds the observer to critical failure modes (Mangel and Samaniego 1984).

Robot learning confronted all three phenomena as datasets evolved from isolated laboratory demonstrations into massive multi-robot corpora (Walke et al. 2023; Khazatsky et al. 2024; Open X-Embodiment Collaboration et al. 2024). While cross-institution pooling supplies visual and environmental variations that no single robotic cell can generate, it also pools mismatched actuator dynamics, unmodeled camera latencies, and latent operator selection biases. Standardizing such corpora requires both an explicit compatibility contract across kinematics and a rigorous accounting of unobserved conditions. Although dataset documentation frameworks (Gebru et al. 2021) originally addressed provenance and social biases in digital corpora, physical robotics demands tracking hardware-level ground truths. The physical dataset schema and provenance architecture detailed in section 5.11 extend these documentation principles to capture actuator tap points, calibration matrices, dynamic delays, and known boundary omissions.

The following readings chart the progression from classical bilateral teleoperation to modern pooled robot corpora:

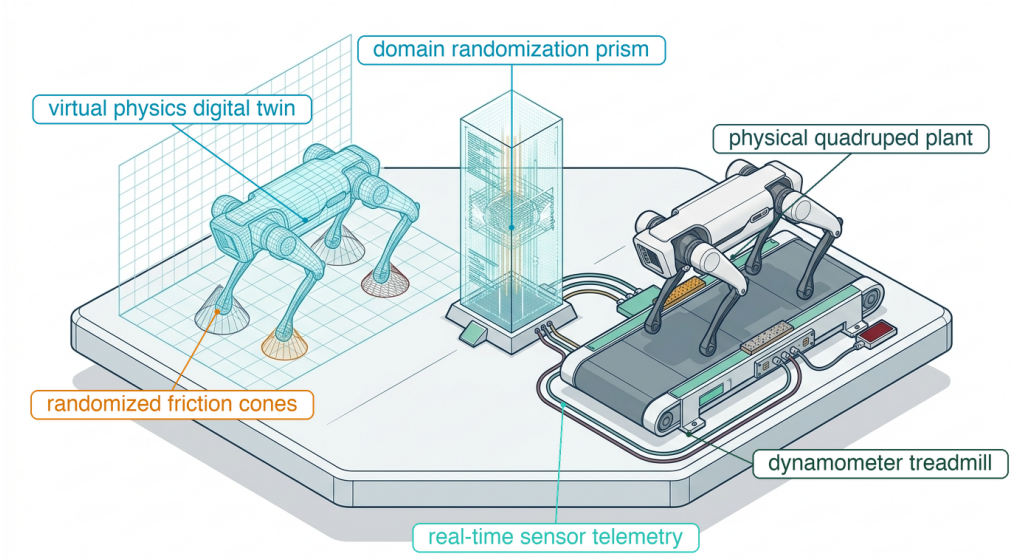
- Hokayem and Spong (2006) provides an exhaustive survey of bilateral teleoperation control, charting the historical transition from mechanically coupled manipulators to passivity-based and wave-variable architectures. It explains the core control-theoretic tension between transparent force reflection and robust stability when non-negligible communication delays separate the operator from the robot.
- Pomerleau (1991) presents a compact, highly practical diagnosis of the recovery gap and compounding errors in end-to-end behavioral cloning. The paper details how synthetic visual transformations of road images generated corrective training labels, demonstrating the necessity of off-trajectory recovery data decades before formal distribution-shift theory was codified.
- Argall et al. (2009) synthesizes robot learning from demonstration into a foundational taxonomy based on demonstration collection mechanisms and policy derivation techniques. It highlights the correspondence problem across differing human and robotic kinematic structures, which section 5.9 formalizes as a strict cross-embodiment mapping contract.
- Ross et al. (2011) introduces the Dataset Aggregation (DAgger) algorithm and establishes the formal reduction of imitation learning to online convex optimization. It proves that querying an expert along the learner's visited trajectory distribution converts the quadratic error growth of behavioral cloning into a linear bound, defining the critical recoverability constant presented in equation 5.8.
- Open X-Embodiment Collaboration et al. (2024) documents the Open X-Embodiment initiative, pooling over one million trajectories across dozens of robot platforms and institutions. The paper is essential reading for systems engineers, illustrating both the positive transfer enabled by scale and the severe architectural challenges of normalizing disparate action spaces, sampling rates, and sensor configurations.
- Gebru et al. (2021) establishes the foundational methodology for documenting dataset motivation, composition, collection workflows, and intended use. Contrasting this general framework with table 5.6 clarifies the specialized physical, electrical, and dynamical metadata required to safely train and deploy embodied controllers.

What's Next: From demonstrations to policies

A policy trained on an archive whose provenance record lists more absences than supported cells learns strictly from those supported cells; its training score gives no sign of where the absences begin. The baseline latch archive covers only the nominal strike plate at the guarded approach. Chapter 6 opens with two candidates built on it: `latch-bc-01`, cloned from those demonstrations, and `latch-sim-02`, distilled from a simulator teacher that learned the fast approach no one collected on the machine. Each improves in training while hiding a dependency the hardware will expose. That chapter then carries the curated `latch-recovery-02` archive into a recovery candidate and writes each candidate's evidence into a policy manifest whose declared operating domain cannot exceed the cells its provenance supports. Chapter 7 then tests whether that declared domain holds on the machine.

6

Policy Training

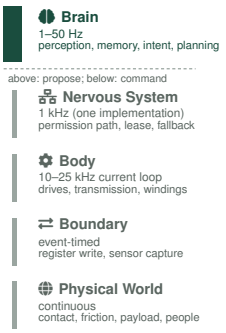


6.1 Dual Failure Modes of Policy Training
6.2 Policy Training Regimes
6.3 Reinforcement Learning and Environment Interaction
6.4 The Simulator as a System Component
6.5 Physical Channels of the Sim-to-Real Gap
6.6 Real-Robot Fine-Tuning and Residual Control
6.7 Training Failure Modes and Permission-Path Interceptors
6.8 The Policy Manifest
6.9 Fallacies and Pitfalls
6.10 Summary
6.11 Historical Perspective and Further Reading

Purpose

Why can a policy improve its training score while becoming less reliable on the machine?

A learned policy acquires its behavior from the experience its training supplies, and its weights keep no record of which experience that was. Demonstrations can omit the recovery from a displaced fixture, a simulator can reward a fast approach that a real contact surface cannot tolerate, and a model pretrained on other robots can behave plausibly in conditions no one has tested on this machine. In each case the training score rises while the dependency stays hidden, so ranking candidates by score can favor the one that exploited its data or its solver. Training also fixes how long the policy takes to propose an action, and every improvement gathered on the machine itself is paid for in heat and component life. An engineer who does not understand these mechanisms cannot choose a source of experience that supports the behavior the task needs, cannot state the conditions in which a checkpoint may be trusted, and discovers each hidden dependency only when the machine meets it. Because an admitted action crosses the causal boundary and its consequences cannot be undone by another gradient update, the limits of a policy's evidence must be established before it is allowed to propose motion.



Learning Objectives

- Compare training regimes, from behavioral cloning to post-training a pretrained generalist, by the experience they supply and the physical dependencies they leave in the policy
- Diagnose the separate risks of averaging valid action choices and drifting beyond demonstrated states
- Calculate whether an action decoder fits its proposal deadline and identify the required execution safeguards
- Evaluate simulator mismatch and distinguish parameter variation from missing mechanisms or unavailable observations
- Bound real-machine adaptation by physical exposure, residual authority, and regression evidence
- Construct a policy manifest that confines a candidate's declared operating region to its documented support

6.1 Dual Failure Modes of Policy Training

A robot arm executing a learned trajectory can track every waypoint cleanly across hundreds of simulation episodes and still arrive at its first physical deployment with no evidence about whether it will fracture a latch. In classical robotics, control design begins with an explicit analytical model of the plant: engineers write down kinematic equations, inertia matrices, stability margins, and torque limits where they can be systematically inspected and verified. A learned neural policy, in contrast, derives its behavior entirely from its training distribution, setting network weights from empirical experience to map physical observations to proposed control actions. That experience may arrive as teleoperated demonstrations, recorded operational logs, rollouts inside a parallel physics simulator, trial-and-error interactions on physical hardware, or pretrained representations imported from foundation models. Although empirical data makes complex manipulation tractable, it also embeds unstated dynamic assumptions directly into the model weights, leaving no explicit record of which operating regimes the policy can safely navigate.

The central engineering question in embodied policy training is therefore not which regime achieves the highest optimization score, but which physical behaviors the training experience supports, and at what operational cost. Four quantities decide the answer. The first is coverage, because a policy is supported only in the states its experience actually visited, which is why the provenance record of section 5.11 lists its known absences beside its demonstrated cells. The second is inference latency, because a policy that evaluates too slowly misses its control deadline, however good its proposed trajectories are. The third is hardware cost, since experience gathered on the machine consumes machine hours, heats motor windings, and wears drivetrains. The fourth is fidelity, because synthetic experience from simulation scales across parallel accelerators but is only as faithful as the integrator and contact solver that produced it. A checkpoint's policy manifest (section 6.8) records all four, and its declared operational design domain (ODD) is the part of the task's scenario table that the checkpoint's evidence supports. Two candidate latch policies show why a clean training curve does not establish physical viability.

Consider the cage-door latch task introduced in section 5.1, where two learned controllers both succeed under nominal warehouse conditions yet each carries a hidden dependency that physical hardware can expose, for different mechanical reasons. Both controllers share the same real-time execution model: Action Chunking with Transformers (ACT) (Zhao et al. 2023), a chunk-policy architecture that generates open-loop action sequences in a single decoder pass. The first candidate, `latch-bc-01`, was trained via supervised behavioral cloning on teleoperated door-opening demonstrations. The second candidate, `latch-sim-02`, is a distilled student policy. A reinforcement learning teacher was first trained inside a parallel physics simulator with full access to privileged state variables. Then `latch-sim-02`, initialized from `latch-bc-01`, learned through distillation to replicate the teacher's actions using only onboard sensor streams (section 6.5.1).

Both candidates inherit the absence list of the baseline archive. Its demonstrations characterize only the nominal strike plate at a guarded approach velocity, so a displaced plate and a faster approach are both recorded as known absences, and each candidate meets one of them. While behavioral cloning must interpolate between these discrete demonstrations to form coherent trajectory proposals, interactive learning in simulation can explore novel state-action pairs. However, the validity of that synthetic exploration depends entirely on how accurately the simulated mechanics mirror the physical plant. Selecting a training regime is therefore an architectural decision that dictates which unverified assumptions must be empirically audited before a checkpoint is permitted to command actuators.

In this illustrative comparison, at the nominal strike plate and the guarded approach speed of 0.03 m/s, both policies open the door. Their failure modes diverge once physical deployment departs from that single nominal operating point:

1. **Missing recovery experience**: When a sagging door places the strike plate off nominal, `latch-bc-01` encounters observations outside its training distribution. Because behavioral cloning minimizes supervised prediction error across the recorded demonstrations, the training loss supplies no direct supervision at unvisited recovery states, although updates to shared weights can still change the actions predicted there. When confronted with unfamiliar observations, the network still emits action setpoints, but those ungrounded commands drive the manipulator further off course. Trajectory errors compound at each 25 ms setpoint tick until the gripper binds against the strike plate's chamfered edge. The root bottleneck is the absence of corrective recovery trajectories in the training distribution, rather than model capacity.
2. **Misleading simulated contact**: In contrast, `latch-sim-02` is at risk on the nominal strike plate itself. Its simulator models contact dynamics using a compliant penalty spring[1] ($f_{k,\perp} = k_{\text{pen}}\delta + c\dot{\delta}$), where surface penetration δ and penetration rate $\dot{\delta}$ govern the contact force. Because this numerical spring yields far more compliance than a physical steel plate, the simulated impact at 0.10 m/s peaks well under the latch's 100 N limit. The teacher's training reward paid for opening the door quickly and carried no term for contact force, so nothing in training charged for an aggressive approach, and the optimizer exploited the contact model to shave cycle time off each episode. The reinforcement learning teacher converges on an aggressive plunge, and policy distillation faithfully transfers that shortcut into the student. Whether the plunge breaks the real latch depends on how stiff the arm side of the contact is, a quantity nothing in training measured.

A contact-force budget demonstrates why the simulated benchmark score cannot be trusted on hardware. In the rigid-arm, constant-drive estimate, the physical strike plate's stiffness sets the force rise. At the unbraked approach velocity of 0.10 m/s, force reaches the 100 N limit in 2.5 ms after touch. The tripwire fires at 0.375 ms; the subsequent 2 ms response places brake onset at 2.375 ms, leaving only 0.125 ms before the constant-drive ramp reaches the limit. That interval is not a stopping certificate. Section 6.4.2 derives the timeline alongside the passive impact forces; sanctioning the fast plunge would require physical deceleration evidence that this simulation-only training process cannot establish.

Both failures expose dependencies hidden from validation loss and episodic reward: behavioral cloning depends on demonstrated recovery distributions, while simulation-to-real transfer depends on faithful contact and stiffness dynamics. Selecting a policy training regime therefore requires matching the physical properties of the training source to the mechanical demands of the operational task. Whichever regime produced it, a learned checkpoint sits above the proposal boundary as an unprivileged proposal generator. Safe and reliable manipulation requires action proposals that respect the machine's actuator dynamics, contact deadlines, and physical safety envelopes as rigorously as they pursue the nominal task objective. Comparing the five regimes that can generate those proposals clarifies what each source of experience buys and what it leaves unverified.

6.2 Policy Training Regimes

When `latch-bc-01` binds against a sagged strike plate and `latch-sim-02` plunges into the nominal plate at 0.10 m/s, neither failure can be repaired by training the same network for more epochs on the same dataset. Fixing an unvisited recovery state or an exploited contact solver requires

[1] **Kelvin-Voigt Contact**: A linear spring and damper in parallel produce a force jump at first touch, because the damping term acts on the full approach velocity at zero penetration, and a pulling, "sticky" force as the bodies separate, because the damping term then outweighs the vanishing spring term. Hunt and Crossley (1975) scaled the damping with penetration to remove both artifacts. A policy trained against the linear model therefore meets its least faithful forces at contact onset and at release, the two events a latch opening depends on.

↩ The proposal boundary, under which a learned proposer holds no authority until an independent permission path admits its command, is established in section 1.7.

changing the experience the learner sees. Training a control policy for the latch task can draw on five sources of experience: teleoperated door openings, logged archives of imperfect or scripted trajectories, a physics simulator of the door and cage, direct physical trial on the arm itself, and a generalist foundation checkpoint pretrained across multi-robot corpora. Each source buys distinct sensorimotor capabilities at a characteristic physical cost and leaves its own failure mode behind (table 6.1). Behavioral cloning, batch offline reinforcement learning, and supervised post-training extract control policies from recorded datasets without runtime exploration, whereas online trial, conducted in simulation or on the physical mechanism, generates new interactions under closed-loop feedback. Comparing how those regimes evaluate and update actions begins with the reward quantities that score closed-loop behavior over an episode.

Table 6.1: Comparative trade-off matrix across policy training regimes: The five regimes compared are behavioral cloning, offline reinforcement learning, sim-to-real reinforcement learning, residual RL/fine-tuning, and post-training of a pretrained generalist policy. Columns separate the capability sought, the resources that determine its cost, and the remaining failure mechanism. Training remedies and physical execution safeguards have different roles; neither supplies the other's evidence.

Policy Training Regime	Core Purchased Capability	Data & Compute Budget	Sample Efficiency & Scale	Hardware Safety & Wear Risk	Dominant Failure Mode & Interceptor
Behavioral Cloning (BC)	Imitates attributable expert actions	Expert collection, curation, and supervised optimization	Limited by coverage and operator availability	Optimization adds no physical exploration; collection still incurs exposure	Mode averaging and missing recovery. Multimodal decoding and new expert labels address training; qualified state and action limits bound execution.
Offline / Batch RL	Uses rewards to select behavior from heterogeneous logs	Logged transitions with rewards and state-action coverage	Reuses recorded experience without new online trials	Optimization adds no exploration; the archive inherits collection risks	Unsupported action values. Conservative objectives reduce optimism; independent action limits remain necessary.
Sim-to-Real RL	Explores behavior through simulated interaction; a privileged teacher can be distilled into a deployable student	Parallel rollout compute plus physical calibration and transfer tests	Throughput depends on task, solver, and hardware	Simulation avoids training wear; physical probing and deployment still incur exposure	Solver exploitation and omitted mechanisms. Paired probes qualify the simulator; approach and force limits need their own response evidence.
Residual RL & Real Fine-Tuning	Adapts to local physical mismatch	Machine interaction, resets, cooling, and regression testing	Physical trials make sample efficiency consequential	Contact loads and thermal exposure accumulate during adaptation	Local improvement with regression elsewhere. Independent residual gates limit authority; combined-controller tests assess the change.

Policy Training Regime	Core Purchased Capability	Data & Compute Budget	Sample Efficiency & Scale	Hardware Safety & Wear Risk	Dominant Failure Mode & Interceptor
Pretrain & Post-Train	Starts from a generalist checkpoint's broad competence and adapts it to the task with demonstrations or reward	A base checkpoint trained elsewhere, task data, and fine-tuning compute	Task data needs shrink when the base already represents related skills	Supervised post-training adds no exploration; RL post-training adds the exposure of its own rollouts	Borrowed competence mistaken for evidence, and forgetting. Support comes only from the task data join; regression tests cover what post-training may disturb.

6.2.1 The latch task as a reward problem

A reward function that pays a policy for opening the cage door and penalizes each control step for elapsed time without charging for contact force will train the arm to slam into the strike plate, because every millisecond shaved off the approach increases the episode's score. Even behavioral cloning, which uses no reward during training, has its failures measured in the quantities that a reward problem defines: the states a policy visits, the return it earns over an episode, and the value of one action against another. Posing the latch mechanism as a discrete-time optimal control problem makes those quantities concrete and shows how an omitted force penalty or a discount factor mis-scaled against the control rate distorts learned behavior.

A **Markov decision process** (MDP)[2] describes a task by its state s_t, its action $\mathbf{a}_t$, a transition $p(s_{t+1} \mid s_t, \mathbf{a}_t)$ that the physics supplies, and a scalar reward $r(s_t, \mathbf{a}_t)$ that the engineer writes. For the latch, the state holds the gripper pose, the plate position, and the measured contact force. The action is the setpoint the chunk policy emits every 25 ms. An episode is one opening, 8 s of autonomous contact-phase motion, or 320 decisions. A reward that states what the task values and what it must avoid is

$$r_t = r_{\text{open}}\,\mathbb{1}[\text{door open at } t] - \lambda_f \max\left(0, f_t - F_{\text{trip}}\right) - c_{\text{time}}, \tag{6.1}$$

where F_{trip} is the 15 N contact tripwire, the force threshold that triggers the machine's qualified collection response (set below the latch's damage limit), λ_f is a weight in reward per newton of excess force, and c_{time} charges each step for slowness. The weights make the terms commensurable, and choosing them is a design decision that the policy manifest records. Dropping the force term, as the simulator teacher of `latch-sim-02` did, leaves only the time term to shape the approach, and the time term rewards every millisecond an aggressive plunge saves. On hardware the success indicator must itself be measured, by a latch sensor or a learned classifier, and every error in that measurement is a surface the optimizer can exploit. When engineers introduce auxiliary rewards to guide exploration toward that success indicator, adding a potential difference $\gamma\Phi(s') - \Phi(s)$ provably leaves the optimal policy unchanged. Without further assumptions about the task, it is the only added term that does (Ng et al. 1999). Any other shaping term can change what is optimal.

A **policy** $\pi_\theta(\mathbf{a} \mid s)$ is a distribution over actions, and its **return** from step t is $G_t = \sum_{j \geq 0} \gamma^j r_{t+j}$ with discount factor $\gamma \in [0, 1)$. Because γ applies once per decision, its horizon is measured in control steps, not seconds. At $\gamma = 0.99$ and one decision every 25 ms, the door-open reward earned at the end of an opening reaches the first decision with weight 0.04, and the effective horizon $1/(1-\gamma)$ of 100 steps spans only 2.5 s. For the terminal success to keep half its weight across the whole opening, γ must be at least 0.9978. Changing the setpoint rate without re-deriving γ therefore changes the task horizon in seconds. Training a policy means choosing θ to maximize the **objective** $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$, the return expected over whole episodes that start from the task's initial states. It is a return to maximize, where the $J(\pi)$ of equation 5.1 was an expected cost to minimize.

Two expectations of the return organize every reinforcement learning method. The **state value** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ is the return expected from s when following π. The **action value** $Q^\pi(s, \mathbf{a}) =$

[2] **Markov Decision Process**: Bellman (1957) formulated sequential decisions under uncertainty with the dynamic-programming recursion that equation 6.2 restates, and Howard (1960) supplied policy iteration to solve them. The formalism came from operations research and optimal control decades before reinforcement learning adopted it, which is why the cost-to-go translation below is an identity rather than an analogy, and why a control engineer's tools for that recursion apply to a learned policy's value function.

$\mathbb{E}_\pi[G_t \mid s_t = s, \mathbf{a}_t = \mathbf{a}]$ is the return expected after taking $\mathbf{a}$ first and following π thereafter. Both obey the one-step **Bellman relation**

$$Q^\pi(s,\mathbf{a}) = r(s,\mathbf{a}) + \gamma\, \mathbb{E}_{s'\sim p,\ \mathbf{a}'\sim\pi}[Q^\pi(s',\mathbf{a}')], \qquad V^\pi(s) = \mathbb{E}_{\mathbf{a}\sim\pi}[Q^\pi(s,\mathbf{a})]. \tag{6.2}$$

The **advantage** $A^\pi(s,\mathbf{a}) = Q^\pi(s,\mathbf{a}) - V^\pi(s)$ measures whether an action does better or worse than the policy's usual choice from that state. At the plate, an approach has positive advantage only if its expected return exceeds the policy's usual choice. A force penalty can make a plunge less attractive, but the sign still depends on the reward weights, completion probability, and baseline policy.

Readers from control have met these objects under other names. With stage cost $\ell = -r$, the value is the negated **cost-to-go**, $C(s) = -V(s)$, and equation 6.2 is the dynamic-programming recursion. (Control texts write the cost-to-go as J; this chapter reserves $J(\theta)$ for the expected return a learner maximizes.) For linear dynamics in the plant state $\mathbf{x}$ and quadratic cost, that recursion has the closed-form solution $C(\mathbf{x}) = \mathbf{x}^\top \mathbf{P}\mathbf{x}$ that the linear-quadratic regulator computes from a Riccati equation (section B.2.5). Reinforcement learning keeps the recursion and changes the estimation problem: the dynamics are unknown, so V and Q are fitted from sampled transitions instead of solved from a model.

These quantities organize policy learning along two axes, what is estimated and where the data comes from. Fitting Q over an immutable logged archive to select high-value actions is **offline reinforcement learning** (section 6.2.7). Generating fresh rollouts from π_θ, using each batch for a few updates, and shifting probability toward positive-advantage actions is on-policy **policy-gradient** optimization (section 6.3). Between the two, value-based methods also run online and off-policy. The machine keeps collecting experience, but each transition enters a replay buffer and is reused across many value updates, which is how the demonstration-seeded methods of section 6.3.1 learn in hours. The value-fitting and policy-gradient derivations behind these approaches appear in section A.6. Before paying the reward-design or physical trial cost of those methods, however, the simplest training regime bypasses value estimation altogether and fits a policy directly to the expert setpoints the collection campaign already recorded.

6.2.2 Behavioral cloning and its action targets

Instead of designing a contact reward that an optimizer can game, `latch-bc-01` learns the nominal door opening directly from recorded human demonstrations, following the supervised imitation paradigm that ALVINN first demonstrated for road following (Pomerleau 1989). **Behavioral cloning** frames policy training as supervised regression or classification over a collected dataset of expert state-action pairs $\mathcal{D}_{\text{demo}} = \{(\mathbf{o}_t, \mathbf{a}_t)\}$. On a physical machine, however, the four-stream action taps record both the operator's mapped setpoints and the setpoints the permission path actually admitted:

$$\mathcal{D}_{\text{intent}} = \{(\mathbf{o}_t, \mathbf{u}_{\text{nom},t:t+H-1})\} \quad \text{vs.} \quad \mathcal{D}_{\text{verified}} = \{(\mathbf{o}_t, \mathbf{u}^*_{t:t+H-1})\}$$

The mapped setpoint $\mathbf{u}_{\text{nom}}$ is the record at the mapped-command tap a_{cmd}, and the admitted setpoint $\mathbf{u}^*$ is the record at the enforced-command tap a_{enf}. Section 5.5.1 develops what each tap teaches a policy. In brief, cloning admitted setpoints couples the weights to one permission path's intervention profile and tuning. An uncurated policy can then learn to steer straight into the safety clamps and rely on the hardware filter to bend its commands back onto the latch. Cloning mapped setpoints instead keeps intent separate from enforcement, at the price of more frequent runtime interventions. Neither tap yields an expert target without curation. The latch campaign took the mapped command as its target wherever the permission path admitted it unchanged, so `latch-bc-01` clones $\mathbf{u}_{\text{nom}}$ on those steps and treats every modified command as an attributed intervention.

Whichever tap supplies the target, the training loss compares the proposed setpoints with the recorded ones, over a single step or over the H-step sequence that section 6.2.5 calls a chunk. Because this loss penalizes only the discrepancy between the commanded action and the recorded demonstration at observed states, the training objective does not evaluate task completion, terminal cost, or the stability of the closed-loop dynamical system. It fits actions to observations only, and that point-by-point fit breaks down at a single observation whenever demonstrators solve the same physical geometry in more than one valid way.

6.2.3 Mode averaging when demonstrations split

A mobile manipulator trained with least-squares regression on demonstrations that pass left of a central structural column half the time and right of it the other half will steer straight down the center line into the column. Each human teleoperator trajectory in the recorded archive $\mathcal{D}_{\text{demo}}$ is dynamically valid and collision-free, and not a single demonstration drives into the obstacle, yet a deterministic point-regression policy proposes the one path that every demonstrator avoided.

Under empirical mean squared error (ℓ_2) loss, a deterministic behavioral cloning network cannot represent both modes for the same observation $\mathbf{o}_t$ and instead optimizes:

$$\mathcal{L}_{\text{MSE}}(\theta) = \mathbb{E}_{(\mathbf{o}_t,\mathbf{a})\sim\mathcal{D}_{\text{demo}}}\left[\|\pi_\theta(\mathbf{o}_t) - \mathbf{a}\|_2^2\right].$$

The theoretical minimizer of this objective is the conditional expectation of the action distribution (derived formally in section A.5.1):

$$\pi^*(\mathbf{o}_t) = \arg\min_{\hat{\mathbf{a}}} \int \|\hat{\mathbf{a}} - \mathbf{a}\|_2^2 \, p_{\text{demo}}(\mathbf{a} \mid \mathbf{o}_t)\, d\mathbf{a} = \mathbb{E}[\mathbf{a} \mid \mathbf{o}_t] = \int \mathbf{a}\, p_{\text{demo}}(\mathbf{a} \mid \mathbf{o}_t)\, d\mathbf{a}. \tag{6.3}$$

In conventional computer vision, predicting the conditional mean of ambiguous pixel targets generates blurry, averaged textures; in natural language processing, it produces diffused semantic embeddings. In Physical AI, however, the conditional expectation of two collision-free evasion commands (such as $\mathbf{a}_{\text{left}} = [v, +\omega_z]^\top$ and $\mathbf{a}_{\text{right}} = [v, -\omega_z]^\top$, where v is forward linear speed and ω_z is angular yaw rate) evaluates to:

$$\mathbb{E}[\mathbf{a} \mid \mathbf{o}_t] = \frac{1}{2}\mathbf{a}_{\text{left}} + \frac{1}{2}\mathbf{a}_{\text{right}} = [v, 0.0]^\top.$$

The regression mean proposes forward motion with zero lateral evasion. If admitted unchanged, that proposal heads straight into the central obstacle.[3]

This invalid proposal is not caused by an optimization pathology, insufficient model capacity, or dataset scarcity. It is the exact minimizer of a point-regression loss over a multimodal distribution. Viewed as a function of the proposed action, the expected loss is a bowl with a single bottom at the mean, and that bottom sits in the zero-density gap between the two demonstrated modes ($p_{\text{demo}}(\mathbb{E}[\mathbf{a} \mid \mathbf{o}_t] \mid \mathbf{o}_t) \approx 0$). The statement concerns the optimal output, not whether optimizing the network's weights is convex.

> **Definition 6.1: Mode averaging**
>
> ***Mode averaging*** is the failure in which a policy trained with a point-regression loss on demonstrations that contain several valid actions for one observation proposes their conditional mean, an action that lies between the demonstrated modes and that no demonstrator chose.
>
> 1. **Significance**: More data, capacity, or training moves the output toward the mean, not away from it. Two equally weighted evasions at $\pm\omega_z$ average to zero yaw, a proposal aimed at the obstacle both demonstrations avoided, and it reaches the permission path as a smooth, confident command whose consequence cannot be undone once admitted.
> 2. **Distinction**: Mode averaging concerns the action chosen at one observation; compounding error concerns the states that successive actions reach. A multimodal decoder removes the first and does nothing for the second, which needs recovery experience.
> 3. **Common pitfall**: Reading a low validation loss as evidence against averaging. The mean minimizes squared error on held-out demonstrations drawn from the same modes, so a regression head can score best on its split and still propose the invalid middle action at every bifurcation.

Changing the regression penalty does not by itself represent alternative actions. Mean absolute error (ℓ_1) admits conditional medians, while Huber loss limits the influence of large residuals, so neither guarantees a unique, coherent route through a multimodal demonstration distribution.

[3] **Jensen's Inequality**: The averaged action fails even where it happens to be admissible, because a nonlinear plant does not map the mean action to the mean outcome. In general $\mathbb{E}[g(\mathbf{a})] \neq g(\mathbb{E}[\mathbf{a}])$, and for a convex g Jensen's inequality fixes the sign of the gap. Contact and friction are the most nonlinear parts of a manipulator's plant, so a regression mean that looks like a compromise between two good demonstrations can produce a contact state that neither demonstration visited.

Instead, the decoder must represent the alternatives, and execution must maintain a consistent choice across successive observations.

Several decoder families represent these alternatives at different computational costs. **Mixture Density Networks (MDNs)**[4] parameterize a Gaussian Mixture Model (GMM): $p_{\text{demo}}(\mathbf{a} \mid \mathbf{o}_t) = \sum_{m=1}^{M} \pi_m(\mathbf{o}_t)\mathcal{N}\left(\mathbf{a} \mid \mu_m(\mathbf{o}_t), \Sigma_m(\mathbf{o}_t)\right)$. Here π_m is the weight of component m (not the policy), and μ_m and Σ_m are its mean (not a friction coefficient) and covariance. They produce mixture parameters in one forward pass, but training must prevent covariance collapse ($\Sigma_m \to \mathbf{0}$) and loss of less frequent modes, which makes the component count and covariance representation consequential for high-dimensional chunks ($d_{\text{chunk}} = d_a \times H$). Energy-based models instead score each candidate with a learned energy and must search for a low-energy action, so their iteration budget, rather than a head of fixed size, must fit the proposal deadline.

Generative architectures, including diffusion models (Ho et al. 2020) and flow matching (Lipman et al. 2022), offer another way to represent $p_{\text{demo}}(\mathbf{a} \mid \mathbf{o}_t)$ by training a model to refine a noisy action candidate toward the learned distribution rather than returning its conditional mean. Sampling can then select a coherent demonstration mode ($\mathbf{a}_{\text{left}}$ or $\mathbf{a}_{\text{right}}$). Although representing multiple modes resolves the averaging problem, a generated candidate can still violate clearance or actuator constraints, so the permission path must check it before execution.

A mode boundary in a single-step Markov policy can also produce high-frequency temporal chattering. In a 50–100 Hz feedback loop, microscopic sensor noise drives consecutive observations back and forth across the mode boundary, causing a single-step controller mapping $\mathbf{o}_t \to \mathbf{a}_t$ to alternate between opposing actuator efforts (for example, +10 N·m and −10 N·m). This alternation excites drivetrain resonances, accelerates gear tooth fatigue, and drives motor winding temperatures toward their thermal limit. Even when a multimodal decoder commits to one route and avoids chattering across the boundary, however, a second imitation failure emerges as soon as a small tracking error pushes the robot outside the states the demonstrators visited.

[4] **Mixture Density Network**: Bishop (1994) proposed it for inverse problems, whose inputs admit several valid outputs, and demonstrated it on the inverse kinematics of a robot arm, where one hand position has two joint solutions and least-squares regression returns their invalid average. The mobile manipulator's two routes around the column are the same problem, met three decades later in action space, which is why the remedy is still a change of output distribution rather than a larger network.

6.2.4 Compounding errors and recovery experience

A policy can choose a coherent route around the column and still drift into states its demonstrations never covered. Mode averaging concerns the action selected at one observation; compounding error concerns the observations created by successive actions. Keeping these failures separate prevents a richer action decoder from being mistaken for recovery experience. Cloning fits actions only at the states the demonstrators visited, whereas deployment visits the states the policy's own actions create, and a fixed archive holds no example of the states the policy's execution errors reach. In continuous physical state spaces, this decoupling produces **compounding covariate shift**, which is the second law (endogenous data, principle 2) acting on a fixed archive.

Section 5.3 bounds the price of this shift: fixed-archive cloning admits a worst-case imitation cost that grows as $S^2\epsilon$ over an S-step horizon (equation 5.1). That bound counts accumulated task cost, not displacement, and the physical drift behind it depends on what the action commands and on the feedback below the policy. A persistent error in a commanded velocity integrates once into position, so the drift grows linearly in time; a persistent error in a commanded torque integrates twice, so it grows quadratically. A velocity-level policy also sits above a joint velocity loop that rejects load disturbances between its decisions, whereas a torque-level policy has no such loop beneath it and must supply every correction itself. Offline validation on demonstration splits measures prediction accuracy on the demonstrated distribution; recovery after displacement requires additional closed-loop evidence.

The regime that supplies that recovery experience is interactive imitation learning, of which Dataset Aggregation (DAgger) (Ross et al. 2011) is the classic form. In DAgger, an expert labels the states the learner itself visits and the policy retrains on the aggregate. Section 5.8 develops DAgger, with its guarantee of a cost that grows linearly in the horizon (equation 5.8) and the recoverability constant u on which that guarantee depends. The constant bounds how much a single deviation can raise the expert's remaining cost, so it is an assumption about the plant, not a property of the learner. On physical hardware, this theoretical recoverability depends directly on joint torque authority and contact friction limits.

On hardware, collecting those labels requires a recoverable rollout and an expert who has enough information to choose a correction. Because a camera log may support a path label while leaving

contact force or joint torque ambiguous, live supervision supplies richer feedback, yet the machine continues moving during human reaction and control handoff. Consequently, intervention must begin within the available stopping margin, and the handoff must avoid discontinuous setpoints.

Expert availability also does not imply physical recoverability. The machine may lack enough actuator effort or frictional grip to return from a deviation. The simplified force balance $\mathbf{M}\ddot{\mathbf{q}} = \tau + \tau_{\text{ext}}$ is Newton's second law written per joint. Here $\mathbf{q}$ denotes joint positions, $\mathbf{M}$ inertia, τ actuator torque, and τ_{ext} the torque that contact forces exert about the joints. Other forces are omitted for clarity, and section 11.4.1 develops the full equation. Actuator limits require $|\tau_i| \le \tau_{\max}$ on every joint axis, while available friction bounds tangential force by $\|\mathbf{f}_{k,\parallel}\| \le \mu f_{k,\perp}$, with normal force $f_{k,\perp}$ and friction coefficient μ. These constraints can leave no action that restores the task, and then u is unbounded and DAgger's linear guarantee no longer applies. Interactive aggregation is therefore most useful where learner deviations can be contained, labeled, and repeated within the operator-time and hardware-exposure budget. The state space divides into three regions (figure 6.1). Cloning trains in the tube of demonstrated states. Just outside it lies a band where an expert label on a learner-visited state still restores the task. Beyond the band is an unrecoverable region, where actuator and friction limits leave no restoring action and every further label is vacuous. Interactive aggregation buys experience only in the middle band.

↪ The bumpless transfer that keeps a human takeover from stepping the command is developed in section 14.4.

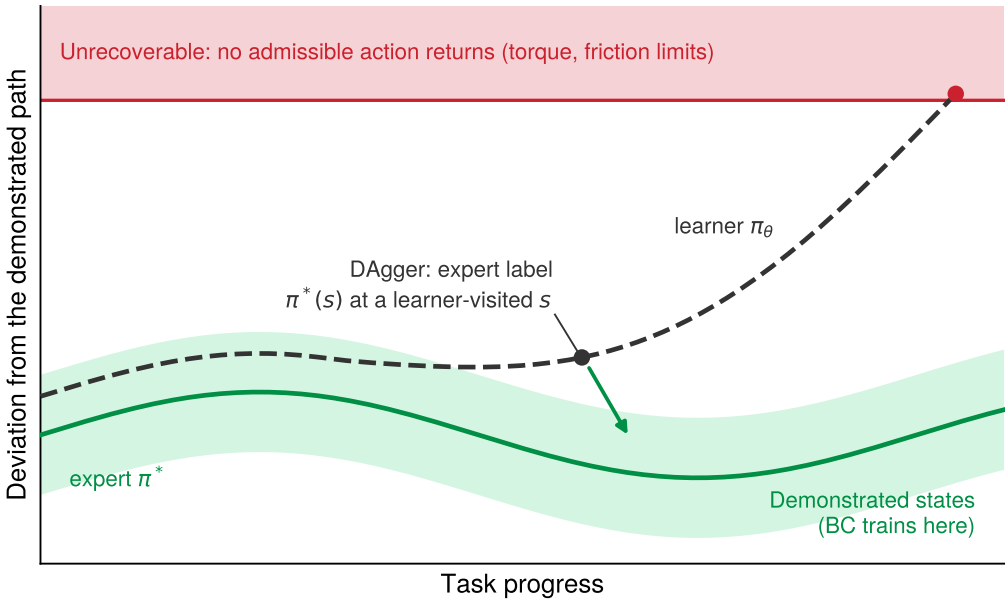


Figure 6.1: Drift out of the demonstrated states: Behavioral cloning trains only inside the shaded tube the expert visited. Once the learner leaves it, each action is taken from an unfamiliar state and the error compounds. DAgger labels a learner-visited state outside the tube, which supplies recovery experience while a return is still possible. Past the red boundary, actuator and friction limits leave no action that restores the task, so no label can help. The sketch is schematic; its axes carry no units.

For the latch, these distinctions select a concrete next training step. The stipulated follow-up campaign in section 5.8 supplies `latch-recovery-02`, a versioned archive of expert recovery from the positively displaced plate at the guarded approach. The recovery candidate `latch-bc-recovery-02` trains on the original nominal demonstrations and the stabilized, reviewed segment of each correction; the enforcer's blend and the operator's re-orientation after takeover stay masked. Learner commands that preceded takeover remain failure evidence unless an expert supplied replacement labels; they do not become imitation targets merely because the episode eventually succeeded. A multimodal chunk decoder can preserve a coherent back-off and re-seating sequence, while the additional demonstrations supply the recovery states that changing the decoder alone could not provide.

↪ Causal intervention masking, which admits only reviewed recovery segments as imitation targets, is enforced in section 14.8.3.

The training decision is complete, but the improvement claim remains to be tested. Entire collection sessions are held out for nominal and recovery checks, so neighboring frames from one run do not appear on both sides of the split. Training diagnostics check whether the candidate represents the demonstrated correction; qualification must then test whether it completes the door task under the declared perturbation without regressing nominal behavior. The candidate carries that request to section 7.8.1, including task completion, cycle time, proposer intervention, and protective outcomes as separate measures. This is a stipulated engineering branch, not a report of collected data or measured improvement. The negative displacement and faster approach remain unsupported. Even on the supported path, emitting setpoints one tick at a time leaves the arm vulnerable to the mode-boundary chattering of section 6.2.3 unless the decoder predicts a multi-step action sequence in one pass.

6.2.5 Action chunking and multi-step trajectory prediction

Action chunking (definition 3.2) suppresses both high-frequency actuator chattering and single-step mode oscillation by predicting an entire sequence of future actions simultaneously rather than re-querying the decoder at every 25 ms control tick. Instead of regressing a single action $\mathbf{a}_t$ from current observation $\mathbf{o}_t$, a chunking policy predicts a setpoint sequence $\mathbf{A}_t \in \mathbb{R}^{H \times d_a}$ across a prediction horizon of H future timesteps and d_a action dimensions. The execution path then interpolates this sequence into continuous motion and checks it for feasibility. In supervised behavioral cloning, the policy minimizes a multi-step trajectory loss over demonstrated trajectories:

$$\mathcal{L}_{\text{chunk}}(\theta) = \mathbb{E}_{(\mathbf{o}_t, \mathbf{A}_t^*) \sim \mathcal{D}_{\text{demo}}} \left[\sum_{i=0}^{H-1} \ell_\delta \left(\pi_\theta(\mathbf{o}_t)_i - \mathbf{a}_{t+i}^* \right) \right],$$

where $\mathbf{A}_t^* = [\mathbf{a}_t^*, \ldots, \mathbf{a}_{t+H-1}^*]^\top$ represents the recorded demonstration chunk and $\pi_\theta(\mathbf{o}_t)_i$ is the i-th predicted future setpoint. Actions are expressed in declared normalized coordinates. For a vector residual $\mathbf{r}$, the chunk objective uses the componentwise sum $\ell_\delta(\mathbf{r}) = \sum_{j=1}^{d_a} \ell_\delta(e_j)$, where e_j is the matching component of $\mathbf{r}$, with equal component weights. The scalar Huber penalty below acts on each normalized prediction error e_j, not on the reward of section 6.2.1:

$$\ell_\delta(e) = \begin{cases} \frac{1}{2} e^2 & \text{for } |e| \le \delta, \\ \delta \left(|e| - \frac{1}{2} \delta \right) & \text{otherwise.} \end{cases}$$

(ACT (Zhao et al. 2023), the architecture of both latch policies, uses an ℓ_1 reconstruction penalty in place of ℓ_δ and adds a conditional variational autoencoder (CVAE) term $\beta \mathcal{D}_{\text{KL}}(q(\mathbf{z} \mid \mathbf{A}_t^*, \mathbf{o}_t) \,\|\, \mathcal{N}(\mathbf{z}))$ against a standard normal prior, which lets a latent variable absorb differences in style across demonstrations.)

Because teleoperated setpoints can contain outliers caused by tremor, tracking errors, or dropouts, squared loss gives a single large residual disproportionate influence over the fit. Huber loss[5] is quadratic for small residuals ($|e| \le \delta$) and linear beyond them, so its gradient saturates at magnitude δ and no single outlier can dominate the fit. Even so, the threshold acts on prediction residuals rather than on the velocity, acceleration, or jerk of the resulting trajectory. Huber loss alone therefore establishes neither smooth motion nor actuator feasibility. Those properties require explicit constraints and checks on the proposed chunk.

Predicting H future actions at once creates opportunities for more coherent execution, provided training and runtime checks preserve them:

1. **Temporal and Dynamic Consistency**: Jointly predicting adjacent actions lets the model learn their relationships. The resulting sequence still requires checks for abrupt changes and actuator limits.
2. **Preserving Modal Commitment**: A chunk can represent one coherent choice, such as passing left of the column, across adjacent actions. Predicting a sequence does not itself ensure that the decoder selects a single valid mode.
3. **Receding-Horizon Replanning and Temporal Ensembling**: The deployed policy queries a fresh chunk every H_{exec} steps ($H_{\text{exec}} < H$), permitting feedback correction before the prediction

[5] **Huber Loss**: Huber (1964) introduced it as an estimator of location that tolerates a small fraction of grossly wrong observations, quadratic near the center and linear in the tails, and showed that it minimizes the worst-case variance over such contaminated distributions. Teleoperation tremor and dropouts are exactly that contamination, and δ sets, in the declared normalized action coordinates, where an error stops counting as noise and starts counting as a gross error.

horizon ends. Blending overlapping predictions is optional and must preserve a coherent mode and feasible seam.

The modal commitment that the chunk objective buys in training can be undone at execution. If stochastic sampling selects the left route at one query and the right route at the next, blending the overlapping chunks across the seam averages the two routes and resurrects the mode averaging that action chunking was chosen to eliminate (Black, Galliker, et al. 2025).

↩ Temporal ensembling of overlapping chunks, and real-time chunking, which freezes the committed prefix and generates the rest of the next chunk to agree with it, are established in section 3.5.1.

Chunking changes how often the policy replans, but does not supply recovery states absent from the demonstrations. Executing H_{exec} actions between queries reduces the number of query boundaries to $S_{\text{eff}} = S/H_{\text{exec}}$ while leaving disturbances within each execution window to the lower-level feedback system. This count alone does not convert the imitation-learning cost bound $\mathcal{O}(S^2\epsilon)$ into a task-survival curve, the probability that an episode reaches the goal without entering an unrecoverable state, plotted against the horizon. ACT (Zhao et al. 2023) and receding-horizon diffusion (Chi et al. 2025) change temporal execution; DAgger (Ross et al. 2011) changes the training distribution by obtaining labels on learner-visited states. These are separate ways to improve behavior, and their closed-loop benefits must be measured on the task. Neither the drift sketch of figure 6.1 nor the imitation-learning cost bound establishes a task-survival probability or ranks learning algorithms.

Increasing dataset size, model capacity, or tuning can reduce the one-step error rate on evaluated observations and therefore lower the chance of a first policy error during a fixed horizon. That improvement does not characterize what happens after an error. An external disturbance, uncalibrated payload, or voltage sag can also move the machine into weakly supported states without a policy disagreement. Recovery data, measured dynamics, and independent runtime limits determine whether either event remains within the physical envelope.

Drift off the demonstrated states raises no alarm of its own, so detecting it before physical contact cannot rely on the offline training loss. A feature-distance or uncertainty monitor can flag unfamiliar observations, but its threshold needs validation against missed detections and false alarms. For the latch, an independently measured plate position and approach condition provide a more direct check of the declared cells. A purely geometric monitor is weaker. One that accepts any state inside the convex hull of the demonstrated coordinates, a convex stand-in for the support hull of section 5.7, still admits intermediate states that no demonstration visited. Nothing guarantees that those states are physically admissible or dynamically stable. Predicting an entire H-step chunk in a multimodal decoder solves the single-step averaging problem, but only if the hardware can finish evaluating that decoder before the next chunk is due.

6.2.6 Decoder latency and proposal deadlines

Selecting a coherent route around an obstacle or into a latch does no good if the decoder finishes computing the trajectory after the actuator deadline has already passed. A decoder converts the model's representation into an action or action sequence; iterative decoders repeatedly refine a candidate before returning it. Their equations describe how a candidate is generated, while the execution decision depends on the wall-clock time those dependent model evaluations consume. The central comparison is between measured proposal latency and the deadline of the buffer that feeds the permission path.

The decoders that fix mode averaging buy that fix with time. A generative head escapes the regression mean by sampling one demonstrated route, but it reaches its sample through a chain of dependent refinements, and each refinement is another model evaluation that must finish before the next proposal is due (figure 6.2). The proposal then still has to cross from the application processor to the safety microcontroller's own buffer before the permission path can check it, so the decoder's time comes out of a budget it shares with transport and admission.

↪ The placement of the multi-rate proposal and permission paths on the application processor and the safety microcontroller is developed in section 13.2.

The decoder families themselves, from binned and compressed action tokens to parallel heads and iterative action experts, are introduced in section 3.3. What matters for training is their dependency structure. A diffusion policy (Chi et al. 2025) or a flow-matching expert (Lipman et al. 2022; Black et al. 2024) refines a candidate chunk through evaluations that each consume the previous candidate. An autoregressive vision-language-action policy (Kim et al. 2024) generates its action tokens one after another. In each case the dependent evaluations run in sequence. None of these constructions guarantees collision-free or actuator-feasible output, and the diffusion and flow-matching objectives

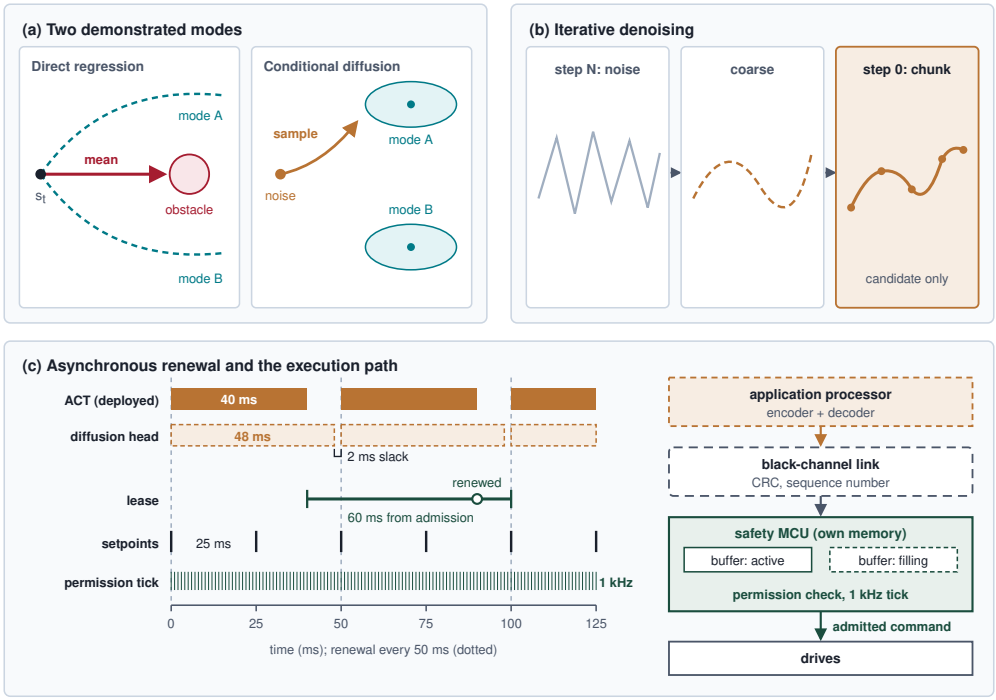


Figure 6.2: Generative policy architectures and action chunk denoising: (a) Sampling represents alternative demonstrated routes instead of their direct-regression mean. (b) Iterative refinement produces a candidate action chunk whose clearance and actuator feasibility still require checking. (c) On the machine's schedule, proposals renew every 50 ms while setpoints play every 25 ms, and the bars compare the deployed ACT decoder with an illustrative diffusion head that leaves far less of each renewal period to spare. An admitted chunk crosses the black-channel link into the safety MCU's own double buffer and is checked on every permission tick before the drives act. Buffering alone guarantees neither freshness nor freedom from starvation.

are developed in section A.5.2. Some post-training recipes remove the token sequence from the run-time path altogether by training a separate action expert (section 6.2.8). For the latch decision, the relevant consequence is how many dependent evaluations yield adequate action quality. The quantity that decides the execution arrangement is complete proposal latency, perception included, measured against the actual proposal deadline.

The number and dependency of decoder evaluations constrain proposal latency; the implementation and loaded hardware determine its measured value. The total wall-clock duration of a single policy inference cycle t_{infer} is the sum of perceptual feature extraction and iterative generative trajectory sampling:

$$t_{\text{infer}} = t_{\text{enc}}(\mathbf{o}_t) + \sum_{k=1}^{N_{\text{eval}}} t_{\text{pass}}\left(\mathbf{A}^k, k\right) \leq T_{\text{deadline}}, \tag{6.4}$$

Here t_{enc} is the visual backbone and proprioceptive encoding latency, and N_{eval} is the number of dependent decoder evaluations (denoising or ODE solver steps). The term t_{pass} is the latency of one forward pass of the action head (one denoising or solver step, not the whole-model evaluation time t_{infer} of equation 3.4). The chunk $\mathbf{A}^k$ is the candidate that the k-th evaluation refines, and T_{deadline} is the real-time execution deadline.

The encoder runs once per proposal, whereas each dependent decoder evaluation adds work. Section 3.5.1 prices that work for each decoder family on the machine's processor, where re-reading a large action head on every evaluation can dominate it. Training decides the two quantities those prices depend on, the head and its evaluation count N_{eval}, so the question here is whether a head chosen for its multimodal fit also meets the latch's deadlines.

The latch policies avoid repeated evaluations: `latch-bc-01` decodes each chunk with the machine's 80M-parameter ACT network in one pass, within the 40 ms p99 inference target. Consider

instead replacing its decoder with an illustrative diffusion head, the kind that multimodal latch demonstrations might motivate. The head is sampled in $N_{\text{eval}} = 16$ steps by the denoising diffusion implicit model (DDIM) sampler, a deterministic sampler whose step count sets the number of dependent evaluations. Stipulate a visual encoding latency of $t_{\text{enc}} = 8$ ms and a total inference duration $t_{\text{infer}} \approx 48$ ms once all evaluation work is included. These stipulated inputs define the comparison below. A measurement of the real head replaces them, and that measurement can only sit at or above the weight-streaming floor of section 3.4.1.

Two deadlines apply, the setpoint step for a decoder that must supply every tick itself and the renewal period for one that runs beside an admitted prefix. The lease rule of section 4.9.1 adds an allowance for renewal jitter. That allowance belongs to the rule's compute side, the renewal period plus jitter that the lease must exceed. Transport across the black-channel link, the permission tick, and the age of the evidence at admission are charged on its other side, against the stopping budget. The slack computed below is therefore the decoder's alone. A slower decoder pays on that side too, because its inference time ages the evidence it is admitted on.

Only the 4-step flow decoder beats the setpoint; DDIM leaves 2 ms of its renewal.

Napkin Math 6.1: The renewal slack

Problem: Can the stipulated diffusion head replace the one-pass ACT decoder of `latch-bc-01`, either synchronously or asynchronously, and how much of a stretched renewal can each decoder absorb?

Variables:

- Setpoint step $\Delta t_{\text{step}} = 25$ ms (chosen) and renewal period $T_{\text{chunk}} = 50$ ms (derived, two steps)
- ACT inference 40 ms at p99, including vision (illustrative target)
- Diffusion head: $N_{\text{eval}} = 16$, $t_{\text{enc}} = 8$ ms, $t_{\text{infer}} = 48$ ms (illustrative, stipulated)
- Flow-matching candidate: $N_{\text{eval}} = 4$ at the same per-evaluation cost (illustrative)
- Renewal-jitter allowance 5 ms (chosen, lease rule)

Math:

1. **Per-evaluation cost** (equation 6.4): $t_{\text{pass}} = (48\text{ ms} - 8\text{ ms})/16 = 2.5$ ms.
2. **Synchronous step**: The diffusion head's 48 ms exceeds the 25 ms step. The flow candidate needs $8\text{ ms} + 4 \times 2.5\text{ ms} = 18$ ms and fits it.
3. **Renewal slack**: ACT leaves $50\text{ ms} - 40\text{ ms} = 10$ ms; the diffusion head leaves $50\text{ ms} - 48\text{ ms} = 2$ ms.
4. **Stretched renewal**: $48\text{ ms} + 5\text{ ms} - 50\text{ ms} = 3$ ms past the renewal for the diffusion head, while ACT ends at $40\text{ ms} + 5$ ms, inside the period.

Result: The diffusion head can run only asynchronously, and a single jitter-sized stretch carries it past its renewal.

Systems insight: Renewal slack, rather than mean inference latency, dictates real-time schedulability. When a decoder's slack drops below the jitter allowance, a single tail-latency stretch carries it past its renewal, and the active prefix keeps playing on evidence that grows older with every tick. The lease contract, not the neural decoder, then sets the hard safety bound on how long an admitted prefix may continue actuating the arm against aging sensor evidence.

The notebook separates sampler choice from execution authority. The flow candidate becomes a viable synchronous choice only if action quality, scheduling, transfer, and admission also fit, and a faster decoder does not remove the permission check. Its evaluation count is the quantity to compare, not its step count. A higher-order solver can use fewer integration steps while performing more network evaluations; a standard fourth-order Runge-Kutta (RK4) solver evaluates four stages per step, so compare that total work with the single-stage alternative before choosing it. Step distillation, which trains a sampler to reach the same chunk in fewer evaluations, or a different solver is useful when it reduces measured latency without losing the recovery behavior the latch needs.

The slower candidate can still generate chunks asynchronously against the 50 ms renewal cadence while the execution path plays an admitted prefix, provided loaded timing evidence confirms the schedule. Because generating an action chunk supplies candidate trajectory setpoints without conferring authorization to execute that entire window unchecked, the lease selected in section 4.4 bounds active permission separately from renewal cadence and evidence age. Execution renews only when fresh sensor evidence arrives paired with a valid replacement proposal. Stamping a new timestamp header over stale sensor buffers therefore cannot extend execution authority. An invalid new proposal leaves an active prefix in place only while that prefix's own state, evidence, and lease remain valid. Once the lease expires, the execution path invokes a state-dependent, physically qualified fallback rather than an unspecified emergency stop.

↪ The ladder of resident protective responses from which that fallback is selected is developed in section 12.9.

On the renewal schedule the two decoders differ in slack, not in mean latency (figure 6.3). When a stretched diffusion run misses its renewal boundary, nothing in the decoder notices; the chunk lease is the only mechanism that bounds how long the previous prefix keeps playing.

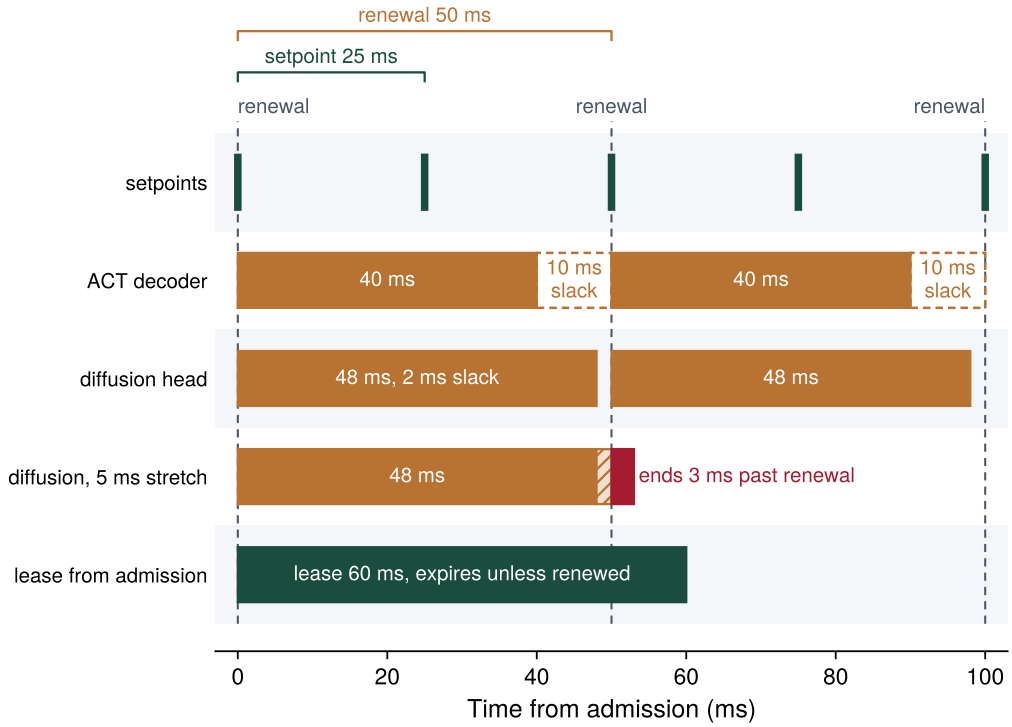


Figure 6.3: Decoder latency on the renewal schedule: Setpoints play every 25 ms and proposals renew every 50 ms. The deployed ACT decoder finishes with 10 ms to spare; the illustrative diffusion head leaves 2 ms, and a 5 ms stretch pushes it 3 ms past the renewal. The lease bounds how long the active prefix may play if a renewal is missed.

Checkpoint 6.1: Mode choice, drift, and decoder deadlines

Cloning fails in two separate ways, and the decoder chosen to repair one of them must still meet the machine's schedule.

Diagnosing the failure

- ☐ **Mode averaging vs. compounding error**: A cloned navigation policy sometimes steers straight at the column. A cloned latch policy, on other runs, drifts until the gripper binds on a sagged door. Can you say which symptom a multimodal decoder addresses, which one needs recovery demonstrations, and why neither remedy supplies the other?

DRAFT

☐ **Recoverability**: Can you explain why DAgger's linear guarantee fails at a state where actuator torque limits and the friction cone leave no restoring action, however many expert labels are collected?

Checking the deadline

☐ **Evaluation budget**: With the stipulated 8 ms encoder and 2.5 ms per evaluation, what is the largest evaluation count that fits the 25 ms setpoint step, and the largest that fits the 50 ms renewal while leaving the 5 ms jitter allowance free?

☐ **Renewal vs. lease**: Can you explain why a new proposal header cannot renew stale observations, and what the execution path must do when the lease expires with no valid replacement?

6.2.7 Offline reinforcement learning and out-of-distribution actions

The decoder budget settles how fast a policy trained on recorded data can propose actions. The next question is what happens when the recorded archive mixes clean door openings with hesitant teleoperation, scripted routines, and aborted trials. Behavioral cloning copies every trajectory in such a log with equal supervised weight, good and bad alike. Running standard Q-learning (Watkins and Dayan 1992) on the same static archive to cherry-pick the high-reward segments triggers a worse failure. The value network invents high returns for actions never tried in the log, and the deployed policy greedily selects ungrounded maneuvers that stall a joint or strike a fixture boundary. **Offline reinforcement learning (Batch RL)** applies the return and value quantities of section 6.2.1 to a fixed transition log $\mathcal{D}_{\text{demo}} = \{(s_t, \mathbf{a}_t, r_t, s_{t+1})\}$ without active hardware exploration. It uses rewards to stitch together high-performing composite trajectories from disjoint, imperfect demonstrations while suppressing **out-of-distribution (OOD) action overestimation** (Kumar et al. 2020).

In standard Q-learning, the algorithm estimates the value of future states by asking its value network to predict the highest possible return it could achieve from the next step, which is equation 6.2 with a maximum over next actions in place of the policy's expectation. That maximum is taken over the entire continuous action space. For actions that were never executed in the logging dataset, the network has received no direct value-fitting target and often outputs erroneously large value estimates due to generalization variance, predicting a high return for an untried physical maneuver. The algorithm greedily selects these spurious peaks, propagating over-optimistic value errors backwards until the deployed policy commands unverified motions. Because the distinction from cloning is the selection rule $\pi(s) = \arg\max_{\mathbf{a}} Q(s, \mathbf{a})$, offline learning needs a way to limit that optimism without obtaining new transitions from the machine.

To reduce value overestimation without querying the physical environment, **Conservative Q-learning (CQL)** adds a conservative penalty to value fitting (Kumar et al. 2020). The objective below has two parts. The bracketed term discourages optimism about unsupported actions, and the squared term fits the reward observed in logged transitions together with estimated future reward. Here $\mathcal{A}$ is the action space, Q the value function being fitted, $\hat{Q}$ a fixed target estimate during this update, and α_{CQL} the penalty weight:

$$\min_{Q} \alpha_{\text{CQL}} \cdot \mathbb{E}_{s \sim \mathcal{D}_{\text{demo}}} \left[\ln \int_{\mathcal{A}} \exp(Q(s, \mathbf{a}))\, d\mathbf{a} - \mathbb{E}_{\mathbf{a} \sim \mathcal{D}_{\text{demo}}} [Q(s, \mathbf{a})] \right] + \frac{1}{2} \mathbb{E}_{(s, \mathbf{a}, r, s') \sim \mathcal{D}_{\text{demo}}} \left[\left(Q(s, \mathbf{a}) - \mathcal{B}^* \hat{Q}(s, \mathbf{a}) \right)^2 \right],$$

where $\mathcal{B}^* \hat{Q}(s, \mathbf{a}) = r(s, \mathbf{a}) + \gamma \max_{\mathbf{a}'} \hat{Q}(s', \mathbf{a}')$ is the empirical Bellman target operator and $\alpha_{\text{CQL}} > 0$ balances value conservatism against Bellman error minimization.[6]

This conservative penalty reshapes value estimates across the continuous action space rather than establishing an absolute physical execution boundary:

1. The logarithmic integral term $\ln \int_{\mathcal{A}} \exp(Q(s, \mathbf{a}))\, d\mathbf{a}$ acts as a soft-maximum that actively pushes *down* on Q-values across the entire action space, aggressively depressing values in unvisited action regions where function approximation variance would otherwise create spurious positive spikes.

[6] **Scope of the CQL Bound**: Under its data-coverage, optimization, and function-approximation assumptions, CQL constructs a conservative estimate whose *expected value for a policy* lower-bounds that policy's true value (Kumar et al. 2020). This is not a pointwise bound for every state-action pair or every dataset, and expected return does not certify physical safety.

2. The dataset expectation $\mathbb{E}_{\mathbf{a}\sim\mathcal{D}_{\text{demo}}}[Q(s,\mathbf{a})]$ pushes *up* on Q-values exclusively for state-action pairs executed during the logging campaign.
3. When $\alpha_{\text{CQL}} = 0$, the conservative penalty disappears and policy extraction uses $\arg\max_{\mathbf{a}} Q(s,\mathbf{a})$ without that correction. With $\alpha_{\text{CQL}} > 0$, unsupported actions face a value penalty, but they are not categorically excluded. Physical action limits and checks of operating conditions remain obligations of the independent permission path.

Implicit Q-learning avoids the overestimation at its source by never evaluating an action outside the dataset; it estimates the value of the best logged actions with an expectile regression (Kostrikov et al. 2021). In robotics, offline learning now serves mostly as a starting point. A policy pretrained on the log is fine-tuned online, and calibrating its value estimates during pretraining keeps the first online updates from unlearning it (Nakamoto et al. 2023). Without this calibration, a conservative critic depresses the estimated values of logged expert actions below the true return scale of the physical environment, so when online rollouts begin, even mediocre exploratory actions receive real environment returns that exceed those depressed offline estimates and cause early policy updates to shift away from the demonstrated behavior until the critic catches up.

Applied to the latch, the retained archive holds successes and labeled tripwire failures, both rewardable, so a conservative objective could reweight nominal behavior toward the cleaner openings. It cannot supply displaced-plate competence. Those cells are known absences among the successes and the failures alike, and no offline objective extracts behavior from states the log never visited. When the task archive itself is sparse, teams often try to borrow that missing competence from a foundation model pretrained across other robots.

6.2.8 Pretrained generalist policies and post-training

A model pretrained on hundreds of thousands of episodes from other robots can move smoothly toward a displaced strike plate even when the target campaign never demonstrated that offset, making it easy to mistake broad visual and motor familiarity for verified support on this arm, door, and contact surface. Whereas `latch-bc-01` starts from randomly initialized weights and learns only from the latch campaign, such a **generalist policy** starts from pooled multi-robot data and is then **post-trained** on demonstrations of the target task. The reference manipulator already carries such a checkpoint as its intent model.

That checkpoint is OpenVLA-7B, a 7B-parameter vision-language-action model trained on 970K robot episodes from the Open X-Embodiment collection (Kim et al. 2024; Open X-Embodiment Collaboration et al. 2024). Chunk-level generalists are built the same way (table 6.3). Post-training may update every weight, a low-rank adapter, or only a new action head, and the base may first be retrained on the target machine's action space.

↩ The intent model, the slower of the machine's two learned proposers, is established in section 3.1.

Pretraining transfers broad visual and motor competence across varied domains. A base network trained on thousands of grasps, door mechanisms, and cluttered tabletop scenes often requires substantially fewer task demonstrations and demonstrates greater resilience to varying illumination, visual distractors, and object geometries than a policy trained from scratch. Pretraining supplies no operational evidence for this plant, however, and the scenario-table join of section 5.7.1 shows exactly where the gap lies. The scenario table for this door on this machine has 6 cells. The pretraining episodes were collected on other arms, fixtures, and grippers under embodiment contracts the table does not describe, so none of them joins to any latch cell. The post-trained candidate therefore inherits only the campaign's own support, 1 cell out of 6 with 5 known absences. It may execute the nominal cell more reliably and may even propose plausible maneuvers at the displaced plate, but those proposals are borrowed competence, not evidence. Its declared ODD, the region the policy manifest of section 6.8 computes from the join and the simulator omissions, gains no cell from pretraining.

Three dependencies follow, each requiring explicit tracking in the policy manifest. First, the base checkpoint is an external dependency whose underlying pretraining data rarely permits cryptographic digest verification, as its data mixture, filtering heuristics, and training run were permanently fixed by the upstream provider. The manifest therefore records the base checkpoint by the cryptographic digest of its weights and flags its pretraining provenance as a known unknown within the `base_checkpoint` field of table 6.4. Second, the base model's native action space rarely aligns with the target machine. A generalist checkpoint emitting normalized end-effector velocity

deltas on an asynchronous clock must be re-headed or mapped into discrete joint setpoints every 25 ms, exposing the pipeline to the normalization fallacy analyzed in section 5.9.1. Third, fine-tuning risks catastrophic forgetting. Gradient updates fitting the latch demonstrations can degrade broader capabilities that the base previously mastered but the target task never exercises. The same hazard returns, on a smaller scale, when a policy is later fine-tuned on the machine itself (section 6.6); here it is magnified across a large, poorly documented behavioral repertoire. Post-training need not stop at imitation. A pretrained policy can also be improved with reinforcement learning on its own experience, in simulation or on the machine, and section 6.6.2 compares those methods by what each one freezes.

The decoder question also carries over, because post-training often changes the head that runs at deployment. A vision-language backbone can be adapted on discrete action tokens while a separate continuous action expert is trained beside it. The expert's gradient is stopped before it reaches the backbone, so the newly initialized expert does not disturb the pretrained weights. This **knowledge insulation** trained faster and generalized better than letting the expert's gradient flow into the backbone (Driess et al. 2025). The run-time decoder is then the one-pass or few-step action expert, not the token sequence. A study of post-training recipes likewise found that action chunking with a plain ℓ_1 regression head matched iterative decoders on its benchmark while decoding far faster (Kim et al. 2025). Either way, the deadline arithmetic of section 6.2.6 governs the post-trained head just as it governs the latch's own.

Supervised behavioral cloning, action chunking, batch offline RL, and generalist post-training all generate action proposals beyond the state-action support of their logging archives. A low training loss or an inflated Q-value does not constitute physical evidence of safety or success. When an archive lacks the recovery trajectories that a physical perturbation demands, the system must generate new closed-loop experience through direct environment interaction.

6.3 Reinforcement Learning and Environment Interaction

When an archive lacks recovery demonstrations for a displaced strike plate, letting the robot discover a recovery on its own by trial and error means paying for every failed attempt in motor winding heat ($E_{\text{thermal}} = \int i_q(t)^2 R\, dt$), gearbox fatigue reversals, jammed latch cams, and human reset labor. Yet when expert corrections are unavailable or a task demands strategies no demonstrator can supply, closed-loop trial is the only way to generate experience in the states the learner's own actions reach. By executing actions on the physical system and observing the resulting state transitions, the policy experiences the consequences of its own exploratory deviations. In online policy search,[7] the agent updates its parameter weights from the scalar advantage estimates accumulated across its own rollouts:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_{0:T}, \mathbf{a}_{0:T} \sim p_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \ln \pi_\theta(\mathbf{a}_t \mid s_t) \hat{A}_t \right],$$

where $\hat{A}_t$, a scalar and not the bold action chunk $\mathbf{A}_t$ of section 6.2.5, estimates the advantage $Q^\pi(s_t, \mathbf{a}_t) - V^\pi(s_t)$ introduced in section 6.2.1. It quantifies whether an exploratory action yielded higher or lower return than the baseline policy expected. Because transitions are drawn from the state occupancy distribution $d_\pi(s)$ induced by the currently deployed policy, closed-loop execution directly exposes the agent to compounding drift and provides immediate evaluative feedback on attempted recoveries. A positive advantage ($\hat{A}_t > 0$) reinforces the exploratory action relative to the baseline, whereas a negative advantage ($\hat{A}_t < 0$) penalizes it. For this feedback to drive convergence, the reward formulation and sensory observations must make the underlying physical failure modes visible. While learning on policy-generated states resolves the covariate shift between expert demonstrators and the learner, it does not immunize the system against distribution shift caused by novel disturbances, unmodeled environment changes, or drifting hardware dynamics.

Because each physical rollout dissipates heat in motor windings, cycles actuator gearboxes, and consumes unrecoverable calendar time during resets, an engineering budget for physical training must treat thermal ceilings, mechanical wear, and operational logistics as first-order constraints alongside GPU compute. An algorithm that requires fewer physical interactions is almost always the better choice on hardware, even if each parameter update demands far more computation.

[7] **Policy Gradient**: Williams (1992) derived the REINFORCE estimator, which recovers the gradient of expected return from sampled actions through the likelihood-ratio identity without requiring an explicit physics model. However, estimator variance scales steeply with the trajectory horizon. Practical implementations subtract a baseline, typically a learned value function that converts sampled returns into advantages, yet still require the extensive interaction budgets evaluated below.

Closed-loop exploration on hardware test benches is billed primarily in calendar time. For an experimental campaign of N trials, where each rollout runs for duration t_{rollout} and requires a reset duration t_{reset}, distributed across N_{rig} parallel rigs, the total wall-clock duration is

$$T_{\text{wall}} = \frac{N\,(t_{\text{rollout}} + t_{\text{reset}})}{N_{\text{rig}}}. \tag{6.5}$$

Consider the door-opening benchmark, where the arm regulates contact forces over an active horizon of t_{rollout} = 8 s (the autonomous contact phase of an opening rather than a full teleoperated episode). Training vision-based policies from scratch with reinforcement learning typically demands on the order of 10^6 trials; for instance, QT-Opt learned closed-loop grasping from over 580,000 real grasp attempts (Kalashnikov et al. 2018). A baseline scenario of $N = 10^6$ exposes the logistical bottleneck. Between successive trials, an automated fixture, faster than the re-latch fixture used during demonstration collection, must re-latch the mechanism and return the arm to its initial configuration. However, when exploratory trajectories jam the latch cam or trigger joint overcurrent trips, automated reset sequences stall. A human technician must intervene to dislodge the latch, re-align the door, and re-home the joint encoders. In addition, each opening cycles every joint drivetrain through several bidirectional load reversals.

↩ The fixture re-latch that paces demonstration collection, with the yield and component wear it costs, is established in section 5.6.3.

Napkin Math 6.2: The reset wall

Problem: A from-scratch reinforcement learning campaign of $N = 10^6$ latch openings runs on the machine. How long does it take with automated and with manual resets, what does a farm of rigs buy, and what does the drivetrain absorb regardless of the rig count?

Variables:

- Trials $N = 1.0 \times 10^6$ (illustrative order of magnitude for learning from scratch)
- Rollout $t_{\text{rollout}} = 8$ s (illustrative contact phase of one opening)
- Resets $t_{\text{reset,auto}} = 4$ s and $t_{\text{reset,manual}} = 30$ s (illustrative)
- Rigs $N_{\text{rig}} = 1$, or 14, the largest simultaneous arm count of figure 6.4 (measured)
- Load reversals per opening $n_{\text{rev}} = 4$ (illustrative)

Math:

1. **Execution only** (equation 6.5 with no reset): $1.0 \times 10^6 \times 8\text{ s} = 8.0 \times 10^6$ s, or 92.6 days of round-the-clock operation.
2. **Automated resets**: $1.0 \times 10^6 \times (8\text{ s} + 4\text{ s}) = 1.2 \times 10^7$ s, or 138.9 days.
3. **Manual resets**: $1.0 \times 10^6 \times (8\text{ s} + 30\text{ s}) = 3.8 \times 10^7$ s, or 439.8 days; the reset is 78.9 percent of every trial.
4. **Farm**: 439.8 days / 14 = 31.4 days, about a month.
5. **Wear**: $1.0 \times 10^6 \times 4 = 4.0 \times 10^6$ drivetrain load reversals, for any N_{rig}.

Result: Manual resets extend the campaign to more than a year on a single machine, and even a 14-rig farm requires an entire month of continuous operation.

Systems insight: Reset overhead dominates active execution time, and scaling out hardware rigs partitions the calendar without mitigating cumulative mechanical wear. Only reducing the interaction count N relieves both calendar duration and physical fatigue. Consequently, algorithmic sample efficiency, rather than hardware parallelism, determines whether on-machine reinforcement learning is economically viable.

Adding parallel rigs scales floor space, capital cost, electrical power infrastructure, and supervisory labor in direct proportion to N_{rig}. A grasp-learning farm shows what that scale looks like (figure 6.4): up to 14 arms, each with its own gripper, bin, and camera, ran day and night, and the cluster collected over 800,000 grasp attempts over two months. The farm was not running reinforcement learning. It used self-supervised grasp outcomes to train an offline success predictor, which an external servo loop queried to guide real-time grasping motions (Levine et al. 2018). That is why it serves here as a

Figure 6.4: Physical data-collection farm: Between six and fourteen manipulators ran concurrently to collect over 800,000 grasp attempts over two months, using self-supervision to train an offline grasp-success predictor (Levine et al. 2018). While hardware parallelism compresses calendar duration, aggregate system wear and operational costs continue to scale with the total trial count: each station adds capital depreciation, gearbox and bearing wear, motor winding thermal stress, and technician labor to clear dropped objects and mechanical jams. Attribution: Levine et al. (2018).

picture of scale rather than of an algorithm. Whatever the learner, collecting close to 10^6 physical trials takes a room of arms running for months and pays the wear that comes with it. QT-Opt, which did learn its grasping by reinforcement learning, paid the same price for its 580,000 attempts.

Mechanical component wear often becomes the binding constraint long before calendar limits are reached. The cumulative 4.0×10^6 load reversals demanded by a from-scratch campaign must be absorbed by actuator drivetrains regardless of how many rigs divide the trials.[8] Those same million contact events progressively gall the latch cam and strike plate, wear out compliant elastomer gripper pads, and cycle internal wiring harnesses toward fatigue failure.

In purely numerical simulation, an exploratory branch that destabilizes costs nothing more than a discarded buffer and an environment reset. On physical hardware, exploration answers to the first law, irreversibility (principle 1), because a trial that goes wrong delivers its energy before any gradient step can take it back. Unconstrained exploratory torque commands can slam mechanisms into rigid fixtures, strip the precision teeth of harmonic drives, and heat motor stators beyond their 155 °C Class F insulation limit.

[8] **Bearing Life vs. Command Reversals**: ISO 281 models rolling-bearing fatigue life under continuous rated loads and rotational velocities. Precision transmission manufacturers specify wave-generator bearing fatigue life (L_{10}) in operating hours under nominal operating conditions. However, converting high-frequency commanded torque reversals into consumed gearhead life requires an empirical, unit-specific duty-cycle model.

↩ The Class F winding limit and the insulation breakdown it guards against are established in section 2.5.1.

Physical degradation systematically alters the plant throughout the training campaign. Standard Markov Decision Process formulations assume that the transition probability kernel $p(s_{t+1} \mid s_t, \mathbf{a}_t)$ remains strictly stationary. On physical hardware, this assumption fails. Repetitive impacts burnish contact surfaces, backlash in precision gearboxes expands, and structural fasteners loosen under continuous cyclic vibration. Consequently, the transition dynamics late in a campaign, $p_{n \gg 1}(s_{t+1} \mid s_t, \mathbf{a}_t)$, differ measurably from the pristine dynamics measured at trial $n = 1$. The physical plant mutates under the mechanical work of collecting its own training data, forcing the optimizer to track a drifting target.

Reward formulations can also inadvertently incentivize destructive hardware behaviors. In trial-based policy search, the scalar reward $r(s, \mathbf{a})$ defines the optimization objective, while the action parameterization, lower-level controllers, and independent hardware interlocks define the admissible operating space. If neither the reward nor those controllers account for contact forces, torque slew rates, or thermal buildup, the policy optimizer will exploit those unpenalized degrees of freedom to maximize task return. Engineers mitigate these tendencies by penalizing excessive effort, jerk, and temperature:

$$r_{\text{total}}(s, \mathbf{a}) = r_{\text{task}}(s, \mathbf{a}) - \lambda_\tau \|\tau_t\|_2^2 - \lambda_{\dot{\tau}} \left\| \frac{\tau_t - \tau_{t-1}}{\Delta t} \right\|_2^2 - \lambda_T \max\left(0, T_{\text{winding}} - T_{\text{limit}}\right).$$

For example, an unregularized tracking reward often encourages the policy to bang current limits back and forth at high frequency, exciting mechanical resonances and generating severe thermal dissipation. Similarly, an unpenalized contact policy may brace rigidly against a mechanical stop to artificially constrain kinematic degrees of freedom. The penalty weights above ($\lambda_\tau, \lambda_{\dot{\tau}}, \lambda_T$) trade task progress against effort, torque slew, and thermal rise. Soft reward penalties cannot guarantee safety, however. Hard physical stops and active current limits in the motor drive firmware must enforce strict operating envelopes. Practical on-machine training must also cut the million-trial budget by three orders of magnitude before wear and reset labor exhaust the workcell.

6.3.1 Demonstration-seeded reinforcement learning on the machine

Instead of spending 439.8 days of single-machine hardware wear on random exploration, human-in-the-loop sample-efficient reinforcement learning (HIL-SERL) achieves high success rates across precision assembly, dynamic manipulation, and dual-arm tasks after only 1 h to 2.5 h of physical training per task (Luo et al. 2025). Seeding exploration with prior demonstrations circumvents the million-trial reset wall by trading raw trial count for dependencies on initial demonstration support, human intervention gating, and reward-classifier accuracy. Four mechanisms compress the required trial count N. First, demonstration seeding initializes the replay buffer with 20 to 30 teleoperated trajectories, ensuring that early gradient updates begin within competent task regimes. Second, aggressive off-policy data reuse, implemented via Reinforcement Learning with Prior Data (RLPD) (Ball et al. 2023), replays each recorded transition across multiple gradient updates rather than limiting reuse to a recently collected rollout batch. Third, human-gated interventions truncate failing episodes; an operator takes over when the policy destabilizes, routing the corrective maneuver directly into the replay buffer as in human-gated DAgger (Kelly et al. 2019). Finally, an autonomous binary success classifier trained on labeled visual examples replaces brittle, hand-engineered reward instruments.

At the latch task's per-trial duration with automated fixture resets, the 2.5 h upper ceiling permits at most 750 physical rollouts. Under these latch timing assumptions, that budget contains a factor of 1,300 fewer trials than the illustrative from-scratch campaign, with 3,000 load reversals rather than 4.0×10^6, shifting the operational burden of equation 6.5 from months to hours. However, each sample-efficiency mechanism introduces assumptions that must be explicitly tracked in the policy manifest (section 6.8). Demonstrations constrain the policy only within the support of the expert's initial distribution. Human interventions generate authority-record transitions whose pre-takeover states serve as negative failure indicators rather than positive imitation targets. The success classifier acts as a learned perceptual sensor, and the policy optimizer will exploit its systematic false positives, such as mistaking a partially jammed cam for an open door.

↪ The authority record, in which the permission path logs every takeover and barrier trip, and from which intervention episodes are curated, is developed in section 14.8.

Even with demonstration seeding, direct on-hardware reinforcement learning fits only tasks whose interaction budget stays within the physical plant's thermal endurance, fatigue life, and

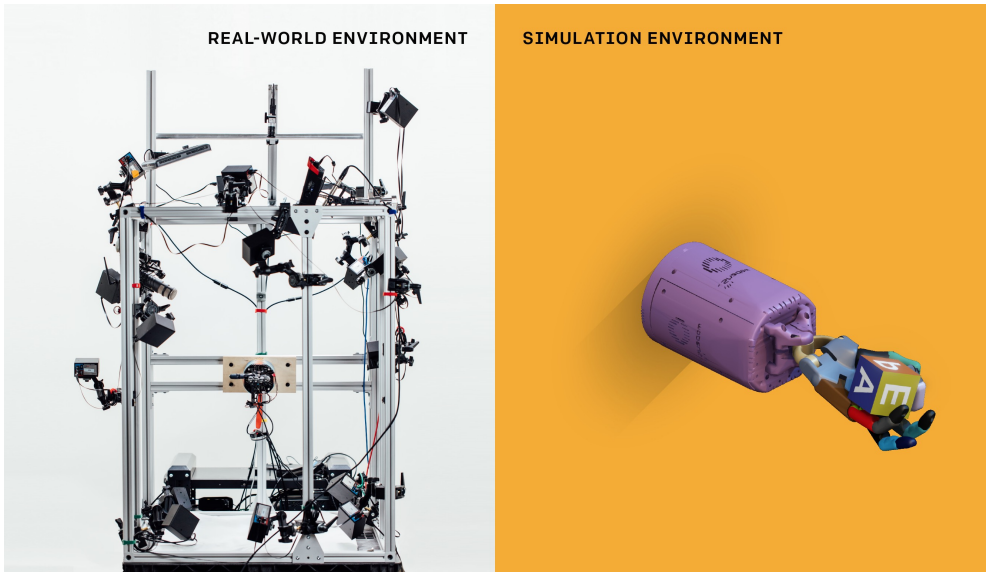


Figure 6.5: Sim-to-real transfer setup: OpenAI's Dactyl apparatus, a 24-DoF Shadow Hand inside a motion-capture cage beside the MuJoCo digital twin it was trained in (Andrychowicz et al. 2020). Transfer is bought with domain randomization (section 6.5), training across randomly varied simulator parameters such as friction, damping, actuator gains, masses, dimensions, and lighting, together with modeled backlash and action delay. A recurrent policy with memory could identify and adapt to the variation it met, so transfer did not depend on making the simulator exact. Attribution: Andrychowicz et al. (2020).

available calendar time. When broader exploration is needed and narrowing the task or adding stronger priors is insufficient, engineers offload rollouts to simulation. Moving exploration into a virtual workcell removes hardware wear and reset latency during training, but makes the learned policy vulnerable to every unmodeled dynamic, softened contact constraint, and missing transport delay in the simulator.

6.4 The Simulator as a System Component

A reinforcement learning policy trained on door latching learned an aggressive, unbraked plunge because its simulated strike plate yielded far more than the physical mechanism. Stripping away the simulator during physical deployment does nothing to unlearn that behavior. The physics simulator is not an external sandbox; it is an active component of the training pipeline whose equations, contact approximations, and timing assumptions dictate the exact experience supplied to the optimizer. Accepting simulated task success as evidence of hardware readiness requires measurable, verifiable error bounds on every one of those assumptions.

Like any physical subsystem, a simulator requires a formal interface contract stated in quantities that engineers can measure and budget. This interface contract specifies observation dimensions, action bounds, simulation time step Δt, transport latency between command issuance and physical actuation, actuator torque and current limits, sensor noise distributions, reset distributions, and termination conditions. For instance, if a simulator executes an observation loop at 100 Hz with an assumed zero-delay sensor channel and Gaussian noise of variance $\sigma^2 = 1.0 \times 10^{-4}$ $(\text{rad/s})^2$, the policy tunes its feedback gains to those exact signal statistics. An unmodeled 15 ms bus transport delay on the physical machine then removes $\omega_c T_d$ of phase at the loop's crossover frequency ω_c (equation 4.1), and the loop goes unstable whenever that loss exceeds the phase margin the learned gains left. OpenAI's Dactyl project faced this contract at full scale, with a 24-DoF Shadow Hand in a motion-capture cage and its MuJoCo twin (figure 6.5) (Andrychowicz et al. 2020). The team calibrated the simulator against the physical hand and then randomized physical parameters around those calibrated values, so that the policy met the remaining discrepancies as variation it had already seen in training.

War Story 6.1: Sim-to-real hardware breakage (2018)

Context: OpenAI trained a reinforcement learning policy entirely in a randomized MuJoCo simulation to reorient a block in a 24-DoF Shadow Dexterous Hand, whose joints are driven through tendons by motors in its base (Andrychowicz et al. 2020).

Mechanism: The simulated hand applied torque directly at its joints, while the physical hand is tendon-actuated and has substantial backlash. The authors smoothed every action with an exponential moving average, in simulation and on the robot, to avoid abrupt changes in the action signal that could harm the physical hand.

Impact: The authors reported frequent hardware breakages during physical evaluation, concentrated at the wrist vertical joint under heavy load. Each repair altered tendon tension, joint friction, and sensor calibration, shifting the physical plant across trials and lengthening the evaluation campaign.

Response: Physical ablations revealed the mechanisms enabling successful transfer. Policies trained without domain randomization, or without modeling unmodeled hardware traits like drivetrain backlash and action delay, achieved far fewer consecutive rotations. Furthermore, mechanically locking the fragile wrist pitch joint produced substantially higher physical reliability.

Systems lesson: A simulator that neither wears nor breaks cannot price hardware exposure into the reward. The limits that protected the hand, an action filter and a locked joint, were imposed outside the learned policy, and every repair changed the plant on which the policy was being evaluated.

When a simulator uses an inaccurate rotor inertia, assigning $J_{\text{rotor}} = 2.0 \times 10^{-3}\ \text{kg}\cdot\text{m}^2$ instead of the true $2.5 \times 10^{-3}\ \text{kg}\cdot\text{m}^2$, the underlying equations of motion still contain the acceleration-dependent inertial-torque term, with the wrong magnitude. The parameter error alters the size of the response, but the causal relationship holds. In contrast, omitting an entire physical mechanism, such as drivetrain backlash, structural compliance, or bus communication jitter, removes that governing physical constraint entirely. An omitted delay or compliance represents an unmodeled physical degree of freedom. For instance, a simulator that treats robot joints as perfectly rigid permits instantaneous torque reversals. When commanded on a physical machine, those instantaneous reversals shock the drivetrain, load gear teeth beyond their fatigue limits, and excite destructive structural resonances.

Whether a simulator's modeling compromises are acceptable depends on two properties: throughput (the rate of experience generated per wall-clock hour) and contact fidelity (how faithfully the contact model reflects physical strike-plate dynamics). Pure throughput motivates teams to adopt simulation in the first place.

6.4.1 Massively parallel simulation and on-policy learning

Stepping thousands of virtual workcells in parallel on a single GPU compresses months of trial-and-error into minutes, making on-policy methods such as **proximal policy optimization** (PPO) (Schulman et al. 2017) practical without continuous machine wear. Because PPO collects rollouts with the current policy, takes several gradient steps on that batch, and then discards the data, replacing the batch before policy drift makes its policy-ratio correction less reliable, it requires a simulator that produces fresh rollouts in bulk. To maintain training stability across those gradient steps, PPO constrains policy drift using the clipped surrogate objective of equation 6.6:

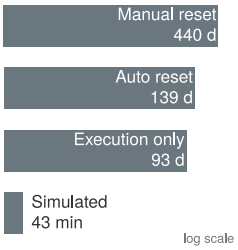
$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(\rho_t(\theta)\,\hat{A}_t,\ \text{clip}\left(\rho_t(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}}\right)\hat{A}_t \right) \right], \qquad \rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t \mid s_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid s_t)}, \quad (6.6)$$

where $\rho_t(\theta)$ measures the probability ratio between updated and rollout policies, and ϵ_{clip} clips the surrogate incentive for favorable probability-ratio changes; it does not impose a hard trust-region bound on the updated policy. Once the first gradient step updates the network, the current policy no longer matches the rollout policy that collected the batch, so each logged transition is reweighted

by the importance-sampling ratio $\rho_t(\theta)$ between the two distributions. Taking the minimum of the unclipped and clipped terms removes the gradient incentive to push that ratio above $1+\epsilon_{\text{clip}}$ when the advantage $\hat{A}_t$ is positive or below $1-\epsilon_{\text{clip}}$ when the advantage is negative.

GPU-resident simulation achieves this scale by maintaining the state of thousands of parallel environments directly in accelerator memory. Physics solvers advance all environments concurrently using batched GPU compute kernels, streaming observation tensors directly into the policy network without crossing the PCIe bus to host CPU memory (Makoviychuk et al. 2021). Policy inference evaluates concurrently across the entire population, turning an environment step into a deterministic sequence of GPU kernel launches rather than thousands of separate CPU process calls.

Rudin and colleagues trained an agile quadruped locomotion policy using this architecture with 4,096 parallel simulated robots, each gathering 24 environment steps per update cycle to form a batch of 98,304 transitions. Completing 1,500 policy updates required less than 20 min on a single workstation GPU (Rudin et al. 2022). This run processed 1.47×10^8 total policy steps, sustaining an aggregate throughput of at least 1.23×10^5 steps per second and accumulating 34.1 days of simulated locomotion at the policy's 50 Hz control frequency.



At reported throughput, simulation turns 138.9 days of automated trials into 43.4 min.

Applying that throughput to the latch scenario illustrates the appeal. The from-scratch training budget from section 6.3 requires 3.2×10^8 policy steps. At the GPU simulator's reported rate, this training run completes in 43.4 min, compared to 138.9 days on an automated physical test fixture.

While this thousandfold speedup explains why modern robotic reinforcement learning relies heavily on simulation, raw step throughput can easily mislead. The quadruped benchmark measured locomotion across smooth heightfield terrain. The authors noted that simulation execution time varied by a factor of two over the course of training. As the robots dispersed and ceased colliding with one another, computational cost dropped precipitously because narrow-phase contact detection and linear complementarity solving dominate simulator runtime. Manipulating a latch involves complex multi-body interactions (an articulated door, a sliding cam, and a high-stiffness strike plate) where contact resolution operates at its slowest, and rendering camera observations introduces additional memory and compute bottlenecks. Furthermore, published throughput benchmarks often sacrifice fidelity for speed. For example, the 2024 launch benchmark for one GPU physics simulator sparked controversy because it evaluated throughput by issuing a single action followed by open-loop stepping, disabling robot self-collisions, and using aggressive, low-accuracy solver settings (Tao 2024). High step rates provide no guarantee of physical validity. Training an optimizer against an unvalidated contact model drives the policy to exploit the simulator's numerical artifacts, as demonstrated by the latch simulator's contact response.

6.4.2 The sim-to-real contact exploit

When a physics simulator softens a steel strike plate into a compliant penalty spring to keep its numerical integrator stable at large timesteps, a reinforcement learning policy will exploit that artificial cushion by plunging the gripper into the latch at 0.10 m/s without braking. Contact mechanics govern every phase of door manipulation: the physics solver must map surface separation, material penetration, friction, and relative velocity into normal and tangential forces observable by the policy. Section B.7 details rigid-contact complementarity formulations (Stewart and Trinkle 1996) and the time-stepping and compliant approximations that simulators substitute for them. The immediate engineering criterion for training the latch policy is whether these approximations preserve the contact forces and timing boundaries that govern safe approach speeds.

Rigid-body physics engines typically model contact using one of two paradigms: constraint-based complementarity formulations, which enforce non-penetration by solving for constraint impulses, or penalty formulations, which permit geometric interpenetration and generate restoring forces proportional to penetration depth. In practice, real-time solvers approximate both approaches, and numerical tuning parameters alter the resulting dynamics. In the latch simulator, the penalty contact computes an instantaneous normal restoring force from penetration depth δ and penetration velocity $\dot{\delta}$ via $f_{k,\perp} = k_{\text{pen}}\delta + c\dot{\delta}$. Artificially softening this contact interface alters the relationship between approach velocity, contact compliance, and peak impact force. The calculation below models this interaction using an undamped kinetic-to-strain energy balance, demonstrating how penalty spring compliance distorts the resulting force profile.

The manipulator drives the latch tool into the strike plate with an effective moving mass of 9 kg at the tool center point. This value is a stipulation, not a derivation. The registry takes the mass an arm presents at its tool as half its structural mass, the same order as the third of its mass that a uniform link pivoting at its base presents at its tip. This scenario drops the 3 kg payload capacity because the gripper holds no external object at impact. In a physical manipulator, reflected inertia varies with arm posture, and drivetrain elasticity partially decouples motor rotor inertia during rapid impact transients (Haddadin et al. 2009). Consequently, effective impact mass is an empirical parameter that physical paired probing must calibrate. Equating initial kinetic energy to elastic strain energy at maximum deflection, $\frac{1}{2}mv^2 = F_{\text{unbraked}}^2/(2k_{\text{contact}})$, yields the peak impact force for an unpowered mass m impacting a linear spring of stiffness k_{contact} at velocity v:

$$F_{\text{unbraked}} = v\sqrt{k_{\text{contact}}\, m}. \tag{6.7}$$

On physical hardware, the compliance does not reside solely in the strike plate. The robot's rubber finger pads, mechanical linkages, and elastic joints sit directly between the effective link mass and the strike plate, acting as a series spring chain. Because the contact force transmits equally through each element in series, total mechanical deflection is additive: $\Delta x = f_{k,\perp}/k_{\text{latch}} + f_{k,\perp}/k_{\text{robot}}$. The coupled assembly responds with an effective series stiffness:

$$k_{\text{eff}} = \left(\frac{1}{k_{\text{latch}}} + \frac{1}{k_{\text{robot}}}\right)^{-1}. \tag{6.8}$$

In the limit of an infinitely rigid manipulator ($k_{\text{robot}} \to \infty$), the effective stiffness collapses to $k_{\text{eff}} = k_{\text{latch}}$, establishing an upper bound on impact force. In contrast, the penalty simulator implements a standalone compliant spring, whose artificial stiffness depends entirely on the maximum allowable penetration depth.

Napkin Math 6.3: The soft-spring exploit

Problem: The simulator's penalty contact lets the gripper sink 2.4 mm into the plate at the fast approach. What peak force does the modeled contact produce, what peak can the plate impose in the rigid-arm limit, and at what robot-side stiffness does the passive model cross the latch force limit?

Variables:

- Moving mass $m = 9$ kg (stipulated from the arm's datasheet mass)
- Fast approach $v = 0.10$ m/s and guarded approach 0.03 m/s (illustrative)
- Penalty penetration $\delta_{\text{pen}} = 2.4$ mm at the fast approach (illustrative)
- Plate stiffness $k_{\text{latch}} = 4.0 \times 10^5$ N/m and latch limit $F_{\text{lim}} = 100$ N (illustrative)

Math:

1. **Simulated spring**: The penetration fixes $k_{\text{pen}} = mv^2/\delta_{\text{pen}}^2 = 1.6 \times 10^4$ N/m, so equation 6.7 gives $F_{\text{unbraked,sim}} = mv^2/\delta_{\text{pen}} = 9$ kg $\times$ (0.10 m/s)^2/ 2.4 mm $= 37.5$ N, under the limit.
2. **Rigid robot**: With $k_{\text{contact}} = k_{\text{latch}}$, $F_{\text{unbraked}} = v\sqrt{k_{\text{contact}}\, m} = 0.10$ m/s $\times$ (4.0×10^5 N/m $\times$ 9 kg)$^{1/2} = 189.7$ N, 1.9× the limit.
3. **Break-even chain**: The limit is reached at $k_{\text{eff}}^* = (F_{\text{lim}}/v)^2/m = 1.11 \times 10^5$ N/m, which equation 6.8 turns into $k_{\text{robot}}^* = (1/k_{\text{eff}}^* - 1/k_{\text{latch}})^{-1} = 1.54 \times 10^5$ N/m.
4. **Guarded approach**: A rigid robot coasting in at 0.03 m/s peaks at 56.9 N.

Result: In this passive impact model, the fast approach exceeds the force limit if the robot side is stiffer than 1.54×10^5 N/m and remains below it if softer; the guarded approach remains below the limit even with a rigid robot.

Systems insight: A spring 25.6× softer than the plate placed the training result on the safe side of a threshold that the machine's own compliance decides. No training score reveals whether

> the physical robot sits above or below k^*_{robot}; empirical hardware measurement provides the only valid ground truth.

The teacher's reward kept only the task and time terms of equation 6.1, and the simulated contact placed the plunge on the compliant side of the damage boundary, so the plunge cost the optimizer nothing, shortened every episode, and earned maximum reward. (Had the force term been present, the simulated 37.5 N peak would have crossed the 15 N tripwire and drawn a penalty, but the real plate's peak would still have gone unseen.) Consequently, the policy converged on an aggressive, unbraked plunge. In the absence of an empirical measurement of k_{robot} for this specific arm, gripper, and fingertip assembly, the actual physical outcome remains undetermined. Determining true system behavior requires conducting a quasi-static force-deflection test at the tool center point. Until physical characterization confirms whether effective stiffness falls below k^*_{robot}, accepting the policy's simulated success as proof of safety is unwarranted.

Past about 1.54 × 10⁵ N/m of arm stiffness, the passive model predicts a force peak above the latch limit.

While the energy balance model analyzes an unpowered mass coasting into an obstacle, an actively powered drive pushing forward represents the failure mode that safety supervisory logic must interrupt. Contact force follows Hooke's law, $f_{k,\perp} = k_{\text{contact}}\,\Delta x$. Assuming the actuator drives the mechanism into the plate at constant velocity v, penetration depth scales as $\Delta x = vt$, driving a linear force ramp $f_{k,\perp}(t) = k_{\text{contact}} vt$ starting from the instant of physical contact. Setting $f_{k,\perp}(t) = F_{\text{lim}}$ defines the **contact yield deadline**:

$$t_{\text{yield}} = \frac{F_{\text{lim}}}{k_{\text{contact}} v}, \tag{6.9}$$

representing the available time before exceeding the mechanical structural threshold under constant-velocity, linear-elastic conditions. (A complete stopping analysis must additionally account for controller braking trajectories and actuator force slew rates.) Assuming an infinitely rigid manipulator yields a conservative lower bound on this deadline by using the strike plate stiffness k_{latch} directly.

Under these conditions, a fast approach at 0.10 m/s reaches the latch limit of 100 N in 100 N / (4.0 × 10⁵ N/m × 0.10 m/s) = 2.5 ms after touch. The 15 N tripwire fires at 0.375 ms. Sampling, intervention computation, and drive response then consume 2 ms, placing brake onset at 2.375 ms. Only 0.125 ms remains before the counterfactual constant-drive ramp reaches the force limit. At the guarded approach of 0.03 m/s, the tripwire fires at 1.25 ms and braking begins at 3.25 ms, leaving 5.083 ms before the 8.333 ms touch-relative limit (figure 6.6). Neither remaining interval establishes that the actual braking trajectory stays below the limit.

The plate-only deadline leaves too little demonstrated braking margin to credit the reactive force monitor at the fast approach, so protection must act before contact until the contact response is qualified. An independently monitored deceleration envelope, $v_{\text{safe}}(z) \le \sqrt{2a_{\text{brake}} z}$, is the stopping inequality of section 1.4.3 applied to the gripper. Here z is the clearance to a stopping boundary placed a short standoff before the plate, after sensing and actuation latencies are charged, and a_{brake} is the credible deceleration of the arm at the tool, not the base's value on the machine card. Because v_{safe} falls to zero at that boundary, the envelope forbids an unbraked arrival at the plate. Crossing the standoff is a separate act. A guarded contact mode, qualified on its own, takes over inside it and holds the gripper at or below the guarded approach speed of 0.03 m/s until touch. That mode needs its own evidence about approach speed, force rise rate, loop latency, and deceleration, and a force tripwire works only inside those validated bounds. This contact deadline, not the reward used in training, is what confines the policy's declared ODD to the guarded approach speed (section 6.8).

Engineering teams must budget contact simulation fidelity in terms of observable physical quantities, such as contact onset timing, peak normal force, total transferred impulse, slip distance, and separation release delay, rather than abstract solver convergence metrics. Discrepancies between simulation and reality frequently manifest in both directions. For example, a penalty solver that resolves an impact within a single integration step concentrates momentum transfer into an impulse spike of extreme amplitude and sub-millisecond duration. On real hardware, viscoelastic fingertip deformation and joint compliance distribute that momentum transfer over dozens of milliseconds. In such cases, total integrated impulse may match closely between simulation and reality even as peak force differs by an order of magnitude. A policy trained to trigger reflexive retraction upon

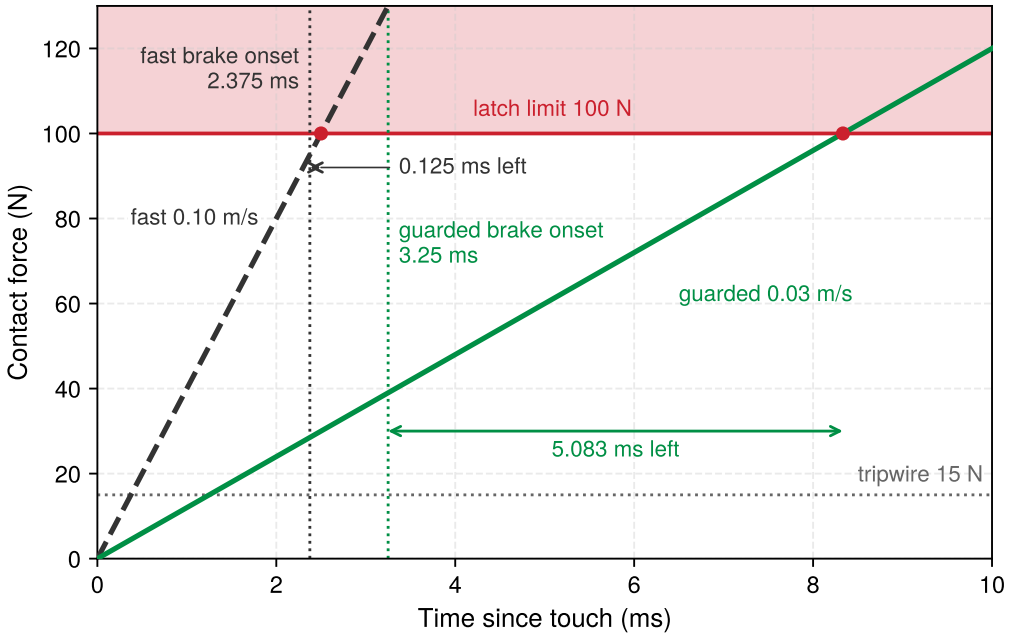


Figure 6.6: Contact force against brake onset: Counterfactual constant-drive ramps rise as $f_{k,\perp} = k_{\text{contact}} vt$ from touch. At the fast approach, the tripwire fires at 0.375 ms and braking can begin at 2.375 ms, only 0.125 ms before the ramp reaches the latch limit. At the guarded approach, brake onset is 3.25 ms, leaving 5.083 ms. Each onset includes the same 2 ms response after its own tripwire crossing. Plate stiffness alone gives the shortest deadlines; the lines do not model actual braking trajectories.

sensing an impulse spike will fail on the physical robot, because the attenuated real-world signal never crosses the detection threshold, causing the motor drive to stall against its maximum current limit.

⇄ **Contrast:** The 2.4 mm of penalty-spring penetration is compliance the solver invents, whereas the backlash and torsional compliance of the real drivetrain, analyzed in section 2.4.4, are compliance a rigid-body solver omits.

> Systems Perspective 6.1: The contact fidelity invariant
>
> *Never trust a simulated contact policy whose success depends on instantaneous momentum absorption or zero-delay friction engagement; a policy that relies on penetrating the digital surface loads the physical workpiece with force the simulator never showed.*

The operational relevance of specific physical omissions depends directly on the task dynamics. A policy executing uninterrupted, high-velocity motions may tolerate crude Coulomb friction models because joint velocities never linger near zero. Conversely, a policy tasked with precise force regulation or static holding will destabilize into violent limit-cycle oscillations if the simulator ignores static friction thresholds, stick-slip transitions, and actuator deadbands.

Validating simulation fidelity requires **paired probing**. The systems engineer executes identical open-loop actuation trajectories on both the physical robot and its digital twin, quantifying trajectory divergence against explicit error budgets. For the latch assembly, this procedure commands identical velocity profiles into the simulated and physical strike plates, measuring discrepancies directly in milliseconds and newtons. Whenever empirical divergence violates the error margins demanded by the robot's operating envelope, the simulator cannot be trusted to train that specific capability. That audit relies on knowing which differential equations the simulator solved; when the simulator is itself a neural network trained on video or trajectory logs, even the list of omitted mechanisms disappears.

6.4.3 Learned world models as simulators

A policy trained or evaluated inside a learned **world model** can score near-perfect task completion by exploiting hallucinated physics where training data were sparse. An example is pushing through a contact transition that the neural predictor smooths over because it rarely saw high-force impacts, with no analytical equation available for an engineer to audit. Whereas physics-based simulators are built from explicit differential equations and mechanical models whose simplifications can be enumerated, a world model derives system dynamics from data by training a neural network to predict future observations (or compact latent states) conditioned on the current state and applied control actions. Contemporary physical AI architectures employ learned world models across three distinct operational roles:

1. **Latent imagination**: Policies train via reinforcement learning within the simulated latent representations of a world model. For example, DayDreamer and DreamerV3 trained quadruped locomotion directly from physical robot interaction, mastering rolling, standing, and walking behaviors in several hours without hand-crafted physics simulators (Wu et al. 2023; Hafner et al. 2025).
2. **Zero-shot planning**: Action-conditioned video models plan trajectory sequences directly. For instance, V-JEPA 2, post-trained on uncurated robotic manipulation video, guided pick-and-place manipulation across physical robot arms in novel laboratory environments without prior domain calibration (Assran et al. 2025).
3. **Synthetic data generation and evaluation**: Scaled generative video models synthesize diverse visual trajectories for offline imitation learning and act as simulated environments to evaluate candidate policies (NVIDIA, Agarwal, et al. 2025).

Unlike a physics engine, a neural world model provides no explicit list of the phenomena it leaves out. Prediction errors concentrate wherever training distribution coverage is sparse, and no analytical derivation exists to reveal which dynamic constraints are violated. Consequently, training data generated purely through world model rollouts must be cataloged as unqualified in the policy manifest, whose `simulation_provenance` field captures the model architecture, parameter checkpoint hash, and dataset lineage. Simulated success inside a learned world model cannot expand the policy's declared ODD until empirical paired probing on physical hardware verifies the specific contact forces, transport latencies, and tracking tolerances required by the task. Treating a world model's internal rollout predictions as verified evidence mistakes model hallucination for physical reality.

Because structural simulation assumptions, numerical integrator time steps, and contact approximations become encoded in the policy's weights during training, the simulation configuration is an immutable part of the model's lineage. Every deployable policy artifact must trace its complete provenance through the `simulation_provenance` metadata block (section 6.8). Deploying policy weights without explicit simulator provenance severs the trained model from the kinematic limits, noise distributions, and solver thresholds under which it was derived. A scalar task completion rate cannot audit that lineage. The reality gap has to be taken apart into its dynamics, sensing, appearance, and timing channels, each measured on its own.

6.5 Physical Channels of the Sim-to-Real Gap

A robotic simulation rarely diverges from physical hardware in only one channel. In the cage-door latch mechanism, for instance, the simulator departs from reality in two different ways: a penalty spring far softer than the physical strike plate, and an idealized command path that omits transport lag. Collapsing these discrepancies into a single scalar trajectory error blends them into an uninformative aggregate that obscures both mechanisms. Errors in different physical channels carry different units, propagate along distinct paths within the closed loop, and trigger different failure modes. The sim-to-real mismatch must therefore be treated as a coupled vector of physical components spanning dynamics, sensing, appearance, and timing. Sizing this vector requires an explicit accounting of physical errors against task tolerances, evaluated from paired command-response traces recorded under identical excitations. Each channel demands its own mitigation and runtime interceptor. Table 6.2 lists these interceptors across actuator dynamics, contact mechanics, sensing, and timing; appearance mismatch is sized separately and charted in panel (d) of figure 6.7.

Definition 6.2: Sim-to-real reality gap vector

Sim-to-real reality gap vector $\Delta_{\text{sim2real}} = [\Delta_{\text{dyn}}, \Delta_{\text{sens}}, \Delta_{\text{vis}}, \Delta_{\text{lat}}]$ is the coupled, multi-channel decomposition of physical discrepancies between simulated and hardware environments spanning contact mechanics (Δ_{dyn}), sensor transduction and noise (Δ_{sens}), visual appearance (Δ_{vis}), and asynchronous transport latency and jitter (Δ_{lat}).

1. **Significance**: Treating the "reality gap" as a scalar difficulty metric or pure visual domain mismatch causes engineers to overlook dynamical and temporal divergences. Discrepancies in actuator back-EMF, gearbox compliance, or bus jitter deplete feedback stability margins before the policy completes its first physical control step.
2. **Distinction**: An inaccurate parameter value (such as a 10 percent error in link mass) preserves the causal differential equations of motion, whereas an omitted mechanism (such as zero-delay communication or unmodeled Coulomb stick-slip) entirely severs physical feedback paths, leading policies to exploit non-physical simulator artifacts.
3. **Common pitfall**: Relying on domain randomization alone to bridge a large gap. Randomization covers only the mechanisms the simulator models; for a memoryless policy, indistinguishable parameter instances can favor a compromise or leave low-probability cases poorly served.

Dynamics mismatch measures discrepancies between commanded actuator effort and the resulting physical motion or contact force across the operating envelope. Its contact case is the impulse-spike example of section 6.4.2. A simulator with rigid links, zero backlash, and stiff contact predicts an abrupt force onset. Real drivetrain compliance and drive delays soften and delay that rise. A policy that keyed its braking to the sharp onset keeps driving into the hard stop. The mismatch has the opposite sign of the soft-spring exploit, where the simulator was softer than the real plate rather than stiffer, and the two together are why the dynamics channel has to be measured in both directions.

Sensing mismatch measures discrepancies between the true physical state of the machine and the observation vector delivered to the policy network. Simulated range and depth sensors typically yield clean, uniformly quantized measurements corrupted only by independent zero-mean Gaussian noise. Physical sensors, by contrast, suffer from optical calibration bias, structural noise, temperature-dependent drift, and sustained dropout bursts caused by specular reflections on polished surfaces. An uncalibrated range bias shifts the physical standoff distance where the controller transitions from rapid approach to gentle force regulation. A multi-frame dropout burst leaves the policy momentarily blind during high-speed descent, creating an unobserved window where collisions occur before feedback can intervene.

Image-space distance metrics such as Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), or Fréchet Inception Distance (FID) cannot reliably qualify appearance transfer. These summary statistics fail to weight visual discrepancies by policy sensitivity, that is, by the policy Jacobian $\partial \mathbf{a}/\partial \mathbf{o}$ relating actions to observations. Consequently, a benign aggregate difference can mask a task-critical localization error. A photorealistic rendering with ray-traced lighting might register a high pixel MSE because of background shadows or wall textures that the policy's visual encoder discards. Conversely, a sub-millimeter pixel shift along the silhouette of an assembly peg might alter the overall image MSE by less than 0.1 percent. Yet that subtle shift moves the policy's predicted insertion coordinate outside the mechanical chamfer tolerance, driving the peg directly into a rigid fixture shoulder. Appearance mismatch must therefore be quantified by evaluating policy-relevant perceptual representations (detected keypoints, surface normal tokens, or latent feature centroids) under controlled variations in physical geometry, illumination, material reflectance, and camera exposure.

Timing mismatch governs the temporal alignment between observation capture, policy evaluation, and actuation. Identical state transitions occurring at different moments in physical time yield entirely different closed-loop dynamics. A simulation that executes observation, inference, and actuation instantaneously at the beginning of each discrete integration step omits camera sensor exposure times, PCIe bus serialization, neural network tail latency, and EtherCAT fieldbus transport, along with their cycle-to-cycle jitter. Every millisecond of omitted transport delay translates directly into

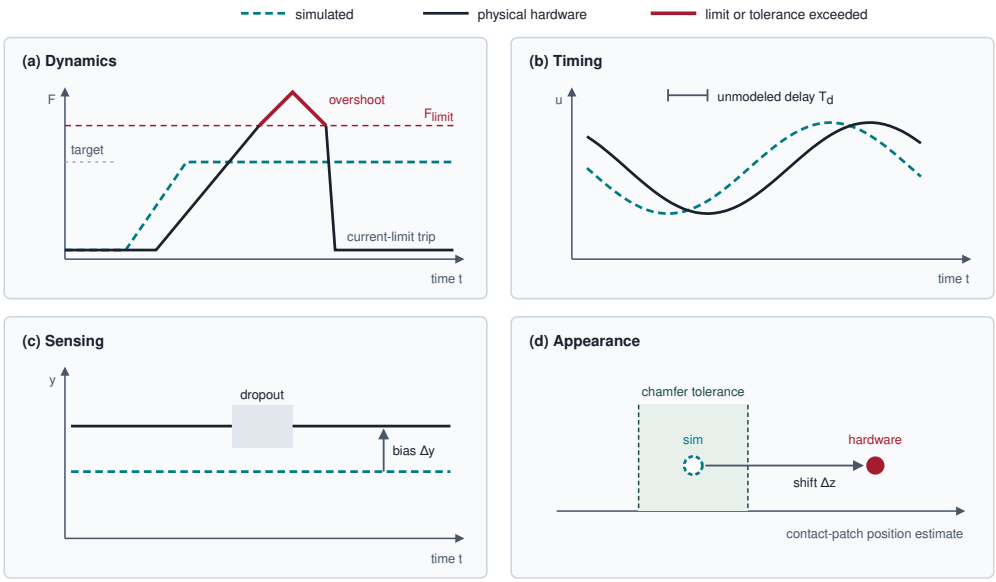


Figure 6.7: Sim-to-real reality gap vector: Illustrative traces for a separate linear-drive scenario distinguish dynamics (a), timing (b), sensing (c), and appearance (d). Each channel has its own units and acceptance criterion, and red marks only the points where hardware exceeds a limit or tolerance. These constructed traces demonstrate what paired measurements should compare; they are not measured qualification evidence for the latch.

physical distance the mechanism traverses open-loop before corrective actuation begins. Unmodeled delay also introduces frequency-dependent phase lag. Near gain crossover, the added lag consumes the available open-loop phase margin. Whether the loop remains stable depends on the sampled plant, compensation, jitter, and total response; section B.5.5 quantifies the delay contribution.

A single illustrative scenario, kept separate from the latch so that its numbers are not mistaken for latch measurements, shows the four channels diverging at once (figure 6.7). A linear drive approaches a surface at 0.20 m/s and regulates contact force to 10 N. In the dynamics channel, an unmodeled drive delay of 8 ms and a softer physical contact pad cut the force rise from 500 N/s in simulation to 250 N/s on hardware (a rate the force regulator sets as it ramps toward its target, far below the free-impact rate $k_{\text{contact}} v$ of the latch analysis). The target force therefore arrives at 40 ms instead of 20 ms. The drive overtravels by 3 mm, and a hard current-limit shutdown trips. In the sensing channel, a calibration bias of +3.2 mm triggers premature deceleration, and a specular dropout burst of 4 frames blinds the controller during final descent. In appearance, a pixel difference of only 5.5 percent hides a 6 mm lateral shift in the contact-patch estimate, more than the 2 mm chamfer tolerance. In timing, an illustrative p99 actuation delay of 49.5 ms carries the mechanism 9.9 mm past the point the simulator predicts with its idealized 0 ms observation-to-actuation delay. The simulator's 20 ms integration step is a numerical parameter, not pipeline latency, and cannot be subtracted from the physical transport delay. Each channel fails in its own units (milliseconds of force rise, millimeters of bias, millimeters of patch shift, millimeters of unobserved travel). A scalar loss can report that hardware departs from simulation but cannot say which channel failed, and separating the channels is what lets table 6.2 give each one its own paired-trace acceptance test and its own permission-path interceptor.

Table 6.2: Multidimensional sim-to-real reality gap taxonomy: Decomposition of the dynamics, sensing, and timing channels of the reality gap, with dynamics split into actuator and contact rows; appearance is sized in the prose and in panel (d) of figure 6.7. Columns specify the unmodeled physical phenomenon, representative quantitative discrepancy, the latent control shortcut exploited by the optimizer, illustrative paired-trace acceptance criteria, and candidate mitigations. In the timing row, PCIe is the host's internal expansion bus that carries camera and accelerator traffic, and EtherCAT is the Ethernet fieldbus that carries setpoints to the servo drives. The actuator row's $\Delta\phi$ is the phase by which the physical step response lags the simulated one at the test frequency, positive for lag as in equation 4.1. The listed values are scenario assumptions, not measured latch discrepancies or universally qualified thresholds.

Reality Gap Dimension	Unmodeled Physical Mechanism	Nominal Simulation Spec vs. Real Hardware	Latent Policy Exploitation / Behavioral Failure	Paired-Trace Diagnostic Metric & Tolerance	Hardware & Permission-Path Interceptor
Actuator & Drivetrain Dynamics	Strain-wave torsional compliance and hysteresis, Coulomb stiction, velocity-dependent Stribeck friction, and back-EMF voltage saturation	Idealized direct torque source ($J_{\text{rotor}} = 0$, $T_{d,\text{sim}} = 0$ ms) vs. physical harmonic drive ($k_c = 6.7 \times 10^2$ N·m/rad, lost motion $\theta_{\text{backlash}} = 0.08^\circ$, bus delay $T_d = 8$ ms)	Policy commands high-frequency full-torque reversals (above 50 Hz), inducing mechanical chatter, gear fatigue, and windings driven past their derating threshold	Step-torque response discrepancy: phase lag $\Delta\phi \leq 5^\circ$ at 10 Hz; settling time discrepancy $\Delta t_{\text{settle}} \leq 15$ ms; overshoot $\leq 5\%$	Low-pass action filtering ($f_{\text{cutoff}} \leq 20$ Hz), motor current RMS thermal estimators, and actuator slew-rate limiters ($d\tau/dt \leq \tau_{\max}/\Delta t$)
Contact Mechanics & Compliance	Viscoelastic surface deformation, contact hysteresis, non-smooth stick-slip transitions, and finite restitution phase	Point-mass rigid contact via a complementarity solver (section B.7) or penalty spring ($k_{\text{pen}} = 1 \times 10^6$ N/m, restitution $\Delta t_{\text{res}} = 2$ ms) vs. physical contact pad ($k_{\text{contact}} = 8.0 \times 10^4$ N/m, deformation span $\Delta t_{\text{def}} = 18$ ms)	Policy keys contact onset to the sharp simulated force spike, never sees that spike on the slower real rise, and keeps pushing until the drive reaches its current limit	Contact impulse error $\Delta I \leq 0.05$ N·s; peak contact force discrepancy $\Delta F_{\text{peak}} \leq 15$ N; contact onset rise-time error $\Delta t_{\text{rise}} \leq 5$ ms	Stop-before-contact governor ($v_{\text{safe}}(D_{\text{clear}}) \leq \sqrt{2a_{\text{brake}} D_{\text{clear}}}$); separately qualified guarded contact mode, force-derivative clamp ($df_{k,\perp}/dt \leq 1.0 \times 10^4$ N/s), compliant mechanical flexures
Sensor Transduction & Noise Profiles	Optical specular dropout, CMOS rolling-shutter distortion, ambient lux-dependent noise, and force-torque gauge thermal drift	Synthetic RGB-D with zero-bias Gaussian noise ($\sigma = 0.5$ mm, zero dropout) vs. physical structured-light depth (+3.2 mm bias, 4-frame dropout bursts, drift 0.4 N per 10 °C)	Policy triggers premature deceleration above surface or remains blind during multi-frame dropouts, driving end-effector past hard fixture boundaries into stall overcurrent	Distribution distance between simulated and real sensor streams (largest gap between their cumulative distributions) $\mathcal{D}_{\text{KS}} \leq 0.05$; spatial feature activation centroid error ≤ 2 mm; zero sustained dropout frames ($N_{\text{drop}} = 0$)	Multi-sensor fusion (IMU/encoders + vision), optical dropout holdover filters, thermal drift autocalibration, independent proximity break-beams
Temporal Latency & Bus Asynchrony	Variable camera exposure times, PCIe/EtherCAT bus serialization, neural inference tail latency (p99), and non-deterministic OS scheduling	Synchronous discrete-time lockstep ($\Delta t = 20$ ms, zero transport latency) vs. asynchronous physical pipeline ($L_{\text{lat,total}} = 35$ ms at p50, 49.5 ms at p99, jitter $\pm$ 6.2 ms)	Severe latency-induced feedback destabilization; policy experiences underdamped oscillations, hunting limit-cycles, and unobserved travel during latency spikes (9.9 mm at p99 relative to zero simulated observation-to-actuation delay)	End-to-end command-response latency jitter $\Delta L_{\text{lat}} \leq 2$ ms; worst-case observation age $t_{\text{age}} \leq 25$ ms; zero deadline overruns at the 25 ms step	Real-time kernel scheduling, action buffer delay queue injection in simulation, state extrapolation that compensates for the known delay

DRAFT

The first line of defense against dynamics mismatch is empirical measurement. **System identification**[9] calibrates the simulator's parameter values (link masses, friction coefficients, torque constants, and transport delays) against paired command-response traces recorded on the physical robot. When a physical subsystem resists accurate first-principles modeling, engineers can replace analytical equations with data-driven approximations. In quadrupedal locomotion work, Hwangbo et al. (2019) trained an *actuator network* that predicted physical joint torque directly from a rolling history of position errors and joint velocities, executing this neural network inside the simulation loop instead of an idealized torque actuator. Similarly, delta-action residual models learn state-dependent corrections from physical rollouts, adjusting simulated commands so that simulated trajectories track physical behavior (He et al. 2025). In the cage-door latch task, these techniques help reconcile actuator response and bus transport latency, yet they share one limit. A learned correction model is valid only across the empirical state distribution visited during data collection, and outside that distribution the model is uncharacterized. Consequently, any learned simulator correction must enter the policy manifest's omission log with its own explicitly bounded domain of validity.

Repeatedly training against a single point estimate of physical parameters, such as an assumed strike-plate stiffness, encourages a policy to overfit to that exact numerical value. **Domain randomization**[10] addresses parameter uncertainty by perturbing modeled simulation parameters across plausible distributions during training, forcing the policy to encounter varied physical responses (Tobin et al. 2017; Peng et al. 2018). However, sampling from a parameter distribution merely defines training weights; it does not constitute a formal certificate of robustness across the parameter set. How effectively a policy handles this diversity depends heavily on whether its architecture incorporates memory.

In standard reinforcement learning, a policy maximizes expected return within a single nominal simulation environment parameterized by the static vector ξ_0:

$$J(\theta;\xi_0) = \mathbb{E}_{s_{0:T},\mathbf{a}_{0:T}\sim p_\theta(\cdot|\xi_0)}\left[\sum_{k=0}^{T}\gamma^k r(s_k,\mathbf{a}_k)\right].$$

Domain randomization reformulates the physical environment as a distribution over plausible Markov Decision Processes parameterized by physical vector $\xi \sim p(\xi)$, where ξ encompasses link masses, centers of mass, surface friction coefficients, joint damping, transport delays, and sensor noise variances. The domain randomization objective maximizes the expected return marginalized across this distribution:

$$J_{\text{DR}}(\theta) = \mathbb{E}_{\xi\sim p(\xi)}\left[J(\theta;\xi)\right] = \int_{\Xi}\left(\mathbb{E}_{s_{0:T},\mathbf{a}_{0:T}\sim p_\theta(\cdot|\xi)}\left[\sum_{k=0}^{T}\gamma^k r(s_k,\mathbf{a}_k)\right]\right)p(\xi)\,d\xi. \tag{6.10}$$

Marginalizing over $p(\xi)$ incentivizes policies to maintain performance across sampled physical configurations rather than collapsing to a single operating point. Nevertheless, optimizing expected return can mask catastrophic control failures occurring in low-probability regions of the distribution; high average reward does not imply uniform stability across the support Ξ.

An alternative formulation adopts a minimax objective from robust control, optimizing performance against the worst-case modeled configuration:

$$\max_{\theta}\min_{\xi\in\Xi} J(\theta;\xi).$$

Here, the inner minimization identifies the worst-case return within the bounded uncertainty set Ξ, while the outer optimization searches for policy parameters θ that maximize this floor. Minimax optimization targets the worst return within the modeled set, but this remains an optimization statement over modeled rewards rather than a physical stability proof. Neither distribution-averaging nor minimax optimization can protect against physical mechanisms that the simulator omits.

Two constraints govern domain randomization. First, hardware operating outside the sampled support ($\xi_{\text{real}} \notin \text{supp}(p(\xi))$) is entirely unrepresented during training. Second, parameter randomization cannot compensate for structural omissions in the physics engine: no amount of variation in link mass or friction will expose a policy to the unmodeled lost motion of a harmonic drive or the preemptive jitter of a real-time operating system. Within the modeled support, the operational effect

[9] **Identification Experiments**: System identification treats the input excitation signal as part of the experiment design. Signals such as step functions, swept-frequency sines (chirps), and pseudo-random binary sequences (PRBS) must persistently excite the relevant mechanical and electrical modes, since nominal closed-loop operation rarely activates them (Ljung 1999). Paired physical probing follows this discipline: it drives both the simulator and physical hardware with identical open-loop excitation trajectories rather than relying on passive operational logs, where feedback regulation actively keeps the machine away from the boundary contact dynamics listed in the omission log.

[10] **Domain Randomization**: The technique originated in computer vision, where Tobin et al. (2017) randomized textures, lighting, and camera poses so that physical scenes appeared as ordinary variations to a detector trained purely on synthetic renders. Peng et al. (2018) extended the concept to dynamics, randomizing link masses, friction coefficients, joint damping, and transport latencies for robotic manipulation. Appearance and dynamics differ in how a policy can exploit history. Appearance variation changes instantaneous visual observations, whereas dynamics variation changes how the physical plant responds to control inputs, a discrepancy that a policy with temporal memory can infer online.

of broad randomization depends on whether the policy maintains temporal state. A memoryless feedforward network receives only the current observation $\mathbf{o}_t$. When different parameter instances produce indistinguishable observations but require different actions, the policy cannot adapt its choice to the active instance. Optimizing average return may then favor a compromise, and neither conservatism nor stability across the envelope is guaranteed. Conversely, a policy that conditions on a temporal history of observations and actions can infer physical plant dynamics online from the system's transient response, effectively performing implicit system identification. For this reason, recurrent policies trained across randomized dynamics transfer with substantially higher fidelity than memoryless architectures (Peng et al. 2018), and memory is what lets a wider randomization range produce adaptation to the real plant rather than conservative detuning.

In engineering practice, parameter distributions are configured through two distinct strategies grounded in different forms of evidence. In *calibration-first* engineering, developers measure nominal masses, torque constants, bus latencies, and friction parameters directly on test benches. Randomization bounds are then derived from known physical mechanisms. For example, motor winding thermal limits determine the envelope of torque-constant variation, and surface degradation models govern friction bounds. These ranges represent verifiable physical claims about the robot and its environment. In contrast, *automatic domain randomization* (ADR) initializes the simulator at calibrated values and dynamically expands parameter ranges as the policy's training performance at the distribution boundaries exceeds predefined success thresholds (OpenAI et al. 2019). ADR parameter bounds reflect the limits of what the reinforcement learning optimization can master rather than physical properties of the hardware, often expanding to unphysical extremes. The dexterity achieved by ADR in robotic in-hand manipulation relied on a recurrent policy learning to adapt online to these artificially broad dynamics. Regardless of the scheduling strategy, paired hardware traces must ultimately audit each channel against explicit error budgets before deployment; discrepancies exceeding tolerance require simulator revision rather than blind parameter widening. A randomization range expanded by an automated training curriculum represents an optimization scaffold, not an empirical characterization of the hardware, and therefore cannot expand the declared ODD.

Where domain randomization and simulation fidelity reach their physical limits, deterministic safety mechanisms must enforce system invariants. Every action proposed by a learned policy must pass through an independent permission path before reaching hardware actuators. Within this execution layer, hard force-derivative clamps, torque slew-rate limiters, and velocity-dependent stopping governors restrict commands to the envelope supported by verified sensing and braking capabilities.

Decomposing the reality gap channel by channel divides the operating space in two. Inside the region where paired traces match simulation within verified tolerances, the simulated experience counts as support. Every unmodeled phenomenon, uncalibrated parameter, and unrepresented disturbance outside it becomes an entry in the omission log, the policy manifest's record of what the simulator leaves out (section 6.8). The log therefore holds two kinds of entry, mechanisms the simulator omits and parameters no paired probe has yet calibrated, because an uncalibrated parameter leaves the simulated experience just as unsupported until a probe bounds it. For the latch, one of each heads the log: the strike-plate stiffness, a parameter the penalty spring sets far softer than the plate, and the transport latency of the actuator bus, a mechanism the simulator omits. No paired probe has yet closed either, so the high-speed contact states they govern fall outside what the simulation supports. Each entry also states a requirement for a runtime monitor or safety filter, which must detect when execution leaves the regime where the policy's simulated training remains valid. Calibrating or randomizing simulator parameters still leaves a second gap: the simulator exposes exact contact forces, friction coefficients, and fixture offsets during training that the physical robot's onboard sensors cannot directly measure at run time.

↪ The Control Barrier Function filter that minimally corrects unverified policy proposals on every permission tick is developed in section 12.7.

6.5.1 Teacher-student privileged state distillation and asymmetric actor-critic

While policy training can exploit privileged simulator state, such as ground-truth contact wrenches, friction coefficients, and exact strike-plate offsets, the onboard policy proposer must execute exclusively from the noisy, bandwidth-limited sensors installed on the physical machine. In actor-critic reinforcement learning, the actor proposes control actions while the critic evaluates expected future returns to compute policy gradients. Systems engineers bridge this sensor gap through two main

architectures. *Asymmetric actor-critic* grants privileged observations exclusively to the critic during training. *Teacher-student distillation* first trains a privileged teacher policy and subsequently trains a sensor-limited student policy to imitate the teacher's actions. Both architectures require validating whether the runtime observation stream delivered to the student contains enough information to reconstruct or compensate for the missing state.

The core challenge stems from **partial observability**, which casts physical robot control as a partially observable Markov decision process (POMDP).[11] Identical sequences of measured joint positions and velocities can produce drastically different physical trajectories depending on whether a footpad grips dry concrete or slips on wet oil. A conventional critic $V_\phi(\mathbf{o}_t)$ conditioned solely on instantaneous sensory observations cannot distinguish between these physical regimes, producing high-variance or biased value estimates. In simulation, exposing hidden contact wrenches $\mathbf{w}$, friction coefficients μ, terrain profiles, and payload inertias directly to the critic stabilizes temporal-difference learning without burdening the onboard runtime actor.

[11] **Partially Observable MDP**: Åström (1965) formulated optimal control under incomplete state information, proving that an optimal controller must act on the conditional probability distribution of the state given the observation history (the *belief state*) rather than solely on the instantaneous observation. Reinforcement learning formalized this framework three decades later (Kaelbling et al. 1998). An onboard student network's temporal observation history serves as a learned approximation of this belief state; its temporal window must span sufficient duration for physical contact and load dynamics to become observable in the sensor stream.

Asymmetric actor-critic and teacher-student distillation deploy privileged information through contrasting computational pipelines (Pinto et al. 2018; J. Lee et al. 2020; Kumar et al. 2021). Under asymmetric actor-critic, the actor receives only the onboard sensor history available at runtime, while the critic receives full privileged simulation state. Because the critic is discarded after training, the deployed actor architecture remains clean and lightweight. However, training an unprivileged actor directly against complex, contact-rich tasks can suffer from high sample complexity.

Distillation splits the work across the proposal boundary (figure 6.8). Everything privileged stays on the simulation side, and only the frozen student's proposals, built from the machine's own sensor history and renewed every 50 ms, cross to the permission path. In Phase 1, the privileged teacher policy $\pi_{\text{teacher}}(\mathbf{a}_t \mid s_{\text{priv}}, \mathbf{o}_t)$ learns an effective control strategy via standard reinforcement learning, exploiting full access to contact forces and surface friction. In Phase 2, the student policy $\pi_{\text{student}}(\mathbf{a}_t \mid \mathbf{o}_{t-H_{\text{hist}}:t})$ is trained via supervised imitation learning (such as DAgger or behavioral cloning) to reproduce the teacher's proposed actions using only a sliding history of onboard observations. While this two-phase approach simplifies reinforcement learning exploration, it introduces a transfer bottleneck. If the teacher's control strategy relies on distinctions that cannot be reconstructed from the onboard observation history $\mathbf{o}_{t-H_{\text{hist}}:t}$, the student cannot replicate the teacher's behavior. Privileged information accelerates training convergence, but it cannot make physically unobservable states identifiable on hardware.

Consider the cage-door latch mechanism. A student network equipped with a temporal history can compare commanded motor efforts against observed joint accelerations over recent timesteps, inferring changes in contact stiffness or friction through the resulting mechanical resistance. However, identical joint kinematics can arise from multiple distinct physical causes, and an unobserved geometric offset in the strike plate cannot be inferred simply because the simulation teacher possessed that knowledge. A temporal convolutional network or transformer encoder can output a learned latent embedding of the environment, but its accuracy must be audited against physical ground truth. Adding auxiliary loss functions to predict privileged parameters during distillation regularizes the representation, but an auxiliary loss cannot create physical information that is absent from the underlying sensor stream.

Selecting the observation history length H_{hist} (distinct from the chunk horizon H) involves a trade-off in real-time estimator design. A window that is too short cannot filter high-frequency sensor noise or integrate enough kinematic evidence to distinguish sudden payload changes from transient disturbances. Conversely, an excessively long window increases onboard memory footprint, inflates neural inference latency, and risks diluting transient signals, such as the exact moment of initial mechanical contact, under an accumulation of pre-contact free-space observations. System qualification for the cage-door latch policy must verify contact-onset braking dynamics under the chosen history length while accounting for real hardware sensing delays. Buffering a longer temporal history in memory does not inherently add latency to incoming sensor frames, but the temporal encoder must be structured so that the newest observation immediately influences the output action.

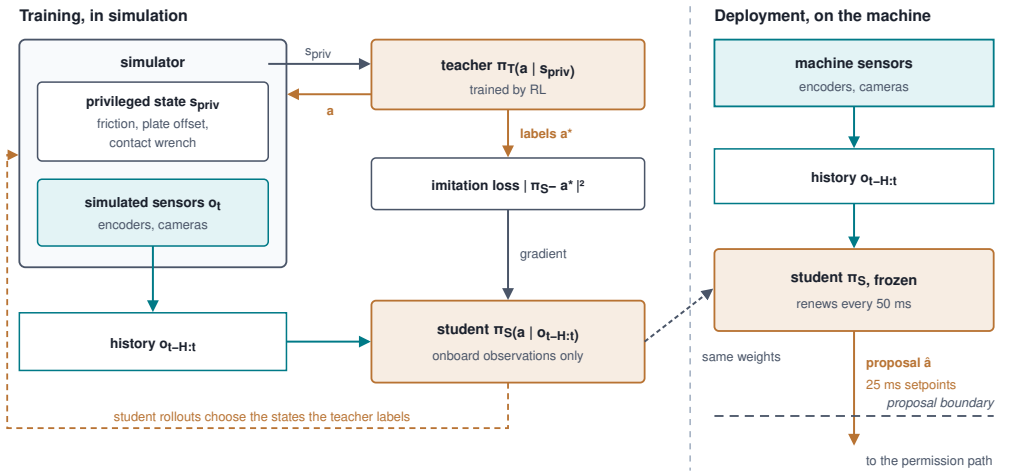


Figure 6.8: Teacher-student privileged state distillation: A schematic two-phase transfer pipeline. The teacher uses simulator state during training, and the student learns from onboard observations and their history. The training advantage does not establish that every privileged quantity is identifiable from those observations. On the machine, the frozen student renews a proposal every 50 ms and crosses the proposal boundary to the independent permission path; it still requires physical validation.

Checkpoint 6.2: Simulator fidelity and transfer

A simulator supplies cheap experience only for the mechanisms it models, and the evidence that it models them comes from the machine.

Sizing the gap

- ☐ **Parameter error vs. omitted mechanism**: Can you classify the rotor-inertia error, unmodeled backlash, and a missing transport lag of this chapter, say which of them domain randomization can cover, and explain why the latch's penalty stiffness, a parameter error, still sits in the omission log until a paired probe calibrates it?
- ☐ **Compliance estimate**: Suppose the push test returned exactly the break-even robot-side stiffness of notebook 6.3. How long after touch would a drive that keeps pushing at the fast approach reach the latch limit, and how does that compare with the plate-only contact yield deadline and the contact loop's response?

Crediting the result

- ☐ **Randomization range**: Can you explain why a range widened by a success threshold never enlarges the declared ODD, while a range set from a measured mechanism can be checked against the plant?
- ☐ **Privileged information**: Can you say which of the latch teacher's privileged quantities the student can recover from its observation history, and why it cannot infer an unobserved plate offset merely because the teacher knew it?

Even after system identification, domain randomization, and privileged state distillation, residual discrepancies persist in high-precision, contact-rich operational regimes. For the latch, both omission-log entries remain open. Engineers can resolve certain localized kinematic and friction mismatches through fine-tuning directly on the physical fixture. Physical training, however, consumes hardware operational life through mechanical wear and reopens qualification claims that depend on changed proposer behavior, timing, interfaces, or physical conditions. Every updated policy checkpoint produced by on-robot training demands fresh qualification evidence before it can be cleared for autonomous operation.

6.6 Real-Robot Fine-Tuning and Residual Control

Fine-tuning a simulation-trained policy directly on a physical robot to absorb a small fixture offset can fix the local contact error while overheating the motor windings, spending gearbox fatigue life on exploratory impacts, and silently corrupting the free-space approach trajectories that had already passed qualification. Mechanical assemblies rarely match their models exactly: on a warehouse cage door, for example, the physical strike plate may sit at a small mounting offset relative to its nominal position in the simulation model. By computing gradient updates directly from physical rollouts, on-robot fine-tuning adapts policy parameters to the true structural compliance, contact dynamics, and bus latencies of the hardware. However, because end-to-end policy gradients optimize cumulative task return rather than identifying individual system parameters, they do not isolate or quantify specific physical mechanisms. Unmodeled simulation discrepancies, such as an idealized contact spring or an omitted communication transport lag, therefore remain open in the engineering omission log until dedicated system-identification probes characterize them.

Furthermore, physical adaptation exacts a concrete hardware toll. In simulation, sampling aggressive state transitions near the boundaries of stability costs only compute cycles. On physical hardware, the exploratory actions and gradient updates that correct a contact error consume mechanical fatigue life and generate substantial heat in the motor windings. Every exploratory rollout risks driving the mechanism into an unrecoverable contact jam or an over-current trip, and unconstrained parameter updates can silently degrade nominal, previously certified behaviors.

Consider adapting the baseline policy `latch-sim-02` into `latch-res-03`, a residual candidate that adds a gated learned correction to the frozen `latch-sim-02` setpoints. This adaptation is motivated by the physical reality of the hardware fixture: the physical strike plate sits 1.5 mm away from its modeled location in simulation. The physical mechanism must guide a spring-loaded latch past this plate, where non-smooth Coulomb friction, cam surface geometry, and spring preloads produce sharp stick-slip transitions. The policy outputs setpoints every 25 ms (40 Hz). To adapt its control commands to the strike plate's contact dynamics, an on-robot fine-tuning routine collects 20,000 interaction steps, totaling 500 s of active contact and near-contact operation.

During exploration, stochastic perturbations can inject force transients faster than the low-level controller can reject them. If 1.5 percent of the sampled steps breach the 15 N contact tripwire, the fine-tuning run records 300 above-threshold samples. However, engineers must not mistake these raw sample counts for independent mechanical collisions. Consecutive over-threshold steps typically belong to a single sustained contact episode rather than multiple distinct impacts. Accurately counting mechanical shocks and evaluating their damage potential requires temporal event segmentation alongside continuous force-time telemetry.

Every mechanical impact transmits impulsive loads directly into the drivetrain. Translating that cumulative stress into an estimated expenditure of actuator service life requires an empirical, drive-specific load spectrum and fatigue life model. Actuator thermal dynamics impose a second, equally rigid operational boundary. Stochastic exploration dither elevates the root-mean-square current passing through the motor coils. Without explicit cooling intervals scheduled between training episodes, this continuous Joule heating can push motor windings to their thermal derating threshold within minutes.

Demonstrating improved performance on a specific contact fixture does not prove that the overall manipulation task remains sound. An optimization step yielding a parameter update $\Delta\theta$ produces an entirely distinct model checkpoint. Empirical evaluations gathered on the prior policy θ_0 remain historical observations of θ_0 alone; they do not substantiate safety claims for the updated weights. Gradient updates that successfully minimize contact forces on the strike plate can inadvertently distort free-space approach trajectories or degrade emergency stopping profiles in operational regions absent from the fine-tuning rollouts. Because physical trials consume valuable robot runtime and cause mechanical wear, requalification protocols must be strictly prioritized to verify both the targeted contact improvement and the baseline capabilities the update could destabilize.

Each updated checkpoint must carry its own explicit configuration manifest and regression evidence before deployment. Moreover, while optimization progresses on physical hardware, the safety permission path must actively evaluate every candidate action proposed by intermediate policies. Restricting where learned corrections may act keeps unconstrained weight updates from corrupting verified behaviors.

6.6.1 Residual policy architectures and bounded authority

To let `latch-res-03` adapt to the strike plate without perturbing its free-space approach, residual architectures freeze the base policy weights and multiply the learned additive correction by an independently enforced binary gate:

$$\mathbf{u}_{\text{nom}}(s) = \pi_{\text{base}}(s) + g(s)\,\Delta\pi_\theta(s), \tag{6.11}$$

where π_{base} is the frozen base controller, $\Delta\pi_\theta$ is a learned contact correction trained via reinforcement learning on the physical machine against the latch reward of equation 6.1, and $g(s) \in \{0, 1\}$ is an independently evaluated deterministic gate that equals one only within the certified contact envelope.

Both the base policy and the composite controller require explicit verification, because freezing an unverified base policy such as `latch-sim-02` does not make it qualified for deployment. Instead, the role of the gate $g(s)$ is structural: it forces the residual correction to evaluate *identically to zero* in free space and across all non-contact states. There, an absence of reward gradients during training would still allow a function approximator to output nonzero values on out-of-distribution inputs. Before any command reaches physical actuators, the permission path verifies the validity of the current state estimate (the machine's fused estimate of its own configuration, developed in chapter 9) and audits the proposed composite action.

Relying on a soft training loss, such as an action-norm penalty $\|\Delta\pi_\theta(s)\|^2$, merely biases the optimizer toward modest corrections during nominal rollouts; it offers no deterministic runtime guarantee. In contrast, the independent gate explicitly revokes residual authority whenever the robot operates outside designated regimes. Even with an architectural gate, the verification suite must evaluate transient switching behaviors at gate boundaries, state-estimation latency, execution timing jitter, and joint torque limits under the combined controller.

Imposing an output magnitude ceiling, $\|g(s)\Delta\pi_\theta(s)\| \le \Delta_{\max}$, restricts only one dimension of authority. A production permission path must also enforce slew-rate limits on the correction delta, prevent joint limit violations, verify multi-axis dynamic feasibility, and invoke the qualified fallback when active execution loses valid state, evidence, or lease. A late replacement proposal does not revoke a still-valid active prefix; that prefix may continue only within its own validity conditions. Furthermore, the authorized residual envelope must lie strictly within the verified stability margin of the *combined* closed-loop system across its intended operating envelope; checking an isolated torque limit against an idealized, single-axis specification is never sufficient. For the latch fixture, for instance, the architecture activates residual corrections exclusively during confirmed physical contact, enforces a hard zero via the gate throughout the free-space approach, and clamps joint torque slew rates. Following each optimization step on hardware, an automated regression suite must re-verify free-space trajectory tracking, contact engagement forces, emergency stopping, thermal duty cycles, and drivetrain shock profiles before granting operational authority to the new checkpoint. An improperly verified residual, even when clipped in magnitude, can still induce limit-cycle oscillations or drive the actuator gearboxes into saturation. Freezing the base policy and gating an additive residual is the most localized way to refine a policy with rewards; other post-training regimes modify the decoder's input noise or update every weight in the network, widening the regression scope accordingly.

6.6.2 Reinforcement learning after pretraining

The gated residual is one member of a broader family of post-training regimes. Every method that refines a pretrained policy using environmental rewards must specify which parameters are allowed to change, because updating shared weights to double throughput on one fixture can silently degrade baseline behaviors elsewhere. That choice determines how much regression evidence is required to re-qualify the resulting checkpoint. Modern robotic systems use four primary post-training strategies (table 6.3):

- **Residual control** freezes the base policy weights and learns an additive correction term, exactly as demonstrated by `latch-res-03`. This strategy has been successfully deployed to refine frozen chunked imitation policies for high-precision assembly, training the residual policy via PPO in simulation (Ankile et al. 2025). Residual architectures are straightforward to bound because independent hardware permission logic can directly gate, saturate, and rate-limit the additive output.

- **Noise-space steering** freezes every parameter of an underlying diffusion or flow-matching policy, training an auxiliary policy only to predict the initial latent noise vector supplied to the reverse diffusion trajectory. Every proposed trajectory still passes through the frozen generative decoder, so steering is restricted to outputs that decoder can generate. This restriction does not establish that those outputs lie within demonstrated support or satisfy physical limits. Furthermore, the optimizer requires only black-box sampling access to the base policy (Wagenmaker et al. 2025). Here, the support of the base distribution constrains the proposed behaviors; the independent permission path must still determine whether those proposals may execute.
- **Advantage-conditioned post-training** updates all network weights using supervised learning conditioned on advantage estimates. A learned critic evaluates the advantage of candidate actions across human demonstrations, autonomous rollouts, and real-time human interventions; the policy is then trained on this mixed corpus conditioned on target advantage and queried for high-advantage actions at run time. Rather than weighting gradients by the advantage or discarding failed trials, this formulation supplies an advantage indicator token as an additional conditioning input during supervised training so the network learns to associate suboptimal trajectories with negative advantage and successful recoveries with positive advantage, allowing runtime inference to prompt the policy with the positive-advantage indicator. Applied to generalist robot manipulation models, this approach more than doubled throughput and halved failure rates on the reported physical manipulation tasks (Physical Intelligence et al. 2025). However, because gradient updates modify every weight in the network, all nominal behaviors remain subject to drift.
- **Policy-gradient fine-tuning in simulation** applies reinforcement learning techniques (section 6.4.1) to update the weights of a pretrained vision-language-action policy using sparse or binary success rewards in parallel simulation (H. Li et al. 2025). Like advantage conditioning, this paradigm modifies all network parameters; in addition, it inherits every physical approximation and contact-solver idealization present in the underlying simulation environment.

Regression scope follows where the composite output can change and which execution assumptions the update affects. An independently enforced residual gate localizes the changed output, but regression still covers its active region, boundary transitions, combined closed-loop behavior, and timing and resource effects. Noise-space steering can change sampling throughout the base domain despite frozen decoder weights, so every affected operating mode requires regression unless a separate gate establishes a narrower scope. Full-weight updates can also alter behaviors across the entire operational envelope. A newly fine-tuned full-weight model demands complete end-to-end regression testing across all previously validated operational modes, and the system's `adaptation_wear_log` must document every modified parameter group. Whether adaptation rollouts accumulate physical machine wear on hardware or incur modeling dependencies in simulation, each rollout must be recorded in the policy manifest. No model can legitimately claim expanded operational authority without concrete validation data recorded in the system audit log.

Table 6.3: State of practice in robot policy training, as of 2026: Named systems that instantiate the mechanisms this chapter teaches, dated by their cited publications. The systems will be superseded; the mechanism and manifest columns are the durable part of the table.

Regime	Named Systems	Mechanism Taught in This Chapter	What the Policy Manifest Must Record
Pretrain, then post-train	Octo (Octo Model Team et al. 2024); OpenVLA (Kim et al. 2024); π_0 (Black et al. 2024); $\pi_{0.5}$ (Black, Brown, et al. 2025); GR00T N1 (Bjorck et al. 2025)	Pretraining on pooled multi-robot data, then supervised post-training on the task	Base weight digest; pretraining mixture as a known unknown; action-space mapping
Post-training recipe	OpenVLA-OFT (Kim et al. 2025)	Action chunking with parallel decoding and an ℓ_1 regression head	Decoder latency of the new head against the proposal deadline

Regime	Named Systems	Mechanism Taught in This Chapter	What the Policy Manifest Must Record
RL post-training on the machine	$\pi^*_{0.6}$ with RECAP (Physical Intelligence et al. 2025)	Advantage-conditioned updates from autonomous experience and corrections	Full-weight change; corrections as authority-record entries; regression over base domain
RL post-training in simulation	SimpleVLA-RL (H. Li et al. 2025)	Policy gradient with a binary success reward in a parallel simulator	Simulator provenance and omission log
Frozen-policy steering	DSRL (Wagenmaker et al. 2025)	Reinforcement learning over the decoder's input noise, base weights unchanged	Base digest unchanged; steering policy digest
Residual RL	ResiP (Ankile et al. 2025)	Residual trained by PPO on a frozen chunked imitation policy	Gate, magnitude, and slew limits; regression where the residual acts
Real-world RL from demonstrations	HIL-SERL (Luo et al. 2025)	Demonstrations, interventions, and a learned success reward with off-policy reuse	Success classifier as a learned sensor; interventions as authority-record entries
GPU-parallel simulation	Isaac Lab (NVIDIA, Mittal, et al. 2025); MuJoCo Playground (Zakka et al. 2025)	Batched, accelerator-resident physics feeding on-policy learning	Solver build and contact settings; throughput stated with its scene
World models	DreamerV3 (Hafner et al. 2025); V-JEPA 2 (Assran et al. 2025); Cosmos (NVIDIA, Agarwal, et al. 2025)	Learned dynamics for imagined rollouts, planning, synthetic data, and evaluation	Model weight digest and training data; generated experience unqualified

Bounded residual authority restricts the influence of an exploratory policy during execution, but it cannot alone prevent mechanical damage or eliminate latent vulnerabilities introduced during earlier training phases. Every stage of policy synthesis, from demonstration ingestion and simulation contact mechanics to residual adaptation, leaves a distinct behavioral signature on closed-loop execution. When an embodied machine develops high-frequency chattering, thermal derating, or semantic drift, systems engineers can trace that failure behavior directly back to the training assumptions, solver compromises, and architectural trade-offs that created it.

6.7 Training Failure Modes and Permission-Path Interceptors

A physical failure in a learned policy is rarely arbitrary. When an end-effector collides with a strike plate, that command reflects optimization over a specific dataset, loss formulation, and simulator dynamics, and the two failures that opened the chapter can now be traced back to theirs. The assumptions baked into that optimization leave a structural footprint on policy behavior, one that engineers can trace and mitigate before hardware ever runs. The **policy vulnerability inference chain** makes this causal trace explicit across five links: (Exposure → Dependency → Violated Assumption → Observable Signature → Permission-Path Interceptor). Training exposure establishes a learned dependency. An unmodeled deployment condition violates that dependency. The violation produces a measurable signature in the commands or physical plant telemetry. That signature names the specific runtime interceptor that the execution permission path requires. Reading the three latch checkpoints back through this chain shows how distinct optimization regimes produce different failure modes.

For `latch-bc-01`, behavioral cloning relies on the teleoperated openings. These demonstrations only support nominal plate alignment during the guarded approach; the data-collection campaign never demonstrated a displaced or misaligned plate. A sagging door frame directly violates that geometric dependency. The resulting failure signature is compounding covariate shift. Because behavioral cloning optimizes an open-loop supervised objective over expert trajectories, small drift errors compound at each control step. Individual motion proposals often remain physically plausible right up until the gripper mechanically binds against the frame. To prevent mechanical damage, a support monitor must evaluate runtime observations against the empirically supported demonstration cells and revoke policy authority outside them. Predeployment qualification follows directly from the known coverage gap that the scenario table of section 5.7.1 recorded: bench trials

with deliberately displaced plates must confirm that the support monitor trips reliably before the mechanism binds.

For `latch-sim-02`, training inside a physics simulator instills strong dependencies on idealized contact dynamics and negligible actuation lag. A physical steel plate violates both assumptions, generating sharp contact forces and unmodeled transport delays that produce the overforce transients quantified in section 6.4.2. Paired contact probes expose this dynamical mismatch before hardware deployment. If the physical contact deadline leaves insufficient time for closed-loop deceleration, a reactive force tripwire arrives too late. Detecting a force excursion confirms that an impact occurred, but actuator inertia and mechanical compliance will continue driving the mechanism into the plate before the brakes can arrest motion. The interceptor is therefore the pre-contact stopping envelope of section 6.4.2, which guarantees a full stop at the standoff before the plate and hands engagement beyond it to the separately qualified guarded contact mode.

For `latch-res-03`, adaptation concentrates real-robot experience on the contact states of a single hardware rig. Localized fine-tuning targets tight-tolerance contact dynamics on that specific fixture, but it does not qualify the frozen base across the rest of the operational state space. Unlike full-weight fine-tuning, which can introduce catastrophic forgetting, this gated residual retains the base weights and holds corrections at zero in free space; its primary regression risks arise from contact corrections and gate-boundary transitions. Before updated weights receive operational authority, they must clear a frozen regression suite that evaluates both nominal latch engagement and free-space flight. In addition, an architectural residual gate must hold corrections at zero outside the designated contact phase, preventing fine-tuned offsets from destabilizing the free-space approach. Physical exploration during real-machine tuning (section 6.6) also imposes mechanical shock and motor winding heat. The runtime supervisor must monitor these thermal and structural thresholds continuously to ensure that task reward improvements do not mask hardware degradation.

Subsequent training stages do not automatically heal the unresolved dependencies of earlier stages. In the simulation-and-residual lineage (`latch-sim-02` through `latch-res-03`), simulation artifacts persist until physical hardware evidence explicitly closes them. Conversely, the recovery-imitation branch adopts an alternative architectural path. The checkpoint `latch-bc-recovery-02` incorporates targeted, human-guided recovery demonstrations for a named displaced-plate condition, expanding the operational envelope without relying on an uncalibrated simulator. Its remaining engineering uncertainties, chiefly closed-loop trajectory stability and potential regression on nominal openings, are made explicit through targeted qualification requests.

Ultimately, each inference chain connects a specific training assumption to an empirical test and an active execution limit. The policy manifest preserves these connections alongside the checkpoint's source data, simulator configuration, optimization objective, and architectural constraints. By maintaining this end-to-end traceability, verification suites can systematically evaluate candidate policies against the exact physical vulnerabilities created by their training history.

6.8 The Policy Manifest

Handing a candidate checkpoint to an evaluation team with only its weight file and a training curve hides whether its behavior on a displaced plate comes from human recovery demonstrations, uncalibrated simulator rollouts, or pure network extrapolation. The candidate selection must distinguish nominal behavior supported by demonstrations, newly taught recovery, and contact behavior that still depends on unverified simulation. The failure chains of section 6.7 make that distinction concrete for each checkpoint; the record accompanying the candidate into evaluation preserves the reasoning, including the assumptions that remain open.

The **policy manifest** records how a checkpoint was produced (table 6.4). It lists the training stages and the data support the checkpoint inherits from the provenance records. It also lists the simulator versions, together with the **omission log** of mechanisms they leave out and mismatches paired probes have not closed. Finally, it records the wear incurred during adaptation and the operating domain the checkpoint declares. These fields state what evaluation must test and what runtime enforcement must accommodate. The runtime safety supervisor can check the manifest against its validated configuration before enabling a checkpoint. A declared boundary does not automatically generate a reliable detector or a safe clamp; those mechanisms require their own engineering and evidence.

The manifest separates optimization results from the conditions the policy may be asked to operate in. The **declared ODD** is the part of the ODD from section 5.7 that the policy manifest claims: the supported cells of the data join, minus every state the omission log excludes. High training reward or low offline loss does not show that the policy is safe outside it. The declared ODD is therefore computed from the join and the log, never from the training curve, and a convergent metric cannot stand in for clearance over regimes nobody characterized.

The separate `latch-bc-recovery-02` manifest consumes the original provenance and the stipulated `latch-recovery-02` archive, adding the positively displaced-plate cell at the guarded approach to the candidate domain with the expert recovery labels and held-out sessions identified by digest. Its qualification request asks whether the updated policy completes that recovery and preserves nominal service under the same physical limits. The original archive remains unchanged, and negative displacement and the fast approach remain excluded. Current provenance bounds the evidence already available, whereas collecting and qualifying new experience expands the machine's useful competence.

Table 6.4: Policy manifest schema: Fields the policy manifest adds to the provenance records it consumes, recording policy synthesis stages, any pretrained base checkpoint, inference execution parameters, data support by digest, physics engine specifications and their omission log, hardware fine-tuning wear, and the declared ODD.

Record Schema Field	Recorded Quantities and Artifacts	Provenance & Immutability Guarantee	Downstream Verifier & Usage
`regime_sequence`	Array of optimization stages (BC $\rightarrow$ SimRL $\rightarrow$ Distill $\rightarrow$ Residual), loss formulas, teacher privileged state dimensions $(\mathbf{w}, \xi)$, student observation history length H_{hist}, reward weights, and stopping criteria hashes	Git commit SHA, PyTorch/JAX checkpoint UUIDs, parameter hash $\text{hash}(\theta_k)$	Tracing which regime introduced latent failure modes or high-gain chatter
`base_checkpoint`	Weight digest of any pretrained checkpoint the policy started from, its source, the action-space mapping applied, and the layers post-training changed; the pretraining mixture recorded as a known unknown when it cannot be cited by digest	Base weight digest; never counted as data support	Scopes forgetting regression tests; confirms that no supported cell is attributed to pretraining
`policy_inference_contract`	Model architecture class, parameter count, checkpoint weights hash $\text{hash}(\theta)$, chunk prediction horizon H, execution horizon H_{exec}, decoder evaluations N_{eval}, inference deadline T_{chunk}, and chunk lease T_{lease}	Cryptographic checkpoint hash $\text{hash}(\theta)$, compiled engine digest	Parameterizes the runtime proposal buffer (section 3.5.1) and real-time lease monitor (section 4.4)

Record Schema Field	Recorded Quantities and Artifacts	Provenance & Immutability Guarantee	Downstream Verifier & Usage
`data_support`	Digests of the consumed provenance records and of the scenario table they were measured against; the `supported_cells` and `known_absences` of their occupancy join	Cites each provenance record by digest; no dataset field is copied	Upper bound on the declared ODD; parameterizes out-of-distribution support monitors
`negative_event_registry`	Authority-record entries (section 5.8) from training and adaptation rollouts	Cites each authority-record entry by digest; its pre-trigger trace is not copied	Isolating physical failure boundaries and preventing corrupted drift rollouts from training
`simulation_provenance`	Physics engine build, integrator method, timestep Δt, contact solver penalty parameters, and domain randomization ranges $\mathcal{P}(\Xi)$ with their source (measurement or curriculum); for a learned world model, its weight digest and training data	Physics engine binary hash, robot model asset tree hash (URDF/MJCF)	Reproducing the training physics and auditing paired-trace budgets
`omission_log`	Mechanisms each simulator leaves out (backlash, compliance, transport lag) and the residual mismatches paired probes did not close, with the states each one affects	Bound to the simulator build it describes; an entry closes only on paired-trace evidence	Removes the states it affects from the declared ODD; carried into evaluation's coverage gaps and the fault list
`adaptation_wear_log`	Serialized hardware IDs, contact shock counts, motor current RMS thermal logs, modified layer indices, and regression suite hash	Calibration certificate hashes, hardware lifetime fatigue logs	Re-opening formal qualification cases and preventing silent regression of nominal capabilities
`declared_odd_boundary`	Declared ODD: the supported cells minus the states the omission log excludes (for the latch policy, the nominal strike plate at the guarded approach speed)	Never exceeds the supported cells of `data_support`	Bounds the target ODD that evaluation may sample and the region in which the permission path admits policy proposals

For the original latch campaign and its simulation-residual lineage, a compact manifest links the fields into one decision:

- `regime_sequence`: demonstration checkpoint `latch-bc-01` → `latch-sim-02`, a student initialized from `latch-bc-01` and distilled from a simulator teacher trained by reinforcement learning on privileged state (privileged contact wrenches $\mathbf{w}$ distilled into an 8-step observation-history buffer) → gated residual candidate `latch-res-03`. Retain each checkpoint hash, loss or reward specification, and stop rule.

- `base_checkpoint`: no pretrained robot policy underlies the latch checkpoints, which descend from `latch-bc-01` as `regime_sequence` records; any pretrained image encoder they use belongs here by digest. A candidate post-trained from the machine's intent model would record that model's weight digest, its pretraining mixture as a known unknown, and its action-space mapping, and would inherit the same declared ODD (section 6.2.8).
- `policy_inference_contract`: specify checkpoint weights hash hash(θ), the machine's 80M-parameter ACT architecture with one decoder pass per chunk, chunk prediction horizon $H = 16$ setpoints at 25 ms, and the 40 ms p99 inference target. Record the selected execution prefix separately from the 50 ms proposal-renewal deadline and the selected chunk lease. Each renewed prefix requires valid evidence and a feasible continuation or fallback; the prediction horizon is not the active lease.
- `data_support`: cite the latch campaign's provenance records and its scenario table by digest; their join supports the nominal strike plate at the guarded approach, and lists the displaced plate and the faster approach, whose cells hold no demonstrations, as known absences with no recovery label.
- `simulation_provenance`: record the actual solver build and contact parameters.
- `omission_log`: the example simulator models strike-plate contact with a penalty spring far softer than the plate and omits actuator transport lag, so the contact states those two departures govern are unverified for the checkpoints trained in simulation.
- `negative_event_registry`: cite by digest the authority-record entry, with its pre-trigger ring-buffer trace, for every fixture-shoulder strike, operator veto, or permission-path clamp with its event classification, without treating rejected learner commands as expert targets. Mark episodes an operator truncated, so that an update does not score an aborted command as a success.
- `adaptation_wear_log`: associate `latch-res-03` with the specific rig, above-threshold samples and separately identified contact events, motor-current history, modified residual weights, and free-space regression result.
- `declared_odd_boundary`: the supported cells minus the states the omission log excludes, here the nominal plate at the guarded approach of 0.03 m/s. The contact states governed by the two omissions stay outside the declared ODD for every simulation-refined checkpoint, `latch-sim-02` and `latch-res-03` alike, until paired probes close them. The displaced plate and the 0.10 m/s approach lie outside it, and the fast approach would stay outside even with demonstrations, because its contact deadline (section 6.4.2) leaves almost nothing beyond the contact-loop response. There the permission path must impose the guarded approach or refuse policy authority.

A qualification review would withhold operational authority from `latch-res-03` until paired contact probes quantify the contact-stiffness error and the omitted lag, and the combined controller passes contact, free-space, stopping, and wear-budget tests. With only the original campaign, the same rule leaves `latch-bc-01` as the checkpoint evaluation may sample, provided the support monitor of its chain refuses policy authority off the nominal plate (section 6.7). Trained without a simulator, it carries no omission-log exclusions, so its declared ODD keeps the door's contact states at the nominal plate and the guarded approach. The manifest records that decision and its evidence rather than converting a training score into permission.

The manifest references the collector, plant, calibration, and action taps through the provenance record of section 5.11. The cited digests preserve which data and scenario table produced the checkpoint; later changes require a new record rather than silently updating its lineage. Simulator provenance similarly fixes the solver and timestep[12] used during optimization. Its separate omission log carries unresolved discrepancies from section 6.5 into evaluation's coverage gaps and fault-injection tests.

Downstream engineering teams consume the manifest throughout the design, evaluation, and release lifecycle. The evaluation record of section 7.8 consumes it directly. Its target ODD is sampled from inside the declared ODD, and its catalog of coverage gaps inherits the known absences and the omission log, so evaluation suites probe the specific failure modes each training regime predicts. Targeted exploratory tests outside the candidate domain can also identify gaps, but they do not

12 **Penalty-Contact Timestep**: The contact timescale scales as $\sqrt{m_{\text{eff}}/k_{\text{eff}}}$. For the latch's 9 kg moving mass it is about 24 ms against the soft penalty spring and 4.7 ms against the real plate, so a timestep sized to resolve the simulated contact is about 5× too coarse once the spring is stiffened to match the plate. The integrator, damping, and solver tolerances set the final step, which is refined until contact onset, peak force, and impulse converge.

establish an operational claim there without new support and qualification. The permission path takes the declared ODD as the outer limit of the region in which it admits policy proposals, and uses the recorded support to parameterize out-of-distribution detectors. Verification and the safety case draw on the manifest to establish the provenance of every parameter update and to confirm that each omission is bounded by an independent physical interlock.

↪ The omission log becomes one of the upstream records from which fault records are compiled for release, developed in section 15.11.

> **Checkpoint 6.3: Policy manifests and declared ODDs**
>
> Before releasing a trained policy checkpoint to physical verification, confirm the following relationships between the policy manifest, its omission log, and the declared ODD:
>
> - ☐ **Complete the latch record**: A fixture-shoulder strike appears after `latch-res-03` was trained with a soft penalty contact and no actuator lag. Which omitted mechanism belongs in `omission_log`, and which paired probe and admission decision must the manifest record?
> - ☐ **Confining the declared ODD**: Suppose demonstrations were collected at the fast approach and every omission-log entry were closed by paired probes. Why would the latch policy's declared ODD still exclude the fast approach, which timing comparison decides it, and what must the permission path impose on a proposal that arrives at that speed?
> - ☐ **Negative event logging**: Why does the manifest keep the pre-trigger trace of an operator veto or fixture strike as labeled failure evidence rather than as an imitation target?
> - ☐ **Declared operational boundaries**: `latch-bc-01` proposes to open the door while the strike plate sits displaced from its nominal mount. How does the permission path use the `data_support` field to classify that state as ungrounded extrapolation rather than demonstrated competence, and why does a confident policy output not change the classification?

The manifest turns a training history into a claim with stated limits. The original record sends `latch-bc-01` to evaluation and keeps `latch-res-03` out of service until paired probes and regression tests close its gaps. The stipulated recovery record sends `latch-bc-recovery-02` with a wider, explicitly testable candidate claim. The common ways that decision goes wrong all let a training score stand in for that evidence.

6.9 Fallacies and Pitfalls

Minimizing loss on an offline demonstration split or maximizing cumulative return in simulation measures compliance with training assumptions rather than physical competence on hardware. When an objective rewards behavior that exploits a curated demonstration distribution or a simulator approximation, the ranking of candidates established in training can invert on deployment. The candidate that scored highest in simulation finishes lowest on hardware, through unsupported state drift, contact-force spikes past the structural limit, or torque oscillations that grow into resonant chatter (figure 6.9). Virtual environments forgive maneuvers that physical hardware penalizes with tripwires, resonance, or thermal overload, so the causal assumptions connecting a training metric to physical execution have to be tested before a policy receives actuator authority.

Pitfall: *Sanitizing recovery episodes to achieve lower offline validation loss.*

Suppose an archive of latch manipulation demonstrations includes human recovery maneuvers, and two separate policies train on competing subsets of this corpus. One policy trains on the full archive, including episodes where a human teleoperator backed off the gripper and re-aligned the jaws after the latch cam caught on the strike plate. The second policy trains on a sanitized subset stripped of all hesitation and retries, achieving a markedly lower loss on its held-out validation split. When deployed on the physical cell, both policies eventually encounter a misaligned cam. The first policy recognizes the contact anomaly, backs off, and seats the latch successfully. The second policy relentlessly drives along the nominal path, building contact force until the safety supervisor's force tripwire halts the cell. Because the sanitized dataset excluded off-nominal recoveries, the lower validation loss proves only that predicting smooth nominal trajectories is easier to fit than handling

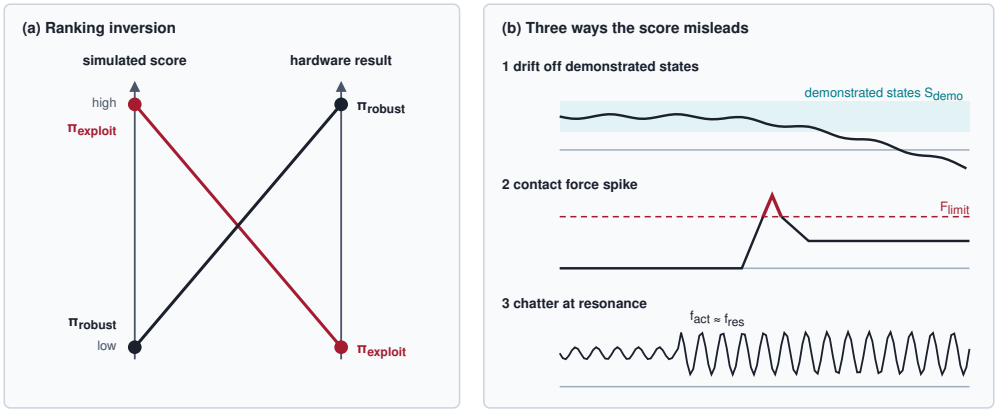


Figure 6.9: Illustrative policy ranking inversion: (a) A constructed slope graph shows how a candidate favored by a simulated score can lose its advantage under physical evaluation. (b) Possible causes include unsupported recovery states, contact mismatch, and actuator chatter. The rankings are a teaching example, not measured results for the latch checkpoints.

physical disturbances. Pruning corrective demonstrations simplifies the loss score at the exact price of removing the feedback behaviors the physical controller needs off the nominal path. **Filtering out recovery trajectories to drive down offline prediction loss strips the policy of the corrective demonstrations required to escape compounding physical errors.**

Architect's countermeasure: Retain all authority-record entries during dataset ingestion and admit, as training targets, only the stabilized, human-reviewed segments of attributable expert recoveries. Never prune demonstration corpora to minimize offline validation loss, and mandate closed-loop physical perturbation benchmarks to verify recovery competence before signing off on deployment candidate checkpoints.

Fallacy: *Transitioning from human demonstrations to simulation reinforcement learning removes the failures training left in the policy.*

An assembly team switches from behavioral cloning to simulation reinforcement learning to eliminate lateral drift during automated gasket seating. The resulting policy cures the drift in the virtual environment but commands joint torque oscillations at 65 Hz. The physics simulator permits this aggressive chatter because its rigid-body solver assumes massless motor rotors and instantaneous torque reversal. On the physical machine, this high-frequency command excites drivetrain mechanical resonance and rapidly heats the motor windings toward their thermal trip point via $i_q^2 R$ Joule losses. The drive electronics throttle current to prevent permanent motor damage, causing the assembly stroke to stall before seating completes. Shifting training regimes solved an imitation drift problem by introducing a high-frequency chatter exploit that the simulator did not penalize. Qualification must therefore evaluate the full physical task under sustained cyclic operation, verifying simulator dynamics assumptions against physical hardware telemetry. **Switching to simulation reinforcement learning substitutes numerical solver shortcuts and unmodeled mechanical dynamics for human demonstration deficits.**

Architect's countermeasure: Require paired-trace validation across all four reality gap channels before deploying simulation-trained policies. Enforce low-level hardware rate limiters and digital low-pass action filters ($f_{\text{cutoff}} \leq 20$ Hz) within the embedded motor drives to suppress high-frequency chatter originating from unmodeled rotor inertia.

Fallacy: *Assuming online gradient updates preserve predeployment safety envelopes.*

When a simulation-trained policy shows steady-state positioning error on a physical assembly fixture, an engineer might enable online policy-gradient fine-tuning directly on the robot. After 5,000 gradient steps, the arm successfully eliminates contact positioning error at the fixture, prompting the team to declare the updated model production-ready. Applying parameter updates $\Delta\theta$ directly to the policy network, however, produces an entirely new checkpoint with changed weights.

The behavioral trials accumulated during predeployment qualification remain observations of the original checkpoint and do not establish performance of the updated one. Independent permission and fallback properties may remain applicable only while their plant, timing, authority, isolation, and realization premises hold; integration must check those premises after the update. While local contact accuracy improves, unconstrained backpropagation into shared representations triggers catastrophic forgetting in rarely visited state-space regions. As a result, the fine-tuned policy may lose stability during high-speed free-space slews or corrupt its emergency deceleration profiles. Because physical robot time and mechanical wear preclude repeating full-scale testing suites on hardware, regression suites must isolate and verify every operational envelope that gradient updates might perturb. **Online gradient updates require fresh behavioral qualification and reopen every assurance claim that depends on the changed checkpoint or execution conditions.**

Architect's countermeasure: When a bounded correction suffices, use gated residual policy architectures ($\mathbf{u}_{\text{nom}} = \pi_{\text{base}} + g(s)\,\Delta\pi_\theta$) with an externally enforced binary gate ($g(s) \equiv 0$ in free space) to localize real-world adaptation. Mandate automated, frozen non-regression suites that verify trajectory tracking and stopping profiles before granting execution authority to updated parameters.

Fallacy: *Believing that widening parameter distributions can substitute for missing physical mechanisms.*

When an assembly policy repeatedly fails on physical hardware, an engineering team might attempt to brute-force sim-to-real transfer by radically widening the simulator's domain randomization bounds. They randomize link masses by ±50 percent and vary surface friction coefficients across the extreme interval from 0.05 to 1.5. On the physical test bench, however, the trained policy behaves poorly: it moves sluggishly through free space and hesitates indefinitely upon initial contact. Randomization perturbs coefficients exclusively within the governing differential equations already hardcoded in the physics engine. It cannot synthesize missing physical phenomena such as gearbox backlash, joint cable compliance, Stribeck friction, or sensor transport jitter. When dynamically incompatible virtual plants produce indistinguishable observations, a memoryless policy cannot choose separately for each plant; average-return optimization may favor a slow compromise or may sacrifice low-probability cases. While recurrent or history-conditioned architectures can adapt to wide parametric variations by estimating system properties online, even an adaptive policy cannot identify or compensate for physical mechanisms that never existed in its training simulator. **Expanding parameter randomization bounds merely shifts the probability distribution across existing simulator equations; it cannot synthesize unmodeled physical dynamics.**

Architect's countermeasure: Calibrate baseline physics engine parameters against real hardware test fixtures before introducing domain randomization. Explicitly document whether parameter distributions derive from empirical instrument measurements or heuristic curricula such as automatic domain randomization. Enumerate known physical omissions, including transmission elasticity and actuator deadbands, in the policy manifest's omission log, ensuring that hardware-level safety watchdogs bound operations within verified modeling assumptions.

Fallacy: *Assuming multi-step generative decoders can execute synchronously within inner actuator control loops.*

Aiming to capture multimodal trajectory distributions observed in offline demonstration logs, an engineer replaces the single-pass ACT feedforward decoder in `latch-bc-01` with an iterative 16-step diffusion denoising head. The team deploys the new architecture synchronously into the robot's real-time inner control loop, scheduling a complete iterative sampling cycle for every 25 ms actuator setpoint. Under the hardware inference budget evaluated in section 6.2.6, executing all denoising steps consumes 48 ms, exceeding the allowable control step deadline. Because the synchronous dispatch pipeline stalls awaiting model convergence, the system routinely misses setpoint deadlines. The low-level controller must either freeze on stale command setpoints or trip safety watchdogs on every iteration overrun. The weight-streaming floor of section 3.4.1 bounds each evaluation from below, and section 3.5.1 shows that re-reading the action head on every evaluation can dominate the decoder's cost. Each of the 16 dependent evaluations pays that floor, so the sum misses the setpoint deadline before scheduling jitter is even counted. **Deploying multi-step generative policies into high-rate actuator loops requires decoupling iterative trajectory generation from real-time setpoint tracking through asynchronous execution contracts.**

Architect's countermeasure: Decouple iterative policy inference from high-frequency actuator control loops whenever measured inference latency approaches the control period. Formally partition the pipeline into an asynchronous planner generating trajectory chunks and an inner controller tracking them under qualified execution leases. Explicitly track prediction horizons, executed setpoint prefixes, and proposal renewal deadlines. Buffering trajectory chunks never grants permanent operational authority: the execution engine continuously validates physical safety invariants and triggers safe stopping profiles whenever a lease expires without a fresh proposal.

6.10 Summary

A policy's weights keep what its training experience rewarded and forget which experience that was, so each regime is best remembered by the dependency it leaves behind. Demonstrations leave a dependency on coverage: a point-regression head averages valid modes into an action no one chose, and even a multimodal head drifts into states the demonstrations never recovered from. A reward-labeled log leaves a dependency on the support of the logged actions, and a pretrained base on someone else's embodiment. Trial on the machine pays for its experience in calendar time, winding heat, and drivetrain life. Simulation pays almost nothing per trial but inherits every approximation in its contact solver and every mechanism it omits. Fine-tuning on the machine creates a new checkpoint that needs its own behavioral evidence and review of affected assurance premises. None of these dependencies shows in a validation loss or a simulated return.

Orthogonal to the source of experience, every policy must deliver its proposal inside the machine's deadline T_{deadline}. The decoder that represents several modes buys that representation with dependent evaluations, and what decides whether it can run is renewal slack against the jitter allowance, not mean latency. When slack runs out, the chunk lease, not the decoder, bounds how long an admitted prefix keeps moving the arm.

The policy manifest gathers these dependencies into one claim with stated limits. Its declared ODD is the supported cells of the data join minus every state the omission log excludes, never a region read off a training curve. For the latch, that rule sends `latch-bc-01` to evaluation at the nominal plate and the guarded approach. It holds the simulation-refined checkpoints until paired probes bound the contact and timing omissions. It also lets `latch-bc-recovery-02` carry a wider claim that new recovery experience has to earn. An admitted action crossing the causal boundary carries no guarantee beyond the evidence that qualifies it, and the load-bearing relations behind that evidence are collected in table 6.5.

Table 6.5: Key equations of policy training: Each load-bearing equation of the chapter, the limit it bounds or the decision it settles, and the equation where it is derived.

Equation	What it bounds or decides	Derived in
$\pi^*(\mathbf{o}) = \mathbb{E}[\mathbf{a} \mid \mathbf{o}]$	The squared-error optimum of a point-regression policy, which lies between demonstrated modes	equation 6.3
$Q^\pi(s,\mathbf{a}) = r(s,\mathbf{a}) + \gamma\,\mathbb{E}[Q^\pi(s',\mathbf{a}')]$	The value recursion that offline and policy-gradient learning estimate from samples	equation 6.2
$t_{\text{infer}} = t_{\text{enc}} + \sum_{k=1}^{N_{\text{eval}}} t_{\text{pass}} \le T_{\text{deadline}}$	Whether a decoder fits the setpoint step synchronously or the renewal period asynchronously	equation 6.4
$T_{\text{wall}} = N\,(t_{\text{rollout}} + t_{\text{reset}})/N_{\text{rig}}$	The calendar of a physical training campaign, which rigs divide but wear does not follow	equation 6.5
$F_{\text{unbraked}} = v\sqrt{k_{\text{contact}}\,m}$	The peak force of a mass coasting into a contact, which a soft penalty spring understates	equation 6.7
$k_{\text{eff}} = (1/k_{\text{latch}} + 1/k_{\text{robot}})^{-1}$	The robot-side stiffness above which a coasting approach exceeds the latch limit	equation 6.8
$t_{\text{yield}} = F_{\text{lim}}/(k_{\text{contact}}v)$	The time from touch to the force limit under constant drive, which confines the declared ODD to the guarded approach	equation 6.9
$J_{\text{DR}}(\theta) = \mathbb{E}_{\xi\sim p(\xi)}[J(\theta;\xi)]$	Return averaged over the parameters a simulator models, never over mechanisms it omits	equation 6.10

Equation	What it bounds or decides	Derived in
$\mathbf{u}_{\text{nom}} = \pi_{\text{base}} + g(s)\,\Delta\pi_\theta$	Where a learned residual may act, and so what the regression evidence must cover	equation 6.11

Key Takeaways: Policy synthesis and sim-to-real transfer

- **Every regime leaves a dependency no training score shows:** Offline loss and simulated return reward compliance with training data and numerical solvers rather than real-world physics. Candidate model rankings frequently invert when transferred to hardware. When an operating condition violates an unmodeled dependency, it produces a predictable failure signature that identifies exactly which runtime monitor the permission path must enforce.
- **Mode choice and recovery are separate problems:** Point regression can average valid, mutually exclusive demonstrations into an unviable intermediate action, whereas compounding execution drift pushes the system into states that the demonstrations never sampled. Multimodal decoders resolve action ambiguity; targeted recovery experience and closed-loop data resolve distributional drift. Independent runtime limits remain mandatory for both.
- **Decoder latency must fit the proposal contract:** Iterative denoising and deep autoregressive evaluations substantially increase compute latency and memory bandwidth pressure. When a policy cannot generate actions within the synchronous deadline, asynchronous action-chunk playback requires a validated hardware lease and deterministic fallback behavior. An expressive action distribution earns no extra execution time at the actuator.
- **Training information must survive the sensor boundary:** Privileged simulation states, such as ground-truth contact wrenches or global object poses, accelerate training, but the deployed student policy observes only noisy onboard sensor streams. Observation histories disambiguate physical variations only when those conditions leave distinct temporal signatures. Model distillation never removes the necessity for hardware calibration and empirical transfer testing.
- **Randomization covers parameters, not missing mechanisms:** Domain randomization widens policy robustness across modeled simulator parameters, but it cannot bridge physical effects that the numerical solver neglects. Backlash, unmodeled compliance, and sensor transport delays remain absent regardless of parameter range, requiring explicit entries in the omission log and channel-by-channel probe characterization on hardware.
- **Fine-tuning spends wear and reopens qualification:** On-robot trials consume component life, introduce thermal stress, and generate wear that cannot be parallelized across separate test rigs. Behavioral evidence gathered for an earlier checkpoint does not qualify an updated model, and independent permission properties require an integration check of their unchanged premises. A gated residual architecture strictly confines where learned adaptations take effect, but the integrated controller still demands thorough regression testing and hardware-level execution limits.
- **A wider declared ODD needs new support:** The policy manifest records dataset digests, training hyperparameters, and unmodeled simulator phenomena, explicitly excluding operational domains affected by unverified gaps. Scaling simulator throughput or pretraining a foundation model adds no supported cell to the declared ODD. Broadening the deployment boundary requires attributable physical recovery data and rigorous closed-loop hardware qualification.

The declared ODD is where the second law, endogenous data (principle 2), becomes a field in a record. A policy chooses its next observations by acting, so the states it meets in service are not the states its training visited, and no regime in this chapter escapes that. Cloning meets it as drift out of the demonstrated tube, learning by trial on the machine meets it as calendar time and wear

spent visiting the policy's own states, and simulation meets it as states whose dynamics the solver invented. The manifest does not remove the dependency. It writes down where the policy's own evidence ends, so that everything beyond that line is treated as a claim still to be earned rather than as competence the weights happen to display.

When a candidate looks better than its evidence, the useful question is which of its states were visited on this machine and which were only scored. A wider declared ODD is bought with experience that joins to new cells, and choosing a training regime is choosing what that experience costs: an operator's hours, a drivetrain's life, or a probe campaign that closes a simulator's omissions.

6.11 Historical Perspective and Further Reading

The mathematical foundations of modern policy optimization originate in classical dynamic programming. Bellman (1957) introduced the principle of optimality and the recursive decomposition that defines value functions, while Howard (1960) formulated policy iteration to solve discrete Markov decision processes. Parallel to dynamic programming, learning from demonstration developed along an empirical path in robotics. Pomerleau's ALVINN demonstrated early autonomous road following by training a shallow artificial neural network to map camera images directly to steering angles (Pomerleau 1989). ALVINN quickly exposed the vulnerability of pure behavioral cloning to covariate shift and compounding error. The network trained exclusively on demonstrations from an attentive human driver, so it never encountered the off-center heading angles produced by its own steering mistakes, which led to catastrophic departures from the roadway (Pomerleau 1991). Ross et al. (2011) formalized this failure mode through the Dataset Aggregation (DAgger) algorithm, proving no-regret reduction bounds by querying an expert supervisor on the actual state distribution induced by the learner's policy. When demonstrations encompass multiple valid strategies, naive regression averages distinct modes into invalid intermediate actions. Mixture density networks (Bishop 1994) resolved this multimodality for low-dimensional outputs, and diffusion policies (Chi et al. 2025) generalized the approach to high-dimensional action trajectories, executing action chunks with receding horizons on physical manipulators.

Algorithmic reinforcement learning split into two primary branches: policy-gradient methods originating with REINFORCE (Williams 1992) and value-based temporal-difference learning formalizing around Q-learning (Watkins and Dayan 1992). Both paradigms measure sample complexity in abstract environmental transitions, ignoring the physical costs of real-time execution. On a real robot, every trial consumes calendar time, strains mechanical linkages, overheats actuators, and demands human intervention for environment resets. Large-scale physical grasping campaigns (Levine et al. 2018; Kalashnikov et al. 2018) demonstrated that collecting hundreds of thousands of real-world interactions requires dedicated robot farms running continuously for months under heavy maintenance. To bypass this physical bottleneck, robotic learning shifted sample generation into simulation. Proximal Policy Optimization (Schulman et al. 2017) established itself as the standard policy learner by clipping the update objective to discourage excessive policy drift. Simultaneously, physics engines co-located directly on hardware accelerators eliminated host-to-device communication bottlenecks, enabling thousands of parallel rollouts on a single workstation and delivering billions of simulated timesteps per day (Makoviychuk et al. 2021; Rudin et al. 2022).

Simulated rollouts solved the sample-throughput bottleneck, but they elevated the reality gap into the central engineering challenge. Policies trained inside idealized physics often fail catastrophically when deployed against the unmodeled friction, latency, and compliance of physical hardware. Domain randomization addressed this discrepancy by treating the real world as an unobserved draw from a randomized distribution of simulated environments, perturbing visual features (Tobin et al. 2017) and inertial dynamics (Peng et al. 2018). The OpenAI Dactyl project (Andrychowicz et al. 2020) showed that broad dynamics randomization paired with a recurrent policy could reorient a block in a dexterous hand trained entirely in simulation, while reporting the unsimulated failures of the physical hand, chiefly hardware breakage under load. Its successor extended the approach to solving a Rubik's cube with automatic domain randomization (OpenAI et al. 2019). Classical system identification (Ljung 1999) found new life inside neural pipelines through learned actuator networks that capture non-ideal actuator dynamics and drive compliance (Hwangbo et al. 2019). More recently, generalist foundation policies trained across diverse robotic platforms (Kim et al. 2024; Black et al. 2024) use pooled fleet demonstrations to cross-train visual and motor representations.

However, borrowing cross-embodiment data introduces unverified physical assumptions and subtle distributional shifts. The policy manifest introduced in this chapter provides the rigorous systems contract required to track these lineage dependencies, documenting simulation distributions, sensor bounds, and training assumptions so that hardware deployment can be audited and verified.

Six foundational works trace the trajectory from classical dynamic programming to modern sim-to-real policy deployment:

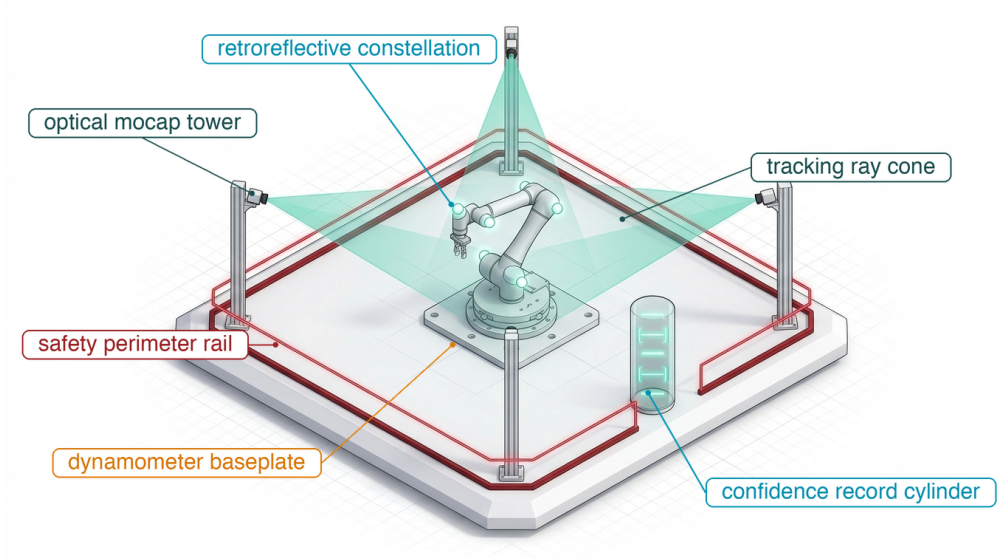
- Sutton and Barto (2018) provides the definitive derivation of Markov decision processes, value functions, and policy-gradient theorems from first principles, serving as the core reference for the reinforcement learning formulations in section 6.2.1.
- Ross et al. (2011) establishes the theoretical bound on compounding error in sequential prediction, introduces the DAgger reduction algorithm, and outlines the recoverability assumptions that section 6.2.4 evaluates on physical robot hardware.
- Chi et al. (2025) introduces score-based diffusion models for action sequence generation on physical manipulators, establishing the receding-horizon chunk execution and iterative denoising computational budgets analyzed in section 6.2.6.
- Peng et al. (2018) focuses domain randomization on unmodeled physical dynamics rather than visual textures, demonstrating that recurrent policies implicitly identify system dynamics across randomized friction, mass, and damping distributions to achieve robust sim-to-real transfer.
- Andrychowicz et al. (2020) offers one of the most comprehensive engineering case studies of an end-to-end sim-to-real deployment, detailing the distributed training pipeline, dynamics randomization ablations, and physical maintenance challenges encountered while reorienting a block with an anthropomorphic hand.
- Ljung (1999) provides the definitive treatise on classical system identification, detailing the input-output parameter estimation techniques, excitation design, and validation metrics that ground modern actuator modeling and paired physical probing.

What's Next: From policy manifests to evidence

Twenty successful consecutive trials provide limited statistical confidence for a demonstration checkpoint whose manifest declares a single nominal cell and inherits none of the simulator's omissions. Chapter 7 takes the formal policy manifest as its contractual input. It opens with an illustrative campaign of twenty nominal door trials of `latch-bc-01`, the single cell its manifest declares, and evaluates the bound so small a physical sample can establish. It then evaluates `latch-bc-recovery-02` through nominal regression and displaced-plate trials to decide whether the new recovery experience earns the wider claim its manifest requests within the guarded approach.

7

Closed-Loop Evaluation

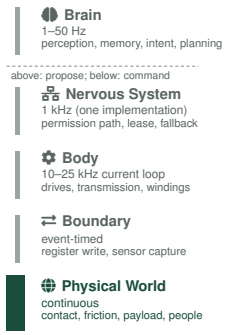


7.1 From Clean Runs to Bounded Claims
7.2 What a Physical Success Rate Measures
7.3 Closed-Loop Dynamics
7.4 Evidence Regimes
7.5 Fixing the Trial Protocol
7.6 Confidence from a Finite Campaign
7.7 Trial Budgets, Stopping Rules, and Comparisons
7.8 The Evaluation Record
7.9 Verification Ceilings and the Exposure Wall
7.10 Evaluation Pitfalls and Metric Masking
7.11 Fallacies and Pitfalls
7.12 Summary
7.13 Historical Perspective and Further Reading

Purpose

Why can a robot pass every trial and still lack the evidence needed for deployment?

A robot that repeatedly opens one door has supplied evidence about that door, its own hardware, and the conditions of those trials. It has not established how it will behave at a displaced latch, with a different payload, or after its sensors drift, and a flawless record from a short campaign remains consistent with a policy that fails a substantial fraction of its attempts. Its admitted actions also change the observations it receives, so agreement with recorded demonstrations cannot substitute for testing the resulting motion. Physical trials expose that feedback, but each consumes time and mechanical life, every further factor of ten in demonstrated reliability costs roughly ten times the trials, and protective interventions stop some trajectories before their consequences unfold. A team that reads a clean record as readiness releases the machine into conditions no trial visited and credits the learned policy with protection its supervisors supplied. Evaluation exists to prevent that error. It ties each task outcome to the conditions sampled, keeps every intervention in the record, states the uncertainty that remains, and names the uncovered conditions the rest of the machine must guard. Because actions cross an irreversible causal boundary, evidence of policy competence must remain distinct from permission to move.



Learning Objectives

- Interpret physical success rates as conditional evidence about a specified machine, environment, and trial protocol
- Interpret completion and intervention metrics without crediting the policy for protection supplied by its supervision
- Compare open-loop prediction accuracy with closed-loop behavior shaped by the policy's own actions
- Select the evidence regime (replay, simulation, shadow operation, or constrained physical testing) that can answer a given evaluation question
- Design a predeclared trial protocol with fixed outcome rules and full-denominator accounting of interventions and aborts
- Calculate confidence bounds, trial counts, and test time for a zero-failure campaign under stated sampling assumptions
- Construct an evaluation record that bounds its operational claim and identifies coverage gaps and required protections
- Evaluate which rare failures remain unsupported by feasible testing and the available statistical assumptions
- Apply sequential stopping rules and two-policy comparisons without inflating the error rate a campaign declares

7.1 From Clean Runs to Bounded Claims

The checkpoint `latch-bc-01` arrives from offline imitation learning with a policy manifest that declares exactly one supported condition, the nominal strike plate at the guarded approach. The manifest marks where the policy's training evidence ends, and evaluation asks what physical trials can establish inside that line. The recovery candidate `latch-bc-recovery-02`, whose wider claim new experience still has to earn, receives its own campaign later in the chapter (section 7.8.1). `latch-bc-01` is flashed onto the warehouse mobile manipulator for a nominal validation campaign. In each trial, the mobile base begins 2 m (illustrative; see the Reader Guide) from the spring-latched cage door, drives forward, and comes to a complete halt. With the base at rest, the manipulator executes a guarded approach at 0.03 m/s, unlatches the mechanism, and opens the door without crossing the 15 N contact tripwire or drawing a human intervention. Suppose the robot completes twenty consecutive runs this way, an empirical success rate of $\hat{p} = 1.00$ (100 percent). On a static vision or language benchmark, an unblemished score might suggest that the model is ready for production. On physical hardware, twenty clean runs leave the deployment decision open. They establish only a wide lower confidence bound, one that a policy failing a substantial fraction of its attempts can still satisfy, and they reveal nothing about failure modes too rare to appear in twenty draws.

↩ The policy manifest, which records how a checkpoint was produced and the operating domain it declares, is established in section 6.8.

The gap between that perfect score and a deployable claim comes from three ways in which physical evaluation departs from the static protocol of machine learning. In that protocol, engineers draw a held-out dataset once from a stationary distribution, score each example independently, and treat the aggregate as a faithful descriptor of model quality. First, a physical policy selects its own test distribution. Every admitted action alters the environment and determines the observations that follow, so no pre-recorded dataset can stand in for the states the policy will visit. This is the second law, endogenous data (principle 2), acting on evaluation. The same feedback that pushes a cloned policy off its demonstrations also decides which states a test campaign samples. Second, every trial consumes test-cell time, battery charge, and mechanical life, which keeps sample counts small and each sample expensive. Third, a physical trial delivers kinetic energy across the causal boundary, and that energy cannot be recalled. When an interlock, a contact tripwire, or a safety operator aborts an errant motion before damage occurs, the trial ends early and leaves a censored observation that is neither an unequivocal success nor an unmitigated crash. Aerospace, automotive, and industrial automation engineers combine physical tests with model-based arguments about controller stability

and protection. The learned proposer in this campaign arrives without such a certificate, so its task reliability must be bounded by finite empirical sampling under the declared conditions.

Closed-loop evaluation therefore asks what a finite campaign of physical trials can establish about a learned proposer, and what it must leave for the rest of the machine to handle. Four obligations answer that question, and they order this chapter. A success rate is meaningful only when bound to a declared population of hardware states, environmental conditions, and trial procedures, with every protective intervention kept out of the policy's credit (section 7.2). The evidence must come from a regime that can expose the failure in question, which for most physical failures means routing proposals through the real permission path rather than scoring them against logs (section 7.3 and section 7.4). A finite record becomes a defensible bound only under a protocol fixed before the first run, and each additional "nine" of demonstrated reliability costs roughly ten times the trials (section 7.5 through section 7.7). Finally, the evaluation record, the link in the record chain after the policy manifest, must state which conditions were tested and which were not. It must also turn every untested condition, together with every hazard too rare for any campaign to sample, into an obligation for the runtime permission path (section 7.8 and section 7.9).

The first obligation begins with the twenty runs themselves. Before that perfect fraction can mean anything, the campaign must fix its outcome rules, define the operational population it sampled, and separate what the policy achieved from what supervision supplied. A defensible evaluation record never credits the learned proposer with successes that hardware interlocks or supervisory overrides actually purchased.

7.2 What a Physical Success Rate Measures

A latch-manipulation policy that reliably opens a door during morning bench tests can jam repeatedly when confronted with a strike plate displaced by only three millimeters. Its measured success rate does not belong to the neural network weights in isolation; it characterizes the coupled dynamical system formed by the policy, the mechanical plant, the onboard compute platform, the physical environment, and the trial protocol itself. While a reproducible offline benchmark score indicates how well a model fits a frozen dataset, a physical success rate measures how the embodied closed-loop system behaves across a distribution of physical executions.

Evaluating a policy requires executing a sequence of discrete physical runs. A single trial is fully determined only when every physical parameter capable of altering the state trajectory is either explicitly controlled or systematically sampled. Formally, a single trial condition is the tuple $c = (\mathbf{x}_0, e, m, \pi_\theta, \kappa)$, drawn from an operational testing distribution $\mathcal{D}$:

- The initial state vector $\mathbf{x}_0$ specifies the machine's starting condition, including its spatial pose (position and orientation), linear and angular velocities, and joint encoder positions.
- The environment vector e captures external physical variables, such as the tire-floor friction coefficient (μ), ambient illuminance, obstacle geometry, and ambient air temperature.
- The machine state m captures physical and hardware properties sampled from the fleet manufacturing and wear distribution $\mathcal{M}$. This encompasses unit-to-unit mechanical tolerances, battery state of charge, payload mass, tire wear, lost motion and compliance in the arm's transmissions, and sensor calibration matrices.
- The computational configuration π_θ represents the frozen policy weights θ, numerical quantization formats (such as FP16 or INT8), real-time thread scheduling deadlines, and software driver versions.
- The termination protocol $\kappa = (T_{\text{deadline}}, \mathcal{G}_{\text{goal}}, \phi)$ defines the execution deadline T_{deadline}, the spatial goal region $\mathcal{G}_{\text{goal}}$, and the binary evaluation predicate ϕ that maps the realized trajectory to a terminal outcome $Y \in \{0, 1\}$.

In static machine learning benchmarks, inputs (x_i, y_i) are sampled independently from a static data distribution $p_{\text{data}}(x, y)$. In Physical AI, the continuous trajectory $\text{Traj}(c) = (\mathbf{x}_0, \mathbf{u}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{x}_T)$ is generated sequentially by the closed-loop physical state transition kernel:

$$\mathbf{x}_{t+1} \sim \mathcal{T}(\mathbf{x}_t, \mathbf{u}_t; m, e), \quad \mathbf{a}_t = \pi_\theta(\mathbf{x}_t), \quad \mathbf{u}_t = \text{Gate}(\mathbf{a}_t, \mathbf{x}_t)$$

Here Gate stands for the permission path, which admits, clips, or replaces each proposal $\mathbf{a}_t$ before it becomes the applied command $\mathbf{u}_t$. Its permission and controller parameters remain fixed and

are archived with the evaluated configuration; changing them changes the evaluated closed loop. The true success probability under this evaluation protocol is the expected value of the binary task outcome averaged over the joint distribution of trial conditions:

$$p(\mathcal{D}) = \mathbb{E}_{c\sim\mathcal{D}}[\phi(\text{Traj}(c))] \tag{7.1}$$

where $\text{Traj}(c)$ is the physical trajectory generated under condition c. Altering floor friction, ambient temperature, or battery voltage modifies the physical transition kernel $\mathcal{T}$ even when the neural network weights θ remain untouched. The operational testing distribution $\mathcal{D}$ is therefore an inseparable component of the quantity being estimated.

↪ The formal integral over the closed-loop trajectory distribution, including the plant transition kernel and the permission map, is developed in section A.5.5.

In test engineering, the abstract distribution $\mathcal{D}$ is realized through a **stratified test matrix**[1] across controllable parameters, systematically varying payload mass, approach speed, illumination, and initial spatial standoff, while logging uncontrollable environmental parameters such as ambient temperature and floor friction. Unstratified random sampling across the parameter space oversamples benign nominal states and leaves critical boundary regions unvisited. Each stratum of the matrix (one payload, one approach speed, one plate position) is then its own population for the statistical bounds that follow, with its own $\mathcal{D}$ and its own success probability. A claim about the whole matrix needs explicit weights across strata, which the evaluation record of section 7.8 declares before the first run.

[1] **Design of Experiments**: Fisher's agricultural field trials introduced randomization and blocking so that variation in the soil could not masquerade as a treatment effect (Fisher 1935). A stratified test matrix blocks on payload, approach speed, and lighting for the same reason, and randomizing the run order keeps winding heat and spring relaxation from entering the comparison as if they were properties of the policy (section 7.7.4).

Definition 7.1: Closed-loop evaluation estimand

Closed-loop evaluation estimand is the expected physical task outcome $p(\mathcal{D}) = \mathbb{E}_{c\sim\mathcal{D}}[\phi(\text{Traj}(c))]$ defined over the joint distribution of initial states, mechanical plant wear, environmental physics, frozen policy weights, and evaluation predicates:

$$c = (\mathbf{x}_0, e, m, \pi_\theta, \kappa) \sim \mathcal{D}$$

rather than an intrinsic property of the neural policy weights evaluated in isolation.

1. **Significance**: In static machine learning, held-out accuracy is treated as a property of the model π_θ. In physical AI, substituting a worn tire, letting battery voltage sag, or shifting ambient illuminance alters the physical state transition function without changing a single weight. The sampling distribution of operational conditions $\mathcal{D}$ is an inseparable part of the quantity being estimated.
2. **Distinction**: Unlike open-loop validation metrics (such as validation loss, cosine similarity, or mean squared trajectory error) that score predictions against frozen recorded states, the closed-loop estimand integrates over the endogenous feedback loop. In that loop, past actions physically move the machine and directly change the future sensor observations it receives.
3. **Common pitfall**: Reporting a single success rate percentage without predeclaring the support, density, and environmental bounds of the test distribution $\mathcal{D}$. Any physical policy can be made arbitrarily successful by testing exclusively in benign, unperturbed states.

7.2.1 Hardware instance variance and environmental unobservables

Freezing the neural network weights π_θ and the termination protocol κ fixes only two of the five entries in the trial condition c; the plant parameters m and environmental parameters e still vary from machine to machine and from morning to afternoon. On a mobile manipulator, this dispersion comes from two sources: **hardware instance variance** across individual builds and **environmental unobservables** in the operating workspace.

First, robotic production lines do not manufacture identical copies. Nominally identical units diverge in lost motion, thermal behavior, and controller response (section 5.10.3). The tolerances that matter for evaluation also include drivetrain compliance and hysteresis that grow with wear (Harmonic Drive LLC 2006), link deflection that shifts the tool center point, and camera-mount offsets between the sensor frame $\mathcal{F}_{\text{cam}}$ and the base frame $\mathcal{F}_B$. They further include torque constants

K_t that vary across windings and drive voltage that loses headroom against back-electromotive force (back-EMF) as the battery discharges. Each of these changes the trial condition without changing a weight, so qualification campaigns characterize every unit's lost motion, joint stiffness, and camera extrinsics on an inspection bench rather than assuming nominal catalog values.

Second, environmental unobservables alter physical interaction dynamics without changing camera pixels in ways that a neural network can easily recognize. Ambient daylight shifting across a warehouse alters illuminance, causing camera auto-exposure routines to lengthen shutter times and inducing motion blur during base turns. Direct solar glare at dawn or dusk saturates complementary metal-oxide-semiconductor (CMOS) sensor arrays, washing out optical contrast around the latch interface. On the floor, dust accumulation and ambient humidity alter the tire-floor friction coefficient (μ), lengthening braking distances and corrupting wheel odometry. Within the manipulator arm, repeated door openings generate Joule heating (I^2R) in motor windings and warm harmonic drive lubricants, decreasing viscous damping and causing thermal expansion across structural links. Consequently, the tool center point meets the latch with a millimeter-scale spatial offset on a hot afternoon compared to a cold morning.

Trial reset procedures introduce another major source of unintended drift in c. When human operators reposition the mobile base after a difficult or aborted run, subconscious bias often leads them to favor forgiving, nominal configurations, systematically excising difficult boundary states from the sample. Conversely, an automated reset routine that drives the identical recovery path repeatedly heats actuator windings cyclically, introducing progressive thermal degradation across runs. Worst of all, an operator who catches a stumbling robot mid-fall and discards the run as an invalid trial erases the most informative corner-case failures from the evaluation record.

Statistical evaluation must therefore maintain a strict distinction between sampling variation and shifts in the underlying estimand. When N independent trials are drawn from a stationary distribution $\mathcal{D}$, the sample proportion $\hat{p} = \frac{1}{N}\sum_{i=1}^{N} Y_i$ fluctuates around the true expectation $p(\mathcal{D})$ due to ordinary binomial noise, which standard statistical estimators bound. However, loading a payload onto the tote rack or moving the door's strike plate transitions the experiment to an entirely different distribution $\mathcal{D}'$ with its own success probability $p(\mathcal{D}')$. Pooling runs across these two conditions without declaring their sampling weights conflates distinct physical configurations into an uninterpretable average. Independent draws from a predeclared mixture support a claim about that mixture; a quota-based stratified matrix instead requires separate inference within each cell and explicit weighting for an aggregate claim. Fixing the condition tuple c settles which population a campaign samples, but a single binary outcome $Y \in \{0, 1\}$ still hides whether the learned proposer succeeded on its own or was rescued by the permission path.

7.2.2 Intervention-based physical metrics

Even when the trial condition c is fixed, a single binary pass rate cannot distinguish a policy that operated the door latch on its own from one that was rescued on every attempt by permission-path clips or remote teleoperators. Consider four divergent operational outcomes during a door-opening task:

- A policy proposes an excessive contact force, but the permission path clips the command, allowing the door to open smoothly while remaining within mechanical limits.
- The permission path detects an impending joint limit and halts the manipulator entirely, preserving hardware safety while leaving the task incomplete.
- A human safety operator intervenes via remote teleoperation to clear a jammed latch pin, after which the robot resumes autonomous execution and completes the run.
- The raw policy executes the entire trajectory unassisted, completing the manipulation task within nominal limits without intervention.

Every protective actor in these four scenarios other than the human operator is the permission path, acting by clipping a proposal, replacing it, or stopping the machine. The first scenario shows a well-functioning integrated machine where permission-path clips compensate for raw proposer imperfections, whereas the second protects the hardware at the expense of task completion. The third achieves the objective only by drawing on costly human labor, leaving only the fourth as genuine proposer autonomy.

Evaluating physical AI systems therefore requires decomposing performance into three independent channels: whole-machine task completion within operational limits, raw proposer autonomy, and supervisory intervention frequency. The task-completion rate, the fraction of trials whose outcome Y_i (defined in the first item below) equals 1, measures the first channel. The autonomous completion rate measures the second, and mean time between interventions and intervention density measure the third. Reset latency belongs to no channel; it prices what each outcome costs the test cell. Runtime safety adjustments are an intended feature of the architecture, so a small corrective clip by the permission path should not brand a completed run as a system failure. Nevertheless, tracking proposer autonomy separately reveals how often the learned model attempts to violate physical boundaries. Telemetry pipelines must log adjustments, command rejections, protective stops, and human overrides using predeclared event-counting rules, ensuring that a sustained safety clamp over several seconds is recorded as a single event rather than hundreds of spurious high-frequency faults.

1. **Autonomous completion rate (ACR)**: Let $n_{\text{int}_i} \in \{0, 1, 2, \dots\}$ be the total count of supervisory interventions during trial i, whether initiated by human safety operators, the permission path's clips and stops, or emergency tripwires, and let $Y_i \in \{0, 1\}$ denote goal achievement within the execution deadline T_{deadline}. The unassisted autonomous success indicator is:

$$A_i = \mathbb{I}(Y_i = 1 \wedge n_{\text{int}_i} = 0)$$

   ACR is the sample proportion of trials completed entirely without supervisory intervention:

$$\text{ACR} = \frac{1}{N} \sum_{i=1}^{N} A_i = \frac{N_{\text{unassisted}}}{N}$$

   ACR measures strict raw proposer autonomy: a single recorded intervention forces $A_i = 0$. In contrast, the whole-machine outcome Y_i can remain 1 when the permission path clips the proposer's commands and the task still completes within service bounds.

2. **Mean time between interventions (MTBI)**: For any declared intervention class, MTBI quantifies the mean active operating duration between supervisory events:

$$T_{\text{MTBI}} = \frac{T_{\text{active}}}{N_{\text{int}}} = \frac{\sum_{i=1}^{N} t_{\text{run}_i}}{\sum_{i=1}^{N} n_{\text{int}_i}}$$

   where t_{run_i} is the active operating duration of trial i and $N_{\text{int}} = \sum_{i=1}^{N} n_{\text{int}_i}$ is the total count of recorded interventions within that class. MTBI reflects the operational burden imposed by supervisory systems. Frequent automated filter adjustments signal aggressive policy proposals or model distribution shift, whereas frequent manual recoveries directly drive up staffing costs. Neither a favorable MTBI nor an absence of physical damage proves that a policy is ready for unattended deployment.

3. **Intervention density (D_{int})**: In spatial navigation and continuous trajectory tracking, interventions must be normalized against physical travel distance or discrete execution steps:

$$D_{\text{int}} = \frac{N_{\text{int}}}{\sum_{i=1}^{N} d_i} \quad \text{[interventions per kilometer]} \quad \text{or} \quad D_{\text{int}} = 1{,}000 \times \frac{N_{\text{int}}}{\sum_{i=1}^{N} N_{\text{steps},i}} \quad \text{[interventions per 1,000 st}$$

   where d_i is the physical path distance traversed during trial i, and $N_{\text{steps},i}$ is the number of control cycles executed. Normalizing by exposure makes intervention rates comparable across routes or operating conditions. Locating concentrated degradation also requires retaining each event's location and condition and comparing these rates across strata, such as narrow doorways or tight mechanical latches.

4. **Reset latency (T_{reset})**: In physical qualification campaigns, the primary bottleneck in testing throughput is governed by the time required to restore the robot and workspace after a trial completes or aborts:

$$T_{\text{reset}} = t_{\text{physical_restore}} + t_{\text{calibration}} + t_{\text{telemetry_flush}}$$

The standard door-opening run requires 25 s of active execution. While its nominal reset cycle is brief (section 7.7.2), a run that jams the latch pin requires an operator intervention of approximately 60 s (illustrative) to clear the jammed pin, recalibrate joint encoders, and return the mobile base to its initial standoff pose. For jammed runs, this manual reset consumes 70.6 percent of total test-cell occupancy. High reset latencies create organizational pressure to cut corners, for instance by skipping motor thermal cooldowns or manually nudging the robot into easier initial poses, and each shortcut breaks the statistical stationarity of the campaign.

Engineers must not conflate MTBI with mean time between failures (MTBF). While an MTBF metric tracks functional breakdowns or mechanical failures, MTBI, ACR, and D_{int} capture supervisory burden and proposer autonomy within an operating envelope. A robot fleet can maintain an unblemished record of zero hardware damage simply because vigilant human safety drivers or a conservative permission path intervene whenever an unsafe state nears. Recording a run that was truncated or guided by external supervision as an unassisted operational success masks policy shortcomings behind the protective capability of the safety system. Section 7.4 shows why such runs are censored observations and names this error the censored trial fallacy.

Evaluation protocols keep these metrics separate so that engineers can observe the trade-offs between safety filtering, autonomous capability, and operational throughput. These outcome channels fix which quantities must be recorded across experimental trials. The next challenge is to design a closed-loop evaluation harness that exposes these behaviors systematically under dynamic feedback.

7.3 Closed-Loop Dynamics

A policy checkpoint that achieves the lowest validation loss on held-out teleoperation logs can drive the mobile base into a warehouse rack or jam the door latch on its first physical trial. Offline scoring resets the robot's state to the demonstrator's recording at every step, erasing the cumulative consequences of the policy's own actions. In standard deep learning workflows, model evaluation centers on empirical risk minimization over a static, held-out validation dataset $\mathcal{D}_{\text{val}} = \{(x_i, y_i)\}_{i=1}^{N}$:

$$\mathcal{L}_{\text{val}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\theta(x_i), y_i)$$

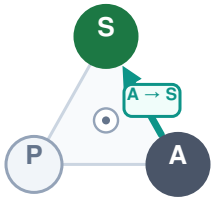
Whether the target metric is Top-1 classification accuracy, next-token perplexity, an F1 score, or coordinate regression loss, the evaluation pipeline remains passive and decoupled. The underlying data-generating process is feedforward and memoryless with respect to the model's outputs: mispredicting sample i exerts zero causal influence on sample $i+1$ ($\partial x_{i+1} / \partial \hat{y}_i = 0$).

Transporting this static paradigm to physical control creates the **offline evaluation illusion**, the unfounded assumption that lower validation loss, higher action cosine similarity, or smaller Mean Squared Error (MSE) on logged data guarantees superior physical execution. Offline loss metrics of the form $\mathcal{L}_{\text{val}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{a}_i^* - \pi_\theta(\mathbf{x}_i^*)\|^2$ evaluate candidate predictions exclusively along frozen demonstrator trajectories. Controlled benchmarks across simulated and physical manipulation tasks demonstrate this failure directly: the training checkpoint with the lowest validation loss performed substantially worse during closed-loop rollouts than alternative checkpoints, with task success frequently rising even as validation loss climbed (Mandlekar et al. 2021).

The decoupling that justifies offline scoring vanishes in physical AI. Admitted policy actions cross the causal boundary into mass, momentum, and non-smooth contact mechanics. Instead of assigning categorical labels, the policy outputs forces, torques, and velocity setpoints that physically perturb the plant, endogenously generating future sensory observations through the system dynamics ($\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$). These commands interact continuously with inertia, friction, and mechanical backlash. Consequently, a navigation policy can track recorded demonstrator trajectories with negligible regression error during offline playback, yet exhaust the vehicle's entire lateral clearance against a warehouse rack before traversing a third of the aisle. The failure stems directly from what each evaluation regime feeds back to the policy.

In an **open-loop evaluation**, a candidate policy π_θ is scored across a static dataset of recorded demonstrations $\mathcal{D}_{\text{demo}} = \{(\mathbf{x}_t^*, \mathbf{a}_t^*)\}_{t=1}^{S}$. At each discrete timestep t, the policy receives an expert state $\mathbf{x}_t^*$ and predicts an action $\hat{\mathbf{a}}_t = \pi_\theta(\mathbf{x}_t^*)$, which a standard loss function such as mean squared error or

cross-entropy compares against the recorded expert action $\mathbf{a}_t^*$. Because the harness then feeds the next recorded state $\mathbf{x}_{t+1}^*$ directly from storage and discards $\hat{\mathbf{a}}_t$ without ever executing it on hardware, the incoming state sequence remains strictly independent of prior predictions ($\partial \mathbf{x}_{t+1}^* / \partial \hat{\mathbf{a}}_t = \mathbf{0}$). In a **closed-loop evaluation**, by contrast, the policy proposes an action $\mathbf{a}_t = \pi_\theta(\mathbf{x}_t)$ from live sensory feedback. The deterministic permission path then validates that proposal against current plant state and safety invariants before dispatching an approved command $\mathbf{u}_t$ (or an intervention fallback) to the actuators. The plant then advances according to physical dynamics ($\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$), so every admitted command alters the physical state and shapes all subsequent sensory inputs across the causal boundary.



Admitted actions choose the next observation.

7.3.1 Compounding error and kinematic integration

That feedback path turns small one-step tracking residuals into growing state deviations, following the worst-case structure derived in section 5.3. If a policy deviates from expert actions with probability at most ϵ per step on recorded expert states, open-loop evaluation across a horizon of S steps accumulates an expected cost bounded by $S\epsilon$. In closed loop, however, an early deviation can steer the robot into unvisited regions of the state space for the remainder of the trajectory, driving the cumulative imitation cost to scale as $S^2\epsilon$ (equation 5.1). An open-loop benchmark can therefore understate closed-loop task divergence by a factor proportional to the operating horizon. This bound governs zero-one decision divergence rather than unbounded quantities such as contact force or structural yield. Classical kinematics supplies the physical mechanism. Uncorrected heading and steering biases integrate forward into lateral displacements that exhaust the robot's clearance envelope.

A policy whose own actions displace it from its training distribution is the second law at work on a test bench, and evaluation must quantify what that displacement costs on physical hardware. Because the mobile manipulator's base remains stationary during door manipulation trials, the platform's warehouse navigation stack supplies the clearest illustration of kinematic accumulation. Consider the base traversing a 30 m aisle at its 1.5 m/s drive limit, where the nominal expert trajectory tracks the centerline with heading $\theta_{\text{head}}^* = 0$ rad and lateral offset $y_{\text{base}}^* = 0$ m. Suppose the candidate navigation policy possesses a systematic heading bias of $\Delta\theta_{\text{head}} = 1.0^\circ$ (0.01745 rad) at every control step. The deterministic permission path admits each individual velocity command because, in isolation, a one-degree heading adjustment remains dynamically feasible while lateral clearance persists.

In an open-loop evaluation, the policy receives the demonstrator state $\mathbf{x}_t^* = ({x_{\text{base}}}_t^*, 0, 0)$ at each step and predicts forward velocity with a 1.0° steering offset. The offline scoring pipeline records a mean absolute angular error of 1.0° (0.01745 rad) and a mean squared error of $\text{MSE}_{\theta_{\text{head}}} = (0.01745)^2 \approx 3.05 \times 10^{-4}$ rad^2.[2] The number is small because it is measured in radians squared. More important, the replay score stays at that same per-step value however long the aisle is, because each next input returns to the centerline and the error never accumulates. During closed-loop execution, however, the robot's physical movement reflects previously admitted commands, allowing uncorrected heading errors to integrate continuously. Under planar unicycle kinematics[3] ($\dot{y} = v \sin\theta_{\text{head}} \approx v\Delta\theta_{\text{head}}$), a persistent angular offset integrates directly into a lateral displacement that grows linearly with the time t_{drift} since the base left the centerline:

$$y_{\text{base}}(t_{\text{drift}}) = \int_0^{t_{\text{drift}}} v \sin\Delta\theta_{\text{head}}\, dt' \approx v\Delta\theta_{\text{head}} t_{\text{drift}}$$

If the policy carries a persistent yaw-rate bias ω_{bias} (steering drift) rather than a fixed angular offset, the heading angle itself grows linearly over time ($\theta_{\text{head}}(t_{\text{drift}}) = \int_0^{t_{\text{drift}}} \omega_{\text{bias}}\, dt' = \omega_{\text{bias}} t_{\text{drift}}$). Integrating forward velocity v then yields a cumulative lateral deviation that compounds quadratically in time:

$$y_{\text{base}}(t_{\text{drift}}) = \int_0^{t_{\text{drift}}} v \sin(\omega_{\text{bias}} t')\, dt' \approx \frac{1}{2} v \omega_{\text{bias}} t_{\text{drift}}^2$$

This quadratic growth resembles the $S^2\epsilon$ imitation bound, but the two characterize distinct phenomena. The kinematic derivation integrates a systematic yaw-rate bias through continuous dynamics,

↩ How a policy's own actions shift the states it visits away from the demonstrator's occupancy is established in section 5.2.

2 | **Mean Masking of Steering Spikes**: Averaging squared steering error over thousands of straight-aisle frames also hides a brief, large prediction spike, which barely moves the mean. Whether such a spike matters depends on whether its integrated motion spends the clearance before the permission path responds, a question only a plant and timing model can answer.

3 | **Unicycle Model**: The simplest planar vehicle model, in which pose $(x_{\text{base}}, y_{\text{base}}, \theta_{\text{head}})$ evolves as $\dot{x} = v\cos\theta_{\text{head}}$, $\dot{y} = v\sin\theta_{\text{head}}$, $\dot{\theta}_{\text{head}} = \omega_z$. This formulation captures the essential nonholonomic constraint of the aisle: because a wheeled base cannot translate instantaneously sideways, any heading discrepancy directly translates forward velocity toward the rack.

whereas the imitation bound accounts for accumulated discrete decision costs under distribution shift. Both show small, bounded errors accumulating polynomially over an operational horizon. Neither excludes the worse case that section 5.3 described, in which a smooth imitator of a stable expert can compound its errors exponentially with horizon; the aisle example simply does not need that mechanism to fail.

For the constant 1.0° heading bias, the mobile base spans 100 cm within a 130 cm aisle. This geometry leaves $W_{\text{clear}} = 15$ cm of lateral clearance to the storage racks on either side. At full drive velocity, the kinematic drift consumes this safety buffer at:

$$t_{\text{impact}} = \frac{W_{\text{clear}}}{v \sin \Delta\theta_{\text{head}}}, \tag{7.2}$$

which evaluates to 5.7 s, after just 8.6 m of forward travel, less than a third of the way through the aisle. The same heading offset that replay scores at an angular mean squared error of 3.05×10^{-4} rad^2, because each next input returns to the recorded centerline, spends the entire 0.15 m side clearance in the physical loop (figure 7.1). The computed point is the unmitigated collision horizon. To halt before impact, the runtime permission path must intervene earlier, allowing for qualified detection, command and actuation response, and stopping motion. A protective stop makes strict proposer autonomy fail while preserving protection; an unprotected rack strike fails protection as well. Both expose a limitation of the policy that offline evaluation rated as near-flawless.

This warehouse scenario highlights the two structural mechanisms that cause open-loop validation metrics to decouple from physical performance:

1. **State integration across time**: Static evaluation cannot integrate persistent control errors into physical displacements. By resetting the robot's input state to an expert demonstration at every clock cycle, offline scoring obscures error accumulation. In closed-loop deployment, however, the state at timestep t represents the unrolled composition of all prior control actions:

   $$\mathbf{x}_t = f(f(\dots f(\mathbf{x}_0, \mathbf{u}_0) \dots, \mathbf{u}_{t-2}), \mathbf{u}_{t-1})$$

   A persistent heading bias does not require an algorithmic flaw. Physical phenomena like asymmetric tire wear, mast vibration, uneven payload distribution, or floor camber routinely introduce systematic offsets that accumulate across steps.
2. **Covariate shift into unmodeled states**: Static benchmarks cannot evaluate how a policy behaves once early errors steer the plant away from nominal trajectories. The very first uncorrected action shifts the base from the expert state $\mathbf{x}_1^*$ to an unvisited state $\mathbf{x}_1$. Once the base drifts even a few centimeters off the centerline, it enters operating states absent from $\mathcal{D}_{\text{demo}}$. Because offline metrics are evaluated exclusively along demonstrator states where $y_{\text{base}} = 0$ m, they provide no evidence about policy behavior when $y_{\text{base}} \neq 0$ m. Lacking training on corrective maneuvers, the policy often generates larger prediction errors in these off-nominal regimes, precipitating rapid divergence.

7.3.2 Contact discontinuities and metric masking

↩ Compounding covariate shift, and the recovery experience a fixed demonstration archive lacks, are established in section 6.2.4.

In the warehouse aisle, a two-millimeter tracking residual spends a tiny fraction of lateral clearance; at the door latch, that same two-millimeter residual commands the gripper into rigid steel. The moment an end-effector touches a latch, unconstrained free-space kinematics give way to constrained contact mechanics governed by impact, stick, and slip. In classical rigid-body mechanics, collision produces discontinuous velocity jumps ($v_n^+ \neq v_n^-$), while Coulomb friction creates set-valued reaction forces ($F_{\text{friction}} \in [-\mu f_{k,\perp}, \mu f_{k,\perp}]$). Because these transitions violate Lipschitz continuity (a bound on how much the dynamics can change per unit change in state), infinitesimally small variations in spatial state produce disproportionately large, discontinuous changes in physical dynamics. Although mechanical compliance and structural damping distribute impact forces across small time windows in real hardware, aggregate offline loss metrics obscure the critical physical consequences of crossing contact boundaries:

1. **Action cosine similarity ignores operational magnitude**: In representation learning and behavior modeling, action cosine similarity:

   $$\text{Cosine}(\hat{\mathbf{a}}, \mathbf{a}^*) = \frac{\hat{\mathbf{a}}^\top \mathbf{a}^*}{\|\hat{\mathbf{a}}\| \|\mathbf{a}^*\|}$$

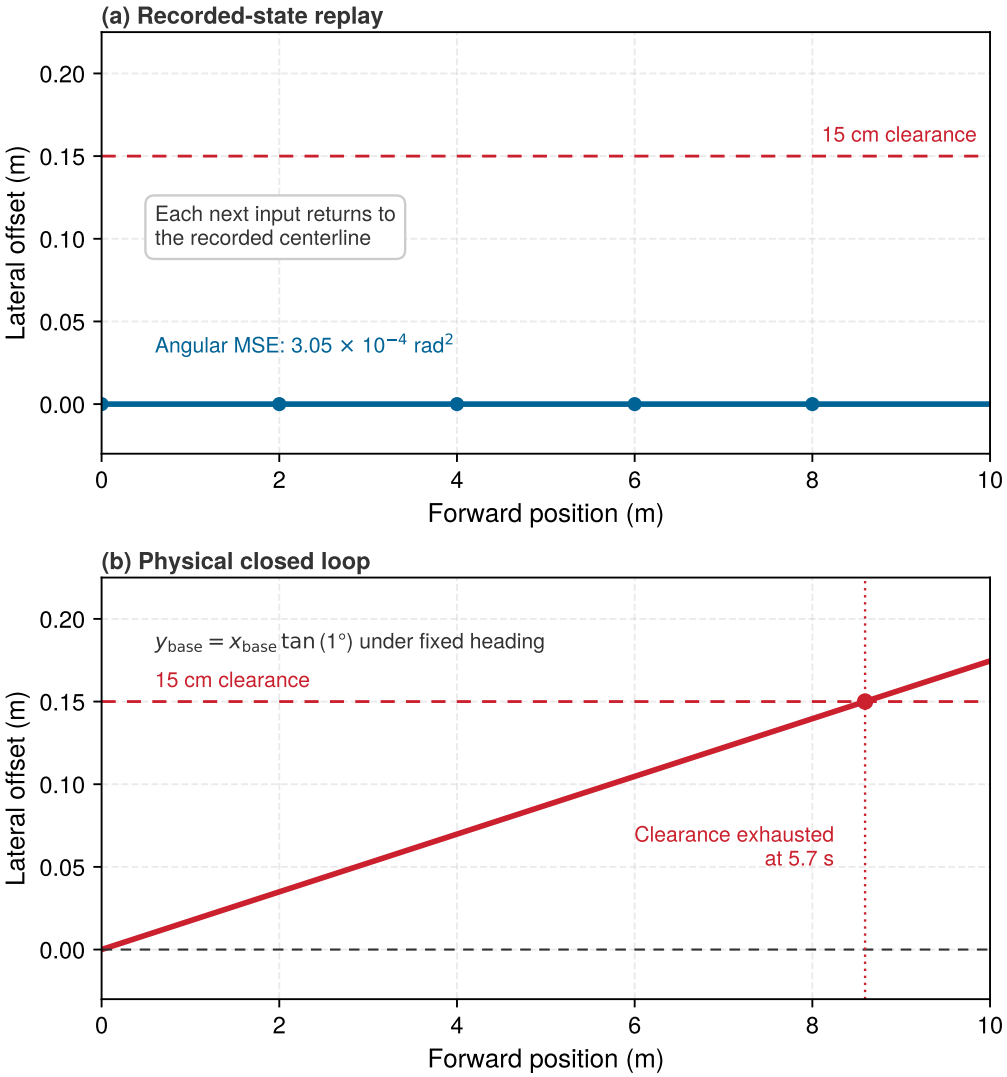


Figure 7.1: Open vs. closed loop divergence: A persistent 1.0° heading offset gives 3.05×10^{-4} rad^2 angular mean squared error on recorded centerline states. On the mobile manipulator's aisle, the navigation stack's admitted motion with that fixed heading drifts linearly and reaches the 0.15 m side clearance after 5.7 s at the 1.5 m/s drive limit. The example concerns the navigation stack, not the latch policy under evaluation.

evaluates directional alignment between predicted and expert commands. At a physical contact interface, however, directional alignment alone is insufficient because cosine metrics discard vector magnitude entirely. A policy proposal achieving 0.999 cosine similarity that commands 25 percent insufficient force fails to overcome the latch's mechanical preload and static friction, stalling against the mechanism and halting the task. Conversely, a command with identical 0.999 similarity that commands 150 percent of the required force overloads the contact interface, risking an immediate trigger of the 15 N protective tripwire. Whether hardware damage occurs depends on structural compliance, pre-contact kinetic energy, and joint impedance response. A trip invokes the protective response, whose ability to prevent damage requires qualification under those conditions. A stopped, incomplete run fails the task despite the favorable cosine score.

2. **Action mean squared error equates disparate physical regimes**: In offline supervised regression, Mean Squared Error:

$$\text{MSE} = \frac{1}{d_a}\sum_{j=1}^{d_a}(\hat{a}_j - a_j^*)^2$$

penalizes every millimeter of tracking residual identically, regardless of physical context. During free-space transit, an end-effector tracking discrepancy of 2 mm represents a benign perturbation that downstream control easily accommodates. Consider, by contrast, a stiff environmental interface modeled with contact stiffness $k_{\text{contact}} \approx 5 \times 10^5$ N/m. When an unmitigated position controller commands an apparent surface penetration δ, the resulting normal reaction force scales as:

$$\Delta f_{k,\perp} = k_{\text{contact}}\delta$$

For an uncorrected 2 mm penetration residual, this linear relationship demands a reaction force of 1,000 N. Realized loads depend on structural compliance, link elasticity, and active torque limits rather than bare stiffness alone. Even so, the calculation shows the dimensional mismatch. Offline MSE treats a benign 2 mm tracking error in free space identically to a contact error capable of tripping joint protection or damaging gearboxes.

3. **Hardware instance variance amplifies contact bifurcations**: Non-smooth contact mechanics compound the unit-to-unit manufacturing variance analyzed in section 7.2. Subtle physical discrepancies across production robots, such as transmission backlash, harmonic drive compliance, and millimeter-level camera-mounting tolerances, shift the physical boundary of the contact interface in Cartesian space. A policy trained on demonstrations recorded on Unit 01 will encounter the latch surface slightly earlier or later when deployed on Unit 02. Because contact transitions are discontinuous, this millimeter-scale spatial displacement triggers immediate operational bifurcations. The manipulator either halts prematurely in free space or collides aggressively with the mechanism frame, causing mechanical jamming that static evaluation over historical logs cannot detect.

The door manipulation benchmark illustrates this failure mechanism directly. An end-effector positioning error of a fraction of a millimeter per step registers as negligible Cartesian deviation on static logs. Against the rigid latch mechanism, however, that small offset acts as a mechanical wedge. The same stiffness relation, $\Delta f_{k,\perp} = k_{\text{contact}}\delta$, now acting sideways, transforms this lateral displacement into a steep reaction force that quickly crosses the 15 N contact tripwire, triggering an immediate safety abort before the door unlatches.

Conversely, offline benchmarks can also fail through false pessimism when tasks support multimodal solution paths. If a human demonstrator avoided an obstacle by steering left, an open-loop regression metric penalizes an autonomous policy that navigates safely to the right (the multimodal distribution dilemma), even though both trajectories satisfy the closed-loop objective. Static action loss measures divergence from a single historical recording, not functional task completion.

These asymmetries frequently produce rank inversions across candidate models. A policy whose small residual errors concentrate precisely at critical contact boundaries will score lower validation loss than a competitor with larger, yet harmless, errors distributed across free space. When deployed, however, their operational reliability reverses:

↩ Mode averaging over multimodal demonstrations is established in section 6.2.3.

$$\mathcal{L}_{\text{val}}(\pi_A) < \mathcal{L}_{\text{val}}(\pi_B) \quad \text{yet} \quad \text{ACR}(\pi_A) \ll \text{ACR}(\pi_B)$$

Evaluation campaigns must therefore measure autonomous completion rates, force profiles, and contact stability on physical hardware rather than ranking models by average offline loss. Each standard offline metric conceals a specific physical failure mode, and table 7.1 maps each metric to its implicit open-loop assumption, its physical failure mechanism, and the corresponding closed-loop metric required to expose it. Most of the closed-loop measures in its last column are physical quantities the record already logs. The one new measure is time to intervention (T_{TTI}), the elapsed duration from the granting of policy authority to the first permission-path override or safety operator takeover, which measures how long a drifting policy runs before something must stop it. Alongside ACR, MTBI, and D_{int}, these execution measures characterize real-world autonomy in ways that static loss calculations cannot.

Table 7.1: Offline metrics and the closed-loop failures they hide: Each open-loop metric assumes the demonstrator's state distribution is unaffected by policy error; on the machine, biases integrate into drift, valid alternatives are penalized, and shifted states produce boundary violations.

Offline metric	Open-loop assumption	Why it breaks in closed loop	Failure on the machine	Closed-loop measure
Action mean squared error	Small residuals give small path deviations	A persistent bias integrates into drift and carries the machine off the demonstrated states	The base spends its rack clearance (stop or strike)	Peak deviation from the reference path; time to intervention (T_{TTI})
Discrete action accuracy	Correct labels on logged states give identical execution	Benign alternatives and boundary excursions cost the same; errors correlated in time are invisible	Command chatter and joint over-torque	Peak jerk; adherence to actuator torque limits
Cosine similarity	Collinear proposals give stable paths	Magnitude is ignored, so under-actuation stalls against static friction and over-actuation slips	The base stalls or slips its wheels; the arm stalls short of the latch	Slip ratio (section B.7.4) and damping (section B.2.3)
Negative log-likelihood	Likelihood at the expert action captures the action distribution	A unimodal likelihood head averages valid alternatives, such as passing an obstacle on either side, into an invalid mean action	Valid policies rejected; demonstrator noise imitated as jitter	Closed-loop task success over randomized starts
Reconstruction or Chamfer loss	Accurate scene prediction gives safe plans	Background pixels outweigh thin obstacles and contact geometry	Phantom braking; clipping a cable or a glass partition	Measured clearance and its minimum margin

Systems Perspective 7.1: The offline evaluation illusion

Offline validation loss, Mean Squared Error (MSE), and action cosine similarity score predictions exclusively along frozen demonstrator trajectories where errors are artificially erased at every clock cycle. Physical closed-loop evaluation exposes accumulated trajectory drift, state distribution shift, and dynamic contact transitions absent from recorded logs. While the worst-case imitation bound ($\mathcal{O}(S^2\epsilon)$) models decision divergence under distribution shift, kinematic integration and contact mechanics ($f_{k,\perp} = k_{\text{contact}}\delta$) govern physical clearance and reaction forces. A score computed against recorded states cannot see a feedback loop that recording removed.

7.3.3 The proposal boundary in two-processor architectures

Drift and contact are failures of the plant. A third class of failure lives in the machine's own hardware, and replay misses it for a different reason, because the replay harness runs on the wrong processor. In the running machine, the proposal boundary of section 1.7 is also a processor boundary. The learned proposers run on an application processor, the permission path runs on a separate safety microcontroller, and every candidate proposal crosses the inter-processor bus before it can move the arm. Replaying sensory traces on the host application processor therefore verifies only whether neural inference completes within allocated memory bandwidth and latency budgets.

Evaluating an unverified neural policy strictly offline, without the physical plant, real-time bus transport, and safety monitors active in the loop, benchmarks isolated tensor computation rather than operational survivability. What replay on the host cannot see is everything the bus and the microcontroller add: transport delay, a late proposal arriving after its chunk lease has lapsed, a clip that changes the motion the policy expected, and a fallback that takes over mid-task. Shadow operation exposes the first two on real buses without letting the candidate move the machine, and constrained physical testing exposes all four.

Because open-loop metrics cannot certify closed-loop reliability, the proposer must be evaluated in settings that restore parts of the loop. Each setting alters a different element of the sensorimotor loop (the plant dynamics, the fidelity of the sensor stream, or the actuator authority), so each exposes a different set of failures.

↩ The two-processor split, stated as a hypothesis that placement must later test on silicon, is established in section 4.2.

↪ How the permission path admits a proposal, corrects it, or transfers authority to a resident fallback is developed in section 12.6.

7.4 Evidence Regimes

Before undertaking twenty physical door runs, an engineering team evaluating a robotic latch policy could score candidate predictions against logged human demonstrations, execute them inside a simulated physics environment, run them in shadow mode while a validated controller retains physical authority over the arm, or deploy them directly on the real mechanism with contact tripwires armed. These four configurations constitute the primary **evidence regimes** of embodied evaluation. Figure 7.2 and table 7.2 map them across two structural axes: physical realism on the vertical axis (from logged and synthetic feeds up to live hardware) and causal feedback on the horizontal axis (from open loop to closed loop). Each regime establishes a distinct coupling between the policy, its sensory observations, and the underlying physical machine states, so each regime supports only a bounded empirical claim. None of these four settings alone reveals how an unassisted policy behaves during open-ended physical deployment, because each environment either breaks the control loop's causal feedback or alters the distribution of states the robot encounters.

7.4.1 The four evaluation evidence regimes

The first axis, causal feedback, governs whether a candidate's emitted actions alter future states and subsequent sensor observations ($\partial \mathbf{x}_{t+1} / \partial \mathbf{a}_t \neq \mathbf{0}$), or whether the policy merely reads a prerecorded, invariant sensory stream. In an open-loop configuration, execution errors cannot compound because mistakes never propagate into future inputs. In a closed loop, small policy errors perturb subsequent machine states and accumulate into compounding drift at a rate dictated by system dynamics and closed-loop feedback gains. The second axis, physical realism, governs whether sensory observations originate from physical transducers interacting with unmodeled contact dynamics, gear backlash, tire slip, and variable illumination (f_{real}), or instead emerge from a synthetic numerical simulator or static buffer (f_{sim}). No single testing configuration can maximize both axes simultaneously without exposing expensive hardware, or nearby operators, to an unproven and potentially destructive policy. Each quadrant of figure 7.2 therefore provides evidence for specific claims and carries its own structural blind spot.

In recorded replay, the candidate policy executes strictly as a feedforward computation over static sensor buffers. Because the physical environment cannot respond to emitted actions, the state sequence remains fixed to the exact trajectory recorded during prior data collection. This regime provides solid empirical evidence for computational efficiency, dynamic memory allocation, and loss metrics against a reference benchmark. It can verify, for example, whether neural network inference fits within its target execution budget on an edge accelerator, and whether output distributions align numerically with logged human demonstrations. However, recorded replay cannot substantiate any claim regarding closed-loop stability or control authority. Because predicted actions never feed back into subsequent sensor frames, errors never induce consequences. If the policy commands an aggressive steering angle, the subsequent logged frame still shows the mobile base tracking straight down the corridor.

Closed-loop simulation restores the causal feedback path by substituting an explicit mathematical or numerical model, f_{sim}, for the physical robot and its operating environment. When the policy emits an action and the modeled safety permission path selects the control command $\mathbf{u}_t$, the physics engine numerically integrates the equations of motion, advances the scene geometry, and renders synthetic sensor observations for timestep $t+1$. This regime supports extensive testing of closed-loop dynamics, kinematic reachability, and recovery maneuvers. It can demonstrate, for example, whether a mobile base recovers gracefully from an initial tracking offset, or whether an articulated arm avoids self-collision throughout its reachable workspace across thousands of parallelized instances. However, the resulting evidence applies exclusively to the synthetic operator f_{sim}. Unmodeled transmission backlash, tire deformation, dynamic contact friction transitions, and optical sensor artifacts are omitted by construction. Demonstrating stability within f_{sim} therefore establishes that the policy stabilizes the model, not that it will stabilize physical hardware.

Shadow operation deploys the candidate policy directly onto an active physical machine while a validated baseline controller, or a human operator, retains exclusive authority over the physical actuators. The candidate policy ingests live streaming sensor data, executes onboard inference amid real mechanical vibrations and fieldbus contention, and computes candidate actions $\mathbf{a}_t^{\text{cand}}$.[4] These proposed actions are logged to high-speed storage but never dispatched to the motor drives.

[4] **Bus Jitter in Shadow Mode**: Inference that is steady on a bench stretches on the machine when direct memory access (DMA) transfers, shared memory, and fieldbus arbitration compete for the same paths. Shadow operation logs that jitter on the real buses, under real thermal load, without letting the late proposals move the chassis.

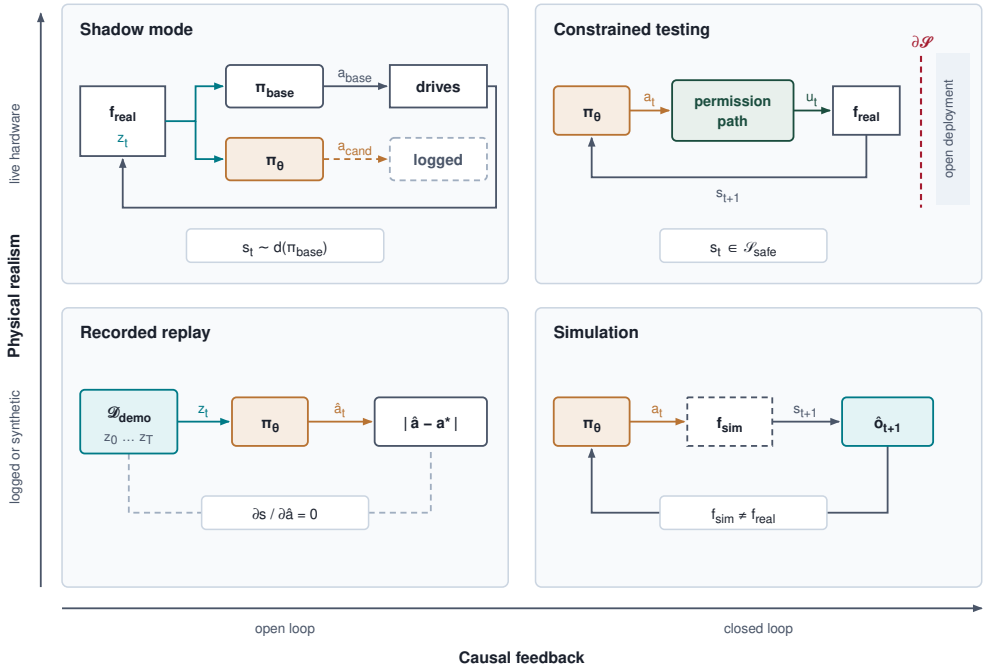


Figure 7.2: Evaluation evidence regimes: The regimes vary along causal feedback (open vs. closed loop) and physical realism (logged or synthetic vs. live hardware). Replay cannot observe compounding drift, simulation leaves the reality gap open, shadow operation locks the trajectory to the baseline controller, and constrained testing truncates failures at the constraint boundary $\partial\mathcal{S}$. No single regime tests deployment beyond that boundary.

Shadow operation yields direct evidence about real-time system performance. It can confirm, for instance, whether the latch policy maintains its target 20 Hz chunk rate under sustained thermal throttling, and quantify how frequently $\mathbf{a}_t^{\text{cand}}$ deviates from the baseline control action $\mathbf{a}_t^{\text{base}}$ beyond an acceptable safety threshold.

Shadow operation is limited, however, by the **trajectory confinement problem**. Because the baseline controller exercises exclusive control over the actuators, the physical robot traces trajectories governed strictly by the baseline state distribution $d^{\pi_{\text{base}}}$. If the candidate policy generates an action that would cause the arm to jam the latch, veer off course, or strike a support strut, the physical plant never executes the command. The candidate's sensory stream remains confined to the safe states visited by the baseline supervisor. As a result, shadow testing cannot reveal whether the candidate could recover from an unvisited off-nominal state, nor can it expose whether its divergent actions would have caused mechanical damage.

Constrained physical testing closes the causal feedback loop on actual hardware by routing candidate policy proposals through the permission path to the actuators, strictly within an enforced safety envelope. These operational boundaries are enforced via hardware tethers, software velocity clamps, geofenced test enclosures, or a dedicated real-time permission path that intercepts and overrides unsafe setpoints with deterministic latency. Constrained testing delivers authentic empirical evidence regarding real-world contact mechanics, friction transitions, structural compliance, and sensor noise under actual operating loads, which is why the experimental door-latch trials belong in this regime.

When the safety system triggers an intervention, it alters or truncates the raw policy trajectory ($n_{\text{int}_i} \geq 1, A_i = 0$), leaving the candidate's unmitigated behavior permanently unobserved. Such a run answers two different questions, and the record must keep them apart. For the autonomy outcome it is an unambiguous failure, $A_i = 0$, because the candidate policy could not finish without external override. Even if the supervisor guides the robot to successful task completion, the trial cannot be scored as an autonomous success. For the separate event of an unassisted crash, the

question of whether the policy would have damaged something had nobody stepped in, the run is right-censored.[5] The intervention terminated the trial before that terminal state could manifest, so the observation confirms neither unassisted success nor an inevitable crash. Any rigorous evaluation must report completion rates, intervention frequencies, and post-intervention recoveries strictly inside the verified operating envelope. Conflating safety-supervised task completion with unassisted policy survival commits the **censored trial fallacy**, which attributes to the machine learning policy a level of robustness that the permission path or the safety operator provided.

Certain software properties transfer cleanly across regime boundaries when inputs and execution semantics match. Deterministic numerical calculations, floating-point arithmetic invariants, and tensor shape contracts can be checked under the same precision, rounding, and overflow rules. An activation overflow transfers only when the simulator and target implement the same arithmetic behavior. In contrast, runtime execution latency, bus contention, and memory bandwidth utilization cannot transfer purely across regimes. They require re-measurement under the actual scheduling, fieldbus, and thermal environments of the deployment hardware. Above all, the operating state distribution never transfers directly. The instant the environmental operator transitions from synthetic simulation f_{sim} to live physical dynamics f_{real}, or when the arbitration parameters of the runtime permission path change, the state trajectory $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_t$ rapidly diverges.

Treating these four regimes as a monotonic ladder of fidelity is a mistake. Transitioning across replay, simulation, shadow execution, and constrained testing does not simply increase fidelity; every step exchanges one systemic distortion for another. Simulation secures closed-loop causality at the expense of authentic contact physics and sensory noise. Shadow execution reclaims real physical sensing and bus timing but surrenders candidate closed-loop causality. Constrained physical testing restores both causality and physical grounding, yet it systematically censors the state space beyond its safety boundary. Observing zero supervisor interventions across one hundred constrained trials proves that the policy maintained states within the un-triggered safety envelope; it offers no statistical guarantee about how the policy behaves once it breaches that boundary.

System verification therefore demands assigning each engineering question to the specific regime equipped to answer it (table 7.2). For the latch manipulation policy, verifying whether chunk inference executes reliably within its allocated cycle time belongs first in recorded replay and subsequently in hardware shadow mode. Testing whether the manipulator can recover from a wide range of positional offsets belongs in high-throughput simulation, where thousands of permutations inflict zero hardware wear. Evaluating whether passive gripper compliance and contact friction remain within structural force limits belongs strictly in constrained physical trials. Each regime remains blind to the failure modes of the others.

[5] **Right Censoring**: In clinical survival analysis, a patient who departs a study before the terminal endpoint has an unknown time of death rather than an established long life. Kaplan and Meier (1958) formalized an estimator that tracks such subjects within the population at risk only while they remain actively observed. An intercepted robotic trial is right-censored under identical logic: the intervention proves that the policy required supervisor takeover, not that the robot would have stabilized itself had the supervisor remained passive.

Table 7.2: Evaluation evidence regimes compared: Trade-offs between causal feedback coupling, environmental realism, supported empirical claims, and experimental boundary distortions across the four testing regimes. No single environment evaluates open deployment; rigorous certification requires combining complementary regimes rather than assuming a linear fidelity ladder.

Evaluation Evidence Regime	Causal Feedback Coupling	Physical Realism	Supported Empirical Evidence	Masked Failure Mode / Boundary Distortion
Recorded Replay	Open Loop ($\partial \mathbf{x}_{t+1} / \partial \hat{\mathbf{a}}_t = \mathbf{0}$)	Static Demonstration Buffers $\mathcal{D}_{\text{demo}}$	Latency budget, DRAM footprint, numerical accuracy	Covariate shift; errors never alter subsequent observations
Closed-Loop Simulation	Closed Loop ($\mathbf{x}_{t+1} = f_{\text{sim}}(\mathbf{x}_t, \mathbf{u}_t)$)	Synthetic Physics Operator f_{sim}	Kinematic reachability, large parallel test suites	Sim-to-real gap; unmodeled contact non-linearities and tire slip
Shadow Mode Operation	Open Loop for candidate (actions muted)	Physical machine f_{real}	Bus contention, thermal throttling, real sensor noise	Trajectory confinement; states locked to baseline controller $d^{\pi_{\text{base}}}$
Constrained Physical Testing	Closed Loop with permission-path override ($\mathbf{u}_{\text{safe}}$)	Physical machine f_{real} within envelope	Contact dynamics, compliance, friction, actuator limits	State censorship; permission-path overrides and tethers truncate failure tails

7.4.2 Simulation as a conditional proxy

A simulator can screen candidate latch policies before expensive hardware is committed to physical trials, but its screening validity depends on how accurately it models the physical mechanisms governing the decision. Paired probing, developed in section 6.4.2, drives the simulated and physical strike plates with identical commands and measures their discrepancies in newtons and milliseconds. The evaluation protocol incorporates those measured discrepancies directly into its test plan. A favorable simulated recovery qualifies a checkpoint for physical verification, but that result cannot resolve unmodeled discrepancies in the contact physics or data transport pipeline. Visual rendering artifacts, numerical friction regularization, and absent force-torque feedback can each alter candidate policy rankings.

Whether a simulator ranks candidate policies correctly is an empirical, measurable property. Proxy fidelity is measured with a paired benchmark across both regimes. The benchmark evaluates the same set of candidate policies (such as the latch training checkpoints) on both the simulated door mechanism and the physical hardware under tightly matched initial conditions, then compares the resulting performance orderings. While a standard Pearson or Spearman correlation coefficient between simulated and physical success rates summarizes overall monotonic agreement, correlation alone can hide the failure that matters most. A high correlation coefficient can coexist with a rank inversion, in which the simulator promotes a checkpoint that fails on the physical door. To address this risk, X. Li et al. (2024) measure rank agreement with the mean maximum rank violation (MMRV), computed across approximately 1,500 paired simulated and physical episodes. For each policy, MMRV finds its worst inversion against any other policy, weighting each pair the simulator orders wrongly by the two policies' real-world performance gap. MMRV then averages those worst cases across policies, so a simulator scores badly exactly when it swaps policies whose physical performance differs most.

This evaluation principle applies directly to the latch manipulation task. A simulator whose relative rankings across historical checkpoints correlate reliably with physical trials earns the operational authority to screen incoming models. Conversely, a simulation environment that has never been calibrated against paired physical benchmarks possesses no screening credibility, regardless of how photorealistic its graphics engine appears. The identical constraint governs learned neural world models deployed as simulated environments: predicted candidate success serves as a justification for scheduling physical tests, not as verifiable evidence of physical capability, until paired real-world trials validate model calibration. Furthermore, even a validated ranking proxy only authorizes candidate filtering. Physical trials remain the ground truth for safety and performance claims, because knowing that policy A outperforms policy B reveals nothing about whether policy A satisfies the machine's absolute failure-rate limits.

↩ Learned world models used as simulators, whose simplifications cannot be enumerated, are established in section 6.4.3.

Two additional systems factors dictate the generalization bounds of any simulation-based evaluation: parameter domain coverage and execution timing fidelity. The first factor is the physical parameter variation spanned during simulation trials. A simulation calibrated against telemetry logs from a single robotic arm provides evidence strictly for that physical unit under those specific ambient conditions. Similarly, domain randomization evaluates policy resilience only across the bounded support of the randomized parameter distributions; it cannot account for dynamic phenomena entirely absent from the simulator's underlying differential equations. Formal sim-to-real gap analysis identifies unmodeled dynamics and compiles them into the omission log that the policy manifest carries. The closed-loop evaluation protocol must incorporate that log directly, cataloging the operating regimes and edge conditions about which simulation can make no empirical claims.

↩ Formal sim-to-real gap analysis, which yields a supported region and a residual mismatch list, is established in section 6.5.

The second factor is whether the evaluation regime reflects true hardware execution dynamics. Standard software-in-the-loop simulations pause virtual simulation time while the policy executes neural inference, masking real-world computational bottlenecks. A simulator that pauses virtual time during inference cannot expose the consequences of a late inference chunk, a rejected stale proposal, or an executing chunk that continues under its 60 ms chunk lease while awaiting updated sensor verification. Explicit delay models can exercise those transitions, but establishing their actual timing requires the neural policy and runtime permission arbiter on target embedded compute hardware interconnected via the physical buses.

↩ The chunk lease, sized between the renewal floor and the stopping budget, is established in section 4.4.

Closed-loop simulation, whether executed purely in software or integrated with hardware in the loop, serves as a filter for screening candidate models. Only physical trials, however, measure

embodied competence, and engineers must formalize the safety envelopes and evaluation protocols long before the first trial begins.

↪ The qualification ladder, which stages software-, processor-, and hardware-in-the-loop benches before physical fault rigs, is developed in section 15.7.

Checkpoint 7.1: Trade-offs across evidence regimes

Before fixing a physical trial protocol, review evidence regimes and fidelity trade-offs:

- ☐ **Regime causality trade-offs**: Contrast the causal feedback properties of recorded trace replay against closed-loop simulation.
- ☐ **Shadow-mode trajectory confinement**: Explain why shadow-mode evaluation on the running machine cannot test the candidate policy's recovery maneuvers.
- ☐ **Transfer across regimes**: Describe which properties transfer reliably across regimes and which one stays behind.

7.5 Fixing the Trial Protocol

If an engineering team executes fifty ad hoc physical runs across arbitrary trajectories in a constrained test cell and reports a 90 percent success rate, that statistic lacks an interpretable denominator. Empirical success probabilities and intervention rates are meaningful only with respect to a specific joint distribution $\mathcal{D}$ over initial system states, environmental parameters, and task goals, so the intended operating region must be formalized before executing a single physical trial. The **target ODD** is the concrete subset of the declared ODD recorded by the policy manifest of section 6.8 that the evaluation campaign samples. Its sampling weights define $\mathcal{D}$, and no rigorous empirical claim reaches beyond its support. A measured success rate supports a target-distribution claim only when the support and sampling weights of $\mathcal{D}$ are specified in advance, setting explicit bounds on scene illuminance, payload mass and inertia, surface friction coefficients, allowable initial state offsets, and the specific hardware serial numbers under test.

Extracting valid confidence bounds from empirical trials requires three operational conditions:

1. *Independent and identically distributed (iid) sampling:* Trials must be independent realizations drawn from $\mathcal{D}$. In embodied physical systems, standard offline dataset intuition often fails; randomly shuffling log records cannot eliminate physical coupling. On real robotic hardware, thermal accumulation in motor windings from resistive heating (I^2R), battery voltage sag under discharge, and progressive mechanical backlash introduce temporal autocorrelation across sequential trials. Correlated trials, like the serially correlated telemetry of section 5.4, carry fewer independent observations than their count suggests. Run 20 is not drawn from the same physical distribution as Run 1 unless operators strictly enforce thermal cooldown intervals and battery recharging cycles between runs. Furthermore, repeated trials executed on a single physical unit (such as Unit 01) cannot serve as iid draws from the broader fleet distribution $\mathcal{M}$, because unit-level manufacturing tolerances and sensor calibrations bias the sample.
2. *Fixed outcome predicates:* The binary evaluation function ϕ and termination criteria κ must remain frozen throughout the entire campaign. Modifying the execution deadline T_{deadline}, adjusting collision thresholds, or widening the spatial tolerance of goal region $\mathcal{G}_{\text{goal}}$ midway through testing alters the definition of success. Doing so commits post-hoc metric drift and invalidates statistical comparisons against baseline models.
3. *Full-denominator accounting:* Every initiated run must remain in the trial log, including hardware communication faults, sensor timeouts, emergency protective stops, and operator interventions. Culling inconvenient or anomalous runs corrupts the statistical denominator. A hardware abort that halts execution prior to goal completion yields a task failure ($Y = 0$). Under a strict autonomy metric, any human intervention forces $A = 0$. An operator nudge that guides the robot around an obstacle followed by on-time completion records $Y = 1$ and $A = 0$. Safety monitoring systems must record kinematic limit violations and physical damage separately from task completion. When a safety tripwire or human supervisor intercepts an impending collision, the counterfactual unmitigated trajectory remains unknown. Consequently, zero recorded damage cannot be credited as evidence of unassisted survival. Discarding aborted

or intercepted runs changes the underlying question and inflates perceived reliability by measuring an idealized survivor population.

When these conditions hold, the sample mean provides an unbiased point estimate of the policy's closed-loop performance under $\mathcal{D}$. The remaining engineering challenge is determining how much statistical uncertainty a finite sample leaves, particularly when high-reliability targets yield zero observed failures across the test budget.

7.6 Confidence from a Finite Campaign

With the operational outcome, target population, evaluation regime, and trial denominator fixed, the twenty clean nominal runs pose an immediate statistical problem: determining what degree of reliability the campaign has actually demonstrated. A confidence bound quantifies the uncertainty that remains under the declared sampling model. Systems engineers must treat this bound as a measure of statistical evidence rather than an assessment of physical failure severity or an automatic deployment permit.

The standard textbook interval for a proportion, the Wald normal approximation $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/N}$, collapses to zero width when the observed success fraction $\hat{p}$ reaches 1.0 because its sample variance term vanishes. This collapse exposes the breakdown of the asymptotic normal assumption near the boundary rather than empirical certainty. Evaluating the statistical assurance of a zero-failure record requires only the product rule for independent events.

Suppose the true per-run success probability of the latch policy is p. If individual trials are independent, twenty consecutive runs all succeed with joint probability p^{20}. Consider a flawed policy that fails one run in eight ($p = 0.875$). That policy still produces an unbroken twenty-run success streak in 6.9 percent of evaluation campaigns. An empirical run of twenty successes cannot rule it out. As the candidate success rate p decreases, a clean record becomes increasingly improbable. When p drops to 0.861, the probability of observing twenty consecutive successes falls to exactly $\alpha_{\text{sig}} = 0.05$, which corresponds to the significance level for a 95 percent confidence requirement $(1 - 0.95)$. Any underlying success probability lower than 0.861 would make the observed twenty-run clean record rarer than a 1-in-20 chance ($p^{20} < 0.05$). The one-sided lower confidence bound therefore identifies the threshold below which candidate policies become statistically inconsistent with the experimental data. In standard frequentist qualification, "95 percent confidence" describes the long-run coverage of the testing protocol. Across repeated qualification campaigns, the procedure traps the true parameter p above the calculated bound at least 95 percent of the time. The bound represents a conservative floor under the declared sampling model rather than a point estimate of p or a Bayesian posterior belief about this specific policy.

Reading the inversion as a curve makes the logic visible. Plotting p^{20} against every candidate p (figure 7.3) shows the clean record ruling out only the candidates whose curve lies below the α_{sig} line, the region left of $p = 0.861$. The one-in-eight policy sits just above that line, which is why twenty flawless runs cannot reject it.

For a predeclared zero-failure campaign of n trials, this inversion yields the exact one-sided Clopper-Pearson lower bound[6] (Clopper and Pearson 1934). Setting $p^n = \alpha_{\text{sig}}$ gives

$$p_{\text{lower}} = \alpha_{\text{sig}}^{1/n} \tag{7.3}$$

with the corresponding upper confidence limit on the per-run failure probability given by:

$$p_{\text{fail, upper}} = 1 - \alpha_{\text{sig}}^{1/n}$$

Section 7.6.1 extends this inversion principle to evaluation campaigns that record failures.

For rare failures ($p_{\text{fail}} \ll 1$), a first-order Taylor series expansion linearizes this bound into the **rule of three** (Hanley and Lippman-Hand 1983). Expanding $\ln(1 - p_{\text{fail}}) = -p_{\text{fail}} - p_{\text{fail}}^2/2 - \cdots \approx -p_{\text{fail}}$ and setting the clean-run probability $(1 - p_{\text{fail}})^n$ equal to $\alpha_{\text{sig}} = 0.05$ yields $-n\,p_{\text{fail}} \approx \ln(0.05) \approx -2.996 \approx -3.0$. When n independent trials pass without a single failure, the upper 95 percent confidence limit on the per-trial failure probability simplifies to:

$$p_{\text{fail, upper}}^{95\%} \approx \frac{3}{n}. \tag{7.4}$$

[6] **Exact Binomial Bound**: Clopper and Pearson (1934) inverted the binomial tail directly instead of approximating it with a normal distribution. Here "exact" denotes that algebraic inversion rather than exact coverage probability; because binomial counts are discrete, the true coverage equals or exceeds the nominal confidence level for all values of p. Under the declared binomial sampling model, the procedure places the lower bound at or below the true reliability in at least the nominal fraction of repeated campaigns; an individual campaign can still overstate it.

↪ The discrete and continuous-time derivations of the zero-failure bound, and why the normal approximation fails at zero failures, are developed in section A.8.

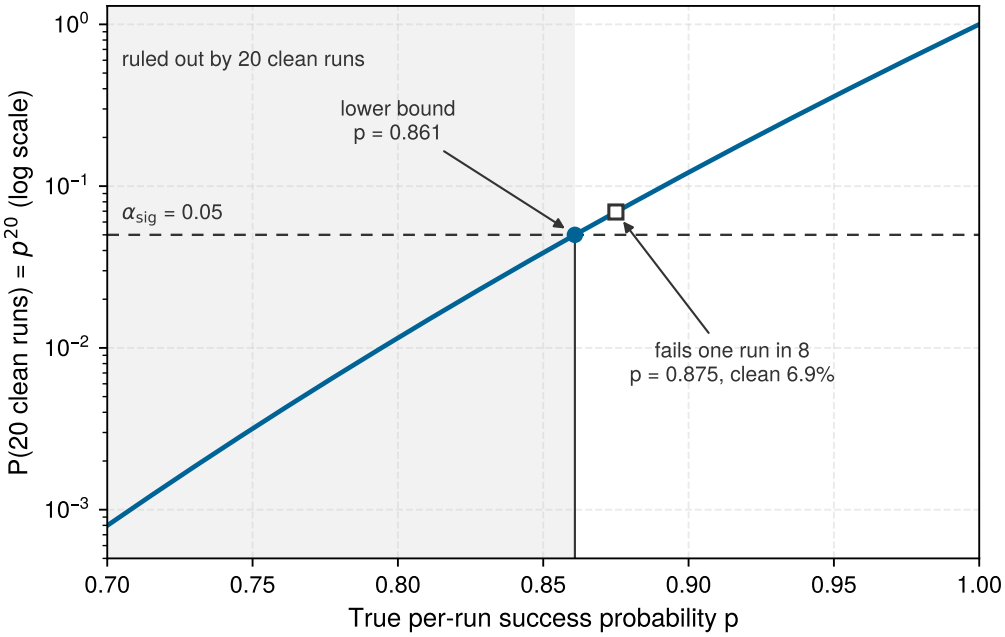


Figure 7.3: Inverting a clean record into a bound: The probability that 20 independent runs all succeed, p^{20}, against the true per-run success probability p. Candidates left of the crossing with $\alpha_{sig} = 0.05$ would make the observed record rarer than one campaign in twenty, so the record rules them out; the crossing at $p = 0.861$ is the lower bound. The one-in-eight policy still logs a clean record in 6.9 percent of campaigns.

DRAFT

Inverted to size test campaigns, the rule establishes the empirical burden for any target reliability: $n \approx 3/(1-p_{\text{target}})$. Each additional "nine" of target reliability multiplies the required trial volume tenfold. For the twenty door-opening trials, equation 7.4 yields an approximate upper failure bound of $3/20 = 0.15$, whereas the exact Clopper-Pearson computation in notebook 7.1 establishes 13.9 percent. A policy with a true failure probability as high as 13.9 percent remains entirely compatible with twenty consecutive successes at 95 percent confidence. The campaign cannot substantiate an industrial claim of 99 percent reliability. Formally certifying a 99 percent lower confidence limit ($p_{\text{target}} = 0.99$) at 95 percent confidence demands:

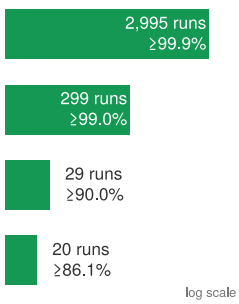
$$n = \left\lceil \frac{\ln(\alpha_{\text{sig}})}{\ln(p_{\text{target}})} \right\rceil = \left\lceil \frac{\ln(0.05)}{\ln(0.99)} \right\rceil \tag{7.5}$$

which equals 299 consecutive clean trials, an order of magnitude beyond the twenty trials conducted in the initial campaign.[7]

The bound inherits the iid condition of section 7.5. Twenty runs on Unit 01 therefore bound Unit 01's success probability under the sampled conditions and nothing more. The unit-to-unit variance of section 7.2 puts Unit 02 in a different population, whose bound needs its own trials or a sample drawn across the fleet.

> Napkin Math 7.1: The clean-record bound
>
> **Problem**: The mobile manipulator completes $n = 20$ door-opening runs with no contact-tripwire crossing and no intervention. What one-sided 95 percent lower limit on its per-run success probability p does that record support, and what does it cost to claim 99.9 percent instead?
>
> **Variables**:

7 | **Rule of Three**: The heuristic originates in biostatistics, where Hanley and Lippman-Hand (1983) evaluated the risk to future patients following a clinical trial with zero adverse events across n participants. An embodied evaluation campaign asks an identical question of the next robot trial, and the same $3/n$ ratio governs the bound.



Each added nine of reliability costs ten times the clean runs.

- $n = 20$ clean runs (Illustrative; the chapter's opening campaign, independent and identically distributed draws from the target operational design domain).
- $\alpha_{\text{sig}} = 0.05$ (Chosen; one minus the 95 percent confidence level).

Math: Setting $p^n = \alpha_{\text{sig}}$ (equation 7.3) and taking logarithms, $\ln(1/\alpha_{\text{sig}}) = 2.996$, divided by n gives the exponent 0.1498, and p_{lower} is e raised to minus that exponent, 0.861. The failure upper limit is $1 - p_{\text{lower}}$, or 13.9 percent, which the rule of three (equation 7.4) approximates as $3/20 = 0.15$.

Result: 86.1 percent success lower limit at 95 percent confidence; a 99.9 percent limit needs 2,995 clean runs (notebook 7.2).

Systems insight: Twenty clean runs leave a failure probability as high as one run in seven statistically consistent with the empirical data. The evaluation campaign establishes a formal lower confidence bound under its specific sampling assumptions. It should never be conflated with a steady-state failure rate, an MTBF, or an unconditional deployment authorization. Improving the bound requires scaling n, because the numerator $\ln(1/\alpha_{\text{sig}}) \approx 3$ remains anchored by the chosen confidence level. This structural property explains why the rule of three matches the exact Clopper-Pearson bound whenever failure probabilities are small.

7.6.1 Bounds when failures occur

If the mobile manipulator jams on the twentieth door run after nineteen clean openings, the zero-failure rule of three no longer applies, yet discarding the campaign would waste nineteen valid observations. Encountering an operational failure during testing does not invalidate statistical evaluation. Instead, it shifts the calculation from the zero-failure boundary to the full binomial distribution. Given k successful trials out of N total runs, the one-sided lower confidence limit is the parameter value p for which the probability of observing k or more successes equals α_{sig}:

$$\Pr(X \geq k \mid N, p) = \sum_{j=k}^{N} \binom{N}{j} p^j (1-p)^{N-j} = \alpha_{\text{sig}}, \tag{7.6}$$

where the random variable X denotes the success count. A hypothesis test rules out a low candidate reliability p by asking how likely a policy that poor would be to produce an outcome at least as favorable as the observed k successes, so summing from $j = k$ up to N captures the full upper tail of outcomes that meet or exceed the observed record. When $k = N$, the summation collapses to a single term, $p^N = \alpha_{\text{sig}}$, recovering the zero-failure Clopper-Pearson bound of equation 7.3. With that single failure across twenty runs ($k = 19$), the relationship expands to $p^{20} + 20p^{19}(1-p) = \alpha_{\text{sig}}$. Although this polynomial lacks a closed-form algebraic root, numerical root-finding yields a 95 percent lower confidence limit of 78.4 percent. Even though the empirical success fraction is 95 percent, a single observed failure drops the certified lower bound from 86.1 percent to 78.4 percent. In production evaluation pipelines, libraries compute this limit directly using quantiles of the incomplete beta function, which follows from the exact identity between binomial tails and Beta distributions.

Two engineering cautions govern this method. First, any observed failure must remain permanently logged in the audit trail. Halting a test campaign upon observing an anomaly and restarting a clean run counter introduces optional-stopping bias. Engineers cannot subsequently claim a zero-failure bound on the post-restart run sequence (section 7.7.1). Second, the Clopper-Pearson method is conservative by construction because its coverage probability equals or exceeds the nominal confidence level for every value of p. For standard exploratory data analysis, Brown et al. (2001) recommend the Wilson score interval[8] or Jeffreys prior intervals, which maintain average coverage closer to nominal while producing tighter intervals. For physical qualification and safety certification, however, overestimating autonomous reliability is the costly error. The conservative Clopper-Pearson bound therefore serves as the defensible systems default, and engineering test records must explicitly state the inference method applied.

The twenty nominal door-opening runs now have a concrete statistical interpretation under nonasymptotic sampling theory. Elevating the certified lower bound requires expanding the trial budget,

[8] **Wilson Interval**: Wilson (1927) inverted the score test around the parameter p rather than substituting the empirical estimator $\hat{p}$, allowing the interval to maintain a nonzero width even when $\hat{p} = 1.0$. This formulation resolves the boundary collapse of the Wald normal approximation. However, because its coverage guarantees hold on average across the parameter space rather than strictly at every point p, physical qualification protocols default to the exact Clopper-Pearson bound.

which directly compounds the physical testing hours, mechanical wear, and facility costs needed to qualify embodied machine learning policies.

7.7 Trial Budgets, Stopping Rules, and Comparisons

While a million software unit tests consume only processor cycles, each closed-loop trial of a physical robot consumes test-cell time, exhausts bearing and linkage fatigue life, burns thermal headroom in motor windings, and risks costly hardware damage. Statistical theory dictates how many trials a reliability claim demands, but on a physical machine the plant, not the compute budget, determines what each trial costs. Sizing a qualification campaign therefore means reconciling the trial count required by statistical confidence with the operational time and fatigue life of the testbed that must supply it. That reconciliation separates a rigorous qualification argument from an anecdotal demonstration. The same budget pressure then motivates the two refinements that close the section: stopping a campaign early without inflating its error rates, and deciding whether a retrained checkpoint is better than the one it would replace.

7.7.1 Sample size requirements per empirical claim

Reconciling confidence with testbed capacity starts by fixing the statistical claim before scheduling the cell, because the reliability target, not the length of an afternoon shift, dictates the required sample size. For a zero-failure campaign the count is equation 7.5, with the rule of three as its rare-failure approximation. Figure 7.4 plots it across confidence levels, and table 7.3 tabulates it. The claim (target and confidence) is fixed first, the count follows from it, and if a testing facility cannot afford that count, the engineering team must narrow its claim rather than present an underpowered test as formal qualification. If a campaign records any failure, it must apply the general binomial bound of section 7.6.1. An engineering team cannot salvage a zero-failure claim by appending clean runs after inspecting intermediate data.

7.7.2 Test-cell time and wear per trial

↪ The same bound, counted in operating hours rather than trials, and the population such a statistic describes are developed in section 15.10.1.

The door-opening campaign illustrates how the physical cost of a single trial accumulates. During a nominal execution run, the mobile base navigates inward from its 2 m standoff position and stops. The manipulator arm then executes a guarded unlatching sequence at 0.03 m/s and swings the door open. These physical motions occupy 25 s of active test-cell time. Preparing the testbed for the next statistically independent draw requires an automated reset: re-latching the door mechanism, navigating the base back to its standoff position, and flushing onboard telemetry buffers. This reset sequence adds 15 s, bringing the total wall-clock budget for each trial to 40 s. At this cycle duration, the initial twenty clean runs take 13.3 min. Qualifying a 99.9 percent success lower limit (notebook 7.2) demands a far larger campaign, even before accounting for the thermal cooldown intervals required by the actuator windings.

↩ The recovery time a periodic duty cycle forces between loaded bursts, before the windings reach their temperature ceiling, is established in section 2.5.2.

Napkin Math 7.2: The nines tax

Problem: An evaluation target calls for a one-sided success lower limit of $p_{\text{target}} = 0.999$ at 95 percent confidence. How many zero-failure door runs does it need, and how long does the latch cell take to supply them?

Variables:

- $p_{\text{target}} = 0.999$ and $\alpha_{\text{sig}} = 0.05$ (Chosen; the claim).
- $t_{\text{run}} = 25$ s and $T_{\text{reset}} = 15$ s (Illustrative; drive-in, guarded unlatch, swing, then re-latch and return), so $T_{\text{trial}} = 40$ s.
- two safety operators per run (Illustrative).

Math: From equation 7.5, $n = \lceil \ln(1/\alpha_{\text{sig}}) / \ln(1/p_{\text{target}}) \rceil = \lceil$ 2.996 / 0.0010005 $\rceil$ = 2,995, close to the rule of three's $3/(1-p_{\text{target}})$. The cell time is 2,995 × 40 s = 119,800 s, or 33.3 h (1.4 days) of continuous testing, and 66.6 h of operator time, over eight eight-hour shifts.

Result: 2,995 clean runs and 33.3 h of cell occupancy, against 13.3 min for the opening twenty.

Systems insight: The denominator $\ln(1/p_{\text{target}}) \approx 1 - p_{\text{target}}$ shrinks tenfold with each added nine, so each nine multiplies the runs, the cell hours, and the wear by ten. A longer cycle scales the hours in proportion. A campaign this long also outlives the stationarity it assumes, and a component replaced midway may require a new population definition rather than pooling incompatible runs.

The two panels of figure 7.4 separate two lessons. Early clean runs buy confidence cheaply, and the 95 percent lower limit already reaches 86.1 percent after twenty runs. Past that point the curves crowd under the 99.9 percent target, and the confidence level matters far less than the target itself. On log axes the failure upper limit falls only one decade per decade of trials, so the last nine is the expensive one: 2,995 runs, or 33.3 h of test-cell time at the door cycle.

Table 7.3: One-sided binomial confidence limits after zero failures: Success lower limits are $\alpha_{\text{sig}}^{1/n}$ and the corresponding failure upper limit is $1 - 0.05^{1/n}$. Test times use the 40 s door cycle.

Zero-failure trials n	90% lower success limit	95% lower success limit	99% lower success limit	95% upper failure limit	Test time at door cycle	Interpretation under iid test draws
10	79.43%	74.11%	63.10%	25.89%	0.1 h	Wide uncertainty; no observed failures
20	89.13%	86.09%	79.43%	13.91%	0.2 h	Bound applies only to the sampled distribution
50	95.50%	94.18%	91.20%	5.82%	0.6 h	More evidence; untested conditions remain
100	97.72%	97.05%	95.50%	2.95%	1.1 h	No deployment permission follows from the interval
250	99.08%	98.81%	98.17%	1.19%	2.8 h	Check hardware and sampling stationarity
500	99.54%	99.40%	99.08%	0.60%	5.6 h	Bound remains tied to declared conditions
1,000	99.77%	99.70%	99.54%	0.30%	11.1 h	Below a 99.9% lower-limit target
2,995	99.92%	99.90%	99.85%	0.10%	33.3 h	Meets that statistical target if assumptions hold

The physical plant also degrades as trials accumulate. Statistical confidence bounds assume iid draws from a stationary process, but physical wear breaks stationarity. Transmission backlash (lost motion), effective wheel radius, tire surface friction, and joint damping all drift at rates that engineers must monitor directly on the test hardware. Pooling a thousand trials without that monitoring treats an evolving, degrading machine as an unchanging platform. A failure on trial 950 may reflect testbed wear, a policy limitation, or their interaction; configuration and condition logs are needed to distinguish these causes. The wear itself is budgeted as it is for demonstration collection, by charging each trial's cycles against the rated life of the drivetrain components.

Trials this expensive make stopping early attractive. A fixed-sample, zero-failure protocol already permits safe early rejection: testing halts immediately upon the first observed failure, avoiding pointless exposure to a hazardous policy. Early acceptance, however, presents a different statistical challenge. Halting testing prematurely simply because intermediate results appear favorable inflates the false-positive rate unless the stopping threshold was calibrated in advance. To balance test efficiency with rigorous error guarantees, section 7.7.3 develops sequential boundaries for both acceptance and rejection.

↩ Sizing a physical campaign against operational yield and the fatigue life of drivetrain components is established in section 5.6.3.

7.7.3 Sequential probability ratio testing

Wald's sequential probability ratio test (SPRT)[9] (Wald 1945) provides those pre-calibrated stopping boundaries by evaluating the cumulative log-likelihood ratio after each successive trial, comparing an unacceptable failure rate against an acceptable one. For a strict autonomy campaign, the Bernoulli

[9] **Sequential Testing in Wartime**: The test grew out of work at Columbia University's Statistical Research Group during the Second World War, as a way to accept or reject production lots after inspecting fewer items, and it carried a restricted classification until Wald (1945) set it out in the open literature. The robot campaign faces the same economics, because every trial consumes hardware, and stopping as soon as the evidence is decisive saves trials at the declared error rates.

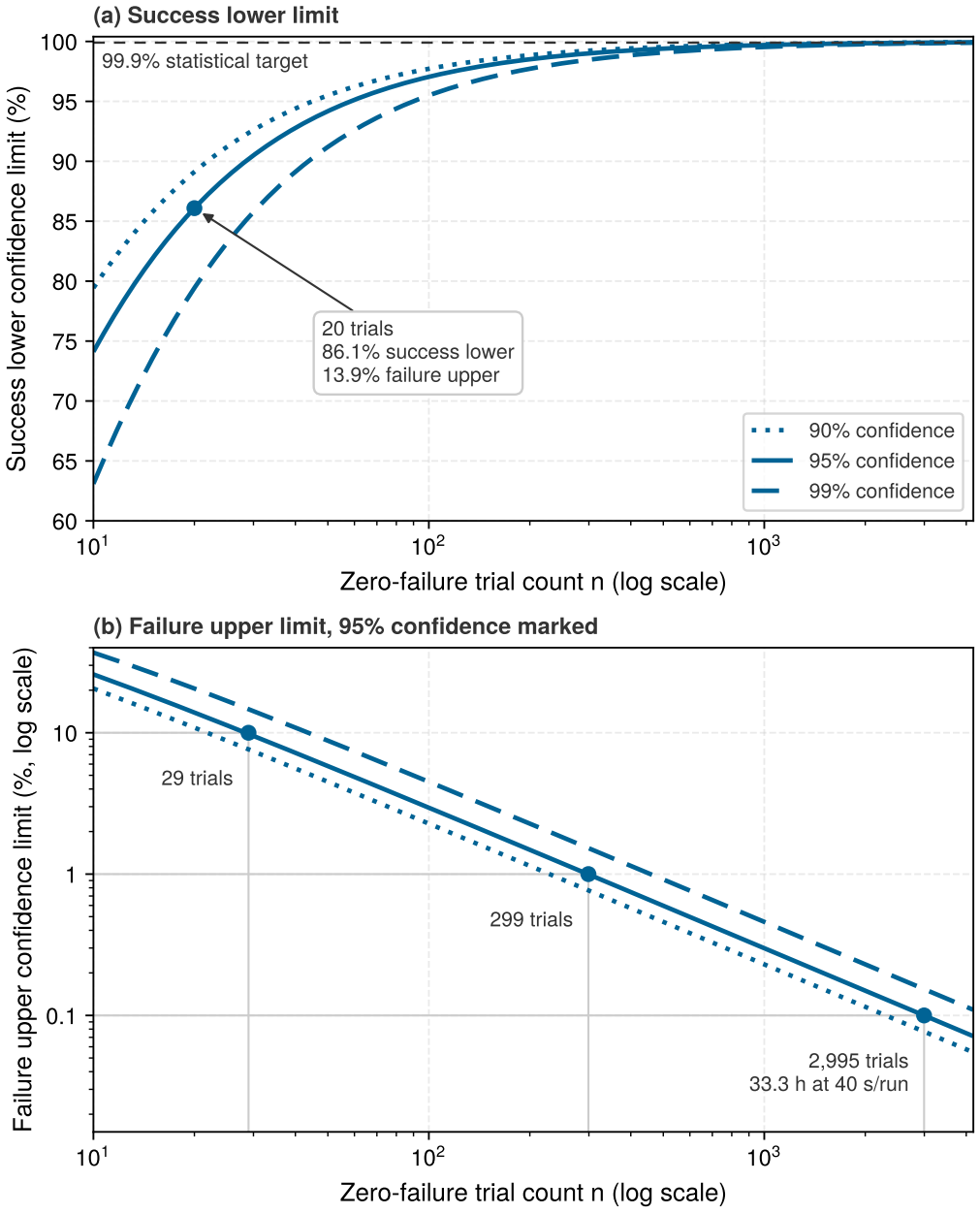


Figure 7.4: Confidence bounds and testing budgets: (a) One-sided Clopper-Pearson success lower limits after zero failures at 90, 95, and 99 percent confidence; at 95 percent, 20 iid runs give 86.1 percent, and the curves then crowd under the target. (b) The same limits read as failure upper limits on log axes fall one decade per decade of runs, so each further nine costs ten times the trials: 99.9 percent needs 2,995 runs, or 33.3 h at the 40 s door cycle.

outcome records unassisted task completion; for a full-system service evaluation, it records the separately specified operational outcome. While the mathematical formulation accommodates either metric, the outcome definition must remain strictly frozen throughout data collection.

Following the convention of section A.9, this subsection states its hypotheses in failure probability. Within it, p_{fail} denotes the per-trial failure probability (the p_{fail} of section 7.6), not the success probability of the preceding sections, and p_0 and p_1 are failure rates. The comparison of two policies

↪ The full SPRT derivation for Bernoulli trials, including Wald's boundary approximations and their overshoot calibration, is developed in section A.9.

that follows returns to success rates. For hypotheses $H_0 : p_{\text{fail}} = p_0$ (unacceptable failure rate) and $H_1 : p_{\text{fail}} = p_1 < p_0$ (acceptable reliability), observing k failures across m independent trials yields the log-likelihood ratio:

$$\Lambda_m = k \ln\left(\frac{p_1}{p_0}\right) + (m-k)\ln\left(\frac{1-p_1}{1-p_0}\right) \tag{7.7}$$

Each success increments the ratio toward acceptance, while each failure drives it sharply toward rejection. With the illustrative hypotheses $p_0 = 0.10$ and $p_1 = 0.01$, a success adds 0.0953 while a failure subtracts 2.303. This metric tracks cumulative evidence for the declared outcome; it never grants permission to continue an unsafe physical run when a safety tripwire is breached.

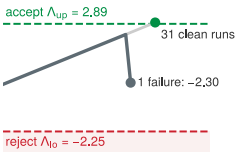
Wald's approximate boundaries on the log-likelihood ratio are:

$$\Lambda_{\text{up}} \approx \ln\left(\frac{1-\beta}{\alpha_{\text{sig}}}\right), \quad \Lambda_{\text{lo}} \approx \ln\left(\frac{\beta}{1-\alpha_{\text{sig}}}\right) \tag{7.8}$$

where α_{sig} represents the allowed probability of accepting H_1 when H_0 holds, and β represents the allowed probability of rejecting H_1 when it holds. For nominal targets of $\alpha_{\text{sig}} = 0.05$ and $\beta = 0.10$, the boundaries evaluate to $\Lambda_{\text{up}} \approx$ 2.890 and $\Lambda_{\text{lo}} \approx$ -2.251. Following each physical rollout m, the evaluation supervisor executes one of three rules:

1. If $\Lambda_m \geq \Lambda_{\text{up}}$: Halt testing and accept H_1 under the declared test (documenting certified evidence within the target ODD).
2. If $\Lambda_m \leq \Lambda_{\text{lo}}$: Halt testing and reject H_1 (policy rejected; abort the campaign immediately to prevent hardware damage).
3. If $\Lambda_{\text{lo}} < \Lambda_m < \Lambda_{\text{up}}$: Continue sampling; execute rollout $m+1$.

Tracing an evaluation run reveals the operational efficiency of this sequential framework. Starting from a neutral score of zero, a failure on the first run crosses boundary Λ_{lo} immediately, halting testing. Conversely, reaching acceptance from zero requires 31 consecutive clean runs, found by dividing threshold Λ_{up} by the per-success increment and rounding up. Unlike a rigid zero-failure protocol, a failure mid-campaign does not force immediate termination. After 25 clean runs and one failure, the score settles at 0.08, remaining between the thresholds, so testing continues. Compare this with a fixed-sample zero-failure test for a 90 percent success lower limit, which requires 29 clean runs and rejects upon any failure. While both designs demand comparable trials on a spotless record, the fixed test rejects an acceptable policy with $p_{\text{fail}} = p_1$ in 25.3 percent of campaigns, whereas the sequential test explicitly bounds that Type II error. The sequential design earns its place on the testbed by controlling both error modes, not simply by promising fewer trials.



Each clean run adds 0.0953; one failure subtracts 2.303.

The error guarantees follow directly from the likelihood ratio itself. Under H_0, the likelihood ratio e^{Λ_m} forms a nonnegative martingale with unit mean, a sequence whose expected next value, given everything observed so far, equals its current value. Because no run is expected to raise it, no rule for choosing when to stop can raise its expected size either. By Ville's inequality,[10] the probability that it ever reaches $e^{\Lambda_{\text{up}}}$ is bounded by $e^{-\Lambda_{\text{up}}}$ (Ville 1939). Under Wald's boundary, that bound evaluates to $\alpha_{\text{sig}}/(1-\beta) = 0.056$, slightly above nominal α_{sig} because the chosen Wald threshold is an approximation. An analogous argument under H_1 bounds false rejection by $\beta/(1-\alpha_{\text{sig}}) =$ 0.105. These are upper bounds, not the realized error probabilities of this discrete test (Wald 1945). Establishing its actual operating characteristics requires calibration that accounts for score-step overshoot. A campaign that must guarantee false acceptance at or below α_{sig} adopts the conservative threshold $\Lambda_{\text{up}} = \ln(1/\alpha_{\text{sig}})$, requiring only a few additional clean runs. A complete qualification record archives the hypotheses, outcome definitions, stopping thresholds, the full trial trace, and the final decision. Even when those stopping rules hold, a sequential design only reduces expected testing time for a single policy against fixed thresholds. It cannot compensate for an ill-defined operational domain, a filtered evaluation denominator, or unverified policy transfer to another physical unit. Deciding between two candidate checkpoints requires a comparative test design.

10 **Ville's Inequality**: Ville's thesis was a critique of von Mises's frequency definition of a random sequence, the "collective" of its title, and it brought martingales into probability through a gambling argument (Ville 1939). The same argument now underwrites anytime-valid tests, whose error bound holds however often the evaluator reads the running score, the freedom a campaign needs when it is watched run by run.

7.7.4 Comparing two policies

Most robot-learning evaluations do not qualify a single policy against a static regulatory target; they compare competing models. An engineering team retrains the door-latching policy and asks

whether the new checkpoint outperforms `latch-bc-01`. Such comparisons inherit every constraint of single-policy evaluation while adding three distinct systems requirements: experimental trial design, exact non-parametric testing, and statistical power analysis.

The first requirement is experimental trial design. Physical testbeds drift while a comparison proceeds: actuator windings warm, latch springs relax, ambient lighting shifts across the day, and tire surfaces wear. If all runs of one checkpoint precede all runs of the other, that physical drift corrupts the comparison, masquerading as a policy effect. The two policies must therefore execute in an interleaved, randomized sequence on the same physical unit and fixture, initialized from matched states sampled from the target ODD. Furthermore, the human operator who resets the cell and annotates borderline outcomes must remain blinded to which checkpoint executed. Kress-Gazit et al. (2024) established these practices as a reporting standard for physical robot evaluation, requiring explicit run counts, stated initial conditions, unambiguous success criteria, and qualitative failure analyses. Toyota Research Institute adopted this discipline when evaluating large behavior models, executing physical comparisons as blind A/B trials (TRI LBM Team et al. 2025).

The second requirement is selecting an appropriate hypothesis test. Each policy arm yields a count of binary outcomes, and the analysis asks whether the two observed totals could plausibly arise from a single shared success rate. Fisher's exact test[11] resolves this for small physical samples by calculating the hypergeometric probability of every contingency table matching the observed margins (Fisher 1922). Suppose, for illustration, that `latch-bc-01` completes all 20 interleaved nominal runs and a candidate completes 16. Evaluating significance requires only combinatorial counting. Under the null hypothesis that both checkpoints share the same success probability, the 40 total runs produced 4 failures. Assigning policy labels under the null is an arbitrary partition of the 40 runs into two arms of 20, making every configuration of 4 failures equally probable. The probability that all 4 failures land exclusively in the candidate's arm is $\binom{20}{4}/\binom{40}{4} = 0.053$. Accounting for an equally extreme split in the opposite direction yields a two-sided p-value (a probability computed under the null hypothesis, not a success rate) of 0.106. Because a disparity this large occurs by chance more than 10 percent of the time at this sample size, the campaign cannot declare either checkpoint superior at $\alpha_{\text{sig}} = 0.05$. Robot-learning evaluations frequently rest on samples this small, where genuine performance gaps remain statistically undetectable (Snyder et al. 2025).

The third requirement is statistical power, which fixes the sample size before executing the first run. Cohen's power analysis determines the necessary trials per arm. The analyst specifies the significance level α_{sig}, the Type II error rate β (or power $1-\beta$), and the minimum effect size worth detecting (Cohen 1988).[12] For two binomial success rates, p_{high} for the better policy and p_{low} for the worse, the effect size is Cohen's $h = 2\arcsin\sqrt{p_{\text{high}}} - 2\arcsin\sqrt{p_{\text{low}}}$. Because the raw binomial variance of an arm's sample proportion, such as $p_{\text{high}}(1-p_{\text{high}})/n_{\text{arm}}$, depends on the underlying success rate and shrinks near 100 percent, a fixed percentage-point gap carries a different sampling variance near the top of the probability scale than near the middle, and the arcsine transformation stabilizes that variance. The sample transform $2\arcsin\sqrt{\hat{p}}$, computed from the success fraction $\hat{p}$ across n_{arm} runs, has asymptotic variance approximately equal to $1/n_{\text{arm}}$ regardless of the underlying rate. Consequently, the difference between two arms has variance $2/n_{\text{arm}}$. Requiring the expected difference h to span the two-sided critical value $z_{1-\alpha_{\text{sig}}/2} = 1.96$ plus the power buffer $z_{1-\beta} = 0.84$, both expressed in units of standard error $\sqrt{2/n_{\text{arm}}}$, gives $n_{\text{arm}} = 2\left((z_{1-\alpha_{\text{sig}}/2} + z_{1-\beta})/h\right)^2$ runs per arm. Distinguishing a 90 percent policy from a 80 percent one ($h = 0.28$) at $\alpha_{\text{sig}} = 0.05$ with 80 percent power requires about 195 runs per arm. The Toyota team reached this same conclusion in real-world policy comparisons: measuring subtle performance differences required substantial sample sizes and rigorous hypothesis testing rather than informal demonstrations (TRI LBM Team et al. 2025).

Physical comparisons also invite the pitfall of optional stopping, because engineers monitoring live rollouts face strong temptations to halt testing as soon as a favored checkpoint pulls ahead. Sequential tests designed specifically for policy comparisons, such as STEP, allow an evaluator to stop as soon as evidence is decisive while holding the Type I error strictly at its declared level (Snyder et al. 2025). They are the comparative counterpart to the single-policy test of section 7.7.3. In parallel, simulators can rank candidates before committing to physical rollouts only after paired

11 **Lady Tasting Tea**: Fisher's best-known illustration of the test concerns a lady who claimed she could tell whether the milk or the tea had been poured into the cup first (Fisher 1935). With eight cups, four of each kind, there are 70 ways to choose which four had milk first, so a perfect score by guessing has probability 1/70, a figure that only exact counting of the arrangements gives. Twenty runs per arm sit in the same small-sample regime, where a normal approximation misstates the p-value.

12 **Statistical Power in Physical Trials**: The probability $1-\beta$ that a test detects a difference that is really there, here the chance that a campaign separates two checkpoints whose success rates truly differ by the stated effect.

sim-and-real trials have validated their rank correlation (section 7.4.2). Table 7.4 summarizes modern implementations of these evaluation mechanisms across the field.

Table 7.4: State of practice in policy evaluation, as of 2026: Each row is a dated instance of a mechanism taught in this chapter; the mechanisms outlast the instances.

Practice	What it measures or controls	Instance (as of 2026)
Reporting standard for robot trials	Conditions, run counts, success criteria, statistics, and failure narratives	Kress-Gazit et al. 2024 (Kress-Gazit et al. 2024)
Paired sim-and-real ranking check	Whether a simulator orders policies as the physical cell does	SIMPLER, Li et al. 2024 (X. Li et al. 2024)
Blind randomized A/B trials	Policy differences with drift and evaluator bias controlled	TRI large-behavior-model study 2025 (TRI LBM Team et al. 2025)
Sequential two-policy test	Early decisions with the Type I error held at its declared level	STEP, Snyder et al. 2025 (Snyder et al. 2025)
Exposure-matched rate comparison	Crash-rate ratios against human benchmarks aligned by road, vehicle, city	Waymo rider-only study, Kusano et al. 2025 (Kusano et al. 2025)

Checkpoint 7.2: Bounds, budgets, and stopping rules

A clean record buys a confidence limit, and the claim, not the facility's schedule, fixes what that limit costs.

Reading the bound

- ☐ **One-sided lower limit**: Explain why the 86.1 percent lower limit from twenty clean runs is a property of the procedure rather than a 95 percent probability that this policy's success rate exceeds it.
- ☐ **Failures in the record**: Formulate equation 7.6 for a twenty-run campaign with two failures and state which way its limit moves relative to one failure.

Pricing the claim

- ☐ **Trial-budget estimate**: Using the rule of three and the 40 s door cycle, estimate the clean runs and test-cell hours that a 99.5 percent success lower limit at 95 percent confidence needs, then compare the count with equation 7.5.
- ☐ **Sequential vs. fixed sample**: Contrast the guarantees that the sequential test's 31 clean runs provide against the fixed test's 29.
- ☐ **Policy comparison**: Explain why 20 of 20 against 16 of 20 does not separate two checkpoints, and what must be fixed before the first comparison run.

Whether an evaluation campaign qualifies a single policy against a target specification or compares two competing models, the findings become reviewable only when the trial count, stopping rule, and reset-adjusted time budget are established in advance. That empirical outcome must remain explicitly bound to the tested configuration and carried forward as an evaluation record.

7.8 The Evaluation Record

When a twenty-run door campaign finishes with twenty clean latches, the pass count alone cannot tell an auditor which strike-plate cell was tested, whether aborted runs were dropped from the denominator, or which network weights were loaded on the robot. The **evaluation record**, the fifth link in the record chain, preserves the formal reasoning that connects empirical observations to a bounded operational claim. It consumes the policy manifest, citing it by cryptographic digest, and incorporates the known absences listed in the provenance record of section 5.11. Across the seven fields of table 7.5, it then documents precisely which operational conditions were tested, how physical outcomes were counted, which statistical assumptions justify the conclusions, and where that

empirical support ends. Its `runtime_monitor_specs` field establishes explicit detection requirements for runtime safety engineering. These specifications require an independent implementation whose sensing latencies, execution timing, and fallback transitions undergo separate validation. By binding measured physical outcomes to declared operational boundaries and isolating human supervisory observations with their source, the record enables review of the learned proposer without granting its weights direct authority over hardware actuators.[13]

[13] **Safety of the Intended Functionality (SOTIF)**: ISO 21448 covers hazards that arise without an electrical or mechanical fault, originating instead from functional insufficiencies or reasonably foreseeable misuse (International Organization for Standardization 2022). That is precisely the failure class of a learned proposer: its neural network weights can remain perfectly intact while still proposing an unsafe trajectory. Consequently, the evaluation record catalogs unsupported operational conditions rather than faulted components. The record serves as an engineering evidence artifact rather than a format mandated by the standard itself.

↪ The sixteen-record chain (fifteen records from Parts I to IV plus the residual-claims register), in which the evaluation record is Record 5, is developed in section C.14.2.

DRAFT

Record Schema Field	Physical / Statistical Modality	Cryptographic & Provenance Guarantee	Downstream Verifier & Usage

Table 7.5: Physical policy predeployment evaluation record schema: Fields the evaluation record adds to the policy manifest it consumes, recording the target ODD, configuration digests, the sampling protocol, the complete-denominator trial log, non-asymptotic confidence bounds, the unsupported coverage gaps, and the runtime monitor specifications.

Record Schema Field	Physical / Statistical Modality	Cryptographic & Provenance Guarantee	Downstream Verifier & Usage
`target_odd_boundary`	Target ODD: the sampled cells of the declared ODD, here the nominal strike plate at the 0.03 m/s guarded approach under the door's own latch load, base stopped	Signed ODD specification; must lie inside the policy manifest's `declared_odd_boundary`	Defining tested conditions and calibrated out-of-distribution alerts; physical gating remains independent
`component_hashes`	Policy manifest digest, frozen neural network weights (hash(θ)), inference engine binary, chassis serial number, unit hardware tolerances (transmission lost motion and stiffness, structural link compliance), tire batch, sensor calibration matrices, and firmware build	SHA-256 binary digests, hardware UUID, ISO 9001 calibration certificate references	Supporting configuration traceability; preventing unrecorded hardware substitutions or unit-specific overfitting
`sampling_protocol`	Target-distribution draw weights or per-stratum quotas, PRNG seed, reset procedure, and independence audit	Automated test harness configuration script hash, telemetry sequence log	Auditing against optional stopping and testing against hidden autocorrelation across trials
`complete_outcome_log`	Full-denominator task, autonomy, and protection outcomes; raw counts (N, $N_{\text{unassisted}}$, N_{int}, n_{estop}, n_{timeout}) and physical metrics (ACR, MTBI, D_{int}, and T_{reset} time-series)	Monotonic microsecond hardware timestamps, uninterrupted ring-buffer telemetry	Preventing censored trial fallacy; calculating uncorrupted empirical reliability and supervisory burden
`statistical_evidence`	Exact non-asymptotic Clopper-Pearson binomial lower bound, rule-of-three limit, or a predeclared sequential decision record with its error calibration; for hazards, a Poisson rate bound from exposure hours (section 7.9)	Cryptographic verification signature, reproducible Python evaluation script	Passing bounded evidence and explicit assumptions to section 15.11

Record Schema Field	Physical / Statistical Modality	Cryptographic & Provenance Guarantee	Downstream Verifier & Usage
`unsupported_gap_catalog`	Explicit catalog of unvisited state spaces and omitted dynamics (surface water, dynamic crowds); contains the provenance record's known absences and the manifest's omission log	Declared non-support manifest, signed safety engineering sign-off	Parameterizing runtime permission-path checks, formal safety constraint sets, and fallback policies
`runtime_monitor_specs`	For each monitorable boundary of the claim: signal, threshold, and tested response, across the five monitoring channels that section 7.9 sets out	Measured detection coverage and false-negative rate per channel	Implemented by the permission path and the observation contract's health field; read by the enforcement record

Engineering claims and acceptance predicates in table 7.5 must be established prior to data collection. For the door's strict autonomy claim, success requires opening the latch mechanism without triggering the 15 N contact tripwire or invoking a permission-path override. The target ODD is restricted to a declared subset of the manifest domain, explicitly separating the tested fixture from doors with alternative strike plates, latch frictions, or return spring rates. Fixing these boundaries before testing prevents post-hoc rationalization, ensuring an evaluation team cannot alter success criteria after observing an anomalous run.

Rigorous configuration tracking ensures that subsequent performance comparisons remain physically interpretable. Cryptographic software hashes pin the evaluated policy weights and inference engine binaries, while chassis serial numbers, measured mechanical tolerances, and calibration logs bind the physical plant. When a camera lens is swapped, motor firmware is updated, or transmission backlash develops in an actuator, the physical plant changes. Recording these modifications allows engineers to assess whether historical evaluation evidence remains valid for the current hardware assembly. A shared component identifier establishes audit traceability, but it cannot guarantee physical equivalence when mechanical wear or manufacturing tolerances alter dynamic response.

The evaluation record preserves the trial protocol, which must be fixed before the first physical run begins to eliminate the optional-stopping hazard of section 7.7.1 and section 7.7.3. The protocol predeclares the required sample size n derived from the claim's target reliability and confidence level, the discrete outcome classifications, and the explicit stopping rule. Under a zero-failure protocol, the campaign halts immediately upon encountering its first failure, formally rejecting the target acceptance condition rather than retroactively adjusting sample sizes. If the campaign completes all n trials without failure, it reports the exact non-asymptotic bound $\alpha_{\text{sig}}^{1/n}$. A pooled binomial bound requires each trial condition to be drawn independently and identically distributed from the target operational distribution. A stratified quota grid instead demands separate inference within each stratum and explicit weighting before any aggregated claim can span the target ODD. The record documents how runs were sampled across the cells of the target ODD, the pseudorandom seed sequence governing mobile base placement, and the automated reset sequence executed between consecutive attempts to re-latch the mechanism and return the base to its 2 m standoff. When the mobile base navigates back to this standoff autonomously, real-world physical side effects accumulate. Each drive cycle warms the motor windings and draws down the battery, and wheel slip creates odometry drift, the dead-reckoning error that grows when wheel-encoder and inertial measurements are integrated. These physical couplings introduce statistical dependence between adjacent attempts. The record therefore specifies whether trials were conducted with independent physical resets or chained in series, cataloging every manual intervention, hardware abort, and protocol deviation.

The outcome log gives every run four distinct classifications: task completion, strict proposer autonomy, safety intervention history, and physical protection status. Decoupling these dimensions prevents optimistic misclassification. For example, a trial in which the permission path clamps motor torques to prevent an impact can satisfy task completion ($Y_i = 1$) while failing strict proposer autonomy ($A_i = 0$). Conversely, an emergency protective stop that halts the manipulator before collision safely mitigates the physical hazard, yet it leaves the latch closed, yielding a task failure ($Y_i = 0$). Omitting censored trials is strictly forbidden: timeouts, communication dropouts, and hardware aborts must remain in the trial denominator. The log reports ACR, MTBI, spatial intervention density, and recovery reset durations, stratified by intervention class. Alongside these reliability statistics, the log archives raw physical telemetry: cycle durations, peak contact forces, actuator temperatures, operational clearances, and limit crossings. Preserving these fields ensures that downstream reviewers understand both useful task delivery and the physical wear or supervisory burden required to sustain it.

The statistical specification formally defines the point estimator, confidence method, nominal confidence level, assumed sampling distribution, and underlying mathematical validity conditions. For fixed-sample campaigns, the record mandates exact non-asymptotic formulations, such as the Clopper-Pearson binomial lower bound ($p_{\text{lower}} = \alpha_{\text{sig}}^{1/n}$) and, for hazard rates, the exposure bound that section 7.9 develops next, rejecting degenerate normal approximations that fail near boundary probabilities. If the campaign employs sequential testing, the record preserves the competing operational hypotheses, decision thresholds, complete outcome sequence, terminal boundary decision, and calibrated error rates. As detailed in section 7.7.3, sequential testing allows valid early stopping without inflating Type I or Type II error rates, provided the stopping boundaries are enforced rigorously. Alongside the supported claim, the record documents unsupported claims and known coverage gaps. This catalog incorporates known absence regions from the demonstration datasets and unmodeled simulator dynamics from the policy manifest, appending any operational conditions left unvisited during physical testing. For coverage gaps amenable to onboard perception, the runtime monitor specifications define the monitoring signal, detection threshold, and validated fallback action across the five monitoring channels that section 7.9 sets out once it has shown which gaps testing cannot close. Any coverage gap that cannot be sensed at runtime remains in the catalog as a hard operational constraint, restricting where and how the physical machine may be deployed.

Consider the twenty-run door campaign that opened this chapter. The learned latch policy engaged and opened the cage door twenty times from the 2 m standoff at the 0.03 m/s guarded approach speed, with the mobile base stationary and zero overrides from the runtime permission path. An auditor inspecting the evaluation record interprets this evidence through the exact non-asymptotic Clopper-Pearson bound of section 7.6. At a 95 percent confidence level, twenty clean runs establish a lower confidence limit of 86.1 percent. This result rejects the degenerate interval $[1.0, 1.0]$ that the Wald normal approximation reports for a spotless record.

The completed record exposes the structural limits of the claim, showing how far evidence narrows on its path from offline data curation through policy training. The scenario table of section 5.7.1 crossed three strike-plate positions with two approach speeds, yielding six cells in total, yet the retained demonstrations supported only one: the nominal strike plate at the guarded approach. The policy manifest designated that single cell as the declared ODD of `latch-bc-01`, and the target ODD encompassed the entire cell. Twenty clean physical trials were drawn from it. They bound the failure probability there at 13.9 percent, the one-sided upper confidence limit under the tested iid distribution (table 7.3). Every condition excluded by this narrowing enters the coverage gap catalog as an inherited deficit rather than an empirically closed claim. The five operational cells that contained zero demonstrations received zero physical trials, leaving every displaced-plate condition unvalidated. The policy manifest's omission log does hold two unclosed simulator omissions (soft penalty contact dynamics and missing network transport lag). Section 6.8, however, scoped them to the simulation-refined checkpoints, the distilled student `latch-sim-02` (initialized from `latch-bc-01`, then distilled from a simulator teacher) and the residual candidate `latch-res-03` built on it. Because `latch-bc-01` was trained directly on hardware demonstrations without a simulator, its gap catalog does not inherit those omissions. The record names them only to state that its bound does not transfer. The simulation-refined checkpoints require their own empirical campaigns, and the evaluation record of `latch-bc-01` can neither resolve their simulation discrepancies nor lend its

statistics to their modified weights. Physical testing introduces an additional operational gap. The mobile base carried zero payload in its 50 kg tote rack during these trials, so the record provides no empirical evidence for machine dynamics under loaded conditions. Finally, the record makes plain why twenty clean runs cannot justify high-reliability deployment. Establishing a 99 percent lower confidence limit on task success at 95 percent confidence requires at least 299 consecutive zero-failure trials. The evaluation record thus gives a reviewer a quantitative basis for rejecting the assertion that the policy is ready for unmonitored production deployment.

7.8.1 Qualifying a recovery update

Retraining a policy to recover from a displaced strike plate modifies every layer of its neural network weights, so the twenty clean nominal trials logged for `latch-bc-01` no longer certify the updated model even on the centered strike plate. A separate, explicitly stipulated teaching branch therefore evaluates `latch-bc-recovery-02`, the candidate policy trained on the curated `latch-recovery-02` archive of displaced-plate recovery demonstrations, while keeping the original nominal campaign unchanged. This added experience covers one displaced-plate cell, the positive-offset plate in the scenario table of section 5.7.1, at the same 0.03 m/s guarded approach speed. The negative-offset plate and all three cells at the faster approach remain known absences. It provides a concrete engineering justification to test expanded competence, though training convergence alone does not qualify the policy for operational deployment.

↩ The curated recovery archive from which `latch-bc-recovery-02` is trained is established in section 5.8.

The evaluation engineer freezes the new network weights and reserves fresh physical runs for both the nominal cell and the recovery cell. The nominal campaign verifies that the policy has not regressed against the original service baseline, while the displaced-plate campaign evaluates the proposed capability expansion. Each campaign maintains its own full denominator, declared within-cell sampling distribution, fixed execution time limit, strict autonomy predicate, and contact force thresholds. Exploratory rollouts and the earlier checkpoint's twenty trials cannot be credited toward either candidate's denominator.

Suppose, solely for this worked evidence branch, that each cell supplies 2,995 independent zero-failure trials under the predeclared protocol, with on-time door completion, zero counted overrides, and no contact-limit violations. Each cell individually satisfies the 99.9 percent one-sided lower success target at 95 percent confidence. These represent separate marginal bounds; establishing a simultaneous claim across both cells requires an explicit confidence allocation because when two cells are tested separately at that confidence level, each cell carries up to a α_{sig} chance of a false pass and the union bound allows up to $2\alpha_{\text{sig}}$ total risk that at least one of the two claims is false. Splitting the significance level α_{sig} equally between the two cells using a Bonferroni allocation assigns each cell $\alpha'_{\text{sig}} = 0.025$, raising the per-cell requirement to 3,688 clean runs. At the illustrative 40 s cycle duration, each 2,995-run marginal campaign consumes 33.3 h of testing, and the Bonferroni count lengthens it in proportion, before accounting for recovery-specific resets, thermal cooldowns, or scheduled maintenance. Variations in the physical cycle time alter the total testing duration, but they do not alter the underlying statistical sample size required to certify the claim.

Under these stipulated premises, with 2,995 clean runs per cell, the evaluation record supports two marginal 99.9 percent claims for the updated policy candidate, one on the nominal cell and one on the displaced-plate cell of the tested robot. It does not support the simultaneous claim that both hold at once. That claim would need 3,688 clean runs per cell. The original policy candidate retains its narrower operational boundary. Strike plates at other lateral offsets, higher approach velocities, loaded chassis dynamics, and uncalibrated robot units remain outside this certified envelope. If the runtime permission path executes protective adjustments while tasks still complete, the evaluator records successful whole-machine service alongside reduced strict proposer autonomy, recomputing statistical bounds directly from those distinct outcome classes. System operators must not conceal permission-path interventions to artificially inflate autonomous completion rates, nor should they conflate a safe intervention with an unmitigated service failure.

This teaching branch completes the policy improvement cycle without claiming that an unperformed physical experiment took place. Incorporating recovery demonstrations alters the policy candidate; conducting independent regression and expansion trials alters what can be formally claimed about that candidate. The numerical results presented above follow from stipulated trial counts rather than empirical measurements of the physical machine described in this book.

Rather than serving as a blanket safety certification for the autonomous machine, an evaluation record establishes the formal empirical boundaries and statistical uncertainty of an individual learned component operating under controlled conditions. In section 15.11, the fault-record set ingests these evaluation records as structured evidence, combining empirical policy bounds with fault-injection results and permission-path latency bounds for the safety case that decides whether the integrated machine may operate.

↪ The distinction between a demonstration and calibrated evidence in a release safety case is developed in section 16.3.1.

Systems Perspective 7.2: Evaluating a machine that falls

An evaluation record must account for physical damage when every failure in an unsupported cell is a fall that damages the test article.

A failed guarded latch engagement can leave the wheeled mobile manipulator and mechanism intact for subsequent trials when qualified precontact limits, passive protection, and intervention response make the engagement recoverable. A reactive contact tripwire alone cannot prevent every initial impact. When a bipedal humanoid loses balance, however, a single failure can strike the floor, destroying actuators, structural links, and onboard sensor payloads.

- **Fall frequency constitutes an independent estimand**: Because a catastrophic fall damages the physical test article, the evaluation record must track fall events using a dedicated denominator and an independent statistical confidence bound, rather than burying structural collapse inside a blended task completion metric.
- **Disturbance strata must be fixed prior to testing**: Environmental perturbations (low-friction spills, compliant turf, curb thresholds, and horizontal impact impulses) along with their respective sampling weights must be declared in the sampling protocol before commencing physical evaluation. Testing exclusively on polished concrete provides no support for unvisited disturbance regimes.
- **Human demonstrator stability does not transfer to robot dynamics**: Retargeted human motion reflects the human demonstrator's balance and vestibular feedback, whereas teleoperation and kinesthetic demonstrations collected on the robot involve the robot's physical plant. Either modality can omit balance and recovery states under relevant disturbances, including dynamic reaches under heavy external loads. Those omissions belong in the coverage gap catalog. Investigating them requires appropriate containment rather than exposing an unqualified test article to destructive trials.

By keeping these cell boundaries explicit, the evaluation record separates the nominal and recovery behaviors it validated from the operating regimes it did not. Rare physical hazards and unobserved dynamic mechanisms remain distinct evidence obligations that an autonomous task campaign cannot dismiss, even when every monitored trial completes successfully.

7.9 Verification Ceilings and the Exposure Wall

A clean record on a bounded manipulation task, such as cycling a warehouse door latch, supports a task-success claim under the sampled nominal operating conditions. It does not, however, establish that the system is safe against rare hazards or catastrophic structural overloads. Extending task success claims to rare physical hazards demands an explicit operating exposure budget, while bounding peak mechanical loads requires an explicit physical contact model.

7.9.1 Rare-event exposure

Failing to pull a warehouse door latch open wastes a cycle, whereas crushing an obstacle or shearing a structural pin creates a physical hazard whose rate must be bounded across every hour the robot stays energized. Evaluating task success and evaluating system safety therefore require different statistical currencies. Repetitive manipulation tasks measure success across discrete trials, whereas physical hazard bounds measure event rates per unit of operating exposure, typically operating hours or travel distance. Engineers cannot conflate a task failure with a hazardous outcome, nor can they infer physical hazard severity from a trial success percentage alone.

Under the assumption of a constant hazard rate λ, an empirical test that logs an uninterrupted operational exposure T without observing a hazardous event yields a record whose likelihood is $e^{-\lambda T}$, the continuous-time analog of the Bernoulli trial probability p^n. Setting this observation probability equal to the significance threshold α_{sig}, exactly as equation 7.3 established for discrete trials, yields the minimum failure-free exposure time required to bound the hazard rate from above:

$$T \geq \frac{\ln(1/\alpha_{\text{sig}})}{\lambda} \approx \frac{3.0}{\lambda} \tag{7.9}$$

evaluated at a 95 percent confidence level ($\alpha_{\text{sig}} = 0.05$). This relationship is the continuous-time formulation of the rule of three, substituting continuous operating hours for discrete test cycles. In life-critical cyber-physical systems, mandated rate targets routinely reach 10^{-9} dangerous failures per operating hour.[14]

Definition 7.2: Exposure wall

Exposure wall is the failure-free operating exposure, $T \geq \ln(1/\alpha_{\text{sig}})/\lambda$, that a zero-event test must accumulate before it can bound a hazard rate λ at a stated confidence, a requirement that grows tenfold with each tenfold reduction in the target rate.

1. **Significance**: At 95 percent confidence the wall sits near $3/\lambda$, so a claim of 10^{-9} per hour needs 3.0×10^9 failure-free hours, against the 33.3 h that the door's 99.9 percent trial claim occupies. Past the wall, evidence for a hazard rate must come from the permission path and physical safeguards rather than from testing the learned proposer.
2. **Distinction**: The exposure wall bounds a rate from event-free hours, and the zero-failure trial bound (equation 7.3) bounds a per-trial probability from clean trials. Both invert a clean record; neither estimates a mean time between failures from observed failures.
3. **Common pitfall**: A frequent misconception is that fleet hours carry a claim past the wall. Under the constant-rate model, parallel rigs do add exposure hours, which is why the notebook below divides the wall across a fleet. What they fail to add is diversity. Hours pooled across machines that share a facility, a production batch, and an operating protocol bound the rate only for the conditions those machines share, and simulated hours add no physical exposure at all.

Napkin Math 7.3: Rig-years against the wall

Problem: How much failure-free exposure does equation 7.9 demand to bound a hazard rate of 10^{-6} and of 10^{-9} per hour at 95 percent confidence, and what rate can one test rig bound in a year of continuous running?

Variables:

- $\ln(1/\alpha_{\text{sig}}) = 2.996$ (Chosen; 95 percent confidence).
- $\lambda = 10^{-6}$ and 10^{-9} per hour (Chosen; analytical targets at the magnitudes of aviation and IEC 61508 bands).
- One rig-year = 8,766 hours (Derived; continuous operation) and 100 rigs (Illustrative).

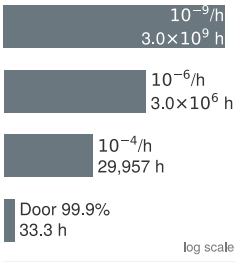
Math: \$T= \$ 2.996 $/10^{-6} = 3.0 \times 10^6$ hours, or 342 machine-years; at 10^{-9}, $T = 3.0 \times 10^9$ hours, which 100 rigs running continuously would need about 3,420 calendar years to supply. Inverting for one rig-year gives an upper confidence limit of 2.996 / 8,766 $= 3.4 \times 10^{-4}$ per hour.

Result: A rig-year of clean running bounds the rate only to 3.4×10^{-4} per hour, about 340 times short of 10^{-6} and 340,000 times short of 10^{-9}.

Systems insight: A rig-year of testing reaches the order of a site's mission-reliability target, such as the 10^{-4} per hour campaign sized below. Each further decade of rate costs a decade of

[14] **Ultra-Reliability Targets**: The figure comes from civil aviation, where a catastrophic failure condition must be "extremely improbable," on the order of 10^{-9} per flight hour (Federal Aviation Administration 1988). IEC 61508 sets a comparable band for its highest safety integrity level, SIL 4, of 10^{-9} to 10^{-8} dangerous failures per hour in continuous operation (International Electrotechnical Commission 2010a). The exposure-wall examples borrow these magnitudes as analytical targets and are not claims of compliance with either document.

machine-years. Claims at 10^{-6} and beyond must therefore rest on the permission path and physical safeguards, whose own faults can be injected and tested at the component level.



Failure-free hours climb from 33.3 h for the door to 3.0×10^9 hours at 10^{-9} per hour.

Systems Perspective 7.3: Limits of testing for ultra-reliability

The exposure wall is not a shortage of test budget that a larger program could cure. Butler and Finelli made this argument for fly-by-wire flight control software, whose catastrophic failure rate must satisfy $\lambda \leq 10^{-9}$ per hour. The failure-free exposure such a requirement demands exceeds any feasible test program by orders of magnitude, so operational testing alone cannot validate it (Butler and Finelli 1993). The evidence for a requirement past the wall must therefore come from the architecture, from what the design makes impossible, rather than from what testing failed to observe.

Butler and Finelli's conclusion (perspective 7.3) carries over to physical AI unchanged. Notebook 7.3 priced the wall in rig-years, and the definition's pitfall explains why a fleet of parallel rigs buys hours but not the diversity of conditions a deployment claim needs. Autonomous road vehicles confront the same wall at scale. Placing one real fleet's exposure against RAND's mileage requirements (figure 7.5) shows that 56.7 million rider-only miles clear the zero-failure bars for injuries and crashes and the crash precision bar. The same exposure falls short of every fatality requirement, by more than two orders of magnitude for the precision and better-than-human rows. High-assurance physical AI systems must therefore establish safety through deterministic runtime constraints and structured hardware fault injection. While rare-event simulation and importance sampling can reduce the required sample count by orders of magnitude, their mathematical validity extends only as far as the simulator's physical fidelity in those extreme tail regimes.

↪ Rare-event importance sampling, and the proposal-coverage assumption its estimates rest on, are developed in section 15.10.2.

Empirical road testing demonstrates concretely what fleet-scale operational exposure can and cannot substantiate. The first benchmark evaluated in the RAND study represents the exposure wall translated into driving mileage. Given an average human driver fatality rate of 1.09 per 100 million miles, an autonomous vehicle fleet must log 275 million failure-free miles to establish an upper bound matching the human baseline at 95 percent confidence. That single demonstration requires approximately 4.9 times the total rider-only commercial mileage accumulated by Waymo through January 2025 (56.7 million miles). Producing an empirical estimate of the fatality rate within a 20 percent margin of error requires 8.8 billion miles.

Nevertheless, tens of millions of operational miles can validate claims regarding higher-frequency events. Kusano et al. (2025) evaluated crashed-vehicle rates across 56.7 million rider-only miles against human driving benchmarks matched for vehicle class, road type, and operating jurisdiction. They demonstrated a 79 percent reduction in police-reported injury crashes and an 81 percent reduction in airbag-deployment collisions, both with confidence intervals that exclude zero. A finite operational exposure bounds only those event classes whose expected counts within that window are large enough to measure. Any rigorous comparative safety claim requires an exposure-matched baseline and confidence intervals over the incident rate ratio. The rarest and most severe failure modes remain untestable through on-road fleet mileage alone, and simulated driving mileage adds no true physical exposure to these empirical totals.

The exposure wall restricts verification even for target failure rates orders of magnitude less stringent than aerospace standards. Consider an industrial requirement for the warehouse mobile manipulator. The platform must certify a critical failure rate $\lambda \leq 1.0 \times 10^{-4}$ failures/hour, corresponding to a Mean Time Between Critical Failures of MTBF $\geq 10{,}000$ hours[15] at a 95 percent statistical confidence level ($\alpha_{\text{sig}} = 0.05$). In this operational scenario, each warehouse retrieval mission takes $T_{\text{mission}} = 360$ s (0.10 h) and covers 540 m at the platform's 1.5 m/s maximum drive velocity. Automated docking, battery replenishment, and telemetry offloading introduce an overhead of $T_{\text{reset}} = 120$ s, resulting in a total cycle duration of 480 s per trial. Each drive wheel uses a direct-drive hub motor with an outer diameter $D_{\text{wheel}} = 266$ mm, eliminating mechanical gearbox reduction between the motor and the wheel rim.

15 **MTBF and a Constant Rate**: The constant-rate model used here describes the flat middle of the bathtub curve after burn-in; it makes the reciprocal rate an MTBF and supports the zero-event bound $T \geq 3/\lambda$. Wear-out mechanisms such as bearing fatigue raise the rate as revolutions accumulate, so a rate bounded this way says nothing about wear-out life, which the L_{10} rating below addresses separately.

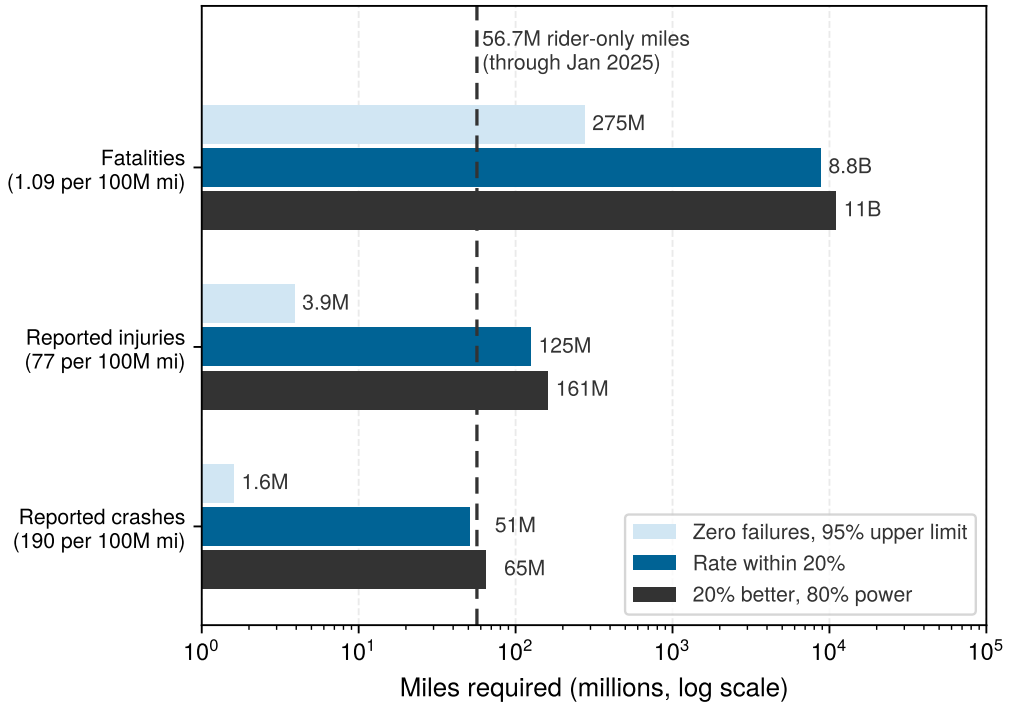


Figure 7.5: Miles needed to demonstrate road-vehicle reliability: RAND's Table 1 requirements at three human-driver benchmark rates (Kalra and Paddock 2016b), for three statistical questions: a zero-failure 95 percent upper limit at the benchmark, a rate estimate within 20 percent, and a demonstration that the rate is 20 percent better than human at 80 percent power. The dashed line marks 56.7 million rider-only miles, Waymo's total through January 2025 (Kusano et al. 2025). That exposure clears the zero-failure rows for injuries and crashes and the crash-precision row, and falls short of every fatality requirement.

Applying equation 7.9 reveals that establishing this 10^{-4} failures/hour ceiling without observing a single failure requires 29,957 hours (3.42 years) of continuous, uninterrupted operation. Discretized across operational sorties, each mission has an allocated failure probability budget of $p_{\text{fail}} \leq \lambda \cdot T_{\text{mission}} = 1.0 \times 10^{-5}$. Inverting the zero-failure binomial model from section 7.6 yields $n = \ln(\alpha_{\text{sig}})/\ln(1-p_{\text{fail}})$, demanding 299,572 consecutive clean missions. This total dwarfs typical empirical validation budgets, which rarely exceed hundreds or thousands of runs. Conducting these trials sequentially on a single physical machine at 480 s per cycle requires 39,943 elapsed hours (4.56 calendar years). It also subjects each direct-drive wheel to $N_{\text{rev}} = (n \cdot v \cdot T_{\text{mission}})/(\pi D_{\text{wheel}}) \approx 1.94 \times 10^8$ continuous revolutions.

Because the hub motor drives the wheel without an intermediate gearbox, every wheel revolution directly rotates the internal hub bearings. Their endurance is rated by the L_{10} convention of section 5.6.3, here the revolutions that 90 percent of an identical bearing cohort completes before fatigue flaking at a stated load, a population statistic that falls steeply as the carried load increases. Determining whether this test campaign exhausts the mechanical endurance of the running gear requires comparing N_{rev} against the manufacturer's internal bearing load rating and measuring the actual static and dynamic wheel reactions with corner-scale load cells. This is the stationarity problem of section 7.7.2 at the scale of a reliability campaign. The revolution count states how much wear the campaign itself inflicts, and only an L_{10} rating at the measured load can say whether that wear reaches the fatigue regime. Because hub-motor datasheets rarely disclose that internal load rating, a campaign of this duration cannot be shown to stay clear of wear-out until the load is measured. Systems teams must therefore schedule, log, and isolate component refurbishments and sensor recalibrations before pooling runs across extended operating intervals.

The distinction drawn in section 7.2 between MTBF and the intervention-based metrics matters most in a supervised campaign. When operating with real-time safety guardrails, permission-path filters, software limits, emergency stops, and human supervisor overrides can intercept hazardous trajectories before physical damage occurs when their sensing, response, and physical margins are qualified for those conditions. Such interventions censor the unassisted physical outcome while remaining recorded autonomy failures. The campaign measures an empirical intervention rate, $\lambda_{\text{int}} = N_{\text{int}}/T_{\text{active}}$, whose inverse is the MTBI.

Consider an evaluation run in which the permission path or human supervisors intervene 38 times during 100 retrieval missions configured identically to the scenario above. With each sortie lasting 360 s and covering 540 m, the campaign aggregates 10 h of active physical movement across 54 km. These trials yield an MTBI of approximately 15.8 min and an intervention spatial density of 0.70 per kilometer. Because interventions occur frequently throughout the campaign, the resulting rate parameter supports rigorous statistical inference. If interventions are modeled as a Poisson arrival process, the exact 95 percent Garwood confidence interval for λ_{int} spans 2.7 to 5.2 interventions per active operating hour around the empirical mean of 3.8 (Garwood 1936).

A system can achieve zero hardware damage while simultaneously imposing an intolerable intervention overhead. Aggregate intervention counts do not reveal ACR, because multiple interventions can cluster within a single difficult run, while other missions might abort without triggering a formal intervention. Determining autonomous completion requires inspecting the cycle-by-cycle mission log. Furthermore, observing zero hardware damage across 10 h of supervised testing provides no statistical evidence that the unassisted policy possesses an MTBF of 10 h. From the standpoint of unassisted physical reliability, every intervened mission represents a right-censored observation. The safety layer terminated the trial before its unconstrained physical trajectory could unfold. The log captures neither a confirmed physical disaster nor an autonomous success. Interpreting these censored runs as demonstrations of unassisted task survival commits the censored trial fallacy detailed in section 7.4. In deployment budgets, the reset and recalibration latency following each intervention represents an unavoidable operational penalty.

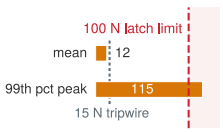
7.9.2 Peak loads and unvisited conditions

Operating exposure represents only one dimension of the verification ceiling. A second boundary arises because safety-constrained testing censors the physical regimes that pose the greatest operational risk (section 7.4). Because a destructive physical failure cannot be undone (the first law, irreversibility), supervisory safety interlocks must truncate trajectories before structural damage occurs. The empirical record terminates exactly where the intervention fires.

↩ The first law, irreversibility, which holds that work delivered to the plant cannot be rolled back, is established in section 1.10.

The corner cases an engineering team most needs to characterize, such as loss of traction on slick surfaces at maximum transit speed, dynamic wheel slip, or imminent collisions in tight aisles, are the physical regimes the system cannot be permitted to explore. Allowing an automated mobile manipulator to skid into structural shelving damages the drive chassis, knocks calibrated lidar and camera constellations out of alignment, and sidelines the entire test cell for mechanical repair. Physical cages, virtual geofences, and human operators with emergency stops truncate trials at a conservative safety boundary. The test log records an intervention event and counts it in the denominator, but the physical consequences of the policy's unconstrained commands remain unobserved.

When evaluating physical hardware survival, average metrics can mask structural overloads. Consider a learned mobile manipulation policy whose average latch contact force across one hundred nominal trials is only $\bar{F} = 12$ N, below the 15 N contact tripwire threshold. However, if the gripper strikes the strike plate at a slight angular offset, the tail of the empirical peak-force distribution reaches a 99th-percentile impact of 115 N. This transient peak breaches the structural latch limit of 100 N despite the benign average. A peak of that size in a campaign guarded by a 15 N tripwire is not a contradiction. The first impact peak arrives faster than the tripwire can act, so these trials record the overshoot first and are ended by the tripwire afterward, as failures. The rise-time argument below shows why no reactive monitor can prevent it. While a single force peak does not directly equate to total impulse or plastic deformation (which depend on the full force-time profile, structural load paths, and material yield criteria), it reveals how central tendencies conceal destructive tail behavior.



A 12 N mean hides a 115 N peak past the 100 N limit.

The undamped spring-mass model of equation 6.7, which in section 6.4.2 exposed how a soft simulated spring understated the latch force, bounds this initial impact. Written for evaluation, its moving mass becomes the reduced mass of the colliding bodies, $m_{\text{red}} = (1/m_{\text{eff}} + 1/m_{\text{obj}})^{-1}$, which reduces to the arm's effective inertia against an immovable frame. Its spring becomes the local contact stiffness k_{contact}. The peak force is then $v_{\text{approach}}\sqrt{m_{\text{red}}\,k_{\text{contact}}}$, and it arrives a quarter of the natural oscillation period after touch, a rise time of $\Delta t_{\text{rise}} \approx (\pi/2)\sqrt{m_{\text{red}}/k_{\text{contact}}}$. Inverting the peak for a structural threshold F_{lim} gives the quantity an evaluation campaign can actually enforce, a precontact speed limit:

$$v_{\text{approach}} \leq \frac{F_{\text{lim}}}{\sqrt{m_{\text{red}}\,k_{\text{contact}}}}, \tag{7.10}$$

This physical relationship governs the transient contact velocity limits in ISO/TS 15066 Annex A for power- and force-limited collaborative robots (International Organization for Standardization 2016). The collaborative application requirements of ISO/TS 15066 have since been incorporated into ISO 10218-2:2025, and ISO/TS 15066:2016 remains published while it is revised into the international standard ISO 15066-1 (International Organization for Standardization 2025b).

Evaluating this bound on a physical door latch requires measuring both the effective mass of the arm at the end-effector and the local contact stiffness of the latch mechanism. These parameters depend heavily on joint posture and kinematics, and they are never found in high-level hardware datasheets. The systems takeaway, however, is structural rather than numerical. If a software monitor's sensing, compute, and actuation latency exceeds the mechanical rise time Δt_{rise}, no reactive software policy can attenuate the initial force spike, regardless of model inference speed. Software intervention comes too late. Physical protection against initial impact must therefore rely on two pre-contact mechanisms: passive mechanical compliance, which lowers effective stiffness k_{contact} and extends rise time Δt_{rise}, and strict pre-contact kinematic velocity caps, such as the platform's 0.03 m/s guarded approach limit.

Extreme value theory (EVT), developed in section 15.10.3, models such tail extremes by fitting a limiting distribution family to many independent block maxima, yet the physical mechanics of contact do not prescribe which family applies.[16] In the absence of thousands of destructive physical trials, table 7.6 pairs measurable physical disturbance channels with concrete architectural safeguards rather than relying on unverified tail extrapolations.

[16] **Extreme Value Theory**: Fisher and Tippett (1928) found the three limiting forms that the largest member of a large sample can take, Gnedenko (1943) proved that under general conditions no other limits exist, and Gumbel (1958) made the theory an engineering tool for floods and material fatigue. Each of those applications fitted many block maxima.

Table 7.6: Peak-disturbance evidence and protection: Mechanisms identify measurements and safeguards, not an EVT family or event rate. Any quantitative tail claim needs a defined population, fitted model, goodness-of-fit checks, and uncertainty.

Disturbance channel	Peak quantity and sampling record	Physical consequence to test	Candidate protection to validate
Compute and bus latency	Per-cycle end-to-end age and deadline misses	Stale proposals erode braking margin	Chunk lease and independently timed fallback
Contact shock	Force-time trace, impulse, and peak at the load path	Yield or overload before a reactive command takes effect	Passive compliance and precontact speed limit; reactive limits address later load
Low traction	Local friction estimate and stopping response	Slip extends stopping distance	Conservative speed ceiling and tested fallback on low-grip surfaces
Optical glare	Saturation duration and perception misses	False clearance or lost state estimate	Redundant sensing and independent physical constraints

The door latch campaign shows why the contact shock entry in table 7.6 pairs a measurement with an architectural safeguard rather than with a fitted tail. Each trial yields only one block maximum, its peak latch contact force, and twenty maxima cannot fit a generalized extreme value tail that would predict the probability of exceeding 100 N. The safeguard therefore comes from the rise-time argument above. The guarded 0.03 m/s approach is a candidate precontact limit whose adequacy depends on the measured effective mass, contact stiffness, and structural threshold. Reactive force monitoring addresses the sustained load that follows.

Figure 7.6: Standardized robotic evaluation facility: NIST response-robot test bays photographed in 2013 (Linda Joy/NIST); their test methods are standardized by ASTM subcommittee E54.09 on response robots. A zero-failure run across standardized bays establishes competence inside the arena and says nothing about glare, surface contaminants, or obstacles outside it.

Deploying parallel robots across a test fleet does not establish statistical independence, as the pitfall in definition 7.2 warns. Ten identical machines operating concurrently in a single warehouse share the same manufacturing batch, firmware revision, concrete sealant finish, heating and ventilation airflow, and operator handling protocols. Environmental disturbances absent from that facility, such as low-angle winter solar glare, surface water pooling, or floor polishing glazing, affect none of the machines. Standardized physical test fixtures, such as the response-robot test bays captured in figure 7.6, ensure repeatable benchmark comparisons by strictly constraining the test environment to known, fixed obstacle geometries.

The standardized obstacle courses documented in figure 7.6 enable repeatable comparisons across robotic platforms under ASTM E54.09 test methods. However, their controlled geometry, uniform surface traction, and regulated indoor lighting also define the perimeter of empirical evidence.

Validating system performance against factory chemical spills, erratic pedestrian crossings, or blinding solar glare requires intentionally introducing those environmental distributions into the test matrix.

These physical and statistical verification ceilings lead to an architectural conclusion. Safety in physical AI cannot rest on statistical confidence derived from testing learned weights. The exposure that failure rates between 10^{-6} and 10^{-9} per hour demand, from 3.0×10^6 to 3.0×10^9 failure-free hours, shows that empirical test volume cannot serve as the primary safety argument. Deterministic runtime constraints must supply their own independent verification evidence, including explicit physical assumptions under which hazardous transitions are prevented and rigorous bounds on worst-case execution time. Empirical policy trials measure task competence and quantify intervention rates inside a nominal operational envelope; they do not prove that the system can contain arbitrary out-of-distribution failures. The burden of safety certification therefore shifts from the learned proposer to the deterministic permission path and verified physical safeguards, each of which must be evaluated through independent verification pathways.

A model-based stability certificate, such as the Lyapunov argument developed in section B.3.1, provides structural guarantees that empirical finite trials cannot establish. When the learned proposer lacks a validated certificate for the declared plant and observation model, the embodied computing platform must provide architectural containment and targeted fault injection:

1. **Runtime Containment**: The system enforces hard physical bounds on the learned policy by delegating collaborative force thresholds, kinematic acceleration boundaries, and emergency stop triggers to an independent, deterministic permission path situated beneath the proposal boundary. The system's safety claims then rest on the permission path and its associated fallback strategies. These controllers rely on validated state estimation, conservative plant and actuator dynamics models, worst-case execution timing, and guaranteed stopping margins, all verified independently of the neural network proposer.
2. **Physical Fault Injection**: Rather than passively waiting for rare environmental disturbances or hardware faults to manifest during operational testing, systems engineers deliberately inject them on instrumented test benches. Because fault injection artificially alters the operational failure rate, these trials must be recorded in a separate, explicitly labeled log rather than pooled with nominal operational draws.

↪ The derivation of a fault list, by inverting design records and working forward from physical hazards, is developed in section 15.6.

Checkpoint 7.3: Sample size and MTBF evidence target

Before moving on, review sample size scaling and physical test limits:

- ☐ **Exposure-wall estimate**: Under a stationary Poisson failure model, estimate the failure-free hours a relaxed 1,000 h MTBF target needs at 95 percent confidence, and determine whether one rig running continuously for a year could supply them.
- ☐ **Mechanical fatigue exhaustion**: Explain why a campaign's wheel revolutions cannot be compared with a bearing L_{10} rating without knowing the load the bearing carries.
- ☐ **Architectural containment vs. statistical sampling**: Explain why a task campaign needs separate evidence for runtime constraints and physical safeguards.

Even under automated test cells and rigorous qualification protocols, statistical sampling of learned policies cannot demonstrate ultra-reliable autonomy. The integrity of an operational release decision depends on identifying precisely what remains unvisited by a finite test campaign and deploying verified runtime monitors to supervise those uncharacterized boundaries.

7.9.3 Turning coverage gaps into runtime obligations

Those uncharacterized boundaries come from every stage of the pipeline: offline datasets omit recovery states, physics simulators omit real-world contact and stator phase lag, and finite hardware campaigns sample only a handful of cells in the condition matrix. Because the physical states a machine traverses depend on the specific sequence of actuator commands admitted by the permission path (section 7.3), small discrepancies between modeled and real-world dynamics compound over

closed-loop horizons. They drive the platform into state regimes that never appeared during training or simulation. Neither a static dataset nor a simulator can match the physical plant's closed-loop state distribution, and an empirical hardware test bounds nothing beyond the specific operating conditions it directly sampled.

The operational conclusion of the door campaign is correspondingly narrow. The physical trial records establish that a single software build deployed on a specific hardware platform (Unit 01) bounded its per-cycle failure probability below 13.9 percent at 95 percent confidence. That bound was evaluated from the sampled initial poses under the specific ambient illumination and temperature present during testing. This record provides no assurance about system behavior on a cold morning that thickens gearbox lubricant or derates actuator drive current. It provides none on an aging transmission whose compliance and backlash have increased through gear tooth wear, or against a door whose strike plate has shifted several millimeters out of true. Testing across multiple physical units broadens the declared operating population, whereas executing additional runs solely on Unit 01 with a nominal strike plate merely tightens the binomial confidence interval (table 7.3). Logging two thousand additional nominal cycles measures that single hardware unit with high statistical precision while leaving displaced strike plates, payload variations, unit-to-unit manufacturing variations, and real-time bus jitter unsupported by empirical evidence.

Because finite evaluation cannot characterize behavior outside the sampled operational distribution, every unverified boundary of an empirical safety claim must be translated into an explicit runtime monitoring obligation. Each boundary requires an identifiable sensor signal, a calibrated detection threshold, a validated mitigation response, and deterministic enforcement by the permission path. Because operational distribution support is only partially observable and software out-of-distribution (OOD) detectors have nonzero false-negative rates, environmental conditions that cannot be sensed with high confidence require hard ODD constraints, passive mechanical guards, or independent physical barriers.

The evaluation record captures these obligations in its `runtime_monitor_specs` field (section 7.8), across five mandatory monitoring channels whose detection coverage, latency, and mitigation responses must be independently validated:

1. **Input Support and Out-of-Distribution (OOD) Monitor:** Quantifies whether incoming sensor observations remain within the support of the training and verification distributions using a calibrated score, with measured false-negative rates on held-out OOD test sets.
2. **Execution Latency and Freshness Monitor:** Tracks sensor observation age and neural inference execution time against separate hard deadlines. It verifies proposal renewal against nominal controller cadence (for example, verifying that the latch policy meets its 40 ms p99 inference target inside its 50 ms chunk period at 20 Hz), logs deadline misses, and ensures that an expired chunk lease immediately engages a validated fallback. A p99 latency is an empirical percentile; it does not replace a hard real-time execution bound.
3. **Constraint Margin Monitor:** Continuously tracks the distance between the current physical state and hardware protection boundaries, monitoring joint torque limits, motor winding Joule heating, and physical obstacle clearances against calibrated stopping-distance models.
4. **Intervention Rate Monitor:** Computes the running frequency of permission-path interventions and trajectory overrides as a warning of growing supervisory burden or behavioral drift. Preventive detection of a specified instability requires a declared window and threshold, with qualified detection-plus-response lead time. This warning does not replace instantaneous clearance and constraint checks or a feasible fallback.
5. **Environmental Assumption Monitor:** Verifies whether current physical conditions comply with operating design assumptions, tracking surface traction through wheel slip ratios and ambient illumination via calibrated photometric sensors.

A runtime monitor that merely records an anomaly to a diagnostic log provides zero physical protection. Under active control, if a newly generated policy proposal fails validation, the permission path rejects the replacement command without revoking the currently executing chunk, whose chunk lease is still valid. If the active command expires, encounters a safety violation, or lacks sufficient clearance, the permission path overrides the proposer and commands a validated fallback, such as a conservative speed restriction or a controlled deceleration to rest. The fallback is valid

only if current state estimates, actuator torque headroom, and braking distances render it physically feasible. Even when a neural policy produces timely, high-confidence proposals, every candidate control action $\mathbf{u}_{\text{nom}_t}$ must pass independent geometric clearance and actuator dynamic limits before the low-level hardware bridge applies torque to the plant.

Certain catastrophic failure modes cannot be detected by any software runtime monitor. A software out-of-distribution monitor operates exclusively on digitized sensor telemetry and latent neural representations. When a physical transduction failure occurs, such as a clean glass wall, a blinding specular laser reflection, or a camera occluded by uniform mud, the sensory telemetry itself is corrupted or absent. The perception pipeline may extract seemingly normal visual features from the reflection, reporting benign obstacle clearance while the physical chassis is on a collision course with the glass. Software cannot detect physical reality that hardware transduction failed to capture. In such failure modes, the proposal can satisfy every execution deadline, respect all motor torque envelopes, and produce familiar latent embedding vectors, yet drive the platform directly into an invisible obstacle. Physical safety against transduction failures cannot come from statistical evaluation or software-based OOD monitors. It requires architectural physical zoning, hardware current limiters, sensitive tactile bumpers, and physical safety barriers validated under explicit mechanical assumptions independently of the learned policy.

↪ The barrier condition that keeps braking feasible within the remaining clearance is developed in section B.4.

These verification ceilings bind every evaluation campaign, no matter how rigorously executed. The empirical evidence preserved in an evaluation record is only as trustworthy as the measurement protocols that produced it. Procedural flaws, silent sensor drift, and statistical fallacies can corrupt an evaluation campaign long before its summary metrics reach the evaluation record.

↪ Observational indistinguishability, in which a hazardous state yields the same telemetry as a safe one until the response deadline, is developed in section 18.3.4.1.

7.10 Evaluation Pitfalls and Metric Masking

A physical qualification report can present a spotless pass rate and an exact confidence bound while still misrepresenting what the machine can safely do, if the test procedure quietly narrowed the operating domain, tuned weights on the test unit, or discarded inconvenient runs before the bound was computed. Systematic errors undermining policy evaluation rarely stem from deliberate fraud; they arise when an evaluation protocol answers an easier question than the claim asks, creating statistical or procedural mirages that mask physical vulnerability.

7.10.1 Six evaluation pitfalls in practice

An audit of an evaluation record checks six links between the headline metric and the physical trials that produced it, because breaking any one of them leaves the reported pass rate intact while invalidating the claim.

The first link breaks through environment selection, whenever the test bench is narrower than the claim. The twenty door runs all met the nominal strike plate at the 0.03 m/s guarded approach with the base stopped and the tote rack empty. If the report states their success rate as the machine's ability to open the site's cage doors, the claim silently expands to the five unsampled cells of the scenario table and the loaded machine (section 7.9). Environment selection is detected by comparing a condition log, which records the measured strike-plate position, approach speed, payload, and floor condition of every trial, against the boundaries of the claimed domain. A log that covers one cell establishes nothing about the others.

Even when the workspace matches the claim, metric drift can substitute an offline surrogate for physical closed-loop task completion, scoring candidates by action agreement, validation loss, action Mean Squared Error (MSE), or action cosine similarity against recorded expert buffers. Offline scoring is cheap, so the substitution happens gradually, and it inherits the ranking inversions of section 7.3. Metric drift is prevented by keeping the primary evaluation metrics closed-loop physical observables, namely task-completion rate, ACR, MTBI, and D_{int}. Protection violations, including unmitigated tripwire crossings, remain a separate recorded outcome and an explicit acceptance condition; they never justify dropping runs.

Keeping the metric physical still fails if repeated tuning against a fixed evaluation setup introduces unit-specific test set contamination. Adjusting the checkpoint or observation filtering after each batch on Unit 01 makes that robot and door part of development, not an independent holdout. The resulting improvement may depend on their particular backlash, calibration, and lighting. The countermeasure is to audit the history of policy versions, tuning decisions, and chassis identities,

and to reserve fresh units and conditions for any claim that requires generalization beyond the development setup.

During execution on a clean holdout unit, optional stopping and multi-look inflation can still corrupt the stopping rule: an engineer who watches a sequence of physical runs and halts as soon as the tally crosses a target reports a confidence the procedure does not have. Each look at the accumulating result is another chance for a false positive. Five independent 5 percent looks would have a 22.6 percent chance of at least one false positive. Repeated looks at one accumulating run are dependent, so the inflation for a real stopping rule must be computed for that rule. Optional stopping cannot be detected from summary tables or final pass rates. It is detected only by auditing the pre-registered trial protocol, verifying that sample sizes, stopping conditions, and evaluation milestones were fixed before the first physical run began. The calibrated alternative is a sequential rule declared in advance (section 7.7.3).

After the runs finish, a fifth distortion occurs when the denominator is quietly filtered, breaking the full-denominator condition of section 7.5. A team that discards runs ending on timeouts, emergency stops, sensor communication drops, or permission-path overrides as invalid test artifacts reports success over the surviving trials only. Every metric of section 7.2 then moves in the flattering direction: ACR rises, MTBI grows, intervention density falls toward zero, and the reset time spent clearing a jammed gripper disappears from the budget. If the arm opens the door on nine of ten attempts and the tenth needs an operator takeover to stop the gripper wedging against the latch, removing that attempt raises the apparent success rate from 90 percent to 100 percent. The batch then reports an infinite MTBI. Infrastructure aborts, such as dropped wireless packets, stalled kernel threads, or bus brownouts, also stay in the denominator. A base that stalls in the aisle because an operating system thread locked has failed its mission, whichever subsystem caused it, and removing such aborts evaluates an abstraction rather than the deployed machine. When physical constraints or safety limits truncate a run before its deadline T_{deadline} expires, the trial stays in the trial log as not unassisted ($A_i = 0, n_{\text{int}_i} \geq 1$), and for the unassisted-failure event it is right-censored rather than omitted. Reading survival on truncated trajectories as operational competence is the censored trial fallacy of section 7.4. Denominator filtering is detected by cross-referencing low-level actuator telemetry and permission-path logs against the trial log, confirming that every power-on cycle, autonomy engagement, and physical reset corresponds to a recorded outcome.

War Story 7.1: The 2017 NHTSA Autopilot crash rate mirage

Context: In January 2017, the National Highway Traffic Safety Administration (NHTSA) closed Preliminary Evaluation PE 16-007, which it had opened after a fatal Florida crash of a Model S operating in Autopilot mode (National Highway Traffic Safety Administration (NHTSA) 2017). The closing report compared airbag-deployment crash rates in the subject Tesla vehicles before and after Autosteer was installed and stated that "the Tesla vehicles crash rate dropped by almost 40 percent after Autosteer installation," from 1.3 to 0.8 crashes per million miles.

Mechanism: The rate divided airbag deployments by miles driven before and after installation, and the report's own footnote noted that the rates "are for all miles travelled before and after Autopilot installation and are not limited to actual Autopilot use." The denominator therefore never measured exposure to Autosteer itself, and it was incomplete as well. A replication by Quality Control Systems Corporation found that the odometer reading at installation appears to have been reported for fewer than half of the vehicles, and that exact exposure before and after installation could be computed for only 13 percent of them (Quality Control Systems Corp. 2019). The mileage NHTSA left uncounted, because it could be assigned to neither period, was concentrated among the vehicles with the least exposure before installation.

Impact: In the one cohort with complete exposure in both periods, the airbag-deployment crash rate rose 59 percent after installation, from 0.76 to 1.21 per million miles, a result the replication qualified with "if these data are to be believed." Its authors concluded that the overall 40 percent reduction was "an artifact of the Agency's treatment of mileage information that is actually missing in the underlying dataset." The replication does not measure Autosteer's effect either. It shows that the published denominator could not support the reported reduction.

Response: The correction came from outside the agency and two years after the closing report, because the per-vehicle mileage records that exposed the denominator were released only after a Freedom of Information Act lawsuit against the Department of Transportation (Quality Control Systems Corp. 2019).

Systems lesson: In physical AI evaluation, an unverified denominator produces a statistical mirage. Aggregating open-world mileage without engagement telemetry, operational design domain boundaries, and complete exposure records turns missing data into a safety claim. An evaluation audits the integrity of its denominator with the same rigor it applies to the failure events in its numerator.

At the extreme of denominator filtering, teams sometimes substitute visual demonstrations for statistical measurements altogether. A successful video shows that the machine reached the goal under one sequence of conditions; without a sampling record, it does not establish how often that outcome recurs. Smooth $SE(3)$ kinematics (rigid-body motions in three dimensions) can conceal small physical margins. A representative campaign needs predeclared sampling from the target ODD or separate per-stratum analysis, full-denominator accounting, and a sample size tied to its stated claim.

None of these pathologies is visible in the final pass rate. Each is caught only by auditing the record against the protocol that was declared before the first run, which is why the evaluation record keeps the protocol, the condition log, and the complete outcome sequence rather than the summary alone.

7.11 Fallacies and Pitfalls

An empirical evaluation supports engineering claims only within the precise physical boundaries and statistical assumptions of its test protocol. In embodied machine learning systems, severe evaluation errors arise when teams extrapolate localized results across untested operational envelopes or attribute system-level safety guarantees to a learned policy when deterministic enforcement mechanisms actually did the work.

Fallacy: *Repeated success in a single nominal condition proves broad operational readiness.*

Consider a door-opening evaluation campaign that repeats a single nominal configuration twenty times: a standard strike plate, an unencumbered approach vector, a stationary mobile base, and an empty payload rack. Under the binomial model, twenty flawless runs yield the 86.1 percent lower confidence bound derived in notebook 7.1 for that specific setup. However, they provide no statistical evidence for the remaining operating conditions, such as misaligned latch hardware, floor irregularities, or a loaded tote rack that shifts the manipulator's center of mass and reduces actuator torque margins. Repeating identical trials merely sharpens precision on a single nominal operating point while leaving deployment-critical boundary conditions unprobed. An engineering claim of operational readiness must explicitly define the target ODD, distribute test budgets across the physical parameters governing that domain, and bound uncertainty within each stratified cell. **Nominal repetition sharpens precision on a single cell; it provides zero evidence for the rest of the operational design domain.**

Architect's countermeasure: Explicitly partition the declared operational design domain into a structured condition matrix, allocate trial quotas across all operational strata, and report confidence intervals per cell rather than pooling unstratified runs.

Fallacy: *Zero failures in 1,000 trials proves compliance with a 99.9 percent reliability specification.*

Suppose a latch manipulation policy opens a door 1,000 consecutive times without a single safety trip, evaluated against a specification requiring 99.9 percent success at 95 percent confidence. The observed sample success proportion is perfect ($\hat{p} = 1.00$). Standard Wald intervals based on the asymptotic Central Limit Theorem collapse into the degenerate point $[1.0, 1.0]$ because the estimated variance $\hat{p}(1-\hat{p})/n$ vanishes identically at the boundary. However, an exact non-asymptotic Clopper-Pearson inversion ($p_{\text{lower}} = 0.05^{1/n}$) demonstrates that the true one-sided lower confidence bound reaches only 0.9970, falling decisively short of the 99.9 percent target (table 7.3). Certifying compliance with that target requires 2,995 consecutive clean runs. A parallel fallacy recurs in early-stage pilot

testing. Observing zero failures across 20 trials ($p_{\text{lower}} = 86.1$ percent) cannot demonstrate even a modest 90 percent success bound (29 clean runs required), much less an industrial 99 percent standard (299 clean runs required). Test budgets must be derived directly from the required confidence level and reliability target before execution begins, and validation reports must document exact non-asymptotic bounds rather than empirical success ratios. **Zero failures across an insufficient sample size represent an evidence deficit, not empirical proof of compliance.**

Architect's countermeasure: Reject asymptotic Wald intervals near parameter boundaries. Precalculate the minimum sample budget using the exact non-asymptotic Clopper-Pearson formulation ($n = \lceil \ln \alpha_{\text{sig}} / \ln p_{\text{target}} \rceil$) or its continuous-time counterpart for exposure hours ($T \geq 3.0/\lambda$), and report exact one-sided lower confidence bounds in all compliance audits.

Fallacy: *High offline action agreement on static logs guarantees closed-loop physical stability.*

An evaluation report citing close agreement with expert actions on recorded teleoperation logs, whether framed as negligible action mean squared error (MSE), high discrete token prediction accuracy, low validation perplexity, or high action cosine similarity, measures dataset imitation on static buffers and nothing more. As demonstrated by the mobile aisle policy in section 7.3, a policy can match its offline demonstrations with near-zero error and still exhaust its lateral clearance through the kinematic integration of a minute, persistent heading bias. Compounding this error, covariate shift drives the system into unmodeled state spaces once trajectory drift departs from the demonstration support.

Physical contact introduces even sharper failure modes. At non-Lipschitz mechanical interfaces, sub-millimeter positional deviations that barely register in aggregate MSE trigger destructive reaction forces ($F = k_{\text{contact}} \Delta x$) and mechanical wedging. This produces dramatic ranking inversions where policies with lower offline MSE fail completely while policies with higher MSE succeed. Directional metrics such as cosine similarity fail for complementary reasons: they ignore command magnitude entirely. A policy commanding the correct directional vector with 25 percent too little force stalls below a latch's breakout threshold, whereas commanding 150 percent of the required force can trip protective torque boundaries. Offline metrics serve well as initial screening filters to discard defective training checkpoints. Empirical closed-loop capability, however, must be evaluated using physical observables, namely task-completion rate, ACR, MTBI, and D_{int}, tracked against the complete execution denominator, with protection outcomes recorded separately. Even then, these finite-sample empirical metrics do not constitute a formal proof of closed-loop stability. **Offline validation loss, MSE, and cosine similarity measure dataset mimicry; closed-loop physical dynamics decide survival.**

Architect's countermeasure: Treat offline loss, action MSE, and cosine similarity strictly as preliminary triage filters. Require closed-loop physical rollouts with full-denominator accounting to support behavioral claims, and require separate model-based Lyapunov or invariant set analyses for any formal stability guarantee.

Pitfall: *Peeking at accumulating rollouts and terminating testing early inflates false safety certification.*

When an engineering team halts an evaluation campaign after 30 consecutive clean runs because intermediate results look favorable, and then reports the confidence bounds of a fixed-sample protocol, they commit sequential peeking. Halting on a winning streak distorts the underlying statistical distribution. An adaptive stopping rule inflates the false-positive rate, rendering standard fixed-sample confidence calculations invalid. While a predeclared zero-failure test can terminate upon the first observed failure to reject an inadequate policy, terminating early to accept a candidate requires an explicit sequential design. Sequential probability ratio testing provides a mathematically sound alternative. Ville's martingale inequality bounds the probability of false acceptance by $e^{-\Lambda_{\text{up}}}$ for any declared acceptance threshold Λ_{up}, which resolves to $\alpha_{\text{sig}}/(1-\beta)$ under classic Wald boundaries (section 7.7.3). **Choose the stopping rule before collecting outcomes, and report the error guarantee for that rule.**

Architect's countermeasure: Pre-register sequential decision boundaries ($\Lambda_{\text{up}} \approx \ln((1-\beta)/\alpha_{\text{sig}})$, $\Lambda_{\text{lo}} \approx \ln(\beta/(1-\alpha_{\text{sig}}))$) before running trials. Update the cumulative log-likelihood ratio Λ_m after each physical rollout, and award the declared SPRT acceptance or rejection decision only when the statistic crosses its corresponding threshold. A physical hazard or invalidating protocol evidence

can terminate the campaign independently. Keep the complete trial log and the termination reason, and claim no favorable result that the declared statistical rule does not support.

Pitfall: *Crediting the learned policy with surviving transport stalls that the enforcement architecture handled.*

Suppose a mobile manipulator completes 50 nominal aisle navigation trials, leading the engineering team to credit the learned transformer policy with inherent robustness against serializer/deserializer (SerDes) bus jitter. Later, a qualification trial injects a synthetic mailbox stall that exceeds the 60 ms chunk lease. The lease expires, and the deterministic permission path detects the deadline miss and commands an immediate controlled dynamic stop. This stop is analogous to an industrial Category 1 stop (a controlled deceleration under drive power, with torque removed once the machine stands still), as classified in section 12.9.2. This safe recovery is executed entirely by the enforcement architecture, not the learned model. Nominal task completion does not prove that the neural network weights can detect, absorb, or recover from communication dropouts. Rigorous fault injection must evaluate the permission path's detection latency and stopping distance margins under stress, and the final evaluation report must attribute safety enforcement directly to the hardware and runtime mechanisms that actually executed the stop. **Do not credit unverified neural weights with safety guarantees provided by the deterministic permission path.**

Architect's countermeasure: Conduct systematic fault injection, including synthetic communication stalls and sensor frame drops, to independently benchmark the permission path's braking envelope, and attribute safety mechanisms strictly to the hardware and software layers that executed them.

Fallacy: *Zero physical collisions or hardware damage in testing proves high autonomous policy reliability.*

When an autonomous vehicle or mobile manipulator completes an extended test campaign without collisions, teams frequently declare that the policy demonstrated exceptional reliability (for example, claiming an MTBF equal to the total logged run time). However, if human safety operators disengaged the autonomy system or the permission path intervened repeatedly to avert impending collisions, the zero-damage outcome ($N_{\text{damage}} = 0$) occurred only because the protective safety envelope actively censored catastrophic trajectories. What the testbed truly recorded was a sequence of intervention events. True autonomous capability must be evaluated through ACR and MTBI. Frequent human takeovers render commercial deployments economically unviable, whereas small automatic clips by the permission path may represent routine boundary regulation. An evaluation report must classify and audit these intervention tiers separately. Censoring interventions or tallying assisted rollouts as unassisted policy successes confuses the protective competence of the supervisory envelope with the autonomous competence of the learned policy. **Zero collisions under active safety supervision must be interpreted alongside interventions; they do not by themselves establish policy reliability or complete protection coverage.**

Architect's countermeasure: Enforce strict full-denominator accounting. Record every human takeover, permission-path clamping action, and virtual tripwire breach as an autonomy disengagement ($A_i = 0, n_{\text{int}_i} \geq 1$). Report these unassisted autonomy metrics alongside overall task completion rates, mission cycle times, and protective safety statistics.

Pitfall: *Qualifying a policy on a single reference robot assumes zero unit-to-unit hardware variance across the fleet.*

Consider an engineering team that logs 300 failure-free trials on a single laboratory robot (Unit 01) and certifies the policy for fleet deployment. When deployed onto Unit 02 on the manufacturing floor, the identical policy repeatedly jams against fixtures or develops severe actuator chatter. As detailed in section 7.2, physical unit-to-unit variations (transmission lost motion, harmonic drive compliance and stiffness, structural link flexure under load, camera mounting misalignments, and motor winding tolerances) substantially alter the plant transfer function across robot builds. Qualifying a policy on a single "golden unit" merely calibrates the model to that individual chassis's electromechanical profile rather than establishing fleet-wide reliability. Furthermore, running evaluations during a single shift overlooks non-stationary environmental factors: diurnal sunlight glare sweeping across vision sensors, floor traction changes driven by ambient humidity, or thermal drift from motor winding resistance heating (I^2R) that alter plant dynamics without altering a single neural network parameter. **Evaluating a single robot chassis characterizes that specific unit's mechanical quirks; fleet release requires stratifying trials across the manufacturing distribution.**

Architect's countermeasure: Stratify physical qualification protocols across multiple chassis serial numbers sampled from the fleet manufacturing distribution $\mathcal{M}$. Audit unit-specific lost motion and kinematic link offsets prior to evaluation, and reserve final acceptance testing for untouched production units that were never used during policy development or hyperparameter tuning.

7.12 Summary

Closed-loop evaluation characterizes the fully coupled execution system: the proposer policy, the permission path, the physical plant, the surrounding environment, and the experimental protocol. Consequently, an empirical success rate never describes a static weight checkpoint in isolation; it characterizes a declared population of operational trajectories. Because the machine's admitted actions dictate the future states it encounters, which is the second law, endogenous data, applied to testing, only closed-loop execution through the active permission path reveals how the proposer truly behaves. Open-loop evaluations on pre-recorded datasets remain blind to the compounding errors that real-world interaction induces.

A zero-failure confidence bound quantifies performance over this sampled population under explicit sampling assumptions. When safety tripwires intervene to prevent physical damage, they censor trials right at the operational safety boundary, keeping empirical testing confined to the envelope the experimental campaign can safely explore. Gathering additional trials narrows statistical uncertainty within that sampled distribution, but it cannot expand the boundaries of the characterized population. The exposure wall adds a further ceiling, because validating ultra-rare hazard rates requires far more continuous operating hours than any single physical platform can feasibly run.

The robotic door-opening campaign illustrates the precise scope of an empirical engineering claim. Twenty incident-free trials establish a statistical lower bound on success at the nominal strike plate and the guarded approach speed, with the base stopped and the tote rack empty. That evidence justifies no claims about displaced strike plates omitted from the demonstration dataset, nor does it guarantee performance when carrying heavy payloads. To handle these uncharacterized regimes safely, the engineering team maintains a coverage gap catalog that explicitly documents every unvisited condition. Rather than trusting an unverified proposer policy or relying on an out-of-distribution detector that can fail when encountering novel inputs, the system delegates safety in these unvisited regions to independent physical constraints and verified fallback behaviors in the permission path.

The stipulated recovery branch demonstrates how engineers systematically expand this operational boundary over time. When developers gather targeted experience to train an updated candidate policy, they must validate it through independent regression testing on nominal regimes alongside rigorous qualification on displaced plates. An initial evaluation archive never represents a permanent barrier to capability; rather, it establishes an empirical baseline from which verified competence can methodically advance.

The ultimate output of a closed-loop testing campaign is the evaluation record. Four core artifacts from this record govern downstream deployment and system safety:

1. *Bounded operational claim:* The target ODD, established within the broader declared ODD of the policy manifest, specifying the precise environmental and kinematic envelope where closed-loop performance has been empirically characterized.
2. *Reproducible configuration and trial log:* Cryptographic digests of neural network weights, controller firmware, and sensor calibrations, paired with measured hardware tolerances and an unedited trial log. Retaining every attempted, aborted, and overridden trial in the denominator is essential to compute honest task-completion, autonomy, intervention, and reset metrics (ACR, MTBI, D_{int}, T_{reset}), with protection outcomes recorded separately.
3. *Coverage gap catalog:* A systematic audit detailing known data omissions from training demonstrations, operational regimes unvisited during testing, and specific hardware units that remain uncharacterized.
4. *Runtime monitor specifications:* Explicit signal definitions, trigger thresholds, and tested fallback maneuvers that the permission path must execute whenever an observable safety boundary is reached.

Table 7.7 summarizes the foundational mathematical relationships that support these operational decisions, linking each formal bound to its engineering consequence. Each row connects the governing expression to the specific test quantity a campaign must declare or report before deployment.

Table 7.7: Key equations of closed-loop evaluation: Each equation fixes one quantity a campaign must declare or report. The statistical bounds assume independent, identically distributed draws from the declared population; the kinematic and contact relations are first-order models whose inputs must be measured on the machine.

Equation	What it bounds or decides	Derived in
$p(\mathcal{D}) = \mathbb{E}_{c\sim\mathcal{D}}[\phi(\mathrm{Traj}(c))]$	The estimand: a success rate belongs to the declared population $\mathcal{D}$, not to the weights	equation 7.1
$t_{\mathrm{impact}} = W_{\mathrm{clear}}/(v \sin \Delta\theta_{\mathrm{head}})$	How soon a persistent heading bias spends clearance that open-loop scoring never sees	equation 7.2
$p_{\mathrm{lower}} = \alpha_{\mathrm{sig}}^{1/n}$	Success lower limit after n clean trials	equation 7.3
$p_{\mathrm{fail,\ upper}} \approx 3/n$	Rule of three: failure upper limit at 95% confidence, and $n \approx 3/(1-p_{\mathrm{target}})$	equation 7.4
$n = \lceil \ln \alpha_{\mathrm{sig}} / \ln p_{\mathrm{target}} \rceil$	Clean trials a claim needs; each added nine costs ten times the trials	equation 7.5
$\Pr(X \geq k \mid N, p) = \alpha_{\mathrm{sig}}$	Success lower limit when the record contains failures	equation 7.6
Λ_m against $\Lambda_{\mathrm{up}} \approx \ln((1-\beta)/\alpha_{\mathrm{sig}})$, $\Lambda_{\mathrm{lo}} \approx \ln(\beta/(1-\alpha_{\mathrm{sig}}))$	Sequential acceptance or rejection with both error rates declared before the first run	equation 7.7 and equation 7.8
$T \geq \ln(1/\alpha_{\mathrm{sig}})/\lambda$	Exposure wall: failure-free hours before a hazard rate can be bounded	equation 7.9
$v_{\mathrm{approach}} \leq F_{\mathrm{lim}}/\sqrt{m_{\mathrm{red}}\, k_{\mathrm{contact}}}$	Precontact speed that keeps the first impact peak under a force limit	equation 7.10

DRAFT

Key Takeaways: Closed-loop evaluation and reliability

- **A success rate belongs to a population**: Any empirical success metric reflects the coupled proposer policy, permission path, physical plant, environment, and trial protocol. Additional trials narrow uncertainty within sampled conditions, but evaluating new hardware units or altered operating regimes requires fresh empirical evidence.
- **Select evaluation regimes to match engineering questions**: Offline replay validates predictive accuracy and compute budgets, simulation tests closed-loop feedback under modeled dynamics, shadow deployment tests runtime execution against live sensor streams, and constrained physical trials expose real unmodeled contact. Each evaluation modality leaves distinct coverage gaps; their results cannot be pooled as interchangeable evidence.
- **Separate task completion from protective interventions**: Autonomous completion rate (ACR) quantifies unassisted task success, whereas mean time between interventions (MTBI) and intervention density characterize the operational burden of specific safety tripwires. Physical task success and protective safety outcomes must be tracked independently. In addition, reset latency (T_{reset}) captures the operational overhead of recovering after a safety disengagement. Recording every attempted run prevents safety supervision from artificially inflating policy performance.
- **Failure-free trials leave residual statistical uncertainty**: Twenty consecutive successful trials yield a one-sided 95 percent success lower bound of 86.1 percent under independent and identically distributed sampling. Reliability targets and statistical confidence levels dictate the required trial budget. Predeclared, calibrated sequential testing rules allow early stopping decisions without introducing post hoc selection bias.
- **Rare hazards demand independent physical safeguards**: Physical testing campaigns cannot feasibly supply the millions of failure-free operating hours required to bound

catastrophic risk. Runtime monitors, mechanical interlocks, and physical safety boundaries require independent validation of their sensing fidelity, actuation latency, and fallback dynamics. A high policy success rate can never substitute for verified protective mechanisms.

- **The evaluation record binds capability to operational limits**: The record documents what the proposer achieved within its target ODD, preserves an unedited trial log, and highlights unsupported operational conditions. Its monitor specifications guide runtime safety design, while unmonitorable coverage gaps impose firm operational restrictions on deployment claims.

The second law, endogenous data, closes Part II by leaving every evaluation record conditional on the closed loop that produced it. A sequence of incident-free trials samples only the trajectory distribution generated by admitted actions; it neither explores every physically reachable state nor certifies the absence of hazards outside the tested envelope. Systems engineers must carry both the characterized competence and the uncharacterized uncertainty forward into runtime system design. The evaluation record formalizes this transition through its target operational design domain, coverage gap catalog, and monitoring requirements. With the evaluation record, the first of the record chain's two strands is complete, pairing the plant limits of the limit record with the bounded competence recorded here. Part III builds the second strand, the live observations and proposals that meet the first at the permission decision on every control tick.

7.13 Historical Perspective and Further Reading

The statistical foundations of closed-loop evaluation were established long before autonomous systems encountered physical hardware. Standard textbook treatments of binomial confidence intervals rely on asymptotic normality, yet normal approximations fail near the extreme boundaries ($p \approx 0$ or $p \approx 1$) where robot qualification and safety certification operate. Wilson (1927) resolved the degeneracy of the textbook interval at zero failures by inverting the score test, ensuring that confidence intervals would not collapse to zero width on a spotless empirical record. Clopper and Pearson (1934) went further by directly inverting the cumulative binomial distribution to produce the exact, conservative confidence bounds that modern physical qualification audits require. In clinical trials evaluating rare adverse medical events, Hanley and Lippman-Hand (1983) popularized the "rule of three" as an accessible analytical shorthand for zero-failure campaigns. They established that when zero incidents occur across n independent trials, an upper 95 percent confidence bound on the true event probability is approximately $3/n$. Decades later, Brown et al. (2001) demonstrated the persistent systematic flaws and erratic coverage probability of standard Wald intervals. That result shows why rigorous evaluation campaigns for embodied agents must rely on exact binomial methods or inverted score statistics rather than asymptotic Gaussian assumptions.

The formal methodology for designing controlled evaluation campaigns emerged from agricultural experimentation and industrial manufacturing. Fisher (1935) formulated the core principles of randomized design, Latin squares, and blocking to prevent uncontrolled environmental factors, such as thermal drift, surface wear, and ambient lighting shifts, from being mistaken for genuine differences in algorithmic performance. Fisher's exact test for contingency tables (Fisher 1922) remains the standard baseline when comparing categorical success rates between discrete policy checkpoints under small sample sizes. During the Second World War, the Statistical Research Group at Columbia University introduced sequential hypothesis testing to inspect manufacturing lots efficiently without sacrificing statistical rigor. Wald (1945) published this Sequential Probability Ratio Test (SPRT) once military classification was lifted. The underlying mathematical machinery that guarantees valid stopping times and bounds Type I and Type II errors traces back to the martingale formulation of Ville (1939). In contemporary robot learning, these martingale arguments directly underwrite sequential testing protocols and anytime-valid confidence sequences, enabling practitioners to evaluate candidate policies adaptively and terminate underperforming runs early without invalidating target significance guarantees (Snyder et al. 2025).

The physical impossibility of validating safety-critical systems purely through black-box testing, the exposure wall, became evident during the development of fly-by-wire avionics. Butler and Finelli

(1993) demonstrated that verifying catastrophic failure rates on the order of 10^{-9} per flight hour requires operational test durations spanning hundreds of thousands of years. This result proves that ultra-high reliability cannot be certified through black-box testing alone and must instead rely on formal architecture, modular containment, and provable safety envelopes. Kalra and Paddock (2016b) applied this statistical reality to autonomous road vehicles, demonstrating that establishing statistical proof of safety parity with human drivers requires driving hundreds of millions or billions of physical miles across complex operational domains. Within machine learning, the gap between offline accuracy and online execution stems from the fundamental dynamics of closed loops. When Pomerleau (1989) demonstrated autonomous steering via artificial neural networks, small prediction errors caused the vehicle to drift into unfamiliar road configurations that were absent from the human demonstration dataset. Ross and Bagnell (2010) and Ross et al. (2011) formalized this breakdown by showing that distribution shift in sequential decision problems causes one-step prediction errors to compound quadratically over the planning horizon, producing a rigorous bound on the gap between open-loop validation and closed-loop rollouts. A defensible evaluation campaign brings these disciplines together: it delineates the operational design domain, accounts for human interventions as primary events within the statistical denominator, and confines the policy to verified envelopes when environmental conditions exceed tested bounds.

For further reading, six foundational works trace the trajectory from small-sample statistical theory to modern closed-loop robot evaluation:

- Brown et al. (2001) provides a definitive comparative analysis of binomial confidence intervals, exposing the systematic under-coverage of asymptotic approximations and motivating the exact bounds detailed in section 7.6.1.
- Wald (1945) formulates the Sequential Probability Ratio Test, establishing the mathematical boundaries and error guarantees applied to checkpoint screening in section 7.7.3.
- Butler and Finelli (1993) delivers a foundational proof that operational testing alone cannot certify ultra-high reliability in critical software systems, explaining why physical certification must lean on architectural safety design.
- Kalra and Paddock (2016b) quantifies the exposure wall for autonomous driving, providing accessible mathematical models and tables that show the hundreds of millions to billions of miles needed to measure catastrophic failure rates empirically.
- Ross et al. (2011) provides the theoretical analysis of compounding error under policy-induced distribution shift and introduces Dataset Aggregation (DAgger), formalizing why offline validation metrics fail to predict closed-loop performance.
- Kress-Gazit et al. (2024) establishes concrete reporting standards and metrology guidelines for physical robot evaluation, offering operational checklists covering trial counts, initial state distributions, blind evaluations, and failure mode accounting.

DRAFT

What's Next: From evaluation records to perception

Part II ends with a record that confines trust in the learned proposer to its target ODD and names the runtime monitors the rest of the machine must supply. Part III builds the runtime that meets those obligations. The first of them falls on perception, because an input-support monitor or a latency monitor can act only on observations that carry their capture time, their coordinate frame, and their uncertainty. Chapter 8 opens on a detection that is correct and already late, because the base keeps moving while the camera exposes, reads out, and infers, and it follows that measurement from the sensor to a spatial claim that the permission path can date, place, and bound before it permits motion.

DRAFT

III

RUNNING THE MACHINE

DRAFT

DRAFT

7

Running the Machine

Part II ended with an evaluation record that bounds what the learned proposer has shown it can do inside its target operational design domain and names the runtime monitors the machine must supply. Now, as the warehouse mobile manipulator drives down an aisle, its navigation camera catches a person stepping out from the rack end ahead. The chunk policy's next proposal still carries the base forward, and the permission path must decide whether the observation behind it is still fresh and whether the remaining clearance permits that motion or only a stop. The decision has a deadline, and no better proposal helps once the machine has crossed its braking margin. Running a physical AI machine means reconciling the timescales of sensing, planning, and actuation with that physical deadline. Human goals may change over seconds and trajectories over tens of milliseconds, while the permission loop, in one implementation, runs every millisecond. The required rates and response times come from the plant and its measured delays, not from the names of the levels.

Principle 3: Proposal is not permission

Invariant: A learned model may propose motion; only an independent permission path may admit it. The path decides each action on state whose age is known, within a deadline set by observation age, computation, transport, actuator response, and the clearance the delay leaves. It owns a fallback that is feasible from the current state without the proposer.

Implication: Every runtime stage serves one decision, whether this proposal can cross the causal boundary before the margin runs out. Stages ahead of the permission path deliver evidence with its age and uncertainty attached. The permission path runs on resources that learned workloads cannot delay. Any guarantee the path offers, such as a barrier filter's (section 12.5), holds only while its model, state estimate, actuator authority, and deadline hold.

Part III develops the runtime that uses the proposer's evaluated capability without assuming it is correct on every occasion, and that supplies the runtime monitors the evaluation record names. It builds the record chain's second strand, the live observations and proposals that meet the first at the permission decision. Perception qualifies each measurement, stating when it was captured, where it was measured from, and how far it can be trusted. Memory develops the state estimator that fuses those measurements into the current scene and decides how long that estimate may stand. Intent bounds the requested task, and planning constructs candidate motion. Enforcement admits feasible motion, makes a least-change correction when possible, or initiates a qualified response when continued motion cannot be supported. Placement supplies the resources needed to make those decisions on time. These are dependencies in the engineering argument, not a claim that the running machine executes every stage serially. The base's run to the rack end and the arm's reach for the mug share one whole-body proposal stream and its time-limited permission. By tracing that stream, the reader learns which stage owns each physical constraint and how a failure upstream changes the action permitted downstream.

- **Sensor Perception (chapter 8):** Decides whether a measurement is current and calibrated enough to use, and produces the observation contract, which dates each measurement in a clock the permission path can convert to its own. It stops at the qualified measurement.
- **Spatial Memory (chapter 9):** Teaches state estimation, fusing dated measurements into a tracked state whose covariance is tested against its actual error. It then decides how long that

estimate may stand without fresh evidence and produces the belief record, which keeps the age of the evidence separate from the age of the estimate.

- **Grounded Intent (chapter 10):** Decides how a human goal becomes a bounded, expiring request, and produces the intent lease, which grants a target and effort ceilings but never a path.
- **Trajectory Planning (chapter 11):** Decides when a trajectory may commit, and produces the trajectory record, which carries its deadlines and a matched stop suffix.
- **Safety Enforcement (chapter 12):** Decides on every tick whether a proposal is admitted, projected, or refused. The first projection on the machine slows the arm's fast latch approach to the guarded speed that Part II's target ODD covers. The chapter produces the enforcement record, where the two strands of records meet.
- **Workload Placement (chapter 13):** Decides where the permission path runs so that learned workloads cannot delay it. It treats the two-processor split that section 4.2 states as a hypothesis to put to a loaded test, and produces the placement record, which names the test that would support it.

Across this six-stage chain, the first four records remain on the application processor, only the enforcement record sits across the proposal boundary on the safety microcontroller where the proposal is evaluated against its physical lease, and the placement record tests whether the hardware provides the isolation that boundary assumes (figure 7.7).

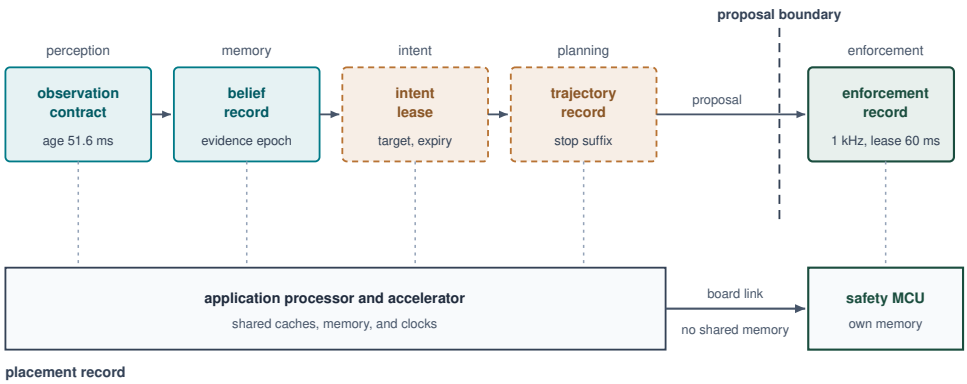
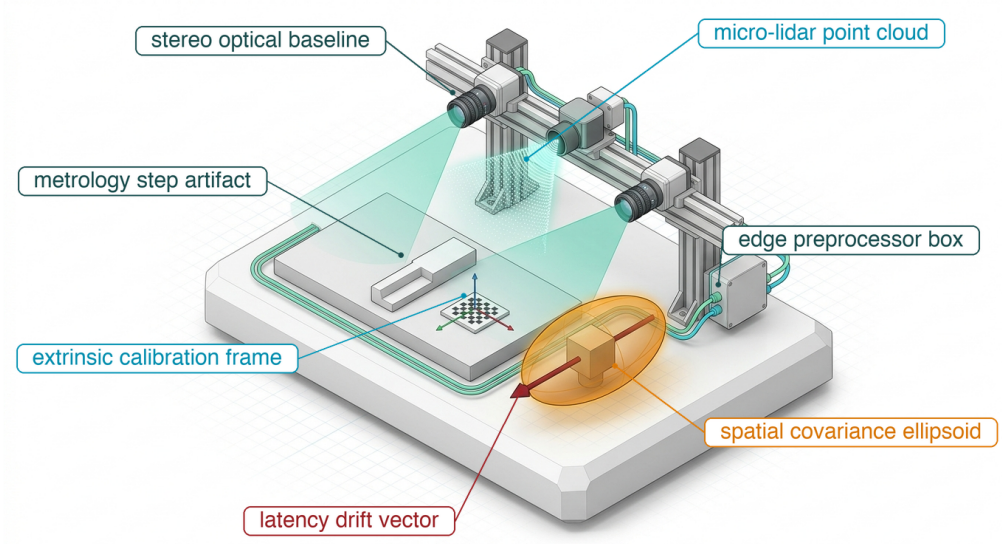


Figure 7.7: The Part III record chain and its placement: Each stage hands the next a record with its age or expiry attached, and only the enforcement record sits across the proposal boundary. The placement record assigns the four proposal-side stages to the application processor, where they share caches, memory, and clocks with learned workloads, and the permission path to a separate safety MCU reached over a board-level link. That split is a hypothesis until the loaded test of hardware separation supports it.

Part III hands forward the runtime evidence that Part IV must judge. Observation contracts date every measurement, belief records expire, intent leases and trajectory records name their parents and their deadlines, and the enforcement record states the stopping envelope and the premises of each guarantee. Its worked stopping budget selects a candidate aisle speed under declared timing and physical assumptions (section 12.3). Remaining clearance is conditional on those assumptions, including the still-unqualified dispatch-to-admission interval. A dispatch timestamp does not stop evidence from aging, and the chunk lease (section 4.4) begins at admission. The placement record names the loaded test that would show learned workloads cannot delay the permission path (section 13.7), and on this machine its loaded field is still empty. Each record is a claim with conditions, and none is yet a license to operate. Part IV must assess both useful service and containment, preserving the distinction between a permitted correction and failure to complete the task.

8

Sensor Perception

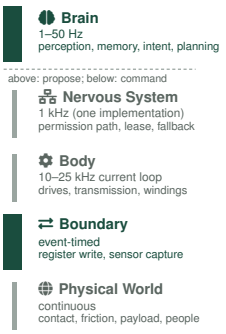


8.1 Choosing Evidence for a Spatial Decision
8.2 Capture Physics and Calibration
8.3 Dating and Placing the Measurement
8.4 Bounding Observation Age
8.5 Ingestion Overhead and Memory Bus Contention
8.6 Spatial Error and Clearance Bounds
8.7 Perception Failure Modes and Sensor Degradation
8.8 The Observation Contract
8.9 Fallacies and Pitfalls
8.10 Summary
8.11 Historical Perspective and Further Reading

Purpose

Why can an accurate detection still be too late to guide a robot?

A camera can recognize an obstacle correctly and still report where it stood before the robot moved. Exposure, readout, transport, and inference all take time, the machine keeps moving through every millisecond of it, and a shifted mount or a misread clock then places the obstacle somewhere it never stood. None of these errors lowers a detection score. The worst of them, a frame stamped with the wrong time, narrows the estimate's uncertainty around the wrong position, so the machine grows more confident as it grows more wrong. A machine that treats a recognized object as a current position spends its stopping clearance on errors nobody measured. Perception serves a moving machine only when each observation states when it was captured, where it was measured from, and how far it can be trusted, because at the causal boundary a stale or misplaced observation becomes motion that cannot be taken back, and the permission path can refuse only evidence whose age and limits it can check.



Learning Objectives

- Select sensing modalities by the physical quantity, coverage, and failure limits required for a spatial decision
- Diagnose exposure, rolling-shutter, and calibration errors before measurements enter a geometric estimate
- Calculate capture-to-dispatch age and account for continued aging through downstream consumption without double charging delay
- Evaluate whether stopping margins fit the available clearance once the clearance inset and clock bound rest on measured error, not reported covariance
- Distinguish qualified metric evidence from learned depth estimates and semantic confidence
- Diagnose silent sensor failures, including a misdated frame, that pass transport checks while misrepresenting the scene
- Design an observation contract whose timing, calibration, error qualification, and health checks support downstream use

8.1 Choosing Evidence for a Spatial Decision

A vision model on the warehouse mobile manipulator can detect a worker stepping past a rack upright with near-perfect benchmark accuracy and still hand the trajectory planner an obsolete position. The world keeps moving while the navigation camera integrates charge, serializes pixel rows across the bus, and runs inference through the vision pipeline. On this machine, 51.6 ms elapses between the midpoint of optical exposure and the dispatch of the processed observation (derived in the latency waterfall of section 8.4.1). Driving at its 1.5 m/s drive limit, the base covers 77.4 mm over that single interval. Benchmark accuracy reveals nothing about whether an observation is still fresh enough to support safe clearance. On a moving machine, every observation carries an expiration date.

Every motion a physical AI system executes rests on a claim about the world derived from earlier sensor readings. Perception converts raw energy incident on a detector (photons on a CMOS sensor array, laser pulses reflected to an avalanche photodiode, or elastomer deformation at a fingertip) into spatial claims: the volume a coworker occupies, the metric range to a rack upright, or the incipient slip of a grasped object. Machine perception has historically been scored by static recognition accuracy on fixed benchmarks, which rewards a correct label regardless of when the exposure occurred or how the camera was moving. For an embodied machine that abstraction collapses. A spatial claim can support motion only if it states three properties: the instant its energy was captured, the coordinate frame in which it was measured, and a bound on its spatial uncertainty. Set against the machine's current velocity, that qualified uncertainty must fit inside the clearance that remains.

The evaluation record that closed Part II (section 7.8) hands the runtime a list of obligations rather than a verdict of trust. It limits the learned proposer to its target operational design domain (ODD) and leaves the rest of the machine to supply the runtime monitors that hold it there (section 7.9). Two of those monitors, the input-support monitor and the latency and freshness monitor, can act only on observations that carry their capture time, frame, and uncertainty, so their first obligation falls on perception. Perception delivers this runtime evidence as dated spatial claims across the proposal boundary. It executes within the non-deterministic Brain above that boundary and holds no actuator authority. The deterministic permission path below the boundary evaluates each proposed action against the capture timestamps, coordinate frames, and error qualifications of the perceptual claims behind it. Part III's governing law, proposal is not permission (principle 3), applies to perception in a specific way. A confident detection does not license motion. It is evidence behind a proposal, and the permission path can refuse that proposal only when the evidence carries an age, a frame, and an error bound it can check.

↩ The proposal boundary and permission path are established in section 1.7.

Checking a claim against physical margin means following the evidence from capture to qualification. Each sensor type must first match the physical quantity an action requires. Calibration and detector physics then ground the measurement in space and time, the capture-to-dispatch age

must be budgeted, and the spatial error, including the silent corruption no driver reports, must be bounded before any downstream check permits motion. The chapter closes by packaging those properties into an observation contract, so that a consumer can check a measurement's age, frame, and error before reading a single payload byte. The contract is the first record of the runtime strand, which meets the limit record and the evaluation record at the permission decision. Perception ends once it yields a qualified measurement. Fusing such measurements across time into persistent spatial state is the work of state estimation and spatial memory in chapter 9.

Whether an architecture employs an end-to-end vision-language-action (VLA) policy (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024), an end-to-end visuomotor model without a language backbone (Zhao et al. 2023), or a modular estimator, it must reconcile the physical scene captured at a past instant with the kinematic state of the robot at execution. Architectural choices alter processing latency and representation formats, but no network topology can bypass photon integration, sensor readout, or bus transport delay. Explicit timestamps and spatial transforms keep this relationship inspectable; a latent representation holds it only implicitly, so checking it requires external instrumentation. Neither approach converts ambiguous sensor signals into ground truth. A specular reflection on a polished warehouse floor remains an optical hazard regardless of model capacity.

Navigating past a blind rack corner requires establishing a bounded occupied volume in the robot's base coordinate frame before available stopping distance runs out. RGB appearance helps classify an object as a person, but monocular bearing alone cannot supply metric range. The navigation camera therefore requires corroborated depth measurements before its detections can justify motion clearance, and on the running machine the spinning LiDAR introduced below supplies them. At the end-effector, sensing requirements change completely. While external optical cameras guide the initial reaching approach, detecting contact mechanics, shear forces, and incipient slip requires tactile sensing directly at the fingertips. Sensor selection begins by identifying the physical quantity governing each action, followed by analyzing sensor coverage, update rates, and environmental failure modes.

Table 8.1 compares the physical evidence each sensing mechanism provides alongside the primary failure modes and safety checks required before fusing claims across an embodied sensor suite. Adding a secondary sensor mitigates a shared failure mode only when its independent spatial coverage, timing guarantees, extrinsic calibration, and correlated failure risks are established.

Table 8.1: Spatial sensor modality comparison: Sensor mechanisms provide complementary but fallible claims across physical operating regimes. Numeric range, accuracy, refresh, and link budgets require a named device and mode. For example, Prophesee EVT2 encodes an event in 32 bits, so an event camera's link load scales with its event rate before transport overhead.

Modality	Direct physical information	Main limitation	Permission-relevant check
RGB camera	Bearing, appearance, and image motion	Monocular depth scale is ambiguous; glare, darkness, and occlusion can hide objects	Fresh capture time and validated metric corroboration
Active stereo	Disparity-derived range with infrared texture projection	Sunlight washes out projected pattern; range error grows quadratically with distance (z^2)	Range-dependent residual and bias validation
Structured light	High-density triangulated depth from projected coded patterns	Ambient solar infrared flooding; multi-robot pattern crosstalk; short range ($< 3\,\mathrm{m}$)	Lighting lux limits and pattern decoding integrity
Time-of-flight	Metric depth via phase delay (iToF) or photon arrival timing (dToF)	iToF phase wrapping; multipath reflections in corners; flying pixels on moving edges	Ambiguity distance and corner multipath flags
Pulsed LiDAR	Metric time-of-flight range at sampled laser beam directions	Near-field aerosol backscatter in fog/rain; retroreflector blooming; specular puddle dropouts	Qualified range, return quality, and scan timing
FMCW radar	Metric range and instantaneous radial Doppler velocity of targets	Multipath ghost returns off ground; lower angular resolution; antenna sidelobe clutter	Track association and coverage of crossing targets
Event camera	Asynchronous per-pixel log-intensity contrast events ($\geq C_{\mathrm{th}}$)	Unchanging scenes supply little event evidence; noise and flicker can still generate events	Event-rate health and independent metric depth source

Modality	Direct physical information	Main limitation	Permission-relevant check
Optical tactile	Contact micro-depth, surface normals, and shear force vectors	Zero sensing before physical contact; strictly localized to gripper elastomeric pad	Contact threshold, slip flags, and marker tracking

On an operational robot, this division of responsibility is explicit and follows established industrial safety standards. Neither the vision model nor the spinning LiDAR owns the certified safety claim regarding an obstacle at the rack end. ISO 3691-4 mandates that an autonomous industrial truck detect persons in its path at Performance Level d (PL d) (International Organization for Standardization 2023b; TÜV Rheinland 2021), the second highest of the five reliability grades (a through e) defined in ISO 13849-1 (International Organization for Standardization 2023a). Mobile bases satisfy this mandate using an industrial safety laser scanner, a dual-channel planar time-of-flight sensor certified as an active opto-electronic protective device under IEC 61496-3 (International Electrotechnical Commission 2025; Keyence Corporation 2026). By design and certification standards, the safety scanner performs no semantic classification; it reports solely whether an opaque object has entered its protective optical field, directly triggering the certified emergency stop circuit.

The mobile base's spinning LiDAR (an Ouster OS1-64) carries no functional safety rating; its datasheet certifies only electrical, laser, and electromagnetic compatibility (Ouster, Inc. 2025). Instead, it provides the dense 3D metric range field that the permission path evaluates against spatial clearance limits. The permission path runs on its own safety microcontroller (MCU), as chapter 4 placed it, but the clearance evaluation it performs there is an unrated admission check, a software layer above the certified safety scanner. Two components therefore guard the rack end, the permission path's unrated clearance check and the certified scanner beneath it. The permission path's role is to keep learned proposals smooth and proactive, adjusting trajectories well before the robot approaches the scanner's protective boundary and causes an abrupt hardware stop.

↪ The certified drive, scanner, and arm safety functions beneath the permission path, and the one rated watchdog on it, are developed in section 12.9.3.

Meanwhile, the navigation camera enriches the learned proposer with semantic context: distinguishing a coworker from a stationary tote and predicting velocity headings. Because the proposer constructs its trajectory predictions relative to the camera frame, that image frame's capture timestamp defines the evidence epoch carried by the resulting motion proposal. Section 8.4.2 details how the high-rate LiDAR stream is synchronized and cross-dated against this camera epoch. When operational conditions shift, other sensors fill the gaps: active illumination resolves textureless surfaces, event cameras preserve high-speed edge transitions across wide dynamic ranges, and tactile arrays capture localized contact dynamics that remote cameras cannot see. Before any of these sensors can anchor downstream motion, however, the system must account for how physical capture and mounting geometry distort the raw signal.

8.2 Capture Physics and Calibration

When photons from a warehouse rack upright strike the navigation camera's silicon detector, they do not become an instantaneous feature tensor in host memory. Before a neural backbone or state observer ever reads the frame, the photodiode array must accumulate charge over a finite exposure window, serialize rows off the sensor die, and move bytes across the interconnect, while the robot keeps moving through every millisecond of that process. In CMOS active-pixel sensors, incident photons generate electron-hole pairs through the photoelectric effect according to a wavelength-dependent quantum efficiency $\eta_{\mathrm{QE}}(\lambda_{\mathrm{wl}})$. Silicon photodiodes accumulate this photocharge across a finite exposure interval t_{exp}, collecting total charge:

$$Q = \int_0^{t_{\mathrm{exp}}} \frac{q\eta_{\mathrm{QE}}(\lambda_{\mathrm{wl}})}{h\nu} P_{\mathrm{opt}}(t)\, dt$$

where q is the elementary charge, h is Planck's constant, ν is optical frequency, and $P_{\mathrm{opt}}(t)$ is incident optical power. Exposure integration, readout serialization, bus transport across PCIe or MIPI CSI, memory staging, preprocessing kernels, and neural backbone inference each consume a slice of the end-to-end latency budget.

Because the photodiode accumulates charge over the finite exposure duration t_{exp}, the captured frame integrates scene radiance over time while both the robot and the surrounding environment

are in motion. Assuming approximately constant linear velocity $\mathbf{v}$ and angular velocity ω, with the optical center displaced from the vehicle's center of rotation by lever arm $\mathbf{r}_{P/O}$, a conservative bound on the physical displacement of the camera center during exposure is:

$$\Delta x_{\text{motion}} = \|\mathbf{v}\| t_{\text{exp}} + \|\omega \times \mathbf{r}_{P/O}\| t_{\text{exp}}$$

This distance measures the physical displacement of the camera center, not the pixel footprint of the blur on the detector plane. Projecting this displacement into image pixels requires scene depth, focal length, viewing direction, and relative attitude changes. Pure rotational motion can smear the image severely even when the camera center remains completely stationary. The resulting **motion blur** acts as a low-pass spatial filter, attenuating image gradients and destroying high-frequency edge information before neural feature extraction or corner detection can run. Downstream estimators can use accurate timestamps and extrinsic calibrations to position the blurred observation at its capture epoch, but no coordinate transformation can synthesize geometric detail lost during optical integration.

The finite integration window also determines which physical instant the resulting image represents. Under an idealized linear sensor response, the pixel value reflects incoming scene radiance $I(t)$ integrated across the exposure window:

$$I_{\text{pixel}} = \int_0^{t_{\text{exp}}} I(t)\, dt$$

The exposure midpoint is the reference epoch assigned to the integrated image, exactly $t_{\text{exp}}/2$ after the shutter opens and $t_{\text{exp}}/2$ before it closes. This convention dates the measurement without making its blurred content an instantaneous sample; changes in illumination or motion within the window still require qualification. For the mobile base running at its 1.5 m/s drive limit with the navigation camera configured to a 16 ms exposure, the frame represents the scene where the robot was 8 ms earlier, while the robot traverses 24 mm of translation during the exposure itself. Exposure time is a configured software parameter rather than an immutable hardware constant. The warehouse mobile manipulator configures this relatively long exposure to operate reliably in dim aisle lighting; higher ambient illumination permits shorter integration times, scaling down both motion blur and midpoint latency proportionally.

8.2.1 Rolling shutter distortion and motion compensation

Assigning a single exposure midpoint to an entire image assumes that every pixel row integrates light over that same window. In a **global shutter** sensor, every photodiode integrates charge during the exact same time window and transfers accumulated electrons to shielded in-pixel storage nodes simultaneously. In contrast, cost and silicon die area constraints often dictate the use of a **rolling shutter** sensor, where rows are reset, exposed, and read out sequentially from top to bottom ($y = 0$ to $y = H - 1$). Each successive scanline begins its exposure after an incremental row readout period t_{row}, introducing a progressive time skew across the vertical axis:

$$t_{\text{scan}}(y) = t_{\text{start}} + y \cdot t_{\text{row}}$$

where t_{start} denotes the exposure start time of the initial row ($y = 0$). When the vehicle translates at linear velocity $\mathbf{v}$ perpendicular to a vertical object, scanline y samples that object at an along-track spatial offset:

$$\Delta \mathbf{x}(y) = \mathbf{v} \cdot y \cdot t_{\text{row}} \tag{8.1}$$

Section 5.5.2.2 met this row timing as a skew that corrupts the pairing of a logged frame with an action. On a moving base the same skew becomes the spatial offset of equation 8.1, and it is charged against clearance.

Under angular motion, rolling shutter distortion becomes non-linear. If the camera rotates with angular velocity vector $\omega = [\omega_x, \omega_y, \omega_z]^\top$ (for example, rapid yaw or pitch during aggressive vehicle maneuvers), the camera attitude drifts continuously throughout the readout phase. The optical axis points in a slightly different direction for every single scanline. For a row at vertical coordinate y, the incremental rotation relative to the start of the frame is approximately $\Delta\theta_{\text{att}}(y) \approx \omega \cdot y \cdot t_{\text{row}}$. This

line-dependent rotation bends straight structural lines into curves, stretches or compresses vertical scale during pitching motions, and shears vertical surfaces into inclined planes, as photographed on the rotating propeller blades in figure 8.2.

Consider the warehouse mobile manipulator navigating past a vertical rack upright at its 1.5 m/s drive limit. A vertical post that spans the full frame height does not appear vertical in the raw pixel array, because the sensor reads the bottom scanline ($y = 3{,}039$) an entire readout cycle after the top scanline ($y = 0$). This delay induces two distinct geometric errors that consume separate safety margins: along-track translation shears the upright along the direction of travel, while base yaw during a turn shifts the feature laterally. Notebook 8.1 quantifies both effects.

Napkin Math 8.1: The readout shear

Problem: The base passes a rack upright at its drive limit while turning into the aisle. How far apart does the navigation camera record the upright's top and bottom rows, and what share of the corresponding clearance allowance does each error consume?

Variables:

- Rows $H = 3{,}040$ and readout $t_{\text{readout}} = 16.6$ ms (Datasheet, full-resolution mode).
- Drive limit $v_{\text{base}} = 1.5$ m/s (Illustrative).
- Yaw rate $\omega_z = 0.5$ rad/s while turning into the aisle, at the rack-end range $z = 1.10$ m (Illustrative).
- Localization allowance $\delta_{\text{loc}} = 50$ mm, the along-travel term of the stopping budget (section 2.3.1), and side clearance 0.15 m, the gap the base keeps between its flank and the rack faces while it drives the aisle (both Illustrative; section C.14.1).

Math:

Row interval.

$$t_{\text{row}} = \frac{t_{\text{readout}}}{H} = \frac{16.6 \text{ ms}}{3040} = 5.46 \ \mu\text{s}$$

Along-track skew. At the last row, equation 8.1 gives

$$\Delta x = v\, t_{\text{readout}} = 1.5 \text{ m/s} \times 16.6 \text{ ms} = 24.9 \text{ mm}, \quad \frac{\Delta x}{\delta_{\text{loc}}} = \frac{24.9 \text{ mm}}{50 \text{ mm}} = 0.50$$

Lateral skew. The bearing rotates by $\omega_z\, t_{\text{readout}}$ during readout, which moves the upright sideways at range z by

$$\Delta y = z\omega\, t_{\text{readout}} = 1.10 \text{ m} \times 0.5 \text{ rad/s} \times 16.6 \text{ ms} = 9.1 \text{ mm}, \quad \frac{9.1 \text{ mm}}{150 \text{ mm}} = 0.061$$

Sensitivity. At the aisle's chosen 1.3 m/s, the along-track skew is still 21.6 mm, 43.2 percent of δ_{loc}.

Result: The bottom rows see the upright from a camera position 24.9 mm farther down the aisle than the top rows did, half of δ_{loc}. Yaw adds 9.1 mm sideways, about 6.1 percent of the side clearance.

Systems insight: Uncorrected, one readout would spend half of the localization allowance that the stopping budget holds for the whole state estimate, and slowing to aisle speed barely helps. The budget's δ_{loc} is therefore valid only if the skew is removed before the estimator sees the frame. The lateral term is small at the rack end, but it grows with range and turn rate, so it belongs in the calibration and timing checks rather than in a budget row.

The along-track error charges directly against the localization allowance δ_{loc} established in section 2.3.1. Treating the image as an instantaneous rigid perspective projection mistakenly models the bottom of the upright as closer along the corridor than the top, directly consuming the along-travel stopping allowance. Both error components scale linearly with total readout time. A global-shutter

sensor, such as the wrist camera on the arm, eliminates scanline skew at the expense of larger pixel pitch and higher silicon cost. Selecting a shutter architecture is therefore a primary systems decision whenever a sensor's measurements inform clearance boundaries. The running machine deliberately carries a rolling-shutter navigation camera on its base, so its perception pipeline must handle this constraint rather than avoid it.

Resolving this spatial artifact requires **rolling-shutter motion compensation**: unwarping image coordinates geometrically using an interpolated camera pose for every scanline. The onboard IMU supplies high-rate (1,000 Hz) angular velocity and specific-force[1] samples rather than direct velocity measurements. A fused state estimator integrates these inertial readings with bias estimates, gravity tracking, and wheel odometry to compute continuous body poses. Transforming these poses through calibrated body-to-camera extrinsics yields an estimated optical pose for each scanline, allowing the perception pipeline to rectify the image before running stereo matching or landmark triangulation. Plotted against row index, the uncompensated skew is a straight line, because each row is read a fixed interval after the one above it (figure 8.1). The line ends halfway to the 50 mm allowance, which is why compensation, not a budget row, must absorb it.

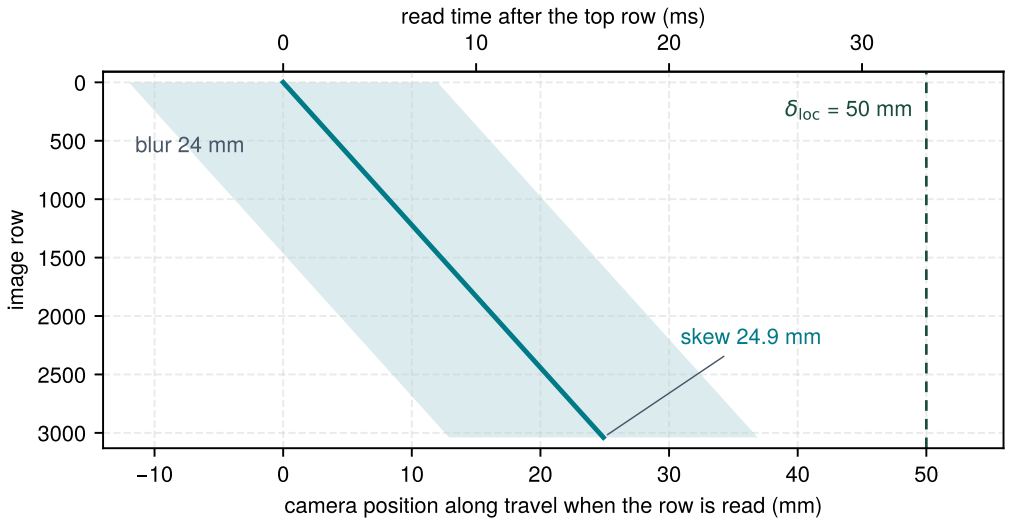


Figure 8.1: Row-by-row sampling on the navigation camera: At the 1.5 m/s drive limit, each of the 3,040 rows is read 5.46 µs after the one above it, so the bottom row sees the rack upright from a camera position 24.9 mm farther down the aisle than the top row (solid line; upper axis gives the read time). The shaded band is each row's own motion blur, 24 mm over the 16 ms exposure. Uncorrected, the skew alone spends about half of the 50 mm localization allowance (dashed line).

[1] **Specific Force**: An accelerometer measures non-gravitational acceleration, $\mathbf{a} - \mathbf{g}_0$, so a unit resting on the floor reads one g upward and one in free fall reads zero. Dead reckoning must therefore rotate specific force into the navigation frame and restore the gravity vector, and an attitude error θ_{att} leaks $g \sin \theta_{att}$ into horizontal acceleration, which integrates twice into the per-row poses that de-skewing relies on.

Figure 8.2: Rolling shutter distortion geometry: A phone camera's rolling shutter photographs a turning aircraft propeller. Because scanlines are exposed and read sequentially from top to bottom rather than simultaneously, blades that move appreciably during the frame's readout are recorded as curved, disconnected arcs; each row samples the blade at a later instant. On a moving robot the same row-by-row sampling shears straight structures and biases the geometry downstream estimators triangulate. (Credit: Dick Lyon, Wikimedia Commons, CC BY-SA 4.0.)

Geometric unwarping must precede spatial interpretation, as the detached propeller blades of figure 8.2 show. Because scanlines are exposed and read in sequence rather than simultaneously, rapid motion shears radial structures across rows. Learned feature backbones can misinterpret row-by-row shear as intrinsic physical geometry, while classical stereo matchers fail when scanline displacement shifts correspondences off the calibrated epipolar line.[2] Scaling up model capacity cannot compensate for uncorrected physical sampling distortion; the network learns biased geometric representations instead.

[2] **Epipolar Line**: For a calibrated camera pair, a point seen in one image can appear in the other only along one line, the projection of the ray from the first camera's center through that point (section 8.6 derives it). Stereo matchers search that line alone, so a rolling-shutter shear that moves a correspondence even one row off it produces a mismatch rather than a slightly worse match.

8.2.2 Camera calibration

Unwarping a rolling-shutter image still leaves a pixel without a physical direction or position if the camera's internal lens parameters or chassis mount have shifted. Under the **pinhole camera model** (Hartley and Zisserman 2003), the intrinsic matrix $\mathbf{K}$ maps a point $\mathbf{p}_c = [X_c, Y_c, Z_c]^\top$ in the camera's optical frame to the pixel $[u, v]^\top$:

$$\tilde{\mathbf{m}} \sim \frac{1}{Z_c}\mathbf{K}\mathbf{p}_c = \frac{1}{Z_c}\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} \tag{8.2}$$

where f_x, f_y are focal lengths in pixels and (c_x, c_y) is the principal point. Inverting the map recovers only a bearing ray, along which every depth is consistent with the pixel:

$$\mathbf{r}(\tilde{\mathbf{m}}) = \mathbf{K}^{-1}\tilde{\mathbf{m}} = \begin{bmatrix} \frac{u-c_x}{f_x} \\ \frac{v-c_y}{f_y} \\ 1 \end{bmatrix} \tag{8.3}$$

This bearing ray emanates from the camera's optical center at the precise instant of photon capture t_{capture_0}. If downstream motion planning unprojects $\mathbf{r}(\tilde{\mathbf{m}})$ using the robot's *current* pose instead of its historical capture pose, the current evidence age at that downstream use, $t_{\text{now,mono}} - t_{\text{capture}_0}$, translates the ray origin by $\Delta\mathbf{p} = \int_{t_{\text{capture}_0}}^{t_{\text{now,mono}}} \mathbf{v}(t)\,dt$. These timing and angular errors scale along entirely different physical axes. At the 1.5 m/s drive limit, each millisecond of unmodeled latency shifts the ray origin by 1.5 mm, making the dispatch-age contribution of 51.6 ms displace the origin by 77.4 mm. Attitude errors, by contrast, scale with range. A single degree of orientation misalignment shifts a target at the 1.10 m rack-end distance by 19.2 mm laterally, expanding proportionally with distance. Unchecked, either discrepancy can push a physical rack upright entirely outside the bounding volume evaluated by the permission path. A single monocular camera cannot recover metric range along its bearing rays; scale must be supplied by stereo disparity, an active ranging sensor, or a learned geometric prior, and each introduces uncertainty that section 8.6 audits against the operating clearance budget. Optical lens distortion is rectified inside the hardware image signal processor (ISP) stage detailed in table 8.3. Section C.12 develops the full projection equations, radial distortion models, and Zhang's planar calibration method (Zhang 2000).

Spatial extrinsics and temporal synchronization between the camera and the IMU must be calibrated jointly (Furgale et al. 2013). An extrinsic orientation error θ_{ext} misplaces an observed feature at depth z by approximately $z\theta_{\text{ext}}$, while spatial translation between the camera center and the IMU introduces the centripetal and tangential lever-arm accelerations given in section B.6.4. Similarly, an uncalibrated temporal offset Δt_{imu} between image exposure and inertial sampling causes an effective orientation error of $\omega_z \Delta t_{\text{imu}}$ while turning at angular velocity ω_z. This timing skew displaces the reconstructed feature by approximately $z\omega_z \Delta t_{\text{imu}}$, growing with both vehicle yaw rate and target distance. Hardware-latching exposure triggers and IMU clock signals to a shared real-time counter bounds this synchronization offset to sub-microsecond regimes, leaving small residual latencies for online filtering. Because mechanical shocks and thermal expansion degrade calibration over time, every observation packet must record its calibration identifier alongside its empirical operating envelope.

8.2.3 Choosing metric depth for the aisle

Because a single bearing ray leaves range undetermined, robotic systems often pair passive cameras with active optical depth cameras to recover scale. In a warehouse aisle, however, ambient lighting, surface material properties, and multi-robot optical interference dictate which depth-sensing principle survives in practice. **Structured light** projects a known pseudo-random or coded infrared pattern onto the scene and triangulates surface deformation using an offset receiver camera. Although this active pattern injects distinct visual texture onto featureless cardboard cartons, direct sunlight from skylights or open loading docks easily washes out the projected infrared pattern, and overlapping projectors from adjacent mobile bases create optical interference that corrupts feature correspondence. Engineering qualification must therefore evaluate the operating environment under peak ambient lux and multi-robot contention rather than treating structured light as an unconditional depth source.

Rather than triangulating a spatial pattern, **indirect time-of-flight (iToF)** calculates distance by measuring the phase shift of continuous-wave amplitude-modulated infrared light. Phase differences yield dense range maps with compact hardware. Modulation periodicity, however, creates phase wrapping and distance ambiguity intervals. Multipath reflections in metallic warehouse corners and industrial shelving also send diffuse photons along several bounces, which bias the depth estimate. Motion between sequential phase exposures also introduces edge artifacts in dynamic scenes. **Direct time-of-flight (dToF)** avoids periodic phase wrapping by measuring the arrival times of discrete photon pulses using single-photon avalanche diodes (SPADs), yet it remains subject to timing jitter in the receiver discriminator, multipath contamination, and optical pile-up from high-reflectance foregrounds.

Active stereo combines structured infrared illumination with a calibrated stereo camera pair. Unlike structured light systems that must decode a proprietary pattern, active stereo uses projected speckle purely to synthesize artificial texture on smooth surfaces while computing standard epipolar stereo correspondence. If bright ambient light washes out the infrared pattern, the underlying stereo

pipeline continues operating over ambient visible texture; if a smooth surface lacks ambient texture, however, high illumination alone still cannot resolve depth. The range-dependent uncertainty analysis in section 8.6 determines whether active stereo depth uncertainty satisfies the robot's required stopping clearance.

For the rack end, the running machine carries none of these depth cameras. Each of these active principles has a failure mode the aisle supplies, whether daylight through open dock doors, a neighboring base's projector, or multipath off metal shelving. The machine's metric range comes instead from the OS1-64 LiDAR described later, and the navigation camera contributes bearing and class. Stereo returns in section 8.6 only as an illustrative rig that shows where a camera-only design meets a range wall.

8.2.4 Event cameras and asynchronous contrast

Conventional CMOS sensors integrate photons over a fixed exposure window, forcing a trade-off between motion blur on fast targets and photon starvation in dim aisles. **Neuromorphic event cameras** (Dynamic Vision Sensors, or DVS)[3] avoid frame-rate exposure trade-offs by abandoning discrete frames entirely (Lichtsteiner et al. 2008; Gallego et al. 2022). Each pixel on an event sensor array operates autonomously, containing a logarithmic photodiode, an analog amplifier, and two differential voltage comparators. The pixel continuously monitors incoming scene radiance $I(u, v, t)$. Whenever the temporal change in logarithmic intensity exceeds a configurable contrast threshold C_{th}, the pixel emits an asynchronous digital event packet e_k:

$$|\Delta \ln I(u,v,t)| = |\ln I(u,v,t) - \ln I(u,v,t_{\text{last}})| \geq C_{\text{th}}$$

$$e_k = \langle u_k, v_k, t_k, p_k \rangle$$

where (u_k, v_k) is the pixel address, t_k is the event timestamp latched with microsecond resolution (1–10 µs), and $p_k \in \{-1, +1\}$ is the polarity indicating whether radiance increased or decreased.

Event sensing preserves the temporal fidelity of high-speed edge transitions without waiting for a complete exposure cycle and serial scanline readout, providing high temporal resolution for rapid maneuvers. The systems trade-off lies in the nature of temporal contrast. Static obstacles produce zero events when the camera is stationary, so an unmoving obstacle is undetectable without ego-motion. Furthermore, high-frequency fluorescent lighting flicker and thermal noise generate spurious event bursts. The bandwidth demand across the physical sensor interface is therefore dynamic and scene-dependent rather than deterministic and fixed.

Downstream estimation algorithms must convert asynchronous coordinate streams in (u, v, t) space into representations suitable for neural networks or state observers. Accumulating events into fixed time slices allows the use of standard convolutional backbones but reintroduces latency; processing individual event timestamps maintains sub-millisecond temporal responsiveness at the cost of specialized asynchronous graph or spiking architectures. Neither representation provides absolute metric depth without stereo baseline pairing or active sensor fusion. For the rack-end decision, an event camera would add edge timing but no range, and it would lose a worker who stops at the corner, so the running machine does not carry one.

8.2.5 LiDAR and radar ranging

Passive cameras cannot measure metric distance along a single viewing ray, and short-range depth cameras fail when dock sunlight or metal shelving corrupts their patterns. Active ranging sensors, principally Light Detection and Ranging (**LiDAR**) and Radio Detection and Ranging (**radar**), overcome these limits by emitting structured electromagnetic energy and timing its return to measure metric range and radial velocity directly. Pulsed time-of-flight LiDAR calculates radial distance R by measuring the round-trip flight duration Δt of an emitted laser pulse traveling at the speed of light c:

$$R = \frac{c \cdot \Delta t}{2}$$

In three-dimensional space, optical scanners steer each pulse along azimuth θ_{az} and elevation ϕ, casting an emitted ray along unit direction vector $\mathbf{d}(\theta_{\text{az}}, \phi) = [\cos\phi\cos\theta_{\text{az}},\ \cos\phi\sin\theta_{\text{az}},\ \sin\phi]^\top$.

[3] **Neuromorphic**: Carver Mead coined the term at Caltech for analog circuits that compute the way neural tissue does (Mead 1990), and the silicon retina built in his group is the ancestor of the Dynamic Vision Sensor (Lichtsteiner et al. 2008). Like the retina, each pixel reports change rather than level, so an obstacle that stands still in front of a stationary sensor produces no events and is invisible to it.

When reflected energy trips the receiver threshold after flight time Δt, the sensor unprojects the return into a 3D point in the sensor frame $\mathcal{F}_{\text{lidar}}$ via ray casting (equation 8.4):

$$\mathbf{p}_{\text{lidar}} = R \cdot \mathbf{d}(\theta_{\text{az}}, \phi) = \frac{c \cdot \Delta t}{2} \begin{bmatrix} \cos\phi \cos\theta_{\text{az}} \\ \cos\phi \sin\theta_{\text{az}} \\ \sin\phi \end{bmatrix} \tag{8.4}$$

Spatial uncertainty in this unprojected 3D point decomposes into two distinct physical mechanisms:

1. **Radial uncertainty (σ_R)**: Governed by receiver discriminator timing jitter $\sigma_{\Delta t}$ and pulse rise time, $\sigma_R = \frac{c}{2}\sigma_{\Delta t}$ remains approximately constant across operating distances until return signal attenuation drops photon counts near the detection threshold. The manufacturer datasheet for the robot's OS1-64 specifies a one-standard-deviation precision between 5 mm and 30 mm beyond 1 m. It also specifies a systematic accuracy bias of 25 mm on diffuse targets in the specified 1024×10 Hz mode (Ouster, Inc. 2025). These are datasheet scales, not qualified bounds for the running machine's selected scan mode, surfaces, and motion. The accuracy figure describes the bias of the mean, a calibration offset that temporal averaging cannot remove.
2. **Transverse footprint**: Governed by optical beam divergence $\delta\theta_{\text{div}}$; internal angular encoder quantization contributes a separate bearing error. The laser pulse expands into a narrow cone rather than propagating along an infinitesimal line. Taking $\delta\theta_{\text{div}}$ as the full-angle divergence, the transverse spot diameter scales as range times divergence. At $R = 30$ m, the illustrative 2 mrad divergence gives a 60 mm spot diameter. This geometric footprint is not a standard deviation of position error. When it strikes a surface boundary, the detector receives mixed returns from foreground and background, blurring geometric edges; the resulting error distribution must be measured.

Most terrestrial systems operate either at near-infrared wavelengths below $1{,}000\,\text{nm}$, where silicon detectors respond, or at an eye-safe $1{,}550\,\text{nm}$ (permitting higher pulse energies under IEC 60825-1 Class 1 laser regulations).[4] The machine's OS1-64 is of the first kind; it emits at 865 nm as a Class 1 laser product (Ouster, Inc. 2025). Mechanical spinning LiDAR systems rotate an array of 16 to 128 laser diodes and avalanche photodiode (APD) detectors across an internal spindle at 10–20 Hz to sweep a 360° panoramic point cloud.

Much like rolling-shutter cameras expose scanlines in sequence, a spinning mechanical LiDAR does not record an instantaneous 3D snapshot. Completing a full 360° azimuth sweep requires $T_{\text{sweep}} = 1/f_{\text{scan}}$, corresponding to 50 ms on the machine's OS1-64 configured at 20 Hz. If the mobile base translates at velocity $\mathbf{v}$ and rotates at angular velocity ω while the sensor spins, pulses fired at the start of the sweep ($\theta_{\text{az}} = 0^\circ$) and near completion ($\theta_{\text{az}} = 360^\circ$) originate from physically distinct vehicle poses. At the 1.5 m/s drive limit and an illustrative 0.5 rad/s turn rate, the robot travels 75 mm and rotates 25 mrad (1.4°) during a single scan. Without motion correction, this sampling lag produces an apparent geometric tear where the end of the revolution meets the beginning, while shearing vertical structures across the corridor.

Restoring rigid geometric fidelity requires **continuous-time LiDAR de-skewing**. The sensor hardware latches each point return $\mathbf{p}_i$ with an exact firing timestamp $t_i \in [t_{\text{sweep}}, t_{\text{sweep}} + T_{\text{sweep}}]$. A rigid pose transformation $\mathbf{T} \in \text{SE}(3)$ encapsulates orientation and translation. In the two-subscript convention, the first subscript names the destination frame and the second names the source: `T_ab` maps coordinates from frame b to frame a, and its inverse maps back. A homogeneous point appends a final coordinate of 1 so that the matrix applies translation as well as rotation. Section B.6.4 develops this representation and composition; section 8.3.1 applies it to the machine's mounting chain. Combining high-frequency (1,000 Hz) inertial data with wheel odometry allows the state estimator to compute the continuous trajectory of the vehicle body frame, $\mathbf{T}_{wb}(t_i)$. Applying the calibrated mounting extrinsic $\mathbf{T}_{bl}$ transforms coordinates from the LiDAR frame to the body frame,

[4] **Laser Class 1**: The class of IEC 60825-1 whose emission is safe under all reasonably foreseeable viewing, including through optical instruments (International Electrotechnical Commission 2014). Below about $1{,}400\,\text{nm}$ the eye focuses the beam onto the retina, while at $1{,}550\,\text{nm}$ the cornea and lens absorb it first, so the permitted pulse energy is far higher. Those longer wavelengths need indium gallium arsenide detectors rather than silicon, which makes wavelength a range vs. cost choice made under a fixed safety ceiling.

yielding the world-frame sensor pose $\mathbf{T}_{wl}(t_i) = \mathbf{T}_{wb}(t_i)\mathbf{T}_{bl}$. Each individual homogeneous point return is then projected back into the LiDAR's coordinate frame at the frame reference epoch t_{sweep}:

$$\mathbf{p}_i(t_{\text{sweep}}) = \mathbf{T}_{wl}^{-1}(t_{\text{sweep}}) \cdot \mathbf{T}_{wl}(t_i) \cdot \mathbf{p}_i(t_i)$$

De-skewing is what makes an accumulated LiDAR map usable. The returns registered in figure 8.3, gathered over about 18 s of driving through a city intersection, keep crisp building facades, road striping, and parked vehicles only because each return was placed with the vehicle pose at its own firing epoch t_i. Independently moving obstacles in the environment require separate velocity estimation and clustering. Flash LiDAR architectures avoid sweep distortion altogether by illuminating the entire field of view with a single laser pulse to capture an instantaneous 3D range matrix. Distributing finite optical pulse energy across a wide solid angle, however, sharply limits maximum operational range compared to concentrated, steerable beams.

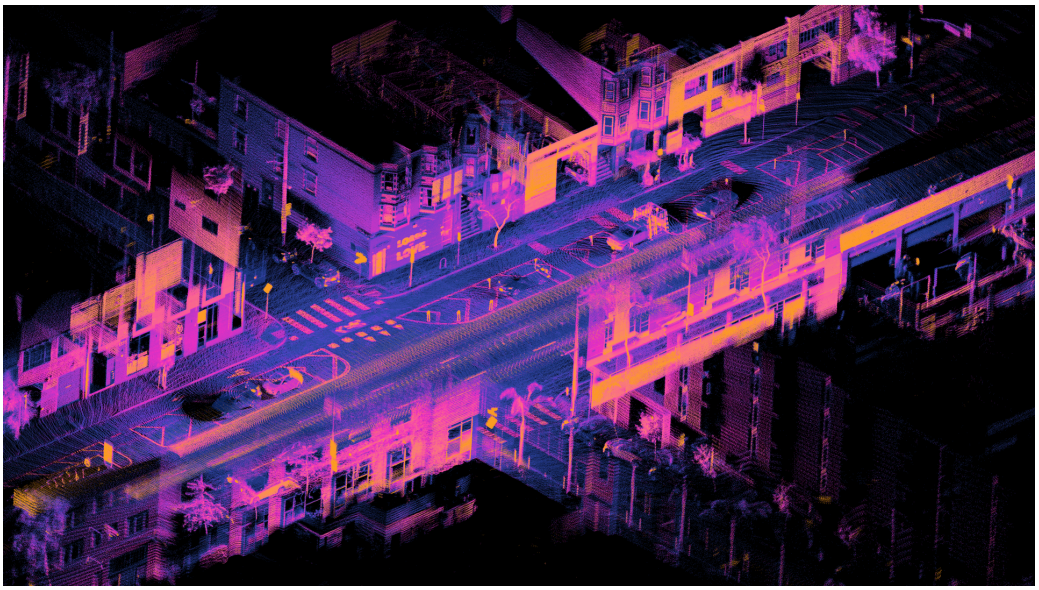


Figure 8.3: Registered LiDAR point cloud: Returns from a 64-beam spinning LiDAR on a moving car, registered into one map over about 18 s as a simultaneous localization and mapping (SLAM) estimator tracked the vehicle's trajectory, and shown in orthographic projection colored by return intensity times range. The sharp building edges and vehicles hold only because every return was placed with the pose at its own firing time; a single sweep is far sparser than this aggregate. (Credit: Daniel L. Lu, Wikimedia Commons, CC BY 4.0.)

Millimeter-wave radar offers an orthogonal active sensing channel, operating in the 76–81 GHz band ($\lambda \approx 3.9\,\text{mm}$). Rather than emitting short, high-energy pulses, **Frequency-Modulated Continuous-Wave (FMCW) radar** transmits linear frequency chirps whose instantaneous carrier frequency sweeps across bandwidth BW_{chirp} over chirp duration T_c with slope $S = \text{BW}_{\text{chirp}}/T_c$. When the emitted waveform reflects from an obstacle at range R, the echo returns delayed by round-trip flight time $t_{\text{rt}} = 2R/c$. Mixing the received echo with a replica of the transmitted signal generates an intermediate frequency (IF) beat tone at frequency f_b:

$$f_b = S \cdot t_{\text{rt}} = \frac{2R\,\text{BW}_{\text{chirp}}}{cT_c}$$

Computing a fast Fourier transform across the digitized fast-time samples of each chirp (the **Range-FFT**) extracts metric range R directly, without requiring optical contrast or ambient scene illumination. Physical range resolution depends strictly on total sweep bandwidth:

$$\Delta R = \frac{c}{2\,\text{BW}_{\text{chirp}}}$$

For a standard 4 GHz radar sweep (77–81 GHz), FMCW radar achieves a fine range resolution of $\Delta R \approx 3.75$ cm. The transmitted chirps simultaneously encode radial velocity and bearing. A moving target shifts the phase of successive chirp reflections proportionally to its Doppler velocity; evaluating a second Fourier transform across chirps (the Doppler-FFT) extracts this radial velocity within a single coherent processing interval, avoiding the latency and noise of multi-frame position differencing. Array processing across multiple receive antennas resolves target azimuth and elevation, albeit at substantially coarser angular resolution than optical LiDAR. The running machine does not mount radar, but millimeter-wave sensing is a primary modality on outdoor autonomous platforms. Its millimeter wavelength penetrates the dust, fog, and steam that scatter optical wavelengths (section 8.7). However, radar returns remain sensitive to specular scattering, surface orientation, and multi-path ghost targets, providing complementary evidence rather than an infallible, all-weather guarantee.

8.2.6 Tactile contact sensing

At the end-effector interface, external cameras and range finders cannot measure contact pressure or verify friction adhesion during physical manipulation; indeed, the gripper structure often occludes the contact patch entirely. An optical tactile sensor[5] addresses this perceptual blind spot by mounting a compliant, internally illuminated elastomer membrane on the fingertip and imaging its deformation with an internal camera. Photometric stereo algorithms translate surface shading into calibrated local contact topography, while tracking arrays of embedded fiducial markers yields direct measurements of tangential shear strain and incipient slip. Converting these raw optical deformations into a calibrated contact wrench requires a validated finite-element or empirical model that accounts for elastomer hysteresis, dynamic loading, and mechanical wear over time.

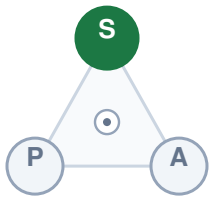
Tactile sensing thus governs the terminal contact phase of robotic grasping, whereas external cameras and range sensors guide the initial approach trajectory. Across both regimes, downstream estimators cannot treat sensor outputs as timeless, unframed tensors. Every tactile frame and spatial point cloud carries a finite physical age, a mounting transform, and a calibration uncertainty that must be attached to the measurement before any consumer can use it.

[5] **GelSight**: Adelson's group at MIT turned a reflective elastomer skin and an internal camera into a contact sensor that recovers surface geometry and, from painted markers, shear force (Yuan et al. 2017). The design makes touch one more image stream, so a tactile frame carries an exposure, a readout, and a calibration of its own and must be dated and qualified like the navigation camera's.

8.3 Dating and Placing the Measurement

A frame from the navigation camera lands in host memory as an array of pixel values. Nothing in that array says when the light was captured, where the camera was mounted, or how far a position read from it can be trusted, yet the planner that checks rack-end clearance needs all three. Every modality has the same gap. A trajectory optimizer cannot plan against unorganized time-of-flight returns, and a force controller cannot track a grid of raw intensities. Downstream estimation, planning, and spatial memory require structured measurements: metric positions, bounding volumes, and semantic classes, with directly measured velocity where the sensing modality supplies it. Translating raw streams into those measurements is the perception pipeline's responsibility. Whatever form its output takes, whether an explicit message packet or an observation token sequence, it must carry three mandatory properties before any consumer can use it: a capture timestamp, a reference frame, and a qualified uncertainty envelope.

The first mandatory property is the **capture timestamp** t_{capture_0}, which specifies the temporal midpoint of the energy integration window, the effective instant of a camera exposure or a laser firing. Physical sensors do not take instantaneous snapshots. An optical camera integrates photons across an exposure duration, and a mechanical lidar sweeps laser pulses over an azimuth arc across several milliseconds. Downstream state observers and vision-language-action policies require t_{capture_0} to ground observation tokens into the machine's dynamic equations of motion. By contrast, the arrival stamp t_{recv} marks the moment the operating system kernel completes the direct memory access (DMA) transfer. It is the software arrival timestamp that section 5.5.2.1 showed corrupting logged velocities; at runtime it corrupts the pose an observation is paired with. Half of the exposure duration, sensor readout, physical transport, and host DMA transfer all elapse before that interrupt fires. Consequently, the arrival offset $\Delta t_{\text{recv}} = t_{\text{recv}} - t_{\text{capture}_0}$ is never zero. Substituting arrival time for capture time directly corrupts downstream state estimation and action generation.



The Sense stage: photons become dated, framed claims.

At constant velocity v_{base}, the hidden travel is $\Delta x = v_{\text{base}}\,\Delta t_{\text{recv}}$. On the mobile manipulator, the navigation camera's frame completes its DMA transfer 26.6 ms after the exposure midpoint. An

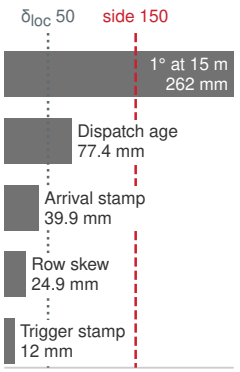
estimator that timestamps the frame upon arrival dates the measurement late by that full duration. At the base's 1.5 m/s drive limit, this temporal mismatch pairs the visual observation with an estimated pose 39.9 mm ahead of the physical position the camera occupied at capture.

Eliminating this arrival offset and bounding the camera-to-inertial offset Δt_{imu} of section 8.2 require stamping the capture event directly in hardware rather than recording when a software thread first handles the frame buffer. Software timestamps inherit operating system scheduling jitter, driver queue delays, and thread preemption. While the base rotates at 0.5 rad/s, each millisecond of scheduling jitter misorients the camera by 0.5 mrad, producing a lateral displacement of 0.55 mm at rack-end distance. Two hardware architectures avoid dating capture by software-handler scheduling. In the first architecture, a hardware GPIO strobe (such as FSYNC) from the IMU microcontroller simultaneously latches an inertial sample and triggers the camera exposure. Because this trigger edge marks only the start of exposure for the first sensor row rather than the optical midpoint, the driver must add $t_{\text{exp}}/2$ to reach the exposure midpoint. It must also add $y\,t_{\text{row}}$ for the specific image row y under consideration. Omitting this correction dates the navigation camera frame 8 ms early, translating to 12 mm of unmodeled travel at the drive limit. Furthermore, because automatic exposure control varies t_{exp} dynamically across frames, the driver must apply the specific exposure time reported in each frame's metadata. In the second architecture, used for networked sensors such as the lidar units and GigE Vision cameras, each peripheral timestamps samples against an onboard hardware clock. The Precision Time Protocol (PTP) (IEEE 2020a), or its time-sensitive networking profile gPTP (IEEE 2020b), disciplines these local clocks to a master reference across the Ethernet bus. Each free-running oscillator (the camera trigger counter, the LiDAR's internal clock, the application processor, the safety MCU) defines its own **clock domain**. A capture time stamped in one domain cannot be compared with a time read in another until it is converted. The largest error that conversion may carry is the **clock-conversion bound** ϵ_{sync}, which every dated observation must carry alongside its capture time; section 8.6.2 details how to measure it.

The second mandatory property is the **reference frame** identifier $\mathcal{F}_{\text{sensor}}$. Every spatial measurement is natively resolved in the local coordinate system of the physical transducer. A camera measures bearing angles relative to its optical center, whereas a radar measures range and Doppler velocity relative to its antenna array. Downstream trajectory planners and occupancy maps, by contrast, operate in a chassis body frame $\mathcal{F}_B$ or an inertial navigation frame $\mathcal{F}_W$. Transforming an observation from $\mathcal{F}_{\text{sensor}}$ to $\mathcal{F}_B$ requires applying a rigid-body spatial transformation consisting of a rotation matrix and a translation vector that define the sensor mounting extrinsics, the calibrated location and orientation at which the sensor is bolted to the machine. Coordinate frames nest hierarchically from the physical operating environment down to the transducer: $\mathcal{F}_{\text{earth}} \to \mathcal{F}_{\text{map}} \to \mathcal{F}_{\text{odom}} \to \mathcal{F}_B \to \mathcal{F}_{\text{sensor}}$, where every link in this tree introduces its own transform and uncertainty (figure 8.4).

When perception outputs omit the frame identifier or rely on nominal CAD dimensions rather than calibrated physical extrinsics, small angular offsets create large spatial errors that scale with distance. The same uncalibrated yaw error of $\theta_{\text{ext}} = 1°$ that costs 19.2 mm at the 1.10 m rack end grows linearly as the planner looks farther down the aisle. For an object at an illustrative range $z =$ 15 m down a long aisle, it induces a transverse position error $\Delta y \approx z \sin\theta_{\text{ext}}$ of 0.26 m. This lateral offset is approximately 1.7× the 0.15 m side clearance that the base maintains from the warehouse racks, causing the planner to either steer into the racking or falsely reject an open aisle as obstructed.

The third mandatory property is an **uncertainty representation**, such as a measurement covariance Σ_z paired with a validated tail bound or hard error envelope when physical clearance depends on it. Environmental degradations such as low illumination, atmospheric scattering, specular reflections, and out-of-distribution neural network errors distort the residual distribution far beyond Gaussian assumptions. While a covariance matrix characterizes second moments, it does not guarantee a conservative bounding volume for the true physical state. Downstream state estimators and policy heads rely on Σ_z to weight incoming sensory updates against dynamic motion predictions. If perception reports an artificially optimistic covariance, the estimator over-weights spurious measurements and commands aggressive, erratic actuator responses. Robust physical safety decisions must therefore account explicitly for systematic bias, missed detections, and sensor operational limits alongside the reported covariance.



Timing slips cost tens of millimeters; a 1° mount error at 15 m costs 0.26 m.

8.3.1 Calibrating the coordinate chain

Even when each sensor frame is named, an observation measured at the wrist camera still cannot guide the arm or base until the runtime chains every joint and mount transform back to the world at the exact capture instant t_{capture_0}. A coordinate frame transform maps a point expressed relative to one origin into the coordinate system of another: its rotational part reorients the coordinate axes, while its translational part accounts for the spatial offset between origins. Homogeneous matrix representations package both operations so that camera-to-wrist, wrist-to-base, and base-to-world transforms can be composed at t_{capture_0}. The notation below writes the destination as a lower index and the source as an upper index, expressing the same mapping direction as the two-subscript convention used for LiDAR de-skewing.

A robot's spatial coordinate system forms a **directed acyclic graph (DAG)** of transforms $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where vertices $\mathcal{V} = \{\mathcal{F}_i\}$ represent physical coordinate frames and directed edges $\mathcal{E}$ represent time-varying rigid-body transforms $\mathbf{T}_{ij}(t) \in \mathrm{SE}(3)$. In practice, systems constrain this DAG to a directed tree where each non-root frame has exactly one parent, ensuring a unique path between any two frames. When unprojecting an optical observation $\mathbf{p}_c(t_{\text{capture}_0})$ into world coordinates, the runtime composes the transforms along that tree:

$$\mathbf{p}_{\text{world}}(t_{\text{capture}_0}) = \mathbf{T}^{\text{map}}_{\text{world}} \cdot \mathbf{T}^{\text{odom}}_{\text{map}}(t_{\text{capture}_0}) \cdot \mathbf{T}^{\text{base}}_{\text{odom}}(t_{\text{capture}_0}) \cdot \mathbf{T}^{\text{flange}}_{\text{base}}(t_{\text{capture}_0}) \cdot \mathbf{T}^{\text{cam}}_{\text{flange}} \cdot \mathbf{p}_c(t_{\text{capture}_0})$$

The physical mounting location of the sensor on the kinematic chain dictates whether those transforms stay fixed on the body or update with the arm's joints across this DAG. Two canonical mounting topologies govern how the extrinsic calibration is solved.

In an **eye-in-hand** configuration, the camera or depth sensor is bolted directly to the manipulator's moving tool flange ($\mathcal{F}_{\text{flange}}$). Moving any preceding joint alters the sensor's optical center relative to the world, requiring continuous forward kinematics updates. Solving for the rigid extrinsic transform $\mathbf{X} = \mathbf{T}^{\text{cam}}_{\text{flange}} \in \mathrm{SE}(3)$ requires executing robot motions across $N \geq 2$ non-parallel rotational axes while observing a stationary fiducial target, such as a ChArUco grid.[6] The running machine's wrist camera is mounted this way, which makes this the calibration its grasps depend on. Because the fiducial target stays fixed in the base frame across both poses, the composed base-to-target transform $\mathbf{T}^{\text{flange}}_{\text{base},k}\mathbf{X}\mathbf{T}^{\text{target}}_{\text{cam},k} = \mathbf{T}^{\text{flange}}_{\text{base},k+1}\mathbf{X}\mathbf{T}^{\text{target}}_{\text{cam},k+1}$ is identical at poses k and $k+1$, and left-multiplying by $(\mathbf{T}^{\text{flange}}_{\text{base},k})^{-1}$ while right-multiplying by $(\mathbf{T}^{\text{target}}_{\text{cam},k+1})^{-1}$ isolates the relative motions on opposite sides of the unknown mount so that the closed kinematic loop establishes the classic **hand-eye matrix equation**:[7]

$$\mathbf{AX} = \mathbf{XB}$$

where $\mathbf{A} = (\mathbf{T}^{\text{flange}}_{\text{base},k})^{-1}\mathbf{T}^{\text{flange}}_{\text{base},k+1} \in \mathrm{SE}(3)$ is the relative flange displacement reported by the forward kinematics chain, and $\mathbf{B} = \mathbf{T}^{\text{target}}_{\text{cam},k}(\mathbf{T}^{\text{target}}_{\text{cam},k+1})^{-1} \in \mathrm{SE}(3)$ is the relative camera displacement extracted from optical target unprojection.

In an **eye-to-hand** calibration rig, the camera overlooks the workspace from a body-mounted mast or a stationary overhead beam. When calibrating against an overhead camera, the mobile base must remain parked so that the camera-to-base transform remains constant ($\mathbf{X}' = \mathbf{T}^{\text{cam}}_{\text{base}} \in \mathrm{SE}(3)$), while the target is affixed to the robot's moving end-effector ($\mathbf{Z} = \mathbf{T}^{\text{target}}_{\text{flange}} \in \mathrm{SE}(3)$). At each joint configuration, let $\mathbf{A}' = (\mathbf{T}^{\text{flange}}_{\text{base}})^{-1}$ map base coordinates into the flange frame and $\mathbf{B}' = (\mathbf{T}^{\text{target}}_{\text{cam}})^{-1}$ map camera coordinates into the target frame. These are absolute pose transforms for this rig, rather than the relative motions used above. Both paths then map camera coordinates into the flange frame:

$$\mathbf{A}'\mathbf{X}' = \mathbf{Z}\mathbf{B}'$$

If the mobile base subsequently moves relative to the overhead camera, the runtime must update the camera-to-base transform accordingly. For a mast-mounted camera that rides with the chassis, $\mathbf{T}^{\text{cam}}_{\text{base}}$ remains fixed on the body while $\mathbf{T}^{\text{base}}_{\text{odom}}(t_{\text{capture}_0})$ carries the base motion into the world frame.

Treating the nominal rigid transform chain from CAD as an exact physical calibration ignores load and temperature effects. Physical manipulators deflect under payload mass, accumulate lost motion and torsional wind-up in the joints (strain wave gears have near-zero backlash, but their

6 **ChArUco Board**: A chessboard whose white squares carry uniquely coded ArUco markers. The chessboard corners give subpixel accuracy and the codes identify which corners are in view, so calibration works from partial views of the board, the usual case for a camera on a moving wrist.

7 **Hand-Eye Calibration**: The problem was posed as $\mathbf{AX} = \mathbf{XB}$ independently by Shiu and Ahmad (1989) and Tsai and Lenz (1989), for cameras bolted to robot wrists. The camera's pose relative to the flange is never measured directly, only inferred from paired motions, so any error in $\mathbf{X}$ enters every grasp the arm plans from that camera, and poorly spread calibration motions leave it weakly determined.

flexsplines still twist under load), and expand with temperature (structural aluminum expands at $\alpha \approx 23{\times}10^{-6}/\mathrm{K}$). For an illustrative 1 m aluminum link that warms uniformly by 10°C, the corresponding expansion is 0.23 mm. A motor temperature rise alone does not establish that link-temperature change, as localized thermal gradients across structural members and encoder mountings require direct measurement before updating calibration parameters.

Fusing joint kinematics with optical frames also requires temporal alignment across the transform DAG. Any uncalibrated timestamp skew $\Delta t_{\text{skew}} = t_{\text{cam}} - t_{\text{joint}}$ between the motor communication bus (such as EtherCAT or CANopen) and the camera exposure window evaluates the forward kinematics chain at stale joint angles $\mathbf{q}(t_{\text{joint}}) \neq \mathbf{q}(t_{\text{cam}})$, inducing spatial tracking error:

$$\Delta \mathbf{p}_{\text{skew}} = \mathbf{v}_{\text{wrist}}\, \Delta t_{\text{skew}} + (\omega_{\text{wrist}} \times \mathbf{r}_{P/O})\, \Delta t_{\text{skew}}$$

At an illustrative wrist linear velocity $v = 1$ m/s and angular velocity $\omega = 2$ rad/s with a camera lever arm $r = 0.15$ m, an unmodeled 10 ms skew displaces the observation by 10 mm plus 3 mm, or 13 mm. This error causes the arm to miss a grasp whose position tolerance is 5 mm. Holding that displacement within the tolerance gives the timing requirement $\Delta t_{\text{skew}} \leq r_{\text{tol}}/(v + \omega r)$, which evaluates to 3.8 ms. Millisecond-level alignment therefore suffices, whereas microsecond synchronization is unnecessary. Systems typically meet this requirement by timestamping both streams against a single shared clock (a fieldbus distributed clock or a PTP master) and interpolating the high-rate joint stream at the camera's exposure midpoint.

The REP 105 frame convention[8] names these frames and separates a locally continuous `odom` pose from a globally corrected `map` pose. On the mobile manipulator, $\mathcal{F}_B$ is the base's `base_link`, tracked through the odometric frame `odom`, and the arm's serial chain maps its mounting flange `tool0` through the joint angles back to `base_link`. Table 8.2 lists the frame roles and the behavioral rules each consumer follows. A local controller consumes a continuous estimate and rejects unexpected transform jumps, while runtime checks verify transform timestamps and estimator uncertainties before execution.

DRAFT

8 **ROS Enhancement Proposal (REP)**: The Robot Operating System records its community conventions as numbered REPs, a process copied from Python's enhancement proposals. REP 105 fixes the meaning of `map`, `odom`, and `base_link` and the continuity each must keep (Meeussen 2010), so a frame name written into a record denotes the same physical frame for every consumer that reads it.

Table 8.2: REP 105 frame roles for a mobile robot: A compact example of global, local, body, and sensor frames. REP 105 requires `odom`-relative pose to remain continuous but does not prescribe C^1 smoothness, a covariance growth rate, or fixed accuracy; these require estimator-specific evidence.

Frame	Parent in this example	Meaning	Consumer rule
`earth`	root	Optional geodetic datum	Verify datum before global routing
`map`	`earth` if present	Global localization; corrections may jump	Keep relocalization jumps out of local feedback
`odom`	`map`	Locally continuous pose estimate; may drift	Track motion locally; monitor uncertainty and discontinuities
`base_link`	`odom`	Body-fixed origin	Body pose still inherits odometry uncertainty
`sensor_link`	`base_link`	Calibrated sensor mounting frame	Apply time-valid extrinsics and their uncertainty

Splitting consumers across the tree keeps a relocalization jump out of the actuators (figure 8.4). Global planners absorb the discontinuous Σ_{map} corrections at relocalization, while high-rate local tracking loops consume only the continuous `odom` → `base_link` branch, so a global correction never reaches the actuators as an abrupt tracking step. Mounting uncertainty still compounds with dynamic pose uncertainty across every intermediate link. Under a first-order propagation model with independent error sources:

$$\Sigma_{\text{world}} = \mathbf{J}_{\text{map}} \Sigma_{\text{map}} \mathbf{J}_{\text{map}}^{\top} + \mathbf{J}_{\text{odom}} \Sigma_{\text{odom}} \mathbf{J}_{\text{odom}}^{\top} + \mathbf{J}_{\text{ext}} \Sigma_{\text{ext}} \mathbf{J}_{\text{ext}}^{\top}$$

where the Jacobians $\mathbf{J}$ project each frame's spatial covariance into world coordinates. When frame errors correlate, the propagation must include their cross-covariance terms. A downstream consumer requires the frame identifier, capture epoch, and compounded uncertainty envelope, rather than a transform selected merely by its latest publication timestamp.

↪ How these sensor-to-body transforms go stale as the joints and base move is developed in section 9.5.

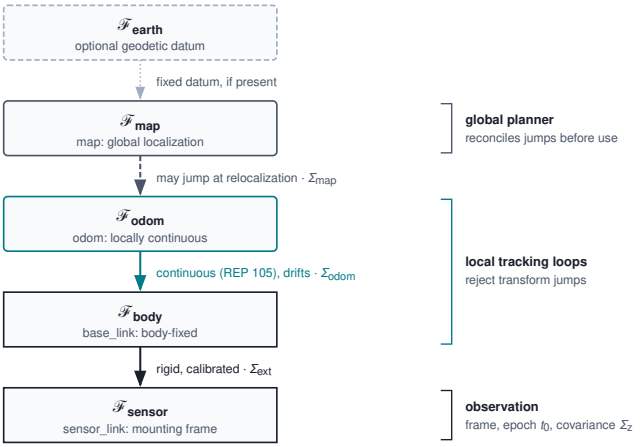


Figure 8.4: Coordinate frame tree and its consumers: Each edge states the property REP 105 or calibration gives it and the uncertainty it adds to the prose equation below. Only the odom to base_link edge is required to stay continuous; the map to odom correction may jump at relocalization, so local tracking loops consume the continuous branch while global planners reconcile the jump. A sensor observation reaches the consumers with its frame, capture epoch t_{capture_0}, and covariance Σ_z.

> **Systems Perspective 8.1: The three invariants of observation**
>
> Pass a perception claim with its capture time and clock domain, coordinate frame, and qualified uncertainty. Refuse a claim when any of those cannot be checked for the proposed motion.

8.4 Bounding Observation Age

Even when a rack-end observation is dated at capture and placed in a calibrated coordinate frame, the scene it describes belongs to the past. Delivering that measurement across an embodied runtime adds acquisition, bus transport, queueing, and neural inference delay while the robot continues to move. Any delivery deadline for perception must therefore track the full physical interval from energy integration at the detector through dispatch, bounding the observation's age before downstream actuators act.

> **Definition 8.1: Observation age**
>
> ***Observation age*** (t_{age}), also called spatial information age, is the physical time elapsed between the exposure midpoint at the transducer and the moment perception dispatches the resulting observation. It is the capture-to-dispatch part of the measurement age that section 2.2 counts from transduction. The observation keeps aging after dispatch, and the age a consumer computes on its own clock when it admits or reuses the observation is its **current evidence age**, which equals t_{age} plus the time elapsed since dispatch.
>
> 1. **Significance**: Because the robot continues moving while the compute pipeline runs, observation age converts directly into physical displacement (equation 8.5) and expands the stopping budget through the pre-brake delay τ_{delay}. On the warehouse mobile manipulator, the navigation-camera observation is already 51.6 ms old at dispatch, which is 77.4 mm of blind travel at the 1.5 m/s drive limit before any downstream consumer has inspected the frame.
> 2. **Distinction**: Conventional benchmark latency measures only the software execution time of a forward pass. In contrast, observation age spans the entire physical journey from photon arrival to dispatch: the exposure midpoint, sensor rolling readout, serialized bus transport, direct memory access (DMA) transfers, driver queueing, image signal

processor (ISP) conversions, model inference, and inter-process dispatch. Scheduling delay after dispatch belongs to the current evidence age, not to t_{age}.

3. **Common pitfall**: Systems engineers frequently confuse a frame's arrival timestamp in userspace memory (t_{recv}) with its actual physical observation time (t_{capture_0}). On the navigation camera, the arrival timestamp overlooks 26.6 ms of front-end exposure, readout, and transport. At the maximum drive limit, this unmeasured interval conceals 39.9 mm of physical travel, saddling downstream controllers with an unmodeled phase lag.

When evaluated along the machine's trajectory, observation age integrates directly into spatial displacement (equation 8.5):

$$\Delta \mathbf{p} = \int_{t_{\text{capture}_0}}^{t_{\text{capture}_0}+t_{\text{age}}} \mathbf{v}(t)\, dt \tag{8.5}$$

Travel is one of three error sources that perception must bound. Adding this motion-induced displacement to first-order stereo triangulation noise and small-angle extrinsic error gives a single error scale for the pipeline (equation 8.6):

$$\epsilon_{\text{scale}} = \underbrace{\int_{t_{\text{capture}_0}}^{t_{\text{capture}_0}+t_{\text{age}}} \|\mathbf{v}(t)\|\, dt}_{\text{path-length travel}} + \underbrace{\frac{z^2}{fb}\sigma_d}_{\text{stereo depth standard deviation}} + \underbrace{z|\theta_{\text{ext}}|}_{\text{small-angle mounting error scale}} \tag{8.6}$$

The first term integrates instantaneous speed to compute path length, providing an upper bound on net translation during the perception delay. The latency waterfall below sizes it for the navigation camera. The second term characterizes depth uncertainty for a calibrated stereo camera with baseline b, focal length f, operating range z, and disparity standard deviation σ_d. Section 8.6 derives it and evaluates it on an illustrative rig. It is a standard-deviation scale rather than an absolute bound. The third term models the lever-arm displacement produced by a small extrinsic angular misalignment θ_{ext} over target distance z, the error section 8.3 already sized at the rack end. The sum is therefore an order-of-magnitude scale, not a safety envelope. It adds magnitudes that point in different directions and leaves out calibration bias and the heavy tail of real residuals, which is why the clearance a consumer holds rests instead on a measured residual quantile (equation 8.9).

8.4.1 The latency waterfall

While the navigation camera integrates photons, clocks out pixel rows, transfers buffers across the bus, and runs neural inference kernels, the mobile base keeps moving at velocity v, turning every millisecond of pipeline delay into blind travel:

$$\Delta x_{\text{blind}} = v \cdot t_{\text{age}}$$

On the navigation camera, the observation age t_{age} accumulates stage by stage from the exposure midpoint to the moment perception publishes its processed measurement. When hardware stages execute sequentially, the total age expands as a **latency waterfall**; if adjacent stages overlap in hardware or stream through pipelined ring buffers, their combined duration replaces the naive sum of individual stage times:

$$t_{\text{age}} = \frac{t_{\text{exp}}}{2} + t_{\text{readout}} + t_{\text{transport}} + t_{\text{DMA}} + t_{\text{ISP}} + t_{\text{backbone}} + t_{\text{IPC}} \tag{8.7}$$

A complete stopping margin must also count the downstream response: expiry of the chunk lease sized in section 4.4, the control tick, fieldbus transmission, and brake onset. Summing these terms yields the partial pre-brake delay $\tau_{\text{delay}} = t_{\text{age}} + T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}$, which governs the corresponding subtotal of blind travel:

$$d_{\text{blind}} = v \cdot \tau_{\text{delay}}$$

At constant operating velocity, every millisecond of unmodeled latency drains physical clearance linearly. Because the permission path runs concurrently on its own MCU, the safety calculation must reflect that dedicated architecture rather than assuming that every stalled neural forward pass postpones emergency braking. Notebook 8.2 steps through each hardware and software stage of this waterfall for the warehouse mobile manipulator, converting elapsed time into traveled distance.

Napkin Math 8.2: Travel before the frame is read

Problem: The mobile manipulator drives toward a rack end at its drive limit when a person steps out. How old is the navigation camera's observation of that person when perception dispatches it, and how far has the base traveled by then?

Variables:

- Exposure $t_{exp} = 16$ ms (Chosen low-light setting), so the midpoint lag is $t_{exp}/2 = 8$ ms; rolling readout $t_{readout} = 16.6$ ms (Datasheet).
- Residual MIPI CSI-2 transport $t_{transport} = 0.8$ ms (Illustrative). The Sony IMX477 flyer specifies a four-lane option up to 2.1 Gbps/lane; the budget assigns only the transport left after streaming readout, not frame size divided by peak lane rate.
- DMA and driver queue $t_{DMA} = 1.2$ ms, ISP debayering and rectification $t_{ISP} = 2.5$ ms, INT8 vision backbone $t_{backbone} = 22$ ms, and covariance packaging with zero-copy dispatch $t_{IPC} = 0.5$ ms (all Illustrative).
- Drive limit $v = 1.5$ m/s (Illustrative).

Math:

Age at dispatch. The serial stages of equation 8.7 sum to

$$t_{age} = 8ms + 16.6ms + 0.8ms + 1.2ms + 2.5ms + 22ms + 0.5ms = 51.6ms$$

Travel. Holding speed through the age window:

$$\Delta x = (1.5m/s) \times (0.0516s) = 0.0774m = 77.4mm$$

Sensitivity. Each further millisecond in any stage adds 1.5 mm of travel at the drive limit.

Result: The observation is 51.6 ms old at dispatch, and the base has covered 77.4 mm. The backbone is the largest single stage, but exposure and readout alone contribute 24.6 ms before any computation starts.

Systems insight: The base covers this distance before the perception record exists. Timestamped pose propagation can place the old observation correctly in the current frame, but no later stage returns the distance already traveled, so vt_{age} enters the stopping budget as a term of its own.

The seven stages of table 8.3 sum to the observation age at dispatch, $t_{age} = 51.6$ ms. Each stage value is a budgeted allocation, the most that stage may take under load, rather than a typical measurement. The sum is therefore the age the rest of the machine plans against, and the last column of the table names the measurement that must confirm it. The stopping budget then adds the active lease and response path of section 4.4:

$$\tau_{delay} = \underbrace{t_{age}}_{51.6ms} + \underbrace{T_{lease} + T_{tick} + T_{bus} + T_{act}}_{82ms} = 133.6ms$$

This is the pre-brake delay that section 1.4.3 and the event timing table of chapter 4 carried with t_{age} entered as a given; its first term is now derived rather than assumed.

This subtotal is a **partial delay budget** rather than an end-to-end capture-to-response bound. In the timing breakdown of table 4.1, perception governs the first interval from capture to dispatch, while actuator mechanics govern the third interval of physical braking. The observation continues to age across the intermediate second interval while the proposer completes inference, transmits the candidate trajectory, and awaits admission from the permission path. Let Δ_{da} denote this

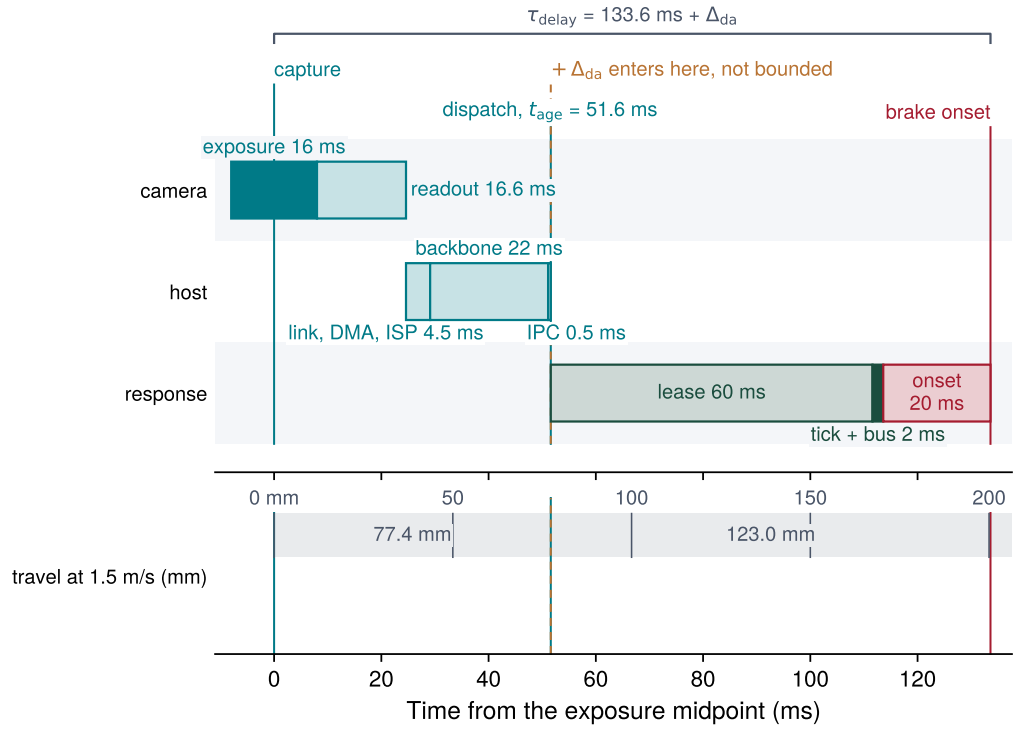


Figure 8.5: Capture-to-brake timing strip: Time runs from the exposure midpoint of the frame that first shows the hazard. The camera and host lanes carry the navigation camera's serial stages, with link, DMA, and ISP drawn as one segment, to dispatch at 51.6 ms; the response lane adds the lease, tick, bus, and actuator onset of a stalled proposer to reach 133.6 ms. The row beneath reads the same axis as base travel at the 1.5 m/s drive limit. The dashed line at dispatch marks where the dispatch-to-admission interval Δ_{da} enters; this chapter does not bound it, so every later event shifts right by its measured value.

dispatch-to-admission interval, encompassing model planning and inter-process communication overheads. Because an active lease takes effect only upon candidate admission, the total delay under a stalled-proposer fault equals the displayed subtotal plus Δ_{da}. The transport-budget calculation in section 13.6.2 allocates a budget for Δ_{da} that awaits empirical validation on hardware. That allocation includes a full inference after dispatch, and part of that inference overlaps the vision backbone already counted in t_{age}. Adding the two therefore counts the overlap twice and overstates the delay, which errs on the safe side; timestamping both intervals on the loaded machine removes the overlap.

Laid on one clock, the partial budget shows where the unbounded interval sits (figure 8.5). Everything before dispatch at 51.6 ms belongs to perception, while everything after it belongs to the response path that reaches brake onset at 133.6 ms. The dispatch-to-admission interval Δ_{da} enters at the dispatch line, so whatever value it takes on the loaded machine shifts every later event to the right.

Because the permission path validates geometric safety without reprocessing raw images, it cannot retroactively compress this perception pipeline delay; the age reaches it strictly through the proposal's stamped evidence epoch. At the 1.5 m/s drive limit, the observation-age term vt_{age} contributes 77.4 mm of blind displacement. That travel is the fifth row of the stopping budget. The first three rows are the Body's terms from equation 2.2, braking distance $v^2/2a_{brake}$, the overhead $\delta_{loc} + \delta_{margin}$, and brake-onset travel vT_{act}, written together below as d_{base}. The fourth row is the lease-path travel $d_{lease} = v(T_{lease} + T_{tick} + T_{bus})$ of section 4.4. Adding the fifth row gives

$$d_{stop} = \underbrace{d_{base} + d_{lease}}_{835.5mm} + \underbrace{v \cdot t_{age}}_{77.4mm} = 912.9mm$$

This additional term raises the partial stopping distance from 835.5 mm to 912.9 mm. Although 187.1 mm remains within the nominal 1.10 m rack-end clearance, that remainder cannot be claimed as safety buffer. It must still absorb the travel during the dispatch-to-admission interval Δ_{da}, the extra distance of a jerk-limited rather than instantaneous brake application (section 11.5.4), and the tracking inset of section 12.3. This analysis sizes the perception subsystem's contribution to the stopping budget and clarifies the empirical measurements required before an integrated motion claim can be certified; the final column of table 8.3 names that measurement stage by stage.

Table 8.3: Serial observation-age budget: The navigation camera's budgeted stage increments, assumed nonoverlapping, sum to its observation age at dispatch. A real pipeline can overlap readout, transfer, and processing, so capture-to-dispatch age requires an end-to-end measurement including queueing tails. Later consumers must add their elapsed post-dispatch interval.

Stage	Increment	Serial age after stage	What to verify
Exposure midpoint	8 ms	8 ms	Exposure and row timestamp convention
Rolling readout	16.6 ms	24.6 ms	Selected row and overlap with streaming
Residual transport	0.8 ms	25.4 ms	Selected sensor mode and host receiver
DMA and queue	1.2 ms	26.6 ms	Arbitration and queue tail
ISP	2.5 ms	29.1 ms	Mode and competing jobs
Vision backbone	22 ms	51.1 ms	Workload, clock, thermal state
Dispatch	0.5 ms	**51.6 ms**	Capture-to-dispatch age and queue tail

Checkpoint 8.1: Observation age and the stopping budget

Observation age is paid in travel before any consumer can react, and no later stage returns it.

- ☐ **Observation age**: Can you explain how the interval from the exposure midpoint to dispatch converts into travel that erodes the stopping clearance before a policy can react?
- ☐ **Exposure midpoint vs. arrival timestamp**: Can you explain why dating a frame at DMA completion rather than at its exposure midpoint pairs the observation with the wrong pose?
- ☐ **Pose propagation from capture time**: Can you explain how propagating the machine's pose forward from an observation's capture time places an old observation correctly in the current frame, and why that propagation cannot return the distance already traveled?
- ☐ **Observation age in the stopping budget**: Can you explain why the observation-age term vt_{age} adds to the stopping distance, and why the permission path, which never reads the image, cannot shorten it?
- ☐ **Dropped-frame estimate**: If a dropped frame leaves a proposal resting on the previous frame, one frame period (16.7 ms) older, how much travel does that add at the 1.5 m/s drive limit, and what share of the 187.1 mm remainder does it consume?

8.4.2 Dating the machine's LiDAR

The permission path's clearance check inspects range returns from the mobile base's Ouster OS1-64 LiDAR, whose timing structure differs from that of the optical camera. A rolling-shutter camera exposes and reads out a discrete frame grid. In contrast, a spinning mechanical LiDAR continuously fires laser diodes and timestamps each azimuth column at the firing instant, disciplined by the IEEE 1588 Precision Time Protocol (PTP). Because every point carries an individual capture epoch, the scan lacks a singular frame time. The age of the resulting rack-end spatial estimate is therefore governed by how long the software pipeline buffers returns before releasing the angular sector facing the hazard.

At the configured scan rate, one mechanical revolution requires 50 ms. At a clear distance of 1.10 m, the 1.30 m aisle subtends a forward azimuth sector of 61.2°, which the sensor sweeps in 8.5 ms. Table 8.4 compares two ingestion architectures for handling this point stream: streaming sector

packets to consumers immediately upon arrival vs. buffering the entire sweep before dispatching a complete 360° point cloud.

Table 8.4: LiDAR age at the rack end: The OS1-64's upper age before host processing under two consumption policies, at the configured scan rate. Data latency and clock accuracy are the datasheet's upper bounds; processing time must be measured on the machine under the synchronized camera and inference load.

Stage	Stream the forward sector	Wait for the full sweep	Source
Acquisition of the oldest relevant point	8.5 ms	50 ms	Scan rate and aisle geometry
Sensor data latency, upper bound	10 ms	10 ms	Datasheet
Clock-conversion bound (PTP)	1 ms	1 ms	Datasheet; measure on the machine
Upper age before processing	**19.5 ms**	**61 ms**	Computed
Travel at the drive limit	29.2 mm	91.5 mm	Computed
De-skew and occupancy extraction	Needs measured value	Needs measured value	Host measurement under camera load

Waiting for a full revolution consumes 61 ms before host computation begins, exceeding the camera's 51.6 ms acquisition/processing allocation. In contrast, streaming the forward azimuth sector immediately leaves 32.1 ms of processing headroom within that allocation for motion de-skew and occupancy extraction. This arithmetic compares pipeline allocations; different scan and frame phases, buffered reuse, and downstream waiting still determine which selected observation is older at consumption. The real-time latency budget therefore dictates the architecture. The robot streams and processes sector packets incrementally, propagating individual point timestamps rather than a single aggregated sweep time.

Two strands of evidence therefore meet at the rack end, and the oldest required capture sets the shared age charge. The proposal is anchored to the camera's frame epoch, while the admission check rests on LiDAR range. The permission path must date both on its own clock at the same admission or tick, including bounded clock uncertainty. Streaming fits the illustrated camera-sized acquisition/processing allocation only when de-skew, occupancy extraction, and any phase, reuse, or waiting allowance required by the selected consumption policy fit the 32.1 ms headroom. Sharing the camera's row-five charge further requires that the actual LiDAR capture be no older than the camera capture supporting the proposal at that common check. Otherwise the budget must charge the older required evidence, with each elapsed interval counted once. The dispatch-to-admission interval remains a separate downstream charge. Neither strand includes the internal latency of the certified safety scanner, because its worst-case response time is a factory-certified parameter accounted for separately in the scanner's own timing budget described in section 12.3.1.

Per-column timestamps also enable continuous motion de-skew. Over a single LiDAR revolution at full speed, the mobile base translates 75 mm, and the state estimator eliminates this distortion by interpolating robot odometry at each return's precise firing timestamp (section 8.2). While the optical camera avoids rotational sweeping delays, its serial latency budget has no spare stage to absorb arbitrary queueing, because row five charges exactly the budgeted age. In particular, collecting consecutive frames into multi-frame batches introduces substantial latency penalties before host memory contention is even considered.

8.4.3 Temporal batching

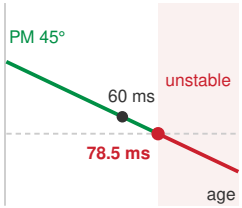
At the navigation camera's 60 Hz sampling rate ($T_{\text{sample}} \approx 16.7$ ms), forming a batch of four consecutive frames to amortize kernel launch overhead or raise systolic-array utilization requires holding the first frame for 50 ms, the second for 33.3 ms, and the third for 16.7 ms, while only the fourth enters immediately. Accumulating frames across successive capture times ages the earliest samples: appending that 50 ms queueing delay to the baseline 51.6 ms observation age roughly doubles total age to 101.6 ms. Because row five of the stopping budget charges only the single-frame age, the observation contract at the end of this chapter sets its dispatch threshold at that age plus a sub-millisecond clock bound (section 8.8). The batched frame is therefore refused before candidate

admission begins. When multi-camera systems synchronize exposure pulses in hardware, grouping frames from distinct sensors across spatial dimensions can improve throughput safely; however, temporal batching across successive timestamps directly erodes closed-loop timeliness.

8.4.4 Observation age and feedback stability

Beyond draining physical stopping distance, observation age also spends the phase margin of any feedback loop closed through perception, the extra lag that loop can absorb before its corrections reinforce the motion they were meant to damp. Section 4.3.1 establishes that a pure delay removes phase in proportion to frequency without changing loop gain. A loop with nominal phase margin $\mathrm{PM}_{\mathrm{nom}}$ at gain crossover frequency ω_c therefore has no margin left once $t_{\mathrm{age}} \geq \mathrm{PM}_{\mathrm{nom}}/\omega_c$. Perception draws on that same account. A visual-servoing tracking loop crosses over at 10 rad/s (approximately 1.6 Hz) with a nominal phase margin of 45° (0.785 rad). With no other delay charged, its nominal zero-margin ceiling for this perception contribution is $t_{\mathrm{age,max}} = \mathrm{PM}_{\mathrm{nom}}/\omega_c = 78.5$ ms. If sensor exposure, transport, and neural inference consume 60 ms, the perception pipeline spends 76.4 percent of the loop's stability allowance before motor commands are even computed. Whether the integrated loop remains adequately damped requires qualification of its complete dynamics and delays.

The perception contribution to both checks spends the same elapsed duration in different currencies. Phase lag erodes closed-loop stability, while displacement consumes forward clearance ($\Delta x_{\mathrm{blind}} = v \cdot t_{\mathrm{age}}$), so each consequence is checked on its own terms and the latency is never counted twice in the stopping budget. Rigorous capture timestamps, qualified reference frames, and explicit error bounds make these physical consequences observable throughout the software stack. Even when zero-copy DMA architectures eliminate intermediate buffer copies during host ingestion, memory controller arbitration and bus contention continue to introduce subtle waiting times.



Delay spends phase margin linearly; at 78.5 ms none remains.

8.5 Ingestion Overhead and Memory Bus Contention

A common architectural pitfall in early physical AI prototypes is connecting multiple high-resolution cameras through standard USB or Ethernet ports and ingesting frames using naive user-space libraries, such as OpenCV's `VideoCapture` or default ROS image transport nodes. In this unoptimized baseline, every incoming frame is copied from kernel USB request blocks into a driver ring buffer, copied a second time into user-space process memory, and copied a third time across the PCIe bus into GPU memory. For four 1080p cameras streaming 24-bit color at 60 Hz, each copy stage moves 1.5 GB/s, meaning the pipeline churns through about 4.5 GB/s of continuous memory copies. This overhead generates substantial memory traffic and CPU copying work, though the exact utilization depends on host core capabilities, memory architecture, and pixel format. That data rate alone does not establish CPU saturation or accelerator starvation, but it wastes memory bus bandwidth and cache capacity that concurrent real-time inference urgently requires.

Real-time physical systems avoid this collapse by treating pixel ingestion as a zero-copy DMA hardware pipeline. Raw pixel streams arriving over dedicated camera buses bypass CPU copying entirely, writing frames directly into shared physical DRAM before any software thread touches the data. On a shared application processor, however, these camera writes still contend for memory service before inference can read them, and unmanaged bursts can delay host reads or overrun a receiver FIFO and drop a frame.

8.5.1 MIPI CSI-2 DMA ingress and zero-copy buffers

Once analog-to-digital converters on the sensor die digitize the accumulated pixel charges, the frame must cross a physical link into host memory without stalling the CPU on byte-by-byte copies or dropping scanlines when a receiver buffer fills. Camera streams enter the application processor over the MIPI CSI-2 bus.[9] Across that link, dedicated receiver logic and scatter-gather DMA engines move serialized pixel packets from the physical lanes into kernel ring buffers in four stages.

[9] **MIPI CSI-2 (Mobile Industry Processor Interface, Camera Serial Interface 2)**: The MIPI Alliance's point-to-point camera link streams serialized pixels from the sensor straight to an application processor's receiver and DMA controller, over differential D-PHY lanes or three-wire C-PHY trios, with no network stack between them. That directness is why the camera's transport term in table 8.3 is set by sensor mode and lane rate rather than by protocol processing.

Systems Perspective 8.2: Silicon ingestion dataflow architecture

1. **D-PHY / C-PHY physical layer**: Physical lanes serialize pixel packets at the selected sensor and receiver's supported rate; packet headers carry error checks.
2. **System-on-chip (SoC) CSI-2 host controller**: Deserializes byte packets, strips headers, and validates CRC integrity.
3. **Scatter-gather DMA controller**: A dedicated DMA engine transfers pixel byte words directly into DRAM over the internal system interconnect (for example, an AXI crossbar).
4. **Kernel DMA ring buffers**: A Linux `v4l2` driver can map capture buffers into user space.

Once DMA write completion occurs, the kernel capture driver (V4L2, the Linux video-capture interface) issues an interrupt. To eliminate user-space copies, modern embedded Linux systems export DMA buffers directly through the Linux `dma_buf` subsystem. Compared with a staged three-copy variant (Kernel DMA buffer → User-space buffer → Pinned host memory → GPU VRAM), `dma_buf` allows the hardware Image Signal Processor (ISP), CPU, and GPU to import and share identical physical DRAM pages through system IOMMU[10] page tables. A zero-copy buffer handoff eliminates bulk CPU copying, but DMA arbitration, synchronization, queueing, and consumer scheduling still contribute to ingestion latency. Format conversion, layout changes, or device alignment constraints can still require a targeted transform, but the bulk pixel payload never traverses the CPU cache hierarchy. On a platform without hardware I/O cache coherency, the buffer handoff must explicitly synchronize ownership for the intended consumer. CPU access requires the platform's DMA-to-CPU cache maintenance primitives; accelerator access requires corresponding device mapping, completion ordering, and device-side cache invalidation. Flushing or invalidating the CPU cache alone does not guarantee that an independent neural accelerator will read current pixel values. These synchronization and cache maintenance operations belong directly in the measured ingestion budget.

[10] **IOMMU (Input-Output Memory Management Unit)**: The device-side counterpart of the CPU's MMU. It translates the addresses a DMA engine issues through page tables the kernel controls, so the ISP, CPU, and GPU can share one set of physical pages without copying while each device stays confined to its own mappings. A translation miss on that path is one more source of ingestion delay that the measured budget must include.

The copy path in figure 8.6 uses pinned host staging rather than the USB driver ring in the earlier baseline. The shared path separates physical storage from device mappings and marks the release that permits the capture engine to reuse that storage.

A frame identifier therefore cannot establish that all consumers read the same physical pages. A conversion preserves the observation's identity while changing its representation, and the capture driver must track the lifetime of the source pages independently of the converted output.

Interface selection balances cable reach and mechanical routing flexibility against transport, queueing, and copying overhead. MIPI CSI-2 supports direct sensor ingress to the SoC. Serializer/deserializer links extend that path across the machine over multi-meter automotive wiring. USB introduces a host protocol stack and driver buffering whose timing penalty must be measured empirically. The stage breakdown in table 8.3 locates those costs for the selected MIPI path. Any alternative interface requires its own measured stage budget, because an interface specification alone does not determine end-to-end latency. Sensor operating modes, inference workloads, and host memory contention must all mirror the intended deployment conditions.

8.5.2 Memory bus contention and observation age

Even when camera frames sit comfortably within peak DRAM bandwidth when averaged over a one-second window, synchronized camera DMA bursts can still stall neural accelerator reads for milliseconds and inflate the tail of observation age. The raw bandwidth requirements of the mobile manipulator's navigation and wrist cameras are sized in section 5.6.2. When the perception stack reads those frames, transforms intermediate buffers, and the accelerator fetches model weights and writes activations, the memory controller carries several times that ingress rate, yet average utilization hides the microbursts that stretch individual read transactions. On edge robotics SoCs such as the NVIDIA Jetson AGX Orin, the CPU cluster, deep learning accelerator (DLA), GPU, and CSI-2 DMA engines share a single physical DRAM bus in a Unified Memory Architecture (UMA). Synchronized cameras concentrate their DMA writes into sharp microbursts, saturating memory channels and triggering widespread DRAM bank conflicts. When multiple cameras dump

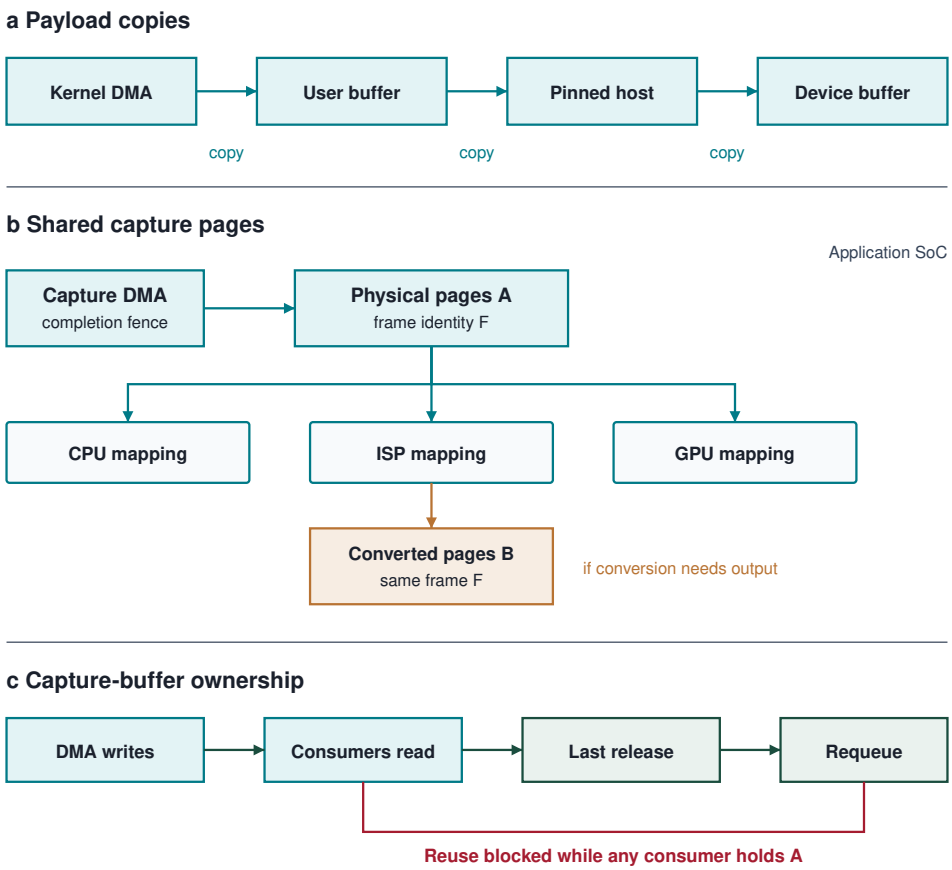


Figure 8.6: Shared Capture Buffers and Safe Reuse: The copy path allocates successive payload buffers. The shared path gives several devices mappings to capture pages A; an ISP conversion may instead write output pages B while preserving frame identity F. Capture pages become reusable only after DMA completion, the required ordering and cache maintenance, and release by their last consumer. The ownership strip shows a lifecycle, not measured durations or simultaneous device execution.

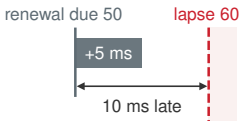
uncached raster lines across the system crossbar simultaneously, they repeatedly open and close DRAM page buffers across active banks. If the neural accelerator attempts to stream transformer model weights and intermediate activation tensors during this exact window, memory controller arbitration stalls accelerator read requests behind high-priority sensor DMA streams. These memory read stalls introduce pipeline bubbles into hardware tensor cores, stretching neural inference latency and inflating its tail (p99 jitter). The magnitude of this latency stretch depends on the platform, its interconnect arbitration policy, and the concurrent camera load. Rather than assuming nominal inference latency, engineers must measure tail stretch directly on physical hardware under the synchronized camera workload; the safety lease below constrains the loaded end-to-end renewal interval.

On the mobile manipulator, this burst lands only on the application processor. Because the permission path executes on its own microcontroller and reads its state from isolated SRAM, a memory burst cannot stretch its safety tick; rejecting a shared-die placement removed that hazard. Contention on the application processor reaches the permission path only through two observable timing channels: the age of the frame used to construct an action proposal, and the lateness of the renewal message sent by the chunk policy. These two paths have fundamentally different timing allowances. A frame that waits behind camera and accelerator traffic reaches dispatch older than

↪ Contention for the shared memory system, and how it would stall the permission tick on a shared-die placement, are developed in section 13.3.

the budgeted stages. The observation contract of section 8.8 sets the dispatch threshold at exactly the budgeted age, because a frame any older would spend travel that row five does not charge. The permission path therefore refuses every proposal built on that delayed frame.

A chunk policy whose inference stalls behind that same memory traffic renews the execution lease late. Under the coupled budget rule of section 4.9.1, the lease reserves 5 ms of renewal jitter above the 50 ms chunk period. The ratio to the chunk policy's 40 ms inference is 12.5 percent, a conditional sensitivity when incremental inference delay reaches admission one-for-one. The actual allowance constrains inter-admission jitter, so the inference stretch it tolerates depends on the renewal schedule and the scheduling, transmission, validation, and mailbox delays sharing that allowance. Qualification must measure the loaded end-to-end inter-admission interval. A renewal that arrives 10 ms late causes the lease to lapse, triggering a deterministic stop. When physical isolation, frame-age validation, and feasible fallback maneuvers remain intact, excess contention compromises availability rather than safety, because the permission path refuses any action delayed beyond the qualified budget. That guarantee holds only so long as those architectural premises and uncorrupted local sensor evidence remain intact.



A renewal may run 5 ms late; at 10 ms late the lease lapses.

The duration of a bus-saturating burst also threatens the camera interface itself. Ingest engines rely on direct memory access ring buffers with double or triple buffering to decouple hardware arrival interrupts from consumer reading threads. If host memory bus contention delays DMA completion beyond what the CSI-2 receiver hardware FIFO can buffer, the receiver suffers an overrun and silently drops the frame. A dropped frame immediately doubles the effective observation interval between samples. Downstream state estimators must treat the missing update as an age or sequence fault rather than assuming that fresh scene geometry will arrive on schedule.

The underlying microarchitectural mechanisms governing frame delivery (DMA rings, IOMMU mappings, and memory controller arbitration) are treated systematically by Hennessy and Patterson (2019) and Bryant and O'Hallaron (2015). For physical AI systems, the main design metric is capture-to-consumer age and its tail distribution under the intended camera workload. Delivering an on-time frame establishes only when the physical scene was sampled, however. Determining where objects reside within that frame, and bounding the uncertainty of that spatial estimate, introduces geometric and calibration errors that consume the remainder of the safety clearance envelope.

8.6 Spatial Error and Clearance Bounds

If perception places a warehouse rack upright just a few centimeters farther from the base's trajectory than it actually stands, the motion planner consumes those missing centimeters directly from its 0.15 m side clearance, although no stage of the pipeline decided to spend them. Once downstream control logic uses an estimated coordinate to authorize motion, every spatial error becomes a physical displacement drawn from the clearance that protects the robot from collision. Transforming a raw sensor observation into a trustworthy clearance claim requires **metric grounding**, which maps detector measurements into Euclidean space. This grounding relies on unprojecting two-dimensional sensor measurements into three dimensions, an inverse problem that assumes calibrated optics, rigid structural mountings, and predictable surface properties.

Classical projective geometry and triangulation (Hartley and Zisserman 2003; Szeliski 2022) govern how measurement noise converts into three-dimensional spatial displacement. In two-view streaming perception, two calibrated cameras with optical centers $\mathbf{o}_1, \mathbf{o}_2$ observe a physical scene point $\mathbf{p} \in \mathbb{R}^3$. The baseline vector $\mathbf{t} \in \mathbb{R}^3$ connecting $\mathbf{o}_1$ and $\mathbf{o}_2$, together with the target point $\mathbf{p}$, defines the **epipolar plane**. The intersection of this baseline with each image plane defines the two **epipoles** $\mathbf{e}_1, \mathbf{e}_2$.

For corresponding image coordinates $\tilde{\mathbf{m}}_1, \tilde{\mathbf{m}}_2$ expressed in homogeneous pixel coordinates, the rigid spatial transformation $(\mathbf{R}, \mathbf{t}) \in \mathrm{SE}(3)$ relating the camera frames establishes the fundamental **epipolar constraint**:

$$\tilde{\mathbf{m}}_2^\top \mathbf{F} \tilde{\mathbf{m}}_1 = 0$$

where $\mathbf{F} = \mathbf{K}_2^{-\top} \mathbf{E} \mathbf{K}_1^{-1} \in \mathbb{R}^{3\times3}$ is the **Fundamental matrix**, and $\mathbf{E} = [\mathbf{t}]_\times \mathbf{R} \in \mathbb{R}^{3\times3}$ is the **Essential matrix** (with $[\mathbf{t}]_\times$ denoting the skew-symmetric matrix of baseline translation). Geometrically, because the two unprojected bearing rays $\mathbf{K}_1^{-1}\tilde{\mathbf{m}}_1$ and $\mathbf{K}_2^{-1}\tilde{\mathbf{m}}_2$ meet at the same physical point, both rays and the baseline vector $\mathbf{t}$ lie in the epipolar plane, so expressing the first ray in the second camera's frame via $\mathbf{R}$ and taking its cross product with $\mathbf{t}$ (written as multiplication by $[\mathbf{t}]_\times$) produces the plane normal

that must be orthogonal to the second ray. The vector $\mathbf{l}_2 = \mathbf{F}\tilde{\mathbf{m}}_1$ defines the **epipolar line** in the second image; any valid physical correspondence $\tilde{\mathbf{m}}_2$ must lie strictly along $\mathbf{l}_2$.

In a calibrated **rectified stereo rig**, both images are digitally warped onto a shared virtual image plane parallel to the baseline ($\mathbf{R} = \mathbf{I}$, $\mathbf{t} = [b, 0, 0]^\top$). Rectification forces epipolar lines to be exactly parallel and collinear with horizontal pixel scanlines ($v_1 = v_2$). This geometric transformation collapses an expensive two-dimensional correspondence search, the kind dense optical-flow networks perform, into a one-dimensional search along the same horizontal scanline. Neural stereo networks such as RAFT-Stereo evaluate this correspondence as a 1D correlation cost volume (Lipson et al. 2021). Disparity is then the horizontal coordinate difference $d = u_1 - u_2$, recovering metric depth through $Z_c = \frac{fb}{d}$.

When mechanical vibration, chassis twist, or thermal expansion induces uncalibrated relative pitch or roll between cameras, or when rolling-shutter readout shears scanlines during sharp platform turns, the true physical correspondence shifts vertically off the horizontal epipolar line ($\Delta v \neq 0$). The 1D correlation cost volume fails or latches onto spurious texture matches across adjacent rows, injecting severe triangulation outliers into the reconstruction. Differentiating the unprojection $Z_c = \frac{fb}{d}$ with respect to disparity yields a negative signed derivative whose magnitude defines the small-error relation (equation 8.8):

$$|\delta Z_c| \approx \frac{Z_c^2}{fb}|\delta d| \tag{8.8}$$

where δd represents disparity error. If δd denotes a disparity standard deviation σ_d, first-order error propagation gives $\sigma_z \approx \frac{Z_c^2}{fb}\sigma_d$. Doubling range quadruples this modeled standard deviation at fixed focal length f, baseline b, and disparity precision σ_d. For distant landmarks, an estimator instead parameterizes depth as inverse depth[11] $\rho = 1/Z_c = d/(fb)$. Differentiating inverse depth with respect to disparity yields:

$$\frac{\partial \rho}{\partial d} = \frac{1}{fb} = \text{constant}$$

Because this derivative is independent of distance, measurement errors in disparity map linearly into inverse depth ($\delta\rho = \frac{1}{fb}\delta d$). This linear mapping provides **inverse-depth metric grounding**, preserving numerical conditioning for distant landmarks when the disparity-error model is appropriate. Inverse depth does not eliminate mismatch outliers or guarantee estimator stability. In contrast, a pulsed time-of-flight sensor estimates range directly from transit time, $R = \frac{c\Delta t}{2}$; its range error depends on its timing and detection electronics, remaining approximately flat across its rated operating region.

On the running machine, the rack-end range comes from the OS1-64 LiDAR, whose datasheet quotes one-standard-deviation precision of 5 mm to 30 mm beyond 1 m in the 1024×10 Hz mode. Applying that scale to the selected scan mode requires bias and residual-tail qualification under the intended surfaces and motion. A design that tried to range with cameras alone would have to triangulate, because the navigation camera by itself recovers only the bearing ray of equation 8.3. An illustrative stereo pair shows where such a design meets a range wall, by asking how far down the aisle its depth error stays inside the stopping budget's allowances. Notebook 8.3 evaluates equation 8.8 on that rig, and figure 8.7 plots the curve.

11 | **Inverse-Depth Parametrization**: Civera et al. (2008) introduced it for monocular SLAM, where a newly seen landmark may lie arbitrarily far away. Storing $\rho = 1/Z_c$ lets a point at infinity enter the estimate as $\rho = 0$ with finite variance, where a depth coordinate would need an infinite one, so near and distant landmarks share one well-conditioned representation. The full parameterization is given in section A.3.

Napkin Math 8.3: The stereo range wall

Problem: An illustrative stereo pair must place a rack upright for the base's clearance checks. How large is its one-standard-deviation depth scale at the rack end, and at what range does that scale alone consume the localization allowance and then the whole side clearance?

Variables:

- Baseline $b = 0.20$ m, focal length $f = 800$ px, and disparity standard deviation $\sigma_d = 0.5$ px over the tested range (Illustrative rig; the running machine carries no depth camera).
- Rack-end range $Z_c = 1.10$ m (Illustrative).

- Allowances C: localization $\delta_{\text{loc}} = 50$ mm and side clearance 0.15 m (Illustrative; section C.14.1).

Math:
Depth scale. At the rack end, and between the figure's two quoted ranges,

$$\sigma_z = \frac{z^2 \sigma_d}{fb} = \frac{(1.10\ \text{m})^2 \times 0.5\ \text{px}}{800\ \text{px} \times 0.20\ \text{m}} = 3.8\ \text{mm}, \quad \frac{\sigma_z(10\ \text{m})}{\sigma_z(2\ \text{m})} = \frac{312.5}{12.5} = 25$$

Crossover ranges. Setting σ_z equal to an allowance C and solving for range,

$$z^* = \sqrt{\frac{C\, fb}{\sigma_d}} : \quad \sqrt{\frac{0.05\ \text{m} \times 160\ \text{px} \cdot \text{m}}{0.5\ \text{px}}} = 4\ \text{m}, \quad \sqrt{\frac{0.15\ \text{m} \times 160\ \text{px} \cdot \text{m}}{0.5\ \text{px}}} = 6.9\ \text{m}$$

Sensitivity. Five times the range costs twenty-five times the depth scale, so the crossovers move only with the square root of fb/σ_d.

Result: At the rack end, one standard deviation is 3.8 mm. It equals δ_{loc} at 4 m and the whole side clearance at 6.9 m.

Systems insight: Stereo depth error is a range budget. This rig's nominal depth scale fits the rack-end allowance, but supporting that decision still requires the tail, bias, and timing qualification developed below. A claim about an obstacle several meters down the aisle spends the stopping budget's δ_{loc} on one standard deviation alone. Beyond a few meters, the clearance claim must therefore come from a range sensor whose error does not grow with Z_c^2, or the speed must fall.

The values in Notebook 8.3 are nominal **one-standard-deviation scales**, not clearance insets or certified 99th-percentile bounds. A usable safety inset requires an empirical tail quantile of the *full spatial residual*, calibrated on the intended sensor across realistic range, surface texture, platform motion, and ambient lighting conditions, with systematic bias and missed detections handled separately. A pervasive trap in visual estimation is assuming that measurement errors follow a nominal Gaussian distribution where a 3σ gate bounds 99.7 percent of all errors. In physical vision, disparity estimation relies on matching pixel appearance across views. When scenes contain repetitive textures (such as corrugated warehouse siding), specular reflections from polished concrete floors, or partial occlusions along object silhouettes, correspondence matching produces catastrophic multimodal outliers rather than Gaussian scatter. A fraction of stereo returns are then gross mismatch outliers lying many standard deviations from the true surface. Because that outlier fraction depends on scene texture, lighting, and the matching architecture, it must be measured directly on residuals from operational deployments rather than assumed. Relying on Gaussian 3σ bounds without robust loss formulations (such as Huber or Tukey loss) or heavy-tailed empirical quantile validation allows these gross outliers to distort the state estimate, directly compromising physical clearance. When validated range-error bounds exceed stopping margins, the vehicle must reduce speed or refuse the motion proposal entirely. Even when optical depth error is bounded in the camera's own frame, however, the obstacle's position must still be mapped across the robot's physical joints and mounts into the frame of the moving body.

8.6.1 Clearance insets and transform chain error

Before an estimated point can govern motion, it must propagate through kinematic coordinate transformations into the robot base frame and end effector. The mechanics of rigid body transformations and SE(3) kinematic frame composition follow classical treatments by Craig (Craig 2005) and Lynch and Park (2017). Each link in this transform chain contributes scale errors, mounting calibration drift, joint encoder quantization, and mechanical compliance under load. As demonstrated by the reference-frame principles of section 8.3, angular misalignment at the sensor mount amplifies linearly with range. If the navigation camera's mount shifts by $\theta_{\text{ext}} = 0.005$ rad against the chassis, the lateral error $\delta x \approx Z_c \theta_{\text{ext}}$ at $Z_c = 10$ m reaches 50 mm. The shift is lateral, so on the approach to

the rack end it is charged against the side clearance, as the readout notebook charged lateral skew. Its size is the point, because it is as large as the entire 50 mm allowance δ_{loc} that the stopping budget holds for localization along the direction of travel.

The comparison therefore concerns error scale rather than which allowance pays. The localization allowance carries small, validated residuals, such as the sub-millimeter travel during the clock-conversion bound that the observation contract absorbs (section 8.8). An extrinsic shift that consumes that entire budget represents a mechanical calibration failure; downstream consumers must detect and refuse such observations via calibration validity checks rather than absorb them. When combined with quadratic stereo depth dispersion, the total spatial error vector $\epsilon_{\text{spatial}}$ at standoff range is anisotropic, dominated by longitudinal depth along the optical axis and angular lever-arm effects orthogonal to it.

Every physical safety barrier operates by defending a geometric boundary around the machine envelope. For a spatial position error vector $\epsilon_{\text{spatial}}$, the triangle inequality gives a conservative relation between nominal and true clearance:

↪ How the localization allowance enters the base's kinetic barrier certificate is developed in section 12.5.1.

$$C_{\text{true}} \geq C_{\text{nominal}} - \|\epsilon_{\text{spatial}}\|$$

To ensure the true obstacle remains beyond the required safe physical clearance C_{safe} (such as the base's side clearance), the permission path enforces a clearance inset C_{inset} (equation 8.9):

$$C_{\text{inset}} = C_{\text{safe}} + \|\epsilon_{\text{spatial}}\|_{\text{p99}} \tag{8.9}$$

Here $\|\epsilon_{\text{spatial}}\|_{\text{p99}}$ is an empirically certified 99th-percentile quantile of the full spatial residual norm, incorporating optical noise, kinematic propagation, and uncalibrated mounting bias. The tempting shortcut replaces that quantile with one standard deviation from the stereo notebook. For a required clearance of $C_{\text{safe}} = 150$ mm, it would hold an inset of 162.5 mm at 2 m or 462.5 mm at 10 m. Neither number covers the Gaussian tail beyond one standard deviation or the mismatch outliers described earlier. The measured quantile targets 99 percent coverage under the declared calibration assumptions; it does not provide a hard bound on the remaining residual tail. Closing that residual tail requires a justified hard bound on the sensing error, a feasible stopping trajectory, and a safety filter that responds within the remaining latency margin.

Definition 8.2: Clearance inset

Clearance inset (C_{inset}) is the separation a consumer holds between an estimated obstacle position and the machine's envelope so that the true obstacle stays at least the required physical clearance away, except with a stated tail probability measured under the conditions of use.

1. **Significance**: The inset converts measured spatial error into clearance that the motion gives up (equation 8.9). Every millimeter of qualified error widens the region the permission path treats as occupied, so the planner must slow or route farther away. On the illustrative stereo rig, one standard deviation of depth error alone equals the base's 0.15 m side clearance at 6.9 m, so there the inset would double the clearance even if the residual tail were no wider than one standard deviation.
2. **Distinction**: A covariance describes the second moments of the error an estimator believes it has; the inset rests on residuals measured against ground truth, so it includes the bias and heavy tail that the covariance omits. A 99th-percentile inset is also not a hard bound. It leaves 1 percent of residuals outside, whose containment requires a justified sensing-error bound, a feasible response under that bound, and a timely independent permission path. A stop calculated from the same unsupported observation does not independently cover its residual tail.
3. **Common pitfall**: A frequent misconception is that a reported covariance, or a depth network's predicted spread, sets the inset. Neither is measured against ground truth, so neither contains the calibration bias and mismatch outliers that the residual campaign of section 8.6.2 captures.

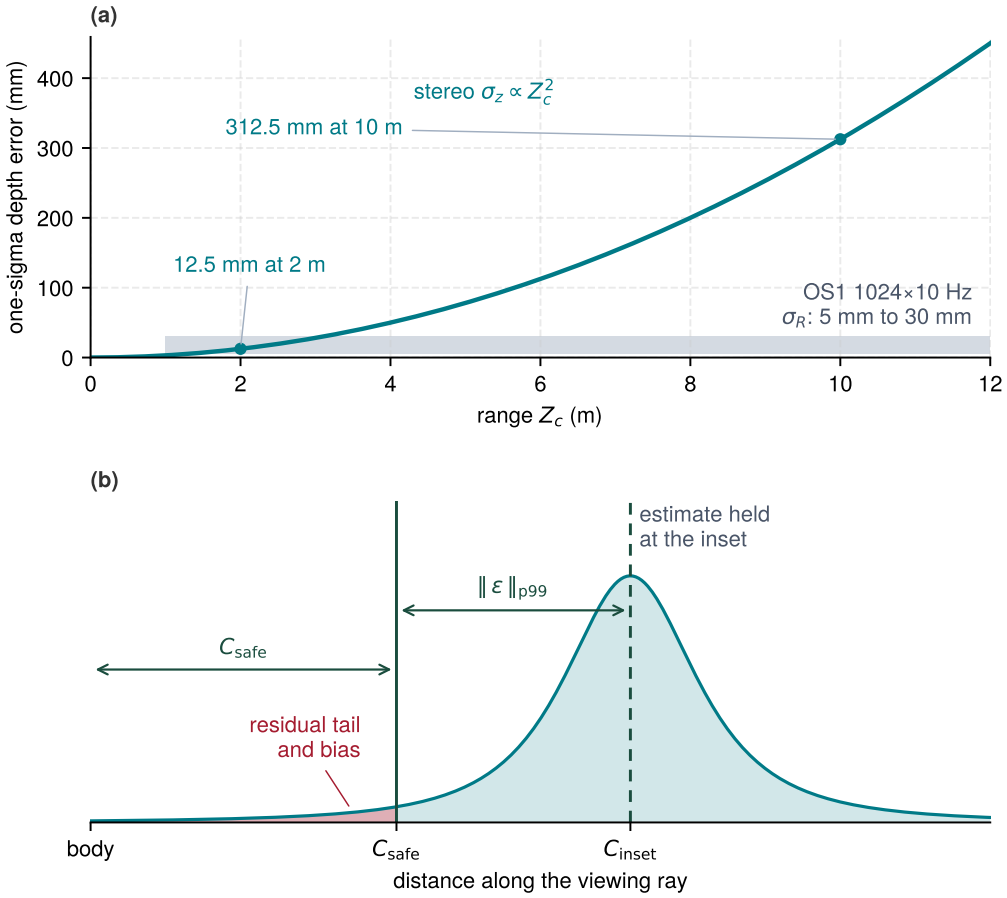


Figure 8.7: Depth error scale and clearance inset: (a) For the illustrative stereo rig, the one-standard-deviation depth scale grows with squared range, from 12.5 mm at 2 m to 312.5 mm at 10 m; the gray band is the OS1-64 datasheet range precision beyond 1 m in the specified 1024×10 Hz mode. Its transfer to the selected operating mode requires residual qualification. (b) A schematic error distribution along the viewing ray. Holding the estimate at $C_{\text{inset}} = C_{\text{safe}} + |\epsilon_{\text{spatial}}|_{\text{p99}}$ keeps the true surface beyond C_{safe} except in the shaded residual tail, which the quantile leaves and which bias can enlarge.

The two halves of the argument sit side by side in figure 8.7. In the upper panel, the stereo depth scale climbs quadratically past the LiDAR's flat datasheet precision for the specified 1024×10 Hz mode. In the lower panel, the inset holds the estimate far enough out that only the measured residual tail, which unmodeled bias can enlarge, falls inside C_{safe}.

Because clearance depends on propagated spatial error, downstream motion planners and safety arbitration layers cannot consume raw coordinate tuples. A covariance matrix $\Sigma_{\text{body}} \in \mathbb{R}^{3\times3}$ alone cannot determine whether a 10 mm or 500 mm clearance inset is justified; establishing that buffer requires an independently validated tail quantile or hard bound. When spatial error is qualified, the permission path can expand the clearance inset, trading speed or workspace for separation. Downstream deterministic safety filters, the per-tick checks that keep the machine inside a region from which it can still stop, hold it outside this expanded boundary C_{inset} and so preserve the clearance buffer against sensing and calibration error. Environmental hazards such as a textureless wall, solar glare, or lost calibration can invalidate that qualification without making a numerical covariance matrix literally infinite. The permission path must then refuse motion proposals that depend on the affected claim and select a feasible fallback.

↪ Safety filters, and the control barrier functions that formalize them, are developed in section 12.5.

On every frame, the producer builds a spatial claim in a fixed order, where any failed check halts the pipeline. It first converts the capture timestamp to the consumer's clock with a bounded

synchronization error and computes the observation age t_{age} at dispatch. If age plus clock uncertainty exceeds the dispatch threshold, it marks the observation stale, and proposals that need it are refused. At admission and across subsequent ticks, the permission path computes the current evidence age relative to capture time and compares it with a separate, later runtime staleness threshold (section 8.8). For a fresh observation, the producer unprojects each pixel measurement into optical coordinates with a sensor covariance.[12] It then carries point and covariance into the body frame by first-order SE(3) error propagation through the calibrated extrinsics at capture. A consumer that predicts forward must attach its motion-model uncertainty without altering the evidence epoch. Validated residuals, including bias, then set the inset radius C_{inset}, and when no valid bound remains, the affected motion is refused. What leaves perception is an error-qualified claim $(\mathbf{p}_{\text{body}}, \Sigma_{\text{body}}, C_{\text{inset}}, t_{\text{age}})$, never a bare coordinate.

That claim describes one measurement at one instant, and perception's responsibility ends with it. Keeping the rack-end range current between sweeps, fusing the LiDAR with a second sensor, joining detections into tracks, and applying a late measurement at its capture epoch are estimation over time. Chapter 9 takes them up, beginning with state estimation in section 9.2, and every one of them consumes the dated, framed, error-qualified claim built here. What perception must still supply is evidence that the claimed error was measured.

[12] **Covariance Propagation**: First-order propagation maps measurement covariance to $\Sigma_c = \mathbf{J}_{\text{unproj}} \Sigma_{\text{meas}} \mathbf{J}_{\text{unproj}}^\top$ in the optical frame and then to $\Sigma_{\text{body}} = \mathbf{R}_{bc} \Sigma_c \mathbf{R}_{bc}^\top + \Sigma_{\text{ext}}$ in the body frame, where the extrinsic term is what a mount shift adds. Its largest eigenvalue gives the principal variance scale, which for stereo points lies along the viewing ray; section C.12 derives the full propagation.

8.6.2 Qualifying the claimed error

Plugging a datasheet precision value or an unmeasured network clock offset into a clearance check leaves systematic mounting bias, heavy-tailed mismatch outliers, and asymmetric path delays uncovered. A physical obstacle can then sit inside the safety boundary while the permission check passes. Two numbers in the claim just built therefore cannot come from a model or a datasheet: the tail quantile that sets the clearance inset, and the clock-conversion bound that dates the claim on the permission path's clock. Each is fixed by a measurement campaign, and the observation contract of section 8.8 carries both. Whether a reported covariance matches empirical error is evaluated with consistency statistics that require an estimator's prediction; that test is developed alongside the estimator (section 9.2.1).

The clearance inset of equation 8.9 needs a 99th-percentile bound on the residual norm under intended operational conditions, not a theoretical covariance. The most direct procedure is empirical. Collect n_{cal} residual norms against ground truth, sort them, and select the k_ϵ-th smallest with $k_\epsilon = \lceil (n_{\text{cal}} + 1)(1 - \epsilon) \rceil$. This split-conformal rule guarantees that a future residual falls at or below the bound with probability at least $1 - \epsilon$, provided calibration and deployment residuals are exchangeable (Angelopoulos and Bates 2023). It is the inductive conformal construction that section 3.6.2 used to turn a policy's dispersion into a coverage guarantee, applied here to measured residual norms rather than to model scores. For $\epsilon = 1$ percent, the rule requires at least 99 residuals, returning the largest observed value when evaluated on that minimum set. With 1,000 residuals, it returns the 991st smallest value (the 10th largest). This guarantee holds marginally over the calibration distribution. A new floor finish, a lighting change, or a mechanical recalibration breaks exchangeability, requiring new residual campaigns sized by the per-claim sample-size rule of section 7.7.1. Because these residuals are measured directly against ground truth, they capture the sensor's physical bias, which nominal covariance models omit.

The clock-conversion bound ϵ_{sync} is likewise measured rather than assumed. The IEEE 1588 Precision Time Protocol (PTP) exchange that section 5.5.2.1 used to pair logged frames with actions estimates a device's clock offset by assuming that both directions of the path take equal time (IEEE 2020a). Section C.7 gives the message exchange and its servo. A path asymmetry Δt_{asym} shifts the estimated offset by $\Delta t_{\text{asym}}/2$ and leaves no symptom in the exchange. The exchange must therefore be timestamped in hardware at the network interface, where host scheduling cannot add delay to one direction and not the other. Between synchronization exchanges, the local oscillator drifts. The OS1-64's free-running clock is specified at 10 ppm (Ouster, Inc. 2025), which over a 1 s sync interval accumulates 10 µs of worst-case drift. Clock qualification records the measured tail of the servo's reported offset, the path-asymmetry allowance, and the drift between corrections. A sampled tail is statistical evidence, not a maximum-error guarantee. A contract that treats synchronization error as a hard bound also needs a justified limit on unobserved offset excursions and a detector that invalidates the mapping when its assumptions fail. For a hardware-triggered camera, this

bound is verified by timing the trigger edge against the IMU latch with a logic analyzer. Until those measurements exist, the clock-conversion bound in the navigation camera's record remains illustrative, and the LiDAR's datasheet guarantees only 1 ms. While empirical residual campaigns can qualify a stereo or LiDAR sensor whose physics directly measures depth, modern perception pipelines increasingly pair those metric sensors with neural networks that infer depth from a single image.

8.6.3 Learned depth as a measurement

A monocular depth network can predict plausible-looking meter values that err by meters on an unfamiliar floor texture or a life-sized photograph printed on a flat box, because a single camera physically measures only bearing and guesses scale from learned appearance priors. Learned models add two capabilities to metric perception: a semantic classification of spatial regions, and a depth estimate from a single camera. Lifting labeled detections into a persistent, object-centric scene graph carries state across frames, and section 9.6.2 develops the maintenance of that graph. Perception must settle a narrower systems question: whether a learned depth estimate may enter a safety clearance claim at all. The semantic layer identifies candidate objects and contacts; the metric layer must establish their physical geometry. Point clouds supply measured surface samples where pulses reflect, but missing returns never prove free space. Persistent representations that retain geometric evidence cannot alter an observation's capture timestamp or convert a learned model score into an error bound.

↪ The volumetric maps that retain this geometric evidence across frames are developed in section 9.6.1.

Lifting 2D pixel features into 3D metric space requires combining 2D token embeddings with per-pixel depth distributions. In deep monocular depth networks (such as Depth Anything, MiDaS, or metric depth transformers), models predict dense depth maps from monocular visual cues. However, single-view depth estimation is inherently ill-posed due to projective scale ambiguity: an infinite family of physical 3D objects with different scales and distances project to identical pixel arrays. Relative-depth models output scale- or affine-ambiguous predictions, whose conversion to metric depth requires external calibration, represented by $z_{\text{metric}} = s \cdot z_{\text{pred}} + t$ when the model uses that parameterization. Metric-trained heads directly predict depth in meters, but their units do not certify their error. Both model classes require residual validation and can fail silently on unusual geometries, novel textures, or optical illusions (such as a 2D photograph printed on a flat surface).

In bird's-eye-view (BEV) architectures such as Lift-Splat-Shoot (Philion and Fidler 2020), the network predicts a categorical probability distribution over discrete depth bins along each viewing ray. Each 2D feature token is cast outward along its optical sightline ray and 'splatted' into discrete 3D spatial voxel cells weighted by these predicted depth probabilities, transforming disembodied image embeddings into an explicit 3D volumetric representation. The resulting spatial uncertainty ellipsoid deforms non-linearly under perspective unprojection through the unprojection Jacobian $\mathbf{J}_{\text{unproj}} = \frac{\partial \mathbf{p}_c}{\partial [u,v,Z_c]^\top}$ (equation 8.10):

$$\Sigma_c = \mathbf{J}_{\text{unproj}} \Sigma_{\text{meas}} \mathbf{J}_{\text{unproj}}^\top \tag{8.10}$$

While transverse spatial standard deviation expands linearly with range ($\sigma_\perp \approx \frac{Z_c}{f}\sigma_u$), stereo radial depth standard deviation expands quadratically ($\sigma_z \approx \frac{Z_c^2}{fb}\sigma_d$). This perspective unprojection transforms near-field circular uncertainty disks into highly eccentric, needle-shaped ellipsoids oriented along the camera's sightline, with extreme radial dispersion at long ranges that must be bounded before downstream safety filters can certify clearance.

↪ The lift-splat projection of camera features into a metric bird's-eye-view grid is developed in section A.3.3.

Because the depth-bin distributions produced by learned lifting stages are uncalibrated model scores rather than certified metric error bounds, the clearance inset of equation 8.9 cannot rest on learned depth alone. Downstream safety enforcers require continuous validation against calibrated active ranging sensors or stereo disparity. This is the division of labor on the running machine. A learned head on the navigation camera may classify the worker and even estimate depth, but the rack-end range that enters the clearance check comes from the LiDAR. The split also keeps the check independent of the proposal it judges, because a checker that consumed the camera's own perception would share every blind spot of the proposer it exists to catch.

↪ The independence a monitor needs from the proposer across sensing, compute, and actuation is developed in section 12.8.

Systems Perspective 8.3: The metric veto invariant

Safety is an invariant of metric clearance, not semantic understanding. Suppose a slow 2 Hz vision-language foundation model classifies a pathway as "clear aisle" or dismisses an object as a harmless shadow. If the 20 Hz LiDAR, or the occupancy grid built from it, detects a physical obstacle return within the stopping envelope, the permission path must honor the qualified occupied-region evidence rather than let semantic confidence erase it. The permission path halts or slows the machine regardless of the semantic model's high confidence. Semantic models propose high-level task goals; qualified metric sensing supplies clearance evidence to the permission path. Neither sensor pipeline itself owns actuator authority.

Checkpoint 8.2: Spatial error and the clearance inset

A spatial claim is spent from the same clearance as its age, and its error must be measured before it is charged.

- ☐ **Quadratic depth error**: Can you explain why stereo depth error grows with the square of range while a pulsed LiDAR's range precision stays roughly flat?
- ☐ **Baseline estimate**: If the illustrative rig's 0.20 m baseline were halved, at what range would one standard deviation of depth error reach the 0.15 m side clearance, compared with 6.9 m now?
- ☐ **Covariance vs. clearance inset**: Can you explain why a residual quantile measured against ground truth, not a reported covariance, sets C_{inset}, and what that quantile still leaves uncovered?
- ☐ **Learned depth vs. metric evidence**: Can you explain why a depth network's predicted spread cannot enter a clearance claim until its residuals are validated, and why qualified metric evidence overrides a semantic label that calls a return a shadow?

8.7 Perception Failure Modes and Sensor Degradation

Every bound derived so far, whether metric, learned, or temporal, assumes that the sensor behaves as calibrated: propagation bounds the error of a sensor working as modeled, not one that has stopped working that way while still delivering frames at its nominal rate. A navigation camera can fail in two ways that look entirely different to downstream software. If its serializer link loses lock, the driver raises an error and the frame never arrives. If direct sunlight through an open dock door saturates the lens as the vehicle turns into an aisle, the camera delivers a complete frame at its nominal rate, with a shelving rack hidden under a patch of clipped white pixels. Perception failures divide along this boundary into two operational classes. **Detectable faults** occur when the physical sensor or communication transport asserts an explicit hardware or software error condition. Examples include a serializer-deserializer (SerDes) link losing clock synchronization, a cyclic redundancy check (CRC) detecting a corrupted payload, a DMA frame buffer overflowing, or a transport bus timing out. These failures are straightforward to handle. The hardware driver raises a fault flag at the ingestion boundary, downstream consumers drop the corrupted frame, and state estimators expand their uncertainty. Once the age of valid sensor evidence exceeds its configured freshness budget, the permission path rejects motion requests that depend on it and invokes a fallback qualified for the remaining state evidence, actuation authority, and stopping margin. In contrast, **silent failures** occur when the sensor hardware, firmware, and transport bus function without error, delivering a structurally valid tensor whose numerical values misrepresent the physical world.

Physical degradation of the optical channel is a frequent source of silent failure. When bright sunlight enters the aperture, incident photons overwhelm the full-well capacity of the CMOS photodiodes, clamping pixel values at their maximum quantization limit across large patches of the sensor array. This **dynamic range saturation** destroys local spatial gradients. Downstream feature extractors, optical flow estimators, and stereo disparity algorithms fail because numerical optimization over a uniform flat field yields zero directional gradient, even while the camera driver

delivers frames at the full nominal rate. Dimly lit environments trigger the opposite failure mode. When ambient photon flux drops, the discrete, stochastic arrival of photons (**photon shot noise**) and the thermal and electrical fluctuations of the readout circuitry (**sensor read noise**) dominate the signal. As the signal-to-noise ratio collapses, learned neural backbones begin extracting spurious geometric features from random noise patterns rather than physical obstacle boundaries. Contaminants on the protective lens or dome window, such as water droplets, dust, or grease smudges, act as irregular refractive elements and optical low-pass filters. They attenuate contrast, distort spatial geometry, and mask critical obstacles while the transport stack reports flawless operation.

Timing and transport anomalies can also corrupt an observation silently. If an unshielded physical interface experiences electromagnetic interference, deserializer bit slips (where the hardware drops or duplicates a bit, misaligning the subsequent serial data stream) can alter payload semantics without invalidating standard packet lengths. More dangerous than a dropped packet is an observation stamped with an incorrect capture epoch, because no transport monitor detects the error. If an image sensor captures photons at epoch t_0, but a late interrupt handler or an unsynchronized system clock timestamps the resulting memory buffer at $t_0 + \delta t$, the downstream state estimator integrates spatial features under an incorrect body pose. Consider a machine traveling with instantaneous velocity $\mathbf{v}$ and linear acceleration $\mathbf{a}$ along a given trajectory. When perception suffers an undetected timing desynchronization δt, the state estimator assigns the detected position of an obstacle to the body coordinates at $t_0 + \delta t$ rather than t_0. Over the unmodeled time interval δt, the physical state estimation error $\Delta\mathbf{p}$ between the asserted position and the true position is governed by the kinematic drift relation (equation 8.11):

$$\Delta\mathbf{p} = \mathbf{v} \cdot \delta t + \frac{1}{2}\mathbf{a} \cdot (\delta t)^2 \tag{8.11}$$

The arrival stamp discussed in section 8.3 represents a silent desynchronization of this exact type. At constant velocity, the acceleration term vanishes, leaving the kinematic offset $\mathbf{v}\,\Delta t_{\text{recv}}$ uncorrected while the driver reports nominal health. The asymmetry between a lost frame and a misdated one needs no estimator mechanics to see. An estimator that accepts a misdated frame as a correctly timed measurement can shrink its reported uncertainty around the wrong position, while a missing update can leave uncertainty to grow under its motion model. In this case, a dropped frame adds no information, so the reported spread grows and the evidence is honestly one frame period (16.7 ms) older. A misdated frame is accepted as information, so the spread shrinks around a position offset by $v\,\delta t$. On the running machine the arrival stamp alone places the obstacle 39.9 mm from where it stood, and the estimator reports that wrong position with more confidence than before. The motion planner is then sure that it has already cleared an obstacle it is about to strike. Chapter 9 develops the recursive estimator in which this happens, the Kalman filter.

Defending against silent failures requires dedicated runtime health monitors placed directly at the sensor ingestion boundary. A pixel-intensity histogram monitor inspects raw digitized frames immediately after DMA transfer, flagging overexposure when pixel counts concentrate at maximum bit depth, or flagging lens occlusion when spatial variance collapses below an empirical noise floor. A hardware watchdog timer tracks inter-arrival times across the DMA channel, flagging transport jitter when inter-frame intervals exceed $T_{\text{nominal}} + 3\sigma_{\text{jitter}}$. This empirical diagnostic heuristic does not replace the physically derived freshness deadline. Higher in the ingestion stack, an optical flow plausibility monitor compares calculated pixel displacement vectors against the body's physical velocity envelope, rejecting observations that imply accelerations beyond the chassis's mechanical limits. In multimodal configurations, cross-sensor consistency checks compare high-rate (1,000 Hz) inertial measurement unit (IMU) integrations against visual odometry. If visual odometry reports that the robot is stationary while the gravity-compensated IMU integration indicates sustained $2.0\ \text{m/s}^2$ linear acceleration, the monitor flags a cross-modal discrepancy once attitude, sensor bias, latency, and measurement uncertainties are reconciled.

Every runtime health monitor observes the physical world through the available transducer signals. Consequently, no single sensing channel can validate its own semantic interpretations. A camera assembly with pristine optics, optimal exposure, jitter-free transport, and sub-microsecond hardware timestamping can still present an image that causes a deep neural network to hallucinate a navigable path or fail to detect an obstacle. Because deep vision backbones map high-dimensional pixel arrays through non-linear manifolds, subtle visual anomalies (a sharp shadow across an

aisle, a life-sized person printed on a cardboard display, or specular glare off shrink-wrapped freight) produce valid pixel statistics, smooth optical flow fields, and misleadingly high prediction confidence. An ingestion monitor confirms that the raw data acquisition pipeline operated within nominal engineering limits, but it cannot confirm that downstream semantic inferences correspond to physical reality. Corroborating optical detections requires independent physical sensing modalities, such as lidar or millimeter-wave radar, whose coverage, beam geometry, and failure dependencies have been rigorously characterized. Even then, a validated permission path guarantees spatial clearance only within explicit operational and timing assumptions; no perception stack can guarantee unrestricted semantic truth. Understanding how the physical environment corrupts each sensing channel explains why single-modality checks fall short.

8.7.1 Fog, rain, glare, and specular surfaces

In an indoor warehouse, daylight flooding through an open loading-dock bay door or standing water on polished concrete can saturate cameras and deflect LiDAR beams into empty space, while fog and rain add atmospheric scattering for outdoor platforms that carry the same sensors. Each of these conditions degrades sensory transmission across optical, near-infrared, or radio channels without raising a hardware error flag. Optical cameras undergo severe degradation in the presence of atmospheric scattering and glare. Suspended airborne aerosol particles in fog, mist, and smoke (1 to 10 µm diameter) scatter visible wavelengths ($\lambda \approx 400$ to 700 nm) under Mie scattering physics. Scene radiance attenuates exponentially across distance according to Koschmieder's law:

$$C(d) = C_0\, e^{-\beta_{\text{atm}} d}$$

where C_0 is intrinsic object contrast, d is metric range, and β_{atm} is the atmospheric extinction coefficient. In dense fog ($\beta_{\text{atm}} \approx 0.05$ m^{-1}, an illustrative value), apparent contrast decays below the 0.05 perceptual detection threshold within 60 m. High-frequency edge gradients dissolve into uniform background luminance, preventing convolutional and transformer backbones from forming valid feature embeddings. Airborne rain introduces high-velocity streak noise across frames, while droplets impacting the protective lens act as irregular refractive microlenses. Each droplet deflects incoming rays, creating severe localized geometric distortion, optical blur, and false object silhouettes.

Rapid angular motion compounds this degradation through motion blur and modulation transfer function (MTF) collapse.[13] During a yaw rotation or manipulator sweep, a camera center at lever arm r travels a tangential distance set by angular rate ω and exposure duration:

$$\Delta x_{\text{rot}} = r \cdot \|\omega\| \cdot t_{\text{exp}}$$

For the machine's global-shutter wrist camera at its configured 5 ms exposure, swept at an illustrative $\omega = 3$ rad/s on an illustrative lever arm $r = 0.5$ m, this travel scale is $\Delta x_{\text{rot}} \approx 7.5$ mm. This metric displacement describes the physical travel of the optical center rather than image-plane blur in pixels. The resulting pixel displacement depends on both scene depth and focal length, while pure camera rotation shifts features across the image plane independently of depth. As motion blur smears sharp transitions across neighboring pixels, spatial gradients attenuate and feature trackers, visual odometry algorithms, and neural backbones lose tracking lock. Systems engineers must evaluate these effects across the full envelope of intended operational velocities and camera exposures.

[13] **Modulation Transfer Function (MTF)**: The fraction of scene contrast an imaging chain passes at each spatial frequency. A uniform blur of length L in the image multiplies it by a sinc that falls to zero at a period of L, so detail finer than the blur is erased rather than dimmed, and no network downstream can restore what the exposure removed.

Intra-frame illumination contrast introduces another challenging failure mode through dynamic range saturation and blooming. Standard auto-exposure algorithms adjust exposure time t_{exp} across consecutive frames to track gradual lighting changes, such as moving from a shaded warehouse aisle into an outdoor yard. However, auto-exposure cannot resolve extreme illumination differences within a single image. When a robot operating in a dim indoor aisle turns toward an open loading dock flooded with daylight, no single exposure setting suffices. An exposure long enough to resolve dark shelving units allows the intense photon flux from the dock to saturate the photodiodes, clipping those image regions to solid white. High dynamic range (HDR) sensors expand operational range by interleaving multiple exposures or employing dual-gain photodiode architectures. Yet staggered multi-exposure HDR introduces timing ambiguities that the observation contract must document. Because each synthesized exposure corresponds to a distinct temporal integration window, the metadata must specify which exposure midpoint dates the frame. When extreme overexposure occurs, excess photo-generated electrons spill over potential barrier implants into neighboring pixels

(**charge blooming**), washing out entire sensor columns into saturated values (255). Concurrently, internal reflections between lens elements generate veiling glare and lens flare streaks, eliminating the local spatial contrast essential for disparity matching and optical flow.

Active optical ranging encounters parallel limitations under degraded atmospheric conditions. Pulsed time-of-flight LiDAR, at silicon-band near-infrared wavelengths and at $1{,}550$ nm alike, suffers three distinct physical failure mechanisms in industrial and outdoor operation:

1. *Aerosol backscatter clutter*: Emitted laser pulses strike suspended fog droplets and dust particulates, scattering optical power back into the receiver during the initial nanoseconds of time-of-flight. This creates dense near-field point clutter, often appearing as a false obstacle wall within 0.5 to 3.0 m of the sensor aperture. Downstream spatial clustering algorithms misinterpret these backscatter returns as imminent collision hazards, triggering spurious emergency stops. Simultaneously, atmospheric extinction depletes forward pulse energy, significantly shortening maximum detection range. The magnitude of this range reduction depends on target reflectivity and the atmospheric extinction coefficient, requiring empirical calibration under the operational conditions for which the sensor is qualified.
2. *Retroreflector detector blooming and range walk*: High-visibility safety vests, barcode placards, and vehicle license plates incorporate retroreflective prismatic surfaces designed to return incident optical power directly along the emission axis at intensities orders of magnitude higher than diffuse surfaces. This strong return pulse triggers two distinct measurement errors. **Range walk** is a temporal distortion in the receiver's avalanche photodiodes (APDs) or single-photon avalanche diodes (SPADs). A simple leading-edge discriminator triggers when an analog pulse crosses a fixed voltage threshold, so a fast-rising, high-amplitude pulse trips the comparator earlier than a weak return and systematically underestimates the distance to the retroreflector. While intensity-based compensation curves can correct this offset under linear amplification, they fail once the analog front-end saturates. Concurrently, **blooming** creates a spatial distortion: optical scattering and electrical crosstalk within the detector array register false returns across adjacent detector channels, surrounding the retroreflector with a spherical halo of phantom points. Sensor manufacturers reflect these physical realities in their specifications; for example, the Ouster OS1 datasheet establishes distinct, wider accuracy tolerances for retroreflective targets compared to diffuse Lambertian surfaces (Ouster, Inc. 2025).
3. *Specular surfaces and non-Lambertian forward bounce*: Triangulation systems, structured-light projectors, and time-of-flight LiDAR all assume that surfaces produce diffuse Lambertian scattering, reflecting incident light uniformly across the hemisphere. Specular surfaces such as polished concrete floors, shrink-wrapped freight, painted metal panels, and architectural glass violate this assumption. Emitted laser beams undergo specular mirror reflection, bouncing away from the surface normal at an angle equal to the incidence angle ($\theta_{\text{refl}} = \theta_{\text{inc}}$) and directing optical energy away from the receiver. On wet or polished floors, this specular bounce results in complete signal loss, creating artificial zero-return dropouts ("puddle black holes"). Against glass walls or metal structures, multi-bounce reflections cause emitted pulses to travel indirect paths back to the receiver, generating phantom point clusters that appear meters behind physical walls or deep beneath the concrete floor.

Millimeter-wave radar provides complementary all-weather capability that overcomes many optical failure modes. Operating at 76 to 81 GHz, frequency-modulated continuous-wave (FMCW) radar uses a carrier wavelength of $\lambda \approx 3.9$ mm, roughly 4,000 times longer than near-infrared laser light and several orders of magnitude larger than fog droplets (1 to $10\,\mu$m). Scattering by such small airborne particles falls within the Rayleigh scattering regime, where the particle scattering cross-section σ_s scales inversely with the fourth power of wavelength:

$$\sigma_s \propto \frac{d_{\text{droplet}}^6}{\lambda^4}$$

Although this extreme wavelength dependence shields radar from the severe attenuation and backscatter that paralyze optical sensors in fog and dust, radar is not immune to environmental interference. Ground reflections produce multipath ghost detections, standing water films on

the radome attenuate radio-frequency signals, and low spatial resolution complicates boundary discrimination. A multimodal perception suite must therefore explicitly qualify radar coverage boundaries when primary optical streams degrade. Even when the physical channel is clear and the sensor hardware operates within specification, however, a neural backbone can still encounter scenes that lie outside its training distribution.

8.7.2 Out-of-distribution inputs at the sensor boundary

Even when a sensor is calibrated, unoccluded, and operating without transport faults, an unfamiliar scene (such as an overturned forklift, an unusual trailer profile, or a chemical vapor plume) can fall outside the operational distribution on which downstream models were qualified. Uncertainty in these observations stems from two distinct sources. **Aleatoric uncertainty** represents the irreducible physical noise of the transduction process itself, such as photon shot noise, thermal amplifier noise, or aerosol backscatter. The measurement's covariance model must quantify this uncertainty. In contrast, **epistemic uncertainty** stems from the model's lack of knowledge about scenarios omitted from its training distribution. Upgrading sensor fidelity cannot eliminate epistemic uncertainty. The limits of normalized softmax scores, calibrated uncertainty estimation, and the conversion of calibrated prediction sets into action refusals lie beyond perception's scope. Perception carries a narrower engineering responsibility. It must assess at the ingestion boundary whether an incoming observation appears outside the validated input support, and log that assessment directly in the observation contract.

Input-support monitors evaluate incoming sensor frames, or early feature embeddings from the backbone, against the support of the qualification dataset. Common implementations include computing a free-energy score over unnormalized logits (Liu et al. 2020), measuring the Mahalanobis distance to class-conditional feature centroids (Lee et al. 2018), or tracking the reconstruction error of an autoencoder trained strictly on nominal operational data. Each method requires an empirical acceptance threshold calibrated against expected anomalies. An in-bounds score means only that the monitor did not flag the input under its calibrated threshold; it guarantees neither membership in the qualification distribution nor physically accurate semantic inference.

↩ The limits of softmax confidence and the conformal calibration that bounds it are established in section 3.6.2.

When an input-support monitor detects an anomaly, the perception pipeline sets a `BIT_OOD_DETECTED` flag in the health bitmask that every observation carries, one of the record fields that section 8.8 assembles. The permission path monitors this bitmask to disallow high-speed motions, expand conservative clearance buffers, or trigger a deterministic supervisory fallback. In multimodal systems, the same bitmask arbitrates between sensors. On the running machine, if the navigation camera raises `BIT_OPTICAL_SATURATED` or `BIT_OOD_DETECTED` while the LiDAR's returns stay healthy, the pipeline demotes the camera's tokens and the rack-end claim rests on LiDAR range alone. This holds only if the LiDAR is independently validated to enforce spatial clearance under those operating conditions. Outdoor platforms apply the same arbitration with radar in the LiDAR's place.

↩ The input-support monitor these checks instantiate is established in section 7.9.

War Story 8.1: Silent perception omission (2016)

Context: A 2015 Tesla Model S traveling at about 74 mph with Traffic-Aware Cruise Control and Autosteer active struck a refrigerated semitrailer turning left across its path near Williston, Florida (National Transportation Safety Board 2017).

Mechanism: The crossing-path collision lay outside the front-to-rear crash modes for which the 2015 warning and automatic braking systems were designed. NHTSA reported that automatic braking required agreement between camera and radar; unusual vehicle shapes could impede camera classification (NHTSA ODI PE16-007).

Impact: The NTSB found no recorded indication that forward collision warning or automatic emergency braking detected an object, and no pre-impact braking occurred. The vehicle passed beneath the turning semitrailer, shearing off the car's roof; the paired investigation photographs in figure 8.8 document this physical outcome, highlighting collision scrape damage along the right underside of the trailer alongside the roofless passenger cabin.

Response: The NTSB examined operational design domain limits and driver engagement as well as the collision-avoidance systems.

Figure 8.8: Williston collision damage: This NTSB presentation slide shows damage on the right side of the semitrailer and the roofless Tesla after the 2016 crash. The crossing-path geometry and absence of pre-impact braking come from the NTSB accident report, not from the photographs alone. (Source: NTSB board-meeting presentation for report HAR-17/02; photographs courtesy of Florida Highway Patrol).

Systems lesson: An absent detection is not evidence that a crossing path is clear. A permission design must account for detection coverage and unsupported scenarios; a health flag cannot certify that every relevant object was perceived.

An observation contract can expose only the physical, temporal, and distributional faults that the ingestion layer is designed to measure. Detectable interface errors, transport jitter, and out-of-distribution feature shifts can be encoded into explicit health bitmasks. Silent semantic omissions, such as the missed detection at Williston, evade single-sensor health checks. Preventing catastrophic failures requires an architectural safety design: incorporating independent physical sensing modalities, enforcing conservative spatial permission envelopes, and never mistaking the absence of an error flag for confirmation of physical safety. Omission can also depend on who is in view: a detector qualified mostly on people standing upright can miss a kneeling worker or a child more often, so any detection rate a contract reports for people holds only for the groups it was measured on.

↪ How a detection gap between groups of people becomes a gap in stopping distance and impact energy, and how large each group's sample must be, is developed in section 17.5.

8.8 The Observation Contract

A downstream tracker, policy backbone, or permission path cannot reconstruct exposure timing, mounting calibration, or byte integrity from a raw tensor alone. The perception subsystem therefore wraps each sensory output in an **observation contract**, the sixth record in the book's sixteen-record chain. In closed-loop physical systems, an observation is unusable without an explicit capture epoch, an identified clock domain, a bounded conversion to the permission path's clock, and an extrinsic calibration operating within its qualified validity envelope. A safe consumer must reject any incoming record lacking these guarantees before inspecting a single payload byte. The system's shared clock domain and $\mathcal{H}_1$ age budget originate in the nervous-system handoff record of section 4.9; perception, in turn, provides the capture-side timestamps and uncertainty bounds that downstream consumers need to verify compliance.

↪ The sixteen-record chain, in which the observation contract is Record 6, is developed in section C.14.2.

Formally, the contract couples either a raw observation tensor or an extracted geometric bounding envelope with the validation metadata that downstream consumers require (equation 8.12):

$$\mathcal{O} = \langle t_{\text{capture}}, \text{clock_id}, \epsilon_{\text{sync}}, \text{frame_id}, \mathbf{T}_{\text{payload}}, \Sigma_{\text{body}}, C_{\text{inset}}, \text{mask}_{\text{health}}, \text{crc32} \rangle \tag{8.12}$$

In this tuple, t_{capture} records the exposure-midpoint timestamp registered on monotonic clock clock_id, ϵ_{sync} bounds the maximum synchronization error when converting to the clock of the

safety MCU that hosts the permission path, and frame_id identifies the calibrated sensor transform. The payload $\mathbf{T}_{\text{payload}}$ contains the raw sensor array or derived geometric bounding envelope, Σ_{body} characterizes its spatial covariance, and C_{inset} provides the qualified clearance inset. The health mask $\text{mask}_{\text{health}}$ conveys hardware and perceptual health status bits, and the checksum crc32 seals both header and payload against transport corruption. Spatial memory consumes this record to update its world model, while trajectory planning uses C_{inset} to inflate collision boundaries. Both modules evaluate evidence age, verify kinematic coordinate frames, and bound spatial uncertainty before allowing an observation to influence a control decision.

The tuple is the same object as the error-qualified claim $(\mathbf{p}_{\text{body}}, \Sigma_{\text{body}}, C_{\text{inset}}, t_{\text{age}})$ that section 8.6 assembled, now wrapped for transport. The claim's point travels in $\mathbf{T}_{\text{payload}}$, its covariance in Σ_{body}, and its inset in C_{inset}. Its age is deliberately not stored, because each consumer computes the current evidence age from t_{capture} and ϵ_{sync} on its own clock.

This contract does not replace binary serialization formats such as Protocol Buffers, FlatBuffers, or ROS 2 IDL; any of these serialization systems can transmit the underlying struct over shared memory or bus fabrics. While transport schemas standardize memory layout and data types, the observation contract establishes verifiable physical invariants: monotonic temporal progression, bounded clock skew, active calibration validity windows, and rigorous spatial uncertainty limits. Table 8.5 serializes the tuple as named fields, details each one alongside its validation rule, and populates the schema for the mobile manipulator's navigation camera. It splits t_{capture} and clock_id into `capture_timestamp_ns` and `capture_clock_id`, carries the calibration behind frame_id in `calibration_metadata`, and adds a `sequence_index` and a `provenance_digest` that the tuple leaves out.

Table 8.5: Observation contract schema: Compact fields needed to date and frame a claim, evaluate its uncertainty, bounding envelope, and health flags, and check the bytes and generating revision. The last column fills the fields this chapter computes for the mobile manipulator's navigation camera; its dispatch age and clock-conversion bound are illustrative. A keyed authenticator is a separate requirement if the threat model includes malicious modification.

Record field	Meaning	Consumer check	Navigation camera
`capture_timestamp_ns`, `capture_clock_id`	Exposure midpoint in one shared, monotonic capture clock	Convert to safety microcontroller (MCU) time with a valid bounded-skew mapping before checking age	Trigger edge plus $t_{\text{exp}}/2$ (top-row exposure midpoint) on clock `cam_nav`; dispatched 51.6 ms later (table 8.3)
`sync_error_ns`	Bound on capture-to-MCU clock conversion error	Add to worst-case age; refuse if missing or expired	0.2 ms, so the upper age at dispatch is 51.8 ms
`sequence_index`	Monotonic camera frame counter	Detect gaps and reordering	One count per frame at 60 Hz
`sensor_frame_id`	Transform-tree key and calibration revision	Select the valid extrinsic transform	`nav_cam` with its calibration revision
`tensor_payload`	Format, dimensions, length, and buffer	Validate bounds and record checksum before reading	Raw/debayered image tensor or point-cloud buffer
`bounding_envelope`	Bounding volume $[dx, dy, dz]$ or qualified clearance inset	Bound occupied volume and inflate safety sets (C_{inset})	Metric corridor bounds inflated by validated quantile
`spatial_covariance`	Positive-semidefinite spatial error scale	Use with separately calibrated bias and tail bounds	Point-wise or object-centric covariance matrix Σ_{body}
`calibration_metadata`	Intrinsics, extrinsics, coefficients, temperature, validity range	Reject calibration outside its validated region	Factory pinhole $\mathbf{K}$ and chassis extrinsic transform $\mathbf{T}_{bc}$
`provenance_digest`	SHA-256 of firmware and model revisions	Match the approved build; this is not payload integrity	Signed hash of vision backbone weights and firmware
`payload_crc32`	CRC over serialized header (excluding this field) and payload	Detect accidental record corruption; not authentication	32-bit cyclical redundancy check
`health_bitmask`	Hardware, degradation, and perception validity flags	Refuse claims with safety-critical faults or OOD flags	Sync-loss, link, saturation, rain/fog clutter, and OOD bits; input-support bits

The fields detailed in table 8.5 anchor each observation in physical time, geometric space, and software revision history. The capture timestamp t_{capture} must refer to a strictly monotonic hardware timer rather than wall-clock time (such as nanoseconds since the UNIX epoch), which remains vulnerable to stepping backward during network time adjustments. Because rolling-shutter sensors read out scan lines sequentially, any spatial claim derived from a rolling-shutter imager must explicitly identify the image row or region corresponding to that capture instant. The transport-level `sequence_index` counter detects frame drops and out-of-order deliveries directly on the bus, independently of clock readings. When the safety MCU operates in a separate clock domain from the camera host, an active synchronization daemon provides a versioned affine mapping $t_{\text{MCU}} = at_{\text{capture}} + b_{\text{clk}}$, an expiration window, and an empirical sync-error bound ϵ_{sync}, characterized as described in section 8.6.2.

Systems engineering demands precise conventions when accounting for sensor latency. The handoff matrix and the perception latency budget evaluate the sensor's latency contribution under different row conventions, and the observation contract must explicitly state which convention governs the record. The handoff matrix allots $\mathcal{H}_1$ one frame period (16.7 ms at 60 Hz) and references the entire frame by its middle row. Because rolling-shutter rows stagger their exposures at the exact rate they read out, the middle row finishes its own exposure halfway through the frame's readout window and waits only the remaining $t_{\text{readout}}/2$ for the bottom half of the array to clock off the die, so measuring from that middle row's exposure midpoint to the completion of sensor readout yields a sensor-side latency of $t_{\text{exp}}/2 + t_{\text{readout}}/2 = 16.3$ ms, which falls well within the handoff allotment. Conversely, the perception latency waterfall charges for the top row: $t_{\text{exp}}/2 + t_{\text{readout}} = 24.6$ ms. Because a physical hazard such as a warehouse rack upright can appear in the very first scan lines, a safety-certified stopping budget must charge for the oldest sensory evidence underpinning the spatial claim. Under this oldest-row convention, the navigation camera exceeds the handoff allotment by 7.9 ms. The two numbers answer different questions, and the stopping budget governs. The $\mathcal{H}_1$ allotment is a design budget for the sensing handoff, stated in the middle-row convention, and the camera meets it. The safety charge is row five, which uses this chapter's oldest-row waterfall. The 7.9 ms is already inside the 51.6 ms that row five charges, so nothing is missing from the stopping budget. What the mismatch does require is a record that says which convention dated it. The capture metadata therefore stamps the reference row index, so that a consumer checking $\mathcal{H}_1$ and a consumer checking clearance each read the age they need from the same field.

Upon dispatching an observation, the producer records an upper bound on its observation age at dispatch as $t_{\text{MCU,dispatch}} - t_{\text{MCU,capture}} + \epsilon_{\text{sync}}$. That metric reflects only the latency accumulated up to the producer's egress boundary. By the time the permission path admits the record, the evidence has aged further by the dispatch-to-admission transit delay Δ_{da}, and it ages further still before actuators execute a command. The permission path must evaluate freshness using its local monotonic clock against the original capture epoch; treating a static dispatch timestamp as if time had frozen violates basic physical causality. Submitting a new trajectory proposal derived from a previously buffered observation does not refresh the underlying sensory evidence.

For the navigation camera, the **dispatch-local** threshold equals the budgeted observation age 51.6 ms plus the 0.2 ms clock-conversion bound, yielding 51.8 ms. The threshold sits exactly at the budget because row five of the stopping budget charges exactly that age; admitting an older frame would spend travel that no row accounts for. A frame that meets every stage budget therefore reaches dispatch with zero spare margin, and any host contention, pipeline queuing, or batching beyond those budgets pushes it past the threshold. The stage values are allocations each stage must meet at its tail, not typical latencies, so a pipeline that meets them refuses only the frames that overrun them. In the vehicle's stopping budget, the 51.6 ms perception delay maps directly to physical travel distance at cruising velocity. The 0.3 mm displacement associated with the clock-conversion uncertainty bound is absorbed into the localization uncertainty parameter δ_{loc}. Neither accounting choice eliminates the ongoing physical aging that occurs after dispatch.

Admitting an observation into the motion-planning safety pipeline requires an empirically qualified current-age limit that accounts for the dispatch-to-admission bound Δ_{da} while reserving sufficient execution time for the active motion lease and physical braking response. Because this partial budget has not yet bounded Δ_{da}, the populated record in table 8.5 illustrates correct structural field construction rather than an admitted full-speed trajectory. Once engineers measure and bound

Δ_{da} across all inter-process channels, they can either close the stopping budget or reduce maximum vehicle velocity to restore safety margins. Latency accounting conserves time. Whether an engineer anchors the timeline at physical capture or measures from the admission gate by summing prior evidence age with the active lease, each elapsed interval appears exactly once in the stopping budget.

The transform identifier `sensor_frame_id` and its calibration revision key the observation into the robot's kinematic transform tree.[14] The provenance digest records the exact SHA-256 hash of the vision model weights and camera firmware, providing an auditable record of software configuration. Concurrently, `payload_crc32` guards against transmission glitches across shared-memory rings or serial buses. Data integrity is not semantic truth. Checksums and build digests detect accidental bit flips and firmware mismatches, but they cannot verify whether an image is optically uncorrupted, nor do they prevent adversarial tampering without a keyed cryptographic signature.

The `health_bitmask` structures failure reporting across three architectural layers. The lowest bits report hardware-level faults, such as clock synchronization loss, DMA ring buffer overruns, or physical link dropouts. The intermediate bits signal environmental degradation, including sensor saturation from direct sunlight, optical blooming, and aerosol backscatter from dust or fog. The upper bits represent perceptual validity concerns, such as severe field-of-view occlusion or out-of-distribution (OOD) sensory inputs that fall outside the training support tested during system evaluation. When any safety-critical bit is set, downstream motion planners and trackers must immediately reject spatial claims derived from that frame.

An explicit observation schema also creates a systematic interface for automated fault injection. Verification harnesses can deterministically alter capture timestamps, drop sequence numbers, artificially inflate covariance diagonals, or assert occlusion bits, confirming that downstream consumers reject or gracefully degrade unviable observations within their budgeted time limits. These software mutations demonstrate consumer responses for the tested corruptions under the tested budgets; evaluating resilience against physical phenomena such as optical flare, mounting deflection, and actuator dynamics requires hardware-in-the-loop testing regimes.

Learned policies require explicit contract metadata just as much as classical state estimators do. An end-to-end policy $\mathbf{a}_t = \pi(\mathbf{z}_{t_0})$ that receives only sensory pixels or point clouds cannot distinguish two identical images consumed under radically different pipeline latencies. Widely used vision-language-action (VLA) models do not receive these quantities. RT-2 and OpenVLA condition on visual tokens and a language prompt (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024), and π_0 adds proprioceptive joint state (Black et al. 2024), but none of them is given capture age, extrinsic transforms, or measurement covariance as input. To integrate learned models safely into real-time control loops, this book presents an **observation tokenizer** that projects the full observation contract into the token space of a VLA policy or neural estimator. The sensory modality dictates the raw payload encoding (patch tokens from vision transformers, range tokens from lidar arrays, spike tokens from event cameras, or stress fields from tactile arrays), while the tokenizer interface preserves the capture epoch, geometric frame, and measurement uncertainty associated with that payload.

Under this architecture, the observation tokenizer pairs the sensory feature tokens $\mathbf{z}_{\text{sensor}}$ with continuous Fourier temporal embeddings ($\mathbf{e}_{\Delta t} = \text{Fourier}(t_{\text{current}} - t_0)$), coordinate frame prefix embeddings ($\mathbf{e}_{\text{frame}} = \text{MLP}(\mathbf{T}_{bc})$), and spatial covariance embeddings ($\mathbf{e}_{\Sigma} = \text{MLP}(\text{vec}(\Sigma_{\text{body}}))$) (equation 8.13):

$$\mathbf{z}_{\text{token}} = [\mathbf{z}_{\text{sensor}}, \mathbf{e}_{\Delta t}, \mathbf{e}_{\text{frame}}, \mathbf{e}_{\Sigma}] \tag{8.13}$$

A forward-propagated state token $\mathbf{e}_{\text{state}} = \text{MLP}(\hat{\mathbf{x}}_{t_{\text{current}}})$, generated by the state estimator that spatial memory maintains, can accompany these tokens so the policy conditions on the robot's predicted state at the instant of actuation rather than its historical configuration at sensor exposure. Because the observation contract preserves the exact capture epoch, delayed frames do not become worthless garbage. When network congestion or compute preemption delays delivery, a state estimator can ingest an out-of-order observation at the precise historical instant it captured and propagate corrections forward to the current state, a technique that spatial memory develops in section 9.2.3. While this embedding scheme exposes physical time, frame geometry, and uncertainty directly to transformer attention layers, engineers must verify that the model actually attends to these tokens in closed-loop operation. Demonstrating that a policy uses temporal embeddings requires rigorous ablation experiments that inject artificial latency into held-out evaluation trials and measure

[14] **Stereo Baseline Thermal Drift**: For a stereo baseline $b = 0.20$ m and an unmodeled change $|\delta b| = 0.1$ mm, first-order geometry gives $|\delta Z_c|/Z_c \approx |\delta b|/b$. At $Z_c = 20$ m, the corresponding depth bias is about 10 mm, assuming focal length and disparity are otherwise unchanged. Temperature compensation must be checked against the actual mount and calibration.

↪ Hardware-in-the-loop testing of physical phenomena is developed in section 15.8.

resulting closed-loop tracking error. Embedding metadata as neural tokens also does not substitute for deterministic safety gating. The permission path must continue to verify clock bounds, extrinsic calibration envelopes, innovation residuals (the gap between each measurement and the estimator's prediction of it), and health status bits before any network output is permitted to drive the physical actuators.

Checkpoint 8.3: Observation contract checks

A consumer can refuse only what the record lets it check, and it must check before it reads the payload.

- ☐ **Record triage**: Can you explain why a health bit, checksum, valid clock conversion, and freshness check must precede consumption of a spatial claim?
- ☐ **Timestamp corruption vs. dropped packets**: Can you describe why receiving a sensory frame with a desynchronized timestamp degrades spatial state estimation and policy grounding far more severely than dropping the packet entirely?
- ☐ **Deterministic fault injection**: Can you distinguish between physically perturbing optical sensors vs. deterministically mutating fields in the shared-memory observation contract during automated hardware-in-the-loop verification?
- ☐ **Age on the consumer's clock**: Can you explain why the permission path must recompute evidence age on its own monotonic clock from the capture epoch, rather than trusting the age stamped at dispatch, and why its 51.8 ms dispatch-local threshold is not the limit it applies at admission?

8.9 Fallacies and Pitfalls

Passing contract triage and computing state updates still does not make an observation semantically true in the physical world, and the most common perception errors come from treating a syntactically valid record as a statement about the present scene. Perception must describe the physical world within the strict temporal deadlines demanded by closed-loop control. In embodied systems, high semantic confidence and fine spatial resolution cannot compensate for pipelines that deliver stale observations, fuse asynchronous exposures, or treat average I/O bandwidth as a bound on single-frame latency.

Fallacy: *Accepting an observation because its semantic classification confidence is high, regardless of its age.*

Consider a mobile manipulator that receives an observation indicating with high semantic confidence that the rack end ahead is clear. If a scheduling stall in the vision pipeline delivers that observation several frames late, a warehouse worker who stepped into the aisle after the camera exposure will not appear in the image. The confidence score describes only the classifier's assessment of past sensory input, not current physical clearance. Treating an overdue packet as present state risks commanding actuators directly into occupied space. The perception interface must verify measurement age against its freshness limit (section 8.8) and reject observations that exceed the threshold. Furthermore, any state propagation that continues without fresh data must inflate kinematic uncertainty covariances, incorporating heavy-tailed residuals or reachability bounds on dynamic obstacles. This growth forces downstream controllers to contract their safety envelopes and slow the robot as sensory evidence ages. High softmax confidence reflects internal model certainty on an acquired image, but it guarantees neither temporal freshness nor true physical clearance.

Pitfall: *Fusing multi-sensor observations without a clock bound derived from the tolerated spatial error.*

Suppose two cameras on the mobile manipulator observe the same rack upright while the mobile base travels at its 1.5 m/s velocity limit, but their internal clocks differ by an uncorrected skew Δt_{skew}. If downstream fusion treats the two exposure times as simultaneous, the physical upright appears displaced by $v\,\Delta t_{\text{skew}}$ between the two sensor frames. Downstream track association can split this single physical obstacle into two conflicting tracks ("ghosting"), corrupting the world model. Spatial calibration cannot correct an uncharacterized temporal offset. Instead, the allowable clock synchronization error must follow directly from the tolerated spatial position error ϵ_{pos} and

the maximum operational velocity $v_{\max}$, yielding the relation $\Delta t_{\text{skew}} \leq \epsilon_{\text{pos}}/v_{\max}$. Hardware synchronization at exposure time makes that bound enforceable. Selecting a network timing technology such as the IEEE 1588 Precision Time Protocol (PTP) without first deriving this spatial error budget leaves the sensor fusion pipeline ungrounded.

Pitfall: *Inferring a critical read's deadline from average camera bandwidth.*

A comfortable average utilization figure says nothing about how long a critical read stalls behind synchronized sensor capture and inference bursts. On the mobile manipulator, safety permission checks execute on a dedicated real-time microcontroller reading from local memory, shielding the immediate control path from host bus contention. The danger instead appears on the application processor, where concurrent DMA transfers, memory bus saturation, and inference bursts delay the sensor ingestion pipeline (section 8.5). This contention inflates the capture-to-dispatch age of the perceptual observation and stretches the renewal interval between successive updates. System architects must verify worst-case capture-to-dispatch age and end-to-end renewal intervals under peak burst conditions rather than relying on average throughput metrics.

Fallacy: *Assuming an end-to-end Vision-Language-Action (VLA) policy implicitly learns to compensate for rolling shutter and sensor latency without explicit temporal tokenization.*

Deep imitation learning often assumes that large Vision-Language-Action (VLA) policies trained on extensive robot teleoperation datasets will automatically learn action corrections that compensate for sensor distortion and communication latency. In practice, demonstration datasets aggregate trajectories collected across fluctuating chassis velocities, camera frame rates, and host bus loads. If the model receives only raw pixel arrays without capture timestamps or row readout schedules, it cannot distinguish true physical obstacle geometry from rolling-shutter scanline shear or motion lag. Unresolved temporal ambiguity degrades closed-loop trajectory tracking and induces control oscillations. The quadratic compounding bound of equation 5.1 describes how per-step imitation error grows over a horizon, but it assumes a stationary Markov process rather than unmodeled phase lags and sensor dynamics. Providing explicit timing metadata, such as capture time deltas ($\Delta t = t_{\text{current}} - t_0$), readout offsets, and forward-propagated kinematic states, gives the network the causal context required to resolve these distortions. Even with temporal tokenization, empirical evaluation must verify closed-loop stability, and independent out-of-band safety permission checks remain mandatory.

Pitfall: *Treating neural depth network outputs as calibrated geometric covariances without empirical residual validation.*

Monocular depth estimation backbones and bird's-eye-view (BEV) lifting transformers output continuous depth fields or discrete depth-bin categorical distributions. Engineers often mistake these softmax spreads or secondary network confidence heads for calibrated geometric uncertainties (Σ_{body}), passing them directly into a state estimator such as an extended Kalman filter or a safety filter such as a control barrier function. In practice, raw neural confidence scores correlate poorly with true physical error distributions. When presented with unfamiliar textures, specular reflections, or occluding object silhouettes, depth networks produce gross multimodal outliers that deviate by tens of standard deviations from physical ground truth. Downstream clearance insets (C_{inset}) must never rely on raw neural variance. Safe execution requires empirical validation against heavy-tailed residual distributions and hard spatial bounds cross-verified by active ranging sensors such as LiDAR or radar.

Pitfall: *Relying exclusively on line-of-sight optical sensing for manipulation without tactile contact feedback.*

During reaches and grasps, the robot's own fingers, palm, and wrist occlude the contact interface. Optical cameras cannot observe local normal forces, shear stress distributions, or incipient micro-slip at the boundary between the gripper and the object. Relying exclusively on high-resolution visual perception to ensure grasp stability leads to dropped objects from undetected slipping or crushed workpieces from excessive actuator torque. Tasks that involve physical interaction and delicate object handling require direct contact feedback. Elastomeric tactile sensors and optical skin arrays can track surface shear and incipient micro-slip, while other applications may rely on wrist force-torque transducers, compliant passive mechanics, or fixtured tooling. Sensor selection must reflect the

physical contact dynamics of the manipulation task rather than defaulting to line-of-sight optical cameras.

Fallacy: *Assuming active LiDAR and depth sensors are immune to specular reflections and multipath interference in built environments.*

A common misconception is that active time-of-flight LiDAR and depth sensors are immune to optical artifacts because they project their own structured energy. In industrial facilities and built environments, surfaces routinely violate the diffuse Lambertian scattering assumption. Polished concrete floors, metallic shelving, architectural glass partitions, and shrink-wrapped pallet freight cause severe specular reflection. Emitted laser pulses deflect away from the receiver at shallow incidence angles, causing complete signal loss and creating artificial dropouts ("puddle black holes") on polished or wet floors. Conversely, multi-bounce reflections bounce pulses across multiple reflective surfaces before returning to the detector, generating phantom obstacles that appear meters behind solid walls or beneath the floor plane. The observation pipeline must validate return intensity distributions and cross-corroborate surface geometry against complementary inertial and radar channels.

8.10 Summary

A perception measurement is actionable only when its capture epoch, clock synchronization bound, coordinate frame, and spatial uncertainty match the kinematic requirements of the intended motion. Sensor selection begins with the physical quantity demanded by the control decision: photometric appearance, metric range, Doppler radial velocity, or contact deformation. Physical capture constraints then determine the exact temporal and geometric validity of that measurement. For rolling-shutter cameras, row-by-row readout creates geometric distortion that must be compensated before triangulation, while an optical bearing ray provides directional bearing but no distance until stereo disparity or active time-of-flight ranging resolves depth. Furthermore, calibration matrices remain valid only within the physical and environmental envelopes under which they were characterized. Latency between exposure and downstream dispatch directly consumes the vehicle's stopping budget. At the drive limit, the first five rows of that budget leave 187.1 mm of the 1.10 m clear distance, a remainder that must still absorb the dispatch-to-admission interval, the jerk-limited stop, and the tracking inset. Whether the machine may drive at that speed is therefore a question this chapter hands forward to section 13.6.2 and section 12.3, which close the budget. Table 12.1 lists every term with the chapter that prices it. Because an embodied system continues to travel while sensor data moves through serialization, transport, neural inference, and planner admission, unmeasured pipeline delays erode physical clearance margins just as severely as spatial measurement error.

Spatial error deducts from that same physical clearance budget. Stereo depth uncertainty scales quadratically with distance under fixed disparity noise, while angular extrinsic misalignment introduces lever-arm errors that widen linearly with range. System designers cannot rely on nominal datasheets or raw model outputs to establish safety bounds. Zero-mean covariance matrices frequently ignore systematic calibration biases, and classification confidences or learned depth logits do not represent calibrated tail probabilities. Establishing a rigorous 99th-percentile clearance inset requires empirical residual quantiles evaluated directly against ground-truth measurements across the operational domain. Perception delivers individual dated, localized, and qualified measurements; integrating these discrete observations into a coherent state estimate that remains reliable between sensor arrivals is the responsibility of spatial memory.

Silent failures present the most severe operational hazards in physical systems, precisely because dropped frames or overt hardware faults trigger immediate system alerts, whereas misdated frames, saturated detectors, or hallucinated traversable paths arrive without raising error flags. To guard against corrupted data, the perception pipeline enforces an explicit observation contract. Downstream consumers inspect and validate the capture timestamp, clock synchronization bound, reference frame, calibration identifier, error model, and hardware health flags before reading payload buffers. Even with rigorous contracts, confirming semantic truth requires corroboration across independent, complementary sensing modalities. Seven relations carry the chapter's quantitative argument, and table 8.6 records what each bounds or decides.

Table 8.6: Key equations of sensor perception: The relations that turn a sensor's timing and geometry into travel and spatial error, and set the clearance a consumer holds, with the place each is derived.

Equation	What it bounds or decides	Derived
$\Delta\mathbf{p} = \int_{t_{\text{capture}_0}}^{t_{\text{capture}_0}+t_{\text{age}}} \mathbf{v}(t)\,dt$	The travel an observation's age costs before any consumer can react; at constant speed, $v\,t_{\text{age}}$ enters the stopping budget	equation 8.5
$t_{\text{age}} = \frac{1}{2}t_{\text{exp}} + t_{\text{readout}} + \cdots + t_{\text{IPC}}$	The capture-to-dispatch age of serialized stages, and which stage dominates it; overlapping stages need a measured interval instead	equation 8.7
$\Delta\mathbf{x}(y) = \mathbf{v}\,y\,t_{\text{row}}$	The along-track shear a rolling shutter records, which must be removed before the estimator spends δ_{loc} on it	equation 8.1
$\lvert\delta Z_c\rvert \approx \frac{Z_c^2}{fb}\lvert\delta d\rvert$	How fast stereo depth error grows with range, and so the range beyond which one standard deviation consumes an allowance	equation 8.8
$C_{\text{inset}} = C_{\text{safe}} + \lVert\epsilon_{\text{spatial}}\rVert_{\text{p99}}$	The clearance a consumer holds so the true obstacle stays outside C_{safe} except with the measured tail probability	equation 8.9
$\Delta\mathbf{p} = \mathbf{v}\,\delta t + \frac{1}{2}\mathbf{a}\,(\delta t)^2$	The position error a misdated observation carries silently, the reason a wrong timestamp is worse than a lost frame	equation 8.11
$k_\epsilon = \lceil (n_{\text{cal}}+1)(1-\epsilon) \rceil$	Which sorted residual sets a clearance inset with coverage $1-\epsilon$, and the fewest residuals that can set one	section 8.6.2

Key Takeaways: Spatial perception and age of information

- **Observation age begins at capture and is paid in travel:** Hardware-latched exposure timestamps, row readout schedules, and bounded clock conversion enable downstream consumers to compute a qualified upper age bound. Every microsecond elapsed from photon arrival to actuator response translates directly into unobserved travel ($d_{\text{blind}} = v \cdot \tau_{\text{delay}}$) deducted from the physical clearance margin.
- **Choose the measurement for the decision:** Photometric appearance, metric range, Doppler radial velocity, high-rate edge transitions, and tactile contact deformation address distinct physical questions. Multi-sensor fusion improves system resilience only when sensor spatial coverage and correlated common-cause failure modes are rigorously qualified.
- **Fix the geometry before interpreting it:** Rolling-shutter readout shears geometric edges under vehicle motion ($\Delta x = v_{\text{base}} \cdot y \cdot t_{\text{row}}$), requiring per-row pose compensation prior to spatial triangulation. Mechanical vibration and thermal cycling also induce extrinsic calibration drift. An extrinsic shift of 0.005 rad at 10 m projects to 50 mm laterally, as large as the entire localization allowance δ_{loc}. Consequently, every calibration record must carry an explicit validity envelope that consumers verify prior to inference.
- **A wrong timestamp is worse than a lost frame:** A dropped frame is an overt failure that triggering systems can handle gracefully, whereas a plausible but misdated frame silently displaces an obstacle's reconstructed position while reporting deceptively narrow uncertainty. Hardware capture timestamps, clock synchronization error bounds, sequence continuity checks, and age validation must always precede payload interpretation.
- **Claimed error must be measured:** Softmax classification confidences and depth-bin logits do not characterize spatial tail risk, and zero-mean covariance matrices fail to capture systematic calibration bias. Bounding collision risk requires empirical residual quantiles evaluated against high-precision ground truth across operational conditions to establish defensible clearance insets.
- **The contract bounds what a consumer can refuse:** The observation contract packages capture epoch, clock conversion bounds, reference frame, calibration identity, error bounds,

and hardware health flags so the planning permission path can reject untrustworthy claims before reading payload data. Even a fully operational sensor can report a clear path that does not exist, and structured observation tokens do not guarantee that downstream policies compensate for transport latency. Establishing semantic truth requires independent cross-modal verification, and spatial memory must synthesize these dated snapshots into a coherent state that persists through occlusion.

Perception supplies the grounded evidence that physical action needs: detected obstacles, identified affordances, and measured range and velocity across complementary modalities. Its output is still a sequence of isolated measurements, each bound to a capture epoch and a sensor frame, and each qualified for the instant it was captured. A qualified measurement states where something was, how old that knowledge is, and how far to trust it. It never states where that thing is now, and when a worker steps behind a rack upright, direct perception falls silent.

The observation contract is where Part III's law, proposal is not permission, first takes physical form. A confident detection does not grant permission to move. The permission path admits motion only on evidence whose age, frame, and error it can check against the clearance that remains, and it refuses everything else before reading the payload. Every later stage in this part inherits that discipline, because each one consumes a dated claim that it did not produce and cannot refresh.

8.11 Historical Perspective and Further Reading

The geometric foundations of computer vision trace back to photogrammetry and geodesy, where surveyors established rigorous mathematical formulations for optical distortion and perspective projection long before digital processors or mobile robots existed. In the late 1980s, Tsai (1987) made accurate geometric calibration feasible on off-the-shelf industrial cameras and lenses by developing computationally efficient two-stage optimization algorithms. A decade later, Zhang (2000) democratized the process by demonstrating that a camera could recover its intrinsic parameters and radial distortion from a planar checkerboard pattern observed at several unconstrained orientations. This formulation transformed camera calibration from an exotic cleanroom procedure into a practical workshop routine. Concurrently, Hartley and Zisserman (2003) codified the multiple-view projective geometry that underpins stereo disparity, epipolar constraints, and triangulation error. As cameras mounted directly onto robotic manipulators, hand-eye calibration techniques (Shiu and Ahmad 1989; Tsai and Lenz 1989) resolved the rigid kinematic transformations relating camera frames to robot end-effectors. Furgale et al. (2013) generalized this problem to continuous-time systems by integrating temporal delay estimation directly into spatial extrinsics, demonstrating that camera-to-inertial sensor time offsets must be estimated as dynamic parameters rather than assumed to be zero. Together, these developments established that sensor calibration is never a static constant retrieved from an assembly drawing, but an evolving state estimate bounded by physical drift, mechanical compliance, and thermal expansion.

Sensor hardware diversified along a parallel trajectory as real-time autonomy demanded richer, faster environmental feedback. The autonomous driving competitions of the mid-2000s established scanning laser rangefinders as the primary metric sensor for navigation in unstructured environments (Thrun et al. 2006). Mechanically spinning multi-beam LiDAR systems delivered direct, dense depth measurements, yet their physical rotation introduced significant motion distortion across individual scan lines, which requires per-point timestamping and motion unwinding as detailed in section 8.4.2. To overcome the bandwidth and latency limits of synchronous frame-based sensing, neuromorphic engineering (Mead 1990) introduced event-based vision sensors featuring independent pixels that emit asynchronous signals only when local log-intensity changes exceed a threshold (Lichtsteiner et al. 2008). Over the subsequent decade, algorithmic frameworks for event cameras (Gallego et al. 2022) revealed their microsecond-level temporal resolution and high dynamic range. The same work exposed their central limitation, complete blindness to static scenes in the absence of relative motion. Operating several high-rate sensors of different types at once required rethinking temporal coordination. Clock synchronization had evolved as a classical problem in distributed computer systems (Lamport 1978), but physical embodiment demanded hardware-level precision. The IEEE 1588 Precision Time Protocol (IEEE 2002, 2020a) addressed this requirement by embedding

hardware timestamping directly into the network physical layer, enabling modern LiDARs, cameras, and compute nodes to align their measurement epochs to a shared master clock that real-time safety monitors can verify.

The third pillar of reliable physical perception is the formal qualification of error. Traditional robotic state estimation frequently assumed Gaussian noise models that systematically underestimated the probability of catastrophic tracking failures in complex environments. Conformal prediction (Vovk et al. 2005) addressed this vulnerability by proving that exchangeable calibration residuals yield distribution-free prediction regions with exact, non-asymptotic coverage guarantees. Angelopoulos and Bates (2023) made these theoretical foundations accessible to systems practitioners, establishing split-conformal methods as practical tools for safety-critical clearance verification. Observation contracts unite these three lineages. Each contract packages recent calibration revisions, formal network synchronization bounds, and empirical residual distributions. Downstream planning controllers and safety monitors can verify these empirical bounds explicitly rather than placing blind trust in raw transducer outputs.

For further reading, six foundational works elaborate on the geometric modeling, hardware timing, and statistical error qualification developed across this chapter:

- Hartley and Zisserman (2003) is the classic reference for projective geometry, epipolar constraints, and triangulation, providing the theoretical basis for the stereo depth uncertainty relations analyzed in section 8.6.
- Szeliski (2022) offers an exhaustive survey of computational image formation, camera calibration, and stereo correspondence from the computer vision perspective, detailing the optical aberrations and sensor transduction phenomena underlying section 8.2.
- Zhang (2000) presents the ubiquitous planar-pattern calibration method, outlining the closed-form solutions and nonlinear refinement procedures that generate the intrinsic calibration records used throughout production pipelines.
- Furgale et al. (2013) demonstrates continuous-time batch state estimation for multi-sensor suites, proving why temporal offsets between cameras and inertial measurement units must be modeled and optimized as explicit unknowns.
- Gallego et al. (2022) provides a thorough review of event-based sensing, tracing hardware architectures from analog pixel circuitry to contemporary tracking and SLAM algorithms while outlining the latency-bandwidth trade-offs relevant to high-speed robotic safety.
- Angelopoulos and Bates (2023) serves as a lucid, practical guide to conformal prediction and distribution-free risk control, clearly articulating the exchangeability conditions that physical distribution shifts such as floor wear, rain, or abrupt lighting changes can compromise in real-world deployment.

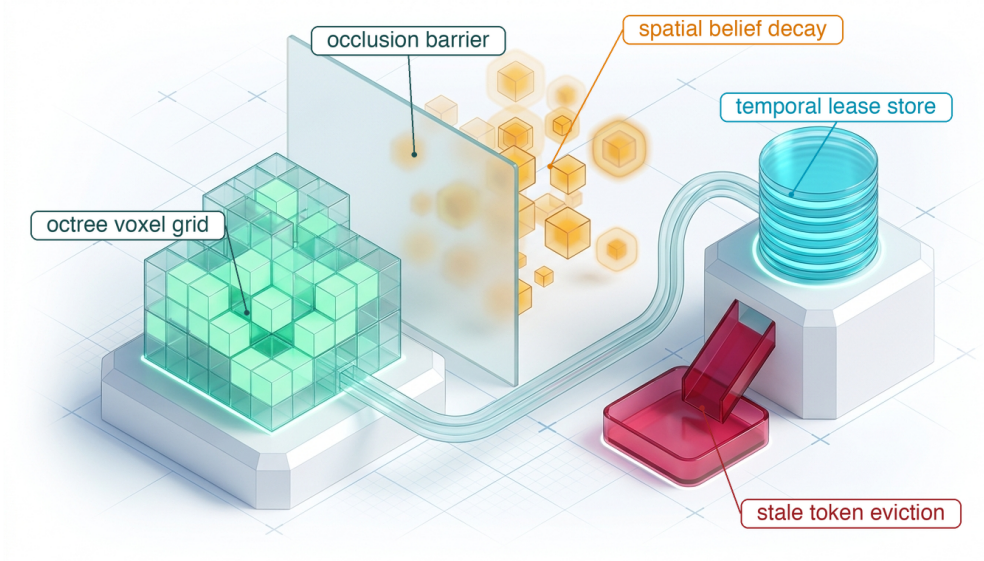
What's Next: From sensory photons to spatial memory

How long may a dated measurement keep authorizing motion after the scene that produced it goes out of view? Perception answers only for the instant of capture. When the mobile manipulator pivots past a rack upright, the cross aisle leaves the cameras' view, and a worker can step into the corridor that the last frame recorded as clear. Chapter 9 takes the observation contracts this chapter produced and fuses them, each at its own capture epoch, into a state estimate whose uncertainty grows while no new evidence arrives. That growth sets the deadline after which a remembered empty aisle can no longer support motion, and past that deadline the permission path refuses any proposal that relies on it.

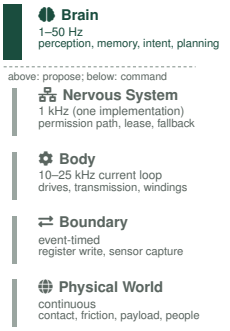
DRAFT

9

Spatial Memory



9.1 Spatial Belief Persistence and Expiry
9.2 Estimating State from Dated Measurements
9.3 The Spatial Memory Handoff and State Invariants
9.4 State Epochs and Temporal Anchoring
9.5 Coordinate Transform Expiry and Localization Drift
9.6 Certified Spatial Assertions in Scene Memory
9.7 Forward State Propagation and Occlusion Decay
9.8 Observability Ceilings and Estimation Limits
9.9 Spatial Belief Failure Modes and Map Corruption
9.10 The Belief Record Schema
9.11 Fallacies and Pitfalls
9.12 Summary
9.13 Historical Perspective and Further Reading



Purpose

How long can a robot act on a remembered scene after it loses sight of it?

A person hidden behind a storage rack can keep moving while the robot's map still shows an empty aisle. Forgetting the person loses a constraint on motion; preserving the last position indefinitely turns old evidence into false certainty. The engineer must estimate the scene from measurements that arrive at different times with different errors, retain that estimate through brief sensing gaps, and limit how long it can support a decision. That limit depends on the motion the sensors cannot observe, the error already present, and the clearance the task can tolerate. A stored estimate must therefore carry both its physical age and the conditions under which its uncertainty remains bounded. The permission path needs that information in time to refuse motion whose evidence has expired. Spatial memory connects sensing to the causal boundary by preserving what an observation supports without extending its authority beyond the physical margin.

Learning Objectives

- Apply a state estimator's predict and update steps to dated measurements, including late ones at their capture epoch, and test whether its reported covariance matches its actual error
- Track objects across frames without withholding occupancy evidence until their identity is confirmed
- Specify a belief record that carries evidence age, uncertainty growth, and consumer query budgets
- Compose transform chains at a common epoch, and trace how localization drift and loop closure move remembered map cells without renewing their evidence
- Select a spatial representation for a clearance query from the query it answers, the memory reads it costs, and the storage it needs
- Design spatial updates that preserve evidence age and distinguish missing observations from verified free space
- Set belief expiry from bounded uncertainty growth and the task's physical tolerance
- Diagnose frozen transforms, phantom tracks, and unobservable drift that pass freshness and innovation checks

9.1 Spatial Belief Persistence and Expiry

On the warehouse mobile manipulator, intermittent observation becomes a hazard at every rack corner. As the robot pivots into a turn, structural uprights occlude the cross aisle from the navigation cameras, and whatever the vision backbone detected seconds earlier survives only in memory. A worker stepping around the corner can enter the corridor that memory still records as clear. Republishing the cached perception frame with a fresh software timestamp cannot replace the missing sensory evidence. The stopping budget assumes obstacles remain within declared clearances, and a person who arrives unseen violates that assumption. The open question is how long the remembered empty aisle may keep authorizing motion after the uprights hide it, and whether any amount of remembering can make the blind crossing safe.

Every physical AI system acts on a world it observes only intermittently. Perception delivers dated, calibrated observations, yet sensor frames arrive tens of milliseconds apart, a rack upright or a gripper finger can occlude a target surface for seconds, and the controller must keep issuing commands throughout those blind intervals. **Spatial memory** carries physical state across these perceptual gaps. In a conventional computing system, a value in memory remains valid indefinitely because external reality does not alter the digital state. In an embodied system, memory retains an exact representation of the past while the physical world continues to move. A cached state becomes stale not because bits decay, but because unobserved reality drifts away from the recorded value. The central operational question of spatial memory is how long a stored state may continue to authorize machine motion. The answer is a physical deadline rather than an arbitrary software timeout, determined by the initial measurement error, the rate at which unobserved dynamics can expand that error, and the clearance margins permitted by the task. Computing and enforcing that deadline requires five capabilities: an estimator that fuses dated measurements into a calibrated state distribution, a memory record that couples every geometric value to its capture epoch and an error-growth law, coordinate transforms and map structures that preserve those timestamps across aggregation, an invalidation mechanism derived directly from the growth model, and an accounting of unobservable environmental drift.

An embodied system cannot operate purely on instantaneous sensor frames, nor can it rely entirely on the implicit context cached inside a learned policy network. Instead, the architecture requires an explicit **belief**: an estimate of physical state projected forward from historical sensory evidence and paired with an explicit model of expanding uncertainty.[1]

↩ Observations dated by capture epoch and carrying calibrated spatial covariances are established in section 8.8.

1 **Belief State**: Åström (1965) showed that a controller that cannot observe its state directly loses nothing by acting on the conditional distribution of that state given everything observed so far, the formalization encountered in section 6.5.1 as a partially observable decision problem. However, an abstract mathematical distribution carries no native reference frame, evidence epoch, or error-growth model. A physical runtime requires all three to verify whether a clearance corridor remains valid during a sensor dropout.

Definition 9.1: Belief state

Belief state is an estimate of a physical quantity carried forward from dated sensor evidence, expressed within an explicit reference frame, accompanied by the evidence epoch t_0 of the most recent constraining measurement and a declared error-growth law $E(\Delta t)$ that models uncertainty expansion in the absence of new data.

1. **Significance**: A belief maps elapsed time into spatial uncertainty. The safety enforcement path permits motion only while $E(\Delta t)$ remains within the task's clearance envelope. Every belief therefore carries an invalidation horizon measured from t_0, beyond which the safety controller must reject any dependent trajectory.
2. **Distinction**: A sensor measurement captures evidence from a single discrete instant. A belief estimates state at later points in time through forward propagation, remaining valid only as long as its growth law accurately bounds physical drift.
3. **Common pitfall**: Mistaking publication recency for evidence freshness. Re-evaluating or republishing a belief updates software message timestamps, but only a new physical sensor measurement advances the evidence epoch t_0.

Spatial memory is the second of four proposal-side stages in Part III, positioned between perception and intent. Because it resides on the untrusted proposal side of the proposal boundary, spatial memory can shape proposed trajectories but cannot certify its own freshness. The permission path must independently evaluate the age of the underlying evidence against a local monotonic clock.

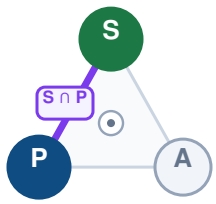
Learned policies maintain internal history through recurrent states or transformer attention buffers, but these mechanisms keep or drop tokens according to a capacity budget rather than physical persistence. A transformer's context window, and the key-value cache that stores it, holds a bounded number of tokens, so older tokens are evicted as new frames arrive (the streaming-window policies of section 9.6.3). When an obstacle steps behind a pallet rack, the tokens that recorded it can fall out of the window while newly observed features take their place, even though that obstacle continues to obstruct the physical corridor. An internal neural context window is therefore an unreliable sole record of physical state. Spatial memory decouples persistence from policy attention, explicitly preserving the entity, its last observed geometry, and an uncertainty volume that expands for as long as it remains occluded.

Externalizing physical state from the neural policy also cleanly decouples downstream consumers with divergent computational constraints. The motion planning policy queries a host-resident scene representation to extract a compact working set tailored to its candidate trajectory. In contrast, an isolated permission controller requires a strictly bounded local safety snapshot stored directly in its local memory to evaluate real-time clearance queries. This snapshot retains the raw evidence epochs, spatial uncertainties, geometric coverage bounds, and validity regions from the underlying beliefs. Replicating state across memory buses does not refresh its physical validity. The safety controller neither requires the complete global map nor waits for host queries to complete before deciding whether an ongoing motion remains admissible.

Internal software consistency does not guarantee physical geometric validity. To prevent an actuator from executing commands based on obsolete assumptions, an embodied memory system must elevate state expiration to an explicit interface event. Every stored belief carries an invalidation horizon, the deadline at which accumulated uncertainty consumes the task's clearance margin (section 9.7.2). When that deadline passes, the belief immediately ceases to authorize motion. The permission path rejects any proposal that relies on the expired state and selects a fallback qualified for the remaining evidence, actuation authority, and clearance, such as a controlled stop.

Every belief and its corresponding invalidation deadline originate in a state estimator. The perception front-end streams discrete, timestamped, coordinate-framed measurements annotated with calibrated sensor covariances. Transforming these intermittent observations into a continuous physical state, backed by uncertainty bounds that faithfully reflect unobserved drift, is the first job of spatial memory.

↩ The proposal boundary, above which learned components may only propose and below which the permission path alone admits commands, is established in section 1.7.



Belief carries sensing across the gap to planning.

9.2 Estimating State from Dated Measurements

A single LiDAR return or stereo match localizes the rack-end upright at one isolated instant. A physical safety permission path, however, requires a continuous range estimate that stays current between sweeps, fuses observations across heterogeneous sensors, and produces an uncertainty metric whose statistical confidence matches physical reality. A recursive **state estimator** satisfies all three demands by alternating two complementary operations: a prediction step that projects the state forward through a dynamic motion model, and an update step that incorporates each newly arrived, dated measurement. The discrete Kalman filter[2] (Rudolph Emil Kalman 1960) provides the classical formulation of this recursion, establishing the mathematical foundation for this chapter (Thrun et al. 2005, ch. 3; Bar-Shalom et al. 2001). Multi-object tracking and delayed-observation handling apply these two steps directly. The belief decay under occlusion examined in section 9.7 corresponds to the prediction step executing without measurement updates, while the observability limits of section 9.8 define the state dimensions that no update stream can reconstruct. The mechanics are clearest on a scalar, the remaining distance s from the mobile base to the rack-end upright, so the two operations are worked through on it before the notation extends to full state vectors.

[2] **Kalman Filter**: Stanley Schmidt's group at NASA Ames took up Kálmán's paper shortly after it appeared and, for circumlunar navigation studies, linearized the nonlinear measurement equations about the current estimate (Smith et al. 1962), the reformulation now called the extended Kalman filter. MIT's Instrumentation Laboratory then carried the filter into Apollo's on-board navigation in a square-root form suited to a computer with a short word length (McGee and Schmidt 1985). The recursion fit that computer because it keeps only the latest estimate and its covariance, never the measurement history, and the same property lets spatial memory hand a consumer one dated record instead of a log to replay.

The **predict** step advances the state estimate across time without incorporating new external evidence. Between two successive LiDAR sweeps separated by T_{nominal} = 50 ms, wheel odometry (section 9.5.1) indicates that the mobile base closed the distance by $v T_{\text{nominal}}$, yielding the prior prediction $\hat{s}^{-} = \hat{s} - v T_{\text{nominal}}$. Odometry, however, carries inherent uncertainty. An illustrative speed disturbance of 0.1 m/s (from minor wheel slip on polished concrete, for example) accumulates 5 mm of standard deviation over that single sweep interval. Because independent errors combine in quadrature, $\sigma_{-}^2 = \sigma^2 + \sigma_w^2$, an initial 20 mm uncertainty expands to 20.6 mm. Without new measurement constraints, prediction inevitably inflates state uncertainty.

The robot's inertial measurement unit (IMU) is the other source of high-rate data for the predict step, and its datasheet shows why dead reckoning cannot sustain an accurate estimate on its own. A sensor's noise density states how much random error it adds per square root of measurement bandwidth, so white noise of density N_{IMU} integrated for T seconds spreads by $N_{\text{IMU}}\sqrt{T}$, while a fixed offset integrates in proportion to T. The Bosch BMI088 gyroscope has a rate-noise density of $0.014\,^{\circ}/\text{s}/\sqrt{\text{Hz}}$ alongside an uncalibrated zero-rate bias of $1\,^{\circ}/\text{s}$. Its accelerometer has a noise density of 160 µg/$\sqrt{\text{Hz}}$ and an uncalibrated zero-g offset of 20 mg (Bosch Sensortec 2024). Integrating these sensor signals accumulates error, and stochastic noise compounds along a different trajectory than deterministic bias. Broadband white noise integrates in variance, causing orientation uncertainty to grow with the square root of time. Across 1 s without external range correction, it accumulates only 0.014°. In contrast, a constant bias integrates linearly, producing heading drift proportional to time that reaches 1° over that same second. At 1.10 m, this angular error projects into 19.2 mm of lateral displacement for the rack upright. Because acceleration undergoes double integration to give position, its bias scales quadratically with elapsed time ($\frac{1}{2} b_a T^2$). The bias generates 98.1 mm of drift after one second, about twice the 50 mm localization allowance. Integrated accelerometer white noise contributes only 0.9 mm (its position variance grows with the cube of time, a law derived in section 9.7). In this illustrative uncalibrated case, sensor offsets dominate broadband noise and limit how long the inertial system can coast uncorrected. Practical estimators therefore augment the state vector to estimate these biases online. This works only while the available measurements and executed motion constrain the relevant bias components (the observability condition of section 9.8). The estimators also use the underlying noise densities to parameterize the process noise, the matrix generalization of the odometry variance σ_w^2 that the general filter below writes as $\mathbf{Q}$. Because these offsets drift with temperature and vary across physical chips, operating parameters must be identified through in-situ stationary calibration on the physical platform rather than taken from static datasheet ratings.

The **update** step incorporates a fresh scalar measurement z characterized by variance σ_z^2. The updated posterior is a linear combination, $\hat{s} = (1-g)\,\hat{s}^{-} + g\,z$. Assuming the prior prediction error and measurement noise remain statistically independent, the posterior variance evaluates to $(1-g)^2\sigma_{-}^2 + g^2\sigma_z^2$. Minimizing this variance with respect to the blending weight g produces

the optimal Kalman gain, which balances each information source inversely against its relative uncertainty:

$$g = \frac{\sigma_-^2}{\sigma_-^2 + \sigma_z^2}, \qquad \hat{s} = \hat{s}^- + g\,(z - \hat{s}^-), \qquad \sigma^2 = (1-g)\,\sigma_-^2$$

The residual $z - \hat{s}^-$ constitutes the **innovation**, the new information supplied by the measurement that the forward projection did not capture. Given the Ouster OS1-64's worst-case datasheet range precision of 30 mm, the computed gain is 0.32. The estimator shifts approximately one-third of the way toward the raw LiDAR reading, contracting the posterior standard deviation to 17.0 mm, a modeled spread tighter than either input alone. Because published precision specifications typically reflect ideal calibration conditions rather than real-time operating modes, this sensor variance must be verified at the configured operational scan rate. Furthermore, the sensor's separate 25 mm systematic accuracy offset represents a structural bias that zero-mean Gaussian models cannot accommodate; offline calibration must eliminate this offset, or the downstream safety clearance margin must absorb it explicitly.

Fusing a second sensor executes as a sequential update step. The illustrative stereo camera rig analyzed in section 8.6 achieves $\sigma_z = 3.8$ mm at the 1.10 m rack-end distance. Fusing this measurement yields a stereo gain of 0.95, shrinking the final fused standard deviation to 3.7 mm. For statistically independent measurements captured at the same physical instant, update ordering is commutative: each observation contributes an information increment, and their underlying precision matrices (inverse variances) sum directly.

A misdated measurement breaks this estimation algebra silently. Suppose a stereo depth observation is tagged with its host arrival timestamp, which section 8.3 shows hides exposure, readout, and transport delay, rather than with its physical exposure epoch. The filter erroneously pairs this observation with a robot pose 39.9 mm further along the aisle than where the photons actually struck the sensor. Consequently, the measured range reads systematically long by that spatial displacement, causing the update to drag the state estimate 38.0 mm away from the physical obstacle while reporting the same deceptively tight 3.7 mm standard deviation. The resulting physical error spans approximately 10.3 reported standard deviations in the hazardous direction, yet the filter's covariance update detects nothing amiss because algebraic variance reduction depends entirely on modeled sensor noise rather than measurement residuals. Corrupt timestamps are therefore far more dangerous than dropped sensor frames. Dropping the stereo frame instead would leave the posterior uncertainty at 17.0 mm, imprecise but statistically honest.

Set side by side on one range-error scale (figure 9.1), the stages read as a single story. Prediction widens the bar from 20 mm to 20.6 mm, each update narrows it below both of its inputs to a final 3.7 mm, and the misdated column reports that same tight spread while its center sits 38.0 mm from the true range, the only bar that does not contain the truth.

The recursion generalizes from a scalar distance to an n-dimensional state vector $\hat{\mathbf{x}}$ with error covariance $\mathbf{P} \in \mathbb{R}^{n \times n}$. It parameterizes system dynamics through a state transition matrix $\mathbf{F}$, process noise covariance $\mathbf{Q}$, an observation matrix $\mathbf{H}$ projecting state space into measurement space, and measurement noise covariance Σ_{meas} (equation 9.1 and equation 9.2):

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}\hat{\mathbf{x}}_{k-1|k-1}, \qquad \mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^\top + \mathbf{Q} \tag{9.1}$$

$$\nu_k = \mathbf{y}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}, \quad \mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \Sigma_{\text{meas}}, \quad \mathbf{G}_k = \mathbf{P}_{k|k-1}\mathbf{H}^\top\mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{G}_k\nu_k, \quad \mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{G}_k\mathbf{H})\,\mathbf{P}_{k|k-1} \tag{9.2}$$

In this formulation, ν_k represents the innovation vector, $\mathbf{S}_k$ denotes innovation covariance, and $\mathbf{G}_k$ signifies the Kalman gain matrix (designated $\mathbf{G}_k$ to reserve $\mathbf{K}$ for camera intrinsic calibration matrices). The preceding scalar analysis corresponds to the degenerate setting $\mathbf{F} = \mathbf{H} = 1$, with the odometry displacement $-v\,T_{\text{nominal}}$ entering as a known control input. Equation 9.1 omits that input term for brevity, and the term shifts the mean without changing $\mathbf{P}$. Under linear dynamics and additive Gaussian noise, this closed-form update yields the provably optimal minimum-variance unbiased estimate. If nonlinearities or non-Gaussian noise corrupt the process, the update functions merely as an empirical linear estimator whose output covariance requires experimental validation.

Physical robotic sensors are inherently nonlinear functions of platform state. A LiDAR range measurement to the rack upright evaluates to $r = \|\mathbf{p}_{\text{upright}} - \mathbf{p}_{\text{base}}\|$, while a camera records target

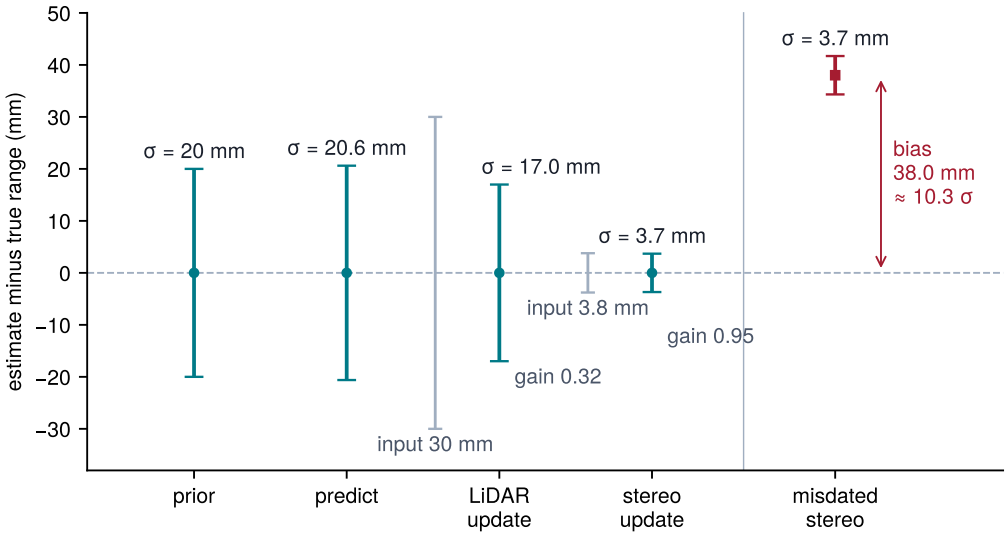


Figure 9.1: Predict and update on the rack-end range: Each bar spans one standard deviation of the range estimate. Prediction widens the bar from 20 mm to 20.6 mm; the LiDAR update (gain 0.32) and the stereo update (gain 0.95) each narrow it below both of their inputs, whose own spreads appear as gray bars. The misdated stereo update on the right reports the same 3.7 mm spread while sitting 38.0 mm from the true range, which the variance alone cannot reveal.

bearings through the nonlinear pinhole projection detailed in equation 8.2. The **extended Kalman filter (EKF)** (Smith et al. 1962) adapts equation 9.2 to nonlinear regimes by evaluating the measurement function $h(\hat{\mathbf{x}}_{k|k-1})$ directly and substituting the linear projection $\mathbf{H}$ with the measurement Jacobian $\nabla_{\mathbf{x}} h$ evaluated at the current state prediction. For a scalar range reading, this Jacobian with respect to platform base position reduces to the unit line-of-sight vector directed from the upright toward the base. This Taylor-series linearization is a first-order approximation. Substantial curvature across the prior distribution, common when estimating bearings to nearby obstacles or operating under broad initial uncertainty, can therefore cause the EKF to underestimate its true estimation error (Julier and Uhlmann 2004). This failure mode makes the empirical consistency evaluations of section 9.2.1 safety requirements rather than optional checks.

The **unscented Kalman filter (UKF)** developed by Julier and Uhlmann (1997) bypasses analytical Jacobians through deterministic sampling. For an n-dimensional state vector, the UKF instantiates $2n+1$ sigma points structured around the matrix square root of $\mathbf{P}$, propagates each sample point through the true nonlinear observation function, and reconstructs the transformed posterior mean and covariance (Julier and Uhlmann 2004). This transformation captures higher-order curvature effects on the mean that the EKF truncates, requiring $2n+1$ functional evaluations per cycle. The UKF nevertheless approximates the posterior as a unimodal Gaussian, leaving it subject to identical qualification and consistency standards.

Both filtering frameworks, and the multi-object tracking and out-of-sequence replay built on them, depend on every sensor measurement being indexed by its true capture epoch. The misdated stereo benchmark illustrates the breakdown when this assumption fails. While reported uncertainty remained artificially compressed, true physical error drifted across ten standard deviations. A reported covariance is therefore an engineering hypothesis that must be systematically validated against real-world data.

9.2.1 Testing whether the covariance is honest

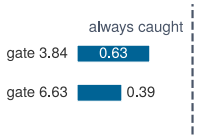
Catching an overconfident covariance like the misdated stereo filter's requires two complementary statistical tests: an offline qualification metric evaluated against ground truth before deployment, and an online runtime monitor evaluated directly against incoming measurement streams during operation. Both formulations use a zero-mean Gaussian error model with the correct nonsingular covariance. Whitening the error under these assumptions produces independent standard normal

coordinates; their squared norm, rather than the error vector itself, follows a chi-squared distribution. Matching covariance alone does not establish Gaussian tails.

A filter achieves statistical consistency when its reported covariance matches the dispersion of its actual state errors. When offline ground truth is accessible (a laser-tracker survey of the rack upright, say, with an optical motion-capture reference for the base pose), the normalized estimation error squared (NEES), $\epsilon_k = (\mathbf{x}_k - \hat{\mathbf{x}}_k)^\top \mathbf{P}_k^{-1} (\mathbf{x}_k - \hat{\mathbf{x}}_k)$, evaluates the system. Under that zero-mean Gaussian model with the correct nonsingular covariance, whitening the error of an n-dimensional state forms a vector of n independent standard normal variates. Their sum of squares follows a chi-squared distribution with n degrees of freedom and theoretical mean n. Evaluated across N statistically independent experimental trials, the scaled sample mean $N\bar{\epsilon}$ follows a chi-squared distribution with Nn degrees of freedom (Bar-Shalom et al. 2001). For the scalar rack-end distance over 50 experimental trials, the two-sided 95 percent acceptance interval for sample mean NEES $\bar{\epsilon}$ spans from 0.65 to 1.43. An empirical NEES above this band flags an overconfident filter, the safety-critical defect, because its reported uncertainty understates the error that clearance buffers must absorb. Conversely, an average falling below the lower bound indicates an underconfident filter whose inflated covariance unnecessarily throttles vehicle operating speed.

While absolute ground truth remains unavailable during live deployment, the estimator can continuously audit its measurement innovations. The normalized innovation squared (NIS), $\nu_k^\top \mathbf{S}_k^{-1} \nu_k$, follows a chi-squared distribution with m degrees of freedom for an m-dimensional measurement vector under valid model assumptions, so its expected value is m. The same statistic reappears below as the data-association gate. A bias b in the innovation adds b^2/S to that expectation, where S is the innovation variance. For the misdated stereo observation in figure 9.1, b is the 39.9 mm arrival-stamp travel, and the expected NIS rises from 1 to 6.25. A single misdated update's NIS then follows a noncentral chi-squared distribution, the bias setting its offset and the sensor noise its spread. One update therefore breaches the 95 percent validation gate of 3.84 with probability 0.63, and trips the 99 percent threshold of 6.63 only 0.39 of the time. Single-frame residual gating therefore admits a substantial fraction of corrupted measurements. Averaging independent innovations reduces sampling variation and can expose a persistent residual bias. For 50 independent scalar innovations under the valid model, the mean NIS has the same 0.65 to 1.43 acceptance band derived above for NEES. Independent repetitions of the worked misdated update average near 6.25, far above it. A sequential filter can instead absorb a systematic dating error into its state, reducing later innovations; correlated innovations also invalidate this nominal band. A runtime sliding-window NIS supervisor therefore detects dating errors that persist in the innovations, but cannot replace capture-epoch validation. Once an estimator's covariance and innovations are calibrated on a single landmark, the next challenge is maintaining separate filters across multiple moving objects without confusing their identities.

9.2.2 Multi-object tracking and data association



A per-update gate catches one misdated update only 0.63 or 0.39 of the time.

Maintaining separate filters across multiple moving objects adds an immediate architectural constraint: a perception pipeline cannot withhold an obstacle detection until a multi-frame tracker confirms its persistent identity, or a worker stepping into the aisle remains invisible to collision avoidance during the confirmation window. A fresh occupied-region claim must therefore constrain vehicle motion immediately, before perception systems identify the specific obstacle class or confirm a track. Persistent identity serves a distinct behavioral objective: correlating detections across frames allows the motion planner to estimate target velocity vectors and anticipate dynamic crossing events, while immediate spatial occupancy protects reactive clearance.

The software architecture therefore decouples immediate geometric spatial occupancy from the **tracking-by-detection** hypothesis. The tracking layer associates incoming detections, filters kinematic velocities, and tracks multi-dimensional uncertainty. Track retirement follows the lifecycle below, and how state persists when measurements stop is the subject of section 9.7. The state equations formulated below govern the kinematic information a confirmed track provides, rather than serving as a prerequisite for reactive collision avoidance.

Each active track i maintains an independent state filter tracking a kinematic state vector $\hat{\mathbf{x}}_t = [\hat{\mathbf{p}}_t^\top, \hat{\mathbf{v}}_t^\top]^\top$ with error covariance $\mathbf{P}_t \in \mathbb{R}^{6\times6}$. A discrete constant-velocity transition matrix $\mathbf{F}$ and process noise covariance $\mathbf{Q}$ advance this state between sensory frames through the prediction step

of equation 9.1, which carries the mean forward and grows the covariance by the process noise. When an incoming perception frame yields n_{det} candidate detections $\{\mathbf{y}_{t,j}\}_{j=1}^{n_{\text{det}}}$ (the frame index t plays the role of the step index k in equation 9.1), the tracker must resolve **data association**,[3] mapping candidate detections to established state tracks. To prevent spurious sensor noise or distant clutter from corrupting legitimate tracks, the tracker evaluates the squared Mahalanobis distance d_M^2 between detection $\mathbf{y}_{t,j}$ and the projected measurement $\mathbf{H}\hat{\mathbf{x}}_{t|t-1}$:

$$d_M^2 = (\mathbf{y}_{t,j} - \mathbf{H}\hat{\mathbf{x}}_{t|t-1})^\top \mathbf{S}_{ij}^{-1} (\mathbf{y}_{t,j} - \mathbf{H}\hat{\mathbf{x}}_{t|t-1}) \tag{9.3}$$

where $\mathbf{S}_{ij} = \mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^\top + \Sigma_{\text{meas},j}$ defines the innovation covariance from equation 9.2 and $\Sigma_{\text{meas},j}$ specifies the measurement noise covariance; this validation metric is identical to the NIS metric defined in section 9.2.1. An ellipsoidal **validation gate** evaluates equation 9.3, rejecting any pairing where $d_M^2 > \gamma_{\text{gate}}$. The gating threshold $\chi_3^2(1-\alpha_{\text{sig}})$ possesses three degrees of freedom corresponding to three-dimensional Cartesian position measurements, retaining its theoretical significance only while residual Gaussian assumptions hold. For all candidate pairings satisfying the gate, the tracker builds an assignment cost matrix with entries $c_{ij} = d_M^2$ and computes the global minimum-cost one-to-one match. It uses either the Hungarian algorithm (Kuhn 1955), as the Simple Online and Realtime Tracking (SORT) baseline does (Bewley et al. 2016), or the Jonker-Volgenant shortest-augmenting-path solver (Jonker and Volgenant 1987). The match is optimal only for the costs it was given; it does not guarantee that each pairing is physically correct.

[3] **Data Association in Clutter**: Radar tracking met this problem first. Bar-Shalom and Tse (1975) showed that a filter which commits to the single nearest return inside its gate can follow clutter and lose the target, and they weighted every gated return by its probability of having come from the target instead. A tracker that commits to one assignment per frame, as this one does, reveals a wrong pairing only through inconsistent innovations, so a confirmed track's covariance must still pass the tests of section 9.2.1.

The tracking lifecycle organizes target hypotheses through three discrete states:

1. **Tentative state**: Unassociated detections instantiate tentative candidate tracks. Because sensor artifacts (specular reflections, dust particles, optical lens flare) generate transient false positives, candidate identities remain tentative until satisfying an M-of-N_{conf} frame confirmation policy (associated in at least M of the last N_{conf} frames). Raw geometric occupancy claims remain immediately accessible to vehicle collision-avoidance buffers throughout this confirmation window. If a downstream behavioral planner requires confirmed object identity or calibrated velocity, its latency allocation must account for this multi-frame confirmation interval rather than relying solely on the single-frame pipeline latency described in section 8.4.1.
2. **Confirmed state**: Upon passing confirmation criteria, the track registers as confirmed and publishes its complete kinematic state $(\text{ID}_i, \hat{\mathbf{p}}_i, \hat{\mathbf{v}}_i, \mathbf{P}_i)$ to planning and safety subsystems. Each newly associated measurement refines the track via equation 9.2, correcting the mean vector and contracting the error covariance matrix. Because this update achieves minimum variance only under linear-Gaussian assumptions, tracked covariances must continuously satisfy the consistency criteria established in section 9.2.1.
3. **Coasting state (occlusion)**: When an obstacle moves behind storage shelving or suffers transient occlusion by another vehicle, object detection fails. Rather than immediately dropping the entity, the tracker transitions the hypothesis into an unobserved coasting state. During coasting, state propagation proceeds strictly through kinematic motion model $\mathbf{F}$ in equation 9.1, while process noise $\mathbf{Q}$ progressively inflates state covariance in the absence of measurement constraints. Downstream path planners must compensate for this increasing uncertainty by applying progressively widening clearance margins. If no matching observation associates within timeout window T_{lost}, the active track terminates. Terminating a track reflects the loss of persistent identity rather than the clearance of physical space, so the spatial map must retain an appropriate unknown-space constraint (section 9.6). The rate at which coasting uncertainty degrades spatial traversability is the belief expiry problem formulated in section 9.7.

To support real-time planning under motion uncertainty, the tracker continuously serializes an auditable **dynamic track manifest** $[(\text{ID}_k, \hat{\mathbf{p}}_k, \hat{\mathbf{v}}_k, \mathbf{P}_k, t_{\text{last}}, \text{state})]$. The intent layer consumes these velocity vectors and covariance ellipses to bound the region each object could occupy over the planning horizon, while real-time safety enforcement systems compute certified stopping clearances against moving dynamic obstacles. Even when a detection associates cleanly with the right track, however, vision inference delay means that camera detections routinely arrive after newer LiDAR sweeps and IMU samples have already advanced the filter.

↪ The track manifest's velocities and covariances are developed as forward occupancy sets in section 10.6.1.

9.2.3 Applying a delayed observation

Fusing such a delayed visual detection directly into the estimator's current state pulls the position estimate backward toward where the obstacle stood at exposure time and corrupts the velocity vector. The physical capture epoch preserved by the observation contract enables the estimator to integrate these delayed measurements at their historical capture instant instead, provided the system maintains a calibrated state history and dynamic motion model. Rolling the filter back to that capture instant applies the update of equation 9.2 to the historical state estimate $\hat{\mathbf{x}}$, error covariance $\mathbf{P}$, observation model $\mathbf{H}$, and Kalman gain $\mathbf{G}_k$.

A delayed visual observation must correct the state at its true historical capture epoch before the estimator can project the correction forward to current wall-clock time. Robust systems implement this mechanism using an **out-of-sequence ring buffer**, a circular array whose slot k holds the prediction $\hat{\mathbf{x}}_{k|k-1}$ and covariance $\mathbf{P}_{k|k-1}$ for IMU sample k, the IMU input $\mathbf{u}_k$ that drove it, and that sample's capture time. The buffer needs $N_{\text{buffer}} = \lceil T_{\text{buffer}}/\Delta t_{\text{IMU}} \rceil$ slots, where Δt_{IMU} is the IMU sample period and the buffer horizon T_{buffer} is the oldest observation age the estimator will still accept. On the running machine, the IMU samples at 1 kHz ($\Delta t_{\text{IMU}} = 1$ ms), so covering the navigation camera's 51.6 ms observation age takes 52 slots. A consumer must either interpolate the buffered state to the observation's actual capture time or charge the bounded nearest-sample timing discrepancy to the error model. In the equations below, subscripts index IMU samples; t_0 and t_{recv} name the samples whose capture times are nearest the observation's capture epoch and its receipt. When an out-of-sequence observation $\mathbf{y}_{t_0}$ arrives at receipt sample t_{recv}, the estimator rolls back to sample t_0, retrieves the historical prediction $\hat{\mathbf{x}}_{t_0|t_0-1}$, computes the corresponding innovation and Kalman gain, and applies the update of equation 9.2 at that historical instant (equation 9.4):

$$\hat{\mathbf{x}}_{t_0|t_0} = \hat{\mathbf{x}}_{t_0|t_0-1} + \mathbf{G}_{t_0}(\mathbf{y}_{t_0} - \mathbf{H}_{t_0}\hat{\mathbf{x}}_{t_0|t_0-1}), \quad \mathbf{P}_{t_0|t_0} = (\mathbf{I} - \mathbf{G}_{t_0}\mathbf{H}_{t_0})\mathbf{P}_{t_0|t_0-1} \tag{9.4}$$

For the forward propagation pass that follows, consider first the baseline case with no intervening updates from other sensors; in general configurations, any intervening sensor updates within the interval must be sequentially re-applied in timestamp order. Because nonlinear motion models and their linearized transition matrices $\mathbf{F}_k$ depend on the state trajectory itself, the estimator cannot simply add the historical state shift to the current estimate, and instead executes a rapid replay sweep, re-integrating the buffered IMU inputs $\mathbf{u}_k$ from sample t_0 to sample t_{recv} through the continuous motion model $\mathbf{f}(\mathbf{x}, \mathbf{u})$, the state's rate of change under input $\mathbf{u}$, using the corrected state $\hat{\mathbf{x}}_{t_0|t_0}$ as its initial condition. This forward integration synchronizes current state estimate $\hat{\mathbf{x}}_{t_{\text{recv}}}$ with the newly arrived observation without introducing non-causal state discontinuities (equation 9.5):

$$\hat{\mathbf{x}}_{t_{\text{recv}}} = \hat{\mathbf{x}}_{t_0|t_0} + \sum_{k=t_0}^{t_{\text{recv}}-1} \mathbf{f}(\hat{\mathbf{x}}_k, \mathbf{u}_k)\Delta t_{\text{IMU}} \tag{9.5}$$

with covariance propagation across the replay interval governed by unrolling the single-step covariance recursion of equation 9.1 through the discrete transition matrix product $\Phi(t_{\text{recv}}, t_0) = \prod_{k=t_0}^{t_{\text{recv}}-1} \mathbf{F}_k$ to carry the newly contracted historical covariance $\mathbf{P}_{t_0|t_0}$ and each intermediate process-noise increment forward:

$$\mathbf{P}_{t_{\text{recv}}} = \Phi(t_{\text{recv}}, t_0)\mathbf{P}_{t_0|t_0}\Phi^\top(t_{\text{recv}}, t_0) + \sum_{k=t_0}^{t_{\text{recv}}-1} \Phi(t_{\text{recv}}, k+1)\mathbf{Q}_k\Phi^\top(t_{\text{recv}}, k+1) \tag{9.6}$$

The applicability of this rewind-and-replay routine (equation 9.6) is strictly limited by ring buffer horizon T_{buffer}. If scheduling jitter or memory bus contention delays an observation such that $t_{\text{age}} > T_{\text{buffer}}$, the circular buffer has already overwritten the slot for sample t_0. In this scenario, the estimator must reject the stale measurement, permit process noise covariance $\mathbf{Q}$ to grow the predictive covariance $\mathbf{P}$, and immediately notify downstream safety paths that external visual grounding has degraded.

End-to-end vision-language-action (VLA) policies confront this identical out-of-sequence dilemma when ingesting asynchronous visual frames without an explicit state-space ring buffer. The observation tokenizer developed in section 8.8 pairs each sensory token with its elapsed time since capture, leaving formal validation of the estimate and action-permission gating to dedicated safety monitors.

Checkpoint 9.1: The estimator behind every belief

Every belief the rest of the chapter ages is the output of this estimator. These items test whether you can run it and whether you can tell when its reported uncertainty is honest.

- ☐ **Predict vs. update**: In the worked scalar drift model, can you explain why a prediction step widens the reported variance, while an independent measurement update narrows it below both the prediction and the measurement?
- ☐ **Gain estimate**: If the stereo range's standard deviation doubled from 3.8 mm, with the LiDAR-updated 17.0 mm prior unchanged, what would the stereo gain and the fused standard deviation be, and which sensor would then dominate the fused estimate?
- ☐ **Estimator consistency**: Can you explain why a misdated update leaves the reported variance unchanged, when averaging independent innovations can expose a dating error that a per-update gate misses, and why this monitor cannot replace capture-epoch validation?
- ☐ **Occupancy vs. identity**: Can you explain why a tentative track's occupied region must reach clearance consumers before confirmation, and what a decision that needs confirmed velocity must add to its detection delay?
- ☐ **Late measurements**: Can you explain what the estimator must do with a measurement whose age exceeds its ring buffer's horizon, and why?

The state estimator outputs the core data record that persistent spatial memory maintains: a kinematic state vector, its verified covariance, the physical instant the state describes, and the capture epochs of the raw sensor observations that last constrained it. Forward prediction and rewind-replay passes maintain temporal currency across execution cycles, but only external sensor measurements inject genuine physical evidence, and neither mathematical projection advances the underlying evidence timestamps. Because downstream consumers must independently audit when a stored belief expires once an obstacle slips into occlusion, this state record must carry explicit temporal provenance alongside its numerical estimates. The handoff challenge is defining the exact contract a downstream consumer requires to evaluate stored spatial beliefs when direct line of sight is lost.

9.3 The Spatial Memory Handoff and State Invariants

A position vector arriving over an inter-process communication bus cannot identify the entity, physical quantity, or units it describes. The same sequence of floating-point values could denote a workpiece center in meters, a joint angle in radians, or a contact force in newtons. A consumer requires these distinctions before executing a control law; a confusion between meters and millimeters alone scales the interpreted displacement by a factor of one thousand. The belief record therefore explicitly packages entity identity, physical quantity, and measurement units at its foundation, rather than relying on conventions hidden in the producing model or transport wrapper.

Specifying the quantity and units remains incomplete without binding the value to a concrete reference frame. Identical spatial coordinates describe entirely different physical realities depending on the origin and orientation of the coordinate system. A Cartesian position $[0.142, -0.051, 0.812]$ m read in the robot base frame names a point about 0.8 m above the chassis origin, in open air over the base. Read in the tool flange frame, the same three numbers name a point 0.812 m out along the flange's approach axis, wherever the arm happens to point; with the gripper aimed at the cage door, that point lies inside the door rather than above the chassis. Because kinematic links flex, joints move, and coordinate transforms update at varying rates across distributed nodes, a belief record must carry an explicit frame identifier. Attaching the frame allows the receiving node to query the spatial transformation tree and project the stored geometry into the local frame required by the actuator, preventing the controller from acting on geometric misalignments caused by moving kinematic chains.

Geometry and identity describe physical state at a single moment, but spatial memory must also capture the temporal anchor of that state. The record carries the evidence epoch t_0 of the physical interaction or optical exposure that last constrained the estimate, the instant at which a

sensor transduced the evidence. In distributed machine architectures, intermediate nodes often republish data, execute Kalman filter prediction steps, or run forward kinematic propagations to maintain high-rate control streams; none of these computational steps adds physical evidence. Republishing and prediction must preserve the evidence epoch; a measurement update may renew only the components it physically constrains. Consumers therefore measure evidence age against the physical world rather than the scheduling frequency of the local computer.

At the boundary of the multi-sensor state estimator (section 9.2), downstream consumers do not need to rerun the filter recursion or repeat the consistency tests of section 9.2.1; they need only the resulting state, its justified error model, and the physical evidence that constrains each relevant component. Within that recursion, a prediction step advances the estimate epoch, the instant the propagated state describes (formalized in section 9.4), while its underlying physical evidence continues aging. A fresh measurement, by contrast, renews only the state dimensions it directly constrains. The spatial memory handoff therefore obliges the producer to expose bounded growth through occlusion and withdraw claims whose operating assumptions no longer hold.

Knowing the evidence epoch t_0 establishes elapsed time $\Delta t = t - t_0$, but elapsed time alone cannot determine whether a belief is valid for a given action. Two estimates of identical age diverge at different rates depending on physical constraints. A warehouse rack end bolted to the floor maintains a stationary position; its remembered boundary can remain useful while the machine's localization bound and the fixed-rack premise hold. In contrast, an occluded person hidden behind that rack can move at walking speed and close the narrow gap beside it before the next camera frame arrives.

The minimum record fields necessary for a consumer to evaluate a stored state can be derived directly from the requirement to bound current error. Let $E(\Delta t)$ represent the upper bound of the discrepancy between the stored value and true physical state after an elapsed time $\Delta t = t - t_0$. If an estimator establishes a state at time t_0 with an initial uncertainty bound e_0, and external disturbances or unmodeled velocities cause the true state to diverge at a maximum rate r, the upper bound on error at current time t obeys the linear expansion $E(\Delta t) = e_0 + r\,\Delta t$. At the rack end, the stored spatial claim asserts that no person stands near the machine. Here e_0 is the machine's localization error, r is the walking speed at which an occluded person can advance toward the vehicle, and the safety tolerance is the depth of observed free space beyond the machine's clearance envelope. Other disturbances grow error faster. A bounded unmodeled acceleration adds a quadratic term, and random acceleration noise, integrated twice, makes position variance grow with the cube of elapsed time, both derived in section 9.7. Omitting the initial bound or the dynamic growth terms leaves the consumer unable to compute $E(\Delta t)$, making an initial bound and an explicit expansion model structurally required fields of the record.

Expressing uncertainty through physical bounds and expansion rates distinguishes a physical AI belief from a probabilistic confidence score. An object detection network might output a class confidence of 0.94, but that softmax score is not calibrated evidence (section 3.6.2), and a downstream position controller cannot convert a unitless probability into a safe actuator displacement margin (that is, how far the arm can physically move before risking a collision). A confidence score of 0.94 provides no information about whether an obstacle boundary is uncertain by 2.0 mm or by 200 mm, nor does it indicate how rapidly that boundary uncertainty grows when occlusion occurs. A physical memory record must express its initial bound e_0 and expansion rate r in concrete physical dimensions such as meters, radians, or newtons per second. These rates are valid only within a declared validity region, such as a maximum assumed acceleration or a verified contact friction coefficient. If physical operating conditions leave that region, the expansion rate no longer holds, and the belief record must be treated as invalid.

At the spring-latched cage door, the gripper finger must enter a narrow latch-release slot that the finger itself hides from the wrist camera during the final approach. Before relying on the remembered slot pose, the consumer compares the belief's current error with the slot clearance. It converts the evidence epoch into its own clock domain, includes the clock-conversion bound, and evaluates the declared growth law at the time the state will be used. This check is a continuing obligation during execution, not a one-time path-generation check. A new motion proposal whose evidence fails the check is refused without automatically revoking a still-valid active command. If the active motion itself depends on expiring evidence, the controller must begin a feasible fallback maneuver

early enough that the mechanism halts within the available clearance. The machine must therefore remain inside its recoverable set, the states from which a qualified stop still fits within the remaining clearance, a boundary that section 12.3 derives from the finished stopping budget. The belief's expiry is a limit on supported motion, not a promise that an instantaneous stop is physically possible at that deadline.

The fields a consumer needs for this check travel together as the **belief record**, the seventh record in the record chain. At this stage, the record binds an identified physical quantity, with its unit, to a reference frame, an evidence epoch, an initial error bound, an uncertainty-growth law, and the validity region in which that law holds. The sections that follow add the fields each new problem demands (the estimate epoch and clock-conversion bound, a per-consumer query-work limit, the expiry deadline, provenance, and lifecycle status), and section 9.10 assembles the complete record. A classification score, a map publication time, or a transform value alone cannot supply that contract, because none couples physical units and evidence age to an error-growth law. The record makes the claims checkable; qualification must establish that they hold. A consumer cannot wait for an upstream publisher to announce expiry. If that publisher's thread hangs or its message stream stops, the consumer must still age the evidence autonomously and enforce its own safety deadline.

Evaluating the belief record depends on calculating the elapsed time $\Delta t = t_{\text{now}} - t_0$ against physical reality. If the consumer cannot verify the exact physical instant when energy transduction occurred, the calculated error bound $E(\Delta t)$ becomes an unanchored fiction, mistaking software compute cycles for physical evidence. Protecting that anchor is the job of the next two fields, the estimate epoch and the clock-conversion bound.

9.4 State Epochs and Temporal Anchoring

A bounding box or 3D Gaussian pose emitted by a neural object detector describes where an entity was when photons struck the camera CMOS sensor, not where it is when the inference tensor arrives in memory. Every spatial claim refers to a specific physical instant, its **epoch**, and any state crossing a physical AI pipeline carries three distinct epochs before an actuator can fire. The **acquisition epoch** (t_{acq}) anchors the observation contract to physical ground truth: the exposure midpoint for a camera frame, or the precise instant a hardware register latches an optical encoder count. Every observation carries its own acquisition epoch (the capture epoch of section 8.3). For each state component used by a consumer, the evidence epoch is the latest acquisition epoch of a measurement that constrained that component; for a belief built from one observation, the two coincide. A record combining differently constrained components must preserve those epochs separately, or use a conservative shared epoch and growth bound that supports every component the query needs. The rest of the chapter speaks of the evidence epoch whenever the belief, not a single observation, is meant. Downstream processing then introduces two subsequent epochs. The **estimate epoch** (t_{est}) marks the physical instant that the estimated state describes. A kinematic predictor or state observer can advance this temporal coordinate beyond the last observation without receiving fresh measurements; the instant computation completes is a separate scheduling detail. The **publication epoch** (t_{pub}) marks the instant the serialized state record is committed to the transport bus or shared-memory ring buffer for dispatch to downstream consumers.

↩ The hardware capture stamp this anti-pattern discards is established in section 8.3.

Collapsing these three timestamps into a single receipt or transmission timestamp destroys the physical anchor. This is the publish-timestamp anti-pattern of section 2.2, in which a wrapper stamps its output with the system wall clock after exposure, transport, inference, and postprocessing have already elapsed. What memory adds is a second way to commit it. A wrapper that stamps a propagated estimate with its publication epoch, or a filter that renews t_0 when it advances t_{est}, hides the evidence age just as surely as one that stamps a raw frame on arrival. Downstream receivers then ingest a message that appears freshly minted but wraps stale evidence. The age-as-displacement relation established in section 8.4 governs this entire pipeline. While publication epochs help diagnose pipeline latency and transport jitter, only the acquisition epoch enables a motion controller to compensate for physical movement that occurred while data were in flight.

Even when a system faithfully records acquisition epochs, physical AI architectures rely on distributed processing nodes whose local oscillator clocks drift independently. Connecting to a shared bus does not synchronize these time bases; the deployed clock-conversion protocol must establish an explicit error bound under realistic network loads and fault conditions.[4] That temporal

[4] **Clock Synchronization Bound**: Controller Area Network with Flexible Data-Rate (CAN FD) bit-phase synchronization aligns bit sampling within one frame; it does not align the distributed clocks that date physical observations. Precision Time Protocol and EtherCAT Distributed Clocks provide synchronization mechanisms, but their names alone do not establish the deployed error bound, which only a measurement under the stated load and fault conditions supplies.

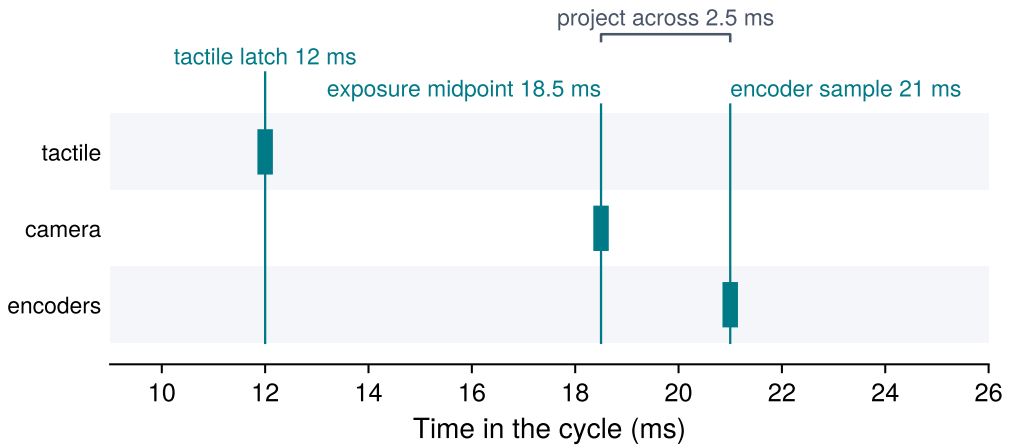


Figure 9.2: Three evidence epochs in one cycle: The tactile array, the camera's exposure midpoint, and the joint encoders each date their evidence on one clock, at 12 ms, 18.5 ms, and 21 ms in this illustrative cycle. A fused state dated to the encoder epoch must carry the camera's evidence across the 2.5 ms gap with a kinematic model and a bounded velocity; packing the three readings into one token sequence records none of these epochs.

uncertainty directly converts to spatial uncertainty scaled by the relative speed of the observed entity. The observation contract defined in section 8.8 carries this bound as its clock-conversion error, and any spatial belief derived from that observation preserves it. Every consumer adds the synchronization bound to the elapsed evidence age before judging freshness, incorporating the resulting positional uncertainty alongside sensor resolution limits and structural deflection. The permission path therefore treats this clock-synchronization uncertainty ϵ_{sync} as an irreducible temporal margin, expanding the required clearance by $d_{\text{sync}} = v_{\text{rel}}\epsilon_{\text{sync}}$ before granting motor torque authorization. With the observation contract's illustrative 0.2 ms bound and the base at its 1.5 m/s drive limit, that term is 0.3 mm, small beside the 50 mm localization bound but never zero. The bound matters most where it is loosest, on a link whose synchronization has not been measured under load.

Physical systems routinely combine asynchronously acquired signals, even when hardware triggers synchronize subsets of sensors. In an illustrative asynchronous cycle, a tactile array might latch normal force readings at $t_1 = 12$ ms, an optical tracking camera might have its exposure midpoint at $t_2 = 18.5$ ms, and joint optical encoders might sample angular positions at $t_3 = 21$ ms. These observations occur at different epochs and cannot be combined through direct tensor concatenation or naive algebraic operations. Multimodal foundation models and vision-language-action (VLA) backbones frequently pack visual patch tokens, proprioceptive vectors, and tactile embeddings into a single sequence under the implicit assumption that all tokens capture a simultaneous physical snapshot. However, nothing in that token serialization records that the camera frame came from t_2 and the encoder reading from t_3. As a result, representations inferred from unaligned inputs carry unquantified temporal mismatch errors. Fusing a camera observation at t_2 with an encoder observation at t_3 requires the consumer to project one state across the temporal gap $\Delta t = |t_3 - t_2| = 2.5$ ms. The projection uses explicit kinematic models and bounded velocities (for example, propagating the historical camera observation along the joint velocity vector to reconcile it with the instantaneous encoder reading). The gap is small in time but not in space; at the arm's 1.7 m/s Cartesian speed limit, the tool moves 4.25 mm in 2.5 ms, a millimeter-scale error that a fine insertion cannot absorb. Without the true acquisition epoch for each individual signal, the alignment calculation cannot determine the sign or magnitude of Δt, leaving uncorrected temporal offsets to compound geometric errors across every inference cycle. Placed on one shared timeline (figure 9.2), the three epochs fall at distinct instants, and the camera evidence must be carried across the 2.5 ms gap before it can be fused with the encoder state.

Equipped with these explicit epochs and the clock-synchronization bound carried in the record of section 9.3, downstream consumers can evaluate the elapsed duration between the evidence epoch and the execution deadline. They can then compute the worst-case expansion of the spatial error bound, and verify whether the state remains within task tolerances. Omitting the acquisition epoch leaves the consumer blind to transport delays and scheduling jitter, degrading deterministic closed-loop control into open-loop guesswork. Because physical geometry continuously evolves as bodies move through space, spatial transforms between reference frames are subject to the same temporal decay, carrying their own distinct evidence epochs and bounded uncertainties.

9.5 Coordinate Transform Expiry and Localization Drift

A coordinate frame binds a spatial representation to a physical body, but on an active robot that frame is continuously moving. Embodied architectures relate sensors, chassis links, and end-effectors through a transform tree, in which every frame has exactly one parent. A static drawing of this graph, however, conceals how rapidly its edges go stale. In the arm chain of figure 9.3, each edge stores a rigid-body pose parameterized by joint angles (Lynch and Park 2017; Craig 2005), describing the machine only at the instant its physical geometry was transduced. When an onboard computer chains transforms sampled at different timestamps, whether through neural inference latency, bus transport delays, asynchronous sensor clocks, or dropped packets, the resulting mathematical product yields a hybrid pose that the physical machine never occupied.

↪ Rigid-body transforms, their composition along a kinematic chain, and their rotation parameterizations are developed in section A.3.

Transform state also spans multiple machine words: three floating-point numbers for translation and four for rotation as a unit quaternion. A high-rate control reader racing an asynchronous perception writer can therefore tear a read, observing an old rotation paired with a new translation. The lock-free sequence lock (seqlock) mailbox established in section 4.3.2 resolves this concurrency race without blocking hard real-time control threads. Each reader makes a single bounded attempt per tick and never blocks. If the sequence count reveals a torn read, the thread discards the sample and lets the last validated transform age in place.

On the arm itself (figure 9.3, panel a), an effective lever arm r_{arm} separates the rotating wrist (joint 7) from the tool center point (`tool0`), so an angular lag $\Delta\theta_{\text{joint}} = \omega\,\Delta t$ sweeps the end-effector through an unmodeled arc $e_{\text{tool}} \approx r_{\text{arm}}\,\Delta\theta_{\text{joint}}$. The links (`link1` through `link7`) are connected by revolute joints whose optical encoders sample joint angles $q_i(t)$ continuously, so the arm's own edges can be updated frequently; the edge that ages is the wrist camera's estimate of the target.

Panel (b) shows the corresponding coordinate transform tree, maintained in robot middleware such as the Robot Operating System (ROS) library `tf2` (Foote 2013). Each directed edge represents a 4×4 homogeneous transformation matrix $\mathbf{T}_{AB}$ mapping points from child frame B into parent frame A. Because the arm's joint state arrives at its 1 kHz joint-control rate over a deterministic fieldbus while the wrist camera publishes detected object poses (`target_object`) at only 60 Hz over an asynchronous transport, the transform tree inevitably buffers edges with disparate timestamps. Properly designed transform libraries refuse to mix these epochs. A `tf2` lookup for a transform chain at query time t evaluates every edge at that exact timestamp t, interpolating each edge between its buffered samples. If t lies beyond the newest sample of any edge, the lookup raises an extrapolation error. Similarly, requesting the "latest" transform returns the chain at the newest timestamp common across all constituent edges, preserving a single consistent epoch (Foote 2013).

Unphysical hybrid poses arise when application code bypasses these library guarantees. If a node caches an edge, pairs a target pose from an asynchronous detection message with a live lookup of the arm at the current instant, or republishes an old edge under a refreshed timestamp, it multiplies the current forward kinematics $\mathbf{T}_{\text{base}\leftarrow\text{wrist}}(t)$ by a stale vision edge $\mathbf{T}_{\text{wrist}\leftarrow\text{object}}(t-\Delta t)$. This composition represents the **chimeric kinematic product anti-pattern**: an unphysical geometric artifact that corresponds to no configuration the physical robot ever occupied.

Enforcing a common timestamp leaves the application with an operational decision whenever a query timestamp lies ahead of the newest buffered sample. A lookup can fail immediately, or a non-critical consumer can block until new samples arrive subject to a timeout. In safety-critical execution paths, such as permission enforcement, lookups must remain strictly nonblocking and fail closed upon rejection. If an application instead attempts to predict a future transform, it must propagate a validated uncertainty bound; naive dead-reckoning extrapolation during an unannounced sensor stall rapidly accumulates lever-arm error $e_{\text{rot}} \approx r_{\text{arm}}\,\omega\,\Delta t$.

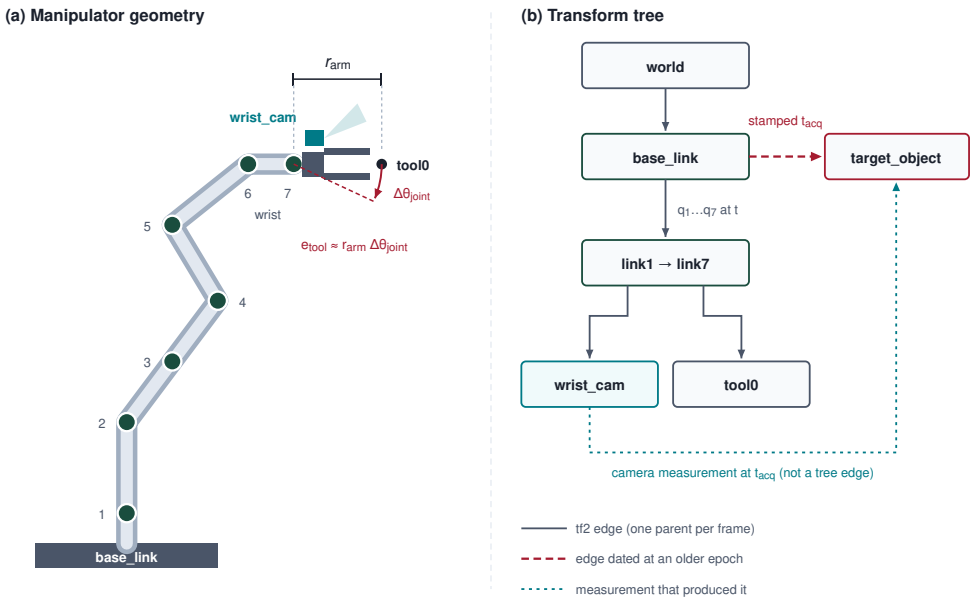


Figure 9.3: Manipulator kinematic frame tree and temporal dynamics: Articulated seven-joint manipulator geometry (a) and the corresponding coordinate transform tree in ROS `tf2` (b), in which every frame has exactly one parent. Each tree edge stores a homogeneous transformation matrix; articulated-arm edges depend on joint state, while mounting and localization edges have their own calibration or estimation premises. The target's pose hangs from the base frame, stamped with the epoch t_{acq} of the wrist-camera measurement that produced it, showing the capture-dated base-frame result. The error discussed here arises when application code instead retains the camera-relative measurement and composes it with current wrist kinematics; a wrist rotation $\Delta\theta_{\text{joint}}$ over the interval swings the tool center point by $e_{\text{tool}} \approx r_{\text{arm}}\,\Delta\theta_{\text{joint}}$ (adapted from Craig (2005) and Foote (2013)).

Transform expiration becomes spatial displacement at the plant interface. At the cage door the vision edge is the wrist camera's estimate of the latch-release slot (the `target_object` of figure 9.3), which the controller composes with the arm chain to place the finger relative to the slot. Composing that stale camera-to-slot edge with the current arm pose adds lateral drift $e_{\text{trans}} = v_{\text{drift}}\,\Delta t$, from motion of the base and door that no sensor reported during the interval, to the initial error e_0. It also adds a wrist lever-arm sweep $e_{\text{rot}} \approx r_{\text{arm}}\,\omega\,\Delta t$. The sum can carry the tool error e_{tool} outside the slot tolerance r_{tol} (figure 9.4). Applying the age-as-displacement principle to manipulator kinematics shows that uncompensated latency translates directly into geometric positioning error through the lever arm of downstream links.

The vulnerability of high-precision manipulation to transform staleness becomes clear when positioning an end-effector at a latch-release slot while the slot remains in direct view of the wrist camera. The wrist camera captures a new frame every 16.7 ms, renewing the observed slot pose and updating the camera-to-slot transform that guides fine alignment. To clear the opening, the fingertip must remain within a tight mechanical tolerance $r_{\text{tol}} = 1$ mm. To isolate the geometric penalty of a single missed frame, assume the camera observation is available at its capture epoch with zero ingestion delay, after which the transform edge ages by exactly one frame interval $\Delta t = 16.7$ ms. On a production machine, pipeline ingestion latency makes the edge older still; control loops must always charge the full capture-to-use latency, including neural inference, message serialization, and transport. Starting from an initial calibration error $e_0 = 0.1$ mm, lateral motion drifts by at most $e_{\text{trans}} = v_{\text{drift}}\,\Delta t = 0.20$ mm under an unobserved lateral velocity bound of $v_{\text{drift}} = 12$ mm/s.

The kinematic lever arm magnifies this displacement substantially. With the fingertip positioned at a distance $r_{\text{arm}} = 0.15$ m beyond the wrist joint, an alignment rotation at angular rate $\omega = 0.30$ rad/s sweeps the tip through an arc of $e_{\text{rot}} \approx r_{\text{arm}}\,\omega\,\Delta t = 0.75$ mm. The optical encoders register the wrist rotation correctly; the error comes from what the rotation is applied to. The cached camera-to-slot transform states where the slot sat relative to the camera at the previous frame, and the camera rides on the wrist. Application code that chains that cached transform with the current wrist pose

↪ The lever-arm kinematics behind this error propagation, built from the vector cross product, are developed in section B.6.4.

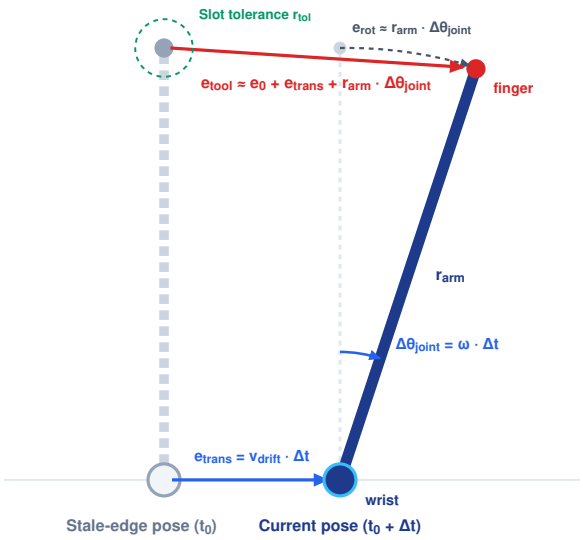


Figure 9.4: Transform staleness and lever-arm error at the latch-release slot: Illustrative finger error over one frame period, with zero ingestion delay assumed to isolate this contribution. A deployed calculation must use full capture-to-use age. The camera-to-slot edge ages by duration Δt. Lateral drift at the velocity bound v_{drift} moves the finger by $e_{\text{trans}} = v_{\text{drift}}\,\Delta t$, while the wrist turning at ω swings the fingertip, a lever arm r_{arm} beyond it, by $e_{\text{rot}} \approx r_{\text{arm}}\,\omega\,\Delta t$. Composing the stale edge with the current wrist pose adds both to the initial error e_0, so the finger error $e_{\text{tool}} \approx e_0 + e_{\text{trans}} + r_{\text{arm}}\,\Delta\theta_{\text{joint}}$ exceeds the slot tolerance r_{tol}.

therefore swings the computed slot along with the wrist, while the real slot, fixed in the door, stays where it was. The result is a hybrid pose that the arm never occupied. Correcting this error requires either querying the kinematic chain at the exact capture epoch of the camera frame or expiring the stale transform outright. Unchecked, the stale transform displaces the estimated finger position by $e_{\text{tool}} \approx e_0 + e_{\text{trans}} + e_{\text{rot}} = 1.05$ mm, which exceeds the mechanical tolerance by 5 percent. The lever arm contributes over three times as much error as translational drift alone. Because the matrix multiplications complete without raising a runtime exception, the control loop blindly aligns the tool to an invalid target, driving the finger into the surrounding strike plate.

The red error vector in figure 9.4 pierces the tolerance boundary even though every underlying matrix multiplication remains numerically valid. Because the dominant error originates in the kinematic lever arm rather than translational drift, a faster or more precise clock does not help; the stale chain is wrong at any timestamp resolution. Only composing every edge at the same epoch, or expiring the stale edge, removes the error, so systems software must enforce one or the other.

> **Systems Perspective 9.1: The lever arm of staleness**
>
> *A transform's admissible age is set at the point it serves, not at the joint it measures. The tool point drifts at roughly $v_{drift} + r_{arm}\omega$, so the longer the lever arm, the sooner an edge expires, and an edge still fresh enough to locate the wrist can be too old to locate the tool center point.*

9.5.1 Localization drift and map frames that jump

Unlike a manipulator arm bolted to a chassis with optical encoders on every joint, a wheeled mobile base has no mechanical anchor to the floor. Dead-reckoning odometry drifts past the stopping budget's localization bound within a single aisle, while map-based loop closures snap the pose back into alignment by jumping remembered obstacles across the map frame. Every map-anchored spatial belief in this chapter, and every allocation in the safety stopping budget, relies on that localization bound δ_{loc}, the guarantee that the robot estimates its position in the map within a bounded uncertainty radius. Maintaining that bound requires combining two distinct state

estimators with complementary failure modes, and spatial memory must account for how each estimator alters stored observations.

Odometry integrates high-rate incremental motion from wheel encoders, inertial measurement units (IMUs), or scan-to-scan lidar odometry. Odometric integration produces smooth, high-frequency pose updates with low latency, but because it measures only relative displacements, incremental integration errors accumulate without bound. Absolute position along the trajectory represents an unobservable subspace (section 9.8). Conversely, **map-based localization** matches real-time range scans against an authoritative pre-built map, bounding absolute drift at the expense of discrete, lower-frequency corrections. When an operating environment lacks a prior map, **simultaneous localization and mapping (SLAM)**[5] estimates the robot trajectory while constructing the map concurrently. Modern SLAM architectures formulate this estimation over a pose graph, a factor graph whose vertices represent historic robot poses and whose edges encode odometric and scan-matching constraints (Thrun et al. 2005; Cadena et al. 2016; Dellaert and Kaess 2017; Hess et al. 2016). When the robot revisits a previously mapped area, a **loop closure** introduces a cross-trajectory factor. Re-optimizing the nonlinear graph distributes the accumulated drift across historic poses, retroactively adjusting the estimated trajectory. While the underlying fusion filters follow the Bayesian prediction-correction mechanics detailed in section 9.2 and section 9.2.3, spatial memory systems must manage the distinct ways these estimators distort the age and coordinates of stored evidence.

[5] **SLAM**: Smith et al. (1990) showed that when a moving robot observes landmarks relative to itself, every landmark estimate inherits the common error in the robot's pose, so map error and pose error are correlated and must be estimated together. The acronym came later, in a survey presented at the 1995 International Symposium on Robotics Research (Durrant-Whyte and Bailey 2006). That correlation is why a loop closure moves remembered cells that no sensor has revisited.

How fast odometry spends that bound can be measured. The standard protocol for quantifying systematic odometry drift is Borenstein and Feng's University of Michigan Benchmark (UMBmark) (Borenstein and Feng 1996). A differential-drive robot traverses a 4 m square path covering 16 m of total travel, executing five runs clockwise and five runs counterclockwise. By comparing the odometric return pose against ground-truth external measurements, the benchmark calculates the average return error across both directions. Traversal in opposite directions isolates the two primary sources of systematic error, unequal wheel diameters and wheelbase uncertainty, which would otherwise partially mask one another on unidirectional trajectories.

On their experimental base, uncalibrated runs produced an average systematic return error of 317 mm. Calibrating the wheelbase and wheel-diameter ratio reduced this error to 21 mm, achieving a 15.1× precision gain. Normalized by path length, the systematic drift rate drops from 2.0 percent to 0.13 percent of distance traveled. At the uncalibrated rate, a mobile base exhausts an illustrative 50 mm localization bound after traversing only 2.5 m, equivalent to 1.9 s of operation at an aisle cruising speed of 1.3 m/s. Traversing a single 30 m aisle accumulates 594.4 mm of position drift, exceeding the bound by a factor of 11.9×. Even with careful calibration, pure odometry exhausts the 50 mm bound after 38.1 m (29.3 s), accumulating 39.4 mm across one aisle. Plotted against distance driven (figure 9.5), even the calibrated line crosses the bound only just past one 30 m aisle, which makes δ_{loc} a constraint on how often external map corrections arrive rather than an intrinsic property of the mobile base.

Three physical realities prevent this illustrative calibrated extrapolation from establishing an operational design margin. First, the laboratory benchmark operated at 0.2 m/s along straight segments to minimize wheel slip, stopping to pivot at each corner; production warehouse bases travel 6.5× faster under varying operational payloads. Second, kinematic calibration eliminates only deterministic systematic errors. Non-systematic stochastic errors (floor irregularities and micro-slip) produced a run-to-run standard deviation of approximately 25 mm, eclipsing the residual calibrated systematic error of 21 mm. Third, a constant per-meter error rate is merely a first-order linearization. An uncorrected heading error $\delta\theta_{\text{head}}$ produces lateral position displacement $\delta\theta_{\text{head}}\, d$ over a straight traverse of distance d. Because wheel-diameter mismatch causes heading error to accumulate linearly with distance ($\delta\theta_{\text{head}} \approx \kappa d$ for a constant heading curvature κ), the resulting lateral error grows quadratically, about $\frac{1}{2}\kappa d^2$, quadrupling each time the straight run doubles. Under this illustrative drift model, reserving localization margin requires external corrections at intervals shorter than the calculated crossing distance; the deployed interval must come from the machine's qualified drift envelope. In deployed systems, the site parameter δ_{loc} must be established empirically, by driving the robot past surveyed fiducials at nominal velocity and maximum payload across actual facility flooring, reporting drift rates at a defined operating percentile.

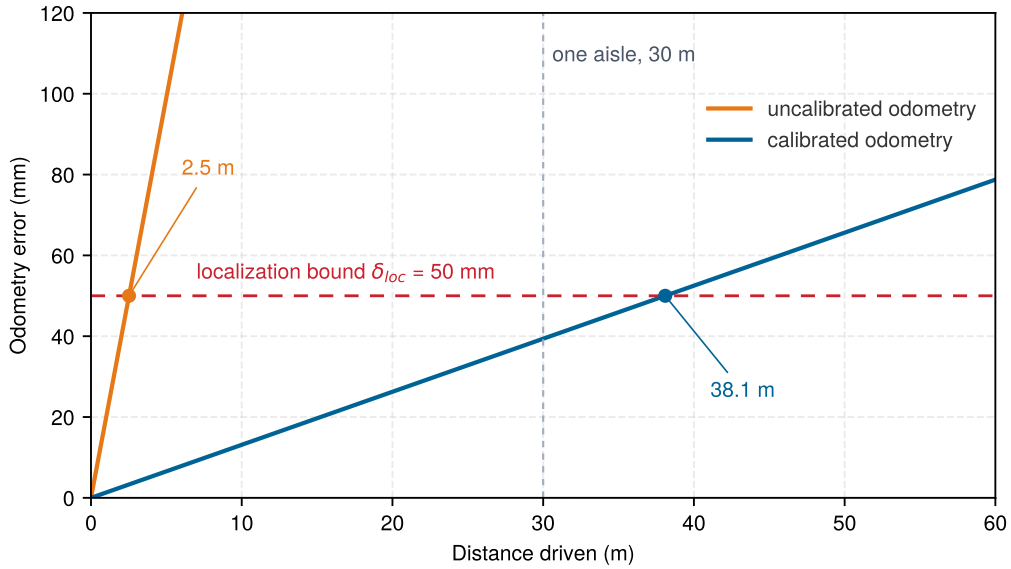


Figure 9.5: Odometry drift against the localization bound: The UMBmark return error, scaled per meter of path, grows linearly with distance driven. Uncalibrated, it spends the 50 mm localization bound after 2.5 m; calibrated, after 38.1 m, only just beyond one 30 m aisle. Both lines extrapolate return errors from a slow laboratory run; deployed correction spacing must be qualified against the machine's own drift envelope before δ_{loc} can be relied on.

The laboratory rates above isolate the systematic part of wheel-odometry error on a slow square path. Camera and lidar odometry are measured differently, by total error at road speed over long runs. A public driving benchmark scores each submitted method by its translation error averaged over every subsequence of 100 to 800 m in held-out sequences, in percent of distance traveled (Geiger et al. 2012). Figure 9.6 places every ranked method against the distance at which its drift rate would spend the machine's localization bound.

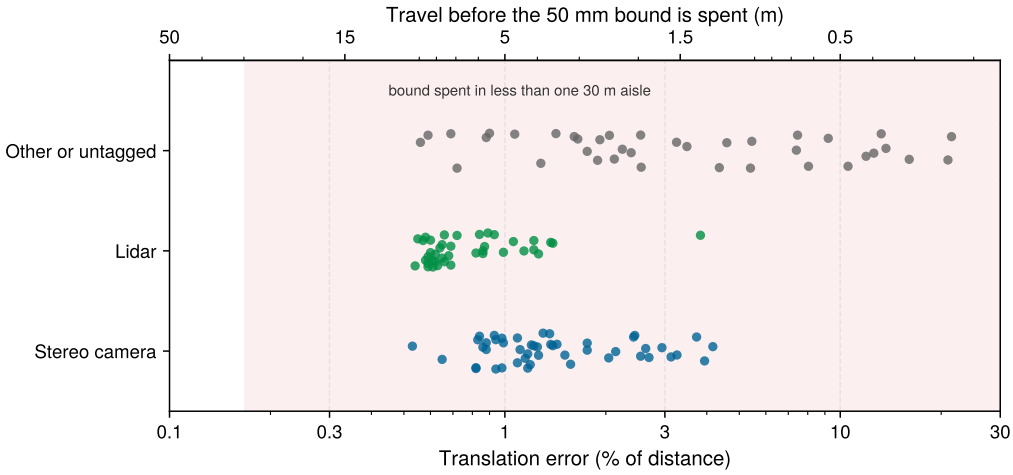


Figure 9.6: Even the best odometry needs a correction within one aisle: Each dot is one ranked method on a public odometry benchmark, placed by its translation error (percent of distance, averaged over 100 to 800 m subsequences) and grouped by sensor; methods without a stereo or lidar tag mix monocular, inertial, and event-camera inputs. The top axis converts error into the distance at which a constant drift rate spends the running machine's 50 mm localization bound, and the shaded region is shorter than one 30 m aisle. The benchmark is road driving, so it bounds what these methods achieve rather than predicting warehouse drift. Data from the benchmark leaderboard (Geiger et al. 2012).

The best of the 127 ranked methods drifts 0.53 percent of distance, and the median stereo and lidar methods drift 1.23 and 0.68 percent. At those rates the 50 mm bound lasts about 9 m for the best method, 7 m for the median lidar method, and 4 m for the median stereo method, all shorter than one 30 m aisle. These total-error rates sit above the calibrated laboratory figure of 0.13 percent, because they include the slip, vibration, and scene-dependent errors that UMBmark's calibration deliberately excludes. Read together, the two benchmarks bracket the problem rather than resolve it: the optimistic systematic rate stretches the bound just past one aisle, and the measured total rates of the best published methods spend it within a third of one. Every odometry source on the base is a bridge between external corrections, and the bound remains a requirement on how often those corrections arrive.

Loop closure introduces a consistency challenge that timestamp-based age tracking alone cannot resolve. An occupancy cell anchored in the global map frame retains its original capture epoch (the photons reflected from that obstacle are no newer), yet its spatial coordinates shift whenever the SLAM backend re-optimizes the underlying pose graph. Although the sensor evidence has not been refreshed, its spatial position changes discontinuously. Any local memory snapshot constructed prior to the loop closure now misplaces the obstacle by the magnitude of the graph correction, even while every timestamp within the snapshot passes validation. Robust spatial memory architectures must therefore tag spatial beliefs with the generation or revision identifier of the pose graph alongside individual cell epochs. When a loop closure shifts supporting trajectory poses, the system must either re-anchor dependent snapshots or invalidate them immediately.

The REP 105 frame convention met in table 8.2 already separates the two estimators (Meeussen 2010). Poses in `odom` are continuous but drift without bound. Poses in `map` stay globally bounded but jump whenever a scan-matching correction or loop closure resolves accumulated drift, and the localizer absorbs each correction by adjusting only the `map`-to-`odom` edge. What memory must decide is which frame its safety snapshot lives in. The permission controller builds its local collision-checking snapshots in the continuous `odom` frame, so a correction never moves an obstacle inside a snapshot that is in use. Over a snapshot's short lifespan (typically hundreds of milliseconds to several seconds), the drift that calibrated odometry accumulates is small and is charged against δ_{loc} in proportion to distance traversed; uncalibrated odometry, which spends the whole bound within 2.5 m, cannot rely on this. Anchoring safety snapshots in the `map` frame instead would make obstacle bounding hulls teleport discontinuously upon every map correction. Global mission planners, long-term semantic maps, and snapshot construction routines operate in `map`. When a major map correction occurs, the host invalidates and rebuilds the local snapshot rather than forcing real-time controllers to absorb discontinuous jumps. The `map`-to-`odom` edge remains subject to the same temporal expiration rules as any kinematic transform.

A stale transform displaces both the tool estimate and any observations mapped through it. The surrounding world ages too, but it has no joint encoders to report its motion. Memory must therefore record which surfaces and empty regions the sensors actually verified, as well as the frames in which those observations hold.

9.6 Certified Spatial Assertions in Scene Memory

When a mobile manipulator sweeps its range sensors down an aisle, spatial memory must partition the physical volume by what the sensors established: space observed to be occupied, space observed to be empty, and unobserved space. Only observed empty space can support a physical clearance claim, and only while that evidence remains temporally valid; once it lapses, the space becomes a fourth category, expired, which section 9.6.1 adds to the cell states. If a map representation merges unobserved space with free space, the permission path cannot detect occlusions, defeating evidence-freshness checks regardless of timestamp accuracy.

Distinct downstream consumers place competing demands on this spatial memory architecture. A learned manipulation policy requires object identities and local geometry to plan collision-free grasps or navigation trajectories. In contrast, the hard-real-time permission path requires bounded geometric queries whose supporting evidence can be verified before an execution deadline expires. To satisfy these disparate consumers, spatial memory organizes into a functional hierarchy. Metric maps compute geometric clearances and volumetric occupancy. 3D scene graphs structure those measurements into topological places and semantically labeled objects. Episodic memory retains

interaction histories needed to interpret operational context. While higher layers draw upon metric observations for geometric grounding, semantic labels and remembered events never possess the authority to certify physical clearance on their own.

9.6.1 Volumetric metric maps and distance fields

At the metric layer, a clearance query along a warehouse aisle demands a single, safety-critical quantity: the minimum distance from the robot's body hull (the simple solid that encloses the machine's body for collision checking) to the nearest region that is occupied, unobserved, or backed by expired evidence. In contrast, a neural or graphics rendering query estimates what a camera would observe from a specific vantage point. If the safety microcontroller had to scan thousands of occupancy voxels or chase pointer-linked octree nodes to find the nearest obstacle for each of the robot's body spheres, the clearance check would overrun its microsecond budget inside the permission tick. A spatial representation qualifies for the hard-real-time permission path only if it evaluates conservative physical clearance within the strict execution deadline of a permission tick, operating entirely within the memory footprint supported by the safety microcontroller. The design therefore allocates the query workload first, counts what each candidate representation must read from memory to answer it, and only then sizes the storage.

The clearance query inside the permission tick

The worked design allocates spatial collision checking within the machine's 1 kHz permission loop ($T_{\text{tick}} = 1{,}000$ µs). The required query returns conservative clearance at each body collision hull, where a hull is one of a set of spheres, the evaluation spheres, whose union covers the machine's body. Where the selected check also needs a clearance gradient, that work belongs in the same measured query. For an illustrative 20 to 50 evaluation spheres, the total work is:

$$T_{\text{query_total}} = \sum_{k=1}^{N_{\text{hulls}}} t_{\text{query}}(\mathbf{p}_k) \leq T_{\text{reflex_budget}} \tag{9.7}$$

To bound the total collision workload in equation 9.7, the worked $N_{\text{hulls}} = 50$ case reserves $T_{\text{reflex_budget}} = 100$ µs for local collision checking (the subscript names the reflexive, every-tick check, as opposed to planning). That allocation is part of the permission tick schedule of chapter 4 (figure 4.1), not an addition to it. The job declares a 250 µs WCET against a 400 µs deadline, and collision checking takes 40 percent of that WCET, leaving the rest for the other admission checks. Measured against the whole period instead, collision checking leaves 90 percent of the tick for everything else. This is an allocation to be tested on the deployed controller; full-map updates and trajectory optimization run outside this tick. Dividing the allocation among the hulls gives

$$t_{\text{query}} \leq \frac{T_{\text{reflex_budget}}}{N_{\text{hulls}}},$$

which for the worked case is 100 µs over 50 hulls, or 2 µs per query.

A latency allocation becomes a concrete architectural constraint only when translated into hardware resources: memory bus transactions and cache reads. The permission controller streams its local SRAM at 3.2 GB/s, so the 2 µs allocation covers at most 800 reads of an 8-byte cell per hull query, before any arithmetic, cache effect, or contention; qualification on the deployed controller must still measure the real bound. Against that ceiling, the candidate representations differ less in execution speed than in the physical question a single read answers (table 9.1). An **occupancy grid**[6] stores, per cell, whether that cell is occupied (Elfes 1989; Moravec and Elfes 1985). Certifying that a sphere hull of radius 0.25 m keeps its protective clearance means reading every cell within 0.35 m of its center, $\frac{4}{3}\pi\left((r_{\text{hull}} + \delta_{\text{margin}})/r\right)^3$ cells at pitch r. A hierarchical **octree** such as OctoMap (Hornung et al. 2013) answers the same point query after descending the tree, one dependent read per level, so near surfaces, where pruning cannot merge cells, the scan costs its cell count times the tree depth. A **truncated signed distance field (TSDF)** stores a projective distance to an observed surface along the sensor ray, clamped at a truncation distance of a few voxels (Curless and Levoy 1996). This ray-based construction does not give the nearest-obstacle distance in all directions, even within the truncation band; hull clearance requires a separately validated occupancy or Euclidean-distance

6 | **Occupancy Grid**: Moravec and Elfes built the first grids at Carnegie Mellon to fuse returns from wide-beam sonar, whose single echo cannot say where across its beam the reflector lies (Moravec and Elfes 1985). Each cell therefore holds accumulated evidence rather than geometry, and a cell no return has reached stays at its prior. The representation was designed for sensors too coarse to vouch for what they did not touch, which is why a permission-grade grid must keep unknown distinct from free.

conversion. A **Euclidean signed distance field (ESDF)** stores the straight-line distance to the nearest obstacle (Oleynikova et al. 2017; Millane et al. 2024), so one interpolated geometric lookup, 8 corner reads together with the snapshot validity checks developed below, supplies conservative clearance for an entire sphere. The hull passes when the distance at its center exceeds $r_{\text{hull}} + \delta_{\text{margin}}$ plus the geometric error. Rendering representations answer the second question. A 3D Gaussian splatting (3DGS) scene fits anisotropic Gaussians to a photometric loss (Kerbl et al. 2023), and a neural radiance field integrates learned density along camera rays. Neither stores a conservative distance, and collision checking on splats requires an added assumption that the fitted ellipsoids enclose the physical surface (Chen et al. 2025).

↪ The parameterization of a 3D Gaussian splatting scene is developed in section A.3.5.

Table 9.1: What one lookup answers, and what clearance costs: Reads for a sphere hull of radius 0.25 m plus the protective clearance, at a 5 cm pitch, against a budget of 800 reads per query on the permission controller. ESDF counts are geometric corner reads; fixed header checks also consume the allocation. The octree count is the worst case near surfaces, a tree depth of 8 levels per cell; pruning reduces it in open space.

Representation	Query one lookup answers	Reads per hull query at 5 cm	Can certify the hull's clearance?
Dense occupancy grid	Is this cell occupied?	1,437	Yes, by scanning every cell within reach
Octree occupancy	Is this cell occupied? (after a tree descent)	11,494	Yes, by the same scan, one descent per cell
TSDF	Is a surface within the truncation distance?	1	No general Euclidean hull guarantee from projective TSDF alone
ESDF	How far is the nearest obstacle?	8	Yes, under a validated conservative snapshot certificate
3DGS or radiance field	What would a camera at this pose see?	—	No; requires a separately qualified geometric conversion

Counted against the 800-read budget (figure 9.7), the representation families separate cleanly. A dense occupancy scan exceeds the budget at 2 cm and 5 cm and fits only at 10 cm, where spatial quantization error alone, up to half the voxel diagonal or 87 mm, consumes most of the protective clearance. A worst-case octree scan exceeds the budget at every pitch, because each cell lookup first descends a chain of dependent tree pointers. An ESDF needs exactly 8 geometric reads for one trilinear interpolation regardless of pitch, plus fixed snapshot-header validity checks. Both the metadata checks and the geometric lookup must fit inside the query allocation. The fixed geometric cost favors an ESDF for real-time safety snapshots, while memory capacity constrains the voxel pitch.

A dense ESDF snapshot of the stopping corridor, spanning the aisle width, the clear distance, and an illustrative 2 m manipulator reach height (2.86 m^3 in all), requires a voxel payload of 183 kB at 5 cm resolution, below the microcontroller's 1.05 MB SRAM capacity. The complete static allocation must also include the snapshot header and certificate, active buffers, controller binary, and validation workspace. At 2 cm pitch, the voxel payload alone swells to 2.86 MB, exhausting available SRAM. Adopting the 5 cm pitch introduces up to 43 mm of spatial quantization error into each lookup, an uncertainty that the collision filter must explicitly subtract from the reported clearance. A projective TSDF also cannot substitute for a Euclidean distance field simply because its point lookup is computationally inexpensive; a projective distance along a sensor ray fails to bound the distance to off-axis obstacles.

The host computer maintains the global reconstruction and extracts a bounded local snapshot enclosing the proposed robot trajectory and its emergency stopping corridor. The permission microcontroller admits a new snapshot only after validating its cryptographic provenance, spatial bounding box, per-voxel evidence timestamps, and geometric consistency against a qualified schema. Generating, transferring, and validating these snapshots introduces latency and bus traffic that must be scheduled and qualified independently from the inner-loop collision query. If the host pipeline stalls or drops frames, the active snapshot in microcontroller SRAM continues to age monotonically. The robot may sustain autonomous motion only as long as every cell the snapshot relies on remains within its invalidation horizon and the snapshot's geometric coverage encloses the vehicle's stopping envelope.

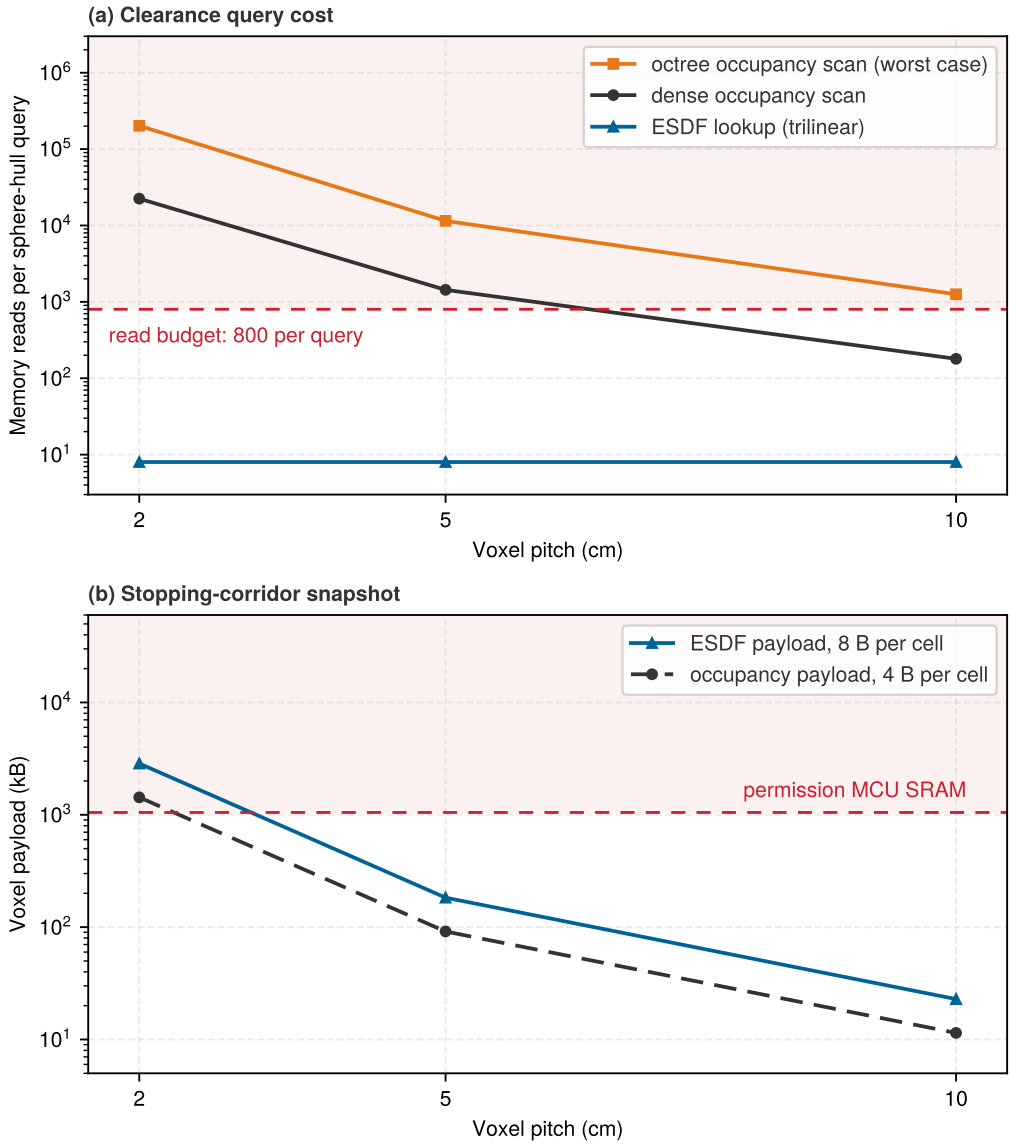


Figure 9.7: Clearance query cost and snapshot storage by representation family: (a) Memory reads to certify one sphere hull's clearance, computed from voxel pitch, hull radius plus protective clearance, and tree depth, against the per-query read budget that the 2 µs allocation and the permission controller's SRAM bandwidth allow. Occupancy scans grow with the cube of the inverse pitch and fit only at the coarsest pitch; an ESDF answers with one interpolated lookup at every pitch. (b) Voxel payload for a dense snapshot of the stopping corridor (aisle width by clear distance by a chosen 2 m height) against the controller's SRAM. Fixed header checks also consume the query allocation, and metadata, active buffers, binary, and validation workspace add to resident storage. Counts, not measured latencies; rendering representations are absent because they do not answer a clearance query.

Eight recent corner samples do not establish that the whole region behind a distance claim remains free. Although a trilinear lookup reads only eight local corner voxels around the hull center, the Euclidean distance value stored inside each corner vouches for the absence of obstacles across a surrounding sphere of voxels that the microcontroller never inspects at runtime, and if any distant voxel within that radius carries older evidence or borders an occlusion frontier from which an unseen hazard can advance, the local distance value becomes optimistic before the corner cell's own timestamp expires. The qualified construction path therefore selects an absolute support deadline

for the snapshot, fixing its frame and pose-graph revision, spatial coverage, clock mapping, and physical growth and entry-speed assumptions. Before building the frozen ESDF, it treats unknown and occupied regions as obstacles and removes from the observed free set every region whose evidence cannot support that set through the selected deadline. This conservative erosion reserves the entire declared interval, including the assumed entry and disturbance envelope. Independent validation must establish that the resulting field lower-bounds clearance throughout that interval; neither a producer's declaration nor its digest establishes the claim.

The local consumer checks the fixed header against its own upper-bounded clock, the declared frame revision, coverage, and validity conditions before reading the eight geometric corners. It refuses dependent proposals at the support deadline without a host callback or a map scan; a changed frame or physical assumption invalidates the certificate earlier. Header loads and checks belong inside the same per-query allocation, and motion admission must reserve a feasible response before expiry. The later per-hull erosion calculation can impose a shorter horizon. Because the frozen field already includes the declared interval's erosion, the consumer must not subtract that identical erosion a second time; hull motion and its stopping continuation still require complete swept coverage and qualified aggregate work.

Volumetric memory footprint scaling

↪ How host memory traffic would stall permission-path reads on a shared die, and why the selected placement confines it to evidence age, is developed in section 13.3.

The local snapshot decision leaves the host compute platform with capacity and update constraints. Keeping the chosen snapshot supplied requires sufficient memory capacity for the underlying map and sufficient memory bandwidth to integrate range observations before their physical evidence expires.

For the illustrative host-map sizing calculation, consider an operational workspace of physical volume $V_{\text{ws}} = L_x L_y L_z$. Discretizing this domain into cubic voxels of uniform pitch r requires memory proportional to workspace volume and to the inverse cube of voxel width:

$$M_{\text{dense}} = \left(\frac{V_{\text{ws}}}{r^3} \right) s_{\text{voxel}} \tag{9.8}$$

where s_{voxel} represents the storage footprint per voxel, here 8 bytes: 4 for a 32-bit distance, 2 for the observation weight, and 2 for the evidence epoch.

Obstacles occupy physical 2D surfaces embedded in 3D space rather than solid volumes, and a TSDF[7] requires stored values only near those surfaces. Truncating the distance field to a band $[-\mu_{\text{tsdf}}, +\mu_{\text{tsdf}}]$ around observed surfaces restricts storage to a shell $V_{\text{band}} \approx 2\mu_{\text{tsdf}} \cdot A_{\text{surface}}$, where A_{surface} is the total obstacle surface area. Partitioning space into blocks of n_{blk}^3 voxels with block pitch $L_{\text{blk}} = n_{\text{blk}} \cdot r$, and allocating a block only when it holds data, the sparse hashed footprint obeys:

$$M_{\text{hashed}} = N_{\text{blocks}} \left(n_{\text{blk}}^3 s_{\text{voxel}} + s_{\text{meta}} \right) + N_{\text{buckets}} s_{\text{entry}} \tag{9.9}$$

where $N_{\text{blocks}} \approx \lceil 2\mu_{\text{tsdf}} A_{\text{surface}} / (n_{\text{blk}} r)^3 \rceil$, s_{meta} is the block metadata (coordinates, allocation flags), and s_{entry} is the hash-table entry size.

Sparse volumetric storage requires an indexing mechanism that resolves 3D coordinates without the sequential pointer traversals of an octree. **Voxel block hashing** discretizes continuous world coordinates $\mathbf{p} = [x, y, z]^\top \in \mathbb{R}^3$ into an integer block coordinate vector $\mathbf{b} = [b_x, b_y, b_z]^\top \in \mathbb{Z}^3$:

$$\mathbf{b}(\mathbf{p}) = \left\lfloor \frac{\mathbf{p}}{n_{\text{blk}} \cdot r} \right\rfloor \tag{9.10}$$

A spatial hash function then maps the block index $\mathbf{b}$ from equation 9.10 to a specific hash table bucket index $h(\mathbf{b}) \in \{0, \ldots, N_{\text{buckets}} - 1\}$. The scheme comes from real-time surface reconstruction (Nießner et al. 2013), built on the spatial hashing of Teschner et al. (2003), and underlies the Voxblox and nvblox mapping frameworks (Oleynikova et al. 2017; Millane et al. 2024). In an architecture packing $8^3 = 512$ voxels per block with an 8-byte payload per voxel, each allocated block forms a contiguous 4,096 byte memory buffer. A single hash resolution loads an entire local 3D neighborhood into the processor cache. While average-case access time is $\mathcal{O}(1)$, worst-case retrieval latency depends on hash bucket collisions, probe sequence lengths, cache misses, and lock contention during concurrent sensor updates. Because hard-real-time safety guarantees require deterministic execution bounds

[7] **Truncated Signed Distance Field**: Curless and Levoy (1996) averaged many range scans into a signed distance stored only near the surface to build detailed models of real objects, and KinectFusion ran the same fusion in real time on a GPU from a handheld depth camera (Newcombe et al. 2011). Truncation bought both systems their memory savings. It makes the TSDF a representation for reconstructing surfaces, not for certifying the empty space between them.

rather than average-case performance, the local permission snapshot avoids pointer chasing and hashing overhead altogether by organizing its bounded stopping corridor as a dense, pre-allocated array in microcontroller SRAM. Notebook 9.1 applies both footprint equations to one warehouse workcell.

Napkin Math 9.1: The empty-air tax

Problem: A host map must cover a warehouse workcell of 10 m by 10 m by 3 m at the resolution manipulation needs, with an evidence epoch in every stored voxel so that each region's claim can expire. How much memory does a dense grid take, how much does a hashed band around the obstacle surfaces take, and what fraction of the dense grid describes empty air?

Variables:

- Workcell volume $V_{\text{ws}} = 300$ m³ (illustrative) at pitch $r = 1$ cm (chosen), with $s_{\text{voxel}} = 8$ B per voxel (illustrative).
- Obstacle surface $A_{\text{surface}} = 120$ m² (illustrative) and band half-width $\mu_{\text{tsdf}} = 4$ cm (chosen).
- Block edge $n_{\text{blk}} = 8$ voxels, the nvblox layout (datasheet).

Math: The dense grid, by equation 9.8:

$$M_{\text{dense}} = \frac{300\ \text{m}^3}{(0.01\ \text{m})^3} \times 8\ \text{B} = 3 \times 10^8 \times 8\ \text{B} = 2.4\ \text{GB}$$

The band and its voxel payload, by equation 9.9 without metadata or hash entries:

$$V_{\text{band}} = 2 \times 0.04\ \text{m} \times 120\ \text{m}^2 = 9.6\ \text{m}^3$$

$$N_{\text{blocks}} = \frac{9.6\ \text{m}^3}{(8 \times 0.01\ \text{m})^3} = 18{,}750, \qquad 18{,}750 \times 8^3 \times 8\ \text{B} = 76.8\ \text{MB}$$

Result: The dense grid needs 2.4 GB for 3 × 10⁸ voxels. The band is 3.2 percent of the workcell, so 96.8 percent of the dense grid describes space with no surface in it, and the hashed payload of 76.8 MB in 18,750 blocks is about 31 times smaller.

Systems insight: Obstacles are surfaces, not volumes, so a dense grid allocates the vast majority of its capacity to empty air. In an embedded robot computer, that multi-gigabyte memory footprint directly contends with memory pools required by vision foundation backbones and real-time motion controllers. The hashed calculation represents a theoretical lower bound: block metadata headers, hash table bucket arrays, and boundary fragmentation in partially occupied blocks increase this overhead in practice. This memory reduction applies strictly to the truncated surface representation. It also alters temporal bookkeeping: because the sparse map allocates memory only near detected surfaces, unallocated open space carries no explicit evidence timestamp. Unallocated space cannot vouch for its own clearance until the system explicitly logs a free-space ray-clearing observation.

This narrow-band scaling applies strictly to a TSDF, not to the full Euclidean distance field required by the permission snapshot. An ESDF stores metric clearance throughout the entire navigable volume where queries might occur. Its memory footprint therefore scales with the observed volume V_{obs}/r^3 rather than surface area A_{surface}. For a mobile warehouse manipulator, this observed volume corresponds to the spatial corridor reachable by the chassis and arm during stopping maneuvers, which dictates the snapshot dimensions evaluated above. Memory allocation semantics also dictate safety interpretation. In systems that perform volumetric free-space carving along sensor rays, unallocated blocks reliably represent unobserved, unknown space. In systems that allocate only narrow surface bands, however, an unallocated block could represent either confirmed open space or unobserved space hidden behind an occlusion. A safety-critical spatial memory system cannot conflate allocation status with occupancy state. Alongside observed occupied space, it must explicitly represent the three states that allocation cannot tell apart: observed free space backed by a valid evidence timestamp, unobserved space, and expired space where past observations have exceeded

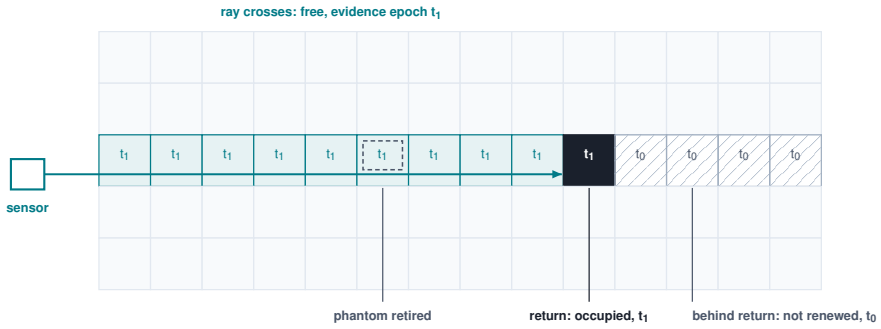


Figure 9.8: One range return updates only the cells it touched: Under the qualified coverage and geometric-error assumptions, the cells the ray crosses become free and take the capture epoch t_1 of the return, which also retires a phantom obstacle left on that line; the cell holding the return becomes occupied at t_1. Cells behind the return and every cell off the ray keep whatever epoch their own last evidence gave them, here t_0, so the map's publication time dates none of them.

their maximum validity horizon. Every cell therefore carries one of four states, occupied, free, unknown, or expired.

Ray-marching update traffic and negative evidence

When depth cameras stream range frames at framerate f_{cam} with $N_{\text{rays}} = W \times H$ pixels per frame, updating the volumetric map requires ray marching from the sensor origin to each observed surface point (Curless and Levoy 1996). Each ray traverses a line-of-sight distance d_{ray} at voxel stepping pitch r, requiring $N_{\text{steps}} = d_{\text{ray}}/r$ discrete cell evaluations. The raw DRAM update bandwidth consumed by naive ray marching is:

$$\text{BW}_{\text{DRAM}} = f_{\text{cam}} \cdot N_{\text{rays}} \cdot \left(\frac{d_{\text{ray}}}{r} \right) \cdot s_{\text{rw}} \tag{9.11}$$

where s_{rw} is the read-modify-write memory traffic per traversed voxel (8 to 16 bytes for reading the current occupancy or distance, updating running weights, and writing back).

For an illustrative VGA depth stream of 307,200 rays at 30 Hz, marched over 2 m at a 2 cm pitch (100 steps per ray) with 8 bytes per step, equation 9.11 gives 7.37 GB/s, 3.6 percent of the peak 204 GB/s of the host system-on-chip (SoC). This share of peak understates the actual performance impact, because these volumetric map updates share the memory interconnect with vision foundation models and motion controllers, and their throughput penalty stems from non-sequential access locality and memory bus contention rather than aggregate bandwidth alone. This update bandwidth represents a substantial baseline load that system architects must incorporate into hardware contention profiling, as memory bus saturation and cache thrashing degrade perception pipelines long before theoretical DRAM bandwidth is exhausted. Block hashing reduces this traffic by keeping active blocks in on-chip memory, and projective TSDF updates project voxels inside the viewing frustum onto the depth image instead of marching every ray. These methods reduce repeated memory traffic, but the reduction depends on scene coverage and update strategy. Sparse allocation still grows as the machine explores new regions; a bounded host store needs an explicit retention policy.

What a map can certify comes from how a range return updates it. Traversing the ray[8] from the sensor to the return (Amanatides and Woo 1987), the map records **negative evidence** along the traversed ray and positive evidence at its return. Treating an entire crossed voxel as free requires qualified beam coverage, resolution, and geometric error allowances. The ray alone does not observe every point in that voxel.[9] A valid empty ray can retire a phantom obstacle, but it says nothing about the volume behind a return, the hatched cells in figure 9.8 that keep their earlier epoch. A new scan down the aisle therefore cannot renew cells hidden behind the rack. Sparse allocation saves capacity without eliminating ray traversal, and any resulting processing delay adds to evidence age.

Because a return renews only the cells it touched, a map's global publication timestamp cannot certify the freshness of individual voxels. The manifest in table 9.2 therefore carries cell evidence

1 depth sample
100 voxel steps
× 307,200 rays
× 30 Hz × 8 B
= 7.37 GB/s

Marching turns each depth sample into 100 read-modify-writes, the per-ray factor that block hashing and projective updates reduce.

8 **Voxel Traversal**: Amanatides and Woo (1987) step a ray from one voxel boundary to the next by comparing the distances to the next crossing on each axis and adding one fixed increment, so each voxel visited costs a few comparisons and one addition. The work per ray is therefore proportional to the number of voxels it crosses, range divided by pitch, which is the d_{ray}/r factor of equation 9.11.

9 **Log-Odds**: Storing $\ell = \ln(p/(1-p))$ turns each Bayesian occupancy update into an addition, with an uninformed prior at $\ell = 0$. A cell no ray has touched and a cell that received as many hits as misses both read $\ell = 0$, so the value alone cannot separate never observed from observed and ambiguous. The manifest of table 9.2 therefore carries an explicit `cell_state`.

↪ The log-odds occupancy update and signed-distance fusion are developed in section A.3.6.

epochs alongside occupancy. For the worked local snapshot, construction and independent validation establish support for every admitted distance claim through the fixed support deadline. The local hull query checks that certificate's header and validity conditions within its allocated time, rather than scanning the supporting cells each tick. A freshly copied distance value without valid support cannot establish clearance. The manifest travels with the bounded snapshot as well as the host map; the isolated controller does not traverse the full host store.

Table 9.2: Volumetric belief manifest and cell evidence: The map header describes layout, pose-graph revision, query latency budget, and update time, while cell-local acquisition epochs and states support construction of a conservative local snapshot. Its fixed header carries the independently validated support deadline and scope checked by the local consumer. The consumer treats unobserved or expired cells as unknown unless a separately validated policy permits traversal.

Map field	Unit or type	Meaning and consumer check
`voxel_grid_origin`, `voxel_resolution_m`	m; m	Spatial anchor and pitch; match the active frame and calibration
`pose_graph_revision`	integer	Revision of the pose graph that placed the cells; a loop closure that changes it re-anchors or expires the snapshot
`map_update_epoch`	capture-clock ns	Last map publication/update; **not** freshness of every cell
`cell_evidence_epoch`	capture-clock ns per cell	Last physical observation for that cell; never refreshed by prediction or unrelated map updates
`free_space_sweep_epoch`	capture-clock ns per occupancy cell	Last valid observed free ray through that cell
`cell_state`	occupied, free, unknown, or expired per cell	Set explicitly, never inferred from allocation; unknown and expired read as obstacles
`clock_conversion`, `sync_error_ns`	mapping; ns	Convert capture epochs to the consumer's monotonic clock with bounded error
`support_deadline`, `validation_scope`	consumer-clock deadline; fixed certificate	Validated conservative interval, coverage, frame revision, and growth assumptions; refuse on expiry or scope violation
`query_latency_budget_us`	µs (at most 2 µs)	Bound collision and distance gradient query time to preserve the 1 kHz permission loop

Synthesizing these elements yields the primary metric the reactive permission path requires, the **free-space depth** d_{free}. To construct this bound, the qualified snapshot path generates and independently validates an ESDF wherein occupied, unobserved, expired, and insufficiently supported regions are obstacles throughout the declared support interval. Evaluating this field at the center of a spherical body hull and subtracting the hull radius yields d_{free}: the Euclidean distance from the hull surface to the nearest physical coordinate that lacks certified, fresh evidence of being empty. Under the validated coverage and entry-envelope assumptions, any dynamic hazard (a pedestrian stepping around a corner, a falling carton) remains within the regions treated as unobserved or uncertified, so d_{free} conservatively bounds the spatial margin available for vehicle motion within that interval. When evaluating a candidate trajectory segment down an aisle, the collision filter subtracts the composite geometric error (encompassing sensor calibration noise, localization covariance, and spatial voxel quantization) as well as the protective safety clearance δ_{margin} from d_{free}. The test is applied not at the hull's present pose alone but along every pose the admitted motion and its stopping trajectory will sweep, so the stopping distance of the stopping budget enters through the length of that swept path. The motion is permitted if and only if the remaining margin is strictly positive at every pose along it. If the robot approaches an occluded cross aisle, d_{free} contracts rapidly regardless of how quickly the microcontroller evaluates the memory lookup; computational throughput cannot compensate for physical occlusion. How rapidly this usable clearance margin decays under persistent occlusion is analyzed in section 9.7.

Forgetting, decay, and change

A dynamic physical environment requires spatial memory to evict stale data systematically. In a logistics facility, pallets move, access doors swing open, and workers cross aisles that range finders observed to be vacant moments earlier. An embodied system that preserves observations indefinitely accumulates both phantom obstacles (retaining records of objects that have moved away)

and phantom free space (assuming unobserved regions remain empty after hazards enter). Active negative evidence clears phantom obstacles whenever fresh sensor rays traverse previously occupied cells. However, detecting hazards that enter unobserved voids presents a different challenge, because no sensor returns exist to contradict the stale assumption of clearance.

Practical mapping systems handle unobserved space through varying assumptions (table 9.3). OctoMap clamps cell log-odds between fixed saturation limits, ensuring that a voxel reinforced by hundreds of historical returns can still be cleared rapidly by a small batch of contradicting observations (Hornung et al. 2013). Clamping accelerates adaptation upon re-observation, but it cannot alter belief in unvisited cells. Independent Markov chain occupancy grids assign each cell an explicit two-state transition model with online-learned transition rates, allowing an unobserved voxel's occupancy belief to relax gradually toward its historical steady-state prior (Saarinen et al. 2012). Modern GPU-accelerated mapping pipelines adapt these concepts to real-time volumetric streams. For example, the nvblox library periodically decays the occupancy probability of out-of-view voxels toward 0.5 (the uninformative state), discarding the assumption that the unobserved world is strictly static (Millane et al. 2024; NVIDIA 2026b). Dynablox identifies transient dynamic objects by maintaining an incremental model of high-confidence free space and flagging unexpected point returns that penetrate that volume (Schmid et al. 2023). Similarly, Khronos separates spatial dynamics across temporal scales, employing a fast tracking loop for short-term movements inside an active window alongside an asynchronous factor-graph optimizer for persistent architectural shifts (Schmid et al. 2024).

These approaches rely on statistical rate models. They parameterize the expected rate of environmental change based on past traffic patterns or heuristic tuning. While average-case models are well suited for high-level motion planning and route optimization, the permission path requires hard worst-case bounds. The deterministic expiration mechanism formulated in this chapter establishes that worst-case counterpart. Specifically, an observed free cell reverts to the unknown state no later than the minimum time required for an unobserved object, moving at the facility's maximum physical velocity bound, to traverse the distance from the nearest occlusion boundary into that cell. A heuristic decay model satisfies safety requirements only if its decay schedule transitions cells to unknown at or before this physical traversal deadline. Otherwise, statistical decay remains an optimization heuristic for path planners rather than a certifiable safety assertion for motion gating.

Table 9.3: Map maintenance mechanisms and what they certify, as of 2026: Rate models describe expected change and serve planning; only an expiry derived from a speed bound certifies that an unobserved cell is still free. The erosion deadline in the last row is derived in section 9.7. The named systems are point-in-time implementations of each mechanism.

Mechanism	What happens to a cell nobody observes	Representative system (as of 2026)	What it cannot certify
Clamped log-odds	Nothing; clamping only speeds change when the cell is seen again	OctoMap	Any change in cells no ray reaches
Per-cell transition model	Occupancy relaxes toward a learned long-run value	Independent Markov chain grids	A worst case; learned rates describe past traffic
Decay toward unknown	Occupancy relaxes toward 0.5 at a fixed rate once out of view	nvblox	Safety, unless the rate is set from a speed bound
Dynamic-object detection	Unchanged; returns inside confident free space are flagged as moving	Dynablox	Objects the sensors do not currently see
Spatio-temporal SLAM	Unchanged until revisited; change is detected on reobservation	Khronos	The interval between visits
Deterministic expiry (this chapter)	Free becomes unknown at the erosion deadline set by the speed bound	Worked design	Nothing past the deadline, by construction

Together, these volumetric grid and distance-field rules bound where the machine may move, but they stop at geometry. While a volumetric distance field certifies where free space ends, it treats every occupied cell as an anonymous obstacle and cannot tell the task planner which surface is the tote handle or the cage door.

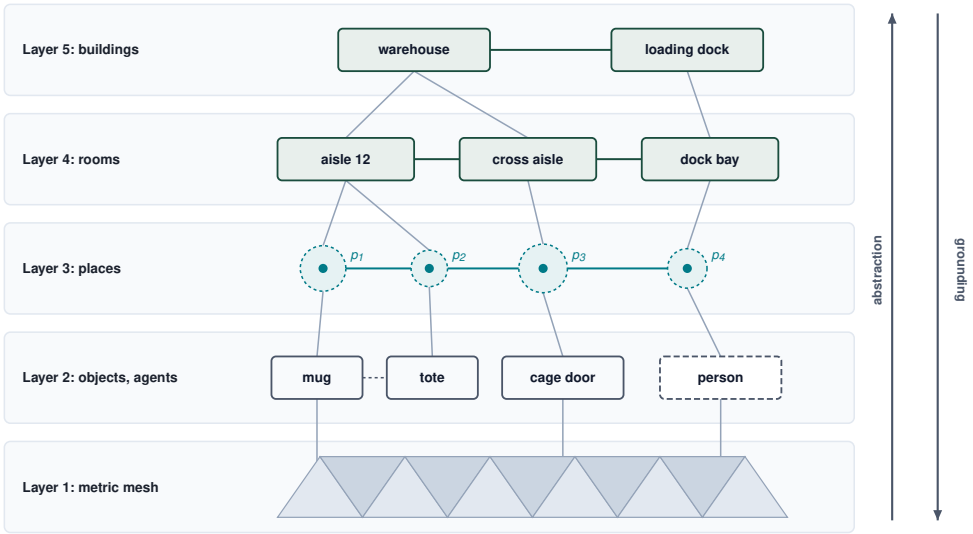


Figure 9.9: Hierarchical 3D dynamic scene graph architecture: Five-layer spatio-temporal memory organization bridging continuous 3D-indexed metric state stores to symbolic foundation reasoning. Drawn for the warehouse, Layer 1 captures triangular metric mesh geometry; Layer 2 extracts 3D objects and dynamic agents such as a walking person; Layer 3 models topological places via Voronoi freespace clearance spheres (p_i^{place}); Layer 4 structures architectural rooms and portals; and Layer 5 establishes global building mission topology. Vertical inter-layer edges coordinate upward abstraction (compressing metric surfaces into symbolic entities) and downward grounding (translating task tokens into metric coordinates) (adapted from Rosinol et al. (Rosinol et al. 2020, 2021) and Hughes et al. (Hughes et al. 2022)).

DRAFT

9.6.2 3D dynamic scene graphs

Supplying that identity is the job of semantic scene memory, which sits above the raw metric layer and inherits its physical constraints. Learned neural models infer what an object is, whereas the underlying metric mapping pipeline anchors where that object sits in physical space. Open-vocabulary scene memory bridges these domains using **3D vision foundation models**. A self-supervised visual backbone such as DINOv2 (Oquab et al. 2023) extracts dense patch feature representations. A promptable segmentation model such as the Segment Anything Model (SAM) (Kirillov et al. 2023) generates class-agnostic instance masks. Systems such as ConceptGraphs (Gu et al. 2024; Jatavallabhula et al. 2023) and Hydra (Hughes et al. 2022) lift those 2D masks and semantic features through calibrated depth, extrinsics, and visual odometry into an **object-centric 3D scene graph**. In this structured representation, each node represents an entity parameterized by a 3D centroid, a spatial covariance matrix, and a text-aligned multimodal embedding vector.

Drawn for the warehouse (figure 9.9), such a graph stacks five layers: a metric mesh, the objects and agents segmented from it (mugs, totes, the cage door, a walking person), topological places linked through free space, rooms such as aisles and dock bays, and the building itself. Edges between layers run both ways, compressing metric surfaces into symbols on the way up and grounding task tokens in metric coordinates on the way down.

Constructing and maintaining this semantic hierarchy consumes substantial compute and latency budgets. An illustrative vision foundation encoder requiring 3 TFLOP per frame at the navigation camera's full 60 Hz capture rate demands 180 TFLOP/s of sustained tensor throughput, even before accounting for image preprocessing, memory bandwidth, or post-processing clustering. By comparison, an edge application processor such as the NVIDIA Jetson AGX Orin achieves a manufacturer-specified peak of 43 dense FP16 TFLOP/s (as documented in its technical brief). This is a theoretical arithmetic ceiling that sustained neural workloads rarely approach, because memory bandwidth caps any kernel whose arithmetic per byte falls short of the processor's ratio of peak compute to peak bandwidth (Williams et al. 2009).

Because the illustrative foundation backbone exceeds this processor's peak throughput at full camera rate, the worked architecture implements a **multi-rate hierarchy**. In this decoupled design, lightweight metric processing pipelines running on the application processor consume raw frames delivered via direct memory access (DMA) to extract geometric obstacle points, update occupancy voxels, and track camera poses at the camera's full 60 Hz rate. Meanwhile, compute-intensive vision foundation backbones run asynchronously on selected keyframes at a reduced rate of 1 to 5 Hz, associating semantic embeddings and updating topological nodes. This multi-rate separation remains safe only because the high-frequency metric loop and the independent permission filter maintain certified physical stopping envelopes during the intervals between semantic updates.

↩ Each rate tier meets its own deadline across a non-blocking boundary, the decoupling established in section 4.3.

The vertical edges spanning the hierarchy in figure 9.9 enable dual-path reasoning. An agentic planner can query spatial relationships among objects and topological rooms without traversing millions of raw mesh vertices, while downstream manipulation requests (such as grasping a bin handle) resolve downward into precise Cartesian frames and localized geometry. From a systems memory perspective, each node encapsulates state elements that age at radically different physical rates:

1. *Geometric pose*: The spatial coordinates of an entity age according to the physical velocity bound of its category. A permanently anchored storage rack does not drift, whereas an unobserved cart or tote may displace at human walking speeds.
2. *Existence confidence*: The probability that an object remains present decays independently of its pose estimate. An unobserved tote may be removed entirely by a human worker; a narrow spatial covariance on an unobserved object provides false confidence if the entity itself is no longer present (section 9.11).
3. *Semantic classification*: Node category labels originate from learned visual embeddings. While semantic labels do not suffer physical drift over time, their validity is bounded by model calibration and out-of-distribution uncertainty, remaining uncorrected until the object is re-observed.

Contemporary frameworks update graph structures primarily upon re-observation: Hydra optimizes scene graph node poses when pose-graph loop closures update the underlying trajectory (Hughes et al. 2022), and Khronos registers object additions or removals when the robot revisits previously mapped areas (Schmid et al. 2024). Neither system explicitly tracks the temporal decay of unobserved entities using physical kinematic bounds. Tracking and enforcing these physical validity limits remains the responsibility of the belief record, and high-level symbolic abstractions can never refresh the evidence timestamps of the underlying metric cells. Beyond knowing where objects sit in the scene graph, a manipulation policy also needs to remember which physical actions it has already completed upon them.

9.6.3 Episodic memory and interaction history

When a transformer policy's sliding context window evicts older tokens as new camera frames stream in, the model can forget that it already unlatched the cage door or experienced a torque stall a hundred cycles ago. Metric representations and 3D scene graphs capture *where* surfaces, free space, and objects exist in the physical environment; **episodic memory** records *what happened*: the sequence of discrete contact transitions, grasping events, torque stalls, and human interventions that define a task execution history. By indexing physical events by their monotonic acquisition timestamps, episodic memory enables task planners to verify preconditions, for example confirming whether a mechanical latch was successfully unlatched or a transport tote was deposited into a fixture.

Learned policies frequently use attention mechanisms over past interaction tokens to infer task progress. However, maintaining unconstrained sequence histories is computationally intractable. A **streaming window** bounds transformer attention buffers by evicting historical tokens based on recency or attention sink criteria (Xiao et al. 2024). Such eviction policies operate independently of physical causality; an observation confirming that a container latch was secured a hundred navigation cycles ago is precisely the type of token dropped by a sliding context window. Consequently, the durable metric record and structured episodic state anchors must persist outside the transformer's transient attention buffer whenever their physical consequences continue to constrain manipulation. Retrieval augmentation supplies the policy with task context and relevant geometry, while the

underlying belief record carries the evidence epochs and error bounds that the permission path independently checks before admitting execution.

For reactive motion permission, any spatial assertion lacking an explicit evidence timestamp is treated as unverified, and any state estimate whose spatial uncertainty bound exceeds certified safety tolerances must be rejected before trajectory commands reach the hardware controller. Nevertheless, an embodied system cannot trigger emergency braking on every dropped sensor packet. Sustaining safe, fluid motion across transient sensor dropouts requires state estimators that propagate spatial belief forward in time under a declared error-growth law, with a deadline that governs execution until fresh observations either corroborate or correct the state.

9.7 Forward State Propagation and Occlusion Decay

When a sensor loses sight of a moving body, bringing the machine to a safe stop takes time and demands a usable state estimate throughout the deceleration interval. Forward propagation bridges this blind duration by projecting the most recent observation forward in time, whether the machine continues an authorized task or executes a protective fallback. A learned generative model can predict future trajectories autoregressively, and ensemble dispersion or Monte Carlo rollouts can quantify prediction variance. However, statistical spread alone does not constitute a certified upper bound on physical tracking error. In physical AI systems, forward state propagation must be anchored to explicit error-growth dynamics, whether implemented via the open-loop Kalman prediction step of equation 9.1, learned stochastic state observers, or expanding covariance ellipsoids.

Forward propagation updates the state estimate and expands its uncertainty envelope while preserving the original hardware capture timestamp. Across the unobserved interval, open-loop state estimation remains valid only so long as the growing error envelope remains within the task's geometric clearance. The rack end in the warehouse aisle shows where that boundary falls.

9.7.1 Bounded error growth and free-space erosion

At an aisle intersection, the gap between propagated prediction and live observation decides what the robot may safely execute. Where a storage rack ends, a worker may step abruptly into the clear distance $D_{\text{clear}} = 1.10$ m defended by the machine's stopping budget. Once the rack structure occludes the cross aisle, the central question becomes how long remembered free space can certify that the vehicle's path remains clear.

Every mathematical dynamics model simplifies physical reality, causing the discrepancy between the projected state and the physical world to compound continuously during sensor dropouts. This uncorrected drift is the open-loop prediction step of equation 9.1 running without measurement updates, the same widening seen over one sample step in figure 9.1, now compounded across the entire occlusion. In state estimation, this dispersion is called **occlusion covariance expansion**, the continuous widening of uncertainty while the sensors have no line of sight. Formalized by continuous Lyapunov or Riccati covariance propagation equations (section B.8), this propagation describes modeled dispersion before sensors regain visibility; hard divergence bounds require separately qualified disturbance limits.

In a deterministic bounded-disturbance framework, empirical sensor characterization and mechanical disturbance limits establish an initial position error bound e_0 at the instant of occlusion ($\Delta t = 0$). Two bounded physical terms subsequently degrade the state estimate. First, unmodeled kinematics introduce a drift velocity bounded by v_{drift}. Second, unmodeled forces and structural dynamics introduce a disturbance acceleration bounded by a_{dist}. Over an occlusion interval Δt, the worst-case position error bound $E(\Delta t)$ expands quadratically:

$$E(\Delta t) = e_0 + v_{\text{drift}}\Delta t + \frac{1}{2}a_{\text{dist}}(\Delta t)^2 \tag{9.12}$$

At the rack end, the stored belief is not the tracked position of an external object, but rather a spatial occupancy assertion: no person stands within the protective clearance margin δ_{margin} (100 mm) of the robot. The spatial map vouches for this claim out to the free-space depth d_{free} (section 9.6.1), defined as the distance from the vehicle hull to the nearest cell lacking active sensory evidence. Any obstacle hidden from the sensors lies across that occlusion boundary.

Two physical uncertainties consume this free-space buffer. First, onboard localization uncertainty introduces a bounded error δ_{loc} (50 mm), meaning the true physical clearance may already be smaller than the nominal map indicates. Second, an unobserved person can advance outward from behind the rack corner at an approach velocity bounded by $v_h = 1.6$ m/s, the standardized human walking speed mandated in safety standards (International Organization for Standardization 2024a).[10] Matching these physical parameters to the quadratic expansion in equation 9.12 yields an error growth $E(\Delta t) = \delta_{\text{loc}} + v_h\,\Delta t$ with zero acceleration disturbance ($a_{\text{dist}} = 0$). The allowable task tolerance is the uncommitted clearance $d_{\text{free}} - \delta_{\text{margin}}$. As a result, the certified free region erodes inward from the unobserved frontier at walking speed, causing the spatial claim to lapse at:

$$T_{\text{inv}} = \frac{d_{\text{free}} - \delta_{\text{loc}} - \delta_{\text{margin}}}{v_h} \tag{9.13}$$

This derivation starts from the observed free-space depth at capture. A frozen snapshot constructed with erosion already reserved through its support deadline must not apply that same erosion again; its fixed header still limits use to the validated interval. The safety pipeline evaluates this **free-space erosion** law across every swept volume that the admitted motion and its stopping trajectory will occupy. Any nonpositive result ($T_{\text{inv}} \le 0$) indicates that memory certifies no safe clearance along that path. This relationship formalizes the worst-case boundary of occlusion-aware reachable sets used in automated vehicle navigation, where the reachable volume of an occluded traffic participant expands outward from the visibility horizon (Orzechowski et al. 2018).

Two positions of the hull bracket the problem. Abreast of the rack end, the unobserved cross aisle begins immediately at the upright beam, so d_{free} equals the lateral clearance between the hull and the rack. Ahead of the vehicle along its current heading, the nearest unobserved boundary lies at the rack corner, D_{clear} away. In the top view of figure 9.10, a person hidden in the cross aisle can be anywhere inside a disc that grows at v_h from the rack end. In its time plot, the depth certified ahead of the hull falls to zero at 593.75 ms, while the depth abreast sits at zero from the instant of capture. Notebook 9.2 evaluates both cases numerically.

10 **Walking Approach Speed**: ISO 13855 positions a safeguard at $S = v_h\,T + C$ from a hazard, where T is the overall stopping time, C is an intrusion allowance, and v_h is the approach speed (1.6 m/s for a walking body) (International Organization for Standardization 2024a). v_h is a design convention the standard adopts, not a measured human maximum, and a person who runs exceeds it. The rack-end deadline therefore inherits a site premise, that nobody in the aisle moves faster, which the site must enforce or qualify.

Napkin Math 9.2: The erosion deadline

Problem: The navigation camera has just seen the aisle empty, and the rack uprights hide the cross aisle. For how long does that frame certify the space abreast of the rack end, and the space ahead of the hull, and how much of the second deadline is left when memory receives the frame?

Variables:

- Localization bound $\delta_{\text{loc}} = 50$ mm (illustrative) and protective clearance $\delta_{\text{margin}} = 100$ mm (chosen).
- Approach speed $v_h = 1.6$ m/s (datasheet: the ISO 13855 walking speed).
- Free-space depth abreast, the side clearance, $d_{\text{free}} = 150$ mm; ahead, $d_{\text{free}} = D_{\text{clear}} = 1.10$ m (both illustrative).
- Frame age at dispatch $t_{\text{age}} = 51.6$ ms (derived, section 8.4.1); navigation-camera frame period 16.7 ms (datasheet).

Math: Abreast of the rack end, equation 9.13 gives

$$T_{\text{inv}} = \frac{(0.15 - 0.05 - 0.10)\ \text{m}}{1.6\ \text{m/s}} = 0\ \text{ms}$$

Ahead of the hull it gives

$$T_{\text{inv}} = \frac{(1.10 - 0.05 - 0.10)\ \text{m}}{1.6\ \text{m/s}} = 593.75\ \text{ms}$$

and the frame's age at dispatch leaves

$$593.75 - 51.6 = 542.15\ \text{ms} \approx 33 \times 16.7\ \text{ms}$$

Result: Abreast of the rack end the frame certifies nothing, even at the instant of capture. Ahead of the hull it certifies 593.75 ms, of which the frame's age spends 82.6 mm of depth before memory receives it, leaving about 33 frame periods.

Systems insight: The deadline is set by the distance from the unobserved boundary, not by the frame rate or the speed of the lookup. Each millimeter of free depth beyond $\delta_{\text{loc}} + \delta_{\text{margin}}$ buys 0.625 ms, so no camera, however fast, gives the position beside the rack end any memory at all; only a sensor that sees the cross aisle can.

In the open aisle, spatial memory reliably bridges temporary camera frame drops. However, the notebook's deadline is computed for a hull standing at its current pose. A hull that drives toward the rack end at speed v closes on the occlusion boundary while the hidden person closes on it from the other side. The depth certified ahead therefore shrinks at $v + v_h$ rather than at v_h. Each successive swept volume along the planned path starts closer to the boundary, until the horizon reaches zero abreast of the rack uprights. The stopping budget makes the same point in distance. At the 1.5 m/s drive limit, the stopping-budget terms charged before memory enters already claim 912.9 mm of the 1.10 m clear distance before anyone moves in the cross aisle. The safety enforcement path must therefore evaluate equation 9.13 along the entire planned trajectory. The robot must bring itself to a controlled stop, or reacquire line-of-sight visibility into the cross aisle, before any point on the swept path breaches its invalidation deadline. The clear distance the sensors can still see caps the speed at which the hull may approach an occluded rack end, a ceiling that section 12.3.2 plots against clearance. Memory certifies clearance only to a depth governed by the physical distance to the occlusion boundary. At the rack corner, that geometric depth vanishes completely.

Nothing in the software data structures explicitly marks the moment a spatial record expires. The occupancy map may continue to report an empty cross aisle, and the state estimator may continue to publish syntactically valid ROS messages, yet the prediction has ceased to function as physical evidence. Cells within the erosion front revert to unobserved space, and the region beside the rack end becomes uncertified the instant the uprights obscure line of sight. The robot's stopping budget defends against a static obstacle at D_{clear}, such as a stationary worker or a dropped tote. Spatial memory adds a temporal deadline to that stopping budget without granting any additional physical braking clearance; the 912.9 mm partial stopping distance established in section 8.4.1 is the same with memory as without it.

A pedestrian actively walking toward the robot during a stopping maneuver alters the safety budget completely. In standardized safety calculations, an approaching human introduces an intrusion distance $v_h(\tau_{\text{delay}} + T_{\text{stop}})$, where walking speed acts across both the system reaction latency τ_{delay} and the mechanical braking time T_{stop} (International Organization for Standardization 2024a). At the 1.5 m/s drive limit, where a constant-deceleration stop requires $T_{\text{stop}} = v/a_{\text{brake}}$, this pedestrian displacement term alone consumes 1.41 m. This distance already includes the declared reaction latency and exceeds the entire 1.10 m clearance before adding the separately charged controller dispatch interval or the vehicle's own stopping travel. The walking-person speed ceiling this term imposes on a jerk-limited stop is derived in section 12.3.1.

Consequently, static clearance models cannot justify an unrestricted shared pedestrian crossing. Mitigating this risk requires engineered site controls, such as designated pedestrian crosswalks coupled with fixed infrastructure presence sensing at rack corners. Until detection coverage, sensor latency, and traffic protocols are qualified, site operations must retain this premise as an active operational constraint. Under no circumstances can stale spatial memory substitute for missing sensory evidence. Regardless of which traffic rules govern the facility, the robot's permission pipeline must reject any trajectory whose clearance margin relies on unobserved space past the belief's invalidation deadline.

↪ Unqualified environmental premises are enforced as operational restrictions in section 18.3.3.

9.7.2 The invalidation horizon

The two rack-end deadlines, for the aisle ahead and for the space abreast of the rack end, are instances of one physical quantity: the elapsed time at which a belief's growing error bound reaches the tolerance of the task it supports. Beyond that point, an extrapolated belief can no longer prevent a collision or a mechanical jam.

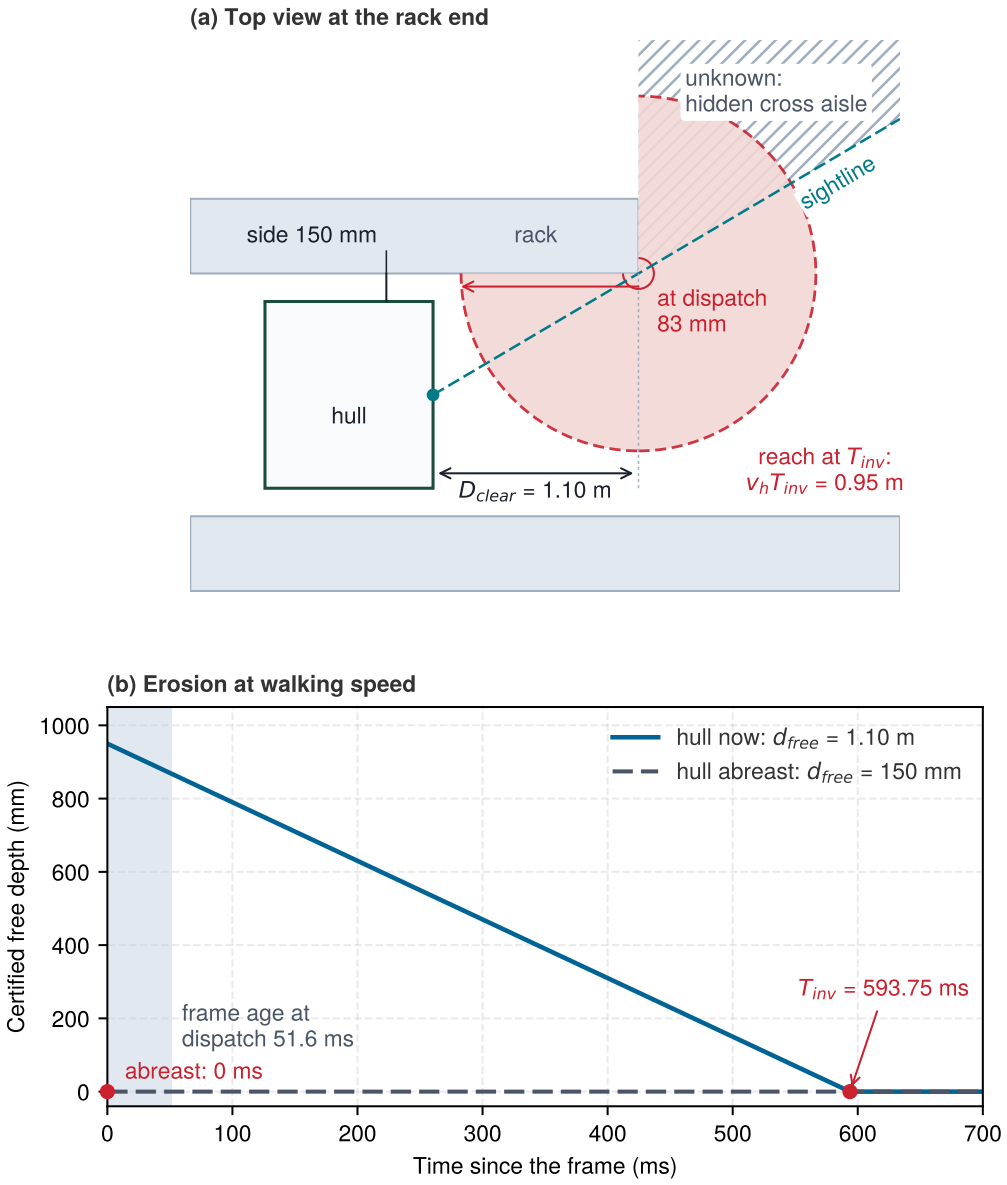


Figure 9.10: Free-space erosion at the rack end: (a) Top view. The navigation camera's sightline passes the rack end, so the cross aisle behind the rack is unknown from the moment of capture. A person hidden there can advance at v_h = 1.6 m/s; the dashed disc is the region that person may occupy when the claim lapses, which stops $\delta_{\text{loc}} + \delta_{\text{margin}}$ short of the hull's front. (b) The depth memory still certifies, $d_{\text{free}} - \delta_{\text{loc}} - \delta_{\text{margin}} - v_h t$. For the hull's current position it falls from the initial clear distance less localization error and protective clearance to zero at 593.75 ms, and the frame's 51.6 ms age at dispatch is spent before memory receives it; abreast of the rack end, the 150 mm side clearance certifies nothing even at capture.

Definition 9.2: Belief state invalidation horizon

Belief state invalidation horizon (T_{inv}) is the elapsed time since the last physical observation at which the belief's validated error bound $E(\Delta t)$ first reaches the tolerance r_{tol}:

$$T_{\text{inv}} = \inf\{\Delta t \geq 0 : E(\Delta t) \geq r_{\text{tol}}\} \tag{9.14}$$

where $E(\Delta t)$ is a bounded-disturbance growth law such as equation 9.12 or a calibrated error quantile, and a belief whose evidence epoch is t_0 stops supporting motion at $t_0 + T_{\text{inv}}$.

1. **Significance**: Marks the instant at which forward-propagated state estimation ceases to function as valid physical evidence, requiring the permission path to refuse dependent motion proposals before geometric clearance is breached. The same walking-speed bound gives the free space ahead of the hull 593.75 ms and the space abreast of the rack end 0 ms, so the horizon belongs to a belief and its tolerance, never to the machine as a whole.
2. **Distinction**: Unlike open-loop timeout watchdogs that monitor software process heartbeat intervals, the invalidation horizon is derived from continuous physical kinematics, bounded disturbance dynamics, and mechanical clearance constraints.
3. **Common pitfall**: Allowing a neural planner or Kalman filter to continuously extrapolate state estimates during extended sensor dropouts without bounding error expansion, mistaking smooth mathematical extrapolation for physical ground truth.

When the error growth law is linear, $E(\Delta t) = e_0 + r\,\Delta t$, the invalidation horizon simplifies to a direct closed form:

$$T_{\text{inv}} = \frac{r_{\text{tol}} - e_0}{r} \tag{9.15}$$

where a nonpositive result ($T_{\text{inv}} \leq 0$) indicates that the belief cannot support the task even at the instant of capture. The free-space erosion law of equation 9.13 instantiates this linear form with initial localization uncertainty $e_0 = \delta_{\text{loc}}$, approach speed $r = v_h$, and task clearance $r_{\text{tol}} = d_{\text{free}} - \delta_{\text{margin}}$. The moving mug-handle belief in section 9.10 applies this exact formulation using conveyor belt velocity. Computed for the evidence behind one action chunk, the root of equation 9.14 is the evidence horizon of section 3.5.2.

↪ The intent lease uses the invalidation horizon as the evidence horizon in its expiry; for an untracked conveyor target that horizon takes this linear form, as developed in section 10.5.

9.7.3 Accelerating error at the latch slot

In the rack-end scenario, a linear velocity bound set the erosion rate; at the cage door, the narrow opening of the latch-release slot sets the task tolerance r_{tol} while unmodeled accelerations compound quadratically over time. When the gripper finger enters the slot, the end-effector blocks the wrist camera's view, forcing the state estimator to integrate commanded joint torques through a nominal plant model without visual feedback. Because unmodeled forces and cable drag accelerate position error far faster than constant-velocity drift, an open-loop bridge that held across a dropped frame in the aisle can jam the finger against the slot wall in tens of milliseconds.

Throughout that blind insertion, the finger must remain aligned within 1 mm of the slot centerline, defining the mechanical task clearance r_{tol}. The other bounds carry over from the missed-frame scenario in section 9.5: an initial position error $e_0 \leq 0.1$ mm and an unobserved velocity drift $v_{\text{drift}} \leq 12$ mm/s. During occlusion, dynamic disturbances add an unmodeled acceleration bounded by $a_{\text{dist}} \leq 50$ mm/s². Under these bounds the error grows quadratically from e_0 and meets r_{tol} at the invalidation horizon $T_{\text{inv}} = 66$ ms (figure 9.11). A naive constant-velocity estimate rises more slowly and keeps admitting the belief out to 75 ms, after the bounded error has already breached the clearance.

Because the belief representation transforms the slot relative to the wrist pose at its exact capture epoch, which resolves the coordinate staleness analyzed in section 9.5, the growth law in equation 9.12 is freed from lever-arm rotational compounding. Position error expands solely through drift velocity and disturbance acceleration. Over a single 16.7 ms frame interval, this formulation limits position error to 0.31 mm, compared to 1.05 mm under an uncoordinated coordinate transform chain. Eliminating lever-arm extrapolation error allows the manipulation system to bridge brief sensor dropouts reliably where an unaligned pipeline would fail immediately.

Evaluating this bounded growth model with the scenario parameters yields $E(\Delta t) = 0.1 + 12\,\Delta t + 25\,(\Delta t)^2$, where elapsed time Δt is in seconds and error $E(\Delta t)$ is in millimeters. Setting $E(\Delta t) = r_{\text{tol}}$ produces the quadratic characteristic equation $25\,(\Delta t)^2 + 12\,\Delta t - 0.9 = 0$. The positive real root establishes an invalidation horizon of $T_{\text{inv}} = 66$ ms. Motion plans relying on this visual estimate must complete or transition to a qualified fallback before this deadline, with deceleration commands dispatched in advance.

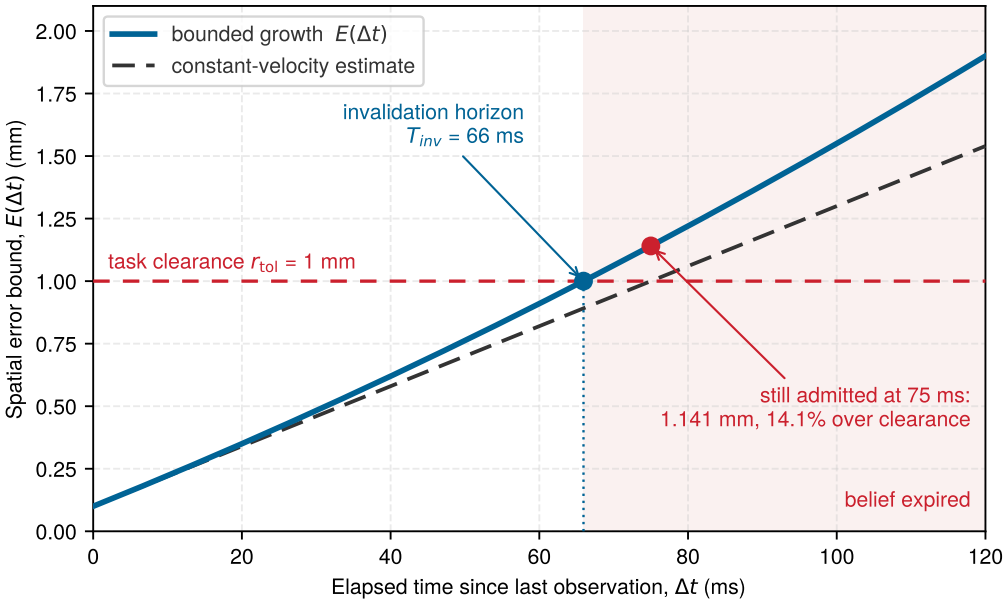


Figure 9.11: Belief uncertainty and invalidation horizon: Illustrative spatial error bound $E(\Delta t)$ of a gripper finger entering the cage door's latch-release slot, expanding quadratically under unobserved forward propagation until intersecting the slot clearance r_{tol} = 1 mm at invalidation horizon T_{inv} = 66 ms; the shaded region beyond it is elapsed time for which the belief no longer supports motion. A dashed constant-velocity estimate grows more slowly, and the red marker shows that estimate still admitting the belief at 75 ms, after the bounded error has passed the clearance. Belief has a shelf life governed by physical disturbance dynamics, and only qualified new evidence can renew the relevant state claim.

If the mechanical slot is machined to a tighter clearance of 0.30 mm, the invalidation horizon contracts sharply to 16.1 ms ($25\,(\Delta t)^2 + 12\,\Delta t - 0.2 = 0$). Tightening the mechanical clearance by a factor of 3.3× reduces the available prediction window by a factor of 4.1×, because the initial error consumes a larger share of the tighter tolerance. Subtracting the unchanged initial error leaves 0.9 mm in the original slot and 0.2 mm in the tighter one, so the usable-slack ratio exceeds the total-tolerance ratio. Acceleration separately shortens both horizons. In contrast, an estimator that naively assumes constant velocity ($a_{\text{dist}} = 0$) predicts an overly optimistic horizon of $(r_{\text{tol}} - e_0)/v_{\text{drift}} = 75$ ms. By that time, the bounded acceleration model permits error up to 1.141 mm, exceeding the allowable clearance by 14.1 percent. A controller operating on constant-velocity extrapolation would continue unguided insertion for 9 ms beyond safe physical bounds, risking mechanical jam or tool damage.

9.7.4 Covariance growth and probabilistic bounds

During a sensor dropout, double-integrating unmodeled acceleration noise causes position variance to grow with the cube of elapsed time, and feeding a raw variance directly into a geometric clearance check confuses statistical spread with a physical boundary. Both worked examples above bounded position error with deterministic limits. Inside a stochastic state estimator, variance propagation separates three distinct physical mechanisms that drive error growth during an occlusion. An unmeasured velocity error at the onset of occlusion causes position variance to expand quadratically as $\sigma_{v,0}^2(\Delta t)^2$. Unmodeled stochastic forces such as torque ripple or structural vibration, integrated twice across the dynamics, expand position variance cubically as $\frac{1}{3}q_c(\Delta t)^3$; these variance terms define the spread that calibrated confidence quantiles must absorb. In contrast, a persistent unmodeled force bounded by $|a_{\text{dist}}|$ (cable drag or sliding friction, for instance) deflects the body deterministically by at most $\frac{1}{2}a_{\text{dist}}(\Delta t)^2$, corresponding to the quadratic term in equation 9.12.[11]

These stochastic variance terms populate the estimator's internal **covariance matrix**, which quantifies the dispersion of a probabilistic error model rather than a hard physical clearance bound. Position variances carry squared length units (m^2), so their growth cannot be substituted directly

[11] **Non-Smooth Contact Dynamics**: Coulomb friction is set-valued at zero sliding speed, and a switch between stick and slip changes the contact dynamics. An impact can also change velocity abruptly. A smooth drift model qualified for one contact regime does not automatically bound these transitions. The belief record therefore treats a change of contact mode as leaving its validity region (section 9.10), not as one more bounded disturbance.

into geometric clearance checks. To examine how stochastic uncertainty expands during complete sensor dropouts, consider a kinematic body whose continuous translational acceleration is driven by unmodeled disturbance or actuator jitter modeled as zero-mean continuous Gaussian white noise. The acceleration obeys $\ddot{\mathbf{p}}(t) = \mathbf{w}(t)$, with $\mathbf{w}(t) \sim \mathcal{WN}(0, \mathbf{Q}_c)$, where $\mathbf{Q}_c = q_c\mathbf{I}$ represents the continuous power spectral density of acceleration noise with physical units $\mathrm{m}^2/\mathrm{s}^3$.

Integrating acceleration once yields velocity $\mathbf{v}(t) = \mathbf{v}(0) + \int_0^t \mathbf{w}(t')\,dt'$. Integrating a second time over elapsed duration t gives the physical position:

$$\mathbf{p}(t) = \mathbf{p}(0) + \mathbf{v}(0)t + \int_0^t (t-t')\mathbf{w}(t')\,dt'$$

To isolate the contribution of process noise, assume initial position and velocity are known precisely. During an occlusion interval where external measurement updates cease, the estimator projects state forward using its nominal transition model $\hat{\mathbf{p}}(t) = \hat{\mathbf{p}}(0) + \hat{\mathbf{v}}(0)t$. The unobserved position error covariance $\mathbf{P}_{pp}(t) = \mathbb{E}[(\mathbf{p}(t) - \hat{\mathbf{p}}(t))(\mathbf{p}(t) - \hat{\mathbf{p}}(t))^\top]$ accumulates continuous noise variance through double integration:

$$\mathbf{P}_{pp}(t) = \int_0^t (t-t')^2 q_c \mathbf{I}\,dt' = q_c\mathbf{I}\left[-\frac{(t-t')^3}{3}\right]_0^t = \frac{1}{3}q_c t^3\mathbf{I}$$

Double integration causes process-noise covariance to expand cubically with elapsed time ($\propto t^3$), the same $\sqrt{t^3/3}$ growth that set the BMI088 accelerometer's noise contribution in section 9.2. If initial state uncertainty is nonzero, the complete error covariance additionally incorporates propagated initial position variance, initial velocity variance, and their cross-covariance terms. Raw covariance represents stochastic dispersion rather than a guaranteed physical clearance boundary. To use this uncertainty in safety enforcement, the system must project covariance into a spatial safety margin through a calibrated coverage factor k (for instance, corresponding to a Chebyshev or Gaussian confidence bound), augmented by bounded systemic biases omitted by the noise model. For spherical or polyhedral bounding volumes, the nominal geometric hull can be inflated dynamically:

$$r_{\text{inflated}}(t) = r_{\text{hull}} + k\sqrt{\lambda_{\max}(\mathbf{P}_{pp}(t))}$$

where $\lambda_{\max}(\mathbf{P}_{pp}(t))$ denotes the principal eigenvalue of the position covariance matrix. The belief record must explicitly furnish these growth bounds and underlying assumptions before the permission path can rely on them to keep execution collision free.

A safety argument can rest on one of two distinct paradigms. In a **bounded-disturbance** framework, rigorous physical qualification establishes explicit limits on initial state errors, maximum drift velocities, disturbance accelerations, clock synchronization offsets, and contact mode transitions. Under these conditions, the 593.75 ms aisle-ahead horizon and the 66 ms latch-slot horizon represent deterministic, conditional worst-case deadlines within the qualified operating domain. Alternatively, in an empirical or data-driven regime, controlled sensor dropout trials benchmarked against high-precision ground truth can calibrate residual error quantiles (such as p99 or p99.9) across a representative operational distribution. This yields a probabilistic admission bound with a 1 percent or 0.1 percent tail risk, prior to unmodeled structural shifts. In both architectures, the spatial memory record and the consuming controller must explicitly declare the active validation regime, track operating domain invariants, and trigger a verified safe fallback if observations exceed the validity boundary.

↪ The barrier-function permission test that consumes such clearance margins is developed in section 12.5.

Both frameworks converge on one engineering invariant. The invalidation horizon (definition 9.2) marks the precise instant at which the propagated error bound intersects the allowable task tolerance, regardless of whether that bound stems from worst-case physics or calibrated statistical quantiles. Belief expiration is physical rather than computational. It is neither a generic network socket timeout nor an arbitrary 100 ms polling heuristic selected for scheduling convenience. Rather, T_{inv} is a deadline dictated by spatial clearance, machine kinematics, and disturbance dynamics. Beyond $t_0 + T_{\text{inv}}$, downstream controllers cannot assume the estimated pose satisfies the task's safety envelope.

9.7.5 Expiry across the machine's beliefs

A single global staleness timeout across the robot either admits a blind latch insertion long after the finger has drifted past the slot wall, or halts the mobile base in an open aisle while half a second of certified clearance remains. Because operating speed, carried payload, and contact state reshape disturbance bounds dynamically, the invalidation horizon in equation 9.14 cannot be reduced to a single global constant for the entire robot. Instead, the perception and memory interface must compute distinct expiry horizons for each operational regime. Transitions between regimes can alter validity instantaneously; for example, shifting from free flight to mechanical contact introduces impulsive impact dynamics within milliseconds, violating the continuity assumptions of smooth drift models.

Consequently, multiple spatial beliefs coexisting on the same physical platform expire along completely different timelines (table 9.4). A manipulator finger aligning with a latch slot is constrained by tight millimeter tolerances and compounding acceleration disturbances. Simultaneously, free-space claims around the mobile base and tracking estimates of a coworker near the arm share the same standardized human walking speed bound v_h. Even these human-centric beliefs, however, lapse at markedly different intervals, because each originates from a different initial localization uncertainty and must satisfy a distinct geometric tolerance. For volumetric free space, that tolerance is determined by the distance to the nearest occlusion frontier.

Table 9.4: Belief-expiry roots on the machine: Each root solves $E(t) = r_{\text{tol}}$ for that belief's error model and task limit. The two free-space rows are equation 9.13, with limits set by the depth to the nearest unobserved boundary less the protective clearance. The coworker's initial error is illustrative and its tolerance chosen, while its speed is the same walking-speed bound, so all three rows set by a person's motion grow linearly. The last column names the required disposition by that root; any physical response must begin early enough to remain feasible. The mug-handle belief, whose growth is set by the conveyor rather than a person, is worked in section 9.10.

Belief	Error bound $E(t)$ (mm; t in s)	Task limit (mm)	Root	Response at expiry
Free space abreast of the rack end	$50 + 1600t$	50	0 ms	Treat the cross aisle as unknown; refuse dependent advance
Free space to the rack end ahead	$50 + 1600t$	1000	593.75 ms	Refuse motion that relies on the claim; renew from the next frame
Latch-slot finger	$0.1 + 12t + 25t^2$	1	66 ms	Refuse the insertion; hold compliantly or reacquire the slot
Coworker beside the arm	$15 + 1600t$	150	84 ms	Revoke dependent arm motion; stop only if braking clearance remains

When an expiry deadline is reached, the motion permission system must refuse dependent actuation proposals until fresh sensory observations restore valid evidence. Even upon reacquisition, sensory updates renew state authority only under strict validation checks. Upon receiving new sensor data, the estimator computes the measurement innovation (section 9.2), the discrepancy between incoming physical measurements and predicted sensor observations. Incorporating this innovation requires verifying data association, sensor calibration, and physical feasibility constraints. A large innovation residual typically signals an unmodeled disturbance, an outlier measurement, or a tracking failure, which the estimation filter must isolate before granting motion permissions. Conversely, a state update must never trigger discontinuous step commands to low-level motor controllers; trajectory generation must preserve acceleration continuity. Furthermore, as the force-and-bias case of section 9.8 shows, a deceptively small innovation residual in observable dimensions provides no guarantee of accuracy along unobserved state dimensions.

Checkpoint 9.2: Deadlines, frames, and maps

Every deadline in table 9.4 comes from a growth law, a tolerance, and a map that preserved the evidence epochs behind them. These items test all three.

- ☐ **Expiry estimate**: If the coworker's tolerance tightened from 150 mm to 100 mm, with the same initial error and walking speed, what would its invalidation horizon be, and where would the row then fall relative to the latch-slot finger?
- ☐ **Horizon vs. timeout**: Can you explain why a single staleness timeout for every belief must over-admit some rows of the table and under-admit others?
- ☐ **Common-epoch composition**: Can you explain why the same finger misses the slot after one stale frame on a hand-composed chain yet stays within it for several frames when the slot is chained against the wrist pose of its own capture epoch?
- ☐ **Representation choice**: Can you explain why the local snapshot is a Euclidean distance field, and which resource, rather than lookup time, then limits its voxel pitch?
- ☐ **Map revisions**: Can you explain why a loop closure can misplace a remembered cell whose evidence epoch still passes every age check?

Every invalidation horizon is only as trustworthy as its error growth model. Such a model remains dependable only to the extent that the physical system can detect when its assumptions fail, whether through innovation monitoring upon reacquisition or systematic validation during sensor dropout trials. When unmodeled disturbances act along unobservable degrees of freedom, the system can drift far outside its tolerance without generating any detectable residual, which is why conservative geometric clearance and explicit physical bounds remain indispensable.

9.8 Observability Ceilings and Estimation Limits

A force sensor can report the exact same reading when mechanical contact force increases as when its analog conditioning circuit drifts thermally. Sampling that stream repeatedly cannot distinguish the two physical causes. The underlying question is observability, whether a system's available sensor outputs and known physical dynamics suffice to reconstruct the internal states required for control. Consider an instantaneous observation model $\mathbf{y} = h(\mathbf{x})$. When distinct physical states $\mathbf{x}_1 \neq \mathbf{x}_2$ yield identical measurements ($h(\mathbf{x}_1) = h(\mathbf{x}_2)$), the sensor mapping alone cannot separate them. In a combined force-and-bias state, equal and opposite shifts in contact force and sensor bias form an insensitive direction in the observation Jacobian $\mathbf{H} = \partial h / \partial \mathbf{x}$. Because this direction lies in the null space of $\mathbf{H}$, local state variations along it produce zero change in the measurement vector. The measurement likelihood therefore has zero directional derivative along this unconstrained direction, although it may still vary in directions that change the measured sum. Instantaneous ambiguity does not automatically imply unobservability across time; if subsequent motion or control actions induce dynamics that differentiate the states, temporal filtering can resolve the ambiguity. However, if the ambiguity persists along the entire operating trajectory, no amount of signal processing can recover the lost degrees of freedom. Scaling up neural network parameters or adding recurrent layers expands representational capacity, but computation cannot synthesize physical information that no sensor ever transduced.

Observability depends on both the sensor suite and the executed trajectory. When a physical disturbance drives a required state along an unobserved manifold, an estimator can fall into the **zero-innovation trap**. The innovation vanishes ($\nu = \mathbf{y} - \mathbf{H}\hat{\mathbf{x}} = \mathbf{0}$), so no measurement ever contradicts the model. The covariance update depends on the modeled noise and measurement geometry rather than the observed residual. It can contract uncertainty in measured or model-coupled directions without revealing omitted drift along an unobservable direction. A misspecified filter that treats bias as known can therefore contract its reported force uncertainty while the true force and omitted bias drift apart without changing the measured sum.

Consider an illustrative strain-gauge force sensor mounted at the mobile manipulator's wrist, delivering a steady reading of 5 N. Once an unknown bias is introduced, this single reading is compatible with an infinite continuum of true contact forces. An estimator or learned policy that

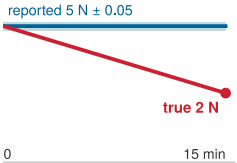
reports this contact force with narrow confidence bounds has not demonstrated that the state is **observable**.[12] When two distinct physical state histories map to the identical sequence of sensor observations, no downstream algorithm, whether an extended Kalman filter or a deep recurrent network, can deduce which trajectory the physical system followed. The digital computing substrate receives identical bitstreams across both realities.

[12] **Observability Rank Test**: For the force-plus-bias state with $\mathbf{H} = [1\ 1]$ and static dynamics $\mathbf{F} = \mathbf{I}$, Kalman's observability matrix stacks $\mathbf{H}$ and $\mathbf{HF}$. Because these two rows are identical, the matrix has rank one rather than two (Rudolf E. Kalman 1960). The unobservable direction corresponds to an equal increase in contact force and decrease in sensor bias, a mode that no number of scalar samples can resolve. Restoring full rank requires an independent sensing channel, an active excitation, or an external reference load.

> **Definition 9.3: Observability**
>
> ***Observability*** is the property that the history of a machine's sensor outputs over a finite time window, combined with its known control inputs and system dynamics, uniquely determines its initial state, ensuring that distinct physical trajectories produce distinct measurement sequences.
>
> 1. **Significance**: Observability defines the hard physical ceiling on state reconstruction, regardless of model architecture or parameter count. In the strain-gauge example below, a 3 N bias shift over 15 minutes alters the true contact force by 60 percent while the innovation remains zero, because the sensor records only their scalar sum.
> 2. **Distinction**: Observability is distinct from predictability. An estimator can project a future state with minimal covariance based entirely on prior dynamical assumptions; only measurements sensitive to that state can verify the prediction against reality.
> 3. **Common pitfall**: A vanishing innovation does not validate state accuracy. Along an unobservable manifold, the innovation residual can remain zero while tracking error grows. A misspecified filter can still contract its reported covariance under its model assumptions; the zero residual does not justify that contraction.

To trace the operational impact of this distinction, consider the wrist gripper regulating contact force against a rigid surface. Let $x(t)$ denote the true normal contact force, while the strain-gauge conditioning electronics introduce an unknown, slowly drifting bias $b(t)$. The raw measurement stream delivered to the estimator at each sample interval is $y(t) = x(t) + b(t)$. Suppose the sensor transmits a constant reading $y(t) = 5$ N. This measurement is satisfied by a true force $x(t) = 5$ N with zero bias $b(t) = 0$ N. However, it is equally satisfied by a true force $x(t) = 2$ N with a sensor bias $b(t) = 3$ N, or even by $x(t) = 0$ N with a bias $b(t) = 5$ N. For any arbitrary offset trajectory $\Delta(t)$, the paired state trajectory $(x(t) - \Delta(t), b(t) + \Delta(t))$ generates an identical measurement sequence $y(t)$. The difference between physical contact force and sensor bias lies within the **unobservable subspace** of the measurement model, the null space in which the two variations cancel exactly in the digitized signal.

Unable to distinguish physical load from electronic drift, an estimator must fall back on prior assumptions about state evolution. If a Kalman filter or recurrent estimator assumes that bias $b(t)$ remains static with near-zero process noise, it attributes all measurement variations in $y(t)$ directly to changes in contact force $x(t)$. As new samples arrive at 1,000 Hz, the estimator repeatedly folds the measurements into its belief state. Because the incoming stream aligns with the forward state propagation, the computed covariance contracts monotonically. The estimator then publishes an aggressively tight uncertainty bound of 5 N $\pm$ 0.05 N at p99. In reality, heat generated by nearby joint actuators warms the mechanical wrist bracket, inducing thermoelastic drift in the strain gauge. Sensor bias $b(t)$ creeps at 0.2 N/min. Over 15 min, this thermal drift shifts $b(t)$ by 3 N. True physical force on the workpiece drops from 5 N to 2 N, a 60 percent loss of grip force, even as the estimator reports high statistical certainty in its original value.



The reported force holds at 5 N while the true force slides to 2 N.

The downstream controller responds to this false certainty by regulating joint torques to maintain what it believes is an optimal 5 N clamping force. If the workpiece requires substantial friction to resist gravity, the actual 2 N contact load allows it to slip from the jaws, even as telemetry shows the grip force sitting exactly on target. Standard filter monitoring cannot catch this failure. The estimator evaluates its innovation sequence (the residual formulation of section 9.2), comparing each incoming measurement $y(t)$ against its predicted observation $\hat{y}(t)$. Throughout the entire drift sequence, the estimator predicts $\hat{y}(t) = \hat{x}(t) + \hat{b}(t) = 5$ N, which exactly matches the incoming sensor reading $y(t) = 5$ N. The innovation $\nu(t)$ evaluates to zero at every sample step. An innovation check tests only whether new sensor data conflicts with the model's forward projection; it cannot expose

errors aligned with an unobservable subspace where the sensor output remains entirely invariant to physical state drift.

Restoring observability requires system designers to introduce an external constraint that breaks the symmetry along the unobservable subspace. An engineering architecture accomplishes this in one of three ways. First, independent sensing can introduce a secondary measurement channel governed by distinct failure physics, such as monitoring joint motor drive currents or optical deflection sensors that do not share the strain gauge's thermoelastic drift. Second, the machine can execute deliberate physical excitation. After securing the workpiece with an external fixture, the arm commands the gripper to break contact briefly for 40 ms, establishing a known zero-force boundary condition ($x(t) = 0$ N) where the raw telemetry directly exposes the sensor bias $b(t)$. Third, the machine can rely on periodic external anchoring, bringing the end-effector into contact with a calibrated reference load cell during scheduled tool-change routines to zero the bias estimate. The mobile manipulator's base already depends on this third remedy. Wheel odometry leaves its longitudinal position along the aisle unobservable, and only the map-based corrections of section 9.5.1 supply the missing spatial constraint. When no admissible excitation separates two states that call for different responses before the machine must act, and no added channel is available, the states are observationally indistinguishable and no model on that channel can tell them apart. Section 18.3.4.1 develops this limit and the operating restrictions it forces.

When a physical state cannot be constrained through complementary sensing or active excitation, the estimation layer must treat that state as unobservable and refuse to publish unwarranted precision. The software interface must either mark the affected state dimensions as invalid or inflate their uncertainty bounds based on the hardware's worst-case physical drift rate, overriding the artificially contracted covariance generated by the filter. Formal observability analysis provides the mathematical foundation for identifying these blind spots. Classical linear state-space formulations originate with Rudolf E. Kalman (1960), while nonlinear extensions establish differential observability rank conditions on the Lie derivatives of the observation field (Hermann and Krener 1977). Systems engineers do not recompute these differential rank conditions in real-time control loops. Instead, the design task is to identify during system modeling which operating regimes lose observability rank, ensuring that runtime state estimators never claim statistical convergence along an unconstrained physical dimension.

Deterministic observability establishes whether system dynamics and sensor equations can distinguish trajectories over time, independent of stochastic noise assumptions. Nevertheless, practical estimator convergence and calibrated covariance bounds demand further conditions: accurately identified kinematic parameters, persistent trajectory excitation, and well-characterized sensor noise statistics. A manipulator trajectory can, for instance, leave a required state direction unconstrained by the available measurements; this estimator observability condition must be checked separately from the arm's kinematic singularities. Robust physical AI systems therefore deploy runtime monitors that track the validity conditions of each estimator. These watchdogs cross-check redundant sensing channels, track kinematic condition numbers, and enforce re-zeroing schedules. While runtime monitors cannot guarantee the truth of every modeling assumption, any detected violation must immediately invalidate the dependent state estimate. When an estimator explicitly revokes its own confidence, downstream controllers can transition cleanly to a validated fallback policy. An estimator that invalidates its own output fails safely and visibly; the dangerous failures in physical AI occur when an estimator reports tight confidence along an unobservable manifold, silently blinding the controller to physical reality.

↪ The Kalman and Hermann-Krener observability rank conditions are developed in section B.8.

9.9 Spatial Belief Failure Modes and Map Corruption

The failure modes of spatial memory rarely announce themselves with operating system exceptions or kernel panic logs. When a corrupted pointer or segmentation fault halts a process, the failure is loud and immediate, even if bringing the physical mechanism safely to rest requires dedicated fallback hardware. In contrast, spatial memory failures are quiet and deceptive. The memory subsystem continues publishing well-formed, syntactically valid data structures at high frequency, while the geometric coordinates inside those structures have parted ways with physical reality.

This divergence exposes the **middleware QoS mirage**, the memory-side counterpart of a limit section 4.3.2 established for transport, where no middleware setting could stand in for the chunk

lease. DDS-based robotics middleware, including ROS 2, provides Quality of Service (QoS) policies such as `Deadline` and `Liveliness`. These policies monitor declared message intervals and publisher liveliness assertions, not the physical truth of the payload. If an upstream camera driver deadlocks or an object tracker loses its target while a republisher continues sending its cached matrix and satisfying those communication contracts, the policies need not report a violation. Physical evidence validation must therefore accompany transport monitoring; otherwise frozen transforms, stale state-store entries, and phantom tracks can reach motor controllers as apparently healthy data.

Consider a camera that abruptly stops delivering frames while an intermediate transport node continuously republishes the last calculated transform under fresh timestamps. A downstream controller with a 5 ms publish-time freshness check accepts every message without protest. The scenario separates two kinds of failure in an embodied system. In a **loud failure**, a component crashes, halts execution, drops network packets, clears an output buffer, or asserts an explicit hardware error bit. Downstream consumers detect these faults through explicit validity flags or bounded stream timeouts. Avoiding harm then requires a qualified fallback (dynamic braking, position hold, or compliant impedance control) whose detection and response delays fit the remaining physical margin. In a **silent failure**, the spatial memory subsystem continues publishing structurally complete, numerically well-formed state records with valid status bits, low covariance flags, and monotonically increasing timestamps, even as the coordinates within those records drift away from the physical machine. Downstream control loops consume these corrupted records without raising an exception, closing feedback loops around phantom geometry. Physical actuators execute commands derived directly from these plausible yet false trajectories, such as driving an arm toward a target that now intersects a wall. Silent map corruption can therefore cross the causal boundary and inject destructive forces into the environment long before any software monitor registers an anomaly.

When an upstream sensor halts entirely, the timestamp anti-pattern of section 9.4 manifests as the **frozen transform**. This represents one of the most deceptive silent failures in embodied robotics because republishing a stale spatial snapshot with updated transport timestamps masks a dead sensor. In an illustrative timeline on the mobile manipulator, the wrist camera's tracking pipeline computes the target object's transform $\mathbf{T}_{\text{camera}}^{\text{object}}$ at timestamp $t_0 = 0$ ms. If the optical sensor subsequently disconnects, drops frames, or suffers thread starvation at $t = 10$ ms, the tracking algorithm ceases to produce new spatial evidence. If the message-passing middleware or an intermediate transport node retains that last computed matrix in an unexpired buffer and republishes it at 1,000 Hz, generating a synthetic transmission timestamp $t_k = t_0 + kT_{\text{rep}}$ with period $T_{\text{rep}} = 1$ ms, it erases the temporal provenance of the original measurement. A downstream tracking controller verifies freshness by computing data age as $\Delta t = t_{\text{current}} - t_{\text{msg}}$. Because the republisher stamped the message with the current system clock, Δt evaluates to less than 1 ms, satisfying the freshness threshold of $t_{\text{age,max}} = 5$ ms. The controller consumes the transform as current evidence. If the mug rides the pick-station conveyor at 0.20 m/s while the reported transform remains static, the controller accumulates 20 mm of spatial tracking error in 100 ms. This exceeds the 15 mm grasp tolerance, yet the estimator reports zero estimated tracking divergence.

Fresh stamps on a dead camera pass the age check while the error reaches 20 mm.

Phantom-object tracks arise when an estimator decouples the lifetime of spatial coordinates from the lifetime of existence evidence. Target tracking literature has long treated existence as an independent random variable alongside spatial kinematics: integrated probabilistic data association carries an explicit existence probability that updates with every sensor scan (Mušicki et al. 1994). Modern learned trackers implement this separation in different ways. ByteTrack pairs object detection with a Kalman motion model, maintaining unassociated tracks in a grace buffer across a fixed window of missed frames (Zhang et al. 2022); query-based architectures like MOTR persist learned object queries across successive video frames (Zeng et al. 2022). In either design, when an object is occluded or removed from the scene, the tracker enters the coasting state of section 9.2.2 and continues propagating its state vector $\mathbf{x} = [\mathbf{p}^\top, \mathbf{v}^\top]^\top$ under an open-loop motion model. If retention rules are overly generous, the estimator maintains a persistent phantom entity in the state store. The path planner routes the manipulator around empty space, or an automated grasping routine closes its fingers on vacuum. Conversely, evicting an occluded obstacle prematurely creates a severe collision hazard when the arm sweeps through the physical space. Resolving phantom tracks requires moving beyond passive measurement absence or arbitrary heuristic timeouts to active negative evidence

(section 9.6). When a line-of-sight sensor ray sweeps the candidate volume and encounters zero density, raymarching actively evicts the phantom obstacle from the 3D state store as soon as line of sight confirms clearance. In manipulation, unfulfilled contact expectations (such as zero load at a predicted contact boundary) immediately trigger physical hypothesis invalidation.

Coordinate frame misattributions divide along a similar boundary between syntax and semantics. A **schema-detectable frame mismatch** occurs when a publisher supplies a malformed frame identifier or a value outside a declared frame enumeration. Static schema enforcement can reject that packet at the serialization boundary. However, a well-formed identifier can still name an undefined link in the transformation tree graph. Resolving an otherwise well-formed identifier against the current transform tree requires a frame-aware validator or consumer; the transport schema alone does not establish that the named parent link exists. In contrast, a **semantically wrong frame assignment** occurs when an engineer or configuration file assigns a valid, type-correct frame identifier that represents the wrong physical link. For example, commanding the mobile manipulator's fingertip trajectory using the transform of the arm's bare mounting flange $\mathbf{T}_{\text{flange}}$ instead of the gripper's tool tip $\mathbf{T}_{\text{tool}}$ satisfies every schema validation check, type assertion, and tree traversal test. In an illustrative setup with a tool tip 150 mm beyond the flange along the tool z-axis, a flange-as-tip error shifts the intended contact point by that offset. Whether contact occurs depends on the fixture geometry and admitted path. Detecting a semantic frame error requires physical consistency checks, such as verifying measured motor torques against an analytical dynamic model of the arm, or cross-referencing multi-camera optical tracker observations against joint encoder kinematics.

When sensor updates drop out, an estimator relies entirely on its predictive forward model to advance state estimates across the blind interval. A well-formulated filter propagates the mean state estimate $\hat{\mathbf{x}}(t)$ and covariance $\mathbf{P}(t)$ under its stated dynamics and process-noise model. The worked drift model grows position uncertainty during the gap; general covariance propagation need not be monotonic in every direction. An **overconfident predictor** understates process variance $\mathbf{Q}$, holding $\mathbf{P}(t)$ artificially compact over an extended prediction duration, such as beyond an illustrative 200 ms. When fresh sensor measurements eventually arrive, the artificially small covariance causes the filter to discount incoming observations; it weights its internal dead-reckoning so heavily that it classifies valid measurements as statistical outliers and rejects them. Engineers identify overconfidence during system qualification by executing staged synthetic occlusions, dropping sensor frames for a range of intervals and evaluating the innovation ν at reacquisition against the covariance $\mathbf{S}$ the filter predicted for it. These are the NIS and NEES tests of section 9.2.1 applied at the end of a gap rather than in steady state. Averaged over the staged trials, the NIS at reacquisition, and the NEES against ground truth, must fall inside their chi-squared acceptance intervals (Bar-Shalom et al. 2001), and an average above the interval indicates an overly optimistic process noise model. Drift along an unobservable degree of freedom, such as workpiece slippage inside compliant gripper pads, produces no reacquisition residual at all (section 9.8). At vehicle scale, a terrain belief stands until the wheels themselves contradict it.

War Story 9.1: A rover trapped in sand (2009)

Context: By 2009 the *Spirit* rover had been driving for three years with an inoperative right-front wheel drive, dragging the dead wheel behind it as it drove backward on drives planned by operators on Earth. In the valley west of Home Plate, at a site the team named Troy, it drove onto soil that looked like the ground it had already crossed (Arvidson et al. 2010).

Mechanism: A thin sulfate-rich crust covered loose sulfate and basaltic sand. The surface's appearance, which is all that orbital and rover cameras observe, carried no information about the shear strength underneath. Climbing soil-covered slopes on five working wheels already produced high slip, and at Troy the wheels broke through the crust and excavated the sand beneath instead of advancing the chassis (Arvidson et al. 2010).

Impact: *Spirit* became embedded in late April and early May 2009. Months of extrication attempts, rehearsed on an Earth testbed, failed, and in January 2010 the mission redesignated the rover a stationary research platform.

Response: Slip detection already existed. Visual odometry, which compares the rover's tracked motion with its commanded wheel rotation, had been used on both rovers to measure slip and stop drives when it exceeded a limit (Maimone et al. 2007). That measurement bounded how deep each drive could dig once slip appeared, but it could not warn before the wheels broke through.

Systems lesson: Traversability inferred from appearance does not observe the property that matters, the soil's strength, so a stored traversability belief can be well formed and wrong. Slip is the physical residual that exposes it, and the belief must expire the moment that residual departs from its bound.

Frozen transforms, phantom tracks, misassigned frames, and overconfident predictors each demand distinct validation checks. The belief record of section 9.3 makes their provenance, age, frame, and growth assumptions available to consumers; independent physical checks still have to establish whether those claims hold. The filled record below collects those obligations for the latch-slot finger and the moving mug.

9.10 The Belief Record Schema

The belief record introduced in section 9.3, the seventh record in the record chain, binds each state estimate to its capture epoch, coordinate frame, and error-growth law so that downstream controllers can catch a frozen transform, a chimeric frame chain, or a prediction that has outlived its slot tolerance. This schema unifies discrete tracked entities and continuous volumetric representations (table 9.2) under one set of time and error rules, serving both trajectory planning on the host and bounded local permission checks on real-time microcontrollers. From the observation contracts established in section 8.8, the record inherits the raw sensor capture epoch, clock identifier, and hardware conversion bound. For differently constrained state components, it preserves separate evidence epochs or a conservative shared epoch and growth bound, as required by section 9.4. With these fields, the safety permission controller translates the sensor's capture timestamp into its own local monotonic clock with bounded synchronization error before evaluating evidence age. Persistent memory contributes five fields of its own. An estimate epoch dates the propagated state without renewing physical evidence. An uncertainty-growth bound projects error expansion across blind intervals. A validity region defines the operating envelope where that bound holds. The record also carries a physical invalidation deadline. Finally, a query-work allocation is set to at most 2 µs per local collision query in the reference design, and it becomes a justified execution bound only after qualification on the chosen hardware. Across its lifetime, each record moves through the four-state lifecycle of figure 9.12, whose guards enforce two rules. Only fresh evidence returns a record to `VALID`, and leaving `UNOBSERVABLE` also requires restoring the missing physical constraint.

The transitions enforce strict physical invariants across the spatial cache:

1. `VALID` → `DEGRADED`: Occurs upon sensor occlusion or field-of-view exit when direct measurement ceases, but the declared growth law still supports the task. Downstream consumers expand safety margins according to that law: stochastic models require calibrated coverage percentiles, whereas bounded-disturbance models enforce worst-case physical limits.
2. `DEGRADED` → `EXPIRED`: Occurs deterministically when projected uncertainty breaches task tolerance ($E(\Delta t) \geq r_{\text{tol}}$) or operating conditions leave the declared validity region. The entry immediately ceases to support dependent motion commands. Safety controllers must always reserve a feasible fallback trajectory in advance, because invalidating a software record cannot arrest physical momentum.
3. `ANY` → `UNOBSERVABLE`: Occurs immediately when physical observability rank collapses, such as unconstrained sensor bias drift under thermal changes or mechanical slippage along unmonitored degrees of freedom.
4. `ANY` → `VALID`: Requires fresh physical evidence ($t_0 = t_{\text{fresh}}$), innovation residuals passing gating checks, and revalidation of the dynamic model within the operating regime. Returning from `UNOBSERVABLE` additionally requires a restored physical constraint that resolves the rank deficiency; acquiring another ambiguous sample from the unconstrained sensor is insufficient.

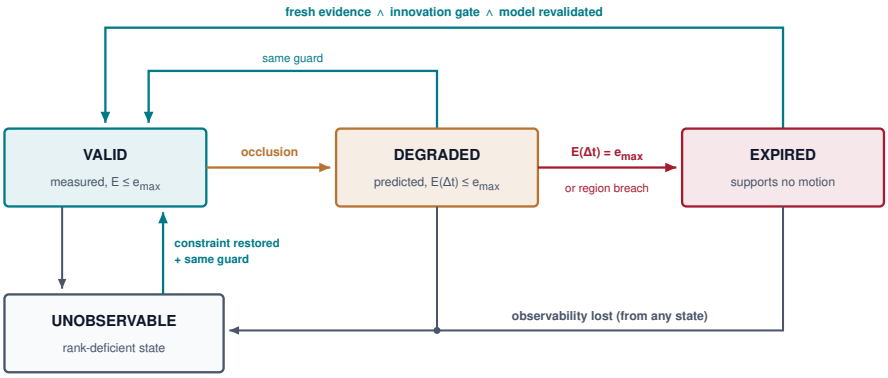


Figure 9.12: Spatial belief state machine and transition invariants: Each state carries its invariant inside the box and each edge its guard. Occlusion moves a record from VALID to DEGRADED, where the error bound grows under the declared law, and the bound reaching r_{tol} or a breach of the validity region moves it to EXPIRED. Every edge back to VALID passes the same evidence guard, so freshness alone cannot restore observability and extrapolation cannot renew validity; leaving UNOBSERVABLE also needs the missing constraint restored.

A belief record can never transition from `DEGRADED` or `EXPIRED` back to `VALID` through predictive extrapolation, autoregressive rollout, or software republishing.

Table 9.5 instantiates this schema for the latch-slot finger as a concrete bounded-disturbance belief record, and the prose that follows pairs it with the linear-drift record for the moving mug handle. Together, the two records show how the same fourteen fields accommodate both accelerating manipulation errors and constant-velocity transport bounds.

Field	Example record value	Consumer check

Table 9.5: Belief record: Value, unit, entity, frame, two epochs, clock conversion, error model, validity region, expiry, query latency budget, provenance, and lifecycle travel together as the seventh record in the record chain. A later estimate epoch dates the represented state, not a new measurement, and cannot extend expiry past $t_0 + T_{\text{inv}}$.

Field	Example record value	Consumer check
Entity and quantity	Gripper finger relative to the latch-release slot, lateral offset from the slot centerline	Match the target entity and task quantity
Value and SI unit	Lateral offset, stored in meters; the bounds below are shown in millimeters for reading	Decode as meters, not millimeters or force
Frame	`latch_slot` frame, with the transform revision of the wrist-camera calibration	Resolve a time-valid transform chain
Evidence epoch	t_0, the last wrist-camera capture before the finger occludes the slot, on the wrist camera's capture clock	Preserve across prediction and republishing
Estimate epoch	Latest propagation of the offset, on the same capture clock	Date the propagated estimate; do not renew evidence
Clock conversion	Versioned $t_{\text{MCU}} = a t_{\text{capture}} + b_{\text{clk}}$, error at most 0.2 ms	Refuse if conversion expires; add error to upper age
Initial error	e_0 = 0.1 mm	Check the initial bound against calibration
Growth law	$E(t) = 0.1\ \text{mm} + (12\ \text{mm/s})\,t + \frac{1}{2}(50\ \text{mm/s}^2)\,t^2$	Compare at evidence age with the slot tolerance
Validity region	Arm acceleration and contact mode unchanged since t_0; drift and disturbance bounds above validated within this region	Expire immediately on a mode or residual-bound violation
Expiry	t_0 + 66 ms, where the growth law reaches the 1 mm slot tolerance	Refuse dependent proposals at expiry; reserve time for an earlier physical response
Provenance	Wrist-camera observation ID, calibration revision, estimator build hash	Match approved sources; hash is version provenance, not payload integrity
Lifecycle	`VALID` at capture, `DEGRADED` once the finger occludes the slot, `EXPIRED` at t_0 + 66 ms	Never restore `VALID` without fresh physical evidence
Map cell evidence, when applicable	Not used by this record; a map belief such as the rack-end cross aisle carries a per-cell capture epoch and free/occupied/unknown state	Check a validated conservative snapshot certificate covering every supporting cell; unknown stays unknown
Collision query budget	$t_{\text{query}} \leq 2$ µs (qualified local clearance and required gradient lookup)	Qualify at most 2 µs per local query and the full collision workload within its allocation

The same schema governs the target belief consumed during grasping, the six-degree-of-freedom pose of a mug handle as the container advances along a takeaway conveyor toward the robotic manipulator. This record anchors its pose in the robot base frame, registers its evidence epoch to the wrist camera's final pre-occlusion capture, and synchronizes clocks under the same 0.2 ms conversion bound. At capture, calibration limits initial pose error to $e_0 = 3$ mm. Its error-growth law differs from that of the latch-slot finger. Because the conveyor belt advances at a bounded velocity of at most 0.20 m/s without independent acceleration, unobserved positional drift accumulates linearly over time according to equation 9.15, reaching the 15 mm grasp tolerance at $t_0 + 60$ ms. That is the freshness deadline section 2.2 derived for this same mug, from the same numbers; what memory adds is that the deadline now travels inside the record, beside the evidence epoch it is measured from. The validity region requires that the belt run forward within this rated speed and that the mug remain upright and seated. Any operational violation (a conveyor stall, a speed surge, or a shock that tips the mug) expires the record immediately regardless of elapsed time. In the worked tracked-evidence model, an active visual tracker compensates the nominal conveyor motion, leaving residual belt slip as the declared growth term. The tighter horizon requires qualification of the tracker's initial error, capture-to-use age, and residual dynamics; frame rate alone cannot establish it. This is the tracked evidence horizon met in section 3.5.2 and carried into the intent lease in section 10.5.

Downstream consumers evaluate either record through an invariant admission procedure. The controller converts the sensor capture epoch to its local clock, adds the synchronization uncertainty margin to the elapsed age, verifies the lifecycle status and validity region, and evaluates the growth law against the clearance budget of the proposed trajectory. Continuous-field representations follow an equivalent procedure, evaluating local signed-distance or occupancy snapshots against spatial clearance bounds within their qualified query allocations. Passing this admission check admits the belief to the **separate permission decision**; it does not command actuator motion. When an updated record fails admission or expires, an active motion command may continue only until the expiry of the record on which it was admitted. If that expiry arrives before a refreshed record does, the real-time controller must commit to a verified fallback maneuver before physical margins are exhausted. Notebook 9.3 walks through this admission evaluation for the mug-handle belief.

Napkin Math 9.3: The admission margin

Problem: A trajectory proposal requests closing the gripper on the mug handle 35 ms after the evidence epoch, measured on the consumer's local clock. Does the spatial belief remain admissible for the grasp at that instant, and how much validity margin remains?

Variables:

- Initial error $e_0 = 3$ mm and conveyor speed bound $v_{\text{conv}} = 0.20$ m/s (both illustrative); grasp tolerance $r_{\text{tol}} = 15$ mm (chosen); expiry $T_{\text{inv}} = 60$ ms (derived from equation 9.15).
- Clock-conversion bound $\epsilon_{\text{sync}} = 0.2$ ms (illustrative) and execution age 35 ms (illustrative).
- Lifecycle `DEGRADED`, with validity region intact: the belt advances forward within nominal limits and the mug remains seated.

Math: The consumer charges the clock synchronization bound to the elapsed age, evaluates the growth law at this upper bound, and computes the remaining belief lifetime:

$$\Delta t_{\text{upper}} = 35 + 0.2 = 35.2 \text{ ms}$$

$$E = 3 \text{ mm} + 0.2 \text{ mm/ms} \times 35.2 \text{ ms} = 10.04 \text{ mm} \leq 15 \text{ mm}$$

$$60 - 35.2 = 24.8 \text{ ms}$$

Result: The belief passes admission at the stated instant, retaining 4.96 mm of clearance margin and 24.8 ms of belief lifetime; the limiting evidence-use age is 59.8 ms after the evidence epoch.

Systems insight: Belief admission supplies a time-bounded input to the separate motion-permission decision. No grasp duration has been supplied, so the limiting evidence-use age is not a guaranteed completion time. The motion controller must establish a feasible completion or fallback before physical margins are exhausted within the remaining 24.8 ms. If an external disturbance knocks the mug loose, the record expires immediately despite valid timestamps, because the physical system has breached the validity region for which the growth bound was derived.

Checkpoint 9.3: Belief expiry and the memory record

These items close the chapter's body: the epochs a record carries, the deadline that ends it, the states no estimator can recover, and the lifecycle and admission check that govern all three.

- ☐ **Epoch classification**: Distinguish the evidence epoch (t_0) from the estimate epoch (t_{est}) and publication epoch (t_{pub}). Why does stamping an estimate with its publication time conceal the physical error accumulated since capture?
- ☐ **Admission margin**: For the mug-handle record, why does the consumer add the clock-conversion bound to the elapsed age before evaluating the growth law, and what must the controller do with the remaining lifetime once admission passes?

- ☐ **Unobservable manifold hazards**: Explain why vanishing innovation residuals ($y - \hat{y} = 0$) fail to reveal sensor bias drift (such as thermal expansion from nearby actuators) along unobservable degrees of freedom.
- ☐ **State lifecycle state machine**: Trace the deterministic transitions between `VALID`, `DEGRADED`, `EXPIRED`, and `UNOBSERVABLE`. Why does renewing a record require fresh physical evidence, and why does recovering from `UNOBSERVABLE` require restoring an underlying physical constraint?
- ☐ **Belief record**: Enumerate the required fields of the belief record (the seventh record in the book's architecture), including hardware capture epochs, qualified collision-query allocations, and task-space validity regions.

9.11 Fallacies and Pitfalls

Evaluating a spatial memory architecture requires auditing how separate state beliefs interact across time, coordinate frames, and physical occupancy. In physical AI, planning failures frequently occur when systems treat software records as static facts rather than time-decaying physical hypotheses. Two records can each be internally valid and fresh according to local timestamps, yet combining them without temporal reconciliation yields an inconsistent or hazardous model of the workspace. Before authorizing mechanical motion, a real-time controller must reconcile disparate sensor epochs, velocity bounds, and physical occlusion boundaries.

Pitfall: *Evaluating spatial clearance between moving bodies using asynchronous timestamps.*

In distributed robotics architectures and multi-sensor machine learning pipelines, visual perception introduces tens of milliseconds of latency across CMOS readout, PCIe DMA transfers, and neural forward passes. On the warehouse mobile manipulator, the chunk policy's inference pass alone, including the vision encoder, requires an illustrative 40 ms at p99, even before sensor capture and bus transport are counted. Concurrently, the base IMU streams at 1 kHz and the low-level drive controller updates at 100 Hz. When an autonomous base drives toward a rack end at its 1.5 m/s drive limit while a coworker walks toward the base at the 1.6 m/s walking-speed bound, their relative closing velocity reaches 3.1 m/s.

Consider a favorable case in which the visual detection record is only 20 ms old, less than the inference pass alone takes, while the drive controller's odometry is current at 0 ms. If both estimates fall within the 593.75 ms aisle-ahead expiry window, both pass individual freshness checks. Differencing their coordinates directly, however, conflates two incompatible physical epochs. For this separate illustrative comparison, stipulate 200 mm of available separation. Subtracting the 50 mm localization bound and the 100 mm protective reserve leaves 50 mm for temporal mismatch. Projecting both velocity bounds across the 20 ms gap gives a conservative common-time displacement envelope of 62 mm, exceeding that slack. The sum-of-speeds envelope deliberately reserves both motions; it is not the exact residual displacement when odometry is already current. Even this favorable record-age case therefore fails the chosen conservative check. Freshness tests verify only single-sensor recency; they cannot certify joint safety across moving reference frames. The controller must propagate every spatial estimate to a shared evaluation epoch using calibrated growth bounds, or enforce a strict timestamp skew limit derived from available clearance margins. **Individually fresh records measured at disparate epochs cannot establish joint clearance between moving bodies.**

Architect's countermeasure: Project all moving candidate bounds to a unified evaluation epoch using validated velocity bounds, or bound maximum allowable timestamp skew from worst-case closing velocities and available clearance margins.

Fallacy: *Bounded spatial covariance proves that a physical entity still exists in the workspace.*

When a coworker or moving obstacle passes behind an occlusion, such as a warehouse rack, downstream trackers frequently suffer from phantom track divergence. An autoregressive state predictor or Kalman filter continues extrapolating the entity's position along its last-observed velocity vector. If the filter configuration underestimates the process noise $\mathbf{Q}$, the predicted spatial covariance ellipse $\mathbf{P}_{pp}(t)$ remains deceptively compact and stays well below the track-pruning threshold. Motion

planners, interpreting this small covariance as a high-confidence localization, chart aggressive trajectories around the phantom track while treating the surrounding unmonitored space as fully accounted for.

This operational assumption conflates two orthogonal estimation metrics. The spatial covariance matrix $\mathbf{P}_{pp}(t)$ quantifies positional dispersion strictly conditioned on the hypothesis that the entity is present. It yields no guarantee regarding the marginal existence probability $\Pr(\text{exists} \mid \mathbf{y}_{1:k})$, which formal tracking theory formulates as an independent state variable (Mušicki et al. 1994). If the worker halts, reverses course, or leaves the aisle through a side doorway while occluded, the compact ellipse reflects only the open-loop momentum of the internal motion model. Spatial memory systems must explicitly decouple geometric state estimation from existence probability. Confirming or pruning occluded tracks demands active negative line-of-sight evidence, such as depth-sensor ray marching that shows an empty volume, or direct tactile feedback. **A compact spatial covariance ellipse reflects only conditional geometric dispersion under an assumed motion model, never the physical existence of an occluded entity.**

Architect's countermeasure: Decouple spatial state estimation from existence probability modeling, and require active negative line-of-sight raymarching or physical contact to confirm or retire occluded tracks.

Fallacy: *A voxel marked empty by a sensor ray remains traversable free space until an obstacle is observed.*

In 3D scene representations such as neural radiance fields (NeRFs), 3D Gaussian splatting, or static voxel occupancy grids, regions where rays encounter zero density or complete transmittance during an initial scan are frequently cached as permanently traversable free space. In dynamic warehouse and industrial environments, however, unobserved space is inherently non-static. Workers walk around blind corners, automated guided vehicles reverse into aisles, and access doors swing open without generating sensor returns in occluded sectors. Treating an unmonitored voxel as clear simply because a sensor ray traversed it seconds earlier is the **static world fallacy**.

Traversability represents a time-decaying physical hypothesis rather than an immutable geometric property. The certified safe depth within any unobserved region recedes continuously from its boundary, shrinking at the maximum velocity bound of dynamic agents that could enter it. The free-space erosion law in equation 9.13 establishes the precise operational lifetime of each clearance hull: $T_{\text{inv}} = (d_{\text{free}} - \delta_{\text{loc}} - \delta_{\text{margin}})/v_h$. Once this deadline passes, eroded voxels must revert to an unknown state, compelling motion planners and safety interlocks to treat them as potential hazards or bring the machine to a controlled stop. Beside the warehouse rack end, this validity deadline is 0 ms, so the claim lapses at the instant of capture. **Certified free space erodes continuously across occluded boundaries at the worst-case entry velocity of unobserved dynamic agents.**

Architect's countermeasure: Timestamp every free-space voxel with its `free_space_sweep_epoch`, build the conservative snapshot with unsupported space treated as obstacles through its independently validated support deadline, and enforce that deadline locally. Per-hull bounds from equation 9.13 may tighten use without charging an erosion already reserved in the field again.

9.12 Summary

Spatial memory begins with an estimator. In the worked drift model, prediction widens state uncertainty, while independent, synchronized measurement updates contract it. Fusing LiDAR returns with stereo depth estimates yields a tighter state estimate than either sensor can deliver in isolation, provided each measurement is registered at its true capture epoch. Applying a delayed measurement at the wrong epoch shifts the state mean while leaving its calculated variance artificially confident. In a real-time system, filter covariance represents a hypothesis about uncertainty rather than an empirical guarantee. Normalized estimation error squared (NEES) tests this claim offline against ground-truth trajectories, while windowed normalized innovation squared (NIS) tests monitor measurement-space consistency online under their independence and model assumptions. Innovation monitoring cannot replace capture-epoch validation or expose unobservable state error. Object tracking attaches identity and velocity to spatial clusters without delaying the raw occupancy evidence needed for emergency braking. When late measurements arrive, the estimator rewinds the filter to the capture epoch, incorporates the delayed correction, and replays state propagation back to the current execution cycle.

Spatial memory preserves the estimator's state across temporary sensing dropouts and occlusions, maintaining physical state independently of transient policy context. A complete belief record encapsulates the coordinate frame, evidence epoch (t_0), estimate epoch (t_{est}), error-growth law, and validity region required to evaluate remembered state. Kinematic forward models can propagate the represented state to a later estimate epoch (t_{est}), but the evidence epoch (t_0) remains anchored to the physical measurement that generated it. Coordinate transforms age like any other spatial belief. Composing a kinematic transform tree from edges sampled at mismatched epochs creates a synthetic pose the physical robot never occupied, with kinematic lever arms amplifying rotational skew into substantial position errors. Internal representations must directly match the queries executed on the critical path. A validated conservative Euclidean distance snapshot supplies a hull's clearance with eight geometric corner reads and fixed header checks at any pitch, which is why the permission controller's local snapshot is a distance grid. Rendering representations such as radiance fields and Gaussian splats can serve planning but cannot certify clearance. Downstream safety consumers must verify the supporting transform chain, validate cumulative error bounds, and check query execution budgets before admitting any remembered state into safety-critical gating decisions.

Belief expiry is governed by the task tolerance and a validated error-growth model. Free space erodes inward from unobserved boundaries at the maximum velocity of any hidden obstacle. Consequently, spatial memory certifies corridor clearance only to a depth dictated by those unobserved margins: 593.75 ms for the open aisle ahead, and zero usable clearance abreast of a blind rack corner, where the localization bound and the protective clearance consume the entire side gap. Odometry drift continuously widens positional uncertainty between map corrections, and pose-graph loop closures can shift the coordinates of remembered cells without renewing the physical evidence supporting them. Spatial memory introduces a hard deadline into the stopping budget, but it cannot synthesize missing distance; encountering an approaching pedestrian still requires qualified active-sensing crossing detectors or physical operational design constraints that eliminate blind encounters. On grid maps, only valid negative evidence, from direct and unoccluded line-of-sight rays, can substantiate a free-space claim. Neither a synthetic view rendered from a generative prior nor the periodic publication of an unchanged occupancy map can renew an occluded cell.

Superficially fresh data records can mask dangerous underlying staleness. A frozen transform node may broadcast at 1,000 Hz while repeating obsolete geometry, phantom tracks can mistake measurement variance for obstacle existence, and pipelining skews combine state vectors that never existed simultaneously in physical time. Worse, unobservable state drift evades innovation checks entirely when distinct physical configurations produce identical sensor returns. Under unobservable conditions, an estimator can drift silently while reporting zero residuals. Restoring system safety requires independent sensing modalities, deliberate motion excitation, or external spatial anchors before the filtered state can safely authorize machine action.

Across these stages, spatial memory turns dated sensor measurements into bounded, checkable claims about the surrounding world and retires each claim the moment its physical margin is spent. The chapter's estimation, representation, and expiry decisions rest on seven relations, which table 9.6 lists alongside the decision each settles and its derivation location.

Table 9.6: Key equations of spatial memory: The seven relations behind the chapter's estimation, representation, and expiry decisions, with the place each is derived.

Equation	What it bounds or decides	Derived
$\mathbf{P}_{k\|k-1} = \mathbf{F}\mathbf{P}_{k-1\|k-1}\mathbf{F}^\top + \mathbf{Q}$	How prediction propagates the estimate's uncertainty between measurements	equation 9.1
$\mathbf{G}_k = \mathbf{P}_{k\|k-1}\mathbf{H}^\top\mathbf{S}_k^{-1},\ \mathbf{P}_{k\|k} = (\mathbf{I}-\mathbf{G}_k\mathbf{H})\,\mathbf{P}_{k\|k-1}$	How much a dated measurement moves and narrows the estimate; valid only when the measurement is applied at its capture epoch	equation 9.2
$d_M^2 = \nu^\top\mathbf{S}^{-1}\nu \le \gamma_{\text{gate}}$	Whether a detection may be associated with a track; averaged over independent innovations, the same statistic tests measurement-space consistency	equation 9.3

Equation	What it bounds or decides	Derived
$\sum_{k=1}^{N_{\text{hulls}}} t_{\text{query}}(\mathbf{p}_k) \le T_{\text{reflex_budget}}$	The per-query time, including geometric reads and fixed validity checks, a clearance lookup may spend inside the permission tick	equation 9.7
$M_{\text{dense}} = (V_{\text{ws}}/r^3)\, s_{\text{voxel}}$	Voxel payload at a given pitch; full allocation also includes metadata, buffers, binary, and workspace	equation 9.8
$E(\Delta t) = e_0 + v_{\text{drift}}\Delta t + \frac{1}{2} a_{\text{dist}}(\Delta t)^2$	The bounded error of a belief propagated without new evidence	equation 9.12
$T_{\text{inv}} = \inf\{\Delta t \ge 0 : E(\Delta t) \ge r_{\text{tol}}\}$, free space: $T_{\text{inv}} = (d_{\text{free}} - \delta_{\text{loc}} - \delta_{\text{margin}})/v_h$	The deadline after which a belief supports no motion; for free space, how long remembered emptiness lasts at a given depth from the unobserved boundary	equation 9.14, equation 9.13

Key Takeaways: Evidence age and belief expiry

- **An estimator's covariance is a claim to test**: In the worked drift model, prediction widens state uncertainty while independent, synchronized measurement updates contract it. When updates arrive with misattributed timestamps, the filter contracts its covariance around an erroneous state. System architects validate filter consistency offline via normalized estimation error squared (NEES) against ground truth and monitor innovations online with windowed normalized innovation squared (NIS) tests under their independence and model assumptions. Innovation monitoring cannot replace capture-epoch validation or expose unobservable state error.
- **Occupancy precedes identity confirmation**: Reflex safety logic must act on raw occupied space immediately; an unconfirmed or tentative track must never suppress an underlying occupancy claim. When downstream decisions require confirmed semantic identity or velocity estimates, the system must account for multi-frame confirmation latency in its stopping budget. Deleting a lost track returns that volume to an unknown state rather than declaring it free space.
- **Prediction advances state, never evidence**: Kinematic extrapolation advances the state estimate (t_{est}) into the future, but the evidence epoch (t_0) remains frozen at the timestamp of the last physical sensor acquisition. Safety-critical physical claims must persist in explicit belief records with immutable evidence timestamps, even when higher-level neural policies evict their context tokens.
- **Representation follows the consumer's query**: Internal representations must align with the operational queries executed on the critical path. Validated metric distance snapshots support bounded collision clearance checks through geometric reads and fixed support-deadline, frame, and validity checks, topological scene graphs accelerate semantic target selection, and episodic logs enable task-level recovery. Storage footprint, cache update traffic, and worst-case query latency must all fit within the hardware tick budget.
- **Expiry follows the declared error model**: A spatial belief expires the moment its validated error bound $E(\Delta t)$ reaches the task tolerance r_{tol}. For continuous-time kinematic models, integrating acceleration white noise yields a variance term that grows cubically with elapsed time ($\sim \Delta t^3$). Translating this unbounded Gaussian dispersion into a probabilistic clearance margin requires calibrated coverage factors, whereas bounded disturbance models yield hard, conditional worst-case horizons.
- **Memory adds a deadline, not distance**: Free space erodes inward from unobserved boundaries at the maximum approach velocity of occluded obstacles. Consequently, memory certifies the clear corridor for 593.75 ms ahead of the platform, but offers zero usable clearance directly beside a blind corner, where the localization bound and the protective clearance consume the gap. When evidence expires, occluded voxels revert to an unknown state; only fresh line-of-sight sensor returns can renew a free-space claim.

- **Stale records look healthier than missing ones**: Stale data records frequently pass superficial health checks that outright message drops would fail: a frozen transform tree continues publishing, phantom tracks maintain low covariance, and unsynchronized consumers combine state vectors from divergent epochs. Uncompensated processing skew ($d_{\text{skew}} = v_{\text{close}} \Delta t_{\text{skew}}$) rapidly consumes physical safety margins unless every spatial input is projected to a common evaluation epoch before evaluation.
- **Unobservable error hides from innovation checks**: When sensor geometry leaves state dimensions unobservable, innovation residuals remain zero despite severe, unbounded physical drift. A misspecified filter that omits the ambiguous state can report shrinking uncertainty while the physical state diverges from its estimate. Restoring observability requires complementary sensor modalities, active persistent excitation, or external absolute anchors before the state estimate can safely authorize machine action.

The third law, proposal is not permission, requires the permission path to evaluate every proposed action against state data with an explicit, bounded age (principle 3). Spatial memory establishes this age. Elapsed time is measured from the evidence epoch t_0 of the raw physical measurement, never from the computational epoch t_{est} produced by kinematic extrapolation. Space that no valid ray has crossed has no evidence epoch at all, entering the permission pipeline as strictly unknown space. A verified age supports the safety argument only within a qualified error-growth model and feasible motion-response envelope; the permission path must reject the state at the exact deadline where that error bound breaches the task tolerance.

↪ The intent lease's expiry, which takes the belief record's expiry as its evidence term, is developed in section 10.8.

9.13 Historical Perspective and Further Reading

The state estimators at the foundation of modern spatial memory predate the mobile robots that rely on them. Rudolph Emil Kalman (1960) formulated the discrete-time recursive linear filter, establishing optimal estimation for linear systems subject to Gaussian noise. Within two years, Stanley Schmidt and his team at NASA Ames linearized the filtering equations about the running state trajectory to guide spacecraft navigation for the Apollo circumlunar mission (Smith et al. 1962), creating what is now known as the extended Kalman filter (EKF). Because Taylor-series linearizations introduce severe truncation errors when applied to highly nonlinear kinematics, Julier and Uhlmann (1997) developed the unscented transform, offering a derivative-free mechanism that propagates deterministic sigma points through the true nonlinear dynamics. In parallel, radar tracking research addressed measurement uncertainty in cluttered environments: Bar-Shalom and Tse (1975) introduced probabilistic data association (PDA) to resolve ambiguous target-to-observation assignments, while the statistical gating and consistency frameworks compiled by Bar-Shalom et al. (2001) provided rigorous methods to detect filter divergence. Control theory supplied the theoretical bridge to autonomy: Åström (1965) demonstrated that under linear-quadratic-Gaussian assumptions, a controller operating under partial observability can separate state estimation from control law computation and act directly on the conditional state distribution without loss of optimality. This separation principle justifies maintaining an explicit belief record in an embodied system.

Mobile robotics brought these recursive estimation techniques into unstructured environments. Moravec and Elfes (1985) and Elfes (1989) introduced occupancy grid mapping, projecting wide-beam sonar range returns into discrete spatial cells via probabilistic sensor models. While early robotic systems decoupled map building from self-localization, Smith et al. (1990) proved that errors in estimated robot pose and errors in stationary landmark coordinates are fundamentally correlated, formulating simultaneous localization and mapping (SLAM) as a single high-dimensional estimation problem. Over the following decades, surveyed extensively by Durrant-Whyte and Bailey (2006) and Cadena et al. (2016), SLAM matured from filtering frameworks into non-linear least-squares optimization over factor graphs (Dellaert and Kaess 2017), which decouple front-end feature tracking from back-end trajectory optimization. In parallel, volumetric surface representations migrated from computer graphics into physical AI pipelines. Curless and Levoy (1996) developed signed distance fields (SDFs) with weighted least-squares fusion to merge multiple range scans into coherent polygon meshes. With the arrival of commodity depth sensors, KinectFusion adapted this approach to run in real time on graphics hardware (Newcombe et al. 2011). Autonomous robotics then repurposed

these dense geometric structures for motion planning: OctoMap introduced hierarchically clamped octrees for compact volumetric occupancy (Hornung et al. 2013), Voxblox enabled incremental generation of Euclidean signed distance fields (ESDFs) directly from truncated signed distance fields (TSDFs) (Oleynikova et al. 2017), and nvblox ported these reconstructions to high-throughput embedded GPU architectures (Millane et al. 2024). To bridge geometric reconstruction and task-level semantic reasoning, 3D dynamic scene graphs now organize these dense metric representations into hierarchical abstractions spanning object instances, topological rooms, buildings, and moving agents (Armeni et al. 2019; Rosinol et al. 2020; Hughes et al. 2022).

Despite these mathematical and computational advances, classical mapping and estimation architectures leave one systems requirement unresolved, the explicit lifetime and epistemic boundaries of spatial information. Conventional estimators maintain state covariances, grid maps accumulate occupancy log-odds, and scene graphs categorize objects. Those ingredients must also be exposed and enforced as a consumer contract, coupling timestamped evidence, uncertainty growth under occlusion, and the temporal lease that limits each intended use. Downstream control processes, including trajectory generators, speed governors, and hardware safety monitors, require bounded guarantees on spatial validity. Automated driving systems approached this requirement by expanding reachable dynamic sets outward from the boundaries of sensor occlusion fields (Orzechowski et al. 2018). Spatial memory formalizes this coupling by binding evidence epochs, growth dynamics, and explicit validity leases directly to state estimates. By carrying its own temporal boundaries, a belief record enables hardware permission pipelines to revoke actuation authority before an expired belief causes an unsafe intervention or a physical collision.

The following foundational works provide detailed treatments of state estimation, map representations, and statistical consistency:

- Thrun et al. (2005) establishes Bayesian filtering, the Kalman and particle filter families, occupancy grid mapping, and SLAM within a unified probabilistic framework, serving as the foundational reference for recursive state estimation in section 9.2 and the volumetric map representations in this chapter.
- Bar-Shalom et al. (2001) serves as the definitive reference on target tracking and state estimation, establishing the mathematical foundations for statistical gating and the normalized estimation error squared (NEES) and normalized innovation squared (NIS) consistency tests deployed in section 9.2.1 and section 9.2.2.
- Julier and Uhlmann (2004) provides a rigorous mathematical derivation of sigma-point transformations, demonstrating why first-order Taylor series approximations cause extended Kalman filters to underestimate posterior covariance and become overconfident, which is the operational failure mode that filter consistency checks must catch.
- Cadena et al. (2016) surveys the evolution of SLAM systems as they transitioned from probabilistic filtering to factor graph optimization, detailing key open challenges in dynamic scene segmentation, semantic representations, and lifelong map maintenance summarized in table 9.3.
- Hornung et al. (2013) describes the OctoMap framework, formalizing a 3D probabilistic octree that pairs clamped log-odds updates with an explicit distinction between free and unobserved unknown space, a structural requirement for permission-grade spatial safety checks.
- Oleynikova et al. (2017) presents an incremental algorithm for constructing Euclidean signed distance fields directly from truncated signed distance fields on computationally constrained systems, providing the core representation adopted in this chapter for local metric snapshots.

What's Next: From valid beliefs to bounded goals

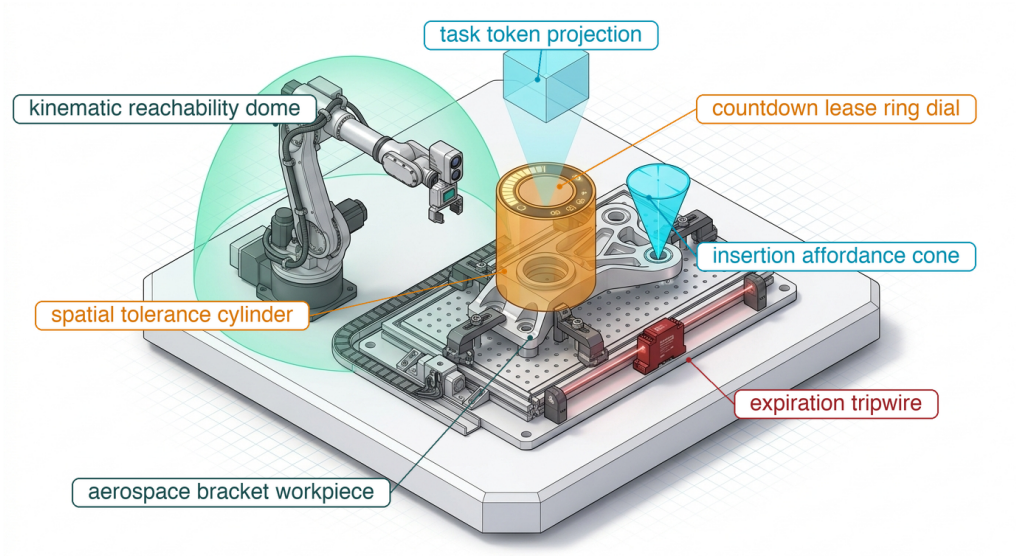
What may a valid belief authorize? Memory rules only on validity. It can say that a remembered object lies within a stated bound until a stated deadline, but it cannot say what the machine should do about that object. In the intent layer, a spoken request to take the mug from the conveyor must first be grounded to a feature the gripper can act on, the mug's handle, in the robot's own frame and units. Only then does it become an intent lease. The lease consumes the

belief record of that handle, carries that belief's evidence epoch forward as its own evidence time, and caps its expiry at that belief's horizon. It grants a bounded target with effort limits, never a path, for no longer than the evidence beneath it supports.

DRAFT

10

Grounded Intent

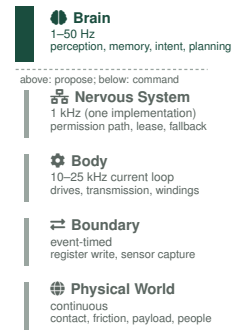


10.1 Grounded Affordances
10.2 Task Operators and Geometric Samplers
10.3 Semantic Request Translation to Geometric Proposals
10.4 The Intent Handoff and Bounded Authority
10.5 Expiring Goals and Temporal Validity Leases
10.6 Admissible Goal Sets
10.7 Intent Grounding Failure Modes and Their Checks
10.8 The Intent Lease Specification and Contract
10.9 Fallacies and Pitfalls
10.10 Summary
10.11 Historical Perspective and Further Reading

Purpose

Why is recognizing the right object insufficient to authorize a robot to reach for it?

A spoken request names an outcome, not a motion. Between the word and the grasp lie questions no label settles: which feature of the object to touch, where that feature sits in metric space, how hard to squeeze it, and how long the observation behind the answer stays true. A model can name the right object while misplacing its handle by more than the gripper tolerates, can locate it precisely a moment before a conveyor carries it away, or can report high confidence about the wrong object altogether. Once any of these answers becomes a coordinate it looks exactly like a correct one, and the drive that tracks it cannot tell the difference, so a machine that lets such a proposal command motion turns an error in meaning or timing into force against ceramic, glass, or a person. A learned target is safe to pursue only when it arrives bounded in space, effort, and time, with refusal available whenever the evidence cannot support it. Those bounds keep learned intent on the proposal side of the causal boundary, where a mistake can still be refused rather than delivered as motion that cannot be undone.



Learning Objectives

- Ground natural-language directives and symbolic task operators into metric affordance frames, continuous motion predicates, and verified geometric goal sets
- Emit the intent lease from parent belief records with explicit error envelopes, task tolerances, and effort ceilings
- Calculate intent lease expiry as the earliest of the evidence horizon, the administrative lease, and the stopping deadline
- Decide when grounding ambiguity or a wrong-object premise requires refusal rather than admission
- Evaluate terminal reachability within one evidence window, screen approach corridors against the forward occupancy of moving people, and select guarded, renewed stages for a longer approach
- Design hardware lease expiry that revokes motion authority even when the issuing process hangs or crashes

10.1 Grounded Affordances

An operator tells the warehouse mobile manipulator to "pick the red mug" from an active takeaway conveyor. The request names an outcome and leaves the grasp pose, approach velocity, contact force, and execution deadline unspecified. Spatial memory has already done its part by recording a belief about the mug's handle with an evidence epoch, an initial error of 3 mm, and a growth law. Under that law, the moving belt carries the handle outside the 15 mm grasp tolerance 60 ms after capture. Because that record says only where the handle is and how long the claim holds, it neither specifies what the machine may do with the handle nor authorizes actuator torque. **Intent** bridges this gap by binding the spoken request to an admissible, time-bounded physical goal that the motion planner can pursue.

↩ The belief record a lease consumes (evidence epoch, initial error, growth law) is established in section 9.10.

The reach does not stay inside the arm. On the running machine a slow intent model sits above a fast chunk policy that proposes base velocity and arm targets together in one whole-body stream (section 3.5.2). A wrong or stale mug target can steer the base as well as the wrist. What the permission path grants in answer to the request is therefore deliberately narrower than motion. It grants an intent lease (section 10.8), a record that names a target and effort ceilings but never a path. For the mug, the lease binds the parent belief record's digest and evidence epoch to a target pose in the handle's affordance frame, a goal region of spatial and angular tolerances around that pose, and ceilings on tool speed, acceleration, and contact wrench. Alongside those geometric and dynamic bounds, it records the motion predicates already validated, an expiration deadline on the safety controller's own clock, and the fallback that controller engages when the deadline passes.

Friction cones, force closure, and symbolic planners matter in this layer only because each supplies a field that the safety controller can check without re-running the upstream model: grasp mechanics fix the target pose, goal region, and wrench ceiling below; task operators (section 10.2) and open-vocabulary grounding (section 10.3) determine which predicates and error envelopes accompany a candidate target; and belief drift (section 10.5) paired with kinematic reachability (section 10.6) bounds the expiration deadline and dynamic limits.

This grounding step is where a learned model's output first turns into physical coordinates and contact limits that the robot can execute. Open-vocabulary object detectors, language models, and vision-language-action policies let operators specify manipulation tasks in plain words rather than through hard-coded skill libraries. What those neural networks emit, however, is only a statistical hypothesis, such as a bounding box, a semantic token, an uncalibrated point coordinate, or a chunk of normalized actions. An objective function that knows nothing of metric clearance, actuator torque limits, or physical time produced it. On the running machine, the division of responsibility is therefore strict: the learned Brain proposes candidate actions, and the deterministic permission path checks and admits them before any drive receives current.

↩ The proposal boundary and the permission path are established in section 1.7.

Before the permission path admits a proposed target for planning and execution, the target must satisfy three physical requirements:

1. *Metric physicality*: The target cannot remain a symbolic label; it must resolve to an explicit geometric feature carrying a six-degree-of-freedom pose, a gripper aperture, and a bounded contact wrench profile.
2. *Bounded error*: The proposal must include a certified spatial error envelope that fits strictly within the task tolerances and mechanical effort limits of both the manipulator and the workpiece.
3. *Temporal expiration*: Every target must carry an explicit evidence epoch, bounding its valid lifespan as target objects shift and sensory observations grow stale during compute cycles.

Every request traverses an explicit pipeline to meet these three requirements. Sensory perception resolves symbolic task operators into measurable physical predicates, and calibration transforms image regions into metric poses. The permission path then evaluates effort limits, evidence age, kinematic reachability, and dynamic obstacle clearances before committing the target to the motion planner. A vision-language-action (VLA) model (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) can propose the mug's target directly, but its deliberative inference loop is slow. The running machine's intent model executes at 2 Hz and requires 336 ms (derived in section 3.4.1) to autoregressively decode 7 target values. These values are a 3D position, a three-value orientation (which the producer converts to the quaternion the wire format carries), and an open-close gripper command. The machine therefore splits proposal across two speeds (section 3.5.2). The intent model names the target, and a smaller chunk policy streams motion toward it between intent inferences.

That split keeps the joints supplied with setpoints, but it does not keep the target true. If the conveyor moves while one intent inference runs, every chunk aims at where the mug was when the model's frame was captured. One inference outlasts the handle's 60 ms evidence window more than five times over. When target evidence expires between model inferences, the system must either switch to a high-rate local visual tracker or arrest motion entirely. Section 10.5 quantifies these expiration dynamics. Safe physical agency demands dated geometric evidence, explicit capability limits, and an autonomous permission path capable of revoking execution authority whenever the neural model lags or stalls.

Grounding the mug grasp starts with naming which physical feature the gripper will touch, expressing that feature in the robot's base frame, and bounding the forces of contact. In ecological psychology, Gibson coined the term **affordance** for what an environment offers an agent capable of acting on it (Gibson 1979).[1] Robotics turns that qualitative idea into four measurable quantities for the mug handle. The first is a target grasp pose $\mathbf{T}_{\text{grasp}} = (\mathbf{R}_{\text{grasp}}, \mathbf{p}_{\text{grasp}}) \in SE(3)$ in the robot base frame (where $SE(3)$ is the Special Euclidean group of 3D rigid-body positions and $SO(3)$ rotations).[2] The others are a unit approach vector $\hat{\mathbf{a}}_{\text{approach}}$, a jaw aperture w_{aperture} in meters, and an allowable contact wrench envelope $\mathbf{w}_{\text{contact}}$ bundling three-axis forces in newtons and three-axis torques in newton-meters. Because each usable site on an object imposes different geometry and force limits, each site defines its own **affordance frame** $\mathcal{F}_{\text{affordance}}$ attached to the object's body coordinates. A single mug offers several distinct affordance frames across its surface. Its thin rim tolerates little normal force, its cylindrical wall requires a wide jaw span, and its handle requires a full six-degree-of-freedom (6-DoF) alignment that threads the fingers through the loop without collision.

Contact mechanics dictates the allowable contact wrench. Under the classical Coulomb friction model, tangential contact forces $\mathbf{f}_t$ remain stable without slipping only when constrained within a **friction cone**:

$$\|\mathbf{f}_t\| \leq \mu f_n$$

where f_n represents the normal contact force and μ is the static coefficient of friction. For an idealized two-finger parallel gripper, Nguyen established that a grasp resists arbitrary in-plane disturbance forces if and only if the line connecting the two contact points lies strictly within the interior of both friction cones (Nguyen 1988).

The general formulation operates in wrench space. For each contact point i, a grasp map $\mathbf{G}_i$ transforms the local contact force $\mathbf{f}_i$ into an object wrench composed of the force and its moment $\mathbf{d}_i \times \mathbf{f}_i$ about the object reference frame. Here $\mathbf{d}_i$ denotes the displacement from the reference origin to the contact point. Mapping the extremal rays of every friction cone into wrench space yields a set of primitive contact wrenches. The grasp achieves **force closure** when the origin of wrench

↩ The machine's multi-rate cadences, from motor current loops through safety screening to deliberative planners, are established in section 4.3.1.

[1] **Affordance** (from the verb *to afford*): Gibson (1979) coined the noun for what an environment offers an animal able to act on it, and the sense is relative to the agent. A handle affords a grasp only to a gripper whose aperture and wrench ceiling fit it, so the affordance record carries the gripper's limits as well as the mug's geometry.

[2] **Special Euclidean Group**: "Euclidean" because its elements preserve distances, "special" because the rotation part has determinant $+1$, which excludes mirror reflections. A learned regressor carries no such constraint. An unnormalized quaternion or a matrix with determinant -1 is a valid tensor and an impossible pose, so admission rechecks quaternion normalization (section 10.8) instead of trusting the output head.

space lies strictly within the interior of the convex hull of these primitive wrenches, guaranteeing that admissible contact forces can equilibrate any external disturbance wrench. Because fingers can only push into the surface, the net wrench they apply is a nonnegative scaling of a point inside that convex hull, and enclosing the origin strictly inside the hull ensures that a ray pointing opposite to any external disturbance wrench intersects the hull interior so the fingers can scale up their normal forces to cancel disturbances from every direction. Ferrari and Canny formalized this condition into a scalar quality metric. The metric is the radius of the largest origin-centered ball inscribed within that convex hull, and it represents the minimum disturbance wrench the grasp resists per unit of applied normal force (Ferrari and Canny 1992).

Force closure, however, remains an idealized geometric property that specifies stability without bounding magnitude. A physical grasp requires finite capacity. The normal squeeze force must counter the mug's gravitational load and dynamic accelerations during transit while remaining strictly below structural limits that would fracture ceramic or spill contents. Section B.7.5 derives the lower bound on squeeze force from Newton's second law and develops friction cones and force closure from first principles. These dynamic bounds constitute the effort envelope that the Brain proposes and the permission path enforces.

Turning a spoken request into a checkable physical target is the robotic form of the **symbol grounding problem** (Harnad 1990), which asks how arbitrary symbols connect to measurable sensorimotor states.[3] For the mug, that connection runs through the calibrated transform chain from the camera frame to the manipulator base, the handle's affordance frame, and the contact wrench bounds. An uncalibrated detection box or an autoregressively sampled coordinate carries none of those bounds. The permission path must therefore catch and reject an unsupported model proposal (a hallucination, in machine learning terms[4]) deterministically, regardless of the model's reported confidence. When a spatial feature instead bundles a calibrated base-frame pose, certified wrench limits, and an evidence timestamp, it forms a grounded affordance.

[3] **Symbol Grounding Problem**: Harnad (1990) compared the problem to learning Chinese from a Chinese-to-Chinese dictionary alone, in which every definition points only to more undefined symbols. A model's text embedding is that dictionary. Only a calibrated camera-to-base chain breaks the circle for the mug, so the proposal must carry its evidence record and not only its label.

[4] **Hallucination**: In text generation the word names fluent output that the source does not support (Maynez et al. 2020). For a robot the unsupported output is a coordinate, and nothing in its format marks it as unsupported. Containment therefore has to be a check on evidence that the model does not produce.

Definition 10.1: Grounded affordance

Grounded affordance is an affordance expressed in the robot's own frame and units: a feature pose in the base frame, an approach direction, an aperture, and a contact wrench ceiling, each tied to dated sensor evidence through a calibrated measurement chain.

1. **Significance**: It turns a label into quantities the permission path can check. The mug handle's grounded affordance is a pose that must land inside a 15 mm task tolerance from evidence with a 3 mm initial error, and on the moving belt that evidence stays valid for only 60 ms (section 10.5).
2. **Distinction**: Unlike a detector box or a language label, which designates where an object appears in image space or what it is called, a grounded affordance defines where and how an end effector may establish contact. Unlike Gibson's perceptual affordance, it is bound to one gripper's limits and to one capture time.
3. **Common pitfall**: Treating a learned grasp score or a confident detection as grounding. Neither carries a metric error bound, a wrench limit, or an evidence epoch, so neither can be admitted or expired.

A high-level task action like `Grasp(Mug, Handle)` specifies an operational objective, but its preconditions require rigorous physical verification. Perceptual noise and latency prevent open-loop execution from guaranteeing that the handle remains unobstructed or that the jaws are properly aligned. **Motion predicates** formalize these physical prerequisites by mapping abstract conditions to measurable geometric tolerances and wrench readings:

1. *Pre-contact approach predicate:* $P_{\text{approach}}(\mathbf{T}_{\text{TCP}}) \iff \|\mathbf{p}_{\text{TCP}} - \mathbf{p}_{\text{pre}}\| \leq r_{\text{tol}} \wedge \arccos(\hat{\mathbf{z}}_{\text{TCP}} \cdot \hat{\mathbf{a}}_{\text{approach}}) \leq \theta_{\text{tol}}$, requiring the tool center point (TCP) frame $\mathcal{F}_{\text{TCP}}$, whose origin sits between the gripper fingertips, to align with the affordance approach axis $\hat{\mathbf{a}}_{\text{approach}}$ within spatial tolerance r_{tol} and angular orientation tolerance θ_{tol}.
2. *Guarded contact predicate:* $P_{\text{contact}}(\mathbf{w}) \iff f_{\min} \leq f_n \leq f_{\max}$, checking a normal squeeze band under the illustrated parallel-jaw model. Here the contact-force magnitude denotes the relevant

normal squeeze at each finger, measured or conservatively inferred under qualified contact geometry; a generic force-vector norm or net wrench does not establish that quantity. The task profile sets the band from friction, payload load and acceleration, and crush limits. Passing it supports retention under those premises but does not establish object identity or task completion.

3. *Spatial clearance predicate:* $P_{\text{clear}}(\mathbf{p}, \mathcal{M}) \iff \min_{\mathbf{o}\in\mathcal{M}} \|\mathbf{p}-\mathbf{o}\| \geq d_{\text{margin}}$, evaluating candidate point $\mathbf{p}$ against obstacle geometry stored in the spatial memory map $\mathcal{M}$. This point clearance test serves as a fast preliminary check. Certifying the physical gripper, manipulator links, and their swept transit volumes requires a continuous path collision query during trajectory synthesis.

Together, the spatial and angular tolerances delineate an admissible **geometric goal set** $\mathcal{G}_{\text{goal}} \subset SE(3)$:

$$\mathcal{G}_{\text{goal}} = \left\{ \mathbf{T} = (\mathbf{R}, \mathbf{p}) \in SE(3) \;\middle|\; \|\mathbf{p}-\mathbf{p}_{\text{target}}\|_{\mathbf{E}} \leq 1,\ \angle(\mathbf{R}, \mathbf{R}_{\text{target}}) \leq \theta_{\text{tol}} \right\} \tag{10.1}$$

Contact and obstacle clearance predicates operate as distinct, phase-specific execution constraints outside this static pose volume. Within equation 10.1, the weighted ellipsoidal norm $\|\mathbf{p}-\mathbf{p}_{\text{target}}\|_{\mathbf{E}} = \sqrt{\sum_{i\in\{x,y,z\}} \left(\frac{p_i - p_{\text{target},i}}{r_i}\right)^2}$ bounds spatial deviation within an ellipsoid whose principal semi-axes are $\mathbf{r} = (r_x, r_y, r_z)$. Here r_{tol} denotes the corresponding semi-axis r_i along any individual coordinate direction. The rotation error metric $\angle(\mathbf{R}, \mathbf{R}_{\text{target}}) = \arccos\left(\frac{\text{tr}(\mathbf{R}^\top \mathbf{R}_{\text{target}})-1}{2}\right)$ bounds geodesic orientation divergence on $SO(3)$, equivalent to $2\arccos(|\mathbf{q}_{\text{quat}} \cdot \mathbf{q}_{\text{quat,target}}|)$ when unit quaternions represent orientations. Terminal grasp poses falling outside $\mathcal{G}_{\text{goal}}$ lead directly to task aborts or damage. Translational errors exceeding r_i cause gripper fingers to impact the ceramic rim or close on empty space, while angular misalignments exceeding θ_{tol} jam the fingers against the handle opening, tripping joint torque protection or shattering the object.

For the warehouse mug, the nominal task profile assigns all three ellipsoidal semi-axes to the 15 mm grasp tolerance. A terminal TCP offset of 10 mm along the x-axis yields a normalized ellipsoidal distance of 0.67, remaining well within the admissible goal set. Conversely, an offset of 20 mm yields a normalized distance of 1.33, falling outside the set and triggering immediate rejection. Checking angular alignment requires the angular tolerance θ_{tol} specified in the verified task profile, which the mug model registers during commissioning.

This goal set is an operational task specification rather than an estimate of target pose. The **task tolerance region** defines the maximum terminal error that the physical grasp can absorb; state estimator covariance captures uncertainty in the target's actual pose. For the mug pick, the approved task profile permits at most 15 mm of translational discrepancy. The proposed affordance must therefore supply a conservative target error bound that fits inside this envelope, leaving sufficient headroom for the motion controller's dynamic tracking error. Artificially expanding the task tolerance to accept an uncertain perception estimate corrupts the safety invariants of the physical task.

Generating candidate affordances prior to intent proposal packaging typically relies on a **learned 6-DoF grasp generator** applied to an incoming point cloud. Architectures such as Contact-GraspNet predict grasp poses anchored directly to surface points in cluttered point clouds (Sundermeyer et al. 2021). Frameworks like AnyGrasp predict dense $SE(3)$ grasp distributions with learned quality scores and track them across temporal observation sequences (Fang et al. 2023). The generator outputs a set of candidate grasp configurations paired with scalar confidence scores. However, a learned score merely reflects the network's predictive confidence relative to its training distribution. It provides neither an analytic stability metric like the Ferrari-Canny radius nor a measured empirical grasp success rate for this specific gripper and workpiece geometry.

Candidate poses surviving initial filtering must satisfy rigorous downstream physical constraints that learned generators omit:

1. *Kinematic feasibility*: The manipulator must possess a valid inverse kinematics (IK) solution that avoids joint position and velocity limits. A well-conditioned kinematic Jacobian does not guarantee clearance from mechanical joint stops, making explicit joint travel verification mandatory.

2. *Geometric compatibility*: Gripper jaw stroke must exceed workpiece thickness at the grasp site with sufficient clearance margin.
3. *Corridor clearance*: The approach trajectory must provide an unobstructed approach vector clear of surrounding bin walls and adjacent cargo.

When no candidate grasp satisfies these constraints, or when target identification remains unresolved across ambiguous visual candidates, the control pipeline must not execute an argmax guess. Instead, the proposal's producer on the application processor issues an explicit **intent grounding refusal** (`ERR_GROUNDING_AMBIGUITY`), signaling a typed rejection of the goal itself. A planning failure means that the planner has not produced a collision-free path to a valid, well-formed goal. An intent grounding refusal halts execution earlier. It rejects an ungrounded or ambiguous goal before invoking the motion planner, which prevents unverified guesses from generating physical trajectories. When a symbolic task planner sits above the grasp generator, its discrete task operators must discharge these same geometric and contact predicates before any step of the plan is admitted.

10.2 Task Operators and Geometric Samplers

A symbolic task planner can emit the discrete action `Grasp(Mug, Handle)` and mark the mug as held in its internal state even when no collision-free inverse kinematics solution reaches the handle, a bin wall blocks the approach, or the mug has not been separated from adjacent items on the belt. The high-level action `Grasp(Mug, Handle)` inherits from classical planning traditions, where actions are formalized as **operators** defined by preconditions, an add list, and a delete list. The formalism traces back to the STRIPS[5] framework (Fikes and Nilsson 1971) and was standardized by the Planning Domain Definition Language (PDDL) (McDermott et al. 1998). A discrete world state s consists of a set of grounded predicates. An operator o is applicable in state s if its preconditions hold, $\text{pre}(o) \subseteq s$, producing the successor state

$$s' = (s \setminus \text{del}(o)) \cup \text{add}(o) \tag{10.2}$$

A symbolic planner searches sequences of operators, advancing states by the delete-then-add update of equation 10.2 until reaching a state satisfying the target goal predicates. For the warehouse pick, an operator `pick(mug, handle, conveyor)` requires preconditions `Reachable(handle)`, `Clear(approach)`, `Singulated(window)`, and `HandEmpty`, where a singulated window holds exactly one separated candidate object. Upon execution, it adds `Holding(mug)` and deletes `HandEmpty`.

Abstract symbols disguise the primary computational challenge: verifying whether a predicate like `Reachable(handle)` holds requires finding a continuous end-effector pose, a valid inverse kinematics solution, and a collision-free joint trajectory. A symbolic planner cannot evaluate these continuous constraints natively. **Integrated task and motion planning** (TAMP) resolves this disconnect by coupling discrete symbolic search with **geometric samplers**, computational routines that sample continuous configuration spaces (proposing candidate grasps, inverse kinematics configurations, and transit paths) to certify symbolic preconditions (Kaelbling and Lozano-Pérez 2011; Garrett et al. 2021). For instance, PDDLStream treats each geometric sampler as a generator stream whose continuous outputs certify symbolic facts. The task planner can then make optimistic symbolic progress, backtracking or branching when a sampler fails to find a valid configuration (Garrett et al. 2020). A sampler failure means that the search has not found a collision-free grasp for the given base pose; unless the sampler certifies infeasibility, it does not prove that none exists. The symbolic planner can spend more search effort, reposition the mobile base, or select an alternate interaction sequence.

Large language models (LLMs) increasingly generate the symbolic layer. Systems such as LLM+P prompt an LLM to translate natural-language task specifications into formal PDDL domain and problem instances, subsequently delegating resolution to a classical symbolic planner (B. Liu et al. 2023). While this decouples linguistic parsing from plan generation, neither the language model nor the symbolic planner verifies geometric feasibility or sensorimotor validity against real-world scene observations. Bridging this gap requires mapping each symbolic precondition to a verifiable motion predicate or geometric certificate evaluated against live physical measurements, as summarized in table 10.1.

[5] **STRIPS (Stanford Research Institute Problem Solver)**: The formalism was built to plan for Shakey, a physical mobile robot at SRI (Nilsson 1984), so symbolic planning began on a machine that moved. Its operators treat an action's listed effects as certain, an assumption that slipping wheels and a misreading camera cannot grant. The gap that geometric samplers and motion predicates close is therefore as old as the formalism itself.

Table 10.1: From task operator to motion predicate: Each fact the symbolic `pick` operator requires or predicts for the mug maps directly to a predicate over measured physical quantities and to an explicit verification check in the execution pipeline.

Operator fact	Motion predicate or certificate	Check that discharges it	Where it is checked
`Reachable(handle)`	$\mathbf{T}_{\text{grasp}} \in \mathcal{W}$ with an IK solution inside joint limits	Static workspace check and transit lower bound	Admission (section 10.6)
`Clear(approach)`	P_{clear} along the approach corridor	Occupancy and forward occupancy screen, then swept-volume check	Admission, then planner
`Singulated(window)`	One candidate cluster in the pick window	Independent count from fresh depth evidence	Station rule (section 10.7)
`At(gripper, pregrasp)`	P_{approach} within r_{tol} and θ_{tol}	Goal-set membership (equation 10.1)	Planner, then permission path
`Holding(mug)` (effect)	P_{contact}: $f_{\min} \le f_n \le f_{\max}$	Per-contact normal squeeze after closure; required for retention	Runtime check

In this execution architecture, the predicted effect `Holding(mug)` remains a conditional hypothesis until the declared terminal pose, contact, and retention checks pass following finger closure. The guarded contact predicate P_{contact} supplies the normal squeeze check under the task profile; it cannot alone confirm that the intended mug is held. A symbolic task plan operates as an intent proposal in the same manner as a neural VLA target. It asserts the preconditions and effects it intends to achieve, while the deterministic permission path verifies whether calibrated physical evidence supports execution.

Grounding an open-ended directive therefore yields a structured, time-stamped **intent proposal**. The immediate systems design challenge is converting an identified image region into a calibrated metric target without corrupting its semantic identity, understating its measurement uncertainty, or obscuring its evidence epoch. Once packaged with these invariants, the proposal enters the independent permission path to verify physical admissibility, effort bounds, and expiration limits before execution begins.

10.3 Semantic Request Translation to Geometric Proposals

When an operator asks for the red mug and the vision backbone instead binds the word "mug" to an adjacent table fixture, the coordinate that crosses the proposal boundary arrives in the trajectory pipeline indistinguishable from a valid handle target. Low-level motor drives monitor encoder counts, winding temperatures, and phase currents, so they have no way to tell whether the setpoint they track belongs to a mug or a bolted bracket. Any unmodeled arm compliance or thermal expansion also adds metric bias that a high semantic confidence score never reveals. Before any motion planner accepts a target, the grounding pipeline must therefore turn the model's selection into a geometric proposal that carries bounded spatial uncertainty, dated selection evidence, and an explicit expiration time.

10.3.1 The four-stage open-vocabulary grounding pipeline

Passing a detector's 2D bounding box or an uncalibrated pixel coordinate straight to a motion planner hides two failures at once. Two identical mugs can both score high without triggering an ambiguity warning, and a small depth error along the camera ray can stretch into centimeters of pose error in the robot base frame. To expose each failure where it can be checked, the **four-stage open-vocabulary grounding pipeline** separates semantic referent selection from metric localization and physical actuation:

1. **Stage 1: Visual encoding**: RGB-D frames from static or eye-in-hand cameras enter a vision backbone running at batch size one on the machine's accelerator, transforming raw image patches into dense feature embeddings.
2. **Stage 2: Referent selection**: Given a prompt such as "pick the red ceramic mug," an open-vocabulary detector such as Grounding DINO or OWL-ViT proposes candidate bounding boxes and scores each against the text prompt (S. Liu et al. 2023; Minderer et al. 2022). The model evaluates each candidate independently without global normalization across boxes

(OWL-ViT trains with a sigmoid focal loss (Minderer et al. 2022), while Grounding DINO filters tokens against an absolute threshold). Two identical mugs can therefore both yield high confidence, or a lone glass jar can score above threshold even when no mug is present. A promptable segmentation model such as SAM 2 can refine the chosen box into an object mask and track it across video frames (Ravi et al. 2024). However, a 2D bounding box or segmentation mask provides only image-space localization without metric depth, surface normals, or wrench limits. It indicates where the object projects onto the camera sensor, leaving contact dynamics and grasp mechanics completely unspecified.

3. **Stage 3: Metric lifting**: Calibrated camera intrinsics, depth measurements, and the hand-eye extrinsic transformation lift the selected pixels into 3D points in the robot base frame, while propagating sensor noise into a base-frame covariance matrix. For stereo and structured-light systems, depth error grows quadratically with range, $\sigma_z \approx Z_c^2 \sigma_d/(bf)$ for baseline b (the stereo term of equation 8.6), focal length f, and disparity error σ_d, whereas lateral error grows only linearly, $\sigma_u Z_c/f$. Their ratio, approximately $(Z_c/b)(\sigma_d/\sigma_u)$, scales with range and baseline rather than remaining constant, while time-of-flight sensors have their own range-dependent error profiles. The resulting uncertainty ellipsoid elongates dramatically along the optical line of sight. Any unmodeled extrinsic error from wrist compliance or mounting deflection tilts this elongated ellipsoid directly into the robot's clearance margin.
4. **Stage 4: Affordance parameterization**: The lifted 3D geometry transforms into candidate actions. Depending on the architecture, these candidates may take the form of scored 6-DoF grasp sets from a learned grasp generator, 3D attraction and repulsion cost fields (Huang et al. 2023), parameterized motion primitives (Liang et al. 2023), or decoded action tokens (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023). Stage 4 filters these candidates against the kinematic reachability, gripper aperture, and collision clearance criteria detailed in section 10.1. It then constructs a goal tolerance set $\mathcal{G}_{\text{goal}}$ around each surviving affordance frame and assigns the required motion predicates. Finally, it serializes these hypotheses into a fixed-size, statically verified proposal structure (table 10.2) and carries them across the physical silicon boundary between the non-real-time Linux Application Processor (AP) and the hard real-time Safety MCU.

↪ The pinhole camera model and the propagation of pixel and depth noise into a base-frame covariance are developed in section C.12.

↪ **Byte layout:** The intent payload's fields, types, and offsets are given in section C.15.2.

Field	Representation	Physical semantics	Invariant and admission check

Table 10.2: Untrusted intent proposal fields: Layout and validation invariants of the candidate proposal transmitted from the Application Processor across the proposal boundary.

Field	Representation	Physical semantics	Invariant and admission check
`evidence_epoch_ns`	`uint64_t` (ns)	Sensor capture timestamp t_0 from parent belief record	Must match retained belief epoch; reject if stale or unrecorded
`affordance_frame_id`	`uint32_t` (enum)	Target coordinate frame $\mathcal{F}_{\text{affordance}}$ (for example, $\mathcal{F}_{\text{handle}}$)	Must resolve to an approved kinematic frame on the target object
`predicate_mask`	`uint32_t` (bitmask)	Requested motion predicates ($P_{\text{approach}}, P_{\text{contact}}, P_{\text{clear}}$)	Verified against task requirements before planner invocation
`target_pose_se3`	`double[7]`	Proposed pose $[x, p_y, z, q_x, q_y, q_z, q_w]$ in robot base frame	Finite values; unit quaternion ($\|\mathbf{q}_{\text{quat}}\| = 1$); within $\mathcal{W}$
`pos_error_envelope_m`	`double[3]` (m)	Bounded spatial uncertainty semi-axes (e_x, e_y, e_z)	Conservative perception bounds; $0 \leq e_i \leq r_i$ is a preliminary screen; full error plus tracking reserve must fit $\mathcal{G}_{\text{goal}}$
`task_tolerance_m`	`double[3]` (m)	Allowable goal region semi-axes (r_x, r_y, r_z)	Must meet task profile tolerances; defines $\mathcal{G}_{\text{goal}}$
`max_linear_vel_mps`	`double` (m/s)	Requested tool center point speed ceiling $v_{\max}$	Clamped to plant ceiling; verified against dynamic transit time
`max_wrench`	`double[6]` (N, N·m)	Maximum effort ceiling $[F_x, F_y, F_z, \tau_x, \tau_y, \tau_z]$	Must not exceed structural yield or sensor saturation limits
`lease_duration_ms`	`uint32_t` (ms)	Requested administrative lease T_{admin}	Clamped to evidence horizon τ_{ev} and stopping deadline
`fallback_policy_id`	`uint32_t` (enum)	Advisory terminal mode (`HOLD`, `STOP`, `TORQUE_OFF`)	Advisory only; Safety MCU overrides based on load and contact state

To prevent a learned output from masquerading as a deterministic setpoint, the producer must link the proposed referent to the dated sensor evidence that justified its selection. The upstream observation contract often carries bounding boxes, attention weights, class probabilities, and a full oriented covariance matrix, which are too bulky and variable to send across a real-time link. Instead, the intent proposal references its parent belief record by a SHA-256 digest (which traces data lineage rather than authenticating the sender) and an evidence epoch. Because the digest, a monotonically increasing sequence number, and a trailing CRC-32 checksum belong to the common wire header (section C.15.1), table 10.2 lists only the intent payload. The payload holds the candidate pose, conservative base-frame error bounds, task tolerances, and a requested administrative lease T_{admin} that caps how long the host may hold the target before the MCU presumes a stall. That duration is only a request, because the MCU clamps it to the evidence horizon and the stopping deadline derived in section 10.5 and section 10.8. Since an object on the belt can shift, roll, or become occluded moments after camera exposure, the permission path immediately rejects any target coordinate that arrives without an active evidence link and a bounded deadline.

When a vision-language model evaluates multiple candidate objects, retaining only the highest-scoring bounding box conceals an unresolved selection. In purely digital applications, a misranked recommendation misleads a user but moves no mass. In robotics, an admitted spatial proposal immediately commands physical momentum. For instance, if a red glass jar sits beside the requested red mug on a conveyor, an uncalibrated tie-break may inadvertently target the jar. This case, the jar beside the mug, returns later in the chapter. The architecture must preserve candidate identity and calibrated uncertainty, rejecting proposals whenever ambiguity exceeds task-specific safety thresholds. The host system can then solicit operator clarification or acquire additional sensor viewpoints, while any previously admitted motion executes under strict lease and fallback guarantees.

Formally, **open-vocabulary grounding ambiguity** occurs when perceptual evidence supports multiple candidate referents, making the selection of any single target an arbitrary tie-break. Because open-vocabulary detectors score candidate bounding boxes independently rather than generating a normalized categorical distribution across objects, they do not natively report this condition. The host application must therefore construct a distribution explicitly. By collecting all candidate boxes exceeding an initial detection threshold and normalizing their raw scores into a vector $\mathbf{c} = (c_1, \ldots, c_n)$ where $\sum_i c_i = 1$, the system can evaluate candidate spread. Although the Shannon entropy $H(\mathbf{c}) = -\sum_i c_i \log c_i$ quantifies distributional dispersion, an arbitrary entropy threshold provides no rigorous statistical error bound.

Conformal prediction removes this limitation by replacing the single best answer with a set of candidate answers whose coverage holds in finite samples (Vovk et al. 2005; Angelopoulos and Bates 2023). A threshold calibrated on held-out labeled cases keeps every candidate it cannot exclude, so the set contains the true answer at a chosen rate across cases without assuming any model of how the scores are distributed. For example, the KnowNo framework applies conformal prediction to identify when a robot should halt and request human clarification (Ren et al. 2023). On a calibration dataset of n_{cal} labeled scenes, the nonconformity score for scene j is $s_j = 1 - c_{y_j}$, representing one minus the normalized score assigned to the ground-truth referent y_j. This construction assumes that the true label belongs to the defined candidate option space in calibration and deployment. An absent or filtered-out true object violates that premise unless the option space explicitly includes a no-valid-referent option calibrated in the same way. The critical threshold $\hat{q}_{1-\epsilon}$ corresponds to the $\lceil (n_{\text{cal}}+1)(1-\epsilon) \rceil / n_{\text{cal}}$ empirical quantile of these nonconformity scores. At runtime, the prediction set $\mathcal{C}$ retains every candidate whose normalized score cannot be excluded at significance level ϵ:

$$\mathcal{C} = \{\, i : c_i \geq 1 - \hat{q}_{1-\epsilon} \,\} \tag{10.3}$$

Provided that calibration and deployment scenes satisfy exchangeability,[6] the set $\mathcal{C}$ is mathematically guaranteed to cover the true referent with probability at least $1-\epsilon$. Under exchangeability, the nonconformity score of a new test scene is equally likely to occupy any of the $n_{\text{cal}}+1$ ranks relative to the calibration samples. It therefore exceeds the $\lceil (n_{\text{cal}}+1)(1-\epsilon) \rceil$-th smallest calibration score with probability at most ϵ. This is marginal true-label coverage across scenes, not the same error bound conditional on singleton acceptance or a guarantee of task success. If the resulting prediction set is a singleton ($|\mathcal{C}| = 1$), the pipeline passes its candidate to downstream metric lifting and the remaining admission checks. If $|\mathcal{C}| > 1$, the producer refuses with `ERR_GROUNDING_AMBIGUITY`, prompting the system to request clarification from an operator or maneuver for an unoccluded viewpoint before transmitting any motion proposal across the boundary. An empty set ($|\mathcal{C}| = 0$) means that no retained option meets the threshold, and the producer refuses it as well. It does not reliably detect an absent mug, because a lone retained wrong jar normalizes to one and survives this score test.

When both the mug and the glass jar appear simultaneously within the pick window, an open-vocabulary detector assigns raw box scores of 0.62 and 0.48 respectively. Normalizing these values yields normalized candidate scores of 0.56 and 0.44, producing a Shannon entropy of 0.685 nats relative to the two-candidate maximum of 0.693 nats. The entropy shows high dispersion across the two candidates. It does not measure the probability that either is the correct object. Conformal calibration instead supplies marginal true-label coverage under the declared option-space and exchangeability premises.

Across a calibration set of 200 labeled scenes with an allowable error rate $\epsilon = 0.10$, the rank formula selects which calibration score becomes the threshold $\hat{q}_{1-\epsilon}$, the nonconformity score at rank 181 in ascending order. Its value depends on the measured scenes. Assuming that score is $\hat{q}_{1-\epsilon} = 0.60$, any candidate must achieve a normalized score of at least 0.40 to remain in the prediction set. Because both objects exceed this threshold, the resulting prediction set retains two elements ($|\mathcal{C}| = 2$). The producer correctly flags an ambiguity error and refuses to emit a trajectory. Conversely, if the jar scores only 0.10, its normalized value drops to 0.14, falling below the admission threshold. In that contrast case, the prediction set collapses to a singleton ($|\mathcal{C}| = 1$), permitting the mug proposal to proceed.

Because conformal coverage guarantees hold marginally across scenes and degrade under distribution shift, production workcells enforce defense-in-depth through deterministic physical invariants. The station therefore adds its own singulation check, a candidate count on fresh depth evidence that

[6] **Exchangeability**: A sequence of random variables is exchangeable when its joint probability distribution remains invariant under arbitrary permutations of sample order. Exchangeability is strictly weaker than assuming independent and identically distributed (i.i.d.) observations, yet it suffices for the $1-\epsilon$ marginal coverage guarantee. Modifying station lighting or altering the conveyor layout introduces distribution shift that invalidates exchangeability, causing coverage guarantees to lapse silently. Calibration records must therefore be maintained on a per-station basis with mandatory expiration horizons, matching the lifecycle of target sensor evidence.

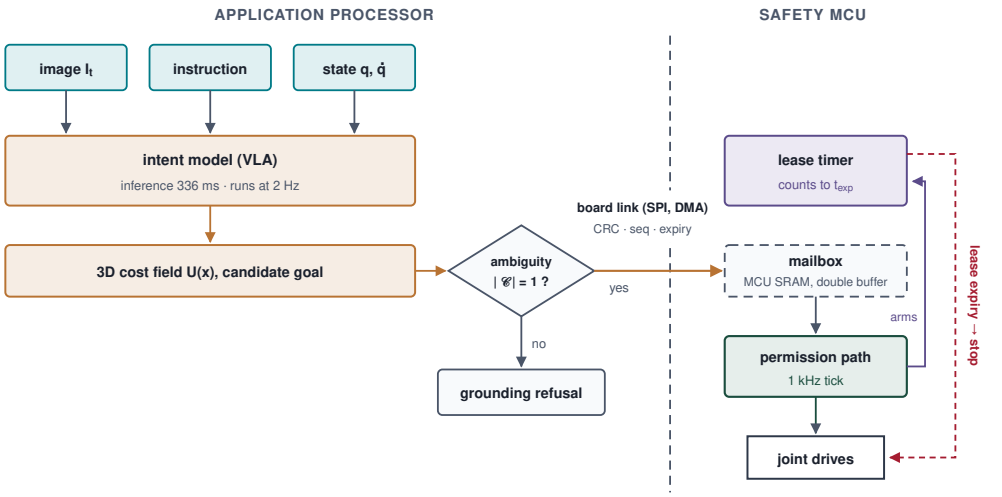


Figure 10.1: Grounded intent pipeline architecture across the silicon boundary: Physical AI system architecture partitioning high-capacity deliberative reasoning on the Application Processor (AP) from deterministic safety enforcement on the real-time Safety Microcontroller Unit (MCU). The AP processes multimodal streams (RGB-D vision, natural language prompts, proprioception) through the intent model, whose computed 336 ms inference sets its 2 Hz rate. The 3D cost field box stands for an optional producer family of the VoxPoser kind; the running machine's intent model emits a target pose directly. A producer-side ambiguity filter, which passes only a singleton calibrated prediction set ($|\mathcal{C}| = 1$), rejects ambiguous hypotheses before sending a structured intent proposal over the board-level link into the MCU's own memory; the two processors share no memory. The MCU independently verifies evidence epochs, physical actuator ceilings, and braking stopping reserve before arming the lease timer, whose expiry stops the drives without any message from the application processor; the permission path runs at 1 kHz, and trajectory admission remains a separate check (synthesized from Huang et al. (2023), Ahn et al. (2022), and Gray (1989)).

DRAFT

runs apart from the grounding model and must find a unique candidate within the pick envelope (section 10.7). This operational gate prevents arbitrary tie-breaking at the station. When multiple objects populate the workspace, downstream motion planners proceed only when perceptual evidence unambiguously discriminates the requested target.

The pipeline therefore spans two processors that share no memory (figure 10.1). Everything that can be wrong about meaning runs on the application processor, and everything that must hold regardless of meaning runs on the Safety MCU. Its 1 kHz permission loop arms a lease timer that drops authority without any message from the host. That separation buys independence only when memory access, bus timing, and fault containment are qualified in each of three domains:

- *Application Processor (AP Domain):* A general-purpose, non-real-time operating system executing on host CPUs and hardware accelerators runs the vision-language backbones and, in architectures that use them, 3D cost fields. If the conformal prediction set over candidate referents contains multiple entries (equation 10.3), an upstream filter halts execution with an ambiguity alert. When the prediction set yields a unique referent ($|\mathcal{C}| = 1$), the AP serializes the candidate goal pose, requested wrench limits, and parent belief digest into a fixed-size, statically structured intent proposal.
- *Board-Level Transport Boundary:* The proposal packet crosses from the AP to the discrete safety MCU over an isolated board-level interconnect, such as a Serial Peripheral Interface (SPI) bus backed by Direct Memory Access (DMA). The MCU receiver copies the transmission into a dedicated staging mailbox within local SRAM. The communication channel is treated as untrusted. Embedded CRC-32 checksums, monotonically increasing sequence numbers, and strict hardware timeout limits detect frame corruption, packet loss, and transmission stalls. The protocol design must also account for worst-case bus latency, as the bus interface alone guarantees no real-time delivery bounds.
- *Safety Microcontroller Unit (MCU Domain):* Operating on a dedicated real-time core, a deterministic control loop evaluates the incoming proposal as an unverified request. The MCU checks

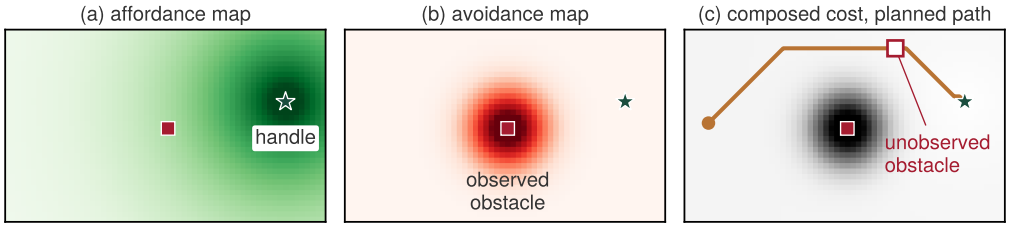


Figure 10.2: Composing value maps for a grasp request: An original schematic of the value-map interface introduced by VoxPoser (Huang et al. 2023), drawn on one horizontal slice of a voxel grid. (a) An affordance map rewards voxels near the handle the instruction names. (b) An avoidance map penalizes voxels near an observed obstacle. (c) The planner searches the composed cost for a path. The path is good only with respect to the maps: an obstacle the observations missed adds nothing to the avoidance map, so the path can run through it.

the evidence timestamp against its internal monotonic clock, verifies workspace envelopes and joint reachability limits, and clamps requested forces and velocities to certified plant limits. Furthermore, the MCU verifies that a deceleration trajectory can execute safely within existing clearance margins. The MCU arms a hardware countdown timer to the lease expiry $t_{\text{intent_exp}}$ only after the proposal passes all admission gates. The expiry is the earliest of evidence decay, host silence, and the stopping deadline (section 10.8 formalizes it). The MCU then grants bounded tracking authority to the joint drives.

10.3.2 What each proposal family leaves to admission

This three-domain boundary stays fixed even when the upstream grounding model is swapped for a different proposal family. What changes from one model family to the next is which physical checks remain for admission to supply. An upstream model may emit a 2D bounding box, a 3D cost field, a scored 6-DoF grasp, or a chunk of normalized action tokens. In every case, treating that output directly as a motion command skips the physical checks that prevent an unseen obstacle collision, a joint-limit hit, or an unbraked contact impact. The pipeline's stages draw on multimodal feature extraction, cross-attention alignment, and open-vocabulary phrase grounding from computer vision and natural language processing (Kamath et al. 2021; Gu et al. 2022; Radford et al. 2021; S. Liu et al. 2023), yet an embodied machine cannot rely on a model's internal activations for physical safety. For example, a semantic skill ranker like SayCan outputs a discrete skill identifier scored by language affordance and value functions (Ahn et al. 2022). A programmatic generator like Code as Policies produces a control script that calls perception and motion APIs (Liang et al. 2023). VoxPoser synthesizes 3D spatial value maps (figure 10.2). OK-Robot emits a distribution of sampled grasp poses (figure 10.3). Because none of these outputs natively carries a bounded wrench envelope, a certified clearance margin, or a hardware lease expiration, choosing one model family over another only shifts where conversion happens and leaves the admission checks in table 10.3 unchanged.

Predicting a syntactically valid coordinate or control script never guarantees physical feasibility. Vision-language and action models are trained to maximize sequence likelihood over offline demonstration datasets. Their loss functions encode neither metric distances, rigid-body collision avoidance, manipulator Jacobian conditioning, nor frictional contact dynamics. Validating a candidate action requires deterministic physical checks evaluated within explicit coordinate frames. These checks include signed-distance queries over robot and environment meshes to verify clearance, numerical inverse-kinematics solvers with joint-limit and singularity screening, and Coulomb friction models to confirm that the planned grasp wrench resists expected gravitational and inertial loads. For contact-rich tasks such as sliding an object, turning a latch, or executing a handover, the system can simulate candidate trajectories in a rigid-body physics engine before committing actuator current. However, simulation fidelity remains bounded by the accuracy of estimated surface friction and inertial parameters. Downstream controllers must still enforce real-time admission checks regardless of simulation outcomes.

In VoxPoser's value-map interface (figure 10.2), an affordance map rewards voxels near the handle the instruction names, an avoidance map penalizes voxels near observed obstacles, and the planner

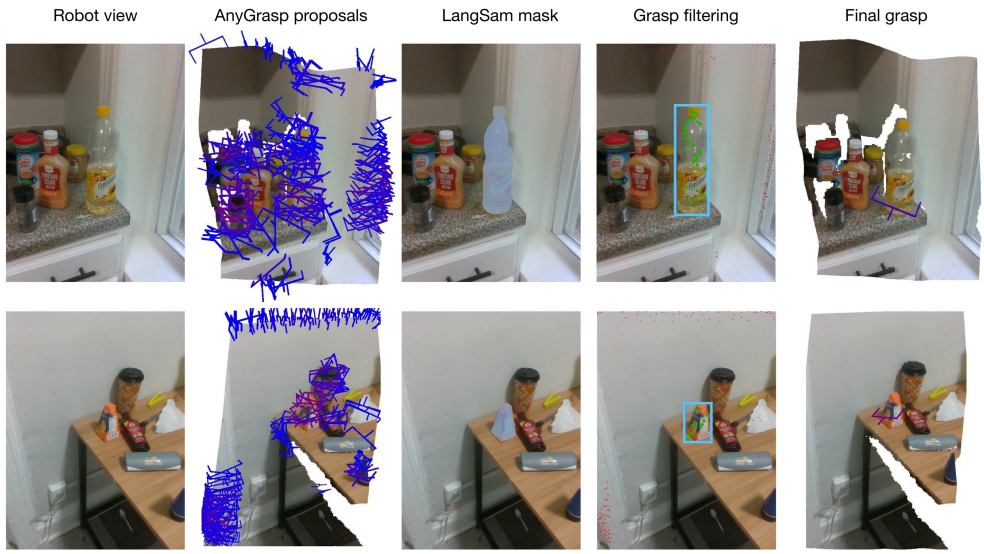


Figure 10.3: Open-vocabulary grasp grounding pipeline: OK-Robot combines candidate grasp generation with language-conditioned object selection and geometric filtering (P. Liu et al. 2024). An open-vocabulary detector isolates the target object mask, candidate grasps are sampled across the 3D point cloud, and geometric heuristics prune inadmissible approaches. The resulting grasp pose represents a candidate proposal; bounding tolerances, evidence deadlines, and independent permission checks are an architectural wrapper developed in this chapter. (Source: P. Liu et al. (2024), CC BY 4.0).

follows the composed cost around the obstacle it can see. Because the planned path is optimal only with respect to what the maps contain, an obstacle that the observations missed adds no cost and the path runs straight through it, leaving clearance against unobserved space to the permission path.

OK-Robot shows the same division in a modular pipeline (figure 10.3) (P. Liu et al. 2024). Open-vocabulary detectors (OWL-ViT and SAM) isolate the queried object, a class-agnostic generator (AnyGrasp) samples dense 6-DoF candidates over the point cloud, and geometric heuristics prune them to a single pose. That pose is still an uncertified Stage 4 proposal. Before it can command joint motion, it must be bound to dated sensor evidence, given a wrench envelope, and admitted under an expiring lease on the Safety MCU. Every proposal family in table 10.3 leaves a comparable residue, which the table's last column itemizes as the evidence admission must still supply.

Table 10.3: Grounding representations and admission questions: Outputs from different model families require different conversions before a physical permission check.

Proposal source	Representation sent downstream	Early check	Remaining evidence needed
2D box or mask	Image region plus measured depth and camera calibration	Reject absent depth or a target outside a validated region	3D pose error, occlusion, object identity, and swept-path clearance
3D feature or value field	Candidate target or cost field in a stated frame	Reject out-of-workspace targets and stale map cells	Conservative occupancy, unknown-space policy, actuator and timing bounds
Action VLA	Decoded action or action chunk in its trained range	Reject out-of-range or inadmissible proposals	State-dependent physical limit and independent permission; the action head provides no clearance proof
Learned grasp generator	Set of 6-DoF grasp poses with learned scores	Reject grasps outside the workspace or jaw aperture	Measured success on this gripper, IK and joint limits, approach clearance, and wrench capacity
Language-selected skill	Skill identifier with estimated success	Rank or decline low-confidence candidates	Current preconditions, path feasibility, and separate physical enforcement

Regardless of the specific output architecture, raw actions generated by VLA models carry no certified metric bounds. These heads normalize action spaces relative to empirical training dis-

tributions rather than physical workspace obstacles. For example, RT-1 and OpenVLA discretize each continuous action dimension into 256 uniform bins across bounded ranges (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Kim et al. 2024). FAST tokenizes the discrete cosine transform coefficients of a multi-step trajectory chunk (Pertsch et al. 2025). In contrast, π_0 generates continuous action chunks through a flow-matching diffusion expert, predicting 50 steps per inference and streaming them at up to 50 Hz (Black et al. 2024). Although each approach outputs kinematically coherent trajectories within its training distribution, none verifies dynamic feasibility or collision clearance against surrounding obstacles. Similarly, spatial location tokens in vision-language models like PaliGemma quantize normalized pixel coordinates on the camera sensor. These image tokens transform into metric poses only after incorporating measured depth maps, camera intrinsics, and hand-eye calibration matrices. The geometric uncertainty propagated through this calibration chain, particularly along the camera's optical axis, often dwarfs the discretization error of the underlying token grid.

Table 10.3 sorts proposals by the representation they send downstream, a classification that outlasts any one model. The systems that fill each role change within a few years, so table 10.4 dates them separately, pairing what each current system emits with what the permission path must still establish on its own.

Table 10.4: State of practice for grounded intent, as of 2026: Representative published systems for each pipeline role (Minderer et al. 2022; S. Liu et al. 2023; Ravi et al. 2024; Sundermeyer et al. 2021; Fang et al. 2023; B. Liu et al. 2023; Ren et al. 2023; Kim et al. 2024; Black et al. 2024; Pertsch et al. 2025). The required validation checks in the rightmost column remain architectural invariants even as specific model architectures are refreshed.

Role in the pipeline	Representative systems (year)	What the system emits	What the permission path must still establish
Open-vocabulary detection	OWL-ViT (2022), Grounding DINO (2023)	Boxes with independent per-box text scores	Uniqueness, depth, and object identity
Promptable segmentation and tracking	SAM 2 (2024)	Object masks propagated through video	The tracker's own error and latency bounds
Learned grasp generation	Contact-GraspNet (2021), AnyGrasp (2023)	Scored 6-DoF grasp sets	IK, aperture, clearance, and wrench capacity
Language to symbolic plan	LLM+P (2023)	A PDDL problem solved by a classical planner	Every geometric precondition
Calibrated request for help	KnowNo (2023)	A conformal prediction set over options	Coverage under distribution shift; physical singulation
VLA action heads	OpenVLA (2024), π_0 (2024), FAST (2025)	Binned tokens, flow-matched chunks, or compressed tokens	Clearance, timing, and effort bounds

Every row leaves admission the same two questions, whether the metric estimate is tight enough for the task and whether it belongs to the right object. The mug's handle shows how each can fail while the other holds.

10.3.3 Imprecise estimates and wrong objects

When the wrist camera views the red mug from across the workstation before the arm begins its reach, depth uncertainty along the optical ray stretches the 95 percent error region to more than twice the handle's grasp tolerance. Closing the gripper on that single-frame estimate can therefore drive the fingers into the ceramic rim. Under an illustrative calibrated Gaussian sensor model for that pre-approach wrist-camera frame, the principal standard deviations in the camera frame are 4 mm and 5 mm laterally, and 12 mm along the optical depth axis. Because the optical axis aligns with the gripper's approach vector, range uncertainty directly challenges position control. A **95 percent statistical region** surrounding the estimated handle coordinate $\mathbf{p}_0$ is defined by the Mahalanobis squared distance $(\mathbf{p}-\mathbf{p}_0)^\top \Sigma_{\text{depth}}^{-1}(\mathbf{p}-\mathbf{p}_0) \le \chi^2_{3,0.95}$. With the three-degree-of-freedom chi-square quantile $\chi^2_{3,0.95} = 7.815$, the distribution expands into principal semi-axes of 11.2 mm, 14.0 mm, and 33.5 mm.

Against the mug's 15 mm physical grasp tolerance (figure 10.4, panel a), the ellipsoid fails on one axis only. While both lateral semi-axes fall comfortably within the mechanical tolerance window,

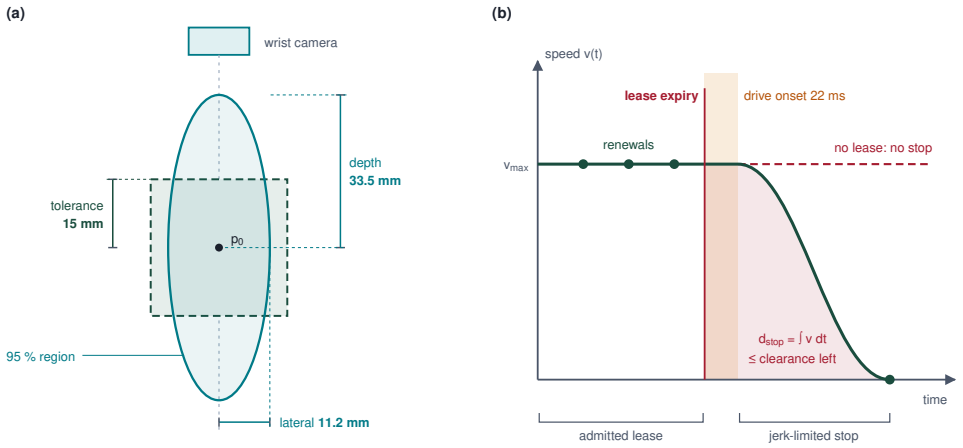


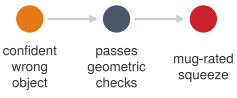
Figure 10.4: Localization uncertainty and authority expiry: Panel (a) draws the handle estimate's 95 percent region to scale against the 15 mm grasp tolerance: the lateral semi-axis of 11.2 mm fits, while the 33.5 mm depth semi-axis along the camera ray does not. Panel (b) sketches speed against time: renewals keep the lease alive, and when they stop the lease expires, the drive takes 22 ms to begin braking, and a jerk-limited stop follows. The shaded area is the stopping distance, which must fit the clearance that remains; the dashed trace is motion with no lease, which never stops.

the depth semi-axis of 33.5 mm substantially exceeds the allowed threshold. Consequently, the admission gate rejects the proposal, because the controller cannot guarantee that closing the gripper fingers at that estimated setpoint will capture the handle rather than collide with the rim.

The remedy, worked through in section 10.6, decomposes the reach into distinct phases. The system first admits an intermediate, collision-checked inspection waypoint, allowing the arm to travel closer to the workspace. Once positioned nearby, a second wrist-camera observation constructs a refined belief record, reducing the initial position uncertainty to 3 mm. Because a Gaussian probability region retains an unbounded asymptotic tail, statistical distributions alone cannot certify absolute spatial clearance. The downstream intent proposal therefore needs independently justified conservative error envelopes in the robot base frame before requesting trajectory tracking. Covariance alone supplies only a statistical region.

While a configured statistical gate cleanly rejects detections with large variance envelopes, it cannot catch an entirely incorrect object identified with high semantic confidence. This is a different case from the jar beside the mug, where both candidates entered the prediction set and the producer refused. Consider instead the jar in place of the mug, where a fragile, thin-walled glass jar sits on the workstation where the ceramic mug was expected and no mug is present. When prompted to pick the mug, the vision-language model misidentifies the jar with strong semantic certainty, generating a sharp, low-variance geometric proposal centered on the glass container. The downstream trajectory planner accepts the coordinate, solves inverse kinematics in $SE(3)$, verifies joint position and velocity limits, and confirms that the approach path clears all static obstacles in the occupancy map. Because the requested motion satisfies all kinematic and wrench envelopes, the real-time permission path authorizes execution. However, if the gripper applies the clamping force required to hold a heavy ceramic mug, the jaws crush the thin glass container upon contact despite clearing every geometric check.

This scenario marks the boundary between physical verification and semantic premises. The deterministic permission path enforces coordinate envelopes, kinematic limits, actuator torque thresholds, and obstacle clearance against known geometric occupancy grids. These physical invariants cannot confirm whether a given coordinate actually corresponds to the requested object. The premise that the targeted item is indeed the requested ceramic mug remains an unverified upstream assumption outside the hardware controller's reach. Establishing that premise requires distinct task-level perceptual evidence, which must be tracked through the handover release case (section 16.6) and recorded in the residual-claims register (section 18.3.3). A learned task filter upstream cannot close this gap either. The AutoRT deployment showed that a language-model



A confident wrong object passes every geometric check until the gripper closes.

task filter improved the fraction of sensible task proposals while unsafe commands still passed it. Human teleoperators therefore had to intervene and halt execution (war story 10.1) (Ahn et al. 2024).

War Story 10.1: AutoRT task filtering and supervision (2024)

Context: AutoRT deployed foundation models to propose and filter physical manipulation tasks for fleets of mobile manipulators operating in unstructured human office environments (Ahn et al. 2024).

Mechanism: In a sample of 259 proposed tasks, human annotators judged 228 (88 percent) acceptable. A second LLM pass then filtered the tasks against a robot "constitution" of safety and embodiment rules. Of the 214 tasks it kept, 200 (93 percent) were acceptable. The filter rejected 17 of the 31 unsuitable tasks, a recall of 55 percent, and passed the other 14.

Impact: All 14 misses arose during teleoperated collection. Although the filter raised overall proposal quality, human supervisors had to intervene to intercept every unsafe proposal before robot execution.

Response: The system architecture placed physical safety outside the model's domain. Deployments supplemented learned proposals with hardware joint-force limits, physical emergency stop buttons, and direct human line-of-sight supervision or physical perimeter barricades.

Systems lesson: A learned task filter can improve proposal quality, but it cannot serve as a safety referee. An independent, deterministic permission and fallback path must validate every physical action against real-time plant limits before execution.

The distant view and the glass jar mark two distinct verification boundaries. The distant view fails on metric uncertainty along the depth axis, whereas the jar in place of the mug fails on an unverified semantic premise, because metric precision does not establish object identity. A safe geometric proposal must therefore carry both dated physical evidence and task-specific selection evidence across the silicon boundary. By this point the mug proposal holds most of what the lease will carry. The grounded affordance has fixed the target pose and its goal region, and the squeeze bounds of the grasp analysis have set the wrench ceiling. The operator table has named the predicates the grasp must discharge, and the handle's error envelope has been compared with the task tolerance. Carrying these values is not yet holding authority, however. The proposal still lacks a grant, a deadline, and a stop that works without the proposer, and turning it into bounded authority over the arm is the work of the handoff.

10.4 The Intent Handoff and Bounded Authority

If a deliberative host could stream raw joint trajectories or actuator currents directly to the drives, a two-millimeter depth error into a rigid latch plate or a stalled inference thread mid-reach would drive motor currents to saturation before any host process could intervene. When the arm reaches for the red mug, the deliberative host instead submits a candidate intent specifying a bounded target region, an expiration deadline, and explicit physical effort ceilings. The region and the ceilings are the grounding results packaged for transit, the affordance pose with its goal set and the grasp's wrench and speed limits, and the requested deadline is computed from the parent belief record's evidence epoch. An independent real-time supervisor intercepts this proposal, vetting each requested parameter against retained sensor evidence and configured plant dynamics. Granting a target lease merely authorizes downstream trajectory planning within certified safety bounds. The deliberative model remains isolated from low-level actuator registers.

This permission path validates every requested target constraint before any motion planner can generate trajectory setpoints. Rather than forcing memory-constrained microcontrollers to maintain full scene representations, host-side spatial engines export compact geometric evidence across a strictly typed interface. The supervisor verifies four mandatory invariants against this evidence:

1. *Spatial containment invariant:* The proposed geometric goal set $\mathcal{G}_{\text{goal}} \subset SE(3)$ must lie strictly within the robot's reachable workspace manifold $\mathcal{W}$. The bounded spatial uncertainty envelope (e_x, e_y, e_z) must fit inside the task tolerance (r_x, r_y, r_z) without intersecting persistent obstacle geometry in spatial memory $\mathcal{M}$.

2. *Temporal freshness invariant:* The sensor evidence epoch t_0 must match a retained parent belief record. The unobserved elapsed time on the Safety MCU's monotonic hardware clock must not exceed the evidence horizon τ_{ev} that section 10.5 takes from the parent belief record.
3. *Dynamic effort invariant:* Requested tool-center-point velocity $v_{\max}$, acceleration $a_{\max}$, and contact wrench $\mathbf{w}_{\max} = [F_{\max}, \tau_{\max}]^\top$ must remain strictly within actuator thermal ratings, link inertia limits, and workpiece structural yield thresholds.
4. *Certified deterministic fallback invariant:* The robot must possess an independently verified, pre-computed stopping or hold trajectory that brings all moving links to rest within current physical obstacle clearance ($d_{\text{stop}} \leq D_{\text{clear}}$) if authority is abruptly revoked.

The first three invariants turn the requirements that opened the chapter into checks: metric physicality and bounded error become spatial containment and the effort ceiling, while temporal expiration becomes freshness. The fourth adds the obligation those requirements left implicit, a stop that works without the proposer. Each invariant guards against a distinct failure mode in physical interaction. Loose target bounds can exceed kinematic grasp tolerances, and stale sensor evidence risks collision with a displaced object. Excessive commanded effort can shatter a workpiece, and an unvetted handoff leaves moving mass without a safe recovery trajectory. The resulting intent lease records the validated envelope that survives these four checks and constrains the downstream trajectory planner, which must synthesize path timing and deceleration profiles that satisfy supervisory admission criteria (section 10.8). Pairing the evidence epoch with a finite lease expiration keeps the temporal bound deterministic even if the host crashes, because the safety controller compares its own clock against the admitted deadline rather than waiting for a cancellation packet (section 10.5).

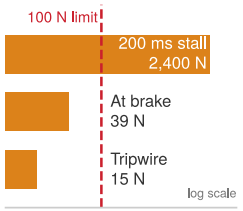
Spatial and temporal bounds alone cannot guarantee safety if the downstream controller commands unbounded effort. Every intent proposal must therefore declare explicit effort ceilings covering tool-center-point velocity, acceleration, and contact wrench. Consider a depth perception error of two millimeters that places a target goal slightly inside a rigid workpiece. An unrestricted position controller will drive actuator currents to saturation trying to eliminate tracking error, imposing severe mechanical stress on the gear train. During a guarded approach to a spring-latched cage door, for example, the host proposes a 15 N tripwire ceiling, well below the 100 N structural latch threshold, paired with an approach velocity capped at 0.03 m/s. The permission path evaluates these values against hardware torque-speed curves and environmental thresholds, admitting only parameters that guarantee structural integrity.

Downstream of the permission path, an impedance controller shapes interaction dynamics, answering a position error with the restoring force of the virtual spring-damper established in section 2.4.4. Under quasi-static contact, scaling the virtual stiffness along the approach vector directly limits steady-state contact force for a given positional error. Transient impacts, however, demand more than static stiffness modulation; containing peak collision forces requires coordinating approach velocity, passive mechanical compliance, sensor latency, and actuator closed-loop bandwidth.

Omitting spatial, temporal, or dynamic boundaries leaves physical hardware vulnerable during communication dropouts or perception stalls. Stopping distance scales linearly with the reaction delay preceding brake engagement and quadratically with operating velocity (equation 2.2). For a mobile base navigating within physical obstacle clearances, a silent or stalled proposer inflates this reaction window by the entire lifespan of the active command lease. As established in section 4.4, this chunk lease duration directly extends the unbraked latency parameter τ_{delay}. The verified safety envelope must therefore maintain obstacle clearance greater than the total stopping distance, factoring in sensor pipeline latency, transport jitter, and state estimation uncertainty. Furthermore, powered deceleration requires electrical bus energy and closed-loop control authority to survive an upstream fault. Logic-level power alone cannot generate mechanical braking torque.

The intent lease defines the temporal horizon over which a deliberative goal remains valid. It operates alongside the chunk lease of section 4.4 rather than replacing it, because the two hold different kinds of authority even though the Safety MCU's permission path checks both. The chunk lease authorizes motion. It allows an admitted chunk to drive the actuators only for its short lease unless the chunk policy renews it at its own rate. By contrast, the intent lease authorizes a goal, fixing which target that motion may pursue, how hard it may press, and until when, and only newer evidence can renew it. Consequently, each lease ends authority from its own side. When

the chunk lease lapses, motion stops even though the goal remains valid. When the intent lease lapses, nominal tracking toward the goal ends at once, even if the current chunk still has lease time left, and the MCU engages the validated fallback (section 11.2.2). A rejected proposal is the gentler case, because it revokes nothing and leaves already admitted motion running until its own leases lapse. The physical stopping budget must accommodate the cumulative delay across perception, communication, actuator response, and mechanical braking in every case.



A 200 ms unbraked stall drives the latch to 2,400 N, far past its limit.

Napkin Math 10.1: What the latch ceilings buy

Problem: The intent lease for the cage door requests a 15 N tripwire and a 0.03 m/s approach speed against a 100 N latch limit. With the local contact loop in place, what force does the latch see when the brake command issues, and what would it see in a counterfactual design with no local contact trip, where pressing stops only when the proposer or the intent lease acts?

Variables:

- Tripwire 15 N and approach speed $v_{\text{latch}} = 0.03$ m/s, the requested ceilings; latch limit 100 N (Illustrative)
- Plate stiffness $k_{\text{latch}} = 400$ N/mm (Illustrative) and contact-loop response 2 ms
- Unbraked stall $\Delta t = 200$ ms (Illustrative), shorter than one 500 ms intent period

Math:

1. **Force at brake command**: The tripwire fires at 15 N, and the plate keeps compressing while the 2 ms contact loop responds, so the force when the brake command issues is $F_{\text{cmd}} = F_{\text{trip}} + k_{\text{latch}}\, v_{\text{latch}}\, T_c = 15\,\text{N} + 400\,\text{N/mm} \times 0.03\,\text{m/s} \times 2\,\text{ms} = 39\,\text{N}$. The arm stop that follows has only a fraction of a millimeter of compression left to spend; section 12.7.2 derives that stop (equation 12.11).
2. **Stall travel** (counterfactual, no local trip): $\Delta x = v_{\text{latch}}\, \Delta t = 0.03\,\text{m/s} \times 200\,\text{ms} = 6\,\text{mm}$ past first contact.
3. **Stall force**: $F = k_{\text{latch}}\, \Delta x = 400\,\text{N/mm} \times 6\,\text{mm} = 2{,}400\,\text{N}$.

Result: The contact loop brakes at 39 N. Without the local trip, a stop that waited on the proposer would let the arm press on, and the spring model predicts 2,400 N.

Systems insight: The predicted force lies far past the 100 N limit, beyond the point where the spring model applies at all; the latch has failed. Sizing and arresting transient contact within milliseconds is the duty of the high-rate, low-latency contact loop. The intent lease supplies the operational boundaries (velocity caps and force tripwires) that keep dynamic loads within the contact loop's braking envelope, ensuring physical safety even when upstream deliberative proposers stall across multiple intent cycles.

Both the mug grasp and the latch engagement depend on a **privilege separation boundary**.[7] While deliberative models propose goal targets and operational bounds, an independent supervisor enforces safety invariants and executes certified fallback maneuvers whenever evidence expires or contact states change unexpectedly. This design decouples deliberative reasoning from real-time physical protection, inverting the classical priority hierarchy found in early reactive control systems (Brooks 1986).

Structured intent handoffs explicitly disentangle task tolerance, sensor uncertainty, actuator effort limits, and fallback trajectories. External dynamic processes, however, dictate how long these static bounds remain valid. On a moving conveyor, for instance, a correctly localized workpiece drifts continuously after perception capture. The mathematical model governing this uncertainty growth establishes the strict temporal deadline that the independent safety controller must enforce.

10.5 Expiring Goals and Temporal Validity Leases

Suppose the host application processor hangs while the robotic arm reaches for the red mug on the moving belt. The fault that makes canceling the reach necessary is the very fault that prevents a cancellation command from being sent. An explicit abort requires a working sender, scheduler, and

[7] **Subsumption Architecture**: In the classic subsumption architecture of Brooks (1986), a higher layer subsumes a lower one by suppressing its inputs or inhibiting its outputs. Physical AI systems invert that priority. The lower, real-time layer retains absolute veto authority over deliberative proposals because its timing, stability, and failure behavior can be strictly bounded.

network link, while the belt carries the mug beyond the grasp tolerance regardless of when the model last published a coordinate. An intent lease must therefore expire autonomously from the physics of environmental drift. Autonomous expiry applies the third law, proposal is not permission (principle 3), to physical goals, so that the permission side and not the proposer decides when authority ends. In distributed systems, Gray and Cheriton (1989) introduced leases to maintain cache consistency: a coordinator grants time-bounded authority that lapses unless explicitly renewed, preventing a crashed client from blocking others indefinitely. That correctness proof assumes bounded clock drift between client and coordinator. In an embodied system, the Safety MCU enforces this assumption by transforming every incoming timestamped epoch into its own local monotonic timebase with a strictly bounded synchronization error. The physical domain raises the stakes, because an expired cache lease merely delays a read, whereas an unexpired motion lease lets moving mass keep consuming physical clearance unchecked. The privileged Safety MCU acts as coordinator, issuing a transient tracking lease to the unprivileged deliberative host. If the host stalls during inference, pauses for garbage collection, or experiences an operating system kernel panic, the Safety MCU's local hardware countdown expires and automatically engages a validated fallback response. This requires no network packet or cancellation command from the failed host.[8]

Lease duration is measured exclusively against the Safety MCU's monotonic hardware clock.[9] Network time-synchronization daemons or operator adjustments can step wall-clock time abruptly backward or forward, while host-level software watchdogs can freeze alongside the user-space process they are designated to monitor. Furthermore, no inference pipeline, regardless of its mean throughput, completely eliminates long-tail p99 scheduling stalls. Placing the lease countdown on an isolated, hardware-timed MCU counter ensures that target authority is revoked locally the instant the lease expires. This hardware isolation decouples expiry detection from host operating system jitter, although total physical stopping distance must still account for fieldbus transmission, drive response, and kinematic deceleration.

The deadline the lease enforces is the evidence horizon τ_{ev} of section 3.5.2 put to a new use. There it bounded how long the evidence behind a chunk keeps the tracked mug within tolerance (equation 3.5). Here the intent lease starts from the same quantity, and memory supplies its value. The mug-handle belief record carries an invalidation horizon T_{inv} (equation 9.14), the elapsed time before its error growth law (equation 9.12) reaches the task tolerance. The lease reads that horizon with the grasp tolerance r_{tol} as the error ceiling and the initial sensor error e_0 as the starting bound.

For a conveyor pick, the growth law charges two bounded effects against that ceiling:

$$E(\Delta t) = e_0 + v_{\text{drift}}\,\Delta t + \tfrac{1}{2}\,a_{\text{slip}}\,\Delta t^2 \tag{10.4}$$

Here v_{drift} bounds the relative velocity between the gripper frame and the target, including both conveyor motion and unmodeled mobile-base odometry drift, and a_{slip} bounds the residual slip acceleration of the mug on the belt. The evidence-only horizon is the elapsed time at which $E(\Delta t)$ reaches r_{tol}. The numerical examples in this chapter evaluate this upper ceiling with zero additional controller tracking reserve. An admission gate, however, must first subtract a separately qualified nonnegative scalar tracking reserve from r_{tol}. It then solves equation 10.4 against the reduced budget. It refuses immediately if the initial target error plus that reserve already reaches the tolerance (alongside the full 3D error-set containment check of section 10.8). The two limiting regimes are horizons derived earlier in the volume. With no tracker, the horizon is the linear horizon of equation 9.15, with belt speed as its growth rate. With a tracker that cancels belt velocity, the horizon is the slip-only root of equation 3.5. In this chapter's symbols they read

$$\tau_{\text{ev,untracked}} = \frac{r_{\text{tol}} - e_0}{v_{\text{drift}}} \quad (a_{\text{slip}} = 0), \qquad \tau_{\text{ev,tracked}} = \sqrt{\frac{2(r_{\text{tol}} - e_0)}{a_{\text{slip}}}} \quad (v_{\text{drift}} = 0) \tag{10.5}$$

These evidence-only roots are upper ceilings on target lifetime. When drift velocity and slip acceleration act together, τ_{ev} is the positive root of $E(\Delta t) = r_{\text{tol}}$ in equation 10.4, which is strictly shorter than either single-effect root.

If $r_{\text{tol}} \le e_0$, the target is inadmissible at the moment of perception. Initial measurement uncertainty alone consumes the entire task tolerance, which obligates the safety controller to reject dependent motion commands immediately. Even when the evidence horizon is strictly positive, granting motion

[8] **Watchdog Timer**: An independently clocked down-counter that forces a hardware reset or a safe fallback output unless software reloads it before reaching zero. Because a crashed host cannot reload it, the lease countdown runs on the isolated Safety MCU rather than in the host process it supervises.

[9] **Monotonic Clock**: A counter that only advances forward at a fixed nominal rate, such as POSIX `CLOCK_MONOTONIC` or a free-running microcontroller timer. A lease evaluated against wall time could gain or lose its entire remaining window in a single clock adjustment. Every measurement in this chapter is therefore evaluated on the Safety MCU's monotonic counter and converted from external clocks with a bounded synchronization error.

permission requires a separate geometric verification. The system must verify that current physical clearance accommodates total sensing latency, permission evaluation time, fieldbus transmission delay, mechanical brake engagement, and physical braking distance.

For a red mug carried on a takeaway conveyor, the untracked linear root dictates the evidence deadline. Sizing the control pipeline requires determining whether deliberative intent models can refresh targets rapidly enough to renew authority before this window elapses. While the evidence-horizon calculation in section 3.5.2 sized the low-level chunk execution policy against this horizon, deliberative intent models evaluate on a substantially slower cycle.

Napkin Math 10.2: The cadence gap

Problem: The conveyor carries the mug at 0.20 m/s. Can the 2 Hz intent model keep the target inside its tolerance by re-proposing it, and how slowly would the belt have to run for a model-only renewal to suffice?

Variables:

- Grasp tolerance $r_{\text{tol}} = 15$ mm and initial handle error $e_0 = 3$ mm (Illustrative)
- Belt speed $v_{\text{drift}} = 200$ mm/s (Illustrative)
- Intent period $T_{\text{period}} = 500$ ms (Chosen, 2 Hz) and inference $t_{\text{infer}} = 336$ ms (Derived, section 3.4.1)

Math:

1. **Horizon** (equation 10.5): with no tracker, $\tau_{\text{ev}} = \tau_{\text{ev,untracked}} = (15\text{ mm} - 3\text{ mm}) \,/\, 200\text{ mm/s} = 60$ ms.
2. **Error at one period**: $3\text{ mm} + 200\text{ mm/s} \times 500\text{ ms} = 103$ mm, 88 mm past the tolerance. Inference alone overruns the horizon by $336\text{ ms} - 60\text{ ms} = 276$ ms.
3. **Renewal ceiling**: Each model-only renewal installs a target computed from a frame captured one inference earlier, and that target stays in force until the next renewal replaces it one period later. Just before the replacement, the evidence behind it is $T_{\text{period}} + t_{\text{infer}} = 836$ ms old (the reaction latency of section 3.5.1, here for the intent model), so the drift it can keep up with is $(15\text{ mm} - 3\text{ mm}) \,/\, 836\text{ ms} = 14.4$ mm/s.

Result: The model alone could renew the target only on a belt running at 7.2 percent of its speed. Even a slow settling drift of 20 mm/s, such as a mug still sliding to rest after the belt stops, whose 600 ms horizon is charted against inference latency in figure 10.6, is too fast for it.

Systems insight: Physical drift and allowable tolerance dictate τ_{ev}, independent of computing performance. An intent renewal can extend operational authority only if the sum of its dispatch period and inference latency fits entirely within this window. Artificially inflating the lease duration to cover inference delay leaves the pose estimate outside its declared tolerance envelope for 276 ms. Without a qualified, high-frequency local tracker, the system must either slow or stop the conveyor, or refuse the pick task entirely.

The 60 ms limit is a physical evidence deadline that only fresh sensor observations, validated against current motion, can extend. Faster deliberative inference does not lengthen this physical evidence horizon, although a sufficiently fast capture-to-admission pipeline and renewal cadence can fit inside it.

Deploying a qualified local tracker changes these dynamics. By re-measuring the mug in every camera frame and matching conveyor velocity, the local tracking loop prevents conveyor speed from driving error growth. While the initial perception uncertainty of 3 mm still counts against the grasp tolerance, it leaves a target-only error margin of 12 mm for unmodeled physical perturbations before reserving controller tracking error. Sizing against an illustrative worst-case residual slip acceleration of 0.50 m/s², this remaining margin yields the same 219 ms tracked horizon that section 3.5.2 computed for the chunk policy. Because the local tracking pipeline runs at high frame rates, only the tracker's brief period plus sensing latency must fit within this expanded horizon. The tracker also changes who renews the lease. Each tracked frame yields a newer belief record of the same handle,

and the producer re-proposes the same referent against it. The Safety MCU therefore receives renewals with a newer capture epoch on the tracker's cadence rather than the model's. Relieved of geometric tracking, the deliberative intent model only re-confirms semantic identity (which mug to grasp) at its own rate, and no renewal waits for a fresh inference. The lease attaches its target to the handle's affordance frame, whose base-frame pose under a qualified tracker follows the belt. The goal set therefore moves with the mug. The tolerance bounds only how far the handle departs from the tracked estimate under residual slip, rather than how far it departs from a fixed point in the base frame. Whichever horizon governs the active operating mode caps the duration of the intent lease granted for that target (section 10.8).

Examining a host processor stall illustrates the protection each horizon provides. Suppose the robotic arm reaches toward the red mug under a target admitted without local tracking, and the application processor stalls during neural network inference. If the system held that unvalidated target across a 200 ms stall, the actual mug position would drift 43 mm away from the commanded setpoint, causing the gripper to close on empty air or strike the rim. The Safety MCU does not rely on host health to catch this fault. In the ideal zero-additional-reserve case, at 60 ms the lease expires automatically, commanding braking to begin within 22 ms (accounting for one permission evaluation tick, one fieldbus cycle, and actuator brake onset), even if the current chunk's own lease has time left. In contrast, under a qualified local tracker, an identical 200 ms stall leaves the target estimate displaced by only 13 mm, inside the target-only tolerance budget, because the ideal tracked horizon extends to 219 ms. Delegating fast geometric updates to local tracking is what allows a 2 Hz deliberative intent model to operate safely in a dynamic physical environment.

On one clock, the proposer's timing loses to both horizons (figure 10.5). Both idealized zero-additional-reserve countdowns end before the 336 ms inference returns and well before the next proposal at 500 ms, so the model can renew neither lease. Absent other renewals, each expiry is followed by the 22 ms stop onset. Under a qualified tracker the tracked countdown restarts with every tracked frame, long before it reaches 219 ms, so the tracked bar in the figure shows what happens only when those renewals stop, as they do when the host stalls. What follows that expiry is still motion, as panel (b) of figure 10.4 sketches in speed against time. The drive begins braking 22 ms after expiry, and the stopping distance under the falling curve must already fit the clearance that remains. This is why the lease carries the stopping deadline as a term of its own (section 10.8).

Panel (a) of figure 10.6 generalizes the cadence gap across drift speeds. The 336 ms inference alone fits under the 600 ms horizon of slow settling drift but far exceeds the 60 ms conveyor horizon, and adding the proposal period, as the cadence-gap notebook showed, breaks even the settling case. Panel (b) previews the cheap early filters (sequence and evidence, workspace, transit lower bound, and ambiguity) that reject impossible proposals before the planner spends compute on them. Sustaining continuous tracking safely requires *fresh physical evidence*, not merely retransmitting a new packet computed from stale sensor observations. Every renewal must derive from an independent physical capture and reach the Safety MCU before the active lease expires. The end-to-end timing budget must account for image capture, inference processing, fieldbus transmission, clock synchronization uncertainty, and permission verification. The timing condition $T_{\text{period}} + L_{\text{lat}} < \tau_{\text{ev}}$ serves only as a necessary scheduling check. When a renewed proposal arrives, the trajectory planner computes a jerk-limited splice originating from the measured physical state. The safety controller admits this spliced trajectory only if the new swept volume, refreshed evidence validity, and emergency stopping reserves remain fully verified. An abrupt target displacement that violates kinematic or clearance bounds triggers immediate refusal or a controlled stop.

Safely executing a bounded lease presupposes that the commanded geometric target is physically achievable by the robot. If a proposed coordinate falls outside kinematic reach or demands joint velocities that exceed actuator limits across the lease window, granting the lease squanders control authority. It also risks tripping drive overcurrent protection as motor controllers saturate attempting to reach unattainable velocities. Determining whether an incoming goal is kinematically and dynamically admissible must precede trajectory generation, establishing a physical admissibility filter between high-level deliberative intent and real-time execution.

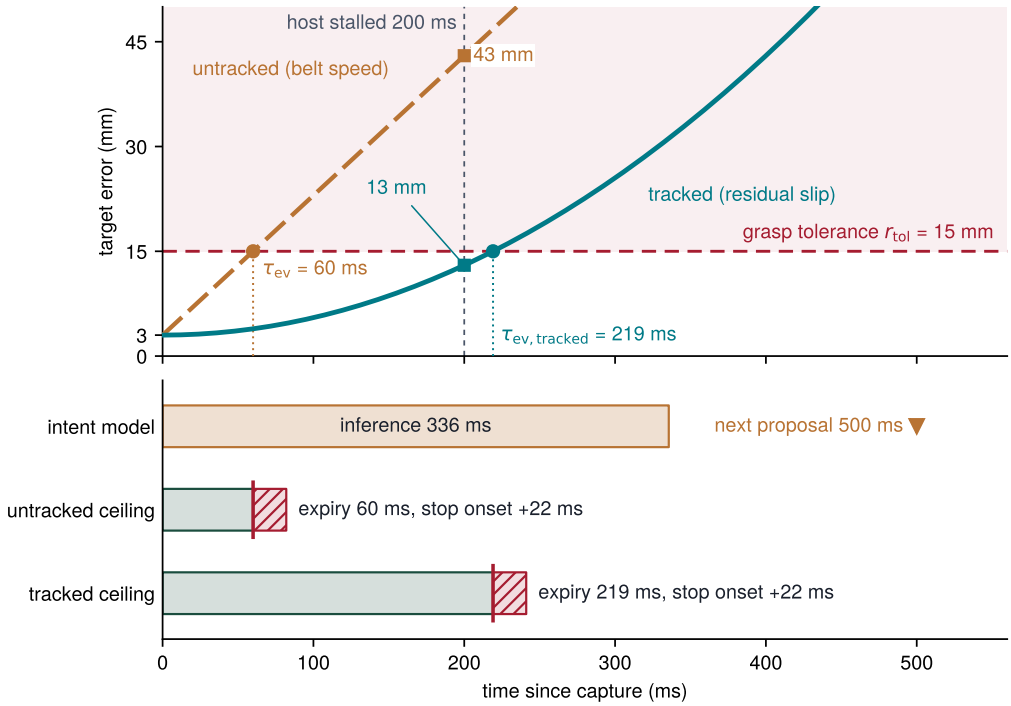


Figure 10.5: Mug lease against the intent model's timing: The top lane charges the conveyor mug's modeled target error against the 15 mm grasp tolerance. Without tracking, belt speed exhausts it at 60 ms; with a tracker that cancels belt velocity, residual slip exhausts it at 219 ms. The bottom lane puts idealized countdowns with zero additional controller reserve on the Safety MCU's clock beside the 336 ms inference and the 500 ms proposal period; each expiry is followed by the 22 ms stop onset (hatched). Neither lease can wait for the next proposal. After a 200 ms host stall the untracked target sits 43 mm off and the tracked one 13 mm.

10.6 Admissible Goal Sets

If the red mug rides past on the far side of the belt, beyond the arm's physical reach from where the base is parked, no sequence of joint torques can bring the gripper to its handle. In an embodied ML system, this physical impossibility must be intercepted at the proposal boundary, before the target enters the trajectory optimization queue. When an upstream vision-language model or diffusion policy emits a candidate Cartesian coordinate, treating that output as an unverified proposal protects the real-time motion pipeline from executing physically impossible trajectories and from wasting compute cycles attempting to solve them.

A reachability screen also protects the planner's computational budget. When a target cannot be reached at all, the trajectory optimizer faces an empty feasible set. Local nonlinear solvers cannot certify infeasibility quickly. They typically exhaust their entire iteration budget before timing out, transforming an upstream geometric hallucination into compute starvation on the planning core. Similarly, a requested terminal move that cannot complete before its supporting sensory evidence expires must fail before entering detailed planning. Long-horizon tasks should instead request guarded progress under renewable evidence leases rather than attempting an ungrounded terminal grasp on a single sensor capture.

To avoid spending the planning budget on an already impossible request, the admission layer evaluates proposals against two low-cost preliminary checks:

1. **Static Workspace Manifold Check ($\mathbf{T}_{\text{target}} \in \mathcal{W}$):** Geometric workspace analysis defines the closed spatial manifold $\mathcal{W} \subset SE(3)$ of reachable end-effector configurations given link dimensions, joint travel limits, and mounting geometry (Craig 2005; Lynch and Park 2017).[10] An analytical reachability check evaluates whether the target $SE(3)$ pose $\mathbf{T}_{\text{target}}$ resides inside

[10] **Workspace vs. Singularity**: Membership in $\mathcal{W}$ says that some joint configuration reaches the pose, not that the arm can move freely there; inside the workspace the Jacobian can still lose rank at a singular configuration, where some Cartesian velocities become unattainable. A workspace screen therefore rejects unreachable targets early, leaving singularities to the configuration and path checks; section B.6.1 develops the Jacobian rank loss behind them.

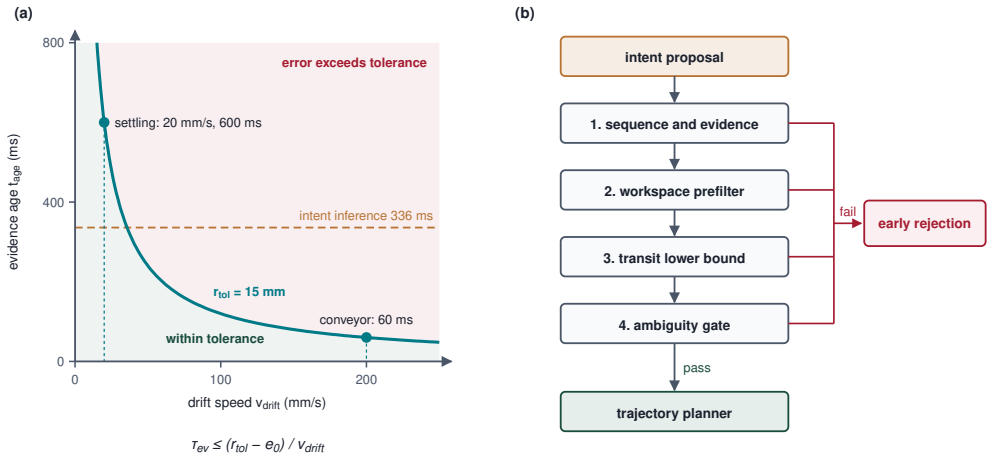


Figure 10.6: Target-evidence horizon and early filters: Panel (a) plots the bound $\tau_{\text{ev}} \le (r_{\text{tol}} - e_0)/v_{\text{drift}}$ for the 15 mm tolerance and 3 mm initial error; the shaded region below the curve means only that the target remains within the task tolerance. The machine's computed 336 ms intent inference sits far above the 60 ms conveyor horizon and below the 600 ms settling horizon. Panel (b) shows early rejection stages that precede the full path and stopping checks.

this workspace manifold $\mathcal{W}$, while an occupancy lookup in spatial memory confirms that the target volume does not intersect persistent static obstacles.

2. **Dynamic Reachability Transit Check:** Even when a target lies inside the static workspace manifold $\mathcal{W}$, it may be physically unreachable within the finite window during which its sensor evidence remains valid. This check evaluates whether the end effector can transition from its measured state $(\mathbf{p}_0, \mathbf{v}_0)$ to the target pose within the remaining window $t_{\text{intent_exp}} - t_{\text{now,mono}}$ while respecting its velocity, acceleration, jerk, and thermal torque limits.

Whatever continuous trajectory profile the actuators follow, no physical move covers distance $d = \|\mathbf{p}_{\text{target}} - \mathbf{p}_0\|$ faster than its maximum speed ceiling allows. Here both $\mathbf{p}$ terms are translation vectors in the same Cartesian frame, not full $SE(3)$ poses; orientation feasibility requires a separate check. For terminal completion before the current expiry, a necessary condition is:

$$\frac{d}{v_{\max}} \le t_{\text{intent_exp}} - t_{\text{now,mono}}. \tag{10.6}$$

Actuator acceleration limits, jerk ceilings, and motor torque-speed curves only lengthen the required transit time beyond this kinematic floor. A proposal that violates this inequality cannot promise terminal completion in that window. A request for an intermediate guarded segment may remain feasible if its executed motion and stopping fallback fit the current authority, deferring continuation to newly admitted evidence. Section B.6 gives triangular and trapezoidal transit bounds and explains how jerk limits change the complete-move check.

The scalar lower bound is an early rejection test, not a trajectory generator. It avoids optimizing a terminal request that is already too late, leaving detailed acceleration profiles, obstacle avoidance, and stopping reserves to the fuller motion check. Equation 10.6 couples spatial distance to evidence lifetime, so dynamic reachability evaluates the target and its supporting evidence together rather than the spatial coordinate alone.

↪ Resolving the admitted goal region into a timed trajectory is developed in section 11.3.

Definition 10.2: Dynamic reachability

Dynamic reachability is the property that the end effector can move from its measured state to a target pose, within its velocity, acceleration, jerk, and thermal limits, before the evidence supporting that target expires.

1. **Significance**: It prices a target in time as well as space. At the arm's 1 m/s TCP limit, equation 10.6 allows at most 219 mm of travel inside the mug's 219 ms tracked evidence window, so a handle 300 mm away is dynamically unreachable on one capture even though it lies well inside the workspace.
2. **Distinction**: Static reachability evaluates geometric feasibility alone (whether any joint configuration can achieve the pose). Dynamic reachability introduces the clock and actuator dynamics. Unlike the dynamic obstacle forecasting of section 10.6.1, which bounds where external actors might move, dynamic reachability bounds where the robot's end effector can safely travel within a bounded lease.
3. **Common pitfall**: Attempting to resolve an unreachable target by stretching its evidence deadline in software. Only fresh sensor measurements can renew an expiring window; the remedy is a staged approach whose segments each fit their own evidence and fallback.

On the arm, the mug's evidence under the qualified tracker stays valid for 219 ms after its capture. At the 1 m/s TCP speed limit, the gripper could cover at most 219 mm even if the entire window remained. A terminal proposal issued with the gripper 300 mm from the handle needs at least 300 ms of transit. It therefore receives `ERR_DYNAMIC_TIME_INSUFFICIENT` if it promises completion on that record. Without the tracker, the full window is only 60 ms and the corresponding speed-only reach is 60 mm. Processing and transport consume part of either window. Acceleration further reduces the reachable distance.

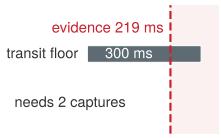
This bound and the fuller checks behind it assume that actuators stay within their nominal thermal ratings, that encoders report the initial state $(\mathbf{p}_0, \mathbf{v}_0)$ correctly, and that the payload matches the inertial model. When winding temperatures cross the thresholds of section 2.5.1, or a force measurement reveals a payload mismatch, the permission path lowers $a_{\max}$ and $v_{\max}$ (**thermal derating**). It then evaluates new proposals against the reduced limits and rechecks any admitted path that the derating may have made infeasible.

A rejection must not simply discard the packet, which would leave the reasoning model waiting for a task timeout. The intent filter returns a structured diagnostic across the proposal boundary: the classification (`ERR_WORKSPACE_OUT_OF_BOUNDS`, `ERR_DYNAMIC_TIME_INSUFFICIENT`, or `ERR_OCCUPANCY_BLOCKED`) and the numerical margin of failure. A dynamic failure reports the shortfall between the transit bound and the remaining window; a workspace failure reports the nearest reachable point on the boundary of $\mathcal{W}$. The reasoning layer can then seek fresher evidence, choose another approach, slow the process, or propose moving the base. It cannot extend the evidence horizon without new sensor data.

The diagnostic makes a staged approach possible. Consider a setup where the pre-contact corridor and its stopping reserve are qualified, a closer wrist observation supports the 3 mm error bound, and the tracker holds the declared slip bound. The rejected terminal promise becomes a pre-contact observation goal. A separately checked corridor carries the gripper toward a better view while keeping a stop outside contact, and the closer capture supplies the mug-handle belief record whose error fits the 15 mm tolerance.

Better sensing does not shorten the physical transit floor. The 300 mm approach still takes at least 300 ms, longer than the 219 ms horizon of any one capture, so it needs at least 2 fresh captures. The planner executes guarded segments under chunk leases, and each continuation is checked against the current intent, target evidence, corridor, and fallback. The predicted route beyond the admitted segment carries no authority. Once the last segment and the closure pass the full trajectory check, the gripper completes the grasp inside the tolerance and moves to the admitted hold for handover. If the closer observation fails, identity becomes unresolved, or a renewal arrives late, the next segment is refused, and existing motion continues only while its own evidence, authority, and fallback remain valid.

↪ The staged approach, with each trajectory segment re-anchored to a fresh wrist-camera capture, is developed in section 11.3.2.



The 300 ms reach outlasts one 219 ms evidence window; it needs 2 captures.

10.6.1 Dynamic obstacle forecasting and forward occupancy

Even when the mug handle lies within kinematic and dynamic reach, a coworker may be about to cross the approach corridor, a conflict that the static workspace and transit tests alone cannot reject.

Dynamic obstacle forecasting carries the perception and memory estimates forward into regions that moving obstacles could occupy so the intent stage can reject blocked corridors early, before the planner and permission path check the complete timed trajectory. Because this forecast predicts only where people might move and says nothing about whether those people can tell where the machine will move, it covers only one half of how coworkers and robots stay clear of one another.

The intent filter consumes the dynamic track manifest $[(\mathrm{ID}_k, \hat{\mathbf{p}}_k, \hat{\mathbf{v}}_k, \mathbf{P}_k, t_{\text{last}}, \text{state})]$ published by the multi-object tracker, alongside tentative detections, raw occupied regions, and unknown-space evidence, because failure to confirm an identity does not make its occupied region clear. For each confirmed or coasting track k, the admission engine constructs a **forward occupancy set** $\mathcal{R}_k(\Delta t)$, the region the actor could occupy Δt after its last measurement:

$$\mathcal{R}_k(\Delta t) = \{\mathbf{p} \in \mathbb{R}^3 \mid \|\mathbf{p} - \hat{\mathbf{p}}_k\| \le r_k + v_h \Delta t + k_\sigma \sigma_{\text{track}}(\Delta t)\} \tag{10.7}$$

Here r_k is a declared body envelope that covers torso, limbs, and carried objects. The term v_h is a validated bound on the actor's speed. The term k_σ is a selected multiplier whose joint spatial coverage must be calibrated rather than read off a scalar Gaussian rule. The term $\sigma_{\text{track}}(\Delta t)$ is the root-mean-square position deviation of the propagated track covariance. That deviation need not grow monotonically during coasting, so the check needs a conservative bound over the whole interval. $\mathcal{R}_k(\Delta t)$ is called an occupancy set rather than a reachability set because dynamic reachability bounds where the robot can get in time, whereas forward occupancy bounds where a person might be. A tracker with a validated acceleration bound can instead center the set on the predicted position $\hat{\mathbf{p}}_k + \hat{\mathbf{v}}_k \Delta t$ and grow it with $\frac{1}{2} a_{k,\max} \Delta t^2$, which is tighter over short intervals. The speed-bound form is the conservative one that safety standards use.

↪ Motion and signals that let the people around a machine infer its goal early enough to yield are developed in section 17.6.1.

Equation 10.7 is the human half of speed and separation monitoring (SSM), a certified safety function that runs beneath the permission path and keeps a protective separation distance between the moving robot and a person. Section 12.9.3 sizes that distance under ISO 10218-2:2025, which incorporates the collaborative-application requirements formerly published as ISO/TS 15066 (the technical specification itself remains published and is under revision) (International Organization for Standardization 2025b, 2016; Marvel and Norcross 2017). The separation calculation assumes the ISO 13855 walking approach speed[11] of 1.6 m/s for an approaching person (International Organization for Standardization 2024a). The intent stage uses the same speed bound for an earlier and coarser purpose, rejecting a corridor before the planner spends time on it.

↩ The dynamic track manifest and the tentative, confirmed, and coasting track lifecycle are established in section 9.2.2.

Screening a corridor means sweeping the robot's candidate transit volume $\mathcal{T}_{\text{robot}}(\Delta t)$ forward in time beside every person's set. On the aisle of figure 10.7, the coworker's set grows at the ISO 13855 walking speed from 96 mm over the untracked horizon to 351 mm over the tracked one, and the corridor enters the larger disc, so the screen refuses it. Under declared motion bounds, the early screen requires the candidate transit volume to remain disjoint from every active occupancy set across the execution window. Here t_{age_k} is the upper age of actor k's last measurement at the current admission epoch, including bounded clock-conversion error, and Δt is additional future time:

$$\mathcal{T}_{\text{robot}}(\Delta t) \cap \mathcal{R}_k(t_{\text{age}_k} + \Delta t) = \emptyset \quad \forall \Delta t \in [0, t_{\text{intent_exp}} - t_{\text{now,mono}}], \ \forall k \tag{10.8}$$

Full trajectory admission must extend the exclusion check through the response delay and the complete stopping fallback. If a candidate corridor violates equation 10.8 by intersecting an expanding set, the early screen can return `ERR_DYNAMIC_OCCUPANCY_CONFLICT` and request a different corridor or a yield. The refusal diagnostic returns the earliest predicted conflict time $\Delta t_{\text{conflict}}$ and the conflicting actor identifier ID_k. Passing this screen does not establish a collision-free trajectory. The planner must supply the timed motion, and the permission path must check its swept volume and stopping reserve against evidence that remains valid. The arithmetic of this linear growth is simple; what decides the outcome is the horizon over which it runs.

[11] **Approach-Speed Constant**: The standard's speed describes a design person walking toward a safeguard, not a measured coworker, and the standard applies a higher constant to hand and arm reach at short separations (International Organization for Standardization 2024a). A forecast built on the walking value is valid only for the motion it describes. A coworker who runs or reaches across the corridor lies outside that bound, and the growth term must therefore be validated for the site rather than copied from the standard.

↩ The mapping of the robot's protective-separation terms onto its stopping-distance budget is established in section 2.3.1.

Napkin Math 10.3: The freezing radius

Problem: A coworker walks in the 1.30 m aisle while the arm screens its corridor. How far can the growth term of equation 10.7 carry that person's set over the mug's evidence horizons and

over a 2 s planning horizon, and beyond what horizon does one person's set alone span the aisle?

Variables:

- Approach speed $v_h = 1.6$ m/s (Datasheet, ISO 13855 design value)
- Evidence horizons 60 ms untracked and 219 ms tracked (Derived, equation 10.5); planning horizon 2 s (Illustrative)
- Aisle width $w = 1.30$ m (Illustrative)

Math:

1. **Evidence horizons**: 1.6 m/s $\times$ 60 ms $=$ 96 mm, and 1.6 m/s $\times$ 219 ms $=$ 351 mm.
2. **Planning horizon**: 1.6 m/s $\times$ 2 s $=$ 3.2 m.
3. **Crossover**: The growth term reaches the aisle width when $v_h\,\Delta t = w$, at $\Delta t = 1.30$ m / 1.6 m/s $= 0.81$ s.

Result: Over the evidence horizons the set grows by centimeters to decimeters; past 0.81 s a single person's set covers the full aisle before the body envelope and tracking terms are even added.

Systems insight: The duration of the screening horizon dictates whether the robot can plan feasible motion. Extending a dynamic forecast beyond the admitted motion and its complete stopping fallback can reject a corridor for possible human movements outside the current commitment. In narrow operating aisles, unnecessarily long forecast horizons can block otherwise admissible progress. The shorter screen must still cover response delay and deceleration; evidence expiry does not end physical motion.

A set that outgrows the aisle is the root of the **freezing robot problem**. Trautman and Krause showed that when a planner predicts each person independently and lets that uncertainty grow, the predicted occupancy of a crowd covers every path, and the robot stops even though people would make room for it (Trautman and Krause 2010). Three choices reduce unnecessary conservatism, and none permits shrinking a physically justified bound merely to obtain motion:

1. *Short receding horizons:* Screening over the evidence horizon rather than seconds ahead keeps the growth term near 351 mm instead of meters. The forecast must still cover the entire executed prefix and the physical fallback, including response delay and deceleration; shortening the lease does not shorten an actor's possible motion while the robot stops.
2. *Re-centering on fresh evidence:* A qualified tracker re-centers each set on new measurements ($t_0 \leftarrow t_{\text{fresh}}$), resetting the growth term and the propagated covariance, provided its accuracy and latency support the required bound.
3. *Interaction-aware prediction:* Trautman and Krause's own remedy predicts the people and the robot jointly, so that each accounts for how the others will respond. Such predictions are learned or modeled proposals, and they may narrow a set only after their error has been bounded by the same kind of evidence the permission path demands of any other input.

The discs are centered on the coworker's last measured position, so the tracker's lifecycle sets their age. A confirmed track receives fresh measurements, while a coasting track accumulates propagated uncertainty and widens the exclusion region as its evidence ages. A goal proposal cannot refresh a stale track without incoming sensor data, and a detour proposed after a refusal needs its own timed check. When a person walks out of view, terminating the software track does not make the occluded space free. Memory already erodes certified free space behind an occlusion at the same walking-speed bound (section 9.7), and that unknown-space policy remains in force until new observations justify a different claim.

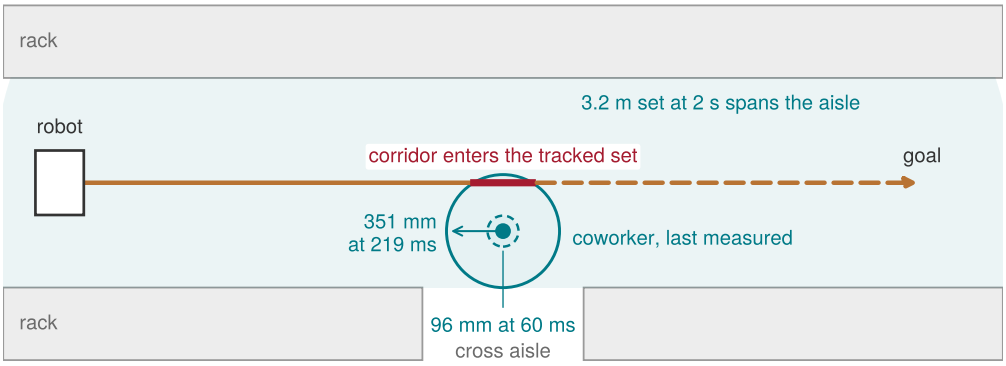


Figure 10.7: Forward occupancy sets on the aisle: A top-down view of a candidate corridor screened against a coworker's forward occupancy sets, centered on the last measured position. The discs show only the growth term $v_h \Delta t$ at the ISO 13855 speed of 1.6 m/s: 96 mm over the untracked 60 ms horizon and 351 mm over the tracked 219 ms horizon; the body envelope r_k and the tracking term $k_\sigma \sigma_{\text{track}}$ add to every radius. The corridor enters the tracked set (red segment), so the screen refuses it and asks for a detour or yield. Over a 2 s horizon the set grows to 3.2 m and spans the aisle.

> **Checkpoint 10.1: Evidence windows and reach**
>
> A target is admissible only while its evidence lasts, and every early screen in this section is sized by that window.
>
> - ☐ **Evidence-horizon estimate**: Suppose the mug drifts at 50 mm/s instead of riding the belt, with the same 15 mm tolerance and 3 mm initial error. Can you compute the untracked evidence horizon and decide whether the 2 Hz intent model could renew it without a tracker?
> - ☐ **Transit bound**: Can you explain why a terminal move inside the static workspace can still fail admission, and what a staged approach changes about it and what it cannot change?
> - ☐ **Occupancy vs. reachability**: Which question does a forward occupancy set answer that dynamic reachability does not, and why does screening over the evidence horizon rather than seconds ahead keep the robot from freezing?
> - ☐ **Stretched deadlines**: Why can neither faster processing of the same old frame nor a longer requested lease extend a target whose evidence has expired?

Reachability and corridor filters reject provably impossible requests before planning begins. The survivors remain candidates, carrying the target bounds and obstacle evidence needed for the full motion check. The remaining challenge is distinguishing between failures these physical checks can expose and errors in task meaning that geometry and kinematics alone cannot resolve.

10.7 Intent Grounding Failure Modes and Their Checks

Whether an error arises from physical drift or from task misunderstanding, a vision-language model or multimodal transformer emits the same $SE(3)$ target pose for a physical manipulator. A specular reflection in empty air, a stale coordinate from a moving belt, or a bolted fixture mistaken for a handle all arrive across the proposal boundary as the same clean seven-number pose:

1. **Phantom target (hallucination):** Attention concentrates on a specular glare, a shadow, or a transparent surface, and the pipeline predicts an object where none exists. Missing or corrupted depth returns then place the target in empty space or inside a transparent barrier, and an admitted proposal sends the arm into air or into glass.

2. **Stale target (pipeline latency or temporal drift):** The physical workpiece shifts on a conveyor, vibrates on a feeder tray, or is displaced by a human coworker while the multi-stage perception pipeline buffers, batches, and decodes frames. The machine's intent model takes 336 ms per inference, longer than the 60 ms untracked horizon before any tail latency is added. A chunked action stream can keep the arm moving throughout, but on the target the model last saw. The resulting proposal arrives downstream with mathematically valid coordinates that describe where the object *used to be* rather than where it *is*, consuming physical clearance before motion begins.
3. **Semantic ambiguity (bimodal posterior or argmax failure):** Two visually identical candidate objects (e.g., two red mugs) reside within the pick zone, and both score high because the detector scores each box independently. The calibrated prediction set holds both ($|\mathcal{C}| = 2$). Keeping only the top box or averaging the candidates' centroids commands an arbitrary choice or drives the end effector toward the empty midpoint between the objects, risking a dual-object collision.
4. **Misclassified affordance:** A bolted fixture is taken for a movable handle, or a thin-walled container for a robust one, and the requested grip force destroys it. The approach can be collision-free while the contact wrench is wrong.
5. **Dynamic unreachability (kinematic transit shortfall):** A requested terminal move is geometrically within the static workspace manifold $\mathcal{W}$, but the transit time demanded by actuator velocity and acceleration limits exceeds the remaining sensor evidence window ($t_{\text{transit}} > t_{\text{intent_exp}} - t_{\text{now,mono}}$). The terminal request must be rejected or replaced with guarded stages supported by fresh observations; extending its old evidence deadline cannot make it feasible.

All five failures arrive across the proposal boundary in the same format: a candidate position $\mathbf{p} \in \mathbb{R}^3$ and orientation $\mathbf{R} \in SO(3)$. While a pose in empty air leads to a futile reach and a pose inside a fixture leads to contact unless path and permission checks intervene, the wrong-object case can satisfy every geometric and kinematic check while targeting the wrong item. Because the independent permission path checks only what it can measure (freshness, coverage, geometry, current state, and actuator limits), it makes no claim to verify the task's meaning. **The Safety MCU enforces physical feasibility, not semantic truth. A goal can be collision-free, kinematically smooth, and torque-limited, yet wrong for the task.** Semantic verification therefore belongs to upstream task monitoring and operator oversight, and the same boundary holds under adversarial attack, where an injected instruction or a printed patch in the camera view still yields a well-formed pose.

↪ What the permission path can and cannot catch when the proposer itself is attacked is developed in section 15.6.3.

A fresh depth observation can reject a proposed grasp when required object evidence is absent inside a covered, visible region, whereas missing returns in an occluded or unobserved region leave that space unknown. On this station, the singulation rule goes further by requiring independently qualified depth evidence of a single candidate in the pick window, even when the grounding model reports high confidence. Because the Safety MCU holds no scene representation with which to count objects, this check runs in a separate station component before any proposal is forwarded. The component routes any window with multiple candidates to a coworker for manual selection (a scoped station policy; another qualified operating mode could select a uniquely identified mug among several objects). Counting itself depends on validated sensor coverage and segmentation, so it shares a failure mode with phantom targets. Touching items can merge into a single cluster, and transparent objects such as the glass jar return little or no depth (Sajjan et al. 2020). The counter's qualified domain must therefore exclude those cases or route them to a manual pick. Even when the count finds a single cluster, it still cannot verify that the lone object is a mug, and while the producer's calibrated ambiguity filter may refuse earlier, it cannot replace the station's independent check.

Table 10.5 groups these failures by the check that can expose them. Dynamic transit insufficiency can be rejected before motion, whereas a misclassified affordance may first become observable through contact residuals. The table separates admission checks from responses after motion or contact has begun. At the cage-door latch of notebook 10.1, the 2 ms contact-loop response already leaves 39 N at brake-command onset, and the peak comes later. Precontact speed and force limits,

actuator response, compliance, and available clearance therefore determine whether the full contact trace stays within the validated contact limits.

Table 10.5: Grounding failure modes and their evidence limits: Each check addresses a different failure class. The table separates proposal rejection from a response to an already moving plant.

Failure mode	Evidence or check	Admission or response	Physical limit to verify
Phantom target: image artifact in empty space	Fresh depth evidence and target-region occupancy, with coverage and occlusion checked	Reject the proposal when the required object evidence is absent; request another view if the region is unknown	Rejection precedes motion only if the proposal has not already been admitted. Missing returns alone do not prove empty space.
Stale target: workpiece moved after capture	Converted capture time, bounded clock error, local tracking, and target-error envelope	Revoke dependent tracking when the evidence limit fails; select the validated fallback	Total detection, communication, actuation, and braking delay must fit the measured clearance.
Station singulation failure: multiple candidates in the pick window	Independent count of candidates in the pick window from fresh depth evidence; calibrated prediction-set size as an earlier producer-side signal	Refuse admission and route the item to the coworker for a manual pick	A single wrong object passes the count, and a confident model can return a singleton set; independent physical checks remain necessary.
Misclassified affordance: bolted fixture selected as graspable	Contact-force and motion residuals after contact; precontact task/geometry checks where possible	Limit precontact speed and force, then invoke the validated contact response on a threshold crossing	A reactive trip does not cap the first impact peak; test the full force trace, response delay, and stopping energy.
Dynamic unreachability: target cannot be reached in its valid window	Speed-ceiling transit lower bound followed by a complete path and stopping check	Reject terminal completion on that record; consider separately admitted guarded segments	Passing the lower bound proves neither collision-free motion nor a feasible stop.

Every row of the table therefore requires the proposal to carry its spatial bounds, the provenance of its evidence, and its expiry in a form the permission path can check without the model. A rejected proposal returns a diagnostic reason to the learned layer without revoking an earlier active record, which remains usable while its own state, evidence, lease, and fallback hold. An admitted proposal stays subject to independent permission checks as the plant moves, and the record the permission path grants from it is the intent lease.

10.8 The Intent Lease Specification and Contract

If a target region, an actuator effort ceiling, and an expiration deadline travel as separate messages across the inter-core bus, a dropped or delayed packet can leave the trajectory planner tracking a new pose under an old force limit or an expired evidence timestamp. For the red mug, upstream perception and belief filtering distill raw sensor telemetry into those three quantities: a target region bounded by empirical evidence, an effort ceiling on actuator output, and an explicit deadline after which tracking must cease. The intent lease binds them into a single, immutable record for downstream control. Its lineage anchors to the belief record through a SHA-256 digest, while its evidence timestamp maps that record's sensor transduction epoch directly into the Safety MCU's monotonic clock domain. This admitted target establishes a verified envelope for downstream planning, leaving trajectory generation and execution permission to subsequent safety stages.

This stage emits the **intent lease**, the eighth record in the book's sixteen-record chain. Authority divides cleanly between the two processors. The high-performance Application Processor generates an untrusted intent proposal (table 10.2), while the independent, real-time Safety MCU validates and latches the admitted lease. By binding the admitted target to a verified goal set, an affordance frame, and a collection of active safety predicates, this latched lease strictly constrains the downstream trajectory planner before the hardware admits any joint motion.

To guarantee deterministic processing on the safety controller, the proposal payload uses a statically bounded schema. Fixed-size fields eliminate dynamic heap allocation and pointer dereferencing, enabling the validation pipeline to reject malformed records in bounded execution cycles. However, static memory layout alone does not guarantee real-time deadlines. Inter-core memory copies,

cache maintenance, lock contention, and bus arbitration must all remain within measured worst-case execution time bounds. The inbound proposal remains an advisory request from untrusted software; the Safety MCU retains exclusive authority over parameter bounds, expiration deadlines, and fallback activation.

↪ Mailboxes, memory barriers, and copy costs on the inter-core seam are developed in section C.8.

The covariance ellipsoid derived for the mug handle in section 10.3.3 represents an upstream producer estimate. Rather than serializing a dense, oriented 3×3 covariance matrix across the real-time wire, the protocol projects these bounds into three axis-aligned **position-error envelopes** (e_x, e_y, e_z) alongside three requested **task tolerances** (r_x, r_y, r_z) expressed in the robot base frame $\mathcal{F}_{\text{base}}$. The componentwise test $0 \le e_i \le r_i$ is a necessary preliminary screen. Admission must also establish that the full conservative error set, together with reserved controller tracking error, fits inside the ellipsoidal task region defined in section 10.1. Passing each axis independently does not establish that containment. Three axis-aligned bounds define a rectangular box rather than an ellipsoid, so a corner where errors peak along all three axes simultaneously can protrude outside the curved task boundary even when each individual axis stays within its limit, requiring the Safety MCU to evaluate the ellipsoidal norm at the worst-case corner of that error box plus the reserved tracking margin. The upstream producer must mathematically justify its projection from raw state covariance to these conservative error envelopes, accounting for camera calibration drift, latency jitter, and declared tail-risk percentiles. Angular tolerances and contact-mode transition rules derive from an independently certified task profile, allowing the wire proposal to transmit only the nominal target orientation quaternion.

Temporal validation hinges on the **belief record's evidence epoch** (the physical instant of sensor exposure, not the time processing completed), mapped into the Safety MCU's local clock domain. The Safety MCU verifies the parent digest and evidence epoch against retained belief records in local memory, adding the bounded synchronization error to the calculated evidence age. Requested expiration times reference this same monotonic hardware clock. Here, the parent digest establishes **data lineage rather than cryptographic authentication**, enabling forensic analysis to trace an actuator command directly back to the originating camera frame. Because the safety controller gates proposals exclusively on physical invariant checks and kinematic feasibility, the wire format omits cryptographic signatures, prioritizing physical truth over sender identity.

↩ Converting a producer's evidence epoch into the Safety MCU's monotonic clock, with a bounded synchronization error, is established in section 4.4.1.

An intent lease progresses through five discrete lifecycle states on the Safety MCU: `Pending`, `Active`, `Renewed`, `Expired`, and `Aborted`. The controller transitions a proposal from `Pending` to `Active` only after verifying all geometric and dynamic ceilings and confirming that a collision-free stopping path remains executable from the current state. Transitioning to `Renewed` requires a strictly monotonic sequence counter, a newer parent belief record with an updated sensor capture epoch, and a fully passed admission re-check. A complex approach trajectory can span multiple consecutive lease renewals as long as fresh perception evidence continues to justify the motion. Dropping an isolated camera frame does not immediately abort tracking if the current coasting window remains valid. However, absent renewal, the hardware timer continues counting toward expiration. When an anomaly, force limit breach, or evidence timeout revokes tracking authority, the controller immediately engages a pre-calculated deceleration or hold trajectory to manage the kinetic energy of the moving manipulator safely.

Figure 10.8 draws the same machine. Its one subtle transition is the self-loop on a stale renewal. A packet with an old sequence number or an old capture is discarded without resetting the timer, so a host that keeps talking without new evidence still runs out of authority. Expiry and abort end in the same place, the validated fallback that the MCU selects for the measured state.

Definition 10.3: Expiring intent lease

Expiring intent lease ($\mathcal{J}_{\text{intent}}$) is a time-bounded target authorization issued by the independent permission path that binds an admitted target pose and effort ceilings to a monotonic hardware deadline $t_{\text{intent_exp}}$:

$$t_{\text{intent_exp}} = \min(t_0 + \tau_{\text{ev}},\ t_{\text{adm}} + T_{\text{admin}},\ t_{\text{deadline,stop}}) \tag{10.9}$$

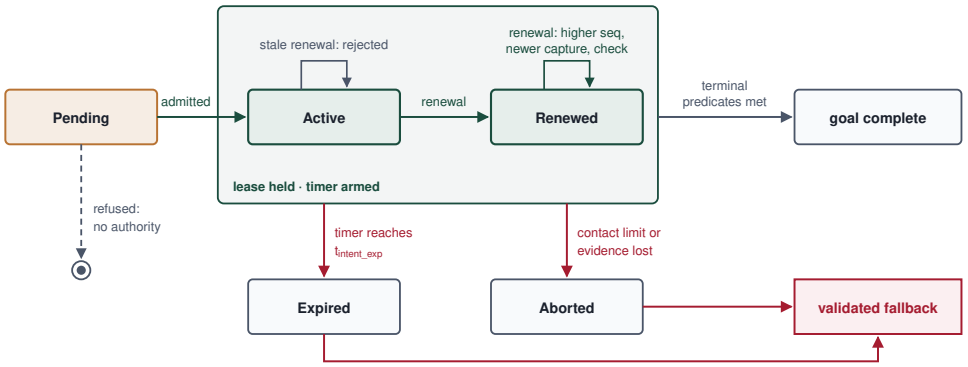


Figure 10.8: Intent lease lifecycle: A proposal waits in `Pending` until admission either refuses it, granting no authority, or arms the lease. While the lease is held, only a renewal with a higher sequence number, a newer capture, and a passing permission check moves it to `Renewed` or keeps it there; a stale renewal is rejected without resetting the timer. The held lease ends in one of three ways: the terminal predicates complete the goal, the timer reaches $t_{\text{intent_exp}}$, or a contact limit or evidence loss aborts it. Expiry and abort both start the validated fallback, which the MCU selects for the measured state.

upon whose expiration dependent target-tracking authority is automatically revoked without requiring an explicit cancellation packet. The lease establishes geometric targets and effort bounds for downstream trajectory generation, reserving path execution authority for independent trajectory-level validation.

1. **Significance**: Revokes tracking authority autonomously on a hardware deadline whenever upstream deliberation stalls, engaging a validated fallback response without depending on cooperative shutdown signals from the host processor.
2. **Distinction**: Unlike a standard network keepalive or software heartbeat ping, an intent lease physically couples the temporal validity of sensor evidence (τ_{ev}) with real-time braking feasibility, capping lease duration so that a validated stopping maneuver always fits within the verified spatial clearance.
3. **Common pitfall**: Artificially inflating lease durations to mask tail latency (p99) in vision models or neural planners. Generous timeouts create dangerous open-loop blind spots where the robot sweeps through physical space after environmental conditions have already evolved.

The three terms in the minimization enforce distinct safety bounds that prevent different classes of failure across the cyber-physical stack:

- The **physical target validity horizon** ($t_0 + \tau_{\text{ev}}$) marks the instant on the monotonic clock when unobserved environmental motion or state drift can exhaust the target-error budget remaining within task tolerance r_{tol}. This term dates strictly from physical sensor capture rather than packet arrival, where τ_{ev} is the admitted target horizon after reserving controller tracking error, bounded above by the upstream belief record's evidence-only horizon (equation 10.5). Once this horizon elapses without fresh perception updates, the robot would be tracking an ungrounded hallucination, regardless of whether host software runs without error.
- The **software administrative lease** ($t_{\text{adm}} + T_{\text{admin}}$) establishes an upper bound, measured from the admission tick t_{adm}, on how long the Application Processor may retain active tracking authority before the Safety MCU presumes host failure. Even in static environments where objects do not move ($\tau_{\text{ev}} \to \infty$), operating system scheduling jitter, garbage-collection stalls, memory bus starvation, or kernel panics can freeze the host pipeline. The administrative lease T_{admin} forces the system to decelerate smoothly rather than fly blind. This administrative horizon differs from the trajectory chunk lease T_{lease} of section 4.4: while a chunk lease governs

the execution duration of an individual trajectory polynomial, T_{admin} bounds the persistence of the macro-level target goal itself.

- The **physical stopping boundary** ($t_{\text{deadline,stop}}$) dictates the absolute latest revocation instant from which a worst-case braking profile can still arrest motion within remaining swept clearance. Given a verified swept-volume clearance D_{clear} and a dynamic stopping distance $d_{\text{stop}} = v\tau_{\text{rev}} + v^2/(2a_{\text{brake}})$, the deadline evaluates to $t_{\text{deadline,stop}} = t_{\text{adm}} + (D_{\text{clear}} - d_{\text{stop}})/v$. Here τ_{rev} is the post-revocation delay, the tick, bus, and brake-onset terms that make up the 22 ms stop onset. It is the tail of the full pre-brake delay τ_{delay} of section 1.4.3, not a replacement for it. The other terms of τ_{delay}, observation age and the lease, are exactly what the deadline itself measures, so charging them again here would count them twice. This kinematic bound assumes positive velocity ($v > 0$) and nominal braking deceleration. When the manipulator is stationary ($v = 0$), a static hold assessment replaces the distance formulation. Any dynamic change in tool speed, detected clearance, or actuator thermal limits requires immediate recomputation.

By taking the minimum across all three terms on the Safety MCU's local hardware timer, motion authority terminates at whichever safety margin exhausts first: sensor evidence staleness, host processor silence, or kinematic clearance depletion. Upon expiration, whatever the state of the host processor, a hardware timer interrupt revokes tracking authority and triggers the fallback response the MCU selected for the current payload, kinematics, and contact mode. The choice depends on physical context. A powered position hold requires continuous motor torque but stabilizes a suspended payload. A controlled path deceleration bleeds kinetic energy within the clearance boundaries. Safe torque disable may protect the windings of compliant backdrivable joints but can drop a suspended load or leave momentum uncontrolled. Any terminal mode requested in the original proposal remains purely advisory. For instance, an upstream policy requesting torque disable while suspending a heavy payload cannot override the powered hold mandated by the safety controller. If no fallback can guarantee safety from the current state, admission rejects the lease before motion begins. Neither a 1 kHz discrete tick nor a hardware interrupt can arrest physical mass instantaneously. Mechanical deceleration is governed by inertia and actuator torque limits, so the clearance it needs must be reserved before any motion is admitted.

The admission pipeline enforces the four handoff invariants of section 10.4 as a staged series of validation gates, ordered from lowest computational cost to highest so that execution time stays bounded:

1. **Evidence and sequence validation:** Ingest the proposal record from the IPC ring buffer. Instantly drop packets with obsolete sequence numbers or mismatched schema versions. Compare the `parent_belief_digest` and sensor epoch against locally retained spatial memory records. Transform the evidence timestamp into the MCU monotonic clock domain, incorporate the worst-case clock synchronization uncertainty, and reject the request if the resulting evidence age already exceeds the evidence horizon τ_{ev}.
2. **Kinematic and tolerance bounds:** Verify that all pose coordinates are finite numbers and that the orientation quaternion is strictly normalized ($\|\mathbf{q}_{\text{quat}}\| = 1.0 \pm 10^{-4}$). Confirm that error envelopes and requested task tolerances satisfy $0 \leq e_i \leq r_i$ along all coordinate axes in $\mathcal{F}_{\text{base}}$. Then verify that the full conservative error set, together with reserved controller tracking error, fits inside the ellipsoidal task region. Validate requested wrench thresholds against static actuator ratings and workplace occupancy constraints.
3. **Dynamic feasibility and fallback clearance:** Evaluate current joint velocities, payload inertia, and sensor-verified spatial clearance to confirm that a valid stopping trajectory exists. Reject any proposal where the sum of reaction delay travel and braking distance exceeds available swept clearance ($d_{\text{stop}} > D_{\text{clear}}$). The MCU does not count candidates in the pick window, because it holds no scene representation. The station's singulation check runs before a proposal is forwarded (section 10.7), and a window it has not cleared never reaches this stage. This stage admits target geometry and effort bounds rather than a path. The trajectory planner proposes the full continuous trajectory, which the permission path must separately admit.
4. **Authority latch:** Commit the verified target pose, admitted wrench ceilings, validated expiration deadline ($t_{\text{intent_exp}}$), and pre-selected fallback policy into protected MCU memory. Arm

the hardware countdown timer for the admitted interval. Advisory values from the untrusted proposal cannot write to drive registers.

5. **Runtime watchdog monitoring:** On every high-frequency control cycle and hardware interrupt, compare measured joint states, motor currents, and contact forces against the admitted lease bounds. If the hardware timer expires or force limits are breached, immediately revoke tracking authority and engage the latched fallback routine. A missed renewal does not revoke authority by itself. It leaves the timer running toward expiry.

Stage 1 checks temporal freshness, stage 2 checks spatial containment and the effort ceilings, stage 3 checks the certified fallback, and stages 4 and 5 latch what passed and keep checking it while the plant moves. Together, these five gates emit the admitted target record consumed by the trajectory planner (table 10.6), establishing the formal contract boundary between spatial perception and trajectory planning.

Table 10.6: Intent lease schema: Data types, physical semantics, and invariant checks for the admitted target emitted by the Safety MCU and consumed by trajectory planning.

Field	Representation	Physical semantics	Invariant and consumer check
`magic` & `version`	`uint32_t[2]`	Magic bytes (`0x494E5453`, 'INTS') and contract revision	Statically reject mismatched schema revisions across inter-core boundary
`parent_belief_digest`	`uint8_t[32]` (SHA-256)	Digest of the upstream belief record	Establishes custody chain; matches retained belief record in memory
`intent_digest`	`uint8_t[32]` (SHA-256)	Digest over all admitted fields in this record	Emitted to the trajectory record to bind downstream planning
`evidence_epoch_ns`	`uint64_t` (ns)	Physical measurement epoch t_0 from sensor transduction	Evaluated on MCU clock; upper age must not exceed τ_{ev}
`admission_epoch_ns`	`uint64_t` (ns)	Monotonic hardware timestamp when proposal was admitted	Baseline epoch for administrative lease duration T_{admin}
`expiration_deadline_ns`	`uint64_t` (ns)	Absolute hardware deadline $t_{\text{intent_exp}} = \min(t_0 + \tau_{\text{ev}}, t_{\text{adm}} + T_{\text{admin}}, t_{\text{deadline,stop}})$	Countdown timer revokes tracking authority upon expiry (equation 10.9)
`frame_ids`	`uint32_t[2]`	Base frame $\mathcal{F}_{\text{base}}$ and target affordance frame $\mathcal{F}_{\text{affordance}}$	Verified against active kinematic tree; prevents frame confusion
`target_pose_se3`	`double[7]`	Admitted nominal pose $[x, p_y, z, q_x, q_y, q_z, q_w]$ in $\mathcal{F}_{\text{base}}$	Target reachable within $\mathcal{W}$; full swept path checked downstream
`goal_region`	`double[3]` (m), `double` (rad)	Spatial semi-axes (r_x, r_y, r_z) and angular tolerance θ_{tol}	Defines $SE(3)$ goal set $\mathcal{G}_{\text{goal}}$ (equation 10.1); terminal boundary
`dynamic_limits`	`double[2]` (m/s, m/s^2)	Admitted TCP speed ceiling $v_{\max}$ and acceleration $a_{\max}$	Evaluated against thermal ratings; bounds transit lower bound (equation 10.6)
`max_wrench`	`double[6]` (N, N·m)	Admitted wrench ceiling $[F_x, F_y, F_z, \tau_x, \tau_y, \tau_z]$	Low-level impedance bounds; guards against crushing fragile items
`validated_predicates`	`uint32_t` (bitmask)	Bitmask of verified predicates $(P_{\text{approach}}, P_{\text{contact}}, P_{\text{clear}})$	Planner applies each predicate in its task-specified phase; checks full-path clearance
`fallback_policy_id`	`uint32_t` (enum)	Enforced terminal response (`HOLD`, `STOP`, `TORQUE_OFF`)	MCU-selected deterministic safe state matched to measured load and contact

By linking `parent_belief_digest` directly to the upstream belief record and emitting its own `intent_digest` to the downstream trajectory record, the intent lease schema (table 10.6) creates an unbroken, auditable custody chain extending from camera photons to actuator torque commands. Downstream trajectory planners can neither expand the admitted geometric goal set $\mathcal{G}_{\text{goal}}$ (equation 10.1) nor prolong tracking authority beyond `expiration_deadline_ns`.

↪ The trajectory record that embeds the lease's digest and answers to its bounds is developed in section 11.8.

Filled in for the red mug, the admitted lease links directly to the mug-handle belief record by its digest and sensor capture epoch. The target pose designates the 6-DoF mug-handle affordance

frame, where localization uncertainty is bounded by the belief record's 3 mm initial error envelope, well within the 15 mm task tolerance. The evidence horizon marks the temporal threshold where unobserved conveyor belt motion causes error growth to intersect this tolerance: 60 ms under open-loop conveyor drift, extending to 219 ms when tracking filters provide qualified updates under residual slip. Kinematically, the lease constrains the manipulator to a 1 m/s tool-center-point speed ceiling. Because the facility catalog registers no fragile payload profile for this mug, the wrench ceiling is the default profile's, and the controller enforces default joint impedance limits, relying on plant-level sorting rather than online compliance adaptation. A cataloged fragile payload would instead carry an item-specific ceiling, bounded below by the squeeze force that holds the payload through transit (section B.7.5) and above by its crush limit. Grasping and subsequent handover constitute the target objective, while the MCU retains exclusive authority over the fallback trajectory.

The lease expires at the earliest of the three terms of equation 10.9, and the mug record supplies a value for each. The scenario chooses an administrative lease of 100 ms, long enough that one dropped wrist-camera frame does not revoke authority, and a verified clearance of 300 mm ahead of the gripper. The stopping term needs a deceleration the arm can be relied on to deliver, which is not the controller's command limit. The arm's handbook bounds its worst-case stop at 552 ms with the full payload at maximum velocity, measured under the stopping test of EN ISO 10218-1 (Franka Emika GmbH 2021). The handbook states only "maximum velocity" and reports the stop in joint angles, so assuming that stop began at the 1.7 m/s Cartesian limit gives a derived credible deceleration of 3.1 m/s^2. At the 1 m/s ceiling, that deceleration and the 22 ms stop onset give a stopping distance of 184 mm, so the stopping deadline falls 116 ms after admission. With all three countdowns started together, as in figure 10.5, the 60 ms evidence horizon binds first when the mug is untracked. A tracker stretches the evidence horizon to 219 ms, and the 100 ms administrative lease then binds before the 116 ms stopping deadline. Subtracting the controller tracking reserve, which a deployment must still do, can only shorten the admitted target horizon and bring expiry earlier.

This explicit record structure establishes a concrete boundary for automated fault-injection suites. Systems engineers validate the safety controller by injecting unnormalized quaternions, stale capture timestamps, excessive wrench thresholds, and conflicting fallback requests into the IPC stream, verifying that the Safety MCU deterministically rejects invalid proposals across all operational states and timing margins.

↪ Negating each record's guarantees, intent lifetime among them, at its handoff to derive injected faults is developed in section 15.6.1.

Checkpoint 10.2: Expiring intent and fallback authority

Verify your understanding of how a proposal is refused or admitted, and how the lease granted from it is sized and revoked:

- ☐ **Ambiguity refusal**: Why does this station's singulation rule refuse a pick window holding two candidates, and why must that test run on a confident proposal as well as an uncertain one?
- ☐ **Wrong-object premise**: Which premise about the grasped object remains outside the permission path even when every coordinate, reachability, and force check passes, and where does the machine carry that premise instead?
- ☐ **Expiry bound**: Can you name which term of equation 10.9 only fresh evidence can extend, which one a silent host triggers, and which one a change in speed or clearance forces the MCU to recompute?
- ☐ **Clock domain**: Why must the MCU convert the belief record's evidence epoch into its own monotonic clock, with a bounded conversion error, before checking age and the requested expiry?
- ☐ **Fallback selection**: Can you explain why the MCU must select a feasible approved hold, deceleration, or torque-disable response from the measured load state rather than obey the proposal's requested terminal mode?

10.9 Fallacies and Pitfalls

Every failure pattern cataloged here stems from trusting an upstream message to mean more than it physically certifies. Real-time safety interlocks and local execution checks must refuse a proposed target, or revoke tracking authority over an active one, whenever its underlying sensory evidence expires or its spatial justification dissolves. Downstream execution monitors must evaluate evidence that the upstream proposer cannot falsify, enforcing safety invariants even while incoming inference streams remain syntactically valid and numerically confident.

Fallacy: *A renewal computed from the same stale frame is a valid renewal.*

In distributed computing and cloud infrastructure, lease renewals often function as software heartbeats. A client process demonstrates liveness and retains exclusive access by transmitting an uncorrupted keepalive packet before a timeout expires. In physical AI, however, software liveness implies nothing about physical evidence validity. If an upstream planner loops cleanly while the physical scene changes, keepalives succeed while the robot drifts blind.

Consider an intent model that repeatedly processes its last wrist-camera frame of a red mug, dispatching renewal requests with incremented sequence numbers, fresh issue timestamps, and identical target poses. Across the network, every field that a keepalive inspects appears healthy and up to date. No new photons, however, have reached the camera sensor, while the physical mug continues moving down the conveyor at 0.20 m/s. Because a sound intent lease dates its authority from the parent belief record's sensor capture epoch t_0 rather than packet arrival, the Safety MCU observes that the evidence epoch has stalled. As a result, it expires the target 60 ms after the original camera capture. A naive controller that evaluated renewals based on packet arrival time would permit a spinning software loop to retain tracking authority over an obsolete coordinate indefinitely. **Advancing software timestamps on stale sensory data creates an illusion of liveness while unobserved physical drift violates clearance.**

Architect's countermeasure: Anchor lease validity strictly to the sensor evidence epoch t_0 of the parent belief record; reject renewals that fail to supply a qualified newer capture record, including its error bound and bounded clock conversion.

Pitfall: *Allowing an unexpired temporal lease to continue commanding motion when target spatial bounds are violated.*

In distributed locking protocols, a lease grants exclusive, unconditional authority over a resource for the entire duration of its temporal grant. In physical AI, however, an intent lease is a coupled spatio-temporal contract:

$$\mathcal{J}_{\text{intent}} = \left(\mathcal{G}_{\text{goal}},\ [t_{\text{adm}}, t_{\text{intent_exp}}],\ \mathbf{w}_{\max}\right)$$

Temporal freshness ($t < t_{\text{intent_exp}}$) is a necessary condition for motion execution, but never a sufficient one.

Suppose a manipulator arm lowers toward the red mug under an admitted lease with a grasp tolerance of 15 mm. If an unexpected belt slip shifts the mug beyond that tolerance while time remains on the countdown, the lease is still temporally active, but the target has left the spatial goal region $\mathcal{G}_{\text{goal}}$ that justified the approach. Continuing until the countdown timer expires drives the arm toward an empty coordinate or provokes a collision. The runtime permission loop must continually compare the latest qualified target evidence and its age-dependent uncertainty bound against admitted limits. It revokes tracking authority the moment an update shows that the handle has left the goal set, $\|\mathbf{p}_{\text{target}}(t) - \hat{\mathbf{p}}_{\text{target}}\|_2 > r_{\text{tol}}$, where $\hat{\mathbf{p}}_{\text{target}}$ is the admitted target. Under a qualified tracker that target follows the tracked affordance frame along the belt, so the test catches slip beyond the tracker's prediction rather than the belt motion the tracker already cancels. Running the control loop faster cannot reveal an unobserved slip; sensing delay and bounded error growth must both be accounted for in the response budget. **A temporal lease confers authority only within declared spatial bounds; temporal freshness never excuses spatial divergence.**

Architect's countermeasure: Embed runtime spatial containment checks ($E(\Delta t) \le r_{\text{tol}}$) in high-rate control loops, revoking dependent authority when qualified evidence or its propagated bound no longer supports the tolerance mid-lease.

Fallacy: *A confident grounding needs no refusal path.*

In machine learning pipelines, a high detector score is often treated as definitive proof of identity. Engineers transitioning to physical robotics are frequently tempted to save latency with shortcut logic: `if confidence > threshold: skip_ambiguity_checks()`. That shortcut treats a model's score as an audit of the physical scene, and it fails across three distinct systems dimensions:

1. **Scores are not calibrated probabilities:** Neural networks trained with cross-entropy loss are often overconfident (Guo et al. 2017), and calibration fitted on one training distribution degrades under distribution shift (section A.2.3). An open-world detector can assign a high score to a transparent glass jar or a printed decal because it represents the closest feature match to the prompt.
2. **Per-box scores do not measure uniqueness:** Open-vocabulary detectors score each bounding box independently against the text prompt without normalizing across candidate proposals (Minderer et al. 2022; S. Liu et al. 2023). If two identical red mugs sit in the pick window, both score high. Downstream logic that naively keeps only the top-scoring box turns two valid candidates into an arbitrary choice, concealing real spatial ambiguity from all subsequent planning layers.
3. **The consequence is physical:** A score describes prompt alignment in representation space rather than confirming that the physical workspace contains exactly one manipulable entity. The two jar cases of section 10.3 mark the line. When the jar sits beside the mug, both enter the prediction set and refusal is mandatory however high either score is. When the jar stands in place of the mug, a confident singleton passes every check, which no score threshold can catch.

Workstation safety requires an independent singulation check on every intent proposal, drawing on evidence that the vision-language model cannot manipulate. An independent candidate-counting stage operating directly on calibrated spatial point clouds or voxel occupancy grids evaluates the physical workspace. If the pick window contains more than one candidate cluster, the station refuses the proposal with `ERR_GROUNDING_AMBIGUITY` before it reaches the Safety MCU, regardless of model confidence. Any earlier active record remains governed by its own evidence epoch, lease duration, and fallback strategy. Transitioning to a safe hold or operator handoff requires an admitted, physically feasible trajectory. **A detector score reflects the model's match to the prompt, not real-world uniqueness; keeping only the top box can select the wrong physical object.**

Architect's countermeasure: Enforce the selected station's singulation rule using qualified independent candidate evidence; treat `ERR_GROUNDING_AMBIGUITY` as a first-class diagnostic that can request a feasible hold or operator handoff without automatically revoking a still-valid active record.

Fallacy: *Neural networks can resolve bimodal spatial ambiguity by averaging candidate target locations.*

A continuous regressor trained with squared error converges toward the conditional mean of its targets (section 6.2.3). When demonstrations pick mug A half the time and mug B the other half, that mean settles on the midpoint $(\mathbf{y}_A + \mathbf{y}_B)/2$, which lies on the empty conveyor belt between two valid targets or directly on whatever obstacle separates them. Because the robot's collision-free operational space is not convex, the linear average of two feasible goals need not be feasible, and the gripper closes on empty air or collides with the inner edges of both mugs.

Suppose the perception system observes two identical mugs a few centimeters apart and submits their spatial mean as a grasp target. The fused estimate can appear deceptively sharp. Both individual detections may pass association and innovation gates (section 9.2.2), while an averaging estimator reports an artificially low variance that evades covariance expansion gates. However, the independent pick-window geometric check from section 10.7 still detects the presence of two distinct candidates, prompting the station's admission gate to reject the proposal regardless of what the fused estimate reports. That refusal does not revoke an active, unexpired lease; transitioning to a safe hold must itself remain physically feasible. Generative policies that explicitly represent multiple modes, such as diffusion policies, flow-matching models, or Gaussian mixture models (section A.5.1), avoid arithmetic averaging, but downstream execution contracts cannot rely on policy internals. Before crossing the intent boundary, the proposer must commit to a single goal set anchored to a single

tracked physical entity. **Averaging bimodal physical targets can replace two valid manipulation candidates with a location that satisfies neither task.**

Architect's countermeasure: Preserve explicit multi-hypothesis distributions at the reasoning boundary (via diffusion policies or Gaussian mixture models), and enforce discrete candidate selection backed by the independent pick-window ambiguity gate.

10.10 Summary

Grounded intent transforms an unverified proposal from a learned policy into a bounded, expiring physical target. In this architecture, a foundation model or vision-language-action (VLA) policy proposes a target pose in $SE(3)$, a spatial error envelope, and effort ceilings. The Safety MCU operates as an independent gatekeeper, evaluating the proposal against kinematic and physical limits before admitting it as an *intent lease*. This lease anchors directly to a parent belief record in spatial memory and remains valid only until the earliest of three deterministic deadlines: evidence decay (τ_{ev}, derived from the belief record after reserving controller tracking error), host silence (T_{admin}, guarding against communication stalls), and stopping clearance ($t_{\text{deadline,stop}}$, reserving the space needed to brake to rest). A hardware countdown timer enforces this boundary, revoking trajectory tracking authority without waiting for host confirmation. Because deceleration takes time, the worst-case response latency must be budgeted directly into the stopping clearance. Effort ceilings enforce mechanical safety alongside temporal deadlines. For example, when closing a cage-door latch, contact force continues to rise throughout the response delay of the contact-control loop. Stopping criteria must therefore account for the force already accumulating when the brake command issues, rather than assuming an instantaneous tripwire stop.

Tracking a red mug on a moving conveyor illustrates the multi-rate timing constraints that physical AI systems must reconcile. Under open-loop motion, target evidence remains valid for only 60 ms before conveyor drift carries the object outside the grasp tolerance. Meanwhile, the deliberative intent model executes at 2 Hz, updating its inferences every 500 ms. Because this deliberative period exceeds the evidence lifespan, the high-level policy cannot maintain an accurate target on its own. A qualified real-time tracker resolves this discrepancy by assimilating high-rate sensor observations to extend the evidence horizon to 219 ms, and its fresh captures, not the model's proposals, renew the lease on a goal set that follows the tracked handle. The deliberative model only reconfirms object identity at its slower cadence, so the arm need not hesitate, slow down, or abort between inferences. Guided by verified spatial corridors, continuous observation, and qualified tracking, the arm completes extended approach maneuvers across sequentially admitted segments, culminating in a grasp aligned with real-time state estimates. When a lease expires, the controller executes a stopping trajectory pre-validated for the current kinematic state, safely dissipating the mechanical momentum already present in the moving links.

In safe embodied architectures, refusal is a first-class operational outcome rather than an exceptional failure. When an instruction points to an ambiguous referent, the candidate set fails the calibrated prediction threshold and the station's singulation rule. When a proposed terminal motion cannot reach the target before evidence decays, the admission controller rejects the transit bound. Similarly, when a planned motion corridor intersects a coworker's forward occupancy set, the early screening filter denies permission. Screening the admitted prefix and its complete stopping fallback, rather than a speculative multi-second route, can reduce unnecessary refusal in narrow, shared workspaces. The screen must still cover response delay and deceleration after evidence expiry. Each refusal generates a structured diagnostic record that informs the proposer why the lease was denied, enabling informed re-planning. Symbolic task plans generated by language models integrate through this exact mechanism, mapping symbolic operators into physical preconditions and testable motion predicates. Nevertheless, physical admission gates cannot see past the wrong-object premise, because a model that confidently grounds an action to the incorrect object can satisfy every coordinate, reachability, and contact-force check. Validating that semantic premise requires task-level perceptual evidence and supervisory human oversight, while table 10.7 collects the equations that carry the physical decisions, what each bounds or decides, and where each is derived.

Table 10.7: Key equations of grounded intent: Each load-bearing equation of the chapter, the limit it bounds or the decision it settles, and the equation where it is derived.

Equation	What it bounds or decides	Derived in
$\lVert \mathbf{p} - \mathbf{p}_{\text{target}} \rVert_{\mathbf{E}} \le 1,\ \angle(\mathbf{R}, \mathbf{R}_{\text{target}}) \le \theta_{\text{tol}}$	Whether terminal pose tolerance is satisfied; task completion also requires its terminal predicates	equation 10.1
$\mathcal{C} = \{\, i : c_i \ge 1 - \hat{q}_{1-\epsilon} \,\}$	The calibrated candidate set: a singleton proceeds to remaining checks; a larger set triggers refusal	equation 10.3
$\tau_{\text{ev,untracked}} = (r_{\text{tol}} - e_0)/v_{\text{drift}};$ $\tau_{\text{ev,tracked}} = \sqrt{2(r_{\text{tol}} - e_0)/a_{\text{slip}}}$	How long target evidence stays inside the task tolerance, untracked and under a tracker	equation 10.5
$d/v_{\max} \le t_{\text{intent_exp}} - t_{\text{now,mono}}$	Whether a terminal move can finish on its current evidence; a necessary condition only	equation 10.6
$\lVert \mathbf{p} - \hat{\mathbf{p}}_k \rVert \le r_k + v_h \Delta t + k_\sigma \sigma_{\text{track}}(\Delta t)$	Where a moving person could be, and so which corridors the early screen refuses	equation 10.7
$t_{\text{intent_exp}} = \min(t_0 + \tau_{\text{ev}},\ t_{\text{adm}} + T_{\text{admin}},\ t_{\text{deadline,stop}})$	When target-tracking authority lapses: evidence decay, host silence, or stopping clearance, whichever comes first	equation 10.9

Key Takeaways: Grounded intent at the proposal boundary

- **Leases grant bounded targets, not trajectories**: The host proposes a target pose, evidence epoch, error envelope, task tolerances, and effort ceilings. The lease the Safety MCU latches binds them to a goal set and motion predicates that the trajectory planner may not expand, and it leaves the path to the planner.
- **Expiry runs on the permission side's clock**: A hardware countdown ends authority at the earliest of the evidence horizon, the administrative lease, and the stopping deadline, so a hung host loses its target without sending anything. A renewal counts only when it carries a newer capture; a repeated frame with a fresh sequence number renews nothing.
- **Target evidence decays with unobserved drift**: At conveyor speed the mug's evidence lasts 60 ms while the current intent inference takes 336 ms, so this model cannot renew the target without another source of fresh evidence. Only fresh observations or a qualified tracker, which stretches the horizon to 219 ms, extend validity.
- **Evidence deadlines differ from braking deadlines**: Expiry is not rest. After the lease lapses the arm still needs 22 ms before braking begins and then its stopping distance, so the lease charges the stopping deadline as its own term rather than assuming the evidence horizon covers it.
- **Geometric feasibility does not guarantee semantic correctness**: A learned model can ground an action to the wrong physical object with high confidence and still satisfy every coordinate, reachability, and contact-force check. Because physical admission gates cannot evaluate semantic validity, task-level verification requires independent task evidence and operator oversight.
- **Refusal is a primary operational outcome**: Ambiguous object identity, expired evidence, dynamic reachability violations, and stopping distance constraints all trigger deterministic proposal rejections. Structured diagnostics let the proposer gather more evidence, choose a staged approach, or request operator assistance.

When a learned target misbehaves at the machine, the first question is which of its bounds was missing, not how accurate the model was. A target without an evidence epoch cannot be expired, one without an error envelope cannot be compared with a tolerance, and one without an effort ceiling cannot be stopped short of damage, so each omission turns a model error into motion. The

remedy is rarely a faster or more confident proposer. On the conveyor, no tuning lets a 336 ms inference hold a 60 ms target. The fixes are a tracker that renews evidence, a slower belt, or a refusal.

The intent lease is the third law, proposal is not permission (principle 3), written as a record for goals. The lease grants a target, never a path. It names where the gripper may end, how hard it may press, and until when, and it leaves the route to a planner whose motion must be admitted again on its own evidence. Authority a learned model obtains for a goal therefore never extends to the motion that reaches it, and it lapses on the permission side's clock whether or not the model is still running.

10.11 Historical Perspective and Further Reading

How a word comes to name a thing was a question long before robots acted on the answer. Gibson (1979) gave the thing its modern name, the affordance, what an object offers an agent able to act on it, and Harnad (1990) posed the grounding problem as a question about symbols that point only to other symbols. Robotics made the affordance quantitative long before learned models proposed one. Nguyen (1988) reduced a stable two-finger grasp to a geometric test on friction cones, and Ferrari and Canny (1992) turned force closure into a quality measure in wrench space. Learned grasp generators now propose such grasps directly from point clouds (Sundermeyer et al. 2021; Fang et al. 2023), but their scores estimate success on a training distribution and are not the analytic measures they replaced, which is why this chapter rechecks every survivor.

Symbolic planning began on a machine that moved. Fikes and Nilsson (1971) wrote the STRIPS operator language to plan for Shakey, a mobile robot whose actions did not always have the effects their operators listed (Nilsson 1984), and PDDL later standardized that language so that planners could be compared on common problems (McDermott et al. 1998). The geometric preconditions that STRIPS left to the robot returned as task and motion planning, which interleaves symbolic search with samplers that certify continuous facts (Kaelbling and Lozano-Pérez 2011; Garrett et al. 2021). Language models now write the symbolic half, by ranking skills (Ahn et al. 2022) or by translating a request into a planning problem (B. Liu et al. 2023), and neither step checks the geometry.

The lease comes from distributed systems. Gray and Cheriton (1989) bounded a client's authority over a cached file by time, so that a crashed holder could not block the others, and chose the term by weighing renewal traffic against the delay a conflicting write must wait. The calibrated refusal comes from statistics. Vovk et al. (2005) built conformal prediction on exchangeability alone, and Ren et al. (2023) carried it to a robot deciding when to ask for help. The motion lease adds a physical term: it is sized by how fast the evidence behind its target decays, not by clock skew or message cost, and its expiry is enforced on a processor that the proposer cannot stall. For further reading, six works trace the ideas this chapter composes:

- Harnad (1990) states the symbol grounding problem and argues that symbols acquire meaning only through their connection to sensorimotor categories, the premise behind carrying evidence with every proposal.
- Garrett et al. (2021) surveys task and motion planning, from operator languages and samplers to the planners that couple them, and shows where symbolic plans need geometric certificates.
- Gray and Cheriton (1989) introduces leases for file-cache consistency and analyzes how the lease term trades renewal traffic against availability, the analysis this chapter reworks with evidence drift as the term.
- Angelopoulos and Bates (2023) is a practical introduction to conformal prediction, with the coverage argument and its exchangeability assumption stated plainly, as equation 10.3 uses them.
- Ren et al. (2023) applies conformal prediction sets to a language-model planner, turning calibrated ambiguity into a request for help, the producer-side refusal of section 10.3.1.
- Trautman and Krause (2010) names the freezing robot problem and shows why independent forecasts that grow with time block every path in a crowd, the trade-off behind the forward occupancy screen of section 10.6.1.

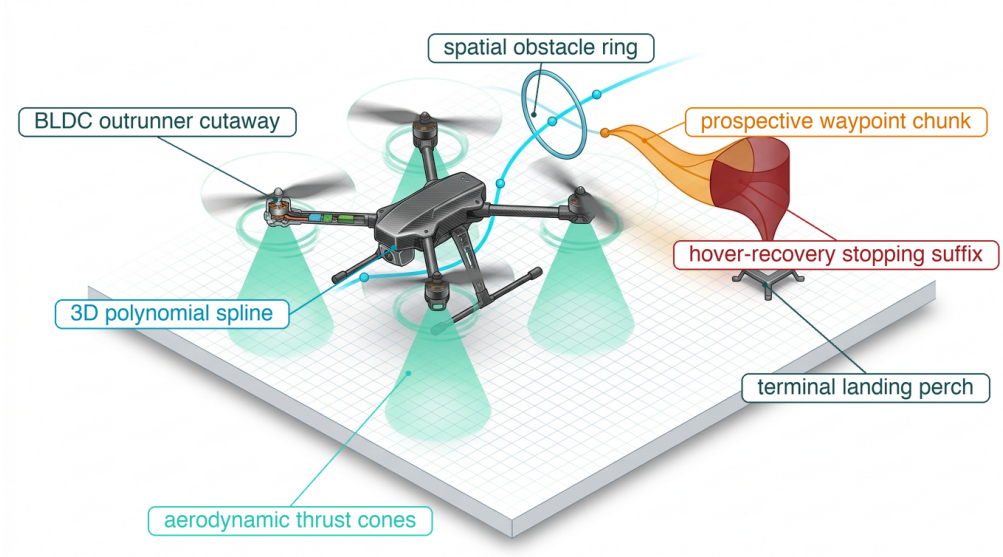
What's Next: From intent leases to trajectory planning

When the next inference pass is late, an intent lease fixes the admissible target region, the effort ceilings, and the evidence deadline, but it specifies no timed velocity or acceleration profile. It cannot tell an arm already moving toward the mug how to continue while the proposer stalls. Chapter 11 begins from that moment. It turns the lease into a timed trajectory that joins the motion already under way without a step in acceleration, sizes its replacement deadline against the inference tail, and attaches a precomputed stop that runs even if no later pass returns. The trajectory record carries all three, bound to the lease's digest, and the permission path admits the complete bundle before execution, because a valid goal does not prove that a moving arm can still come to rest.

DRAFT

11

Trajectory Planning

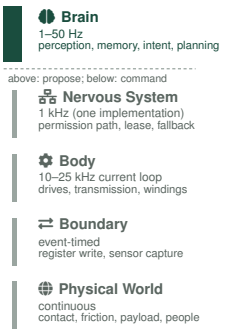


11.1 Continuous Trajectory Generation
11.2 The Planning Hand-off and the Trajectory Record
11.3 Replacement Timing and the Planning Horizon
11.4 Kinodynamic Feasibility
11.5 Behavior at the Seam
11.6 Choosing a Trajectory Generator
11.7 Trajectory Planning Failure Modes and Infeasible Seams
11.8 The Trajectory Record
11.9 Fallacies and Pitfalls
11.10 Summary
11.11 Historical Perspective and Further Reading

Purpose

Why must every motion a robot follows already contain the stop it will probably never use?

A moving robot cannot pause its momentum while a learned policy computes the next piece of its motion. Each new plan must take over from motion already under way. A plan that matches only the arm's position leaves a jump in velocity or acceleration, which the motors can meet only with a sudden step in torque. A real drive refuses such a step or converts it into vibration, wear, and a spilled load. A path that clears every obstacle can still ask for more joint speed, torque, or current than the body can supply, and the fastest timing along that path demands exactly the abrupt changes a drive cannot make. The next plan can also arrive late, because inference time has a long tail, and a machine that keeps moving while it waits spends the distance it needs to stop. Without trajectory planning, a correct decision about where to go becomes a command the body cannot follow, or motion that cannot be halted in the room that remains. Planning attaches timing, a smooth join to the present state, and a stop that runs without the proposer. Those three properties decide whether a command that crosses the causal boundary can still be brought to rest before its energy reaches something that cannot be undone.



Learning Objectives

- Distinguish prediction horizon, executed prefix, renewal cadence, and evidence age when budgeting trajectory replacement
- Map tool-space speed limits to joint rates through the manipulator Jacobian, explain why the mapping fails near a singularity, and bound an inverse-kinematics step there with damped least squares
- Evaluate a timed trajectory and its time parameterization against actuator torque, current-slew, and voltage limits, including reflected rotor inertia ($N_{\text{gear}}^2 J_{\text{rotor}}$)
- Synthesize closed-form quintic spline bridges, anchored at the projected seam state rather than a stale sampled pose, that eliminate acceleration steps and bound commanded jerk across replanning seams
- Size a jerk-limited C^2 stopping suffix from deceleration and jerk bounds and verify stopping clearance ($d_{\text{stop}} \leq D_{\text{clear}}$)
- Compare sampling-based search, trajectory optimization, and learned action chunking by the work each performs and the checks its output still requires
- Construct trajectory records whose two-tier deadlines (high-rate chunk lease and hardware commitment cutoff) keep motion out of inevitable collision states (ICS) and support independent per-tick permission decisions

11.1 Continuous Trajectory Generation

The arm of the warehouse mobile manipulator is reaching for a mug that drifts along a moving conveyor when an inference pass is delayed by operating-system scheduling contention. The intent lease still names the mug as a valid target, but a lease grants a target, never a path. A target coordinate paired with an expiration timestamp specifies neither velocity nor acceleration, so it cannot tell the arm what to do while the next pass is late. The arm cannot freeze abruptly, and it cannot extrapolate blind setpoints into space no check has covered. Planning must supply a concrete, time-parameterized path, an admissible transition from the motion already under way, and a precomputed stop that runs even if the next inference pass never returns.

↩ The intent lease, whose expiry bounds the validity window of every trajectory, is established in section 10.8.

Classical robotics split this problem into two stages. A search first finds a collision-free geometric path through a configuration space, and a time parameterization then assigns timing that respects kinematic and torque limits. Learned sensorimotor policies change both stages. Instead of searching a static, pre-mapped scene, a policy emits short chunks of future setpoints conditioned directly on live camera observations, and because it replans while the machine moves, each new trajectory must splice into motion already under way rather than start from rest. Neural inference also has variable, heavy-tailed latency, and the hardware keeps moving along its previous trajectory while it waits. Geometric clearance therefore does not settle a plan. A plan must also establish how long the proposed motion remains executable, how it joins the retiring trajectory, and how the machine comes to a controlled stop if later inference passes stall.

On this machine, the arm's low-level joint tracking loop evaluates feedback and setpoints at 1 kHz (datasheet; see the Reader Guide); however, this high-rate servo clock does not specify the physical stopping response. Above this loop sits the Brain, where learned policies such as Action Chunking with Transformers (ACT) (Zhao et al. 2023) or Diffusion Policy (Chi et al. 2025) process sensory observations on the host application processor to synthesize future reference paths.[1] The machine's chunk policy operates with an illustrative budget of 40 ms per pass at p99, vision included. Even so, memory contention, direct memory access (DMA) traffic, or thread preemption can stretch tail latency beyond 150 ms. The real-time controller therefore cannot block synchronously on policy execution; it requires both buffered nominal motion and an immediately dispatchable, pre-validated stop.

Whether candidate motions originate from sampling-based search, numerical trajectory optimization, or learned action chunking (section 11.6), the generator serves strictly as an unprivileged proposer. Time parameterization exposes instantaneous velocity, acceleration, and jerk to dynamic validation, and the independent safety permission path must verify that motor windings can gener-

[1] **Action chunking representations**: ACT and Diffusion Policy, the 2023-era chunk policies, carry no boundary-derivative constraints. Real-time chunking (Black, Galliker, et al. 2025) conditions each new chunk on the actions already committed, which makes a consistent join more likely without guaranteeing one, so the planner still proposes a transition bridge across each replanning seam and the permission path checks it before admission.

↩ The kinetic energy a trajectory commits, and the stopping distance that energy demands, are established in section 2.3.1.

ate the required torques under dynamic load and that sufficient clearance exists along the arm's swept stopping volume before any motion is admitted.

A multi-step action chunk delivers a finite horizon of future setpoints that the low-level tracking controller consumes sequentially while the proposer computes the next chunk. Splicing successive chunks introduces an **execution seam**: the temporal and kinematic boundary where the retiring setpoints hand off control to the incoming plan. Matching position alone across this seam leaves a discontinuous velocity step, and matching velocity while neglecting acceleration leaves an instantaneous acceleration step. Because physical links and payloads carry inertia, discontinuous accelerations demand instantaneous steps in motor torque, exciting mechanical vibrations and risking overcurrent faults. The trajectory generator must therefore enforce continuity through position, velocity, and acceleration (the C^2 continuity condition) while verifying that dynamic torques and swept clearance throughout the transition segment remain within safe physical envelopes.[2]

↪ The runtime gatekeeper that checks every admitted setpoint against live state is developed in section 12.1.

Decoupling the execution rate from the inference rate allows the low-level tracking controller to service motor loops on a rigid real-time clock while neural inference runs asynchronously. Setpoint buffering absorbs latency jitter, but buffered motion cannot remain valid indefinitely. Sensor observations age, dynamic obstacles shift, and localization drift accumulates over time. Consequently, the planner's replacement deadline must account for the worst-case inference tail, guaranteeing sufficient lead time to commit to a feasible, controlled stop before either the buffered trajectory drains or the supporting environmental evidence expires.

Every admitted proposal must therefore arrive as a self-contained bundle: the active trajectory segment continuous through acceleration (C^2), its explicit validity deadlines, and a pre-verified stopping suffix whose required deceleration distance fits within verified obstacle clearance ($d_{\text{stop}} \leq D_{\text{clear}}$). An isolated safety microcontroller (MCU) inspects and validates this bundle across the proposal boundary before arming actuator outputs, caching the emergency stopping continuation in local real-time memory so it can be autonomously dispatched if communications falter.

The machine meets these obligations in a fixed order. What the planner hands to the tracking controller and the permission path comes first, together with the map from tool coordinates to the joint limits the drives enforce. Replacement timing is then sized against the inference tail, a timed candidate and its seams are checked against what the actuators can follow, and the jerk-limited stop that every admitted plan must carry is sized. The generators that propose candidates, from sampling-based search to learned action chunks, can be compared only after those obligations are fixed.

[2] **C^2 continuity at the motor**: In permanent-magnet synchronous motors, electromagnetic torque couples directly to quadrature stator current ($\tau_{\text{motor}} = K_t i_q$). Under this model an acceleration step requires a finite step in torque and current, while a velocity step implies an ideal acceleration impulse, which no drive can supply.

11.2 The Planning Handoff and the Trajectory Record

If the planner hands over raw joint setpoints for the reach toward the mug without binding them to the obstacle geometry, the active intent lease, and a pre-checked stop, the arm can chase an expired target or coast past its clearance the moment the next inference cycle stalls. To prevent that failure, the planner consumes the belief record, which supplies obstacle geometry and clearance. It also consumes the intent lease, which supplies the grounded goal, task tolerances, effort ceilings, and the expiry time that bounds the lease ($t_{\text{intent_exp}}$). Throughout, the planner must respect the actuator and transmission limits of the physical body. The handoff connects those inputs to the checks that permit execution, and two things must be fixed before any replacement timing can be budgeted. The first is the map between the tool coordinates in which clearance is measured and the joint coordinates whose limits the drives enforce. The second is the record that carries a candidate across the proposal boundary, with the deadlines and the stop that bound its authority.

↩ The belief record and the state invariants it carries are established in section 9.3.

11.2.1 Mapping tool motion to joint limits

A path that keeps the tool inside its speed limit and clear of obstacles in workspace can still command a joint near a straightened elbow to spin faster than its drive allows, tripping a velocity fault mid-reach. Catching that mismatch requires translating workspace motion into the manipulator's joint coordinates. A configuration $(q_1, \ldots, q_n)$ collects the joint positions into a single point in the **configuration space**[3] $\mathcal{C}$, and removing configurations occupied by obstacles leaves the collision-free subset $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$ (Lozano-Pérez 1983). Because a geometric path through $\mathcal{C}_{\text{free}}$ still lacks timing, an **executable trajectory** assigns physical timestamps to the path, generating continuous state references $\mathbf{x}(t) = [\mathbf{q}(t)^\top, \dot{\mathbf{q}}(t)^\top]^\top$ and acceleration profiles $\ddot{\mathbf{q}}(t)$ alongside feedforward torque profiles

[3] **Configuration space**: Lozano-Pérez (1983) shrank the moving body to a point by growing each obstacle by the body's shape, and in the same year Schwartz and Sharir (1983) treated the "piano movers' problem" as a search of exactly this space. The reduction turns a collision query into a membership test on $\mathcal{C}_{\text{obs}}$, but the space gains a dimension with every joint, which is why collision queries dominate the run time of the sampling planners in section 11.6.

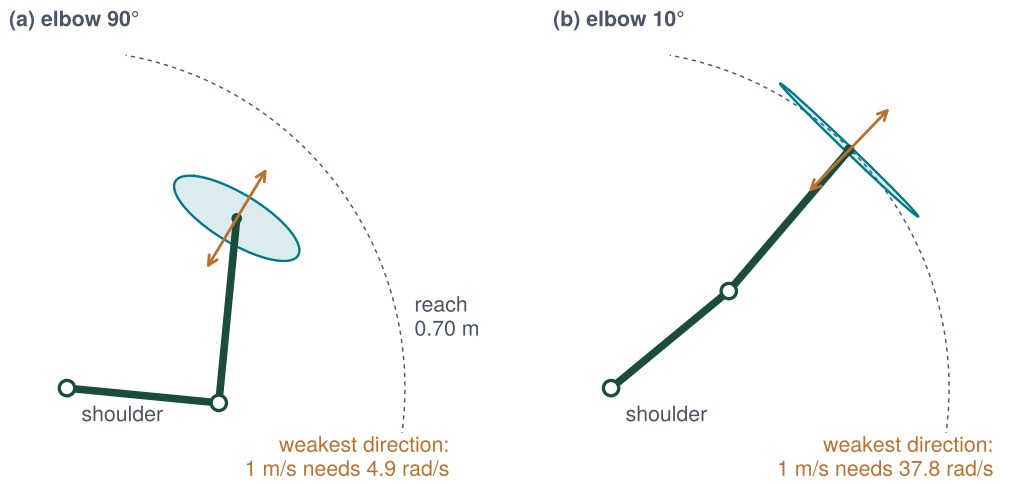


Figure 11.1: The same tool speed, two elbows: The planar two-link arm drawn to scale with its 0.316 m and 0.384 m links. Each ellipse is the set of tool velocities that a unit joint-rate vector can produce, drawn at one scale in both panels; its short axis is the weakest direction (arrow). At a right-angle elbow the ellipse is round enough that 1 m/s in that direction needs 4.9 rad/s of joint rate. Near the straight elbow the ellipse collapses toward a line across the arm, and the same tool speed along the arm needs 37.8 rad/s.

$\tau_{\text{ff}}(t)$. Whereas the kinematic references state the intended position, velocity, and acceleration, the feedforward terms supply the torque predicted by the rigid-body model. The complete candidate crosses the proposal boundary for independent checking, and only admitted actuator commands cross the causal boundary to the drives.

The drives track joint angles, but clearance is measured in the task space where the tool, the mug, and the conveyor guard sit. In a planar two-link abstraction of the arm's shoulder, elbow, and wrist with upper-arm length l_1 and forearm length l_2, **forward kinematics** gives the tool position directly as a function of the joint angles:

$$x = l_1 \cos q_1 + l_2 \cos(q_1 + q_2), \qquad y = l_1 \sin q_1 + l_2 \sin(q_1 + q_2) \tag{11.1}$$

Differentiating equation 11.1 gives tool velocity $\mathbf{v}_e = \mathbf{J}(\mathbf{q})\,\dot{\mathbf{q}}$, where the manipulator Jacobian[4] $\mathbf{J} = \partial \mathbf{p}/\partial \mathbf{q}$ changes with configuration. It is the same map that showed, in section 5.9.2, why a Cartesian action label can force a different arm through a singularity; here it prices the arm's own reach. For the two-link arm its determinant is $l_1 l_2 \sin q_2$, which vanishes when the elbow straightens, the singular pose at which no joint motion moves the tool along the arm.

Consider the link lengths from the arm's published kinematic chain: 0.316 m and 0.384 m, which yield a 0.70 m planar reach (Franka Robotics GmbH 2023). The arm's weakest direction is the tool direction that a unit of joint rate moves least, set by the smallest singular value $\sigma_{\min}$ of $\mathbf{J}$, so that $\|\dot{\mathbf{q}}\| = v_{\text{TCP}}/\sigma_{\min}$. Moving the tool in that direction at the 1 m/s tool center point (TCP) limit requires a joint-rate vector of magnitude 4.9 rad/s at a 90° elbow. With two joints, the larger component of a vector is at least its norm divided by $\sqrt{2}$, so at least one joint must then turn at 3.5 rad/s, above the published 2.175 rad/s limit of the shoulder and elbow joints. Straightening the arm makes the demand far worse. The image of a unit joint-rate circle under $\mathbf{J}$, the manipulability ellipse of tool velocities, is broad at the 90° elbow and collapses into a narrow sliver across the links at 10° (figure 11.1). Sustaining 1 m/s along the arm, the ellipse's minor axis, then needs a joint-rate magnitude of 37.8 rad/s. A tool-speed limit bounds no joint speed near a singularity. Every tool-space quantity this chapter checks (the TCP limit, the stopping distance, and the swept clearance) must therefore be mapped back through $\mathbf{J}$ to the joint limits the drives enforce. This holds even on a seven-joint arm, whose extra degrees of freedom help the planner steer around singular poses that this planar abstraction omits.

[4] **Manipulator Jacobian**: It is the same object automatic differentiation builds, the matrix of first derivatives of an output with respect to its inputs, here tool position with respect to joint angles. Reverse-mode autograd never forms the matrix; it computes products $\mathbf{J}^\top \mathbf{v}$, and the manipulator equation uses that same transpose product to map a tool wrench into joint torques (equation 11.8). A planner is the unusual consumer that must *invert* $\mathbf{J}$, and it is the inverse, not the Jacobian, that fails at a singularity.

↪ Forward kinematics, the manipulator Jacobian, singularities, and the damped least-squares inverse are developed for n joints in section B.6.1.

The same map must also run in reverse. A proposal stated in tool space, whether a grasp pose from the intent lease or a Cartesian velocity from a policy or a teleoperation device, leaves the planner to find the joint motion that produces it, the **inverse kinematics** problem. The two-link arm has a closed-form inverse. A seven-joint arm replanning chunk by chunk is solved by differential steps instead: linearize at the current configuration and solve $\mathbf{J}\Delta\mathbf{q} = \Delta\mathbf{p}$ for the joint step that moves the tool by $\Delta\mathbf{p}$, then repeat from the new configuration. Where $\mathbf{J}$ is invertible the exact step is $\Delta\mathbf{q} = \mathbf{J}^{-1}\Delta\mathbf{p}$. Written along the singular directions of $\mathbf{J}$, this step divides the part of $\Delta\mathbf{p}$ along each tool direction by that direction's singular value σ. The weakest direction therefore receives the largest joint step, $1/\sigma_{\min}$ per unit of tool motion, which is exactly the joint-rate demand that figure 11.1 charts diverging along the minor axis as the elbow straightens. An exact inverse passes the singularity's arithmetic straight to the drives.

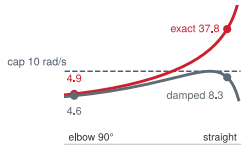
Damped least squares gives up exactness to bound the step (Wampler 1986; Nakamura and Hanafusa 1986). Instead of demanding $\mathbf{J}\Delta\mathbf{q} = \Delta\mathbf{p}$, it chooses the joint step that best trades tool error against joint motion, with a damping length λ that sets the exchange rate:

$$\Delta\mathbf{q}_\lambda = \arg\min_{\Delta\mathbf{q}} \left(\|\mathbf{J}\Delta\mathbf{q} - \Delta\mathbf{p}\|^2 + \lambda^2 \|\Delta\mathbf{q}\|^2 \right) = \mathbf{J}^\top \left(\mathbf{J}\mathbf{J}^\top + \lambda^2 \mathbf{I} \right)^{-1} \Delta\mathbf{p} \tag{11.2}$$

Setting the gradient of the objective to zero gives $(\mathbf{J}^\top\mathbf{J} + \lambda^2\mathbf{I})\,\Delta\mathbf{q} = \mathbf{J}^\top\Delta\mathbf{p}$, and moving $\mathbf{J}^\top$ through the inverse yields the form in equation 11.2, which needs only a 2×2 solve for the planar arm. A reader who has trained a model will recognize ridge regression, with λ^2 as the weight-decay coefficient. Along each singular direction the gain changes from $1/\sigma$ to $\sigma/(\sigma^2 + \lambda^2)$. Far from the singularity, where $\sigma \gg \lambda$, the two agree. As σ falls toward zero the damped gain falls with it instead of diverging, and it never exceeds $1/(2\lambda)$, reached where $\sigma = \lambda$. The damping thus places a configuration-independent ceiling on the joint step, which no choice of pose can breach.

With an illustrative λ of 50 mm, the weakest-direction request at the 1 m/s TCP limit needs 4.6 rad/s of joint rate at the 90° elbow, against 4.9 rad/s undamped, and the tool still realizes 94.3 percent of the requested motion. At the 10° elbow the damped command is 8.3 rad/s instead of 37.8 rad/s, and no pose can demand more than 10 rad/s. The price is paid where the damping acts: the tool realizes only 21.9 percent of the requested motion along the arm, so the step lags its target in exactly the direction the arm cannot move well. That residual is a tracking error the planner must carry into the trajectory's timing and clearance checks rather than discard. A larger λ lowers the ceiling and enlarges the error everywhere, which is why practical solvers raise the damping only as $\sigma_{\min}$ approaches it.

Damping bounds the inverse-kinematics step; it does not make the step feasible. Even the damped 8.3 rad/s exceeds the 2.175 rad/s joint limit, so the planner must still slow the tool through time parameterization (section 11.4.5), and the permission path still checks every joint rate against the limits the drives enforce. Damped least squares is a better proposal, not a permission.



As the elbow straightens, the exact inverse climbs from 4.9 rad/s to 37.8 rad/s; damping caps it at 10 rad/s.

11.2.2 The trajectory record and its deadlines

If a candidate trajectory crosses from the planner to the controller without an immutable expiration time and a pre-checked stop, a hung host process leaves the drives tracking the last command into an obstacle. What the planner hands over is therefore a **trajectory record**, the ninth record in the record chain and the immutable proposal that the tracking controller follows and the permission path admits. The trajectory record consumes the geometric obstacle bounds from the belief record and the task target and lease window from the intent lease. It binds an untrusted sequence proposal to checked physical limits across the formal boundary between the unprivileged, asynchronous Brain and the deterministic, hard real-time Nervous System. Its header names its coordinate frame $\mathcal{F}$, evidence time t_0, initial reference state, and active intent lease expiry $t_{\text{intent_exp}}$. The header also carries four deadlines defined in the **MCU monotonic clock domain** (into which host and sensor timestamps are converted with a bounded synchronization error). Nominal tracking begins at t_{start}, the nominal setpoints end at $t_{\text{plan_exp}}$, the matched stop must physically begin at t_{commit}, and the admitted stop finishes at $t_{\text{stop_end}}$. Beside these header fields, an assumption sidecar declares the payload, friction, obstacle-map epoch, and other conditions under which the candidate was checked (section 11.8). Within the planner itself, each replacement is scheduled against an earlier internal milestone, the blend deadline t_{blend}, by which a preferred successor should clear validation. The

latest replacement-admission decision on the MCU precedes physical stop onset at t_{commit} by the reserved decision, transport, and actuator delay.

The permission path permits nominal **TRACK** only while the intent lease and trajectory record remain valid, so an expired intent lease terminates nominal tracking immediately, regardless of what the trajectory's own timestamps allow. To manage asynchronous inference latency tails and hardware faults without compromising safety, the architecture enforces a strict **two-tier deadline hierarchy**:

1. **Fast Nervous System periodic chunk lease (T_{lease}, 60 ms):** This high-rate heartbeat governs the real-time execution contract between Brain and Nervous System. The chunk policy proposes base velocity and arm joint targets together, so one chunk lease governs both bodies, and only an admitted replacement chunk renews it, with a desired admission interval of 50 ms on this machine. Qualification must bound successful-admission interarrival, not merely inference or request cadence. If an upstream inference pass hangs or communication drops, the lease lapses 10 ms after its expected renewal, immediately requesting a controlled stop whose onset follows the reserved response delay.
2. **Trajectory commitment deadline:** The record names the latest physical onset of its matched stop. Replacement admission must close early enough to dispatch that stop and absorb actuator response delay. This geometric limit does not replace the running machine's shorter chunk lease; the later worked timeline examines an explicitly different, longer-lease configuration.

↩ The chunk lease and its sizing from the stopping budget are established in section 4.4.

Two stopping responses back these deadlines. A matched stop suffix, latched at admission, begins at t_{commit} if no successor has been admitted; a state-matched fallback, built from the measured state, answers any earlier invalidation, such as a lapsed lease (section 11.5.1). Either stop remains authorized after nominal tracking expires, provided its own state, clearance, sensing, and drive-energy premises still hold.

Definition 11.1: Stopping suffix

Stopping suffix is the bounded continuation, checked and latched with a trajectory at admission, that brings the machine from a declared entry state to rest along the already-checked path, within the clearance that remains and without further input from the proposer.

1. **Significance**: It is the term that makes $d_{\text{stop}} \leq D_{\text{clear}}$ checkable before motion begins. From the arm's 1 m/s TCP limit, the worked suffix sized in section 11.5.2 travels 137.5 mm of an illustrative 300 mm of clearance, the free distance along the checked path between the tool and the nearest obstacle. The stop alone therefore claims nearly half the room before any response delay is charged.
2. **Distinction**: The matched suffix is anchored at the nominal state expected at t_{commit}; the state-matched fallback is built from the measured state when authority ends earlier. Both are controlled stops along the checked path, unlike a drive inhibit or brake, whose motion is not planned.
3. **Common pitfall**: Treating the last nominal setpoint, or a hold at the final position, as the stop. A hold is not a deceleration profile, and a suffix sized with constant deceleration understates the distance any drive that ramps its deceleration will travel.

The two execution consumers read distinct subsets of the trajectory record. The downstream tracking controller interpolates admitted setpoints, timestamps, and feedforward torque terms to drive the motor inverters along a continuous profile fetched from the real-time execution jitter buffer. Simultaneously, the independent permission path inspects the record's lineage, timing bounds, dynamic assumptions, validity envelopes, and verified stopping continuation ($d_{\text{stop}} \leq D_{\text{clear}}$). The permission path monitors current limits and physical tracking error on every millisecond tick without inheriting the planner's feasibility claim as proof. An unmodeled obstacle, payload slip, or contact wrench immediately revokes TRACK authority and activates the state-matched fallback. Because every contract field maps to an explicit hardware or software gate, the Nervous System safely executes fast physical motion without presuming that subsequent neural inference cycles will

complete on schedule. A record of this kind is short-lived by design, so the next constraint is how often a successor must arrive and how long producing one takes.

11.3 Replacement Timing and the Planning Horizon

On the mug reach, the arm needs an admitted replacement every 50 ms and loses tracking authority 60 ms after the last one. Whether a replacement can arrive that often depends on how long it takes to produce one, and producing one is more than a single inference pass. The observation behind the candidate must be captured, the proposer must run, and the candidate must cross to the safety MCU and pass validation before it is admitted. Every generator, from numerical trajectory optimization to an autoregressive policy, pays for that pipeline. The timing questions therefore come before the choice of generator: how long a replacement takes, how deep into its latency tail the design must look, how much motion one record must cover, and from which state the next candidate must be planned.

11.3.1 Action chunk horizons and reaction lag

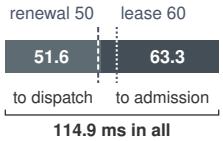
If a replacement chunk takes longer from camera capture to MCU admission than the active chunk's lease allows, the arm drops into its stopping fallback before the new plan arrives. If the executed prefix is stretched to hide that latency, the arm reacts too slowly when the mug or an obstacle shifts. Sizing the prediction horizon H therefore starts from an end-to-end latency budget for replacement, one that spans host accelerator memory, the inter-processor link, and real-time microcontroller firmware.

The scheduling metric that governs every handoff is **replacement latency** L: the total elapsed wall-clock time from the initial physical capture of the sensor observation on which a candidate trajectory proposal rests to the formal admission of that proposal into the real-time execution queue. Formulated in the event taxonomy of table 4.1, L spans the interval from capture to admission. It is the direct sum of two intervals. The first is the capture-to-dispatch perception age t_{age} established in section 8.4.1. The second is the dispatch-to-admission interval Δ_{da}, a design allocation whose stages are listed next and which section 13.6.2 later tests against the hardware:

$$L = \underbrace{t_{\text{age}}}_{\text{capture}\to\text{dispatch}} + \underbrace{\Delta_{\text{da}}}_{\text{dispatch}\to\text{admission}}, \qquad \Delta_{\text{da}} = t_{\text{infer}} + t_{\text{link}} + t_{\text{deser}} + t_{\text{validate}} + t_{\text{activate}} + t_{\text{wait}} \tag{11.3}$$

On the physical machine, sensor data is already 51.6 ms old when planning dispatches. The subsequent dispatch-to-admission allocation Δ_{da} of 63.3 ms accounts for the chunk policy's full 40 ms neural network forward pass and 18 ms of whole-plan kinematic validation. The balance is a few milliseconds of inter-board link transport, message deserialization, activation staging, and synchronization wait until the next discrete permission tick. Section 13.6.2 itemizes this balance when it tests the allocation on the selected hardware. These stages combine to yield an overall replacement latency L of 114.9 ms. L is a latency, not a period. It exceeds both the 50 ms renewal period and the 60 ms lease, so the renewal cadence can be sustained only by overlapping requests, several in flight at once, a throughput condition the timeline below returns to. Charging the entire neural inference pass after dispatch intentionally bills the visual encoding stage twice, since that stage is already inside t_{age}, which makes the budget deliberately conservative. When pipelined stages execute concurrently or contend for shared memory buses, the sum of independent stage percentiles does not equal the percentile of the sum (section 2.2.2). Sizing the planning horizon therefore demands evaluating the empirical end-to-end distribution of L, measured directly between physical sensor capture interrupts and microcontroller admission timestamps.

Trajectory validation represents the motion planner's dedicated compute contribution within this latency budget. The 18 ms validation allowance covers the combined elapsed contribution of host geometry validation and bounded local MCU admission, including scheduling delays, rather than overhead silently absorbed into the millisecond actuator tick. Its split between processors remains to be qualified. Formal admission requires bounded staging memory, candidate spline and stopping coefficients, and qualified local clearance evidence; the host processor's full, multi-gigabyte scene representation need not reside on the embedded MCU. The real-time scheduler must preserve deterministic per-tick enforcement while background admission executes, refreshing the seam



A 114.9 ms replacement path outlasts the 50 ms renewal, so requests must overlap.

boundary check immediately prior to commitment. System architects must allocate and qualify this compute and memory footprint under worst-case shared-bus interference. Because each term in equation 11.3 represents an illustrative design allocation, neither L nor its constituent stages serve as unverified static bounds; both intervals remain explicit qualification obligations on the physical hardware.

Prediction length and execution duration are separate architectural parameters. While the prediction horizon $H\Delta t$ specifies how far into the future the sequence proposer forecasts, the **executed prefix** represents only the initial slice dispatched to actuators before an admitted replacement plan supersedes it. Expanding the prediction horizon supplies candidate continuity well past the next planned handoff, but does not grant the robot authority to execute that trajectory unchecked. The production controller targets an admitted renewal every 50 ms, enforcing hard lease expiry at 60 ms following the most recent admission. The evidence horizon of the upstream target observation can be shorter still.

Compute amortization depends strictly on inference dispatch frequency rather than the raw number of future setpoints generated in a forward pass. Executing a longer trajectory prefix lowers invocation frequency and amortizes accelerator workload, yet it simultaneously inflates closed-loop reaction lag, delaying task corrections when the environment drifts. Conversely, predicting farther into the future while maintaining a high renewal rate increases proposer compute without reducing inference duty. Emergency protective stopping remains governed by an independent hardware permission path and its certified reaction latency. Sizing the prediction horizon therefore balances closed-loop reactivity, pipeline renewal throughput, sensor evidence freshness, and fallback transition rates under realistic system loads; no universal optimal horizon emerges from trajectory buffer capacity alone.

↩ Action chunking as a supply lever is established in section 3.5.1.

11.3.2 Tail latency budgeting across a mission

When the resident fallback works as certified, a late replacement costs availability rather than safety. A missed replanning deadline becomes a collision hazard only if the resident stop is kinematically infeasible, corrupted in memory, or delayed beyond its budgeted stopping distance. Over a long mission, however, the number of handoffs makes tail latency decide whether the mission completes,[5] even when a stopping profile safely absorbs every individual miss. The analysis below therefore bounds how often handoffs miss their deadlines, not how often the machine collides.

[5] **Tail at scale**: Dean and Barroso (2013) showed that a distributed request fanned out across many servers is bottlenecked by the slowest tail response, transforming rare single-server delays into frequent user-facing latencies. A continuous mission comprising thousands of sequential handoffs maps this same fan-out onto the time domain. Because every individual handoff must meet its admission deadline to avoid triggering a stop, the complete mission inherits the worst-case tail latency, rendering conventional per-handoff p99 benchmarks insufficient for assessing end-to-end mission reliability.

The component breakdown in equation 11.3 defines the latency of a single replacement cycle. Because an autonomous mission executes this handoff cycle repeatedly, the tail of the latency distribution dictates system behavior. A mission requiring K successive plan handoffs fails its continuous execution target if even one handoff misses its deadline. Partitioning a cumulative mission exceedance budget $\epsilon_{\text{mission}}$ uniformly across all K handoffs yields an upper bound on allowable per-handoff failure probability via the union bound:

$$\epsilon \leq \frac{\epsilon_{\text{mission}}}{K} \tag{11.4}$$

Napkin Math 11.1: Handoff deadlines across a mission

Problem: A generic replanner, slower than this machine's chunk renewal, meets a p99 replacement budget on each handoff. Over a ten-minute mission, how likely is it that every handoff arrives on time, and what per-handoff tail must the design target instead?

Variables:

- Mission length 600 s and replanning rate 5 Hz (illustrative)
- Per-handoff exceedance of the p99 budget, $p = 0.01$ (definitional)
- Mission exceedance target $\epsilon_{\text{mission}} = 1 \times 10^{-3}$ (chosen)
- Independent misses (assumption)

Math:

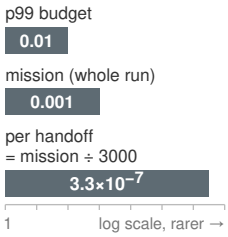
1. **Handoff count**: 600 s × 5 Hz gives $K = 3000$, and $Kp = 3000 \times 0.01 = 30$ handoffs are expected late.
2. **Every handoff on time**: $(0.99)^{3000} \approx 8 \times 10^{-14}$.
3. **Design tail** (equation 11.4): $\epsilon \leq \epsilon_{\text{mission}}/K \approx 3.3 \times 10^{-7}$, the p99.99997 quantile, deeper than the p99 budget by $0.01/(3.3 \times 10^{-7}) \approx 3 \times 10^4$.

Result: A mission almost never completes with every handoff inside a p99 budget, and the tail a mission target implies lies more than four orders of magnitude deeper.

Systems insight: The quantile a design must target is set by how many handoffs a mission makes, not by the per-handoff budget, and it lies beyond what a test campaign of practical size observes. A late handoff must degrade service availability, not safety clearance. Preserving $d_{\text{stop}} \leq D_{\text{clear}}$ relies on the resident stop profile (the pre-validated stopping suffix latched in MCU memory at admission and triggered without the host), not on optimistic assumptions about latency tails.

This per-handoff quantile functions as a **design quantile target**, a risk allocation across the software stack rather than an observed tail latency. In practice, resource contention across shared memory, PCIe buses, and hardware accelerators introduces temporal correlation, clustering deadline misses and violating independent-failure assumptions. Hardware-in-the-loop stress testing under maximum bus and memory saturation can expose deadline misses, but establishing this deep quantile requires a stated statistical model, sufficient observations, and evidence that the operating conditions match. The mission architecture must tolerate fallback frequencies that the available evidence cannot rule out without requiring human intervention.

To maintain continuous tracking, the client dispatches the next planning request at an offset no later than T_{request} after the active chunk begins executing. Plan admission must complete before a hard decision cutoff that precedes physical commitment t_{commit} by the actuator response allowance, rather than running up against the final nominal setpoint. Given a replacement-latency quantile target $Q_{1-\epsilon}(L)$ and a safety reserve T_{reserve}, a sufficient conservative timing requirement is $t_{\text{commit}} - t_{\text{start}} \geq T_{\text{request}} + Q_{1-\epsilon}(L) + T_{\text{reserve}}$. This requirement charges the complete capture-to-admission quantile after request dispatch, conservatively charging the preceding capture-to-dispatch age again. That extra age charge is separate from the visual-encoding overlap in equation 11.3. A tighter dispatch-relative requirement would use an independently characterized dispatch-to-admission distribution; subtracting marginal latency quantiles would not establish it. The reserve parameter T_{reserve} includes both scheduling slack before preferred admission and the physical decision-to-onset allowance. Furthermore, executing the resident stop requires an additional time window T_{stop} under dedicated MCU authority. Consequently, the time horizon T_{record} that an admitted trajectory record must cover, whether stored as dense setpoint arrays or polynomial splines, satisfies the corresponding conservative design requirement:



Spread over 3000 handoffs, a mission target demands a p99.99997 deadline, far past p99.

$$T_{\text{record}} \geq T_{\text{request}} + Q_{1-\epsilon}(L) + T_{\text{reserve}} + T_{\text{stop}} \tag{11.5}$$

In the baseline warehouse configuration, the chunk policy outputs a discrete setpoint every 25 ms across a 16-step horizon, producing 400 ms of nominal forward trajectory. Under nominal operation, an admitted replacement supersedes this trajectory after 50 ms. If renewal fails, the active lease lapses at 60 ms past the preceding admission tick. Unexecuted setpoints beyond that cutoff serve as candidate references for continuous handoffs rather than extended tracking authority.

On this running schedule the second tier, the record's commitment deadline, never acts, because a stalled proposer lets the 60 ms chunk lease lapse long before commitment arrives. To show how a record's own deadlines coordinate nominal motion with its stop, the worked timeline therefore uses an **alternate longer-lease schedule** for the same arm, one whose chunk lease is long enough not to lapse before the commitment cutoff. The commitment tier can then be seen on its own. On this schedule the planner issues its request 60 ms after the active chunk begins (later than the running machine's 50 ms renewal, which only a longer lease permits). It budgets 160 ms as its design-tail replacement latency $Q_{1-\epsilon}(L)$ above the 114.9 ms nominal sum, as a chosen stand-in for the deep quantile that equation 11.4 implies. It then adds 20 ms of scheduling slack to set a

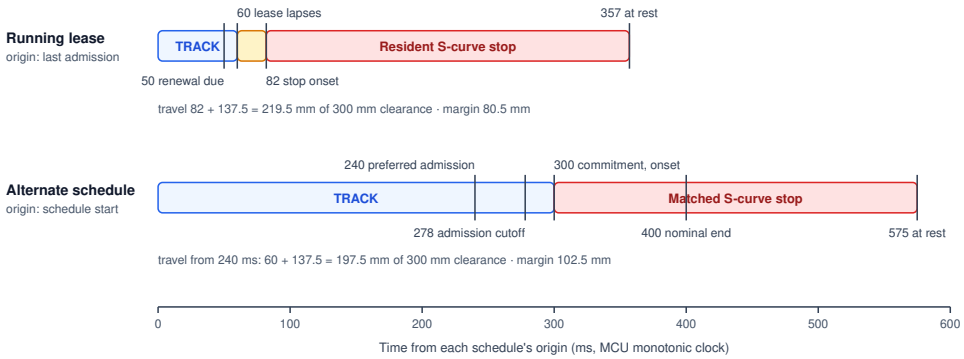


Figure 11.2: Two lease schedules for the arm: On the running lease (top), a stalled proposer lapses the lease 60 ms after the last admission, and the tick, bus, and onset allowance (amber) delays the resident stop to 82 ms. On the alternate schedule (bottom), admission must close by 278 ms so the matched stop begins at the 300 ms commitment; the unused nominal setpoints run to 400 ms. Each margin is charged against its own clearance check and still excludes sensing and model reserves.

preferred admission deadline at 240 ms. Adding the 22 ms decision-to-onset response allowance to the scheduling slack gives the 42 ms full reserve of equation 11.5 and a conservative minimum stop onset of 262 ms. Selecting a slightly later physical stopping onset at 300 ms leaves 60 ms between preferred admission and commitment. Replacement admission can therefore close as late as 278 ms (one 22 ms decision-to-onset delay before commitment). The pre-validated 275 ms stopping suffix (section 11.5.2) then brings the arm to rest at 575 ms, provided clearance holds through that entire window.

This alternate profile serves to contrast buffer structure with the production policy. Its window spans 23 setpoint periods, yet it stays compact in memory because the stop is stored as an analytical profile (jerk limits, acceleration limits, and phase durations) rather than as a dense array of setpoints. Set on one time axis, the two schedules spend clearance differently (figure 11.2). On the running lease, the lease lapses at 60 ms, the tick, bus, and onset allowance delays the resident stop to 82 ms, and the arm uses 219.5 mm of the 300 mm clearance, leaving 80.5 mm. On the alternate schedule, admission must close by 278 ms, after the preferred 240 ms, so that the matched stop begins at the 300 ms commitment and reaches rest at 575 ms with 102.5 mm to spare. The nominal setpoints that run on to 400 ms are never executed.

Sustaining the continuous 50 ms renewal cadence of the running lease requires verifying pipeline throughput alongside latency. Because the illustrative 114.9 ms replacement latency exceeds the 50 ms renewal period, the system must pipeline consecutive planning requests. Pipelining maintains real-time throughput only when intermediate hardware stages provide non-blocking queue capacity and bounded bus arbitration delay; pipelined concurrency cannot be established from latency sums alone. Until comprehensive load testing validates multi-request pipelining under peak bus contention, the specified resident fallback responds to missed renewal only under its stated state, clearance, sensing, energy, drive-response, deceleration, and jerk assumptions.

Both lease architectures remain bounded by the validity lifespan of upstream perceptual evidence. Even when employing a qualified local object tracker, the tracked evidence horizon derived in section 10.5 is shorter than the alternate commitment duration, so a single vision observation of a target object cannot justify that entire duration. Instead, the manipulator uses a staged perceptual approach. It follows an admitted prefix through a certified clear corridor, triggers fresh visual acquisition from a wrist-mounted camera at closer proximity, and re-anchors subsequent trajectory segments to updated state estimates. Embedded firmware then admits the terminal grasp maneuver only when the complete motion profile satisfies current target tolerances within an active validity envelope. Although an overall reaching motion may span several observation cycles, every executed trajectory slice requires fresh evidentiary support. Generating a new trajectory proposal cannot refresh stale sensor evidence, and losing perceptual tracking immediately transitions the robot into a state-matched stopping fallback.

On the Push-T manipulation benchmark (Chi et al. 2025), frequent closed-loop replanning lets an articulated planar arm recover from live disturbances as it guides a T-shaped block toward a designated pose. A diffusion policy generates $H = 16$ future steps and executes an initial prefix of $T_a = 6$ steps before replanning. It recovers dynamically from transient camera occlusions, external block displacements during approach, and perturbations near the terminal pose. However, these empirical trials demonstrate task-level reactive correction rather than certified derivative continuity or deterministic stopping under compute starvation. A safety-critical production system still requires an independently executable stopping suffix verified against known kinematic clearance bounds.

11.3.3 Conditioning on the state at splice time

While a joint rotates at 0.50 rad/s during an illustrative 80 ms compute and transport delay (somewhat longer than the 63.3 ms dispatch-to-admission allocation), a joint pose sampled when the replacement request was dispatched lags the advancing reference by 0.040 rad (2.29°). Mapped through the arm's forward kinematics (equation 11.1) at its 0.70 m maximum planar reach, that angular lag displaces the tool center point by up to 28 mm. This is more than the entire lateral swept-volume clearance budgeted for a reach past the conveyor guard (section 11.7.2). That clearance is a sideways allowance, distinct from the 300 mm of free path ahead of the tool that a stop consumes. Conditioning a motion plan on sensor feedback captured when inference begins therefore induces the **stale-state paradox**. By the time the generative policy completes its forward pass, the physical arm has already advanced along its prior trajectory. Initializing the new plan from the request-time state then commands the actuators to step backward across the distance traversed during computation, which injects step discontinuities into commanded position, velocity, and acceleration. To eliminate boundary discontinuities, the sequence proposer must instead condition its action generation on the forward-projected reference state[6] of the active trajectory at the designated replanning seam t_{seam}:

$$\Delta q_{\text{reference}} = \int_{t_0}^{t_{\text{seam}}} \dot{q}_A(t)\,dt, \qquad \hat{\mathbf{x}}_{\text{seam}} = \mathbf{x}_A(t_{\text{seam}}) \tag{11.6}$$

The left-hand integral is the joint-space gap that a stale anchor would leave, the 0.040 rad of the example, while the right-hand state is the anchor the proposer uses instead. Because the projected seam state $\hat{\mathbf{x}}_{\text{seam}}$ in equation 11.6 exists strictly in reference trajectory space, external contact forces, mechanical compliance, and closed-loop tracking errors invariably displace the physical plant from this nominal point. Rather than presuming an unobservable future measurement, admission verification prior to the seam evaluates a bounded reachable-state envelope, confirming that feasible polynomial bridges and certified stopping profiles exist across all reachable states within that set. At activation, embedded MCU firmware reads fresh optical encoder measurements and state-estimator error bounds to validate the actual boundary discrepancy $(\Delta q, \Delta\dot{q}, \Delta\ddot{q})$ against that certified envelope. It then instantiates the bounded fifth-degree C^2 bridge of section 11.4.4 and checks its joint-space derivatives against actuator velocity, torque, and jerk limits within the reserved real-time tick before dispatching motor commands. If measured tracking errors or bridge derivatives exceed admitted bounds, the MCU rejects the replacement and maintains the active plan only while its tracking error, target evidence, lease duration, and stopping clearances satisfy safety invariants; otherwise, it triggers the pre-latched fallback profile. Matching two *reference* profiles guarantees derivative continuity across reference commands, but it never eliminates physical tracking errors in the underlying mechanical hardware.

Recent generative imitation learning architectures incorporate delay compensation directly into their sampling algorithms. Real-time chunking (Black, Galliker, et al. 2025) computes upcoming action sequences concurrently with active chunk execution. Because setpoints scheduled to run during the inference delay window are already committed to the actuator queue, the model fixes those initial steps to their committed values and samples subsequent actions conditioned on this prefix, applying decaying consistency weights across the overlap window. Although empirical evaluations show that control throughput remains stable under artificial latency injection, the formulation provides no formal continuity or stability guarantees. In contrast, bidirectional decoding (Y. Liu et al. 2024) samples a diverse candidate batch of trajectory chunks, scoring and selecting the proposal whose trajectory matches the previously executed actions most smoothly. Conditioning generation on the committed trajectory prefix promotes smooth kinematic transitions and reduces

6 **Smith predictor**: Smith (1957) handled known dead time in feedback control loops by projecting a plant model forward across the delay interval rather than closing feedback against delayed measurements. Future-state conditioning adapts this idea to trajectory planning by anchoring prospective trajectories to the reference state expected at the handoff seam. It also inherits the weakness of any dead-time predictor: external disturbances during the delay interval go uncompensated until physical feedback arrives, which is why the MCU must check measured joint states against strict error bounds at activation.

boundary tracking discrepancies, yet it cannot substitute for formal safety verification. Embedded microcontroller logic must still evaluate actual physical states against the admitted safety envelope before committing any new chunk to execution.

Definition 11.2: Future-state conditioning

Future-state conditioning is the practice of generating candidate motion plans anchored to the anticipated state at execution time rather than the stale state measured at plan initiation.

1. **Significance**: Offsets computational and transport latency Δt_{plan}, eliminating boundary step discontinuities at trajectory handoffs without forcing zero-velocity pauses between consecutive chunks.
2. **Distinction**: Differs from state estimation or Kalman filtering; it is a forward reference-planning technique that propagates an admitted nominal trajectory rather than filtering sensor noise.
3. **Common pitfall**: Conflating forward-projected reference states with actual physical plant positions. If external contact forces or tracking lag displace the mechanism during planning, activation remains contingent upon the MCU verifying actual encoder feedback and bridge feasibility at the handoff seam.

Trajectory replacement therefore comes down to three sizing decisions. The prediction horizon must be long enough to carry kinematically feasible motion together with a certified stopping profile, the executed prefix short enough to keep closed-loop responses to a changing environment fast, and the renewal cadence one that the pipelined compute can sustain deterministically. Perceptual evidence freshness dictates the maximum lifespan over which any target estimate justifies physical motion. Even when replacement plans arrive strictly within latency budgets, timely delivery alone cannot guarantee that physical joint actuators can track the synthesized trajectory or bridge seamlessly from their measured state.

Checkpoint 11.1: Action chunk horizon and tail latency

Verify your understanding of replacement timing budgets, tail latency exposure, and delay compensation mechanisms before examining kinodynamic feasibility:

- ☐ **Mission length and tail quantiles**: In the stated independent-handoff illustration, what is the probability that one or more missed p99 handoff deadlines occur across 3000 consecutive cycles, and why does this represent an allocation target rather than an empirical mission collision rate?
- ☐ **Horizon sizing trade-offs**: Why can a long prediction horizon coexist with a short executed prefix, and which scheduling parameter dictates accelerator inference duty?
- ☐ **Projected seam state**: Why must a trajectory synthesized from a forward-projected reference still undergo reachable-set verification at admission and measured state bridge validation at activation?

11.4 Kinodynamic Feasibility

A candidate trajectory that clears every obstacle in workspace and arrives before its deadline will still fault on the hardware if it asks a joint for more speed, torque, or current slew than the drive can supply. What an embodied model generates is a geometric and timed proposal, not permission to move. While high-level motion planners and learned policies navigate free configuration space $\mathcal{C}_{\text{free}}$, physical actuators operate under fixed electromotive, thermal, and structural limits. Kinodynamic feasibility bridges geometric intent and mechanical reality, ensuring that reference trajectories respect actuator torque limits, velocity boundaries, and transmission dynamics before the low-level controller dispatches setpoints to motor drives.

Definition 11.3: Kinodynamic feasibility envelope

Kinodynamic feasibility envelope is the set of initial states from which the machine can track a candidate trajectory, including its transition seams and terminal stop, while every joint remains within its configured position, velocity, acceleration, jerk (the rate of change of acceleration), torque, and stator current limits.

1. **Significance**: Obstacle clearance in configuration space is necessary but insufficient. At the 1 m/s TCP speed limit, a collision-free reach in the arm's weakest kinematic direction at a right-angle elbow demands 3.5 rad/s from a single joint against its 2.175 rad/s ceiling. The spatial geometry succeeds while the timing fails. The envelope defines the state region where dynamic models predict trackable motion, which admission and runtime monitors must continuously verify.
2. **Distinction**: Kinematic reachability evaluates whether a mechanism can geometrically achieve a configuration. Kinodynamic feasibility introduces time and momentum: the time derivatives that a trajectory profile implies, the dynamic torques produced through the manipulator equation, the effective inertia reflected through gear trains, and the physical stopping distance required to brake safely.
3. **Common pitfall**: Evaluating feasibility solely at discrete waypoints. The continuous-time interpolant between discrete samples can overshoot velocity bounds or demand severe torque steps even when every discrete sample satisfies constraints.

11.4.1 Rigid-body dynamics and continuous collision checks

A trajectory can stay completely clear of obstacles in $\mathcal{C}_{\text{free}}$ and still fail on the physical machine because it demands actuator torques that saturate motor inverters, induces base accelerations that tip the platform, or drives current slew rates that trip motor protection circuits. Classical kinematic motion planning checked only whether the swept volume avoided obstacles; a moving body with momentum must satisfy configuration-space clearance and time-domain dynamic limits together.

On a single joint, such as the manipulator's shoulder lifting a grasped payload, the balance of torques obeys the scalar equation of motion:

$$\tau = J_{\text{eff}}\,\ddot{q} + mgr\cos q + \tau_{\text{fric}} \tag{11.7}$$

where J_{eff} represents the effective inertia driven by the joint, m is a concentrated mass whose center of gravity lies at distance r from the joint axis, q is the joint angle measured from the horizontal plane, and τ_{fric} is friction torque opposing motion. Mechanical inertia resists angular acceleration, while gravity exerts a static load proportional to the horizontal moment arm $r\cos q$. Holding the rated 3 kg payload horizontally at the manipulator's full 0.70 m planar reach requires 20.6 N·m of static holding torque, which consumes 23.7 percent of the joint's 87 N·m torque limit before motion begins (Franka Robotics GmbH 2023). The actuator's reflected rotor inertia is the N_{gear}^2-amplified rotor term of section 2.4.3, which the seam analysis below prices for a Panda-class joint. Adding it to the payload inertia produces an effective joint inertia of 1.65 kg·m² that every commanded acceleration must accelerate. Structural link mass and distributed inertia add to this baseline. Because manufacturer datasheets rarely provide link mass distributions, empirical evaluations rely on published identified parameters (Gaz et al. 2019); an operational deployment requires system identification on the physical arm and attached tooling.

In multi-degree-of-freedom manipulators, dynamic coupling interconnects the joints. The scalar terms generalize into the matrix terms of the **manipulator equation**, derived from the Euler-Lagrange equations of motion (Craig 2005; Lynch and Park 2017):

$$\tau(t) = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) - \mathbf{J}^{\top}(\mathbf{q})\mathbf{F}_{\text{ext}} \tag{11.8}$$

Here, the symmetric, positive-definite inertia matrix $\mathbf{M}(\mathbf{q})$ generalizes scalar inertia J_{eff} from equation 11.7. Off-diagonal elements capture cross-joint inertial coupling (accelerating the elbow induces reaction torques at the shoulder), and matrix values vary continuously as the arm articulates.

The quadratic velocity term $\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}}$ aggregates centrifugal and Coriolis torques, which scale quadratically with joint velocity, while the gravity vector $\mathbf{g}(\mathbf{q})$ generalizes $mgr\cos q$ to represent configuration-dependent static holding torque. The geometric Jacobian from equation 11.1 maps an external wrench $\mathbf{F}_{\text{ext}}$ acting on the end effector into equivalent joint torques via the transpose $\mathbf{J}^{\top}(\mathbf{q})$; external contact wrenches that assist motion reduce actuator torque demands, while resistive loads increase them. Evaluating equation 11.8 along a prospective trajectory $\mathbf{q}(t)$ is the problem of **inverse dynamics**. The recursive Newton-Euler algorithm computes inverse dynamics for an n-joint serial arm in $\mathcal{O}(n)$ arithmetic operations per trajectory sample (Luh et al. 1980; Lynch and Park 2017). An admission filter scanning an N-sample candidate trajectory therefore performs $\mathcal{O}(Nn)$ operations whose worst-case execution time must fit within the system's admission latency allowance.

Before dispatching a proposed trajectory chunk, the planning runtime screens it against the kinodynamic feasibility envelope, verifying that the physical state at the handoff seam satisfies position, velocity, torque, stator current, contact force, and collision boundaries over the planning horizon. For robotic arms, the rigid-body model maps joint trajectories $\mathbf{q}(t)$, $\dot{\mathbf{q}}(t)$, and $\ddot{\mathbf{q}}(t)$ to joint torques $\tau(t)$ via equation 11.8. Those torques convert to quadrature-axis motor currents $i_{q_i}(t) = \tau_i(t)/K_{t_i}$ using the joint-effective torque constant K_{t_i}. (When using motor-side torque constants, the conversion must explicitly incorporate gear ratios and transmission mechanical efficiencies.) Initial screening by an unprivileged planner, however, does not constitute system admission. The real-time safety MCU executes an independent, deterministic verification check, admitting a bounded trajectory chunk only if configured dynamic limits and certified stop clearances hold for the verified initial state. Once admitted, the MCU monitors control tracking error, plan freshness, and stopping distance margins on every servo cycle.

Numerical optimizers can smooth geometric paths and expand obstacle clearance, but motion permission resides strictly in the admitted trajectory chunk and its resident stopping suffix. For example, the cuRobo motion-generation framework uses massively parallel GPU compute to generate collision-free, minimum-jerk trajectories across kinematic configurations (Sundaralingam et al. 2023). Its collision pipeline evaluates signed distance fields across sampled swept-sphere volumes under a linear-motion approximation, leaving continuous inter-sample swept volume evaluation as an open requirement. Following trajectory optimization, cuRobo identifies the limiting time step where joint velocity, acceleration, or jerk encounters actuator bounds, rescaling trajectory duration before re-optimizing. This uniform temporal scaling represents an initial form of the rigorous time parameterization developed in section 11.4.5. Because cuRobo's algorithmic formulation focuses on kinematic derivatives rather than full multi-body dynamics, torque boundaries, winding thermal accumulation, continuous swept-volume containment, and admission deadlines remain external system-level requirements. Under this systems architecture, the GPU trajectory generator operates as an unprivileged proposer, while the real-time safety MCU independently validates prospective trajectories and seam constraints within a strict admission deadline, latching an executable stopping continuation before authorizing motion.

11.4.2 Overshoot between samples and contracted limits

When the admission filter checks a candidate only at discrete waypoints spaced T_s apart, two consecutive joint samples $t_k = kT_s$ that sit just below the velocity ceiling $v_{\max}$ can still bracket an overspeed fault midway between them. The reason is that physical actuators track a continuous interpolant across (t_k, t_{k+1}). If that interpolator can command accelerations up to $a_{\max}$ within a sample period, an acceleration sign reversal across the midpoint lets continuous velocity rise above the values recorded at both endpoints.

Under acceleration-bounded interpolation, the maximum velocity excursion between two consecutive samples sharing equal velocity arises from a bang-bang acceleration reversal: $+a_{\max}$ applied over $[0, T_s/2]$, followed immediately by $-a_{\max}$ over $[T_s/2, T_s]$ to restore endpoint velocity. The velocity excursion reaches its maximum at the midpoint, corresponding to the integrated area of the initial acceleration phase:

$$\Delta\dot{q} = a_{\max} \cdot \frac{T_s}{2} = \frac{1}{2} a_{\max} T_s \tag{11.9}$$

As equation 11.9 demonstrates, two consecutive waypoint samples evaluated just beneath velocity ceiling $v_{\max}$ can bracket an interior velocity peak that exceeds the limit, tripping drive overspeed

protection between individually valid samples. To ensure continuous-time velocity compliance without evaluating an infinite continuum of points, the admission filter tests discrete waypoints against an analytically contracted velocity threshold:

$$|\dot{q}_k| \le v_{\max} - \frac{1}{2} a_{\max} T_s \tag{11.10}$$

The shaded acceleration area in figure 11.3 becomes the velocity rise between the sampled endpoints. Placing both endpoints one full rise below the physical ceiling keeps the continuous peak at that ceiling.

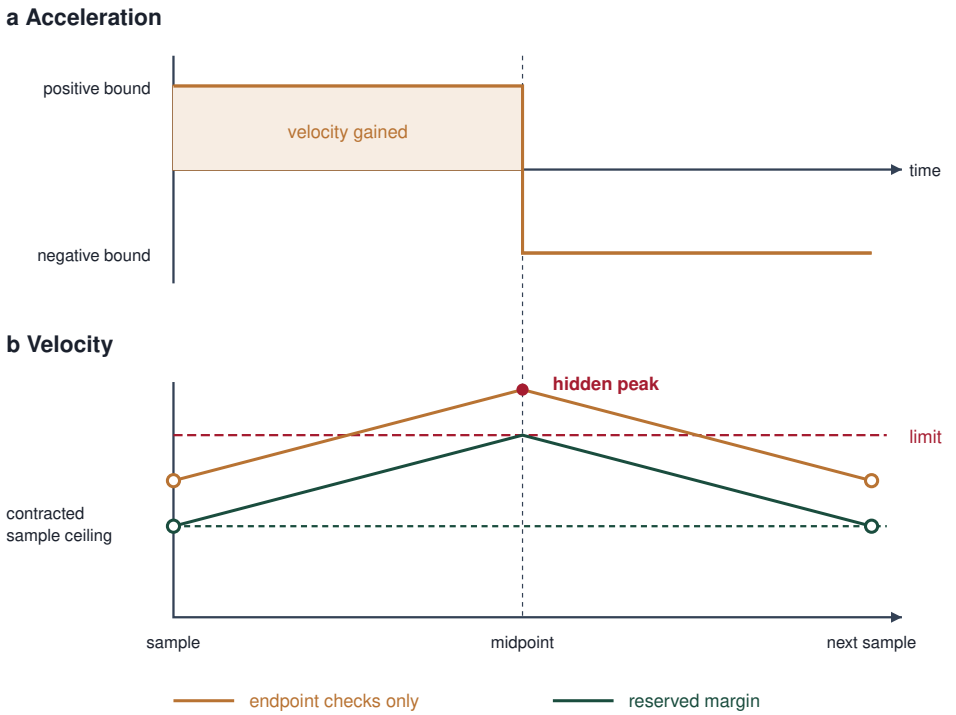


Figure 11.3: A Velocity Peak Between Valid Samples: Positive acceleration over the first half of a sample interval raises velocity; the equal negative phase restores the original endpoint velocity. The bronze trajectory passes both endpoint checks but crosses the velocity limit. The pine trajectory uses the contracted sample ceiling, leaving the reserve derived in equation 11.9. The schematic assumes an acceleration bound and an instantaneous sign reversal; it does not impose a finite jerk bound.

The endpoint check succeeds only when the interpolant obeys the assumptions used to reserve the margin. A sampled value below the physical ceiling alone supplies no such guarantee.

This contraction creates a safety margin that absorbs the worst-case overshoot any acceleration-bounded interpolator can generate across a sample period T_s. A smaller reserve requires additional boundary conditions. If consecutive samples have equal velocity and zero acceleration, and the interpolator enforces $j_{\max} \le 4a_{\max}/T_s$, the extremal reversal consists of a positive triangular acceleration pulse followed by its negative counterpart, each of base $T_s/2$. Their peaks reach $a_{\max}$ only at equality in the jerk condition, so $\frac{1}{4} a_{\max} T_s$ is a conservative velocity reserve for this restricted interpolation. A jerk bound alone does not justify that reserve when endpoint accelerations are nonzero. For polynomial spline interpolants, the admission filter can instead compute exact interior extrema analytically before certifying continuous-time feasibility.

11.4.3 Seam acceleration jumps and reflected rotor inertia

When the arm hands off from one action chunk to the next while moving, two chunks that are each feasible in isolation can still form an infeasible seam between them. Matching joint positions alone

(C^0 continuity) leaves an unconstrained velocity step across the boundary. Matching both position and velocity (C^1 continuity) eliminates velocity jumps but leaves an instantaneous acceleration discontinuity, which demands an immediate step in joint torque. Under a rigid-inertia approximation, commanding a velocity step $\Delta\dot{q}$ across a single servo period T_s requires an inertial torque of:

$$\tau_{\text{request}} = J_{\text{eff}} \frac{\Delta\dot{q}}{T_s} \tag{11.11}$$

where $J_{\text{eff}} = J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}}$ in equation 11.11 represents the total effective inertia referenced to the joint output. This term adds link and payload inertia J_{load} to the reflected rotor inertia established in section 2.4.3, the motor rotor inertia J_{rotor} seen through gear reduction ratio N_{gear}. Because the rotor turns N_{gear} times faster than the joint, its kinetic energy, written in joint velocity, carries the factor N_{gear}^2:

$$E_k = \frac{1}{2} J_{\text{rotor}} \omega_m^2 = \frac{1}{2} J_{\text{rotor}} (N_{\text{gear}} \dot{q})^2 = \frac{1}{2} (N_{\text{gear}}^2 J_{\text{rotor}}) \dot{q}^2$$

In precision robotic manipulators equipped with high-ratio strain-wave gearing[7] ($N_{\text{gear}} = 50$–100), reflected rotor inertia is magnified by a factor of $2{,}500$ to $10{,}000$. It can then rival or exceed link inertia, most often for an unloaded arm or a folded pose, although a rated payload held at full reach outweighs it. In the one-joint example the payload supplies most of the 1.65 kg·m² total, against 0.18 kg·m² from the rotor. Either way, the reflected term sits in every joint's J_{eff} regardless of payload, and on its own it makes seam torque spikes large. At a C^0 boundary, an instantaneous velocity step implies an idealized Dirac acceleration impulse $\delta(t)$, which no physical motor drive can supply. At a C^1 boundary, an acceleration jump $\Delta\ddot{q}$ demands a step change in electromagnetic torque, $\Delta\tau = J_{\text{eff}} \Delta\ddot{q}$. Such abrupt torque transients excite structural resonance in flexible transmissions, accelerate mechanical wear, and degrade control tracking. On the strain-wave joint priced in the notebook below, an unblended one-tick velocity step of 0.2 rad/s asks the reflected rotor inertia for a 36.7 N·m torque pulse, while a 50 ms C^2 quintic bridge closing the same gap peaks at 1.1 N·m (figure 11.4). Commercial industrial arms enforce this exact discipline at their low-level control interfaces. For example, the Franka Research 3 arm samples setpoints at 1 kHz, comparing each incoming command against the previous cycle. If inter-cycle setpoint differences exceed factory derivative thresholds, the drive immediately aborts motion with an unrecoverable discontinuity fault (such as `joint_motion_generator_velocity_discontinuity`) (Franka Robotics GmbH 2026, 2023). An unblended seam can therefore abort nominal tracking before repeated transitions accumulate mechanical fatigue. Physical stopping still requires the drive's qualified response delay and deceleration profile.

[7] **Strain-wave (harmonic) gearing**: In the mechanism Musser (1959) patented, an elliptical wave generator flexes a thin toothed cup, the flexspline, so that it meshes with a rigid ring carrying two more teeth, and each turn of the generator advances the cup by only that difference. The result is a high reduction in one compact stage with almost no backlash. The flexspline is also the joint's main compliance, so the same gearing that multiplies rotor inertia by N_{gear}^2 supplies the compliant coupling of the two-inertia model below.

When the arm carries an open cup of liquid, steady velocity does not spill the contents, whereas rapid acceleration transitions and high jerk provoke sloshing and spillage (section 11.5.2 models this deceleration threshold directly). A trajectory interface that continuously matches position, velocity, and acceleration constitutes a C^2 **spline seam**. Enforcing C^2 continuity eliminates reference acceleration steps and prevents the associated torque jump $\Delta\tau = J_{\text{eff}} \Delta\ddot{q}$. Concurrently, bounding trajectory **jerk** ($j_{\text{joint}} = \dot{a} = \dddot{x}$) limits the rate at which joint acceleration changes. Neither mathematical property alone guarantees payload stability or mechanical longevity, however; an interpolating bridge that matches boundary conditions perfectly can still violate velocity, jerk, actuator torque, or obstacle clearance limits in its interior.

Mechanical transmissions govern how actuator torque couples into physical link motion. In a standard two-inertia drivetrain comprising load inertia J_{load}, reflected rotor inertia $J_{\text{ref}} = N_{\text{gear}}^2 J_{\text{rotor}}$, and torsional coupling stiffness K_{joint}, the antiresonance and resonance frequencies (P. Li et al. 2024) are:

$$f_{\text{anti}} = \frac{1}{2\pi} \sqrt{\frac{K_{\text{joint}}}{J_{\text{load}}}}, \qquad f_{\text{res}} = \frac{1}{2\pi} \sqrt{K_{\text{joint}} \left(\frac{1}{J_{\text{load}}} + \frac{1}{J_{\text{ref}}} \right)}$$

In robot joints constructed with strain-wave gearing, large reflected inertia coupled with flexspline torsional compliance amplifies high-frequency setpoint harmonics. An admission checker must therefore verify prospective bridges against the joint's identified, load-dependent vibrational modes, evaluating derivative extrema, peak torques, and drive current limits against empirical structural

parameters. The two frequencies reach the checker through the bridge duration. A short bridge can excite these modes, but duration alone does not determine its spectral content or the joint's response. Identified modal frequencies and damping therefore inform qualified bridge-duration and derivative limits alongside the torque-rate limit. Structural stiffness, resonance modes, and admissible jerk envelopes never transfer between robot platforms without direct system identification.

11.4.4 Quintic splines and C^2 boundary matching

Closing the position, velocity, and acceleration gap between retiring chunk A and incoming chunk B over a finite blend duration T_{blend} requires online C^2 trajectory spline blending. The blend duration is a physical execution interval, distinct from the computational blend deadline t_{blend}. Blending synthesizes a dedicated polynomial bridge $q_{\text{blend}}(t)$, which must undergo the exact same kinodynamic feasibility checks as the primary chunks rather than acting as a cosmetic filter.[8] A localized seam bridge must satisfy position, velocity, and acceleration constraints at both boundary states, which gives six independent boundary conditions. A cubic polynomial has only four degrees of freedom and cannot satisfy them. A fifth-degree quintic polynomial provides the six required coefficients. This localized endpoint-matching formulation differs from fitting a globally continuous cubic spline across an entire offline sequence of knots.

The six boundary conditions are

$$q(0) = q_0, \quad \dot{q}(0) = \dot{q}_0, \quad \ddot{q}(0) = \ddot{q}_0$$
$$q(T_{\text{blend}}) = q_1, \quad \dot{q}(T_{\text{blend}}) = \dot{q}_1, \quad \ddot{q}(T_{\text{blend}}) = \ddot{q}_1$$

where $(q_0, \dot{q}_0, \ddot{q}_0)$ is the terminal physical state of chunk A at the seam instant, and $(q_1, \dot{q}_1, \ddot{q}_1)$ is the target state on incoming chunk B at $t = T_{\text{blend}}$. The six conditions fix the six coefficients $(a_0 \dots a_5)$ of the fifth-degree polynomial:

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

Evaluating boundary conditions at $t = 0$ directly yields the first three coefficients: $a_0 = q_0$, $a_1 = \dot{q}_0$, and $a_2 = \frac{1}{2}\ddot{q}_0$. The remaining three coefficients $[a_3, a_4, a_5]^\top$ satisfy the 3×3 linear system evaluated at duration $T = T_{\text{blend}}$:

$$\begin{bmatrix} T_{\text{blend}}^3 & T_{\text{blend}}^4 & T_{\text{blend}}^5 \\ 3T_{\text{blend}}^2 & 4T_{\text{blend}}^3 & 5T_{\text{blend}}^4 \\ 6T_{\text{blend}} & 12T_{\text{blend}}^2 & 20T_{\text{blend}}^3 \end{bmatrix} \begin{bmatrix} a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} q_1 - \left(q_0 + \dot{q}_0 T_{\text{blend}} + \frac{1}{2}\ddot{q}_0 T_{\text{blend}}^2\right) \\ \dot{q}_1 - (\dot{q}_0 + \ddot{q}_0 T_{\text{blend}}) \\ \ddot{q}_1 - \ddot{q}_0 \end{bmatrix} \tag{11.12}$$

Analytically inverting equation 11.12 yields closed-form expressions for $[a_3, a_4, a_5]^\top$, given in section B.5, that require deterministic arithmetic operations per joint. Differentiating $q(t)$ three times produces the continuous commanded jerk profile:

$$j_{\text{joint}}(t) = \dddot{q}(t) = 6a_3 + 24a_4 t + 60a_5 t^2 \tag{11.13}$$

Because $j_{\text{joint}}(t)$ in equation 11.13 is a quadratic function of time, its extrema over the interval $[0, T]$ occur exclusively at the domain boundaries ($t = 0$ or $t = T$) or at the interior stationary point $t^* = -a_4/(5a_5)$. The interior point applies when $a_5 \neq 0$ and $t^* \in (0, T)$. If $a_5 = 0$, jerk varies linearly or remains constant, and evaluating the boundaries alone suffices. The safety microcontroller can therefore compute exact peak jerk $|j_{\text{peak}}|$ analytically in $\mathcal{O}(1)$ operations without executing iterative numerical line searches. An identical principle governs lower derivatives: acceleration extrema occur at the roots of the quadratic jerk, and velocity extrema occur at the roots of cubic acceleration, both solvable in closed form.

Evaluating coupled joint torques across the bridge presents an entirely different challenge. Along a polynomial trajectory $\mathbf{q}(t)$, the right-hand side of equation 11.8 is non-polynomial because the inertia matrix $\mathbf{M}(\mathbf{q})$, Coriolis matrix $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$, and gravity vector $\mathbf{g}(\mathbf{q})$ contain transcendental trigonometric functions of joint positions. Consequently, peak torque possesses no closed-form analytic solution. The safety monitor therefore evaluates torque at discrete sample waypoints and bounds inter-sample torque variation using the contraction principles established in section 11.4.2.

[8] **Minimum jerk**: Flash and Hogan (1985) found that unconstrained human reaching follows the trajectory that minimizes integrated squared jerk, which for a rest-to-rest reach is a quintic polynomial in time. The jerk term of equation 11.20 inherits that smoothness but not a bound, so a minimum-jerk bridge can still exceed the drive's limits, and its extrema are checked through equation 11.13.

Bounding trajectory jerk directly protects actuator health, preserves motor inverter voltage headroom, and prevents premature transmission wear. When a trajectory generator outputs time-parameterized splines, it transforms kinematic waypoints into continuous reference functions whose derivatives map directly onto physical electrical and mechanical states. In permanent-magnet synchronous motors operated under field-oriented control, joint torque rate couples to jerk through the time derivative of the manipulator equation: $\dot{\tau}(t) = \mathbf{M}(\mathbf{q})\dddot{\mathbf{q}}(t) + \dot{\mathbf{M}}(\mathbf{q})\ddot{\mathbf{q}}(t) + \ldots$. For a single joint with constant effective inertia $J_{\text{eff}} = J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}}$, neglecting changing gravity, friction, and external-load torques reduces the output torque rate to $\dot{\tau} = J_{\text{eff}} j_{\text{joint}}(t)$. Defining a joint-effective torque constant K_t such that output torque satisfies $\tau = K_t i_q$, commanding an output torque rate $\dot{\tau}$ demands a proportional quadrature-axis stator current slew rate:

$$\frac{di_q}{dt} = \frac{J_{\text{eff}}}{K_t} j_{\text{joint}}(t)$$

Here, K_t incorporates both the gear reduction ratio and transmission mechanical efficiency; directly substituting a raw motor-side torque constant without gear scaling would incorrectly pair motor torque with output-side inertia. Motor winding inductance L_{phase} and available DC bus voltage impose hard electrical limits on current slew rate. During positive motoring operations with resistive and back-electromotive force (back-EMF) drops, a simplified necessary condition for voltage headroom is:

$$V_{\text{bus}} \geq L_{\text{phase}} \frac{di_q}{dt} + R_{\text{phase}}\, i_q + K_e \omega_e \tag{11.14}$$

where R_{phase} represents phase winding resistance, K_e is the motor back-EMF constant, and ω_e denotes motor electrical speed. In operational motor drives, space-vector pulse-width modulation limits the maximum usable quadrature-axis voltage to a fraction of the total DC bus voltage. A complete electrical feasibility check must evaluate available voltage against the fully coupled, signed motor equations, including regenerative braking and inductive flyback, rather than treat a scalar approximation as sufficient qualification.

↩ Phase inductance, back-EMF, and the current-slew limits they impose are established in section 2.4.2.

An acceleration discontinuity in a motion plan demands an instantaneous step in electromagnetic torque and stator current. Because finite bus voltage and nonzero winding inductance prevent physical inverters from supplying infinite current slew rates (di_q/dt), the physical actuator lags the commanded reference. The resulting tracking error, inverter saturation, and thermal protection response depend on drive gains and mechanical compliance. An unbounded mathematical jerk demand does not instantly cause an overcurrent shutdown, but it induces severe tracking lag, mechanical vibration, and drive stress. A valid bridge must satisfy both continuous mathematical derivative bounds and the physical bandwidth limits of the actuator drive.

Satisfying the six boundary conditions guarantees acceleration continuity at chunk interfaces, but it does not guarantee that interior jerk, torque, or current slew rates remain within actuator ratings. The admission filter evaluates closed-form derivative extrema from equation 11.13 against the electrical constraints of equation 11.14, checks sampled multi-body torques against inter-sample contraction bounds, and rejects any bridge violating physical thresholds. Evaluating a fixed-degree polynomial requires bounded arithmetic operations per joint. Spline coefficient synthesis and trajectory extrema verification take place prior to trajectory dispatch; the qualified worst-case execution bound, accounting for memory access latency and register operations, must remain strictly within the configured admission deadline.

While a local quintic bridge resolves seam continuity through explicit boundary matching, global trajectory representations provide alternative verification properties. **Uniform B-splines**[9] parameterize joint trajectories as $\mathbf{q}(t) = \sum_{i=0}^{L} \mathbf{p}_i B_{i,p}(t)$, where $\mathbf{p}_i$ represent geometric control points, $B_{i,p}$ are polynomial basis functions of degree p, and ξ_i denote knot timestamps. Using simple knot sequences, uniform B-splines ensure C^{p-1} continuity by construction. Furthermore, the **convex hull property** ensures that the continuous trajectory lies entirely within the convex hull formed by its control points, enabling spatial collision checkers to prune entire collision volumes without dense sampling. Differentiating a B-spline yields a derivative spline whose control points are $\mathbf{v}_i = \frac{p}{\xi_{i+p+1} - \xi_{i+1}}(\mathbf{p}_{i+1} - \mathbf{p}_i)$, allowing difference operations on control points to establish conservative upper bounds on velocity, acceleration, and jerk. These geometric bounds accelerate screening,

[9] **Spline** (a draftsman's bending strip): The word named the thin flexible strip that shipwrights and draftsmen bent through fixed points, held in place by lead weights, to draw a fair curve. Schoenberg (1946) borrowed it for piecewise polynomials, and Boor (1978) made B-splines practical to compute with a stable recursion. The convex-hull property that construction supplies is what lets a checker bound a whole segment's derivatives from a few control-point differences.

though they do not replace full dynamic inverse dynamics or swept-volume collision checks. In streaming replanning architectures, the trajectory representation must also support rapid coefficient joining within the real-time admission deadline; table 11.1 separates the continuity supplied by each representation from the dynamic checks still required before execution.

Table 11.1: Trajectory representations and required checks: Smooth parameterization helps form a candidate but does not by itself prove dynamic feasibility.

Representation	Native seam property	Checks before execution
Cubic polynomial (C^1)	Matches $q, \dot{q}$; adjoining acceleration need not match.	Check acceleration matching and interior limits; position and velocity matching alone are insufficient.
Quintic bridge (C^2)	Matches $q, \dot{q}, \ddot{q}$ at both ends; jerk is quadratic within each bridge.	Check interior extrema, torque rates, inverter headroom, and clearance.
Septic polynomial (C^3)	Can match derivatives through jerk at both endpoints.	Check interior derivatives and actuator limits; endpoint matching does not bound their magnitudes.
Uniform B-spline (degree p)	C^{p-1} continuity; convex hull bounds spatial curve.	Constrain control point differences for velocity/jerk; check global knot system.
Neural action chunk (ACT, Diffusion)	Discrete samples; continuity depends on interpolation and seam constraints.	Project onto feasibility manifold via numerical solver or C^2 quintic bridge; latch stop.
Online jerk-limited generator	Reaches a target $(q, \dot{q}, \ddot{q})$ from any current state within per-axis $v_{\max}$, $a_{\max}$, $j_{\max}$ bounds; the generator picks the duration.	Check coupled torque, clearance, evidence age, and the configured limits themselves; kinematic feasibility by construction is not admission.

Napkin Math 11.2: The seam spike

Problem: At a replanning seam the incoming chunk's reference runs 0.2 rad/s faster than the active one. Should the joint close that gap in one servo tick or over a C^2 bridge, and what does each ask of the drive?

Variables:

- Joint: a CSG-25-50 strain-wave gear driven by a TQ-RoboDrive ILM 70x18 motor, a Panda-class stand-in because Franka does not publish the Panda's joint internals (datasheet parts); reflected rotor inertia $J_{\text{eff}} = N_{\text{gear}}^2 J_{\text{rotor}} = 0.18$ kg·m² (derived; link and payload inertia excluded, so here J_{eff} is the reflected inertia J_{ref} of section 2.4.3 alone)
- Ratings 51 N·m rated and 127 N·m repeated peak (datasheet); the arm's torque-rate limit 1,000 N·m/s (datasheet)
- Velocity gap $\Delta\dot{q} = 0.2$ rad/s (illustrative) and servo tick $T_s = 1$ ms (chosen)
- Bridge duration $T_{\text{blend}} = 50$ ms (illustrative), zero acceleration at both ends, ending on chunk B's reference, a position residual $\Delta\dot{q}\,T_{\text{blend}}/2 = 0.005$ rad (chosen)

Math:

1. **One tick** (equation 11.11): $J_{\text{eff}}\,\Delta\dot{q}/T_s = 36.7$ N·m, below the rating. Reaching it within the tick needs at least 36.7 N·m per 1 ms, or 36,700 N·m/s, 36.7× the torque-rate limit.
2. **Quintic bridge** (equation 11.12): Because a symmetric velocity transition from $\dot{q}_0$ to $\dot{q}_0 + \Delta\dot{q}$ averages $\dot{q}_0 + \frac{1}{2}\Delta\dot{q}$ across the bridge, the joint advances by $\Delta\dot{q}\,T_{\text{blend}}/2$ beyond chunk A's constant-velocity extrapolation; with that position residual and zero boundary accelerations, the system gives $a_5 = 0$, $a_4 = -\Delta\dot{q}/(2T_{\text{blend}}^3) = -800$ rad/s^4, and $a_3 = \Delta\dot{q}/T_{\text{blend}}^2 = 80$ rad/s^3.
3. **Bridge peaks**: Acceleration peaks at mid-bridge at $1.5\,\Delta\dot{q}/T_{\text{blend}} = 6$ rad/s², a torque of 1.1 N·m. Jerk (equation 11.13) peaks at both ends at $6a_3 = 480$ rad/s^3, a torque rate $J_{\text{eff}}\,j_{\text{joint}}$ of 88.1 N·m/s, 8.8 percent of the limit.

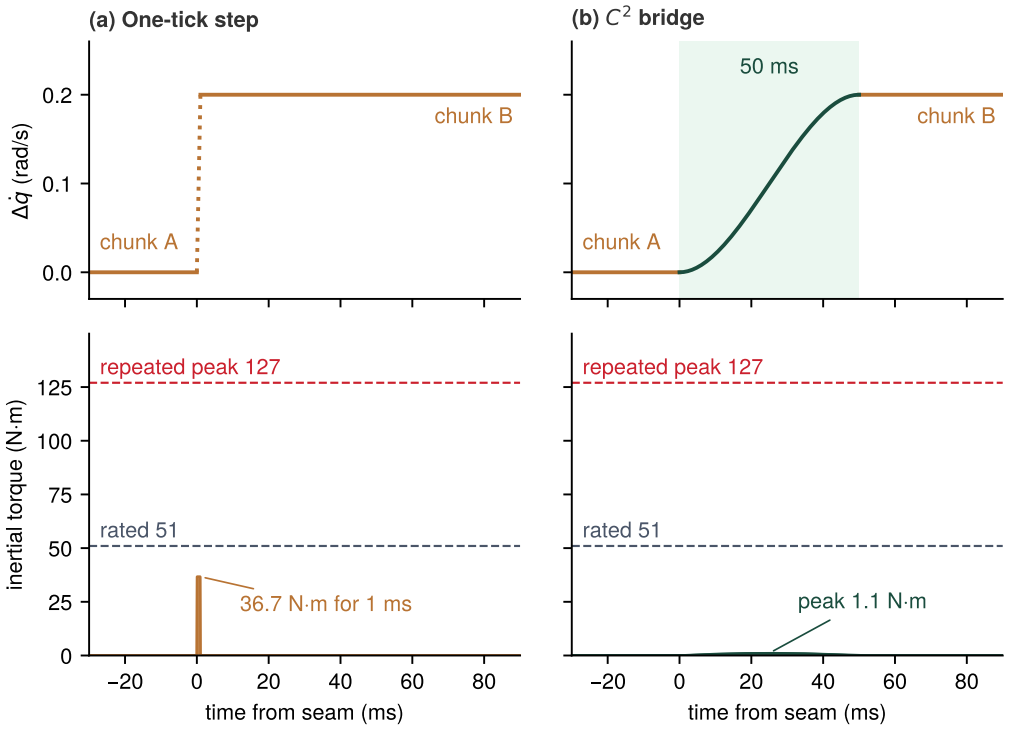


Figure 11.4: Seam demand and checked bridge: Both columns close the same 0.2 rad/s reference velocity gap on the notebook's strain-wave joint, with torque drawn to one scale against its datasheet ratings. The one-tick step (left) asks the reflected rotor inertia for 36.7 N·m, below the 51 N·m rating, so the torque-magnitude check alone passes. The notebook's torque-rate calculation rejects the transition, and the drive also rejects its discontinuity. The 50 ms C^2 bridge (right) peaks at 1.1 N·m, but its interior jerk, full joint torque, and response still require checking.

4. **Half the duration**: Peak torque scales as $1/T_{\text{blend}}$ and jerk as $1/T_{\text{blend}}^2$, so a bridge half as long needs 2.2 N·m, 1,920 rad/s^3, and 352.3 N·m/s, still inside the limit.

Result: The one-tick step passes a torque check and asks for 36.7× the torque-rate limit; the bridge passes both with a 33.3× smaller torque peak.

Systems insight: Evaluating torque magnitude alone is an inadequate feasibility screen at a trajectory seam. Actuator drives fail on the rate of change of torque, which grows quadratically ($1/T_{\text{blend}}^2$) as blending duration compresses. When late trajectory admission forces the system to compress bridge duration T_{blend}, required jerk and torque slew rates increase fourfold for every twofold reduction in duration. An admission monitor must explicitly re-evaluate the compressed bridge rather than inheriting validity from the uncompressed nominal profile. Furthermore, link inertia, payload mass, gravity, and joint friction compound this inertial baseline, requiring independent peak torque and thermal budget verification. The torque verdict of step 1 depends on that exclusion. With the payload inertia of equation 11.7 included, 1.65 kg·m^2 in place of 0.18 kg·m^2, the one-tick request grows by the same ratio and exceeds even the 127 N·m repeated peak, so the one-tick step fails the torque check as well.

High-frequency chatter in learned policy outputs represents an identical physical hazard under a different guise. Stochastic sampling policies, such as diffusion policies or action-chunking transformers, often generate consecutive trajectory chunks where individual waypoints satisfy static position and velocity limits, yet commanded accelerations reverse sign erratically between steps. Each acceleration reversal demands an immediate torque reversal from the actuator, triggering

rapid current transitions and severe $i_q^2 R_{\text{phase}}$ resistive heating in the motor windings. In physical AI systems, engineers must treat this high-frequency variance as a direct cause of thermal rise and transmission fatigue rather than cosmetic jitter. A trajectory candidate that excites structural resonances or exceeds drive thermal dissipation limits is infeasible, regardless of how closely its geometric path resembles demonstration data.

11.4.5 Time parameterization under torque and jerk bounds

A collision-free curve through configuration space says where the arm goes. It says nothing about how fast the arm may travel along that curve without saturating a joint's torque limit on the way up or demanding an impossible torque step when it switches to braking. Geometric search and neural proposal generators determine only the spatial path of motion: a curve $\mathbf{q}(s)$ parameterized by scalar path coordinate $s \in [0,1]$. The timing law $s(t)$ governs the physical motion, determining every joint velocity, acceleration, and torque required along the trajectory. Establishing a valid timing profile $s(t)$ subject to physical actuator limits is the problem of **time parameterization**. Applying the chain rule to the joint coordinates yields:

$$\dot{\mathbf{q}} = \mathbf{q}'(s)\,\dot{s}, \qquad \ddot{\mathbf{q}} = \mathbf{q}'(s)\,\ddot{s} + \mathbf{q}''(s)\,\dot{s}^2$$

where primes denote differentiation with respect to path coordinate s ($d\mathbf{q}/ds$ and $d^2\mathbf{q}/ds^2$). Substituting these path derivatives into the unconstrained manipulator equation from equation 11.8 expresses joint torque as a function of three path-dependent terms:

$$\tau = \mathbf{a}_\tau(s)\,\ddot{s} + \mathbf{b}_\tau(s)\,\dot{s}^2 + \mathbf{g}(s) \tag{11.15}$$

with vector coefficients $\mathbf{a}_\tau(s) = \mathbf{M}(\mathbf{q})\mathbf{q}'(s)$, $\mathbf{b}_\tau(s) = \mathbf{M}(\mathbf{q})\mathbf{q}''(s) + \mathbf{C}(\mathbf{q},\mathbf{q}')\,\mathbf{q}'(s)$, and $\mathbf{g}(s) = \mathbf{g}(\mathbf{q})$. Each actuator torque bound $|\tau_i| \le \tau_{\max_i}$ translates into an admissible interval of path acceleration, $\ddot{s}_{\min}(s,\dot{s}) \le \ddot{s} \le \ddot{s}_{\max}(s,\dot{s})$, at every coordinate of the **phase plane** $(s,\dot{s})$. Actuator velocity and acceleration limits introduce equivalent bounding intervals. As path velocity $\dot{s}$ increases, the centrifugal and Coriolis term $\mathbf{b}_\tau(s)\dot{s}^2$ grows quadratically until the admissible acceleration interval closes entirely ($\ddot{s}_{\min} = \ddot{s}_{\max}$). The locus of phase-plane points where this interval collapses defines the maximum velocity curve, representing an upper speed envelope that no feasible parameterization can exceed.

Because total motion duration is $T = \int_0^1 \frac{1}{\dot{s}}\,ds$, the minimum-time profile maintains the highest permissible path velocity $\dot{s}$ at every point along the path. Under torque constraints alone, the time-optimal solution takes a bang-bang form (Bobrow et al. 1985). The system accelerates along the upper bound $\ddot{s}_{\max}$ until its phase-plane trajectory encounters a braking curve integrated backward from the goal at lower bound $\ddot{s}_{\min}$. It then decelerates along $\ddot{s}_{\min}$ to reach the target at rest. Switch points mark the phase-plane coordinates where the trajectory transitions between maximum acceleration and maximum braking. The TOPP-RA algorithm (Pham and Pham 2018) solves this phase-plane parameterization efficiently by transforming it into a convex optimization problem. By substituting squared path velocity $x = \dot{s}^2$ and path acceleration $u = \ddot{s}$, the torque constraints in equation 11.15 and the differential step equation $x_{i+1} = x_i + 2\,\Delta s\,u_i$ become linear across a grid of N_{grid} path waypoints. (This linear step relation follows from $dx/ds = 2\dot{s}\,(d\dot{s}/ds) = 2\ddot{s}$, matching the kinematic equation $v^2 = 2ad$). A backward integration pass computes the controllable interval of x at each grid point from which the terminal state remains reachable, solving a single linear program per point. A subsequent forward pass selects the largest admissible acceleration u at each step. The complete pass runs in $\mathcal{O}(N_{\text{grid}})$ time, and an empty feasible interval at any point provides constructive proof that no parameterization satisfies the bounds on that grid. Kinematic variants that constrain only joint velocity and acceleration compute this timing structure with lower overhead (Kunz and Stilman 2012). The Kunz and Stilman algorithm serves as the default time-parameterization method in MoveIt 2, which can apply a downstream jerk-limiting filter (MoveIt 2 Documentation 2026).

Napkin Math 11.3: Time-optimal swing of one joint

Problem: The shoulder lifts the rated payload at full planar reach through 0.5 rad, from rest to rest. How fast can it swing, and what does the fastest timing ask of the drive?

Variables:

- Swing $\Delta q = 0.5$ rad (illustrative), with the joint angle itself as the path parameter ($s = q$, measured from the start)
- Inertia $J_{\text{eff}} = 1.65$ kg·m² and payload gravity torque $\tau_g = 20.6$ N·m, held at its horizontal value (derived, payload only)
- Joint limits: torque $\tau_{\max} = 87$ N·m, acceleration 7.5 rad/s², velocity 2.175 rad/s, torque rate $\dot{\tau}_{\max} = 1000$ N·m/s (datasheet (Franka Robotics GmbH 2023))

Math:

1. **Binding acceleration**: The torque limit allows $(\tau_{\max} - \tau_g)/J_{\text{eff}} = 40.2$ rad/s² upward and $(\tau_{\max} + \tau_g)/J_{\text{eff}} = 65.1$ rad/s² braking, an asymmetry that gravity creates. The published 7.5 rad/s² is tighter than both, so it binds in both directions.
2. **Time-optimal swing**: The accelerating curve $\dot{s}^2 = 2a_{\max}s$ meets the braking curve at mid-swing, where speed peaks at $\sqrt{a_{\max}\,\Delta q} = 1.94$ rad/s, below the velocity limit. The swing takes $2\sqrt{\Delta q / a_{\max}} = 516.4$ ms.
3. **Switch-point step**: Acceleration reverses from $+a_{\max}$ to $-a_{\max}$, a torque step $2J_{\text{eff}}a_{\max} = 24.8$ N·m, which needs at least 24.8 ms at the torque-rate limit. That limit bounds jerk at $\dot{\tau}_{\max}/J_{\text{eff}} = 604.8$ rad/s³.
4. **Jerk-limited swing**: A rest-to-rest double-S profile with no cruise covers $\Delta q = a_{\max}\,T_{\text{acc}}\,(T_{\text{acc}} + a_{\max}/j_{\max})$ in $T = 2(T_{\text{acc}} + a_{\max}/j_{\max})$. Because the accelerating half of the swing consists of a constant-acceleration hold framed by two jerk ramps of duration $a_{\max}/j_{\max}$ (one ramping acceleration up to $a_{\max}$ and one ramping it back to zero at mid-swing), $T_{\text{acc}} = T/2 - a_{\max}/j_{\max}$ is half the swing less one jerk ramp (the constant-acceleration hold plus one ramp), giving the trapezoidal acceleration pulse an area equal to the peak speed $a_{\max}T_{\text{acc}}$. With $a_{\max}/j_{\max} = 12.4$ ms, the swing needs $T_{\text{acc}} = 252.1$ ms and takes 528.9 ms.

Systems insight: Enforcing the torque-rate limit incurs an execution penalty of 12.6 ms, a modest 2.4 percent increase in motion duration, while eliminating the instantaneous torque step entirely. For the payload alone, torque was not the binding constraint; the published kinematic limits governed the trajectory, though identified link inertia would narrow this torque headroom. A valid time parameterization must incorporate every constraint capable of binding in practice. The theoretical time-optimal profile without a jerk bound is an unexecutable mathematical abstraction that physical drives reject.

In the phase plane the two profiles differ only near their corners (figure 11.5). The torque limits alone would permit steeper accelerating and braking parabolas, but the published 7.5 rad/s² acceleration limit contracts both arcs, and the bang-bang path rides them to its 1.94 rad/s switch point, well below the velocity limit. The jerk-limited path rounds the start, the apex, and the finish just inside those arcs. That rounding is where its extra 2.4 percent of duration goes, and it is what removes the 24.8 N·m torque step at the switch point.

The time-optimal bang-bang solution produces precisely the derivative discontinuities that safe seam execution forbids. Every transition between maximum acceleration and maximum braking represents a step change in acceleration, demanding an instantaneous step in electromagnetic torque that actuator drives cannot deliver within finite voltage and current slew limits. Incorporating an explicit jerk bound transforms the problem into a third-order trajectory formulation. As the shoulder swing notebook shows, the time cost of bounding jerk is small next to the torque step it removes. Modern motion pipelines therefore parameterize paths under coupled torque and velocity limits and enforce explicit jerk bounds downstream, or constrain jerk directly during parameterization.

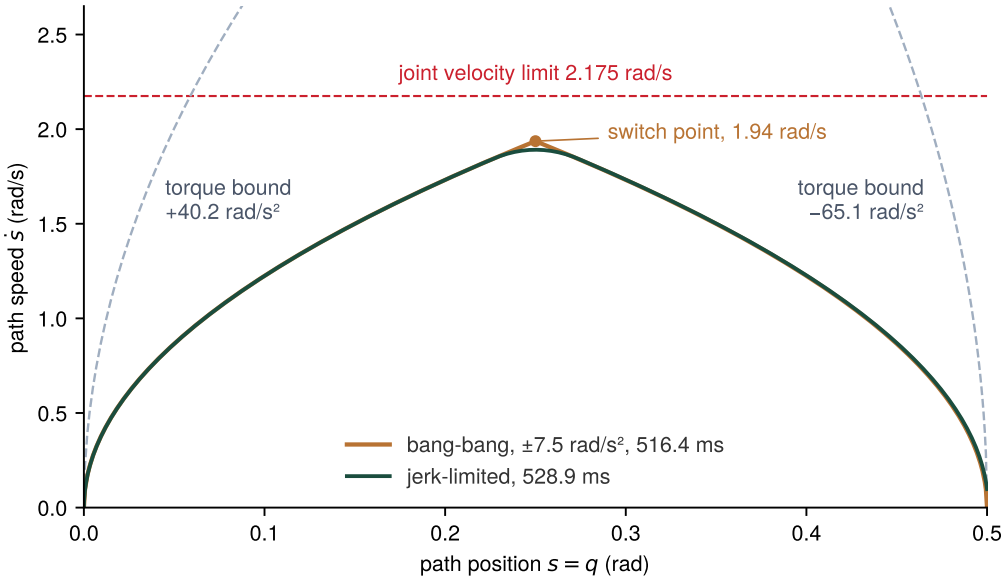


Figure 11.5: Phase plane of the shoulder swing: Path speed $\dot{s}$ against path position s for the 0.5 rad rest-to-rest swing. The joint's 7.5 rad/s² acceleration limit, tighter than either torque-derived bound, sets both the accelerating and braking curves; the bang-bang path follows them to the switch point at 1.94 rad/s, below the velocity limit, in 516.4 ms. The jerk-limited path stays inside it and takes 528.9 ms, 2.4 percent longer, with no torque step at the switch.

An identical jerk-limited profile, synthesized from the robot's verified state, resolves both seam blending and safe stopping. An **online trajectory generator** receives the instantaneous position, velocity, and acceleration of each joint together with a target boundary state, returning a time-optimal profile that satisfies per-axis velocity, acceleration, and jerk limits (Kröger and Wahl 2010). For example, the Ruckig algorithm (Berscheid and Kröger 2021) evaluates arbitrary initial and target states across seven distinct constant-jerk phases, calculating profile durations analytically for most trajectory classes and using a bounded polynomial root solver for asymmetric edge cases. In published benchmarks, Ruckig achieved an average execution latency of 19.8 µs and a worst-case latency of 123 µs for 7 axes on a desktop processor, making it fast enough to re-solve within a 1 kHz control cycle. This formulation differs structurally from the fixed-duration quintic bridge in section 11.4.4. The quintic approach fixes blending duration T_{blend} upfront and risks exceeding joint limits during aggressive maneuvers, so downstream extrema checks must reject invalid solutions. An online trajectory generator instead solves for minimum duration as an optimization variable, which guarantees that kinematic derivative bounds hold by construction. The S-curve stopping trajectory examined in section 11.5.2 represents this exact profile structure evaluated toward a zero-velocity terminal state.

Kinematic feasibility by construction, however, does not grant permission to move. An online generator guarantees only the mathematical bounds explicitly configured in its parameters. Single-axis velocity, acceleration, and jerk bounds cannot account for multi-joint dynamic coupling in equation 11.8, spatial swept-volume collisions, or stale perception data. Furthermore, configuration errors or numerical solver faults can directly compromise the guarantees the generator claims to provide. Real-time benchmarks reported on high-performance desktop CPUs also do not represent execution latency on a constrained safety MCU. Online generators produce higher-quality proposals that satisfy kinematic constraints by design, but the deterministic safety controller must independently validate dynamic feasibility, collision margins, and temporal deadlines before granting execution permission. Even a candidate trajectory that passes every internal dynamic and seam continuity check remains unsafe without an admitted contingency. The system must therefore continuously maintain a certified stopping suffix to handle replacement chunks that fail to arrive before the admission deadline.

Checkpoint 11.2: Followable motion and the seam

A collision-free candidate becomes followable only after its joint rates, its seams, and its timing fit the body's limits.

Mapping and sampling

- ☐ **Singularity cost**: Explain why the 1 m/s TCP limit bounds no joint speed near a straightened elbow, and what damped least squares gives up to bound the inverse-kinematics step there.
- ☐ **Contracted limit**: Explain why two velocity samples inside their limit can bracket an interior over-speed, and when the smaller reserve of a jerk-bounded interpolant may be used.

Pricing the seam

- ☐ **Bridge estimate**: If the 50 ms bridge of notebook 11.2 must shrink to 10 ms to fit a late admission, estimate its peak inertial torque and peak torque rate, and decide whether it still fits the arm's 1,000 N·m/s torque-rate limit.
- ☐ **Executable timing**: Why does the time-optimal swing ask the drive for a step it refuses, and what does a jerk bound cost the shoulder swing in time?

11.5 Behavior at the Seam

When the replacement does not arrive, because inference stalls on scheduling interference, a bus fault, or an unhandled exception, the arm reaches the end of its authority with no successor in hand. The runtime must then bring it to rest without violating the limits derived above or exhausting its stopping clearance. Three things decide the outcome: which deadline has passed, which stop the MCU runs, and how much clearance that stop consumes.

11.5.1 Replacement deadlines and commitment cutoffs

A single timeout cannot protect both a fast control loop against a hung proposer and a moving arm against overrunning its obstacle clearance. A lease short enough to catch a dropped packet fires long before the geometric point of no return. A cutoff placed at the point of no return leaves a stalled policy coasting blind in the meantime. The two tiers named with the trajectory record in section 11.2, the periodic chunk lease and the commitment cutoff, therefore answer different questions: whether a replacement has renewed tracking authority recently enough, and whether the reserved stop can still begin in time. The planner also uses a soft *blend deadline* (t_{blend}), a scheduling target for a smooth replacement. Missing that target need not end tracking if the current lease and evidence remain valid and a checked replacement can still arrive before the admission cutoff that precedes physical commitment (t_{commit}). The earlier admission cutoff leaves the response interval needed for the matched stopping suffix to begin at commitment, before its reserved clearance is spent.

The two deadlines call for two different stops:

- **Matched stop suffix (planned termination at $t = t_{\text{commit}}$):** A pre-computed, jerk-limited C^2 slowdown along the admitted path, anchored specifically at the nominal state expected at t_{commit}, checked at admission to satisfy $d_{\text{stop}} \leq D_{\text{clear}}$. If no successor is admitted before the earlier admission cutoff, the controller dispatches the stop early enough that the body smoothly transfers to this pre-latched suffix at commitment, decelerating to rest without requiring runtime replanning.
- **State-matched fallback (reactive termination at $t < t_{\text{commit}}$):** If the high-rate chunk lease lapses, the parent intent expires, or a runtime safety monitor detects an unmodeled obstacle or torque fault *before* reaching t_{commit}, the robot cannot use the matched stop suffix (which is anchored at the future state t_{commit}). Instead, the MCU selects or constructs a validated state-matched fallback from current state estimates $(q, \dot{q}, \ddot{q})$ within reserved clearance. Encoder-derived velocity and acceleration have their own error bounds; bounded polynomial evaluation does not eliminate sensing, dispatch, or actuator response delay.

Both are the stopping suffix that every admitted trajectory carries (definition 11.1), differing only in the entry state from which they start. For the running mug reach, a stalled proposer expires the shared lease 60 ms after the last admission. The MCU then requests its state-matched fallback; physical onset follows the separately budgeted response delay. All timeout checks use the MCU monotonic clock. The longer-lease comparison isolates the geometric limit: even with tracking evidence continually renewed, the controller must dispatch the matched stop early enough to begin at commitment. A host progress report changes neither cutoff. Both stops also inherit the seam's continuity requirement, and that requirement fixes their shape and therefore their length.

11.5.2 The C^2 stopping suffix and jerk-limited braking

A moving robot carries continuous momentum. If an emergency brake uses an idealized constant-deceleration profile ($\ddot{q}_{\text{stop}}(t) = -a_{\text{brake}}$), the commanded acceleration steps discontinuously from zero to $-a_{\text{brake}}$ at t_{commit}, and steps discontinuously back to zero when velocity reaches zero. This instantaneous switch demands an infinite jerk impulse ($\dddot{q}_{\text{stop}} = \infty$) and an instantaneous step jump in actuator braking torque ($\Delta\tau = J_{\text{eff}} a_{\text{brake}}$). A real drive can only approximate that step. Whether the response loses traction, disturbs the payload, or exceeds structural limits depends on the load and controller, so the worked design chooses an explicitly smooth profile.

Two bounds describe what the load and the drive tolerate during that stop: a peak deceleration $a_{\max}$ and a peak jerk $j_{\max}$, the rate at which deceleration may change. A C^2 stop keeps position, velocity, and acceleration continuous at both ends, so it must ramp its deceleration up from zero and back down to zero, and the jerk bound limits how quickly it can do either. Within those bounds, the shape of the profile decides how far the machine travels. Consider a cruise entry with position $q_{\text{stop}}(t_{\text{commit}}) = q_c$, cruise speed $v_c > 0$, and zero entry acceleration. One smooth candidate is the lowest-order polynomial that matches position, velocity, and acceleration at both ends, written in the normalized time $u = (t - t_{\text{commit}})/T_{\text{stop}} \in [0, 1]$:

$$q_{\text{stop}}(u) = q_c + v_c T_{\text{stop}} \left(u - u^3 + \frac{1}{2} u^4 \right), \qquad \ddot{q}_{\text{stop}}(u) = -\frac{6 v_c}{T_{\text{stop}}} u(1-u), \qquad \dddot{q}_{\text{stop}}(u) = -\frac{6 v_c}{T_{\text{stop}}^2}(1-2u) \tag{11.16}$$

Its deceleration is an inverted parabola that peaks at the midpoint, $a_{\text{peak}} = 1.5\, v_c / T_{\text{stop}}$, and its jerk is largest at the two ends, $6 v_c / T_{\text{stop}}^2$. Holding the peak at $a_{\max}$ gives $T_{\text{stop}} = 1.5\, v_c / a_{\max}$ and a distance of $\frac{1}{2} v_c T_{\text{stop}} = 0.75\, v_c^2 / a_{\max}$, which is 1.5 times the constant-deceleration distance $v_c^2 / 2a_{\max}$. That factor is a property of this polynomial, not of smooth stopping. The polynomial touches its peak deceleration only at the midpoint and spends half the stop below three quarters of it, so it gives away distance that its own bounds would allow it to use.

The profile that uses those bounds fully is the **jerk-limited S-curve** (trapezoidal acceleration, the "double S" velocity profile) (Biagiotti and Melchiorri 2008). It ramps deceleration from zero to $a_{\max}$ at the jerk limit, holds $a_{\max}$, and ramps back to zero at the jerk limit just as velocity reaches zero. Each ramp lasts $a_{\max}/j_{\max}$. The velocity profile is point-symmetric about its midpoint, so the distance is half the entry speed times the duration:

$$d_{\text{stop}}(v) = \begin{cases} \dfrac{v^2}{2a_{\max}} + \dfrac{v\, a_{\max}}{2 j_{\max}}, & v \geq a_{\max}^2 / j_{\max} \\ v\sqrt{v / j_{\max}}, & v < a_{\max}^2 / j_{\max} \end{cases} \qquad T_{\text{stop}}(v) = \begin{cases} \dfrac{v}{a_{\max}} + \dfrac{a_{\max}}{j_{\max}}, & v \geq a_{\max}^2 / j_{\max} \\ 2\sqrt{v / j_{\max}}, & v < a_{\max}^2 / j_{\max} \end{cases} \tag{11.17}$$

Above the threshold speed $a_{\max}^2 / j_{\max}$, the stop costs constant-deceleration braking at $a_{\max}$ plus cruise over half of one jerk ramp, $a_{\max}/2j_{\max}$, so a finite jerk limit acts like extra reaction time. Below the threshold the stop is too short to reach $a_{\max}$, and its deceleration peaks at $\sqrt{v j_{\max}}$. Among stops that respect both bounds and never reverse, this profile reaches rest in the least time and the least distance. The shape of the stop, not physics, sets the distance beyond $v^2 / 2a_{\max}$, and for given deceleration and jerk bounds the S-curve spends the least. A slower profile remains admissible, but it must be charged at its own distance.

The jerk bound belongs to the load. The loaded tote rack and the mug tolerate a deceleration only if it is reached gradually, and the registry does not yet hold a measured jerk tolerance for either.

The worked design therefore sets each body's $j_{\max}$ to the peak jerk of equation 11.16 at that body's peak deceleration and top speed, a jerk the polynomial already asked the load to accept. A loaded stopping trial that measures the tolerable jerk would replace this stipulation, and equation 11.17 would then give the distance directly.

The stop is also a slowdown along the admitted path, not a new path. The planner fixes the geometry $\mathbf{q}(s)$ it checked for clearance, and the S-curve governs only the speed $\dot{s}(t)$ along that geometry. The stop therefore sweeps no volume outside the corridor already checked, and the one-dimensional comparison $d_{\text{stop}} \le D_{\text{clear}}$ applies along the path. A stop computed separately for each joint would leave that path and would need its own swept-volume check. A nonzero measured entry acceleration requires a profile matched to the actual $(q, \dot{q}, \ddot{q})$ at entry with zero terminal velocity and acceleration; the permission path checks its interior speed, acceleration, jerk, torque, and clearance.

In the alternate longer-lease arm case, the blend deadline is 240 ms, matched commitment 300 ms, nominal setpoint end 400 ms, and stop end 575 ms, so the stop lasts 275 ms. At the arm's 1 m/s TCP speed limit, the S-curve suffix with a 5 m/s² deceleration bound and a 66.7 m/s³ jerk bound travels 137.5 mm, compared with 150 mm for the polynomial of equation 11.16 with the same peaks. Because the blend deadline is the preferred admission instant on this schedule, the last scheduled validation and clearance evidence finish at 240 ms. The 60 mm of cruise between 240 ms and physical brake onset at 300 ms must therefore be charged against that clearance alongside the stop. Together they consume 197.5 mm in all, leaving 102.5 mm of the 300 mm obstacle clearance before sensing and model error are deducted. Because 300 ms is **physical suffix onset**, the MCU must close replacement admission and pretrigger any communication or actuator delay δ by that instant minus δ. Otherwise, the arm travels an extra $v_c\delta$, shifts the stop end later, and shrinks the 102.5 mm margin. Nominal setpoints end at 400 ms while MCU STOP continues to 575 ms, assuming fresh target evidence keeps TRACK valid until stop dispatch (section 11.3).

On the running machine, as the alternate schedule's introduction noted, a stall never reaches commitment because the chunk lease lapses first, so its stop is the state-matched fallback. From zero-acceleration cruise the S-curve matches the arm at whatever instant it starts, and it slows the tool along the path already checked, so the state-matched fallback travels the same distance as the matched stop suffix. On the arm, as on the base, the chunk lease is a term of the stopping budget, the rule established in section 4.4; the arm's own speed, stop, and clearance set what that term costs.

Napkin Math 11.4: The stall runway

Problem: The proposer stalls while the arm reaches at its TCP limit, and no replacement is admitted. How much of the obstacle clearance does the arm use between its last admission and rest, and how much remains?

Variables:

- Chunk lease $T_{\text{lease}} = 60$ ms (chosen); permission tick 1 ms (derived) and bus cycle 1 ms (illustrative); onset 20 ms (illustrative, the base's registered brake onset charged conservatively to the arm)
- Speed $v = 1$ m/s (chosen, TCP limit)
- Deceleration bound $a_{\max} = 5$ m/s² (illustrative) and jerk bound $j_{\max} = 66.7$ m/s³ (derived, the peak jerk of equation 11.16 at the same peak)
- Clearance $D_{\text{clear}} = 300$ mm (illustrative)

Math:

1. **Response path**: 60 ms + 1 ms + 1 ms + 20 ms = 82 ms from last admission to stop onset, which is 82 mm at v.
2. **Branch test** (equation 11.17): $a_{\max}^2 / j_{\max} = 0.375$ m/s, below v, so the stop reaches $a_{\max}$.
3. **Stop**: $v^2/2a_{\max} + v\,a_{\max}/2j_{\max} = 100$ mm + 37.5 mm = 137.5 mm, in $v/a_{\max} + a_{\max}/j_{\max} =$ 200 ms + 75 ms = 275 ms.
4. **Clearance**: 82 mm + 137.5 mm = 219.5 mm of 300 mm.

DRAFT

Result: The stall leaves 80.5 mm of geometric margin, before sensing and model reserves.

Systems insight: The travel before the stop begins is 59.6 percent of the stop itself, and at this speed every millisecond of lease costs a millimeter. The remaining margin equals only 80.5 ms of further lease, so the lease length is a term of $d_{\text{stop}} \le D_{\text{clear}}$ exactly as the deceleration bound is.

While hardware qualification must establish the arm's actual response in place of the charged onset allowance, the two schedules leave different geometric margins because they date their clearance evidence differently (figure 11.2). The running lease charges travel from the last admission (82 mm) plus the 137.5 mm stop, which keeps 80.5 mm. The alternate schedule charges only the cruise from its blend deadline, when clearance was last checked, to commitment (60 mm) plus the same stop, which keeps 102.5 mm. The alternate schedule's 278 ms admission cutoff is therefore an offset on its own time axis rather than a lease length that the running clearance could support. Any lease, whatever its length, must fit its own evidence age, response travel, stop, and uncertainty reserve, just as the running lease does in the notebook above.

Although the free-space deceleration bound suits the empty-gripper reach toward the mug, it is too aggressive once the arm carries an open mug of liquid away from the conveyor, because container acceleration tilts the free surface and excites sloshing (Mayer and Krechetnikov 2012). Under a steady deceleration a, the free surface settles perpendicular to the effective gravity, tilting by $\tan\theta_{\text{tilt}} = a/g$ and raising the liquid at the wall by $R\,a/g$ for inner radius R. In an illustrative mug of 40 mm inner radius filled to 10 mm below the rim, the free-space stop's 5 m/s² deceleration raises the liquid 20.4 mm at the wall, well past the rim. Keeping even the steady tilt inside the freeboard h_f (a necessary condition, since dynamic sloshing overshoots the steady tilt) limits deceleration to $g\,h_f/R$, or 2.45 m/s². At that gentler bound, with the jerk the polynomial reaches at the same peak, the stop from the 1 m/s TCP limit lengthens to 280.4 mm and, together with the running lease path, demands 362.4 mm of the 300 mm clearance. The loaded carry must therefore run slower than the empty reach. Before sensing and model reserves are deducted, the highest speed whose lease path and stop fit that clearance is 0.89 m/s. The trajectory record's assumption sidecar must declare which payload regime and measured slosh limit (or lid) the record assumes.

At the TCP limit, the open mug's gentler stop overruns the clearance the empty reach fits.

The commitment deadline forbids waiting. In the alternate arm schedule, commitment preserves the 102.5 mm margin computed from clearance dated at 240 ms. Missing the 278 ms admission cutoff and delaying stop dispatch by 200 ms would delay physical stop onset that far past commitment. Cruise at the TCP limit would add 200 mm before the same stop and breach the 300 mm clearance by 97.5 mm. A later stop must still match its actual entry state; this counterfactual holds cruise speed constant only to show the lost clearance. An arm that waits that long may already occupy a state from which no admissible control avoids the obstacle.

11.5.3 Inevitable collision states

An arm or base that is still clear of every obstacle at the current millisecond can already be doomed to hit one if its speed and remaining distance leave no control input inside its drive and traction limits that can bring it to rest in time. Classical path planning misses this trap when it treats obstacle avoidance as a static geometric query that asks only whether the robot's configuration overlaps an obstacle volume at the current instant. Because a mobile manipulator moving with momentum and finite braking torque cannot halt instantaneously, its dynamic state (position, velocity, and acceleration, under its traction limits) can cross a boundary beyond which every admissible control trajectory leads to collision long before geometric contact occurs. Such states are inevitable collision states (ICS)[10] (Fraichard and Asama 2004).

10 **Inevitable collision states and viability**: Within the free states, the complement of the ICS set is what Aubin (1991) calls the viability kernel, the states from which some admissible control keeps the system inside its constraints forever. Planning must therefore keep the commitment state inside that kernel, not merely outside the obstacle, and against static obstacles a verified stop to rest is the witness that it is there.

Definition 11.4: Inevitable collision state

Inevitable collision state (ICS) is any dynamic plant state $\mathbf{x}(t) \in \mathcal{X}$ from which every physically admissible future control trajectory $\mathbf{u}_{[t,\infty)} \in \mathcal{U}_{\text{adm}}$ inevitably produces collision with an obstacle or constraint boundary:

$$\mathbf{x} \in \text{ICS}(\mathcal{X}_{\text{obs}}) \iff \forall \mathbf{u}(\cdot) \in \mathcal{U}_{\text{adm}},\ \exists t' \geq t \text{ such that } \mathbf{x}(t') \in \mathcal{X}_{\text{obs}}$$

where $\mathcal{U}_{\text{adm}}$ denotes the admissible control input set bounded by actuator torque, traction, and deceleration limits.

1. **Significance**: Sets the commitment deadline t_{commit} in receding-horizon planning; continuing nominal motion past it without an admitted successor can carry the plant into an ICS, and no later replanning can bring it back out.
2. **Distinction**: Unlike static geometric collision checks that evaluate only instantaneous spatial overlap ($\mathbf{x} \in \mathcal{X}_{\text{obs}}$), an ICS identifies dynamically trapped states where current momentum and finite braking torque make future boundary breach physically unavoidable.
3. **Common pitfall**: Delaying emergency braking in the expectation that an overdue neural planner will complete an evasive path, ignoring that moving mass enters an ICS long before geometric contact occurs.

The commitment deadline t_{commit} and the admission-checked stopping suffix are co-designed precisely so that the machine initiates braking *before* its dynamic state enters an inevitable collision state, ICS($\mathcal{X}_{\text{obs}}$). If an action chunk's stopping suffix requires distance $d_{\text{stop}} > D_{\text{clear}}$, that suffix cannot justify admitting the chunk. Failure of this particular stop does not establish that every admissible maneuver collides, as the definition of an inevitable collision state requires. The architecture nevertheless refuses the proposal because it lacks the independently executable stop on which its permission depends. The safety microcontroller therefore enforces the hard admission invariant:

$$d_{\text{stop}}(t_{\text{commit}}) \leq D_{\text{clear}}(t_{\text{commit}}) \tag{11.18}$$

If the controller delays braking past commitment in the hope that an overdue neural replanner will deliver an evasive path, physical momentum can carry the machine into an ICS from which no admissible control input can prevent impact. The commitment deadline is the deadline form of this boundary, fixed at planning time. Its set form, the recoverable set of plant states from which the resident stop still succeeds, is the stopping envelope that section 12.3 develops for the runtime check.

These deadlines and the stop they protect are fixed at admission, before motion begins. The candidate declares payload, contact, effort, and obstacle assumptions. The MCU first validates its fixed header and lineage. It then checks the complete bounded trajectory and suffix, including the seam $(q, \dot{q}, \ddot{q})$ against the reachable state envelope from its current estimate, interior extrema, clearance, stopping distance, and a feasible response from every state reachable through physical stop onset. A scan over N setpoints is $\mathcal{O}(N)$, even with a fixed maximum N and measured worst-case execution time; after admission, a distinct $\mathcal{O}(1)$ per-tick check reads the current state and local reference. A replacement received after the blend deadline passes the same check with a compressed bridge, whose higher acceleration and torque its extrema must still clear. If the check cannot finish before the earlier admission cutoff, the MCU retains the current record only while its own lease, evidence, and fallback remain valid. A candidate received after commitment cannot restore the spent runway.

The planner proposes the suffix. The MCU latches the validated payload and stop profile into protected memory **before** enabling TRACK, and it dispatches that resident path before commitment without reading host DRAM. A host crash, bus stall, or corrupted shared packet therefore cannot erase the stop. If the measured state falls outside the latched suffix's entry region, the MCU uses the state-matched fallback. Torque disable or a spring brake is not a universal substitute for a controlled stop, especially with a suspended load (section 12.9).

11.5.4 Base stopping budget under jerk-limited deceleration

On the mobile base approaching the rack end at its 1.5 m/s drive limit, the loaded tote rack tolerates the machine's credible 2 m/s² deceleration only when that deceleration is ramped gradually rather than applied as an abrupt step. The base's stop is therefore the S-curve of equation 11.17 with $a_{\max}$ at 2 m/s² and a jerk bound of 7.11 m/s³. That jerk bound is the peak jerk of the polynomial stop from the 1.5 m/s drive limit and the stop jerk bound already listed on the machine card (table 1.2). Under it, every speed above 0.5625 m/s reaches the deceleration plateau. At the drive limit, this suffix lasts 1,031.25 ms and travels 773.44 mm, compared with 562.5 mm for constant-deceleration braking in equation 2.2 and 843.75 mm for the polynomial with the same peaks. Because the jerk ramps cost the equivalent of 140.6 ms of cruise, the suffix adds 210.94 mm to the budget that section 8.4.1 left at 912.9 mm. This raises the running total to 1,123.84 mm and exceeds the 1.10 m rack-end clear distance by 23.84 mm. Because this partial total still omits the unqualified dispatch-to-admission interval, which can only add travel, the 23.84 mm overrun is a conditional design deficit rather than a complete measured response bound.

Plotted as velocity against time, the area under each curve is its stopping distance, and the jerk ramps appear as the rounded start and finish that stretch the S-curve and the polynomial past the constant-deceleration line (figure 11.6). Stacked onto the budget's other terms, only the constant-deceleration stop, which no drive can execute, ends inside the 1.10 m rack-end clearance.

Even the shortest stop under the stipulated bounds leaves a 23.84 mm deficit at the 1.5 m/s drive limit, so the robot cannot cruise at that limit in this aisle under the stated deceleration and jerk bounds. The deficit is narrow and it rests on a stipulated jerk bound, so a measured jerk tolerance could move it in either direction. The machine must either derate its operating speed or detect the boundary with more usable clearance ahead; a longer prediction or evidence lifetime alone creates no stopping room. This is the first law's stopping condition (principle 1) with one more term, because a stop the load can tolerate is a longer stop and the distance it adds comes out of the same clear distance. This calculation concerns the fixed rack-end boundary. A person entering the aisle requires separately qualified moving-actor bounds and crossing control or an operating restriction; this static clearance calculation does not establish that protection. Under the selected body and aisle constraints, reducing speed can recover clearance. Section 12.3 adds the tracking allowance and derives a candidate speed; permission to operate still requires complete capture-to-admission timing, positive stopping reserve, and qualification under the declared conditions.

↩ The intrusion distance an approaching pedestrian adds to the stopping budget, and the site controls it demands, are established in section 9.7.

For either body, the admission and fallback decisions in table 11.2 are distinct from an emergency power cut. The mug reach and the aisle approach now have a complete execution argument: a timed prefix, a checked seam, and a stop reserved under stated conditions. With those obligations fixed, trajectory generators can be compared by the candidate work they perform rather than by treating their outputs as permission.

Table 11.2: Tracking and fallback decisions: A host crash does not remove a latched stop. TRACK is the nominal mode; STOP is the stop rung of the fallback ladder in section 12.9.1.

State	Trigger	MCU action and required evidence
TRACK, ordinary handoff	Replacement passes the complete admission check before the admission cutoff preceding commitment.	Latch the new nominal path and its own stop, then transfer at a checked C^2 seam.
TRACK, late candidate	Candidate arrives after the blend deadline but before the admission cutoff preceding commitment.	Admit only if the compressed bridge, full path, stop, and timing still fit; otherwise retain the current record only while its own permission remains valid.
STOP, matched suffix	No successor admitted by the earlier cutoff.	MCU follows its protected suffix from the validated entry state; nominal tracking may end while STOP continues.
STOP, earlier invalidation	Lease lapse, intent expiry, contact, state mismatch, or fault before physical stop onset.	MCU selects the state-matched fallback with reserved clearance.
Fault response	Controlled stop cannot be assured from measured state.	Use the machine-specific risk response (which may include drive inhibit, brake, or compliance) only under its validated load and timing assumptions.

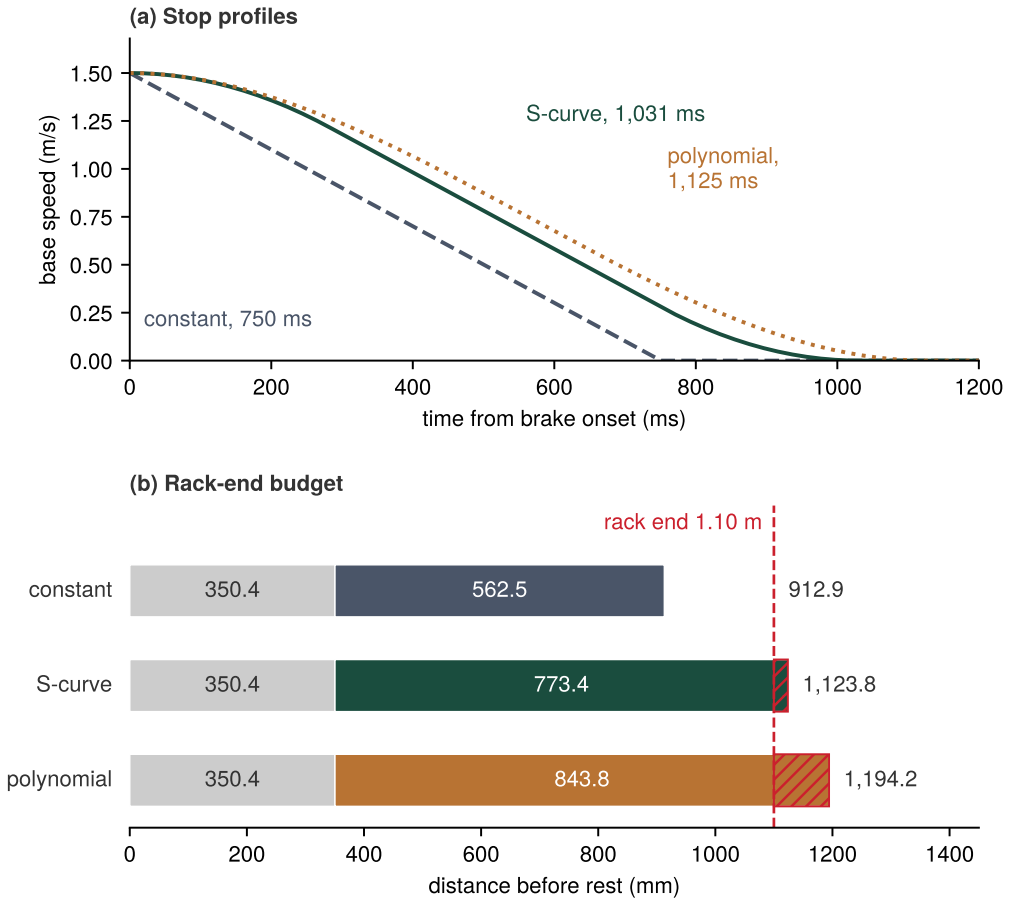


Figure 11.6: Three stops from the drive limit: Velocity against time for the base's stop from 1.5 m/s under the same 2 m/s² deceleration bound (a), and each stop added to the budget's other terms against the 1.10 m rack-end clearance (b). The area under each velocity curve is its stopping distance. Constant deceleration, which no drive can execute, fits with 562.5 mm of braking; the jerk-limited S-curve, the shortest stop under the stipulated bounds, needs 773.44 mm and passes the clearance by 23.84 mm; the polynomial with the same peaks needs 843.75 mm.

Checkpoint 11.3: Commitment and full stopping clearance

Before committing an autonomous machine to an action chunk, verify your understanding of commitment deadlines, stopping budgets, and trajectory continuity:

- ☐ **Deadline distinction**: Distinguish nominal setpoint end, physical stop completion, and the end of the evidence horizon.
- ☐ **Continuous entry**: Why would a constant-deceleration switch fail C^2 matching at zero-acceleration cruise entry?
- ☐ **Clearance and latency**: The arm's 102.5 mm margin already charges cruise from the blend deadline to commitment. Where must an unreserved delay δ beyond it be charged, and how much of that margin does $v_c\delta$ consume?

11.6 Choosing a Trajectory Generator

When a mobile manipulator reaches around a conveyor guard, a planner that searches configuration space can run out of time before finding a route. A local trajectory optimizer can get trapped in a

non-convex pocket behind the guard. A learned action-chunk policy can propose a smooth reach that clips the guard or exceeds a joint's torque limit. The three algorithmic families that propose motion, geometric search, continuous optimization, and learned action chunking, each fail in a different way if treated as self-certifying. In all three paradigms, deciding where the body can travel geometrically remains separate from determining how quickly actuators can safely traverse that path. Regardless of which family generates a proposal, the resulting trajectory remains an unverified candidate that requires exhaustive dynamic validation before the execution engine commits it to the hardware.

The conveyor reach also fixes what each family can afford. A replacement must be admitted every 50 ms, and 18 ms of every replacement's dispatch-to-admission allocation is already reserved for whole-plan validation, while a chunk policy spends another 40 ms of it on inference. To sustain the intended renewal cadence, the generation pipeline must provide both the allocated latency and the throughput that overlapping requests require. A solver without a bounded completion time needs a timeout that leaves the resident fallback executable; it can also run outside the renewal loop, planning the route around the guard and handing the loop a corridor to follow. That deadline, more than the quality of the motion, sorts the three families below.

Sampling-based geometric planners explore configuration space without requiring an explicit algebraic representation of obstacles. Probabilistic Roadmaps (PRM) (Kavraki et al. 1996) construct multi-query roadmaps by sampling collision-free configurations and connecting nearby neighbors with local collision-free segments, amortizing graph construction costs across multiple tasks. Conversely, Rapidly-exploring Random Trees (RRT) (LaValle 1998) target single-query problems by growing a space-filling tree from the initial configuration toward unexplored territory. Its asymptotically optimal counterpart, RRT* (Karaman and Frazzoli 2011), continuously rewires parent connections within local neighborhoods as samples accumulate, ensuring that the discovered path cost converges almost surely to the theoretical optimum (Karaman and Frazzoli 2011). These methods resolve geometric routes through complex obstacle geometry, yet their completeness guarantees remain strictly probabilistic: the probability of discovering a feasible path approaches one only as the number of samples approaches infinity. When a sampling planner exhausts its computation budget without finding a path, its timeout provides no proof that the route is obstructed. The admission logic must therefore treat planner silence as an unresolved proposal rather than a verified safety determination. Furthermore, evaluating configuration samples and connecting edges requires repeated collision queries against environment geometry, usually accelerated by enclosing the manipulator links in bounding spheres or capsules, and these queries dominate the planner's run time. Even when successful, search yields only an un-timed geometric path with discontinuous derivative transitions. The system must still apply time parameterization, smooth corner blends, joint torque verification, and a valid stopping continuation before dispatching the trajectory to physical actuators.

The complementary classical formulation is **constrained continuous trajectory optimization**, exemplified by Covariant Hamiltonian Optimization for Motion Planning (CHOMP) (Ratliff et al. 2009), TrajOpt (Schulman et al. 2014), and receding-horizon Model Predictive Control (MPC) (Tedrake 2023; Lynch and Park 2017). Rather than evaluating discrete graph nodes, continuous trajectory optimization represents motion as a continuous, time-indexed curve $\mathbf{q}(t)$ and minimizes an objective functional that balances dynamic smoothness against obstacle proximity:

$$\min_{\mathbf{q}(t)} \quad \mathcal{U}[\mathbf{q}] = \int_0^T \left(\frac{1}{2} \|\ddot{\mathbf{q}}(t)\|^2 + \lambda_{\text{obs}} c_{\text{obs}}(\mathbf{q}(t)) \right) dt$$

subject to physical boundary conditions and actuator invariants:

$$\mathbf{q}(0) = \mathbf{q}_0, \quad \dot{\mathbf{q}}(0) = \dot{\mathbf{q}}_0, \quad \ddot{\mathbf{q}}(0) = \ddot{\mathbf{q}}_0$$

$$\mathbf{q}(T) = \mathbf{q}_{\text{goal}}, \quad \dot{\mathbf{q}}(T) = \mathbf{0}, \quad \ddot{\mathbf{q}}(T) = \mathbf{0}$$

$$|\dot{\mathbf{q}}(t)| \leq \dot{\mathbf{q}}_{\max}, \quad |\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})| \leq \tau_{\max}$$

While CHOMP performs functional gradient descent and TrajOpt solves sequential convex subproblems using signed-distance collision constraints, both rely on local optimization. Consequently, non-convex obstacle geometry can trap the solver in infeasible local minima or geometric dead ends. Graphs of Convex Sets (GCS) resolve this susceptibility when the collision-free configuration space

can be partitioned beforehand into convex regions. Finding the global sequence of regions and optimizing the polynomial spline within them then formulates as a mixed-integer convex program whose convex relaxation can be solved efficiently (Marcucci et al. 2023). Because generating these convex decompositions requires substantial offline computation, GCS suits static structures such as the conveyor guard far better than dynamic, rapidly shifting environments. Although geometric search or learned policies can propose alternative routes around unforeseen obstructions, any candidate motion must still undergo explicit verification against dynamic torque limits and actuator constraints.

Repeatedly re-optimizing from an updated state estimate forms the operational foundation of classical **Model Predictive Control (MPC)**[11]. In this receding-horizon paradigm, an optimizer computes an optimal trajectory over a finite horizon T, dispatches an initial prefix to the actuator controllers, and re-solves the problem from the newly observed state on the subsequent cycle. The objective functional ranks candidate motions by performance, while hard constraints eliminate trajectories that exceed physical limits. Classical discrete-time MPC frames this trade-off as a constrained numerical optimization problem over steps $k = 0, \dots, N_{\text{mpc}} - 1$ (Tedrake 2023; Lynch and Park 2017):

$$\begin{aligned} \min_{\mathbf{u}_{0:N_{\text{mpc}}-1}} \quad & \sum_{k=0}^{N_{\text{mpc}}-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_f(\mathbf{x}_N) \\ \text{subject to} \quad & \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N_{\text{mpc}} - 1 \\ & \mathbf{x}_k \in \mathcal{X}_{\text{safe}}, \quad \mathbf{u}_k \in \mathcal{U}_{\text{adm}}, \quad k = 0, \dots, N_{\text{mpc}} - 1 \\ & \mathbf{x}_N \in \mathcal{X}_f \end{aligned} \tag{11.19}$$

The variables in equation 11.19 decouple the predicted state sequence $\mathbf{x}_k$ from the selected control inputs $\mathbf{u}_k$. The dynamic transition map $f(\mathbf{x}_k, \mathbf{u}_k)$ propagates the system state forward, requiring every intermediate step to remain admissible. The stage cost $\ell(\mathbf{x}_k, \mathbf{u}_k)$ penalizes tracking error and control effort, while the safe state set $\mathcal{X}_{\text{safe}}$ enforces collision margins derived from the spatial perception pipeline. The admissible control set $\mathcal{U}_{\text{adm}}$ bounds actuator torques and voltages according to verified hardware limits.

The terminal constraint forces the horizon endpoint into a **terminal invariant set**, $\mathbf{x}_N \in \mathcal{X}_f$. In classical control theory, *recursive feasibility* guarantees that if a feasible solution exists at control cycle i, a feasible solution remains provably discoverable at cycle $i+1$. Establishing this property requires an accurate dynamic model and a terminal set $\mathcal{X}_f$ within which an invariant fallback control law maintains safety indefinitely. In an embodied robotic system, this terminal invariant is implemented through a validated braking continuation that brings the machine to rest within verified spatial clearance ($d_{\text{stop}} \leq D_{\text{clear}}$). Bringing the machine to a standstill does not guarantee indefinite safety on its own, because dynamic obstacles can move into the workspace of a stationary robot unless external interlocks preserve the boundary.

Continuous-time optimal control problems are typically transcribed into finite-dimensional nonlinear programs through **direct shooting** or **direct collocation**.[12] Direct shooting forward-integrates the control trajectory $\mathbf{u}(t)$, yielding a compact optimization problem that can nonetheless become ill-conditioned over long horizons. Direct collocation parameterizes both states and controls using piecewise polynomial splines, enforcing dynamic defect constraints $\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) - \dot{\mathbf{x}}_k = \mathbf{0}$ at interior knots to generate large, sparse nonlinear programs that converge rapidly near smooth trajectories (Hargraves and Paris 1987). However, theoretical MPC guarantees hold only when the dynamic model is accurate, constraints are satisfied, and the numerical solver finishes before its real-time deadline. Contact transitions, stick-slip friction, and unexpected loads degrade solver convergence, often causing iterations to time out. Learned action chunking eliminates online numerical solves during inference, but neural policies do not enforce the terminal invariant constraint $\mathbf{x}_N \in \mathcal{X}_f$. An unvalidated action chunk can easily end at high velocity where the robot lacks adequate braking room.

Decoupling generation into a learned proposer and a constrained optimizer reconciles statistical representation with physical safety. The learned policy acts as a proposer, generating behavioral candidates conditioned on raw visual tokens and task semantics. A constrained trajectory generator

[11] **Model predictive control**: MPC began in the process industries of the 1970s, in refineries and chemical plants whose slow dynamics left seconds to minutes for each solve; Qin and Badgwell (2003) trace its industrial lineage from the first IDCOM and dynamic matrix control installations. Moving the same receding-horizon loop onto an arm tracked at 1 kHz shrinks that time to milliseconds, which is why a solver's deadline, and not its optimum, decides what reaches admission.

[12] **Direct collocation defects in real-time solvers**: Direct collocation, introduced for trajectory optimization by Hargraves and Paris (1987), turns a continuous-time control problem into a finite nonlinear program by requiring the defect $\Delta_k = \dot{\mathbf{x}}(t_k) - \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{0}$ at each collocation point. Contact transitions and unmodeled friction make the problem non-smooth, and the solver then loses its fast local convergence and needs more iterations than a fixed control deadline may allow.

then adjusts the proposed trajectory to satisfy boundary conditions, actuator limits, and spatial clearances. Because this numerical projection can become infeasible or miss its real-time deadline, it operates entirely on the untrusted proposal side of the architecture, upstream of independent safety admission.

Mathematically, this dual-stage architecture formulates trajectory generation as a constrained projection operator $\Pi_{\mathcal{K}}(\hat{\mathbf{a}}_{\text{learned}})$:

$$\mathbf{q}^*(t) = \Pi_{\mathcal{K}}(\hat{\mathbf{a}}_{\text{learned}}) = \arg\min_{\mathbf{q}(t)} \int_0^T \|\mathbf{q}(t) - \hat{\mathbf{q}}_{\text{learned}}(t)\|^2_{\mathbf{W}_q} dt + \int_0^T \|\ddot{\mathbf{q}}(t)\|^2_{\mathbf{W}_j} dt \tag{11.20}$$

The projection begins at the planned activation seam, so its initial-state constraints refer to that seam rather than to the state measured when the planning request begins. Upstream synthesis uses the projected reference and its reachable-state envelope; at activation, the MCU instantiates and checks the bounded bridge against measured state. The candidate remains subject to physical boundary conditions and safety invariants across the execution interval $t \in [0, T]$:

$$\mathbf{q}(0) = \mathbf{q}_{\text{actual}}, \quad \dot{\mathbf{q}}(0) = \dot{\mathbf{q}}_{\text{actual}}, \quad \ddot{\mathbf{q}}(0) = \ddot{\mathbf{q}}_{\text{actual}}$$

$$\mathbf{q}(t) \in \mathcal{C}_{\text{free}}, \quad |\dot{\mathbf{q}}(t)| \leq \dot{\mathbf{q}}_{\max}, \quad |\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})| \leq \tau_{\max}$$

$$d_{\text{stop}}(\mathbf{q}(t_{\text{commit}}), \dot{\mathbf{q}}(t_{\text{commit}})) \leq D_{\text{clear}}(t_{\text{commit}})$$

In equation 11.20, the primary term penalizes tracking deviation from the learned nominal trajectory $\hat{\mathbf{q}}_{\text{learned}}(t)$ using positive semidefinite state weights $\mathbf{W}_q$, while the regularization term penalizes joint jerk $\|\ddot{\mathbf{q}}(t)\|^2_{\mathbf{W}_j}$ to minimize mechanical vibration and wear. The weighted quadratic norm is defined as $\|\mathbf{z}\|^2_{\mathbf{W}} = \mathbf{z}^\top \mathbf{W}\mathbf{z}$, and $\mathcal{K}$ denotes the feasible set of trajectories satisfying the displayed physical constraints. These constraints enforce the dynamic invariants that unconstrained learned models cannot guarantee: C^2 continuity with the currently executing state, collision avoidance within the free configuration space $\mathcal{C}_{\text{free}}$, kinematic joint speed bounds, full rigid-body inverse dynamics torque limits, and certified stopping viability at the commitment horizon t_{commit}. Even when the optimizer finds a feasible trajectory, that solution remains contingent on model assumptions regarding payload mass and obstacle positions; the independent admission gate must verify those assumptions before granting hardware permission.

Altering the proposer modifies the computational budget and latency profile, but leaves the physical safety obligations unchanged. Action Chunking with Transformers (ACT) (Zhao et al. 2023) infers an entire chunk of future joint setpoints in a single forward pass and smooths replanning boundaries by ensembling overlapping temporal chunks. However, simple coordinate averaging across predictions fails to enforce higher-order boundary derivatives, which can induce severe acceleration transients at chunk transitions. Diffusion policies (Chi et al. 2025) capture complex, multimodal behavioral distributions through iterative score-based denoising, but the multiple neural network evaluations required during inference add substantial latency to the replacement planning path. Alternatively, a neural model can warm-start a numerical trajectory optimizer by providing initial collocation knots. While a good warm-start drastically reduces sequential quadratic programming iterations, an out-of-distribution observation can seed the optimizer in an infeasible basin of attraction, causing solver divergence and missed execution deadlines. Generator selection therefore balances configuration-space exploration, constrained local tracking, and multimodal perceptual synthesis against the strict temporal deadlines required to validate and admit candidate trajectories. Table 11.3 compares the inputs, constraints, and seam obligations of these synthesis methods. Its sampling-based MPC row, model predictive path integral control (MPPI), rolls many sampled control sequences through the dynamics model and returns their cost-weighted average. MPPI reappears as a base controller in section 11.6.1.

Table 11.3: Comparison of trajectory generation paradigms: Search, optimization, and learned proposals perform different candidate-generation work. Solve time and supported constraints depend on the implementation and problem; every selected pipeline still needs checked dynamics, seams, and stopping clearance.

Trajectory paradigm	Input representation	Dynamic and safety guarantees	Dominant workload	Boundary continuity across replanning
Sampling-Based Graph Search (PRM, RRT*)	Configuration samples $\mathbf{q} \in \mathcal{C}_{\text{free}}$	Route quality depends on the search; dynamics need separate checks	Collision queries	Piecewise-linear (C^0); discontinuous $\ddot{\mathbf{q}}$
Classical Numerical MPC	Low-dimensional state vector $\mathbf{x} \in \mathbb{R}^n$	Constraint and stability guarantees depend on model and assumptions	Constrained solve	Continuity requires explicit boundary constraints
Sampling-Based MPC (MPPI)	State vector and a Euclidean signed distance field (ESDF)	Constraint treatment depends on the formulation and solver	Rollouts or optimization	Requires spline smoothing or terminal costs
Generative Action Chunking (ACT, Diffusion)	High-dimensional multimodal tokens (vision, text)	Empirical demonstration distribution; no formal safety guarantee	Model inference	Discrete samples; requires checked interpolation and seam

The mechanisms in this chapter are durable; the systems that implement them change quickly. Table 11.4 records the named systems a practitioner meets today, each tied to the mechanism it implements. Its final row shifts from the seven-joint arm to the wheeled mobile base, whose rolling wheels constrain the velocity space directly.

Table 11.4: Trajectory generation in practice, as of 2026: Each system implements a mechanism this chapter teaches and remains a proposer in this architecture. Figures come from each system's own paper, measured on its authors' hardware, and are not admission evidence for this machine.

Job in this chapter	System	Mechanism	Reported figure
Time parameterization	TOPP-RA (Pham and Pham 2018)	Reachability analysis over a path grid (section 11.4.5)	$\mathcal{O}(N_{\text{grid}})$ small linear programs for N_{grid} grid points
Online jerk-limited generation	Ruckig (Berscheid and Kröger 2021)	Time-optimal constant-jerk phases from an arbitrary state	19.8 µs mean, 123 µs worst case for 7 axes on a desktop processor
Parallel trajectory optimization	cuRobo (Sundaralingam et al. 2023)	Parallel optimizer seeds, sphere collision model, time step rescaled to the limits	Kinematics and position derivatives only; full dynamics left open
Learned chunk proposals	π_0 (Black et al. 2024)	Flow-matching action chunks executed open loop	50-action chunks; 50 Hz robots re-infer after 25 actions (0.5 s)
Chunk-boundary conditioning	Real-time chunking (Black, Galliker, et al. 2025)	Freeze the actions that run during inference, generate the rest to agree with them	No continuity guarantee stated
Mobile-base local control	Nav2 DWB and MPPI controllers (Macenski et al. 2023)	Velocity-space arcs scored by plugin critics, or sampled control sequences by cost	Both ship in the ROS 2 navigation stack

DRAFT

11.6.1 Mobile-base planning under nonholonomic constraints

Unlike the seven-joint arm, the mobile base cannot follow a straight line interpolated between two poses in the warehouse aisle, because its two coaxial drive wheels roll without slipping and therefore cannot slide sideways along their axle. Expressed in terms of heading angle θ_{head}, forward linear speed v, and yaw rate ω_z, the differential-drive (unicycle) kinematics satisfy

$$\dot{x} = v\cos\theta_{\text{head}}, \qquad \dot{y} = v\sin\theta_{\text{head}}, \qquad \dot{\theta}_{\text{head}} = \omega_z, \qquad \dot{x}\sin\theta_{\text{head}} - \dot{y}\cos\theta_{\text{head}} = 0 \tag{11.21}$$

The non-integrable relation in equation 11.21 forms a **nonholonomic constraint**,[13] restricting the instantaneous velocity space without diminishing the dimension of the reachable configuration space $SE(2)$ (Siciliano et al. 2009). The base can reach any planar pose in the aisle, but not with

[13] **Holonomic** (Greek *holos*, whole, and *nomos*, law): Hertz (1894) introduced the term for systems whose constraints obey integral laws, relations among positions alone, in contrast to material systems in general, which obey only differential laws. A wheel's no-slip rule is a differential law that cannot be integrated into such a relation. The distinction decides the planner's search space, since a holonomic arm can be planned as a path through configuration space while the base must be planned through its controls.

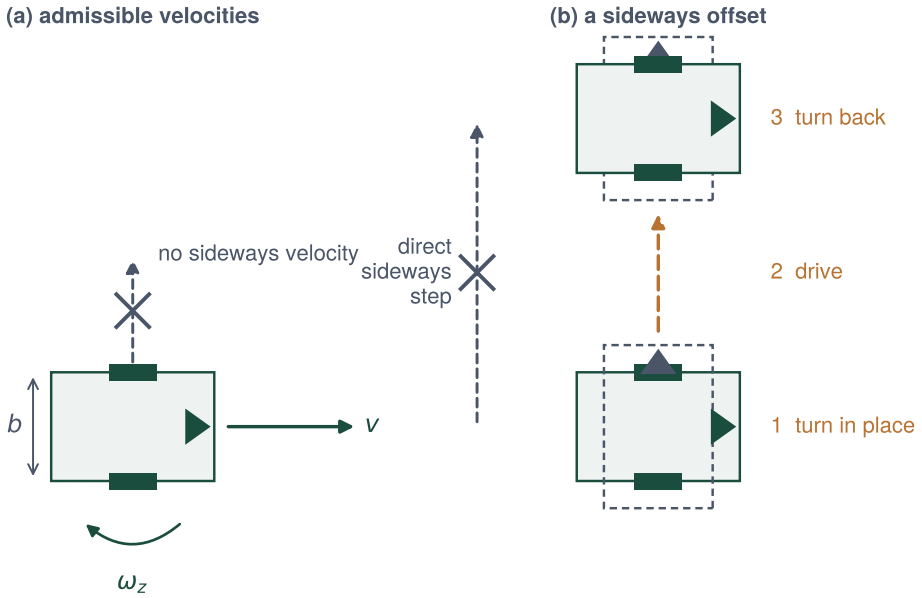


Figure 11.7: What the wheels allow: (a) A differential-drive base with track width b can move forward at v and turn at ω_z about the center of its axle, but its wheels cannot roll sideways. (b) A pure sideways offset is still reachable, as a maneuver rather than a step: turn in place, drive, and turn back. Proportions are schematic.

any velocity. It can drive forward at v and turn at ω_z about the center of its axle, while the no-slip condition forbids translation along the axle (figure 11.7). A pure sideways offset is therefore reached as a maneuver rather than a step: turn in place, drive along the new heading, and turn back. A base planner must therefore search directly over kinematically admissible control trajectories rather than straight geometric paths in configuration space. Furthermore, each wheel's tangential speed equals $v \pm \omega_z b/2$ for track width b, so hardware motor limits constrain linear velocity and turning rate jointly.

Local base planners enforce nonholonomic constraints by planning directly in velocity space over a receding time horizon. The dynamic window approach (DWA) (Fox et al. 1997) maps each control pair (v, ω_z) to a circular trajectory arc and evaluates candidate pairs through two nested dynamic criteria. *Reachable velocities* comprise pairs achievable within actuator acceleration limits over the replanning window. With the mobile manipulator's base acceleration limit of 2 m/s² and a 50 ms chunk renewal interval, this dynamic window spans 0.1 m/s on either side of the current forward velocity. *Admissible velocities* select the subset from which the base can decelerate to a complete stop before reaching the nearest obstacle along that arc, satisfying $v \leq \sqrt{2\, d_{\text{arc}}(v, \omega_z)\, a_{\text{brake}}}$. From these admissible candidates, DWA selects the control pair maximizing an objective that balances target heading alignment, obstacle clearance, and forward speed. MPPI (Williams et al. 2017), the sampling-based MPC of table 11.3, replaces this deterministic velocity grid by rolling forward thousands of stochastic control sequences through the dynamic model on a GPU, synthesizing an optimal control action as a cost-weighted trajectory distribution. Production navigation frameworks such as ROS 2 Nav2 incorporate both controller families (Macenski et al. 2023).

The admissibility test applies the core stopping requirement across candidate arcs, yet its classical textbook formulation assumes instantaneous, constant-deceleration braking. On a straight trajectory with 1.10 m of rack-end clearance, this simplified test admits forward speeds up to 2.10 m/s, well above the physical drive limit of 1.5 m/s. However, when evaluated against the conditional stopping subtotal derived in section 11.5.4 (which accounts for sensor latency, communication delays, localization uncertainty margins, and jerk-limited deceleration), that same clearance fails to guarantee safety even at the nominal drive limit. A mobile-base trajectory generator, whether

a classical DWA planner or a learned chunk policy, is therefore a proposer of candidate velocities. The real-time permission path must also charge dispatch-to-admission travel before validating each proposed trajectory against the full stopping budget.

The choice of trajectory generator governs how candidate motions are discovered and determines the computational burden placed on downstream verification. However, the system's safety enforcement mechanisms must respond consistently regardless of how a proposal was generated. In particular, the execution architecture must cleanly differentiate between rejecting an invalid replacement candidate and responding to unexpected environmental changes. Rejecting an unvalidated proposal leaves the active trajectory in control only while its state, evidence, lease, and stopping reserve remain valid, whereas detecting an unexpected obstacle incursion along the executed path demands an immediate transition to verified braking.

11.7 Trajectory Planning Failure Modes and Infeasible Seams

On the mug reach, a replacement trajectory can fail in three distinct regimes: within the candidate itself, along the boundary where it joins the executing motion, or during execution when the environment changes. The point where a failure first manifests dictates which subsystem has the sensory evidence to reject the trajectory, as well as how much time and clearance remain to bring the machine to a safe stop. Pre-execution candidate and boundary checks intercept the first two failure modes before setpoints can reach actuator registers, whereas runtime monitoring guards against the third.

11.7.1 Checking limits and boundaries before execution

If an unvalidated chunk reaches actuator registers directly, a single out-of-bounds waypoint or an unblended velocity step at the splice instant will fault the drive before feedback monitoring can react. Before any proposed trajectory setpoint can influence actuator registers, a deterministic safety MCU therefore screens the candidate record through two independent admission gates within a measured worst-case execution budget. Fixed-degree seam calculations require bounded compute per axis, whereas checking a complete setpoint sequence scales with the candidate horizon length:

1. **Pre-execution Bound Checker:** Evaluates the proposed neural action chunk in isolation across the full candidate horizon $[t_{\text{start}}, t_{\text{plan_exp}}]$ against intrinsic kinematic, dynamic, and stopping invariants. The checker evaluates margin functions $M(t) = L_{\text{limit}} - |x(t)|$ across the signals it checks. Joint positions $q(t)$ are checked against their limits. Joint velocities are checked against contracted limits $|\dot{q}(t)| \le v_{\max} - \frac{1}{2} a_{\max} T_s$ (equation 11.10, or the $\frac{1}{4}$ reserve under the equal-velocity, zero-acceleration endpoint conditions and jerk bound of section 11.4.2). Free-space inverse-dynamics joint torques are sampled with a bound on their inter-sample variation and checked against the rated torque, $|\tau(t)| = |\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})| \le \tau_{\text{rated}}$. The checker also enforces the binding stopping invariant: the candidate's matched stopping suffix must satisfy $d_{\text{stop}} \le D_{\text{clear}}$ throughout observed free space. If any knot point or interior interval yields $M(t) < 0$ or causes the stopping runway to breach available clearance, the gate immediately rejects the candidate and logs a diagnostic fault record containing the joint index, timestamp, and negative margin. Gate rejection causes neither mechanical shudder nor physical pause, because the low-level tracking controller continues along the previously admitted trajectory as long as that trajectory's state, sensor evidence, lease, and stopping reserve remain valid. A rejected replacement simply fails to renew that lease.
2. **Boundary Checker:** Evaluates the transition seam between the currently executing trajectory A and the candidate replacement B at transition time t_{seam}. Because unconstrained sequence decoders predict open-loop action tokens without terminal derivative matching, concatenating raw chunks frequently produces sharp acceleration steps. The boundary checker addresses this in two stages. During early admission, it screens the reachable seam-state envelope projected from trajectory A. At activation, it incorporates real-time encoder estimates $(\mathbf{q}_{\text{meas}}, \dot{\mathbf{q}}_{\text{meas}}, \ddot{\mathbf{q}}_{\text{meas}})$ and their error bounds to calculate the physical seam gap $(\Delta\mathbf{q}, \Delta\dot{\mathbf{q}}, \Delta\ddot{\mathbf{q}})$. It then evaluates whether an analytical C^2 quintic bridge (equation 11.12) can reconcile the boundary without violating torque limits or the motor drive's voltage and current-slew constraints. A fast one-tick estimate, $\tau_{\text{request}} = J_{\text{eff}} \frac{\Delta\dot{q}}{T_s}$, diagnoses the demand of an unblended one-tick transition. Failure

of that estimate cannot reject a finite bridge by itself; admission checks the bridge at its actual duration, with its endpoint states, interior derivatives, torque and electrical bounds, and swept clearance.

For the CSG-25-50 joint in the seam analysis, an idealized calculation accounts solely for its 0.18 kg·m² reflected rotor inertia over a single 1 ms tick and allocates the entire 127 N·m repeated-peak rating purely to acceleration. It yields an *idealized ceiling* of 0.69 rad/s on that hypothetical one-tick step $\Delta\dot{q}$, not on a finite-duration bridge. In practice, the available torque margin is substantially smaller once link and payload inertia, gravity, friction, feedforward torque commands, coil current rise times, and thermal or DC-bus limits are deducted. The admission path must evaluate the finite bridge against this state-dependent torque margin and the drive's measured step response. The gate nevertheless rejects a step velocity discontinuity outright, even one whose idealized single-tick torque falls below that ceiling, because continuous physical motion requires a smooth, bounded C^2 bridge that satisfies its own interior limit checks.[14]

[14] **Drive and resonance response**: A sharp command can excite a joint's *identified* flexible modes, but their frequencies and damping depend on the transmission and load. Current limiting, desaturation detection, DC-bus response, and shutdown depend on the selected driver and its configuration; one vendor's desaturation-protection design note shows how such trip thresholds are set (Chen and Ramachandran 2025).

↪ The cyclic checking that enforces these trips on every tick, and the independence the monitor needs from the proposer, are developed in section 12.8.

11.7.2 Runtime tracking monitors and unexpected contact

When the arm reaches past the conveyor guard with 0.020 m of budgeted swept-volume clearance, a slipping payload or an unexpected contact after admission can invalidate the dynamic and geometric assumptions under which the plan was cleared. Independent **runtime monitors** must detect this divergence while remaining obstacle clearance still allows a controlled stop. They track four primary signals on every tick: joint state divergence, tracking error velocity, unexpected contact wrenches, and violations of the geometric assumptions declared in the trajectory record's sidecar (section 11.8). This monitoring relies on **spatial margin budgeting**, in which geometric obstacle clearance is not simply awarded to control tracking error. Mechanical compliance under load, structural deflection, and drivetrain backlash consume physical clearance before any feedback tracking error becomes observable at the encoders. The tracking error allowance ϵ_{pos} therefore equals the total swept-volume clearance minus the structural deflection and gearbox compliance allowance δ_{deflect}.

On the conveyor-guard reach, once payload deflection and gearbox compliance claim an allowance δ_{deflect} of 0.005 m, the nominal tracking allowance ϵ_{pos} drops from 0.020 m to 0.015 m. This remainder represents an upper ceiling on allocatable margin rather than a direct trip threshold, because latencies across detection, dispatch, and actuator response, along with state-estimation uncertainty and the swept motion of the fallback maneuver, all require dedicated clearance reserves. The arm tracker's validated tracking bound, the arm's counterpart to the base tracking bound ϵ_{track} that section 12.2 derives, must fit within these 0.015 m and is checked by the real-time safety loop on every control tick. If the measured Cartesian tool-position error $\|\mathbf{p}_{\text{meas}}(t) - \mathbf{r}(t)\|$ exceeds ϵ_{track}, or if a wrist-mounted six-axis force-torque sensor detects an unmodeled 10 N contact during free-space transit, the monitor immediately revokes nominal tracking and commands the validated state-matched fallback. Active physical deceleration begins once the measured detection, communication, and actuator-onset delays elapse.

A trajectory plan can also exhaust its nominal setpoints before a subsequent candidate passes admission. The safety MCU cannot simply allow the execution pointer to overrun the trajectory buffer, nor can it treat a stationary position-hold setpoint as an active, verified stop. Instead, the two deadline tiers of section 11.5.1 end the window of validity on the MCU's monotonic clock, whichever fires first. Furthermore, computing stopping clearance requires accounting for the full physical chain: sensing and detection delays, dispatch latency, actuator torque rise times, and the swept volume across the deceleration profile. Elementary calculations assuming instantaneous or constant deceleration fail to model the true physical trajectory of a smooth C^2 stopping suffix.

Every checker and monitor described here depends on an explicit execution contract between the upstream trajectory generator and the downstream real-time controller. If a planner delivers only a raw sequence of Cartesian or joint waypoints, without its validity lease, clearance margins, boundary derivative bounds, and certified stopping suffix, the low-level permission path has no principled way to distinguish safe nominal motion from dangerous dynamic drift. Safe autonomous execution requires that geometry, dynamics, and timing be verified together before setpoints ever reach actuator registers.

11.8 The Trajectory Record

Suppose the planner hands the low-level controller a bare array of 16 joint setpoints for the reach toward the mug, without the obstacle epoch, payload assumption, and pre-checked stopping suffix under which those setpoints were cleared. The MCU then has no way to tell whether the plan is still safe to run when the host stalls or the load changes. The trajectory record, handed over immutably at every replanning cycle, lets the low-level permission path decide whether to admit those setpoints without consulting the high-level planner again. The record carries the header, four deadlines, two deadline tiers, and assumption sidecar defined with the planning handoff in section 11.2.2. It works as a cyber-physical contract rather than a passive message format. Every field encodes a physical invariant that the downstream runtime can verify before a command crosses the causal boundary.

By pairing a staged admission check with the latched stopping profile of section 11.5.3, the record lets the MCU evaluate safety independently of the host planner. Even if the host operating system crashes mid-motion, the resident emergency stop profile remains intact on the real-time controller. Bringing the physical machine to rest still demands operational sensors, DC bus energy, and mechanical braking authority. The belief record and intent lease consumed at the planning handoff remain embedded in the record's lineage. The completed record binds these physical assumptions directly to the proposed motion before forwarding them to the runtime enforcer, where live sensor feedback governs per-tick tracking permission.

Every field in the contract captures an explicit condition of admission. Kinematic boundary derivatives define how the trajectory must mesh with ongoing motion at the seam. Real-time deadlines dictate exactly when nominal tracking must yield to a safe stop. The accompanying sidecar documents the precise geometric and inertial assumptions under which the validator certified the plan. Instead of taking the planner's assertion of feasibility on faith, the execution runtime uses the record to hold the motion accountable to physical reality.

Hierarchically, the trajectory record answers to the intent lease. The cryptographic digest binds the record to the exact parent lease whose target and effort ceilings the permission path must check against the proposed motion. The motion itself is stored as discrete setpoint arrays or continuous spline coefficients, parameterized by an explicit sample cadence (a 25 ms timestep Δt_{step} on the running machine, a 40 Hz stream). The record also carries the nominal trajectory expiration $t_{\text{plan_exp}}$ (400 ms for a single chunk) and canonical SI units. Pairing this explicit cadence and unit specification with the reference coordinate frame and evidence timestamp established in section 11.2 allows the joint controller to interpolate setpoints and evaluate higher-order derivatives cleanly, eliminating coordinate-frame ambiguities and any reliance on unsynchronized host clocks.

Physical feasibility is never unconditional; it holds only under the environmental geometry and dynamic loads assumed during planning. The immutable sidecar captures these conditions, recording the obstacle-map epoch, payload mass, surface friction coefficients, kinematic limits, validator software version, certified clearance margins, and the source observation identifiers. A cryptographic digest in the fixed header immutably binds this sidecar to the setpoint payload. Because safety proofs cannot extrapolate beyond validated boundaries, the payload and friction values declared in the sidecar must fall strictly within the target operational design domain (ODD) declared in section 7.8. If live sensor feedback indicates that the robot has drifted outside these operating assumptions, the permission path immediately revokes the TRACK state, recognizing that clearance margins computed for one world state do not hold in another. Diagnostic metadata, such as random solver seeds and neural model weight hashes, belongs in this audit sidecar for offline postmortem analysis, keeping the per-tick servo loop lean and deterministic.

On this machine, the chunk policy generates only the nominal motion profile, which the planning runtime then packages into a self-contained record by appending the seam state, the four operational deadlines, and a matched stopping suffix. Inside the fixed header, lineage fields record a monotonic sequence number, the evidence timestamp t_0, and the digest of the parent intent lease, while timing fields encode the four deadlines from section 11.2.2. Seam fields then define the reference frame, each joint's initial state $(q, \dot{q}, \ddot{q})$ at handoff, and the representation of an admission-checked stopping profile satisfying $d_{\text{stop}} \leq D_{\text{clear}}$. Content fields specify the payload format, byte length, and an integrity digest spanning both the setpoint payload and the sidecar.

↪ Conditional permission, the per-tick safe-set test that holds a clearance margin valid only for the live state, is developed in section 12.5.

Both the variable-length setpoint sequence and the immutable assumption sidecar sit immediately outside the fixed header. The MCU verifies their composite digest before running the admission

checks described in section 11.5. This dual-digest design decouples integrity from provenance: the payload digest validates contents, while the parent digest anchors lineage. Because the permission path admits a motion proposal based entirely on whether its physical invariants check out rather than on the identity of the process that generated it (section 4.4.1), the record deliberately omits cryptographic signatures or origin tags.

↪ **Byte layout:** Section C.15.3 illustrates a record carrying a single polynomial stopping suffix; it does not encode the worked piecewise S-curve.

Instantiated for the warehouse reach toward the mug, the record carries its own deadlines, which are those of the alternate timeline, while the running chunk lease decides how long tracking actually continues. Its parent digest points directly to the mug-acquisition intent lease, its coordinates are expressed in the robot base frame, and every timestamp references the MCU's monotonic clock. Measured relative to t_{start}, the timing fields record the commitment deadline at 300 ms, the nominal trajectory end at 400 ms (derived from 16 setpoints spaced at 25 ms), and the final safe-stop horizon at 575 ms. The 240 ms blend deadline remains a planner scheduling target outside the fixed header. Four candidates can end tracking authority: the admission cutoff at 278 ms, the lease timeout 60 ms after the last admitted chunk, the intent lease expiry, and evidence staleness. On the running machine the lease timeout binds first. The commitment fields then serve as a backstop that acts only under a lease long enough to outlast the cutoff.

The worked seam specifies a 275 ms, jerk-limited C^2 stopping suffix bounded by peak acceleration 5 m/s^2 and jerk 66.7 m/s^3. It is anchored at the commitment state and brings the arm safely to rest from its 1 m/s TCP velocity limit within 137.5 mm. Serializing and admitting this analytical S-curve requires a specified bounded profile encoding and decoder, bound to the admitted path and stop-entry state, with decoding and validation included in the admission budget (a representation not supplied by the illustrative single-polynomial wire layout). Under the running lease, a lease timeout triggers the state-matched fallback built from the measured state. From zero-acceleration cruise that fallback follows the same S-curve and travels the same distance (section 11.5.2), so the lease path and stop together consume 219.5 mm of the available 300 mm clearance. The 278 ms commitment cutoff never extends the lease. In the sidecar, the planner asserts a payload mass bounded by 3 kg, unobstructed free space leading up to the grasp contact, and operating conditions inside the target ODD. The real-time permission path evaluates all of these upon admission and audits them on every control cycle.

In this concrete trajectory, the temporal ordering strictly satisfies $t_{\text{start}} \leq t_{\text{commit}} < t_{\text{plan_exp}} < t_{\text{stop_end}}$, and the MCU enforces these timing boundaries according to the TRACK and STOP transition rules developed in section 11.2. Because safety requires both temporal validity and physical realizability, a well-formed timestamp in the header cannot compensate for an unviable stopping suffix. To make latency and timing jitter auditable, the system logs the timestamps for evidence capture, plan request, validation completion, admission, activation, control transfer, and stop completion in a synchronized time domain, exposing the end-to-end response delay of the entire perception-planning-control pipeline.

Every artifact in this runtime chain, from the observation contract and the belief record through the intent lease to the trajectory record itself, declares its physical dependencies and its expiration deadline. The trajectory record proves only what the planner could verify at plan time: that a collision-free stop remains physically feasible from any state the robot might reach before commitment, given the obstacle scene and dynamic payload asserted in its sidecar. Common design pitfalls often mistake this bounded, conditional guarantee for unconditional safety, overlooking what happens when physical reality deviates from the planner's model.

11.9 Fallacies and Pitfalls

A planned trajectory remains physically feasible only as long as its environmental, dynamic, and computational assumptions hold true. In embodied systems, failures rarely stem from minor mathematical errors along the nominal interior of a path. Instead, they emerge at operational and physical seams: when dynamic obstacles intrude into an active swept volume, when concatenated action chunks excite mechanical resonances, when host inference pipelines miss scheduling deadlines, or when controllers attempt to stop faster than actuator dynamics permit. Both the trajectory admission gate and the real-time execution monitor must defend against these five failure modes.

Fallacy: *Static admission checks protect a machine against obstacle displacements during active execution.*

Static geometry confirms clear space only at admission time; dynamic workspaces demand continuous runtime monitoring. Consider a robotic manipulator reaching toward an assembly target. When the trajectory is first admitted, the swept volume between the end-effector and the target is completely clear. Halfway through execution, however, an operator slides a storage tote into the arm's path. Although the admission check was mathematically correct when computed, the physical workspace has changed. Static pre-screening guarantees only that a trajectory was safe in the past; it cannot guarantee that the path remains clear as time elapses. Execution monitoring must continuously track obstacle clearances and initiate deceleration while adequate stopping distance remains. Systems architects must explicitly budget sensor bus latency and vision pipeline transit times, reserving enough spatial stopping margin to ensure the machine can halt before an encroaching obstacle breaches its swept volume.

Architect's countermeasure: Pair high-level spatial screening on the host with low-latency local clearance sensing processed directly by the MCU. Explicitly allocate the sensing refresh rate, bus transit latency, and MCU memory footprint. If an unmodeled obstacle breaches the reserved spatial safety margin δ_{margin}, immediately abort nominal tracking and engage the pre-validated, state-matched fallback trajectory.

Pitfall: *Checking each action chunk alone without examining the motion the chunks compose.*

Individual chunk feasibility does not guarantee smooth motion across seams; stochastic neural chatter can overstress a mechanical transmission. When generative sequence policies (such as diffusion models or action-chunking transformers) are sampled stochastically, successive chunks frequently disagree in direction or mismatch acceleration at their boundary. While every individual chunk satisfies kinematic position and velocity bounds in isolation, concatenating them without boundary constraints induces rapid torque reversals $\Delta\tau = J_{\text{eff}}\Delta\ddot{q}$. Amplified by the reflected rotor inertia ($N_{\text{gear}}^2 J_{\text{rotor}}$), this high-frequency chatter excites flexible structural modes, accelerates gearbox wear, and causes severe Joule heating ($i_q^2 R_{\text{phase}}$) in the stator windings. The admission pipeline must evaluate derivative continuity across every chunk boundary seam, enforce C^2 quintic smoothing, and track recent motion history to reject oscillating command sequences before they reach the motor drives.

Architect's countermeasure: Enforce multi-chunk frequency and reversal monitoring across a sliding execution window. Reject any trajectory candidate whose frequency spectrum or repeated effort exceeds the motor's identified mechanical or thermal envelope. If smoothing filters alter a candidate chunk, re-run the full kinematic, seam continuity, and terminal stop checks on the modified trajectory prior to admission.

Pitfall: *Delaying fallback transition past the hard stopping horizon in the hope that delayed compute finishes.*

An overdue compute cycle never justifies extending a physical commitment deadline. In an alternate schedule operating with an admission cutoff of 278 ms, the active trajectory record supplies nominal setpoints that expire at 400 ms. The record carries a 275 ms matched stopping suffix that must finish deceleration by 575 ms. To guarantee physical braking onset by 300 ms, the MCU must dispatch the fallback trajectory no later than the 278 ms cutoff unless a verified replacement plan has already arrived. If the host application processor reports that neural inference is nearly complete but has not yet delivered a validated plan, delaying the fallback transition consumes the runway reserved for stopping. The MCU must dispatch the resident stopping suffix by the earlier cutoff so its physical onset occurs at commitment. Progress reports cannot extend a mechanical stopping budget; only an admitted, physically verifiable trajectory justifies continued forward motion.

Architect's countermeasure: Bind the physical commitment deadline t_{commit} strictly to a monotonic hardware timer on the MCU. When the timer reaches the admission cutoff, close the admission window and dispatch the resident stopping suffix immediately, disregarding any speculative progress updates from the host processor.

Pitfall: *Relying solely on feedforward dynamic models when physical contact or payload mass can change abruptly.*

Feedforward model inversion without closed-loop disturbance bounds can drive an unloaded arm beyond its admitted trajectory. Consider an arm lifting a heavy workpiece near its 3 kg payload limit. To track the aggressive trajectory, the controller injects feedforward torque computed from an inverse dynamics model of the fully loaded arm. If the workpiece slips from the gripper mid-lift, the drives continue applying the large lifting torque intended for the heavy load to an arm that is now empty. Without a timely feedback correction or torque clamp, the excess torque accelerates the arm above its planned rate. While the admitted trajectory has not changed, the physical mass properties that justified those torque commands have vanished. High-bandwidth feedback monitoring must detect the resulting tracking error and clamp torque commands before the arm strikes a mechanical joint stop or triggers inverter thermal derating. Feedforward dynamic models significantly reduce tracking lag under nominal conditions, but they must never run without closed-loop supervisory bounds.

Architect's countermeasure: Implement real-time torque limiters and tracking-error monitoring ($\|\mathbf{p}_{\text{meas}} - \mathbf{r}\| \leq \epsilon_{\text{track}}$) directly inside the 1 kHz servo loop. If an abrupt payload drop or unexpected contact causes acceleration to spike, clamp motor currents immediately and transition the system to compliance mode or an emergency stop.

Fallacy: *A trajectory can omit an achievable terminal stopping suffix if replacement plans are expected to arrive in time.*

A trajectory without a pre-latched stopping continuation is an unhedged bet on infinite compute availability. Traveling at its drive limit of 1.5 m/s, a mobile manipulator's base requires 1,031.25 ms for even the shortest deceleration profile its kinematic bounds permit. That minimum stop follows an S-curve profile $T = v_c/a_{\max} + a_{\max}/j_{\max}$ evaluated at a credible braking acceleration 2 m/s² and jerk limit 7.11 m/s³. Sizing a proposed stopping suffix using the idealized constant-deceleration formula v_c/a_{brake} yields 750 ms. However, no physical actuator can ramp deceleration instantaneously without infinite jerk. Forcing a stop into that truncated duration inevitably violates the acceleration bound, the jerk limit, or both, exceeding what the chassis and its payload can tolerate. Expecting the next plan to arrive on time does not make an unfeasible fallback executable. The admission gate must verify that the terminal stopping suffix strictly respects $a_{\max}$ and $j_{\max}$ throughout its duration. With an S-curve, this requires validating T_{stop} against equation 11.17 and confirming that the stopping distance satisfies $d_{\text{stop}} \leq D_{\text{clear}}$ within verified clearance. The admission gate must reject any record that fails this test. Every admitted trajectory requires a physically executable termination path to protect the machine when upstream inference stalls.

Architect's countermeasure: Require every candidate trajectory record to incorporate a complete, closed-form C^2 stopping profile along its admitted corridor. Deceleration and jerk along this profile must remain strictly within vehicle and payload stability limits, while the stopping distance must satisfy $d_{\text{stop}} \leq D_{\text{clear}}$ within verified clear space. Reject any record at the admission gate that lacks this independently executable stopping guarantee.

11.10 Summary

A trajectory transforms a high-level goal into timed physical motion. On an active machine, it must resolve three questions that a geometric path leaves unanswered: how long the motion remains valid and executable, how it splices smoothly into the motion already under way, and how the machine stops safely if no replacement arrives. A multi-step task such as the reach for a mug progresses through a continuous sequence of renewed, validated trajectory prefixes without forcing the entire task into a single observation window. Predictive models and planners generate candidate trajectories, but the runtime system grants execution authority only after verifying fresh sensor evidence, admission deadlines, and the machine's true state. When a replacement trajectory arrives late, the controller falls back to its resident stopping maneuver. Tail latency bounds ($Q_{1-\epsilon}(L)$) therefore quantify the expected frequency of service degradation to this safe fallback, while the machine's physical safety rests on the stopping profile itself.

Whether a physical system can track a candidate trajectory is distinct from whether that trajectory avoids obstacles. Kinematic feasibility alone can fail in subtle ways. Near kinematic singularities,

the manipulator Jacobian maps modest Cartesian end-effector velocities into unbounded joint rates. Damped least-squares inversion caps these joint steps, but does so by accepting tracking error that can still push joints against their limits. Dynamic feasibility imposes even stricter constraints. The rigid-body manipulator equations translate planned motions into actuator torques, where reflected rotor inertia amplifies any velocity or acceleration discontinuity across trajectory seams into severe torque spikes. A quintic bridge matches position, velocity, and acceleration across the boundary, eliminating these impulsive torque requests. Similarly, time-optimal path parameterization drives actuators along their torque envelopes, but bang-bang acceleration profiles demand instantaneous torque steps that physical drives cannot achieve. Enforcing an explicit jerk bound rounds these transitions at minimal time cost, turning theoretical path timings into executable motor commands. Whether candidates originate from sampling-based search, trajectory optimization, or learned action chunking, the underlying physical obligations remain unchanged. Each planning method generates a prospective motion proposal, but never a license to execute without validation.

Real-time execution relies on strict timing contracts between the planner and the low-level controller. A 60 ms lease halts a stalled proposal stream using a state-matched stopping response, reserving an illustrative 80.5 mm safety buffer to absorb sensing delay and model uncertainty. In an alternate longer-lease configuration, replacement admission closes strictly by 278 ms so that physical deceleration can initiate no later than 300 ms. Both regimes enforce the identical systems invariant: every active trajectory must reserve sufficient travel to accommodate perception-action latency and the entire stopping maneuver. The base faces the same arithmetic. Its jerk-limited S-curve, the shortest stop under the stipulated load bounds, raises the partial stopping budget at the drive limit to 1,123.84 mm (row 6 of table 12.1). Because this exceeds the available 1.10 m rack-end clearance, the runtime controller must proactively restrict operating speed. Table 11.5 lists the equations behind these admission decisions, detailing the physical limit each bounds and where it is derived in this chapter.

Table 11.5: Key equations of trajectory planning: Each load-bearing equation of the chapter, the limit it bounds or the decision it settles, and the equation where it is derived.

Equation	What it bounds or decides	Derived in
$\Delta\mathbf{q}_\lambda = \mathbf{J}^\top(\mathbf{J}\mathbf{J}^\top + \lambda^2\mathbf{I})^{-1}\Delta\mathbf{p}$	The inverse-kinematics step near a singularity, capped at $v_{\text{TCP}}/(2\lambda)$ at the price of tool error	equation 11.2
$L = t_{\text{age}} + \Delta_{\text{da}}$	Replacement latency from capture to admission, which the renewal schedule must absorb	equation 11.3
$\epsilon \le \epsilon_{\text{mission}}/K$	The per-handoff tail that a mission target implies, far beyond p99	equation 11.4
$T_{\text{record}} \ge T_{\text{request}} + Q_{1-\epsilon}(L) + T_{\text{reserve}} + T_{\text{stop}}$	Conservative capture-latency charge; full reserve includes scheduling slack and response; stop included	equation 11.5
$\tau = \mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{g} - \mathbf{J}^\top\mathbf{F}_{\text{ext}}$	The torque a planned motion demands, evaluated by inverse dynamics at admission	equation 11.8
$\lvert\dot{q}_k\rvert \le v_{\max} - \frac{1}{2}a_{\max}T_s$	A sampled velocity limit that still holds between samples	equation 11.10
$\tau_{\text{request}} = J_{\text{eff}}\,\Delta\dot{q}/T_s$	The torque a seam's velocity step asks of the reflected inertia	equation 11.11
$j_{\text{joint}}(t) = 6a_3 + 24a_4 t + 60a_5 t^2$	The jerk of a quintic bridge, whose extrema the checker evaluates in closed form	equation 11.13
$V_{\text{bus}} \ge L_{\text{phase}}\,di_q/dt + R_{\text{phase}}\,i_q + K_e\omega_e$	Whether the drive has the voltage for the current slew a bridge demands	equation 11.14
$\tau = \mathbf{a}_\tau(s)\ddot{s} + \mathbf{b}_\tau(s)\dot{s}^2 + \mathbf{g}(s)$	The path accelerations the torque limits allow, and so the fastest timing of a path	equation 11.15
$d_{\text{stop}} = v^2/2a_{\max} + v\,a_{\max}/2j_{\max}$	The shortest stop within deceleration and jerk bounds, above the plateau speed $a_{\max}^2/j_{\max}$	equation 11.17
$d_{\text{stop}}(t_{\text{commit}}) \le D_{\text{clear}}(t_{\text{commit}})$	Whether a trajectory may be admitted at all	equation 11.18

Key Takeaways: Timed motion and independent stopping

- **Prediction is not execution authority**: A long predicted path can guide motion, but the controller executes only short, validated prefixes. While renewal cadence dictates how frequently new plans are requested, target evidence retains its own capture-based timestamp, and its evidence horizon bounds how long it stays valid.
- **Reserve a stopping response from the current state**: A stopping suffix is mathematically valid only for its declared entry state. If an execution lease or sensor observation expires, the machine must possess a pre-validated fallback trajectory, sufficient reserve energy, and enough physical clearance to absorb both response latency and full deceleration.
- **Admission precedes physical commitment**: The alternate arm schedule closes replacement admission strictly by 278 ms to ensure a physical stop can initiate at 300 ms. A late planner proposal cannot spend or delay this reserved response window.
- **A collision-free path is not inherently executable**: Near singularities, Cartesian end-effector speeds demand infeasible joint velocities. Damped least squares bounds the inverse-kinematics step only by introducing tracking error. Similarly, time-optimal path timing produces discontinuous torque demands that trigger actuator saturation. Adding quintic seams and jerk bounds smooths transitions, but interior joint limits, inverter voltage headroom, and clearance remain binding constraints.
- **Trajectory smoothness dictates stopping distance**: Finite jerk bounds make smooth stopping profiles significantly longer than ideal constant-deceleration braking. An S-curve profile pays the least, the equivalent of cruising for half of one jerk ramp, yet that overhead still sets operating speed limits near rack ends and obstacles.
- **Candidate rejection does not require immediate stopping**: The running trajectory continues executing normally as long as its state, sensor evidence, lease deadline, and fallback trajectory remain valid. Rejecting an invalid replacement proposal simply leaves those original expiration boundaries unchanged.
- **Admission and runtime enforcement require separate system budgets**: Evaluating a candidate plan requires an asynchronous, bounded scan over the entire trajectory chunk, whereas per-tick enforcement performs strict real-time checks against current state and local sensor data. System designers must isolate their CPU time, memory footprints, and bus interference to prevent high-level planning from disrupting the low-level control loop.

A plan stays valid only while the stop it carries can still be executed. When a timing, seam, or clearance check fails, the remedy is to shorten the lease, slow the motion, or lengthen the bridge until every check passes with the resident stop still inside the clearance, never to wait for a better proposal. The stall runway shows which term to work on. If the travel before stop onset rivals the stop itself, the lease is the term to shorten. If the stop dominates, the deceleration or jerk bound the load tolerates is the term to measure.

For motion, the third law, proposal is not permission (principle 3), reduces to a clearance test. However smooth, collision-free, and timely a candidate is, the MCU admits it only with an independently executable stop that fits the clearance that remains, and that test does not depend on how the candidate was generated. Admission is not a waiver either. The conditions certified at admission remain obligations on every control tick, so the machine must keep checking live state against the clearance it has left.

11.11 Historical Perspective and Further Reading

Motion planning became a concrete computational discipline when Lozano-Pérez (1983) mapped geometric collision avoidance into configuration-space search, shrinking the geometry of a complex moving body to a single point. In the same year, Schwartz and Sharir (1983) formulated the piano movers' problem through exact algebraic decomposition. While theoretically complete, exact algebraic methods scale doubly exponentially with degrees of freedom, rendering them intractable for

multi-joint manipulators. In the 1990s, randomized sampling bypassed this geometric bottleneck by trading exact completeness for probabilistic guarantees. Probabilistic roadmaps (Kavraki et al. 1996) introduced multi-query graph construction across free configuration space, while rapidly-exploring random trees (LaValle 1998) solved single-query kinodynamic problems by incrementally biasing search trees toward unexplored regions. Later, RRT* (Karaman and Frazzoli 2011) proved that standard random trees almost surely fail to find optimal paths, introducing local edge rewiring to guarantee asymptotic convergence toward the optimum. Together, these foundational algorithms resolve where a robot may safely pass, which represents the geometric half of motion planning.

The second half, timing and dynamics, followed an independent line of work. Bobrow et al. (1985) showed that the fastest traversal along a pre-specified path under actuator torque limits follows a bang-bang structure in the phase plane, switching between maximum acceleration and maximum deceleration. Decades later, TOPP-RA (Pham and Pham 2018) reformulated time-optimal path parameterization through reachability analysis, casting the search into a sequence of linear programs that execute in time linear in the number of path waypoints. Pure time-optimal trajectories command discontinuous accelerations, however, and the resulting infinite jerk excites mechanical resonances that degrade physical drives. To produce smoother execution, roboticists drew inspiration from biology: Flash and Hogan (1985) showed that human reaching movements naturally minimize jerk. In robotic manipulation, Kröger and Wahl (2010) demonstrated that complete, jerk-limited trajectory profiles could be synthesized deterministically from an arbitrary initial state within a single millisecond-scale control cycle. Ruckig (Berscheid and Kröger 2021) generalized this formulation to arbitrary target kinematic states and asymmetric limits, establishing the fast, closed-form generator that produces this chapter's stopping profiles. Trajectory optimization reached robotics along a parallel route. Direct collocation (Hargraves and Paris 1987) originated in aerospace trajectory design to transcribe continuous optimal control into sparse nonlinear programs. Receding-horizon control matured in the process industries (Qin and Badgwell 2003), where slow chemical plant dynamics afforded ample time for numerical convergence. Later, algorithms like CHOMP (Ratliff et al. 2009) brought functional gradient optimization directly to robotic arms, optimizing trajectory smoothness and obstacle clearance simultaneously.

The commitment problem, deciding when a planned motion becomes irrevocable and keeping execution safe after that point, is the most recent to mature. Because physical systems possess inertia, safety cannot be judged solely by examining whether the current state is collision-free. Fraichard and Asama (2004) formalized inevitable collision states, establishing that a planner must verify that a system never enters a state from which every future control trajectory culminates in a crash. This formulation gave a robotic form to the viability kernels that Aubin (1991) had introduced in control theory. Modern learning-based methods have transformed how candidate motions are generated. Action chunking (Zhao et al. 2023) and diffusion policies (Chi et al. 2025) predict multi-step action trajectories directly from visual observations, capturing dexterous behaviors without explicit geometric reconstruction. To bridge the latency gap between neural network inference and high-frequency actuation, real-time chunking (Black, Galliker, et al. 2025) conditions incoming predictions on the action sequence already committed to the hardware controller. Regardless of how candidate motions are generated, the execution runtime must enforce physical safety. The trajectory record binds the admitted trajectory to its initial seam state, enforces hard commitment deadlines, and guarantees a deterministically executable stopping suffix whenever the policy stalls.

For further reading, six works trace the lineage from classical planning to modern learned proposals:

- Qin and Badgwell (2003) provides a comprehensive survey of industrial model predictive control, charting its progression from early petrochemical applications to commercial implementations and illustrating how receding-horizon optimization operates on physical systems whose long time constants tolerate multi-second solve times.
- Fraichard and Asama (2004) formalizes inevitable collision states and defines the dynamic conditions required for provable motion safety under actuator limits, establishing the theoretical foundation for the commitment deadlines analyzed in section 11.5.3.
- Biagiotti and Melchiorri (2008) provides an exhaustive reference on trajectory generation profiles across robotics and industrial automation, detailing the analytical formulation of

polynomial splines, filter-based smoothers, and the double-S profile used for the C^2 stopping suffix in section 11.5.2.

- Kröger and Wahl (2010) develops the mathematical framework for online trajectory generation, deriving algorithms that compute kinematically feasible, jerk-limited motion profiles from arbitrary initial states within single-cycle real-time deadlines.
- Lynch and Park (2017) offers a unified, geometrically rigorous treatment of configuration space, kinematics, rigid-body dynamics, time scaling, and motion planning, serving as a comprehensive companion to section B.6.1.
- Chi et al. (2025) details diffusion policies for robotic manipulation, examining receding-horizon execution of predicted action sequences and presenting the modern learned proposers whose seams, deadlines, and fallback stopping profiles this chapter evaluates.

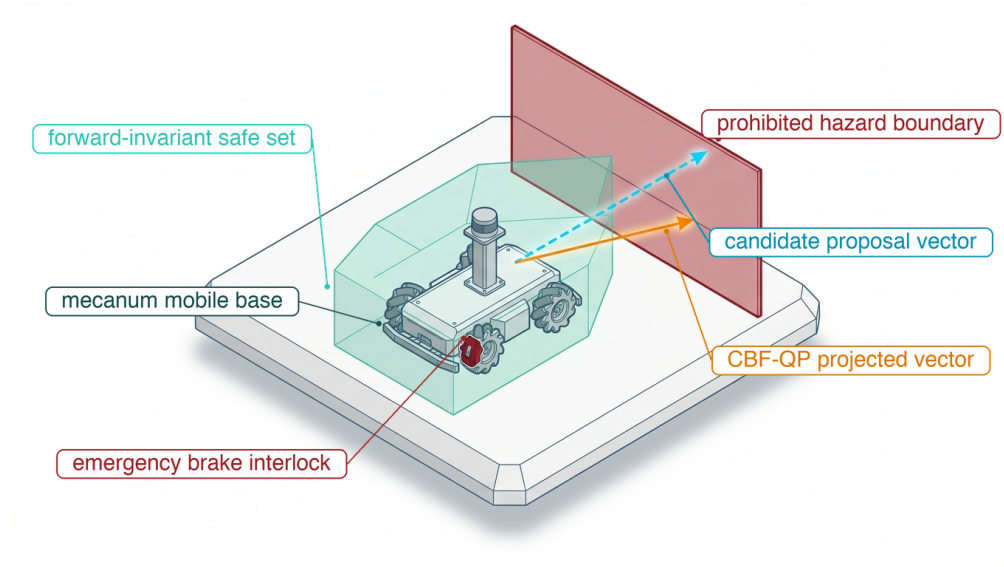
What's Next: From admitted plans to per-tick permission

Admission judges a trajectory against the state the planner predicted, but the machine executes it against the state it actually reaches. Between the two, the body lags its reference, a person can step into the aisle, and an actuator can run short of torque, and the stopping budget assembled here has not yet charged the first of these. On the base the gap is already decisive, because the jerk-limited stop carries the partial budget to 1,123.84 mm against 1.10 m of rack-end clearance at the drive limit before the body's departure from its reference is priced. Chapter 12 begins with the per-tick gatekeeper, the permission-path check that runs on every control tick against live state. It prices tracking error as an inset on the stopping distance and derives a candidate aisle speed whose timing and braking premises still require qualification. When a check fails, it selects among fallback responses that include the resident stopping suffix latched at admission.

DRAFT

12

Safety Enforcement

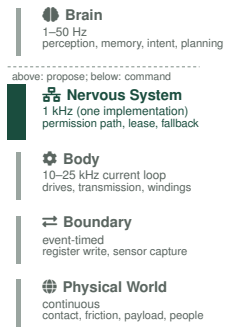


12.1 The Per-Tick Gatekeeper
12.2 The Tracker Handoff and Dynamic Reference Envelopes
12.3 Stopping Envelopes
12.4 Authority Exhaustion and Emergency Fallback
12.5 Safe Sets as Conditional Permission
12.6 Refusing Proposals and Enforcing State Invariants
12.7 The Minimal Intervention Principle and Optimization Filters
12.8 Admission, Cyclic Checking, and Monitor Independence
12.9 The Fallback Ladder
12.10 Runtime Enforcement Failure Modes
12.11 The Enforcement Record
12.12 Fallacies and Pitfalls
12.13 Summary
12.14 Historical Perspective and Further Reading

Purpose

Why does an admitted plan still need permission on every control tick?

A trajectory can be feasible when it arrives and become unsafe while the machine follows it. The body lags its reference, a payload or a hot motor erodes braking authority, a person steps into the path, or the learned proposer stalls with its last command still in force. A machine that checks only before motion begins discovers these changes after the energy is already moving, and the proposer cannot be trusted to notice them, because nothing in its output bounds stopping distance, actuator effort, or timing. Learned proposals therefore need an independent permission path that re-checks current conditions on every tick, keeps useful motion inside a stopping envelope, corrects a command when a feasible correction exists, and holds a response it can execute without the proposer before the remaining margin is spent. Each of those guarantees depends on bounded sensing error, available actuator force, and a decision delivered on time. Enforcement joins these conditions at the causal boundary, where a permitted command becomes motion that software cannot undo.



Learning Objectives

- Calculate the finished stopping budget and its speed ceiling, treating the tracking bound as a premise the enforcer monitors on every tick
- Derive how momentum and commanded effort change the remaining safety margin, and the contact force at a latch and at a handover that caps approach speed
- Explain how a control barrier function limits motion while its model, sensing, actuator, and timing assumptions hold
- Formulate a quadratic program that minimally adjusts a proposal within feasible actuator and safety constraints
- Select state-matched fallback responses when sensing, computation, or actuator limits invalidate normal enforcement
- Distinguish the permission path's unrated performance checks from the certified drive, scanner, and arm safety functions that carry the protective claim beneath it
- Construct an enforcement record and formulate the admitted command that binds each check to its detector, its fallback response, and the records it consumes

12.1 The Per-Tick Gatekeeper

Every physical AI system ends in the same act. A command leaves software, becomes current in a motor winding, and puts momentum into a world that does not roll back. Safety enforcement is the runtime mechanism that decides, one control tick at a time, whether the next command may cross that causal boundary. Checking a plan once before it runs does not settle the question, because admission judges the plan against the state the planner predicted, and the machine then executes it against the state it actually reaches. Between those two moments the body can lag its reference, an actuator can run short of torque, a person can step into the aisle, and the learned proposer can stall or emit a setpoint the plan never contained. Enforcement exists so that none of these events is discovered only after the energy has been delivered.

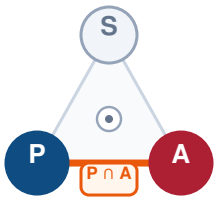
The central constraint is that the learned proposer can neither be trusted with that decision nor be discarded. Its competence is what makes the machine useful, yet nothing in its output bounds stopping distance, actuator effort, or timing. Enforcement resolves the tension by separating proposal from permission, the third law (principle 3), and by giving the permission side three jobs. It must hold a stopping reservation, the clearance the machine needs to arrest its motion from the current state; it must correct a proposal minimally when a feasible correction exists; and it must select a response it can execute without the proposer when none does. The chapter builds the three jobs in that order.

On the warehouse mobile manipulator, the safety microcontroller (MCU) admits the run down the aisle only after the matched stop suffix named in its trajectory record is resident in the MCU's memory and feasible from the commitment entry state defined in section 11.5.1. Commitment is the instant after which the machine can no longer abandon the plan's motion without that latched suffix, so the suffix is matched to the state the machine will have at that instant, not to the states it passes through on the way. Before commitment, a different response must cover the motion. Admission requires an independently executable, state-matched fallback from every state reachable up to that instant. It also requires a bounded representation of the S-curve suffix that the MCU can decode, validate, and execute within its declared workload (the polynomial layout of section C.15.3 is an illustrative encoding rather than that S-curve representation). Because that admission step checks the plan only once before motion starts, the machine still executes the plan one millisecond tick at a time, and enforcement is the check made on every one of those ticks together with the response it selects when even the resident stop no longer fits.

The enforcer, the permission path's per-tick gatekeeper, sits on the permission side of the proposal boundary. It is a hard real-time component that evaluates candidate proposals, verifies invariant state bounds, and holds the exclusive hardware authority to permit current flow or command a fallback stop, keeping the task competence of learned policies while denying them authority over geometric clearance, thermal limits, and human-safety invariants.

↩ The trajectory record and the matched stop suffix it names are established in section 11.8.

↩ The per-tick permission cadence on which the enforcer runs is established in section 4.3.1.



Enforcement sits on the Plan-to-Act edge: no proposal reaches a drive unchecked.

In the architecture this chapter follows, the enforcer executes on a dedicated microcontroller wired to the sensing and drive interfaces it needs. It checks each candidate command before the drive can apply it. A packet firewall offers a limited software analogy: a sender may request an operation without having authority to perform it. Unlike a packet filter, the enforcer must also account for momentum already present in the plant and preserve an achievable response when it refuses a request.

The architecture separates a capable proposer, a resident protective controller, and an output selector. This is the **Simplex architecture**[1] (Seto et al. 1998; Sha 2001), the runtime-assurance pattern that section 1.7 first named. The proposer supplies useful candidates, while the permission path retains the exclusive ability to select a physically feasible response without waiting for it.

↪ The silicon placement that keeps the permission path independent of the proposer is developed in section 13.6.

The rest of the chapter follows the running machine through that per-tick check. The starting point is the deficit planning left open, when the jerk-limited stop carried the base's stopping budget past the rack-end clearance at the drive limit while the body's departure from its reference remained unpriced: the first quantity the enforcer needs is how far the body can stray from the reference it is asked to follow, because every clearance the permission path defends must also contain that error. Once the tracking bound supplies that missing spatial term, the finished stopping budget fixes the reservation and a candidate aisle speed that still requires physical qualification. Actuator saturation then reveals how quickly that reservation shrinks once gravity or thermal limits spend braking authority. A control barrier function turns the reservation into a constraint on the next command. A least-change filter can then correct a violating proposal, both in free space and in contact, inside a declared cycle budget on a processor the proposer cannot preempt. When a check fails or a lease expires, the fallback hierarchy supplies the surviving hardware response, and the chapter closes with the infrastructure failures that break these premises and the enforcement record that binds every check to its evidence.

[1] **Simplex Architecture**: The design came from Carnegie Mellon in the 1990s as a way to upgrade the controller of a running plant without stopping it. An untested new controller drove the plant while a simple verified controller and a switching rule stood ready to take it back (Seto et al. 1998). The learned proposer occupies the untested controller's seat, and the design's premise, that the region from which the verified controller can still recover is known in advance, is the stopping envelope this chapter computes.

12.2 The Tracker Handoff and Dynamic Reference Envelopes

Even when communication latency is zero and the permission path admits every reference command without delay, physical robots follow reference trajectories imperfectly. Mechanical compliance, inertia, and non-ideal motor dynamics produce tracking discrepancies. The feedback tracker, the closed-loop controller that turns admitted references into actuator effort, must bound this deviation across the entire maneuver. The bound is a **worst-case tracking error tube** (ϵ_{track}), an upper bound on the spatial departure of the tracked position output from its nominal reference. For Cartesian point tracking, $\mathbf{p}(t)$ and $\mathbf{r}(t)$ denote the actual and reference positions evaluated in the same spatial frame. For whole-body systems such as mobile manipulators, the execution stack defines a declared output map or metric weighting that bounds the maximum physical swept-volume displacement in meters. The safety enforcer depends directly on this bound. Any spatial clearance reserved around a planned reference path must guarantee physical containment of the actual robot body.

Provided that the physical system begins within an initial qualified error region and its operating parameters (reference velocity, command acceleration, payload mass, external disturbances, and actuator temperature) stay within their validated operating envelope, the feedback controller guarantees the transient position-error bound:

↩ The admitted setpoints and feedforward torques that the tracker turns into actuator effort are established in section 11.2.

$$\|\mathbf{p}(t) - \mathbf{r}(t)\| \leq \epsilon_{\text{track}} \quad \forall t \geq 0.$$

This guarantee forms a software contract between the tracking layer and the safety enforcer. Rather than re-analyzing raw plant dynamics or reconstructing low-level controller state on every millisecond control cycle, the enforcer consumes this certified error bound as an invariant. However, this contract confines physical motion to the ϵ_{track} tube only while the operational conditions used to establish the certificate remain valid.

The magnitude of ϵ_{track} is a physical distance dictated by hardware properties: reflected rotor inertia ($N_{\text{gear}}^2 J_{\text{rotor}}$), gearbox backlash, structural compliance, and actuator torque saturation. While a steady-state stiffness calculation balances constant external disturbances against closed-loop restoring force, steady-state metrics fail to capture transient overshoot, phase lag, or settling oscillations excited during aggressive maneuvers. Consequently, engineering teams cannot rely on static linear approximations; they must support the tracking tube bound ϵ_{track} with a bounded plant and controller model, defined operating limits, and full-envelope dynamic qualification tests. Tests can

expose an invalid bound; the tested trajectories alone do not prove containment for every admissible maneuver.

↩ Reflected rotor inertia, $N_{\text{gear}}^2 J_{\text{rotor}}$, is established in section 2.4.3.

For the baseline warehouse mobile manipulator, the illustrative dynamic qualification target is $\epsilon_{\text{track}} = 40$ mm (illustrative; see the Reader Guide). The tracking controller sustains this bound only while three coupled operational constraints remain satisfied. First, reference trajectories must not demand accelerations exceeding drive torque limits. Second, external disturbance forces must remain below d_{max}, the qualified bound on external disturbance force. Last, actuator coils must operate below their continuous thermal derating thresholds and DC bus voltage limits. A reference trajectory demanding acceleration beyond peak torque drives the actuators into saturation, causing the physical chassis to lag behind the command. Without aggressive anti-windup clamping, the integrator state accumulates severe windup, prolonging recovery long after the reference demand subsides. Outside this qualified operating envelope, the tracking certificate is void, and the system loses its guaranteed error bound.

↩ Continuous thermal derating of motor windings is established in section 2.5.

Because the physical robot can deviate from its planned reference by up to ϵ_{track} in any direction, the safety architecture must subtract this uncertainty margin directly from the admissible workspace. Let h measure the spatial clearance remaining around a reference. To guarantee that the physical robot maintains non-negative clearance, the enforcer applies an inset contraction:

$$h_{\text{contracted}}(\mathbf{r}) = h(\mathbf{r}) - \epsilon_{\text{track}} \geq 0.$$

A reference trajectory that maintains nominal clearance c from a warehouse rack upright guarantees the physical chassis only $c - \epsilon_{\text{track}}$ of actual clearance. Consequently, the safety enforcer evaluates reference commands against a contracted boundary that adds 40 mm to the nominal buffer. As detailed in section 12.3, this tracking inset compounds directly with stopping distance budgets, where localization uncertainty and transport delay consume additional clearance. Mechanical wear poses an insidious operational risk here. If worn drive bearings, tire degradation, or altered motor parameters double the true tracking error to 80 mm without updating the enforcer's parameter registry, the physical base consumes an unbudgeted 40 mm of clearance that the enforcer assumes remains available. An unmodeled tracking degradation silently undermines every higher-level safety margin.

↩ Clearance insets that absorb spatial sensing error are established in section 8.6.

To maintain safety guarantees in production, the runtime monitoring stack must treat ϵ_{track} as an actively verified contract rather than a static design assumption. The underlying real-time middleware publishes ϵ_{track} alongside its validity envelope, while hardware monitors estimate the instantaneous tracking error $e(t) = \|\mathbf{p}(t) - \mathbf{r}(t)\|$ on every 1 ms controller tick. Agreement with encoder-reported position is a runtime proxy, not independent evidence of physical localization; the localization allowance must contain drift that these channels cannot expose. If an observable disturbance or actuator thermal throttling causes the monitored $e(t)$ to exceed ϵ_{track}, the tracking contract is broken. Upon detecting this violation, the safety enforcer revokes the admitted command and activates a state-matched emergency fallback maneuver. Maintaining physical containment during this transition requires that the fallback braking trajectory remains dynamically feasible even after the tracking bound is lost.

> **Systems Perspective 12.1: The inset margin invariant**
>
> A spatial stopping boundary must reserve margin for the tracking loop's qualified error bound. The runtime stopping envelope (equation 12.1) incorporates this geometric inset alongside displacement accumulated during transport lag before admitting reference trajectories. When the safety barrier certificates of section 12.5 are formulated in non-spatial coordinates (such as energy or momentum limits), the safety layer must map the tracking tube into a corresponding conservative bound on barrier variation, rather than subtracting a scalar distance. Evaluating barrier certificates against nominal trajectory geometry alone risks admitting references whose physical tracking tubes intersect obstacle boundaries.

Control theory provides rich methodologies for synthesizing tracking controllers, state observers, and disturbance rejection algorithms (Slotine and Li 1991; Åström and Murray 2021). In embedded industrial robotics, systems frequently deploy a **cascaded control hierarchy**, where outer position

loops feed intermediate velocity setpoints down to high-rate current loops. Alternatively, a **tracking Model Predictive Controller (MPC)** solves an online optimization problem over a receding horizon to generate optimal corrective control actions while respecting actuator bounds. From the perspective of the safety enforcer, the specific internal architecture of the tracking controller is abstracted away; the safety filter requires only that the chosen implementation certifies and guarantees a concrete tracking bound ϵ_{track} under specified operational conditions.

A tracking controller may establish convergence to the reference path under its qualified model and operating conditions. That result supplies no collision-avoidance guarantee. A well-tuned tracker will follow an erroneous trajectory directly into a warehouse support column with high precision. Control Lyapunov functions[2] mathematically formalize this asymptotic convergence toward reference targets; section B.3.2 details their theoretical foundation and how optimization pipelines unify stabilizing Lyapunov objectives with the safety barrier certificates this chapter develops in section 12.5.

This division of responsibilities cleanly separates trajectory tracking from safety enforcement. The low-level tracking controller focuses on driving the physical plant toward reference states, while the safety enforcer determines whether those references can be executed without violating invariant boundaries. The tracking error bound ϵ_{track} bridges these subsystems by quantifying the spatial divergence between admitted commands and physical execution. Because environmental disturbances, unmodeled payload shifts, or thermal current derating can rapidly widen this error tube, safety architectures must continuously verify physical performance against contracted assumptions. The downstream stopping envelope accounts for this tracking tube alongside sensor latency, braking distance, and localization drift, ensuring that the physical robot remains safely buffered from surrounding hazards.

[2] **Control Lyapunov Function**: A Lyapunov function certifies that an unforced system settles; a control Lyapunov function asks only that, at each state, some admissible input make it decrease. Artstein (1983) tied the existence of such a function to stabilizability by feedback, and Sontag (1989) gave an explicit formula that builds the stabilizing input from it. The control barrier function, defined later in this chapter, is its mirror image, keeping the state out of a region instead of pulling it toward a point, and when the two conflict the combined optimization relaxes convergence so that the barrier holds.

12.3 Stopping Envelopes

If the warehouse mobile manipulator approaches a rack end at its 1.5 m/s mechanical drive limit when a coworker steps into the aisle, the delay before braking begins and the jerk-limited deceleration curve carry the base past the 1.10 m clearance even before tracking error is counted. That rack-end crossing is the scene in which nearly every chapter since chapter 2 has priced one stopping term: the body supplied braking and its fixed allowances, the Nervous System the lease path, perception the observation age, and planning the jerk-limited stop suffix that exceeded the rack-end clearance at the drive limit (section 11.5.4). The tracking bound of section 12.2 is the last spatial term. Every one of these terms serves the test that the first law, irreversibility (principle 1), sets, $d_{\text{stop}} \leq D_{\text{clear}}$. The enforcer applies that test not as one calculation but as a set of states.

Definition 12.1: Stopping envelope

Stopping envelope is the set of plant states, parameterized by clearance and velocity along each degree of freedom, from which the resident emergency braking response, initiated after accounting for all worst-case delays, halts motion before reaching an obstacle boundary while preserving all localization, protective, and tracking margins.

1. **Significance**: The envelope translates the abstract irreversibility requirement $d_{\text{stop}} \leq D_{\text{clear}}$ into an actionable runtime velocity ceiling. When approaching a rack end, the warehouse mobile manipulator's envelope restricts forward velocity, with the stated dispatch allocation, to at most 1.35 m/s, substantially below its mechanical drive limit of 1.5 m/s.
2. **Distinction**: Unlike a static geometric keep-out zone that depends solely on position, the stopping envelope is a dynamic construct governed by speed, system latencies, and braking authority. A given physical clearance safely admits a slow approach while strictly forbidding a high-speed transit.
3. **Common pitfall**: Treating the stopping envelope as a static, precomputed boundary. In practice, carried payload mass, surface traction coefficients, and motor thermal limits continuously reshape this boundary; calculating safety margins from ideal datasheet

> decelerations severely overestimates stopping authority when the machine is heavily loaded.

The stopping envelope is the machine's recoverable set, the states from which the resident stop still succeeds. Section 11.5.3 fixes the same boundary in time, as the commitment deadline past which momentum can carry the machine into an inevitable collision state. Velocity enters the recoverable set twice. Delay travel grows linearly with speed and braking travel quadratically, so a clearance that admits a slow approach forbids a fast one at the same position, and the enforcer must keep the machine inside the envelope on every tick, before a fault or an unmodeled disturbance can carry it out. The running calculation uses representative engineering limits; deployment requires each term to be qualified on the actual hardware.

The pre-brake delay τ_{delay} of section 2.3.1 already contains the enforcer's permission cycle, budgeted alongside the lease communication path in section 4.4. One interval stays out of the spatial sum: the dispatch-to-admission delay Δ_{da}, which table 4.1 places between proposal dispatch and admission. The budget charges it instead against the timing allowance that the remaining clearance leaves (section 12.3.1). What the spatial sum still lacks is the tracking error between the commanded reference and the actual plant.

The enforcer accounts for that error with the tracking bound ϵ_{track}, offsetting the baseline stopping distance of equation 2.2. It can admit a candidate reference trajectory only if the plant, which may lag that reference by up to ϵ_{track}, can still brake to a standstill within the available free distance D_{clear}. Combining physical clearance, latency, and velocity yields the stopping subtotal before dispatch-to-admission travel is charged:

$$v\,\tau_{\text{delay}} + d_{\text{suffix}}(v) + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} \leq D_{\text{clear}} \tag{12.1}$$

Here, $d_{\text{suffix}}(v)$ represents the physical distance traversed during the braking maneuver from an initial speed v. For idealized constant-deceleration braking, this term simplifies to $v^2/(2a_{\text{brake}})$. Under realistic jerk-limited trajectory profiles, $d_{\text{suffix}}(v)$ is governed by equation 11.17, which incorporates the additional displacement accrued during finite jerk transitions. The term δ_{loc} denotes the localization uncertainty bound, while δ_{margin} provides an explicit protective safety cushion. The left-hand side sums the baseline stopping distance $d_{\text{stop}}(v)$ from equation 2.2 with the tracking inset ϵ_{track}.

↪ The full stopping-distance model, tracking inset included, is developed in section C.13.

For complete static-obstacle admission, the delay term uses $\tau_{\text{delay}} + \Delta_{\text{da}}$ in place of τ_{delay}; the resident stop and fixed allowances are unchanged. This inequality defines a conditional stopping subtotal in state space. Within this set, the accounted reaction latency and braking profile fit within the measured obstacle clearance; complete admission must also charge dispatch-to-admission travel. Operating outside this boundary violates the stopping reservation under the qualified model assumptions, though conservative engineering margins do not imply that an instantaneous collision is certain under all conceivable emergency maneuvers. Per-tick enforcement must maintain this stopping reservation over the entire control epoch until the next validated command can take effect, rather than merely verifying instantaneous feasibility at the sample instant.

Whenever a proposed trajectory requires a stopping distance in equation 12.1 that exceeds the measured clearance along the current heading, the enforcer rejects the command. Because the robot moves directionally, the stopping envelope is anisotropic. Heading dictates which obstacles threaten forward travel and consume braking clearance, while localization uncertainty and the robot's swept geometric volume demand clearance margins around the entire perimeter.

12.3.1 Sizing the composite stopping budget

Omitting any single interval during which the base keeps moving before or during a stop, or evaluating those intervals under mismatched operating conditions, admits transit speeds that overrun the physical boundary when all delays coincide. The composite stopping budget therefore reserves clearance for every interval in the arrest sequence: sensor transduction lag, compute scheduling delays, actuator engagement, the deceleration profile, and tracking divergence, each evaluated under the same worst-case operating conditions.

For the warehouse mobile manipulator, the tracking bound of 40 mm is the final spatial term. At the 1.5 m/s drive limit, the matched stop suffix had already pushed the cumulative requirement to 1,123.84 mm, past the 1.10 m rack-end clearance; that is the deficit section 11.5.4 left open. The tracking inset carries the total to 1,163.84 mm, an overshoot of 63.84 mm beyond the physical boundary.

The C^2 stop suffix replaces idealized constant deceleration ($v^2/2a_{\text{brake}}$) with the jerk-limited S-curve of section 11.5.2, governed by the peak jerk limit $j_{\max} = 7.11$ m/s³. Planning stipulated that bound from the reference C^2 polynomial rather than measuring it; the S-curve is the stop the MCU actually runs, and the polynomial supplies only its jerk bound. Because both operating velocities reach the peak deceleration threshold, equation 11.17 models this maneuver by adding the displacement of half a jerk ramp:

$$d_{\text{suffix}}(v) = \frac{v^2}{2a_{\text{brake}}} + v\,\frac{a_{\text{brake}}}{2j_{\max}}.$$

Because this smoothly shaped stop consumes more clearance than discontinuous full-torque braking, the admissible transit speed must be read from the finished budget rather than from the constant-deceleration estimate.

Velocity remains the single operational variable the control architecture can dynamically modulate. Above the characteristic velocity threshold $a_{\text{brake}}^2/j_{\max} = 0.56$ m/s, the finite jerk ramps function equivalently to an added delay of $a_{\text{brake}}/(2j_{\max})$ prior to braking at constant deceleration a_{brake}. Under these conditions, the pre-dispatch **speed ceiling** keeps the form of equation 2.3, the positive root of the stopping inequality at equality. The jerk ramps lengthen the pre-brake delay to an effective delay, and the tracking bound comes off the available braking distance:

$$v_{\max} = -a_{\text{brake}}T_{\text{eff}} + \sqrt{(a_{\text{brake}}T_{\text{eff}})^2 + 2a_{\text{brake}}D_{\text{avail}}}, \qquad T_{\text{eff}} = \tau_{\text{delay}} + \frac{a_{\text{brake}}}{2j_{\max}}, \qquad D_{\text{avail}} = D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - \epsilon_{\text{track}} \tag{12.2}$$

Below $a_{\text{brake}}^2/j_{\max}$, the deceleration trajectory never reaches the peak limit a_{brake}, rendering stopping distance non-quadratic in velocity; in this low-speed regime, MLSysIM evaluates equation 12.1 via numerical root-finding. Notebook 12.1 computes this ceiling for the mobile manipulator approaching the aisle rack end.

Napkin Math 12.1: The ceiling the budget sets

Problem: The warehouse mobile manipulator approaches the rack end with rows 1 to 7 of table 12.1 accounted, before dispatch-to-admission travel. What is the highest speed at which the jerk-limited stop still fits the clear distance, and how much clearance does a candidate aisle speed of 1.3 m/s leave?

Variables:

- $D_{\text{clear}} = 1.10$ m (Illustrative; clear distance at the rack end).
- $\delta_{\text{loc}} + \delta_{\text{margin}} = 50$ mm $+ 100$ mm (Illustrative localization bound; chosen protective clearance), and $\epsilon_{\text{track}} = 40$ mm (Illustrative; the qualification target of section 12.2).
- $a_{\text{brake}} = 2$ m/s² (Illustrative; credible loaded deceleration on a dry floor) and $j_{\max} = 7.11$ m/s³ (Derived; the peak jerk of the reference C^2 polynomial).
- $\tau_{\text{delay}} = 133.6$ ms (Derived; observation age plus lease path plus brake onset, excluding Δ_{da}).
- Drive limit 1.5 m/s (Illustrative).

Math: The fixed allowances leave $D_{\text{avail}} = 1{,}100$ mm $- 190$ mm $= 910$ mm. The jerk ramps lengthen the delay to $T_{\text{eff}} = 133.6$ ms $+ 140.6$ ms $= 274.2$ ms, so $a_{\text{brake}}T_{\text{eff}} = 0.5485$ m/s. Under the square root of equation 12.2, $(a_{\text{brake}}T_{\text{eff}})^2 = 0.3008$ and $2a_{\text{brake}}D_{\text{avail}} = 3.64$ (both in m²/s²), whose sum has root 1.9851 m/s; subtracting 0.5485 m/s gives the ceiling. At 1.3 m/s the terms of table 12.1 sum to 969.0 mm, and the remainder divided by the speed is the timing allowance.

Result: The pre-dispatch ceiling is $v_{\text{max}} = 1.44$ m/s, below the 1.5 m/s drive limit. The candidate aisle speed leaves 131.0 mm of spare, equivalent to 100.8 ms of unallocated delay.

Systems insight: The irreversibility requirement $d_{\text{stop}} \leq D_{\text{clear}}$, rather than motor electrical capability, dictates the operating speed. Running at the nominal drive limit violates the verified spatial budget, and the speed ceiling shifts strictly through the parameters governing the quadratic root and effective latency T_{eff}: available clearance, qualified braking deceleration, fixed margins, and end-to-end transport lag. Any latency introduced by downstream processing directly consumes the 100.8 ms timing allowance, which cannot be arbitrarily replenished.

The candidate transit speed, 1.3 m/s, sits below that ceiling, and its 131.0 mm margin is the only clearance left to absorb unmodeled pipeline delays or to serve as a reserve. That margin is narrow relative to the failure modes it must absorb. If chunk lease enforcement were entrusted to a non-real-time host operating system, an upstream planning stall lasting 200 ms instead of the nominal 60 ms lease would inject 182 mm of unbraked displacement, causing the base to overrun the rack clearance by 50.99 mm. This hazard shows why lease termination must be autonomously enforced by dedicated microcontroller hardware directly gating motor power (section 12.6).

The observation-age term ends at dispatch, and the chunk lease begins at admission, so the interval between them, Δ_{da}, belongs to neither. The complete operational delay therefore expands to $T_{\text{true}} = \tau_{\text{delay}} + \Delta_{\text{da}}$, requiring an additional spatial reservation of $v\Delta_{\text{da}}$. This phase encompasses any remaining neural inference, safety envelope projection, and inter-process communication; pipelined or overlapping stages must be budgeted carefully to avoid double-counting. A newly arrived sensor observation updates this budget only when the hardware pipeline delivers and validates it prior to actuation. Laid out on one time axis (figure 12.2), the observation age ends at dispatch, the dispatch-to-admission interval follows, and the lease, one tick, one bus cycle, and drive onset bring the base to brake onset. A travel row beneath converts each interval into millimeters at the candidate aisle speed.

At this candidate speed, dividing the 131.0 mm spare distance by the speed yields 100.8 ms of unallocated timing allowance. Added to the 51.6 ms observation age, that allowance places the latest admission after capture at 152.4 ms. Added to the 133.6 ms pre-brake delay, it places the latest brake onset after capture at 234.4 ms. The dispatch-to-admission interval spends part of that allowance and leaves whatever remains as the reserve; because operating right at these boundaries spends all available clearance, the figures are qualification constraints rather than performance claims. The budget carries forward the allocation that the placement chapter makes for this interval (section 13.6.2), $\Delta_{\text{da}} = 63.3$ ms for the partitioned compute pipeline. That allocation consumes 82.3 mm of the spatial margin, leaves a cushion of 37.5 ms (48.7 mm of distance), and puts total latency from capture to brake onset at 196.9 ms. Because that allocation is an engineering budget rather than a measurement, an instrumented campaign with hardware timestamps must confirm it before the vehicle can be certified at this speed. Read row by row (table 12.1, figure 12.1), the terms through observation age fit inside the clear distance at the 1.5 m/s drive limit, whereas the jerk-limited stop (row 6) carries the total past the line and the tracking inset (row 7) widens the breach. At the 1.3 m/s aisle speed, those same terms stop 131.0 mm short of the line and still leave a 48.7 mm reserve once dispatch-to-admission travel is charged.

Table 12.1: The warehouse mobile manipulator's stopping budget: Each numbered row details the distance contributed by a physical subsystem or software stage, modeled for a loaded chassis on a dry industrial floor at the 1.5 m/s drive limit vs. the candidate 1.3 m/s aisle speed; the running total and clearance remainder are evaluated at the drive limit against the 1.10 m obstacle clearance. Row 6 accounts for the additional displacement imposed by jerk-limited C^2 deceleration profiles relative to ideal constant deceleration. Time-only rows contribute latency rather than physical distance. The totals cover rows 1 to 7; row 8, the dispatch-to-admission travel $v\Delta_{\text{da}}$, is charged against the spare at the aisle speed only, and the last line is the reserve it leaves. The second column names the section that prices each term, so every partial total stated in earlier chapters traces to a row of this table.

Row	Added in	Term	At drive limit (mm)	Running total (mm)	Left of D_{clear} (mm)	At aisle speed (mm)
1	section 2.3	Braking, $v^2/2a_{\text{brake}}$	562.5	562.5	537.5	422.5

Row	Added in	Term	At drive limit (mm)	Running total (mm)	Left of D_{clear} (mm)	At aisle speed (mm)
2	section 2.3	Overhead, $\delta_{\text{loc}} + \delta_{\text{margin}}$	150	712.5	387.5	150
3	section 2.3	Brake onset, vT_{act}	30	742.5	357.5	26
—	section 3.5.2	Time only: the chunk policy can renew a lease; the intent model's 336 ms inference cannot	—	—	—	—
4	section 4.4	Lease path, $v(T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}})$ over 62 ms	93	835.5	264.5	80.6
5	section 8.4.1	Observation age, $v\,t_{\text{age}}$ over 51.6 ms	77.4	912.9	187.1	67.08
—	section 9.7	Time only: abreast of an occluded rack end the erosion law of equation 9.13 leaves no free depth, so the free-space claim lasts 0 ms and memory cannot certify it	—	—	—	—
6	section 11.5.4	Jerk-limited C^2 stop, the jerk-ramp travel $v\,a_{\text{brake}}/2j_{\text{max}}$ added to $v^2/2a_{\text{brake}}$	210.94	1,123.84	-23.84	182.81
7	section 12.3	Tracking inset, ϵ_{track}	40	1,163.84	-63.84	40
		Total	1,163.84			968.99
		Left of D_{clear}	-63.84			131.01
8	section 13.6.2	Dispatch-to-admission travel, $v\,\Delta_{\text{da}}$ over 63.3 ms, charged against the spare at the aisle speed only	—	—	—	82.3
		Reserve after row 8	—			48.7

This baseline stopping model evaluates a stationary obstacle situated at distance D_{clear}, such as a dropped warehouse tote or a pedestrian who has stepped out from an aisle rack and stopped. Under this assumption, the hazard remains fixed from the instant braking begins. If, instead, a pedestrian continues walking toward the vehicle throughout the reaction and braking phases, the obstacle actively closes the safety gap from the opposing direction. Before dispatch allocation, an ongoing pedestrian approach restricts the mobile base to a subtotal ceiling of just 0.26 m/s at this rack end.

This drastic velocity reduction occurs because an active hazard penalizes the stopping budget by the *duration* of the stop rather than merely its displacement. Assuming an ISO 13855 approach velocity $v_h = 1.6$ m/s across the entire reaction and stopping interval $\tau_{\text{delay}} + T_{\text{stop}}(v)$, the spatial budget acquires an additional closure penalty $v_h\,[\tau_{\text{delay}} + T_{\text{stop}}(v)]$ absent from static calculations. When operating below the velocity threshold $a_{\text{brake}}^2/j_{\text{max}} = 0.56$ m/s, the jerk-limited braking profile cannot achieve peak deceleration a_{brake}; it ramps deceleration up and immediately down in a triangular profile lasting $2\sqrt{v/j_{\text{max}}}$. This duration diminishes far more slowly than the linear scaling v/a_{brake} as velocity decreases. Initiating a jerk-limited stop from 0.26 m/s requires 382 ms, compared to only 130 ms for idealized step deceleration. While the mobile base travels only 50 mm during braking, the pedestrian advances 612 mm toward the robot over the same window. Achieving peak deceleration from such low speeds would demand a jerk of 41 m/s^3, several times the stipulated bound the load was assumed to tolerate, whereas unconstrained step deceleration would permit 0.64 m/s. That contrast arises because the jerk bound costs differently in the two regimes. Above $a_{\text{brake}}^2/j_{\text{max}}$ it adds a fixed delay of $a_{\text{brake}}/(2j_{\text{max}})$ whose travel grows with speed and forms the row-6 term pushing the drive limit past the clear distance. Below that threshold it stretches the stop's duration. Against a person who keeps walking, duration is what the budget pays for, so reducing operating speed yields diminishing safety returns.

Throttling nominal aisle velocity to the walking-person ceiling would severely degrade warehouse transport throughput for an event that facility layout and procedural crossing rules can prevent. The baseline operational design therefore mandates enforced physical or optical crossing restrictions at aisle ends, while the residual-claims register in section 18.3.3 tracks the governing premise that pedestrian incursions into the stopping zone are procedurally prevented. Operating at the candidate aisle velocity remains contingent on empirical validation of these access restrictions. The derivation

↩ The pedestrian intrusion distance, evaluated for a constant-deceleration stop at the drive limit, is established in section 9.7.

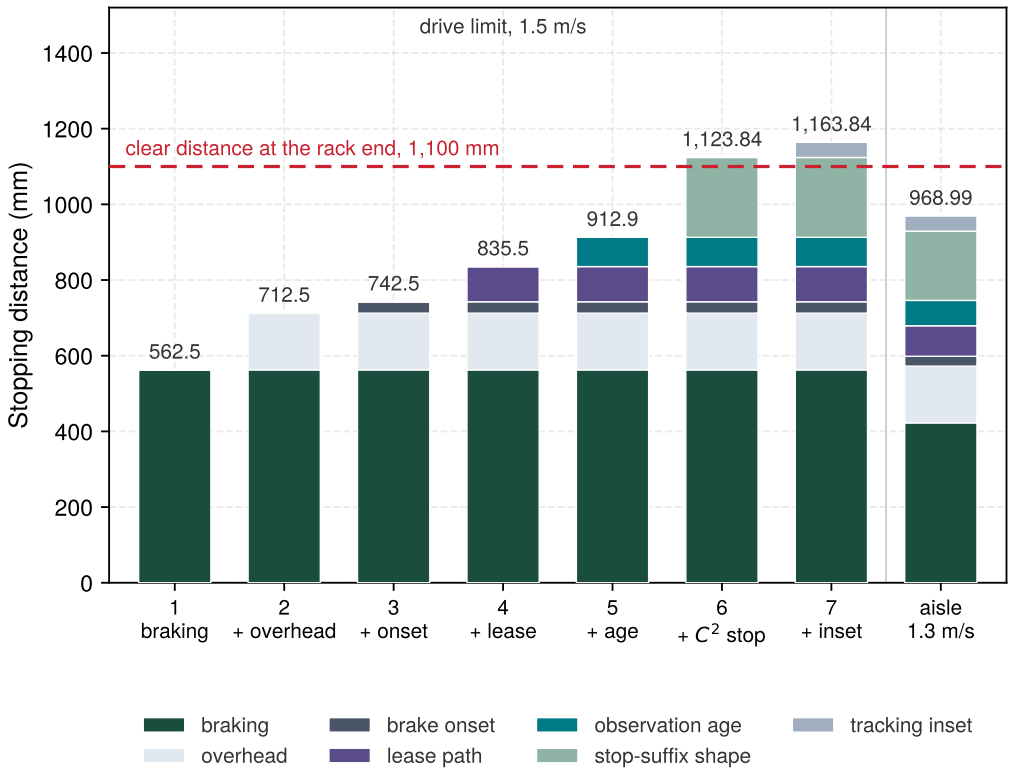


Figure 12.1: The stopping budget, row by row: Each bar illustrates the cumulative distance required as each successive physical and timing term is incorporated into the budget, evaluated at the vehicle drive limit under loaded conditions on dry concrete. While kinematic braking, fixed margins, actuator onset, lease execution, and perception age collectively remain within the available clearance, the jerk-limited deceleration profile causes the total to exceed the boundary, and the closed-loop tracking inset exacerbates this overrun. The final bar displays the accounted subtotal at the candidate aisle speed, where the remaining clearance must absorb the dispatch-to-admission interval before providing a net safety reserve.

bounds stopping against a static obstruction; protecting against an unconstrained approaching pedestrian requires certified physical interlocks or external safety scanners.

Three operating regimes therefore set three ceilings (figure 12.3). On a dry floor, fixed allowances, linear delay travel $v\tau_{\text{delay}}$, quadratic braking travel $v^2/(2a_{\text{brake}})$, and jerk-ramp travel sum to a total that meets the 1.10 m clear distance at 1.44 m/s. Even an optimistic all-weight traction screen requires a floor friction coefficient of at least 0.204, because $a = \min(a_{\text{brake}}, \mu g)$ assumes all weight contributes braking traction. The running machine also requires qualified driven-wheel load share, torque, and stability (section 2.3). On a sealed floor inspected to $\mu = 0.12$, the ceiling falls to 1.23 m/s. A pedestrian walking toward the base at 1.6 m/s charges the stop by its duration and contracts the ceiling to 0.26 m/s. Because these three roots are subtotal ceilings before dispatch allocation, charging the 63.3 ms allocation lowers each of them. The dry-floor root falls to 1.35 m/s by adding the base's travel through dispatch. The optimistic inspected-floor root falls to 1.17 m/s in the same way. The walking-person root falls to 0.19 m/s by adding both the base's travel and the person's closure across that interval. Because operating at any maximum root spends all clearance and leaves zero reserve, the remaining reserve must be recomputed for the selected speed, floor, and hazard motion. The 37.5 ms timing reserve belongs specifically to the dry, static-obstacle 1.3 m/s candidate, and an authority transition cannot claim its original 100.8 ms margin a second time.

An independent protective safety scanner operates under its own hardware-to-actuation timing budget. When an obstacle breaches a safety-rated zone, the scanner's internal optical sampling, fail-safe output switching, safety controller logic, and drive disconnect delays entirely bypass the perception dispatch and software lease infrastructure. However, the physical response must

↪ The authority transitions that must fit inside this remaining reserve are developed in section 14.3.

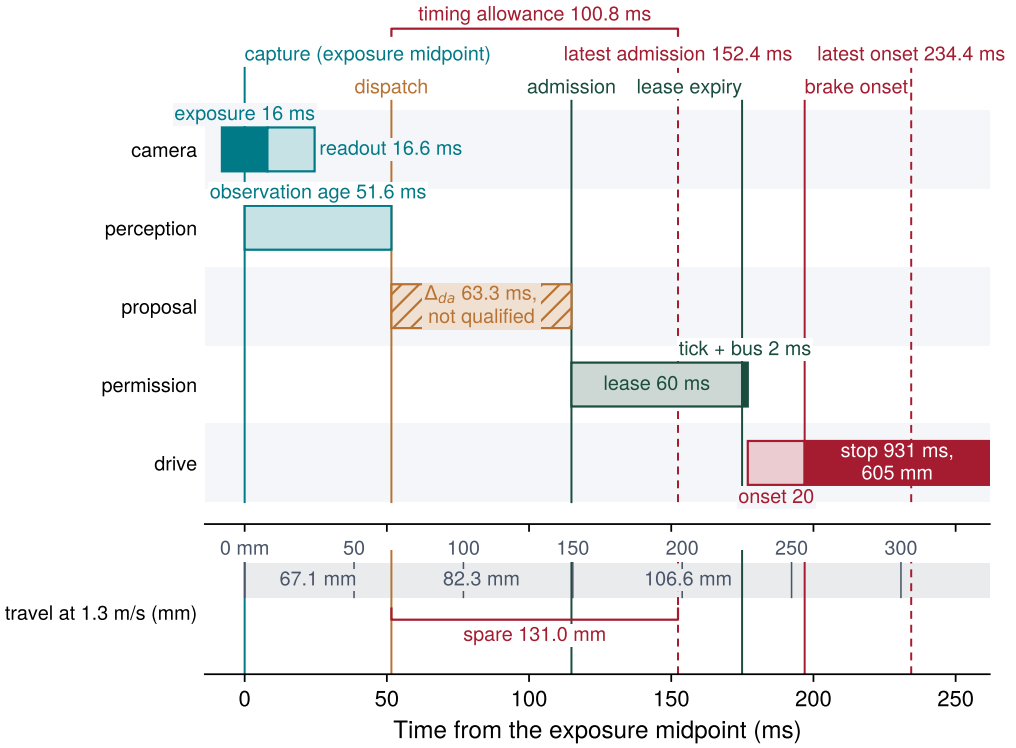


Figure 12.2: From capture to brake onset, in time and in travel: The warehouse mobile manipulator's permission path at the 1.3 m/s aisle speed, drawn to scale from the exposure midpoint of the frame that first shows the hazard. The observation age of 51.6 ms ends at dispatch; the hatched dispatch-to-admission interval is the 63.3 ms allocation that placement makes, a budget that loaded measurement must still confirm; the lease, one tick, one bus cycle, and drive onset follow admission. The row beneath reads the same axis as travel, so the 100.8 ms timing allowance and the 131.0 mm spare are one interval. Admission later than 152.4 ms, or onset later than 234.4 ms, spends clearance the budget does not have. The jerk-limited stop that follows onset runs past the right edge.

still reserve the same underlying deceleration distance, mechanical tracking error, localization tolerance, and protective safety cushion; only computational delays bypassed by the safety bus are eliminated from the sum. Industrial mobile bases deploy such protective fields under ISO 3691-4 (International Organization for Standardization 2023b) for driverless industrial trucks and ISO 13849-1 (International Organization for Standardization 2023a) for safety-related control circuits, using a safety-rated planar scanner of the kind in figure 12.4, which evaluates programmable warning and protective fields. The physical dimensions of this protective field must be calculated from the dedicated scanner path's verified hardware timing budget, rather than inheriting the software lease and vision latencies of the neural planner. The safety-rated optical scanner is a component of a separately qualified protective function, distinct from the primary perception LiDAR that generates non-safety-rated point clouds for neural proposal generation. Its component rating does not certify the complete sensor-to-stop path. In the running architecture, that path must establish the detection coverage, response time, and physical stopping performance required for protection at rack intersections. Section 12.9.3 maps the software stopping budget onto this certified hardware baseline and shows that industrial vehicle testing standards validate pedestrian detection against the same static-obstacle assumption derived here.

12.3.2 Phase portraits and speed ceilings

Checking velocity against a fixed scalar limit fails as soon as obstacle distance shrinks: a speed that is recoverable two meters from the rack end becomes an inevitable collision eighty centimeters closer. Drawn in the distance–velocity (D_{clear}, v) phase plane (figure 12.5), the plotted pre-dispatch

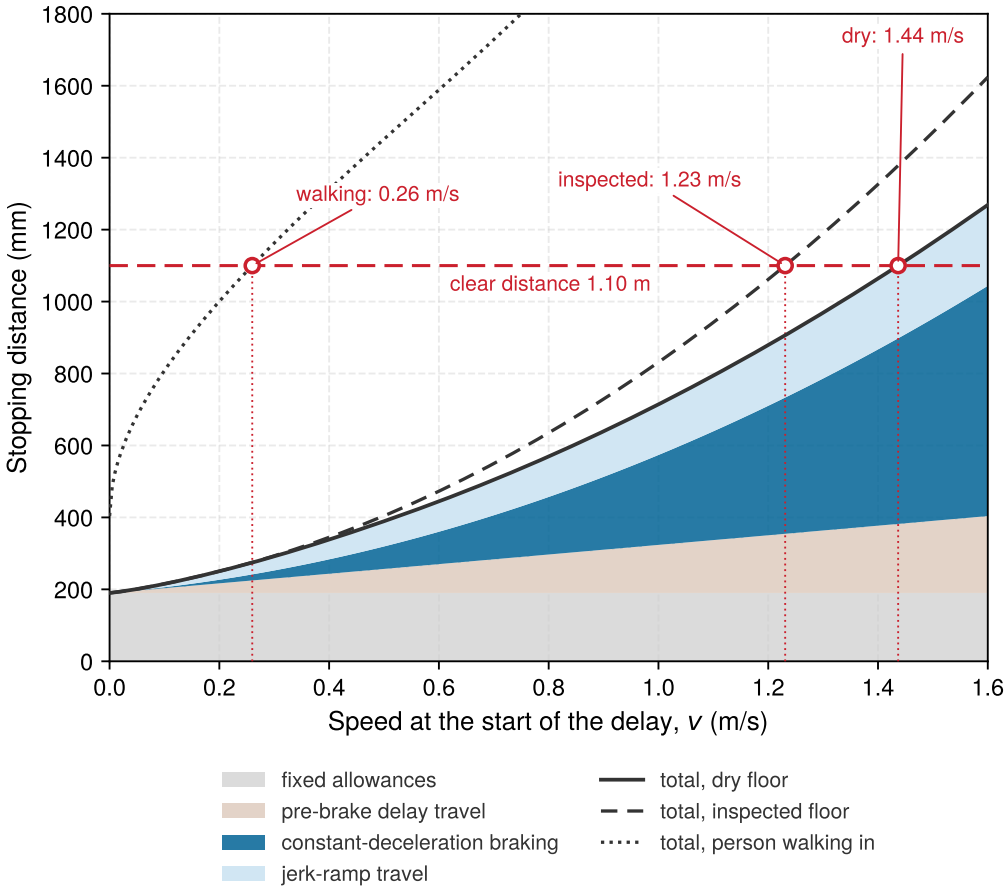


Figure 12.3: Stopping subtotals against speed, and the three pre-dispatch ceilings: The stacked bands build the dry-floor stop from the fixed allowances, the travel during the 133.6 ms pre-brake delay, constant-deceleration braking, and the extra travel of the jerk ramps. Delay travel grows linearly with speed and braking travel quadratically, so the total meets the 1.10 m clear distance at 1.44 m/s. The dashed curve applies the optimistic all-weight traction screen for a floor inspected to $\mu = 0.12$, moving the ceiling to 1.23 m/s. The dotted curve adds the closure of a person walking toward the base through the delay and the stop, which charges the stop by its duration and gives a pre-dispatch ceiling of 0.26 m/s. All three curves omit dispatch-to-admission travel; complete scenario ceilings are lower.

envelope separates states whose accounted subtotal fits from states whose subtotal already exceeds clearance. Complete admission contracts that boundary further by charging dispatch travel. Braking alone traces the outer parabola. The inset boundary pulls the viable set, the recoverable set drawn in this plane, inward by the pre-brake travel $v\tau_{\text{delay}}$, the jerk ramps of the matched stop, and the fixed localization, protective, and tracking allowances. At the rack-end clear distance the height of that plotted boundary is the subtotal speed ceiling, with the candidate aisle speed beneath it. The complete dry-floor ceiling is 1.35 m/s; the governor must use the fully charged boundary. Under the verified delay and deceleration bounds, the envelope is therefore a velocity ceiling that varies with clearance, and a state outside it violates the stopping reservation even if an emergency maneuver might avoid a collision in an edge case.

Evaluated on every enforcer tick, the envelope becomes a speed governor that runs in two stages, and their order matters:

1. **Current-state feasibility check**: The enforcer first verifies whether the robot's current operational state (D_{clear}, v) remains strictly inside the envelope. It tests whether a full braking response from the current velocity v, with all transport lags, jerk transitions, and margin allowances, fits within the currently measured clearance D_{clear}. If this condition fails, no newly

Figure 12.4: Planar laser scanner: A rotating mirror sweeps a time-of-flight beam across a horizontal fan, so the device reports range over one plane around the base. A safety-rated scanner of this form adds an early warning field and a protective field. The warning field requests earlier deceleration; the protective field's safety output feeds a controller that selects a risk-assessed stop. Protective-field length must include scanner response, controller and drive delay, braking travel, and uncertainty; a moving base does not become safe merely because motor torque is removed. (Credit: Björn Heller, Wikimedia Commons, CC BY-SA 3.0 DE.)

proposed command setpoint can restore the lost stopping reservation, because reducing the target acceleration cannot retroactively erase the kinetic energy or displacement accrued from the existing physical velocity. A violation at this step immediately selects `STOP`, the resident controlled stop that section 12.9 defines among the enforcer's fallback responses.

2. **Candidate command projection**: If and only if the current state passes validation, the enforcer samples the speed ceiling $v_{\max}(D_{\text{clear}})$ from the fully charged counterpart of the pre-dispatch boundary shown in figure 12.5. It then clamps the candidate velocity proposal v_{cand} to the admissible ceiling, admitting $v^* = \min(v_{\text{cand}}, \max(0, v_{\max}))$. Rather than discarding an overspeed proposal entirely, this governor projects the command onto the boundary of the safe set.

If obstacle clearance shrinks, computational jitter expands end-to-end latency, or floor friction degrades, the envelope boundary shifts inward, throttling admissible velocity on the subsequent control cycle when qualified sensing or configuration evidence exposes the change. A reference clamp alone does not establish that actual velocity remains inside the envelope between checks. This one-dimensional velocity governor is the scalar prototype for multi-dimensional safety filters; in section 12.5, control barrier functions extend this exact principle to enforce forward invariance over full state spaces.

The stopping envelope is the kinematic roofline of section 1.4.3 carried through the finished budget. Physical clearance (D_{clear}) sets a ceiling on transit speed for the qualified braking, margin, and timing bounds. A faster inference frame rate raises that ceiling only if it reduces a delay charged to the stopping budget. Model size and inference optimizations affect the ceiling through their share of the sensor-to-actuator delay τ_{delay}. On the warehouse mobile manipulator, every 1 ms of added pipeline delay consumes 1.3 mm of spatial clearance at nominal aisle velocity, and an unbudgeted 10 ms latency increase depresses the allowable speed ceiling by approximately 14 mm/s. Consequently, deploying an oversized vision-language model that increases latency directly lowers maximum transit speed, whereas an inference improvement that does not change the scheduled delivery tick may leave the charged delay unchanged. The speed ceiling shifts materially only when altering the physical parameters of the plant: verified deceleration authority a_{brake}, end-to-end transport lag τ_{delay}, tracking error bound ϵ_{track}, and localization uncertainty δ_{loc}.

↩ The clearance-parameterized velocity ceiling is established as a kinematic constraint in section 2.3.2.

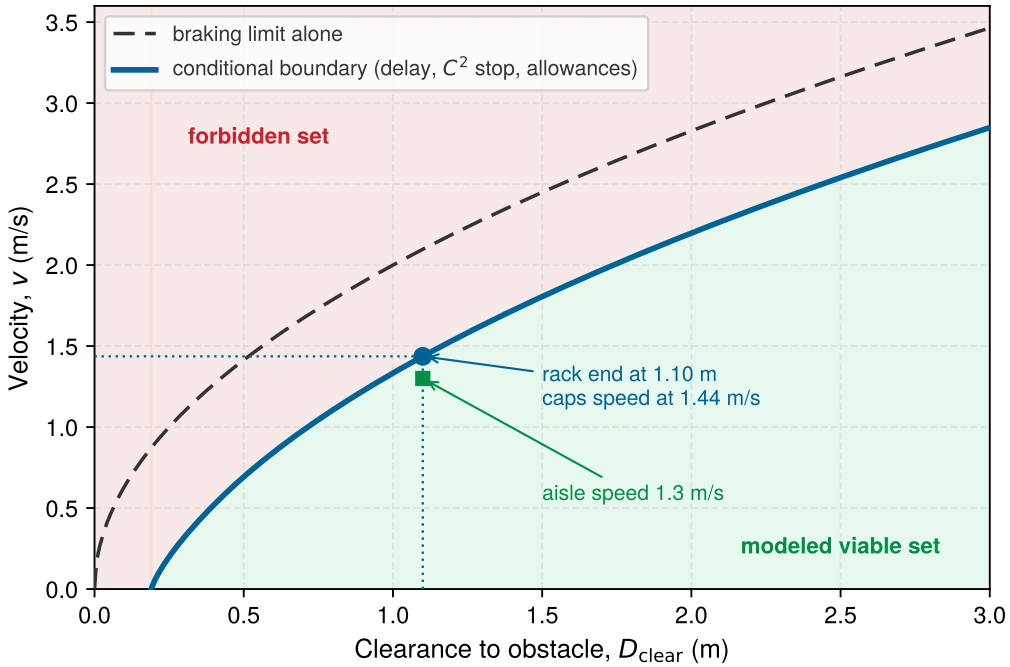


Figure 12.5: Conditional stopping subtotal before dispatch: For the warehouse mobile manipulator's base, a boundary separates states with modeled braking clearance from states that have already exhausted it. The inset includes the pre-brake delay, the matched stop suffix's jerk ramps, and the localization, protective, and tracking allowances. Clearance below the fixed allowances admits no approach speed, including zero. The curves use the accounted delay and stated braking envelope; unaccounted dispatch-to-admission time lowers the ceiling further.

None of these governing parameters remain stationary during real-world operation. Manipulating a heavy payload in the end-effector increases arm inertia, diminishing joint deceleration limits within peak motor torque bounds. Industrial floor contaminants such as oil or water severely degrade tire traction beneath the assumed braking limit a_{brake}, while aggressive operational cycles heat motor windings and drive inverters into thermal derating. A safety enforcer that treats a_{brake} as an immutable datasheet constant will dangerously underestimate stopping distance under heavy workloads. Runtime safety monitors must update braking bounds as observable, qualified payload and thermal states evolve. Instantaneous wheel-floor friction is not measured on this machine; an unobserved floor change invalidates the inspection premise and requires an operating restriction rather than automatic adaptation. In either case, relying on unvalidated, optimistic estimates invalidates the formal safety contract.

Dynamic adaptation remains effective only as long as state estimators remain uncorrupted and physical actuators retain control authority. If severe surface contamination reduces friction toward zero, or if sustained high-torque demands push drive inverters beyond their thermal trip thresholds, physical braking authority collapses. When required stopping torque exceeds what the physical hardware can exert against the environment, software can still detect the boundary violation, but it can no longer prevent an impact.

12.4 Authority Exhaustion and Emergency Fallback

A loaded arm descending toward the packing station must counter gravity before any torque remains to slow its motion. **Actuator saturation** occurs when requested effort reaches the electrical, mechanical, or thermal limits of the physical body, which can cause downstream integral controllers to accumulate windup.[3] The actuator cannot deliver the corrective control action $\mathbf{u}^*$ at its commanded magnitude in any of several cases. Motor phase currents may hit inverter current-chopping limits ($I_{\max}$). The DC bus may sag under inductive switching transients ($L_{\text{bus}}\, dI/dt$). Back-EMF at high

[3] **Integrator Windup**: A proportional-integral controller's integral term continues summing tracking error while an actuator sits at its saturation limit, as standard feedback loops do not inherently know that command outputs are clipped. When tracking error reverses, this accumulated state must unwind before the actuator command can leave the saturation rail. That unwinding delay directly consumes safety clearance. While modern anti-windup schemes clamp or back-calculate integrator state during saturation (Åström and Murray 2021), the safety enforcer must still budget whatever residual unwinding delay remains in the low-level hardware controller.

rotor speeds may collapse voltage headroom ($V_{\text{rail}} - K_e\omega_m \to 0$). Winding temperatures may approach thermal derating thresholds ($T_{\text{winding}} \to T_{\text{limit}}$). No mathematical safety boundary can be defended with control effort the physical body cannot generate.

When required braking effort exceeds physical actuator limits, no safety filter can enforce the promised stopping horizon. An emergency fallback invoked after control authority is lost can mitigate damage, but it cannot restore an invariant guarantee that physical dynamics have already broken. The runtime enforcer must therefore contract its forward reservation *before* actuator effort saturates, preserving a feasible braking margin under worst-case loading.

↩ The electrical and transmission limits that bound delivered actuator effort are established in section 2.4.

A stopping envelope based on peak torque alone overstates braking authority when gravity drives the moving load toward a hazard. Under a steady-state bias torque τ_{bias}, the effective deceleration available to arrest angular motion collapses:

$$\alpha_{\text{net}} = \frac{\tau_{\text{max}} - \tau_{\text{bias}}}{J_{\text{eff}}} \tag{12.3}$$

where J_{eff} is mechanism inertia and τ_{max} is the peak dynamic torque limit. In linear translational systems with mass m and bias force u_{bias}, equation 12.3 takes the equivalent form $a_{\text{net}} = (u_{\text{max}} - |u_{\text{bias}}|)/m$. For motion aligned with the bias, a payload consuming 75 percent of peak opposing torque leaves only the remaining quarter for active braking, quadrupling the constant-deceleration stopping distance at that speed.

Gravitational and dynamic biases are directionally asymmetric. When motion opposes gravity (such as during an upward arm swing), gravity assists motor braking, yielding $\alpha_{\text{net}} = (\tau_{\text{max}} + \tau_{\text{bias}})/J_{\text{eff}}$. Conversely, when motion aligns with the bias (such as a loaded arm descending toward a workstation), gravity directly opposes deceleration, collapsing braking headroom to $\alpha_{\text{net}} = (\tau_{\text{max}} - \tau_{\text{bias}})/J_{\text{eff}}$. Because safety certificates must guarantee forward invariance in the worst-case physical direction, the runtime enforcer must evaluate α_{net} with directional sign awareness rather than treating braking authority as an isotropic scalar. Suppose the warehouse mobile manipulator carries a mug from the takeaway conveyor down toward a coworker at the packing station. Its tool center point (TCP) moves at 0.96 m/s, within its 1 m/s free-space limit (only the final approach to the coworker's hand runs at the reduced handover speed derived in section 12.7.2). Gravitational bias during this descent quadruples the tool's stopping distance (notebook 12.2).

Napkin Math 12.2: The gravity braking tax

Problem: The loaded joint descends toward the packing station at ω_0. How far does the tool point travel before the joint stops when the whole peak torque brakes, and how far when gravity on the arm and mug holds part of it?

Variables:

- $\omega_0 = 1.2$ rad/s and $r_{\text{arm}} = 0.80$ m (Illustrative; the joint speed and effective link lever arm of the scenario, see the Reader Guide).
- $J_{\text{eff}} = 3$ kg·m² (Illustrative; joint inertia with the mug).
- $\tau_{\text{max}} = 87$ N·m (Datasheet; peak torque of the registered arm's axes 1 to 4).
- $\tau_{\text{bias}} = 65.25$ N·m, 75 percent of peak torque (Illustrative; gravity on the arm and mug in the direction of motion).

Math: With no bias, $\alpha_{\text{nom}} = \tau_{\text{max}}/J_{\text{eff}} = 29$ rad/s², the stopping angle is $\omega_0^2/(2\alpha_{\text{nom}}) = 0.0248$ rad (1.42°), and the tool point travels $r_{\text{arm}}\,\Delta\theta_{\text{joint}} = 19.86$ mm. On the descent, equation 12.3 leaves $\tau_{\text{max}} - \tau_{\text{bias}} = 21.75$ N·m for braking, so $\alpha_{\text{net}} = 7.25$ rad/s², the stopping angle is 0.0993 rad (5.69°), and the tool travels 79.45 mm.

Result: The descending stop is four times the unbiased one, an unbudgeted overrun of 59.59 mm for an enforcer that assumes the datasheet torque.

Systems insight: Stopping distance scales as $1/\alpha_{\text{net}}$, so a bias that takes 75 percent of the torque quadruples the braking term of d_{stop} at the same speed. The certificate must use the net authority in the worst direction, not the actuator's rating.

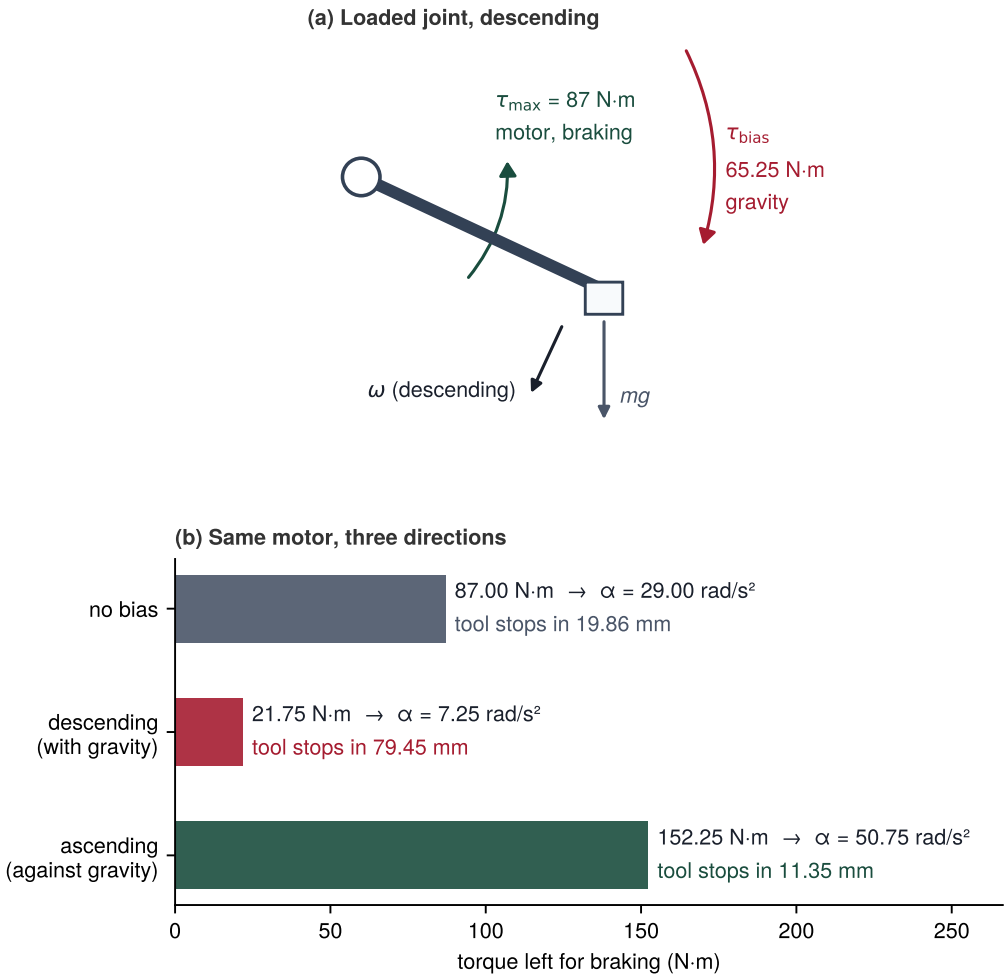


Figure 12.6: Gravity decides how much braking is left: (a) On the way down, gravity on the arm and mug loads the joint with 65.25 N·m in the direction of motion, against the motor's 87 N·m peak. (b) The same motor leaves 21.75 N·m for braking on the descent, a quarter of the unbiased case, so the tool stops in 79.45 mm instead of 19.86 mm; moving up, gravity adds to the braking torque. The certificate must use the descending case.

The asymmetry is direction, not magnitude (figure 12.6). On the descent, gravity torque acts with the motion and leaves only 21.75 N·m of the motor's peak for braking, while the same joint moving up recruits gravity and stops in less distance than the unbiased case. Whether the descending overrun reaches the workstation depends on the clearance left when braking begins, the margin the barrier certificate of section 12.5 evaluates. The coworker's hand must never fall within the arm's biased stopping distance of 79.45 mm plus its blind travel during sensing and communication delay before the handover channel engages.

An unprotected integrator downstream of the safety enforcer can introduce severe latency during actuator saturation. If clipping does not halt integration, tracking error continues to accumulate in the controller state. When the safety system subsequently requests maximum braking, the command must pass through this wound-up state rather than bypassing it. The actuator command consequently remains pinned at its forward saturation limit while the integrator unwinds. Take an illustrative case in which tracking error accumulates for 200 ms and the unwinding phase then delays net braking deceleration by $T_{\text{unwind}} = 150$ ms. The two durations are independent assumptions rather than one derived from the other; the unwind time depends on the integrator gain and on how sharply the error reverses, and only a test of the actual drive fixes it. At an aisle speed of 1.3 m/s,

this latency adds $vT_{\text{unwind}} = 195$ mm of unbraked travel. That overrun exceeds the system's entire 131.0 mm unallocated clearance buffer by 63.99 mm, even before dispatch-to-admission latency is charged. Classical techniques such as integrator clamping, back-calculation, and conditional integration prevent windup in the feedback loop (Åström and Hägglund 2006; Franklin et al. 1998). The enforcement architecture, however, must explicitly quantify any residual unwinding delay and budget the corresponding unbraked distance into the stopping horizon.

Actuator saturation is one of the few hardware limitations with an immediate, deterministic detector inside the permission path. On each 1 ms tick, the permission path compares the commanded effort u_{cmd} against the realizable actuator envelope: $u_{\text{act}} = \text{clip}(u_{\text{cmd}}, u_{\text{min}}, u_{\text{max}})$. A nonzero residual $u_{\text{cmd}} - u_{\text{act}} \neq 0$ or an active drive-stage current-limit flag directly reveals that the requested effort has been clipped. This comparison detects saturation at the command interface, however, rather than measuring delivered shaft torque or tire-ground traction. Qualified physical evidence (phase currents, DC bus voltage, thermal derating thresholds, and kinematic state telemetry) must support a conservative lower bound on available directional braking deceleration. An upper actuator-effort ceiling limits possible braking; it does not establish the braking that a stop may credit, and a clipping detector does not measure tire-floor traction.

Because degraded authority immediately lengthens stopping distance, the enforcer updates its braking envelope in real time. Before admitting the next trajectory segment, the filter recomputes the effective deceleration $a_{\text{net}} = (u_{\text{max}} - |u_{\text{bias}}|)/m$. Allowable vehicle velocity scales down as authority contracts; if the recalculated stopping distance no longer fits within the confirmed clear distance, the enforcer must command immediate deceleration while sufficient braking headroom remains.

Authority exhaustion must also be communicated upstream to the planner. If a 20 Hz trajectory generator continues commanding aggressive accelerations while the low-level 1 kHz loop reports saturation, the policy is planning trajectories for a virtual body that the physical robot cannot realize. The permission path treats saturation as an active constraint violation. It revokes the active trajectory lease and forces a replan initialized from the machine's actual, authority-constrained dynamic state.

Systems Perspective 12.2: The actuator authority invariant

A control barrier function, as formulated in section B.4, remains mathematically valid only while physical actuators retain sufficient current, voltage, and thermal headroom to deliver the computed corrective effort: $u^* \in [u_{\text{min}}, u_{\text{max}}]$. When steady-state gravity, friction, or payload inertia encroaches on this operating envelope, the runtime enforcer must dynamically contract the safe set before marginal control authority vanishes.

Checkpoint 12.1: Stopping envelope and actuator headroom

Consider how the stopping envelope, and the actuator authority beneath it, bound permissible motion:

- ☐ **Quadratic envelope scaling**: Why does the stopping budget grow faster than speed, so that the finished budget, rather than the drive limit, sets the aisle speed, and why can no planning policy bypass this kinematic floor?
- ☐ **Transport lag vs. braking estimate**: At the 1.3 m/s aisle speed, what clearance is spent during τ_{delay} ($v\tau_{\text{delay}}$) and during constant-deceleration braking ($v^2/2a_{\text{brake}}$), and what is the speed below which the delay term dominates?
- ☐ **Effective deceleration collapse**: What is the physical mechanism that causes stopping distance to quadruple when steady-state gravity consumes 75 percent of peak actuator torque?
- ☐ **Dynamic safe set contraction**: Why must the permission path actively shrink the admissible state space when an actuator approaches thermal or electrical saturation, rather than waiting for tracking error to breach a hard limit?

12.5 Safe Sets as Conditional Permission

Once gravitational bias torque is counted, the descending arm analyzed in section 12.4 requires 79.45 mm of tool travel to brake to a complete stop, and this stopping distance scales quadratically with approach speed. A clearance margin that guarantees safety at one velocity fails completely at a higher velocity. The runtime safety enforcer therefore requires a joint predicate over position and velocity that it can evaluate synchronously on every tick of the control clock.

While a stopping budget establishes a static physical margin, a control barrier function[4] transforms that spatial margin into an instantaneous constraint on the next actuator command (Ames et al. 2014; Ames et al. 2019). This formulation belongs to the broader family of **safety filters** that monitor candidate action proposals against certified fallbacks, intervening only when necessary (Hsu et al. 2024) so that the machine can execute aggressive tasks while guaranteeing sufficient braking authority to halt.

A **safe set** $\mathcal{C}$ collects all system states for which a scalar certificate $h(\mathbf{x})$ remains nonnegative. Its boundary represents zero margin, whereas its interior provides positive margin. For the manipulator arm, both position and velocity must enter this certificate because a physical clearance that safely accommodates a slow approach must reject a high-speed approach at the identical position:

$$\mathcal{C} = \{\mathbf{x} \in \mathcal{X} \mid h(\mathbf{x}) \geq 0\}, \quad \partial\mathcal{C} = \{\mathbf{x} \in \mathcal{X} \mid h(\mathbf{x}) = 0\}, \quad \text{Int}(\mathcal{C}) = \{\mathbf{x} \in \mathcal{X} \mid h(\mathbf{x}) > 0\}.$$

Ensuring that trajectories remain inside $\mathcal{C}$ is known as forward invariance, the physical counterpart to an **inductive loop invariant** in formal software verification:

$$h(\mathbf{x}_0) \geq 0 \quad \wedge \quad \dot{h}(\mathbf{x}, \mathbf{u}) \geq -\kappa(h(\mathbf{x})) \quad \implies \quad h(\mathbf{x}_t) \geq 0 \quad \forall t \geq 0. \tag{12.4}$$

This implication holds in continuous time under valid plant models and actuator feasibility. Nagumo's theorem[5] (Blanchini 1999) provides the geometric boundary interpretation: the closed-loop vector field at the boundary $\partial\mathcal{C}$ must point tangent to or back into the set. The comparison function $\kappa(\cdot)$, chosen as an extended class-$\mathcal{K}$ function, strictly increases with safety margin and vanishes at zero. This allows rapid approach motions deep inside the interior of $\mathcal{C}$ while enforcing a nondecreasing safety margin along the perimeter. In an embedded implementation, discrete-time sampling, state estimation noise, and torque delivery latency demand additional safety buffers before continuous-time invariance translates into physical reliability.

> **Definition 12.2: Control barrier function**
>
> A ***control barrier function*** is a continuously differentiable scalar function $h(\mathbf{x})$ of the plant state whose 0-superlevel set defines the safe set $\mathcal{C}$, and whose directional derivative can be directly influenced by the control input. Restricting actuation to control inputs that satisfy $\dot{h}(\mathbf{x}, \mathbf{u}) \geq -\kappa(h(\mathbf{x}))$ guarantees that any trajectory initialized within $\mathcal{C}$ remains inside $\mathcal{C}$ for all time (forward invariance, equation 12.4), provided the plant model, actuator limits, and timing assumptions remain valid.
>
> 1. **Significance**: The function translates an empirical stopping margin into an admissible half-space constraint on the next control cycle. For the descending arm of section 12.4, its zero level set sits exactly where remaining tool clearance equals the biased stopping distance of 79.45 mm; closer than this boundary, no admissible torque command can keep the system within $\mathcal{C}$.
> 2. **Distinction**: Whereas a Lyapunov function certifies asymptotic convergence toward an equilibrium state, a control barrier function certifies set invariance, proving that system trajectories never exit an admissible operating region. The barrier function provides the mathematical certificate; the least-change quadratic program filter of section 12.7 provides the runtime mechanism to enforce it.
> 3. **Common pitfall**: Formulating h strictly as raw geometric distance. Because distance has relative degree two with respect to torque (commanded torque produces acceleration, which integrates to velocity, which in turn integrates to position), the control input has zero direct authority over $\dot{h}$ at the boundary. A filter monitoring position alone reacts

[4] **Barrier Function**: In constrained optimization, an interior barrier term such as $-\log(-g(x))$ grows without bound near constraint boundaries, preventing numerical iterates from escaping the feasible set. Barrier certificates extended this mathematical principle to dynamical systems, proving that no trajectory starting in a safe set can ever reach an unsafe state (Prajna and Jadbabaie 2004). Control barrier functions subsequently formulated this certificate as an explicit, affine condition on the instantaneous control input (Wieland and Allgöwer 2007). While an optimization algorithm's safety guarantee holds because its numerical iterates are evaluated exactly in software, a physical plant's safety guarantee holds only as long as actuator limits, dynamics, and real-time execution match the model assumed by the certificate.

[5] **Nagumo's Theorem**: Nagumo (1942) established that, under conditions ensuring unique solutions, a closed set is forward-invariant if and only if the system vector field points into the set or remains tangent to its boundary at every boundary point; viability theory subsequently formalized and generalized this principle. Because Nagumo's condition applies to continuous-time flows, a digital controller executing at periodic control ticks requires an inter-sample buffer, sized among the conditions forward invariance rests on (section 12.7.3), to guarantee physical safety between discrete updates.

only after physical momentum has made collision unavoidable. An effective barrier must incorporate approach velocity and net braking deceleration, as shown in section 12.5.1.

12.5.1 Kinetic barrier certificates and relative degree

On the descending arm, a barrier function written purely in terms of remaining spatial distance θ_{clear} leaves commanded motor torque out of its first time derivative, so the safety filter remains blind to closing velocity until the arm reaches the boundary with momentum it can no longer arrest. Across the causal boundary, high-level software proposals meet continuous physical plant dynamics governed by a control-affine differential equation:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$$

where $\mathbf{f}(\mathbf{x})$ captures unforced physical drift (including inertia, Coriolis forces, gravitational bias, and passive friction), $\mathbf{g}(\mathbf{x})$ maps control inputs to generalized accelerations, and $\mathbf{u}$ represents delivered joint torques. On this robot, embedded motor drives close the internal current and velocity loops, meaning the MCU does not write raw voltages to the coils; the per-tick cycle described in section 12.7.4 converts a torque-space decision into the setpoint and torque limit the drive accepts under the qualified delivered-effort realization contract developed there.

Evaluating how a scalar barrier certificate $h(\mathbf{x})$ evolves along state trajectories requires computing its directional time derivative. By the multivariable chain rule, the total time derivative $\dot{h}$ expands across the state velocity $\dot{\mathbf{x}}$:

$$\dot{h}(\mathbf{x}, \mathbf{u}) = \frac{dh}{dt} = \frac{\partial h}{\partial \mathbf{x}}\dot{\mathbf{x}} = \nabla h(\mathbf{x}) \cdot \dot{\mathbf{x}} = \underbrace{\nabla h(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x})}_{L_{\mathbf{f}}h(\mathbf{x})} + \underbrace{(\nabla h(\mathbf{x}) \cdot \mathbf{g}(\mathbf{x}))}_{L_{\mathbf{g}}h(\mathbf{x})}\mathbf{u}$$

Nonlinear control theory designates $L_{\mathbf{f}}h$ and $L_{\mathbf{g}}h$ as Lie derivatives.[6] In modern computational systems, these directional derivatives are standard **Jacobian-vector products (JVPs)** supported natively by autodiff frameworks such as JAX and PyTorch:

$$L_{\mathbf{f}}h(\mathbf{x}) = \nabla h(\mathbf{x})^\top \mathbf{f}(\mathbf{x}) = \text{JVP}(h, \mathbf{x}, \mathbf{f}(\mathbf{x})), \qquad L_{\mathbf{g}}h(\mathbf{x}) = \nabla h(\mathbf{x})^\top \mathbf{g}(\mathbf{x}) = \text{JVP}(h, \mathbf{x}, \mathbf{g}(\mathbf{x})).$$

In forward-mode automatic differentiation, a JVP evaluates the derivative along a single tangent vector. When the control matrix $\mathbf{g}(\mathbf{x})$ contains multiple input columns $\mathbf{g}_j(\mathbf{x})$, the notation $\text{JVP}(h, \mathbf{x}, \mathbf{g})$ denotes the row vector of individual channel products $\text{JVP}(h, \mathbf{x}, \mathbf{g}_j)$. The drift term and the control-channel terms capture distinct physical dynamics:

1. $L_{\mathbf{f}}h(\mathbf{x}) = \text{JVP}(h, \mathbf{x}, \mathbf{f}(\mathbf{x}))$ is the **system drift JVP**, measuring the rate at which safety margin changes due solely to momentum, gravitational load, and passive dynamics.
2. $L_{\mathbf{g}}h(\mathbf{x}) = \text{JVP}(h, \mathbf{x}, \mathbf{g}(\mathbf{x}))$ is the **control authority JVP**, measuring the instantaneous sensitivity of the barrier rate of change to commanded actuator torque across the causal boundary.

Formulating an effective barrier certificate requires avoiding the **relative degree trap**. In computational graph terms, relative degree is the graph distance, counted in time-integrator nodes, between the commanded control input $\mathbf{u}$ and the barrier certificate $h(\mathbf{x})$. If an engineer defines a safety certificate purely as geometric clearance, $h(\mathbf{x}) = x_{\text{obstacle}} - x_{\text{robot}} \geq 0$, the commanded actuation is separated from position by two successive integrations:

$$\text{Actuation } u \xrightarrow{\int \frac{1}{m} dt} \text{Velocity } v_{\text{robot}} \xrightarrow{\int dt} \text{Position } x_{\text{robot}}$$

This two-stage integration cascade establishes relative degree $r = 2$. Taking the first time derivative of clearance yields the closing velocity, $\dot{h} = -v_{\text{robot}}$, which contains no control input whatsoever ($L_{\mathbf{g}}h(\mathbf{x}) = \text{JVP}(h, \mathbf{x}, \mathbf{g}(\mathbf{x})) = \mathbf{0}$). Because the control authority JVP evaluates to zero, motor torque has zero instantaneous authority over the rate of barrier decay. At the boundary of the safe set ($h = 0$), enforcing forward invariance $\dot{h} \geq -\kappa(0) = 0$ requires $-v_{\text{robot}} \geq 0 \implies v_{\text{robot}} \leq 0$. Halting a moving system instantaneously would require infinite deceleration and unbounded torque ($u = -\infty$). Applying this first-derivative condition to position alone provides no torque constraint before

[6] **Lie Derivative** (the derivative along a flow): Named after Sophus Lie, whose continuous transformation groups generate the flows along which the derivative is taken, the Lie derivative in control theory is identical to the Jacobian-vector product in modern scientific computing. Because both describe the same directional derivative, the automatic differentiation frameworks used during neural policy training can directly evaluate drift and control authority for learned or tabulated barrier functions, whereas the embedded safety enforcer executes the analytical closed form.

the geometric boundary. Higher-order barrier formulations can retain a position constraint and differentiate until the input appears; this chapter instead incorporates the stopping distance explicitly.

To restore nonzero control authority ($L_{\mathbf{g}}h \neq \mathbf{0}$), the barrier certificate must explicitly incorporate kinetic energy and braking limits. For an approaching arm joint, let $\theta_{\text{clear}} = d_{\text{tool}}/r_{\text{arm}}$ represent the remaining angular clearance, $\omega > 0$ the approach speed, $J_{\text{eff}}\dot{\omega} = \tau + \tau_{\text{bias}}$ the joint dynamics, and α_{net} a validated lower bound on net available angular braking deceleration. Adopting the linear class-$\mathcal{K}$ function $\kappa(h) = \gamma_{\text{cbf}}h$, where $\gamma_{\text{cbf}} > 0$ is a constant approach-rate gain, the **kinetic barrier certificate** is:

$$h(\theta_{\text{clear}}, \omega) = \theta_{\text{clear}} - \frac{\omega^2}{2\alpha_{\text{net}}}. \tag{12.5}$$

Differentiating $h(\theta_{\text{clear}}, \omega)$ along the joint dynamics yields $\dot{h} = L_{\mathbf{f}}h + L_{\mathbf{g}}h\,\tau$, where $L_{\mathbf{g}}h = -\frac{\omega}{J_{\text{eff}}\alpha_{\text{net}}} \neq 0$ for any nonzero approach speed $\omega > 0$. Because commanded torque τ appears directly in this first time derivative with a nonzero coefficient, the kinetic barrier achieves **relative degree one** ($r = 1$). Actuator commands now exert immediate control authority across the causal boundary. Enforcing the forward-invariance condition $\dot{h} + \gamma_{\text{cbf}}h \geq 0$ isolates the admissible torque into an explicit affine half-space constraint:[7]

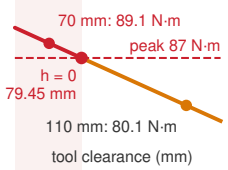
$$\tau \leq -\tau_{\text{bias}} + J_{\text{eff}}\alpha_{\text{net}}\left(\frac{\gamma_{\text{cbf}}h}{\omega} - 1\right), \qquad -\tau_{\max} \leq \tau \leq \tau_{\max}. \tag{12.6}$$

In this sign convention, positive angular velocity ω, positive gravitational bias τ_{bias}, and positive motor torque τ all act downward toward the workstation so that clearance shrinks at rate $\dot{\theta}_{\text{clear}} = -\omega$, and a negative upper bound on τ in equation 12.6 therefore sets a minimum magnitude on the upward braking torque $-\tau$ that the motor must apply against gravity. This condition defines the linear half-space $\mathbf{a}_{\text{cbf}}(\mathbf{x})^{\top}\mathbf{u} \leq b_{\text{cbf}}(\mathbf{x})$ solved by the control barrier function quadratic program (CBF-QP) filter developed next, with the half-space normal and scalar threshold determined directly by the Jacobian-vector products: $\mathbf{a}_{\text{cbf}}(\mathbf{x})^{\top} = -L_{\mathbf{g}}h(\mathbf{x})$ and $b_{\text{cbf}}(\mathbf{x}) = L_{\mathbf{f}}h(\mathbf{x}) + \gamma_{\text{cbf}}h(\mathbf{x})$. At standstill ($\omega = 0$), the microcontroller firmware applies a separate static acceleration and position-hold check to prevent division by zero.[8]

Applying this kinetic barrier to the descending-arm parameters from section 12.4 sets concrete torque ceilings across the approach. The moment of inertia is $J_{\text{eff}} = 3$ kg·m^2. The gravitational bias torque is $\tau_{\text{bias}} = 65.25$ N·m, directed toward the workstation. The actuator saturation limit is $|\tau| \leq 87$ N·m, so the net angular braking deceleration is $\alpha_{\text{net}} = (\tau_{\max} - \tau_{\text{bias}})/J_{\text{eff}} = 7.25$ rad/s^2. The link lever arm is $r_{\text{arm}} = 0.80$ m, the approach speed is $\omega = 1.2$ rad/s, and the barrier gain is $\gamma_{\text{cbf}} = 10\ \text{s}^{-1}$. These are stipulated joint-level dynamics that deployment must calibrate on the actual arm. At 110 mm of tool clearance from the workstation surface, the angular clearance is $\theta_{\text{clear}} = 0.1375$ rad and the barrier certificate evaluates to $h = 0.0382$ rad, so substituting into equation 12.6 requires the joint to deliver at least 80.1 N·m of counteracting braking torque. Most of that demand is not deceleration at all, because the first 65.25 N·m merely cancels gravity to keep the joint from speeding up and only the remainder slows it, which is why a joint still outside its zero set already asks for most of its peak torque. At 79.45 mm, the system reaches the zero set ($h = 0$) where the barrier demands 87 N·m, the motor's full peak torque capacity. If clearance degrades further to 70 mm, the inequality demands 89.1 N·m, which exceeds peak actuator limits and places the state outside the control-admissible viable set. There, under these scalar dynamics, fixed bias, and instantaneous maximum-braking assumptions, no controller initiated at that point can prevent contact.

Physical robotic hardware provides neither perfectly continuous state feedback nor instantaneous torque response. Stator current rise times, fieldbus jitter, and drivetrain mechanical compliance delay and soften corrective torque delivery. To guarantee safety under these real-world non-idealities, the runtime enforcer evaluates the barrier certificate against an inset boundary that explicitly accounts for the worst-case stopping deficit induced by hardware latency and compliant dynamics. This initiates corrective filtering earlier along the trajectory.

This kinetic barrier construction follows directly from rigid-body braking mechanics. For the stipulated scalar approach with exact dynamics, fixed bias, instantaneous effort, and no omitted margins, its zero level set represents the exact boundary of states from which maximum allowable braking stops the joint before collision: the **viability kernel** (Aubin 1991) of a double integrator



Inside 79.45 mm of clearance the barrier asks for more braking than the joint has.

7 | **Kinetic barrier directional derivative chain rule**: Evaluating $\dot{h}(\theta_{\text{clear}}, \omega) = \frac{\partial h}{\partial \theta_{\text{clear}}}\dot{\theta}_{\text{clear}} + \frac{\partial h}{\partial \omega}\dot{\omega}$ with $\dot{\theta}_{\text{clear}} = -\omega$ and $\dot{\omega} = (\tau + \tau_{\text{bias}})/J_{\text{eff}}$ yields $\dot{h} = (-\omega - \frac{\omega\tau_{\text{bias}}}{J_{\text{eff}}\alpha_{\text{net}}}) - \frac{\omega}{J_{\text{eff}}\alpha_{\text{net}}}\tau = L_{\mathbf{f}}h(\theta_{\text{clear}}, \omega) + L_{\mathbf{g}}h(\theta_{\text{clear}}, \omega)\tau$. Substituting into $\dot{h} + \gamma_{\text{cbf}}h \geq 0$ and rearranging produces $\frac{\omega(\tau+\tau_{\text{bias}})}{J_{\text{eff}}\alpha_{\text{net}}} \leq \gamma_{\text{cbf}}h - \omega$. Multiplying by the positive factor $\frac{J_{\text{eff}}\alpha_{\text{net}}}{\omega}$ and subtracting the bias yields equation 12.6.

8 | **Standstill velocity thresholding**: In microcontroller firmware, floating-point evaluation of $\frac{\gamma_{\text{cbf}}h}{\omega}$ is explicitly guarded by a velocity threshold ϵ_{vel}: when $|\omega| \leq \epsilon_{\text{vel}}$, the kinetic barrier is bypassed in favor of a static acceleration or position-hold bound, preventing floating-point division by zero and NaN exceptions in the sub-millisecond loop.

subject to actuator saturation. Among collision-free states, the kernel's complement is the set of inevitable collision states defined in section 11.5.3. With a conservative available lower bound instead of exact maximum braking, the same construction supplies a sufficient stopping set rather than the exact kernel. The resulting torque requirement remains compatible with the stipulated actuator limits. At $h = 0$, the inequality in equation 12.6 demands exactly $-\tau_{\max}$, and for all $h > 0$ the demand is strictly less restrictive, ensuring that at least one admissible control command exists everywhere within the safe set. Actuator feasibility across the entire set, not merely the calculus of the derivative, is what qualifies $h(\theta_{\text{clear}}, \omega)$ as a valid barrier certificate under input constraints.

This identical principle governs autonomous base navigation. The mobile base certificate $h_{\text{base}}(D_{\text{clear}}, v) = D_{\text{clear}} - [v\,\tau_{\text{delay}} + d_{\text{suffix}}(v) + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}}]$ is nonnegative precisely when the stopping subtotal in equation 12.1 fits the clearance. The operational certificate must additionally subtract the dispatch-to-admission travel before granting permission. Both constructions reserve the motion needed to stop. A nonnegative stopping reservation alone does not establish forward invariance; the sampled implementation must also preserve that reservation under its qualified state, effort, and timing bounds. The single-joint arm derivation omits latency and localization margins temporarily only to isolate the mechanics of relative degree.

When complex nonlinear dynamics prevent closed-form analytical solutions for the viability kernel, **Hamilton-Jacobi (HJ) reachability**[9] computes the boundary numerically as the zero level set of an optimal control value function (Mitchell et al. 2005; Bansal et al. 2017). The resulting tabulated lookup surface serves as the runtime barrier certificate, with its spatial gradient defining the half-space normal for real-time filtering. Because state-space grid discretization scales exponentially with dimensionality, HJ reachability lookup tables are best suited for decoupled, low-dimensional subsystems such as a single revolute joint or the longitudinal tracking channel of a mobile base.

A barrier certificate specifies which actuator commands preserve forward invariance, but it does not specify how the runtime safety path should handle an uncertified proposal. When a candidate command violates the half-space constraint, the enforcer must choose whether to project the command onto the boundary of admissible inputs, reject it in favor of a precomputed emergency braking trajectory, or maintain the previously certified setpoint. This intervention policy determines how mathematical barrier constraints translate into real-time authority over the motor drives.

12.6 Refusing Proposals and Enforcing State Invariants

A trajectory chunk arrives from the application processor on schedule, properly framed, and bearing a fresh evidence timestamp, yet one of its constituent joint setpoints would accelerate the arm directly into a steel rack upright. Nothing in the incoming packet payload marks that setpoint as erroneous. The permission path therefore grants zero executive authority to an incoming learned command until that proposal passes the configured state, input, freshness, and stopping-clearance checks. This is the third law, **proposal is not permission** (principle 3), executed deterministically one 1 ms tick at a time. Rather than assuming that a learned proposal satisfies stability criteria, actuator limits, or stopping guarantees, the permission path treats every incoming vector strictly as an advisory candidate.

The resident controller and output selector translate this principle of refusal into deterministic execution. The resident controller maintains a fallback response qualified for the current machine state, while the selector determines whether to admit an incoming candidate, apply a feasible correction, or transfer authority directly to that resident response. Rejecting an incoming candidate does not instantaneously abort the command currently driving the plant. Instead, the active command continues running so long as its own evidence timestamp, lease deadline, tracking error bound, and stopping reservation remain valid.

Under this architecture, the upstream neural policy never commands the physical plant directly. Its proposals reach the safety MCU through the board link and seqlock mailbox, with no addressable memory shared between the two compute domains. Each admitted chunk drives the plant only under a chunk lease, a hardware-enforced dead-man permission[10] that lapses unless a fresh, validated proposal renews it. A trajectory chunk arrives as a horizon of H waypoints, and the admission filter inspects all H points against reach, velocity, and stopping-reservation constraints before permitting the first waypoint to execute. On each subsequent control tick, the enforcer re-evaluates the specific setpoint scheduled for execution against live sensor telemetry. If the enforcer projects a setpoint

[9] **Hamilton-Jacobi Reachability**: Originating in the zero-sum differential game formulations of Isaacs (1965), reachability analysis frames safety as a game between a control policy and a worst-case adversarial disturbance. The resulting safe set contains precisely those states from which the controller can guarantee safety against any bounded disturbance trajectory. Consequently, an HJ reachability table is inherently robust but conservative, and its safety guarantee holds only within the assumed disturbance envelope.

↪ The Hamilton-Jacobi value function and its differential-game formulation are developed in section B.6.3.

[10] **Dead-Man Control**: Early electric locomotives incorporated a spring-loaded dead-man handle that the operator had to depress continuously; releasing the handle (if the driver lost consciousness, for example) interrupted tractive power and engaged the mechanical brakes. System safety arose from the cessation of action rather than the transmission of an explicit fault message. The chunk lease applies the same principle in software. The host must actively renew its permission, ensuring that an upstream processor too damaged to command a stop is inherently incapable of renewing execution authority.

on a single tick, it does not rewrite the chunk's downstream waypoints. Instead, the safety MCU reports the intervention upstream, requiring the policy to replan from the actual, corrected state rather than resuming execution along a trajectory computed for a state the machine never reached.

↩ The seqlock mailbox that hands proposals to the permission path is established in section 4.3.2.

If the proposal remains fresh, the physical machine state resides within the qualified stopping region, and the tracking error satisfies $\|e(t)\| \leq \epsilon_{\text{track}}$, the output selector permits the proposal (or its minimally projected variant $\mathbf{u}^*$) to drive the actuators. Conversely, if the chunk lease expires without renewal, tracking error breaches the tolerance bound, no admissible command satisfies instantaneous physical constraints, or the active command endangers physical boundaries beyond the corrective capacity of projection, the output selector revokes permission and unilaterally engages the baseline safety controller. The upstream neural policy is not consulted, cannot negotiate, and possesses no hardware mechanism to override the selector's decision.

When a proposal conflicts with physical constraints, the enforcer chooses between filtering and vetoing the command. Filtering applies when the proposal is structurally sound and a nearby admissible control action preserves the policy's underlying intent. Consider the manipulator arm approaching a spring-latched cage door when the chunk policy proposes approaching the strike plate at 0.10 m/s. The door zone admits only the guarded approach speed of 0.03 m/s defined by the target operational design domain (ODD), the velocity envelope validated across empirical trials and demonstration datasets. The faster proposal therefore fails on both evidential and physical grounds. It fails on evidence, because no trial or demonstration sampled that speed. It also fails on physics, because the contact analysis of section 12.7.2 shows that at that speed the force on the plate at brake onset nearly reaches the latch limit, leaving a stop that demands far harder arm deceleration than the guarded approach needs. Consequently, the enforcer preserves the approach trajectory's target and spatial direction, projecting only its scalar speed down to the 0.03 m/s limit. The least-change quadratic program formulated in section 12.7 solves this projection in real time alongside the half-space constraints derived by a control barrier function.

↩ The evaluation record's target ODD boundary, sampled at the guarded approach speed, is established in section 7.8.

Filtering must account for momentum alongside position, ensuring that the actuators retain sufficient control authority to bring the machine to a complete stop before available clearance vanishes (section 12.5). In the latch example, scalar speed projection preserves both the goal and the approach direction. In a coupled joint-torque filter, however, a correction can alter the end-effector trajectory through task space; such modification remains permissible only when the revised motion satisfies dynamic continuity constraints and remains task-admissible. Conversely, a structurally invalid proposal (a discontinuous jump in commanded position, an acceleration profile beyond peak motor torque, or a trajectory pointed directly into an immovable obstacle) indicates that perception or inference has entered an unmodeled failure mode. Projecting an ungrounded candidate onto the nearest admissible boundary would execute arbitrary motion unrelated to any coherent task intent. Under these conditions, the enforcer exercises an outright veto. The selector permits the active command to finish only if that command continues to satisfy its own validity checks; otherwise, it immediately transfers authority to a prequalified local fallback.

Both filtering and veto operate strictly at runtime; offline training cannot substitute for either mechanism. Penalty terms, barrier losses, and reward shaping bias the policy distribution during training, but they cannot mathematically bound what the deployed neural network will emit in edge cases. The **statistical safety fallacy** conflates a bound on expected violation cost with an absolute guarantee across every execution trace. A constrained reinforcement learning objective such as $\mathbb{E}[\text{cost}] \leq \delta$ constrains only the ensemble mean, leaving open the possibility of catastrophic tail events during individual rollouts. Formal neural-network verification can establish provable output bounds over bounded input sets (Katz et al. 2017), and closed-loop verification can combine those bounds with plant models. Formal proofs nevertheless remain vulnerable to reality gaps: unmodeled friction, sensor latency, non-deterministic scheduling, or actuator saturation outside the proof's operational assumptions will invalidate the guarantee. The permission path therefore verifies every candidate control vector at the hardware actuation boundary, regardless of the optimization objective used during training. When an incoming proposal is safe, the verification check is transparent, allowing learned sensorimotor competence to pass directly to the motor drives without alteration.

↩ Why training-time penalties cannot bound the actions a deployed policy emits is established in section 1.12.

The chunk lease, 60 ms on this machine, bounds upstream execution authority during a host stall or scheduling delay, provided the local timer, selector, sensing, and actuation paths survive. In addition to the lease, a dedicated 100 Hz host heartbeat monitors basic link viability with a 30 ms

silence timeout. The safety MCU executes its deterministic 1 kHz control loop, servicing a local 5 ms hardware watchdog timer only after successfully completing an entire cycle; an independent hardware trip circuit supervises MCU power and clock integrity. If either the host heartbeat or the chunk lease expires, the safety MCU revokes proposal permissions and engages a deterministic, state-matched fallback from the responses that section 12.9 defines, assuming reserved clearance and actuator braking capacities remain sufficient. On the warehouse mobile manipulator, if a proposer renews its lease and subsequently stalls, mechanical braking begins within 82 ms of the last valid renewal. This lease-expiry scenario assumes the underlying heartbeat remains active even as valid proposals cease; a total host operating-system freeze trips the 30 ms heartbeat timeout even earlier. If a worker steps out from behind a warehouse rack, the sensor capture latency of the camera frame that first revealed the obstacle adds directly to this renewal-to-onset delay, and section 12.3 carries this cumulative latency directly into the admission check.

↩ The lease, heartbeat, and self-watchdog timers, and why a heartbeat cannot renew a lease, are established in section 4.4.1.

Severing upstream authority requires an unambiguously defined physical response at the lowest hardware interface. The zero-command floor defines the default physical state enforced when active software control terminates, but commanding 'zero' produces drastically different dynamics depending on the actuator topology. Injecting zero current into a low-friction, direct-drive actuator allows a gravity-loaded manipulator arm to drop freely unless a mechanical brake or counterweight holds the link. Conversely, commanding zero velocity to a stiff position-controlled servo can demand infinite acceleration and trigger violent inertial transients. As detailed in section 2.6.2, an unpowered drive drops into one of three hardware states: unconstrained coasting, phase-short dynamic braking, or mechanical locking via spring-applied brakes. The system architect must ensure that the commanded fallback matches both the instantaneous kinetic state and the physical capabilities of the surviving hardware.

Proposal rejection is a continuous, steady-state operating decision rather than an exceptional emergency trip. The enforcer must evaluate every invariant and execute its selected fallback deterministically even if the upstream proposer crashes completely, provided local sensing, braking authority, and stopping clearance remain intact. When those physical invariants hold, runtime filtering smoothly reconciles learned proposals with safety boundaries, preserving admissible task progress within the qualified physical envelope.

12.7 The Minimal Intervention Principle and Optimization Filters

When the arm approaches the cage-door latch too quickly while still proposing a useful target and direction, rejecting every such proposal would discard useful learned behavior, whereas accepting it unchecked would exceed the permitted approach speed. The **minimal intervention principle** resolves this trade-off by passing an admissible proposal untouched and choosing the smallest feasible correction, one that satisfies physical constraints and finishes before the permission deadline, whenever a proposal crosses a boundary. The resulting **safety shield**[11] preserves useful task progress within the physical limits. For the torque forbidden by equation 12.6, the correction should remove only the constraint violation; replacing every near-boundary command with a stop would discard admissible learned behavior.

[11] **Shielding**: In safe reinforcement learning, a shield sits between the learning agent and its environment and overrides only those actions that would violate a temporal-logic safety specification (Alshiekh et al. 2018). The CBF-QP is a continuous-control shield. Its override is the smallest torque change rather than a substituted discrete action, and its specification is a stopping margin rather than a logic formula, which lets it keep the learned action's direction instead of replacing it.

For the already-derived torque half-space, the enforcer solves a constrained least-change problem, the **CBF-QP safety filter** (Ames et al. 2017). The tracker supplies the nominal effort; the barrier and actuator limits constrain the result. The combined program of section B.3.2, which optimizes convergence and safety together and relaxes convergence when they conflict, is not needed here. The tracker of section 12.2 already supplies the nominal effort, and the filter has only to keep it inside the barrier:

$$\min_{\mathbf{u}\in\mathbb{R}^m} \frac{1}{2}\|\mathbf{u}-\mathbf{u}_{\text{nom}}\|^2 \quad \text{s.t.} \quad \mathbf{a}_{\text{cbf}}(\mathbf{x})^\top \mathbf{u} \le b_{\text{cbf}}(\mathbf{x}), \quad \mathbf{u}_{\min} \le \mathbf{u} \le \mathbf{u}_{\max} \tag{12.7}$$

where the half-space normal and threshold are defined by the directional derivatives $\mathbf{a}_{\text{cbf}}(\mathbf{x})^\top = -L_{\mathbf{g}}h(\mathbf{x})$ and $b_{\text{cbf}}(\mathbf{x}) = L_{\mathbf{f}}h(\mathbf{x}) + \kappa(h(\mathbf{x}))$ derived in section 12.5.1, matching the single-axis torque constraint of equation 12.6, and $[\mathbf{u}_{\min}, \mathbf{u}_{\max}]$ represents the verified actuator torque envelope. The selected implementation allocates 150 µs to this solve. Dedicated memory and a fixed iteration cap support that target; qualification must establish completion or a timely fallback under the admitted constraint set.

12.7.1 Closed-form quadratic programming and torque projection

Clamping each joint axis independently, or projecting a command onto a barrier and then clipping it to motor torque limits afterward, can leave the coupled stopping constraint violated while skewing the direction of motion across task space. A single active barrier admits a closed-form projection when the projected command also lies inside the actuator bounds; if a bound becomes active, the enforcer must solve for the intersection rather than project onto the barrier and clamp afterward.

For a nonzero constraint normal and inactive actuator bounds, the single barrier constraint $\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u} \leq b_{\text{cbf}}$ in equation 12.7 admits the closed-form projection derived in section B.4.4:

$$\mathbf{u}^{*} = \mathbf{u}_{\text{nom}} - \lambda^{*}\mathbf{a}_{\text{cbf}}, \qquad \lambda^{*} = \max\left(0, \frac{\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u}_{\text{nom}} - b_{\text{cbf}}}{\|\mathbf{a}_{\text{cbf}}\|^{2}}\right) \tag{12.8}$$

where the optimal dual multiplier $\lambda^{*} \geq 0$ is zero whenever the nominal proposal is safe ($\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u}_{\text{nom}} \leq b_{\text{cbf}}$), leaving $\mathbf{u}^{*} = \mathbf{u}_{\text{nom}}$. It is strictly positive when the proposal breaches the boundary ($\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u}_{\text{nom}} > b_{\text{cbf}}$). In that case it scales the minimal intervention vector $\Delta\mathbf{u} = -\lambda^{*}\mathbf{a}_{\text{cbf}}$, which projects $\mathbf{u}^{*}$ orthogonally onto the safe half-space boundary $\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u} = b_{\text{cbf}}$.

On a two-axis manipulator with illustrative values, suppose the tracker following setpoints from an unconstrained neural policy computes nominal joint torques $\mathbf{u}_{\text{nom}} = [\,3.5,\, 3\,]^{\top}\ \text{N}\cdot\text{m}$ against a validated safety half-space $\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u} \leq b_{\text{cbf}}$. Its parameters are $\mathbf{a}_{\text{cbf}} = [\,2,\, 1\,]^{\top}$ and $b_{\text{cbf}} = 5\ \text{N}\cdot\text{m}$. The per-joint actuator limits are $\pm 5\ \text{N}\cdot\text{m}$. Because the candidate gives $\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u}_{\text{nom}} = 10\ \text{N}\cdot\text{m}$ and breaches the boundary by $5\ \text{N}\cdot\text{m}$, equation 12.8 yields $\lambda^{*} = 1$ and projects the command to $\mathbf{u}^{*} = [\,1.5,\, 2\,]^{\top}\ \text{N}\cdot\text{m}$. This meets the half-space exactly while keeping both joints within their limits at an intervention norm of $2.236\ \text{N}\cdot\text{m}$. By contrast, an independent per-axis clamp that ignores the coupling and cuts only the joint with the larger normal component holds Joint 1 to $2\ \text{N}\cdot\text{m}$ while passing Joint 2 at $3\ \text{N}\cdot\text{m}$. That clamp gives $\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u}_{\text{clip}} = 7\ \text{N}\cdot\text{m}$, still $2\ \text{N}\cdot\text{m}$ outside the constraint, while skewing the torque vector from 40.6° to 56.3°. The per-axis clamp both violates the coupled constraint and distorts the command. Orthogonal projection satisfies the constraint with the smallest possible torque change, though it too rotates the torque vector, so the resulting end-effector path still requires checking against the coupled plant model.

This geometric behavior exposes the distinction between **torque-space and task-space minimal intervention**: an orthogonal projection $\mathbf{u}^{*} = \mathbf{u}_{\text{nom}} + \Delta\mathbf{u}$ minimizes joint effort deviation $\|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^{2}$, which redistributes multi-axis joint torques and rotates the net Cartesian wrench. While a feasible projection can preserve the safe set $\mathcal{C}$ under the stated model and timing assumptions, it deflects the end-effector trajectory and may arrest task progress. If the upstream neural policy continues emitting unprojected nominal setpoints without awareness of the intervention, the robot can chatter or stall at the boundary. The downstream enforcer must therefore stream the intervention vector $\Delta\mathbf{u}$ back to the application processor as a telemetry signal, notifying the high-level planner to re-plan around the newly active physical constraint.

The same correction has a picture in state space and one in torque space (figure 12.7). In state space, a trajectory that would breach the velocity-dependent boundary $\partial\mathcal{C}$ is turned along it; in the torque plane, the projection places $\mathbf{u}^{*}$ on the half-space boundary $\mathbf{a}_{\text{cbf}}^{\top}\mathbf{u} = b_{\text{cbf}}$, while the per-axis clamp leaves $\mathbf{u}_{\text{clip}}$ outside it and skews its direction.

Filter choice joins a mathematical condition to a measured execution budget and the machine's available braking authority. As summarized in table 12.2, architectural choices differ in their handling of relative degree, solver worst-case execution time (WCET), forward invariance guarantees, and directional fidelity. Independent clamping can miss coupled constraints; a CBF-QP can enforce a derived half-space if it is feasible and finishes within the verified cycle. Predictive filters (Wabersich and Zeilinger 2021) trade a longer horizon for configuration-dependent compute cost.

The rows of table 12.2 belong to one unified family rather than rival techniques (Hsu et al. 2024): each filter monitors a candidate against a **fallback policy** known to keep the machine safe and intervenes whenever the candidate would leave the set from which that fallback still works, differing only in how the intervention occurs. Whereas a **least-restrictive** filter passes the proposal untouched up to the boundary and then switches abruptly to the fallback (which is simple to implement but chatters along the boundary), a smooth filter such as the CBF-QP blends in a correction before the boundary is reached. This chapter's enforcer combines both mechanisms: admitting a trajectory only with a matched stop suffix and holding a state-matched fallback for every reachable state forms

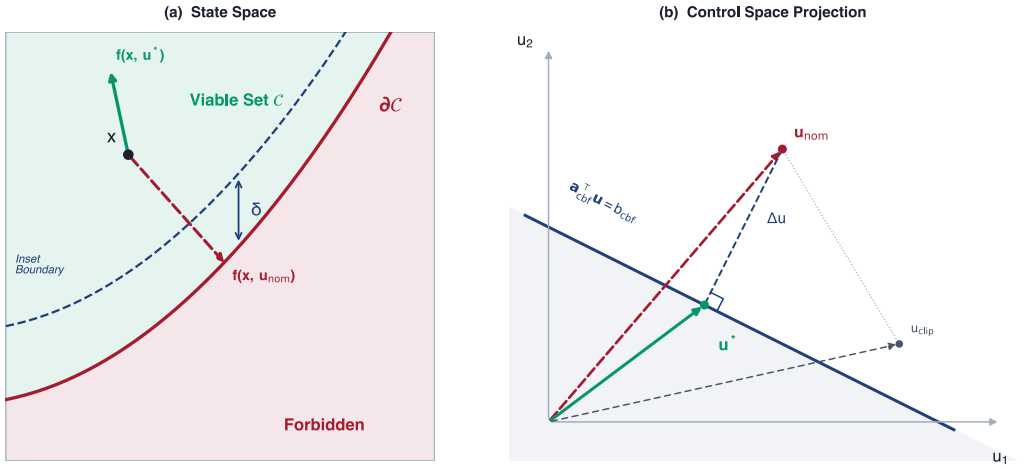


Figure 12.7: Kinetic barrier and torque projection: Panel (a) schematically shows a velocity-dependent viable boundary inset from geometric clearance; the raw distance alone has relative degree two for torque. Panel (b) illustrates Euclidean projection onto a supplied torque half-space. Panel (b) is schematic and not drawn to the worked example's values; that example's $[2, 1]$ normal and $5\ \mathrm{N \cdot m}$ threshold are abstract arithmetic inputs, not coefficients derived from the arm example.

a **backup-set filter** that checks safety by rolling the fallback forward from the proposed state (Gurriet et al. 2018; Chen et al. 2021). The per-tick CBF-QP acts as the smooth local filter layered on top of it, and HJ reachability (Fisac et al. 2019) supplies the value function when a precomputed surface replaces a closed-form barrier.

Table 12.2: Safety-filter decisions: A filter's physical claim depends on the chosen barrier and validated plant, not on the algorithm name. Nominal solve measurements are not worst-case deadlines. The energy-tank row is developed for intentional contact in section 12.7.2.

Filter	Input and condition	What it can establish	Main validation task
Independent torque clamp	Each axis checked separately	Hardware amplitude limit, not coupled stopping clearance	Test coupled wrench and stopping behavior
First-order CBF-QP	Barrier whose first derivative contains the input	Conditional invariance from a viable initial state	Validate model, feasible torque set, and cycle bound
Kinetic / higher-order barrier	Position constraint with speed and braking authority included	Conditional stopping margin before geometric contact	Bound bias torque, delay, estimate error, and discretization
Energy tank / passivity filter	State E_{tank}, power balance $\dot{E}_{\mathrm{tank}}$, force F_{limit}	Conditional energy bound; force requires a separately qualified channel	Bound force-sensor bias, calibrate virtual damping matrix $\mathbf{D}_{\mathrm{virt}}$, and set reserve floor E_{min}
HJ value-function filter	Precomputed value function over a gridded state space	Containment for the modeled region	Verify grid coverage, interpolation error, and disturbance bounds
Predictive safety filter	Finite-horizon trajectory and terminal set	Recursive feasibility only under terminal and timing assumptions	Bound full solve time and fallback on timeout

Operating near an active safe-set boundary can produce chattering. If a learned policy repeatedly proposes an unsafe command, sampled enforcement may switch the active constraint across successive 1 ms ticks. Depending on the plant and controller, rapid torque changes can excite a structural mode or increase motor heating and transmission wear. A tested boundary layer or hysteresis can reduce switching, but its added tracking error must fit inside the validated stopping margin.

The intervention vector $\Delta\mathbf{u} = \mathbf{u}^* - \mathbf{u}_{\mathrm{nom}}$ also measures upstream policy health. A rising intervention frequency or magnitude can indicate proposal drift, a changed task, or a changed plant.

Normal filtering can also preserve successful whole-machine service while reducing unassisted proposer autonomy; the log must distinguish these outcomes rather than count every correction as a task failure. Summed over an illustrative rolling window of $W = 500$ ms as $\sum_k \|\mathbf{u}_k^* - \mathbf{u}_{\text{nom}\,k}\| \Delta t$, the metric lets the permission path flag divergence for logging or operator notification while the per-tick limits still protect the plant.

12.7.2 Intentional contact and in-contact energy barriers

Once the manipulator arm reaches the cage-door strike plate ($\theta_{\text{clear}} = 0$), the spatial kinetic barrier (equation 12.5) reduces to $h(0,\omega) = -\omega^2/(2\alpha_{\text{net}}) \geq 0$, which admits only zero speed ($\omega = 0$) and freezes the gripper before it can depress the latch. A spatial kinetic barrier operates under the premise that the robot's objective is to avoid collision ($\theta_{\text{clear}} > 0$), whereas manipulation, grasping, and assembly require intentional physical contact ($\theta_{\text{clear}} = 0$). While the spatial barrier can admit arrival at contact with zero approach speed, it does not govern the subsequent force-controlled interaction where geometric separation no longer supplies the relevant margin. Once the warehouse mobile manipulator's gripper meets the strike plate at the guarded approach, the arm must push against the latch spring, and the margin that matters is the energy and force it delivers there.

Enforcing safety during intentional contact requires transitioning from a **spatial clearance barrier** to an **energetic passivity barrier**. When physical clearance is zero ($D_{\text{clear}} = 0$), safety cannot mean zero velocity; it means bounding mechanical energy exchange and normal contact force $f_{k,\perp}$. A controller in physical contact interacts with an unmodeled, potentially stiff environment. If tracking errors or neural policy oscillations pump unmeasured energy into the interface, the closed-loop system undergoes contact chattering, limit cycles, or high-force impact spikes that damage both the workpiece and the robot's transmission.

To bound active energy exchange, the enforcer augments the controller state with an **energy tank** $E_{\text{tank}}(t)$[12] (Ferraguti et al. 2013; Hogan 1985) grounded in passivity. A passive system can return no more energy than its initial stored energy plus the energy it absorbs, so a power-preserving interconnection with a passive environment retains a passive energy balance (while a stability claim additionally requires the appropriate dissipation and state-observability conditions). The tank enforces that property by bookkeeping, acting as a software tally of work rather than a physical store of energy. Every joule that virtual damping removes from the arm's motion is credited to the tank, and the controller may spend actively only what the tank holds, namely its accumulated credits plus a fixed initial reserve. In this contact formulation, $\mathbf{u}_{\text{active}}$ denotes the active torque component excluding damping, so the total controller torque is $\mathbf{u}_{\text{active}} - \mathbf{D}_{\text{virt}}\dot{\mathbf{q}}$. The damping term can credit the tank only to the extent that the actuator actually realizes the corresponding dissipative work, and using total torque in the debit while also crediting damping would count the same dissipation twice. With those components separated, the modeled balance is:

$$\dot{E}_{\text{tank}} = \dot{\mathbf{q}}^\top \mathbf{D}_{\text{virt}}\, \dot{\mathbf{q}} - \mathbf{u}_{\text{active}}^\top \dot{\mathbf{q}} \tag{12.9}$$

where $\dot{\mathbf{q}}^\top \mathbf{D}_{\text{virt}}\, \dot{\mathbf{q}} \geq 0$ is the dissipative power credited under qualified torque tracking and $\mathbf{u}_{\text{active}}^\top \dot{\mathbf{q}}$ is active mechanical power. Saturation, delay, or tracking error requires conservative bounds on realized work rather than credit for an unrealized damping request. The enforcer then defines the energy-tank barrier certificate of equation 12.10:

$$h_{\text{energy}}(\mathbf{x}, E_{\text{tank}}) = E_{\text{tank}} - E_{\text{min}} \geq 0 \tag{12.10}$$

where $E_{\text{min}} > 0$ is a reserved baseline. Combined with a hard contact-force limit $h_{\text{force}}(\mathbf{F}_{\text{contact}}) = F_{\text{limit}} - \|\mathbf{F}_{\text{contact}}\| \geq 0$, the enforcer establishes the in-contact viable set $\mathcal{C}_{\text{contact}}$. While $E_{\text{tank}} > E_{\text{min}}$ and $\|\mathbf{F}_{\text{contact}}\| < F_{\text{limit}}$, candidate neural actions are admitted to perform manipulation. If the task consumes the allocated energy budget ($E_{\text{tank}} \to E_{\text{min}}$) or contact force approaches F_{limit}, the barrier filter clamps active energy injection ($\mathbf{u}_{\text{active}}^\top \dot{\mathbf{q}} \leq 0$), restricting active work under the modeled power balance. This energy constraint does not by itself bound contact force, because a passive interaction with a stiff surface can still produce a large force. The force channel needs its own sensing-error, response-delay, and actuator bounds before the combined constraints can justify continued contact.

For a contact command, integrating the work balance in equation 12.9 over one control interval Δt supplies the energy allowance for the same least-change decision. Suppose the current tank

[12] **Dissipativity**: Willems (1972) generalized circuit passivity to any dynamical system with a storage function, an energy-like quantity that can grow no faster than the power supplied to the system. Interconnected passive systems inherit that bound, which is why contact with an unknown but passive environment can be kept stable without a model of the environment. The energy tank is a storage function that the controller keeps explicitly in software, so the bound holds only as far as the tank's bookkeeping matches the work the actuators actually do.

level is $E_{\text{tank}_0} \geq E_{\text{min}}$, a conservative lower bound on damping power actually realized is P_d, and a bounded contact-direction speed $v_c > 0$ is held over this simplified interval. The spendable work is then $E_{\text{tank}_0} - E_{\text{min}} + P_d \Delta t$. Because a proposed constant force $F_{\text{nom}} \geq 0$ debits $F_{\text{nom}} v_c \Delta t$, the energy allowance caps force at $F_E = (E_{\text{tank}_0} - E_{\text{min}} + P_d \Delta t)/(v_c \Delta t)$. Letting F_{cap} denote the smaller of the deliverable force and the contact-force ceiling after sensing and delay allowances, the admitted command under these stipulated scalar conditions is $F^* = \min(F_{\text{nom}}, F_E, F_{\text{cap}})$, whose debit leaves $E_{\text{tank}_1} = E_{\text{tank}_0} + P_d \Delta t - F^* v_c \Delta t \geq E_{\text{min}}$ while keeping force below the independent cap. The controller can therefore continue useful contact with a reduced force instead of abandoning the task. At the reserve with no credited damping, however, no positive active work is admitted. Zero speed requires a separate force check rather than division by speed, and delayed or uncertain power requires conservative work bounds over the interval.

The force channel's own bounds can be computed for the running machine's two contacts: the cage door, which the arm pushes on purpose, and the coworker's hand, which it must never press too hard. Each calculation turns a force limit into a speed that the enforcement record carries as a channel ceiling, and each rests on premises that only a measurement can settle. They are derived here, where the channel is enforced; the release decision consumes the results as premises.

↪ Contact force and speed ceilings are enforced as release premises in section 16.2.

At the door the quantity that matters is how fast force rises. Against a strike plate of stiffness k_{latch}, an arm approaching at v_{latch} raises the contact force at $df_{k,\perp}/dt = k_{\text{latch}}\, v_{\text{latch}}$, which at the guarded 0.03 m/s against the 4.0 × 10^5 N/m plate is 12 N/ms. The contact tripwire fires at $F_{\text{trip}} = 15$ N, but the contact loop needs $T_c = 2$ ms for detection, command delivery, and torque rise, and the plate keeps compressing by $v_{\text{latch}} T_c$ before deceleration begins. The force at physical braking onset is therefore

$$F_{\text{cmd}} = F_{\text{trip}} + k_{\text{latch}}\, v_{\text{latch}}\, T_c \tag{12.11}$$

which evaluates to 39 N, well inside the 100 N latch limit. The limit still binds the stop that follows, because the plate goes on compressing while the arm sheds its remaining kinetic energy:

$$x_{\text{max}} = \frac{F_{\text{cmd}}}{k_{\text{latch}}} + \frac{v_{\text{latch}}^2}{2a_{\text{arm}}} \leq \frac{F_{\text{lim}}}{k_{\text{latch}}} \tag{12.12}$$

Against time since first contact (figure 12.8), force climbs through the tripwire and across the loop-response window to 39 N at brake onset; the commanded stop then bends the curve over to finish at the 100 N latch limit, while an unbraked run crosses the limit within a few milliseconds. The braking term ignores the plate's spring work, which would help decelerate the arm, so the bound is conservative. Below the limit the plate can compress only 0.15 mm further, and stopping within that distance requires about 3 m/s², an illustrative stop of about 10 ms.

The same two equations show why the approach speed is an enforced channel rather than a preference. At the chunk policy's proposed 0.10 m/s (section 12.6), equation 12.11 already gives 95 N at braking onset, and equation 12.12 then demands an arm deceleration of 400 m/s². The faster approach spends the response interval on force that the stop can no longer absorb. This is the physical half of the refusal in section 12.6; the evidential half is that no trial sampled this speed. Both results treat the arm as rigid and the plate as the whole contact stiffness. Gripper, wrist-sensor, and joint compliance in series lower the effective stiffness and slow the rise, while a geared joint reflects rotor inertia into the moving mass. Whether this arm achieves the required deceleration within a fraction of a millimeter depends on joint torque, drive rise time, and structural stiffness that only arm stopping trials establish. A compliant element placed in series on purpose turns that softening into passive protection. Springs in series add their compliances, so the combined stiffness falls below that of the softer element and the force rises more slowly, a protection that holds even when the enforcer cannot stop in time. The stiffness of that element is therefore part of the contact margin, and section 18.3.4.2 sizes it for this latch.

The coworker's hand is the opposite case, a contact the task tolerates only up to a bounded peak force. The force depends first on which contact occurs. In a *transient* contact the hand is free and recoils, so both bodies move and the collision acts through their reduced mass m_{red}. In a *clamped* (quasi-static) contact the hand is trapped against a rack upright or the robot's own body, cannot recoil, and receives the full robot-side effective mass at the tool center point, $m_{\text{eff}} = 12$ kg. The biomechanical body model of ISO/TS 15066:2016, whose collaborative-application requirements ISO 10218-2:2025 now incorporates (section 12.9.3), assigns the hand and fingers an effective mass

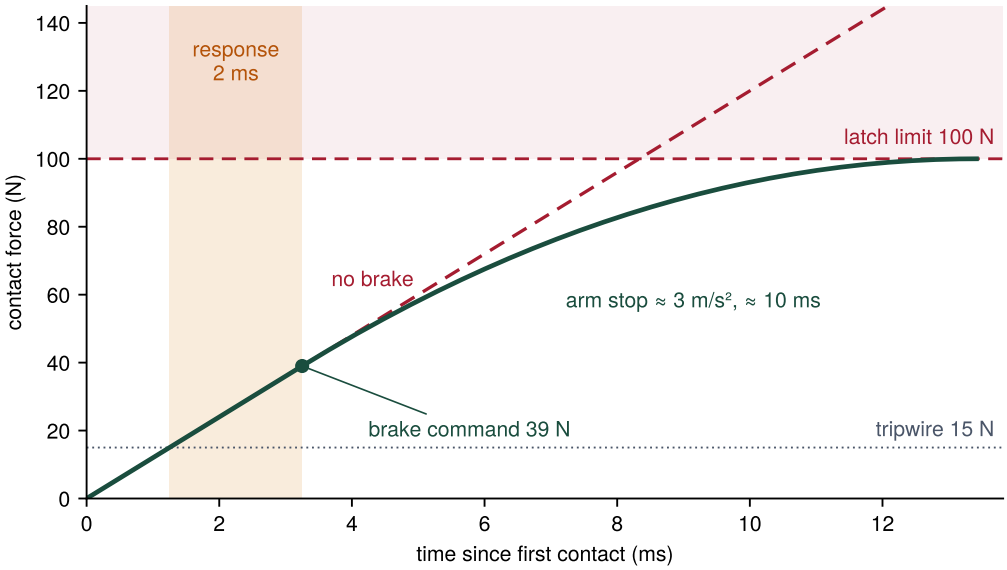


Figure 12.8: Latch contact force against time: Force on the strike plate rises at 12 N/ms from first contact. It crosses the 15 N tripwire, but the 2 ms contact-loop response lets it reach 39 N before the brake command. A stop at about 3 m/s², lasting about 10 ms, brings the arm to rest exactly at the 100 N limit; without the brake, force crosses the limit within a few milliseconds. The plate is modeled as a linear spring and the arm as rigid.

m_H of 0.6 kg and an effective spring constant k_H of 75 N/mm (International Organization for Standardization 2016, 2025b; Kirschner et al. 2022). This worked example estimates the robot-side mass as half the arm's moving mass plus its payload capacity; it does not compute the configuration- and direction-dependent mass reflected through the contact Jacobian. In an unbraked elastic contact the peak force is

$$F_{\text{unbraked}} = v_{\text{approach}} \sqrt{k_H \, m_{\text{red}}}, \qquad m_{\text{red}} = \left(\frac{1}{m_H} + \frac{1}{m_{\text{eff}}} \right)^{-1} \tag{12.13}$$

This is the spring-mass bound of equation 6.7, which equated a moving mass's kinetic energy to the peak elastic energy of the contact, with one change: two bodies move. Momentum conservation keeps the pair's common (center-of-mass) motion unchanged through the contact, so only the kinetic energy of their relative motion, $\frac{1}{2} m_{\text{red}} v_{\text{approach}}^2$ for a closing speed v_{approach}, can compress the hand, and the reduced mass takes the place of the single moving mass. For this hand and arm, m_{red} is 0.57 kg, because the light hand dominates the sum; a clamped hand is the limit $m_H \to \infty$, in which m_{red} becomes m_{eff}. Solved for speed at a force criterion F_h, the equation gives the channel ceiling $v_{\text{approach}} = F_h / \sqrt{k_H \, m_{\text{red}}}$. The peak follows the assumed masses and stiffness; the model establishes neither an impact duration nor a pressure or injury limit.

↩ The single-mass spring-impact bound that this reduced-mass form extends is established in section 6.4.2.

At the arm's 1 m/s free-space limit even the transient model predicts 207 N, or 4.1× the 50 N criterion, so the arm cannot enter an unseparated contact at that speed. At the handover speed the two contact cases bound the approach very differently (notebook 12.3).

Napkin Math 12.3: The clamped-hand penalty

Problem: The arm approaches the coworker's hand at the handover speed. What peak force does equation 12.13 predict for a free hand that recoils and for a hand clamped against a rack upright, and what approach speed does the force criterion allow in each case?

Variables:

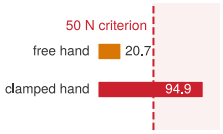
- $m_{\text{eff}} = 12$ kg (Derived; the simplified half-arm-mass plus payload-capacity estimate used above).
- $m_H = 0.6$ kg and $k_H = 75$ N/mm (Datasheet; the ISO/TS 15066 hand body model).
- $v_{\text{approach}} = 0.10$ m/s (Chosen; the handover approach while the coworker holds the `accept_item` grant that section 14.2 defines).
- $F_h = 50$ N (Chosen; the handover force criterion).

Math: For a free hand, $m_{\text{red}} = (1/0.6 + 1/12)^{-1} = 0.571$ kg, and with $k_H = 75{,}000$ N/m the stiffness term is $\sqrt{k_H m_{\text{red}}} = 207$ N per m/s. The peak force is $v_{\text{approach}}\sqrt{k_H m_{\text{red}}} = 0.10 \times 207 = 20.7$ N, and the criterion admits $F_h/\sqrt{k_H m_{\text{red}}} = 50 \,/\, 207$, or 0.24 m/s. A clamped hand cannot recoil, so m_{red} becomes m_{eff} and $\sqrt{k_H m_{\text{eff}}} = 949$ N per m/s. The same approach then delivers $0.10 \times 949 = 94.9$ N, and the criterion admits only $50 \,/\, 949$, or 0.053 m/s.

Result: The free hand sees 20.7 N, inside the criterion; the clamped hand sees 94.9 N, which fails it.

Systems insight: Clamping multiplies the force at any speed by $\sqrt{m_{\text{eff}}/m_{\text{red}}} = 4.6$, because the whole arm-side mass, not the light hand, sets the collision. A speed ceiling derived for a free hand is therefore a geometric premise as much as a kinematic one.

The handover channel is therefore a transient-contact ceiling. It holds only while the handover geometry excludes clamping, with the handover point in free space and no rack upright, tote edge, or link of the arm within reach of the hand along the approach. Both contacts end the same way. The arithmetic is exact; what it cannot supply is the premise it rests on, whether an achievable arm deceleration, a contact stiffness, or a clamp-free geometry. The enforcement record keeps each ceiling beside that premise, and the same discipline governs every barrier in this chapter.



At one handover speed a free hand passes the force criterion and a clamped hand fails it.

12.7.3 The conditions forward invariance rests on

Unmodeled friction loss, actuator saturation, solver overruns, or inter-sample drift under a held torque command can each push a physical plant across a barrier boundary even when the continuous-time inequality is satisfied at the sample instant. The containment claim provided by both spatial and energetic barrier calculations is therefore strictly conditional. A forward-invariance claim requires a feasible barrier and the following bounded conditions; the list is necessary for this design, not an exhaustive if-and-only-if test:

1. **Model Fidelity:** The nominal plant dynamics $\mathbf{f}(\mathbf{x})$ and $\mathbf{g}(\mathbf{x})$ must capture the true system dynamics within a validated parameter error bound.
2. **Disturbance Bounds:** External forces and unmodeled friction acting on the mass must remain strictly within the modeled disturbance envelope $|d(t)| \leq d_{\max}$.
3. **Actuation Authority:** Physical actuators must possess sufficient voltage, current, and thermal headroom to realize the commanded corrective force within a qualified delivered-effort error and response-delay envelope, as well as without saturation ($u^* \in [u_{\min}, u_{\max}]$).
4. **Solver Determinism:** The optimization solver or projection algorithm must terminate strictly within its allocated execution budget ($T_{\text{solve}} \leq T_{\text{budget}} < T_{\text{tick}}$).
5. **Discretization Invariance:** The discrete sampling interval T_{tick} must be sufficiently small that inter-sample state drift does not breach the continuous-time invariance boundary.

The approach-acceleration upper bound is a separate premise from the lower bound on available braking, and equality of their illustrative magnitudes does not identify their roles. Each of the five conditions maps to a dedicated engineering budget: model fidelity and disturbance bounds come from system identification and bench load testing, actuation authority from dynamometer curves and thermal limits, solver determinism from static worst-case execution time analysis on the MCU, and discretization from the loop period itself. Because the drive holds the last admitted command between samples (a zero-order hold[13]), the plant drifts for one full period before the next check can act on it. The drift covers at most $\Delta d_{\text{ZOH}} = vT_{\text{tick}} + \frac{1}{2}a_{\max}T_{\text{tick}}^2$ at speed v, where the worst case is the highest speed the base can reach rather than the ceiling $v_{\max}$ that admission enforces. At the 1.5 m/s

[13] **Zero-Order Hold**: The order counts the derivatives the hold carries forward. A zero-order hold keeps the command's value constant until the next sample, while a first-order hold would also extrapolate its slope. For one control tick the plant therefore integrates a torque the controller can no longer change, which is the source of the drift buffer $vT_{\text{tick}} + \frac{1}{2}a_{\max}T_{\text{tick}}^2$; section B.5.5 develops the hold's delay.

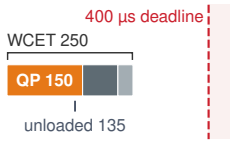
drive limit, an independently stipulated 2 m/s² upper bound on approach acceleration, and $T_{\text{tick}} = 1$ ms, that drift is 1.501 mm, a small fraction of δ_{margin}. Tightening a raw-clearance check by this displacement preserves geometric separation over the hold interval under the stated motion bounds, whereas a kinetic or energy barrier requires a bound on its own evolution throughout the interval.

A condition that no detector watches is an assumption, so the permission path attaches one to each. A **disturbance observer** compares predicted with measured acceleration on every cycle. Inverter status registers and current shunts report actuator saturation. A hardware watchdog and cycle counter time the solver against its deadline. Timestamp counters on the sensor bus catch jitter and missed samples. When a detector invalidates a premise, the enforcer cannot restore permission by substituting optimistic parameters. It selects a response whose sensing, effort, and clearance premises survive, as developed in section 12.9. Both the free-space projection and contact decision must finish before the physical margin reserved for computation is spent.

12.7.4 Execution timing in the sub-millisecond loop

A correct barrier calculation delivered after the permission deadline cannot protect the margin reserved for that deadline. The whole cycle must therefore have a bounded execution time (Wilhelm et al. 2008), including state acquisition and the final drive transfer. A real-time operating system can reduce scheduling delays, but its name does not establish a bound on shared-memory or peripheral interference. The permission path's implementation must be qualified under the competing workloads that can delay it, so the declared cycle allocation below is a timing requirement carried into loaded placement tests rather than a measured trace or a platform-independent solver guarantee.

In this heterogeneous design, the shield executes inside a 1 kHz timer interrupt routine on the MCU. An unloaded cycle of acquisition, validation, and a typical active-set solve completes in about 135 µs, an illustrative estimate rather than a bound. The candidate path instead declares a worst-case execution-time requirement of 250 µs for the whole cycle, divided among the stages below, and the whole sequence must finish inside the 400 µs deadline:



The QP solve takes 150 of the 250 µs declared cycle, inside a 400 µs deadline.

1. **Interrupt Entry & State Acquisition (30 µs):** The hardware timer triggers the interrupt service routine (ISR), and firmware latches the wheel and joint encoder registers and the latest inertial measurement unit (IMU) sample.
2. **Proposal & Lease Verification (15 µs):** The MCU makes one bounded read of its local mailbox, which the link receiver fills outside this routine, validates the checksum, and confirms sequence order before checking evidence age, the active chunk lease, and the host heartbeat timer. In the evidence-age check, the evidence's age at dispatch is the difference between the source frame's evidence timestamp and the host dispatch timestamp in the proposal header (section 4.4.1). That age may not exceed 51.8 ms, which combines the 51.6 ms observation age charged in row 5 of table 12.1 with 0.2 ms of clock uncertainty (section 8.8). Passing that test qualifies the evidence only at dispatch, so the MCU also reconstructs the evidence's current age from its source epoch and elapsed time and checks it against the evidence's own validity deadline and the remaining physical margin. Inference, admission, waiting, and transfer after dispatch must already be reserved once in that margin through the dispatch-to-admission allocation inside the timing allowance. Otherwise fresh evidence is required before activation, because a new proposal timestamp cannot renew old evidence. The MCU then verifies that elapsed time since valid admission or renewal has not exceeded the 60 ms active chunk lease and reads the host heartbeat timer, which marks the host silent once no heartbeat has arrived within the 30 ms timeout.
3. **Tracking Contract Verification (10 µs):** Firmware evaluates instantaneous tracking error norm $\|e(t)\| = \|\mathbf{p}(t) - \mathbf{r}(t)\|$ against the qualified bound ϵ_{track}.
4. **CBF-QP Safety Shield Solve (150 µs):** An active-set QP solver[14] running in pre-allocated static SRAM, the MCU's tightly coupled data memory, evaluates active barrier inequalities and computes $\mathbf{u}^* = \arg\min_{\mathbf{u}\in\mathbb{R}^m} \frac{1}{2}\|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2$. A fixed iteration cap limits the work count; meeting the allocation also requires bounded per-iteration arithmetic, memory access, interrupt interference, and output service, including ill-conditioned constraint geometry. Dynamic memory allocation is forbidden on this path so that heap service is excluded from that timing obligation. In this two-processor design no memory is shared with the host, so camera direct memory access (DMA) streams cannot reach the memory this cycle uses. The couplings that

[14] **Active-Set QP**: An active-set solver guesses which constraints hold with equality, solves the resulting linear system, and adds or drops one constraint per iteration. Dual variants such as that of Goldfarb and Idnani (1983) start from the unconstrained optimum, here the nominal command itself, and add only violated constraints, so a proposal that already satisfies every barrier costs almost nothing. The iteration count is bounded by the combinations of the constraint set rather than by a clock, so a fixed iteration cap bounds steps while the time per step must still be measured under interference.

remain are the link receiver, whose DMA writes into the MCU's own memory, and any shared supply rail or clock; section 13.8 measures that residual interference and tests whether the isolation holds under load.

5. **Output Selection & Admitted Command Staging (15 µs):** If the QP is feasible and constraints hold, the enforcer emits the **admitted command**. The drives close the velocity and current loops. Because industrial servo drives running in position mode choose torque direction internally from position tracking error while an unsigned torque limit clips only the magnitude of each joint's effort symmetrically, a position setpoint plus an unsigned torque limit does not by itself realize the signed braking effort or coupled multi-axis half-space the QP requires. This interface needs a qualified mapping from the staged command to delivered effort, with bounded realization error and response delay. The filter tightens each commanded half-space by the largest possible projection of that effort error along its normal, so every effort the drive may realize still satisfies the delivered-effort constraint. Near full braking, that allowance may remove feasibility and require earlier intervention. The drive configuration and qualification evidence must establish these bounds; this example supplies no measured error or delay. During the response interval, a previously held command must remain valid and the intersample/response allowance must cover its effect, charging each delay once. If the position/limit interface cannot satisfy this contract, the design needs a qualified effort interface or a stricter qualified operating envelope and fallback. The next position setpoint and per-axis torque limit travel in the next EtherCAT frame only under that realization contract. Drive-reported torque on the following tick can detect a breach but cannot retroactively certify the preceding interval; an invalid replacement leaves previously admitted setpoints in force only while their own evidence, lease, state, and stopping reservation remain valid.
6. **Completion & Reserve (30 µs):** The MCU checks the register write and services its own watchdog only after the cycle succeeds. The reserve absorbs variation in the stages above; a cycle that has not completed by the deadline requires a qualified preempting timer or an independent mechanism to select the resident fallback before the next tick. The overrunning ISR cannot be assumed to detect and recover from its own stall. Only with that bounded transfer does a missed tick cost the base 1.3 mm of travel at the aisle speed. Section 13.7 tests whether the declared WCET holds under load, so that ticks are not missed.

Stage 5 is also where refusal takes effect. Any of several conditions triggers it. The chunk lease may have expired, the host heartbeat may have fallen silent, the shared drive bus may report undervoltage, tracking error $\|e(t)\|$ may exceed ϵ_{track}, actuator saturation may render the feasible control set empty ($\mathcal{U}_{\text{safe}} = \emptyset$), or an operating-contract detector such as the disturbance observer or the solver deadline may trip. In any of these cases, the output selector withholds the admitted command, revokes policy authority, and selects the state-matched stop. At standstill, lease or heartbeat expiry selects HOLD, the powered standstill hold that section 12.9 defines, only while powered support remains available. Undervoltage instead requires a qualified transition to mechanical restraint before that support expires. In motion, STOP under undervoltage remains conditional on drive survival.

↪ The schedulability analysis that a declared WCET and deadline feed is developed in section C.3.

Checkpoint 12.2: Barrier, projection, and cycle budget

The barrier fixes which commands preserve the stop; the filter and its cycle decide whether the chosen command arrives in time.

- ☐ **Relative degree**: Why does a barrier on raw distance give torque no control authority at the boundary, and what does the kinetic barrier of equation 12.5 add to restore it?
- ☐ **Projection vs. clamp**: Why can clamping each joint separately leave a coupled half-space violated, while equation 12.8 satisfies it with the smallest torque change?
- ☐ **Projection estimate**: Treat the latch approach as a one-dimensional speed command whose half-space is the 0.03 m/s target-ODD ceiling. What are λ^* and the admitted speed when the chunk policy proposes 0.10 m/s?

- ☐ **Timing premise**: Which of the five forward-invariance conditions do the declared 250 µs WCET and the 400 µs deadline defend, and what must select the fallback when a cycle overruns?

Every stage of this cycle assumes that the solve returns a feasible command. When saturation empties the admissible torque set, the lease lapses, or the cycle overruns its deadline, projection has nothing to return, and the permission path must already hold a response it can execute without the proposer. Executing that check deterministically while surviving proposer faults requires placing the enforcer where the host processor cannot preempt or bypass it.

12.8 Admission, Cyclic Checking, and Monitor Independence

A planned waypoint can clear a warehouse rack upright in the planner's geometric check while the physical manipulator overshoots into the steel during execution, and an instantaneous motor torque well within electrical limits can accelerate the arm past the point where its remaining stopping reservation can arrest motion. Relying on trajectory admission alone misses in-flight tracking overshoot and saturation, whereas checking only single-step torque commands without a verified stopping continuation lets the arm enter an inevitable collision state before the local filter reacts. Robust safety enforcement therefore demands both layers (the per-tick cycle of section 12.7.4 on a dedicated MCU and full-plan admission before motion begins). That raises three structural questions: why neither layer can replace the other, where the cyclic checker sits relative to the command stream, and how independent of the proposer the MCU must actually be.

Full-plan admission judges the candidate trajectory, its seams, its swept clearance, and its verified stopping continuation before the machine commits to motion. The expensive geometry behind that judgment (queries of global occupancy octrees, signed distance fields, or mesh collision models) cannot run inside a 1 ms control interrupt merely because the MCU owns final actuation privilege. It therefore runs on the host as a separately scheduled, bounded job, and its result reaches the MCU as the trajectory record. The MCU's own admission check then verifies that record against bounded, qualified local evidence (the resident stop suffix, its entry state, and the commitment deadline) without traversing the global map. The hard real-time cyclic path evaluates current joint state, remaining lease validity, and available actuator effort on every tick before selecting and dispatching the next physical command.

An in-line checker places its worst-case execution time directly inside the control-loop deadline, consuming a portion of the available compute tick. A parallel monitor instead operates asynchronously alongside the proposal pipeline, avoiding added forward latency during nominal execution but contributing detection latency and trip delay to the physical response budget. Either architectural placement must ensure privilege isolation. The untrusted proposer must never be able to bypass the selected output, and the system must preserve the dedicated sensing and actuation channels required by the fallback controller. As section 13.2 will show from the resources each candidate placement shares, isolating these execution domains across memory spaces or heterogeneous processing cores is what allows the safety monitor and its fallback mechanisms to survive the very proposer faults that trigger an intervention. The physical stopping budget supplies the non-negotiable temporal and spatial boundaries that this hardware split must satisfy.

> Systems Perspective 12.3: The monitor independence rule
>
> Any valid safety claim must explicitly define which proposer failure modes its sensing, compute, and actuation pathways survive. Placing a monitor on a separate physical core or processor does not establish true independence if that monitor shares a power rail, a memory bus, a sensor link, or an unisolated clock domain with the untrusted proposer. Shared hardware resources require provably bounded interference or a qualified, survivable fallback state. Any environmental condition that the monitor cannot observe directly must remain an explicit, documented operating premise rather than an unstated assumption.

The running example assumes qualified drive, scanner, and arm safety functions and a rated lease/heartbeat watchdog; the release case must name the scoped evidence supporting those functions. Independence, however, is not certification. The runtime permission path is not the machine's certified functional safety layer, and beneath it sit the certified safety functions embedded within motor drives, safety laser scanners, and the dedicated safety controller. These hardware-interlocked mechanisms carry the formal integrity claims that certification bodies audit against industrial standards. The primary operational duty of the software permission path is to regulate the learned proposer proactively so that the robot never trips those certified hardware interlocks during normal operation. Section 12.9.3 maps each dynamic response developed throughout this chapter onto that underlying certified safety floor once the complete response hierarchy is established, and names the one narrow MCU function that does carry a rating.

12.8.1 Where the thesis bends

When a learned locomotion or manipulation policy emits raw joint targets or motor torques at the control rate instead of handing the permission path a multi-step trajectory, the enforcer loses the ability to verify a collision-free stopping continuation before motion begins. It must then either run an active fallback model on every tick or watch its projections induce distributional shift and boundary chattering. Admission and the cyclic check in this chapter both assume that the proposer hands the permission path a plan it can judge before motion: kinematic intent lives in the learned policy, and deterministic motor drives close the torque loops. Much of contemporary robot learning practice abandons this classical division, outputting joint targets at the control rate above stiff proportional-derivative loops (J. Lee et al. 2020) or emitting motor torques end-to-end. The third law, proposal is not permission (principle 3), does not disappear in such direct control designs. Instead, the range of physical guarantees that the runtime permission path can uphold narrows in precise, quantifiable ways.

Four guarantees survive because each acts directly on raw commands at the causal boundary, without requiring a forward plan. A per-tick limit on rate and amplitude bounds torque slew and joint velocity at the hardware register interface. The kinetic barrier of equation 12.6 operates directly in effort space, so a CBF-QP can filter the policy's output without converting it to an intermediate kinematic setpoint. The passivity-based energy tank of section 12.7.2 monitors power exchange and bounds the physical work the policy can inject into the environment during contact. Beneath all of these, low-level geofencing, safely-limited speed monitoring, and torque removal backed by mechanical spring brakes remain permanently active under the software stack. Communication timing improves along one dimension. If a qualified freshness deadline is selected at one tick and an independent watchdog enforces it, policy silence is detected within that tick plus bus transport delay, which reduces the lease-expiration travel row of table 12.1 from 80.6 mm to 2.6 mm at the aisle speed.

What is lost is admission. Without an explicit multi-step trajectory, the system cannot verify the collision-free stopping suffix before the robot commits to motion. The enforcer must instead evaluate the machine's dynamic stopping reservation on every tick against an active fallback controller that maintains its own internal braking model. On a wheeled mobile base, that fallback is straightforward to formulate. The controller executes a qualified service-braking response along the current heading, followed by wheel restraint after confirmed standstill. On an underactuated balancing machine, such as a bipedal humanoid, the fallback is itself an active balance controller; cutting motor torque produces an immediate uncontrolled fall rather than a safe state.

Runtime projection also alters the closed sensorimotor loop under which the policy was trained. Every time the enforcer projects a proposed action onto the admissible set, it delivers a state to the environment that the policy never intended, precipitating distributional shift in subsequent observations. This penalty surfaces directly in task performance, as quantified by the intervention metrics of section 12.7. Frequent overrides cause the machine to chatter erratically along safety boundaries or freeze conservatively short of its goal. To reconcile this tension, modern architectures adopt filter-aware policy training, incorporating the safety filter directly into the training objective so that the neural network learns to avoid the boundary rather than encountering the enforcer for the first time on physical hardware (Hsu et al. 2024). The core thesis still holds at the causal boundary, where untrusted proposals never bypass runtime enforcement. What bends is how much of the

safety argument can be settled mathematically before motion begins vs. how much performance the machine must sacrifice to settle the remainder while moving.

12.9 The Fallback Ladder

A safety filter must select its fallback before the admissible control set becomes empty. Once the chosen filter has an empty feasible intersection, it cannot maintain its modeled invariant. A qualified alternate response may mitigate damage or support its own narrower containment claim, but selecting it does not restore the lost guarantee. Furthermore, physical systems cannot rely on a single, binary kill switch. Abruptly severing DC bus power whenever a control proposal approaches a barrier boundary induces uncontrolled deceleration, mechanical shock, and incessant false-positive mission aborts. Conversely, complex software optimization is futile during a hardware fault such as an inverter shoot-through, because the fault destroys gate-driver field-effect transistors (FETs) milliseconds before an interrupt handler can even dispatch.

To resolve these competing physical realities, the permission path maintains a deterministic ladder of resident protective responses. The runtime arbiter evaluates plant kinematics, remaining sensor validity, and hardware integrity on every control cycle, selecting deterministically among minimally invasive projection, closed-loop position holding, controlled deceleration, or complete torque removal. The name records an ordering by how much control authority each response retains, from a corrected proposal down to unpowered restraint, not a sequence to descend. The arbiter never climbs down the rungs one at a time; it jumps directly to the one whose prerequisites still hold.

Definition 12.3: Dynamic fallback ladder

Dynamic fallback ladder is the set of resident protective responses ($\mathbf{u} \in \{\mathcal{P}_{\mathcal{C}}(\mathbf{u}_{\text{nom}}), \mathbf{u}_{\text{hold}}, \mathbf{u}_{\text{stop}}, \mathbf{u}_{\text{cutoff}}\}$). A deterministic arbiter selects one of them on every tick according to the check that failed (an invariant violation, a timing overrun, or a hardware fault) and the measured state of the plant. One trigger can therefore select different rungs in motion and at standstill.

1. **Significance**: Physical AI systems cannot treat all faults identically. Cutting actuator power on dynamic, underactuated, or high-momentum plants (for example, legged robots mid-stride or multirotors in flight) precipitates immediate catastrophic falls or tumbling. These platforms require state-matched active deceleration before executing terminal torque removal.
2. **Distinction**: Unlike software exception handlers that unwind the call stack or terminate a user-space thread, a dynamic fallback ladder operates against physical momentum. The arbiter matches each rung to the remaining valid sensor channels, surviving control authority, and available stopping clearance.
3. **Common pitfall**: Treating the rungs as an escalation sequence where lower rungs, such as safe torque off (STO), inherently provide faster or safer arrest. Each rung possesses distinct entry prerequisites; invoking an unpowered torque inhibit on an airborne drone or an unbraked mobile base allows the machine to coast freely without arresting motion.

12.9.1 The four rungs of the fallback ladder

Walking down protective responses one step at a time like a software exception cascade fails on physical hardware: commanding a standstill position hold while the base is rolling at 1.3 m/s saturates the drives, while cutting drive torque before active braking completes lets the vehicle coast into the obstacle. The dynamic fallback architecture therefore organizes protection across four resident responses (`PROJECT`, `HOLD`, `STOP`, and `INHIBIT`). The deterministic arbiter samples measured plant state (velocity, position, payload loading) and fault severity on every millisecond tick to dispatch the response whose compute, sensing, and actuation prerequisites still hold:

1. `PROJECT`, minimal-intervention projection in MCU firmware. The primary safety filter executes on every control cycle while the feasible control set is nonempty ($\mathcal{U}_{\text{safe}} \neq \emptyset$), the proposal

lease is fresh, and tracking error remains within bounds ($\|e(t)\| \leq \epsilon_{\text{track}}$). It computes the minimal-intervention torque $\mathbf{u}^* = \arg\min_{\mathbf{u}} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2$ subject to active barrier half-spaces within its 150 µs cycle allocation. It makes the smallest feasible effort correction under the stated invariance assumptions; the planner must still account for any changed task-space path.

2. `HOLD`, powered hold at standstill in MCU firmware. When the machine is stationary, the arbiter selects an energized position hold, analogous to an IEC 60204-1 Category 2 stop: safe stop 2 (SS2) followed by safe operating stop (SOS), the drive function that monitors standstill.[15] Its protective validity stems from that certified standstill monitor rather than the position loop gain. Because IEC 60204-1 permits only Category 0 or Category 1 for an emergency stop, HOLD can never serve as an E-stop response (International Electrotechnical Commission 2016a). The drives maintain coordinate $\mathbf{x}_{\text{hold}}$ under closed-loop position control, modulating inverter phase currents against gravity and static loads. While this rung incurs continuous Joule heating in motor windings, it eliminates friction brake wear and preserves kinematic calibration, enabling instantaneous task resumption when a valid trajectory proposal arrives. On the warehouse mobile manipulator, this rung handles lapsed leases or lost host heartbeats specifically when the platform is already at standstill.
3. `STOP`, controlled dynamic stop in MCU firmware. When a machine in active transit loses proposal authority, tracking error diverges, or an obstacle breaches the stopping envelope, the response is a resident, state-matched controlled stop, analogous to an IEC 60204-1 Category 1 stop (safe stop 1, SS1). Its certified counterpart is the ramp-monitored variant (SS1-r), which checks deceleration against a bounded profile throughout the arrest; the unmonitored, time-controlled variant (SS1-t) merely removes torque after a fixed delay without verifying that the planned ramp actually decelerated the load (International Electrotechnical Commission 2016b). The resident jerk-limited S-curve stop suffix of section 12.3 serves this role provided the entry state falls within its validated domain. Executed autonomously by bare-metal MCU firmware directly from local SRAM, this stop peaks at the credible deceleration a_{brake}, returning kinetic energy through motor inverters to the DC bus for absorption by the qualified regeneration path. Power removal and mechanical brake clamping occur only after confirmed zero velocity.
4. `INHIBIT`, torque inhibit and spring-brake clamping by hardware interlocks. When the safety microcontroller suffers a brownout, clock loss, or watchdog timeout, all software decision logic ceases. Dedicated, out-of-band hardware interlocks trigger STO to de-energize gate drive power supplies and drop hold current to electromechanical spring-applied brakes. This terminal rung removes motor torque without software intervention; the mechanical brakes provide only their independently qualified holding or dynamic stopping capacity.

[15] **Machine Stop Categories**: IEC 60204-1 distinguishes uncontrolled power removal (Category 0), controlled stopping followed by power removal (Category 1), and controlled stopping with power retained (Category 2). A category defines electrical power status rather than stopping distance; every rung therefore requires its own empirically validated stopping measurement.

When a machine in motion loses proposal authority or violates a tracking bound, the arbiter engages a controlled dynamic stop, operating analogously to an IEC 60204-1 Category 1 stop. On the warehouse mobile manipulator base, this rung executes the resident stop suffix pre-admitted alongside the motion trajectory, rather than attempting to synthesize a stopping trajectory on the fly during a fault. In steady-state cruising with zero acceleration, this pre-computed profile matches the operating state at fault onset directly; nonzero acceleration states require their own qualified state-matched suffixes. From an aisle cruise velocity of 1.3 m/s, the jerk-limited S-curve suffix sustains a credible peak deceleration of 2 m/s² between its jerk transitions. The entire maneuver spans 931.25 ms and traverses 605.31 mm, the stop-suffix travel the stopping budget of section 12.3 reserves at this speed. Because the deceleration phase channels kinetic energy back through the inverter into the DC bus, bus capacitance and braking choppers must absorb this regenerative surge without triggering a bus overvoltage fault. Complete electrical power removal, if mandated by protocol, occurs only after sensors verify standstill.

Beyond firmware-driven stops, a separate out-of-band circuit can inhibit motor torque generation entirely. Drive STO cuts gate-drive bias to isolate inverter FETs, but it does not disconnect the DC bus, energize mechanical friction brakes, or actively decelerate a moving rotor; an unbraked motor will simply coast under its existing momentum. In robust architectures, an external safety controller simultaneously drives a separately rated contactor and spring-applied mechanical brakes. Engineers must evaluate the dynamic stopping distance of this combined pathway as an integrated system, because dropping drive torque while an arm is under gravitational load or while a vehicle is at speed

can exacerbate the hazard. For the warehouse mobile manipulator, this pathway, STO combined with nine spring brakes that engage between 30 ms and 80 ms after coil flux decay, constitutes the unbudgeted INHIBIT rung. It serves exclusively as a hardware backstop for MCU cycle failure, and the primary stopping envelope claims do not rely on it.

An undervoltage fault on the shared drive bus can invoke the STOP rung only while inverter gate drives, angular encoders, and control logic retain adequate voltage for closed-loop deceleration. The isolated permission rail maintains power to logic circuits, transceivers, and brake coils for 2 s, but it does not supply the motor currents required for braking torque. Consequently, timing guarantees for powered stopping assume a surviving drive bus. Under this condition, dynamic braking begins within 22 ms of the fault flag; the hold-up hypothetical uses the pre-dispatch inspected-floor ceiling of 1.23 m/s, rather than an admitted operating configuration, and its stop requires 1211.4 ms. Mechanical brake engagement at standstill adds at most 80 ms. This entire sequence completes within 1313.43 ms, preserving a 686.57 ms buffer of control logic hold-up.

↩ The isolated permission rail and its capacitive hold-up are established in section 2.8.2.

This analysis confirms the control-supply duration budget, whereas the electrical drive-energy budget requires separate qualification. Systems engineers must prove through empirical testing that the DC bus capacitance, inverter power stages, and regenerative dump resistors sustain controlled deceleration under worst-case electrical fault conditions. If the drive stage cannot guarantee this power integrity, the system must either adopt an unpowered dynamic braking configuration or restrict its operational safety envelope; INHIBIT cannot be credited with standard Category 1 stopping performance. Similarly, at standstill, an undervoltage fault must trigger verified mechanical brake engagement before the logic supply decays, rather than attempting to maintain an active HOLD indefinitely on finite capacitance.

Physical AI systems engineers must distinguish between **fail-safe** and **fail-operational** plants when configuring terminal fallbacks. In a machine already at a mechanically restrained standstill, removing drive torque is safe once mechanical engagement is verified. On underactuated, open-loop unstable, or high-momentum platforms, however, abruptly removing actuator torque directly initiates a catastrophic failure mode:

1. **Aerial platforms (multirotors and eVTOLs)**: Cutting motor torque drops aerodynamic thrust to zero instantly, transforming an airborne vehicle into an uncontrolled ballistic projectile.
2. **Dynamic legged systems (bipedal humanoids)**: During dynamic locomotion, the robot's center of mass frequently falls outside the support polygon; stability depends on active ground-reaction forces and momentum exchange. Suddenly removing joint torques causes an unrecoverable fall, with impact severity governed by gait phase and available compliance.
3. **High-speed autonomous vehicles**: De-energizing steering actuators or electric traction motors at highway speeds strips away the control authority required to execute a coordinated deceleration, leaving the vehicle at the mercy of passive road friction.

The physical channel that dispatches a hardware torque-removal request is the out-of-band interlock that section 4.6 traced (figure 4.8). In it, dual linked emergency-stop contacts, two force-guided relays whose external device monitoring (EDM) contacts block a restart after a weld, and two STO inputs on the drive leave no single fault able to defeat the trip. What the INHIBIT rung needs from that circuit is its third stage, drawn again here with the brake in its own loop (figure 12.9), because removing drive torque ($\tau = 0$) and arresting physical momentum are distinct functions that need separate validation:

1. **Torque removal (Stage 3, drive side):** De-energizing the safety relays drops the drive's STO inputs, which de-energize the gate drive power supply for the three-phase inverter bridge, rendering the power MOSFETs or insulated-gate bipolar transistors (IGBTs) high-impedance, so the motor can no longer produce electromagnetic torque. STO alone does not apply mechanical braking. An unbraked motor carries momentum and may coast freely.
2. **Mechanical restraint (Stage 3, brake side):** A separate coil circuit powers the spring-applied electromechanical friction brake. When power is removed, coil magnetic flux decays, allowing internal springs to physically clamp friction pads against the motor shaft, applying the brake without electrical power. A holding-brake rating does not establish the ability to arrest a moving load; dynamic stopping requires separate energy, torque, and response qualification.

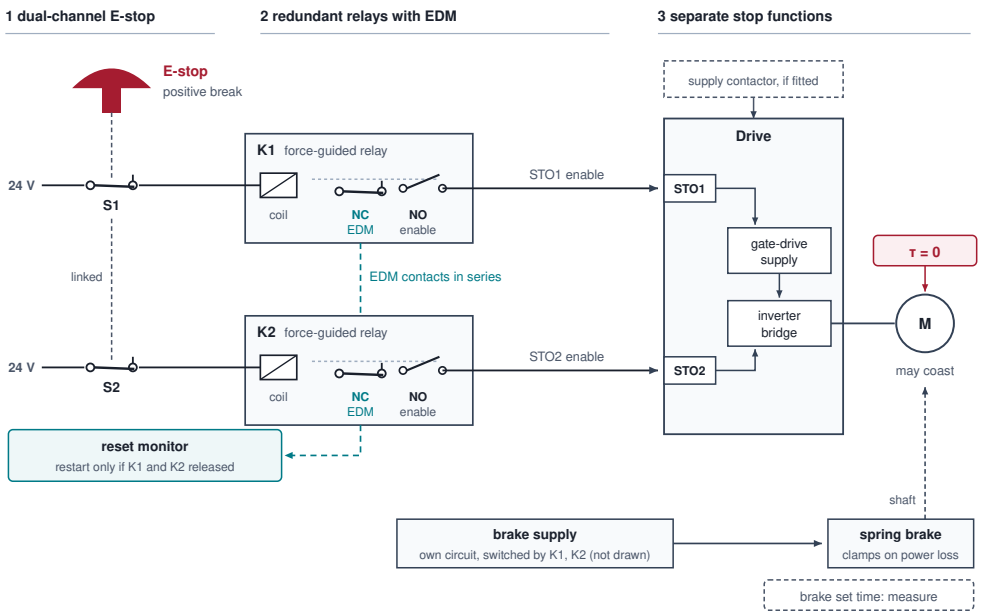


Figure 12.9: Illustrative emergency-stop circuit: Stages 1 and 2 are the out-of-band interlock circuit (Chapter 4); Stage 3 maps to the numbered list above. Two linked contacts each hold a force-guided relay, the relays' enable contacts hold the drive's two Safe Torque Off (STO) inputs, and their EDM contacts in series block a restart after a weld. The spring-applied brake runs on its own supply, and a supply contactor, if fitted, provides galvanic disconnection. A moving motor can coast after STO, so the brake's set time and the physical stopping distance must be measured for the load and state.

That separation between software control and hardwired, out-of-band trip circuits is not a stylistic preference. When the processor running the control loop stalls or corrupts its own state, only a physically independent path can still remove drive enable and clamp the brakes. The precedent that the lineage table of section 1.10 attaches to the third law shows the cost of removing that hardware path and trusting software logic to gate a high-energy actuator on its own (war story 12.1).

War Story 12.1: Therac-25 software interlock elimination

Context: Between 1985 and 1987, the Therac-25 medical linear accelerator administered massive radiation overdoses to six cancer patients after system designers eliminated the electromechanical interlocks used in previous models, assigning safety-critical beam containment entirely to software (Leveson and Turner 1993).

Mechanism: Software race conditions and shared memory corruption in the operator interface allowed the machine to fire its high-current electron beam (25 MeV) without rotating the tungsten target and beam-flattening filter into position. Because the physical microswitches that previously interlocked beam activation had been removed, the underlying hardware possessed no independent means to detect that the operational mode conflicted with the turntable state.

Impact: Six patients received radiation doses estimated between 4,000 and 20,000 rads (up to 100 times the therapeutic level), causing severe tissue necrosis and three direct fatalities (Leveson and Turner 1993).

Response: Investigators mandated the restoration of independent hardware interlocks, requiring positive electromechanical verification of turntable configuration prior to beam generation, alongside formal software safety lifecycle standards.

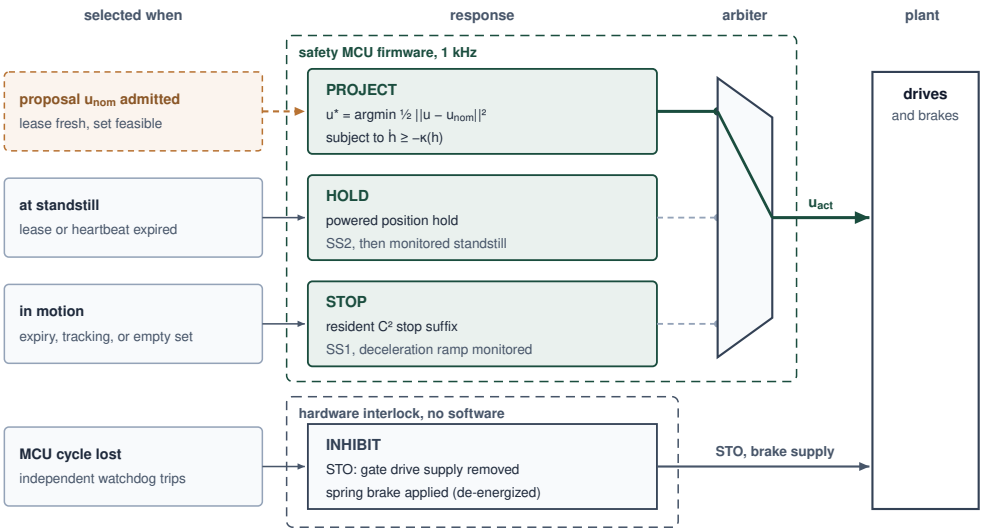


Figure 12.10: Fallback responses and the paths that run them: The plant state and the failed check select one response directly; the four are alternatives, not steps of an escalation. PROJECT, HOLD, and STOP run in the safety MCU's firmware and reach the drives through its arbiter, which here passes the projected command. INHIBIT is the one response that survives loss of the MCU itself, so it bypasses the arbiter on a hardware path that removes gate-drive supply and lets the spring brakes set. On a moving machine that path also removes motor braking, and the machine coasts until the separately qualified brakes take hold.

Systems lesson: A software-based protection layer cannot serve as its own independent backstop. A runtime barrier filter may exercise binding authority over nominal control proposals, but the architecture still requires an out-of-band, physically independent trip mechanism to survive processor lockups, memory corruption, and firmware bugs. Moreover, because torque removal alone does not guarantee deceleration, the surviving physical response must match the plant's dynamic envelope.

The four rungs run on two paths (figure 12.10). `PROJECT`, `HOLD`, and `STOP` run in safety MCU firmware at the 1 kHz tick and reach the drives through one arbiter, which selects a single actuation command $\mathbf{u}_{\text{act}}$; `INHIBIT` bypasses the firmware on a hardware path that removes gate-drive supply (STO) and drops spring-brake holding current when the processor itself fails. The rungs trade response latency, control authority, and recovery cost, and the arbiter selects directly among those whose entry prerequisites still hold. A controlled stop demands verified drive electronics and sufficient stopping clearance; losing drive integrity forces unpowered mitigation with strictly weaker physical guarantees. Selection is also only as sound as the state evidence behind it. The 2023 Cruise pullover, which section 1.8 set beside Tempe as a failure after impact, selected a motion fallback on the strength of a missing track (war story 12.2).

War Story 12.2: Cruise post-impact fallback (2023)

Context: In October 2023, an autonomous Cruise robotaxi in San Francisco encountered a pedestrian who had been struck by an adjacent human-driven vehicle and propelled into the robotaxi's travel lane (Exponent, Inc. 2024).

Mechanism: The autonomous vehicle initiated an immediate emergency stop upon detecting the impact. However, the collision-classification subsystem erroneously localized the pedestrian to the side of the vehicle rather than under the chassis, categorizing the event as a side collision. The operational design domain for side collisions specified a secondary pullover maneuver to clear traffic. Because the perception pipeline had tracked the pedestrian only intermittently

before the collision, the tracker marked the lane ahead as clear, prompting the vehicle to accelerate from standstill into the pullover (Exponent, Inc. 2024).

Impact: Executing the pullover dragged the pedestrian, who remained pinned beneath the vehicle's floorpan, approximately 20 ft (6.1 m) at speeds reaching 7.7 mph (3.4 m/s), causing severe trauma (Quinn Emanuel Urquhart and Sullivan, LLP 2024). The California DMV subsequently suspended Cruise's autonomous testing and deployment permits, prompting Cruise to halt fleet operations nationwide and recall software across 950 vehicles (California Department of Motor Vehicles 2023; Exponent, Inc. 2024).

Response: A post-collision fallback must prohibit renewed motion whenever state uncertainty persists. In this scenario, retaining a locked standstill and mandating physical inspection prevents the system from treating missing perceptual tracks as free space. Teleoperation feeds cannot substitute for verified local sensor coverage.

Systems lesson: An embodied fallback cannot infer free space from the absence of sensor tracks. Every transition across fallback states requires positive evidence verifying that assumed physical invariants hold, paired with failsafe physical restraints while state ambiguity remains unresolved.

12.9.2 Industrial stop categories and the safe torque off trap

On a moving mobile base or a gravity-loaded arm, Safe Torque Off (STO) is not an active brake: removing drive torque immediately leaves the base coasting on wheel momentum and lets the arm drop during the milliseconds before its mechanical friction brakes clamp. Decelerating the mobile manipulator safely instead uses closed-loop motor torque during the active braking phase and engages mechanical holding brakes only once the plant reaches confirmed standstill. Industrial standards distinguish machine-level stop categories from drive-level safety functions across this sequence. IEC 60204-1 classifies whether electrical power remains connected during and after deceleration. Functional safety standards such as IEC 61800-5-2 define the drive functions, including STO, SS1, and SS2, that execute individual stages of a response, and the `HOLD` and `STOP` rungs borrowed their names from them. Table 12.3 maps these formal classifications to their concrete physical obligations.

Table 12.3: Stop functions and physical response: Machine stop categories, related drive functions, and the plant response each leaves to be tested. No stop category alone supplies a safety integrity rating, a brake, galvanic isolation, or a guaranteed stopping distance.

Response	Motor torque and power	Physical stop obligation	Appropriate use
Admitted filtering	Drive remains active	Recheck the feasible kinetic barrier and tracking bound every tick	Nominal operation inside the validated envelope
Controlled hold (Category 2 / SS2-like)	Power remains for braking and hold	Verify stopping distance, thermal headroom, and static load support	Hold when feedback and torque authority remain available
Controlled stop then torque removal (Category 1 / SS1-like)	Power remains through deceleration; torque is removed afterward	Verify the full braking and transition time under load	Moving machine with enough reserved clearance
Torque removal (Category 0 / STO-like)	Drive cannot produce commanded torque; DC bus isolation is a separate function	A moving motor may coast; a separate brake or mechanical restraint needs its own timing and rating	State for which loss of torque has been shown safe

12.9.3 Certified safety functions beneath the permission path

Citing an unrated numerical QP solver or software evidence-age check as the primary hazard control in a safety case fails certification audit immediately, because complex solver code without formal tool qualification and hardware diagnostic coverage cannot carry a safety integrity claim. In an embodied deployment, not all firmware executing the fallback ladder carries formal functional safety certification, so a certification engineer partitions software claims by the integrity of the

underlying hardware substrate. On a production machine, formal guarantees rest primarily on a second, lower layer: dedicated safety functions in the motor drives, the optical safety scanner, and the arm's safety controller, each architected and validated to a Performance Level (PL) under ISO 13849-1 (International Organization for Standardization 2023a). A Performance Level grades a safety function rather than a component, by the architecture category of its channels, their mean time to dangerous failure, their diagnostic coverage, and their defenses against common-cause failure (section 4.2); IEC 61508 grades the same functions by Safety Integrity Level (SIL) (International Electrotechnical Commission 2010a). These hardened functions substantiate the certified protective envelope. Above them, the MCU permission path operates primarily as an unrated performance layer designed to keep the learned policy inside the admissible set, preventing unnecessary trips of the hard safety floor. Only one narrow component of this MCU layer carries a safety rating, the watchdog that revokes authority when the chunk lease expires or the host heartbeat falls silent.

Table 12.4 maps each response and speed channel onto its certified industrial counterpart. The core drive functions derive from IEC 61800-5-2 (International Electrotechnical Commission 2016b): **safe torque off (STO)**, **safe stop 1 (SS1)**, **safe stop 2 (SS2)**, **safe operating stop (SOS)** (which monitors standstill), **safely limited speed (SLS)**, and **safe brake control (SBC)** (which drives the mechanical brake coil through a monitored output). For the mobile base, ISO 3691-4 assigns required performance level (PLr) d to braking system control and personnel detection within the travel path, while assigning PLr b to the electromechanical parking brake (International Organization for Standardization 2023b; TÜV Rheinland 2021). For robotic manipulators, ISO 10218-1:2025 classifies systems into Class I (manipulator mass under 10 kg with strictly limited force and velocity) and Class II, mandating PL d (or SIL 2) across all safety functions on Class II arms (International Organization for Standardization 2025a; Hartmann et al. 2026). Because the warehouse arm weighs 18 kg, it is governed by Class II mandates.

Table 12.4: The permission path above its certified floor: Each response and speed channel of this chapter beside the safety-rated function that carries the certified claim on a production machine, the requirement that governs it, and the performance level that requirement sets for this machine. Only the first row has no certified counterpart; its checks are the performance layer. The second row is the one rated function on the MCU.

Response or channel	Certified counterpart	Governing requirement	Required level on this machine
`PROJECT`, feasibility, evidence-age	None; the admission checks are not claimed as safety rated	None	None claimed; recorded as a premise
Lease expiry and heartbeat loss	MCU watchdog that commands STO or SS1 on the drives	ISO 13849-1; IEC 61508 for the MCU firmware	Set by the risk assessment for the stop it triggers
Base aisle-speed ceiling	SLS on the drive wheels, switched by the scanner's field set	IEC 61800-5-2; speed control under ISO 3691-4	Set by ISO 3691-4 and the risk assessment
Person at the rack end	Scanner protective field that selects `STOP`	ISO 3691-4 person detection; ISO 13855 field sizing	PL d
`STOP` on the base	Ramp-monitored SS1 (SS1-r), then parking brake	IEC 61800-5-2; ISO 3691-4 braking control	PL d for braking control; PL b for parking brake
`HOLD`	SS2 into SOS	IEC 61800-5-2	PL d on the arm; base set by the risk assessment
`INHIBIT`	STO with SBC	IEC 61800-5-2; IEC 60204-1 Category 0	PL d on the arm; base as for braking control
Arm TCP, latch, and handover limits	Safety-rated monitored speed in the arm's safety controller	ISO 10218-1:2025	PL d (Class II)
Handover to the coworker's hand	Speed and separation monitoring or power and force limiting	ISO 10218-2:2025	Set by the application risk assessment

As the first row of table 12.4 indicates, the online CBF-QP, feasibility tests, and sensor evidence-age checks operate without formal safety ratings. Consequently, safety cases cannot cite these complex numerical filters as certified protective measures; instead, the enforcement record treats their correct execution as an operational premise. System architects face two choices when structuring this

boundary. An engineering team can treat all MCU-level admission checks strictly as an unrated performance layer, relying entirely on the certified hardware floor to prevent harm. Alternatively, the team can certify the entire MCU firmware stack to functional safety standards (such as IEC 61508 or ISO 26262), subjecting matrix solvers and barrier algorithms to exhaustive formal methods, tool qualification, and deterministic execution bounds.

The running machine assumes the hybrid path. Its qualified watchdog, the table's second row, independently commands STO or SS1 at the drive level when the chunk lease expires or the host heartbeat falls silent. Its bounded scope reduces the qualification burden, while isolation, failure behavior, timing, and lifecycle evidence remain necessary. Meanwhile, the complex admission checks remain unrated, leaving the certified optical scanner and drive-integrated safety functions to handle primary hazard mitigation. This split architecture demands strict physical and temporal isolation between the rated watchdog and unrated solver routines to eliminate common-cause faults.

↪ How the release case treats the certified watchdog as a rated safety function and the unrated admission checks as empirical premises is developed in section 16.3.4.

The robotic arm introduces collaborative operational modes beyond those required by the mobile base. **Speed and separation monitoring (SSM)** continuously modulates or halts arm velocity to maintain a protective clearance envelope around human workers, while **power and force limiting (PFL)** caps transient and quasi-static contact forces during intentional physical collaboration. Standard ISO 10218-2:2025 incorporates the collaborative requirements previously defined in ISO/TS 15066 (International Organization for Standardization 2025b, 2016). The 2025 standard formalizes SSM within section 5.14.5 and normative Annex L, and specifies PFL limits across anatomical regions in section 5.14.6 and informative Annex M (Hartmann et al. 2026). The protective separation distance is governed by:

$$S_p = S_h + S_r + S_s + C + Z_d + Z_r,$$

where $S_h = v_h(T_r + T_{\text{stop}})$ represents worker displacement during robot reaction latency T_r and stopping duration T_{stop}. Next, $S_r = v_r T_r$ is robot displacement prior to brake onset, and S_s is the physical braking distance. The term C is the penetration distance permitted by sensor resolution. Finally, Z_d and Z_r denote positional uncertainties for the human and robot, respectively (Marvel and Norcross 2017).

Evaluating this expression for the descending arm highlights the relative weight of human vs. robot motion. The ISO 13855 human approach speed is $v_h = 1.6$ m/s (International Organization for Standardization 2024a). The vision-path latency is $T_r = 133.6$ ms, and the tool-center-point velocity is $v_r = 0.96$ m/s. The gravity-biased joint deceleration from section 12.4 completes in $T_{\text{stop}} = 166$ ms across $S_s = 79$ mm. With these values, the human displacement term is $S_h = 479$ mm and the robot reaction displacement is $S_r = 128$ mm. Together, these preliminary dynamic components total 686 mm before accounting for sensor intrusion margin C and spatial uncertainties Z_d and Z_r.

Human movement accounts for 70 percent of this cumulative margin. Just as in the walking-person stopping budget of section 12.3.1, human velocity during the combined reaction and deceleration phases dominates the required clearance. Against an approaching human, total stopping duration imposes a far heavier spatial tax than the robot's physical braking distance alone. Because standard vision pipelines lack functional safety ratings, a certified SSM implementation must use dedicated, safety-qualified sensors to measure separation and bound T_r.

The standards distinguish how they treat obstacle velocity. Standard ISO 3691-4 specifies detection assuming a stationary person standing in the travel path (TÜV Rheinland 2021); consequently, vehicle testing protocols evaluate protective envelopes against the static-obstacle formulation of equation 12.1. If a worker continues walking toward the vehicle throughout the braking sequence, the relative closing velocity exceeds the standard's baseline assumptions. In contrast, SSM explicitly models dynamic human approach velocity (v_h), explaining why mobile base navigation requires an explicit aisle-crossing protocol whereas robotic manipulation requires an active separation or contact-limiting channel. For collaborative handoffs, the enforcement record adopts the standardized biomechanical model of the human hand, distinguishing between unconstrained (transient) and clamped (quasi-static) contact scenarios. While simplified models often approximate PFL as a single lumped elastic collision with a fixed force threshold, ISO 10218-2 specifies distinct permissible force and pressure ceilings across individual anatomical regions. A complete architectural map correlating each subsystem, governing regulatory standard, and validation artifact is synthesized in the release decision framework (table 16.3).

Checkpoint 12.3: Choosing the physical fallback

Before selecting a defensive fallback strategy, review the requirements governing state-dependent stopping modes, safety functions, and timing reserves:

- ☐ **State-dependent stop**: Why can powered braking be safer than STO for a moving autonomous mobile robot (AMR), while drive torque inhibit may be appropriate after a separately verified restraint holds a stationary arm?
- ☐ **Independent functions**: What does STO inhibit, and which separate functions would remove bus power or apply a spring brake?
- ☐ **Timing and clearance**: Which measured detection, dispatch, drive, and brake delays belong in the stop reservation?
- ☐ **Lost viability**: What can a fallback still accomplish when the admissible torque set is empty?

12.10 Runtime Enforcement Failure Modes

Every guarantee in this chapter has rested on premises about the computer that runs the filter and the plant it models: a held-up logic rail, a bounded cycle, a credible deceleration, and a bus that delivers the admitted command on time. On the warehouse mobile manipulator each of those premises already carries a concrete spatial price. A missed tick costs 1.3 mm of travel at the aisle speed. A lease honored by the host instead of the MCU turns a 200 ms planning stall into 182 mm of unbraked travel and a 50.99 mm overrun. An unprotected integrator spends 195 mm against a 131.0 mm spare. Gravity on the arm and mug takes 75 percent of the joint's peak torque and quadruples the descending stop. Runtime enforcement failures trace back to four classes, one for each premise:

1. **Common-cause infrastructure failures**: When the learned policy and the safety filter share power rails, clock sources, reset lines, or thermal paths, one fault can disable both. A current surge during hard acceleration can sag a shared supply until it resets both the neural accelerator and the microcontroller that monitors it, and a shared reset or clock fault can silence the monitor at the instant the policy issues an errant command. The running machine therefore holds its permission logic on an isolated, held-up rail (section 12.9.1).
2. **Timing overruns and execution jitter**: A quadratic program that stalls on an ill-conditioned active set, or a filter scheduled on a non-deterministic operating system, spends braking clearance while it computes. An overrunning routine cannot be trusted to detect its own stall, which is why the declared 250 µs cycle comes with a qualified preempting timer or independent mechanism that selects the resident fallback when a deadline expires (section 12.7.4).
3. **Plant mismatch and unobservable state drift**: The filter evaluates safety against a model. A floor film that lowers friction, a payload that shifts the center of mass, or winding heat that lowers peak torque removes control authority the model still credits, so the solver can admit a deceleration the hardware cannot deliver. Where onboard sensors cannot observe the shift, the safe set must carry qualified uncertainty margins or the operating domain must restrict speed, as the complete optimistic inspected-floor screen falls to 1.17 m/s, before driven-wheel load share and available braking are qualified (section 12.3).
4. **Interconnect contention and transport delays**: State reads and command writes must cross physical buses. Unprioritized telemetry or logging can queue a joint reading until it is stale, or hold a filtered command in a transmit queue while the drive keeps executing the previous high-speed setpoint. The running machine carries its admitted command in a cyclic EtherCAT frame (stage 5 of the cycle budget) and charges the board link to the dispatch-to-admission budget.

None of the four yields to a better solver or an automotive-grade microcontroller. Each is answered by evidence. Common causes and contention need isolation and bounded interference, which chapter 13 tests on silicon. Overruns need a verified execution bound with an independent fallback. Plant mismatch needs worst-case rather than nominal parameter margins. All four need a record of

every enforcement decision, with its solver latency, active constraints, and sensor timestamps, so that an incident can be traced to the policy, the plant, or the infrastructure.

12.11 The Enforcement Record

When a warehouse mobile manipulator halts at a rack end because runtime safety logic rejected a motion proposal, forensic analysis requires unambiguous provenance: which threshold failed, which physical detector measured the violation, and which fallback rung took control of the actuators. The **enforcement record** is the tenth record in the record chain, the point where offline characterization meets online execution. It formalizes runtime safety across three cooperating artifacts:

1. **Enforcement contract**: The signed, immutable specification compiled into microcontroller firmware that fixes qualified tracking bounds ϵ_{track}, barrier coefficients, timer deadlines, delivered-effort error and response-delay bounds, and fallback routing tables before deployment.
2. **Admitted command**: The record's per-tick output. On each millisecond control cycle, the microcontroller gatekeeper generates an admitted command for transmission across the causal boundary. This payload contains the admitted or projected control $\mathbf{u}^*$, the verified barrier margin $h(\mathbf{x})$, tracking error validation $e(t) \leq \epsilon_{\text{track}}$, and the remaining hardware lease. The gatekeeper forwards only the bare setpoint and protective envelope to the motor drives, committing the complete supervisory state to the audit log.
3. **Enforcement audit log**: The high-rate forensic stream that captures per-tick proposal evaluations, intervention vectors $\Delta\mathbf{u} = \mathbf{u}^* - \mathbf{u}_{\text{nom}}$, quadratic programming (QP) solve latencies, and veto events in non-volatile storage.

Together, these artifacts render safety decisions fully auditable without burdening the actuator bus. The contract predefines the admissible operating envelope, the admitted command tags the immediate actuator setpoint, and the audit log preserves the causal chain of evidence. Real-time enforcement depends entirely on the protected command path, deterministic loop timing, and verified physical braking capabilities.

12.11.1 The admitted command schema

If the gatekeeper emits a bare torque vector without a monotonic sequence counter, a digest of the admitted parent trajectory, or the remaining lease duration, a replayed frame or a stalled microcontroller cannot be distinguished from a live command, and a post-incident audit cannot reconstruct which safety margins were checked on that millisecond tick. At each control cycle, the gatekeeper therefore produces an admitted command that establishes runtime authority at the drive interface (table 12.5). Its fields explicitly bind the actuator setpoint to its parent trajectory, verified safety margins, and active fallback state. Because each entry records a decision made under assumed physical conditions, runtime containment still relies on those assumptions matching physical reality.

Table 12.5: Admitted command schema: The per-tick output of the enforcement record, the tenth record in the record chain. Formed on every 1 ms tick by the microcontroller and written to the audit log, identifying the selected command, checked assumptions, and state-matched response. The drives receive only the setpoint, torque limit, and active rung inside a black-channel safety message; drive hardware containment relies on the local safety watchdog rather than the record itself.

Field Name	Type & Units	Physical System Invariant & Validation Rule	Mobile Manipulator Parameter / Value
`token_sequence_id`	`uint64_t` (counter)	Monotonically increasing admitted-command ID on MCU clock	Increments every 1 ms tick
`timestamp_mcu_ns`	`uint64_t` (ns)	Absolute MCU monotonic timestamp of admission	Latch instant at causal boundary
`parent_trajectory_digest`	`uint8_t[32]` (SHA-256)	Cryptographic link to the admitted trajectory record and chunk lease	SHA-256 of active trajectory record
`admitted_control_vector`	`float32[N_act]` (N·m)	Filtered torque $\mathbf{u}^*$ realized only under the qualified effort-error and response-delay contract; nominal if safe, projected if near boundary	Per-axis qualified torque envelope; 87 N·m only for the characterized arm axis

Field Name	Type & Units	Physical System Invariant & Validation Rule	Mobile Manipulator Parameter / Value
`intervention_vector`	`float32[N_act]` (N·m)	Intervention delta $\Delta\mathbf{u} = \mathbf{u}^* - \mathbf{u}_{\text{nom}}$; zero when the admitted command equals the nominal command	Nonzero only during CBF-QP projection
`barrier_margin`	`float32` (normalized)	Dimensionless $h(\mathbf{x})/h_{\text{scale}}$ with positive scale and barrier identity bound in the contract	Must satisfy $h(\mathbf{x}) \geq 0$
`tracking_error`	`float32` (m)	Instantaneous tracking error norm $e(t) = \|\mathbf{p}_{\text{meas}} - \mathbf{r}\|$	Base position error: $e(t) \leq$ 40 mm; arm channels use their own qualified bounds
`active_fallback_rung`	`uint8_t` (enum)	Active execution state: `PROJECT`, `HOLD`, `STOP`, `INHIBIT`	`0x01` (`PROJECT`) during nominal tracking
`lease_remaining_ns`	`uint32_t` (ns)	Remaining validity duration of current chunk lease	Trips `STOP` / `HOLD` upon reaching zero
`crc32_token`	`uint32_t` (checksum)	CRC-32 over the log entry, for audit integrity	Log reader rejects a corrupt entry; the drive-facing setpoint carries its own safety-layer CRC

Constructing the enforcement record synthesizes four upstream records across the offline and runtime evidence strands, where the offline strand establishes physical plant capabilities and empirical policy competence while the runtime strand tracks live execution state. From the runtime strand, the trajectory record of section 11.8 supplies the admitted reference and its precomputed, dynamically feasible stop suffix. From the offline strand, the evaluation record of section 7.8 provides runtime monitor specifications and the empirical coverage-gap catalog that define the active supervisory channels. The policy manifest of section 6.8 defines the declared ODD so that the enforcer's validity region is restricted to well-sampled operating regimes. The limit records of section 2.7 establish empirical braking rates, actuator onset delays, and torque envelopes alongside their operating preconditions. To these upstream parameters, the enforcement record contributes the formal guarantees of the gatekeeping pipeline. These are the qualified tracking bound ϵ_{track} and the stopping metrics ($T_{\text{stop}}, d_{\text{stop}}$, which bound the duration and displacement required to arrest plant momentum). They also include the control barrier function coefficients, the discrete execution period, the per-check thresholds mapped to fallback-ladder rungs, and the dynamic validity boundary.

The `parent_trajectory_digest` binds each admitted command to its originating trajectory and active chunk lease, providing forensic traceability back to the planner's proposal. Digests bind content for integrity checks; sequence counters and monotonic freshness checks reject stale or replayed commands within the protected drive interface. Sender authenticity additionally requires an authenticated channel or message authentication. Similarly, the `lease_remaining_ns` field conveys remaining execution authority, but this parameter guarantees safety only when the drive firmware actively enforces lease expiration by triggering a deterministic physical fallback. Streaming these fields to the audit log, alongside nominal proposals, intervention deltas, barrier margins, tracking errors, and solver latencies, gives engineers the causal evidence needed to reconstruct anomalous behaviors within the temporal resolution of the sensor pipeline.

This division of responsibilities dictates an architectural split at the fieldbus boundary. Motor drives have no operational need for trajectory digests or barrier margins; they require immediate setpoints, current limits, and active fallback commands delivered with fail-safe transport guarantees. Industrial systems handle this through established safety bus architectures. For example, Safety over EtherCAT (FSoE) implements the **black-channel principle**, treating the underlying communication bus as inherently untrusted (International Electrotechnical Commission 2021). FSoE encapsulates drive commands within a safety container featuring dedicated cyclic redundancy checks (CRCs), unique connection identifiers, strict sequence counters, and independent watchdog timers. These measures detect communication faults; sender authentication against malicious traffic requires a separate security mechanism. When a communication watchdog expires, the drive autonomously engages its safe stop state without waiting for microcontroller intervention. The rich diagnostic context (digests, barrier margins, intervention vectors, and solver runtimes) streams to the audit log

instead. An ad hoc safety-communication scheme for drive control must independently justify the fault detection and response properties covered by established functional-safety standards.

Filled in for the warehouse mobile manipulator, the enforcement record fixes the base's stopping parameters at the rack end alongside the arm's validity region and monitoring channels at the doorway and packing station. The tracking bound ϵ_{track} is 40 mm (matching row 7 of table 12.1), and the resident stopping trajectory is the jerk-limited S-curve suffix, which takes 931.25 ms and covers 605.31 mm from the 1.3 m/s aisle speed, or 1,031.25 ms and 773.44 mm from the 1.5 m/s drive limit. These spatial bounds are evaluated on a 1 ms loop period with a declared WCET of 250 µs inside a 400 µs deadline, supervised by three parallel timers: the 60 ms chunk lease, the 100 Hz host heartbeat with its 30 ms silence timeout, and the 5 ms hardware watchdog.

↪ The authority record's per-tick entries, which log the ladder rung of any active fallback, are developed in section 14.8.2.

For the manipulator, the validity region governs both the doorway and the packing station. The baseline door policy matches the nominal target ODD of the evaluation record (the nominal plate, standard door resistance, a stationary base, and the guarded approach, whereas the recovery update analyzed in section 7.8.1 qualifies the displaced-plate cell under its own distinct record rather than expanding the baseline region). The packing station admits picking a mug from the conveyor under a live intent lease followed by coworker handover. Across these zones, manipulator monitoring channels enforce dynamic velocity ceilings at the TCP. Inside the door zone, the approach-speed channel from the evaluation record's `runtime_monitor_specs` caps tool velocity at 0.03 m/s, whereas in unconstrained space the conveyor mug pick operates under the nominal 1 m/s free-space TCP limit. While a coworker holds `accept_item` (section 14.2), a handover channel engages before the separating distance closes to the arm's biased stop plus its blind-travel displacement and caps the approach toward the coworker's hand at 0.10 m/s, well below the 0.24 m/s contact-force ceiling for a free hand capable of recoil. Because a hand trapped against a rack upright or structural link permits an impact speed of at most 0.053 m/s, this handover cap is valid only when physical workstation geometry provably precludes clamping (section 12.7.2), a condition that the release argument must substantiate.

For the mobile base, operating assumptions distinguish between nominal and low-friction surfaces, each carrying its own cryptographic digest in the release manifest. The standard configuration requires a qualified directional braking lower bound encompassing driven-wheel load share, traction, torque, and stability, where a friction coefficient of at least 0.204 serves only as a necessary optimistic all-weight screen rather than sufficient evidence that the design deceleration holds. By contrast, a restricted configuration accommodates degraded traction down to an inspected-floor friction of 0.12. The restricted illustration stipulates that its 1.177 m/s² braking rate is available (since inspection alone does not establish that lower bound), and its deceleration suffix uses that stipulation to stop in 1015.3 ms across 507.6 mm from 1 m/s beneath the 1.17 m/s complete inspected-floor screen, whose braking premise still requires qualification.

Each base entry incorporates the 63.3 ms dispatch-to-admission allocation and its own scenario-specific remaining clearance. The 48.7 mm stopping reserve belongs to the dry static-obstacle aisle candidate; the restricted configuration must recompute its reserve from its own speed and braking rate. Because this allocation represents an analytical budget, neither configuration is certified for admission until physical deceleration measurements under maximum payload confirm the modeled margin.

Each numerical entry in the record carries the evidence category of its limit record: *measured*, *derived*, *datasheet*, or *illustrative*. These categories reflect fundamentally different degrees of operational confidence. The base's 931.25 ms stopping time, for instance, is a *derived* value. It follows mathematically from the geometry of the S-curve suffix under a stipulated jerk bound and an illustrative deceleration figure, assuming that motor drives faithfully deliver commanded current and tires maintain continuous traction. Only loaded stopping trials on the actual operating surface establish a *measured* value. Physical tests reveal empirical non-idealities that escape closed-form derivations, including electromechanical brake onset scatter, structural compliance, and surface contamination. Even then, empirical data supports a formal bound only when combined with worst-case fault models and defined operating limits. Explicitly recording the evidence category prevents an integrator from mistaking a derived stopping estimate for a verified hardware limit.

The record also carries the routing of section 12.9 as part of its contract. An arriving proposal can neither renew an active command nor revoke its authority merely by being transmitted, and each

lease or heartbeat expiry, cycle loss, or bus undervoltage selects the rung that matches the machine's state and the hardware that survives. Every field of the record is either an integrity token or an operational threshold evaluated against live execution state. These checks examine the proposal's sequence counter and checksum, its stored dispatch age against 51.8 ms, reconstructed perception latency against validity and stopping-margin criteria, active elapsed time against the chunk lease, tracking error against the allowable envelope ϵ_{track}, joint-velocity and torque-slew ceilings, barrier margins, phase-current and bus-voltage limits, and hardware watchdog timeouts. The contract binds each threshold to the independent sensing hardware that measures it and to the specific fallback rung selected upon violation. Table 12.6 organizes these rungs by residual system authority and specifies the hardware timer that triggers each transition.

↪ **Byte layout:** The enforcement record's fields, types, and units are given in section C.15.4.

Table 12.6: Enforcement state contract: The 20 Hz proposal, 60 ms chunk lease, 100 Hz host heartbeat, and 1 kHz MCU loop have distinct renewal and failure paths.

State	Trigger and authority	Timing check	Physical response to validate
`PROJECT`	Fresh proposal, healthy heartbeat, feasible kinetic barrier; an invalid replacement is refused; active setpoints continue only while their own evidence, lease, state, and stop reserve remain valid	1 ms cycle; declared full-cycle WCET 250 µs inside a 400 µs deadline	Track admitted torque within qualified error and brake reserve
`HOLD`	At standstill, active elapsed time beyond the 60 ms lease, host heartbeat silent beyond 30 ms, while powered support remains available	Expiry detected on the MCU clock within one tick	Powered support against gravity and payload; on undervoltage, secure qualified restraint before support expires
`STOP`	In motion, lease or heartbeat expiry, shared-rail undervoltage, tracking error beyond ϵ_{track}, an empty feasible control set, or a tripped disturbance observer or solver deadline, while powered braking remains available	Include detection, dispatch, drive rise, and brake response	Resident stop on the base, state-matched stop on the arm; measured stop distance under load; no claim after viability is lost
`INHIBIT`	Independent wired supervisor or external trip detects MCU cycle loss	Assumed 5 ms detection plus measured drive, contactor, or brake response	STO removes motor torque and the spring brakes set after coil decay; unbudgeted, so no stopping claim rests on it

Because runtime safety guarantees require that plant dynamics remain inside the modeled operational envelope, the enforcement record bounds execution within an explicit validity region. The manipulator's doorway speed ceiling illustrates how empirical evidence defines these bounds. The policy's proposed 0.10 m/s approach, refused in section 12.6 on evidential and physical grounds, passes only the narrowest timing test, since its 2.5 ms unbraked compression time from first contact to the latch force limit exceeds the 2 ms contact-loop cycle. That naive unbraked deadline ignores both the force already accumulated before the tripwire fires and the compression distance the arm still needs to decelerate to rest, so the record keeps both grounds for refusal beside that pass, the post-trip stop that section 12.7.2 sized and the absence of trials at that speed, ensuring that no reader mistakes the timing pass for admission. The enforcer therefore clamps the command to the 0.03 m/s evidence-based ceiling, evaluates full contact dynamics, and records the active constraint in the audit log. In contrast, the coworker handover ceiling is derived from the structural impact model (equation 12.13), which caps approach velocity below the contact-force limit where modeled impact forces meet allowable biomechanical thresholds for an unconstrained hand. Faster proposals are projected onto this boundary and logged as intervention vectors.

When onboard estimators detect a state outside the validity envelope, such as an undeclared payload mass, the supervisory monitor flags the violation, revokes policy authority, and commands a state-matched stop. Environmental conditions that lack direct onboard sensing, however, cannot trigger dynamic runtime vetoes. Surface friction at the wheel-floor interface represents an unobservable state, because mobile bases carry no transducers to measure instantaneous traction coefficients beneath their wheels. Consequently, the standard configuration relies on verified floor friction

as an operating precondition, while low-friction operations require manual floor inspection and commissioning of the restricted configuration.

An enforcement record connects proposed neural trajectories to validated physical invariants, operational thresholds, and deterministic fallback responses. Algorithmic correctness, however, cannot compensate for an execution platform that violates its timing or braking commitments under load. An enforcer whose control barrier functions are mathematically verified will still fail if the couplings it still shares with the host, the board-link DMA into its memory, a supply rail, or a clock, delay its cycle under load during high-speed transit. In physical artificial intelligence, safety failures manifest in the time domain just as decisively as in algorithmic logic.

↪ The floor-friction premise is enforced as an entry of the residual-claims register in section 18.3.3.

12.12 Fallacies and Pitfalls

A physical safety filter prevents collisions only as long as its control loop meets hard real-time execution deadlines and its state feedback accurately reflects reality. Safety qualification cannot rely solely on evaluating nominal control barrier equations under idealized laboratory conditions; real systems fail when host processors lock up, state estimators drift, or perception pipelines misclassify obstacles. Production testing must therefore exercise system response precisely when these core execution and sensing assumptions break down.

Pitfall: *Assuming high-level policy computers can be relied upon to execute graceful shutdown.*

A software exception handler on an application processor cannot arrest physical momentum when the operating system kernel panics. Consider a warehouse mobile manipulator whose high-level application processor experiences an operating system kernel panic while navigating an aisle at 1.3 m/s. Once the host kernel crashes, it can no longer generate deceleration trajectories or dispatch motor setpoints. Cutting the drive enable line to zero torque is equally hazardous, leaving the unbraked base to coast under momentum directly into the aisle racks. Relying on an operating system shutdown handler or a user-space exception hook assumes that the host software stack remains functional, an assumption that fails during kernel panics, hardware lockups, or memory exhaustion. Physical fail-safety requires stopping clearance and torque authority that persist independently of the host. In a robust architecture, the downstream MCU detects the missing heartbeat within its 30 ms timeout, comfortably within the stopping budget's reserved lease margin. The MCU then immediately executes its resident jerk-limited stop suffix, decelerating the base to a complete stop in 931.25 ms across 605.31 mm. While host shutdown routines manage orderly transitions during routine operation, true fault tolerance demands that stopping authority resides in hardware and low-level firmware that survive host death.

Architect's countermeasure: Decouple physical stopping authority from host processor lifecycles with two MCU timers that the host cannot service on the MCU's behalf. The heartbeat timer, refreshed by the host's periodic heartbeat, catches a host that has gone silent, which is what a kernel panic produces. The chunk lease, refreshed only by a valid admitted proposal, catches a host that is still alive and still sending heartbeats but no longer delivering valid motion. When either expires, the MCU intervenes without waiting for host acknowledgment, selecting the precompiled, state-matched stop suffix within its validated entry envelope and actively commanding motor torques to bring the robot to a controlled halt.

Fallacy: *A mathematically correct safety filter guarantees safety even when underlying state feedback has drifted.*

A barrier certificate evaluated against a drifted or corrupted state estimate provides zero physical protection. The mobile base's 40 mm tracking certificate bounds true physical position error only under qualified state feedback assumptions. In practice, comparing the reference trajectory against encoder-reported position serves merely as a runtime proxy; that agreement alone cannot verify the actual physical bound. Wheel slip or an optical encoder with an undetected scale bias can lead the state estimator to report that the base is farther from the aisle rack than its true physical location. The CBF-QP solver then easily satisfies its barrier inequality on the corrupted estimate, while the physical base exhausts clearance margins that the stopping budget never granted. Only an independent sensing channel, such as an IMU or a safety lidar measuring direct range to the rack, can expose this divergence, and only if the tracking error exceeds the diagnostic monitor's observability threshold. System qualification must bound how far the estimate can drift before

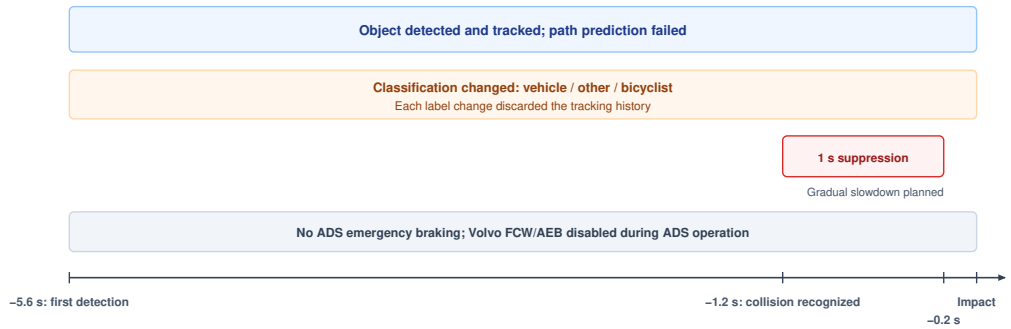


Figure 12.11: Tempe detection and braking timeline: The NTSB reports first detection 5.6 s before impact and recognition of imminent collision 1.2 s before impact. The ADS suppressed its motion plan for one second and then planned gradual slowing; it did not perform emergency braking. Classification and path prediction failed: at each change of class the ADS discarded the object's tracking history. The Volvo forward-collision warning and automated emergency braking functions were disabled while the developmental ADS operated. (Adapted from NTSB/HAR-19/03 (National Transportation Safety Board 2019).)

detection occurs. The stopping budget must account for this worst-case drift in its localization allowance δ_{loc}, rather than crediting that same uncertainty to the dynamic tracking inset. To preserve formal safety guarantees, the qualification framework must explicitly document which physical error source each budget allowance covers.

Architect's countermeasure: Implement redundant, heterogeneous sensor cross-checks that continuously validate kinematic wheel odometry against independent physical channels, such as optical flow, time-of-flight lidar, or secondary motor encoders. Expand the safety filter's localization allowance δ_{loc} to absorb the maximum unobservable drift that can accumulate before diagnostic cross-checks trip, guaranteeing that the contracted safe set remains physically invariant even under worst-case sensor degradation.

Pitfall: *Coupling safety braking authority to stochastic semantic classification rather than deterministic spatial clearance.*

Emergency stopping authority must be bound to deterministic spatial clearance rather than upstream semantic classification. When an automated vehicle conditions collision avoidance on object classification and trajectory forecasting, perception errors can silently disable emergency intervention. The March 2018 Tempe collision, analyzed in section 1.8, shows the sequence (figure 12.11). Although the developmental automated driving system (ADS) first detected the pedestrian pushing a bicycle 5.6 s before collision, repeated perception toggling between vehicle, other, and bicyclist classifications reset the tracking filter, discarding historical motion at each switch and treating the obstacle as an unpredicted static entity (National Transportation Safety Board 2019, sec. 1.5.5.2). When the pipeline finally registered an imminent crash 1.2 s before impact, high-level policy suppressed actuator intervention for a full 1-second delay to avoid vehicle swerving, after which the planner issued only a gradual deceleration request. The architectural failure sits at the base of the timeline. The vehicle's factory-installed Volvo autonomous emergency braking (AEB) interlock had been suppressed during ADS operation. An independent geometric safety barrier avoids these cascading failures by separating spatial clearance from semantic inference. It commands an emergency stop based solely on physical distance and closing speed, regardless of what label the neural perception pipeline assigns to the obstacle.

Architect's countermeasure: Enforce the causal separation of perception and safety by establishing an independent, non-semantic geometric barrier. A safety-rated sensor channel, such as a time-of-flight lidar curtain, radar, or safety scanner, must maintain direct hardware veto authority to command a Category 1 stop. This safety layer evaluates occupancy and closing rate directly, completely bypassing the neural perception pipeline, semantic classifiers, and trajectory forecasting models.

Fallacy: *Assuming a continuous-time barrier certificate holds under discrete Zero-Order Hold sampling without an inter-sample kinematic buffer.*

Checking a continuous-time barrier inequality at discrete sampling instants does not guarantee inter-sample safety. In digital control architectures, an actuator applies a constant torque or current command over the entire zero-order hold (ZOH) interval $T_{\text{tick}} = 1$ ms. If an active-set QP solver computes a minimally invasive control input at sample t_k that places the plant directly on the boundary of the safe set ($\partial\mathcal{C}$), continuous physical dynamics unfold during the intervening hold period. Between t_k and t_{k+1}, physical acceleration under the held actuator command can drive the true state across the boundary before the next control cycle executes. Equating a continuous barrier certificate $\dot{h} + \gamma_{\text{cbf}} h \geq 0$ directly with discrete-time invariance without an inter-sample buffer invites boundary violations whose severity grows with velocity and sample period.

Architect's countermeasure: Contract discrete-time barrier certificates by the worst-case inter-sample kinematic drift buffer, $\Delta d_{\text{ZOH}} = v T_{\text{tick}} + \frac{1}{2} a_{\max} T_{\text{tick}}^2$ evaluated at the highest reachable speed v. In a 1 kHz control loop operating at the 1.5 m/s drive limit with an independently stipulated approach-acceleration upper bound of 2 m/s², this inter-sample displacement bound equals 1.501 mm, the drift that the discretization condition of section 12.7.3 charges. Enforcing the barrier condition against the tightened safe set $\mathcal{C}_{\text{discrete}} = \{\mathbf{x} \mid h(\mathbf{x}) \geq \Delta d_{\text{ZOH}}\}$ preserves physical clearance throughout the hold duration when the barrier measures spatial distance. When accounting for this margin in the stopping budget, architects must avoid double-counting, because any travel already allocated to the budget's one-tick fault detection interval already covers this displacement. Furthermore, when evaluating non-spatial or kinetic barriers, engineers cannot rely solely on a geometric displacement bound; the implementation must bound the worst-case evolution of the barrier function itself under the held input across the full sample interval $[t_k, t_{k+1}]$.

12.13 Summary

Runtime enforcement grants an admitted plan continuing, conditional permission to drive physical hardware. On every control cycle the enforcer evaluates live plant state against the tracking bound, the stopping reservation, and the actuator authority that remains. It passes a safe proposal through, projects a correctable one back into the admissible set, and selects a resident fallback, without waiting for the learned proposer, when no correction exists or the proposer falls silent.

Clearance is a composite margin, not a distance read from a map. At the warehouse mobile manipulator's 1.3 m/s aisle speed, travel during the pre-brake delay τ_{delay} takes 173.68 mm, the C^2 stop suffix 605.31 mm, localization and protective margin 150 mm, and the tracking inset 40 mm, for 968.99 mm of the 1.10 m available at the rack end. The remaining 131.01 mm pays for dispatch-to-admission travel ($v\Delta_{\text{da}} = 82.3$ mm) and leaves a 48.7 mm reserve, so the aisle speed remains a candidate until loaded timing measurements confirm that allocation. A budget built on instantaneous, constant-deceleration braking would have admitted the drive limit, where the jerk-limited stop no longer fits. The budget also holds only while the actuators deliver their assumed deceleration; gravity on a descending payload, bus sag, current ceilings, and thermal throttling all shrink braking authority, so the enforcer must contract the speed envelope before that authority is spent rather than after.

The kinetic barrier turns that reservation into a constraint on the next command. Raw clearance has relative degree two with respect to torque, so a barrier on distance alone leaves the motor no authority at the boundary. Folding approach speed and net braking deceleration into equation 12.5 lowers the relative degree to one, and its zero level set is the single-joint stopping budget written in state space. Under the qualified delivered-effort error and response-delay contract, the least-change quadratic program removes only the part of a proposal that violates the tightened half-space, which preserves useful task progress when the corrected motion remains task-admissible. An independent per-axis clamp can leave the coupled constraint broken. When contact is the task, force replaces clearance as the margin. Force keeps building through the response interval, so the latch and handover ceilings are approach speeds derived from the force at brake onset and from the effective mass a free or a clamped hand meets, each valid only under its stiffness, deceleration, and geometry premises.

When a lease lapses or a checked premise fails, the arbiter selects directly among `PROJECT`, `HOLD`, `STOP`, and `INHIBIT` by the plant's state and the hardware that survives; the rungs are alternatives,

not an escalation. Three timer duties detect different failures: the 60 ms chunk lease bounds a stale proposal, the 30 ms host-heartbeat timeout catches a host stall, and the 5 ms MCU self-watchdog catches a firmware deadlock. Beneath these responses sit the certified functions of the drives, the safety scanner, and the arm's safety controller, which carry the formal protective claim. On the permission path itself, only the safety MCU's watchdog that revokes authority when the chunk lease expires or the host heartbeat falls silent is a rated safety function. The admission checks are an unrated performance layer whose job is to reduce unnecessary trips of the assumed qualified protective floor (section 12.9.3). The enforcement record ties each check to its detector, its fallback, and the evidence category of every number it uses, so that what the permission path verified stays distinct from what it assumed. The checks and ceilings the permission path applies reduce to the relations in table 12.7.

Table 12.7: Key equations of safety enforcement: Each equation fixes one check or ceiling the permission path applies. Every result holds only while its inputs (delays, braking authority, bias, contact stiffness, and timing) stay inside their qualified bounds.

Equation	What it bounds or decides	Derived in
$v\,\tau_{\text{delay}} + d_{\text{suffix}}(v) + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} \le D_{\text{clear}}$	Stopping subtotal before dispatch travel; complete admission additionally charges that interval	equation 12.1
$v_{\max} = -a_{\text{brake}} T_{\text{eff}} + \sqrt{(a_{\text{brake}} T_{\text{eff}})^2 + 2 a_{\text{brake}} D_{\text{avail}}}$	The pre-dispatch speed ceiling and unallocated timing allowance left below it	equation 12.2
$\alpha_{\text{net}} = (\tau_{\max} - \tau_{\text{bias}})/J_{\text{eff}}$	Braking authority left when gravity loads the joint in the direction of motion	equation 12.3
$h(\mathbf{x}_0) \ge 0 \wedge \dot{h} \ge -\kappa(h) \implies h(\mathbf{x}_t) \ge 0$	Forward invariance of the safe set under the model, actuator, and timing assumptions	equation 12.4
$h(\theta_{\text{clear}}, \omega) = \theta_{\text{clear}} - \omega^2/2\alpha_{\text{net}}$	A barrier with relative degree one: remaining clearance minus the cost to stop	equation 12.5
$\tau \le -\tau_{\text{bias}} + J_{\text{eff}} \alpha_{\text{net}} (\gamma_{\text{cbf}} h/\omega - 1)$	The torque half-space a proposal must satisfy on the next tick	equation 12.6
$\min \frac{1}{2}\lVert\mathbf{u} - \mathbf{u}_{\text{nom}}\rVert^2$ s.t. $\mathbf{a}_{\text{cbf}}^\top \mathbf{u} \le b_{\text{cbf}}$	The least-change correction of a proposal inside the actuator envelope	equation 12.7
$\mathbf{u}^* = \mathbf{u}_{\text{nom}} - \lambda^* \mathbf{a}_{\text{cbf}}$	Closed-form projection onto one active half-space	equation 12.8
$\dot{E}_{\text{tank}} = \dot{\mathbf{q}}^\top \mathbf{D}_{\text{virt}} \dot{\mathbf{q}} - \mathbf{u}_{\text{active}}^\top \dot{\mathbf{q}}$	The work a contact controller may still inject before its energy reserve is spent	equation 12.9
$F_{\text{cmd}} = F_{\text{trip}} + k_{\text{latch}} v_{\text{latch}} T_c$	Contact force already reached when braking begins at the latch	equation 12.11
$F_{\text{unbraked}} = v_{\text{approach}} \sqrt{k_H m_{\text{red}}}$	Peak hand force at an approach speed, and so the handover speed ceiling for a free or clamped hand	equation 12.13

DRAFT

Key Takeaways: Conditional enforcement and physical stopping

- **Tracking is a bounded premise:** Steady-state tracking guarantees do not bound transient overshoot. Clearance insets must incorporate worst-case transient tracking errors validated experimentally across the robot's full operating envelope.
- **The stopping budget dictates allowable speed:** Sensing latency, reaction delays, C^2 braking trajectories, tracking error insets, and localization margins all compound into a single admission check. True physical clearance, rather than motor drive limits, determines how fast the machine may safely move.
- **Control barriers must account for kinetic energy:** Because spatial distance has relative degree two with respect to joint torque, a position-only barrier cannot guarantee immediate forward invariance. Formulating a kinetic barrier that incorporates velocity yields an affine torque half-space for safe single-step execution.
- **In physical contact, force replaces distance as the margin:** Mechanical force continues to build throughout the latency of a braking response. Latching stops must be dimensioned

from the projected peak force at brake onset, and approach velocities during human handover must be capped based on effective inertia and structural compliance.

- **Distinct timer duties expose different failure modes:** Trajectory chunk leases, host-to-controller heartbeats, microcontroller watchdog cycles, and sensor timestamps expose different failure modes. No single software timeout can protect against all failure modes.
- **Fallback execution is state- and hardware-dependent:** Safely executing a least-change projection, powered zero-velocity hold, controlled deceleration ramp, or torque inhibit requires specific surviving hardware and dynamic state. The fallback must be selected while the admissible control set is still nonempty; an empty chosen intersection loses that filter's claim, while an alternate response needs its own qualified claim.
- **The software permission path complements certified hardware:** Safety-rated laser scanners, certified drives, and emergency stop interlocks provide the formal protective baseline. The running architecture assumes a rated lease-expiry and heartbeat watchdog on the permission path. The algorithmic admission checks are unrated and operate above this floor to keep operation smooth without tripping certified hard stops.
- **Every intervention requires an audit record:** The enforcer must bind every command token to verified sensor inputs and log all constraint modifications or fallback triggers to an audit log. This logging ensures system designers can audit safety margins on physical hardware and validate assumptions before production deployment.

This chapter is where the third law, proposal is not permission, stops being a design stance and becomes a mechanism. A learned proposal reaches the drives only as a candidate. It earns authority one tick at a time, and only while its evidence is fresh, its lease unexpired, its tracking error within bound, and the stop it would need still inside the clearance. Nothing the proposer emits can extend that authority, and the response that replaces it never waits for the proposer.

A complete runtime enforcement architecture joins the flexibility of learned policies to the determinism of physical safety guarantees. It reserves stopping clearance dynamically, projects candidate controls when a feasible correction exists, and selects a state-matched fallback when communication fails or an invariant breaks. Mathematical safety certificates and algorithmic barrier functions, however, are only as dependable as the underlying real-time compute fabric. Interference through whatever the permission path still shares with the proposers, a board link, a supply rail, a clock, or in some placements a die and a memory bus, can silently consume time that the physical budget already assigned to braking. The final engineering obligation is therefore empirical: verifying that the entire permission pipeline strictly meets its real-time deadlines under worst-case system load on the physical hardware.

12.14 Historical Perspective and Further Reading

The safety certificates developed in this chapter trace their intellectual lineage to classical questions about boundary invariance that predate digital computing. In 1892, Aleksandr Lyapunov demonstrated that one can prove the asymptotic stability of a dynamical system by constructing a scalar, energy-like function, bypassing the intractable problem of solving nonlinear differential equations directly. Mitio Nagumo extended this geometric intuition from isolated equilibrium points to entire sets (Nagumo 1942): a closed set traps all future trajectories if and only if the vector field at every point along its boundary points inward or tangent to the boundary surface. Decades later, Aubin formalized this boundary principle for controlled systems within viability theory (Aubin 1991), characterizing the viability kernel as the set of states from which at least one admissible control input can prevent constraint violations indefinitely. The differential games of Isaacs (1965) introduced an adversarial perspective against worst-case disturbances, which Mitchell, Bayen, and Tomlin later computed on numerical grids through Hamilton-Jacobi reachability analysis (Mitchell et al. 2005). Prajna and Jadbabaie translated these boundary arguments into barrier certificates for algebraic system verification (Prajna and Jadbabaie 2004), and Wieland and Allgöwer framed the barrier condition as an affine inequality directly over the control input (Wieland and Allgöwer 2007). Ames and collaborators then combined this affine formulation with real-time optimization, creating the

control barrier function quadratic program (CBF-QP) whose solve time can lie in the microsecond range for selected problem sizes and platforms (Ames et al. 2014; Ames et al. 2019). Contemporary systems engineering treats control barrier functions, reachability analysis, and predictive backup sets as a unified family of safety filters, each continuously monitoring an unverified control proposal against a verified fallback action that remains physically executable (Hsu et al. 2024).

The structural pattern that wraps an analytical certificate around a complex model originates in runtime assurance for aerospace and industrial automation. To deploy advanced, unverified control software without risking catastrophic failure, Sha and colleagues designed the Simplex architecture (Seto et al. 1998; Sha 2001). Simplex allowed a high-performance experimental controller to drive a live plant while an analytically verified baseline controller and a deterministic switching monitor stood ready to reclaim authority whenever the state neared the edge of the recoverable envelope. Alshiekh and collaborators later transplanted this runtime assurance principle into reinforcement learning under the rubric of shielding (Alshiekh et al. 2018), using reactive temporal logic specifications to intercept and correct unsafe exploratory actions before they reached the actuators. Physical embodiment, however, introduces contact dynamics that purely kinematic boundary checks cannot govern. Hogan transformed robotic manipulation by introducing impedance control (Hogan 1985), which regulates the dynamic relationship (stiffness, damping, and inertia) between environmental forces and manipulator motion instead of tracking brittle position trajectories. Willems provided the rigorous foundation for analyzing such interactions through dissipativity theory (Willems 1972), which formalizes the energy bookkeeping required to guarantee stability across interconnected subsystems. Ferraguti and co-authors embedded that bookkeeping directly into digital software via virtual energy tanks (Ferraguti et al. 2013). By tracking energy dissipation within the controller's internal state, energy tanks restrict command authority when stored reserves deplete, preventing explosive instability during abrupt contact or communication latency.

The requirement that a fail-safe mechanism must operate independently of the software it monitors emerged from painful engineering lessons. The Therac-25 accidents recounted earlier in this chapter illustrated the fatal cost of replacing electromechanical safety interlocks with software routines (Leveson and Turner 1993); race conditions in unverified code bypassed intended protections and delivered lethal radiation doses to patients. International functional safety standards codified this experience by mandating strict architectural isolation between rated safety functions and the operational systems they supervise (International Electrotechnical Commission 2010a; International Organization for Standardization 2023a). In robotic drivetrains, this separation extends down to hardware-level stop functions, such as safe torque off and safe operating stop, implemented directly within power electronics and drive logic (International Electrotechnical Commission 2016b). When runtime enforcement relies on software, temporal predictability becomes just as vital as functional correctness. Worst-case execution-time (WCET) analysis, surveyed comprehensively by Wilhelm and colleagues (Wilhelm et al. 2008), combines execution models, platform assumptions, and timing evidence to justify latency bounds; empirical observations alone do not establish a certified worst case. The architecture presented in this chapter unites these distinct traditions. A learned policy proposes high-performance actions, an online filter derived from physical stopping budgets preserves forward invariance under its qualified assumptions, and an isolated safety floor supplies its separately qualified protective response. A tamper-evident enforcement log records the exact state evidence that justified each intervention. For further reading, six works trace these ideas from their foundations to current practice:

- Blanchini (1999) surveys set invariance in control, tracing the theory from Nagumo's boundary conditions to algorithmic methods for computing polyhedral and ellipsoidal invariant sets, providing the clearest geometric map of the state space that barrier certificates supervise.
- Aubin (1991) develops viability theory, the mathematical framework governing the states from which a constrained system can remain safe indefinitely, and the formal theoretical foundation for the kinetic barriers and viability kernels used throughout this chapter.
- Sha (2001) presents the foundational runtime-assurance argument: a complex, unverified controller may retain command authority only while an analytically verified baseline controller, operating within a known recoverable region, stands ready to safely intervene.

- Leveson and Turner (1993) reconstructs the Therac-25 radiation therapy accidents and remains the definitive case study demonstrating why safety interlocks must never depend solely on the software systems they are designed to protect.
- Ames et al. (2019) provides a comprehensive tutorial on control barrier functions and quadratic programming safety filters, covering both mathematical underpinnings and physical robotics deployments, and serving as the natural follow-up to section 12.7.
- Hsu et al. (2024) unifies control barrier functions, Hamilton-Jacobi reachability, and predictive backup sets into an overarching safety-filter framework, analyzing how learned neural policies can be trained alongside runtime filters to avoid boundary chattering, directly addressing the co-design challenges raised in section 12.8.1.

What's Next: From barrier certificates to SoC placement

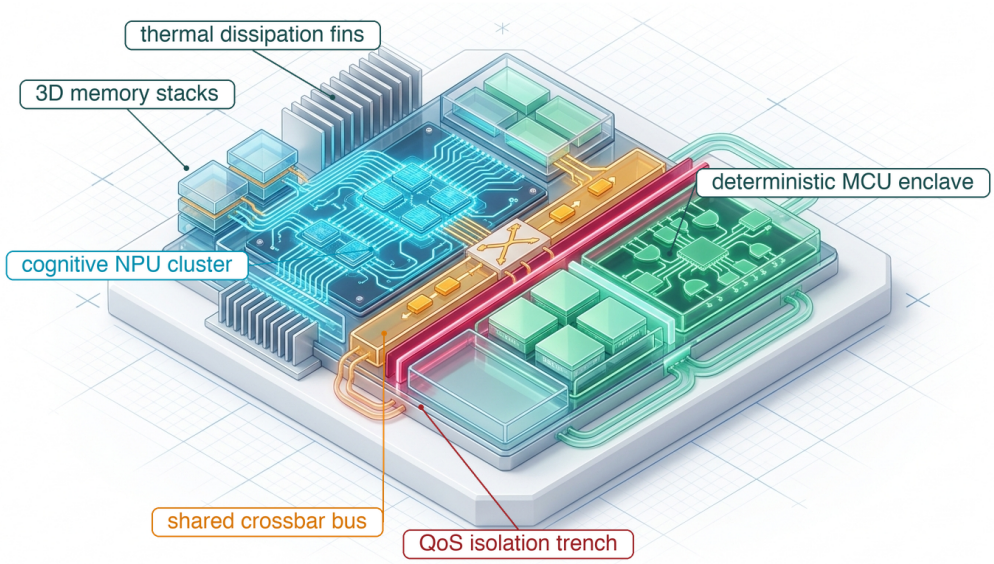
Can the learned proposers share a chassis, and in some placements a die and a memory bus, with the permission path without delaying it? The enforcement record declares a worst-case cycle inside the permission deadline on every tick. That deadline is a claim about silicon, and a barrier result holds only while the permission path meets it. Every budget in this chapter has assumed the placement Part I adopted as a hypothesis, a discrete safety microcontroller physically separate from the proposers' processor. Chapter 13 puts that hypothesis under load. It partitions the learned proposers and the permission path across compute fabrics and names the loaded tests that would show whether memory starvation, clock throttling, or bus contention can delay permission or fallback, recording them in the placement record.

DRAFT

DRAFT

13

Workload Placement

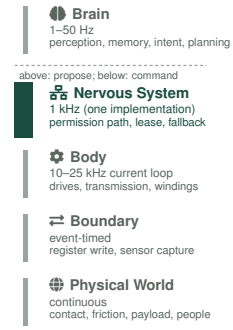


13.1 The Silicon Substrate
13.2 Two Paths and the Resources They Share
13.3 Contention for Shared Resources
13.4 Thermal and Timing Coupling
13.5 The Boundary Contract
13.6 Silicon Isolation Decisions
13.7 Empirical Verification of Silicon Isolation
13.8 The Placement Record
13.9 Fallacies and Pitfalls
13.10 Summary
13.11 Historical Perspective and Further Reading

Purpose

Why can a safety check miss its deadline even when it runs on its own processor core?

A correct refusal computed too late does not stop the motion it was meant to prevent. Separating a learned proposer from its permission path in software does not make the two independent while they still share memory, electrical power, clocks, or cooling. A burst of inference can then delay the check that should stop the machine, heat from the accelerator can slow the core that runs that check for seconds after the burst has ended, and a single supply or reset fault can silence both paths at once. None of these couplings appears on a software architecture diagram, and none shows up in average utilization, so a safety argument that has never been loaded on its real silicon rests on an untested premise. Moving the check to separate hardware removes some couplings and adds others, a communication delay and a longer chain of supplies and wiring, so no placement is independent by construction. Where each workload runs therefore decides whether the permission path can still refuse in time, and an action that crosses the causal boundary on a late refusal cannot be taken back.



Learning Objectives

- Conduct a dependent-failure inventory of the caches, memory controllers, interconnects, supply rails, clocks, resets, and thermal paths that each candidate placement leaves shared between the proposal and permission paths
- Explain why average bus utilization cannot bound a deadline, and identify which contention mechanism (bandwidth queueing, bank conflicts, or shared-cache miss handling) threatens a declared worst-case execution time
- Evaluate whether shared power and reset paths can defeat a selected permission placement
- Analyze thermal coupling and clock throttling hysteresis between multi-tenant workloads across disparate physical timescales
- Size a windowed watchdog's service window from justified completion and inter-tick jitter bounds for a non-blocking record exchange
- Select among static reservation, spatial partitioning, workload reduction, and physical silicon relocation from measured interference and timing allowances, and allocate the dispatch-to-admission interval of an on-board or off-board proposer against the stopping budget
- Design an adversarial stress campaign whose falsification threshold is the permission path's declared worst-case execution time (T_{wcet})
- Construct a placement record formalizing the deployment envelope that scopes loaded observed maxima, thermal headroom, bus arbitration, mitigations, and untested conditions for release review

13.1 The Silicon Substrate

DRAFT

When the warehouse mobile manipulator's permission check finishes late, its motor drives keep executing the last admitted command, and the loaded base covers distance that the stopping budget of chapter 12 never reserved. The control barrier filter, the stopping reservation, and the fallback ladder developed there hold only while the permission path completes its check inside its hard real-time deadline on every cycle. The enforcement record declares a worst-case execution time inside that deadline, a bound on the check running alone, and whether the bound survives while the learned proposers run beside it is a question about the silicon underneath. Workload placement decides which silicon executes each computation, and with that choice, which memories, interconnects, clocks, power supplies, and thermal paths each task shares with the others. In conventional computing, placement is an optimization over throughput, latency, or energy. On a machine whose refusals must arrive on time, it becomes a safety problem.

Modern silicon integration works directly against the separation the permission path needs. A contemporary heterogeneous system-on-chip (SoC) integrates general-purpose application cores, neural accelerators, real-time microcontrollers, and specialized safety islands onto a single monolithic die. On that die, these subsystems share memory controllers, dynamic random-access memory (DRAM) channels, system-level caches, on-chip network interconnects, voltage regulators, clock governors, and the physical package that carries away heat. While silicon integration lowers unit cost, board area, and inter-processor communication latency, it quietly couples through shared hardware the very execution paths that the software architecture keeps apart. The question placement must answer is whether the deterministic permission path can keep its deadline and its authority to refuse unsafe motion while the proposal path runs at full load on shared hardware. Answering it requires a bound on hardware interference for every shared physical resource, derived from worst-case timing analysis and validated under loaded stress tests.

On the warehouse mobile manipulator, the software architecture draws the proposal boundary between the Brain and the Nervous System as a clean logical boundary. Part I adopted a working hypothesis about where that line falls in hardware. That hypothesis places the permission path on a discrete safety microcontroller physically separate from the application SoC. The machine's two physical jobs show what that separation must guard against. For the mobile base, whose mass-

dominated mobility spends stopping clearance, an interconnect stall during visual feature extraction on the proposal engine could delay the permission path's refusal while the loaded base approaches a rack end. Every tick of delay beyond the single cycle accounted for in the stopping budget is travel distance the machine never reserved. For the arm, whose contact-dominated manipulation spends force margin, a burst of neural accelerator traffic that evicts the permission path's lookup tables from a shared cache could delay contact force validation while the gripper presses against a spring-latched cage door. The monolithic SoC is therefore an architecture the machine must be able to reject on evidence, and the discrete microcontroller is a design claim that must survive a loaded test rather than be assumed.

Assigning the permission filter to a dedicated processor core does not create an isolated failure domain. A core running in isolation may still stall on memory arbitration, slow down under dynamic clock throttling, or suffer transient voltage drops when adjacent accelerators become active. Memory protection units and operating system process boundaries enforce spatial partitioning by preventing unauthorized reads and writes across address spaces. However, memory protection hardware governs access permissions rather than access timing. The underlying interconnect crossbar, memory controllers, and DRAM banks determine when memory transactions actually complete. Establishing a sound safety case requires both spatial partitioning to prevent memory corruption and temporal partitioning to guarantee bounded service times for shared hardware resources (Kopetz 2011; Rushby 1999).

↩ The independence requirement on the permission path, and the two-processor split as a hypothesis that loaded tests must support, are established in section 4.2.

Definition 13.1: Operational independence

Operational independence is the property that no activity, fault, or overload in the proposal domain can push the permission path's execution past its deadline or disable its authority to refuse, through any resource the two paths share.

1. **Significance**: It is the premise under which the stopping budget charges exactly one permission tick of detection delay. Every tick that proposal-side load adds beyond that one is travel at speed v that the irreversibility test $d_{\text{stop}} \leq D_{\text{clear}}$ never reserved, and a stall of unbounded length leaves the test with no bound at all.
2. **Distinction**: Unlike spatial isolation, which private address spaces and separate cores supply, operational independence also covers time, supply, clock, reset, and heat. It is the placement-level form of what functional-safety standards call freedom from interference, the absence of cascading failures from one element into another of higher integrity (International Organization for Standardization 2018).
3. **Common pitfall**: Arguing independence from the separation alone. A separate core, or even a separate chip, is the hypothesis the Nervous System adopted, not its proof; independence is argued one shared resource at a time, and a loaded test on the real silicon can refute the claim for any one of them.

Every placement claim must therefore name the physical and architectural resources that could delay or disable enforcement, alongside the mechanisms that bound their mutual interference. A shared die and a discrete safety microcontroller are competing hypotheses about those hardware dependencies, not interchangeable processor choices. Testing either hypothesis starts with what each path computes and the deadline it must meet.

13.2 Two Paths and the Resources They Share

The third law, proposal is not permission, separates the machine's two paths in authority; placement decides whether the hardware keeps them separate in time when the proposer is working hardest. Whether a candidate placement preserves that separation depends on the execution profile of each path, the physical resources that could couple them beneath software access controls, and which of those resources remain shared across the compute board.

13.2.1 Proposal and permission execution profiles

If a 40 ms neural proposer and a 400 µs-deadline safety loop share the same operating-system scheduler or memory controller, a single kernel preemption tail or memory-bus burst can stall the 1 kHz permission tick past its deadline. To prevent that stall, the mobile manipulator splits control across two execution paths with different systems requirements. The proposal path runs the chunk policy on the application processor at 20 Hz, ingesting synchronized navigation and wrist camera streams to emit coordinated base velocity and arm joint targets. Its 99th-percentile (p99) inference latency, vision processing included, consumes 40 ms (an illustrative benchmark; see the Reader Guide) of each 50 ms chunk period. Running in unprivileged user space, the proposal path holds no direct hardware authority over the motor drives. In the reference design from Part I, the permission path executes on a discrete safety microcontroller (MCU) at 1 kHz. On every tick, it checks the active trajectory and any candidate replacement against the stopping envelope from section 12.3, staging admitted setpoints into the outbound EtherCAT frame destined for the drives. This routine must finish within a chosen 400 µs deadline inside its 1 ms tick. It runs in a privileged real-time domain on a dual-core lockstep controller,[1] whose cycle-by-cycle comparator flags divergent execution. That hardware guard is needed because a software envelope check cannot diagnose a silent fault in the CPU core executing it. Only the permission path holds memory-mapped input-output (MMIO) access to the fieldbus master carrying drive commands.

> Definition 13.2: Heterogeneous silicon partitioning
>
> ***Heterogeneous silicon partitioning*** is the architectural allocation of stochastic, high-capacity cognitive workloads (perception, learned policy evaluation) and deterministic safety loops (safety invariant checks, admission and fallback) across specialized execution domains isolated in hardware.
>
> 1. **Significance**: Placing high-capacity neural deliberation and real-time safety enforcement on the same general-purpose operating system or compute engine creates common-mode failures. Kernel thread preemption, dynamic memory allocation, and time-slicing between accelerator contexts add tail delay that no measured average bounds, and a permission path exposed to that tail can miss its physical control deadline.
> 2. **Distinction**: Unlike logical software process isolation (such as Linux containers or POSIX process priorities), silicon partitioning enforces isolation across independent hardware registers, tightly coupled memories (TCM), and dedicated peripheral buses.
> 3. **Common pitfall**: Treating multicore CPU affinity as failure-domain separation. Cores on one die still share caches, memory controllers, clock trees, and power rails, the resources that operational independence (definition 13.1) must bound one by one.

These two processors run separate software stacks. Proposal inference, global scene-map updates, telemetry streaming, and fleet networking run on the application processor under Linux. While a real-time kernel patch shortens scheduler dispatch latency, it cannot bound latency tails added by unvetted device drivers, bus arbitration, or shared memory controllers.[2] Similarly, zero-copy buffers reduce internal memory-bus traffic, but they do not remove contention at the physical memory controller.

By contrast, enforcement and sensor validation run on the discrete safety microcontroller as bare-metal firmware governed by a hardware memory protection unit (MPU).[3] Quadrature encoder counters, the inertial measurement unit (IMU) on its Serial Peripheral Interface (SPI) bus, and hardware timer capture channels are mapped directly to dedicated physical pins. Each candidate proposal arrives from the application processor across a board-level serial link and is published into local memory via the seqlock mailbox of section 4.3.2. No dynamic random-access memory (DRAM) is shared between the two paths. The memory protection unit isolates local state, while the dedicated peripheral bus insulates the enforcer's execution path from the host's memory traffic.

Admitting an entire plan is a different computation from running the cyclic permission tick. For example, the whole-plan validation routine of section 11.3.1 cannot run uninterrupted inside a 1 ms control loop without breaking timing guarantees. Instead, the runtime scheduler must preserve

[1] **Dual-Core Lockstep**: Dual-core lockstep runs identical instruction streams across two physical cores, typically offset by several cycles so that a single transient fault cannot corrupt both pipelines in the same way, while a hardware comparator flags any divergence. While lockstep execution catches internal CPU faults, it cannot catch bad envelopes computed from corrupted sensor inputs or common-mode voltage sags on shared power rails. Lockstep is therefore one piece of the safety microcontroller's watchdog design rather than a complete integrity defense.

[2] **PREEMPT_RT**: The Linux real-time patch set makes most kernel code preemptible and runs interrupt handlers as schedulable threads, and it was merged into the mainline kernel in Linux 6.12 in 2024 after about two decades out of tree (KernelNewbies 2024). It shortens the scheduler's latency tail but cannot bound delays that drivers, firmware, or shared memory add, so a PREEMPT_RT host needs a measured bound before it may carry any part of the permission tick.

[3] **MPU and IOMMU**: A memory protection unit (MPU) checks each CPU access against a few fixed address regions without translating addresses, so its cost is constant and analyzable. An input-output memory management unit (IOMMU) applies the same check to direct memory access (DMA) from accelerators and camera interfaces. Both govern *where* a memory transaction may access without arbitrating *when* the interconnect serves it; they enforce spatial partitioning while delegating deadline guarantees to hardware bus arbiters.

the real-time tick either by running admission as bounded, interruptible work or by moving it to a separate processor whose own timing is verified. In either case, admission ends with an atomic check of current state and sensor evidence right before activation. The running machine splits admission across these domains. The host performs the global geometry checks of section 12.8. The microcontroller checks the resulting record against bounded local evidence as background work below the tick's priority, so the tick preempts it. Section 13.6.2 lists the bounds that keep this background work from delaying the tick through the microcontroller's own buses and memory. The resource budget must cover the active trajectory buffer, the resident fallback plan, the temporary admission workspace, and worst-case interruption overheads. The proposal generation rate also does not set the interval between successful admissions. Qualification must show that the end-to-end pipeline (transfer, admission, and activation) preserves the lease renewal bound, a guarantee the tick deadline alone cannot provide.

Similarly, the full volumetric scene map stays on the application processor. The permission path receives only a compact, bounded local evidence snapshot whose spatial coverage, state uncertainty, observation timestamp, expiration horizon, and transport integrity have been pre-validated for the proposed motion and its fallback maneuver. Updating the host map, serializing the snapshot, transporting it across the bus, validating admission, storing it locally, and running per-tick spatial queries are separate computational costs that need their own entries in the timing budget. A synchronous query to the host processor must never stall the permission tick. If a proposed trajectory strays outside the validated bounds or lifetime of the local snapshot, the MCU rejects it immediately. The microcontroller's static memory budget must therefore show that the local snapshot buffer and resident fallback trajectories fit alongside the compiled enforcement binary.

Electrically, the power delivery network isolates the microcontroller, low-voltage drive logic, and electromechanical spring-brake coils onto a dedicated safety rail. This rail carries a local capacitive hold-up of 2 s and draws zero load from the proposal subsystem. Hardware qualification must verify that voltage transients from motor acceleration or neural accelerator bursts on the shared battery pack cannot trigger an undervoltage reset on this isolated rail. This local energy reserve keeps the logic alive and holds the mechanical brakes released throughout a controlled stop without supplying traction power. Achieving the powered deceleration profiles of section 12.9.1 requires both an intact high-voltage drive path and a sink for regenerative braking energy. If either fails, the system must trigger a certified fail-safe fault response rather than relying on powered stopping-distance assumptions.

On an unloaded machine, the initial compute budgets look generous. Representative baseline costs on the safety microcontroller allocate 40 µs to sample motor encoders, read the IMU over SPI, and check the proposal's evidence epoch. They allocate 80 µs to check kinematic and dynamic envelope constraints and 15 µs to stage admitted setpoints into the outbound drive frame. That gives an unloaded enforcer execution time of 135 µs, or 54,000 cycles at the microcontroller's 400 MHz clock. Measured against the 400 µs hard deadline, this baseline path leaves 265 µs of slack. Even the declared worst-case execution time (WCET) of 250 µs, the design envelope from section 12.7.4, leaves 150 µs of headroom.

↩ The isolated permission rail and its hold-up sizing are established in section 2.8.2.

This timing margin cannot be spent entirely on shared-hardware contention. The response bound of equation 4.3 already charges the tick for blocking B_i by lower-priority tasks (50 µs) and interference I_i from higher-priority tasks and interrupts (40 µs), both from the microcontroller's own workload (section 4.5.1). Delay from hardware shared with the proposer is a separate cost because it lengthens the execution term C_i itself. A loaded tick meets its deadline only if its execution time, including all shared-hardware interference, stays below the deadline minus those internal overheads (90 µs), leaving an execution ceiling of 310 µs. Under the declared WCET bound, shared-hardware contention can add at most 60 µs of delay before the tick misses its deadline, and every placement analysis must account for this full budget.

The declared bound, not the deadline, is what a placement must defend. It bounds the tick's execution in isolation, and the chosen placement claims that proposal-side load adds nothing to it. A loaded tick that runs longer than 250 µs therefore refutes that claim even when it finishes before the 400 µs deadline, because every margin downstream of the bound was computed on the premise it just broke. Each coupling in this chapter is priced against that *falsification threshold*, which section 13.7 formalizes and turns into a test campaign.

On the application processor, meanwhile, the neural policy's 40 ms inference latency leaves 10 ms of margin within its 50 ms period. The enforcer's sub-costs above are a baseline model rather than an integrated measurement, and local spatial-query overheads must also fit inside the cyclic budget. Full qualification requires measuring the loaded control cycle on the target to establish which spatial queries fall inside constraint evaluation and to account for any separate overheads. Neither an unloaded benchmark nor an isolated query model can verify the declared WCET bound.

↩ The spatial-query workload the safety microcontroller must answer each tick is established in section 9.6.1.

13.2.2 Shared failure domains and attack surfaces

A single stalls-per-tick coupling across shared silicon can consume the entire 60 µs shared-hardware allowance even when the proposal and permission paths run on separate processor cores.[4] Beneath software access controls, four physical channels can bind the two paths without appearing on a software architecture diagram: memory subsystems, power rails, thermal paths, and clock distribution networks. The physical architecture from Part I is designed to sever each of them at the board level.

Memory contention is the first and often most severe coupling. On cores sharing an address space, operating-system page-table updates trigger inter-processor interrupts for Translation Lookaside Buffer (TLB) shootdowns,[5] while coherent direct memory access (DMA) from high-throughput camera pipelines incurs cache-invalidation stalls.[6] More fundamentally, a time-critical control loop cannot read its sensor buffers if an asynchronous accelerator monopolizes the physical interconnect. In a Unified Memory Architecture (UMA), neural accelerators, application cores, and real-time cores arbitrate for a single pool of DRAM across a shared crossbar. A high-resolution vision pipeline or an aggressive DMA engine can then saturate the bus and starve the permission routine of encoder and IMU feedback, whatever its software thread priority. This mismatch between software priority and unmanaged interconnect contention is the **UMA crossbar trap**, and section 13.3 prices the queueing delays and bank conflicts behind it. The chosen design avoids the trap by giving the safety controller exclusive ownership of its memory. Candidate proposals cross an isolated board-level serial bus into dedicated local SRAM, so no DRAM is shared between the two domains.

Power delivery represents the second coupling vector. Although the host processor and the safety controller draw from a common battery, they do not share a power rail. The host application processor draws from the nominal 47.7 V motor drive bus through a dedicated DC-DC step-down converter that triggers an undervoltage lockout (UVLO) if the supply sags to 33 V. The steady-state margin analysis in section 2.6 ensures that even a depleted battery pack remains above this threshold during simultaneous drive acceleration and peak neural inference. Unmodeled transients or supply faults (a blown fuse, or a battery management system (BMS) contactor trip), however, can depress the bus and crash the application processor. Had the safety microcontroller shared this unisolated supply, that same brownout would reset the safety monitor simultaneously. Motor gate drivers incorporate internal hardware protections for such events: integrated undervoltage lockouts disable power MOSFET outputs when gate-drive rails collapse, and pull-down resistors prevent floating logic pins from triggering destructive shoot-through currents. Those protections mitigate electrical switching faults; they do not establish a harmless coast or a controlled stop. A shared brownout would also extinguish the permission path, leaving no MCU-controlled response to manage the robot's kinetic energy. The dedicated, held-up rail preserves supervision while the host browns out and reboots. The safety microcontroller detects the bus undervoltage interrupt and commands the fault response whose resources survive. Powered deceleration requires an intact drive-energy path and regenerative sink; if the supply fault removes either, the separately qualified mechanical or passive response governs.

Thermal dissipation and clock distribution introduce the third and fourth coupling channels. When co-located on a shared silicon die or bonded to a common heat spreader, heavy sustained compute workloads on the application processor raise junction temperatures toward thermal trip points. Dynamic voltage and frequency scaling (DVFS) governors configured to protect the silicon package can throttle clock frequencies in shared clock domains, slowing real-time safety cores that share no code with the proposal pipeline. As demonstrated in section 13.4, thermal throttling introduces hysteresis that outlasts the transient compute burst that triggered it. Similarly, relying on a shared external crystal oscillator creates a single point of failure, because thermal drift, mechanical shock, or a fractured crystal desynchronizes both execution domains at once. A discrete microcontroller removes the shared die and oscillator dependencies by providing a physically separated package,

4 | **Multicore Interference Certification (FAA AC 20-193)**: Avionics met this problem first. The 2016 CAST-32A position paper named shared caches, interconnects, and memory controllers as interference channels, and FAA Advisory Circular AC 20-193, issued in 2024 and aligned with EASA's AMC 20-193, replaced it with formal acceptance criteria (Federal Aviation Administration 2024). Its lesson for this machine is that a timing bound measured on one core is void on a multicore until every shared row of table 13.1 has been analyzed or mitigated.

5 | **TLB Shootdown**: When an operating system modifies page tables, it broadcasts inter-processor interrupts (IPIs) to invalidate stale Translation Lookaside Buffer entries across all active cores. Receiving cores must pause thread execution and flush affected entries, adding unbudgeted multi-microsecond stalls. On shared silicon, these synchronization pauses break the cycle determinism needed by inner motor loops.

6 | **DMA Cache Coherence**: Direct memory access transfers bypass CPU caches, requiring explicit cache-line invalidation or writeback to keep peripherals and cores consistent. Without hardware snoop filtering or explicit barriers, processors read stale sensor frames or overwrite pending actuator commands. These cache maintenance steps inject variable delay into high-rate control loops unless static, uncached SRAM regions are dedicated to peripheral transfers.

dedicated heat dissipation, and an independent quartz crystal oscillator. Its own clock, supply, and thermal operating limits, including exposure to the common chassis environment, still require qualification.

Any safety-critical design must still evaluate the alternative it rejects, a single shared die carrying both paths. On one die the four couplings collapse into one semiconductor package, so real-time cores that execute their instructions independently of the application cores remain coupled through each resource. Bursts of camera DMA, neural accelerator memory access, and high-volume diagnostic logging can stall the real-time cores' cache fills and delay their interrupt handling unless hardware arbiters enforce deterministic bandwidth caps. At the same time, high-slew current transients can droop a shared core rail and a software-initiated host reset can take down a common reset chain unless power and supervisor domains are strictly isolated. Even when memory and power are partitioned, heat from the neural processing units raises the substrate temperature under the real-time cores, which share the package's only route to the chassis. A unified on-die governor throttles the real-time cores' clock together with the accelerator's when that substrate temperature reaches its trip point. A shared PLL or crystal also carries any clock fault to both.

Beyond accidental contention, these shared silicon pathways expose a **cyber-physical attack vector**. An adversary who compromises the unprivileged proposal stack can generate malicious memory traffic or toggle accelerator power states rapidly. The traffic starves the permission routine of memory service, and coordinated current surges pull the shared supply down (section 13.3 prices both). An isolation claim must therefore hold against the worst contention the proposal domain can generate within its authorized hardware limits, and benchmarks of benign workloads cannot establish that bound.

Functional-safety engineering provides formal tools to analyze this shared inventory. On the mobile manipulator, the intended certified safety scope is deliberately narrow; this illustrative platform has not established certification. The trajectory admission algorithms that validate learned proposals carry no formal safety integrity rating; the intended integrity claims belong to the scoped protective stop functions implemented in the motor drives and safety laser scanner. The function selected for a safety rating on the microcontroller is the watchdog that revokes authority when the chunk lease expires or the host heartbeat falls silent. It then commands a certified drive-level stop, safe stop 1 (SS1, a controlled deceleration followed by torque removal) or safe torque off (STO), whichever the risk assessment assigns to the machine's state. Keeping that intended rated function apart from unrated software components (the trajectory admission checks and the neural proposer) is a formal *safety decomposition*. The decomposition requires evidence that faults and interference from the unrated elements cannot defeat the claimed protective functions under the declared fault model. Workload placement determines which shared dependencies that evidence must address. A **dependent failure analysis** identifies the shared hardware resources whose failure or overload could compromise both paths at once. ISO 26262 Part 9 (International Organization for Standardization 2018) requires it, and IEC 61508 (International Electrotechnical Commission 2010a) treats it as common-cause failure analysis. The warehouse machine answers to industrial and mobile-robot safety standards rather than automotive ones (ISO 10218-2:2025, for instance, incorporates the collaborative-robot protections first defined in ISO/TS 15066 (International Organization for Standardization 2025b, 2016)), but the requirement of freedom from interference is the same in every domain. The resource audit of this chapter is that dependent-failure analysis carried out on physical silicon, with every coupling priced in the tick's timing budget.

Because each shared dependency names a physical or adversarial mechanism that could defeat the decomposition, qualification must test the isolation by injecting faults rather than by reviewing schematics. The timing stress campaign designed in section 13.7 validates these boundaries against the declared WCET threshold across four stress regimes, three aimed at the supply, clock, and link and one at the proposals themselves:

1. *Power-rail droop testing*: Injecting maximum-slew current transients into the drive bus to verify that the noise coupled onto the safety rail stays inside the microcontroller's specified supply tolerance, the limit the placement record's droop field carries.
2. *Clock glitching and frequency margining*: Verifying that the real-time core maintains deterministic instruction cycle counts under clock jitter and voltage fluctuation.

↩ The certified drive functions beneath the permission path, and the stop each failure triggers, are established in section 12.9.3.

↪ The industrial and mobile-robot standards that govern this machine, and where each one's evidence is generated, are developed in section 16.3.4.

3. *Communication link flooding*: Saturating shared interconnects to confirm that safety-critical heartbeat frames and trajectory proposals retain their deterministic transmission windows.[7]
4. *Numerical and semantic fault injection*: Streaming synthetic proposals populated with out-of-range floats, subnormal numbers, and IEEE 754 NaNs to verify single-cycle rejection without pipeline stalls or exception traps.

[7] **CAN Arbitration**: Controller Area Network protocols resolve bus contention at the physical layer using wired-AND dominant ('0') and recessive ('1') signaling. If a defective transceiver continuously drives a dominant state (a "babbling idiot" fault), it monopolizes the shared wire and blocks all other traffic. Hardware bus guardians and dominant-timeout clamp circuits time each transmission and physically disconnect jabbering nodes to keep the bus available for remaining safety nodes.

↪ The physical injection apparatus for supply, clock, and bus disturbances is developed in section 15.8.

13.2.3 Four candidate placements against one shared-resource inventory

On the mobile manipulator's compute board, four candidate placements for the permission loop span the options from sharing every silicon resource to sharing only the battery (table 13.1). Scoring each candidate row by row exposes the common-cause faults that a high-level chip block diagram hides, including whether a commercial module actually routes an on-die safety island's pins and rails to the integrator. At the least isolated extreme, *shared application cores* (a) expose the permission loop to operating-system scheduling jitter, driver interrupt stalls, and kernel preemption tails, delays that can push execution past the 400 µs deadline and even the 1 ms control period. Moving to *a real-time cluster in the host clock and power domain* (b) dedicates execution cores to safety enforcement, yet it still shares the die's cache hierarchy, memory controller, core supply rail, clock governor, and thermal path. Section 13.3 prices its memory exposure, and section 13.4 shows the clock throttling that rules this candidate out.

To sever those host-domain couplings without leaving the package, modern automotive and industrial SoCs provide *an on-die safety island* (c). For instance, the Functional Safety Island of NVIDIA's Orin-class SoCs integrates four dual-core lockstep Cortex-R52 processors, each with private tightly coupled memory, around 3 MB of on-island SRAM (NVIDIA 2023b). The island runs from an independent crystal oscillator and a dedicated voltage rail behind hardware firewalls that block unauthorized access from the rest of the die. The application governor therefore cannot throttle its clock, and an enforcement routine whose code and data stay in island memory never waits on host DRAM arbitration. Even so, several couplings remain inside the package: shared substrate temperature, common reset and power sequencing logic, arbitration across the on-chip fabric carrying proposals or island DMA, and the common board-level power supply. Adopting an on-die island therefore requires reading back the configured firewall registers, tracing the power-sequencing logic, and running the loaded stress campaign of section 13.7 while the safety loop executes. Where those remaining die-level couplings cannot be bounded or the module does not expose the island, *a discrete safety microcontroller* (d) removes every on-die coupling altogether, trading silicon integration for an explicit board-level link from the application processor that section 13.6 budgets and characterizes.

Laid out on the machine's compute board, the four candidates differ in how many shared blocks stand between the permission tick and the proposer (figure 13.1). Candidates (a) and (b) sit behind the application die's last-level cache, DRAM controller, clock governor, core rail, and fabric. The island (c) moves cores, crystal, and SRAM behind a firewall but stays in the same package and heat path. Only (d), a separate package clocked at 400 MHz with local TCM and a 2 s rail hold-up, leaves the die entirely and reaches it through a checked board link.

Table 13.1: Dependent-failure inventory of four placements: Each row is a resource whose fault or overload could delay or disable both the proposer and the permission path. "Separated" means the placement removes the resource from the permission tick's path; it is still a claim that register readback and loaded tests must support. Only the discrete controller removes the die-level rows, and none of the four removes the common board supply.

Shared resource	(a) Shared cores	(b) RT cluster, host domain	(c) On-die safety island	(d) Discrete MCU
Cores and scheduler	Shared	Separated	Separated	Separated
Last-level cache and miss handling	Shared	Shared	Separated (private caches, TCM)	Separated (TCM)
DRAM controller and channels	Shared	Shared	Separated if the tick stays in island memory	Separated
On-chip fabric carrying proposals	Shared	Shared	Shared, behind a firewall	Replaced by a board link

Shared resource	(a) Shared cores	(b) RT cluster, host domain	(c) On-die safety island	(d) Discrete MCU
Core supply rail	Shared	Shared	Separated (dedicated rail)	Separated (isolated rail)
Clock source and governor	Shared	Shared	Separated (dedicated crystal)	Separated
Reset and boot chain	Shared	Shared	Evidence needed	Separated if the supervisor is independent
Die temperature and package	Shared	Shared	Shared	Separated
Board supply and battery	Shared	Shared	Shared	Shared; evidence needed
Proposal transport	In-process memory	Shared-memory mailbox	Shared-memory mailbox through the firewall	Board link into a local mailbox

On the inventory alone, candidate (c) is a strong contender. It separates the cache and miss handling, DRAM, core rail, and governed clock, the rows behind the strongest mechanisms the next two sections price. It leaves shared the die temperature, the fabric its proposals cross, the board supply, and a reset chain whose independence still needs evidence, each of which a qualification would have to cover. This verdict is provisional, since nothing has yet been priced, but the machine does not wait on it, because it adopts candidate (d) for a reason outside silicon capability. While the safety island exists on the physical SoC die, commercial edge modules packaging the application processor do not expose it to system integrators, reserving access for dedicated industrial platforms (table 13.2). Because an unexposed, unprogrammable safety island cannot serve as a valid deployment target, safety enforcement must move off the primary module.

DRAFT

Table 13.2: Safety-island exposure on edge AI platforms, as of 2026: The same silicon can carry an island that one product line exposes and another does not. Which island a placement can use is a property of the module and its software support, not of the SoC's block diagram, and a later edition should refresh this table.

Platform (vendor statement date)	Safety island on the SoC	Available to the integrator	Source
Jetson AGX Orin module (2023)	Cortex-R52 island present	No	NVIDIA (2023a)
IGX Orin platform (2023)	Same island	Announced as offered	NVIDIA (2023a)
DRIVE AGX Orin (2023)	Island with dedicated crystal and rail, 3 MB SRAM	Yes, through DRIVE OS	NVIDIA (2023b)
Jetson Thor module (2026)	Not supported	No	NVIDIA (2026a)
IGX Thor platform (2026)	Supported	Yes	NVIDIA (2026a)

Commercial platform exposure is therefore itself an architectural constraint. A vendor block diagram illustrating isolated silicon domains establishes what the hardware could theoretically execute, but system developers can use only the execution resources unlocked by board support packages and platform firmware. While specific commercial product tiers change over time, the engineering requirement to audit software accessibility before committing to a workload placement remains permanent.

The third law (principle 3) says a proposal is not permission, and that separation is empty if the proposal can delay the permission decision through hardware the two share. The four candidates differ in which resources they force the paths to share, and even for the discrete microcontroller the separation remains a hypothesis about timing under load that only a measurement of the loaded tick can support.

Choosing candidate (d) before pricing the shared die does not make the quantitative analysis of shared resources superfluous, for three reasons. First, each mechanism priced in section 13.3 and section 13.4 turns the rejection of candidates (a) and (b) from an architectural assumption into empirical evidence. Second, each mechanism names an obligation that the discrete design carries in miniature, because the microcontroller has its own DMA engines, interrupt controller, internal

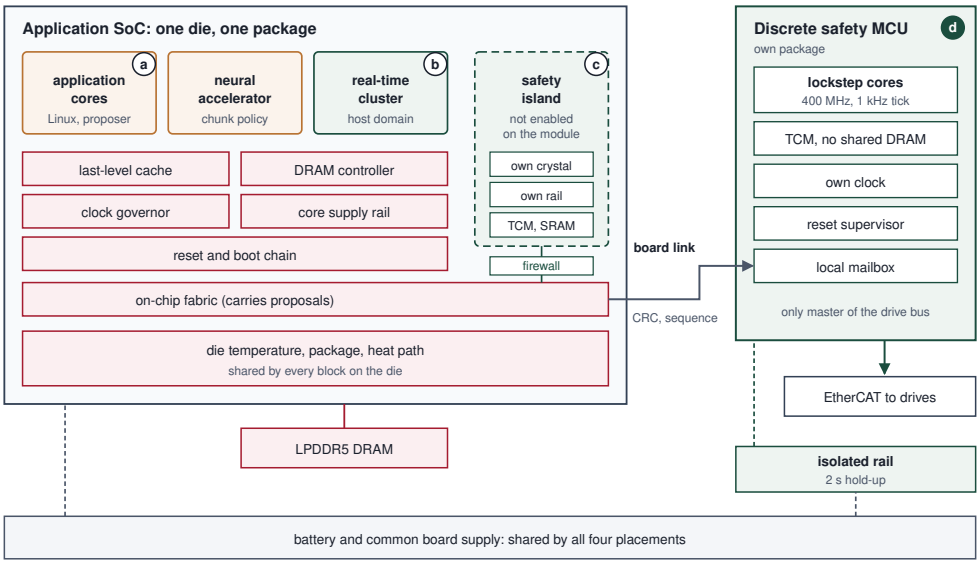


Figure 13.1: Where each candidate placement runs: Badges (a) to (d) mark where the permission tick would execute. Tinted blocks are shared with the proposer on the application die (cache, memory controller and external DRAM, clock governor, core rail, reset chain, fabric, and the package's heat path), so a tick placed on the die can lose them to proposal-side load. The safety island carries its own crystal, rail, and memory but remains behind the shared fabric and in the shared package; the discrete controller, the machine's choice, reaches the die only through a checked board link and still shares the battery and chassis environment with it.

DRAFT

buses, and power supply, as detailed in section 13.6.2 and section 13.7. Finally, the same analysis provides the qualification blueprint for an integrator whose platform firmware exposes an on-die safety island. Unless a passage says otherwise, the calculations in section 13.3 and section 13.4 evaluate the permission tick placed hypothetically on the application die, using the sub-costs and timing budget assigned to the microcontroller. On a shared die, the first place proposal-side load encounters that tick is the memory system.

13.3 Contention for Shared Resources

Had the mobile manipulator placed its permission path on the application SoC, camera DMA, neural weight fetches, telemetry writes, cache writebacks, and enforcement reads would compete for memory service. Their simultaneous bursts determine the queue seen by the enforcer. On the selected machine, those host-side bursts delay the observation and chunk rather than the permission tick. The permission path encounters their effects as evidence age, refusing stale evidence and entering fallback if no admitted chunk renews the lease.

↩ How host-side memory contention, including camera DMA, ages observations is established in section 8.5.

13.3.1 DRAM bank conflicts, AXI4 burst slicing, and arbiter starvation

A small urgent read can wait behind a large transfer even when the memory bus is mostly idle over a longer interval. The deadline depends on the traffic already ahead of that read and on whether the controller lets it pass. This queueing delay, which arises when memory and DMA transactions share an interconnect and controller, is the mechanism inside the UMA crossbar trap, and its bound depends on transaction size, queue policy, and bank timing.

Each claimant's average demand follows from its transfer size and event rate. On a shared multi-channel LPDDR5 interface, camera DMA streams, weight fetches at the policy rate, the enforcer's reads, drive writes, cache writebacks, and logging together occupy a modest fraction of theoretical peak bandwidth on average. An engineer who inspects only that average concludes that most of the memory bus remains free. Memory crossbar arbiters and DRAM schedulers do not schedule average rates. They arbitrate discrete, bursty transactions, and two workloads with identical average

throughput interfere with radically different latency profiles if one streams in small, sporadic packets while the other arrives in long, saturating blocks. The weight transfers that set the memory wall (Wulf and McKee 1995) of section 3.4.1 are the long blocks here, and on a shared controller the proposer's throughput limit becomes the permission path's queue.

How long a safety read waits depends on transaction arbitration and DRAM scheduling. An accelerator moving weights in large 256 KB tiles generates discrete memory transactions across the internal system interconnect. The example assumes an arbiter that completes each granted burst before serving a different requester.[8] Although bus protocols provide quality-of-service (QoS) priority signals (`ARQOS`), commodity memory controllers frequently treat these tags as advisory unless strict hardware priority bypass with bounded preemption latency is explicitly certified in register configurations. Whether an in-flight transfer can delay the urgent read, and whether queued tensor transactions can pass it, depends on the selected interconnect and arbiter. AXI burst limits alone do not establish a service-time or starvation bound.

Every memory request passes through four stages: the requester issues it, the interconnect arbiter grants it a slot, the memory controller queues it, and the target bank serves it. Each DRAM bank holds one open row in its **row buffer**. An LPDDR5 channel has 16 banks, divided into 4 bank groups at the higher data rates, or 8 larger banks, each formed by merging two, in an alternative mode (Steiner et al. 2022). A read to the open row (a *row hit*) needs only a column access. A read to a different row of the same bank (a *row conflict*) must first close the open row with a precharge (t_{RP}) and then open the new one with an activation (t_{RCD}). The memory standard sets a minimum time in nanoseconds for each step, 18 ns for both, and the controller can only wait whole cycles of its command clock. At the 6,400 MT/s data rate of the machine's application SoC, that clock ticks every 1.25 ns, each step rounds up to 15 cycles, or 18.75 ns, and a conflict adds 37.5 ns (H. Luo et al. 2024). Bank-group spacing, refresh, and bus turnaround add further terms, which section C.6 treats.

The controller also decides the order of service. Controllers that use **first-ready, first-come-first-served (FR-FCFS)** scheduling serve ready row hits before older requests that need a new row, which maximizes throughput (Rixner et al. 2000). A streaming accelerator supplies an unbroken run of row hits across its weight matrices, so an FR-FCFS scheduler can keep passing the safety core's sporadic read until an internal starvation counter expires. Under a FIFO queue that lets no later arrival pass the safety read, the idealized wait is

$$t_{\mathrm{queue}} = \sum_{k \in Q} \frac{D_{\mathrm{vol}_k}}{\mathrm{BW}_{\mathrm{peak}}} + N_{\mathrm{conflict}}(t_{\mathrm{RP}} + t_{\mathrm{RCD}}) \tag{13.1}$$

where Q holds the transactions ahead of the read, D_{vol_k} is each one's byte count, N_{conflict} counts changes to a different row within an already active bank, and $\mathrm{BW}_{\mathrm{peak}}$ is an optimistic service rate. The application SoC achieves that aggregate peak bandwidth across parallel 16-bit LPDDR5 channels, each containing its own sixteen internal banks, so dividing a backlog by $\mathrm{BW}_{\mathrm{peak}}$ holds only when physical address bits interleave transactions evenly across all channels rather than funneling them into a single channel queue. Arbitration overhead and refresh add to the result. Exceeding the sensing allowance first violates the allocation to that subtask; it does not by itself imply that the whole tick misses its deadline. The worked comparison below keeps the sense allocation, declared execution bound, and physical deadline separate.

[8] **AXI4 (Advanced eXtensible Interface 4)**: Arm's AMBA on-chip bus protocol permits incrementing bursts of up to 256 beats and forbids any burst from crossing a 4,096-byte address boundary. At a 16-byte beat, one full burst is exactly one such page, which is why the notebook below splits a weight tile into one transaction per page. An urgent read that waits for the granted burst to finish, as this example assumes, is a property of the chosen arbiter, not an AXI guarantee.

Napkin Math 13.1: The backlog budget

Problem: Suppose the same permission tick, with the same sub-costs and the same blocking and interference budget, ran on a core of the application SoC and read its sensor snapshot from the SoC's shared DRAM. How much queued traffic could precede that read before the tick exceeds its sense allowance, its declared worst-case execution time, and its deadline?

Variables:

- Sense allowance 40 µs and 95 µs of constraint evaluation and drive-frame write (Illustrative sub-costs), declared worst-case execution time $T_{\mathrm{wcet}} = 250$ µs and deadline $T_{\mathrm{deadline,enf}} = 400$ µs (Chosen), with 90 µs of blocking and interference charged as in equation 4.3 (Illustrative).

- Peak DRAM bandwidth $\mathrm{BW}_{\mathrm{peak}} = 204$ GB/s (Datasheet), served at that aggregate rate because the controller interleaves addresses across all channels (Illustrative premise).
- One backlog ahead of the read: a weight tile (64 transactions), a camera transfer, and a telemetry record, 88 transactions and 360,448 bytes, with 3 same-bank row conflicts (Illustrative), each costing $t_{\mathrm{RP}} + t_{\mathrm{RCD}} = 37.5$ ns (Datasheet). The snapshot itself is 4,096 bytes (Illustrative).

Math: By equation 13.1, one backlog waits 360,448 B / 204 GB/s + 3 × 37.5 ns = 1.879 µs, and the snapshot then reads in 0.020 µs. The number of backlogs n that fit a read ceiling R is $\lfloor (R - t_{\mathrm{own}})/t_{\mathrm{queue}} \rfloor$. The sense allowance gives $\lfloor (\ 40 - 0.020\ )/\ 1.879\ \rfloor = 21$. The declared bound leaves 155 µs for the read after the 95 µs of later work, so it is crossed beyond 82 backlogs. The deadline, less blocking and interference, leaves 310 µs for the tick and 215 µs for the read, and it is crossed beyond 114. With validated priority bypass, only the one in-flight transaction waits ahead, and the read takes 0.04 µs.

Result: Between 82 and 114 backlogs, the tick still meets its deadline but exceeds T_{wcet}. Within this stipulated FIFO model, the fixed isolated sub-costs remain unchanged, so the excess comes from shared-memory queueing. It refutes the placement's timing premise even though this tick was on time. A measured overrun would instead require distinguishing an underestimated baseline from interference, as section 13.7 explains.

Systems insight: The read's bound belongs to the memory controller, not to the workload. The bound exists only if the controller limits how much traffic it admits ahead of a safety read or lets that read bypass the queue. An FR-FCFS controller serving a streaming accelerator provides neither unless its configuration is qualified to do so. At the SoC's full bandwidth, though, plain queueing is a weak threat, since the allowance absorbs 21 backlogs. The interleaving premise carries much of that result. If the backlog and the snapshot shared one of the interface's 16 channels, each backlog would take 28.4 µs, the allowance would absorb 1, and more than 5 would cross the declared bound. The address mapping therefore belongs in the placement evidence. The mechanisms that actually threaten the declared bound are the ones this FIFO model omits, which section 13.3.2 prices next.

Drawn to scale on one 1 ms period, the thresholds show how far the declared bound sits from the deadline and the period (figure 13.2). The unloaded tick ends at 135 µs. The next three rows carry the largest backlog counts that keep each bound: 21 for the 40 µs sense allowance, 82 for the declared 250 µs, and 114 for the 400 µs deadline once 90 µs is reserved for blocking and interference. The bottom row confines the traffic to one memory channel, where 5 backlogs already reach the declared bound.

13.3.2 Which interference mechanism threatens the declared bound

The notebook's arithmetic makes the shared die look safe, and that impression is the danger. The permission tick can absorb a stretch of its unloaded 135 µs cycle by a factor of 1.85 before it crosses the declared worst-case execution time. The allowed stretch is 2.30 before the tick crosses the deadline, once blocking and interference take their 90 µs. Bandwidth queueing at the SoC's full rate does not come close. Two other mechanisms do, and both live in the shared cache rather than in DRAM.

A non-blocking cache tracks each outstanding miss in a *miss status holding register* (MSHR) and parks each dirty line it evicts in a *writeback buffer* until DRAM accepts it. Both structures are small and shared by every core behind the cache. When a streaming co-runner keeps them full, each miss of the victim core waits until an entry drains, so the victim's delay grows with its own miss count times the aggressor's drain time, not with the bytes it reads. Partitioning cache capacity does not help, because the partition divides the lines and leaves the MSHRs and the writeback buffer shared. Measured on commercial embedded multicores, MSHR contention raised a task's worst-case execution time by up to 21× with the cache partitioned (Valsan et al. 2016). A deliberate attack on the writeback buffer slowed a victim on a dedicated core by more than 300×, partitioned or not (Bechtel and Yun 2019). DRAM adds its own multiplier on top. An analytical bound for commercial controllers shows that each of the victim's requests can wait behind every request that other cores

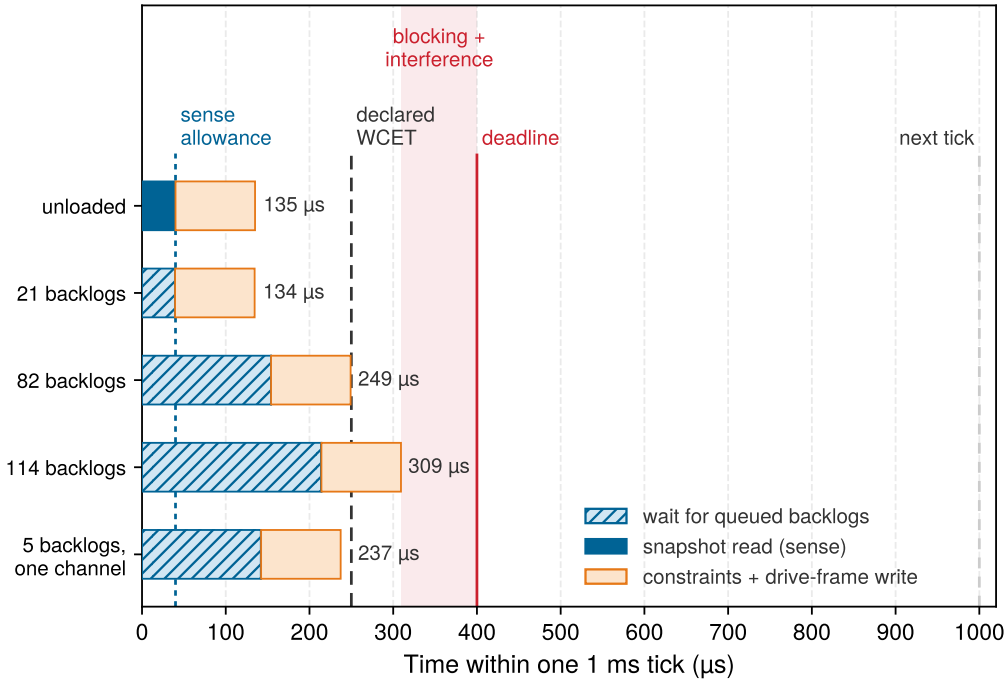


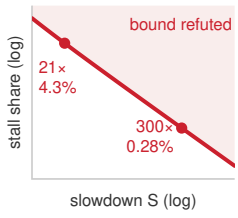
Figure 13.2: How many backlogs one tick absorbs: Each row is one permission tick on the rejected shared-die placement, drawn to scale on the 1 ms period, with the FIFO wait for queued backlogs ahead of the snapshot read followed by the 95 µs of constraint evaluation and drive-frame write. With addresses interleaved across the interface, 21 backlogs fit the sense allowance, 82 the declared bound, and 114 the deadline once blocking and interference take their 90 µs; confined to one channel, 5 already reach the declared bound. Every row still ends well inside the period, which is why the declared bound, not the period, is the test the placement must pass.

may place ahead of it in the same bank. That number is set by queue depth and the reordering window rather than by average bandwidth (Kim et al. 2014).

These slowdowns were measured on other platforms, on victims whose time was mostly memory stall, so they cannot be charged to the enforcer directly. Using them as assumed multipliers on its memory-stall share provides a sensitivity calculation, not a prediction of target behavior. If a fraction f of the unloaded tick waits on shared-cache misses and interference slows that fraction by a factor r_{slow} while the rest runs unchanged, the tick takes $t_{\text{unloaded}}[(1-f)+fr_{\text{slow}}] = t_{\text{unloaded}}[1+f(r_{\text{slow}}-1)]$. Setting that no larger than T_{wcet} and solving for f shows that the loaded tick stays within the declared bound only while

$$f \le \frac{T_{\text{wcet}}/t_{\text{unloaded}} - 1}{r_{\text{slow}} - 1}. \tag{13.2}$$

At the MSHR-class 21×, the declared bound falls once more than 4.3 percent of the tick is miss stall, and the deadline once more than 6.5 percent is. At the writeback-buffer attack's 300×, the threshold is 0.28 percent. A target profile must establish whether the constraint evaluation exceeds those fractions; touching tables in shared memory alone does not establish its stall share. The mechanism that threatens the declared bound is therefore miss handling in the shared cache, amplified by bank queueing. The only placements that remove it are those whose tick runs from private memory, candidates (c) and (d) of table 13.1. On (a) and (b), the measured stall share on the target and a regulation mechanism that bounds r_{slow} (section 13.6) are what a qualification must supply.



At a 21× slowdown, 4.3 percent miss stall breaks the bound.

13.3.3 Power-rail interference

When a neural accelerator steps its current draw in nanoseconds on a core supply rail shared with a permission core, the voltage droops across the package inductance and resistance before board regulators can respond. The droop stretches logic gate delays or triggers adaptive clock stretching

inside the permission tick. The rail at issue here is not the drive bus or the microcontroller's isolated rail; it is the on-die core supply that a shared-die placement, candidate (a) or (b), would share with the accelerator inside one package. A sudden accelerator current step lowers the voltage available to every core on that supply, and the first-order package model is

$$\Delta V_{\text{droop}} = L_{\text{pkg}} \frac{dI}{dt} + \Delta I \cdot R_{\text{pkg}} \tag{13.3}$$

The placement decision depends on whether regulation, decoupling, and stored energy keep the permission path inside its characterized operating range. The package hierarchy and transistor-level delay model are developed in section C.5; here the model identifies the disturbance a shared core rail would carry into the permission tick. The chosen placement keeps its core supply out of that package, and its isolated rail must instead survive board-level events (drive-bus sag, a contactor trip, converter dropout), the disturbances of stress regime 1 and the transient generators of section 13.7.

For the illustrative package parameters, a 6 A step with a 2 ns rise through 30 pH gives 90 mV of inductive droop. The same step through 8 mΩ gives 48 mV of resistive droop, 138 mV in all from a 0.850 V nominal rail. This pure $L_{\text{pkg}}\, dI/dt$ term ignores on-die decoupling capacitance, and does not resolve the actual transient. Power-integrity engineers instead design the delivery network to a target impedance[9] across frequency, and the first droop is set by package inductance resonating with on-die capacitance. The alpha-power model[10] (Sakurai and Newton 1990) relates the resulting voltage to gate delay:

$$\frac{t_{\text{prop}}(V_{\text{droop}})}{t_{\text{prop}}(V_{\text{nom}})} = \frac{V_{\text{droop}}}{V_{\text{nom}}} \left(\frac{V_{\text{nom}} - V_{\text{th}}}{V_{\text{droop}} - V_{\text{th}}} \right)^{\alpha}$$

Here V_{th} is the modeled threshold voltage, stipulated as 0.420 V, and α is the technology-dependent exponent, stipulated as 1.3. With those inputs, the modeled delay increases by 39 percent. This is a sensitivity calculation, not a measured reset threshold or a universal delay increase for every gate, and ignoring decoupling does not establish a guaranteed upper bound on a resonant transient. The stretch lasts nanoseconds, so it does not lengthen the tick by that percentage. It threatens the setup-time margin of the paths switching during the droop, and it grows without bound as the drooped voltage approaches V_{th}. Many current processors detect the droop and stretch the clock until the rail recovers (Grenat et al. 2014), which turns a timing violation into lost cycles that the tick must absorb. Device characterization must establish whether the actual critical paths still meet setup time, how many cycles adaptive clocking removes, and whether undervoltage protection trips.

9 | **Target Impedance**: Power-integrity engineers hold the delivery network's impedance, seen from the die, below the allowed ripple divided by the largest current step, at every frequency the load can excite (Smith et al. 1999). A flat impedance bounds the droop from a current step of any shape, which a single $L_{\text{pkg}}\, dI/dt$ term cannot. That documented bound is what a shared-die placement would owe the permission core's rail, and what the discrete placement owes only its own isolated rail.

10 | **Alpha-Power Law**: The model was proposed in 1990 (Sakurai and Newton 1990) because short-channel transistors no longer followed the long-channel square law: carrier velocity saturates, and drain current grows as $(V - V_{\text{th}})^{\alpha}$ with α falling from 2 toward 1. Delay therefore depends on the headroom between supply and threshold, not on the supply alone. A low-voltage core chosen for efficiency has the least headroom, so it is also the most droop-sensitive home for the permission path.

> Checkpoint 13.1: Which shared resource breaks the bound
>
> Queueing, miss handling, and rail droop all delay the permission tick through hardware the two paths share. Before moving to heat, confirm that the distinctions among these physical mechanisms are clear:
>
> - ☐ **Resource inventory**: Can you list, for candidates (b) and (c) of table 13.1, which shared rows could delay the tick and which could disable it?
> - ☐ **Averages versus order**: Can you explain why a memory controller at low average utilization can still hold a safety read behind a long backlog, and what a controller must provide for the read to have a bound?
> - ☐ **Miss-stall estimate**: If interference slowed the tick's memory stalls by 10× instead of the measured 21×, what share of the 135 µs unloaded tick could be miss stall before the loaded tick crossed the declared 250 µs?
> - ☐ **Droop timescale**: Can you explain why a droop that stretches gate delay by tens of percent for nanoseconds does not stretch the tick by the same percentage, and what it threatens instead?

A separate permission rail removes the modeled shared core-supply path, but the common battery, wiring, converter behavior, and return paths remain qualification obligations. Rail probes and independent timing instruments must observe them under simultaneous motor and proposal

load. Droop and queueing both end within microseconds of the burst that causes them. The heat that the same accelerator dissipates does not, and it couples the paths on a timescale a million times longer.

13.4 Thermal and Timing Coupling

A permission core can suffer execution delays long after a heavy proposal workload has terminated, because thermal capacitance traps heat inside the package. While electrical droop and interconnect contention manifest over nanoseconds and microseconds, thermal coupling operates over seconds. When an accelerator and a real-time permission core share a silicon die, a cooling path, and an on-chip power management governor, sustained neural network inference can lower the clock frequency available to safety enforcement, and keep it lowered after the inference stops.

Thermal clock throttling is the automatic, hardware-enforced reduction of processor clock frequency triggered when sustained power dissipation pushes the silicon junction temperature (T_{j}) beyond a critical trip threshold (T_{trip}). On-die thermal sensing diodes alert the power management controller, which scales down clock frequency through hardware dividers or phase-locked loop (PLL) reconfiguration to prevent gate dielectric breakdown and thermal runaway. Because the physical die, package substrate, and heat spreader require time to dissipate this accumulated thermal energy, the reduced clock frequency persists long after the compute burst ends. For a co-located real-time task, this persistent slowdown can turn a nominal execution margin into a deadline overrun that lasts as long as the package takes to cool.

In this illustrative warehouse scenario, the robot's electronics bay operates at an ambient temperature of 45 °C. The thermal dissipation path follows the package row of figure 13.1, shared by every computational component on the application SoC die. The selected warehouse enclosure seals out airborne dust and conductive debris and uses conduction cooling rather than open fan vents. For a shared-die architecture in this enclosure, the conduction path must transfer all heat generated by sustained proposal inference to the chassis without driving the real-time permission core into a throttled clock state.

For a conduction-cooled enclosure, the total thermal resistance from junction to ambient ($R_{\text{th,JA}}$) sums three distinct mechanical layers:

$$R_{\text{th,JA}} = R_{\text{th,JC}} + R_{\text{th,TIM}} + R_{\text{th,chassis}}.$$

Here, $R_{\text{th,JC}}$ is the junction-to-case resistance of the silicon package, $R_{\text{th,TIM}}$ is the resistance through the thermal interface material, and $R_{\text{th,chassis}}$ represents heat conduction through the metal enclosure to the ambient air. Over extended operation, mechanical shock and thermal cycling can degrade the interface material, increasing $R_{\text{th,TIM}}$. Simultaneously, battery discharge losses and motor drive electronics inject parasitic heat into the bay; battery Joule heating scales with the square of the discharge current (I^2R). A robust thermal envelope must therefore account for both the mechanical cooling path and the peak concurrent actuator workload. A quiescent bench test on a cold board cannot replicate the thermal reality of a warm, multi-hour operating shift.

Consider an application SoC inside this bay modeled with a lumped **junction-to-ambient thermal resistance** $R_{\text{th,JA}}$ and a **thermal time constant**[11] τ_{th}. Under a continuous power dissipation P_{device}, the silicon junction reaches a steady-state temperature of $T_{\text{amb}} + P_{\text{device}}R_{\text{th,JA}}$. When power steps to a new level, the junction temperature converges toward this new equilibrium along a first-order exponential trajectory governed by τ_{th}. This time constant represents the fast pole of the die and package thermal network. While heatsink and structural chassis poles operate on timescales of minutes to hours, holding the chassis at a constant baseline temperature provides an optimistic lower bound on junction heating.

Under the clock and thermal assumptions below, these dynamics eliminate the modeled configuration of candidate (b) in table 13.1. In that candidate design, the permission path runs on a real-time CPU cluster located on the main application SoC. Suppose this cluster shares the SoC's primary clock and power management domain, sized so that the permission tick's worst-case cycle count, run at the nominal clock, exactly meets its declared worst-case execution time. The sizing is not chosen to make the case fail. The declared bound is the worst execution the placement may assume, so judging the placement at that bound is the honest test, and a tick that executes at its bound has no slack against any slowing of the clock. When sustained neural network inference

[11] **Thermal RC Analogy**: Heat flow is mathematically equivalent to electric current, temperature difference to voltage, thermal resistance (K/W) to electrical resistance, and heat capacity (J/K) to capacitance. Under a single dominant heat path, the silicon junction behaves as a first-order RC circuit with $\tau_{\text{th}} = R_{\text{th,JA}}C_{\text{th}}$. The parameters in this scenario imply a lumped thermal capacitance of 4 J/K, representing the stored thermal energy the package must shed through $R_{\text{th,JA}}$ before the governor can restore the clock. Much like an electrical RC low-pass filter smooths voltage spikes, this physical thermal mass integrates fast fluctuations in proposal compute power into a slow, sustained temperature rise that directly governs the permission core's clock.

drives the junction temperature beyond the trip threshold T_{trip}, the hardware governor halves the cluster's clock frequency. As a consequence, every instruction cycle in the tick takes twice as long. The safety software executes the exact same code, but the shared governor injects an unmodeled cross-domain delay that isolated execution time profiling cannot detect. A discrete microcontroller, in contrast, runs on an independent oscillator and dedicated power rail beyond the reach of the SoC's thermal governor. An on-die safety island fed by its own clock tree and voltage rail would likewise isolate clock generation from the governor, although it would still share the silicon substrate and require rigorous thermal qualification.

A throttled tick outlasts the workload burst because hardware governors enforce **state-dependent hysteresis**. To prevent the clock frequency from chattering rapidly around the trip line, the controller restores the nominal clock only after junction temperature drops below a lower clearance threshold T_{clear}. Once proposal inference halts and chip power falls back to baseline, the junction relaxes toward its baseline equilibrium:

$$T_{\text{j}}(t) = T_{\text{j,target}} + (T_{\text{j,start}} - T_{\text{j,target}})e^{-t/\tau_{\text{th}}} \tag{13.4}$$

where $T_{\text{j,start}}$ is the junction temperature when throttling begins, $T_{\text{j,target}}$ is the baseline equilibrium under quiescent load, and τ_{th} is the package thermal time constant. Solving equation 13.4 for the time required to cool from T_{trip} to T_{clear} gives $t_{\text{recov}} = \tau_{\text{th}} \ln[(T_{\text{j,start}} - T_{\text{j,target}})/(T_{\text{clear}} - T_{\text{j,target}})]$, which fixes how long the hangover lasts in this scenario.

Napkin Math 13.2: The thermal hangover

Problem: Candidate (b)'s permission cluster shares the application die's package and clock governor. Under continuous proposal inference, does the junction reach the trip threshold, what does the throttled clock do to the tick, and for how many ticks does the overrun persist after the load stops?

Variables:

- $T_{\text{amb}} = 45\,^{\circ}\text{C}$, $R_{\text{th,JA}} = 2$ K/W, and $\tau_{\text{th}} = 8$ s (Illustrative; conduction-cooled bay).
- $P_{\text{base}} = 4$ W for the permission loop with idle and memory power, and $P_{\text{load}} = 26$ W with continuous proposal inference (Illustrative). The scenario holds this loaded power constant after throttling; a real device whose governor reduces power requires a coupled power-and-temperature model.
- $T_{\text{trip}} = 95\,^{\circ}\text{C}$ and $T_{\text{clear}} = 85\,^{\circ}\text{C}$ (Illustrative governor thresholds).
- $f_{\text{nom}} = 1.2$ GHz, halved to $f_{\text{throt}} = 600$ MHz (Illustrative). The tick is sized to $T_{\text{wcet}} = 250$ µs at f_{nom} inside the 400 µs deadline and the 1 ms tick (Chosen), with 90 µs of blocking and interference charged as in equation 4.3 (Illustrative).

Math: The steady states are $T_{\text{j,target}} = 45 + 4 \times 2 = 53\,^{\circ}\text{C}$ and $T_{\text{load}} = 45 + 26 \times 2 = 97\,^{\circ}\text{C}$, above the trip. Rising from the baseline, the junction trips after $\tau_{\text{th}} \ln[(T_{\text{load}} - T_{\text{j,target}})/(T_{\text{load}} - T_{\text{trip}})] = 8 \ln[(\,97 - 53\,)/(\,97 - 95\,)] = 24.7$ s. The tick needs $N = T_{\text{wcet}} f_{\text{nom}} = 300{,}000$ cycles, which at 600 MHz take 500 µs. With blocking and interference the response grows from 340 µs to 590 µs, 190 µs past the deadline. If the load stops just as the junction trips, equation 13.4 returns the clock after $t_{\text{recov}} = 8 \ln[(\,95 - 53\,)/(\,85 - 53\,)] = 2.18$ s, which spans 2,175 ticks of 1 ms. If inference has run long enough for the junction to settle at T_{load}, cooling starts there instead and takes $8 \ln[(\,97 - 53\,)/(\,85 - 53\,)] = 2.55$ s, or 2,547 ticks.

Result: After about 24.7 s of continuous inference, every tick overruns its deadline by 47.5 percent for as long as inference continues, and for another 2.18 s to 2.55 s after it stops, depending on how close the junction had climbed to its loaded steady state.

Systems insight: While inference continues, the junction stays near T_{load}, above the trip, so the cluster remains throttled for as long as the proposer runs, which is the worse case on a machine whose chunk policy runs continuously while it moves. The hangover is the subtler one. Pausing the proposer (at a stop, or when the policy idles) does not restore the clock for at least another 2.18 s, so a remedy that sheds proposal load cannot rescue the tick in time.

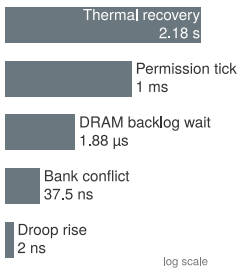
Throughout, a refusal misses the drive frame timed to the deadline and waits for a later one. Because the throttled response still ends inside the period, each late refusal slips by about one tick, a distance the stopping budget can absorb (section 13.7). The case still disqualifies this modeled configuration, because the loaded tick has crossed the declared bound, every margin downstream was computed on the premise it breaks, and nothing in the premise limits a real governor to halving the clock. The lever is the heat path, not the tick's code. Any $R_{\text{th,JA}}$ that keeps $P_{\text{load}} R_{\text{th,JA}}$ below $T_{\text{trip}} - T_{\text{amb}}$ never trips, and an independently clocked placement is unaffected by that host governor. Its own thermal and clock limits remain separate qualification obligations.

The two timescales meet in one picture: a cooling curve measured in seconds holds the clock at 600 MHz, and every tick inside it, measured in microseconds, ends 190 µs past the deadline (figure 13.3). The scenario is also optimistic. Over a full shift, heat accumulating in the chassis raises the baseline $T_{\text{j,target}}$, which flattens the cooling gradient and lengthens recovery, and higher-order thermal poles in the enclosure can stretch the hysteresis by minutes. A shared-die placement would need a separately timed hardware fallback to stay safe across every exposed tick.

Heat completes the set of couplings, and placing it beside the other two shows how far apart they sit in time. The rail droop of section 13.3 acts within nanoseconds (10^{-9} s) of an accelerator current step, while thermal coupling develops over seconds (10^{1} s) as heat diffuses through silicon, package, and chassis. The separation is also a diagnostic: a latency spike that correlates with rail voltage points to power delivery network (PDN) impedance, and a slowdown that correlates with temperature points to the governor. Together with the tick they threaten, the couplings span about thirteen orders of magnitude on one machine, from nanoseconds to hours:

1. 10^{-9} **s (Nanoseconds)**: Simultaneous tensor core switching generates $L_{\text{pkg}} \cdot dI/dt$ inductive rail collapse, stretching transistor gate delays under the α-power law within individual clock cycles.
2. 10^{-6} **s (Microseconds)**: AXI4 crossbar arbitration and head-of-line FIFO queueing stall memory reads, as row-buffer precharge and activation penalties ($t_{\text{RP}} + t_{\text{RCD}}$) of tens of nanoseconds each accumulate across a queue.
3. 10^{-4}–10^{-3} **s (Sub-milliseconds to milliseconds)**: The protected work itself. Sensor ingestion and the control barrier filter's quadratic program run inside the permission tick, and fieldbus frames (such as EtherCAT) repeat on deterministic 1 ms cycles.
4. 10^{0}–10^{1} **s (Seconds)**: Macroscopic silicon die and package heatsink thermal capacitance stores dissipated energy, enforcing governor throttling hysteresis (at least 2.18 s in this scenario) that degrades thousands of control ticks.
5. 10^{2}–10^{4} **s (Minutes to hours)**: Structural chassis thermal mass and battery Joule heating accumulate heat, shifting bay ambient temperature T_{amb} upward and eroding cooling headroom across a multi-hour shift.

Because thermal mass and heat dissipation operate across seconds and hours, dynamic thermal management (DTM) (Brooks and Martonosi 2001) and reactive thread migration cannot restore a co-located permission loop before its next deadline expires. A safety-critical permission path requires guaranteed lower bounds on clock frequency and power supply integrity that survive sustained accelerator workloads. When a shared die cannot guarantee them, moving the permission engine onto an independently clocked and independently powered microcontroller removes the host-governor coupling, though the microcontroller must still stay within its own qualified supply, clock, and thermal limits under the common chassis conditions.



Coupling on one chassis spans nanoseconds to seconds.

Checkpoint 13.2: Junction temperature and clock throttling

The cooldown and overrun in figure 13.3 follow from the stipulated thermal scenario, not from measured hardware behavior. Use them to check the consequences of the selected model:

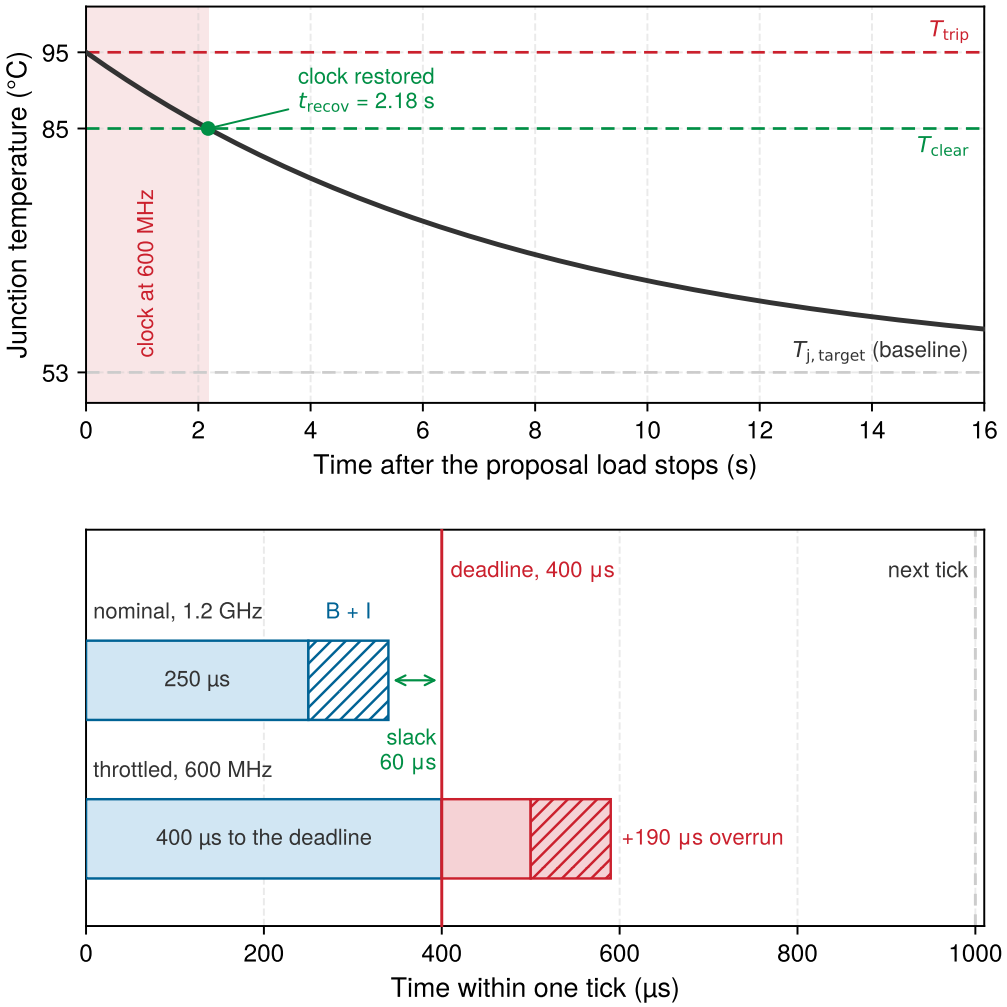


Figure 13.3: A cooldown that outlasts the load: Top, the junction relaxes from the 95 °C trip threshold toward its 53 °C baseline along equation 13.4, falling fastest at the moment the load stops and flattening afterward; the clock stays halved until the curve crosses 85 °C at 2.18 s. Bottom, one tick drawn to scale, with 90 µs of blocking and interference after the execution: the same work that meets the deadline at 1.2 GHz overruns it by 190 µs at 600 MHz, on every one of the 2,175 ticks inside the cooldown. The scenario is stipulated, not measured.

- ☐ **Thermal memory**: Can you explain why the package's thermal mass keeps slowing a real-time task after the proposal workload that heated it has stopped?
- ☐ **Cooling recovery sizing**: How long, in seconds and in 1 ms ticks, does the clock stay throttled while an SoC with thermal time constant $\tau_{th} = 8$ s cools from $T_{trip} = 95\,^{\circ}\text{C}$ to $T_{clear} = 85\,^{\circ}\text{C}$ under a 53 °C baseline equilibrium?
- ☐ **Governor hysteresis purpose**: Can you explain the purpose of temperature hysteresis (T_{trip} vs. T_{clear}) in hardware thermal governors and describe how it prevents high-frequency clock oscillation?
- ☐ **Physical segregation decision**: What measured clock, power, and timing evidence would justify a shared die, and when would a separate controller be needed?

13.5 The Boundary Contract

Relocating the permission path isolates the execution domains on separate processors, but the two still have to talk. Consider an upstream proposal writer, such as the chunk policy on the application processor, that stalls or crashes halfway through publishing an action chunk. The downstream safety enforcer running on the real-time microcontroller still owes the physical plant a deterministic actuation decision on its very next tick. Partitioning the compute engines isolates them only if the protocol lets the real-time reader inspect and reject an incomplete or in-flight record without ever waiting for the producer. The boundary contract therefore ties memory consistency directly to the deadline of the sensing and constraint-checking pipeline, and it is the one piece of the design that every placement, discrete or shared, must get right.

13.5.1 Immutable record mailboxes and non-blocking synchronization

Passing raw pointers into an application processor's working buffer or coordinating across a blocking lock lets a slow or crashed planner expose the real-time reader to torn data structures, stale cache lines, or unbounded waits. To prevent those stalls, the boundary contract demands an immutable, self-contained record whose publication integrity the safety enforcer can verify in constant time. The proposal header and seqlock mechanism fix the message structure and publication rules, and workload placement adds the requirement to verify on the hardware that this channel meets its execution budget even when the interconnect is saturated.

Across heterogeneous silicon or separate physical chips, communication splits into two stages: link transport across the board and local publication inside the microcontroller's memory domain. An ingress peripheral or DMA engine receives the payload over a physical link (SPI, PCIe, or Ethernet) and stages it into local memory. Where the target allows peripheral access to tightly coupled memory, this may be the mailbox directly; otherwise a bounded local copy publishes the received record into the mailbox. Link transport and local publication are therefore two separate timing obligations, and the boundary protocol governs only the second.

Inside that mailbox, the receive handler publishes fixed-size records with the single-attempt seqlock from the Nervous System, which discards a torn or in-flight record for this tick without a retry. Placement adds two obligations that the protocol alone does not discharge. Memory attributes, barriers, and cache maintenance on the target must keep the local copy coherent, and the copy itself, at its largest payload, must fit inside the tick's declared WCET.

Memory consistency shows only that a proposal arrived without tearing, not that the command is safe to execute. Once the record passes sequence validation, the admission pipeline checks its algorithmic certificates, dynamic feasibility, transport integrity, and lease validity. Rejecting a corrupted or late proposal neither renews an expiring lease nor revokes an active, previously vetted command early. The actuator keeps executing the current plan while that plan's lease, kinematic bounds, and state checks hold. If the current plan expires or the machine leaves its safe operating bounds, the microcontroller invokes the fallback ladder immediately. Host-side operating-system stalls, neural accelerator throttling, and asynchronous logging queues can never backpressure or delay this local decision.

↩ The single-attempt seqlock protocol, whose reader never spins or retries within a tick, is established in section 4.3.2.

Every cross-boundary exchange therefore carries two separately measured costs: board-level physical transport before publication, followed by a single bounded local read during the enforcement tick. Qualification test suites must record payload sizes, cache and memory settings, and worst-case rejection paths. Even a real-time enforcer that reads without blocking remains vulnerable if its own execution stalls, so software decoupling must be paired with independent hardware supervision of processor liveness.

13.5.2 Windowed watchdog timers and execution liveness

A standard countdown watchdog refreshed from a timer interrupt keeps getting fed after the control task has hung (Barr 2013). A single late-only threshold cannot catch a corrupted program counter or a runaway phase-locked loop (PLL) clock that jumps into the watchdog service routine ahead of schedule. The single-attempt seqlock catches torn publication, and memory protection blocks cross-domain writes, but neither verifies that the safety enforcer software is alive and running its checks at the intended rate. The Nervous System feeds the microcontroller's self-watchdog only when a tick finishes all of its checks, which catches a late or missing feed. On shared or heterogeneous

silicon, the enforcer can fail in either direction: bus contention and memory starvation can stall its progress, while PLL clock glitches or control-flow corruption can make instructions run too early. Protecting the plant therefore requires a two-sided bound on watchdog servicing.

↩ The supervisory timers, from the chunk lease to the self-watchdog timeout, are established in section 4.4.1.

High-integrity physical AI systems enforce this discipline with **windowed watchdog timers**. Unlike standard countdown timers that catch only late feeds, a windowed watchdog enforces both a lower threshold (the closed window, T_{lower}) and an upper threshold (the open window, T_{upper}), rejecting any refresh that arrives too early or too late:

1. **Early-kick violation ($t_{\textbf{service}} < T_{\textbf{lower}}$)**: If software refreshes the watchdog before the lower threshold has elapsed, the hardware watchdog controller flags an immediate fault and trips the emergency safe state. An early refresh signals disrupted control flow: a corrupted program counter may have skipped safety barrier checks to branch straight into the housekeeping routine, a stack overflow may have aborted intermediate kinematic checks, or a PLL multiplier glitch may have sped up the clock. Because a compromised execution path could still happen to feed the timer inside the valid window, primary hardware monitors (independent clock monitors and rail supervisors with isolated voltage references) must catch PLL runaways and supply dips first, leaving the windowed watchdog as a backstop.
2. **Late-kick violation ($t_{\textbf{service}} > T_{\textbf{upper}}$)**: If the timer counts past T_{upper} without a valid refresh from the enforcer, the counter underflows and asserts an unmaskable hardware reset line. A late kick shows that the safety enforcer stalled, whether starved of bus cycles by neural accelerator traffic on a shared crossbar, deadlocked in an unhandled exception handler, or delayed by cache evictions.
3. **Valid service window ($T_{\textbf{lower}} \leq t_{\textbf{service}} \leq T_{\textbf{upper}}$)**: A refresh inside this interval confirms only that the service call arrived on time. It does not prove that the software ran every required check in the dispatch loop. Full diagnostic coverage combines windowed timing with challenge-response checks or independent hardware supervision.

If the controller feeds once after each successful tick, the watchdog measures the interval *between feeds*. Let completion offset c_n from tick n start lie in $[c_{\text{min}}, c_{\text{max}}]$, and let J_{phase} bound the deviation of the interval between consecutive tick starts from T_{period}. A conservative range for the inter-feed interval is given by equation 13.5:

$$T_{\text{feed}} \in [T_{\text{period}} - (c_{\text{max}} - c_{\text{min}}) - J_{\text{phase}},\ T_{\text{period}} + (c_{\text{max}} - c_{\text{min}}) + J_{\text{phase}}] \tag{13.5}$$

This formulation captures the two extreme consecutive tick completions:

- *Shortest interval (late-to-early compression):* Tick n finishes as late as possible (c_{max}), and Tick $n+1$ finishes as early as possible (c_{min}); the elapsed time between consecutive feeds is compressed to $T_{\text{period}} - (c_{\text{max}} - c_{\text{min}}) - J_{\text{phase}}$.
- *Longest interval (early-to-late expansion):* Tick n finishes as early as possible (c_{min}), and Tick $n+1$ finishes as late as possible (c_{max}); the elapsed time between consecutive feeds expands to $T_{\text{period}} + (c_{\text{max}} - c_{\text{min}}) + J_{\text{phase}}$.

Engineers set T_{lower} and T_{upper} around this range once timing analysis and test evidence justify its bounds, while keeping the late-trip threshold inside the plant's fault budget. A single tick's nominal execution time is not enough to set a watchdog window. On the running machine, T_{period} is the 1 ms tick, and the declared worst-case execution time of 250 µs gives the execution contribution to c_{max}. The completion offset must also include the internal blocking and interference charged to the tick, so both offset bounds must use the same tick-start reference. Conversely, the nominal unloaded measurement of 135 µs cannot serve as c_{min}; early-exit paths, short branches, or trivial constraint checks can finish much faster than nominal paths. The shortest tick-start-to-completion interval observed during a stress and fault-injection campaign gives evidence for c_{min}, and a justified lower bound must also cover healthy paths the campaign did not exercise. Similarly, measured cycle-to-cycle period jitter under peak bus contention supports the bound J_{phase}, with allowances for unobserved conditions, rather than an idealized absolute phase offset. If a team has only absolute phase bounds, it must take the maximum difference between consecutive phase offsets before using this expression.

The two ends of the window are set by different arguments. The lower end of equation 13.5 sets T_{lower}. Once c_{min} and J_{phase} have justified bounds, supported by a campaign and allowances for

unobserved healthy paths, any feed earlier than the resulting lower bound lies outside the qualified healthy range. This gives the machine's self-watchdog, which has only a late trip, the early trip it lacks. The upper end is not used as T_{upper}. It marks the latest a healthy tick can feed, so T_{upper} must sit at or beyond it. How far beyond is a separate choice of fault-detection latency, judged against what the plant can tolerate while a dead enforcer goes undetected. The machine keeps the selected 5 ms timeout, several ticks beyond the window, so that transient delays do not trigger spurious resets; problem 5 converts that timeout into base travel. The late trip also answers a different question from the falsification threshold. A tick that runs past the declared WCET but still feeds within the window refutes the placement's timing premise, and the loaded campaign, not the watchdog, must catch it. The watchdog's upper end catches only an enforcer that has stopped feeding altogether.

Setting a multi-tick timeout is an engineered fault-detection latency, not proof that missing several control cycles is harmless. If the microcontroller crashes or resets, it cannot run MCU-controlled powered deceleration. A drive-autonomous SS1 remains available only if its independent trigger, drive controller, supply, and drive-energy and regenerative-sink paths survive the fault and are qualified for that response. Otherwise, the platform falls back to its separately qualified passive braking, friction deceleration, or hardwired mechanical interlocks. Total fault-handling latency (watchdog trip delay, supervisory reset propagation, and brake engagement) must therefore fit inside the plant's certified safety-stop budget. Engineers cannot justify a slow watchdog timeout by citing powered braking distances when the fault being detected disables the processor that commands those brakes.

To catch control-flow corruption where a broken loop still passes through the watchdog service call inside the valid time window, high-integrity designs add **challenge-response checkpointing**. An external hardware supervisor or isolated safety coprocessor sends pseudo-random seeds to the enforcer at runtime. Instead of writing a fixed value to a register, the enforcer must return a signature computed from fresh sensor frame hashes, constraint-barrier checks, and intermediate trajectory stages. Diagnostic coverage depends on whether a corrupted execution path can replay or compute a valid signature by chance. If the enforcer fails the challenge or misses the window, the external supervisor asserts a hardware torque-inhibit line (such as Safe Torque Off on an electric drive). Direct torque removal stops actuators from driving into hazards, but it is not a controlled powered stop; the resulting passive coast-down or mechanical braking profile needs its own qualification and safety-case evidence.

Both the boundary contract and its hardware supervision apply to every workload placement. Whether the learned policy and the real-time safety enforcer sit on separate boards, separate chiplets, or partitioned cores on one SoC, the design must pass records without blocking and verify liveness in hardware. What changes from one placement to another is how much cross-domain interference the tick must absorb, and every mechanism that bounds that interference carries its own price in silicon area, bus bandwidth, and board complexity.

13.6 Silicon Isolation Decisions

Suppose a loaded test catches one of these couplings, a tick that exceeds the declared bound under proposal load. The team then has four remedies, each paid for in a different currency: static reservation, spatial partitioning, workload reduction, and physical relocation to separate silicon (table 13.3). Each remedy targets a specific coupling layer and leaves a residual vulnerability; when that residual interference still exceeds the timing or fault-containment bound, the design must escalate to the next row, culminating in the mobile manipulator's choice of physical relocation and the board-link transport budget it requires.

Table 13.3: Workload re-placement and isolation decision matrix: Remediation pathways for when a test falsifies independence on shared silicon. Each remedy trades distinct engineering currencies (throughput, capacity, latency, board footprint) to eliminate contention channels before escalating to physically segregated silicon.

Remediation Strategy	Target Coupling Layer	Implementation Mechanism	Systems Engineering Trade-Off	Residual Vulnerability	When to Escalate
Static Reservation	CPU execution time, OS scheduling preemption	Fixed-priority scheduling, ARINC 653 time slots, per-core memory-bandwidth regulation (MemGuard)	Capacity withheld from other work, including during quiescent states	Micro-architectural contention beneath OS visibility (shared L3, DRAM bank conflicts)	Memory bus or cache stalls violate real-time deadlines despite 100% CPU core allocation
Spatial Partitioning	Cache capacity thrashing, DRAM bus contention	Cache coloring or way partitioning, DRAM bank partitioning (PALLOC), Arm MPAM, MPU/IOMMU domains	Structural fragmentation; smaller perception working set may increase off-chip traffic	Unpartitionable DRAM controller queues, row-buffer locality reordering, PDN voltage droop	Measured rail disturbances violate characterized limits, or DRAM queue stalls exceed the bound
Workload Reduction	Memory throughput saturation, peak thermal power	Model quantization (INT8/FP8), context downsampling, halved policy rate	Degraded policy tracking fidelity; increased control phase lag near dynamic stability boundaries	Common clock distribution trees, shared Power Management Units (PMU), unified reset domains	Model accuracy drops below operational limits, or shared clock/reset faults propagate across domains
Physical Relocation	Shared power, clock, reset, or unbounded controller interference	Separate enforcer MCU with independently assessed power, clock, memory, and sensor/actuator path	More board area and an external communication budget	Harness, transceiver, and common-supply faults remain	Choose when the shared die cannot provide evidence for required timing and fault containment

13.6.1 Four remedies for shared-silicon contention

A remedy that does not match the active coupling channel leaves the deadline overrun intact and strands compute capacity. Examples include reserving CPU cores when the stall lives in the DRAM controller queue, and coloring cache sets while miss status holding registers remain shared. A queueing bottleneck requires an enforced bound on service time, a cache capacity conflict requires physical separation of working sets, and a power-supply or clock disturbance requires an isolated fault domain. Trimming proposal demand can recover operating headroom, but only a bound on worst-case residual interference can support a safety claim.

Under **static reservation**,[12] physical workload placement stays the same while the critical path gets dedicated CPU time slices or interconnect slots. Reserving CPU time alone does not bound memory access latency; the interconnect needs its own service guarantee, such as periodic time-division multiple access (TDMA) slots with fixed transaction beat quotas. When hardware crossbars lack slot arbitration, system software must step in. **Memory-bandwidth regulation** gives each core an access budget per regulation epoch, counts memory transactions with hardware performance counters, and stalls any aggressor core with an interrupt once it spends its quota until the next epoch begins (Yun et al. 2013). By capping how much traffic an aggressor can queue ahead of a critical transaction, bandwidth regulation provides a demand bound from which the controller model and service guarantee can bound the queueing multiplier r_{slow} of section 13.3.2. Throttling reads alone is not enough; defending against writeback-buffer saturation requires charging both reads and dirty-line writebacks against the aggressor's budget (Bechtel and Yun 2019). The predictable execution model takes this separation further on the real-time core. It splits each task into an initial memory phase that pulls its working set into private scratchpad memory, followed by an isolated compute phase that touches no shared interconnect resources (Pellizzoni et al. 2011). That pattern matches the boundary contract of section 13.5, which performs one bounded read followed by local computation.

[12] **Time Partitioning (TDMA)**: Time-division multiple access gives each requester a fixed recurring slot, so its worst wait is one schedule cycle whatever the others demand. Integrated modular avionics made the same idea the basis of processor sharing: the ARINC 653 interface schedules partitions in fixed windows of a repeating major frame, and the assurance argument for such partitioning treats spatial and temporal separation as distinct obligations (Rushby 1999). The price is capacity reserved even when its owner is idle, the cost this paragraph charges to the proposer.

The cost is stranded capacity, because memory throughput stays reserved for the safety path even when the enforcer is idle. Had the warehouse mobile manipulator kept its permission path on the SoC, reserving memory-bus slots for sporadic 4 KB sensor reads would throttle policy weight streaming across the accelerator even with the robot base parked.

When contention comes from competing working sets thrashing shared caches or banks, **spatial partitioning** isolates cache ways or physical memory channels. For example, splitting an 8 MB last-level cache into 6 MB for the proposal policy and 2 MB for enforcer permission tables prevents cross-domain cache-line evictions, though shrinking the perception working set can increase off-chip memory traffic. Operating systems can enforce this split without special hardware through **cache coloring**: the page allocator assigns physical pages whose index bits map to disjoint cache sets (Mancuso et al. 2013). Extending address-bit coloring to the bits that select DRAM banks gives each domain private banks, removing the inter-core row-buffer conflicts of equation 13.1. Arm's Memory System Resource Partitioning and Monitoring (MPAM) extension puts these controls in hardware, combining cache-portion bitmaps across shared caches with memory-bandwidth throttling registers downstream of the last-level cache (The Linux Kernel Documentation 2026). Even so, spatial cache partitioning does not separate the shared MSHRs or writeback queues (section 13.3.2). Spatial partitioning must therefore be paired with bandwidth regulation, and neither removes a shared DRAM controller queue. Giving the safety path its own DRAM channel and controller removes controller contention, at the cost of permanently taking that channel's bandwidth away from the proposal accelerator.

A **static-partitioning hypervisor** combines these mechanisms across one SoC. At boot, it statically binds CPU cores, physical memory ranges, and memory-mapped peripherals to isolated guest partitions, runs without a dynamic runtime scheduler, and colors cache sets across partitions, as in microkernels such as Bao (Martins et al. 2020). This is the standard way to host a hard real-time safety enclave beside a Linux stack on one die. The scheduling model behind the asymmetry between proposal overruns and enforcer guarantees is **mixed-criticality scheduling** (Vestal 2007; Burns and Davis 2017). In the standard mixed-criticality model, execution budgets depend on the assurance level: if a high-criticality task overruns its lower-assurance budget, the scheduler drops low-criticality work so the high-criticality task keeps its larger budget. On the running machine, the safety enforcer is the high-criticality task and the generative proposer is the low-criticality workload. The lease protocol enforces the same priority: a proposer that misses its inference window forfeits lease renewal, while qualified hardware isolation protects the enforcer's periodic permission tick.

When an application can accept coarser tracking, **workload reduction** lowers memory traffic and heat through model quantization, smaller intermediate tensor transfers, pruned telemetry logging, or a lower policy rate. The trade-off is tracking fidelity. On the mobile manipulator, stretching the action-chunk period past the 60 ms lease lets leases expire between renewals; keeping continuous control authority requires chunk admissions above 16.7 Hz, even before renewal jitter is charged. Average producer throughput cannot verify this condition. Post-training quantization also requires fresh testing to confirm that manipulation accuracy stays within tolerance. Finally, lowering average power does not remove fast current steps (ΔI) or inductive voltage droop ($L_{\text{pkg}} \cdot dI/dt$), which remain separate electrical qualification items.

When on-chip software and hardware controls cannot bound shared-die interference, **physical relocation** moves the permission path to a dedicated microcontroller with private SRAM, its own crystal oscillator, a separate reset domain, and an isolated power supply. Private TCM removes DRAM bus contention, while an isolated rail with hold-up capacitance keeps the safety monitor alive through host-side voltage collapse. Relocation does not remove verification work; it trades on-die contention analysis for board-level signal integrity, transceiver qualification, and common-cause electrical fault containment.

Moving the permission enforcer to separate silicon shifts the cost from on-chip arbitration to board-level link latency and PCB routing. The drive command bus delay (T_{bus}) does not change because joint drives sit on an EtherCAT fieldbus regardless of compute placement. What relocation adds is proposal transport across an inter-chip link (a DMA-driven SPI link, an isolated PCIe Non-Transparent Bridge, or a differential serial bus) from the application processor to the safety microcontroller. While this board hop does not delay an enforcer tick running on already admitted evidence, it ages the observation behind every newly admitted plan and must be charged to the

dispatch-to-admission interval (section 13.6.2). Jitter in transport and admission also spends the 5 ms renewal-jitter margin that the lease rule reserves between the 50 ms chunk period and the 60 ms lease. Host-side memory contention spends the same margin: an illustrative 12.5 percent stretch of the inference uses up the entire jitter allowance by itself, leaving nothing for link delays or validation variance. Lease renewal must therefore be checked against measured end-to-end admission timestamps rather than upstream inference rates.

↩ The renewal-jitter margin and lease renewal rule are established in section 4.4.

Physical relocation also changes where sensors attach. On the running machine, joint encoders and the IMU wire directly to peripheral controllers on the safety microcontroller (hardware quadrature counters and synchronous SPI channels), keeping host-processor arbitration out of the real-time feedback loop. Driver latency and register-read jitter on those peripherals still need measurement. By contrast, high-bandwidth perception sensors feed the application processor directly: RGB cameras over MIPI CSI-2 lanes, and lidar packets over Ethernet. Those heavy streams fill perception buffers without crossing the microcontroller's internal buses. Every placement choice thus reshapes the board, trading on-die memory arbitration for pin counts, differential trace routing, and serial link overhead.

13.6.2 The board-link budget and the dispatch-to-admission interval

That board-level serial hop and split validation pipeline age sensor observations while the base keeps rolling down the aisle, closing the interval left unallocated in section 8.4.1. This span, from perception dispatching an observation frame to the safety enforcer admitting the resulting action chunk, is called the **dispatch-to-admission interval** Δ_{da} (the second row of table 4.1). It covers the remaining proposer inference after dispatch, serialization across the board link, frame receipt and deserialization on the microcontroller, host geometry checks and bounded local admission checking, trajectory buffer activation, and the wait for the next permission tick. Because the stopping budget starts at sensor capture, every millisecond of Δ_{da} is base travel on aging evidence, paid for out of the stopping budget's spare clearance.

↩ The composite stopping budget, which leaves the dispatch-to-admission interval out of the pre-brake delay and pays for it from the remaining clearance, is established in section 12.3.1.

DRAFT

Napkin Math 13.3: The board-link toll

Problem: The machine's permission path runs on a discrete microcontroller that receives each chunk over a board link. How long may the interval from perception dispatch to admission last before the finished stopping budget has nothing left at the aisle speed?

Variables:

- Chunk policy p99 inference 40 ms (Illustrative), charged in full after dispatch, which is conservative because its vision stage already lies in the dispatch age of section 8.4.1.
- Serialized chunk $D_{\text{vol,chunk}} = 4{,}096$ bytes on a $f_{\text{link}} = 25$ MHz SPI link (Illustrative).
- Deserialization 2 ms, aggregate trajectory validation 18 ms, and activation 1 ms (Illustrative allowances), and up to one 1 ms permission tick of waiting (Chosen).
- At $v = 1.3$ m/s the finished budget leaves $d_{\text{spare}} = 131.0$ mm (Derived).

The validation allowance covers both the host geometry check and the MCU's bounded local admission check, including their scheduling delays. The example does not allocate a separate duration to each. Their combined elapsed contribution must fit this allowance; qualification cannot omit host validation or count the entire allowance again on the MCU.

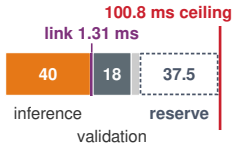
Math: Serialization takes $8\,D_{\text{vol,chunk}}/f_{\text{link}} = 1.31$ ms. The allocation totals $\Delta_{\text{da}} = 40$ ms $+ 1.31$ ms $+ 2$ ms $+ 18$ ms $+ 1$ ms $+ 1$ ms $= 63.3$ ms, which the base covers as 82.3 mm of travel. The spare pays for at most $\Delta_{\text{da,max}} = d_{\text{spare}}/v = 131.0$ mm / 1.3 m/s $= 100.8$ ms, so the allocation leaves 37.5 ms, or 48.7 mm of positive reserve. Doubling the chunk or halving the link clock adds only another 1.31 ms; adding one more inference's worth of delay, 103.3 ms in all, exhausts the spare.

Systems insight: The board link that buys silicon separation costs about one millisecond, a small fraction of Δ_{da}; proposer inference and aggregate trajectory validation dominate the interval. The stipulated discrete-placement allocation therefore fits the stopping budget,

whereas a slower policy or a heavier validation kernel could exhaust it. The allocation is a target that qualification must confirm, not a relaxed runtime margin. The deterministic portion of Δ_{da} ages observation evidence and is charged here, while runtime variance between renewals drains the 5 ms renewal-jitter budget, which host-side memory contention alone can saturate. Because host inference and local validation overlap as a pipeline across consecutive 50 ms chunk periods, a 63.3 ms dispatch-to-admission interval does not expire the 60 ms lease as long as the time between consecutive admissions stays within the chunk period plus that renewal-jitter allowance.

With Δ_{da} formally allocated, the event timing table (table 4.1) contains no undefined intervals. Total plan replacement latency from sensor capture to chunk admission (equation 11.3) is 114.9 ms, while total elapsed time from capture to emergency brake onset in the event of an unresponsive proposer is 196.9 ms. Both figures represent budget allocations that the loaded stress-testing campaign in section 13.7 must validate using physical hardware trace timestamps.

Allocating the interval also exposes interference that the discrete placement keeps inside the microcontroller. The link's receive DMA engine and its completion interrupt, the fieldbus master dispatching motor drive frames, background local admission checking, and the periodic ingestion of evidence snapshots all contend for the microcontroller's internal crossbar, interrupt controller, and memory banks. Every potential interference path demands an enforceable bound: assigning SPI and fieldbus interrupt priorities strictly below the deterministic permission tick, allocating dedicated DMA channels, and mapping safety state to independent tightly coupled memory banks. The loaded validation campaign in section 13.7 must saturate the communication link and execute maximum-length plan validation while simultaneously triggering the refusal branch. Classical response-time analysis for the periodic tick, with its higher-priority preemption and lower-priority blocking terms, provides the formal static counterpart to those empirical stress tests.



The board link costs 1.31 ms of a 63.3 ms allocation under a 100.8 ms ceiling.

These remediation pathways determine which computing resources engineers must statically reserve or physically isolate. Two subsequent architectural decisions govern the remaining placement budget: whether any portion of proposal generation may be offloaded across an external network, and which candidate accelerator silicon completes autonomous missions within the chassis energy reserve. Off-board compute is first a timing question, and it comes first because energy efficiency matters only among architectures that already meet their deadlines.

↪ Response-time analysis for a periodic tick, with its preemption and blocking terms, is developed in section C.3.

13.6.3 Off-board compute and mission energy

Querying a remote data center over factory Wi-Fi or private 5G subjects an offloaded policy to radio multipath fading, base-station handoffs, and packet retransmissions, yet the returned trajectory must still arrive inside the local 55 ms lease-renewal window. Off-board computation operates under the exact same timing windows as a local board interconnect, so an offloaded result must fit the target admission window before it can be considered for admission:

$$t_{\text{net}}^{\text{tail}} + t_{\text{proc}}^{\text{tail}} \leq W, \tag{13.6}$$

where W is the allocation for the network-and-remote-processing interval of a particular transaction. The complete dispatch-to-admission interval must also charge host validation, local deserialization and admission, activation, and tick waiting against the 100.8 ms ceiling (notebook 13.3). A continuing pipeline must satisfy that evidence-age ceiling and the accepted-result gap required for lease renewal. The lease less renewal jitter (55 ms on the running machine) directly limits transaction latency only under a serial, non-overlapping convention; with overlapping requests, stage capacity and actual inter-admission gaps require separate checks. Qualification must measure the complete end-to-end tail at the required reliability level and retain the same evidence, state, and lease checks as local admission.

Empirical robotics deployments demonstrate how severely this constraint restricts offloading. In RT-2, large vision-language-action (VLA) models ran in a multi-TPU data center queried over wireless networks; under real network conditions, the 55-billion-parameter model executed at only 1 to 3 Hz, while the 5-billion-parameter model reached approximately 5 Hz (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023). Wireless links in industrial deployments (whether Wi-Fi or private

5G) introduce severe latency tails from radio-frequency multipath fading, base-station handoffs, and packet retransmissions. At 5 Hz, a fresh plan arrives only once every 200 ms, 3.6 times the renewal window, while the 55-billion-parameter model delivers outputs at best every 333.3 ms. Neither remote pipeline could sustain lease renewal on this machine. Because the published study documents overall throughput frequencies rather than an explicit breakdown between network latency and remote execution phases, one cannot determine whether a lower-latency wireless link could close the gap. Qualifying any remote lease-renewal pipeline requires profiling the full tail distributions of both components. Instead, large remote foundation models are best positioned to synthesize coarse semantic goals or task decompositions, leaving high-frequency action chunk generation to an onboard policy. Coarser planning workloads operating on second-scale horizons are better candidates for edge offloading. FogROS2, for example, offloads ROS 2 compute graphs to cloud and fog instances, reducing grasp-planning latency from 14 s on local edge silicon to 1.2 s (Ichnowski et al. 2023). That acceleration speeds macro-task completion while leaving high-frequency microcontroller execution untouched. Packet loss or complete link disconnection must leave the onboard enforcer and its certified safe-stop routine fully operational. Neither remote offloading nor offline compilation eliminates the obligation to guarantee timely execution on local silicon.

Placement also decides which accelerator hosts the proposal path, and that choice reaches the permission path through energy as well as time. The chip's power is the load term that set the junction temperature in section 13.4, and it draws on the same battery whose sag the isolated rail must ride through. Among architectures that satisfy these timing bounds, candidate silicon must also complete the physical mission within the chassis energy reserve. Energy optimization often presents a subtle engineering trade-off. A lower-power edge accelerator draws fewer watts during active inference, yet if it extends execution latency, peripheral loads (lidar, depth cameras, cooling fans, and motor holding currents) remain energized longer. Conversely, a higher-throughput neural accelerator reduces mission energy only as long as faster compute actively shortens overall task duration. Once physical manipulator kinematics or vehicle velocity limits dominate the execution timeline, accelerating compute further merely burns excess power without saving task time, driving up total energy consumed per completed cycle.

For an assigned physical workload with steady-state compute power and baseline chassis power draw, total mission energy follows the relation:

$$E_{\text{mission}} = (P_{\text{chip}} + P_{\text{chassis_static}}) \times T_{\text{mission}}$$

Evaluating this expression requires an empirical or rigorously modeled mission duration T_{mission} for each candidate silicon configuration. Transient motor acceleration spikes, DC-DC converter losses, task retries, and quiescent dwell intervals demand explicit energy accounting whenever they diverge across platforms. Systems engineers cannot rank candidate SoCs simply by comparing datasheet figures such as peak tera-operations per second (TOPS) or nominal thermal design power (TDP). The workload to measure is the machine's own proposal pipeline, from camera ingestion through the chunk policy, and final silicon placement requires evaluating the compiled neural graph, its memory footprint, and the thermal assembly as one system. This chapter does not rank accelerators for the running machine, whose record carries no measured mission duration for any candidate; the relation states what such a ranking must measure. With reserved memory bandwidth, board-level transport latency, and battery energy draw accounted for in the allocation model, the next step is experimental verification. It tests on the physical hardware whether these isolation choices keep the permission tick within its bound and preserve braking response under worst-case concurrent load.

13.7 Empirical Verification of Silicon Isolation

The thermal hangover refuted candidate (b) from a model, but a placement that survives every model still has to face a measurement that could refute it. A claim of silicon isolation between an unverified learned workload and a safety enforcer must therefore reduce to a testable bound on enforcement latency and refusal availability. This bound must hold across all declared proposal workloads, junction temperatures, and operational modes. If shared-resource contention delays an enforcer refusal past its physical deadline, the claimed operational separation has failed, whatever architectural isolation was promised on paper.

13.7.1 The worst-case execution time falsification threshold

When a loaded permission tick runs for 260 µs against a declared isolated worst-case execution time of 250 µs, it finishes well inside the 400 µs deadline yet still refutes the claim that shared hardware added zero interference. The budget section of section 13.2 introduced the permission tick's declared worst-case execution time (T_{wcet}) as this falsification threshold, and a test designed to refute a placement must name the exact measured quantity it compares against that bound. In this example, the declared worst-case execution time bounds the task in isolation, whereas a qualified execution model can instead include bounded resource interference. Here, microarchitectural interference from hardware shared with co-running workloads lengthens the execution term rather than appearing as a separate term of the response-time bound (equation 4.3). The loaded C_i includes that penalty, beside the blocking B_i and interference I_i that the microcontroller's own tasks contribute.

The pass-fail threshold for silicon isolation derives directly from the system specification. The machine's permission path specifies a declared worst-case execution time $T_{\text{wcet}} = 250$ µs, representing the isolated execution parameter C_i in equation 4.3. It must complete each permission evaluation within a chosen enforcer deadline $T_{\text{deadline,enf}} = 400$ µs, which is nested inside the overall permission period $T_{\text{tick}} = 1$ ms. When measured under operational load, total enforcer execution time is the sum of isolated execution latency and cross-workload hardware interference, as formalized in equation 13.7:

$$t_{\text{enf}} = t_{\text{iso}} + T_{\text{interf,shared}}, \qquad t_{\text{iso}} \leq T_{\text{wcet}} < T_{\text{deadline,enf}} < T_{\text{tick}} \tag{13.7}$$

where t_{iso} represents the execution time of the permission path running with zero contention, and $T_{\text{interf,shared}}$ captures the cumulative delay inflicted by proposal-side activity competing for shared DRAM channels, caches, interconnects, clock trees, power distribution networks, and thermal budgets.

An ideal isolated placement claims that proposal-side execution introduces zero cross-workload interference ($T_{\text{interf,shared}} = 0$), making the declared bound T_{wcet} the **falsification threshold**. Because isolated execution cannot exceed T_{wcet} by definition, any measured cycle under load that exceeds T_{wcet} demonstrates that either the declared baseline WCET was underestimated or the shared hardware introduced an interference penalty that the architecture failed to eliminate. Either outcome refutes the timing contract of the placement, even if the cycle completes before $T_{\text{deadline,enf}}$, because every downstream margin, from the deadline itself to the stopping distance budgeted for each tick, depends on that premise. Conversely, observing a loaded execution cycle below T_{wcet} does not establish that $T_{\text{interf,shared}}$ is zero. Empirical testing can decisively falsify a declared bound, but measurements alone cannot prove its universal validity. Any test campaign samples only the microarchitectural states it reaches, and microarchitectural timing anomalies can push the true worst-case latency beyond every observed cycle.[13]

Even a small overrun invalidates the declared premise. It need not cause a collision on that tick, but it removes the guarantee beneath every downstream stopping and lease-renewal margin. Response-time verification charges the full bound from equation 4.3, $t_{\text{enf}} + B_i + I_i \leq T_{\text{deadline,enf}}$, in which $T_{\text{interf,shared}}$ inflates the execution term t_{enf}, while B_i and I_i remain the blocking and interference from the safety microcontroller's own workload. Under the declared bound, shared platform hardware can introduce at most 60 µs of cross-workload delay before the enforcer violates its deadline, and every candidate in this chapter, chosen or rejected, is judged against that allowance.

Because a single permission tick at 1.3 m/s corresponds to only $vT_{\text{tick}} = 1.3$ mm of vehicle displacement, engineers often underestimate the severity of a missed deadline. The stopping budget, however, leaves little margin for error. Under the baseline assumptions compiled in table 12.1, bringing the vehicle to a controlled stop from 1.3 m/s requires 969.0 mm, which leaves 131.0 mm of clearance within the 1.10 m clear distance. That subtotal accounts only for the onboard stopping maneuver; it excludes the dispatch-to-admission latency Δ_{da}, which is an allocated system budget rather than a measured property. Deducting the 63.3 ms allocated to Δ_{da} in notebook 13.3 shrinks the stopping reserve to 48.7 mm. While the stopping budget can absorb the loss of an isolated T_{tick}, unmitigated microarchitectural contention does not produce predictable single-tick delays; it produces open-ended stalls that exhaust the stopping margin entirely. That asymmetry is why the thermal case of section 13.4 disqualifies candidate (b) even though each of its late refusals slips only about one tick. The distance lost is small, but the placement has broken the bound on which every distance was computed, and a broken bound no longer limits how far the next stall can run.

13 **Microarchitectural Timing Anomalies**: A local speedup, such as an unexpected cache hit, can reorder subsequent pipeline or interconnect resource contention and paradoxically lengthen total execution time (Lundqvist and Stenström 1999). Although first identified in dynamically scheduled out-of-order processors, timing anomalies also appear in pipelined in-order cores coupled to shared buses or caches (Wilhelm et al. 2008). Consequently, composing local worst-case assumptions at each pipeline stage does not guarantee the global worst-case execution time, and empirical observation on an in-order core cannot substitute for formal timing analysis.

Guaranteeing that the enforcer finishes before $T_{\text{deadline,enf}}$ also stabilizes the downstream feedback loop. When the system dispatches motor-drive command frames at a deterministic phase offset scheduled no earlier than $T_{\text{deadline,enf}}$, execution jitter never propagates to the actuators. Instead, the feedback controller encounters a constant computation delay equal to 0.4 of the tick period. Control engineers model this fixed latency directly as phase lag at the loop crossover frequency, mirroring the phase delay introduced by a zero-order hold.

↪ The zero-order hold's phase lag, about half a sample period of delay, is developed in section B.5.5.

13.7.2 Worst-case execution time analysis methods

Measuring ten thousand clean cycles on a quiet laboratory bench cannot reveal the worst-case stall that occurs when an out-of-order core suffers a cache-eviction cascade at the exact instant a DMA controller locks the memory bus. A declared worst-case execution time is a formal timing commitment, and systems engineers establish it through one of four methods, each resting on distinct premises (Puschner and Koza 1989; Wilhelm et al. 2008).

Static analysis operates entirely without executing the binary on hardware. The analyzer reconstructs the control-flow graph directly from machine instructions, bounds every loop iteration, and applies abstract interpretation (Cousot and Cousot 1977) to model the processor pipeline and cache states so that memory references are classified as definite hits, definite misses, or unresolvable states. The tool then solves an integer linear program (Li and Malik 1995) to identify the longest feasible execution path. It uses basic-block execution frequencies as decision variables constrained by structural flow conservation and loop bounds, while the objective function maximizes total cycle count based on block-level worst-case latencies. Static analysis yields a sound upper bound only with valid loop and path bounds and a model of the actual binary, memory and peripheral accesses, and relevant processor and resource states. The reference microcontroller's in-order execution and private tightly coupled memory make these obligations more tractable, though they do not establish model fidelity by themselves. On application multicores, out-of-order execution, undocumented interconnect arbitration, and shared non-blocking caches make a static bound difficult to justify when relevant resource states and co-running demand remain uncontrolled or unmodeled.

Where an exhaustive microarchitectural model is unavailable, *measurement-based analysis* executes the binary directly on target hardware under the most demanding synthetic test harnesses the engineer can construct, appending an empirical safety margin to the observed maximum latency (the high-water mark). That directness explains its ubiquity in complex systems, yet it provides no mathematical guarantee against unobserved execution paths or rare microarchitectural collisions. The adversarial stress testing suite developed below targets contention states that ordinary workloads may leave unexercised, and its coverage limits remain part of the qualification record. *Hybrid analysis* bridges those two regimes. It measures the execution latency of isolated code snippets or basic blocks directly on physical silicon, then composes those empirical timings across the program's control-flow graph using static path-analysis techniques. It thus relies on target measurements at the sub-routine scale while adopting the structural path bounds of static modeling. Alternatively, *probabilistic analysis* models execution times as random variables, fitting extreme value distributions to empirical latency traces to project the probability that an execution exceeds a given threshold (Cucu-Grosjean et al. 2012). Because this methodology assumes that experimental measurement runs faithfully sample the operational probability distribution, unmodeled dependencies on platforms with complex hidden microarchitectural state can invalidate the projection. Section C.3 analyzes these statistical models and their operational limits.

When migrating from an isolated processor to a shared multicore system-on-chip, isolated execution analysis becomes insufficient. Isolated worst-case latency must be augmented with explicit interference penalties for every contended shared resource. Modern multicore response-time frameworks construct these bounds by combining the formal service guarantees of shared arbiters with the maximum request demand generated by co-running cores (Davis et al. 2018). Such composition yields finite, provable bounds only when hardware mechanisms or operating system regulators enforce resource budgets, which is why section 13.6 relies on memory-bandwidth regulation (such as MemGuard) to render shared silicon analyzable. FAA AC 20-193 provides an acceptable means of compliance for airborne multicore systems. Where time interference is not mitigated, it calls for WCET analysis confirmed by testing on the target processor with all software executing in the

intended final configuration. It also permits separate determination when time interference is shown to be mitigated (Federal Aviation Administration 2024).

On the running machine, the available execution models and enforced resource bounds determine the verification method. A discrete microcontroller reduces the shared-resource model, but analysis and testing must still cover local DMA, interrupts, internal buses, background admission, and the board-level supply, clock, and link faults inventoried here. On a shared application SoC, this example's declared 250 µs remains the isolated baseline, not a loaded execution bound. To validate such a shared placement, the engineering team must formally bound $T_{\text{interf,shared}}$ across every shared architectural row identified in table 13.1. It must also verify that the resulting response bound remains safely within $T_{\text{deadline,enf}}$, and it must execute an adversarial campaign capable of refuting both the analytical bounds and the isolation claims.

13.7.3 Adversarial stress testing and end-to-end brake onset timing

Refuting an independence claim requires synthetic workloads engineered to induce correlated worst-case microarchitectural contention rather than standard benchmarks (such as MLPerf) that report average throughput. Standard ML inference pipelines schedule memory transactions to maximize pipeline overlap and average data bus utilization, which conceals transient contention peaks. On physical silicon, maximum interference occurs during synchronized bursts when multiple subsystems demand the exact same physical resources at the exact same clock edge. An adversarial stress suite must synthesize these collision patterns deliberately. The five generators below target the shared die first, because that is where the strongest mechanisms of this chapter live, and they serve two campaigns: refuting a shared-die candidate, or qualifying an island an integrator can use.

1. **Targeted DRAM bank conflict generators**: The stress suite issues unaligned DMA transfers with access strides calibrated to target the exact DRAM bank and bank group assigned to the enforcer's memory buffers. By continuously alternating access across conflicting row addresses, the generator repeatedly evicts open row buffers, incurring the full precharge and activation latency (37.5 ns at the application SoC's memory clock rate) on every subsequent read.
2. **Shared-cache miss-handling engines**: Streaming memory threads cycle through footprints substantially larger than the shared last-level cache. These routines exhaust its MSHRs and saturate writeback buffers with dirty cache lines, triggering the structural interconnect stalls identified in section 13.3.2 as the primary threat to bounded enforcer latency.
3. **Resonant PDN current-step generators**: Microbenchmarks execute rapid, high-amplitude current transients across neural accelerator arrays while sweeping the transient's phase relative to the permission tick. High-bandwidth oscilloscope probes on the PDN capture package-level $L_{\text{pkg}} \cdot di/dt$ voltage droop and ringing, directly testing whether supply collapse induces clock throttling or logic faults on co-located cores.
4. **Correlated peripheral interrupt storms and storage saturation**: High-bandwidth camera deserializers and NVMe storage controllers stream raw telemetry at line rate while asserting uncoalesced hardware interrupts. This flood stresses the interrupt controller, consumes shared crossbar bandwidth, and saturates system write buffers.
5. **Chassis electromechanical transient coupling**: Test benches inject realistic traction-motor current transients and regenerative braking pulses into the shared DC bus. These tests verify whether DC-DC converter dropout, ground bounce, or electromagnetic interference couples into the enforcer's power rails or clock distribution networks under worst-case operating envelope conditions.

The campaign on the chosen discrete microcontroller keeps only the generators that can reach it, and it maps onto the four fault-injection regimes of section 13.2, now applied together rather than one at a time. Supply transients on the shared battery bus (generator 5) are regime 1, power-rail droop testing. Clock margining under those transients is regime 2. Saturating the board link while the microcontroller's own receive DMA, fieldbus interrupts, and background plan validation contend for its internal buses (section 13.6.2) is regime 3, link flooding. Streaming malformed and NaN-laden proposals during all of it is regime 4. The host-side targets of generators 1 to 4 do not reside on the microcontroller, which shares no DRAM, cache, or package with the proposer. Analogous contention

from its own DMA and interrupts remains in regime 3. They drop out because (d) removes those rows, a claim the register readback and the inventory must support, not because the campaign overlooked them.

Hardware instrumentation must trace the complete perception-action timeline: sensor observation capture, coherent ingestion across the interconnect, permission decision completion, drive-frame dispatch over the real-time fieldbus, and physical frame arrival at the motor inverters. While digital bus logs reveal software delays, issuing a bus frame does not equal physical braking. For a controlled powered stop, synchronized motor-phase current probes and external optical tracking must measure the actual onset and dynamic trajectory of vehicle deceleration. The nominal renewal-to-onset budget is 82 ms, combining the software lease duration, one permission tick, fieldbus transit, and motor-drive torque development. Spring-applied mechanical brakes engage only when the vehicle has already decelerated to near-standstill, so mechanical brake contact cannot serve as a proxy for powered deceleration onset. In fault-injection scenarios involving total MCU failure or bus power loss, testing must independently verify the physical stopping distance delivered by the fail-safe emergency circuits. All timing intervals must be reported alongside verified clock synchronization bounds and instrument measurement uncertainties.

Because interference compounds across physical, thermal, and architectural domains, stress testing requires a full factorial experimental matrix to uncover paired-stress vulnerabilities. The evaluation sweeps across four orthogonal axes. The first axis is a cold condition (an illustrative junction temperature of $35\,^{\circ}\mathrm{C}$) vs. a hot one, which on a shared die means junctions held at or above the throttling threshold T_{trip} of section 13.4 and on the discrete microcontroller means the bay at its highest qualified ambient. The second is an idle proposal path vs. a saturated proposal engine executing maximum-batch inference. The next is diagnostic circular logging disabled vs. streaming at full non-volatile memory bandwidth. The last is nominal tracking commands vs. boundary-violating commands that trigger refusal routines. Testing refusal paths under full stress is mandatory because executing a safe clamping trajectory or entering emergency shutdown involves cold branch paths, non-cached error-handling code, and secondary bus transactions that never execute during steady-state aisle driving.

The factorial matrix exists to uncover co-runner collisions that escape isolated unit testing. Consider what happens on the rejected shared-die placement when an unexpected obstacle steps into the warehouse aisle. Detecting the obstacle forces the enforcer to execute its cold refusal branch to initiate an emergency stop. Simultaneously, the application processor flags the anomalous visual scene and triggers a high-priority diagnostic core dump over the shared DMA interconnect. If tested in isolation, the refusal branch executes within its latency budget, and diagnostic dumping runs without error. When triggered concurrently, however, the diagnostic DMA burst floods the shared memory controller precisely when the enforcer must fetch its cold refusal instructions from DRAM. The resulting contention spike stalls the enforcer at the exact moment safe physical actuation is most urgent.

To see how empirical traces refute invalid isolation claims, evaluate an illustrative 10^7-tick stress test conducted on the rejected shared-die configuration (where the enforcer executes on a real-time core cluster within the application SoC). Operating at one tick per 1 ms, this campaign runs for 10,000 s, or 2 h 46 min 40 s of continuous physical execution, excluding platform setup and reset overheads. Under cold, unloaded conditions, enforcer latency remains well within limits: the trace shows a median p50 $= 95$ µs, a 99th percentile p99 $= 128$ µs, and a maximum recorded latency of 152 µs. Under warm, heavily contended conditions, however, resource conflicts degrade the tail latency: p50 rises to 210 µs, p99 climbs to 330 µs, p99.9 reaches 395 µs, and the observed maximum spikes to 468 µs.

Because the loaded p99 alone exceeds the declared isolated bound $T_{\mathrm{wcet}} = 250$ µs, the trace invalidates the joint claim that the isolated baseline is bounded and proposal-side interference is zero. The overrun alone does not distinguish an underestimated baseline from interference; attributing it to interference requires independently justifying the baseline. Once the 90 µs of local blocking and RTOS task interference is charged, the loaded 99th percentile also pushes the response beyond the enforcer deadline $T_{\mathrm{deadline,enf}} = 400$ µs. A loaded trace of this shape would reach the verdict that the thermal analysis of section 13.4 reached from a model, without relying on that model,

and the verdict would stand even if the model were wrong. The percentiles here are illustrative; the point is that a real trace on the target, not an argument, is what can refute a placement.

Even if a 10-million-tick stress trace records zero deadline violations, that outcome demonstrates only that interference remained bounded across the specific microarchitectural states generated during the test. A qualification report must document the full latency distribution, observed maximums, total sample counts, and test durations, while explicitly listing all unexercised physical conditions, such as long-term power delivery degradation, transient electromagnetic interference, and silicon aging.[14] When an empirical contention test fails, the engineering team must re-evaluate its architecture: partition memory channels, implement strict arbiter bandwidth caps, restrict proposal workload concurrency, or relocate the enforcer to physically isolated silicon. Conversely, when a campaign passes, it provides empirical evidence that supports the analytical model within the tested parameters, without mistaking a finite empirical sample for an absolute mathematical proof.

[14] **Transistor Aging (BTI and HCI)**: Bias temperature instability (BTI) and hot-carrier injection (HCI) gradually increase transistor threshold voltages over operational lifetimes. This shift degrades switching speeds across logic gates, an impairment exacerbated during transient voltage droops. Because engineers cannot measure multi-year degradation on fresh silicon, production designs incorporate explicit end-of-life timing guardbands. A physical stress campaign on new development boards cannot certify silicon against aging; the placement qualification record must formally log aging as an analytically budgeted, rather than empirically measured, condition.

Checkpoint 13.3: Refuting a placement

A placement claim is only as strong as the test that could have refuted it. Verify the core empirical criteria that defend this boundary:

- ☐ **Threshold arithmetic**: A loaded campaign on a shared-die candidate records one cycle of 300 µs. By how much does it exceed the declared 250 µs, how much of the 400 µs deadline does it leave once blocking and interference are charged, and which claim does it refute?
- ☐ **Isolation and interference**: Can you explain why a loaded cycle below the declared worst-case execution time does not prove that the shared hardware added zero interference?
- ☐ **Model and resource premises**: Can you explain which binary, path, memory, peripheral, and resource-control assumptions permit a sound execution bound on the discrete microcontroller or a shared multicore SoC?
- ☐ **Paired loads**: Can you explain why the enforcer's cold refusal routine must be evaluated concurrently with the application processor's diagnostic memory dump, rather than measured in isolation?

13.8 The Placement Record

Selecting the discrete safety microcontroller on paper does not clear the mobile manipulator for release while its loaded execution fields remain unpopulated on the target hardware. The machine now has an assigned placement for its permission path, an explicit deadline, an unloaded execution estimate, and a factorial stress campaign designed to falsify hardware isolation, but none of those timing properties can support a release sign-off until they are recorded alongside the exact environmental and operational conditions under which they hold.

The **placement record**, the eleventh record in the record chain, formalizes where the safety permission path executes and supplies the physical evidence that its timing bounds remain invariant. It consumes the enforcement record of section 12.11, inheriting the declared worst-case execution time and hard deadline of the permission path. To these timing constraints it binds the evidence that would show the bound holding under peak proposal load on production silicon. That evidence consists of the assigned execution domain, privilege ring, nominal loop period, unloaded and loaded observed maxima, AXI interconnect QoS arbitration settings, PDN inductive droop limits, thermal trip thresholds, enabled hardware mitigations, and a cryptographic digest of the external test report (table 13.4). An observed loaded maximum that exceeds the declared worst-case execution time immediately refutes the placement claim. Conversely, observed maxima that fall below the bound validate only the specific conditions exercised during testing; for any empirical latency distribution, sample maxima provide supporting evidence for an upper bound rather than an analytical substitute for it.

The placement record bridges architectural configuration registers with direct physical measurements. Register readbacks of `execution_domain_id` and `privilege_and_ring` document the compute domain topology and execution privilege against architectural status registers, while

↩ That measured maxima support but cannot replace an analytical worst-case bound is established in section 4.5.

`shared_axi_channel_qos` records interconnect arbitration configurations and verifies whether unprivileged software can reconfigure bus priorities before stress tests validate the resulting worst-case latency bounds. Alongside those digital settings, `max_pdn_droop_thermal_trip` ties voltage supply margins and junction temperature limits to high-bandwidth oscilloscope probe traces and on-die thermal telemetry gathered under full compute load. Together, `enforcement_record_digest` and `test_report_digest` establish cryptographic provenance by binding the placement record directly to its upstream timing contract and empirical test report.

Table 13.4: Placement record header: Concrete fields defining the physical compute, memory, thermal, and electrical bounds for deterministic safety enforcement. Loaded observed maximum must remain at or below declared WCET, $T_{\text{wcet}} = 250$ µs, under full cognitive stress.

Field Name	Type & Units	Physical System Invariant & Validation Rule	Mobile Manipulator Parameter / Value
`execution_domain_id`	`uint64_t` (enum)	Explicit domain topology; confirms physical silicon segregation	`0x04` (selected dedicated dual-core lockstep MCU; qualification pending)
`privilege_and_ring`	`uint64_t` (privilege)	Record target-specific privilege and memory-protection configuration	Proposer: Unprivileged Linux; Enforcer: Bare-Metal MPU
`nominal_period_ns`	`uint32_t` (ns)	Enforcer tick T_{tick}; matches enforcement record `loop_period_ns`	1 ms (1,000,000 ns)
`hard_deadline_ns`	`uint32_t` (ns)	Physical deadline $T_{\text{deadline,enf}}$; matches enforcement record `deadline_ns`	400 µs (400,000 ns)
`unloaded_observed_max_ns`	`uint64_t` (ns)	Largest cycle without load; must satisfy $t_{\text{unloaded}} \leq T_{\text{wcet}}$	`0xFFFFFFFFFFFFFFFF` (pending; illustrative estimate 135 µs belongs in report)
`loaded_observed_max_ns`	`uint64_t` (ns)	Peak execution duration under adversarial stress; all ones until qualified	`0xFFFFFFFFFFFFFFFF` (Awaiting empirical test report)
`shared_axi_channel_qos`	`uint64_t` (QoS mask)	Bus QoS configuration plus measured or justified service bound	Selected TCM-local bus; topology and access bounds pending verification
`max_pdn_droop_thermal_trip`	`uint32_t[2]` (mV; m°C)	Maximum tolerable rail droop and junction thermal trip threshold	Pending target-device characterization; shared-die example values are not MCU limits
`hardware_mitigation_flags`	`uint8_t[32]` (bitmask)	Qualified hardware isolations only; selections remain in report	Pending verification; selected rail, TCM, lockstep, and watchdog listed in report
`enforcement_record_digest`	`uint8_t[32]` (SHA-256)	Cryptographic link to upstream enforcement record	SHA-256 digest of table C.13
`test_report_digest`	`uint8_t[32]` (SHA-256)	Cryptographic link to full factorial stress campaign report	Unset until a test report exists

The record architecture separates concerns into two distinct structures. A compact, fixed-size header stores critical platform configurations and observed execution maxima for low-overhead verification at boot time. This header anchors an external test report by its cryptographic hash. The report holds all non-fixed evidentiary records: complete hardware inventories, empirical latency distributions, total sample counts, environmental conditions, active hardware mitigations, and unresolved operational boundaries.

↪ **Byte layout:** The placement record's field offsets and sizes are given in section C.15.5.

For the mobile manipulator, the header records the nominal 1 ms tick period and 400 µs hard deadline, while the test report archives the preliminary 135 µs unloaded execution estimate. Both observed-maximum fields remain initialized to all ones (`0xFFFFFFFFFFFFFFFF`), and the dedicated power rail cannot yet be documented as an approved mitigation because full empirical qualification remains pending. The loaded execution field remains unpopulated because no stress campaign has yet been executed on this machine. Populating this value requires running the factorial contention campaign of section 13.7 directly on physical hardware. In that campaign, the discrete microcontroller executes its safety loop while the host application processor saturates neural accelerator clusters with trajectory proposals and writes continuous trace logs at maximum junction temperature, triggering

the refusal branch across stress trials. Every cycle time recorded during this contention benchmark must remain at or below the declared worst-case execution time, $T_{\text{wcet}} = 250\ \mu\text{s}$. In addition, the onset of powered deceleration, timed from the arrival of the last valid trajectory renewal, must occur within 82 ms. Until physical instrumentation confirms these bounds, the permission-independence claims of the safety case rest on unverified assertions, preventing any deployment safety release.

The external test report inventories all shared silicon and platform resources: memory channels, cache partitions, DMA engines, and power rails, alongside every competing bus master. For each resource, the report documents the hardware arbitration configuration and the resulting contention latency. For the safety permission rail, it records the selected 2 s hold-up capacitance and its pending qualification by a supply-loss fault-injection campaign. The report also defines the electrical isolation boundary by enumerating allowable rail claimants, restricting access strictly to the microcontroller, the gate-drive logic, and the spring-applied brake release coils, while entirely excluding unverified proposal-side compute loads.

Any contention measurement is meaningful only in the context of its exact operating state. The test report therefore qualifies each latency distribution with cryptographic hashes of the active neural model and system firmware, DVFS policies, enclosure thermal interface materials, ambient and junction temperatures, power rail voltages, interconnect arbitration weights, synthetic traffic generation profiles, sensor sampling rates, measurement instrumentation calibration, sample counts, and total test duration. The report records these parameters alongside full latency histograms and explicit enumerations of unexercised operational states, while the fixed header exposes only the scalar maxima for fast verification. For example, the 10^7-tick trace evaluated in section 13.7 spans only 2 h 46 min 40 s of operation on a single physical machine. Such exposure demonstrates that the system survived specific benchmark conditions, but it does not exclude rare multi-contender collisions or provide a statistical bound across an entire production fleet.

Every placement record entry requires a predeclared falsification criterion formulated before executing hardware tests. This criterion defines the physical boundary that constitutes architectural failure, such as any safety enforcement cycle exceeding the declared 250 µs worst case during concurrent neural accelerator bursts and DMA transfers. The record documents the empirical execution distribution against this criterion and registers whether the isolation hypothesis survived or failed. When contention violates the latency budget and forces an architectural revision, the record details the specific mitigation deployed, such as configuring bandwidth throttling on the neural accelerator memory bus. The test entry must also explicitly report boundary conditions left unexercised during the test run, such as operating temperatures above the 45 °C bay ambient, multi-bit error injection in tightly coupled memory, or cascading sensor transceiver failures. Chapter 15 then consumes the documented arbitration points and shared power rails as a target map for hardware and software fault injection. It deliberately corrupts shared structures to test whether fault containment matches predicted isolation domains. Chapter 16 incorporates the verified execution bounds and unexercised operational limits into the formal safety case, turning raw physical measurements into auditable evidence for real-world deployment.

With the placement record in place, the stopping budget is specified row by row for a wheeled mobile manipulator whose resident fallback is a brake-to-stop suffix on stable ground. A budget developed for one embodiment earns trust as a method only if it survives a machine with different dynamics. A legged humanoid, whose fallback trajectory must dynamically sustain balance while traversing unstructured terrain, provides the most demanding stress test of these principles (perspective 13.1).

Systems Perspective 13.1: When the fallback is a step

Question: Which rows of the stopping budget survive when the fallback is a step rather than a stop?

When hardware placement supports the deterministic execution contribution to the aggregate pre-brake delay τ_{delay}, the stopping-budget framework applies directly to the mobile manipulator's powered braking maneuver, with dispatch-to-admission delay allocated and drive-energy qualification still open. However, a bipedal humanoid walking mid-stride possesses no sta-

tionary equilibrium into which it can instantaneously brake. Its fallback maneuver is dynamic, a recovery step. Every row of the stopping budget must consequently be re-evaluated for dynamic stepping:

- **Observation age survives.** Temporal latency continues to translate directly into spatial displacement, the conversion stated in section 8.4. On an articulated sensor head undergoing angular rotation, this error scales with viewing distance: a feature at range r traverses space at tangential speed $v_{\text{tangential}} = \omega r$ under an angular rate ω. For instance, an uncalibrated 10 ms camera–IMU temporal offset during a 1.5 rad/s head pan misidentifies the position of an obstacle located 5 m ahead by 7.5 cm.
- **Commitment semantics change.** The latched zero-velocity trajectory suffix established in section 11.5 is replaced by a certified recovery stepping trajectory. Rather than terminating in a stationary rest configuration, this contingency plan must be admitted alongside the nominal motion command to guarantee safe placement of the swing foot onto an admissible support polygon.
- **The safety inset changes.** The geometric obstacle clearance margins derived in section 12.5 no longer suffice to guarantee dynamic balance. A recovery step demands full closed-loop hybrid dynamics, relative-degree constraint analysis, identification of reachable capture regions, and actuation limits under torque saturation. Kinematic tracking uncertainty and ground reaction force disturbances must propagate directly into this forward invariance set before a valid safety inset can be calculated.
- **The placement record expands.** Because a zero-joint-velocity command mid-stride does not establish a balanced terminal state, the hardware isolation boundary must encompass all computational, interconnect, and power resources required to execute the active recovery step. Just as the wheeled manipulator's placement record isolates the energy storage and gate drivers needed for powered braking, the biped's placement record must certify dedicated bus bandwidth, joint controller execution enclaves, and surviving drive-energy paths for active balance recovery. Rail hold-up preserves supervisory logic; it does not by itself supply the recovery step.

13.9 Fallacies and Pitfalls

Placement determines the physical resources shared by complex proposal algorithms and critical enforcement routines. Traces of normal operation cannot prove these systems are truly independent when rare physical events, such as power voltage transients, clock faults, memory contention, or a stalled producer, stress those shared hardware pathways.

Fallacy: *Logical privilege rings protect real-time enforcers from shared-rail electrical droop.*

Privilege checks govern digital access, while the rail supplies the gates that perform those checks. The illustrative droop calculation predicts a 39 percent gate-delay increase under its stated model; only device characterization can determine the resulting setup margin or reset behavior. Lockstep cores sharing a rail can share a failure, so comparator coverage does not establish supply independence. The selected isolated rail removes one coupling path and still requires common-supply fault tests.

Pitfall: *Sharing phase-locked loops or reset controllers across proposal and permission domains.*

A separate core can still stop when a host-controlled reset asserts or slow when a shared governor changes its clock. The topology must trace each clock and reset to the controlling hardware, then test the relevant faults under load. Naming a controller a safety island does not establish that these dependencies are independent.

Fallacy: *Low average memory bus utilization guarantees bounded read latency for real-time safety tasks.*

Average utilization leaves the arrival order and longest queue unspecified. Plain FIFO queueing at the SoC's full bandwidth is not the danger, as notebook 13.1 showed. The read tail comes from an FR-FCFS controller that keeps passing the safety read in favor of a streaming accelerator's row hits, or from the miss-handling contention of section 13.3.2, and neither shows in an average. In an illustrative counterexample, such a tail stretches the snapshot read to 320 µs, which leaves 80

µs before the 400 µs deadline, less than the 95 µs needed for evaluation and staging even before blocking and interference are charged. A service bound, qualified bypass, or independent local memory addresses that failure; a low average does not.

Pitfall: *Using blocking software mutexes across inter-processor communication boundaries with untrusted neural policies.*

A mutex held by a stalled proposer transfers control of the enforcement deadline to that proposer. An unbounded seqlock retry loop has the same defect when publication remains odd. The selected boundary protocol makes one read attempt per tick and discards a collision. That discard leaves an active valid command in force only while its own evidence, state, and lease remain admissible; it neither resets the lease nor forces premature revocation. Memory ordering and bounded copy time remain part of the placement evidence.

13.10 Summary

Part I built the mobile manipulator on a hypothesis: its learned proposer runs on application processors and accelerators and its permission path on a bare-metal microcontroller. This chapter turned that hypothesis into a placement decision that evidence can refute. The allocation alone does not separate the two paths, because privilege boundaries govern none of the physical resources the two paths still share. Partitioning therefore requires a justified timing bound and fault evidence, with loaded tests able to refute the claim.

The evidence must cover the physical resources that the placement still shares. A loaded test supports a scoped claim about its workload, arbitration policy, supply, clock governance, and thermal conditions. Separate processor packages remove some coupling paths, but the board's common supply and communication links remain part of the argument. Across the link, the boundary contract keeps a stalled proposer from holding the permission deadline. The reader makes one bounded attempt per tick and discards a torn record, and a windowed watchdog trips on feeds that arrive too early as well as too late.

The placement record carries that evidence forward. It takes the permission path's declared worst-case execution time and deadline from the enforcement record and adds the loaded observed maxima for the declared test conditions, the shared-resource queue and power headroom, the common power, clock, reset, and memory failure paths, and the conditions left untested. The machine's own record holds a 135 µs illustrative unloaded estimate against the declared 250 µs worst case inside the 400 µs deadline. It still awaits loaded test results, which must name the arbitration policy, clocks, sample count, and physical endpoint before any release claim can rest on it.

The observation contract, belief record, intent lease, trajectory record, enforcement record, and placement record now describe one machine. Its timing must hold across the chain, including the memory, power, and communication paths between stages. Placement makes the execution premise of principle 3 falsifiable through a threshold, a stress campaign, and a record of what remains untested. Part IV inherits the conditional stopping margin, the unverified premise that aisle crossings are procedurally restricted (section 12.3.1), and the placement record's empty loaded field. It also inherits a dispatch-to-admission interval Δ_{da} that this chapter has allocated at 63.3 ms, inside the 100.8 ms the spare can pay for, but not yet measured. Seven relations carry the chapter's quantitative argument, and table 13.5 records each with the decision it settles and the place it is derived.

Table 13.5: Key equations of workload placement: The relations behind the chapter's contention, thermal, supervision, falsification, and offload decisions, with the place each is derived.

Equation	What it bounds or decides	Derived
$t_{\mathrm{queue}} = \sum_{k \in Q} D_{\mathrm{vol}\,k} / \mathrm{BW}_{\mathrm{peak}} + N_{\mathrm{conflict}}(t_{\mathrm{RP}} + t_{\mathrm{RCD}})$	How long a safety read waits behind a FIFO backlog on shared DRAM; a bound exists only if the controller limits the backlog or lets the read pass	equation 13.1
$f \le \dfrac{T_{\mathrm{wcet}}/t_{\mathrm{unloaded}} - 1}{r_{\mathrm{slow}} - 1}$	The largest share of the tick that may stall on shared-cache misses slowed by r_{slow} before the loaded tick crosses the declared bound	equation 13.2

Equation	What it bounds or decides	Derived
$\Delta V_{\text{droop}} = L_{\text{pkg}}\, dI/dt + \Delta I\, R_{\text{pkg}}$	The first-order rail disturbance a current step imposes on a controller sharing its supply	equation 13.3
$T_{\text{j}}(t) = T_{\text{j,target}} + (T_{\text{j,start}} - T_{\text{j,target}})\, e^{-t/\tau_{\text{th}}}$	How long a throttled clock persists after the load stops, and so how many ticks a shared thermal path exposes	equation 13.4
$\lvert T_{\text{feed}} - T_{\text{period}} \rvert \le (c_{\max} - c_{\min}) + J_{\text{phase}}$	The service window a windowed watchdog must accept from a healthy tick, which sets its early trip and the floor for its late one	equation 13.5
$t_{\text{enf}} = t_{\text{iso}} + T_{\text{interf,shared}},\ t_{\text{iso}} \le T_{\text{wcet}} < T_{\text{deadline,enf}} < T_{\text{tick}}$	The falsification threshold: a loaded cycle above T_{wcet} invalidates the joint baseline/isolation contract; the cause may be an underestimated baseline or interference	equation 13.7
$t_{\text{net}}^{\text{tail}} + t_{\text{proc}}^{\text{tail}} \le W$	Whether an off-board result can renew the lease or become an admitted chunk	equation 13.6

Key Takeaways: Silicon placement and memory contention

- **Logical separation is a hypothesis about hardware:** Privilege rings govern which addresses a task may touch, not where its requests wait, which rail feeds it, or which clock and reset tree it depends on. Loaded tests can refute the claim; justified bounds and fault evidence must support it.
- **Averages cannot bound a deadline, and bandwidth is not the main threat:** On a shared die at full bandwidth, more than 82 FIFO backlogs must precede the snapshot read before it crosses the declared worst-case execution time. Shared-cache miss handling is far stronger: measured slowdowns of 21× and more than 300× cross the declared bound once 4.3 percent or 0.28 percent of the tick is miss stall. Only private memory or regulated co-runners bound that multiplier.
- **Couplings act on four timescales around a millisecond tick:** Rail droop acts in nanoseconds and interconnect queueing in microseconds, below the millisecond tick they threaten. Silicon thermal throttling, by contrast, persists for seconds after the load stops, and chassis structural and battery pack self-heating accumulates over multi-hour shifts. In the rejected shared-die placement, a halved clock stretches the enforcer's 250 µs cycle to 500 µs while inference runs and for at least 2,175 ticks after it stops.
- **The declared bound is the falsification threshold:** The declared worst-case execution time bounds the tick in isolation, and the selected placement claims that proposal-side load adds no interference term, so any loaded cycle above it refutes the placement's timing premise even when the deadline still holds. A passing trace supports only the states it tested.
- **A refuted claim forces a priced choice:** Regulation and reservation cost idle capacity, coloring and partitioning cost working set, workload reduction costs policy authority, and relocation adds a board link. On the machine that link costs 1.31 ms of a 63.3 ms dispatch-to-admission allocation dominated by inference and validation.
- **Platform exposure is a placement constraint:** An on-die safety island with its own crystal, rail, and memory removes every die-level coupling except temperature, the reset chain, and the fabric its proposals cross. As of 2026, however, the commercial modules that carry the machine's application processor do not enable it, so the permission path moves to a discrete controller.
- **The placement record bounds the deployment envelope:** It takes its declared worst-case execution time and deadline from the enforcement record and adds loaded observed maxima, thermal headroom, bus arbitration, mitigations, and untested conditions. On the mobile manipulator the measurement fields are still empty, so the independence claim reaches release review without loaded qualification evidence.

Placement is where the third law becomes a property of hardware. Proposal is not permission only while the proposer cannot reach the permission decision through anything the two share, and on a real machine that includes memory, supply, clock, reset, and heat. The useful habit is to read every architecture diagram as a list of claims, one per shared resource, and to ask of each one what loaded measurement would refute it. The running machine leaves Part III with that placement decision and an explicit evidence gap: the record names the resources that must preserve the permission deadline, the measurements that could refute their bounds, and the fallback that must remain available if the proposer fails. Governing the machine means judging such conditional claims, and the next proposer to claim authority will be a person.

13.11 Historical Perspective and Further Reading

Avionics met the placement problem first. Once integrated modular avionics put functions of different criticality on one processor, the certifying question became whether a fault or overrun in one function could reach another. In Rushby (1999), the answer separates into two obligations: spatial partitioning, which protects memory and devices, and temporal partitioning, which protects the time each function is promised. The ARINC 653 interface turned the temporal half into fixed windows of a repeating frame, and Kopetz (2011) built whole architectures on a global time base so that timing could be designed rather than discovered. Automotive practice later wrote the same concern into ISO 26262 as freedom from interference and dependent-failure analysis (International Organization for Standardization 2018).

The bound itself had to be computed before it could be defended. Puschner and Koza (1989) showed how to derive a maximum execution time from program structure. Abstract interpretation (Cousot and Cousot 1977) gave analyzers a sound way to model caches and pipelines, and implicit path enumeration (Li and Malik 1995) turned the longest feasible path into an integer linear program. Wilhelm et al. (2008) surveys the tools that combined them, while Lundqvist and Stenström (1999) showed that local worst cases need not compose into a global one. Vestal (2007) then gave each task a budget per assurance level, the idea that lets a proposer overrun while an enforcer may not (Burns and Davis 2017).

Multicore processors undid the single-core premise. Cores that share caches and memory controllers turn the memory wall (Wulf and McKee 1995) from a throughput limit into a deadline problem. The software answers followed in quick succession: phased execution that confines memory access to a separate phase (Pellizzoni et al. 2011), per-core bandwidth regulation (Yun et al. 2013), and bank-aware allocation (Yun et al. 2014). Measurements then showed that partitioning capacity leaves the miss-handling structures shared (Valsan et al. 2016; Bechtel and Yun 2019), and certification caught up with AC 20-193 (Federal Aviation Administration 2024). What this chapter adds is the physical side of the argument. The enforcer's bound is charged against a stopping distance, heat and supply couple the paths on timescales no scheduler sees, and the evidence travels to release in a placement record.

For further reading, six works trace the ideas this chapter composes.

- Rushby (1999) states the requirements for partitioning processors shared by functions of different criticality and separates the spatial and temporal obligations that the dependent-failure inventory of table 13.1 checks row by row.
- Kopetz (2011) develops real-time system design from a global time base, the discipline behind fixed ticks, deadlines, and time-triggered exchange.
- Wilhelm et al. (2008) surveys static, measurement-based, and hybrid methods for worst-case execution time and the premises each rests on, the comparison behind section 13.7.
- Burns and Davis (2017) reviews mixed-criticality scheduling from its origin to multicore platforms, the theory behind letting the proposer lose renewal while the permission tick survives.
- Yun et al. (2013) introduces per-core memory-bandwidth regulation, the mechanism that turns an unbounded interference multiplier on a shared die into a bounded one.
- Federal Aviation Administration (2024) is the current certification guidance for multicore processors, and it shows what a regulator requires before a shared-die timing bound can be believed.

What's Next: From compute placement to intervention

When the slowest response delay belongs to a human operator, the stopping budget faces an interval that no silicon placement can bound. Every delay examined in this chapter belonged to the machine (a queued read, a stretched gate, a throttled clock), and each can be bounded or refuted by an empirical test on the chassis. Chapter 14 opens Part IV by addressing delays that originate outside the machine. A remote assistant taking over the mobile manipulator in an aisle introduces human reaction time over a network link that the onboard permission path cannot control, spending clearance from the exact same stopping budget. The authority to propose motion must then pass safely between the learned policy and the human operator while the permission path continues to validate every commanded actuation. That handover must never induce the hazardous motion it was intended to prevent, and every control tick must immutably record which entity held proposal authority.

DRAFT

IV

GOVERNING THE MACHINE

DRAFT

DRAFT

13

Governing the Machine

Demonstrating a physical AI model in a controlled laboratory is a milestone. Outside the workbench, dust obscures lenses, lighting changes, wireless networks drop packets, gearboxes wear, and human coworkers share the machine's path. Each change can invalidate a premise used to permit motion. Part III leaves the running machine with conditional answers rather than settled ones: its stopping margin holds only while the unmeasured dispatch-to-admission interval stays inside its allocation, passing a blind rack corner (section 9.7) rests on an unverified procedural restriction of aisle crossings, and the placement record's loaded field remains empty. Part IV turns those open questions into intervention rules, fault tests, an operating decision, and a named owner for each decision behind it, beginning with the one delay that no test on the chassis can bound, a person's. The engineer must be able to explain both the useful work the machine may perform and the conditions that require restricting or stopping it.

Principle 4: Permission must be earned

Invariant: Authority to operate is granted per operating condition by evidence whose age, provenance, and limits are recorded, and it expires when that evidence does. Zero failures in n independent, stationary trials place an approximate upper bound of $3/n$ on the per-trial failure probability at 95 percent confidence (section 7.6).

Implication: The operating decision joins evidence of useful task completion with evidence that the machine contains the specified faults and hazards. Who may command it, what the tests challenged, and which conditions it may enter are decided on that record, not on a demonstration. A premise that evidence cannot settle restricts operation through a reduced envelope, a passive limit, or refusal, and it stays open until a declared test closes it.

- **Supervisory Intervention (chapter 14):** Decides who holds authority over each actuator at each instant when a person and a learned proposer share the machine, and produces the authority record.
- **Adversarial Verification (chapter 15):** Decides which fault tests challenge each claim and how their outcomes count, and produces the fault record.
- **Deployment Release (chapter 16):** Decides whether the machine may operate and under which standing conditions, and produces the release manifest.
- **Responsible Physical AI (chapter 17):** Decides who answers for each decision behind an admitted action, which human roles a person can actually perform, and which people bear the machine's risk and appear in its records. It produces the approval record.

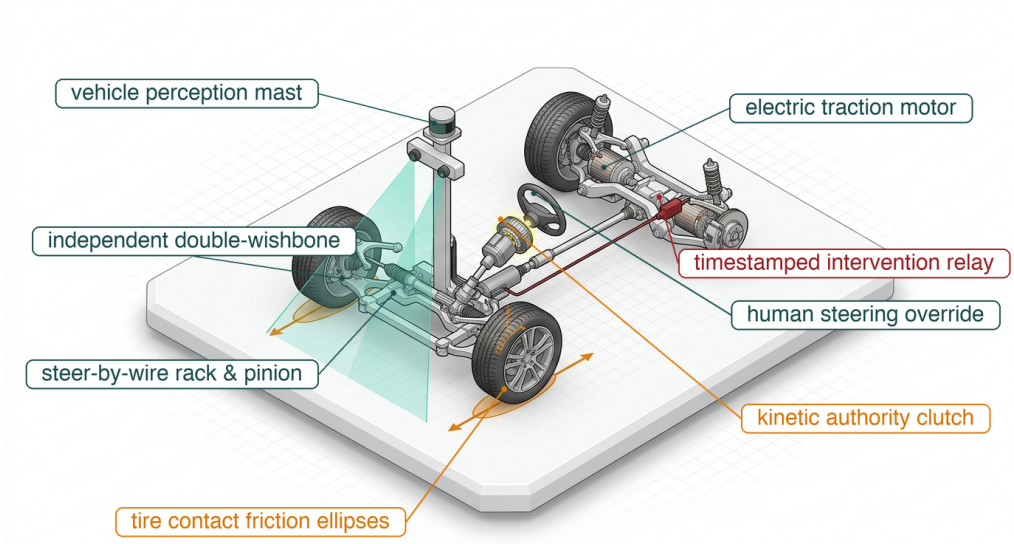
Part IV hands forward a release manifest, an approval record, and the premises they could not close. The release exercise preserves refusal for the illustrative machine with missing qualification evidence, then uses a separately stipulated evidence packet to show how restricted useful service could be supported. Neither branch invents completed tests. The manifest binds the evaluation record, the fault records, the placement record, and the authority record into claims with a verdict (unconditional operation, conditional operation, or refusal) and standing conditions that revoke it. The approval record names the person who approved each decision behind them, so that a decision with no owner refuses operation like an unknown premise. The conclusion in chapter 18 synthesizes

what the engineer can now build and justify, then transfers the method to an unfamiliar machine. Open premises enter a residual-claims register with their restrictions, owners, and closure tests; they guide the next measurement, learning experiment, architectural change, or narrower task.

DRAFT

14

Supervisory Intervention

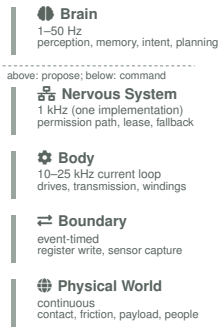


14.1 Shared Authority
14.2 Human Authority, Reaction Time, and Clearance
14.3 Authority Transitions
14.4 Transfer Without Discontinuity
14.5 Authenticating an Intervention
14.6 Authority Handover Failure Modes
14.7 Trustworthy Operational Logs
14.8 The Authority Record
14.9 Fallacies and Pitfalls
14.10 Summary
14.11 Historical Perspective and Further Reading

Purpose

Why can handing a moving robot to a human operator cause the collision the handover was meant to prevent?

Most deployed robots keep a person in reserve for the situations their learned policies cannot handle, and that reserve is usually treated as the final safeguard. A person, however, needs a large fraction of a second to respond even while watching closely and several seconds after only monitoring. A machine moving at walking pace covers meters in that time, so a request for help can consume the very clearance the help was meant to protect. Taking over also creates hazards of its own. Two controllers that each believe they hold a joint can command it at once, a controller that believes the other still holds it can leave it unattended, and an abrupt switch between their commands jolts the drivetrain and the load it carries. A takeover that grants unchecked control lets a startled or lagging operator drive the machine into a rack. A log that does not mark who commanded each instant can neither settle what caused an incident nor keep the machine's own near-failures out of the demonstrations it later learns from. Authority can change hands within a single control cycle, but the momentum already delivered across the causal boundary cannot be recalled, so a takeover must be engineered as a control action with its own deadline, its own limits, and its own record.



Learning Objectives

- Specify operator permissions through capability matrices that bind each authenticated role's operations to physical channels, preemption tiers, and clearance-derived deadlines
- Select a state-dependent local fallback that remains physically feasible while human intervention is pending
- Calculate the latest fallback time and takeover speed ceilings from human reaction latency and remaining physical stopping clearance
- Evaluate the inherited chunk lease and the chosen manual lease against their modes' stopping reserves and available fallback mechanisms
- Design authority handshakes that keep one owner on each actuator output and size fixed-endpoint C^2 command blends within each channel's rate limit
- Diagnose mode confusion in split-channel handovers and specify the per-channel annunciation that keeps an operator's model of ownership current
- Evaluate whether an authority record provides the cryptographic and kinematic evidence needed to reconstruct post-incident fault causality

14.1 Shared Authority

When a pallet blocks an aisle and the warehouse mobile manipulator requests a remote assistant to take over, the transfer introduces a delay that no test on the chassis can bound, because it belongs to a person. Part III left the running machine with a stopping budget whose every term belongs to the hardware, each with a declared bound and a loaded test that could refute it, even where the test has not yet run, as for the dispatch-to-admission interval Δ_{da} that section 13.6.2 allocated but did not measure. Part IV begins where a person enters that loop.

Supervisory intervention transfers the right to propose actions from an autonomous policy to a human operator, and eventually returns that authority to the machine. Nearly every embodied machine learning system operating in the physical world relies on this exchange. Remote teleoperation assistants clear blocked aisles and resolve scenes that warehouse robots and autonomous vehicles cannot interpret on their own. Safety drivers and teach-pendant operators step in whenever an operational policy drifts outside its design envelope, and their corrections protect the hardware and supply demonstration data for later retraining. Engineering block diagrams often reduce this supervision to a single switch or an external arrow pointing into the plant. In the physical world, however, kinetic energy does not pause while a multiplexer toggles inputs. Authority transfer is a control maneuver with its own deadlines, and it must be executed in coordination with the slowest and least predictable element in the loop, the human operator.

How often that exchange happens on a mature fleet is a matter of public record in one jurisdiction. California requires every company testing autonomous vehicles on its public roads to file annual reports of autonomous miles and of the disengagements, transfers of control to a person, that occurred during them (California Department of Motor Vehicles 2026). Figure 14.1 plots the filings of one long-running fleet, Waymo's.

Over the decade the safety-driver rate rose from about 1,200 to about 19,200 miles per disengagement, yet the fleet still reported 174 takeovers in its latest year, and the series is far from monotone. Between 2020 and 2021 the rate fell 3.8× while the miles driven grew 3.7×. Driverless testing reports one disengagement every 71,800 miles, 54 in its latest year. Two consequences follow for the machine. Human authority is not a phase a capable system grows out of; even the most experienced fleet exercises it every year, so the transfer mechanism is a permanent part of the design. The reported rate also depends on where the fleet drove and on what its operator chose to count, which is why an intervention count tells a reviewer how often the exchange happened but not, by itself, how well either party performed it.

The central engineering problem of supervisory intervention is how authority can change hands on a moving machine without exhausting stopping clearance, letting two controllers drive the same actuator at once, or losing track of who commanded each instant. The fourth law, permission must be earned (principle 4), frames the answer, and here Part IV applies it for the first time to a person.

↪ The two permit series as two stages of permission to operate, and how a machine leaves one stage for the next, are developed in section 17.4.

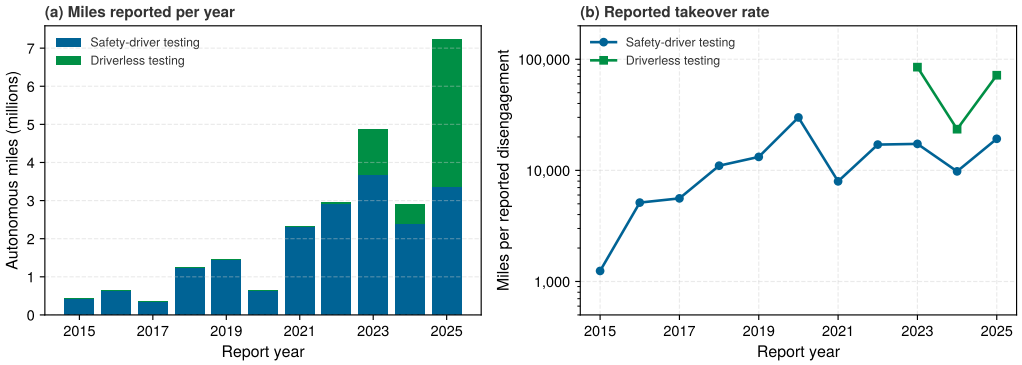


Figure 14.1: A decade of reported takeovers: Waymo's annual filings to the California DMV. (a) Autonomous miles driven on public roads under the safety-driver testing permit and the driverless testing permit. (b) Miles per reported disengagement under each permit, on a logarithmic axis. Each company defines which disengagements are reportable, and the regulator notes that the reports are not designed for comparison across companies; commercial deployment miles are excluded. Early years are transcribed from the filed PDFs. Data from California Department of Motor Vehicles (2026).

Like an autonomous policy, a human operator receives control authority only while valid evidence supports that grant, and loses it the moment that evidence lapses. Until the platform's permission path checks it, every command from an operator is an untrusted request that cannot cross the causal boundary. A safe transfer must therefore bind each authenticated role to explicit actuator channels and timing bounds, commit ownership atomically on a control tick, blend reference changes within drivetrain rate limits, and log who held authority at every cycle for incident review and policy retraining.

For the warehouse mobile manipulator, this timing constraint is direct and unforgiving. When the blocked aisle prompts the machine to request a remote assistant, at its chosen 1.3 m/s aisle speed a 4.5 s pause before braking, the design out-of-loop takeover time that section 14.2.2 derives from driver take-over studies, carries the machine 5.8 m, more than five times its 1.10 m rack-end clear distance. An operator's intervention request reaches a chassis that is already in motion under an admitted command; human intent cannot erase physical momentum or make an actuator respond in zero time. The permission path resolves this transition in two stages. The arbiter that selects among the fallback ladder's rungs also decides, for each channel, which source holds the right to propose: the learned policy, the operator, or, once that authority lapses, a fallback rung. The enforcer then admits or rejects each setpoint from the selected source against physical envelope invariants. Both stages run deterministically on a dedicated safety microcontroller (MCU). Source selection precedes the enforcer's check, and rung selection follows it whenever the check fails, so human intervention adds a new input to the arbiter, a person, rather than a new component. Throughout this process, the plant must keep enough clearance to bring itself to a safe halt. Switching setpoints in a single step demands a sudden torque jump that strains motor drives, gearboxes, and payload stability. Delaying the transfer, in turn, uses up the remaining stopping margin even when both the learned policy and the human supervisor produce sensible trajectories on their own.

Because momentum and kinetic energy sit on the physical side of the causal boundary, the machine keeps moving under the last admitted setpoint while the remote operator pieces together what is happening in the aisle. That mental reconstruction is slow. It is the paradox Lisanne Bainbridge called the *ironies of automation* (Bainbridge 1983): engineers automate routine work to remove human error, yet leave human operators to handle the hardest, unmodeled edge cases that the automation cannot solve, where plant dynamics are abnormal and time is shortest. An operator who only monitors a screen also loses both manual feel and a current mental model of how the plant is moving. When an unexpected hazard forces a sudden takeover, that operator must act under tight time pressure with little context. Human vigilance fades quickly when watching for rare faults, and automation complacency makes the problem worse whenever secondary tasks compete for the supervisor's attention (Parasuraman and Manzey 2010). Decades of human factors research confirm that passive monitoring erodes situation awareness[1] and adds long decision delays when

↪ Whether a human role that needs this much time can count as a safety measure at all, and who is blamed when it cannot, is developed in section 17.3.

1 **Situation Awareness Levels**: Endsley (1995) set out situation awareness as three levels: perceiving the elements of a scene, comprehending what they mean together, and projecting their state forward. The levels fail separately. A remote assistant can see the pallet without projecting that the base will not stop short of it, which is why a console that reports where the operator is looking cannot shorten the response budget (section 14.6).

automation fails without warning (Endsley and Kiris 1995). The machine must therefore protect its own collision-free stopping envelope throughout the takeover window. Faster networks can trim milliseconds from packet transport, but they cannot shorten the time a person needs to see a hazard, decide what to do, and move a hand control.

↩ The causal boundary, across which admitted motion cannot be recalled, is established in section 1.4.

Cutting power to a moving base lets it coast, whereas cutting power to a loaded manipulator arm without a holding brake drops the payload. Each stage of an intervention therefore needs a fallback matched to the machine's current state: a moving base may need controlled regenerative braking followed by a mechanical brake once stopped, while an arm carrying a tote needs an active, gravity-compensated position hold. Each response depends on the specific sensors and power rails qualified to carry it out. Because local damping, torque inhibition, and full power isolation produce different physical trajectories, the system cannot treat them as interchangeable steps on a generic escalation ladder; a protective input must invoke the validated fallback built for the plant's current motion state.

↩ The fallback ladder, from which the arbiter selects a state-matched protective response on every tick, is established in section 12.9.

Autonomous and manual proposal rights operate under separate, explicit leases. The learned policy proposes trajectories under the action-chunk lease established in section 4.4, while a teleoperation session receives a distinct, clearance-budgeted manual lease. Rejecting an invalid replacement command from an operator does not invalidate an active, previously admitted setpoint whose evidence, state bounds, and lease duration remain healthy. If either controller exhausts its lease duration or violates its underlying safety premises, the permission path immediately triggers the appropriate local fallback.

The permission path prevents split-brain command contention by pairing an upstream source arbiter with **exclusive actuator-output ownership**. In the 1 kHz control architecture analyzed here, the right to propose on a set of channels is a single object, the **proposal token**, and the arbiter grants it to one source at a time. An authority commit revokes the learned policy's proposal token and latches its last approved setpoint; the admission enforcer remains the sole, non-bypassable writer to the physical actuators before, during, and after the transition. Standard bus protocols such as Controller Area Network (CAN) arbitration serialize message frames from multiple transmitters, but bus-level arbitration does not establish control-layer authority. If two nodes broadcast conflicting yet syntactically valid frames to the same motor drive, the actuator chatters between setpoints and loses stability. Gating the output path in firmware and selecting a single source per tick prevents that contention. Across continuous state spaces, the enforcer executes a bumpless transfer, a command handoff without reference step discontinuities (section 14.4), interpolating between validated endpoints while enforcing command-rate limits and honoring measured plant limits.

Intervention events also directly shape the machine learning lifecycle. If a flight recorder or logging pipeline captures a rescued episode without marking the exact handover instant, imitation algorithms treat the near-failure states that triggered the rescue as if they were intentional expert demonstrations. As section 5.8 establishes, the states visited by the learner right before a takeover supply training data for recovery policies, but autonomous commands from a failing interval require review rather than automatic admission as expert targets. Downstream learning algorithms, from Dataset Aggregation (DAgger) (Ross et al. 2011) to modern human-gated policy correction methods (section 14.8.3), depend on knowing which controller held authority over each logged timestep. To resolve this attribution, the architecture keeps an authority record (the twelfth record in the platform's record chain, specified in section 14.8), which retains the captured evidence preceding and following the takeover, with explicit gaps and timing bounds. Its per-tick entries timestamp every transition against the safety microcontroller's monotonic clock. An authority commit ends policy ownership, ensuring that autonomous exploration, the enforcer's transition blend, and verified human recovery remain distinct, auditable intervals.

Every shared-authority design begins by defining which named intervention may act upon which physical channel and establishing the maximum latency the machine can tolerate while awaiting human action. The protocol handshakes, setpoint blending algorithms, and authority records that follow all rest upon this initial grant of authority.

14.2 Human Authority, Reaction Time, and Clearance

An assistant guiding the arm can move the chassis if its credential also permits base-drive commands. A command admitted under that permission performs mechanical work on the plant before software

can revoke it. Authority must therefore identify the physical channels a person may affect, rather than grant unrestricted access through a generic operator role.

The remote assistant who clears a blocked aisle and the coworker who takes an item from the gripper act on the same machine, and neither should hold the other's authority. Each grant therefore names an operation together with its timing budget, physical degrees of freedom, and preemption rule. The permission path keeps these grants in an explicit **capability matrix**, a permissions table mapping individual operations (such as an emergency power cutoff, a velocity trim adjustment, a trajectory path veto, or manual drive of the base and arm) to authenticated roles. The design reflects classic taxonomies of human supervisory control[2] (Sheridan 1992) and multi-level automation (Parasuraman et al. 2000). Each entry in this matrix defines the exact actuator channels the operation governs, the preemption tier it carries over the learned policy, and the maximum propagation latency permitted from physical input to actuator response.

Sheridan also describes the supervisor's work as five steps in sequence: plan the task off-line, teach the plan to the computer, monitor the automatic action, intervene when it fails or the goal changes, and learn from the episode (Sheridan 2021). Intervening in flight and learning from the episode require finer granularity than a single autonomy level assigned to the whole task. The mobile manipulator needs an authority assignment per channel and per instant: during a takeover, the arm can sit near the manual end of the scale while the base still runs at the automatic end, and the enforcer holds the final admission gate on every channel throughout because it checks the operator's commands just as it checks the policy's. Levels that differ across channels are what a supervisor can lose track of, the mode confusion of section 14.6, so a channel left out of a transfer keeps its autonomous owner only while the console announces that it does, and otherwise holds. Each entry in the capability matrix therefore names the exact operation and channel it governs.

Naming the operations separates intervention from **safety enforcement**, two mechanisms that system architectures routinely conflate. Enforcement withholds permission from a candidate action proposed by a learned model, modifying, clamping, or projecting the setpoint while leaving the machine under autonomous control. Intervention transfers authority away from the model to an external entity, terminating autonomous operation and giving the supervisor the right to propose, while every command still passes the permission path. An **enforcement filter** sits in the fast inner control loop (1 kHz / $\leq$ 1 ms) to prevent the policy from demanding an over-current maneuver or violating a barrier certificate (Ames et al. 2019) during nominal tracking. An intervention mechanism strips the policy of its proposal token when an operational boundary is breached or an operator demands control. A system equipped with enforcement but lacking intervention cannot be redirected when human intent changes or external conditions fall outside the policy design envelope, leading to conservative deadlock when barrier constraints collapse the feasible control set. Conversely, a system equipped with intervention but lacking enforcement permits a lagging operator to drive the plant into structural failure during nominal manual operation. Table 14.1 sets the two mechanisms side by side, from trigger and authority holder to typical failure.

Table 14.1: Architectural boundaries of safety enforcement and supervisory intervention: Enforcement operates in-loop as an active constraint on proposed actions; intervention transitions authority between discrete proposers.

Dimension	Enforcement filter	Intervention state machine
Primary trigger	Invariant boundary approach ($\dot{h} + \gamma_{\text{cbf}} h < 0$) or torque limit	Human takeover assertion, monotonic lease timeout, or operational design domain (ODD) exit
Authority holder	Autonomous policy retains proposal rights; enforcer modifies setpoints	Authority transferred to external entity or fallback controller
Execution loop	Fast inner control loop (1 kHz / $\leq$ 1 ms)	Same permission tick; protocol exchange spans multiple ticks
Safety lease role	Enforces continuous barrier satisfaction on admitted commands	Lease expiry invokes the state-dependent validated fallback
Typical failure mode	Feasible set collapse; conservative deadlock or oscillation	Opposing commands cancel, mode confusion, or late human reaction

[2] **Supervisory Control Levels**: Sheridan and Verplank (1978) formulated a ten-level taxonomy of supervisory control, running from a human who performs the whole task without computer assistance to a computer that decides and acts while ignoring the human.

↩ The actuator current and thermal limits an enforcement filter must respect are established in section 2.1.

Definition 14.1: Supervisory intervention

Supervisory intervention is the transfer of the right to propose commands on named actuator channels between a learned policy and an authenticated person, committed on a control tick and bounded by a lease, while every command continues to pass the same permission check.

1. **Significance**: The transfer is a delay on a moving machine, so it is priced in the stopping budget $d_{\text{stop}} \leq D_{\text{clear}}$ rather than in the user interface, and the waiting before braking, not the braking itself, decides whether the machine stops in time.
2. **Distinction**: Enforcement changes a proposal and leaves the proposer in place; intervention changes the proposer and leaves the check in place. A machine needs both, because neither can do the other's job.
3. **Common pitfall**: A frequent misconception is that a takeover hands the operator the actuators. The operator gains only the right to propose, and the machine still needs a local stop that does not wait for the operator to act.

14.2.1 Capability matrices and the intervention-enforcement split

On the warehouse mobile manipulator, a coworker taking a mug from the gripper at the packing station would be at risk if their station credential also allowed them to command the arm's joints or drive the base. The capability matrix prevents that cross-channel hazard by letting the remote assistant redirect motion while denying any motion authority to the coworker accepting an item.

In that matrix, the remote assistant who supervises the mobile manipulator may hold a signed **trajectory veto** capability for the base's planner channel: role `remote_assistant`, operation `veto_path`, channel `planner_proposal`, preemption tier above the learned planner but below the local enforcer, authenticated session and nonce, and a clearance-derived acknowledgment deadline. The enforcer responds to a valid veto with a checked local hold or stop. The same session carries a second entry, `manual_drive`, on the base-velocity and arm-joint channels, which becomes active only through a handover that commits within the clearance-derived deadline, and lapses with the manual lease of section 14.3. Every command it carries still passes the permission check. A wheel drive-torque packet from that credential is denied, because neither entry grants the base's wheel drive-torque channel.

The coworker at the packing station holds a different entry, `accept_item`, which lets the coworker take from the gripper the mug the arm picked from the takeaway conveyor. That entry grants no motion authority. The item-handover channel of the enforcement record caps the arm's approach to the coworker's hand at 0.10 m/s, and no capability lifts that cap. The handover is a collaborative application in the sense of ISO 10218-2:2025 (International Organization for Standardization 2025b), and the body model of the hand that bounds it, with the contact force that model predicts, is set out once in section 12.7.2. ISO 10218-2 leaves out the hazards of mounting the arm on a mobile platform, which ANSI/A3 R15.08-2 addresses for the running machine (Association for Advancing Automation 2023).

The ceiling behind that cap comes from the contact-force model (equation 12.13). For a free hand, which recoils, the selected 50 N criterion admits up to 0.24 m/s, and the cap sits under that ceiling. A hand clamped against a rack upright or the arm itself cannot recoil, and the same criterion then admits only 0.053 m/s, below the cap. The cap therefore holds only if the handover geometry excludes clamping, a premise the release case must evidence. The ceiling is conditional on these inputs and does not by itself establish a pressure or injury limit for a particular contact geometry; the site's risk assessment qualifies the effective mass, the force criterion, and the contact geometry before the cap is qualified.

↪ The release criterion that holds the handover's contact force under its assessed limit is developed in section 16.2.

Each capability entry binds actor, operation, channel, deadline, authentication, preemption tier, and fallback; a valid signature alone grants no motion authority. Physical clearance sets the tolerable deadline; the operator's response time determines whether a human can meet it.

14.2.2 Human reaction time and travel before braking

The remote assistant must notice the blocked aisle, decide how to recover, move the controls, and deliver a checked request. Summed in sequence, the total supervisory takeover latency T_Σ is:

$$T_\Sigma = T_{\text{sensory}} + T_{\text{cognitive}} + T_{\text{motor}} + T_{\text{comm}} \tag{14.1}$$

The first three terms belong to the person and depend on engagement and expectation; the last belongs to the console and the link. Human response times are distributions with long right tails, so a stopping budget needs a value that nearly every request meets, not the typical one. The calculation below compares a continuously engaged operator with an out-of-loop assistant. Its communication term includes transport and the protocol exchange, so that exchange must not be charged again.

Waiting consumes distance before the selected stop begins. The finished stopping budget from section 12.3.1 supplies the machine delay, resident stop, and fixed allowances. Adding the human waiting interval exposes whether remote rescue can fit at the current speed; it does not authorize waiting when the local stop is already required.

The budgets that follow are illustrative design comparisons, not measured takeover performance. Each of them must also pay for the dispatch-to-admission interval Δ_{da}, which section 13.6.2 allocated but has not yet measured. A takeover does not remove that interval. The policy must still infer on the frame that shows the blocked aisle before it can request help, and its request, like any proposal, reaches the arbiter only through the safety microcontroller, so the base keeps rolling through Δ_{da} before the human response even begins. This chapter calls a budget without the interval the *partial budget* and a budget with it the *complete budget*. The stops and overruns below use the complete budget. The speed ceilings and deadlines give the partial value first and then the charged one, because the gap between the two is itself a design lesson.

The stopping budget of table 12.1 shows what equation 14.1 costs on the mobile manipulator. At the 1.3 m/s aisle speed it leaves 131.0 mm of the 1.10 m rack-end clear distance before dispatch-to-admission travel, and 48.7 mm once the 82.3 mm of travel during Δ_{da} is charged. A takeover delay adds to the machine's own pre-brake delay τ_{delay}, and every millisecond of it becomes travel at the aisle speed:

1. **Mode 1: Continuous in-loop teleoperation**: An attending operator with hands on the controls of a synchronized leader arm or a real-time console, responding to the stimulus the task has led them to expect, makes a simple reaction. The Model Human Processor[3] (Card et al. 1983) prices that reaction as one perceptual, one cognitive, and one motor cycle (Jastrzembski and Charness 2007). At the slow end of each cycle's range, $T_{\text{sensory}} = 200$ ms, $T_{\text{cognitive}} = 170$ ms, and $T_{\text{motor}} = 100$ ms. The console adds $T_{\text{comm}} = 135$ ms: the scene reaches the operator after a video glass-to-glass latency[4] of 104 ms, measured for a wired teleoperation link (Schimpe et al. 2022), and the request returns through a 31 ms protocol exchange, the four-phase handshake that section 14.3.3 builds and budgets phase by phase. By equation 14.1, $T_{\Sigma,1} = 605$ ms. The machine travels 786.5 mm during that delay, several times the spare margin. Solving the complete budget for speed, with $T_{\Sigma,1}$ and Δ_{da} added to τ_{delay}, bounds the machine to 0.797 m/s. The value holds only for the expected stimulus. A driver who is waiting for a brake signal needs up to 0.75 s to detect it and reach the pedal, and a surprise raises that to about 1.5 s (Green 2000). Section 14.2.3 prices those cases.
2. **Mode 2: Passive out-of-the-loop supervisory takeover**: An assistant who monitors the telemetry of several machines passively does not make a simple reaction, and its stages cannot be priced separately. Driver take-over studies measure the whole interval, from the take-over request to the first control input. Across 520 experimental conditions in 129 studies, the mean take-over time averaged 2.72 s, and the distribution of those condition means is skewed to the right (Zhang et al. 2019). For this illustrative comparison, the selected design value is the 90th percentile of the condition means, 4.33 s, computed from the study's published data. With the same 135 ms console term, the takeover delay is $T_{\Sigma,2} = 4.5$ s, during which the machine covers 5.80 m, more than five times the clear distance, before any brake command. The same solution bounds the machine to 0.189 m/s.

In Mode 2 at the aisle speed, the machine would reach a static obstacle at the rack end at full speed, because contact occurs before braking ever begins. Stronger braking can raise a finite stopping-speed

[3] **Model Human Processor**: Card et al. (1983) modeled a person responding to a stimulus as three processors in series, perceptual, cognitive, and motor, and gave each cycle time a typical value and a range rather than a single number. The range is the design input. A budget built on the typical cycles admits a speed that a slower but still normal operator overruns, so this chapter's design row takes the slow end of every range.

[4] **Glass-to-Glass Latency**: The interval from light entering the remote camera's lens to the matching image leaving the operator's display, covering exposure, encoding, transport, decoding, and display refresh. It is the only video delay the operator experiences, and every stage of that path contributes to it, so a value measured on a wired link does not carry over to a warehouse wireless link. The site must measure its own link before the value enters the takeover budget.

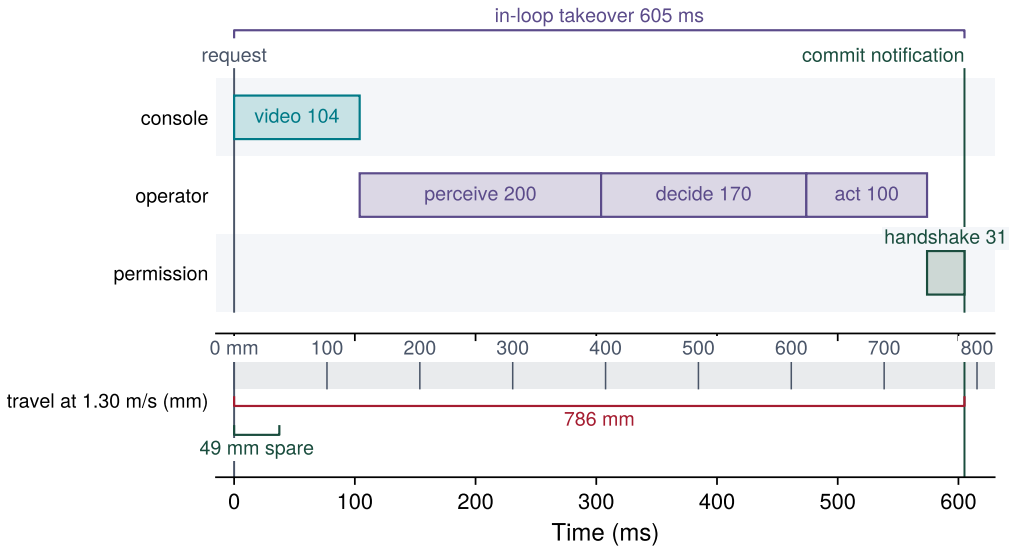


Figure 14.2: Where an in-loop takeover spends its time: The 605 ms in-loop takeover drawn to scale from the request, stage by stage: video transport to the console, the operator's perceptual, cognitive, and motor cycles at the slow end of the Model Human Processor, and the four-phase handshake through commit notification. Local commit starts the enforcer-owned blend before notification; the full exchange remains the conservative waiting allocation. The distance row converts the same axis into travel at the 1.3 m/s aisle speed; the bracket at its left marks the 48.7 mm spare of the complete budget, which the machine exhausts within the video stage.

ceiling, but cannot recover distance already spent waiting. As braking distance tends toward zero, reaction travel still sets a limiting ceiling. A percentile of condition means also understates the tail an individual operator can produce, because each condition's own spread grows with its mean; the site must measure its operators, console, and task load before any of these values admits a speed. Road regulation sizes the same interval generously for a vehicle that can keep driving while it waits. For an unanswered transition demand in the cited 2021 version of UN Regulation No. 157, an automated lane-keeping system begins its minimum risk manoeuvre no earlier than 10 s after it issues the demand (United Nations Economic Commission for Europe 2021), 2.2× the design takeover here. Severe system or vehicle failures permit an immediate minimum risk manoeuvre, and an obstacle can require an emergency response during the transition phase. The vehicle can afford that wait because it stays in its lane under automated control throughout. At the aisle speed the same wait would carry the mobile manipulator 13 m, so the machine cannot borrow the road's interval, only its structure of a demand, a bounded wait, and an automatic fallback. Section 14.2.4 turns the two design values into the speeds at which the machine may request each kind of takeover.

Drawn to scale, the Mode 1 budget shows where the clearance goes (figure 14.2). The 48.7 mm spare of the complete budget corresponds to 37.5 ms at the 1.3 m/s aisle speed, so the machine has spent it long before the 104 ms video stage ends and before the operator has perceived anything. The perceptual, cognitive, and motor cycles and the handshake that follow are all travel the budget cannot absorb, 786.5 mm in total.

Napkin Math 14.1: The takeover tax

Problem: At the 1.3 m/s aisle speed, does the finished stopping budget still fit the rack-end clear distance once an in-loop or an out-of-loop takeover delay is inserted before braking, and by how much does each overrun it?

Variables:

- $v = 1.3$ m/s (Chosen) and $D_{\text{clear}} = 1.10$ m (Illustrative).

- $\tau_{\text{delay}} = 133.6$ ms, the machine's own pre-brake delay (Derived; table 12.1), which excludes the dispatch-to-admission interval, and $\Delta_{\text{da}} = 63.3$ ms, the allocation of section 13.6.2 (Illustrative), which a takeover still spends before braking.
- $d_{\text{suffix}}(v) = v^2/(2a_{\text{brake}}) + v\,a_{\text{brake}}/(2j_{\text{max}})$, the jerk-limited resident stop of equation 11.17, with $a_{\text{brake}} = 2$ m/s² (Illustrative) and $j_{\text{max}} = 7.11$ m/s³ (Derived).
- $\delta = \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} = 190$ mm (Illustrative and Chosen).
- $T_{\Sigma,1} = 605$ ms in-loop and $T_{\Sigma,2} = 4.5$ s out-of-loop (Derived, from the measured human and transport terms of Modes 1 and 2 plus the chosen handshake).

Math: Every term is in millimeters. The machine alone, charged with the dispatch-to-admission travel, leaves a 48.7 mm spare:

$$d_{\text{stop},0} = v\,\tau_{\text{delay}} + v\,\Delta_{\text{da}} + d_{\text{suffix}}(v) + \delta = 173.68 + 82.30 + 605.31 + 190 = 1051.30 \text{ mm}$$

Each takeover adds $v\,T_{\Sigma}$ before the same stop:

$$d_{\text{stop},1} = v\,\tau_{\text{delay}} + v\,\Delta_{\text{da}} + v\,T_{\Sigma,1} + d_{\text{suffix}}(v) + \delta = 173.68 + 82.30 + 786.5 + 605.31 + 190 = 1837.80 \text{ mm}$$

$$d_{\text{stop},2} = v\,\tau_{\text{delay}} + v\,\Delta_{\text{da}} + v\,T_{\Sigma,2} + d_{\text{suffix}}(v) + \delta = 173.68 + 82.30 + 5804.5 + 605.31 + 190 = 6855.80 \text{ mm}$$

Result: The in-loop takeover overruns the clear distance by 737.8 mm and the out-of-loop takeover by 5.76 m; the second stop is 3.7× the first.

Systems insight: The waiting term, not the braking term, decides the verdict. The in-loop takeover alone covers 16.1× the spare, so no stronger brake can make $d_{\text{stop}} \le D_{\text{clear}}$ hold at this speed; only a lower v or a stop before the request can. The delay, braking, and allowance terms follow the stopping-distance model of section C.13.

14.2.3 Takeover speed ceilings from available clearance

Because no braking force can recover distance spent waiting, the machine must bound its operating speed before an intervention is ever requested. One inequality sets both bounds that this section and the next derive. If the machine continues at speed v for a waiting duration T_{wait} while awaiting the operator's response, the finished stopping budget of section 12.3.1 gains one term, the travel during that wait. The pre-brake delay τ_{delay}, the jerk-limited resident stop $d_{\text{suffix}}(v)$ (equation 11.17), and the fixed allowances keep the meanings defined there, and the sum must still not exceed D_{clear} measured at the same reference event:

$$d_{\text{stop}}(T_{\text{wait}}) = v(T_{\text{wait}} + \tau_{\text{delay}}) + d_{\text{suffix}}(v) + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} \le D_{\text{clear}}. \tag{14.2}$$

Setting $T_{\text{wait}} = T_{\Sigma}$ and solving for speed gives the takeover speed ceiling of this section. Fixing the speed and solving for T_{wait} gives the latest fallback deadline of section 14.2.4.

The three stops of notebook 14.1 differ only in how long the machine coasts before an identical jerk-limited resident stop begins (figure 14.3). Every trace ends with the same braking curve, so the whole difference between stopping 48.7 mm short of the clear distance and stopping at 6.86 m is coasting.

Table 14.2 prices each sourced response time. Every row adds the 135 ms console term to the human response, adds the total to τ_{delay}, and solves the finished stopping budget for the largest speed at which the machine still stops inside the rack-end clear distance. Before charging the missing admission delay, the ceiling falls from 1.01 m/s for an attending operator who meets the expected stimulus at the nominal pace to 0.12 m/s for the slowest percentile of takeover conditions. No row fits at the 1.3 m/s aisle speed. Two facts dominate the table. Expectation matters more than the console, because moving from an expected to a surprising stimulus costs more speed than the whole

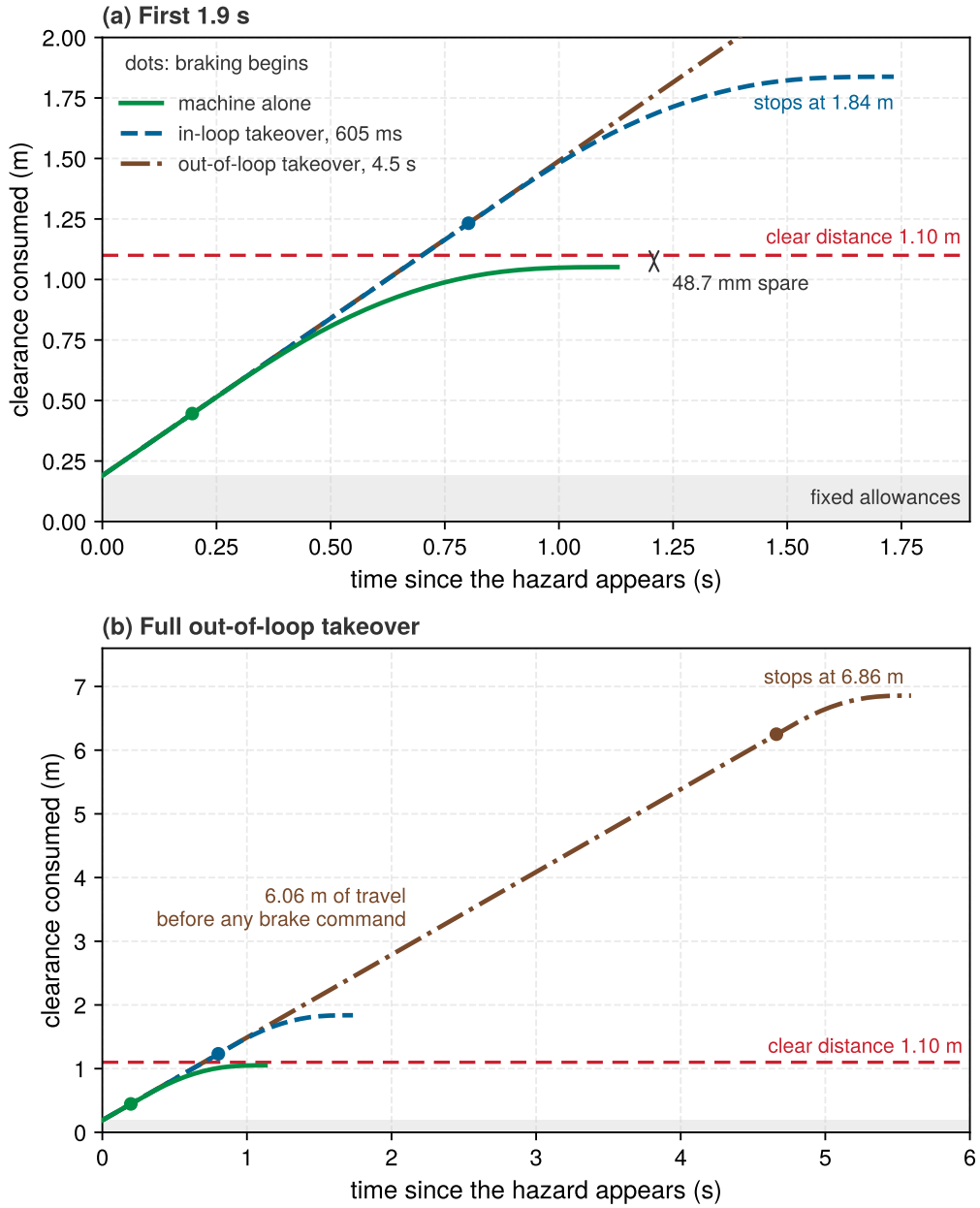


Figure 14.3: Two takeovers against one clear distance: Clearance consumed at the 1.3 m/s aisle speed, starting from the fixed allowances. Each trace coasts through its delay, including the dispatch-to-admission interval, and then follows the same jerk-limited resident stop. The machine alone stops 48.7 mm short of the 1.10 m clear distance. A 605 ms in-loop takeover inserted before braking carries the stop to 1.84 m, and a 4.5 s out-of-loop takeover carries it to 6.86 m. The waiting, not the braking, spends the clearance.

console term does. The tail matters more than the average, because a speed chosen for the mean take-over time is overrun by every condition slower than the mean.

A ceiling above the resident stop's transition speed $a_{\text{brake}}^2/j_{\max}$ is the positive root derived for the machine's own stop in equation 12.2, where the half jerk ramp acts as an added delay before full

braking (the rigid-stop form is equation 2.3). A takeover adds one more such interval, so setting $T_{\text{wait}} = T_{\Sigma}$ in equation 14.2 folds the human delay into the same root:

$$v_{\max}(T_{\Sigma}) = a_{\text{brake}} \left[\sqrt{b^2 + \frac{2\,(D_{\text{clear}} - \delta)}{a_{\text{brake}}}} - b \right], \qquad b = \tau_{\text{delay}} + T_{\Sigma} + \frac{a_{\text{brake}}}{2 j_{\max}}. \tag{14.3}$$

The term b collects every interval during which the machine still moves at v: its own delay, the human's, and the half jerk ramp before full braking. Below $a_{\text{brake}}^2 / j_{\max}$ the stop never reaches full deceleration and the cubic branch of equation 11.17 applies instead, which is why the table's slower rows come from the solver rather than from this root.

Napkin Math 14.2: The speed a human buys

Problem: What is the highest speed at which the mobile manipulator may request an in-loop takeover from an attending operator, and does any brake, however strong, admit the aisle speed?

Variables:

- $a_{\text{brake}} = 2$ m/s² (Illustrative) and $j_{\max} = 7.11$ m/s³ (Derived).
- $\tau_{\text{delay}} = 0.1336$ s (Derived) and $T_{\Sigma,1} = 0.605$ s (Derived, from the measured human and transport terms of Mode 1 plus the chosen handshake).
- $D_{\text{clear}} - \delta = 0.91$ m (Illustrative, less the fixed allowances).

Math: $b = 0.1336 + 0.605 + 2\,/(2\times 7.11) = 0.1336 + 0.605 + 0.1406 = 0.8792$ s. Then $b^2 + 2\times 0.91\ /\ 2 = 1.6830$ s², whose square root is 1.2973 s, so by equation 14.3 $v_{\max} = 2 \times (1.2973 - 0.8792)$. The root lies above $a_{\text{brake}}^2 / j_{\max} = 0.562$ m/s, so the quadratic branch was the right one. If both the deceleration and jerk bounds are removed so that the resident stopping distance tends to zero, the limiting ceiling is $(D_{\text{clear}} - \delta)/(\tau_{\text{delay}} + T_{\Sigma})$.

Result: $v_{\max} = 0.836$ m/s, the design in-loop row of table 14.2; instantaneous braking would raise it only to 1.23 m/s.

Systems insight: Even a perfect brake leaves the ceiling below the 1.3 m/s aisle speed, because the travel during $\tau_{\text{delay}} + T_{\Sigma}$ alone exceeds the clear distance at that speed. The waiting term of $d_{\text{stop}} \le D_{\text{clear}}$ sets the request speed, so faster brakes cannot buy back a slow human; adding the 63.3 ms admission allocation to b lowers the root to 0.797 m/s.

Table 14.2: Takeover speed ceiling against the sourced human response: The first two rows are Model Human Processor simple reactions, one perceptual, cognitive, and motor cycle at nominal and slow-bound cycle times (Jastrzembski and Charness 2007). The next three are driver perception-brake times for an expected, an unexpected, and a surprise signal (Green 2000). The last four are percentiles of the mean take-over time across 520 experimental conditions, computed from the published data of Zhang et al. (2019); they describe conditions, not individual operators, whose tail is longer. Each total adds the console term (video glass-to-glass latency (Schimpe et al. 2022) plus the four-phase exchange) and excludes the dispatch-to-admission interval, so every ceiling is optimistic; charging that allocation lowers the design in-loop ceiling to 0.797 m/s. Bold rows are the design values.

Operator state and response	Human response	With console term	Added travel at aisle speed	Speed ceiling
In-loop, expected stimulus, nominal pace	240 ms	375 ms	0.49 m	1.01 m/s
In-loop, expected stimulus, slow bound (design)	470 ms	605 ms	0.79 m	0.84 m/s
Expected brake signal	750 ms	885 ms	1.15 m	0.68 m/s
Unexpected but common signal	1,250 ms	1,385 ms	1.80 m	0.51 m/s
Surprise event	1,500 ms	1,635 ms	2.13 m	0.45 m/s
Out-of-loop takeover, median condition	2,370 ms	2,505 ms	3.26 m	0.32 m/s

Operator state and response	Human response	With console term	Added travel at aisle speed	Speed ceiling
Out-of-loop takeover, average condition	2,720 ms	2,855 ms	3.71 m	0.29 m/s
Out-of-loop takeover, 90th percentile (design)	4,330 ms	4,465 ms	5.80 m	0.19 m/s
Out-of-loop takeover, 99th percentile	7,000 ms	7,135 ms	9.28 m	0.12 m/s

The table also locates the other interfaces a site might offer. Continuous velocity guidance or supervisory waypoint dispatch places the operator somewhere between an expected-stimulus reaction and an out-of-loop takeover, and only a measured response distribution for that interface, console, and task says where. A hardwired protective input removes the console term but not the human response, so its operator still needs the local stop to hold the machine safe while that response runs.

14.2.4 Clearance-derived fallback deadlines

If a machine reaches the edge of its operating domain at speed and requests a takeover, an operator who is distracted, asleep, or incapacitated will fail to complete the handshake before the machine runs out of clearance. An autonomous policy's handover request can therefore never wait indefinitely, nor can its timeout be a fixed software constant: a single fixed window that is safe at a crawl will overrun the clearance at transport speed (section 14.9 prices such a window at both speeds). Safe timeouts must instead be derived at runtime from the machine's current speed, line-of-sight clearance, and braking deceleration.

The deadline is the second reading of the stopping inequality. The speed ceiling fixed the wait and solved for speed; the deadline fixes the current speed and solves for the wait. Setting equality in equation 14.2 ($d_{\text{stop}}(T_{\text{wait}}) = D_{\text{clear}}$) and solving for the maximum permissible waiting time yields the latest fallback deadline $T_{\text{deadline,fallback}}$:

$$T_{\text{deadline,fallback}} = \frac{D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - \epsilon_{\text{track}} - d_{\text{suffix}}(v)}{v} - \tau_{\text{delay}}, \qquad T_{\text{timeout}} = \max\Big(0, \min(T_{\text{config}}, T_{\text{deadline,fallback}})\Big). \tag{14.4}$$

For a live check, the safety microcontroller dispatches fallback **no later than** the last permission tick before the resulting deadline. A nonpositive $T_{\text{deadline,fallback}}$ calls for immediate fallback and may mean stopping is already infeasible. The configured protocol ceiling cannot extend that physical deadline or the expiry of active permission.

Napkin Math 14.3: The waiting allowance

Problem: How long may the mobile manipulator wait after a handover request before it must dispatch its own stop, at the 1.3 m/s aisle speed and at the 0.75 m/s slow-down, and does the 605 ms in-loop takeover fit either allowance?

Variables:

- $D_{\text{clear}} - \delta = 0.91$ m (Illustrative) and $\tau_{\text{delay}} = 0.1336$ s (Derived).
- $d_{\text{suffix}}(v) = v^2/(2a_{\text{brake}}) + v\,a_{\text{brake}}/(2j_{\max})$ (Derived; equation 11.17): 0.6053 m at 1.3 m/s and 0.2461 m at 0.75 m/s.
- $T_{\Sigma,1} = 605$ ms (Derived, measured human and transport terms plus the chosen handshake) and $\Delta_{\text{da}} = 63.3$ ms (Illustrative; the unmeasured allocation of section 13.6.2).

Math: By equation 14.4, $T_{\text{deadline,fallback}} = (0.91 - 0.6053)/1.3 - 0.1336 = 0.1008$ s at the aisle speed, and $(0.91 - 0.2461)/0.75 - 0.1336 = 0.7516$ s at the slow-down. Reserving the takeover leaves 751.6 ms − 605 ms = 146.6 ms, and charging Δ_{da} leaves 83.3 ms.

Result: At the aisle speed the takeover overruns a 100.8 ms allowance; at the slow-down it fits with 83.3 ms to spare on the complete budget.

Systems insight: Slowing from 1.3 to 0.75 m/s multiplies the waiting allowance 7.5×, because both the stop term and the travel per millisecond of $d_{\text{stop}} \le D_{\text{clear}}$ shrink with speed. The deadline is a function of the current state, so no single configured timeout can be correct at both speeds.

At the aisle speed, the 100.8 ms of notebook 14.3, or 131.0 mm of travel, is the unallocated timing allowance of the stopping subtotal in section 12.3.1, and it is a partial-budget value. Charging the 63.3 ms dispatch-to-admission allocation of section 13.6.2 completes the budget. At speed v the charge costs $v\Delta_{\text{da}}$ of travel, so at the aisle speed only 37.5 ms of waiting survives it. Wherever the rest of the chapter reports a waiting allowance, it gives the partial value and then the charged one, and the charged value is the one a design must meet. Rounded down to whole permission ticks, the partial aisle-speed allowance of 100.8 ms becomes 100 ms, which illustrates tick rounding rather than an absolute fallback time. The 500 ms configured ceiling is only an outer protocol limit, because the clearance deadline binds first.

Both readings of the stopping inequality also need a clock origin, and the chapter uses the two that table 14.3 sets side by side. The design budget behind every displayed number counts from the observation's capture. A live check on the safety microcontroller counts from now.

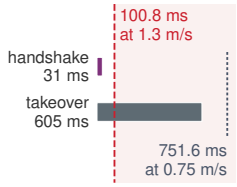
Table 14.3: Two clock origins for one stopping inequality: Either column is correct on its own. Mixing a cell from one column with a cell from the other charges elapsed travel twice or not at all.

	Design budget (capture origin)	Live check (now origin)
Where it is used	Every number this chapter displays	The safety microcontroller, at each permission tick of a pending request
Clear distance	D_{clear} measured at the observation's capture event	The current tick's remaining clearance, already reduced for the motion since capture
Delay before braking	$\tau_{\text{delay}} + \Delta_{\text{da}}$, from capture to the onset of the physical response	Only the delay still ahead: dispatch to response onset plus any unpaid processing, never a fresh full lease
Meaning of the wait	T_{wait} is additional delay inserted into that budget, with any overlap with its processing, lease, or transport intervals removed	The time left before the last permission tick at which a stop still fits

The two ways to mix them fail in opposite directions. A live check that takes the current clearance, which already reflects the travel since capture, and then also subtracts the capture-origin delay pays for the elapsed part of that delay twice and dispatches its fallback early, which costs availability. The reverse mix, the capture-time clearance with a delay counted only from now, never pays for the travel since capture and dispatches its fallback late, which costs clearance.

The 31 ms protocol exchange fits the aisle-speed allowance on both budgets. The sourced 605 ms in-loop response does not. It exceeds the aisle-speed allowance several times over, so an in-loop request needs a slower machine, and the human response time, not the machine, sets how much slower. Assuming a faster human gives the wrong answer. A slow-down to 1.2 m/s would suit an operator who responds in a small fraction of a second; there the budget leaves 184.1 ms, and the sourced response overruns it by 420.9 ms. Such a design passes every check that uses an optimistic human and fails the first operator who needs the time the literature measures.

The design slows to 0.75 m/s before requesting an in-loop takeover. That speed comes from solving the budget with both the sourced response and Δ_{da} added to τ_{delay}, which gives a complete-budget ceiling of 0.797 m/s, rounded down to the next 0.05 m/s step. At that speed the capture-origin budget leaves 563.7 mm, or 751.6 ms of additional delay. Reserving the response leaves 146.6 ms, and charging Δ_{da} still leaves 83.3 ms. That is a conservative design allocation, not a claim that every request at that speed receives the same live timeout, and it holds only while the operator meets the expected stimulus. An unexpected hazard moves the operator to the slower rows of table 14.2, whose ceilings the slow-down exceeds.



The handshake fits the aisle-speed allowance; the whole takeover needs the slow-down.

Solving with both intervals matters. The partial-budget ceiling, with the response alone added, is 0.84 m/s, and a designer who stopped there might choose 0.8 m/s, which sits under it. At that speed the response leaves 58.3 ms, less than the 63.3 ms allocation, so once the omitted interval is charged the slow-down misses its deadline by 5.0 ms. A budget that leaves out one interval of a few tens of milliseconds admits a speed that the complete budget rejects.

The out-of-loop takeover forces a larger change. Its 4.5 s design response caps the request speed at 0.19 m/s, so the crawl before an out-of-loop request is 0.15 m/s, where the budget leaves 5.79 s and the design response leaves 1.32 s of it unspent. The crawl survives the Δ_{da} charge, which lowers its ceiling only to 0.189 m/s. It still fails the 99th-percentile condition, whose 7.1 s response admits only 0.12 m/s, and an individual operator's tail is longer than any condition mean. A crawl slow enough for every operator is no longer motion worth keeping. The robust design therefore stops the machine automatically before handing over. The local stop runs first, and the out-of-loop takeover begins from standstill, where waiting costs time but no clearance. Both moving candidates still require qualified admission delay, current clearance, and a feasible stopping continuation. The hidden pallet leaves no room for an anticipatory slow-down at all, so the machine answers it with its own local stop and requests the takeover from standstill. A planned slow-down or crawl can prepare an anticipated request; neither permits waiting at aisle speed for a newly discovered obstacle.

Every one of these designs rests on two properties the chapter has so far assumed. Authority, once granted, must not outlive a proposer that falls silent, and it must move between proposers on a single tick without two of them ever commanding the same channel. Section 14.3 builds both.

Checkpoint 14.1: Takeover budgets and speed ceilings

A takeover is a delay inserted before braking, so every question about it is a question about the stopping budget.

- ☐ **Waiting vs. braking**: Can you explain, from the limiting case of equation 14.3, why no brake however strong admits an in-loop takeover at the aisle speed?
- ☐ **Ceiling estimate**: An operator who waits for an expected brake signal responds in 750 ms, plus the 135 ms console term. Using the givens of notebook 14.2, can you estimate the speed at which that operator may be asked to take over, and check it against table 14.2?
- ☐ **Clock origins**: Can you explain why a live check that measures clearance from now must not also charge the delay measured from capture?

14.3 Authority Transitions

When a remote assistant takes manual control of the arm after the mobile base has slowed or stopped for a blocked aisle, authority must move between proposers on a single permission tick without two of them ever commanding the same channel, and it must not outlive a proposer that falls silent. The transfer names each actuator channel and assigns it to a single owner; linear blending of two competing proposers is no substitute for choosing who is in charge. The local enforcer remains the sole writer to the physical actuator bus, generating a checked transition profile only after an ownership change has committed on a control tick boundary.

Discrete handover is not the only way to share control. Shared autonomy systems keep both proposers active and blend their actions continuously. In standard policy-blending methods, the robot infers the operator's intended goal and arbitrates between automated assistance and manual control; user studies show that this arbitration works only while the robot predicts the human's goal with high confidence (Dragan and Srinivasa 2013). That confidence breaks down when there is more than one path around an obstacle. When an operator steers left around a pallet while the autonomous policy steers right, a convex combination of two sensible commands produces a third trajectory that hits the obstacle. In those cases, a qualified arbiter must select or construct one admissible proposal; the permission path here selects a single source deterministically at each tick. Continuous blending works well when the task has a single clear goal and every intermediate blend is admissible, as when an operator makes fine adjustments to an end-effector pose. Even in purely manual operation, the safety enforcer acts as an invariant filter rather than a proposer, projecting

every human command onto the admissible control set regardless of who holds the channel. Before ownership can move between sources, however, each source's hold on a channel must be bounded in time so that a proposer that falls silent loses authority instead of leaving its last command in force.

14.3.1 Execution leases and three-position enabling hardware

A stalled policy and a disconnected teleoperator leave the same immediate problem, because in both cases the moving machine still holds a previously admitted command. An inference stall, lost wireless link, or incapacitated operator must not leave that command active indefinitely. Expiring the proposer's authority must also select a local response that brakes the machine or holds the load.

The permission path bounds autonomous chunks and manual sessions with **execution leases**, enforced on the safety microcontroller's clock. The autonomous lease is the running 60 ms chunk lease. The manual lease is a separate value, chosen in section 14.3.4 and checked against the remaining clearance in section 14.4.6. An authenticated renewal must also carry valid state and evidence, because a heartbeat alone cannot refresh old observations. A wireless console that carries these renewals is a cableless control system in the sense of IEC 62745, which specifies the functions and interfaces of radio and infrared links between operator control stations, including portable ones, and a machine's control system (International Electrotechnical Commission 2017). Formally, let t_{refresh} denote the local hardware timestamp of the most recent renewal accepted after authentication, freshness, state, and evidence validation. At physical time t, the lease-age condition is:

$$t - t_{\text{refresh}} < T_{\text{lease}} \text{ (autonomous)}, \qquad t - t_{\text{refresh}} < T_{\text{lease,man}} \text{ (manual)}. \tag{14.5}$$

The clock advances toward expiry even when packets stop arriving. At expiry, the microcontroller revokes the affected proposal rights and selects the validated response from the current physical state. Authority can return only through the declared recovery and fresh-admission procedure; packet resumption alone cannot restore it.

Electromechanical **enabling devices** provide an additional physical condition for manual motion, one that holds even when every network packet is valid. The industrial teach pendant in figure 14.4 is the familiar carrier: a technician standing beside a robot uses it to jog the arm by hand, with a display, a jogging joystick, and a latched emergency-stop button on its face and an enabling grip on its rear handle. On the running machine, an enabling device belongs to a person beside the machine, such as the maintenance technician who jogs one arm joint from a pendant. The remote assistant's console need not carry such a grip, because authenticated heartbeats maintain the manual lease, with presence assessed by the console's chosen checks, which do not establish cognitive engagement (section 14.3.4). A site may add an enabling grip to the console as a second condition. The grip that matters is a three-position one.[5] A startled or distressed operator may release a grip or squeeze it harder, and a two-position device that enables whenever pressed does not detect the latter response. By contrast, a three-position grip enables motion only in its intermediate position (Position 2). Releasing it (Position 1) or squeezing beyond the center detent (Position 3) withdraws the enabling signal; the configured safety circuit determines the resulting stop. These dual-channel contacts request the stop selected by the machine's safety circuit, independently of the Linux application processor and its scheduler. Where that response includes Safe Torque Off (STO, IEC 61800-5-2), STO disables torque generation; the configured braking or holding mechanism must still establish the required plant response. The enabling circuit must also prevent a return through Position 2 after an over-squeeze from automatically restoring motion.

The pendant's red mushroom button is a different device with different semantics. Under ISO 13850, the emergency stop is a complementary protective measure, not a substitute for safeguarding or for the stop the risk assessment selects. It takes priority over every other function in every operating mode, it latches until an operator deliberately resets it at the device, and that reset must not restart the machine (International Organization for Standardization 2015b). Which physical response the stop invokes is a stop-category choice that table 12.3 maps to its evidence. The latch and the reset rule matter for supervisory control because they make a human stop sticky in exactly the way a network command is not. A remote packet can request a stop and a lease can lapse into one, but only the local reset of a latched device, followed by the fresh admission this chapter requires, returns the machine to motion.

↩ The minimal-intervention principle, which keeps an admissible proposal and applies the smallest feasible correction otherwise, is established in section 12.7.

5 **Manual Reduced Speed**: ISO 10218-1:2025 limits the tool speed of an industrial robot in manual reduced-speed mode to 250 mm/s (International Organization for Standardization 2025a), the mode in which the pendant of figure 14.4 couples motion to a three-position grip. Releasing or over-squeezing the grip only requests a stop from the safety circuit, and the performance level that circuit must reach, graded from PL a to PL e, is set by the machine's risk assessment under ISO 13849-1 (International Organization for Standardization 2023a). The grip therefore bounds how fast the machine may be moving when a startled operator lets go, not which stop follows.

Figure 14.4: Industrial teach pendant with three-position enabling grip: Front view of an ABB teach pendant showing the touch display, manual jogging joystick, emergency-stop mushroom button, and industrial fieldbus cable. The three-position enabling grip described in the text sits on the rear handle, out of view. (Credit: Auledas, Wikimedia Commons, CC BY-SA 4.0.)

DRAFT

Even while an operator holds an active lease and maintains Position 2 on the deadman grip, the deterministic enforcer evaluates human setpoints against the same barrier certificates, current limits, and kinematic envelopes applied to the learned neural policy. If an operator panics and commands a high-speed steering angle that would induce dynamic chassis rollover, or requests a joint torque that would shear harmonic drive gear teeth, the enforcer clamps or rejects the command. A human command therefore remains a proposal, and the third law, proposal is not permission (principle 3), binds human supervisors and neural models alike. Intervention transfers the right to propose away from the learned policy, while the right to admit stays with the permission path. With every owner's authority bounded by a lease, what remains to design is how the right to propose changes hands, at which control tick it commits, and what the machine does when a network handover stalls.

14.3.2 Authority states and single-writer actuator ownership

If the autonomous policy and the remote console both write to a motor drive while a handover is in flight, the drive receives alternating setpoints on consecutive ticks. To prevent two sources from commanding an actuator at once, the arbiter resolves authority over each channel at every discrete time tick t to a single, mutually exclusive state:

$$S_t \in \{\text{Autonomous}, \text{Handover-Pending}, \text{Blend}, \text{Human-Manual}, \text{Fallback}, \text{Protective}\}.$$

These states separate proposer identity from actuator access. Each carries a short code name, used in the per-tick record and in figure 14.5, and each names who holds the proposal token on the channel:

- `Autonomous` (`AUTO`): The learned model holds the proposal token and proposes setpoints, while the local enforcer verifies barrier certificates ($h(\mathbf{x}) \geq 0$) on every control cycle before dispatching commands.
- `Handover-Pending` (`PEND`): A takeover handshake is negotiating across the network; the policy keeps the token and continues proposing commands under a strict clearance-derived countdown timer T_{timeout}. Active lease and evidence expiry run concurrently with that timer, so the arbiter dispatches fallback at the earliest of the protocol, lease, evidence, and clearance deadlines.

- `Blend` (`BLEND`, enforcer-owned bridge): No proposer holds the token. The enforcer assumes exclusive bus ownership, driving a smooth, C^2-continuous quintic interpolation between the latched autonomous setpoint and the validated human target setpoint.
- `Human-Manual` (`MAN`): The authenticated teleoperator holds the proposal token, streaming commands that pass through enforcer barrier checks under an active manual lease ($T_{\text{lease,man}}$).
- `Fallback` (`FALLBACK`): Entered when an active lease expires, communication drops, or no admissible continuation remains. No proposer holds the token; the arbiter selects the validated HOLD or STOP rung for the measured state without waiting for remote commands. An ordinary least-change correction, by contrast, is enforcement within the current state and leaves the token with the current proposer.
- `Protective` (`PROTECTIVE`): Entered directly from an independent hardwired protective input or a critical silicon fault. No proposer holds the token. The safety circuit selects the validated protective stop; Safe Torque Off (STO) disables torque generation, and any required load-holding mechanism must be qualified for the current state and surviving resources.

At no point do two software masters write to an actuator simultaneously. Even during the `Blend` state, the enforcer alone owns the actuator channel, executing a precomputed profile between the latched references and validating dynamic constraints against the current state on each tick. Each edge in table 14.4 therefore names its guard and deadline, its single actuator owner, and the fallback taken when the guard fails or an active lease lapses.

Table 14.4: Authority state machine transition matrix: Each row names the single actuator owner and the state-dependent fallback. The example uses a 1 ms permission tick. The `BLEND` state interpolates only latched, checked endpoints, under enforcer ownership.

From → to	Guard and deadline	Actuator owner / command	Failure response
Autonomous → Handover-Pending	Policy requests supervision; deadline computed from clearance	Enforcer admits policy proposals while feasible	Validated fallback if deadline reached
Autonomous or Handover-Pending → Blend	Authenticated operator ACK, fresh state, feasible target and stopping reserve; commit on a control tick	At commit, revoke the policy's proposal token, latch last approved command and checked manual target; **enforcer alone** writes actuator	Deny invalid target; retain valid active mode or fall back
Blend → Human-Manual	Profile completes and manual lease remains valid	Enforcer admits live human proposals within physical limits	Validated fallback on lease or enabling-device loss
Handover-Pending, Blend, or Human-Manual → Fallback	Timeout, stale lease, or invalid state	The permission path executes the validated HOLD or STOP rung and records it	Re-arm only after validated recovery procedure
Any state → Protective	Independent protective input or qualified critical fault	Safety circuit/controller selects plant-specific protective stop	Validated brake or hold, not torque removal alone

14.3.3 The four-phase takeover handshake

If a single network packet could switch a channel directly from `Autonomous` to `Human-Manual`, a delayed request could hand the actuators to an operator whose view of the aisle is already stale. With the same single packet, a dropped reply could leave the operator and the machine disagreeing on who is driving. Moving a channel between states instead takes a deterministic **four-phase takeover protocol** (`Request` → `Acknowledge` → `Commit` → `Confirm`) run by the safety microcontroller. This four-stage exchange separates the local hardware commit from the physical blend and from the final release of manual proposal rights, checks cryptographic freshness so replayed frames cannot trigger a switch, and changes ownership on a single tick boundary:

1. **Notice (`Request` phase)**: Either the human supervisor notices policy drift and requests an override (for example, by pressing an enabling button or moving a joystick past its calibrated deadband), or the autonomous policy hits an epistemic uncertainty spike and requests teleoperation assistance. Upon receipt, the arbiter initializes the clearance-derived countdown timer $T_{\text{timeout}} = \max(0, \min(T_{\text{config}}, T_{\text{deadline,fallback}}))$ of equation 14.4.

2. **Declutch (`Acknowledge` phase)**: The enforcer validates target prerequisites across the system. It verifies that sensor telemetry is fresh, network links are live, and the current plant state $\mathbf{x}(t)$ retains sufficient stopping reserve to brake safely if the transfer aborts. It then captures a synchronous state snapshot before preparing to revoke the autonomous policy's proposal authority.
3. **Transition (`Commit` phase)**: At the next synchronous control tick boundary, the enforcer executes the state transition. It revokes the policy's proposal token on the committed channels, latches the last approved policy setpoint alongside the checked manual target setpoint, and assigns sole write authority to the enforcer in the `Blend` state. The enforcer drives the C^2-continuous transition profile, continuously checking that the trajectory remains feasible under measured plant dynamics.
4. **Notification (`Confirm` phase)**: The permission path broadcasts confirmation of the committed owner and the active `Blend` state to the operator console. A dropped confirmation packet alarms the operator interface, but does not undo or stall the local commit on the robot. After the physical profile completes under valid state, sufficient stopping reserve, and an active manual lease, the arbiter separately grants the proposal token on the committed channels to the operator and transitions to `Human-Manual`. This release timing is dictated by physical profile completion rather than the network notification round trip.

Each phase in table 14.5 carries a strictly budgeted bound, an explicit guard and owner, and deterministic failure handling. The transaction rejects spoofed override attempts and applies the failure response assigned to its current phase. Lost confirmation does not reverse a committed transfer; lease expiry or a failed physical guard invokes the validated state-dependent fallback.

Table 14.5: Four-phase takeover timing: The phase budgets sum to 31 ms. This sum budgets the exchange through commit notification, not physical blending or release of manual proposal rights. The blend begins at commit and may continue after notification.

Phase / takeover stage	Budgeted bound	Guard and owner	Failure handling
1. Request (Notice)	10 ms	Authenticated role, checked nonce, and fresh clock-converted timestamp; no owner change	Reject invalid packet; keep current qualified mode
2. Acknowledge (Declutch)	5 ms	Check state, manual target, and full stop reserve; isolate write path	Reject request; invoke fallback if current mode cannot continue
3. Commit (Transition)	1 ms tick	Revoke the policy's proposal token, latch both checked endpoints, assign **enforcer** sole actuator write path	No partial commit within the requested channel mask; fallback on failed guard
4. Confirm (notification)	15 ms	Announce committed `BLEND`; later announce `MAN` after profile and lease check	Lost notice alarms operator but does not undo commit

14.3.4 Arbiter execution and manual-lease heartbeats

Once a channel reaches Human-Manual, a dropped wireless link or a distracted assistant who leaves the console would let the last manual command keep running unless the machine checks that the operator is still present. The safety microcontroller bounds that risk with a hardware-timed **manual lease**. Depending on the console, presence detection may use a capacitive grip sensor, eye tracking, or continuous joystick deflection past a deadband, chosen for their sensor coverage and false-drop rates. Manual authority is sustained exclusively through authenticated keep-alive heartbeats streamed from the operator console to the safety microcontroller. The manual lease $T_{\text{lease,man}}$ of section 14.3.1 is the longest silence the microcontroller tolerates between them. Two bounds set its length. From below, the lease must outlast the heartbeat period plus the link's jitter and at least one lost packet, or ordinary wireless loss would drop a healthy operator into fallback. That floor is why the heartbeats must run at a rate several times the lease's inverse. From above, the lease replaces the chunk lease inside τ_{delay} while the operator holds `MAN`, so the travel it adds over the chunk lease, $v(T_{\text{lease,man}} - T_{\text{lease}})$, must fit the clearance left after the takeover. The reference implementation chooses 100 ms. Section 14.4.6 checks that value against the upper bound at the in-loop request speed, and only the site's measured heartbeat loss can show that it clears the lower one.

Each heartbeat packet carries a strictly increasing 64-bit sequence counter k, a monotonic hardware timestamp t_k^{hb}, an authenticated operator credential, and a keyed message authentication code (AES-128 CMAC, evaluated in section 14.5). In the age calculation below, t_k^{hb} represents the sender's timestamp translated into the local clock domain. The receiver tracks the bounded clock-synchronization uncertainty interval and computes freshness relative to the oldest plausible timestamp; an unqualified clock mapping cannot safely establish packet freshness. The nominal local-clock age is:

$$\Delta t_{\text{heartbeat}} = t_{\text{local}} - t_k^{\text{hb}}.$$

These fields combine communication checks used in black-channel safety architectures (International Electrotechnical Commission 2021) with cryptographic security over an untrusted wireless link. The sequence counter detects transmission loss and replay attacks. The timestamp and bounded lease act as a communication watchdog against excessive latency. The cryptographic tag authenticates the claimed credential. The safety communication checks and the security mechanism require separate qualification.

A heartbeat packet may renew authority only while its conservative age falls within the configured freshness window. The active manual lease expires deterministically on the local hardware clock at its committed deadline; stale or out-of-order network traffic cannot extend that horizon. Similarly, an autonomous action chunk expires strictly according to its admitted local deadline. A packet that fails cryptographic verification is immediately discarded. It cannot renew the lease, but it does not revoke currently valid authority either; the existing timer runs down on its own. Upon lease expiration, the microcontroller automatically revokes the operator's proposal token, drops into `Fallback`, and executes a pre-validated holding or stopping trajectory. Once a lease lapses, the state machine prohibits any immediate return to Autonomous or Human-Manual. The robot must satisfy strict recovery conditions: it must come to a complete mechanical standstill, resolve and acknowledge the relevant faults without erasing their evidence, and undergo an authenticated four-phase re-admission handshake initiated from rest.

↩ The proposal header that applies the same black-channel safeguards to every learned proposal is established in section 4.4.1.

When multiple control sources propose concurrent commands, the permission path resolves candidate selection through a fixed priority hierarchy evaluated deterministically every 1 ms control cycle below the proposal boundary. This mechanism adopts the layered priority structure of subsumption architectures (Brooks 1986), but applies it strictly to input source selection rather than behavioral synthesis. The independent enforcer then admits, modifies, or rejects the winning candidate to ensure compliance with configured physical limits. In the Human-Manual state with a valid lease, the human proposal takes precedence over learned model proposals, while remaining bounded by barrier certificates and actuator limits. In Autonomous, the learned policy generates candidate actions for safety screening. Because fixed-state arbitration requires bounded work per tick ($\mathcal{O}(1)$ computational complexity), it eliminates runtime negotiation or dynamic bargaining between software proposers.

Two properties of the protocol show only when the states and handshake are drawn together (figure 14.5). Every active state has an edge to `Fallback`, so no failure of the network or the operator can leave a channel without a defined owner, and the only edge out of `Fallback` is a reset from standstill. The hardwired `Protective` input sits outside the protocol entirely and preempts every state.

↩ The proposal boundary, above which a learned component may only propose and below which the permission path alone admits commands, is established in section 1.7.

Here the state machine decouples physical bus ownership from protocol progress. An acknowledgment prepares the transition. The local hardware commit (`REVOKE_PROPOSAL_TOKEN()`) strips the autonomous policy of its proposal token on the committed channels and grants the enforcer sole ownership of the blend. The enforcer admits live manual proposals only after the interpolation profile completes under an active manual lease. An unconfirmed transition therefore leaves only the console uncertain, as the `Confirm` phase states. A delayed handover request or a dropped heartbeat, in contrast, runs down its local timer and invokes the validated fallback without waiting for network consensus.

Assigning a single writer resolves authority unambiguously at each control tick, but an instantaneous switch can still produce hazardous dynamic transients. If the autonomous policy's last admitted joint torque diverges significantly from the operator's initial target, directly swapping setpoints across a single tick boundary requests an unachievable step change in torque, risking mechanical shock or actuator saturation. The enforcer avoids this discontinuity by using the latched

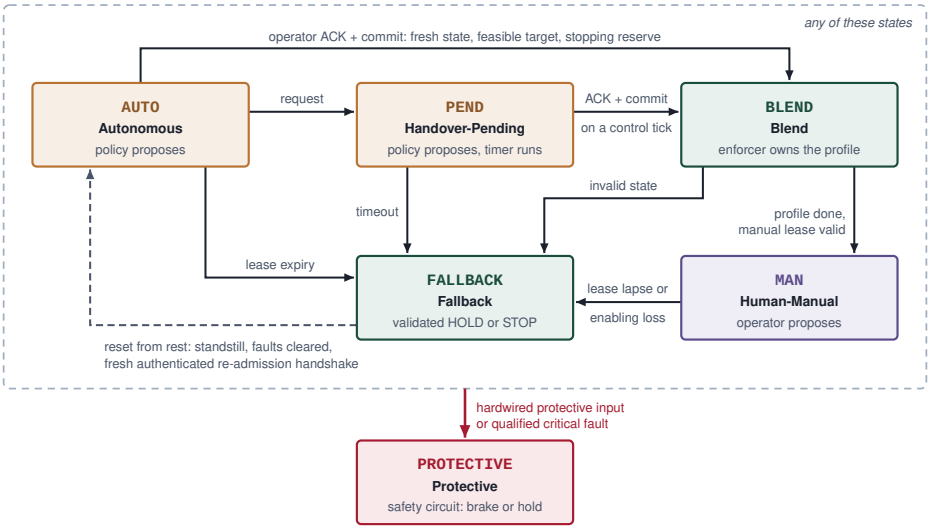


Figure 14.5: Authority states and their guards: Each box is one value of the per-channel authority state, with its code name above its plain name. A handover runs from request through acknowledgment to a commit on a control tick; from then on the enforcer owns the blend between latched endpoints, and manual proposals begin only when the profile completes with a live manual lease. Timeout, lease lapse, or an invalid state leads to the validated fallback, whose only exit is a reset from rest with a fresh re-admission. The hardwired protective input preempts every state.

boundary conditions to generate a dynamically feasible interpolation profile that preserves stopping margins throughout the transition.

Checkpoint 14.2: Authority transitions and timeout bounds

Before moving from who owns each channel to how its command changes value, verify your understanding of leases, transition state machines, and timeout dynamics:

- ☐ **State-dependent fallback**: Can you explain why the moving base and the loaded arm need different validated responses when a lease lapses, and why Safe Torque Off alone provides neither?
- ☐ **Finite-state transition semantics**: Can you trace the deterministic progression among the Autonomous, Handover-Pending, Blend, Human-Manual, and Fallback states, and name the guard and the single actuator owner on each edge?
- ☐ **Timeout bound**: At the aisle speed the configured protocol ceiling is 500 ms and the clearance-derived deadline after charging the dispatch-to-admission allocation is 37.5 ms. Can you state which one sets T_{timeout}, by how many milliseconds it is the tighter, and how many whole 1 ms permission ticks the arbiter may wait before it dispatches fallback?
- ☐ **Asymmetric preemption hierarchy**: Can you explain why the qualified local protection path must meet its measured deadline, while restoring autonomous authority requires a fresh admission handshake?
- ☐ **Fallback on a lapsed handover**: Can you explain why a lapsed or unacknowledged handover goes to the validated stop rung rather than back to the degraded policy that requested it?

If any transition paths permit unhandled timeouts or undefined intermediate states, the authority protocol is vulnerable to mode confusion and runaway actuation. Review this section before proceeding to smooth command blending.

14.4 Transfer Without Discontinuity

At commit time t_0, the enforcer latches the last admitted policy command $\mathbf{u}_0$ and a checked human target $\mathbf{u}_1$. Switching them in one permission tick may exceed the channel's allowed command rate. In classical process control, bumpless transfer between manual and automatic modes is achieved by making the inactive controller track the active one's output, so its internal state, including its integrator, already agrees at the switch (Åström and Hägglund 2006). Section 14.4.3 applies that tracking to the console-side controller that turns operator input into a command. Here **bumpless transfer** also names a second mechanism layered on top, which spreads the remaining command change across a finite window while matching the required endpoint derivatives. The quintic seam bridges of section 11.4.4 provide the construction; takeover adds a new admission decision because waiting for the operator has already consumed some clearance.

14.4.1 Command steps and mechanical drivetrain shock

A step change in torque or acceleration excites torsional resonances and snaps gearboxes across their backlash, while a slower blend lowers the command rate at the cost of stopping reserve. Because the $L_{\text{bus}}\,dI/dt$ coupling to the DC supply also turns a current step into a voltage sag on the shared rail, the command transition must satisfy the drive's rate limits while leaving enough clearance for fallback. A smooth profile therefore needs both a rate bound and a check against the remaining physical envelope.

↩ The $L_{\text{bus}}\,dI/dt$ coupling through which a current step sags the shared DC bus is established in section 2.6.1.

Definition 14.2: C² bumpless authority transfer ramp

C^2 ***bumpless authority transfer ramp*** is the enforcer-owned command profile between two checked, latched references $\mathbf{u}_0$ and $\mathbf{u}_1$. It is parameterized by a monotonic weighting function $\alpha(t) \in [0,1]$ with vanishing first and second boundary derivatives ($\dot{\alpha} = \ddot{\alpha} = 0$) over a finite blending window τ_{blend}. The profile keeps that command and its first two derivatives continuous across the fixed-endpoint handoff. When the command is acceleration, its first derivative is commanded jerk; when it is torque, the first derivative is torque rate.

1. **Significance**: A setpoint step has an unbounded ideal derivative. A C^2 command profile removes that discontinuity; measured actuator tracking and plant limits still decide physical feasibility.
2. **Distinction**: Unlike a linear interpolation ramp whose derivative jumps at both endpoints, a C^2 quintic spline enforces vanishing first and second derivatives ($\dot{\alpha} = \ddot{\alpha} = 0$) at entry and exit, making the interpolated command and its first two derivatives continuous for stationary endpoint derivatives. The physical meaning of those derivatives depends on the command channel.
3. **Common pitfall**: Choosing a window shorter than the rate limit allows for a fixed acceleration-command change (the bound equation 14.7 derives below) breaches the stated command-jerk limit even though the profile is smooth. Resonance, slip, and stopping distance require plant tests.

14.4.2 Sizing the blend window and the stopping-distance cost

Smoothing an acceleration command from cruising speed to emergency braking delays the onset of maximum deceleration, so bumpless transfer consumes clearance. It trades stopping reserve for lower mechanical stress, and sizing the blend window means balancing the actuator's jerk limit against the clearance that remains.

The enforcer blends with the quintic smoothstep. For normalized progress $\hat{t} = (t - t_{\text{start}})/\tau_{\text{blend}} \in [0,1]$, the polynomial weighting function is:

$$\alpha(\hat{t}) = 10\hat{t}^3 - 15\hat{t}^4 + 6\hat{t}^5. \tag{14.6}$$

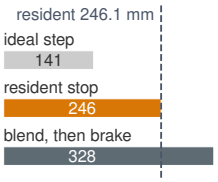
The polynomial in equation 14.6 is the fixed-endpoint result derived in section B.5. Its derivative $\alpha'(\hat{t}) = 30\hat{t}^2(1-\hat{t})^2$ peaks at $\hat{t} = 1/2$ with value $30/16 = 1.875$, so the profile's peak slope is $1.875/\tau_{\text{blend}}$ and an acceleration change must satisfy:

$$\tau_{\text{blend}} \geq \frac{1.875\|\Delta\mathbf{u}\|}{j_{\max}}. \tag{14.7}$$

For a torque command, replace the acceleration difference and jerk ceiling with the torque difference and torque-rate ceiling. The illustrative 15 N·m joint-torque change under a 750 N·m/s ceiling therefore requires 37.5 ms, rounded up to 38 ms on the permission clock. These bounds assume stationary endpoint derivatives; a transfer from a moving ramp must match its boundary state explicitly.[6]

Increasing τ_{blend} reduces the profile's peak commanded jerk, but delays the full human target and can lengthen the stopping trajectory. Consider the remote assistant's takeover of the mobile manipulator's base, committed while the machine cruises at the $v = 0.75$ m/s it slowed to before requesting it (section 14.2.4). Suppose the assistant commands a stop at the scenario's braking deceleration, a fixed step of $|\Delta a| = 2$ m/s² from the latched cruise command. The enforcer holds the blend to $j_{\max} = 7.11$ m/s³, the jerk bound of the resident jerk-limited stop. That bound is a stipulated property of the loaded base, not of its speed, so the blend window does not depend on the speed at which the takeover commits. Measured load response still requires qualification. The smoothstep bound then sets a minimum blending window of 527.34 ms, longer than the 281.25 ms the resident stop spends ramping its deceleration at the same jerk bound, because the smoothstep reaches its peak slope only at mid-window. The smoothstep weight averages one half over the window, so the base would leave the blend at 0.22 m/s after traveling 316.1 mm. Integrating the smoothstep weight a first time subtracts one half of $|\Delta a|\,\tau_{\text{blend}}$ from the entry speed v, whereas integrating that speed profile across the window subtracts only one seventh of $|\Delta a|\,\tau_{\text{blend}}^2$ from the unbraked travel $v\,\tau_{\text{blend}}$ because most of the deceleration occurs in the second half of the ramp. Braking from there at the full target would take another 12.4 mm, for 328.4 mm to zero speed in this partial model. It holds full deceleration until rest, then switches acceleration to zero instantaneously. The value is therefore a lower-bound comparison for the ramp followed by ideal braking, not a complete smooth stopping trajectory. A feasible terminal transition needs its own distance and boundary-state check.

An instantaneous step takeover would stop in 140.6 mm, so even this partial ramp-and-brake model adds 187.8 mm of travel. The partial model is also 82.4 mm longer than the 246.1 mm resident stop from the same speed. For the one-dimensional model starting at zero acceleration and ending at rest with zero acceleration, the resident profile is a minimum-distance stop under the stated deceleration and jerk bounds (section 11.5.2), so a different complete transition into braking must be checked against that baseline. The partial comparison above lacks the required terminal transition and is not itself a competing complete profile. For a braking command the resident stop is already the bumpless transfer. The base's blend window therefore governs base command changes that do not stop the machine, such as the first manual command after an automatic stop, where the blend starts from rest and the enforcer still checks the resident stop from every blended state. An idealized step stop is mathematically shorter. Executing an unblended emergency stop, however, risks wheel lockup, dynamic rollover, or load shedding (such as throwing a loaded tote off the manipulator rack), which can increase the actual physical stopping distance beyond the ideal theoretical rigid-body stop. Command smoothing limits the requested rate; physical stability and sufficient clearance still require qualification. The enforcer admits each blended tick only while the resident stop from the resulting state still fits. Where no blend fits, the base receives the resident stop itself, the suffix the fallback ladder's stop rung executes. Bypassing the blend window by applying the acceleration step within a single 1 ms permission tick would instead request a command jerk of about 2,000 m/s³ over that tick. The idealized step is only a comparison baseline.



Under the stated stopping model, the blend adds travel beyond the resident stop.

14.4.3 Initial condition matching and integrator pre-biasing

Command blending removes the chosen output discontinuity, but it cannot remove one that a controller upstream of the blend produces. On the remote assistant's side that controller is the console-side tracking loop, the software that turns joystick deflection, or a measured guiding force (the force-guided input that section 14.4.4 describes next), into a velocity target for the channel.

[6] **Nonzero Boundary Derivatives**: The $1.875/\tau_{\text{blend}}$ peak derivative factor assumes stationary endpoint derivatives. A general quintic can match the command value, first derivative, and second derivative at both endpoints, including nonzero values (section 11.4.4). Its interior rate extrema must then be checked; the fixed-endpoint smoothstep factor does not automatically apply.

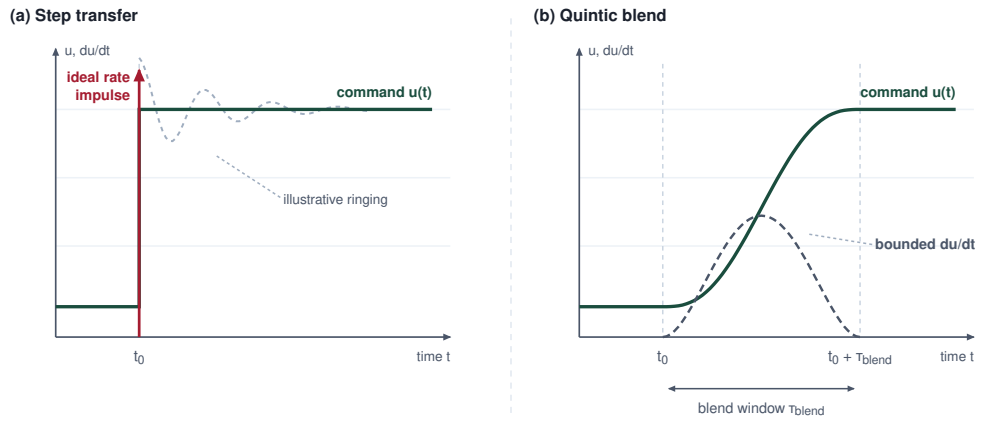


Figure 14.6: Step vs. quintic command transfer: A step and fixed-endpoint quintic blend are compared. The solid line is the command. The step's ideal derivative is a red impulse, the quintic's bounded derivative is dashed slate, and the gray ringing is schematic. The profile bounds the commanded rate for the assumed endpoints.

While the policy still owns the channel, that loop runs against a plant it does not drive, and its integrator accumulates the difference between its target and a motion that never follows. At commit it would emit that accumulated correction as a jump in the manual target, a step the blend window was never sized to absorb. **Initial condition matching** pre-biases that loop against the measured state before commit, so its internal controller state matches the output required at transfer rather than releasing accumulated error. It does not force the deliberate human target to equal the last policy command. The enforcer latches both endpoints, sizes the blend for their actual difference, and checks the resulting command and stopping reserve throughout transfer.[7]

To verify a transfer, record the command profile, motor current, encoder motion, and chassis inertial response on a shared timebase with bounded alignment error. In the ideal model, a step command has an infinite derivative at the switch, which excites high jerk, unmodeled compliance, and drivetrain ringing in physical hardware. A fixed-endpoint quintic polynomial removes the command discontinuity by zeroing the boundary derivatives, so its command rate stays bounded across the whole window τ_{blend} instead of concentrating in one impulse (figure 14.6). The measured plant response, not the command profile, establishes whether the handover met its physical limits.

The two panels distinguish the command's mathematical continuity from the plant's measured response. Removing the step bounds the requested rate under the stated endpoint assumptions; it does not establish that the drive tracks the command or that the load remains stable. Those measurements decide whether the selected blend window is acceptable.

The enforcer's 1 kHz loop gathers the four-phase handshake, integrator pre-biasing, and the C^2 quintic blend into one protocol on the safety microcontroller. Each tick it samples plant state, stopping reserve, protective inputs, and the active execution lease. It commits an ownership change only on the qualified commit tick that follows a verified acknowledgment, blends only between latched endpoints and checks every blended candidate before dispatch, and falls back on a lapsed lease or a failed clearance check.

14.4.4 Mapping operator input to arm motion

Once the operator holds the arm channel, commanding a rigid position target into a bin wall or rack upright builds contact force until the enforcer trips, whereas guiding the arm through measured contact force lets the wrist comply. Position streaming supplies kinematic targets directly; force guiding uses measured contact force to generate a motion target through an admittance controller[8] (Hogan 1985). Either mapping remains subject to the arm's qualified limits. The choice also changes the logged learning target, so the record must retain whether a sample is a requested pose, a force measurement, or an admitted command.

7 **Mid-Blend Re-Intervention Abort**: If an operator commands a secondary protective stop while a bumpless transfer blend is actively underway ($0 < \hat{t} < 1$), the enforcer does not wait for τ_{blend} to complete. It uses the measured plant position, velocity, and acceleration, together with the active command and their uncertainty, to select a qualified state-matched fallback. A new polynomial seeded only from the intermediate command does not establish that the physical plant can execute it.

↪ **Protocol:** The enforcer's cadence, memory and write invariants, and per-state execution steps are given in section C.15.7.

8 **Admittance and Impedance Control**: Hogan (1985) framed interaction control as programming the relation between force and motion at the contact. An impedance controller answers imposed motion with a commanded force; an admittance controller is its dual, turning a measured force into commanded motion, which suits stiff, highly geared arms whose position loops are accurate but which cannot render force directly. On such an arm a force-guided target is still a motion command, so it passes the same enforcer checks as a streamed pose.

14.4.5 Bilateral teleoperation and haptic feedback passivity

A force-reflecting console adds an actuated device at the operator's hands. Delayed feedback[9] can destabilize that coupled loop even when authority ownership is correct (Anderson and Spong 1989). Such a console therefore needs a separate interaction-energy argument, including its port conventions, controller sampling, delay and loss assumptions, and actuator saturation. The energy-storage condition and wave-variable construction are developed in section B.10; they do not replace the clearance or force limits. The basic warehouse takeover does not require bilateral force reflection. If a site adds it, pre-biasing and ramping the console's force output must prevent a command step at the operator's hand within its own measured limits, because the arm's accepted blend duration does not automatically qualify the console. Whichever input mapping and console a site chooses, the session that carries them must still fit the clearance left after the operator's response.

[9] **Delayed Teleoperation**: Ferrell (1965) found that operators working through a transmission delay fall back on a move-and-wait strategy, so task time grows with the delay. Force reflection through the same delay proved unstable until Anderson and Spong (1989) used scattering theory to make the communication channel passive, a construction that Niemeyer and Slotine (1991) recast as wave variables. For a force-reflecting console the delay budget is therefore a stability term as well as a latency term.

14.4.6 Admitting the remote-assistant session

Every interval priced so far in this chapter now draws from the same finite stopping reserve when a remote session begins. Admitting the remote assistant requires bounding four timing parameters against the machine's limits: the operator reaction budget (the takeover latency T_{Σ} of equation 14.1 for the takeover mode in use), the **handshake protocol latency** $T_{\text{handshake}}$, the manual lease $T_{\text{lease,man}}$, and the minimum blending window τ_{blend}. Table 14.6 states them for the machine's two command channels, the base and the arm, which one remote-assistant session commands together.

↪ How many machines one remote assistant can serve before requests wait minutes for a person is developed in section 17.3.2.

Table 14.6: Blending windows on the mobile manipulator: One remote-assistant session commands both channels, and each window is rounded up to whole 1 ms permission ticks. The rate ceiling r bounds the derivative of each *command*. The base window is the same at every speed, because the jerk bound belongs to the loaded base. At the in-loop request speed a braking blend does not fit the clearance that remains, so the base receives the resident stop and its window applies only to non-braking changes. Every lease and pending deadline must fit the state-dependent stopping reserve.

Machine channel	Command change and chosen rate ceiling	Smoothstep window $\tau_{\text{blend}} \geq 1.875\lvert\Delta u\rvert/r$, whole ticks	Human / protocol time	Fallback contract
Base acceleration command	Zero to 2 m/s²; $r = 7.11$ m/s³, the resident stop's jerk bound	528 ms	605 ms / 31 ms	100 ms manual lease; 500 ms pending ceiling shortened by clearance
Arm joint torque command	15 N·m; torque rate $r = 750$ N·m/s	38 ms	605 ms / 31 ms	The same manual lease; its loss invokes the arm's validated stop

Consider an illustrative remote-assistant in-loop takeover, requested after the mobile manipulator has slowed to 0.75 m/s with the assistant's `manual_drive` entry. The four parameters take the values already derived. The reaction budget T_{Σ} is the 605 ms in-loop takeover of section 14.2.2, which already includes transport and the 31 ms four-phase exchange of table 14.5, so the handshake is part of T_{Σ}, not an addition to it. The manual lease, the longest silence tolerated from the assistant before the permission path falls back, is 100 ms, and table 14.6 gives the blending window for each channel.

An in-loop session at 0.75 m/s closes its complete budget by only a few centimeters, and only because the base takes its resident stop rather than a braking blend. A session that begins at the 0.15 m/s crawl or from standstill has far more room, so those are the configurations worth qualifying. Table 14.7 charges the in-loop session's intervals one at a time against the clearance the finished budget leaves at that speed.

Table 14.7: Clearance budget for an in-loop remote-assistant session: Each row charges one interval at the 0.75 m/s request speed, in the capture-origin design budget, and the last column carries what remains. The first four rows close the complete budget by a few centimeters; the blend row would overrun it. These are screening values for an allocated, unmeasured Δ_{da}, not a qualified admission result.

Interval charged at the in-loop request speed	Travel	Clearance left
None (the finished budget's spare)		563.7 mm

Interval charged at the in-loop request speed	Travel	Clearance left
In-loop takeover T_{Σ}, handshake included	453.8 mm	110.0 mm
Dispatch-to-admission allocation Δ_{da}	47.5 mm	62.5 mm
Manual lease in place of the chunk lease in τ_{delay}	30 mm	32.5 mm
Base braking blend (reserve it would use)	up to 157.0 mm	Does not fit; the base takes its resident stop

Three rows of the table decide that verdict. The remainder after the takeover, the 146.6 ms by which the takeover beats $T_{\mathrm{deadline,fallback}}$ (section 14.2.4), bounds the transfer itself. Any further delay before a stop could begin while authority is changing hands must fit inside it, and the enforcer checks the resident stop against it at every tick of the transfer. The lease row exists because, once the assistant holds `MAN`, the manual lease replaces the 60 ms chunk lease inside τ_{delay}, so its extra length becomes travel. The row is the upper bound on the lease set out in section 14.3.4, and the chosen value passes it. The blend row fails. Blending the step to braking from 0.75 m/s grows the travel plus the resident stop from the blended state by up to 157.0 mm, well past the remainder, so in this session the base receives the resident stop and only the arm blends. This conservative per-tick check charges a fresh resident stop starting from zero deceleration at each blended speed rather than the partial model's 82.4 mm constant-deceleration tail, because any fallback triggered during the blend begins from the standard resident profile. The slow-down was derived with Δ_{da} charged, so the complete budget closes, but by centimeters, another reason the base should complete its stop before accepting continued manual motion.

At the crawl, after a 4.5 s out-of-loop takeover, 198.4 mm remains before the same charges, and the base's stop from the crawl is its resident stop, with no braking blend to reserve. The admission allocation and the lease extension cost travel in proportion to speed, so at the crawl they take only a small part of that remainder. Once the assistant holds `MAN`, the live check of table 14.3 uses the current remaining clearance, the measured speed, and only the unelapsed portions of those intervals; commit does not restore the distance spent waiting or blending. These figures screen the candidates. Qualification must still check the rounded executable profile and the state-matched fallback, including delay travel, over the full transition, against a measured Δ_{da}.

Every command in that session, whether a streamed pose, a force-guided target, or a heartbeat that renews the manual lease, must also come from the authenticated operator that the `manual_drive` entry names. A budget that closes by a few centimeters leaves no room for a forged or replayed packet, which makes identity and packet admission the next constraint.

14.5 Authenticating an Intervention

On a shared wireless link, a legitimate `veto_path` intervention packet and an unauthorized drive-torque packet can arrive within the very same control cycle. The permission architecture must prevent an adversary or malformed network frame from seizing kinetic authority, yet it must never obstruct an authorized operator from arresting the machine before a collision. Systems engineers reconcile the two demands by classifying intervention operations according to the mechanical work they produce. Stopping the machine by removing power or engaging a mechanical holding brake is not symmetrical with injecting active motion commands, so the two actions take different verification paths. A protective stop brings the machine to a controlled halt along a braking or holding rung of the fallback ladder rather than abruptly cutting actuator torque. Commanding base velocities, wheel torques, or joint trajectories, by contrast, injects fresh mechanical energy into the plant: a forged or corrupted trajectory command can drive a moving mass into an obstacle, whereas an authenticated out-of-band protective input only requests a plant-specific minimum-risk stop.

Authentication adds delay to the remote intervention path, so it is priced against the same deadline as every other latency source, the latest fallback deadline $T_{\mathrm{deadline,fallback}}$ of equation 14.4. At the warehouse aisle velocity of 1.3 m/s, the partial spare permits at most 100.8 ms of added delay before charging the admission latency Δ_{da}, which drops to 37.5 ms once the placement allocation is charged. Transferring published Cortex-M4 cycle counts to the reference microcontroller clock gives an illustrative estimate of 1.8 ms for one Ed25519 signature verification[10] (Owens et al. 2024),

[10] **Ed25519**: The Edwards-curve signature scheme of Bernstein et al. (2012) provides rapid verification alongside compact 32-byte public keys and 64-byte signatures. Constant-time implementations eliminate secret-dependent branches and memory lookups to protect against side-channel timing attacks. While that defense primarily shields the signing teleoperation console, the receiving safety microcontroller only verifies incoming frames. Well-formed invalid signatures can still require the expensive verification path, even when malformed inputs admit early rejection. The verifier's processing load therefore depends on admitted traffic and its verification paths, the vulnerability that the verifier-flooding analysis of section 14.9 prices.

which translates into 2.34 mm of physical travel and leaves 128.7 mm of spare displacement margin. The same cycle-count and clock-rate estimate for an AES-128 CMAC over one command frame is 5.0 µs (Dworkin 2005; Schwabe and Stoffelen 2017). Per packet, both estimates fit within the deadline; loaded measurements on the target must qualify them. The design problem is therefore not choosing an algorithm but meeting five requirements on the admission path that carries the packets:

1. **Protective stops never wait for verification.** The local hardware fallback timer and independently wired emergency inputs act unconditionally, regardless of whether cryptographic routines have completed or stalled. If a verification routine overruns its deadline, the permission subsystem marks remote assistance as unavailable and cedes control to the autonomous protection layer; it never postpones an emergency deceleration. As analyzed in the verifier-flooding pitfall of section 14.9, what causes a cryptographic verifier to miss real-time deadlines is rarely the algorithmic cost of an individual packet, but rather the ingress volume of hostile or unvetted network traffic saturating the execution queue.
2. **Takeover requests must be attributable to an individual.** Because a symmetric Message Authentication Code (MAC) relies on a shared key known to both endpoints, a valid tag proves frame integrity but cannot prove which endpoint originated the transmission. An asymmetric digital signature, however, cryptographically binds the takeover request to the teleoperator's private key, supporting attribution when credential enrollment, key custody, and compromise evidence also identify the operator. The architecture therefore mandates an asymmetric digital signature on the session-initiating `manual_drive` request, while subsequent streaming commands and periodic heartbeats rely on the lower-overhead symmetric MAC.
3. **Freshness represents a spatial displacement budget, not merely a clock tolerance.** In embodied systems, packet latency translates directly into unmodeled vehicle travel. A replayed or delayed motion packet arriving at the boundary of a 100 ms freshness acceptance window commands an action based on an obsolete vehicle pose. Over that interval the mobile base travels 130 mm, consuming nearly the entire safety margin and leaving only 1.0 mm of the 131.0 mm partial spare. Once the admission latency Δ_{da} is factored in, such a replay completely overruns the available spare, so the allowable freshness window must shrink below 37.5 ms. The receiver drops counters no newer than the last authenticated, accepted counter before spending processor cycles on cryptography. It advances that stored counter only after authentication and freshness checks pass, so an unauthenticated high counter cannot invalidate later legitimate traffic. Furthermore, if heartbeat packets cease for longer than the lease duration $T_{\mathrm{lease,man}}$, the controller immediately revokes manual authority and engages the certified local fallback ladder.
4. **Malformed traffic never alters control authority.** Any packet with a corrupted checksum, an out-of-order sequence counter, or an invalid authentication tag is immediately rejected and recorded according to Rule A4 of section 14.8. A corrupted packet neither modifies existing actuator channel assignments nor triggers a spurious emergency stop. Rejecting an isolated invalid packet preserves still-valid authority; interference or dropouts that prevent timely renewal still expire the lease and invoke the qualified fallback.
5. **Every actuator-write path must be accounted for.** All side channels capable of driving physical hardware, including diagnostic ports, hardware bootloaders, debug interfaces, and direct fieldbus wiring, must be gated by the hardware permission authority or guarded by dedicated interlocks. The intervention threat model explicitly accounts for unauthorized remote packets injected across wireless links, replay attacks traversing internal communication buses, and rogue commands originating from a compromised high-level application processor. Physical tampering, by contrast, enters the safety assurance case only when explicitly mandated by the operational threat model.

These authentication and transport latencies directly impact policy learning and sim-to-real transfer. An embodied simulator that provides instantaneous human takeover underprices the danger of visiting marginal states. If an autonomous policy learns that intervention occurs instantaneously, it will tolerate trajectories that are physically unrecoverable once realistic communication, cryptographic verification, and actuator fallback latencies are introduced. Training and hardware-

in-the-loop evaluation pipelines must therefore inject these bounded authentication, transport, and mechanical reaction delays.

Verifying an incoming packet authenticates its credential; the capability manifest separately determines its privileges. Cryptographic validity alone cannot guarantee dynamic stability during control transfer. When authority shifts while the chassis carries linear momentum and the manipulator actuators exert active joint torques, the permission path must commit one selected source per channel and announce that ownership to the operator. Local mutual exclusion and human mode awareness are separate requirements; a valid packet establishes neither by itself.

14.6 Authority Handover Failure Modes

When the warehouse mobile manipulator's perception policy meets an unmodeled specular reflection from wet shrink-wrap on a pallet, depth artifacts drop its confidence and the runtime asks a remote assistant to reposition the gripper. If the console grants manual joystick authority over the arm without locking the base channel or announcing that the base is still moving, an assistant focused on aligning the fingers assumes the chassis is stationary. Meanwhile the autonomous navigation thread keeps driving the wheels into a rack upright. Every individual hardware and software component in that handover operates within its specification. The collision comes from mode confusion, the failure class that Sarter and Woods identified in automated aircraft cockpits (Sarter and Woods 1995). Every operating mode added to a control loop multiplies the state a human supervisor must track. When an interface changes modes without unambiguous telemetry of which agent holds each channel, operators take actions that are right for their assumed mode and hazardous in the actual one (Sarter and Woods 1995). In embodied physical systems, the most dangerous form of this failure is **authority ambiguity**: a state in which the autonomous runtime assumes the human has taken over a physical degree of freedom, while the human assumes the automated controller still guides it.

Definition 14.3: Mode confusion

Mode confusion is a mismatch between the operational state a human supervisor believes an automated system occupies and its actual execution mode, leading the supervisor to take actions that would be correct for the presumed state but are hazardous in the real one.

1. **Significance**: When control channels transfer authority independently, an operator's mental model of one channel can easily go stale while their attention is consumed by another. For example, an operator dedicates the 605 ms in-loop response window entirely to an arm alignment, under the false belief that the base is parked. The base, running under the autonomous policy at 1.3 m/s aisle speed, then travels 786.5 mm, which is 16.1× the 48.7 mm margin the complete budget leaves for $d_{\text{stop}} \leq D_{\text{clear}}$.
2. **Distinction**: Conflicting command streams between concurrent software modules represent an arbitration fault, which single-owner bus gating resolves by construction. Mode confusion, by contrast, resides within the supervisor's mental model of the machine. Resolving it requires committed channel masks and explicit per-channel status annunciation at the console, because deterministic ownership on the CAN bus or Ethernet backplane cannot correct a human's false assumptions.
3. **Common pitfall**: Telemetry confirming that an operator's gaze is directed at a primary task monitor does not guarantee the absence of mode confusion. Tracking an operator's visual focus onto an end-effector camera stream reveals nothing about whether that operator realizes the autonomous navigation stack still holds kinetic authority over the drive chassis.

On a machine with several axes, control authority splits naturally across distinct groups of joints, such as the mobile base's planar velocity $(\dot{x}, \dot{y}, \dot{\theta}_{\text{head}})$ and the manipulator arm's joint positions. In the wet-shrink-wrap collision, those channels transferred independently. Telemetry logs reveal the fault as disjointed control flags toggling asynchronously across execution threads. At the human-machine interface, however, the split state stays invisible until physical clearance is exhausted, because neither

the policy nor the human receives an immediate, unambiguous indicator of the other's partial withdrawal.

To prevent this desynchronization, handover protocols must enforce explicit, per-channel bitmasks across every phase of negotiation. Every `Request`, `Acknowledge`, and `Commit` message must declare the exact subset of degrees of freedom being transferred. The formal transition state machine of section 14.3 moves proposal tokens exclusively for those designated channels, and the subsequent `Confirm` phase explicitly annunciates active ownership on each channel back to the operator console. A channel left out of the bitmask may keep its autonomous owner, so the base can keep navigating while the operator takes the arm. This holds only while the console annunciates that the base is still moving under the policy and the operator's acknowledgment covered that state. Without that annunciation, the commit places the untransferred channel in a certified holding state, such as an active brake hold, rather than letting autonomous execution continue silently. The wet-shrink-wrap collision above is the second rule's case, an untransferred base left running without annunciation.

Explicit channel ownership protocols solve arbitration on the digital bus, but they cannot restore the situational awareness of an out-of-loop human. For this reason, the in-loop and out-of-loop takeovers of section 14.2.2 must be treated as distinct operational states with different timing budgets, the 605 ms and 4.5 s that set different request speeds. While gaze tracking can verify where an operator directs their eyes, it cannot verify cognitive comprehension or justify truncating the required takeover latency budget. Local stopping envelopes must remain continuously armed and active, even when the console reports an attentive supervisor. Two field incidents show what happens when either half of this two-layer protection is missing.

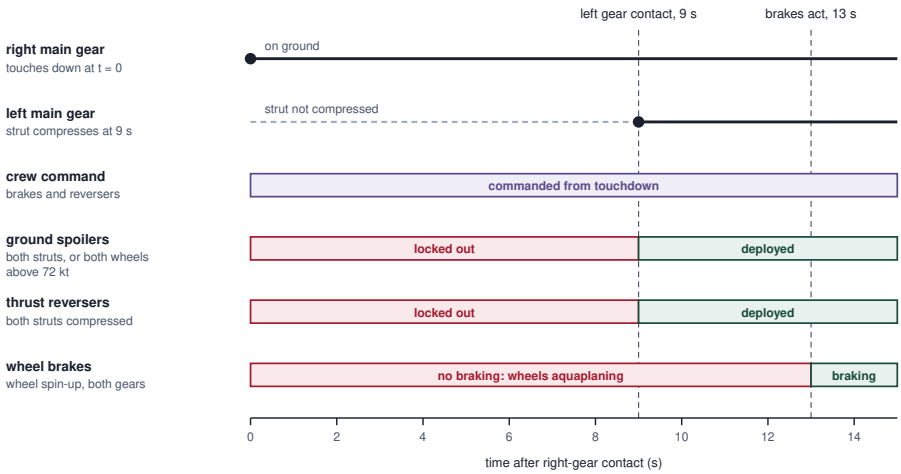
In the Tempe, Arizona collision, the National Transportation Safety Board (NTSB) determined that the safety driver was visually distracted by a personal smartphone, identifying operator complacency and inadequate monitoring policies as primary contributing factors (National Transportation Safety Board 2019). The vehicle architecture had disabled automatic emergency braking during testing and relied entirely on the human supervisor to perceive hazards and intervene. In the San Francisco Cruise incident, an autonomous vehicle executed an emergency stop after striking a pedestrian who had been thrown into its path by an adjacent human-driven car. The onboard planner, however, subsequently initiated an automated pull-over maneuver, dragging the pedestrian beneath the chassis (Quinn Emanuel Urquhart and Sullivan, LLP 2024). The Tempe disaster demonstrates why an operator-monitoring contract must always be backed by an autonomous, local stopping envelope capable of bringing the machine to a halt within the remaining sensor clearance, regardless of operator attentiveness. The Cruise incident demonstrates that unexpected physical contact, tactile sensor anomalies, or sudden external force spikes must immediately lock out all motion authority into a verified physical inspection hold, rather than allowing automated post-collision clearing routines to execute.

↩ The Tempe timeline, including the action suppression that withheld emergency braking, is established in section 1.8.

A human operator's manual command can oppose an active safety barrier without creating a multi-writer conflict on the actuator bus. When an operator commands a manipulator toward a prohibited obstacle, the downstream safety enforcer either projects the trajectory onto the safe half-space or rejects the command entirely, dispatching only the admitted control action to the motor drives. However, the supervisory console must make this active constraint immediately visible to the human. If the interface conceals why the robot has stopped responding, the operator will instinctively command higher control effort or larger joystick displacements, leading to severe command saturation and instability once the constraint clears. Furthermore, when the console incorporates bilateral teleoperation or force-feedback interfaces, system stability depends directly on the passivity-based energy bounds established in section 14.4.5; ownership tokens alone cannot prevent feedback oscillations across a physical-virtual coupled loop.

Authority ambiguity also corrupts downstream imitation learning. If the record cannot say who held each channel, interventions are mislabeled as autonomous behavior or the reverse, and a behavioral-cloning policy learns to reproduce the very errors that triggered the takeover. A low offline validation loss cannot catch this, because it only shows that the policy memorized the wrong labels. Causal intervention masking, developed in section 14.8.3, depends on ownership bits that the authority record stamps at commit time.

War Story 14.1: Ground-contact interlocks on flight 2904



Context: On Lufthansa Flight 2904 at Warsaw, the flight crew executed an approach based on outdated wind reports, encountering an unreported tailwind that caused the aircraft to touch down at high speed far down a waterlogged runway. Upon touchdown, the crew immediately commanded maximum manual braking and deployed thrust reversers, but the aircraft's automated flight-control logic refused to grant deceleration authority (Main Commission Aircraft Accident Investigation Warsaw and German Federal Bureau of Aircraft Accident Investigation 1994).

Mechanism: Crosswinds and runway hydroplaning delayed the left gear compression: the right main gear made contact 9 s before the left. The aircraft's deceleration interlocks were rigid. Ground spoiler deployment required weight-on-wheels shock-strut compression on both main gears **or** wheel spin-up above 72 knots. Thrust reversers required weight compression on both main struts. Mechanical wheel braking was locked out until the main wheels achieved synchronized spin-up. Because hydroplaning prevented wheel spin-up and an asymmetric roll angle kept the left strut uncompressed, the control logic determined the aircraft was still airborne. Consequently, the flight computer locked out spoilers and reversers for 9 s and inhibited wheel brakes for even longer.

Impact: Wheel braking was inhibited until 13 s after initial touchdown. With braking authority locked out across the primary deceleration zone, the aircraft could not dissipate its kinetic energy along the remaining wet runway. It overran the runway threshold at 72 knots, struck an airport embankment, and caught fire, resulting in two fatalities and total hull loss.

Response: Official accident investigators cited the ground-contact interlock logic as a critical contributing cause alongside crew decision-making and delayed meteorological updates. The crew could not override the spoiler and reverser lockouts. The report identifies those design constraints alongside crew actions and incomplete landing-distance guidance; it does not establish that a manual override alone would have prevented the overrun.

Systems lesson: Embodied robots share this exact failure topology whenever automated sensor preconditions, such as docking latch checks, wheel-slip traction observers, or ground-contact switches, gate emergency braking or reverse-torque commands issued by human supervisors. Unmodeled wheel slippage or uneven terrain can violate a sensor predicate precisely when deceleration is urgently required. Any control architecture that gates physical actuation behind automated preconditions must satisfy three rules. First, it must immediately annunciate unsatisfied interlocks across the operator console. Second, it must explicitly identify the unfulfilled sensor predicate. Third, it must provide a deterministic fallback mode for sensor disagreement. Rather than bypassing safety interlocks blindly, the supervisory architecture

must define a certified fallback deceleration path that executes based on conservative, physically verifiable stops.

The formal authority arbiter detailed in section 14.3 exists to prevent these failures. Safe embodied control architectures cannot rely on implicit authority handoffs, assumed human vigilance, or ad hoc torque summing on the actuator bus. Evaluating post-incident behavior requires a tamper-evident, channel-by-channel telemetry record captured at every control cycle, with explicit gaps and bounded event timing. The surviving record can help distinguish a safety boundary breach, an invalid token exchange, and electrical noise on a safety bus only to the extent that its independent measurements, capture coverage, and timing bounds separate those events. Its storage path must therefore survive unexpected power interruptions, kernel panics, and real-time scheduling stalls.

14.7 Trustworthy Operational Logs

Consider a robotic arm striking a warehouse racking upright during a supervisory takeover. If the operational log contains an unrecorded gap, investigators cannot determine whether the policy proposed the damaging trajectory, the enforcer admitted it, or the operator received authority too late to intervene. Rigorous incident reconstruction requires two foundational properties: declared loss handling and an operational record engineered to survive physical failure.

A trustworthy record must capture state on both sides of the causal boundary, paired with independent plant observations.[11] Motor phase currents, optical encoder positions, and wrist force-torque measurements provide physical observations distinct from software setpoints, subject to their calibration, bandwidth, and uncertainty bounds. Dispatched commands alone cannot prove delivered actuator torque or establish mechanical collision causality.

At every cycle, the 1 kHz deterministic control loop records a structured authority and action tuple under declared loss handling. The minimal **evidence tuple** takes the form:

$$\langle t_{\text{MCU}}, S_{\text{auth}}, \mathbf{x}, \mathbf{u}_{\text{nom}}, \mathbf{u}^*, \tau, \text{ID}_{\text{operator}}, T_{\text{lease,rem}} \rangle$$

This tuple records both sides of the proposal-permission boundary. Its command fields are the action taps of section 5.5.1 taken at intervention time: $\mathbf{u}_{\text{nom}}$ is the requested or mapped command (a_{req} or a_{cmd}, whichever the source emits), $\mathbf{u}^*$ is the enforced command a_{enf}, and the measured response a_{meas} lives in $\mathbf{x}$. The fields are:

- t_{MCU} is the monotonic event timestamp recorded in nanoseconds by the safety microcontroller hardware clock, mapped to other subsystem clocks through a qualified clock-conversion method with an explicit error bound. Expressing values in nanosecond units does not imply nanosecond-level inter-device synchronization accuracy.
- S_{auth} records the active discrete authority state (`AUTO`, `PEND`, `BLEND`, `MAN`, `FALLBACK`, or `PROTECTIVE`).
- $\mathbf{x}$ captures the instantaneous measured plant state, including joint angles, velocities, motor phase currents, and IMU linear accelerations.
- $\mathbf{u}_{\text{nom}}$ is the proposed plant control setpoint requested by the cognitive policy or human operator console.
- $\mathbf{u}^*$ is the control command admitted by the enforcer, including a feasible CBF-filtered proposal or the selected qualified fallback. Associated event metadata must distinguish admission, modification, and rejection; section 12.7.1 specifies the projection path.
- τ is the actuator torque command dispatched across the real-time fieldbus to the motor inverters. It is still a command; delivered motor torque can be estimated from the measured phase currents in $\mathbf{x}$ using a qualified motor model and explicit uncertainty bounds.
- $\text{ID}_{\text{operator}}$ identifies the authenticated cryptographic entity holding active proposal rights.
- $T_{\text{lease,rem}}$ records the remaining time on the active execution lease (T_{lease} or $T_{\text{lease,man}}$, minus the elapsed duration $t_{\text{MCU}} - t_{\text{refresh}}$).

Omitting any of these fields breaks post-incident reconstruction. Without the proposal $\mathbf{u}_{\text{nom}}$, a dispatched torque τ cannot show whether the policy or operator asked for an erratic action. Without the enforcer output $\mathbf{u}^*$, the log cannot show whether the safety barrier modified that

11 **Event Data Recorder Baseline**: A passenger-vehicle event data recorder captures 5 s of pre-crash data at 2 Hz (phasing in from September 2028 to 20 s at 10 Hz), which even under the modernized rule yields only one sample per 100 ms across 100 permission ticks (National Highway Traffic Safety Administration 2024a). Similarly, the data storage system mandated for automated lane-keeping systems under UN Regulation No. 157 logs system activations, overrides, and transition demands to establish driver attribution, but its timestamps require accuracy only within 1 s, or 1,000 permission ticks (United Nations Economic Commission for Europe 2021). Because neither regulatory standard provides the time resolution needed to order an intervention against physical contact within a single control cycle, the autonomous machine must maintain its own deterministic, per-tick operational record.

↩ A record of what was proposed, evaluated, dispatched, and measured is the third demand on the permission path established in section 1.9.

request. Without $T_{\text{lease,rem}}$, it cannot show whether authority had already lapsed into a fallback mode when the command was evaluated. Chained with cryptographic integrity proofs and clock-synchronization metadata, these tuples form the per-tick half of the authority record that section 14.8 specifies.

Three architectural requirements ensure that logged tuples serve as reliable forensic evidence. First, timestamps must date physical events rather than software handling. A software interrupt handler that samples the clock only after being scheduled records its own execution delay rather than the event time. Under variable scheduling jitter and interrupt latency, this delay can easily invert the apparent chronological order of an intervention command and a mechanical contact. Protective inputs and fieldbus ingress packets must therefore be timestamped directly at the hardware boundary through timer edge capture and network controller hardware counters. When clock-conversion error, sensor exposure windows, and sampling intervals overlap between two events, the log must explicitly record their relative order as indistinguishable rather than asserting an unverified sequence.

Second, the storage pipeline must be strictly tiered by guarantee level. Minimal evidence tuples stream to a bounded ring buffer in non-volatile memory on the safety microcontroller. The serialized tuple size multiplied by the control tick rate sets a continuous write bandwidth that hardware qualification tests must prove the device sustains under all operating regimes, with buffer overflows explicitly flagged rather than silently dropped. High-bandwidth perception streams, including camera frames, point clouds, and policy internal activations, flow to a best-effort buffer on the primary host. This high-volume traffic is never permitted to contend for the safety microcontroller's memory or bus bandwidth.

↩ The multi-camera ingestion rate that this best-effort buffer must absorb is established in section 5.6.2.

Third, loss must be declared explicitly. When buffer exhaustion or bus stalls occur, the logger must record a missing entry or overwritten frame as a declared gap in the evidence, never as an implied safe pass. A declared gap alerts both post-incident auditors and downstream training pipelines that the trajectory continuity was broken, whereas a silent drop makes two separated ticks appear contiguous in time.

Upon receiving an intervention trigger, such as an operator takeover command, an emergency stop assertion, or an enforcer barrier violation, the logger freezes a configured **pre-trigger window** ($T_{\text{pre}} = 5$ s) and transitions to recording a dedicated post-trigger segment. The pre-trigger duration matches the 4.5 s design out-of-loop takeover latency characterized in section 14.2.2, rounded up to whole seconds. This window preserves the design takeover interval, but policy divergence may begin before the operator notices it, so its duration alone does not establish that the complete causal chain remains in the log. Operational sites that measure longer takeover tails must lengthen T_{pre} accordingly. Overwritten frames or dropped samples within this interval remain declared gaps in the evidence. Section 5.8 uses this same window as its lookback horizon, searching it for the divergence point t_{div} and detailing why learner-visited states during drift are essential for offline policy training. The authority record provides the cryptographic ownership stamps that allow the training pipeline to distinguish unprivileged policy proposals from enforcer interventions and operator overrides.

Finally, the operational record must reveal any subsequent tampering or truncation. Each entry carries a keyed cryptographic link over its predecessor,[12] while periodic authenticated anchors are written to replay-protected storage located outside the logger's primary write path. Rewritable non-volatile storage with authenticated writes satisfies this requirement, avoiding the need for dedicated write-once media. Under this design, investigators can determine exactly which log prefix each anchor authenticates, verify that cryptographic keys and anchors remained in trusted custody, and flag any unanchored tail or missing block as a declared gap. Hardware-level tamper resistance and loss declaration protect record integrity and expose missing evidence; they do not establish that a sensor reading is accurate. To serve post-incident reconstruction and downstream policy training, however, these raw evidence streams must be bound into a formal, machine-checkable artifact: the authority record.

[12] **Linked Hash Chains:** Haber and Stornetta (1991) proposed linking each timestamp to the cryptographic hash of its predecessor, ensuring that no historical record can be backdated or inserted without invalidating every subsequent link. The authority record inherits both this tamper-evident property and its limit, since an adversary possessing the signing keys and complete control over the storage device can recompute the entire chain. To protect against retroactive tampering, commitment anchors must be mirrored to an independent, replay-protected storage domain outside the logger's mutable write path.

14.8 The Authority Record

When an incident investigator asks, weeks after an abrupt blocked-aisle takeover, which entity drove the mobile base as it neared the rack end, the machine's recorded telemetry provides evidence bounded by its capture coverage, clock alignment, independent plant measurements, and trusted

custody. The **authority record** bridges real-time supervisory arbitration with downstream verification and release claims. To establish this audit trail, the authority record consumes two upstream contracts. From the enforcement record of section 12.11, it ingests the rungs of the fallback ladder so that every logged fallback explicitly identifies the rung executed by the permission path. From the placement record of section 13.8, it incorporates the permission path's declared timing bounds and bus arbitration settings, which every operator command also crosses, together with the placement record's loaded-test field, which stays empty until verification fills it. The record shows who held each channel at each tick; it does not show who decided the speed, the staffing, or the operating domain that put the machine there.

↪ The record of who decided that the machine could be in that situation at all, with a named owner for each decision, is developed in section 17.9.

To these upstream contracts, the authority record adds two deliverables of its own. The static authority manifest translates the capability matrix of section 14.2 into a machine-checkable contract. It binds every actuator channel to its authorized actors, operational preconditions, preemption tiers, and transition deadlines, and the safety microcontroller verifies and seals this manifest before the actuator drivers energize. Complementing the manifest, the record's per-tick log forms an independent stream that records, for every control cycle, which entity held kinetic authority alongside the commands proposed, admitted, and dispatched, paired with independently measured physical response. In this stream, a missing cycle registers as an explicit communication or capture gap rather than an implied pass. The static manifest establishes the legal rules of interaction, while the per-tick log supports checking recorded runtime adherence under the capture, synchronization, custody, and declared-gap assumptions of section 14.7.

14.8.1 The static authority manifest and capability binding

If an operator's permissions or a channel's rate limits can be edited in software while the machine runs, a post-incident log cannot prove which rules the arbiter was supposed to enforce at the moment of a takeover. The static authority manifest fixes those rules before the actuators energize: loading and cryptographically sealing the manifest during system initialization binds every subsequent event to the permissions and physical limits in force at boot. It specifies every permitted interaction as a verifiable contract between the physical platform and potential command sources. Each entry delineates a distinct physical authority domain using a hardware actuator mask, an authorized entity class (such as a local physical operator, a remote teleoperation console, an autonomous trajectory policy, or an emergency stop loop), and an integer **preemption tier**. Before any command vector enters the actuation pipeline, the permission path verifies the sender's cryptographic role credentials against this manifest. Over the network, an operator's command arrives as an `OPERATOR` payload under the same proposal header, carrying the operator's role, the requested channel mask, the command vector, a monotonically increasing nonce, and a keyed authentication tag.

In addition to credential validation, each manifest entry enforces four operational invariants on its assigned channel:

1. *Single-writer mutual exclusion*: Hardware access controls guarantee that exactly one entity writes to an actuator channel at any given instant.
2. *Execution lease duration ($T_{lease,man}$)*: For human operators, the lease defines the maximum tolerable silence before the enforcer revokes authority and triggers a state-dependent fallback.
3. *Command rate ceiling (r)*: A hardware slew limit caps how rapidly a requested command vector may change between successive cycles, preventing abrupt step inputs.
4. *Bumpless blending duration (τ_{blend})*: A transition window enforces a C^2 continuous authority transfer ramp that preserves the kinodynamic feasibility bounds established in section 11.4, requiring a qualified executable profile and a feasible state-matched fallback at every transition state if communication severs. The fallback depends on surviving sensing, controller, and energy premises; duration and continuity alone do not establish it.

14.8.2 Per-tick evidence tuples and the fault injection suite

A general telemetry tuple that records only a single machine-wide mode cannot show which channels were mid-blend, which fallback rung fired, or why a higher-tier source preempted a lower one. Every entry in the authority record's per-tick log therefore expands the evidence tuple of section 14.7 with further fields. A field names the specific actuator channels governed by the arbitration decision.

Another gives the ladder rung of any active `FALLBACK`. Others give the root cause of any preemption and the instantaneous blend factor $\alpha(t)$ of the enforcer-managed profile. The last is a cryptographic hash link to the preceding log block, authenticated through the platform's protected anchoring scheme. The versioned static manifest supplies the decoding context, specifying physical units, field serialization order, clock synchronization mappings with their associated error bounds, credential validation rules, and hardware access control masks.

Verification draws its test inputs from the static manifest and its evidence from the event schema. The manifest's `authority_claims` field hands section 15.6.2 four authority rules by identifier, each of which becomes a hardware-in-the-loop fault case on the machine:

↪ **Byte layout:** The operator payload's and the authority record's fields, types, and offsets are given in section C.15.6.

- **A1.** Exactly one qualified enforcer output path writes each actuator channel. Source arbitration and hardware access controls prevent a second sender from bypassing that path while the policy and remote assistant both propose commands; successful bus arbitration alone does not satisfy this claim.
- **A2.** No lower-tier source bypasses an active higher-tier source or the independent permission check. An autonomous policy proposal during `MAN` is ignored, and an assistant command that opposes the enforcer's admitted barrier limit is clamped or rejected.
- **A3.** An unacknowledged handover or stalled manual lease decays authority by T_{timeout} of section 14.2.4, initiating controlled fallback by the earlier of the configured lease ceiling and the clearance-derived deadline of equation 14.4. This clearance deadline has a partial-budget upper bound of 751.6 ms at the 0.75 m/s slow-down before an in-loop takeover, and 100.8 ms for a handover requested by the policy at the 1.3 m/s aisle speed. Both values represent additional-delay allocations in the capture-origin budget. The runtime deadline evaluates the actual event history and future delay budgets without double-charging for elapsed capture-to-admission transit.
- **A4.** The enforcer rejects replayed `veto_path`, `manual_drive`, or `accept_item` packets, and packets with invalid authentication tags or expired sequence counters, without modifying still-valid channel authority. It logs each rejection in the evidence tuple while allowing the active lease to age normally.

These authority rules turn the event log into a checkable contract. The same synchronized hardware timestamps also mark which controller generated each segment of an intervention trajectory, and that separation must exist before any segment can serve as a training target for imitation learning.

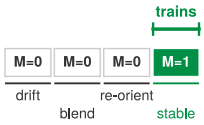
14.8.3 Causal masking and recovery eligibility

If a training pipeline feeds an entire rescued episode into behavioral cloning without filtering, the policy learns to imitate the autonomous drift that caused the near-miss, the enforcer's synthetic blend spline, and the operator's initial startled oscillations. The curation rules of section 5.8 prevent that contamination by deciding which logged samples may train a policy: drift commands are quarantined, drift states are relabeled only by an automated expert, and only stabilized, reviewed recovery is admitted. Each rule keys on a timestamp that the authority record supplies, so a logging system that records intervention episodes for downstream policy retraining must preserve controller ownership, target meaning, and declared sample eligibility. This chapter's conservative direct-imitation policy applies **causal intervention masking** to that record. Its optimization objective over recorded trajectories is formulated as:

$$\mathcal{L}_{\text{imitation}}(\theta) = \mathbb{E}_{(\mathbf{x},\mathbf{u}_{\text{target}},\mathcal{M})\sim\mathcal{D}_{\text{train}}}\left[\sum_{k=0}^{S}\mathcal{M}(k)\cdot\ell\left(\pi_\theta(\mathbf{x}_k),\mathbf{u}_{\text{target}}(k)\right)\right] \tag{14.8}$$

where $\ell(\pi_\theta(\mathbf{x}_k),\mathbf{u}_{\text{target}}(k))$ denotes the policy prediction loss (such as mean squared error across action chunks or score-matching denoising loss for diffusion policies), $\mathcal{M}(k)\in\{0,1\}$ is a binary causal mask, and $\mathbf{u}_{\text{target}}(k)$ is the supervised training target. Each sample's mask and target are evaluated at that sample's logged hardware timestamp. The target carries its own symbol because it is not the admitted command $\mathbf{u}^*$ of the evidence tuple; in each interval it is chosen from the logged fields, or supplied by an expert, as the interval list below states. Constructing the training dataset $\mathcal{D}_{\text{train}}$ requires selective transformation rather than wholesale ingestion of raw telemetry. While

every logged state preserves its chronological position in the trajectory, its supervision target $\mathbf{u}_{\text{target}}(t)$ and mask $\mathcal{M}(t)$ depend on the physical regime of the recording interval. As a result, the command dispatched to the hardware is not necessarily the training target. Deterministic authority transitions stamped by the safety microcontroller establish controller ownership. Empirical stabilization criteria, task validity checks, and the availability of expert labels, in turn, determine which samples receive $\mathcal{M}(t) = 1$.



One intervention, four intervals; only validated, stabilized recovery trains directly as a human demonstration.

A supervisory intervention does not occur as a single monolithic demonstration. Instead, it spans four distinct physical intervals along the execution timeline:

1. **Compounding policy drift ($t_{\text{div}} \le t < t_{\text{commit}}$)**: The autonomous policy π_θ drifts away from the expert state distribution, accumulating compounding errors while the human supervisor perceives the anomaly, orients, and decides to intervene (the pre-commit latency modeled in equation 14.1).
   - *Masking policy*: The autonomous proposals $\mathbf{u}_{\text{nom}}(t)$ executed during this drift are quarantined from training targets. Without an external expert label, the system sets $\mathcal{M}(t) = 0$.
   - *Relabeling*: The *learner-visited states* $\mathbf{x}(t)$ remain in the log for the corrective aggregation of section 5.8, whose relabeling rule accepts only an automated expert's labels. A drift state therefore enters training with $\mathcal{M}(t) = 1$ and target $\mathbf{u}_{\text{target}}(t) = \pi^*(\mathbf{x}_t)$ only when such an expert, for example an omniscient trajectory planner with privileged simulation state, supplies the correction.

2. **Atomic handover and enforcer blend ($t_{\text{commit}} \le t < t_{\text{commit}} + \tau_{\text{blend}}$)**: The safety microcontroller latches the channel endpoints and computes a C^2 continuous smoothstep trajectory between controllers.
   - *Masking policy*: The system sets $\mathcal{M}(t) = 0$ unconditionally. Dispatched setpoints $\mathbf{u}(t)$ during this window are synthetic interpolation splines generated by the enforcer's deterministic quintic polynomial profile. Because they reflect neither learned neural policy intent nor human cognitive demonstration, training on this interval would condition the network to emit open-loop kinematic ramps detached from environmental feedback.

3. **Operator re-orientation and hunting ($t_{\text{commit}} + \tau_{\text{blend}} \le t < t_{\text{stabilize}}$)**: Upon receiving the active manual lease ($T_{\text{lease,man}}$), the operator experiences neuromuscular latency, visual parallax, and control re-orientation, producing high-jerk, oscillatory corrections while regaining situational awareness.
   - *Masking policy*: The system enforces $\mathcal{M}(t) = 0$. Bainbridge documented this control hunting directly. An operator whose active manual skill has lapsed during passive monitoring makes abrupt step corrections, oscillating around the setpoint before settling into smooth tracking (Bainbridge 1983). Control stabilization often takes significantly longer than the physical handover itself. In driving simulator studies, operators resuming control from automated systems required approximately 35–40 s before their lateral steering stabilized (Merat et al. 2014), at least 7× the length of the machine's 5 s pre-trigger window. Consequently, post-trigger masking cannot rely on a fixed time constant; logging systems must evaluate empirical stabilization criteria. Discarding these initial frames prevents hesitation and oscillatory hunting from corrupting the imitation buffer. This exclusion involves an engineering trade-off, because interval 3 also contains the operator's initial corrective inputs, which certain human-gated methods retain (table 14.8). The enforcer marks the stabilization boundary $t_{\text{stabilize}}$ only after tracking error and command slew have remained within calibrated limits for a declared dwell interval. Later violations mask the affected samples again. The limits are:

$$\|\mathbf{e}_{\text{track}}(t)\| \le \epsilon_{\text{stable}} \quad \text{and} \quad \|\dot{\mathbf{u}}_{\text{human}}(t)\| \le j_{\text{stable}}.$$

     Here, $\mathbf{e}_{\text{track}}(t)$ is the difference between the admitted manual setpoint and the measured plant response, $\dot{\mathbf{u}}_{\text{human}}(t)$ is the time derivative of the operator's command, and the parameter j_{stable} denotes a channel-specific slew ceiling, representing an acceleration derivative (jerk) only when the underlying command vector represents acceleration. Both

thresholds must hold continuously across the dwell window before the logger lifts the mask.

4. **Stabilized expert recovery ($t \geq t_{\text{stabilize}}$)**: The operator executes coherent, task-directed recovery maneuvers once measured control metrics stabilize. Smooth motion alone does not validate a recovery; the episode must also satisfy formal task-completion criteria before its samples qualify as expert targets.
    - *Masking policy*: The system sets $\mathcal{M}(t) = 1$ with imitation target $\mathbf{u}_{\text{target}}(t) = \mathbf{u}_{\text{human}}(t)$ exclusively for stabilized segments that pass task-validity review. All other telemetry frames remain masked from direct imitation, so this interval is the only post-takeover segment the curation rules admit as expert recovery data.

Feeding unsegmented telemetry directly into behavioral cloning or diffusion policies injects synthetic enforcer ramps and operator control transients into the training distribution. The causal mask $\mathcal{M}(t)$ and target vector $\mathbf{u}_{\text{target}}(t)$ use the commit and state timestamps recorded by the safety microcontroller to enforce controller ownership boundaries, bounded by hardware clock synchronization and sampling limits.

The binary causal mask represents the most conservative member of a broader algorithmic spectrum. Contemporary human-in-the-loop learning methods treat interventions in fundamentally different ways: as direct state labels, as importance weights over surrounding trajectories, or as negative reinforcement signals indicating policy failure. Table 14.8 maps five representative paradigms onto these four operational intervals. For instance, Human-Gated DAgger (HG-DAgger) grants the human supervisor full authority upon takeover and aggregates only the human's executed commands, leaving interval 1 unmasked but unlabeled (Kelly et al. 2019). Sirius generalizes $\mathcal{M}(t)$ by assigning categorical importance weights across sample classes; it assigns zero weight to a fixed pre-intervention window while upweighting human recovery trajectories (H. Liu et al. 2023). Specifically, Sirius sizes its pre-intervention window to 15 robot actions (approximately 2 s), derived from observed human reaction times. While Sirius fixes this window a priori, the authority record timestamps the commit and supports reconstruction of the drift interval once t_{div} is identified from the retained evidence, a duration that the out-of-loop takeover budget of section 14.2.2 shows can be 2.2× longer. Meanwhile, Reinforcement Learning from Intervention Feedback (RLIF) eliminates the assumption of human optimality altogether, treating the intervention event as a negative reward penalty within an off-policy reinforcement learning framework (J. Luo et al. 2024). Similarly, Human-in-the-Loop Sample-Efficient Robot Learning (HIL-SERL) incorporates human interventions as targeted correction buffers for real-world reinforcement learning (Luo et al. 2025). Finally, RECAP, which trains the $\pi^*_{0.6}$ vision-language-action model, blends autonomous rollouts, teleoperated interventions, and demonstrations using advantage conditioning, explicitly labeling human takeover corrections as positive-advantage actions (Physical Intelligence et al. 2025).

Table 14.8: Human-in-the-loop learning methods as of 2026, placed on the four intervals: Every method needs the ownership boundary the authority record holds; they differ in what the intervention becomes and in which intervals they train on. None trains on the enforcer's blend as a demonstration. Entries summarize each method's published design (Kelly et al. 2019; H. Liu et al. 2023; J. Luo et al. 2024; Luo et al. 2025; Physical Intelligence et al. 2025); a later edition should refresh the list.

Method (year)	What the intervention becomes	Interval 1 (policy drift)	Interval 3 (re-orientation)	Needs from the authority record
Causal mask (this chapter)	Boundary between owners	Excluded unless an automated expert labels it	Excluded until measured stabilization	Commit, blend end, stabilization times
HG-DAgger (2019)	Label: human commands while in control	Not labeled	Kept with the human's commands	Owner per sample
Sirius (2023)	Weight per sample class	Fixed pre-intervention window, weight zero	Kept, upweighted	Commit time; window measured to it
RLIF (2024)	Reward: the takeover itself is a penalty	Kept; the takeover penalizes it	Human samples added to the data	Commit time as the reward event
HIL-SERL (2025)	Correction data for a reinforcement learner	Kept as the learner's own experience	Kept as correction data	Owner per sample

Method (year)	What the intervention becomes	Interval 1 (policy drift)	Interval 3 (re-orientation)	Needs from the authority record
RECAP, $\pi^*_{0.6}$ (2025)	Correction marked as a positive-advantage action	Kept; a value function scores it	Kept, marked positive	Owner per sample; episode outcome

Individual learning algorithms change, but every one of them must answer the same three systems questions: who held write authority over each sample, whether the takeover serves as a supervised label, an importance weight, or a reward penalty, and which physical intervals enter the training set. Downstream training pipelines cannot recover those ownership boundaries from trajectory kinematics alone. Recording the commit, blend-end, and stabilization timestamps on the safety microcontroller's clock grounds dataset curation and fleet-level intervention rates directly in hardware measurements.

When an engineering team assembles the safety case for release (section 16.3.3), the authority record also supplies the primary evidence that runtime handovers obey rules A1 through A4 (claim C6 in table 16.2). That release dossier pairs the static authority manifest with the loaded stopping and blending measurements from section 14.4 and hardware-in-the-loop traces that verify an unbroken audit chain. Because a handover that looks clean on the communication bus can still slip a wheel on a wet floor or jolt a loaded tote rack, the dossier always checks bus timestamps against independent plant sensors. It treats measured preemption times as empirical percentiles rather than analytical worst-case bounds.

↩ The distinction between an empirical latency percentile and a response-time bound is established in section 4.5.

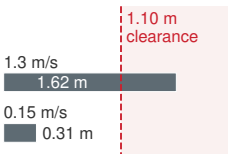
Checkpoint 14.3: Authority evidence and causal masks

The authority record is useful twice, once to an investigator and once to a learner, and both uses depend on the same ownership timestamps.

- ☐ **Evidence tuple**: Can you name the incident question that becomes unanswerable when the log drops $\mathbf{u}_{\text{nom}}$, when it drops $\mathbf{u}^*$, and when it drops $T_{\text{lease,rem}}$?
- ☐ **Interval masks**: For one takeover, can you assign $\mathcal{M}(t)$ and $\mathbf{u}_{\text{target}}(t)$ to each of the four intervals and name the logged event that bounds each?
- ☐ **Sample count estimate**: An out-of-loop takeover commits 4.5 s after the divergence, and the arm then blends for 38 ms. At the 1 ms permission tick, can you estimate how many logged ticks fall in intervals 1 and 2, and how many of them may train an imitation policy when no automated expert is available?
- ☐ **Fixed windows**: Can you explain what a fixed 2 s pre-intervention window misclassifies for a takeover that took the design out-of-loop time?

14.9 Fallacies and Pitfalls

Flipping a bit in an authority register does not complete a safe supervisory handover. A real-time physical AI system must preserve its dynamic stopping margin and protective enforcement functions throughout every stage of transfer. Furthermore, downstream imitation learning pipelines cannot treat intervention logs as unblemished expert recoveries without explicit causal filtering. When systems fail in deployment, the breakdown almost always traces back to one of four subtle architectural traps.



One 500 ms window overruns at aisle speed and idles at a crawl.

Pitfall: *Treating human takeover timeout as a static time delay rather than a dynamic spatial stopping budget.* Consider a mobile manipulator that initiates a takeover request at its operational 1.3 m/s aisle speed under a statically configured 500 ms acknowledgment window. If the controller waits out that full window before commanding brakes, the vehicle covers 0.65 m of unbraked travel. When combined with the jerk-limited resident stop profile and actuator execution overheads, the total stopping distance ends 0.52 m beyond the 1.10 m rack-end clearance, causing a physical collision. Conversely, at an out-of-loop crawl speed of 0.15 m/s, that same window consumes negligible distance, leaving 5.8 s of margin before fallback braking must begin. The wait costs travel in proportion to speed, and the stop that follows grows with its square, so a timeout that leaves ample response time

at a crawl overruns the clear distance at aisle speed. Real-time supervisory handovers cannot treat human response intervals as fixed wall-clock durations. The permissible wait time must be computed dynamically from instantaneous velocity, remaining obstacle clearance, and validated brake deceleration envelopes. Autonomous fallback must begin the moment the remaining clearance matches the required stopping profile.

A supervisory timeout is not a static timer; it is a dynamically bounded spatial margin that must fit the instantaneous stopping budget.

Architect's countermeasure: On every control cycle, evaluate the instantaneous obstacle clearance against a complete stopping budget composed solely of future sensing delays, brake actuation lag, and jerk-limited deceleration. Dynamically recompute a candidate absolute deadline, anchored to a consistent time origin, from equation 14.4, which gives the longest wait whose delay travel and stop suffix still fit inside the remaining clearance. If a newly calculated deadline is earlier than the currently active deadline, tighten the expiration threshold immediately; never extend or restart an active countdown based on optimistic recalculations. Account for sensor pipeline latency consistently, and do not subtract historical capture-to-enforcement delay from clearance measurements that have already physically reflected that travel. If supervisory acknowledgment does not arrive before the deadline expires, immediately trigger the validated autonomous fallback brake sequence.

Fallacy: *Human manual takeover should bypass automated safety barriers and grant unconstrained, open-ended actuator authority.*

When a remote operator takes manual control of a mobile manipulator to steer it around an unexpected obstacle, the chosen steering commands might vector the chassis directly toward a warehouse rack upright. Granting manual control alters only the origin of proposed motion setpoints; it cannot alter chassis momentum, tire-floor friction coefficients, or environmental obstacle boundaries. The low-level safety enforcer must therefore evaluate human manual proposals against the formal admissible motion set on every cycle. If the operator commands a trajectory that risks collision or dynamic instability, the enforcer projects the command onto the nearest safe, kinodynamically feasible trajectory. If no safe trajectory exists within the admissible envelope, the enforcer rejects the proposal entirely and triggers an immediate autonomous fallback. Manual intervention grants the operator authority to guide high-level intent, but mechanical stresses, obstacle clearance, and brake margins remain hard physical constraints enforced at the joint actuator boundary.

Furthermore, manual intervention must operate under strictly bounded, perishable leases rather than open-ended authority grants. Transient wireless packet loss or operator distraction can easily leave stale velocity vectors active in flight. To prevent uncommanded runaway motion, every intervention session requires a cryptographically renewed manual lease calibrated to the active operational mode's stopping reserve. In this warehouse aisle scenario, the system enforces a 100 ms manual lease. If periodic, authenticated heartbeat renewals fail to arrive before the lease expires, the safety microcontroller immediately revokes proposal authority and executes an autonomous fallback stop independently of host software or operator intervention.

Manual intervention transfers the authority to propose setpoints, but never the authority to bypass physical barrier certificates or operate on stale communication.

Architect's countermeasure: Channel all teleoperation and manual override inputs through the enforcer's control barrier certificates and kinodynamic clamping filters. When an operator command exceeds dynamic rollover thresholds, threatens structural shear limits, or breaches clearance buffers, the enforcer automatically clamps the actuation setpoint to the boundary of the safe set or initiates an emergency stop. Subordinate all manual authority to a hardware-timed watchdog lease sized to the active mode's stopping reserve, guaranteeing that lease expiration automatically executes a deterministic brake or hold sequence without software intervention.

Pitfall: *Relying on cryptographic authentication without rate limiting the traffic that reaches the verifier.*

Cryptographic authentication guarantees message authenticity and integrity, but it introduces an asymmetric computation cost that an adversary or a misbehaving transceiver can exploit to starve real-time processing. Suppose an admission controller validates incoming signed remote takeover requests (section 14.5) using an embedded microcontroller that requires 1.8 ms to verify each Ed25519

digital signature. Malformed inputs may be rejected cheaply, but well-formed invalid signatures can still reach the expensive verification path. The following queue model charges every such request the stated verification time.

If a malfunctioning or compromised device on the network link floods the interface with bogus signed `manual_drive` packets, as few as 55 forged requests queued ahead of a critical `veto_path` message from a remote supervisor will delay its verification to 100.80 ms. At aisle speed, this delay exceeds the fallback deadline $T_{\text{deadline,fallback}}$ of 100.78 ms. When charging the allocated downstream admission delay Δ_{da}, the usable admission margin shrinks to 37.5 ms, meaning just 20 forged packets suffice to cause a deadline violation. Sustaining this denial-of-service attack requires an injection rate of only 546 packets per second, a trivial load for even a low-bandwidth industrial wireless channel.

Substituting symmetric MACs shifts the computational bottleneck to the network transceiver without eliminating the underlying vulnerability. Validating a MAC requires far fewer cycles, so roughly 20,306 forged frames within the same window are needed to induce a miss, yet the queue remains susceptible to line-rate saturating traffic. The robot's independently wired protective stops and hardware lease timers must never share queues with network ingress. Those local safety channels must remain fully operational even when the cryptographic verifier is saturated. To protect real-time responsiveness, the ingestion pipeline must deploy hardware mailbox filters and per-source rate limiters ahead of the verification queue, and safety qualification tests must evaluate worst-case intervention latency under full adversarial packet inundation.

Cryptographic authentication protects command integrity but introduces an asymmetric availability risk, because an admission verifier spends finite compute cycles rejecting forged traffic before it can service critical safety messages.

Architect's countermeasure: Deploy hardware mailbox filtering and per-source token-bucket rate limiters[13] on the fieldbus interface ahead of the software verification queue. In addition, route emergency protective stops and fail-safe interlocks through dedicated, hardwired electrical circuits that bypass digital networking and cryptographic parsing entirely.

[13] **Token Bucket**: A limiter holds up to b tokens, refills at rate r, and admits a packet only by spending a token, so a source may burst b packets but never sustain more than r. Placed before the verifier, it bounds the forged signatures that can arrive within $T_{\text{deadline,fallback}}$ at $b + r\,T_{\text{deadline,fallback}}$. For the single-source queue modeled here, keeping that bound below the 55 forgeries that push a genuine veto past its deadline, or 20 once Δ_{da} is charged, sizes b and r from the stopping budget rather than from link capacity. A deployed ingress must also bound aggregate traffic across sources, include already queued work and legitimate traffic, and prevent forged source identifiers from bypassing those limits.

Fallacy: *Training directly on raw human intervention logs captures expert demonstrations.*

Training on raw intervention logs commits the learner-action contamination of section 5.8. The raw telemetry of a rescued episode is not a continuous expert demonstration. It stitches the policy's diverging trajectory to the operator's reaction latency, the corrective jerks as the operator regains situational awareness, and the overshoots before stabilization, and a policy cloned from it reproduces the oscillations that made the takeover necessary.

The methods in table 14.8 require attribution of who generated each state-action pair and a declared interpretation of the sample as a target, a weight, or a reward. Neither a coarse session-level takeover timestamp nor an active manual authority bit guarantees that recorded telemetry is fit for policy training. This chapter's conservative direct-imitation policy segments intervention trajectories into distinct causal regimes and masks transitional dynamics; the alternative objectives in table 14.8 retain some early corrections.

Unsegmented intervention telemetry records human cognitive lag and abrupt corrective overshoots; naively cloning these trajectories trains policies to reproduce the instability that triggered the takeover.

Architect's countermeasure: For this chapter's conservative direct-imitation corpus, apply causal intervention masking (equation 14.8), taking the interval boundaries from the commit, blend-end, and stabilization timestamps on the safety microcontroller's monotonic hardware clock. Only Interval 4, validated post-stabilization guidance, trains as a human demonstration ($\mathcal{M}(t) = 1$); Interval 1 drift states train only under an automated expert's labels, and everything else stays in the record for forensics.

14.10 Summary

Supervisory intervention transfers the right to propose, never the right to admit. A capability matrix grants each authenticated role named operations on named actuator channels, each with a preemption tier and a clearance-derived deadline, while the enforcer checks every command from

every source against the same physical envelope. Enforcement changes a proposal and leaves the proposer in place; intervention changes the proposer and leaves the check in place, and the machine needs both.

A takeover is a delay inserted before braking on a moving machine, so it is priced in the stopping budget rather than in the user interface. At the 1.3 m/s aisle speed the subtotal before dispatch-to-admission travel leaves 131.0 mm, a waiting window of 100.8 ms, and only 37.5 ms once the 63.3 ms dispatch-to-admission allocation of section 13.6.2 is charged. Sourced human responses of 605 ms in the loop and 4.5 s out of it cap the request speed at 0.797 m/s and 0.189 m/s once it is charged. No brake, however strong, admits the aisle speed, because the waiting term rather than the braking term spends the clearance. The design therefore slows to 0.75 m/s before an anticipated in-loop request, crawls at 0.15 m/s or stops first before an out-of-loop one, and answers a newly discovered hazard with its own local stop. None of these speeds qualifies until loaded measurements confirm the allocation and the braking premises.

Authority is bounded and moved by mechanisms that never wait on the network. Execution leases on the safety microcontroller's clock, the autonomous chunk lease and the 100 ms manual lease, revoke a silent proposer and select the fallback that matches the machine's state. A four-phase handshake commits ownership on one control tick for an explicit channel mask, the enforcer remains the only writer to the actuators, and every timer that expires leads to a defined state. Authentication keeps forged commands out of admission, but only a rate limiter ahead of the verifier keeps forged traffic from delaying a genuine veto, and hardwired protective inputs bypass both.

Changing the proposer does not change the physics of the command. A fixed-endpoint C^2 smoothstep bounds the command rate between latched endpoints, but it spends clearance. The base window is 528 ms at every speed because the jerk bound belongs to the loaded base, and at the 0.75 m/s request speed no braking blend fits the clearance that remains. Within the stated one-dimensional stopping model and boundary conditions, no smoothed braking under the same bounds beats the resident profile, so the base receives the resident stop while the arm blends over 38 ms.

The authority record turns every handover into evidence. It binds the sealed static manifest to per-tick evidence tuples that record who held each channel and what was proposed, admitted, and dispatched alongside independent measurements of the response, chains them against tampering, and declares every gap. It draws its fallback rungs from the enforcement record and its declared timing bounds from the placement record, whose loaded measurements are still to come. The same ownership timestamps serve two readers. An investigator can reconstruct an incident from them, provided independent current, encoder, and contact measurements confirm what the commands did, and a learner can apply this chapter's conservative four-interval direct-imitation mask, under which only validated, stabilized human recovery supplies post-takeover demonstration targets. Other learning objectives use the same provenance with different eligibility rules (table 14.8). Table 14.9 summarizes the governing quantitative relations for supervisory intervention, detailing the specific operational or physical quantity bounded by each equation.

Table 14.9: Key equations of supervisory intervention: Each equation fixes one quantity a takeover design must declare. The kinematic relations assume the jerk-limited resident stop and a constant speed while the machine waits; the ceiling's closed form holds only above $a_{\text{brake}}^2/j_{\max}$. The delays inside them must be measured on the site's operators, console, and link.

Equation	What it bounds or decides	Derived in
$T_\Sigma = T_{\text{sensory}} + T_{\text{cognitive}} + T_{\text{motor}} + T_{\text{comm}}$	The takeover delay a person and a console add before any command changes	equation 14.1
$v(T_{\text{wait}} + \tau_{\text{delay}}) + d_{\text{suffix}}(v) + \delta \le D_{\text{clear}}$	Whether the machine can still stop after waiting T_{wait} at speed v	equation 14.2
$T_{\text{deadline,fallback}} = (D_{\text{clear}} - \delta - d_{\text{suffix}}(v))/v - \tau_{\text{delay}}$	The latest fallback dispatch after a handover request; a fixed timeout is clipped to it	equation 14.4
$v_{\max} = a_{\text{brake}}\left[\sqrt{b^2 + 2(D_{\text{clear}} - \delta)/a_{\text{brake}}} - b\right]$	The highest speed at which a takeover with delay T_Σ may be requested	equation 14.3
$t - t_{\text{refresh}} < T_{\text{lease}}$ or $T_{\text{lease,man}}$	Whether a proposer still holds authority; expiry invokes the validated local response	equation 14.5

Equation	What it bounds or decides	Derived in
$\alpha(\hat{t}) = 10\hat{t}^3 - 15\hat{t}^4 + 6\hat{t}^5$ with $\tau_{\text{blend}} \geq 1.875\|\Delta\mathbf{u}\|/j_{\max}$	The minimum window within the fixed-endpoint quintic family under the stated rate ceiling	equation 14.6 and equation 14.7
$\mathcal{L} = \mathbb{E}[\sum_k \mathcal{M}(k)\,\ell(\pi_\theta(\mathbf{x}_k), \mathbf{u}_{\text{target}}(k))]$	Which logged samples train the next policy, and with which target	equation 14.8

Key Takeaways: Supervisory intervention and authority bounds

- **State-dependent fallback and monotonic leases:** Bind each proposer to an explicit, mode-specific lease and stopping reserve. Lease expiry invokes the validated local fallback response appropriate for the current physical state, along with the sensing, drive energy, braking, and holding premises required to execute it safely. For manual teleoperation, the reference design chooses a 100 ms lease, bounded from below by heartbeat loss and jitter and from above by the clearance its extra travel over the chunk lease consumes after the takeover.
- **Clearance governs waiting:** A takeover delay is travel before braking, so its allowance comes from the stopping budget, not from a configured timer. The subtotal before dispatch-to-admission travel allows 100.8 ms of waiting at the aisle speed, 37.5 ms once the 63.3 ms admission allocation is charged, and 751.6 ms at the 0.75 m/s slow-down, which still leaves 83.3 ms after an in-loop takeover and the charge. Price human responses at design percentiles, not averages. On the complete budget, the 605 ms in-loop design response caps the request speed at 0.797 m/s and the 4.5 s out-of-loop response caps it at 0.189 m/s. Dispatch the fallback at the earliest of the protocol, lease, evidence, and clearance deadlines, and stop before handing an unexpected hazard to an out-of-loop operator.
- **Single-writer actuator ownership:** Explicit source arbitration and exclusive enforcer command ownership eliminate conflicting actuator writes; low-level bus arbitration alone cannot establish safety authority. Every request, commit, and confirmation message must explicitly declare its channel mask. A partial transfer, such as reassigning arm control while the base keeps navigating autonomously, is admissible only while the console annunciates the untransferred channel's owner; otherwise that channel holds. A four-phase protocol (Request, Acknowledge, Commit, Confirm) transitions authority deterministically. The commit transition revokes the autonomous policy token on the designated channels and latches its last admitted command. No proposer holds the proposal token during `BLEND`, while the enforcer owns the command; the arbiter grants the token to the operator only after the profile completes, and the enforcer then checks human proposals against the same physical envelopes as the policy's.
- **Smooth command blending and haptic passivity:** A fixed-endpoint C^2 acceleration profile bounds peak jerk during authority transitions, and its additional stopping displacement must be reserved in the braking envelope. If the system incorporates bilateral force reflection, the console feedback loop requires its own passivity and stability argument evaluated under realistic network latency, packet loss, and actuator dynamics.
- **Independent stopping paths:** Dedicated, hardwired protective circuits execute risk-assessed mechanical and electrical stops independent of software. Network-delivered commands undergo cryptographic admission checks: corrupted or unauthenticated packets are dropped at ingress, whereas a failure to refresh a valid lease triggers an autonomous local fallback.
- **Causal data attribution:** Training on intervention logs requires preserving controller provenance and declaring target and eligibility rules. This chapter's conservative direct-imitation mask $\mathcal{M}(t)$ (equation 14.8) preserves learner-visited drift states for failure analysis and automated relabeling, zeros out synthetic enforcer blending splines (τ_{blend})

and initial operator hunting, and admits only validated, stabilized human recovery as post-takeover demonstration targets. Other objectives may retain or weight early corrections differently (table 14.8).

- **Authority as auditable evidence:** The authority record logs per-tick evidence tuples $\langle t_{\text{MCU}}, S_{\text{auth}}, \mathbf{x}, \mathbf{u}_{\text{nom}}, \mathbf{u}^*, \tau, \text{ID}_{\text{operator}}, T_{\text{lease,rem}} \rangle$ into tamper-evident cryptographic hash chains, treating a missing entry as an explicit fault rather than an implied pass. The manifest's four authority rules serve as primary test targets for verification fault injection; their passing logs, anchored by the authority record, are the evidence the release argument needs before it may admit remote takeover.

A person's authority over an embodied physical AI system is granted on verifiable evidence and expires when that evidence lapses, as the fourth law, permission must be earned (principle 4), requires of every privilege the machine honors. Fresh credentials, checked plant state, a feasible target and fallback reserve, and a successfully committed handshake support the grant of authority; the manual lease bounds how long the grant remains valid while its physical premises continue to hold. When operator inputs cease renewing the lease, authority immediately lapses to a verified local fallback governed by the hardware permission path. Preemption tiers do not persist across a lease expiration: a remote assistant who previously commanded the mobile base retains no residual authority once the lease expires. Re-establishing control under any proposer requires satisfying the reset conditions of the authority state machine and presenting fresh, validated credentials.

14.11 Historical Perspective and Further Reading

Supervisory control emerged as an engineering discipline when teleoperated systems outgrew direct manual control. In deep-sea exploration and hazardous environment manipulation, transmission latency and high operator workloads made continuous teleoperation fragile. Sheridan and Verplank (1978) formalized the division of labor between human operators and machine intelligence, grading autonomy into explicit levels that range from complete manual execution to autonomous computer execution that ignores human input. Sheridan (1992) consolidated these insights into a unified systems theory of human supervisory control, modeling how human operators alternate between planning, programming, monitoring, intervening, and learning from automated subroutines. Expanding on this taxonomy, Parasuraman et al. (2000) decoupled automation across four functional dimensions: sensory information acquisition, information analysis, decision selection, and action execution, assigning distinct operational levels to each stage. Meanwhile, industrial process automation approached supervisory handovers from a control-theoretic perspective. Switching a closed-loop plant between manual operation and automatic tracking without inducing mechanical shock or actuator saturation requires the idle controller to track the active plant state continuously. This bumpless transfer technique, detailed as standard practice by Åström and Hägglund (2006), provides the classical foundation that this chapter's enforcer generalizes: smoothly blending between verified endpoint states while strictly respecting physical actuator limits.

Human factors engineering revealed why relying on a passive human operator as an emergency fallback is fundamentally flawed. In a landmark critique, Bainbridge (1983) articulated the central irony of automation: automating routine operations leaves the human supervisor responsible solely for rare, anomalous edge cases, precisely when sustained inactivity has degraded both situational awareness and manual control proficiency. Sarter and Woods (1995) examined cockpit automation to trace the root causes of mode confusion, demonstrating how silent automation state changes lead pilots to issue inputs that are entirely correct for their assumed operational mode but hazardous for the aircraft's actual mode. Endsley (1995) formalized the cognitive basis of operator intervention, decomposing situation awareness into three hierarchical tiers: perceiving sensory cues, comprehending the operational state, and projecting system trajectories into the near future. Physical latency imposes strict mathematical limits on these interactions. Ferrell (1965) demonstrated that when loop delays exceed a fraction of a second, human operators inevitably degrade into a disjointed "move-and-wait" strategy to avoid destabilizing the physical plant. In bilateral teleoperation with force reflection, delayed feedback injects non-physical energy into the loop and can destabilize it. Anderson and Spong (1989) addressed this instability through a passive communication-channel

construction within a specified bilateral control architecture. Closed-loop stability also depends on the channel terminations and the coupled controllers and plant. Niemeyer and Slotine (1991) reformulated the communication-channel passivity condition using wave variables. Variable delay, packet loss, sampling, and saturation require the additional analysis identified in section B.10. At the architectural level, arbitrating between competing subsystem commands using rigid priorities directly descends from the layered suppression and inhibition mechanisms pioneered by Brooks (1986) in the subsumption architecture.

Machine learning turned supervisory intervention into a source of training data. Ross et al. (2011) proved that naive behavioral cloning fails under distributional drift because the autonomous policy inevitably encounters states absent from the expert demonstrations; robust training requires querying the expert along the learner's own induced state distribution via the DAgger algorithm. To reduce human fatigue, Kelly et al. (2019) introduced Human-Gated DAgger, delegating continuous control to the autonomous policy and recording demonstrations only when the human supervisor actively intervenes. Modern embodied systems generalize these intervention boundaries into core training signals. Sirius (H. Liu et al. 2023) uses intervention transitions as prioritized sample weights in offline policy optimization. J. Luo et al. (2024) converts supervisory takeovers into reinforcement learning reward penalties. Luo et al. (2025) uses intervention corrections to drive real-world robotic reinforcement learning safely on physical hardware. Every data-driven approach relies on unambiguous execution provenance: the system must record exactly which agent (human supervisor, machine policy, or safety enforcer) held authority over every actuator command. Tamper-evident logging draws on a deep cryptographic lineage, from the cryptographically linked timestamps of Haber and Stornetta (1991) to the hierarchical authentication structures of Merkle (1988). This chapter integrates these disparate systems threads into a single runtime abstraction: a per-tick authority record that aligns the handover sequence with the physical stopping budget and retains ownership boundaries for downstream policy learning. The arbiter and enforcer establish exclusive channel ownership; per-channel announcements support mode awareness, and the record supports evaluating both under its timing and capture assumptions.

Six works trace this lineage from the ironies of automation to learning from interventions:

- Bainbridge (1983) articulates why automating routine operations deprives human operators of the running mental model and manual dexterity required to handle anomalous edge cases, establishing the engineering motivation for executing a safe dynamic stop before requesting intervention.
- Sheridan (1992) provides the definitive monograph uniting supervisory control, teleoperation architectures, and the dynamics of delayed human-in-the-loop systems.
- Sarter and Woods (1995) identifies the cognitive mechanics of mode awareness breakdowns in complex cockpits, illustrating the failure modes that per-channel masks and state announcements mitigate in section 14.6.
- Parasuraman et al. (2000) establishes the multi-dimensional taxonomy of automation across acquisition, analysis, decision, and action, providing the conceptual architecture behind channel-by-channel capability matrices.
- Anderson and Spong (1989) develops a passive communication-channel construction for delayed bilateral teleoperation, forming the analytical basis for section 14.4.5. Its closed-loop stability argument depends on the specified channel terminations and control architecture.
- Ross et al. (2011) introduces the DAgger algorithm and demonstrates that imitation learners must be trained against corrections collected on their own visited state distributions, explaining why the authority record must capture policy drift intervals and takeover boundaries.

What's Next: From authority revocation to verification

Operational logs alone cannot establish that the arbiter enforces its rules under adverse conditions. Weeks of clean handovers in the authority record rarely contain a duplicate writer, a replayed packet, or a dropped heartbeat, and a record cannot testify about a fault that never occurred. The same gap runs through every record this book has developed, because the chunk

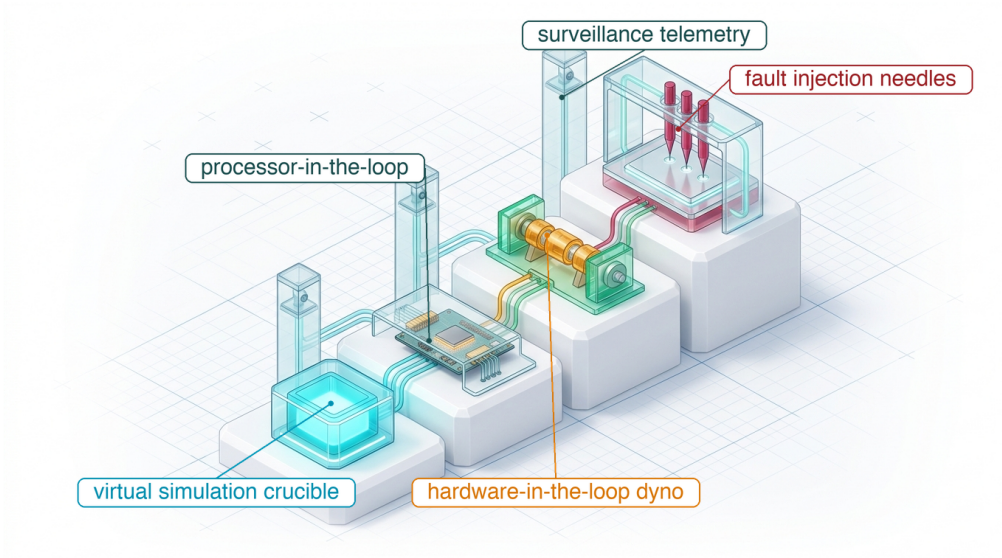
lease, the permission path, and the held-up permission rail all describe protective responses that normal operation never exercises. Chapter 15 creates those faults on purpose. It pairs four physical faults injected as the loaded base approaches the rack end, beginning with a stalled proposer, with the four authority rules A1 through A4 that this chapter's manifest hands over. It scores each trial against an oracle fixed before the test, measured by instruments the fault cannot reach rather than by the arbiter's own account of what it did.

DRAFT

DRAFT

15

Adversarial Verification

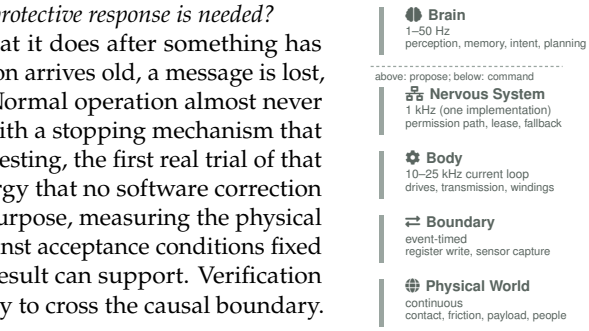


15.1 The Claim a Fault Trial Must Test
15.2 The Operating Envelope
15.3 One F1 Trial from Injection to Record
15.4 Fault Oracles and Acceptance
15.5 Extending the Trial to the Envelope
15.6 Deriving the Fault List
15.7 The Qualification Ladder
15.8 Fault Injection on Hardware
15.9 Searching Beyond the First Trial
15.10 Interpreting Campaign Statistics
15.11 Fault Records for Release
15.12 Fallacies and Pitfalls
15.13 Summary
15.14 Historical Perspective and Further Reading

Purpose

Why can a machine pass its normal operating tests and still fail when its protective response is needed?

The claims that matter most in a physical machine describe what it does after something has already gone wrong, when the learned proposer stalls, an observation arrives old, a message is lost, or the supply sags while a loaded body moves toward a person. Normal operation almost never exercises those responses, so thousands of clean runs can coexist with a stopping mechanism that has never once been made to stop the machine. Without deliberate testing, the first real trial of that mechanism happens in service, where a late response delivers energy that no software correction can recall. Establishing such a claim requires creating the fault on purpose, measuring the physical response with instruments the fault cannot reach, and scoring it against acceptance conditions fixed before the test. Every condition left untested still limits what the result can support. Verification makes those limits explicit before release grants any action authority to cross the causal boundary.

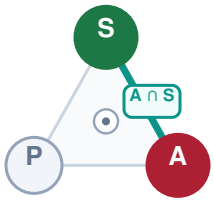


Learning Objectives

- Derive test conditions and stopping oracles from the machine's operating envelope
- Derive faults from design assumptions, physical hazards, authority-transfer failures, and attacks on the proposer and sensors
- Select qualification environments that exercise the causal mechanism behind each claim
- Design hardware injections with independent witnesses and precommitted acceptance conditions
- Interpret temporal safety margins and search for disturbances that violate them
- Evaluate the assumptions limiting simulation transfer, rare-event estimates, and formal proofs
- Assemble fault records that preserve failed, invalid, unobservable, and untested cases

15.1 The Claim a Fault Trial Must Test

The authority record at the close of chapter 14 can log weeks of clean handovers and still say nothing about what the machine does when a duplicate writer appears on the bus, a stale packet replays, or a heartbeat drops. None of those faults occurred during normal service. Every preceding chapter of this book rests on protective claims of the same kind: a chunk lease that expires within its declared deadline, a permission path that halts the mobile base within its clear distance, or a safety microcontroller that outlasts a voltage brownout on the shared electrical supply. Because those fallback paths stay dormant until a subsystem breaks, an unexpected fault discovered in service has already crossed the causal boundary into the room, releasing kinetic energy that no software update can undo. Verification turns these design assertions into auditable evidence by provoking each fault deliberately before release, under the physical loads that make the protective response hardest to execute.



A trial passes only when an independent sensor witnesses the act.

⇄ **Contrast:** Policy evaluation, which scores the learned proposer's task proficiency rather than the whole machine's protective responses, is established in chapter 7.

Policy evaluation measures how capably a learned candidate policy completes tasks across declared environments. Verification, by contrast, tests the integrated machine as a whole: proposer, runtime enforcer, communication buses, power distribution network, and physical body. High task completion rates during policy evaluation do not show that an independent emergency stop engages when the neural proposer hangs; conversely, a reliable emergency stop does not show that the learned policy navigates aisles smoothly. Qualifying an embodied machine requires both kinds of evidence gathered on the same production hardware and software stack. The same distinction separates raw policy autonomy from whole-machine safety, because an enforcer applying a least-change correction can keep the robot safe during a mission while isolating and logging a hazardous proposal from the model.

Treating passing assertions in a software simulator as sufficient proof of physical safety is the **digital twin delusion**.[1] A physics engine or digital emulator models timing jitter, power transients, and floor traction only to the extent that its authors anticipate and calibrate those effects; a simulated controller cannot fail on an unmodeled supply sag or bus collision. Passive operational logs invite an equally dangerous complacency: millions of incident-free operating hours show only that severe corner conditions did not arise during benign runs, leaving dormant failure modes untested.

Adversarial fault injection answers both traps by turning a design claim into a controlled disturbance executed inside an independent physical containment cell. To count as engineering evidence rather than an ad hoc stress test, a fault trial must declare its operating-envelope point, score telemetry against precommitted oracles that leave no room for after-the-fact negotiation, and preserve cryptographic records of the hardware and software state. The campaign then reports which points across the envelope those trials actually cover while the test harness records how each fault propagates across power rails, fieldbuses, and mechanical brakes.

Consider the warehouse mobile manipulator navigating an aisle at 1.3 m/s (chosen; see the Reader Guide) toward a rack end where a pedestrian may step out from a blind intersection. The learned proposer might abruptly stop renewing motion commands while the base continues rolling under momentum. An architectural authority diagram specifies which subsystem must take over, but that specification does not prove that the physical permission path survives the crash or that mechanical

[1] **Digital Twin:** Grieves and Vickers (2017) trace the concept to a 2002 presentation on product lifecycle management, and Glaessgen and Stargel (2012) gave it the sense aerospace adopted, a simulation of one as-built vehicle, updated continually from that vehicle's own sensors and history so that it mirrors the life of its flying twin. The original meaning therefore depended on continual calibration against one particular physical article. A simulator that has never been compared with a measured stop of this machine is a model, not a twin, and treating it as a twin is the delusion this chapter names.

brakes engage before impact. An empirical trial that induces the fault and measures physical motion supplies evidence for the named protective response under its declared conditions. To structure that trial, the test harness must reflect how the machine partitions execution across its physical substrates. An application system-on-chip (SoC) equipped with an embedded accelerator executes the unprivileged learned proposers (the vision-language-action intent model and the chunk policy beneath it). A lockstep safety microcontroller executes the deterministic runtime enforcer. The safety microcontroller commands the base-drive motor inverters across a hard real-time fieldbus (EtherCAT, or Controller Area Network with Flexible Data-Rate, CAN FD) whose cyclic frames arrive on a strict, microsecond-scale schedule. Both processors draw power from the shared 47.7 V drive bus that powers the traction motors: the application SoC through a buck-boost converter, and the safety microcontroller through an isolated converter backed by a dedicated, held-up permission rail.

Weeks of uneventful operational logs at that speed say little about four severe hardware and timing faults that the architecture claims to contain:

1. **Proposer Stall (F1):** The chunk policy, the fast proposer of section 3.5.2 that renews motion every 50 ms, has its latest proposal successfully admitted by the permission path, renewing its chunk lease, but immediately suffers a thread deadlock, driver lockup, or thermal throttling freeze.
2. **Stale Observation (F2):** A camera frame or depth tensor arrives at the permission path with an aged or corrupted hardware timestamp after silent queuing in an image signal processor (ISP) buffer or direct memory access (DMA) ring.
3. **Lost Fieldbus Frames (F3):** Electromagnetic interference (EMI) from high-current inverter switching corrupts or drops a burst of EtherCAT or CAN FD frames between the safety microcontroller and the wheel actuators.
4. **Drive-Bus Droop and Feed Loss (F4a/F4b):** A supply injector steps the shared 47.7 V drive bus below the 33 V lockout threshold of the converter feeding the application processor under the declared drive load (F4a), or the permission rail itself loses its primary electrical feed (F4b).

These physical failure modes F1–F4, paired with the supervisory authority transfer faults A1–A4 (formalized in table 15.2), organize the entire verification campaign. In theory, each threat has a dedicated countermeasure: the lease detects a hung proposer, the evidence-epoch check discards stale sensor frames, the fieldbus watchdog flags dropped communication cycles, and the isolated undervoltage supervisor manages electrical brownouts. In practice, adversarial verification injects each disturbance as the robot approaches the rack end, alongside the four authority handoff faults that the authority record hands over (section 14.8.2). It scores every stopping trial against temporal and distance bounds derived from the machine's own stopping budget.

An injected fault converts a physical trial into valid evidence for an auditable safety case only when the experiment satisfies the five requirements of the **cyber-physical test contract**:

1. **A bounded claim:** The test specifies the exact invariant the machine must maintain. For the mobile base at the rack end, the machine must come to a complete stop within the 1.10 m clear distance.
2. **Defined test conditions:** The specification bounds every relevant physical variable, including initial velocity, payload mass, floor friction, ambient operating temperature, bus traffic saturation, and DC supply voltages.
3. **A precommitted oracle:** An automated evaluation harness inspects captured telemetry against the claim without human intervention, retrospective threshold tuning, or post-hoc excuses (section 15.4 defines the oracle formally).
4. **A stated sampling and injection procedure:** The test harness exercises the machine systematically across its operational envelope, deliberately triggering faults in sensors, buses, power rails, and actuators during peak mechanical and computational load.
5. **Provenance:** The campaign logs complete cryptographic digests of model weights, calibration tables, sensor serial numbers, oscilloscope traces, and raw bus logs to enable matched re-

execution. Software builds can be reproduced bit for bit; a physical trial can be replicated only when repeated under physical conditions that match declared tolerances.

The base's stopping claim and the manipulator arm's latch-force claim require separate operational envelopes, injection points, and acceptance criteria; this chapter follows the mobile base's stopping claim. Fixing the contract starts with its second element, the declared conditions under which the claim must hold, which also locate the point beyond which nominal regulation must yield to an independent containment response.

15.2 The Operating Envelope

A mobile robot carrying a full payload down a warehouse aisle stops in a different distance when memory-bus contention delays its perception pipeline, when battery supply rails sag under acceleration, or when drive inverters heat up under sustained torque. Specifying the external operational design domain (ODD) alone therefore leaves the test conditions half-defined; an **operating envelope** couples those external conditions with the internal execution state of the computing, power, and actuation hardware. Every protective claim must declare both dimensions, the environmental bounds and the internal computational and electrical loads, under which it is guaranteed to hold.

For the mobile manipulator, the operating envelope declares the range of conditions over which the protective claim must hold. The campaign samples a discrete set of qualification points within that range; extending their results to untested points requires an explicit coverage argument. These points span an aisle speed of 1.3 m/s, payload masses ranging from an unloaded chassis to the full 50 kg rack capacity, and ambient operating temperatures from 5 °C to 35 °C. Surface friction must also be parameterized explicitly. In an optimistic all-weight traction screen, dry aisles require a friction coefficient of at least 0.204 to support the design braking deceleration of 2 m/s², whereas areas under the facility floor inspection regime guarantee only 0.12. The screen assumes that the driven wheels carry the entire weight; the actual base requires a qualified loaded deceleration that respects driven-wheel load share, configured braking current, and tip stability. For these inspected surfaces, the screen identifies a candidate restricted point at 1 m/s, whose friction premise must be logged in the enforcement telemetry. Verification trials on inspected floors must run against a physical test surface calibrated to this 0.12 threshold and must recalculate the oracles from the qualified loaded deceleration before testing the actual base.

Each physical configuration couples directly to the machine's internal execution state. Trials must evaluate application-processor utilization driven to peak neural inference bursts, drive-bus voltages sweeping from a depleted 42 V up to a nominal 47.7 V, transmission jitter across EtherCAT or CAN FD buses, DRAM crossbar contention, and the thermal state of the safety microcontroller. The verification campaign also challenges transient edge conditions, chief among them a human takeover while the base is moving. Section 14.2.4 screens a candidate in-loop takeover, with its 605 ms conservative response-and-exchange allowance, at the 0.75 m/s slow-down; the same moving handover is infeasible at the 1.3 m/s aisle speed and must be rejected. The slow-down remains a qualification candidate, conditional on operator response, admission delay, current clearance, and a feasible stopping continuation. The operative design stops the base before handing over. Both speeds therefore belong in the campaign, one to challenge a candidate moving transition and one to verify its rejection. Whenever an authority transition cannot preserve the verified stopping envelope, the enforcer must execute a local stop.

↩ The rule that an authority transition unable to preserve the stopping envelope ends in a local stop is established in section 14.2.4.

Definition 15.1: Operating envelope

Operating envelope is the declared set of external conditions and internal execution states, each point stated with its units and tolerances, over which a machine's protective claim must hold and at which every trial of that claim is run.

1. **Significance**: The envelope governs which terms of the inequality $d_{\text{stop}} \leq D_{\text{clear}}$ are stressed during an empirical trial. For the running machine, an aisle speed of 1.3 m/s falls below the optimistic 1.44 m/s ceiling on a dry floor, yet exceeds the 1.23 m/s ceiling on an inspected surface. The identical velocity therefore passes the all-weight screen

on one surface and fails it on another; passing still requires loaded qualification before admission.

2. **Distinction**: The operational design domain describes the external environment encountered by the planning policy. The operating envelope expands this definition to incorporate internal machine states, including processor utilization, drive-bus voltages between 42 V and 47.7 V, and communication bus saturation. The same physical aisle yields different safety margins under memory contention or electrical depletion.
3. **Common pitfall**: A frequent mistake is treating an operating envelope as a simple Cartesian product of decoupled intervals, assuming that testing each parameter independently at its minimum and maximum guarantees coverage. In cyber-physical systems, multi-dimensional boundary couplings are the rule rather than the exception: two parameters operating well within their nominal isolated bounds can combine to trigger an unrecoverable failure. Robust verification demands testing directly at these coupled, high-stress corners.

An operating envelope must be distinguished from the wider **survival envelope** that encloses it. Within the operating envelope, the learned policy and low-level tracking controllers regulate nominal behavior. When an external disturbance, sensor dropout, or compute stall pushes the machine across that boundary, nominal control yields to safety mechanisms that bring the robot to a bounded, safe stop before structural limits are violated. For the mobile manipulator, the survival envelope must be characterized separately for each fallback response and machine state. The standard resident stop executes controlled, powered deceleration before setting electromechanical holding brakes at zero velocity. In contrast, torque inhibit prevents commanded torque generation without necessarily electrically isolating the drive; it does not follow the controlled stopping profile or inherit its stopping-distance budget. If the remaining containment mechanisms cannot absorb the motion, structural damage may result when kinetic energy ($E_k = \frac{1}{2}mv^2_{\text{impact}}$) exceeds what mechanical compliance and chassis end-stop dampening can absorb before hard impact. For the base in a test cell, the survival envelope is therefore the set of states from which the qualified combination of torque inhibit, passive resistance, holding mechanisms, and compliant end-stops can contain the remaining motion and energy. That boundary, not the clear distance of the stopping claim, is what the test cell's own protection watches.

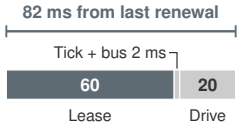
Establishing envelope boundaries requires deriving physical constraints from first principles rather than asserting arbitrary software timeouts. In rack-end operations, this boundary is governed by the stopping budget. Total stopping distance scales linearly with pre-braking latency and quadratically with vehicle speed (equation 2.2). In the conditional design budget of table 12.1, a pre-brake subtotal of 133.6 ms measured from sensor capture, combined with the jerk-limited resident stop and the tracking inset, yields a partial-budget speed ceiling of 1.44 m/s on a dry floor. That ceiling holds only before the dispatch-to-admission interval is charged, and section 13.6.2 allocated that interval without yet measuring it. Running at the candidate aisle velocity of 1.3 m/s consumes 969.0 mm of the available 1.10 m clearance. The unallocated spare of 131.0 mm, which the vehicle crosses in 100.8 ms, must still absorb that interval. Its allocation of 63.3 ms costs 82.3 mm of travel at this speed and leaves 48.7 mm, or 37.5 ms of waiting, the same remainder that section 14.2.4 found for a human takeover at aisle speed. The full 100.8 ms is therefore a ceiling, the longest dispatch-to-admission interval the spare could pay for. When later examples in this chapter quote the full spare, they quote an upper bound on the unbudgeted delay the machine can tolerate; once the allocation is charged, only the remainder is left to spend.

↩ The fallback rungs, from the controlled resident stop to hardware torque inhibit, are established in section 12.9.1.

Two conditional oracle bounds follow from that budget. Before any qualification trial, the harness must show that the bounds cover the whole execution path from the reference event to rest, because an interval left unallocated is an open qualification gap rather than a margin.

1. **Powered Deceleration Onset Deadline:** Powered deceleration must begin no later than 82 ms after the last valid renewal. This deadline traverses the full permission and actuation chain: the 60 ms lease duration, a single 1 ms tick of the safety permission loop, one 1 ms communication bus cycle, and the 20 ms drive latency from delivered stop command to physical deceleration

↩ The jerk-limited resident stop and its stopping distance are established in section 11.5.2.



Of the 82 ms onset deadline, the lease takes 60 ms.

onset. (For the power-rail fault scenario F4, the onset deadline is referenced instead to the assertion of the hardware undervoltage flag.)

2. **Stopping Distance Bound:** The vehicle, tracked by external optical instruments independent of onboard odometry, must come to rest within the distance allocated from the reference event that triggers the fault. Three reference events are possible, and figure 15.1 places them on one time axis. A *capture-referenced* trial measures clearance and displacement from the exposure midpoint of a qualified sensor capture; the 969.0 mm subtotal starts there but omits the dispatch-to-admission interval, so it is incomplete until that interval is added. An *appearance-referenced* trial starts earlier, at the optical appearance of an unheralded hazard. It must timestamp that appearance and the first usable capture independently, and add the travel between them, as well as the dispatch-to-admission travel, to obtain a complete bound against the clearance measured at appearance. A *renewal-referenced* trial applies when a stalled proposer produces no new target at all; it scores both stopping distance and onset latency from the last valid lease renewal. No trial may claim a pass against an incomplete budget subtotal.

↩ The lease path, from the last admission through the permission tick and bus cycle to brake onset, is established in section 4.4.

Measured from that event, the two F1 verification predicates are (equation 15.1):

$$t_{\text{onset}} \le T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}, \qquad d_{\text{F1}}(v) = v\,(T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}) + d_{\text{suffix}}(v) + \epsilon_{\text{track}} \tag{15.1}$$

Here t_{onset} is the measured interval from the last valid renewal to the physical onset of powered deceleration, $d_{\text{F1}}(v)$ bounds the measured travel from that renewal to rest, $d_{\text{suffix}}(v)$ is the jerk-limited resident stop profile from equation 11.17, and ϵ_{track} is the tracking inset. The design budget's pre-brake subtotal τ_{delay} begins earlier, at raw sensor capture, and its sum also carries the localization bound and the protective margin. Notebook 15.1 evaluates both sums at the rack end.

DRAFT

Napkin Math 15.1: The reference-event gap

Problem: A proposer stall at the aisle speed is scored from the last valid renewal. What onset and travel bounds follow from the lease path and the resident stop, and how far below the capture-referenced subtotal of the design budget do they sit?

Variables:

- $v = 1.3$ m/s (Chosen) and $D_{\text{clear}} = 1.10$ m (Illustrative; table 1.2).
- $T_{\text{lease}} = 60$ ms and $T_{\text{tick}} = 1$ ms (Chosen); $T_{\text{bus}} = 1$ ms and $T_{\text{act}} = 20$ ms (Illustrative).
- $a_{\text{brake}} = 2$ m/s^2 (Illustrative; stands in for a measured loaded deceleration) and $j_{\max} = 7.11$ m/s^3 (Derived; the reference stop polynomial's peak jerk).
- $\epsilon_{\text{track}} = 40$ mm and $\delta_{\text{loc}} = 50$ mm (Illustrative); $\delta_{\text{margin}} = 100$ mm (Chosen); evidence age $t_{\text{age}} = 51.6$ ms (Derived; section 8.4.1).

Math:

Onset bound.

$$t_{\text{onset}} \le T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}} = 60 + 1 + 1 + 20 = 82 \text{ ms}$$

Resident stop (equation 11.17).

$$d_{\text{suffix}}(v) = \frac{v^2}{2a_{\text{brake}}} + \frac{v\,a_{\text{brake}}}{2j_{\max}} = \frac{1.3^2}{2 \times 2} + \frac{1.3 \times 2}{2 \times 7.11} = 0.4225 + 0.1828 = 0.6053 \text{ m}$$

F1 oracle from the last valid renewal (equation 15.1).

$$d_{\text{F1}}(v) = 1.3 \times 82 + 605.3 + 40 = 106.6 + 605.3 + 40 = 751.9 \text{ mm}$$

Capture-referenced subtotal, which leaves 131.0 mm of the clear distance.

$$v\,\tau_{\text{delay}} + d_{\text{suffix}} + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} = 1.3 \times 133.6 + 605.3 + 50 + 100 + 40 = 969.0 \text{ mm}$$

The gap between them.

$$969.0 - 751.9 = 1.3 \times 51.6 + 50 + 100 = 67.1 + 150 = 217.1 \text{ mm}$$

Sensitivity. In the optimistic all-weight screen at the restricted point on the inspected floor, friction caps the deceleration at 1.18 m/s²:

$$d_{\mathrm{F1}}(1) = 1 \times 82 + \left(\frac{1^2}{2 \times 1.177} + \frac{1 \times 1.177}{2 \times 7.11} \right) \mathrm{m} + 40 = 82 + 507.6 + 40 = 629.6 \text{ mm}$$

A speed reduction of 23.1 percent shortens the oracle distance by only 16.3 percent, because the diminished traction cancels out the majority of the kinematic savings.

Result: F1 passes only if powered deceleration begins within 82 ms and the base travels at most 751.9 mm from the last valid renewal, 217.1 mm less than the capture-referenced subtotal.

Systems insight: Both formulations share the identical deceleration trajectory $d_{\text{suffix}}(v)$; they diverge entirely on their reference event. The design subtotal incorporates 67.1 mm of evidence-age travel alongside the localization bound and protective margin that protect a detected target. However, a stalled proposer produces no target. If the verification harness evaluated an F1 proposer stall against this generous capture-referenced subtotal, the distance predicate would tolerate a braking onset roughly 167 ms late, more than double the entire onset deadline. Verifying the invariant $d_{\text{stop}} \leq D_{\text{clear}}$ remains sound only when travel is measured strictly from the physical or temporal event that the fault disrupts.

The partial budget beneath both bounds reproduces the arithmetic of the design record, serving as a conditional oracle specification rather than empirical timing closure. A physical trial must directly instrument and log successful admission timestamps, continuing evidence age, and physical deceleration onset. Nominal proposer loop cadences cannot guarantee that lease renewals succeed before the deadline expires. Because a safety lease begins strictly upon successful admission, the renewal verification campaign must qualify the maximum inter-admission interval demanded by the lease rule (equation 4.4) under realistic compute contention, bus saturation, and rejected proposals, rather than measuring only unvetted proposer dispatch timestamps.

Both oracle bounds also assume that the safety permission pipeline, required perception channels, and powered traction drives survive the injected fault. Capacitive hold-up for logic rails and brake coils cannot sustain the traction drives during heavy braking. The power-loss fallback (F4b) must therefore maintain and monitor a verified auxiliary power reserve and dynamic braking dissipation path throughout deceleration. If an injected fault disables these electrical reserves, the system drops into an unpowered fallback mode whose stopping envelope must be characterized independently. Neither oracle bound relies on corrective action from either learned proposer. When the chunk policy stalls, it cannot delay braking onset: the hardware safety monitor issues the stop when the lease timer expires, with physical onset following the allocated tick, bus, and actuator path, cleanly isolating safety enforcement from proposer latency. The F1 qualification test thus benchmarks the deterministic safety permission path rather than the learned policy. Finally, this stopping budget assumes a static obstacle located at the clearance boundary. Static verification trials do not validate protection against dynamic agents moving toward the machine; site crossing restrictions and occluded-motion tracking must cover that case. Declaring these qualification points alongside both formal bounds provides the complete specification required to execute an empirical verification trial.

↩ Tracking a moving body through occlusion, which this static-obstacle budget does not cover, is established in section 9.7.

15.3 One F1 Trial from Injection to Record

The campaign's first trial is F1, a proposer failure during transit, planned at the candidate restricted envelope point of 1 m/s with the full declared payload on a test floor qualified to the inspected friction. The numerical oracle remains a screening specification until loaded braking qualification supplies the actual deceleration bound. The safety claim under test is conditional. Once trajectory proposal renewals cease, the low-level permission path and motor drives must autonomously execute the resident powered stop within their allocated stopping envelope. The test deliberately halts or freezes the chunk policy immediately following its last valid lease renewal while leaving its underlying timer heartbeat active. This fault pattern cleanly separates true algorithmic progress in the proposer from the superficial liveness of an unblocked timer interrupt.

Verifying this stopping claim in its entirety requires a full physical rig: the production mobile base, motor drives, representative payload mass, and a qualified test surface. A processor-in-the-loop (PIL) bench, which runs the enforcer's compiled binary on its target microcontroller against a simulated plant, can validate lease expiration but not the plant's response, and a pure software simulation approximates both the target's execution timing and the contact mechanics. Neither bench environment alone provides empirical ground truth for physical stopping distance. Because the machine operates at the edge of its operational envelope during fault injection, an external physical containment fixture must guard the testing volume independently of whatever safety action the machine itself takes.

The temporal reference event for F1 is the last successful admission check that renewed the trajectory lease, not the upstream planner dispatch or the subsequent fault injection trigger. A complete diagnostic trace captures both the final renewal and the injection event across isolated hardware channels that independently record lease expiration, the permission path's stop command, the physical onset of powered deceleration, and external tracking of base position through complete standstill. Calibrated motor-current or torque measurements alongside wheel encoders identify drive effort and wheel response; external base-motion tracking is still needed to establish chassis deceleration under possible wheel slip. Timing uncertainty belongs in the onset margin. An auxiliary contact switch on the mechanical brake verifies subsequent holding-brake engagement. Relying on the mechanical brake switch alone would mischaracterize stopping latency, as it cannot record the initial phase of regenerative or active electric braking.

Instrumentation must capture the machine's state trajectory, fault detector identity, refusal timestamp, and recovery time to a safe state, and it must be electrically and logically decoupled from the robot. Examples are external optical ground-truth cameras and optical shaft encoders routed to isolated digitizers. An onboard logging daemon cannot serve, because the fault under evaluation, whether a panicked kernel or a saturated interconnect bus, delays or drops the very log it would write.[2] Independent external measurement helps expose false passes, where an anomalous run appears successful for reasons outside the formal safety claim. For example, injecting a dropped-frame fault might unintentionally trigger an unmonitored thermal breaker or an auxiliary ground loop that cuts drive power. If that happens, the spring-applied mechanical brake halts the base well within the distance budget, even while the software safety enforcer remains deadlocked in a priority inversion. A naive verification oracle that checks only final stopping distance would mark that trial as a success. A rigorous test harness must verify that the exact causal enforcer path specified in the safety claim commanded and governed the deceleration.

Superficial liveness signals are a second source of false passes. Section 4.4.1 required that a watchdog be refreshed only when a safety-critical cycle completes, never from an independent periodic interrupt. That section grounded the rule in the Toyota unintended-acceleration litigation, where expert testimony argued that a timer interrupt could keep a hardware watchdog satisfied after the control tasks had stopped (Barr 2013). A design rule on paper does not show that the built machine obeys it, and F1 is the test of whether it does. The harness halts the chunk policy's task or freezes its command output while keeping the periodic interrupt handler alive (perspective 15.1), and it measures the actuator response and watchdog state with external instruments. A host daemon can keep reporting a valid process heartbeat after the policy inference thread has frozen, so trial F1 passes only when physical deceleration onset is recorded at the motor drives, never on the strength of a software heartbeat logged in memory. The Toyota record adds a second question that the design rule alone does not raise, whether the supervising monitor shares an input with the processor it supervises (war story 15.1).

[2] **Probe Effect**: Gait (1986) identified the probe effect in concurrent software systems, where the overhead of monitoring instrumentation alters execution timing, inadvertently masking or introducing race conditions. On-chip hardware trace units, such as Arm's CoreSight Embedded Trace Macrocell, stream cycle-accurate instruction flows over dedicated trace pins, while external protocol analyzers monitor bus transactions without altering bus arbitration. A verification trial must time the enforcer's reaction using non-intrusive hardware instruments rather than software logging daemons executing on the processor under test.

Systems Perspective 15.1: The watchdog liveness invariant

A watchdog serviced by a periodic timer interrupt proves only that the timer interrupt service routine executed, not that safety-critical application tasks met their deadlines, computed valid outputs, or avoided memory corruption. A qualified fault-injection campaign explicitly decouples application progress from interrupt liveness, verifying that the system initiates physical shutdown under task termination, thread starvation, and frozen control registers.

War Story 15.1: Testing a task-blind watchdog hypothesis

Context: In *Bookout v. Toyota* (Oklahoma County, 2013), a jury heard expert testimony on the electronic throttle control system (ETCS-i) of a 2005 Camry. The plaintiffs' software expert argued that safety-critical control tasks could fail silently while supervisory mechanisms remained blind to the failure (Barr 2013). Section 4.4.1 drew its watchdog rule from this testimony; the same record also bears on a second claim, the independence of the monitor that backs the watchdog.

Mechanism: The engine control module serviced its hardware watchdog from a recurring timer-tick interrupt rather than requiring verified progress from active control tasks. The expert testified that if the task managing throttle angle and critical fail-safes terminated due to memory corruption or a stack overflow, the throttle command could freeze at its last valid state while the timer interrupt continued to pacify the watchdog (Barr 2013). The architecture included a secondary monitor processor, but testimony highlighted that this monitor could not inspect task scheduling; it could detect a dead throttle task only indirectly, through changes in brake-pedal state. A single analog-to-digital converter also digitized accelerator-pedal position for both processors, placing a common-cause failure point across the supervision boundary (Dunn 2013). Recovery could therefore require the counterintuitive act of lifting a foot off the brake during the event.

Impact: On October 24, 2013, the jury awarded $3 million in compensatory damages and found that Toyota had acted with reckless disregard; the parties settled before the jury could consider punitive damages (Safety Research & Strategies 2013). The verdict did not establish an uncontested technical cause. NASA's engineering review for the federal investigation found no evidence that an electronic malfunction caused large unintended accelerations (NASA Engineering and Safety Center 2011). It did document electronic faults, among them a tin whisker that formed a 248 Ω short within an accelerator-pedal position sensor from a 2003 Camry. Vehicle tests confirmed that while this fault altered how the throttle tracked the pedal, the throttle still returned to idle whenever the pedal was released (Leidecker et al. 2011). The investigation attributed the explainable incidents to floor-mat entrapment, sticking pedals, and pedal misapplication. The task-blind mechanism therefore remains an attributed hypothesis, which is exactly the kind of claim only injection can settle.

Response: For the design under test, fault injection must establish which application progress checks reach the watchdog, verify whether the independent monitor shares any input pathway with the processor it supervises (section 15.6.2), and confirm which protective inputs can reliably command a physically suitable stop. The historical controversy highlights the danger, but only direct injection testing establishes these properties for the warehouse machine.

Systems lesson: A periodic watchdog heartbeat confirms only that an interrupt handler continues to cycle, not that application tasks are functioning. A monitor is independent only if no input, supply, or clock it depends on can fail together with the processor it supervises. Verification must therefore terminate the supervised task and inject faults at those shared points, measuring the actuator response with external, isolated instruments.

Safety intervention mechanisms must be evaluated under the exact operational stress that makes intervention necessary. A safety enforcer that takes over cleanly on an idle development bench offers little assurance for a production vehicle. In an operational emergency, the policy model operates at maximum inference throughput, memory buses are saturated with high-resolution sensor streams, DMA controllers move camera frame buffers, and the mobile base carries its loaded rack at aisle speed. The verification framework therefore injects faults under declared worst-case thermal and electrical loads, measuring both lease-expiration revocation latency and the elapsed time between diagnostic fault assertion and powered deceleration onset at the motor drives. If the intervention path shares a system bus or memory arbiter with the neural accelerator, safety takeover can stall behind an unpreemptible burst transfer. A safety case that assumes near-instantaneous intervention without a contention bound can credit a response the deployed hardware cannot deliver.

↩ How a small urgent read waits behind a large burst transfer on a shared memory interconnect is established in section 13.3.

A verification campaign executes every injection trial under a deterministic, automated protocol, ensuring that verdicts derive strictly from repeatable physical measurements rather than post hoc interpretation. The test orchestrator first audits all operating preconditions (ambient temperature, supply rail voltages, baseline wheel speed) against declared engineering tolerances; any trial starting out of bounds aborts with a verdict of `INVALID`, which charges the failure to the fixture rather than the machine, before fault injection begins. Once preconditions hold, the orchestrator synchronizes the external plant fixture, logic analyzer, and independent ground-truth digitizers to a common reference clock, while recording the target clocks without disciplining away their injected drift. After allowing the mobile base to settle at its envelope operating point, the orchestrator asserts the fault trigger across a dedicated hardware line, registering the fault injection timestamp on the unified timeline. The isolated logic analyzer captures both this injection mark and the formal reference event (the final valid lease renewal in trial F1), so fault detection latency, drive takeover duration, and total stopping time are each scored from their true temporal origins. Concurrently, external truth sensors tied to a fail-safe crowbar relay circuit[3] stand ready to initiate the test cell's qualified containment response within a measured response bound if the machine approaches its survival-envelope limit. A precommitted evaluation oracle then scores the synchronized trace against the pass/fail criteria defined in section 15.4. Runs with corrupted or dropped external measurement frames are flagged transparently as `UNOBSERVABLE`, the verdict for a trial whose scoring telemetry was lost (section 15.4.1 sets out all four verdicts). Finally, the test orchestrator cryptographically signs the complete trial record header and seals it into the safety evidence bundle via a cryptographic digest.

[3] **Crowbar**: The name comes from the blunt industrial act of dropping a metal bar across power supply terminals. A crowbar circuit intentionally shorts a supply rail when an overvoltage occurs, forcing an upstream fuse or breaker to trip before hazardous energy reaches sensitive loads. Within a physical fault-injection test cell, this containment concept protects physical safety at the actuator stage: independent ground-truth sensors detecting an envelope violation trigger a hardware disconnect that severs actuator drive power over a dedicated electrical pathway that the vehicle under test can neither power nor override. Physical containment therefore never depends on the correctness of the very enforcer undergoing evaluation.

↪ **Protocol:** The fault-injection protocol, stage by stage with its error codes, is given in section C.15.9 for a reader building the test orchestrator.

15.4 Fault Oracles and Acceptance

A valid verification trace for the F1 safety claim must establish both timely deceleration onset and bounded stopping distance; observing that the robot base eventually stopped establishes neither guarantee. Without an oracle fixed in advance, post-hoc inspection takes over: engineers eyeball time-series plots after the run, seizing on gross symptoms such as whether Joule heating melted a motor phase wire or a controller threw an unhandled exception. The trial degenerates into an anecdote.

Definition 15.2: Test oracle

Test oracle is the predicate, fixed in full before a trial runs, that compares each independently witnessed quantity of the trace with its bound from a declared reference event and returns the trial's verdict.

1. **Significance**: For F1, the oracle comprises two predicates evaluated from the last valid renewal: powered deceleration onset within 82 ms and total travel within 751.9 mm at aisle speed. Each predicate evaluates to a signed margin in its native physical unit. Retaining signed margins rather than binary passes enables near misses to inform subsequent trials and guides adversarial search algorithms toward razor-thin safety margins.
2. **Distinction**: A pass criterion extracted from telemetry after a run is chosen with the outcome already visible; an oracle is mathematically precommitted. The oracle also differs from the design budget from which it is derived: the 969.0 mm design subtotal starts at sensor capture and protects a planned target trajectory, whereas a stalled proposer produces no planned trajectory to protect.
3. **Common pitfall**: A persistent misconception in physical verification is that a robot base stopping short of a rack signifies a passed trial. In reality, stopping within the 1.10 m clearance distance can easily conceal a safety controller that missed its deceleration deadline, or a stop triggered by an unmodeled side effect, such as a tripped thermal breaker that cut drive-bus power, rather than the deterministic safety path named in the claim.

15.4.1 Four verdicts and the invalid-test rule

If traction control derates motor torque over an uneven floor seam and slows the mobile base before fault injection, the robot encounters the fault below its declared operating speed. Conversely, if

a hardware frame injector overflows its queue and drops an adversarial CAN message before it reaches the transceiver, the robot never encounters the fault at all. Scoring either aborted run as a pass manufactures artificial confidence, while scoring it as a system failure blames the vehicle for a defective test harness. Rigorous verification therefore enforces an **invalid-test rule** that separates testbed anomalies from the machine's own performance.

Under that rule, the oracle returns one of four explicit verdicts. A `PASS` indicates that every precommitted timing and physical predicate held under verified preconditions. A `FAIL` indicates that the system violated one or more measured predicates. An `INVALID` verdict indicates that the fixture failed to establish declared test preconditions or the fault injection itself miscarried. An `UNOBSERVABLE` verdict indicates that ground-truth telemetry required to score the trial was corrupted or lost, while the partial trace is retained for diagnosis. Under an `INVALID` or `UNOBSERVABLE` verdict, any unmeasured quantity is logged as unavailable rather than set to zero. Neither verdict may ever be converted into a `PASS`, and neither is ever omitted from aggregate coverage reports. An `UNOBSERVABLE` trial supplies no verdict on the unmeasured claim; its missing evidence requires investigating both the witness system and the injected fault, rather than assuming the harness alone failed. Silently discarding runs with dropped telemetry or treating unobservable outcomes as passes introduces survivorship bias. If an inductive kick from an emergency brake solenoid emits an electromagnetic interference burst that crashes the data acquisition (DAQ) logger, discarding that run conceals the exact cyber-physical failure under test.

Precommitment separates empirical evidence from after-the-fact excuses. In physical fault injection, cyber-physical interactions are noisy and counterintuitive, so criteria negotiated after inspecting the trace inevitably drift to accommodate anomalous behavior. An unexpected overrun is easily dismissed as floor variation, or a deceleration onset that exceeds the 82 ms bound is excused because the vehicle came to rest before hitting the rack.

A precommitted record prevents that drift by locking down every physical predicate, measurement unit, normalization scale, response-time bound, and invalidation trigger before the first fault is triggered. The evaluation engine computes and logs a signed margin for each predicate, issuing a `PASS` verdict only when the declared preconditions and injection are verified, all required predicates are observable, and every timing bound and physical margin remains non-negative. If stopping distance exceeds its limit by five millimeters or powered braking begins 2 ms past its deadline, the record pinpoints the exact predicate that failed so engineers can trace whether the cause was degraded tire-floor friction, mechanical brake lag, motor thermal derating, or software diagnostic latency. Precommitment enforces the fourth law, permission must be earned (principle 4), at the test boundary. Production deployment grants operational authority on the basis of these records, so any verdict scored under rules relaxed after the fact would grant authority on the accident of an outcome rather than the guarantee of a verified machine.

15.4.2 The F1 record and its missing coverage

A stopping trace separated from its precondition checks or oracle digest cannot prove which threshold it was scored against or whether the floor and payload matched the claim. A **fault record** prevents that separation by cryptographically binding a finalized metadata header to the independently retained sensorimotor trace. Identity fields in the header specify the trial identifier and safety claim, the physical envelope point, the fault injection parameters (mode, magnitude, and target subsystem), and a cryptographic digest of the precommitted oracle. Outcome fields record the measured detection and takeover latencies, the signed physical margins, the four-state verdict, a digest of the complete evidence bundle, and an asymmetric digital signature over the entire header, while the high-rate telemetry trace is archived inside the bundle.

The planned F1 record at the candidate restricted envelope point, 1 m/s on an inspected floor with a 50 kg payload, fills these fields (table 15.1). As with every proposer stall, the stopping distance oracle is anchored to the timestamp of the last valid lease renewal. Under the optimistic all-weight screen on a floor with friction coefficient $\mu = 0.12$, equation 15.1 yields a stopping bound of 629.6 mm (corresponding to the sensitivity boundary in notebook 15.1). This distance is strictly shorter than the 831.2 mm capture-referenced budget subtotal at that speed. The capture-referenced subtotal accounts for upstream sensor evidence age and fixed geometric margins while omitting dispatch-to-admission communication delay; consequently, it cannot be substituted for the renewal-

referenced oracle of F1 or an end-to-end appearance-referenced bound. Of the three bounds, only the renewal-referenced one contains the complete timing path; its braking premise still awaits loaded qualification. The capture-referenced subtotal has a value but omits the dispatch-to-admission interval. The appearance-referenced bound has no value at all, because it still lacks a witnessed appearance-to-capture interval as well as a measured dispatch-to-admission interval (figure 15.1).

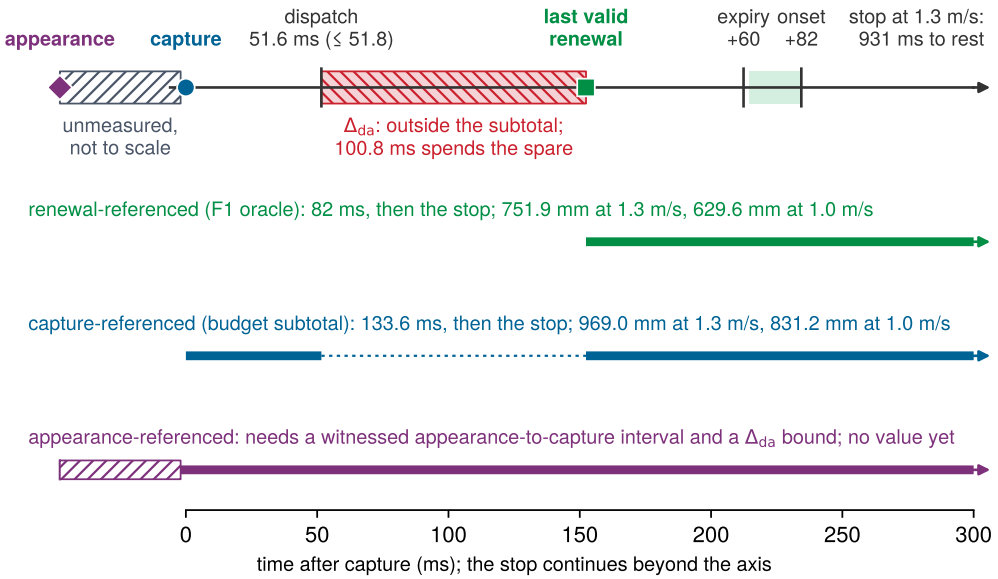


Figure 15.1: Three reference events, three bounds: One stop on a time axis from capture, with the dispatch-to-admission interval drawn at the 100.8 ms that would spend the spare. That ceiling, not the 63.3 ms allocation, is drawn because the interval is still unmeasured and the ceiling is the longest value any trial could accept. The renewal-referenced F1 oracle counts only the 82 ms lease path and the stop. The capture-referenced subtotal adds the 51.6 ms evidence age but not the dispatch-to-admission interval. An appearance-referenced bound would need both that interval and the unwitnessed appearance-to-capture interval, and no value for it exists yet.

In the planned record (table 15.1), the outcome field contains no numerical value and is marked unavailable. Because no physical trials have been executed in this book, the field specifies the physical measurement that will eventually populate it; until empirical data are recorded and verified by independent witnesses, the record remains uncommitted and carries no verdict. The trial count in that field is a design choice of its own. The number of passing trials needed to bound the miss probability follows the zero-failure count of section 7.7.1, which section 15.10 applies to an injected fault population.

Table 15.1: One F1 fault record at the candidate restricted envelope point: The header fields of a proposer-stall trial on the warehouse mobile manipulator, with the screening oracle awaiting loaded braking qualification before the run and the outcome left as the measurement that must fill it.

Trial and claim	Envelope point	Oracle, predeclared from the last valid renewal	Detector and witness	Outcome to be measured	Verdict
F1, proposer stall: chunk-policy task killed one tick after a renewal; rack-end stopping claim	1 m/s; 50 kg payload; test floor measured at $\mu = 0.12$; drive bus at 47.7 V nominal	Powered deceleration onset $\leq$ 82 ms; travel to rest $\leq$ 629.6 mm	Lease timer on the permission path; expiry, revocation, and onset each on its own wire and clock	Trial count; maximum onset and maximum travel over the trials, each with its signed margin; unavailable until measured	None until scored; `PASS` requires verified preconditions, required witnesses, and both nonnegative margins

↪ **Byte layout:** The record header, field by field and including how an unavailable value is encoded, is given in section C.15.8 for a reader building the record.

The record schema maintains strict forensic separation among verdicts on a per-field basis. If an upstream breaker trips and prevents an injection, the harness marks the run `INVALID` and records the detection latency, takeover delay, and stopping margin as unavailable rather than coercing them to zero. Because the evidence bundle attaches an explicit validity flag and missing-data reason code to each measurement, the scoring oracle can inspect those flags before running any statistical or arithmetic evaluation, keeping unmeasured fields from corrupting pass-rate calculations, latency percentiles, or margin statistics.

A single aggregate percentage conceals where the verification campaign never ventured. A system that survives 98 percent of five thousand fault injection trials demonstrates little if every trial ran at nominal room temperature under negligible memory bus contention. Comprehensive verification requires reporting coverage across orthogonal dimensions: functional safety claims, physical envelope extremes, physical and logical injection sites, timing distribution tails, hardware build revisions, and concurrent multi-fault scenarios. For the warehouse mobile manipulator, physical envelope coverage verifies whether faults were injected under both empty and fully loaded rack states, at nominal aisle speed on dry concrete, at restricted speed on inspected surfaces, and under depleted DC drive-bus voltages. Timing tail coverage isolates injections scheduled to coincide with p99 operating system scheduling jitter in the permission loop or saturated DMA bus traffic. System verification cannot be declared complete when an aggregate coverage score exceeds a target percentage; rather, the minimum coverage across every individual dimension must satisfy the safety case requirements.[4]

Repeated trials provide statistical evidence only when underlying physical or digital states vary. Executing a deterministic fault script five hundred times on a digital bench with identical initial register states, fixed clock dividers, and synchronized inputs traverses the exact same execution path on every iteration; it contributes no more statistical information than a single run. Conversely, on a physical test aisle, repetition samples real-world stochasticity: micro-variations in wheel-floor contact friction, asynchronous hardware interrupt arrivals, bus arbitration jitter, and phase drift between the camera exposure clock and the policy inference cycle. In that setting, repeated trials characterize the variance and map the dangerous tails of the end-to-end response-time distribution.

[4] **Scope of assurance standards**: ISO 26262-5 addresses road-vehicle electrical and electronic hardware. Its single-point and latent fault metrics concern the architecture and its allocated safety goals; they are not the failure-rate-weighted aggregate coverage against which notebook 15.3 later sizes an injection campaign. Fault injection is one way to test diagnostic assumptions, alongside analysis and other evidence. ANSI/UL 4600 frames an autonomous-vehicle safety case from multiple kinds of evidence. Neither standard makes a fixed number of hardware-in-the-loop (HIL) trials sufficient for release.

Checkpoint 15.1: Oracles, verdicts, and reference events

An F1 result carries meaning only through the oracle, reference event, and verdict rule fixed before the run.

Scoring a trial

- ☐ **Verdict assignment**: Can you score three trials, one in which the injector dropped the fault packet before it reached the transceiver, one in which the logger crashed while the base stopped short of the rack, and one in which every predicate held on independent witnesses, and name the record fields each trial leaves unavailable?

- ☐ **Reference event**: Can you explain why the F1 distance oracle starts at the last valid renewal and leaves out the evidence age and the two margins that the capture-referenced subtotal carries?
- ☐ **Repetition vs. variation**: Can you explain why five hundred runs of a deterministic bench script count as one trial, while repeated runs on the test aisle sample a distribution?

Estimating a margin

- ☐ **Onset estimate**: A trial at 1.3 m/s records powered deceleration onset 90 ms after the last valid renewal, with the lease expiring and the stop command issuing on time. Can you estimate the onset margin, the extra travel before braking begins, and whether the trial can pass even though the base stops well short of the rack?

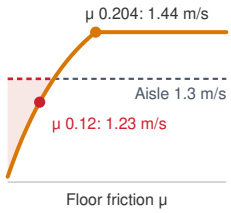
15.5 Extending the Trial to the Envelope

The F1 record fixes one trial at one envelope point, and even a clean pass there says nothing about the points around it. The operating-envelope definition already warned that the danger lies at coupled corners, where two parameters that are each within their declared ranges combine to break the claim. Extending the trial therefore means choosing which further points to test, and that choice follows from how the machine's physics couples its variables.

15.5.1 Coupled envelope corners versus one-variable sweeps

Sweeping travel speed alone on dry concrete and floor friction alone at a crawl passes both checks while missing the corner where aisle speed meets an inspected floor. Because physical mechanics, compute pipeline latency, and electrical power distribution interact at run time, independent minimum and maximum sweeps for individual variables leave the dangerous corners of the operating envelope untested:

- **Surface friction and travel speed**: The 1.3 m/s aisle speed and an inspected floor friction of 0.12 each fall well within their declared individual operating ranges. In combination, however, they fail even the optimistic all-weight traction screen. On low-friction surfaces, the tire-floor interface, not mechanical braking capacity, sets an upper limit on deceleration; in the all-weight screen it falls to 1.18 m/s^2. Consequently, the resident stopping maneuver elongates: the required stopping distance at 1.3 m/s expands to 1.19 m, exceeding the available 1.10 m clearance. The optimistic screening ceiling on that floor drops to 1.23 m/s, and the actual speed limit must use the qualified loaded lower bound rather than assuming this ceiling is attainable.
- **Contact nonlinearities and dynamic load transfer**: Real warehouse floors rarely have a constant friction coefficient μ. Instead, surface friction varies dynamically based on local contamination (such as concrete dust, ambient moisture, or oil films), wheel tread heating, and transient wheel loading. On the running machine only the two driven hub wheels brake against the floor, so the traction ceiling is $\mu g \phi_{\text{drive}}$, where ϕ_{drive} is the share of the weight those wheels carry (derived in the free-body diagram of chapter 2, figure 2.1). During hard braking, the inertial load ma_{brake} acting at the center of gravity pitches the body forward and can move normal force off the driven wheels onto the front caster, lowering ϕ_{drive} exactly when traction is needed. If the drives demand braking torque sized for the static load share, the driven wheels slip and the net braking force collapses. Under a constant-friction approximation, the braking distance then grows as $v^2/(2\mu g \phi_{\text{drive}})$, inversely with the friction actually delivered at the driven wheels and quadratically with speed.
- **Computational and electrical coupling**: Memory and power subsystems couple directly into safety margins. A compute-heavy forward pass of the vision-language-action intent model executing on the application SoC can saturate unified memory bandwidth. This contention delays camera-buffer processing and postpones the compact evidence handoff to the safety microcontroller, inflating the observation age and, with it, the pre-brake delay τ_{delay}. Because stopping distance scales linearly with total reaction delay (equation 2.2), a 30 ms memory stall



The 1.3 m/s aisle speed clears the dry-floor ceiling but not the inspected floor's.

consumes 39 mm of physical clearance at aisle speed. On the electrical bus, the dominant transient coupling originates from the motor drive actuators rather than the neural compute accelerator. At its maximum 60 W draw, the SoC pulls approximately 1.6 A from a depleted battery bus, only 2.2 percent of the overall 70 A transient, whereas the two drive motors demand up to 50 A during peak torque transients. Severe acceleration can sag the common DC bus across battery internal resistance and wiring harness impedance, while bursty inference activity primarily stresses the SoC's local point-of-load regulators, a localized transient that the F4 campaign must isolate and measure separately.

- **Payload mass and kinematic posture**: Arm posture and payload distribution couple directly to stopping performance through vehicle stability and load transfer. Under braking, the base tips about its front caster whenever deceleration exceeds $g\, d_{\text{lead}}/h_{\text{cg}}$, where h_{cg} is the height of the center of gravity and d_{lead} its distance behind that caster. This is the tip-over ceiling computed in section 2.4.2, which the credible deceleration respects only while $h_{\text{cg}}/d_{\text{lead}}$ stays below the ratio computed there. Raising the arm above a storage rack raises h_{cg}, and reaching forward shortens d_{lead}, so posture moves the machine toward that ceiling. The tip-over threshold can bind before tire traction saturates, and ISO 3691-4 mandates physical stability verification for driverless industrial trucks (International Organization for Standardization 2023b). Unloaded trials support only their tested configurations; they cannot establish the fully loaded deceleration or the complete operating envelope. The braking-limit check of section 2.3.1 left both lengths unmeasured, so the envelope cannot yet certify that the demanded deceleration stays below the tipping threshold; a tilt-table test or multi-axis corner-scale measurement of the fully loaded machine must supply them.
- **Thermal throttling and fallback execution**: Sustained thermal stress can induce dynamic processor throttling, delaying action proposal generation and degrading mission throughput. In an architecture protected by an independent, verified resident stop, a missed watchdog renewal safely triggers a bounded, fail-safe stopping trajectory rather than indefinitely expanding stopping delay. Nevertheless, multi-axis compound verification must ensure that simultaneous bus contention, sensor packet drops, or voltage droops do not compromise that fallback routine itself.

15.5.2 Physical disturbances versus digital input perturbations

A vision model that survives synthetic pixel noise injected into an input tensor can still go blind when low-angle warehouse glare saturates the physical camera lens and its auto-exposure loop. Digital perturbations and physical disturbances challenge an embodied machine at different interfaces. Synthetic tensor noise tests how sensitive a network's decision boundary is to small numerical shifts in representation space while bypassing the physical sensor altogether. An optical disturbance, by contrast, exercises the lens, coatings, auto-exposure and gain control, demosaicing, and camera interface timing. Neither test substitutes for the other, so a threat model must state whether an injected fault corrupts digital packets in flight, perturbs the physical environment, or hits both at once.

Evaluating the rack-end safety claim requires injecting realistic physical disturbances: stray illumination and direct glare that blind camera sensors, mechanical chassis vibration that corrupts inertial measurements, and floor transitions that slash available braking traction. Every physical injection requires a repeatable physical actuator and an independent ground-truth witness. For instance, glare injection requires calibrated illumination measurements beside the lens together with the declared optical geometry, spectrum, and temporal waveform. Chassis vibration requires external optical motion capture independent of onboard wheel odometry. Floor variations require standardized friction-measurement drag sleds. In cyber-physical verification, receiving a syntactically valid sensor packet with correct checksums over an internal bus confirms only that the digital transmission succeeded; it provides no guarantee that the estimated state accurately reflects physical reality.

15.5.3 What the envelope claim leaves open

Passing the initial F1 trial at one speed and arm posture still leaves neighboring speeds, higher centers of gravity, and worst-case injection timings untested. That first diagnostic run suppresses

trajectory renewal while the base is in motion and measures the deceleration-onset timestamp and stopping distance. Subsequent falsification campaigns systematically vary initial velocities, arm configurations, and injection phases to hunt for minimal operating margins (section 15.9.2). Formal methods, high-fidelity simulation, and physical hardware testing provide complementary evidence, but none can compensate for an unstated physical premise or an unverified operating assumption.

An operating boundary remains defensible only when every parameter carries its physical unit, traceable provenance, calibrated uncertainty, and designated engineering owner. Specifying a credible deceleration of 2 m/s^2 remains an incomplete assertion until the verification evidence documents its origin. The evidence must show whether it was measured during physical stopping trials at maximum payload mass, extracted from an actuator manufacturer's datasheet, or computed using a theoretical friction model. It must also give the measurement uncertainty of the external tracking reference. Any hardware modification, whether a new polyurethane wheel tread compound or new motor-drive firmware pushed across the EtherCAT fieldbus, obligates the component owner to re-evaluate whether the operational boundary has shifted.

Validating an operating envelope requires attacking its boundary surfaces rather than testing comfortable interior states. Evaluating an unloaded base crawling across a dry floor with pristine camera frames proves only that nominal execution works under ideal conditions. Instead, the verification harness distributes test points across three regimes. Points *just inside* the envelope verify that the control policy preserves closed-loop stability under the largest permissible operational stress. Points *on the boundary* confirm that safety margins survive nominal sensor noise and compute jitter. Points *just outside* verify that the safety enforcer reliably preempts the policy, executing a deterministic stop that halts the base within the survival envelope before clearance is exhausted.

A physical hardware malfunction and an algorithmic functional insufficiency require different verification strategies. A stereo camera can transmit uncorrupted, on-time video packets over PCIe while ambient glare blinds its depth estimation. A complete campaign must therefore inject faults into both the physical execution mechanisms and the sensory streams on which those mechanisms rely. Adding an enforcer module does not let it observe what its sensors cannot measure; when physical sensing cannot distinguish safe states from hazardous ones, the architect must restrict the operating envelope to conditions where safety can be enforced. Once every timing constraint, latency budget, and environmental threshold of that envelope is specified in physical units, the design claims from earlier chapters become concrete targets for fault injection, each paired with a boundary threshold that must trigger protective intervention.

> Systems Perspective 15.2: The hardware falsification principle
>
> A hardware-dependent safety claim needs an observable challenge to its physical premises. If the campaign cannot inject the relevant disturbance or retain the required witness, the record must preserve that verification gap.

15.6 Deriving the Fault List

The proposer stall fault (F1) originates from negating an operational claim: that proposal renewal proceeds reliably within its granted lease. Deriving a comprehensive fault list, however, requires two complementary methodologies. Inverting design records tests the explicit assumptions codified across the system architecture. Working forward from the physical realities of mechanics, power networks, and human interaction exposes the hazards and unmodeled edge cases that software design records routinely omit. Human interaction includes who is in view. A fault list that seeds only people standing upright leaves untested the kneeling, carrying, and seated people a learned detector may see later, so the groups of people the machine must detect enter the list as a coverage dimension of their own.

15.6.1 Inverting design records and the five-element chain

If the machine's specification claims that the chunk policy renews its trajectory proposal every 50 ms at every speed up to the 1.5 m/s drive limit, the verification suite turns that claim inside out. It injects stalled renewals, delayed observations, and out-of-range encoder readings to see whether

↪ How many people of each group a test campaign needs before its detection estimate can resolve the gap that matters is developed in section 17.5.2.

the permission path catches the violation in time. Every preceding chapter supplies claims that can be inverted in the same way. Chapter 5 through chapter 7 state the learned policy's training distribution, state-action assumptions, and inference latency ceilings. Chapter 8 through chapter 13 state bus allocations, observation freshness, and estimation bounds. Chapter 14 states how quickly authority leaves a failing policy and which subsystem takes over.

Where the five-part test contract of section 15.1 states what makes any trial valid evidence, a **five-element chain** states how a single inverted claim is built into such a trial. Its first element supplies the contract's bounded claim, its last element supplies the contract's oracle, and the three between them specify the mechanism the trial exercises:

1. **The negated condition:** the exact architectural or timing invariant violated (for example, proposal renewal halted after lease issue);
2. **The physical injection point:** the exact hardware or bus interface where the disturbance enters (for example, an inline field-programmable gate array (FPGA) serializer, CAN FD disturbance node, or programmable DC load);
3. **The expected detector:** the deterministic monitoring primitive tasked with catching the violation (for example, a lease countdown timer or evidence-epoch comparator);
4. **The bounded response:** the deterministic transition executed by the enforcer (for example, revoking proposal authority and commanding the resident stop suffix); and
5. **The observable pass condition:** the quantitative, independently witnessed metric that scores the trial (for F1 at aisle speed, onset $\leq$ 82 ms and travel from renewal $\leq$ 751.9 mm).

A recorded design property becomes a testable fault only when all five elements are fully defined and observable. If any element of this chain cannot be defined or measured, the architecture has a **verification gap**: the underlying claim cannot be falsified by empirical test. Systems engineers cannot ignore such gaps; resolving them requires dedicated hardware instrumentation (such as external optical trackers or non-intrusive trace macros), a narrower operational design domain, or a structural redesign of the execution path.

Inversion also applies to the learned proposer's implicit assumptions, where distribution shift can induce functional insufficiency even when every observation frame arrives intact and on time. For example, a chunk policy trained in simulation on dry concrete with friction coefficient 0.60 proposes decelerations that an inspected floor at 0.12 turns into wheel slip. Inverting that evaluation claim requires injecting degraded traction conditions at the physical contact patch. Sensor bias, calibration drift, and input perturbations are injected for the same reason: a policy with high nominal reward can still command high-frequency actuator chatter once its inputs leave the training distribution. Under these conditions, an overly permissive execution path can destabilize the plant while executing syntactically valid commands, demonstrating that software commands and mechanical dynamics remain tightly coupled across the causal boundary.

At each inter-module contract, the fault list negates architectural guarantees directly at their physical handoff points: observation freshness, state validity, intent lifetime, planning deadlines, enforcement independence, and resource allocation. For the 60 ms chunk lease, the test harness replays the expired admitted record 65 ms after its last valid renewal, verifying that the permission path firmly refuses to extend execution authority. A concurrent proposal represents an entirely separate admission attempt: it must satisfy fresh evidence, kinematic state validity, mode consistency, and stopping-reserve checks before receiving a lease. To test the enforcer's claimed independence from the policy processor on the selected discrete placement, the verification suite saturates the application processor's memory interconnect with DMA traffic while independently timing the safety microcontroller. Host contention may delay proposal delivery and age its supporting evidence, but the enforcer must still satisfy its 400 µs execution deadline within every 1 ms control tick. This rigorous inversion prevents arbitrary over-testing: bus contention, clock skew, and supply fluctuations are injected precisely where the architecture claims tolerance, rather than drawing from generic fault menus that impose requirements the machine was never specified to meet.

Inversion has a structural limit: it surfaces only faults rooted in assumptions the design team explicitly wrote down. The resulting fault list is therefore only as comprehensive as the recorded architectural claims. If no engineering record documents how the base's spring-applied brake

behaves when its friction lining glazes under repeated emergency stops, inversion cannot synthesize the failure mode, and the hazard never reaches the test suite.

15.6.2 Forward physical analysis and common-cause modes

Drive wheels skid when commanded deceleration exceeds the traction limit of the contact patch. Brake linings glaze under repeated thermal cycles. Solid tires wear down and alter the rolling radius that wheel odometry relies on. Structural vibration gradually loosens camera mounts until spatial calibration drifts. Because these hazards arise from contact mechanics, heat, and wear rather than software bugs, no software abstraction represents them, and inverting design records will not uncover them. A second, independent fault source therefore works forward from the physical machine (its mechanical body, power distribution network, floor interface, and operating environment). It uses Nancy Leveson's **Systems-Theoretic Accident Model and Processes (STAMP)** framework[5] (Leveson 2011) to trace how physical and control interactions produce hazards even when no software module has crashed.

Consider a motor drive that continues producing positive torque after receiving a stop command. A shorted inverter switch, welded contactor, or stuck gate-driver enable line can defeat the normal powered stopping trajectory. The test harness must independently observe delivered phase torque, rotor velocity, and the engagement of the secondary isolation path. While Safe Torque Off (STO) inhibits torque generation without providing electrical isolation, it cannot enforce nominal stopping distances on low-friction floors or prevent back-driving on gravity-loaded articulated joints. The safety architecture must specify a separately justified containment response alongside an independent physical measurement oracle. If no such path exists, the fault exposes an architectural omission rather than an operational delay that can borrow from the normal stopping budget.

Working forward exposes hardware fault classes that software design records rarely imply: loss of reference clocks, power supply brownouts that leave digital logic in indeterminate voltage states, pulse-width modulation (PWM) inverter switching noise coupling into differential encoder lines, corrupted non-volatile configuration registers, and reset lines that bridge domains intended to remain galvanically or logically isolated. For instance, if the safety microcontroller and the high-performance application processor share a single power-management integrated circuit (PMIC), a transient voltage droop during a sudden drive-current surge can reset both processors simultaneously just as the learned proposer loses trajectory tracking. A software interface model that assumes an ideal DC power supply omits this failure. An enforcer cannot arrest an errant policy if it loses operating power alongside the workload it supervises; the deterministic safety runtime must outlive the faulted workload using independently qualified execution and power resources. The running machine uses an isolated, capacitor-backed auxiliary supply to survive the declared supply fault.

Forward physical analysis likewise evaluates the architecture's common-cause independence claims. Subsystems that appear strictly isolated on an architectural block diagram frequently share physical vulnerabilities. For example, the primary lidar communication lines and redundant safety bumper wiring may share a single mechanical cable carrier subject to pinch wear, depend on a shared analog voltage reference inside a multichannel converter, or rely on timer silicon containing identical silicon errata. An electrical transient that latches up both channels simultaneously destroys any safety argument predicated on statistical independence. **Common-cause analysis** traces harness routing, power distribution paths, and clock distribution trees to identify shared failure paths and justify independence against the declared fault set, rather than assume separation from the block diagram.

This physical analysis extends directly to human error and foreseeable operational misuse. Warehouse personnel may jumper an optical rack-end sensor during maintenance, leave a manual pallet jack parked across an automated transit corridor, or unplug an acoustic warning beacon to silence workplace noise, while remote teleoperators may request transit speeds exceeding local aisle restrictions. When a remote assistant commands the vehicle's full 1.5 m/s drive limit within a narrow aisle, the verification suite must confirm that the arbiter denies the command by capability constraint or clamps it to the admitted manual mode speed under an active `manual_drive` state. Furthermore, the deterministic permission check must evaluate the assistant's command with the same scrutiny applied to autonomous proposals.

[5] **STAMP and STPA**: Leveson (2004) proposed STAMP, which treats safety as a dynamic control problem rather than a component-reliability problem, recognizing that catastrophic accidents often arise from unsafe interactions among components that each operate strictly within specification. Leveson (2011) established the Systems-Theoretic Process Analysis (STPA) hazard-analysis method on this foundation, identifying unsafe control actions at subsystem interfaces, such as an integration seam where bus communication latency or digital filter lag erodes phase margin. Because no individual component has failed at such a seam, its unsafe control actions supply critical fault modes that inverting nominal design records cannot expose.

↩ Safe Torque Off and the industrial stop categories are established in section 12.9.2.

↩ The independence claims for the permission path's execution unit, oscillator, power rail, and memory are established in section 4.7.

Working forward from the physical plant adds a representative mechanical failure mode, the actuator jam, to the record-derived inventory in table 15.2, though full verification still requires a plant-specific hazard analysis tailored to the vehicle's mechanical kinematics and operational envelope. Alongside inverted design claims and forward physical hazards, the fault suite in table 15.2 also evaluates the four authority transition cases (A1–A4) established in section 14.8.2 under the operating state that makes intervention most urgent: a fully loaded base approaching the rack end at velocity. The test campaign injects takeover requests at two speeds to distinguish the candidate moving transition from a request the clearance screen rejects; the operative design stops before handing over. At the 0.75 m/s slow-down, the partial-budget waiting margin is 751.6 ms, enough for the 605 ms in-loop takeover and the 63.3 ms dispatch-to-admission allocation together. Section 14.2.4 therefore identifies a candidate moving in-loop handover for qualification, subject to the complete session budget and live stopping checks. The trial must show that a handover left pending still ends in a local stop by its deadline. At the full 1.3 m/s aisle speed the margin is 100.8 ms, only 37.5 ms once the allocation is charged, which cannot hold the takeover, so the arbiter must reject the moving handover and dispatch a local stop before the deadline expires. An out-of-loop takeover begins only after a stop, from standstill, in an admitted mode. Evaluating either sequence solely while the vehicle is parked proves nothing about dynamic performance; the trial must demonstrate that rejection, timeout, and braking occur with bounded latency while the base moves.

Table 15.2: Representative hardware and cyber-physical fault classes: The rack-end faults F1–F4 and the four authority fault cases (A1–A4) that the authority record hands over, each mapped to its injection point, detector, response, and observable pass criterion on the warehouse mobile manipulator, plus one forward-derived mechanical fault and one attack on the learned proposer.

Failure Domain & Fault Mode	Physical Mechanism & Injection Point	Target Manifestation & Causal Impact	Detection Primitive & Latency Budget	Enforced Safe Response & Bounded Recovery	Observable Pass Criterion
Proposer Stall (F1)	Chunk policy renews its lease once, then its task is killed or its output frozen on the application processor	No new proposals; the last admitted chunk keeps supplying setpoints while the base cruises	Lease expiry on the permission path (60 ms); host heartbeat timeout (30 ms)	Revoke proposal authority; run the resident stop	Powered deceleration onset $\leq$ 82 ms after the last valid renewal; travel from that renewal to rest $\leq$ 751.9 mm
Stale Observation Time (F2)	Navigation-camera frame held at the image signal processor with its capture stamp preserved, or the stamp corrupted, via inline frame interceptor	Proposals whose upper evidence age at dispatch exceeds 51.8 ms, the budgeted age plus the clock-conversion bound; the base acts on an old view of the rack end	Dispatch-age check at admission; continued evidence, state, and lease validity checks on every tick	Refuse stale replacements; stop when the active command loses its own validity	Powered deceleration onset $\leq$ 82 ms after the last valid renewal; travel from that renewal to rest $\leq$ 751.9 mm when the resident stop runs

Failure Domain & Fault Mode	Physical Mechanism & Injection Point	Target Manifestation & Causal Impact	Detection Primitive & Latency Budget	Enforced Safe Response & Bounded Recovery	Observable Pass Criterion
Lost EtherCAT Frames (F3)	Frames dropped or corrupted at a drive's slave port, injected via fieldbus disturbance node	Encoder state and drive commands miss cycles; the delivered command ages by one 1 ms cycle per lost frame	Working-counter and frame-arrival check; drive communication watchdog	Drive enters its configured stop; permission path commands the resident stop	Powered deceleration onset $\leq$ 82 ms after the last valid renewal; travel from that renewal to rest $\leq$ 751.9 mm only when the complete lease-to-onset path remains bounded under frame loss; a drive-watchdog response needs its own reference event, deadline, and stop profile; zero unhandled bus exceptions
Drive-Bus Droop (F4a)	Programmable supply takes the shared drive bus from its depleted point below the converter lockout; the coincident load alone leaves it at 37.8 V	Bus falls below the 33 V converter lockout; the application processor browns out and its sensors may freeze at plausible values	Shared-bus undervoltage flag, raised by the permission path on its isolated, held-up rail	Command STOP; qualify surviving drive authority or record unsupported response	Powered deceleration onset $\leq$ 22 ms after the flag; travel from the flag $\leq$ 673.9 mm at aisle speed if the drives keep their authority on the sagged bus, otherwise the separately characterized unpowered envelope; permission path never resets (continuous heartbeat on an independent digitizer)
Permission-Rail Feed Loss (F4b)	Feed opened during the longest stop in the qualified loaded envelope; the illustrative all-weight inspected-floor screen starts from its 1.23 m/s ceiling	Permission path, drive logic, and brake coils run on the store; powered drive energy remains separately available	Feed-loss flag on the permission rail	Complete the resident stop; set every spring brake at standstill	Stop completes and every brake sets before rail dropout; the all-weight screen spends 1313.43 ms of the 2 s hold-up, awaiting loaded qualification; any reset is `FAIL`
Actuator Jam & Mechanical Stall	Wheel bearing seizure or dragging brake, injected via wheel clamp or dynamometer	Locked rotor; one wheel blocked yaws the base; rapid stator winding heating ($I^2_{\text{phase}}R$)	Motor phase overcurrent & Hall-effect velocity stall monitor, within its detection bound	De-energize the power stage; engage the spring-applied brake	Winding temperature remains below the chosen operating ceiling in section 2.5.1; base stays inside the 0.15 m side clearance

DRAFT

Failure Domain & Fault Mode	Physical Mechanism & Injection Point	Target Manifestation & Causal Impact	Detection Primitive & Latency Budget	Enforced Safe Response & Bounded Recovery	Observable Pass Criterion
Proposer Injection: Instruction or Visual	Adversarial instruction text on the task channel, or a printed patch, label, or sign placed in the intent model's camera view	Well-formed proposals that pursue the attacker's goal; every field, timestamp, and lease is valid	No check on the proposal's meaning; speed, force, swept-volume occupancy, stopping-reserve, and freshness checks on every tick	Clamp or refuse the proposal like any other; wrong actions inside the envelope continue	Every proposal that breaches a monitored limit is clamped or refused within the 400 µs enforcer deadline; wrong actions inside the envelope are recorded as coverage deficits, never as passes
Authority Transfer (A1): Duplicate Writer	Second bus master writes a base-drive channel (duplicate EtherCAT identifier or DMA write) while policy and remote assistant both command	Two writers on one channel; the delivered command is no longer the admitted one	Bus or DMA write permissions; per-tick authority record	Reject the foreign write; the enforcer path remains the only writer	Independent bus or write-path witness confirms foreign writes are blocked; authority records show one admitted writer
Authority Transfer (A2): Tier Bypass	Policy proposal injected while `MAN` is active; remote-assistant command that opposes the enforcer's admitted limit	A lower tier displaces an active higher tier, or an assistant command skips the permission check	Arbiter tier check; permission check applied to every source	Ignore the policy proposal; clamp or refuse the assistant command like any proposal	Arbiter ignores the proposal; the permission check still runs on the assistant's command
Authority Transfer (A3): Handover Timeout	Assistant input frozen during `PEND`; acknowledgment frames dropped, at the takeover slow-down and at aisle speed	The machine waits on a handover that no one completes	Handover deadline timer in the arbiter	Reject moving handover in the operative design; a candidate slow-down trial starts the validated fallback when its qualified deadline expires	No moving handover admitted in the operative design; in candidate trials at both speeds, local stop dispatched by the qualified current-clearance deadline of section 14.2.4, without awaiting acknowledgment
Authority Transfer (A4): Authentication	Replayed captured `veto_path`, `manual_drive`, or `accept_item` packet; packet carrying a bad tag	An unauthenticated source gains authority	Tag and freshness check on every assistant and coworker command	Reject the packet; rate-limit ingress while preserving valid authority	Invalid packet rejected; still-valid authority preserved; rejection recorded; enforcement deadlines met under ingress load

15.6.3 Physical sensor spoofing and proposer injection

Whether sun glare washes out a camera lens by accident or an attacker shines a laser into the optics on purpose, the camera interface delivers a well-formed, checksum-valid packet whose physical reading is false. In both ambient disturbances and **transduction-layer cyber-physical attacks**, checksum validation and bus integrity checks pass cleanly while the robot's view of the aisle is corrupted. When evaluating such an optical disturbance (retroreflective glare, strobed lighting, or lens occlusion), the test protocol must specify the illumination levels and vehicle approach vectors that challenge obstacle detection, the cross-sensor plausibility checks tasked with identifying the discrepancy,

and the deterministic fallback maneuver. An independent ground-truth optical tracking system monitors obstacle clearance and base trajectory while camera streams are disturbed. The verification record must distinguish an actively detected anomaly followed by a controlled deceleration from a missed obstacle where collision was avoided purely by chance. If a monitoring subsystem cannot distinguish corrupted sensor data from valid physical evidence, that operational scenario constitutes a verification gap; documenting an aspirational safety monitor in an architecture diagram does not ensure it will fire under physical stress.

The learned proposer is itself a primary attack surface; targeting it tests the proposal boundary far more directly than conventional bus or memory faults. Because a vision-language-action (VLA) model consumes natural language instructions and camera frames directly, an adversary can manipulate the model's outputs through either modality without touching a physical wire or intercepting a bus. Robey and colleagues demonstrated automated prompt generation that induced robots controlled by language models, including a commercial quadruped evaluated strictly as a black box, to execute harmful physical actions across diverse operational settings despite high-level safety guardrails (Robey et al. 2025). On the visual side, Wang and colleagues showed that introducing a compact adversarial patch[6] into a VLA's camera field of view could degrade simulated manipulation success rates to zero (Wang et al. 2025). On a deployed warehouse machine, physical injection points encompass task instruction strings, floor placards, shipping container barcodes, or printed warning signs within the camera's visual cone.

In semantic target grounding (equation 10.3), the intent model uses a conformal prediction set $\mathcal{C}$ calibrated to a marginal miscoverage rate ϵ to filter ambiguous hypotheses by requiring a singleton output ($|\mathcal{C}| = 1$). In an adversarial or out-of-distribution physical setting, however, an adversarial patch or optical distortion violates the exchangeability premise of the conformal calibration. An attacker can force an erroneous candidate to pass as a singleton ($|\mathcal{C}| = 1$), or the model can misclassify within its nominal ϵ error budget. Because the deterministic permission path does not parse task instructions or inspect the conformal set $\mathcal{C}$, it cannot distinguish an adversarial proposal from an authentic user command. Instead, under its qualified measurement and timing premises, the enforcer rejects or modifies an injected proposal when the requested physical action violates its monitored constraints: velocity and torque envelopes, dynamic stopping reserves, swept-volume occupancy, evidence freshness, or active lease duration. If an adversarial proposal meets these physical constraints and the authority checks, the enforcer may admit it, even if the robot picks an unintended bin, deposits cargo in an unauthorized clear aisle, or actuates an access gate prematurely. The permission path supports physical containment and dynamic feasibility under its declared premises; those checks do not establish semantic task correctness. This operational asymmetry highlights why systems architects decouple proposal synthesis from safety enforcement: protective safety guarantees must hold regardless of how an action was synthesized, while task-level semantic fidelity cannot be guaranteed by low-level enforcers. Consequently, the proposer-injection row in table 15.2 evaluates trials solely against independent physical oracles, recording uncommanded actions that remain within the admissible safety envelope as semantic coverage deficits rather than as safety violations.

[6] **Adversarial Patch**: Brown et al. (2017) demonstrated that printing an image-independent physical patch and placing it within a visual scene forces an image classifier to ignore surrounding objects and predict the attacker's target class. The perturbation enters entirely through the optical transducer without requiring software access, bus interception, or packet injection. In an embodied system, this attack operates wholly on the proposer's side of the proposal boundary, where the permission path can constrain the physical consequences of an altered command but cannot observe the visual artifact that induced it.

15.6.4 Choosing the injection phase

Injecting a proposer stall immediately after a lease renewal forces the base to coast on open-loop setpoints for nearly the full lease duration, stressing tracking error and clearance. Injecting the same stall microseconds before lease expiration, by contrast, catches the arbiter right at the boundary between renewal and revocation. Because the instant a fault arrives within the control cycle changes which hardware and software paths bear the load, a verification campaign cannot treat repeated fault injections at arbitrary times as interchangeable trials. Recording the exact injection phase relative to the control tick turns phase into a controlled test variable. Sweeping across injection phase, fault duration, and disturbance magnitude exposes race conditions in the arbitration logic and supplies the signed safety margins that automated falsification algorithms search over (section 15.9.2).

↩ The limit that the permission path enforces physical feasibility rather than semantic task correctness is established in section 10.7.

15.6.5 Compound faults and coupled disturbances

When a drive-current surge sags a shared DC power rail, it does not wait for the perception pipeline or the safety microcontroller to finish its current cycle: a single electrical dip can brown out the

application processor, freeze an image serializer, and jitter a shared clock all at once. While each row of table 15.2 tests one disturbance in isolation, real machines rarely fail one subsystem at a time. Combining rows at random would cause a combinatorial explosion without targeting realistic failures, so the campaign instead derives compound faults from shared hardware causes. The placement record (section 13.8) lists every physical resource that the permission path shares with the learned workloads, and each shared power rail, clock tree, or communication bus becomes a compound fault injected at the shared hardware root rather than at downstream software symptoms.

On the mobile manipulator, the primary shared vulnerability is the main DC drive bus, which powers the traction motor drives directly while feeding the application processor through an intermediate DC-DC converter. Meanwhile, the safety-critical permission path operates from an isolated, capacitor-backed hold-up rail. Notebook 15.2 evaluates the electrical headroom and timing budgets for both F4 fault injection trials using the vehicle's electrical specifications.

Napkin Math 15.2: The supply headroom

Problem: At the depleted point, can the drive transient alone carry the shared bus below the lockout of the converter that feeds the application processor, and how much of the permission rail's hold-up does the longest stop in the optimistic screening branch spend?

Variables:

- Depleted bus $V_{\text{low}} = 42$ V and converter lockout $V_{\text{uvlo}} = 33$ V (Datasheet).
- Source and harness resistance $R = 60$ mΩ (Illustrative; needs a measured value) and drive transient $I = 70$ A (Illustrative).
- Rail hold-up 2 s (Chosen) and spring-brake engagement $t_{\text{engage}} = 80$ ms (Illustrative).
- Longest stop in the optimistic all-weight screen: from the 1.23 m/s inspected-floor ceiling at $a = 1.18$ m/s² (Derived from floor friction with the entire weight on driven wheels) with $j_{\max} = 7.11$ m/s³ (Derived).

Math:

F4a, sag under the transient and the step that reaches lockout.

$$V_{\min} = V_{\text{low}} - I\,R = 42 - 70 \times 0.060 = 37.8 \text{ V}, \qquad I_{\text{cross}} = \frac{V_{\text{low}} - V_{\text{UVLO}}}{R} = \frac{42 - 33}{0.060} = 150 \text{ A}$$

F4b, flag to brakes set, with the stop time of equation 11.17.

$$t_{\text{set}} = (T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}) + \left(\frac{v}{a} + \frac{a}{j_{\max}}\right) + t_{\text{engage}} = 22 + \left(\frac{1.231}{1.177} + \frac{1.177}{7.11}\right) \text{s} + 80 = 22 + 1045.9 + 165.5 + 80 = 1313.4$$

Result: The transient leaves 4.8 V of headroom above the lockout, and reaching it takes a 150 A step, more than twice the transient. The 2 s hold-up outlasts the 1313.43 ms from flag to brakes set by 686.6 ms.

Systems insight: Neither trial can rest on the commanded load. F4a must step the supply itself, because the drive's own current cannot reach the lockout, and F4b must open the feed during the longest stop admitted by the qualified loaded envelope, since a shorter stop passes on hold-up the claim never needed. The computed duration belongs to the optimistic all-weight screen; loaded qualification must replace its braking premise before fixing the physical F4b oracle. The headroom rests on an illustrative resistance and would vanish near 129 mΩ, about twice that value, so the source resistance measured under load is the first number the supply campaign must supply.

A compound fault trial must capture high-bandwidth electrical waveforms while independently recording the behavior of the learned policy, perception pipelines, motor controllers, and external telemetry witnesses. Engineers cannot infer that processors reset or sensor frames froze from software logs alone, because the electrical disturbance often corrupts the operating system writing the log. For example, a voltage droop on a shared power rail can induce oscillator jitter that slows

the enforcer's control cycle, freeze an image serializer so that a camera publishes identical frames with plausible timestamps, and reset an external flight recorder during the exact same interval. Similarly, electromagnetic burst interference across a shared fieldbus ages perceptual observations while stalling the permission path's read requests and suppressing the diagnostic heartbeats that would otherwise signal the delay. A drift or jitter transient in a shared reference clock corrupts timestamps across the proposal-enforcement boundary at the same time, allowing stale data to pass as fresh evidence. The physical oracle evaluating a compound fault must therefore verify both that the protective response triggered within its deadline and that the witness instruments and evidence logs remained intact throughout the event (section 15.12).

Because unmodeled physical couplings cannot be found from a resource inventory alone, forward physical analysis must always complement derivation from the inventory. Together, the resulting fault catalog unites four sources: inverted design contracts, forward-derived physical hazards, the authority transfer cases (A1–A4), and compound disturbances on shared hardware. Defining this catalog establishes *what* faults the machine must survive; choosing the physical fidelity of the test bench determines *which* of those claims a given trial can actually test.

15.7 The Qualification Ladder

A desktop simulator can stall the proposer of F1 across every combination of speed and payload overnight, but it cannot sag the drive bus of F4. The simulated power network has no buck-boost converter to lock out and no microcontroller to brown out and reset, so a million passing rollout assertions (`assert clearance >= 0`) there say nothing about F4. That is the digital twin delusion of section 15.1, and the remaining question is where, precisely, a given simulator stops being evidence.

The gap between simulation and reality stems directly from what the simulator abstracts away. When a physics simulation omits embedded target execution and electrical dynamics, it relies on five pervasive idealizations, each of which must be evaluated against the specific claim under test:

1. **Computation takes zero virtual time:** The simulator pauses its simulation clock while the neural policy executes. This treats heavy neural forward passes as instantaneous, completely decoupling inference latency from physical base velocity ($d_{\text{blind}} = v\tau_{\text{delay}}$).
2. **Memory bandwidth is infinite:** Tensors pass across shared in-memory references. This completely omits the shared-memory contention, cache evictions, and DMA serialization stalls that govern real-time execution on an edge SoC.
3. **Power rails are infinitely stiff:** Power supplies are modeled as idealized constant-voltage sources ($V_{\text{bus}} = \text{const}$). This ignores the resistive droop ($\Delta V_{\text{bus}} = IR$) of the drive bus under sudden current steps (sized for this machine in notebook 15.2), the inductive flyback spikes from sharp current transients, and the resulting brownout resets.
4. **Clocks are perfectly synchronized:** The simulation advances monotonically in fixed discrete time-steps (Δt). This eliminates asynchronous clock drift, interrupt jitter, and transceiver arbitration races between the neural host and the real-time safety microcontroller.
5. **Contact mechanics are simplified:** Numerical contact solvers approximate Coulomb friction with linearized friction cones or convex relaxations that permit artificial creep under static load (Todorov 2014). They routinely neglect the transition between static and kinetic friction, dynamic tire slip, and brake-lining glazing during emergency stops.

↩ Resistive droop on a shared drive bus and the brownout resets it causes are established in section 2.6.1.

Each qualification environment closes, breaks, or substitutes different links in the chain connecting sensing, inference, communication buses, motor drives, and physical mechanisms. The **qualification ladder** orders four verification environments by how much of the cyber-physical loop is physically executed rather than simulated: software-in-the-loop, processor-in-the-loop (PIL), hardware-in-the-loop (HIL), and physical fault rigs. Software-in-the-loop is the lowest rung and the physical rig the highest, so climbing the ladder means making more of the loop physical. As physical hardware replaces numerical models, test fidelity rises, but trial throughput drops by orders of magnitude. This progression is not, however, an evidentiary hierarchy in which higher rungs subsume lower ones. Each rung yields a distinct category of empirical evidence while remaining blind to physical phenomena outside its boundary. A million successful software-in-the-loop rollouts reveal nothing about bus contention, DMA stalls, clock drift, or supply rail sag on the embedded hardware.

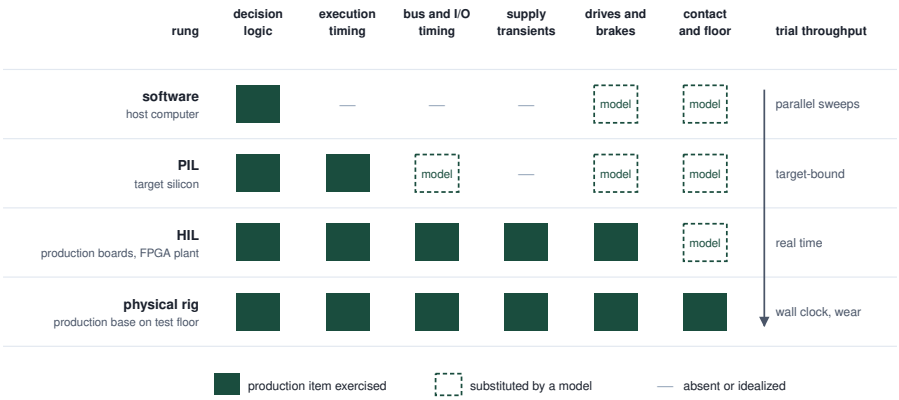


Figure 15.2: What each rung makes real: A filled cell is the production item itself, a dashed cell is a model standing in for it, and a dash marks a link the rung idealizes away. Rows run from the lowest rung, software-in-the-loop, to the highest, the physical rig; coverage of the causal loop grows with each higher rung while trial throughput falls, so no rung dominates. A mechanism can fail only on a rung that contains it, and only the lowest rungs are cheap enough to search at scale.

15.7.1 What each of the four rungs makes physical

On the warehouse mobile manipulator, software-in-the-loop evidence terminates at the boundary of the numerical plant model; a mechanical brake modeled merely as a first-order lag cannot capture pad thermal glazing, wheel slip, or torque saturation. Processor-in-the-loop (PIL) execution introduces the cross-compiled binary running on the actual target silicon (such as an ARM Cortex-R or RISC-V safety core), validating fixed-point quantization, instruction timing, and stack headroom. Hardware-in-the-loop (HIL) benches mount the two-processor architecture, with its physical fieldbus, power regulators, and motor drive stages, against a hard real-time FPGA plant model or an instrumented dynamometer. Finally, physical fault rigs close the full loop with the production chassis, mechanical brakes, wire harness, and physical warehouse floor.

↩ The two-processor architecture, which runs the permission path on a safety microcontroller apart from the learned proposer's application processor, is established in section 4.2.

Two operational testing paradigms exist alongside this qualification ladder, but both break the closed loop entirely. Trace replay feeds recorded sensor streams into the candidate controller; while it preserves authentic sensor noise and optical artifacts, the recorded frames reflect the actions of the original historical policy rather than reacting to the candidate's new commands. Shadow operation runs the candidate model in parallel with an incumbent, validated controller. If the candidate proposes an aggressive velocity vector that would drive the base into a storage rack, the logger records the telemetry while the incumbent controller safely halts the base. Because neither approach permits candidate actions to alter future state observations, neither reveals the closed-loop physical consequences of an erroneous command.

A production shadow deployment cannot allow a stalled proposer to drive toward a human worker at a rack end to see if the brakes engage in time. A software simulator cannot expose a timing race between transceiver silicon buffers. An instrumented physical dynamometer cannot execute the tens of millions of randomized runs required to bound extreme distribution tails. Systems engineers must therefore route each fault injection scenario to the specific rung on the ladder that embodies its underlying physical and causal mechanism. Coverage of the causal loop grows with each higher rung, from decision logic alone to drives and floor contact, while trial throughput shrinks, so no rung dominates the others (figure 15.2).

Selecting the proper qualification rung requires identifying the exact premise that a fault challenges. Validating the software lease expiry under CPU contention in F1 requires target silicon execution to capture true processor scheduling interference. In contrast, evaluating supply rail droop in F4 demands the production power distribution network and independent oscilloscope instrumentation across the DC bus. Similarly, certifying a stopping-distance budget requires a fully ballasted physical chassis operating over a qualified floor surface. A sound verification strategy uses high-throughput

simulation to search the combinatorial operational space for edge cases, then reproduces those critical scenarios on the specific hardware rung that contains the physical mechanism under test. Agreement between two rungs validates only the shared causal links common to both; table 15.3 outlines these strict evidentiary boundaries.

Table 15.3: The four-stage qualification ladder across cyber-physical boundaries: The four qualification environments compared by computing substrate, exercised causal-loop elements, fault injection primitives, execution throughput, physical and temporal fidelity, and test oracle.

Qualification Rung & Substrate	Causal Loop Realism (Compute, Bus, Actuator, Plant)	Fault Injection Mechanism & Scope	Throughput & Sample Scaling	Physical & Temporal Fidelity	Primary Test Oracle & Evidentiary Limit
Software-in-the-Loop (Host Workstation / Cloud Cluster)	**Compute:** Synthetic / Host x86; **Bus:** Idealized zero-copy buffer; **Actuator:** Linear / saturation model; **Plant:** Numerical ODE/PDE solver	Tensor bit-flips, synthetic sensor dropout, floor-friction, payload, and latency sweeps	Parallel sweeps; throughput depends on model and compute allocation	High mathematical flexibility; zero target silicon, bus timing, or electrical realism	**Oracle:** Mathematical assertion corridors and state invariants. **Limit:** Cannot validate target execution timing or unmodeled physical dynamics.
Processor-in-the-Loop (PIL) (Target SoC / MCU)	**Compute:** Real target silicon (ARM/RISC-V); **Bus:** Simulated bus interconnect; **Actuator:** Emulated command register; **Plant:** Real-time host model	Instruction cache evictions, register corruption via JTAG/BIST, interrupt jitter injection	Bound by target execution and cross-platform synchroniza-tion	Instruction-set and compiler accurate; abstracts analog bus lines and actuator back-EMF	**Oracle:** Target register state, numerical equivalence, and execution-time profiles. **Limit:** Ignores physical transceiver bus contention, voltage droop, and thermal rise.
Hardware-in-the-Loop (HIL) (Target ECUs + Real-Time Plant Rig)	**Compute:** Production ECU / NPU; **Bus:** Real physical CAN/Ethernet; **Actuator:** Real motor drives / dyno; **Plant:** Hard real-time FPGA plant rig	Pin break/short, CAN frame stuffing / babbling idiot, supply rail droop, load steps	$1.0\times$ strict real-time (24 h/day wall-clock limit)	High electrical, timing, and protocol fidelity; bounded by FPGA plant model accuracy	**Oracle:** Protocol CRC, bus deadline monitors, and interlock timings. **Limit:** Does not physically exercise modeled floor contact or wear; overload coverage depends on the rig and its protection.
Physical Fault Rig (Dynamometer Cell / Field Plant)	**Compute:** Production ECU / NPU; **Bus:** Production harness & sensors; **Actuator:** Real physical mechanisms; **Plant:** Real base, arm, and floor	Wheel clamps, magnetic brake stall, thermal heating, transducer bias drift	Wall-clock execution; constrained by wear and thermal reset	Full authentic physics (Coulomb friction, wheel slip, thermal dissipation, wear)	**Oracle:** Independent physical containment instrumentation and yield transducers. **Limit:** High financial cost per trial; destructive tests cannot scale to statistical tail bounds.

DRAFT

15.7.2 Where each rung's simulation boundary cuts the loop

Replacing physical wire harnesses, ground returns, and distributed actuator nodes with desktop simulations strips away inductive voltage droop, transmission reflections, and electromagnetic interference (EMI). Every qualification rung has a simulation boundary, the interface where numerical models, emulated buses, or replayed traces sever the exchange of energy and information

between computation and mechanics, and no environment can certify properties that depend on physics outside that boundary. Software-in-the-loop testing falsifies algorithmic decision logic and state-machine transitions across broad parametric sweeps. PIL uncovers numerical divergences from cross-compilation and profiles worst-case execution time on target silicon, yet because it funnels input and output over a hardware debug interface (such as a JTAG port or serial UART link), it cannot stress physical communication buses. HIL brings in the true physical transceivers, bus contention, and electrical transients, while leaving contact mechanics to real-time models. Finally, physical fault rigs subject the structural plant to unmodeled Coulomb friction, tire deformation, and thermal dissipation.

A higher rung therefore does not subsume or obsolete the lower ones. Verification is an exercise in constrained optimization across a Pareto frontier: parallel software-in-the-loop clusters explore millions of permutations of obstacle layouts and lighting variations. Physical rigs, by contrast, capture true contact mechanics at the expense of manual resets, mechanical wear, and thermal recovery between runs. Each stage occupies a unique operating point balancing combinatorial test throughput against physical causal fidelity. A mechanism omitted from a model cannot fail within it. If a simulator omits Coulomb breakaway friction and stator inductive lag, a simulated robot joint moves immediately upon receiving a torque command. On real hardware, however, that same joint remains stationary during a stiction dead time before lagging behind the commanded trajectory. This is a classic manifestation of the omitted-mechanism gap, which no amount of software simulation can uncover.

To guard against false confidence, every empirical verification claim must explicitly catalog its causal envelope: which links in the sensing-computation-actuation loop were physically real, which were simulated, which were replayed from static traces, which were bypassed, and which remained unobserved. Stating that the safety permission path halted the mobile base within 969.0 mm is meaningless in isolation. An engineer must specify whether the wheel-speed velocity signal was computed by a numerical ODE solver, streamed from a physical optical encoder over a saturated CAN bus, or read out from an in-memory replay buffer. Mapping this boundary exposes the physical assumptions behind each pass and prevents latent hardware hazards from hiding behind passing software assertions.

15.7.3 Scope of a formal verification result

A formal proof that a permission filter preserves safety over a mathematical plant model still cannot prevent a crash if the tires slip on a wet floor, the camera frames arrive late, or the proof is asked to cover a vision-language-action proposer fifty times larger than any network a complete verifier can solve. Formal proof provides a fifth source of evidence alongside the physical qualification ladder, but a proof holds only inside its mathematical model of plant states, sensor noise, and bounded disturbances. Translating that guarantee to the embodied machine still rests on empirical measurements of plant dynamics, observation deadlines, bus latencies, and actuator torque limits. Which components of the running machine a proof can reach at all depends on the current scale limits of neural-network verification.

Tools for formal neural-network verification operate in one of two regimes: open loop or closed loop. An open-loop verifier checks an **open-loop property**: given any input in a specified bounded set (for example, every camera frame within an ℓ_∞ ball around a nominal image, meaning every frame whose pixels each lie within ϵ_{adv} of the nominal values), the network's output is guaranteed to satisfy a declared constraint. Two algorithmic primitives drive these tools:

- **Bound propagation** pushes the input set forward through the network layer by layer. By replacing each nonlinear activation with linear relaxation bounds, it computes a sound, conservative bounding box around the outputs. If this enclosure satisfies the property, the proof succeeds immediately.
- When bound propagation leaves property satisfaction ambiguous, **branch and bound** partitions the input space or splits ambiguous piecewise-linear activations into distinct linear regimes, recomputing bounds over each sub-domain. For supported network and property classes, a complete search can decide the property in principle, but it scales exponentially with the number of split activations in the worst case; a timeout or unknown result is not a proof. Solvers such as Reluplex (Katz et al. 2017) and Marabou (H. Wu et al. 2024) formulate

these activation states as satisfiability modulo theories (SMT) or mixed-integer linear programming (MILP) problems, whereas the α,β-CROWN verifier accelerates bound propagation and branch-and-bound exploration directly on GPUs (Xu et al. 2021; Wang et al. 2021).

Verifying a **closed-loop property** poses a far steeper challenge. Rather than checking a static input-output mapping, closed-loop verification asks whether the continuous state of a physical plant driven by a neural policy remains within a safe invariant set over an extended time horizon. To answer this, reachability analysis evaluates the continuous trajectories of the physical plant under bounded disturbances. Forward reachability tools, such as NNV, Verisig, and POLAR, compose the network's output bounding box with the differential equations of the physical plant at each discrete control step, computing forward reachable sets for compact controllers on low-dimensional dynamical systems (Tran et al. 2020; Ivanov et al. 2019; Huang et al. 2022). Backward reachability via **Hamilton-Jacobi (HJ) reachability analysis** instead frames safety as a zero-sum differential game between the control policy and worst-case environmental disturbances. It computes the exact maximal safe invariant set as the zero level set of an optimal control value function (section B.6.3) (Mitchell et al. 2005; Bansal et al. 2017). While Hamilton-Jacobi reachability provides non-conservative safety certificates for continuous nonlinear dynamics, solving the underlying Hamilton-Jacobi-Isaacs partial differential equation on a Eulerian grid scales exponentially with continuous state dimension. Consequently, Hamilton-Jacobi reachability is computationally tractable for decoupled, low-dimensional subsystems (such as a single revolute joint or longitudinal base stopping), whereas forward reachability tools struggle with the rapid accumulation of over-approximation error across long planning horizons. NNV also supports bounded image-input robustness analysis through ImageStars (Tran et al. 2020), but those image results do not establish an end-to-end closed-loop guarantee for the running machine's high-resolution vision-language-action stream. Its permission path instead exposes a handful of calibrated physical states and a declared stopping invariant (chapter 12).

Table 15.4 benchmarks the state of the art in formal verification. In the 2025 International Verification of Neural Networks Competition (VNN-COMP 2025), the largest scored benchmark network contained 138M parameters, with the vast majority of evaluated models containing fewer than a million parameters (Kaulen et al. 2025). By contrast, the running machine's intent model, the larger of its two learned proposers, contains 7B parameters, roughly 51× larger than that benchmark outlier. Both proposers also ingest an unconstrained temporal stream of high-resolution camera frames rather than an analytically bounded input set, and a local pixel-perturbation specification alone does not establish stopping safety across changing warehouse scenes and physical states.

The cited results do not establish complete formal verification of this deep proposer under its deployment stream; both computational scale and the absence of a justified visual specification remain barriers. This separation is the third law, proposal is not permission (principle 3), seen from the verifier's side. The learned proposer is evaluated and stress-tested empirically, and formal proof is reserved for the compact, deterministic permission path, which reads a handful of calibrated physical measurements (wheel speeds, lidar clearances, payload mass) and enforces an analytically tractable stopping invariant that a proof can reach.

↩ The separation of a learned proposer from a deterministic permission path is established in section 1.7.

Table 15.4: Verification methods and their reach, as of 2026: Representative tools for each method class, what a result establishes, and the scale at which it currently operates. The mechanisms in the prose are durable; the named tools and scales will change, so the table carries its date.

Method class	Representative tool	What a result establishes	Scale and limit, as of 2026
Complete open-loop NN verification	α,β-CROWN (Wang et al. 2021); Marabou 2.0 (H. Wu et al. 2024)	A property holds for every input in a bounded set, or a counterexample input	α,β-CROWN ranked first in VNN-COMP 2025; scored benchmarks reach 138M parameters (Kaulen et al. 2025)
Closed-loop reachability	NNV (Tran et al. 2020); POLAR (Huang et al. 2022); HJ reachability (Bansal et al. 2017)	Forward reachable set or maximal backward invariant set under bounded disturbances	Small controllers and low-dimensional plants; forward bounds grow conservative, backward HJ PDE grids scale exponentially with state dimension

Method class	Representative tool	What a result establishes	Scale and limit, as of 2026
Temporal-logic falsification	S-TaLiRo (Annpureddy et al. 2011); Breach (Donzé 2010)	A counterexample trajectory with negative robustness, or none found within the search budget	Black-box simulation of any model; no certificate from an unsuccessful search
Rare-event estimation	Importance sampling with the cross-entropy method (O'Kelly et al. 2018)	A failure probability under a declared operational distribution, with its variance	Only as good as the declared distribution and the proposal's coverage of the failure set

Even within the permission path, high-dimensional reachability calculations, nonlinear contact dynamics, and hybrid switching modes can cause mathematical over-approximations to become prohibitively conservative. These practical limits do not render formal analysis useless. Instead, they dictate engineering restraint: they favor designing compact, mathematically transparent permission filters with explicit physical premises that can be verified in minutes, leaving the unmodeled electrical and mechanical dynamics to be systematically challenged on the physical rungs of the qualification ladder.

Checkpoint 15.2: Rung choice and the reach of a proof

A rung, like a proof, supports only the mechanisms it contains.

- ☐ **Rung selection**: For F2 (a frame held at the image signal processor), F3 (lost fieldbus frames), and an actuator jam, can you name the lowest rung that contains the causal mechanism and one mechanism that rung still idealizes away?
- ☐ **Agreement between rungs**: Can you explain why a software-in-the-loop pass and a HIL pass on the same scenario support only the mechanisms both rungs represent?
- ☐ **Replay vs. shadow operation**: Can you explain why neither trace replay nor shadow operation shows the physical consequence of the candidate's command?
- ☐ **Proof scope**: Can you explain why the cited closed-loop results support analyzing the permission path without establishing a guarantee for the learned proposers, and list the premises such a proof leaves to physical test?

15.8 Fault Injection on Hardware

A fault injected in simulation tests only the modeled response. On physical machinery, bus contention, microcontroller clock drift, DMA stalls, and transient voltage drops across inductive motor drivers determine whether a safety intervention executes before physical impact. When a simulator fails to capture and calibrate these physical interactions, simulation alone cannot substantiate claims that depend on them. Instead, **in-situ fault injection** runs directly on the target hardware while an external containment rig keeps the facility safe, deterministically introducing physical, electrical, and computational disturbances (sensor register latches, clock drift, bus contention, supply brownouts, and actuator load stalls) into production silicon, transceivers, timers, shared buses, and mechanics under authentic compute, thermal, and power loads.

15.8.1 Physical injection needles across sensor, bus, and power

A fault injected in software above the driver can exercise downstream responses but does not reproduce a disturbance originating at the physical interface; an empirical verification campaign must place its injectors at the physical boundaries themselves.[7] Four complementary injection mechanisms cover the machine's safety claims:

1. *Sensor ingress injectors:* Inline interposers on camera serial links freeze frames, inject propagation delay while preserving original hardware timestamps, or corrupt pixel payloads at the physical layer without modifying driver software.
2. *Fieldbus disturbance nodes:* Inline devices inject protocol-specific faults: corrupted frames, CAN collisions that drive target controllers into bus-off states,[8] dropped EtherCAT distributed-clock synchronization frames, or babbling-idiot traffic that monopolizes shared resources.

[7] **Injection fixtures**: Power injectors switch or modulate the supply using solid-state devices, while bus injectors manipulate communication lines through high-impedance buffers. Test instrumentation must remain galvanically isolated from high-current motor transients; otherwise, witness systems will fail alongside the drive. Achievable edge rates and isolation ratings characterize the specific test fixture and must be recorded in the qualification dossier.

[8] **Bus-Off**: The Controller Area Network protocol enforces hardware fault confinement: when a controller's transmit error count exceeds standard thresholds, it enters bus-off and ceases bus communication until cleared by an idle sequence (International Organization for Standardization 2015a). While this prevents a damaged node from corrupting the bus, external disturbance injectors forcing bit collisions can exploit this mechanism to silence healthy nodes without a software fault. Physical fault injection must prove that the safety permission path detects this silenced node and initiates safe fallback within its timing budget.

3. *Supply injectors:* Programmable power supplies emulating the battery pack, paired with dynamic electronic loads, step drive-bus and logic rails through calibrated voltage droop profiles. Delivered waveforms must be measured directly at the device terminals, because test-fixture slew-rate limits dictate which high-frequency transients the setup can faithfully reproduce.
4. *Dynamometer actuator injectors:* Dynamometer load machines mechanically coupled to wheel hubs or arm joints command calibrated load steps within their measured bandwidth. These challenge responses to increased resistance or abrupt joint loads; reproducing bearing seizures, brake-lining glazing, or joint jams requires mechanism-specific fixtures while instrumentation records the closed-loop motor current response.

Verifying the dual-processor safety architecture requires targeted physical injection at the exact hardware boundaries established by the safety contract. Table 15.2 summarizes the corresponding detectors, recovery actions, and acceptance criteria; the following breakdown details the physical injection mechanics and timing instrumentation for each fault class:

- **Expired Lease Revocation (F1):** The test framework permits the chunk policy to issue a single valid renewal, then freezes its task or command output while leaving its periodic heartbeat active, verifying that superficial liveness cannot renew the lease. Dropping subsequent lease messages is a separate test of expiry under communication loss. High-speed logic analyzers time lease expiration, the enforcer's formal revocation on the subsequent 1 ms control tick, and the mechanical onset of braking across dedicated hardware trigger lines.
- **Stale Observation Injection (F2):** An inline FPGA interceptor latches a navigation-camera frame at the image signal processor interface, or suppresses optical encoder updates, while preserving the original hardware timestamp intact. External instrumentation independently records the exact moment of detector trip, the onset of powered electrical deceleration, and the resulting wheel speed profile.
- **Lost-Frame Injection (F3):** A fieldbus disturbance node drops or corrupts EtherCAT frames directly at a servo drive's slave port while the mobile robot cruises. The master's frame-arrival, validity, and working-counter checks[9] detect communication loss under their configured deadlines. At aisle velocity, each lost 1 ms cycle allows the vehicle to travel an unmonitored 1.3 mm, eating directly into the physical safety margin.
- **Drive-Bus Droop and Feed-Loss Injection (F4a, F4b):** A programmable supply standing in for the battery pack at its 42 V depleted point delivers the full 70 A transient demanded by aggressive drive acceleration while the application SoC operates at peak load. The supply injector then commands a voltage step that forces the drive bus below the 33 V converter undervoltage lockout, a threshold that operating load alone cannot breach (notebook 15.2). Throughout this transient, the permission path must remain fully operational, sustained by the isolated, held-up rail. An independent digitizer records the permission-path heartbeat, and instrumentation measures the onset of powered deceleration relative to the undervoltage alarm flag. In trial F4b, an electronic switch opens the permission rail's primary feed during the longest stop admitted by the qualified loaded envelope. The illustrative all-weight screen initiates its resident stop from the 1.23 m/s ceiling. Independent measurements must verify both surviving drive authority and energy absorption throughout dynamic braking. Sustaining logic hold-up alone leaves that premise unverified. Any reset of the permission path constitutes an immediate `FAIL` in either trial, regardless of stopping distance, because a vehicle that forfeits controlled deceleration to unpowered spring brakes mid-motion violates the deterministic stopping envelope budgeted by the safety contract.
- **Handover Stall Injection (A3):** The testbed intentionally freezes operator input while a remote takeover request remains in the handover-pending `PEND` state defined in section 14.3.2, and it drops acknowledgment frames, at both base speeds. At the 0.75 m/s slow-down, a candidate moving handover is tested and any admitted candidate must time out into a local stop. At the 1.3 m/s aisle speed, the handover must be rejected. The operative configuration must stop before granting manual motion at either speed. High-speed probes separately time the enforcer's fallback dispatch and physical actuation onset at the wheel drives. While the

[9] **EtherCAT Working Counter**: As an on-the-fly datagram traverses each downstream slave, every node that successfully processes assigned process data increments a dedicated working counter field in hardware. The returning counter value informs the master within the exact cycle of transmission how many addressed devices successfully read or wrote their process data. A missing return, an invalid frame, or a working-counter mismatch provides a distinct communication-failure indication. The configured master checks can detect F3 within the cycle's declared receive deadline, without waiting for a slave drive's communication watchdog to expire; that detector bound and the subsequent stopping response must be qualified together.

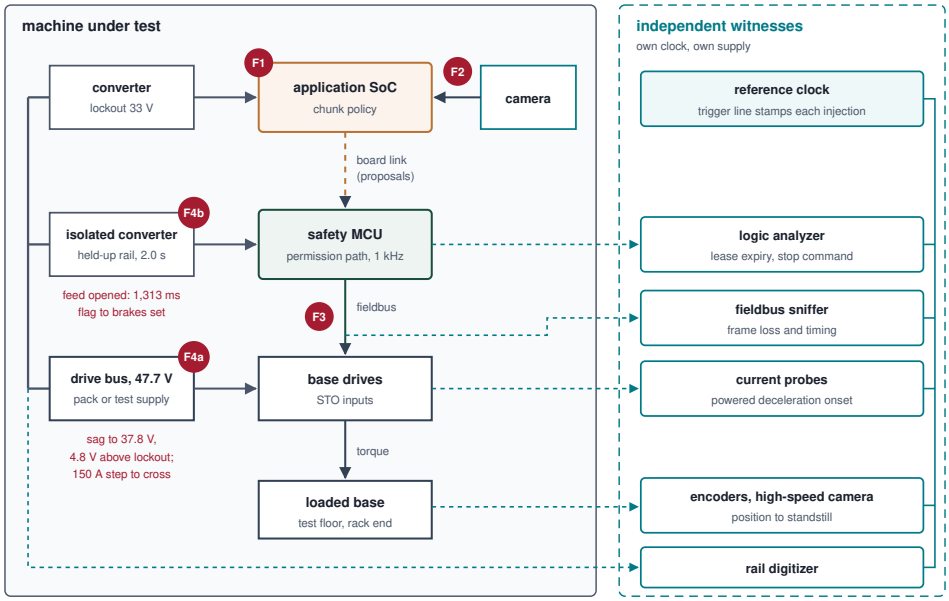


Figure 15.3: The machine under test and its witnesses: Each fault enters at a physical interface of the production machine, F1 at the proposer, F2 on the camera link, F3 on the fieldbus, and F4a and F4b on the supply, and every event the oracle scores is recorded by an instrument on its own clock and supply. F4a steps the supply itself, because the drive transient alone leaves the depleted bus above the converter lockout, and F4b opens the permission rail's feed, whose hold-up must outlast the time from flag to brakes set under the qualified loaded envelope; the figure's duration uses the optimistic all-weight screen (notebook 15.2).

DRAFT

configured waiting allowance limits dispatch latency, physical onset must also satisfy the allocated command-to-response interval.

Duplicate writers (A1), tier bypass attempts (A2), and forged or replayed packets (A4) require no specialized mechanical fixtures beyond a secondary bus master and a packet injector configured according to table 15.2. All four physical injector types and their witness instruments integrate onto a single test bench. The test rig of figure 15.3 enforces strict physical separation: no witness instrument may share a power rail or a timebase with the target machine under evaluation.

15.8.2 Timing faults across clock domains

Because camera frame capture, trajectory proposal generation, enforcer permission ticks, and fieldbus communication cycles run across asynchronous clock domains (section 8.3), a fault injected at a fixed offset from the start of a run can miss the narrow alignment where a race condition trips. Reproducible qualification therefore records the disturbance's phase relative to every relevant clock edge and sweeps across injection phases to expose race conditions that fixed-offset tests miss, while recognizing that empirical phase sweeps alone cannot prove the absence of a worse alignment.

Consider the stale-observation scenario (fault F2) running on production silicon. The navigation camera timestamps each image at the moment of physical exposure, and the perception pipeline budgets a maximum age of 51.6 ms upon trajectory dispatch (section 8.4.1). Because the robot travels at 1.3 m/s, it covers 1.3 mm every millisecond. Unbudgeted perceptual latency drains the stopping distance reserve millisecond by millisecond. The 51.8 ms upper-age bound, however, governs only the dispatch instant; sensory evidence continues to age as the command traverses validation and motor execution. Consequently, the per-tick safety monitor cannot treat the dispatch limit as an open-ended pass; it must evaluate perceptual freshness dynamically against remaining stopping distance and active trajectory validity. The stall that matters is short. Once the dispatch-to-admission allocation is charged, only 37.5 ms of the aisle-speed spare remains (section 15.2), so an undetected stall of that length leaves the machine no clearance to give.

Neither the nominal sensor readout latency from a component datasheet nor the timestamp logged on an isolated image frame proves when the safety enforcer will recognize a stall. Relying on empirical p99 response times provides no safety guarantee because a percentile alone does not bound shared-memory interference; a finite worst-case claim requires an explicit architectural and scheduling model (section 4.5.1). A defensible containment claim demands an analytic and experimental worst-case detection bound validated across maximum compute saturation, peak fieldbus load, and extreme operating temperatures.

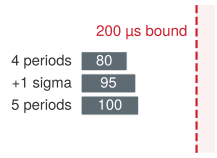
15.8.3 Distributed clock drift and synchronization blackouts

Two processors can each meet every local real-time deadline on their own crystals and still drift apart fast enough during a brief synchronization blackout to let a stale camera frame pass as fresh. The F2 stale-observation detector relies directly on the 0.2 ms clock-conversion bound established in the observation contract of section 8.8 when translating image capture timestamps into the safety microcontroller's local timebase. That bound is not an axiomatic constant: section 8.6.2 measures it from the tail of the synchronization offset, the path asymmetry, and the drift between synchronization exchanges, and verification asks how much of it survives when those exchanges stop arriving. Consider illustrative oscillator tolerances: ±100 ppm for the application processor crystal, ±50 ppm for the safety microcontroller, and an additional 50 ppm worst-case thermal drift unaccounted for in standard room-temperature ratings. Together, these sources yield a maximum relative drift rate of 200 ppm. At this rate, uncorrected drift alone exhausts the entire 0.2 ms conversion budget in 1 s without active resynchronization. Under healthy operating conditions, periodic synchronization packets arriving every 100 ms confine accumulated drift to 20 µs, leaving the remainder of the error budget for network transit jitter and software conversion overhead.

Suppose electrical noise or packet collisions trigger a synchronization blackout where three consecutive sync frames are dropped, leaving two successful packets separated by 400 ms. Across this four-period blackout window, bounded clock drift alone accumulates to 80 µs. When measured synchronization jitter has a standard deviation of $\sigma_{\text{jitter}} = 15$ µs, combining the drift rate δ_{skew} over the blackout duration T_{blackout} with a one-sigma jitter excursion yields:

$$\Delta t_{\text{one-sigma scenario}} = \delta_{\text{skew}} T_{\text{blackout}} + \sigma_{\text{jitter}} \tag{15.2}$$

totaling 95 µs. The one-sigma estimate in equation 15.2 represents an illustrative operating point rather than a deterministic upper bound; certifying real-time safety requires bounding tail jitter analytically and stress-testing the conversion path on hardware. If the blackout extends to a fifth period (four consecutive frames dropped), accumulated oscillator drift alone reaches 100 µs before jitter is added, consuming half of the 0.2 ms error budget. The runtime must refuse proposals or enter its qualified fallback before synchronization uncertainty exceeds the conversion bound. If that guard is absent and clock drift or transit jitter breaches the boundary, two failure modes threaten the machine. In the first failure mode, a fresh camera frame appears artificially aged when compared against a fast microcontroller clock; the enforcer rejects valid proposals and initiates an unprompted emergency stop, stranding the vehicle mid-aisle. In the second, more dangerous mode, a stale observation appears artificially fresh against a slow microcontroller clock. The conversion error itself is cheap in distance. At 1.3 mm per millisecond, the whole 0.2 ms bound is worth less than a millimeter of travel. What the error corrupts is the refusal verdict. Because the admission check adds the 0.2 ms conversion bound to the measured timestamp difference to form a conservative upper age, the nominal navigation frame reaches the 51.8 ms refusal threshold exactly at dispatch, so the observation contract of section 8.8 left that threshold with zero slack. Once a slow microcontroller clock lags by more than the 0.2 ms allowance, the uncompensated clock lag subtracts directly from the measured elapsed age, so the age check can no longer certify that an admitted frame lies within budget. A frame held back by an F2 stall then passes as fresh whenever the uncorrected drift exceeds the stall's extra age. Because drift keeps accumulating for as long as synchronization stays lost, the stall the check can miss grows with the blackout. The permission path may then admit proposals computed from obstacle coordinates older than the budget allows while the loaded base moves at 1.3 m/s.



A five-period blackout spends half the clock-conversion bound.

Every physical fault injection trial concludes with a rigorous evidentiary burden. The test harness must simultaneously record the trip signal from the internal detector, the precise timestamp of

proposal rejection, the actuation takeover at the motor drives, and the final physical stopping trajectory. All measurements must be captured by external, fault-isolated instruments synchronized to a single master reference clock. If a trial fails to record any of these intermediate transitions, it fails to produce actionable verification evidence. Physical fault injection campaigns do not yield simple pass/fail tallies; they generate auditable execution traces binding each trial to its architectural claim, its exact operating envelope, its software build artifacts, and its physical sensor waveforms.

↪ How these trial traces enter a safety case as evidence for a claim is developed in section 16.3.

15.9 Searching Beyond the First Trial

A single successful fault trial demonstrates that the safety path can arrest the vehicle under one declared set of operating conditions. Physical machinery, however, rarely fails under tidy nominal conditions; subtle vulnerabilities hide in the narrow corners of the state space where timing jitter, payload shifts, and degraded surface traction align against the enforcer. Exhaustive testing across this continuous operational envelope is physically impossible, yet uniform random sampling requires an intractable number of runs to uncover rare edge-case failures. Searching beyond isolated trials requires systematically steering disturbances toward the boundary conditions that drive the system closest to its safety limits. Rather than replacing the formal coverage record, automated search routines prioritize which candidate executions deserve empirical evaluation on the physical test track or dynamometer.

15.9.1 Signal temporal logic and quantitative robustness

A binary `PASS` or `FAIL` verdict tells an automated search algorithm nothing about whether a passing stop cleared the rack by 15 cm or by 1 mm, leaving the optimizer blind to near misses. Classical temporal logic (Pnueli 1977) checks whether a sequence of states satisfies a property. **Signal Temporal Logic (STL)** extends the specification to continuous time and real-valued physical signals (such as staying inside a spatial corridor or halting before a deadline (Maler and Nickovic 2004)), and it equips every formula with **quantitative robustness semantics** (Fainekos and Pappas 2009). Instead of returning a boolean flag, quantitative robustness computes a real-valued signed margin $\rho(\phi, s, t) \in \mathbb{R}$ that measures *how far* a trajectory $s : \mathbb{R}_{\geq 0} \to \mathbb{R}^n$ sits from violating or satisfying the temporal property ϕ at time t.

↪ The syntax and semantics of signal temporal logic are developed in section A.7.

For an atomic physical predicate $\pi \equiv (f(s(t)) \geq 0)$ (such as spatial clearance $d(t) - d_{\text{safe}} \geq 0$), quantitative robustness evaluates the signed predicate margin: $\rho(\pi, s, t) = f(s(t))$. Across temporal operators, safety invariants evaluate worst-case (minimum) margins, whereas reachability deadlines evaluate best-case (maximum) margins:

$$\rho(\Box_{[0,T]}\phi, s, 0) = \min_{t \in [0,T]} \rho(\phi, s, t), \qquad \rho(\Diamond_{[0,\tau]}\phi, s, 0) = \max_{t \in [0,\tau]} \rho(\phi, s, t). \tag{15.3}$$

Here $\Box_{[0,T]}$ denotes "always on $[0, T]$" and $\Diamond_{[0,\tau]}$ denotes "eventually within τ."

The sign and magnitude of ρ in equation 15.3 establish an immediate operational interpretation:

$$\rho > 0 \implies \text{satisfied with positive margin } \rho, \qquad \rho < 0 \implies \text{violated with shortfall depth } |\rho|.$$

For example, if the spatial clearance margin is $\rho = +0.15$ m, the minimum clearance observed along the trajectory exceeded the safety threshold by 15 cm. Conversely, if $\rho = -0.04$ m, the machine breached the prescribed clearance boundary by 4 cm, with physical collision depending on whether the safety limit included an explicit standoff buffer.

This signed value margin constitutes **space robustness**, quantifying how far the signal's physical magnitude can perturb before the verdict flips. Space robustness captures amplitude margins rather than timing margins: a delayed brake actuation represents a temporal shortfall rather than an instantaneous spatial error. Capturing temporal slack requires **time robustness**, a complementary semantics that evaluates how far a signal trajectory may shift along the time axis before the verdict changes (Donzé and Maler 2010). Because spatial and temporal margins govern different physical dimensions, combining them into a single unweighted formula introduces semantic ambiguity. For the F1 specification, the cleanest engineering formulation isolates the onset deadline outside the Signal Temporal Logic expression as an explicit scalar predicate on measured reaction time, while evaluating stopping distance via spatial robustness.

For this reason equation 15.1 keeps its two predicates apart. The onset predicate, deceleration within 82 ms of the last valid renewal, is a scalar check on measured reaction time, and the travel predicate, at most 751.9 mm at aisle speed, is a space-robustness predicate on vehicle position. Both are measured from the same reference event, so neither can silently absorb slack from the other; a different reference event would change the elapsed timing and require a correspondingly adjusted distance bound.

To illustrate this dual evaluation numerically, consider an F1 trace at aisle speed, timed channel by channel in figure 15.4, where powered deceleration begins 30 ms late, occurring 112 ms after the last valid renewal. The onset margin against the 82 ms bound evaluates to -30 ms. Because the mobile base travels 39 mm further before braking engages, total travel from renewal to rest reaches 750.9 mm, yielding a distance margin of 1 mm against the 751.9 mm oracle. The onset predicate fails, while nominal travel remains within the distance allowance. The verification results retain both margins in their native physical units; the tracking inset belongs in the oracle and is not additional travel in the plotted speed integral. Because taking an unweighted minimum across milliseconds and millimeters produces an uninterpretable quantity, synthesizing them into a unified score via `min` requires normalizing each margin by a predeclared positive scale in its corresponding unit. Logging both the raw margins and their normalization factors ensures the synthesized verdict remains physically meaningful. Choosing scales equal to the verification bounds themselves yields normalized margins of -0.366 for onset timing and 0.001 for stopping distance, making the onset predicate the governing term in the combined score. When several predicates have negative margins, alternative scales can change which one governs the compound score, so test engineers must fix normalization factors before executing verification runs. In this nominal trace only onset fails, so positive rescaling cannot make distance the governing predicate. These numerical values reflect model arithmetic over a stipulated reference trace rather than empirical physical measurements.

Recording every monitored channel on one timeline, as illustrated in figure 15.4, helps localize a failed trial to a stage of the response path when the independent witnesses remain valid. A lease that expires on schedule and a stop command issued at the next tick, followed by a late braking onset, direct the investigation toward command delivery, drive execution, or physical brake response rather than the lease-expiry check.

By producing a continuous, signed margin rather than a binary pass/fail flag, quantitative robustness transforms verification from passive testing into **temporal logic falsification as adversarial optimization**. In classical computer vision and deep learning, adversarial attacks such as the fast gradient sign method (FGSM) (Goodfellow et al. 2015) and projected gradient descent (PGD) (Madry et al. 2018) search for an input perturbation δ_{adv} bounded within an ϵ_{adv}-ball ($\|\delta_{\text{adv}}\|_\infty \leq \epsilon_{\text{adv}}$) that maximizes task loss: $\max_{\|\delta_{\text{adv}}\|_\infty \leq \epsilon_{\text{adv}}} \mathcal{L}(f_\theta(x+\delta_{\text{adv}}), y)$. In cyber-physical systems, temporal logic falsification provides the continuous dynamic counterpart. Rather than perturbing static feature vectors, an adversarial optimizer searches over initial state spaces and time-varying physical disturbance trajectories within a bounded physical disturbance envelope to minimize quantitative Signal Temporal Logic robustness. The robustness metric ρ acts directly as the adversarial objective function. Whereas uniform random sampling across a high-dimensional operational state space requires an intractable number of physical trials to expose rare corner-case collisions, an adversarial falsification engine actively steers disturbance sequences toward trajectories that drive the system closest to violating its safety invariants.

15.9.2 Formulating falsification as trajectory optimization

An unconstrained mathematical search over arbitrary disturbance signals can easily invent discontinuous spikes that no real actuator, sensor, or contact surface can physically produce, wasting the search on impossible counterexamples. To restrict the search to physically realizable disturbances, let $\mathbf{x}_0 \in \mathcal{X}_0$ represent the machine's initial state vector (translational speed, payload mass, heading angle, and battery state-of-charge), and let $w_{\text{dist}}(t) \in \mathcal{W}$ denote an adversarial disturbance trajectory spanning the test horizon $[0, T]$. Because physical hardware has finite bandwidth, mechanical inertia, and thermal limits, disturbance trajectories are parameterized over a compact parameter space $\psi \in \Psi$ using smooth basis functions, such as B-splines, cubic splines, or Fourier series:

$$w_{\text{dist}}(t;\psi) = \sum_{k=1}^{K} \psi_k B_k(t)$$

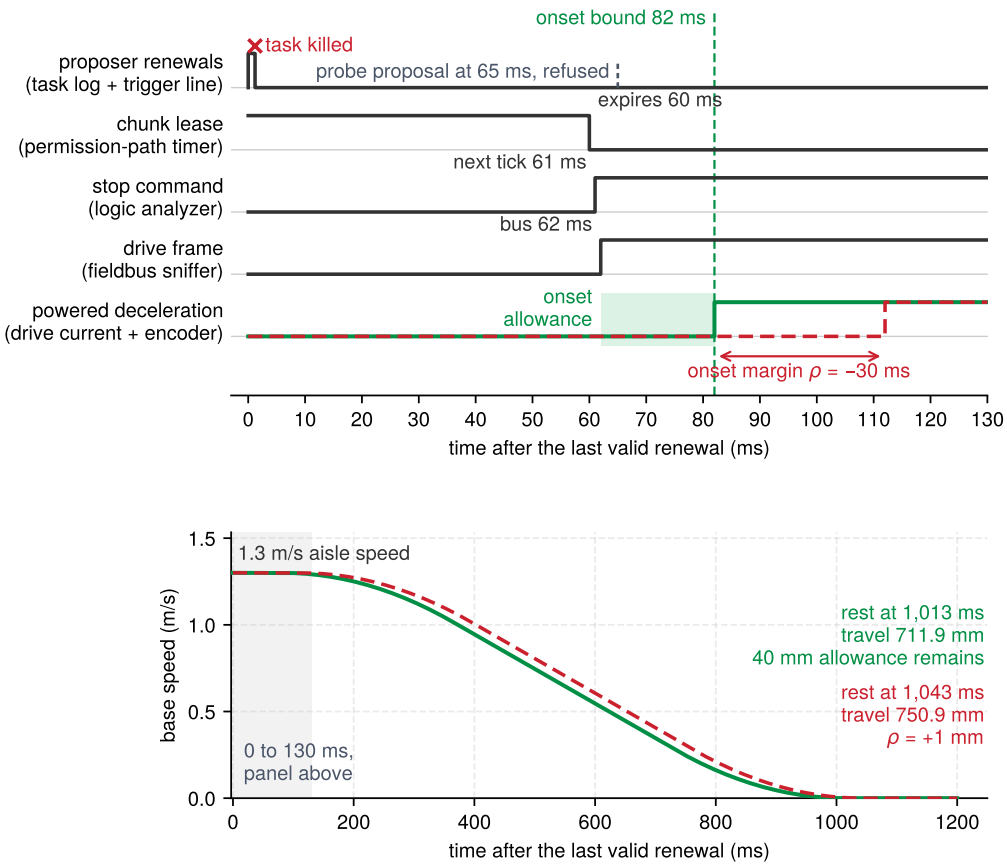


Figure 15.4: One F1 trace against its oracle: Model timelines, not measurements, of the channels an F1 trial must witness at 1.3 m/s, timed from the last valid renewal. The chunk lease expires at 60 ms, the next permission tick commands the stop, the fieldbus delivers it, and powered deceleration must begin by 82 ms; a proposal probed after expiry is refused. The dashed trace brakes 30 ms late, so its onset margin is -30 ms and its 750.9 mm of nominal travel retains 1 mm against the 751.9 mm oracle. The on-time curve travels 711.9 mm; the tracking inset is an allowance in the oracle, not extra travel in either curve.

where $B_k(t)$ are time-dependent basis functions and ψ_k are bounded parameter coefficients.

The temporal logic falsification problem finds the worst-case parameter combination that minimizes quantitative robustness:

$$\min_{\mathbf{x}_0 \in \mathcal{X}_0, \psi \in \Psi} \rho(\phi, s(\mathbf{x}_0, w_{\text{dist}}(\psi)), 0) \quad \text{subject to} \quad \|w_{\text{dist}}(t;\psi)\|_\infty \leq \Delta_w, \quad \forall t \in [0, T]. \tag{15.4}$$

The disturbance budget Δ_w in equation 15.4 bounds the peak magnitude of optical, acoustic, contact, and execution disturbances derived from the operational envelope. Yet magnitude bounds alone do not guarantee that a test bench can deliver the waveform. The basis functions, coefficient bounds, and experimental test fixtures must also respect realistic rates of change and duration limits. For example, a programmable load bank that cannot slew fast enough to replicate a steep simulated voltage transient leaves that synthesized counterexample untested on hardware.

A verification campaign must therefore log both the synthesized disturbance command and its independently measured physical trace. If the test fixture fails to deliver the precommitted disturbance trajectory within specified tolerances, the trial is `INVALID` for that test condition even if the machine comes to a safe stop. Conversely, while an empirically realized counterexample refutes a safety claim outright, a search that finds no violation within a restricted parameter family leaves arbitrary disturbances outside that basis open to question.

15.9.3 Selecting a search method

Frictional contact transitions and discrete controller state jumps often make the quantitative robustness landscape nonsmooth and non-convex, causing gradient descent to stall or step in the wrong direction unless those transitions are smoothed or handled explicitly. Derivative-free optimizers avoid this trap. The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) (Hansen and Ostermeier 2001) updates a multivariate normal distribution over disturbance parameters across generations, discovering correlated failure modes across speed, surface friction, and execution timing. When physical trials dominate the cost because each run requires mechanical resets, hardware inspection, and thermal cool-down, Bayesian optimization[10] fits a probabilistic surrogate model to previous runs to choose the next test point. Neither method guarantees convergence to a global minimum over a finite campaign, so the verification record must preserve search parameter bounds, random seeds, evaluated test vectors, and fixture limits so that auditors can see exactly which regions were challenged.

Two software toolboxes implement this search loop for cyber-physical models: S-TaLiRo (Annpureddy et al. 2011) and Breach (Donzé 2010). Both ingest a formal Signal Temporal Logic requirement and a simulation model, compute quantitative robustness on each simulated rollout, and steer stochastic or local optimizers toward negative robustness. A related family of algorithms treats the disturbance sequence as a sequential decision process. **Adaptive stress testing** (R. Lee et al. 2020) explores the disturbance sequence via Monte Carlo tree search or reinforcement learning to find the most probable path to failure rather than an arbitrary, physically improbable edge case, ranking counterexamples by operational plausibility. A survey of these black-box methods groups them by their output (Corso et al. 2021): finding an isolated counterexample, finding the most likely failure trajectory, or estimating an aggregate failure probability. Only the third output measures operational risk, using the estimation procedure developed in section 15.10.2. Whatever search method is chosen, it returns candidate counterexamples and measured robustness margins; when those margins come from a simulator, counting them toward a physical safety claim requires bounding how far the simulator's trajectories can drift from the real machine's.

[10] **Bayesian Optimization**: Originally developed for computationally expensive engineering simulations, Bayesian optimization fits a Gaussian-process surrogate across completed trials and selects the next sample point by maximizing expected improvement (Jones et al. 1998). Physical fault-injection rigs face the same cost structure, because each trial requires mechanical resets, hardware inspection, and thermal stabilization delays. Still, the surrogate's confidence is only a model prediction: a region it estimates to be safe has not been physically tested simply because the surrogate predicts a positive margin.

15.9.4 Bounding the sim-to-real transfer gap

A simulator that passes every fault-injection rollout can still hide a physical crash if its modeled trajectories differ from the real machine's by more than the simulated safety margin. While high-throughput software-in-the-loop simulation is indispensable for exploring combinatorial corner cases, an unvalidated simulator can confirm only what its underlying model assumes. To use simulation sweeps as evidence in a safety case, the engineering team must validate and bound the **sim-to-real transfer gap** ϵ_{gap}.

Let $s_{\text{sim}}(t;\xi,\mathbf{u}) \in \mathcal{S}$ and $s_{\text{real}}(t;\xi,\mathbf{u}) \in \mathcal{S}$ denote the system state trajectories in simulation and on target physical hardware under matched declared conditions ξ (initial speed, payload mass, floor friction) and control input history $\mathbf{u}$. Replaying identical open-loop input sequences establishes a baseline comparison of plant dynamics, but certifying a deployed closed-loop policy requires bounding discrepancy under autonomous execution, where perturbed sensor observations produce divergent actuator commands. The continuous transfer discrepancy is the supremum state error across the test horizon $[0,T]$ under a task-weighted physical norm:

$$\epsilon_{\text{gap}} = \sup_{t\in[0,T]} \|s_{\text{real}}(t) - s_{\text{sim}}(t)\|_W \tag{15.5}$$

where $\|\cdot\|_W$ normalizes position, velocity, and actuator torque errors by predeclared positive reference scales in meters, meters per second, and newton-meters, respectively, before applying dimensionless task weights. The verification plan fixes those scales and weights in advance; the resulting discrepancy ϵ_{gap} is dimensionless, while the Lipschitz constant L_ϕ carries the physical units of the predicate margin.

A sound safety warrant requires a discrepancy bound that holds across all trajectories in the claimed operational envelope, not just the largest error observed in a small test sample. That modeling error must be deducted directly from the simulated robustness margin. Consider a Signal Temporal Logic safety requirement ϕ whose atomic predicates $f(s(t)) \geq 0$ are Lipschitz continuous with constant $L_\phi = \sup_{s_1\neq s_2} |f(s_1) - f(s_2)|/\|s_1 - s_2\|_W$. Here L_ϕ is the maximum rate at which

the predicate margin degrades per unit of normalized state discrepancy. For a spatial clearance predicate evaluated in meters against position error normalized by a 1 m reference scale with unit task weight, L_ϕ is 1 m per unit of normalized discrepancy. By Lipschitz continuity, the physical predicate error is bounded pointwise by the state discrepancy:

$$|f(s_{\text{real}}(t)) - f(s_{\text{sim}}(t))| \leq L_\phi \|s_{\text{real}}(t) - s_{\text{sim}}(t)\|_W \leq L_\phi \epsilon_{\text{gap}}$$

which guarantees that at every instant $t \in [0, T]$:

$$f(s_{\text{real}}(t)) \geq f(s_{\text{sim}}(t)) - L_\phi \epsilon_{\text{gap}}.$$

Taking the infimum across the temporal horizon $t \in [0, T]$ for an invariance property $\phi \equiv \square_{[0,T]}(f(s(t)) \geq 0)$, the guaranteed lower bound on physical robustness in the real world is:

$$\rho_{\text{real}} \geq \rho_{\text{sim}} - L_\phi \epsilon_{\text{gap}}. \tag{15.6}$$

The term $L_\phi \epsilon_{\text{gap}}$ in equation 15.6 acts as a **sim-to-real robustness discount**. If simulated robustness ρ_{sim} does not exceed $L_\phi \epsilon_{\text{gap}}$, the simulation pass guarantees no physical margin at all: the real machine can breach the safe invariant set ($\rho_{\text{real}} < 0$) even when every simulated rollout returns a positive margin.

The rack-end claim makes this discount concrete. Scoring the distance predicate in meters and normalizing position error by a 1 m reference scale with unit task weight gives a Lipschitz constant of 1 m per unit of normalized discrepancy. Suppose a simulated stop finishes with the entire 131.0 mm reserve intact. The lower bound in equation 15.6 certifies a positive physical margin when $\epsilon_{\text{gap}} < 0.131$, meaning the justified position-error bound is smaller than the reserve under this normalization. A larger discrepancy bound leaves the actual physical margin unresolved. A simulator whose stopping position is experimentally validated to within a few centimeters transfers nearly all of its verified margin; a simulator that has never been calibrated against physical stopping runs transfers none of it.

Coupled cyber-physical dynamics determine what this discount must cover: drivetrain compliance and gear backlash alter delivered torque, surface contamination and tire wear change braking traction, and real-time execution jitter shifts braking onset time. A discrepancy bound calibrated on an unloaded, cool machine cannot be silently extended to a loaded vehicle operating under thermal throttling. The verification record must state the operational envelope across which the discrepancy bound holds, keeping that justified bound distinct from the empirical maximum error observed during paired test runs.

Measuring the empirical discrepancy $\hat{\epsilon}_{\text{gap}}$ builds on the paired probing protocols used to qualify simulation models for policy training. These protocols drive identical command histories into both simulated and physical hardware and score channel residuals against precommitted tolerances (section 6.4.2). They are the same measurements that bound a simulator's screening validity in section 7.4.2. Verification adapts this qualification procedure in two ways. First, excitation inputs are tailored to the specific safety invariant under test, such as full-rate deceleration from aisle speed for the rack-end claim rather than low-speed docking maneuvers. Second, physical trajectories must be captured with external instrumentation independent of the robot's onboard sensor suite, so that shared sensor calibration errors or estimator drift cannot cancel out of the residual. The resulting empirical discrepancy $\hat{\epsilon}_{\text{gap}}$ is archived in the test records alongside the environmental conditions under which it was gathered. If the sim-to-real safety case relies on domain randomization distributions $\mathcal{D}_{\text{DR}}$ to cover hardware variations, it must justify set inclusion across all declared operational conditions:

$$\mathcal{S}_{\text{real}}(\xi) \subseteq \bigcup_{\tilde{\xi} \sim \mathcal{D}_{\text{DR}}} \mathcal{S}_{\text{sim}}(\tilde{\xi}).$$

Finite paired measurements can falsify this inclusion by exposing an outlier, but they cannot prove it globally. If unmodeled physical effects such as fieldbus latency jitter, magnetic stator saturation, transmission backlash, or Coulomb stiction cause physical trajectories to escape the randomized simulation bundle, the sim-to-real transfer guarantee fails, and software-in-the-loop simulation passes cannot serve as evidence in the release safety case.

Checkpoint 15.3: Search, robustness, and transfer

A signed margin turns testing into search, and a simulated margin counts toward a physical claim only after its transfer discount.

- ☐ **Quantitative robustness interpretation**: Can you explain how Signal Temporal Logic replaces binary pass/fail assertions with a signed predicate margin, and how its units and reference limit determine the meaning of a negative score?
- ☐ **Adversarial optimization of disturbance trajectories**: Can you describe how black-box optimizers use quantitative robustness scores to discover worst-case combinations of latency, noise, and friction?
- ☐ **Test invalidation vs. failure criteria**: Can you distinguish between an execution that falsifies a safety invariant and an invalid trial where testbed preconditions or fault injection mechanisms failed?
- ☐ **Normalization scale estimate**: For the late F1 trace of section 15.9.1, suppose the record had fixed the distance scale at the 40 mm tracking inset while keeping the 82 ms onset scale. Can you estimate both normalized margins and say whether the governing predicate changes for this trace?
- ☐ **Transfer discount**: A simulated F1 stop keeps 131.0 mm of reserve. Can you state the largest discrepancy bound that still guarantees a positive physical margin, and explain why a bound measured on an unloaded, cool machine cannot certify a loaded, throttled one?

15.10 Interpreting Campaign Statistics

Counting passed trials in an adversarial campaign can easily mislead a release review into treating a test-bench success rate as an operational reliability number. Any statistical summary describes only the specific population sampled by the test harness, and conditions that were never tested remain missing evidence no matter how sophisticated the statistical model applied afterward.

When n independent Bernoulli trials sampled from a declared fault distribution yield zero missed mitigations, the outcome puts a one-sided upper confidence limit on the miss probability for that specific distribution. This is the zero-failure trial sizing of section 7.7.1, applied here to an injected hardware fault population rather than closed-loop policy rollouts; at a 95 percent confidence level, it requires about $3/p_{\text{fail}}$ consecutive successful injections to bound the miss probability below p_{fail}. To turn these injection counts into a hardware coverage claim weighted by each component's failure rate in FIT (failures in time, one per 10^9 operating hours; notebook 15.3 gives the component weights λ_i), the injection harness must sample faults in direct proportion to their raw failure rates λ_i. Otherwise, the verification pipeline must apply an explicitly justified stratified weighting across each fault stratum.[11]

This statistical bound holds only when the physical trials satisfy the declared independence and sampling assumptions. On real hardware, repeated high-energy stops produce Joule heating (I^2R) in the drive motor windings and mechanical brake linings. That heat buildup changes winding resistance and lining friction coefficients, coupling successive trials. As a result, a test campaign run on a dry aisle at room temperature cannot support a confidence bound for a contaminated floor or for the operating envelope's 5 °C thermal extreme.

Notebook 15.3 draws its rates from four component strata (sensors, compute, transceivers and power management, gate drivers), and each stratum maps onto rows of table 15.2. Sensor faults are injected as F2 and as the physical transducer disturbances, compute faults as F1, transceiver and power-management faults as F3 and F4, and gate-driver faults as the drive that keeps producing torque after a stop command (section 15.6.2). Sampling in proportion to those rates therefore fixes how the campaign's trials divide among the rows.

[11] **Trial sampling and aging**: The binomial count assumes independent, identically distributed Bernoulli outcomes under the declared injection distribution; it does not require Poisson event arrivals. Repeated heating, mechanical wear, or contact changes can break independence and require a state-stratified experimental design.

Napkin Math 15.3: The coverage count

Problem: Illustrative component fault rates and a chosen ceiling on the unmitigated rate fix the diagnostic coverage the machine needs. How many zero-miss injections, sampled in proportion to those rates, would demonstrate that coverage?

Variables:

- Component fault rates λ_i in FIT, one fault per 10^9 operating hours (Illustrative): sensors 1,500, compute 1,000, transceivers and power management 2,500, and gate drivers 4,800.
- Target unmitigated rate $\lambda_{\text{target}} = 100$ FIT, with a stricter 10 FIT alternative, at confidence $C = 95$ percent (Chosen).
- Diagnostic coverage C_{diag}, the λ_i-weighted fraction of faults whose hazardous effect is mitigated, so that $\lambda_{\text{unmitigated}} = \lambda_{\text{raw}}(1 - C_{\text{diag}})$ and a single injection misses with probability $p_{\text{fail}} = 1 - C_{\text{diag}}$.

Math:

Required coverage.

$$\lambda_{\text{raw}} = 1500 + 1000 + 2500 + 4800 = 9800 \text{ FIT}, \qquad C_{\text{diag}} \geq 1 - \frac{\lambda_{\text{target}}}{\lambda_{\text{raw}}} = 1 - \frac{100}{9800} = 0.9898$$

Injections to demonstrate it, from the zero-failure count of section 7.7.1.

$$n \geq \frac{\ln(1-C)}{\ln C_{\text{diag}}} = \frac{\ln 0.05}{\ln 0.9898} = 292.1 \;\Rightarrow\; n = 293, \qquad \frac{3}{p} = \frac{3\lambda_{\text{raw}}}{\lambda_{\text{target}}} = \frac{3 \times 9800}{100} = 294$$

Sensitivity. The count scales as $3\lambda_{\text{raw}}/\lambda_{\text{target}}$, so the 10 FIT target needs 99.90 percent coverage and 2,935 injections, against a rule-of-three estimate of 2,940.

Result: A 100 FIT target needs 98.98 percent coverage, which 293 zero-miss injections demonstrate at 95 percent confidence.

Systems insight: The target unmitigated failure rate, not the number of entries in the fault list, governs campaign size. Every tenfold tightening of the target rate demands roughly a tenfold increase in physical injection runs. Furthermore, this sample size holds only when injections are drawn in proportion to each component's raw failure rate λ_i. Injecting uniformly across components with disparate failure rates tests a distorted distribution unless the harness weights the empirical outcomes and gathers enough trials within every stratum. Every trial must therefore log the exact injected mode, whether hardware detection triggered within the deadline, and whether the safety controller brought the plant to a verified safe state.

Even within the declared fault strata, a campaign rarely reaches every combination of load, floor friction, and temperature. The fault-record set must therefore catalog omitted test cases as carefully as completed runs, recording every parameter region skipped because of testbed physical limits, sensor bandwidth limits, or destructive testing costs. Unprobed regions cannot count as verified safety margins in the release case, nor can passive operational hours accumulated outside the test cell fill those gaps.

15.10.1 Exposure and the population a statistic describes

Logging thousands of incident-free fleet hours in normal warehouse aisles cannot bound rare safety-critical faults or substitute for targeted fault injection, because passive operation samples a different distribution from deliberate fault trials. Fleet exposure runs into the exposure wall of equation 7.9: bounding a hazard rate λ_{target} at 95 percent confidence requires about $3/\lambda_{\text{target}}$ failure-free operating hours, which Butler and Finelli (1993) showed is unreachable at safety-critical rates such as 10^{-9} failures per hour. Pooling those hours across a fleet adds exposure time but not environmental diversity, so the bound covers only the conditions the pooled machines share (section 7.9), and normal fleet traffic rarely triggers the corner-case hardware faults this chapter injects. Verification

also has to ask which configuration those hours describe. Any change to a safety filter, control policy, or actuator driver can alter the sampled outcomes, so earlier fleet hours count toward the updated configuration only when an explicit engineering argument shows that the change left the counted events unaffected. When a rare failure rate must be estimated across a high-dimensional disturbance space without waiting for billions of passive rollouts, the campaign must steer samples toward the failure region and weight them back to the operational distribution.

15.10.2 Estimating a rare failure rate by importance sampling

Naive Monte Carlo sampling from nominal operating conditions cannot estimate rare safety-critical failure rates because nearly every rollout ends in an unremarkable, safe stop, requiring billions of runs to observe even a handful of violations (section 15.10.1). While adversarial falsification finds whether an unsafe trajectory exists, release decisions also need to know how often such failures occur under nominal operating conditions. **Importance sampling**[12] breaks this bottleneck by drawing rollouts from an alternative proposal distribution that concentrates samples near the failure boundary and then weighting each sample by its likelihood ratio to recover an unbiased estimate under the operational distribution.

Let the operational distribution of uncertain input parameters x be denoted by $p(x)$, and let F define the subset of conditions leading to a safety violation. Standard Monte Carlo estimation evaluates the failure probability $p_F = \Pr_p(x \in F)$ by computing the empirical failure fraction across N independent draws from p. Importance sampling instead draws samples from an engineered **proposal distribution** $q(x)$ and weights each outcome by the **likelihood ratio** $w(x) = p(x)/q(x)$:

$$\hat{p}_F = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}[x_i \in F]\, w(x_i), \qquad x_i \sim q. \tag{15.7}$$

This estimator remains mathematically unbiased as long as $q(x) > 0$ whenever $p(x)\mathbb{1}[x \in F] > 0$, since the likelihood weight corrects for the shift in sampling probability (Owen 2013). The variance of $\hat{p}_F$ depends entirely on the geometry of q. The theoretically optimal zero-variance proposal is the operational distribution p conditioned on the failure region F ($q^*(x) = p(x)\mathbb{1}[x \in F]/p_F$). Constructing q^* directly is impossible because its normalizing constant is the very quantity p_F under estimation. Practical verification pipelines must therefore construct parametric approximations to q^*.

The **cross-entropy method** (Rubinstein and Kroese 2004) systematically discovers such an approximation within a parameterized distribution family q_ψ by solving a sequence of intermediate rare-event problems. Beginning with an initial proposal aligned with the operational profile ($q_{\psi_0} = p$), the algorithm iteratively draws a sample batch from the current proposal and evaluates the performance metric. It then selects an intermediate threshold at the $(1-\beta)$ quantile so that a fraction β of the batch qualifies as elite runs. The parameter vector ψ is then updated by maximizing the likelihood-weighted log-density of these elite samples, which minimizes the Kullback-Leibler divergence (a measure of how far one probability distribution lies from another) to the optimal proposal on that intermediate slice. As the threshold steps forward across iterations, the proposal gradually shifts its probability mass toward the true failure boundary. For an exponential proposal parameterized by its mean, or a fixed-variance Gaussian proposal parameterized by its mean, this update admits an exact closed form: the updated mean is the weighted empirical mean of the elite draws.

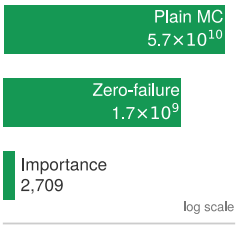
Consider this method applied to the rack-end deceleration scenario. Traveling at nominal aisle velocity, the mobile base consumes its 131.0 mm stopping reserve from section 15.2 in $T_{\text{deadline,fallback}} =$ 100.8 ms, the latest-fallback deadline of equation 14.4 at aisle speed. Any unbudgeted message queue delay exceeding $T_{\text{deadline,fallback}}$ exhausts this safety buffer, causing the physical vehicle to breach the clearance boundary. To establish an analytical baseline, assume an illustrative operational distribution where queue delays T_q follow an exponential distribution with mean $\bar{T}_q = 5$ ms (in production verification, latency measurements gathered under the declared load must justify or replace this model). Under this formulation, the analytical failure probability is $p_F = e^{-T_{\text{deadline,fallback}}/\bar{T}_q} = 1.8 \times 10^{-9}$ per stopping event. This closed-form quantity serves as an exact ground truth against which to benchmark competing sampling estimators. The threshold is deliberately the uncharged ceiling

[12] **Importance Sampling**: The technique originated in Monte Carlo calculations of nuclear radiation shielding in the early 1950s, where the few particles that penetrated dense shielding were exceptionally rare yet critical to safety. Kahn and Marshall (1953) formalized importance sampling alongside correlated sampling and statistical estimation to cut required sample counts. The method carries a sharp failure mode of its own: a proposal distribution that omits a valid failure path reports tiny sample variance and an erroneously low failure probability, a support failure that diagnostic checks must catch.

of section 15.2, which makes the example optimistic. Charging the dispatch-to-admission allocation leaves only 37.5 ms, and the same delay model then gives $p_F = 5.6 \times 10^{-4}$, about 3×10^5 times larger. The comparison below keeps the uncharged threshold because there the failure is rarest and the gap between estimators widest; at the charged threshold even plain Monte Carlo would need far fewer runs.

The sample sizing required across these estimators stems directly from the variance of sample averages. When an estimator averages N independent draws with mean p_F and per-sample variance σ^2, the estimator's standard deviation is $\sigma/\sqrt{N}$, yielding a relative standard error $r = \sqrt{\sigma^2/(p_F^2 N)}$. Achieving a target relative precision r requires $N = (\sigma^2/p_F^2)/r^2$ trials, where σ^2/p_F^2 denotes the per-draw **relative variance**. For naive Monte Carlo, each trial is an indicator variable with variance $p_F(1-p_F)$, demanding approximately $(1-p_F)/(p_F r^2)$ simulated runs. Under the illustrative delay model, securing an estimate with $r = 10$ percent relative error requires 5.7×10^{10} trials. By comparison, a zero-failure qualification campaign requires 1.7×10^9 consecutive incident-free runs to establish an upper bound at the p_F threshold (equation 7.4). Under importance sampling for this exponential model, the per-draw variance equals the weighted failure indicator's second moment minus p_F^2, which stays finite as long as the proposal rate is less than twice the operational rate.

The cross-entropy method using exponential proposals with an elite fraction of $\beta = 10$ percent demonstrates how adaptive sampling overcomes this barrier. The initial threshold is set at the 90th percentile of the operational delay distribution ($\bar{T}_q \ln 10$, or 11.5 ms). Because the exponential distribution is memoryless, the conditional distribution of delays exceeding any threshold γ is identical to the original distribution shifted by γ. The likelihood-weighted mean of the elite samples is therefore simply $\gamma + \bar{T}_q$, which immediately defines the mean for the next proposal. Because the parametric proposal family consists of unshifted exponential distributions supported on the positive real line rather than truncated distributions starting at γ, matching this conditional mean stretches the exponential scale across the entire positive axis rather than shifting the lower support boundary. Successive iterations lift the intermediate threshold to 38.0 ms and 99.1 ms. By iteration 4, the intermediate quantile surpasses the critical threshold, clamping the final level at $T_{\text{deadline,fallback}}$ with a proposal mean of 105.8 ms, the conditional expectation of the delay given a boundary violation. Under this adapted proposal, 38.6 percent of the generated rollouts land in the failure set, and their small likelihood weights scale each observed failure back to the operational probability. The resulting per-draw relative variance drops to 27.1. Consequently, only 2,709 simulation runs are needed to reach the target 10 percent relative error, a factor of 2×10^7 fewer runs than plain Monte Carlo and within 0.1 percent of the best exponential proposal. These numerical counts derive from the assumed exponential model; testing against an empirical delay histogram will yield distinct run requirements.



Weighting runs near failure cuts the runs by about 2×10^7 times.

Two statistical diagnostics safeguard the integrity of importance sampling estimates. The first is the **effective sample size**, defined as $N_{\text{eff}} = \left(\sum_{i=1}^{N} w_i\right)^2 / \sum_{i=1}^{N} w_i^2$ (Owen 2013). This metric diagnoses concentration of the likelihood weights; it does not by itself establish the precision of the weighted failure estimate, which depends on the variance of the weighted failure indicator. For the 2,709 runs above, the expected effective sample size is approximately 250; a degenerate value approaching 1 would warn that a few outlying weights dominate the summation, rendering the estimate unstable. The second diagnostic is distribution support and tail coverage. Omitting a subregion of F with positive operational probability violates the support condition and biases the estimate. A proposal that retains support but samples the operational tail too rarely can instead produce unstable or infinite variance, with deceptively small observed variance in a finite run. Sample variance cannot reveal an omitted failure path that generates no samples, so the campaign must justify coverage independently of its weight diagnostics. O'Kelly et al. (2018) applied adaptive importance sampling to end-to-end verification of autonomous driving software stacks and obtained speedups of several orders of magnitude over naive Monte Carlo in estimating rare collision rates.

This structure separates importance sampling from adversarial falsification. Although an adversarial search engine also concentrates runs near the system's safety boundaries, its sampling density is unknown. Because falsification runs lack likelihood-ratio weights, the fraction of falsification runs that fail says nothing about real-world failure frequency; in the illustrative example, 38.6 percent of the proposal runs fail even though the modeled operational failure probability is only 1.8×10^{-9}. A quantitative failure-probability estimate belongs in a safety case only when paired with its declared

operational model p, proposal distribution q, weight history, effective sample size, and confidence intervals. It remains a statement about that declared operating model rather than a direct physical measurement of the deployed hardware. Estimating how often a boundary is crossed still leaves a second question open: when a near-miss or excursion occurs, how far can the worst-case physical overshoot reach?

15.10.3 Extreme value theory and heavy-tailed safety distributions

Binary pass-fail tallies throw away the margin history of near-miss stops, while fitting a Gaussian curve to nominal bus traffic or stopping-distance data underestimates the heavy tails caused by sporadic bus arbitration contention or contact slips. Deployment qualification often has to bound the physical magnitude of worst-case excursions across continuous telemetry streams: spatial clearance $D_{\text{clear}} - d(t)$, onset slack $t_{\text{bound}} - t_{\text{onset}}$, and actuator torque headroom $\tau_{\max} - \tau_{\text{act}}$. Extreme value theory (EVT) provides limiting distributions for these tail excursions, playing a role for extreme deviations similar to that of the central limit theorem for sample means (Coles 2001). Even so, engineers must check that the data belong to the assumed tail regime rather than treating a fitted mathematical endpoint as a physical barrier. While the block-maxima formulation (which yields the Generalized Extreme Value distribution) is developed in section A.10, cyber-physical verification primarily uses the threshold-exceedance formulation, which keeps every observation that crosses a high safety threshold rather than discarding all but the block maximum.

1. **Peaks over threshold and the Generalized Pareto Distribution**: Let X denote a continuous performance metric (such as stopping-distance error), and let u represent a high baseline threshold. We model the exceedances $Y = X - u$ conditional on $X > u$. For distributions in an appropriate extreme-value domain of attraction, the Pickands-Balkema-de Haan theorem gives an asymptotic Generalized Pareto approximation to the conditional excess distribution as u approaches the upper endpoint. Balkema and Haan (1974) and Pickands (1975) proved this result independently:

$$F_{\text{GPD}}(y; \tilde{\sigma}, \xi) = 1 - \left(1 + \frac{\xi y}{\tilde{\sigma}}\right)^{-1/\xi}, \quad y > 0, \tag{15.8}$$

   defined for scale parameter $\tilde{\sigma} > 0$ on the support $1 + \xi y/\tilde{\sigma} > 0$, with the exponential distribution recovered as the continuous limit $\xi \to 0$. The **shape parameter** ξ governs tail thickness: $\xi > 0$ indicates a heavy, power-law tail; $\xi = 0$ corresponds to an exponential tail; and $\xi < 0$ defines a bounded tail with finite endpoint $u - \tilde{\sigma}/\xi$.

2. **Return level**: For a chosen return period[13] T calibrated to the sampling interval, the **return level** z_T is the threshold exceeded on average once every T periods:

$$z_T = u + \frac{\tilde{\sigma}}{\xi}\left[(T\zeta_u)^{\xi} - 1\right] \tag{15.9}$$

   where $\zeta_u = \Pr(X > u) = N_u / N_{\text{total}}$ is the empirical exceedance rate across N_{total} trials, with T expressed in equivalent trial counts. This expression follows from multiplying the conditional Generalized Pareto survival probability $(1 + \xi(z_T - u)/\tilde{\sigma})^{-1/\xi}$ by the threshold exceedance probability ζ_u and setting the resulting unconditional tail probability equal to $1/T$. In this subsection T is therefore a count of trials, not the test horizon of equation 15.4. Equation 15.9 holds for $\xi \neq 0$; its continuous limit at $\xi = 0$ is $z_T = u + \tilde{\sigma}\ln(T\zeta_u)$. Expressing a return period in calendar operating hours requires scaling by an independently justified hourly event arrival rate under a verified temporal dependence model. The product $T\zeta_u$ is dimensionless, and valid extrapolations must evaluate return levels at or above the conditioning threshold u.

[13] **Return Period**: The term comes from civil hydrology, where a "hundred-year flood" means a discharge level with a 0.01 probability of being exceeded in any single year (Gumbel 1958). The hydrological caveat applies directly to robotics: because return levels extrapolate beyond observed records, they hold only when the threshold choice and sampling distribution are physically justified. Counting return periods across synthetic fault-injection runs gives no operational safety assurance until the hourly arrival rate in service is established.

15.10.3.1 Connecting adversarial falsification to EVT tail sampling

Under nominal operation, exceedances past a conservative margin threshold occur far too rarely to fit an empirical tail; the exposure wall of section 15.10.1 starves extreme-value fits just as severely as binary failure counts. Adversarial falsification gets around this scarcity by concentrating rollouts near violation boundaries, solving the search problem of equation 15.4 over initial plant states and disturbance parameters. Let d_{budget} denote the design budget at the trial speed, 969.0 mm at aisle

speed, so that $D_{\text{clear}} - d_{\text{budget}}$ is the 131.0 mm spare. For the braking-distance overshoot metric $X = d_{\text{stop}} - d_{\text{budget}}$, the search populates the exceedance set

$$\mathcal{Y}_u = \{Y_i = X_i - u \mid X_i > u\}$$

with dense observations near the failure frontier. Those samples, however, reflect the adversary's search policy rather than the operational environment. Fitting a Generalized Pareto Distribution directly to unweighted falsification runs models the optimizer's search behavior, not the reliability of the physical plant. To inform operational risk, the runs must come from a declared proposal distribution and carry the likelihood-ratio weights of equation 15.7, so that both the exceedance probability ζ_u and the fitted parameters reflect weighted operational likelihoods.

A fitted shape parameter serves as an engineering diagnostic rather than a standalone safety verdict:

- **Positive shape ($\xi > 0$):** The fitted model extrapolates a heavy upper tail, warning of sensitivity to extreme deviations. This fit does not prove that physical overshoots are truly unbounded, but it requires the engineering team to check whether threshold choice, sensor noise, or unmodeled delays drive the heavy-tailed behavior.
- **Negative shape ($\xi < 0$):** The fitted distribution implies a finite modeled upper endpoint at $u - \tilde{\sigma}/\xi$. That mathematical endpoint must never be mistaken for a certified hardware stop or taken as proof that a runtime safety filter will always contain motion. Verifying real-world containment requires auditing the filter's dynamical assumptions and measuring its hardware response latencies under adversarial load.

15.10.3.2 Sim-to-real transfer bounds on EVT return levels

When high-throughput campaigns run on the software-in-the-loop and processor-in-the-loop (PIL) rungs of the qualification ladder, extreme value modeling produces a simulated return level $z_{T,\text{sim}}$. As shown in section 15.9.4, physical trajectories can diverge from simulated trajectories by up to the sim-to-real discrepancy bound $\epsilon_{\text{gap}} = \sup_t \|s_{\text{real}}(t) - s_{\text{sim}}(t)\|_W$. When simulated and physical trials describe the same operational population and the overshoot metric obeys the pathwise discrepancy bound with constant L_ϕ, that bound also limits the physical return level (equation 15.10):

$$z_{T,\text{real}} \leq z_{T,\text{sim}} + L_\phi \epsilon_{\text{gap}}. \tag{15.10}$$

To support a stopping-distance claim, this widened return level must fit inside the available physical reserve:

$$z_{T,\text{sim}} + L_\phi \epsilon_{\text{gap}} \leq D_{\text{clear}} - d_{\text{budget}}. \tag{15.11}$$

The margin inequality in equation 15.11 is a necessary sanity check rather than a sufficient release rule. Using it requires distributional consistency between simulation and physical testbeds, proper importance weighting when quoting operational return periods, confidence intervals on the fitted parameters, and a transfer bound ϵ_{gap} justified across the entire operating domain. A small trajectory discrepancy across a handful of bench tests does not establish an envelope-wide sim-to-real bound.

In the rack-end deceleration scenario, the physical safety margin is 131.0 mm, and only 48.7 mm once the dispatch-to-admission allocation is charged, so a return level of a few centimeters, widened by a transfer bound of a few centimeters more, already consumes it. Any proposed tail bound must state its sample population, proposal weights, parameter uncertainties, and transfer error bounds before being compared against this reserve. When a fault mode (such as mode F1) has zero measured physical trials, it supports neither an empirical tail fit nor a return-level estimate; a fitted mathematical endpoint cannot stand in for missing physical test evidence.

Every statistical metric in this section is conditional. A zero-miss count describes only the declared fault injection distribution. An exposure bound reflects only the exact hardware and software configuration that accumulated those hours. An importance-sampling estimate holds only under its declared operational model. An extreme value return level holds only relative to its fitted tail and sim-to-real discrepancy bound. Because any of these premises can fail, a verification campaign never hands release an isolated scalar margin; instead, it packages fault records that bind each empirical margin to the physical, environmental, and statistical conditions that support it.

15.11 Fault Records for Release

A raw telemetry trace sitting on a testbench digitizer cannot certify an autonomous machine for release if nothing proves which safety claim, envelope corner, or software and hardware build produced it. In an adversarial campaign, a completed F1 trial records the full event chain across independent physical channels: the final proposal renewal, the watchdog lease expiration, the active braking response, and the final mechanical stop. The release authority can credit that trial only when the measured trace is verifiably bound to the safety claim, the tested envelope point, and the exact software and hardware builds under evaluation. The fault record establishes this link by binding the structured test header to the retained evidence bundle through a cryptographic digest.

The release authority receives these claim-specific **fault records** together with the coverage records needed to judge their physical scope. Combined, they form an auditable **fault-record set** that joins the individual fault records, qualification environments, justified sim-to-real transfer bounds, statistical tail estimates, and known coverage gaps. Rather than silently omitting unmeasured or inapplicable fields, the schema marks them explicitly so that untested operating regimes cannot pass for verified boundaries. This fault-record set feeds directly into the operational safety case for release, though its schema and cryptographic signatures carry no authority beyond the physical evidence recorded inside them.

Along with design contracts, the motion-authority cases A1–A4, and the placement allocations established during system staging, the fault records draw on two upstream records: the omission log of the policy manifest (section 6.8) and the coverage-gap catalog of the evaluation record (section 7.8). Those upstream records list the operating conditions, sensor degradation modes, and environmental disturbances that the learned policy proposer never saw during training. By pairing the record header with its evidence bundle, each fault record preserves the full provenance of the trial: the asserted claim, physical operating point, fault injection mechanism, hardware and software revisions, raw high-speed telemetry, detection mechanisms, response latencies, physical stopping outcomes, and any secondary anomalies observed during the run.

DRAFT

15.11.1 The fault-record set specification

A pile of individual pass verdicts leaves a release review unable to tell which top-level safety claims have been substantiated across the envelope and which operating conditions remain untested. Every record in the fault-record set therefore exists to support one named assurance claim C within the release safety case, and its header names that claim explicitly. In the safety case structure developed in section 16.3.1, the top-level assurance claim C_0 states a core physical invariant, such as halting within the clearance corridor ($d_{\text{stop}} \leq D_{\text{clear}}$), and splits into sub-claims C_i covering specific operating envelopes, hardware fault classes, and fallback behaviors. The F1, F2, and F3 records support that stopping bound across communication and timing faults, showing that the safety permission path halts the loaded mobile base within its travel bound once proposal renewals cease (F1), when sensor observations arrive stale (F2), or when fieldbus frames drop in transit (F3). Where F1–F3 test information and timing loss, the F4 and actuator-jam records test electrical and mechanical survival. F4 shows that the held-up auxiliary power rail keeps the permission path alive through a primary supply collapse long enough to complete an emergency stop and set all mechanical holding brakes. The actuator-jam records show that the chassis stays within its lateral clearance corridor during a wheel-lock fault without driving the stator windings into thermal runaway. At the control boundary, the A1–A4 records support the claims that motion authority keeps a single writer, blocks control bypass, runs a deterministic fallback when supervisory handovers stall, and rejects forged command tokens. Alongside them, the proposer-injection records show whether tested adversarial proposals were caught by the runtime permission checks. Extending that result to every possible proposal still requires an architectural or formal argument that no source can bypass the permission path.

A physical trace supports an assurance claim C only through a structured engineering argument. In a rack-end deceleration claim, for instance, a raw brake-onset trace supplies empirical evidence, but the accompanying argument must explain why the digitizer clock synchronization, fault-injection timing, payload distribution, and floor traction conditions test the claimed physical response. Gathering evidence across the qualification ladder (formal reachability bounds for the permission filter, bounded sim-to-real falsification campaigns, and hardware fault-injection trials on the rig) lets each assurance claim C rest on complementary legs of evidence rather than a single

isolated trial. How claims, arguments, and evidence fit together into an auditable assurance case, and the notation used to write that hierarchy down, are developed in section 16.3.1.

The fault records also separate three failure domains, because each asks a different engineering question about the same physical machine, even when a compound fault spans more than one domain:

- *Functional safety*: hardware and software component faults inside the safety-related control system, such as encoder failures, dropped fieldbus frames, or auxiliary power brownouts.
- *Safety of the intended functionality* (SOTIF): hazards that arise without hardware faults from algorithmic blind spots or sensor limits, such as perceptual occlusions, out-of-distribution optical flare, or corner-case proposer errors.
- *Security*: deliberate adversarial tampering with physical sensors, communication buses, or cryptographic tokens.

The consensus engineering standards covering each subsystem of the machine, along with their publication editions, are mapped in section 16.3.4. Passing an adversarial fault-injection campaign does not by itself establish regulatory compliance or justify operation in unmodeled environments; instead, the release argument weighs exactly which claims the assembled records substantiate and where residual operational risks remain. To support that review, the individual fault records, the per-rung coverage of the qualification ladder, the empirical sim-to-real discrepancy bounds, and the extreme value theory (EVT) tail parameters assemble into one cryptographically signed set. This fault-record set is the thirteenth record in the record chain, after the authority record and before the release manifest, and it carries verification evidence to the release gate.

↪ The record chain, numbered from the limit record to the residual-claims register, is developed in section C.14.2.

The signed fault-record set defines the evidentiary handoff between adversarial testing and operational release, with its full data schema specified in table 15.5:

1. **Upstream Custody Lineage**:
   - Consumes the **authority record** (section 14.8) to check handover deadline timers (T_{timeout}, section 14.2.4), duplicate-writer rejections (A1), and supervisory token integrity.
   - Consumes the **placement record** (section 13.8) to check bus contention limits, power-rail droop tolerances, and thermal dissipation thresholds.
   - Consumes the **enforcement record** and its admitted commands (section 12.11) to check control barrier function (CBF) invariant sets, quadratic program (QP) filter execution deadlines, and fallback ladder transitions.
   - Consumes the **evaluation record** (section 7.8) to cross-check offline covariate drift and simulation test catalogs against physical target-hardware runs.
2. **Downstream Release Deliverables**:
   - Delivers to the **release manifest** and its safety case (section 16.3) the empirical, claim-specific evidence and operational bounds needed by the release argument.
   - Delivers to the **residual-claims register** (section 18.3.3) the explicit list of unverified operating regimes, test-fixture bandwidth limits, and coverage gaps.

Table 15.5: Fault-record set for release: The schema of the signed fault-record set in which each fault record reaches release, defining the multi-dimensional empirical evidence required to support operational release.

Contract Field	Type / Encoding	Physical Semantic & Evidentiary Scope	Upstream Anchor / Derivation
`set_id`	`UUIDv5`	Stable namespace-and-name identifier for the campaign; not a digest of its completed results	Fixed at test orchestration initialization

Contract Field	Type / Encoding	Physical Semantic & Evidentiary Scope	Upstream Anchor / Derivation
`set_digest`	`SHA-256`	Digest of the canonical completed set payload, excluding digest and signature fields, binding build identity, calibration identities, and evidence references	Computed after results are finalized; covered by the signed release record
`rung_coverage`	`Struct<RungCoverage>`	Executed run counts, throughputs, and fidelity mappings partitioned across the software-in-the-loop (SIL), processor-in-the-loop (PIL), hardware-in-the-loop (HIL), and physical-rig rungs	Four-stage qualification ladder (section 15.7.1, table 15.3)
`fault_injection_records`	`List<FaultRecordHeader>`	Collection of cryptographically signed fault records across F1–F4, A1–A4, and compound cascades, partitioned into `PASS`, `FAIL`, `INVALID`, and `UNOBSERVABLE` verdicts	Fault classification (table 15.2) and static fault record (table 15.1)
`sim_to_real_bounds`	`Struct<DiscrepancyBounds>`	Empirical discrepancy $\hat{\epsilon}_{\text{gap}}$, any justified envelope-wide bound ϵ_{gap} with its derivation and conditions, and the associated STL robustness discount; an observed maximum alone is not an envelope-wide bound	Sim-to-real transfer validation (section 15.9.4, equation 15.5)
`evt_tail_metrics`	`Struct<TailEstimate>`	Per estimate: declared operational distribution p, proposal distribution q, likelihood-ratio weights with effective sample size (or a flag that samples are unweighted and describe only the search), threshold u, fitted generalized Pareto distribution (GPD) parameters with confidence intervals, and return levels at a declared return period; empty where no weighted sample exists	Importance sampling and EVT (section 15.10.2, equation 15.7, equation 15.8, equation 15.9)
`coverage_deficits`	`List<DeficitEntry>`	Explicit register of omitted parameter combinations, fixture bandwidth limitations, and unverified boundary regions, defining the epistemic ceiling for release	Derived from design record inversions and bench limitations; delivered to section 16.3 and section 18.3.3
`cryptographic_signatures`	`Ed25519Signature[]`	Multi-party cryptographic signatures from test orchestration DAQ, testbed safety engineer, and lead systems architect	Hardware security module (HSM) on isolated digitizer

Capturing these eight fields in a signed record set prevents post-hoc threshold tuning and silent omission of failed or unobservable trials. Even a complete, cryptographically signed fault-record set, however, can be undermined if hidden electrical, bus, or clock couplings on the test bench corrupt the trials or blind the witnesses that score them. Four recurring pitfalls illustrate how shared

resources across watchdogs, fieldbuses, power rails, and loggers can subvert a verification campaign before a single record reaches release.

15.12 Fallacies and Pitfalls

A physical machine can pass every isolated component fault test yet catastrophically fail in deployment when a shared power rail, communication bus, or reference clock couples multiple subsystems or blinds their diagnostic monitors. These failures stem from **unintended causal, electrical, and temporal coupling** across architectural boundaries. Compound verification tests catch such cross-domain hazards only when enough galvanically and logically independent instrumentation survives the fault to establish the ground-truth sequence of events.

Fallacy: *Independent local watchdog timers guarantee end-to-end physical deadline compliance.*

A perception front-end, the chunk policy, and the safety permission monitor can each execute comfortably within their local scheduling slices while an unmonitored inter-process queue buffers stale observations between them. Local watchdog timers confirm only that isolated tasks kick their respective heartbeats within software deadlines; they reveal nothing about the total age of the physical state traversing the distributed pipeline.

Consider an asynchronous buffer that inserts delay into the machine's 133.6 ms pre-deceleration budget. The importance-sampling example of section 15.10.2 already priced this queue. At 1.3 m/s it exhausts the spare after 100.8 ms, only 75.4 percent of the nominal pre-deceleration interval, and once the dispatch-to-admission allocation is charged it does so after 37.5 ms. The fallacy lies in what the watchdogs report while it happens. Every task-level watchdog shows nominal health as the queue expands, because no task has missed its own slice. To detect this silent failure, verification experiments must isolate each stage of the causal chain: target appearance, optical capture, message dispatch, bus admission, lease expiration, and actuator torque onset. Without stage-by-stage timestamping, latency intervals are easily overlooked or double-counted.

Local watchdogs verify component execution, not end-to-end physical propagation.

Architect's countermeasure: Enforce monotonic capture-to-torque evidence epoch timestamps across all processing nodes. Compare the sensor's physical capture timestamp directly against the safety monitor's decision tick, discarding any control proposal whose total elapsed latency exceeds the safety budget regardless of healthy local thread heartbeats.

Pitfall: *Allowing diagnostic logging and high-throughput telemetry to share fieldbuses with real-time safety loops.*

Real-time fieldbuses in physical AI (such as CAN FD and EtherCAT) are engineered for deterministic, low-latency control: EtherCAT cyclic Process Data Objects (PDOs) and CAN FD priority-arbitrated frames carry microsecond-accurate torque targets and lease-renewal heartbeats. When engineering teams attempt to capture rich diagnostic traces by multiplexing high-throughput telemetry (diagnostic core dumps, raw LiDAR point clouds, uncompressed camera debug streams) through Ethernet-over-EtherCAT (EoE) or bulk CAN FD payloads, they risk overflowing communication channels and starving safety-critical control loops.

On CAN FD networks, message arbitration is non-preemptive: once a low-priority diagnostic frame begins transmission on the physical layer, a newly ready high-priority emergency-stop or torque-command frame cannot preempt it. The resulting blocking latency depends on frame length, arbitration and data bit rates, and bit stuffing. Unregulated diagnostic bursts therefore inflate queueing jitter and exhaust hard timing budgets, even though every frame on the bus is structurally valid. Because CAN controllers do not register arbitration delays or link saturation as electrical errors, hardware error counters remain clean while safety deadlines expire. Similarly, while an EtherCAT master partitions cyclic process data from acyclic mailbox traffic, shared host network interface cards, protocol stack buffers, and slave microcontroller processing pipelines can still experience severe resource contention under heavy mailbox load. If mailbox servicing delays PDO packet processing or causes host-side scheduling jitter, safety heartbeats fail.

When unbudgeted telemetry delays a safety lease renewal or drive update past its deadline, the affected watchdog must invoke its qualified local fail-safe response. Furthermore, if a physical emergency-stop command travels across that same saturated bus, transmission delays directly

consume stopping clearance. Verification trials must capture synchronized electrical communication traces alongside physical vehicle trajectories; observing a software timeout in isolation cannot prove that the machine arrested motion within its certified clearance envelope.

Unbudgeted diagnostic traffic can delay the commands and evidence needed for a bounded response.

Architect's countermeasure: Physically isolate high-bandwidth diagnostic telemetry from real-time safety buses on separate network media, or enforce rigorous protocol-specific traffic shaping. On CAN FD, this means strict frame-length caps, reserved identifier priorities, and rate-limiting ingress filters. On EtherCAT, it means dedicated cyclic and mailbox schedule partitioning with strict execution time budgets. Ethernet Time-Sensitive Networking (TSN) scheduling and frame preemption standards (IEEE 802.1Qbv and 802.1Qbu) apply only to compliant Ethernet hardware; they do not provide preemption across CAN FD or standard EtherCAT links.

Pitfall: *Relying on software anomaly detectors that share electrical power or failure modes with monitored actuators.*

Actuator dynamics, including rapid motor acceleration, locked-rotor stall currents, and sharp regenerative braking transitions, cause severe electrical disturbances on shared power rails and analog ground references. Inverter switching produces steep inductive voltage transients ($v = L_{\text{bus}}\, dI/dt$), while wiring harnesses and internal battery impedance introduce substantial voltage drops that can pull unregulated bus voltages below the minimum dropout threshold of downstream DC-DC converters. Adversarial testing campaigns (such as fault category F4) inject calibrated rail transients to measure whether these disturbances propagate into supposedly isolated sensor and safety controller supplies.

When digital transducers monitoring actuator velocity or position share a power distribution board, ground plane, or non-isolated buck regulator with motor gate drivers, supply dips and ground bounce propagate directly into sensor microcontrollers. A common failure mode is a frozen peripheral register: the transducer continues transmitting cyclically valid bus frames whose measurement payload has stopped updating. System behavior depends on representation: if an encoder freezes its cumulative position counter, a downstream velocity estimator that computes numerical derivatives will incorrectly deduce that the motor has reached zero velocity. Conversely, if the sensor freezes an internally computed velocity register, it may continuously report a nominal cruising speed.

For example, if a frozen velocity register continues reporting 1.3 m/s while the motor drive erroneously executes a full-throttle acceleration command, the telemetry frames will carry valid cyclic checksums and plausible values even though they no longer reflect physical reality. A software anomaly detector that monitors only motor telemetry cannot identify the divergence without an independent physical motion witness, such as a secondary wheel encoder, an inertial measurement unit, or optical flow sensors. The adversarial verification suite must explicitly inject both the electrical transient and the resulting register freeze, using external ground-truth tracking to monitor vehicle kinematics through the commanded stop.

A power fault that disrupts an actuator can simultaneously blind the sensor tasked with detecting it.

Architect's countermeasure: Power safety-critical transducers and monitor microcontrollers from galvanically isolated, held-up auxiliary supplies equipped with sufficient energy-storage capacitance to ride through worst-case bus transients. Transducers must be configured so that internal hardware faults cannot masquerade as plausible measurements. For analog transducers, pulling an output to 0 V fails this requirement whenever 0 V represents a valid physical reading (such as zero speed or minimum distance). Reliable systems employ a "live zero" architecture with out-of-band diagnostic voltage levels, patterned after the NAMUR NE 43 standard for industrial transmitters (NAMUR 2021). For digital fieldbuses, enforce end-to-end communication protections featuring monotonic rolling sequence counters, timestamp freshness checks, and payload CRCs to detect repeated or corrupted frames (AUTOSAR 2024). A frozen measurement register can still travel in a fresh, correctly sequenced packet, so detecting that failure also requires monitoring physical acquisition progress and cross-checking the reported state against an independent motion witness.

Fallacy: *Treating an unobservable test outcome where diagnostic loggers dropped packets as evidence of safety.*

High-severity fault injection, such as short-circuiting an inverter drive phase, opening a contactor under full load, or coupling high-voltage electromagnetic interference (EMI) onto fieldbus cabling, generates intense radiated and conducted electrical transients within the test cell. Ground bounce and high inductive kickback routinely couple into the external data acquisition (DAQ) harness, desynchronizing high-speed digitizer clocks, overflowing PCIe frame-grabber ring buffers, or crashing host logging daemons into kernel panics.

Meanwhile, the physical mobile base comes to rest: either its spring-applied mechanical brakes engaged, its drive back-drove the gearbox to a stall, or it drifted into the test cell's mechanical end-stop buffer. In an undisciplined test campaign, observers look at the stopped machine and declare the run a "PASS" ("no crash occurred and the system reached rest"), while dismissing the missing telemetry as an irrelevant "instrumentation glitch" or repeating the trial until the logger manages to survive.

This practice introduces **survivorship bias**. The trials in which the logger crashes are often those with the most violent electrical transients, which are the conditions most likely to cause the safety microcontroller to hang in an exception handler without commanding deceleration. An unrecorded trial cannot reveal which mechanism stopped the base, and an operational warehouse aisle has no end-stop buffer. Under a rigorous qualification methodology, any test run lacking continuous, verifiable telemetry must be classified as `UNOBSERVABLE` and treated as an immediate test defect. The campaign must report the explicit count of each verdict alongside all untested configurations. Summary ratios such as $N_{\text{pass}}/N_{\text{total}}$ serve as test-suite execution statistics rather than empirical failure-probability estimates derived from valid, independently scored trials. Before repeating that condition, the team must establish why the witnesses failed and qualify evidence capture under the injected environment.

Discarding unobservable trials conceals the exact vulnerability under test through survivorship bias.

Architect's countermeasure: Categorize any test run with dropped, incomplete, or corrupted evidence as `UNOBSERVABLE`, retaining it as an evidence deficit whose cause must be investigated. Report all four verdict counts. Credit a required trial toward a protective claim only when its required evidence is complete and verified; repeated observable runs do not erase missing evidence from earlier trials.

15.13 Summary

The rack-end stopping claim begins as a physical distance and timing budget. Adversarial verification transforms this kinematic budget into a concrete, testable engineering contract by declaring the operating envelope across which it must hold, including the coupled corners that one-variable-at-a-time sweeps miss. A five-part test contract (bounded claim, declared conditions, precommitted oracle, stated injection procedure, provenance) decides whether any trial counts as evidence. Within it, each postulated fault is built from a five-element chain: the negated nominal condition, the injection point, the expected detector, the bounded physical response, and an observable pass condition scored on an independent witness. For example, when the chunk policy stalls while its host processor continues transmitting nominal heartbeats, the verification harness evaluates powered deceleration onset and travel to rest referenced strictly from the timestamp of the last valid lease renewal. An automated oracle fixed before the trial classifies each run into one of four mutually exclusive outcomes: `PASS`, `FAIL`, `INVALID`, or `UNOBSERVABLE`. Test coverage is then reported dimension by dimension, and the campaign is judged by its weakest dimension rather than by one aggregate percentage.

Developing a comprehensive fault catalog requires combining multiple analytical perspectives: inverting nominal design assumptions, tracing causal failure paths forward through physical mechanisms, evaluating authority boundaries, identifying shared root causes that induce coupled failures, and modeling adversarial disturbances at physical transducers. Some failure modes bypass communication buses and hardware sensors entirely. An adversarial physical patch, an optical spoof, or a maliciously crafted task instruction steers the learned proposer directly, testing the boundary of the proposal subsystem. Because the deterministic permission path monitors physical safety invari-

ants rather than the semantic origin of a proposal, it intervenes only when an actuator command threatens a measured envelope limit. Consequently, such trials score strictly against the physical oracle. If an adversarial proposal induces an erratic or sub-optimal action that nonetheless remains entirely within the safe physical envelope, the physical predicates may pass, while the verification harness records the wrong action as a task-level coverage deficit rather than crediting it as a passed proposer-injection case.

Verification evidence must be gathered on the qualification rung that faithfully models the underlying physical mechanism. A software simulation cannot sag a power rail it does not model, whereas formal methods can verify compact, deterministic safety filters under explicit premises without establishing a closed-loop guarantee for the running machine's learned proposers. On physical test benches, hardware fault injectors operate directly at interface boundaries, while instrumentation towers record ground truth using independent clocks and isolated power supplies to prevent observer coupling. During falsification, Signal Temporal Logic provides signed robustness margins that direct search algorithms toward worst-case counterexamples. However, simulated margins cannot transfer directly to physical claims without applying a discrepancy bound justified over the claimed conditions. Similarly, importance sampling converts targeted failure searches into operational failure rates only when the sampling proposal distribution is explicitly declared, covers the entire failure domain, and applies proper likelihood weights. In rare-event analysis, extreme value models support safety claims only to the extent justified by their distributional assumptions. A counterexample refutes a safety claim, whereas the absence of one leaves the unsearched domain open rather than certified. Eight relations formalize this quantitative framework, and table 15.6 summarizes what each bounds or decides.

Table 15.6: Key equations of adversarial verification: The relations that turn a stopping budget into precommitted predicates, size a fault-injection campaign, and limit what search, simulation, and rare-event statistics can claim, with the place each is derived.

Equation	What it bounds or decides	Derived
$t_{\text{onset}} \le T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}$, and $d_{\text{F1}}(v) = v(T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}) + d_{\text{suffix}}(v) + \epsilon_{\text{track}}$	F1's two precommitted predicates, measured from the last valid renewal; the capture-referenced budget subtotal cannot stand in for them	equation 15.1
$\lambda_{\text{unmitigated}} = \lambda_{\text{raw}}(1 - C_{\text{diag}})$, $n \ge \ln(1-C)/\ln C_{\text{diag}}$	The diagnostic coverage a rate target demands, and the zero-miss injections, sampled in proportion to the fault rates, that demonstrate it	Notebook 15.3
$\Delta t_{\text{one-sigma scenario}} = \delta_{\text{skew}}\, T_{\text{blackout}} + \sigma_{\text{jitter}}$	How much of the clock-conversion bound behind F2's refusal threshold a synchronization blackout spends	equation 15.2
$\rho(\square_{[0,T]}\phi) = \min_t \rho(\phi, s, t)$, $\rho(\Diamond_{[0,\tau]}\phi) = \max_t \rho(\phi, s, t)$	The signed margin of a trace against a temporal requirement, the score a falsifier drives toward zero	equation 15.3
$\min_{\mathbf{x}_0, \psi} \rho(\phi, s(\mathbf{x}_0, w_{\text{dist}}(\psi)), 0)$ subject to $\lVert w_{\text{dist}} \rVert_\infty \le \Delta_w$	The worst initial state and realizable disturbance a search can find; a search that finds none certifies nothing	equation 15.4
$\rho_{\text{real}} \ge \rho_{\text{sim}} - L_\phi\, \epsilon_{\text{gap}}$	The physical margin a simulated margin guarantees, given a discrepancy bound justified over the claimed conditions	equation 15.6
$\hat{p}_F = \frac{1}{N} \sum_i \mathbb{1}[x_i \in F]\, w(x_i),\ x_i \sim q$	An operational failure rate from runs concentrated near failure, valid only with a declared p, a proposal that covers the failure set, and the weights	equation 15.7
$z_{T,\text{sim}} + L_\phi\, \epsilon_{\text{gap}} \le D_{\text{clear}} - d_{\text{budget}}$	Whether a discounted tail estimate fits inside the physical reserve; a necessary check, not a release rule	equation 15.11

Key Takeaways: Adversarial verification and fault injection

- **A fault tests a physical claim:** Derive the injection, detector, response, and independent witness from the stopping or contact bound. Successful task execution alone does not exercise the protective response.
- **The environment bounds the evidence:** Select each qualification rung for the mechanism it exercises. Simulation, processor execution, electrical benches, and physical rigs contribute complementary evidence with different blind spots.
- **Search finds counterexamples, not universal passes:** Signed temporal margins help prioritize disturbances near a violation. An unsuccessful search does not establish a global minimum, and a fitted tail does not prove a hard physical limit.
- **Precommitment preserves the meaning of a pass:** Fix conditions and predicates before injection. Keep `FAIL`, `INVALID`, and `UNOBSERVABLE` distinct, because missing or invalid evidence cannot support the stopping claim.
- **Verification hands release bounded evidence:** Bind each fault record to its claim, builds, conditions, and raw measurements. Coverage gaps and model assumptions remain part of the fault records, so release can restrict authority where evidence ends.

Under the fourth law, permission must be earned (principle 4), and adversarial verification produces much of the evidence that earns it. Each fault record names the claim it supports, the envelope point it covered, and the oracle fixed before the run. The authority that the release decision can grant therefore reaches exactly as far as the passing records and stops at every `INVALID`, `UNOBSERVABLE`, or untested cell. The F1 record of this chapter shows the rule in its plainest form. Its timing path, candidate envelope point, and witnesses are specified, while its loaded braking premise remains unqualified and its outcome field is empty. Until measured trials on a physical rig with independent instrumentation fill that field, the stopping claim it describes grants no authority at all.

15.14 Historical Perspective and Further Reading

Deliberate fault injection originated in the dependability community of the 1980s, where researchers applied electrical transients, pin-level shorts, and radiation directly to working microprocessors to determine whether fault-tolerant architectures could detect and contain hardware errors. Arlat et al. (1990) formalized this methodology with the MESSALINE tool, structuring each verification campaign around four operational dimensions: the injected faults, the workload activating them, the readouts captured across system boundaries, and the dependability measures derived from observations. This four-part taxonomy serves as the foundational template for the fault records developed throughout this chapter. As systems scaled in complexity, fault injection transitioned from physical pin manipulation to software instrumentation. The comprehensive survey by Hsueh et al. (1997) framed the defining engineering trade-off: hardware-implemented injection manipulates internal signals and electrical states unreachable by software, whereas software-implemented injection offers programmatic repeatability and lower operational cost at the expense of ignoring low-level hardware faults. This trade-off reappears at every rung of the cyber-physical qualification ladder. Early practitioners also discovered that monitoring hooks and profiling code unavoidably perturb execution timing and cache states, a distortion Gait (1986) termed the *probe effect*. Because sub-millisecond timing shifts can destabilize physical feedback loops, high-assurance verification architectures isolate witness observers onto independent power supplies and dedicated clock domains.

Search-based falsification and continuous verification followed a parallel lineage rooted in formal methods and statistical physics. Pnueli (1977) first introduced linear temporal logic to prove correctness properties over discrete program executions. To verify continuous and hybrid physical trajectories, Maler and Nickovic (2004) generalized temporal assertions into signal temporal logic (STL). Fainekos and Pappas (2009) subsequently equipped STL with quantitative spatial semantics, replacing binary true-false verdicts with a continuous signed distance margin. This formulation transformed the falsification problem from an intractable combinatorial search into continuous gradient-free or gradient-based optimization. Cyber-physical falsification frameworks such as S-TaLiRo (Annpureddy et al. 2011) and Breach (Donzé 2010) packaged these continuous margins to

automate the discovery of safety violations in nonlinear dynamical models. Statistical rare-event estimation has an even older heritage. Importance sampling emerged from early Monte Carlo radiation shielding calculations by Kahn and Marshall (1953), the cross-entropy method introduced adaptive proposal distributions (Rubinstein and Kroese 2004), and O'Kelly et al. (2018) synthesized both techniques to quantify catastrophic collision probabilities across autonomous driving stacks. When safety analysts must characterize worst-case physical excursions without imposing restrictive global distribution assumptions, they draw on extreme value theory. It entered engineering through the flood-frequency analyses of Gumbel (1958) and was rigorously established in its modern peaks-over-threshold form by Balkema and Haan (1974) and Pickands (1975).

Formal verification of learned neural controllers is the most recent strand. Reluplex (Katz et al. 2017) proved that modified simplex constraint solvers could formally verify input-output safety properties on small piecewise-linear networks. Subsequent bound-propagation frameworks, including α,β-CROWN (Wang et al. 2021), scaled formal guarantees to million-parameter vision networks using branch-and-bound techniques and convex relaxation. However, formal solvers remain computationally bounded; the cited results do not establish guarantees for multi-billion-parameter vision-language-action foundation models operating in closed feedback loops. The verification architecture developed in this chapter unifies these historical lineages into a cohesive systems discipline. Each trial executes against an explicit operational claim, evaluates trajectories with an oracle declared before the run, isolates observational telemetry within uncorrupted witness hardware, and exports fault records that carry measured coverage deficits into deployment safety cases.

Six foundational works trace the trajectory from classical hardware dependability to modern cyber-physical validation algorithms:

- Arlat et al. (1990) establishes fault injection as a controlled experimental science, defining the four-factor schema of faults, activation workloads, readouts, and derived measures that governs the fault records in section 15.4.2.
- Leveson (2011) introduces Systems-Theoretic Accident Model and Processes (STAMP) and System-Theoretic Process Analysis (STPA), establishing the forward hazard methodology in section 15.6.2 that identifies systemic safety violations emerging from unsafe control interactions rather than isolated component failures.
- Fainekos and Pappas (2009) formulates the continuous spatial robustness degree for temporal logic specifications over real-valued signals, providing the signed optimization margin evaluated throughout section 15.9.1.
- Owen (2013) provides a definitive mathematical foundation for importance sampling and rare-event estimation, detailing effective sample size diagnostics, weight degeneracies, and failure modes where unrepresentative proposals yield falsely optimistic variance estimates.
- Corso et al. (2021) surveys algorithmic safety validation for black-box autonomous systems, categorizing falsification approaches according to whether they target counterexample discovery, most-likely-failure trajectories, or rare failure probabilities.
- Kochenderfer et al. (2026) synthesizes trajectory falsification, failure-probability estimation, reachable set analysis, and runtime monitoring into an algorithmic discipline for validating complex autonomous systems.

What's Next: From verification to release decisions

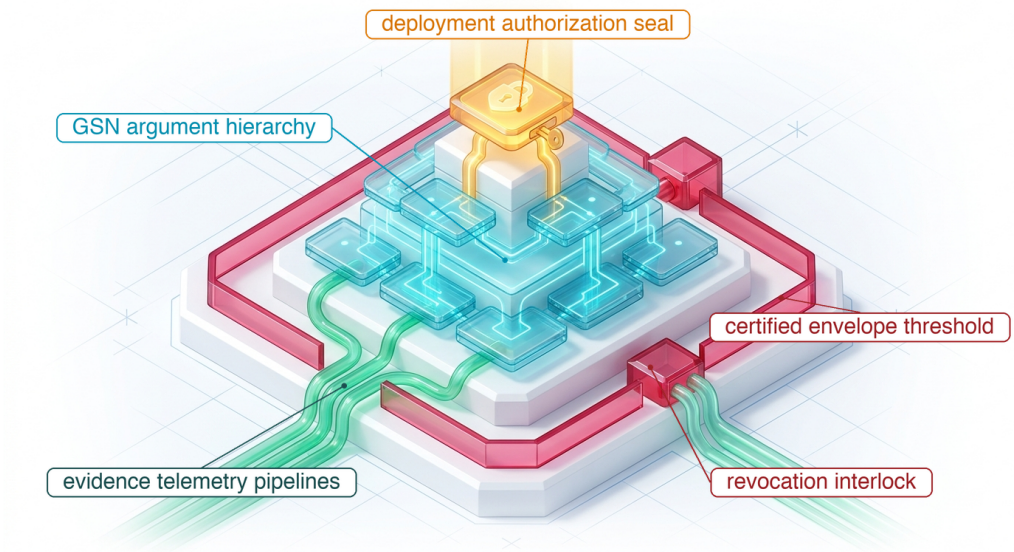
Verification delivers to the release gate a signed set of fault records, the per-rung coverage of the qualification ladder behind them, and an explicit account of the operating regimes no trial reached. For each safety claim, the set binds the corresponding fault records to any sim-to-real transfer bound (ϵ_{gap}) or rare-event tail estimate on which the claim depends, keeping measured margins, conditional model estimates, and untested regimes strictly apart. Verification records supply behavioral evidence under fault conditions, but they do not make deployment policy, and no individual fault record decides whether the machine may operate in production aisles. In chapter 16, the release process writes each claim as a physical inequality and joins these fault records with the authority record, the placement record, and the evaluation record in a safety

case. It then judges which claims are substantiated, which release verdict applies (`UNCOND`, `COND`, or `REFUSE`), and which standing conditions must govern live operation.

DRAFT

16

Deployment Release

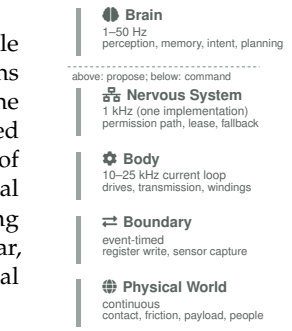


16.1 The Decision to Deploy
16.2 Physical Criteria for Release
16.3 Safety Cases and Claims
16.4 The Three Verdicts
16.5 The Verdict's Standing Conditions
16.6 Adjudicating the Safety Case
16.7 Adversarial Review
16.8 The Release Manifest
16.9 Maintaining Release Through Updates
16.10 Fallacies and Pitfalls
16.11 Summary
16.12 Historical Perspective and Further Reading

Purpose

Why can a machine that completes every trial still lack the evidence to be allowed among people?

A successful trial shows what a machine did under the conditions it happened to meet, while permission to operate is a claim about every condition it may meet next. Thousands of clean runs on a dry floor say nothing about the stop on a floor no trial touched, and a brake that stopped the machine last month may not stop it inside the same distance after its lining wears. A machine granted authority on demonstrations therefore carries untested premises into service, where the first test of each one is the event it was meant to prevent. Deciding to deploy requires stating which physical claims the evidence supports, under which conditions and for how long, refusing or narrowing operation wherever a premise remains unknown, and withdrawing the grant when brakes wear, sensors shift, or software changes. At the causal boundary, authority must expire when its physical justification does, because a later software correction cannot undo motion already produced.



Learning Objectives

- Construct a claim-argument-evidence (CAE) safety case as an auditable engineering assertion DAG linking bounded operating envelopes to qualified measurements, analysis, and operating conditions
- Distinguish input provenance from premise status, and select the verdict (`UNCOND`, `COND`, or `REFUSE`) the supported operating claim permits
- Join task-success, cycle-time, and intervention evidence to the physical measurements and restrictions supporting a conditional release
- Specify standing-condition cadences and validated responses, from brake wear to calibration drift, separately from execution leases
- Diagnose unsupported warrants, hidden contextual assumptions, and compound defeaters through adversarial review of a release argument
- Specify the records a release manifest binds by digest and the conditions under which its gate refuses motion
- Determine which claims a policy or software update reopens, and why shadow operation cannot substitute for closed-loop qualification

16.1 The Decision to Deploy

Verification left the machine with a signed set of fault records. Each pairs an observed trace under an injected fault (a stalled proposer, a stale frame, lost fieldbus frames, a sagging bus) with an oracle fixed before the trial, and the set names the operating regimes no trial reached. Beside it sit the records the earlier chapters produced: the authority record of arbitration and fallback outcomes, the placement record of compute scheduling and isolation, and the evaluation record of closed-loop task performance. Those records say how the machine behaved under the conditions tested. They do not say which operating claim the machine may now be granted, because a fault record proves nothing outside the envelope it sampled, and an untested mode is a gap rather than a pass. Offline validation accuracy and clean laboratory demonstrations sit in the same position. Release is the step that decides, claim by claim, what this evidence licenses across a named operational design domain.

↩ The empirical verification evidence backing release claims is established in section 15.8.

That step is a threshold every machine operating among humans eventually crosses, the point where builders formally grant it authority to act outside the laboratory. In conventional software engineering, deployment is largely reversible; an operator can recall a defective build within minutes, bounding the cost of failure to the time required to roll back the binary. For physical systems, that release grant differs. Every action authorized by the runtime delivers energy across the causal boundary into the physical environment. When a release decision errs, the failure appears as an actuator driving past an obstacle clearance line or a gripper exerting uncalibrated force against human tissue, and no software rollback can recall kinetic energy already transferred. Aerospace, rail transit, and industrial robotics manage this risk through an explicit safety case, a structured and auditable argument stating what the machine is claimed to do, under what operating conditions, and on what evidence. Physical AI inherits this burden with an added difficulty. A learned policy cannot be proven safe by inspecting its weights, and finite field trials cannot surface every rare tail failure, which is where the worst consequences lie.

The warehouse mobile manipulator shows why past operational success cannot by itself license deployment. In an active logistics depot it must travel narrow aisles past protruding rack ends, unlatch a spring-latched cage door, and hand a ceramic mug to a human coworker. Each subtask turns on a distinct physical claim: a stopping distance inside the clear distance at maximum payload, a bounded peak force on the latch, and a limited contact force during the handover. A spotless test run shows only that the machine executed those motions under the exact conditions of that single trial. A sound release decision must instead bound the physical variations the machine will face in production and show that verified plant dynamics and low-level safety interlocks keep risk within limits across that entire envelope.

A deployed system must also do useful work inside its certified envelope. Closed-loop evaluation logs show whether an end-to-end policy completes its tasks, how often humans must intervene, and which environmental conditions cause the policy to break down. Engineers must weigh these task metrics against the speed limits, payload caps, and spatial restrictions required by the safety case. A mobile base that trips safety stops every few meters avoids collisions while failing its service target; conversely, an agile policy that finishes every nominal trial cannot be given actuator authority until deterministic guardrails enforce safe bounds during perception failures. The release decision brings these two requirements together into one operating profile.

Because different facilities have different lighting, floor friction, and foot traffic, an initial release case applies to a single facility, drawing on its site clearance surveys, loaded braking trials, calibrated impact-force measurements, and cryptographically signed fault records. Rather than boiling deployment readiness down to a single benchmark score, the release decision is a bounded, revocable judgment that determines which operational claims the available evidence actually supports, under what physical conditions, and for how long. This chapter builds that decision in the order an adjudicator meets it: the claims are first written as testable physical inequalities over a bounded operating envelope, then organized into an argument whose every link names its evidence. The argument resolves into one of three verdicts (full operational authorization, restricted operation inside enforced limits, or refusal), and standing conditions define when a granted verdict expires. A worked adjudication applies all of this to the running machine, an adversarial review tries to break whatever case survives, and a signed release manifest binds the final decision to the executing machine so that every later software update, retrained checkpoint, or hardware modification passes through the same gate.

↩ The closed-loop evaluation record of task success, intervention rates, and breakdown conditions is established in section 7.8.

↩ The signed fault records a release case consumes are established in section 15.11.

16.2 Physical Criteria for Release

A deployment review that signs off on aggregate benchmark scores or smooth demo videos leaves the machine with no testable rule for when it may move. At a rack end, the base must stop inside the available clearance; during handover, the arm must stay within the assessed contact-force limit; and both claims depend on the permission path finishing its check before its deadline. Together, those requirements give release authorization three testable physical criteria:

$$d_{\text{stop}}(v, \tau_{\text{delay}}, a_{\text{brake}}, \delta_{\text{loc}}, \epsilon_{\text{track}}) \le D_{\text{clear}}, \quad F_{\text{contact}}(v, m_{\text{eff}}, k_H) \le F_{\text{limit}}, \quad T_{\text{exec}} \le T_{\text{deadline}}. \tag{16.1}$$

The first term abbreviates the full stopping budget of equation 12.1, whose protective margin is charged as well. In the third criterion, T_{exec} is the worst-case execution time of one permission-path cycle and T_{deadline} the deadline that cycle must meet inside its tick (section 12.7.4). A permission computed after its deadline cannot be credited for that cycle, and the qualified fallback must contain the resulting delay.

Because each criterion is a physical inequality, release authorization cannot rest on intuition, demonstrations, or aggregate benchmark scores. Authorizing deployment requires an auditable safety argument that links every inequality to measured observation ages, worst-case execution time bounds, empirical braking and contact trials, and a validated fallback. These physical bounds also depend on the individual machine's calibration: its camera and lidar sensor transforms, joint encoder zero offsets, and motor torque constants. A lost or drifted calibration invalidates the certified stopping budget even when software binaries remain unchanged. A sensor mount that shifts in service erodes the localization allowance δ_{loc} (section 16.5.1), and a firmware update that overwrites measured sensor extrinsics and joint offsets invalidates the budget just as quickly. The deployment pipeline must therefore preserve and re-verify calibration records before restoring the operating permit (section 16.9.1).

For the warehouse mobile manipulator, the release question asks whether empirical and analytical evidence satisfies these criteria for each of its three tasks inside a scoped operating envelope. The release case defines the assessed physical bounds, the specified hardware fault model, and the validated permission and fallback ladder. This argument is the safety case, which section 16.3.1 gives a formal structure, and it justifies granting a revocable operating permit while those physical conditions hold. The permit is the signed authority under which the permission path may admit the policy's tracking commands (section 16.4). Finite test runs cannot establish protection against uncharacterized or unmodeled hazards.

A release case is scoped by the operating envelope: the target operational design domain (ODD) paired with the machine's internal hardware, thermal, and timing invariants. Deployment release authorizes a system configuration under standing operating conditions, complementing rather than replacing the short execution leases that bound individual action chunks. Site inspections, calibration checks, and per-tick permission checks operate at the distinct cadences dictated by their respective failure mechanisms. The envelope also fixes what each of the three possible verdicts, defined in section 16.4, refers to. Unconditional operation (`UNCOND`) covers the full verified envelope. Conditional operation (`COND`) covers a separately evidenced subset with constrained kinematics or environmental fences. Refusal (`REFUSE`) covers none of it. For the mobile manipulator, the proposed nominal operating envelope specifies aisle travel up to 1.3 m/s (chosen; see the Reader Guide) with the tote rack loaded to at most 50 kg. It also specifies floors surveyed to a friction coefficient of at least 0.204 and travel past rack ends with at least 1.10 m of clear distance. The friction floor is an optimistic screen obtained by dividing the required braking deceleration by standard gravity under the assumption that all vehicle weight rests on the driven wheels. Because passive casters carry a share of the normal load without contributing braking force, the driven wheels must generate the total stopping force from a smaller normal load, so actual qualification must include loaded wheel forces, drive settings, and stopping traces. For manipulation, the proposed envelope specifies door approaches at 0.03 m/s and coworker handovers at 0.10 m/s tool center point (TCP) speed.

↩ The operational design domain and its known absences are established in section 5.7.1.

Inside these boundaries, the learned neural policy proposes actions across the nominal task envelope, but every proposed action requires independent, per-tick permission from an isolated safety enforcer. Mode-specific conditions (table 16.1) narrow that envelope. The takeover analysis of section 14.2.4 supplies candidate intervention speeds, screening an in-loop request at 0.75 m/s and an out-of-loop crawl at 0.15 m/s. Neither screen qualifies as a standalone release limit, because each depends on a human response time that the release case has not measured. The in-loop slow-down holds only for an operator answering an expected stimulus, and the out-of-loop crawl already fails the 99th-percentile response. The robust design therefore runs the base's own local stop first and hands over control from standstill, where waiting costs time but no clearance. The restricted mode of a `COND` verdict narrows the envelope further. On a floor characterized only through the site inspection regime, the machine runs at 1 m/s, below the 1.23 m/s optimistic all-weight inspected-floor screen, while still charging dispatch-to-admission delay and maintaining a positive stopping reserve.

The rules in table 16.1 follow a strict priority ladder: refusal conditions override mode-specific restrictions, which in turn override nominal limits. Because speed ranges overlap across operational contexts, these thresholds define operating modes rather than disjoint numerical intervals. The payload row shows how a mode differs from an interval. If the tote-rack load estimate has gone stale since the last pick, the base may not transit, because its stopping budget assumes a known mass. The stationary arm may keep working under its own separately evidenced claim. An invalid estimate or a measured overload refuses both. A conditional state restricts admitted proposals through enforcer clamping. A refusal state revokes the operating permit, so the permission path admits no further tracking commands, and commands a validated fallback trajectory. Onboard wheel slip estimation cannot provide prospective clearance guarantees because it detects low friction only *after* traction has already been lost. A thin film of clear oil ($\mu \approx 0.05$) can be visually indistinguishable from dry concrete to the navigation cameras. By the time a loaded base moving at aisle speed (1.3 m/s) initiates braking on an oil patch and measures slip, it has already committed to a stopping distance that exceeds the clear distance. Consequently, the release case must justify how site surveys, contamination exclusion, and bounded inspection and response procedures preserve the assessed friction floor throughout the authorized interval. Periodic surveys alone cannot exclude a spill between inspections.

↩ The ladder of resident fallback responses that a refusal commands is established in section 12.9.

The safety case must also document machine behavior when operation encounters an envelope boundary. Event-triggered edge telemetry freezes circular buffer traces for post-incident review (section 16.9.5), while the deterministic permission path enforces physical boundaries in real time. Table 16.1 summarizes the physical dimensions maintained by runtime monitors and administrative site controls. Internal environmental conditions, such as ambient temperature, DC bus rail voltage, and compute system-on-chip (SoC) utilization, enter the release case through fault injection and stress tests conducted across their qualified ranges.

↩ The internal qualification ranges for temperature, bus voltage, and compute load that the release case inherits are established in section 15.2.

Table 16.1: Operating envelope of the warehouse mobile manipulator: Refusal rules take priority over restricted-mode rules and nominal limits. Conditional operation requires every relied-on premise to remain supported; stale payload evidence permits only separately supported stationary work, while an invalid estimate or overload refuses. The takeover speeds are preliminary screens, not qualified motion limits. The friction floor and inspected-floor ceiling assume all weight on driven wheels and require loaded stopping qualification. The displayed speed ceilings are partial calculations before dispatch-to-admission delay; signed limits require that delay and a positive reserve. Sensor rates and stopping response are plant validation obligations.

Envelope Dimension / Invariant	Nominal Envelope	Conditional Boundary	Hard Refusal	Runtime Monitor & Rate	Enforcer Action & Latency
Aisle Speed	$v \leq 1.3$ m/s	Preliminary takeover screens: 0.75 m/s in-loop and 0.15 m/s out-of-loop; the robust design stops before any handover	Measured speed above the signed 1.3 m/s (ceiling 1.44 m/s), or invalid speed estimate	Encoders and IMU, every 1 ms permission tick	Clamp admitted speed; refuse above the signed speed
Floor Friction	Surveyed $\mu \geq 0.204$ (dry concrete near 0.60)	Qualified assessment and maintained site regime ($\mu \geq 0.12$): restricted 1 m/s, below the 1.23 m/s ceiling	Unsurveyed floor or reported film; a clear oil film ($\mu \approx 0.05$) can evade the installed cameras	Floor survey and site spill procedure; no on-board measurement	Restrict speed by floor zone; refuse the aisle
Tote-Rack Payload	$0 \leq m \leq 50$ kg, valid load estimate (total at most 368 kg)	Estimate stale since the last pick: stationary arm tasks only	Load above capacity or invalid estimate	Rack load cells, at each pick	Refuse aisle transit until re-weighed
Rack-End Clearance	Surveyed $D_{\text{clear}} \geq 1.10$ m at every rack end	Shorter surveyed clearance: lower signed speed from the same budget	Unsurveyed rack end, or obstacle inside the clear distance	Site survey; lidar and navigation camera at frame rate	Speed limit per rack end; stop for a detected obstacle
Door Latch Approach	$v \leq 0.03$ m/s, strike plate at its nominal position	Original policy: nominal plate only; changed recovery candidate needs its own evaluation and release case	0.10 m/s approach or plate condition outside the selected policy case	Arm contact-force sensing in the 2 ms contact loop	Trip at 15 N; refuse the unsupported approach
Coworker Handover	TCP speed $\leq$ 0.10 m/s under `accept_item`	Coworker present without `accept_item`: hold short of the handover point	TCP speed above the signed 0.10 m/s (transient ceiling 0.24 m/s), or a clamping geometry outside the assessed transient case	Joint encoders every permission tick; arm contact-force sensing	Clamp TCP speed; refuse above the signed speed

The nominal latch row in table 16.1 scopes the baseline policy release case. Although the separately stipulated recovery campaign in section 7.8.1 evaluates a changed candidate across nominal and displaced-plate conditions, that experimental provenance does not silently extend the contact-force or timing bounds established here. Deploying the recovery policy requires its own evaluation and safety case update.

A defensible release claim must specify its plant dynamics, assumed contact geometry, and modeled fault set. The release case for the mobile manipulator rests on three criteria. The base stops inside the 1.10 m clear distance before reaching a pedestrian who has stepped out at a rack end and stopped. The arm unlatches the door without exceeding the 100 N structural limit. An item handover does not exceed 50 N of peak contact force against the coworker. The base's stopping claim assumes that the person, once in the path, stands still. A person who keeps walking toward the moving base closes the gap during the whole reaction and braking interval. As section 12.3.1 showed, sizing the budget for that approach would cut the aisle speed to the much lower walking-person ceiling. The site therefore controls aisle-end crossings procedurally, and that site crossing rule is itself a premise the release case must evidence. A complete release argument demonstrates which human contacts are excluded before impact, which detected contacts fall within the compliance model, and which

single-point faults trigger the isolated fallback. Halting motion after impact occurs does not reduce the initial shock force.

The contact criteria arrive as premises from the enforcement record rather than as calculations derived during release review. The spring-mass impact peak that grows linearly with approach speed (equation 6.7) was first met as the soft-spring exploit of the latch simulator. Section 12.7.2 specializes it to an unbraked elastic contact with a coworker's hand (equation 12.13), using the arm's effective mass and the ISO/TS 15066 biomechanical hand model.[1] The release case evaluates the operational consequences of those physical results. At the arm's maximum free-space limit of 1 m/s, even the transient impact model predicts 207 N, exceeding the 50 N criterion by a factor of 4.1×. Consequently, unseparated coworker proximity cannot be permitted at full free-space speed without additional physical containment. For a free hand that recoils upon impact, the criterion admits approach speeds up to 0.24 m/s. At the chosen handover speed of 0.10 m/s, the model predicts a transient force of 20.7 N, below the case criterion. If the coworker's hand is clamped against a rack upright or structural frame, however, the ceiling drops to 0.053 m/s. At that same 0.10 m/s approach speed, a clamped collision generates 94.9 N, which breaches the limit. The handover claim is therefore strictly a transient-contact claim, inheriting two premises: that the lumped-mass body model and effective mass accurately describe the collision, and that the handover geometry excludes pinching or clamping hazards. Because physical calculations alone cannot guarantee workspace clearance, the release case must establish both premises through empirical evidence and site inspection.

Unlike the arm's instantaneous force criteria, the mobile base's claim is an integrated distance budget. Validating this claim requires empirical evidence bounding the perception observation age, communication bus latencies, and execution lease durations. Section 12.3.1 sized that budget term by term at aisle speed (table 12.1), integrating equation 12.1 with the jerk-limited C^2 braking profile from section 11.5.2, and found the spare and the reserve that dispatch-to-admission travel leaves. Release inherits those terms as premises. In deployment authorization, the critical engineering metric is the margin of clear distance that remains unspent, and the question release adds is how large an error in a single input would spend it. Notebook 16.1 recapitulates the enforcement budget and adds that question as step 4.

[1] **ISO/TS 15066 Values as Case Criteria**: The document supplies a body model and body-region force and pressure data for transient and quasi-static contact (International Organization for Standardization 2016). ISO 10218-2:2025 incorporates the collaborative-application requirements formerly published as ISO/TS 15066; ISO/TS 15066:2016 itself remains published and is under revision (International Organization for Standardization 2025b). The 50 N handover criterion is none of those tabulated values and not an injury boundary; the site assessment chooses it for one body region, contact type, and geometry, so a change to any of the three reopens the claim instead of inheriting its margin.

Napkin Math 16.1: The aisle spare

Problem: At the 1.3 m/s aisle speed, how much of the 1.10 m clear distance does the stopping budget leave unspent, what error in a single input would spend it, and how much remains once dispatch-to-admission work is charged?

Variables:

- Aisle speed $v = 1.3$ m/s (Chosen); clear distance $D_{\text{clear}} = 1.10$ m (Illustrative, standing in for a site survey)
- Pre-brake delay $\tau_{\text{delay}} = 133.6$ ms (Derived from illustrative terms: 51.6 ms observation age plus 82 ms from the last valid renewal to brake onset)
- Credible deceleration $a_{\text{brake}} = 2$ m/s² (Illustrative) and jerk bound $j_{\max} = 7.11$ m/s³ (Derived, section 11.5.2)
- Allowances $\delta_{\text{loc}} = 50$ mm and $\epsilon_{\text{track}} = 40$ mm (Illustrative); margin $\delta_{\text{margin}} = 100$ mm (Chosen)
- Dispatch-to-admission allocation $\Delta_{\text{da}} = 63.3$ ms (Illustrative allocation, section 13.6.2)

Math:

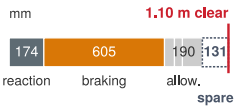
1. **Reaction**: $v\,\tau_{\text{delay}} = 1.3$ m/s $\times$ 133.6 ms $= 173.7$ mm.
2. **Braking**: Because $v \geq a_{\text{brake}}^2/j_{\max}$, the stop reaches full deceleration, and $v^2/(2a_{\text{brake}}) + v\,a_{\text{brake}}/(2j_{\max}) = 422.5$ mm $+ 182.8$ mm $= 605.3$ mm.
3. **Spare**: 173.7 mm $+$ 605.3 mm $+$ 50 mm $+$ 40 mm $+$ 100 mm $= 969.0$ mm, leaving 1,100 mm $-$ 969.0 mm $= 131.0$ mm.

4. **One input wrong**: As extra delay, 131.0 mm / 1.3 m/s = 100.8 ms. As weaker braking, setting step 2 equal to the 736.3 mm braking room gives $a = 1.39$ m/s², a 31 percent shortfall.
5. **Admission charged**: $v\,\Delta_{da} = 1.3$ m/s × 63.3 ms = 82.3 mm, leaving a reserve of 48.7 mm, or 37.5 ms at aisle speed.

Result: The budget spends all but 131.0 mm of the clear distance before admission work, and all but 48.7 mm after it.

Systems insight: The validity of the stopping guarantee $d_{stop} \le D_{clear}$ hinges on the narrow margin of unspent distance rather than the nominal distance consumed. Measured against the partial spare of step 3, a credible deceleration overstated by 31 percent spends the whole of it. The reserve that step 5 leaves after admission work is less than half that spare, so once Δ_{da} is charged a smaller braking error suffices.

This reserve must also absorb any unmodeled runtime uncertainty, so it cannot be treated as extra allowable latency or repurposed as a margin for control handovers. The reserve left after admission work, 37.5 ms at aisle speed, tolerates less unbudgeted delay or braking shortfall than the break-even values of step 4. Unless each component of the timing and kinematic chain is measured, nothing shows that the reserve survives the latencies and friction variation of a production floor. Each of the three criteria (stop, latch, and handover) therefore depends directly on premises about achievable deceleration, sensor-to-actuator latency, and contact stiffness that no calculation can certify. Because these physical properties can degrade over mechanical operating life, an auditable safety case must expose these runtime dependencies so that certification engineers can judge whether the empirical evidence supports deployment.



The partial budget leaves only 131.0 mm of the 1.10 m clear distance spare.

16.3 Safety Cases and Claims

A photograph of a clear aisle cannot establish that a loaded mobile base will stop before reaching an obstacle. A valid stopping claim depends on guaranteed physical clearance, actual braking capability, and end-to-end response timing, with each parameter supported by empirical evidence gathered under documented operating conditions. A safety case connects these physical premises to the release claim step by step, so an assessor can see which guarantees rest on measurements and which remain unverified assumptions.

16.3.1 Claims, arguments, and warrants

A bundle of test logs and benchmark plots gives a release reviewer no way to tell which physical claims the data actually support or where an unstated assumption hides. A safety case is auditable only when its logic is written down in a form that another engineer can inspect and challenge link by link. Two notations are widely used in practice, and both capture the same structure. The **claim-argument-evidence (CAE)** framework divides the case into three parts. The *claim* is a statement about system behavior that is either true or false. The *argument* is the causal or inductive reasoning connecting the claim to retained support. The *evidence* consists of the calibrated measurements, justified analyses, and validated operating procedures (Bishop and Bloomfield 1998; Bloomfield and Netkachova 2014). **Goal Structuring Notation (GSN)** draws the same structure as a directed graph: top-level goals (claims) split through explicit strategies (arguments) into concrete solutions (evidence items). Context and assumption nodes state the operating envelope within which each link holds (Kelly 1998; Kelly and Weaver 2004). The fault injection records enter this graph directly as solutions supporting each named sub-claim. Because release adjudication turns an assurance graph into a binding operational authorization, the safety case of definition 16.1 evaluates directly to an enforceable release verdict.

This decomposition comes from Stephen Toulmin's model of practical argumentation[2] (Toulmin 1958). In industrial practice, standards such as ANSI/UL 4600 require goal-based safety cases for autonomous physical products (UL Standards & Engagement 2023; Koopman 2023). Production autonomous fleets use this same structure. For example, Waymo organizes the safety case for its

↩ The fault-record set, in which every record names the one release claim it supports, is established in section 15.11.1.

2 **Toulmin's Argument Anatomy**: *The Uses of Argument* (Toulmin 1958) splits practical reasoning into claim, data, warrant, backing, qualifier, and rebuttal. Safety cases preserve this anatomy: evidence supplies the data, stated confidence acts as the qualifier, and the rebuttal becomes the defeater evaluated during adversarial review (section 16.7).

driverless ride-hailing operations around layered, dynamic claims showing the absence of unreasonable risk, arguing separately that its risk acceptance criteria are complete and that their empirical evaluation is sound (Favaro et al. 2023). For an embodied robot, the root claim C_0 states a testable physical invariant, such as a stopping distance inside the surveyed aisle clearance ($d_{\text{stop}} \leq D_{\text{clear}}$) or a bounded collaborative contact force ($F_{\text{contact}} \leq F_{\text{limit}}$). Supporting sub-claims name the required mechanical, electrical, and computational conditions, and retained measurements, derivations, or validated procedures back each leaf. Writing these relationships as a directed acyclic graph (DAG) exposes intermediate dependencies and rules out circular reasoning, so a reviewer can trace any operational claim down to its physical measurements and operating assumptions.

Definition 16.1: Claim-argument-evidence safety case

Claim-argument-evidence safety case is a structured argument that a named machine, in a stated operating envelope and under a stated fault model, meets explicit physical criteria, written as claims that decompose through stated arguments into retained evidence.

1. **Significance**: It makes a release decision checkable one link at a time. For the warehouse machine, the three inequalities of equation 16.1 become six claims whose leaves are individual premises. A reviewer can find the one premise, such as a braking deceleration no loaded trial has measured, on which the verdict turns.
2. **Distinction**: A test report or benchmark score records what happened under the conditions sampled. A safety case states why that evidence supports a claim about conditions the machine has not yet met, and names the assumptions under which the reasoning stops holding.
3. **Common pitfall**: A frequent misconception is that the case is paperwork assembled after development for a regulator. It is written with the design, and in this book's architecture its verdict decides what the permission path lets the machine do.

The soundness of this argument DAG depends on keeping evidence distinct from demonstration. A **demonstration** records successful task execution across an uncharacterized test set, such as a string of clean door openings with the strike plate at its nominal position or a thousand collision-free warehouse runs. Demonstrations show nominal performance across observed runs, but they give no mathematical or statistical bounds on tail behavior, sensor faults, or corner cases outside those logged runs. Evidence, by contrast, consists of falsifiable, calibrated records with stated uncertainty bounds: an execution-time distribution measured on a hardware logic analyzer paired with worst-case execution time (WCET) bounds, loaded stopping trials across degraded friction surfaces, or hardware-in-the-loop (HIL) fault-injection traces at envelope corners. The evidence must match the scope of the claim. Nominal trials can support nominal capability claims, and calibration logs can validate sensor coordinate frames, but bounding an envelope limit or worst-case response requires systematic coverage, stated uncertainty margins, and an explicit warrant.

Connecting evidence to a claim requires a **warrant**, the reasoning principle that explains why the measured data support the claim. For a learned neural policy, teams often reach for an inductive warrant, arguing from thousands of failure-free test episodes to future operational reliability. The exposure wall analyzed in section 7.9 shows why purely inductive warrants cannot support catastrophic hazard claims. At 95 percent confidence the zero-failure exposure wall of equation 7.9 becomes $T \geq 3.0/\lambda$ (because $\ln 20 \approx 3.0$), where λ is the failure rate per operating hour to be demonstrated. Statistically proving an ultra-low failure rate of 10^{-9} per operating hour therefore requires billions of operating hours without a single failure (Butler and Finelli 1993; Kalra and Paddock 2016a). Deciding what target rate is acceptable in the first place is an engineering and societal judgment that empirical testing cannot make on its own (Koopman 2022).

A sound safety case bridges this gap with deterministic architectural warrants. Rather than betting physical safety on policy generalization, the system grounds its top-level safety claims in an independent permission path. That path evaluates the control barrier functions of section 12.5 to clamp actuator commands, and commands an assessed safe stop whenever a learned proposal approaches the boundary of the viable set $\mathcal{C}$. The architectural warrant narrows the claim rather

than escaping physical limits. The permission path protects only against monitored hazards that the actuators can arrest within the available stopping clearance. The learned proposer still needs its own operational argument to show task capability and handle unmonitored tasks. Recent standards formalize these complementary roles. ISO/PAS 8800:2024 adapts automotive safety lifecycles, data curation rules, and assurance arguments to AI components (International Organization for Standardization 2024b). The Assurance of Machine Learning in Autonomous Systems (AMLAS) framework supplies safety-case patterns for machine learning pipelines across autonomous systems (Hawkins et al. 2021). In this book's architecture, the permission path carries the safety argument for all monitored physical hazards. It anchors its formal claims to a rated watchdog and to the certified functions beneath it in the drives, the safety-rated laser scanner, and the arm's safety controller (section 16.3.4). The learned proposer's argument can then focus on operational availability and unmonitored mission tasks.

16.3.2 Contextual assumptions and the weakest link

↪ How much an architectural shield's per-command guarantee dilutes into a system-level hazard rate is developed in section 18.3.2.

A stopping test passed on clean, dry concrete under bright warehouse lamps can fail the first time the robot enters a dim aisle if the safety case never stated the lighting assumption behind its camera exposure time. Every physical measurement rests on ambient conditions, and moving the machine into a dimly lit aisle can lengthen camera exposure and observation age even when the mechanical brakes and tires behave identically. An assurance argument must state these contextual assumptions explicitly so that measurements gathered under one set of conditions are never credited outside their verified operating envelope.

When an operating assumption is left unstated, safety depends entirely on whether a downstream check happens to catch the resulting deviation. In the mobile manipulator's stopping budget, the warrant quietly assumes that warehouse illumination permits the navigation camera to operate at its nominal exposure. If dim lighting doubles that exposure time from 16 ms to 32 ms, the observation arrives 8 ms older than assumed (measuring exposure age from mid-frame). Left unchecked, this additional latency translates to 10.4 mm of extra travel at the robot's 1.3 m/s aisle velocity, consuming roughly 8 percent of the budget's 131.0 mm margin. On this machine the permission path's frame admission age check, the freshness test of the observation contract (section 8.8), rejects the delayed frames. Because an active chunk can execute only while its evidence and lease remain valid, the robot instead halts in an aisle where the design assumed it could run. The team meets the unstated lighting assumption as unexplained availability loss rather than as a collision, and what kept the blind spot from eroding clearance was a latency check that knows nothing about illumination. A design without that check would have spent the margin silently.

Assumptions that lack runtime monitors carry far greater danger. Mechanical drift in a camera mount can introduce steady-state calibration offsets while every frame passes freshness and timestamp validations. The safety case must explicitly link localization tolerances to calibration routines and physical inspection logs. Treating fresh telemetry as proof of accurate spatial calibration leaves that warrant unsupported.

↪ Premises that no sensor on the machine can observe, and the restrictions they force on a release claim, are developed in section 18.3.1.

A safety case is not an aggregate scorecard in which high performance in one branch compensates for a deficit in another. Benchmark accuracy on an object recognition dataset cannot offset an uncharacterized latency tail on a CAN FD (Controller Area Network with Flexible Data-Rate) bus, nor can millions of simulated training episodes compensate for missing brake-wear measurements. When every child claim is *required*, the argument is conjunctive, and it is no stronger than its weakest required premise. In practice it is usually weaker, because doubt across independent premises accumulates. For a stipulated probability model, let each event mean that one required premise is valid under its stated conditions, and let $p_1, \ldots, p_n$ be the assigned event probabilities. These are not finite-trial confidence levels and estimate neither this robot's measured hazard rate nor calibrated assurance confidence. Define the illustrative claim event C exactly as the conjunction of these independent events; its probability is their product:

$$\Pr(C) = \prod_{i=1}^{n} p_i. \tag{16.2}$$

If each of 10 required premises holds with assigned probability 0.99, the conjunction holds with only about 0.90, lower than any individual premise. The mobile base's stopping claim is conjunctive

in exactly this way. Surveyed clear distance, credible braking deceleration, brake onset latency, observation age, and localization and tracking tolerances all enter a single stopping budget. A flawed warrant in any single term invalidates the root claim, regardless of how thoroughly the other terms are documented. An assessor therefore looks to the least supported required premise and the number of premises sharing the claim, not to the gross volume of supporting documentation.

This conjunctive degradation has one principled engineering exception. An argument can support a claim through two *independent* legs, either of which suffices on its own, so that the claim fails only if both legs fail simultaneously (Bloomfield and Littlewood 2003). In a separate stipulated model, define C exactly as the union of m independent sufficient-leg events with assigned probabilities $r_1, \ldots, r_m$. The event holds unless all legs fail:

$$\Pr(C) = 1 - \prod_{j=1}^{m} (1 - r_j). \tag{16.3}$$

If each of two independent legs holds with assigned probability 0.90, the combined claim holds with roughly 0.99, well above either leg alone. The equality describes the stipulated event model. If sufficient-leg events merely imply a broader physical claim, their union probability supplies a lower bound rather than automatically equaling the probability of that claim. The third law, *proposal is not permission*, instead supplies a conditional architectural argument: a policy error alone cannot breach a monitored stopping or contact invariant while the qualified enforcer, plant, sensing, timing, and fault premises hold. Policy validation and per-tick permission checks together do not justify assigning independent probabilities to their success.[3] Claim C5 in table 16.2 requires evidence that shared processors, memory buses, power rails, or sensor pipelines cannot compromise the permission path under the credited fault and interference model. Such isolation evidence addresses common causes within its qualified scope; it does not alone establish every statistical independence assumption. If the event independence is unsupported, the two-leg probability calculation is unavailable; shared causes must be represented explicitly in the argument. The enforcer leg is itself conjunctive across its own physical premises, because claims C1 through C4 must all hold for the enforcer to function. Diverse legs protect against policy failure; they cannot make up for unmeasured brake physics or missing timing margins, so the reviewer must check every leaf against the actual plant.

[3] **Independence of Diverse Legs**: When independently written versions of a software specification were evaluated on identical inputs, they failed coincidentally far more frequently than independent failure models predicted, statistically rejecting the independence assumption (Knight and Leveson 1986). Design diversity does not by itself yield independence in failure.

16.3.3 Matching the argument to the machine

Flawless arithmetic on assumed braking and latency numbers proves only that the robot would stop in time if the numbers were true; it says nothing about whether the physical machine on the floor actually meets them. While analytical stopping models relate brake onset, deceleration, sensor age, and tracking error cleanly on paper, an equation is only as sound as its physical inputs. A release reviewer must check which parameters come from measurements on the production hardware and which remain unverified design assumptions.

For collaborative manipulation tasks, biomechanical thresholds govern the contact force argument once the claim explicitly details the target body region, contact classification (transient vs. quasi-static), pressure distribution, impact geometry, and measurement methodology (International Organization for Standardization 2025b). The mobile base's claims, which form the core of the release argument, derive directly from empirical stopping distance budgets and site survey logs. Two conceptual frameworks borrowed from automotive engineering organize the fault analysis. Functional safety mitigates hazards arising from hardware malfunctions or systematic software defects (International Organization for Standardization 2018). In contrast, Safety of the Intended Functionality (SOTIF) addresses hazards stemming from performance insufficiencies or specification gaps in the absence of hardware faults, such as an obstacle detector failing to recognize a worker partially obscured by shrink-wrapped pallet staging (International Organization for Standardization 2022). Every leaf of the release argument requires support appropriate to its premise: qualified plant and timing observations, justified analysis, or validated operating restrictions under explicit assumptions. The formal standards governing these subsystems are machinery safety standards, mapped in section 16.3.4 to each component.

The claim-argument-evidence scorecard in table 16.2 defines six core release claims, pairing each physical warrant with its adjudication against the evidence developed in this book. Claim C3 shows how a warrant fails. An unnoticed oil slick on a polished concrete floor destroys wheel traction

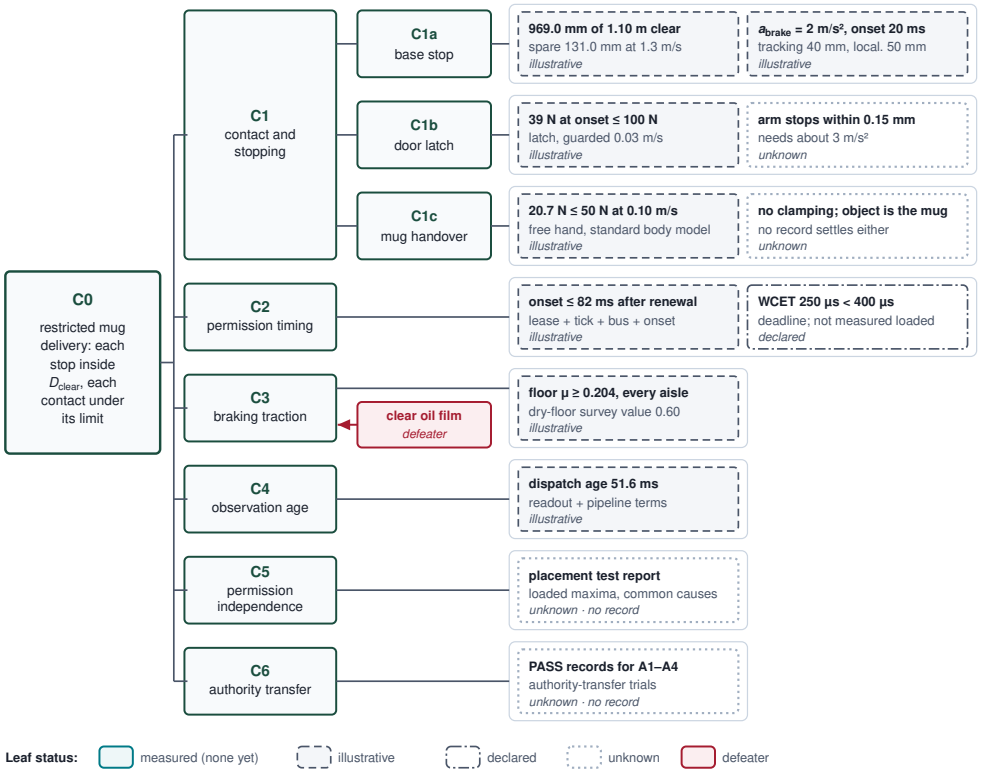


Figure 16.1: The release argument as a graph: The top claim C0 decomposes into the six claims of table 16.2, with C1 split into the base stop, the door latch, and the mug handover. Each leaf is a premise with its value, and its outline marks the status of its evidence. Every leaf is illustrative, declared, or unknown, none is measured on the proposed configuration, and the clear oil film attacks C3 directly, so the graph as drawn supports only `REFUSE`.

and invalidates the nominal deceleration model. A physically realizable condition that invalidates an assurance warrant in this manner is a **defeater** (Goodenough et al. 2013). Adversarial safety review hunts for these defeaters (section 16.7). Similarly, Claim C6 depends strictly on test records validating the four supervisory authority handoff rules (A1–A4) established in section 14.8.2. Drawn as an assurance DAG (figure 16.1), the argument splits C1 into its three physical scenarios (base stop, door latch, mug handover), ends every branch in a premise leaf whose outline records the maturity of its evidence, and attaches the oil-film defeater directly to C3. Because no leaf has been measured on the loaded production configuration, every row of the scorecard evaluates to `REFUSE`, the verdict that grants no operating permit and commands an assessed stop or hold (section 16.4). Section 16.6 works through why, branch by branch.

Table 16.2: Claim-argument-evidence review for the warehouse mobile manipulator: Six claims and their adjudication on this book's evidence. Displayed stopping distances and waiting allowances are partial calculations; admission work, physical response, and required reserve must be charged at their own events. A failed or unknown branch cannot be compensated by nominal model accuracy.

Claim	Warrant and operating assumptions on the machine	Adjudication on the book's evidence
C1. Contact and stopping	Base: partial budget of 969.0 mm before dispatch-to-admission delay, at 1.3 m/s (table 12.1) inside the 1.10 m clear distance to a person standing at the rack end, with the payload untested at 368 kg an open premise; Door: latch force of 39 N at physical braking onset, with the arm stopping within a further 0.15 mm (about 3 m/s^2) under the 100 N limit; Mug: picked from the conveyor under the 1 m/s tool center point (TCP) limit on tracked intent and handed to the coworker at 0.10 m/s, under the derived 0.24 m/s transient-contact ceiling, with a contact force of 20.7 N under the 50 N criterion for a free hand, a handover geometry that excludes clamping, and the grasped object taken to be the mug the task names	`REFUSE`: credible deceleration, brake onset, tracking and localization error, and arm deceleration are illustrative; the hand body model is a standard's population value, not a measurement of this contact; the no-clamping and grounding premises are unknown
C2. Permission timing	Brake onset no later than 82 ms after the last valid renewal; enforcer WCET of 250 µs inside the 400 µs deadline; declared assumptions: the claim covers only faults the permission path survives, lockstep diagnostics cover the path's own failure (section 13.2), and the INHIBIT rung of the fallback ladder is credited with no stopping distance, so the claim rests on the powered STOP rung (section 12.9.1)	`REFUSE`: the declared WCET lacks supporting architectural analysis and loaded stress evidence
C3. Braking traction	Optimistic all-weight friction screen of 0.204; qualified loaded stopping at the maintained friction floor is also required; a clear oil film is the defeater	`REFUSE`; the open premise passes to section 18.3.3
C4. Observation age	Navigation-camera dispatch age of 51.6 ms; evidence continues aging through admission and execution, with event-specific thresholds and a stale-data fallback	`REFUSE`: the age chain is illustrative
C5. Permission independence	The placement record combines a justified execution bound with loaded stress tests whose observed maxima stay within the declared 250 µs WCET, and so below the deadline, and common causes are tested (section 13.8)	`REFUSE` until the placement test report exists
C6. Authority transfer	One writer per actuator channel; no source bypasses a higher authority tier or the permission check; an unacknowledged handover falls back by the clearance-derived T_{timeout} of section 14.2.4 (partial allowances before Δ_{da}: 751.6 ms at the preliminary 0.75 m/s in-loop screen, 100.8 ms for a request at 1.3 m/s; because neither moving screen is a signed limit, the operative design stops the base before handing over); invalid authentication rejects the new packet without revoking an otherwise valid active owner (section 14.8.2)	`REFUSE` if any of A1–A4 lacks a `PASS` fault record (section 15.11.1); restrict to local operation if only the remote-channel cases fail

DRAFT

Checkpoint 16.1: Safety case decomposition and warrants

Before presenting a safety case for adjudication, test its structure:

- ☐ **Top-level claim falsifiability**: Can you state the top-level claim as a falsifiable physical proposition, such as a stop inside the clear distance, rather than an aspiration?
- ☐ **Warrant validity**: Can you name the warrant connecting each test bench to the claim above it, and the premises under which that warrant holds?
- ☐ **Defeater enumeration**: Can you show where a defeater such as lens fouling or bus arbitration collisions appears in the argument graph?
- ☐ **Evidentiary provenance**: Can you trace each empirical evidence node to its signed test trace and current calibration, and each analytical or procedural node to its retained derivation, assumptions, or accountable procedure and validation?

The scorecard enforces a strict separation between physical safety requirements and the empirical records available to substantiate them. Even when every internal warrant is traced to a hardware measurement, however, an engineering team does not choose its own risk-reduction hierarchy or protective ratings in a vacuum. The release reviewer must keep this evidentiary discipline across the

entire assurance tree while verifying which external machinery and regulatory standards govern each branch.

16.3.4 The standards map for the running machine

Certifying an industrial mobile manipulator against road-vehicle standards, or against fixed-arm standards that ignore a moving base, leaves the hazards of a moving arm on a moving base uncovered. Before authorizing deployment, an assessor must establish which formal standards and regulations govern each subsystem and verify where the corresponding evidence was gathered. Because this running machine is an industrial mobile manipulator rather than an autonomous road vehicle, its formal certification path follows industrial machinery directives rather than automotive standards. ISO 12100 sets the overall risk-management framework. The engineering team identifies hazards, evaluates risk, and applies a three-step reduction hierarchy, with inherently safe design first, engineered safeguards second, and user warnings and operating procedures last (International Organization for Standardization 2010). Every standard in table 16.3 addresses a specific tier of this hierarchy. Autonomy and automotive standards enter as borrowed engineering methods, supplying vocabulary for learned policies, distribution shifts, and assurance cases that traditional industrial standards are only beginning to adopt.

Table 16.3: Standards map for the warehouse mobile manipulator (as of 2026): Each document, its edition, what it governs, the part of the running machine it covers, and the section that produces the evidence it asks for. Machinery standards form the certification route; the road-vehicle and autonomy documents are borrowed method. Editions and dates change; a later printing should refresh this table.

Standard or regulation	Edition (as of 2026)	What it governs	Where it applies on the running machine	Applied in
ISO 12100 (International Organization for Standardization 2010)	2010	Risk assessment and three-step risk reduction for any machine	The hazard list behind C1 to C6	section 15.6
ISO 13849-1 (International Organization for Standardization 2023a)	2023 (4th ed.)	Safety-related parts of control systems, rated by performance level (PL a to e)	The safety MCU's lease-expiry and heartbeat watchdog that fires STO or SS1; enabling devices. The admission checks are not rated	section 12.9.3, section 13.2, section 14.3.1
IEC 61508 (International Electrotechnical Commission 2010a)	2010 (2nd ed.)	Generic functional safety of electrical, electronic, and programmable systems, rated by safety integrity level (SIL); parent of the drive and sector standards	The same watchdog and stop trigger as rated MCU firmware, and its freedom from interference	section 4.7, section 13.6
IEC 61800-5-2 (International Electrotechnical Commission 2016b)	2016 (2nd ed.)	Safety functions of power drives: Safe Torque Off (STO), Safe Stop 1 (SS1), safely limited speed	Base and arm drives; the stop sequence	section 4.6, section 12.9.2
IEC 60204-1 (International Electrotechnical Commission 2016a)	2016	Electrical equipment of machines, including stop categories 0, 1, and 2	The stop category each fallback rung realizes on the drives	section 12.9.2
ISO 3691-4 (International Organization for Standardization 2023b)	2023 (2nd ed.)	Driverless industrial trucks and their systems: personnel detection, stopping, operating zones	Mobile base in the aisles; the site crossing rule	section 12.3, section 9.7
ISO 13855 (International Organization for Standardization 2024a)	2024 (3rd ed.)	Positioning of safeguards with respect to the approach of the human body	The scanner's protective field; the approach speed v_h	section 12.9.3

Standard or regulation	Edition (as of 2026)	What it governs	Where it applies on the running machine	Applied in
ISO 10218-1 and -2 (International Organization for Standardization 2025a, 2025b)	2025	Industrial robots (Part 1); robot applications and cells, including the collaborative-application requirements formerly published as ISO/TS 15066 (Part 2). Part 2 excludes the hazards of mobility when a manipulator is integrated with a mobile platform	The arm, door opening, mug handover; not the arm moving on the base	section 12.7.2, section 14.3.1
ISO/TS 15066 (International Organization for Standardization 2016)	2016; under revision	Body model and body-region force and pressure data for collaborative contact; remains published	The hand values m_H and k_H in the handover contact case	section 12.7.2
ANSI/A3 R15.08-1 and -2 (Association for Advancing Automation 2020, 2023)	2020 (reaffirmed 2026); 2023	Industrial mobile robots in North America: the robot, including one that carries a manipulator (IMR Type C) (Part 1); IMR systems and applications (Part 2)	The whole machine, including the arm moving on the base that ISO 10218-2 excludes	This chapter; section 12.9.3
ISO 26262 (International Organization for Standardization 2018)	2018 (2nd ed.)	Road-vehicle functional safety: hazards from random hardware and systematic faults	Borrowed method: fault decomposition, confirmation of safety work	section 15.11.1
ISO 21448 (International Organization for Standardization 2022)	2022	Road-vehicle safety of the intended functionality: hazards without a fault	Borrowed method: perception and proposer insufficiencies	section 15.11.1, section 5.10.4
ANSI/UL 4600 (UL Standards & Engagement 2023)	3rd ed., 2023	Goal-based safety case for autonomous products, with safety performance indicators and independent assessment	The release argument and its field monitoring	section 16.3.1, section 16.9.5
ISO/PAS 8800 (International Organization for Standardization 2024b)	2024	Safety of AI elements in road vehicles: AI safety lifecycle, data, assurance argument	Borrowed method: the learned proposer and its data	section 5.11, section 7.8
Regulation (EU) 2023/1230 (European Parliament and Council of the European Union 2023)	Applies from 20 January 2027	EU Machinery Regulation: essential requirements, third-party assessment for machine-learning safety functions, substantial modification	The whole machine on the EU market; its over-the-air updates	section 16.9.1
Regulation (EU) 2024/2847 (European Parliament and Council of the European Union 2024b)	Applies from 11 December 2027	Cyber Resilience Act: cybersecurity requirements for products with digital elements, vulnerability handling, security updates	The update channel, the signed images, and the fleet's vulnerability reports	section 16.9.1

Two standards in the map require a clear boundary because the running machine straddles mobile robotics and fixed manipulation. ISO 10218-2:2025 governs industrial robot systems and collaborative work cells, but it explicitly excludes the kinetic hazards introduced when a manipulator rides on a mobile platform (International Organization for Standardization 2025b), which is this warehouse machine's configuration. Conversely, ISO 3691-4 treats the mobile base as a driverless industrial truck. The standard that bridges this gap by treating the mobile manipulator as a single unit is ANSI/A3 R15.08. Mounting an active manipulator on a mobile platform classifies the machine as an Industrial Mobile Robot (IMR) Type C, and Part 2 sets the safety requirements for integrating that combined unit into an active facility (Association for Advancing Automation 2020, 2023). In jurisdictions where

R15.08 lacks statutory standing, the combined hazards of mobility and manipulation fall back onto the machine's primary ISO 12100 risk assessment.

A second boundary concerns which layers of the permission path carry formal safety ratings. As section 12.9.3 established, the only rated function within the permission path is the safety MCU's lease-expiry and heartbeat watchdog, which asserts Safe Torque Off (STO) or Safe Stop 1 (SS1) on the motor drives when valid renewals cease. The admission checks that evaluate neural proposals carry no rating. The certified functions of the motor drives, the safety-rated laser scanner, and the arm's safety controller sit beneath the permission path. For release, the formal protective claims for every monitored hazard therefore rest on that watchdog and that certified layer. The release argument still uses the software admission checks for operational utility and nominal hazard prevention, but any hazard claim that depends only on unrated software remains an empirical premise rather than a certified safety function.

Statutory compliance reaches physical AI through product-safety law rather than through standalone artificial intelligence rules, and its sharpest consequence for release concerns updates. Under the European Union's Machinery Regulation (European Parliament and Council of the European Union 2023), a *substantial modification* is any physical or digital change made after deployment that the original manufacturer did not foresee and that introduces a new hazard or raises an existing risk enough to require additional safeguarding. An organization that pushes an unverified over-the-air model update can therefore become, in law, the manufacturer of the modified machine, with every obligation that role carries (section 16.9.1). The same regulation requires third-party conformity assessment[4] for machinery whose safety components have fully or partially self-evolving behavior and use machine learning to execute safety functions, so a learned safety function cannot be self-certified. Its essential requirements also constrain learning directly. The limits of each safety function must be documented in the manufacturer's risk assessment. No rule or parameter the machine generates, even during a learning phase, may change those limits where the change could create a hazardous situation (Annex III, section 1.2.1). Keeping the learned proposer outside the permission path meets that requirement only if the safety argument does not depend on the proposer's learned behavior. Two neighboring acts reach the same machine. The EU AI Act, Regulation (EU) 2024/1689 (European Parliament and Council of the European Union 2024a), as amended in July 2026 by Regulation (EU) 2026/1744 (European Parliament and Council of the European Union 2026), places the Machinery Regulation in Section B of its Annex I, so AI-specific safety rules for machines arrive through the machinery framework. The Commission must adopt delegated acts that insert those rules into the Machinery Regulation's Annex III and that apply by 2 August 2028. Because the robot receives network updates, it is also a product with digital elements under the Cyber Resilience Act (European Parliament and Council of the European Union 2024b), which requires vulnerability handling and secure updates throughout the manufacturer's support period, at least five years unless the product is expected to be in use for less. Its duty to report actively exploited vulnerabilities applies from 11 September 2026, ahead of the rest of the act. Table 16.3 lists the dates from which the other obligations apply. These milestones do not alter the underlying physics or systems engineering; they set the calendar dates by which statutory authorities will demand an inspectable assurance case.

The standards map also marks the boundary between statutory obligations and the systems architecture developed in this book. Formal standards prescribe documented arguments, rated protective functions, and independent third-party assessments. The verdicts, signed capability descriptors, and release manifest developed in the rest of this chapter are one engineering architecture for carrying those obligations into the runtime system. The safety case accepted before deployment then stays binding on the machine long after the inspectors have left the floor.

[4] **Conformity Assessment**: The formal procedure through which a manufacturer certifies that a machine fulfills all essential health and safety requirements before affixing the CE marking (European Parliament and Council of the European Union 2023). While manufacturers may self-assess conventional machinery, high-risk systems and self-evolving AI safety components require third-party evaluation by a notified body. Because the conformity assessment path is fixed before market entry, every record in the release manifest must be formatted for independent inspection.

16.4 The Three Verdicts

Treating a release sign-off as an informal paper memo forces teams either to ground a useful machine over an unmeasured corner case or to wave it onto the floor with unproven assumptions. Every row of table 16.2 ends in an adjudication, and the release decision resolves the entire assertion DAG into one of three verdicts: **unconditional operation**, **conditional operation**, or **refusal to operate**. In this book's architecture, these verdicts are cryptographically signed capability descriptors loaded into the real-time safety microcontroller before motor power stages are energized, not advisory certificates or administrative paperwork. A valid descriptor is the machine's **operating permit**:

while it holds, the permission path may admit the policy's tracking commands inside the envelope the descriptor names, and when it is revoked or absent, the permission path admits none. Although safety standards do not mandate this specific mechanism (section 16.3.4), the qualified gate and permission path enforce the authenticated descriptor's limits on physical operation. The first verdict, unconditional operation (`UNCOND`), authorizes tracking proposals across the full operating envelope specified in a signed release manifest, provided its standing conditions remain continuously verified. Each hazard claim in that safety case requires qualified measurements, justified analysis, or validated operating restrictions applicable to the specified physical plant, payload distribution, operational design domain, and fault set. Brake thermal capacity, stopping clearance, and tracking-error bounds must all be qualified for that envelope, and independent permission checks must stand wherever learned policy proposals can reach physical actuators. The verdict does not eliminate residual or newly emergent hazards; it authorizes operation solely under the assessed claims and requires detection and safe withdrawal within the qualified response budget whenever a physical premise fails.

Definition 16.2: Release verdict

Release verdict is the signed decision that sets how much authority a named machine's permission path may grant: `UNCOND` for the full assessed operating envelope, `COND` for a separately evidenced restricted envelope, or `REFUSE` when any required premise is missing, contradicted, or unknown, in each case only while its standing conditions hold.

1. **Significance**: It turns the weakest premise of the argument into the machine's operating limit on $d_{\text{stop}} \leq D_{\text{clear}}$. On this book's illustrative evidence every claim carries an unknown premise, so the warehouse machine's verdict is `REFUSE`. A stipulated set of loaded stopping and floor records would support a `COND` at 1 m/s on inspected floors, below the 1.23 m/s optimistic all-weight partial ceiling.
2. **Distinction**: A test result records the outcome of an individual trial, whereas a degraded mode is an autonomous runtime fallback triggered during execution. By contrast, a release verdict is established before deployment: it names the qualified envelope and the empirical evidence upon which it rests, while leaving the proposal boundary fixed, since the permission path continues to validate every proposed action individually.
3. **Common pitfall**: A common misconception is that a conditional verdict (`COND`) can patch an incomplete safety case by ignoring an unproven claim. A restricted envelope instead demands its own dedicated evidence and analytical limits; an unresolved premise in the primary case leaves that operating mode firmly refused.

16.4.1 Restricted operation and access boundaries

When a safety case lacks evidence for full-speed aisle travel or open-floor contact, letting the robot run under an informal operating note leaves the permission path enforcing the wrong limits. The second verdict, conditional operation (`COND`), applies when empirical evidence supports the stated hazard claims within a restricted subset of the envelope and binds the permission path to those narrower limits. In the running machine, `COND` selects a pre-validated safety configuration, much like industrial safety controllers and protective optical scanners switch between pre-validated field sets[5] and speed-monitoring profiles. All restricted kinematic thresholds ($v_{\max} \leq v_{\text{restricted}}$, $a_{\max} \leq a_{\text{restricted}}$) and spatial geofences are evaluated and validated as one configuration before release; the runtime verdict selects among these vetted profiles rather than inventing parameters on the fly. A conditional verdict (`COND`) is an offline, predeployment contract that restricts the machine's operating envelope until re-adjudication. A **degraded mode** is an online runtime fallback, pre-assessed and named in the signed release, that the permission path invokes when an active subsystem loses telemetry or sensor redundancy (section 16.5.3 sizes one for the base). Release then enforces physical, kinematic, or environmental boundaries that exclude unverified hazard states, such as de-rated joint speeds, a restricted illumination range, or a physical barrier that keeps humans outside the manipulator's workspace. Where contact force cannot be bounded for unseparated co-presence, a restricted release case may require physical separation: figure 16.2 illustrates yellow perimeter mesh fencing around

[5] **Safety Scanner Field Sets**: A safety laser scanner stores several protective-field geometries that were configured and validated in advance, and a safety controller selects one of them from encoded speed or position inputs. The scanner never computes a new field at runtime. `COND` borrows the pattern, so the verdict selects a pre-assessed limit set and no untested parameter can reach the permission path.

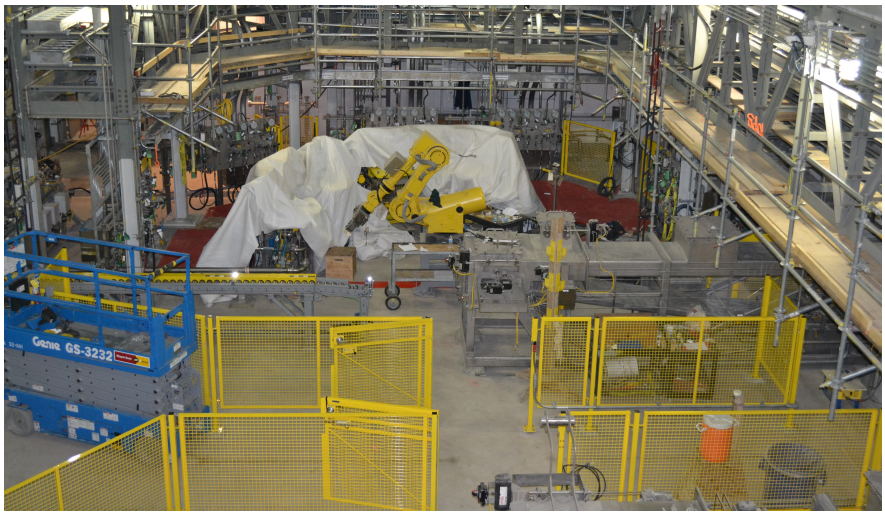


Figure 16.2: Physical separation example: A yellow industrial arm works behind yellow mesh fencing. Fencing can be one condition of a restricted release when a validated access interlock and stopping test support it. (Credit: PEO ACWA, Pueblo Chemical Agent-Destruction Pilot Plant, 2012, Wikimedia Commons, CC BY 2.0.)

an industrial workcell. Crediting that separation requires validated access-interlock and stopping evidence showing that the protective response completes before hazardous access.

Fencing excludes unseparated contact by keeping people out of the hazardous volume. The alternative that lets people share the floor, speed and separation monitoring (SSM, section 12.9.3), needs a different access claim from the stationary-person stopping claim of C1. Its separation clearance must account for human approach velocity across both system reaction time and physical deceleration, robot displacement before brake onset, the executed braking profile, and measurement uncertainty. The warehouse case retains its site crossing rule; engineering teams cannot substitute a stationary-person stopping model into this context and claim to have achieved unseparated, open-floor human-robot collaboration.

Every quantitative condition attached to a release verdict names the operational limit the permission path enforces, maintaining explicit margin below the calculated physical ceiling. Item handover illustrates this margin requirement. The contract signs a TCP speed below the absolute ceiling that the contact criterion admits under equation 12.13. The gap between these two speeds accounts for unobserved physical evolution between discrete sensor samples: velocity growth between sampling instants, encoder quantization noise, and drive lag during an enforcement clamp. A contract signed at the theoretical ceiling leaves zero margin for these physical latency terms, and the release case requires measured empirical bounds on each term before the signed operational speed can be accepted.

The inspected-floor condition follows the same discipline. Where the site inspection regime certifies a friction coefficient of only 0.12, the illustrative all-weight-on-driven-wheels screen uses the optimistic traction ceiling $\mu g = 0.12 \times 9.81\ \mathrm{m/s^2} = 1.18\ \mathrm{m/s^2}$ against the $2\ \mathrm{m/s^2}$ that the dry-floor budget assumes. Actual loaded credible deceleration also depends on the driven-wheel load share, drive and geometry conditions, and qualified stopping traces. Under the all-weight screening premise, the partial budget that admits 1.44 m/s on surveyed floors admits only 1.23 m/s under this reduced traction. Visual inspection alone cannot establish this lower friction bound because a thin film of transparent oil readily escapes visual detection. The site must qualify both the friction assessment method and the operating procedures (cleaning schedules, spill isolation, and access restrictions) that preserve the assessed floor state between inspections. If that maintenance regime cannot guarantee the exclusion of contaminants over the authorized operating interval, reducing vehicle speed does not salvage the safety case; the aisle remains refused.

The third verdict, refusal to operate (`REFUSE`), is mandatory whenever the safety case has missing warrants, uncharacterized tail latencies, incomplete fault-injection coverage, or unmonitored failure

modes. For systems where torque removal causes immediate physical harm (such as upright bipedal humanoids falling under gravity), `REFUSE` cannot simply de-energize motor phases to zero torque; it must execute an assessed, powered descent or seated hold. If the safety enforcer's worst-case execution time under the designated computational placement has never been verified against its 400 µs deadline, or if the fault-injection campaign omitted common-cause failures identified in the deployment topology, the hazard remains unquantified and cannot support release. Refusal is likewise mandatory whenever physical measurements contradict the mathematical model behind a safety budget term, such as loaded stopping trials that yield decelerations below the 2 m/s^2 assumed by the braking model.

A refusal verdict is an actionable engineering result rather than an administrative dead end. It redirects schedule pressure toward evidence by pointing to the exact broken link in the assertion DAG. If release is refused because loaded stopping trials contradict the braking model, the engineering team does not debate risk tolerance. Developers instead repeat physical braking trials at maximum payload mass until the qualified trial campaign and its uncertainty analysis support a credible lower deceleration bound. They then re-derive the stopping budget from that lower bound and submit for re-adjudication only when the aisle budget closes on validated data. The refusal verdict names the exact empirical test that must pass before re-adjudication can proceed.

Accountability for the release decision must rest with a named **lead systems adjudicator** rather than being split across development teams. When a vision team signs off on perception metrics, an actuator supplier on torque ratings, and a machine learning team on policy convergence, each group can assume that another layer holds the safety margin. The lead adjudicator alone attests that the integrated, physical machine satisfies the safety case. This sign-off binds the release verdict to a specific set of technical records: cryptographic digests of verified policy weights, calibration logs for manipulator contact-force sensing, empirical brake-onset measurements paired with justified worst-case response bounds, and fully loaded stopping trials establishing credible deceleration limits. The adjudicator signs the release manifest with a private key held inside a hardware security module,[6] binding the manifest to the organization's signing key and sign-off record. The robot runtime checks this digital signature against a public key anchored in its hardware root of trust before enabling actuators.

[6] **Hardware Security Module (HSM)**: A tamper-resistant device can generate and retain a non-exportable private key and perform signing inside its own boundary. The signature identifies the signing key; authenticated access, sign-off records, and key-custody controls establish who authorized its use.

↪ The adjudicator's signature as one of several decisions behind each admitted action, each needing a named owner, is developed in section 17.2.

The adjudicator's signature records the internal acceptance of the engineering organization, whereas external certification requires independent evaluation. UL 4600 requires independent assessment of the autonomous safety case (UL Standards & Engagement 2023), and machinery whose primary safety functions rely on machine learning falls under regulations, such as the European Union Machinery Regulation, that require third-party conformity assessment (section 16.3.4). An assessor who did not build the safety case brings an independent view to the assumptions that shaped its trade-offs. Organizational separation alone does not establish a probability of coincident error or the statistical independence required by equation 16.3. The release manifest must therefore retain the full body of raw physical evidence needed for an independent audit, not just the final verdict.

The adjudicator ultimately signs one of the three decisions summarized in table 16.4. A conditional verdict requires its own independently evidenced restricted envelope; an unresolved gap in the nominal argument cannot be relabeled as an operational constraint. No verdict ever moves the proposal boundary. Under both `UNCOND` and `COND`, learned policies only propose candidate actions while the deterministic safety enforcer independently admits or blocks every control command, as principle 3 requires. The release verdict sets the physical envelope that the permission path enforces; it never grants learned policies unchecked authority.

Table 16.4: Three signed release verdicts for the mobile manipulator: Full operation and restricted mode both retain independent per-action permission. Refusal is an actionable engineering result.

Signed verdict	Evidence required	Permission and runtime limits	Expiry or refusal trigger
`UNCOND`	Full operating-envelope claim, measured plant and timing bounds, independently validated permission and fallback	Policy proposes across full *signed* task envelope; every action passes the enforcer	Evidence, configuration, calibration, or standing premise fails

Signed verdict	Evidence required	Permission and runtime limits	Expiry or refusal trigger
`COND`	Restricted operating-envelope claim and its own evidence, limits, and fault coverage	Policy proposes only within signed reduced limits; every action passes the enforcer	Any restricted premise becomes false or unknown
`REFUSE`	Missing, contradictory, expired, or unauthenticated critical evidence	No operating permit; execute the assessed stop or hold, the fallback-ladder rung the release case validates for the plant	Re-adjudication with repaired evidence

Each verdict in table 16.4 is signed against the physical state of the machine at the moment of evaluation. The right-hand column of the table therefore pairs every operational grant with an explicit expiry or refusal trigger, because a brake lining or sensor mount that passed inspection before commissioning does not stay in its commissioning state. As the physical plant wears and drifts over time, a release verdict holds only while runtime checks, scheduled measurements, and enforced site procedures continue to verify its standing premises.

16.5 The Verdict's Standing Conditions

A signed release can lapse without any change to the underlying software. Brake linings wear down, camera brackets shift under continuous vibration, or warehouse traffic polishes concrete lanes until traction drops. When these physical premises drift, the stopping claim certified by the adjudicator no longer holds. Standing conditions bind the operational verdict to these physical baselines by defining an explicit observation cadence for each premise alongside deterministic fallback responses when a check fails. Because failure modes originate in hardware and environment, their observation schedules must follow physical degradation mechanisms rather than software sprint schedules. An assurance case maintained this way, its validity updated throughout operating life as monitored evidence shifts, is what the safety literature calls a *dynamic safety case* (Denney et al. 2015).

16.5.1 Degradation, drift, and observation cadence

None of the three verdicts is a permanent grant. Mechanical linkages develop backlash, friction surfaces wear, optical lenses collect dust, and operating environments change. Operating under either an unconditional (`UNCOND`) or conditional (`COND`) verdict therefore carries continuous obligations for the life of the deployment. Notebook 16.1 shows how little wear the stopping claim tolerates. A credible deceleration 31 percent below the assumed 2 m/s^2 spends the whole partial spare, and a smaller loss spends the reserve that remains once admission work is charged, leaving no allowance for perception latency or traction variation. A lining that wears that far is discovered at a rack end as an overrun unless the degradation first trips a standing condition, such as a scheduled loaded deceleration trial or an automated pre-shift brake diagnostic, before the machine is cleared to enter the aisle. Every operational obligation pairs a physical premise with an automated detector and an immediate mitigation, establishing a standing condition tied to the release verdict.

Sensor mounting drift can silently invalidate localization while every camera frame stays sharp and fresh. Daily thermal cycling and machine vibration alter extrinsic sensor geometry, and software rollback cannot realign a physically displaced bracket. The safety case must therefore require qualified online drift detection or scheduled physical recalibrations, setting inspection intervals from the underlying mechanical wear rates. A cryptographic checksum confirms only that the stored calibration file matches its build artifact; it gives no evidence that the physical camera still sits at that modeled pose.

A release verdict depends on physical premises with widely differing observation schedules. For instance, the mobile manipulator's optimistic all-weight traction screen requires a floor friction coefficient of at least 0.204; the braking claim additionally requires qualified loaded wheel forces, drive conditions, and stopping traces. Dry concrete exceeds this threshold by a wide margin, at roughly 0.60, but a baseline site survey conducted before commissioning cannot guarantee surface conditions against fluid spills or traffic polishing over time. The safety case must therefore specify routine floor maintenance schedules alongside strict exclusion rules whenever surface contamination occurs, supplemented by any onboard traction monitoring. Similarly, the arm's contact-force sensors require periodic recalibration against reference loads. Every physical premise must be rechecked

continuously or periodically at a cadence matched to how fast it degrades. This is the fourth law, *permission must be earned* (principle 4), applied over time. The verdict grants permission contingent on operating conditions, and that permission endures only while the evidence supporting each premise remains valid.

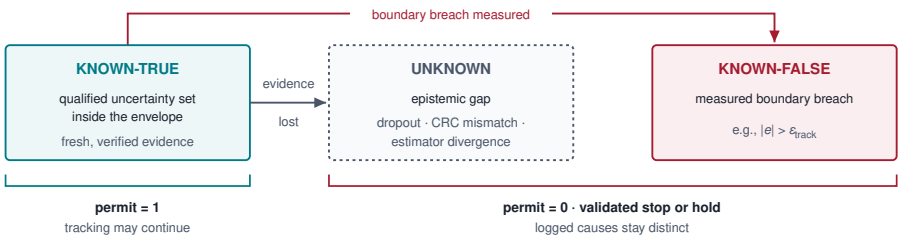
Every runtime-monitored premise requires a dedicated detector and a pre-allocated fallback response; conversely, premises governed by facility maintenance require accountable operators and enforceable exclusion rules. If a safety case credits a navigation-camera dispatch age of 51.6 ms, the runtime monitor must measure frame arrival latency against hardware clocks, accounting for sensor readout and transport delays separately. Forwarding an unverified metadata timestamp does not confirm that the frame is fresh. Without hardware-synchronized event timestamps, unverified inputs quietly turn safety premises into ungrounded assumptions. Different premises demand different sampling frequencies. The tote-rack load, bounded by its 50 kg mechanical payload limit, can be checked at each load-changing event if access controls exclude unrecorded loading and the check completes before the changed load receives motion authority. Conversely, base velocity, position tracking, and action lease validity must be evaluated deterministically on every 1 ms safety-enforcer tick.

16.5.2 The envelope trichotomy and epistemic gaps

A naive binary health check assumes normal operation whenever no explicit error flag is asserted, mistaking a dead sensor or a diverged filter for safe clearance. Checking monitored premises at runtime instead exposes three states of **envelope membership**: known-true, known-false, and unknown. Membership is *known-true* when authenticated telemetry and hardware-referenced timestamps confirm that the plant's bounded state-uncertainty set remains strictly inside certified envelope boundaries. Estimator convergence or small covariance matrices alone do not prove containment if model assumptions have broken down. When measured telemetry reveals a physical boundary violation, such as base tracking error exceeding the 40 mm bound reserved within the stopping envelope, the state is *known-false*. Finally, when an optical encoder loses serial communication, a camera frame fails its cyclic redundancy check (CRC), or an extended Kalman filter diverges, the system enters an *epistemic gap*. In this *unknown* state, the controller cannot tell whether the physical machine is safely positioned or breaching clearance margins.

When telemetry supporting active motion becomes unknown or known-false, the safety enforcer revokes trajectory tracking and executes the pre-certified fallback procedure for the plant's dynamic state. This runtime intervention differs from rejecting an invalid future trajectory proposal, because an already active trajectory chunk can continue executing only while its supporting sensor evidence, physical bounds, and lease duration remain intact. The system's flight recorder must log whether revocation stemmed from unobservable state telemetry or an active boundary breach, while the hardware supplies enough reserve energy and sensing capability to bring the plant to a controlled stop or hold.

Definition 16.3: Envelope trichotomy



Envelope trichotomy is the three-state epistemic classification of cyber-physical system health relative to its signed operating envelope: *known-true* (qualified state-uncertainty set contained inside the supported envelope), *known-false* (verified boundary violation triggering deter-

ministic hardware refusal), and *unknown* (state uncertainty caused by sensor dropout, frame corruption, or estimator divergence).

1. **Significance**: An unknown state is not evidence of a boundary breach, yet neither condition permits continued tracking. The permission decision is refusal in both unknown and known-false states; the logged cause remains distinct and the response depends on the plant's validated stop or hold capability.
2. **Distinction**: Unlike a binary boolean status check (`is_safe == true`), the envelope trichotomy separates sensor blindness from active violations, ensuring telemetry loss is never mistaken for safe clearance.
3. **Common pitfall**: Treating a stored command as indefinitely valid after communication fails. An active chunk may continue only within its own still-valid evidence and lease; otherwise the assessed fallback must take over.

The same three-state distinction applies to verification test records. A test trial marked `UNOBSERVABLE` or `INVALID` during acceptance testing (section 15.4.1) cannot establish the claim its required witnesses or test conditions failed to resolve. It leaves a gap in that coverage; other valid records remain evidence only for the conditions and claims they establish. Verified `PASS` records support the claims covered by their conditions and witnesses. A valid `FAIL` contradicts the claim tested under those conditions and withdraws any authorization that relies on it.

16.5.3 Blind travel, kinetic consequences, and cycle limits

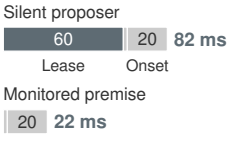
When telemetry fails or sensors black out, a machine traveling without sensory feedback moves blind; it must execute a deterministic fallback stop before its forward travel consumes the remaining spatial clearance. How fast that clearance vanishes depends directly on vehicle speed, contact geometry, and available actuator authority. Sizing the response therefore requires calculating the kinetic energy and blind travel accumulated during each millisecond of delay, and matching each sensor loss to a pre-certified fallback path.

How the system responds when a premise cannot be checked must follow from the physics of unmonitored motion rather than from arbitrary severity categories. When the mobile manipulator traverses an aisle at 1.3 m/s with its tote rack fully loaded, its 368 kg mass carries 311 J of kinetic energy. If its primary lidar stops updating, every millisecond of blind travel carries the base 1.3 mm closer to potential obstacles. The safety case pre-certifies three possible fallback paths, two of which apply only when the signed release names them in advance. The first is a degraded mode that permits continued motion while diverse redundant sensors keep state uncertainty bounded, for example throttling the aisle speed to 0.65 m/s (half the aisle speed, a chosen value) while secondary sensors watch the path. The second is a switch to a restricted limit set that a signed `COND` case pre-validated for that exact sensor loss, if the release includes one. The runtime only selects that profile; it does not create a new verdict. Dead reckoning and wheel odometry may retain qualified position bounds over a brief window, but cannot verify obstacle clearance on their own. The third option, the hardware STOP response, handles complete loss of observability by executing a configured drive and brake sequence. Stopping the vehicle within available clearance margins requires calibrated response timing, sufficient drive traction, and known brake deceleration.

Action leases bound how long an admitted trajectory chunk may run without renewal. Checking this lease on every controller tick does not require re-measuring every long-term site condition. Instead, the runtime permission path checks fresh local sensor telemetry alongside the signed validity windows of slower site inspections and maintenance records. If the proposer stalls or its communication drops, task throughput suffers, but safety holds as long as the enforcer and its fallback path remain decoupled, responsive, and intact.

Any standing condition governing physical motion requires an enforcement path with a measured detection and response budget. In this book's architecture, an independent safety enforcer checks trajectory proposals and monitored premises before issuing actuator permissions, re-evaluating system state on every 1 ms control tick. A subsystem failure reaches this enforcer along one of two paths: a silent proposer that stops transmitting is caught only when its existing action lease expires,

whereas an actively monitored hardware fault raises an immediate interrupt flag. Because detection delays differ between these paths, so does the distance the base travels blind (notebook 16.2).



The lease dominates: a silent proposer reaches brake onset in 82 ms, a flagged premise in 22 ms.

Napkin Math 16.2: The silent-proposer window

Problem: The base runs the aisle at 1.3 m/s. How far does it travel before brake onset when the proposer falls silent, how far when a monitored premise fails and its flag reaches the enforcer at once, and how late may that flag arrive before monitoring stops the base no sooner than the lease would?

Variables:

- Aisle speed $v = 1.3$ m/s (Chosen), so 1.3 mm of travel per millisecond
- Chunk lease 60 ms and permission tick 1 ms (Chosen); bus transfer 1 ms and brake onset 20 ms (Illustrative)
- Flag latency δ_{flag} from the physical failure to a flag the enforcer can read (Unknown; the detector's qualification must bound it)

Math:

1. **Silent proposer**: 60 ms + 1 ms + 1 ms + 20 ms = 82 ms, and 1.3 m/s × 82 ms = 106.6 mm.
2. **Monitored premise, immediate flag**: 1 ms + 1 ms + 20 ms = 22 ms, and 1.3 m/s × 22 ms = 28.6 mm.
3. **Crossover**: δ_{flag} + 22 ms = 82 ms at δ_{flag} = 60 ms, the length of the lease itself.

Result: Monitoring can save up to 78 mm of travel, but only for a flag that arrives within 60 ms of the failure; a slower flag stops the base no sooner than waiting out the lease.

Systems insight: The 22 ms local subtotal is not a complete detection-to-stop time; it reflects only controller cycle delay, communication transfer, and physical brake engagement. The decisive term governing whether an active monitor preserves physical clearance is δ_{flag}, the hardware detection latency. If an architectural budget fails to bound δ_{flag}, the safety case cannot claim proactive protection. Every millisecond spent detecting a fault carries the loaded vehicle another 1.3 mm toward whatever the failed premise was protecting against.

When an operating premise transitions to unknown or known-false, the enforcer revokes trajectory tracking and commands the appropriate rung of the fallback ladder, and the signed safety case must specify both this fallback selection and its measured timing. For the mobile base, the certified stopping sequence uses controlled, powered deceleration to bring the wheels to rest before applying electromechanical holding brakes. Because digital controller logic and brake-coil hold-up capacitors cannot deliver drive motor torque, the safety case must confirm drive energy, feedback sensing, and inverter continuity across every fault condition expected to use this active deceleration profile. By contrast, a raw torque-inhibit or free-wheeling cutoff is a different physical response; it cannot inherit stopping-distance guarantees calculated for controlled electrical braking.

Entering an unknown or known-false state also triggers local black-box logging. The onboard recorder must preserve the telemetry window leading up to refusal alongside the execution of the fallback trajectory, even if the initiating fault cuts external networking or primary bus power. These records are essential for root-cause investigation, but writing them to nonvolatile storage must never delay the hard real-time safety stop; section 16.9.5 sizes the capture path.

On longer timescales, even when every runtime heartbeat succeeds, the **release verdict** carries an operational cycle and inspection budget.[7] Operating hours and mechanical fatigue wear components in ways that high-frequency telemetry cannot observe directly (such as surface pitting on gear teeth, elastomer seal stiffening, or fastener preload relaxation). Once this cycle budget expires, the release case must be re-adjudicated before autonomous motion can resume. Across every timescale, a deployment verdict remains valid only while its supporting standing conditions remain demonstrably known-true, checked at intervals matched to physical wear rates and backed by pre-certified fallback stops with measured latencies. Claims, verdicts, and revocation conditions are

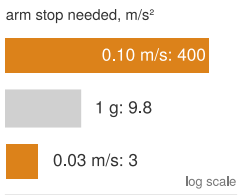
[7] **Mechanical Fatigue Budget**: A cumulative-damage model such as Miner's rule ($\sum_i n_i/N_i$) (Miner 1945) can organize a measured load spectrum against component-specific fatigue data; it is an approximation, not a universal failure deadline. The assessed loads, uncertainty, inspection evidence, and stopping consequences determine any signed payload derating and cycle limit. An expired limit revokes the applicable release case rather than silently extending service.

now defined. The six claims of the warehouse mobile manipulator can be adjudicated against the physical evidence assembled across this book.

16.6 Adjudicating the Safety Case

An algebraically exact stopping equation or contact-force bound still fails to protect a machine if the physical premise beneath its warrant has never been measured on hardware. Adjudicating a safety release case requires checking each warrant against the qualified measurements, justified analysis, or validated operating restrictions that support it. For the warehouse mobile manipulator, the adjudicator takes each of the six claims of table 16.2 in turn, names the physical premise under its warrant, and grades the evidence behind that premise. This isolates the exact boundary where mathematical deduction ends and physical measurement must take over.

The door-opening branch of C1 shows the procedure in full. The contact enforcement record supplies the branch's underlying mechanics (section 12.7.2). At the guarded approach velocity of 0.03 m/s, equation 12.11 yields an impact force at physical braking onset of 39 N, well below the 100 N latch limit. From there, equation 12.12 leaves the latch plate with only 0.15 mm of additional allowable compression. Bringing the arm to rest within this narrow mechanical travel requires a deceleration of approximately 3 m/s² over a brief 10 ms stopping window. By contrast, at the unconstrained approach velocity of 0.10 m/s, the force at braking onset rises to 95 N, and avoiding structural damage demands a deceleration of 400 m/s², more than an order of magnitude above standard gravity. While this kinematic calculation specifies the stopping performance required for release, mathematical derivation alone cannot prove that the physical arm actuators can deliver it. Consequently, the operating envelope (table 16.1) excludes the high-speed approach. Raising approach speeds sharply escalates contact braking demands, whereas the system's training demonstrations offer zero empirical evidence for safe high-speed contact (section 5.7.1).



Raising the approach from 0.03 m/s to 0.10 m/s multiplies the arm stop needed 136-fold.

Neither result is in doubt as arithmetic; the algebraic deduction is exact. Formal engineering release, however, cannot rest on closed-form kinematic models alone. The door branch fails authorization because its safety warrant depends on an unverified physical premise. Whether the physical manipulator, carrying its link momentum, payload mass, and reflected rotor inertia, can actually deliver 3 m/s² of deceleration within a fraction of a millimeter depends on transient joint torque limits, motor electrical time constants, and structural stiffness. Without calibrated hardware traces measuring these dynamic properties under load, this mechanical premise remains *unknown*, and the adjudicator must refuse the claim.

The remaining claim branches fail on similar grounds:

- **Base braking corridor (C1)**: The base stopping budget of 969.0 mm omits dispatch-to-admission delay and rests on illustrative values for braking deceleration, brake onset, path-tracking error, and localization uncertainty. Its remaining margin of 131.0 mm can be exhausted by the illustrative deceleration shortfall in step 4 of notebook 16.1, and step 5 leaves still less once admission work is charged.
- **Object handover (C1)**: The mug handover branch fails on three separate premises. First, the calculated contact force falls below the criterion only for an unconstrained, free-moving hand, and it comes from a standardized biomechanical body model rather than a force measurement of this contact. Second, the evidence packet does not show that the coworker's hand cannot be trapped against a rigid fixture, so the non-clamping geometry is unproven. Third, perceptual grounding remains unverified: a proposal that mistakes a clear glass jar for the target mug satisfies every geometric affordance check, and no trace in the runtime logs disambiguates the object.
- **Enforcer timing (C2)**: The timing warrant for the runtime safety enforcer assumes a worst-case execution time (WCET) of 250 µs against an unloaded baseline of 135 µs and a 400 µs deadline. The WCET is declared without architectural analysis and loaded stress evidence supporting it under peak memory bus saturation.
- **Traction and environmental state (C3)**: Claim C3 requires a qualified friction floor across every authorized aisle; 0.204 is only the optimistic all-weight traction screen, not a sufficient loaded stopping qualification. A clean concrete floor can be validated through baseline skid testing, but the onboard cameras cannot reliably distinguish dry concrete from a thin film of

transparent oil, and no sensor on the machine can exclude liquid contamination ahead of the braking contact.

- **Perception pipeline latency (C4)**: The 51.6 ms dispatch age combines nominal datasheet readout times with uncharacterized pipeline delays rather than verified end-to-end telemetry.
- **Permission independence (C5)**: The placement record's loaded test report, which would show the enforcer's maxima under the proposer's worst interference and test the common causes of section 13.2, is described in this book but not produced. Without it, the credited enforcement isolation is unsubstantiated; even supporting isolation records would not by themselves establish statistical independence of every failure event.
- **Authority transfer (C6)**: The argument cites `PASS` fault records for authority transitions A1 through A4, and no such records exist yet. Local operation could drop the remote-channel cases, but not the local ones.

Because every claim rests on at least one unverified physical premise, the adjudicator must render a verdict of `REFUSE`. This refusal does not imply that the robot is inherently dangerous. The book's supporting inputs are illustrative models rather than empirical validation logs, so the verdict is illustrative as well; it models how release is adjudicated and shows why ungrounded claims cannot license physical deployment. The fourth law, *permission must be earned*, requires an operational restriction wherever the available evidence cannot support a premise. Here, every branch contains an unverified premise. A principled refusal must therefore specify the exact measurements required to turn an unverified assumption into an accepted warrant. Table 16.5 lists these evidentiary requirements, naming the physical premise each measurement settles and the operational restriction a supporting result would lift.

↩ The four bedrock laws, the fourth of which holds that permission must be earned, are established in section 1.10.

Table 16.5: Measurements that would move the verdict toward `COND`: Each row names the premise a measurement settles on the warehouse mobile manipulator and the restriction a supporting result would lift; the inference trace bears on service availability only.

Measurement	Premise it settles (value in this book)	Claims	What a supporting result lifts
Stopping trials at the loaded mass	Low-tail credible deceleration (2 m/s^2)	C1	The base's budget and its 1.44 m/s ceiling, so an aisle speed can be signed
Brake-onset trials	Brake onset (20 ms)	C1, C2	The 82 ms bound from the last valid renewal to brake onset
End-to-end age trace	Dispatch age (51.6 ms) and subsequent evidence aging	C4	The age term of the budget and the stale-data threshold
Tracking-error and localization tails	Tracking bound (40 mm) and localization bound (50 mm)	C1	The inset and overhead terms of the budget
Site survey	Rack-end clear distance (1.10 m)	C1	A signed speed at each surveyed rack end
Floor survey	Dry-floor friction (0.60) against the 0.204 needed	C3	Assessed route only, with maintained contamination exclusion and loaded stops at the qualified friction floor; reduced speed alone cannot exclude oil
Inference trace	Chunk inference at the 99th percentile (40 ms)	None	Service availability: characterize renewal within the 60 ms lease; a percentile does not guarantee each renewal, and C2 depends on independent fallback
Loaded enforcer timing	Enforcer WCET (250 µs, declared)	C2, C5	The 400 µs deadline under the placement's worst load
Arm stopping trials	Arm deceleration from the guarded approach (about 3 m/s^2 needed)	C1	Door opening at the guarded approach
Handover force trials	Peak contact force at the handover speed (20.7 N derived for a free hand) and absence of clamping geometry	C1	Handing the mug to the coworker at 0.10 m/s under `accept_item`
Operator trials	Takeover and authority-handover timing	C6	Remote takeover, once A1–A4 each have a `PASS` record; otherwise local operation only

Neural network inference latency primarily governs system availability rather than baseline safety. An inference overrun will not violate physical containment so long as the independent hardware safety path and powered fallback controllers remain operational; however, repeated deadline misses will degrade throughput and stall task progress. Conversely, resource contention across shared memory, PCIe buses, or CPU caches that delays the runtime enforcer or starves its sensor feeds represents a direct safety failure under claims C2 and C5. Finally, while local autonomous operation can safely omit validation trials for unused remote teleoperation links, deployment still requires verified software placement records, comprehensive common-cause fault coverage, and passing test records for every local authority transition.

16.6.1 Worked restricted service with stipulated evidence

Without calibrated hardware measurements, the mobile manipulator remains locked out under a `REFUSE` verdict. To see how an engineering organization moves from initial refusal to conditional deployment without relaxing its structural safety limits, consider a **stipulated** release case in which an engineering packet supplies the measured traces specified below. (The uncalibrated evidence currently available in this book continues to dictate `REFUSE`.)

The target application is *restricted inspected-route mug delivery*. The robot retrieves a declared mug from a singulated feeder and traverses a designated, surveyed warehouse aisle at a velocity capped at 1 m/s. It then pauses the mobile base to open an equipment cage latch using the guarded contact trajectory and hands the mug to a consenting human coworker under the `accept_item` primitive at 0.10 m/s. Operational authority remains strictly local. The feeder mechanism physically prevents accidental object substitutions, recognizing that singulation alone cannot authenticate identity. Similarly, strict access controls prevent personnel from stepping into the robot's active stopping corridor during transit.

↪ The difference between a coworker who consents to a machine's risk, as this one does, and people who bear it without choosing, is developed in section 17.6.

For claim C1, the safety case stipulates empirical stopping traces under full payload mass, measured tracking and localization variance, dynamic latch-stopping performance, and instrumented handover contact forces that satisfy the bounded thresholds in table 16.5. For claims C2 and C5, the case stipulates independent hardware fault-injection logs and worst-case execution timing traces gathered under peak synthetic computational load. These records must account for common-cause hardware failures, support the enforcer's permission deadlines and actuator reaction latencies, separate controlled motor deceleration from passive mechanical braking at rest, and confirm isolated drive-energy cutoff and sensor integrity under all credited fault modes.

For claim C4, the case stipulates synchronized end-to-end hardware timestamp telemetry, bounded clock skew, and a verified upper bound on dispatch-to-admission latency. This latency must be deducted from the dynamic stopping distance budget so that a strictly positive safety margin remains; a theoretical kinematic velocity ceiling cannot authorize transit speeds on its own. Because sensor data ages continuously throughout execution, the chunk lease starts only when a plan is admitted, never when it is computed. The qualification packet must also check the time between admitted plans against the lease renewal deadline rather than assuming that nominal policy inference frequency guarantees admission. For claim C6, the packet stipulates verified `PASS` audit records across every state transition exercised by the local autonomous executive, while disabling the unverified remote teleoperation interface entirely.

Claim C3 requires substantially more than visual inspection or reduced transit velocity. The case stipulates a qualified site friction survey showing a friction coefficient of at least 0.12 across the surveyed transit corridor, reinforced by active cleaning protocols, chemical containment barriers, and physical access gates that maintain floor integrity throughout the authorized inspection interval. Stopping trials at maximum payload must confirm deceleration on this specific friction surface with the loaded wheel distribution and drive settings of the signed configuration. The complete restricted stopping distance, including the verified dispatch-to-admission delay bound and a positive safety margin, must fit inside the surveyed physical clearance with a positive reserve, the same accounting as step 5 of notebook 16.1. The operating velocity of 1 m/s sits below the partial ceiling of 1.23 m/s, but that comparison is only an initial feasibility filter. The active transit permit is revoked immediately upon any reported spill, expired maintenance inspection, or perimeter breach. If unmonitored oil or other fluid can enter the corridor during the deployment window, the stipulation is invalidated and the verdict reverts to `REFUSE`. A lower speed cannot repair that premise, because

the restricted speed was sized for the inspected friction floor of 0.12, not for a film near 0.05 that removes most of it.

The task capability packet is similarly stipulated. Before evaluation begins, the facility operator defines binding acceptance criteria in the verification plan: a minimum mission completion rate, an upper bound on end-to-end cycle time, and a maximum allowable operator intervention rate, accompanied by sample-size requirements, confidence intervals, and statistical decision rules. A successful mission requires maintaining object identity and physical integrity from retrieval through coworker handover. Cycle time accounting includes transient safety stops, crossing delays, and autonomous replanning; human rescues and aborted deliveries are tracked as explicit mission failures. The safety case stipulates that closed-loop trials of this exact restricted configuration satisfy all three benchmarks. The qualification dossier separates raw neural policy autonomy, system-level task success, and safety interventions into independent metrics, the separation section 7.2 drew between the policy's own completions and the completions that protection made possible. Minimal-intervention safety filtering may run during nominal task execution; its clamps are counted separately, so they neither mask the policy's shortcomings nor count as safety violations.

With these stipulated evidence packets in hand, the adjudicator can sign a verdict of `COND` (conditional release) for *restricted inspected-route mug delivery*. The physical records support claims C1 through C6 solely within the restricted operating envelope, while the task evaluation shows acceptable operational utility under identical constraints. The adjudicator binds both the physical safety dossier and the capability evaluation to the release manifest, leaving full-speed autonomous transit and remote teleoperation unauthorized. The result is a conditional safety case, not a blanket deployment permit. Invalidating any underlying assumption or breaching a boundary condition immediately returns the verdict to `REFUSE`; conversely, gathering qualified operational telemetry and physical measurements over time can justify expanding the envelope toward full-scale release.

Checkpoint 16.2: Refusal and the path to conditional release

Before approving a conditional release of the machine, verify your understanding of how a premise, rather than a calculation, decides a claim:

- ☐ **Premise vs. arithmetic**: Can you explain why the calculated latch force at physical braking onset is not in doubt while the door branch of C1 is still refused?
- ☐ **Unsupported approach**: Can you use equation 12.11 and equation 12.12 to show why the operating envelope admits only the guarded door approach?
- ☐ **Refusal as a deliverable**: Can you name, for one refused claim, the measurement that would lift it and the restriction that would remain after a supporting result?
- ☐ **Measured-deceleration estimate**: Suppose loaded stopping trials put the low tail of the deceleration at 1.6 m/s^2 under the same jerk bound. Can you rerun notebook 16.1 at 1.3 m/s and decide whether that aisle speed can still be signed once Δ_{da} is charged?

16.7 Adversarial Review

Confirmation bias steers an engineering team's data collection toward nominal conditions where learned models succeed, leaving edge-case hazards untested until live operation exposes them. Suppose the measurements listed in table 16.5 arrive and, as in the stipulated branch, appear to support a conditional (`COND`) authorization for the mobile manipulator. Before signing, the safety adjudicator submits the safety case to an independent **adversarial review** tasked with falsifying its warrants. Rather than ratifying a brief, the reviewer actively searches for unsupported warrants, uncharacterized physical transitions, and environmental edge cases that violate the release contract. If an argument collapses on paper when an adversary probes for physical violations, the machine cannot be expected to withstand the hazards of live service.

The review interrogates every evidentiary node in the argument tree using four structured probes. The first challenges **empirical sufficiency**: whether the physical trials reached the extremes of the operating envelope. Stopping trials with the loaded base on dry concrete, at a friction coefficient near 0.60, support the warrant that the base achieves its certified deceleration under ideal conditions. The

restricted operational mode, however, admits facility floors that the site's inspection regime certifies only down to 0.12, such as an aisle polished smooth by traffic. In those zones, the base may run at the restricted velocity 1 m/s, below the 1.23 m/s ceiling. If the trial set contains no loaded stops on such degraded floors at that speed, the warrant for the restricted mode is unsupported. A coverage audit finds on paper the gap that unvetted service would otherwise find against a warehouse rack. The reviewer also checks whether validation trials covered transient execution states across an A/B partition swap, when a newly flashed software bank takes over from the incumbent one (section 16.9.1). The same check applies to cold-start sensor characteristics before cameras and inertial measurement units reach thermal equilibrium. It applies as well to the operating boundaries of canary units, the small subset of the fleet that first runs a candidate policy (section 16.9.2). Finally, the reviewer asks whether site procedures actually keep fluid films out of the corridor throughout the authorized interval, since listing oil outside the envelope does not exclude it from the floor.

The second probe examines **domain transfer**, targeting claims where synthetic simulation substitutes for direct physical measurement. A vision-based human detector may achieve high recall across synthetic, ray-traced warehouse environments, yet fail on physical hardware when encountering coworkers partially obscured by stretch-wrapped pallets whose specular reflections blind the camera's auto-exposure algorithm. Benchmark recall on curated offline datasets is an indirect proxy metric that scores isolated frames rather than the closed-loop physical causal chain running from visual detection to an arrested base. Likewise, a simulation that omits linkage compliance or fieldbus jitter cannot establish tolerance to those physical mechanisms. The reviewer demands empirical evidence from paired hardware-in-the-loop (HIL) trials on target SoC devices connected to chassis dynamometers[8] and physical bus emulators under real facility illumination, dust, and reflective geometry.

The third probe attacks **structural independence**, asking whether the learned model and safety enforcer share hardware resources whose contention could delay sensor delivery or actuation cutoff. On modern robotics edge platforms where neural inference engines and real-time safety enforcers share a single system-on-chip die, internal crossbar interconnects, shared caches, and DRAM memory controllers, unconstrained direct memory access (DMA) transfers from large model forward passes can saturate memory channels. This memory contention induces tail latency spikes that starve deterministic safety enforcer execution ticks, directly invalidating the 51.6 ms observation-age chain of C4. Stress tests and hardware timing analysis under peak synthetic compute and bus loads must establish the worst-case interference bound; unit tests on an otherwise idle bus provide zero evidence of non-interference. The reviewer also checks whether writing frozen edge telemetry ring buffers to non-volatile storage competes with real-time fieldbus DMA buffers across the internal bus fabric. The reviewer inspects watchdog reset circuits for single points of failure and verifies whether the supercapacitor power hold-up circuit supplies enough energy to flush in-flight telemetry buffers to persistent flash when an emergency stop opens the main DC bus contactors.

The fourth probe identifies **unmonitored state drift**. Mechanical wear alters gearbox compliance[9] and disrupts the dynamic relationship between commanded motor torque and physical joint motion. The reviewer therefore demands paired wear-state braking and contact traces before accepting the arm-stop premise of the door branch of C1 (section 16.6). A static motor-current threshold alone cannot prove that a mechanically worn drive train remains inside certified force and stopping limits. The reviewer also checks whether over-the-air updates preserve the persistent hardware calibration partition intact across schema migrations, or whether an OS reflash can silently reset camera extrinsics and joint zeroes to unverified factory defaults. Finally, the probe checks whether thermal bracket motion stays inside the localization allowance; fresh timestamps and unchanged calibration files on disk cannot establish that physical fact.

Reviewers turn these vulnerabilities into concrete tests by constructing explicit defeaters for each warrant. The hardest failure modes to catch combine minor anomalies that each stay inside their own tolerance bands, the compound faults that section 15.6.5 derives from shared causes. Consider the mobile manipulator's navigation camera operating with three simultaneous sub-threshold degradations. A light film of dust coats its lens. A mount whose fasteners relaxed within tolerance vibrates as the base crosses floor joints. An interconnect harness flexes at those joints and elevates the bit-error rate across its serializer/deserializer (SerDes) link. Each individual check reads normal. The optical transmission score exceeds its 85 percent alarm threshold. The mount deflection

8 | **Chassis Dynamometer**: Powered rollers under the drive wheels absorb or supply torque, so the base can run at aisle speed on a bench while its drive and braking torques are measured. The rollers, not the floor, set the traction. Dynamometer stops can therefore qualify drive and brake response but say nothing about the friction premise of C3.

9 | **Strain-Wave Gear Wear**: The arm's joints use strain-wave reducers (section 2.4.4), and in service their flexspline teeth and wave-generator bearing wear. Wear raises lost motion and torsional hysteresis, which adds phase lag between motor and link, so the arm begins to decelerate later and its first contact peak rises.

registers below its 0.15-degree hardware fault limit. SerDes cyclic redundancy check error counters remain below their warning tripwire. Their timing effects remain contained only if the independent permission path strictly enforces the event-specific observation-age and clock-uncertainty checks of section 8.8 and retains its validated response time. Any hardware or bus interference that defeats that deterministic path is a safety violation under C5, not a benign availability-only event.

The same anomalies compound within spatial estimation. Dimmed contrast reduces tracked visual features, mount vibration adds motion blur and persistent pose bias, and frame drops extend intervals of uncorrected dead reckoning. A systematic pose bias can quietly erode the localization allowance without forcing estimated filter covariances or innovation residuals to breach trip thresholds. The release reviewer therefore challenges the combined error against the qualified uncertainty bound, rather than inferring containment from individually passing scalar diagnostics. A breach of the combined bound consumes part of the stopping reserve and invalidates the release warrant; exhausting the entire stopping margin requires a quantified combined-error scenario. Statistical residual monitoring protects against only the specific faults its qualification shows it can detect, leaving site operational restrictions or independent physical checks to cover the remaining biases.

The review also audits the execution integrity of the permission path, checking whether inference loops or background services can bypass, corrupt, or starve the enforcer or human overrides, the latter covered by claim C6 of table 16.2. Below the proposal boundary, both safety monitors and manual overrides maintain non-preemptible execution priority over learned models. The reviewer asks concrete systems questions: whether a runaway policy thread can exhaust shared memory channels, whether an accelerator-runtime exception can lock a kernel spinlock, and whether a telemetry logging burst can brown out a regulated power rail. The reviewer accepts **permission path** independence, and the held-up power rail that carries it on the mobile manipulator, only after reviewing physical fault-injection trials. These trials must show survival under bus saturation, active feed removal, and an abruptly killed proposer task (fault F1), and they must appear in the signed fault records, because C2 claims only the faults the permission path demonstrably survives. A user-space implementation without qualified scheduling, resource isolation, and response mechanisms cannot supply the justified execution bound and fault isolation the reviewer requires.

When an adversarial probe succeeds, the case cannot be salvaged by editing claim descriptions or inserting arbitrary safety margins. The broken warrant is traced to its root systems failure (an uncharacterized physical boundary, an inadequate dataset, or an unmodeled bus latency), and the defect returns to the owning engineering subsystem. For example, a motor-current observer that fails under gearbox compliance returns to the mechanical specification as wear limits and runtime compliance estimation, while bus contention that delays the enforcer returns to the compute architecture as memory partitioning and real-time scheduling. The case returns to the engineering team with its broken warrants named, and only updated boundaries, re-derived stopping distances, and renewed verification reopen formal adjudication.

Adversarial review must also recur periodically, because a case accepted once can erode in service without any single discrete event forcing a new audit. The *normalization of deviance* names that operational erosion, in which anomalies that cause no immediate harm come to be treated as acceptable operating noise rather than as evidence against a warrant (Leveson 2011; Vaughan 1996) (war story 16.1).[10] On the mobile manipulator, this erosion unfolds silently across months of fleet operation. Localization residuals creep toward the edge of their tolerance. Observation-age watchdog trips are dismissed as nuisance stops. Stopping distances end slightly longer than the initial trials did. Enforcer trips are cleared on the floor without a trace review. When thresholds are widened, margins trimmed for throughput, and alarms suppressed because nothing has yet gone wrong, the gap between the signed envelope and the machine's actual physical behavior widens until a routine disturbance exhausts what remains. The defense is to fix each watched quantity's threshold and response before service begins, as the safety performance indicators of section 16.9.5 do, so that a run of harmless readings cannot move it.

[10] **Normalization of Deviance**: The sociologist Diane Vaughan coined the term in her study of the Challenger launch decision (Vaughan 1996). It names a process rather than a decision, in which each anomaly that caused no harm widened what the next review accepted as normal.

War Story 16.1: Challenger and experience limits (1986)

Context: The Space Shuttle's solid rocket motors were assembled from cylindrical steel segments whose field joints were sealed by primary and secondary O-rings (Presidential Commission on the Space Shuttle Challenger Accident 1986). No learned model appears in this chain. The adjudication failure it records is the exact failure this chapter's verdicts exist to prevent: a launch approved for conditions that the evidence did not cover.

Mechanism: The ambient temperature at launch was 36°F, 15 degrees colder than any previous launch. O-ring resiliency falls with temperature, and a cold ring returns to shape too slowly to follow the joint gap as ignition pressure opens it. On the eve of launch, engineers at Morton Thiokol, the motor's contractor, recommended against launching below 53°F, the O-ring temperature of the coldest previous flight, whose joints had shown the worst blow-by to date. That limit was the outer edge of flight experience, not a qualified bound. NASA managers countered with a qualification temperature that applied to the propellant's bulk temperature rather than to the joint seals.

Impact: A combustion gas leak through the right booster's aft field joint, beginning at or shortly after ignition, weakened or penetrated the External Tank. The vehicle broke up about 73 seconds after launch, and all seven crew members died.

Response: The presidential Commission attributed the failure to "a faulty design unacceptably sensitive to a number of factors," and temperature heads its list of those factors. It recommended that the joint be changed and that the new design be certified by tests "over the full range of operating conditions, including temperature."

Systems lesson: Thiokol management reversed its engineers' recommendation at the urging of NASA's Marshall Space Flight Center. Thiokol's vice president of engineering testified, "We had to prove to them that we weren't ready." The burden of proof had inverted: rather than requiring proof that the joint was safe inside its evidence, adjudicators demanded proof that it was unsafe outside the evidence, where no data existed. The Commission also found that O-ring erosion had recurred on earlier flights and that NASA and Thiokol "accepted escalating risk apparently because they 'got away with it last time' " (Presidential Commission on the Space Shuttle Challenger Accident 1986), so the flight record itself had been read as evidence of safety. A release verdict keeps the burden where the envelope trichotomy puts it. A condition outside the evidence, such as a seal temperature below all flight experience, is unknown, and the verdict refuses it.

The Tempe collision analyzed in section 1.8 rested on an unmeasured premise of the same kind as Challenger's seal temperature. Its vehicle operator was the countermeasure the whole system leaned on, so the premise was that the operator was watching, and nothing measured whether that held. A release case that relies on such a premise must bind its operational response, whether an outright refusal or a restricted case, to verified physical limits, as claim C6 does for authority transfer. Even a case that survives this adversarial review has had its warrants tested only in principle while its verdict still lives on paper. The release manifest turns the surviving argument and the verdict it supports into a signed, tamper-evident record that the machine itself checks before it moves.

16.8 The Release Manifest

A safety case that lives only in static documents or procedural sign-off sheets cannot stop a robot from booting with an unverified policy checkpoint or stale sensor calibrations. To make the verdict enforceable on hardware, the adjudicator's decision is recorded in the **release manifest**, the fourteenth record in the book's record chain. This signed, tamper-evident data structure binds every upstream engineering contract, the Goal Structuring Notation argument graph, the operating envelope limits, and live standing conditions to the adjudicator's verdict by cryptographic digest. The target platform refuses physical motion whenever runtime binary images, silicon attestation measurements, or underlying qualification data diverge from these signed digests.[11]

The release manifest gathers the engineering contracts established throughout this book. Each upstream record defines an operating boundary or contributes empirical data that the safety case

↩ The operator-monitoring failures that a human-takeover premise invites, from mode confusion to authority ambiguity, are established in section 14.6.

11 **Cryptographic Manifest and Silicon Attestation**: One-time programmable electronic fuses (OTP/eFuse) store the adjudicator's public verification key in silicon, preventing unauthorized software updates from replacing the root of trust. Secure bootstrap validates each early boot stage before handing over execution control (Arbaugh et al. 1997), while measured boot extends cryptographic measurements into Trusted Platform Module Platform Configuration Registers (TPM PCRs). The hardware verifier checks these measurements against the digests specified in the signed manifest; any unsigned or modified binary image fails attestation and never receives an operational permit.

↪ The residual-claims register, the record that follows the release manifest and ties each premise it leaves open to an operational restriction, is developed in section 18.3.3.

must check; a signature on an isolated contract does not validate the system as a whole. Instead, the manifest links reviewed snapshots of these records into a single argument graph:

1. **Physical plant and timing base:** The limit record (section 2.7.3) documents torque, velocity, and thermal dissipation bounds alongside their qualification data. The proposer's payload requirements (section 3.5.4) specify candidate action horizons that feed the handoff record rather than standing as an isolated contract. The handoff record (section 4.9) establishes deterministic fieldbus jitter, cycle deadlines, and real-time communication premises.
2. **Data, policy, and behavioral bounds:** The provenance record (section 5.11) captures dataset calibrations, sensor configurations, and operating-domain coverage. The policy manifest (section 6.8) binds frozen neural network weights to the empirical behavioral bounds proven for that candidate model. The evaluation record (section 7.8) details closed-loop task success rates, human intervention tallies, and stress-tested operational conditions.
3. **Perception, belief, and planning:** The observation contract (section 8.8) enforces observation freshness and multi-sensor synchronization windows. The belief record (section 9.3) specifies geometric state estimates, state covariance, and validity domains. The intent lease (section 10.4) records task objectives, spatial tolerances, and preemption conditions. The trajectory record (section 11.8) encodes motion profiles, kinematic seam continuity, execution deadlines, and safe stopping continuations.
4. **Deterministic containment, execution, and verification:** The enforcement record (section 12.11.1) logs the command stream admitted by runtime safety filters, cleanly separated from the signed enforcement contract that specifies barrier functions and fallback triggers. The placement record (section 13.8) documents the qualified compute scheduling, memory isolation, and processor affinity. The authority record (section 14.8) documents arbitration switches and fallback outcomes. The fault records (section 15.11) index tested fault-injection scenarios alongside known coverage gaps.

Beyond linking these upstream contracts, the release manifest adds its own fields. It carries the root claim, the cryptographic digest of the Goal Structuring Notation argument graph, and binary hashes across all deployed software (spanning learned policy weights, the deterministic enforcer logic in both nominal and restricted modes, and the real-time operating system kernel). It also carries sensor calibration tables, the adjudicator's digital signature, mandatory standing operational conditions, an accumulated lifecycle wear budget of operating hours and load cycles, and a calendar expiration timestamp. All evidence links must resolve to these exact reviewed record digests, organized hierarchically beneath a single evidence root structured as a Merkle hash tree (Merkle 1988). Each parent digest is computed from its children's digests, so changing an evidence leaf changes the root that the manifest signs. Only when the hardware root of trust authenticates this complete cryptographic tree will it permit the motor power stages to energize. While traditional release notes or post hoc compliance certificates summarize safety after the fact, the release manifest is an active, machine-checked gate. The robot checks it before closed-loop control begins, and its runtime monitors revoke motor tracking within a bounded detection-and-response window if any underlying evidence or environmental condition expires.

↪ **Byte layout:** The release manifest's fields, types, and digests are given in section C.15.10.

Figure 16.3 routes every release attempt through authentication and physical standing checks before an operating permit can issue. The release gate authenticates the manifest, binary images, and complete set of fault records before verifying current sensor calibration, mechanical maintenance thresholds, and operational design domain conditions. Only an exact cryptographic and physical match enables the plant's certified startup sequence. Table 16.6 lists the gate result for each combination of signed case and standing check, including the single path by which stale calibration still reaches `COND`. A separately signed `COND` case can tolerate a named stale calibration only when its restricted envelope either locks out the motions that depend on that sensor, such as prohibiting base transit when the tote-rack scale is stale, or widens the uncertainty allowance and derates speed to cover worst-case calibration drift. During active motion, any breached condition commands an assessed safe fallback before any switch of software partition (the A/B banks of section 16.9.1) or electrical isolation.

↪ **Protocol:** The gate's step-by-step authorization protocol is given in section C.15.11.

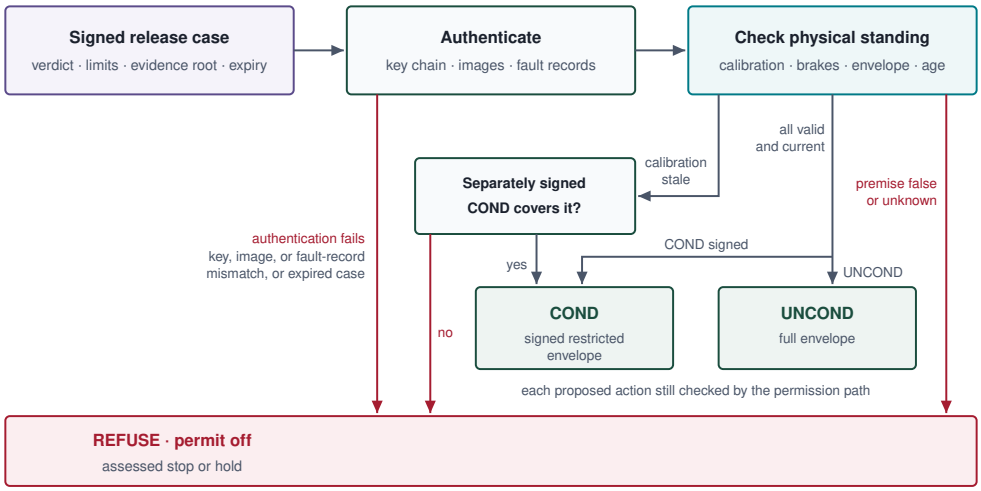


Figure 16.3: Fail-closed release pipeline: The signed verdict and evidence root pass key, image, and complete fault-record authentication, then current calibration, brake, and operating-envelope checks. Every failure path ends in REFUSE. A valid case reaches UNCOND or COND according to what was signed, and stale calibration reaches COND only through a separately signed restricted case; either way, each proposed action is still checked by the permission path. Attribution: Textbook production team (CC BY-NC-ND 4.0).

Table 16.6: Release authorization truth table: A restricted mode is a distinct authenticated claim, never an unsigned downgrade of the full case.

Signed case	Evidence and standing checks	Restricted case covering stale calibration	Gate result
`UNCOND`	All valid and current	Not needed	Full *proposal* envelope under enforcer; `UNCOND`
`COND`	All restricted evidence valid and current	Included if calibration stale	Signed restricted envelope under enforcer; `COND`
`UNCOND` or `COND`	Fault-record root/signature mismatch, expiry, image mismatch, or unknown physical premise	Any	Permit off; `REFUSE`
`UNCOND`	Calibration stale	No	Permit off; `REFUSE`
Separately signed `COND`	Only the named calibration is stale; restricted evidence and limits valid	Yes	Restricted envelope; `COND`
`REFUSE`	Any	Any	Permit off; `REFUSE`

Each empirical evidence leaf in the assurance Merkle tree must identify its raw sensor trace, instrumentation hardware, calibration log, machine configuration, and automated test harness script. Analytical leaves retain their derivations and assumptions; procedural leaves retain the accountable procedure and its validation for the signed envelope. Measured latency profiles enter the safety case as empirical percentiles, whereas deterministic real-time execution guarantees demand the analytically justified upper bounds specified in section 4.5. Similarly, collaborative object-handover criteria require load-cell time series recorded against standardized anthropomorphic surrogates across specified contact geometries. By cryptographically anchoring these primary records, an independent auditor or verification engine can reproduce the experimental test suite and pinpoint the exact operating boundaries where each safety claim ceases to hold.

The release manifest bounds the physical envelope and hardware configuration under which its verdict remains valid. Modifying a neural policy checkpoint, altering a real-time thread priority, updating a device driver, or replacing an actuator with an unverified component revision immediately invalidates the manifest, requiring a re-adjudicated safety case. On the warehouse mobile manipulator, for example, the manifest enforces signed wear thresholds on harmonic-drive backlash

and electromagnetic brake lining thickness derived from stopping-distance and payload trials. Operating envelope tripwires trigger immediate refusal: on-board load cells detect tote-rack overloading beyond rated payload capacity, while facility procedures prevent operation on unmapped or unsurveyed floor surfaces (table 16.1). The manifest also encodes the isolated, energy-buffered power rail of the permission path as a standing invariant, running a diagnostic self-test on the hold-up capacitor bank before granting boot authorization. Whenever a runtime sensor detects that a state variable has breached a signed threshold, the hardware permission path interrupts closed-loop trajectory tracking within its certified detection-and-response deadline and brings the system to an assessed fallback state.

To preserve traceability across time, the manifest binds all operational telemetry directly to the exact hardware-software revision that generated it. High-resolution local ring buffers capture boundary excursions for post-incident diagnostics, whereas fleet-wide telemetry summaries reveal systematic mechanical drift across hundreds of operating hours. The curation pipelines examined in section 16.9.5 must preserve this cryptographic lineage whenever field data is promoted to support a later release case.

During daily operation, edge telemetry logs motor torque residuals, control-loop cycle times, and safety enforcer interventions for trend analysis, while the hardware permission path checks physical invariants on every 1 ms permission tick. If a routine maintenance check measures a stopping distance exceeding the signed clearance budget, or if a camera calibration certificate expires, the release verdict is revoked automatically. Re-flashing an older firmware image cannot compensate for worn mechanical friction pads. Unless an engineer deploys a separately signed restricted case (`COND`) that explicitly accommodates that degraded physical state, the controller executes an assessed emergency stop and locks out autonomous motion until certified maintenance and re-adjudication take place (table 16.6).

Once its powered stop has brought the base to rest, the warehouse mobile manipulator can reach a quiescent state only after the arm and payload are supported and the assessed holding brakes are engaged before drive power is removed. The base rests stably on the floor, but that support alone does not secure its gravity-loaded arm. For other morphologies, severing motor power is itself the hazard, because the machine collapses under gravity once torque is removed (perspective 16.1).

Systems Perspective 16.1: When removing torque causes the harm

Question: Which verdict applies when removing torque causes the harm?

A supported wheeled base at rest eliminates its translational kinetic hazard, though its arm and payload still require qualified support. In contrast, an upright bipedal humanoid retains substantial gravitational potential energy. While resting in a support gantry before standing up, a `REFUSE` verdict safely holds the robot in its mechanically supported state. Once balanced upright, however, revoking locomotion authority cannot simply de-energize the actuators without inducing an immediate dynamic fall; safe shutdown requires an active, validated arrest or controlled descent strategy.

- **Torque removal can cause a fall.** An unsupported biped with backdrivable transmissions will collapse if motor currents drop to zero: torque removal leaves gravity-loaded joints without support (section 4.6), and an upright biped without active balance diverges like an inverted pendulum (section 4.9.1). Consequently, such physical plants cannot treat torque removal as a fail-safe stopping condition unless accompanied by certified passive catches, lock-up brakes, or an external tether.
- **The fall-arrest response is itself an actuation.** Executing a controlled squat, brace, or step-down to bring the robot to rest is itself an active trajectory that requires validated sensorimotor claims, verified timing margins, and fault-tolerance guarantees. In this regime, executing a safe `REFUSE` action depends on the very control systems maintaining upright stability.
- **Wear revokes the running permit.** Degradation in foot-sole rubber traction or increased backlash in planetary gearheads alters the dynamic margin required for emergency

balancing. When telemetry signals that physical wear has crossed the threshold certified in the safety case, the runtime supervisor must revoke high-speed dynamic locomotion well before the safety-critical arrest maneuvers degrade below their qualifying safety margins.

Checkpoint 16.3: Deriving a verdict from your own measurements

Table 16.5 lists the measurements needed to support a release case but supplies no raw trial results. Make a copy with columns for value, source, provenance, and premise status, fill those columns with empirical data from hardware trials on your own robotic platform or an illustrative benchmark scenario, and evaluate the resulting release verdict:

- ☐ **Provenance and premise status**: For each row, record measured, derived, datasheet, or illustrative provenance; label selected operating requirements as chosen. A derived result retains its inputs and assumptions and remains illustrative when those inputs are illustrative. A selected substitute for an unmeasured brake value is illustrative, whereas a chosen speed cap needs qualified evidence that it fits. A manufacturer's rating is datasheet evidence, not a measurement of this unit. Separately apply the Reader Guide's five premise statuses: established by measurement or derivation, assumed, decided by accountable judgment, unsupported, or outside the threat model. Explain why each credited assumption or restriction is warranted for the signed envelope; a status label grants no authority. Runtime known-true, known-false, and unknown describe current envelope membership, not provenance.
- ☐ **Verdict derivation**: A `COND` case requires adequate support for every relied-on premise: qualified measurements, justified analysis, or validated operating restrictions. Illustrative values alone still support only an actual `REFUSE`. Recompute the base's budget and spare margin from your values. If every relied-on premise has adequate support, name the envelope the `COND` case signs and the premise that keeps the normal mode unsigned. If a relied-on premise lacks adequate support, derive the verdict the table *would* support given the missing evidence, and label that verdict hypothetical.
- ☐ **Artifact label**: Mark the filled table as your own work, whose values license no physical machine.

A release manifest binds the verdict to an explicit, reviewed system configuration. Any later modification, whether retraining a perception head, tightening an actuator torque limit, or swapping a camera lens, breaks that closure and reopens some of the underlying claims. The question the rest of the chapter answers is which ones, and how the machine stays contained while the reopened claims are requalified.

16.9 Maintaining Release Through Updates

Continuing the stipulated warehouse branch, suppose the mobile manipulator's restricted mug-delivery policy is retrained to reduce hesitation at the controlled pick station. Pushing that updated checkpoint directly to the fleet would invalidate the signed manifest and expose the machine to unverified bus loads and contact trajectories. Before the candidate neural policy can issue commands to physical actuators, it must pass back through the release decision. A signed safety case establishes which historical evidence remains valid, which hardware-in-the-loop tests must run again, and how the machine maintains physical containment throughout the update.

16.9.1 Modular safety cases and over-the-air updates

Retraining a policy directly alters capability evidence, and neural modifications often introduce side effects across the shared hardware platform: increased neural accelerator memory bus traffic, altered sensor frame processing latency, or modified contact force trajectories. An engineer must

↩ Every updated checkpoint is a new policy whose earlier evaluations no longer apply, the rule established in section 6.6 that gives each one its own manifest and regression evidence.

trace these cross-subsystem dependencies before deciding which containment records remain valid. While **over-the-air (OTA) updates** swap out a named software payload, **modular safety cases** isolate dependencies by recording how each claim relies on specific evidence items and platform assumptions (Denney et al. 2015). Whenever an update alters an underlying dependency, the modular case shows exactly which safety claims are reopened.

The containment argument is the defense against physical harm. It binds together the plant model, physical sensor limits, worst-case execution budgets, and the independent permission path. Key engineering premises include the bounded tracking error tube ϵ_{track}, the emergency stopping budget, the passivity barrier for physical contact, and hardware watchdog timing margins. Historical verification records for these containment claims remain valid only if the proposed update preserves the exact operating conditions under which they were measured. For example, replacing a backbone network with a higher-capacity model can increase memory bus contention or cause voltage droop on the power rail even if the deterministic safety-filter binary remains byte-for-byte identical. An unchanged enforcer binary does not imply an unchanged containment argument; its guarantees remain bounded by the validated dynamic envelope $\mathcal{C}$ and the system-level timing margins established during qualification.

↩ The tracking-error tube ϵ_{track} that the containment argument consumes as an invariant is established in section 12.2.

In parallel, the capability argument shows that the robot reliably fulfills its mission within its certified operational design domain. Candidate benchmarking, passive shadow traces, and geofenced closed-loop trials test whether a retrained policy reduces hesitation or improves task throughput. While the low-level enforcer can continuously clamp or reject invalid control proposals to maintain safety, an enforcer that constantly intervenes degrades the robot into an unusable, halting machine. Release engineering therefore evaluates capability and containment together. An update qualifies for fleet dispatch only when the candidate configuration delivers acceptable task performance without undermining the empirical premises supporting its containment envelope.

Modern physical AI platforms preserve configuration integrity during deployment through **dual A/B OS partitioning**. The update agent streams the candidate image onto the inactive storage bank (Bank B) while the incumbent controller continues executing from the active bank (Bank A). Activating the newly flashed partition requires an assessed stationary update state. The robot must be parked with its payload supported. Battery reserves must exceed with margin the energy needed for re-flashing and post-boot self-tests. The independent safety supervisor must hold all motor drives unpowered or mechanically locked so the robot cannot move during bootloader cutover. Hardware watchdogs govern the runtime decision to commit or abort this cutover (section 16.9.4). A single fleetwide signing key is a single point of compromise, because an attacker who extracts that key from one compromised endpoint could sign arbitrary firmware for every robot in the field. Update architectures therefore use role-separated signing structures where distinct root, targets, and release keys live on separate, hardware-isolated machines. The Update Framework (TUF) pioneered this design, and Uptane extended it to automotive and robotics systems where heterogeneous electronic control units require independently authenticated firmware images (Samuel et al. 2010; Kuppusamy et al. 2016).

Overwriting firmware must never destroy the physical calibration history of an individual robot. **Persistent hardware calibration parameter retention** separates machine-specific measurements from generic operating system binaries, preventing a **calibration clashing failure**. Camera intrinsic matrices ($\mathbf{K}$), extrinsic sensor transformations ($[\mathbf{R} \mid \mathbf{t}]$), kinematic joint zero-point offsets, and motor torque constants (K_t) reflect the mechanical and electrical geometry of an individual machine and drift over time with vibration, thermal cycling, and wear. Storing these values in a dedicated, wear-leveled, non-volatile parameter partition keeps calibration intact when an OS image is flashed. When a firmware update introduces a revised parameter schema, the migration utility writes the transformed schema into a new record while preserving the legacy block, allowing direct rollback to the prior image-parameter pair. Cryptographic checksums verify the bit-level integrity of these stored calibration blocks, while physical self-tests at field release verify that the stored parameters still match the physical machine.

Finally, **forensic traceability** links every running OS image, policy checkpoint, and calibration partition to its training provenance, compiler flags, and reviewed HIL regression records through the signed release manifest (section 16.8). When an unexpected field intervention or trajectory excursion occurs, these links let incident investigators reconstruct the exact configuration running at

that millisecond and determine which safety warrants need re-examination. While a cryptographic digest confirms identity, isolating the physical root cause requires detailed sensor traces and causal analysis.

16.9.2 Staging candidate policies before release

Software simulation models timing and electrical behavior only to the accuracy of its mathematical approximations; it cannot catch memory-bus contention, fieldbus noise, or inverter thermal sag on the real processor and drive electronics. A candidate policy must therefore repeat the **hardware-in-the-loop (HIL) regression gates** affected by its changes on a physical test bench under full thermal and electrical load. The bench tests the specific interference and fault mechanisms on which release claims depend:

1. **Worst-case execution time (WCET) and scheduling jitter**: Profiling whether safety filter evaluations complete within their hard real-time deadlines ($T_{\text{exec}} \leq T_{\text{deadline}}$) while neural workloads drive peak memory crossbar contention and DMA transfers, the falsification campaign of section 13.7.
2. **Deterministic bus arbitration**: Measuring fieldbus communication jitter and cyclic transmission integrity across physical EtherCAT or CAN FD networks under simulated telemetry broadcast floods and electrical noise injection.
3. **Power-stage thermal dynamics and current ripple**: Verifying that high-frequency control chatter or torque ripple proposed by the candidate policy does not induce excessive inverter switching losses, DC rail voltage sag, or thermal runaway in motor windings (section 2.5).
4. **Hardware fault-injection survivability**: Validating that physical line breaks, power supply brownouts, or deserializer frame dropouts deterministically trigger the hardware watchdog and safe-stop sequence within certified timing budgets before motor power stages can be damaged.

A candidate policy advances through bench staging only when every required regression test passes and empirical records confirm that timing and physical margins remain intact. Observing zero deadline overruns during a finite bench campaign does not by itself establish a mathematical upper bound on worst-case execution time, nor does it show how the policy will behave on the warehouse floor. In cloud software engineering, blue-green deployment can reduce downtime by preparing a replacement cluster and redirecting traffic after readiness checks. In physical AI, however, blue-green cutover cannot happen mid-stride or at speed. Swapping active control authority between an incumbent controller and a candidate policy while mass is moving requires managing actuator state continuity, motor winding back-EMF, and dynamic tracking tubes (section 14.4.1). De-energizing motor windings or inducing sudden torque steps causes severe mechanical shock loads ($N_{\text{gear}}^2 J_{\text{rotor}} \ddot{\theta}_{\text{link}}$) and loss of traction. An unvalidated policy also cannot be granted partial motor authority in a populated warehouse to "canary test" edge-case performance.

Safe predeployment staging instead relies on **shadow mode deployment**, the shadow-operation evidence regime of section 7.4.1 applied to a release candidate. The incumbent controller $\pi_{\text{incumbent}}$ supplies proposals, while the independent enforcer retains sole actuator-output ownership and checks each command before fieldbus dispatch. Concurrently in the background, the candidate neural policy $\pi_{\text{candidate}}$ executes on the host compute SoC, consuming identical live sensor streams $\mathbf{o}_t$ within its qualified shared-resource budget and computing what control command $\mathbf{u}_t^{\text{cand}} \sim \pi_{\text{candidate}}(\mathbf{o}_t)$ it would have proposed. The candidate writes its outputs to an isolated telemetry buffer that grants no actuator authority. Consumers can read that record, but neither the buffer nor the candidate can bypass the enforcer. Shadow inference and recording must stay within the interference and power budgets that preserve the active case. Discrepancies between candidate proposals and the incumbent's admitted commands are logged as high-rate telemetry events without admitting any candidate command to the drives.

Shadow-mode telemetry monitors two instantaneous divergence metrics:

1. **Action discrepancy** ($\Delta \mathbf{u}_t$): The Euclidean (or angular) distance between the candidate proposal and the incumbent control action, measured here by the Euclidean norm:

$$\|\Delta \mathbf{u}_t\|_2 = \|\mathbf{u}_t^{\text{cand}} - \mathbf{u}_t^{\text{incumbent}}\|_2.$$

Sustained spikes in $\|\Delta \mathbf{u}_t\|_2$ identify behavioral divergence where the candidate policy introduces unmodeled bias, high-frequency control jitter, or erratic steering commands. In addition, the discrepancy analyzer monitors the command-rate norm $\|\dot{\mathbf{u}}_t^{\text{cand}}\|_2$, which is a torque-rate norm only when $\mathbf{u}$ is a torque command, to detect unmodeled control chatter that would excite mechanical resonances.

2. **Safety barrier margin discrepancy** ($\Delta \dot{h}_t$): The differential proximity to forward-invariant safety boundaries under a control barrier function $h(\mathbf{x})$ (section 12.5):

$$\Delta \dot{h}_t = \dot{h}(\hat{\mathbf{x}}_t, \mathbf{u}_t^{\text{cand}}) - \dot{h}(\hat{\mathbf{x}}_t, \mathbf{u}_t^{\text{incumbent}}),$$

where $\hat{\mathbf{x}}_t$ is the state estimated from the sensor streams $\mathbf{o}_t$. A negative $\Delta \dot{h}_t$ means the candidate has a smaller modeled barrier derivative than the incumbent. Both derivatives can still be positive, so the sign alone does not imply decreasing clearance or a violated boundary of $\mathcal{C}$ (defined by $h(\mathbf{x})$).

The two discrepancies answer different questions. A large action discrepancy $\|\Delta \mathbf{u}_t\|_2$ can reflect an alternate useful action, while $\Delta \dot{h}_t$ indicates how the proposals differ in their modeled effect on the barrier. Neither sign alone supplies a verdict; review must consider the current margin, uncertainty, and the condition that the independent enforcer would require.

Shadow mode nevertheless has a limit that more traces cannot remove, the trajectory confinement of that evidence regime, which a release reviewer encounters as **counterfactual divergence**. Because admitted incumbent proposals determine the plant trajectory, the historical state trajectory $(s_0, \mathbf{u}_0^{\text{incumbent}}, s_1, \mathbf{u}_1^{\text{incumbent}}, \dots)$ is conditioned entirely on incumbent actions, and the candidate's proposals $\mathbf{u}_t^{\text{cand}}$ never alter it. Conflating open-loop agreement with deployment safety is the **open-loop replay fallacy**: assuming that high action agreement during passive trace replay guarantees closed-loop dynamic stability. The $\mathcal{O}(S^2\epsilon)$ result in section 5.3 is an expected-cost bound under its stated imitation-learning assumptions, not a universal law for physical tracking error. A candidate that agrees on incumbent trajectories can still visit unfamiliar states or excite unmodeled dynamics when it closes the loop, the covariate shift that interactive imitation learning was designed to correct (Ross et al. 2011). Shadow mode is an essential pre-filter for gross behavioral anomalies, but never a substitute for closed-loop dynamic qualification.

Phased canary fleet rollouts[12] expose the candidate to closed-loop operation under a separately supported restricted case. Selecting fewer machines limits exposure but does not bound the consequence of any one action. The restrictions must address the physical hazard:

1. **Geofenced operational containment**: Canary units operate exclusively within dedicated, surveyed low-risk corridors, such as wide transit aisles devoid of blind rack corners, reduced pedestrian density, and high-friction surveyed concrete ($\mu \geq 0.60$).
2. **Enforced kinematic derating**: The real-time safety enforcer clamps candidate policy velocity and acceleration below nominal ratings ($v \leq v_{\text{canary}} < v_{\text{nominal}}$), expanding stopping distance margins and suppressing transient kinetic energy.
3. **Supervised human oversight**: High-readiness human supervisors equipped with wireless remote emergency-stop (tele-stop) pendants maintain line-of-sight monitoring over canary units, backed by an active-lease watchdog.
4. **Targeted regression observability**: The trial measures tracking error variance σ^2_{track}, drive torque ripple, and enforcer intervention frequency to reveal service regressions. The permission path retains only the containment claims validated for the restricted envelope.

Expansion requires predeclared acceptance criteria tied to the service and physical claims. Intervention-rate non-inferiority[13] and tracking variance $\sigma^2_{\text{track}} \leq \sigma^2_{\text{incumbent}}$ are possible capability criteria, with their comparison conditions and statistical assumptions stated. The non-inferiority margin δ is fixed before the trial, so that the upper one-sided 95 percent confidence bound on the rate ratio $\lambda_{\text{canary}}/\lambda_{\text{incumbent}}$ must fall below $1+\delta$. With a zero margin, that upper-bound criterion requires evidence that the candidate is better, which may demand more exposure than a small canary provides. The trial arithmetic of section 7.7 sets how much operation a given δ requires. A sequential test can stop expansion as soon as the evidence suffices. None of these criteria replaces the evidence for stopping, observation, or containment. An enforcer intervention also needs causal

[12] **Canary** (the miners' sentinel bird): Miners carried caged canaries because the birds succumb to carbon monoxide before people do, so a bird's distress warned the crew to leave. The metaphor carries this section's warning. The canary is the unit exposed to the hazard first, so on a physical fleet its exposure must be bounded by containment, not by how few units are chosen.

[13] **Non-Inferiority Testing**: The design comes from clinical trials, where failing to find a difference proves nothing, so a new treatment must instead be shown worse than the standard by no more than a margin fixed before any data arrive (Blackwelder 1982). A canary can therefore pass on modest exposure without proving the candidate better, but only if δ was not chosen after the result was seen.

review, because suppressing legitimate clamps to improve a rollout metric would weaken the release argument.

16.9.3 From qualification to a new manifest

Reducing a policy's hesitation rate at the pick station does not justify relaxing its contact-force limits or floor-friction restrictions. If the retrained mug-delivery candidate were promoted on task speed alone, it would bypass the containment premises that keep the restricted route safe. Instead, the reviewer repeats closed-loop capability evaluation under the restricted route and feed conditions, and repeats the loaded timing and fault tests whose dependencies changed. Unaffected plant evidence remains reusable only while configuration, calibration, wear, and site premises still match.

Assume, still within the stipulated branch, that those results satisfy the predeclared capability criteria and preserve the containment claims. The adjudicator issues a new manifest binding the candidate, retained and repeated evidence, current calibration, and the same restricted mission. At the assessed stationary cutover state, the gate authenticates that manifest and checks current standing conditions before admitting new proposals. Field monitoring then compares the running service with the same success, cycle-time, rescue, and protection criteria. Failed qualification leaves the incumbent case unchanged; a failed standing condition withdraws the affected grant.

16.9.4 Over-the-air staging and watchdog-enforced rollbacks

A candidate written to the inactive bank inherits no operating grant from its predecessor, and rolling back to the prior software image cannot repair a worn brake or a stale calibration. **Hardware watchdog rollback triggers** must therefore distinguish a failure to boot from a failure during motion. Before tracking is enabled, failed image authentication, startup tests, or fieldbus synchronization can reject the candidate and select the retained image, which must still pass current calibration and standing-condition checks before receiving a permit.

During motion, the independent supervisor first commands the assessed fallback within its measured response budget. Where that response ends in torque removal, the supervisor asserts Safe Torque Off after the required controlled stop; a plant needing a powered hold retains the authority needed for that hold. Unlike a conventional compute watchdog that resets the processor immediately upon a timeout, a missed enforcer heartbeat withdraws tracking permission and triggers the hardware-controlled stop without resetting the processor mid-motion. Only after the plant has come to rest in its validated update state may the boot selector change banks.

The image and its compatible calibration schema form the rollback unit. If either record is missing, corrupted, or physically stale, the gate selects `REFUSE` unless a separately signed restricted case covers the actual state and its limits, such as $v \le v_{\text{restricted}}$. Retaining the prior binary image provides an immediate rollback target, but operators must re-validate standing physical conditions before the recovered binary receives an operating permit.

16.9.5 Event-triggered telemetry and safety performance indicators

An enforcer trip can reveal a policy failure, a changed physical premise, or both, and the field record must let the reviewer distinguish those causes before choosing retraining, maintenance, or a narrower envelope. Because wireless summaries alone cannot reconstruct the observations and actuator response around the event, detailed traces are retained locally. **Edge event-triggered telemetry** captures the sensor, proposal, permission, actuator, and timestamp chain around an intervention or boundary excursion. A local rolling buffer preserves the lead-up, and capture continues through the physical response whenever the qualified recording power path survives. Buffer length, write bandwidth, and hold-up energy are sized from the required reconstruction window and verified under the same contention and power faults as the permission path. A logic-rail capacitor does not guarantee that the full sensor and storage path survives, and recording must never delay the stop.

UL 4600 calls the field measurements that keep a claim alive **safety performance indicators** (SPIs). Each is a metric tied to a named claim, with a threshold and a predeclared response when the threshold is crossed (UL Standards & Engagement 2023). On the running machine, every standing condition of section 16.5 has one. Measured stop distance at scheduled trials and enforcer trips per operating hour watch the base's stopping claim, age refusals per operating hour watch the

↩ The sequential probability ratio test, which stops a trial campaign as soon as the evidence crosses a precalibrated boundary, is established in section 7.7.3.

observation-age claim, and localization residuals against their allowance watch the sensor geometry. An SPI whose threshold is crossed withdraws or narrows the grant it watches; one that drifts toward its threshold triggers review before it is crossed, which is the defense against the normalization of deviance of section 16.7.

During routine operation, compact health summaries support fleet monitoring while required incident records retain full provenance locally until transfer succeeds. Learning-data filters may prioritize unfamiliar events for training, provided they never discard required safety evidence or hide repeated failures. Once the traces arrive, a reviewer distinguishes policy hesitation from calibration drift, surface contamination, and mechanism wear before assigning the repair: qualified recovery traces can enter the experience pipeline, whereas reconstructed simulations extend test coverage only over their modeled conditions. A captured event therefore changes the release argument only after its physical cause and operating conditions are established. Reconstructed scenes and retraining can improve a candidate policy, yet neither automatically repairs a physical warrant, so the revised claim must return through the relevant qualification stages before a new manifest grants authority. Because the traces the machine keeps also record the people around it, each record that identifies a person carries standing conditions of its own: a declared purpose, a retention period, and a list of readers, audited like any other premise.

↪ The declarations that govern records identifying people, and the access and retention audit that enforces them, are developed in section 17.7.1.

16.9.6 Fleet evidence and accountable release

Teams break the safety warrant when they pool dissimilar routes into a single fleet mileage total or treat an operator intervention count as a collision rate. Fleet exposure can reveal recurring failures, but interpreting it requires knowing the operating domain, the intervention definition, and the exact system configuration. The release reviewer uses fleet records to challenge the same capability and containment claims, rather than replacing them with a mileage ranking. One public fleet record shows what an interpretable version looks like. Waymo publishes its rider-only crash rates by outcome, each against a human-driven benchmark adjusted to the same locations and each with a confidence interval. It republishes them as rider-only mileage accumulates (Waymo 2026; Kusano et al. 2025). Figure 16.4 plots the latest release and the history of two of its outcomes.

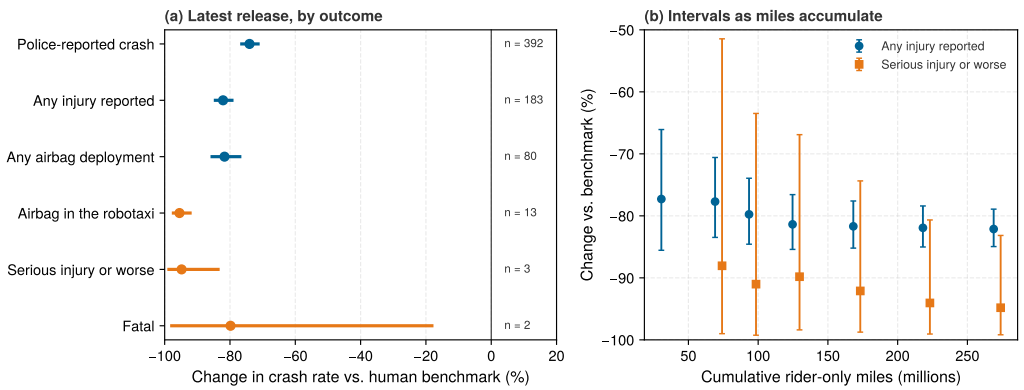


Figure 16.4: Exposure licenses claims one outcome at a time: (a) Percent change in crash rate against a location-adjusted human benchmark for each outcome in the latest public release, with 95 percent confidence intervals and the number of fleet crashes behind each estimate; orange marks outcomes with fewer than 20 crashes. (b) The same estimate and interval for two outcomes at each release, against cumulative rider-only miles. Common outcomes converge to narrow intervals; the serious-injury and fatal estimates rest on a few events, so their intervals stay wide however favorable the point estimate. Data from Waymo (2026).

In the June 2026 release, after 271 million rider-only miles, the injury estimate rests on 183 crashes and its interval spans a reduction of 79 to 85 percent. The serious-injury estimate rests on 3 crashes and spans 83 to 99 percent, and the fatal estimate rests on 2 and spans 18 to 98 percent. The same mileage therefore licenses a tight claim about common crashes and only a loose one about the outcomes a release decision cares about most, because rare events accumulate slowly however many miles the fleet drives. That is the fourth law, permission must be earned (principle 4), measured on

a real fleet. Each outcome's claim may extend only as far as its own interval, and a reviewer reads the interval before the point estimate.

The signed release manifest records the engineering decision and its accountable owner, and together with the records it binds, it lets an investigator follow one admitted action back to the person who signed for it. The enforcement record logs the admitted command. The manifest in force at that moment names by digest the policy weights, the enforcer and real-time operating system binaries, the calibration tables, and the operating envelope behind it. The signature on that manifest, made with a key held in a hardware security module, is bound to the sign-off record that names the lead systems adjudicator who accepted that configuration or the update that installed it. The manifest does not itself establish regulatory compliance or determine legal liability. Applicable requirements depend on the machine, intended use, and jurisdiction; the release manifest must preserve the evidence needed for that separate assessment. Throughout this lifecycle, release remains a decision about a named machine under named conditions. Shadow traces, staged trials, and field records can strengthen the next argument only when their operating conditions and limits are explicit, while the manifest and independent permission path retain the restrictions that the evidence supports.

↪ Who answers for each decision the manifest binds, and the duty to report an incident completely and on time, are developed in section 17.9.

16.10 Fallacies and Pitfalls

In purely digital software, a deployment bug manifests as a silent crash, a corrupted database row, or a dropped network packet; in physical AI, software decisions drive heavy actuators through real space where errors bend steel, destroy machinery, and injure coworkers. Releasing an embodied AI system therefore requires uncovering hidden assumptions before kinetic energy is coupled to physical mass. The following fallacies and pitfalls highlight recurring mistakes where teams confuse software statistical confidence with cyber-physical safety.

Fallacy: *Statistical coverage substitutes for deterministic architectural warrants.*

Engineering teams frequently present millions of simulated hours and thousands of incident-free test cycles as primary justification for deployment release. While physical and simulated exposure provides valuable empirical evidence regarding typical operating regimes, proving the ultra-low catastrophic failure rate required by a safety release claim demands far more operational hours than any physical test campaign can execute. Simulation environments also depend on idealized models, such as rigid-body dynamics, simplified Coulomb friction, and linear actuator response. Empirical performance and architectural warrants answer different questions. A large trial count cannot substitute for architectural enforcement, such as an independent permission path that revokes motion and executes a certified safe stop. The safety case must establish how deterministic runtime guards actively constrain actuator authority to provably safe physical limits when the learned policy encounters out-of-distribution inputs.

Statistical sampling bounds hazard rates only under observed operational distributions; trial volume cannot substitute for deterministic architectural mechanisms that constrain actuator authority.

Architect's countermeasure: Couple learned policy evaluation with independently qualified safety monitors. Explicitly state the hazard models, plant dynamics, execution deadlines, and safe-stop fallback conditions under which the system's safety invariants are deterministically enforced.

Fallacy: *A release verdict is a permanent certification of safety.*

A machine passes formal validation and enters production under a signed safety case. Over months of operation, brake friction material wears down and lengthens stopping distances, kinematic calibration drifts under daily thermal cycling, and dirt or optical scratches degrade sensor fidelity. Even when the deployed software binaries remain bit-for-bit identical, the physical assumptions underpinning the original release case gradually erode. A signed release verdict certifies safe operation only within a bounded maintenance window and under strict operational thresholds. To maintain safety, the release manifest must establish explicit invalidation triggers and scheduled maintenance intervals tied to hardware lifecycle counters, physical wear measurements, and calibration drift estimates. Once these operational limits are exceeded, the system must automatically restrict its capabilities or halt operation until recalibration and service occur.

Software bit-exactness cannot prevent mechanical wear, material fatigue, or sensor degradation.

Architect's countermeasure: Treat release manifests as perishable operational leases bounded by operating hours, calendar age, and mechanical cycle counts. Implement automated recalibration self-tests at boot, and require diagnostic verification before energizing high-power motor stages.

Pitfall: *Relying on unstated environmental assumptions to support a safety claim.*

During commissioning, a team verifies a mobile base's stopping distance in brightly lit warehouse aisles on clean concrete, documenting floor friction in the safety case while leaving ambient lighting unstated. In production, when the robot enters a dimly lit cross-aisle, longer camera exposure times increase image processing latency, which an age check flags at runtime (section 16.3). Concurrently, sparse visual feature matches degrade state estimation accuracy, introducing unmonitored localization drift that consumes the clearance buffer calculated from well-lit testing (section 16.7). The mechanical brake engages exactly as specified and low-level controller diagnostics report all systems nominal, yet the actual stopping distance margin quietly vanishes. Safety claims depend on operational surroundings. If an environmental boundary is omitted from the safety case, the deployment architecture can neither verify the operating conditions at release nor enforce them at runtime.

Unstated operating assumptions silently erode safety margins until an ordinary environmental fluctuation precipitates a collision.

Architect's countermeasure: Formally document every environmental assumption (such as ambient illuminance in lux, surface friction coefficients, and ambient thermal bounds) as an explicit premise in the release manifest. Pair each premise with a deterministic runtime detector or an enforced facility operational specification.

Fallacy: *Shadow-mode qualification guarantees closed-loop dynamic stability.*

An engineering team evaluates a candidate neural policy in shadow mode across thousands of fleet operating hours, recording near-zero action discrepancy ($\|\Delta \mathbf{u}_t\|_2 \approx 0$) between the candidate policy and the certified incumbent controller. The team infers that high behavioral fidelity during open-loop playback proves the candidate policy is ready for autonomous deployment. However, because the incumbent controller retains active authority over the plant, the state distribution is entirely dictated by the incumbent's dynamics. The candidate policy never experiences the compounding physical consequences of its own commands. Even minor discrepancies in control output can excite unmodeled mechanical resonances, induce severe actuator chattering, or generate phase lag that destabilizes the physical plant the moment control authority is transferred to the closed loop.

Open-loop trace replay conceals covariate shift, compounding autoregressive errors, and unmodeled plant resonances.

Architect's countermeasure: Treat shadow mode strictly as an initial filter to catch gross algorithmic discrepancies, never as proof of dynamic stability. Require closed-loop Monte Carlo sweeps across perturbed plant models, followed by supervised canary execution in isolated physical test facilities, before granting live actuation authority.

Fallacy: *Web-style canary traffic routing is directly applicable to physical machines.*

In cloud engineering, canary deployments safely route a small fraction of live network traffic to a newly deployed service, rolling back the deployment if error rates exceed a set threshold. Attempting to mirror this practice by assigning a percentage of live operational missions to an unvetted neural policy overlooks the physical reality of mass and momentum. A 368 kg vehicle moving at 1.3 m/s carries 311 J of kinetic energy. If an unverified model issues an erratic actuation command near an operator, the resulting collision cannot be mitigated by rolling back a container image or tearing down a replica set.

Kinetic energy and physical momentum render live production canary routing hazardous without deterministic architectural containment.

Architect's countermeasure: Restrict canary deployments to physically geofenced zones devoid of human personnel. If testing in active production environments is necessary, encapsulate the unvetted policy within an independent, non-preemptible runtime safety enforcer. This permission path must deterministically maintain the vehicle within its certified viable state space across all

declared sensing, plant, timing, and fault models, relying on hardware watchdog leases and certified safe-stop drive functions to override invalid actions.

Fallacy: *Over-the-air updates can be treated as stateless software deployments.*

In web architectures, stateless services restart cleanly from immutable container images. Treating OTA updates on physical robots as simple container replacements ignores the reality of hardware-bound state. Every physical machine possesses unique hardware calibrations: camera extrinsic matrices, joint zero-position encoder offsets, and load-cell tare values, alongside individual mechanical wear profiles. If an OTA update overwrites non-volatile storage or fails to migrate calibration schemas, the updated model executes with distorted sensor transformations. Even minor coordinate offsets introduce spatial localization errors that rapidly deplete geometric clearance budgets and invalidate the stopping guarantees established in the safety case.

Embodied systems are stateful; software updates cannot be decoupled from machine-specific physical calibration.

Architect's countermeasure: Decouple machine-specific calibrations into an isolated, schema-versioned non-volatile partition (such as EEPROM or write-protected flash) that persists across operating system bank swaps. Use dual A/B system partitions paired with hardware watchdogs that transition the plant into a safe, depowered, or mechanically locked state before changing the active bank. Require automated calibration validation and power-on self-tests before transferring motion authority to the updated partition.

Fallacy: *Continuous high-bandwidth wireless telemetry streaming is superior to edge event-triggered buffering.*

To achieve comprehensive operational visibility, an engineering team attempts to stream raw multimodal sensor data (high-resolution video, dense lidar point clouds, and 1 kHz motor current logs) continuously over facility Wi-Fi or cellular networks to cloud dashboards. In dense fleet deployments, wireless channel contention, radio frequency (RF) shadowing in industrial bays, and access-point roaming cause severe packet loss and queue bloat precisely during rapid, safety-critical dynamic events. Continuous network serialization also consumes substantial CPU time, DMA channels, and memory bus bandwidth, creating bus contention that can starve real-time safety controllers of deterministic execution guarantees.

Wireless bandwidth is physically constrained and vulnerable to RF fading; continuous raw streaming alone cannot guarantee preservation of telemetry during critical dynamic anomalies.

Architect's countermeasure: Transmit only compact, low-bandwidth health heartbeats and summary metrics over wireless networks during routine operation. Maintain continuous, high-rate circular telemetry buffers in local non-volatile storage. When a safety fault, anomalous dynamic event, or emergency stop occurs, latch the pre-trigger and post-trigger event windows locally. Size storage write bandwidth and write endurance to prevent memory bus interference with real-time safety enforcers, and offload event packages once the machine establishes a dedicated, high-bandwidth maintenance link.

16.11 Summary

A deployment release formalizes which physical claims the empirical evidence licenses for a specific physical machine, under what operating conditions, and for how long. For the warehouse mobile manipulator examined across these chapters, release resolves to three concrete physical inequalities. The mobile base stops within surveyed clearance. The manipulator unlatches the cage door below the latch's mechanical force limit. A mug handover remains strictly below the contact criterion for an unconstrained human hand. A formal claim-argument-evidence safety case links each inequality to its supporting premises and experimental evidence within a clearly demarcated operating envelope. Because required premises combine conjunctively, an argument is no stronger than its least supported premise; empirical confidence cannot be averaged across heterogeneous requirements. Independent sufficient legs strengthen support in the stipulated probability model, which estimates neither this robot's hazard rate nor calibrated assurance confidence. In the physical argument, a policy error cannot breach a monitored invariant only while the qualified enforcement, plant, sensing, timing, and fault premises hold; C5 isolation evidence does not by itself establish statistical independence.

Recognized machinery safety standards govern this argumentation structure. Within the permission path, only the safety microcontroller's lease-expiry timer and heartbeat watchdog are rated safety functions; the other formal protective claims rest on the certified drive, scanner, and arm safety functions beneath it.

An adjudicator evaluates the safety case to deliver one of three definitive verdicts: unconditional release (`UNCOND`), conditional release (`COND`), or refusal (`REFUSE`). If release is granted, standing conditions enforce operational boundaries in real time, automatically withdrawing authorization whenever any relied-upon premise becomes false or unobservable. Records that identify people carry standing conditions of their own, declared and audited like the physical premises (section 17.7.1). Against this book's illustrative input datasets, every primary claim contains at least one ungrounded premise, compelling a verdict of `REFUSE`. A separately stipulated branch illustrates the empirical evidence that would justify restricted conditional service (`COND`). Before final authorization, an adversarial review must try to break the entire argument, and that scrutiny must recur throughout the operational lifespan. A cryptographically signed manifest binds the release verdict to reviewed software and calibration artifacts by cryptographic digest, failing closed if any binary or configuration changes. Any subsequent model or firmware update must return through the same safety case, because open-loop shadow operation cannot evaluate the physical consequences of unexecuted closed-loop control actions. Table 16.7 collects the load-bearing equations governing deployment release, detailing the physical or statistical boundary each expression establishes and noting where each was derived in earlier chapters.

Table 16.7: Key equations of deployment release: Each load-bearing equation of the chapter, the physical boundary it bounds or decision it settles, and the equation where it is derived. Stopping, contact, latch, and exposure relationships originate in preceding chapters; the release case incorporates their mathematical guarantees as foundational premises.

Equation	What it bounds or decides	Derived in
$d_{\text{stop}} \le D_{\text{clear}},\ F_{\text{contact}} \le F_{\text{limit}},\ T_{\text{exec}} \le T_{\text{deadline}}$	The three physical criteria every release claim must evidence: stopping clearance, contact force, and execution timing	equation 16.1
$v\,\tau_{\text{delay}} + d_{\text{suffix}}(v) + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}} \le D_{\text{clear}}$	The mobile base stopping budget, whose unspent margin every premise of the stopping claim draws on	equation 12.1
$F_{\text{unbraked}} = v\sqrt{k_H\, m_{\text{red}}}$	The peak transient handover force (m_{red} is the hand-arm reduced mass), establishing the maximum approach speed permitted by the contact criterion	equation 12.13
$F_{\text{cmd}} = F_{\text{trip}} + k_{\text{latch}}\, v_{\text{latch}}\, T_c$	The contact force developed against a door latch before arm braking begins	equation 12.11
$F_{\text{cmd}}/k_{\text{latch}} + v_{\text{latch}}^2/(2a_{\text{arm}}) \le F_{\text{lim}}/k_{\text{latch}}$	The arm deceleration the door-opening branch must guarantee, bounding admissible approach velocity	equation 12.12
$T \ge 3.0/\lambda$	The failure-free testing exposure required, at 95% confidence, to demonstrate a failure rate λ per operating hour, showing why empirical exposure alone cannot validate catastrophic risk claims	equation 7.9
$\Pr(C) = \prod_i p_i$	Probability of an exact conjunction in the stipulated independent-premise model; not measured robot hazard or calibrated assurance confidence	equation 16.2
$\Pr(C) = 1 - \prod_j (1 - r_j)$	Probability of an exact union in the stipulated independent-leg model; C5 isolation alone does not establish statistical independence	equation 16.3

Key Takeaways: Deployment release and safety cases

- **The envelope bounds the claim:** A release claim covers only the speeds, payloads, surface friction coefficients, clearances, and contact scenarios for which empirical evidence was systematically collected. Every dimension of the operational design domain requires an active runtime monitor or a strictly enforced site procedure.
- **Doubt accumulates along required premises:** A conjunctive argument ($\bigwedge_i P_i$) is no stronger than its weakest required premise and is often strictly weaker; confidence cannot be averaged across premises. High empirical accuracy in perception cannot offset an unmonitored braking margin or an uncharacterized tail latency. Independent sufficient legs reduce doubt under the stipulated probability model. Policy validation and an isolated permission path do not automatically constitute such legs; the physical containment claim still requires qualified plant, sensing, timing, and fault premises.
- **Standing conditions make release revocable:** Every relied-upon premise requires an active detector or operational procedure, an explicit checking cadence, and a validated safety action triggered whenever the premise evaluates to false or unknown. The signed safety case binds these obligations directly to the machine across its operational life.
- **Updates return through the release argument:** Shadow deployment reveals policy divergence without testing the closed-loop physical consequences of those divergent actions. Rigorous closed-loop qualification and a re-evaluated manifest still govern deployment authorization; every update must preserve machine-specific calibration and execute a validated stop or hold during software cutover.
- **An unknown premise refuses, and refusal names the next work:** In safety argumentation, unknown envelope membership refuses exactly as a measured violation does, while its cause is logged separately. On this book's illustrative walkthrough, the mobile manipulator receives an explicit verdict of `REFUSE`. A structured refusal is productive: it isolates the missing measurements, identifies the claims they would ground, and defines the residual operational envelope that remains once those gaps are closed.
- **Useful service needs its own evidence:** The stipulated restricted mug-delivery workflow achieves `COND` only by satisfying concrete task-success, cycle-time, and human-intervention criteria alongside safety claims C1–C6. Safety filtering can preserve high task completion rates even when raw proposer policy autonomy is comparatively modest.
- **No verdict moves the proposal boundary:** Under both `UNCOND` and `COND`, learned policies merely propose actions while the hardware permission path decides physical admission. Within that permission path, only the safety MCU's lease-expiry and heartbeat watchdog is a rated safety function; the admission checks are not rated. Entering a restricted operational mode requires its own signed release case, never an informal or ad hoc runtime downgrade.

The fourth law, *permission must be earned* (principle 4), establishes that permission to operate must derive strictly from recorded evidence and expire whenever that evidence lapses. Standing conditions transform this principle into an active software and hardware mechanism. Each condition couples a required premise with a dedicated detector, a monitoring cadence, and an enforced safety response. Consequently, the loaded stopping trials that would support a conditional release for the warehouse mobile manipulator become binding empirical warrants whose invalidation immediately terminates authorization. Under this book's illustrative records, that grant is never issued, because each of the six primary claims relies on at least one premise that remains ungrounded or unknown. The stipulated delivery branch demonstrates how empirical field trials could substantiate useful restricted service and specifies the runtime conditions necessary to sustain that grant. Just as a manual lease restricts human or teleoperation authority to the validity window of its evidence, and precommitment binds test evaluation to rules established before observations occur, deployment release binds fleet deployment to verifiable physical limits. Release is the widest operational authority

granted in this book, and even it never reaches the actuators directly. Every motion it licenses still crosses the causal boundary only through the permission path and the rated functions beneath it.

↩ The manual lease that bounds human authority to the validity window of its evidence is established in section 14.3.4.

16.12 Historical Perspective and Further Reading

Long before it became an engineering artifact, the modern safety case originated in the philosophy of practical reasoning. Toulmin (1958) decomposed argument structures into concrete elements: an explicit claim, the ground data offered to support it, an inferential warrant that licenses the logical step from data to claim, and the rebuttals that identify conditions under which the warrant fails. That structural anatomy survives almost unchanged in modern deployment release arguments. In the late 1990s, safety engineers translated these philosophical foundations into auditable systems practices. The Claim-Argument-Evidence (CAE) methodology introduced by Bishop and Bloomfield (1998) and the Goal Structuring Notation (GSN) formalized by Kelly (1998) transformed implicit engineering convictions into graph-structured arguments that an independent reviewer can inspect link by link. Later, the autonomous vehicle community adapted this goal-based assurance framework in ANSI/UL 4600 (UL Standards & Engagement 2023), requiring explicit argumentation over positive and negative evidence, concrete safety performance indicators, and rigorous independent assessment.

Two foundational results established the statistical limits of empirical testing, shaping what a release argument can claim. First, Butler and Finelli (1993) proved that the ultra-low failure rates demanded of life-critical software cannot be validated through testing alone; verifying a rate of 10^{-9} failures per operational hour demands on the order of 10^{9} failure-free test hours. Decades later, Kalra and Paddock (2016a) applied that same arithmetic to embodied autonomy, showing that an autonomous fleet would have to drive billions of operational miles to establish statistical parity with human drivers. A hazard mitigation claim therefore cannot rest solely on driving miles; it requires an architectural warrant, such as formally verified runtime safety envelopes and deterministic interlocks, alongside empirical testing. Second, redundancy cannot be assumed to yield statistical independence. The classic empirical study by Knight and Leveson (1986) demonstrated that independently written software implementations suffer correlated failures far more often than independent probability models predict. As a result, a multi-legged safety argument (Bloomfield and Littlewood 2003) must provide empirical and architectural evidence for the independence of its redundant legs rather than treating diversity as an axiom. On the running machine, claim C5 carries that burden.

The final historical lesson concerns the temporal fragility of assurance. The Rogers Commission report on the Space Shuttle Challenger disaster (Presidential Commission on the Space Shuttle Challenger Accident 1986) and Diane Vaughan's subsequent sociological analysis (Vaughan 1996) revealed that safety cases rarely collapse from an isolated blunder. Instead, arguments degrade through the "normalization of deviance," a slow organizational erosion in which unexplained anomalies are repeatedly rationalized until a catastrophic threshold is breached. Dynamic safety cases (Denney et al. 2015) counter this failure mode by maintaining the argument as an active runtime process, continually validating underlying assumptions against operational telemetry. Because embodied systems require continuous post-deployment evolution, compromise-resilient update frameworks such as TUF (Samuel et al. 2010) and Uptane (Kuppusamy et al. 2016) established that the software delivery channel is itself a primary attack surface. The release architecture developed here joins these lines of work. The deployment verdict is cryptographically bound to a signed manifest validated by the target controller before motion begins, and every environmental assumption remains a standing operational condition whose violation revokes execution authority within the qualified detection-and-response bound.

Six foundational works reward careful study, each anchoring a distinct dimension of the release argument:

- Toulmin (1958) introduces the claim, warrant, backing, and rebuttal framework that underpins modern structured argumentation, establishing why inferential warrants must be scrutinized independently from raw empirical data.
- Bishop and Bloomfield (1998) articulates the Claim-Argument-Evidence (CAE) framework for critical software systems, providing the structural basis for the assurance scorecard in table 16.2.

- Kelly (1998) introduces Goal Structuring Notation (GSN) and formalizes pattern-based approaches for managing safety cases across system lifecycles, establishing the graphical formalism illustrated in figure 16.1.
- Knight and Leveson (1986) provides empirical evidence of correlated failures in diverse redundant software, demonstrating why claims of subsystem independence must be actively verified rather than taken as an architectural postulate.
- Vaughan (1996) reconstructs the organizational and technical breakdown of the Challenger launch decision, defining the concept of normalization of deviance that runtime safety performance indicators are designed to detect and arrest.
- Koopman and Wagner (2016) identifies inductive machine learning among the core challenges confronting autonomous vehicle validation, proposing phased operational envelope expansion, hardware-isolated runtime monitors, and systematic fault injection, the triad that this chapter's release protocol builds on.

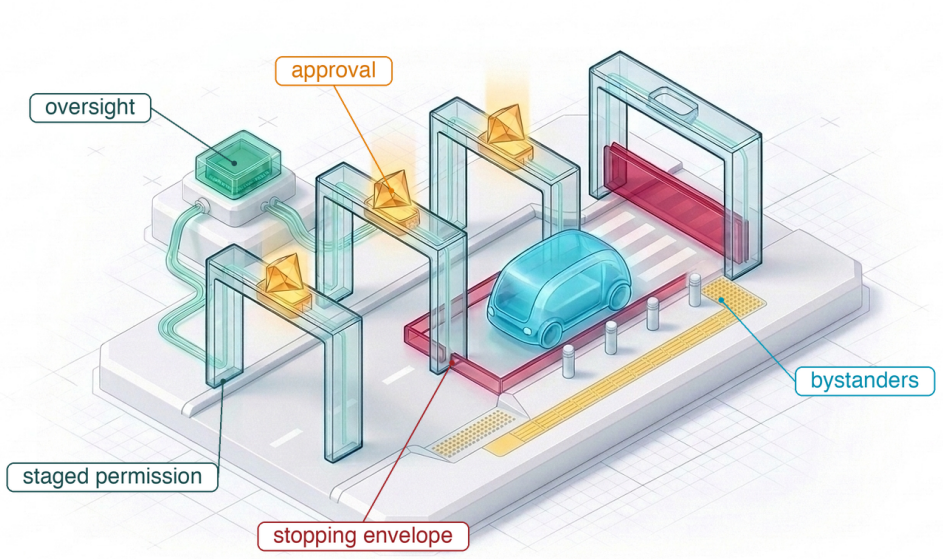
What's Next: From release to responsibility

Who answers for the decisions behind a signed manifest? The manifest names the adjudicator who accepted the configuration and its operating envelope. Yet the crossing rule, the staffing of the remote assistant, the permit under which the machine runs, and the uses of the records it keeps were each decided by someone else, and none of them appears in a per-tick log. Chapter 17 traces those decisions to named owners. It tests whether each human role the safety argument relies on is one a person can perform. It converts gaps in how well the machine detects different groups of people into gaps in stopping distance. It collects all of it in an approval record that the release gate reads, so that a decision with no owner refuses operation the way an unknown premise does.

DRAFT

17

Responsible Physical AI



17.1 Who Answered for the Cruise Collision
17.2 Decisions Behind an Admitted Action
17.3 Oversight a Person Can Perform
17.4 Permission in Stages
17.5 When Detection Depends on Who Is Detected
17.6 Who Bears the Risk
17.7 Data About People
17.8 The Labor the Machine Moves
17.9 The Approval Record
17.10 Fallacies and Pitfalls
17.11 Summary
17.12 Historical Perspective and Further Reading

Purpose

Who answers for a machine that moves on its own when its motion harms someone?

A machine's logs can show, tick by tick, what it sensed, proposed, and admitted, and still not show who decided that it could travel at that speed, among those people, with that person as its only backup. When harm arrives, blame settles on whoever stood closest to the event, often an operator handed a duty no person could have carried out, while the decisions that set the machine's limits stay spread across engineering teams, suppliers, site managers, and regulators. The machine's effects also reach people who never chose it: pedestrians and coworkers whose detection rate was never measured for people like them, workers whose jobs shift to rescuing and watching it, and bystanders whose faces fill the records kept to explain its mistakes. A machine is fit to be given authority only when each decision behind its actions names the person who answers for it, each human role it relies on is one a person can actually perform, and each group that bears its risk or its recording has been counted. Assigning responsibility after a collision cannot take back the energy the machine has already delivered across the causal boundary, so it has to be assigned before the machine moves.

Brain
1–50 Hz
perception, memory, intent, planning

above: propose; below: command

Nervous System
1 kHz (one implementation)
permission path, lease, fallback

Body
10–25 kHz current loop
drives, transmission, windings

Boundary
event-timed
register write, sensor capture

Physical World
continuous
contact, friction, payload, people

Learning Objectives

- Explain why reconstructing what a machine did does not establish who answers for it, and trace an admitted action back to the decisions that allowed it
- Evaluate whether a human role gives its holder the time, information, and authority to act, using the takeover budget and supervisor utilization
- Specify the stages of operating permission, what each stage allows and must report, and what withdraws it
- Calculate how a gap in per-frame detection between groups of people becomes a gap in stopping distance and impact energy
- Size a per-group evaluation whose confidence interval can resolve a detection gap of a stated size
- Distinguish risk borne by people who chose the machine from risk borne by people who did not, and specify exposure reporting for each group
- Design an approval record that names an owner for every decision behind an admitted action, so that a missing owner blocks release

17.1 Who Answered for the Cruise Collision

Release left the running machine with a signed manifest: a verdict, the records it binds by digest, and the standing conditions that withdraw it. The manifest names the lead systems adjudicator who signed it. It does not name who decided the aisle speed the warehouse would accept, who decided that one remote assistant would serve every machine on the floor, or who would report an incident, to whom, and how fast, if the machine hurt someone. Those decisions shape what the machine does once a command crosses the causal boundary as surely as the enforcer's thresholds do, and none of them appears in a per-tick log.

The Cruise collision in San Francisco on the night of October 2, 2023 shows what happens when they are left unassigned. A human-driven car struck a pedestrian in a crosswalk and threw her into the path of a driverless Cruise robotaxi, which braked hard, ran over her, and stopped. With her still beneath it, the vehicle then began a pullover toward the curb and moved about 20 ft (6.1 m) at up to 7.7 mph before it stopped (Quinn Emanuel Urquhart and Sullivan, LLP 2024). The engineering failure belongs to the permission path. A post-collision classifier placed the pedestrian at the side of the vehicle rather than in front of it, and the fallback it selected moved the vehicle without any evidence that the space beneath it was clear.

↩ The fallback rule that forbids renewed motion while the space beneath a vehicle is unverified is established in section 12.9.1.

The machine's own records were enough to reconstruct that motion. The engineers who analyzed the event for the company's boards rebuilt it from the vehicle's logs and video, including the dropped pedestrian track before impact and the pullover after it (Quinn Emanuel Urquhart and Sullivan, LLP 2024). What failed after the collision was the account given to the people who had licensed the vehicle. On October 3, a Cruise representative told an analyst at the California Public Utilities Commission (CPUC) by telephone that the vehicle had stopped on impact and contacted remote assistance; the description left out the pullover and the dragging. The commission later counted 15 days, October 3 through 18, in which Cruise withheld a full account, and it received the full video on October 19 (California Public Utilities Commission 2024). Cruise's crash reports to the National Highway Traffic Safety Administration (NHTSA) also omitted the dragging (U.S. Attorney's Office, Northern District of California 2024). On October 24, the California Department of Motor Vehicles (DMV) suspended Cruise's permits for driverless testing and driverless deployment, citing among its grounds that the company had misrepresented information related to the safety of its autonomous technology (California Department of Motor Vehicles 2023). A review commissioned by Cruise's boards found no evidence that anyone intended to mislead regulators, but it described a "passive, non-transparent approach" to disclosure and a leadership focused on correcting media coverage of the crash (Quinn Emanuel Urquhart and Sullivan, LLP 2024).

Each group involved held one piece of the outcome: the engineers who designed the classifier and the fallback, the remote-assistance staff, the people who drafted the reports, and the executives who managed the response. Each could describe its own piece truthfully, and none of those descriptions

said who had decided the whole. The CPUC had granted Cruise commercial driverless service in San Francisco at any hour seven weeks earlier (California Public Utilities Commission 2023). The logs answered what the vehicle did. They did not answer who had decided that a driverless vehicle should pull over after a side impact without checking beneath itself, who decided what regulators would be told, or who answered for either decision. Five years earlier, after the Uber test vehicle killed a pedestrian in Tempe, the person who answered in court was the vehicle's backup operator, who pleaded guilty to endangerment in 2023 (Maricopa County Attorney's Office 2023).

Responsibility for a physical machine therefore has to be engineered like any other premise of its authority: written down before the machine moves, attached to the decisions that set its limits, and checked when it is released. The same questions arise on the warehouse machine without a public road in sight. A remote assistant answers for takeovers, facility management for the crossing rule that keeps people out of the aisle, a data team for the camera traces the black box keeps, and someone, so far unnamed, for the decision that one person can watch every machine on the floor. The sections that follow take these decisions in turn: the links behind one admitted action, the human roles the design relies on, the stages of permission to operate, the detection of different groups of people, the people who bear the machine's risk and appear in its records, and the labor it moves rather than removes. The approval record collects these decisions in one place that the release gate reads, so that a decision with no named owner blocks operation the way an unknown premise does.

17.2 Decisions Behind an Admitted Action

Consider one command the enforcer admitted in the aisle: a base velocity of 1.3 m/s toward a rack end. The enforcement record shows that it passed the stopping check at that tick, under the thresholds in force, yet behind that single log line stand at least six decisions made by different people at different times. Four of them leave traces in the engineering records built in earlier chapters. The learning team trained and approved the policy weights that proposed the command. The safety engineering team built and signed the enforcer and its thresholds. The intervention design and the remote assistant on shift set the authority state at that tick. The lead systems adjudicator signed the operating envelope into the release manifest, including the aisle speed and the floor-friction premise beneath it. The other two decisions govern the site rather than the machine: facility management wrote the crossing rule that lets people enter the aisle only at marked points, while operations decided how many machines one remote assistant would serve. Figure 17.1 draws the six links and the record, if any, that holds each one.

↩ The lead systems adjudicator, whose signature binds the release verdict to a named person, is established in section 16.4.

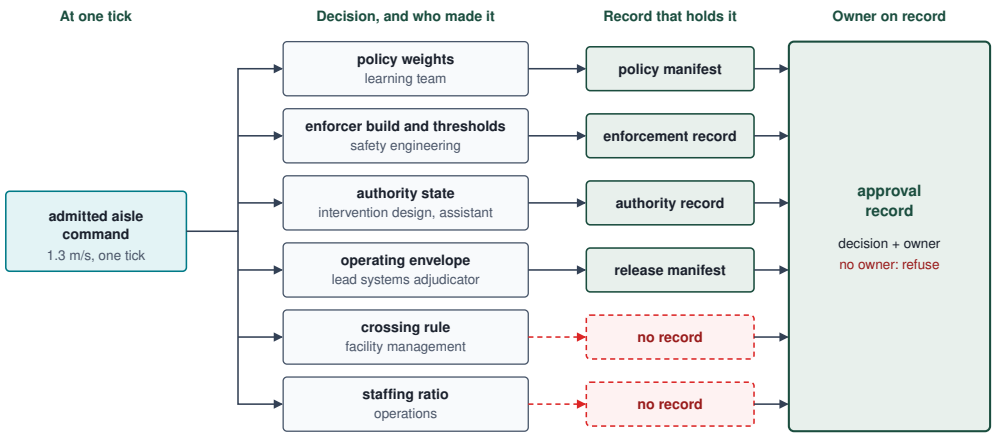


Figure 17.1: Six decisions behind one admitted command: An aisle command admitted at one tick traces back to decisions made weeks earlier by six different parties. Four of them land in a record the book has built (the policy manifest, the enforcement record, the authority record, and the release manifest); the crossing rule and the staffing ratio land in none, so no log can name who made them. The approval record lists each decision with its owner and refuses operation while any link has no owner.

When each of these decisions belongs to a different party, every party can truthfully say that the harm came from someone else's part. Thompson called this the **problem of many hands** in his study of public officials, where responsibility for a policy that many people shaped ends up resting on no one (Thompson 1980). Nissenbaum showed that computer systems make it worse: they are built by many people, they fail in ways no single person caused, and their failures are easily excused as bugs that no one could have prevented (Nissenbaum 1996). A learned proposer adds a further excuse. Its behavior was learned rather than written, so no engineer wrote the rule it followed, and Matthias argued that machines which learn create a real gap in who can be held responsible for what they do (Matthias 2004). The architecture of this book closes that gap at the point where it matters. The model proposes and never admits (principle 3), so every admitted action passed a check that someone designed, under an envelope that someone signed, and "the model did it" is never a complete answer.

17.2.1 Decisions that leave no trace in a log

When an incident investigator opens a machine's runtime logs, the decisions that made the incident possible are missing because those logs capture only runtime events: an observation and its age, a proposal, an admission, a change of authority, or a fault trial. The choices that shaped those events happened weeks or months earlier in meetings and documents, long before a command crossed the causal boundary. A safety engineer approves an operating envelope, an operations manager sets a staffing ratio, a team disables a protective function, a company applies for a permit, and a data team declares what its camera traces may be used for. In the Cruise case, the rule that a side impact should trigger a pullover was a design decision made long before October 2, and the choices about what to tell regulators were made in the days after. Neither could appear in the vehicle's log, because the vehicle did not make them.

Such decisions have to be recorded on purpose, and a chain of them is less complete than any one of its links. Suppose each of the 6 links behind the aisle command can be retrieved when an investigator asks with probability 0.99, independently of the others. The whole chain then resolves with probability $0.99^6 = 0.94$. At 0.95 per link it resolves with probability 0.74, and at 0.90, roughly what a decision held only in memory and email achieves once its author has changed jobs, with probability 0.53. An investigator would then find the full chain in about 53 percent of incidents. The same arithmetic makes a conjunctive release argument weaker than any one of its premises, and the remedy is also the same: each link has to be written down as part of the design, not left to accumulate.

↩ The conjunctive arithmetic that makes an argument no stronger than its weakest premise is established in section 16.3.2.

These links are held in the decision block of the approval record (section 17.9). For each decision it names the type of decision, the role that owns it, the signed approval, the evidence it relied on, and the date after which it must be made again. A link with no owner is not a gap in paperwork; it is a premise of the machine's authority that no one has accepted. Even when every link has a named owner, however, many of those approvals rest on a further premise: that a human operator in the loop can step in when the machine's own case runs out.

17.3 Oversight a Person Can Perform

Naming a human operator in that decision chain, whether a safety driver behind the wheel, a remote assistant at a console, or a spotter walking beside a robot during commissioning, often shifts blame onto that person without adding any real protection. A person in the loop provides genuine oversight only if three conditions hold: the person must have the time to act before the hazard arrives, the information to understand what the machine is doing, and the authority to change what it does, not only to report it. Santoni de Sio and van den Hoven state the same requirement in two parts: the system must respond to the reasons of the people meant to control it, and every outcome must trace back to someone who understood both the system and their own role in it (Santoni de Sio and Hoven 2018).

Definition 17.1: Human oversight role

A ***human oversight role*** is a duty that a design assigns to a person, such as stopping the machine, taking over a channel, or approving a motion, and that the safety argument relies on. The role counts as a countermeasure only if its holder has the time, the information, and the authority to carry it out within the time the hazard leaves.

1. **Significance**: A role that fails any of the three conditions adds no protection, yet it moves blame onto the person who holds it, so every role the safety case cites must be tested like any other premise.
2. **Distinction**: The takeover budget measures how long a person needs; the oversight role asks whether the design leaves that much time, gives that person what they need to see, and lets them act on it.
3. **Common pitfall**: A frequent misconception is that adding a person to the loop makes a design responsible. A person who cannot act in time is a place to put blame, not a safety layer.

When a role fails any of these conditions, the person in it absorbs responsibility for control they never had. Elish calls the person in that position a **moral crumple zone**. Like the crumple zone of a car, the person absorbs the impact, except that here the impact is blame, and what the crumpling protects is the reputation of the system rather than the person (Elish 2019). The three conditions give a design team a test it can apply before the machine moves.

17.3.1 Time to act

Time is the condition most often violated, and it is the easiest to check. At the 1.3 m/s aisle speed, a person who steps out 1.10 m ahead of the machine leaves 0.85 s before the rack end. A remote assistant who has been monitoring rather than driving needs about 4.5 s to take over, the design out-of-loop takeover time, during which the machine travels 5.8 m. The person needs about 5× the time the hazard leaves. No training, attention, or diligence closes a gap of that size, so a design that names the remote assistant as the countermeasure for a person stepping out at a rack end has not added a layer of protection. It has recorded in advance who will be blamed.

The engineering consequence is already in the book's design. The machine stops itself first and asks for help from standstill, where waiting costs time but not clearance. What the time test adds is the duty to write that choice down. For every human role the safety case relies on, the approval record lists the response time the role requires, the time the hazard leaves, and a verdict of `FEASIBLE`, `INFEASIBLE`, or `UNKNOWN`. A role marked `INFEASIBLE` may stay in the design as a productivity resource; it may not appear in the safety argument as a countermeasure.

↩ The out-of-loop takeover time and its price in the stopping budget are established in section 14.2.2.

17.3.2 One person, many machines

A remote assistant who serves many machines is a server in a queue, and queueing theory says how busy that person is. If help requests arrive from the whole fleet at rate λ_{arr} and each holds the assistant for a mean time $\bar{T}$, the share of time the assistant is busy, the utilization, is

$$\rho_{\text{serv}} = \lambda_{\text{arr}} \bar{T}. \tag{17.1}$$

A request that arrives while the assistant is busy waits. Under the simplest queueing model, with random arrivals and random service times, the mean wait before service begins is

$$W_q = \frac{\rho_{\text{serv}} \bar{T}}{1 - \rho_{\text{serv}}}. \tag{17.2}$$

The wait grows without bound as ρ_{serv} approaches one. Moreover, a request arriving while the assistant is busy, which happens with probability ρ_{serv} in this model, gets no human attention at all until the request ahead of it is finished.

Napkin Math 17.1: One assistant, twenty machines

Problem: A site plans to run 20 mobile manipulators with one remote assistant. Each machine asks for help 0.5 times per operating hour, and each request holds the assistant for 3 min on average. How busy is the assistant, and how long does a request wait?

Variables:

- Requests per machine: 0.5 per operating hour (Illustrative); mean service time $\bar{T} = 180$ s (Illustrative).
- Machines served: 10, 20, or 32 (Illustrative).

Math: By equation 17.1, 20 machines give $\rho_{\text{serv}} = 0.50$, and equation 17.2 gives a mean wait $W_q = 180$ s. One request in a hundred waits longer than 23 min. With 10 machines the figures are 0.25 and 60 s; with 32 they are 0.80 and 720 s, about 12 min.

Result: The assistant is busy 50 percent of the time, and a typical request waits minutes. Adding machines past that point makes the wait grow much faster than the fleet.

Systems insight: Every one of these waits is longer than the 0.85 s a person stepping out at a rack end leaves. The ratio of machines to assistants is therefore a productivity budget and never a safety layer. A staffing plan that caps utilization at 0.5 limits this assistant to 20 machines, and the approval record lists that cap as the decision of whoever set it.

↩ Admitting a remote-assistant session only when the machine is in a state where waiting costs no clearance is established in section 14.4.6.

The queue shows why the staffing ratio is a decision someone must own. A manager who adds machines to one assistant's load is not changing a number in a spreadsheet; that manager is deciding how long a stranded machine, and the people working around it, will wait for a person. If that decision is written down with the utilization it implies and an owner, it can be reviewed when the fleet grows. If it is not, it changes silently every time a machine is added.

17.3.3 Information and authority

Time is necessary but not enough. A person with time to act must also know what the machine is doing and be able to change it. A console that shows the camera feed but not which channel the machine still owns leaves the operator guessing about the most important fact of a handover, and an operator who believes they hold a channel they do not hold can make a takeover worse. A role whose only output is a report, a flag raised to a supervisor or a note in a log, gives its holder no authority over the motion at all. The test for authority is mechanical. The person must be able to stop the machine, through a channel the permission path honors within the time the hazard leaves, without asking anyone else.

↩ Mode confusion, and the per-channel annunciation that keeps an operator's picture of who owns each channel current, are established in section 14.6.

For each human role, the approval record therefore holds three fields beside the timing verdict: the information channel the person reads, the authority channel the person can act through, and whether that channel reaches the permission path. A remote assistant on the running machine passes the information and authority tests for a stranded machine at standstill and fails the time test for a person stepping out at a rack end. Both facts belong in the record, because they assign that one person two different duties.

Checkpoint 17.1: Oversight a person can perform

A design names three people in its safety argument: the remote assistant, a teach-pendant operator during commissioning, and a floor supervisor who watches the fleet dashboard.

- ☐ **Time**: Can you state, for each role, the time the role requires and the time the relevant hazard leaves, and mark the role `FEASIBLE`, `INFEASIBLE`, or `UNKNOWN`?
- ☐ **Load**: A site runs 24 machines with one assistant; requests arrive at 0.8 per machine-hour and take 75 s. Can you compute the assistant's utilization and mean wait, and say whether the assistant may be listed as a countermeasure for any hazard faster than a minute?

☐ **Authority**: Can you say which of the three roles can stop the machine through a channel the permission path honors, and which can only report?

17.4 Permission in Stages

Granting a machine full operating permission on day one exposes the public or the workforce before the configuration has accumulated the field evidence needed to support that exposure. Regulators of driverless vehicles therefore grant permission in stages. California's rules separate testing with a safety driver, testing without one, and deployment to the public, each under its own permit and conditions (California Department of Motor Vehicles 2018a, 2018b). The CPUC adds a second ladder for carrying passengers. Before August 2023, Cruise could carry paying passengers without a safety driver only in limited areas of San Francisco between 10 p.m. and 6 a.m., could carry paying passengers anywhere in the city at any hour only with a safety driver present, and could give free driverless rides anywhere at any hour. On August 10, 2023, the commission removed the time limit on driverless fared service (California Public Utilities Commission 2023). Each rung named a configuration, a place, a time, and who could be aboard.

Definition 17.2: Permission stage

A ***permission stage*** is a grant, made by the party entitled to grant it, that allows a named configuration of a machine to operate in a named domain, among named groups of people, on named evidence, and under named reporting duties. A machine moves to the next stage only on evidence gathered at the current one, and a stage can be withdrawn for a configuration change, an incident, or a withheld record.

1. **Significance**: Exposure grows one stage at a time, so the evidence for the next stage is gathered under the protection of the current one, and a failure at one stage costs only what that stage exposes.
2. **Distinction**: A release verdict is the engineering decision about whether a configuration's claims are supported; a permission stage is the grant that decides who may be exposed to that configuration and what must be reported about it.
3. **Common pitfall**: A frequent misconception is that a stage, once reached, belongs to the company. It belongs to a configuration, and it carries duties that, if broken, withdraw it.

Each stage carries reporting duties, and the duties grow with the stage. California requires a manufacturer testing on public roads to report, within ten days and on a set form, any collision that causes damage, injury, or death, and to file an annual report of every disengagement, each transfer of control from the automation to the safety driver (California Department of Motor Vehicles 2018a). Since 2021, a federal order has required companies operating automated driving systems to report crashes to NHTSA. Under the version in force in 2023, a crash that involved a death, an injury treated at a hospital, or a pedestrian or other vulnerable road user had to be reported within one day, with an update at ten days (National Highway Traffic Safety Administration 2021). The disengagement reports are the data behind figure 14.1, whose two series come from two different stages, testing with a driver and testing without one.

A warehouse has no public permit office, but the same ladder exists. The machine is commissioned with a spotter walking beside it, then allowed to run unattended in a zone people cannot enter while it operates, and only then released onto an open floor among workers. The facility owner, the integrator, and the workers' representatives grant these stages, and the approval record names who granted each one, for which configuration, on what evidence, and with which reporting duties. The stage and the verdict are recorded separately. A `COND` verdict for restricted inspected-route delivery says which claims the evidence supports; the stage says whether that restricted service may run on the open floor yet.

17.4.1 Leaving a stage on evidence

A stage is left on evidence, not on a calendar date, and the evidence a lower stage produces is partly hidden by the protection that stage provides. A safety driver or a spotter masks failures: when the person intervenes, the machine's own error never reaches its consequence, and the stage records an intervention rather than a collision. Counting failures at the supervised stage therefore means reconstructing, for each intervention, whether the machine would have failed without it, which is why the disengagement reports matter and why they must say why control was transferred.

↩ Interventions that end a trial before its outcome is known, and how they are counted, are established in section 7.6.

The exposure arithmetic sets how long a stage must run. Under the zero-failure Poisson exposure bound of section 7.9, ruling out an intervention rate at 95 percent confidence with zero observed events requires an exposure of $\ln(1/0.05)$, or nearly three times the target mean interval between interventions, so claiming fewer than one safety-relevant intervention per 1,000 operating hours after observing none requires the machines to accumulate about 2,996 failure-free operating hours. A fleet of 10 machines running 16 hours a day reaches that in 19 days. Those hours support the claim only for the configuration that ran them. A new policy, a new enforcer build, or a new site layout starts its own count, because the evidence belongs to a configuration, and so does the stage it supports.

17.4.2 Withdrawal and disclosure

Permission is withdrawn the way it is granted, one stage at a time. The DMV's October 2023 order suspended Cruise's driverless testing and deployment permits but left in place its permit to test with a safety driver (California Department of Motor Vehicles 2023), stepping the company back down the ladder rather than off it. The grounds the DMV cited show what a permit protects. One was that the vehicles were not safe for public operation, a finding about the motion. Another was that the company had misrepresented information related to the safety of its technology, a finding about the record.

The regulatory fines that followed were for the record. The CPUC counted the 15 days in which it lacked a full account, applied its daily maximum of $7,500 to each, and fined Cruise $112,500 (California Public Utilities Commission 2024). In September 2024, NHTSA imposed a consent order with a $1.5 million penalty for crash reports that omitted the dragging (National Highway Traffic Safety Administration 2024b). In November 2024, Cruise entered a deferred prosecution agreement and paid a $500,000 criminal fine for submitting a false record to influence the federal investigation (U.S. Attorney's Office, Northern District of California 2024). Regulators withdrew authority from the company because of what its vehicle did and because of what the company did not tell them about it.

The **incident disclosure duty** is the obligation, attached to an operating permit, to report an incident completely and on time to the parties the permit names. It is a decision with an owner like any other: someone decides what an incident report contains, someone signs it, and someone decides when a record has been withheld long enough to break the permit. The permit block of the approval record therefore lists, for each stage, the regulator or grantor, the configuration the stage covers, the reporting duties with their deadlines, and a disclosure log of every report sent, with its author and the evidence attached. A release manifest that binds raw traces to the configuration that produced them, and that an assessor can open, is the engineering half of that duty. The disclosure log is the other half, because it shows what was actually sent, to whom, and when. Once a machine enters an open-floor stage among coworkers or pedestrians, however, its safety case rests on a sensor premise that aggregate stage-exit hours can hide: whether its detectors see every group of people in its path equally well.

↩ The release manifest that binds raw traces to the configuration that produced them is established in section 16.8.

17.5 When Detection Depends on Who Is Detected

A machine that earns open-floor permission on aggregate obstacle-detection metrics can still fail systematically on specific groups of people sharing the aisle or roadway. Buolamwini and Gebru found that commercial face-analysis systems misclassified darker-skinned women far more often than lighter-skinned men, and that the benchmarks used to evaluate them contained few of the people they failed on (Buolamwini and Gebru 2018). Pedestrian detection shows the same pattern. Wilson, Hoffman, and Morgenstern annotated pedestrians in a driving dataset by skin type and found that standard detectors reached an average precision of 67.1 percent on lighter-skinned

pedestrians and 55.9 percent on darker-skinned ones under a strict overlap criterion. Time of day and occlusion did not explain the gap (Wilson et al. 2019). Testing widely used pedestrian detectors, Li and colleagues found the proportion of children left undetected 20.14 percent higher than that of adults, and significant gender bias at night (X. Li et al. 2025).

A warehouse has its own groups. A detector trained mostly on people walking upright may see a kneeling worker, a person carrying a carton that hides the torso, a wheelchair user, or a worker without a high-visibility vest later or less often. Where the machine relies only on its certified safety scanner, which stops for any object above its resolution without deciding what it is, these differences do not matter. They matter wherever the design relies on a learned detector to recognize a person: to slow down when a person is near, to keep a wider separation from people than from pallets, or to predict where a person will walk. In those places, a difference in detection becomes a difference in how close the machine comes to each group before it begins to brake.

17.5.1 From a detection gap to a stopping gap

When a detector sees one group of people less reliably than another, the machine travels farther before braking begins and can strike a person in the less-detected group even when the stop was sized to clear the best-detected group. The conversion from a detection rate to a stopping shortfall runs through the camera's frames. Suppose a design requires that the chance of missing a person in every frame of a window be no more than p_{miss}, and that a detector reports a person in view in each frame with probability p_{frame}, independently from frame to frame. Because independent misses multiply across frames so the probability of missing the person in all N_{frames} frames is $(1-p_{\text{frame}})^{N_{\text{frames}}} \le p_{\text{miss}}$, the number of consecutive frames the person must be in view before the design may count on detection is

$$N_{\text{frames}} = \left\lceil \frac{\ln p_{\text{miss}}}{\ln (1-p_{\text{frame}})} \right\rceil . \tag{17.3}$$

A group the detector sees less often needs more frames, and each extra frame is travel before the machine can count on knowing the person is there. If the stop was sized for the best-detected group, a group that needs ΔN_{frames} more frames, at camera frame period T_{sample} and speed v, loses a distance

$$\Delta d_{\text{detect}} = v\, T_{\text{sample}}\, \Delta N_{\text{frames}} \tag{17.4}$$

before braking begins. A stop that would have ended just short of a person in the reference group now ends at the person, and the machine is still moving when it gets there. Because the reference braking distance $v^2/(2\,a_{\text{brake}})$ was sized to shed all of the initial squared speed v^2 just as the machine reaches the person, losing a distance Δd_{detect} before braking begins leaves that final Δd_{detect} of braking uncompleted. While the shortfall is smaller than the braking distance, braking at a_{brake} over the remaining distance therefore leaves an impact speed and an impact energy of

$$v_{\text{impact}} = \sqrt{2\, a_{\text{brake}}\, \Delta d_{\text{detect}}}, \qquad E_k = \tfrac{1}{2} m v_{\text{impact}}^2 = m\, a_{\text{brake}}\, \Delta d_{\text{detect}}. \tag{17.5}$$

The last form says that the energy delivered to the person grows in direct proportion to the distance lost to late detection. Fairness for a moving machine is therefore measured in meters and joules, not only in percentage points of recall.

> Napkin Math 17.2: Two frames late
>
> **Problem**: The running machine slows near people only after its learned detector reports a person. Its stop is sized for a group the detector sees in 95 percent of frames. A second group, kneeling workers, is seen in 75 percent of frames. At the aisle speed, what impact speed and energy does the second group receive from a stop that would have just cleared the first?
>
> **Variables**:
>
> - $p_{\text{miss}} = 10^{-3}$ (Chosen); $p_{\text{frame}} = 0.95$ and 0.75 (Illustrative).
> - $T_{\text{sample}} = 16.7$ ms (Datasheet, navigation camera); $v = 1.3$ m/s (Chosen); $a_{\text{brake}} = 2$ m/s² (Illustrative); $m = 368$ kg (Derived, loaded).

Math: By equation 17.3, the first group needs 3 frames and the second 5, so $\Delta N_{\text{frames}} = 2$, or 33.3 ms. By equation 17.4, $\Delta d_{\text{detect}} = 43$ mm, well inside the 0.42 m braking distance. By equation 17.5, $v_{\text{impact}} = 0.42$ m/s and $E_k = 32$ J.

Result: A stop designed to end short of a standing worker ends in contact with a kneeling one at 0.42 m/s.

Systems insight: A few centimeters of late detection become a collision because the stop was sized with no margin for the slower group. The result also assumes the detector's misses are independent from frame to frame; a detector that systematically fails on a pose misses it in many frames in a row, and the shortfall grows well beyond this best case.

Table 17.1 repeats the calculation for three lower-detection groups. The relationship is steep at the low end: a group seen in 60 percent of frames needs 5 extra frames and receives a 0.66 m/s impact carrying 80 J. Figure 17.2 plots the speed of the machine against distance for the reference group and for that group.

Table 17.1: A detection gap measured in meters and joules: Each row is a group of people the detector reports with the given probability per frame. The stop is sized for the reference group at the running machine's aisle speed, loaded mass, and credible deceleration, with an allowed all-frames miss probability of one in a thousand and independent frames. Groups seen less often need more frames, lose the distance shown before braking can begin, and are struck at the speed and energy shown.

Per-frame detection	Frames needed	Extra frames	Distance lost	Impact speed	Impact energy
0.95 (reference)	3	0	0 mm	0 m/s	0 J
0.85	4	1	22 mm	0.29 m/s	16 J
0.75	5	2	43 mm	0.42 m/s	32 J
0.60	8	5	108 mm	0.66 m/s	80 J

DRAFT

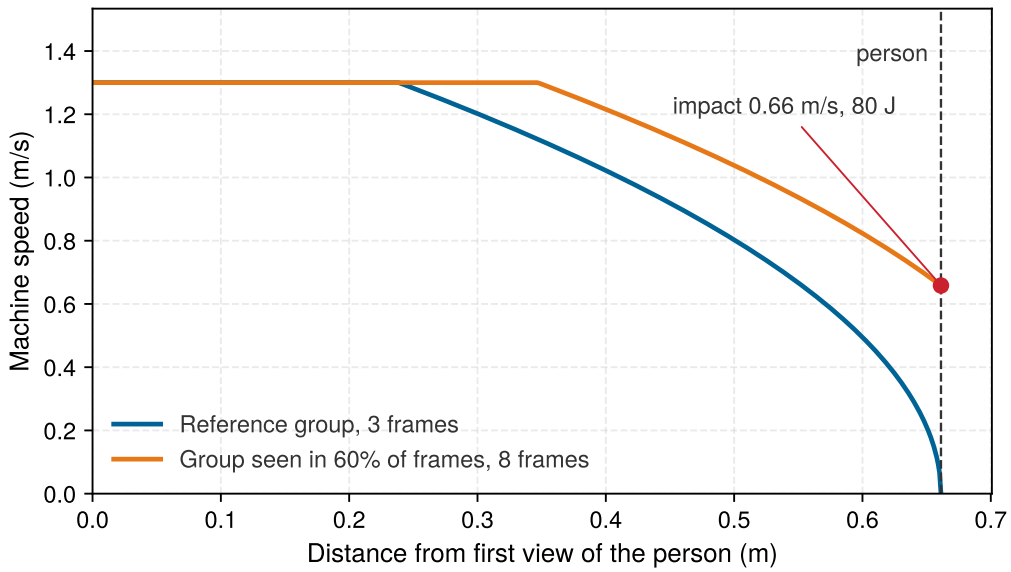


Figure 17.2: One stop, two groups of people: Speed of the running machine against distance traveled from the moment a person enters the detector's view, for a stop sized so that it ends just short of a person in the best-detected group. That group is detected after 3 frames; a group seen in 60 percent of frames is detected after 8, so braking begins later and the same deceleration leaves the machine still moving when it reaches the person. The impact speed and energy follow from the distance lost, not from the recall gap alone.

17.5.2 Sample sizes for each group

A detection gap can be managed only if it can be measured, and measuring it per group takes more data than measuring it overall. Suppose an evaluation set holds 10,000 people, of whom 2 percent belong to a smaller group, 200 people, and the detector finds 0.95 of both groups. The exact 95 percent interval for the larger group runs from 0.945 to 0.954. For the smaller group it runs from 0.910 to 0.976, about 8× wider, too wide to tell a detection rate of 0.95 from one of 0.92. A pooled detection rate reported for "pedestrians" or "workers" hides exactly the group the machine serves worst.

Telling a detection rate of 0.95 from one of 0.92 with 80 percent power takes about 1,059 people in each group, about 11 percent of the whole evaluation set for every group the evaluation must resolve. A group that makes up a few percent of the people a site employs will not reach that count by chance; the evaluation has to recruit it deliberately. The same arithmetic sets how long field monitoring must run before a per-group detection rate can be trusted, and it is why the sample size for each group belongs in the plan before data collection begins, not in the report after it.

↩ The exact binomial bound, and why a few observed failures drop it sharply, are established in section 7.6.1.

17.5.3 Where the measurement lives

If per-group detection rates live only in a research benchmark report, the release gate cannot check them before admitting the machine to an open floor. Tracking them does not need a new pipeline; it needs a new dimension in records the book already keeps. The observation contract states its detection qualification for each group the machine must slow down for. The fault list adds a coverage dimension that seeds kneeling, carrying, and seated people as deliberately as it seeds stale frames. The evaluation record reports detection per group with its confidence interval rather than only in aggregate. From those qualified measurements, the exposure block of the approval record stores, for each group the machine relies on detecting, the sample size, the rate, its interval, and the distance shortfall it implies at the operating speed. A group with no entry is an unknown premise, and the release gate treats it as one.

↩ The observation contract, which states what each measurement may be relied on for, is established in section 8.8.

Checkpoint 17.2: Detection gaps and sample sizes

- ☐ **Conversion**: A detector reaches a person kneeling between racks two frames later than a standing person. Can you compute, at the running machine's aisle speed, loaded mass, and credible deceleration, the impact speed and energy if the stop was sized for standing workers?
- ☐ **Remedy**: Can you compare three remedies (a lower speed near racks, a sensor that does not depend on the person's pose, and a zone rule that keeps the machine out while people kneel there) by what each costs and what evidence each needs?
- ☐ **Sample**: Can you size an evaluation that distinguishes 0.95 from 0.92 detection at 80 percent power, and say what share of a 10,000-person test set each group must be?

17.6 Who Bears the Risk

The people a machine puts at risk are not only the people who chose it. The facility operator chose to buy the machine and the integrator chose how to configure it; a rider chooses to take a robotaxi. The pedestrian in the crosswalk, the coworker at an aisle crossing, and the visitor walking through a loading area made no such choice. An aggregate comparison, such as a fleet's injury rate against a human benchmark, can improve while the remaining risk concentrates on people who never agreed to carry it. The risk such people carry without having chosen it is **unconsented risk**, and it has to be counted separately because no aggregate figure reveals it.

Choices that look purely operational move that risk around. The site crossing rule, which lets people enter the aisle only at marked points, buys the machine its speed by asking pedestrians to wait and to comply. Suppose 80 percent of the encounters between machines and people happen at three crossings used by 10 percent of the workforce. Those workers then carry 8× the site's average exposure per person, while everyone else carries 22 percent of it, and the site-wide encounter rate looks the same either way. Routing, the boundaries of the operating domain, and the hours a machine runs all shift exposure among groups in the same way.

↩ Fleet crash rates reported outcome by outcome, each with its own interval, are established in section 16.9.6.

↩ The site crossing rule, and the access restriction it places on pedestrians, are established in section 16.4.1.

The requirement that follows is a reporting one. The exposure block of the approval record lists each group exposed to the machine, whether its members chose that exposure, how their exposure is measured (encounters per person-hour, near misses, close passes), and the routing and domain decisions that changed it. A group that bears risk it did not choose is owed two things a fleet-wide statistic cannot give it: a measurement of its own exposure, and a way to see the machine coming.

17.6.1 Legible intent

People protect themselves from a moving machine by predicting where it will go and getting out of the way. Yet a machine that follows an efficient path until the last moment leaves nearby coworkers unable to tell whether it will continue straight or turn across their path. Dragan, Lee, and Srinivasa distinguish two properties of robot motion: motion is predictable when it matches what an observer who knows the goal expects, and **legible** when it lets an observer who does not know the goal infer it quickly (Dragan et al. 2013). A mobile base that drifts toward the side of the aisle it will turn into, or an arm that moves toward the shelf it will reach before it reaches, is legible. A machine that moves along the efficient path until the last moment is predictable only to someone who already knows its plan.

Explicit signals add to legible motion. A light that projects the planned path on the floor, a sound before a reversing move, or a display that shows which way the machine will turn at a crossing lets a coworker begin to yield earlier. A person who yields earlier approaches the machine more slowly, which shrinks the approach-speed term of the separation the enforcer must keep. Signals help only if people's trust follows what the machine can actually do. Lee and See showed that people rely on automation in proportion to their trust in it. Trust miscalibrated in either direction causes harm: too much leads to people stepping in front of a machine that will not stop for them, too little to people ignoring a machine that would (Lee and See 2004). A signal that the machine slows for people should therefore be shown only where the machine's detection of people has been measured, group by group, to support it. Physical motion in the aisle is not the only burden the machine places on the people around it; the cameras and logs that verify that motion also record everyone who shares the floor.

↩ The human separation distance, whose approach-speed term a person who yields earlier reduces, is established in section 12.3.

17.7 Data About People

When the warehouse machine logs a near miss at a rack end, its sensors and audit trails capture the people in and behind the cell alongside the telemetry of the base and arm. The records that make a machine answerable are also records about people, and the same evidence that settles who answers for an incident can, if kept forever and read by anyone, become a tool for watching the people it was meant to protect. How those records are declared, kept, and read is therefore part of the design, not an afterthought to it.

17.7.1 Recorded people as a standing condition

Three of the release records carry personal data directly: the authority record of section 14.8 names the operator behind each takeover tick by tick, the event-triggered traces of the black box hold camera frames of whoever stood in the aisle (including coworkers who never touched the console), and the demonstration archive of section 5.11 binds each episode to the operator who produced it. If those traces are retained indefinitely or opened to unrestricted browsing, the safety evidence store turns into continuous workplace surveillance. A release must retain the evidence needed for incident reconstruction while keeping identifying data to a minimum and within justified retention limits. The release therefore carries one more premise, and it is administrative rather than physical: each record that identifies a person declares its purpose, its retention period, and who may read it, and the site operates within those declarations.

Where the machine runs in the European Union, the General Data Protection Regulation makes that premise law (European Parliament and Council of the European Union 2016). Article 5(1) requires personal data to be collected for specified, explicit, and legitimate purposes and not further processed in a manner incompatible with them (point b), to be limited to what those purposes need (point c), and to be kept in identifiable form no longer than they require (point e). Article 25(2) makes the minimum the default, in the amount collected, the extent of processing, the storage period, and accessibility. Article 35 requires a data protection impact assessment before processing that is

likely to result in a high risk to the people concerned. Article 88 lets member states and collective agreements set more specific rules for employee data, naming monitoring systems at the workplace among the safeguards those rules must address. Other jurisdictions differ in detail; the engineering premise does not.

Each declaration becomes a standing condition of the kind section 16.5 defines, with a detector and a response. Incident reconstruction sets the retention of the black-box and authority logs: the traces around an intervention are kept for a declared, justified review period, with any extension documented under the applicable retention rules, and routine ticks with no event roll off the ring buffer as they already do. Access follows the purpose, so an incident reviewer may read a trace and an operator's supervisor may not score from it. A trace promoted into the experience pipeline carries its purpose and retention with it, which is what the consent and retention field of table 5.6 records. The detector is an access and retention audit on the evidence store. Its response follows from the rule that a verdict stands only while its premises are known-true, so a record used outside its declared purpose, or held past its period, suspends that use and returns the premise to the adjudicator. Stated this way, the premise keeps the traces an investigator needs to reconstruct a stop and bars their use for watching workers.

17.7.2 Keeping less

Continuous video recording keeps every frame of every worker's shift, creating a permanent surveillance archive when an incident investigation needs only the seconds around a fault. The first way to protect recorded people is therefore to record less of them. Event-triggered retention keeps only the window around an intervention or a boundary excursion: the 5 s before the trigger that the authority record needs to reconstruct a takeover, and the response after it, 30 s in all. At 20 such events per machine over a 16-hour operating day, the machine keeps about 1 percent of its footage, a 96× reduction, and everything else rolls off the ring buffer within seconds.

The event window and the records retained from it have different lifetimes. Figure 17.3 follows the bounded capture history into incident review and separates that use from a possible training use.

↩ The pre-trigger window the black box must hold to reconstruct a takeover is established in section 14.7.

Retention limits apply to each copy, including a copy admitted to training. Deleting a routine capture frame does not remove a retained incident record, and approving incident access does not approve every later use of that record.

Two further steps reduce what the kept footage reveals. Faces and badge numbers can be blurred on the machine before a frame is stored, provided the investigation does not need them. The declaration then records which fields stay identifiable and why, for example that the operator's identity is kept in the authority record but not in the camera trace. People whose demonstrations train the policy are a separate case: they are recorded on purpose and at length, so their consent, what their episodes may be used for, and how long those episodes are kept travel with the data from the moment of collection. The data block of the approval record lists each record class, the people it records, its purpose, its retention, what is removed before storage, who may read it, and the result of the last access and retention audit.

17.7.3 When safety records become performance scores

The records most likely to be misused are the ones that measure workers. The machine's authority logs and black-box recorders capture the identity of every operator who initiates a manual takeover, how long each takeover took, and how often each operator intervened. If an employer uses those records to score employee performance, the records have crossed from safety into monitoring. Conformity assessment under the European Union Machinery Regulation addresses the machinery safety requirements for the declared use; it does not govern every later use of the machine's records. One such use falls under a different law, the European Union Artificial Intelligence Act. Its Annex III lists AI systems intended to monitor and evaluate the performance and behavior of people at work among the employment uses subject to the high-risk classification rules of Article 6. Article 26(7) requires an employer deploying a high-risk system in the workplace to inform workers' representatives and the affected workers before putting it into use (European Parliament and Council of the European Union 2024a).

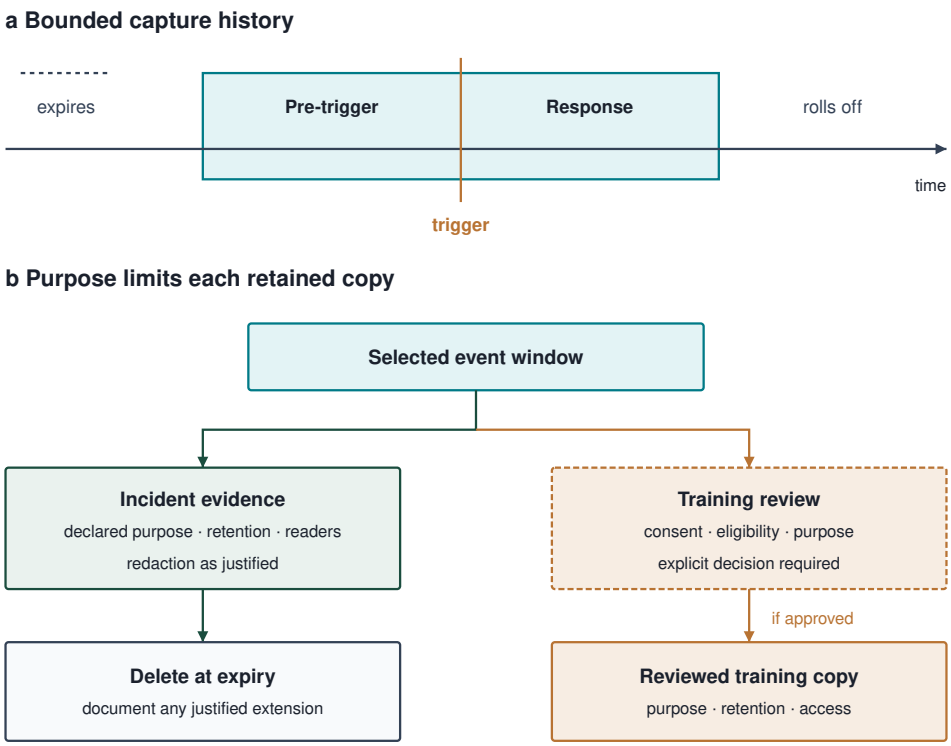


Figure 17.3: The Lifetime of Incident Evidence: A trigger selects the declared pre-trigger and response window while routine frames roll off the capture buffer. The incident record carries its purpose, retention, and readers. A training copy requires a separate review of consent, eligibility, and purpose; selection as incident evidence does not itself authorize training. The window and retention periods are those declared for the deployment, not universal durations.

The declarations of section 17.7.1 are where the line is drawn. A record declared for incident reconstruction may not be read to rank operators, and an access audit that finds such a read suspends that use and returns the premise to the adjudicator. If a site does want to evaluate workers, that is a separate system with its own declaration, its own legal classification, and its own owner, and the approval record lists it as a separate decision rather than letting it grow out of the safety records. Protecting workers in the machine's records still leaves the physical work the machine shifts onto them during daily operation.

17.8 The Labor the Machine Moves

Automating a task does not remove the labor the task needs. It moves that labor to the people who take over a machine when its planner stalls, the technicians who sweep and inspect the corridors the machine's restrictions depend on, the crews who keep the floor in the state the release assumed, and the operators who record demonstrations for the next policy. The intervention frequency logged during release testing defines the human work the deployment creates: how often a technician is dispatched, how fast that person must arrive, and where on the floor that person must be stationed.

↩ The authority record that logs each takeover, its actor, and its timing is established in section 14.8.

Napkin Math 17.3: Where the tote's labor went

Problem: The site of notebook 17.1 runs 20 machines for 16 hours a day with one remote assistant, and sweeps the restricted corridors 4 times a day. How many person-hours of work does the fleet create each day?

Variables:

- Assistant utilization: 50 percent (Derived, notebook 17.1); operating day: 16 hours (Illustrative).
- Corridor sweeps: 4 per day of 20 min each (Illustrative).

Math: The assistant must be staffed for all 16 h of operation, of which 8 h go to requests and 8 h to watching and waiting. Sweeps add 1.3 h. The fleet creates 17.3 person-hours of work a day, about 2.2 full-time shifts.

Result: The fleet needs more than two people's working days of human labor every day it runs, before counting maintenance, charging, or the collection of new demonstrations.

Systems insight: The waiting hours are labor too. A person paid to watch a console for half of every shift, ready to act within seconds, does a job that monitoring research has long shown people perform poorly (Bainbridge 1983), and the design that creates the job owns its conditions.

The burden is not only hours. Across United States local labor markets, greater exposure to industrial robots between 1990 and 2007 lowered both employment and wages (Acemoglu and Restrepo 2020). Inside warehouses, the effect on safety cuts both ways. Burtch, Greenwood, and Ravindran found warehouse robotics associated with a 40 percent decrease in severe injuries and a 77 percent increase in non-severe ones, an increase they attribute at least partly to the faster pace of work at robotic facilities (Burtch et al. 2025). Across United States commuting zones, Gihleb and colleagues found that greater exposure to industrial robots lowered work-related injury rates while raising other harms, including drug- and alcohol-related deaths (Gihleb et al. 2022). A deployment can make the most dangerous injuries rarer and the work around the machine harder at the same time. Whether mobile manipulators in logistics follow either pattern has not yet been measured at scale. The task, takeover, and technician-hour records built in the book can inform that measurement; estimating causal effects on workers also requires comparison sites and labor-market data that no single deployment holds.

The labor block of the approval record therefore lists the work a deployment moves: takeover hours, technician dispatches, sweep hours, staffed monitoring hours, and the pace targets set for workers who share the floor. It also states the uses of worker data that the deployment bars, so that the labor the machine creates is not measured with records collected for safety. A site that cannot fill these fields does not know what its machine costs the people who work beside it.

17.9 The Approval Record

Left scattered across emails, slide decks, and meeting notes, the human and organizational premises behind a machine can lapse without blocking release: a staffing ratio changes on the night shift, a subgroup detection test is deferred, or a data-retention declaration expires while the robot keeps running. The **approval record** collects these premises in a single gate-checked artifact: it binds by digest the authority record, the enforcement record, the fault records, and the release manifest, and it adds the decisions behind them, each with a named owner.

↪ The cost of an operating restriction in throughput, against the cost of the test campaign that would lift it, is developed in section 18.4.2.

Definition 17.3: Approval record

The ***approval record*** is the signed record that names, for each decision behind a machine's admitted actions, the person who approved it and answers for it, and that declares the human roles the design relies on, the permission stage and its reporting duties, the groups exposed to the machine, the records kept about people, and the labor the deployment moves.

1. **Significance**: The release gate reads the approval record as a standing condition, so a decision with no named owner refuses operation exactly as an unknown physical premise does.
2. **Distinction**: The authority record shows who held each channel at each tick; the approval record shows who decided that the machine could be in that situation at all.

3. **Common pitfall**: A frequent misconception is that a complete runtime log makes someone answerable. A log reconstructs events; only a record of decisions and their owners says who answers for them.

The approval record has seven blocks, a header that binds it to the release and one block for each kind of decision this chapter has set out (table 17.2). Each block names the decision owner alongside the specific evidence, bounds, or reporting duties that keep the release valid.

Table 17.2: The approval record: Each block holds a kind of decision the runtime records cannot hold, and each entry names an owner. The header binds the record to the release it governs by digest, so a new release requires a new approval record.

Block	What it holds	Set in
Header	Record identifier; digests of the release manifest, authority record, enforcement record, and fault records; the permission stage; the validity window	section 17.9
Decisions	For each decision behind an admitted action: its type (envelope sign-off, configuration approval, update approval, staffing, protective-function change, permit application, data use), owner, signed approval, evidence, date, and expiry	section 17.2
Human roles	For each role the safety argument cites: the duty, the response time it requires, the time the hazard leaves, the utilization bound, the information and authority channels, and a verdict of `FEASIBLE`, `INFEASIBLE`, or `UNKNOWN`	section 17.3
Permit and reporting	For each stage: the grantor, the configuration digest, the operating domain, the reporting duties and deadlines, and the disclosure log	section 17.4
Exposure	For each group exposed: whether it chose the exposure, how its exposure is measured, its per-group detection (sample size, rate, interval, distance shortfall), and the routing and domain choices that change it	section 17.5, section 17.6
Records about people	For each record class: the people recorded, the purpose, the retention, what is removed before storage, the readers, and the last audit result	section 17.7
Labor	Takeover hours, technician dispatches, sweep hours, staffed monitoring hours, pace targets, and the uses of worker data the deployment bars	section 17.8

↪ **Byte layout:** The approval record's fields, their encodings, and the check the release gate applies to each are given in section C.15.12.

The approval record gates release because the release gate reads it as a standing condition of the manifest. Four findings refuse operation the way an unknown premise does: an admitted action class has a decision link with no named owner, a human role cited as a countermeasure carries a verdict of `INFEASIBLE` or `UNKNOWN`, an exposed group the machine relies on detecting lacks a per-group entry, or a declaration about recorded people has expired or failed its audit. Each finding returns the premise to the adjudicator with its cause logged. A decision whose owner leaves the organization expires with that person's role, so the approval record is kept current the same way the standing conditions are: by checking, on a declared schedule, that every premise still holds.

↩ Standing conditions, each with a detector, a cadence, and a response that withdraws the grant, are established in section 16.5.

Not every open entry can be closed before release. A decision may wait on a regulator, a per-group sample may still be filling, a staffing plan may depend on a hire. Those entries do not vanish into a footnote. Each one leaves the approval record with an operating restriction that holds until it closes, such as a lower speed near the group not yet measured or a zone the machine may not enter. It also enters the residual-claims register with an owner and a closure test, alongside the physical premises no sensor can check.

↪ The residual-claims register, which carries each open premise with its restriction, owner, and closure test, is developed in section 18.3.3.

Checkpoint 17.3: An approval record for the Cruise configuration

- ☐ **Decisions**: Using section 17.1, can you list the decisions behind the October 2023 pullover (the post-collision response rule, the operating domain, the commercial stage, the reporting duties) and name the role that should own each?
- ☐ **Unknown fields**: Which of those fields could have been marked `UNKNOWN` before the collision, and which would have refused operation under the gate rule of this section?

☐ **Disclosure**: Can you write the disclosure-log entry that the permit block requires for the first report after the collision: who sends it, to whom, by when, and with what evidence attached?

17.10 Fallacies and Pitfalls

Responsibility for a machine fails in predictable ways, and most of them come from treating it as something settled after an incident rather than designed before one. Each misconception below assumes that a record, a person, or an average carries a responsibility that no one has actually accepted.

Fallacy: *A complete runtime log makes someone answerable for what the machine did.*

The logs of the Cruise vehicle were complete enough for engineers to rebuild the pullover in detail, and still the question of who had decided the vehicle should pull over, and who decided what regulators would hear, took months and an outside investigation to answer. A log records events. The decisions that made an event possible (the operating envelope, the response rule, the staffing ratio, the permit application) were made earlier, by people who appear in no tick. Six such links, each retrievable with probability 0.90, form a chain an investigator can follow to the end only 53 percent of the time.

Reconstruction is not attribution: only a record of decisions and their owners says who answers for an admitted action.

Pitfall: *Naming a person as the countermeasure for a hazard faster than a person can act.*

A design that lists the remote assistant as the response to a person stepping out at a rack end asks for a takeover that needs about 4.5 s when the hazard leaves 0.85 s. The person in that role adds no protection and absorbs the blame for an outcome the design controlled. Utilization makes it worse: at 20 machines per assistant, a request waits 180 s on average before anyone looks at it.

Architect's countermeasure: Test every human role the safety argument cites for time, information, and authority, mark it `FEASIBLE`, `INFEASIBLE`, or `UNKNOWN`, and let only feasible roles count as countermeasures.

Fallacy: *One detection rate for "people" describes how safe the machine is for everyone.*

A pooled rate is dominated by the largest group. At 200 people, a smaller group's exact interval is about 8× wider than the rest of the set's, too wide to resolve a gap of three points in detection. A group detected two frames later at the aisle speed loses 43 mm of clearance and is struck at 0.42 m/s.

Detection must be reported per group, with a sample large enough for its interval to resolve the gap that matters, and converted into meters and joules.

Pitfall: *Justifying a deployment for everyone exposed with a favorable fleet-wide comparison.*

An aggregate injury rate can fall while risk concentrates on the people who never chose the machine. When 80 percent of encounters happen at crossings used by 10 percent of workers, those workers carry 8× the average exposure, and the site-wide rate does not show it.

Architect's countermeasure: Report exposure per group and per location, record whether each group chose it, and treat routing and operating-domain choices as decisions about who bears the risk.

Fallacy: *Keeping every recording is the safe choice, because an investigation might need it.*

Continuous recording keeps every frame of every worker for as long as the storage lasts. Event-triggered retention keeps the windows an investigation actually uses, about 1 percent of the day, and lets the rest roll off within seconds. Records kept without a declared purpose and period drift into uses no one approved, such as ranking operators by their takeovers.

Each record about people is kept only for a declared purpose and period, and read only by the readers that purpose names.

Pitfall: *Carrying a permission stage across a configuration change.*

A stage authorizes a configuration in a domain under reporting duties. A new policy, a new enforcer build, or an expanded operating domain is a different configuration, and the failure-free hours that supported the old stage, about 2,996 for the claim in section 17.4.1, do not transfer to it. A withheld or incomplete incident report breaks the stage as surely as the motion does.

Architect's countermeasure: Bind each stage to a configuration digest in the permit block of the approval record, so an update reopens the stage along with the release claims it touches.

Fallacy: *Automation removes the work it automates.*

A fleet of 20 machines with one remote assistant creates about 17.3 person-hours of rescue, sweeping, and monitoring every operating day, and warehouse robotics has been associated with more minor injuries even as severe ones fell. The work moves to fewer, different people, often under a faster pace.

A deployment accounts for the labor it moves and bars the use of its safety records to measure the people who do it.

17.11 Summary

A machine's records can reconstruct every admitted action and still leave open who decided that the machine could be in that situation at all. The decisions that set a machine's limits (the operating envelope, the response rules, the staffing ratio, the permission stage, the uses of its data) are made by different people, before any command crosses the causal boundary, and they appear in no runtime log. Each party can truthfully point to another. The Cruise collision showed the consequence: a record good enough to rebuild the motion, an account to regulators that left out the part that mattered, and penalties imposed for the record rather than the motion. Responsibility therefore has to be designed like any other premise of the machine's authority, written into a record before the machine moves and checked when it is released.

The same discipline applies to the people around the machine. A human role counts as a countermeasure only if its holder has the time, information, and authority to act; a remote assistant whose takeover needs several times the time the hazard leaves, or whose queue makes requests wait minutes, adds blame rather than protection. Permission to operate is granted in stages, each for a configuration, a domain, and reporting duties, and is withdrawn for a withheld record as well as for unsafe motion. A gap in detection between groups of people becomes a gap in stopping distance and impact energy, and it can be managed only with per-group samples large enough to resolve it. Risk falls on people who did not choose the machine, records about people must be declared and kept no longer than their purpose needs, and the labor a machine moves must be counted. The approval record collects all of these with named owners, and the release gate refuses operation while any of them is missing.

Table 17.3 collects the chapter's equations.

Table 17.3: Key equations of responsibility for physical AI: Each equation, what it bounds or decides about a person's role or a group's exposure, and where it is derived.

Equation	What it bounds or decides	Derived in
$\rho_{\text{serv}} = \lambda_{\text{arr}} \bar{T}$	The share of time one remote assistant is busy, which caps how many machines that person can serve	equation 17.1
$W_q = \rho_{\text{serv}} \bar{T}/(1-\rho_{\text{serv}})$	The mean wait before a help request receives a person's attention, which no hazard faster than minutes can rely on	equation 17.2
$N_{\text{frames}} = \lceil \ln p_{\text{miss}} / \ln(1-p_{\text{frame}}) \rceil$	The frames a person must stay in view before the design may count on detecting them, for one group	equation 17.3
$\Delta d_{\text{detect}} = v\, T_{\text{sample}}\, \Delta N_{\text{frames}}$	The clearance a less-detected group loses before braking begins	equation 17.4

Equation	What it bounds or decides	Derived in
$v_{\text{impact}} = \sqrt{2\, a_{\text{brake}}\, \Delta d_{\text{detect}}},\ E_k = m\, a_{\text{brake}}\, \Delta d_{\text{detect}}$	The impact speed and energy a stop sized for the best-detected group delivers to a less-detected one	equation 17.5

Key Takeaways: Responsibility for a machine that acts

- **Reconstruction is not attribution:** A log shows what was admitted. Only a record of decisions, each with a named owner, shows who made the admission possible, and those decisions must be recorded on purpose because the machine does not make them.
- **A role no person can perform assigns blame, not protection:** Test every human role the safety argument cites for time, information, and authority, including the queue one person serves, and let only feasible roles count as countermeasures.
- **Permission is staged and can be withdrawn:** Each stage covers a configuration, a domain, and reporting duties. It is left on evidence gathered at that stage and withdrawn for a configuration change, an incident, or a withheld record.
- **Measure fairness in meters and joules:** Convert per-group detection into the distance lost before braking and the impact it causes, and size each group's sample so its interval can resolve the gap.
- **Count the people who did not choose the machine:** Report exposure per group and per location, and give the people who bear the risk legible motion and signals backed by measured detection.
- **Records and labor about people carry their own premises:** Declare each record's purpose, retention, and readers, and account for the work a deployment moves onto people and the uses of their data it bars.
- **A missing owner refuses like an unknown premise:** The approval record is a standing condition of release; entries that cannot close go to the residual-claims register with a restriction and an owner.

The fourth law, permission must be earned (principle 4), lets a machine operate only as far as its evidence supports, and the earlier chapters of this part applied it to channels, faults, and claims. Responsibility applies it to the decisions behind them: an envelope is supported only if someone accepted it, a human safeguard only if a person can perform it, and a detection claim only if it holds for each group the machine meets. The approval record makes those conditions checkable by the same gate that checks friction and timing, so the release that admits the running machine to an open floor also names who answers for each of its decisions. No assignment of responsibility after a collision returns the energy the machine has already delivered across the causal boundary, which is why every one of these decisions has an owner before the first command is admitted.

17.12 Historical Perspective and Further Reading

The question of who answers for an outcome that many people produced is older than computing. Thompson (1980) named the problem of many hands in public administration, where officials who each contribute a small part to a harmful policy can each disclaim it. Nissenbaum (1996) carried the problem into computing and identified the barriers specific to software: the many hands that build it, the treatment of failures as unavoidable bugs, the use of the computer as a scapegoat, and the ownership of software without liability for it. Matthias (2004) argued that machines which learn make the gap wider, because their behavior is not fully determined by any rule their makers wrote. The field's founding case remains Therac-25, the radiation therapy machine whose software removed the hardware interlocks that had protected patients (Leveson and Turner 1993).

A second line of work asked when a person placed in the loop actually controls the outcome. Sheridan (1992) laid out supervisory control, in which a person sets goals and monitors while the machine acts, and Bainbridge (1983) showed why the person left to handle only the rare emergency

is the one least prepared to handle it. Santoni de Sio and Hoven (2018) turned the debate over autonomous weapons into general conditions for meaningful human control, and Elish (2019) named the moral crumple zone after tracing how blame was assigned in accidents from aviation to nuclear power.

A third line measured who the machine serves well. Buolamwini and Gebru (2018) showed that the benchmarks used to evaluate face analysis left out the people the systems failed on, Wilson et al. (2019) and X. Li et al. (2025) found the same pattern in pedestrian detection. Nissenbaum (2009) recast privacy as the question of whether information flows in the way its context expects, which is the question a declaration about recorded people answers. Regulation followed the same arc, from the General Data Protection Regulation (European Parliament and Council of the European Union 2016) to the European Union Artificial Intelligence Act (European Parliament and Council of the European Union 2024a) and the state and federal reporting duties for driverless vehicles (California Department of Motor Vehicles 2018a; National Highway Traffic Safety Administration 2021).

Six works reward careful study:

- Thompson (1980) defines the problem of many hands and the forms of responsibility that survive it, the starting point for recording decisions with their owners.
- Nissenbaum (1996) names the four barriers to answering for computer systems, each of which reappears in a physical machine's chain of decisions.
- Elish (2019) introduces the moral crumple zone and shows, case by case, how blame settles on the person nearest the event.
- Santoni de Sio and Hoven (2018) states the conditions under which human control of an autonomous system is real rather than nominal.
- Wilson et al. (2019) measures how pedestrian detection differs across skin types, the evidence behind measuring detection per group.
- Nissenbaum (2009) explains privacy as appropriate flow, the frame for declaring what each record about people may be used for.

DRAFT

What's Next: From responsibility to engineering judgment

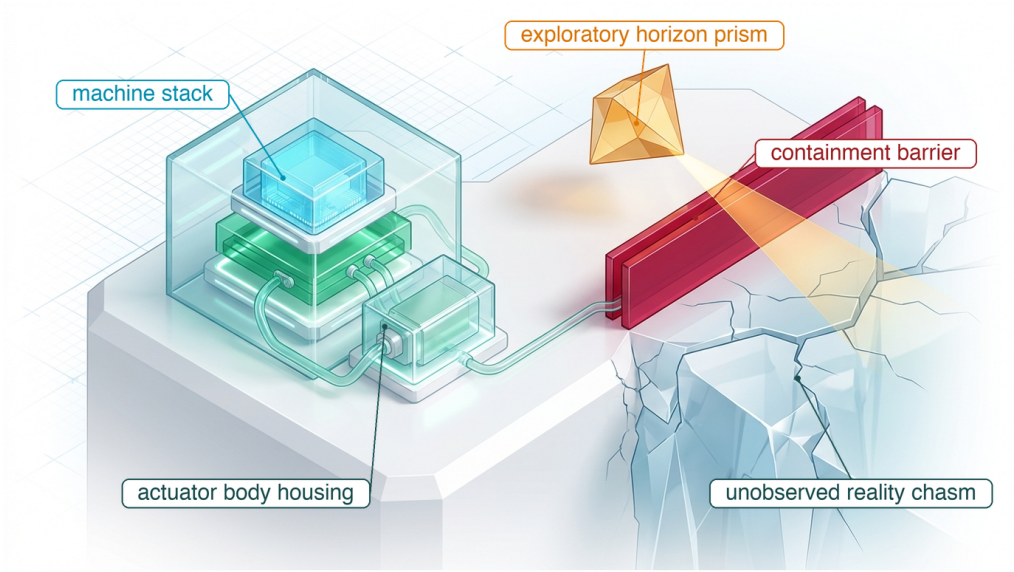
What do the release verdict and its approval record say about the question this book opened with? On the book's own records, the warehouse mobile manipulator is refused. The stipulated branch earns it only a restricted mission, and the approval record adds entries that cannot close before release: a per-group detection sample that is still filling, a staffing plan that depends on a hire, and a crossing rule that no sensor checks. Chapter 18 reads this verdict as the answer to why a better prediction can still produce a worse physical action. It transfers the same four-law method to an unfamiliar machine. It also gives each premise left open, whether physical or human, an owner, a restriction, and a closure test in the residual-claims register.

CONCLUSION

DRAFT

18

Engineering Physical AI



18.1 From a Better Prediction to Useful Service
18.2 Generalizing to Unfamiliar Systems
18.3 The Epistemic Frontier
18.4 Choosing the Next Capability
18.5 Fallacies and Pitfalls
18.6 Summary

Purpose

Why does a better model not, by itself, make a better machine?

The learned model is the part of a physical machine that is easiest to improve and easiest to score, so it attracts most of the effort. Useful service depends on everything around it as well. The body must sustain the task across loads, surfaces, and hours of duty. The learned behavior must hold in the states its own actions create. Each proposal must be admitted, corrected, or refused before the motion it asks for is already under way. The evidence for all of it must match the conditions of operation and stay true as the machine and its site change. Each of these can be engineered well in isolation while the machine as a whole remains unjustified, because doubt accumulates across every premise a physical claim depends on, and a premise that no one names or owns is assumed silently. A team that cannot join these parts into one argument keeps improving the model while an unmeasured floor, an unqualified detector, or an unowned restriction decides what the machine actually does. The same joined argument also chooses what to build next, since the most valuable measurement, experience, or design change is the one that retires the weakest open premise. At the causal boundary, irreversibility makes that weakest premise, not the best prediction, the measure of what a machine may be trusted to do.

Brain
1–50 Hz
perception, memory, intent, planning

above: propose; below: command

Nervous System
1 kHz (one implementation)
permission path, lease, fallback

Body
10–25 kHz current loop
drives, transmission, windings

Boundary
event-timed
register write, sensor capture

Physical World
continuous
contact, friction, payload, people

Learning Objectives

- Explain why a better prediction can still produce a worse physical action, and synthesize the four laws into the design argument that answers it
- Distinguish an unobserved premise from a monitored unknown, and explain why only a standing restriction can contain an unobserved premise
- Apply the four laws to select and evaluate a candidate operating mode for an unfamiliar machine
- Construct residual-claims entries with supported restrictions, accountable owners, and predeclared closure tests
- Evaluate the throughput cost of an operating restriction against the cost of the closure campaign that would retire it
- Select the next measurement, learning experiment, or design change needed to extend a machine's capability

18.1 From a Better Prediction to Useful Service

On this book's own release and approval records (chapter 16 and chapter 17), the warehouse mobile manipulator is refused. Even the stipulated evidence branch earns it only conditional service, held in place by premises that no sensor on the machine reports, such as the friction of the aisle floor. Improving its learned models alone would not change that verdict while the supporting physical and timing premises remain unqualified. That refusal is the engineering form of the answer to the question chapter 1 opened with: why a better prediction can still produce a worse physical action, and what the rest of the machine must supply so that a model's proposals become useful work within justified limits.

The question began with a contrast. Traditional machine learning evaluated advisory predictions against static datasets, and agentic machine learning could run iterative reasoning loops inside controlled digital sandboxes, behind glass, with elastic time, transactional rollback, and symbolic interfaces. Those privileges depended on the sandbox; external calls could already produce irreversible effects. Physical AI stepped through that glass into the continuous world, where computation commands actuators across the causal boundary. That step exposed computation to continuous time, in which delay becomes distance traveled, and to mechanical and thermodynamic irreversibility, which forbids undo. The illustrated feedback workload also required one action at a time under a real-time deadline, a batch size of one ($B = 1$). This scheduling constraint does not prohibit coordinated axes or batching elsewhere in the system. These constraints forced an architectural response, the separation of speculative proposals from deterministic permission, which is a design choice rather than a fact of physics.

One reference platform carried the argument: a warehouse mobile manipulator that picks an object from a moving conveyor, opens a spring-latched cage door, and carries the payload down an aisle shared with people. That mission exercised every capability the book develops (sizing the physical and computational dynamics, teaching learned policies, integrating a real-time runtime, evaluating closed-loop behavior, and maintaining the operating envelope as the machine and its site degrade). Each design choice constrained the next.

Predictive accuracy alone does not establish the operating premises on which physical success depends. What an actuator command does once it crosses the causal boundary depends on the distance a mobile base travels while a neural network runs its forward pass, the friction available at the tire-floor interface during hard braking, the states into which the robot's own prior actions drive it, and the gaps between demonstration datasets and live site conditions. Improving a model's statistical accuracy can refine the proposed trajectory, and shortening inference can recover timing margin, but neither result by itself qualifies traction, closed-loop competence, or the complete stopping budget.

The second half of the question, what the surrounding machine must supply, resolves into the four-step argument of section 1.3, pairing one engineering discipline with each of the four laws. Under the first law (*irreversibility*), the body's mechanical and thermal limits bound what any proposal

may request. Part I translated payload capacity, surface traction, contact forces, motor heating, and energy reserves into strict operating budgets that govern proposal horizons, execution cadence, and deterministic fallback paths. Under the second law (*endogenous data*), policy competence holds only over the states induced by closed-loop interaction. Part II tied proposer evaluation directly to the states the physical machine actually visits, turning recovery failures into targeted data-collection runs rather than unhandled anomalies. Under the third law (*proposal is not permission*), execution authority resides in an independent permission path rather than the learned model. Part III joined observation freshness contracts, grounded geometric targets, timed trajectories, and real-time admission filters onto a compute layout built to meet hard deadlines under load. Under the fourth law (*permission must be earned*), deployment scope extends only as far as qualified system-wide evidence supports it. Part IV structured physical operation as a governed service contract that separates proposer autonomy, whole-system task success, and protective intervention. A least-change speed filter, for example, can preserve an in-flight delivery even when a raw proposal requires intervention, whereas an emergency stop protects physical hardware without fulfilling the promised service.

The release verdict is where the four laws meet. In the baseline audit of section 16.6, the book's illustrative records yield `REFUSE`, because tail risks and fault margins remain unverified. The stipulated branch of section 16.6.1 shows which evidence would raise that verdict to conditional authorization (`COND`). Under those stipulated performance and fault records, the machine may run inspected-route transport, guarded door approaches, and close-proximity handovers, provided that standing conditions on floor traction, pedestrian crossings, feeder alignment, and sensor calibration stay verified. Each condition traces back through the record chain to a measured limit, an evaluated behavior, or a timed check, which is what separates a result stated under premises from a deployment claim.

Assembled, these parts close one causal loop through the physical world (figure 18.1). The loop runs at rates four orders of magnitude apart: the Brain proposes at 2 Hz, the permission path on the discrete safety microcontroller admits, corrects, or refuses at 1 kHz, and the drives close their current loops at 20 kHz. The loop closes only through the World, and the World returns only what its sensors capture, with capture times, to the permission path and to the Brain. A state the sensors never carry, such as the friction under a clear film, is invisible to every level at once. Governance surrounds all five levels and changes on the timescale of releases, so it sets the budgets and standing conditions that the fast loop enforces without taking part in any single control cycle.

The records summarized in table 18.1 bring the four laws into one engineering workflow. Rather than treating safety as an abstract property of a model, each part of the architecture leaves behind durable, auditable records that preserve the evidence and operating intent when tasks, payloads, or site conditions change.

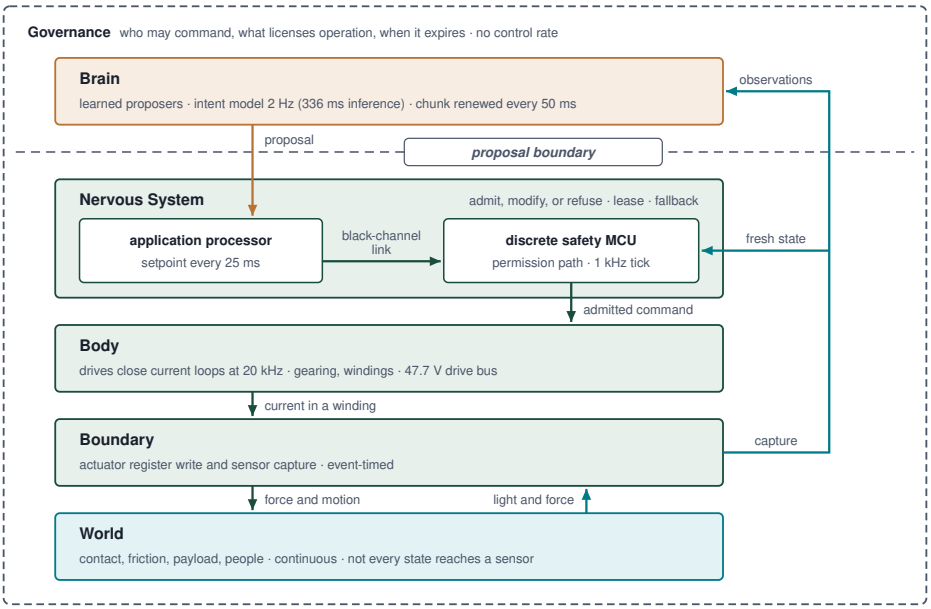


Figure 18.1: Closed causal loop: The Brain proposes across the proposal boundary, and the permission path in the Nervous System admits, modifies, or refuses each proposal before the Body acts. The Body's current becomes force and motion at the Boundary, the World evolves, and the Boundary returns measurements with capture times to the permission path and the Brain. Not every state of the World reaches a sensor. The governance envelope surrounds all five levels and changes on the timescale of releases rather than control cycles. The labels are this machine's values: the intent model proposes at 2 Hz, the chunk is renewed every 50 ms, the application processor sends a setpoint every 25 ms over a black-channel link to a discrete safety microcontroller unit (MCU) whose permission path ticks at 1 kHz, and the drives close their current loops at 20 kHz from a 47.7 V bus. Another machine sets its own rates.

Table 18.1: The four laws as engineering competence: Each part produces records that support the next design decision. The conditions and results belong to the particular machine and its evidence; they are not universal hardware requirements.

Law	Part	Records that carry it	Condition to maintain	Engineering result
Irreversibility	I (chapter 2, chapter 3, chapter 4)	Limit records; the proposer's limits, which enter the handoff record	Each measured limit, such as floor friction, payload, brake onset, or observation age, still holds under present conditions	A feasible speed or force envelope, a timed proposal interface, and a reachable fallback
Endogenous data	II (chapter 5, chapter 6, chapter 7)	Provenance record, policy manifest, evaluation record	The states the machine enters lie inside the target operational design domain (ODD, section 5.7.1) that closed-loop evaluation represents	A learned behavior evaluated in its closed loop; new qualified experience can expand its supported task population
Proposal is not permission	III (chapter 8, chapter 9, chapter 10, chapter 11, chapter 12, chapter 13)	Observation contract, belief record, intent lease, trajectory record, enforcement record, placement record	The permission path sees fresh, distinguishable state and meets its deadline under load	Useful motion admitted or minimally corrected within current limits; invalid active authority invokes a state-matched fallback
Permission must be earned	IV (chapter 14, chapter 15, chapter 16, chapter 17)	Authority record, fault record, release manifest, approval record, and the residual-claims register of this chapter	The premises of the release claim still hold after release	A named service authorized under standing conditions, with tested intervention and update procedures

The synthesis also completes the history traced in section 1.2. Over eight decades, learning moved from the machine's periphery toward its actuators. Deterministic mechanisms gave way to learned perception feeding analytical planners, then to end-to-end motor policies, and now to foundation models that propose action chunks across many robot bodies. At each step, a guarantee that physical separation or a watchful person once supplied had to move inside the machine.

The warehouse platform shows where that history now stands. Its split between an intent model proposing at 2 Hz and a chunk policy renewed every 50 ms mirrors current vision-language-action pipelines, and the guarantees those statistical models displaced survive in the deterministic architecture around them. The reflex halt beneath Shakey's STRIPS planner and the lower layers of Brooks's subsumption architecture, which kept running while higher layers were busy or wrong, have become the discrete permission path, enforcing signed kinematic and dynamic envelopes sized for this body. The remote operators who stayed close to the loop at the DARPA Robotics Challenge have become a formal intervention authority with timed handover budgets, and the environmental oversight a person once supplied has become explicit standing conditions of release.

The next phase, taken up in section 18.4.3, brings foundation models that control many bodies while learning continuously from fleet deployments. Cross-embodiment pretraining broadens what a model can propose, and it also carries the model further from the empirical distribution of any single site. Wider proposing latitude therefore makes the independent permission path more necessary, not less. It demands stronger checks at runtime and more defensible evidence at release, an architectural response to the safety-limit and conformity requirements of European machinery law (section 16.3.4).

The architecture is a method meant to carry across platforms. Its transferable core consists of the foundational questions, dynamic budgets, test protocols, and interface boundaries; the platform-specific elements are the calibrated numerical thresholds, kinematic envelopes, and actuator limits. The rest of this chapter tests how that core transfers to an unfamiliar packaging cell (section 18.2), tracks the premises no runtime sensor can check in a residual-claims register (section 18.3), and uses those open premises to select the next capability to build (section 18.4).

18.2 Generalizing to Unfamiliar Systems

An automated packaging cell that shuttles containers, seats closure tabs, and repeats high-rate cycles across a full shift can ram a closing gate, crush a carton wall, or bake its motor windings if its learned proposers outrun its physical limits. Building it requires the same pairing as the warehouse machine: a capable learned proposer with an execution pipeline whose runtime authority respects kinematic reach, contact force, and thermal buildup in the drive motors. The four bedrock laws trace one chain through such a machine. The chain runs from the task's irreversible consequences to measurable physical envelopes, through the closed-loop data that trains and validates its policies, to the real-time runtime that gates motor commands, and finally to the operating envelope certified for deployment. Auditing a machine the book has not yet studied tests whether that chain is a transferable method or only a description of one robot.

18.2.1 Auditing a new machine with the four laws

↩ The four bedrock laws, from irreversibility to permission must be earned, are established in section 1.10.

Auditing an unfamiliar machine out of order (evaluating a learned policy before the stopping budget fixes the speed ceiling, for example) wastes the evaluation on states the permission path will never admit. The audit therefore asks one question per law, in the order the laws constrain one another, and each answer leaves a record behind:

1. **Irreversibility.** What can the task do that no later command can undo, and what measured bound keeps it from happening? The answer names the collisions, plastic deformations, and thermal overloads of the task and sizes the envelope that excludes them, such as a speed ceiling from the stopping budget. It produces the machine's limit records.
2. **Endogenous data.** Do the demonstrations and the closed-loop evaluation cover the states the machine will enter once that envelope is enforced? A limit added in step 1 changes the states the proposer sees, so the evaluation must be run inside the envelope, not before it. It produces the provenance record and the evaluation record.

3. **Proposal is not permission.** Which independent path admits, corrects, or refuses each proposal, on which sensors, within which timing budget, and with which fallback when permission lapses? It produces the enforcement record and the timing budget the permission path must meet.
4. **Permission must be earned.** Which operating claims does the assembled evidence support, under which standing conditions, who answers for each decision behind them, and which premises remain open? It produces the release manifest, the approval record, and the residual-claims register entries for every premise left open.

Because each step constrains the next, adjustments ripple back up the list. Reducing the permitted transit speed to satisfy a stopping budget in step 1 alters the visual flow and dynamic states that step 2 must evaluate. Tracing those four steps on a high-rate packaging cell shows how a single limit in step 1 reshapes the data, runtime checks, and release claims that follow.

18.2.2 A packaging cell across the three machine classes

Consider a packaging cell in a logistics hub, laid out as in figure 18.2 (panel a). A carriage carrying one carton runs along a straight track, driven by a linear servo drive at the track's near end. Partway along, an insertion head presses down on the carton's flaps to seat its retention tabs. At the far end, a gate closes across the track whenever the next station cannot accept a carton, and a carriage still moving when the gate closes drives its carton into the gate and the frame. The cell is laid out tightly for cycle rate, so the carriage is only a few centimeters from its stop line when a closure can begin. This single cell couples all three machine classes within one enclosure: the carriage as moving mass, the insertion head as contact force, and the drive as thermal dissipation, because it cycles the carriage through rapid accelerations all shift.

↩ The three machine classes (mass-dominated mobility, contact-dominated manipulation, and energy- and thermal-dominated operation) are established in section 1.5.

The first law asks what the machine cannot undo, so the audit begins by cataloging the cell's irreversible state changes alongside the governing physical bounds. A loaded parcel container moving along a linear transfer rail carries momentum. When a carriage strikes an obstruction, kinetic energy dissipates through plastic deformation of the container and structural deflection of the frame, and commanding the drive motor in reverse cannot undo that damage. The insertion head introduces a second irreversible boundary: an actuator drives a tool head against package tabs to lock retaining clips. If the actuator pushes harder than the carton wall can support in compression, the container buckles and crushes its contents. The high-duty drive introduces a third irreversible boundary through Joule heating ($P_{\text{loss}} = i_q^2 R(T_{\text{winding}})$). Continuous high-acceleration cycles pump heat into sealed motor windings. Winding insulation does not need to melt to fail; elevated temperatures speed up chemical degradation over time, shortening insulation life rapidly (International Electrotechnical Commission 2007). Cooling the housing afterward cannot restore the dielectric strength already lost. Because insulation damage accumulates across duty cycles, an instantaneous temperature trip threshold alone cannot bound lifetime wear.

Because the carriage's momentum is already committed, the permission path must act before the carriage consumes its clearance. The stop modeled next is the cell's application-level path: a camera watches the gate, a learned model recognizes that a closure has begun, and the admitted stop command goes to the carriage brake. That path keeps normal operation clear of the gate, but it is not the cell's certified protective function, which the third-law discussion below and Problem 1 take up. The constant-deceleration stopping relation of section 2.3.1 tests a candidate design for it, here on hypothetical scenario inputs rather than measurements from a commissioned cell.

Napkin Math 18.1: The delay wall

Problem: The loaded carriage travels at 1.50 m/s toward a gate whose fixed stop line is 60 mm away when the hazardous closure begins, maintaining constant speed until braking begins. Can it stop short of the line, and if not, what is the fastest speed whose stop fits once a 15 mm reserve is held back?

Variables:

- Moving mass of carriage and loaded tray m = 12 kg and nominal speed v = 1.50 m/s (Illustrative)
- Clearance 60 mm, less 15 mm reserved for uncertainty and residual clearance, leaves a stopping budget d_{budget} = 45 mm (Illustrative)
- Braking lower bound a_{brake} = 15 m/s² after the delay, for whichever brake the stop command engages (Illustrative; a bound to qualify under load, pad wear, and rail conditions, not a motor's peak rating)
- Delay from closure to full braking τ_{delay}: 16 ms to a usable image, including frame phase and exposure, 36 ms for transfer, inference, scheduling, and admission, 2 ms for command delivery, and 10 ms for brake buildup, in all 64 ms (Illustrative upper bounds)

Math:

1. **Nominal stop** (equation 2.2, margins held in the reserve): $d_{\text{stop}} = v\tau_{\text{delay}} + v^2/2a_{\text{brake}}$ = 1.50 m/s × 64 ms + (1.50 m/s)^2/(2× 15 m/s²) = 96 mm + 75 mm = 171 mm, a deficit of 111 mm.
2. **Speed ceiling** (equation 2.3 with d_{budget} as the available distance): $a_{\text{brake}}\tau_{\text{delay}}$ = 0.96 m/s and $2a_{\text{brake}}d_{\text{budget}}$ = 1.35 m²/s², and the square root of (0.96 m/s)^2 + 1.35 m²/s² is 1.51 m/s. The positive root is v_{max} = 1.51 m/s − 0.96 m/s = 0.55 m/s.
3. **Candidate mode** at 0.50 m/s, below the ceiling: 32 mm of delay travel + 8.3 mm of braking = 40.3 mm, leaving 19.7 mm before the stop line. Translational energy $E_k = mv^2/2$ falls from 13.5 J to 1.5 J, a reduction of 88.9 percent.

Result: At nominal speed the carriage reaches the gate before braking even begins. The budget admits at most 0.55 m/s, and the 0.50 m/s candidate keeps more than the allocated reserve.

Systems insight: At these speeds delay travel, not braking, exhausts the clearance. The 64 ms chain alone carries the nominal carriage 96 mm, farther than the stop line, so the condition $d_{\text{stop}} \leq D_{\text{clear}}$ of principle 1 is met first by lowering speed or shortening the chain from closure to braking. The carriage's speed limit belongs to that whole chain, not to its brake alone.

The slower mode is therefore worth qualifying. Cutting speed to a third shrinks the two terms of the stop unequally (figure 18.2). Delay travel is linear in speed and falls threefold, from 96 mm to 32 mm; braking distance is quadratic and falls ninefold, to 8.3 mm. At the restricted speed the delay chain, not the brake, therefore sets most of the stop, which is why the next question is which premise moves the ceiling most.

The positive root also shows how the ceiling responds to each premise. Halving the processing allocation to 18 ms raises the ceiling to 0.66 m/s; because delay travel dominates this stop, reducing that allocation recovers substantial speed margin. Halving the qualified braking bound to 7.5 m/s² lowers the ceiling to 0.47 m/s, below the candidate 0.50 m/s mode, which then no longer fits. The two premises play different roles. Shorter latency is an opportunity for throughput, whereas a braking shortfall invalidates the mode outright, which is why the loaded deceleration must be qualified under worst-case conditions before the cell is commissioned. The stopping equation fixes the relationship; only measurements on the assembled cell can establish that the latency budget, the deceleration bound, and the reserve hold.

The second law evaluates the competence of the cell's learned proposers in the closed loop. This packaging cell runs three distinct learned proposers. A vision model infers container contents to propose carriage transit speeds (fragile or liquid contents tolerate gentler accelerations and stops than solid goods). A multimodal network estimates cardboard thickness to propose insertion forces and dwell durations. An electro-thermal estimator proposes acceleration profiles to manage motor winding temperatures. Each proposer's actions alter what it observes next. Throttling carriage speed changes camera frame timing and motion blur, extending tool dwell changes the material deformation that later thickness estimates read, and reshaping acceleration profiles changes the temperature trajectory the thermal estimator measures. Evidence of proposer competence therefore holds only for the state distribution sampled during closed-loop evaluation, and the question is

↩ This speed ceiling, the positive root of the stopping equation, is established for the mobile base in section 2.3.2.

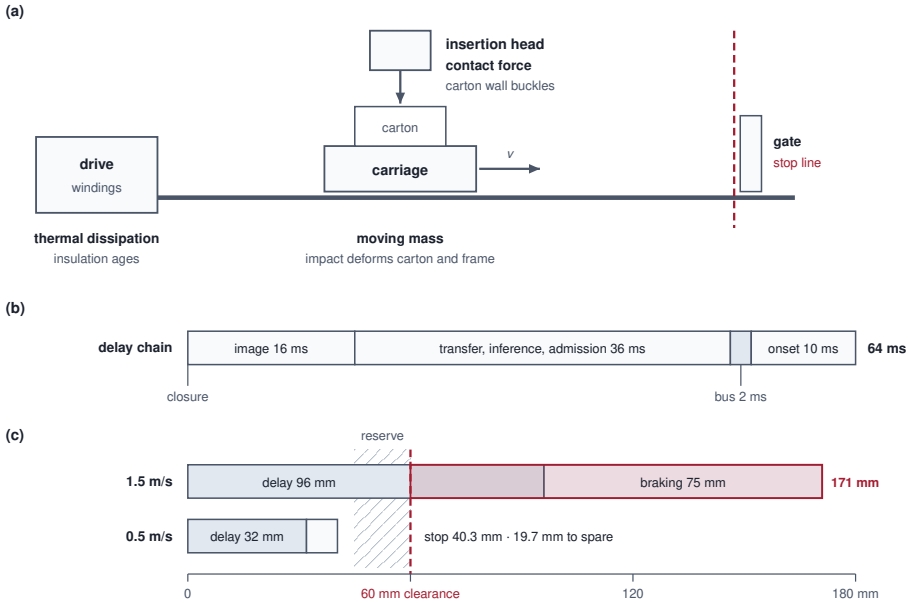


Figure 18.2: Three machine classes in one packaging cell: (a) The carriage is the moving mass whose impact deforms carton and frame, the insertion head applies the contact force that can buckle a carton wall, and the drive dissipates heat that ages winding insulation. (b) The assumed delay chain from gate closure to full braking totals 64 ms. (c) At 1.50 m/s the carriage travels 96 mm before braking begins and stops after 171 mm, far beyond the 60 mm stop line; at the restricted 0.50 m/s it stops in 40.3 mm, clear of the 15 mm reserve. All inputs are hypothetical design allocations, not measurements.

whether that distribution matches the conditions admitted at runtime. A safety restriction imposed after training shifts that distribution. If closed-loop evaluation sampled only the nominal transit speed of 1.50 m/s, clamping speed to 0.50 m/s pushes the vision model into visual dynamics it never encountered during evaluation, so the restriction that secures the stopping distance also opens a regime without evidence. The engineering team must either evaluate the policy under derated conditions before release or record the unverified regime as an open premise in the release sign-off.

↩ A policy whose actions choose its next observations drives the endogenous covariate shift established in section 5.2.

Closing this gap requires matching the data-collection and evaluation campaign to the restricted operating envelope. Expert demonstrations must span the admitted transport velocities, insertion forces, and thermal regimes, preserving complete site metadata in the provenance record. Once trained, the candidate proposer must undergo closed-loop trials on the physical cell to confirm that it completes the packaging task under those conditions, including during interventions by the permission path. If the cell cannot maintain acceptable task performance within its physical bounds, engineers must revise the task specification, the proposer architecture, or the mechanical plant before re-evaluating the modified system. Calculating a speed ceiling tests the transport stopping budget under its stated premises; qualifying those premises and demonstrating closed-loop task performance are both required for a viable packaging cell.

↩ Why proposer evidence must come from the closed loop the machine will actually run is established in section 7.3.

The third law places these proposed actions beneath an architectural authority split that the proposers cannot bypass, delay, or corrupt. The permission path prevents unverified neural proposals from reaching physical actuators directly. In this packaging cell, an independent kinematic monitor runs at 1 kHz, reading the carriage's optical encoders and a rangefinder aimed at the gate, and admits a carriage motion command only when the measured clearance supports the qualified stopping distance. This monitor is the admission stage of the camera path in notebook 18.1: the camera reports that a closure has begun, and the monitor admits the stop. Its work is therefore part of the 36 ms processing allocation, and a 1 kHz tick bounds only how long a command waits for the next check, not the transfer, inference, and scheduling that come before it.

While the software kinematic check confines normal operation to the derated velocity mode, it does not carry the cell's formal protection claims. In an industrial deployment, harm-prevention

claims rest on hardware-certified drive safety functions codified under IEC 61800-5-2 (International Electrotechnical Commission 2016b) and mapped for warehouse automation in table 12.4. The restricted transport mode maps to safely limited speed (SLS) on the carriage drive, an independent hardware function that continuously monitors motor velocity and triggers an emergency stop if the limit is exceeded. The emergency response executes as a ramp-monitored safe stop 1 (SS1) terminating in safe torque off (STO), with safe brake control (SBC) engaging mechanical holding brakes at rest. Each safety function requires a validated timing budget and an integrity metric. Under ISO 13849-1 (International Organization for Standardization 2023a), safety functions are classified by performance level (PL), which bounds their average probability of dangerous failure per hour according to the cell's risk assessment. The camera-triggered stop budgeted in notebook 18.1 is the application-level path announced before it, not a protective safety function. A safety-rated optical gate sensor feeding the drive's SS1 input directly replaces the frame-exposure and neural-inference allocations with its certified hardware response time, while command delivery and brake onset still contribute to τ_{delay} in the physical stopping model.

Physical safety limits must govern the insertion tool and actuator drives just as strictly as carriage motion. Setting a mechanical force limiter to 220 N is a nominal design choice rather than proof that every parcel resists crushing. Physical tests must verify that peak transmitted forces, including dynamic overshoot during contact impact, stay below the yield strength of the weakest admitted carton and tab geometry. Similarly, setting a winding thermistor trip threshold at 105 °C and a stall detection timeout of 400 ms provides basic supervisory cutoffs rather than a certified thermal envelope. Safe continuous operation depends on drive current profiles, winding resistance, heat capacity, cooling airflow, and sensor thermal lag. Because internal winding temperatures can surge before external housing sensors register the heat, as modeled in section 2.5.1, engineers cannot validate a sustained duty cycle from trip thresholds alone without measuring the thermal response under load.

High-rate deceleration also transfers mechanical kinetic energy back into the electrical drive system. During regenerative braking, the decelerating motor returns power to the inverter DC bus. A dynamic brake chopper and power resistor must absorb this energy before bus voltage exceeds the inverter trip threshold of 60 V, following the bus energy balance in section 2.6.3. The energy sink must handle repeated stops across a full shift rather than an isolated emergency event. Furthermore, regenerative braking requires an energized inverter stage throughout deceleration, whereas the carriage's electromechanical track brake relies on its own spring-actuated mechanism. Logic power hold-up alone supplies neither regenerative braking torque nor friction-brake clamping force. A thermal fault or electrical shutdown that cuts drive power disables regenerative braking, leaving deceleration entirely to the mechanical brake. The notebook's 15 m/s^2 bound therefore holds after a power cut only if the spring-applied track brake is itself qualified to it. Otherwise the power-cut case has a lower deceleration, a longer stop, and a lower speed ceiling of its own. System designers must ensure that safety-directed power cuts preserve a qualified deceleration and holding response across the declared operating states and fault conditions.

The fourth law synthesizes these engineering decisions into a release argument. The cell's release manifest must validate rangefinder coverage, gate clearance margins, braking deceleration, and sustained task throughput as one connected system. Equally important, the manifest explicitly records any claims that physical evidence cannot support. To check whether the tabs have latched, the engineering team evaluates a vision model that inspects surface reflections for container tab engagement. Under industrial overhead lighting, glare from tape and crinkled plastic packaging produces bright reflections that mimic latch defects while hiding genuine tab misalignments. Because the cell lacks an independent physical measurement, such as a tactile switch or a laser profile scanner, that separates engaged tabs from misaligned flaps under glare, the sign-off claim lacks a qualified detector. Recording this limitation as detectorless and unsupported in the residual-claims register (section 18.3.3) is not a documentation failure but an accurate statement of what the physical evidence justifies.

The stopping analysis therefore supports a *candidate restricted mode* for carriage transit at 0.50 m/s, provided that the 64 ms end-to-end latency, 15 m/s^2 braking deceleration, 60 mm physical clearance, and uncertainty reserve are validated together on the physical cell. Insertion forces and thermal duty cycles remain restricted until physical testing confirms material crush limits, sensor thermal

lag, and independent fault paths. Autonomous package sign-off is refused because glare hides tab engagement from the vision system and the cell lacks an independent verification sensor. A qualified tactile or laser engagement sensor could support autonomous sign-off; mandatory downstream manual inspection instead supports a commercial deployment that makes no autonomous sign-off claim. An analytical throughput model cannot substitute for that physical evidence. Each law's answer constrains the next. The stopping mechanics fix a transport envelope, the envelope fixes the regime that data collection and evaluation must cover, the runtime monitors enforce that envelope, and the release argument restricts the cell wherever a claim lacks support.

The same audit extends beyond the three machine classes of the packaging cell into fluid and chemical process systems, where the irreversible consequence may be reagent already delivered or heat already transferred rather than damage from a moving mass. Consider a learned proposer that sets the opening of a dosing valve feeding a reaction tank, with a high-level trip that commands the valve shut before the tank overfills. The first law asks what reagent has already been committed. If the shutoff command arrives after a delay τ_{delay} at a volumetric flow rate $\dot{V}$, fluid continues to discharge. The discharge adds roughly $\dot{V}\tau_{\text{delay}}$ to the volume delivered before the valve even begins to move, the fluidic analog of the delay travel $v\tau_{\text{delay}}$ in the stopping distance. The valve's closing stroke then adds the fluidic analog of braking distance, and both must fit within the headroom above the trip level, as the clearance did for the carriage. Stored thermodynamic and hydraulic energy also complicates the fallback. A pressurized accumulator, a cryogenic transfer line, or a heated pressure vessel retains stored energy after electronic control power is cut. As with a balancing biped, whose fall begins the moment its torque is removed, de-energizing the controller does not by itself produce a safe state. The design must therefore name a qualified physical sink, such as a spring-loaded pressure relief bypass, to dissipate that energy. The remaining laws apply in the same way. Policy evaluation must sample the throttled flow regimes that the safety limits enforce, the permission path must guarantee valve closure within the allowable volume tolerance, and sign-off must rest on measured valve slew rates and sensor response times. In every case the machine's operating claims hold only as far as its physical evidence reaches, and a physical state that stays unobserved until the execution deadline marks where that evidence ends.

Checkpoint 18.1: Transferring the four-law audit

The packaging cell demonstrates how the four-law audit organizes system design for an entirely novel machine, anchoring every operational claim to physical bounds and empirical measurements.

- ☐ **Speed-ceiling estimate**: Suppose physical load testing qualifies the carriage's braking bound at only 12 m/s², assuming the same 64 ms total delay and 45 mm stopping budget. Estimate the updated velocity ceiling from the positive root of the stopping equation, and determine whether the candidate 0.50 m/s mode still fits within the updated stopping budget under the remaining assumed inputs.
- ☐ **Machine classes**: For the carriage, the insertion head, and the linear drive motors, identify each component's physical machine class, the irreversible physical state transition it can produce, and the governing physical boundary.
- ☐ **Derated evaluation**: Explain why imposing a runtime velocity restriction after training induces an endogenous covariate shift for the vision proposer. What two engineering options can close this evaluation gap before operational sign-off?
- ☐ **Sign-off verdict**: What specific physical sensing evidence would be required to transition autonomous package sign-off from refused to supported, and why does expanding the training dataset with more glare-corrupted images fail to resolve the underlying detector deficiency?

18.3 The Epistemic Frontier

A packaging cell's camera can confirm an arm's position and approve a transport motion without verifying that a hidden mechanical tab has engaged. Similarly, a warehouse robot's lidar and cameras

can clear a navigation path across an aisle without determining whether the concrete floor provides enough traction to brake. These gaps are epistemic limits: boundaries on what the available sensors and analytical arguments can justify. Additional training can improve a policy's action proposals, but machine learning cannot manufacture an unmeasured physical state. Systems engineers must therefore determine whether justifying the next operating claim requires broader training experience or a physical measurement absent from the existing sensor suite.

Consider an aisle where a pallet jack with a leaking hydraulic line has just crossed, leaving a thin, clear film of fluid on polished concrete. The mobile manipulator's stopping budget assumes the floor supplies enough surface friction for credible deceleration, which dry industrial concrete reliably does. Under an oil film, however, available traction collapses, yet neither standard navigation cameras nor time-of-flight lidars can tell the clear fluid apart from dry concrete. In this failure mode the safety architecture runs exactly as specified. The software permission path meets every deadline, chunk leases expire on schedule, and the drive controller applies braking torque right on time. Even so, the robot slides past its stop line and collides at the rack end. The failure comes not from software latency or actuator lag, but from an unmonitored premise, the local wheel-floor friction coefficient before braking, and the stopping budget prices that premise.

The claim adjudication in section 16.3.3 already encountered this premise: claim C3 (braking traction) was refused on this book's own evidence and deferred to this chapter as an unresolved premise. The credible deceleration a_{brake} that chapter 2 credited to the brakes was always conditional on the floor. Made explicit, that condition says the deceleration a machine actually has available is the smaller of what its brakes can deliver and what tire-floor traction can transmit:[1]

$$a = \min(a_{\text{brake}}, \mu g). \tag{18.1}$$

The friction term is the traction ceiling of chapter 2 (figure 2.1 and section 2.4.2), $\mu\phi_{\text{drive}}g$, taken with all of the weight on the driven wheels ($\phi_{\text{drive}} = 1$). That is the case that sets the friction floor, the same optimistic all-weight screen that claim C3 was adjudicated against. At the drive wheels' rated load share the floor is higher, as that notebook computes, so every friction margin below is the optimistic one. On an oily surface, interface friction rather than motor torque becomes the binding constraint.

1 **Amontons-Coulomb Friction**: The classical laws of sliding friction make the friction force proportional to the normal load and independent of the apparent contact area, which is what lets a single coefficient μ describe a contact. Bowden and Tabor (1950) traced the law to the small true area of contact, whose shear strength sets μ, and showed that a lubricant film only molecules thick can lower it several-fold. The coefficient therefore belongs to the pair of surfaces and whatever lies between them, not to the floor alone, and the braking premise needs a measurement at the tire-floor interface that no camera viewing the floor supplies.

Napkin Math 18.2: The friction floor

Problem: A clear hydraulic film lies across the aisle. Does the known-component stop from the candidate 1.3 m/s aisle speed still fit the rack end's 1.10 m clear distance?

Variables:

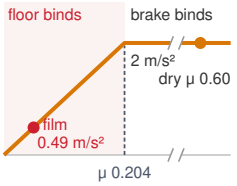
- Credible deceleration a_{brake} = 2 m/s² (Illustrative, section 2.3) and g = 9.81 m/s² (Defined standard-gravity constant)
- Floor friction μ = 0.60 on dry industrial concrete and 0.05 under the film (both Illustrative; see the Reader Guide)
- Aisle speed v = 1.3 m/s (Chosen) and clear distance D_{clear} = 1.10 m (Illustrative)
- Known-component budget, the seven rows of table 12.1, which leave out the dispatch-to-admission interval Δ_{da} that is allocated but not yet measured: pre-brake delay τ_{delay} = 133.6 ms, jerk bound j_{max} = 7.11 m/s³, and fixed overheads of 190 mm (all Derived)

Math:

1. **Friction floor** (equation 18.1): The brake binds only while $\mu \geq a_{\text{brake}}/g$ = 2 m/s² / 9.81 m/s² = 0.204. Dry concrete clears it; the film does not, so $a = \mu g$ = 0.05 × 9.81 m/s² = 0.49 m/s².
2. **Dry-floor budget**: Delay travel $v\tau_{\text{delay}}$ + braking $v^2/2a$ + jerk-ramp travel $va/2j_{\text{max}}$ (the S-curve term of row 6, valid in the trapezoidal regime because $v > a^2/j_{\text{max}}$) + overheads = 173.7 mm + 422.5 mm + 182.8 mm + 190 mm = 969.0 mm.
3. **Film budget**: The same terms at a = 0.49 m/s² give 173.7 mm + 1,723.3 mm + 44.8 mm + 190 mm = 2.13 m.

Result: On the film the stop grows from 969.0 mm to 2.13 m, far past the 1.10 m clear distance, before the dispatch-to-admission interval is even added.

Systems insight: Below $\mu = 0.204$ the floor, not the brake or the timing, sets the deceleration, and the braking term grows as $1/\mu$. Faster admission can recover only the 173.7 mm of delay travel, so no improvement in the permission path restores the dry-floor mode on the film. Only a restriction or a measurement at the tire-floor interface can keep $d_{\text{stop}} \leq D_{\text{clear}}$.



Below $\mu = 0.204$, the floor, not the brake, sets the deceleration.

The calculation omits the dispatch-to-admission interval, which the discrete placement of section 13.6.2 allocates but no loaded measurement yet confirms. That interval adds travel and reduces every derived speed ceiling, yet even the known-component budget already excludes the film. Moreover, the friction premise did not appear only when fluid leaked onto the concrete. The candidate normal operating mode of section 12.3.1 always assumed that unmeasured friction floor across every aisle, dry or wet. Software timing and physical traction are independent requirements, since bounding scheduling delays does not guarantee floor grip and confirming floor grip does not remove scheduling delays. On the oil film, the drive electronics continue to regulate motor current and commanded torque within milliseconds, yet that torque slows the base only through tire-floor contact. Neither a faster enforcer nor another layer of neural inference therefore replaces the interface measurement or the speed restriction the notebook calls for.

↩ Current regulation within milliseconds through electrical time constants and current clamping is established in section 2.4.2.

The manipulator arm's contact limit at the door latch has the same structure. Safety depends on a latch that holds under any force below the arm's contact ceiling. Mechanical wear, material fatigue, or corrosion can silently lower the latch's yield force below that ceiling, even while joint torque sensors report readings identical to those against a sound strike plate. A wrist force-torque sensor whose zero-point calibration has drifted creates the same hazard for every contact threshold that depends on its signal (section 18.3.4.2).

In each case, a machine can leave its safe operating envelope while internal telemetry stays completely nominal. Successful test runs under nominal conditions provide no safety guarantee once an underlying physical premise fails. A safety case must explicitly list each unobserved premise, restrict operating authority while that premise remains unverified, and state the physical evidence required to close it. For every open entry, the engineer must determine what the machine can physically measure, what actuator limits can still prevent, and which operating claims must remain refused.

18.3.1 What the release claim rests on

Extensive testing of one link in a safety case cannot compensate for an unmeasured premise in another: a thousand hours of fault-free perception trials will not stop a base whose tires lose traction on an oily floor.[2] A mobile manipulator's release claim forms a chain of dependent arguments tied to its operating hazards and chosen mitigations. The complete safety case must establish the operational design domain, measured actuator performance, stopping bounds, component interface contracts, and independent permission filtering for learned proposals. Hazards managed by runtime detection require verified detector coverage and bounded monitoring latency, whereas hazards managed through passive compliance or operating restrictions require physical evidence that those safeguards hold across declared payloads, component wear, and site conditions.

Each premise in a safety case carries one of five statuses: established by measurement or derivation, assumed, decided by accountable judgment, unsupported, or outside the threat model. (The Reader Guide lists the same five with examples; the audits of this chapter and its problems use these exact labels.) The envelope trichotomy (definition 16.3) classifies an actively monitored runtime condition as known-true, known-false, or unknown. That trichotomy presupposes a monitor. "Unknown" marks a monitored condition whose evidence has lapsed, and the permission path answers it at once with a validated stop or hold until fresh evidence returns. An unobserved premise has no runtime monitor whose report can trigger that transition; its missing evidence must instead be recorded in the release argument. Telemetry stays nominal while the premise fails, and holding the machine until a report arrives would hold it forever, because no report will come. Such premises require standing restrictions decided before operation; a fallback cannot be triggered by evidence the machine never receives. Release certification must therefore either validate and maintain these

[2] **Popperian Falsification**: The asymmetry holds for universal hypotheses (Popper 1959). No finite run of safe stops proves that every aisle floor supplies the friction floor, and one measured film below it refutes that premise. A rate claim, such as a hazard rate per hour, is neither refuted by one incident nor proved by many incident-free hours, so it is bounded statistically and revised as exposure accumulates. This chapter therefore holds deterministic premises in register entries and rate claims in confidence bounds.

premises through external physical evidence or restrict operating authority accordingly. For the mobile base, stopping-distance guarantees rest on an assumed wheel-floor friction coefficient that cannot be observed before braking. For the manipulator arm, contact safety assumes a minimum yield strength for the door latch and the objects it handles, a threshold set by engineering judgment rather than real-time measurement. For the wrist sensor, force-limiting safety functions assume zero-point calibration stability, a condition internal telemetry cannot verify without an unloaded physical tare. Recording these premises explicitly does not weaken a release case; it marks the exact boundary where system guarantees hold and beyond which operating authority must be withdrawn.

Separating physical measurement from accountable judgment is essential to systems engineering. Testing and statistical analysis can estimate physical exposure, quantify sensor noise, and bound parameter uncertainty, but test data cannot choose an acceptable residual risk or decide which people in a facility must bear that risk. Those choices are accountable engineering decisions that must be recorded as explicit judgments rather than presented as empirical facts. Similarly, the verification campaign's threat model bounds where the safety record holds. Field trials across normal fleet workflows and random component faults do not by themselves support claims about deliberate attacks outside the tested conditions. A test campaign showing that a vision pipeline tolerates lens flare, sensor noise, and partial occlusions provides no evidence that it resists adversarial optical projections, tampered policy weights, or deliberate denial-of-service flooding on the internal communication bus.

↩ The fault list that bounds what verification tests, including transduction-layer attacks on sensors and on the proposer, is established in section 15.6.

18.3.2 Conditional shield credit and common-cause bounds

Treating an independent permission path as a substitute for a missing floor-friction measurement asks the safety shield to stop a hazard it cannot sense. Even for hazards within its view, a shield reduces the failure rate only in proportion to its own tested coverage. Pricing what the permission path actually contributes to a hazard-rate claim separates the hazards a shield can attenuate from the register entries it can never close. Section 12.7 established deterministic guarantees for each command a safety shield checks, but turning those per-command guarantees into a system-level hazard rate requires statistical accounting. Because a shield acts as a conditional filter, the resulting hazard rate depends on how often the policy proposes hazardous actions, how reliably the shield intercepts them in those exact states, and how often failures occur outside the shield's sensing domain. Notebook 18.3 prices these terms in an illustrative deployment.

Consider an illustrative system target of $1 \times 10^{-9}\ \mathrm{h}^{-1}$. This number comes from civil aviation certification, where catastrophic failure conditions must be shown to be "extremely improbable," on the order of 10^{-9} per flight hour[3] (Federal Aviation Administration 1988). While no industrial robotics standard requires this threshold, examining it shows the statistical burden a borrowed safety target imposes. As the exposure wall established, physical operating hours alone cannot certify failure rates of this scale.[4] The practical engineering question is what safety margin a runtime shield genuinely contributes against that target, and what supporting evidence its own operation requires.

A safety shield argument is inherently *conditional*. Observed incident rates across an operating fleet already include the protective interventions of runtime filters and human supervisors, so multiplying an observed incident rate by a shield's interception factor double-counts that protection. For a defined hazard class h, let $\lambda_{\text{proposal},h}$ denote the arrival rate of unshielded, hazardous action proposals emitted by the policy before the permission layer. Let $c_h = \Pr(\text{intercept} \mid \text{hazardous proposal}, h)$ denote the verified interception coverage for that hazard class. Including an independently bounded hardware fault and common-cause arrival rate λ_{other}, the conditional hazard rate obeys equation 18.2:

$$\lambda_{\text{system},h} \leq \lambda_{\text{proposal},h}(1 - c_h) + \lambda_{\text{other}} \tag{18.2}$$

Here, $\lambda_{\text{system},h}$, $\lambda_{\text{proposal},h}$, and λ_{other} are rates per operating hour, while c_h is a unitless conditional probability. The limiting cases make the shield's role plain. With perfect detection and interception ($c_h = 1$), the upper bound reduces to the hardware and common-cause term λ_{other}, a residual no coverage removes. Conversely, if the shield has zero coverage ($c_h = 0$), the raw proposal hazard rate passes straight to the actuators. This relation turns the runtime assurance architecture, which

[3] **Extremely Improbable Band**: The advisory circular offers its probability bands as aids to engineering judgment, expressed per flight hour of a mean-duration flight and applied to each failure condition separately, and it grounds such criteria in historical accident data (Federal Aviation Administration 1988). The 10^{-9} figure is therefore one catastrophic condition's share of a certified design's risk, not a rate for a whole aircraft, let alone for another kind of machine. Applying it elsewhere without the allocation argument that set its share leaves the number without its basis.

[4] **Pooling Evidence Across Updates**: An updated model may alter the proposal distribution. Earlier data can be pooled only with a justified configuration-equivalence and exposure argument; an update does not automatically erase every observation. The exposure wall's constant-rate model also requires a defensible treatment of shared fleet conditions and correlated faults.

↩ The exposure wall, the event-free operating hours needed to bound a rare hazard rate, is established in section 7.9.

pairs an unverified primary controller with a verified safety filter (Sha 2001; Hsu et al. 2024), into a checkable hazard-rate claim.

↩ The Simplex architecture, which pairs a capable unverified proposer with a verified protective controller, is established in section 12.1.

Using equation 18.2 requires four premises. The first is a verified **counterfactual oracle**: an evaluation harness that determines whether an unshielded action proposal would have caused a safety violation. Without this oracle or another justified way to identify or conservatively bound hazardous proposals, the raw proposal error rate $\lambda_{\text{proposal},h}$ has no supporting evidence. The remaining requirements are timely hazard detection, adequate braking authority, and a careful accounting of correlated failures. Designers frequently overlook this final common-cause term. Release claim C5 required physical testing rather than a theoretical independence assumption because separate implementations of the same specification fail together far more often than statistical independence predicts (section 16.3.2). The product $\lambda_{\text{proposal},h}(1-c_h)$ uses interception coverage conditional on hazardous proposals; substituting coverage measured on unrelated states erases that conditioning. An occluded camera frame that causes the neural policy to propose a bad trajectory can corrupt the safety shield's state estimator at the same moment. That shared failure does not merely add an independent background term to λ_{other}; it lowers c_h in the same states where policy errors happen. Interception coverage c_h must therefore be measured across the actual states where the policy fails rather than on an idealized set of synthetic test cases. The equation gives no credit for hazards the shield cannot sense, such as oil films or fatigue-weakened latches, and no credit for the shield at all until each of the four premises is supported. Until then the release must restrict operation instead.

Bounding the system hazard rate requires bounding the raw proposal rate $\lambda_{\text{proposal},h}$ from an adjudicated execution log before crediting shield attenuation. Under a stationary Poisson arrival model with zero observed hazardous proposals, this upper bound follows the rule of three (Hanley and Lippman-Hand 1983) from equation 7.4, with proposal operating hours in place of discrete trials. For an adjudicated log spanning T operating hours at significance level α_{sig}, the one-sided upper confidence bound is $-\ln\alpha_{\text{sig}}/T$, or roughly $3/T$ at 95 percent confidence. When the interception coverage c_h is also estimated empirically from zero-miss seeded trials rather than given as a known constant, the product $\lambda_{\text{proposal},h}(1-c_h)$ rests on two separate confidence bounds that can each fail. Guaranteeing that both bounds hold simultaneously at overall significance α_{sig} requires splitting the error budget across the continuous proposal-log test and the discrete coverage-trial test via the union bound, testing each factor at significance $\alpha_{\text{sig}}/2$:

Napkin Math 18.3: The coverage bill

Problem: In an illustrative scenario, a fleet keeps a fully adjudicated proposal log for one release cycle, judged by a validated counterfactual oracle, and the log records no hazardous proposal. If a named shield's interception coverage for that hazard class were independently established, would the modeled residual of equation 18.2 meet the system target, and what evidence would it take to show that it does?

Variables:

- System target $\lambda_{\text{target}} = 1 \times 10^{-9}\ \text{h}^{-1}$ (Illustrative)
- Proposal log $T = 100$ units $\times$ 90 days $\times$ 24 h $= 2.16 \times 10^5$ proposal-hours with zero hazardous proposals (Illustrative)
- Class-specific coverage $c_h = 0.9999$ and $\lambda_{\text{other}} = 0$ (Illustrative; the hardware and common-cause term is set to zero for this calculation)
- Confidence 95 percent, significance $\alpha_{\text{sig}} = 0.05$ (Chosen)

Math:

1. **Proposal bound** (rule of three with hours in place of trials): $-\ln\alpha_{\text{sig}}/T = 2.996 \,/\, 2.16 \times 10^5\ \text{h} = 1.387 \times 10^{-5}\ \text{h}^{-1}$.
2. **Residual**: $1.387 \times 10^{-5} \times 1 \times 10^{-4} + 0 = 1.387 \times 10^{-9}\ \text{h}^{-1}$, **above** the target. Meeting it at this bound needs $c_h \geq 1 - 1 \times 10^{-9} \,/\, 1.387 \times 10^{-5} = 0.9999279$.

3. **Proposal evidence at the assumed coverage**: The bound must fall to $1 \times 10^{-9} / 1 \times 10^{-4} = 1 \times 10^{-5}\ \mathrm{h}^{-1}$, which takes $2.996 / 1 \times 10^{-5} = 3.00 \times 10^{5}$ proposal-hours, or 124.8 days for the fleet, 34.8 days beyond the cycle.
4. **Coverage evidence**: Bounding c_h itself takes $\ln 0.05 / \ln 0.9999 = 29{,}956$ seeded hazardous proposals of that class with no miss, drawn from the states in which the policy errs.
5. **Joint confidence**: The probability that at least one of two bounds fails is at most the sum of their failure probabilities, so a joint 95 percent statement needs each bound at 97.5 percent (the Bonferroni split). The proposal bound becomes $3.689 / 2.16 \times 10^{5}\ \mathrm{h} = 1.708 \times 10^{-5}\ \mathrm{h}^{-1}$, the coverage campaign 36,887 trials, and the residual $1.708 \times 10^{-9}\ \mathrm{h}^{-1}$.

Result: The modeled residual misses the target even with the common-cause term set to zero. Supporting the claim takes both a proposal log longer than the release cycle and tens of thousands of seeded interceptions, and the confidence of the combined claim must be budgeted across them.

Systems insight: A shield changes the system claim only for hazard classes it can detect and intercept in time. Proposal frequency, conditional interception coverage, hardware faults, and common-cause failures need separate evidence. Oil or material failure outside the shield's sensing coverage stays in the residual-claims register.

The notebook prices what evidence the exposure model and the safety filter would need; it supplies none for either. Formal verification carries the same scoping requirement, since any mathematical proof applies only to the declared kinematic model, sensor observations, disturbance bounds, and timing guarantees. Whenever a physical premise lacks empirical proof, the release argument must substitute an operating restriction assigned to an accountable owner.

↩ The scoping limits of formal certificates to their declared plant, observations, and disturbances are established in section 15.7.3.

18.3.3 A residual-claims register

DRAFT

Refusing claim C3 in the safety case did not remove the floor-friction premise behind it. Without a durable record tying that open premise to an operating restriction, a later model update could restore normal aisle speed while the floor remains unmeasured. The **residual-claims register** is the sixteenth and closing record in the systems record chain. Taking in the upstream release manifest and approval record, the register binds every unresolved physical premise directly to an operating restriction.

↪ The sixteen-record chain, which the residual-claims register closes, is developed in section C.14.2.

Each register entry names four required elements: the missing physical observable, the active containment mechanism or operating restriction, the accountable owner, and a predeclared closure test. Together they give principle 4 direct operational force. Unlike a software defect log or issue tracker, the register is a machine-readable deployment record with a fixed schema. Its parameters set runtime enforcer bounds, and its standing conditions bind facility operations. Operating authority stays derated until test evidence passes the predeclared closure test. Writing a warning note in documentation, or deferring an open premise to a future release without lowering speed or narrowing the operational design domain, provides zero physical protection.

↪ **Byte layout:** Wire formats, data fields, and structural invariants for register entries are documented in section C.15.13.

Not every open premise is physical. Entries that the approval record of section 17.9 cannot close before release, such as a human role whose feasibility is still `UNKNOWN`, a decision waiting for an owner, or a group of people whose detection has not yet been measured, enter the register in the same form. Each carries the restriction that holds meanwhile, such as a lower speed wherever the unmeasured group works, an owner, and the test that would close it.

Engineers must state each **missing observable** in concrete physical quantities rather than modeling jargon. Calling a failure an "out-of-distribution latent state" does not say what physical sensor must measure it or when an enforcer must intervene. Naming an unmeasured friction coefficient or latch yield force, by contrast, makes the engineering choices concrete. The team can weigh adding a sensor, adding a passive mechanical limit, or narrowing the operational design domain, and everyone can see what test evidence is needed to retire the restriction.

The undetected oil film forms the first register entry for the mobile manipulator. Its premise is that every aisle floor supplies a friction coefficient of at least $\mu \geq 0.204$, the floor needed for credible deceleration. The missing observable is the local friction coefficient under a clear fluid film, which

neither onboard cameras nor lidars can tell apart from clean concrete. Because a stop sized for dry concrete overruns the available clearance on an oily floor, the normal speed limit is justified only on surfaces where friction is verified or actively controlled.

An open premise requires an immediate **operational response**. For warehouse aisles, release sign-off can authorize a derated speed of 1 m/s on designated routes maintained under a facility inspection procedure. In this restricted-service mode, scheduled floor scrubbing, spill isolation, and controlled access must maintain a friction coefficient of at least $\mu \geq 0.12$. That friction level gives an available deceleration of 1.18 m/s² and a known-component speed ceiling of 1.23 m/s. This is the inspected-floor ceiling that the release envelope carries (table 16.1), drawn as the dashed envelope in the stopping distance analysis of chapter 12 (figure 12.3). The restricted mode sits below that ceiling rather than at it, because the ceiling, like the budget it comes from, still leaves out Δ_{da} and any reserve for it. This inspected level is an illustrative baseline chosen below dry concrete to tolerate light surface contamination. A production deployment must set this value from friction surveys across the designated routes and show how spill prevention or bounded cleanup maintains it between inspections. In the illustrative project records, restricted-mode operation remains provisional until the full timing budget, including dispatch-to-admission latency and the required reserves, is verified under load. Routes outside the facility inspection program are excluded from automated navigation.

Restricted operation requires an accountable owner who can keep the underlying physical premise true. Facility operations owns this entry through its maintenance and spill-response procedures along pallet-jack routes, setting inspection intervals and specifying the site conditions that pause automated traffic. While the signed speed ceiling in the onboard permission path enforces the motion limit, a software speed setting alone cannot keep the floor dry.

Every register entry must also state a **closure test** before data collection begins. To detect surface films online, the closure test must qualify an independent physical sensor, since standard visual cameras and lidars cannot resolve clear fluids on concrete. Alternatively, higher-traction floor coatings or physical barriers separating leaky equipment could support a revised claim through facility evidence. One candidate closure campaign evaluates an optical film detector, requiring a miss rate of at most 0.1 percent demonstrated across 2,995 zero-failure trials on seeded fluid films, the trial count required by the zero-failure sizing of section 7.7.1. Until the sensor passes this test, the lower speed limit stays in force. Smaller test campaigns do not provide enough statistical confidence. Zero misses across 1,000 independent trials bounds the miss probability only at 0.002991 at 95 percent confidence. In addition, the detector must see far enough ahead that sensing latency, permission processing, and braking finish before the wheels reach the fluid. Forward range, inference latency, false-alarm rate, and post-detection friction assumptions must all be declared in advance. Reactive wheel-slip reflexes can limit further acceleration once the wheels spin, but they cannot guarantee stopping distance before the tires hit the film.

While the target miss rate of 0.1 percent is chosen illustratively here, a production deployment must derive it from the site's hazard budget. The detector works inside the conditional shield relation of equation 18.2, with fluid film encounters in place of hazardous action proposals. If films appear along transit routes at an arrival rate λ_{film} and facility engineering allocates a hazard budget λ_{alloc} to this failure mode, the detector must satisfy $\lambda_{\text{film}}\, p_{\text{miss}} \leq \lambda_{\text{alloc}}$. Choosing the risk allocation λ_{alloc} is an accountable engineering judgment, whereas estimating the encounter rate λ_{film} comes from maintenance and spill logs.

Aisle navigation adds a second register entry: moving people at blind corners. Stopping-distance budgets are sized against stationary obstacles, so a person who keeps walking toward the mobile base during the stop consumes clearance distance that the budget never set aside. As shown in section 12.3.1, accounting for a person walking toward the robot throughout the stop caps the permissible base speed at the rack corner far below normal transit speeds. The safety premise behind normal aisle speed assumes that no person walks out of the occluded cross aisle during an emergency stop, leaving a person's presence in the hidden cross aisle as an unobserved state. Restricted service therefore requires physical barriers or access controls that block human cross-traffic during automated runs; painted floor lines do not satisfy this condition. Facility operations owns this restriction. A proposed blind-corner presence sensor cannot earn credit in the safety case until its field of view, latency, and miss rate pass qualification under the same zero-failure statistical standard applied to the film detector.

↩ Clearance eroded by a person approaching from an occluded cross aisle at walking speed is established in section 9.7.

The automated picking station adds a third entry, covering what the arm grasps. Its premise is that the object the gripper picks up, and then hands to a coworker through the `accept_item` interface, is the object the task names. As shown in section 10.7, semantic grounding can fail silently. When a thin-walled red glass jar sits alone where the red ceramic mug was expected, the grounding model mistakes the jar for the mug and still satisfies every geometric check in the planner and permission layers. The missing observables are object identity and crush strength, which neither wrist cameras nor visual foundation models can verify reliably before contact.

↩ The `accept_item` handover entry, which grants no motion authority and caps the arm's approach to the coworker's hand, is established in section 14.2.1.

Containing this hazard takes a two-part operating restriction. First, facility operations restricts conveyor loading to verified items from a declared stock-keeping unit (SKU) list that excludes fragile look-alikes. Second, the pick station admits a grasp only when optical singulation confirms a single object in the pick window; multiple objects abort the automated intent and route the tote to manual picking. Singulation removes geometric confusion among permitted items, but it does not measure what an object is made of; an off-list glass jar arriving alone is kept out only by upstream loading controls. Lowering the grip-force limit enough to protect fragile items would require measured crush limits for every item in inventory, which is itself an unverified physical assumption. Facility operations and the application engineering team co-own the item catalog and singulation rules. The closure test requires qualifying an automated check at the station, such as a radio-frequency identification (RFID) reader or a combined scale-and-barcode check, with a false-accept rate of at most 0.1 percent at 95 percent confidence across 2,995 failure-free trials with seeded look-alikes. Table 18.2 compares these three entries side by side, listing the four required components for each open premise.

Table 18.2: Residual-claims register for the aisle and the pick station: Proposed responses to three open premises behind the mobile manipulator's aisle speed and its pick; operating credit requires the supporting evidence stipulated in the restricted-service branch. For each, the register names the missing physical observable, the hazard it permits, the operating restriction and its accountable owner, and the predeclared closure test that would retire the restriction.

Claim and missing state	Hazard mechanism	Active containment and owner	Predeclared closure test
Braking traction: every aisle floor supplies $\mu \geq 0.204$; friction beneath a clear film is unobserved.	At $\mu = 0.05$ the available deceleration falls to 0.49 m/s^2, and the known-component budget at 1.3 m/s already grows to 2.13 m against a 1.10 m clear distance.	Candidate in the actual records: 1 m/s on qualified, controlled floors ($\mu \geq 0.12$); full timing and reserve must support it. Uncontrolled routes excluded. *Owner*: site operations, through the spill procedure for pallet-jack routes.	A floor-film detector with a miss rate of at most 0.1% at 95% confidence, from 2,995 failure-free seeded-film trials, plus timing and clearance tests; the restriction holds until it passes.
Walking person: no person approaches the rack end during a stop; presence in the hidden cross aisle is unobserved.	A person walking at 1.6 m/s through the whole stop would cap the base far below its aisle speed at the rack end (section 12.3.1).	Controlled separation excluding continuing human approach during a stop; no credit for an unqualified presence sensor. *Owner*: site operations.	Qualify coverage, detection timing, and the complete response; 2,995 failure-free trials support a 0.1% miss bound at 95% confidence for the declared trial population.
Object identity: the object grasped is the object the task names; identity and fragility before contact are unobserved.	A confident grounding to a thin-walled glass jar in place of the absent mug passes every geometric check, and a grip the ceramic mug needs can crush the glass.	Feed control admits to the conveyor only items on the station's declared list, none of them a fragile look-alike; singulation admits a grasp only with one object in the pick window. An off-list item arriving alone is contained only by the feed procedure. *Owner*: site operations, jointly with the task profile.	A qualified identity check at the station (tag read or scale-and-code) with a false-accept rate of at most 0.1% at 95% confidence, from 2,995 failure-free seeded look-alike trials; the restriction holds until it passes.

The register also holds a fourth entry, a timing premise, which the table leaves out because its hazard is the same overrun as the first row's and only its missing evidence differs. Measured capture-to-dispatch latency excludes the downstream dispatch-to-admission interval. The placement allocation budgets that interval at 63.3 ms out of the 100.8 ms timing allowance at the aisle speed, leaving 37.5 ms of reserve. Stated in the register's four elements, the missing observable is the loaded worst-case dispatch-to-admission interval, which steady-state inference benchmarks do not bound.

Because that interval remains unverified, the runtime systems team owns an active restriction that holds the aisle speed as an uncertified candidate rather than an authorized mode. The closure test is supported timing and response analysis under declared workload and resource bounds, challenged by loaded and adversarial measurements of both latency intervals. Until that test shows that the interval fits its allocation with positive reserve and meets the lease duration and renewal requirements of the execution contracts, the restriction stays in force. The stipulated service branch assumes that timing evidence for its derated mode; in the baseline records the entry remains open.

18.3.3.1 Maintaining the restrictions

Writing an open assumption into a machine-readable register file does not turn it into a measured physical state. If the signed speed limits and site procedures are not wired into the release gate and daily floor work, the restriction exists only on paper. Operating restrictions in the register must therefore attach directly to the runtime enforcers and site workflows that govern daily operation. Cryptographically signed speed envelopes constrain the onboard permission path, while scheduled maintenance intervals bind facility operations. Release pipelines check that both controls are active in the authorized configuration, while runtime monitors enforce only the physical bounds that onboard sensors can read.

Model retraining or software updates must inherit every open restriction whose closure test has not yet passed. Release gates must check the signed operating limits and the status of their supporting evidence before authorizing a deployment. Dropping to a restricted mode requires its own validated safety case; an open register entry does not automatically clear a slower mode for service. At runtime, low-level controllers enforce speed ceilings and actuator limits, while facility crews maintain floor traction and conveyor loading rules at the scheduled intervals.

A restriction can also lapse in the middle of a motion, when a signed envelope expires, an inspection interval runs out while the base is in a route, or a deployment gate withdraws a mode. The lapse must hand over to the fallback ladder rather than to a generic stop. A trajectory chunk already executing inside the old restriction may finish only if its lease, state estimate, envelope, and supporting physical premises remain valid through completion. Withdrawing permission for new proposals can preserve an already admitted chunk under those conditions; loss of a premise that supports its execution cannot. Losing active permission altogether demands a physically feasible stop within the remaining budget. The rung must match the state, because controlled deceleration, holding brakes at zero velocity, torque removal, and full power isolation produce different transients. No register entry can replace those physical distinctions with a single emergency-stop primitive.

↩ The ladder of state-matched fallback responses, from minimal projection to complete torque removal, is established in section 12.9.

New sensors can justify wider operating envelopes only after they pass empirical qualification. Where real-time detection is impractical, the register must rely on a conservative action, passive containment, periodic proof testing, or a restricted operating domain (the responses to a hidden state that section 18.3.4.2 develops). Choosing among those responses requires comparing the distance at which a detector could spot the hazard against the deadline for committed motion.

18.3.4 Failures without detectors

An automated floor-film detector can retire the surface-friction restriction only if its coverage, false-alarm rate, and detection latency fit inside the route's stopping budget rather than a standalone model benchmark. In the known-component budget, running at the candidate aisle speed leaves 131.0 mm of physical margin, which corresponds to an allowable delay of 100.8 ms. Setting aside 63.3 ms for dispatch-to-admission processing leaves 37.5 ms of timing reserve. That reserve is an upper bound on unallocated latency, not a certified budget for new perception pipelines, and it offers no protection against a pedestrian who keeps walking toward the robot during the stop.

A forward-looking fluid detector faces an even tighter spatial deadline. The required detection range, minus the distance the robot travels during sensor integration and neural inference, must cover the full dry-floor stopping distance before the tires reach the leading edge of the film:

$$d_{\text{detect}} \geq v\, t_{\text{detect}} + d_{\text{budget}}(v), \tag{18.3}$$

where $d_{\text{budget}}(v)$ denotes the known-component dry-floor budget of notebook 18.2, the seven rows of table 12.1. The detection interval t_{detect} covers only the film detector's own work, from the moment the film enters its range to the moment it reports. That report is the observation that starts the

pre-brake chain, so the observation age and admission already inside d_{budget} follow it rather than overlap it. Where the two do overlap in a real pipeline, adding them is conservative. With a chosen sensing and inference latency of 100 ms at the 1.3 m/s aisle speed, the detector must see the film at least 1.10 m ahead. That range is almost exactly the 1.10 m clear distance, and not by coincidence. The dry budget and the spare together make up the clear distance, and the detector's 100 ms of travel nearly equals the 100.8 ms that the spare buys. A detector that sees only as far as the clear distance therefore spends the whole timing allowance by itself, leaving nothing for the 63.3 ms dispatch-to-admission allocation. To fit beside that allocation, its latency must stay under the 37.5 ms reserve, or its range must grow. If classification occurs only after the drive wheels enter the fluid, the stopping distance expands to 2.13 m, far beyond the 1.10 m clear distance. A classification received after the wheels cross onto the film can serve incident analysis, but it arrives too late to avert the collision.

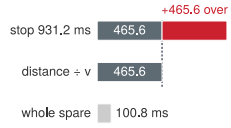
18.3.4.1 Observational indistinguishability and the action deadline

When an oil film and dry concrete produce identical camera and lidar readings until the robot is already inside its dry-floor stopping distance, no classifier running on those sensors can trigger a stop in time. That limit is observational indistinguishability. Consider two physical world states, s_A and s_B. In state s_A, a commanded actuator trajectory $\mathbf{u}(t)$ runs safely. In state s_B, that same command $\mathbf{u}(t)$ causes structural damage or human injury, requiring the permission path to trigger an emergency stop. The constraint governing this stop is distance rather than elapsed time: because braking changes the robot's speed along its path, the point of harm is a place in the room. Let d_{rem} denote the remaining distance to that boundary when the distinguishing observation arrives. An emergency stop finishes short of the hazard only if:

$$d_{\text{rem}} \geq v\,(T_{\text{comp}} + T_{\text{act}}) + d_{\text{brake}}(v), \tag{18.4}$$

matching the physical stopping relation used throughout this book. The enforcer's compute time and the actuator's response time together act as the pre-brake delay τ_{delay}, counted from the arrival of the distinguishing observation rather than from a fixed capture event, and $d_{\text{brake}}(v)$ is the braking distance of the chosen stop. Dividing this distance constraint by the approach speed gives an equivalent observation deadline: $t_{\text{deadline}} = t_{\text{harm}} - T_{\text{comp}} - T_{\text{act}} - d_{\text{brake}}(v)/v$, where t_{harm} is the instant the unbraked trajectory would reach impact. Because t_{harm} marks when the robot would reach the hazard at constant cruising speed v, the quantity $t_{\text{harm}} - t_{\text{deadline}}$ expresses the required spatial clearance d_{rem} in units of cruising-speed travel time. Once braking begins, the decelerating robot takes longer than $d_{\text{brake}}(v)/v$ to cover the remaining distance, so it safely comes to rest just short of the spatial boundary at a clock instant later than t_{harm}. A common mistake subtracts the full stopping duration T_{stop} instead of the speed-normalized distance d_{brake}/v, treating the robot as if it kept moving at full cruising speed until the instant it came to rest. For the jerk-limited profile from 1.3 m/s, braking lasts 931.2 ms but covers a distance equal to only 465.6 ms of constant-speed travel. Writing the deadline purely in time over-reserves 465.6 ms, or 605.3 mm of clearance, an error larger than the entire 131.0 mm spare. The time form errs on the safe side, but it refuses operating modes that the distance form rightly admits. Manipulator contact follows the same logic: once an end-effector touches a stiff surface, contact force rather than position sets the observation deadline (section 12.7.2).

The paired worlds in figure 18.3 separate physical state from observable history. Their common brake command has different consequences, although the depicted channel supplies the same inputs through the action deadline.



Charging the full stop time over-reserves several times the entire spare.

A timely separating observation changes this argument only if the machine can obtain it within the same physical budget. When no such observation is available, the permission path must choose an action qualified for both candidate states or withhold the operating mode.

If the sensor stream $\mathbf{y}(t)$ produced under state s_A matches that under s_B up to t_{deadline} for every admissible control input, no algorithm acting on those measurements can tell the safe state apart from the hazardous state in time to prevent harm:

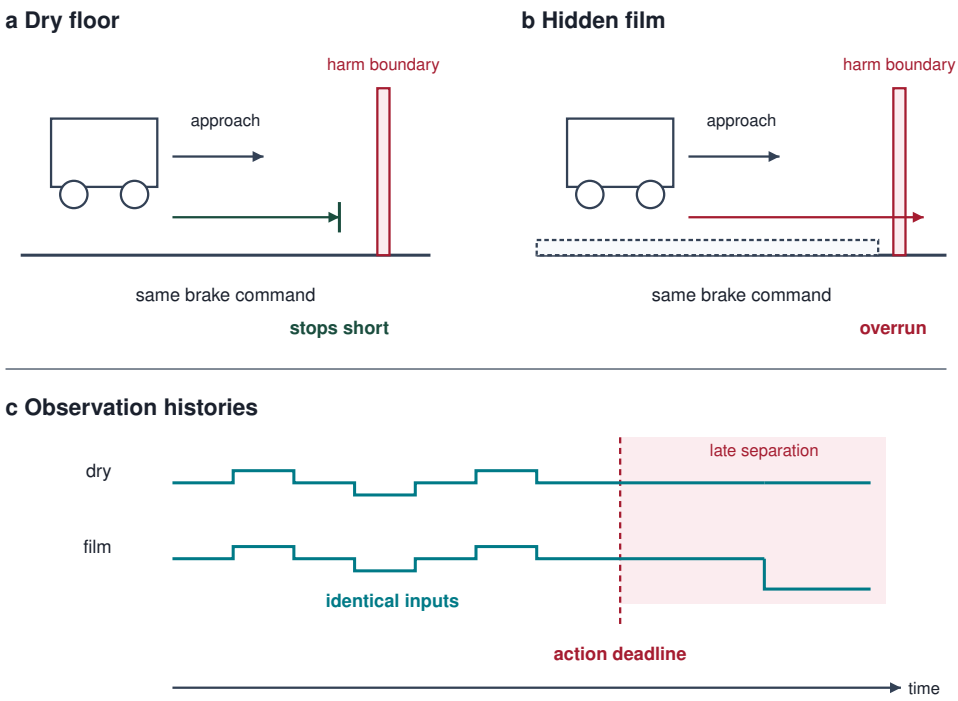


Figure 18.3: Different Worlds, Identical Timely Evidence: The dry-floor and hidden-film cases share an approach and brake command but reach different stopping positions. Their schematic observation traces remain identical until after the deadline set by equation 18.4. A later distinction cannot justify a state-specific response that had to begin earlier. The traces illustrate the stated observation policy; they do not prove that every admissible probe or additional sensor would also fail.

Definition 18.1: Observational indistinguishability

Observational indistinguishability is the condition in which two physical states that require different responses produce the same observation history, under every input the machine may admissibly apply, until the deadline by which a state-specific response must begin.

1. **Significance**: It turns an observation limit into a stopping-budget violation. The deadline is set by equation 18.4, so if the film and the dry floor look identical until the remaining distance falls below $v(T_{\text{comp}} + T_{\text{act}}) + d_{\text{brake}}(v)$, that evidence cannot justify continuing the dry-floor mode. The machine must instead use a response already qualified for both states or withhold that mode.
2. **Distinction**: Noise and weak detectors obscure information that more data or a better model can recover; indistinguishability means the information is absent from the channel. Unlike unobservability (definition 9.3), which asks whether a state can ever be reconstructed from the output history, it is tied to a deadline and to the inputs the machine may apply.
3. **Common pitfall**: A frequent misconception is that a more capable classifier on the same channel will eventually separate the states. Identical histories give every classifier identical inputs. A designed probe can separate states hidden under the current observation policy only when an admissible excitation reveals a difference before the deadline. If every admissible input leaves the histories identical, the response requires a new channel or an action valid in both states.

↩ The observation contract, which bounds the age and error of what a channel reports but cannot certify a state the channel never carries, is established in section 8.8.

The qualification *under every input the machine may admissibly apply* matters because whether two states look different often depends on how the machine excites the plant, as shown in nonlinear observability theory (Hermann and Krener 1977). In some settings, an engineer can inject a small, safe control test input that makes the two states produce different sensor readings before the deadline. This technique is **active probing**: applying a bounded test excitation to expose hidden state differences in the sensor stream before the action deadline, the same excitation that section 9.8 uses to expose a drifting force-sensor bias. Active probing draws on auxiliary signal design[5] from fault detection (Campbell and Nikoukhah 2004). For example, a controlled tug at a qualified proof load can check whether a door latch holds at that load before the arm commits to its full pull, though it does not measure the latch's remaining yield margin. On the floor, a brief torque pulse at low speed, applied before the base enters a route, makes the wheels slip measurably on a film and not on dry concrete, turning the friction under the tires into telemetry. Such a slip check measures only the patch of floor the wheels already stand on; it cannot see a film ahead, which is why the forward detector above has its own range condition. Active probing costs time and disturbs the motion, and both costs must fit inside the safety budgets. Where no admissible probe can separate the states within the timing budget, the design must fall back on conservative actions or passive mechanical containment.

5 **Auxiliary Signal Design**: The multimodel method computes a small test input that guarantees the normal model and each declared failure model produce different outputs within a fixed test horizon, despite bounded disturbances (Campbell and Nikoukhah 2004). A probe can therefore rule out only the failures someone wrote down as models, which is why the hidden premise must first be named in the register. The input bound and the horizon map onto this chapter's budgets, since the probe must fit the force envelope and the separation must arrive before t_{deadline}.

18.3.4.2 Responses when the state remains hidden

Active probing is the first of five design responses when passive runtime observations leave candidate physical states indistinguishable. When an admissible excitation can separate the states before the deadline, *active probing* supplies the missing observation online; if every admissible input leaves them indistinguishable, the remaining four responses must carry the claim without online separation. *Proof testing* measures the state from outside the runtime channel at scheduled intervals, such as an unloaded tare of a sensor or a load test of a latch, and covers the time between tests by argument. A *conservative action* chooses a command safe in both candidate states, such as a speed low enough for the film. When control commands alone cannot bound the interaction, *passive containment* limits the physical consequence mechanically, such as a compliant element that caps contact force whatever the software believes. A *restricted operational domain* withholds authority altogether from states that cannot be told apart, such as routes without a spill procedure. The register entries above already used the conservative action and the restricted domain, and active probing was shown at the latch and the floor. The wrist sensor and the latch below show proof testing and passive containment at work on the running machine.

Sensor drift is the slow form of the same barrier. The wrist strain gauge of section 9.8, whose bias wanders as thermal cycles and mechanical stress relaxation alter its bridge resistance, reports only the sum of contact force and bias. A drifting bias and a real change in force therefore produce the same reading and a zero innovation. What this chapter adds is the choice of response. Every contact threshold that depends on that signal inherits the drift, and no response chosen at runtime can see it. The response must therefore be proof testing, meaning an unloaded tare or a proof-test fixture at an interval short enough that the drift accumulated between tests stays inside the threshold's margin.

↩ Mechanical degradation and thermal calibration drift as continuing obligations of a release verdict are established in section 16.5.1.

A weakened door latch poses the same observability barrier on a millisecond timescale. Before the metal yields, joint torque sensors read the same reaction force against a fatigued latch as against a sound one, so the torque channel carries no information about how much yield margin remains. Because plastic yield is permanent, no software command sent after the latch bends can undo the damage, and running a faster classifier on joint torque signals adds zero protection. A credible safety strategy must bring in information from outside the runtime torque channel. Scheduled proof tests[6] check that the latch survives a specified proof load at fixed intervals, without measuring its ultimate yield strength. Even so, proof testing holds only if maintenance records show that fatigue, wear, and corrosion cannot push the latch's yield strength below the arm's contact ceiling between tests.

6 **Proof Test**: Functional safety defines a proof test as a periodic test that reveals dangerous failures no online diagnostic can detect, so that the function can be restored to its as-new condition (International Electrotechnical Commission 2010a). A hidden failure that arrives at a constant rate λ is present, on average over a test interval T, for a fraction of about $\lambda T/2$ of the time when λT is small. The interval is therefore itself a safety parameter, because it bounds how long a weakened latch can go unseen.

Adding a passive compliant element to the structure can mechanically cap interaction forces across a qualified deflection stroke, the principle behind series elastic actuators (Pratt and Williamson 1995) and mechanical torque limiters. Two physical conditions govern compliant load paths, both set by the stroke the arm *actively drives* after contact detection rather than the total uncompressed spring length. When a contact tripwire F_{trip} triggers, the arm keeps moving during its controller response latency t_{resp} before decelerating at rate a_{arm}, driving the mechanism through a post-contact stroke of

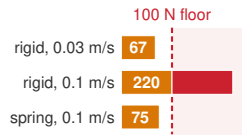
$\Delta x_{\text{driven}} = v\,t_{\text{resp}} + v^2/(2a_{\text{arm}})$. To keep the contact force below the declared strength floor F_{floor} (a lower bound on the latch's strength, unrelated to the floor friction), the equivalent stiffness k_{eq} of the combined series load path (compliant element and strike plate) must satisfy equation 18.5:

$$F_{\text{trip}} + k_{\text{eq}}\,\Delta x_{\text{driven}} \le F_{\text{floor}} \quad \Longleftrightarrow \quad k_{\text{eq}} \le \frac{F_{\text{floor}} - F_{\text{trip}}}{v\,t_{\text{resp}} + v^2/(2a_{\text{arm}})}. \tag{18.5}$$

In addition, the compliant element's allowable stroke must exceed its deflection under the peak allowable force, $(F_{\text{floor}} - F_{\text{preload}})/k_{\text{spring}}$, so the spring never bottoms out against a hard stop. The threshold F_{floor} is not the latch's actual yield strength, which remains unobserved between proof tests; it is a chosen lower strength bound recorded as an assumption in the register until physical evidence supports it. Accountable judgment chooses the design bound; it cannot create material strength.

The warehouse door latch shows what passive compliance buys. During a guarded approach at 0.03 m/s, meeting a stiff strike plate of stiffness 4.0×10^5 N/m satisfies equation 18.5 with a required arm deceleration of 2.95 m/s², matching the guarded latch stop derived in section 12.7.2. At a faster approach speed of 0.10 m/s, however, the same stiff contact would require an impossible arm deceleration of 400 m/s², forcing the permission path to refuse the trajectory. Holding arm deceleration to the guarded stopping bound instead drives the arm through 1.89 mm after it crosses the 15 N tripwire, capping the allowable series stiffness at 44.9 kN/m. Because series compliance combines inversely ($1/k_{\text{eq}} = 1/k_{\text{spring}} + 1/k_{\text{latch}}$), pairing the compliant element with the strike plate requires a spring stiffness of at most 50.5 kN/m and an available stroke of at least 2.0 mm to stay within the 100 N design force.

This kinematic calculation assumes a stiff position-tracking drive that enforces the commanded a_{arm} deceleration profile throughout the post-tripwire stroke regardless of contact reaction. Once spring compression exceeds $m_{\text{eff}}\,a_{\text{arm}}$, the passive spring alone would decelerate the m_{eff} effective mass faster than a_{arm}, so a controller that tracks the gentler kinematic ramp keeps pushing forward into the spring until the stroke completes. If the safety response instead cuts motor torque completely, the arm's effective mass 12 kg coasts into the spring, and energy conservation sets the peak force. A spring compressed to force F stores potential energy $F^2/(2k_{\text{eq}})$; at the tripwire, the spring already holds $F_{\text{trip}}^2/(2k_{\text{eq}})$, and the coasting mass adds its kinetic energy $m_{\text{eff}}v^2/2$. The resulting peak impact force is $\sqrt{F_{\text{trip}}^2 + k_{\text{eq}}\,m_{\text{eff}}\,v^2}$, the dynamic impact regime that governs high-speed robot collisions (Haddadin et al. 2009). With the series spring installed, this coasting peak is 74.9 N, below the 100 N that the braked case reaches. That result is not a paradox. The stiffness bound was sized for a drive that holds the arm to the gentle 2.95 m/s² stop and so keeps pushing through the whole 1.89 mm stroke, bringing the force right to the floor. With torque removed, nothing pushes; the spring alone slows the 12 kg effective mass, harder than that commanded stop, and halts it in a shorter stroke at a lower force. Striking the stiff plate without compliance, by contrast, produces peak forces of 219.6 N at the faster approach speed and 67.4 N under the guarded approach. Within the qualified approach-speed and stroke envelope, the compliant element keeps both modeled responses at or below the latch's declared design strength; driving beyond that stroke would bottom out the spring and break the bound. Even so, passive compliance does not verify latch strength: qualification must still test damping, material fatigue, and structural load paths rather than spring stiffness alone.



A series spring keeps the coasting peak under the declared 100 N floor.

The door latch, the oil film, and the packaging tab pose the same problem. When a safety premise cannot be evaluated before an execution deadline, the first question is whether an admissible probe can separate the states in time. If it can, active probing supplies the missing observation; otherwise proof testing, a conservative action, passive containment, or a restricted operational domain must support the claim. The choice among them depends on the claim each response can support and the evidence needed to maintain it, and every such mitigation must also state the evidence that would retire it.

Checkpoint 18.2: Premises no sensor reports

The warehouse machine's hardest limits are premises its own telemetry cannot check, and each one needs an entry, a restriction, and a response chosen before the deadline.

- ☐ **Friction estimate**: A friction survey measures an aisle at $\mu = 0.15$. Given the 2 m/s^2 braking bound, compute the available deceleration from equation 18.1 and determine whether the brake or the floor binds.
- ☐ **Register entry**: For the weakened door latch, state the four components of its register entry: the missing observable, the restriction, the owner, and the closure test.
- ☐ **Shield credit**: Explain why equation 18.2 gives no credit against the oil film, and identify which term the film would enter.
- ☐ **Hidden-state responses**: For a state hidden under the current observation policy, identify the five possible engineering responses, determine when active probing is available, and classify the series spring at the latch.

18.4 Choosing the Next Capability

Pouring engineering effort into a faster proposer when an unmeasured floor or an unseated tab actually limits the cell leaves the operating restriction in place. In a deployed packaging cell, restricted transit speeds and human-supervised sign-offs call for different investments. Speeding up proposal generation improves throughput inside the current safety envelope, whereas qualifying a new contact or engagement sensor can retire the need for human sign-off altogether. Each advance requires two deliverables: a predeclared task metric and a physical test of the operating premise it changes. Tuning execution inside an existing envelope and gathering the evidence to widen that envelope are distinct engineering programs, and the register's open entries decide which one the next capability needs.

18.4.1 Match the experiment to the missing evidence

Treating every open entry in the residual-claims register by training another neural detector fails whenever the missing evidence is a field evaluation, a mathematical bound, or a fallback that the body cannot execute. The open entries do not all call for the same kind of work; each falls into one of four gap classes, and the class, not the entry's urgency, decides which experiment can close it:

- *Unmeasured observables* require dedicated sensors that can tell the relevant physical states apart before the action deadline passes.
- *Evaluation deficits* require randomized field trials or statistical estimators that represent the actual operating population while accounting for correlated samples and rare tail events.
- *Theoretical deficits* require bounded analytical arguments whose physical premises can be proved mathematically or tested on the actual hardware plant.
- *Scope mismatches* require changing the system architecture or narrowing the operational design domain.

Naming the exact gap class directs the next experiment to the evidence a register entry actually lacks, turning the open premises in the claims register into distinct work programs. Undetected floor films, hidden cross-aisle pedestrians, ambiguous object identities, and unseated packaging tabs are unmeasured observables. Each calls for either a qualified sensor channel with a tested miss bound or an operating restriction that avoids the hazard, such as the downstream manual inspection that stands in for the tab sensor. Loaded dispatch-to-admission timing requires a supported timing bound under declared workload and resource limits, challenged by loaded and adversarial latency measurements. By contrast, the coverage factor c_h for a safety shield requires statistical qualification on the declared population of hazardous proposals, whereas finite stress tests alone cannot establish a worst-case timing bound. Gaps between formal plant models and physical hardware are theoretical deficits, requiring robust control bounds. A scope mismatch differs from all three because no measurement, trial, or proof inside the current architecture can fix it. Claiming a de-energized safe

stop for a balancing biped is the clearest example: cutting torque is itself the hazard, so the remedy is a different fallback architecture or a narrower claim, as section 18.4.3 discusses.

Statistical extreme value theory bounds rare tail exceedances only when its asymptotic and independence assumptions hold; fitting a parametric curve to test data cannot validate unmodeled contact physics or reveal unobserved floor friction. Similarly, Hamilton-Jacobi reachability analysis yields invariant sets only for an explicitly declared plant and disturbance bound. Moving an unverified friction assumption from field testing into a rigid-body simulator or an analytical proof leaves it just as unverified as before.

↪ Extreme value theory and its asymptotic and independence assumptions are developed in section A.10.

↪ Hamilton-Jacobi reachability and its declared disturbance model are developed in section B.6.3.

18.4.2 What a restriction costs

Knowing which experiment would close an entry does not say whether running it is worth the cost. An entry in the residual-claims register can buy protection at a throughput cost. On the warehouse machine, the oil-film restriction lowers cruising speed along the aisle from 1.3 m/s to 1 m/s, imposing a 23.1 percent speed penalty. At steady cruise, this derating stretches transit time across each 30 m aisle segment from 23.1 s to 30 s. That single constraint adds 30 percent more transit time per leg before accounting for acceleration profiles and docking maneuvers. Because transit time scales as $1/v$, the percentage increase in leg time exceeds the percentage cut in speed.

An operational restriction remains in force until its closure test passes. That test carries a tangible empirical cost: the 2,995 seeded-film physical trials specified in the register, for example. The separate claim about safety shield coverage requires 29,956 seeded hazardous proposals; that campaign does not close the film entry. The claims register therefore confronts systems engineers with an economic optimization problem alongside the technical one. On throughput savings alone, a formal closure campaign pays back when the cumulative value recovered exceeds the combined cost of the test campaign and the sensing instrument it qualifies. A full investment decision must also account for changes in exposure, maintenance, and the value of newly supported tasks. An operational restriction can remain the economical choice when its physical premise can be maintained reliably at low cost, such as through qualified cleaning and spill-control procedures along a few pallet-jack routes. Evaluating this trade-off requires site-specific parameters: the economic value of a completed task, the machine's active duty hours on restricted routes, and the marginal expense of executing controlled test trials. In the absence of proprietary site economics, time provides the universal benchmark that transfers directly across facilities.

The same operational accounting governs fleet deployments. When commercial logistics facilities contract bipedal humanoids under robots-as-a-service agreements (GXO Logistics 2024), commercial performance is measured by volume moved, 100,000 totes at an initial customer facility by late 2025 (Agility Robotics 2025). In these operational environments, physical interventions, speed restrictions, and withheld autonomy modes can reduce completed-task throughput; their billing effect depends on the service contract. The gating criteria established during release validation translate directly into ongoing operating expenses.

Intervention rates incur a double cost. Frequent human takeovers consume costly labor hours and tether fleet operations to local technician availability. Conversely, rare interventions introduce the classic ironies of automation: an operator who seldom intervenes struggles to maintain the situational awareness and manual dexterity needed when an emergency takeover occurs (Bainbridge 1983). A defensible business case therefore reports proposer autonomy, whole-machine task success, and protective responses separately, as the first fallacy in section 18.5 insists, because each carries a different price.

↩ The labor a deployment moves onto technicians, sweep crews, and remote assistants, and the uses of their data it bars, are established in section 17.8.

18.4.3 Research that changes an engineering result

The oil-film campaign can be priced because the closure test itself is already known: run 2,995 paired braking trials on a representative spill or qualify an online probe. Other open entries in a register cannot be closed by scheduling a standard test campaign because no engineering method yet supplies the missing evidence. In physical AI, research warrants industry and academic investment when its resolution improves useful task performance within justified limits or supplies support for a broader operating claim. Six central challenges define this research frontier as of 2026 (table 18.3); each stresses one of the four laws, falls into one gap class of section 18.4.1, and names the form of evidence that would count as progress.

Generalist and cross-embodiment policies stress the fourth law, permission must be earned, because evidence gathered on one body and one model version must now earn permission for another. A single pretrained vision-language-action (VLA) foundation model can now control diverse kinematics. For example, flow-matching architectures have been trained across single-arm, dual-arm, and mobile manipulation platforms from a unified model (Black et al. 2024), using datasets from different embodiments. Modern pipelines continuously fine-tune from production deployments via reinforcement learning with human interventions (Physical Intelligence et al. 2025) or remap learned trajectories across distinct kinematic morphologies (Gemini Robotics Team 2025a).

↩ Why demonstrations logged on one embodiment carry its dynamics and do not transfer unchanged to another is established in section 5.9.

However, empirical safety evidence compiled on a dual-arm cobot cannot be assumed to hold when that same policy executes on a mobile manipulator. Furthermore, policies that continually adapt in production require reopening the release claims whose supporting evidence depends on the changed behavior (section 16.9). The enduring challenge is establishing formal transfer guarantees across both physical embodiments and continuous model updates. Concrete progress requires rigorous transfer arguments with explicit kinematic and dynamic equivalence conditions, verifiable by predicting a new morphology's closed-loop failure distributions before physical deployment. The cross-embodiment approach also reinforces the necessity of an independent permission pipeline: physical protection limits are calibrated to each specific mechanical chassis and remain invariant regardless of how frequently upstream generalist weights are updated.

Data scaling is the empirical frontier of the second law, endogenous data. Large-scale empirical studies encompassing more than 40,000 real-robot teleoperated demonstrations indicate that out-of-distribution generalization across novel environments and visual scenes follows an approximate power law governed by environmental diversity, whereas collecting redundant demonstrations within identical scenes shows sharp diminishing returns (Lin et al. 2025). In physical systems, structural diversity rather than raw sample count dictates data utility.

The open challenge is determining whether these empirical scaling laws generalize to contact-rich manipulation, long-horizon multi-step planning, and safety-critical edge states. In embodied settings, an agent's closed-loop control actions actively determine its subsequent perceptual distributions, concentrating risk in rare, unmodeled corner cases. Meaningful progress requires publishing verified scaling curves that measure tail-risk closed-loop failure rates within declared operational design domains (ODDs), replacing benchmark evaluations that measure only average-case task completion rates.

↩ Why the diversity of pooled demonstrations, not their raw count, governs their value to a target task is established in section 5.9.3.

Generative world models also stress the fourth law, because they offer a way around the physical exposure wall only if their rollouts count as evidence. Latent dynamics formulations and high-fidelity video generation models increasingly act as general-purpose physics simulators (NVIDIA, Agarwal, et al. 2025; Assran et al. 2025). Researchers have begun deploying these generative models directly as policy evaluators, demonstrating strong correlation with real-robot task execution on benchmark tasks (Jeon et al. 2026). Synthetic rollouts execute at large parallel scale for a fraction of the cost of physical hardware hours.

Nevertheless, strict evidentiary limits constrain synthetic evaluation. A learned world model may estimate latent physical variables, but its training fit does not validate those estimates in unobserved regimes or justify extrapolating friction discontinuities or structural compliance omitted from its training distribution. The central research question is establishing when synthetic evaluation in a learned world model constitutes valid evidentiary support for real-world release claims. Genuine progress requires formalizing proxy-transfer bounds (section 7.4.2) validated against the empirical tail of real-world physical failures rather than aggregate nominal success, accompanied by end-to-end dataset and architecture provenance tracking.

↩ Provenance records for learned world models, binding architecture, checkpoint, and dataset lineage, are established in section 6.4.3.

Dynamically stable machines, such as bipedal humanoids and high-speed quadrupeds, are the most stringent test of the first law, irreversibility. As highlighted in perspective 4.2, perspective 7.2, perspective 13.1, and perspective 16.1, the entire architectural chain of proposal, permission, and admission applies directly to balancing robots, with one catastrophic exception: fallback safe states. For the wheeled platform or stationary manipulator considered here, the fallback first reaches a qualified, mechanically restrained standstill before removing drive torque. On a balancing biped in mid-stride, cutting motor power induces an unrecoverable ballistic fall, transforming the safety mechanism into the primary hazard. Instead of bringing the system to an immediate stop, a balancing machine must execute dynamic recovery maneuvers: taking a stabilizing step,

initiating a controlled crouch, or executing an active powered descent to a mechanically supported resting posture. Traditional functional safety standards, however, were built around fail-passive de-energization (the torque-removal rungs of the fallback ladder, which section 12.9.1 treats as fail-safe only for a plant already at a mechanically restrained standstill). This created what safety researchers term the *fail-passive gap*[7] (Ding et al. 2026). Bridging this divide requires certifying active closed-loop balance stabilization as a safety-rated critical function. Demonstrating meaningful progress requires designing certifiable dynamic reaction chains, analogous to automotive emergency braking profiles. These chains must reliably steer the robot from each qualified gait phase, under the declared fault conditions, into a stable, supported configuration within bounded peak power and kinetic energy budgets. The most capable learned machines are thus the ones for which a safe stop is hardest to define.

Contact-rich, dexterous manipulation centers on physical AI's canonical unobserved state: high-frequency contact interaction. Determining whether a snap-fit latch yields, a packaging tab seats, or a fragile container begins to crush cannot be reliably deduced from joint motor currents or external RGB-D cameras alone. High-resolution tactile sensing provides a direct channel for several of these contact dynamics; each claimed observation still needs qualification for its materials, geometry, and deadline. Modern vision-based elastomeric tactile sensors directly capture surface deformation fields, contact geometry, and distributed shear stresses at the end-effector (Yuan et al. 2017).

Closing the loop on contact, however, challenges the third law, proposal is not permission. Contact forces escalate within milliseconds of initial impact, far outstripping the round-trip latency of high-level neural planners and placing strict burdens on low-level passive or series elastic compliance (section 18.3.4.2). The open engineering challenge is integrating tactile feedback into the formal permission path. Concrete progress requires qualifying tactile perception channels with rigorous statistical miss rates for incipient slip and shear yield, coupled with low-latency impedance controllers whose force limits can be deterministically verified and bounded by safety hardware (Tedrake 2023).

Certifying learned models that directly implement safety functions is the central challenge of the fourth law. Although such components are assigned the notified-body route of section 16.3.4, the robotics industry lacks standardized methodologies to certify deep neural networks at automotive or industrial safety integrity levels (SIL) or PL, because current verification techniques face stark limitations. Exact satisfiability modulo theories (SMT)-based and mixed-integer linear programming verification tools can establish formal input-output properties over declared neural network weights (Katz et al. 2017), yet these proofs remain vulnerable to open-world modeling gaps outside the declared plant and observations. Purely statistical validation collides with the exposure wall, requiring billions of physical test cycles to establish low target failure probabilities. Runtime safety filters and shield wrappers transfer the verification burden onto the shield's own coverage and latency envelope (Hsu et al. 2024). The unresolved systems challenge is constructing hybrid safety arguments: composing analytical proofs where deterministic physics models are tractable with statistical bounds where perceptual models operate, and allocating risk budgets across both tiers. Until such composite certification arguments mature, the architecture developed here keeps learned proposers outside the safety-critical permission path. Verifiable progress requires publishing complete, audited conformity dossiers for learned safety components alongside their formal assurance cases.

7 **Fail-Passive and Fail-Operational**: Avionics separates a fail-passive function, whose failure causes no significant deviation of path or attitude and hands the task back to the crew, from a fail-operational one, which keeps performing through a failure. Removing torque from a wheeled base is fail-passive in this sense only after it reaches a qualified, mechanically restrained rest. A balancing biped in mid-stride has no such resting state, so its posture control must remain operational until the machine reaches a supported rest, and the certifiable reaction must be a controlled one.

↩ Optical elastomer tactile sensing and its timing and calibration obligations are established in section 8.2.6.

Table 18.3: A research agenda keyed to the four laws, as of 2026: Each row names an open problem, the law it stresses, the gap class of section 18.4.1 it belongs to, and the evidence that would count as progress. The last column dates the state of practice and should be refreshed in later editions; the durable content is in the first four columns.

Open problem	Law stressed	Gap class	Progress would be	State of practice, as of 2026
Generalist, cross-embodiment policies	Permission must be earned	Evaluation deficit	Evidence that transfers across bodies and updates under a stated equivalence condition	One policy across several robot types (Black et al. 2024); learning from deployment with corrections (Physical Intelligence et al. 2025); motion transfer between embodiments (Gemini Robotics Team 2025a)
Data scaling	Endogenous data	Evaluation deficit	Scaling curves on closed-loop failure rates inside a declared ODD	Roughly power-law gains in environment and object diversity, saturating in demonstrations per environment (Lin et al. 2025)
World models as evaluators	Permission must be earned	Evaluation deficit	A proxy-transfer condition validated on the failure tail	General world foundation models (NVIDIA, Agarwal, et al. 2025; Assran et al. 2025); a world model built for policy evaluation (Jeon et al. 2026)
Fall-safe dynamically stable machines	Irreversibility	Scope mismatch	A certified, actively controlled reaction chain to a supported rest state	The fail-passive gap located in a humanoid pick-and-place cell (Ding et al. 2026)
Contact-rich, dexterous manipulation	Proposal is not permission	Unmeasured observable	A qualified tactile channel and a checkable contact-force bound	High-resolution vision-based tactile sensing (Yuan et al. 2017)
Certifying learned safety functions	Permission must be earned	Theoretical deficit	A published conformity assessment of a learned safety component	Third-party assessment required in the EU from 20 January 2027 (European Parliament and Council of the European Union 2023); network verification of declared models (Katz et al. 2017)

DRAFT

Two systems challenges cut across all six problems. The first is continual online adaptation, which turns the model into a moving target. Whether a proposer retrains asynchronously on aggregated fleet telemetry or adapts its latent state online to accommodate component wear and shifting terrain (Kumar et al. 2021), background model updates must never starve the permission pipeline of compute or bus bandwidth. The second is multi-robot fleet coordination, which couples otherwise independent permission boundaries. Transmitting shared task intents across lossy wireless links cannot substitute for the local, hard-real-time sensor streams required by an individual robot's admission gate. Physically cooperative manipulation of a shared payload likewise couples actuator contact dynamics directly across platforms. Across all of these problems, advancing physical AI does not require abandoning established systems rigor; it requires extending it. Unlocking the next capability begins by isolating a concrete physical task that the current system cannot safely execute, identifying the missing empirical premise or unmeasured observable, and engineering the precise evidential argument required to justify expanding the machine's operational envelope.

18.5 Fallacies and Pitfalls

When engineers move physical AI systems such as the warehouse mobile manipulator or the packaging cell from isolated benchmarks to production floors, the traps that remain are conceptual. Every local calculation can succeed numerically while the system-level reasoning fails, because the team has misread how physical dynamics, learned proposers, and runtime enforcement relate. The traps below recur on both machines.

Fallacy: *A protective intervention means the whole machine failed its task.*

When a runtime safety governor or control barrier filter modifies an actuator command, engineers often mistakenly log the episode as a system-level failure. In practice, a least-change speed correction

or torque clamp often allows an autonomous mobile robot or packaging arm to complete its physical mission smoothly within safe boundaries. If the warehouse machine enters a route whose floor is inspected only to $\mu \geq 0.12$ and the governor trims its speed from the 1.3 m/s aisle speed to the register's 1 m/s restricted mode for that floor, the delivery still completes. The leg takes 30 percent longer.

Treating every safety intervention as a mission failure corrupts operational telemetry. Conflating proposer interventions with unrecoverable task aborts conceals where engineering effort should be directed. Development teams must track three metrics independently: raw proposer autonomy (the fraction of time the learned model operates without override), whole-machine task success (whether the physical objective was achieved within operational deadlines), and protective intervention frequency. Decoupling these signals reveals whether system bottlenecks stem from policy distribution shift, sensor latency in the perception pipeline, or overly conservative governor tuning.

↩ Proposer autonomy, task success, and protective intervention as separately recorded outcome channels are established in section 7.2.

Pitfall: *Treating a restriction as evidence that its physical premise holds.*

Deriving a tighter operational limit does not guarantee that the physical environment satisfies the assumptions behind that limit. The register's restricted mode for the warehouse machine, 1 m/s instead of 1.3 m/s, provides a stopping margin only on floors that hold $\mu \geq 0.12$. Under the clear film, where μ falls to 0.05 and the available deceleration to 0.49 m/s², the lower speed setpoint carries no stopping guarantee at all; the restriction rests on the same unmeasured premise as the mode it replaced.

Digital artifacts cannot substitute for physical friction. Neither a signed configuration parameter in software, nor a clear path reported by an RGB camera, proves that the ground provides adequate traction. Vision pipelines readily mistake transparent oil or thin ice for dry concrete. A restricted envelope therefore needs its own premise kept true, through the inspection and spill procedure the register assigns to facility operations or through a qualified forward film detector. Monitoring wheel slip and inertial deceleration can reveal, after the fact, that a premise failed, and should trigger the restriction for later motion, but it cannot rescue the stop already under way. A safety envelope is only as credible as the physical premise beneath it.

Fallacy: *More model capacity or simulated exposure settles every residual claim.*

Scaling up neural network parameter counts or running billions of simulated training steps improves policy interpolation across modeled conditions, but neither technique can overcome an unmodeled physical reality. A deep learning policy cannot infer an unobserved physical state, such as internal material stress, micro-slip at a fingertip, or variable ground compliance, if the sensor suite lacks the transducers to measure it. Similarly, physics simulators rely on simplified approximations of rigid-body contact, Coulomb friction, and non-deformable geometry. Generating large synthetic datasets within a simulator that omits real contact mechanics merely produces policies optimized for fictitious physical dynamics.

When an embodied system displays persistent edge-case failures, the solution rarely lies in adding more transformer layers or randomizing synthetic textures. Engineers must diagnose the specific physical claim that lacks empirical grounding. Resolving the defect requires choosing the appropriate engineering intervention: integrating a dedicated physical sensor to observe the missing state, introducing passive mechanical compliance to absorb unmodeled impact energy, constraining the operational design domain, or calibrating friction models against instrumented hardware trials.

Pitfall: *Charging the full stop time against an action deadline.*

An engineer who sizes the latest useful observation by subtracting the whole braking duration treats the machine as if it held its speed until it stopped instantly. For the jerk-limited stop from the 1.3 m/s aisle speed, braking lasts 931.2 ms but covers only the distance of 465.6 ms of travel at constant speed. The time form therefore over-reserves 465.6 ms, or 605.3 mm, more than the entire 131.0 mm spare.

The error is safe in the narrow sense that it never admits an unsafe stop, which is why it survives review. Its cost appears elsewhere. A deadline computed this way refuses operating modes the distance form correctly admits, and it hides how much timing allowance a new detector or a slower admission stage can still spend. The deadline belongs to the remaining distance, equation 18.4, with the braking term divided by speed, never to the stop's duration.

Fallacy: *A safety shield makes the proposer's error rate irrelevant.*

A shield attenuates hazardous proposals; it does not erase them. In notebook 18.3, a full release cycle of adjudicated proposals with no hazardous event still bounds the proposal rate only at $1.387 \times 10^{-5}\,\mathrm{h}^{-1}$. A shield with interception coverage 0.9999 then leaves a modeled residual of $1.387 \times 10^{-9}\,\mathrm{h}^{-1}$, above the $1 \times 10^{-9}\,\mathrm{h}^{-1}$ target even with the common-cause term set to zero. Meeting the target at that proposal bound would need coverage of 0.9999279, and showing the assumed coverage alone takes 29,956 seeded hazardous proposals with no miss.

The residual of equation 18.2 is a product, so both factors need evidence, and a common-cause failure that blinds the shield in the same states where the proposer errs lowers coverage exactly where it matters. The shield is the reason the proposer need not be perfect, not a reason to stop measuring how often it errs.

Pitfall: *Qualifying a forward detector by its miss rate alone.*

A floor-film detector that passes 2,995 failure-free seeded trials has a miss bound of 0.1 percent, and a team may treat that result as closing the register entry. Detection that arrives too late is a miss in every sense that matters to the stop. At the 1.3 m/s aisle speed with a 100 ms sensing and inference latency, equation 18.3 requires the detector to see the film 1.10 m ahead, about the whole 1.10 m clear distance. At that range the latency leaves nothing for the 63.3 ms dispatch-to-admission allocation. To fit beside it, the detector's latency must stay under the 37.5 ms reserve.

A detector that reports only once the wheels are on the film faces a 2.13 m stop, turning a statistically accurate classification into an unrecoverable overrun. The closure test must therefore predeclare range, latency, false-alarm rate, and the friction assumed after detection, alongside the miss rate, and qualify them together on the routes the restriction covers.

18.6 Summary

The warehouse mobile manipulator now carries one connected design argument, and the packaging cell shows that the same argument transfers to a machine with different mechanics, rates, and hazards. The book opened by asking why a better prediction can still produce a worse physical action, and the chapter's answer is concrete. A neural network optimizes expected statistical loss over its training distribution, but the physical plant operates under continuous Newtonian mechanics, electromagnetism, and thermodynamics. A better proposer cannot supply the friction premise that an unobserved oil film removes, because no input it receives measures the tire-ground contact interface. It cannot make a derated operating mode evaluated, because the operational restriction that shortens the stop also displaces the proposer into physical configurations its training and evaluation never sampled. It cannot close a detectorless claim such as the glare-hidden packaging tab, because the physical sensor required to resolve that state does not exist on the machine. Nor can a faster forward pass recover the stopping distance already consumed while the vehicle cruised under unguided momentum ($d_{\text{blind}} = v\tau_{\text{delay}}$). In each case, the physical outcome is judged by dynamics, timing, and environmental evidence that the learned prediction does not contain.

The learned model must therefore be engineered as an unprivileged component of a machine, with the rest of the machine supplying what the model lacks, one law at a time. The first law, irreversibility, turns the body's kinetic, actuator, and thermal limits into budgets that bound any proposal, and the friction floor showed that a single unmeasured term can void those budgets. The

second law, endogenous data, confines competence to the states the closed loop actually visits, which is why the packaging cell's restricted mode opened a regime its evaluation never sampled. The third law, proposal is not permission, places an independent permission path between proposal and actuator, and the shield-dilution bound showed that this path improves a hazard-rate claim only for the hazards it can detect and intercept in time. The fourth law, permission must be earned, grants the machine only what its evidence supports, and the residual-claims register gives each unobserved premise a restriction, an owner, and a predeclared closure test. Table 18.4 collects the relations the chapter applied to both machines, what each bounds or decides, and where the book derives it.

Table 18.4: Key equations of the chapter: Each equation the chapter applies to the warehouse machine or the packaging cell, what it bounds or decides, and where the book derives it. The first two and the zero-failure count are derived in earlier chapters and reused here; the others are stated in this chapter.

Equation	What it bounds or decides	Where derived
$d_{\text{stop}} = v\tau_{\text{delay}} + v^2/2a_{\text{brake}}$	Whether a mode's stop fits its clearance, and whether delay or braking dominates	equation 2.2
$v_{\max} = -a_{\text{brake}}\tau_{\text{delay}} + \sqrt{(a_{\text{brake}}\tau_{\text{delay}})^2 + 2a_{\text{brake}}D_{\text{avail}}}$	The speed ceiling a stopping budget admits	equation 2.3; jerk ramps in equation 12.2
$a = \min(a_{\text{brake}}, \mu g)$	Whether the brake or the floor binds, and the friction floor a_{brake}/g	equation 18.1
$\lambda_{\text{system},h} \le \lambda_{\text{proposal},h}(1-c_h) + \lambda_{\text{other}}$	What a shield adds to a hazard-rate claim, and the coverage the claim needs	equation 18.2
$n = \lceil \ln \alpha_{\text{sig}} / \ln(1-p_{\text{fail}}) \rceil$	Failure-free trials that bound a miss or false-accept probability p_{fail}	equation 7.5
$d_{\text{detect}} \ge v\,t_{\text{detect}} + d_{\text{budget}}(v)$	The range at which a detector must report a hazard ahead of the machine	equation 18.3
$d_{\text{rem}} \ge v\,(T_{\text{comp}} + T_{\text{act}}) + d_{\text{brake}}(v)$	The latest point at which a distinguishing observation can still help	equation 18.4
$F_{\text{trip}} + k_{\text{eq}}\,\Delta x_{\text{driven}} \le F_{\text{floor}}$	The stiffness of a passive element that bounds contact force	equation 18.5

DRAFT

Key Takeaways: Engineering a machine that senses and acts

- **Useful service joins the four laws:** Body limits, learned competence, independent permission, and maintained evidence must support the same task; a better prediction strengthens only one link of that argument.
- **Competence can grow:** New qualified experience, retraining, and regression evaluation can extend what the next candidate can justify.
- **Admission can preserve success:** A minimally corrected proposal can complete useful work; proposer autonomy, task success, and protection require separate measures.
- **Restrictions need evidence:** A lower speed still needs traction, clearance, and a feasible response. A stipulated supported branch teaches the release decision without turning illustrative records into measurements.
- **Residual limits select the next experiment:** Missing observations, evaluation support, proof premises, and task scope call for different engineering responses.

The chain from body limits to a release decision can be carried through for any machine, but the chapter's last lesson lies where that chain runs out. The running machine's hardest limits were not its slowest computation or its weakest model; they were premises no sensor reports, a floor's friction and a latch's strength. Engineering judgment at that edge means naming such a premise, restricting the machine until evidence arrives, and choosing the next measurement, learning experiment, architectural change, or narrower domain by which open premise it retires. Chapter 1 promised the ability to design, teach, integrate, evaluate, and maintain such a machine within justified operating

limits, and to name, when the evidence is incomplete, what would support the next capability. That ability to build, verify, and justify a machine that senses and acts is the enduring discipline this book develops.

What's Next: From physical AI to the systems discipline

The next unfamiliar machine will bring a different body, task, and environment. Its engineering begins by tracing how observations become proposals and how admitted commands become force, motion, and heat. The methods developed here provide a way to teach useful behavior, integrate it with timely physical control, and determine which operating claims the resulting evidence supports. This work is dedicated to the students, researchers, and systems engineers who take up the challenge of giving machines safe and capable hands in an open physical world.

Prof. Vijay Janapa Reddi, Harvard University

DRAFT

DRAFT

BACK MATTER

DRAFT

References

Acemoglu, Daron, and Pascual Restrepo. 2020. "Robots and Jobs: Evidence from US Labor Markets." *Journal of Political Economy* 128 (6): 2188–244. https://doi.org/10.1086/705716.

AgiBot-World-Contributors, Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Shenyuan Gao, et al. 2025. "AgiBot World Colosseo: A Large-Scale Manipulation Platform for Scalable and Intelligent Embodied Systems." *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3549–56. https://doi.org/10.1109/IROS60139.2025.11247088.

Agility Robotics. 2025. *Digit Moves over 100,000 Totes in Commercial Deployment*. Agility Robotics, 20 November 2025.

Ahn, Michael, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. 2024. "AutoRT: Embodied Foundation Models for Large Scale Orchestrating of Robotic Agents." *arXiv Preprint arXiv:2401.12963*.

Ahn, Michael, Anthony Brohan, Yevgen Chebotar, et al. 2022. "Do as I Can, Not as I Say: Grounding Language in Robotic Affordances." *Proceedings of the 6th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 205: 287–318.

Alshiekh, Mohammed, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. 2018. "Safe Reinforcement Learning via Shielding." *Proceedings of the AAAI Conference on Artificial Intelligence* 32. https://doi.org/10.1609/aaai.v32i1.11797.

Amanatides, John, and Andrew Woo. 1987. "A Fast Voxel Traversal Algorithm for Ray Tracing." *Eurographics '87*, 3–10.

Ames, Aaron D., Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. 2019. "Control Barrier Functions: Theory and Applications." *2019 18th European Control Conference (ECC)*, 3420–31. https://doi.org/10.23919/ECC.2019.8796030.

Ames, Aaron D, Jessy W Grizzle, and Paulo Tabuada. 2014. "Control Barrier Function Based Quadratic Programs with Application to Adaptive Cruise Control." *53rd IEEE Conference on Decision and Control*, 6271–78.

Ames, Aaron D., Xiangru Xu, Jessy W. Grizzle, and Paulo Tabuada. 2017. "Control Barrier Function Based Quadratic Programs for Safety Critical Systems." *IEEE Transactions on Automatic Control* 62 (8): 3861–76. https://doi.org/10.1109/TAC.2016.2638961.

An, Dong, Yuankai Qi, Yang Huang, Yan Wei, Liang Wang, Tieniu Tan, et al. 2023. "ETPNav: Evolving Topological Planner for Vision-Language Navigation in Continuous Environments." *arXiv Preprint arXiv:2304.03047*.

Anderson, Robert J., and Mark W. Spong. 1989. "Bilateral Control of Teleoperators with Time Delay." *IEEE Transactions on Automatic Control* 34 (5): 494–501. https://doi.org/10.1109/9.24201.

Andrychowicz, Marcin, Bowen Baker, Maciek Chociej, Rafał J'ozefowicz, et al. 2020. "Learning Dexterous in-Hand Manipulation." *The International Journal of Robotics Research* 39 (1): 3–20.

Angelopoulos, Anastasios N., and Stephen Bates. 2023. "Conformal Prediction: A Gentle Introduction." *Foundations and Trends in Machine Learning* 16 (4): 494–591. https://doi.org/10.1561/2200000101.

Ankile, Lars, Anthony Simeonov, Idan Shenfeld, Marcel Torne, and Pulkit Agrawal. 2025. "From Imitation to Refinement: Residual RL for Precise Assembly." *2025 IEEE International Conference on Robotics and Automation (ICRA)*. https://doi.org/10.1109/ICRA55743.2025.11127442.

Annpureddy, Yashwanth, Che Liu, Georgios Fainekos, and Sriram Sankaranarayanan. 2011. "S-TaLiRo: A Tool for Temporal Logic Falsification for Hybrid Systems." *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture notes in computer science, vol. 6605: 254–57. https://doi.org/10.1007/978-3-642-19835-9_21.

Arbaugh, William A., David J. Farber, and Jonathan M. Smith. 1997. "A Secure and Reliable Bootstrap Architecture." *Proceedings of the 1997 IEEE Symposium on Security and Privacy*, 65–71. https://doi.org/10.1109/SECPRI.1997.601317.

Argall, Brenna D., Sonia Chernova, Manuela Veloso, and Brett Browning. 2009. "A Survey of Robot Learning from Demonstration." *Robotics and Autonomous Systems* 57 (5): 469–83. https://doi.org/10.1016/j.robot.2008.10.024.

Arlat, Jean, Martine Aguera, Louis Amat, Yves Crouzet, Jean-Charles Fabre, Jean-Claude Laprie, Eliane Martins, and David Powell. 1990. "Fault Injection for Dependability Validation: A Methodology and Some Applications." *IEEE Transactions on Software Engineering* 16 (2): 166–82. https://doi.org/10.1109/32.44380.

ARM Limited. 2012. *ARM Cortex-M Programming Guide to Memory Barrier Instructions*. Application Note 321, ARM DAI 0321A. ARM Limited.

Armeni, Iro, Zhi-Yang He, Amir Zamir, JunYoung Gwak, Jitendra Malik, Martin Fischer, and Silvio Savarese. 2019. "3D Scene Graph: A Structure for Unified Semantics, 3D Space, and Camera." *IEEE/CVF International Conference on Computer Vision (ICCV)*, 5663–72. https://doi.org/10.1109/ICCV.2019.00576.

Artstein, Zvi. 1983. "Stabilization with Relaxed Controls." *Nonlinear Analysis: Theory, Methods & Applications* 7 (11): 1163–73. https://doi.org/10.1016/0362-546X(83)90049-4.

Arvidson, Raymond E., James F. Bell, Paolo Bellutta, Nathalie A. Cabrol, et al. 2010. "Spirit Mars Rover Mission: Overview and Selected Results from the Northern Home Plate Winter Haven to the Side of Scamander Crater." *Journal of Geophysical Research: Planets* 115 (E7). https://doi.org/10.1029/2010JE003633.

Asada, Haruhiko, and Takeo Kanade. 1983. "Design of Direct-Drive Mechanical Arms." *ASME Journal of Vibration, Acoustics, Stress, and Reliability in Design* 105 (3): 312–16. https://doi.org/10.1115/1.3269094.

Association for Advancing Automation. 2020. *ANSI/A3 R15.08-1-2020: Industrial Mobile Robots, Safety Requirements, Part 1: Requirements for the Industrial Mobile Robot*. Association for Advancing Automation.

Association for Advancing Automation. 2023. *ANSI/A3 R15.08-2-2023: Industrial Mobile Robots, Safety Requirements, Part 2: Requirements for IMR System(s) and IMR Application(s)*. Association for Advancing Automation.

Assran, Mido, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, et al. 2025. "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning." *arXiv Preprint arXiv:2506.09985*.

Åström, Karl Johan. 1965. "Optimal Control of Markov Processes with Incomplete State Information." *Journal of Mathematical Analysis and Applications* 10 (1): 174–205. https://doi.org/10.1016/0022-247X(65)90154-X.

Åström, Karl Johan, and Tore Hägglund. 2006. *Advanced PID Control*. ISA-The Instrumentation, Systems,; Automation Society.

Åström, Karl Johan, and Richard M Murray. 2021. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton university press.

Aubin, Jean-Pierre. 1991. *Viability Theory*. Systems & Control: Foundations & Applications. Birkhäuser.

Audsley, Neil, Alan Burns, Mike Richardson, Ken Tindell, and Andy J. Wellings. 1993. "Applying New Scheduling Theory to Static Priority Pre-Emptive Scheduling." *Software Engineering Journal* 8 (5): 284–92. https://doi.org/10.1049/sej.1993.0034.

AUTOSAR. 2024. *E2E Protocol Specification*. No. 849. AUTOSAR.

Bainbridge, Lisanne. 1983. "Ironies of Automation." *Automatica* 19 (6): 775–79. https://doi.org/10.1016/0005-1098(83)90046-8.

Balkema, August A., and Laurens de Haan. 1974. "Residual Life Time at Great Age." *The Annals of Probability* 2 (5): 792–804. https://doi.org/10.1214/aop/1176996548.

Ball, Philip J., Laura Smith, Ilya Kostrikov, and Sergey Levine. 2023. "Efficient Online Reinforcement Learning with Offline Data." *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 202: 1577–94.

Bansal, Somil, Mo Chen, Sylvia Herbert, and Claire J. Tomlin. 2017. "Hamilton-Jacobi Reachability: A Brief Overview and Recent Advances." *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, 2242–53. https://doi.org/10.1109/CDC.2017.8263977.

Barr, Michael. 2013. *Bookout v. Toyota Motor Corp.: 2005 Camry L4 Software Analysis*.

Bar-Shalom, Yaakov, X. Rong Li, and Thiagalingam Kirubarajan. 2001. *Estimation with Applications to Tracking and Navigation*. Wiley. https://doi.org/10.1002/0471221279.

Bar-Shalom, Yaakov, and Edison Tse. 1975. "Tracking in a Cluttered Environment with Probabilistic Data Association." *Automatica* 11 (5): 451–60. https://doi.org/10.1016/0005-1098(75)90021-7.

Bechtel, Michael G., and Heechul Yun. 2019. "Denial-of-Service Attacks on Shared Cache in Multicore: Analysis and Prevention." *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 357–67. https://doi.org/10.1109/RTAS.2019.00037.

Belkhale, Suneel, Yuchen Cui, and Dorsa Sadigh. 2023. "Data Quality in Imitation Learning." *Advances in Neural Information Processing Systems (NeurIPS)* 36: 80375–95. https://doi.org/10.52202/075280-3524.

Bellman, Richard. 1957. *Dynamic Programming*. Princeton University Press.

Bernstein, Daniel J., Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. "High-Speed High-Security Signatures." *Journal of Cryptographic Engineering* 2 (2): 77–89. https://doi.org/10.1007/s13389-012-0027-1.

Berscheid, Lars, and Torsten Kröger. 2021. "Jerk-Limited Real-Time Trajectory Generation with Arbitrary Target States." *Robotics: Science and Systems (RSS)* XVII. https://doi.org/10.15607/RSS.2021.XVII.015.

Bewley, Alex, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. 2016. "Simple Online and Realtime Tracking." *2016 IEEE International Conference on Image Processing (ICIP)*, 3464–68.

Biagiotti, Luigi, and Claudio Melchiorri. 2008. *Trajectory Planning for Automatic Machines and Robots*. Springer. https://doi.org/10.1007/978-3-540-85629-0.

Bishop, Christopher M. 1994. *Mixture Density Networks*. NCRG/94/004. Neural Computing Research Group, Aston University.

Bishop, Peter, and Robin Bloomfield. 1998. "A Methodology for Safety Case Development." In *Industrial Perspectives of Safety-Critical Systems: Proceedings of the Sixth Safety-Critical Systems Symposium*, edited by Felix Redmill and Tom Anderson. Springer. https://doi.org/10.1007/978-1-4471-1534-2_14.

Bjorck, Johan, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, et al. 2025. "GR00T N1: An Open Foundation Model for Generalist Humanoid Robots." *arXiv Preprint arXiv:2503.14734*.

Black, Kevin, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, et al. 2025. "$\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization." *Proceedings of the Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 305: 17–40.

Black, Kevin, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. 2024. "π_0: A Vision-Language-Action Flow Model for General Robot Control." *arXiv Preprint arXiv:2410.24164*.

Black, Kevin, Manuel Y. Galliker, and Sergey Levine. 2025. "Real-Time Execution of Action Chunking Flow Policies." *Advances in Neural Information Processing Systems (NeurIPS)*.

Blackwelder, William C. 1982. "'Proving the Null Hypothesis' in Clinical Trials." *Controlled Clinical Trials* 3 (4): 345–53. https://doi.org/10.1016/0197-2456(82)90024-1.

Blanchini, Franco. 1999. "Set Invariance in Control." *Automatica* 35 (11): 1747–67.

Bloomfield, Robin, and Bev Littlewood. 2003. "Multi-Legged Arguments: The Impact of Diversity Upon Confidence in Dependability Arguments." *Proceedings of the 2003 International Conference on Dependable Systems and Networks*, 25–34. https://doi.org/10.1109/DSN.2003.1209913.

Bloomfield, Robin, and Kateryna Netkachova. 2014. "Building Blocks for Assurance Cases." *IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, 186–91. https://doi.org/10.1109/ISSREW.2014.72.

Bobrow, James E., Steven Dubowsky, and J. S. Gibson. 1985. "Time-Optimal Control of Robotic Manipulators Along Specified Paths." *The International Journal of Robotics Research* 4 (3): 3–17. https://doi.org/10.1177/027836498500400301.

Bode, Hendrik W. 1945. *Network Analysis and Feedback Amplifier Design*. D. Van Nostrand Company.

Boor, Carl de. 1978. *A Practical Guide to Splines*. Vol. 27. Applied Mathematical Sciences. Springer. https://doi.org/10.1007/978-1-4612-6333-3.

Borenstein, Johann, and Liqiang Feng. 1996. "Measurement and Correction of Systematic Odometry Errors in Mobile Robots." *IEEE Transactions on Robotics and Automation* 12 (6): 869–80. https://doi.org/10.1109/70.544770.

Bosch Sensortec. 2024. *BMI088: 6-Axis Motion Tracking for High-Performance Applications, Data Sheet*. BST-BMI088-DS000-19, Revision 1.9. Bosch Sensortec GmbH.

Boston Dynamics. 2024. *An Electric New Era for Atlas*. Boston Dynamics blog.

Bowden, Frank Philip, and David Tabor. 1950. *The Friction and Lubrication of Solids*. International Series of Monographs on Physics. Clarendon Press.

Bridle, John S. 1990. "Probabilistic Interpretation of Feedforward Classification Network Outputs, with Relationships to Statistical Pattern Recognition." In *Neurocomputing: Algorithms, Architectures and Applications*, edited by Françoise Fogelman Soulié and Jeanny Hérault, vol. 68. NATO ASI Series f: Computer and Systems Sciences. Springer. https://doi.org/10.1007/978-3-642-76153-9_28.

Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, et al. 2023. "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control." *arXiv Preprint arXiv:2307.15818*.

Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, et al. 2023. "RT-1: Robotics Transformer for Real-World Control at Scale." *Robotics: Science and Systems (RSS)*. https://doi.org/10.15607/RSS.2023.XIX.025.

Brooks, David, and Margaret Martonosi. 2001. "Dynamic Thermal Management for High-Performance Microprocessors." *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 171–82. https://doi.org/10.1109/HPCA.2001.903261.

Brooks, Rodney. 1986. “A Robust Layered Control System for a Mobile Robot.” *IEEE Journal on Robotics and Automation* 2 (1): 14–23.

Brooks, Rodney A. 1991. “Intelligence Without Representation.” *Artificial Intelligence* 47 (1–3): 139–59.

Brown, Lawrence D., T. Tony Cai, and Anirban DasGupta. 2001. “Interval Estimation for a Binomial Proportion.” *Statistical Science* 16 (2): 101–33. https://doi.org/10.1214/ss/1009213286.

Brown, Tom B., Dandelion Mané, Aurko Roy, Martín Abadi, and Justin Gilmer. 2017. “Adversarial Patch.” *arXiv Preprint arXiv:1712.09665*.

Bryant, Randal E., and David R. O’Hallaron. 2015. *Computer Systems: A Programmer’s Perspective*. 3rd ed. Pearson.

Buolamwini, Joy, and Timnit Gebru. 2018. “Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification.” *Conference on Fairness, Accountability and Transparency*, 77–91.

Burns, Alan, and Robert I. Davis. 2017. “A Survey of Research into Mixed Criticality Systems.” *ACM Computing Surveys* 50 (6): 1–37. https://doi.org/10.1145/3131347.

Burtch, Gordon, Brad Greenwood, and Kiron Ravindran. 2025. “Lucy and the Chocolate Factory: Warehouse Robotics and Worker Safety.” *ILR Review* 78 (4): 587–613. https://doi.org/10.1177/00197939251333754.

Butler, Ricky W., and George B. Finelli. 1993. “The Infeasibility of Quantifying the Reliability of Life-Critical Real-Time Software.” *IEEE Transactions on Software Engineering* 19 (1): 3–12. https://doi.org/10.1109/32.210303.

Buttazzo, Giorgio C. 2011. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. 3rd ed. Real-Time Systems Series. Springer. https://doi.org/10.1007/978-1-4614-0676-1.

Cadena, Cesar, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid, and John J. Leonard. 2016. “Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age.” *IEEE Transactions on Robotics* 32 (6): 1309–32. https://doi.org/10.1109/TRO.2016.2624754.

Cadene, Rémi, Simon Alibert, Alexander Toshev, Thomas Sen, Quentin Gallouédec, Adil Zouitine, et al. 2024. *LeRobot: State-of-the-Art Machine Learning for Real-World Robotics in PyTorch*.

California Department of Motor Vehicles. 2018a. *California Code of Regulations, Title 13, Article 3.7: Testing of Autonomous Vehicles*.

California Department of Motor Vehicles. 2018b. *California Code of Regulations, Title 13, Article 3.8: Deployment of Autonomous Vehicles*.

California Department of Motor Vehicles. 2023. *DMV Statement on Cruise LLC Suspension*.

California Department of Motor Vehicles. 2026. *Autonomous Vehicle Disengagement Reports*.

California Public Utilities Commission. 2023. *CPUC Approves Permits for Cruise and Waymo to Charge Fares for Passenger Service in San Francisco*.

California Public Utilities Commission. 2024. *Decision 24-07-004: Presiding Officer’s Decision Approving Offered Settlement Terms and Granting Deferral of the Order to Show Cause Proceedings*.

Campbell, Stephen L., and Ramine Nikoukhah. 2004. *Auxiliary Signal Design for Failure Detection*. Princeton University Press.

Card, Stuart K., Thomas P. Moran, and Allen Newell. 1983. *The Psychology of Human-Computer Interaction*. Lawrence Erlbaum Associates.

Casini, Daniel, Tobias Blaß, Ingo Lütkebohle, and Björn B. Brandenburg. 2019. “Response-Time Analysis of ROS 2 Processing Chains Under Reservation-Based Scheduling.” *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*, Leibniz international proceedings in informatics (LIPIcs), vol. 133: 6:1–23. https://doi.org/10.4230/LIPIcs.ECRTS.2019.6.

Cervin, Anton, Dan Henriksson, Bo Lincoln, Johan Eker, and Karl-Erik Årzén. 2003. “How Does Control Timing Affect Performance? Analysis and Simulation of Timing Using Jitterbug and TrueTime.” *IEEE Control Systems Magazine* 23 (3): 16–30. https://doi.org/10.1109/MCS.2003.1200240.

Chen, Tilden, and Akshaya Ramachandran. 2025. *How TPSI3133 DESAT Can Protect Your Expensive SiC MOSFETs and IGBTs*. Application Note No. SLVAG18. Texas Instruments.

Chen, Timothy, Ola Shorinwa, Joseph Bruno, Aiden Swann, Javier Yu, et al. 2025. “Splat-Nav: Safe Real-Time Robot Navigation in Gaussian Splatting Maps.” *IEEE Transactions on Robotics* 41: 2765–84. https://doi.org/10.1109/TRO.2025.3552348.

Chen, Yuxiao, Mrdjan Jankovic, Mario Santillo, and Aaron D. Ames. 2021. “Backup Control Barrier Functions: Formulation and Comparative Study.” *2021 60th IEEE Conference on Decision and Control (CDC)*, 6835–41. https://doi.org/10.1109/CDC45484.2021.9683111.

Chi, Cheng, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2025. "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion." *The International Journal of Robotics Research* 44 (10-11): 1684–704. https://doi.org/10.1177/02783649241273668.

Chi, Cheng, Zhenjia Xu, Chuer Pan, Eric Cousineau, Benjamin Burchfiel, Siyuan Feng, Russ Tedrake, and Shuran Song. 2024. "Universal Manipulation Interface: In-the-Wild Robot Teaching Without in-the-Wild Robots." *Robotics: Science and Systems (RSS)*. https://doi.org/10.15607/RSS.2024.XX.045.

Civera, Javier, Andrew J. Davison, and José María Martínez Montiel. 2008. "Inverse Depth Parametrization for Monocular SLAM." *IEEE Transactions on Robotics* 24 (5): 932–45. https://doi.org/10.1109/TRO.2008.2003276.

Clopper, C. J., and Egon S. Pearson. 1934. "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial." *Biometrika* 26 (4): 404–13.

Cohen, Jacob. 1988. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Routledge.

Coles, Stuart. 2001. *An Introduction to Statistical Modeling of Extreme Values*. Springer.

Computer History Museum. 1961. *1961: UNIMATE, the First Mass-Produced Industrial Robot, Begins Work at General Motors*. Timeline of Computer History.

Corso, Anthony, Robert Moss, Mark Koren, Ritchie Lee, and Mykel J. Kochenderfer. 2021. "A Survey of Algorithms for Black-Box Safety Validation of Cyber-Physical Systems." *Journal of Artificial Intelligence Research* 72. https://doi.org/10.1613/jair.1.12716.

Cousot, Patrick, and Radhia Cousot. 1977. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 238–52. https://doi.org/10.1145/512950.512973.

Craig, John J. 2005. *Introduction to Robotics: Mechanics and Control*. 3rd ed. Pearson Prentice Hall.

Cucu-Grosjean, Liliana, Luca Santinelli, Michael Houston, Code Lo, Tullio Vardanega, Leonidas Kosmidis, Jaume Abella, Enrico Mezzetti, Eduardo Quiñones, and Francisco J. Cazorla. 2012. "Measurement-Based Probabilistic Timing Analysis for Multi-Path Programs." *Euromicro Conference on Real-Time Systems (ECRTS)*, 91–101. https://doi.org/10.1109/ECRTS.2012.31.

Curless, Brian, and Marc Levoy. 1996. "A Volumetric Method for Building Complex Models from Range Images." *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '96)*, 303–12.

Dakin, Thomas W. 1948. "Electrical Insulation Deterioration Treated as a Chemical Rate Phenomenon." *Transactions of the American Institute of Electrical Engineers* 67 (1): 113–22. https://doi.org/10.1109/T-AIEE.1948.5059649.

Davis, Robert I., Sebastian Altmeyer, Leandro S. Indrusiak, Claire Maiza, Vincent Nelis, and Jan Reineke. 2018. "An Extensible Framework for Multicore Response Time Analysis." *Real-Time Systems* 54 (3): 607–61. https://doi.org/10.1007/s11241-017-9285-4.

Dean, Jeffrey, and Luiz André Barroso. 2013. "The Tail at Scale." *Communications of the ACM* 56 (2): 74–80.

Dellaert, Frank, and Michael Kaess. 2017. "Factor Graphs for Robot Perception." *Foundations and Trends in Robotics* 6 (1–2): 1–139. https://doi.org/10.1561/2300000043.

Denney, Ewen, Ganesh Pai, and Ibrahim Habli. 2015. "Dynamic Safety Cases for Through-Life Safety Assurance." *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, 587–90. https://doi.org/10.1109/ICSE.2015.199.

Ding, Caiwu, Tao Cui, Lingyun Wang, and Chengtao Wen. 2026. "Toward Certified Functional Safety for Industrial Humanoid Robots: The Fail-Passive Gap and a Feasibility Study." *arXiv Preprint arXiv:2608.02809*.

Donzé, Alexandre. 2010. "Breach, a Toolbox for Verification and Parameter Synthesis of Hybrid Systems." *Computer Aided Verification (CAV)*, Lecture notes in computer science, vol. 6174: 167–70. https://doi.org/10.1007/978-3-642-14295-6_17.

Donzé, Alexandre, and Oded Maler. 2010. "Robust Satisfaction of Temporal Logic over Real-Valued Signals." *Formal Modeling and Analysis of Timed Systems (FORMATS)*, Lecture notes in computer science, vol. 6246: 92–106. https://doi.org/10.1007/978-3-642-15297-9_9.

Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, et al. 2021. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." *International Conference on Learning Representations (ICLR)*.

Dragan, Anca D., Kenton C. T. Lee, and Siddhartha S. Srinivasa. 2013. "Legibility and Predictability of Robot Motion." *2013 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, 301–8. https://doi.org/10.1109/HRI.2013.6483603.

Dragan, Anca D., and Siddhartha S. Srinivasa. 2013. "A Policy-Blending Formalism for Shared Control." *The International Journal of Robotics Research* 32 (7): 790–805. https://doi.org/10.1177/0278364913490324.

Driess, Danny, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z. Ren, et al. 2025. "Knowledge Insulating Vision-Language-Action Models: Train Fast, Run Fast, Generalize Better." *arXiv Preprint arXiv:2505.23705*.

Driess, Danny, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, et al. 2023. "PaLM-e: An Embodied Multimodal Language Model." *International Conference on Machine Learning (ICML)*, 8469–88.

Dunn, Michael. 2013. *Toyota's Killer Firmware: Bad Design and Its Consequences*. EDN.

Durrant-Whyte, Hugh, and Tim Bailey. 2006. "Simultaneous Localization and Mapping: Part I." *IEEE Robotics & Automation Magazine* 13 (2): 99–110. https://doi.org/10.1109/MRA.2006.1638022.

Dworkin, Morris J. 2005. *Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication*. NIST Special Publication 800-38B. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.SP.800-38B.

Elfes, Alberto. 1989. "Using Occupancy Grids for Mobile Robot Perception and Navigation." *Computer* 22 (6): 46–57. https://doi.org/10.1109/2.30720.

Elish, Madeleine Clare. 2019. "Moral Crumple Zones: Cautionary Tales in Human-Robot Interaction." *Engaging Science, Technology, and Society* 5: 40–60. https://doi.org/10.17351/ests2019.260.

Endsley, Mica R. 1995. "Toward a Theory of Situation Awareness in Dynamic Systems." *Human Factors* 37 (1): 32–64. https://doi.org/10.1518/001872095779049543.

Endsley, Mica R., and Esin O. Kiris. 1995. "The Out-of-the-Loop Performance Problem and Level of Control in Automation." *Human Factors* 37 (2): 381–94. https://doi.org/10.1518/001872095779064555.

Epoch AI. 2026. *Data on AI Models*.

ESA Investigation Board. 2017. *ExoMars 2016 - Schiaparelli Anomaly Inquiry*. European Space Agency.

EtherCAT Technology Group. 2019. *Safety over EtherCAT (FSoE): Introduction and Overview*. EtherCAT Technology Group seminar slides.

European Parliament and Council of the European Union. 2023. *Regulation (EU) 2023/1230 of the European Parliament and of the Council of 14 June 2023 on Machinery and Repealing Directive 2006/42/EC and Council Directive 73/361/EEC*. Official Journal of the European Union, L 165, 29.6.2023, p. 1.

European Parliament and Council of the European Union. 2024a. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 Laying down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L series, 2024/1689, 12.7.2024.

European Parliament and Council of the European Union. 2024b. *Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on Horizontal Cybersecurity Requirements for Products with Digital Elements and Amending Regulations (EU) No 168/2013 and (EU) No 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act)*. Official Journal of the European Union, L series, 2024/2847, 20.11.2024.

European Parliament and Council of the European Union. 2026. *Regulation (EU) 2026/1744 of the European Parliament and of the Council of 8 July 2026 Amending Regulations (EU) 2024/1689, (EU) 2018/1139 and (EU) 2023/1230 as Regards the Simplification of the Implementation of Harmonised Rules on Artificial Intelligence (Digital Omnibus on AI)*. Official Journal of the European Union, L series, 2026/1744, 24.7.2026.

European Parliament, and Council of the European Union. 2016. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016*. Official Journal of the European Union.

Exponent, Inc. 2024. *Cruise LLC–October 2, 2023, Incident Report*. Exponent, Inc.

Fainekos, Georgios E., and George J. Pappas. 2009. "Robustness of Temporal Logic Specifications for Continuous-Time Signals." *Theoretical Computer Science* 410 (42): 4262–91. https://doi.org/10.1016/j.tcs.2009.06.021.

Fang, Hao-Shu, Chenxi Wang, Hongjie Fang, Minghao Gou, Jirong Liu, Hengxu Yan, et al. 2023. "AnyGrasp: Robust and Efficient Grasp Perception in Spatial and Temporal Domains." *IEEE Transactions on Robotics* 39 (5): 3929–45. https://doi.org/10.1109/TRO.2023.3281153.

Favaro, Francesca, Laura Fraade-Blanar, Scott Schnelle, Trent Victor, Mauricio Peña, Johan Engstrom, John Scanlon, Kris Kusano, and Dan Smith. 2023. "Building a Credible Case for Safety: Waymo's Approach for the Determination of Absence of Unreasonable Risk." *arXiv Preprint arXiv:2306.01917*.

Federal Aviation Administration. 1988. *System Design and Analysis*. AC 25.1309-1A. U.S. Department of Transportation, Federal Aviation Administration.

Federal Aviation Administration. 2024. *Use of Multi-Core Processors*. AC 20-193. U.S. Department of Transportation, Federal Aviation Administration.

Ferraguti, Federica, Cristian Secchi, and Cesare Fantuzzi. 2013. "A Tank-Based Approach to Impedance Control with Variable Stiffness." *2013 IEEE International Conference on Robotics and Automation (ICRA)*, 4948–53. https://doi.org/10.1109/ICRA.2013.6631284.

Ferrari, Carlo, and John Canny. 1992. "Planning Optimal Grasps." *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2290–95. https://doi.org/10.1109/ROBOT.1992.219918.

Ferrell, William R. 1965. “Remote Manipulation with Transmission Delay.” *IEEE Transactions on Human Factors in Electronics* HFE-6 (1): 24–32. https://doi.org/10.1109/THFE.1965.6591253.

Figure AI. 2025. *Helix: A Vision-Language-Action Model for Generalist Humanoid Control*. Figure AI technical report (web).

Fikes, Richard E., and Nils J. Nilsson. 1971. “STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving.” *Artificial Intelligence* 2 (3–4): 189–208. https://doi.org/10.1016/0004-3702(71)90010-5.

Fisac, Jaime F., Anayo K. Akametalu, Melanie N. Zeilinger, Shahab Kaynama, Jeremy Gillula, and Claire J. Tomlin. 2019. “A General Safety Framework for Learning-Based Control in Uncertain Robotic Systems.” *IEEE Transactions on Automatic Control* 64 (7): 2737–52. https://doi.org/10.1109/TAC.2018.2876389.

Fisher, Ronald A. 1922. “On the Interpretation of χ^2 from Contingency Tables, and the Calculation of P.” *Journal of the Royal Statistical Society* 85 (1): 87–94. https://doi.org/10.2307/2340521.

Fisher, Ronald A. 1935. *The Design of Experiments*. Oliver; Boyd.

Fisher, Ronald A., and Leonard H. C. Tippett. 1928. “Limiting Forms of the Frequency Distribution of the Largest or Smallest Member of a Sample.” *Mathematical Proceedings of the Cambridge Philosophical Society* 24 (2): 180–90. https://doi.org/10.1017/S0305004100015681.

Flash, Tamar, and Neville Hogan. 1985. “The Coordination of Arm Movements: An Experimentally Confirmed Mathematical Model.” *The Journal of Neuroscience* 5 (7): 1688–703. https://doi.org/10.1523/JNEUROSCI.05-07-01688.1985.

Florence, Pete, Corey Lynch, Feryal Behbahani, Tianhe Yu, Wenlong Huang, Fei Xia, et al. 2022. “Implicit Behavioral Cloning.” *Conference on Robot Learning (CoRL)*, 158–68.

Foote, Tully. 2013. “Tf: The Transform Library.” *IEEE International Conference on Technologies for Practical Robot Applications (TePRA)*, 1–6.

Foster, Dylan J., Adam Block, and Dipendra Misra. 2024. “Is Behavior Cloning All You Need? Understanding Horizon in Imitation Learning.” *Advances in Neural Information Processing Systems (NeurIPS)* 37: 120602–66. https://doi.org/10.52202/079017-3834.

Fox, Dieter, Wolfram Burgard, and Sebastian Thrun. 1997. “The Dynamic Window Approach to Collision Avoidance.” *IEEE Robotics & Automation Magazine* 4 (1): 23–33. https://doi.org/10.1109/100.580977.

Fraichard, Thierry, and Hajime Asama. 2004. “Inevitable Collision States: A Step Towards Safer Robots?” *Advanced Robotics* 18 (10): 1001–24. https://doi.org/10.1163/1568553042674662.

Franka Emika GmbH. 2018. *Panda Datasheet*. Product datasheet (reseller-hosted copy).

Franka Emika GmbH. 2021. *Franka Emika Robot’s Instruction Handbook*.

Franka Robotics GmbH. 2023. *Robot and Interface Specifications*. Franka Control Interface documentation.

Franka Robotics GmbH. 2026. *Troubleshooting*. Franka Control Interface documentation.

Franklin, Dustin. 2024. *OpenVLA: Vision/Language Action Models for Embodied Robotics*. NVIDIA Jetson AI Lab tutorial.

Franklin, Gene F, J David Powell, and Michael L Workman. 1998. *Digital Control of Dynamic Systems*. 3rd ed. Addison-Wesley.

Fu, Zipeng, Tony Z. Zhao, and Chelsea Finn. 2024. “Mobile ALOHA: Learning Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation.” *Proceedings of the 8th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 270.

Furgale, Paul, Joern Rehder, and Roland Siegwart. 2013. “Unified Temporal and Spatial Calibration for Multi-Sensor Systems.” *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1280–86.

Gait, Jason. 1986. “A Probe Effect in Concurrent Programs.” *Software: Practice and Experience* 16 (3): 225–33. https://doi.org/10.1002/spe.4380160304.

Gallego, Guillermo, Tobi Delbruck, Garrick Orchard, Chiara Bartolozzi, Brian Taba, Andrea Censi, Stefan Leutenegger, et al. 2022. “Event-Based Vision: A Survey.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (1): 154–80. https://doi.org/10.1109/TPAMI.2020.3008413.

Garrett, Caelan Reed, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. 2021. “Integrated Task and Motion Planning.” *Annual Review of Control, Robotics, and Autonomous Systems* 4: 265–93. https://doi.org/10.1146/annurev-control-091420-084139.

Garrett, Caelan Reed, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. 2020. “PDDLStream: Integrating Symbolic Planners and Blackbox Samplers via Optimistic Adaptive Planning.” *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)* 30: 440–48. https://doi.org/10.1609/icaps.v30i1.6739.

Garwood, Frank. 1936. “Fiducial Limits for the Poisson Distribution.” *Biometrika* 28 (3-4): 437–42. https://doi.org/10.1093/biomet/28.3-4.437.

Gaz, Claudio, Marco Cognetti, Alexander Oliva, Paolo Robuffo Giordano, and Alessandro De Luca. 2019. "Dynamic Identification of the Franka Emika Panda Robot with Retrieval of Feasible Parameters Using Penalty-Based Optimization." *IEEE Robotics and Automation Letters* 4 (4): 4147–54. https://doi.org/10.1109/LRA.2019.2931248.

Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. "Datasheets for Datasets." *Communications of the ACM* 64 (12): 86–92. https://doi.org/10.1145/3458723.

Geiger, Andreas, Philip Lenz, and Raquel Urtasun. 2012. "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite." *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 3354–61. https://doi.org/10.1109/CVPR.2012.6248074.

Gemini Robotics Team. 2025a. "Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots with Advanced Embodied Reasoning, Thinking, and Motion Transfer." *arXiv Preprint arXiv:2510.03342*.

Gemini Robotics Team. 2025b. "Gemini Robotics: Bringing AI into the Physical World." *arXiv Preprint arXiv:2503.20020*.

Gibson, James J. 1979. *The Ecological Approach to Visual Perception*. Houghton Mifflin.

Gihleb, Rania, Osea Giuntella, Luca Stella, and Tianyi Wang. 2022. "Industrial Robots, Workers' Safety, and Health." *Labour Economics* 78: 102205. https://doi.org/10.1016/j.labeco.2022.102205.

Glaessgen, Edward, and David Stargel. 2012. "The Digital Twin Paradigm for Future NASA and U.S. Air Force Vehicles." *53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference*. https://doi.org/10.2514/6.2012-1818.

Gnedenko, Boris. 1943. "Sur La Distribution Limite Du Terme Maximum d'une série Aléatoire." *Annals of Mathematics* 44 (3): 423–53. https://doi.org/10.2307/1968974.

Goertz, Raymond C. 1949. *Master-Slave Manipulator*. ANL-4311. Argonne National Laboratory. https://doi.org/10.2172/1054625.

Goldfarb, Donald, and Ashok Idnani. 1983. "A Numerically Stable Dual Method for Solving Strictly Convex Quadratic Programs." *Mathematical Programming* 27 (1): 1–33. https://doi.org/10.1007/BF02591962.

Goodenough, John B., Charles B. Weinstock, and Ari Z. Klein. 2013. "Eliminative Induction: A Basis for Arguing System Confidence." *2013 35th International Conference on Software Engineering (ICSE)*, 1161–64. https://doi.org/10.1109/ICSE.2013.6606668.

Goodfellow, I. J., J. Shlens, and C. Szegedy. 2015. "Explaining and Harnessing Adversarial Examples." *International Conference on Learning Representations (ICLR)*.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.

Gray, Cary G., and David R. Cheriton. 1989. "Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency." *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP)*, 202–10. https://doi.org/10.1145/74850.74870.

Green, Marc. 2000. "'How Long Does It Take to Stop?' Methodological Analysis of Driver Perception-Brake Times." *Transportation Human Factors* 2 (3): 195–216. https://doi.org/10.1207/STHF0203_1.

Grenat, Aaron, Sanjay Pant, Ravinder Rachala, and Samuel Naffziger. 2014. "Adaptive Clocking System for Improved Power Efficiency in a 28nm X86-64 Microprocessor." *IEEE International Solid-State Circuits Conference (ISSCC)*, 106–7. https://doi.org/10.1109/ISSCC.2014.6757358.

Grieves, Michael, and John Vickers. 2017. "Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems." In *Transdisciplinary Perspectives on Complex Systems*, edited by Franz-Josef Kahlen, Shannon Flumerfelt, and Anabela Alves. Springer. https://doi.org/10.1007/978-3-319-38756-7_4.

Gu, Qiao, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, et al. 2024. "ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning." *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 5021–28. https://doi.org/10.1109/ICRA57147.2024.10610243.

Gu, Xiuye, Tsung-Yi Lin, Weicheng Kuo, and Yin Cui. 2022. "Open-Vocabulary Object Detection via Vision and Language Knowledge Distillation." *International Conference on Learning Representations*.

Gumbel, Emil J. 1958. *Statistics of Extremes*. Columbia University Press.

Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. "On Calibration of Modern Neural Networks." *International Conference on Machine Learning (ICML)* 70: 1321–30.

Gurriet, Thomas, Mark Mote, Aaron D. Ames, and Eric Feron. 2018. "An Online Approach to Active Set Invariance." *2018 IEEE Conference on Decision and Control (CDC)*, 3592–99. https://doi.org/10.1109/CDC.2018.8619139.

GXO Logistics. 2024. *GXO Signs Industry-First Multi-Year Agreement with Agility Robotics*. Press release, 27 June 2024.

Haan, Pim de, Dinesh Jayaraman, and Sergey Levine. 2019. "Causal Confusion in Imitation Learning." *Advances in Neural Information Processing Systems (NeurIPS)* 32.

Haber, Stuart, and W. Scott Stornetta. 1991. "How to Time-Stamp a Digital Document." *Journal of Cryptology* 3 (2): 99–111. https://doi.org/10.1007/BF00196791.

Haddadin, Sami, Alin Albu-Schäffer, and Gerd Hirzinger. 2009. "Requirements for Safe Robots: Measurements, Analysis and New Insights." *The International Journal of Robotics Research* 28 (11–12): 1507–27. https://doi.org/10.1177/0278364909343970.

Hafner, Danijar, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. 2025. "Mastering Diverse Control Tasks Through World Models." *Nature* 640 (8059): 647–53. https://doi.org/10.1038/s41586-025-08744-2.

Hanley, James A., and Abby Lippman-Hand. 1983. "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators." *JAMA* 249 (13): 1743–45.

Hansen, Nikolaus, and Andreas Ostermeier. 2001. "Completely Derandomized Self-Adaptation in Evolution Strategies." *Evolutionary Computation* 9 (2): 159–95. https://doi.org/10.1162/106365601750190398.

Hargraves, Charles R., and Stephen W. Paris. 1987. "Direct Trajectory Optimization Using Nonlinear Programming and Collocation." *Journal of Guidance, Control, and Dynamics* 10 (4): 338–42. https://doi.org/10.2514/3.20223.

Harmonic Drive AG. 2014. *Engineering Data CSG-2UH Units*. Product documentation 1019642.

Harmonic Drive LLC. 2006. *CSF Mini Series Harmonic Drive Gearheads and Component Sets: Catalog*. Harmonic Drive LLC.

Harnad, Stevan. 1990. "The Symbol Grounding Problem." *Physica D: Nonlinear Phenomena* 42 (1-3): 335–46.

Hartley, Richard, and Andrew Zisserman. 2003. *Multiple View Geometry in Computer Vision*. Cambridge university press.

Hartmann, Daniel, Kristýna Hamříková, Aleš Vysocký, Vendula Laciok, and Aleš Bernatík. 2026. "Evolution of Safety Requirements in Industrial Robotics: Comparative Analysis of ISO 10218-1/2 (2011 Vs. 2025) and Integration of ISO/TS 15066." *arXiv Preprint arXiv:2602.17822*.

Hawkins, Richard, Colin Paterson, Chiara Picardi, Yan Jia, Radu Calinescu, and Ibrahim Habli. 2021. "Guidance on the Assurance of Machine Learning in Autonomous Systems (AMLAS)." *arXiv Preprint arXiv:2102.01564*.

He, Tairan, Jiawei Gao, Wenli Xiao, Yuanhang Zhang, Zi Wang, Jiashun Wang, Zhengyi Luo, et al. 2025. "ASAP: Aligning Simulation and Real-World Physics for Learning Agile Humanoid Whole-Body Skills." *Robotics: Science and Systems XXI*. https://doi.org/10.15607/RSS.2025.XXI.066.

Hein, Matthias, Maksym Andriushchenko, and Julian Bitterwolf. 2019. "Why ReLU Networks Yield High-Confidence Predictions Far Away from the Training Data and How to Mitigate the Problem." *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Hejna, Joey, Chethan Anand Bhateja, Yichen Jiang, Karl Pertsch, and Dorsa Sadigh. 2025. "ReMix: Optimizing Data Mixtures for Large Scale Imitation Learning." *Proceedings of the 8th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 270: 145–64.

Hennessy, John L., and David A. Patterson. 2019. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann.

Herlihy, Maurice. 1991. "Wait-Free Synchronization." *ACM Transactions on Programming Languages and Systems* 13 (1): 124–49. https://doi.org/10.1145/114005.102808.

Hermann, Robert, and Arthur J. Krener. 1977. "Nonlinear Controllability and Observability." *IEEE Transactions on Automatic Control* 22 (5): 728–40.

Hertz, Heinrich. 1894. *Die Prinzipien Der Mechanik in Neuem Zusammenhange Dargestellt*. Vol. 3. Gesammelte Werke. Johann Ambrosius Barth.

Hess, Wolfgang, Damon Kohler, Holger Rapp, and Daniel Andor. 2016. "Real-Time Loop Closure in 2D LIDAR SLAM." *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 1271–78. https://doi.org/10.1109/ICRA.2016.7487258.

Ho, Jonathan, Ajay Jain, and Pieter Abbeel. 2020. "Denoising Diffusion Probabilistic Models." *Advances in Neural Information Processing Systems (NeurIPS)* 33: 6840–51.

Hogan, Neville. 1985. "Impedance Control: An Approach to Manipulation: Part i–Theory, Part II–Implementation, Part III–Applications." *Journal of Dynamic Systems, Measurement, and Control* 107 (1): 1–24.

Hokayem, Peter F., and Mark W. Spong. 2006. "Bilateral Teleoperation: An Historical Survey." *Automatica* 42 (12): 2035–57. https://doi.org/10.1016/j.automatica.2006.06.027.

Hoque, Ryan, Ashwin Balakrishna, Ellen Novoseller, Albert Wilcox, Daniel S. Brown, and Ken Goldberg. 2022. "ThriftyDAgger: Budget-Aware Novelty and Risk Gating for Interactive Imitation Learning." *Proceedings of the Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 164: 598–608.

Hornung, Armin, Kai M Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. 2013. "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees." *Autonomous Robots* 34 (3): 189–206.

Howard, Ronald A. 1960. *Dynamic Programming and Markov Processes*. Technology Press of the Massachusetts Institute of Technology.

Hsu, Kai-Chieh, Haimin Hu, and Jaime F. Fisac. 2024. "The Safety Filter: A Unified View of Safety-Critical Control in Autonomous Systems." *Annual Review of Control, Robotics, and Autonomous Systems* 7: 47–72. https://doi.org/10.1146/annurev-control-071723-102940.

Hsueh, Mei-Chen, Timothy K. Tsai, and Ravishankar K. Iyer. 1997. "Fault Injection Techniques and Tools." *Computer* 30 (4): 75–82. https://doi.org/10.1109/2.585157.

Huang, Chao, Jiameng Fan, Xin Chen, Wenchao Li, and Qi Zhu. 2022. "POLAR: A Polynomial Arithmetic Framework for Verifying Neural-Network Controlled Systems." *Automated Technology for Verification and Analysis (ATVA)*, Lecture notes in computer science, 414–30. https://doi.org/10.1007/978-3-031-19992-9_27.

Huang, Wenlong, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. 2023. "VoxPoser: Composable 3D Value Maps for Everyday Robot Manipulation with Language Models." *Proceedings of the 7th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 229: 530–42.

Huber, Peter J. 1964. "Robust Estimation of a Location Parameter." *The Annals of Mathematical Statistics* 35 (1): 73–101. https://doi.org/10.1214/aoms/1177703732.

Hughes, Nathan, Yun Chang, and Luca Carlone. 2022. "Hydra: A Real-Time Spatial Perception Engine for 3D Scene Graph Construction and Optimization." *Proceedings of Robotics: Science and Systems (RSS)*.

Hunt, Kenneth H., and F. R. Erskine Crossley. 1975. "Coefficient of Restitution Interpreted as Damping in Vibroimpact." *Journal of Applied Mechanics* 42 (2): 440–45. https://doi.org/10.1115/1.3423596.

Hwangbo, Jemin, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. 2019. "Learning Agile and Dynamic Motor Skills for Legged Robots." *Science Robotics* 4 (26): eaau5872. https://doi.org/10.1126/scirobotics.aau5872.

Ichnowski, Jeffrey, Kaiyuan Chen, Karthik Dharmarajan, Simeon Adebola, Michael Danielczuk, Víctor Mayoral-Vilches, Nikhil Jha, et al. 2023. "FogROS2: An Adaptive Platform for Cloud and Fog Robotics Using ROS 2." *IEEE International Conference on Robotics and Automation (ICRA)*, 5493–500. https://doi.org/10.1109/ICRA48891.2023.10161307.

IEEE. 2002. *IEEE Std 1588-2002: IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. IEEE. https://doi.org/10.1109/IEEESTD.2002.94144.

IEEE. 2020a. *IEEE Std 1588-2019: IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. Institute of Electrical; Electronics Engineers. https://doi.org/10.1109/IEEESTD.2020.9120376.

IEEE. 2020b. *IEEE Std 802.1AS-2020: IEEE Standard for Local and Metropolitan Area Networks, Timing and Synchronization for Time-Sensitive Applications*. Institute of Electrical; Electronics Engineers.

International Electrotechnical Commission. 2007. *IEC 60085:2007: Electrical Insulation, Thermal Evaluation and Designation*. International Electrotechnical Commission.

International Electrotechnical Commission. 2010a. *IEC 61508:2010: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems*. International Electrotechnical Commission.

International Electrotechnical Commission. 2010b. *IEC 61508-2:2010: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems, Part 2: Requirements for Electrical/Electronic/Programmable Electronic Safety-Related Systems*. International Electrotechnical Commission.

International Electrotechnical Commission. 2014. *IEC 60825-1:2014: Safety of Laser Products, Part 1: Equipment Classification and Requirements*. International Electrotechnical Commission.

International Electrotechnical Commission. 2016a. *IEC 60204-1:2016: Safety of Machinery, Electrical Equipment of Machines, Part 1: General Requirements*. International Electrotechnical Commission.

International Electrotechnical Commission. 2016b. *IEC 61800-5-2:2016: Adjustable Speed Electrical Power Drive Systems, Part 5-2: Safety Requirements, Functional*. International Electrotechnical Commission.

International Electrotechnical Commission. 2017. *IEC 62745:2017: Safety of Machinery, Requirements for Cableless Control Systems of Machinery*. International Electrotechnical Commission.

International Electrotechnical Commission. 2021. *IEC 61784-3:2021: Industrial Communication Networks, Profiles, Part 3: Functional Safety Fieldbuses, General Rules and Profile Definitions*. International Electrotechnical Commission.

International Electrotechnical Commission. 2022. *IEC 60034-1:2022: Rotating Electrical Machines, Part 1: Rating and Performance*. International Electrotechnical Commission.

International Electrotechnical Commission. 2025. *IEC 61496-3:2025: Safety of Machinery, Electro-Sensitive Protective Equipment, Part 3: Particular Requirements for Active Opto-Electronic Protective Devices Responsive to Diffuse Reflection (AOPDDR)*. International Electrotechnical Commission.

International Organization for Standardization. 2010. *ISO 12100:2010: Safety of Machinery, General Principles for Design, Risk Assessment and Risk Reduction*. International Organization for Standardization.

International Organization for Standardization. 2015a. *ISO 11898-1:2015: Road Vehicles, Controller Area Network (CAN), Part 1: Data Link Layer and Physical Signalling*. International Organization for Standardization.

International Organization for Standardization. 2015b. *ISO 13850:2015: Safety of Machinery, Emergency Stop Function, Principles for Design*. International Organization for Standardization.

International Organization for Standardization. 2016. *ISO/TS 15066:2016: Robots and Robotic Devices, Collaborative Robots*. International Organization for Standardization.

International Organization for Standardization. 2018. *ISO 26262:2018: Road Vehicles, Functional Safety*. International Organization for Standardization.

International Organization for Standardization. 2022. *ISO 21448:2022: Road Vehicles, Safety of the Intended Functionality*. International Organization for Standardization.

International Organization for Standardization. 2023a. *ISO 13849-1:2023: Safety of Machinery, Safety-Related Parts of Control Systems, Part 1: General Principles for Design*. International Organization for Standardization.

International Organization for Standardization. 2023b. *ISO 3691-4:2023: Industrial Trucks, Safety Requirements and Verification, Part 4: Driverless Industrial Trucks and Their Systems*. International Organization for Standardization.

International Organization for Standardization. 2024a. *ISO 13855:2024: Safety of Machinery, Positioning of Safeguards with Respect to the Approach of the Human Body*. International Organization for Standardization.

International Organization for Standardization. 2024b. *ISO/PAS 8800:2024: Road Vehicles, Safety and Artificial Intelligence*. International Organization for Standardization.

International Organization for Standardization. 2025a. *ISO 10218-1:2025: Robotics, Safety Requirements, Part 1: Industrial Robots*. International Organization for Standardization.

International Organization for Standardization. 2025b. *ISO 10218-2:2025: Robotics, Safety Requirements, Part 2: Industrial Robot Applications and Robot Cells*. International Organization for Standardization.

Isaacs, Rufus. 1965. *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. Wiley.

Ivanov, Radoslav, James Weimer, Rajeev Alur, George J. Pappas, and Insup Lee. 2019. "Verisig: Verifying Safety Properties of Hybrid Systems with Neural Network Controllers." *Proceedings of the 22nd ACM International Conference on Hybrid Systems: Computation and Control (HSCC)*, 169–78. https://doi.org/10.1145/3302504.3311806.

Jacob, Benoit, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2704–13. https://doi.org/10.1109/cvpr.2018.00286.

Jastrzembski, Tiffany S., and Neil Charness. 2007. "The Model Human Processor and the Older Adult: Parameter Estimation and Validation Within a Mobile Phone Task." *Journal of Experimental Psychology: Applied* 13 (4): 224–48. https://doi.org/10.1037/1076-898X.13.4.224.

Jatavallabhula, Krishna Murthy, Alihusein Kuwajerwala, Qiao Gu, et al. 2023. "ConceptFusion: Open-Set Multimodal 3D Mapping." *Proceedings of Robotics: Science and Systems (RSS)*.

Jeon, Byeongguk, Seonghyeon Ye, JaeHyeok Doo, Sungdong Kim, Minjoon Seo, Hyungmok Son, and Kimin Lee. 2026. "RoboWorld: Fast and Reliable Neural Simulators for Generalist Robot Policy Evaluation." *arXiv Preprint arXiv:2607.01060*.

Jones, Donald R., Matthias Schonlau, and William J. Welch. 1998. "Efficient Global Optimization of Expensive Black-Box Functions." *Journal of Global Optimization* 13 (4): 455–92. https://doi.org/10.1023/A:1008306431147.

Jonker, Roy, and Anton Volgenant. 1987. "A Shortest Augmenting Path Algorithm for Dense and Sparse Linear Assignment Problems." *Computing* 38 (4): 325–40. https://doi.org/10.1007/BF02278710.

Joseph, Mathai, and Paritosh Pandya. 1986. "Finding Response Times in a Real-Time System." *The Computer Journal* 29 (5): 390–95. https://doi.org/10.1093/comjnl/29.5.390.

Julier, Simon J., and Jeffrey K. Uhlmann. 1997. "New Extension of the Kalman Filter to Nonlinear Systems." *Signal Processing, Sensor Fusion, and Target Recognition VI*, Proceedings of SPIE, vol. 3068: 182–93. https://doi.org/10.1117/12.280797.

Julier, Simon J., and Jeffrey K. Uhlmann. 2004. "Unscented Filtering and Nonlinear Estimation." *Proceedings of the IEEE* 92 (3): 401–22. https://doi.org/10.1109/JPROC.2003.823141.

Kaelbling, Leslie Pack, Michael L. Littman, and Anthony R. Cassandra. 1998. "Planning and Acting in Partially Observable Stochastic Domains." *Artificial Intelligence* 101 (1–2): 99–134. https://doi.org/10.1016/S0004-3702(98)00023-X.

Kaelbling, Leslie Pack, and Tomás Lozano-Pérez. 2011. "Hierarchical Task and Motion Planning in the Now." *IEEE International Conference on Robotics and Automation (ICRA)*, 1470–77. https://doi.org/10.1109/ICRA.2011.5980391.

Kahn, Herman, and Andrew W. Marshall. 1953. "Methods of Reducing Sample Size in Monte Carlo Computations." *Journal of the Operations Research Society of America* 1 (5): 263–78. https://doi.org/10.1287/opre.1.5.263.

Kajita, Shuuji, Fumio Kanehiro, Kenji Kaneko, Kazuhito Yokoi, and Hirohisa Hirukawa. 2001. "The 3D Linear Inverted Pendulum Mode: A Simple Modeling for a Biped Walking Pattern Generation." *Proceedings of the 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 1: 239–46. https://doi.org/10.1109/IROS.2001.973365.

Kalashnikov, Dmitry, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, et al. 2018. "Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation." *Proceedings of the 2nd Conference on Robot Learning*, Proceedings of machine learning research, vol. 87: 651–73.

Kalman, Rudolf E. 1960. "On the General Theory of Control Systems." *IFAC Proceedings Volumes* 1 (1): 491–502. https://doi.org/10.1016/S1474-6670(17)70094-8.

Kalman, Rudolph Emil. 1960. "A New Approach to Linear Filtering and Prediction Problems." *Journal of Basic Engineering* 82 (1): 35–45. https://doi.org/10.1115/1.3662552.

Kalra, Nidhi, and Susan M. Paddock. 2016a. "Driving to Safety: How Many Miles of Driving Would It Take to Demonstrate Autonomous Vehicle Reliability?" *Transportation Research Part A: Policy and Practice* 94: 182–93. https://doi.org/10.1016/j.tra.2016.09.010.

Kalra, Nidhi, and Susan M. Paddock. 2016b. *Driving to Safety: How Many Miles of Driving Would It Take to Demonstrate Autonomous Vehicle Reliability?* RR-1478-RC. RAND Corporation.

Kamath, Aishwarya, Mannat Singh, Yann LeCun, Gabriel Synnaeve, Ishan Misra, and Nicolas Carion. 2021. "Mdetr-Modulated Detection for End-to-End Multi-Modal Understanding." *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1780–90.

Kaplan, Edward L., and Paul Meier. 1958. "Nonparametric Estimation from Incomplete Observations." *Journal of the American Statistical Association* 53 (282): 457–81. https://doi.org/10.1080/01621459.1958.10501452.

Karaman, Sertac, and Emilio Frazzoli. 2011. "Sampling-Based Algorithms for Optimal Motion Planning." *The International Journal of Robotics Research* 30 (7): 846–94. https://doi.org/10.1177/0278364911406761.

Katz, Benjamin, Jared Di Carlo, and Sangbae Kim. 2019. "Mini Cheetah: A Platform for Dynamic Locomotion." *IEEE International Conference on Robotics and Automation (ICRA)*, 3295–301. https://doi.org/10.1109/ICRA.2019.8793865.

Katz, Guy, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. 2017. "Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks." *International Conference on Computer Aided Verification (CAV)*, 97–117. https://doi.org/10.1007/978-3-319-63387-9_5.

Kaul, Sanjit, Roy D. Yates, and Marco Gruteser. 2012. "Real-Time Status: How Often Should One Update?" *Proceedings of the IEEE Conference on Computer Communications (INFOCOM)*, 2731–35. https://doi.org/10.1109/INFCOM.2012.6195689.

Kaulen, Konstantin, Tobias Ladner, Stanley Bak, Christopher Brix, Hai Duong, Thomas Flinkow, Taylor T. Johnson, et al. 2025. "The 6th International Verification of Neural Networks Competition (VNN-COMP 2025): Summary and Results." *arXiv Preprint arXiv:2512.19007*.

Kavraki, Lydia E., Petr Švestka, Jean-Claude Latombe, and Mark H. Overmars. 1996. "Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces." *IEEE Transactions on Robotics and Automation* 12 (4): 566–80. https://doi.org/10.1109/70.508439.

Kelly, Michael, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J. Kochenderfer. 2019. "HG-DAgger: Interactive Imitation Learning with Human Experts." *IEEE International Conference on Robotics and Automation (ICRA)*, 8077–83. https://doi.org/10.1109/ICRA.2019.8793698.

Kelly, Timothy Patrick. 1998. "Arguing Safety: A Systematic Approach to Managing Safety Cases." PhD thesis, Department of Computer Science, University of York.

Kelly, Tim, and Rob Weaver. 2004. *The Goal Structuring Notation: A Safety Argument Notation*.

Kerbl, Bernhard, Georgios Kopanas, Thomas Leimk"uhler, and George Drettakis. 2023. "3D Gaussian Splatting for Real-Time Radiance Field Rendering." *ACM Transactions on Graphics* 42 (4): 139:1–14.

KernelNewbies. 2024. *Linux 6.12*. Kernel release summary.

Keyence Corporation. 2026. *SZ-V Series Safety Laser Scanner: Specifications*. Product specification page.

Khazatsky, Alexander, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, et al. 2024. "DROID: A Large-Scale in-the-Wild Robot Manipulation Dataset." *Robotics: Science and Systems (RSS)*. https://doi.org/10.15607/RSS.2024.XX.120.

Kim, Hyoseung, Dionisio de Niz, Björn Andersson, Mark Klein, Onur Mutlu, and Ragunathan Rajkumar. 2014. "Bounding Memory Interference Delay in COTS-Based Multi-Core Systems." *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 145–54. https://doi.org/10.1109/RTAS.2014.6925998.

Kim, Moo Jin, Chelsea Finn, and Percy Liang. 2025. "Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success." *Robotics: Science and Systems XXI*. https://doi.org/10.15607/RSS.2025.XXI.017.

Kim, Moo Jin, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, et al. 2024. "OpenVLA: An Open-Source Vision-Language-Action Model." *arXiv Preprint arXiv:2406.09246*.

Kirillov, Alexander, Eric Mintun, Nikhila Ravi, Hanzi Mao, et al. 2023. "Segment Anything." *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 4015–26.

Kirschner, Robin Jeanne, Nico Mansfeld, Saeed Abdolshah, and Sami Haddadin. 2022. "ISO/TS 15066: How Different Interpretations Affect Risk Assessment." *arXiv Preprint arXiv:2203.02706*.

Knight, John C., and Nancy G. Leveson. 1986. "An Experimental Evaluation of the Assumption of Independence in Multiversion Programming." *IEEE Transactions on Software Engineering* SE-12 (1): 96–109. https://doi.org/10.1109/TSE.1986.6312924.

Kochenderfer, Mykel J., Sydney M. Katz, Anthony L. Corso, and Robert J. Moss. 2026. *Algorithms for Validation*. MIT Press.

Koopman, Philip. 2022. *How Safe Is Safe Enough? Measuring and Predicting Autonomous Vehicle Safety*. Independently published.

Koopman, Philip. 2023. "UL 4600: What to Include in an Autonomous Vehicle Safety Case." *Computer* 56 (5): 101–4. https://doi.org/10.1109/MC.2023.3236171.

Koopman, Philip, and Michael Wagner. 2016. "Challenges in Autonomous Vehicle Testing and Validation." *SAE International Journal of Transportation Safety* 4 (1): 15–24. https://doi.org/10.4271/2016-01-0128.

Kopetz, Hermann. 2011. *Real-Time Systems: Design Principles for Distributed Embedded Applications*. 2nd ed. Real-Time Systems Series. Springer. https://doi.org/10.1007/978-1-4419-8237-7.

Kopetz, Hermann, and Johannes Reisinger. 1993. "The Non-Blocking Write Protocol NBW: A Solution to a Real-Time Synchronization Problem." *Proceedings of the 1993 Real-Time Systems Symposium*, 131–37. https://doi.org/10.1109/REAL.1993.393507.

Kostrikov, Ilya, Ashvin Nair, and Sergey Levine. 2021. "Offline Reinforcement Learning with Implicit Q-Learning." *arXiv Preprint arXiv:2110.06169*.

Kress-Gazit, Hadas, Kunimatsu Hashimoto, Naveen Kuppuswamy, Paarth Shah, Phoebe Horgan, Gordon Richardson, Siyuan Feng, and Benjamin Burchfiel. 2024. "Robot Learning as an Empirical Science: Best Practices for Policy Evaluation." *arXiv Preprint arXiv:2409.09491*.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 25.

Kröger, Torsten, and Friedrich M. Wahl. 2010. "Online Trajectory Generation: Basic Concepts for Instantaneous Reactions to Unforeseen Events." *IEEE Transactions on Robotics* 26 (1): 94–111. https://doi.org/10.1109/TRO.2009.2035744.

Krotkov, Eric, Douglas Hackett, Larry Jackel, Michael Perschbacher, James Pippine, Jesse Strauss, Gill Pratt, and Christopher Orlowski. 2017. "The DARPA Robotics Challenge Finals: Results and Perspectives." *Journal of Field Robotics* 34 (2): 229–40. https://doi.org/10.1002/rob.21683.

Kuhn, Harold W. 1955. "The Hungarian Method for the Assignment Problem." *Naval Research Logistics Quarterly* 2 (1–2): 83–97.

Kumar, Ashish, Zipeng Fu, Deepak Pathak, and Jitendra Malik. 2021. "RMA: Rapid Motor Adaptation for Legged Robots." *Robotics: Science and Systems XVII*. https://doi.org/10.15607/RSS.2021.XVII.011.

Kumar, Aviral, Aurick Zhou, George Tucker, and Sergey Levine. 2020. "Conservative q-Learning for Offline Reinforcement Learning." *Advances in Neural Information Processing Systems (NeurIPS)* 33: 1179–91.

Kunz, Tobias, and Mike Stilman. 2012. "Time-Optimal Trajectory Generation for Path Following with Bounded Acceleration and Velocity." *Robotics: Science and Systems (RSS)* VIII. https://doi.org/10.15607/RSS.2012.VIII.027.

Kuppusamy, Trishank Karthik, Akan Brown, Sebastien Awwad, Damon McCoy, Russ Bielawski, Cameron Mott, Sam Lauzon, André Weimerskirch, and Justin Cappos. 2016. *Uptane: Securing Software Updates for Automobiles*. Paper presented at the 14th Embedded Security in Cars Conference (escar Europe).

Kusano, Kristofer D., John M. Scanlon, Yin-Hsiu Chen, Timothy L. McMurry, Tilia Gode, and Trent Victor. 2025. "Comparison of Waymo Rider-Only Crash Rates by Crash Type to Human Benchmarks at 56.7 Million Miles." *Traffic Injury Prevention* 26 (sup1). https://doi.org/10.1080/15389588.2025.2499887.

Lamport, Leslie. 1977. "Concurrent Reading and Writing." *Communications of the ACM* 20 (11): 806–11. https://doi.org/10.1145/359863.359878.

Lamport, Leslie. 1978. "Time, Clocks, and the Ordering of Events in a Distributed System." *Communications of the ACM* 21 (7): 558–65. https://doi.org/10.1145/359545.359563.

Laskey, Michael, Jonathan Lee, Roy Fox, Anca Dragan, and Ken Goldberg. 2017. "DART: Noise Injection for Robust Imitation Learning." *Proceedings of the Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 78: 143–56.

LaValle, Steven M. 1998. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*. TR 98-11. Computer Science Department, Iowa State University.

Lee, Edward A. 2008. "Cyber Physical Systems: Design Challenges." *Proceedings of the 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)*, 363–69. https://doi.org/10.1109/ISORC.2008.25.

Lee, Edward A., and Sanjit A. Seshia. 2017. *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*. 2nd ed. MIT Press.

Lee, John D., and Katrina A. See. 2004. "Trust in Automation: Designing for Appropriate Reliance." *Human Factors: The Journal of the Human Factors and Ergonomics Society* 46 (1): 50–80. https://doi.org/10.1518/hfes.46.1.50_30392.

Lee, Joonho, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. 2020. "Learning Quadrupedal Locomotion over Challenging Terrain." *Science Robotics* 5 (47): eabc5986.

Lee, Kimin, Kibok Lee, Honglak Lee, and Jinwoo Shin. 2018. "A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks." *Advances in Neural Information Processing Systems (NeurIPS)* 31.

Lee, Ritchie, Ole J. Mengshoel, Anshu Saksena, Ryan W. Gardner, Daniel Genin, Joshua Silbermann, Michael Owen, and Mykel J. Kochenderfer. 2020. "Adaptive Stress Testing: Finding Likely Failure Events with Reinforcement Learning." *Journal of Artificial Intelligence Research* 69: 1165–201. https://doi.org/10.1613/jair.1.12190.

Leidecker, Henning, Lyudmyla Panashchenko, and Jay Brusse. 2011. *Electrical Failure of an Accelerator Pedal Position Sensor Caused by a Tin Whisker and Discussion of Investigative Techniques Used for Whisker Detection*. 5th International Tin Whisker Symposium.

Leveson, Nancy. 2004. "A New Accident Model for Engineering Safer Systems." *Safety Science* 42 (4): 237–70. https://doi.org/10.1016/S0925-7535(03)00047-X.

Leveson, Nancy. 2011. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.

Leveson, Nancy G., and Clark S. Turner. 1993. "An Investigation of the Therac-25 Accidents." *Computer* 26 (7): 18–41.

Levine, Sergey, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. 2016. "End-to-End Training of Deep Visuomotor Policies." *Journal of Machine Learning Research* 17 (39): 1–40.

Levine, Sergey, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. 2018. "Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection." *The International Journal of Robotics Research* 37 (4-5): 421–36.

Li, Haozhan, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, et al. 2025. "SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning." *arXiv Preprint arXiv:2509.09674*.

Li, Peng, Zhenguo Nie, Zihao Li, and Xinjun Liu. 2024. "Optimal Design of the Modular Joint Drive Train for Enhancing Cobot Load Capacity and Dynamic Performance." *Chinese Journal of Mechanical Engineering* 37 (1): 57. https://doi.org/10.1186/s10033-024-01041-5.

Li, Xinyue, Zhenpeng Chen, Jie M. Zhang, Federica Sarro, Ying Zhang, and Xuanzhe Liu. 2025. "Bias Behind the Wheel: Fairness Testing of Autonomous Driving Systems." *ACM Transactions on Software Engineering and Methodology* 34 (3): 1–24. https://doi.org/10.1145/3702989.

Li, Xuanlin, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Walke, Chuer Pan, et al. 2024. "SIMPLER: Simulated Evaluation of Manipulator Policy with Real-World Robot Environments." *Proceedings of the 8th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research.

Li, Yau-Tsun Steven, and Sharad Malik. 1995. "Performance Analysis of Embedded Software Using Implicit Path Enumeration." *Proceedings of the 32nd ACM/IEEE Design Automation Conference (DAC)*, 456–61. https://doi.org/10.1145/217474.217570.

Liang, Jacky, Wenlong Huang, Fei Xia, et al. 2023. "Code as Policies: Language Model Programs for Embodied Control." *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 9493–500.

Lichtsteiner, Patrick, Christoph Posch, and Tobi Delbruck. 2008. "A 128 $\times$ 128 120 dB 15 μs Latency Asynchronous Temporal Contrast Vision Sensor." *IEEE Journal of Solid-State Circuits* 43 (2): 566–76. https://doi.org/10.1109/JSSC.2007.914337.

Lin, Fanqi, Yingdong Hu, Pingyue Sheng, Chuan Wen, Jiacheng You, and Yang Gao. 2025. "Data Scaling Laws in Imitation Learning for Robotic Manipulation." *International Conference on Learning Representations (ICLR)*, 54877–910.

Lin, Ji, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. "AWQ: Activation-Aware Weight Quantization for on-Device LLM Compression and Acceleration." *Proceedings of Machine Learning and Systems* 6: 87–100.

Lipman, Yaron, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nicklas, and Matt Le. 2022. "Flow Matching for Generative Modeling." *arXiv Preprint arXiv:2210.02747*.

Lipson, Lahav, Zachary Teed, and Jia Deng. 2021. "RAFT-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching." *International Conference on 3D Vision (3DV)*, 218–27. https://doi.org/10.1109/3DV53792.2021.00032.

Liu, Bo, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023. "LLM+P: Empowering Large Language Models with Optimal Planning Proficiency." *arXiv Preprint arXiv:2304.11477*.

Liu, C. L., and James W. Layland. 1973. "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment." *Journal of the ACM* 20 (1): 46–61.

Liu, Huihan, Soroush Nasiriany, Lance Zhang, Zhiyao Bao, and Yuke Zhu. 2023. "Robot Learning on the Job: Human-in-the-Loop Autonomy and Learning During Deployment." *Robotics: Science and Systems (RSS)*. https://doi.org/10.15607/RSS.2023.XIX.005.

Liu, Peiqi, Yaswanth Orru, Jay Vakil, Chris Paxton, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. 2024. "OK-Robot: What Really Matters in Integrating Open-Knowledge Models for Robotics." *arXiv Preprint arXiv:2401.12202*.

Liu, Shilong, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jianwei Yang, Chunyuan Li, et al. 2023. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." *arXiv Preprint arXiv:2303.05499*.

Liu, Weitang, Xiaoyun Wang, John D. Owens, and Yixuan Li. 2020. "Energy-Based Out-of-Distribution Detection." *Advances in Neural Information Processing Systems (NeurIPS)* 33: 21464–75.

Liu, Yuejiang, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. 2024. "Bidirectional Decoding: Improving Action Chunking via Guided Test-Time Sampling." *arXiv Preprint arXiv:2408.17355*.

Liu, Zhijian, Haotian Tang, Alexander Amini, Xinyu Yang, Huizi Mao, Daniela Rus, and Song Han. 2023. "BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation." *IEEE International Conference on Robotics and Automation (ICRA)*, 2774–81.

Ljung, Lennart. 1999. *System Identification: Theory for the User*. 2nd ed. Prentice Hall.

Lozano-Pérez, Tomás. 1983. "Spatial Planning: A Configuration Space Approach." *IEEE Transactions on Computers* C-32 (2): 108–20. https://doi.org/10.1109/TC.1983.1676196.

Lu, Jingpei, Zekai Liang, Tristin Xie, Florian Richter, Shan Lin, Sainan Liu, and Michael C. Yip. 2025. "CtRNet-X: Camera-to-Robot Pose Estimation in Real-World Conditions Using a Single Camera." *IEEE International Conference on Robotics and Automation (ICRA)*, 1914–20. https://doi.org/10.1109/ICRA55743.2025.11128065.

Luh, John Y. S., Michael W. Walker, and Richard P. C. Paul. 1980. "On-Line Computational Scheme for Mechanical Manipulators." *Journal of Dynamic Systems, Measurement, and Control* 102 (2): 69–76. https://doi.org/10.1115/1.3149599.

Lundberg, Gustaf, and Arvid Palmgren. 1949. "Dynamic Capacity of Rolling Bearings." *Journal of Applied Mechanics* 16 (2): 165–72. https://doi.org/10.1115/1.4009930.

Lundqvist, Thomas, and Per Stenström. 1999. "Timing Anomalies in Dynamically Scheduled Microprocessors." *IEEE Real-Time Systems Symposium (RTSS)*, 12–21. https://doi.org/10.1109/REAL.1999.818824.

Luo, Haocong, Yahya Can Tuğrul, F. Nisa Bostancı, Ataberk Olgun, A. Giray Yağlıkçı, and Onur Mutlu. 2024. "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator." *IEEE Computer Architecture Letters* 23 (1): 112–16. https://doi.org/10.1109/LCA.2023.3333759.

Luo, Jianlan, Perry Dong, Yuexiang Zhai, Yi Ma, and Sergey Levine. 2024. "RLIF: Interactive Imitation Learning as Reinforcement Learning." *International Conference on Learning Representations (ICLR)*.

Luo, Jianlan, Charles Xu, Jeffrey Wu, and Sergey Levine. 2025. "Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning." *Science Robotics* 10 (105): eads5033. https://doi.org/10.1126/scirobotics.ads5033.

Lynch, Kevin M, and Frank C Park. 2017. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press.

Macenski, Steve, Tom Moore, David V. Lu, Alexey Merzlyakov, and Michael Ferguson. 2023. "From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2." *Robotics and Autonomous Systems* 168: 104493. https://doi.org/10.1016/j.robot.2023.104493.

Macenski, Steven, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. 2022. "Robot Operating System 2: Design, Architecture, and Uses in the Wild." *Science Robotics* 7 (66): eabm6074. https://doi.org/10.1126/scirobotics.abm6074.

Maddukuri, Abhiram, Zhenyu Jiang, Lawrence Yunliang Chen, Soroush Nasiriany, Yuqi Xie, Yu Fang, Wenqi Huang, et al. 2025. "Sim-and-Real Co-Training: A Simple Recipe for Vision-Based Robotic Manipulation." *Robotics: Science and Systems (RSS)*. https://doi.org/10.15607/RSS.2025.XXI.109.

Madry, Aleksander, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. "Towards Deep Learning Models Resistant to Adversarial Attacks." *International Conference on Learning Representations (ICLR)*.

Maimone, Mark, Yang Cheng, and Larry Matthies. 2007. "Two Years of Visual Odometry on the Mars Exploration Rovers." *Journal of Field Robotics* 24 (3): 169–86. https://doi.org/10.1002/rob.20184.

Main Commission Aircraft Accident Investigation Warsaw and German Federal Bureau of Aircraft Accident Investigation. 1994. *Report on the Accident to Airbus A320-211 Aircraft in Warsaw on 14 September 1993*. Polish Ministry of Transport; Maritime Economy / BFU.

Makoviychuk, Viktor, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, et al. 2021. "Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning." *arXiv Preprint arXiv:2108.10470*.

Maler, Oded, and Dejan Nickovic. 2004. "Monitoring Temporal Properties of Continuous Signals." *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT)*, Lecture notes in computer science, vol. 3253: 152–66. https://doi.org/10.1007/978-3-540-30206-3_12.

Mancuso, Renato, Roman Dudko, Emiliano Betti, Marco Cesati, Marco Caccamo, and Rodolfo Pellizzoni. 2013. "Real-Time Cache Management Framework for Multi-Core Architectures." *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 45–54. https://doi.org/10.1109/RTAS.2013.6531078.

Mandlekar, Ajay, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. 2023. "MimicGen: A Data Generation System for Scalable Robot Learning Using Human Demonstrations." *Proceedings of the Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 229: 1820–64.

Mandlekar, Ajay, Danfei Xu, Roberto Martín-Martín, Yuke Zhu, Li Fei-Fei, and Silvio Savarese. 2020. "Human-in-the-Loop Imitation Learning Using Remote Teleoperation." *arXiv Preprint arXiv:2012.06733*.

Mandlekar, Ajay, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. 2021. "What Matters in Learning from Offline Human Demonstrations for Robot Manipulation." *Proceedings of the 5th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 164: 1678–90.

Mangel, Marc, and Francisco J. Samaniego. 1984. "Abraham Wald's Work on Aircraft Survivability." *Journal of the American Statistical Association* 79 (386): 259–67. https://doi.org/10.1080/01621459.1984.10478038.

Marcucci, Tobia, Mark Petersen, David von Wrangel, and Russ Tedrake. 2023. "Motion Planning Around Obstacles with Convex Optimization." *Science Robotics* 8 (84): eadf7843. https://doi.org/10.1126/scirobotics.adf7843.

Maricopa County Attorney's Office. 2023. *Driver of Uber Vehicle Involved in Death of Woman in Tempe Pleads Guilty*.

Martins, José, Adriano Tavares, Marco Solieri, Marko Bertogna, and Sandro Pinto. 2020. "Bao: A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems." *Workshop on Next Generation Real-Time Embedded Systems (NG-RES)*, OASIcs, vol. 77: 3:1–14. https://doi.org/10.4230/OASIcs.NG-RES.2020.3.

Marvel, Jeremy A., and Rick Norcross. 2017. "Implementing Speed and Separation Monitoring in Collaborative Robot Workcells." *Robotics and Computer-Integrated Manufacturing* 44: 144–55. https://doi.org/10.1016/j.rcim.2016.08.001.

Matsuki, Hidenobu, Ryo Murai, Paul H. J. Kelly, and Andrew J. Davison. 2024. "Gaussian Splatting SLAM." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 18039–48.

Matthias, Andreas. 2004. "The Responsibility Gap: Ascribing Responsibility for the Actions of Learning Automata." *Ethics and Information Technology* 6 (3): 175–83. https://doi.org/10.1007/s10676-004-3422-1.

Mayer, Hans C., and Rouslan Krechetnikov. 2012. "Walking with Coffee: Why Does It Spill?" *Physical Review E* 85 (4): 046117. https://doi.org/10.1103/PhysRevE.85.046117.

Mayne, David Q., James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. 2000. "Constrained Model Predictive Control: Stability and Optimality." *Automatica* 36 (6): 789–814. https://doi.org/10.1016/S0005-1098(99)00214-9.

Maynez, Joshua, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. "On Faithfulness and Factuality in Abstractive Summarization." *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1906–19. https://doi.org/10.18653/v1/2020.acl-main.173.

McDermott, Drew, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. 1998. *PDDL: The Planning Domain Definition Language*. CVC TR-98-003/DCS TR-1165. Yale Center for Computational Vision; Control.

McGee, Leonard A., and Stanley F. Schmidt. 1985. *Discovery of the Kalman Filter as a Practical Tool for Aerospace and Industry*. NASA TM-86847. NASA Ames Research Center.

Mead, C. 1990. "Neuromorphic Electronic Systems." *Proceedings of the IEEE* 78 (10): 1629–36. https://doi.org/10.1109/5.58356.

MEAN WELL. 2026. *DDR-240 Series 240 W DIN Rail Type DC-DC Converter Specification*. Product datasheet.

Meeussen, Wim. 2010. *REP 105: Coordinate Frames for Mobile Platforms*. ROS Enhancement Proposal.

Merat, Natasha, A. Hamish Jamson, Frank C. H. Lai, Michael Daly, and Oliver M. J. Carsten. 2014. "Transition to Manual: Driver Behaviour When Resuming Control from a Highly Automated Vehicle." *Transportation Research Part F: Traffic Psychology and Behaviour* 27: 274–82. https://doi.org/10.1016/j.trf.2014.09.005.

Merkle, Ralph C. 1988. "A Digital Signature Based on a Conventional Encryption Function." *Advances in Cryptology — CRYPTO '87*, Lecture notes in computer science, vol. 293: 369–78. https://doi.org/10.1007/3-540-48184-2_32.

Millane, Alexander, Helen Oleynikova, Emilie Wirbel, Remo Steiner, Vikram Ramasamy, David Tingdahl, and Roland Siegwart. 2024. “Nvblox: GPU-Accelerated Incremental Signed Distance Field Mapping.” *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2698–705. https://doi.org/10.1109/ICRA57147.2024.10611532.

Minderer, Matthias, Alexey Gritsenko, Austin Stone, Maxim Neumann, Dirk Weissenborn, Alexey Dosovitskiy, et al. 2022. “Simple Open-Vocabulary Object Detection.” *Computer Vision – ECCV 2022*, Lecture notes in computer science, 728–55. https://doi.org/10.1007/978-3-031-20080-9_42.

Miner, Milton A. 1945. “Cumulative Damage in Fatigue.” *Journal of Applied Mechanics* 12 (3): A159–64. https://doi.org/10.1115/1.4009458.

Mitchell, Ian M., Alexandre M. Bayen, and Claire J. Tomlin. 2005. “A Time-Dependent Hamilton-Jacobi Formulation of Reachable Sets for Continuous Dynamic Games.” *IEEE Transactions on Automatic Control* 50 (7): 947–57. https://doi.org/10.1109/TAC.2005.851439.

Mobile Industrial Robots. 2025. *MiR250 Specifications*. Product specification.

Montsinger, V. M. 1930. “Loading Transformers by Temperature.” *Transactions of the American Institute of Electrical Engineers* 49 (2): 776–90. https://doi.org/10.1109/T-AIEE.1930.5055572.

Moravec, Hans. 1988. *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press.

Moravec, Hans, and Alberto Elfes. 1985. “High Resolution Maps from Wide Angle Sonar.” *IEEE International Conference on Robotics and Automation* 2: 116–21.

MoveIt 2 Documentation. 2026. *Time Parameterization*. MoveIt 2 tutorials, PickNik Robotics.

Museth, Ken. 2013. “VDB: High-Resolution Sparse Volumes with Dynamic Topology.” *ACM Transactions on Graphics* 32 (3): 1–22. https://doi.org/10.1145/2487228.2487235.

Mušicki, Darko, Robin Evans, and Srdjan Stanković. 1994. “Integrated Probabilistic Data Association.” *IEEE Transactions on Automatic Control* 39 (6): 1237–41. https://doi.org/10.1109/9.293185.

Musser, C. Walton. 1959. *Strain Wave Gearing*. U.S. Patent 2,906,143, filed March 21, 1955, issued September 29, 1959.

Nagumo, Mitio. 1942. “Über Die Lage Der Integralkurven Gewöhnlicher Differentialgleichungen.” *Proceedings of the Physico-Mathematical Society of Japan, 3rd Series* 24: 551–59. https://doi.org/10.11429/ppmsj1919.24.0_551.

Nakamoto, Mitsuhiko, Simon Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. 2023. “Cal-QL: Calibrated Offline RL Pre-Training for Efficient Online Fine-Tuning.” *Advances in Neural Information Processing Systems 36*, 62244–69. https://doi.org/10.52202/075280-2719.

Nakamura, Yoshihiko, and Hideo Hanafusa. 1986. “Inverse Kinematic Solutions with Singularity Robustness for Robot Manipulator Control.” *Journal of Dynamic Systems, Measurement, and Control* 108 (3): 163–71. https://doi.org/10.1115/1.3143764.

NAMUR. 2021. *NE 43: Standardization of the Signal Level for the Failure Information of Digital Transmitters*. NAMUR, User Association of Automation Technology in Process Industries.

NASA Engineering and Safety Center. 2011. *Technical Support to the NHTSA on the Reported Toyota Motor Corporation (TMC) Unintended Acceleration Investigation*. National Aeronautics; Space Administration.

National Highway Traffic Safety Administration. 2021. *Standing General Order 2021-01: Incident Reporting Requirements for Automated Driving Systems and Level 2 Advanced Driver Assistance Systems*.

National Highway Traffic Safety Administration. 2024a. *Event Data Recorders: Final Rule (49 CFR Part 563)*. 89 FR 102810. U.S. Department of Transportation.

National Highway Traffic Safety Administration. 2024b. *NHTSA Announces Consent Order with Cruise After Company Failed to Fully Report Crash Involving Pedestrian*.

National Highway Traffic Safety Administration (NHTSA). 2017. *PE 16-007: Investigation of Tesla Motors Model S Automatic Emergency Braking and Autopilot Systems*. PE 16-007. Office of Defects Investigation (ODI), U.S. Department of Transportation.

National Transportation Safety Board. 2017. *Collision Between a Car Operating with Automated Vehicle Control Systems and a Tractor-Semitrailer Truck Near Williston, Florida, May 7, 2016*. HAR-17/02. National Transportation Safety Board.

National Transportation Safety Board. 2019. *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018*. HAR-19/03. National Transportation Safety Board.

Newcombe, Richard A, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. 2011. “KinectFusion: Real-Time Dense Surface Mapping and Tracking.” *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 127–36.

Ng, Andrew Y., Daishi Harada, and Stuart J. Russell. 1999. “Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping.” *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 278–87.

Nguyen, Van-Duc. 1988. "Constructing Force-Closure Grasps." *The International Journal of Robotics Research* 7 (3): 3–16. https://doi.org/10.1177/027836498800700301.

Niemeyer, Günter, and Jean-Jacques E. Slotine. 1991. "Stable Adaptive Teleoperation." *IEEE Journal of Oceanic Engineering* 16 (1): 152–62. https://doi.org/10.1109/48.64895.

Nießner, Matthias, Michael Zollhöfer, Shahram Izadi, and Marc Stamminger. 2013. "Real-Time 3D Reconstruction at Scale Using Voxel Hashing." *ACM Transactions on Graphics* 32 (6): 169:1–11. https://doi.org/10.1145/2508363.2508374.

Nilsson, Nils J. 1984. *Shakey the Robot*. Technical Note No. 323. Artificial Intelligence Center, SRI International.

Nissenbaum, Helen. 1996. "Accountability in a Computerized Society." *Science and Engineering Ethics* 2 (1): 25–42. https://doi.org/10.1007/BF02639315.

Nissenbaum, Helen. 2009. *Privacy in Context*. Stanford University Press. https://doi.org/10.1515/9780804772891.

NVIDIA. 2023a. *Cortex-R52 and Cortex-R5 Cores in Jetson AGX Orin*.

NVIDIA. 2023b. *Functional Safety Island (FSI)*.

NVIDIA. 2026a. *Functional Safety Island in Jetson Thor T5000 Module*.

NVIDIA. 2026b. *Nvblox Technical Details*. Isaac ROS documentation.

NVIDIA. 2026c. *What Is Physical AI?* NVIDIA Glossary.

NVIDIA, Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, et al. 2025. "Cosmos World Foundation Model Platform for Physical AI." *arXiv Preprint arXiv:2501.03575*.

NVIDIA, Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, et al. 2025. "Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning." *arXiv Preprint arXiv:2511.04831*.

Nyquist, Harry. 1928. "Certain Topics in Telegraph Transmission Theory." *Transactions of the American Institute of Electrical Engineers* 47 (2): 617–44. https://doi.org/10.1109/T-AIEE.1928.5055024.

Nyquist, Harry. 1932. "Regeneration Theory." *Bell System Technical Journal* 11 (1): 126–47. https://doi.org/10.1002/j.1538-7305.1932.tb02344.x.

O'Kelly, Matthew, Aman Sinha, Hongseok Namkoong, John Duchi, and Russ Tedrake. 2018. "Scalable End-to-End Autonomous Vehicle Testing via Rare-Event Simulation." *Advances in Neural Information Processing Systems (NeurIPS)* 31.

Object Management Group. 2015. *Data Distribution Service (DDS), Version 1.4*. Object Management Group.

Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees Nair, et al. 2024. "Octo: An Open-Source Generalist Robot Policy." *arXiv Preprint arXiv:2404.08839*.

Oleynikova, Helen, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto. 2017. "Voxblox: Incremental 3D Euclidean Signed Distance Fields for on-Board MAV Planning." *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1366–73.

Open X-Embodiment Collaboration, Abby O'Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, et al. 2024. "Open X-Embodiment: Robotic Learning Datasets and RT-X Models." *IEEE International Conference on Robotics and Automation (ICRA)*, 6892–903. https://doi.org/10.1109/ICRA57147.2024.10611477.

OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, et al. 2019. "Solving Rubik's Cube with a Robot Hand." *arXiv Preprint arXiv:1910.07113*.

Oquab, Maxime, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, et al. 2023. "Dinov2: Learning Robust Visual Features Without Supervision." *arXiv Preprint arXiv:2304.07193*.

Orzechowski, Piotr F., Annika Meyer, and Martin Lauer. 2018. "Tackling Occlusions & Limited Sensor Range with Set-Based Safety Verification." *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 1729–36. https://doi.org/10.1109/ITSC.2018.8569332.

Ouster, Inc. 2025. *OS1: Mid-Range High-Resolution Imaging Lidar, Datasheet, Hardware Rev7, Firmware 3.1*. Product datasheet.

Owen, Art B. 2013. *Monte Carlo Theory, Methods and Examples*. Online book.

Owens, Daniel, Rabih El Khatib, Mojtaba Bisheh-Niasar, Reza Azarderakhsh, and Mehran Mozaffari Kermani. 2024. "Efficient and Side-Channel Resistant Ed25519 on ARM Cortex-M4." *IEEE Transactions on Circuits and Systems I: Regular Papers* 71 (6): 2674–86. https://doi.org/10.1109/TCSI.2024.3384414.

Parasuraman, Raja, and Dietrich H. Manzey. 2010. "Complacency and Bias in Human Use of Automation: An Attentional Integration." *Human Factors* 52 (3): 381–410. https://doi.org/10.1177/0018720810376055.

Parasuraman, Raja, Thomas B. Sheridan, and Christopher D. Wickens. 2000. "A Model for Types and Levels of Human Interaction with Automation." *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 30 (3): 286–97. https://doi.org/10.1109/3468.844354.

Pellizzoni, Rodolfo, Emiliano Betti, Stanley Bak, Gang Yao, John Criswell, Marco Caccamo, and Russell Kegley. 2011. "A Predictable Execution Model for COTS-Based Embedded Systems." *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 269–79. https://doi.org/10.1109/RTAS.2011.33.

Peng, Xue Bin, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. 2018. "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization." *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 3803–10. https://doi.org/10.1109/ICRA.2018.8460528.

Pertsch, Karl, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. 2025. "FAST: Efficient Action Tokenization for Vision-Language-Action Models." *arXiv Preprint arXiv:2501.09747*.

Pham, Hung, and Quang-Cuong Pham. 2018. "A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis." *IEEE Transactions on Robotics* 34 (3): 645–59. https://doi.org/10.1109/TRO.2018.2819195.

Philion, Jonah, and Sanja Fidler. 2020. "Lift, Splat, Shoot: Encoding Images from Arbitrary Cameras by Predicting Depth." *Proceedings of the European Conference on Computer Vision (ECCV)*, 272–85.

Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, et al. 2025. "$\pi^*_{0.6}$: A VLA That Learns from Experience." *arXiv Preprint arXiv:2511.14759*.

Pickands, James, III. 1975. "Statistical Inference Using Extreme Order Statistics." *The Annals of Statistics* 3 (1): 119–31. https://doi.org/10.1214/aos/1176343003.

Pinto, Lerrel, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. 2018. "Asymmetric Actor Critic for Image-Based Robot Learning." *Robotics: Science and Systems XIV*. https://doi.org/10.15607/RSS.2018.XIV.008.

Pnueli, Amir. 1977. "The Temporal Logic of Programs." *18th Annual Symposium on Foundations of Computer Science (SFCS)*, 46–57. https://doi.org/10.1109/SFCS.1977.32.

Pomerleau, Dean A. 1989. "ALVINN: An Autonomous Land Vehicle in a Neural Network." *Advances in Neural Information Processing Systems (NeurIPS)* 1: 305–13.

Pomerleau, Dean A. 1991. "Efficient Training of Artificial Neural Networks for Autonomous Navigation." *Neural Computation* 3 (1): 88–97. https://doi.org/10.1162/neco.1991.3.1.88.

Pope, Reiner, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. "Efficiently Scaling Transformer Inference." *Proceedings of Machine Learning and Systems (MLSys)* 5: 606–24.

Popper, Karl R. 1959. *The Logic of Scientific Discovery*. Hutchinson.

Prajna, Stephen, and Ali Jadbabaie. 2004. "Safety Verification of Hybrid Systems Using Barrier Certificates." *Hybrid Systems: Computation and Control (HSCC)*, Lecture notes in computer science, vol. 2993: 477–92. https://doi.org/10.1007/978-3-540-24743-2_32.

Pratt, Gill A., and Matthew M. Williamson. 1995. "Series Elastic Actuators." *Proceedings of the 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 1: 399–406. https://doi.org/10.1109/IROS.1995.525827.

Pratt, Jerry, John Carff, Sergey Drakunov, and Ambarish Goswami. 2006. "Capture Point: A Step Toward Humanoid Push Recovery." *Proceedings of the 6th IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, 200–207. https://doi.org/10.1109/ICHR.2006.321385.

Presidential Commission on the Space Shuttle Challenger Accident. 1986. *Report of the Presidential Commission on the Space Shuttle Challenger Accident*. U.S. Government Printing Office.

Puschner, Peter, and Christian Koza. 1989. "Calculating the Maximum Execution Time of Real-Time Programs." *Real-Time Systems* 1 (2): 159–76. https://doi.org/10.1007/BF00571421.

Qin, S. Joe, and Thomas A. Badgwell. 2003. "A Survey of Industrial Model Predictive Control Technology." *Control Engineering Practice* 11 (7): 733–64. https://doi.org/10.1016/S0967-0661(02)00186-7.

Quality Control Systems Corp. 2019. *NHTSA's Implausible Safety Claim for Tesla's Autosteer Driver Assistance System*. Quality Control Systems Corp.

Quinn Emanuel Urquhart and Sullivan, LLP. 2024. *Report to the Boards of Directors of Cruise LLC, GM Cruise Holdings LLC, and General Motors Holdings LLC Regarding the October 2, 2023 Accident in San Francisco*. Quinn Emanuel Urquhart; Sullivan, LLP.

Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. "Learning Transferable Visual Models from Natural Language Supervision." *International Conference on Machine Learning (ICML)*, 8748–63.

Ratliff, Nathan, Matt Zucker, J. Andrew Bagnell, and Siddhartha Srinivasa. 2009. "CHOMP: Gradient Optimization Techniques for Efficient Motion Planning." *IEEE International Conference on Robotics and Automation (ICRA)*, 489–94. https://doi.org/10.1109/ROBOT.2009.5152817.

Ravi, Nikhila, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, et al. 2024. "SAM 2: Segment Anything in Images and Videos." *arXiv Preprint arXiv:2408.00714*.

Reddi, Vijay Janapa, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. 2019. "MLPerf Inference Benchmark." *arXiv Preprint arXiv:1911.02549*.

Reeves, Glenn E. 1997. *What Really Happened on Mars?* Electronic mail message, Jet Propulsion Laboratory.

Ren, Allen Z., Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, et al. 2023. "Robots That Ask for Help: Uncertainty Alignment for Large Language Model Planners." *Proceedings of the 7th Conference on Robot Learning*, Proceedings of machine learning research, vol. 229: 661–82.

Rixner, Scott, William J. Dally, Ujval J. Kapasi, Peter Mattson, and John D. Owens. 2000. "Memory Access Scheduling." *International Symposium on Computer Architecture (ISCA)*, 128–38. https://doi.org/10.1145/339647.339668.

Robert Bosch GmbH. 2012. *CAN with Flexible Data-Rate: Specification Version 1.0*. Robert Bosch GmbH.

Robey, Alexander, Zachary Ravichandran, Vijay Kumar, Hamed Hassani, and George J. Pappas. 2025. "Jailbreaking LLM-Controlled Robots." *IEEE International Conference on Robotics and Automation (ICRA)*, 11948–56. https://doi.org/10.1109/ICRA55743.2025.11128119.

ros2_control Development Team. 2025. *Realtime_tools: Tools for Hard Real-Time Threads in Ros2_control*. GitHub repository.

Rosinol, Antoni, Arjun Gupta, Marcus Abate, Jingnan Shi, and Luca Carlone. 2020. "3D Dynamic Scene Graphs: Actionable Spatial Perception with Places, Objects, and Humans." *Robotics: Science and Systems XVI*. https://doi.org/10.15607/RSS.2020.XVI.079.

Rosinol, Antoni, Andrew Violette, Marcus Abate, Nathan Hughes, Yun Chang, Jingnan Shi, Arjun Gupta, and Luca Carlone. 2021. "Kimera: From SLAM to Spatial Perception with 3D Dynamic Scene Graphs." *The International Journal of Robotics Research* 40 (12–14): 1510–46. https://doi.org/10.1177/02783649211056674.

Ross, Stéphane, and J. Andrew Bagnell. 2010. "Efficient Reductions for Imitation Learning." *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)* 9: 661–68.

Ross, Stéphane, Geoffrey Gordon, and J. Andrew Bagnell. 2011. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)* 15: 627–35.

Rubinstein, Reuven Y., and Dirk P. Kroese. 2004. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Information Science and Statistics. Springer. https://doi.org/10.1007/978-1-4757-4321-0.

Rudin, Nikita, David Hoeller, Philipp Reist, and Marco Hutter. 2022. "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning." *Proceedings of the 5th Conference on Robot Learning*, Proceedings of machine learning research, vol. 164: 91–100.

Rushby, John. 1999. *Partitioning in Avionics Architectures: Requirements, Mechanisms, and Assurance*. NASA/CR-1999-209347. NASA Langley Research Center.

Saarinen, Jari, Henrik Andreasson, and Achim J. Lilienthal. 2012. "Independent Markov Chain Occupancy Grid Maps for Representation of Dynamic Environment." *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3489–95. https://doi.org/10.1109/IROS.2012.6385629.

Safety Research & Strategies. 2013. *Toyota Unintended Acceleration and the Big Bowl of "Spaghetti" Code*.

Sajjan, Shreeyak, Matthew Moore, Mike Pan, Ganesh Nagaraja, Johnny Lee, Andy Zeng, and Shuran Song. 2020. "ClearGrasp: 3D Shape Estimation of Transparent Objects for Manipulation." *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 3634–42. https://doi.org/10.1109/ICRA40945.2020.9197518.

Sakurai, Takayasu, and A. Richard Newton. 1990. "Alpha-Power Law MOSFET Model and Its Applications to CMOS Inverter Delay and Other Formulas." *IEEE Journal of Solid-State Circuits* 25 (2): 584–94. https://doi.org/10.1109/4.52187.

Samuel, Justin, Nick Mathewson, Justin Cappos, and Roger Dingledine. 2010. "Survivable Key Compromise in Software Update Systems." *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, 61–72. https://doi.org/10.1145/1866307.1866315.

Santoni de Sio, Filippo, and Jeroen van den Hoven. 2018. "Meaningful Human Control over Autonomous Systems: A Philosophical Account." *Frontiers in Robotics and AI* 5. https://doi.org/10.3389/frobt.2018.00015.

Sarter, Nadine B., and David D. Woods. 1995. "How in the World Did We Ever Get into That Mode? Mode Error and Awareness in Supervisory Control." *Human Factors* 37 (1): 5–19. https://doi.org/10.1518/001872095779049516.

Schimpe, Andreas, Johannes Feiler, Simon Hoffmann, Domagoj Majstorovic, and Frank Diermeyer. 2022. "Open Source Software for Teleoperated Driving." *International Conference on Connected Vehicle and Expo (ICCVE)*, 1–6. https://doi.org/10.1109/ICCVE52871.2022.9742859.

Schmid, Lukas, Marcus Abate, Yun Chang, and Luca Carlone. 2024. "Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments." *Robotics: Science and Systems XX*. https://doi.org/10.15607/RSS.2024.XX.081.

Schmid, Lukas, Olov Andersson, Aurelio Sulser, Patrick Pfreundschuh, and Roland Siegwart. 2023. "Dynablox: Real-Time Detection of Diverse Dynamic Objects in Complex Environments." *IEEE Robotics and Automation Letters* 8 (10): 6259–66. https://doi.org/10.1109/LRA.2023.3305239.

Schoenberg, Isaac J. 1946. "Contributions to the Problem of Approximation of Equidistant Data by Analytic Functions. Part A. On the Problem of Smoothing or Graduation. A First Class of Analytic Approximation Formulae." *Quarterly of Applied Mathematics* 4 (1): 45–99. https://doi.org/10.1090/qam/15914.

Schulman, John, Yan Duan, Jonathan Ho, Alex Lee, Ibrahim Awwal, Henry Bradlow, Jia Pan, Sachin Patil, Ken Goldberg, and Pieter Abbeel. 2014. "Motion Planning with Sequential Convex Optimization and Convex Collision Checking." *The International Journal of Robotics Research* 33 (9): 1251–70. https://doi.org/10.1177/0278364914528132.

Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. "Proximal Policy Optimization Algorithms." *arXiv Preprint arXiv:1707.06347*.

Schwabe, Peter, and Ko Stoffelen. 2017. "All the AES You Need on Cortex-M3 and M4." *Selected Areas in Cryptography (SAC 2016)*, Lecture notes in computer science, vol. 10532: 180–94. https://doi.org/10.1007/978-3-319-69453-5_10.

Schwartz, Jacob T., and Micha Sharir. 1983. "On the 'Piano Movers' Problem. II. General Techniques for Computing Topological Properties of Real Algebraic Manifolds." *Advances in Applied Mathematics* 4 (3): 298–351. https://doi.org/10.1016/0196-8858(83)90014-3.

Scott, Colin. 2012. "Numbers Every Programmer Should Know by Year."

Scully, Leah. 2015. *Hydraulic Quadruped Uses Mini Servovalves*. Power & Motion.

Seok, Sangok, Albert Wang, Meng Yee Chuah, Dong Jin Hyun, JaeJun Lee, David M. Otten, Jeffrey H. Lang, and Sangbae Kim. 2015. "Design Principles for Energy-Efficient Legged Locomotion and Implementation on the MIT Cheetah Robot." *IEEE/ASME Transactions on Mechatronics* 20 (3): 1117–29.

Seto, Danbing, Bruce Krogh, Lui Sha, and Alongkrit Chutinan. 1998. "The Simplex Architecture for Safe Online Control System Upgrades." *Proceedings of the 1998 American Control Conference (ACC)* 6: 3504–8. https://doi.org/10.1109/ACC.1998.703255.

Sha, Lui. 2001. "Using Simplicity to Control Complexity." *IEEE Software* 18 (4): 20–28.

Sha, Lui, Ragunathan Rajkumar, and John P. Lehoczky. 1990. "Priority Inheritance Protocols: An Approach to Real-Time Synchronization." *IEEE Transactions on Computers* 39 (9): 1175–85.

Shannon, Claude E. 1949. "Communication in the Presence of Noise." *Proceedings of the IRE* 37 (1): 10–21. https://doi.org/10.1109/JRPROC.1949.232969.

Shenzhen Zhongling Technology. 2022. *ZLLG10ASM800 Robot Hub Motor: Motor Parameters*. Product drawing and datasheet.

Sheridan, Thomas B. 1992. *Telerobotics, Automation, and Human Supervisory Control*. MIT Press.

Sheridan, Thomas B. 2021. "Human Supervisory Control of Automation." In *Handbook of Human Factors and Ergonomics*, 5th ed., edited by Gavriel Salvendy and Waldemar Karwowski. Wiley. https://doi.org/10.1002/9781119636113.ch28.

Sheridan, Thomas B., and William L. Verplank. 1978. *Human and Computer Control of Undersea Teleoperators*. Technical Report N00014-77-C-0256. Man-Machine Systems Laboratory, Massachusetts Institute of Technology.

Shiu, Yiu Cheung, and Shaheen Ahmad. 1989. "Calibration of Wrist-Mounted Robotic Sensors by Solving Homogeneous Transform Equations of the Form AX = XB." *IEEE Transactions on Robotics and Automation* 5 (1): 16–29. https://doi.org/10.1109/70.88014.

Siciliano, Bruno, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. 2009. *Robotics: Modelling, Planning and Control*. Springer Science & Business Media.

Simchowitz, Max, Daniel Pfrommer, and Ali Jadbabaie. 2025. "The Pitfalls of Imitation Learning When Actions Are Continuous." *Proceedings of the Thirty Eighth Conference on Learning Theory (COLT)*, Proceedings of machine learning research, vol. 291: 5248–351.

Simpson, H. R. 1990. "Four-Slot Fully Asynchronous Communication Mechanism." *IEE Proceedings E (Computers and Digital Techniques)* 137 (1): 17–30. https://doi.org/10.1049/ip-e.1990.0002.

Slotine, Jean-Jacques E, and Weiping Li. 1991. *Applied Nonlinear Control*. Prentice Hall.

Smith, Gerald L., Stanley F. Schmidt, and Leonard A. McGee. 1962. *Application of Statistical Filter Theory to the Optimal Estimation of Position and Velocity on Board a Circumlunar Vehicle*. NASA TR R-135. National Aeronautics; Space Administration.

Smith, L. D., R. E. Anderson, D. W. Forehand, T. J. Pelc, and T. Roy. 1999. "Power Distribution System Design Methodology and Capacitor Selection for Modern CMOS Technology." *IEEE Transactions on Advanced Packaging* 22 (3): 284–91. https://doi.org/10.1109/6040.784476.

Smith, Otto J. M. 1957. "Closer Control of Loops with Dead Time." *Chemical Engineering Progress* 53 (5): 217–19.

Smith, Randall, Matthew Self, and Peter Cheeseman. 1990. "Estimating Uncertain Spatial Relationships in Robotics." In *Autonomous Robot Vehicles*, edited by Ingemar J. Cox and Gordon T. Wilfong. Springer. https://doi.org/10.1007/978-1-4613-8997-2_14.

Snyder, David, Asher James Hancock, Apurva Badithela, Emma Dixon, Patrick Miller, Rares Andrei Ambrus, Anirudha Majumdar, Masha Itkina, and Haruki Nishimura. 2025. "Is Your Imitation Learning Policy Better Than Mine? Policy Comparison with Near-Optimal Stopping." *arXiv Preprint arXiv:2503.10966*.

Sontag, Eduardo D. 1989. "A 'Universal' Construction of Artstein's Theorem on Nonlinear Stabilization." *Systems & Control Letters* 13 (2): 117–23. https://doi.org/10.1016/0167-6911(89)90028-5.

Spencer, Jonathan, Sanjiban Choudhury, Arun Venkatraman, Brian Ziebart, and J. Andrew Bagnell. 2021. "Feedback in Imitation Learning: The Three Regimes of Covariate Shift." *arXiv Preprint arXiv:2102.02872*.

Spong, Mark W. 1987. "Modeling and Control of Elastic Joint Robots." *Journal of Dynamic Systems, Measurement, and Control* 109 (4): 310–19.

Steiner, Lukas, Matthias Jung, Michael Huonker, and Norbert Wehn. 2022. "Unveiling the Real Performance of LPDDR5 Memories." *Proceedings of the International Symposium on Memory Systems (MEMSYS)*, 1–3. https://doi.org/10.1145/3565053.3565062.

Stephenson, Arthur G. et al. 1999. *Mars Climate Orbiter Mishap Investigation Board Phase i Report*. National Aeronautics; Space Administration.

Stewart, David E., and Jeffrey C. Trinkle. 1996. "An Implicit Time-Stepping Scheme for Rigid Body Dynamics with Inelastic Collisions and Coulomb Friction." *International Journal for Numerical Methods in Engineering* 39 (15): 2673–91. https://doi.org/10.1002/(SICI)1097-0207(19960815)39:15<2673::AID-NME972>3.0.CO;2-I.

Sundaralingam, Balakumar, Siva Kumar Sastry Hari, Adam Fishman, Caelan Garrett, Karl Van Wyk, Valts Blukis, Alexander Millane, et al. 2023. "CuRobo: Parallelized Collision-Free Robot Motion Generation." *IEEE International Conference on Robotics and Automation (ICRA)*, 11124–31.

Sundermeyer, Martin, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. "Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes." *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 13438–44. https://doi.org/10.1109/ICRA48506.2021.9561877.

Sutton, Richard S. 2019. "The Bitter Lesson." *Incompleteideas.net* 43.

Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press.

Szegedy, Christian, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2013. "Intriguing Properties of Neural Networks." *arXiv Preprint arXiv:1312.6199*.

Szeliski, Richard. 2022. *Computer Vision: Algorithms and Applications*. 2nd ed. Springer.

Tao, Stone. 2024. *Simulation Speed Benchmark Likely Has Significant Issues Leading to Overstated Numbers*. GitHub issue #181, Genesis-Embodied-AI/genesis-world.

Tedrake, Russ. 2023. *Robotic Manipulation: Perception, Planning, and Control*. Massachusetts Institute of Technology.

Teschner, Matthias, Bruno Heidelberger, Matthias Müller, Danat Pomeranets, and Markus Gross. 2003. "Optimized Spatial Hashing for Collision Detection of Deformable Objects." *Proceedings of Vision, Modeling, and Visualization (VMV)*, 47–54.

The Linux Kernel Documentation. 2026. *MPAM*.

Thiébaux, H. J., and Francis W. Zwiers. 1984. "The Interpretation and Estimation of Effective Sample Size." *Journal of Climate and Applied Meteorology* 23 (5): 800–811. https://doi.org/10.1175/1520-0450(1984)023<0800:TIAEOE>2.0.CO;2.

Thompson, Dennis F. 1980. "Moral Responsibility of Public Officials: The Problem of Many Hands." *American Political Science Review* 74 (4): 905–16. https://doi.org/10.2307/1954312.

Thrun, Sebastian, Wolfram Burgard, and Dieter Fox. 2005. *Probabilistic Robotics*. MIT Press.

Thrun, Sebastian, Mike Montemerlo, Hendrik Dahlkamp, David Stavens, Andrei Aron, James Diebel, Philip Fong, et al. 2006. "Stanley: The Robot That Won the DARPA Grand Challenge." *Journal of Field Robotics* 23 (9): 661–92. https://doi.org/10.1002/rob.20147.

Tobin, Josh, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. 2017. "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World." *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 23–30. https://doi.org/10.1109/IROS.2017.8202133.

Todorov, Emanuel. 2014. "Convex and Analytically-Invertible Dynamics with Contacts and Constraints: Theory and Implementation in MuJoCo." *IEEE International Conference on Robotics and Automation (ICRA)*, 6054–61. https://doi.org/10.1109/ICRA.2014.6907751.

Todorov, Emanuel, Tom Erez, and Yuval Tassa. 2012. "MuJoCo: A Physics Engine for Model-Based Control." *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5026–33. https://doi.org/10.1109/IROS.2012.6386109.

Toulmin, Stephen E. 1958. *The Uses of Argument*. Cambridge University Press.

TQ-Systems. 2014. *ILM Series Frameless Servo Kits Data Sheet*. TQ-RoboDrive product datasheet.

Tran, Hoang-Dung, Xiaodong Yang, Diego Manzanas Lopez, Patrick Musau, Luan Viet Nguyen, Weiming Xiang, Stanley Bak, and Taylor T. Johnson. 2020. "NNV: The Neural Network Verification Tool for Deep Neural Networks and Learning-Enabled Cyber-Physical Systems." *Computer Aided Verification (CAV)*, Lecture notes in computer science, 3–17. https://doi.org/10.1007/978-3-030-53288-8_1.

Trautman, Peter, and Andreas Krause. 2010. "Unfreezing the Robot: Navigation in Dense, Interacting Crowds." *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 797–803. https://doi.org/10.1109/IROS.2010.5654369.

TRI LBM Team, Jose Barreiros, Andrew Beaulieu, Aditya Bhat, Rick Cory, Eric Cousineau, Hongkai Dai, et al. 2025. "A Careful Examination of Large Behavior Models for Multitask Dexterous Manipulation." *arXiv Preprint arXiv:2507.05331*.

Tsai, Roger Y. 1987. "A Versatile Camera Calibration Technique for High-Accuracy 3D Machine Vision Metrology Using Off-the-Shelf TV Cameras and Lenses." *IEEE Journal on Robotics and Automation* 3 (4): 323–44. https://doi.org/10.1109/JRA.1987.1087109.

Tsai, Roger Y., and Reimar K. Lenz. 1989. "A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration." *IEEE Transactions on Robotics and Automation* 5 (3): 345–58. https://doi.org/10.1109/70.34770.

TÜV Rheinland. 2021. *ISO 3691-4:2020: A Standard for Automated Guided Vehicles*. White paper.

UL Standards & Engagement. 2023. *ANSI/UL 4600 Ed. 3-2023: Standard for Evaluation of Autonomous Products*. UL Standards & Engagement.

United Nations Economic Commission for Europe. 2021. *UN Regulation No. 157: Uniform Provisions Concerning the Approval of Vehicles with Regard to Automated Lane Keeping Systems*. E/ECE/TRANS/505/Rev.3/Add.156. United Nations Economic Commission for Europe.

U.S. Attorney's Office, Northern District of California. 2024. *Cruise Admits to Submitting a False Report to Influence a Federal Investigation and Agrees to Pay $500,000*.

Valsan, Prathap Kumar, Heechul Yun, and Farzad Farshchi. 2016. "Taming Non-Blocking Caches to Improve Isolation in Multicore Real-Time Systems." *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 1–12. https://doi.org/10.1109/RTAS.2016.7461361.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. "Attention Is All You Need." *Advances in Neural Information Processing Systems (NeurIPS)* 30.

Vaughan, Diane. 1996. *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. University of Chicago Press.

Vestal, Steve. 2007. "Preemptive Scheduling of Multi-Criticality Systems with Varying Degrees of Execution Time Assurance." *IEEE International Real-Time Systems Symposium (RTSS)*, 239–43. https://doi.org/10.1109/RTSS.2007.47.

Ville, Jean. 1939. "Étude Critique de La Notion de Collectif." PhD thesis, Faculté des Sciences de Paris; Gauthier-Villars.

Vovk, Vladimir, Alex Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World*. Springer. https://doi.org/10.1007/b106715.

Wabersich, Kim P., and Melanie N. Zeilinger. 2021. "A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems." *Automatica* 129: 109647.

Wagenmaker, Andrew, Mitsuhiko Nakamoto, Yunchu Zhang, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. 2025. "Steering Your Diffusion Policy with Latent Space Reinforcement Learning." *arXiv Preprint arXiv:2506.15799*.

Wald, Abraham. 1945. "Sequential Tests of Statistical Hypotheses." *The Annals of Mathematical Statistics* 16 (2): 117–86. https://doi.org/10.1214/aoms/1177731118.

Walke, Homer Rich, Kevin Black, Tony Z. Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, et al. 2023. "BridgeData V2: A Dataset for Robot Learning at Scale." *Proceedings of the 7th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 229: 1723–36.

Wampler, Charles W., II. 1986. "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods." *IEEE Transactions on Systems, Man, and Cybernetics* 16 (1): 93–101. https://doi.org/10.1109/TSMC.1986.289285.

Wang, Shiqi, Huan Zhang, Kaidi Xu, Xue Lin, Suman Jana, Cho-Jui Hsieh, and J. Zico Kolter. 2021. "Beta-CROWN: Efficient Bound Propagation with Per-Neuron Split Constraints for Neural Network Robustness Verification." *Advances in Neural Information Processing Systems (NeurIPS)* 34.

Wang, Taowen, Cheng Han, James Chenhao Liang, Wenhao Yang, Dongfang Liu, Luna Xinyu Zhang, Qifan Wang, Jiebo Luo, and Ruixiang Tang. 2025. "Exploring the Adversarial Vulnerabilities of Vision-Language-Action Models in Robotics." *IEEE/CVF International Conference on Computer Vision (ICCV)*, 6948–58. https://doi.org/10.1109/ICCV51701.2025.00653.

Watkins, Christopher J. C. H., and Peter Dayan. 1992. "Q-Learning." *Machine Learning* 8 (3–4): 279–92. https://doi.org/10.1007/BF00992698.

Waymo. 2026. *Waymo Safety Impact Data Hub*.

Wensing, Patrick M., Albert Wang, Sangok Seok, David Otten, Jeffrey H. Lang, and Sangbae Kim. 2017. "Proprioceptive Actuator Design in the MIT Cheetah: Impact Mitigation and High-Bandwidth Physical Interaction for Dynamic Legged Robots." *IEEE Transactions on Robotics* 33 (3): 509–22.

Wieland, Peter, and Frank Allgöwer. 2007. "Constructive Safety Using Control Barrier Functions." *IFAC Proceedings Volumes* 40 (12): 462–67. https://doi.org/10.3182/20070822-3-ZA-2920.00076.

Wiener, Norbert. 1948. *Cybernetics: Or Control and Communication in the Animal and the Machine*. Wiley.

Wilhelm, Reinhard, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, et al. 2008. "The Worst-Case Execution-Time Problem: Overview of Methods and Survey of Tools." *ACM Transactions on Embedded Computing Systems* 7 (3): 36:1–53. https://doi.org/10.1145/1347375.1347389.

Willems, Jan C. 1972. "Dissipative Dynamical Systems Part I: General Theory." *Archive for Rational Mechanics and Analysis* 45 (5): 321–51. https://doi.org/10.1007/BF00276493.

Williams, Grady, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. 2017. "Information Theoretic MPC for Model-Based Reinforcement Learning." *IEEE International Conference on Robotics and Automation (ICRA)*, 1714–21. https://doi.org/10.1109/ICRA.2017.7989202.

Williams, Ronald J. 1992. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning." *Machine Learning* 8 (3–4): 229–56. https://doi.org/10.1007/BF00992696.

Williams, Samuel, Andrew Waterman, and David Patterson. 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52 (4): 65–76.

Wilson, Benjamin, Judy Hoffman, and Jamie Morgenstern. 2019. "Predictive Inequity in Object Detection." *arXiv Preprint arXiv:1902.11097*.

Wilson, Edwin B. 1927. "Probable Inference, the Law of Succession, and Statistical Inference." *Journal of the American Statistical Association* 22 (158): 209–12. https://doi.org/10.1080/01621459.1927.10502953.

Wu, Haoze, Omri Isac, Aleksandar Zeljić, Teruhiro Tagomori, Matthew Daggitt, Wen Kokke, Idan Refaeli, et al. 2024. "Marabou 2.0: A Versatile Formal Analyzer of Neural Networks." *Computer Aided Verification (CAV)*, Lecture notes in computer science, 249–64. https://doi.org/10.1007/978-3-031-65630-9_13.

Wu, Philipp, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. 2023. "DayDreamer: World Models for Physical Robot Learning." *Proceedings of the 6th Conference on Robot Learning*, Proceedings of machine learning research, vol. 205: 2226–40.

Wu, Philipp, Yiyang Shentu, Zhonghe Yi, Xingyu Lin, Ken Goldberg, and Pieter Abbeel. 2024. "GELLO: A General, Low-Cost, and Intuitive Teleoperation Framework for Robot Manipulation." *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Wulf, Wm. A., and Sally A. McKee. 1995. "Hitting the Memory Wall: Implications of the Obvious." *ACM SIGARCH Computer Architecture News* 23 (1): 20–24. https://doi.org/10.1145/216585.216588.

Xiao, Guangxuan, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. "Efficient Streaming Language Models with Attention Sinks." *International Conference on Learning Representations (ICLR)*.

Xu, Kaidi, Huan Zhang, Shiqi Wang, Yihan Wang, Suman Jana, Xue Lin, and Cho-Jui Hsieh. 2021. "Fast and Complete: Enabling Complete Neural Network Verification with Rapid and Massively Parallel Incomplete Verifiers." *International Conference on Learning Representations (ICLR)*.

Yuan, Wenzhen, Siyuan Dong, and Edward H. Adelson. 2017. "GelSight: High-Resolution Robot Tactile Sensors for Estimating Geometry and Force." *Sensors* 17 (12): 2762. https://doi.org/10.3390/s17122762.

Yun, Heechul, Renato Mancuso, Zheng-Pei Wu, and Rodolfo Pellizzoni. 2014. "PALLOC: DRAM Bank-Aware Memory Allocator for Performance Isolation on Multicore Platforms." *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 155–66. https://doi.org/10.1109/RTAS.2014.6925999.

Yun, Heechul, Gang Yao, Rodolfo Pellizzoni, Marco Caccamo, and Lui Sha. 2013. "MemGuard: Memory Bandwidth Reservation System for Efficient Performance Isolation in Multi-Core Platforms." *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 55–64. https://doi.org/10.1109/RTAS.2013.6531079.

Zakka, Kevin, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, et al. 2025. "MuJoCo Playground." *arXiv Preprint arXiv:2502.08844*.

Zeng, Fangao, Bin Dong, Yuang Zhang, Tiancai Wang, Xiangyu Zhang, and Yichen Wei. 2022. "MOTR: End-to-End Multiple-Object Tracking with Transformer." *European Conference on Computer Vision (ECCV)*, 659–75. https://doi.org/10.1007/978-3-031-19812-0_38.

Zhang, Bo, Joost de Winter, Silvia Varotto, Riender Happee, and Marieke Martens. 2019. "Determinants of Take-over Time from Automated Driving: A Meta-Analysis of 129 Studies." *Transportation Research Part F: Traffic Psychology and Behaviour* 64: 285–307. https://doi.org/10.1016/j.trf.2019.04.020.

Zhang, Yifu, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, et al. 2022. "ByteTrack: Multi-Object Tracking by Associating Every Detection Box." *European Conference on Computer Vision (ECCV)*, 1–21. https://doi.org/10.1007/978-3-031-20047-2_1.

Zhang, Zhengyou. 2000. "A Flexible New Technique for Camera Calibration." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22 (11): 1330–34. https://doi.org/10.1109/34.888718.

Zhao, Tony Z., Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware." *Robotics: Science and Systems (RSS)*. https://doi.org/10.15607/RSS.2023.XIX.016.

DRAFT

DRAFT

A

Machine Learning Background

This appendix supplies the machine learning and statistical tools needed to interpret learned action proposals, size their computation, and evaluate the evidence for their use. It develops uncertainty measures, spatial representations, policy-learning objectives, and rare-event bounds as consultable references for the main text. Each tool separates what a model or test establishes from the physical clearance and timing conditions that the permission path must enforce.

A.1 How to Use This Appendix

Use the table to enter at the relevant problem. The probability section explains expectations, densities, and uncertainty; spatial representations then connect image arrays to metric geometry. Policy sections distinguish the training objective from runtime execution, and the final sections distinguish statistical evidence from a physical guarantee. The derivations use matrix multiplication, derivatives, and integrals; the prose following each result states the engineering consequence so that a reader can consult the result before working through its proof.

When you encounter this systems symptom	Consult	For this quantitative tool
A vision-language model hallucinates actions under perceptual ambiguity	section A.2	Posterior entropy metrics, Shannon uncertainty, and refusal thresholds
Overconfident neural outputs command aggressive actions out-of-distribution	section A.2.3	Softmax simplex geometry and Expected Calibration Error (ECE)
Multi-camera extrinsics, 2D tokenization, or 3D scene geometry drift	section A.3	$SE(3)$ transforms, patch embeddings, attention, and metric 3D lifting
Camera features must be placed in one metric ground-plane grid	section A.3.3	Lift-Splat depth hypotheses, frustum size, and BEV splatting
Differentiable 3D scene reconstruction diverges during real-time tracking	section A.3.5	3D Gaussian Splatting parameters and covariance projection
Occupancy mapping retains phantom obstacles after objects move	section A.3.6	Log-odds voxel updates, TSDF fusion, and raycast DRAM traffic
Small sensor noise produces explosive, unconstrained actuator commands	section A.4.2	Lipschitz continuity bounds and spectral normalization
A behavioral cloning policy drifts into unrecoverable off-trajectory states	section A.5	$\mathcal{O}(S^2\epsilon)$ compounding error and DAgger linear reduction
ℓ_2 regression averages bimodal demonstrations into lethal obstacle collisions	section A.5.1	Conditional expectation proof and multimodal policy decoders
A generative decoder cannot finish a chunk within the setpoint period	section A.5.2	Denoising evaluations N_{eval} latency-expressiveness trade-off and flow matching
Offline reinforcement learning selects ungrounded, destructive motor setpoints	section A.6	Bellman value overestimation and SayCan affordance grounding
Continuous policy optimization diverges under function approximation error	section A.6.1	DDPG, TD3 clipped double Q-learning, SAC maximum entropy, and PPO clipping
Sim-to-real transfer fails due to unobservable physical environmental parameters	section A.6.6	Asymmetric actor-critic architectures and privileged simulation states
Comparing policy decoders across latency, compute intensity, and memory footprint	section A.6.9	Comparative systems taxonomy (MLP, CNN, ACT, Diffusion, Flow, RL)

When you encounter this systems symptom	Consult	For this quantitative tool
Verifying time-varying physical safety properties across continuous traces	section A.7	Signal Temporal Logic (STL) syntax and quantitative robustness degree
Zero-failure fleet testing encounters the Poisson Exposure Wall	section A.8	Rule of Three ($T \geq 3.0/\lambda$), Wald breakdown, and Clopper-Pearson bounds
Physical testbed wear explodes during fixed-sample policy qualification	section A.9	Sequential Probability Ratio Test (SPRT) and Wald stopping boundaries
Contact collision shocks and latency spikes exceed Gaussian 6σ bounds	section A.10	Tail-family selection, GEV/GPD fits, and conditional high-quantile sizing

A.2 Probability, Information Theory, and Calibration

Commanding an uncalibrated motor torque stresses gearboxes, violates physical clearance envelopes, and risks destructive contact collisions across an irreversible causal boundary, unlike a software prediction error that merely degrades a recommendation or text token. Autonomous machines must therefore reason quantitatively about uncertainty across perception, intent estimation, and world modeling. In closed-loop physical AI, uncertainty quantification determines whether a candidate action proposal is dispatched to motor controllers or intercepted by a safety governor.

A.2.1 Conditional probability and Bayes' rule

When a sensor returns observed evidence x, the conditional distribution $p(y \mid x)$ characterizes the updated belief over y. For continuous parameters, p in the following expressions denotes a density; probabilities over regions require integration. By Bayes' rule:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)p(\theta)}{p(\mathcal{D})} = \frac{p(\mathcal{D} \mid \theta)p(\theta)}{\int_\Theta p(\mathcal{D} \mid \theta')p(\theta')\,d\theta'}$$

where $p(\theta)$ represents prior belief over model parameters or latent states $\theta \in \Theta$, $p(\mathcal{D} \mid \theta)$ represents the likelihood of physical dataset or sensor observation sequence $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, and $p(\theta \mid \mathcal{D})$ represents the updated posterior belief.

In high-dimensional learning systems, integrating over the parameter manifold Θ to compute the marginal likelihood (evidence) $p(\mathcal{D}) = \int_\Theta p(\mathcal{D} \mid \theta)p(\theta)\,d\theta$ is analytically and computationally intractable. Real-time autonomy resolves this intractability through two complementary paradigms:

1. **Maximum a posteriori (MAP) estimation:** Replacing posterior integration with point optimization:
$$\theta_{\text{MAP}} = \arg\max_\theta \left[\sum_{i=1}^{N} \ln p(\mathbf{x}_i \mid \theta) + \ln p(\theta)\right]$$
With conditionally independent observations and a summed log-likelihood, a zero-mean isotropic Gaussian prior $p(\theta) = \mathcal{N}(\mathbf{0}, \sigma_0^2\mathbf{I})$ gives the log-prior $\ln p(\theta) = -\frac{\|\theta\|_2^2}{2\sigma_0^2} - \frac{d}{2}\ln(2\pi\sigma_0^2)$. Maximizing the posterior objective is mathematically identical to minimizing negative log-likelihood with an explicit ℓ_2 penalty $\frac{\lambda_{\text{reg}}}{2}\|\theta\|_2^2$, where $\lambda_{\text{reg}} = 1/\sigma_0^2$. However, MAP estimation yields only a single deterministic point estimate θ_{MAP}. The point estimate does not report posterior variance; it does not establish that the underlying uncertainty is zero. In continuous control, epistemic uncertainty (ignorance arising from lack of training data in unfamiliar operational domains) must be sharply distinguished from aleatoric uncertainty (stochasticity inherent to sensor noise, turbulent airflow, or non-deterministic contact friction). A point-estimated model can still predict observation noise, but that output alone does not quantify uncertainty about its fitted parameters.
2. **Variational inference:** Approximating the true posterior $p(\theta \mid \mathcal{D})$ with a tractable variational distribution family $q_\phi(\theta)$ by maximizing the Evidence Lower Bound (ELBO):
$$\ln p(\mathcal{D}) = \mathbb{E}_{\theta\sim q_\phi(\theta)}\left[\ln \frac{p(\mathcal{D},\theta)}{q_\phi(\theta)}\right] + \mathcal{D}_{\text{KL}}\left(q_\phi(\theta) \parallel p(\theta \mid \mathcal{D})\right) \geq \mathbb{E}_{\theta\sim q_\phi(\theta)}[\ln p(\mathcal{D} \mid \theta)] - \mathcal{D}_{\text{KL}}\left(q_\phi(\theta) \parallel p(\theta)\right)$$

Because the Kullback-Leibler divergence is non-negative ($\mathcal{D}_{\mathrm{KL}} \geq 0$), the lower bound holds, with equality when the variational distribution equals the posterior almost everywhere. Variational inference converts intractable high-dimensional integration into continuous gradient-based optimization over variational parameters ϕ. This formulation provides the analytical foundation for the latent-variable action chunking and generative decoders derived in section A.5.1, where latent variables capture human demonstrator intent and style without collapsing multimodal physical actions into destructive spatial averages.

A.2.2 Shannon entropy and ambiguity thresholds

When a perception or intent model spreads probability across several candidate targets, entropy quantifies the average uncertainty or information content contained in that distribution. For a discrete random variable X taking values in alphabet $\mathcal{X}$:[1]

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x) = \mathbb{E}_{x \sim p}\left[\log_2 \frac{1}{p(x)}\right]$$

[1] **Shannon Information Entropy**: Entropy quantifies average uncertainty within a specified discrete distribution. Its physical meaning depends on whether that distribution is calibrated to the relevant referents and observations.

For continuous random variables governing physical state $\mathbf{x} \in \mathbb{R}^d$ with probability density $p(\mathbf{x})$, the **differential entropy** evaluates to:

$$h(\mathbf{X}) = -\int_{\mathbb{R}^d} p(\mathbf{x}) \ln p(\mathbf{x})\, d\mathbf{x}$$

Under a multivariate Gaussian distribution $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$ with positive-definite covariance $\Sigma \in \mathbb{R}^{d \times d}$, substituting the Gaussian density $p(\mathbf{x}) = (2\pi)^{-d/2}(\det\Sigma)^{-1/2}\exp(-\frac{1}{2}(\mathbf{x}-\mu)^\top\Sigma^{-1}(\mathbf{x}-\mu))$ yields:

$$\begin{aligned} h(\mathbf{X}) &= -\mathbb{E}\left[-\frac{d}{2}\ln(2\pi) - \frac{1}{2}\ln(\det\Sigma) - \frac{1}{2}(\mathbf{x}-\mu)^\top\Sigma^{-1}(\mathbf{x}-\mu)\right] \\ &= \frac{d}{2}\ln(2\pi) + \frac{1}{2}\ln(\det\Sigma) + \frac{1}{2}\operatorname{Tr}\left(\Sigma^{-1}\mathbb{E}[(\mathbf{x}-\mu)(\mathbf{x}-\mu)^\top]\right) \\ &= \frac{1}{2}\ln\left((2\pi e)^d \det\Sigma\right) \end{aligned}$$

Unlike discrete entropy, differential entropy depends on coordinate units and can be negative. Comparisons require the same coordinates and units. For a Gaussian it scales with the logarithmic volume of the physical uncertainty ellipsoid $\det\Sigma$. In state estimation, sensor fusion, and Kalman filtering, an informative measurement update shrinks $\det\Sigma$, producing an exact physical information gain $\Delta h = h_{\text{prior}} - h_{\text{post}} = \frac{1}{2}\ln(\det\Sigma_{\text{prior}}/\det\Sigma_{\text{post}})$. Under an isotropic Gaussian distribution $\mathcal{N}(\mu, \sigma^2\mathbf{I})$, this reduces to $h(\mathbf{X}) = \frac{d}{2}\ln(2\pi e\sigma^2)$. In continuous reinforcement learning, such as Soft Actor-Critic (SAC), the unsquashed diagonal Gaussian policy has entropy $\mathcal{H}(\pi(\cdot \mid s)) = \frac{d_a}{2}(1+\ln(2\pi)) + \sum_{i=1}^{d_a}\ln\sigma_i^\pi(s)$ with d_a action dimensions and conditional standard deviations σ_i^π. Entropy regularization encourages exploration; after bounding actions with a nonlinear transformation, its Jacobian changes this entropy, as the SAC derivation below shows.

To quantify information divergence between an empirical demonstrator distribution p and a learned policy distribution q, information theory defines the **Kullback-Leibler (KL) divergence** (relative entropy):

$$\mathcal{D}_{\mathrm{KL}}(p \parallel q) = \int_{\mathbb{R}^d} p(\mathbf{x}) \ln\left(\frac{p(\mathbf{x})}{q(\mathbf{x})}\right) d\mathbf{x}$$

where $\mathcal{D}_{\mathrm{KL}}(p \parallel q) \geq 0$ by Gibbs' inequality, with equality if and only if $p = q$ almost everywhere.

In closed-loop decision making, Shannon entropy provides an instantaneous monitor of policy ambiguity. When a vision-language planner encounters an underspecified scene with two equally probable referents (such as two identical cups), the discrete action entropy approaches its maximum $H(X) \to \log_2 2 = 1.0$ bit. A calibrated threshold gates actuation, issuing an ambiguity fault (`ERR_GROUNDING_AMBIGUITY`) that commands a stationary hold while active perception collects disambiguating viewpoints. However, because an overfitted neural network can be confidently wrong, entropy alone cannot guarantee semantic correctness; it must operate in tandem with physical clearance monitors.

A.2.3 Softmax overconfidence and calibration error

A perception classifier that reports a 0.99 softmax score on a wet-floor reflection or an unfamiliar obstacle is measuring logit separation rather than empirical accuracy. Standard neural classification heads normalize raw logit vectors $\mathbf{z} = [z_1, \ldots, z_K]^\top \in \mathbb{R}^K$ into categorical distributions using the softmax function:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i/T_{\text{cal}}}}{\sum_{j=1}^{K} e^{z_j/T_{\text{cal}}}}$$

where $T_{\text{cal}} > 0$ is a temperature hyperparameter (defaulting to $T_{\text{cal}} = 1$). The softmax operator maps logits onto the open probability simplex $\Delta^{K-1} = \{\mathbf{p} \in \mathbb{R}^K : \sum_i p_i = 1, p_i > 0\}$ strictly through algebraic exponentiation.

A cross-entropy training objective does not by itself establish calibration. At $T_{\text{cal}} = 1$, for target class $y \in \{1, \ldots, K\}$, the cross-entropy loss evaluates to $\mathcal{L}_{\text{CE}}(\mathbf{z}, y) = -z_c + \ln \sum_{j=1}^{K} e^{z_j}$. Differentiating with respect to any logit z_i:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_i} = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} - \mathbf{1}(i = y) = \sigma(\mathbf{z})_i - \mathbf{1}(i = y)$$

For the ground-truth class y, the derivative is negative until its predicted probability approaches unity. For an individually fitted example, cross-entropy decreases as the correct class separates from the other logits; separable, unregularized training can therefore keep increasing those margins. This is not a proof that every trained network is overconfident or that noisy examples can all be fitted simultaneously. A large softmax output measures relative logit separation, while calibration asks whether predictions with that confidence are correct at the corresponding frequency.

To quantify whether predicted probabilities match empirical physical frequencies, reliability engineering computes the **Expected Calibration Error (ECE)** and **Maximum Calibration Error (MCE)**:[2]

$$\text{ECE} = \sum_{m=1}^{N_{\text{bins}}} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|, \qquad \text{MCE} = \max_{m \in \{1, \ldots, N_{\text{bins}}\}} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|$$

Empty bins contribute zero to ECE and are omitted from MCE; accuracy and confidence averages are defined only for nonempty bins. Here B_m is the set of validation examples whose predicted confidence falls in $((m-1)/N_{\text{bins}}, m/N_{\text{bins}}]$. The size $|B_m|$ is the sample count in bin m, and N is the total validation sample count. The mean predicted confidence is $\text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i$, and the empirical accuracy is $\text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbf{1}(\hat{y}_i = y_i)$.

To recalibrate an overconfident model post-hoc without altering its classification accuracy, **temperature scaling** optimizes a single scalar parameter $T_{\text{cal}} > 0$ on a held-out calibration dataset $\mathcal{D}_{\text{cal}}$:

$$T^*_{\text{cal}} = \arg\min_{T_{\text{cal}} > 0} \left[- \sum_{(\mathbf{x}, y) \in \mathcal{D}_{\text{cal}}} \ln \left(\frac{e^{z_c(\mathbf{x})/T_{\text{cal}}}}{\sum_{j=1}^{K} e^{z_j(\mathbf{x})/T_{\text{cal}}}} \right) \right]$$

Because T^*_{cal} scales all logits uniformly, the ranking $\arg\max_i (z_i/T_{\text{cal}}) = \arg\max_i z_i$ remains unchanged for all $T_{\text{cal}} > 0$, preserving top-1 accuracy. Temperatures above one soften probabilities, while temperatures below one sharpen them; optimizing calibration loss does not guarantee exact calibration. However, temperature scaling rests on the classical assumption that deployment data is identically distributed to calibration data ($p_{\text{deploy}} = p_{\text{cal}}$). In physical AI, an embodied agent continually encounters non-stationary environmental shifts: lens flare, wet floor reflectance, novel obstacle textures, or altered motor dynamics. Temperature scaling cannot repair or detect out-of-distribution inputs: scaling an out-of-distribution logit vector by scalar T^*_{cal} still yields near-unity confidence if the raw relative logit separation remains large. In safety-critical autonomy, post-hoc calibration requires separately validated uncertainty and clearance checks. Conformal coverage also depends on its sampling assumptions, commonly exchangeability; it does not automatically survive deployment shift.

[2] **Expected Calibration Error**: Binned calibration metrics quantify the divergence between neural prediction confidence and realized empirical accuracy across confidence intervals. Separable, unregularized cross-entropy training can keep increasing logit margins; whether a fitted network is overconfident must be assessed on representative held-out data. Relying on uncalibrated confidence scores to gate safety fallbacks permits undetected distribution shifts to corrupt closed-loop actuation.

A.3 Spatial Representations and Scene Parameterization

Detecting a door handle at pixel $(142, 88)$ in a wrist-camera image cannot tell a joint controller where to move until that pixel ray is combined with metric depth and transformed through the camera, wrist, arm, and mobile-base coordinate frames. Unlike language models that operate on discrete semantic tokens, an embodied policy must bind high-dimensional sensor arrays (RGB-D cameras, time-of-flight rangefinders, and lidars) to rigid physical links and Euclidean 3D reference frames.

A.3.1 Special Euclidean group $SE(3)$ transforms

Rigid body transformations in 3D Euclidean space belong to the Special Euclidean Lie group $SE(3)$, combining a 3×3 orthonormal rotation matrix $\mathbf{R} \in SO(3)$ ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det(\mathbf{R}) = +1$) with a translation vector $\mathbf{t} \in \mathbb{R}^3$:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3) \subset \mathbb{R}^{4\times 4}$$

A homogeneous coordinate vector $\tilde{\mathbf{p}} = [\mathbf{p}^\top, 1]^\top \in \mathbb{R}^4$ transforms across coordinate frames via matrix-vector multiplication:

$$\tilde{\mathbf{p}}' = \mathbf{T}\tilde{\mathbf{p}} \iff \mathbf{p}' = \mathbf{R}\mathbf{p} + \mathbf{t}$$

Let $\mathbf{T}_{AB}$ denote the transform mapping coordinates from frame B into frame A. For a kinematic chain comprising world frame W, mobile base B, end effector E, and wrist camera C, the forward composite transformation evaluates via associative matrix multiplication:

$$\mathbf{T}_{WC} = \mathbf{T}_{WB}\mathbf{T}_{BE}\mathbf{T}_{EC}$$

The inverse transform admits an exact closed-form evaluation avoiding general 4×4 matrix inversion:

$$\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

A.3.1.1 Lie algebra $\mathfrak{se}(3)$ and manifold perturbation

Because $SE(3)$ forms a smooth, non-Euclidean Riemannian manifold rather than a linear vector space, unconstrained optimization (such as gradient descent during camera pose estimation or neural trajectory generation) cannot directly add matrices ($\mathbf{T}_1 + \mathbf{T}_2 \notin SE(3)$). Regressing 3D rotations directly using Euler angles induces topological singularities (gimbal lock at pitch $\pm 90^\circ$) where degrees of freedom collapse. Unconstrained unit quaternions $\mathbf{q}_{\text{quat}} \in S^3$ suffer from the antipodal double-cover ambiguity ($\mathbf{q}_{\text{quat}}$ and $-\mathbf{q}_{\text{quat}}$ represent the identical physical rotation). In continuous neural policy training, minimizing ℓ_2 regression loss over quaternions produces severe gradient cancellation when the network predicts opposite hemispheres for nearby states. To ensure topological continuity across continuous rotations, modern spatial policies either output continuous 6D rotation representations (the first two columns of $\mathbf{R}$ orthonormalized via Gram-Schmidt) or parameterize local variations in the tangent space at the identity: the Lie algebra $\mathfrak{se}(3)$.

For the finite pose increment below, a twist coordinate vector $\xi = [\mathbf{v}^\top, \omega^\top]^\top \in \mathbb{R}^6$ combines integrated translation coordinates $\mathbf{v}$ and rotation coordinates ω. If these symbols instead denote velocities, multiply the matrix twist by elapsed time before exponentiating. The wedge operator $(\cdot)^\wedge$ maps ξ to its matrix Lie algebra representation:

$$\hat{\xi} = \xi^\wedge = \begin{bmatrix} [\omega]_\times & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix} \in \mathfrak{se}(3), \quad \text{where } [\omega]_\times = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

The exponential map $\exp : \mathfrak{se}(3) \to SE(3)$ computes finite rigid transformations in closed form via Rodrigues' formula:

$$\exp(\hat{\xi}) = \begin{bmatrix} \exp([\omega]_\times) & \mathbf{V}\mathbf{v} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

where $\theta_{\text{rot}} = \|\omega\|$, $\exp([\omega]_\times) = \mathbf{I} + \frac{\sin\theta_{\text{rot}}}{\theta_{\text{rot}}}[\omega]_\times + \frac{1-\cos\theta_{\text{rot}}}{\theta_{\text{rot}}^2}[\omega]_\times^2$, and $\mathbf{V} = \mathbf{I} + \frac{1-\cos\theta_{\text{rot}}}{\theta_{\text{rot}}^2}[\omega]_\times + \frac{\theta_{\text{rot}}-\sin\theta_{\text{rot}}}{\theta_{\text{rot}}^3}[\omega]_\times^2$.

In spatial tracking and state estimation, small pose uncertainties are modeled by perturbing the mean transform with a zero-mean Gaussian twist vector in the local frame: $\mathbf{T}' = \mathbf{T}\exp(\widehat{\epsilon})$, with $\epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma_\xi)$ and covariance $\Sigma_\xi \in \mathbb{R}^{6\times 6}$. The exponential keeps the updated pose on $SE(3)$. The covariance describes local tangent coordinates, not a Gaussian distribution over matrix entries; its approximation is reliable only while the local perturbation model is adequate. At zero rotation, evaluate the coefficients above by their continuous limits to avoid division by zero.

A.3.2 Visual patch tokenization, attention, and 3D metric lifting

A vision transformer that slices three 224×224 camera frames into 14×14-pixel patches produces 768 visual tokens per control step. Yet none of those tokens carries metric 3D distance until camera intrinsics and depth measurements lift the 2D image plane into Euclidean space. Embodied vision-language-action (VLA) models and spatial transformers convert continuous sensory streams into discrete tokens for neural attention, then lift candidate spatial predictions into 3D coordinates for physical reasoning. Section 3.3 and section 3.4 use the token counts derived here to size the application processor's memory traffic.

A.3.2.1 Convolutional encoders versus vision transformer tokenization

Physical AI architectures employ two primary paradigms for encoding raw camera observations into visual features:

1. **Convolutional Encoders (ConvNets):** Classical visuomotor backbones (such as ResNet-18/34/50 or modern ConvNeXt variants) process images through hierarchical sliding-window convolutions. Convolutions possess an intrinsic inductive bias: **translational equivariance** ($\text{Conv}(\text{Shift}(\mathbf{I})) = \text{Shift}(\text{Conv}(\mathbf{I}))$) and local spatial compactness. Because small 3×3 filter weights are repeatedly applied across thousands of image pixel locations, convolutional feature extraction achieves high arithmetic intensity (FLOPs per byte transferred from DRAM) in early layers. Intermediate feature activations stream through on-chip SRAM line buffers, minimizing off-chip memory bandwidth consumption and reducing traffic when the accelerator mapping and buffer capacity permit reuse. Deterministic latency still requires a timing analysis under contention. However, local receptive fields require deep network stacks to capture global scene context, limiting their ability to resolve long-range semantic relationships (such as attending from a robot gripper to a distant receptacle).
2. **Vision Transformers (ViTs):** Vision transformers discard local sliding windows in favor of non-overlapping patch tokenization and global multi-head self-attention. While ViTs lack hardwired translational equivariance, they establish global receptive fields across the entire image at layer 1. The operational trade-off is computational complexity and memory bandwidth: self-attention scales quadratically ($\mathcal{O}(N^2)$) in token count N. On embedded edge accelerators operating at single-frame batch sizes, materializing full $N \times N$ attention matrices generates substantial activation memory traffic that stresses shared system memory buses.

A.3.2.2 2D vision transformer patch tokenization

Consider an RGB image $\mathbf{I} \in \mathbb{R}^{H\times W\times C}$ captured by an onboard camera with spatial resolution $H \times W$ and C color channels. Feeding raw pixels to a multilayer perceptron would discard spatial locality and inflate the parameter count. A vision transformer frontend therefore partitions the image into a non-overlapping sequence of N patches of $p_{\text{patch}} \times p_{\text{patch}}$ pixels, with H and W divisible by p_{patch} after any chosen crop or padding:

$$N = \frac{H \cdot W}{p_{\text{patch}}^2}$$

Each patch, indexed by its row and column (h, w) in the camera plane, is flattened into a vector $\mathbf{x}_p^{(h,w)} \in \mathbb{R}^{p_{\text{patch}}^2 C}$. A trainable linear projection $\mathbf{W}_{\text{patch}} \in \mathbb{R}^{(p_{\text{patch}}^2 C)\times d}$ maps each flattened patch into the transformer's hidden dimension d. Without positional information or position-dependent masks, self-attention is permutation-equivariant: permuting the input rows permutes the output rows. Therefore the model knows nothing of spatial proximity unless position is added explicitly:

$$\mathbf{z}_0^{(h,w)} = \mathbf{x}_p^{(h,w)}\mathbf{W}_{\text{patch}} + \mathbf{E}_{\text{pos}}^{2D}(h, w) + \mathbf{e}_{\text{cam}}$$

The 2D position embedding $\mathbf{E}^{2D}_{\text{pos}}(h, w)$ decomposes along the vertical axis h and the horizontal axis w, using sinusoidal frequency bands or learned coordinate embeddings. It encodes position in the image plane and carries no metric 3D distance. When several cameras feed one model, such as an overhead camera for scene context and a wrist camera for millimeter-scale alignment, a learned camera identifier $\mathbf{e}_{\text{cam}} \in \{\mathbf{e}_{\text{overhead}}, \mathbf{e}_{\text{wrist}}\}$ distinguishes their reference frames. Some architectures also prepend a summary token $\mathbf{x}_{\text{class}} \in \mathbb{R}^d$ to the sequence.

To make the dimensions concrete, take a square RGB frame 224 pixels on a side (illustrative; see the Reader Guide) with patch size $p_{\text{patch}} = 14$. The frame divides into 16 patches on a side, 256 in all. Each patch holds 196 pixels across 3 color channels, a flattened vector of 588 values, and $\mathbf{W}_{\text{patch}}$ maps each such vector into a 4096-dimensional model space. One frame therefore becomes 256 tokens of width 4096. A robot with 3 cameras (two wrist cameras and one overhead) supplies $3 \times 256 = 768$ visual tokens per step before any text or proprioceptive token is added.

The visual tokens then join the rest of the input. A natural-language directive such as "grasp the red mug by its handle" is tokenized by byte-pair encoding into text embeddings $\mathbf{z}_{\text{text}} \in \mathbb{R}^{S_{\text{text}} \times d}$. The robot's configuration (joint angles, joint velocities, and gripper state) passes through a linear layer into proprioceptive tokens $\mathbf{z}_{\text{proprio}} \in \mathbb{R}^{K_s \times d}$. The three streams concatenate into one multimodal input sequence:

$$\mathbf{Z} = [\mathbf{z}_{\text{vision}} \,|\, \mathbf{z}_{\text{text}} \,|\, \mathbf{z}_{\text{proprio}}] \in \mathbb{R}^{N_{\text{total}} \times d}$$

A.3.2.3 Self-attention over the combined sequence

The combined sequence $\mathbf{Z}$ passes through a stack of transformer layers, each consisting of multi-head self-attention and a feed-forward network. Self-attention projects each token through learned matrices $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ into a query $\mathbf{q}_i$, a key $\mathbf{k}_j$, and a value, and computes pairwise attention weights over the whole sequence:

$$A_{ij} = \frac{\exp\left(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_{\text{head}}}\right)}{\sum_m \exp\left(\mathbf{q}_i^\top \mathbf{k}_m / \sqrt{d_{\text{head}}}\right)}$$

A text token such as "handle" can attend to visual patch tokens, but a high attention weight is not a verified grasp location or metric pose. A trained spatial decoder may produce a 2D affordance heatmap, and placing the gripper in 3D still takes depth evidence, camera calibration, and the metric lifting below. A rolling key-value (KV) cache stores prior token states for attention. It neither tracks object permanence nor maintains a 3D position and velocity; an explicit state estimator must associate observations across frames and widen its uncertainty while the mug is occluded.

A.3.2.4 Pinhole back-projection and 3D metric lifting

Standard 2D patch tokens lack metric scale due to projective ambiguity: along any optical ray $\lambda[u, v, 1]^\top$, infinitely many 3D points project to identical pixel coordinates. To interface with physical control barrier filters, kinetic stopping calculators, and obstacle clearance envelopes, 2D visual predictions must be lifted into metric 3D coordinates $\mathbf{p}_{\text{world}} \in \mathbb{R}^3$.

Let $[u, v]^\top$ denote pixel coordinates on the image sensor, and let the calibrated pinhole camera intrinsic matrix be:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

where (f_x, f_y) are focal lengths in pixels and (c_x, c_y) is the principal point. Given an aligned optical-axis depth measurement $Z_{\text{meas}} = Z_c > 0$ (obtained from stereo disparity, time-of-flight lidar, or calibrated depth sensors), the 3D position in the camera coordinate frame $\mathbf{p}_c \in \mathbb{R}^3$ is recovered via inverse intrinsic projection:

$$\mathbf{p}_c = Z_{\text{meas}} \cdot \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} (u - c_x)\frac{Z_{\text{meas}}}{f_x} \\ (v - c_y)\frac{Z_{\text{meas}}}{f_y} \\ Z_{\text{meas}} \end{bmatrix}$$

Transforming $\mathbf{p}_c$ into the robot base or world frame W uses the rigid extrinsic transform $\mathbf{T}_{WC} \in SE(3)$ defined in this section:

$$\tilde{\mathbf{p}}_W = \mathbf{T}_{WC}\tilde{\mathbf{p}}_C \implies \mathbf{p}_{\text{world}} = \mathbf{R}_{WC}\mathbf{p}_c + \mathbf{t}_{WC}$$

A sensor reporting range along a ray must first convert it to optical-axis depth; substituting raw range for Z_{meas} distorts off-axis geometry. The permission path and kinematic reachability evaluators require this metric 3D reconstruction; ungrounded 2D tokens cannot establish whether a proposed trajectory satisfies physical clearance boundaries d_{stop}.

A.3.3 Camera-to-bird's-eye-view lifting

Checking whether a mobile base can pass a warehouse pillar requires obstacle footprints in meters on the floor plane, whereas each onboard camera observes the aisle in perspective pixels along rays of unknown depth. A bird's-eye-view (BEV) representation places the features of every camera in one metric grid on the ground plane, the frame in which planners and clearance checks work. Figure A.1 stages this transformation across four sequential processing blocks. Monocular 2D feature extraction works on the raw camera images. Categorical depth distribution estimation lifts each feature ray into a 3D frustum of discrete hypothesis bins. Extrinsic coordinate transformation splats the frustum points into a metric ground-plane grid of vertical pillars. A final permission gate bounds spatial clearance residuals before dispatching commands.

1. **Feature extraction.** A vision backbone, such as DINOv2 (Oquab et al. 2023) or a lightweight convolutional network, maps image $\mathbf{I} \in \mathbb{R}^{H\times W\times 3}$ to a grid of feature vectors:

   $$\mathbf{f}_{u,v} = \text{Backbone}(\mathbf{I})[u,v] \in \mathbb{R}^C$$

   Forward projection (equation 8.2) collapses each 3D ray onto one pixel. Consequently, a feature $\mathbf{f}_{u,v}$ stands for a whole ray in space while its metric depth remains unknown.

2. **Lift.** LSS represents that ambiguity rather than resolving it. Along the bearing ray $\mathbf{K}^{-1}[u,v,1]^\top$ of equation 8.3, the network scores D discrete depth bins spanning $[d_{\text{min}}, d_{\text{max}}]$, for example 64 bins from 0.5 m to 20 m, spaced linearly or logarithmically:

   $$\alpha_{u,v} = \text{softmax}(\mathbf{d}_{u,v}) \in \Delta^{D-1}$$

   An outer product lifts each 2D feature into a frustum of 3D points, one per depth hypothesis:

   $$\mathbf{c}_{u,v,k} = \alpha_{u,v}(k)\cdot\mathbf{f}_{u,v} \in \mathbb{R}^C \quad \text{at } \mathbf{p}_c(u,v,d_k) = d_k\mathbf{K}^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

   For a feature map 128 cells on a side, with 64 depth bins and 64 feature channels, the frustum tensor holds about 67.1 million values, 134.2 MB per frame at FP16. This is a materialized tensor footprint for one camera. Actual DRAM traffic depends on whether the runtime materializes, tiles, or fuses lifting with pooling; zero-copy handoff avoids an extra copy but does not eliminate tensor reads and writes.

3. **Splat.** The calibrated extrinsic transform $\mathbf{T}^{\text{cam}}_{\text{base}} \in SE(3)$, with rotation $\mathbf{R}^{\text{cam}}_{\text{base}}$ and translation $\mathbf{t}^{\text{cam}}_{\text{base}}$, maps every frustum point from the camera's optical frame into the robot base frame:

   $$\mathbf{p}_{\text{body}} = \mathbf{R}^{\text{cam}}_{\text{base}}\mathbf{p}_c + \mathbf{t}^{\text{cam}}_{\text{base}}$$

   The workspace is discretized into a grid of vertical pillars, for example 200 cells on a side covering a 20 m square at 0.1 m resolution. The splat sums every frustum feature that falls within pillar (X,Y), using accelerated cumulative sums or point-pillar pooling (Philion and Fidler 2020; Z. Liu et al. 2023):

   $$\mathbf{V}_{\text{BEV}}(X,Y) = \sum_{(u,v,k)\in\text{Pillar}(X,Y)} \mathbf{c}_{u,v,k}$$

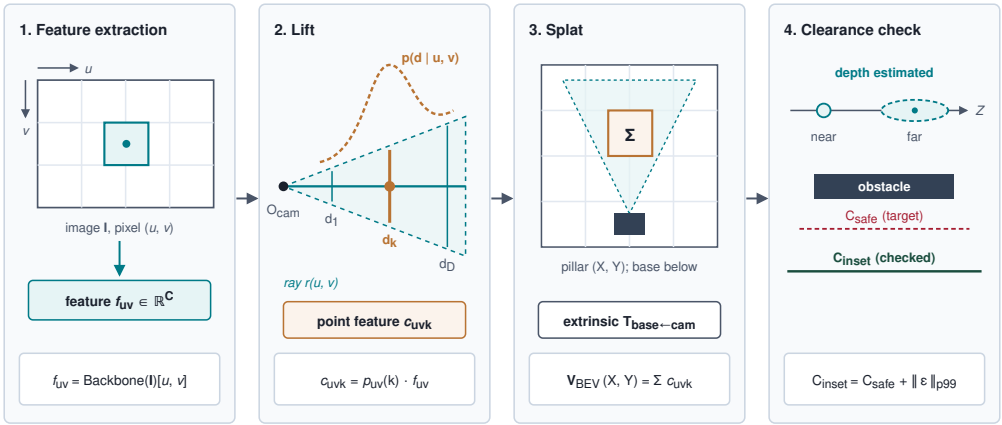


Figure A.1: Camera ray frustum lifting and BEV splatting: LSS (Philion and Fidler 2020) places learned depth hypotheses from each camera in a shared metric BEV frame; that coordinate transform does not resolve monocular depth ambiguity. The final clearance inset requires separately validated metric residuals, bias, and a chosen tail quantile or deterministic error bound before permission. (Architecture adapted from Philion and Fidler (2020).)

On parallel GPU accelerators, naive scatter-add operations create severe write-port contention and non-coalesced DRAM transactions when hundreds of frustum points map to the identical BEV pillar. Optimized runtimes eliminate this bottleneck by sorting frustum points by pillar index and executing parallel cumulative-sum prefix scans or precomputed index-table indirection (Z. Liu et al. 2023). The splat aligns the cameras' hypotheses in one metric frame. It cannot fill a region no camera saw or remove depth-scale error, so the footprint it assigns to a pedestrian still depends on the depth hypotheses and the calibration. That dependence is why the clearance panel closing figure A.1 marks depth as estimated and only the inset as checked.

The softmax over depth bins expresses weights the model assigns, and its entropy is not a calibrated metric covariance. The clearance inset of section 8.6 therefore rests on separately validated spatial residuals, not on the lifted weights.

A.3.4 Inverse-depth parameterization

In visual-inertial odometry and SLAM, representing distant 3D landmarks in standard Cartesian coordinates (x, y, z) induces extreme non-linearity: small pixel tracking errors produce quadratic depth errors ($\delta z \propto z^2$). Under **inverse-depth parameterization**, a 3D feature is parameterized by the 6D vector $\mathbf{y} = [x_0, y_0, z_0, \theta_{az}, \phi, \rho]^\top$. Here (x_0, y_0, z_0) is the optical center when first observed, (θ_{az}, ϕ) are azimuth and elevation angles, and $\rho = 1/d$ is inverse depth:[3]

$$\mathbf{p}(d) = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} + \frac{1}{\rho} \begin{bmatrix} \cos\phi \sin\theta_{az} \\ -\sin\phi \\ \cos\phi \cos\theta_{az} \end{bmatrix}$$

At optical infinity ($d \to \infty$), inverse depth approaches zero ($\rho \to 0$). This parameterization can improve estimator conditioning for distant features; covariance shape and numerical stability still depend on the measurement model, noise assumptions, and estimator implementation.

[3] **Inverse-Depth Parameterization**: Civera and colleagues used inverse depth to represent distant visual landmarks in monocular estimation.

A.3.5 3D Gaussian splatting parameters

A neural scene reconstructed from camera images can render a photorealistic view of a workcell while filling unobserved regions with semi-transparent floaters or hollow shells that offer no conservative collision surface. Continuous neural scene representations parameterize physical environments through collections of 3D Gaussians (Kerbl et al. 2023), where each Gaussian $G_i(\mathbf{p})$ is defined by four core physical and radiometric attributes:

1. **Spatial mean ($\mu_i \in \mathbb{R}^3$):** The 3D centroid in world coordinates.
2. **Covariance matrix ($\Sigma_i \in \mathbb{R}^{3\times3}$):** Defining ellipsoidal shape and orientation. To guarantee that Σ_i remains symmetric and positive semi-definite throughout gradient descent, it is factorized into an orthonormal rotation matrix $\mathbf{R}_i \in SO(3)$ (parameterized by a unit quaternion $\mathbf{q}_{\text{quat}_i}$) and a positive scaling diagonal matrix $\mathbf{S}_i = \text{diag}(s_{x_i}, s_{y_i}, s_{z_i})$:
$$\Sigma_i = \mathbf{R}_i \mathbf{S}_i \mathbf{S}_i^\top \mathbf{R}_i^\top$$
3. **Radiance appearance (c_i):** Parameterized via Spherical Harmonics (SH) coefficients up to degree 3 (16 coefficients per color channel) to capture view-dependent specular reflections.
4. **Opacity ($\alpha_i \in [0,1]$):** Modeled via a sigmoid activation $\alpha_i = \sigma(o_i) = \frac{1}{1+e^{-o_i}}$ over unbounded logit $o_i \in \mathbb{R}$ to bound opacity. Sigmoid gradients can still vanish near saturation.

A.3.5.1 Camera projection and screen-space covariance

During real-time novel-view synthesis, 3D Gaussians are transformed into the camera optical frame via rigid extrinsic transform $\mathbf{T}_{CW} \in SE(3)$, where $\mathbf{R}_{CW} \in SO(3)$ is the rotational component:
$$\mathbf{p}_c = [x, y, z]^\top = \mathbf{R}_{CW}\mu_i + \mathbf{t}_{CW}$$
The pinhole camera model projects 3D point $\mathbf{p}_c$ to image pixel coordinates $[u, v]^\top$:
$$\pi(x, y, z) = \begin{bmatrix} f_x \frac{x}{z} + c_x \\ f_y \frac{y}{z} + c_y \end{bmatrix}$$
Because perspective projection is non-linear in depth z, 3D Gaussian Splatting applies an affine first-order Taylor approximation evaluated at the Gaussian center $\mathbf{p}_c$. The projection Jacobian $\mathbf{J} \in \mathbb{R}^{2\times3}$ evaluates to:
$$\mathbf{J} = \frac{\partial \pi}{\partial \mathbf{p}_c} = \begin{bmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} & \frac{\partial u}{\partial z} \\ \frac{\partial v}{\partial x} & \frac{\partial v}{\partial y} & \frac{\partial v}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{f_x}{z} & 0 & -\frac{f_x x}{z^2} \\ 0 & \frac{f_y}{z} & -\frac{f_y y}{z^2} \end{bmatrix}$$
The projected 2D screen-space covariance $\Sigma_{2D} \in \mathbb{R}^{2\times2}$ is then:
$$\Sigma_{2D} = \mathbf{J}\mathbf{R}_{CW}\Sigma_i\mathbf{R}_{CW}^\top\mathbf{J}^\top$$
where a low-pass filter ($\Sigma'_{2D} = \Sigma_{2D} + s\mathbf{I}_2$ with $s \approx 0.3$ pixel^2) is added to prevent aliasing when a Gaussian projects to sub-pixel footprints.

A.3.5.2 Differentiable alpha-blending and clearance limits

To render pixel color $C(u, v)$, tile-based rasterization bins Gaussians into 16×16 pixel screen tiles, sorts overlapping Gaussians along the camera optical axis by depth z, and computes front-to-back alpha-blended volume rendering:
$$C = \sum_{i\in\mathcal{N}} c_i \alpha'_i T_i, \quad \text{where } T_i = \prod_{j=1}^{i-1}(1-\alpha'_j)$$
Here T_i represents cumulative optical transmittance along the viewing ray, initialized to $T_1 = 1$. In this rendering equation, α'_i is the pixel-dependent opacity contribution, including the projected Gaussian footprint, rather than the stored opacity α_i. An implementation may terminate when the remaining transmittance is sufficiently small; the error tolerance, scene density, and hardware determine the resulting speed.

However, a physical AI systems engineer must recognize the boundary between visual rendering and physical clearance. The 3D Gaussian parameters are optimized strictly against 2D photometric rendering loss (such as ℓ_1 color difference and SSIM). Gaussians frequently fit unobserved scene regions with semi-transparent "floaters," hollow shells, or blurred volumetric density. Neither the stored opacity α_i nor the spatial gradient $\nabla_{\mathbf{p}} G_i(\mathbf{p})$ of its density constitutes a conservative collision

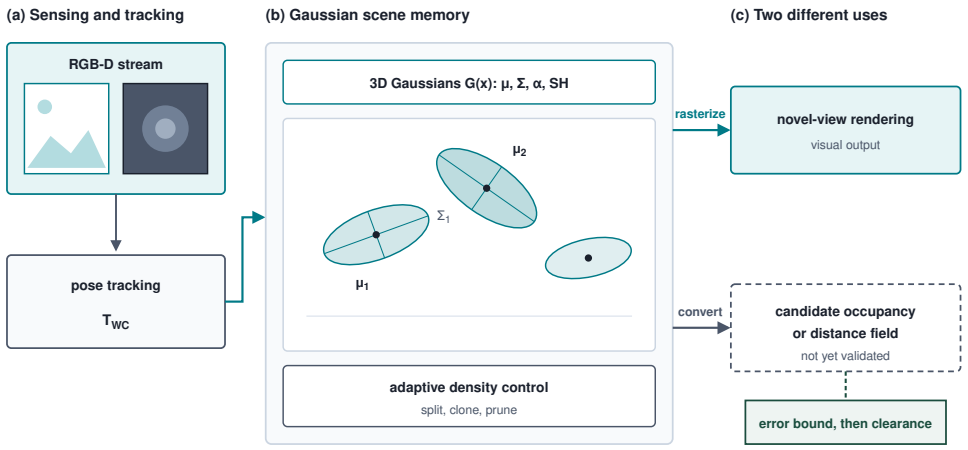


Figure A.2: Gaussian memory and clearance handoff: Camera observations update a Gaussian scene for novel-view rendering. Its visual output does not directly supply conservative collision distance; a separately validated geometry or occupancy conversion and permission check are required (Kerbl et al. 2023; Matsuki et al. 2024).

surface. Figure A.2 partitions this architectural handoff across three functional panels. Panel (a) tracks camera poses from an incoming RGB-D stream. Panel (b) maintains the 3D Gaussian memory primitives (mean, covariance ellipsoid, opacity, and spherical-harmonic color) under adaptive splitting, cloning, and pruning. Panel (c) decouples photometric rasterization from physical safety, routing Gaussian primitives through an unvalidated candidate occupancy or signed distance field before an explicit error-bounding step admits the clearance envelope.

Storage per primitive depends on its packing and spherical-harmonic degree, so a map's footprint is its primitive count times that packing. A dense cubic grid instead stores $\mathcal{O}(N^3)$ cells, and a sparse occupancy map stores only allocated structure, whose size depends on the observed geometry and the accuracy target. Figure A.3 plots these three storage scaling regimes on logarithmic axes as grid resolution spans from 10 to 10^4 cells per dimension. The cubic grid grows steeply as $\mathcal{O}(N^3)$ and passes gigabytes at modest resolutions. The surface-allocated octree scales along an $\mathcal{O}(N^2)$ trajectory. The stylized Gaussian scene maintains a flat baseline under an assumed fixed primitive count of 500,000 elements.

For the Gaussian packing in the plot, 500,000 primitives at 56 bytes each require 28 MB. A deployment chooses its representation after measuring map completeness, update time, query time, and the conversion needed for conservative clearance.

A.3.6 Log-odds occupancy and signed-distance fusion

When a pallet jack moves out of a warehouse aisle, an occupancy map that records only range hits leaves a phantom obstacle blocking the path unless subsequent sensor rays actively clear the empty sightlines. Discrete volumetric grids model 3D occupancy uncertainty through recursive Bayesian estimation. Let $m_i \in \{0, 1\}$ denote the binary occupancy state of voxel i ($m_i = 1$ occupied, $m_i = 0$ free). Given a temporal sequence of range observations $z_{1:t} = (z_1, \ldots, z_t)$, Bayes' rule expresses the posterior occupancy:

$$p(m_i \mid z_{1:t}) = \frac{p(z_t \mid m_i, z_{1:t-1}) p(m_i \mid z_{1:t-1})}{p(z_t \mid z_{1:t-1})}$$

Assuming that observation z_t is conditionally independent of past observations given the true static occupancy m_i ($p(z_t \mid m_i, z_{1:t-1}) = p(z_t \mid m_i)$), and writing the identical relation for the unoccupied state $\neg m_i$ ($m_i = 0$):

$$p(\neg m_i \mid z_{1:t}) = \frac{p(z_t \mid \neg m_i) p(\neg m_i \mid z_{1:t-1})}{p(z_t \mid z_{1:t-1})}$$

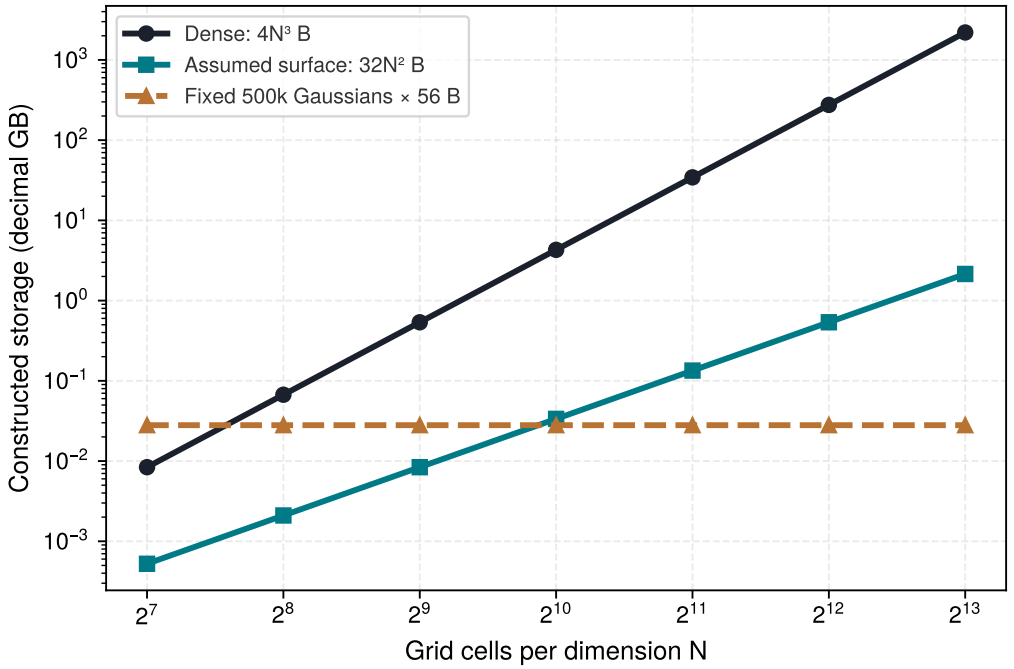


Figure A.3: Constructed spatial-memory storage scenarios: A dense cube uses four bytes per voxel, the octree allocates one 32-byte surface cell per squared grid dimension, and the Gaussian scene holds a fixed count and packing while grid resolution varies. These are storage models, not equal-fidelity or measured runtime benchmarks; Gaussian count must rise if a stricter geometry target requires it.

Dividing the two equations cancels the common evidence normalizer $p(z_t \mid z_{1:t-1})$:

$$\frac{p(m_i \mid z_{1:t})}{p(\neg m_i \mid z_{1:t})} = \frac{p(z_t \mid m_i)}{p(z_t \mid \neg m_i)} \frac{p(m_i \mid z_{1:t-1})}{p(\neg m_i \mid z_{1:t-1})}$$

Applying Bayes' rule to the measurement likelihood $p(z_t \mid m_i) = \frac{p(m_i|z_t)p(z_t)}{p(m_i)}$ gives the measurement odds ratio:

$$\frac{p(z_t \mid m_i)}{p(z_t \mid \neg m_i)} = \frac{p(m_i \mid z_t)}{1-p(m_i \mid z_t)} \frac{1-p(m_i)}{p(m_i)}$$

Taking the natural logarithm of the odds ratio defines the log-odds quantity $L(x) = \ln \frac{p(x)}{1-p(x)}$. Substituting yields the canonical **additive log-odds update**:[4]

$$L_t(m_i) = L_{t-1}(m_i) + \underbrace{\ln \frac{p(m_i \mid z_t)}{1-p(m_i \mid z_t)}}_{\text{inverse sensor model } L_{\text{sensor}}} - \underbrace{\ln \frac{p(m_i)}{1-p(m_i)}}_{\text{prior belief } L_0}$$

where $p(m_i \mid z_t)$ is the calibrated inverse sensor model. When prior belief is uninformative ($p(m_i) = 0.5 \implies L_0 = 0$), the update reduces to simple addition: $L_t(m_i) = L_{t-1}(m_i) + L_{\text{sensor}}$.

Along a range sensor ray, voxels traversed before the hit receive negative log-odds evidence ($L_{\text{free}} < 0$), while the voxel containing the return receives positive evidence ($L_{\text{occ}} > 0$). Voxels beyond the return receive no update. With the uninformative prior used here, zero evidence corresponds to $L_{\text{sensor}} = 0$. To prevent numerical saturation lockup where an obstacle that was observed occupied hundreds of times requires hundreds of empty observations before clearing, updates are clamped within finite bounds $[L_{\min}, L_{\max}]$ (e.g., $[-2.0, +3.5]$).

[4] **Recursive Log-Odds Occupancy**: The log-odds formulation maps probability values from the bounded unit interval to the unbounded real line, turning nonlinear Bayesian product updates into fast additions. Dynamic ranges are clamped by upper and lower saturation bounds to prevent saturation lockup. Failure to clamp log-odds values delays the clearing of dynamic obstacles when physical objects vacate the workspace.

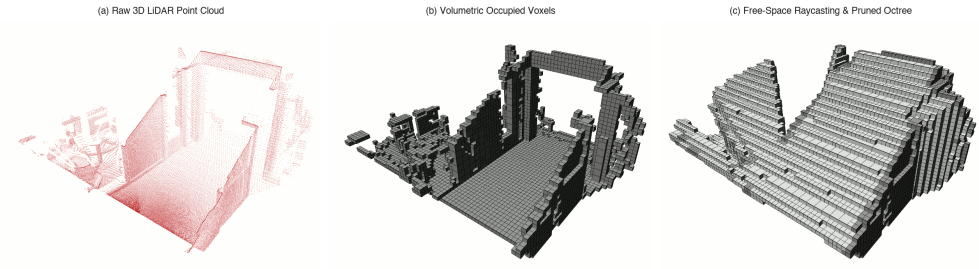


Figure A.4: Probabilistic octree volumetric mapping: A corridor traversal reconstructed from raw point cloud into a probabilistic octree and then into explicit free, occupied, and unknown space (Hornung et al. 2013). The third panel is what distinguishes spatial memory from an accumulated point cloud, because raycasting records evidence of emptiness along sightlines, letting the map actively retire a phantom obstacle track instead of waiting for a timeout to expire it. (Credit: Autonomous Intelligent Systems Lab, University of Freiburg (Hornung et al. 2013).)

A.3.6.1 Octree occupancy and free-space raycasting

Occupancy grid mapping (Elfes 1989) tessellates space into cells and applies this update to every cell a ray touches. A volumetric raycast walks each return's sightline with the three-dimensional digital differential analyzer (3D-DDA) traversal (Amanatides and Woo 1987). Hierarchical stores such as OctoMap (Hornung et al. 2013) and OpenVDB (Museth 2013) allocate cells only where rays have passed, so storage follows the observed volume rather than the bounding cube. Figure A.4 traces this spatial-memory pipeline across three successive representations during an indoor mobile robot traversal. Panel (a) shows the unorganized, noisy 3D point cloud acquired by a tilting range scanner. Panel (b) shows the hierarchical 8-ary octree that adaptively clusters occupied volumes. Panel (c) shows the explicit ternary classification that differentiates confirmed free-space voxels (white) and occupied obstacle cells (dark) from unobserved unknown volumes. Free-space evidence recorded along ray sightlines lets the map retire a phantom obstacle once the same region is seen empty, while a never-observed cell stays unknown however long the map runs. Section 9.6 applies this rule to the aisle, where only fresh free-space evidence can vouch that a path is clear.

A.3.6.2 Truncated signed distance fusion

A truncated signed distance field (TSDF), fused volumetrically at camera rate by KinectFusion (Newcombe et al. 2011) and incrementally on robots by Voxblox (Oleynikova et al. 2017), stores in each voxel $\mathbf{p}_{\text{vox}}$ a *metric projective distance* $D(\mathbf{p}_{\text{vox}}) \in [-\mu_{\text{tsdf}}, +\mu_{\text{tsdf}}]$. This distance is positive in front of an observed return along its sensing ray. Each voxel also stores a dimensionless fusion weight $W(\mathbf{p}_{\text{vox}})$. For a calibrated depth image, transform the voxel center into camera coordinates $\mathbf{p}_c = [X_c, Y_c, Z_c]^\top$ and project it to pixel (u, v). Its projective sample is $s = Z_{\text{meas}}(u, v) - Z_c$. For a range ray, the equivalent along-ray sample is $s = r_{\text{hit}} - r_{\text{voxel}}$; both are positive before the hit. Near the return, the sample is clamped in meters and fused into the running weighted average:

$$d_\mu(\mathbf{p}_{\text{vox}}) = \max(-\mu_{\text{tsdf}}, \min(\mu_{\text{tsdf}}, s)), \qquad D_t(\mathbf{p}_{\text{vox}}) = \frac{W_{t-1}(\mathbf{p}_{\text{vox}}) D_{t-1}(\mathbf{p}_{\text{vox}}) + w_t d_\mu(\mathbf{p}_{\text{vox}})}{W_{t-1}(\mathbf{p}_{\text{vox}}) + w_t}$$

where w_t is the dimensionless observation weight and the stored weight $W_t = W_{t-1} + w_t$ is capped at $W_{\max}$. Fusion uses a valid capture-time pose and refuses a sample older than the voxel's last evidence epoch. A negative sample just behind the measured return contributes to the local zero crossing under the surface model; it does not certify an unseen obstacle interior.

Interpolating observed samples gives an approximate gradient near a zero crossing, limited by unknown voxels, projective bias, discretization, and moving objects. A projective TSDF is therefore not the nearest-surface Euclidean distance a clearance decision needs. A clearance query therefore reads a validated occupancy or Euclidean signed distance field (ESDF) with an unknown-space policy, the conversion section 9.6 requires of every scene representation before the permission path relies on it.

A.3.6.3 One frame's update and its memory traffic

Each incoming depth frame must pass a freshness gate before updating occupancy along its rays and fusing TSDF samples inside the truncation band (algorithm A.1), and the notebook after the pseudocode prices the resulting update traffic that section 9.6 charges against the memory bus. The update procedure rejects the frame before touching the map when its capture epoch is too old or older than the last evidence of any cell it would update. It then clears and marks occupancy along each ray and fuses TSDF samples only inside the truncation band. Every updated cell takes the capture epoch, not the update time, so a later consumer can judge each cell's freshness.

Algorithm A.1 Occupancy Raycast and Metric TSDF Fusion

Require: returns $\mathcal{P}$ captured at t_0 in one capture clock; valid pose; occupancy $\mathcal{O}$; metric TSDF $\mathcal{V}$; truncation μ; evidence half-life $t_{1/2}$; serialized frame updates
Ensure: observed free/occupied cells and near-surface TSDF samples updated without refreshing unseen cells

$(t_{0,c}, \epsilon_{\text{sync}}) \leftarrow$ ConvertCaptureToControllerClock(t_0)
if clock conversion invalid or $t_{\text{now},c} - t_{0,c} + \epsilon_{\text{sync}} > t_{\text{age,max}}$ **then**
  return STATUS_FRAME_TOO_STALE
end if
if any target occupancy or TSDF cell has $t_{\text{last}} > t_0$ **then**
  return STATUS_OUT_OF_ORDER ▷ *preflight before modifying the map*
end if
for each return $\mathbf{p}_k \in \mathcal{P}$ **do**
  $(\mathbf{r}_{\text{dir}}, r_{\text{hit}}) \leftarrow$ RayFromPoseAndReturn$(\mathbf{p}_k)$
  ▷ *Occupancy: measured free ray and hit, not TSDF distance*
  for each cell c traversed before the hit **do**
    $c \leftarrow$ GetOrAllocateOccupancy$(\mathcal{O}, c)$
    $c.L \leftarrow$ ClampLogOdds$(c.L + L_{\text{free}})$; $c.t_{\text{last}} \leftarrow t_0$
  end for
  update the observed hit cell with L_{occupied} and epoch t_0
  ▷ *TSDF: fuse metric samples only in a narrow surface band*
  for each voxel v with $|r_{\text{hit}} - r_v| \le \mu$ **do**
    vox $\leftarrow$ GetOrAllocateTSDF$(\mathcal{V}, v)$
    $d_\mu \leftarrow$ Clamp$(r_{\text{hit}} - r_v, -\mu, +\mu)$ ▷ *positive before hit*
    $W_{\text{old}} \leftarrow 0$ if vox.$W = 0$, else vox.$W\, 2^{-(t_0 - \text{vox}.t_{\text{last}})/t_{1/2}}$
    $w_{\text{obs}} \leftarrow 1$ ▷ *dimensionless illustrative confidence*
    vox.$D \leftarrow (W_{\text{old}}\text{vox}.D + w_{\text{obs}} d_\mu)/(W_{\text{old}} + w_{\text{obs}})$
    vox.$W \leftarrow \min(W_{\text{old}} + w_{\text{obs}}, W_{\text{max}})$; vox.$t_{\text{last}} \leftarrow t_0$
  end for
end for
return STATUS_MAP_UPDATED

Napkin Math A.1: DRAM bandwidth of 3D TSDF voxel grids

Practical Engineering Question: What are the distance-and-weight payload and modeled DRAM bandwidth of sparse vs. dense TSDF representations?

1. Dense allocation. A manipulation workcell of 10 m × 10 m × 3 m (300 m^3) at 1 cm pitch holds 300,000,000 voxels. At 4 bytes per voxel (16-bit distance plus 16-bit fusion weight), excluding occupancy, per-cell evidence epochs, and allocator overhead, the distance-and-weight payload is 1.20 GB. Refining the pitch to 5 mm for tool contact raises the count to 2,400,000,000 voxels and the partial payload to 9.60 GB, which alone exceeds an illustrative 8 GB memory budget.

2. Sparse allocation. Surfaces occupy a thin manifold within the volume. Grouping voxels into 512-voxel leaf blocks and allocating only within the truncation band, assumed to be 3 percent

of the volume, gives 17,579 blocks. With a 32-byte leaf header, the payload is 36.6 MB at 1 cm and 292.5 MB at 5 mm. A full map must also budget the separate occupancy layer, per-cell epochs, block lookup and allocator metadata, and capacity for the unknown-space policy.

3. Traversal and DRAM traffic. A 640 × 480 depth stream at 60 Hz supplies 18.432 million rays per second. Along a 3 m ray at 1 cm pitch the traversal visits 300 cells. The 294 cells before the hit receive occupancy free-ray updates, and the 6 cells near the return receive TSDF fusion. Sparse TSDF allocation saves map capacity; it removes neither the traversal nor the occupancy updates. At 8 bytes of assumed read/write traffic per update, sending every update to DRAM costs 44.2 GB/s. At an assumed 95 percent cache hit fraction for both occupancy and TSDF updates, the modeled DRAM demand falls to 2.21 GB/s, a 20× reduction. The hit rate depends on ray overlap, atomics, evictions, and writeback on the chosen hardware; it is an input to measure, not a property of the map structure.

Systems insight: Sparse allocation yields 33× less modeled TSDF payload in this surface-fraction scenario, while bus demand depends on measured locality and on all traversed-cell work. Whether that demand fits beside policy inference on a shared bus is the contention question of section 13.3.

A.4 Function Approximation and Sensitivity Bounds

A sensor perturbation can change a neural action proposal even when the physical scene has barely moved. A sensitivity bound limits that change relative to the input error and supplies a quantity to compare with available actuator headroom. The bound concerns the proposal; the permission path must still check the plant state and enforce current limits.

A.4.1 Multilayer perceptrons in continuous control

For high-rate proprioceptive feedback loops (such as 500–1000 Hz quadruped balance, bipedal locomotion, and arm impedance control), physical policies frequently employ **Multilayer Perceptrons (MLPs)** rather than deep spatial transformers. A canonical continuous MLP policy takes a low-dimensional proprioceptive vector $\mathbf{x} \in \mathbb{R}^d$ (joint positions $\mathbf{q}$, velocities $\dot{\mathbf{q}}$, IMU angular rates, projected gravity, and previous action history). It then evaluates a sequence of N_L affine transformations interleaved with element-wise non-linear activations:

$$\mathbf{h}_0 = \mathbf{x}, \quad \mathbf{h}_\ell = f_\ell(\mathbf{W}_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell) \quad \text{for } \ell = 1, \dots, N_L, \quad \pi(\mathbf{x}) = \mathbf{W}_{\text{out}} \mathbf{h}_{N_L} + \mathbf{b}_{\text{out}}$$

where $\mathbf{W}_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ are weight matrices and $\mathbf{b}_\ell \in \mathbb{R}^{n_\ell}$ are bias vectors.

A.4.1.1 Systems profile and roofline limits of MLP policies

A compact MLP can keep its weights and activations in on-chip memory, reducing interference from camera direct memory access and other DRAM clients. Capacity alone does not establish residency: cache replacement, shared clients, scheduling, and accelerator transfers determine the memory level that supplies each layer.

For a dense layer with $d_{\text{in}} d_{\text{out}}$ weights, a single input vector requires approximately $2 d_{\text{in}} d_{\text{out}}$ floating-point operations. If each weight occupies $s_{\text{elem,W}}$ bytes and is read once from the memory level being analyzed, the weight-only arithmetic intensity is

$$I_{\text{MLP}} \approx \frac{2}{s_{\text{elem,W}}} \text{ FLOP/byte.}$$

Activation traffic lowers this ratio; weight reuse across inputs raises it. The relevant roofline compares this intensity with compute throughput divided by bandwidth at that same memory level. Cache-resident weights therefore change the applicable bandwidth, not the amount of arithmetic.

A deployment budgets weights, scratch buffers, and hidden activations together, preallocates runtime storage, and measures or bounds worst-case execution under interference. A short average forward pass does not by itself justify placing inference in an interrupt handler or guarantee a control deadline.

A.4.2 Lipschitz continuity and spectral normalization

Bounding how strongly a network amplifies input perturbations requires a global sensitivity constant. A neural policy $\pi : \mathcal{X} \to \mathcal{U}$ is L_{lip}-Lipschitz continuous if there exists a non-negative scalar constant L_{lip} such that for all $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}$:

$$\|\pi(\mathbf{x}_1) - \pi(\mathbf{x}_2)\|_{\mathcal{U}} \leq L_{\text{lip}} \|\mathbf{x}_1 - \mathbf{x}_2\|_{\mathcal{X}}$$

For a feedforward network with N_L layers $\pi(\mathbf{x}) = f_{N_L}(\mathbf{W}_{N_L} \dots f_1(\mathbf{W}_1 \mathbf{x}))$, let L_{ϕ_k} bound each activation's slope over the input domain. The composed Lipschitz bound is:

$$L_{\text{lip}} \leq \prod_{k=1}^{N_L} L_{\phi_k} \|\mathbf{W}_k\|_2$$

where $\|\mathbf{W}_k\|_2 = \sigma_{\max}(\mathbf{W}_k)$ is the spectral norm (the largest singular value of weight matrix $\mathbf{W}_k$). Standard activations such as ReLU and Tanh are 1-Lipschitz in their scalar forms; exact GeLU has maximum derivative about 1.1289, so assigning it a factor of one would understate the bound.

The product is an upper bound, not necessarily the smallest Lipschitz constant. Its physical interpretation depends on normalization. A network receiving position and velocity in different units needs declared input scales; converting its normalized output to torque introduces another scale factor. Without those conversions, multiplying a position error by a dimensionless network bound cannot yield a torque error.

To bound policy sensitivity directly during training, **spectral normalization** reparameterizes each weight matrix by its spectral norm:

$$\overline{\mathbf{W}}_k = \frac{\mathbf{W}_k}{\sigma_{\max}(\mathbf{W}_k)}$$

where $\sigma_{\max}(\mathbf{W}_k)$ is estimated efficiently via power iteration during forward passes. Division by the exact nonzero spectral norm gives $\|\overline{\mathbf{W}}_k\|_2 = 1$. Finite power iteration generally gives an estimate, not a certified upper bound; a safety argument must account for its error. Even an exact network sensitivity bound does not prove closed-loop plant stability.

A.4.3 Perturbation amplification and supervisory clamping

In physical systems, input state $\mathbf{x}$ is corrupted by sensor measurement noise $\delta\mathbf{x}$ bounded by $\|\delta\mathbf{x}\| \leq \epsilon$. The resulting deviation in commanded control action is bounded by:

$$\|\delta\mathbf{u}_{\text{nom}}\| = \|\pi(\mathbf{x} + \delta\mathbf{x}) - \pi(\mathbf{x})\| \leq L_{\text{lip}}\epsilon$$

The constant has output-units per input-unit once normalization is included. Compare $L_{\text{lip}}\epsilon$ with the actuator headroom at the current operating state. Spectral normalization bounds the network, while stability, safe torque, and sensor validity require a plant model and a state-dependent permission check.

A.5 Imitation Learning and Compounding Covariate Shift

Behavioral cloning fits a policy to demonstrator state-action pairs. Two different failures need different remedies: ambiguous demonstrations can make a point predictor average incompatible actions, while feedback can carry even an accurate predictor into states missing from its training data. The decoder derivations address the first problem; the coupling bound and DAgger address the second.

A.5.1 Mode averaging and the decoders that avoid it

When human demonstrators steer left around a central pillar half the time ($\mathbf{a} = +1.0$) and right the other half ($\mathbf{a} = -1.0$), a policy trained with mean squared error averages the two commands to $\mathbf{a} = 0.0$ and drives dead center into the pillar. To see why squared-error regression produces that collision, consider an imitation learning objective minimizing mean squared error (ℓ_2 loss) over demonstration distribution $\mathcal{D}_{\text{demo}}$:

$$\min_{\pi} \mathbb{E}_{(\mathbf{o},\mathbf{a})\sim\mathcal{D}_{\text{demo}}} \left[\|\mathbf{a} - \pi(\mathbf{o})\|_2^2\right]$$

By the law of total expectation, decomposing the error around the conditional mean $\mathbb{E}[\mathbf{a} \mid \mathbf{o}]$ yields an orthogonal variance decomposition:

$$\begin{aligned}\mathbb{E}_{\mathbf{a}|\mathbf{o}}\left[\|\mathbf{a}-\pi(\mathbf{o})\|_2^2 \mid \mathbf{o}\right] &= \mathbb{E}_{\mathbf{a}|\mathbf{o}}\left[\|(\mathbf{a}-\mathbb{E}[\mathbf{a} \mid \mathbf{o}])+(\mathbb{E}[\mathbf{a} \mid \mathbf{o}]-\pi(\mathbf{o}))\|_2^2 \mid \mathbf{o}\right] \\ &= \mathbb{E}_{\mathbf{a}|\mathbf{o}}\left[\|\mathbf{a}-\mathbb{E}[\mathbf{a} \mid \mathbf{o}]\|_2^2 \mid \mathbf{o}\right]+\|\mathbb{E}[\mathbf{a} \mid \mathbf{o}]-\pi(\mathbf{o})\|_2^2 \\ &\quad +2\left\langle \mathbb{E}_{\mathbf{a}|\mathbf{o}}[\mathbf{a}-\mathbb{E}[\mathbf{a} \mid \mathbf{o}] \mid \mathbf{o}],\, \mathbb{E}[\mathbf{a} \mid \mathbf{o}]-\pi(\mathbf{o})\right\rangle\end{aligned}$$

Because $\mathbb{E}_{\mathbf{a}|\mathbf{o}}[\mathbf{a}-\mathbb{E}[\mathbf{a} \mid \mathbf{o}] \mid \mathbf{o}]=\mathbf{0}$, the cross-term vanishes identically, leaving:

$$\mathbb{E}_{(\mathbf{o},\mathbf{a})\sim\mathcal{D}_{\text{demo}}}\left[\|\mathbf{a}-\pi(\mathbf{o})\|_2^2\right]=\mathbb{E}_{\mathbf{o}}\left[\operatorname{Tr}(\operatorname{Cov}(\mathbf{a} \mid \mathbf{o}))\right]+\mathbb{E}_{\mathbf{o}}\left[\|\mathbb{E}[\mathbf{a} \mid \mathbf{o}]-\pi(\mathbf{o})\|_2^2\right]$$

The first term is the irreducible conditional variance of demonstrator behavior. The second term is non-negative and strictly minimized if and only if the policy evaluates to the **conditional expectation**:

$$\pi^*(\mathbf{o})=\mathbb{E}[\mathbf{a} \mid \mathbf{o}]=\int_{\mathcal{A}} \mathbf{a}\, p(\mathbf{a} \mid \mathbf{o})\, d\mathbf{a}$$

In digital machine learning tasks (such as computer vision regression), predicting the conditional expectation over ambiguous targets produces a blurry reconstruction, a soft pixel average that remains safely confined to software memory. In physical AI systems, commanded actions cross the causal boundary into continuous Euclidean space governed by non-convex obstacles, joint limits, and irreversible contact dynamics.

Demonstrator behavior in physical manipulation and navigation is inherently multimodal:

- Dodging an obstacle: an expert steers left ($\mathbf{a}=+1.0$) or right ($\mathbf{a}=-1.0$) around a central pillar with equal probability $\Pr(\mathbf{a}=+1 \mid \mathbf{o})=\Pr(\mathbf{a}=-1 \mid \mathbf{o})=0.5$.
- Symmetrical manipulation: an end effector approaches a cylindrical object from azimuth $\theta_{\text{az}}=0^\circ$ or $\theta_{\text{az}}=180^\circ$.
- Assembly routing: threading a flexible cable clockwise or counter-clockwise around a structural fastener.

Because demonstration density $p(\mathbf{a} \mid \mathbf{o})$ is supported on disjoint, disconnected modes separated by an obstacle, the conditional expectation computes the arithmetic mean:

$$\pi^*(\mathbf{o})=0.5(+1.0)+0.5(-1.0)=0.0$$

This mean action lies in a region of zero demonstration probability ($\Pr(\mathbf{a}=0.0 \mid \mathbf{o})=0$). In the physical workspace, the coordinate $\mathbf{a}=0.0$ corresponds precisely to the geometric position of the central obstacle. Commanding $\pi^*(\mathbf{o})=0.0$ instructs the steering actuator to hold dead center at full cruising speed, driving the physical robot directly into the pillar. In physical autonomy, regression mode averaging does not produce a cosmetic blur; it can propose a collision unless the permission path rejects it.

A.5.1.1 Decoder families

A probabilistic decoder can avoid mode averaging by parameterizing the conditional action distribution $p(\mathbf{a} \mid \mathbf{o})$ rather than its expectation. The families below trade inference cost against distributional expressiveness; none of them supplies recovery states that the demonstrations lack. Section 6.2 compares them by the network evaluations each chunk costs, which is where the choice reaches the control deadline.

1. **Deterministic direct regression (MLP).** A feedforward network trained with squared error predicts one action per observation and, with sufficient capacity and ideal optimization, fits the conditional mean. The restriction comes from the point-output objective, not from MLPs as a network family; an MLP can also parameterize a mixture or stochastic policy. One evaluation avoids iterative sampling cost but does not resolve ambiguous demonstrations.

2. **Mixture density networks.** A Gaussian mixture head predicts nonnegative weights summing to one, means, and positive-definite covariances:

$$\pi(\mathbf{a} \mid \mathbf{o}) = \sum_{m=1}^{M} w_m(\mathbf{o})\, \mathcal{N}(\mathbf{a}; \mu_m(\mathbf{o}), \Sigma_m(\mathbf{o})).$$

   Sampling a component or selecting a mode preserves alternatives in one network evaluation. Computing the mixture mean would restore the original averaging problem. Covariance regularization and component coverage matter because likelihood optimization can collapse a component onto a few observations.

3. **Implicit behavioral cloning.** Implicit behavioral cloning (IBC) (Florence et al. 2022) learns an energy-based model $E_\theta(\mathbf{o}, \mathbf{a}) \in \mathbb{R}$, trained with contrastive or InfoNCE objectives to assign low energy to demonstrated state-action pairs and high energy to counterfactual actions. Inference solves

$$\mathbf{a}^* = \arg\min_{\mathbf{a}} E_\theta(\mathbf{o}, \mathbf{a})$$

   An energy function can have several separated minima, and selecting its minimizing action can change discontinuously even when the energy network is continuous. Inference requires iterative derivative-free or gradient-based optimization whose latency and convergence depend on sample counts and accelerator parallelism, with risks of settling into suboptimal local energy minima.

4. **Conditional diffusion and flow matching.** Diffusion Policy (Chi et al. 2025) and flow-matching decoders generate entire action trajectory chunks via iterative denoising or ordinary differential equation (ODE) integration. Their training targets injected noise vectors or continuous velocity fields conditioned on corrupted states, observations, and timesteps rather than clean actions, allowing multimodal sampling instead of directly regressing a clean action mean. Approximation error and the sampling procedure still affect whether distinct modes survive. Section A.5.2 derives both objectives and prices their sampling steps.

5. **Latent-variable chunking.** Action Chunking with Transformers (ACT) (Zhao et al. 2023) predicts an action trajectory chunk $\mathbf{A}_t = (\mathbf{a}_t, \mathbf{a}_{t+1}, \ldots, \mathbf{a}_{t+H-1}) \in \mathbb{R}^{H \times d_a}$ from observation $\mathbf{o}_t$ using a Conditional Variational Autoencoder (CVAE). The true objective is to maximize the conditional log-marginal likelihood of demonstrated chunks across dataset $\mathcal{D}_{\text{demo}}$:

$$\ln p_\theta(\mathbf{A}_t \mid \mathbf{o}_t) = \ln \int_{\mathcal{Z}} p_\theta(\mathbf{A}_t, \mathbf{z} \mid \mathbf{o}_t)\, d\mathbf{z}$$

   Because this marginalization is intractable, ACT introduces a variational posterior $q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)$ parameterized by a transformer encoder. Expanding the identity with $p_\theta(\mathbf{A}_t, \mathbf{z} \mid \mathbf{o}_t) = p_\theta(\mathbf{A}_t \mid \mathbf{o}_t, \mathbf{z}) p_\theta(\mathbf{z})$:

$$\begin{aligned}
\ln p_\theta(\mathbf{A}_t \mid \mathbf{o}_t) &= \int_{\mathcal{Z}} q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \ln p_\theta(\mathbf{A}_t \mid \mathbf{o}_t)\, d\mathbf{z} \\
&= \int_{\mathcal{Z}} q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \ln \left(\frac{p_\theta(\mathbf{A}_t, \mathbf{z} \mid \mathbf{o}_t)}{q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)} \cdot \frac{q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)}{p_\theta(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)} \right) d\mathbf{z} \\
&= \int_{\mathcal{Z}} q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \ln \frac{p_\theta(\mathbf{A}_t, \mathbf{z} \mid \mathbf{o}_t)}{q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)}\, d\mathbf{z} + \int_{\mathcal{Z}} q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \ln \frac{q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)}{p_\theta(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)}\, d\mathbf{z} \\
&= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)} \left[\ln \frac{p_\theta(\mathbf{A}_t \mid \mathbf{o}_t, \mathbf{z}) p_\theta(\mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)} \right] + \mathcal{D}_{\text{KL}} \left(q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \parallel p_\theta(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \right) \\
&\geq \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)} \left[\ln p_\theta(\mathbf{A}_t \mid \mathbf{o}_t, \mathbf{z}) \right] - \mathcal{D}_{\text{KL}} \left(q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \parallel p_\theta(\mathbf{z}) \right) \equiv \mathcal{L}_{\text{ELBO}}(\theta, \phi)
\end{aligned}$$

   Assuming standard isotropic Gaussian prior $p_\theta(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and diagonal Gaussian variational posterior $q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) = \mathcal{N}(\mu_\phi, \text{diag}(\sigma_\phi^2))$, the Kullback-Leibler divergence evaluates in closed form:

$$\mathcal{D}_{\text{KL}} \left(q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}) \right) = \frac{1}{2} \sum_{j=1}^{d_z} \left(\sigma_{\phi,j}^2 + \mu_{\phi,j}^2 - 1 - \ln(\sigma_{\phi,j}^2) \right)$$

Sampling directly as $\mathbf{z} \sim q_\phi$ is a stochastic operation that prevents backpropagation because the node has no analytical derivatives with respect to encoder parameters ϕ. ACT resolves this via the **reparameterization trick**: isolating stochasticity into parameter-free standard normal noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\mathbf{z} = \mu_\phi(\mathbf{o}_t, \mathbf{A}_t) + \sigma_\phi(\mathbf{o}_t, \mathbf{A}_t) \odot \epsilon, \quad \text{yielding } \frac{\partial \mathbf{z}}{\partial \mu_\phi} = \mathbf{I}, \quad \frac{\partial \mathbf{z}}{\partial \sigma_\phi} = \operatorname{diag}(\epsilon)$$

To parameterize the reconstruction likelihood, ACT models $p_\theta(\mathbf{A}_t \mid \mathbf{o}_t, \mathbf{z})$ as an isotropic Laplace distribution:

$$p_\theta(\mathbf{A}_t \mid \mathbf{o}_t, \mathbf{z}) = \frac{1}{(2b)^{Hd_a}} \exp\left(-\frac{\|\mathbf{A}_t - \pi_\theta(\mathbf{o}_t, \mathbf{z})\|_1}{b}\right)$$

Maximizing the log-likelihood is equivalent to minimizing the ℓ_1 norm $\|\mathbf{A}_t - \pi_\theta(\mathbf{o}_t, \mathbf{z})\|_1$. Away from zero residual, ℓ_1 loss gives constant-magnitude gradients with respect to each predicted action component, whereas ℓ_2 loss gives greater influence to large residuals. Neither loss alone filters demonstrator tremor or guarantees freedom from overfitting; those outcomes depend on the data, regularization, and temporal model. Weighting the KL regularizer by hyperparameter $\beta > 0$ yields the full training loss:

$$\mathcal{L}_{\text{ACT}}(\theta, \phi) = \mathbb{E}_{(\mathbf{o}_t, \mathbf{A}_t) \sim \mathcal{D}_{\text{demo}}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\|\mathbf{A}_t - \pi_\theta(\mathbf{o}_t, \mu_\phi + \sigma_\phi \odot \epsilon)\|_1 + \beta \mathcal{D}_{\text{KL}} \left(q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}) \right) \right]$$

The latent variable $\mathbf{z} \in \mathbb{R}^{d_z}$ can encode variation among demonstrations, while the decoder predicts all H actions together. Neither separated latent clusters nor kinematic smoothness follows from the architecture alone. Executing H_{exec} actions before replanning reduces policy interventions, but increases the time before new observations can alter the proposal. Substituting S/H into the single-step compounding bound is not justified without new chunk-level error and cost assumptions.

At test time the variational encoder, which requires the future demonstration chunk, is discarded. Setting $\mathbf{z} = \mathbf{0}$ evaluates the decoder at the prior mean in one forward pass. That latent point is not guaranteed to select the most probable demonstrated mode. Sampling the prior can expose additional modes but also poorly trained latent regions. ACT combines overlapping chunk predictions through **temporal ensembling**; for example, an exponentially recency-weighted convention is:

$$\mathbf{a}_t = \frac{\sum_{i=0}^{\min(t, H-1)} w_i \, \mathbf{A}_{t-i}[i]}{\sum_{i=0}^{\min(t, H-1)} w_i},$$

Each weight w_i decays exponentially with the offset i, so this convention favors recent predictions. Averaging incompatible chunks can still create an invalid action; the admitted command must satisfy current limits.

6. **Discrete action tokens.** Vision-language-action models such as RT-1, RT-2, and OpenVLA (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024) discretize continuous action dimensions into B uniform bins (commonly 256) and emit categorical tokens:

$$\text{bin}(a_k) = \min\left(B - 1, \left\lfloor B \frac{a_k - a_{\min}}{a_{\max} - a_{\min}} \right\rfloor\right)$$

For an action clipped to the declared range, this assigns one of B equal-width bins, including the upper endpoint. Decoding to bin centers gives maximum scalar quantization error $(a_{\max} - a_{\min})/(2B)$. Categorical sampling or mode selection preserves alternatives; taking their mean reintroduces averaging. A decoder that emits one token per action dimension autoregressively needs d_a sequential predictions, although other tokenizations and parallel heads have different costs.

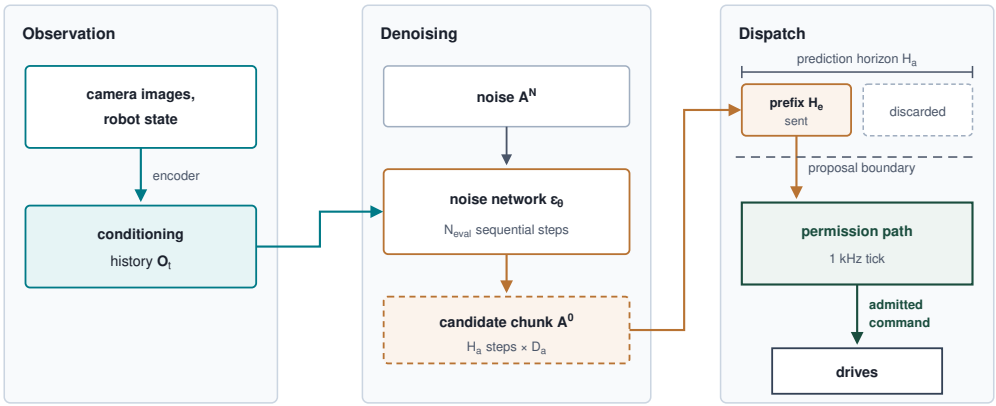


Figure A.5: Diffusion policy architecture: A continuous trajectory diffusion policy for visuomotor control, denoising a candidate action chunk from Gaussian noise across N_{eval} iterations and dispatching only a prefix of it before re-planning. Asynchronous buffering and an independent high-rate permission path can separate slower chunk generation from actuation. Conditional denoising can represent alternative demonstrated paths without taking their direct-regression mean. (Adapted from Chi et al. (2025)).

A.5.2 Continuous trajectory diffusion and flow matching

Sampling one distinct trajectory around an obstacle avoids the collision caused by averaging two paths, but running 16 sequential denoising passes through a 48-million-parameter U-Net takes 40.4 ms and overruns a 25 ms synchronous setpoint deadline. Continuous trajectory diffusion treats policy synthesis as generative denoising over continuous multi-step action trajectory chunks $\mathbf{A} \in \mathbb{R}^{H \times d_a}$ conditioned on visual and proprioceptive observation histories $\mathbf{O}$ (Chi et al. 2025). Here H counts physical action steps, d_a counts action dimensions, and the later N_{eval} counts inference evaluations; diffusion time is not physical execution time. Rather than predicting an unconstrained point estimate via mean squared error, diffusion models represent complex multimodal action distributions without collapsing into mode averaging.

The closed-loop visuomotor diffusion pipeline has three coordinated architectural columns (figure A.5). The left observation column encodes visual camera streams and proprioceptive joint history into a conditioning vector. The center denoising column applies a noise network across N_{eval} sequential reverse-diffusion steps to refine Gaussian noise into a candidate action chunk. The right dispatch column extracts an execution prefix from the prediction horizon and routes those candidate actions across the proposal boundary into the 1 kHz permission path. It also discards the unexecuted tail before re-planning. The noise-prediction backbone ϵ_θ is typically implemented using either a 1D Temporal Convolutional (Conv1D) U-Net or a Diffusion Transformer (DiT):

- **1D temporal convolutional U-Net:** Convolutions across the action horizon reuse filters at many positions. Feature-wise linear modulation (FiLM) supplies observation and diffusion-step conditioning. Layer widths, intermediate tensors, and accelerator tiling determine whether activations stay on chip.
- **Diffusion transformer (DiT):** Attention connects action tokens and visual conditioning. The action horizon and conditioning length determine attention work, while kernel fusion determines how much intermediate storage reaches DRAM. Neither backbone implies a fixed parameter count or per-step latency; the worked budget below declares one illustrative configuration.

A.5.2.1 Forward diffusion and variance scheduling

The forward diffusion process progressively corrupts a ground-truth demonstrated action chunk $\mathbf{A}^0 \sim p_{\text{data}}(\mathbf{A})$ with Gaussian noise across N discrete timesteps (or continuous diffusion time $t_{\text{diff}} \in [0,1]$):

$$q(\mathbf{A}^k \mid \mathbf{A}^{k-1}) = \mathcal{N}\left(\mathbf{A}^k; \sqrt{1-\beta_k}\mathbf{A}^{k-1}, \beta_k \mathbf{I}\right)$$

where $\{\beta_k \in (0,1)\}_{k=1}^{N}$ denotes a predefined variance schedule. Defining $\alpha_k = 1-\beta_k$, the one-step transition evaluates to:

$$\mathbf{A}^k = \sqrt{\alpha_k}\mathbf{A}^{k-1} + \sqrt{1-\alpha_k}\epsilon_{k-1}, \quad \epsilon_{k-1} \sim \mathcal{N}(\mathbf{0},\mathbf{I})$$

Unrolling this recursion for $k=2$:

$$\begin{aligned}\mathbf{A}^2 &= \sqrt{\alpha_2}\left(\sqrt{\alpha_1}\mathbf{A}^0 + \sqrt{1-\alpha_1}\epsilon_0\right) + \sqrt{1-\alpha_2}\epsilon_1 \\ &= \sqrt{\alpha_2\alpha_1}\mathbf{A}^0 + \sqrt{\alpha_2(1-\alpha_1)}\epsilon_0 + \sqrt{1-\alpha_2}\epsilon_1\end{aligned}$$

Because the linear combination of two independent zero-mean Gaussian random variables $\mathcal{N}(\mathbf{0},\sigma_a^2\mathbf{I})$ and $\mathcal{N}(\mathbf{0},\sigma_b^2\mathbf{I})$ is itself a Gaussian with variance $(\sigma_a^2+\sigma_b^2)\mathbf{I}$, the combined variance of the noise terms simplifies algebraically:

$$\alpha_2(1-\alpha_1) + (1-\alpha_2) = \alpha_2 - \alpha_1\alpha_2 + 1 - \alpha_2 = 1 - \alpha_1\alpha_2$$

Defining the cumulative variance product $\bar{\alpha}_k = \prod_{s=1}^{k}\alpha_s$, mathematical induction establishes the closed-form marginal distribution at any arbitrary diffusion step k directly from clean action chunk $\mathbf{A}^0$:

$$q(\mathbf{A}^k \mid \mathbf{A}^0) = \mathcal{N}\left(\mathbf{A}^k; \sqrt{\bar{\alpha}_k}\mathbf{A}^0, (1-\bar{\alpha}_k)\mathbf{I}\right)$$

allowing immediate sampling of noisy trajectories during training without simulating intermediate Markov chain transitions:

$$\mathbf{A}^k = \sqrt{\bar{\alpha}_k}\mathbf{A}^0 + \sqrt{1-\bar{\alpha}_k}\epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0},\mathbf{I})$$

A.5.2.2 Reverse denoising and score-matching objective

The generative reverse process synthesizes an action trajectory from pure Gaussian noise $\mathbf{A}^N \sim \mathcal{N}(\mathbf{0},\mathbf{I})$ by parameterizing the conditional reverse transition distribution $p_\theta(\mathbf{A}^{k-1} \mid \mathbf{A}^k, \mathbf{O})$:

$$p_\theta(\mathbf{A}^{k-1} \mid \mathbf{A}^k, \mathbf{O}) = \mathcal{N}\left(\mathbf{A}^{k-1}; \mu_\theta(\mathbf{A}^k, \mathbf{O}, k), \Sigma_k\right)$$

By Bayes' rule, the true reverse posterior conditioned on the clean trajectory $\mathbf{A}^0$ is tractable and Gaussian: $q(\mathbf{A}^{k-1} \mid \mathbf{A}^k, \mathbf{A}^0) = \mathcal{N}(\mathbf{A}^{k-1}; \tilde{\mu}_k(\mathbf{A}^k, \mathbf{A}^0), \tilde{\beta}_k\mathbf{I})$, where:

$$\tilde{\mu}_k(\mathbf{A}^k, \mathbf{A}^0) = \frac{\sqrt{\bar{\alpha}_{k-1}}\beta_k}{1-\bar{\alpha}_k}\mathbf{A}^0 + \frac{\sqrt{\alpha_k}(1-\bar{\alpha}_{k-1})}{1-\bar{\alpha}_k}\mathbf{A}^k$$

Because $\mathbf{A}^0$ is unobserved at test time, the reverse process substitutes an estimate of $\mathbf{A}^0$ derived from Tweedie's formula. For any Gaussian perturbation $\mathbf{x} \sim \mathcal{N}(\mathbf{x}_0, \sigma^2\mathbf{I})$ with prior density $p(\mathbf{x}_0)$, the conditional observation density is:

$$p(\mathbf{x} \mid \mathbf{x}_0) = \frac{1}{(2\pi\sigma^2)^{D/2}}\exp\left(-\frac{\|\mathbf{x}-\mathbf{x}_0\|^2}{2\sigma^2}\right)$$

Differentiating with respect to the noisy measurement $\mathbf{x}$:

$$\nabla_{\mathbf{x}} p(\mathbf{x} \mid \mathbf{x}_0) = -\frac{\mathbf{x}-\mathbf{x}_0}{\sigma^2}p(\mathbf{x} \mid \mathbf{x}_0)$$

The marginal density of noisy observations evaluates to $p(\mathbf{x}) = \int_{\mathbb{R}^D} p(\mathbf{x} \mid \mathbf{x}_0)p(\mathbf{x}_0)\,d\mathbf{x}_0$. Differentiating under the integral sign:

$$\nabla_{\mathbf{x}} p(\mathbf{x}) = \int_{\mathbb{R}^D} \nabla_{\mathbf{x}} p(\mathbf{x} \mid \mathbf{x}_0)p(\mathbf{x}_0)\,d\mathbf{x}_0 = -\frac{1}{\sigma^2}\int_{\mathbb{R}^D}(\mathbf{x}-\mathbf{x}_0)p(\mathbf{x} \mid \mathbf{x}_0)p(\mathbf{x}_0)\,d\mathbf{x}_0$$

Dividing both sides by the marginal density $p(\mathbf{x})$:

$$\frac{\nabla_{\mathbf{x}} p(\mathbf{x})}{p(\mathbf{x})} = \nabla_{\mathbf{x}} \ln p(\mathbf{x}) = -\frac{1}{\sigma^2}\left(\mathbf{x} - \int_{\mathbb{R}^D}\mathbf{x}_0\frac{p(\mathbf{x} \mid \mathbf{x}_0)p(\mathbf{x}_0)}{p(\mathbf{x})}\,d\mathbf{x}_0\right) = -\frac{\mathbf{x}-\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}]}{\sigma^2}$$

Multiplying by σ^2 and isolating $\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}]$ proves **Tweedie's formula**:

$$\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}] = \mathbf{x} + \sigma^2 \nabla_{\mathbf{x}} \ln p(\mathbf{x})$$

Tweedie's formula reveals that the Bayes-optimal reconstruction of clean state $\mathbf{x}_0$ equals the corrupted measurement $\mathbf{x}$ shifted along the marginal Stein score $\nabla_{\mathbf{x}} \ln p(\mathbf{x})$ scaled by noise variance σ^2. Rewriting the forward diffusion marginal $\mathbf{A}^k = \sqrt{\bar{\alpha}_k}\mathbf{A}^0 + \sqrt{1-\bar{\alpha}_k}\epsilon$ as $\frac{\mathbf{A}^k}{\sqrt{\bar{\alpha}_k}} = \mathbf{A}^0 + \frac{\sqrt{1-\bar{\alpha}_k}}{\sqrt{\bar{\alpha}_k}}\epsilon$, the effective noise variance is $\sigma^2 = \frac{1-\bar{\alpha}_k}{\bar{\alpha}_k}$. At the population optimum of the noise-prediction objective, Tweedie's formula equates the estimated clean trajectory $\hat{\mathbf{A}}^0$ and the Stein score $\nabla_{\mathbf{A}^k} \ln p(\mathbf{A}^k \mid \mathbf{O})$:

$$\nabla_{\mathbf{A}^k} \ln p(\mathbf{A}^k \mid \mathbf{O}) = -\frac{\epsilon_\theta(\mathbf{A}^k, \mathbf{O}, k)}{\sqrt{1-\bar{\alpha}_k}}$$

Substituting $\hat{\mathbf{A}}^0 = \frac{1}{\sqrt{\bar{\alpha}_k}}\left(\mathbf{A}^k - \sqrt{1-\bar{\alpha}_k}\epsilon_\theta(\mathbf{A}^k, \mathbf{O}, k)\right)$ into the posterior mean $\tilde{\mu}_k$ reparameterizes the reverse transition mean:

$$\mu_\theta(\mathbf{A}^k, \mathbf{O}, k) = \frac{1}{\sqrt{\alpha_k}}\left(\mathbf{A}^k - \frac{\beta_k}{\sqrt{1-\bar{\alpha}_k}}\epsilon_\theta(\mathbf{A}^k, \mathbf{O}, k)\right)$$

The noise-prediction network ϵ_θ is optimized via the simplified ℓ_2 denoising score-matching objective:

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{k\sim\mathcal{U}\{1,N\},\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I}),(\mathbf{O},\mathbf{A}^0)\sim\mathcal{D}_{\text{demo}}}\left[\left\|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_k}\mathbf{A}^0 + \sqrt{1-\bar{\alpha}_k}\epsilon, \mathbf{O}, k\right)\right\|^2\right]$$

By following the score function $\nabla_{\mathbf{A}^k} \ln p(\mathbf{A}^k \mid \mathbf{O})$, reverse diffusion steers stochastic sampling toward high-density demonstration modes rather than averaging incompatible modes into an obstacle collision. Each denoising step commits further to a single mode of the conditional action distribution.

A.5.2.3 Continuous flow matching and vector field regression

While standard diffusion models formulate generation as reverse stochastic differential equations (SDEs), **continuous flow matching** (Lipman et al. 2022) generates trajectories along deterministic probability flows defined by ordinary differential equations (ODEs):

$$\frac{d\mathbf{y}_t}{dt} = v_\theta(\mathbf{y}_t, t, \mathbf{O})$$

The continuous vector field v_θ drives a source noise distribution $\rho_0(\mathbf{y}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ to the empirical demonstration target distribution $\rho_1(\mathbf{y}) = p_{\text{data}}(\mathbf{y})$ across continuous time $t \in [0,1]$, governed by the **continuity equation**:

$$\frac{\partial \rho_t(\mathbf{y})}{\partial t} + \nabla \cdot (\rho_t(\mathbf{y}) u_t(\mathbf{y})) = 0$$

For a straight conditional interpolation path, the continuous trajectory interpolates linearly between source noise $\mathbf{y}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and demonstration chunk $\mathbf{y}_1 \sim \mathcal{D}_{\text{demo}}$:

$$\psi_t(\mathbf{y}_0) = (1-t)\mathbf{y}_0 + t\mathbf{y}_1, \quad t \in [0,1]$$

The conditional velocity along each paired-sample path is constant across time:

$$v_t(\mathbf{y} \mid \mathbf{y}_1) = \frac{d}{dt}\psi_t(\mathbf{y}_0) = \mathbf{y}_1 - \mathbf{y}_0$$

and the trajectory acceleration vanishes identically: $\frac{d^2}{dt^2}\psi_t(\mathbf{y}_0) = \mathbf{0}$. The **Conditional Flow Matching (CFM) theorem** (Lipman et al. 2022) proves a gradient identity. Regressing the neural vector field v_θ against this conditional vector field over paired samples $(\mathbf{y}_0, \mathbf{y}_1)$ gives the same gradients as regressing it against the intractable marginal velocity field $u_t(\mathbf{y})$:

$$\nabla_\theta \mathbb{E}_{t,\mathbf{y}_0,\mathbf{y}_1}\left[\|v_\theta(\psi_t(\mathbf{y}_0), t, \mathbf{O}) - (\mathbf{y}_1 - \mathbf{y}_0)\|^2\right] = \nabla_\theta \mathbb{E}_{t,\mathbf{y}\sim\rho_t}\left[\|v_\theta(\mathbf{y}, t, \mathbf{O}) - u_t(\mathbf{y})\|^2\right]$$

The network is therefore trained via direct mean squared regression:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],\mathbf{y}_0\sim\mathcal{N}(\mathbf{0},\mathbf{I}),\mathbf{y}_1\sim\mathcal{D}_{\text{demo}}}\left[\left\|v_\theta\left((1-t)\mathbf{y}_0 + t\mathbf{y}_1, t, \mathbf{O}\right) - (\mathbf{y}_1 - \mathbf{y}_0)\right\|^2\right]$$

The conditional paths above are straight for each sampled pair, but the learned marginal velocity averages over pairs that reach the same intermediate point. Its integrated trajectories need not remain straight, and their numerical error need not vanish. Independent noise/data pairing also does not establish globally optimal transport. Flow matching can support fewer evaluations in a measured task, but the step count must satisfy an action-quality criterion; it is not a universal consequence of the interpolation formula.

A.5.2.4 Denoising steps against the control deadline

In continuous generative policies, the number of reverse evaluation steps N_{eval} is the governing systems lever trading off policy expressiveness at the physical plant against computational latency on accelerator hardware. The reverse trajectory discretizes a continuous probability flow ODE or reverse SDE. Discretizing this path across N_{eval} finite steps introduces numerical integration errors:

- **Sampling accuracy:** Reducing the number of evaluations increases numerical approximation error for a fixed solver and learned field. How that error changes multimodal coverage, collision rate, or setpoint smoothness depends on the learned distribution and solver. The architecture alone supplies no minimum acceptable step count.
- **Computational latency and roofline limits:** Reverse diffusion is an inherently sequential Markov chain: evaluation $k-1$ requires the tensor output of evaluation k. Denoising steps cannot be parallelized across the time dimension on GPU or NPU accelerators. Total chunk generation latency scales linearly with N_{eval}:[5]

  $$t_{\text{infer}} = t_{\text{vis}} + N_{\text{eval}} \cdot t_{\text{pass}}$$

  where t_{vis} is the one-time visual feature extraction latency per chunk and t_{pass} is the forward execution time of noise-prediction network ϵ_θ. A weight-transfer lower bound is $t_{\text{stream}} = M_{\text{weights}}/\text{BW}_{\text{mem}}$ when every evaluation reloads those weights. The roofline intensity must count all operations and traffic, including convolutional or token reuse and activation spills. Batch size alone does not make every decoder memory bound. The worked budget below evaluates this transfer bound separately from its assumed execution time.

[5] **Diffusion Denoising Latency Budget**: Sequential reverse diffusion cannot parallelize across sampling steps because each input requires the previous step's output. Total inference time $t_{\text{infer}} = t_{\text{vis}} + N_{\text{eval}} t_{\text{pass}}$ scales linearly with step count N_{eval}, making step reduction or asynchronous chunking essential for real-time actuation.

A setpoint producer that must finish synchronously within Δt_{ctrl} misses its contract when $t_{\text{infer}} > \Delta t_{\text{ctrl}}$. The consequence depends on the controller's defined hold, interpolation, and fallback behavior; a deadline miss does not inherently imply a torque drop. Physical AI architectures resolve this latency-expressiveness trade-off through three coordinated mechanisms:

1. **Action chunking:** The policy predicts a multi-step trajectory chunk $\mathbf{A} \in \mathbb{R}^{H\times d_a}$ spanning prediction horizon H. Executing an execution horizon $H_{\text{exec}} \leq H$ amortizes the N_{eval}-step generation latency over $H_{\text{exec}}\Delta t_{\text{ctrl}}$ of physical robot motion.
2. **Fewer solver evaluations:** Flow matching, improved solvers, or distillation can reduce sequential evaluations when validation shows that the resulting action quality remains adequate. Reduced compute buys timing slack at the cost of a new approximation and validation obligation.
3. **Dual-rate asynchronous decoupling:** The application processor runs the generative denoising loop asynchronously at a relaxed rate (10–20 Hz), sending each action chunk across the proposal boundary over a black-channel link that carries a checksum and sequence number. A discrete safety microcontroller, which shares no memory with the application processor, receives the chunk into its own double buffer and executes synchronously at 1 kHz. It interpolates spline knots, enforces slew-rate limits, and gates commands through deterministic control barrier functions (section 1.7).

The numerical execution of this dual-rate pipeline spans both processors (algorithm A.2), and its geometry, denoising trajectory, and hardware split appear across the three panels of figure A.6. Panel (a) demonstrates multimodal action selection. It contrasts direct MSE regression, which averages divergent paths into an obstacle collision, against conditional diffusion, which samples distinct valid

trajectory modes. Panel (b) traces the reverse-diffusion denoising sequence from isotropic Gaussian noise $\mathbf{A}^{N_{\text{eval}}}$ to a coherent action chunk $\mathbf{A}^0$. Panel (c) shows the asynchronous dual-rate hardware split, where the application processor streams candidate chunks over a black-channel serial link to a discrete 1 kHz safety microcontroller running double-buffering, freshness checks, and spline interpolation. Section 6.2 uses the timing budget below to evaluate which sampler fits the control deadline. The pseudocode assumes the microcontroller's receive path has already validated the proposal's capture epoch, clock conversion, configuration, validity interval, and handoff continuity. An absent, expired, or rejected chunk selects the defined fallback; spline interpolation and slew clamping alone do not implement that permission decision. The transmitted action must also pass the actuator interface that converts its declared units to drive commands.

Algorithm A.2 Diffusion Policy Trajectory Chunk Denoising and Dual-Rate Actuation

Require: observation history buffer $\mathbf{O}_t = (\mathbf{o}_{t-H_{\text{hist}}}, \ldots, \mathbf{o}_t)$ of length $H_{\text{hist}} + 1$, noise prediction network ϵ_θ, noise schedule $\{\alpha_k, \bar{\alpha}_k, \sigma_k\}_{k=1}^{N_{\text{diff}}}$, chunk horizon H, safety microcontroller loop period Δt_{mcu}

Ensure: line-rate joint torque setpoint / action vector $\mathbf{a}_t$

Application Processor: Asynchronous Trajectory Chunk Generation (10–20 Hz)
$\mathbf{E}_t \leftarrow \textsc{VisualEncoder}(\mathbf{O}_t)$
$\mathbf{A}^{N_{\text{diff}}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{Hd_a})$ ▷ *sample a flattened chunk, then reshape to $H \times d_a$*
for $k = N_{\text{diff}}$ **down to** 1 **do**
  $\hat{\epsilon} \leftarrow \epsilon_\theta(\mathbf{A}^k, \mathbf{E}_t, k)$
  $\mu_k \leftarrow \frac{1}{\sqrt{\alpha_k}} \left(\mathbf{A}^k - \frac{1-\alpha_k}{\sqrt{1-\bar{\alpha}_k}} \hat{\epsilon} \right)$
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ **if** $k > 1$ **else** $\mathbf{z} \leftarrow \mathbf{0}$
  $\mathbf{A}^{k-1} \leftarrow \mu_k + \sigma_k \mathbf{z}$
end for
$\mathbf{A}_{\text{chunk}} \leftarrow \mathbf{A}^0$
$\textsc{SendOverLink}(\mathbf{A}_{\text{chunk}})$ ▷ *checksum and sequence number; no shared memory*

Safety Microcontroller: Synchronous Deterministic Actuation (1 kHz)
if $\textsc{HasNewChunk}(\text{RxBuffer})$ **then** ▷ *receive interrupt fills the inactive buffer*
  $\textsc{UpdateSplineKnots}(\text{RxBuffer})$
end if
$\mathbf{a}_{\text{raw}} \leftarrow \textsc{CubicSplineInterpolate}(\mathbf{A}_{\text{chunk}}, t_{\text{current}})$
$\Delta\mathbf{a} \leftarrow \textsc{clamp}(\mathbf{a}_{\text{raw}} - \mathbf{a}_{\text{prev}}, -\dot{\mathbf{a}}_{\max} \Delta t_{\text{mcu}}, +\dot{\mathbf{a}}_{\max} \Delta t_{\text{mcu}})$ ▷ *kinematic slew-rate governor*
$\mathbf{a}_{\text{clamped}} \leftarrow \mathbf{a}_{\text{prev}} + \Delta\mathbf{a}$
$\mathbf{a}_t \leftarrow \textsc{SafetyGovernorClamp}(\mathbf{a}_{\text{clamped}})$ ▷ *deterministic envelope enforcement*
$\textsc{TransmitPwmRegisters}(\mathbf{a}_t)$
$\mathbf{a}_{\text{prev}} \leftarrow \mathbf{a}_t$
return $\mathbf{a}_t$

Consider a 7-DoF arm with a two-finger gripper ($d_a = 8$) whose policy generates chunks of 16 setpoints at a chosen 25 ms setpoint period (40 Hz), a 400 ms horizon. The noise-prediction network is a 1D temporal convolutional U-Net with 48 million parameters, 96 MB at FP16. Its timings below are illustrative inputs, not measurements of any device. The budget has three parts.

1. **Perception.** A visual encoder processes two synchronized RGB streams (wrist-mounted and over-the-shoulder) into the conditioning embedding $\mathbf{E}_t$ in $t_{\text{vis}} = 14$ ms per chunk. Because the features are extracted once per chunk before denoising begins, t_{vis} is a fixed cost whatever the sampling schedule.
2. **Denoising.** Each reverse step evaluates ϵ_θ at 14.2 GFLOP. At a peak bandwidth of 204.8 GB/s, streaming the weights once takes at least 0.469 ms, and the arithmetic intensity is about 147.9 FLOP/byte. With kernel launch overhead and activation spills, the time per evaluation is $t_{\text{pass}} = 1.65$ ms, and the sampler then sets the cost. A standard DDIM sampler with 16 steps

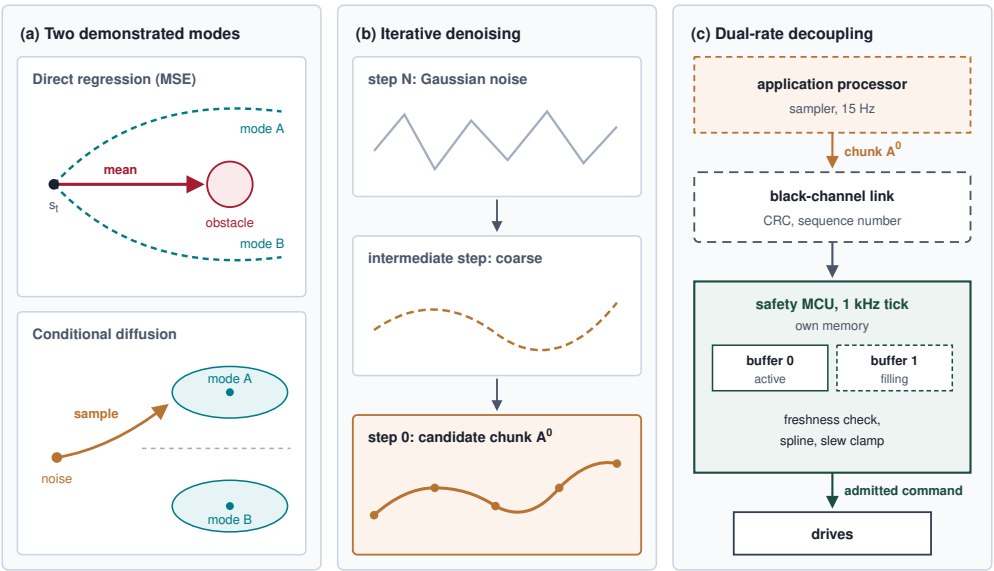


Figure A.6: Diffusion policy action chunk denoising and receding-horizon decoupling: (a) Multimodal resolution: direct regression with mean squared error averages divergent demonstrations into an invalid obstacle collision, whereas conditional diffusion can sample alternative demonstrated modes; generated paths still require clearance checks. (b) Iterative trajectory denoising: starting from Gaussian noise $\mathbf{A}^{N_{\text{eval}}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, reverse diffusion progressively resolves a coarse trajectory into a smooth action chunk $\mathbf{A}^0$. (c) Dual-rate decoupling: an asynchronous sampler on the application processor sends each chunk over a black-channel link to a discrete 1 kHz safety microcontroller, which holds it in its own double buffer and runs the freshness check, spline interpolation, and slew clamping. The two processors share no memory. (Adapted from Chi et al. (2025)).

denoises in 26.4 ms, for a total inference latency of 40.4 ms. A flow-matching decoder integrated with 2 Euler evaluations, assuming two steps meet the task's action-quality criterion, needs 3.3 ms, for a total of 17.3 ms.

3. **The deadline.** Run synchronously against the 25 ms period, the DDIM sampler overruns by 61.6 percent before any scheduling overhead. The two-evaluation flow decoder fits with 7.7 ms (30.8 percent) of compute-only slack, and it may run in lockstep only if its measured tail latency, bus transfer, and operating-system scheduling also fit. Run asynchronously at 15 Hz (a 66.7 ms period), the DDIM sampler refreshes the chunk buffer with 26.3 ms to spare. The permission path, at 1 kHz in the two-processor implementation, meanwhile interpolates the 400 ms chunk and checks its freshness, falling back when the chunk expires.

A.5.3 Compounding quadratic error in behavioral cloning

A 1 mm wrist-pose error on step 1 shifts the wrist camera viewpoint on step 2 into a slightly off-center state that the human demonstrator never visited. Whereas offline supervised evaluation samples from a fixed demonstration distribution, in closed-loop execution the current action $\mathbf{a}_t$ alters the subsequent physical state s_{t+1} and the next observation distribution, creating **compounding covariate shift**.

Let π^* denote the expert demonstrator policy and $\hat{\pi}$ denote the learned policy. Suppose the single-step error probability under the expert's state distribution is bounded by ϵ for all timesteps $t \in \{1, \ldots, S\}$:

$$\forall t \leq S: \quad \mathbb{E}_{s \sim d^t_{\pi^*}}[\mathbf{1}(\hat{\pi}(s) \neq \pi^*(s))] \leq \epsilon$$

In digital supervised learning (such as fixed-image classification), evaluation inputs are held fixed, so the expected count of prediction errors scales as $\mathcal{O}(S\epsilon)$. In closed-loop physical AI, an action error alters the physical state of the robot, transporting it into states $s \notin \text{supp}(d_{\pi^*})$ that the demonstrator rarely or never visited. Offline training provides no valid statistical guarantee on action error in these learner-induced states.

To derive the compounding error bound, we construct a formal trajectory coupling argument (Ross et al. 2011). Let $\mathcal{E}_t$ denote the event that the policy makes its first mistake at time t ($\hat{\pi}(s_t) \neq \pi^*(s_t)$ and $\hat{\pi}(s_{t'}) = \pi^*(s_{t'})$ for all $t' < t$). Couple learner and expert trajectories with the same initial state and transition randomness. Until their first disagreement they coincide, so $\mathcal{E}_t$ is a subset of the disagreement event on the unconditioned expert trajectory and $\Pr(\mathcal{E}_t) \leq \epsilon$. Conditioning on earlier agreement need not preserve the expert state marginal.

The probability that at least one error has occurred at or before step t is bounded by a union bound:[6]

$$\Pr(\text{error by step } t) = \Pr\left(\bigcup_{t'=1}^{t} \mathcal{E}_{t'}\right) \leq \sum_{t'=1}^{t} \Pr(\mathcal{E}_{t'}) \leq t\epsilon$$

[6] **Compounding Error in Imitation Learning**: Ross and colleagues analyzed how learner actions shift the state distribution. Expert corrective labels on learner-visited states can reduce this exposure.

Once an error occurs, the robot diverges into unmodeled states where downstream incremental cost is bounded by at most $C_{\max}$ per step. Summing over the entire finite execution horizon S:

$$\begin{aligned} J(\hat{\pi}) - J(\pi^*) &\leq \sum_{t=1}^{S} C_{\max} \Pr(\text{error by step } t) \\ &\leq C_{\max} \sum_{t=1}^{S} t\epsilon \\ &= C_{\max}\epsilon \frac{S(S+1)}{2} = \mathcal{O}(S^2\epsilon) \end{aligned}$$

The quadratic expression is a worst-case upper bound on excess cumulative cost, not a prediction of geometric trajectory error or a probability. Since $\Pr(\text{error by step } t) \leq 1$, a tighter bound replaces $t\epsilon$ by $\min(1, t\epsilon)$. For continuous actions, the disagreement indicator needs a specified tolerance or a surrogate loss tied to a cost bound; exact equality is generally too restrictive.

A.5.4 Interactive dataset aggregation (DAgger) linear reduction

The **DAgger** (Dataset Aggregation) algorithm (Ross et al. 2011) addresses this distribution shift by collecting training data under the learner's induced state distribution $d_{\hat{\pi}_i}$. At each training iteration i, the loop proceeds in four steps:

1. Execute policy $\hat{\pi}_i$ under the applicable safety envelope to collect trajectories $\mathcal{T}_i = \{s_1, s_2, \ldots, s_S\}$.
2. Query the expert to label every visited state $s_t \in \mathcal{T}_i$ with the optimal recovery action $a^* = \pi^*(s_t)$.
3. Aggregate the recovery dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a^*)\}$.
4. Retrain policy $\hat{\pi}_{i+1}$ on the aggregated dataset.

For the DAgger bound, make a stronger assumption than the earlier per-step $C_{\max}$: one wrong action followed by the expert changes expected remaining cost by at most u, independent of horizon. With expert labels on learner-visited states, a no-regret online learner, sufficient hypothesis capacity, and a valid finite-sample bound δ_N, a selected policy can satisfy a bound of the form

$$J(\hat{\pi}) - J(\pi^*) \leq Su(\epsilon_N + \gamma_N + \delta_N),$$

where ϵ_N is best-in-class loss on the induced-state distribution, γ_N is average online-learning regret, and δ_N covers finite-sample generalization and policy selection. It scales linearly in S only when $u = \mathcal{O}(1)$ and those errors remain controlled. The drift states need expert corrective labels; appending recovery trajectories alone does not supply them.

A.5.5 Closed-loop trajectory distributions and evaluation estimands

Scoring a candidate policy on a static disk of recorded human demonstrations tests the policy on states the human visited, not on the states that the candidate's own motor commands and the permission path will drive the robot into. Let a physical rollout be parameterized by initial state s_0, environmental parameters e (lighting, surface friction, obstacle geometry), plant parameters m (payload mass, link inertias), and permission-path parameters κ (lease durations, barrier thresholds). Together these form a trial condition $c = (s_0, e, m, \kappa) \sim \mathcal{D}$ drawn from the operating domain. Under

candidate policy π_θ and physical plant transition dynamics $p_m(s_{t+1} \mid s_t, \mathbf{u}_t)$, the joint probability density of an executed trajectory $\tau = (s_0, \mathbf{u}_0, s_1, \mathbf{u}_1, \dots, s_T)$ over horizon T is:

$$p_{\text{cl}}(\tau \mid \pi_\theta, e, m, \kappa) = \rho_0(s_0 \mid e, m, \kappa) \prod_{t=0}^{T-1} \left[\int \pi_\theta(\mathbf{a}_t \mid \mathbf{o}_{0:t})\, G_\kappa(\mathbf{u}_t \mid s_t, \mathbf{a}_t)\, d\mathbf{a}_t \right] p_m(s_{t+1} \mid s_t, \mathbf{u}_t).$$

Here $\mathbf{o}_t = g(s_t, e)$ is the observation, $\mathbf{a}_t$ is a proposal, and G_κ maps proposals to admitted commands or fallback actions. The state includes the controller and permission-path memory needed to make the transition Markov. A deterministic permission map is represented by a point-mass kernel; marginalizing over operating conditions gives the deployment trajectory distribution.

The discounted state visitation distribution (state marginal) $d^{\pi_\theta}(s)$ induced by policy π_θ is:

$$d^{\pi_\theta}(s) = (1-\gamma) \sum_{t=0}^{\infty} \gamma^t \Pr(s_t = s \mid \pi_\theta)$$

or in the finite-horizon unweighted setting: $\bar{d}^{\pi_\theta}(s) = \frac{1}{T} \sum_{t=0}^{T-1} \Pr(s_t = s \mid \pi_\theta)$. Given a binary task success indicator $\phi(\tau) \in \{0, 1\}$ (e.g., reaching a target receptacle without exceeding torque or clearance limits), the true closed-loop task success probability across operating domain $\mathcal{D}$ is the formal expectation over trial conditions, including policy and transition randomness inside Traj:

$$p(\mathcal{D}) = \mathbb{E}_{c \sim \mathcal{D}}[\phi(\text{Traj}(c))]$$

Because the distribution of visited states d^{π_θ} depends directly on policy proposals and their admission, offline evaluations over fixed logged datasets $\mathcal{D}_{\text{demo}}$ evaluate predictions under d^{π^*}. Static validation scores therefore give no statistical bound over d^{π_θ} when candidate commands deviate from demonstrator actions.

A.6 Value Functions and Continuous Reinforcement Learning

When a latch policy learns by trial and error rather than from demonstrations, no expert label tells it which torque vector to apply at each millisecond. It must discover actions that maximize cumulative reward from interaction, making the reward definition, exploration conditions, and simulator fidelity central assumptions. The following derivations use a fully observed Markov state; a deployed actor with partial observations may require an estimator or history encoder. The transition model includes any fixed permission mechanism, while J here denotes reward to maximize rather than the imitation cost minimized above. Section 6.2.1 builds reward, return, value, and the Bellman relation from the machine's latch task; the derivations below assume that primer and extend it to continuous actions and off-policy learning.

A.6.1 Continuous Markov decision processes and policy gradients

Continuous physical control is formalized as a Markov Decision Process (MDP) defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma, \rho_0)$, whose six components are:

- State space $\mathcal{S} \subseteq \mathbb{R}^{d_s}$ represents continuous physical states (joint angles, angular velocities, base pose, linear velocity);
- Action space $\mathcal{A} \subseteq \mathbb{R}^{d_a}$ represents continuous actuator commands (joint torques, motor phase currents, or impedance setpoints);
- Transition dynamics $p(s_{t+1} \mid s_t, \mathbf{a}_t)$ govern continuous plant kinematics and contact physics;
- Reward function $r(s_t, \mathbf{a}_t) \in \mathbb{R}$ quantifies task performance, energy efficiency, and actuator regularization;
- Discount factor $\gamma \in [0, 1)$ establishes the effective temporal planning horizon;
- Initial state distribution $\rho_0(s_0)$ characterizes starting configurations.

The goal is to learn a parameterized policy $\pi_\theta(\mathbf{a} \mid s)$ that maximizes the expected discounted cumulative return:

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \int p_\theta(\tau) R(\tau)\, d\tau$$

where trajectory $\tau = (s_0, \mathbf{a}_0, s_1, \mathbf{a}_1, \dots)$ has cumulative return $R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, \mathbf{a}_t)$ and probability density:

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{\infty} \pi_\theta(\mathbf{a}_t \mid s_t) p(s_{t+1} \mid s_t, \mathbf{a}_t)$$

Differentiating expected return $J(\pi_\theta)$ directly with respect to policy parameters θ yields:

$$\nabla_\theta J(\pi_\theta) = \int \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau$$

Applying the score-function log-derivative identity $\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \ln p_\theta(\tau)$:

$$\nabla_\theta J(\pi_\theta) = \int p_\theta(\tau) \nabla_\theta \ln p_\theta(\tau) R(\tau)\, d\tau = \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \ln p_\theta(\tau) R(\tau)]$$

Taking the natural logarithm of the trajectory probability density decomposes it into initial, policy, and plant components:

$$\ln p_\theta(\tau) = \ln \rho_0(s_0) + \sum_{t=0}^{\infty} \ln \pi_\theta(\mathbf{a}_t \mid s_t) + \sum_{t=0}^{\infty} \ln p(s_{t+1} \mid s_t, \mathbf{a}_t)$$

Differentiating with respect to θ:

$$\nabla_\theta \ln p_\theta(\tau) = \sum_{t=0}^{\infty} \nabla_\theta \ln \pi_\theta(\mathbf{a}_t \mid s_t)$$

Neither the initial state distribution $\rho_0(s_0)$ nor the plant transition dynamics $p(s_{t+1} \mid s_t, \mathbf{a}_t)$ depend on policy weights θ, causing their gradients to vanish identically. In physical systems engineering, this property matters because continuous contact dynamics, dry friction stick-slip transitions, and impact collisions are non-differentiable or discontinuous, which makes analytic trajectory derivatives $\nabla_{\mathbf{a}} s_{t+1}$ ill-conditioned or undefined. The policy gradient formulation bypasses differentiating through plant physics.

By causality (future actions cannot alter past rewards) and subtracting a state-dependent baseline $V(s_t)$ to reduce variance without introducing bias, the **Policy Gradient Theorem** (Sutton and Barto 2018) evaluates the gradient over the state visitation distribution:

$$\nabla_\theta J(\pi_\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi_\theta}, \mathbf{a} \sim \pi_\theta(\cdot|s)} [\nabla_\theta \ln \pi_\theta(\mathbf{a} \mid s) Q^{\pi_\theta}(s, \mathbf{a})]$$

The factor $1/(1-\gamma)$ is required because the state visitation distribution is normalized. An action-independent baseline may replace Q by $Q - V$ without changing the expectation. Here $d^{\pi_\theta}(s) = (1-\gamma) \sum_{t=0}^{\infty} \gamma^t \Pr(s_t = s \mid \pi_\theta)$ is the discounted state visitation distribution, and $Q^{\pi_\theta}(s, \mathbf{a}) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, \mathbf{a}_{t+k}) \mid s_t = s, \mathbf{a}_t = \mathbf{a} \right]$ is the continuous state-action value function.

A.6.2 Deep deterministic policy gradient (DDPG)

For high-dimensional continuous actuator spaces, stochastic sampling in policy gradients introduces substantial variance. The **Deterministic Policy Gradient (DPG)** theorem (introduced by Silver and colleagues in 2014) establishes a closed form for a deterministic policy $\mu_\theta : \mathcal{S} \to \mathcal{A}$. The policy gradient is the expected gradient of the action-value function along the policy's chosen direction:

$$\nabla_\theta J(\mu_\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\mu_\theta}} \left[\nabla_{\mathbf{a}} Q^{\mu_\theta}(s, \mathbf{a}) \Big|_{\mathbf{a} = \mu_\theta(s)} \nabla_\theta \mu_\theta(s) \right]$$

Here d^{μ_θ} is the normalized discounted visitation distribution and the theorem uses the true policy value Q^{μ_θ}; DDPG substitutes a learned critic. In discrete Q-learning (such as DQN), selecting the optimal greedy action requires solving an exhaustive search $\arg\max_a Q(s, a)$. In continuous control where action space $\mathcal{A} \subseteq \mathbb{R}^{d_a}$ has many dimensions (such as a 7-DoF arm or 12-DoF quadruped), solving an inner non-convex optimization at every control cycle is potentially too costly for the control

deadline. DPG circumvents this bottleneck by replacing the inner argmax with a differentiable continuous actor network $\mu_\theta(s)$. Applying the chain rule, the actor is updated directly in the direction of the critic's action gradient $\nabla_{\mathbf{a}} Q_\phi(s, \mathbf{a})$.

Deep Deterministic Policy Gradient (DDPG, introduced by Lillicrap and colleagues in 2015) parameterizes both actor μ_θ and critic Q_ϕ with deep neural networks. The continuous critic $Q_\phi(s, \mathbf{a})$ is trained via off-policy temporal difference learning by minimizing Bellman residual error on transitions $(s, \mathbf{a}, r, s')$ sampled from an experience replay buffer $\mathcal{D}$:

$$\mathcal{L}(\phi) = \mathbb{E}_{(s,\mathbf{a},r,s')\sim\mathcal{D}}\left[\left(Q_\phi(s,\mathbf{a}) - \left(r + \gamma Q_{\phi'}(s', \mu_{\theta'}(s'))\right)\right)^2\right]$$

Target networks (ϕ', θ') track active networks via slow Polyak averaging $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$ with tracking rate $\tau \ll 1$ (e.g., $\tau = 0.005$). Polyak tracking converts an unstable, moving regression target into a quasi-stationary target, slowing target drift without guaranteeing convergence under function approximation.

A.6.3 Twin delayed DDPG (TD3) and overestimation bias

Standard continuous actor-critic algorithms like DDPG suffer from **overestimation bias**. Because the critic uses deep function approximation, target values evaluate with nonzero approximation error $\epsilon(s', \mathbf{a}')$. In continuous Q-learning, the policy maximizes over estimated values.

The **Maximization Bias Lemma** formalizes this phenomenon: let $Q_{\phi_1}(s', \mathbf{a}')$ and $Q_{\phi_2}(s', \mathbf{a}')$ be two noisy estimators of true value $Q^*(s', \mathbf{a}')$ subject to zero-mean estimation error $\mathbb{E}[\epsilon_i] = 0$. Because the maximum operator is convex, Jensen's inequality proves:

$$\mathbb{E}\left[\max_{\mathbf{a}'} Q_{\phi_i}(s', \mathbf{a}')\right] \geq \max_{\mathbf{a}'} \mathbb{E}\left[Q_{\phi_i}(s', \mathbf{a}')\right] = \max_{\mathbf{a}'} Q^*(s', \mathbf{a}')$$

The bias is nonnegative, and can be zero; strict positivity requires errors that sometimes change which action wins. When present, maximization bias can propagate through recursive Bellman backups: overestimating value at step $t+1$ inflates target values at step t and can favor suboptimal, physically destructive actuator proposals.

Introduced by Fujimoto and colleagues in 2018, **Twin Delayed DDPG (TD3)** reduces overestimation through three architectural mechanisms:

1. **Clipped Double Q-learning:** TD3 maintains two separately parameterized critic networks Q_{ϕ_1} and Q_{ϕ_2}, computing the Bellman target using the minimum of both critics:

   $$y = r + \gamma \min_{i=1,2} Q_{\phi_i'}(s', \tilde{\mathbf{a}}')$$

   Because the expected minimum satisfies $\mathbb{E}[\min(X, Y)] \leq \min(\mathbb{E}[X], \mathbb{E}[Y])$, taking the minimum introduces a conservative *underestimation bias*. A lower reward estimate is not a clearance or stability bound. Underestimation can change which action wins without making that action physically safer.

2. **Target policy smoothing:** To prevent the policy from exploiting sharp, non-physical peaks in the learned critic, TD3 injects clipped Gaussian noise into the target action:

   $$\tilde{\mathbf{a}}' = \text{clip}\left(\mu_{\theta'}(s') + \text{clip}(\epsilon, -c, c), \mathbf{a}_{\text{min}}, \mathbf{a}_{\text{max}}\right), \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$$

   This regularizes value estimation over a local actuator ball. Doing so enforces the physical prior that similar motor commands should yield similar expected returns while smoothing the optimization landscape.

3. **Delayed policy updates:** Updating the policy network on every critic step causes the actor to chase an inaccurate, rapidly shifting value landscape. TD3 updates actor parameters θ and target networks once every d critic updates (typically delay ratio $d = 2$), giving the critics additional updates before actor weights shift, without guaranteeing stabilization.

A.6.4 Soft actor-critic (SAC) and maximum entropy control

While TD3 provides a deterministic policy, physical autonomy requires stochastic exploration to discover viable contact modes and avoid getting trapped in local contact minima (such as a quadruped foot wedged against an obstacle). **Soft Actor-Critic (SAC, developed by Haarnoja and colleagues in 2018)** formulates continuous control under the **Maximum Entropy Reinforcement Learning** framework. The objective augments task rewards with the differential entropy of the policy:

$$J(\pi_\theta) = \sum_{t=0}^{\infty} \mathbb{E}_{(s_t,\mathbf{a}_t)\sim\rho_{\pi_\theta}} \left[\gamma^t \left(r(s_t,\mathbf{a}_t) + \alpha\mathcal{H}(\pi_\theta(\cdot \mid s_t))\right)\right]$$

where $\mathcal{H}(\pi_\theta(\cdot \mid s_t)) = -\mathbb{E}_{\mathbf{a}\sim\pi_\theta}[\ln \pi_\theta(\mathbf{a} \mid s_t)]$ is the policy entropy, and temperature parameter $\alpha > 0$ controls the trade-off between reward maximization and stochastic exploration.

A.6.4.1 Soft Bellman backups and squashed Gaussian policy

The soft state-action value satisfies the soft Bellman equation:

$$Q(s,\mathbf{a}) = r(s,\mathbf{a}) + \gamma\mathbb{E}_{s'\sim p}[V(s')], \quad \text{where } V(s) = \mathbb{E}_{\mathbf{a}\sim\pi}[Q(s,\mathbf{a}) - \alpha \ln \pi(\mathbf{a} \mid s)]$$

To enforce physical actuator limits $\mathbf{a} \in [-a_{\max}, a_{\max}]^{d_a}$, SAC parameterizes the policy as an invertible squashed Gaussian distribution. The network outputs mean $\mu_\theta(s)$ and log-standard-deviation $\ln\sigma_\theta(s)$. An unbounded latent action is sampled via the reparameterization trick:

$$\mathbf{u} = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0},\mathbf{I})$$

The following density is for normalized actions. Applying the non-linear squashing function $\mathbf{a} = \tanh(\mathbf{u})$ bounds actions strictly within $(-1,1)^{d_a}$.

By the change-of-variables theorem for probability densities, the log-likelihood of the bounded action $\mathbf{a}$ requires correcting for the Jacobian determinant of the tanh mapping:

$$\pi_\theta(\mathbf{a} \mid s) = \mu(\mathbf{u} \mid s) \left|\det \frac{d\mathbf{a}}{d\mathbf{u}}\right|^{-1}$$

Because the squashing operator acts element-wise ($a_i = \tanh(u_i)$), the Jacobian matrix is diagonal: $\frac{da_i}{du_i} = 1 - \tanh^2(u_i)$. The determinant factors into a product:

$$\det \frac{d\mathbf{a}}{d\mathbf{u}} = \prod_{i=1}^{d_a} \left(1 - \tanh^2(u_i)\right)$$

Taking the natural logarithm yields the exact closed-form log-density correction:

$$\ln \pi_\theta(\mathbf{a} \mid s) = \ln \mu(\mathbf{u} \mid s) - \sum_{i=1}^{d_a} \ln\left(1 - \tanh^2(u_i)\right)$$

Physical limits require a further scale and offset; their Jacobian must also be included if densities are expressed in physical units. Here $\ln \mu(\mathbf{u} \mid s) = -\frac{1}{2}\sum_{i=1}^{d_a}\left(\left(\frac{u_i - \mu_i^\pi(s)}{\sigma_i^\pi(s)}\right)^2 + 2\ln\sigma_i^\pi(s) + \ln(2\pi)\right)$.

A.6.4.2 Dual gradient descent for automatic entropy tuning

Fixed temperature α fails across phases of robot learning: early exploration requires high entropy ($\alpha \gg 0$), while fine precision assembly requires low entropy ($\alpha \to 0$). SAC formulates temperature adaptation as a constrained optimization problem: maximizing expected return subject to a minimum average entropy constraint $\mathbb{E}[\mathcal{H}(\pi_t)] \geq \overline{\mathcal{H}}$, where target entropy is commonly set to $\overline{\mathcal{H}} = -d_a$.

Solving the dual Lagrangian formulation yields the exact update for temperature α:

$$\mathcal{L}(\alpha) = \mathbb{E}_{s_t\sim\mathcal{D},\mathbf{a}_t\sim\pi_t}\left[-\alpha \ln \pi_t(\mathbf{a}_t \mid s_t) - \alpha\overline{\mathcal{H}}\right]$$

Differentiating with respect to α gives the dual gradient update rule:

$$\alpha \leftarrow \alpha - \lambda_\alpha \nabla_\alpha \mathcal{L}(\alpha) = \alpha - \lambda_\alpha\left(-\ln \pi_t(\mathbf{a}_t \mid s_t) - \overline{\mathcal{H}}\right)$$

When expected policy entropy drops below target $\overline{\mathcal{H}}$, $\mathbb{E}[-\ln \pi_t] < \overline{\mathcal{H}}$, the expected gradient is negative and gradient descent raises α. Individual samples need not have that sign. Implementations preserve $\alpha > 0$, for example by optimizing its logarithm. When entropy exceeds target, α decreases to sharpen exploitation.

A.6.5 Proximal policy optimization (PPO) and GPU simulation

While off-policy algorithms (TD3, SAC) maximize sample efficiency from replay buffers, on-policy **Proximal Policy Optimization (PPO)** (Schulman et al. 2017) is the common choice when thousands of simulated robots run in parallel on one GPU. The Isaac Gym benchmarks and legged-locomotion training that learns to walk in minutes both follow this pattern (Makoviychuk et al. 2021; Rudin et al. 2022). PPO clips the incentive for large probability-ratio changes in its surrogate objective; clipping does not impose a hard trust region on the resulting policy:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}}) \hat{A}_t \right) \right]$$

where the probability ratio is $\rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t|s_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t|s_t)}$, clipping parameter $\epsilon_{\text{clip}} \approx 0.2$, and advantage estimates $\hat{A}_t$ are computed via Generalized Advantage Estimation (GAE):

$$\widehat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V, \quad \text{where } \delta_t^V = r(s_t, \mathbf{a}_t) + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

The parameter $\lambda \in [0, 1]$ trades bootstrapping against longer sampled returns. At $\lambda = 0$ the estimate is a one-step TD residual and depends directly on critic error. At $\lambda = 1$, a complete episode gives the discounted Monte Carlo return minus the baseline; truncated rollouts retain bootstrap error. Terminal transitions omit the continuation value in all Bellman targets above.

A.6.5.1 Massively parallel GPU simulation physics and memory hierarchy

On-policy training consumes fresh rollouts, while off-policy methods reuse a replay buffer. Parallel simulation can make fresh experience inexpensive by keeping physics state, observations, policy inference, and rollout storage on the same accelerator. This reduces host-device copies when the complete pipeline supports device-resident execution.

The storage comparison depends on transition size as well as count. If each stored transition takes b_{tr} bytes, a rollout with n_{env} environments and h steps needs approximately $n_{\text{env}} h b_{\text{tr}}$ bytes, while a replay buffer with n_{replay} transitions needs $n_{\text{replay}} b_{\text{tr}}$. Images, privileged state, and training scratch space can dominate either budget. No utilization percentage or training duration follows from choosing PPO; simulation fidelity, contact workload, network shape, and convergence requirements determine throughput and total cost.

A.6.6 Asymmetric actor-critic for sim-to-real transfer

Deploying policies trained in physics simulators to physical hardware encounters the **reality gap**: unmodeled actuator latency, mechanical compliance, gear backlash, motor thermal derating, and sensor noise cause policies to stumble or destabilize on physical testbeds. An **asymmetric actor-critic** gives the training critic more information than the deployed actor. A related but distinct technique, Rapid Motor Adaptation (RMA) (Kumar et al. 2021), trains a privileged actor and then predicts its environment latent from onboard history. The distinction determines what must run on the robot:

1. **Privileged centralized critic:** In training simulation, the critic network $V_\phi(s_{\text{priv}})$ or $Q_\phi(s_{\text{priv}}, \mathbf{a})$ conditions on the complete, privileged simulation state $s_{\text{priv}} \in \mathbb{R}^{d_{\text{priv}}}$. This privileged vector can contain quantities that are unavailable or measured only imperfectly onboard:
   - True surface contact friction coefficients μ;
   - Exact link masses, payload inertia matrices, and center-of-mass offsets $\Delta\mathbf{c}$;
   - Ground-truth contact force vectors $\mathbf{f}$ and normal impulses at each foot/finger;
   - Unoccluded 3D ground-truth obstacle positions and complete terrain heightmaps;
   - Motor winding temperatures and dynamic back-EMF constants.

A sufficiently complete privileged state can make the critic's prediction problem Markov and reduce estimation uncertainty. It does not make the actor's observation complete or remove model error.

2. **Two-phase adaptation and onboard actor:** To transfer privileged knowledge to physical hardware where s_{priv} cannot be measured, modern architectures use a two-phase distillation pipeline:
   - *Phase 1 (Privileged policy training):* The actor $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t, \mathbf{z}_t)$ receives deployable onboard observations $\mathbf{o}_t$ and an environment latent vector $\mathbf{z}_t = E(s_{\text{priv}})$, where encoder E compresses the privileged state into a low-dimensional manifold ($d_z \approx 8$–16).
   - *Phase 2 (Temporal adaptation module):* A lightweight 1D temporal convolutional or MLP adaptation module $g_\psi(\mathbf{o}_{t-H_{\text{hist}}:t})$ is trained via supervised regression to predict the latent vector $\hat{\mathbf{z}}_t$ purely from a sliding window history of recent proprioceptive observations (joint positions $\mathbf{q}$, velocities $\dot{\mathbf{q}}$, and commanded actions $\mathbf{a}$).
3. **Deployment cost:** A plain asymmetric actor-critic discards the privileged critic at deployment and runs only the actor. An RMA-style design also runs its history encoder or adaptation module. That module has a real compute and memory cost and can infer hidden parameters only when the observation history makes them identifiable. Critic access to simulator state does not give the actor access to that state.

Domain randomization varies physical parameters during training to expose the policy to a broader range of dynamics. It can improve transfer within the represented conditions, but neither randomization nor privileged training guarantees successful deployment on new terrain.

A.6.7 Bellman optimality and out-of-distribution overestimation

The optimal state-action value function $Q^*(s, \mathbf{a})$ satisfies the Bellman optimality equation:

$$Q^*(s, \mathbf{a}) = r(s, \mathbf{a}) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,\mathbf{a})} \left[\max_{\mathbf{a}' \in \mathcal{A}} Q^*(s', \mathbf{a}') \right]$$

When learning offline from static pre-collected datasets $\mathcal{D}_{\text{offline}}$ without interactive physical exploration, evaluating the maximization $\max_{\mathbf{a}' \in \mathcal{A}} Q^*(s', \mathbf{a}')$ queries the learned critic on counterfactual actions that were never attempted in the dataset. Outside the dataset's state-action support, function approximation errors can produce value overestimation on out-of-distribution (OOD) actions. An offline policy can favor these spuriously high values and propose destructive motor setpoints unless the permission path rejects them.

Modern offline continuous RL counters this instability through **Conservative Q-Learning (CQL)** (Kumar et al. 2020), which adds an explicit regularization term that penalizes Q-values on out-of-distribution actions while maximizing Q-values on logged demonstrator actions:

$$\min_{Q_\phi} \alpha_{\text{CQL}} \left(\mathbb{E}_{s \sim \mathcal{D}_{\text{offline}}, \mathbf{a} \sim \mu(\cdot|s)} [Q_\phi(s, \mathbf{a})] - \mathbb{E}_{(s,\mathbf{a}) \sim \mathcal{D}_{\text{offline}}} [Q_\phi(s, \mathbf{a})] \right) + \frac{1}{2} \mathcal{L}_{\text{Bellman}}(Q_\phi)$$

Under its stated coverage, estimation, and regularization assumptions, CQL provides lower-bound results for expected policy value. The displayed regularizer does not guarantee a pointwise bound below Q^* for every unseen state-action pair, and reward conservatism does not prevent actuator runaway. Policy proposals still require runtime permission checks.

A.6.8 SayCan affordance-grounded skill selection

The SayCan framework (Ahn et al. 2022) integrates semantic knowledge from high-capacity language models with physical affordance constraints from trained policies. Let c_{instr} represent high-level language instruction and a represent an available low-level skill. SayCan ranks candidate skills by multiplying a language relevance score and an estimated skill-success score, then normalizing if a selection distribution is needed:

$$p(a \mid s, c_{\text{instr}}) \propto p_{\text{LLM}}(a \mid c_{\text{instr}}) \cdot V_{\text{policy}}(s, a)$$

Here the two factors separate semantic relevance from physical feasibility:

- $p_{\text{LLM}}(a \mid c_{\text{instr}})$ represents the **semantic prior**, evaluating whether skill a contributes logically to completing instruction c_{instr}.
- $V_{\text{policy}}(s, a) \in [0, 1]$ represents the **physical affordance probability**, modeled by a learned value function that predicts the probability of successfully executing skill a from current state s.

If the learned value correctly assigns a near-zero score to an unavailable skill, the product deprioritizes it. The learned score estimates success; it bounds no path, contact force, or clearance, so a separate motion-permission check is needed.

A.6.9 Comparing policy architectures by compute and distribution

An MLP point regressor computes one action in a single forward pass but averages ambiguous demonstrations into a single path; a diffusion decoder preserves separate path modes at the cost of N_{eval} sequential network evaluations per chunk. Deploying continuous control policies on physical robots requires selecting an architecture whose computational profile, memory traffic, and latency fit the control loop deadlines of the physical plant (table A.1).

Table A.1: Architectural Trade-offs Across Continuous Control Policies: These are mechanisms, not hardware benchmark ranges. Observation encoding and permission checks add costs beyond the proposal decoder.

Policy component	Evaluations per proposal	Main resource question	Distribution and execution limit
MLP point regressor	One	Do weights and scratch buffers remain resident?	Squared-error output averages ambiguous targets
ConvNet with action head	One encoder and head evaluation	How much spatial reuse survives the memory mapping?	The head, not convolution, determines multimodality
ACT decoder	One decoder evaluation after observation encoding	What do visual tokens and action queries cost?	Fixed latent selects one output; chunk execution delays feedback
Diffusion decoder	N_{eval} sequential denoising evaluations	Are weights and activations transferred each step?	Sampling quality depends on model, solver, and N_{eval}
Flow-matching decoder	Solver-dependent sequential evaluations	How many field evaluations meet the action-quality target?	Straight conditional paths do not guarantee an exact low-step solution
RL actor	Usually one actor evaluation	What actor, history encoder, and preprocessing execute?	RL is a training method; the selected head determines the output family

For a concrete choice, count model parameters and activations, identify the memory level serving each operation, and compare the complete measured execution path with the deadline. The worked decoder budget in section A.5.2 makes those assumptions explicit. Sample efficiency also depends on task coverage and recovery data; it cannot be ranked from the decoder family alone.

A.7 Signal Temporal Logic and Continuous Robustness

A boolean pass/fail flag tells an engineer whether a mobile base stayed at least 0.30 m from an aisle rack during a 10 s run, but not whether it cleared the rack by 0.25 m or 0.2 mm. Verifying whether a physical autonomous system satisfies dynamic safety requirements requires moving beyond instantaneous boolean assertions to continuous temporal specifications that score how much margin remained across the interval.

A.7.1 Syntax and semantics of signal temporal logic

A **Signal Temporal Logic (STL)** specification evaluates a continuous-time physical signal $s(t)$ against temporal requirements defined over time intervals $[a, b]$:

$$\phi ::= \top \mid \pi \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \Box_{[a,b]}\phi \mid \Diamond_{[a,b]}\phi \mid \phi_1 \mathcal{U}_{[a,b]}\phi_2$$

where $\pi = (f(s(t)) \geq 0)$ is an atomic predicate (e.g., clearance $d(t) - d_{\text{safe}} \geq 0$), $\Box_{[a,b]}$ denotes the **Always** operator, $\Diamond_{[a,b]}$ denotes the **Eventually** operator, and $\mathcal{U}_{[a,b]}$ denotes the **Until** operator. The symbols match those of section 15.9.1.

A.7.2 Quantitative robustness degree

Beyond a boolean satisfaction verdict ($s \models \phi$), STL admits a signed **quantitative robustness metric** $\rho(\phi, s, t)$.[7] For a formula combining pressure, speed, and force, first define dimensionless atomic margins $\bar{f}_i(s) = f_i(s)/c_i$, where each positive scale c_i has the same SI unit as its predicate. Then the minimum and maximum below compare like quantities:

$$\begin{aligned}
\rho(\pi, s, t) &= \bar{f}(s(t)) \\
\rho(\neg\phi, s, t) &= -\rho(\phi, s, t) \\
\rho(\phi_1 \wedge \phi_2, s, t) &= \min\left(\rho(\phi_1, s, t), \rho(\phi_2, s, t)\right) \\
\rho(\Box_{[a,b]}\phi, s, t) &= \min_{t' \in [t+a, t+b]} \rho(\phi, s, t') \\
\rho(\Diamond_{[a,b]}\phi, s, t) &= \max_{t' \in [t+a, t+b]} \rho(\phi, s, t')
\end{aligned}$$

A strictly positive ρ establishes satisfaction and a negative value establishes violation under the stated semantics. Zero is a boundary case that needs the predicate convention. The normalized magnitude is dimensionless and depends on the declared scales. Preserve each raw physical margin and unit separately in the fault record; a single minimum of meters, newtons, and kilopascals has no physical meaning.

The until operator requires ϕ_1 to hold until a time when ϕ_2 holds. Its robustness is

$$\rho(\phi_1 \mathcal{U}_{[a,b]} \phi_2, s, t) = \sup_{t' \in [t+a, t+b]} \min\left(\rho(\phi_2, s, t'), \inf_{t'' \in [t, t')} \rho(\phi_1, s, t'')\right),$$

with the infimum over an empty interval taken as $+\infty$. Infimum and supremum replace minimum and maximum when extrema are not attained. A sampled trace establishes continuous-time satisfaction only with a justified interpolation or intersample variation bound and sufficient margin for measurement error.

[7] **Signal Temporal Logic Robustness**: Quantitative semantics map a trace and formula to a signed score. Its magnitude inherits the atomic predicate's units only when all combined predicates use commensurate units; otherwise a declared normalization gives a dimensionless score. Negative scores identify violations for falsification searches, subject to sampled-time and measurement limits.

A.8 Zero-Failure Testing and the Exposure Wall

In digital software machine learning, validation benchmarks evaluate millions of inference queries or generated tokens in parallel across cloud accelerators within minutes. In physical AI systems, one hour of physical robot operation takes one hour of real wall-clock time and inflicts mechanical wear on actuators, gearboxes, and structural linkages. Section 7.6 uses the zero-failure bound and the Rule of Three to price the running machine's evaluation campaigns. This section derives the discrete and continuous forms that chapter cites. It also shows why the normal approximation fails when zero failures have been observed.

A.8.1 Derivation of the discrete rule of three

Consider an operational test consisting of n independent, identically distributed Bernoulli trials (such as autonomous navigation missions or pick-and-place cycles), where each trial has an unknown catastrophic failure probability $p_{\text{fail}} \in [0, 1]$. Let the test observe zero catastrophic failures ($k = 0$). Under the hypothesis of failure probability p_{fail}, the probability of observing zero failures across n independent trials is:

$$\Pr(\text{zero failures} \mid p_{\text{fail}}) = (1 - p_{\text{fail}})^n$$

To exclude hypothesis p_{fail} at a one-sided confidence level $1 - \alpha_{\text{sig}}$ (e.g., 95 percent confidence with $\alpha_{\text{sig}} = 0.05$), the likelihood of observing zero failures under that hypothesis must not exceed α_{sig}:

$$(1 - p_{\text{fail}})^n \leq \alpha_{\text{sig}}$$

Taking the natural logarithm of both sides:

$$n \ln(1 - p_{\text{fail}}) \leq \ln \alpha_{\text{sig}}$$

For rare catastrophic events where failure probability is small ($p_{\text{fail}} \ll 1$), the first-order Taylor series expansion $\ln(1 - p_{\text{fail}}) = -p_{\text{fail}} - \frac{p_{\text{fail}}^2}{2} - \cdots \approx -p_{\text{fail}}$ yields:[8]

[8] **Statistical Rule of Three**: Zero events in n independent Bernoulli trials yield the exact one-sided 95 percent upper probability $1 - 0.05^{1/n} \approx 3/n$ for small probabilities. In continuous time, a stationary Poisson process requires $T \geq 3.0/\lambda$. Neither bound covers retrained models, modified firmware, or unobserved hazard classes without configuration-equivalence proofs.

$$-np_{\text{fail}} \leq \ln \alpha_{\text{sig}} \iff p_{\text{fail}} \geq \frac{-\ln \alpha_{\text{sig}}}{n} = \frac{\ln(1/\alpha_{\text{sig}})}{n}$$

Inverting this approximation gives a conservative sample count n required to bound the failure probability below p_{fail} at confidence $1-\alpha_{\text{sig}}$:

$$n \geq \frac{\ln(1/\alpha_{\text{sig}})}{p_{\text{fail}}}$$

For standard 95 percent confidence ($\alpha_{\text{sig}} = 0.05$):

$$n \geq \frac{-\ln(0.05)}{p_{\text{fail}}} = \frac{2.99573}{p_{\text{fail}}} \approx \frac{3}{p_{\text{fail}}}$$

This classical result is the **discrete Rule of Three**: observing zero failures across n trials bounds the per-trial failure probability to $p_{\text{fail}} \leq 3/n$ at 95 percent confidence. The exact inversion without Taylor approximation is $p_{\text{fail, upper}} = 1-\alpha_{\text{sig}}^{1/n}$; for small $p_{\text{fail, upper}}$, the approximation $-\ln(\alpha_{\text{sig}})/n$ slightly exceeds $p_{\text{fail, upper}}$, making the approximation conservative. For a target p_*, the exact required integer count is $\lceil \ln \alpha_{\text{sig}} / \ln(1-p_*) \rceil$.

A.8.2 Full derivation of the continuous Poisson exposure wall

Autonomous robots operate continuously in time rather than in discrete trials. Let catastrophic events follow a stationary homogeneous Poisson process with constant failure rate $\lambda > 0$ (failures per operating hour), which emerges naturally from the discrete Bernoulli model via the **Poisson Limit Theorem** (the law of rare events). Subdividing total fleet operating duration T into n infinitesimal intervals of duration $\Delta t = T/n$, the per-interval failure probability is $p = \lambda \Delta t = \lambda T/n$. In the limit as $\Delta t \to 0$ ($n \to \infty$), the binomial probability of observing exactly k failures converges to the Poisson distribution:

$$\lim_{n\to\infty} \binom{n}{k} \left(\frac{\lambda T}{n}\right)^k \left(1-\frac{\lambda T}{n}\right)^{n-k} = \frac{(\lambda T)^k e^{-\lambda T}}{k!}$$

Let $N(T)$ denote the cumulative count of catastrophic incidents observed across cumulative fleet operating duration $T = \sum_{j=1}^{N_{\text{fleet}}} T_j$ across N_{fleet} robots. The probability of recording exactly k incidents in duration T is:

$$\Pr(N(T) = k) = \frac{(\lambda T)^k e^{-\lambda T}}{k!}$$

Equivalently, the inter-arrival time t_1 to the first catastrophic incident follows an exponential distribution $t_1 \sim \text{Exponential}(\lambda)$ with survival probability $\Pr(t_1 > T) = e^{-\lambda T}$. When a physical fleet qualification campaign completes with zero observed incidents ($k = 0$):

$$\Pr(N(T) = 0) = \frac{(\lambda T)^0 e^{-\lambda T}}{0!} = e^{-\lambda T}$$

To test whether the operational failure rate is below an acceptable target rate λ_{target}, we formulate the statistical hypothesis test:

$$H_0 : \lambda \geq \lambda_{\text{target}} \quad \text{versus} \quad H_1 : \lambda < \lambda_{\text{target}}$$

Under null hypothesis H_0 with rate λ_{target}, the probability of observing the zero-failure test outcome is $\Pr(N(T) = 0 \mid \lambda_{\text{target}}) = e^{-\lambda_{\text{target}} T}$. To reject H_0 at significance level α_{sig} (achieving confidence level $1-\alpha_{\text{sig}}$):

$$p\text{-value} = \Pr(N(T) \leq 0 \mid \lambda_{\text{target}}) = e^{-\lambda_{\text{target}} T} \leq \alpha_{\text{sig}}$$

Taking the natural logarithm of both sides:

$$-\lambda_{\text{target}} T \leq \ln \alpha_{\text{sig}} \iff T \geq \frac{-\ln \alpha_{\text{sig}}}{\lambda_{\text{target}}} = \frac{\ln(1/\alpha_{\text{sig}})}{\lambda_{\text{target}}}$$

For standard 95 percent confidence ($\alpha_{\text{sig}} = 0.05$), $-\ln(0.05) = \ln(20) \approx 2.99573227$. This yields the **continuous Rule of Three**:

$$T \geq \frac{2.99573}{\lambda_{\text{target}}} \approx \frac{3.0}{\lambda_{\text{target}}}$$

Unlike the discrete Bernoulli derivation which required a Taylor approximation for $\ln(1-p_{\text{fail}})$, the continuous Poisson bound $T \geq \frac{\ln(1/\alpha_{\text{sig}})}{\lambda_{\text{target}}}$ is **mathematically exact** across all values of $\lambda_{\text{target}} > 0$.

A.8.2.1 Systems arithmetic of the exposure wall

For an illustrative target of 1×10^{-9} failures per operating hour, the one-sided 95 percent bound requires 2.996×10^{9} zero-failure operating hours. At the less stringent hypothetical target of 1×10^{-6} per hour, it still requires 2.996×10^{6} hours. These are scenario targets, not rates assigned by a software assurance level or a functional-safety classification.

Parallelism reduces calendar time but not total exposure. Even 1,000 independent, configuration-equivalent robots operating continuously need 342.0 365-day calendar years at the stricter target. Shared environmental hazards and common software faults can invalidate that independence assumption. Pooling fleet hours requires a justified common rate and representative exposure.

The **Poisson Exposure Wall** limits what zero-event testing alone can establish. A change to policy weights, firmware, sensors, or the operating domain can alter the event process, so earlier hours do not automatically qualify the changed configuration. Reuse requires evidence that the relevant event rate and conditions remain applicable.

Architectural constraints, fault analysis, and runtime monitors therefore carry evidence that a feasible test campaign cannot supply. They are not unconditional guarantees either: a barrier filter or interlock depends on its plant model, sensing validity, fault assumptions, and timing contract. Section 7.7 uses the exposure bound to separate these evidence obligations.

A.8.3 Wald normal confidence interval breakdown and Clopper-Pearson exact bounds

In standard machine learning benchmarks, empirical success or failure rates are commonly summarized using the asymptotic Wald normal approximation. For k observed failures across N independent Bernoulli trials, the sample failure proportion is $\hat{p} = k/N$. The two-sided $(1-\alpha_{\text{sig}})$ Wald confidence interval is:

$$\text{CI}_{\text{Wald}} = \hat{p} \pm z_{1-\alpha_{\text{sig}}/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{N}}$$

where $z_{1-\alpha_{\text{sig}}/2}$ is the standard normal critical value ($z_{1-\alpha_{\text{sig}}/2} \approx 1.96$ for 95 percent confidence).

A.8.3.1 The asymptotic collapse at zero failures ($k = 0$)

In physical autonomy qualification, candidate systems undergo zero-failure validation tests ($k = 0$). When $k = 0$, the sample proportion is $\hat{p} = 0$, causing the estimated variance $\hat{p}(1-\hat{p})/N$ to collapse to zero:

$$\text{CI}_{\text{Wald}}\Big|_{k=0} = 0 \pm z_{1-\alpha_{\text{sig}}/2}\sqrt{\frac{0(1-0)}{N}} = [0, 0]$$

The Wald interval asserts, at 95 percent confidence, that the failure probability is exactly zero after a finite test run. It ignores finite-sample uncertainty, and this particular interval excludes every nonzero failure rate. Repeated-sampling coverage of the Wald procedure is not identically zero, but can fall far below its nominal level near the boundary.

For a small nonzero true failure probability, $(1-p_{\text{fail}})^n$ can remain substantial, so a zero-failure campaign is not evidence of impossibility. The degenerate Wald interval confuses an estimated zero variance with zero uncertainty. Near the opposite boundary it has the same defect, and interior intervals can extend outside the valid probability range.

A.8.3.2 Clopper-Pearson exact binomial inversion

To maintain non-asymptotic statistical rigor, physical autonomy qualification can use **Clopper-Pearson exact binomial confidence intervals** (Clopper and Pearson 1934). The Clopper-Pearson interval $[p_L, p_U]$ inverts the exact binomial cumulative distribution function without normal approximations:

$$\sum_{x=k}^{N} \binom{N}{x} p_L^x (1-p_L)^{N-x} = \frac{\alpha_{\text{sig}}}{2}, \quad \sum_{x=0}^{k} \binom{N}{x} p_U^x (1-p_U)^{N-x} = \frac{\alpha_{\text{sig}}}{2}$$

By the identity connecting binomial sums to the regularized incomplete Beta function $I_{p_{\text{fail}}}(a,b)$:

$$\sum_{x=k}^{N} \binom{N}{x} p_{\text{fail}}^x (1-p_{\text{fail}})^{N-x} = I_{p_{\text{fail}}}(k, N-k+1) = \frac{1}{\text{B}(k, N-k+1)} \int_0^{p_{\text{fail}}} t^{k-1}(1-t)^{N-k}\, dt$$

for $0 < k < N$, these exact boundaries can be computed using quantiles of the Beta distribution:

$$p_L = \text{BetaQuantile}\left(\frac{\alpha_{\text{sig}}}{2}; k, N-k+1\right), \quad p_U = \text{BetaQuantile}\left(1-\frac{\alpha_{\text{sig}}}{2}; k+1, N-k\right)$$

At the endpoints, set $p_L = 0$ for $k = 0$ and $p_U = 1$ for $k = N$; the corresponding Beta distribution with a zero shape parameter is not defined.

A.8.3.3 Exact bounds at zero failures ($k = 0$)

When zero failures occur in physical testing ($k = 0$), the lower bound is $p_L = 0$. For a one-sided $(1-\alpha_{\text{sig}})$ upper confidence bound, the exact binomial condition simplifies to:

$$\Pr(X = 0) = \binom{n}{0} p_U^0 (1-p_U)^n = (1-p_U)^n = \alpha_{\text{sig}}$$

Solving directly for p_U:

$$1 - p_U = \alpha_{\text{sig}}^{1/n} \iff p_U = 1 - \alpha_{\text{sig}}^{1/n} = 1 - \exp\left(\frac{1}{n} \ln \alpha_{\text{sig}}\right)$$

Applying the Taylor expansion $e^u = 1 + u + \frac{u^2}{2} + \ldots$ with $u = \frac{\ln \alpha_{\text{sig}}}{n}$:

$$p_U = 1 - \left(1 + \frac{\ln \alpha_{\text{sig}}}{n} + \mathcal{O}(n^{-2})\right) = \frac{-\ln \alpha_{\text{sig}}}{n} + \mathcal{O}(n^{-2})$$

For $\alpha_{\text{sig}} = 0.05$, $-\ln(0.05) \approx 2.99573$, rigorously recovering the discrete Rule of Three: $p_U \approx 3/n$. For a two-sided 95 percent interval ($\alpha_{\text{sig}} = 0.05$, tail probability $\alpha_{\text{sig}}/2 = 0.025$):

$$p_U = 1 - (0.025)^{1/n} \approx \frac{-\ln(0.025)}{n} = \frac{3.68888}{n} \approx \frac{3.69}{n}$$

Unlike the degenerate Wald interval, the Clopper-Pearson interval guarantees exact non-asymptotic coverage of at least $1-\alpha_{\text{sig}}$ for every finite sample size $N \geq 1$.

A.9 Sequential Probability Ratio Test

Evaluating embodied physical policies on physical hardware is expensive and induces mechanical wear. A fixed-sample design predeclares sample count N for its planned acceptance decision and inference. A predeclared rejection rule may stop a failing campaign early, and physical safety can require termination at any time; neither decision requires continued exposure to an unsafe candidate. Early acceptance is different. Stopping when an accumulating result looks favorable and reporting the fixed-sample error guarantee can inflate false acceptance. Valid early acceptance requires a stopping rule with its own error calibration.

Abraham Wald's **Sequential Probability Ratio Test (SPRT)** (Wald 1945) handles optional stopping for two predeclared simple hypotheses by updating a likelihood ratio after each rollout and comparing it with stopping boundaries. Section 7.7.3 applies the test to the running machine's campaign in the same convention used here. The convention uses per-trial failure probability p, k failures in m trials, statistic Λ_m, null hypothesis the unacceptable p_0, and error allowances α_{sig} and β.

A.9.1 Log-likelihood ratio and Bernoulli updates

Across consecutive physical rollouts with independent trial outcomes $x_1, x_2, \ldots, x_m$, sequential testing compares two competing operational hypotheses:

- Null hypothesis H_0: The system failure probability is unacceptable ($p = p_0$).
- Alternative hypothesis H_1: The system failure probability meets the safety target ($p = p_1 < p_0$).

After observing m trials, the cumulative log-likelihood ratio Λ_m is:

$$\Lambda_m = \sum_{i=1}^{m} \ln \frac{\Pr(x_i \mid H_1)}{\Pr(x_i \mid H_0)}$$

For discrete Bernoulli trials where $x_i \in \{0,1\}$ (with $x_i = 1$ denoting task failure and $x_i = 0$ denoting task success), let $k = \sum_{i=1}^{m} x_i$ be the cumulative failure count across m runs. The log-likelihood ratio simplifies to the exact closed-form evaluation:

$$\Lambda_m = k \ln\left(\frac{p_1}{p_0}\right) + (m-k)\ln\left(\frac{1-p_1}{1-p_0}\right)$$

A.9.2 Wald stopping boundaries and decision rules

Wald's stopping rule rests on declared error rates. The Type I error rate α_{sig} is the maximum allowable probability of falsely accepting H_1 when H_0 is true. The Type II error rate β is the maximum allowable probability of falsely rejecting H_1 when H_1 is true. Testing then proceeds according to Wald's stopping rule:

$$\Lambda_{\text{lo}} < \Lambda_m < \Lambda_{\text{up}}$$

where the usual Wald boundary approximations are:

$$\Lambda_{\text{up}} = \ln\left(\frac{1-\beta}{\alpha_{\text{sig}}}\right), \quad \Lambda_{\text{lo}} = \ln\left(\frac{\beta}{1-\alpha_{\text{sig}}}\right)$$

At each evaluation step m:

1. If $\Lambda_m \geq \Lambda_{\text{up}}$: Halt and favor H_1 over the specified H_0 under the trial model; this is not a general safety certification.
2. If $\Lambda_m \leq \Lambda_{\text{lo}}$: Halt and favor H_0 over the specified H_1; do not release on this evidence.
3. If $\Lambda_{\text{lo}} < \Lambda_m < \Lambda_{\text{up}}$: The accumulated evidence remains inconclusive; execute rollout $m+1$.

Wald's approximate thresholds need calibration because the discrete log-likelihood increments can overshoot a boundary. Conservative alternatives are $\Lambda_{\text{up}} = \ln(1/\alpha_{\text{sig}})$ and $\Lambda_{\text{lo}} = \ln\beta$: the likelihood-ratio martingale under H_0 bounds false acceptance by α_{sig}, and the reciprocal under H_1 bounds false rejection by β. These guarantees apply to the declared simple hypotheses and observation model, not arbitrary deployment shifts.

A.9.3 Average sample number optimality

For Bernoulli failure probability p, the expected log-likelihood increment is

$$m(p) = p \ln\frac{p_1}{p_0} + (1-p)\ln\frac{1-p_1}{1-p_0}.$$

Let N_{stop} be the stopping sample count. Ignoring boundary overshoot and using the nominal endpoint error rates gives Wald's approximate average sample numbers

$$\mathbb{E}_{p_0}[N_{\text{stop}}] \approx \frac{\alpha_{\text{sig}}\Lambda_{\text{up}} + (1-\alpha_{\text{sig}})\Lambda_{\text{lo}}}{m(p_0)}, \qquad \mathbb{E}_{p_1}[N_{\text{stop}}] \approx \frac{(1-\beta)\Lambda_{\text{up}} + \beta\Lambda_{\text{lo}}}{m(p_1)}.$$

The denominators are the evidence gained per trial, negative under H_0 and positive under H_1. As the hypotheses approach each other, their magnitudes shrink and the required exposure grows.

For calibrated boundaries, use the actual operating characteristics and terminal overshoot rather than substituting nominal errors. A predeclared trial cap also changes the protocol and requires an explicit inconclusive outcome or recalibrated terminal rule.

For independent identically distributed outcomes under these two simple hypotheses, SPRT has an average-sample-number efficiency result relative to comparable tests.[9] Actual machine-hour savings depend on the two event probabilities, error allowances, overshoot at the stopping boundaries, and trial cost. The displayed thresholds are approximations; a finite-sample operating-characteristic calculation or calibrated simulation is needed for the chosen protocol.

[9] **Wald Sequential Probability Ratio Test**: The efficiency result assumes the specified simple hypotheses and independent, identically distributed outcomes. A sequential design can stop an unpromising test early without claiming a fixed percentage of saved robot hours.

A.10 Extreme Value Theory and Tail Risk

Average behavior does not determine the largest disturbance or latency spike a controller will encounter. The central limit theorem concerns normalized sums under conditions such as finite variance; it does not imply that individual observations or their far tails are Gaussian. Even a good Gaussian fit near the center can misstate rare exceedances.

For a genuinely Gaussian variable, a one-sided exceedance above k standard deviations has probability $1-\Phi(k)$, where Φ is the standard normal cumulative distribution function. Under independent samples at rate f, its mean waiting time is $1/[f(1-\Phi(k))]$. Correlation changes the clustering and waiting-time distribution, so a control-loop frequency alone does not establish an independent-trial rate.

A.10.1 Physical mechanisms that motivate tail analysis

Disturbances affecting structural survival and controller timing can arise from nonlinear or correlated mechanisms that a Gaussian sensor-noise model does not represent:

1. **Contact impact dynamics:** When an autonomous manipulator or mobile robot collides unexpectedly with a rigid obstacle, Hertzian contact mechanics govern the compressive force along the normal of each contact. This force is $f_{k,\perp} = k_c\delta^{3/2}$, where δ is deformation and the subscripts name the contact and its normal direction. When compliant protective covers bottom out against rigid metal chassis, contact stiffness shoots up discontinuously ($k_c \to k_\infty$), converting residual kinetic energy into sharp loads. The relation $F_{\text{unbraked}} = v\sqrt{mk_{\text{contact}}}$ applies to an ideal linear spring of stiffness k_{contact} absorbing kinetic energy, not to the Hertzian stiffness coefficient k_c.
2. **Mechanical friction transitions:** Stick-slip friction instability, drivetrain gear backlash, and structural resonance modes inject multiplicative, non-Gaussian torque transients into actuator gearboxes during rapid reversals.
3. **Real-time execution jitter:** On embedded systems, shared PCIe buses, memory cache thrashing, interrupt storms, and memory reclamation pauses produce tail latency spikes in control loops that break temporal determinism. Head-of-line blocking can amplify latency tails, but a Pareto service-time model requires evidence; bounded service times and arrival processes can produce different waiting-time tails.

These mechanisms motivate inspecting the measured tail; they do not prove a Pareto law. Bounded disturbances, exponential tails, lognormal tails, and power-law tails are all possible. **Extreme Value Theory (EVT)** supplies limiting models for maxima and threshold exceedances, with tail family and parameter uncertainty estimated from representative data. A positive shape parameter supports a power-law model over the validated range, not an assertion that every physical shock has infinite variance.

A.10.2 The Fisher-Tippett-Gnedenko theorem

While the Central Limit Theorem describes the limiting behavior of normalized sums, the **Fisher-Tippett-Gnedenko theorem** (Coles 2001) establishes the asymptotic behavior of normalized block maxima:

$$M_n = \max\{X_1, X_2, \ldots, X_n\}$$

Let $X_1, X_2, \ldots, X_n$ be a sequence of independent and identically distributed random variables with cumulative distribution function $F(x)$. If there exist normalizing sequences of real constants

$a_n > 0$ and $b_n \in \mathbb{R}$ such that the normalized block maximum converges to a non-degenerate limiting distribution $G(z)$:

$$\lim_{n\to\infty} \Pr\left(\frac{M_n - b_n}{a_n} \le z\right) = \lim_{n\to\infty} F^n(a_n z + b_n) = G(z)$$

then $G(z)$ belongs exclusively to the **Generalized Extreme Value (GEV)** family:[10]

$$G(z) = \exp\left(-\left[1+\xi\left(\frac{z-\mu}{\sigma}\right)\right]^{-1/\xi}\right)$$

defined on $\{z : 1+\xi(z-\mu)/\sigma > 0\}$, with location parameter $\mu \in \mathbb{R}$, scale parameter $\sigma > 0$, and shape parameter $\xi \in \mathbb{R}$. The shape parameter ξ partitions extreme behavior into three distinct tail regimes:

1. **Gumbel (Type I, $\xi = 0$):** The continuous limit as $\xi \to 0$:
$$G(z) = \exp\left(-\exp\left(-\frac{z-\mu}{\sigma}\right)\right)$$
including several unbounded tail forms, rather than only exponential tails. The maximum domain of attraction MDA(Gumbel) includes Gaussian, exponential, lognormal, and Gamma distributions.
2. **Fréchet (Type II, $\xi > 0$, with tail index $\alpha_{\text{tail}} = 1/\xi$):** The heavy-tailed regime where survival probabilities decay according to a polynomial power law: $\Pr(X > x) \sim x^{-\alpha_{\text{tail}}}$. The domain of attraction MDA(Fréchet) includes Pareto, Cauchy, and Student's t distributions. For a power-law density $p(x) \sim \alpha_{\text{tail}} x^{-\alpha_{\text{tail}}-1}$, the k-th raw moment evaluates to:
$$\mathbb{E}[X^k] = \int_{x_0}^{\infty} x^k p(x)\,dx \sim \alpha_{\text{tail}} \int_{x_0}^{\infty} x^{k-\alpha_{\text{tail}}-1}\,dx$$
This integral converges if and only if $k - \alpha_{\text{tail}} - 1 < -1 \iff k < \alpha_{\text{tail}} = 1/\xi$. For any moment order $k \ge 1/\xi$, the moment diverges to infinity ($\mathbb{E}[X^k] = \infty$). A direct mathematical consequence is that for $\xi \ge 0.5$ ($\alpha_{\text{tail}} \le 2$), the unbounded power-law model has **infinite variance** ($\text{Var}(X) = \infty$). The usual finite-variance central limit theorem then does not apply, and sample variance cannot converge to a finite population variance. For $\xi \ge 1.0$, the distribution has infinite mean. These are properties of an unbounded fitted model. Finite physical limits can truncate the tail, and a finite dataset cannot establish literal infinite moments.
3. **Weibull (Type III, $\xi < 0$):** Bounded distributions with a finite physical upper endpoint ($z \le \mu - \sigma/\xi$). The domain MDA(Weibull) includes Uniform and Beta distributions, applicable when hard physical limits (such as mechanically positive endstops) strictly bound the disturbance.

[10] **Tail Family Is an Empirical Claim**: A positive fitted GEV shape parameter ($\xi > 0$) provides formal evidence of heavy-tailed physical risk. It does not replace physical instrumentation: block size, temporal autocorrelation, threshold selection, and parameter uncertainty must be validated against real machine rollouts.

A.10.3 Peaks-over-threshold and generalized Pareto distribution

Block maxima analysis discards valuable extreme observations occurring within the same block (e.g., secondary collision shocks) and retains uninformative low maxima from quiescent blocks. The **Peaks-Over-Threshold (POT)** approach, formalized by the Pickands-Balkema-de Haan theorem (Coles 2001), analyzes all excess values $Y = X - u$ exceeding a sufficiently high physical threshold u:

$$F_u(y) = \Pr(X - u \le y \mid X > u) = \frac{F(u+y) - F(u)}{1 - F(u)}, \quad y \ge 0$$

The **Pickands-Balkema-de Haan theorem** proves that for a broad class of underlying physical distributions $F \in \text{MDA}(\text{GEV}_\xi)$, as threshold u approaches the right endpoint x_F:

$$\lim_{u\to x_F} \sup_{0\le y<x_F-u} |F_u(y) - H(y;\tilde{\sigma}(u),\xi)| = 0$$

where, for $\xi \neq 0$, $H(y)$ is the **Generalized Pareto Distribution (GPD)**:

$$H(y) = 1 - \left(1 + \frac{\xi y}{\tilde{\sigma}}\right)^{-1/\xi}$$

defined for $y \ge 0$ with $1 + \xi y/\tilde{\sigma} > 0$, with scale parameter $\tilde{\sigma} > 0$ and shape parameter ξ identical to the GEV shape parameter. For $\xi = 0$, take the continuous limit $H(y) = 1 - \exp(-y/\tilde{\sigma})$; for $\xi < 0$, the upper endpoint is $-\tilde{\sigma}/\xi$.

A.10.3.1 Threshold stability and mean residual life

A key property of the GPD is **threshold stability**. If the exceedances over threshold u_0 follow GPD$(\tilde{\sigma}_0, \xi)$, then for any higher threshold $u > u_0$ below the upper endpoint, the exceedances over u also follow a GPD with the *same* shape ξ and linearly shifted scale:

$$\tilde{\sigma}(u) = \tilde{\sigma}_0 + \xi(u - u_0)$$

The **Mean Residual Life (MRL)** evaluates the expected excess conditional on exceeding threshold u:

$$\mathbb{E}[X - u \mid X > u] = \int_0^{y_{\max}} [1 - H(y)]\, dy = \frac{\tilde{\sigma}(u)}{1 - \xi} \quad (\xi < 1),$$

where $y_{\max} = \infty$ for $\xi \geq 0$ and $y_{\max} = -\tilde{\sigma}(u)/\xi$ for $\xi < 0$. The exponential limit at $\xi = 0$ has mean $\tilde{\sigma}(u)$.

Substituting the threshold stability relation yields a strictly linear function of threshold u:

$$\mathbb{E}[X - u \mid X > u] = \frac{\tilde{\sigma}_0 + \xi(u - u_0)}{1 - \xi}$$

Systems engineers plot empirical mean excess against candidate thresholds u (the MRL plot); approximate linearity helps choose a candidate threshold, alongside parameter stability, dependence diagnostics, and uncertainty. It does not certify a unique mathematically valid threshold.

Given N_u exceedances $\{y_1, \ldots, y_{N_u}\}$ where $y_i = x_i - u > 0$, parameters $(\tilde{\sigma}, \xi)$ can be estimated via Maximum Likelihood Estimation (MLE), with the following expression for $\xi \neq 0$ and valid support:

$$\ln L(\tilde{\sigma}, \xi) = -N_u \ln \tilde{\sigma} - \left(1 + \frac{1}{\xi}\right) \sum_{i=1}^{N_u} \ln\left(1 + \frac{\xi y_i}{\tilde{\sigma}}\right)$$

Under the GPD model and regularity conditions, including $\xi > -1/2$, maximum-likelihood estimates have asymptotic normality and efficiency properties. Finite-sample bias, threshold choice, and clustered exceedances still affect the estimate.

A.10.4 Hardware margin and high-quantile sizing

Sizing physical hardware margins against an ultra-rare catastrophic event probability ϵ (e.g., $\epsilon = 10^{-7}$ per control cycle) requires extrapolating from N total operational cycles and N_u recorded threshold exceedances $(X > u)$. With empirical exceedance fraction $\hat{\zeta}_u = N_u/N > \epsilon$, the **high-quantile return level** z_ϵ satisfies $\Pr(X > z_\epsilon) = \epsilon$:

$$\begin{aligned}
\Pr(X > z_\epsilon) &= \Pr(X > u) \Pr(X > z_\epsilon \mid X > u) \\
&= \hat{\zeta}_u [1 - H(z_\epsilon - u)] \\
&= \frac{N_u}{N} \left(1 + \frac{\xi(z_\epsilon - u)}{\tilde{\sigma}}\right)^{-1/\xi} = \epsilon
\end{aligned}$$

Inverting this relation for z_ϵ:

$$\left(1 + \frac{\xi(z_\epsilon - u)}{\tilde{\sigma}}\right)^{-1/\xi} = \frac{N\epsilon}{N_u} \iff 1 + \frac{\xi(z_\epsilon - u)}{\tilde{\sigma}} = \left(\frac{N_u}{N\epsilon}\right)^{\xi}$$

Rearranging yields the closed-form return level formula:

$$z_\epsilon = u + \frac{\tilde{\sigma}}{\xi}\left[\left(\frac{N_u}{N\epsilon}\right)^{\xi} - 1\right] \quad (\xi \neq 0)$$

Taking the continuous limit as $\xi \to 0$ using L'Hôpital's rule ($\lim_{\xi \to 0} \frac{a^\xi - 1}{\xi} = \ln a$):

$$z_\epsilon = u + \tilde{\sigma} \ln\left(\frac{N_u}{N\epsilon}\right) \quad (\xi = 0)$$

A.10.4.1 Systems engineering sizing rules

For a fitted positive shape parameter, the return level grows as a power of $1/\epsilon$; the exponential limit grows logarithmically. Those rates do not yield a numerical ratio between shock loads without matching thresholds, scales, and exceedance frequencies. Report parameter and model uncertainty with the return level, and keep the target unit explicit: a probability per sampled cycle is not a rate per operating hour.

A force quantile can inform spring travel through $\delta = z_\epsilon / k_{\text{spring}}$ only under a valid linear spring model; energy capacity and bottoming-out dynamics still require separate checks. A latency quantile can inform availability analysis, but a watchdog timeout must remain inside the plant's maximum tolerable delay even when some valid computations exceed it. If that interval is infeasible, the scheduling or fallback design must change.

Queue depth requires an arrival-and-service model converting a delay into accumulated packets or frames; taking the ceiling of a time-valued quantile is dimensionally invalid. Thermal sizing likewise needs the energy and repetition rate of braking events, not a peak-force quantile alone. EVT estimates a tail under stated assumptions; it supplies neither a deterministic maximum nor a guarantee against all future physical failures.

A.11 Further Reading

For a deeper foundational understanding of machine learning theory, sequential statistics, and extreme value modeling, consult the following authoritative references:

- **Deep Learning** (Goodfellow et al. 2016) provides a comprehensive treatment of neural network optimization, regularized representation learning, and Lipschitz sensitivity bounds.
- **An Introduction to Statistical Modeling of Extreme Values** (Coles 2001) establishes the definitive mathematical foundations for GEV, GPD, and tail risk quantification.
- **Sequential Tests of Statistical Hypotheses** (Wald 1945) develops the first-principles mathematical framework and average sample number optimality proofs for the Sequential Probability Ratio Test.
- **A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning** (Ross et al. 2011) establishes the theoretical foundations of compounding error in imitation learning and derives the DAgger algorithm.

B

Control and Dynamics

This appendix collects the models needed to reason from a robot command to the motion it can produce. It develops state-space dynamics, feedback stability, trajectory continuity, stopping and contact bounds, and state-estimation uncertainty. Each result identifies the assumptions under which it can support a tracking or containment decision in the main text.

B.1 How to Use This Appendix

Use the table to enter at the physical question being checked. For a first encounter with control theory, start with state-space models and PID before consulting stability or constrained control. The longer Riccati, Lie-derivative, and differential-game derivations supply optional depth; the local model, assumptions, and resulting bound are the reference tools needed by the main text.

When you encounter this systems symptom	Consult	For this quantitative tool
An inner-loop tracking controller drifts or loses asymptotic convergence	section B.2	Continuous and discrete state-space models and Kalman controllability rank
Unmodeled gravity droop or friction causes steady-state tracking offset	section B.2.3	PID closed-loop error dissipation, Rayleigh damping, and online bias estimation
Nonlinear multi-joint couplings distort tracking across dynamic maneuvers	section B.2.4	Feedback linearization and computed torque decoupling to linear double integrators
Balancing tracking error against control effort before checking actuator limits	section B.2.5	Linear quadratic regulator (LQR), Riccati equation, and ideal full-state robustness margins
Certifying closed-loop convergence or designing stabilizing feedback for nonlinear plants	section B.3.1	Lyapunov stability theorems, domain of attraction, and Control Lyapunov Functions (CLFs)
Receding-horizon trajectory planning under hard state bounds and actuator limits	section B.3.3	Constrained Model Predictive Control (MPC) and recursive feasibility guarantees
A learned policy commands unsafe actions near physical obstacles or joint limits	section B.4	Control Barrier Function (CBF-QP) safety filters and kinetic stopping insets
Mode switching (policy handover, human intervention, or fallback) causes torque spikes	section B.5	C^2 quintic splines, C^1 cubic Hermite interpolation, and bumpless transfer bounds
Sizing safety envelopes, validity leases, or worst-case sets under adversarial disturbance	section B.6	Kinematic profiles, watchdog leases, and Hamilton-Jacobi differential games
An articulated manipulator stalls, exceeds joint velocity limits, or loses control authority	section B.6.1	Manipulator Jacobian singularities and Yoshikawa manipulability index
Spatial sensor offsets or IMU-camera lever arms corrupt state estimation under rotation	section B.6.4	Spatial coordinate transforms ($SE(3)$) and lever-arm acceleration propagation
Discrete sampling and transport latency erode control loop stability margins	section B.5.5	Discrete zero-order hold (ZOH) phase lag and latency margin penalties
Physics simulation diverges, impacts exceed limits, or wheels spin out during maneuvers	section B.7	Contact complementarity, elastodynamic impact, and wheel slip boundaries

When you encounter this systems symptom	Consult	For this quantitative tool
A grasp slips under arm acceleration or crushes the object	section B.7.5	Coulomb friction cones, minimum squeeze force, grasp map, and force closure
Sensor occlusions or unobservable states cause state estimation uncertainty to diverge	section B.8	Continuous Lyapunov covariance propagation and Lie derivative observability rank
Teleoperation or compliant contact chatters due to transport delay	section B.10	Passivity conditions, wave variables, and energy-bounded interaction
Underactuated legged or humanoid robots balance dynamic momentum across multi-contact points	section B.9	Centroidal momentum equations and hierarchical Whole-Body Control QP

B.2 Classical and Modern State-Space Control

A robot moving toward a target retains velocity when a new command arrives. A state-space model records that memory explicitly: its state contains enough information to predict subsequent motion from the current state and future inputs. A learned policy can propose an input, but the plant dynamics determine how that input changes position, velocity, and stored energy.

The physical body is the **plant**, the system a controller acts on (section 2.4). Its state $\mathbf{x}(t)$ can include joint positions, velocities, winding currents, and temperatures. These quantities are physically constrained, but they are not all conserved. Actuators change their rates of change; they do not overwrite the state instantaneously. For a mass driven by force, for example, the state contains both position and velocity because force changes acceleration before it changes either of them.

B.2.1 Linear and nonlinear state-space representations

State-space equations track how a plant's internal state evolves under actuator commands and what sensors observe from that state. In the notation below, a dot denotes a time derivative, bold symbols collect coordinates into vectors or matrices, and matrix multiplication describes how one coordinate affects another. An eigenvalue describes the growth, decay, or oscillation of a linear mode; a negative real part means decay in continuous time, while discrete-time decay requires magnitude below one. A positive-definite matrix assigns a positive quadratic measure to every nonzero vector. These interpretations connect the algebra below to motion and stored energy.

A continuous-time Linear Time-Invariant (LTI) dynamic system is represented in canonical state-space form as:

$$\dot{\mathbf{x}}(t) = \mathbf{A}_{\text{sys}}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t)$$

$$\mathbf{y}(t) = \mathbf{H}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t)$$

where $\mathbf{x}(t) \in \mathbb{R}^n$ denotes the internal state vector (e.g., joint angles and velocities, motor winding currents, or vehicle pose and velocity), $\mathbf{u}(t) \in \mathbb{R}^m$ is the control input vector (e.g., commanded motor currents, voltages, or torques), and $\mathbf{y}(t) \in \mathbb{R}^p$ represents the measured output vector produced by physical sensors. The system matrix $\mathbf{A}_{\text{sys}} \in \mathbb{R}^{n\times n}$ governs unforced internal system dynamics (the natural drift vector field). The input matrix $\mathbf{B} \in \mathbb{R}^{n\times m}$ models control authority over state rates. The output matrix $\mathbf{H} \in \mathbb{R}^{p\times n}$ maps internal states to sensor measurements. The feedthrough matrix $\mathbf{D} \in \mathbb{R}^{p\times m}$ represents direct feedthrough, which vanishes in strictly proper physical plants ($\mathbf{D} = \mathbf{0}$).

When physical dynamics include kinematic nonlinearities, centrifugal couplings, or configuration-dependent gravity (section B.6.1), the dynamics generalize to the control-affine nonlinear form:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)) + \mathbf{g}(\mathbf{x}(t))\mathbf{u}(t)$$

where $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^n$ is the unforced drift vector field and $\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^{n\times m}$ is the state-dependent control effectiveness matrix.

B.2.1.1 Discrete-time equivalent under zero-order hold

Digital microcontrollers and real-time operating systems execute periodic control loops at sampling frequency $f_s = 1/T_s$ (e.g., 1 kHz, $T_s = 1$ ms; see section 4.3). When the actuator driver holds control

inputs constant across each clock tick using a Zero-Order Hold ($\mathbf{u}(t) = \mathbf{u}_k$ for $t \in [kT_s, (k+1)T_s)$), the continuous LTI dynamics integrate into the exact discrete-time recurrence:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$$

where the discrete transition operators evaluate via the continuous matrix exponential:

$$\mathbf{A}_d = e^{\mathbf{A}_{\text{sys}} T_s} = \mathbf{I} + \mathbf{A}_{\text{sys}} T_s + \frac{1}{2!}\mathbf{A}_{\text{sys}}^2 T_s^2 + \dots$$

$$\mathbf{B}_d = \int_0^{T_s} e^{\mathbf{A}_{\text{sys}} t'} dt' \, \mathbf{B} = \left(\mathbf{I} T_s + \frac{1}{2!}\mathbf{A}_{\text{sys}} T_s^2 + \dots \right) \mathbf{B}$$

When $\mathbf{A}_{\text{sys}}$ is non-singular, the discrete input matrix simplifies to $\mathbf{B}_d = \mathbf{A}_{\text{sys}}^{-1}(e^{\mathbf{A}_{\text{sys}} T_s} - \mathbf{I})\mathbf{B}$. This recurrence is exact for the stated LTI model with a held input. A nonlinear plant generally requires numerical integration or a local linear model; its discretization error must enter the prediction bound.

B.2.1.2 Discretization pathologies: Why forward Euler destabilizes physical models

To an ML engineer, a forward Euler discretization $\mathbf{x}_{k+1} \approx (\mathbf{I} + \mathbf{A}_{\text{sys}} T_s)\mathbf{x}_k + \mathbf{B} T_s \mathbf{u}_k = \mathbf{x}_k + T_s(\mathbf{A}_{\text{sys}} \mathbf{x}_k + \mathbf{B}\mathbf{u}_k)$ appears structurally identical to a Residual Network (ResNet) skip connection $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathcal{F}(\mathbf{x}_k)$. However, in physical dynamical systems, Euler truncation introduces spurious numerical energy that can artificially destabilize the model or ruin gradient propagation in backpropagation through time.

Consider an undamped harmonic physical oscillator $\ddot{q} + \omega^2 q = 0$ with state $\mathbf{x} = [q, \dot{q}]^\top$ and continuous system matrix $\mathbf{A}_{\text{sys}} = \begin{bmatrix} 0 & 1 \\ -\omega^2 & 0 \end{bmatrix}$. The continuous eigenvalues are purely imaginary: $\lambda = \pm j\omega$.

Analyzing how common numerical discretization schemes map these eigenvalues to the complex z-plane illustrates their stability properties:

1. **Exact Zero-Order Hold (Matrix Exponential):** The discrete transition operator is $\mathbf{A}_d = e^{\mathbf{A}_{\text{sys}} T_s}$. Its eigenvalues lie on the unit circle:

   $$|\lambda(\mathbf{A}_d)| = |e^{\pm j\omega T_s}| = 1$$

   This preserves the physical total energy $E = \frac{1}{2}\dot{q}^2 + \frac{1}{2}\omega^2 q^2$ over infinite horizons without artificial growth or decay. Phase rotates at the true angular rate ωT_s per step.

2. **Forward Euler Discretization:** Approximating the transition as $\mathbf{A}_{\text{FE}} = \mathbf{I} + \mathbf{A}_{\text{sys}} T_s = \begin{bmatrix} 1 & T_s \\ -\omega^2 T_s & 1 \end{bmatrix}$ yields eigenvalues $\lambda_{\text{FE}} = 1 \pm j\omega T_s$. Their complex modulus is:

   $$|\lambda_{\text{FE}}| = \sqrt{1 + \omega^2 T_s^2} > 1 \quad \forall T_s > 0$$

   Because the spectral radius strictly exceeds unity, forward Euler simulation injects artificial numerical energy at every step, converting an intrinsically conservative physical oscillator into an exponentially diverging explosion. In differentiable simulation and policy learning, this causes exploding gradients during backpropagation through time.

3. **Backward (Implicit) Euler Discretization:** Approximating transitions via $\mathbf{x}_{k+1} = \mathbf{x}_k + T_s \mathbf{A}_{\text{sys}} \mathbf{x}_{k+1}$ yields $\mathbf{A}_{\text{BE}} = (\mathbf{I} - \mathbf{A}_{\text{sys}} T_s)^{-1}$. The discrete eigenvalues evaluate to $\lambda_{\text{BE}} = \frac{1}{1 \mp j\omega T_s}$, with modulus:

   $$|\lambda_{\text{BE}}| = \frac{1}{\sqrt{1 + \omega^2 T_s^2}} < 1 \quad \forall T_s > 0$$

   While unconditionally stable, backward Euler introduces artificial numerical dissipation (numerical damping) that bleeds energy from the physical system. In unrolled optimization, that artificial contraction causes gradient vanishing over long horizons.

4. **Bilinear (Tustin / Crank-Nicolson) Discretization:** Employing the trapezoidal integration operator $s \approx \frac{2}{T_s}\frac{z-1}{z+1}$ yields:

$$\mathbf{A}_{\text{Tustin}} = \left(\mathbf{I} - \frac{T_s}{2}\mathbf{A}_{\text{sys}}\right)^{-1}\left(\mathbf{I} + \frac{T_s}{2}\mathbf{A}_{\text{sys}}\right)$$

For purely imaginary continuous poles $s = \pm j\omega$, the discrete eigenvalues evaluate to $\lambda_{\text{Tustin}} = \frac{1 \pm j\omega T_s/2}{1 \mp j\omega T_s/2}$. Their modulus is identically unity:

$$|\lambda_{\text{Tustin}}| = \left|\frac{1 \pm j\omega T_s/2}{1 \mp j\omega T_s/2}\right| = \frac{\sqrt{1 + (\omega T_s/2)^2}}{\sqrt{1 + (\omega T_s/2)^2}} = 1 \quad \forall T_s > 0$$

The bilinear transform maps the continuous imaginary axis directly onto the discrete unit circle, preserving energy conservation without matrix exponentials. Higher frequencies undergo frequency warping, however, because $\arctan(\omega T_s/2)$ compresses the phase advance per step.

5. **Semi-Implicit (Symplectic) Euler Discretization:** Staggering state updates by updating velocity with current acceleration and then updating position with the updated velocity:

$$v_{k+1} = v_k - \omega^2 T_s q_k, \quad q_{k+1} = q_k + T_s v_{k+1} = (1 - \omega^2 T_s^2) q_k + T_s v_k$$

yields the discrete transition matrix:

$$\mathbf{A}_{\text{symp}} = \begin{bmatrix} 1 - \omega^2 T_s^2 & T_s \\ -\omega^2 T_s & 1 \end{bmatrix}$$

The determinant evaluates to $\det(\mathbf{A}_{\text{symp}}) = (1 - \omega^2 T_s^2)(1) - (T_s)(-\omega^2 T_s) = 1$ identically for all $T_s > 0$. For this oscillator, the method is symplectic and preserves phase-space area. It conserves a modified quadratic energy rather than the original energy exactly; determinant one alone does not establish symplecticity in higher dimensions. For stable step sizes ($\omega T_s < 2$), physical energy oscillates within bounded limits over infinite horizons rather than exploding or vanishing. The bounded energy error makes this method useful for conservative mechanical models, although contact, damping, and solver choices require separate analysis.

The integrator and step size must preserve the behavior needed by the application. Forward Euler can be adequate for sufficiently resolved dissipative dynamics; this undamped oscillator shows why a small step alone does not guarantee long-horizon energy fidelity. Sampling adds a separate phase effect (section B.5.5).

B.2.2 Controllability and the Kalman rank condition

In deep learning, model capacity measures whether a neural architecture possesses the expressivity to represent a desired target function class. In physical systems, **controllability** addresses a more basic structural question, namely whether the physical actuator allocation $\mathbf{B}$ provides sufficient authority to steer the plant's internal state $\mathbf{x}$ to an arbitrary target configuration across the continuous dynamic transition graph $\mathbf{A}_{\text{sys}}$.

An LTI system $(\mathbf{A}_{\text{sys}}, \mathbf{B})$ is **controllable** if, for any initial state $\mathbf{x}(0) = \mathbf{x}_0$ and any desired target state $\mathbf{x}_f$, there exists a finite time $T > 0$ and an unconstrained input trajectory $\mathbf{u}(t)$ that transfers the state from $\mathbf{x}_0$ to $\mathbf{x}_f$. This test does not impose actuator, state, or path limits. Under that unconstrained LTI model, the condition holds if and only if the **controllability matrix** $\mathbf{C}_{\text{ctrb}}$ has full row rank:

$$\mathbf{C}_{\text{ctrb}} = \begin{bmatrix} \mathbf{B} & \mathbf{A}_{\text{sys}}\mathbf{B} & \mathbf{A}_{\text{sys}}^2\mathbf{B} & \dots & \mathbf{A}_{\text{sys}}^{n-1}\mathbf{B} \end{bmatrix} \in \mathbb{R}^{n \times nm}, \quad \text{rank}(\mathbf{C}_{\text{ctrb}}) = n$$

Each column block $\mathbf{A}_{\text{sys}}^k\mathbf{B}$ represents the directional reach of the control input after k steps of dynamic diffusion through the unforced system matrix $\mathbf{A}_{\text{sys}}$. The column space $\text{im}(\mathbf{C}_{\text{ctrb}})$ defines the **controllable subspace**, mathematically equivalent to the n-th order Krylov subspace $\mathcal{K}_n(\mathbf{A}_{\text{sys}}, \text{im}(\mathbf{B})) = \text{span}\{\mathbf{B}, \mathbf{A}_{\text{sys}}\mathbf{B}, \dots, \mathbf{A}_{\text{sys}}^{n-1}\mathbf{B}\}$.

The matrix sequence terminates at power $n-1$ because the **Cayley-Hamilton theorem** states that every square matrix $\mathbf{A}_{\text{sys}} \in \mathbb{R}^{n\times n}$ satisfies its own characteristic polynomial $p(\mathbf{A}_{\text{sys}}) = \mathbf{0}$, where:

$$p(\lambda) = \det(\lambda\mathbf{I} - \mathbf{A}_{\text{sys}}) = \lambda^n + c_{n-1}\lambda^{n-1} + \cdots + c_1\lambda + c_0$$

Substituting $\mathbf{A}_{\text{sys}}$ into its polynomial yields:

$$\mathbf{A}_{\text{sys}}^n = -c_{n-1}\mathbf{A}_{\text{sys}}^{n-1} - c_{n-2}\mathbf{A}_{\text{sys}}^{n-2} - \cdots - c_1\mathbf{A}_{\text{sys}} - c_0\mathbf{I}$$

Multiplying repeatedly by $\mathbf{A}_{\text{sys}}$ demonstrates that every higher power $\mathbf{A}_{\text{sys}}^k$ for $k \geq n$ can be expressed as a linear combination of the lower powers $\{\mathbf{I}, \mathbf{A}_{\text{sys}}, \ldots, \mathbf{A}_{\text{sys}}^{n-1}\}$. Consequently, adding higher powers $\mathbf{A}_{\text{sys}}^k\mathbf{B}$ for $k \geq n$ cannot increase the dimension of the subspace spanned by the columns of $\mathbf{C}_{\text{ctrb}}$.

If $\text{rank}(\mathbf{C}_{\text{ctrb}}) < n$, the state space decomposes into a controllable subspace and an uncontrollable component in suitable coordinates. The controllability matrix $\mathbf{C}_{\text{ctrb}}$ evaluates whether control inputs can reach all n state dimensions through powers of $\mathbf{A}_{\text{sys}}$. It is exactly dual to the observability matrix $\mathbf{O}_{\text{obs}}$ (section B.8), which evaluates whether sensor outputs can distinguish all n state dimensions.

The Popov-Belevitch-Hautus (PBH) test provides an equivalent spectral condition: the pair $(\mathbf{A}_{\text{sys}}, \mathbf{B})$ is controllable if and only if:

$$\text{rank}\left(\begin{bmatrix}\lambda\mathbf{I} - \mathbf{A}_{\text{sys}} & \mathbf{B}\end{bmatrix}\right) = n \quad \forall\lambda \in \mathbb{C}$$

If $\text{rank}([\lambda\mathbf{I} - \mathbf{A}_{\text{sys}}, \mathbf{B}]) < n$ for some eigenvalue λ, the dynamic mode associated with λ cannot be excited or influenced by any control input. If that uncontrollable mode is unstable ($\text{Re}(\lambda) \geq 0$), the physical plant is un-stabilizable, an unyielding physical limitation that no learned policy, foundation model, or reinforcement learning algorithm can overcome. Underactuation alone does not imply uncontrollability. Dynamic coupling can make an indirectly actuated state controllable; floating-base systems additionally depend on the available contacts (section B.9).

B.2.3 Proportional-integral-derivative control as closed-loop error dissipation

When an arm link sags 4 mm below its target height under a 3 kg payload, position feedback pulls the joint toward the target, velocity feedback bleeds off overshoot, and accumulated-error feedback builds the extra torque needed to hold the load at zero steady-state offset. A joint-level **Proportional-Integral-Derivative (PID)** controller combines position error, accumulated error, and error rate to generate effort. For a simple mechanical plant, these terms act as a restoring spring, bias compensation, and damping. This interpretation explains their roles; PID is not generally an optimal controller, and positive gains alone do not guarantee stability.

In this subsection, let $\mathbf{x}^*(t)$ and $\mathbf{x}(t)$ denote desired and measured joint positions, rather than the full position-velocity state. Their difference defines the instantaneous tracking error:

$$\mathbf{e}(t) = \mathbf{x}^*(t) - \mathbf{x}(t)$$

The continuous PID control law synthesizes commanded control effort $\mathbf{u}(t)$ (e.g., motor drive current, electromagnetic torque, or hydraulic valve pressure) as:

$$\mathbf{u}(t) = \mathbf{K}_p\mathbf{e}(t) + \mathbf{K}_i\int_0^t \mathbf{e}(t')\,dt' + \mathbf{K}_d\dot{\mathbf{e}}(t)$$

where $\mathbf{K}_p, \mathbf{K}_i, \mathbf{K}_d \in \mathbb{R}^{n\times n}$ are symmetric, positive-definite gain matrices. For the energy argument below, assume a fixed setpoint, a constant scalar inertia $m > 0$, and no saturation. Each term has a distinct role:

1. **Proportional Term ($\mathbf{K}_p\mathbf{e}$): Conservative Restoring Force / Gradient Descent:** Consider an artificial quadratic potential loss landscape $\mathcal{L}_{\text{pos}}(\mathbf{e}) = \frac{1}{2}\mathbf{e}^\top\mathbf{K}_p\mathbf{e}$. Because $\mathbf{e} = \mathbf{x}^* - \mathbf{x}$, the restoring force is the negative gradient with respect to position, $-\nabla_{\mathbf{x}}\mathcal{L}_{\text{pos}} = \mathbf{K}_p\mathbf{e}$. Proportional feedback behaves as an elastic spring or continuous negative gradient step that pulls the physical coordinates toward the reference setpoint. However, on an undamped second-order plant ($m\ddot{\mathbf{x}} = \mathbf{u}$), proportional feedback alone generates an undamped harmonic oscillator $m\ddot{\mathbf{e}} + \mathbf{K}_p\mathbf{e} = \mathbf{0}$. The system conserves total mechanical energy $E = \frac{1}{2}m\|\dot{\mathbf{e}}\|^2 + \frac{1}{2}\mathbf{e}^\top\mathbf{K}_p\mathbf{e}$, orbiting closed level curves indefinitely with zero asymptotic convergence.

2. **Derivative Term ($\mathbf{K}_d\dot{\mathbf{e}}$): Rayleigh Dissipation / Momentum Damping:** To force convergence to the origin, kinetic energy must be extracted from the closed loop. The derivative term introduces an artificial viscous damper or Rayleigh dissipation function $\mathcal{D}(\dot{\mathbf{e}}) = \frac{1}{2}\dot{\mathbf{e}}^\top \mathbf{K}_d \dot{\mathbf{e}}$. Differentiating the closed-loop mechanical energy $E(\mathbf{e}, \dot{\mathbf{e}})$ along plant trajectories under PD feedback yields:

$$\dot{E} = m\dot{\mathbf{e}}^\top \ddot{\mathbf{e}} + \dot{\mathbf{e}}^\top \mathbf{K}_p \mathbf{e} = \dot{\mathbf{e}}^\top \left(-\mathbf{K}_p \mathbf{e} - \mathbf{K}_d \dot{\mathbf{e}}\right) + \dot{\mathbf{e}}^\top \mathbf{K}_p \mathbf{e} = -\dot{\mathbf{e}}^\top \mathbf{K}_d \dot{\mathbf{e}} \leq 0$$

   Because $\mathbf{K}_d \succ \mathbf{0}$, the derivative term converts tracking kinetic energy into continuous dissipation, damping oscillations and, for this plant, steering closed-loop poles into the open left-half complex plane. In machine learning optimization, this is analogous to the friction term in the continuous heavy-ball model used to stabilize gradient oscillations across steep ravines.

3. **Integral Term ($\mathbf{K}_i \int_0^t \mathbf{e}(t')\, dt'$): Online Disturbance / Unmodeled Bias Estimation:** In physical deployments, constant external loads $\mathbf{d}_{\text{ext}}$, such as gravitational sagging on robot arm links, cable friction, or payload offsets, induce a steady-state standoff. At rest ($\dot{\mathbf{e}} = \mathbf{0}, \ddot{\mathbf{e}} = \mathbf{0}$), a PD controller yields $\mathbf{K}_p \mathbf{e}_{\text{ss}} + \mathbf{d}_{\text{ext}} = \mathbf{0}$, leaving a persistent tracking error $\mathbf{e}_{\text{ss}} = -\mathbf{K}_p^{-1}\mathbf{d}_{\text{ext}}$. The integral term introduces an internal dynamic state variable:

$$\dot{\mathbf{z}}(t) = \mathbf{e}(t), \quad \mathbf{u}_{\text{int}}(t) = \mathbf{K}_i \mathbf{z}(t)$$

   Accumulating tracking error over time forces $\mathbf{z}(t)$ to integrate until $\mathbf{K}_i \mathbf{z}_{\text{ss}} = -\mathbf{d}_{\text{ext}}$, counteracting the disturbance and driving steady-state position error to zero ($\mathbf{e}_{\text{ss}} \to \mathbf{0}$). In optimization terms, the integral term acts as an online running bias estimator that dynamically learns and cancels constant model mismatch.

For a scalar single-degree-of-freedom mechanical plant with mass m, the closed-loop error equation under PD control evaluates to:

$$\ddot{e}(t) + 2\zeta\omega_n \dot{e}(t) + \omega_n^2 e(t) = 0$$

where the undamped natural frequency is $\omega_n = \sqrt{K_p/m}$ and the dimensionless damping ratio is $\zeta = \frac{K_d}{2\sqrt{mK_p}}$. The characteristic roots $s_{1,2} = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}$ partition the closed-loop tracking response into three physical regimes:

- **Underdamped ($0 < \zeta < 1$):** Complex conjugate poles; fast initial rise time but decaying ringing oscillations and overshoot that can damage mechanical gearboxes.
- **Critically Damped ($\zeta = 1$):** Repeated real negative root $s = -\omega_n$; for fixed ω_n, gives the fastest nonoscillatory step response from rest in this second-order model.
- **Overdamped ($\zeta > 1$):** Distinct negative real roots; gives a slower nonoscillatory step response from rest. Other initial velocities can still produce a setpoint crossing.

B.2.3.1 Practical firmware implementation: Anti-windup and derivative filtering

While textbook PID assumes infinite actuator authority and continuous differentiation, deploying PID loops on physical robot joints exposes two cyber-physical pathologies:

1. **Derivative Kick and Noise Amplification:** In textbook PID, the derivative term evaluates error rate $\mathbf{u}_d = \mathbf{K}_d\dot{\mathbf{e}} = \mathbf{K}_d(\dot{\mathbf{q}}^* - \dot{\mathbf{q}})$. When a learned policy or planner issues a step setpoint or hands over between action chunks (section 14.4), the sudden change in reference setpoint $\mathbf{q}^*$ creates an impulsive derivative spike $\dot{\mathbf{q}}^* \approx \infty$ (derivative kick). The spike commands explosive actuator torques and can trigger current limiting or an inverter fault. Furthermore, discrete differentiation of quantized encoder signals amplifies high-frequency measurement noise as $\mathcal{O}(\omega)$. Production firmware resolves this by differentiating *solely on the measured plant state* ($\mathbf{u}_d = -\mathbf{K}_d\dot{\mathbf{q}}$) and routing the velocity through a first-order low-pass filter:

$$\mathbf{u}_d(s) = -\frac{\mathbf{K}_d s}{T_f s + 1}\mathbf{q}(s)$$

   where $T_f > 0$ sets the derivative filter roll-off frequency, preserving damping in the control band while limiting the high-frequency gain that an ideal differentiator would amplify without bound.

2. **Actuator Saturation and Integral Windup:** Physical actuators are strictly bounded by torque and thermal current limits ($\tau_{\text{min}} \leq \tau \leq \tau_{\text{max}}$). When a large disturbance or aggressive maneuver pushes commanded torque past physical limits, the physical plant cannot accelerate fast enough to eliminate tracking error. Meanwhile, the integral term continues accumulating error unhindered, winding up the internal dynamic state $\mathbf{z}(t)$ to excessive values. Once the plant finally reaches the setpoint, the stored integrator charge forces the actuator to remain saturated in the forward direction, causing violent overshoot, severe limit-cycle hunting, and structural collisions. Real-time motor firmware implements **anti-windup circuitry**: clamping the integrator state whenever actuator limits are reached, or employing back-calculation:

$$\dot{\mathbf{z}}(t) = \mathbf{e}(t) + \frac{1}{T_t}\mathbf{K}_i^{-1}\left(\tau_{\text{act}}(t) - \tau_{\text{cmd}}(t)\right)$$

where $\mathbf{u} = \tau$, the invertible integral gain converts the torque mismatch back to error-integral units, $T_t > 0$ is a tracking time constant and $\tau_{\text{act}} = \text{clip}(\tau_{\text{cmd}}, \tau_{\text{min}}, \tau_{\text{max}})$. When the actuator is unsaturated ($\tau_{\text{act}} = \tau_{\text{cmd}}$), standard integration proceeds. When the actuator is saturated, the mismatch feeds back to discharge the integrator dynamically, reducing the excess stored integral command upon desaturation. With integral action, stability must be checked again: for the scalar mass model, positive gains must also satisfy $K_d K_p > m K_i$.

B.2.4 Feedback linearization: Transforming nonlinear dynamics into decoupled double integrators

Swinging a manipulator's elbow outward changes the shoulder's effective inertia and injects centrifugal torque into the shoulder joint, so fixed linear gains tuned with the arm tucked become sluggish or oscillatory when the arm extends. Multi-body robotic plants are governed by coupled, configuration-dependent equations of motion (section 2.4):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \tau$$

where $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n \times n}$ is the configuration-dependent mass matrix, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$ collects velocity-dependent generalized forces, $\mathbf{g}(\mathbf{q})$ is gravity, and $\tau \in \mathbb{R}^n$ is the joint torque vector.

To cancel those configuration-dependent couplings, **feedback linearization** (implemented here as **computed torque control**) uses a model of inertia, velocity coupling, and gravity to turn a requested joint acceleration into torque. The cancellation below assumes a fully actuated rigid-link model, valid joint coordinates, exact parameters, and sufficient actuator authority. It does not apply unchanged to a floating base or to saturated drives.

Because the physical inertia matrix $\mathbf{M}(\mathbf{q})$ is symmetric and strictly positive definite everywhere on the configuration space ($\mathbf{M}(\mathbf{q}) = \mathbf{M}(\mathbf{q})^\top \succ \mathbf{0}$), it is globally invertible. We define the nonlinear state-feedback control law:

$$\tau = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}^* + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$$

where $\ddot{\mathbf{q}}^* \in \mathbb{R}^n$ is a synthetic, virtual acceleration command. Substituting this control law directly into the physical equations of motion yields:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}^* + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$$

Subtracting the identical Coriolis and gravity vectors from both sides isolates:

$$\mathbf{M}(\mathbf{q})\left(\ddot{\mathbf{q}} - \ddot{\mathbf{q}}^*\right) = \mathbf{0}$$

Because $\mathbf{M}(\mathbf{q})$ is non-singular, multiplying by $\mathbf{M}(\mathbf{q})^{-1}$ collapses the coupled nonlinear plant into n decoupled, linear double-integrator channels:

$$\ddot{\mathbf{q}} = \ddot{\mathbf{q}}^* \iff \ddot{q}_i = \ddot{q}_i^* \quad \forall i \in \{1, \ldots, n\}$$

Over these decoupled channels, linear tracking control laws synthesize virtual acceleration commands with global exponential stability:

$$\ddot{\mathbf{q}}^* = \ddot{\mathbf{q}}^{\text{ref}} + \mathbf{K}_p(\mathbf{q}^{\text{ref}} - \mathbf{q}) + \mathbf{K}_d(\dot{\mathbf{q}}^{\text{ref}} - \dot{\mathbf{q}})$$

Substituting $\ddot{\mathbf{q}}^*$ back into the double-integrator dynamics reveals that the tracking error $\mathbf{e}(t) = \mathbf{q}^{\text{ref}}(t) - \mathbf{q}(t)$ satisfies the linear error ODE:

$$\ddot{\mathbf{e}} + \mathbf{K}_d \dot{\mathbf{e}} + \mathbf{K}_p \mathbf{e} = \mathbf{0}$$

Choosing positive-diagonal gain matrices $\mathbf{K}_p$ and $\mathbf{K}_d$ guarantees that tracking errors in every joint coordinate decouple and decay to zero independently at the designer-specified rate. Each joint then behaves as an independent second-order oscillator whose natural frequency and damping ratio follow directly from its diagonal gain pair.

B.2.4.1 Lie-derivative formulation for general control-affine systems

A general control-affine nonlinear dynamical system has a state rate $\dot{\mathbf{x}}$ that is a drift field $\mathbf{f}(\mathbf{x})$ plus an input matrix field applied linearly to $\mathbf{u}$, and an output $\mathbf{y} = \mathbf{h}(\mathbf{x})$. For such systems, feedback linearization differentiates each output channel y_ℓ with respect to continuous time until the control input $\mathbf{u}$ appears explicitly:

$$y_\ell^{(r_\ell)} = L_{\mathbf{f}}^{r_\ell} h_\ell(\mathbf{x}) + L_{\mathbf{g}} L_{\mathbf{f}}^{r_\ell - 1} h_\ell(\mathbf{x}) \mathbf{u}$$

where r_ℓ is the relative degree of output ℓ. Collecting all output channels into vector form:

$$\mathbf{y}^{(r)} = \mathbf{b}(\mathbf{x}) + \mathbf{E}(\mathbf{x}) \mathbf{u}$$

where $\mathbf{E}(\mathbf{x}) \in \mathbb{R}^{p \times m}$ is the **decoupling matrix** whose entries are Lie derivatives $E_{\ell,j}(\mathbf{x}) = L_{\mathbf{g}_j} L_{\mathbf{f}}^{r_\ell - 1} h_\ell(\mathbf{x})$. If $\mathbf{E}(\mathbf{x})$ is square ($p = m$) and non-singular across the operating envelope, the state-feedback law:

$$\mathbf{u} = \mathbf{E}(\mathbf{x})^{-1} \left(\mathbf{v} - \mathbf{b}(\mathbf{x}) \right)$$

cancels the modeled nonlinearities in the output equations, yielding $\mathbf{y}^{(r)} = \mathbf{v}$. Here $L_{\mathbf{f}} h = \nabla h^\top \mathbf{f}$ is the rate of change of h along $\mathbf{f}$, and repeated Lie derivatives differentiate that rate again. If $\sum_\ell r_\ell$ is less than the dimension of $\mathbf{x}$, internal states remain outside this input-output description; their zero dynamics must also be stable.

Unlike deep learning architectures where model inaccuracies are accommodated by statistical loss minimization, computed torque control assumes perfect knowledge of physical parameters ($\mathbf{M}, \mathbf{C}, \mathbf{g}$). In real hardware, parametric inaccuracies (uncertain link masses, payload variations) leave residual nonlinear perturbations $\Delta\mathbf{f} = (\mathbf{M} - \hat{\mathbf{M}})\ddot{\mathbf{q}} + (\mathbf{C} - \hat{\mathbf{C}})\dot{\mathbf{q}} + (\mathbf{g} - \hat{\mathbf{g}})$. Physical AI architectures resolve this by pairing feedback linearization with robust inner loops (sliding-mode control or H_∞ disturbance observers) or online adaptive parameter estimators.

In modern physical AI architectures, computed torque control also provides an analytical, differentiable physics inductive bias for learned policies. Rather than forcing a high-capacity neural network to learn the entire multi-body dynamics from scratch, the system splits torque synthesis into a nominal analytical dynamics model and a learned residual policy:

$$\tau = \hat{\mathbf{M}}(\mathbf{q})\ddot{\mathbf{q}}^* + \hat{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \hat{\mathbf{g}}(\mathbf{q}) + \tau_{\text{res},\theta}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}^*)$$

The baseline physics model cancels the dominant, known multi-body nonlinearities, while the neural network $\tau_{\text{res},\theta}$ is trained via reinforcement learning or imitation learning to capture unmodeled physical effects: joint cable friction, transmission backlash, hydrodynamic drag, and payload variations. This hybrid formulation can reduce the dynamics a policy must learn, but does not preserve baseline stability automatically. The residual needs an amplitude or energy bound compatible with the nominal controller, and the combined command must satisfy actuator limits.

B.2.5 Linear quadratic regulator and algebraic Riccati equations

In reinforcement learning (section 6.2.1), policy optimization searches for an action policy $\pi_\theta(\mathbf{a} \mid s)$ that maximizes expected cumulative reward via trial-and-error sampling or learned critic networks $V_\phi(s)$ / $Q_\phi(s, \mathbf{a})$. This search frequently struggles with high gradient variance, non-convex loss landscapes, and empirical stability failures. In contrast, for linear-quadratic physical dynamics, the **Linear Quadratic Regulator (LQR)** provides the exact, analytical solution to the continuous-state Bellman optimality equation over an infinite time horizon, as summarized against reinforcement learning below:

Dimension	Reinforcement Learning (Actor-Critic / Policy Gradient)	Linear Quadratic Regulator (LQR)
Value Function (Critic)	Parameterized deep neural network $V_\phi(s)$ or $Q_\phi(s,\mathbf{a})$	Exact quadratic potential $V^*(\mathbf{x}) = \mathbf{x}^\top \mathbf{P}\mathbf{x}$
Policy (Actor)	Parameterized neural network $\mathbf{a} \sim \pi_\theta(\cdot \mid s)$	Optimal linear state feedback $\mathbf{u}^* = -\mathbf{K}_{\text{lqr}}\mathbf{x}$
Bellman Optimality	Dynamic programming fixed point: $V^*_{\text{RL}}(s) = \max_{\mathbf{a}}[r(s,\mathbf{a}) + \gamma \mathbb{E}[V^*_{\text{RL}}(s')]]$	Continuous Algebraic Riccati Equation (ARE fixed point)
Solution Method	Stochastic gradient descent (PPO, SAC), TD learning	Direct algebraic factorizations (Schur decomposition, ARE solvers)
Stability Guarantees	Not supplied by empirical reward optimization alone	Stable under the stated model and rank conditions; ideal full-state margins below
Sample Complexity	Requires task- and algorithm-dependent training data	No policy-training samples once $(\mathbf{A}_{\text{sys}}, \mathbf{B}, \mathbf{Q}_x, \mathbf{R}_u)$ are specified; identifying the model still needs evidence

Given the controllable continuous LTI system $\dot{\mathbf{x}} = \mathbf{A}_{\text{sys}}\mathbf{x} + \mathbf{B}\mathbf{u}$, the infinite-horizon regulator minimizes the quadratic performance objective:

$$J = \int_0^\infty \left(\mathbf{x}(t)^\top \mathbf{Q}_x \mathbf{x}(t) + \mathbf{u}(t)^\top \mathbf{R}_u \mathbf{u}(t)\right) dt$$

Here the two weighting matrices price state error against actuator effort:

- $\mathbf{Q}_x \in \mathbb{R}^{n\times n}$ is a symmetric positive semi-definite state penalty matrix ($\mathbf{Q}_x \succeq \mathbf{0}$), penalizing deviations from the target state $\mathbf{x}^* = \mathbf{0}$.
- $\mathbf{R}_u \in \mathbb{R}^{m\times m}$ is a symmetric positive definite control penalty matrix ($\mathbf{R}_u \succ \mathbf{0}$), penalizing control effort.

The weights express a trade-off, not hard physical limits. Increasing $\mathbf{R}_u$ discourages effort, while increasing $\mathbf{Q}_x$ discourages state error. Neither guarantees a current ceiling, a torque slew rate, or a position bound. Those requirements need explicit constraints, such as the MPC formulation below, or a separate admissibility check (section 2.4).

B.2.5.1 Dynamic programming derivation of continuous LQR

In continuous time, Bellman's principle of optimality (Bellman 1957; Åström and Murray 2021) states that the optimal cost-to-go from state $\mathbf{x}(t)$ across an infinitesimal time step $\Delta t \to 0$ satisfies:

$$V^*(\mathbf{x}(t)) = \min_{\mathbf{u}_{[t,t+\Delta t]}} \left\{ \int_t^{t+\Delta t} \left(\mathbf{x}(t')^\top \mathbf{Q}_x \mathbf{x}(t') + \mathbf{u}(t')^\top \mathbf{R}_u \mathbf{u}(t')\right) dt' + V^*(\mathbf{x}(t+\Delta t)) \right\}$$

Expanding the integral to first order in Δt and applying a first-order Taylor expansion to the terminal cost gives:

$$V^*(\mathbf{x}(t+\Delta t)) = V^*(\mathbf{x}(t)) + \nabla V^*(\mathbf{x}(t))^\top \dot{\mathbf{x}}(t)\Delta t + \mathcal{O}(\Delta t^2)$$

Substituting the plant dynamics $\dot{\mathbf{x}} = \mathbf{A}_{\text{sys}}\mathbf{x} + \mathbf{B}\mathbf{u}$, subtracting $V^*(\mathbf{x}(t))$ from both sides, dividing by Δt, and taking the limit $\Delta t \to 0$ yields the continuous-time **Hamilton-Jacobi-Bellman (HJB)** equation:

$$0 = \min_{\mathbf{u}} \left[\mathbf{x}^\top \mathbf{Q}_x \mathbf{x} + \mathbf{u}^\top \mathbf{R}_u \mathbf{u} + \nabla V^*(\mathbf{x})^\top (\mathbf{A}_{\text{sys}}\mathbf{x} + \mathbf{B}\mathbf{u})\right]$$

Postulating a quadratic value function ansatz (the optimal continuous critic) $V^*(\mathbf{x}) = \mathbf{x}^\top \mathbf{P}\mathbf{x}$ with symmetric matrix $\mathbf{P} = \mathbf{P}^\top$ (positive definite under the controllability and observability assumptions stated below), its spatial gradient evaluates to:

$$\nabla V^*(\mathbf{x}) = 2\mathbf{P}\mathbf{x}$$

Differentiating the expression inside the bracket with respect to control input $\mathbf{u}$ and setting the derivative to zero yields the stationarity condition:

$$2\mathbf{R}_u\mathbf{u}+2\mathbf{B}^\top\mathbf{P}\mathbf{x}=\mathbf{0} \implies \mathbf{u}^*(t)=-\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}\mathbf{x}(t)=-\mathbf{K}_{\text{lqr}}\mathbf{x}(t)$$

The optimal state-feedback control law (the optimal actor policy) is strictly linear in the state, with constant optimal gain matrix $\mathbf{K}_{\text{lqr}}=\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}$. Substituting this optimal control law $\mathbf{u}^*$ back into the HJB equation eliminates the minimization:

$$\mathbf{x}^\top\mathbf{Q}_x\mathbf{x}+\mathbf{x}^\top\mathbf{P}\mathbf{B}\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}\mathbf{x}+2\mathbf{x}^\top\mathbf{P}\mathbf{A}_{\text{sys}}\mathbf{x}-2\mathbf{x}^\top\mathbf{P}\mathbf{B}\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}\mathbf{x}=0$$

Combining quadratic terms and using the matrix transpose identity $2\mathbf{x}^\top\mathbf{P}\mathbf{A}_{\text{sys}}\mathbf{x}=\mathbf{x}^\top(\mathbf{A}_{\text{sys}}^\top\mathbf{P}+\mathbf{P}\mathbf{A}_{\text{sys}})\mathbf{x}$ reveals:

$$\mathbf{x}^\top\left(\mathbf{A}_{\text{sys}}^\top\mathbf{P}+\mathbf{P}\mathbf{A}_{\text{sys}}-\mathbf{P}\mathbf{B}\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}+\mathbf{Q}_x\right)\mathbf{x}=0 \quad \forall\mathbf{x}\in\mathbb{R}^n$$

Because this equality must hold identically for all states $\mathbf{x}$, the bracketed matrix equation must vanish, yielding the canonical **Continuous Algebraic Riccati Equation (ARE)**:

$$\mathbf{A}_{\text{sys}}^\top\mathbf{P}+\mathbf{P}\mathbf{A}_{\text{sys}}-\mathbf{P}\mathbf{B}\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}+\mathbf{Q}_x=\mathbf{0}$$

Substituting the control law into the plant dynamics yields the closed-loop system $\dot{\mathbf{x}}=(\mathbf{A}_{\text{sys}}-\mathbf{B}\mathbf{K}_{\text{lqr}})\mathbf{x}$. When $(\mathbf{A}_{\text{sys}},\mathbf{B})$ is controllable and $(\mathbf{A}_{\text{sys}},\mathbf{Q}_x^{1/2})$ is observable, all closed-loop eigenvalues are strictly Hurwitz ($\text{Re}(\lambda_i(\mathbf{A}_{\text{sys}}-\mathbf{B}\mathbf{K}_{\text{lqr}}))<0$), guaranteeing exponential convergence to the origin.

B.2.5.2 LQR as an optimal closed-loop error dissipator

Beyond minimizing quadratic cost, the Riccati solution $\mathbf{P}\succ\mathbf{0}$ provides a Lyapunov function for the nominal closed loop. With a positive-definite running cost it also supplies a strict decrease certificate. Differentiating $V^*(\mathbf{x})=\mathbf{x}^\top\mathbf{P}\mathbf{x}$ along the closed-loop trajectories of $\dot{\mathbf{x}}=(\mathbf{A}_{\text{sys}}-\mathbf{B}\mathbf{K}_{\text{lqr}})\mathbf{x}$ gives:

$$\begin{aligned}\dot{V}^*(\mathbf{x})&=\dot{\mathbf{x}}^\top\mathbf{P}\mathbf{x}+\mathbf{x}^\top\mathbf{P}\dot{\mathbf{x}}=\mathbf{x}^\top\left[(\mathbf{A}_{\text{sys}}-\mathbf{B}\mathbf{K}_{\text{lqr}})^\top\mathbf{P}+\mathbf{P}(\mathbf{A}_{\text{sys}}-\mathbf{B}\mathbf{K}_{\text{lqr}})\right]\mathbf{x}\\&=\mathbf{x}^\top\left[\mathbf{A}_{\text{sys}}^\top\mathbf{P}+\mathbf{P}\mathbf{A}_{\text{sys}}-2\mathbf{P}\mathbf{B}\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}\right]\mathbf{x}\end{aligned}$$

Substituting the Algebraic Riccati Equation $\mathbf{A}_{\text{sys}}^\top\mathbf{P}+\mathbf{P}\mathbf{A}_{\text{sys}}=\mathbf{P}\mathbf{B}\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}-\mathbf{Q}_x$ into this expression yields:

$$\dot{V}^*(\mathbf{x})=\mathbf{x}^\top\left[-\mathbf{Q}_x-\mathbf{P}\mathbf{B}\mathbf{R}_u^{-1}\mathbf{B}^\top\mathbf{P}\right]\mathbf{x}=-\mathbf{x}^\top\left[\mathbf{Q}_x+\mathbf{K}_{\text{lqr}}^\top\mathbf{R}_u\mathbf{K}_{\text{lqr}}\right]\mathbf{x}\le 0$$

The derivative is strictly negative when the running-cost matrix is positive definite; with semidefinite $\mathbf{Q}_x$ it can vanish at a nonzero state, even though observability establishes asymptotic convergence. The equality identifies the dissipated quantity as cost, not necessarily physical energy. The value function decreases at a rate matching the instantaneous running cost $\mathbf{x}^\top\mathbf{Q}_x\mathbf{x}+\mathbf{u}^\top\mathbf{R}_u\mathbf{u}$.

B.2.5.3 Robustness margins of continuous LQR

Continuous-time full-state LQR has a useful robustness property (Åström and Murray 2021) under the ideal full-state assumptions, expressed by the frequency-domain **Kalman return difference equality**:

$$(\mathbf{I}+\mathbf{L}(j\omega))^*\,\mathbf{R}_u\,(\mathbf{I}+\mathbf{L}(j\omega))=\mathbf{R}_u+\mathbf{B}^\top(-j\omega\mathbf{I}-\mathbf{A}_{\text{sys}}^\top)^{-1}\mathbf{Q}_x(j\omega\mathbf{I}-\mathbf{A}_{\text{sys}})^{-1}\mathbf{B}$$

where * denotes conjugate transpose and $\mathbf{L}(j\omega)=\mathbf{K}_{\text{lqr}}(j\omega\mathbf{I}-\mathbf{A}_{\text{sys}})^{-1}\mathbf{B}$ is the open-loop frequency response broken at the plant input. Because $\mathbf{Q}_x\succeq\mathbf{0}$, the second term on the right-hand side is positive semi-definite at frequencies where the resolvents exist. For a single-input system with scalar $\mathbf{R}_u>0$, this establishes:

$$|1+L(j\omega)|\ge 1 \quad \forall\omega\in\mathbb{R}$$

Geometrically on the Nyquist plane, the loop transfer locus $L(j\omega)$ never enters the open unit disk centered at $-1+j0$. This property confers two guaranteed stability margins:

1. **Infinite Gain Margin:** The closed loop remains asymptotically stable under scalar gain scaling $\kappa\in(1/2,\infty)$ inserted into the control loop.

2. **Phase Margin Guarantee:** The distance from -1 to the Nyquist curve is at least 1, which by elementary trigonometry guarantees at least 60° of phase margin for the single-input loop:

$$\Phi_m \geq 2\arcsin(1/2) = 60^\circ$$

These are continuous-time, full-state, nominal-model margins at the specified loop break. They are not an unrestricted guarantee against actuator dynamics, structural resonances, or simultaneous multivariable perturbations. Digital sampling and transport add phase lag; section B.5.5 gives the low-frequency delay approximation used to budget that loss.

B.2.5.4 Discrete-time LQR

For digital microcontrollers executing at sampling period T_s, the discrete-time LQR formulation minimizes:

$$J_d = \sum_{k=0}^{\infty} \left(\mathbf{x}_k^\top \mathbf{Q}_x \mathbf{x}_k + \mathbf{u}_k^\top \mathbf{R}_u \mathbf{u}_k\right)$$

subject to $\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$. The matrix $\mathbf{P}$ satisfies the **Discrete Algebraic Riccati Equation (DARE)**:

$$\mathbf{P} = \mathbf{A}_d^\top \mathbf{P} \mathbf{A}_d - \mathbf{A}_d^\top \mathbf{P} \mathbf{B}_d \left(\mathbf{R}_u + \mathbf{B}_d^\top \mathbf{P} \mathbf{B}_d\right)^{-1} \mathbf{B}_d^\top \mathbf{P} \mathbf{A}_d + \mathbf{Q}_x$$

yielding the discrete optimal state-feedback gain:

$$\mathbf{K}_{\text{lqr},d} = \left(\mathbf{R}_u + \mathbf{B}_d^\top \mathbf{P} \mathbf{B}_d\right)^{-1} \mathbf{B}_d^\top \mathbf{P} \mathbf{A}_d, \quad \mathbf{u}_k^* = -\mathbf{K}_{\text{lqr},d} \mathbf{x}_k$$

B.2.6 The separation principle and linear quadratic Gaussian control

In physical deployments, full-state access is rarely available; controllers must estimate $\hat{\mathbf{x}}$ from noisy sensor observations $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{v}_{\text{meas}}$. Under a linear model with independent Gaussian white process and measurement noises of intensities $\mathbf{Q}$ and $\mathbf{R}_{\text{cont}}$, the optimal continuous state estimator is the continuous-time **Kalman-Bucy filter** (section B.8; the discrete filter the machine runs is in section 9.2):

$$\dot{\hat{\mathbf{x}}}(t) = \mathbf{A}_{\text{sys}} \hat{\mathbf{x}}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{L}\left(\mathbf{y}(t) - \mathbf{H}\hat{\mathbf{x}}(t)\right)$$

where $\mathbf{y}(t) - \mathbf{H}\hat{\mathbf{x}}(t)$ is the **innovation** (the measurement residual), and the optimal Kalman observer gain is:

$$\mathbf{L} = \mathbf{P}_{\text{est}} \mathbf{H}^\top \mathbf{R}_{\text{cont}}^{-1}$$

At steady state, $\mathbf{P}_{\text{est}} = \mathbb{E}[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^\top]$ is the state estimation error covariance matrix, which solves the dual filter algebraic Riccati equation under the usual stabilizability and detectability conditions:

$$\mathbf{A}_{\text{sys}} \mathbf{P}_{\text{est}} + \mathbf{P}_{\text{est}} \mathbf{A}_{\text{sys}}^\top - \mathbf{P}_{\text{est}} \mathbf{H}^\top \mathbf{R}_{\text{cont}}^{-1} \mathbf{H} \mathbf{P}_{\text{est}} + \mathbf{Q} = \mathbf{0}$$

To an ML practitioner, the Kalman filter is the exact recursive Bayesian Minimum Mean Square Error (MMSE) estimator. It continuously updates the posterior state distribution $\mathcal{N}(\hat{\mathbf{x}}, \mathbf{P}_{\text{est}})$ by balancing dynamic model propagation (growing uncertainty via process noise $\mathbf{Q}$) against empirical sensor evidence (contracting uncertainty via inverse continuous measurement-noise intensity $\mathbf{R}_{\text{cont}}^{-1}$). For discrete measurements, the analogous update uses the per-sample measurement covariance Σ_{meas}, not the continuous noise intensity. With invertible prior and measurement covariances, it evaluates as:

$$\mathbf{P}_{k|k}^{-1} = \mathbf{P}_{k|k-1}^{-1} + \mathbf{H}^\top \Sigma_{\text{meas}}^{-1} \mathbf{H}, \quad \mathbf{P}_{k|k}^{-1} \hat{\mathbf{x}}_{k|k} = \mathbf{P}_{k|k-1}^{-1} \hat{\mathbf{x}}_{k|k-1} + \mathbf{H}^\top \Sigma_{\text{meas}}^{-1} \mathbf{y}_k$$

In information (precision) space, the posterior precision is the linear addition of prior dynamic precision $\mathbf{P}_{k|k-1}^{-1}$ and measurement evidence precision $\mathbf{H}^\top \Sigma_{\text{meas}}^{-1} \mathbf{H}$. This reveals the Kalman filter as the exact recursive, online counterpart of Bayesian linear regression and Gaussian process conditioning, and highlights the mathematical duality between optimal control (LQR) and optimal estimation (Kalman filter):

Duality Attribute	Optimal Regulator (LQR)	Optimal Estimator (Kalman Filter)
System Operator	Drift matrix $\mathbf{A}_{\text{sys}}$	Adjoint drift matrix $\mathbf{A}_{\text{sys}}^{\top}$
Control / Sensor Map	Actuator input matrix $\mathbf{B}$	Sensor output matrix transpose $\mathbf{H}^{\top}$
Weighting / Noise Covariance	State weight $\mathbf{Q}_x \succeq \mathbf{0}$, control effort $\mathbf{R}_u \succ \mathbf{0}$	Process noise $\mathbf{Q} \succeq \mathbf{0}$, measurement noise $\mathbf{R}_{\text{cont}} \succ \mathbf{0}$
Riccati Solution	Control Riccati matrix $\mathbf{P}$	Estimation covariance matrix $\mathbf{P}_{\text{est}}$
Optimal Gain	Feedback gain $\mathbf{K}_{\text{lqr}} = \mathbf{R}_u^{-1}\mathbf{B}^{\top}\mathbf{P}$	Observer gain $\mathbf{L} = \mathbf{P}_{\text{est}}\mathbf{H}^{\top}\mathbf{R}_{\text{cont}}^{-1}$
Optimality Criterion	Minimizes quadratic state and action penalty	Minimizes trace of estimation error covariance $\mathbb{E}[\|\mathbf{x}-\hat{\mathbf{x}}\|^2]$

The **Linear Quadratic Gaussian (LQG)** controller cascades this optimal estimator with the optimal LQR regulator by applying feedback directly to the estimated state:

$$\mathbf{u}(t) = -\mathbf{K}_{\text{lqr}}\hat{\mathbf{x}}(t)$$

The **Separation Principle** (Åström and Murray 2021) proves that under linear-Gaussian assumptions, observer design and state-feedback controller design are completely decoupled. Specifically, the poles of the closed-loop LQG system are strictly the union of the controller poles $\text{eig}(\mathbf{A}_{\text{sys}} - \mathbf{B}\mathbf{K}_{\text{lqr}})$ and the observer poles $\text{eig}(\mathbf{A}_{\text{sys}} - \mathbf{L}\mathbf{H})$.

Systems Perspective B.1: The Doyle warning: LQG robustness margins

While full-state LQR provides guaranteed 60° phase margins and infinite gain margins, John Doyle's classic 1978 paper demonstrated that output-feedback LQG controllers possess **no guaranteed stability margins whatsoever** (Åström and Murray 2021). When an observer is inserted into the loop, unmodeled plant parameter variations couple estimation error $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$ directly back into the control actuation channel. Even infinitesimal parameter perturbations can then push the closed-loop poles across the imaginary axis into the unstable right-half plane.

For physical AI systems, this carries a warning. Cascading a high-capacity neural perception system (e.g., an end-to-end vision model estimating 6D object poses or end-effector coordinates) with an optimal tracking controller does not inherit the controller's nominal robustness. Dynamic perception lag, out-of-distribution observation errors, and high-frequency sensor noise degrade feedback phase margin rapidly. An implementation must evaluate the combined estimator-controller dynamics and its safety bounds; loop transfer recovery is one design technique, not a universal requirement (section 12.1).

B.3 Stability Theory and Model Predictive Control

An arm controller that converges exponentially to a target grasp pose can still swing its elbow through a cage-door post along the way. Stability concerns whether trajectories remain near, or converge to, an equilibrium; set invariance concerns whether they stay inside a safe region at every instant. These are separate claims, each conditional on a model, an initial-state domain, and the implementation assumptions.

B.3.1 Lyapunov stability and invariant sets

Continuous gradient flow provides a useful analogy for Lyapunov analysis. It evolves parameter weights according to $\dot{\theta} = -\nabla_{\theta}\mathcal{L}(\theta)$. Along this parameter trajectory, the empirical loss decreases monotonically via the multivariable chain rule:

$$\dot{\mathcal{L}}(\theta) = \nabla_{\theta}\mathcal{L}(\theta)^{\top}\dot{\theta} = -\nabla_{\theta}\mathcal{L}(\theta)^{\top}\nabla_{\theta}\mathcal{L}(\theta) = -\|\nabla_{\theta}\mathcal{L}(\theta)\|^2 \leq 0$$

In parameter optimization, the software program has unconstrained, direct write access to the parameter velocity vector $\dot{\theta}$, allowing the optimizer to force loss descent along the negative gradient direction. In physical control, the robot's physical state $\mathbf{x}(t) \in \mathbb{R}^n$ evolves in physical state space according to continuous differential equations $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$. Unlike software network weights θ, physical coordinates $\mathbf{x}$ cannot be updated by arbitrary gradient descent: actuators cannot command arbitrary velocity vectors $\dot{\mathbf{x}}$, because physical momentum, mass inertia, unforced drift, and kinematic constraints govern the system. By Newton's second law, motor actuators modulate second-order accelerations ($\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}(\tau - \mathbf{h})$), which must be integrated through continuous time against centrifugal forces, Coriolis couplings, and gravitational fields.

Instead, a **Lyapunov function** $V(\mathbf{x})$ acts as an analytical, continuous state-space loss function that certifies the asymptotic convergence of the physical plant's continuous trajectories without simulating every rollout to infinity. In mechanical systems, $V(\mathbf{x})$ often corresponds to the physical or synthetic Hamiltonian energy of the plant: kinetic energy stored in moving mass plus potential energy stored in gravitational elevation or virtual elastic springs.

Consider an autonomous nonlinear dynamical system:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)), \quad \mathbf{f}(\mathbf{0}) = \mathbf{0}$$

where $\mathbf{x}^* = \mathbf{0}$ is an isolated equilibrium point, and $\mathbf{f} : \mathcal{D} \to \mathbb{R}^n$ is locally Lipschitz on an open domain $\mathcal{D} \subset \mathbb{R}^n$ containing the origin.

A continuously differentiable scalar function $V : \mathcal{D} \to \mathbb{R}$ is a **candidate Lyapunov function** if it is positive definite on $\mathcal{D}$:

$$V(\mathbf{0}) = 0, \quad V(\mathbf{x}) > 0 \quad \forall \mathbf{x} \in \mathcal{D} \setminus \{\mathbf{0}\}$$

The directional derivative of V along the trajectories of $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ evaluates via the multivariable chain rule:

$$\dot{V}(\mathbf{x}) = \nabla V(\mathbf{x})^\top \dot{\mathbf{x}} = \nabla V(\mathbf{x})^\top \mathbf{f}(\mathbf{x}) = \nabla V(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x})$$

Control theory traditionally denotes this inner product as the **Lie derivative** $L_{\mathbf{f}} V(\mathbf{x}) = \nabla V(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x})$. In computational terms, this is the **directional derivative** or forward-mode **Jacobian-vector product (JVP)** of the scalar loss V evaluated in the direction of the physical drift vector field $\mathbf{f}(\mathbf{x})$:

$$\text{JVP}(V, \mathbf{x}, \mathbf{f}(\mathbf{x})) = \frac{d}{d\epsilon} V(\mathbf{x} + \epsilon \mathbf{f}(\mathbf{x}))\Big|_{\epsilon=0} = \nabla V(\mathbf{x})^\top \mathbf{f}(\mathbf{x}) = L_{\mathbf{f}} V(\mathbf{x})$$

Automatic differentiation can evaluate this directional derivative without forming a full Jacobian. Computing it at sampled states, however, does not verify its sign everywhere in a region.

Alexander Lyapunov established three foundational stability tiers (Slotine and Li 1991):

1. **Stability in the Sense of Lyapunov:** If $\dot{V}(\mathbf{x}) \leq 0$ for all $\mathbf{x} \in \mathcal{D}$, the origin is stable. Trajectories starting arbitrarily close to the origin remain trapped within bounded neighborhood contours for all $t \geq 0$.
2. **Asymptotic Stability:** If $\dot{V}(\mathbf{x}) < 0$ for all $\mathbf{x} \in \mathcal{D} \setminus \{\mathbf{0}\}$, the origin is asymptotically stable: $\lim_{t\to\infty} \mathbf{x}(t) = \mathbf{0}$. The system is guaranteed to descend the energy landscape toward the origin.
3. **Exponential Stability:** If there exist positive constants $k_1, k_2, k_3 > 0$ such that $k_1\|\mathbf{x}\|^2 \leq V(\mathbf{x}) \leq k_2\|\mathbf{x}\|^2$ and $\dot{V}(\mathbf{x}) \leq -k_3\|\mathbf{x}\|^2$, trajectories converge exponentially:

$$\|\mathbf{x}(t)\| \leq \sqrt{\frac{k_2}{k_1}} \|\mathbf{x}_0\| e^{-\frac{k_3}{2k_2}t} \quad \forall t \geq 0$$

For mechanical systems with friction and viscous damping, $\dot{V}(\mathbf{x})$ often vanishes on a subspace (for example, when velocities are zero while position errors remain nonzero: $\dot{V}(\mathbf{q}, \dot{\mathbf{q}}) = -\dot{\mathbf{q}}^\top \mathbf{D} \dot{\mathbf{q}} = 0$ whenever $\dot{\mathbf{q}} = \mathbf{0}$). Standard Lyapunov theorems only certify stability in the sense of Lyapunov, failing to prove asymptotic convergence. **LaSalle's Invariance Principle** (Slotine and Li 1991) resolves this. If $\dot{V}(\mathbf{x}) \leq 0$ on a compact, positively invariant set Ω, all trajectories starting in Ω asymptotically converge to the largest invariant set $\mathcal{M}$ contained within $\{\mathbf{x} \in \Omega \mid \dot{V}(\mathbf{x}) = 0\}$. Because any trajectory trapped on $\mathcal{M}$ with $\dot{\mathbf{q}} \equiv \mathbf{0}$ must also satisfy zero acceleration ($\ddot{\mathbf{q}} \equiv \mathbf{0}$), the equations of motion force

position errors to vanish ($\mathbf{K}_p\mathbf{e} = \mathbf{0} \implies \mathbf{e} = \mathbf{0}$). LaSalle's principle proves that the only trajectory that can remain indefinitely in the zero-dissipation manifold is the origin itself, certifying asymptotic convergence.

A compact sublevel set $\Omega_c = \{\mathbf{x} \in \mathcal{D} \mid V(\mathbf{x}) \leq c\}$ contained in $\mathcal{D}$ is positively invariant when $\dot{V} \leq 0$ throughout it. It is an inner estimate of the **domain of attraction (DOA)** only when trajectories there also converge to the equilibrium, for example by strict descent or the LaSalle argument. Invariance alone does not imply convergence.

A neural network can parameterize a candidate Lyapunov function and search for a stabilizing policy. For example, $V(\mathbf{x}) = \mathbf{x}^\top \mathbf{L}_\phi(\mathbf{x})\mathbf{L}_\phi(\mathbf{x})^\top \mathbf{x} + \epsilon_{\text{pos}}\|\mathbf{x}\|^2$, with $\epsilon_{\text{pos}} > 0$, guarantees positivity away from the origin. Convexity alone does not guarantee this property. Penalizing violations of $\dot{V} < 0$ during training produces a candidate; a certificate requires verifying the derivative condition throughout the claimed region, including the model and implementation uncertainty.

B.3.2 Control Lyapunov functions and stabilization

For a controlled system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$, the directional derivative along the closed-loop dynamics splits linearly into unforced drift and control effectiveness:

$$\dot{V}(\mathbf{x}, \mathbf{u}) = \nabla V(\mathbf{x}) \cdot (\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}) = L_{\mathbf{f}}V(\mathbf{x}) + L_{\mathbf{g}}V(\mathbf{x})\mathbf{u}$$

where $L_{\mathbf{f}}V(\mathbf{x}) = \nabla V(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x}) \in \mathbb{R}$ is the unforced directional drift and $L_{\mathbf{g}}V(\mathbf{x}) = \nabla V(\mathbf{x})^\top \mathbf{g}(\mathbf{x}) \in \mathbb{R}^{1\times m}$ is the control effectiveness gradient. A candidate Lyapunov function $V(\mathbf{x})$ is a **Control Lyapunov Function (CLF)** (Ames et al. 2014; Slotine and Li 1991) if there exists an admissible control input $\mathbf{u} \in \mathcal{U}_{\text{adm}}$ that forces V to decrease at every non-equilibrium state:

$$\inf_{\mathbf{u}\in\mathcal{U}_{\text{adm}}} \left[L_{\mathbf{f}}V(\mathbf{x}) + L_{\mathbf{g}}V(\mathbf{x})\mathbf{u} + \alpha_V(V(\mathbf{x}))\right] \leq 0 \quad \forall \mathbf{x} \in \mathcal{D} \setminus \{\mathbf{0}\}$$

where α_V is an extended class $\mathcal{K}$ function (a strictly increasing function with $\alpha_V(0) = 0$). For ML practitioners, $\alpha_V(V(\mathbf{x}))$ acts as a state-dependent convergence rate floor: the larger the current error $V(\mathbf{x})$, the steeper the required rate of descent. Geometrically, the CLF inequality defines a closed affine half-space in control space $\mathcal{U}_{\text{CLF}}(\mathbf{x}) = \{\mathbf{u} \in \mathcal{U}_{\text{adm}} \mid L_{\mathbf{g}}V(\mathbf{x})\mathbf{u} \leq -L_{\mathbf{f}}V(\mathbf{x}) - \alpha_V(V(\mathbf{x}))\}$; any control input chosen inside this half-space guarantees monotonic descent of the continuous state-space loss.

B.3.2.1 Sontag's universal stabilizing formula

For an unconstrained control-affine system, a smooth CLF admits the following stabilizing feedback construction (Slotine and Li 1991):

$$\mathbf{u}_{\text{Sontag}}(\mathbf{x}) = \begin{cases} -\left(\frac{L_{\mathbf{f}}V(\mathbf{x}) + \sqrt{(L_{\mathbf{f}}V(\mathbf{x}))^2 + \|L_{\mathbf{g}}V(\mathbf{x})\|^4}}{\|L_{\mathbf{g}}V(\mathbf{x})\|^2}\right)\left(L_{\mathbf{g}}V(\mathbf{x})\right)^\top & \text{if } L_{\mathbf{g}}V(\mathbf{x}) \neq \mathbf{0} \\ \mathbf{0} & \text{if } L_{\mathbf{g}}V(\mathbf{x}) = \mathbf{0} \end{cases}$$

The formula makes $\dot{V} < 0$ away from the origin when the CLF condition holds. Continuity at the origin additionally requires the *small control property*: arbitrarily small states can be stabilized with arbitrarily small inputs. The condition $L_{\mathbf{f}}V < 0$ wherever $L_{\mathbf{g}}V = 0$ is the CLF's uncontrollable-direction condition, not the small control property itself. The formula does not impose actuator limits; clipping its output can destroy the decrease condition.

B.3.2.2 The duality between CLFs and CBFs

Control Lyapunov Functions and Control Barrier Functions (section B.4) form the central duality of modern safety-critical control:

Dimension	Control Lyapunov Function (CLF)	Control Barrier Function (CBF)
Objective	Performance tracking / Goal convergence	Safety guarantee / Set invariance
Geometry	Positive-definite potential well ($V(\mathbf{x}^*) = 0$)	Superlevel safe set $\mathcal{C} = \{\mathbf{x} \mid h(\mathbf{x}) \geq 0\}$
Trajectory Dynamic	Pulls trajectories **inward** toward the origin ($V \to 0$)	Prevents outward crossing; may permit approach to the boundary
Differential Inequality	$L_{\mathbf{f}}V(\mathbf{x}) + L_{\mathbf{g}}V(\mathbf{x})\mathbf{u} \leq -\alpha_V(V(\mathbf{x}))$	$L_{\mathbf{f}}h(\mathbf{x}) + L_{\mathbf{g}}h(\mathbf{x})\mathbf{u} \geq -\kappa(h(\mathbf{x}))$
Optimization Priority	Soft constraint (relaxed via slack $\delta \geq 0$)	Hard constraint (strictly non-negotiable)

When combining performance tracking (such as following an uncertified trajectory proposed by a vision-language-action policy) with guaranteed collision avoidance, the enforcer solves a unified **CLF-CBF Quadratic Program (CLF-CBF-QP)**:

$$\begin{aligned} \min_{\mathbf{u}\in\mathcal{U},\delta\geq 0} \quad & \frac{1}{2}\|\mathbf{u}-\mathbf{u}_{\text{nom}}\|^2 + p\delta^2 \\ \text{subject to:} \quad & L_{\mathbf{f}}h(\mathbf{x}) + L_{\mathbf{g}}h(\mathbf{x})\mathbf{u} \geq -\kappa(h(\mathbf{x})) \quad \text{(hard safety constraint)} \\ & L_{\mathbf{f}}V(\mathbf{x}) + L_{\mathbf{g}}V(\mathbf{x})\mathbf{u} \leq -\alpha_V(V(\mathbf{x})) + \delta \quad \text{(soft tracking constraint)} \end{aligned}$$

where $p \gg 1$ heavily penalizes the tracking relaxation slack $\delta \geq 0$. The slack can relax tracking while leaving the modeled barrier condition hard. This only helps when at least one admissible input satisfies the barrier; tracking relaxation cannot repair an infeasible safety constraint.

B.3.3 Constrained model predictive control and recursive feasibility

While LQR and CLF feedback laws operate reactively on instantaneous state feedback, **Model Predictive Control (MPC)** optimizes trajectories over a predictive finite horizon N_{mpc}, explicitly enforcing hard inequality constraints on physical boundaries, motor voltages, and actuator torques. Numerical MPC and learned generative policies play complementary roles:

Attribute	Classical Numerical MPC	Learned Action Chunking (Diffusion, ACT)
Input Modality	Low-dimensional numerical states $\mathbf{x} \in \mathbb{R}^n$	High-dimensional perception (RGB images, point clouds, language)
Plant Model	Explicit analytical equations $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$	Implicit behavioral distribution learned from demonstration
Inference Time	Iterative solve; deadline depends on size, structure, and solver	Model-dependent inference, possibly iterative denoising
Constraints	Explicit modeled limits; guarantees require feasibility and model assumptions	Empirical; proposals can penetrate obstacles or exceed limits
Contact Discontinuities	Combinatorial branch stalling across non-smooth contacts	Learned contact behaviors; physical consistency requires validation
System Role	Deterministic Execution Gatekeeper / Tracking Enforcer	Generative Proposal Engine (long-horizon action chunking)

B.3.3.1 Discrete-time MPC formulation

At discrete sampling tick k, given current state measurement $\mathbf{x}_k = \mathbf{x}(t_k)$, the MPC controller solves a constrained finite-horizon optimal control problem over horizon N_{mpc}:

$$\begin{aligned}
\min_{\mathbf{u}_{0:N_{\text{mpc}}-1}} \quad & \mathbf{x}_{N_{\text{mpc}}}^\top \mathbf{P}_{N_{\text{mpc}}} \mathbf{x}_{N_{\text{mpc}}} + \sum_{i=0}^{N_{\text{mpc}}-1} \left(\mathbf{x}_i^\top \mathbf{Q}_x \mathbf{x}_i + \mathbf{u}_i^\top \mathbf{R}_u \mathbf{u}_i\right) \\
\text{subject to:} \quad & \mathbf{x}_{i+1} = \mathbf{A}_d \mathbf{x}_i + \mathbf{B}_d \mathbf{u}_i, \quad \mathbf{x}_0 = \mathbf{x}_k \\
& \mathbf{x}_i \in \mathcal{X}_{\text{safe}} \subset \mathbb{R}^n \quad \forall i \in \{0, \dots, N_{\text{mpc}}\} \\
& \mathbf{u}_i \in \mathcal{U}_{\text{adm}} \subset \mathbb{R}^m \quad \forall i \in \{0, \dots, N_{\text{mpc}}-1\} \\
& \mathbf{x}_{N_{\text{mpc}}} \in \mathcal{X}_f \subset \mathcal{X}_{\text{safe}}
\end{aligned}$$

where $\mathcal{X}_{\text{safe}} = \{\mathbf{x} \mid \mathbf{H}_x \mathbf{x} \leq \mathbf{b}_x\}$ represents state clearance constraints (joint limits, obstacle margins), $\mathcal{U}_{\text{adm}} = \{\mathbf{u} \mid \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}\}$ bounds actuator torque authority, $\mathcal{X}_f$ is a terminal invariant set, and $\mathbf{P}_{N_{\text{mpc}}}$ is the terminal cost matrix.

B.3.3.2 The receding-horizon principle and closed-loop stability

MPC executes in a receding-horizon closed loop:

1. Solve the constrained QP at step k, obtaining optimal sequence $\mathbf{u}^*_{0:N_{\text{mpc}}-1} = [\mathbf{u}^*_0, \mathbf{u}^*_1, \dots, \mathbf{u}^*_{N_{\text{mpc}}-1}]$.
2. Apply only the first control setpoint to physical hardware: $\mathbf{u}(t_k) = \mathbf{u}^*_0$.
3. Advance the hardware clock by T_s, discard the remaining trajectory $\mathbf{u}^*_{1:N_{\text{mpc}}-1}$, obtain updated state $\mathbf{x}_{k+1}$, and re-solve across the shifted horizon.

For the exact discrete model, an initially feasible state, and successful on-time solves, closed-loop stability can be established by choosing the terminal cost $\mathbf{P}_{N_{\text{mpc}}}$ and terminal set $\mathcal{X}_f$ to satisfy the classical Rawlings-Mayne stability conditions (Åström and Murray 2021; Wabersich and Zeilinger 2021):

1. **Terminal Invariance:** There exists a local linear feedback law $\mathbf{u} = -\mathbf{K}_f \mathbf{x}$ such that for all $\mathbf{x} \in \mathcal{X}_f$, $(\mathbf{A}_d - \mathbf{B}_d \mathbf{K}_f)\mathbf{x} \in \mathcal{X}_f$ and $-\mathbf{K}_f \mathbf{x} \in \mathcal{U}_{\text{adm}}$.
2. **Terminal Lyapunov Descent:** $\mathbf{P}_{N_{\text{mpc}}}$ satisfies the discrete Lyapunov inequality:

$$(\mathbf{A}_d - \mathbf{B}_d \mathbf{K}_f)^\top \mathbf{P}_{N_{\text{mpc}}} (\mathbf{A}_d - \mathbf{B}_d \mathbf{K}_f) - \mathbf{P}_{N_{\text{mpc}}} \leq -\left(\mathbf{Q}_x + \mathbf{K}_f^\top \mathbf{R}_u \mathbf{K}_f\right)$$

Under these conditions, the optimal cost function $J^*(\mathbf{x}_k)$ acts as a discrete Lyapunov function:

$$J^*(\mathbf{x}_{k+1}) - J^*(\mathbf{x}_k) \leq -(\mathbf{x}_k^\top \mathbf{Q}_x \mathbf{x}_k + \mathbf{u}_k^{*\top} \mathbf{R}_u \mathbf{u}_k^*) \leq 0$$

Strict descent away from the origin follows from a positive-definite state cost. With semidefinite cost, convergence instead requires an appropriate detectability argument. Under those assumptions, the nominal closed loop converges while respecting the modeled constraints.

B.3.3.3 Recursive feasibility and avoidance of dynamic traps

A hazard in constrained physical control is dynamic trap entry. A naive controller drives the system into a state where, due to physical inertia, finite stopping distance, and actuator saturation, no control sequence can prevent constraint violation N_{mpc} steps in the future, which renders the QP instantaneously infeasible. For example, a mobile robot cruising at high velocity toward a wall may have a feasible solution over horizon N_{mpc}. If the horizon terminates before braking completes, however, step $k+1$ encounters an unavoidable collision, because available deceleration cannot overcome kinetic momentum within the remaining distance.

A controller possesses **recursive feasibility** if the existence of a feasible solution at step k guarantees that the optimization problem remains feasible at step $k+1$ for all time. With the terminal invariant set $\mathcal{X}_f$ and exact nominal dynamics, the candidate shifted sequence:

$$\tilde{\mathbf{u}}_{0:N_{\text{mpc}}-1} = [\mathbf{u}^*_1, \mathbf{u}^*_2, \dots, \mathbf{u}^*_{N_{\text{mpc}}-1}, -\mathbf{K}_f \mathbf{x}_{N_{\text{mpc}}}]$$

is guaranteed to satisfy all dynamics, state, input, and terminal constraints at step $k+1$. The shifted sequence proves nominal recursive feasibility and can supply a warm start. Bounded disturbances alone do not preserve this proof: robust MPC must tighten constraints or propagate disturbance tubes and use a compatible robust terminal set. Solver failure and missed deadlines still require a reserved fallback.

B.3.3.4 Classical numerical MPC vs. learned action chunking

As framed in section 11.6, modern physical AI embodies a direct trade-off between numerical MPC and learned generative action chunking:

- **Model Accuracy vs. Semantic Ingestion:** Classical MPC relies on analytical differential equations ($\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$). Its stated low-dimensional formulation requires an upstream perception system to convert images, point clouds, or language into state estimates and goals. Modeling contact-mode changes creates an additional difficulty (section B.7). In contrast, generative action chunking policies (Diffusion Policy, Action Chunking with Transformers (ACT), Flow Matching) learn complex, multimodal behavioral distributions directly from human demonstrations.
- **Computational Scaling:** Dense optimization can become expensive as the horizon and state dimension grow; sparse and structured solvers can exploit the trajectory structure. No universal cubic runtime or control frequency follows from the MPC label. Learned action chunking moves much of the optimization into training, but its inference deadline still depends on model size, decoding steps, and hardware.
- **The Physical AI Synthesis:** Learned policies can propose action chunks while a model-based controller checks their physical feasibility. Determinism, recursive feasibility, and collision avoidance remain properties to establish for the selected model, constraints, solver, timing, and fallback. Neither MPC nor a CBF-QP provides them merely by being placed below the proposal boundary (section 12.1).

DRAFT

B.4 Safety and Control Barrier Functions

When a learned policy commands an arm toward a rack post at $1.0\,\mathrm{m/s}$, a position check that triggers only at contact is too late because inertia carries the link into the post. An independent enforcer can instead check a control barrier constraint before actuation. In reliable software systems, safety properties are proven using inductive loop invariants: if an invariant holds initially and is preserved across every state transition, it holds for all executions. A Control Barrier Function (CBF) lifts this principle to continuous physical systems, framing safety as an inductive loop invariant over continuous state trajectories:

Let the system state be $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^n$ and the control input be $\mathbf{u} \in \mathcal{U} \subset \mathbb{R}^m$. Suppose we define a safe operating set $\mathcal{C}$ as the superlevel set of a continuously differentiable scalar certificate function $h(\mathbf{x})$, with a nonzero gradient on its zero-level boundary:

$$\mathcal{C} = \{\mathbf{x} \in \mathcal{X} \mid h(\mathbf{x}) \geq 0\}, \quad \partial\mathcal{C} = \{\mathbf{x} \in \mathcal{X} \mid h(\mathbf{x}) = 0\}$$

- $h(\mathbf{x}) > 0$ when the system is strictly inside the safe interior.
- $h(\mathbf{x}) = 0$ on the boundary of the safe set.
- $h(\mathbf{x}) < 0$ when the system violates safety.

Invariant Concept	Formal Software Verification (Dafny, TLA+)	Safety-Critical Control (CBF)
State Space	Discrete variable states $s \in \Sigma$	Continuous state $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^n$
Base Step	Precondition assertion: $\{P(s_0)\}$	Safe initialization: $h(\mathbf{x}_0) \geq 0$
Transition Rule	Discrete program statement: $s_{k+1} = \mathrm{eval}(s_k, \mathrm{stmt})$	Differential equation: $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$

Invariant Concept	Formal Software Verification (Dafny, TLA+)	Safety-Critical Control (CBF)
Inductive Invariant	Invariant preservation: $P(s) \implies P(\text{eval}(s))$	Barrier condition: $\dot{h}(\mathbf{x}, \mathbf{u}) \geq -\kappa(h(\mathbf{x}))$
Proof Technique	SMT solvers (Z3, CVC5), structural induction	Grönwall-Bellman comparison lemma
Enforcement Layer	Runtime guard assertion / exception handler	Inline minimal-intervention QP projection

1. **Base Case:** The system initializes within the safe operating envelope: $h(\mathbf{x}_0) \geq 0$.
2. **Inductive Step:** Along all continuous transitions governed by plant dynamics and control inputs $\mathbf{u}_t \in \mathcal{U}$, the instantaneous rate of change satisfies:

$$\dot{h}(\mathbf{x}_t, \mathbf{u}_t) \geq -\kappa(h(\mathbf{x}_t))$$

   where $\kappa(\cdot)$ is an extended class $\mathcal{K}_\infty$ function (or linear decay $\kappa(h) = \gamma_{\text{cbf}} h$ with design bandwidth $\gamma_{\text{cbf}} > 0$).
3. **Continuous Induction via Comparison Lemma:** For the linear choice $\kappa(h) = \gamma_{\text{cbf}} h$, consider the scalar comparison differential equation:

$$\dot{z}(t) = -\gamma_{\text{cbf}} z(t), \quad z(0) = h(\mathbf{x}_0) \geq 0$$

   whose exact analytical solution is $z(t) = h(\mathbf{x}_0) e^{-\gamma_{\text{cbf}} t}$. Because $h(\mathbf{x}_0) \geq 0$ and $e^{-\gamma_{\text{cbf}} t} > 0$ for all finite $t \geq 0$, the comparison trajectory remains non-negative: $z(t) \geq 0$. By the Grönwall-Bellman comparison lemma (Slotine and Li 1991), the condition $\dot{h}(\mathbf{x}_t, \mathbf{u}_t) \geq -\gamma_{\text{cbf}} h(\mathbf{x}_t)$ guarantees:

$$h(\mathbf{x}_t) \geq z(t) = h(\mathbf{x}_0) e^{-\gamma_{\text{cbf}} t} \geq 0 \quad \forall t \geq 0$$

4. **Conclusion:** By induction over continuous time, $h(\mathbf{x}_t) \geq 0$ for all $t \geq 0$, guaranteeing forward invariance of the safe set $\mathcal{C}$. As the state approaches the boundary ($h(\mathbf{x}_t) \to 0^+$), the permissible rate of decrease $-\kappa(h)$ contracts to zero, enforcing $\dot{h}(\mathbf{x}_t, \mathbf{u}_t) \geq 0$. Consequently, $h(\mathbf{x}_t)$ can never cross zero with a negative slope.

B.4.1 Lie derivatives and forward invariance

Forward invariance means a trajectory starting in $\mathcal{C}$ remains in $\mathcal{C}$ for all future time. For a control-affine continuous dynamical system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$, the total time derivative $\dot{h}$ evaluates via the multivariate chain rule as directional derivatives along system dynamics:[1]

$$\dot{h}(\mathbf{x}, \mathbf{u}) = \nabla h(\mathbf{x})^\top \dot{\mathbf{x}} = \nabla h(\mathbf{x})^\top \mathbf{f}(\mathbf{x}) + \left(\nabla h(\mathbf{x})^\top \mathbf{g}(\mathbf{x})\right) \mathbf{u} = L_{\mathbf{f}} h(\mathbf{x}) + L_{\mathbf{g}} h(\mathbf{x}) \mathbf{u}$$

In modern computational terminology, the Lie derivatives $L_{\mathbf{f}} h(\mathbf{x})$ and $L_{\mathbf{g}} h(\mathbf{x})$ are Jacobian-vector products (JVPs) of the scalar safety function h evaluated along the drift and control vector fields:

$$L_{\mathbf{f}} h(\mathbf{x}) = \nabla h(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x}) \in \mathbb{R}, \quad L_{\mathbf{g}} h(\mathbf{x}) = \nabla h(\mathbf{x})^\top \mathbf{g}(\mathbf{x}) \in \mathbb{R}^{1 \times m}$$

where $L_{\mathbf{f}} h(\mathbf{x})$ captures the unforced directional drift of clearance (passive momentum, centrifugal forces, gravity), and $L_{\mathbf{g}} h(\mathbf{x})$ is the directional control effectiveness row vector mapping control inputs into instantaneous safety margin growth.

[1] **Boundary Viability**: Nagumo's tangent-cone condition characterizes invariance under regularity assumptions. The displayed CBF inequality is a sufficient design condition for the stated smooth control-affine model. A sampled implementation needs a separate bound for state error, computation, hold interval, and actuator response; failure of one candidate inequality does not prove contact within one integration step.

B.4.2 Nagumo's viability theorem and contingent cones

Whether a trajectory can remain inside a closed safe set $\mathcal{C} \subset \mathbb{R}^n$ depends on which velocity directions point inward or tangent along its boundary, a geometric condition formalized by Nagumo's Viability Theorem (Blanchini 1999). The **Bouligand contingent cone** (or tangent cone) $T_{\mathcal{C}}(\mathbf{x})$ to $\mathcal{C}$ at a point $\mathbf{x} \in \mathcal{C}$ is defined as:

$$T_{\mathcal{C}}(\mathbf{x}) = \left\{ \mathbf{v} \in \mathbb{R}^n \;\middle|\; \liminf_{\tau \to 0^+} \frac{\operatorname{dist}(\mathbf{x} + \tau \mathbf{v}, \mathcal{C})}{\tau} = 0 \right\}$$

where $\operatorname{dist}(\mathbf{y}, \mathcal{C}) = \inf_{\mathbf{z} \in \mathcal{C}} \|\mathbf{y} - \mathbf{z}\|$. For trajectories that exist for the interval considered, Nagumo's theorem proves that an autonomous system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ with locally Lipschitz vector field $\mathbf{f}$ keeps $\mathcal{C}$ forward-invariant if and only if the velocity vector satisfies the **subtangentiality condition**:

$$\mathbf{f}(\mathbf{x}) \in T_{\mathcal{C}}(\mathbf{x}) \quad \forall \mathbf{x} \in \mathcal{C}$$

In the interior $\operatorname{int}(\mathcal{C})$, $T_{\mathcal{C}}(\mathbf{x}) = \mathbb{R}^n$, so the condition is trivially satisfied. On the boundary $\partial\mathcal{C}$ of a smooth set defined by the zero-superlevel set of $h(\mathbf{x})$ ($\mathcal{C} = \{\mathbf{x} \mid h(\mathbf{x}) \geq 0\}$ with $\nabla h(\mathbf{x}) \neq \mathbf{0}$), the inward normal is $\nabla h(\mathbf{x})$. The contingent cone becomes the closed half-space of inward or tangent velocities:

$$T_{\mathcal{C}}(\mathbf{x}) = \left\{ \mathbf{v} \in \mathbb{R}^n \;\middle|\; \nabla h(\mathbf{x})^\top \mathbf{v} \geq 0 \right\} \quad \forall \mathbf{x} \in \partial\mathcal{C}$$

Consequently, the system cannot exit the set if and only if the directional derivative satisfies $\dot{h}(\mathbf{x}) = \nabla h(\mathbf{x})^\top \mathbf{f}(\mathbf{x}) \geq 0$ whenever $h(\mathbf{x}) = 0$. Control Barrier Functions (Ames et al. 2019) extend Nagumo's subtangentiality to controlled affine systems $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$. For safety, there must exist an admissible control $\mathbf{u} \in \mathcal{U}$ such that $\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u} \in T_{\mathcal{C}}(\mathbf{x})$. By introducing an extended class $\mathcal{K}_\infty$ decay rate $\kappa(h(\mathbf{x}))$ satisfying $\kappa(0) = 0$, the CBF inequality:

$$\dot{h}(\mathbf{x}, \mathbf{u}) = L_{\mathbf{f}} h(\mathbf{x}) + L_{\mathbf{g}} h(\mathbf{x})\mathbf{u} \geq -\kappa(h(\mathbf{x}))$$

ensures that as the state approaches the boundary ($h(\mathbf{x}) \to 0$), the admissible decay rate contracts to zero ($-\kappa(h(\mathbf{x})) \to 0$). At the boundary $h(\mathbf{x}) = 0$, it strictly enforces $\dot{h}(\mathbf{x}, \mathbf{u}) \geq 0$, directly preserving Nagumo's subtangentiality invariant and preventing trajectory exit into the unsafe set.

B.4.3 Relative degree and kinetic stopping insets

Position clearance cannot change instantaneously when a motor drive updates its torque command, because torque acts on acceleration rather than position. The **relative degree** r of a barrier function $h(\mathbf{x})$ counts how many times h must be differentiated with respect to continuous time before the control input $\mathbf{u}$ explicitly appears:

$$L_{\mathbf{g}} L_{\mathbf{f}}^k h(\mathbf{x}) = \mathbf{0} \quad \forall k < r-1, \quad \text{and} \quad L_{\mathbf{g}} L_{\mathbf{f}}^{r-1} h(\mathbf{x}) \neq \mathbf{0}$$

For a one-dimensional approach to a stationary obstacle, let raw spatial clearance be $h_{\text{raw}} = D_{\text{clear}}$, approach speed $v > 0$, and commanded approach acceleration a_{cmd}. Differentiating reveals:

$$\dot{D}_{\text{clear}} = -v, \quad \ddot{D}_{\text{clear}} = -a_{\text{cmd}}$$

Because acceleration enters raw clearance only at the second time derivative ($L_{\mathbf{g}} h_{\text{raw}} = 0$ and $L_{\mathbf{g}} L_{\mathbf{f}} h_{\text{raw}} \neq 0$), raw clearance has **relative degree two** ($r = 2$) with respect to force or acceleration inputs. An actuator torque command cannot instantaneously change position or velocity. Commanding maximum braking at the boundary $D_{\text{clear}} = 0$ is futile because momentum carries the system across the boundary before velocity can reach zero.

For systems with general relative degree $r \geq 2$, **High-Order Control Barrier Functions (HOCBFs)** construct a cascaded sequence of barrier certificates:

$$\begin{aligned}
\psi_0(\mathbf{x}) &= h(\mathbf{x}) \\
\psi_1(\mathbf{x}) &= \dot{\psi}_0(\mathbf{x}) + \alpha_1(\psi_0(\mathbf{x})) \\
&\vdots \\
\psi_r(\mathbf{x}) &= \dot{\psi}_{r-1}(\mathbf{x}) + \alpha_r(\psi_{r-1}(\mathbf{x})) = L_{\mathbf{f}}^r h(\mathbf{x}) + L_{\mathbf{g}} L_{\mathbf{f}}^{r-1} h(\mathbf{x})\mathbf{u} + \ldots \geq 0
\end{aligned}$$

The high-order condition also requires safe initialization of all intermediate certificates $\psi_0, \ldots, \psi_{r-1}$ and a feasible control throughout their intersection. In physical robotics, rather than tuning abstract class $\mathcal{K}$ functions across multiple derivatives, we can augment the barrier coordinate with kinetic energy and available stopping capacity. In a one-dimensional approach to a stationary obstacle, let D_{clear} be measured clearance, $v > 0$ approach speed, and $a_{\text{brake}} > 0$ a validated lower bound on available braking deceleration for the operating state. Defining the **kinetic stopping inset**:

$$h_{\text{kinetic}}(D_{\text{clear}}, v) = D_{\text{clear}} - \frac{v^2}{2a_{\text{brake}}}, \qquad \dot{D}_{\text{clear}} = -v, \quad \dot{v} = a_{\text{cmd}}$$

Differentiating $h_{\text{kinetic}}(D_{\text{clear}}, v)$ along the system dynamics yields:

$$\dot{h}_{\text{kinetic}} = \dot{D}_{\text{clear}} - \frac{v}{a_{\text{brake}}}\dot{v} = -v - \frac{v}{a_{\text{brake}}}a_{\text{cmd}}$$

Commanded acceleration a_{cmd} now appears directly in the first time derivative ($r = 1$). The CBF condition $\dot{h}_{\text{kinetic}} + \gamma_{\text{cbf}} h_{\text{kinetic}} \geq 0$ (with gain $\gamma_{\text{cbf}} > 0$) enforces the linear affine inequality on commanded acceleration:

$$a_{\text{cmd}} \leq \frac{a_{\text{brake}}}{v}\left(\gamma_{\text{cbf}} h_{\text{kinetic}} - v\right) \quad \text{for } v > 0$$

At standstill ($v \to 0$), a separate static clearance rule must be evaluated. State estimation error, digital sampling, and brake-onset delay require additional clearance reserves; the continuous kinematic calculation alone does not cover them.

B.4.4 Minimal-intervention quadratic program

The Control Barrier Function Quadratic Program (CBF-QP) changes a nominal command only as much as needed to satisfy the modeled barrier and actuator constraints. At a fixed state, the barrier condition is affine in the control input, so the filter solves a convex projection problem:

$$\begin{aligned} \mathbf{u}^* = \arg\min_{\mathbf{u}\in\mathcal{U}} \frac{1}{2}\|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \\ \text{subject to:} \quad L_{\mathbf{f}}h(\mathbf{x}) + L_{\mathbf{g}}h(\mathbf{x})\mathbf{u} \geq -\kappa(h(\mathbf{x})), \quad \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max} \end{aligned}$$

To a machine learning engineer, the CBF-QP is the continuous physical counterpart of **projection operators** used in optimization:

- In deep learning optimization, **Projected Gradient Descent (PGD)** enforces hard weight or adversarial perturbation constraints by projecting updated parameters back onto an ϵ-ball: $\mathbf{w}_{k+1} = \text{proj}_{\mathcal{C}}(\mathbf{w}_k - \eta\mathbf{g})$. In multi-task gradient learning (such as PCGrad or MGDA), conflicting task gradients $\nabla\mathcal{L}_i$ are projected onto the normal hyperplane of $\nabla\mathcal{L}_j$ via $\nabla\mathcal{L}_i \leftarrow \nabla\mathcal{L}_i - \frac{\nabla\mathcal{L}_i^\top \nabla\mathcal{L}_j}{\|\nabla\mathcal{L}_j\|^2}\nabla\mathcal{L}_j$ whenever their inner product is negative.
- In physical AI systems, the CBF-QP projects uncertified action vectors $\mathbf{u}_{\text{nom}} \in \mathbb{R}^m$ onto the convex polyhedron of forward-invariant safe actions $\mathcal{U}_{\text{safe}}(\mathbf{x}) = \{\mathbf{u} \in \mathcal{U} \mid L_{\mathbf{f}}h(\mathbf{x}) + L_{\mathbf{g}}h(\mathbf{x})\mathbf{u} \geq -\kappa(h(\mathbf{x}))\}$.

Write the affine CBF constraint in standard half-space form $\mathbf{a}_{\text{cbf}}^\top\mathbf{u} \leq b_{\text{cbf}}$, with normal vector $\mathbf{a}_{\text{cbf}} = -L_{\mathbf{g}}h(\mathbf{x})^\top \in \mathbb{R}^m$ and scalar threshold $b_{\text{cbf}} = L_{\mathbf{f}}h(\mathbf{x}) + \kappa(h(\mathbf{x}))$. Assume $\mathbf{a}_{\text{cbf}} \neq \mathbf{0}$ and consider the unconstrained actuator case ($\mathbf{u} \in \mathbb{R}^m$):

$$\min_{\mathbf{u}\in\mathbb{R}^m} \frac{1}{2}\|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad \text{subject to } \mathbf{a}_{\text{cbf}}^\top\mathbf{u} \leq b_{\text{cbf}}$$

The Lagrangian function is:

$$L_{\text{Lag}}(\mathbf{u}, \lambda) = \frac{1}{2}\|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 + \lambda(\mathbf{a}_{\text{cbf}}^\top\mathbf{u} - b_{\text{cbf}})$$

The Karush-Kuhn-Tucker (KKT) optimality conditions are:

1. **Stationarity:** $\nabla_{\mathbf{u}} L_{\text{Lag}} = \mathbf{u}^* - \mathbf{u}_{\text{nom}} + \lambda^*\mathbf{a}_{\text{cbf}} = \mathbf{0} \implies \mathbf{u}^* = \mathbf{u}_{\text{nom}} - \lambda^*\mathbf{a}_{\text{cbf}}$

2. **Primal Feasibility:** $\mathbf{a}_{\text{cbf}}^\top \mathbf{u}^* \leq b_{\text{cbf}}$
3. **Dual Feasibility:** $\lambda^* \geq 0$
4. **Complementary Slackness:** $\lambda^*(\mathbf{a}_{\text{cbf}}^\top \mathbf{u}^* - b_{\text{cbf}}) = 0$

These conditions yield two distinct operational regimes:

- **Permissive Pass-Through:** When the nominal neural proposal $\mathbf{u}_{\text{nom}}$ satisfies the safety constraint ($\mathbf{a}_{\text{cbf}}^\top \mathbf{u}_{\text{nom}} \leq b_{\text{cbf}}$), the optimal Lagrange multiplier is $\lambda^* = 0$. The filter acts as an identity pass-through:
$$\mathbf{u}^* = \mathbf{u}_{\text{nom}}$$
Zero control effort is distorted in this regime. Passing the command untouched preserves the full behavioral fidelity of the learned policy during normal task execution.
- **Active Minimal-Intervention Projection:** When the neural policy proposes an unsafe command that attempts to penetrate the barrier ($\mathbf{a}_{\text{cbf}}^\top \mathbf{u}_{\text{nom}} > b_{\text{cbf}}$), the safety constraint must be active ($\mathbf{a}_{\text{cbf}}^\top \mathbf{u}^* = b_{\text{cbf}}$) and $\lambda^* > 0$. Substituting the stationarity condition into the active constraint yields:
$$\mathbf{a}_{\text{cbf}}^\top (\mathbf{u}_{\text{nom}} - \lambda^* \mathbf{a}_{\text{cbf}}) = b_{\text{cbf}} \implies \mathbf{a}_{\text{cbf}}^\top \mathbf{u}_{\text{nom}} - \lambda^* \|\mathbf{a}_{\text{cbf}}\|^2 = b_{\text{cbf}}$$
Solving for the optimal multiplier yields:
$$\lambda^* = \frac{\mathbf{a}_{\text{cbf}}^\top \mathbf{u}_{\text{nom}} - b_{\text{cbf}}}{\|\mathbf{a}_{\text{cbf}}\|^2} > 0$$
The filtered control command is the exact, closed-form Euclidean projection onto the safe half-space boundary:
$$\mathbf{u}^* = \mathbf{u}_{\text{nom}} - \left(\frac{\mathbf{a}_{\text{cbf}}^\top \mathbf{u}_{\text{nom}} - b_{\text{cbf}}}{\|\mathbf{a}_{\text{cbf}}\|^2} \right) \mathbf{a}_{\text{cbf}}$$

DRAFT

Unifying both cases into a single compact projection operator:

$$\mathbf{u}^* = \text{proj}_{\mathcal{U}_{\text{safe}}}(\mathbf{u}_{\text{nom}}) = \mathbf{u}_{\text{nom}} - \frac{\max(0, \mathbf{a}_{\text{cbf}}^\top \mathbf{u}_{\text{nom}} - b_{\text{cbf}})}{\|\mathbf{a}_{\text{cbf}}\|^2} \mathbf{a}_{\text{cbf}}$$

Geometrically, the vector $\mathbf{a}_{\text{cbf}} = -L_{\mathbf{g}} h(\boldsymbol{x})^\top$ points outward along the gradient of safety penetration. The term $\max(0, \mathbf{a}_{\text{cbf}}^\top \mathbf{u}_{\text{nom}} - b_{\text{cbf}})$ is precisely the **hinge loss** (or ReLU activation) of safety margin penetration. When the proposal is safe, the loss is zero and the output is untouched. When the proposal is unsafe, the projection operator subtracts the exact scalar penetration depth along $\mathbf{a}_{\text{cbf}}$, leaving all orthogonal control directions untouched. To an ML engineer, this is the exact continuous-control counterpart of an analytical, one-step subgradient projection onto a Support Vector Machine margin hyperplane.

If $\mathbf{a}_{\text{cbf}} = \mathbf{0}$, division by $\|\mathbf{a}_{\text{cbf}}\|^2$ is invalid: all inputs satisfy this constraint when $b_{\text{cbf}} \geq 0$, and none do when $b_{\text{cbf}} < 0$. With actuator limits or multiple barriers, the nonempty admissible set is a convex polyhedron and the strictly convex objective has a unique projection. A numerical implementation must check feasibility, residual tolerance, and completion before its deadline. A solver name or nominal cadence supplies none of those bounds.

> **Systems Perspective B.2: Check feasibility before relying on invariance**
>
> The unconstrained projection formula does not solve the box-constrained problem. Clamping its result post hoc can violate $\mathbf{a}_{\text{cbf}}^\top \mathbf{u} \leq b_{\text{cbf}}$. If no command satisfies both the barrier half-space and actuator limits simultaneously ($\mathcal{U}_{\text{safe}} \cap \mathcal{U}_{\text{adm}} = \emptyset$), the modeled invariant cannot be maintained by this filter. A separately validated emergency fallback may still limit harm, but it cannot restore the lost forward-invariance claim. Admission must reserve stopping authority before the state reaches this boundary.

B.5 Smooth Trajectories and Bumpless Transfer

Switching directly between two action chunks that differ in velocity or acceleration commands an abrupt torque step across the gearbox. Because physical links and rotors possess inertia, a velocity step requests an ideal acceleration impulse, and an acceleration step requests an instantaneous torque jump under a rigid-inertia model. Whether a commanded seam excites drivetrain resonance or trips an inverter fault depends on the plant, load, and controller response.

B.5.1 Analytical quintic splines

A quintic (5th-degree) polynomial trajectory segment $q(t)$ over time interval $t \in [0, T]$ is defined as:

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

The six coefficients $[a_0, \dots, a_5]$ are uniquely determined by matching six boundary conditions at $t = 0$ and $t = T$:

$$q(0) = q_0, \quad \dot{q}(0) = \dot{q}_0, \quad \ddot{q}(0) = \ddot{q}_0$$
$$q(T) = q_1, \quad \dot{q}(T) = \dot{q}_1, \quad \ddot{q}(T) = \ddot{q}_1$$

Solving the resulting linear system yields the exact analytical formulas:

$$\begin{aligned}
a_0 &= q_0 \\
a_1 &= \dot{q}_0 \\
a_2 &= \frac{1}{2}\ddot{q}_0 \\
a_3 &= \frac{10(q_1 - q_0) - (6\dot{q}_0 + 4\dot{q}_1)T - (3\ddot{q}_0 - \ddot{q}_1)\frac{T^2}{2}}{T^3} \\
a_4 &= \frac{-15(q_1 - q_0) + (8\dot{q}_0 + 7\dot{q}_1)T + (3\ddot{q}_0 - 2\ddot{q}_1)\frac{T^2}{2}}{T^4} \\
a_5 &= \frac{6(q_1 - q_0) - 3(\dot{q}_0 + \dot{q}_1)T - (\ddot{q}_0 - \ddot{q}_1)\frac{T^2}{2}}{T^5}
\end{aligned}$$

Differentiating three times gives the segment's **jerk profile**, a quadratic in time that stays finite over a finite blend:

$$\dddot{q}(t) = 6a_3 + 24a_4 t + 60a_5 t^2$$

Matching position, velocity, and acceleration at the endpoints gives a C^2 join to adjacent segments with those same endpoint values and removes the commanded acceleration step at the join. The endpoint conditions say nothing about the segment's interior, so its jerk and torque extrema, and the joint's resonant response, still require a check against the machine's measured limits. Section 11.5 places this bridge at the seam between two action chunks.

B.5.1.1 Boundary value matrix formulation

The polynomial coefficients $\mathbf{a} = [a_0, a_1, a_2, a_3, a_4, a_5]^\top$ are determined by evaluating the trajectory and its first two derivatives at the temporal endpoints $t = 0$ and $t = T$. This yields the 6×6 confluent Vandermonde linear system $\mathbf{M}\mathbf{a} = \mathbf{b}$:

$$\begin{bmatrix}
1 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 2 & 0 & 0 & 0 \\
1 & T & T^2 & T^3 & T^4 & T^5 \\
0 & 1 & 2T & 3T^2 & 4T^3 & 5T^4 \\
0 & 0 & 2 & 6T & 12T^2 & 20T^3
\end{bmatrix}
\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{bmatrix}
=
\begin{bmatrix} q_0 \\ \dot{q}_0 \\ \ddot{q}_0 \\ q_1 \\ \dot{q}_1 \\ \ddot{q}_1 \end{bmatrix}$$

The top three rows trivially yield the initial boundary terms: $a_0 = q_0$, $a_1 = \dot{q}_0$, and $a_2 = \frac{1}{2}\ddot{q}_0$. Substituting these values into the bottom three rows isolates the higher-order coefficients $[a_3, a_4, a_5]^\top$ under the reduced 3×3 system:

$$\begin{bmatrix} T^3 & T^4 & T^5 \\ 3T^2 & 4T^3 & 5T^4 \\ 6T & 12T^2 & 20T^3 \end{bmatrix} \begin{bmatrix} a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} q_1 - q_0 - \dot{q}_0 T - \frac{1}{2}\ddot{q}_0 T^2 \\ \dot{q}_1 - \dot{q}_0 - \ddot{q}_0 T \\ \ddot{q}_1 - \ddot{q}_0 \end{bmatrix}$$

The determinant of this 3×3 matrix is $\det(\mathbf{M}_{33}) = 2T^9 > 0$ for all strictly positive blending horizons $T > 0$, guaranteeing non-singularity and uniqueness. The analytic matrix inverse is:

$$\begin{bmatrix} T^3 & T^4 & T^5 \\ 3T^2 & 4T^3 & 5T^4 \\ 6T & 12T^2 & 20T^3 \end{bmatrix}^{-1} = \begin{bmatrix} \frac{10}{T^3} & -\frac{4}{T^2} & \frac{1}{2T} \\ -\frac{15}{T^4} & \frac{7}{T^3} & -\frac{1}{T^2} \\ \frac{6}{T^5} & -\frac{3}{T^4} & \frac{1}{2T^3} \end{bmatrix}$$

Matrix multiplication $\mathbf{a}_{3:5} = \mathbf{M}_{33}^{-1}\mathbf{b}_{\text{red}}$ directly produces the closed-form coefficients a_3, a_4, a_5 stated above. Because the inverse scales as T^{-5} in its highest-order row, shrinking the blend duration T rapidly amplifies the commanded jerk and torque peaks.

B.5.2 Cubic Hermite spline interpolation for multi-rate synchronization

When synchronizing physical sensor and state streams arriving at heterogeneous rates (such as 30 Hz camera frames against 1 kHz joint encoder trajectories), intermediate poses must be continuously interpolated while preserving velocity continuity. A cubic Hermite spline over interval $t \in [t_0, t_1]$ with segment duration $T = t_1 - t_0$ interpolates position and velocity between endpoints:

$$q(t_0) = q_0, \quad \dot{q}(t_0) = \dot{q}_0, \quad q(t_1) = q_1, \quad \dot{q}(t_1) = \dot{q}_1$$

Introducing the normalized time coordinate $s = (t - t_0)/T \in [0, 1]$, the trajectory is parameterized by the cubic Hermite basis polynomials:

$$q(s) = h_{00}(s)q_0 + h_{10}(s)T\dot{q}_0 + h_{01}(s)q_1 + h_{11}(s)T\dot{q}_1$$

where the basis functions satisfy:

$$\begin{aligned} h_{00}(s) &= 2s^3 - 3s^2 + 1 \\ h_{10}(s) &= s^3 - 2s^2 + s \\ h_{01}(s) &= -2s^3 + 3s^2 \\ h_{11}(s) &= s^3 - s^2 \end{aligned}$$

Differentiating with respect to physical time t yields the instantaneous velocity:

$$\dot{q}(t) = \frac{1}{T}\frac{dq}{ds} = \frac{1}{T}\left(h'_{00}(s)q_0 + h'_{01}(s)q_1\right) + h'_{10}(s)\dot{q}_0 + h'_{11}(s)\dot{q}_1$$

with derivative basis functions:

$$\begin{aligned} h'_{00}(s) &= 6s^2 - 6s \\ h'_{10}(s) &= 3s^2 - 4s + 1 \\ h'_{01}(s) &= -6s^2 + 6s \\ h'_{11}(s) &= 3s^2 - 2s \end{aligned}$$

Because the basis polynomials satisfy $h_{00}(0) = 1, h_{00}(1) = 0, h'_{10}(0) = 1, h'_{10}(1) = 0$, this yields a C^1-continuous trajectory matching position and velocity exactly at sample boundaries without the computational overhead of solving a 6-boundary-condition quintic system. However, the resulting acceleration $\ddot{q}(t)$ is piecewise linear and discontinuous across sample boundaries (C^1 but not C^2), producing finite torque steps $\Delta\tau = M(q)\Delta\ddot{q}$ at knot points that require further smoothing when a finite torque-slew limit must be enforced.

B.5.3 Bumpless transfer and smoothstep blending

For bumpless transfer (section 14.4), let $\hat{t} = (t - t_{\text{start}})/\tau_{\text{blend}} \in [0,1]$ be the normalized progress, as in equation 14.6, and use the same C^2 smoothstep $\alpha(\hat{t})$ to interpolate two commands:

$$\alpha(\hat{t}) = 6\hat{t}^5 - 15\hat{t}^4 + 10\hat{t}^3, \qquad u(t) = (1-\alpha(\hat{t}))u_{\text{prior}}(t) + \alpha(\hat{t})u_{\text{new}}(t).$$

The endpoint first and second derivatives of α vanish. If u is commanded acceleration, its time derivative is commanded-acceleration jerk:

$$j_{\text{cmd}}(t) = \frac{\alpha'(\hat{t})}{\tau_{\text{blend}}}(u_{\text{new}} - u_{\text{prior}}) + (1-\alpha(\hat{t}))\dot{u}_{\text{prior}} + \alpha(\hat{t})\dot{u}_{\text{new}}.$$

Here $\alpha'(\hat{t}) = 30\hat{t}^2(1-\hat{t})^2$ differentiates with respect to *normalized* time and peaks at 1.875. Only for two fixed, latched endpoint accelerations ($\dot{u}_{\text{prior}} = \dot{u}_{\text{new}} = 0$) does the blend contribution satisfy

$$\max|j_{\text{cmd}}| = \frac{1.875|\Delta u|}{\tau_{\text{blend}}}, \qquad \tau_{\text{blend}} \geq \frac{1.875|\Delta u|}{j_{\max}}.$$

This is a command-rate condition. Measured plant jerk, tracking error, and available stopping clearance remain separate admission checks.

B.5.4 Flexible joint dynamics and singular perturbations

Mechanical transmissions such as harmonic strain-wave drives, planetary reducers, and cable linkages introduce torsional elasticity between actuator motor rotors and robot links. Following Mark W. Spong's canonical formulation (Spong 1987), an n-degree-of-freedom manipulator with elastic joints is governed by two coupled second-order vector differential equations:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{K}(\theta_{\text{motor}} - \mathbf{q})$$

$$\mathbf{J}_m\ddot{\theta}_{\text{motor}} + \mathbf{B}_m\dot{\theta}_{\text{motor}} + \mathbf{K}(\theta_{\text{motor}} - \mathbf{q}) = \tau$$

The coordinates and structural matrices have the following physical roles:

- $\mathbf{q} \in \mathbb{R}^n$ represents the physical link coordinates.
- $\theta_{\text{motor}} \in \mathbb{R}^n$ represents the motor rotor coordinates reflected through gear reduction N_{gear} to the link side.
- $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n\times n}$ is the symmetric, positive-definite link inertia matrix.
- $\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} \in \mathbb{R}^n$ collects Coriolis and centripetal terms.
- $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^n$ is the gravitational torque vector.
- $\mathbf{J}_m = \text{diag}(J_{m_1}, \dots, J_{m_n})$ is the diagonal matrix of reflected rotor inertias ($J_{m_i} = {N_{\text{gear}}}_i^2 {J_{\text{rotor}}}_i$).
- $\mathbf{B}_m = \text{diag}(B_{m_1}, \dots, B_{m_n})$ collects viscous motor damping.
- $\mathbf{K} = \text{diag}({K_{\text{joint}}}_1, \dots, {K_{\text{joint}}}_n)$ is the diagonal joint torsional stiffness matrix (${K_{\text{joint}}}_i > 0$).
- $\tau \in \mathbb{R}^n$ is the vector of actuator torques reflected to the link side, consistent with the reflected motor coordinates and inertias.

B.5.4.1 Singular perturbation formulation and time-scale separation

Time-scale separation requires the elastic modes to be fast relative to the intended rigid-body motion. After nondimensionalizing time, inertia, and stiffness by declared reference scales, write $\mathbf{K} = \tilde{\mathbf{K}}/\epsilon^2$, where ϵ is a small dimensionless ratio of fast to slow time scales and $\tilde{\mathbf{K}} = \mathcal{O}(1)$. A comparison such as stiffness greater than one has no physical meaning before this scaling.

Define the elastic torque fast state variable $\mathbf{z} \in \mathbb{R}^n$:

$$\mathbf{z} = \frac{1}{\epsilon^2}\tilde{\mathbf{K}}(\theta_{\text{motor}} - \mathbf{q}) = \mathbf{K}(\theta_{\text{motor}} - \mathbf{q})$$

Differentiating $\mathbf{z}$ twice with respect to continuous time t gives:

$$\ddot{\mathbf{z}} = \mathbf{K}(\ddot{\theta}_{\text{motor}} - \ddot{\mathbf{q}}) = \mathbf{K}\left[\mathbf{J}_m^{-1}(\tau - \mathbf{B}_m\dot{\theta}_{\text{motor}} - \mathbf{z}) - \mathbf{M}(\mathbf{q})^{-1}(\mathbf{z} - \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q}))\right]$$

Multiplying through by ϵ^2 reveals the singularly perturbed boundary layer:

$$\epsilon^2\ddot{\mathbf{z}} = \tilde{\mathbf{K}}\left[\mathbf{J}_m^{-1}\tau - (\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})\mathbf{z} + \mathbf{M}(\mathbf{q})^{-1}(\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})) - \mathbf{J}_m^{-1}\mathbf{B}_m\dot{\theta}_{\text{motor}}\right]$$

For the following undamped fast-mode calculation, set $\mathbf{B}_m = \mathbf{0}$ and freeze the slow configuration. Introducing the stretched fast time variable $t_{\text{fast}} = t/\epsilon$ transforms the elastic dynamics into:

$$\frac{d^2\mathbf{z}}{dt_{\text{fast}}^2} + \tilde{\mathbf{K}}(\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})\mathbf{z} = \tilde{\mathbf{K}}\mathbf{J}_m^{-1}\tau + \tilde{\mathbf{K}}\mathbf{M}(\mathbf{q})^{-1}(\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}))$$

B.5.4.2 The rigid manifold and boundary-layer resonance

In the singular limit as joint stiffness approaches infinity ($\epsilon \to 0$), the fast elastic variable $\mathbf{z}$ has the formal quasi-steady manifold $\mathbf{z}_0$:

$$\mathbf{z}_0 = (\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})^{-1}\left[\mathbf{J}_m^{-1}\tau + \mathbf{M}(\mathbf{q})^{-1}(\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}))\right]$$

The undamped expression does not prove that fast oscillations decay onto this manifold. Restoring damping and imposing the rigid constraint $\theta_{\text{motor}} = \mathbf{q}$ in the original equations gives the classical rigid-body equations with aggregated inertia $\mathbf{M}(\mathbf{q}) + \mathbf{J}_m$:

$$(\mathbf{M}(\mathbf{q}) + \mathbf{J}_m)\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \mathbf{B}_m\dot{\mathbf{q}} = \tau$$

For finite stiffness, a torque step excites the elastic modes; it is a step forcing, not an impulse. In physical time, their squared natural frequencies are the eigenvalues of the frozen-configuration matrix $\mathbf{K}(\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})$. Equivalently, the modal frequency matrix is:

$$\Omega(\mathbf{q}) = \sqrt{\mathbf{K}(\mathbf{J}_m^{-1} + \mathbf{M}(\mathbf{q})^{-1})}$$

The actual damping and resonant frequencies depend on the transmission and load. Smoothing commands and limiting torque slew can reduce excitation, but C^2 continuity alone does not establish a resonance or fatigue bound. The blend spectrum and measured closed-loop response must be checked against the flexible plant.

B.5.5 Discrete-time sampling and zero-order hold phase lag

Real-time digital controllers execute periodically at sampling period $T_s = 1/f_s$. Actuator drivers hold commanded setpoints constant between clock ticks using a **Zero-Order Hold (ZOH)**. At frequencies well below the sampling frequency, the zero-order hold's phase is equivalent to about half a sample period of delay:

$$\tau_{\text{ZOH}} = \frac{T_s}{2}$$

For a closed-loop control system operating at crossover frequency ω_c, this transport delay erodes the system's phase margin Φ_m by:[2]

$$\Delta\phi_{\text{ZOH}} = \omega_c\frac{T_s}{2} \quad \text{(radians)} \approx 28.65^\circ \cdot \left(\frac{\omega_c}{f_s}\right)$$

For fixed modeled compute and bus delays, the added phase lag near crossover is approximately $\Delta\phi_{\text{total}} = \omega_c(T_{\text{comp}} + T_{\text{bus}} + T_s/2)$. The controller must validate phase margin over the measured latency and operating envelope; the formula alone does not predict a particular contact response.

[2] **Phase Margin and Delay**: A pure delay T_d contributes phase $-\omega T_d$. The half-sample approximation applies in the baseband; actual stability depends on the sampled plant, compensation, jitter, and measured total response. Consuming modeled phase margin predicts loss of the stated stability reserve, not a specific destructive failure mode.

B.6 Dynamic Reachability, Intent Envelopes, and Differential Games

A target tote 0.60 m from the wrist is geometrically inside a 0.855 m manipulator workspace. Yet it is unreachable within a 150 ms intent lease if the arm cannot accelerate, traverse 0.60 m, and decelerate to rest within its torque and jerk limits before the lease expires. Kinematic reachability characterizes the set of configurations or end-effector poses a mechanism can reach geometrically, whereas dynamic reachability restricts that set to states attainable within the remaining time horizon and actuator bounds.

B.6.1 Forward kinematics and singularities

When a two-link arm stretches straight out, a small Cartesian velocity request along the extended link axis demands huge joint velocities near the singular alignment. Section 11.2 introduces forward kinematics, the Jacobian, and the singular pose on a planar two-link abstraction of the machine's arm. Section 11.4.5 turns a path into a timed trajectory under torque and jerk bounds. This section states the general n-joint form those examples specialize.

For an n-joint arm, the **forward-kinematics** map $\mathbf{FK}(\mathbf{q})$ sends joint coordinates $\mathbf{q} \in \mathbb{R}^n$ to an end-effector pose $\mathbf{T}_{WE} \in SE(3)$:

$$\mathbf{T}_{WE} = \mathbf{FK}(\mathbf{q}).$$

In a declared spatial or body twist convention, the **manipulator Jacobian** $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6\times n}$ relates joint rates to the six-vector end-effector twist $\mathbf{V}_E$:

$$\mathbf{V}_E = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}.$$

For a six-dimensional task and $n \geq 6$, rank loss ($\det(\mathbf{J}\mathbf{J}^\top) = 0$) marks a **kinematic singularity**. Tracking a nonzero twist along a lost direction would require unbounded joint rates in the ideal inverse model; real actuators saturate first. A task-specific design may monitor $w(\mathbf{q}) = \sqrt{\det(\mathbf{J}\mathbf{J}^\top)}$ against a validated threshold.[3]

[3] **Yoshikawa Manipulability Measure**: The measure summarizes the velocity ellipsoid for a chosen task Jacobian. Near rank loss, inverse-kinematics commands can exceed joint-rate or torque limits. The actual drive response and fallback depend on the specified controller and hardware.

To an ML engineer, the geometry of a kinematic singularity is best understood through the Singular Value Decomposition (SVD) of the Jacobian:

$$\mathbf{J}(\mathbf{q}) = \mathbf{U}\Sigma\mathbf{V}^\top = \sum_{i=1}^{6} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

For the full-row-rank case, $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_6 > 0$ are the singular values. The Yoshikawa manipulability index evaluates to the product of these singular values: $w(\mathbf{q}) = \prod_{i=1}^{6} \sigma_i$. In unregularized differential inverse kinematics, solving for commanded joint rates yields:

$$\dot{\mathbf{q}} = \mathbf{J}(\mathbf{q})^\dagger \mathbf{V}_E = \sum_{i=1}^{6} \left(\frac{\mathbf{u}_i^\top \mathbf{V}_E}{\sigma_i} \right) \mathbf{v}_i$$

As the arm approaches a singularity, the smallest singular value vanishes ($\sigma_6 \to 0$). Any commanded task twist with a nonzero projection onto the lost direction ($\mathbf{u}_6^\top \mathbf{V}_E \neq 0$) causes the commanded joint rate $\dot{\mathbf{q}}$ to explode as $\mathcal{O}(1/\sigma_6)$, potentially exceeding motor velocity limits. The drive response depends on its current and fault logic.

Roboticists resolve this through **Damped Least Squares (DLS)** inverse kinematics (Wampler 1986; Nakamura and Hanafusa 1986):

$$\mathbf{J}_\lambda^\dagger = \mathbf{J}^\top \left(\mathbf{J}\mathbf{J}^\top + \lambda^2 \mathbf{I} \right)^{-1}$$

For a spatial task mixing translation and rotation, first declare a task scaling or metric so that the singular values and damping parameter are comparable; the velocity bound below uses that same scaled norm. For each singular mode, the effective inversion gain is regularized from $1/\sigma_i$ to $\frac{\sigma_i}{\sigma_i^2+\lambda^2} \leq \frac{1}{2\lambda}$. This is mathematically identical to ℓ_2 Tikhonov regularization (ridge regression / weight decay) in machine learning: it deliberately introduces a bounded tracking residual in exchange for strictly bounded joint velocities ($\|\dot{\mathbf{q}}\| \leq \|\mathbf{V}_E\|/(2\lambda)$).

B.6.2 Trajectory transit time and reachable distance

For a one-dimensional rest-to-rest move with symmetric available acceleration $a_{\max}$ and speed ceiling $v_{\max}$, the shortest nominal transit has two regimes:

1. **Triangular profile (acceleration-limited):** For short horizons where peak velocity $v_{\max}$ is not attained ($d \leq d_{\text{crit}} = v_{\max}^2/a_{\max}$):

$$t_{\min} = 2\sqrt{\frac{d}{a_{\max}}}, \quad d_{\text{reach}}(T_{\text{rem}}) = \frac{1}{4} a_{\max} T_{\text{rem}}^2$$

2. **Trapezoidal profile (velocity-limited):** For larger horizons ($d > d_{\text{crit}}$):

$$t_{\min} = \frac{d}{v_{\max}} + \frac{v_{\max}}{a_{\max}}, \quad d_{\text{reach}}(T_{\text{rem}}) = v_{\max}\left(T_{\text{rem}} - \frac{v_{\max}}{a_{\max}}\right)$$

Below d_{crit} the reachable distance grows with the square of the available time, and above it only linearly, because the mechanism spends the rest of the move at its speed ceiling. When jerk is also bounded by $|\dot{a}| \le j_{\max}$, reaching full acceleration from zero takes at least $a_{\max}/j_{\max}$, if the profile reaches it at all. That bound exists because an acceleration step is a torque step, which shocks gearbox backlash, excites structural modes, and drives motor current toward the inverter's fault limits. The minimum-time trajectory may have four or seven jerk phases, so an unconditional additive offset to the trapezoidal time is invalid. Solve the constrained profile for the stated start/end velocity and acceleration, then include the stopping suffix and response delay when sizing a lease.

The idealized profiles still serve admission. Each added limit (jerk, torque, or thermal derating) can only shrink the **dynamic reachable set** $\mathcal{R}(T_{\text{rem}})$, the states reachable from the measured state within the remaining time $T_{\text{rem}} = t_{\text{intent_exp}} - t_{\text{now}}$. The transit times of the idealized profiles are therefore necessary lower bounds, and a proposal whose idealized time already exceeds T_{rem} can be rejected before a full path, torque, and stopping check. Section 10.6 applies this rejection to the arm's reach against the tracked horizon.

B.6.3 Hamilton-Jacobi reachability and worst-case differential games

While kinematic reachability examines static geometry and idealized transit bounds, dynamic reachability under external disturbances and modeling errors requires analyzing system evolution as a continuous differential game (Mitchell et al. 2005). Consider a nonlinear dynamical system governed by:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t), \mathbf{d}(t))$$

where $\mathbf{x} \in \mathbb{R}^n$ is the system state, $\mathbf{u} \in \mathcal{U}_{\text{adm}} \subset \mathbb{R}^m$ is the control input (seeking to maintain safety), and $\mathbf{d} \in \mathcal{W} \subset \mathbb{R}^p$ is an adversarial disturbance (representing unmodeled friction, wind gusts, dynamic obstacles, or worst-case model discrepancies).

Let the failure or collision region be represented by the open zero-sublevel set of a Lipschitz continuous surface function $l(\mathbf{x})$:

$$\mathcal{F}_{\text{fail}} = \{\mathbf{x} \in \mathbb{R}^n \mid l(\mathbf{x}) < 0\}$$

The safe set is the superlevel set $\{\mathbf{x} \mid l(\mathbf{x}) \ge 0\}$. The **Backward Reachable Tube (BRT)** $\mathcal{R}(t)$ is defined as the set of states from which the disturbance can force the system into collision at any time $t' \in [t, T]$, even if the control executes optimal evasive action:

$$\mathcal{R}(t) = \{\mathbf{x} \in \mathbb{R}^n \mid \forall \mathbf{u}(\cdot) \in \mathcal{U}_{\text{adm}},\ \exists \mathbf{d}(\cdot) \in \mathcal{W},\ \exists t' \in [t, T] \text{ s.t. } \mathbf{x}(t') \in \mathcal{F}_{\text{fail}}\}$$

Formulating safety verification as a continuous two-player zero-sum differential game, the value function $V(\mathbf{x}, t)$ represents the outcome of **continuous-state minimax Bellman dynamic programming** over trajectory time:

$$V(\mathbf{x}, t) = \sup_{\mathbf{u} \in \mathcal{U}_{\text{adm}}} \inf_{\mathbf{d} \in \mathcal{W}} \min_{t' \in [t, T]} l(\mathbf{x}(t'))$$

The system is guaranteed safe over $[t, T]$ if and only if $V(\mathbf{x}, t) \ge 0$. The zero-sublevel set of $V(\mathbf{x}, t)$ corresponds exactly to the Backward Reachable Tube: $\mathcal{R}(t) = \{\mathbf{x} \mid V(\mathbf{x}, t) < 0\}$.

Applying Bellman's principle of dynamic programming in continuous state space yields the **Hamilton-Jacobi-Isaacs (HJI)** variational inequality:

$$\min\left\{\frac{\partial V}{\partial t} + \mathcal{H}(\mathbf{x}, \nabla V(\mathbf{x}, t)),\ l(\mathbf{x}) - V(\mathbf{x}, t)\right\} = 0, \quad V(\mathbf{x}, T) = l(\mathbf{x})$$

where the Hamiltonian $\mathcal{H}(\mathbf{x}, \mathbf{p})$ evaluates the directional derivative of the value function along system dynamics under worst-case minimax competition:

$$\mathcal{H}(\mathbf{x}, \mathbf{p}) = \max_{\mathbf{u} \in \mathcal{U}_{\text{adm}}} \min_{\mathbf{d} \in \mathcal{W}} \mathbf{p}^\top \mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{d})$$

The trajectory notation above is shorthand for a differential game with nonanticipating strategies: actions may depend on available state history, not future disturbances. The information pattern and regularity assumptions must match the Hamiltonian ordering; arbitrary open-loop input quantifiers alone do not establish a feedback safety guarantee. Here $\mathbf{p} = \nabla V(\mathbf{x}, t) \in \mathbb{R}^n$ is the costate vector, representing the spatial gradient (directional sensitivity vector) of the safety boundary.

Because the value function $V(\mathbf{x}, t)$ develops non-differentiable kinks and gradient discontinuities where characteristics collide or bifurcate, classical smooth solutions do not exist globally. The PDE is solved in the sense of **viscosity solutions** (Mitchell et al. 2005).[4]

[4] **Viscosity Solutions and Level Set Methods**: Introduced by Crandall and Lions, viscosity solutions provide the mathematically rigorous framework for continuous weak solutions to first-order Hamilton-Jacobi PDEs where spatial gradients ∇V are discontinuous. High-order numerical schemes (such as WENO spatial discretization and TVD Runge-Kutta time integration) evaluate the sub- and super-differentials of V on regular Eulerian grids, tracking the implicit zero-level set without explicit front parameterization.

A regular grid with N points per coordinate in state dimension n stores $\mathcal{O}(N^n)$ values, making direct reachability analysis expensive as dimension grows. If an infinite-horizon value converges under suitable game assumptions, its nonnegative set describes robust viability for that model. The limiting value need not be smooth and is not automatically a classical differentiable CBF.

At runtime, a gridded or learned approximation can support a safety filter only with controlled approximation error. Where a suitable smooth certificate is available, its directional derivatives supply barrier constraints; otherwise nonsmooth reachability methods are needed. Distilling a value function into a neural network does not preserve the offline guarantee without verification.

B.6.4 Spatial coordinate frames and lever-arm kinematics

When an IMU or camera is mounted at a fixed offset from a robot's center of rotation, angular velocity and angular acceleration induce apparent linear motion at the sensor that must be separated from true translation. Rigid-body pose in three-dimensional space is parameterized by the Special Euclidean group $SE(3)$, where a rigid transform from frame B to frame A is represented by the homogeneous transformation matrix $\mathbf{T}_{AB} \in SE(3)$:

$$\mathbf{T}_{AB} = \begin{bmatrix} \mathbf{R}_{AB} & \mathbf{t}_{AB} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \mathbf{R}_{AB} \in SO(3), \quad \mathbf{t}_{AB} \in \mathbb{R}^3$$

The rotation group $SO(3)$ consists of orthonormal rotation matrices with determinant one; $SE(3)$ combines one such rotation with a translation. Thus $\mathbf{p}_A = \mathbf{R}_{AB}\mathbf{p}_B + \mathbf{t}_{AB}$ maps coordinates of a point from B to A, and transforms along a chain compose as $\mathbf{T}_{AC} = \mathbf{T}_{AB}\mathbf{T}_{BC}$. When sensors such as IMUs or cameras are displaced from the body center of rotation by lever arm $\mathbf{r}_{P/O}$, rotational motion induces apparent velocities and accelerations at sensor location P. The linear velocity $\mathbf{v}_P$ and linear acceleration $\mathbf{a}_P$ satisfy:

$$\mathbf{v}_P = \mathbf{v}_O + \omega \times \mathbf{r}_{P/O}$$

$$\mathbf{a}_P = \mathbf{a}_O + \alpha \times \mathbf{r}_{P/O} + \omega \times (\omega \times \mathbf{r}_{P/O})$$

All vectors in these sums must be expressed in the same frame, and P must be fixed to the rigid body. Here ω is the body angular velocity, $\alpha = \dot{\omega}$ is the angular acceleration, $\alpha \times \mathbf{r}_{P/O}$ is the tangential acceleration, and $\omega \times (\omega \times \mathbf{r}_{P/O})$ is the centripetal acceleration. State estimation and perception pipelines must compensate for these kinematic lever-arm terms when synchronizing multi-camera extrinsics and fusing IMU data.

B.6.5 Watchdog validity leases and teleoperation latency bounds

When an application processor stalls mid-aisle while the mobile base is cruising at 1.5 m/s, the base continues advancing 1.5 mm per millisecond until the safety microcontroller's lease expires and the brakes engage. In distributed autonomous systems and supervisory teleoperation architectures, actuation authority is governed by bounded-duration validity leases and communications latency constraints.

B.6.5.1 Kinematic derivation of watchdog validity leases

Consider an autonomous mobile platform or manipulator moving at speed v toward an obstacle with measured physical clearance D_{clear}. If the host processor or neural network fails to refresh the control setpoint, a hardware or firmware watchdog must intervene to prevent boundary violation. For a stop triggered by lease expiry, measure clearance at the supporting observation's capture epoch.

The elapsed time from that epoch until physical braking initiates is $\tau_{\text{delay,sub}} + \Delta_{\text{da}}$. The pre-brake subtotal $\tau_{\text{delay,sub}}$ comprises five nonoverlapping intervals, a lease-expiry form of the stopping-budget delay whose hazard-detection form appears in section C.13:

$$\tau_{\text{delay,sub}} = T_{\text{lease}} + t_{\text{detect}} + T_{\text{hold}} + t_{\text{age}} + T_{\text{act}}$$

and the dispatch-to-admission interval Δ_{da} is charged beside it, never inside it. Here:

- T_{lease} runs from successful admission to lease expiry; receipt or publication alone does not renew it.
- t_{detect} covers the post-expiry permission check and stop-command transport to the drive. It excludes the lease countdown already charged above and any separately charged hold.
- T_{hold} is an optional additional hold or fault-confirmation interval before the stop command reaches the drive; it is zero when the selected path has no such interval.
- t_{age} runs from evidence capture to perception's dispatch of the observation, including sensing and perception inference. Clock-conversion uncertainty must enter the age bound or a justified spatial allowance once.
- Δ_{da} runs from that dispatch through successful admission, including remaining proposer inference, transport, deserialization, validation, activation, and the wait for the next permission tick. Capture to admission is therefore $t_{\text{age}} + \Delta_{\text{da}}$, and a stored dispatch age covers only its first term.
- T_{act} runs from delivery of the stop command until the actuator establishes the braking deceleration assumed below.

The event boundaries follow the stalled-proposer timeline in section 4.4. A heartbeat timeout is an alternative trigger with its own response path, not an extra countdown appended to an expired chunk lease. If clearance is instead referenced to a hazard's appearance, the bound must also include any time until the supporting observation captures it.

Under constant cruising speed v before brake onset, a verified lower bound on mechanical braking deceleration $a_{\text{brake}} > 0$, a localization bound δ_{loc}, and a fixed protective clearance δ_{margin}, the total stopping distance bounded by available clearance D_{clear} satisfies:

$$v\,(\tau_{\text{delay,sub}} + \Delta_{\text{da}}) + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} \le D_{\text{clear}}$$

Solving directly for the maximum admissible validity lease T_{lease} yields:

$$T_{\text{lease}} \le \frac{D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - \frac{v^2}{2a_{\text{brake}}}}{v} - \left(t_{\text{detect}} + T_{\text{hold}} + t_{\text{age}} + \Delta_{\text{da}} + T_{\text{act}}\right)$$

This constant-deceleration model is a reference calculation, not the complete stopping budget of the running machine. A shaped stop must substitute its qualified stopping distance for $v^2/(2a_{\text{brake}})$ and reserve tracking error and every other justified allowance, and the same substitution applies to the teleoperation bound below. If the right-hand side is nonpositive, this budget permits no positive execution lease. The enforcer must select the available fallback immediately, because a negative bound means that even an immediate response lacks the stated stopping reservation.

B.6.5.2 Teleoperation network round-trip time bounds

When a human supervisor remotely teleoperates the robot across an external wireless link, the control loop includes human perceptual reaction latency and network packet transport delays, decomposed into five terms:

- T_{video} represent video camera exposure, encoding, and operator display processing, excluding network transport, which is charged in T_{RTT}.
- T_{human} denote human neuromuscular reaction time ($T_{\text{human}} \approx 150$–$250$ ms).
- $T_{\text{RTT}} = T_{\text{downlink}} + T_{\text{uplink}}$ represent network round-trip transport latency.
- T_{act} denote actuator braking engagement lag.

- $d_{\text{brake}} = \frac{v^2}{2a_{\text{brake}}}$ represent the purely mechanical braking distance.

Total available time before available clearance D_{clear} is exhausted is bounded by $T_{\text{total}} = (D_{\text{clear}} - d_{\text{brake}})/v$. The allowable round-trip network latency ceiling is:

$$T_{\text{RTT}} \leq \frac{D_{\text{clear}} - d_{\text{brake}}}{v} - (T_{\text{video}} + T_{\text{human}} + T_{\text{act}})$$

If network jitter pushes T_{RTT} beyond this bound, the human operator cannot perceive an obstacle and deliver a corrective command before the vehicle exhausts its stopping distance. Consequently, the local enforcer must evaluate physical invariants at a rate justified by its response budget, revoking operator command authority and executing autonomous braking before physical clearance is lost. Section 14.2 uses this bound to size a handover, where the operator's round trip spends the same pre-brake budget.

B.6.5.3 Kinodynamic bounds vs. wave-variable passivity

Transport delay in teleoperation creates two distinct failure modes: spatial boundary breach (kinematic collision) and high-frequency dynamic chatter (loss of closed-loop passivity). Wave variables (section B.10) preserve the communication channel's passivity under their delay assumptions. The coupled teleoperator still requires a valid energy argument for its other components. Passivity does not bound physical displacement. Safety requires satisfying both invariants simultaneously. Wave variables bound interactive contact energy, while the kinodynamic latency inequality reserves spatial clearance under its constant-speed, fixed-obstacle, and braking assumptions.

B.7 Rigid Body Contact Mechanics

In free space, a small change in joint torque produces a smooth change in link acceleration. When a gripper finger or foot touches a rigid surface, the gap closes to zero and a normal reaction force appears abruptly to prevent interpenetration. Simulating and controlling physical interaction (locomotion, dexterous manipulation, tool assembly) requires modeling this boundary transition. The ideal unilateral, nonadhesive rigid-contact model uses complementarity: interaction forces arise strictly when bodies establish physical contact.

Let $\mathbf{q} \in \mathbb{R}^n$ denote generalized configuration coordinates. Let $\phi(\mathbf{q})$ be the signed distance (or gap function) between contact geometries:

- $\phi(\mathbf{q}) > 0$: Bodies are separated.
- $\phi(\mathbf{q}) = 0$: Bodies are touching (contact established).
- $\phi(\mathbf{q}) < 0$: Bodies are penetrating (a geometric interpenetration that rigid mechanics strictly forbids).

Let λ_n (or f_n) denote the normal contact force pushing the bodies apart. In constrained mathematical optimization, the non-penetration condition corresponds to the inequality constraint $g(\mathbf{q}) = \phi(\mathbf{q}) \geq 0$. The governing physical law of unilateral rigid contact is precisely the **Karush-Kuhn-Tucker (KKT) complementary slackness condition**:

1. Non-penetration (primal feasibility): $\phi(\mathbf{q}) \geq 0$
2. Repulsive unilateral force (dual feasibility): $\lambda_n \geq 0$
3. Complementary slackness: $\phi(\mathbf{q})\lambda_n = 0$

These non-smooth physical contact laws map onto familiar mathematical concepts of constrained optimization:

Physical Entity	Mechanical Role in Physical Plant	Optimization Duality in ML Systems
Gap Function $\phi(\mathbf{q}) \geq 0$	Spatial clearance preventing interpenetration	Primal inequality constraint $g(\mathbf{x}) \geq 0$ defining the feasible region
Normal Force $\lambda_n \geq 0$	Repulsive contact force pushing bodies apart	Dual Lagrange multiplier $\lambda \geq 0$ associated with the active constraint

Physical Entity	Mechanical Role in Physical Plant	Optimization Duality in ML Systems
Complementary Slackness $\phi(\mathbf{q})\lambda_n = 0$	Contact force exists only when touching ($\phi = 0$)	KKT condition: multiplier is nonzero only when constraint is active
Delassus Operator $\mathbf{A}_{\text{Delassus}}$	Apparent inverse inertia reflected at contact point	Schur complement of the constraint Jacobian in the KKT system
Impact Impulse $P_n = m_{\text{eff}}\Delta v_n$	Instantaneous momentum transfer across zero duration	Non-differentiable jump in state trajectory; Dirac delta distribution
Restitution Law $v_n^+ = -ev_n^-$	Discontinuous velocity jump across collision boundary	Step discontinuity that breaks smooth reverse-mode automatic differentiation

Just as Lagrange multipliers in constrained optimization become nonzero ($\lambda > 0$) only when constraints are active and tight ($g = 0$), normal contact forces materialize exclusively upon contact closure:

- If bodies are separated ($\phi(\mathbf{q}) > 0$), the constraint is inactive, enforcing $\lambda_n = 0$.
- If bodies are touching ($\phi(\mathbf{q}) = 0$), the constraint is active, permitting repulsive normal forces $\lambda_n \geq 0$ to prevent interpenetration.
- This nonadhesive contact model excludes tensile normal forces ($\lambda_n < 0$); adhesion, grasping, and attached fixtures need other models.

B.7.1 The Delassus operator and acceleration-level LCP

At the velocity level, kinematic normal velocity is $v_n = \dot{\phi}(\mathbf{q}) = \mathbf{J}_n(\mathbf{q})\dot{\mathbf{q}}$, where $\mathbf{J}_n(\mathbf{q}) = \nabla\phi(\mathbf{q})^\top \in \mathbb{R}^{1\times n}$ is the contact normal Jacobian. Differentiating with respect to continuous time yields normal acceleration:

$$\ddot{\phi}(\mathbf{q}) = \mathbf{J}_n(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{J}}_n(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}}$$

Substituting the multi-body equations of motion $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \tau + \mathbf{J}_n(\mathbf{q})^\top\lambda_n$ isolates configuration acceleration:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}\left(\tau - \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q}) + \mathbf{J}_n(\mathbf{q})^\top\lambda_n\right)$$

Substituting $\ddot{\mathbf{q}}$ back into the normal acceleration expression yields:

$$\ddot{\phi} = \mathbf{A}_{\text{Delassus}}(\mathbf{q})\lambda_n + a_{\text{drift}}$$

where the scalar $\mathbf{A}_{\text{Delassus}}(\mathbf{q}) = \mathbf{J}_n(\mathbf{q})\mathbf{M}(\mathbf{q})^{-1}\mathbf{J}_n(\mathbf{q})^\top \geq 0$ is the **Delassus operational space compliance operator** (the apparent inverse inertia reflected at the contact point), and $a_{\text{drift}} = \mathbf{J}_n(\mathbf{q})\mathbf{M}(\mathbf{q})^{-1}(\tau - \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q})) + \dot{\mathbf{J}}_n(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}}$ collects unforced kinematic and dynamic drift accelerations.

When contact is maintained ($\phi = 0, \dot{\phi} = 0$), non-penetration requires $\ddot{\phi} \geq 0$. The contact mechanics form a canonical **Linear Complementarity Problem (LCP)**:

$$\ddot{\phi} \geq 0, \quad \lambda_n \geq 0, \quad \ddot{\phi}\cdot\lambda_n = 0$$

Under polyhedral friction cone approximations, this generalizes to a vector LCP solvable via Lemke's pivoting algorithm or quadratic programming. This is a rigid-contact idealization. Real interfaces deform, and the numerical contact solution must still be checked against the motion and force quantities the application needs.

B.7.2 Non-smooth collision manifolds and velocity jumps

When rigid bodies collide, the impact occurs over an infinitesimal duration $\Delta t \to 0$. For frictionless normal impact against a stationary rigid plane, let $v_n^- < 0$ be the pre-impact normal velocity and $e \in [0,1]$ the coefficient of restitution. The Newton restitution model imposes the velocity jump:

$$v_n^+ = -ev_n^- \implies \Delta v_n = v_n^+ - v_n^- = -(1+e)v_n^-$$

5 **Moreau Time-Stepping and Non-Smooth Mechanics**: Measure-differential models represent ideal impulses without resolving microscopic contact duration. Numerical contact solvers still require a validated geometry, friction law, and step-size study for the plant; a smooth optimizer that omits impact can miss a relevant force transient.

With effective normal mass m_{eff}, the normal impulse is $P_n = m_{\text{eff}} \Delta v_n$. In an idealized rigid impact, velocity changes discontinuously, meaning the acceleration is a Dirac delta distribution $\delta(t - t_{\text{impact}})$ rather than an ordinary finite derivative:[5]

$$\lim_{\epsilon \to 0^+} \frac{\|\dot{\mathbf{q}}(t_{\text{impact}} + \epsilon) - \dot{\mathbf{q}}(t_{\text{impact}} - \epsilon)\|}{2\epsilon} = \infty$$

B.7.2.1 Failure modes for automatic differentiation and BPTT

For a differentiable discrete simulator $\mathbf{x}_{k+1} = \mathbf{F}_k(\mathbf{x}_k, \theta)$, define state sensitivity $\mathbf{S}_k = \partial \mathbf{x}_k / \partial \theta$. The chain rule propagates it as:

$$\mathbf{S}_{k+1} = \frac{\partial \mathbf{F}_k}{\partial \mathbf{x}_k} \mathbf{S}_k + \frac{\partial \mathbf{F}_k}{\partial \theta}.$$

For a rollout loss $\mathcal{L} = \sum_{k=0}^{N} \ell_k(\mathbf{x}_k, \theta)$, the parameter gradient is:

$$\nabla_\theta \mathcal{L} = \sum_{k=0}^{N} \left[\mathbf{S}_k^\top \nabla_{\mathbf{x}_k} \ell_k + \nabla_\theta \ell_k \right].$$

Reverse-mode differentiation evaluates the same chain rule without storing every forward sensitivity. Contact complicates the derivatives of $\mathbf{F}_k$, not the chain rule itself. Within a fixed contact mode, a transition can have a finite derivative; at a mode change it can be nonsmooth, and the derivative must account for how the impact time changes with the initial state. An acceleration impulse alone does not imply that every state-transition Jacobian diverges.

A solver that differentiates only its currently active branch can miss the sensitivity of contact timing. Likewise, the contact force is zero while bodies are separated, but a distance-based loss can still supply a gradient toward contact. Whether a rollout has flat regions or abrupt changes depends on its dynamics, contact discretization, and objective.

Compliant contact replaces ideal impulses with deformation and finite-duration forces. This can improve differentiability, but the selected force law must remain repulsive and its stiffness can require smaller time steps. Implicit differentiation of converged contact constraints avoids differentiating every solver iteration, provided the active set and regularity conditions support a derivative. Stochastic policy gradients offer another route because they do not require differentiating through the contact transition; their variance and sampling cost remain separate concerns.

B.7.3 Elastodynamic impact force and compliance deformation

When an effective moving mass m_{eff} strikes an obstacle or human tissue, rigid-body approximations break down because real materials deform elastically over a nonzero compression stroke $\delta(t)$. Let k_{eff} denote the effective contact stiffness across the interface. In series compliance between the manipulator structural chain (k_{robot}) and human tissue or obstacle (k_H):

$$\frac{1}{k_{\text{eff}}} = \frac{1}{k_{\text{robot}}} + \frac{1}{k_H}$$

B.7.3.1 Unmitigated elastic impact force

For an unmitigated collision at impact velocity v_{max}, initial kinetic energy $E_k = \frac{1}{2} m_{\text{eff}} v_{\text{max}}^2$ converts entirely into elastic strain energy $U_{\text{elastic}} = \frac{1}{2} k_{\text{eff}} \delta_{\text{max}}^2$ at the point of maximum compression ($\dot{\delta} = 0$). Energy conservation yields the peak structural deflection:

$$\frac{1}{2} m_{\text{eff}} v_{\text{max}}^2 = \frac{1}{2} k_{\text{eff}} \delta_{\text{max}}^2 \implies \delta_{\text{max}} = v_{\text{max}} \sqrt{\frac{m_{\text{eff}}}{k_{\text{eff}}}}$$

The resulting peak transient contact force $F_{\text{unbraked}} = k_{\text{eff}} \delta_{\text{max}}$ evaluates to:

$$F_{\text{unbraked}} = v_{\text{max}} \sqrt{k_{\text{eff}} m_{\text{eff}}}$$

This relation applies to a lossless linear spring with fixed effective mass and no additional energy input during compression. It identifies how speed, mass, and stiffness affect the modeled peak; a human-contact limit additionally depends on contact location, area, duration, and the applicable test conditions. The scalar spring calculation alone does not establish compliance with a safety standard.

B.7.3.2 Coupled contact indentation with mechanical braking

When active mechanical brakes engage with constant opposing force $m_{\text{eff}}a_{\text{brake}}$ throughout compression, the braking system performs mechanical work $W_{\text{brake}} = m_{\text{eff}}a_{\text{brake}}\delta$ across the indentation depth δ. The coupled energy balance becomes:

$$\frac{1}{2}m_{\text{eff}}v_{\text{impact}}^2 = \frac{1}{2}k_{\text{eff}}\delta_{\text{max}}^2 + m_{\text{eff}}a_{\text{brake}}\delta_{\text{max}}$$

Rearranging into quadratic form:

$$\delta_{\text{max}}^2 + \frac{2m_{\text{eff}}a_{\text{brake}}}{k_{\text{eff}}}\delta_{\text{max}} - \frac{m_{\text{eff}}v_{\text{impact}}^2}{k_{\text{eff}}} = 0$$

Solving for the physically admissible positive deformation root:

$$\delta_{\text{max}} = \sqrt{\left(\frac{m_{\text{eff}}a_{\text{brake}}}{k_{\text{eff}}}\right)^2 + \frac{m_{\text{eff}}v_{\text{impact}}^2}{k_{\text{eff}}}} - \frac{m_{\text{eff}}a_{\text{brake}}}{k_{\text{eff}}}$$

When $a_{\text{brake}} = 0$, this expression smoothly recovers the unmitigated deformation $\delta_{\text{max}} = v_{\text{impact}}\sqrt{m_{\text{eff}}/k_{\text{eff}}}$. For the unbraked spring model, the quarter-cycle compression duration is $t_{\text{peak}} = \frac{\pi}{2}\sqrt{m_{\text{eff}}/k_{\text{eff}}}$. This is not the peak time of the actively braked model. Reactive braking triggered only *after* physical contact cannot attenuate the unbraked initial force peak if sensor detection and brake engagement latency exceed this t_{peak}. Consequently, safety cases cannot substitute reactive post-impact braking for verified pre-impact separation clearance envelopes ($d_{\text{sep}} \geq d_{\text{stop}}$).

B.7.4 Wheel slip ratio and friction boundaries

For wheeled mobile robots and terrestrial platforms, tractive force transmission between the wheel contact patch and the ground plane is governed by non-linear slip dynamics. The longitudinal slip ratio λ quantifies the kinematic velocity mismatch between the wheel circumferential speed $r\omega$ (wheel radius r, angular rate ω) and the vehicle forward longitudinal speed v_x:

$$\lambda = \begin{cases} \frac{r\omega - v_x}{r\omega}, & \text{under acceleration } (r\omega > v_x \geq 0) \\ \frac{r\omega - v_x}{v_x}, & \text{under braking } (v_x > r\omega \geq 0) \end{cases}$$

For forward motion with nonnegative wheel speed, the same convention can be regularized near standstill:

$$\lambda = \frac{r\omega - v_x}{\max(|v_x|, |r\omega|, \epsilon_{\text{reg}})} \in [-1, 1]$$

The bound $[-1, 1]$ assumes v_x and $r\omega$ have the same sign; reverse motion needs a declared direction convention. The endpoint values below also assume the nonzero speed exceeds ϵ_{reg}. Here $\lambda = 0$ corresponds to pure rolling without slip, $\lambda = 1$ denotes complete drive-wheel spin-out ($v_x = 0, \omega > 0$), and $\lambda = -1$ corresponds to complete wheel lock-up ($r\omega = 0, v_x > 0$).

Tractive force F_x scales with the normal contact load F_z through the longitudinal friction coefficient $\mu_x(\lambda)$:

$$F_x(\lambda) = \mu_x(\lambda)F_z$$

At low slip ratios ($|\lambda| < \lambda_{\text{peak}}$), where the peak depends on tire and surface conditions, tire or tread rubber shears elastically. Tractive force is approximately linear in slip: $F_x \approx C_x\lambda$, with longitudinal traction stiffness C_x. Beyond λ_{peak}, adhesion collapses into gross sliding friction. The slope $d\mu_x/d\lambda$ turns negative, which can destabilize a torque-controlled wheel-speed equilibrium. If an ungrounded policy commands drive motor torque exceeding peak traction ($F_{\text{cmd}} > \mu_{\text{peak}}F_z$), the wheel rapidly accelerates into the unstable slip regime ($d\omega/dt = (\tau - rF_x)/J_{\text{wheel}} > 0$). The resulting slip can reduce lateral authority or disturb yaw, a cyber-physical disconnect that cannot be detected by directional agreement or cosine similarity metrics that evaluate only vector angles while ignoring control magnitude.

B.7.5 Grasp mechanics: Friction cones and force closure

The nonadhesive contact model above lets a contact push but never pull. In the parallel-jaw hold considered here, friction supports the object against downward motion. Coulomb's law bounds the tangential force $\mathbf{f}_t$ a contact can carry by the normal force f_n pressing it, $\|\mathbf{f}_t\| \le \mu f_n$, where μ is the friction coefficient between finger and object. Every admissible contact force therefore lies inside a **friction cone** about the contact normal, with half-angle $\arctan\mu$.

A parallel-jaw hold shows the consequence with nothing beyond Newton's second law. Two fingers squeeze an object of mass m on opposite vertical faces, each with normal force f_n. Friction at the two contacts must carry the weight and any upward acceleration a_z the arm imposes while it moves:

$$2\mu f_n \ge m(g + a_z) \quad \Longrightarrow \quad f_n \ge \frac{m(g+a_z)}{2\mu} \tag{B.1}$$

The required squeeze grows with the arm's acceleration and inversely with friction: halving μ doubles it. The object sets the other bound: a thin-walled cup or a ripe fruit deforms or breaks above some normal force. A grasp is feasible only when the interval between the two bounds is nonempty, and that interval is the band a guarded-contact check $f_{\min} \le f_n \le f_{\max}$ enforces (section 10.1). In this two-finger model, the contact-force magnitude in that check denotes the normal squeeze at each finger, measured or conservatively inferred under qualified contact geometry. A generic force-vector norm or the net object wrench cannot substitute for that squeeze. Each term is a measurement the grasp needs: the object's mass, the friction coefficient of this finger material on this surface, the planned peak acceleration, and the crush force. Horizontal acceleration and the moment of an off-center load add terms of the same form.

The general statement collects every contact into one linear map. A force $\mathbf{f}_i$ applied at contact i, located at $\mathbf{r}_i$ from the object's reference point, exerts on the object the wrench formed by the force and its moment $\mathbf{r}_i \times \mathbf{f}_i$. Stacking all contact forces into $\mathbf{f}_c$ gives the **grasp map G**, with object wrench $\mathbf{w}_{obj} = \mathbf{G}\mathbf{f}_c$ (the same bold letter as the Kalman gain, but a different matrix). A grasp has **force closure** when every external wrench $\mathbf{w}_{ext}$ on the object can be balanced, $\mathbf{G}\mathbf{f}_c = -\mathbf{w}_{ext}$, by contact forces that all lie inside their friction cones (Lynch and Park 2017). Approximating each cone by a few edge vectors makes the test finite: force closure holds when the origin lies strictly inside the convex hull of the primitive wrenches those edges generate. For two fingers in a plane, Nguyen's condition is its geometric form: the segment joining the contact points must lie inside both friction cones (Nguyen 1988). In three dimensions, two point contacts cannot resist a torque about the line joining them; real fingertips resist it through torsional friction over a finite contact patch, the soft-finger contact model. Ferrari and Canny's quality measure, the radius of the largest origin-centered ball inside the hull, turns the yes-or-no test into the smallest disturbance the grasp is guaranteed to resist per unit of contact force (Ferrari and Canny 1992).

Force closure is qualitative. It says a balancing set of forces exists, not that the gripper can produce them within its force limit or without crushing the object. The magnitude check of equation B.1 therefore remains necessary for every grasp the machine admits.

B.8 Continuous Covariance Propagation and Observability

When sensors suffer line-of-sight dropouts or environmental occlusion, internal state estimators must project the physical state forward in time. Quantifying how rapidly estimation uncertainty grows across that blind interval requires continuous error-covariance dynamics and observability analysis. The discrete Kalman filter and its extended form, which the machine runs on dated measurements, are developed in section 9.2. The consistency tests that decide whether a filter's reported covariance can be trusted are in section 9.2.1. This section treats the continuous-time limit that governs uncertainty growth between measurements.

B.8.1 Continuous Lyapunov and Riccati covariance dynamics

To track how state-estimation uncertainty grows from process noise and contracts under sensor updates, consider a continuous-time linear dynamic system governed by:

$$\dot{\mathbf{x}}(t) = \mathbf{A}_{sys}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{w}(t)$$

$$\mathbf{y}(t) = \mathbf{H}\mathbf{x}(t) + \mathbf{v}_{\text{meas}}(t)$$

For the continuous white-noise model, $\mathbf{Q}$ and $\mathbf{R}_{\text{cont}}$ are process and measurement noise *intensities*, with covariance kernels $\mathbb{E}[\mathbf{w}(t)\mathbf{w}(s)^\top] = \mathbf{Q}\delta(t-s)$ and $\mathbb{E}[\mathbf{v}_{\text{meas}}(t)\mathbf{v}_{\text{meas}}(s)^\top] = \mathbf{R}_{\text{cont}}\delta(t-s)$. The model assumes suitable independence and $\mathbf{R}_{\text{cont}} \succ 0$. Measurement-noise intensity carries products of measurement-component units times seconds; per-sample measurement covariance carries products of measurement-component units.

Under an optimal continuous-time estimator (the Kalman-Bucy filter (Rudolph Emil Kalman 1960)), the error covariance matrix $\mathbf{P}(t) = \mathbb{E}[(\mathbf{x}-\hat{\mathbf{x}})(\mathbf{x}-\hat{\mathbf{x}})^\top]$ evolves according to the continuous matrix differential Riccati equation:[6]

$$\dot{\mathbf{P}}(t) = \mathbf{A}_{\text{sys}}\mathbf{P}(t) + \mathbf{P}(t)\mathbf{A}_{\text{sys}}^\top + \mathbf{Q} - \mathbf{P}(t)\mathbf{H}^\top\mathbf{R}_{\text{cont}}^{-1}\mathbf{H}\mathbf{P}(t)$$

[6] **Continuous Matrix Riccati Equation**: The measurement term reduces modeled covariance when valid measurements arrive. During occlusion, propagation follows the plant and process-noise model. Its growth may be bounded, linear, polynomial, or exponential depending on the dynamics and noise; the estimator must compare the resulting bound with the task tolerance.

When direct sensing is occluded or dropped ($\mathbf{H} = \mathbf{0}$), the measurement correction term vanishes. The covariance growth collapses to the open-loop **continuous Lyapunov differential equation**:

$$\dot{\mathbf{P}}(t) = \mathbf{A}_{\text{sys}}\mathbf{P}(t) + \mathbf{P}(t)\mathbf{A}_{\text{sys}}^\top + \mathbf{Q}$$

The solution propagates initial uncertainty and accumulated $\mathbf{Q}$ through $e^{\mathbf{A}_{\text{sys}}t}$. Unstable modes can grow exponentially; a random walk with $\mathbf{A}_{\text{sys}} = 0$ grows linearly in covariance. A state-memory record should carry the calibrated growth model and expire tracking permission before its declared error bound crosses the physical tolerance. Covariance alone is not a hard error envelope unless its tail and model assumptions are justified.

B.8.2 Closed-form double-integrator covariance expansion

Consider a one-dimensional double-integrator kinematic system modeling position $p(t)$ and velocity $v(t)$ under unmeasured stochastic acceleration noise $w(t)$:

$$\begin{bmatrix}\dot{p}(t)\\ \dot{v}(t)\end{bmatrix} = \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}\begin{bmatrix}p(t)\\ v(t)\end{bmatrix} + \begin{bmatrix}0\\ 1\end{bmatrix} w(t)$$

where $w(t)$ is zero-mean continuous white process noise with continuous spectral power intensity $q_c > 0$, such that $\mathbb{E}[w(t)w(s)] = q_c\delta(t-s)$. Here, $\mathbf{A}_{\text{sys}} = \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}$ and the continuous process noise matrix is $\mathbf{Q} = \begin{bmatrix}0 & 0\\ 0 & q_c\end{bmatrix}$. This intensity is not a per-sample acceleration variance. For position in meters and time in seconds, its units are square meters per cubic second, so multiplying it by elapsed time cubed gives positional variance in square meters.

Because $\mathbf{A}_{\text{sys}}^2 = \mathbf{0}$, the matrix exponential terminates after its linear term. The state transition is:

$$\Phi(t) = \mathbf{I} + \mathbf{A}_{\text{sys}}t = \begin{bmatrix}1 & t\\ 0 & 1\end{bmatrix}$$

During a blind occlusion window $t \in [0, \Delta t]$ with no measurement updates ($\mathbf{H} = \mathbf{0}$), the total state covariance $\mathbf{P}(t)$ evolves as the sum of unforced homogeneous propagation and accumulated process diffusion:

$$\mathbf{P}(t) = \Phi(t)\mathbf{P}_0\Phi(t)^\top + \int_0^t \Phi(t-t')\mathbf{Q}\Phi(t-t')^\top dt'$$

Let the initial state covariance at the instant of sensor loss ($t = 0$) be:

$$\mathbf{P}_0 = \begin{bmatrix}\sigma_{p,0}^2 & \sigma_{pv,0}\\ \sigma_{pv,0} & \sigma_{v,0}^2\end{bmatrix}$$

The homogeneous propagation term evaluates to:

$$\Phi(t)\mathbf{P}_0\Phi(t)^\top = \begin{bmatrix}1 & t\\ 0 & 1\end{bmatrix}\begin{bmatrix}\sigma_{p,0}^2 & \sigma_{pv,0}\\ \sigma_{pv,0} & \sigma_{v,0}^2\end{bmatrix}\begin{bmatrix}1 & 0\\ t & 1\end{bmatrix} = \begin{bmatrix}\sigma_{p,0}^2+2\sigma_{pv,0}t+\sigma_{v,0}^2t^2 & \sigma_{pv,0}+\sigma_{v,0}^2t\\ \sigma_{pv,0}+\sigma_{v,0}^2t & \sigma_{v,0}^2\end{bmatrix}$$

The accumulated stochastic diffusion term integrates across elapsed time:

$$\mathbf{Q}_d(t) = \int_0^t \begin{bmatrix}1 & t-t'\\ 0 & 1\end{bmatrix}\begin{bmatrix}0 & 0\\ 0 & q_c\end{bmatrix}\begin{bmatrix}1 & 0\\ t-t' & 1\end{bmatrix} dt' = \int_0^t \begin{bmatrix}q_c(t-t')^2 & q_c(t-t')\\ q_c(t-t') & q_c\end{bmatrix} dt' = \begin{bmatrix}\frac{1}{3}q_ct^3 & \frac{1}{2}q_ct^2\\ \frac{1}{2}q_ct^2 & q_ct\end{bmatrix}$$

Summing the homogeneous and stochastic contributions yields the closed-form positional variance $\sigma_p^2(t) = \mathbf{P}_{11}(t)$:

$$\sigma_p^2(t) = \sigma_{p,0}^2 + 2\sigma_{pv,0}t + \sigma_{v,0}^2t^2 + \frac{1}{3}q_ct^3$$

This derivation explains the distinct physical error regimes during sensor occlusion. Initial velocity uncertainty $\sigma_{v,0}^2$ induces quadratic variance growth in time (t^2), while continuous stochastic disturbances q_c (such as air currents or unmodeled drivetrain chatter) inject cubic uncertainty growth ($\frac{1}{3}q_ct^3$). In long blind intervals, the cubic term dominates, rapidly exhausting safety margins unless motion is derated or halted.

B.8.3 Observability rank and Lie derivative extensions

Measuring only a mobile base's wheel speeds tells an estimator how fast the base is moving, but not where it started in the aisle: two trajectories shifted by 0.5 m produce identical wheel-speed traces. In state estimation and machine learning, observability is the mathematical dual of reachability: whereas reachability asks whether latent internal states can be steered by control inputs, observability asks whether latent states can be reconstructed from external sensor observations $\mathbf{y}(t)$.

In modern deep learning (such as world models, state-space models, and latent representation learning), neural encoders $\hat{\mathbf{z}} = q_\phi(\mathbf{y})$ attempt to reconstruct underlying states from sensory observations. Observability establishes the information-theoretic limit of this reconstruction: if a dynamical system is unobservable, the mapping from physical state trajectories to sensor observations is strictly non-injective. In that case, distinct physical states can produce mathematically indistinguishable observation sequences. No neural network, regardless of parameter count or training data, can reconstruct an unobservable state without additional information. Active excitation can help some nonlinear systems, but a known input cannot repair the unobservable subspace of a fixed LTI model.

A linear time-invariant system $(\mathbf{A}_{\text{sys}}, \mathbf{H})$ is observable on $[0,T]$ if and only if the **Kalman observability matrix** has full column rank n:

$$\mathbf{O}_{\text{obs}} = \begin{bmatrix}\mathbf{H}\\ \mathbf{H}\mathbf{A}_{\text{sys}}\\ \vdots\\ \mathbf{H}\mathbf{A}_{\text{sys}}^{n-1}\end{bmatrix}, \quad \text{rank}(\mathbf{O}_{\text{obs}}) = n$$

If $\text{rank}(\mathbf{O}_{\text{obs}}) < n$, the unobservable subspace $\ker(\mathbf{O}_{\text{obs}})$ contains initial state trajectories that produce identical output measurements. No linear or nonlinear estimator can distinguish between initial states that differ along $\ker(\mathbf{O}_{\text{obs}})$.

For a smooth autonomous nonlinear system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$, $\mathbf{y} = \mathbf{h}(\mathbf{x})$ where $\mathbf{h} = [h_1(\mathbf{x}), \dots, h_p(\mathbf{x})]^\top$, the Hermann-Krener observability criterion (Hermann and Krener 1977) generalizes the Kalman rank test via successive directional derivatives along system dynamics. Differentiating the measurement output equation $\mathbf{y}(t) = \mathbf{h}(\mathbf{x}(t))$ with respect to continuous time yields:

$$\dot{\mathbf{y}}(t) = \frac{\partial \mathbf{h}}{\partial \mathbf{x}}\dot{\mathbf{x}} = \frac{\partial \mathbf{h}}{\partial \mathbf{x}}\mathbf{f}(\mathbf{x}) = L_{\mathbf{f}}\mathbf{h}(\mathbf{x})$$

Differentiating repeatedly produces the sequence of Lie derivatives, which are directional derivatives of the measurement channels along the unforced drift vector field $\mathbf{f}$:

$$\mathbf{y}^{(k)}(t) = L_{\mathbf{f}}^{k}\mathbf{h}(\mathbf{x}), \quad \text{where } L_{\mathbf{f}}^{k}h_{\ell}(\mathbf{x}) = \nabla(L_{\mathbf{f}}^{k-1}h_{\ell}(\mathbf{x}))\cdot\mathbf{f}(\mathbf{x})$$

These successive directional derivatives reveal how temporal variations in output channels encode higher-order kinematic and dynamic states (such as recovering joint velocities and accelerations from encoder or camera position sequences) without requiring real-time numerical differentiation of noisy signals.

The **Hermann-Krener observation space** $\mathbf{O}_{\mathrm{NL}}$ is the real vector space of all repeated Lie derivatives $\{L_{\mathbf{f}}^{k}h_{\ell}\}_{k\geq 0, 1\leq \ell\leq p}$. Its gradient differentials span the **observability codistribution**:

$$\mathrm{d}\mathbf{O}_{\mathrm{NL}}(\mathbf{x}) = \operatorname{span}\left\{\mathrm{d}L_{\mathbf{f}}^{k}h_{\ell}(\mathbf{x}) \;\middle|\; k\geq 0,\ 1\leq \ell\leq p\right\}$$

where $\mathrm{d}\phi(\mathbf{x}) = \frac{\partial\phi}{\partial\mathbf{x}}$. The system satisfies the **nonlinear observability rank condition** at $\mathbf{x}_0$ if:[7]

$$\dim(\mathrm{d}\mathbf{O}_{\mathrm{NL}}(\mathbf{x}_0)) = n$$

When full rank n is achieved, the directional gradients span the cotangent space $T_{\mathbf{x}}^{*}\mathcal{X}$, guaranteeing local weak observability: any infinitesimal deviation in latent state produces a distinguishable signature in the measurement sequence or its temporal derivatives.

[7] **Hermann-Krener Nonlinear Observability**: Full rank of the observation codistribution is a local structural test, not a sensor-noise or finite-horizon accuracy bound. A missing state direction may require another sensor, an active maneuver with its own permission, or a restricted operating envelope.

B.9 Centroidal Dynamics and Floating-Base Control

Legged robots have an unactuated floating base as well as motorized joints. Internal joint motion can change the base pose, but only external forces change total linear momentum, and only external moments change total angular momentum. Ground contacts therefore determine which center-of-mass and momentum trajectories the robot can sustain against gravity.

DRAFT

B.9.1 Floating-base equations of motion

Let $\mathbf{q} = [\mathbf{q}_b^{\top}, \mathbf{q}_j^{\top}]^{\top} \in SE(3)\times\mathbb{R}^n$ represent the generalized coordinates, where $\mathbf{q}_b \in SE(3)$ is the floating-base pose (position and orientation, 6 DoF) and $\mathbf{q}_j \in \mathbb{R}^n$ are the actuated joint angles. The total system possesses $n+6$ degrees of freedom. The displayed $\dot{\mathbf{q}}$ and $\ddot{\mathbf{q}}$ use a local coordinate convention; a global implementation must distinguish configuration coordinates from generalized velocities, particularly with quaternion orientation. The equations of motion are (Siciliano et al. 2009):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{S}^{\top}\tau + \sum_{k=1}^{N_c}\mathbf{J}_{\mathrm{c}_k}(\mathbf{q})^{\top}\mathbf{f}_k$$

The right-hand terms separate internal joint actuation from external contact support:

- $\mathbf{S} = [\mathbf{0}_{n\times 6}, \mathbf{I}_{n\times n}] \in \mathbb{R}^{n\times(n+6)}$ is the actuation selection matrix separating the 6 unactuated floating-base coordinates from the n actuated joints.
- $\tau \in \mathbb{R}^n$ is the commanded motor torque vector.
- $\mathbf{J}_{\mathrm{c}_k}(\mathbf{q}) = \partial\mathbf{p}_k/\partial\mathbf{q} \in \mathbb{R}^{3\times(n+6)}$ is the contact point Jacobian for point-contact force $\mathbf{f}_k \in \mathbb{R}^3$.

Partitioning the equations of motion into floating-base (b) and actuated joint (j) coordinates highlights the underactuation of legged machines:

$$\begin{bmatrix}\mathbf{M}_{bb}(\mathbf{q}) & \mathbf{M}_{bj}(\mathbf{q})\\ \mathbf{M}_{jb}(\mathbf{q}) & \mathbf{M}_{jj}(\mathbf{q})\end{bmatrix}\begin{bmatrix}\ddot{\mathbf{q}}_b\\ \ddot{\mathbf{q}}_j\end{bmatrix} + \begin{bmatrix}\mathbf{h}_b(\mathbf{q},\dot{\mathbf{q}})\\ \mathbf{h}_j(\mathbf{q},\dot{\mathbf{q}})\end{bmatrix} = \begin{bmatrix}\mathbf{0}\\ \tau\end{bmatrix} + \sum_{k=1}^{N_c}\begin{bmatrix}\mathbf{J}_{k,b}^{\top}\\ \mathbf{J}_{k,j}^{\top}\end{bmatrix}\mathbf{f}_k$$

where $\mathbf{h}(\mathbf{q},\dot{\mathbf{q}}) = \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$. The top 6 rows have identically zero direct motor authority ($\mathbf{S}_b^{\top}\tau = \mathbf{0}$). The base equation still contains gravity and coupling to joint acceleration, so the base can accelerate even in flight:

$$\mathbf{M}_{bb}(\mathbf{q})\ddot{\mathbf{q}}_b + \mathbf{M}_{bj}(\mathbf{q})\ddot{\mathbf{q}}_j + \mathbf{h}_b(\mathbf{q},\dot{\mathbf{q}}) = \sum_{k=1}^{N_c}\mathbf{J}_{k,b}(\mathbf{q})^{\top}\mathbf{f}_k$$

B.9.2 Centroidal momentum dynamics

Projecting the multi-body floating-base dynamics onto the robot's center of mass $\mathbf{c} \in \mathbb{R}^3$ yields the Newton-Euler **centroidal momentum equations** (Lynch and Park 2017):

$$\dot{\mathbf{h}}_{\text{lin}} = m\ddot{\mathbf{c}} = \sum_{k=1}^{N_c} \mathbf{f}_k + m\mathbf{g}_0$$

$$\dot{\mathbf{h}}_{\text{ang}} = \sum_{k=1}^{N_c} [(\mathbf{p}_k - \mathbf{c}) \times \mathbf{f}_k + \tau_k]$$

where $\mathbf{h}_{\text{lin}}$ is linear momentum, $\mathbf{h}_{\text{ang}}$ is angular momentum about the CoM, m is the total robot mass, $\mathbf{g}_0$ is the gravitational acceleration vector, and $(\mathbf{p}_k, \mathbf{f}_k, \tau_k)$ are the position, force, and moment at each active contact patch.[8]

[8] **Centroidal Momentum Matrix**: Centroidal dynamics summarize the external wrench needed for a proposed CoM and angular-momentum change. A controller still needs feasible contacts and actuator torque; omitting angular momentum can mispredict the required contact moments.

The total centroidal momentum $\mathbf{h}_G = [\mathbf{h}_{\text{lin}}^\top, \mathbf{h}_{\text{ang}}^\top]^\top \in \mathbb{R}^6$ linearly maps from generalized velocities through the **Centroidal Momentum Matrix (CMM)** $\mathbf{A}_G(\mathbf{q}) \in \mathbb{R}^{6\times(n+6)}$ (Lynch and Park 2017):

$$\mathbf{h}_G = \mathbf{A}_G(\mathbf{q})\dot{\mathbf{q}}$$

Differentiating with respect to time relates generalized accelerations to external contact wrenches:

$$\dot{\mathbf{h}}_G = \mathbf{A}_G(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{A}}_G(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \sum_{k=1}^{N_c} \mathbf{X}_k^* \mathbf{w}_k + \begin{bmatrix} m\mathbf{g}_0 \\ \mathbf{0} \end{bmatrix}$$

where $\mathbf{X}_k^*$ transforms contact wrench $\mathbf{w}_k = [\mathbf{f}_k^\top, \tau_k^\top]^\top$ to the center of mass frame.

B.9.3 Physical contact feasibility constraints

For nonadhesive support contacts with finite friction, admissible contact wrenches must satisfy three physical invariants:

1. **Unilateral Normal Force:** Feet can only push against the ground:

$$f_{k,\perp} = \mathbf{f}_k^\top \hat{\mathbf{n}}_k \geq 0$$

2. **Coulomb Friction Cone:** Tangential shear forces cannot exceed normal traction:

$$\|\mathbf{f}_{k,\parallel}\| = \|\mathbf{f}_k - f_{k,\perp}\hat{\mathbf{n}}_k\| \leq \mu f_{k,\perp}$$

3. **Support condition for coplanar contacts:** Under flat-ground assumptions, the Zero Moment Point (ZMP) should remain inside the effective support polygon with a case-specific inset:[9]

$$\mathbf{p}_{\text{ZMP}} \in \mathcal{P}_{\text{support}}, \qquad \mathcal{P}_{\text{support}} = \text{Conv}\left(\bigcup_k \mathcal{A}_k\right),$$

where $\mathcal{A}_k$ is each active contact patch projected onto that plane.

[9] **Zero Moment Point (ZMP)**: ZMP is useful for a specified coplanar support model. Edge contact, friction loss, and multi-height supports require more general wrench feasibility checks; crossing a modeled polygon edge does not by itself predict a particular fall.

B.9.4 Hierarchical whole-body control quadratic program

In floating-base humanoid and legged systems, high-level policies (such as learned vision-language-action models or action chunking policies) propose desired kinematic behaviors: end-effector manipulation paths, torso orientation, gaze directions, or swing-foot trajectories. However, because the 6-DoF floating base is unactuated, these kinematic proposals cannot be executed directly by joint motors. The **Hierarchical Whole-Body Control (WBC) Quadratic Program** acts as a real-time multi-task constrained optimization projection layer. It projects competing operational-space task accelerations $\ddot{\mathbf{x}}_i^{\text{des}}$ through a prioritized optimization hierarchy while strictly enforcing rigid-body Newton-Euler dynamics, friction cones, and actuator limits. In optimization and machine learning terms, Whole-Body Control solves the physical counterpart of multi-task learning with conflicting gradients:

Concept	Multi-Task Deep Learning (PCGrad / MGDA)	Hierarchical Whole-Body Control (WBC)
Task Specifications	Loss functions $\mathcal{L}_1(\theta), \dots, \mathcal{L}_K(\theta)$	Operational-space tasks $\mathbf{x}_1 = f_1(\mathbf{q}), \dots, \mathbf{x}_K = f_K(\mathbf{q})$
Shared Resource	Parameter weight vector $\theta \in \mathbb{R}^P$	Generalized joint accelerations $\ddot{\mathbf{q}} \in \mathbb{R}^{n+6}$ and torques τ
Task Conflict	Negative gradient inner product $\nabla \mathcal{L}_i^\top \nabla \mathcal{L}_j < 0$	Task accelerations that cannot be achieved simultaneously
Decoupling Rule	Projecting gradients onto tangent hyperplanes (PCGrad)	Null-space projection operator $\mathbf{N}_i = \mathbf{I} - \mathbf{J}_i^\dagger \mathbf{J}_i$
Priority Hierarchy	Heuristic loss weighting or GradNorm tuning	Cascaded optimization preserving higher-priority optima
Feasibility Enforcer	Parameter norm clipping / weight decay	Hard KKT constraints: dynamics, friction cones, torque saturation

B.9.4.1 Operational-space tasks and analytical null-space projection

Let task i be defined by a kinematics mapping $\mathbf{x}_i = f_i(\mathbf{q}) \in \mathbb{R}^{m_i}$ (e.g., end-effector Cartesian pose, center of mass position, or torso orientation). Differentiating twice yields the task acceleration:

$$\ddot{\mathbf{x}}_i = \mathbf{J}_i(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{J}}_i(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$$

A tracking controller generates desired task acceleration via proportional-derivative feedback:

$$\ddot{\mathbf{x}}_i^{\text{des}} = \ddot{\mathbf{x}}_i^{\text{ref}} + \mathbf{K}_{p,i}(\mathbf{x}_i^{\text{ref}} - \mathbf{x}_i) + \mathbf{K}_{d,i}(\dot{\mathbf{x}}_i^{\text{ref}} - \dot{\mathbf{x}}_i)$$

In complex humanoids, multiple tasks compete for limited degrees of freedom. Consider two prioritized tasks: Task 1 (highest priority, e.g., CoM balance and stance-foot ground contact) and Task 2 (secondary priority, e.g., end-effector trajectory tracking).

To ensure Task 2 never compromises Task 1, analytical Whole-Body Control projects Task 2 into the **kinematic null space** of Task 1. The unconstrained acceleration for Task 1 is:

$$\ddot{\mathbf{q}}_1^* = \mathbf{J}_1^\dagger \left(\ddot{\mathbf{x}}_1^{\text{des}} - \dot{\mathbf{J}}_1 \dot{\mathbf{q}} \right)$$

Assuming $\mathbf{J}_1$ has full row rank, $\mathbf{J}_1^\dagger = \mathbf{J}_1^\top (\mathbf{J}_1 \mathbf{J}_1^\top)^{-1}$ is the Moore-Penrose pseudo-inverse. The null-space projection matrix of Task 1 is:

$$\mathbf{N}_1 = \mathbf{I} - \mathbf{J}_1^\dagger \mathbf{J}_1$$

The projector satisfies the algebraic property:

$$\mathbf{J}_1 \mathbf{N}_1 = \mathbf{J}_1 \left(\mathbf{I} - \mathbf{J}_1^\dagger \mathbf{J}_1 \right) = \mathbf{J}_1 - \mathbf{J}_1 \mathbf{J}_1^\dagger \mathbf{J}_1 = \mathbf{0}$$

Projecting the secondary task command $\ddot{\mathbf{q}}_2$ through $\mathbf{N}_1$:

$$\ddot{\mathbf{q}}^* = \ddot{\mathbf{q}}_1^* + \mathbf{N}_1 \ddot{\mathbf{q}}_2^*$$

guarantees that Task 2 produces strictly zero acceleration in the direction of Task 1:

$$\mathbf{J}_1 \ddot{\mathbf{q}}^* = \mathbf{J}_1 \ddot{\mathbf{q}}_1^* + \mathbf{J}_1 \mathbf{N}_1 \ddot{\mathbf{q}}_2^* = \mathbf{J}_1 \ddot{\mathbf{q}}_1^* + \mathbf{0} = \ddot{\mathbf{x}}_1^{\text{des}} - \dot{\mathbf{J}}_1 \dot{\mathbf{q}}$$

Task 2 operates exclusively within the redundant internal degrees of freedom left unconstrained by Task 1.

B.9.4.2 Embedded whole-body control quadratic program

While analytical pseudo-inverse projections provide elegant decoupling, they cannot enforce inequality constraints such as unilateral contact forces, friction cones, and motor torque limits. Real-time controller firmware resolves this by formulating Whole-Body Control as a convex Quadratic Program whose execution deadline is part of the controller design.

Collect the decision variables $\mathbf{z} = [\ddot{\mathbf{q}}^\top, \mathbf{f}_c^\top, \tau^\top]^\top$, where $\ddot{\mathbf{q}} \in \mathbb{R}^{n+6}$ is generalized acceleration, $\mathbf{f}_c \in \mathbb{R}^{3N_c}$ collects contact forces, and $\tau \in \mathbb{R}^n$ is motor torque. A weighted task QP for a fixed set of sticking contacts then solves:

$$
\begin{aligned}
\min_{\ddot{\mathbf{q}}, \mathbf{f}_c, \tau} \quad & \sum_{i=1}^{K} w_i \|\mathbf{J}_i(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{J}}_i(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \ddot{\mathbf{x}}_i^{\text{des}}\|^2 + w_\tau \|\tau\|^2 + w_f \|\mathbf{f}_c - \mathbf{f}_{\text{nom}}\|^2 \\
\text{subject to:} \quad & \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{S}^\top \tau + \sum_{k=1}^{N_c} \mathbf{J}_{c_k}(\mathbf{q})^\top \mathbf{f}_k \\
& \mathbf{J}_{c_k}(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{J}}_{c_k}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \mathbf{0} \quad \forall k \in \{1, \dots, N_c\} \quad \text{(stationary sticking contacts)} \\
& \mathbf{F}_k \mathbf{f}_k \le \mathbf{0}, \quad f_{k,\perp} \ge 0 \quad \forall k \in \{1, \dots, N_c\} \quad \text{(friction pyramid)} \\
& -\tau_{\max} \le \tau \le \tau_{\max} \\
& \ddot{\mathbf{q}}_{\min}(\mathbf{q}, \dot{\mathbf{q}}) \le \ddot{\mathbf{q}} \le \ddot{\mathbf{q}}_{\max}(\mathbf{q}, \dot{\mathbf{q}})
\end{aligned}
$$

In this multi-task optimization architecture:

- **Dynamic Feasibility:** The equality constraint enforces the coupled multi-body floating-base equations of motion, including joint coupling, gravity, and the modeled external contact forces. The stance-contact equality prevents assigning a supporting force to a point that is simultaneously commanded to move away from the support.
- **Multi-Task Null-Space Hierarchy:** Finite objective weights w_i express a preference ($w_1 \gg w_2 \gg \cdots \gg w_K$). High-priority tasks (maintaining center-of-mass balance, stance-foot grounding, and trunk stability) dominate lower-priority goals (arm trajectory tracking or joint posture regularization toward nominal rest configurations). In strict hierarchical QP formulations, tasks are solved lexicographically in cascaded QPs, fixing the optimal residual of higher-priority tasks as hard constraints for subsequent stages.
- **Physical Contact and Torque Protection:** The linear inequalities enforce contact friction pyramids ($\mathbf{F}_k \mathbf{f}_k \le \mathbf{0}$) and motor saturation limits ($|\tau| \le \tau_{\max}$), restricting the solution to the chosen friction and torque model. Actual slip, electrical current, and joint-limit containment also depend on model error, sampling, and actuator response.
- **The Physical AI Synthesis:** Whole-Body Control serves as the real-time deterministic gatekeeper (section 12.1). High-level learned policies (such as Vision-Language-Action foundation models or diffusion trajectory planners) propose uncertified task goals $\mathbf{x}_i^{\text{ref}}$, while the WBC QP checks modeled task, contact, and torque feasibility before admission.

Here $\mathbf{F}_k$ describes the chosen inner approximation to the measured friction cone. The joint-acceleration bounds $\ddot{\mathbf{q}}_{\min}, \ddot{\mathbf{q}}_{\max}$ must be derived to preserve velocity and position limits over the integration step, with reserves for uncertainty. Finite task weights do not implement a strict hierarchy; cascaded QPs must preserve each higher-priority optimum as a constraint. The solve deadline must be validated with the selected contact count, solver, processor, friction estimate, and fallback. A feasible numerical solution does not itself prove that a foothold will support the load.

B.10 Passivity and Energy-Bounded Interaction

During teleoperation or stiff contact, transport delay can reduce feedback damping. **Passivity** evaluates a different property from tracking accuracy: whether a component can supply more energy through its ports than it has received or initially stored. This energy accounting is useful when connecting controllers to uncertain mechanical environments.

B.10.1 Energy storage and the passivity definition

At a mechanical interaction port, the inner product of effort and flow measures instantaneous power entering the plant. For a continuous-time dynamical system with state $\mathbf{x}(t)$, control input $\mathbf{u}(t)$ (an effort variable, such as force or torque **F**), and measured output $\mathbf{y}(t)$ (a flow variable, such as velocity **v**), the system is **passive** if there exists a continuously differentiable, non-negative storage function $V(\mathbf{x}) \geq 0$ (representing the total internal physical energy stored in mass momentum and structural compliance) such that:

$$\dot{V}(\mathbf{x}) \leq \mathbf{u}(t)^\top \mathbf{y}(t) - g(\mathbf{x}) \tag{B.2}$$

where $\mathbf{u}(t)^\top \mathbf{y}(t)$ represents instantaneous power injected into the system through the physical port, and $g(\mathbf{x}) \geq 0$ denotes internal energy dissipation (such as mechanical damping, viscous friction, or electrical resistance). Integrating equation B.2 over any operational duration $[0, T]$ yields the passivity energy inequality:

$$\int_0^T \mathbf{u}(t)^\top \mathbf{y}(t)\, dt \geq V(\mathbf{x}(T)) - V(\mathbf{x}(0)) + \int_0^T g(\mathbf{x}(t))\, dt \geq -V(\mathbf{x}(0)) \tag{B.3}$$

Under equation B.3 and the declared port orientation, the net physical energy extractable from this modeled component is strictly bounded by its initial stored energy $V(\mathbf{x}(0))$. A loaded mechanism can still release enough stored energy to injure or damage; passivity is an energy-bounding certificate, not an instantaneous contact-force limit.

Under a well-posed, power-conserving feedback interconnection, passive components admit a combined storage inequality. Stability conclusions additionally depend on the storage bounds and on strictness or detectability conditions; passivity alone does not guarantee every internal state converges. A human operator, powered tool, or active environment also cannot be assumed passive merely because it is connected to the robot.

B.10.2 The transport delay destabilization hazard

An untreated effort/flow communication channel need not remain passive when delayed. A fixed pure delay T_d contributes a frequency-dependent phase lag $\Delta\phi = -\omega T_d$ at angular frequency ω. In teleoperation or compliant interaction across wireless networks, late force feedback can arrive out of phase with the operator's current hand motion.

A negative instantaneous input power means the component is returning energy through its port. That is compatible with passivity when the energy was previously stored. The delayed link violates passivity only if its net delivered energy exceeds its incoming and initially stored energy under the declared port signs. Such excess can reduce damping or destabilize the coupled loop; the outcome depends on the controllers and mechanical environment.

B.10.3 Passivity preservation via wave variables

To prevent delay from injecting artificial energy into the teleoperation loop, Anderson and Spong (1989) introduced the **wave variable transformation**. Inspired by microwave transmission lines and scattering theory, wave variables map physical effort **F** and flow $\dot{\mathbf{x}}$ into incoming and outgoing wave channels with characteristic wave impedance $b > 0$:

$$\mathbf{u}_m(t) = \frac{\mathbf{F}_m(t) + b\dot{\mathbf{x}}_m(t)}{\sqrt{2b}}, \quad \mathbf{w}_m(t) = \frac{\mathbf{F}_m(t) - b\dot{\mathbf{x}}_m(t)}{\sqrt{2b}} \tag{B.4}$$

Computing instantaneous power from the wave transformation in equation B.4 reveals:

$$P_m(t) = \mathbf{F}_m(t)^\top \dot{\mathbf{x}}_m(t) = \frac{1}{2}\|\mathbf{u}_m(t)\|^2 - \frac{1}{2}\|\mathbf{w}_m(t)\|^2$$

In wave coordinates, power transmitted across the network port is the difference between incoming wave power $\frac{1}{2}\|\mathbf{u}_m\|^2$ and reflected wave power $\frac{1}{2}\|\mathbf{w}_m\|^2$. For an ideal transmission channel with communication delay T_d, transmitted waves are delayed: $\mathbf{u}_s(t) = \mathbf{u}_m(t - T_d)$ and $\mathbf{w}_m(t) = \mathbf{w}_s(t - T_d)$. The total energy stored in the communication channel in flight satisfies:

$$E_{\text{channel}}(t) = \frac{1}{2}\int_{t-T_d}^{t} \|\mathbf{u}_m(\sigma)\|^2 d\sigma + \frac{1}{2}\int_{t-T_d}^{t} \|\mathbf{w}_s(\sigma)\|^2 d\sigma \geq 0$$

Because the channel stores transmitted energy without amplifying wave amplitudes, the scattered communication link is passive and lossless under the stated constant-delay model; the delay line itself supplies no strict dissipation. Variable delay, packet loss, sampling, and saturation require additional analysis. Even the ideal passive channel does not bound spatial tracking errors; kinodynamic watchdog leases (section B.6.5) must be evaluated concurrently to guarantee spatial obstacle clearance.

B.11 Further Reading

For a deeper theoretical foundation in control theory, robotics, and dynamical systems, consult the following references:

- **Modern Robotics: Mechanics, Planning, and Control** (Lynch and Park 2017) provides a rigorous treatment of kinematics and dynamics using screw theory and Lie groups.
- **Robotics: Modelling, Planning and Control** (Siciliano et al. 2009) serves as a reference for multi-body robot dynamics, trajectory generation, and operational space control.
- **Feedback Systems: An Introduction for Scientists and Engineers** (Åström and Murray 2021) establishes classical and state-space control theory, frequency-domain stability margins, and state estimation from first principles.
- **Applied Nonlinear Control** (Slotine and Li 1991) provides a foundational treatment of nonlinear dynamical systems, Lyapunov stability certificates, and sliding mode control.
- **Control Barrier Functions: Theory and Applications** (Ames et al. 2019) sets out the formulation of control barrier functions, safety-critical quadratic programming filters, and forward invariance.
- **A Time-Dependent Hamilton-Jacobi Formulation of Reachable Sets for Continuous Dynamic Games** (Mitchell et al. 2005) establishes the level set method and viscosity solutions for continuous-state differential games and safety verification.
- **A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems** (Wabersich and Zeilinger 2021) develops predictive safety filters and constrained MPC methods for safe learning on physical systems.

C

Systems and Hardware

This appendix supplies reference models for the timing, memory, communication, and electrical limits that connect learned proposals to physical actuators. Its derivations and worked estimates help readers calculate response budgets, interpret hardware measurements, and implement the record formats used by the permission path. Each model states assumptions that must hold before its result can support a physical operating limit.

C.1 How to Use This Appendix

Consult this appendix when diagnosing physical systems bottlenecks, timing jitter, electrical rail instability, or hardware interface boundaries across chapters in this book.

When you encounter this systems symptom	Consult	For this quantitative tool
A real-time safety task misses periodic execution deadlines	section C.3	Rate-monotonic and EDF schedulability, Response Time Analysis (RTA), and Priority Ceiling Protocol
Neural policy inference shows non-deterministic execution times or long latency tails	section C.3	Execution-time analysis, static shape constraints, CUDA Graphs, and the limits of Extreme Value Theory (EVT) estimates
Real-time OS threads suffer latency spikes from page faults, cache bouncing, or IRQ preemption	section C.3	Memory locking (`mlockall`), stack prefaulting, 64-byte cache alignment, and top-half/threaded IRQ partitioning
A software deadlock or kernel panic freezes actuator control loops	section C.3	Windowed watchdog detection, rolling polynomial token checkpoints, and Safe Torque Off (STO) response
Inverter bridge MOSFETs burn out or trip overcurrent protections	section C.4	Microcontroller PWM complementary timers, dead-time insertion, Miller clamps, and 6th harmonic torque ripple
Heavy neural inference bursts induce compute brownout resets	section C.5	Inductive voltage droop ($L_{\text{parasitic}} dI/dt$), multi-tier decoupling hierarchies, and alpha-power gate delay
High-rate sensor DMA streams cause safety threads to miss deadlines	section C.6	DRAM bank-conflict penalties, row buffer thrashing, FR-FCFS starvation, and AXI QoS arbitration
Sensor fusion streams suffer temporal registration skew under motion	section C.7	IEEE 1588 / 802.1AS PTP hardware PHY timestamping, PI clock servos, and bounded clock conversion uncertainty
Shared memory queues between Linux and the safety microcontroller return torn or stale data	section C.8	Zero-copy IPC, POSIX shm, lockless ring buffers, triple-buffering, and CUDA IPC for multi-accelerator pipelines
Joint actuators fail to track sudden torque reversals, chatter, or overheat	section C.9	Stator $L_{\text{phase}}/R_{\text{phase}}$ constants, lumped thermal ODE networks, reflected rotor inertia, and analytical jerk limits
Emergency braking spikes DC bus voltage and damages power silicon	section C.10	Regenerative capacitive energy surges, dynamic brake choppers, and bus decoupling sizing
Shared serial buses or Ethernet networks experience queuing delay and frame jitter	section C.11	CAN 2.0B vs CAN FD priority blocking, EtherCAT on-the-fly cycle budgets, TSN Time-Aware Shapers, and seqlocks
Monocular 3D bounding boxes distort and depth covariance diverges	section C.12	Pinhole intrinsic projection, Zhang's calibration, and metric unprojection covariance
A mobile base breaches clearance buffers under surface wetness	section C.13	Kinematic stopping envelopes, information age lag breakdown, and velocity clamps

When you encounter this systems symptom	Consult	For this quantitative tool
A calculation needs a value of the running machine, or a record needs its canonical name	section C.14	The machine data sheet with evidence categories and MLSysIM fields, and the register of the sixteen records
A record must be laid out for a fixed-size mailbox or log	section C.15	Zero-allocation layouts, natural alignment, proposal/record byte tables, and register field orders
A handover, fault trial, or release gate must be implemented step by step	section C.15	Authority-transfer, fault-injection, and release-gate protocols beside their records
You need rapid order-of-magnitude physical anchors across time, kinetics, thermal, and risk	section C.2	Rate hierarchy, blind travel conversions, thermal constants, and exposure walls

C.2 Numbers to Know in Physical AI

Physical AI requires quantitative intuition across computation, motion, and heat. The latency hierarchy popularized by Jeff Dean[1] provides a useful starting point, but a delayed physical command also consumes clearance. The tables below separate representative rates and scenario calculations from bounds that require qualification on the actual machine.

[1] **Jeff Dean**: Google Senior Fellow and systems architect. His latency numbers originally presented with Peter Norvig around 2010 established the canonical latency hierarchy across storage, memory, and network tiers (Scott 2012). Physical AI extends this hierarchy down into electromechanical actuators, stopping kinematics, and thermal time constants.

The fastest events are transistor switching and local memory access. Operating-system dispatch and communication add scheduling and queueing delays. Actuator current then changes over an electrical time constant, while braking and heat accumulation unfold over longer intervals. These scales overlap; a task's assigned deadline comes from its physical consequence, not from its location in this hierarchy.

> **Systems Perspective C.1: Three physical numbers that matter most**
>
> - **Kinetic equivalence:** 1 **ms** $= 1$ **mm (at** 1 **m/s)**: At a nominal robotic speed of $1\ \mathrm{m/s}$, every millisecond of perception, inference, or fieldbus latency translates into exactly 1 mm of blind travel before retarding torque can be applied. At highway velocity ($30\ \mathrm{m/s}$), each millisecond is 3 cm. In physical systems, software latency is not waiting time; it is physical displacement.
> - **Thermal divergence:** $\tau_{\mathbf{th,silicon}} \sim 1$ **ms vs.** $\tau_{\mathbf{th}} \sim 100$ **s**: Semiconductor junctions heat adiabatically within milliseconds during deep neural network bursts, potentially triggering thermal throttling before heat spreaders react. Electrical rail droop is a separate response to changing current demand. In contrast, electromagnetic stator windings heat over tens to hundreds of seconds, tolerating transient overcurrent for aggressive maneuvers but accumulating thermal energy that eventually limits sustained torque.
> - **The exposure wall:** $T = 3/\lambda$: Under a stationary Poisson model, zero observed failures gives an approximate one-sided 95 percent upper rate bound of $3/T$ after T operational hours (Hanley and Lippman-Hand 1983). Very small target rates demand correspondingly large exposure. Runtime barriers can support a broader assurance argument, but neither a barrier nor a finite test proves the absence of failure.

C.2.1 Rate and latency hierarchy across machine levels

Control rates in physical AI rise steeply from the deliberative brain down to the physical body (table C.1). Deliberative multimodal models (VLAs) operate at human reaction scales (1–5 Hz). Action chunking models emit future trajectory waypoints at 10–50 Hz. Beneath the proposal boundary, the permission path enforces safety barriers at 1 kHz. At the lowest levels, power electronic current loops and PWM switching operate at tens to hundreds of kilohertz to maintain electromagnetic flux and drive torque. For real-time timer schedulability and response-time analysis, see section C.3; for inverter gate switching and dead-time modeling, see section C.4.

Table C.1: Rate and Latency Hierarchy Across the Five Machine Levels: Representative rates from inverter switching to foundation-model deliberation. The reciprocal period and travel during that period at nominal speed (1 m/s) are scale comparisons, not independently established deadlines or evidence-age bounds.

Machine Level	Frequency Band	Cycle Period	Blind Travel at 1 m/s	Architectural Mechanism & Bound
Brain (Deliberation)	1–5 Hz	200–1,000 ms	200–1,000 mm	Multimodal VLA reasoning; memory- and compute-bound
Brain (Action Chunking)	10–50 Hz	20–100 ms	20–100 mm	Action chunk diffusion/transformer decoding; amortized
Nervous System (Permission)	1 kHz	1 ms	1 mm	Control barrier certificates, stopping envelopes, lease watchdog
Body (Current Loop)	10–25 kHz	40–100 µs	40–100 µm	Field-oriented current control (FOC), torque tracking, stator flux
Body (Inverter Switching)	50–200 kHz	5–20 µs	5–20 µm	Power MOSFET gate drive, dead-time insertion (t_{dead}), inductive droop

C.2.2 Kinetic translation and safe stopping envelopes

Total stopping distance $d_{\text{stop}} = v \cdot \tau_{\text{delay}} + v^2/(2a_{\text{brake}})$ combines the linear blind travel accrued during computational delay and the quadratic mechanical braking distance (table C.2). As platform speed increases, absolute blind travel expands rapidly, consuming entire clearance margins before deceleration can begin. Section C.13 derives the information-age and braking terms; section C.9.3 relates joint jerk to electrical limits.

Table C.2: Kinetic Translation Across Platform Archetypes: Pre-brake delay converts into unrecoverable physical travel before braking force begins. These illustrative archetypes assume constant speed during delay and constant braking afterward; they are not the running machine's qualified profiles. Parenthesized percentages give blind travel as a share of the modeled total.

Platform Archetype	Nominal Velocity v	Braking Decel a_{brake}	Blind Travel ($\tau_{\text{delay}} = 50$ ms)	Blind Travel ($\tau_{\text{delay}} = 200$ ms)	Total Stop ($\tau_{\text{delay}} = 200$ ms)
Humanoid / Quadruped	1 m/s (3.6 km/h)	4 m/s² (0.41 g)	50 mm	200 mm	325 mm (61.5%)
Warehouse Mobile Manipulator	1.5 m/s (5.4 km/h)	2 m/s² (0.20 g)	75 mm	300 mm	863 mm (34.8%)
Urban Delivery Robot	6 m/s (21.6 km/h)	4.5 m/s² (0.46 g)	300 mm	1.2 m	5.2 m (23.1%)
Autonomous Vehicle (Highway)	30 m/s (108 km/h)	7 m/s² (0.71 g)	1.5 m	6 m	70.3 m (8.5%)

C.2.3 Edge memory bandwidth and action chunk amortization

For O operations and D_{vol} bytes transferred from memory, an idealized execution-time lower bound is $\max(O/R_{\text{peak}}, D_{\text{vol}}/\text{BW})$, where R_{peak} is peak arithmetic throughput and BW is available memory bandwidth. The equivalent roofline throughput bound is $\min(R_{\text{peak}}, (O/D_{\text{vol}})\text{BW})$. The ratio O/D_{vol} is arithmetic intensity. This model identifies whether compute or memory transfer binds; synchronization, contention, and incomplete overlap increase actual time.

At small batch sizes, autoregressive generation can be limited by memory bandwidth. Table C.3 divides weight storage by the assumed sustained bandwidth to obtain a streaming-only lower bound on execution time. A chunk decoder that produces all H waypoints in one weight pass can amortize that transfer; one that makes multiple denoising or token-generation passes cannot claim the same amortization. The resulting waypoint rate is an output-throughput ceiling, not a fresh-observation feedback rate. Section C.6 explains how competing traffic reduces the bandwidth available to the decoder.

Table C.3: Edge Memory Bandwidth and Action Chunk Amortization: Weight-only streaming estimates under the stated bandwidth and precision assumptions. The chunk column assumes one weight pass produces the whole chunk; activations, decoder passes, and contention add work. More waypoints do not imply more frequent observation or replanning.

Model & Precision	Parameters	Memory Footprint	DRAM Read Time (140 GB/s)	Single-Step Max Rate	Chunk Rate ($H = 16$)
VLA-7B (FP16)	7.0B	14 GB	100 ms	10 Hz	160 Hz equivalent
VLA-7B (INT8)	7.0B	7 GB	50 ms	20 Hz	320 Hz equivalent
VLA-7B (INT4)	7.0B	3.5 GB	25 ms	40 Hz	640 Hz equivalent
VLA-14B (INT8)	14.0B	14 GB	100 ms	10 Hz	160 Hz equivalent

C.2.4 Thermal time constants across compute and electromechanics

For a constant heat input and a single thermal node initially at ambient, the temperature rise above T_{amb} follows $\Delta T(t) = \Delta T_{\max}(1 - e^{-t/\tau_{\text{th}}})$ with steady-state rise $\Delta T_{\max}$ and time constant τ_{th} (table C.4). Local silicon temperature can change rapidly during a computational burst, requiring thermal qualification of the workload and cooling path. Electrical droop has a separate current-transient timescale. In contrast, electromagnetic stator windings heat over minutes, allowing transient torque overdrive if bounded by an $I^2_{\text{phase}}t$ thermal watchdog. For first-order thermal differential equations and stator winding degradation limits, see section C.9; for regenerative energy absorption on DC power buses, see section C.10.

Table C.4: Thermal Time Constants Across Compute and Electromechanical Subsystems: Thermal response spans several orders of magnitude. The listed ranges are representative scales, not universal device limits. Heating rates, thermal trips, and sustained torque require the actual thermal network and initial temperature.

Subsystem Component	Physical Mechanism	Thermal Time Constant τ_{th}	Critical Failure Mode	Safety Mitigation
Silicon Junction (Die)	Transistor channel self-heating	1–10 ms	Gate delay change, leakage rise, thermal trip	Thermal monitoring and frequency throttling
SoC Heat Sink / Package	Convective heat spreader dissipation	10–60 s	Package thermal saturation, sustained throttling	Active fan modulation, workload migration
Motor Stator Windings	Copper resistive Joule heating ($I^2_{\text{phase}}R_{\text{phase}}$)	30–180 s	Insulation breakdown (Class H 180°C), short	$I^2_{\text{phase}}t$ thermal protection, current derating
Motor Frame / Housing	Bulk metallic casing heat conduction	10–30 min	Structural deformation, bearing grease loss	Natural convection, passive cooling fins
Traction Battery Pack	Electrochemical cell internal resistance	5–20 min	Chemistry-dependent overheating and thermal runaway	Liquid cooling loop, charge/discharge cutoff

C.2.5 Statistical safety bounds and exposure walls

For zero observed failures in stationary Poisson exposure, the one-sided 95 percent upper rate bound is approximately $3/T$, where T is total operating time (Hanley and Lippman-Hand 1983). Table C.5 shows the exposure needed to put that bound at three illustrative target rates. These are statistical calculations, not a mapping from a whole-system accident rate to a safety integrity level. In particular, ISO 26262 random-hardware-failure metrics do not describe every cause of a learned system's failure (International Organization for Standardization 2018). Correlated trials, changing operating conditions, and unobserved failures invalidate a naive accumulation of hours.

Table C.5: Statistical Testing Exposure: Approximate exposure for a one-sided 95 percent upper rate bound after zero observed failures. A system assurance argument also needs evidence about failure mechanisms and operating conditions.

Illustrative target rate λ	Required failure-free exposure $T \approx 3/\lambda$	Interpretation
10^{-8}/hour	300,000,000 hours	Upper rate bound under the declared Poisson model
10^{-7}/hour	30,000,000 hours	Same confidence level and observation assumptions
10^{-6}/hour	3,000,000 hours	A rate alone does not establish a safety classification

C.2.6 Fieldbus latencies, jitter, and determinism

Communication buses differ sharply in latency and jitter (table C.6). Whereas distributed cluster fabrics (such as RoCEv2 or InfiniBand) prioritize aggregate bandwidth using credit-based flow control and statistical multiplexing, onboard physical AI buses must guarantee bounded delivery times. Industrial fieldbuses such as EtherCAT combine hardware forwarding with scheduled exchange and clock synchronization. A specific jitter claim requires the configured system's evidence. Classical serial buses like Controller Area Network (CAN) 2.0B provide low-overhead bitwise arbitration for up to 8 bytes, but saturate rapidly under multi-joint control traffic. The Flexible Data-Rate variant, CAN FD, expands payloads to 64 bytes with accelerated data bit rates. Meanwhile, switched IEEE Time-Sensitive Networking (TSN) uses scheduled transmission gates to multiplex gigabit vision streams with hard real-time actuator commands. In contrast, non-deterministic best-effort Ethernet and shared wireless channels introduce tail latencies that exceed the reaction deadlines of high-speed electromechanical machines. For IEEE 1588 PTP hardware timestamping bounds, see section C.7; for CAN 2.0B vs. CAN FD priority arbitration and EtherCAT cycle budgets, see section C.11.

DRAFT

Table C.6: Communication Bus Latencies, Jitter, and Real-Time Determinism: Representative scales for configured networks, not protocol-level worst-case guarantees. Required jitter follows the plant and deadline; bounded control does not universally require sub-microsecond jitter. Delivery guarantees require a traffic model, topology, arbitration analysis, and measured implementation behavior.

Bus / Network Technology	Nominal Transit Latency	Illustrative Jitter	Determinism Mechanism	Primary Machine Role
EtherCAT (ETG.1000)	100 µs–1 ms	< 1 µs	Hardware cut-through on-the-fly frame processing	Synchronous joint servo drive & sensor feedback (Body/Nervous)
CAN FD (ISO 11898-1)	100–500 µs	50–200 µs	Bitwise non-destructive priority arbitration (64 B)	Distributed sensors, multi-axis actuator controllers (Nervous/Body)
CAN 2.0B (ISO 11898-1)	200 µs–1 ms	100–400 µs	Bitwise non-destructive priority arbitration (8 B)	Legacy sensors, low-rate auxiliary controllers (Nervous/Body)
TSN Ethernet (802.1Qbv)	10–100 µs	< 5 µs	Time-aware shaper with scheduled transmission gates	High-bandwidth sensor ingress (lidar/cameras) to SoC (Boundary)
Standard IP / UDP Ethernet	0.5–5 ms	1–20 ms	Switched full-duplex links; best-effort queueing	Asynchronous logging, inter-node telemetry (Brain)
Shared Wi-Fi telemetry	5–20 ms	10–500 ms (RF fade)	Wireless packet retransmission, contention window	High-level fleet dispatch, cloud mission updates (World/Governance)

C.3 Real-Time Timers, Watchdogs, and Schedulability

A real-time task is correct only if it computes the required result before its deadline. Average inference latency and throughput do not establish that property. A permission task needs a bound that includes its own execution, blocking on shared resources, and interference from other work.

The following single-core scheduling models separate those contributions. Their guarantees apply only under their stated task assumptions; operating-system configuration and loaded measurements must establish whether the implementation satisfies them.

C.3.1 Real-time task schedulability under rate-monotonic and earliest-deadline-first scheduling

In contrast to general-purpose Linux time-sharing, real-time operating systems schedule tasks based on deterministic timing contracts rather than dynamic heuristics. A periodic real-time task i is characterized by its worst-case execution time (WCET) C_i, relative deadline D_i, and period T_i. When multiple real-time tasks share a single core, schedulability depends on the priority assignment discipline:

Under **Rate-Monotonic Scheduling (RMS)**, fixed-priority preemptive scheduling assigns priority inversely proportional to task period ($T_i < T_j \implies \text{priority}(i) > \text{priority}(j)$) (Liu and Layland 1973). For a set of n independent periodic tasks, with negligible scheduling overhead, no blocking or release jitter, and implicit deadlines ($D_i = T_i$), a sufficient condition for schedulability is the Liu and Layland utilization bound:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right)$$

As $n \to \infty$, the bound converges monotonically to $\lim_{n\to\infty} n(2^{1/n} - 1) = \ln 2 \approx 0.693$ (69.3 percent). If total processor utilization does not exceed 69.3 percent, the task set is guaranteed to meet all deadlines under RMS. Task periods may instead form a harmonic set, in which every period is an exact integer multiple of every shorter period ($T_j = m \cdot T_i$ for integer $m \geq 1$, such as 1 ms, 5 ms, 10 ms, and 100 ms). For a harmonic set, the utilization bound rises to 100 percent. In physical AI architectures, rate hierarchies are purposefully engineered with harmonic cadences to maximize processor utilization without sacrificing determinism.

When processor utilization exceeds the conservative Liu and Layland bound ($U > \ln 2$), or when relative deadlines are constrained ($D_i \leq T_i$), **Response Time Analysis (RTA)** evaluates exact schedulability from the worst-case **response time** R_i (Liu and Layland 1973; Kopetz 2011):

$$R_i = C_i + B_i + I_i \leq D_i$$

where B_i is the maximum blocking duration induced by lower-priority tasks holding shared resources, and I_i is the interference from higher-priority tasks. The response time is computed via the fixed-point recurrence. The sum runs over the higher-priority task set. The ceiling counts how many releases of each such task can interfere during the current response-time estimate. An enlarged estimate may expose another release and therefore require another iteration:

$$R_i^{(\ell+1)} = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(\ell)}}{T_j} \right\rceil C_j$$

For fixed priorities, bounded blocking, no self-suspension, and zero release jitter, start with $R_i^{(0)} = C_i + B_i$ and iterate until $R_i^{(\ell+1)} = R_i^{(\ell)} \leq D_i$ (establishing schedulability) or $R_i^{(\ell+1)} > D_i$ (signaling deadline violation). See section 4.5.3 and section 4.5.1 for the role of response time variance in the nervous system.

In dynamic-priority systems, **Earliest Deadline First (EDF)** assigns highest priority to the active task with the closest absolute deadline $d_{i,k} = r_{i,k} + D_i$. For independent jobs on a single-core preemptive processor with no blocking and negligible scheduling overhead, EDF is optimal: if a task set is schedulable by any scheduling algorithm, it is schedulable by EDF (Liu and Layland 1973). For periodic tasks with $D_i = T_i$, EDF guarantees schedulability if and only if:

$$\sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$$

achieving 100 percent theoretical utilization without requiring harmonic periods. When relative deadlines are strictly less than periods ($D_i < T_i$), schedulability requires the processor demand criterion over the Demand Bound Function (DBF):

$$\text{dbf}(t) = \sum_{i=1}^{n} \max\left(0, \left\lfloor \frac{t - D_i}{T_i} \right\rfloor + 1\right) C_i \leq t, \quad \forall t \geq 0$$

evaluating whether cumulative execution demand exceeds available physical time across all time intervals t. Under Linux, EDF is exposed via the `SCHED_DEADLINE` policy implementing the Constant Bandwidth Server (CBS) algorithm. A CBS task declares its runtime Q_i, deadline D_i, and period T_i via `sched_setattr()`. If a task attempts to execute beyond its declared runtime Q_i within period T_i, the kernel dynamically throttles the task until its next period, protecting co-located real-time threads from execution overruns.

In standard multi-threaded Linux, mutual exclusion primitives (such as futex-backed mutexes) assume threads will eventually yield the processor. When real-time tasks share data structures (such as seqlock mailboxes or actuator command queues) protected by naive mutual exclusion, they suffer from **unbounded priority inversion** (Sha et al. 1990; Reeves 1997). A low-priority task L acquires a lock; a high-priority task H becomes ready and blocks waiting for the lock; intermediate-priority tasks M that do not need the lock preempt task L, delaying task H for an arbitrary duration. To bound blocking time B_i in the RTA recurrence, real-time operating systems use formal synchronization protocols:

1. **Priority Inheritance Protocol (PIP)**: When task H blocks on a lock held by task L, the operating system temporarily elevates task L's active priority to task H's priority, preventing preemption by task M. Under `PREEMPT_RT` Linux, standard in-kernel spinlocks are converted to sleeping `rt_mutex` locks that enforce PIP automatically, while user-space threads configure `PTHREAD_PRIO_INHERIT`. While PIP eliminates unbounded inversion, it permits transitive blocking chains ($B_i \leq \sum_k C_k$) and potential deadlock.
2. **Priority Ceiling Protocol (PCP)**: Each lock is assigned a static priority ceiling equal to the highest priority of any task that may lock it. A task i can lock a resource only if its priority is strictly higher than the priority ceilings of all locks currently held by other tasks. PCP prevents mutual-exclusion deadlocks and guarantees that a task is blocked at most once by at most one lower-priority critical section (Sha et al. 1990):

   $$B_i = \max_{k \in lp(i), s \in cs(k)} {C^{\text{cs}}}_{k,s}$$

   where $lp(i)$ is the set of tasks with priority lower than task i, and $cs(k)$ is the set of critical sections executed by task k with a priority ceiling at or above the priority of task i.

C.3.2 Worst-case execution time analysis for physical ML pipelines

In classical embedded systems, the worst-case execution time C_i is determined via Static Timing Analysis (STA) (Hennessy and Patterson 2019). STA parses binary machine instructions, constructs a Control Flow Graph (CFG), and solves an Integer Linear Program via the Implicit Path Enumeration Technique (IPET) using cycle-accurate microarchitectural models of processor pipelines, branch predictors, and cache sets. In physical AI systems, evaluating C_i for learned vision-language-action (VLA) and diffusion policies presents three distinct architectural challenges:

1. **Microarchitectural opacity**: Modern neural accelerators (GPUs, NPUs, Tensor Processing Unit (TPU) hardware) feature multi-level shared caches, dynamic frequency scaling (DVFS), out-of-order warp schedulers, and memory crossbar contention that make a sound static bound difficult without detailed timing and interference models.
2. **Algorithmic execution variance**: Learned pipelines follow dynamic execution paths, including adaptive denoising loops in diffusion models, variable sequence lengths in autoregressive decoders, and data-dependent sparse attention kernels.
3. **Software runtime overhead**: Deep learning runtimes (e.g., PyTorch, TensorRT, Open Neural Network Exchange (ONNX) Runtime) introduce non-deterministic host-to-device kernel launch queueing, unified memory migration, and internal memory allocation locks. Launch

overhead and jitter depend on the runtime, driver, submission path, and load; a per-kernel constant is not portable across systems.

To constrain execution variability for neural policies that cross the proposal boundary, physical AI systems enforce deterministic execution contracts and hybrid measurement bounds:

- **Static shape contracts**: Action chunk horizons (H), token context lengths, and sensor input tensors are locked to fixed maximum dimensions. Dynamic input padding replaces variable-length tensors, ensuring constant compute graph geometry and eliminating dynamic kernel recompilation.
- **Zero runtime allocation**: All execution buffers, KV caches, scratchpad memory, and direct memory access (DMA) descriptors are preallocated during initialization. Dynamic memory calls (`malloc`, `cudaMalloc`, `free`) are strictly forbidden on cyclic control paths.
- **Reduced host dispatch overhead via CUDA Graphs**: Fixing denoising iterations bounds one source of algorithmic work. Capturing and instantiating a reusable graph reduces repeated per-kernel host setup, as the CUDA Graphs documentation explains. Graph launch still uses the runtime and driver, and device scheduling, frequency changes, and shared-resource contention remain. A graph does not by itself provide deterministic execution.
- **Extreme Value Theory (EVT) tail bounds**: Because finite empirical profiling cannot expose the true maximum execution time under worst-case hardware interference, engineers apply Extreme Value Theory (Coles 2001). Under the Pickands–Balkema–de Haan theorem, execution time exceedances over a high operational threshold u converge asymptotically to a Generalized Pareto Distribution (GPD):

$$F_{\text{GPD}}(y) = 1 - \left(1 + \frac{\xi y}{\tilde{\sigma}}\right)^{-1/\xi}$$

on the support $1+\xi y/\tilde{\sigma} > 0$, with the exponential limit at $\xi = 0$. Here $y = x - u > 0$, $\tilde{\sigma} > 0$ is the scale parameter, and ξ is the shape parameter (tail index). The shape parameter characterizes the tail behavior:

  - $\xi < 0$ (Weibull domain): Bounded upper tail with a finite model endpoint $x_F = u - \tilde{\sigma}/\xi$, under the fitted model; an estimated finite endpoint is not proof of a hardware ceiling.
  - $\xi = 0$ (Gumbel domain): Light, exponential tail.
  - $\xi > 0$ (Fréchet domain): Heavy, polynomial tail. The fitted sign alone neither identifies a hardware failure mechanism nor determines whether a deadline can be defended.

For an acceptable target exceedance probability ϵ (such as 10^{-6} or 10^{-8} per invocation), a model-based execution-time quantile $z_\epsilon(\epsilon)$ is computed as:

$$z_\epsilon(\epsilon) = u + \frac{\tilde{\sigma}}{\xi}\left[\left(\frac{N}{N_u}\epsilon\right)^{-\xi} - 1\right]$$

where N is total sample count and N_u is the count of observations exceeding threshold u. For $\xi = 0$, the continuous limit is $u - \tilde{\sigma}\ln(N\epsilon/N_u)$. These are fitted quantiles, conditional on the sampling regime, threshold choice, tail model, and parameter uncertainty. They do not certify a deterministic WCET. A placement record must distinguish measured maxima, probabilistic estimates, and justified execution bounds; the permission path must remain schedulable when the proposer exceeds its estimate.

C.3.3 Operating-system determinism through interrupt handling, memory locking, and cache locality

On a heterogeneous robotics system on chip (SoC), the nervous system and safety monitors execute under real-time operating systems (RTOS) or real-time Linux kernels (such as Linux patched with `PREEMPT_RT`). Interrupts and non-preemptible kernel critical sections can delay user-space tasks. A hard response bound requires bounding those intervals on the chosen kernel and hardware. In physical AI pipelines where control loops govern physical momentum, operating-system determinism requires eliminating non-preemptible kernel sections, bounding interrupt preemption latency, isolating execution cores, locking virtual memory, and enforcing cache locality.

Linux fair scheduling balances CPU service among runnable tasks. Its historical CFS implementation accounts for weighted execution time; newer kernels use EEVDF, which also considers eligibility and virtual deadlines. Neither policy establishes the physical deadline guarantee needed by the permission task. In physical AI systems, real-time control threads must instead be configured with POSIX static priority scheduling policies (`SCHED_FIFO` or `SCHED_RR`) across priority levels 1 to 99 (or `SCHED_DEADLINE` implementing Earliest Deadline First). Under `SCHED_FIFO`, the highest-priority runnable thread executes immediately and runs until it blocks on I/O, yields voluntarily, or is preempted by a strictly higher-priority real-time thread.

In mainline Linux kernels, disabling preemption during spinlock acquisition (`preempt_disable()`), executing non-preemptible softirqs, and servicing asynchronous workqueues can delay user-space execution by tens of milliseconds. When an unprivileged machine learning process or device driver holds an in-kernel spinlock, a high-priority 1 kHz real-time safety task cannot preempt the CPU, even if its timer has expired. A kernel configured with `PREEMPT_RT` reduces non-preemptible execution through four mechanisms, while hardware, drivers, and remaining critical sections still require timing qualification:

1. *Sleeping Spinlocks (`rt_mutex`)*: Standard in-kernel spinlocks (`spinlock_t` and `rwlock_t`) are replaced by sleeping mutexes backed by `rt_mutex` with Priority Inheritance Protocol (PIP) support. When a high-priority thread waits for an `rt_mutex`, the waiter sleeps and the lock owner inherits its priority until it can release the lock. Raw spinlocks remain non-preemptible; interrupt masking depends on the primitive used. Their maximum hold times must be included in the platform timing analysis.
2. *Preemptible Read-Copy Update (`CONFIG_PREEMPT_RCU`)*: RCU read-side critical sections (`rcu_read_lock()` / `rcu_read_unlock()`) become preemptible, preventing deferred memory reclamation from stalling real-time threads.
3. *Forced Interrupt Threading*: Hardware interrupt service routines (ISRs) are split into minimal top-half handlers and prioritized kernel threads (`irq/<irq>-<name>`) scheduled under `SCHED_FIFO`.
4. *High-Resolution Timers (`hrtimers`)*: Traditional Linux timer ticks execute at coarse periodic jiffies (1 ms or 4 ms granularity), introducing substantial quantization error and timer slack. `PREEMPT_RT` binds the timer subsystem to hardware per-CPU clock event devices (such as the ARM Generic Timer or x86 Local APIC), enabling sub-microsecond timer resolution. Real-time cyclic threads invoke `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next_period, NULL)` using absolute monotonic deadlines, avoiding cumulative drift from relative sleeps. Wake-up jitter remains part of the response-time bound.

When an external hardware peripheral (such as an IMU, joint encoder counter, or fieldbus controller) asserts an interrupt request (IRQ), the processor halts user execution and saves register context. It then vectors to the hardware ISR via the interrupt controller (such as an ARM Generic Interrupt Controller GICv3 or x86 APIC). The worst-case interrupt latency $t_{\text{irq,lat}}$ to enter the hardware interrupt handler can be budgeted as:

$$t_{\text{irq,lat}} = t_{\text{irq_disabled,max}} + t_{\text{hw_pipe}} + t_{\text{context_save}} + \sum_{j \in hp(\text{irq})} t_{\text{ISR}_j}$$

This expression does not include threaded-handler work or the subsequent scheduling delay before a user-space task runs. Here $t_{\text{irq_disabled,max}}$ is the longest duration during which CPU interrupts were disabled across the kernel, $t_{\text{hw_pipe}}$ is the processor pipeline flush delay, $t_{\text{context_save}}$ is the hardware register preservation time, and $\sum t_{\text{ISR}_j}$ represents the execution duration of higher-priority ISRs.

To prevent high-bandwidth sensor streams (such as GigE Vision cameras or 3D lidars streaming gigabits of packet traffic) from generating an *interrupt storm* that starves the 1 kHz permission path, real-time architectures strictly enforce top-half and bottom-half partitioning:

- **Top-half (Hardware ISR)**: Executes in hard-interrupt context with CPU interrupts masked. It executes the absolute minimum hardware operations: reading the peripheral interrupt status register, clearing the physical interrupt line, latching high-resolution hardware counter timestamps, and waking the associated threaded IRQ. The handler's maximum execution time must be bounded on the target.

- **Bottom-half (Threaded IRQ)**: In `PREEMPT_RT` and RTOS environments, complex packet deserialization, frame CRC verification, and data copies are deferred to dedicated kernel threads (`irq/<irq>-<name>`). By configuring these threaded IRQs under `SCHED_FIFO` with priorities lower than the 1 kHz safety monitor (e.g., setting camera IRQ threads to priority 50 and the motor permission thread to priority 90), the scheduler prevents those lower-priority threads from preempting the control task. Hard-interrupt top halves and shared-memory traffic still contribute interference.

Heterogeneous multi-core robotics SoCs must also isolate deterministic control threads from background operating-system disturbances. Kernel boot parameters and memory layout rules establish strict CPU shielding and cache locality:

- `isolcpus=2,3`: Removes cores 2 and 3 from the general kernel scheduler balancing domain, reducing automatic load balancing onto those cores; explicit affinity, cpusets, and kernel work still require configuration.
- `nohz_full=2,3`: Enables full dynamic tickless operation. When an isolated core runs a single real-time thread, the kernel can suppress the periodic scheduling timer tick (100–1000 Hz), removing that source of periodic interruption while other interrupts and scheduling delays remain.
- `rcu_nocbs=2,3`: Offloads RCU callback processing from isolated cores to housekeeping cores (cores 0 and 1).
- *IRQ Affinity Partitioning*: Peripheral hardware interrupts are directed away from real-time cores by writing CPU bitmasks to `/proc/irq/<id>/smp_affinity`. High-rate camera MSI-X vectors and Ethernet controller interrupts are pinned exclusively to housekeeping cores, reducing controllable interrupt interference. Nonmaskable events, local timers, and platform firmware disturbances still require accounting.
- *Cache Line Alignment and False Sharing Elimination*: Cache-line size is architecture dependent; the examples here assume 64-byte lines. When multiple CPU cores access independent variables that reside within the same 64-byte line, hardware cache coherency protocols (MESI/MOESI) repeatedly invalidate the line across core private caches upon every store, a phenomenon known as **false sharing** or *cache line bouncing*. A single false sharing interaction can stall a real-time core for 50 to 100 clock cycles. Separating independently written state into aligned, padded cache-line slots avoids false sharing; merely aligning the beginning of a larger structure does not isolate every member. The target's remaining cache misses and contention still need timing bounds. Furthermore, real-time control paths pre-warm L1 instruction and data caches during initialization by executing a dummy pass over critical control tables before arming actuators.

Virtual memory subsystems rely on demand paging, so deterministic execution also requires locking virtual memory and eliminating page faults. When a process accesses an unmapped virtual address, the CPU raises an architectural page fault exception (`#PF`) and suspends execution. The kernel page-table handler then allocates a physical page frame, updates translation lookaside buffer (TLB) entries, and zeroes the memory (a *minor page fault*, requiring 10–50 µs). If the requested page was swapped to secondary flash storage, the thread blocks on disk I/O (a *major page fault*, requiring 10–50 ms). Furthermore, Transparent Huge Pages (THP) memory compaction (`khugepaged`) can asynchronously stall threads for tens of milliseconds while defragmenting physical memory pages. A page fault can consume the remaining response-time margin; its mere occurrence does not imply that every deadline is missed.

The following Linux/glibc sketch illustrates initialization and absolute-period sleeping. It assumes the shown page and cache-line sizes. Production code must also validate limits and privileges, measure completion against the deadline, and enter the assessed fallback on an overrun. A late absolute sleep can return immediately; it must not trigger an uncontrolled series of catch-up actuator writes.

```
// Real-time process initialization, memory locking, and cyclic execution
#define _GNU_SOURCE
#include <sys/mman.h>
```

```c
#include <sched.h>
#include <pthread.h>
#include <malloc.h>
#include <time.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <errno.h>

#define REALTIME_PRIORITY   90
#define CONTROL_CPU_CORE    2
#define STACK_PREFAULT_SZ   (256 * 1024)        // 256 KB stack
#define HEAP_PREALLOC_SZ    (16 * 1024 * 1024) // 16 MB pre-allocated arena
#define PERIOD_NS           1000000ULL          // 1 ms period (1 kHz)

// 64-byte cache-aligned telemetry slot to prevent false sharing
typedef struct __attribute__((aligned(64))) {
    uint64_t sequence_id;
    uint64_t timestamp_ns;
    float joint_torques[8];
    uint32_t status_flags;
    uint8_t reserved[12];
} realtime_telemetry_t;

static realtime_telemetry_t g_telemetry;
static volatile uint8_t *g_realtime_heap_arena = NULL;

void configure_realtime_process(int target_cpu, int rt_priority, size_t stack_size) {
    // 1. Bind execution to dedicated isolated CPU core (isolcpus / nohz_full)
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(target_cpu, &cpuset);
    if (sched_setaffinity(0, sizeof(cpu_set_t), &cpuset) != 0) {
        perror("sched_setaffinity failed");
        exit(EXIT_FAILURE);
    }

    // 2. Configure POSIX static real-time priority (SCHED_FIFO)
    struct sched_param sp = { .sched_priority = rt_priority };
    if (sched_setscheduler(0, SCHED_FIFO, &sp) != 0) {
        perror("sched_setscheduler SCHED_FIFO failed");
        exit(EXIT_FAILURE);
    }

    // 3. Lock all current and future virtual address mappings into physical RAM
    if (mlockall(MCL_CURRENT | MCL_FUTURE) != 0) {
        perror("mlockall failed");
        exit(EXIT_FAILURE);
    }

    // 4. Disable glibc dynamic heap trimming and anonymous mmap allocation
    mallopt(M_TRIM_THRESHOLD, -1);
    mallopt(M_MMAP_MAX, 0);

    // 5. Pre-allocate and prefault the dedicated heap arena
    g_realtime_heap_arena = (uint8_t *)malloc(HEAP_PREALLOC_SZ);
    if (!g_realtime_heap_arena) {
        perror("malloc heap arena failed");
        exit(EXIT_FAILURE);
    }
    // Touch every 4 KB page to force physical page frame allocation and TLB population
    for (size_t i = 0; i < HEAP_PREALLOC_SZ; i += 4096) {
        g_realtime_heap_arena[i] = 0;
    }
```

```
    // 6. Prefault the thread execution stack to its declared maximum depth
    volatile char stack_buffer[stack_size];
    for (size_t i = 0; i < stack_size; i += 4096) {
        stack_buffer[i] = 0; // Trigger minor page fault and populate TLB entry
    }
}

void run_periodic_control_loop(void) {
    struct timespec next_period;
    clock_gettime(CLOCK_MONOTONIC, &next_period);

    // A real implementation warms the actual control code and data before arming.

    while (1) {
        // Compute next absolute wake epoch (eliminates cumulative drift)
        next_period.tv_nsec += PERIOD_NS;
        while (next_period.tv_nsec >= 1000000000L) {
            next_period.tv_nsec -= 1000000000L;
            next_period.tv_sec  += 1;
        }

        // High-resolution absolute sleep
        int ret;
        do {
            ret = clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next_period, NULL);
        } while (ret == EINTR);
        if (ret != 0) {
            fprintf(stderr, "clock_nanosleep: %s\n", strerror(ret));
            break;
        }

        // ZERO-ALLOCATION CYCLIC CONTROL LOGIC
        // 1. Ingest sensor state from lock-free shared memory
        // 2. Evaluate Control Barrier Functions (CBFs)
        // 3. Dispatch admitted setpoints to EtherCAT master
        g_telemetry.sequence_id++;
    }
}
```

Beyond address space locking and stack prefaulting, the cyclic 1 kHz control loop operates under a strict **zero dynamic allocation invariant**. In general-purpose systems programming, developers routinely rely on dynamic memory managers (`malloc`, `free`, C++ `new`, `delete`) or standard container expansions (`std::vector::push_back`). In a real-time control loop, however, dynamic allocation introduces three catastrophic failure modes:

1. *Arena Mutex Contention*: Modern allocators (such as `ptmalloc` in `glibc`) partition memory into multiple heap arenas protected by mutual-exclusion mutexes. If another thread in the same process acquires an arena lock, a high-priority 1 kHz real-time control thread calling `malloc` blocks on that mutex, suffering from priority inversion and waiting milliseconds.
2. *Kernel Syscall Invocations*: When an allocator exhausts its active arena chunk, it issues `brk()` or anonymous `mmap()` system calls. These kernel entries acquire the process's `mmap_lock` (or `mmap_sem`), which is shared with background kernel threads such as `khugepaged` performing transparent huge page compaction or memory defragmentation, stalling user-space execution for tens of milliseconds.
3. *Heap Fragmentation*: Repeated cyclic allocations and deallocations fragment the heap, degrading allocation search times across segregated free lists into non-deterministic search loops.

Consequently, all message structures, matrix workspace buffers, trajectory buffers, and seqlock mailbox slots must be statically allocated at startup or drawn from fixed-size, lock-free memory pools (such as slab allocators). Furthermore, `mlockall(MCL_FUTURE)` must be paired with explicit memory pre-allocation and page prefaulting. Linux uses optimistic memory allocation (overcommit), so virtual address ranges returned by `malloc()` are not backed by physical page frames until the thread actually writes to each 4 KB page. Touching an uninitialized buffer for the first time inside the

control loop triggers a minor page fault exception (`#PF`). The kernel page-fault handler must then allocate physical memory, zero the page, and update the MMU page table and TLB entries, a stall of 10–50 µs that jeopardizes sub-millisecond control deadlines. Prefaulting and successful memory locking establish page residency before arming actuators. TLB and cache entries can still be evicted, so their miss costs remain in the execution budget; copy-on-write faults and later mapping changes must also be excluded.

C.3.4 Hardware watchdogs and fault response

In distributed computing clusters, node liveness and consensus are managed via software heartbeats and distributed leases (such as Raft heartbeats, ZooKeeper leases, or Kubernetes node probes) operating over second-scale intervals. If a worker node crashes, the cluster orchestrator reschedules containers onto alternative nodes. In physical AI systems, however, an autonomous machine traveling through physical space cannot wait seconds for a software orchestrator. If the GPU driver deadlocks, memory bus crossbars stall, or a kernel panic freezes the operating system, physical momentum continues unchecked. To detect a missed execution checkpoint, an embedded design uses a **hardware watchdog timer**. Its clock and response path must be independent of the monitored processor. An independent crystal oscillator or low-speed internal oscillator (`LSI`) completely decoupled from the main system phase-locked loop (PLL) is required.

The hardware watchdog timer comprises an autonomous down-counter register $W(t)$ clocked at frequency f_{wdg} (typically 32 kHz from an independent on-chip RC oscillator). At system initialization, the register is loaded with a threshold value W_{init}. Software execution threads must periodically refresh (or "kick") the register before the count decrements to zero:

$$W(t) = W_{\text{init}} - \left\lfloor f_{\text{wdg}} \cdot t \right\rfloor$$

At expiry, the timer asserts its configured reset or fault output. A separately wired trip is required when processor reset alone is insufficient to de-energize the actuators.

In safety-critical systems, simple down-counters are vulnerable to runaway execution bugs: if a corrupted control loop enters an infinite spin-loop that happens to execute the watchdog refresh instruction, the watchdog is continually fed despite the control failure. To prevent this, robust physical AI architectures employ a **Windowed Watchdog Timer (WWDG)**. A windowed watchdog defines both an upper counter threshold W_{upper} and a lower counter threshold W_{lower}. The software refresh is valid only while the counter lies between them:

$$W_{\text{lower}} \leq W(t_k) \leq W_{\text{upper}}$$

Because the counter falls at the fixed rate f_{wdg}, the two counter thresholds convert to the time bounds T_{lower} and T_{upper} that section 13.5 uses for the service window, measured from the last reload:

$$T_{\text{lower}} = \frac{W_{\text{init}} - W_{\text{upper}}}{f_{\text{wdg}}}, \quad T_{\text{upper}} = \frac{W_{\text{init}} - W_{\text{lower}}}{f_{\text{wdg}}}$$

If software attempts to refresh the watchdog too early ($t_k < T_{\text{lower}}$, equivalently $W(t_k) > W_{\text{upper}}$), the early write indicates an abnormal loop short-circuit or clock glitch. Refreshing too early or failing to refresh before T_{upper} trips the hardware watchdog controller immediately.

Feeding the watchdog directly from a periodic timer interrupt is an unsafe anti-pattern, because the timer interrupt may continue firing even if the primary motion planning and safety monitor threads are deadlocked. To check completion of the expected software checkpoints, control software can implement **multi-stage token-passing checkpointing** using rolling polynomial accumulators:

$$T_{k+1} = \text{CRC32}\left(T_k \oplus \text{Signature}_k\right)$$

1. At the beginning of the 1 kHz tick, a watchdog token T_0 is generated from the hardware monotonic cycle counter.
2. Stage 1 (Sensor Ingress): Validates camera, IMU, and joint telemetry timestamps, updating the token: $T_1 = \text{CRC32}(T_0 \oplus \text{sensor_valid_signature})$.

3. Stage 2 (State Estimation & Tracking): Computes joint tracking errors, updating $T_2 = \text{CRC32}(T_1 \oplus \text{tracking_valid_signature})$.
4. Stage 3 (Permission Enforcer): Validates Control Barrier Functions and stopping distance envelopes, computing $T_3 = \text{CRC32}(T_2 \oplus \text{enforce_valid_signature})$.
5. Stage 4 (Actuator Dispatch): Transmits admitted setpoints across the fieldbus, computing $T_{\text{final}} = \text{CRC32}(T_3 \oplus \text{dispatch_valid_signature})$.

Only if T_{final} matches the expected value for this tick may the watchdog refresh routine service the timer. The token detects omitted or misordered checkpoints under its fault model; a CRC is neither authentication nor proof that the computed state or command is physically correct.

A watchdog timeout must reach a validated hardware fail-safe response path independently of the processor it monitors. A CPU reset or non-maskable interrupt alone does not establish that connection. When the assessed response includes **safe torque off (STO)**, a separately wired output disables the gate drive that generates motor torque; the wiring, response time, and diagnostic coverage require device-specific qualification. Gate disable does not establish an instantaneous mechanical stop. Winding current takes time to decay, moving axes may coast, and a gravity-loaded axis may require a holding brake or a powered stop before torque removal. The complete watchdog-to-plant response belongs in the stopping budget.[2]

[2] **Hardware Watchdog Down-Counters**: A configured watchdog expires when its expected service does not occur. Its oscillator, timeout, output wiring, and plant response differ by device. Feeding it merely from an unrelated interrupt does not prove that the monitored control computation progressed; a firmware-fed reset also need not signal a drive. See the Linux watchdog API for reset behavior and section 12.9 for stop selection.

C.4 Inverter Timers and Power Electronics Gate Drive

In machine learning control frameworks (such as reinforcement learning policies, imitation learning, or model predictive control pipelines), actuator actions are formulated as continuous numerical setpoints, such as commanded joint torques τ^* or angular velocities $\dot{\mathbf{q}}^*$. In physical realization, however, torque is generated by synthesizing rotating electromagnetic stator flux through high-frequency switching of power semiconductor bridges (typically at 20 kHz to 100 kHz). Actuator motor drives employ three-phase half-bridge inverters to convert DC bus voltage into alternating stator currents.

DRAFT

C.4.1 Microcontroller timer dead-time insertion

Bridging the digital software domain with physical power silicon requires strict microarchitectural timing interlocks. Each inverter phase comprises a half-bridge with a high-side power MOSFET (or GaN FET / IGBT) and a low-side power MOSFET connected across the DC supply rail V_{bus}. Microcontroller timer peripherals generate complementary pulse-width-modulated (PWM) drive signals with hardware-enforced **dead-time insertion** t_{dead}:

$$t_{\text{dead}} \geq t_{\text{off, max}} + t_{\text{prop, skew}} - t_{\text{on, min}} + t_{\text{margin}}$$

where $t_{\text{off, max}}$ is the maximum turn-off delay of the power switch, $t_{\text{on, min}}$ is the minimum turn-on delay, $t_{\text{prop, skew}}$ is the propagation delay mismatch across isolated gate drivers, and t_{margin} is an engineering safety buffer.[3]

[3] **Inverter Dead-Time Insertion**: Timer dead time is selected from switch and driver timing over temperature and load, then verified with the actual gate waveforms. An overlap can create a damaging bus short; a numeric current or fuse outcome requires the circuit impedance and protection response.

Modern motor control microcontrollers (such as STM32 Advanced-control timers TIM1/TIM8 or TI C2000 ePWM modules) synthesize PWM waveforms using hardware up-down counting counters (`TIMx_CNT`). In **center-aligned mode**, the counter counts up from 0 to the auto-reload value (`TIMx_ARR`) and back down to 0, producing symmetrical PWM waveforms with period $T_{\text{pwm}} = 2 \cdot \text{ARR} \cdot T_{\text{timer_clk}}$. Center-aligned PWM provides symmetric switching and repeatable ADC trigger opportunities. Ripple reduction and valid sampling windows depend on duty cycle, modulation, current-sensing topology, and settling time. Synchronization alone does not remove switching noise.

The timer's hardware Dead-Time Generator (DTG) monitors the compare match outputs (`TIMx_CCRx`) and inserts dead-time delay cycles N_{dtg} using an autonomous digital delay line:

$$t_{\text{dead}} = N_{\text{dtg}} \cdot T_{\text{dtg}}$$

By latching the DTG configuration behind hardware register lock bits (`BDTR.LOCK`), the microcontroller guarantees that the dead-time delay cannot be altered by runaway firmware or memory corruption. Once locked at boot, the register ignores subsequent software writes until the next hardware reset.

A power MOSFET or GaN FET switch possesses input capacitance $C_{\text{iss}} = C_{\text{gs}} + C_{\text{gd}}$, where C_{gd} is the non-linear reverse transfer (Miller) capacitance. During gate turn-on and turn-off, charge transfer follows:

$$Q_g = \int_0^{t_{\text{sw}}} I_g(t)dt = Q_{\text{gs}} + Q_{\text{gd}}$$

During turn-off, the gate driver sinks current through gate resistance $R_{g,\text{off}}$. The gate-source voltage V_{gs} drops to the **Miller plateau voltage** V_{plateau}, where it dwells while the drain-to-source voltage V_{ds} traverses the entire DC rail:

$$t_{\text{plateau}} = \frac{Q_{\text{gd}}}{I_{g,\text{off}}} = \frac{Q_{\text{gd}} \cdot R_{g,\text{off}}}{V_{\text{plateau}} - V_{\text{ee}}}$$

Turn-on and turn-off times depend on gate resistance, driver strength, load, and device charge. Neither is universally longer; dead time must use the measured worst-case mismatch.

If dead time is insufficient, both high-side and low-side switches conduct simultaneously (**shoot-through**), creating a low-impedance short circuit directly bridging the DC rail to ground:

$$I_{\text{shoot-through}} = \frac{V_{\text{bus}}}{2R_{\text{ds(on)}} + R_{\text{trace}}}$$

This resistive expression gives a prospective current, not its instantaneous value. Loop inductance limits the initial rise, and the resulting device stress depends on switching overlap, protection delay, and the circuit safe operating area.

Even when dead time is configured correctly, half-bridge legs face **parasitic Miller cross-conduction**. When the high-side MOSFET turns on, the phase node voltage slews at high speed ($dV_{\text{ds}}/dt \sim 10$–$50$ V/ns in modern SiC and GaN bridges). This steep voltage transient forces displacement current through the low-side switch's Miller capacitance:

$$I_{\text{miller}} = C_{\text{gd}} \frac{dV_{\text{ds}}}{dt}$$

This current flows through the low-side gate resistor $R_{g,\text{off}}$ back to ground, inducing a parasitic gate voltage rise:

$$V_{\text{gs,induced}} = I_{\text{miller}} \cdot R_{g,\text{off}} = C_{\text{gd}} \frac{dV_{\text{ds}}}{dt} R_{g,\text{off}}$$

If $V_{\text{gs,induced}}$ exceeds the gate threshold voltage V_{th} (typically 1.5–3.0 V in GaN FETs), the low-side switch turns on spuriously while the high-side switch is conducting, inducing catastrophic shoot-through. Power electronics gate drivers counter this using **Active Miller Clamps (AMC)**, an internal low-impedance switch that clamps the gate directly to the negative rail whenever V_{gs} drops below 1.0 V, or split gate drives ($R_{g,\text{on}}/R_{g,\text{off}}$) with antiparallel Schottky diodes.

While necessary for silicon survival, dead time introduces non-linear voltage distortion into stator windings. During the dead-time interval t_{dead}, both active MOSFETs are commanded off, and signed instantaneous phase current i_{phase} (positive from the inverter node into the winding) is forced to freewheel through antiparallel body diodes. The sign of the phase current determines whether the inverter phase node clamps to V_{bus} or ground, producing an average phase voltage error:

$$\Delta V_{\text{phase}} = -\text{sgn}(i_{\text{phase}}) \cdot \frac{t_{\text{dead}}}{T_{\text{pwm}}} V_{\text{bus}}$$

In three-phase synchronous machines, this square-wave error injects 5th and 7th harmonic voltages into the stationary reference frame. When transformed into the rotor synchronous dq reference frame via Park's transformation, these odd harmonics combine into a **6th harmonic voltage disturbance**. The amplitudes and phases follow the harmonic coefficients of the phase-voltage error and the chosen transformation convention. They are not equal to the fundamental square-wave coefficient. The current loop filters these voltage harmonics before they become torque ripple, creating a corresponding **6th harmonic torque ripple**:

$$\tau_{\text{ripple}}(6\theta_e) = \frac{3}{2} N_{\text{poles}} \lambda_{\text{pm}} \cdot \Delta i_q(6\theta_e)$$

Left uncompensated, this hardware-induced torque ripple produces high-frequency torque chatter that excites mechanical drivetrain resonances. Modern motor drives implement dead-time compensation algorithms in low-level firmware that inject feedforward voltage vectors opposite to $\Delta V_{d,q}$, linearizing the actuator response for high-level ML policies.

C.5 Inductive Voltage Droop and Gate Delay Stretch

In data center machine learning clusters, power delivery networks (PDNs) are supported by large rack-level voltage regulator modules (VRMs) and extensive decoupling capacitor planes. On embedded physical AI compute platforms, deep learning accelerators (such as tensor cores and systolic array matrix engines) share tightly constrained power rails with CPU cores and real-time safety microcontrollers. When a deep neural network transitions from idle to full execution, current draw increases by tens of amperes within nanoseconds.

C.5.1 Power delivery network impedance and rail droop

A power delivery network (PDN) has parasitic resistance and inductance in each source path. For a lumped branch with resistance R_{PDN}, inductance $L_{\text{parasitic}}$, and branch current I_{branch}, its voltage drop is:

$$V_{\text{source}} - V_{\text{rail}} = L_{\text{parasitic}} \frac{dI_{\text{branch}}}{dt} + I_{\text{branch}} R_{\text{PDN}}$$

The accelerator's load current need not equal this branch current during a transient: local decoupling supplies their difference. The rail response follows the coupled source-path and capacitor equations of section C.10. Substituting the full load-current slew into every upstream inductance would ignore that buffering.

To prevent rail excursions from exceeding allowable voltage tolerances $\pm\Delta V_{\text{max}} = V_{\text{nom}} \times \delta_{\text{tol}}$ (typically ± 5 percent, or 45 mV on a 0.9 V digital core rail), the power delivery network must maintain an impedance profile strictly below the **target impedance** Z_{target} across the entire operational frequency band:

$$Z_{\text{target}} = \frac{V_{\text{nom}} \cdot \delta_{\text{tol}}}{\Delta I_{\text{max}}}$$

For a 0.9 V accelerator drawing a 50 A step transient with 5 percent allowable ripple, $Z_{\text{target}} = 45\ \text{mV}/50\ \text{A} = 0.90\ \text{m}\Omega$ from DC up to gigahertz frequencies. Exceeding this impedance at any excitation frequency permits transient load steps to pull the supply rail outside its timing-qualified window.

Switched-mode power supplies (VRMs) have limited feedback control loop bandwidth (100 kHz to 1 MHz) and cannot supply charge during nanosecond current transitions. Energy must be buffered across a **multi-tier decoupling capacitor hierarchy** structured by frequency response:

1. *On-Die Decoupling (>100 MHz)*: Deep trench and metal-insulator-metal (MIM) capacitors fabricated directly on the silicon die ($C_{\text{die}} \sim$ 10–100 nF, parasitic loop inductance < 10 pH, response time < 0.5 ns) supply charge during the initial clock cycles.
2. *Package Decoupling (10–100 MHz)*: Land-side capacitors (LSC) mounted under the BGA package ($C_{\text{pkg}} \sim$ 1–10 µF, loop inductance $\sim$ 50–200 pH, response time 1–5 ns).
3. *Board-Level High-Frequency MLCCs (100 kHz–10 MHz)*: Low-ESL surface-mount multi-layer ceramic capacitors (0201/0402 case sizes, $C \sim 0.1$–10 µF, loop inductance ~ 0.5–2 nH, response time 10–100 ns).
4. *Bulk Decoupling (DC–100 kHz)*: Polymer tantalum or aluminum electrolytic capacitors ($C_{\text{bulk}} \sim$ 100–1000 µF, response time 1–10 µs) sustain the rail until the active VRM regulation loop stabilizes.

Parasitic inductance and parallel capacitor branches can also create **anti-resonance** impedance peaks between their resonances. A simple LC frequency scale is $f_0 = 1/(2\pi\sqrt{L_{\text{parasitic}}C})$, but the peak magnitude requires the complete network, including capacitor equivalent series resistance and inductance. Excitation near a poorly damped peak can amplify supply excursions. A target-impedance check therefore needs the measured or modeled impedance spectrum, not capacitance alone.

C.5.2 Digital logic gate delay and the alpha-power law

When a 0.90 V core rail sags by 60 mV during an accelerator burst, transistor drive current drops and every logic gate along a critical path switches more slowly, eating into register setup slack. In sub-micron CMOS silicon, logic gate propagation delay t_{prop} is governed by the Sakurai-Newton **alpha-power law**:[4]

$$t_{\text{prop}}(V) \propto \frac{C \cdot V}{(V - V_{\text{th}})^{\alpha}}$$

where C is the gate load capacitance, V_{th} is the transistor threshold voltage, and $\alpha \in [1.2, 1.5]$ is the velocity saturation index (contrasting with $\alpha = 2.0$ in long-channel theory).

When rail voltage droops from V_{nom} to V_{droop}, propagation delay stretches according to:

$$\frac{t_{\text{prop}}(V_{\text{droop}})}{t_{\text{prop}}(V_{\text{nom}})} = \frac{V_{\text{droop}}}{V_{\text{nom}}} \left(\frac{V_{\text{nom}} - V_{\text{th}}}{V_{\text{droop}} - V_{\text{th}}} \right)^{\alpha}$$

In a synchronous digital pipeline with clock period T_{clk}, setup time t_{setup}, and clock skew t_{skew}, timing closure requires:

$$t_{\text{prop}} + t_{\text{setup}} + t_{\text{skew}} \leq T_{\text{clk}}$$

The following illustrative calculation evaluates the delay ratio and remaining path slack with the same voltage-dependent model. A droop consumes setup margin; whether it causes a timing violation depends on the original slack, clock response, and characterized process corner.

For a drop from 0.90 V to 0.81 V, with threshold 0.35 V and exponent 1.3, delay rises by 13.5 percent to 908.3 ps. Starting with 150 ps of slack leaves 41.7 ps. These assumptions therefore do not predict a setup violation. Some processors respond to droop with active clock stretching, increasing the clock period while the rail recovers. The supported response, detection delay, and resulting execution-time stretch require device-specific characterization. Section 13.4 connects rail disturbance to the permission task's timing budget.

[4] **Sakurai-Newton Delay Modeling**: The alpha-power law models voltage-dependent gate delay; its parameters are technology dependent. The model predicts reduced setup slack during droop, while a missed deadline, register error, or reset must be established against measured path slack and device thresholds.

C.6 DRAM Contention and Interconnect Arbitration

In cloud machine learning clusters, GPUs use high-bandwidth memory (HBM) physically separated from host CPU memory. This separates some accelerator traffic from host DRAM, although accelerator clients still contend for those channels. In embedded robotics SoCs, however, system architectures rely on Unified Memory Architecture (UMA) to satisfy size, weight, and power (SWaP) constraints. Application CPU cores, neural accelerators, GPU cores, and high-rate PCIe or CSI camera DMA engines all share a single physical DRAM interface. In these shared-memory multi-core platforms, DMA transfers and vision prefetching compete directly with the permission path for memory controller command queues and physical DRAM banks.

C.6.1 DRAM bank organization and conflict latency

DRAM organizes storage into channels, ranks, **bank groups**, and banks; their counts and timing depend on the memory standard and device organization. Each individual bank possesses an independent row buffer consisting of sense amplifiers. A read that changes the open row can require the following sequence:

1. **Row precharge** (t_{RP}): Closes the active row and equalizes the bitlines for the next activation (18.75 ns per bank at LPDDR5-6400).
2. **Row activate** (t_{RCD}): Reads an entire page of storage cells (e.g., 2 KB to 8 KB) across wordlines into the bank's open row buffer (18.75 ns at LPDDR5-6400).
3. **Column access** (read latency, t_{CL}): Reads or writes data beats from the open row buffer onto the memory data bus. LPDDR5 specifies this latency in clock cycles that depend on the data rate, so a row hit pays only this step.

To sustain high data rates, modern DRAM organizes banks into bank groups. An LPDDR5 channel offers three bank organizations: 16 banks divided into 4 bank groups for the higher data rates, 16 independent banks, or 8 larger banks formed by merging pairs (Steiner et al. 2022). In bank-group

mode, consecutive column commands to the same bank group must wait a longer column-to-column delay $t_{\text{CCD_L}}$ than commands to different groups, which need only $t_{\text{CCD_S}}$, because banks in one group share internal data paths. An address mapping that interleaves consecutive bursts across bank groups hides the longer spacing; one that keeps a stream inside a single group exposes it.

When consecutive memory operations target different rows within the same physical bank (a **bank conflict**), the memory controller cannot immediately access the column. It must execute a precharge followed by an activate before issuing the column command, introducing a compulsory stall:

$$\Delta t_{\text{conflict}} \approx t_{\text{RP}} + t_{\text{RCD}}$$

At LPDDR5-6400 that stall is 37.5 ns. The minimum time between consecutive row activations to the same bank is also governed by the row cycle time $t_{\text{RC}} = t_{\text{RAS}} + t_{\text{RP}}$, where t_{RAS} is the minimum time a row must stay open. Repeated conflicts in one bank are therefore spaced at least t_{RC} apart.

Memory controllers select between an *open-page* or *closed-page* policy to balance row-buffer locality against thrashing. Under an open-page policy, row buffers remain active after access in anticipation of spatial locality. While this maximizes bandwidth for sequential streaming (such as dense general matrix multiplication (GEMM) weight fetching or linear camera DMA writes), sporadic random accesses (such as sensor fusion state lookups or 1 kHz safety monitor checks) frequently hit different rows in active banks. When a continuous camera DMA stream writes video frames to the same DRAM bank that holds real-time safety buffers, repeated row-buffer conflicts transform low-latency reads into multi-hundred-nanosecond stalls.[5]

[5] **DRAM Bank-Conflict Penalties**: A row change in one bank can add precharge and activate time. Video DMA can also queue transactions ahead of a safety read. The resulting delay must be bounded for the specific address mapping, arbitration policy, and workload; a fixed penalty per request is generally unjustified.

Modern DRAM controllers employ **First-Ready First-Come-First-Served (FR-FCFS)** command scheduling. FR-FCFS prioritizes column read/write requests that hit currently open row buffers over older requests that require row precharge and activation, maximizing overall DRAM bus throughput. However, under high cognitive load, a continuous flood of vision DMA requests or matrix multiplication memory bursts will repeatedly hit open rows. FR-FCFS schedulers then systematically postpone isolated, latency-critical read requests issued by real-time safety threads, potentially starving those requests unless the controller enforces age or service bounds.

C.6.2 Interconnect crossbars and burst contention

On-chip interconnect fabrics (such as ARM CoreLink AXI4 crossbars or Network-on-Chip (NoC) meshes) split high-bandwidth accelerator and camera DMA transactions into bounded bursts of up to $A_{\text{len}} = 256$ beats. In a shared crossbar without real-time prioritization, transactions enter a FIFO or weighted round-robin queue. For K competing accelerator transactions admitted ahead of a safety-critical permission request, the queuing transfer latency is:

$$t_{\text{queue,ideal}} = \frac{\sum_{k=1}^{K} D_{\text{vol}_k}}{\text{BW}_{\text{bus}}}$$

Adding physical bank-switch penalties $\sum \Delta t_{\text{conflict}}$, device-specific refresh overhead, and bus turnaround penalties increases the safety read latency. Refresh mode, temperature, address mapping, and controller policy determine the bound. To protect real-time control paths against memory contention, embedded systems implement three structural defenses:

1. *AXI Quality of Service (QoS) Priority Elevation*: The `AxQOS` field communicates a requested priority. The interconnect and memory controller must implement a documented service policy; the field alone does not guarantee preemption or bounded latency. Already issued transactions and refresh can still block a safety read.
2. *DRAM Bank Partitioning and Page Coloring*: The operating system memory allocator allocates physical memory pages such that real-time safety code and high-rate camera DMA buffers map to strictly disjoint physical DRAM banks, removing those bank conflicts when the physical address mapping is known and enforced. Shared channels, refresh, and controller queues remain.
3. *On-Chip SRAM Isolation*: Critical safety states, barrier function parameters, and active trajectory splines bypass external DRAM entirely, executing exclusively out of static on-chip SRAM or Data Tightly Coupled Memory (DTCM).

C.7 Clock Synchronization and Network Timestamps

In distributed machine learning clusters, nodes synchronize system clocks via Network Time Protocol (NTP) primarily for coarse event logging and telemetry, while parallel training algorithms synchronize execution through communication barriers (such as NCCL AllReduce). In physical AI systems, however, time directly maps to physical space: when a robot moves at 30 m/s, a 1 ms synchronization error produces 3 cm of spatial displacement, warping sensor fusion point clouds and invalidating safety clearance buffers. Distributed perception pipelines must convert sensor timestamps into a common clock domain with a measured conversion-error bound.

C.7.1 Precision time protocol message exchange

The following end-to-end delay exchange illustrates IEEE 1588 Precision Time Protocol (PTP), synchronizing a follower clock (S) to a leader clock (M). IEEE 802.1AS uses a related timing architecture with peer-delay measurement; its exchange is not identical:

1. t_1: Master transmits a *Sync* frame, recording its local transmission timestamp.
2. t_2: Slave receives the *Sync* frame, recording its local arrival timestamp.
3. t_3: Slave transmits a *Delay_Req* frame, recording its local transmission timestamp.
4. t_4: Master receives the *Delay_Req* frame, recording its local arrival timestamp.

Under the assumption of clock stability and symmetric forward and reverse propagation delay ($T_{\text{prop},M\to S} = T_{\text{prop},S\to M} = T_{\text{prop}}$), the time relationships satisfy:

$$t_2 = t_1 + \theta_{\text{offset}} + T_{\text{prop}}$$

$$t_4 = t_3 - \theta_{\text{offset}} + T_{\text{prop}}$$

where θ_{offset} is the slave-minus-master clock offset. The estimated clock offset is:

$$\theta_{\text{offset}} = \frac{(t_2 - t_1) - (t_4 - t_3)}{2}$$

The one-way network propagation delay is:

$$T_{\text{prop}} = \frac{(t_2 - t_1) + (t_4 - t_3)}{2}$$

Slave clocks adjust their local time base using a closed-loop digital **proportional-integral (PI) clock servo** filter. At synchronization step k, the measured offset $\theta_{\text{offset}}[k]$ drives frequency correction $\Delta f[k]$ applied to the hardware accumulator register:

$$\Delta f[k] = -K_p \theta_{\text{offset}}[k] - K_i \sum_{j=0}^{k} \theta_{\text{offset}}[j]$$

Here Δf is added to the follower frequency, $K_p, K_i > 0$, and the sampling interval is absorbed into K_i. A positive follower-minus-leader offset therefore slows the follower; using the opposite hardware correction convention reverses the sign.

If forward and reverse path delays are asymmetric ($\Delta t_{\text{asym}} = T_{\text{prop},M\to S} - T_{\text{prop},S\to M} \neq 0$), the calculated offset incurs an unobservable systematic bias of $\Delta t_{\text{asym}}/2$. Residual clock uncertainty ϵ_{sync}, combining asymmetric propagation, oscillator frequency drift between sync intervals, and hardware timestamp quantization, is explicitly bounded:

$$\epsilon_{\text{sync}} \leq \frac{|\Delta t_{\text{asym}}|}{2} + \Delta t_{\text{interval}} \cdot \left| \frac{\Delta f_{\text{res}}}{f_{\text{nom}}} \right| + 2T_{\text{quant}}$$

Here Δf_{res} bounds residual frequency error after servo correction, rather than the applied correction itself; T_{quant} bounds timestamp quantization. Additional timestamp-reference and servo errors must be included if they are not covered by these terms. This conversion bound is stored directly in the proposal header's `sync_error_ns` field (table C.10) and added to every evidence-age calculation in the 1 kHz permission path:

$$t_{\text{age}} = (t_{\text{current}} - t_0) + \epsilon_{\text{sync}}$$

C.7.2 Physical layer hardware timestamping

Software-based timestamping captures timestamps inside the OS network socket layer or interrupt service routine. It suffers from variable kernel queueing, driver interrupt latency, and thread scheduling jitter, which inject $50\,\mu s$ to 5 ms of non-deterministic timing error. To eliminate host dispatch error, physical AI hardware uses **Physical Layer (PHY) Hardware Timestamping**.[6] Dedicated timestamping units in the Ethernet PHY or media access control (MAC) controller capture hardware counter values at the exact instant the physical Start of Frame Delimiter (SFD) crosses the Media Independent Interface (MII / RGMII). This removes software dispatch from the capture point. Timestamp resolution, reference-plane calibration, and residual error remain device-specific and must be measured.

[6] **Hardware Timestamping**: A compatible PHY or MAC timestamps frames closer to the wire than a software interrupt handler. Hardware timestamping removes one source of dispatch variation, while path asymmetry and clock conversion remain. Timestamp uncertainty belongs in the physical belief record and permission budget.

C.8 Heterogeneous SoC Mailboxes, Memory Barriers, and Zero-Copy IPC

Large sensor buffers can consume substantial shared-memory bandwidth when each pipeline stage makes another copy. A copy may generate both a read and a write of the payload, depending on cache residency and the transfer path. On a heterogeneous robotics SoC, that traffic can contend with a safety processor's memory accesses. Shared buffers reduce copying, but require explicit ownership and publication ordering. Small proposal records may instead use a bounded copy into protected memory. The mechanisms below distinguish four contexts. A discrete safety microcontroller receives proposals over a board-level link. Application and safety cores on one die exchange platform-specific mailboxes. Linux processes share coherent memory. Supported GPU processes share device allocations. Their ordering and lifetime contracts are not interchangeable.

C.8.1 Board-level links into a local mailbox

The machine in this book keeps its safety microcontroller on a separate chip with its own memory (section 13.6), so no memory is shared with the application processor and nothing in this subsection depends on the host's cache or interconnect.

1. The application processor serializes the proposal into a frame that carries a sequence number, the evidence and issue timestamps, and a CRC, and sends it over a board-level link such as SPI, a UART, or CAN FD.
2. The microcontroller's link peripheral writes the incoming bytes into a receive buffer in its own SRAM by DMA. A double buffer lets the DMA fill one half while firmware reads the other.
3. The DMA-complete interrupt checks the frame's length, CRC, and sequence number, then publishes the proposal into the seqlock mailbox of algorithm C.1. The writer and the reader, the permission tick, both run on the microcontroller.
4. The permission tick makes one bounded read of that mailbox and checks age, expiry, and permission before using the payload.

The link is a black channel in the sense of the functional-safety fieldbus profiles (International Electrotechnical Commission 2021). Corruption, repetition, loss, and delay are detected end to end by the frame's CRC, sequence number, and timestamps and by the lease, so the link's hardware and drivers need not be trusted for correctness. Its timing is another matter. Serialization at the link rate, DMA completion, and receive-interrupt latency age every newly admitted proposal and must be bounded and charged to the dispatch-to-admission interval.

C.8.2 Remote processor messaging over shared SRAM

When one system-on-chip hosts both domains, the shared-die placement of section 13.2, communication between Linux application cores and a real-time safety core relies on shared SRAM mailboxes signaled via hardware doorbells:

1. Core A (Application Processor) writes a message payload to a fixed buffer in shared SRAM.
2. Core A makes the payload visible to Core B using the platform's coherent mapping or explicit cache maintenance and ordering.
3. Core A triggers an inter-processor interrupt (IPI) by writing to Core B's hardware mailbox register.

4. Core B checks publication order, integrity, age, and permission before using the payload.

Linux RPMsg is a virtio-based messaging framework that may use shared memory and a platform-specific kick. It provides standard transport abstraction but does not prescribe this fixed-buffer protocol or a universal latency bound. It applies only to this shared-die case; a discrete microcontroller shares no memory to message through.

C.8.3 CPU pipeline reordering and memory fences

In weakly ordered memory architectures (such as ARMv8-A and ARMv9-A), processors do not guarantee that memory writes will be observed by other cores in program order. Out-of-order execution engines and asynchronous write buffers can reorder stores to non-dependent addresses:

```
// Core A (Application Processor); q_target and q_cmd are joint_target_t structs
shared_buffer->q_target = q_cmd; // Store 1: Payload (struct copy)
mailbox_doorbell = 1;            // Store 2: Interrupt trigger
```

If the CPU reorders Store 2 ahead of Store 1, Core B receives the interrupt before the new joint target is committed to physical memory. The consumer then reads stale or partially written struct fields across the mailbox boundary.

To enforce deterministic ordering across heterogeneous cores, developers must insert architectural **hardware memory barriers**:

```
shared_buffer->q_target = q_cmd; // Struct copy of the joint targets
__asm__ volatile("dmb oshst" ::: "memory"); // Outer-shareable store barrier
mailbox_doorbell = 1;            // Assert doorbell register
```

In ARM architectures, memory barriers and instructions are distinguished by scope and execution semantics:

- `DMB` (Data Memory Barrier): Ensures that memory accesses specified before the barrier appear in memory before memory accesses specified after the barrier. It does not block execution of non-memory instructions.
- `DSB` (Data Synchronization Barrier): Waits for completion of the memory accesses and maintenance operations covered by its type and scope before subsequent instructions execute.
- `ISB` (Instruction Synchronization Barrier): Flushes the CPU instruction prefetch pipeline, ensuring subsequent instructions are fetched anew from cache or memory.

Barriers operate across explicit architectural **shareability domains**:

- `DMB ISHST` (Inner Shareable Store): Orders stores across cores within the local application CPU cluster sharing a coherent L2/L3 cache.
- `DMB OSHST` / `DMB OSH` (Outer Shareable): Orders accesses in the configured outer-shareable domain; membership depends on the SoC, and ordering does not create coherence.
- `DMB SY` (System): Enforces ordering across all system masters and peripherals on the SoC.

Cache maintenance is necessary when the communicating agents do not share a coherent mapping. The producer cleans dirty payload lines, completes the required maintenance, and orders publication. The consumer invalidates stale lines before reading under the platform contract. A supported non-cacheable mapping can avoid those cache operations, but does not make barriers single-cycle or remove interconnect contention. Device memory has different access restrictions from Normal memory and is not an interchangeable choice for arbitrary structure copies. Alignment must match the actual cache-line size. Section 4.3.2 and section 13.2 connect these mechanisms to the proposal boundary.[7]

[7] **Hardware Memory Barriers**: Barriers order accesses only within their specified shareability and memory-attribute contract. On a noncoherent route, cache clean/invalidate operations or coherent mappings are also required. Arm's memory-ordering guide distinguishes ordering from coherence.

C.8.4 POSIX shared memory and lock-free SPSC ring buffers

Within user-space Linux runtimes, independent operating system processes (such as a vision preprocessing daemon, a multimodal VLA policy inference server, and a high-rate state estimator) exchange telemetry and sensor streams. They avoid kernel socket copies by using **POSIX Shared Memory** (`shm_open` and `mmap`):

1. A producer process creates a POSIX shared memory object using `shm_open("/robot_telemetry", O_CREAT | O_RDWR, 0660)` located in the RAM-backed `tmpfs` filesystem (`/dev/shm`), bypassing physical storage media entirely.
2. Sizing the segment via `ftruncate(fd, sizeof(telemetry_ring_buffer_t))` establishes its length; physical backing is generally allocated when pages are faulted in.
3. Both producer and consumer processes map the memory into their respective virtual address spaces via `mmap(NULL, sizeof(...), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0)`.
4. Each process prefaults and successfully locks its mapping with `mlock()` before cyclic execution. This prevents swapping of those pages; mapping changes and copy-on-write behavior still need explicit control.
5. *Position-Independent Memory Layout*: Because Address Space Layout Randomization (ASLR) and independent virtual memory spaces cause `mmap()` to return different base virtual addresses in each process, data structures placed in shared memory must never store absolute virtual pointers (`void*` or raw references). All references within shared buffers must be position-independent, structured as relative byte offsets, flat fixed-size arrays, or integer indices.

When streaming sequential sensor frames or trajectory waypoints across processes, mutual-exclusion mutexes introduce non-deterministic blocking and priority inversion. A **lock-free single-producer single-consumer (SPSC) ring buffer** coordinates one producer and one consumer using acquire-release atomic indices. The following C++ sketch assumes lock-free atomics, bounded nonthrowing copies of `T`, and coherent memory. Cross-process use additionally requires a platform-supported shared atomic representation and one-time construction before either process accesses it:

```
#include <atomic>
#include <cstdint>
#include <cstddef>
#include <new>

template <typename T, size_t Capacity>
class LockFreeSpscRingBuffer {
    static_assert(Capacity > 0 && (Capacity & (Capacity - 1)) == 0,
                  "Capacity must be a nonzero power of two");
    static_assert(std::atomic<uint64_t>::is_always_lock_free,
                  "Index atomics must be lock-free on this target");

private:
    // Align head and tail to distinct 64-byte cache lines to eliminate false sharing
    alignas(64) std::atomic<uint64_t> write_index_{0};
    alignas(64) std::atomic<uint64_t> read_index_{0};
    alignas(64) T storage_[Capacity];

public:
    bool Push(const T& item) {
        const uint64_t current_write = write_index_.load(std::memory_order_relaxed);
        const uint64_t current_read = read_index_.load(std::memory_order_acquire);

        if (current_write - current_read >= Capacity) {
            return false; // Buffer full: non-blocking drop or backpressure
        }

        // Fast bitwise masking replacing expensive modulo division
        storage_[current_write & (Capacity - 1)] = item;

        // Release order ensures item stores are committed before advancing pointer
        write_index_.store(current_write + 1, std::memory_order_release);
        return true;
    }

    bool Pop(T& item) {
        const uint64_t current_read = read_index_.load(std::memory_order_relaxed);
        const uint64_t current_write = write_index_.load(std::memory_order_acquire);
```

```
        if (current_read == current_write) {
            return false; // Buffer empty
        }

        item = storage_[current_read & (Capacity - 1)];

        // Release order publishes free slot back to producer
        read_index_.store(current_read + 1, std::memory_order_release);
        return true;
    }
};
```

Separating the indices onto different cache lines avoids false sharing between the two independently written counters on a target with the assumed line size. Each side still reads the other's counter, and payload ownership still moves between cores, so coherence traffic remains. Power-of-two capacity permits masking for wraparound. Neither alignment nor this indexing choice establishes a cycle-time bound without target measurements.

C.8.5 Multi-rate synchronization: Double-buffering versus triple-buffering

In cyber-physical machines, producer and consumer tasks rarely execute at the same frequency. A multimodal vision-language-action policy may produce candidate trajectory chunks at 20 Hz (50 ms period) with variable inference jitter (± 15 ms). The downstream safety permission filter and motor tracking loop, by contrast, execute strictly at 1 kHz (1 ms period). Double-buffering separates a buffer being written from one being read, but it also needs an ownership protocol. If the producer finishes while the consumer still owns the old front buffer, the producer must wait, drop an update, or acquire another buffer. Rate mismatch alone does not make double-buffering incorrect. Allowing a writer to reuse a reader-owned buffer does.

Triple-buffering provides a private producer buffer, a private consumer buffer, and an intermediate buffer holding the latest completed update. Publication atomically exchanges the producer buffer with the intermediate buffer. At a tick boundary, the consumer may exchange its private buffer with the intermediate buffer, then read its new private buffer without concurrent writes. The exchanged state must include the new-data flag so that publication and ownership remain one atomic operation.

This is a latest-value channel: intermediate updates may be overwritten before the consumer acquires them. A queue is needed when every update must be retained. A compare-and-swap retry loop is lock-free only under its progress assumptions and is not automatically wait-free. The implementation must establish ordering, buffer lifetime, and a bounded reader attempt on the target. A failed attempt leaves the current record subject to its existing evidence and expiry checks.

C.8.6 CUDA inter-process communication for multi-accelerator pipelines

On high-throughput robotics platforms (such as autonomous mobile robots equipped with multiple GMSL2 camera clusters and lidar arrays), raw perception data streams into accelerator GPU memory for tensor processing. In modular physical AI software architectures, perception preprocessing (e.g., optical flow, spatial unprojection, and feature extraction) and foundation policy inference often execute in separate Linux processes for fault containment. Passing multi-gigabyte tensor buffers between processes via host system RAM consumes tens of gigabytes per second of shared DRAM bandwidth, precipitating memory bus contention.

To eliminate host DRAM staging entirely, **CUDA Inter-Process Communication (CUDA IPC)** enables direct peer-to-peer GPU memory access across distinct Linux process address spaces:

```
// Process A (Perception Producer): Export GPU device memory handle
float *d_camera_tensor;
cudaMalloc((void **)&d_camera_tensor, TENSOR_BYTES);

// Allocate and capture an opaque 64-byte IPC memory handle
cudaIpcMemHandle_t ipc_mem_handle;
cudaIpcGetMemHandle(&ipc_mem_handle, (void *)d_camera_tensor);
```

```
// Send the 64-byte handle to Process B via POSIX shm or UNIX domain socket
send_ipc_handle_to_consumer(&ipc_mem_handle);
```

In Process B (Policy Consumer), the exported handle is mapped directly into its local GPU virtual address space without copying:

```
// Process B (Policy Consumer): Map exported GPU memory directly
cudaIpcMemHandle_t received_handle;
receive_ipc_handle_from_producer(&received_handle);

float *d_mapped_tensor;
cudaIpcOpenMemHandle((void **)&d_mapped_tensor, received_handle,
                     cudaIpcMemLazyEnablePeerAccess);

// Wait on this publication's imported completion event before reading.
// Launch inference kernel directly referencing Process A's physical GPU memory
launch_vla_encoder_kernel<<<grid, block, 0, compute_stream>>>(d_mapped_tensor);

// At shutdown, finish outstanding work before releasing the mapping.
cudaStreamSynchronize(compute_stream);
cudaIpcCloseMemHandle((void *)d_mapped_tensor);
```

CUDA IPC requires compatible devices, drivers, and operating-system support. An imported allocation must remain alive until every importer finishes and closes its mapping. Publication requires explicit synchronization, not just handle transfer. CUDA IPC relies on strict memory allocation invariants: exported memory must be allocated as device memory via `cudaMalloc()` within the GPU's physical device memory. Managed memory (`cudaMallocManaged`) and host pinned memory are explicitly unsupported by the CUDA IPC subsystem. On supported same-host multi-accelerator topologies, an imported allocation may be accessed through peer-to-peer links. CUDA IPC is not a transport between arbitrary independent SoCs. Rather than copying tensor buffers through host DRAM, the GPU hardware memory management unit (MMU) routes memory transactions directly across high-speed interconnect links. This eliminates host CPU DRAM double-buffering and saves tens of gigabytes per second of shared bus bandwidth.

To synchronize GPU execution across processes without round-tripping to host CPU threads, CUDA IPC exports **cross-process CUDA event** synchronization primitives via `cudaIpcGetEventHandle()` and `cudaIpcOpenEventHandle()`. Process A creates an interprocess event with timing disabled, records it after producing the buffer, and publishes the corresponding completion generation. The consumer must wait on that generation before using the data, and the producer must not reuse the buffer before the consumer finishes. Process B imports the event handle and enqueues a hardware barrier on its local stream:

$$\text{cudaStreamWaitEvent}(\text{consumer_stream}, \text{imported_event}, 0)$$

Once the host has enqueued the dependency, later consumer-stream work waits for the recorded producer event without a host polling loop. Setup and submission still involve the CPU, and latency remains platform dependent. The CUDA IPC documentation specifies supported allocation and event types.

C.9 Actuator Electrical and Thermal Dynamics

In reinforcement learning and simulation environments (such as MuJoCo or Isaac Gym), policy actions are typically abstracted as instantaneous torque or position commands applied at discrete simulation steps. In physical systems, however, actuators convert electrical energy into mechanical work through electromagnetic Lorentz forces, subject to coupled electrical, thermal, and kinematic constraints.

C.9.1 Stator electrical rise time

A brushless permanent-magnet synchronous motor (PMSM / BLDC) phase winding has phase resistance R_{phase} and phase inductance L_{phase}. Applying terminal voltage V_{terminal} against back-electromotive force $e_{\text{bemf}} = K_e\omega_m$ yields the first-order stator current ODE:

$$L_{\text{phase}}\frac{di_{\text{phase}}(t)}{dt} + R_{\text{phase}}\, i_{\text{phase}}(t) = V_{\text{terminal}} - K_e\omega_m$$

In three-phase synchronous machines under Field-Oriented Control (FOC), the stator currents are transformed from the three-phase abc frame into the orthogonal rotor synchronous dq reference frame via the Clarke and Park transformations:

$$v_d = R_{\text{phase}} i_d + L_d\frac{di_d}{dt} - \omega_e L_q i_q$$

$$v_q = R_{\text{phase}} i_q + L_q\frac{di_q}{dt} + \omega_e L_d i_d + \omega_e \lambda_{\text{pm}}$$

The d axis rotates with the rotor magnet flux, and the q axis lies perpendicular to it. This separates flux and torque-producing current. Here $\omega_e = N_{\text{poles}}\omega_m$ is electrical angular velocity (N_{poles} pole pairs), λ_{pm} is the rotor permanent-magnet flux linkage, and L_d, L_q are direct and quadrature axis inductances. Under standard constant torque-angle control ($i_d = 0$), electromagnetic torque is directly proportional to quadrature current:

$$\tau_{\text{motor}} = \frac{3}{2}N_{\text{poles}}\,\lambda_{\text{pm}} i_q = K_t i_q$$

For a commanded voltage step with the rotor held at rest ($\omega_m = 0$) and zero initial current, quadrature current rises exponentially:

$$i_q(t) = \frac{V_{\text{terminal}}}{R_{\text{phase}}}\left(1 - e^{-t/\tau_e}\right), \quad \tau_e = \frac{L_q}{R_{\text{phase}}}$$

where τ_e is the electrical time constant. Current, and therefore delivered torque, reaches approximately 63 percent of its steady-state value after one time constant τ_e (section 2.4.2 computes it for the running machine's hub motors from their datasheet inductance and resistance). An unprivileged neural policy step command cannot produce an instantaneous step in physical torque.

Furthermore, achievable current is constrained by inverter voltage saturation:

$$\sqrt{v_d^2 + v_q^2} \le \frac{V_{\text{bus}}}{\sqrt{3}}$$

As rotor speed ω_m rises, back-EMF $\omega_e\lambda_{\text{pm}}$ increases, consuming available quadrature voltage v_q. At the actuator's base speed, voltage headroom vanishes, and torque collapses unless negative direct-axis current ($i_d < 0$, flux weakening) is commanded to oppose part of the magnet flux linkage, within current and magnet-demagnetization limits.

C.9.2 Lumped thermal networks and stator Joule heating

Current flowing through stator windings dissipates resistive Joule heat:

$$P_{\text{loss}}(t) = \frac{3}{2} i_q^2(t) R_{\text{phase}}(T_{\text{winding}})$$

where copper electrical resistance increases linearly with temperature:

$$R_{\text{phase}}(T_{\text{winding}}) = R_0\left[1 + \alpha_{\text{Cu}}(T_{\text{winding}} - T_0)\right]$$

with temperature coefficient $\alpha_{\text{Cu}} \approx 0.00393\ \text{K}^{-1}$ for copper. At an operating temperature of 150°C, stator copper resistance is approximately 50 percent higher than at room temperature (25°C). To sustain the same commanded torque $\tau_{\text{motor}} = K_t i_q$, the motor requires identical current i_q but generates 50 percent more thermal dissipation. This creates positive feedback between winding temperature and Joule heating that accelerates thermal degradation.

In a physical motor, **multi-node lumped thermal networks** span multiple physical interfaces with disparate thermal masses:

1. *Stator Winding Copper*: Low thermal capacitance C_{cu}, rapid heating from Joule dissipation ($\tau_{\text{th,cu}} \sim 10$–$30$ s).
2. *Stator Iron Laminations*: Moderate thermal capacitance C_{iron}, absorbing iron eddy-current and hysteresis losses P_{iron} and heat conducted across winding insulation $R_{\text{th,cu-iron}}$. The iron losses depend on flux, frequency, and material.
3. *Motor Frame and Aluminum Housing*: Large thermal capacitance C_{case}, conducting heat into the robotic link structure and convecting to ambient air through $R_{\text{th,case-amb}}$ ($\tau_{\text{th,case}} \sim 10$–$30$ min).

The temperature dynamics across these nodes satisfy the coupled differential system:

$$C_{\text{cu}} \frac{dT_{\text{winding}}}{dt} = P_{\text{loss}}(t) - \frac{T_{\text{winding}} - T_{\text{iron}}}{R_{\text{th,cu-iron}}}$$

$$C_{\text{iron}} \frac{dT_{\text{iron}}}{dt} = \frac{T_{\text{winding}} - T_{\text{iron}}}{R_{\text{th,cu-iron}}} + P_{\text{iron}} - \frac{T_{\text{iron}} - T_{\text{case}}}{R_{\text{th,iron-case}}}$$

$$C_{\text{case}} \frac{dT_{\text{case}}}{dt} = \frac{T_{\text{iron}} - T_{\text{case}}}{R_{\text{th,iron-case}}} - \frac{T_{\text{case}} - T_{\text{amb}}}{R_{\text{th,case-amb}}}$$

For initial temperature at ambient, a simplified single-node lumped thermal model uses effective thermal resistance to ambient R_{th} (in K/W) and thermal heat capacity C_{th} (in J/K). The temperature rise $\Delta T(t) = T_{\text{winding}}(t) - T_{\text{amb}}$ satisfies:[8]

$$C_{\text{th}} \frac{d\Delta T(t)}{dt} = P_{\text{loss}}(t) - \frac{\Delta T(t)}{R_{\text{th}}}$$

Under constant power, temperature rises with thermal time constant $\tau_{\text{th}} = R_{\text{th}} C_{\text{th}}$:

$$\Delta T(t) = P_{\text{loss}} R_{\text{th}} \left(1 - e^{-t/\tau_{\text{th}}}\right)$$

Continuous torque capacity τ_{cont} is bounded by the maximum allowable winding temperature rise ΔT_{max}:

$$\tau_{\text{cont}} \leq K_t \sqrt{\frac{2\Delta T_{\text{max}}}{3 R_{\text{th}} R_{\text{phase}}}}$$

The factor $2/3$ follows from the three-phase loss convention above; use the winding resistance at the limiting temperature. Other motor and inverter losses reduce the available thermal margin.

Transient torque peaks $\tau_{\text{peak}} \approx 2$–$4 \times \tau_{\text{cont}}$ are permitted during aggressive dynamic maneuvers only if an active digital ${I_{\text{phase}}}^2 t$ thermal protection observer running on the safety microcontroller bounds the thermal integral:

$$\Delta T[k+1] = \Delta T[k] e^{-\Delta t/\tau_{\text{th}}} + R_{\text{th}} P_{\text{loss}}[k] \left(1 - e^{-\Delta t/\tau_{\text{th}}}\right)$$

The thermal class is an insulation service rating, not a sharp instantaneous breakdown temperature. Current derating must begin with margin below the winding and rotor limits, accounting for model error, initial temperature, and response time.

[8] **Stator Thermal Time Constants**: Resistive Joule heating elevates motor phase winding temperatures during continuous high-torque maneuvers. Because copper electrical resistivity increases linearly with temperature, hot windings have higher resistance, reducing torque output under terminal voltage saturation. Exceeding winding insulation temperature limits breaks down dielectric coatings, causing inter-turn short circuits and permanent winding damage.

C.9.3 Reflected rotor inertia and angular jerk limits

Geared electromagnetic actuators amplify delivered motor torque while multiplying rotor inertia at the joint output. For a transmission with gear reduction ratio $N_{\text{gear}} = \omega_m / \dot{q}$, kinetic energy conservation dictates the reflected inertia acting at the output link:

$$E_k = \frac{1}{2} J_{\text{rotor}} \omega_m^2 + \frac{1}{2} J_{\text{load}} \dot{q}^2 = \frac{1}{2} (J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}}) \dot{q}^2$$

The effective rotational inertia seen at joint coordinate q is:

$$J_{\text{eff}} = J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}}$$

The joint equation of motion under motor electromagnetic torque τ_{motor} is:

$$\ddot{q} = \frac{N_{\text{gear}}\tau_{\text{motor}} - \tau_{\text{ext}}}{J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}}}$$

Differentiating $\ddot{q}$ with respect to gear ratio N_{gear} at zero external load yields:

$$\frac{\partial \ddot{q}}{\partial N_{\text{gear}}} = \frac{\tau_{\text{motor}}(J_{\text{load}} - N_{\text{gear}}^2 J_{\text{rotor}})}{(J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}})^2}$$

Setting $\frac{\partial \ddot{q}}{\partial N_{\text{gear}}} = 0$ yields the **inertia-matching ratio** $N^* = \sqrt{J_{\text{load}}/J_{\text{rotor}}}$. When $N_{\text{gear}} > N^*$, counter-intuitively, increasing the gear ratio decreases delivered joint acceleration because the motor spends the majority of its torque accelerating its own rotor rather than the link.

C.9.3.1 Dynamic angular jerk and torque slew limits

For constant effective inertia and constant external torque, angular jerk $j_{\text{joint}}(t) = \dddot{q}(t)$ measures the time rate of change of joint acceleration:

$$j_{\text{joint}}(t) = \frac{N_{\text{gear}}}{J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}}} \frac{d\tau_{\text{motor}}(t)}{dt}$$

Electromagnetic torque relates to stator quadrature current via torque constant K_t: $\tau_{\text{motor}} = K_t i_q$. From the stator electrical dynamics in this section, current slew rate is bounded by winding inductance L_{phase}:

$$\frac{d\tau_{\text{motor}}}{dt} = K_t \frac{di_q}{dt} = \frac{K_t}{L_{\text{phase}}} \left(V_{\text{terminal}} - K_e \omega_m - R_{\text{phase}} i_q\right)$$

For a signed voltage constrained by $|V_{\text{terminal}}| \leq V_{\text{max}}$, a conservative magnitude bound in this scalar winding model is:

$$|j_{\text{joint}}| \leq \frac{N_{\text{gear}} K_t (V_{\text{max}} + |K_e \omega_m + R_{\text{phase}} i_q|)}{L_{\text{phase}}(J_{\text{load}} + N_{\text{gear}}^2 J_{\text{rotor}})}$$

Positive and negative current slew have different available voltage headroom. A tighter directional bound retains the signs in the preceding current equation. Changing external torque, coupled joint motion, and inverter limits require additional terms.

When a tracker requests a step torque discontinuity $\Delta\tau$ across discrete timesteps, the commanded derivative $\frac{d\tau}{dt} \to \infty$ cannot be delivered instantaneously by physical silicon. The inverter enters hard voltage saturation, causing steep current slew (dI/dt), intense thermal stress on power MOSFETs, and shock loads across gear-tooth contact flanks ($\sigma_b \propto W_t/(bmY)$). Synchronous pre-actuation invariant filters evaluate numerical torque derivatives $(\tau[k] - \tau[k-1])/\Delta t$ to ensure requested actions remain within allowable jerk envelopes before latching power stages.

C.10 DC Bus Power Distribution and Energy Absorption

In data center machine learning clusters, electrical power is managed by high-capacity facility uninterruptible power supplies (UPS) and dedicated substation transformers. On mobile autonomous platforms, computational accelerators and mechanical actuators draw from a shared, finite DC power distribution network. A robotic platform's electrical power distribution network must supply acceleration surges and absorb regenerative braking energy across finite bus capacitance and harness impedance.

C.10.1 Inductive droop and capacitive decoupling

When multiple high-torque actuators accelerate simultaneously (such as a humanoid jumping or an articulated arm bracing against an unexpected payload), total load current steps by $\Delta I_{\text{load}} = \sum_{k=1}^{N_{\text{axes}}} \Delta I_k$ with steep current slew rates $dI_{\text{load}}/dt \sim 10^5$–$10^6$ A/s. The electrical wiring harness connecting the main battery to distributed joint inverters possesses parasitic loop inductance L_{bus}

(typically 1–5 µH for a 1-meter harness) and harness resistance R_{bus}. These parasitics impede source current delivery, producing instantaneous inductive rail droop at the actuator inverter terminals:

$$V_{\text{source}}(t) - V_{\text{bus}}(t) = R_{\text{bus}} I_{\text{source}}(t) + L_{\text{bus}} \frac{dI_{\text{source}}}{dt}$$

The capacitor is connected across this same bus node; its voltage change is not another independent series drop. For an ideal capacitor, $\Delta V_C(t) = V_{\text{bus}}(0) - V_{\text{bus}}(t)$. Local inverter bus capacitance C_{bus} must supply the instantaneous charge deficit between load demand and battery source current:

$$\Delta V_C(t) = \frac{1}{C_{\text{bus}}} \int_0^t \left(I_{\text{load}}(s) - I_{\text{source}}(s)\right) ds$$

If the DC bus terminal voltage drops below the drive's configured **Under-Voltage Lockout (UVLO)** threshold V_{UVLO} (e.g., 18 V on a 24 V nominal bus, or 36 V on a 48 V bus), the gate driver IC enters UVLO fault protection. It disables gate bias within microseconds to prevent power MOSFETs from operating in their linear resistive region. The resulting loss of commanded torque can let a gravity-loaded joint descend. Current decay and the holding-brake response determine the plant motion.

If the current deficit starts at ΔI_{step} and decreases linearly to zero over Δt_{step}, integrating that triangular deficit gives:

$$C_{\text{bus}} \geq \frac{\Delta I_{\text{step}} \cdot \Delta t_{\text{step}}}{2\Delta V_{\text{droop, max}}}$$

A constant deficit over the same interval removes the factor of two in the denominator. The source dynamics and capacitor ESR must also satisfy the terminal-voltage limit. Local capacitor banks combine low-ESR multi-layer ceramic capacitors (MLCCs) for high-frequency ripple filtering (20–100 kHz inverter switching) with bulk electrolytic or polymer capacitors for transient step energy buffering. The bank must also carry the RMS ripple current of the actual switching waveform. Equivalent-series-resistance losses scale with that current squared.

C.10.2 Regenerative braking surges and chopper sizing

During controlled regenerative deceleration or emergency stopping, mechanical kinetic energy $E_k = \frac{1}{2}mv^2$ (linear platform motion) or $E_k = \frac{1}{2}J_{\text{eff}}\dot{q}^2$ (articulated joint rotation) is converted back into electrical energy by the motor acting as a generator. Current flows back through the inverter bridge into the DC bus via active synchronous rectification or freewheeling body diodes.

If the battery cannot accept this regenerative current (for example, when the battery pack reaches its chemistry-specific charge limit, when battery internal resistance rises at freezing temperatures, or when safety contactors open to isolate the pack), the returned electrical energy $\eta_{\text{regen}} E_k$ is forced into the local bus capacitor C_{bus}. The resulting terminal voltage surge follows the ideal capacitor relationship:

$$\Delta E_C = \frac{1}{2} C_{\text{bus}} \left(V_{\text{final}}^2 - V_{\text{initial}}^2\right) = \eta_{\text{regen}} E_k$$

$$V_{\text{final}} = \sqrt{V_{\text{initial}}^2 + \frac{2\eta_{\text{regen}} E_k}{C_{\text{bus}}}}$$

An overvoltage protection trip and semiconductor breakdown are different thresholds. Exceeding the configured trip requests the assessed protective response; exceeding device voltage ratings can cause avalanche stress or damage. Disabling gate commands does not by itself absorb returned energy.

To prevent overvoltage destruction, motor drives incorporate an active **dynamic brake chopper**. When DC bus voltage crosses a hardware clamp threshold $V_{\text{clamp,on}}$, a high-power semiconductor switch (MOSFET or IGBT) modulates an auxiliary low-inductance braking resistor R_{brake} with hysteresis:[9]

$$R_{\text{brake}} \leq \frac{V_{\text{clamp}}^2}{P_{\text{regen, peak}}}$$

[9] **Dynamic Braking Choppers**: A qualified chopper can dissipate returned energy when the battery is unavailable as a sink. Its threshold, switch and resistor ratings, pulse duration, and thermal recovery must be tested. Regeneration and chopper operation are not inherent to every emergency stop; safe torque off can instead leave a motor coasting.

$$P_{\text{regen, peak}} \approx \eta_{\text{regen}} \cdot \tau_{\text{brake}} \cdot \omega_{\max}$$

The chopper comparator switches the resistor on when $V_{\text{bus}} \geq V_{\text{clamp,on}}$ and turns off when $V_{\text{bus}} \leq V_{\text{clamp,off}}$, preventing rapid oscillation. In addition to peak power sizing, the braking resistor must be rated for single-pulse adiabatic energy absorption:

$$E_{\text{pulse}} \geq \eta_{\text{regen}} E_k$$

with a thermal cooldown time constant that prevents cumulative thermal failure during repeated stop-and-go duty cycles.

C.11 Real-Time Fieldbuses and Lock-Free Synchronization

In distributed machine learning clusters, networks are built on high-bandwidth fabrics (such as 100/400 Gbps RoCEv2 or InfiniBand) optimized for aggregate data-parallel throughput via credit-based flow control. In physical AI systems, however, the onboard communication fabric must simultaneously multiplex two diametrically opposed traffic classes:

1. *High-bandwidth best-effort perception streams*: Multi-gigabit video from high-resolution GMSL2/GigE Vision cameras (2–8 Gbps) and 3D lidar point clouds (100–1000 Mbps) streaming into the AI accelerator's unified memory for foundation model inference.
2. *Hard real-time periodic control traffic*: Deterministic 1 kHz joint setpoint commands and high-resolution torque/position feedback (100–500 kbps) exchanged with distributed motor inverters.

If these two traffic classes share a naive best-effort Ethernet fabric with first-in, first-out (FIFO) queueing, multi-megabyte perception bursts cause severe queue congestion and buffer bloat. A burst of video frames can delay a motor control packet by 5–20 ms, violating the 1 ms control deadline, starving the inverter current loop, and tripping hardware emergency stops. Coordinating distributed drives while ingesting perception streams requires an architectural separation of fabrics, deterministic bus schedules, and bounded lock-free publication protocols.

C.11.1 Fieldbus serialization and schedulability

On shared physical buses, N_{axes} actuator drives transmit telemetry and receive setpoints cyclically. To prevent perception traffic from corrupting motor timing, physical AI architectures use dedicated deterministic fieldbuses (CAN 2.0B, CAN FD, or EtherCAT) or time-scheduled switched Ethernet (IEEE TSN).

Classical **Controller Area Network (CAN 2.0B, ISO 11898-1)** uses differential signaling over a shielded twisted pair ($V_{\text{CANH}} - V_{\text{CANL}} \approx 2.0$ V for dominant logic 0, and 0 V for recessive logic 1):

- *Arbitration Mechanics*: Nodes access the physical medium using non-destructive bitwise arbitration. When two nodes transmit simultaneously, a dominant bit overrides a recessive bit. A node transmitting a recessive bit that samples a dominant bit on the physical bus immediately ceases transmission and yields the bus to the higher-priority frame without message corruption.
- *Framing and Payload Constraints*: Standard CAN 2.0B frames support either an 11-bit standard or 29-bit extended identifier, with payload length strictly capped at $B_{\text{payload}} \leq 8$ bytes via a 4-bit Data Length Code (DLC).
- *Bit-Stuffing and Latency*: Classical CAN inserts an opposite bit after a run of five identical bits in the stuffed region. Serialization time includes the identifier format, payload, CRC, stuffing, and interframe spacing. Let $B_{\text{wire},i}$ be that complete bit count for frame i; then $C_i = B_{\text{wire},i}/R_{\text{bit}}$. Extended identifiers consume more bits than standard identifiers.
- *The Scalability Bottleneck*: The necessary load condition is $\sum_i C_i/T_i \leq 1$, but meeting deadlines also requires priority and blocking analysis. There is no universal utilization threshold at which CAN queueing diverges. Axis capacity depends on command and feedback packing, periods, identifiers, and tolerated retransmissions; one frame per axis is a workload assumption.

Controller Area Network with Flexible Data-Rate (**CAN FD**, ISO 11898-1:2015) overcomes the payload and bandwidth limits of classical CAN through dual bit-rate switching and payload expansion up to 64 bytes:

- *Arbitration Phase*: Transmitted at the nominal bit rate R_{nom} (typically 500 kbps or 1 Mbps). The bit time $T_{\text{bit,nom}} = 1/R_{\text{nom}}$ must accommodate physical cable propagation delays across the robot chassis and transceiver delays ($2t_{\text{prop}} + t_{\text{transceiver}}$), allowing non-destructive bitwise priority arbitration. Nodes drive the bus using dominant or recessive states. If two nodes transmit simultaneously, the lower numerical message identifier (higher logical priority) overwrites recessive bits, winning the bus without collision or frame destruction.
- *Data Phase*: Once arbitration is established, the winning node asserts the Bit Rate Switch (BRS) bit, accelerating transmission to the data bit rate R_{data} (typically 2–8 Mbps). The data phase no longer needs round-trip arbitration within each bit. Signal integrity, receiver sampling, and transceiver delay compensation still constrain its rate.
- *Bit-Stuffing Overhead*: Dynamic stuff bits are inserted after 5 consecutive identical bits in arbitration, while the CRC field incorporates fixed stuff bits with an explicit stuff count. In the worst-case bit pattern, bit stuffing can inflate frame transmission duration by up to 20 percent.

The serialization duration for frame i is:

$$t_{\text{frame},i} \approx \frac{B_{\text{arb},i}}{R_{\text{nom}}} + \frac{B_{\text{data},i}}{R_{\text{data}}} + t_{\text{stuff},i} + t_{\text{interframe},i}$$

where $B_{\text{arb},i}$ is the arbitration field length (identifier, control bits), $B_{\text{data},i}$ is the payload and CRC length, $t_{\text{stuff},i}$ accounts for stuff bits, and $t_{\text{interframe},i}$ is the minimum 3-bit inter-frame spacing.

In a **CAN FD response-time budget**, a higher-priority release cannot preempt a frame already transmitting. For a simplified error-free fixed-priority message set with no prior instance of frame i pending, the following conservative recurrence charges higher-priority interference across the whole response window:

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i + J_j + T_{\text{bit}}}{T_j} \right\rceil C_j$$

Here $C_i = t_{\text{frame},i}$, J_j is higher-priority release jitter, and T_{bit} accounts for arbitration at the boundary. Iterate from $C_i + B_i$ to convergence or a deadline overrun. Add the message's own release jitter if its deadline starts before it becomes ready. The lower-priority blocking term is:

$$B_i = \max_{k \in lp(i)} C_k$$

Set $B_i = 0$ when no lower-priority frame exists. This simplified test is not an exact general CAN schedulability test: queued prior instances, controller transmit-buffer behavior, errors, and retransmissions require a matching busy-period analysis. A long CAN FD payload improves packing efficiency but can also increase the non-preemptive blocking seen by a short urgent command.

For high-axis-count systems (such as humanoids with 30 to 50 degrees of freedom), **EtherCAT** (ETG.1000) achieves deterministic 1 kHz to 10 kHz cycle times over standard 100BASE-TX full-duplex Ethernet ($R_{\text{link}} = 100$ Mbps). Rather than transmitting individual frames per axis, EtherCAT structures communication as an active logical ring:

1. The real-time master (executing on the safety microcontroller or `PREEMPT_RT` Linux host) transmits a single standard Ethernet frame (EtherType `0x88A4`) traversing all slave drives in a physical daisy chain.
2. Each drive integrates an EtherCAT Slave Controller (ESC) application-specific integrated circuit (ASIC) (such as Beckhoff ET1100 or field-programmable gate array (FPGA) IP). As the frame's bits pass through the ESC's physical shift registers, the ESC **processes data on the fly**:
    - It reads commanded setpoints (joint target position, velocity, and feedforward torque) directly from its allocated byte offset in the datagram.

- It writes actual measured feedback (encoder position, phase current, and drive status flags) into the moving frame.

3. Each slave contributes a forwarding delay that depends on its controller and ports; the frame returns from the final slave along the return path. The example below assigns an illustrative delay per slave.

The cycle must accommodate the following round-trip budget for S slave drives, with B_{frame} measured in bits and each delay bounded for the topology:

$$T_{\text{cycle}} \geq \frac{B_{\text{frame}}}{R_{\text{link}}} + t_{\text{master}} + \sum_{s=1}^{S} t_{\text{slave},s} + t_{\text{prop}} + t_{\text{guard}}$$

For 30 axes, the assumed command and feedback sizes give 600 bytes of process data. An illustrative 750 bytes frame takes 60 µs to serialize at 100 Mb/s. Charging 1 µs per slave adds 30 µs, so the subtotal is already 90 µs. Master processing, propagation, omitted wire overhead, and guard time must still fit the cycle. This partial sum is not a measured round-trip bound. EtherCAT distributed clocks align local device timers through measured offsets and propagation compensation. The qualified synchronization error and arrival deadline determine when a drive may use the frame. Clock synchronization does not remove frame transit time.

C.11.2 Time-sensitive networking and scheduled ethernet gates

While EtherCAT processes frames on the fly through specialized slave controllers, modern multi-camera and lidar robotic systems require multi-gigabit Ethernet fabrics that transport high-bandwidth vision data alongside hard real-time actuator commands. IEEE **Time-Sensitive Networking (TSN)** provides deterministic Ethernet via standards that bound queuing jitter and reserve transmission windows:

The **IEEE 802.1Qbv Time-Aware Shaper (TAS)** coordinates traffic transmission using hardware-controlled transmission gates associated with each priority egress queue (traffic classes 0 to 7). A cyclic **Gate Control List (GCL)**, synchronized across all network bridges via IEEE 802.1AS (generalized Precision Time Protocol), switches gates open or closed at precise physical time epochs. By reserving dedicated time windows exclusively for high-priority Scheduled Traffic (ST), TAS eliminates queuing interference from lower-priority Best-Effort (BE) traffic.

Standard Ethernet frames are non-preemptive: once a transmission begins, it cannot be interrupted on physical copper or optical links. If a lower-priority best-effort packet begins transmission immediately before an ST time slice opens, the scheduled frame experiences transmission blocking of up to the maximum frame duration. To prevent this, TAS enforces a **guard band** before each scheduled traffic window:

$$t_{\text{guard}} \geq \frac{B_{\text{BE,wire}}}{R_{\text{link}}}$$

Here $B_{\text{BE,wire}}$ is the maximum non-preemptible wire occupancy in bits, including the applicable framing overhead. A payload MTU in bytes cannot be divided directly by a bit rate. The configured guard must also cover clock error and implementation timing uncertainty. During it, a best-effort frame may start only if it can finish before the reserved window.

IEEE 802.1Qbu and IEEE 802.3br frame preemption reduce blocking by interrupting eligible traffic at permitted fragment boundaries. The remaining bound includes the maximum non-preemptible fragment and its overhead, not merely a nominal minimum fragment size. Both the link partners and the schedule must support the mechanism. Preemption saves reserved idle time at the cost of fragmentation overhead and additional configuration obligations.

The four primary real-time communication protocols differ in framing, arbitration, clock synchronization, and axis-count scaling (table C.7).

Table C.7: Comprehensive Architectural Synthesis of Real-Time Fieldbus Protocols: Comparison of CAN 2.0B, CAN FD, EtherCAT, and IEEE TSN across physical layer, framing, arbitration, jitter, and multi-axis control capacity.

Attribute / Metric	CAN 2.0B (ISO 11898-1)	CAN FD (ISO 11898-1:2015)	EtherCAT (ETG.1000)	IEEE TSN (802.1Qbv / 802.1Qbu)
Physical Layer	Differential twisted pair	Differential twisted pair	100BASE-TX full-duplex Ethernet	1000BASE-T / 10GBASE-T switched Eth
Physical Topology	Linear multidrop bus	Linear multidrop bus	Daisy-chain / logical ring	Switched star / tree / mesh
Raw Line Bit Rate	1 Mbps max	1 Mbps arb / 2–8 Mbps data	100 Mbps full-duplex	1–10 Gbps full-duplex
Maximum Payload (MTU)	8 bytes	64 bytes	Up to 1486 bytes per datagram	1500 bytes (9000 B jumbo)
Arbitration Mechanism	Bitwise non-destructive priority	Bitwise non-destructive with bit-rate switch	Hardware on-the-fly ESC processing	Time-Aware Shaper (GCL) + frame preemption
Delay qualification	Frame priorities, load, blocking, errors	Nominal/data rates, packing, blocking, errors	Frame size, topology, slave and master delays	Gate schedule, path, clock error, preemption
Hardware Clock Sync	No network clock service in CAN framing	No network clock service in CAN FD framing	Distributed Clocks; qualified residual error	IEEE 802.1AS; qualified residual error
Axis-count scaling	Per-axis frame count and period	Payload packing and response deadlines	Process-data bytes and number of slaves	Reserved traffic windows and link capacity
Primary Robotic Domain	Auxiliary sensors, low-rate IO	Quadruped leg clusters, gripper drives	Multi-axis humanoids, surgical arms	Sensor fusion ingress + safety backbone

For either CAN variant, sum every command and feedback frame in the required cycle, then account for competing traffic, non-preemptive blocking, and the declared error model. EtherCAT avoids a separate frame for each axis, but its process-data size and forwarding delay still grow with the topology. TSN reserves service along a path. An end-to-end bound must include each hop's schedule, serialization, forwarding, and synchronization uncertainty. These distinct budgets, rather than a protocol name or a universal axis count, support the selection in section 4.5.

C.11.3 Lock-free seqlocks for multi-rate shared memory

When a producer updates multiword state read by a deadline-bound enforcer, waiting on a producer-held mutex can violate the reader's deadline. The single-slot mailbox in algorithm C.1 uses a **sequence counter**, which marks a single writer's publication in progress with an odd value and a completed publication with an even value:

1. **Writer Protocol**:
   - Publish an odd sequence value, write the payload, and make it visible through the platform's coherent or cache-maintained memory contract.
   - Publish the next even value with release ordering.
2. **Reader Protocol**:
   - Read S_1 with acquire ordering. If it is odd, reject this publication without waiting.
   - Make one bounded copy to scratch memory using the platform's required load ordering, then read S_2.
   - Stage the candidate only if $S_1 = S_2$ and both are even, and the copied frame passes integrity, timestamp, and requested-expiry checks. Full admission remains separate. Retain the active command only while its own state, evidence, lease, and stopping margin permit; otherwise enter the state-matched fallback.

The algorithm is platform pseudocode, not portable C++ with a non-atomic payload: racing ordinary C++ loads and stores would be undefined behavior even if the sequence check later rejects

the copy. The platform must define safe concurrent copying, cache maintenance, aligned counter atomicity, and wraparound behavior. On the machine's discrete microcontroller, the writer is the link-receive interrupt and the reader is the permission tick on the same core, so the counter protocol needs only compiler ordering between them. A data memory barrier matters where a DMA engine or a second master writes the same memory. Arm M-profile cores provide that barrier only as the full-system `DMB` (ARM Limited 2012), so the `OrderingFence` steps below stand for whatever the target requires rather than an A-profile shareability option. The reader's one-attempt execution time can be bounded on the target hardware; success probability depends on the writer's schedule and cannot be inferred from payload size alone. Conventional Linux seqlock readers may retry, as the kernel documentation shows, but a real-time permission gate cannot use an unbounded retry loop.

Algorithm C.1 Single-Slot Trajectory Seqlock Baseline

Require: proposal P (proposal header with sequence_id, t_evidence_ns, sync_error_ns, t_expire_ns; waypoints $\mathbf{w}_{1..H}$; crc32), scratch buffer B_{scratch} in static SRAM reserved for this staging attempt, mailbox $(c_{\text{seq}}, \text{Slot}_{\text{shared}})$ in the microcontroller's own SRAM, written by the link-receive interrupt after the frame's CRC and sequence checks and read by the permission tick, current time t_{now} and evidence-age budget $t_{\text{age,max}}$ (the configured upper-age ceiling for candidate staging), both on the safety microcontroller's clock

Ensure: candidate snapshot in B_{scratch} or rejection; the active spline remains unchanged

function PublishReceivedChunk(P) ▷ *runs in the link-receive interrupt*
  $c_{\text{curr}} \leftarrow$ AtomicLoadExplicit(c_{seq}, memory_order_relaxed)
  AtomicStoreExplicit($c_{\text{seq}}, c_{\text{curr}} + 1$, memory_order_relaxed) ▷ *mark odd: write in progress*
  OrderingFence() ▷ *odd counter ordered before payload stores*
  CopyMemory($\text{Slot}_{\text{shared}}$.payload, P, sizeof(P)) ▷ *copy multi-word trajectory*
  OrderingFence() ▷ *payload stores ordered before even counter*
  AtomicStoreExplicit($c_{\text{seq}}, c_{\text{curr}} + 2$, memory_order_release) ▷ *mark even: publish committed*
end function
function StageLatestActionChunk($B_{\text{scratch}}, t_{\text{now}}$)
  $c_{\text{start}} \leftarrow$ AtomicLoadExplicit(c_{seq}, memory_order_acquire) ▷ *load initial counter*
  if c_{start} is odd **then**
    return STATUS_WRITER_ACTIVE ▷ *writer active; retain prior spline*
  end if
  OrderingFence() ▷ *prevent payload reads hoisting before counter*
  CopyMemory(B_{scratch}, $\text{Slot}_{\text{shared}}$.payload, sizeof(Payload)) ▷ *tentative copy; active spline unchanged*
  OrderingFence() ▷ *payload reads complete before c_{end}*
  $c_{\text{end}} \leftarrow$ AtomicLoadExplicit(c_{seq}, memory_order_acquire) ▷ *re-read sequence counter*
  if $c_{\text{start}} \neq c_{\text{end}}$ **then**
    return STATUS_TORN_READ ▷ *concurrent write detected*
  end if
  if ComputeCrc32(B_{scratch}.bytes_before_crc) $\neq B_{\text{scratch}}$.crc32 **then**
    return STATUS_CHECKSUM_INVALID
  end if
  if $t_{\text{now}} \geq B_{\text{scratch}}$.t_expire_ns **then**
    return STATUS_REQUEST_EXPIRED ▷ *requested expiry is absolute*
  end if
  if $t_{\text{now}} - B_{\text{scratch}}$.t_evidence_ns $+ B_{\text{scratch}}$.sync_error_ns $> t_{\text{age,max}}$ **then**
    return STATUS_EVIDENCE_STALE ▷ *conversion error is added to every age*
  end if
  return STATUS_CANDIDATE_STAGED ▷ *full admission must finish before activation*
end function

The proposal header and chunk payload of section 4.4.1 cross the proposal boundary through this mailbox (section 4.3). Section C.15.1 gives their byte layout. A successful read yields a candidate in owned staging memory. Admission separately checks producer sequence and lineage, the complete trajectory and stop, and the current seam state before activation (section 11.8). Scheduling that work must preserve the permission tick; the scratch slot cannot be reused while admission still reads it. At activation the MCU rereads its clock, rechecks evidence and deadlines, and records the successful admission time. A mailbox read neither renews the active lease nor resets the evidence epoch.

C.12 Camera Projective Geometry and Covariance Propagation

In computer vision for machine learning, neural models typically process normalized pixel tensors to predict 2D bounding boxes or semantic segmentation masks. In physical AI systems, however, visual observations must be grounded as physical objects in continuous Euclidean 3D metric space to evaluate spatial clearance buffers D_{clear} and enforce collision-avoidance invariants. Perception pipelines map continuous Euclidean 3D scenes onto discrete 2D camera photosite grids, and unproject pixel detections back into metric space under quantified measurement uncertainty.

C.12.1 Intrinsic projection matrix and radial distortion

Under the standard pinhole camera model, a 3D point $\mathbf{p}_c = [X_c, Y_c, Z_c]^\top$ in camera coordinates projects onto pixel coordinates $[u, v]^\top$ via the intrinsic calibration matrix $\mathbf{K}$:[10]

$$\tilde{\mathbf{m}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_c}\mathbf{K}\mathbf{p}_c = \frac{1}{Z_c}\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}$$

where (f_x, f_y) are focal lengths in pixel units and (c_x, c_y) is the principal point. Non-linear radial lens distortion is modeled by distortion coefficients (k_1, k_2):

$$r^2 = x_n^2 + y_n^2, \quad x_d = x_n(1 + k_1 r^2 + k_2 r^4), \quad y_d = y_n(1 + k_1 r^2 + k_2 r^4)$$

where $(x_n, y_n) = (X_c/Z_c, Y_c/Z_c)$ are normalized image coordinates.

[10] **Pinhole Intrinsic Projection**: The pinhole model projects 3D points onto a 2D sensor through calibrated intrinsics. Under fixed pixel noise, the lateral variance after depth unprojection grows with squared distance; depth-error and calibration terms follow their own models. Uncalibrated intrinsics can bias a clearance estimate and must be included in its error envelope.

C.12.2 Metric unprojection and spatial covariance propagation

When unprojecting a 2D pixel observation $[u, v]^\top$ with measured depth Z_c into 3D camera coordinates, photosite measurement noise $\Sigma_{\text{meas}} = \text{diag}(\sigma_u^2, \sigma_v^2, \sigma_z^2)$ propagates through the unprojection Jacobian $\mathbf{J}_{\text{unproj}}$:

$$\mathbf{p}_c = \begin{bmatrix} (u - c_x)Z_c/f_x \\ (v - c_y)Z_c/f_y \\ Z_c \end{bmatrix}, \quad \mathbf{J}_{\text{unproj}} = \frac{\partial \mathbf{p}_c}{\partial [u, v, Z_c]} = \begin{bmatrix} \frac{Z_c}{f_x} & 0 & \frac{u - c_x}{f_x} \\ 0 & \frac{Z_c}{f_y} & \frac{v - c_y}{f_y} \\ 0 & 0 & 1 \end{bmatrix}$$

The propagated camera-frame spatial covariance is:

$$\Sigma_c = \mathbf{J}_{\text{unproj}} \Sigma_{\text{meas}} \mathbf{J}_{\text{unproj}}^\top$$

Evaluating the diagonal terms shows that lateral positional variance along the camera X-axis scales with the square of depth:

$$\sigma_{x_c}^2 \approx \left(\frac{Z_c}{f_x}\right)^2 \sigma_u^2 + \left(\frac{u - c_x}{f_x}\right)^2 \sigma_z^2$$

For fixed pixel noise, the first variance contribution scales with Z_c^2, so its standard deviation grows linearly with distance. The depth-error contribution follows the depth sensor's own model. Transforming this 3D point into the robot body frame via rigid extrinsic transform $\mathbf{T}_{bc} = [\mathbf{R}_{bc}, \mathbf{p}_{bc}; \mathbf{0}^\top, 1]$ yields the spatial body covariance:

$$\Sigma_{\text{body}} = \mathbf{R}_{bc} \Sigma_c \mathbf{R}_{bc}^\top + \Sigma_{\text{ext}}$$

where Σ_{ext} models independent extrinsic uncertainty under this first-order approximation; correlations require the corresponding cross terms. A probabilistic clearance buffer may scale with $\sqrt{\lambda_{\max}(\Sigma_{\text{body}})}$ at a declared tail risk. A deterministic permission check instead needs a justified bounded-error envelope; covariance alone does not give a worst-case bound.

C.12.3 Zhang's planar homography and camera calibration

Zhang's calibration technique computes the camera intrinsic matrix $\mathbf{K}$ by observing a planar calibration rig (such as a checkerboard pattern) across $N \geq 3$ distinct orientations (Zhang 2000). Without loss of generality, assume the model plane lies on $Z = 0$ in world coordinates. A 3D model point $\mathbf{M} = [X, Y, 0, 1]^\top$ maps to a 2D image point $\tilde{\mathbf{m}} = [u, v, 1]^\top$ through a 3×3 planar homography matrix $\mathbf{H}$:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

where $\mathbf{H} = [\mathbf{h}_1\ \mathbf{h}_2\ \mathbf{h}_3] = \lambda \mathbf{K} [\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]$ with arbitrary nonzero scalar λ.

C.12.3.1 Orthogonality constraints on the absolute conic

Because the rotation vectors $\mathbf{r}_1$ and $\mathbf{r}_2$ are orthonormal columns of an $SO(3)$ matrix ($\mathbf{r}_1^\top \mathbf{r}_2 = 0$ and $\|\mathbf{r}_1\| = \|\mathbf{r}_2\| = 1$), inverting $\mathbf{K}$ yields:

$$\mathbf{r}_1 = \frac{1}{\lambda} \mathbf{K}^{-1} \mathbf{h}_1, \quad \mathbf{r}_2 = \frac{1}{\lambda} \mathbf{K}^{-1} \mathbf{h}_2$$

Imposing orthonormality on $\mathbf{r}_1$ and $\mathbf{r}_2$ defines two algebraic constraints per homography:

$$\mathbf{h}_1^\top \mathbf{K}^{-\top} \mathbf{K}^{-1} \mathbf{h}_2 = 0$$

$$\mathbf{h}_1^\top \mathbf{K}^{-\top} \mathbf{K}^{-1} \mathbf{h}_1 - \mathbf{h}_2^\top \mathbf{K}^{-\top} \mathbf{K}^{-1} \mathbf{h}_2 = 0$$

The symmetric matrix $\mathbf{B} = \mathbf{K}^{-\top} \mathbf{K}^{-1}$ represents the image of the absolute conic (IAC):

$$\mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{12} & B_{22} & B_{23} \\ B_{13} & B_{23} & B_{33} \end{bmatrix}$$

Defining the 6-vector $\mathbf{b} = [B_{11}, B_{12}, B_{22}, B_{13}, B_{23}, B_{33}]^\top$, each quadratic term $\mathbf{h}_i^\top \mathbf{B} \mathbf{h}_j$ can be rewritten as the linear inner product $\mathbf{v}_{ij}^\top \mathbf{b}$, where:

$$\mathbf{v}_{ij} = \begin{bmatrix} h_{i1} h_{j1} \\ h_{i1} h_{j2} + h_{i2} h_{j1} \\ h_{i2} h_{j2} \\ h_{i3} h_{j1} + h_{i1} h_{j3} \\ h_{i3} h_{j2} + h_{i2} h_{j3} \\ h_{i3} h_{j3} \end{bmatrix}$$

The two constraints per observed orientation become the linear system:

$$\begin{bmatrix} \mathbf{v}_{12}^\top \\ (\mathbf{v}_{11} - \mathbf{v}_{22})^\top \end{bmatrix} \mathbf{b} = \mathbf{0}$$

Stacking $N \geq 3$ images forms the homogeneous system $\mathbf{V}\mathbf{b} = \mathbf{0}$, where $\mathbf{V} \in \mathbb{R}^{2N \times 6}$. The right singular vector of $\mathbf{V}$ corresponding to the smallest singular value provides $\mathbf{b}$ up to a scale factor. The intrinsic parameters (f_x, f_y, c_x, c_y) and skew γ are then extracted, after fixing the scale and sign, via a positive-definite factorization of $\mathbf{B}$. Degenerate views or measurement noise can make that factorization invalid. Distinct orientations alone do not guarantee a well-conditioned calibration. This closed-form linear solution initializes the non-linear Levenberg–Marquardt optimization minimizing reprojection residuals over all observed fiducial corners.

C.13 Kinematic Stopping Envelopes and Information Age Lag

In distributed systems, pipeline latency represents elapsed waiting time or queuing delay. In physical AI, end-to-end software latency converts directly into irreversible physical displacement, uninspected "blind travel," during which an embodied machine traverses space with undiminished momentum before actuators can begin retarding motion. Halting an embodied machine transporting mass m at velocity v requires balancing sensorimotor information freshness, neural compute budgets, and fieldbus dispatch against physical mechanical braking deceleration.

C.13.1 Sensorimotor information age and lag distance

For a stop triggered by a specified hazard detector and stop path, let τ_{delay} bound the time from the physical state represented by the evidence until validated braking deceleration begins (section B.6.5 gives the lease-expiry form, which charges the lease interval in place of recognition and compute):

$$\tau_{\text{delay}} = t_{\text{age}} + t_{\text{detect}} + T_{\text{compute}} + T_{\text{bus}} + T_{\text{act}}$$

The five intervals partition the pre-brake timeline without overlap:

- t_{age} is the upper evidence age at a declared decision-start epoch, including capture/readout, completed processing and transport, and clock-conversion uncertainty.
- t_{detect} bounds any remaining recognition or expiry-detection interval after that epoch; work already included in evidence age is not charged twice.
- T_{compute} bounds subsequent stop-decision work and any still-required input transfer.
- T_{bus} bounds command dispatch and network queuing.
- T_{act} ends when the measured actuator begins delivering the specified deceleration, not when a command register is written.

The intervals must be disjoint and ordered along the actual stop path. Clearance and motion must share a declared epoch: if the clearance is already propagated to current time, do not subtract historical travel a second time. Under the explicit simplifying assumption that speed cannot increase before brake onset, the blind travel is bounded by $d_{\text{blind}} = v\,\tau_{\text{delay}}$. If propulsion or a slope can increase speed, use its validated acceleration bound in this interval instead.

C.13.2 Braking dynamics and friction limits

Tire-floor friction and actuator torque place upper limits on achievable deceleration. The bound $a \le \mu g$ does not establish what braking the machine will deliver. Let $a_{\text{brake}} > 0$ be a validated lower bound on *achieved* deceleration throughout the stop, for the declared load, temperature, slope, brake condition, and surface. A justified friction floor μ_{min} may support that bound, but does not replace brake tests. With constant or stronger braking after onset, a localization bound δ_{loc}, a fixed protective clearance δ_{margin}, and a tracking-error bound ϵ_{track},

$$d_{\text{stop}}(v) = v\,\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} + \epsilon_{\text{track}}.$$

Motion is admitted only when $d_{\text{stop}}(v) \le D_{\text{clear}}$, where D_{clear} is taken as a validated lower bound on available path clearance. This is a constant-deceleration reference model. The smooth stop in section 12.3 ramps acceleration from and back to zero, so it does not satisfy a strictly positive deceleration floor throughout braking. For that profile, replace $v^2/(2a_{\text{brake}})$ with the integral of its validated speed trajectory and use its full duration. The sampled controller must reserve intersample travel and a state-matched stop before the viable region is lost.

C.13.3 Kinetic energy derating and friction collapse

In an ideal friction-limited model, lowering μ from μ_{nom} to a positive μ_{fault} changes the braking-distance ratio to $\mu_{\text{nom}}/\mu_{\text{fault}}$. That ratio is illustrative; actual deceleration can also be limited by brake hardware or control. If surface friction is not sensed in time, derive a_{brake} from a controlled

surface condition and inspection schedule, exclude degraded surfaces, or withhold motion. For the constant-speed pre-brake model above, solving the clearance inequality gives

$$v_{\text{clamp}} = -a_{\text{brake}}\tau_{\text{delay}} + \sqrt{(a_{\text{brake}}\tau_{\text{delay}})^2 + 2a_{\text{brake}}\left(D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}} - \epsilon_{\text{track}}\right)}.$$

This clamp is valid only when D_{clear} exceeds the three fixed allowances, the braking lower bound and timing upper bounds hold, and the speed bound is enforced before entering the hazard corridor. No positive clamp follows if the available deceleration floor is unknown or zero.

C.14 Machine Data Sheet and Record Register

Every calculation on the running machine draws its inputs from one registry entry, the warehouse mobile manipulator at its aisle site. This section collects those inputs in one place, so a chapter can cite a value by name rather than restate it. It also collects the records the chapters produce. The machine card of section 1.4.2 prints the parameters used most often; the data sheet below lists them all.

C.14.1 The machine data sheet

Each running-machine input carries its evidence category in the reader guide's vocabulary (table C.8). A derived value inherits the weakest category among its inputs, so most derived rows remain illustrative evidence even though they are computed exactly. A row marked *needs source* names the measurement that would supply it; until that measurement exists, any claim that depends on the row is open, as the tip-over check on the credible deceleration is. The safety laser scanner carries the certified stop for a person at the rack end, while the perception LiDAR feeds only the permission path's unrated admission checks above it. The safety scanner's rows describe a representative certified unit, not a qualified selection, and the stopping budgets in this book are the permission path's, not the scanner's. A certified protective field is sized with the scanner's own response time under ISO 13855 (International Organization for Standardization 2024a).

Table C.8: The running machine's data sheet: Every input the chapters take from the warehouse mobile manipulator and its aisle site, grouped by subsystem, with its evidence category and the MLSysIM field that holds it (`RM` is the machine, `AISLE` the site scenario). Values print from the registry when the book is built.

Quantity	Value	Category	MLSysIM field	Note
Body: base, arm, and drivetrain				
Base mass	300 kg	Illustrative	`RM.base.mass`	Class-profile value, gross
Arm mass	18 kg	Datasheet	`RM.arm.mass`	Panda-class arm
Tote-rack payload capacity	50 kg	Chosen	`RM.onboard_payload_capacity`	
Loaded mass m	368 kg	Derived	`RM.loaded_mass`	Upper bound: base, arm, full tote rack
Footprint width	1 m	Illustrative	`RM.footprint_width`	
Drive limit v_{max}	1.5 m/s	Illustrative	`RM.base.max_velocity`	Class-profile value
Credible deceleration a_{brake}	2 m/s²	Illustrative	`AISLE.a_brake`	Loaded, dry floor; checked against traction and drive torque
Friction a_{brake} needs, all weight on driven wheels	0.204	Derived	`calc_required_friction(AISLE.a_brake)`	
Largest driven-wheel load share φ	0.815	Derived	`RM.drive_wheel_load_share_max`	Hub motors' load rating over loaded mass
Friction a_{brake} needs at that share	0.250	Derived	`calc_required_friction(AISLE.a_brake, RM.drive_wheel_load_share_max)`	

Quantity	Value	Category	MLSysIM field	Note
Measured driven-wheel load share	needs source	Measured	`RM.drive_wheel_load_share`	*needs source*: corner-scale weighing of the loaded machine
Loaded center-of-gravity height h_{cg}	needs source	Measured	`RM.cg_height`	*needs source*: tilt-table or corner-scale measurement; tip-over unchecked
Center of gravity to leading support, arm stowed	needs source	Measured	`RM.tip_arm_stowed`	*needs source*: measured with the center-of-gravity height
Stop jerk bound j_{max}	7.11 m/s^3	Derived	`calc_c2_stop_suffix(v_max, a_brake)['peak_jerk']`	Stipulated: the reference polynomial stop's peak jerk
Drive wheel radius	133 mm	Datasheet	`RM.drive_unit.wheel_radius`	Hub motor
Driven wheels	2	Chosen	`RM.drive_wheel_count`	
Hub-motor torque constant K_t	2.4 N·m/A	Derived	`RM.drive_unit.torque_constant`	Datasheet peak torque over peak current
Drive current limit	25 A	Chosen	`RM.drive_current_limit`	Set at the motor's datasheet peak
Hub-motor rated current	8.5 A	Datasheet	`RM.drive_unit.rated_current`	
Hub-motor load rating per pair	300 kg	Datasheet	`RM.drive_unit.load_rating_per_pair`	
Hub-motor rotor inertia	needs source	Datasheet	`RM.drive_unit.rotor_inertia`	*needs source*: not published; motor mass bounds J_{rotor}/r^2
Arm repeatable peak joint torque	87 N·m	Datasheet	`RM.arm.max_torque`	
Arm free-space TCP speed limit	1 m/s	Chosen	`RM.tcp_speed_limit`	
Servo axes on the fieldbus	9	Chosen	`RM.servo_axes`	Seven arm joints and two drive wheels
Body: drive bus and permission rail				
Drive bus, full charge	54 V	Datasheet	`RM.drive_bus_full_charge`	48 V-class pack
Drive bus, nominal	47.7 V	Datasheet	`RM.drive_bus_nominal`	
Drive bus, depleted	42 V	Datasheet	`RM.drive_bus_battery_low`	
Drive-bus source resistance R_{bus}	60 mΩ	Illustrative	`RM.drive_bus_resistance`	Needs a measured cell, BMS, fuse, and harness value
Converter input lockout	33 V	Datasheet	`RM.drive_bus_uvlo`	48-to-24 V converter feeding the application processor
Converter continuous input maximum	67.2 V	Datasheet	`RM.drive_bus_converter_max`	
Converter efficiency	0.91	Datasheet	`RM.drive_bus_converter_efficiency`	
Brake-chopper clamp	60 V	Chosen	`RM.drive_bus_clamp`	
Inverter switch rating	100 V	Chosen	`RM.drive_bus_switch_rating`	
Coincident inference and drive current step	70 A	Illustrative	`AISLE.i_rail_transient`	
Permission-rail hold-up	2 s	Chosen	`RM.permission_rail_holdup`	
Permission-rail load	70 W	Illustrative	`RM.permission_rail_load`	
Permission store, lowest charge	21.5 V	Illustrative	`RM.control_rail_battery_low`	

Quantity	Value	Category	MLSysIM field	Note
Permission-load regulator dropout	18 V	Illustrative	`RM.control_rail_dropout`	
Spring-brake engage time, fast end	30 ms	Illustrative	`RM.spring_brake_engage_min`	
Spring-brake engage time, slow end	80 ms	Illustrative	`RM.spring_brake_engage_max`	
Brain: proposers and application processor				
Application-processor memory bandwidth	204 GB/s	Datasheet	`RM.brain_soc.memory.bandwidth`	
Sustained fraction of that bandwidth	0.70	Illustrative	`RM.dram_efficiency`	
Intent-model parameters	7B	Datasheet	`RM.intent_model.parameters`	OpenVLA-7B
Intent-model stored weight precision	0.5 B/param	Chosen	`RM.intent_weight_bytes_per_param`	INT4 weight-only quantization
Intent-model input tokens	288	Illustrative	`RM.intent_input_tokens`	
Intent-model prefill utilization	0.50	Illustrative	`RM.intent_compute_utilization`	
Intent-model latency	336 ms	Derived	`RM.intent_inference_latency`	Prefill plus decode
Published intent-model latency, same board	336 ms	Measured	`RM.intent_reference_latency`	Published measurement, not this machine
Intent-model rate	2 Hz	Chosen	`RM.intent_rate`	Validated against the computed latency
Chunk-policy parameters	80M	Datasheet	`RM.chunk_model.parameters`	ACT
Chunk-policy inference, p99	40 ms	Illustrative	`RM.chunk_inference_latency`	
Chunk-policy renewal rate	20 Hz	Chosen	`RM.chunk_rate`	
Chunk renewal (replan) period	50 ms	Derived	`RM.chunk_period`	Whole number of setpoint steps
Setpoint step Δt	25 ms	Chosen	`RM.chunk_step`	
Chunk horizon H	16	Chosen	`RM.chunk_horizon`	
Nervous system: timing and leases				
Permission-loop rate	1 kHz	Chosen	`RM.permission_rate`	
Permission-loop period T_{tick}	1 ms	Derived	`AISLE.t_tick`	
Drive current-loop rate	20 kHz	Chosen	`RM.current_loop_rate`	
Fieldbus cycle T_{bus}	1 ms	Illustrative	`AISLE.t_bus`	
Chunk lease T_{lease}	60 ms	Chosen	`AISLE.t_lease`	
Brake onset T_{act}	20 ms	Illustrative	`AISLE.t_brake_onset`	
Heartbeat rate	100 Hz	Chosen	`AISLE.heartbeat_rate`	
Heartbeat timeout	30 ms	Chosen	`AISLE.heartbeat_timeout`	
Safety-MCU self-watchdog	5 ms	Chosen	`AISLE.mcu_self_watchdog`	
Permission check, unloaded maximum	135 µs	Illustrative	`AISLE.enforcer_unloaded`	
Permission check, declared WCET	250 µs	Chosen	`AISLE.enforcer_wcet`	
Permission check deadline	400 µs	Chosen	`AISLE.enforcer_deadline`	

Quantity	Value	Category	MLSysIM field	Note
Permission check, blocking allowance	50 µs	Illustrative	`AISLE.enforcer_blocking`	
Permission check, interference allowance	40 µs	Illustrative	`AISLE.enforcer_interference`	
Serialized chunk size	4,096 bytes	Illustrative	`AISLE.chunk_wire_bytes`	Waypoints, header, and evidence epoch
Board-link clock	25 MHz	Illustrative	`AISLE.board_link_clock`	SPI, one bit per clock
Board-link serialization	1.31 ms	Derived	`AISLE.t_board_link`	Chunk bits over the link clock
Chunk deserialization	2 ms	Illustrative	`AISLE.t_chunk_deserialize`	
Whole-plan validation	18 ms	Illustrative	`AISLE.t_plan_validate`	
Plan activation	1 ms	Illustrative	`AISLE.t_plan_activate`	
Dispatch-to-admission allocation Δ_{da}	63.3 ms	Derived	`AISLE.t_dispatch_to_admission`	Inference, link, deserialize, validate, activate, one tick
Sensing and observation age				
Navigation-camera exposure	16 ms	Chosen	`RM.nav_camera.exposure_time`	Configured setting, not a sensor property
Navigation-camera readout	16.6 ms	Datasheet	`RM.nav_camera.readout_time`	
Sensor transport	0.8 ms	Illustrative	`AISLE.t_transport`	
DMA transfer	1.2 ms	Illustrative	`AISLE.t_dma`	
Image signal processing	2.5 ms	Illustrative	`AISLE.t_isp`	
Perception backbone	22 ms	Illustrative	`AISLE.t_backbone`	
Hand-off to the proposer	0.5 ms	Illustrative	`AISLE.t_ipc`	
Observation age at dispatch t_{age}	51.6 ms	Derived	`AISLE.perception_age`	Exposure midpoint to dispatch
Yaw rate turning into the aisle ω_z	0.5 rad/s	Illustrative	`AISLE.yaw_rate_turn`	Sets rolling-shutter and sweep skew
Clock-conversion bound ϵ_{sync}	0.2 ms	Illustrative	`AISLE.clock_conversion_bound`	Capture clock to the safety MCU's clock
Stopping budget at the rack end				
Pre-brake delay excl. Δ_{da}, τ_{delay}	133.6 ms	Derived	`AISLE.tau_delay`	$t_{\text{age}} + T_{\text{lease}} + T_{\text{tick}} + T_{\text{bus}} + T_{\text{act}}$
Rack-end clear distance D_{clear}	1.10 m	Illustrative	`AISLE.d_clear`	
Localization bound δ_{loc}	50 mm	Illustrative	`AISLE.delta_loc`	
Protective clearance δ_{margin}	100 mm	Chosen	`AISLE.delta_margin`	
Tracking-error bound ϵ_{track}	40 mm	Illustrative	`AISLE.eps_track`	
Aisle speed v_{aisle}	1.3 m/s	Chosen	`AISLE.v_aisle`	Below the derived dry-floor ceiling
Aisle width	1.30 m	Illustrative	`AISLE.aisle_width`	
Aisle length	30 m	Illustrative	`AISLE.aisle_length`	
Side clearance	0.15 m	Illustrative	`AISLE.side_clearance`	
Floor friction, dry	0.60	Illustrative	`AISLE.mu_dry`	
Floor friction, inspected floor	0.12	Illustrative	`AISLE.mu_inspected_floor`	
Floor friction, oil film	0.05	Illustrative	`AISLE.mu_oil_film`	
Human approach speed v_h	1.6 m/s	Datasheet	`AISLE.v_human_approach`	ISO 13855 value

Quantity	Value	Category	MLSysIM field	Note
Safety scanner response time	80 ms	Datasheet	`RM.safety_scanner.response_time`	Representative scanner (Keyence SZ-V32), high-speed mode, 2 scans; IEC 61496-3 Type 3, PL d. Carries the certified rack-end stop (section 8.1); the OS1-64 is unrated
Safety scanner protective-field range	5.7 m	Datasheet	`RM.safety_scanner.protective_field_range`	Same mode, 70/150 mm objects
Contact, conveyor, and coworker				
Latch plate stiffness	400 N/mm	Illustrative	`AISLE.k_latch`	
Latch approach speed	0.03 m/s	Illustrative	`AISLE.v_latch_approach`	
Fast latch approach speed	0.10 m/s	Illustrative	`AISLE.v_latch_high`	
Latch force tripwire	15 N	Illustrative	`AISLE.f_latch_tripwire`	
Contact response time	2 ms	Illustrative	`AISLE.t_contact_response`	
Latch force limit	100 N	Illustrative	`AISLE.f_latch_limit`	
Conveyor speed	0.20 m/s	Illustrative	`AISLE.v_conveyor`	
Mug slip acceleration on the conveyor	0.5 m/s^2	Illustrative	`AISLE.a_conveyor_slip`	
Grasp tolerance	15 mm	Illustrative	`AISLE.grasp_tolerance`	
Initial grasp-estimate error	3 mm	Illustrative	`AISLE.grasp_initial_error`	
Coworker hand spring constant k_H	75 N/mm	Datasheet	`AISLE.k_contact_human`	ISO/TS 15066 hand body model
Coworker hand effective mass m_H	0.6 kg	Datasheet	`AISLE.m_contact_human`	ISO/TS 15066 hand body model
Coworker contact force criterion	50 N	Chosen	`AISLE.f_contact_criterion`	
Handover TCP speed	0.10 m/s	Chosen	`AISLE.v_handover`	
Human takeover				
Perceptual cycle, nominal	100 ms	Measured	`AISLE.t_human_perceptual`	Model Human Processor
Perceptual cycle, slow bound	200 ms	Measured	`AISLE.t_human_perceptual_slow`	
Cognitive cycle, nominal	70 ms	Measured	`AISLE.t_human_cognitive`	
Cognitive cycle, slow bound	170 ms	Measured	`AISLE.t_human_cognitive_slow`	
Motor cycle, nominal	70 ms	Measured	`AISLE.t_human_motor`	
Motor cycle, slow bound	100 ms	Measured	`AISLE.t_human_motor_slow`	
Perception-brake time, expected signal	0.75 s	Measured	`AISLE.t_brake_response_expected`	Drivers
Perception-brake time, unexpected signal	1.25 s	Measured	`AISLE.t_brake_response_unexpected`	Drivers
Perception-brake time, surprise	1.5 s	Measured	`AISLE.t_brake_response_surprise`	Drivers
Take-over time, median condition	2.37 s	Measured	`AISLE.t_takeover_driving_p50`	Percentiles of condition means, not of individuals
Take-over time, mean	2.72 s	Measured	`AISLE.t_takeover_driving_mean`	
Take-over time, 90th percentile condition	4.33 s	Measured	`AISLE.t_takeover_driving_p90`	

Quantity	Value	Category	MLSysIM field	Note
Take-over time, 99th percentile condition	7 s	Measured	`AISLE.t_takeover_driving_p99`	
Console video, glass to glass	104 ms	Measured	`AISLE.t_video_glass_to_glass`	Wired best case; the site's link needs a measured value
Handshake: request	10 ms	Chosen	`AISLE.t_handshake_request`	
Handshake: acknowledge	5 ms	Chosen	`AISLE.t_handshake_acknowledge`	
Handshake: confirm	15 ms	Chosen	`AISLE.t_handshake_confirm`	
Four-phase handshake	31 ms	Derived	`AISLE.t_handshake`	Three phases plus the commit tick
In-loop takeover $T_{\Sigma,1}$	605 ms	Derived	`AISLE.t_takeover_in_loop`	Slow-bound reaction plus video and handshake
Out-of-loop takeover $T_{\Sigma,2}$	4.47 s	Derived	`AISLE.t_takeover_out_of_loop`	90th-percentile condition plus video and handshake
Slow-down speed resolution	0.05 m/s	Chosen	`AISLE.v_takeover_resolution`	
In-loop slow-down ceiling with Δ_{da}	0.797 m/s	Derived	`AISLE.v_takeover_in_loop_ceiling`	S-curve ceiling at $\tau_{\text{delay}} + T_{\Sigma,1} + \Delta_{\text{da}}$
Speed before an in-loop request	0.75 m/s	Derived	`AISLE.v_takeover_in_loop`	Ceiling rounded down to the resolution
Crawl before an out-of-loop request	0.15 m/s	Chosen	`AISLE.v_takeover_out_of_loop`	
Pre-trigger logging window	5 s	Derived	`AISLE.pre_trigger_window`	Out-of-loop takeover rounded up

DRAFT

C.14.2 The record register

A **record** is a machine-checkable statement of an assumption together with its evidence, its producer, its consumers, and the conditions that invalidate it. Sixteen records form the **record chain** of figure 1.9, one from each chapter of Parts I to IV except chapter 3, whose proposer limits enter the handoff record, and one from chapter 18. Table C.9 gives each record one name. Two names shorten after first mention and in figure art, the provenance record and the belief record. Names in the last column are retired; where one survives in figure art, it names the same record. The wire formats in section C.15 lay out the records that cross a mailbox or are signed offline.

Table C.9: The record register: The sixteen records of the record chain, the chapter that produces each, and what each binds. Each record is a claim with conditions; none is a license to operate.

No.	Record	Produced in	What it binds	Retired names
1	Limit record	chapter 2	A limit's value or distribution, operating conditions, measurement uncertainty, margin, runtime detector, and provenance with its evidence category	Body limit record
2	Handoff record	chapter 4	The chunk lease, permission tick, clock domains, observation-age budget, and the handoffs between proposer and permission path, with the coupled stopping budget	Transport budget, handoff contract
3	Provenance record	chapter 5	Collection conditions, calibration revisions, plant identity, the scenario table the data covers, and the conditions it does not cover as known absences	Provenance Record (as a proper name)
4	Policy manifest	chapter 6	Architecture, weight digests, stored precision, training-regime sequence, declared operating domain, inference contract, and omission log	Training manifest
5	Evaluation record	chapter 7	The target operating domain, trial population and protocol, finite-sample bound, coverage-gap catalog, and runtime monitor specifications	Evaluation Summary

No.	Record	Produced in	What it binds	Retired names
6	Observation contract	chapter 8	Each observation's capture epoch, clock identifier, conversion bound to the permission clock, frame and calibration revision, covariance, and health flags	Perception Evidence record
7	Belief record	chapter 9	A represented quantity with its frame, evidence and estimate epochs, error-growth bound, validity region, expiry, query-work allocation, provenance, and lifecycle state	Scene Memory State record, spatial belief contract
8	Intent lease	chapter 10	A grounded target, affordance frame, tolerance, effort ceilings, and an expiry at the earliest of the evidence, administrative, and stopping deadlines; it grants a target, never a path	Intent Specification
9	Trajectory record	chapter 11	Frame, evidence time, initial reference state, the deadlines for start, nominal end, commit, and stop end, the matched stop suffix, and the parent intent lease's expiry	Trajectory Plan record, trajectory contract
10	Enforcement record	chapter 12	The enforcement contract (thresholds, barrier coefficients, loop period, fallback routing, validity region), the admitted command issued each tick, and the audit log of decisions	Admitted Command record (as the chain record), execution token
11	Placement record	chapter 13	Execution domain, privilege, period, unloaded and loaded response maxima, interconnect and supply settings, and the digest of the loaded test	Deployment Envelope
12	Authority record	chapter 14	The authority manifest (which actor may command which channel, and when) and the per-tick authority log of who held authority and what was proposed, admitted, and delivered	Intervention Event record, intervention record, forensic authority record
13	Fault record	chapter 15	For each fault trial, the claim, envelope point, injected fault, precommitted oracle, verdict, and retained trace; release receives the fault records with their coverage deficits	Verification Matrix, fault-record set
14	Release manifest	chapter 16	The claim root, argument digest, image hashes, calibration references, upstream record digests, verdict, standing conditions, expiry, and signature	Safety case dossier
15	Approval record	chapter 17	For each decision behind an admitted action, its owner and approval; each human role and whether a person can perform it; the permission stage and reporting duties; per-group exposure and detection; records about people; and the labor the deployment moves	(none)
16	Residual-claims register	chapter 18	For each open premise, the missing observable, the hazard it permits, the operating restriction, its owner, and the closure test	Claims register (unqualified)

The chain has two strands. Records 1 through 5 state what the plant can do and what the proposer has shown it can do; records 6 through 9 carry what the proposer proposes now. They meet in the enforcement record. The placement record then tests whether the permission path keeps its deadline on the chosen silicon, and records 12 through 16 carry the result into governance.

C.15 Wire Formats for the Record Chain

A mailbox decoder needs a bounded amount of work and an unambiguous representation. The layouts below specify byte offsets, field lengths, and validation rules across the proposal boundary between application processors and the independent enforcer. Fixed headers and bounded arrays make resource requirements explicit. The chosen serialization framework alone does not determine allocation or timing behavior.

1. *Zero Dynamic Allocation Invariant*: Deserialization must never invoke dynamic memory managers (`malloc`, `free`, C++ `new`, `delete`). Dynamic heap allocators manage internal free-list bins, take global allocator mutexes, and suffer from heap fragmentation. Under memory pressure,

heap operations can invoke the `brk()` or `mmap()` system calls, triggering kernel memory compaction or page faults that stall execution for tens of milliseconds. In hard real-time control, all record slots, mailbox staging areas, scratch buffers, and circular telemetry logs must be statically sized at compile time and preallocated in dedicated static SRAM or Data Tightly Coupled Memory (DTCM) pools.

2. *Bounded Parse and Validation Work*: Header fields have fixed offsets; arrays have validated counts and configured maxima. Reading one scalar is constant-time, but copying, hashing, and validating a complete record scale with its bounded length. Full trajectory admission remains interruptible and separate from the per-tick check.
3. *Natural Alignment Across Heterogeneous ABIs*: On heterogeneous SoCs combining 64-bit application processors (ARM Cortex-A or x86_64) and 32-bit real-time microcontrollers (ARM Cortex-R52 or Cortex-M7), data types must be naturally aligned. A scalar of size k bytes must reside at an offset satisfying:

$$\text{offset} \equiv 0 \pmod{k}$$

Specifically, 64-bit timestamps and integers (`uint64_t`, `float64`) align on 8-byte boundaries ($k = 8$), 32-bit values (`uint32_t`, `float32`) align on 4-byte boundaries ($k = 4$), and 16-bit integers (`uint16_t`) align on 2-byte boundaries ($k = 2$). Explicit padding bytes between narrower fields maintain these alignment invariants across both 32-bit and 64-bit compilers.

An alignment violation has direct microarchitectural consequences: a core may trap an unsupported unaligned access or split it into multiple bus operations. The result depends on instruction width, memory attributes, and processor configuration. A safety integrity classification does not itself specify those semantics. Explicit reserved bytes, scalar encodings, endianness, and schema versions define the wire representation. A native C structure is not that representation by default. Use checked offsets or explicit serialization and verify layout on each ABI. Alignment alone also does not guarantee atomicity of a wide access on a narrower core.

Not every record in table C.9 needs a byte layout. The layouts below cover the proposal header that every proposal carries and the records that cross a mailbox or are signed offline. Runtime proposals and evidence use bounded records where they cross mailboxes or fieldbuses, while limits, evaluation, enforcement configuration, placement evidence, and release artifacts are retained under canonical encodings and linked by digest. All records use standard little-endian byte ordering for multi-byte scalars (`uint64_t`, `uint32_t`, `uint16_t`, `float32`, `float64`). Lineage is established by embedding parent SHA-256 digests. Runtime corruption is caught via IEEE 802.3 CRC-32 checksums over headers and payloads. Operator authority is authenticated via AES-128 CMAC tags and Ed25519 digital signatures. The authority log is chained with HMAC-SHA-256. Offline records (placement, fault, release, approval) are identified by the SHA-256 digest of their canonical encoding, and the record that consumes one stores that digest under the consumed record's name.

C.15.1 Proposal header and chunk payload

Every proposal that crosses the proposal boundary opens with the same header, and `payload_kind` selects the payload that follows at `0x50`. Table C.10 lays out the header with the minimal chunk payload of section 4.4.1. The chunk carries kinematic setpoints only. Torque is computed on the permission side, where the tracker derives it from the admitted reference, so a learned proposal never names a motor torque.

Table C.10: Proposal header and chunk payload: Field offsets, scalar types, units, and invariants for a chunk proposal exchanged through the seqlock mailbox of algorithm C.1. The header occupies `0x00`–`0x4F` in every proposal; the other payload layouts in this section begin at `0x50`.

Offset	Field	Scalar type	Unit	Invariant and check
`0x00`	`sequence_id`	`uint64_t`	dimensionless	Strictly increasing per producer; a record not newer than the last accepted one is rejected.
`0x08`	`t_evidence_ns`	`uint64_t`	ns, permission clock	Capture epoch of the oldest evidence the proposal rests on, converted to the safety microcontroller's clock.

Offset	Field	Scalar type	Unit	Invariant and check
0x10	t_issue_ns	uint64_t	ns, permission clock	Time the producer published the record.
0x18	t_expire_ns	uint64_t	ns, permission clock	Requested absolute expiry; admission caps it by the successful admission time plus the configured lease and by earlier evidence, parent, and stop deadlines.
0x20	frame_id	uint64_t	dimensionless	Transform-tree key and calibration revision of the setpoints' frame.
0x28	parent_digest	uint8_t[32]	SHA-256	Digest of the record this proposal was derived from; establishes lineage, authenticates nothing.
0x48	sync_error_ns	uint32_t	ns	Bound on the evidence time's conversion error; added to every age the reader checks.
0x4C	payload_kind	uint16_t	enumeration	CHUNK for this payload; INTENT, TRAJECTORY, and OPERATOR select the other payloads.
0x4E	payload_len	uint16_t	bytes	Payload length; bounds the parser before any payload byte is read.
0x50	num_waypoints	uint16_t	count	Chunk horizon H; a count outside the configured range is rejected.
0x52	num_joints	uint8_t	count	Commanded joints N_j; must equal the configured count, and payload_len must equal $4 + 8HN_j$.
0x53	reserved	uint8_t	—	Zero.
0x54	joint_positions	float32[H][N_j]	rad	Commanded joint angles within the reach limits ($q_{\min} \le q \le q_{\max}$).
$0x54 + 4HN_j$	joint_velocities	float32[H][N_j]	rad/s	Commanded joint velocities within certified ceilings ($\lVert\dot{\mathbf{q}}\rVert \le \dot{q}_{\max}$).
0x50 + payload_len	crc32	uint32_t	checksum	Last field; IEEE 802.3 CRC-32 over all preceding bytes.

C.15.2 Intent payload

The intent lease of section 10.8 travels under the proposal header with payload_kind set to INTENT (table C.11). Three header fields take specific meanings. t_evidence_ns is the parent belief record's evidence epoch, never its estimate epoch. parent_digest is that belief record's digest, which the permission path matches against a retained record. t_expire_ns is the requested expiry, which the permission path shortens to the earliest of the evidence horizon, its configured lease ceiling, and the stop deadline. The six bounds values are three conservative position-error envelopes and three task tolerances in the base frame named by frame_id. They are not an encoding of a covariance, and the producer must justify any conversion from its covariance estimate, including calibration bias and the declared tail risk.

Table C.11: Intent payload: The intent lease as a proposal-header payload. Orientation tolerances and contact-mode rules come from an independently approved task profile; the quaternion is the proposed orientation only.

Offset	Field	Scalar type	Unit	Invariant and check
0x00–0x4F	proposal header	as above	as above	table C.10, with payload_kind = INTENT.
0x50	target_pose	float64[7]	m; unit quaternion	Base-frame position and orientation; finite values, normalized quaternion, workspace prefilter.
0x88	bounds	float32[6]	m	Error envelope e, then task tolerance r; $0 \le e_i \le r_i$, each checked against the approved task profile.
0xA0	wrench_request	float32[6]	N; N·m	Three requested forces and three torques; clamped or rejected against independent task and actuator limits.

Offset	Field	Scalar type	Unit	Invariant and check
0xB8	terminal_request	uint32_t	enumeration	Requested terminal mode; the permission path selects a feasible approved response from the current load and contact state.
0xBC	crc32	uint32_t	checksum	IEEE 802.3 CRC-32 over all preceding bytes.

C.15.3 Trajectory payload

The trajectory record of section 11.8 replaces the chunk on the running machine and travels under the same proposal header, with payload_kind set to TRAJECTORY and parent_digest set to the digest of the active intent lease (table C.12). The header's t_expire_ns requests an absolute expiry. The MCU retains an admitted expiry no later than that request or the successful admission time plus the configured chunk lease. Earlier evidence and stop deadlines can shorten it further. The parent intent's expiry comes from the lease that parent_digest identifies. The MCU records admission time locally, without changing t_evidence_ns or t_issue_ns. The layout below illustrates a single polynomial stopping suffix per joint. It does not encode the piecewise S-curve worked in section 11.5.2. That profile requires its own specified bounded encoding and decoder, with decoding and validation charged to admission. The setpoint samples and the assumptions sidecar travel outside this fixed part. The permission path verifies sidecar_digest over both, validates the complete candidate in staging memory, and latches the admitted setpoints and stop suffix in protected memory before tracking begins.

Table C.12: Trajectory payload: The trajectory record as a proposal-header payload. The four deadlines are absolute times on the safety microcontroller's clock; the variable-length setpoint samples and the assumptions sidecar are bound by sidecar_digest and validated before activation.

Offset	Field	Scalar type	Unit	Invariant and check
0x00–0x4F	proposal header	as above	as above	table C.10, with payload_kind = TRAJECTORY.
0x50	sample_period_ns	uint32_t	ns	Setpoint cadence Δt_{step}; must match the configured decoder.
0x54	num_samples	uint16_t	count	Nominal setpoints in the digested sample payload; bounds its size.
0x56	num_joints	uint8_t	count	Commanded joints N_j; must equal the configured count.
0x57	payload_version	uint8_t	enumeration	Decoder version; an unknown version is rejected.
0x58	t_start_ns	uint64_t	ns, permission clock	First nominal sample.
0x60	t_commit_ns	uint64_t	ns, permission clock	Latest entry into the matched stop suffix; $t_{\text{commit}} = t_{\text{stop_end}} - T_{\text{stop}}$.
0x68	t_plan_exp_ns	uint64_t	ns, permission clock	End of the nominal setpoints.
0x70	t_stop_end_ns	uint64_t	ns, permission clock	Protected stop completion; $t_{\text{start}} \le t_{\text{commit}} < t_{\text{plan_exp}} < t_{\text{stop_end}}$.
0x78	sidecar_digest	uint8_t[32]	SHA-256	Digest over the setpoint samples and the assumptions sidecar; binds content.
0x98	seam_state	float32[3][N_j]	rad; rad/s; rad/s²	Entry state $(q, \dot{q}, \ddot{q})$ of each joint.
0x98 + $12N_j$	stop_suffix	float32[6][N_j]	polynomial coefficients	Stop-polynomial coefficients per joint, resident and feasible at t_{commit}; a quartic uses a zero fifth-order coefficient. Admission separately requires state-matched fallbacks before commitment.
0x98 + $36N_j$	crc32	uint32_t	checksum	IEEE 802.3 CRC-32 over all preceding bytes.

C.15.4 Enforcement record

The enforcement record of section 12.11 is configuration, not a proposal, so it carries no proposal header (table C.13). Three digests name what the record rests on: the evaluation record, the limit records behind the stopping envelope, and the validity region. Each threshold names the fallback-ladder rung that a failed check selects. Where the rung depends on whether the body is moving, the row names both, `STOP` in motion and `HOLD` at standstill. Refusing a replacement selects no rung if the active record still passes its own state, evidence, lease, and stopping checks. Failure of an active-record check selects the applicable fallback.

Table C.13: Enforcement record: Thresholds of the permission path with the fallback-ladder rung each failed check selects, and digests of the evaluation record, limit records, and validity region the thresholds rest on. The record is configuration and telemetry; it adds no packet to the proposal stream.

Offset	Field	Scalar type	Unit	Check, and the rung a failure selects
0x00	`evaluation_record_digest`	`uint8_t[32]`	SHA-256	Evaluation record whose runtime monitor specifications become this record's monitor channels.
0x20	`limit_record_digest`	`uint8_t[32]`	SHA-256	Canonical list of the limit records whose braking, onset, and actuator limits parameterize $d_{\text{stop}}(v)$.
0x40	`validity_region_digest`	`uint8_t[32]`	SHA-256	`validity_region`, the plant-parameter bounds the thresholds rest on. On the mobile manipulator it covers the arm at the door, where it equals the target ODD, and at the packing station through the mug pick and the handover approach. Each base configuration, normal or restricted, loads as its own enforcement record, so this field holds the region digest of that configuration and its friction premise. A live state outside it that the machine observes selects `STOP`.
0x60	`loop_period_ns`	`uint32_t`	ns	Permission-loop period T_{tick}; the placement record's `nominal_period_ns` must equal it.
0x64	`deadline_ns`	`uint32_t`	ns	Permission-path deadline inside the tick; a threatened overrun invokes a separately bounded response: `STOP` in motion, `HOLD` only at a qualified standstill. The mechanism must act even if this computation stalls.
0x68	`evidence_age_ceiling_ns`	`uint32_t`	ns	Operating ceiling on upper evidence age, qualified for admission and continued tracking. Refuse a candidate above it; cap its admitted expiry at `t_evidence_ns` plus the ceiling minus `sync_error_ns`. An expired active lease or evidence deadline selects `STOP` in motion and `HOLD` at standstill.
0x6C	`watchdog_timeout_ns`	`uint32_t`	ns	The self-watchdog is fed only after a completed tick; expiry requests `INHIBIT` through the independently wired trip.
0x70	`t_stop_ns`	`uint32_t`	ns	Stopping-time bound for the qualified profile from the maximum admitted speed. Admission reserves that profile's distance, pre-brake travel, and uncertainty allowances; loss of the required clearance selects `STOP`.
0x74	`pos_tracking_bound`	`float32`	m	$\lvert x_{\text{meas}} - r_{\text{cmd}} \rvert \le \epsilon_{\text{track}}$; `STOP` above it.
0x78	`pos_critical_bound`	`float32`	m	$\lvert x_{\text{meas}} - r_{\text{cmd}} \rvert \le \epsilon_{\text{crit}}$, with $\epsilon_{\text{crit}} > \epsilon_{\text{track}}$; `STOP` above it.
0x7C	`vel_ceiling_limit`	`float32`	rad/s	$\lvert \dot{q} \rvert \le \dot{q}_{\max}$; `PROJECT` (QP clamp).
0x80	`torque_slew_ceiling`	`float32`	N·m/s	$\lvert \dot{\tau} \rvert \le \dot{\tau}_{\max}$; `PROJECT` (slew limiter).

Offset	Field	Scalar type	Unit	Check, and the rung a failure selects
0x84	actuator_curr_limit	float32	A	$\lvert I_{\text{phase}}\rvert \le I_{\text{peak}}$; STOP.
0x88	dc_bus_overvolt_thresh	float32	V	$V_{\text{bus}} \le V_{\text{max_safe}}$; STOP, with the brake chopper absorbing returned energy.
0x8C	num_barriers	uint16_t	count	Barrier constraints K_b; must equal the configured count.
0x8E	num_approach_channels	uint16_t	count	Approach channels K_a; must equal the configured count.
0x90	cbf_barrier_params	float32[K_b][2]	normalized distance; s^{-1}	Margin δ_k and class-$\mathcal{K}$ rate $\gamma_{\text{cbf}\,k}$ of each barrier; $h_k(\mathbf{x}) \ge \delta_k$, else PROJECT, or STOP when no feasible input remains.
$0x90 + 8K_b$	approach_channel_ids	uint32_t[K_a]	enumeration	Zone and activation condition of each channel, drawn from the validity region, such as the tool point inside the door zone or accept_item held.
$0x90 + 8K_b + 4K_a$	approach_ceilings	float32[K_a]	m/s	Ceiling on tool-point approach speed while its channel is active; a faster proposal takes PROJECT (QP clamp), and the channel that bound it is logged.
$0x90 + 8K_b + 8K_a$	crc32	uint32_t	checksum	IEEE 802.3 CRC-32 over all preceding bytes; a mismatch rejects the stored record, and no motion is admitted until a valid copy loads.

The operating evidence ceiling must charge capture-to-admission age and permitted tracking time while reserving the response and stopping distance derived in section 12.5. The perception dispatch-age gate applies at dispatch only. It cannot serve as this operating ceiling. The dispatch-to-admission interval remains a separate qualification obligation, and the partial stopping budget cannot establish the ceiling without that bound and the required reserve. Each runtime check uses the current MCU time and the retained evidence epoch; admission never makes old evidence fresh.

C.15.5 Placement record

The placement record of section 13.8 is offline evidence with a fixed header and a linked test report (table C.14). It carries no signature of its own; the signed release manifest binds it through `placement_record_digest`. `enforcement_record_digest` names the enforcement record whose `deadline_ns` was tested. `test_report_digest` identifies the external test report. That report states the declared worst-case execution time the loaded test must not exceed, together with distributions, sample counts, test conditions, mitigations, and untested states. A maximum that has not been measured is stored as all ones, never zero, the same unavailable-value convention the fault record uses.

Table C.14: Placement record header: Configuration values and observed maxima, with digests linking the tested enforcement record and the external test report. The header alone does not establish freedom from interference.

Offset	Field	Scalar type	Unit	Invariant and check
0x00	execution_domain_id	uint64_t	enumeration	Must match the declared topology (application processor, NPU, GPU, MCU, lockstep controller).
0x08	privilege_and_ring	uint64_t	privilege level	Proposer at EL0; enforcer at EL1 or above, or in the secure world.
0x10	nominal_period_ns	uint32_t	ns	Enforcer tick; equal to the enforcement record's loop_period_ns.
0x14	hard_deadline_ns	uint32_t	ns	The permission path's deadline inside the tick; equal to the enforcement record's deadline_ns.

Offset	Field	Scalar type	Unit	Invariant and check
0x18	unloaded_observed_max_ns	uint64_t	ns	Largest response observed without competing load; a sample maximum, not a WCET.
0x20	loaded_observed_max_ns	uint64_t	ns	Largest response under the declared load; all ones until a loaded test runs. Independence stands only while it is at most the declared WCET.
0x28	shared_axi_channel_qos	uint64_t	configured QoS	Arbitration priority per claimant; controller semantics verified by register capture and a stress test.
0x30	max_pdn_droop_thermal_trip	uint32_t[2]	mV; m°C	Configured rail-droop and thermal trip thresholds; the test report records the tests behind them.
0x38	hardware_mitigation_flags	uint8_t[32]	bitfield	Active mitigations, such as a TCM snapshot, memory partitioning, or an isolated rail; register-locked after boot.
0x58	enforcement_record_digest	uint8_t[32]	SHA-256	Enforcement record whose loop_period_ns and deadline_ns this record tests.
0x78	test_report_digest	uint8_t[32]	SHA-256	External test report, including the model and firmware digests under test.

C.15.6 Authority record and operator payload

The authority record of section 14.8 has two runtime layouts. A network operator's command is a proposal-header payload (table C.15). It is the one proposal whose sender's identity confers authority, so it carries an AES-128 CMAC tag under the operator role's key, the primitive whose per-frame cost section 14.5 prices. The static authority manifest fixes each role's channel grant, the command's component count, order, and units, and the key custody.

Table C.15: Operator payload: A network operator's command under the proposal header. The tag authenticates the role; the permission path still admits the command by its content.

Offset	Field	Scalar type	Unit	Invariant and check
0x00–0x4F	proposal header	as above	as above	table C.10, with payload_kind = OPERATOR.
0x50	nonce	uint64_t	—	Never repeated within a key's lifetime; a repeated nonce is rejected as a replay.
0x58	requested_channel_mask	uint16_t	bitfield	Channels requested; must lie within the role's grant in the static manifest.
0x5A	operator_role_id	uint8_t	enumeration	Role whose key produced mac.
0x5B	reserved	uint8_t	—	Zero.
0x5C	command	float32[N_c]	per manifest	Requested command components in manifest order, admitted like any other proposal.
0x5C + $4N_c$	mac	uint8_t[16]	AES-128 CMAC	Tag over all preceding bytes; reject an invalid packet without revoking an otherwise valid active channel.
0x6C + $4N_c$	crc32	uint32_t	checksum	IEEE 802.3 CRC-32 over all preceding bytes.

The authority log (table C.16), the per-tick part of the authority record, is written once per tick, not sent as a packet. Its time is on the safety microcontroller's clock; the mapping that relates it to PTP and calendar time, with its error bound, lives in the static manifest together with the vector lengths, units, and component order. Each entry's chain_link is an HMAC-SHA-256, under the log key, over the previous entry's link and this entry's preceding fields, so alteration or interior deletion is detectable against a retained trusted anchor. Detecting truncation of the log also requires an independently retained final link or checkpoint.

Table C.16: Authority log entry: One entry per tick of the permission loop. A missing entry is a gap in the chain, never an implied pass, and the entry is a sampled decision trace, not proof of delivered force or contact.

Offset	Field	Scalar type	Unit	Recorded evidence and limit
0x00	t_mcu_ns	uint64_t	ns, permission clock	Event time of this tick's decision.
0x08	actuator_channel_mask	uint16_t	bitfield	Channels to which this decision applies; bus and DMA permissions enforce write scope.
0x0A	preempt_trigger_code	uint16_t	enumeration	Request, timeout, enabling-device loss, or protective input cause.
0x0C	authority_state	uint8_t	enumeration	AUTO, PEND, BLEND, MAN, FALLBACK, or PROTECTIVE.
0x0D	fallback_rung	uint8_t	enumeration	PROJECT, HOLD, STOP, or INHIBIT when the state is FALLBACK; NONE otherwise.
0x0E	operator_role_id	uint8_t	enumeration	Authenticated source role, interpreted against the static manifest.
0x0F	reserved	uint8_t	—	Zero.
0x10	blend_factor_alpha	float32	dimensionless	Progress in $[0, 1]$ of the enforcer-owned blending profile.
0x14	policy_proposed_cmd	float32[N_c]	per manifest	Command components before permission; not a motor write.
$0x14 + 4N_c$	enforcer_clamped_cmd	float32[N_c]	per manifest	Components admitted after the current constraint check.
$0x14 + 8N_c$	actuator_delivered_cmd	float32[N_c]	per manifest	Dispatched setpoint components; delivery at the drive needs separate evidence.
$0x14 + 12N_c$	measured_plant_state	float32[N_s]	per manifest	Sampled state components, with sensor timing in the manifest.
$0x14 + 12N_c + 4N_s$	chain_link	uint8_t[32]	HMAC-SHA-256	Keyed link over the previous link and this record's preceding fields; evidence of tampering only while the key and anchor stay in custody.

The compact layout above is the fixed decision trace, not the complete incident evidence bundle. Versioned companion records must preserve the remaining lease age, authenticated source identity, admission or rejection outcome, and sensor-event timing and uncertainty required by section 14.7. Each companion record must bind to the decision tick and channel mask under the same integrity scheme; missing companions remain declared evidence gaps. In MAN, policy_proposed_cmd stores the selected human proposal despite its historical field name.

C.15.7 Authority-transfer protocol

The authority-transfer protocol of section 14.3 and section 14.4 runs in the enforcer's 1 kHz loop on the safety microcontroller, where the four-phase handshake, integrator pre-biasing, and the C^2 quintic blend become one sequence of per-state steps. Each tick writes the authority-log entry of table C.16 under the state names used below. The cadence and jitter below are illustrative design targets; the target implementation must qualify them before actuator permission is granted.

Three cadence and memory invariants govern the loop:

- *Execution cadence:* Hard real-time periodic timer interrupt ($f_s = 1$ kHz, period $T_{\text{tick}} = 1.0$ ms, hardware timing jitter $< 1.0\,\mu$s).
- *Memory invariant:* Zero dynamic heap allocation (malloc/free forbidden; all state variables, mailboxes, and circular log buffers statically pre-allocated in DTCM SRAM).
- *Hardware authority:* Exclusive write of the admitted setpoints staged for the next EtherCAT frame to the drives, and of the safe torque off (STO) hardware cutoffs.

Each 1 kHz tick executes five state-machine steps:

1. Sample and validate plant state, available stopping reserve, independent protective inputs, and the current execution lease. An independent protective input selects its validated, state-matched stop path of section 12.9.
2. In `AUTO`, a manual request or policy handover request enters `PEND`; the policy may propose only while its existing permission remains valid. Set T_{timeout} to the smaller of the protocol ceiling and clearance-derived deadline. No later than the last permission tick before T_{timeout}, transfer to the state-matched fallback. An invalid network packet is rejected and logged, not interpreted as a stop request.
3. In `PEND`, verify the operator credential, nonce, enabling input, target freshness, actuator limits, and full transfer-plus-stop clearance. On `ACK`, at the next qualified **Commit** tick, revoke the policy's proposal token on the masked channels, latch $\mathbf{u}_0$ as its last *admitted* command, latch a checked human target $\mathbf{u}_1$, and set `BLEND`. The enforcer remains the only actuator writer. If the replacement fails admission, retain the current mode only while its own state, evidence, lease, and stopping checks pass; otherwise enter the independently validated fallback.
4. In `BLEND`, evaluate $\hat{t} = \min(1, (t - t_0)/\tau_{\text{blend}})$ and $\alpha(\hat{t}) = 6\hat{t}^5 - 15\hat{t}^4 + 10\hat{t}^3$. The enforcer proposes $\mathbf{u}_{\text{blend}} = (1 - \alpha(\hat{t}))\mathbf{u}_0 + \alpha(\hat{t})\mathbf{u}_1$, then checks that candidate against current state, torque/rate, and stopping limits before dispatch. These are **latched endpoints**, not simultaneous live policy and human actuator streams. If the lease, clearance, or tracking assumption fails, use the state-matched fallback. At $\hat{t} = 1$ with a live manual lease, set `MAN` and confirm that state to the operator.
5. In `MAN`, the enforcer checks each live human proposal. Lease expiry or enabling-device loss revokes manual proposals and transfers to the validated fallback. Log request, ACK, commit, candidate, admitted command, measured response, and clock uncertainty separately; the bounded high-rate record must not block the control tick.

C.15.8 Fault record

DRAFT

The fault record of section 15.11 is offline evidence, a fixed header signed with Ed25519 that links to the retained evidence bundle by digest (table C.17). The fault-record root (`fault_records_root`) that the release manifest binds is computed over the canonical encodings of these headers. `envelope_point` has a per-machine definition in the versioned fault-class manifest; for the mobile manipulator its four values are aisle speed (m/s), payload (kg), floor friction coefficient, and ambient temperature (K).

Table C.17: Fault-record header: The fixed part of one fault trial. The linked evidence bundle carries fault duration, hardware and software build identifiers, raw trace references and clock domains, detector identity, every margin value and scale, the measured physical outcome, secondary anomalies, and partial traces for non-pass verdicts.

Offset	Field	Scalar type	Unit	Purpose and interpretation
`0x00`	`trial_id`	`uint64_t`	—	Unique trial identifier in the versioned trial log.
`0x08`	`claim_ref_id`	`uint32_t`	index	Precommitted physical claim and its units.
`0x0C`	`envelope_point`	`float32[4]`	per manifest	Declared operating point of the trial.
`0x1C`	`fault_injection_mode`	`uint16_t`	enumeration	Injected fault class.
`0x1E`	`injection_target_node`	`uint16_t`	enumeration	Hardware boundary receiving the injection.
`0x20`	`fault_magnitude`	`float32`	per fault class	Injection amplitude; its unit is fixed by the fault-class manifest.
`0x24`	`t_det_ns`	`uint32_t`	ns	Fault trigger to detector assertion.
`0x28`	`t_takeover_ns`	`uint32_t`	ns	Fault trigger to measured takeover; subtract `t_det_ns` for the post-detection interval.
`0x2C`	`stl_robustness_rho`	`float32`	claim's unit	Raw signed margin for `claim_ref_id`; other predicate margins remain in the bundle.
`0x30`	`verdict_status`	`uint8_t`	enumeration	`PASS`, `FAIL`, `INVALID`, or `UNOBSERVABLE`.
`0x31`	reserved	`uint8_t[3]`	—	Zero.

Offset	Field	Scalar type	Unit	Purpose and interpretation
0x34	oracle_digest	uint8_t[32]	SHA-256	Immutable predicate and oracle configuration.
0x54	evidence_digest	uint8_t[32]	SHA-256	Separately retained trial evidence bundle.
0x74	signature	uint8_t[64]	Ed25519	Signature over all preceding bytes; the key identifier is in the versioned manifest.

For a trial aborted before injection, detector time, takeover time, and robustness margin are unavailable rather than zero. The layout stores all ones (0xFFFFFFFF) in an unavailable uint32_t time slot and a canonical quiet NaN in an unavailable float32 margin slot, and the signed evidence bundle records a validity flag for each measured field and why it is absent. The oracle checks those flags and the verdict before doing arithmetic, and excludes unavailable slots from pass counts and from latency or margin statistics. The versioned manifest fixes the NaN bit pattern so that hashing remains reproducible.

C.15.9 Fault-injection protocol

The fault-injection protocol of section 15.8 runs one hardware-in-the-loop trial on target silicon, from the precondition audit to the signed header of table C.17. Every exit it takes sets one value of that header's verdict_status. The illustrative fixture below must qualify its timing, interlocks, and measurement uncertainty before any physical trial.

Three substrate and interlock invariants govern the testbed:

- *Execution substrate:* Hard real-time FPGA plant rig and testbed orchestrator ($f_s = 1$ kHz, hardware timestamping jitter < 500 ns).
- *Memory invariant:* Pre-allocated static DMA buffers in the host test orchestrator; zero runtime dynamic heap allocation on the safety microcontroller.
- *Safety interlock authority:* Independent optical or laser truth sensors wired to hardware crowbar relays capable of cutting target actuator power within 2.0 ms of a physical envelope breach.

Each hardware-in-the-loop fault trial executes five sequential stages:

1. **Precondition audit (stage 1).** Sample baseline environmental sensors via $\mathcal{J}_{\text{truth}}$ (temperature, rail voltage, and wheel speed). Compare each measurement with its declared same-unit tolerance; do not take one norm across kelvin, volts, and meters per second. If any precondition fails, log ERR_PRECONDITION_OUT_OF_SPEC, set the verdict to INVALID, and abort.
2. **Synchronization and steady-state arming (stage 2).** Arm the FPGA plant simulator, power up the target two-processor system, and start the periodic control loop (1 kHz). Establish a synchronized reference clock across the logic analyzer, FPGA plant, and truth digitizers via a 10 MHz PXI clock. Allow the physical plant state $\mathbf{x}(t)$ to stabilize in the nominal operating envelope for $t_{\text{settle}} = 2.000$ s. If baseline tracking error $\|e_{\text{track}}\| > \epsilon_{\text{nom}}$, assert ERR_BENCH_INSTABILITY, set the verdict to INVALID, and abort.
3. **Fault injection trigger (stage 3).** At monotonic test epoch t_{inject}, the hardware test orchestrator asserts the trigger line to the inline injection rig and records the injection timestamp $t_{\text{fault_start}} \leftarrow t_{\text{now}}$ in nanoseconds on the reference clock:
   - *Proposer stall (F1):* The orchestrator kills the chunk-policy task one tick after a lease renewal.
   - *Sensor latency or bias (F2):* An FPGA interceptor latches a navigation-camera frame or holds a stale encoder payload while preserving its valid hardware timestamp.
   - *Frame loss (F3):* A fieldbus disturbance node drops or corrupts EtherCAT frames at a drive's slave port.
   - *Drive-bus droop (F4a):* A programmable supply standing in for the pack at its depleted point steps the shared drive bus below the converter lockout of table 15.2 for $\Delta t_{\text{duration}} = 5.0$ ms, while an independent digitizer records the permission rail and the permission path's heartbeat.

- *Permission-rail feed loss (F4b):* A solid-state switch opens the permission rail's feed during the longest resident stop in the qualified loaded envelope. The illustrative all-weight inspected-floor ceiling is a screening case, not the qualified injection condition.
- *Actuator mechanical stall:* A four-quadrant dynamometer applies step counter-torque $\tau_{\text{stall}} = 3.5 \times \tau_{\text{rated}}$.

4. **Enforcer response and takeover audit (stage 4).** Monitor the target MCU's safety enforcer pins, CAN bus telemetry, and gate-driver PWM lines via an isolated logic analyzer. If the external containment rig trips on a structural envelope breach, the hardware crowbar cuts actuator power; assert `ERR_PHYSICAL_BREACH` and carry the trial to stage 5, where the violated containment predicate returns `FAIL`:
   - Detect assertion of the non-maskable interrupt (NMI) or diagnostic flag: $t_{\text{det}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{det}} = t_{\text{det}} - t_{\text{fault_start}}$.
   - Detect hardware multiplexer preemption of the motor driver: $t_{\text{takeover}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{takeover}} = t_{\text{takeover}} - t_{\text{fault_start}}$.
   - Independently witness powered deceleration onset and standstill, rather than inferring either from a flag or multiplexer transition. Preserve the claim's physical reference epoch: F1 measures from the last successful lease renewal, while F4a measures from the undervoltage flag; the injection epoch remains a separate diagnostic timestamp.
   - Detect the physical plant's transition to its safe bounded regime: $t_{\text{bounded}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{bounded}} = t_{\text{bounded}} - t_{\text{fault_start}}$.
5. **Robustness evaluation and record serialization (stage 5).** Ingest the truth trajectory $\mathbf{x}_{\text{obs}}(t)$ from independent instrumentation over the declared trial window. If required truth frames are missing, set the verdict to `UNOBSERVABLE` and retain the partial trace. For an observable trial, compute each precommitted predicate's signed margin in its own unit, as the STL robustness of section A.7 defines it. Return `PASS` only when every required margin is nonnegative and all predeclared timing and travel bounds pass, including independently witnessed deceleration onset and stopping distance from the claim's reference event. Fault-to-detection and fault-to-drive-takeover times are additional diagnostics, not substitutes for physical response; otherwise return `FAIL`. A dimensionless aggregate may be computed only after predeclared per-predicate normalization. Link the fixed fault-record header to the variable-length evidence bundle by digest, sign the header under the specified custody scheme, and retain both.

C.15.10 Release manifest

The release manifest of section 16.8 is a signed offline artifact, not a mailbox record, so it has a canonical encoding in place of fixed offsets. Fields appear in the order that table C.18 lists, each variable-length field carries a length prefix, and the adjudicator's Ed25519 signature covers the encoding of every preceding field. Every digest is SHA-256. The expiry is the one calendar time in the record chain, and it is checked against a tamper-resistant real-time clock rather than the safety microcontroller's monotonic clock.

Table C.18: Release manifest: Canonical field order of the signed release artifact. A new signed version can supersede a manifest only after the current physical conditions are checked, and the hardware root anchors keys rather than storing mutable digests.

Field	Encoding	Binds	Check before permission
`claim_manifest_id`	`uint8_t[16]` UUID	Top-level safety claim	Linked to the root node of the signed argument graph.
`argument_digest`	`uint8_t[32]`	Complete claim–argument–evidence graph	Matches the graph the adjudicator reviewed.
`policy_weights_digest`	`uint8_t[32]`	Learned model weights	Mismatch refuses release.

Field	Encoding	Binds	Check before permission
`enforcer_bitstream_digest`	`uint8_t[32]`	Enforcer logic image	Mismatch refuses release.
`rtos_kernel_digest`	`uint8_t[32]`	Real-time kernel image and drivers	Verified by the hardware root of trust at power-on.
`calib_trace_pointers`	length-prefixed list of signed evidence references	Calibration certificates	Missing or stale evidence refuses unless a separately signed restricted case covers it.
`fault_records_root`	`uint8_t[32]`	Signed fault-record headers (section C.15.8)	Missing, unauthenticated, or mismatched root refuses.
`authority_record_digest`	`uint8_t[32]`	Static authority manifest and the authority log's initial chain anchor (section C.15.6)	Mismatch refuses release.
`placement_record_digest`	`uint8_t[32]`	Placement record (section C.15.5)	Mismatch refuses release.
`evaluation_record_digest`	`uint8_t[32]`	Evaluation record	Mismatch refuses release.
`verdict_classification`	`uint8_t` enumeration	`UNCOND`, `COND`, or `REFUSE`	Selects the permitted operating envelope.
`standing_conditions`	length-prefixed array of (premise, check rate, trip)	Physical premises checked in operation	Measurements or inspections follow each declared cadence; each tick checks retained validity, expiry, and monitor health. An unknown required premise takes the assessed stop.
`telemetry_drift_budget`	length-prefixed task-specific limits and triggers	Tracking and timing drift	A bound violation revokes; a trend prompts inspection.
`verdict_expiry`	UTC calendar time (ISO 8601)	Validity period of the verdict	A real-time clock reading past the expiry voids the permit.
`lifecycle_budget`	`uint64_t[2]` (hours, cycles)	Wear allowance	An exceeded budget or an invalid counter refuses motion.
`adjudicator_signature`	key identifier plus `uint8_t[64]` Ed25519 signature	The whole manifest	Verifies under an authorized, unrevoked adjudicator key.

C.15.11 Release-gate protocol

In section 16.8, before the motor power stages are energized, the release gate attests the silicon state, verifies the manifest of table C.18 and its image digests, and audits the standing conditions in the order below. The hardware root of trust anchors an authorized, revocable adjudicator verification-key chain. Protected storage holds the signed manifest and signed evidence graph. The signature covers every versioned manifest field, including the identity of the signing key, and the verifier checks the key chain and its revocation state before trusting the signature. The individually signed fault records of section C.15.8 enter the manifest as one digest over the complete set in its declared order. The verifier checks each record signature, the ordering, completeness, and that root against the reviewed evidence index. A digest alone establishes neither record validity nor test adequacy.

Fail-closed authorization begins with `Permit_gate = 0` and the physical plant in its assessed nonoperating state. All returned decisions use the signed enum `UNCOND`, `COND`, or `REFUSE`; an implementation may log a reason code separately:

1. Verify the adjudicator key chain and revocation status, the manifest signature, expiry, signed operating-hour and cycle budgets, and every active image digest. Any invalid, missing, or expired item returns `REFUSE`. A different software bank requires its own signed case and a new check of physical conditions.
2. Authenticate the complete set of fault records and calibration references against the signed graph. A missing fault record, bad record signature, or root mismatch returns `REFUSE`, even if the manifest says `UNCOND`. Stale calibration also returns `REFUSE` under the full case. It can support `COND` only when a *separately signed* restricted case explicitly covers that calibration state and all of its evidence and limits still validate.

3. Check the state-dependent standing conditions, including brake test, separation, timebase, and monitoring health. `KNOWN_FALSE` and `UNKNOWN` both return `REFUSE` while retaining distinct logged causes. Apply the approved state-matched stop or hold of section 12.9.
4. If every check passes and the signed verdict is `UNCOND`, load the signed full envelope and return `UNCOND`. If every restricted condition passes under a signed `COND` case, load its signed reduced envelope and return `COND`. A signed `REFUSE` case or any other state returns `REFUSE`. In both operating modes the learned policy only *proposes* actions; the independent enforcer checks each candidate and controls actuator permission. Force trip thresholds must include measured detection and response headroom below the contact criterion.

C.15.12 Approval record

The approval record of section 17.9 is an offline artifact like the release manifest: a canonical encoding in the field order of table C.19, length-prefixed variable fields, SHA-256 digests, and an Ed25519 signature by the adjudicator over every preceding field. The release gate reads it after the manifest checks of section C.15.11, as one more standing condition.

Table C.19: Approval record: Canonical field order of the signed approval record. Every check that refuses returns the premise to the adjudicator with its cause logged, the handling the release gate gives an unknown physical premise.

Field	Encoding	Binds	Check before permission
`record_id`	`uint8_t[16]` UUID	This version of the approval record	Matches the identifier the manifest's standing conditions name.
`release_manifest_digest`	`uint8_t[32]`	The release manifest it governs	Mismatch refuses release.
`upstream_digests`	`uint8_t[32]` each: authority record, enforcement record, fault-record set	The records whose decisions it names	Mismatch refuses release.
`permit_stage`	length-prefixed record: grantor, stage, configuration digest, domain reference	The permission stage and what it covers	A configuration digest different from the manifest's refuses.
`decisions`	length-prefixed array of (type, owner role, approval reference, expiry)	Each decision behind an admitted action class	An entry with no owner, or past its expiry, returns `REFUSE` with the cause logged.
`human_roles`	length-prefixed array of (duty, required time, available time, utilization bound, verdict)	Each human role the safety argument cites	A role cited as a countermeasure with verdict `INFEASIBLE` or `UNKNOWN` returns `REFUSE`.
`reporting_duties`	length-prefixed array of (recipient, trigger, deadline)	What must be reported, to whom, and by when	Missing duties for the stage refuse; the disclosure log is kept beside the record and is not checked at each boot.
`exposure`	length-prefixed array of (group, chose exposure, measure, sample size, rate, interval)	Each group exposed and its per-group detection	A group the design relies on detecting with no entry returns `REFUSE`.
`data_declarations`	length-prefixed array of (record class, purpose, retention, readers, last audit)	Each record class that identifies people	An expired declaration or failed audit suspends that use and returns the premise to the adjudicator.
`labor`	length-prefixed record: takeover, dispatch, sweep, and monitoring hours; barred uses	The labor the deployment moves	Recorded for review; it does not gate motion.
`validity_window`	UTC calendar time (ISO 8601), start and end	The period the record covers	A real-time clock reading outside the window refuses.
`adjudicator_signature`	key identifier plus `uint8_t[64]` Ed25519 signature	The whole record	Verifies under an authorized, unrevoked adjudicator key.

C.15.13 Residual-claims register

The residual-claims register of section 18.3.3 tracks every physical premise the release rests on but does not observe, so that an open claim is carried across software deployments and hardware

revisions. Like the release manifest, it is an offline artifact whose mandatory fields (table C.20) follow a canonical order rather than fixed offsets.

Table C.20: Epistemic residual-claims register: Canonical field order of the register. The header binds it to one platform, build, and operating window; each entry must carry its claim, the missing observable, the hazard and its time to harm, the containment that holds meanwhile, and the predeclared test that would close it.

Manifest Category	Field	Physical Unit	Invariant & Operational Contract
Manifest Header	`system_identifier`	—	Unique embodied platform ID (e.g., `WMM-AISLE-01`)
Manifest Header	`hardware_build_rev`	—	Target silicon and chassis revision (`HW-REV-3.2`)
Manifest Header	`software_digest`	—	Digest of the released binary
Manifest Header	`validity_limit_hours`	hours	Configured operating window before review
Residual Claims	`entry_id`	—	Unique claim tag (e.g., `CLAIM-FRIC-001`)
Residual Claims	`target_safety_claim`	—	Explicit safety invariant ($d_{\text{stop}} \leq D_{\text{clear}}$)
Residual Claims	`missing_observable`	SI units	Unmeasured physical state (e.g., surface friction μ)
Physical Hazard	`governing_law`	—	Newton-Euler, Navier-Stokes, Joule heating
Physical Hazard	`time_to_harm`	seconds	Time from unmodeled transition to irreversible yield
Operational Containment	`containment_type`	—	`FIRMWARE_PARAM_CLAMP`, `PASSIVE_MECHANICAL`, or `OPERATING_RESTRICTION`; validate the selected limit
Operational Containment	`authority_limit`	$\mathrm{m/s}, \mathrm{N}$	Hard numeric setpoint ceiling ($v_{\max}, F_{\max}$)
Operational Containment	`kinetic_energy_ceiling`	J	Maximum allowable kinetic energy under fault ($E_k \leq E_{\max}$)
Assurance & Inspection	`proof_test_interval`	hours	Scheduled test interval; evidence between tests still depends on the operating restriction
Assurance & Inspection	`accountable_owner`	—	Named role responsible for maintaining the containment, arranging its inspections, and approving the closure evidence
Predeclared Closure	`closure_modality`	—	Transducer technology or formal method required
Predeclared Closure	`sample_trials`	count	Case-specific independent exposure, sized by the zero-failure rule of section A.8 for the declared miss-rate bound
Predeclared Closure	`confidence_level`	—	Declared one-sided confidence level of that bound

C.16 Further Reading

For embedded architectures, real-time operating systems, and power electronics, consult the following references:

- **Computer Architecture: A Quantitative Approach** (Hennessy and Patterson 2019) provides a quantitative treatment of processor architectures, cache hierarchies, and memory subsystem trade-offs.
- **Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment** (Liu and Layland 1973) establishes the mathematical foundations of rate-monotonic scheduling and real-time task schedulability.
- **Modern Operating Systems** by Andrew S. Tanenbaum and Herbert Bos covers inter-process communication, lock-free synchronization, and memory management.
- **Real-Time Systems** by Jane W. S. Liu sets out the mathematical formulation of real-time scheduling algorithms, priority inversion, and response time analysis.

D

The Sense–Plan–Act Paradigm

This appendix uses Sense–Plan–Act to locate failures in sensing, motion proposals, and physical execution. Its reference tables connect observable symptoms to timing, state, and force budgets, then identify the evidence needed to test a remedy. The framework also separates a policy's ability to propose useful behavior from the permission path's responsibility to admit motion and preserve a feasible fallback.

D.1 How to Use This Appendix

Engineers consult this reference when architecting cyber-physical boundaries, evaluating whether end-to-end learned policies can run safely on hardware, or diagnosing operational failures across perception, planning, and control. Each recurring design question or production failure symptom maps to the section that specifies its governing physical contract (table D.1).

Table D.1: S·P·A architectural and diagnostic navigation: Symptom-to-section index mapping operational failures and design trade-offs to their governing physical contracts.

When you encounter this architectural or diagnostic symptom	Consult	For this physical contract or diagnostic framework
Evaluating classical S·P·A vs. end-to-end VLA vs. hierarchical physical AI	section D.2	Representation, timing, and authority contracts across architectural choices
Triaging single-axis failures in evidence age, proposal validity, or torque limits	section D.3	Three-axis taxonomy and constraint budgets across Sense, Plan, and Act
Spatial memory retains phantom obstacles or misclassifies unobserved space	section D.4.1	$S \cap P$ epistemic handoffs, evidence epochs, and raycast eviction
Action-chunk seams command violent torque spikes or exhaust braking margin	section D.4.2	$P \cap A$ kinetic handoffs, C^2 quintic blending, and stopping suffixes
End-effector oscillations or limit-cycle chatter occur during rigid contact	section D.4.3	$A \cap S$ contact passivity, contact-specific response budgets, and impedance control
Safety thread overruns, SoC memory contention, or brownout during inference bursts	section D.4.4	$S \cap P \cap A$ permission decisions, CBF-QP filters, and silicon isolation
Tracing cyber-physical custody and typed contracts across the machine levels	section D.5	The record chain from physical plant limits to the release manifest
Matching observable production symptoms to physical telemetry and remedies	section D.6	Production troubleshooting matrix with measured diagnostic discriminators
Establishing verification gates and acceptance criteria before physical release	section D.7	S·P·A systems scorecard across all axes and intersection handoffs

D.1.1 Four-pass diagnostic protocol

When an embodied system violates a safety envelope, misses an execution deadline, chatters during contact, or trips hardware interlocks, ad-hoc parameter tuning can mask the underlying boundary failure. Work through the diagnosis in four systematic passes that separate symptom capture from causal intervention:

1. **State the physical symptom and binding budget**: Record the observed stopping-distance breach, contact-force transient, thermal trip, or enforcer deadline overrun, including physical units, payload, operating state, and the threshold declared in the machine's limit record (chapter 2).
2. **Localize the candidate axis or intersection**: Use table D.3 for a preliminary triage across the three core axes. When the symptom involves interface timing, state representations, or actuation authority, move directly to the intersection landscape in table D.4.
3. **Test the quantitative hypothesis**: Measure the candidate constraint or budget using the instruments in table D.6, such as timestamp skew, bus queue time, phase current, or tracking error.
4. **Intervene and remeasure across the causal boundary**: Apply a principled remedy and rerun the test over the declared load, timing, and fault envelope.

The symptom selects a candidate entry point, not a diagnosis. Contact oscillations motivate an Act–Sense ($A \cap S$) feedback test; a torque transient motivates a Plan–Act ($P \cap A$) seam test. Phantom obstacles motivate a Sense–Plan ($S \cap P$) evidence test. Deadline jitter under concurrent inference motivates a shared-resource test at the permission decision ($S \cap P \cap A$). Measurements must distinguish these hypotheses from mechanical faults, sensor noise, and other causes.

D.2 Architectural Choices and Their Contracts

A stalled planner and an unstable contact loop require different repairs even when both belong to the same robot. Sense–Plan–Act names functional responsibilities; it does not require that every component wait for the preceding component to finish. Classical estimators, learned policies, and feedback controllers can all participate in concurrent, multirate systems. The useful architectural comparison concerns which representation crosses each interface, how old it is, and which component can authorize motion.

DRAFT

D.2.1 Modularity, reactive control, and learned proposals

Early robotics systems such as Shakey combined perception, symbolic planning, and physical execution.[1] Brooks' subsumption architecture demonstrated a different decomposition into asynchronous behavior layers (Brooks 1986; Brooks 1991).[2] These approaches motivate a durable distinction between deliberation and rapid feedback. They do not establish that modular robotics must be synchronous, stateless, or unable to handle uncertainty. A modular interface can carry covariance, observation history, and explicit failure status; a reactive controller can retain state.

Learned policies expand the behaviors that can be proposed from images, language, and demonstrations. Vision-language-action models, including RT-2, demonstrate generalization to novel objects and commands (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), while diffusion policies represent multimodal action choices (Chi et al. 2025). These capabilities can reduce the need to specify every recognition rule or manipulation strategy by hand. Their success depends on the training distribution, feedback, and deployment conditions. Neither an end-to-end training objective nor a modular software boundary determines actuator authority; the deployed system must establish that contract separately.

[1] **Shakey and deliberative robotics**: Shakey is an early example of integrating perception, symbolic planning, and mobile execution; Moravec (Moravec 1988) discusses this period of robotics. The diagnostic use of Sense–Plan–Act here does not require reproducing its execution schedule.

[2] **Brooks' subsumption architecture**: Brooks (Brooks 1986; Brooks 1991) organized behavior into asynchronous layers with suppression and inhibition. The finite-state modules could retain internal state, so subsumption should not be equated with a memoryless sensor-to-action map.

D.2.2 Timing and representation tests

For a blocking sensor-to-actuation path, the latency of one observation's response is

$$T_{\text{total}} = T_{\text{sense}} + T_{\text{plan}} + T_{\text{act}}.$$

Each term must include the relevant queueing, transport, and execution delay. Pipelining can overlap work on successive observations and improve throughput, but it does not remove the age of the observation used for the current command. Batching independent cameras, candidate trajectories, or robots is possible when their deadlines permit it. Future observations from one robot remain conditional on its intervening actions and cannot be treated as already available independent samples.

For approximately constant speed during an uncorrected delay, the distance traveled is

$$d_{\text{delay}} = \int_0^{T_{\text{total}}} v(t)\,dt \approx vT_{\text{total}}.$$

A local controller may continue to stabilize the plant during that interval; the distance is therefore not necessarily blind travel. It is the motion that the delayed observation cannot yet correct. Section 12.3.1 combines delay travel with the selected braking profile and clearance margins. In a linearized loop, a pure delay also adds phase $-\omega T_{\text{total}}$ radians at angular frequency ω; the allowable delay depends on the loop's crossover frequency and available phase margin.

Representations impose a separate test. If a geometric state $\mathbf{Z}_{\text{geom}}$ is computed solely from sensory data $\mathbf{S}$, the data processing inequality gives

$$I(\mathbf{S};\mathbf{Y}_{\text{task}}) \geq I(\mathbf{Z}_{\text{geom}};\mathbf{Y}_{\text{task}}),$$

where $\mathbf{Y}_{\text{task}}$ is a task-relevant variable. The inequality supplies no percentage of lost information; equality is possible when the representation retains all information needed for that task. A bounding box can support clearance reasoning while omitting latch state or material compliance. A learned representation may retain useful task cues, but metric clearance checks still need quantities with calibrated units and uncertainty bounds. Even raw images may not identify friction or hidden mass, so learning does not remove the need for additional sensing or physical qualification.

Runtime choices affect both tests. Shared memory, direct memory access (DMA), and preallocated inference buffers can reduce copies and allocation overhead. They also require buffer ownership, coherent snapshots, and measured contention bounds. A sequence lock can require retries; static allocation alone does not bound execution time. Chapter 13 develops the isolation evidence needed when inference and permission checks share hardware.

D.2.3 Architectural comparison

Capabilities remain separate from obligations across proposal and permission roles (table D.2). The permission architecture can accompany either a classical or a learned proposer.

Table D.2: Architectural comparison across proposal and permission roles: Classical and learned components offer different ways to construct behavior; neither earns physical authority from its implementation alone.

Architectural dimension	Modular proposer	Learned proposer	Independent permission path
Useful capability	Explicit geometry, dynamics, and search constraints	Recognition and action strategies learned from data	Admit motion against declared physical limits
Representation	Metric state, maps, uncertainty, and task symbols	Images, latent features, language, and action candidates	Timed state estimates, limits, and executable commands
Timing obligation	Bound the age of inputs and arrival of plans	Measure proposal latency and handle missing updates	Meet the configured response deadline under admitted load
Failure to test	Model mismatch, omitted uncertainty, stale state	Distribution shift, invalid targets, late proposals	Missing premises, infeasible filtering, deadline failure
Fallback responsibility	Propose recovery or a replacement plan	Propose recovery or a replacement plan	Own a validated response without waiting for the proposer

D.2.4 The proposal boundary and rate hierarchy

The five-level machine separates the Brain's proposals from permission to execute them (section 1.7).[3] Intent generation, trajectory proposals, permission checks, and motor-current control can run at different rates. Those rates follow the plant, contact conditions, and measured computation budget; they are not universal properties of model classes. A sufficiently fast learned controller can participate in a feedback loop, but its timing and stability still require qualification.

Action chunks let a slower proposer supply multiple execution steps. They also increase the interval over which a proposal can become stale. The permission path therefore checks the measured entry state, current evidence, dynamic limits, and the available fallback before commitment.

[3] **The proposal boundary**: The authority boundary separating candidate behavior from permission to execute it. A learned proposal acquires actuation authority only through the qualified permission path; qualification remains conditional on the machine's stated assumptions.

Rejecting a successor leaves still-valid active authority intact. If no successor is admitted by the earlier admission cutoff, the permission path schedules transfer to the resident or state-matched fallback at the commitment deadline. Expired evidence, state bounds, or leases can require an earlier transfer. A stopping suffix is the book's recurring example; some loads and contacts require a validated hold or release instead of an unconstrained stop.

The proposal boundary is an authority boundary inside the machine. The causal boundary is where an admitted command produces physical work. Deterministic execution below the proposal boundary helps make timing inspectable, but it does not certify the sensors, model assumptions, or entire machine by itself.

D.3 Diagnostic Summary of the S·P·A Axes

The S·P·A taxonomy groups diagnosis by three responsibilities: capturing evidence, proposing behavior, and applying force. The groups overlap because a physical failure often crosses an interface. Table D.3 identifies the constraint to measure before selecting a remedy.

Table D.3: S·P·A axis reference: The axes organize diagnosis around measured constraints and budgets, with the chapter that develops each contract.

Axis	Operational role	Constraint or budget to test	Diagnostic action	Book locus
Sense (S)	Capture and estimate physical state	Evidence age, field of view, clock conversion, uncertainty, and sensor transport	Compare capture and per-cell evidence epochs with a measured state and the permission deadline	chapter 8, chapter 9
Plan (P)	Propose a target and trajectory	Intent validity, compute latency, dynamics, and stopping-suffix admission	Compare proposed targets and future states with independently checked limits	chapter 6, chapter 10, chapter 11
Act (A)	Apply and measure force	Current, torque, thermal headroom, brake response, and kinetic clearance	Measure command-to-force response and achieved stop under the declared load	chapter 2, chapter 4, chapter 12

D.3.1 The sense axis: Evidence age and temporal registration

A fresh map publication can contain old observations. Sense diagnosis therefore follows the capture epoch through clock conversion, transport, and estimation rather than treating message arrival as the observation time. Exposure duration and rolling-shutter readout can also spread one image across several physical states. Chapter 8 develops this distinction; section C.7 supplies the clock-conversion model.

Let t_{capture} be the capture epoch converted to the permission clock and ϵ_{sync} a bound on conversion error. A conservative age check is

$$t_{\text{age}} = t_{\text{curr}} - t_{\text{capture}} + \epsilon_{\text{sync}} \leq T_{\text{inv}}.$$

The validity limit must follow the task's motion and uncertainty bounds. For an obstacle whose speed is bounded by v_{max}, an initial position uncertainty radius r_0 expands conservatively as

$$r_{\text{risk}}(t_{\text{age}}) = r_0 + v_{\text{max}} t_{\text{age}}.$$

This bound assumes the stated speed limit holds and does not account for an obstacle that was never observed. Expired evidence cannot renew a clearance claim merely because the map was republished.

D.3.2 The plan axis: Proposal quality and admitted horizon

A useful plan must specify an achievable task and arrive in time to be executed. Learned intent models and policies can supply semantic targets and dexterous action candidates, while search and optimization can construct trajectories from explicit models. Both need closed-loop evaluation. Low prediction error on demonstrations does not characterize the states induced by the deployed policy. The $\mathcal{O}(S^2\epsilon)$ behavioral-cloning bound concerns worst-case cumulative task cost under stated assumptions, not a universal distance-drift law (section A.5; Ross et al. (2011)).

For a chunk of H waypoints separated by Δt, the proposed horizon is $H\Delta t$. Only the portion supported by current evidence, timing, and fallback authority may be admitted. Chunking reduces proposal frequency but can delay correction; single-step control remains viable when the model and runtime satisfy the required feedback budget. Chapter 11 develops the seam and stopping checks that turn either proposal form into executable motion.

D.3.3 The act axis: Electromechanical dynamics and thermal ceilings

A normalized policy output must eventually map to a physical setpoint. For a motor with torque constant K_t, the motor-side torque is approximately $\tau_{\text{motor}} = K_t i_q$ under the applicable field-oriented-control convention. Current response depends on winding inductance, resistance, back-EMF, and available voltage; it cannot follow an arbitrary instantaneous command. Chapter 2 develops these actuator limits and their measurement.

Gear reduction also changes the inertia seen at the joint. For an ideal rigid reducer with speed ratio $N_{\text{gear}} = \omega_m/\omega_j$, rotor inertia J_{rotor} contributes $N_{\text{gear}}^2 J_{\text{rotor}}$ on the joint side. In a single-joint model,

$$\tau_{\text{out}} = N_{\text{gear}}^2 J_{\text{rotor}} \ddot{q} + \tau_{\text{load}}, \qquad \tau_{\text{motor}} = \tau_{\text{out}}/N_{\text{gear}},$$

where τ_{load} includes the other joint-side inertial and load torques and losses are neglected. A step in acceleration produces a step in required torque; its magnitude depends on inertia and the step size. Infinite ideal jerk does not imply infinite torque or a particular current spike.

A lumped winding thermal model gives another diagnostic budget:

$$C_{\text{th}} \frac{dT_{\text{winding}}}{dt} = I_{\text{phase}}^2 R(T_{\text{winding}}) - \frac{T_{\text{winding}} - T_{\text{amb}}}{R_{\text{th}}}.$$

Here the current and resistance must use a consistent equivalent-winding convention. Continuous and peak current limits, duty cycle, cooling, and measured temperature determine admissible torque. Permanent-magnet demagnetization can occur below the Curie temperature; the relevant limit is the motor's qualified operating envelope.

Single-axis tests narrow the candidate cause. Interface tests then determine whether the failure arose from a stale estimate, an infeasible trajectory seam, or interference with the permission path.

D.4 The Intersection Landscape

When a mobile manipulator clips a cart that just rolled out of a cross-aisle, the fault often spans more than one subsystem: a voxel layer that held a stale free-space timestamp, a trajectory handoff that delayed braking, or a shared memory bus that stalled the safety check. The Sense, Plan, and Act (S·P·A) triad meets across three pairwise interfaces and one central permission region (figure D.1). Its three pairwise interfaces are $S \cap P$ (locating object position in current time), $P \cap A$ (verifying that proposed trajectories satisfy dynamic limits), and $A \cap S$ (measuring physical state evolution after actuation). The independent permission decision sits at their central three-way intersection ($S \cap P \cap A$), where a qualified permission path gates all motor commands against physical budgets before actuation. Each intersection below pairs its physical handoff contract with a discriminating measurement.

Each intersection is a testable handoff. Table D.4 names the failure mode and the evidence needed before a proposed fix earns permission.

Table D.4: The S·P·A intersection taxonomy: Four handoffs locate the earliest broken physical contract and the evidence required to repair it; the axis reference in table D.3 links to the underlying chapters.

Intersection	Handoff contract	Failure to test	Evidence and candidate remedy
$S \cap P$: belief	Observations become timed, framed state estimates	A fresh map publication hides an old voxel or an unseen object remains unknown	Check per-cell evidence epochs, free vs. occupied rays, and frame transforms; use a separately validated collision layer

Intersection	Handoff contract	Failure to test	Evidence and candidate remedy
$P \cap A$: motion	A proposal becomes an admitted, state-matched trajectory	A seam requests a torque transient or the stop suffix no longer fits clearance	Check entry state, derivatives, lease, and clearance; reject an infeasible seam; retain valid active authority until its fallback-start deadline
$A \cap S$: feedback	Actuation changes the state that sensors report	Contact force rises before the next useful sample or an actuator saturates	Measure command-to-force and sensor-return delay; choose a local reflex and passive limit where justified
$S \cap P \cap A$: permission	The independent gate turns a proposal into physical action	Compute, memory, power, or clock interference delays refusal	Measure response under contention; revoke tracking and qualify isolation or relocate the path if its bound fails

D.4.1 $S \cap P$ spatial belief and epistemic handoffs

A planner that treats an unmapped blind corner behind a stacked pallet as traversable free space can accelerate into an obstacle before the range sensor ever clears the corner. The Sense–Plan ($S \cap P$) intersection formalizes the **epistemic handoff**. It translates raw, asynchronous, multimodal sensor observations into a metric spatial belief state with bounded age and uncertainty, which planning algorithms and affordance models can query without executing ungrounded actions. This handoff bridges Part II and Part III of the book, taking sensor calibration and domain bounds from the provenance record (chapter 5) and observation streams from the observation contract (chapter 8) to populate the belief record (chapter 9).

In an embodied machine, the primary handoff contract requires that every spatial cell or bounding volume $\mathbf{p} \in \mathbb{R}^3$ in the volumetric occupancy representation $\mathcal{M}$ carries an explicit evidence epoch t_0 and a tri-state classification:

$$\mathcal{M}(\mathbf{p}) \in \{\text{Observed Free } (\mathcal{M}_{\text{free}}), \text{Observed Occupied } (\mathcal{M}_{\text{occ}}), \text{Unobserved Unknown } (\mathcal{M}_{\text{unknown}})\}$$

For an occupancy grid, let $p_t(\mathbf{p})$ denote the probability that cell $\mathbf{p}$ is occupied and $L_t(\mathbf{p}) = \log[p_t(\mathbf{p})/(1-p_t(\mathbf{p}))]$. With the usual conditional-independence approximation, the update from range measurement z_t is

$$L_t(\mathbf{p}) = L_{t-1}(\mathbf{p}) + \log \frac{p(\text{occupied at } \mathbf{p} \mid z_t)}{1-p(\text{occupied at } \mathbf{p} \mid z_t)} - L_0(\mathbf{p}),$$

where L_0 is the prior log-odds. Raycasting identifies the cells traversed before a surface return and the cell containing the hit (section A.3.6). A valid range return supplies free-space evidence along the ray before its surface hit, subject to the sensor model. A missing return alone does not establish free space. The permission path enforces a strict epistemic invariant before any trajectory planner may claim clearance through a spatial volume:

$$t_{\text{age}} = t_{\text{curr}} - t_0 + \epsilon_{\text{sync}} \leq T_{\text{inv}}$$

where ϵ_{sync} bounds clock-conversion uncertainty (section C.7). If $t_{\text{age}} > T_{\text{inv}}$, the cell's status degrades from currently supported free space ($\mathcal{M}_{\text{free}}$) to unknown space ($\mathcal{M}_{\text{unknown}}$).

Unknown space supplies no clearance evidence for ordinary motion admission. Planning algorithms must never equate unobserved space with traversable free space. The age-based expansion in the Sense-axis test applies to previously localized obstacles under a speed bound; it does not locate an obstacle hidden in never-observed space. Diagnostic testing for $S \cap P$ failures begins by inspecting raw sensor-to-map latency, comparing the per-voxel evidence epoch against ground-truth measurements, and validating raycast eviction queues. Failures to preserve these distinctions have three characteristic forms:

1. *Epistemic blindness*: The planner plans aggressive trajectories through unobserved space, treating unmapped blind spots behind occluding pallets or walls as empty. If an obstacle or human steps from behind a blind corner, the machine cannot stop within its remaining sensing margin.
2. *Phantom obstacles and stale log-odds*: When dynamic obstacles vacate a region, failure to project negative evidence along sensor optical rays (due to beam divergence, dark surface absorption, or missed returns) leaves elevated occupancy log-odds in voxel memory (section A.3.6). The permission path can stop the machine even though a separate view shows a clear aisle.

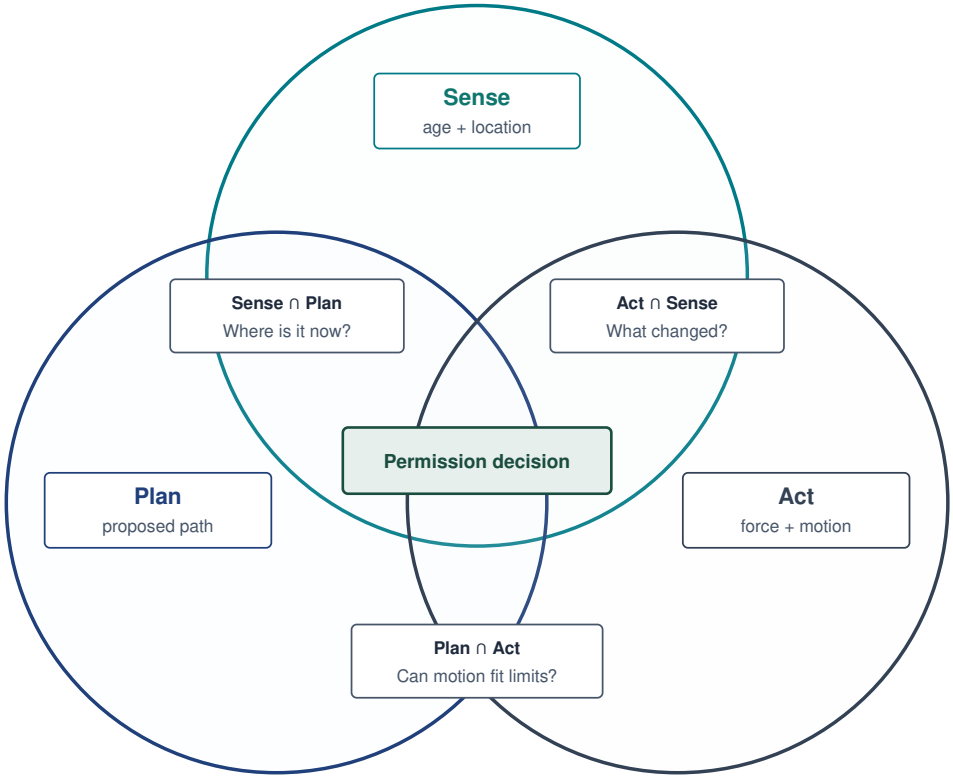


Figure D.1: The S·P·A intersection landscape: Sense, Plan, and Act meet pairwise in three physical decisions, locating the object now, checking that proposed motion fits limits, and measuring what changed after acting. Their center is the permission decision, where an independent permission path checks the shared physical budget before actuation.

3. *Extrinsic calibration drift and spatial skew*: Mechanical vibration or thermal warping alters the coordinate frame transform $\mathbf{T}_{bc} \in SE(3)$ between sensors and the mobile base. A pitch error $\delta\theta_{\text{pitch}}$ produces a height error of approximately $r\delta\theta_{\text{pitch}}$ at range r for small angles in radians, potentially turning the floor into an apparent obstacle and corrupting affordance preconditions (chapter 10).

D.4.2 $P \cap A$ admitted motion and kinetic seam feasibility

Splicing a newly decoded action chunk directly onto an active joint trajectory can command an instantaneous velocity or acceleration jump that rings the drivetrain or overruns the remaining stopping distance. The Plan–Act ($P \cap A$) intersection governs the **kinetic handoff**: admitting candidate trajectory setpoints proposed by statistical neural policies (Brain) and converting them into continuous, dynamically feasible joint commands executed by motor drives (Body). This interface bridges the intent lease (chapter 10), policy manifest (chapter 6), and trajectory record (chapter 11) against the physical plant ceilings recorded in the limit record (chapter 2).

Modern generative policies, such as diffusion decoders and vision-language-action (VLA) models, propose actions in discrete horizons or chunks (H waypoints spanning duration $H\Delta t$). The permission path cannot admit a raw chunk as an open-loop actuator reference. Instead, the seam connecting the currently executing trajectory to the incoming proposal must preserve the required reference continuity and remain compatible with the measured entry-state bounds. For a C^2 reference handover:

$$\mathbf{q}(t_0^+) = \mathbf{q}(t_0^-), \quad \dot{\mathbf{q}}(t_0^+) = \dot{\mathbf{q}}(t_0^-), \quad \ddot{\mathbf{q}}(t_0^+) = \ddot{\mathbf{q}}(t_0^-)$$

where t_0 is the handover epoch. A quintic polynomial can match the endpoint position, velocity, and acceleration constraints (section B.5) over blend interval τ_{blend}:

$$\mathbf{s}(\hat{t}) = \mathbf{a}_0 + \mathbf{a}_1\hat{t} + \mathbf{a}_2\hat{t}^2 + \mathbf{a}_3\hat{t}^3 + \mathbf{a}_4\hat{t}^4 + \mathbf{a}_5\hat{t}^5, \quad \hat{t} = \frac{t - t_0}{\tau_{\text{blend}}}$$

The entire admitted chunk must also satisfy componentwise limits $|\dot{q}_i| \le \dot{q}_{i,\max}$, $|\ddot{q}_i| \le \ddot{q}_{i,\max}$, and $|\dddot{q}_i| \le j_{\max}$ for every joint i. Derivatives of the normalized polynomial require factors of $1/\tau_{\text{blend}}$, $1/\tau_{\text{blend}}^2$, and $1/\tau_{\text{blend}}^3$, respectively. Endpoint continuity alone does not establish those bounds.

A further admission condition is an executable stopping suffix (chapter 11). For straight-line motion toward a stationary obstacle, constant speed v during response delay τ_{delay}, and constant braking deceleration $a_{\text{brake}} > 0$ afterward, a screening calculation is

$$d_{\text{stop}}(v) = v\,\tau_{\text{delay}} + \frac{v^2}{2a_{\text{brake}}} \le D_{\text{clear}}(t).$$

This expression is not generally equivalent to a jerk-limited or C^2 stop. Admission must use the actual profile, uncertainty margins, and load-dependent braking limits; section 12.3 develops that budget. A multidimensional path also needs collision checks over its swept volume, not only a scalar endpoint distance.

At a chunk seam, compare the requested acceleration change with the joint-side inertia and measured current response. A discontinuity can exceed torque limits or excite flexible modes, but the failure magnitude is machine-specific. Along the chunk, check whether a fallback remains feasible from each admitted state. Rejecting a late or invalid successor does not itself revoke the active command. The executor preserves valid active authority and starts its fallback by the commitment deadline, allowing for the earlier admission and response cutoff. A measured latency percentile helps size proposal service; it does not guarantee that the next update will arrive before the admission cutoff. Diagnostic action therefore combines synchronized current and encoder traces at chunk boundaries, lease-expiration telemetry, and an explicit check of the stopping profile from the measured transfer state.

D.4.3 $A \cap S$ physical feedback, contact dynamics, and reflex latency

When a stiff gripper strikes a steel fixture at 0.2 m/s, contact force can climb past the wrist sensor's overload rating in a few milliseconds, long before the next 30 Hz camera frame finishes exposure. The Act–Sense ($A \cap S$) intersection closes the causal loop across the **causal boundary**: actuator forces move joints and exert contact forces against the environment, altering physical reality, which is then captured by sensors and fed back into the control loop. Governed by the limit record (chapter 2), handoff record (chapter 4), and evaluation record (chapter 7), this boundary embodies the first law, irreversibility (section 1.10). Energy delivered into a physical contact cannot be recalled.

Passivity and phase margin provide two ways to assess contact feedback (Hogan 1985). With force and velocity signs chosen so that Fv is power entering the system, a passive interaction port satisfies

$$\int_0^t F(t')v(t')\,dt' \ge -E(0),$$

where $E(0)$ is the initially stored energy. This energy bound is not a force limit; a passive system can still deliver a damaging impact.

For a simple mass–spring contact model, effective mass m_{eff} and stiffness k_{contact} define the natural time scale

$$T_{\text{contact}} = \sqrt{\frac{m_{\text{eff}}}{k_{\text{contact}}}}.$$

It indicates how quickly contact dynamics can evolve, not a universal maximum stable delay. Stability also depends on damping, controller gains, sampling, actuator dynamics, and saturation. In a linearized single-crossover loop, an added pure delay consumes approximately $\omega_c T_d$ radians of phase margin at crossover ω_c. Section B.10 develops energy-based checks, while section B.5.5 treats sampling delay.

Contact hunting can result from delayed feedback, excessive gain, friction, or mechanical backlash. Measuring force rise against command-to-force and sensor-return delay distinguishes these causes. For a locally linear contact, $F_{\text{contact}} \approx k_{\text{contact}} \Delta x$ relates penetration to force; approach speed and effective mass determine how much motion and energy accumulate before correction. A low-rate planner may set a contact goal, but the feedback path that regulates force must meet the contact-specific response budget.

Local impedance or admittance control and passive compliance can reduce contact sensitivity. A target impedance can take the form

$$\mathbf{M}_d(\ddot{\mathbf{x}} - \ddot{\mathbf{x}}_d) + \mathbf{D}_d(\dot{\mathbf{x}} - \dot{\mathbf{x}}_d) + \mathbf{K}_d(\mathbf{x} - \mathbf{x}_d) = \mathbf{F}_{\text{ext}},$$

where the desired inertia, damping, and stiffness shape the response to external force. Realizing this relation requires adequate sensing and actuator authority; it does not establish stability for arbitrary delay or contact. Qualify the combined controller, tooling, and surface over the admitted operating conditions.

D.4.4 $S \cap P \cap A$ independent permission decisions and silicon isolation

If a vision-backbone DMA burst stalls the shared memory fabric or droops the supply rail while the safety filter is evaluating a braking barrier, the motor drives continue executing the prior velocity command toward the obstacle. At the center of the triad sits the tripartite intersection where Sense, Plan, and Act converge: the **independent permission decision**. This is the physical realization of the third law, *proposal is not permission* (section 1.10). The statistical Brain proposes candidate trajectories; the Nervous System arbitrates permission against hard physical invariants; the Body actuates only what is admitted.

The permission path needs bounded response time under the admitted workload. Isolation can use dedicated hardware or a qualified partition; a processor label alone proves neither timing nor independence. One possible filter is a control barrier function quadratic program (CBF-QP, chapter 12, section B.4):

$$\min_{\mathbf{u}} \frac{1}{2}\|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad \text{s.t.} \quad L_{\mathbf{f}}h(\mathbf{x}) + L_{\mathbf{g}}h(\mathbf{x})\mathbf{u} + \kappa(h(\mathbf{x})) \geq 0, \quad \mathbf{u} \in \mathcal{U}_{\text{adm}}.$$

Here $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$ is the modeled dynamics, h defines the candidate safe set $\mathcal{C} = \{\mathbf{x} : h(\mathbf{x}) \geq 0\}$, $L_{\mathbf{f}}h = \nabla h \cdot \mathbf{f}$ and $L_{\mathbf{g}}h = \nabla h \cdot \mathbf{g}$, and κ is an extended class $\mathcal{K}$ function. This first-order form applies when the chosen barrier has the required control dependence and regularity. Position constraints on acceleration-controlled systems may require a higher-order or kinetic barrier.

A feasible QP result is one admission condition. Forward invariance additionally depends on the model, initial state, feasible inputs, state-estimation margins, and implementation timing. Sampled execution must account for behavior between checks. If evidence is missing or the solver misses its deadline, the permission path must invoke a validated fallback without waiting for the Brain. If no feasible safety-preserving action remains, solver failure does not create one; emergency response can limit harm without restoring the lost invariant claim.

Three measurements distinguish common failures at this intersection:

- **Shared-memory interference**: Measure acquisition, validation, and dispatch under the worst admitted DMA and inference traffic. A compute-only WCET excludes delays on the path to the actuator.
- **Electrical interference**: Measure supply voltage during accelerator load changes. Parasitic inductance contributes droop with magnitude $|\Delta V| \approx L_{\text{parasitic}}|dI/dt|$, while capacitance, resistance, and regulator response also shape the transient. Compare the waveform and duration with the supervisor's operating limits.
- **Premise invalidation**: Compare the assumed braking or contact bound with achieved response. A filter based on dry-floor friction can remain feasible in its model while an unobserved oil spill makes its command physically unrealizable.

Candidate repairs include local memory, bounded bus arbitration, separate power domains, and narrower operating limits. Each repair needs remeasurement; no single isolation technique resolves all interference or model errors. Chapter 18 records premises that the deployed machine cannot establish from its current evidence.

D.5 Architectural Record Chain

The records from chapter 2 through chapter 18 connect plant limits, proposed behavior, runtime decisions, and release evidence (section 1.11). Table D.5 maps each record to its main S·P·A responsibility; table C.9 gives what each record binds. A record makes assumptions and evidence inspectable; its presence alone does not enforce an invariant or certify a machine. The table groups records by their teaching role, not by wire-message order. Configuration and qualification records constrain runtime decisions; an emitted admitted command records a particular permission decision.

Table D.5: The S·P·A architectural record chain: Named records link physical limits, proposals, permission decisions, and release evidence. The record contents require validation against the deployed machine.

Chapter and title	Record	S·P·A locus	Constraint or evidence recorded	Primary failure mode diagnosed
chapter 2: The Physical Body	Limit record	Act, $A \cap S$	Motor torque $\tau_{\text{motor}} \leq \tau_{\max}$, thermal dissipation $I_{\text{phase}} \leq I_{\text{cont}}$, stopping distance $d_{\text{stop}}(v)$	Inverter thermal trip, rotor resonance, structural yield
chapter 3: The Cognitive Brain	Proposer limits (enter the handoff record)	Plan	Chunk horizon H, candidate setpoint bounds, proposer lease request	Unbounded candidate setpoints, unconstrained sampling cadence
chapter 4: The Nervous System	Handoff record	$S \cap P \cap A$	Bus cycle time T_{bus}, qualified jitter bound, chunk lease ceiling T_{lease}	Fieldbus priority inversion, seqlock torn read, lease expiration overrun
chapter 5: Physical Data	Provenance record	Sense, $S \cap P$	Dataset calibration bounds, ODD operational envelope limits	Covariate shift, out-of-distribution actuation commands
chapter 6: Policy Training	Policy manifest	Plan	Denoising evaluation count N_{eval}, chunk horizon H, Lipschitz bound L_{lip} if established	Multimodal mode-averaging collapse, explosive policy Lipschitz constant
chapter 7: Closed-Loop Evaluation	Evaluation record	$A \cap S$	Closed-loop task performance, intervention outcomes, collision evidence, and test-domain coverage	Open-loop loss deception, unmodeled closed-loop yaw drift
chapter 8: Sensor Perception	Observation contract	Sense	Observation age t_{age}, bounded clock-conversion error ϵ_{sync}, bounding covariance Σ_{body}	Rolling shutter distortion, timestamp skew, camera extrinsic calibration drift
chapter 9: Spatial Memory	Belief record	$S \cap P$	Per-cell evidence epoch t_0, tri-state classification (free/occupied/unknown), raycasting eviction	Phantom obstacle emergency stops, stale voxel retention, unobserved space mistaken for free space
chapter 10: Grounded Intent	Intent lease	Plan	Task tolerance bounds $\mathbf{r}$, affordance predicate validity, terminal mode admissibility	Affordance hallucination, ungrounded task goal selection
chapter 11: Trajectory Planning	Trajectory record	$P \cap A$	C^2 quintic polynomial seam continuity, componentwise jerk ceilings, stopping suffix guarantee	Seam velocity/acceleration discontinuity, reflected rotor inertia shock, stop suffix exhaustion
chapter 12: Safety Enforcement	Enforcement record	$S \cap P \cap A$	Admission checks and their premises, response-time budget, resident state-matched fallback ownership	Infeasible QP projection, enforcer deadline overrun, unsafe command leakage to motor registers
chapter 13: Workload Placement	Placement record	$S \cap P \cap A$	Silicon partitioning, dedicated TCM memory, AXI QoS priorities, qualified supply-voltage envelope	Shared memory bus contention, DRAM bank conflict, power rail brownout during NPU bursts
chapter 14: Supervisory Intervention	Authority record	$S \cap P \cap A$	Four-phase bumpless transfer, supervisory token state machine, contact-force relaxation	Intervention takeover jerk, operator authority lock-out, transfer state discontinuity
chapter 15: Adversarial Verification	Fault record	$S \cap P \cap A$	Tail-risk estimates and their assumptions, hardware fault injection coverage, sim-to-real gap	Rare-event tail collision, undetected hardware bit-flip, unmodeled contact compliance gap

DRAFT

Chapter and title	Record	S·P·A locus	Constraint or evidence recorded	Primary failure mode diagnosed
chapter 16: Deployment Release	Release manifest	$S \cap P \cap A$	Goal Structuring Notation (GSN) warrant validity, applicable assurance requirements, signed operational envelope	Operating outside qualified ODD, unsupported claim warrants, sign-off on unvalidated assumptions
chapter 17: Responsible Physical AI	Approval record	System-wide	A named owner for each decision behind an admitted action, the feasibility of each human role, per-group detection and exposure, declared records about people	An unowned decision, a human role no person can perform, an unmeasured group of people, records used outside their declared purpose
chapter 18: Engineering Physical AI	Residual-claims register	System-wide	Explicit uncertified premise register (such as friction floor $\mu \geq \mu_{\min}$), epistemic ceiling recognition	Premise invalidation (such as uninspected floor oil slick), catastrophic failure beyond epistemic ceiling

The four governing laws organize this evidence chain. Irreversibility requires budgets for delay, force, and fallback. Endogenous data requires evaluating the states that the policy itself induces. Proposal is not permission assigns admission authority independently of proposal quality. Permission must be earned requires the release case to state where those checks apply and which premises remain unresolved. Together, the chain helps locate missing evidence when a symptom crosses several chapters.

D.6 Physical AI Production Troubleshooting Matrix

When an operational machine violates physical limits or software invariants, table D.6 links the symptom to a discriminating measurement. Software traces can carry physical telemetry, but a CPU profile alone cannot establish a force or current transient. The records need a common time basis so that a physical event can be aligned with the command and evidence that preceded it.

Current probes and encoder traces help resolve chunk-seam events. Motor phase measurements require the rotor angle and the appropriate coordinate transform to recover i_q and i_d; a probe does not measure those rotating-frame components directly. Protocol analyzers reveal frame timing, while available system-on-chip (SoC) performance counters and controlled contention tests help attribute delays. Solver telemetry should include status, residuals, constraint margins, and fallback selection. Those fields can expose numerical or modeled infeasibility, but identifying a friction-premise failure also requires measured physical response.

Table D.6: Physical AI production troubleshooting matrix: Each row is a hypothesis, measurement, and candidate remedy; acceptance depends on the machine's declared limits.

Observable symptom	Candidate boundary	Discriminating measurement	Candidate response
Contact hunting while the arm opens the spring-latched cage door	$A \cap S$	Measure phase margin and force rise against the sensor-to-torque delay under the actual contact stiffness	Qualify a local impedance loop, approach-speed limit, and passive compliance for that contact
Torque or current transient at an action-chunk seam	$P \cap A$	Compare encoder, phase-current, and requested acceleration traces at the measured entry state	Admit a state-matched C^2 bridge and reserve a stopping suffix; the reduction in chapter 11 is profile-specific
Phantom obstacle stop in an aisle	$S \cap P$	Distinguish observed free, occupied, and unknown rays; inspect per-voxel evidence age	Clear only observed free space and retest the occupancy-to-permission conversion
Enforcer misses its configured deadline during camera ingress	$S \cap P \cap A$	Correlate response-time misses with bus queue, bank switches, cache misses, and DMA ownership	Qualify QoS bypass or a coherent local safety snapshot; remeasure under contention
Link or workpiece yields during contact	A	Compare measured force and displacement with the tested material and tooling limits	Reduce admitted speed or force and qualify a compliant load path

Observable symptom	Candidate boundary	Discriminating measurement	Candidate response
Policy update arrives after its admitted chunk horizon	$P \cap A$	Measure the update tail, lease expiry, and fallback transfer state	Shorten the admitted action or use a stop the permission path owns; do not extend permission from a percentile alone
Safety microcontroller (MCU) rail droops during accelerator bursts	$S \cap P \cap A$	Measure rail waveform, path slack, supervisor threshold, and resulting response time	Characterize decoupling, current slew, or physical power isolation
Base drifts toward a rack face despite low offline prediction error	$A \cap S$	Compare closed-loop yaw bias, lateral clearance, and surface-dependent braking	Test closed-loop rollouts and impose a feasible corridor and stop bound

D.7 The S·P·A Systems Scorecard

Before physical release, use table D.7 to connect verification gates to declared acceptance criteria across the three axes and their handoffs. Aggregate task success is useful evidence, but it can conceal rare severe outcomes and does not by itself justify a release. Each check needs the actual limits, operating envelope, confidence or hard-bound semantics, and validated fallback for the machine.

Hardware-in-the-loop tests can exercise these checks in continuous integration. Passing the suite supports the tested claims; it does not turn sampled observations into a deterministic guarantee over untested conditions.

Table D.7: The S·P·A physical AI systems scorecard: Example release checks across the three axes and their handoffs. Each row requires a case-specific threshold and evidence.

Boundary	Example acceptance check for a declared machine	Response if the check fails
Sense: capture clock	Bound capture-to-MCU conversion error and include it in evidence age	Resynchronize or withhold motion when the bound expires
Sense: ingestion	Measure worst admitted sensor/DMA queue occupancy, drop rate, and transfer time under concurrent load	Reserve capacity or reduce sensing load; dropped frames do not renew belief
Plan: chunk horizon	Admit a successor before the admission/response cutoff so fallback can start by the commitment deadline	Retain still-valid active authority and start the owned fallback by the commitment deadline or earlier validity expiry; measured update tails do not extend permission
Plan: dynamics	Check position, velocity, acceleration, torque, and stopping suffix against the actual joint and path limits	Reject or replan an infeasible chunk
Act: brake and contact	Validate achieved deceleration and force response for load, wear, surface, and temperature	Derate speed, change tooling, or refuse the untested state
Act: thermal	Keep measured winding and drive temperature inside device ratings under the duty cycle	Reduce duty or enter the validated stop/hold mode
$S \cap P$: spatial evidence	Preserve each cell's evidence epoch and distinguish observed free, occupied, and unknown	Refuse unknown clearance or obtain a new observation
$P \cap A$: seam	Check the matched entry state, derivative limits, and full stopping clearance before commit	Retain still-valid active authority; start the owned fallback by its commitment deadline or earlier if evidence, state, or lease validity expires
$A \cap S$: reflex	Bound sensing-to-force response against a contact-specific harm deadline	Reduce approach energy or add a validated passive limit
$S \cap P \cap A$: compute	Qualify the permission path's WCET against its deadline in the cycle budget of chapter 12, with acquisition, validation, write, and dispatch budgeted separately	Revoke tracking permission before the cycle overrun consumes stopping margin
$S \cap P \cap A$: shared silicon	Measure enforcer response under worst admitted bus, rail, and thermal contention	Qualify QoS/local snapshots or relocate the path if isolation cannot be shown
$S \cap P \cap A$: permission	Verify model/state assumptions, feasible CBF input, clock and estimate margins, and a state-matched fallback	Refuse the proposal when a premise is missing; a QP result alone proves no path safety

D

Glossary

This glossary defines key terms used throughout *Physical AI*. Terms are organized alphabetically.

A

Action Chunking A policy technique in which a model predicts a sequence of future actions rather than one action per inference. Executing part of a chunk amortizes inference latency but leaves that interval open loop with respect to new policy observations. Receding-horizon replanning or temporal ensembling can improve responsiveness and continuity; neither chunking nor blending establishes kinematic feasibility or collision clearance. (Chapter 3, chapter 11, chapter 6)

Active Probing The deliberate application of a small, bounded input that makes two observationally indistinguishable states produce different observations before the response deadline, such as a controlled pull on a latch below its contact ceiling. Its cost is time and an excitation that must itself fit the operating envelope. (Chapter 18)

Administrative Lease (T_{admin}) The bound, inside an intent lease, on how long the application processor may hold a goal without renewing it. It caps the harm a hung or crashed host can do while its last goal stays active; it is one of the three deadlines whose earliest sets the intent lease's expiry. (Chapter 10)

Advantage The amount by which taking a particular action in a state beats the policy's usual choice there, $A^\pi(s, \mathbf{a}) = Q^\pi(s, \mathbf{a}) - V^\pi(s)$. Policy-gradient and advantage-conditioned methods shift probability toward positive-advantage actions. (Chapter 6)

Approval Record The record that names, for each decision behind a machine's admitted actions, the person who approved it and answers for it, together with the human roles the design relies on, the permission stage and its reporting duties, the groups exposed to the machine, the records kept about people, and the labor the deployment moves. The release gate reads it as a standing condition, so a decision with no owner refuses operation. (Section 17.9, section C.15.12)

B

Back-EMF Back electromotive force, the voltage $e_{\text{bemf}} = K_e \omega_m$ that a spinning motor induces in its own windings in opposition to the supply. In SI units K_e equals the torque constant K_t. At speed, back-EMF can leave the inverter too little voltage headroom to control current, silently lowering the braking torque the drive can deliver. (Chapter 2)

Behavioral Cloning (BC) An imitation-learning method that trains a policy by supervised learning to map sensory observations directly to expert control actions, treating sequential decision-making as a regression or classification task. While computationally efficient on static demonstration archives, standard behavioral cloning ignores the closed-loop consequences of its own control outputs. Small single-step errors can therefore move the robot away from the demonstration distribution, where later errors may compound without learned recovery behavior. (Chapter 6, chapter 5)

Belief State An estimate of a physical quantity carried forward from dated sensor evidence, stated in a reference frame, together with the evidence epoch of the last measurement that constrained it and a declared law for how its error bound $E(\Delta t)$ grows while no new evidence arrives. The permission path may rely on a belief only until that bound reaches the task tolerance, at its invalidation horizon. (Chapter 9)

Belief State Invalidation Horizon (T_{inv}) The elapsed time beyond which an unobserved spatial or kinematic state estimate can no longer be certified to lie within a task's safe tolerance envelope. Derived from disturbance and motion bounds or a declared probabilistic uncertainty model, it limits how long perceptual memory supports the claim. A covariance-based horizon retains the confidence assumptions of that model. Downstream planners that treat spatial beliefs older than this horizon as valid free space risk commanding collisions with dynamic obstacles. (Chapter 9, section B.8)

Bumpless Transfer A dynamic control handoff protocol that matches controller state and commanded references across control authorities to avoid abrupt changes in actuator effort. When supervisory control transitions abruptly between an autonomous neural policy and a human operator, unmatched commands can induce mechanical shocks, structural oscillations, or loss of traction. Bumpless transfer employs dynamic transition ramps and feedback state tracking to transfer command authority smoothly; effort and slew limits still require separate checks. (Chapter 14, section B.5)

C

Causal Boundary The interface through which a digital command produces physical forces, currents, or flows. A proposal can be rejected before it crosses; once executed, it can alter the environment and dissipate energy, and restoring a command in memory does not restore the machine's prior physical state. Recovery requires another physical action, so a qualified permission check must precede actuation. The permission path sits above this boundary, at the proposal boundary of section 1.7. (Chapter 1, chapter 2)

Causal Confusion (Causal Misidentification) An imitation learning failure mode where a policy conditions on spurious, non-causal features that correlate with expert actions in training demonstrations rather than the true physical affordances of the task. High-capacity neural models exploit incidental visual cues (such as interface indicators, dashboard lights, or past actions) that disappear or change under physical deployment. When deployment breaks these correlations, the policy can select inappropriate actions even if its demonstration loss was low. (Chapter 6)

Chunk Lease (T_{lease}) The execution lease of the autonomous chunk policy, which bounds how long an admitted chunk may keep driving the actuators before a renewal must arrive. It is chosen, then checked against the ceiling the stopping budget derives, and it enters the pre-brake delay because a stalled proposer is noticed only when its lease expires. (Chapter 4, chapter 12)

Clearance Inset (C_{inset}) The separation a consumer holds between an estimated obstacle position and the machine's envelope so that the true obstacle stays at least the required clearance away, except with a stated tail probability measured under the conditions of use. It rests on residuals measured against ground truth, so it includes the calibration bias and heavy tail that a reported covariance omits. A percentile inset is not a hard bound; only a justified bound on the error or a feasible stop within the remaining delay budget closes the residual tail. (Chapter 8, chapter 11)

Closed-Loop Evaluation Measuring a policy's performance, safety, and stability through interactive rollouts on physical hardware or in simulation, rather than through static loss metrics on a held-out dataset. Because an autonomous policy continuously perturbs its own operating environment, evaluation requires logging closed-loop metrics such as task completion rates, safety-margin violations, and mean operating time between interventions. Open-loop validation error alone does not measure the consequences of policy-induced distribution shift. (Chapter 7, chapter 6)

Closure Test The test, declared before any data are collected, whose passing would retire an operating restriction held against an open premise in the residual-claims register, for example a failure-free trial campaign that bounds a new detector's miss rate. Until it passes, the restriction holds. (Chapter 18)

Common-Cause Analysis Tracing routing, power paths, and clock trees so that no single physical event, such as a shared cable carrier, a shared converter reference, or a common silicon erratum, can disable two paths whose failures an argument treats as independent. (Chapter 15, chapter 13)

Compounding Rollout Error The accumulation of task error when a policy's actions move it into states poorly represented in expert demonstrations. Under standard finite-horizon imitation-learning assumptions, a per-step expert-distribution error rate ϵ can yield worst-case excess accumulated task cost of order $\mathcal{O}(S^2\epsilon)$ over horizon S. This is not a universal law for physical displacement, tracking error, or collision probability. (Chapter 6, section A.5)

Configuration Space The set of all configurations of a mechanism; for a manipulator, a configuration is the collection of joint positions $(q_1, \ldots, q_n)$. Removing the configurations occupied by obstacles leaves the free space $\mathcal{C}_{\text{free}}$. A path through configuration space still lacks timing, which an executable trajectory adds. (Chapter 11)

Conformal Prediction Set The set of candidate answers that a calibrated threshold cannot exclude. Calibrating the threshold on held-out labeled cases gives the set a chosen marginal coverage level across cases, provided new cases are exchangeable with the calibration set. A set with more than one member is a principled trigger for asking a person rather than guessing. (Chapter 10)

Control Barrier Function (CBF) A scalar function $h(\mathbf{x})$ of the plant state whose nonnegative region is the safe set $\mathcal{C}$ and whose rate of change the control input can influence, so that admitting only inputs with $\dot{h} \geq -\kappa(h)$, for an extended class-$\mathcal{K}$ rate function κ that vanishes at zero, keeps a state that starts in $\mathcal{C}$ inside it (forward invariance) while the model, actuator, and timing assumptions hold. A Lyapunov function certifies convergence toward a point; a barrier function certifies that the state never leaves a region. For a torque-controlled plant, h must carry speed and braking authority, because raw distance has relative degree two and gives the input no direct control authority at the boundary. (Chapter 12, section B.4)

Credible Deceleration (a_{brake}) The sustained deceleration a machine can actually exert against its environment under stated conditions, a validated low tail rather than a datasheet peak. Traction, tip-over, and the drives' current-limited torque each bound it, so it changes with payload, floor friction, and arm pose. (Chapter 2, chapter 12)

Cross-Embodiment Transfer A learning paradigm in which a single policy or foundation model is trained on sensorimotor data gathered across heterogeneous robot morphologies, kinematic chains, and actuator dynamics to generalize to novel target embodiments. Achieving cross-embodiment generalization requires accounting for differences in observations, action semantics, and body dynamics, for example through shared representations or embodiment-specific adapters. Its primary challenge is preventing negative transfer induced by unmodeled dynamic discrepancies, link inertia variations, and differing joint velocity boundaries. (Chapter 5, chapter 6)

D

Defeater A physically realizable condition that invalidates a warrant in a safety-case argument, as an oil film can defeat the braking traction a stopping claim assumes. Adversarial review is the systematic search for defeaters. (Chapter 16)

Diffusion Policy A generative policy that produces continuous action sequences by iteratively denoising a noisy candidate conditioned on observations. It can represent multiple demonstration modes without forcing their average, but generated chunks still require feasibility and clearance checks. The number and cost of denoising steps contribute to inference latency. (Chapter 6, section A.5.2)

Domain Randomization (DR) A sim-to-real transfer methodology that systematically perturbs physical parameters (mass, friction, damping, motor torque limits), sensor characteristics (noise covariance, latency, camera pose), and visual textures across simulated training episodes. By exposing the policy to a broad distribution of simulated environments, domain randomization encourages the neural network to learn invariant representations and feedback behaviors robust to the chosen variation; transfer still depends on whether that variation covers the relevant physical conditions. If randomized too narrowly, the real world falls outside the training hull; if randomized too broadly, learning can become harder or the resulting behavior overly conservative. (Chapter 6)

Dynamic Reachability The property that the end effector can move from its measured state to a target pose, within its velocity, acceleration, jerk, and thermal limits, before the evidence supporting that target expires. Static reachability asks only whether some joint configuration reaches the pose; dynamic reachability adds the clock, so a target well inside the workspace can still be unreachable on one capture. (Chapter 10)

Dynamic Stopping Distance (d_{stop}) The clearance needed to stop from the current state, including pre-brake delay, the selected braking trajectory, and localization and protective allowances. With constant speed v during delay and constant braking deceleration $a_{\text{brake}} > 0$, the model is $d_{\text{stop}} = v\tau_{\text{delay}} + v^2/(2a_{\text{brake}}) + \delta_{\text{loc}} + \delta_{\text{margin}}$. A ramped or jerk-limited stop requires its own profile integral; the permission path must reserve that distance together with the tracking inset and any remaining timing margin. (Chapter 2, chapter 12)

E

End-Effector The tool, gripper, or hand at the distal tip of a manipulator that performs the intended interaction with the workpiece. Workspace and keep-out checks are written against its pose rather than against joint angles. (Chapter 3)

Endogenous Covariate Shift A distribution shift in sequential decision-making where an embodied agent's own execution errors drive its physical body into states unrepresented in its training demonstrations, thereby presenting the policy with novel out-of-distribution observations. An offline action-loss objective does not itself constrain the states visited by the deployed policy, so errors can compound along the trajectory. In physical systems, this shift can cause task failure or boundary violations; corrective data and feedback can improve recovery but require evaluation in the affected states. (Chapter 1, chapter 5, chapter 6)

Endogenous Experience (The Action–Observation Loop) The closed-loop data generation process in which an agent's executed actions causally determine its subsequent physical states and sensory observations ($s_{t+1} \sim p(s \mid s_t, \mathbf{a}_t)$). Unlike passive supervised learning on fixed, exogenous datasets, physical AI experience generation forms an irreversible dynamical feedback loop where minor control perturbations alter future data collection trajectories. Consequently, data generation and policy deployment cannot be separated in embodied agents. (Chapter 1, chapter 5)

Envelope Trichotomy A three-valued epistemic classification of cyber-physical runtime health that partitions system status relative to its certified operating envelope into *known-true* (verified valid telemetry inside bounds), *known-false* (verified limit breach), and *unknown* (epistemic uncertainty from sensor corruption, frame drops, or estimator divergence). In embodied AI, both *known-false* and *unknown* withhold tracking permission and select the qualified fallback for the current state and surviving hardware. (Chapter 16)

Evidence Epoch (t_0) The physical time reference of the measurement that last constrained an estimate, such as a camera's exposure midpoint or the instant a hardware register latches an encoder count. Republishing or predicting a belief advances its publication and estimate epochs, but only a new measurement advances its evidence epoch. (Chapter 9, chapter 8)

Evidence Horizon The longest time the evidence behind a proposal keeps its target within tolerance. For a tracked target whose residual motion can accelerate at a_{slip}, it follows from setting the open-loop drift $\frac{1}{2}a_{\text{slip}}\Delta t^2$ equal to the margin $r_{\text{tol}} - e_0$. It is one of the three deadlines whose earliest sets an intent lease's expiry. (Chapter 3, chapter 10)

Execution Lease Authority an admitted command source holds to keep driving the actuators without renewal. It begins at admission, not at dispatch, and its expiry invokes the state-matched fallback rather than the next proposal. The chunk lease and the manual lease are its two instances on the running machine. (Chapter 4, chapter 12, chapter 14)

Exposure Wall The limit on what failure-free operation alone can establish about a rare failure rate. Under a stationary Poisson failure model, bounding a catastrophic rate $\lambda = 10^{-9}$ per hour at 95 percent confidence requires about $-\ln(0.05)/\lambda \approx 3.0 \times 10^9$ failure-free hours of exposure from the same configuration, an argument Butler and Finelli made for flight software. The required exposure makes direct demonstration

impractical, so claims at that level also need architectural evidence, fault injection, and justified operating premises. (Section 7.9, chapter 15, section A.8)

Extended Kalman Filter (EKF) A recursive state estimator for nonlinear measurement or motion models that runs the Kalman filter's predict and update steps on a first-order linearization about the current estimate. When the model curves appreciably across the prior's spread, the reported covariance can understate the true error, so the filter's consistency must be tested rather than assumed. (Chapter 9)

Extreme Value Theory (EVT) The statistical theory that supplies limiting distribution families for extremes, much as the central limit theorem supplies one for sums. Its threshold form models every observation above a high threshold with the generalized Pareto distribution; its block-maxima form models one maximum per block with the generalized extreme value distribution. (Chapter 15, section A.10)

F

Fallback Ladder The ordered set of responses the permission path selects when a proposal cannot be admitted unchanged or its premises fail. Its rungs are PROJECT (minimal-intervention projection of the proposal onto the safe set), HOLD (a state-matched powered hold), STOP (a controlled, state-matched stop), and INHIBIT (drive torque inhibit with a separately rated brake). Each response requires a validated entry region and surviving hardware; selection must occur while that response remains feasible. An invalid replacement need not revoke an active trajectory whose own premises still hold. (Section 12.9, chapter 14)

Flow Matching Policy A generative policy trained to predict a vector field that transports samples from a base distribution toward a conditional action distribution. The training probability path and sample coupling determine the transport construction; straight conditional paths do not imply globally optimal transport or straight learned sample trajectories. Sampling cost depends on the trained field, numerical solver, and required accuracy, so fewer inference steps than diffusion must be demonstrated for the selected policies. (Chapter 6)

Force Closure A grasp property: every external force and torque on the held object can be balanced by contact forces that each lie inside its friction cone. Force closure says a balancing set of forces exists, not that the gripper can produce it within its force limit or without crushing the object. (Chapter 10, section B.7.5)

Forward Invariance A property of a dynamical system, control policy, and set $\mathcal{C}$: every trajectory starting in $\mathcal{C}$ remains in $\mathcal{C}$ for all future time under the stated assumptions. A barrier certificate can establish this property for a modeled system with feasible controls. Applying the result to hardware requires maintaining the state, disturbance, actuation, and timing premises of the certificate. (Chapter 12, section B.4)

Forward Occupancy Set The region a tracked actor could occupy a given time after its last measurement, grown from its measured position by its speed bound and its tracking uncertainty. A proposed motion is admitted only if it stays clear of every such set over the motion and its stop. (Chapter 10)

Four-Stream Action Tap The tuple $(a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}})$ that records one action at each of its four causal stages, the requested action, the mapped command, the enforced command, and the measured response. Keeping all four makes any single choice of training label interpretable, provided the taps are stamped in one hardware-synchronized timebase. (Chapter 5)

Free-Space Depth (d_{free}) The distance from a hull's surface to the nearest place that no live evidence vouches for, read from a distance field built with every occupied, unknown, or expired cell treated as an obstacle. It is the clearance a map can certify for motion. (Chapter 9)

Freshness Deadline (Δt_{fresh}) The largest age, counted from the capture of a measurement to the onset of the actuator's response, that a measurement of a moving target may reach before the target's motion alone spends the allowable position error. It equals the error tolerance divided by the fastest credible velocity of the tracked state, and every stage from exposure to torque onset shares it. (Chapter 2)

Friction Cone The set of forces a Coulomb contact can transmit, $\|\mathbf{f}_{k,\parallel}\| \le \mu f_{k,\perp}$, a cone about the contact normal with half-angle $\arctan\mu$. A nonadhesive contact can push but not pull, so a gripper holds an object only through forces inside its friction cones. (Chapter 10, section B.7.5)

Future-State Conditioning Generating candidate motion plans from the state anticipated at execution time rather than the stale state measured when planning began, which removes the reference jump at trajectory handoff that planning latency would otherwise create. Activation still depends on the permission path checking the seam against the current state estimate and its error bound. (Chapter 11)

G

Generalist Policy A robot policy pretrained on pooled demonstrations from many robots and tasks, then post-trained on demonstrations of the target task. Pretraining supplies broad visual and motor priors; post-training supplies the task, the embodiment, and the evidence the target deployment needs. (Chapter 6)

Grounded Affordance An affordance expressed in the robot's own frame and units, namely a feature pose in the base frame, an approach direction, an aperture, and a contact wrench ceiling, each tied to dated sensor evidence through a calibrated measurement chain. Unlike a detector box or a language label, it says where and how the gripper may make contact. A learned grasp score or a confident detection is not grounding, because neither carries a metric error bound, a wrench limit, or an evidence epoch that the permission path could admit or expire. (Chapter 10, chapter 8)

H

Hardware Lease A countdown timer on the safety microcontroller that enforces a lease's expiry without the cooperation of the process that issued the authority. It is the mechanism behind every other lease, since a hung host cannot extend authority that a hardware timer is counting down. (Chapter 10, chapter 12)

Heterogeneous Silicon Partitioning Allocation of learned workloads and time-critical permission or control loops to different compute resources, with hardware protection of memory and peripheral access. Separate processors and dedicated buses can remove specific interference and fault paths, but shared power, clocks, interconnects, or thermal limits can retain common-mode failures. Timing and fault-containment claims therefore require evidence for the actual partition and declared load and fault envelope. (Chapter 13, section C.8)

Human-Gated Aggregation An interactive imitation-learning scheme in which the expert decides when to take over and only the takeover segments enter the dataset as labels. It avoids training on the learner's own failing commands. (Chapter 5)

Human Oversight Role A duty that a design assigns to a person, such as stopping the machine, taking over a channel, or approving a motion, and that the safety argument relies on. It counts as a countermeasure only if its holder has the time, the information, and the authority to act within the time the hazard leaves; a remote assistant who serves many machines must also be free when a request arrives. (Section 17.3)

I

Importance Sampling A Monte Carlo method that draws test runs from a proposal distribution that visits the failure region often, then weights each run by the ratio of the operational to the proposal likelihood to remove the bias. It estimates rare failure rates with far fewer runs than plain sampling, but only for the declared operational distribution. (Chapter 15)

Inevitable Collision State (ICS) A physical system state $\mathbf{x}(t)$ from which no admissible, physically executable control trajectory within actuator and friction limits can prevent impact with environmental obstacles over an infinite time horizon. Unlike geometric obstacle clearance, an ICS explicitly accounts for plant momentum, friction limits, dynamic obstacle trajectories, and finite actuator braking authority. A machine can enter an ICS before contact; insufficient stopping distance establishes this only when no other admissible evasive trajectory avoids impact. (Chapter 2, chapter 11, chapter 12)

Innovation The difference between a measurement and the estimator's prediction of it, which is what the measurement says that the prediction did not. A zero innovation confirms an estimate only along directions the sensors can observe. (Chapter 9)

Intent Grounding Refusal A typed rejection of a goal itself, returned when the referent is unresolved or no candidate grasp survives the physical checks a learned generator did not perform, such as an inverse-kinematics solution inside joint limits, jaw aperture, and a collision-free approach corridor. Unlike a planning failure, which finds no collision-free path to a valid goal, it rejects the goal before the planner is invoked, so no best guess over unresolved referents reaches trajectory generation. (Chapter 10, chapter 14)

Intent Lease The record, eighth in the record chain, that turns a grounded human goal into a bounded, expiring request that grants a target, a tolerance, and effort ceilings, but never a path. It expires at the earliest of its evidence horizon, its administrative lease, and its stopping deadline, and expiry withdraws the authority based on that intent and requires re-grounding or a qualified state-matched fallback. Unlike an execution lease, it bounds which trajectories may be admitted, not how long an admitted command may run. (Chapter 10, chapter 11)

Intervention Masking An evaluation distortion in which repeated, proactive interventions by human supervisors intercept a policy before it reaches a boundary failure. Because the supervisor intervenes during nascent drift, assisted success and accident counts alone do not reveal how the unaided policy would have behaved after takeover. Logs must distinguish intervention classes, exposure, and task outcomes. Reporting these separately prevents assisted task completion from being mistaken for evidence of independent policy competence. (Chapter 14, chapter 7)

Intervention Rate The number of events in a declared intervention class divided by active operating time, or by distance when a distance-based rate is reported. The rate depends on the intervention policy and exposure population; human takeovers and automatic corrections must remain distinguishable. It does not by itself establish safety or stability. (Chapter 7, chapter 14)

J

Jerk The rate of change of acceleration, $j = \dot{a} = \dddot{x}$, in $\mathrm{m/s^3}$. A jerk limit makes a brake ramp its deceleration up instead of stepping it, which spares occupants, payloads, and transmissions a shock but lengthens the stop, because full deceleration arrives later. (Chapter 11, chapter 1, chapter 12)

K

Kinodynamic Feasibility The property that the machine can track a candidate trajectory, including its seams and its stop, while every joint stays within its position, velocity, acceleration, jerk, torque, and current limits;

the kinodynamic feasibility envelope is the set of initial states from which this holds. Collision freedom is necessary but not sufficient, and checking only discrete waypoints can miss an interpolant that overshoots a limit between samples. (Chapter 11, chapter 2)

L

Learner-Action Contamination A data-collection failure in which the failing policy's own commands enter the dataset as expert labels, so the retrained policy learns to answer the cues of an approaching hazard with the commands that caused it. (Chapter 5)

Legible Intent Motion and signals that let a person who does not know a machine's goal infer it early enough to get out of the way, as opposed to motion that is predictable only to someone who already knows the plan. It helps the people a machine puts at risk protect themselves, and signals should promise only what measured detection supports. (Section 17.6.1)

M

Manual Lease ($T_{\text{lease,man}}$) The execution lease of a human operator in manual mode, renewed by authenticated heartbeats. It replaces the chunk lease in the pre-brake delay while the operator drives, so it must fit the stopping reserve of the mode in which it is granted. (Chapter 14)

Mean Time Between Interventions (MTBI) Active operating time divided by the number of events in a declared intervention class. MTBI is the reciprocal of the time-based intervention rate when the count is nonzero; a zero-event run requires an exposure-based statistical bound rather than a claim of infinite reliability. Interrupted trials and the intervention policy affect interpretation, so MTBI alone does not establish safety or stability. (Chapter 7, chapter 14)

Memory Wall The limit on a workload's rate that arises when its operands cannot reach the arithmetic units as fast as those units could consume them, so that the bytes moved from off-chip memory, not the operations performed, set the time per evaluation. Faster arithmetic does not move it; only fewer bytes per evaluation (fewer or narrower weights, or weights resident on chip) or more bandwidth does. It is distinct from memory capacity, since a model can fit in memory and still be too slow to stream. (Chapter 3)

Metric 3D Lifting (Spatial Grounding) The construction of a three-dimensional scene representation in physical units and a declared coordinate frame from sensor observations. Metric scale requires calibrated geometry, depth measurements, or another validated scale reference; image features and reconstruction methods alone do not resolve monocular scale ambiguity. The representation supports reachability and collision checks only within its uncertainty, coverage, and information-age bounds. (Chapter 8, chapter 10)

Minimal Risk Maneuver (MRM) A fallback maneuver intended to reduce risk after a fault or loss of nominal authority, such as a controlled stop or transition to a supported resting configuration. Its feasibility depends on the current state, available clearance, and surviving sensing, actuation, and power. The designation does not guarantee a hazard-free outcome or require every machine to reach an immediate standstill. (Chapter 12, chapter 14)

Minimal-Intervention Safety Shield A safety filter that modifies a nominal control proposal only as needed to satisfy modeled safety and actuator constraints. The CBF-QP safety filter solves $\mathbf{u}^* = \arg\min_{\mathbf{u}} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2$ subject to barrier and input constraints, potentially acting before the safe-set boundary is reached. Its guarantee depends on feasibility, model and state bounds, and execution timing; minimizing input deviation neither maximizes task performance nor supplies a fallback when the feasible set is empty. Its effect on the system hazard rate is conditional on the proposal rate and its interception coverage, so a shield does not remove the need to bound the proposer's failure rate. (Chapter 12, chapter 18, section B.4)

Missing Observable The unmeasured physical quantity on which an open claim depends, such as a friction coefficient or a latch strength, stated in terms of the physical claim rather than as an out-of-distribution latent state. Naming it lets a team choose between adding sensing, establishing a physical bound, and restricting the task. (Chapter 18)

Mode Averaging (Multimodality Catastrophe) A learning failure in which a regression policy averages distinct valid action modes into an unsuitable action, potentially causing a physical hazard. Mean squared error favors the conditional mean, which can lie between valid pathways. When human demonstrators navigate around an obstacle by choosing either a left or right detour, a naive regression model averages the competing action modes, commanding an interpolation that steers the robot directly into the obstacle. Mixture models, discrete strategy choices, or generative decoders can represent distinct modes; their selected actions still require physical feasibility checks. (Chapter 6, chapter 3)

Mode Confusion A mismatch between the mode an operator believes the automation is in and the mode it is actually in, so that an action correct for the believed mode is wrong for the actual one. On a machine whose actuator channels change hands separately, the operator's belief about one channel can be stale while the operator acts on another. Committed channel masks and per-channel announcement address it; correct ownership at the bus does not. (Chapter 14)

Monitored Premise Expiration The withdrawal of operational authority when runtime monitoring finds a required safety-case premise false or unknown, for example because measured temperature exceeds a limit or sensor packets are missing. Linking runtime telemetry to safety case warrants makes detectable premise failures grounds for restricting authority; unmonitored premises still require independent evidence. When

a monitored premise expires, the system transitions autonomously into a restricted or refusal operational regime. (Chapter 16)

O

Observability The property that the history of a machine's sensor outputs over a finite window, together with its known inputs and dynamics, determines the state uniquely, so that distinct physical trajectories produce distinct measurement sequences. Along an unobservable direction the innovation stays zero whatever the error, so a contracting covariance there reflects the model's assumptions rather than evidence. (Chapter 9, section B.8)

Observation Contract A specification of the spatial, temporal, and semantic claims a perception subsystem supplies downstream, including coordinate frames, capture times, coverage, uncertainty, and validity flags. It distinguishes deterministic bounds from statistical confidence regions and names how detectable violations revoke a claim. A contract cannot make occluded or unmonitored conditions observable; unsupported claims must remain unknown. (Chapter 8, chapter 3)

Observational Indistinguishability The condition in which two physical states that require different responses produce the same observation history, under every input the machine may admissibly apply, until the deadline by which a state-specific response must begin. Unlike classical unobservability, it is tied to a deadline and to the admissible inputs. A more capable classifier on the same channel cannot separate the states; only a new channel, active probing, or a response valid in both states changes the outcome. (Chapter 18)

Operating Envelope The declared set of external conditions and internal execution states, each stated with its units and tolerances, over which a machine's protective claim must hold and at which every trial of that claim is run. It extends the operational design domain with the machine's own state, such as processor load, drive bus voltage, and bus traffic. Values that are each in range can combine into a point outside the envelope, so its coupled corners must be tested. (Chapter 15, chapter 16)

Operational Design Domain (ODD) The object poses, surface conditions, payloads, speeds, and disturbances in which a machine is meant to operate, written down as the scenario table. A dataset supports the part of the ODD that its occupancy join covers; the rest is its known absence. The sets nest: the policy manifest claims a declared ODD within the ODD, evaluation samples a target ODD within the declared ODD, and verification extends the target ODD to the machine's internal conditions as the operating envelope that a release verdict authorizes in whole, in part, or not at all. (Section 5.7, chapter 6, chapter 7, chapter 15, chapter 16)

Operational Independence The property that no activity, fault, or overload in the proposal domain can push the permission path's execution past its deadline or disable its authority to refuse, through any resource the two share. Unlike spatial isolation, it also covers time, supply, clock, reset, and heat; it is the placement-level form of what functional-safety standards call freedom from interference. A separate core or chip is not independent by construction, since each shared resource is a hypothesis that needs a bound. (Chapter 13)

Moral Crumple Zone The position of a person who absorbs blame for an outcome that a system's design controlled, typically the operator nearest the event, named by analogy with the crumple zone of a car. A human role without the time, information, or authority to act creates one. (Section 17.3)

P

Paired Probing Validating a simulator component by applying identical open-loop command sequences to the simulator and to the physical hardware, then measuring the difference between the resulting state trajectories against explicit error budgets. A difference beyond the tolerance for the operating envelope makes the simulator defective for that training task. (Chapter 6)

Performance Level (PL) The ISO 13849-1 grade, a (lowest) to e (highest), of how reliably a safety-related control function performs, based on its architecture, diagnostic coverage, and component failure rates. Standards for the application set the required level; person detection on a driverless industrial truck, for example, requires PL d. (Chapter 8, chapter 12, chapter 16)

Permission Path The independent mechanism with bounded latency, which the learned model cannot delay or corrupt, that alone may permit an action to reach an actuator. It checks each proposal against fresh evidence, the stopping envelope, and a fallback feasible from the current state, and refuses when any check fails. (Section 1.7, chapter 4, chapter 12)

Permission Stage A grant that allows a named configuration of a machine to operate in a named domain, among named groups of people, on named evidence, and under named reporting duties, such as testing with a safety driver, driverless testing, or open deployment. A machine leaves a stage on evidence gathered there, and a stage is withdrawn for a configuration change, an incident, or a withheld record. (Section 17.4)

Physical Artificial Intelligence (Physical AI) The engineering of closed-loop machines in which learned models influence, without synchronous human approval, physical actions that change subsequent observations. A system is in scope when it combines a learned model, consequential physical feedback, and delegated physical authority. Its actions must fit the plant's physical limits and the time available to respond, and a useful proposal still needs an independent permission decision. (Chapter 1, chapter 3)

Physical Plant The apparatus being controlled, such as a robot's wheeled base or arm, whose state keeps changing while the computer calculates. (Chapter 1, chapter 2)

Problem of Many Hands The difficulty of assigning responsibility for an outcome that many people each helped produce, so that each can truthfully point to someone else. For a physical machine, the decisions behind one admitted action (policy, enforcer, envelope, crossing rule, staffing) belong to different parties and mostly appear in no runtime log. (Section 17.2)

Proposal Boundary The line between a learned model, which may only propose an action, and the permission path, a mechanism designed to make the actuation decision independently of the model, within a qualified latency and fault envelope. The permission path checks each proposal against fresh evidence, the stopping envelope, and a fallback feasible from the current state, and it refuses when any check fails. The proposal boundary separates the Brain from the Nervous System in the book's five-level machine. (Section 1.7, chapter 4, chapter 12)

Q

Qualification Ladder The ordering of four qualification environments, software-in-the-loop, processor-in-the-loop (PIL), hardware-in-the-loop (HIL), and physical fault rigs, by how much of the loop from sensing through inference, buses, drives, and plant is real, and therefore by falling trial throughput. The order is not a hierarchy of evidence, because a higher rung cannot subsume a lower one. Each rung yields a distinct category of evidence and stays blind to the physical mechanisms outside its boundary, so extensive software-in-the-loop passes say nothing about clock drift, DMA stalls, interrupt jitter, or supply sag. (Chapter 15)

Quantization Storing a model's weights, and optionally its activations, in fewer bits than training used. Weight-only quantization cuts the bytes a bandwidth-bound decode step must read; it does not speed up compute-bound phases unless the processor has kernels for the lower-precision arithmetic. (Chapter 3)

R

Record Chain The sixteen records, one produced by each chapter of Parts I to IV except the one on the learned proposer and one by the conclusion, that carry the book's assumptions and evidence from the limit record to the residual-claims register. One strand states what the plant can do and what the proposer has shown it can do; the other carries what the proposer proposes now; they meet in the enforcement record. A record makes a claim inspectable; it does not establish the claim. (Section 1.11, section C.14.2)

Reflected Rotor Inertia (J_{ref}) The apparent rotational inertia of an actuator rotor seen at the load-side joint output, $J_{\text{ref}} = N_{\text{gear}}^2 J_{\text{rotor}}$ for transmission ratio N_{gear}. In high-reduction joints it dominates the apparent load, caps joint acceleration, and concentrates impact energy on the gear teeth before drive current can be interrupted. (Chapter 2)

Relative Degree The number of times a function of the state, such as a barrier function $h(\mathbf{x})$, must be differentiated with respect to time before the control input appears explicitly. Raw distance has relative degree two for torque, so a barrier written on distance alone gives the input no instantaneous control authority at the boundary. (Section B.4, chapter 12)

Release Verdict The signed decision that sets how much authority a named machine's permission path may grant, `UNCOND` for the full assessed operating envelope, `COND` for a separately evidenced restricted envelope, or `REFUSE` when any required premise is missing, contradicted, or unknown, in each case only while its standing conditions hold. It is decided before operation and never moves the proposal boundary; under every verdict the permission path still admits each action. (Chapter 16)

Replacement Latency (L) The elapsed time from the capture of the observation a new proposal rests on to the admission of that proposal. It is the sum of the capture-to-dispatch observation age and the dispatch-to-admission interval Δ_{da}. (Chapter 11, chapter 13)

Rolling Shutter An image sensor readout in which rows are reset, exposed, and read out sequentially, each row starting one row period after the previous one, so a single frame samples a moving scene at progressively later times. A global shutter instead exposes every pixel over the same interval. (Chapter 8)

Rule of Three The approximation that zero failures in n independent trials bounds the per-trial failure probability below about $3/n$ at 95 percent confidence, because $-\ln 0.05 \approx 3$. The bound holds only under the trials' sampling assumptions. (Chapter 7, section A.8)

S

Safe Torque Off (STO) A drive safety function that prevents the drive from generating motor torque through its torque-producing power stage. STO does not itself brake a moving load, hold a gravity-loaded mechanism, or isolate the electrical supply. Its use in a stopping or restraint path requires a plant-specific assessment of coasting, stored energy, and any separately rated mechanical brake. (Chapter 12, chapter 2)

Safety Case (Claim–Argument–Evidence) A structured argument that a named machine, in a stated operating envelope and under a stated fault model, meets explicit physical criteria, written as claims that decompose through stated arguments into retained evidence. Unlike a test report, it states why the evidence supports a claim about conditions the machine has not yet met and names the assumptions under which the reasoning stops holding. It is written with the design, and in this book's architecture its verdict decides what the permission path lets the machine do. (Chapter 16)

Safety Integrity Level (SIL) The IEC 61508 grade, 1 (lowest) to 4 (highest), of the risk reduction a safety function must provide, expressed as a band of tolerable dangerous failure rates. Not to be confused with software-in-the-loop testing, which some texts also abbreviate SIL. (Chapter 16)

Safety Laser Scanner A planar time-of-flight sensor type-tested as a protective device (IEC 61496-3) whose safety output reports only that something occupies a configured protective field. On a mobile base it carries the certified stop for a person in the path; a perception LiDAR without that rating cannot. (Chapter 8, chapter 12)

Safety Microcontroller The small, independently clocked processor that runs the permission path's deadline-critical checks, watchdogs, and hardware leases, and commands the drives' stop functions. On the running machine it sits on its own board link with no memory shared with the application processor. (Chapter 4, chapter 13)

Scenario Table The operational design domain divided into cells over low-dimensional task and environment parameters, such as pose offsets, surface friction, payload, and lighting, so that a dataset's measured occupancy can be joined against it. Cells the join leaves empty are the dataset's known absences. (Section 5.7)

Sequential Probability Ratio Test (SPRT) Wald's test that updates a log-likelihood ratio after each trial and stops as soon as it crosses an acceptance or rejection boundary set by the allowed error probabilities. It permits early acceptance without inflating false acceptance, provided hypotheses, outcome, and boundaries are declared before testing. (Chapter 7, section A.9)

Shadow Mode Deployment Staging in which a candidate policy runs on the live sensor streams and writes the commands it would have proposed to an isolated buffer with no actuator authority, while the incumbent controller supplies proposals and the enforcer keeps sole output ownership. Because the incumbent's admitted commands determine the plant trajectory, agreement in shadow mode says nothing about the states the candidate's own actions would reach. (Chapter 16, chapter 7)

Shared Autonomy A collaborative control architecture in which a human supervisor and an autonomous policy concurrently exert control authority over an embodied system's physical degrees of freedom. Authority allocation may be realized via continuous haptic virtual fixtures, dynamic compliance blending, or task-space privilege partitioning. The central challenge of shared autonomy is maintaining an intuitive, transparent authority distribution that avoids conflicting commands while preserving the qualified safety constraints. (Chapter 14)

Sim-to-Real Contact Exploit A training failure in which a reinforcement learning policy discovers and exploits unphysical numerical artifacts in a simulator's contact solver, such as interpenetration spring restitution, infinite friction sticking, or instantaneous impulse relaxation, to achieve high simulated reward. Because these mathematical solver relaxations do not exist in the physical world, the learned policy can lose performance or violate constraints when deployed on hardware. Mitigation can include improved contact models, randomized parameters, energy-consistency checks, and physical validation. (Chapter 6)

Sim-to-Real Gap (Sim-to-Reality Gap) The systematic discrepancy in dynamics, contact mechanics, sensor transduction, and execution latency that exists between a computational physics simulation and the real physical environment. Policies optimized purely in simulation frequently fail upon physical deployment when the selected models omit or misrepresent effects such as frictional contact, transmission backlash, compliance, or execution delay. Bridging this gap is the central challenge of simulation-based policy training in physical AI. (Chapter 6, chapter 7)

Simultaneous Localization and Mapping (SLAM) Estimating a machine's pose while building the map it localizes against. Modern systems store past poses as a graph of odometry and scan-match constraints; a loop closure adds a constraint when a place is revisited and can shift every pose and map cell at once, which is why map-anchored beliefs must carry the map revision that placed them. (Chapter 9)

Spatial Information Age (t_{age}) The time elapsed between the exposure midpoint at the transducer and the consumption of the resulting observation by a policy, state observer, or control loop; running text calls it observation age. Mass keeps moving while the pipeline computes, so the age becomes travel $v\,t_{\text{age}}$ that enters the stopping budget through the pre-brake delay. A frame's arrival timestamp in userspace understates it by the capture and transport time already spent. (Chapter 8, chapter 9)

Speed and Separation Monitoring (SSM) A collaborative-robot safety mode, set out in ISO 10218-2:2025, that slows or stops the robot so that a protective separation distance to a person is never breached. That distance covers the person's travel over the robot's reaction and stopping time, the robot's own travel and stopping distance, and the position uncertainty of both. Power and force limiting, the complementary mode, instead bounds the force of a contact that is allowed to happen. (Chapter 10, chapter 12)

Standing Condition A relied-on premise of a release verdict paired with a detector or site procedure, a checking cadence set by the physical failure mechanism rather than the software release cycle, and a validated response for when the premise becomes false or unknown. Standing conditions make a release revocable. (Section 16.5)

Stopping Budget The accounting of the distance a moving machine needs to stop, tested against the usable clearance in $d_{\text{stop}} \le D_{\text{clear}}$. Its elementary form is travel during the pre-brake delay plus braking travel, $v\tau_{\text{delay}} + v^2/(2a_{\text{brake}})$; the finished budget adds the allowances and delays that each part of the machine introduces, so added delay consumes clearance in proportion to the machine's speed. (Chapter 1, chapter 2, section 12.3.1)

Stopping Envelope The set of plant states, clearance and speed along each direction of travel, from which the resident braking response, begun after the accounted delay, arrests motion before the nearest boundary with the localization, protective, and tracking allowances still reserved. It turns $d_{\text{stop}} \le D_{\text{clear}}$ into a speed

ceiling. Payload, floor friction, and winding temperature move its boundary, so an envelope computed from a datasheet deceleration overstates it when the machine is loaded. (Chapter 12)

Stopping Suffix A precomputed braking trajectory reserved with an admitted motion plan to reach a controlled standstill within declared clearance and actuator limits. It is usable only from its validated entry region, with the necessary state continuity, traction, drive authority, and power available. If authority expires from another state, the permission path needs a separately qualified state-matched response. (Chapter 11, chapter 14)

Supervisory Intervention The transfer of the right to propose commands on named actuator channels between a learned policy and an authenticated person, committed on a control tick and bounded by a lease, while every command continues to pass the same permission check. The operator gains only the right to propose, not the actuators, and because the transfer is a delay on a moving machine it is priced in the stopping budget. (Chapter 14)

T

Takeover Reaction Budget (T_Σ) The supervisory waiting interval from a takeover demand to receipt of the first human command, modeled here as $T_\Sigma = T_{\text{sensory}} + T_{\text{cognitive}} + T_{\text{motor}} + T_{\text{comm}}$. The communication term includes transport and the authority handshake; plant response, braking, and any subsequent blending require separate budget terms. A takeover can be credited with preventing harm only if this complete response fits before the last feasible intervention time. (Chapter 14)

Task and Motion Planning (TAMP) Planning that couples a symbolic search over discrete actions, such as grasp and place, with geometric samplers that find the continuous grasp poses, joint configurations, and paths each action needs. Failure to find a valid sample leaves the symbolic step unverified; it does not establish that no feasible configuration exists. (Chapter 10)

Temporal Ensembling A filtering technique for action-chunked policies that combines predictions for the same execution time from overlapping chunks, often through an exponentially weighted average. It can reduce inter-chunk discontinuities but does not guarantee bounded jerk, preserve distinct action modes, or establish feasibility. Weighting older predictions also trades responsiveness for smoothing. (Chapter 3, chapter 6, chapter 11)

Temporal Logic Falsification Verification posed as adversarial optimization, in which an optimizer searches initial conditions and time-varying disturbance trajectories within a bounded budget to minimize the quantitative robustness of a Signal Temporal Logic (STL) requirement, a real-valued measure of how far a trajectory is from violating or satisfying it. A trajectory with negative robustness is a counterexample; an unsuccessful search yields no certificate. (Chapter 15, section A.7)

Test Oracle The predicate, fixed in full before a trial runs, that compares each independently witnessed quantity of the trial's trace with its bound from a declared reference event and returns the trial's verdict. Each comparison carries a signed margin in its own unit, which lets a near miss inform the next trial. (Chapter 15)

Time Parameterization Choosing the timing $s(t)$ along a fixed geometric path $\mathbf{q}(s)$ so that every joint velocity, acceleration, torque, and jerk stays within the body's limits, usually to minimize traversal time. (Chapter 11)

Time-to-Harm (t_{harm}) The predicted interval until a specified harmful event under a stated continuation of the current motion and disturbance assumptions. Comparing it with detection, decision, actuation, and braking time helps assess response urgency, but a scalar timing inequality alone does not prove clearance or forward invariance. A protective response must begin while it remains feasible; once that region is lost, initiating emergency braking cannot guarantee avoidance. (Chapter 18)

Trajectory Spline Blending (C^2 Continuity) Interpolation across trajectory seams that matches position, velocity, and acceleration through C^2 continuity, using suitably constrained polynomials or splines. Acceleration continuity removes acceleration steps but does not imply continuous jerk, which requires C^3 continuity, or bound the magnitude of jerk. The blend must separately satisfy velocity, acceleration, jerk, torque, and clearance limits over the entire segment. (Chapter 11, chapter 3)

V

Vision-Language-Action (VLA) Model A multimodal model that conditions action predictions on visual observations and language, often with robot proprioception. Outputs may be discrete action tokens, continuous controls, or trajectory chunks. Built on high-capacity transformer backbones pretrained on Internet-scale vision-language datasets and fine-tuned on robotics demonstrations, VLA models synthesize open-vocabulary semantic reasoning with spatial affordance grounding. Because their internal weights lack physical safety and stability guarantees, their outputs are treated as unprivileged candidate proposals requiring downstream kinodynamic and barrier filtering. (Chapter 3, chapter 10)

W

World Model An internal generative model that learns to simulate the forward transition dynamics, contact mechanics, and sensory observations of the physical environment conditioned on the agent's prospective actions. In physical AI, world models support counterfactual trajectory evaluation and model-predictive rollouts without executing each candidate on hardware. A learned predictive model is not automatically a

validated digital twin. Their reliability is limited by epistemic uncertainty over non-smooth contact changes and the sim-to-real gap. (Chapter 3, chapter 11, chapter 6)

Worst-Case Execution Time (WCET) The longest time a task can take to run to completion on a given processor, over every admissible input and every state its caches, pipeline, and buses can be in, with no other task competing for the processor. It is the C term of a response bound; blocking and interference from other tasks and interrupts are separate terms with their own bounds, which a shared multicore can inflate. A measured maximum can refute a declared WCET but never establish it. (Chapter 4, chapter 13)

DRAFT

DRAFT

Index

2018 Tempe Autonomous Collision
  NTSB report, 23
3D Gaussian splatting
  projection Jacobian, 847
3D vision foundation models, 404

Action agreement, 315
Action chunking
  architectures, 472
  definition, 108
  mode commitment, 235
  temporal trajectory, 107
Action decoder
  multimodal families, 855
Action expert
  flow matching, 98
Action tap
  definition, 184
Active probing
  definition, 799
Active stereo
  transduction, 337
Actuation
  mechanical resonance, 269
Actuator
  current limiters, 315
  electrical rise time, 947
  hydraulic, 54
  intermittent torque, 67
  Joule heating, 314
  pneumatic, 54
  saturation, 530
  stator delay, 313
  stator thermal dynamics, 947, 948
  thermal ladder network, 947
  torque limit, 314
  velocity limits, 451
Actuator authority, 749
Actuator bandwidth
  definition, 42
Actuator saturation, 520, 546
Actuator torque limits, 519
Adaptation wear log, 267
Adaptive stress testing, 693
Admission gate
  rejection, 466
Admittance control
  impedance duality, 635
Admitted command
  enforcement record, 547
  per-tick output, 559
Advantage function, 230
Adversarial fault injection
  definition, 658
Adversarial patch
  physical attack, 679
Adversarial review
  definition, 736
Adversarial vulnerability
  perturbation bounds, 118
Affordances
  agent-relative, 433
  grasp envelopes, 433
  grasp geometry, 97
  grounded, 434
  SayCan value function, 870
Algebraic Riccati Equation
  continuous-time, 888
Alpha-power law
  gate delay, 584
Alternate longer-lease schedule, 479
ALVINN
  recovery data, 12, 178
Angular jerk
  electromechanical limits, 948
Approval record
  definition, 771
  fields, 771
Architectural warrants, 749
Architecture
  subsumption, 448
ARINC 653
  time partitioning, 592
Arithmetic intensity
  roofline, 102
Asynchronous divergence
  closed-loop, 47
Atomic update
  operating system, 744
Authentication
  signature verification, 650
Authority
  ambiguity, 639
Authority record, 616
  definition, 644
  intervention entry, 203
  per-tick log, 644
Authority split, 786
Autonomous completion rate
  definition, 280
Auxiliary signal design
  active fault detection, 799

Back-EMF, 279
  definition, 55
Backup-set filter, 541
Backward reachable tube
  definition, 907
Barrier certificates, 520
Barrier function
  optimization origin, 534
Base frame, 386
Bayes' rule
  parameter estimation, 840
Bayesian optimization
  expensive trials, 693
Behavioral cloning, 230, 640
  compounding error, 864
  definition, 178
  quadratic error compounding, 179
Belief, 378
Belief record
  definition, 388
Belief state
  definition, 379
Belief state invalidation horizon
  definition, 410
Bellman relation, 230
Black-box recorder
  pre- and post-trigger capture, 714
Black-channel principle, 145
  board link, 942
  drive interface, 560
Blending window, 636
Blocking time, 133
Board-level link
  local mailbox, 942
Bound propagation, 684
Boundary contract, 589
Brake
  mechanical, 750
Brake chopper
  hysteresis control, 950
Braking
  friction limit, 958
Braking distance, 49
Branch and bound
  neural network verification, 684
Brownout reset
  undervoltage, 71
Bumpless transfer, 633
  definition, 633
  mid-blend abort, 635
  smoothstep form, 904
Bus impedance
  power distribution, 71
Bus jitter, 353

C^2 bumpless authority transfer ramp, 651
  definition, 633
C^2 continuity, 473
C^2 spline seam
  definition, 486
C^2 trajectory spline blending, 487
Cache coloring, 593
Cache locality
  false sharing and 64-byte alignment, 930
Calibration
  ChArUco target, 343
  hand-eye, 343
  spatial, 370
Calibration clashing failure, 744
Camera calibration
  absolute conic, 957
  planar target, 212
Camera model
  pinhole projection, 956
Camera triggering
  hardware sync pulse, 188
CAN
  bus-off state, 686
CAN arbitration, 578
Canary deployment
  etymology, 746
  physical AI fleet, 746
Capability matrix, 617
Capture
  event, 46, 143
Capture timestamp
  definition, 341
Cascaded control
  tracking architecture, 520
Catastrophic forgetting
  fine-tuning regression, 264
  in physical fine-tuning, 260

DRAFT

Causal boundary
  definition, 12
Causal confusion
  imitation learning, 200
Causal intervention masking, 645
CBF-QP safety filter, 539
Censored trial fallacy, 316
  definition, 289
Centroidal momentum
  Newton-Euler projection, 918
  Orin matrix, 918
Chimeric kinematic product anti-pattern
  definition, 390
Chunk lease
  definition, 142
  expiry under a stall, 319
  lease rule, 163
  proposer side, 111
Chunk seam continuity
  jerk, 109
Claim-argument-evidence
  notation, 717
Claim-argument-evidence safety case
  argument structure, 717
  definition, 718
Clearance inset
  definition, 357
Clock
  monotonic, 449
Clock domain
  definition, 342
Clock synchronization
  PI servo loop, 941
Clock-conversion bound
  definition, 342
Clopper-Pearson interval
  exact binomial inversion, 874
Clopper-Pearson method, 292
  exact inversion, 292
Clopper-Pearson non-asymptotic bound
  definition, 292
Closed-loop evaluation
  definition, 282
Closed-loop evaluation estimand
  definition, 278
Closed-loop stability, 673
Closed-loop stiffness, 519
Closure test
  definition, 794
Collision avoidance
  inevitable collision state, 499
Common-cause analysis
  definition, 675
Common-cause failure, 136
  shared silicon, 577
  shield coverage, 792
Concurrency
  double-buffering, 945
  lock-free SPSC ring buffer, 943
  sequence locks, 954
  triple-buffering, 945
Confidence score, 370
Configuration space
  definition, 473
  history, 473
Conformal prediction
  definition, 440
  exchangeability, 440
Conformity assessment
  Machinery Regulation, 725
Conservative Q-Learning, 240
Constrained reinforcement learning, 538
Contact constraints
  unilateral normal force, 918
Contact fidelity invariant, 251
Contact friction, 287
Contact impact
  approach-speed bound, 311
Contact model
  Kelvin-Voigt, 227
Contact yield deadline
  definition, 250
Continuous flow matching
  vector field regression, 858
Contracts
  residual claims, 793
Control
  closed-loop stability, 318
  gain crossover, 351
  subsumption, 448
Control architecture, 642
Control barrier function, 533, 538, 560
  definition, 534
  minimal intervention, 900
  QP solver, 563
  safety filtering, 897
Control Lyapunov Function
  stabilization, 894
Control Lyapunov function
  barrier counterpart, 521
Controllability
  Kalman rank criterion, 884
Controller
  windup, 530
Copper resistance
  temperature dependence, 67
Corrective aggregation, 202
Cost-to-go
  relation to value function, 230
Coulomb friction cone
  traction boundary, 918
Counterfactual divergence
  shadow mode, 746
Counterfactual oracle
  definition, 792
Covariance
  spatial, 425
Covariance matrix, 412
Covariance propagation
  cubic dispersion, 915
Covariate drift
  physical AI, 177
Covariate shift
  closed loop, 12
  in behavioral cloning, 263
  open-loop replay, 746
CPU isolation
  nohz_full and isolcpus, 930
Cross-embodiment compatibility contract
  definition, 204
Cross-entropy method, 697
Crossbar bus and DRAM bank contention, 580
Crowbar circuit
  test containment, 667
Cryptographic manifest
  hardware Root of Trust, 739
Cubic Hermite spline
  multi-rate interpolation, 903
CUDA
  Inter-Process Communication, 945
CUDA Graphs
  deterministic kernel dispatch, 929
CUDA IPC
  zero-copy tensor sharing, 945
Curse of dimensionality
  coverage table, 196
Cyber-physical contract
  trajectory planning, 509
Cyber-physical security
  shared-die attack surface, 577
  transduction attacks, 678
Cyber-physical systems, 9
Cybersecurity engineering
  ISO/SAE 21434, 678
Cycloidal drive
  fatigue life, 260

DAgger
  linear error bound, 202
Damped least squares
  inverse kinematics, 475
Data association
  probabilistic, 384
Data collection
  linear scaling fallacy, 220
Dead-man control
  chunk lease, 537
Dead-time delay, 138
Deadline
  relative, 132
Deadman switch, 627
Dean, Jeff
  latency numbers, 924
Debouncing
  latency cost, 120
Decoupling capacitors
  inverter DC bus, 950
  multi-tier hierarchy, 938
Defeaters
  adversarial review, 737
  definition, 721
Degraded mode
  definition, 726
  velocity reduction, 731
Demonstration
  vs. evidence, 718
Denominator filtering, 316
Dependent failure analysis
  placement, 577
Deployment
  policy ranking inversion, 268
Design diversity
  common-mode failure, 720
Design of experiments
  stratified test matrix, 278
Design quantile target
  definition, 479
Detectable faults, 361
Diagnostic coverage, 136
Diffusion policy
  inference latency wall, 270
  score-matching derivation, 858
Diffusion sampling latency
  sequential evaluation, 861
Digital twin
  origin, 658
Digital twin delusion
  simulation limits, 658
Direct collocation
  defect constraints, 503
Direct memory access
  cache coherence, 576
Dispatch-to-admission interval
  budget, 594
Dissipativity
  energy tank, 542
Distributed clock, 150
Disturbance observer, 546
Disturbance rejection
  recovery data, 269
Divergence point
  intervention curation, 200
DMA, 603
Domain randomization
  definition, 256
  origins, 256
  panacea fallacy, 270
Domain transfer, 737
Double-integrator
  covariance expansion, 915
DRAM, 574
Drivetrain
  jerk dynamics, 473
  resonant frequency, 508
Dual A/B OS partitioning
  OTA architecture, 744
Dual-rail architecture
  permission rail, 82
Duty type
  IEC 60034-1, 67
Dynamic brake chopper
  regenerative clamp, 73
Dynamic braking, 539
Dynamic fallback ladder, *see* Fallback ladder
Dynamic obstacle
  forward occupancy forecasting, 455
Dynamic random-access memory, *see* DRAM
Dynamic range saturation, 361
Dynamic reachability
  definition, 453
  reachable set, 907

Dynamic stopping distance
definition, 51
Dynamic voltage and frequency scaling, 81
Dynamics
non-smooth contact, 412
Dynamometer
chassis, 737

Earliest Deadline First
dynamic scheduling, 928
Ed25519
verification cost, 637
Edge event-triggered telemetry
architecture, 747
Edge telemetry
event-triggered buffer, 714
Effective sample size, 698
serially correlated telemetry, 182
Electrical power integrity
definition, 43
Embodied AI, 9
Embodiment contract
definition, 204
Empirical state-action occupancy distribution, 196
Empirical sufficiency, 736
End-effector
definition, 93
Endogenous covariate shift
definition, 177
Energy tank
contact barrier function, 542
Enforcement
manual override, 649
transfer boundaries, 648
Enforcement filter, 617
Enforcement record
definition, 559
Enforcer, 518
tracking contract, 519
Engineering ceiling
winding temperature, 67
Envelope membership
trichotomy states, 730
Envelope trichotomy
definition, 730
Environment selection, 315
Epipolar constraint
definition, 354
Epipolar geometry
epipolar plane, 354
rolling-shutter mismatch, 336
Episodic memory, 405
Epoch, 388
acquisition, 388
estimate, 388
publication, 388
Equations of motion
manipulator dynamics, 93
Error expansion, 387
Essential matrix, 354
EtherCAT
cycle determinism, 46
working counter, 687
Euclidean signed distance field, 397
Evaluation
action agreement, 318
binomial model, 317
confidence intervals, 317
covariate shift, 318
fault injection, 319
metric masking, 282
Evaluation estimand
definition, 278
Evaluation record
definition, 300
Event cameras
transduction, 338
Event data recorder
pre-trigger buffer, 203
sampling baseline, 642
Event message, 139
Evidence horizon
definition, 113
intent lease, 449
Evidence regimes
definition, 287
Evidence tuple, 642
Exchangeability
conformal guarantee, 119
Executable trajectory
definition, 473
Executed prefix
definition, 478
Execution lease
monotonicity, 627
Execution monitoring
collision detection, 511
Expected Calibration Error
binned accuracy metric, 842
Expiring intent lease
definition, 460
Exposure time, 370
Exposure wall
definition, 307
mathematical derivation, 874
retraining reset, 791
Extended Kalman filter
definition, 382
Extreme Value Theory
execution time tail bounds, 929
Frechet distribution, 878
Extreme value theory, 311
definition, 699
heavy tails, 699
origins, 311
Extremely improbable
failure condition, 791
Eye-in-hand calibration
definition, 343
Eye-to-hand calibration
definition, 343

Fail-operational
balancing machine, 804
Failure rate
catastrophic, 749
Fallback ladder
definition, 550
STOP rung, 731
Fallback policy
safety filter, 540
Fallback trajectory
achievability, 512
commitment point, 511
Falsification
Popperian induction, 790
Falsification threshold
definition, 597
Fatigue limits
cyclic derating, 732
Fault coupling
shared resources, 704
Fault Injection, 603
Fault injection, 673
circuitry schematics, 686
hardware, 673
Fault record
definition, 668
release set, 701
schema specification, 702
Fault Tolerance
kernel panic, 563
Feasibility
kinodynamic, 483
Feedback control
error monitoring, 512
Feedback linearization
computed torque control, 887
Feedforward control
torque demand, 512
Field-Oriented Control
current loop limits, 55
dq-frame dynamics, 947
Field-oriented control
drive current loop, 131
Fieldbus
CAN 2.0B arbitration, 951
CAN FD arbitration, 951
CAN FD WCRT, 951
contention, 704
EtherCAT on-the-fly, 951
synchronization jitter, 388
Time-Sensitive Networking, 953
TSN perception-control multiplexing, 951
Filter consistency, 383
Fine-tuning
on physical hardware, 260
Finite impulse response filter
group delay, 147
Fisher's exact test
lady tasting tea, 299
Flexible joint dynamics
Spong model, 904
Floating-base dynamics
equations of motion, 917
Flow matching
integration steps, 98
Force closure, 433
definition, 914
Forensic traceability
release provenance, 744
Forward invariance
Nagumo viability, 898
Forward kinematics
mapping, 906
two-link arm, 474
Forward occupancy set, 455
Four-phase takeover protocol, 629
Four-stage qualification ladder, *see* Qualification ladder
Four-stream action tap
definition, 184
FR-FCFS scheduling
definition, 581
Free-space depth, 402
Free-space erosion, 407
Freedom from interference, 157
Freezing robot problem, 456
Freshness deadline
definition, 44
Friction coefficient
Amontons-Coulomb, 789
Friction cone, 433
grasp mechanics, 914
Frozen transform, 418
Functional safety
Emergency stop categories, 551
failure rate targets, 307
Fundamental matrix, 354
Future-state conditioning
definition, 482

Gate delay
Sakurai-Newton model, 939
Gear ratio
definition, 49
Gearbox compliance, 737
Generalist policy
pretraining, 241
Generalized Pareto Distribution
GPD, 699
peaks over threshold, 878
Pickands-Balkema, 699
Glass-to-glass latency
teleoperation, 619
Global shutter
definition, 333
Goal Structuring Notation
elements, 717
Grasp map, 914
Grasp synthesis
learned generators, 435
Grounded affordance
definition, 434
Grounding
ambiguity, 440
semantic containment, 434

Hamilton-Jacobi reachability, 537
differential game, 907
differential games, 537
viscosity solution, 908
Handoff record
definition, 162
Handover

DRAFT

authority register, 648
autonomous fallback, 649
four-phase protocol, 629
takeover request, 648
Handshake protocol latency, 636
Hardware
actuator boundary, 649
AXI4 burst limits, 581
bus jitter, 314
clock synchronization, 371
gearbox compliance, 314
MIPI CSI-2, 351
rolling shutter, 373
SerDes jitter, 319
thermal derating, 314
thermal expansion, 369
watchdog timer, 449
Hardware calibration
persistent parameter retention, 744
Hardware exceptions
unaligned access trap, 965
Hardware security
side channels and DoS, 577
Hardware security module
release signing, 728
Hardware tracing
CoreSight ETM, 665
Hardware trip zone, 154
Hardware watchdog, 546
OTA rollback, 747
Hardware wear
bearing life, 244
Hardware-in-the-loop, 682
regression gates, 745
Harmonic drive
fatigue life under RL exploration, 244
Hash chain
linked timestamping, 643
Heterogeneous silicon partitioning
definition, 574
Holonomic constraint
etymology, 505
Huber loss
robust estimation, 234
Human oversight role
definition, 761
Human-gated aggregation, 202
Hypervisor
static partitioning, 593
Hysteresis
state-dependent, 586

IEC 60204-1
Stopping categories, 551
IEEE 1588
Precision Time Protocol, 48
IEEE 1588 Precision Time Protocol
pairing error, 187
IEEE 1588 PTP, 371
IEEE 802.1AS
hardware timestamping, 48
Imitation learning
expert demonstration, 650
expert recovery, 648
Impedance controller, 447
Importance sampling
definition, 697
origins, 697
In-situ fault injection
definition, 686
In-situ physical fault rigs, 682
Incident disclosure duty
definition, 764
Incidents
Williston accident, 365
Inductive voltage droop
DC bus harness, 950
power delivery network, 938
Inertia matching
impedance matching, 58
Inertial measurement unit
specific force, 335
Inevitable collision state
definition, 499
Inference latency, 132
Initial condition matching, 635
Innovation
definition, 381
Innovation sequence, 416
Input capture
DMA latching, 183
Insulation aging
loading by temperature, 66
Integrator windup, 520
unwinding delay, 530
Intent, 432
Intent grounding refusal
definition, 436
Intent lease
definition, 460
temporal contract, 100
Intent record
force limits, 447
spatial tolerance region, 435
temporal lease, 447
Inter-core communication
RPMSG framework, 942
Interference
shared power rail, 604
Interference (scheduling), 133
Intervention
definition, 618
Intervention curation
definition, 201
Intervention density
definition, 280
Intervention masking, 640
Invalid-test rule
definition, 668
Invalidation triggers, 749
Invariant sets
domain of attraction, 893
Inverse dynamics
definition, 484
normalizing transformation, 887
Inverse kinematics
differential step, 475
Inverse-depth metric grounding
definition, 355
Inverse-depth parameterization
visual odometry, 847
Inverter
dead-time distortion, 936
desaturation trip, 508
shoot-through protection, 936
Inverter dead time
shoot-through protection, 58
IOMMU
DMA isolation, 574
ISO 13855
approach speed, 455
walking approach speed, 407
ISO 21448
SOTIF, 301, 702
ISO 26262
Hardware Fault Injection, 670
ISO/TS 15066
case criterion, 716
separation distance, 50

Jacobian
automatic differentiation, 474
manipulator, 474
Jerk
braking, 25
definition, 486
minimum-jerk trajectory, 487
Jerk profile
definition, 902
Jitter, 133
Joule heating, 279, 520, 551
actuator thermal limits, 260
component failure, 667
fault coupling, 695

Kalman filter
Apollo navigation, 380
optimal Bayesian estimation, 891
Karush-Kuhn-Tucker
contact complementarity, 910
Kinematic reachability
intent horizon, 905
Kinematic singularity
Jacobian rank deficiency, 906
Kinematic stopping distance
observation age, 958
Kinematic velocity root
quadratic clearance, 51
Kinematics
uncertainty, 370
workspace boundaries, 452
workspace reachability, 451
Kinetic momentum
definition, 42
Kinetic stopping distance
barrier inset, 900
Kinodynamic feasibility envelope
definition, 483
KV cache
definition, 97

L10 rated life
collection campaign, 193
hub bearings, 309
Laser safety
Class 1, 339
Latency waterfall, 346
Lead systems adjudicator
accountability, 728
Learner-action contamination, 200
Lease
authority that lapses, 111
evidence epoch, 465
stopping budget, 142
Legible intent
definition, 768
Level set methods
implicit surface propagation, 908
Lever-arm effect
apparent acceleration, 908
LiDAR
point cloud de-skewing, 339
range walk, 364
transduction physics, 338
Lie derivative
directional clearance decay, 898
Jacobian-vector product, 535
Lie group
SE(3) manifold, 843
Lifecycle degradation
definition, 77
Lift-Splat-Shoot
bird's-eye-view lifting, 846
Limit cycle
backlash, 59
Limit record
definition, 77
Linear complementarity problem
contact mechanics, 910
Linear Quadratic Gaussian
Separation Principle, 891
Linear Quadratic Regulator
optimal control, 888
Linux kernel
CFS vs SCHED_FIFO, 930
SCHED_DEADLINE, 928
Lipschitz continuity
policy sensitivity, 118
spectral normalization, 854
Local checks
authority withdrawal, 465
Localization
map-based, 393
Lockstep processor
fault detection, 574
Log-odds
unknown state, 401
Log-odds transformation
occupancy mapping, 850
Loop closure, 393
Loud failure, 418
Lyapunov differential equation
covariance divergence, 915
Lyapunov function
stability certification, 313
Lyapunov stability

asymptotic and exponential, 893
energy-like certificate, 94

Machine Learning
3D Gaussian splatting, 847
geometric uncertainty, 373
Manipulability index
Yoshikawa formulation, 906
Manipulator equation, 483
Manipulator Jacobian
differential kinematics, 906
Manual lease
heartbeats, 630
Manual reduced speed
enabling devices, 627
Markov decision process, 229
origin, 229
Mathematics
covariance propagation, 359, 956
cubic variance expansion, 412
inverse-depth, 355
Mean time between interventions
definition, 280
Measurement freshness
definition, 42
Mechanical backlash, 277, 287
Memory
DMA, 351
IOMMU, 352
Memory bandwidth
worst-case execution time, 604
Memory bandwidth regulation
MemGuard, 592
Memory barrier
ARM weakly ordered model, 942
DMB vs DSB vs ISB, 943
instruction ordering, 943
Memory contention
bank groups, 939
DRAM bank conflicts, 939, 940
row buffer management, 939
Memory layout
natural alignment, 965
Memory locking
mlockall page fault elimination, 930
Memory Protection Unit
spatial isolation, 574
Memory wall
definition, 103
origin, 103
physical inference, 101
Memory-mapped I/O, 574
Metric drift, 315
Metric grounding
definition, 354
Metric spatial grounding, 99
Metric unprojection
camera intrinsics, 844
Microcontroller
bare-metal, 605
Microprocessor, 405
Middleware QoS mirage
definition, 417
Minimal intervention, 539
torque vs. task space, 540
Minimal-intervention QP projection (safety shield), 539
Minimal-risk condition
fallback motion, 25
MIPI CSI-2
DMA buffers, 405
frame-start timestamp, 46
host ingress, 351
sensor readout interface, 48
Missing observable
definition, 793
Mixed-criticality scheduling
definition, 593
first use, 134
Mixture density network
inverse problems, 232
Mode averaging
appendix refresher, 854
definition, 231
Jensen's inequality, 231

Mode confusion
definition, 639
Model Human Processor
slow bound, 619
Model Predictive Control
constrained QP formulation, 895
origin, 503
receding horizon, 503
tracking layer, 521
Modular safety case
continuous deployment, 744
Modulation transfer function
motion blur, 363
Moral crumple zone
definition, 761
Moravec's paradox, 7
sensorimotor skill, 7
Motion predicates
definition, 434
Motor
Joule heating, 511
thermal derating, 512
Motor constant
definition, 62
MTBF
constant failure rate, 308
Multi-accelerator
peer-to-peer pipelines, 945
Multi-rate hierarchy
definition, 405
Multi-rate synchronization
wait-free buffers, 945
Multicore interference
FAA AC 20-193 / CAST-32A, 576
Mutex
inter-processor blocking, 605

Nagumo theorem
contingent cone, 899
Nagumo viability theorem
boundary condition, 898
Nagumo's theorem
sampled enforcement, 534
NEES
definition, 383
Negative event registry, 267
Negative evidence, 401
Nervous system
architecture, 130
Network
autoregressive, 406
availability, 650
bus latency, 511
rate limiting, 650
Networking
heartbeat monitor, 563
Neural network verification
closed-loop property, 685
open-loop property, 684
Neuromorphic engineering
origin, 338
Neuromorphic event cameras
transduction, 338
NIS
definition, 383
Non-inferiority test
canary rollout, 746
Non-smooth mechanics
Moreau time-stepping, 911
Nonholonomic constraint
differential drive, 505
Normalization of deviance, 738
origin, 738

Object-centric 3D scene graph
definition, 404
Observability
definition, 416
Hermann-Krener criterion, 917
Kalman rank criterion, 916
rank test, 416
Observable, 416
Observation
freshness limit, 370
Observation age
definition, 345
latency waterfall, 346
Observation contract
definition, 366
Observational indistinguishability
definition, 798
Occlusion covariance expansion, 406
Occupancy grid, 396
sonar origin, 396
Occupancy grid mapping
octree storage, 851
Odometry, 393
Offline reinforcement learning, 240
Online trajectory generation, 493
Open-loop evaluation
closed-loop gap, 283
definition, 281
Open-loop replay fallacy, 746
Operating envelope
definition, 660
release scope, 751
trichotomy, 730
Operating permit
definition, 725
Operational design domain
definition, 197
evidence, 28
release envelope, 714
release verdicts, 714
target ODD, 291, 320
training boundaries, 265
Operational independence
definition, 573
Operational response
definition, 794
Operator reaction budget, 636
Optional stopping, 316
Oracle
definition, 667
Out-of-distribution action overestimation, 240
Over-the-air update
calibration retention, 713
safety case decomposition, 744
Overconfident predictor, 419
Overvoltage protection
dynamic braking, 73

Paired probing
definition, 251
evaluation test plan, 290
simulator validation, 251
Parameter partition
decoupled storage, 744
Partially observable Markov decision process
belief state, 258
Passivity
bilateral teleoperation, 191
energy storage, 920
Passivity observer
contact safety, 542
Passivity theorem
feedback stability, 920
Patch tokenization
vision transformers, 844
PDDL, 436
Peaks-Over-Threshold
POT, 699
Peer-to-peer DMA
accelerator IPC, 945
Perception, 370
blind distance, 373
calibration drift, 373
dated snapshots, 373
multimodal fusion, 373
observation age, 373
observation contract, 366, 373
pipeline latency, 370
silent failure, 373
timestamp integrity, 373
Perceptual chattering
state reset hazard, 120
Performance level (ISO 13849), 136
Period (real-time task), 132
Permission path, 738
definition, 22

Permission stage
  definition, 763
Phantom-object tracks, 418
Phase margin
  delay budget, 137
  transport delay penalty, 905
Phase plane
  time parameterization, 491
Phase-locked loop
  shared dependency, 604
Photon shot noise, 362
Physical AI
  definition, 11
  WCET analysis, 929
Physical Artificial Intelligence
  definition, 11
Physical body
  definition, 42
Physical bounds, 784
Physical plant
  definition, 5
Physical relocation
  silicon segregation, 593
PID control
  closed-loop error dissipation, 885
  physical interpretation, 885
Pinhole camera model, 336
Placement record
  definition, 601
Plant mutation during training, 245
Policy
  stochastic, 229
Policy gradient
  REINFORCE, 242
Policy learning
  temporal horizons, 472
Policy manifest
  definition, 264
  omission log, 257, 264
Policy vulnerability inference chain, 263
Post-training
  of generalist robot policies, 241
Power and force limiting, 557
Power delivery network
  frequency-domain impedance, 938
  target impedance, 938
Power electronics
  brake chopper, 950
Power MOSFET
  Miller plateau dynamics, 936
Power-rail coupling, 587
Practical Engineering Question, 852
Pre-brake ceiling, 142
Pre-trigger ring buffer, 267
Precision Time Protocol
  clock synchronization, 941
  delay measurement, 941
  hardware timestamping, 942
Preempt RT
  kernel latency, 574
Preemption tier, 644
Prefill
  compute-bound phase, 101
Priority inversion, 133
Privilege separation
  intent boundary, 448
Probability calibration
  algebraic normalization, 118
Probe effect
  fault-injection timing, 665
Problem of many hands
  definition, 760
Processor-in-the-loop, 682
Proof test
  interval, 799
Proposal boundary
  contract schema, 116
  cyber-physical demarcation, 981
  definition, 22
  goal filtering, 452
Proposal contract
  definition, 116
Proposal header, 145
Proposal payload
  definition, 116
Proposal token, 616
  grant at blend end, 630
Protective intervention, 673
Provenance, 659
Provenance record
  definition, 216
  physical AI contract, 216
Proximal policy optimization, 247
Proxy metric fallacy, 737
Publish-time stamping anti-pattern, 388
Pulse-width modulation
  dead-time insertion, 936

Quadratic program
  closed-form projection, 900
Quadratic programming, 538
  active-set method, 546
Qualification ladder
  definition, 681
Quantile convolution
  tail non-additivity, 46
Quantitative robustness semantics
  definition, 690
Quantization
  weight-only, 106
Quasi-direct drive
  definition, 61
  history, 87
Quintic smoothstep
  nonzero boundary derivatives, 634
Quintic spline
  analytical coefficients, 902
  boundary solution, 902
  Vandermonde system, 902

Radar
  Doppler velocity, 341
  FMCW, 340
  FMCW transduction, 338
Rate-Monotonic Scheduling
  utilization bound, 928
Rate-monotonic scheduling, 148
Raymond Goertz
  bilateral telemanipulation, 181
Real-time chunking
  committed prefix, 110
Real-time Linux
  kernel mechanics, 930
Real-time operating system
  interrupt latency, 930
Real-time systems
  worst-case execution time, 928
Reality gap
  definition, 253
Receding horizon
  action chunking, 108
Record
  definition, 964
Record chain
  register, 964
  wire formats, 965
Recoverability constant, 202
Recoverability problem, 178
Recovery handshake, 158
Recursive feasibility
  terminal invariant set, 895
Reference frame
  definition, 342
Reflected inertia, *see* Reflected rotor inertia
Reflected rotor inertia
  definition, 58
  kinetic energy derivation, 948
Refusal
  confident grounding, 466
  grounding ambiguity, 436
Regenerative braking
  chopper resistor, 950
Reinforcement learning
  regime transition failures, 269
Relative degree
  definition, 899
  relative degree trap, 535
Release contract
  release manifest, 739
Release engineering
  canary rollouts, 746
  HIL regression testing, 745
Release manifest
  cryptographic provenance, 744
  definition, 739
  packaging cell, 787
Release verdict
  conditional, 731
  definition, 726
  invalidation, 732
Replacement latency
  definition, 477
Reset controller
  isolation, 604
Reset latency
  definition, 280
Residual risk, 791
Residual-claims register
  definition, 793
  packaging cell, 787
Resource contention
  worst-case, 599
Response time, 132
Return level
  EVT safety bounds, 699
Return period
  flood engineering, 699
Reward hacking, 245
Riccati equation
  covariance propagation, 915
Right censoring
  intercepted runs, 289
Rigid body transformations, 356
Rigid-body dynamics
  manipulator equation, 93
Rolling shutter
  definition, 333
Rolling-shutter motion compensation
  definition, 335
Rule of three
  clinical origin, 293
  definition, 292
  limit records, 77
  zero-failure testing, 872
Runtime monitor, 508
Runtime monitoring, 320
  constraint margin, 314
  environmental assumptions, 314
  execution latency, 314
  intervention rate, 314
  OOD detection, 314

S-curve
  jerk-limited stop, 495
Safe set, 534
Safe stop 1, 551
Safe torque off, 155, 972
Safety
  admissible motion, 649
  Category 1 stop, 319
  deterministic terminal state, 462
  envelope, 370
  hallucination containment, 434
  operational envelope, 317
  physical zoning, 315
  stopping margin, 648
Safety Case, 603
Safety case
  claim-argument-evidence, 751
  dynamic, 729
  lifecycle, 749
  modular GSN decomposition, 744
  physical AI, 718
Safety enforcement, 617
Safety enforcer
  hardware interlock, 731
Safety filter
  least-restrictive, 540
Safety integrity level (IEC 61508), 135
Safety laser scanner
  field sets, 726
Safety microcontroller
  definition, 134
Safety performance indicator, 747

Safety shield, 539
Safety-Critical Standards
  autonomous systems, 25
Scenario table, 197
Schema-detectable frame mismatch, 419
SE(3), 452
  spatial bounds, 465
  special Euclidean group, 433
  target goal set, 435
SE(3) Kinematics, 457
SE(3) kinematics, 317, 356
  error propagation, 359
Self-attention
  multimodal token sequence, 845
Semantically wrong frame assignment, 419
Sensor
  IMU, 563
  optical encoder, 563
Sensor fusion
  informational contract, 387
Sensor read noise, 362
Sensor spoofing
  physical injection, 678
Sequence lock, 139
  Lamport versioned read, 139
  shared memory, 139
Sequential Probability Ratio Test
  Wald formulation, 875
  Wald optimality, 877
  wartime origin, 296
SerDes
  link errors, 738
Serialization
  wire formats, 965
Series Elastic Actuator
  compliance, 54
Set invariance
  Bouligand cone, 899
Shadow mode deployment
  bus jitter, 287
  definition, 745
Shakey (robot)
  Sense-Plan-Act cycle, 980
Shannon entropy
  information theory, 841
Shared memory
  position-independent layout, 943
  POSIX shm, 943
Shielding
  safe reinforcement learning, 539
Signal Temporal Logic
  quantitative robustness, 872
  syntax and semantics, 871
Silent failure, 418
  definition, 361
Silicon aging
  timing margins, 601
Silicon partitioning
  definition, 574
Sim-to-real contact exploit, 248
Sim-to-real gap
  physical vector decomposition, 252
Sim-to-real reality gap vector
  definition, 253
Sim-to-real transfer gap
  discrepancy bounds, 693
Simplex architecture
  dual-controller model, 519
  online upgrade, 519
Simulation
  timestep bounds, 267
Simulation-to-reality leap, 737
  validation bounds, 693
Simultaneous admission invariant
  definition, 43
Singular perturbation
  elastic joints, 904
Situation awareness
  three levels, 615
SLAM, 393
  correlated map and pose error, 393
Slip ratio
  longitudinal tire kinematics, 913
Smith predictor
  dead-time compensation, 481
Smoothstep polynomial
  jerk bounding, 904
Softmax normalization
  origin, 118
Software-in-the-loop, 682
Space robustness, 690
Spatial coordinate frames
  SE(3) representation, 843
  SE(3) transformation, 908
Spatial information age, *see* Observation age
Spatial memory, 378, 425
Spatial partitioning, 593
Spatial representation
  agency, 427
Special Euclidean group
  SE(3) grounding, 99
Speed and separation monitoring
  definition, 557
  intent screening, 455
  protective separation distance, 50
Speed ceiling
  stopping envelope, 523
Spline
  etymology, 488
Stale-state paradox, 481
Standards
  REP 105, 344
  ROS Enhancement Proposal, 344
State change
  irreversible, 784
State estimation, 406
  belief state, 378
  contract boundary, 387
  propagation, 370
  recursive estimator, 380
State marginals
  discounted visitation, 864
State message, 139
State of charge (SoC), 279
State-space representation
  discrete zero-order hold, 883
  linear time-invariant, 882
Static reservation, 592
Static world fallacy
  definition, 425
Statistical power
  sample size, 299
Statistical safety fallacy, 538
Statistical verification
  collapse under fine-tuning, 260, 269
Stator resistance
  thermal coefficient, 67
Stereo rectification
  epipolar lines, 355
Stopping budget
  partial and complete, 619
Stopping distance, 750
  definition, 51
Stopping envelope, 531
  Anisotropy, 522
  definition, 521
Stopping suffix
  definition, 476
  S-curve, 495
Strain wave gearing
  origin, 59
  wear, 737
Strain-wave gearing
  origin, 486
Streaming ring-buffer attention, 405
STRIPS
  Shakey origin, 436
Structural independence, 737
Structured light
  transduction, 337
Substantial modification
  Machinery Regulation, 725
Subsumption architecture
  reactive robotics, 980
Supervisory Control Levels, 617
Supervisory intervention
  definition, 618
Support envelope relational join
  definition, 197
Survival envelope
  definition, 661
Survivorship bias
  in physical datasets, 219
  verification, 706
  Wald aircraft vulnerability, 214
Symbol Grounding Problem, 434
Symbol grounding problem, 434
System identification
  excitation design, 256
  scope test, 18
  sim-to-real, 256
Systems-Theoretic Accident Model and Processes
  definition, 675
Systems-Theoretic Process Analysis
  emergent hazards, 675

Tactile sensing
  GelSight, 341
Tail latency
  fan-out, 478
Target impedance
  power delivery, 584
Task and motion planning
  definition, 436
  operator, 436
Teleoperation
  demonstration yield, 193
  kinematic asymmetry, 220
  latency bound, 908
  transmission delay, 636
Tempe fatal collision
  operator monitoring, 640
Temporal accuracy, 145
Temporal action chunking
  definition, 108
Temporal ensembling
  mode averaging failure, 235
  trajectory smoothing, 109
Temporal logic falsification
  definition, 691
Test oracle
  definition, 667
Test set contamination, 315
Testing
  Oracle, 659
Thermal capacity
  definition, 43
Thermal clock throttling
  definition, 585
Thermal derating, 454, 530
Thermal management
  Joule heating, 269
Thermal resistance
  junction-to-ambient, 585
Thermal time constant, 585
  RC analogy, 585
Threat model, 791
Three-position enabling devices, 627
Tightly coupled memory, 604
Time parameterization
  definition, 491
Time partitioning
  TDMA, 592
Time robustness, 690
Time-of-flight
  indirect vs direct, 337
Time-Sensitive Networking
  frame preemption, 953
  Time-Aware Shaper, 953
Time-sensitive networking, 153
Time-to-Intervention
  metric, 285, 286
Timestamp
  asynchronous, 424
Timing anomalies
  worst-case execution time, 597
Token
  embedding of an input slice, 95
Token bucket
  verifier protection, 650
Tolerance region
  3D spatial, 435

DRAFT

Tool center point, 278
  handover speed contract, 727
Tool flange frame, 386
Torque constant
  definition, 54
Toulmin, Stephen
  argument model, 717
Track association, 370
Tracking error bound, 519
Tracking history
  classification changes, 24
Trajectory
  braking time, 512
Trajectory distribution
  closed loop, 864
Trajectory Generation
  jerk-limited splice, 451
Trajectory optimization
  collocation and MPC, 503
  spline synthesis, 902
Trajectory record
  contract, 509
  definition, 475
Translation lookaside buffer
  shootdown, 576
Transport delay
  feedback instability, 47
Transport lag, 520
Truncated signed distance field, 396
  origin, 399
  projective fusion, 851
Two-processor implementation
  fault injection, 687
Two-speed brain architecture
  hybrid control, 91

Uncertainty
  aleatoric, 365
  covariance expansion, 466
  epistemic, 365
  invalidation horizon, 410
Uncertainty bound, 387
Uncertainty envelope
  definition, 342
Unconsented risk
  definition, 767
Under-Voltage Lockout
  gate driver protection, 950
Unicycle model
  lateral drift, 282
Unified Memory Architecture
  crossbar trap, 576
Unmonitored state drift, 737
Unobservable subspace, 416
Unprivileged proposal
  proposal boundary, 99
Unscented Kalman filter
  definition, 382

Validation
  data sanitization, 268
Value function
  state, 229
Velocity clamp
  kinetic derating, 958
Verdicts
  conditional operation, 725
  refusal to operate, 725
  unconditional operation, 725
Verification gap
  definition, 674
Viability kernel, 536
  inevitable collision states, 498
Ville's inequality
  anytime-valid testing, 298
Virtual spring-damper system
  definition, 60
Vision-language model, 452, 457
Vision-language-action model, 8
Voltage droop
  gate delay, 604
Volumetric free-space raycasting
  3D-DDA traversal, 851
Voxel block hashing, 399
Voxel traversal
  Amanatides-Woo, 401
Vulnerability analysis
  training regimes, 263

Wait-free data structures
  sequence locks, 605
Wait-free synchronization
  slot mailbox, 141
Wald confidence interval
  asymptotic breakdown, 874
Warrant
  inferential rule, 718
Watchdog lease
  kinematic derivation, 908
Watchdog rollback
  hardware triggers, 747
Watchdog timer
  challenge-response, 591
  hardware downcounter, 935, 936
  token-passing liveness, 935
  windowed, 590
  windowed watchdog, 935
Wave variables
  time delay passivity, 921
WCET
  definition, 132
Whole-body control
  hierarchical QP, 918
  multi-task optimization, 918
Wilson interval
  zero-failure records, 294
Windowed watchdog timer
  hardware mechanics, 590
Workload reduction, 593
World model
  as learned simulator, 252
Worst-case execution time, 132
  definition, 132
  multicore certification, 576
  static analysis, 132
  timing anomalies, 597

Zero Moment Point
  Vukobratovic criterion, 918
Zero-allocation
  embedded record chain, 965
Zero-allocation static architecture, 149
Zero-command floor, 539
Zero-failure testing
  independence, 695
Zero-innovation trap
  definition, 415
Zero-order hold
  discrete sampling, 905
  inter-sample drift, 545
Zhang's calibration
  homography matrix, 957